PUBLIC
2026-10-27

SAP Business Technology Platform

© 2026 SAP SE or an SAP affiliate company. All rights reserved.

Content

1 SAP Business Technology Platform. 7

2 Basic Platform Concepts. 10

2.1 Solutions and Services. 16

2.2 Regions. 17

Regions and API Endpoints Available for the Cloud Foundry Environment. 19

Regions and API Endpoints for the ABAP Environment. 43

Regions for the Kyma Environment. 51

Resilience, High Availability, and Disaster Recovery. 59

2.3 Environments. 62

Cloud Foundry Environment. 63

ABAP Environment. 81

Kyma Environment. 86

Neo Environment. 108

2.4 Trial Accounts and Free Tier. 109

2.5 Enterprise Accounts. 111

Commercial Models. 113

Using Free Service Plans. 118

2.6 Account Model. 121

2.7 Entitlements and Quotas. 127

2.8 Introduction to Identity and Access Management on SAP BTP. 131

2.9 User and Member Management. 133

Platform Users. 135

Business Users. 149

2.10 Tools, Programming Models, Programming Languages, and APIs. 150

Tools. 151

Programming Languages. 153

Programming Models. 154

Continuous Integration and Delivery (CI/CD). 155

APIs. 158

Cloud Management Tools. 159

Prerequisites and Restrictions. 159

3 Getting Started. 161

3.1 Getting a Global Account. 162

3.2 Getting Started in the Cloud Foundry Environment. 165

Getting Started with a Trial Account in the Cloud Foundry Environment. 165

Getting Started with an Enterprise Account in the Cloud Foundry Environment 176
3.3 Getting Started in the ABAP Environment. 183
Getting Started with a Customer Account in the ABAP Environment. 184
Getting Started with Custom Code Analysis in the ABAP Environment. 246
3.4 Getting Started in the Kyma Environment. 247
Getting Started with an Enterprise Account in the Kyma Environment. 248
Getting Started with a Trial Kyma Instance. 249

4 Development. 254
4.1 Development in the Cloud Foundry Environment 256
Best Practices. 259
Developing Your First Application. 261
Developing Applications and Services. 261
Developing User Interface. 430
Consuming APIs. 546
Adding Authentication and Authorization. 546
Setting Up Database Artifacts. 612
Deploying to the Cloud Foundry Environment. 614
4.2 Development in the ABAP Environment. 848
Getting Started as a Developer in the ABAP Environment. 849
ABAP Development Tools: User Guides. 852
ABAP Data Models Guide. 853
ABAP Keyword Documentation. 855
ABAP RESTful Application Programming Model 855
Generative AI in ABAP Cloud. 857
Assuring Quality of ABAP Code. 859
Background Processing Framework. 860
ABAP Lifecycle Management. 861
Intelligent Scenario Lifecycle Management. 958
Identity and Access Management (IAM). 977
Business Configuration. 1045
Integration and Connectivity. 1064
SaaS Applications and Multitenancy. 1233
Released Components and Objects. 1367
UI Development. 1843
ABAP Service Instance URL. 1881
4.3 Development in the Kyma Environment. 1882
Kyma Modules. 1884
Access a Kyma Instance Using kubectl. 2265
Using SAP BTP Services in the Kyma Environment. 2267
Deploy Workloads in the Kyma Environment to Extend SAP Systems. 2267
Expose and Secure a Workload with a JWT Using SAP Cloud Identity Services. 2269

Develop Resilient Applications in the Kyma Runtime. 2278
Available Technology. 2279

5 Extensions and Integrations. 2281
5.1 Maintaining Unified Customer Landscape. 2283
5.2 Extensibility and Integration Concepts. 2284
5.3 Requesting Consent for a System or System Type. 2289
Request Consent. 2293
Approve or Reject a Consent Request. 2294
5.4 Adding, Registering and Deregistering Systems. 2295
Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP. 2299
Register an SAP Marketing Cloud System in a Global Account in SAP BTP. 2304
Register an SAP Integrated Business Planning System in a Global Account in SAP BTP. 2308
Register an SAP SuccessFactors System in a Global Account in SAP BTP. 2312
Register an SAP Customer Experience System. 2316
Register a Microsoft 365 Copilot for Joule System. 2319
Register a System of Type SAP BTP Application. 2320
Deregister or Remove a System. 2321
Merge SAP Systems. 2323
5.5 Integrating SAP Solutions. 2324
Automating Integrations Using Formations. 2326
5.6 Extending SAP Solutions 2358
Declaring System APIs and Events as Dependencies for Business Scenarios. 2360
Extending SAP S/4HANA Cloud in the Cloud Foundry and Kyma Environment. 2363
Extending SAP Marketing Cloud in the Cloud Foundry and Kyma Environment. 2426
Extending SAP SuccessFactors in the Cloud Foundry and Kyma Environment. 2430
Extending SAP Customer Experience Products in the Kyma Environment. 2454

6 Administration and Operations. 2456
6.1 Account Administration. 2457
Account Administration in the Cockpit. 2458
Account Administration Using Joule in the Cockpit. 2658
Account Administration Using the SAP BTP Command Line Interface (btp CLI) 2664
Account Administration Using MCP Servers. 2722
Account Administration Using APIs 2729
Account Administration Using Infrastructure as Code. 2777
6.2 Administration and Operations in the Cloud Foundry Environment. 2779
Org Management Using the SAP BTP Command Line Interface (BTP CLI). 2780
About User Management in the Cloud Foundry Environment. 2787
About Roles in the Cloud Foundry Environment. 2788
Migrating Cloud Foundry Environment Instance to SAP Build. 2790
Org Administration Using the Cockpit. 2792

Org Administration Using the Cloud Foundry CLI. . . . 2812
Application Operations in the Cloud Foundry Environment. . . . 2860
Audit Logging in the Cloud Foundry Environment. . . . 2878
6.3 Administration and Operations in the ABAP Environment. . . . 2914
SAP Fiori Apps in the ABAP Environment. . . . 2916
Communication Operations. . . . 3286
Technical Operations. . . . 3378
Troubleshoot Custom Apps. . . . 3446
6.4 Administration and Operations in the Kyma Environment. . . . 3450
Creating Kyma Instances. . . . 3452
Tracking Kubeconfig and Cluster Associations in Kyma. . . . 3454
Managing Kyma Runtime Using the Provisioning Service API. . . . 3455
Available Plans in the Kyma Environment. . . . 3459
Provisioning and Updating Parameters in the Kyma Environment. . . . 3470
Adding and Deleting a Kyma Module. . . . 3511
Setting Your Module to the Unmanaged and Managed State. . . . 3513
Access Kyma Application Logs. . . . 3515
Kyma Environment Backup. . . . 3516
Kyma Modules' Sizing. . . . 3518
Change Storage Size in Kyma. . . . 3521
Assigning Workloads to Worker Node Pools. . . . 3522
Adapting Your Workloads to Kyma Landscape Requirements. . . . 3525
Admission Policies. . . . 3530
Kyma Runtime with Dual-Stack Support. . . . 3533
Ingress and Egress IP Addresses. . . . 3534

7 Security. . . . 3536
7.1 SAP Authorization and Trust Management Service. . . . 3538
What Is the SAP Authorization and Trust Management Service?. . . . 3545
Web Access Control. . . . 3548
API Access Control. . . . 3549
Authorization Entities. . . . 3550
Monitoring and Troubleshooting. . . . 3552
Security Considerations for the SAP Authorization and Trust Management Service. . . . 3591
Configuration Options for the SAP Authorization and Trust Management Service. . . . 3596
Third-Party Cookies and SAP Authorization and Trust Management Service. . . . 3598
Security Recommendations for SAP Authorization and Trust Management Service. . . . 3599
Rate Limiting. . . . 3600
Limits for Technical Artifacts of the SAP Authorization and Trust Management Service. . . . 3605
Configuring Backup. . . . 3607
Accessibility Features in SAP Authorization and Trust Management Service. . . . 3608
7.2 Default Role Collections of SAP BTP. . . . 3608

7.3 Trusted Certificate Authentication. 3616
7.4 Principal Propagation. 3617
Principal Propagation from the Neo Environment to the Multi-Cloud Foundation. 3617
Principal Propagation from the Multi-Cloud Foundation to the Neo Environment. 3624
Principal Propagation from SAP BTP Applications to On-Premise Systems Using Corporate IdP Tokens. 3631
7.5 Data Protection and Privacy. 3633
Glossary for Data Protection and Privacy. 3635
Change Logging and Read-Access Logging. 3637
Personal Data Record. 3637
Deletion. 3638
Consent. 3644
7.6 Security in the Kyma Environment. 3644
Kyma Security Concepts. 3644
Secure Development in the Kyma Environment. 3651
Secure Administration and Operations in the Kyma Environment. 3658

8 Getting Support. 3683
Creating a Support User. 3684
8.1 Providing Details for SAP HANA Service Database Problems. 3685
8.2 Gather Support Information. 3686
8.3 Platform Updates and Notifications. 3687
8.4 Operating Model. 3688
8.5 Support Components. 3692

9 Glossary. 3785

1 SAP Business Technology Platform

SAP Business Technology Platform (SAP BTP) is an offering that integrates the following technology portfolios: application development; process automation; integration; data, analytics, and enterprise planning; artificial intelligence. The platform allows users to turn data into business value, compose end-to-end business processes, connect entire IT landscapes, and personalize, build, and extend SAP applications. This reduces the overall total cost of ownership by maintaining SAP landscapes and third-party software across end-to-end business processes.

SAP BTP in the Intelligent Enterprise

Companies need access to live data to make informed decisions in real time. They also need to apply advanced technologies and best practices within agile, integrated business processes. That's why a key part of SAP's strategy is integrating end-to-end processes, whether the solutions come from SAP, partners, or third parties.

SAP drives integration across its solution portfolio in the following business scenarios: lead to cash, source to pay, design to operate, and hire to retire.

SAP Business Technology Platform offers integration capabilities that connect end-to-end business processes across SAP and third-party applications. The following key aspects, known as Suite Qualities, provide a consistent experience across applications:

- Seamless User Experience: SAP Fiori gives many SAP solutions a common look and feel. This improves user experience and reduces interruptions.
  See: SAP Fiori.
- Consistent Security and Identity Management: SAP Cloud Identity services on SAP BTP enable you to manage identities and use the single sign-on capability across selected end-to-end processes.
  See: SAP Cloud Identity Services on SAP BTP.
- Aligned Domain Models, APIs, and Events: Master Data Integration is a cloud service for master data integration. It provides a consistent view of master data across a hybrid landscape. You can use APIs for process integration among SAP solutions, SAP, and third parties, or by adopting an event-based integration strategy – all based on predefined integration content available on SAP API Business Hub.
  See: SAP One Domain Model, SAP Master Data Integration service, SAP Integration Suite on SAP BTP
- Embedded Analytics across Solutions: The analytical insights in various SAP solutions help you make informed decisions. Embedded analytics from SAP Analytics Cloud are available, for example, in SAP S/4HANA and SAP SuccessFactors.
  See: SAP Analytics Cloud on SAP BTP.
- One Workflow Inbox: With the unified view of pending tasks across SAP solutions in both mobile and desktop environments, you can complete tasks faster.
  See: SAP Task Center on SAP BTP, SAP Mobile Start. For full workflow modeling and visibility capabilities leverage, see: SAP Process Automation on SAP BTP
- Coordinated Lifecycle Management: You can optimize implementation projects and reduce configuration effort because provisioning, setup and operations, and monitoring solutions are harmonized. Automated

provisioning and guided integration setup are available for selected scenarios in lead to cash, source to pay, recruit to retire, and design to operate.
See: SAP for Me, SAP CALM

- End-to-End Process Blueprints: Process blueprints that follow the Industry Reference Architecture standard can reduce planning effort for implementation projects. They also support architecture decisions.
See: SAP API Business Hub

Related information:

- SAP's vision for the Intelligent Enterprise
- SAP's Integration Strategy whitepaper
- SAP Integration Strategy community

About This Guide

Get an overview of the basic platform concepts of SAP BTP and how they relate to each other. Learn how to manage your accounts and how to develop, extend, administer, and secure your cloud setup and applications on the platform.

> → Tip
>
> The English version of this guide is open for contributions and feedback using GitHub. This allows you to get in contact with responsible authors of SAP Help Portal pages and the development team to discuss documentation-related issues. To contribute to this guide, or to provide feedback, choose the corresponding option on SAP Help Portal:
>
> - *Feedback* > *Create issue*: Provide feedback about a documentation page. This option opens an issue on GitHub.
> - *Feedback* > *Edit page*: Contribute to a documentation page. This option opens a pull request on GitHub.
>
> You need a GitHub account to use these options.
>
> More information:
>
> - Contribution Guidelines
> - Introduction Video
> - Introduction Blog Post

More resources that you might find interesting:

Best Practices

Get onboarded to SAP BTP. Learn about basic concepts and see our best practices for your development projects in setting up the correct organizational structure, creating an account and security model, and developing and operating applications: SAP BTP Administrator's Guide.

Services and Solutions

Get an overview on the availability of SAP BTP solutions and services according to region, infrastructure provider, and license model. Access the service details pages for information on features, service plans, related tools, APIs, and documentation: SAP Discovery Center: Service Catalog.

Neo Environment

For more information about the Neo environment, see SAP BTP, Neo Environment.

To learn more about why and how to migrate your scenarios in the Neo environment, see Migrating from the Neo Environment to the Multi-Cloud Foundation.

All SAP BTP Resources

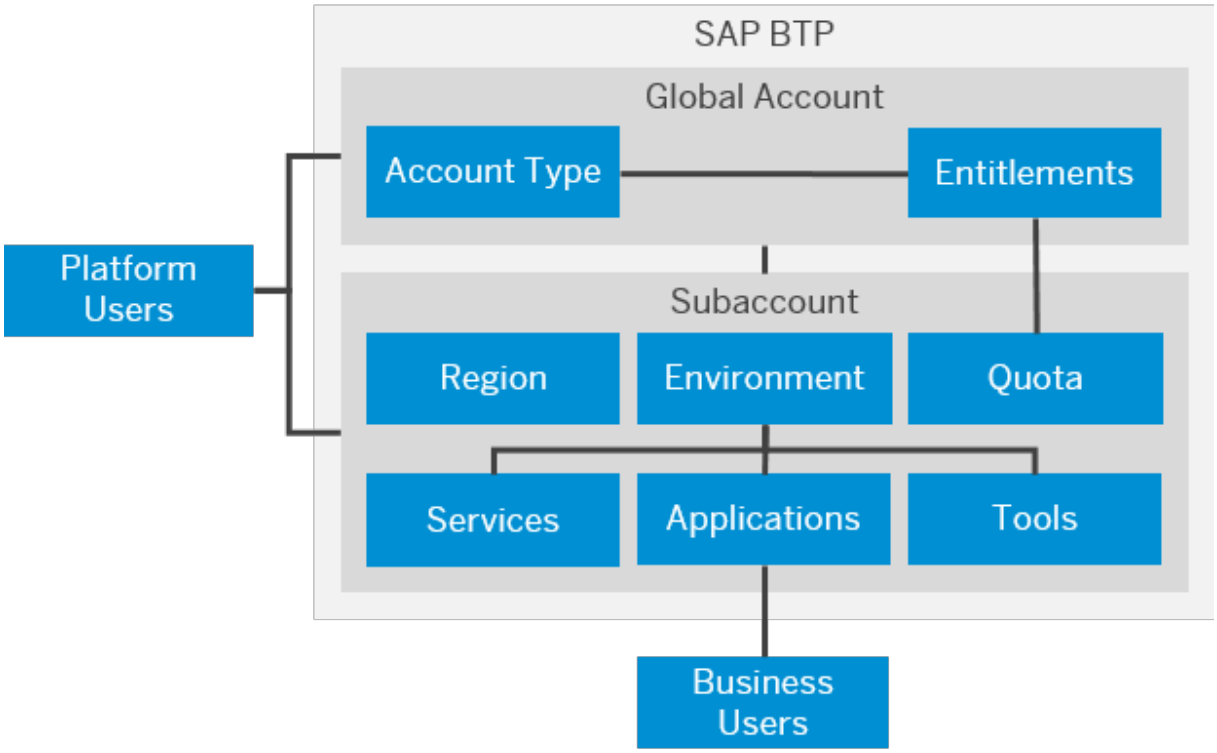
You can find all of our resources on the product page: SAP Business Technology Platform (SAP BTP).

2 Basic Platform Concepts

SAP BTP offers users the ability to turn data into business value, compose end-to-end business processes, and build and extend SAP applications quickly.

SAP Business Technology Platform is built on a multi-cloud foundation, which lets you choose from different infrastructures and runtimes. The services and solutions of SAP BTP are available on multiple cloud infrastructure providers, and it supports different runtimes, such as Cloud Foundry, ABAP, and Kyma, as well as multiple regions, and a broad choice of programming languages.

The central point of entry to the platform is the **SAP BTP cockpit**, where you can access your accounts and applications and manage all activities associated with them.



Relationship between SAP BTP Accounts and Associated Activities

Solutions and Services

SAP BTP offers fast in-memory processing, sustainable, agile **solutions** and **services** to integrate data and extend applications, and fully embedded analytics and intelligent technologies.

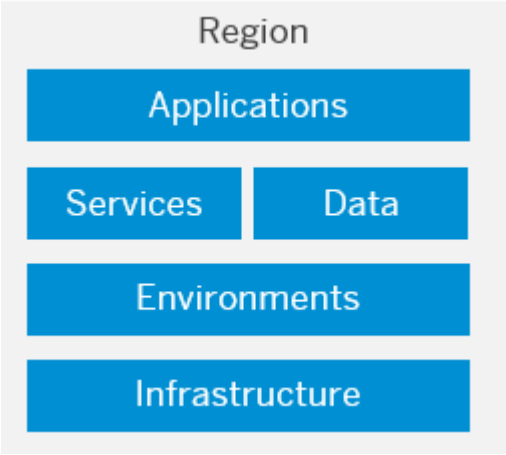
Services enable, facilitate, or accelerate the development of business applications and other platform services on SAP BTP.

For a complete list of services and capabilities, see SAP Discovery Center: Services.

For more information, see Solutions and Services [page 16].

Regions

You can deploy applications in different **regions**. Each region represents a geographical location (for example, Europe, US East) where applications, data, or services are hosted.



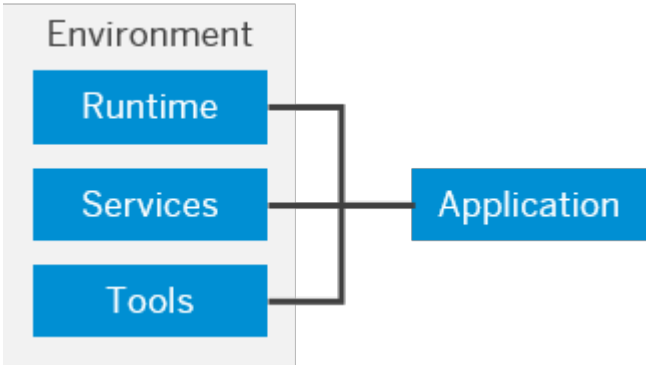
Regions are provided either by SAP or by our Infrastructure-as-a-Service (IaaS) partners Amazon Web Services (AWS), Microsoft Azure, Google Cloud, and Alibaba Cloud. The third-party region providers operate the infrastructure layer of the regions, whereas SAP operates the platform layer and Cloud Foundry.

A region is chosen at the subaccount level. For each subaccount, you select exactly one region (that is one data center).

For more information, see Regions [page 17].

Environments

Environments constitute the actual platform-as-a-service offering of SAP BTP that allows for the development and administration of business applications. Environments are anchored in SAP BTP on subaccount level.



SAP BTP provides the following environments:

- Cloud Foundry Environment [page 63]

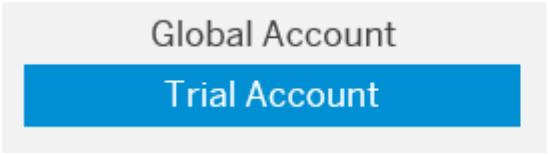
The Cloud Foundry environment enables you to develop new business applications and business services, supporting multiple runtimes, programming languages, libraries, and services. You can leverage a multitude of buildpacks, including community innovations and self-developed buildpacks.

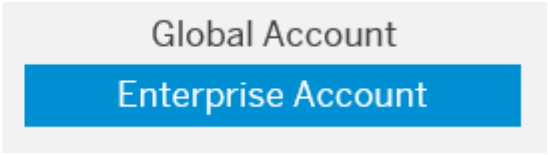
- ABAP Environment [page 81]
  The ABAP environment allows you to create extensions for ABAP-based products, such as SAP S/4HANA Cloud, and develop new cloud applications. You can transform existing ABAP-based custom code or extensions to the cloud.
- Kyma Environment [page 86]
  SAP BTP, Kyma runtime provides a fully managed cloud-native Kubernetes application runtime based on the open-source project "Kyma". Based on modular building blocks, Kyma runtime includes all the necessary capabilities to simplify the development and to run enterprise-grade cloud-native applications.
- Neo Environment [page 108]
  The Neo environment lets you develop HTML5, Java, and SAP HANA extended application services (SAP HANA XS) applications. You can also use the UI Development Toolkit for HTML5 (SAPUI5) to develop rich user interfaces for modern web-based business applications.

> ⚠ Caution
>
> SAP Business Technology Platform, Neo environment will sunset on **December 31, 2028**, subject to terms of customer or partner contracts.
>
> For more information, see SAP Note 3351844.

Enterprise and Trial Accounts

SAP BTP provides different types of global accounts, **enterprise** and **trial**. The type you choose determines pricing, conditions of use, resources, available services, and hosts.

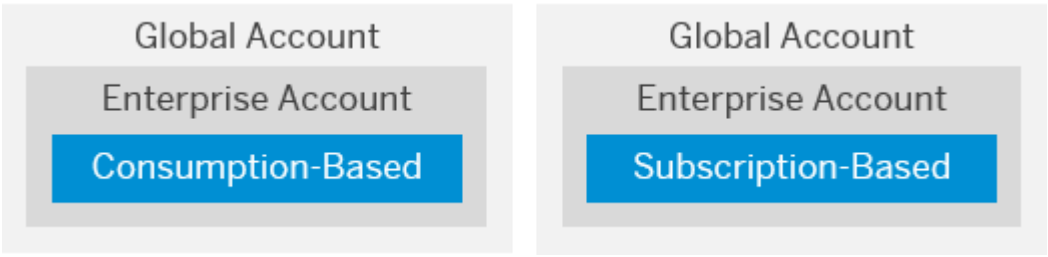




- A **trial account** lets you try out the platform for free. Access is open to everyone. Trial accounts are intended for personal exploration, and not for production use or team development. They allow restricted use of the platform resources and services.
  For more information, see Trial Accounts and Free Tier [page 109].
- An **enterprise account** is usually associated with one SAP customer or partner and contains their purchased entitlements to platform resources and services. It groups together different subaccounts that an administrator makes available to users for deploying applications.
  For more information, see Enterprise Accounts [page 111].

Commercial Models

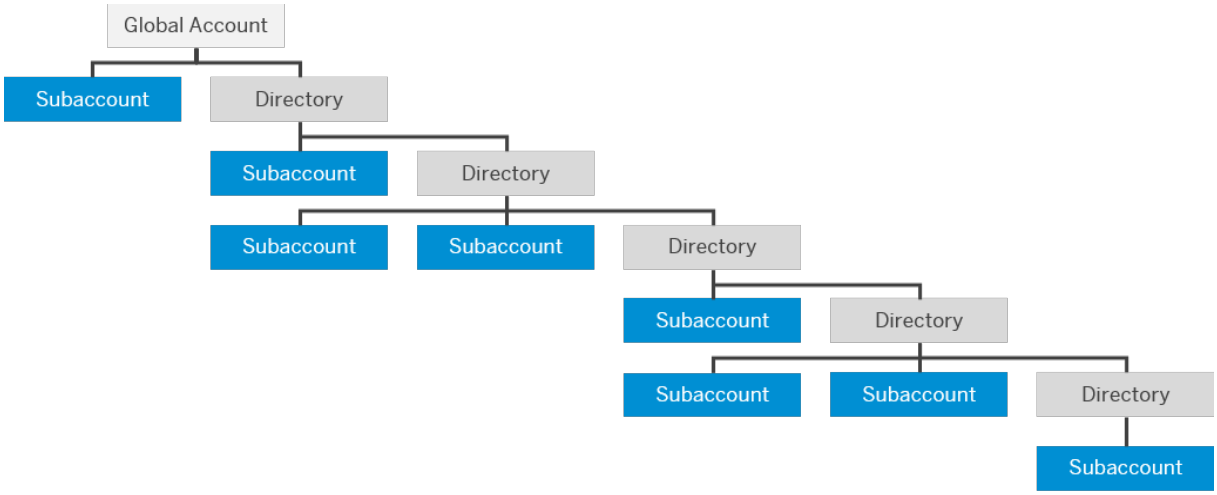
SAP BTP offers two different commercial models:



- **Consumption-based commercial model:** Your organization receives access to all current and future services that are eligible for this model. You have complete flexibility to turn services on and off and to switch between services as your business requires throughout the duration of your contract. This commercial model is available in the following flavors: SAP BTP Enterprise Agreement (SAP BTPEA), Cloud Platform Enterprise Agreement (CPEA), and Pay-As-You-Go for SAP BTP.
  For more information, see What Is the Consumption-Based Commercial Model? [page 114]
- **Subscription-based commercial model:** Your organization subscribes only to the services that you plan to use. You can then use these services at a fixed cost, irrespective of consumption.
  For more information, see What Is the Subscription-Based Commercial Model? [page 117]

Account Model

The SAP BTP cockpit is structured according to global accounts, directories, and subaccounts:



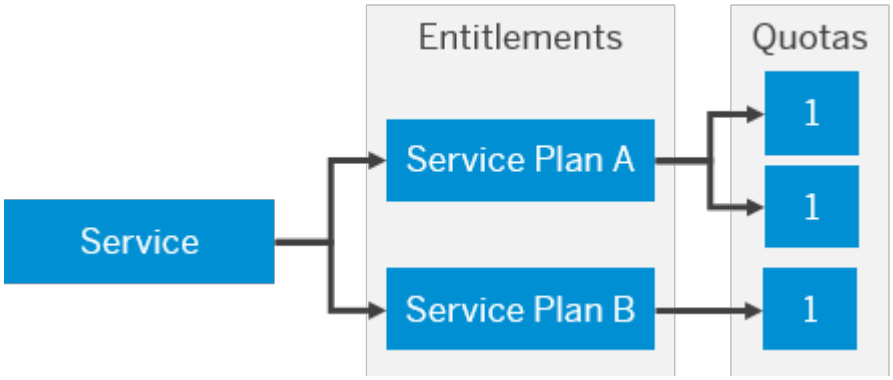
- A **global account** is the realization of a contract you or your company has made with SAP. A global account is used to manage subaccounts, members, entitlements and quotas. You receive entitlements and quotas to use platform resources per global account and then distribute the entitlements and quotas to the subaccount for actual consumption. There are two types of commercial models for global accounts: consumption-based model and subscription-based model. See Commercial Models [page 113]
- With **directories**, you can organize and manage your subaccounts according to your technical and business needs.

- In **subaccounts**, you deploy applications, use services, and manage your subscriptions. You can use them to manage members, authorizations, and entitlements.
  In the SAP BTP, Cloud Foundry environment, the subaccount is divided into one or more spaces, which is where application development, deployment, and maintenance take place.

For more information, see Account Model [page 121].

Entitlements and Quotas

When you purchase an enterprise account, you're entitled to use a specific set of resources, such as the amount of memory that can be allocated to your applications.



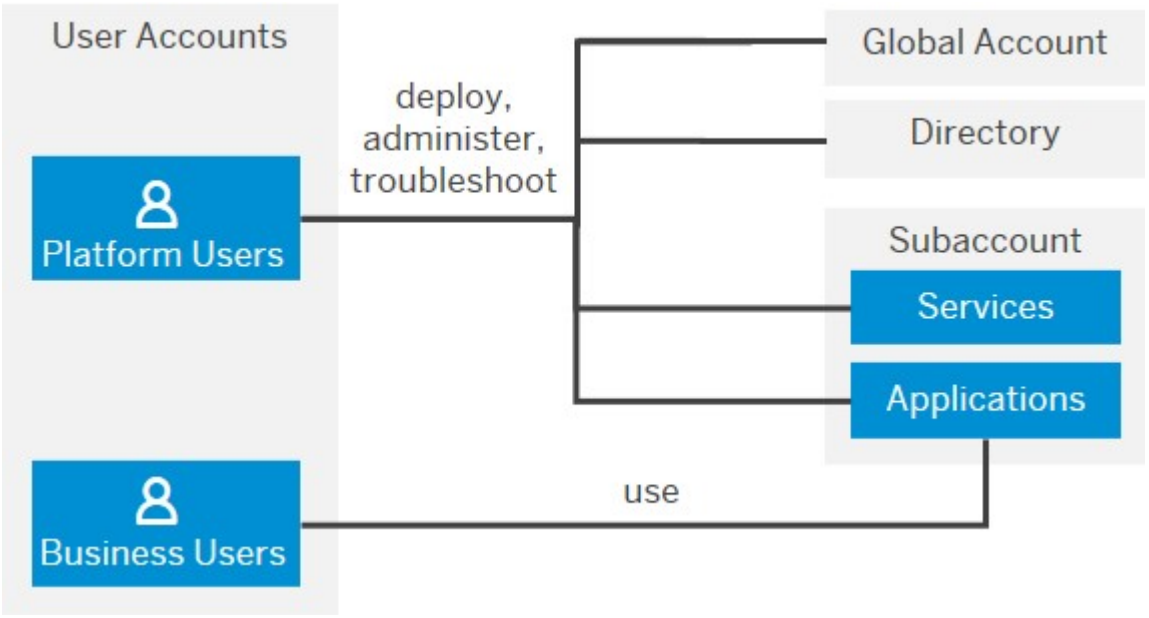
- On SAP BTP, all external dependencies such as databases, messaging systems, files systems, and so on, are **services**. In this context, multitenant applications and environments are considered services.
  Each service has one or more **service plans** available. A service plan is the representation of the costs and benefits for a given variant of a particular service. For instance, a database may be configured with various "T-shirt sizes", each of which is a different service plan.
- An **entitlement** is your right to provision and consume a resource. In other words, entitlements are **the service plans** that you're entitled to use.
- A **quota** represents the numeric quantity of a service plan that you're entitled to consume in your global account and its subaccounts.

For more information, see Entitlements and Quotas [page 127].

User and Member Management

On SAP BTP, user management takes place at all levels from global account to environment. There are different types of users, such as depending on their roles in the company.

User accounts enable users to log on to SAP BTP, access subaccounts, and to use applications according to the permissions granted to them. We distinguish between two types of users:



- **Platform users** are usually developers, administrators or operators who deploy, administer, and troubleshoot accounts, applications and services on SAP BTP.
- **Business users** use the applications that are deployed to SAP BTP. For example, the end users of SaaS apps or services, such as SAP Build Work Zone, or end users of your custom applications are business users.

Member management refers to managing permissions for platform users. A member is a user who is assigned to an SAP BTP global account or subaccount. Administrators can add users to global accounts and subaccounts and assign roles to them as needed. You can use predefined roles, for example the administrator role for managing subaccount members.

User management refers to managing authentication and authorization for your business users.

For more information, see User and Member Management [page 133].

Tools, Programming Models, Programming Languages, and APIs

SAP BTP provides various programming languages and tools for your development project:

- SAP BTP includes many tools to help you develop and manage applications, and connect them to your on-premise systems. For more information, see Tools [page 151].
- SAP BTP supports many different programming languages; the availability of each depends on the development environment you're using. For more information, see Programming Languages [page 153].
- The SAP Cloud Application Programming Model offers a consistent end-to-end programming model that includes languages, libraries, and APIs that are tailored for full-stack development on SAP BTP. For more information, see Programming Models [page 154].
- Depending on your use case, you can choose between different offerings for continuous integration and delivery. For more information, see Continuous Integration and Delivery (CI/CD) [page 155].
- Discover and consume APIs to manage, build, and extend the core capabilities of SAP BTP. For more information, see APIs [page 158].

2.1 Solutions and Services

Consume the solutions and services by SAP BTP according to your preferred development environment and use cases.

Solutions

SAP BTP offers fast in-memory processing, sustainable, agile **solutions** and **services** to integrate data and extend applications, and fully embedded analytics and intelligent technologies.

Services

Services enable, facilitate, or accelerate the development of business applications and other platform services on SAP BTP. Services are grouped into the following service types:

- **Business services**: Services that enable, facilitate, or accelerate the development of business process components or provide industry-specific functionalities or content within a business application.
- **Technical services**: Services that enable, facilitate, or accelerate the development of general or domain independent content within a business application, independent of the application's business process or task.

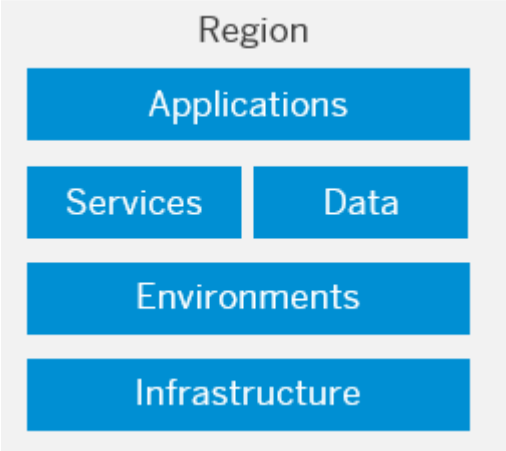
You find all available services, solutions, and use cases in the SAP Discovery Center.

Use the service catalog to access service-specific resources:

2.2 Regions

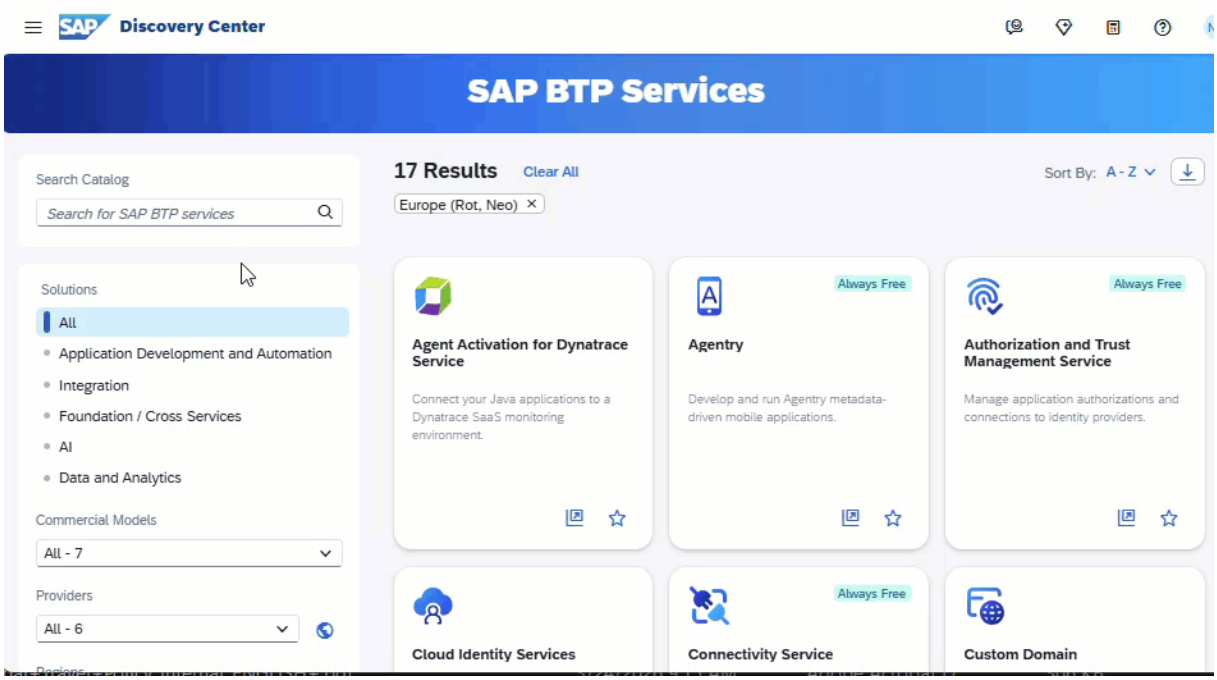
You can deploy applications in different **regions**. Each region represents a geographical location (for example, Europe, US East) where applications, data, or services are hosted.

About SAP BTP Regions



Regions are provided either by SAP or by our Infrastructure-as-a-Service (IaaS) partners Amazon Web Services (AWS), Microsoft Azure, Google Cloud, and Alibaba Cloud. The third-party region providers operate the infrastructure layer of the regions, whereas SAP operates the platform layer and Cloud Foundry.

For an overview of all available regions, see SAP Discovery Center:



Selecting a Region

A region is chosen at the subaccount level. For each subaccount, you select exactly one region. The selection of a region is dependent on many factors: For example, application performance (response time, latency) can be optimized by selecting a region close to the user. For more information, see Selecting a Region.

Deploying Applications in Regions

When deploying applications, consider that a subaccount is associated with a particular region and that this is independent of your own location. You may be located in the United States, for example, but operate your subaccount in a region in Europe. For more information on subaccounts, see Subaccounts [page 122].

To deploy an application in more than one region, execute the deployment separately for each host. For more information, see Deploy an Application [page 2862].

Within a region, there can be multiple instances of the SAP BTP, Cloud Foundry environment. When creating a subaccount, SAP BTP automatically assigns the account to a specific instance of the environment. Several subaccounts of the same global account can be located in different datacenters in one region. This also affects the format of the API endpoint URL that is displayed in the cockpit after enabling Cloud Foundry in your subaccount. There are two possible formats for the API endpoint URL, either displayed with or without an index. Here's an example for **eu10**:

> Example
>
> - https://api.cf.eu10.hana.ondemand.com
> - https://api.cf.eu10-<XXX>.hana.ondemand.com

In both cases, the subaccount is located in the region eu10. The differences in the URLs are only an indicator of technical details on the side of SAP BTP and do not affect the functionality of your applications. For information on enabling Cloud Foundry, see Create Orgs [page 2793].

High Availability

SAP has a number of processes in place to support resilience in SAP BTP, and provides different offerings so that you can support the high availability of your applications. For more information, see Resilience, High Availability, and Disaster Recovery [page 59].

EU Access

Some customer contracts include EU Access, which restricts processing of personal data to EEA/Switzerland. If the global account is marked with EU Access, the actual EU Access compliance status of subaccounts will be displayed during creation of subaccounts.

> **Note**
>
> If you require a subaccount with EU Access, make sure to select a provider and region where EU Access is available. Regions that allow for EU Access are labeled as such in the map view filter in the SAP Discovery Center.
>
> For some services, EU Access is generally not available, not even if the provider and region support EU Access.

Related Information

Regions and API Endpoints Available for the Cloud Foundry Environment [page 19]
Regions and API Endpoints for the ABAP Environment [page 43]
Regions for the Kyma Environment [page 51]

2.2.1 Regions and API Endpoints Available for the Cloud Foundry Environment

Find the IPs you need to configure for inbound and outbound communication according to your region.

> **Note**
>
> If you are using the Cloud Connector and/or the Destination service for connections between your SAP BTP applications and your local network or another (web-based) target system, and you restrict access by allowlisting IP addresses in your firewall rules, you must add connectivity-specific IPs to these rules for the respective SAP BTP region(s).
>
> For the Destination service, both the egress and ingress IPs (NAT and LB IPs) may be relevant.
>
> For more information, see:
>
> - Prerequisites: Network (Cloud Connector)
> - Configure Destination Service IPs in Firewall Rules (Destination service)

Regions for Enterprise Accounts

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Microsoft Azure	eu20	Europe (Netherlands)	cf-eu20	westeurope	**cf-eu20:** 52.149.67.35, 20.82.96.175, 20.82.96.178, 20.82.96.211, 20.82.96.244, 20.82.96.220, 20.82.96.227, 20.82.97.50, 20.82.96.240, 20.82.96.234, 20.82.97.38, 20.82.96.222, 20.82.96.233, 20.82.96.248, 20.82.97.31, 20.82.97.45, 52.149.96.147, 20.56.169.152, 20.56.169.69, 20.56.169.0, 20.56.169.41, 20.56.169.58, 20.56.169.161, 20.56.169.116, 20.56.169.167, 20.56.169.50, 20.56.169.175, 20.56.169.131, 20.56.169.66, 20.56.169.71, 20.56.169.138, 20.56.169.91, 52.142.226.14, 20.86.1.84, 20.86.1.80, 20.86.0.233, 20.86.1.131, 20.86.1.54, 20.86.1.128, 20.86.1.134, 20.86.1.163,	**cf-eu20:** 40.119.153.88 **cf-eu20-001:** 20.82.83.59 **cf-eu20-002:** 98.64.76.0	**cf-eu20:** api.cf.eu20.hana.ondemand.com **cf-eu20-001:** api.cf.eu20-001.hana.ondemand.com **cf-eu20-002:** api.cf.eu20-002.hana.ondemand.com	**cf-eu20:** eu20.hana.ondemand.com **cf-eu20-001:** eu20-001.hana.ondemand.com **cf-eu20-002:**eu20-002.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
					20.86.1.15, 20.86.0.250, 20.86.1.107, 20.86.1.157, 20.86.0.253, 20.86.1.12, 20.86.1.97 **cf-eu20-001:** 20.54.248.90, 20.54.248.142, 20.54.250.88, 20.54.250.91, 20.56.17.109, 20.56.18.130, 20.56.18.132, 20.56.18.179, 20.86.17.114, 20.86.17.142, 20.86.17.153, 20.86.17.159 **cf-eu20-002:** 20.86.83.34, 50.85.208.116/31, 51.105.247.44, 134.149.194.28/31, 172.201.130.235, 172.211.56.76/31				
Microsoft Azure	eu22	Europe (Frankfurt)	cf-eu22	germanywestcentral	4.182.249.47, 20.52.249.79, 20.218.136.116	9.141.129.192	api.cf.eu22.hana.ondemand.com	eu22.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Microsoft Azure	ap20	Australia (Sydney)	cf-ap20	Australia East	40.82.211.52, 40.82.206.131, 20.70.176.247, 20.40.81.59, 20.40.80.246, 20.40.81.36, 20.70.208.178, 20.70.208.228, 20.70.208.235, 20.70.201.155, 20.70.201.89, 20.70.201.66	20.53.99.41	api.cf.ap20.hana.ondemand.com	ap20.hana.ondemand.com	Open Cockpit
Microsoft Azure	ap21	Singapore	cf-ap21	Southeast Asia	40.90.179.153, 20.198.169.214, 20.198.168.45, 20.198.169.5, 40.90.170.226, 20.198.225.78, 20.198.225.102, 20.198.225.27, 40.90.162.117, 20.191.154.174, 20.191.154.191, 20.191.154.193	20.184.61.122	api.cf.ap21.hana.ondemand.com	ap21.hana.ondemand.com	Open Cockpit
Microsoft Azure	br20	Brazil (São Paulo)	cf-br20	Brazil South	20.206.161.114, 20.206.249.230, 191.233.21.237	4.228.118.21	api.cf.br20.hana.ondemand.com	br20.hana.ondemand.com	Open Cockpit
Microsoft Azure	ca20	Canada Central (Toronto)	cf-ca20	canadacentral	4.172.210.105, 4.172.226.192, 4.204.51.0, 4.248.154.171, 20.63.122.64, 20.151.145.75	4.172.17.29	api.cf.ca20.hana.ondemand.com	ca20.hana.ondemand.com	Open Cockpit
Microsoft Azure	cn20	China (North 3)	cf-cn20	ChinaNorth3	159.27.65.75, 159.27.107.117, 159.27.217.233	143.64.224.195	api.cf.cn20.platform.sapcloud.cn	cn20.platform.sapcloud.cn	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Microsoft Azure	uk20	UK South (London)	cf-uk20	uksouth	20.0.136.123, 20.39.216.30, 20.117.187.163	131.145.59.204	api.cf.uk20.hana.ondemand.com	uk20.hana.ondemand.com	Open Cockpit
Microsoft Azure	us20	US West (WA)	cf-us20	West US 2	40.90.195.191, 20.57.129.106, 20.57.128.95, 20.57.128.118, 40.90.209.71, 20.72.210.109, 20.72.209.240, 20.72.209.187, 40.90.200.224, 40.90.201.197, 40.90.201.85, 40.90.200.237	40.91.120.100	api.cf.us20.hana.ondemand.com	us20.hana.ondemand.com	Open Cockpit
Microsoft Azure	jp20	Japan (Tokyo)	cf-jp20	Japan East	52.185.186.130, 20.194.193.229, 20.194.193.167, 20.194.194.97, 40.81.200.207, 20.78.122.9, 20.78.121.237, 20.78.122.8, 20.40.96.175, 20.78.2.104, 20.78.2.106, 20.78.2.107	20.43.89.91	api.cf.jp20.hana.ondemand.com	jp20.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Microsoft Azure	us21	US East (VA)	cf-us21	East US	**cf-us21:** 40.90.251.147, 52.146.1.155, 20.51.255.236, 52.146.1.223, 40.90.232.167, 20.55.49.185, 20.55.49.92, 20.55.49.186, 40.90.231.101, 52.151.248.29, 52.146.15.82, 52.146.10.227 **cf-us21-001:** 20.84.76.184, 20.121.56.69, 40.90.238.252/31, 52.188.229.202, 57.151.106.208/31, 172.174.107.252/31	**cf-us21:** 40.88.52.17 **cf-us21-001:** 20.241.255.189	api.cf.us21.hana.ondemand.com	us21.hana.ondemand.com	Open Cockpit
Microsoft Azure	ch20	Switzerland (Zurich)	cf-ch20	Switzerland North	20.208.128.83, 20.208.128.86, 20.208.128.87, 20.208.128.88, 51.103.208.79, 51.103.208.81, 51.103.208.85, 51.103.208.87, 51.107.2.38, 51.107.2.50, 51.107.2.52, 51.107.2.54	20.208.56.83	api.cf.ch20.hana.ondemand.com	ch20.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Amazon Web Services	br10	Brazil (São Paulo)	cf-br10	sa-east-1	52.67.245.111, 18.231.45.151, 54.207.173.126, 18.230.81.234, 18.229.169.29, 54.94.110.127, 18.231.101.158, 177.71.170.199, 54.232.227.140, 52.67.251.43, 54.232.20.181, 54.94.136.11, 52.67.221.224, 18.229.54.222, 54.232.250.83, 18.228.194.102, 18.228.198.142, 177.71.168.150, 3.44.192.224/27	18.229.91.150, 52.67.135.4, 54.232.179.204, 18.228.53.198, 52.67.149.240, 54.94.179.209, 18.96.66.56/29	api.cf.br10.hana.ondemand.com	br10.hana.ondemand.com	Open Cockpit
Amazon Web Services	jp10	Japan (Tokyo)	cf-jp10	ap-north-east-1	54.238.10.97, 54.250.43.250, 52.192.218.156, 35.74.54.33, 18.177.86.79, 35.74.144.49, 3.114.115.232, 18.179.150.168, 54.249.134.63, 18.179.66.68, 54.250.33.48, 54.95.22.24, 52.198.77.221, 35.73.255.50, 54.178.62.192, 3.113.232.224, 52.198.66.114, 13.230.215.218, 3.44.137.192/27	3.114.248.68, 3.113.252.15, 13.114.117.83, 18.178.155.134, 57.180.140.5, 57.180.145.179, 18.99.86.144/29	api.cf.jp10.hana.ondemand.com	jp10.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Amazon Web Services	ap10	Australia (Sydney)	cf-ap10	ap-southeast-2	52.62.223.36, 13.55.100.204, 13.54.168.75, 13.55.239.117, 13.210.173.131, 13.54.77.205, 13.237.182.31, 52.65.102.82, 54.79.72.145, 13.236.142.207, 54.79.43.227, 13.54.252.220, 54.79.26.135, 13.54.220.129, 13.236.59.235, 13.238.9.23, 54.153.242.51, 3.105.234.54, 18.98.199.192/27	13.236.220.84, 13.211.73.244, 3.105.95.184, 13.55.188.95, 3.105.212.249, 3.106.45.106, 18.98.199.32/29	api.cf.ap10.hana.ondemand.com	ap10.hana.ondemand.com	Open Cockpit
Amazon Web Services	ap11	Asia Pacific (Singapore)	cf-ap11	ap-southeast-1	13.251.40.148, 13.228.68.14, 13.251.49.36, 52.76.185.92, 13.229.13.240, 54.251.74.134, 52.220.111.202, 54.179.77.154, 52.76.123.164, 54.179.253.138, 13.213.119.83, 3.1.38.48, 52.76.114.209, 13.213.105.43, 13.213.132.88, 13.250.92.77, 18.140.150.56, 18.140.255.164, 18.99.38.96/27	18.139.147.53, 3.0.9.102, 18.140.39.70, 13.229.158.122, 18.140.228.217, 52.74.215.89, 3.44.7.128/29	api.cf.ap11.hana.ondemand.com	ap11.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Amazon Web Services	ap12	Asia Pacific (Seoul)	cf-ap12	ap-northeast-2	15.165.116.197, 54.180.53.68, 3.35.252.222, 52.78.49.16, 15.164.33.162, 3.34.19.116, 3.36.2.67, 3.35.57.231, 15.164.254.80, 3.36.165.189, 52.78.38.74, 15.165.249.251, 13.124.251.247, 13.124.16.17, 15.165.83.237, 18.98.130.128/27	3.35.106.215, 3.35.255.45, 3.35.215.12, 13.209.236.215, 43.201.194.105, 43.202.204.5, 18.98.130.112/29	api.cf.ap12.hana.ondemand.com	ap12.hana.ondemand.com	Open Cockpit
Amazon Web Services	ca10	Canada (Montreal)	cf-ca10	ca-central-1	35.182.118.205, 35.182.198.31, 3.98.159.3, 3.96.101.45, 15.222.120.34, 3.96.14.215, 3.97.48.154, 3.97.119.250, 35.182.95.49, 3.97.228.23, 35.182.185.156, 3.98.252.245, 15.223.62.0, 99.79.181.241, 3.98.167.60, 99.79.110.245, 52.60.239.204, 52.60.212.33, 18.99.6.128/27	35.183.74.34, 35.182.75.101, 3.98.102.153, 15.157.88.166, 3.98.202.222, 52.60.210.33, 18.99.6.72/29	api.cf.ca10.hana.ondemand.com	ca10.hana.ondemand.com	Feature Set B

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Amazon Web Services	eu10	Europe (Frankfurt)	cf-eu10	eu-central-1	**cf-eu10:** 52.59.128.222, 52.28.241.88, 18.184.81.94, 3.67.200.70, 3.68.51.135, 3.124.174.204, 3.68.31.37, 3.67.58.183, 3.67.0.172, 3.67.244.62, 3.126.117.58, 3.66.100.105, 3.68.13.226, 3.126.45.133, 3.67.249.135, 18.194.183.183, 3.67.246.74, 3.66.68.201, 3.68.0.70, 52.28.56.202, 3.126.95.250, 3.66.68.127, 18.195.244.40, 3.67.107.121, 3.67.24.253, 18.193.50.255, 3.121.35.143, 18.96.47.192/27 **cf-eu10-002:** 18.198.196.89, 18.193.21.232, 3.65.9.91, 52.29.190.137, 18.197.134.65, 3.67.182.154, 3.67.255.232, 3.66.249.150, 3.68.44.236, 18.96.47.96/27	3.124.208.223, 3.122.209.241, 3.124.222.77, 18.159.31.22, 3.69.186.98, 3.77.195.119, 3.40.3.168/29 **cf-eu10-002:** 3.126.229.22, 18.193.180.19, 3.64.227.236, 18.153.123.11, 3.121.37.195, 3.73.215.90, 3.40.3.152/29 **cf-eu10-003:** 3.127.77.3, 3.64.196.58, 18.156.151.247, 18.197.252.154, 3.79.137.29, 52.58.93.50, 3.40.3.160/29 **cf-eu10-004:** 3.70.38.218, 18.196.206.8, 3.65.185.47, 3.73.109.100, 3.73.8.210, 52.59.18.183, 5.60.96.208/29 **cf-eu10-005:** 3.78.172.245, 3.122.31.132, 18.193.56.244, 3.40.3.184/29	**cf-eu10:** api.cf.eu10.hana.ondemand.com **cf-eu10-002:** api.cf.eu10-002.hana.ondemand.com **cf-eu10-003:** api.cf.eu10-003.hana.ondemand.com **cf-eu10-004:** api.cf.eu10-004.hana.ondemand.com **cf-eu10-005:** api.cf.eu10-005.hana.on	**cf-eu10:** eu10.hana.ondemand.com **cf-eu10-002:** eu10-002.hana.ondemand.com **cf-eu10-003:** eu10-003.hana.ondemand.com **cf-eu10-004:** eu10-004.hana.ondemand.com **cf-eu10-005:** eu10-005.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
					cf-eu10-003: 3.64.131.199, 3.64.88.217, 3.64.142.243, 18.198.18.157, 3.68.40.83, 3.67.235.98, 3.68.17.221, 18.198.149.19, 3.68.38.23, 18.96.47.64/27 **cf-eu10-004:** 3.69.195.103, 3.64.170.167, 3.68.176.248, 3.121.49.211, 18.197.219.60, 3.70.38.84, 5.60.96.176/28 **cf-eu10-005:** 35.159.192.144/28		de-mand.com		
Amazon Web Services	eu11	Europe (Frankfurt)	cf-eu11	eu-central-1	18.156.140.38, 3.121.55.100, 35.156.198.246, 52.59.77.121, 18.185.57.85, 3.121.79.209, 3.67.237.8, 35.156.31.32, 3.65.63.251, 3.122.176.63, 18.198.13.57, 18.157.114.142, 18.184.172.97, 18.159.180.188, 35.157.5.44, 18.96.47.32/27	18.156.209.198, 18.157.105.117, 3.124.207.41, 3.66.26.249, 3.72.216.204, 3.74.99.245, 3.40.3.136/29	api.cf.eu11.hana.ondemand.com	eu11.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Amazon Web Services	eu13	Europe (Milan)	cf-eu13	eu-south-1	15.161.87.249, 15.161.194.156, 18.99.209.0/27, 35.152.224.143	15.161.2.170, 18.102.32.205, 18.102.85.10, 18.99.209.120/29	api.cf.eu13.hana.ondemand.com	eu13.hana.ondemand.com	Open Cockpit
Amazon Web Services	us10	US East (VA)	cf-us10	us-east-1	**cf-us10:** 52.200.16.71, 52.23.123.125, 52.202.170.155, 18.97.43.224/27 **cf-us10-001:** 52.0.214.195, 18.213.153.162, 54.227.144.195, 3.225.73.158, 34.233.151.91, 23.23.172.117, 3.222.22.16, 34.238.1.234, 34.194.239.31, 3.225.44.56, 34.201.208.150, 75.101.157.228, 3.41.186.96/27 **cf-us10-002:** 54.162.233.194, 34.206.160.141, 35.168.80.144, 18.232.28.65, 54.82.224.146, 3.221.4.74, 18.211.12.227, 54.159.45.198, 72.44.51.245, 3.41.186.192/27 **cf-us10-003:** 3.41.185.128/27	**cf-us10:** 18.213.242.208, 3.214.110.153, 34.205.56.51, 18.97.7.240/29 **cf-us10-001:** 3.227.182.44, 52.86.131.53, 3.220.114.17, 44.218.82.203, 44.219.57.163, 50.16.106.103, 18.97.34.248/29 **cf-us10-002:** 107.20.66.86, 54.234.152.59, 34.202.68.0, 3.214.116.95, 54.144.230.36, 54.226.37.104, 18.97.7.216/29 **cf-us10-003:** 3.41.185.0/29, 3.41.185.0, 3.41.185.1, 3.41.185.2	**cf-us10:** api.cf.us10.hana.ondemand.com **cf-us10-001:** api.cf.us10-001.hana.ondemand.com **cf-us10-002:** api.cf.us10-002.hana.ondemand.com	**cf-us10:** us10.hana.ondemand.com **cf-us10-001:** us10-001.hana.ondemand.com **cf-us10-002:** us10-002.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Amazon Web Services	us11	US West (Oregon)	cf-us11	us-west-2	44.244.167.160/28	34.211.82.149, 35.95.238.236, 100.20.19.69, 18.98.1.8/29	api.cf.us11.hana.ondemand.com	us11.hana.ondemand.com	Open Cockpit
Google Cloud	us30	US Central (IA)	cf-us30	us-central1	35.193.171.152, 35.193.168.31, 35.202.69.204, 35.202.175.147, 35.193.69.164, 35.202.1.6, 23.236.63.113, 35.193.30.116, 35.202.66.196, 34.68.152.205, 35.222.158.222, 104.197.20.168, 35.232.105.70, 35.224.211.196, 35.222.192.158, 35.193.8.172, 34.171.4.220, 34.172.37.175, 34.170.206.220, 34.172.145.231, 35.222.38.254, 35.239.28.216, 34.134.91.47, 34.123.17.36, 35.202.205.85, 34.118.207.84, 35.193.6.192, 34.122.222.203, 104.197.157.121, 34.135.159.154, 35.223.208.27, 146.148.74.171, 34.132.192.46, 34.68.109.37, 104.198.49.58, 35.225.164.132	35.184.169.79	api.cf.us30.hana.ondemand.com	us30.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Google Cloud	eu30	Europe (Frankfurt)	cf-eu30	europe-west3	34.107.28.38, 34.141.10.217, 34.141.116.52, 34.141.1.228, 34.141.123.52, 34.141.46.51, 34.89.130.182, 34.89.146.167, 34.89.203.91, 34.89.232.158, 34.89.243.40, 35.198.83.71, 35.234.65.38, 35.242.208.222, 35.246.155.42, 35.246.171.35, 34.141.28.26, 34.159.160.86, 34.107.19.175, 34.159.165.29, 35.242.240.154, 34.141.73.130, 34.159.27.236, 34.89.152.211, 35.242.194.75, 35.246.235.253, 34.159.127.190, 34.141.82.126, 35.234.69.102, 34.89.231.53, 34.159.188.133, 35.246.203.194, 34.159.201.78, 34.141.112.232, 35.198.84.213, 34.89.165.33	35.198.143.110	api.cf.eu30.hana.ondemand.com	eu30.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Google Cloud	in30	India (Mumbai)	cf-in30	asia-south1	34.93.27.36, 34.93.89.145, 34.93.92.210, 34.93.137.163, 34.93.148.247, 34.93.166.164, 34.93.180.0, 34.93.221.129, 35.200.131.125, 35.200.144.1, 35.200.175.62, 35.200.183.224, 35.200.194.175, 35.200.198.26, 35.200.209.142, 35.244.29.120, 35.200.137.225, 34.100.186.241, 35.200.169.254, 35.200.151.131, 35.200.252.103, 35.244.15.103, 35.244.16.76, 34.93.255.115, 35.244.53.153, 35.200.168.60, 35.200.222.30, 34.100.178.164, 35.244.2.193, 34.93.11.49, 34.100.211.195, 34.100.151.15, 34.93.95.83, 34.100.215.143, 34.93.205.174, 34.93.159.24	34.93.125.74	api.cf.in30.hana.ondemand.com	in30.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Google Cloud	il30	Israel (Tel Aviv)	cf-il30	me-west1	34.165.0.14, 34.165.0.115, 34.165.5.181, 34.165.5.246, 34.165.7.73, 34.165.12.173, 34.165.16.177, 34.165.16.210, 34.165.17.27, 34.165.18.240, 34.165.21.242, 34.165.24.112, 34.165.26.162, 34.165.37.171, 34.165.38.114, 34.165.40.240, 34.165.41.254, 34.165.80.207, 34.165.81.54, 34.165.110.15, 34.165.136.9, 34.165.150.108, 34.165.168.74, 34.165.171.197, 34.165.172.4, 34.165.194.20, 34.165.222.104, 34.165.223.90, 34.165.228.95, 34.165.231.165	34.165.59.26	api.cf.il30.hana.ondemand.com	il30.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Google Cloud	jp30	Japan (Osaka)	cf-jp30	asia-northeast2	34.97.10.131, 34.97.19.128, 34.97.24.15, 34.97.27.89, 34.97.33.154, 34.97.36.44, 34.97.42.85, 34.97.52.48, 34.97.71.122, 34.97.75.108, 34.97.76.16, 34.97.79.22, 34.97.116.150, 34.97.117.3, 34.97.126.206, 34.97.131.43, 34.97.140.64, 34.97.141.186, 34.97.147.230, 34.97.177.45, 34.97.185.68, 34.97.186.214, 34.97.190.161, 34.97.201.159, 34.97.209.167, 34.97.210.16, 34.97.210.66, 34.97.221.101, 34.97.246.18, 34.97.246.150	34.97.168.169	api.cf.jp30.hana.ondemand.com	jp30.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Google Cloud	jp31	Japan (Tokyo)	cf-jp31	asia-northeast1	34.84.6.224, 34.84.32.55, 34.84.34.158, 34.84.126.44, 34.84.134.209, 34.84.141.254, 34.84.220.103, 34.84.225.24, 34.84.231.92, 34.85.20.250, 34.85.75.175, 34.146.79.159, 34.146.122.64, 34.146.132.93, 34.146.132.147, 34.146.174.212, 34.146.180.55, 34.146.195.200, 34.146.196.228, 34.146.212.140, 34.146.251.151, 34.146.252.171, 35.189.143.202, 35.189.159.75, 35.189.159.240, 35.194.111.248, 35.194.117.168, 35.200.34.120, 35.221.109.129, 35.243.117.227	34.84.96.118	api.cf.jp31.hana.ondemand.com	jp31.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Google Cloud	sa30	KSA (Dammam - KSA Regulated Customers)	cf-sa30	me-central2	34.166.4.164, 34.166.4.182, 34.166.8.22, 34.166.10.68, 34.166.19.145, 34.166.20.65, 34.166.37.166, 34.166.38.149, 34.166.39.3, 34.166.40.145, 34.166.41.104, 34.166.44.55, 34.166.46.73, 34.166.46.216, 34.166.47.18, 34.166.50.229, 34.166.52.69, 34.166.53.147, 34.166.55.78, 34.166.57.177, 34.166.60.109, 34.166.61.61, 34.166.61.119, 34.166.61.169, 34.166.61.173, 34.166.62.51, 34.166.62.206, 34.166.63.160, 34.166.64.6, 34.166.65.231	34.166.32.46	api.cf.sa30.hana.ondemand.com	sa30.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Google Cloud	sa31	KSA (Dammam - KSA Non-Regulated Customers)	cf-sa31	me-central2	34.166.3.142, 34.166.11.228, 34.166.14.210, 34.166.38.222, 34.166.41.150, 34.166.43.17, 34.166.43.183, 34.166.48.219, 34.166.66.132, 34.166.69.32, 34.166.73.246, 34.166.78.126, 34.166.80.128, 34.166.87.5, 34.166.87.129, 34.166.87.229, 34.166.90.71, 34.166.93.161, 34.166.102.223, 34.166.104.146, 34.166.105.14, 34.166.106.220, 34.166.115.33, 34.166.115.46/31, 34.166.116.103, 34.166.117.89, 34.166.123.179, 34.166.125.172, 34.166.127.149	34.166.72.122	api.cf.sa31.hana.ondemand.com	sa31.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Google Cloud	ap30	Australia (Sydney)	cf-ap30	australia-southeast1	34.40.139.113, 34.40.148.106, 34.40.148.145, 34.87.197.203, 34.87.227.150, 34.87.246.15, 34.87.246.56, 34.116.64.18, 34.116.71.248, 34.116.93.109, 34.116.96.225, 34.116.98.18, 34.151.82.192, 34.151.84.167, 34.151.92.254, 34.151.95.105, 34.151.110.186, 34.151.113.55, 35.189.5.90, 35.189.6.201, 35.189.24.237, 35.189.29.223, 35.197.168.71, 35.201.9.202, 35.201.25.163, 35.201.28.91, 35.244.87.121, 35.244.96.216, 35.244.100.190, 35.244.124.253	35.244.71.16	api.cf.ap30.hana.ondemand.com	ap30.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Google Cloud	br30	Brazil (São Paulo)	cf-br30	southamerica-east1	34.95.136.152, 34.95.141.244, 34.95.169.207, 34.95.175.231, 34.95.181.247, 34.95.184.165, 34.95.200.14, 34.95.201.213, 34.95.225.218, 34.95.243.255, 34.95.246.4, 34.151.201.167, 34.151.205.45, 34.151.212.79, 34.151.214.13, 34.151.221.155, 34.151.234.215, 34.151.243.44, 34.151.246.168, 34.151.248.189, 35.198.10.217, 35.198.12.163, 35.198.39.142, 35.198.49.194, 35.199.67.49, 35.199.97.244, 35.199.110.212, 35.199.114.60, 35.247.219.74, 35.247.250.157	34.95.189.118	api.cf.br30.hana.ondemand.com	br30.hana.ondemand.com	Open Cockpit
Alibaba Cloud	cn40	China (Shanghai)	cf-cn40	cn-shanghai	101.132.190.155, 106.14.165.33, 106.14.184.113	139.224.7.71	api.cf.cn40.platform.sapcloud.cn	cn40.platform.sapcloud.cn	Open Cockpit
SAP Cloud Infrastructure	ae01	UAE (Dubai)	cf-ae01	ap-ae-1	130.214.81.0/25, 130.214.83.128/25	130.214.81.128/26, 130.214.83.0/26	api.cf.ae01.hana.ondemand.com	ae01.hana.ondemand.com	Open Cockpit

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
SAP Cloud Infrastructure	ap01	Australia (Sydney)	cf-ap01	ap-au-1	130.214.42.0/25, 130.214.151.128/25	130.214.42.128/26, 130.214.151.0/26	api.cf.ap01.hana.ondemand.com	ap01.hana.ondemand.com	Open Cockpit
SAP Cloud Infrastructure	eu01	Europe (Frankfurt) - EU Access Only	cf-eu01	eu-de-2	130.214.199.0/25, 130.214.252.0/25	130.214.199.128/28, 130.214.252.224/28	api.cf.eu01.hana.ondemand.com	eu01.hana.ondemand.com	Open Cockpit
SAP Cloud Infrastructure	eu02	Europe (Rot) - SAP EU Access	cf-eu02	eu-de-1	130.214.61.128/25, 130.214.159.0/25	130.214.61.0/26, 130.214.159.128/26	api.cf.eu02.hana.ondemand.com	eu02.hana.ondemand.com	Open Cockpit
SAP Cloud Infrastructure	jp01	Japan (Tokyo)	cf-jp01	ap-jp-1	130.214.44.0/25, 130.214.54.128/25	130.214.44.128/26, 130.214.54.0/26	api.cf.jp01.hana.ondemand.com	jp01.hana.ondemand.com	Open Cockpit
SAP Cloud Infrastructure	us01	US (Sterling)	cf-us01	na-us-1	130.214.4.0/25, 130.214.46.128/25	130.214.4.128/26, 130.214.46.0/26	api.cf.us01.hana.ondemand.com	us01.hana.ondemand.com	Open Cockpit
SAP Cloud Infrastructure	us02	US West (Colorado)	cf-us02	na-us-2	130.214.2.0/25, 130.214.185.128/25	130.214.2.128/26, 130.214.185.0/26	api.cf.us02.hana.ondemand.com	us02.hana.ondemand.com	Open Cockpit

Regions for Trial Accounts

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	Trial NAT IPs (egress, IPs for requests from a Cloud Foundry app)	LB IPs (ingress, for incoming requests)	API Endpoint	Domain	Cockpit Logon
Amazon Web Services	us10	US East (VA)	cf-us10	us-east-1	3.218.99.154, 52.72.147.227, 3.218.112.63	18.213.242.208, 3.214.110.153, 34.205.56.51	api.cf.us10.hana.ondemand.com	us10.hana.ondemand.com	Trial
Microsoft Azure	ap21	Singapore	cf-ap21	Southeast Asia	52.139.216.172, 20.195.24.178, 20.195.9.169	20.184.61.122	api.cf.ap21.hana.ondemand.com	ap21.hana.ondemand.com	Trial

> ### Note
>
> Trial accounts and subaccounts on trial can no longer be created on eu10, Europe (Frankfurt).
>
> Existing trial accounts and subaccounts are not affected.

> ### Note
>
> In the Cloud Foundry environment, IPs are controlled by the respective IaaS provider (AWS, Azure, or Google Cloud). IPs may change due to network updates on the provider side. Any planned changes will be announced at least four weeks before they take effect. For information on how to subscribe to updates, see Platform Updates and Notifications.

> ### Note
>
> In the Cloud Foundry environment, the region in which a global account was created determines the API endpoint of all subaccounts associated with it. For example, subaccounts created in a global account in region **eu10** share the API endpoint URL `api.cf.eu10.hana.ondemand.com`.

Related Information

Regions [page 17]
Regions for the Kyma Environment
Regions and API Endpoints for the ABAP Environment

Regions and Hosts Available for the Neo Environment

IP Allowlisting for Inbound Requests in SAP Cloud Integration

2.2.2 Regions and API Endpoints for the ABAP Environment

Regions for Enterprise Accounts

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from an ABAP System)	API Endpoint	Domain	Cockpit Logon
SAP Cloud Infrastructure	eu01	Germany (Frankfurt)	cf-eu01	eu-de-2	130.214.190.0/26, 130.214.228.32/27, 130.214.164.0/26	api.cf.eu01.hana.ondemand.com	eu01.hana.ondemand.com	https://eu-access.cockpit.btp.cloud.sap/
Alibaba Cloud	cn40	China (Shanghai)	cf-cn40	cn-shanghai	101.132.173.213, 47.100.166.73	api.cf.cn40.platform.sapcloud.cn	cn40.abapenv.platform.sapcloud.cn	https://cockpit.cn40.platform.sapcloud.cn/
Amazon Web Services	ap10	Australia (Sydney)	cf-ap10	ap-southeast-2	54.153.226.137, 54.153.194.85, 54.79.209.86, 13.238.93.75, 13.237.181.0, 13.54.184.162, 13.54.97.19, 15.135.57.55, 3.104.10.32, 54.253.28.16	api.cf.ap10.hana.ondemand.com	ap10.hana.ondemand.com	https://apac.cockpit.btp.cloud.sap/
Amazon Web Services	br10	Brazil (Sao Paulo)	cf-br10	sa-east-1	52.67.140.201, 54.207.133.145, 15.229.97.244, 18.228.92.201, 15.229.223.36, 15.229.57.18, 54.232.121.126, 54.233.172.106, 56.125.124.4, 56.125.36.254	api.cf.br10.hana.ondemand.com	br10.hana.ondemand.com	https://amer.cockpit.btp.cloud.sap/

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from an ABAP System)	API Endpoint	Domain	Cockpit Logon
Amazon Web Services	ca10	Canada (Montreal)	cf-ca10	ca-central-1	15.222.180.159, 15.222.175.12, 52.60.183.108, 3.97.94.144, 15.156.227.151, 15.222.146.76, 16.52.188.152, 16.54.103.237, 3.97.125.247, 3.98.40.155	api.cf.ca10.hana.ondemand.com	ca10.hana.ondemand.com	https://amer.cockpit.btp.cloud.sap/

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from an ABAP System)	API Endpoint	Domain	Cockpit Logon
Amazon Web Services	eu10	Europe (Frankfurt)	cf-eu10	eu-central-1	3.126.219.206, 63.177.3.125, 18.197.217.237, 18.198.153.44, 18.157.206.182, 52.57.94.154, 3.74.95.163, 18.156.20.40, 3.70.85.193, 3.76.177.92, 18.196.196.117,3.127.41.81, 3.69.221.68, 63.177.166.147, 3.67.230.143, 3.74.106.119, 18.199.199.153, 3.78.46.180, 3.71.168.30, 18.192.255.16, 63.183.16.189	cf-eu10: api.cf.eu10.hana.ondemand.com cf-eu10-002: api.cf.eu10-002.hana.ondemand.com cf-eu10-003: api.cf.eu10-003.hana.ondemand.com cf-eu10-004: api.cf.eu10-004.hana.ondemand.com cf-eu10-005: api.cf.eu10-005.hana.ondemand.com cf-eu10-006: api.cf.eu10-006.hana.ondemand.com	cf-eu10: eu10.hana.ondemand.com cf-eu10-002: eu10-002.hana.ondemand.com cf-eu10-003: eu10-003.hana.ondemand.com cf-eu10-004: eu10-004.hana.ondemand.com cf-eu10-005: eu10-005.hana.ondemand.com cf-eu10-006: eu10-006.hana.ondemand.com	https://emea.cockpit.btp.cloud.sap/
Amazon Web Services	eu11	Europe (Frankfurt) EU Access	cf-eu11	eu-central-1	18.157.200.44, 3.121.238.156, 3.67.47.252, 18.195.136.83, 3.120.9.225, 3.68.253.186	api.cf.eu11.hana.ondemand.com	eu11.hana.ondemand.com	https://eu-access.cockpit.btp.cloud.sap/

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from an ABAP System)	API Endpoint	Domain	Cockpit Logon
Amazon Web Services	jp10	Japan (Tokyo)	cf-jp10	ap-northeast-1	35.75.28.56, 35.74.196.78, 54.250.128.197, 52.196.122.86, 13.114.78.204, 13.193.147.200, 13.196.33.83, 18.178.108.61, 18.180.6.147, 57.181.125.121	api.cf.jp10.hana.ondemand.com	jp10.hana.ondemand.com	https://apac.cockpit.btp.cloud.sap/
Amazon Web Services	ap11	Singapore	cf-ap11	ap-southeast-1	54.179.221.168, 18.140.151.124, 54.254.21.208, 54.251.135.238, 13.214.45.63, 13.214.85.235,18.138.160.57,18.138.194.0,47.130.168.23,52.221.161.31	api.cf.ap11.hana.ondemand.com	ap11.hana.ondemand.com	https://apac.cockpit.btp.cloud.sap/
Amazon Web Services	ap12	South Korea (Seoul)	cf-ap12	ap-northeast-2	3.39.57.235, 13.125.0.129, 13.124.63.148, 13.209.63.123, 13.124.110.21, 13.125.41.13, 3.34.120.107, 3.34.76.212, 3.36.9.57, 3.38.223.6	api.cf.ap12.hana.ondemand.com	ap12.hana.ondemand.com	https://apac.cockpit.btp.cloud.sap/
Amazon Web Services	us10	US East (VA)	cf-us10	us-east-1	54.243.29.110, 18.215.92.120, 34.232.200.153, 18.232.247.104, 3.209.189.244, 18.204.158.200, 23.20.221.103, 54.86.32.250, 52.1.255.25	cf-us10: api.cf.us10.hana.ondemand.com cf-us10-001: api.cf.us10-001.hana.ondemand.com cf-us10-002: api.cf.us10-002.hana.ondemand.com	cf-us10: us10.hana.ondemand.com cf-us10-001: us10-001.hana.ondemand.com cf-us10-002: us10-002.hana.ondemand.com	https://amer.cockpit.btp.cloud.sap/

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from an ABAP System)	API Endpoint	Domain	Cockpit Logon
Amazon Web Services	us11	US West (Oregon)	cf-us11	us-west-2	44.231.192.253, 34.212.183.155, 44.247.175.75, 35.167.112.20, 35.160.187.201, 52.36.59.222, 44.245.184.113	api.cf.us11.hana.ondemand.com	us11.hana.ondemand.com	https://amer.cockpit.btp.cloud.sap/
Microsoft Azure	ap20	Australia (Sydney)	cf-ap20	australiaeast	20.211.81.46, 20.70.213.77, 20.211.46.196	api.cf.ap20.hana.ondemand.com	ap20.hana.ondemand.com	https://apac.cockpit.btp.cloud.sap/
Microsoft Azure	ap21	Singapore	cf-ap21	southeastasia	20.6.9.203, 20.198.169.36, 20.195.25.14	api.cf.ap21.hana.ondemand.com	ap21.hana.ondemand.com	https://apac.cockpit.btp.cloud.sap/
Microsoft Azure	br20	Brazil (São Paulo)	cf-br20	brazilsouth	74.163.88.199, 191.235.112.28, 20.197.249.147	api.cf.br20.hana.ondemand.com	br20.hana.ondemand.com	https://amer.cockpit.btp.cloud.sap/
Microsoft Azure	ca20	Canada Central (Toronto)	cf-ca20	canadacentral	20.220.171.110, 20.220.27.242, 20.63.53.134	api.cf.ca20.hana.ondemand.com	ca20.hana.ondemand.com	https://amer.cockpit.btp.cloud.sap/
Microsoft Azure	ch20	Switzerland (Zurich) Azure EU Access	cf-ch20	switzerlandnorth	172.162.240.234, 51.107.40.177, 51.103.221.23	api.cf.ch20.hana.ondemand.com	ch20.hana.ondemand.com	https://eu-access.cockpit.btp.cloud.sap/
Microsoft Azure	cn20	China (North 3)	cf-cn20	chinanorth3	52.130.232.107, 52.130.216.11, 159.27.72.16	api.cf.cn20.platform.sapcloud.cn	cn20.abapenv.platform.sapcloud.cn	https://cockpit.cn40.platform.sapcloud.cn/

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from an ABAP System)	API Endpoint	Domain	Cockpit Logon
Microsoft Azure	eu20	Europe (Netherlands)	cf-eu20	westeurope	74.234.204.238, 98.71.236.50, 108.143.241.97	cf-eu20: api.cf.eu20.hana.ondemand.com cf-eu20-001: api.cf.eu20-001.hana.ondemand.com	cf-eu20: eu20.hana.ondemand.com cf-eu20-001: eu20-001.hana.ondemand.com	https://emea.cockpit.btp.cloud.sap/
Microsoft Azure	eu22	Europe (Frankfurt)	cf-eu22	germanywestcentral	20.170.115.175, 20.218.226.251, 20.218.140.233	api.cf.eu22.hana.ondemand.com	eu22.hana.ondemand.com	https://emea.cockpit.btp.cloud.sap/
Microsoft Azure	jp20	Japan (Tokyo)	cf-jp20	japaneast	20.78.124.111, 20.63.141.36, 20.78.37.7	api.cf.jp20.hana.ondemand.com	jp20.hana.ondemand.com	https://apac.cockpit.btp.cloud.sap/
Microsoft Azure	us20	US West (WA)	cf-us20	westus2	20.9.136.184, 172.179.0.166, 20.9.147.50	api.cf.us20.hana.ondemand.com	us20.hana.ondemand.com	https://amer.cockpit.btp.cloud.sap/
Microsoft Azure	us21	US East (VA)	cf-us21	eastus	172.190.88.27, 20.83.171.94, 74.235.125.210	api.cf.us21.hana.ondemand.com	us21.hana.ondemand.com	https://amer.cockpit.btp.cloud.sap/
Google Cloud	ap30	Australia (Sydney)	cf-ap30	australiasoutheast1	34.40.134.54, 35.244.118.182, 35.189.59.175, 35.244.102.155, 35.244.118.9, 34.116.93.234	api.cf.ap30.hana.ondemand.com	ap30.hana.ondemand.com	https://apac.cockpit.btp.cloud.sap/

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from an ABAP System)	API Endpoint	Domain	Cockpit Logon
Google Cloud	br30	Brazil (São Paulo)	cf-br30	southamerica-east1	34.95.137.184, 35.247.201.68, 34.39.150.8, 35.247.226.31, 34.95.150.147, 34.95.207.43	api.cf.br30.hana.ondemand.com	br30.hana.ondemand.com	https://amer.cockpit.btp.cloud.sap/
Google Cloud	eu30	Europe (Frankfurt)	cf-eu30	europe-west3	34.141.88.79, 34.159.65.75, 34.141.107.207, 34.141.42.177, 34.141.126.80, 34.159.215.19	api.cf.eu30.hana.ondemand.com	eu30.hana.ondemand.com	https://emea.cockpit.btp.cloud.sap/
Google Cloud	il30	Israel (Tel Aviv)	cf-il30	me-west1	34.165.172.174, 34.165.28.224, 34.165.26.44, 34.165.46.220, 34.165.133.5, 34.165.21.8	api.cf.il30.hana.ondemand.com	il30.hana.ondemand.com	https://emea.cockpit.btp.cloud.sap/
Google Cloud	in30	India (Mumbai)	cf-in30	asia-south1	34.93.58.135, 35.200.177.49, 35.244.63.61, 34.93.41.126, 34.93.191.130, 35.200.168.232	api.cf.in30.hana.ondemand.com	in30.hana.ondemand.com	https://apac.cockpit.btp.cloud.sap/
Google Cloud	jp30	Japan (Osaka)	cf-jp30	asia-northeast2	34.97.109.65, 34.97.169.121, 34.97.118.229, 34.97.49.120, 34.97.5.235, 34.97.12.103	api.cf.jp30.hana.ondemand.com	jp30.hana.ondemand.com	https://apac.cockpit.btp.cloud.sap/
Google Cloud	jp31	Japan (Tokio)	cf-jp31	asia-northeast1	34.146.45.152, 34.84.224.174, 35.221.114.117, 34.84.69.15, 34.84.239.245, 34.146.166.32	api.cf.jp31.hana.ondemand.com	jp31.hana.ondemand.com	https://apac.cockpit.btp.cloud.sap/cockpit
Google Cloud	sa30	KSA (Dammam - KSA Regulated Customers)	cf-sa30	me-central2	34.166.122.101, 34.166.13.193, 34.166.133.79, 34.166.95.100, 34.166.102.25, 34.166.180.181	api.cf.sa30.hana.ondemand.com	sa30.hana.ondemand.com	https://emea.cockpit.btp.cloud.sap/

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	NAT IPs (egress, IPs for requests from an ABAP System)	API Endpoint	Domain	Cockpit Logon
Google Cloud	sa31	KSA (Dammam - KSA Non-Regulated Customers)	cf-sa31	me-central2	34.166.162.172, 34.166.125.210, 34.166.181.228, 34.166.70.80, 34.166.99.127, 34.166.92.192	api.cf.sa31.hana.ondemand.com	sa31.hana.ondemand.com	https://emea.cockpit.btp.cloud.sap/
Google Cloud	us30	US Central (IA)	cf-us30	us-central1	34.121.78.84, 34.122.132.185, 104.154.245.19, 34.171.92.122, 35.224.140.73, 34.31.173.8	api.cf.us30.hana.ondemand.com	us30.hana.ondemand.com	https://amer.cockpit.btp.cloud.sap/

Regions for Trial Accounts

IaaS Provider	Region	Region Name	Technical Key	Technical Key of IaaS Provider	Trial NAT IPs (egress, IPs for requests from a Cloud Foundry app)	API Endpoint	Domain	Cockpit Logon
Amazon Web Services	us10	US East (VA)	cf-us10	us-east-1	3.218.99.154, 52.72.147.227, 3.218.112.63	api.cf.us10.hana.ondemand.com	us10.hana.ondemand.com	Trial
Microsoft Azure	ap21	Singapore	cf-ap21	southeastasia	20.6.9.203, 20.198.169.36, 20.195.25.14	api.cf.ap21.hana.ondemand.com	ap21.hana.ondemand.com	Trial

> **Note**
>
> SAP Business AI Platform, ABAP environment is available in other regions than SAP AI Core. To provide access to SAP AI Core services for the available ABAP Environment regions, a mapping between regions might be required, see 3566760.

2.2.3 Regions for the Kyma Environment

To work with the Kyma environment, you must specify the region for both your subaccount and the cluster.

> **Note**
>
> In Kyma, the ingress and egress IP addresses are independent of the subaccount region. For more information, see Ingress and Egress IP Addresses [page 3534].

Subaccount Regions

When you create a subaccount, choose one of the regions listed in the following table.

Subaccount Regions for Kyma

Global Account Type	Region	IaaS Provider	Technical Key	Region Name	Plan ID

Enterprise account
Partner Test, Demo, and Development account
Trial account (available only on request; for more information, see Getting Started with a Trial Kyma Instance [page 249])
	ap21	Microsoft Azure	cf-ap21	Singapore	`azure`
					`azure_lite`
					`trial`
					`build-runtime-azure`

Enterprise account
Partner Test, Demo, and Development account
	us20	Microsoft Azure	cf-us20	US West (WA)	`azure`
					`azure_lite`
					`build-runtime-azure`

Enterprise account
Partner Test, Demo, and Development account
	jp20	Microsoft Azure	cf-jp20	Japan (Tokyo)	`azure`
					`azure_lite`
					`build-runtime-azure`

Enterprise account
Partner Test, Demo, and Development account
	us21	Microsoft Azure	cf-us21	US East (VA)	`azure`
					`azure_lite`
					`build-runtime-azure`

Enterprise account Partner Test, Demo, and Development account	us22	Microsoft Azure	cf-us22	US East 2 (VA)	`azure` `azure_lite`

Global Account Type	Region	IaaS Provider	Technical Key	Region Name	Plan ID

Enterprise account
Partner Test, Demo, and Development account
	eu20	Microsoft Azure	cf-eu20	Europe (Netherlands)	`azure`
					`azure_lite`
					`build-runtime-azure`

Enterprise account Partner Test, Demo, and Development account	eu22	Microsoft Azure	cf-eu22	Europe (Frankfurt)	`azure` `azure_lite`
Enterprise account Partner Test, Demo, and Development account	uk20	Microsoft Azure	cf-uk20	UK South (London)	`azure` `azure_lite`

Enterprise account
Partner Test, Demo, and Development account
	ap20	Microsoft Azure	cf-ap20	Australia (Sydney)	`azure`
					`azure_lite`
					`build-runtime-azure`

Enterprise account
Partner Test, Demo, and Development account
	br20	Microsoft Azure	cf-br20	Brazil (São Paulo)	`azure`
					`azure_lite`
					`build-runtime-azure`

Enterprise account Partner Test, Demo, and Development account	ca20	Microsoft Azure	cf-ca20	Canada (Toronto)	`azure` `azure_lite`
Enterprise account	ch20	Microsoft Azure	cf-ch20	Switzerland (Zurich) EU Access	`azure` The only available cluster region is Switzerland (Zurich) (`switzerlandnorth`). See Cluster Regions [page 56].
Enterprise account	cn20	Microsoft Azure	cf-cn20	China (North 3)	`azure` The only available cluster region is China (North 3) (`chinanorth3`). See Cluster Regions [page 56].

Enterprise account
Trial account (available only on request; for more information, see Getting Started with a Trial Kyma Instance [page 249])
	us10	Amazon Web Services	cf-us10	US East (VA)	`aws`
					`trial`
					`build-runtime-aws`

Global Account Type	Region	IaaS Provider	Technical Key	Region Name	Plan ID

Enterprise account
Trial account (available only on request; for more information, see Getting Started with a Trial Kyma Instance [page 249])
	eu10	Amazon Web Services	cf-eu10	Europe (Frankfurt)	`aws`
					`trial`
					`build-runtime-aws`

Enterprise account	eu11	Amazon Web Services	cf-eu11	Europe (Frankfurt) EU Access	`aws`
					`build-runtime-aws`
					The only available cluster region is Europe (Frankfurt) (`eu-central-1`). See Cluster Regions [page 56].

Enterprise account	eu13	Amazon Web Services	cf-eu13	Europe (Milan)	`aws`
Enterprise account	br10	Amazon Web Services	cf-br10	Brazil (São Paulo)	`aws` `build-runtime-aws`
Enterprise account	jp10	Amazon Web Services	cf-jp10	Japan (Tokyo)	`aws` `build-runtime-aws`
Enterprise account	ca10	Amazon Web Services	cf-ca10	Canada (Montreal)	`aws` `build-runtime-aws`
Enterprise account	ap12	Amazon Web Services	cf-ap12	South Korea (Seoul)	`aws`
Enterprise account	ap10	Amazon Web Services	cf-ap10	Australia (Sydney)	`aws` `build-runtime-aws`
Enterprise account	ap11	Amazon Web Services	cf-ap11	Singapore	`aws` `build-runtime-aws`

Global Account Type	Region	IaaS Provider	Technical Key	Region Name	Plan ID
Enterprise account	us11	Amazon Web Services	cf-us11	US West (Oregon)	`aws`
Enterprise account	us30	Google Cloud	cf-us30	US Central (IA)	`gcp` `build-runtime-gcp`
Enterprise account	eu30	Google Cloud	cf-eu30	Europe (Frankfurt)	`gcp` `build-runtime-gcp`
Enterprise account	eu31	Google Cloud	cf-eu31	Europe (Netherlands)	`gcp`
Enterprise account	in30	Google Cloud	cf-in30	India (Mumbai)	`gcp` `build-runtime-gcp`
Enterprise account	in31	Google Cloud	cf-in31	India (Delhi)	`gcp`
Enterprise account	jp30	Google Cloud	cf-jp30	Japan (Osaka)	`gcp`
Enterprise account	jp31	Google Cloud	cf-jp31	Japan (Tokyo)	`gcp`
Enterprise account	sa30	Google Cloud	cf-sa30	KSA (Dammam) GCP public sector	`gcp`
Enterprise account	sa31	Google Cloud	cf-sa31	KSA (Dammam)	`gcp`
Enterprise account	il30	Google Cloud	cf-il30	Israel (Tel Aviv)	`gcp`
Enterprise account	br30	Google Cloud	cf-br30	Brazil (São Paulo)	`gcp`

Global Account Type	Region	IaaS Provider	Technical Key	Region Name	Plan ID
Enterprise account	ap30	Google Cloud	cf-ap30	Australia (Sydney)	`gcp` `build-runtime-gcp`
Enterprise account	ap31	Google Cloud	cf-ap31	Singapore (Jurong West)	`gcp`
Enterprise account	eu01	SAP Cloud Infrastructure	cf-eu01	Europe (Frankfurt) EU Access	`sap-converged-cloud` available in the Germany (Frankfurt) (`eu-de-2`) region
Enterprise account	eu02	SAP Cloud Infrastructure	cf-eu02	Europe (Rot) EU Access	`sap-converged-cloud` available in the Germany (Rot) (`eu-de-1`) region
Enterprise account	ap01	SAP Cloud Infrastructure	cf-ap01	Australia (Sydney)	`sap-converged-cloud` available in the Australia (Sydney) (`ap-au-1`) region
Enterprise account	ae01	SAP Cloud Infrastructure	cf-ae01	UAE (Dubai)	`sap-converged-cloud` available in the UAE (Dubai) (`ap-ae-1`) region
Enterprise account	us01	SAP Cloud Infrastructure	cf-us01	US East (Sterling)	`sap-converged-cloud` available in the US East (Sterling) (`na-us-1`) region
Enterprise account	us02	SAP Cloud Infrastructure	cf-us02	US West (Colorado)	`sap-converged-cloud` available in the US West (Colorado) (`na-us-2`) region

Global Account Type	Region	IaaS Provider	Technical Key	Region Name	Plan ID
Enterprise account	jp01	SAP Cloud Infrastructure	cf-jp01	Japan (Tokyo)	`sap-converged-cloud` available in the Japan (Tokyo) (`ap-jp-1`) region
Enterprise account	cn40	Alibaba Cloud	cf-cn40	China (Shanghai)	`alicloud`

Cluster Regions

When you enable a Kyma environment for a given subaccount, you must select a service plan and region for your cluster. The available regions vary by IaaS provider and service plan. By default, any cluster region can be used with any subaccount region within a service plan. Exceptions are noted in the Subaccount Regions table. To see the supported regions for specific service plans, expand the table for your preferred IaaS provider.

Microsoft Azure

The following regions are available with these service plans:

- Standard: Microsoft Azure (`azure`)
- Kyma Test Demo and Development (Azure Lite) (`azure_lite`)
- Build Runtime: Microsoft Azure (`build-runtime-azure`)

Microsoft Azure Cluster Regions

Region	Region Name	Azure Subaccount Regions
`centralus`	US Central (IA)	All
`eastus`	US East (VA)	All
`eastus2`	US East 2 (VA)	All
`westus2`	US West (WA)	All
`northeurope`	North EU (Ireland)	All
`uksouth`	UK South (London)	All
`germanywestcentral`	Europe (Frankfurt)	All
`switzerlandnorth` (not available with the `azure_lite` plan)	Switzerland (Zurich)	ch20 only
`westeurope`	Europe (Netherlands)	All
`southeastasia`	Singapore	All
`australiaeast`	Australia (Sydney)	All

Region	Region Name	Azure Subaccount Regions
`japaneast`	Japan (Tokyo)	All
`brazilsouth`	Brazil (São Paulo)	All
`canadacentral`	Canada (Toronto)	All
`chinanorth3`	China (North 3)	cn20 only

Amazon Web Services

The following regions are available with these service plans:

- Standard: Amazon Web Services (`aws`)
- Build Runtime: Amazon Web Services (`build-runtime-aws`)

AWS Cluster Regions

Region	Region Name	AWS Subaccount Regions
`eu-central-1`	Europe (Frankfurt)	All
`eu-west-2`	Europe (London)	All
`eu-south-1`	Europe (Milan)	All
`ca-central-1`	Canada (Montreal)	All
`sa-east-1`	Brazil (São Paulo)	All
`us-east-1`	US East (VA)	All
`ap-northeast-1`	Japan (Tokyo)	All
`ap-northeast-2`	South Korea (Seoul)	All
`ap-south-1`	India (Mumbai)	All
`ap-southeast-1`	Singapore	All
`ap-southeast-2`	Australia (Sydney)	All
`us-west-2`	US West (Oregon)	All

The following regions are available with the trial (`trial`) plan.

AWS Trial Cluster Regions

Region	Region Name	AWS Subaccount Regions
`eu-central-1`	Europe (Frankfurt)	All
`us-east-1`	US East (VA)	All
`ap-southeast-1`	Singapore	All

Google Cloud

The following regions are available with these service plans:

- Standard: Google Cloud (`gcp`)
- Build Runtime: Google Cloud (`build-runtime-gcp`)

Google Cloud Cluster Regions

Region	Region Name	GCP Subaccount Regions
`europe-west3`	Europe (Frankfurt)	All
`europe-west4`	Europe (Netherlands)	All
`us-central1`	US Central (IA)	All
`asia-south1`	India (Mumbai)	All
`asia-south2`	India (Delhi)	All
`asia-northeast2`	Japan (Osaka)	All
`me-central2`	KSA (Dammam)	All
`me-west1`	Israel (Tel Aviv)	All
`australia-southeast1`	Australia (Sydney)	All
`southamerica-east1`	Brazil (São Paulo)	All
`asia-northeast1`	Japan (Tokyo)	All
`asia-southeast1`	Singapore (Jurong West)	All
`us-west1`	North America (Oregon)	All
`us-east4`	North America (Virginia)	All

SAP Cloud Infrastructure

The following regions are available with the SAP Cloud Infrastructure (`sap-converged-cloud`) plan.

SAP Cloud Infrastructure Cluster Regions

Region	Region Name	SAP Cloud Infrastructure Subaccount Regions
`eu-de-1`	Germany (Rot)	eu02
`eu-de-2`	Germany (Frankfurt)	eu01
`na-us-1`	US East (Sterling)	us01
`na-us-2`	US West (Colorado)	us02
`ap-au-1`	Australia (Sydney)	ap01
`ap-jp-1`	Japan (Tokyo)	jp01
`ap-ae-1`	UAE (Dubai)	ae01

Alibaba Cloud

The following region is available with the Alibaba Cloud (`alicloud`) plan.

Alibaba Cloud Cluster Regions

Region	Region Name	Alibaba Cloud Subaccount Regions
`cn-shanghai`	China (Shanghai)	cn40

Load Balancers

Depending on the IaaS provider, the following load balancers are provisioned by default:

Default Load Balancers

IaaS Provider	Default Load Balancer
Microsoft Azure	Standard
Amazon Web Services	Classic Network Load Balancer
Google Cloud	External passthrough Network Load Balancer

For more information about the load balancers and their features, see the official documentation of the respective IaaS provider.

Related Information

Creating Kyma Instances [page 3452]
Available Plans in the Kyma Environment [page 3459]
Provisioning and Updating Parameters in the Kyma Environment [page 3470]

2.2.4 Resilience, High Availability, and Disaster Recovery

SAP has a number of processes in place to support resilience in SAP BTP, and provides different offerings so that you can support the high availability of your applications.

How SAP Provides Resilience

SAP applies resilience principles when developing, updating, and deploying our SAP BTP applications and services.

SAP BTP provides resilience through the following:

Processes and Offerings	Description
Availability Zones	To achieve better fault-tolerance, we deploy our services across multiple availability zones (AZs), which improves the availability of a service if there are issues with the infrastructure of one AZ. Both Kyma and Cloud Foundry utilize multiple AZs provided by the cloud providers to ensure high availability of the platform. For more information on AZs, see: • Cloud Foundry runtime: Availability Zones in the Cloud Foundry Environment [page 66]. • Kyma runtime: Availability Zones in the Kyma Environment. In addition to what SAP BTP offers, you can follow the Best Practices for Resilient Applications [page 60] to ensure high availability of your applications.
Backups	SAP provides platform-level backups for some foundation services. However, for most of the services, managing backups is the customers' responsibility. For more information on backups, see SAP BTP Service Configuration Backups Managed by Customers and Data Backups Managed by SAP.
Disaster Recovery	The SAP BTP Disaster Recovery (DR) Plan is part of the overall SAP BTP Business Continuity Plan, which includes crisis management and process continuity activities that are triggered by a declared disaster. For more information, see Disaster Recovery as Part of the Business Continuity Plan [page 61].
In-Metro Disaster Recovery	In-Metro DR is a disaster recovery solution that utilizes synchronous data replication. It has been designed to protect our customers from the impact of local disasters that may affect a single availability zone (AZ). By deploying services across multiple AZs within a single region, In-Metro DR ensures that if a disaster occurs in one AZ, the issue is contained within that zone. Meanwhile, the other unaffected AZs continue to function normally, efficiently handling incoming requests. For more information, see Disaster Recovery as Part of the Business Continuity Plan [page 61].

Best Practices for Resilient Applications

In addition to the services offered by SAP BTP, you can follow our best practices for developing and deploying applications, which allow you to make your application running on SAP BTP stable and highly available.

- **Develop Resilient Applications**
  When developing your applications, apply the principles and patterns of resilient software design that fit your use case. For more information, see Developing Resilient Apps on SAP BTP. For information specifically about developing applications in the Kyma runtime, see Develop Resilient Applications in the Kyma Runtime. For situations where the load is highly available and where applications need to react by scaling, consider using Application Autoscaler. For more information, see What Is Application Autoscaler?
- **Working with Availability Zones**

To benefit from the high availability mechanisms in Cloud Foundry, set up your applications with multiple instances. For more information, see Developing Resilient Applications.

2.2.4.1 Disaster Recovery as Part of the Business Continuity Plan

The cloud platform disaster recovery (DR) plan is part of the overall cloud platform business continuity plan, which includes crisis management and process continuity activities that are triggered by a declared disaster.

Standard Disaster Recovery

SAP can restore productive tenants from backups as soon as practicable in case of a disaster resulting in the loss of the primary production data center.

As the magnitude of a disaster is unpredictable, a region might not be restored in a reasonable time. In addition, a new infrastructure might need to be set up at a different location, which might require the purchase and setup of new hardware. Therefore, we can't guarantee any fixed recovery timelines.

In-Metro Disaster Recovery

In-Metro DR is a disaster recovery solution that utilizes synchronous data replication. It has been designed to protect our customers from the impact of local disasters that may affect a single availability zone (AZ). By deploying services across multiple AZs within a single region, In-Metro DR ensures that if a disaster occurs in one AZ, the issue is contained within that zone. Meanwhile, the other unaffected AZs continue to function normally, efficiently handling incoming requests.

- In-Metro DR is offered to customers of SAP BTP cloud services as contractual obligations and includes:
  - Recovery Point Objective (RPO): No more than 5 minutes of data loss.
  - Recovery Time Objective (RTO): Full-service restoration within 2 hours.
- In-Metro DR solution leverages an active/active HA set up across 2 or more availability zones with a failover for all customers from primary to secondary site. That is, availability zones within a single region
- Fully functioning availability zones are paired with every primary availability zone, application stack is pre-built with same capacity and compute as the primary availability zone.
- A written, customer facing DR plan exist and is updated at least once every 12 months.
- A DR test is performed at least once every 12 months, and the DR documents are revised as needed based on the results.
- The In-Metro DR solution passes annual audits and receives certifications (for example, SOC2).
- Scope:
  - For the scope of SAP BTP services, see SAP BTP Disaster Recovery Overview.
  - Includes SAP BTP regions on AWS and Azure

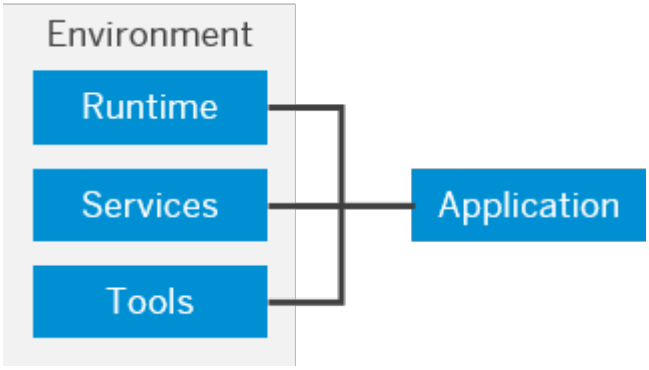
For more information, see Announcing the New "In-Metro" Disaster Recovery Solution for SAP BTP.

2.3 Environments

Environments constitute the actual platform-as-a-service offering of SAP BTP that allows for the development and administration of business applications. Environments are anchored in SAP BTP on subaccount level.

Each environment comes equipped with specific tools, technologies, and runtimes that you need to build applications. So a multi-environment subaccount is your single address to host a variety of applications and offer diverse development options. One advantage of using different environments in one subaccount is that you only need to manage users, authorizations, and entitlements once per subaccount, and thus grant more flexibility to your developers.

> **Note**
>
> The multi-environment subaccount functionality is not applicable for the Neo environment.



Environment Instances

To actually use an environment in a subaccount, you must **enable** it, which creates an instance of that environment. There are several ways to create **environment instances**:

- In the SAP BTP cockpit, on the subaccount overview, choose *Enable*.
- In the SAP BTP cockpit, under *Service Marketplace*. Here, you get more information, such as the available plans and links to further information.
- Using the BTP CLI command `btp create accounts/environment-instance`

Cloud Foundry Environment [page 63]
: The Cloud Foundry environment allows you to create polyglot cloud applications in Cloud Foundry. It contains the SAP BTP, Cloud Foundry runtime service, which is based on the open-source application platform managed by the Cloud Foundry Foundation.

ABAP Environment [page 81]
: The ABAP environment allows you to create extensions for ABAP-based products, such as SAP S/4HANA Cloud, and develop new cloud applications. You can transform existing ABAP-based custom code or extensions to the cloud.

Kyma Environment [page 86]
: SAP BTP, Kyma runtime provides a fully managed cloud-native Kubernetes application runtime based on the open-source project "Kyma". Based on modular building blocks, Kyma runtime includes all

the necessary capabilities to simplify the development and to run enterprise-grade cloud-native applications.

Neo Environment [page 108]
: The Neo environment lets you develop HTML5, Java, and SAP HANA extended application services (SAP HANA XS) applications. You can also use the UI Development Toolkit for HTML5 (SAPUI5) to develop rich user interfaces for modern web-based business applications.

Related Information

Account Administration [page 2457]

2.3.1 Cloud Foundry Environment

The Cloud Foundry environment allows you to create polyglot cloud applications in Cloud Foundry. It contains the SAP BTP, Cloud Foundry runtime service, which is based on the open-source application platform managed by the Cloud Foundry Foundation.

The Cloud Foundry environment enables you to develop new business applications and business services, supporting multiple runtimes, programming languages, libraries, and services. You can leverage a multitude of buildpacks, including community innovations and self-developed buildpacks. It also integrates with SAP HANA extended application services, advanced model.

For more information about Cloud Foundry, see the official Cloud Foundry documentation at https://docs.cloudfoundry.org/ .

Supported and Unsupported Cloud Foundry Features [page 64]
: Find out which Cloud Foundry features the Cloud Foundry environment on SAP BTP supports and doesn't support.

SAP BTP-Specific Configurations [page 65]
: Learn about technical configurations for Cloud Foundry that are specific to SAP BTP.

Availability Zones in the Cloud Foundry Environment [page 66]
: The Cloud Foundry environment follows the recommendations of our partner IaaS providers by leveraging the availability zones (AZ) concept.

Additional Information About Cloud Foundry [page 68]
: Links to additional information about Cloud Foundry that is useful to know but not necessarily directly connected to the SAP BTP, Cloud Foundry environment.

Service Plans and Metering for Cloud Foundry Runtime [page 69]
: This page explains the relationship between the service plans in the SAP Discovery Center and those in the SAP BTP cockpit, and provides information to help you understand how the service is billed.

Related Information

Getting Started in the Cloud Foundry Environment [page 165]
Development in the Cloud Foundry Environment [page 256]
Administration and Operations in the Cloud Foundry Environment [page 2779]

2.3.1.1 Supported and Unsupported Cloud Foundry Features

Find out which Cloud Foundry features the Cloud Foundry environment on SAP BTP supports and doesn't support.

Supported Features	Unsupported Features
• Diego runtime. See https://docs.cloudfoundry.org/concepts/diego/diego-architecture.html . • SSH. See https://docs.cloudfoundry.org/devguide/deploy-apps/app-ssh-overview.html . • Custom Domains. See https://docs.cloudfoundry.org/devguide/deploy-apps/routes-domains.html#domains . • Docker. See https://docs.cloudfoundry.org/adminguide/docker.html . • Running Tasks. See https://docs.cloudfoundry.org/devguide/using-tasks.html . • Request Tracing • Zipkin Tracing. See https://docs.cloudfoundry.org/adminguide/zipkin_tracing.html . • Websockets. See https://docs.cloudfoundry.org/adminguide/supporting-websockets.html . • Space-Scoped Service Brokers. See https://docs.cloudfoundry.org/services/managing-service-brokers.html . • Route Services (only user-provided and fully-brokered services). See https://docs.cloudfoundry.org/services/route-services.html . • Sharing Service Instances (not all services support instance sharing). See https://docs.cloudfoundry.org/devguide/services/sharing-instances.html . • HTTP/2. See https://docs.cloudfoundry.org/adminguide/supporting-http2.html#application . • Streaming Logs to Log Management Services. See https://docs.cloudfoundry.org/devguide/services/log-management.html . • Configuring per-route options. See https://docs.cloudfoundry.org/devguide/custom-per-route-options.html	• Container-to-Container Networking. See https://docs.cloudfoundry.org/concepts/understand-cf-networking.html . • Isolation Segments. See https://docs.cloudfoundry.org/adminguide/isolation-segments.html . • TCP Routing. See https://docs.cloudfoundry.org/adminguide/enabling-tcp-routing.html . • Secure Service Credential Delivery (with Credhub). See https://docs.cloudfoundry.org/credhub/index.html or https://github.com/cloudfoundry/credhub/blob/main/docs/secure-service-credentials.md

2.3.1.2 SAP BTP-Specific Configurations

Learn about technical configurations for Cloud Foundry that are specific to SAP BTP.

The following technical configurations are specific to SAP BTP and differ from the default configuration:

- SAP BTP supports the Cloud Foundry command line interface (CF CLI) version 8 or newer. Older versions of the CF CLI are not supported.
- By default, a newly pushed (or started) Cloud Foundry application needs to respond to a health check within the first 60 seconds, otherwise the application is considered to have failed. For more information, see https://docs.cloudfoundry.org/devguide/deploy-apps/healthchecks.html#health_check_timeout . On SAP BTP, however, you can override this timeout to up to 10 minutes. For instructions, see https://docs.cloudfoundry.org/devguide/deploy-apps/large-app-deploy.html .
- On SAP BTP, application SSH access is disabled by default. For more information on SSH, see https://docs.cloudfoundry.org/devguide/deploy-apps/app-ssh-overview.html .
- SAP BTP supports the Cloud Foundry API version 3. The Cloud Foundry API v2 is not available. For more information, see https://v3-apidocs.cloudfoundry.org/ .
- On SAP BTP, the Cloud Foundry API is protected by a rate limit against misuse. The limit is in the range of a few 10k requests per hour per user on average.
- In addition to the general rate limit on the Cloud Foundry API, requests for certain API endpoints related to services face a separate limit on concurrent requests. The Cloud Foundry API responds with HTTP status code `429` if a rate limit is reached and provides a Retry-After Header suggesting when the client can attempt a retry. For more information, see https://docs.cloudfoundry.org/running/rate-limit-cloud-controller-api.html#Rate%20Limit%20Responses:%20Service%20Brokers .
- In the SAP BTP, Cloud Foundry environment, the total HTTP Request Header and HTTP Response Header size is limited to 64 KB to protect against misuse.
- In the SAP BTP, Cloud Foundry environment, for both HTTP Request Headers and HTTP Response Headers the total amount of headers is limited to 101. Components that are handling the requests will add system-relevant headers such as `X-Vcap-Request-Id` or `X-Ssl-*` when mTLS is used. This addition reduces the amount of headers that can be set by the client (for requests) or the application (for responses).
- In the SAP BTP, Cloud Foundry environment, the limit of concurrent HTTP connections between client and application is 3000 per application container.
- In the SAP BTP, Cloud Foundry environment, the HTTP `keep-alive` timeout towards the client is set to 60s to protect against misuse. 60s is the maximum time span allowed to wait for a new HTTP request to appear if `keep-alive` is enabled.
- In the SAP BTP, Cloud Foundry environment, an internal HTTP `keep-alive` is set to 90s. A higher value must be set on application-side to avoid intermittent disruptions. For more information, see step 4 of 3406978.
- In the Cloud Foundry environment, there's a logging rate limit to guard against malicious applications. By default, the limit is 4000 logs per second per application instance, but in exceptional high load scenarios it may be lowered by the platform. If this limit is exceeded, additional logs from the application instance are dropped and a warning message is injected into the application instance's log stream every second. This message also contains the exact current log rate limit.
- Applications requiring sent envelopes to be delivered to external Log Management Services should use the Cloud Foundry syslog drain capability. See https://docs.cloudfoundry.org/devguide/services/log-management.html .
- In the SAP BTP, Cloud Foundry environment, the time between signaling a container to shut down gracefully and forcefully stopping it is set to 60 seconds. The default in Cloud Foundry is 10 seconds, see

https://docs.cloudfoundry.org/devguide/deploy-apps/app-lifecycle.html#shutdown . This time interval will not be taken into account if there are no explicit kernel signal handlers implemented in the application.

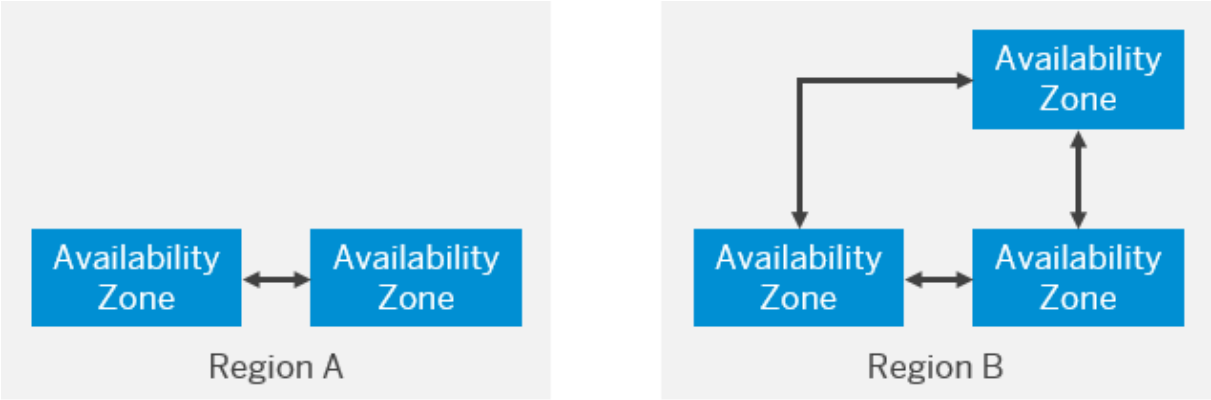
- In the SAP BTP, Cloud Foundry environment, applications get a guaranteed CPU share of ¼ core per GB instance memory. As the maximum instance memory per application is 16 GB, this allows for vertical scaling up to 4 CPUs.
  If applications running on the same virtual machine don't use their guaranteed CPU, other applications might get more CPU. This isn't guaranteed and might be subject to change in the future. If you encounter performance problems, scale up your application or increase the application start timeout.
  The number of running threads per application instance is limited to 10,420. Reaching this limit can cause performance issues.
- When pushing or scaling your application, you can define a `disk_quota` that can be up to 10 GB. For more information, see https://docs.cloudfoundry.org/devguide/deploy-apps/manifest-attributes.html#disk-quota .
- When deploying applications on SAP BTP, the maximum application package size is 1.5 GB.
  If your application is larger than that, the deployment fails. For more information, see https://docs.cloudfoundry.org/devguide/deploy-apps/large-app-deploy.html .
- In the SAP BTP, Cloud Foundry environment, the hard limit for open file descriptors is 32,768 (32K) per container.
- In global accounts that support the consumption-based commercial model you might see a quota limit for certain services. This is a technical limit only, not a business limit. If you need to increase this limit, report an incident to SAP support for component BC-NEO-CIS.
- In the SAP BTP, Cloud Foundry environment, the SAP HANA database supports a limited number of simultaneous connections per database. For more information, see Connecting to the SAP HANA Database in SAP HANA Cloud.
- In the SAP BTP, Cloud Foundry environment, each application can be mapped to approximately 1000 routes (128 KB). The total length of the routing information must not exceed this limit.
- Cloud Foundry Audit Events have a retention period of 14 days. For more information on Audit Events, see https://docs.cloudfoundry.org/running/managing-cf/audit-events.html .
- To ensure the stable and fair operation of our platform, we enforce rate limiting on container network bandwidth. Specific thresholds are not disclosed to prevent abuse. These limits are carefully set to fully support all expected customer workload.

2.3.1.3 Availability Zones in the Cloud Foundry Environment

The Cloud Foundry environment follows the recommendations of our partner IaaS providers by leveraging the availability zones (AZ) concept.

About Availability Zones

Availability zones (AZ) are single failure domains within a single geographical region and are separate physical locations with independent power, network, and cooling. Multiple AZs exist in one region and are connected with each other through a low-latency network.



2-AZ and 3-AZ Deployments

To achieve better fault-tolerance, our partners recommend deploying services across multiple AZs, which improves the availability of a service if there are issues with the region infrastructure of one AZ.

High Availability at Platform and Application Level

The SAP BTP Cloud Foundry environment follows these recommendations to support high availability at the platform and application level:

- **High availability of the platform components:**
  - The building blocks of Cloud Foundry and the virtual machines on which the Cloud Foundry application instances are scheduled run in a high availability setup. These instances are distributed across different AZs.
  - The technology that manages the deployment of the Cloud Foundry environment monitors the health of the platform. If there are infrastructure failures, it re-creates the faulty components.
- **High availability on the application level:**
  - We recommend running multiple application instances to increase availability. For more information, see Run Multiple Instances to Increase Availability . On SAP BTP, there are three ways to increase application instances:
    - Scaling your application using the application manifest. The `manifest.yml` allows you to make and save configurations for your application. To scale, you can configure the instance count in the manifest and push the application again with the new configuration. See App Manifest Attribute Reference . To avoid downtimes when updating your application configuration, you can also consider using rolling application deployments. See Rolling App Deployments .
    - Scaling your application using the `cf scale` command in the Cloud Foundry command line interface (CF CLI). See Scaling an App Using cf scale .
    - Scaling your application using the SAP BTP cockpit. See Add or Remove Application Instances.
  - The Cloud Foundry container scheduler takes care of distributing the different instances of one application on virtual machines in different AZs. For more information, see How Diego Balances App Processes .
  - Cloud Foundry is constantly monitoring the health state of application instances and restarts instances that are considered unhealthy. See Using App Health Checks .

- When the number of desired instances doesn't match the number of actually running instances, Cloud Foundry reschedules the missing instances, for example, when the virtual machines that an application instance was initially scheduled on become unresponsive.

For more information on high availability configuration, see High Availability in Cloud Foundry .
For more information on application stability and resilience, see Developing Resilient Applications.

2.3.1.4 Additional Information About Cloud Foundry

Links to additional information about Cloud Foundry that is useful to know but not necessarily directly connected to the SAP BTP, Cloud Foundry environment.

Content	Location
BOSH	http://bosh.cloudfoundry.org
BOSH documentation	http://bosh.io/docs
Buildpacks	http://docs.cloudfoundry.org/buildpacks
Components of Cloud Foundry	http://docs.cloudfoundry.org/concepts/architecture/
Cloud Foundry Concepts	http://docs.cloudfoundry.org/concepts/
Deployment of Cloud Foundry	http://docs.cloudfoundry.org/deploying
Developer Guide for Cloud Foundry	http://docs.cloudfoundry.org/devguide
Diego Application Process Balancing	https://docs.cloudfoundry.org/concepts/diego/diego-auction.html
Glossary for Cloud Foundry	http://docs.cloudfoundry.org/concepts/glossary.html
Overview of Cloud Foundry	http://docs.cloudfoundry.org/concepts/overview.html
Sample applications for Cloud Foundry	https://github.com/cloudfoundry-samples
Security settings for Cloud Foundry	http://docs.cloudfoundry.org/concepts/security.html
Cloud Foundry Services	http://docs.cloudfoundry.org/services http://docs.cloudfoundry.org/devguide/services/user-provided.html
Considerations for designing and running an application in the cloud	http://docs.cloudfoundry.org/devguide/deploy-apps/prepare-to-deploy.html
Installing the Cloud Foundry command line interface	http://docs.cloudfoundry.org/devguide/installcf/install-go-cli.html

Content	Location
Blog about Cloud Foundry	http://blog.cloudfoundry.org/

2.3.1.5 Service Plans and Metering for Cloud Foundry Runtime

This page explains the relationship between the service plans in the SAP Discovery Center and those in the SAP BTP cockpit, and provides information to help you understand how the service is billed.

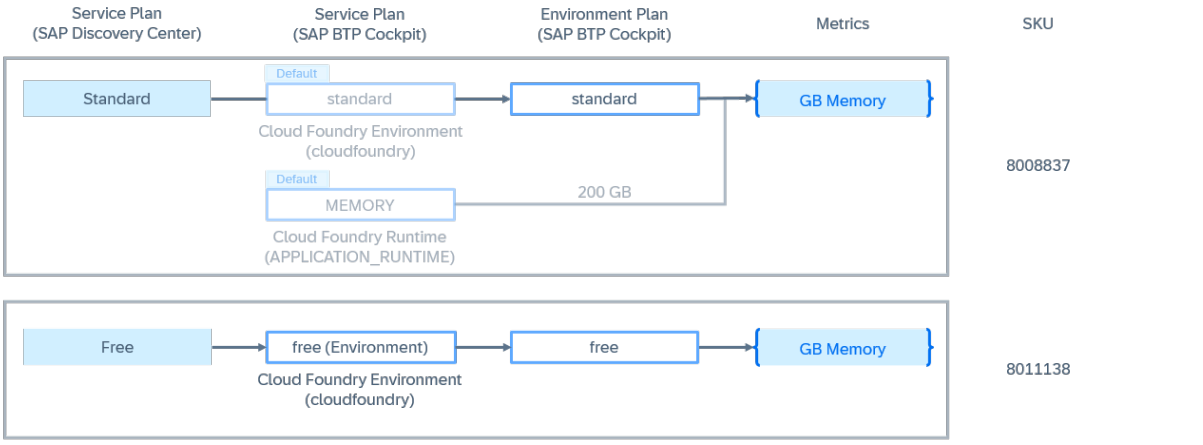
Service

Overview

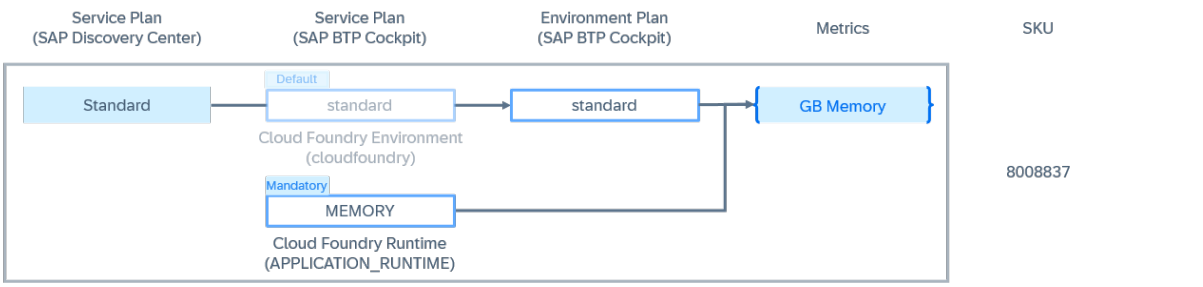
The diagram below shows how the service plans listed in the SAP Discovery Center correspond to the plans you choose in the SAP BTP cockpit, depending on the commercial model of your enterprise global account. For more information about the commercial models offered by SAP, see Commercial Models [page 113].

This image is interactive. Hover over rectangles in the diagram to see the description. Choose the highlighted areas for more information.

Consumption-Based Commercial Model



Subscription-Based Commercial Model

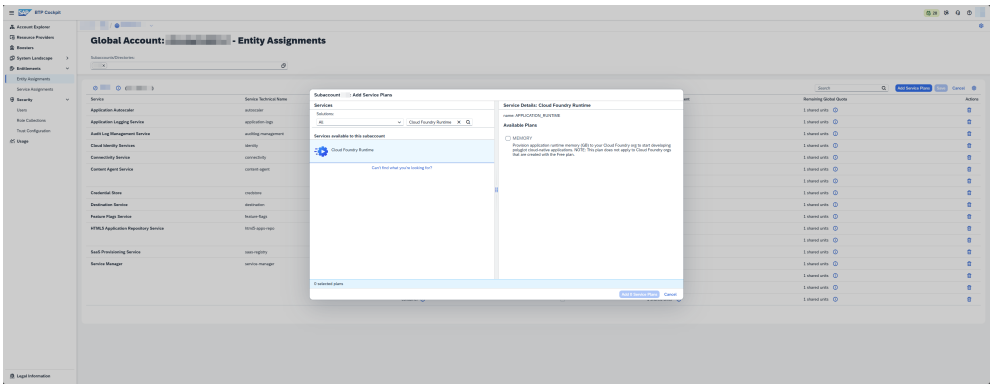


- https://discovery-center.cloud.sap/serviceCatalog/cloud-foundry-runtime?region=all&tab=service_plan&commercialModel=btpea [https://discovery-center.cloud.sap/serviceCatalog/cloud-foundry-runtime?region=all&tab=service_plan&commercialModel=btpea]
- Restricting the Usage of Standard Plan for Cloud Foundry Runtime
- Enabling Standard Plan for Cloud Foundry Runtime (Consumption-Based)
- Metrics [page 77]
- (optional) Adjusting Runtime Memory Quota for Cloud Foundry Runtime (Consumption-Based)
- https://discovery-center.cloud.sap/serviceCatalog/cloud-foundry-runtime?region=all&tab=service_plan&commercialModel=btpea [https://discovery-center.cloud.sap/serviceCatalog/cloud-foundry-runtime?region=all&tab=service_plan&commercialModel=btpea]
- Assigning Free Service Plan with Runtime Memory Quota for Cloud Foundry Runtime
- Enabling Free Plan for Cloud Foundry Runtime
- Metrics [page 77]
- https://discovery-center.cloud.sap/serviceCatalog/cloud-foundry-runtime?region=all&tab=service_plan&commercialModel=subscription [https://discovery-center.cloud.sap/serviceCatalog/cloud-foundry-runtime?region=all&tab=service_plan&commercialModel=subscription]
- Restricting the Usage of Standard Plan for Cloud Foundry Runtime
- Enabling Standard Plan for Cloud Foundry Runtime (Subscription-Based)
- Metrics [page 77]
- Assigning Runtime Memory Quota for Cloud Foundry Runtime (Subscription-Based)

Service Plan (SAP BTP Cockpit)

The plan that the global account administrator can assign to a subaccount or a directory on the *Entitlements* > *Entity Assignments* page by choosing *Edit* > *Add Service Plans*.

> **→ Tip**
>
> Choose *Zoom In* for the full-sized version of the screenshot.



Screenshot: Assigning a service plan to a subaccount in SAP BTP cockpit

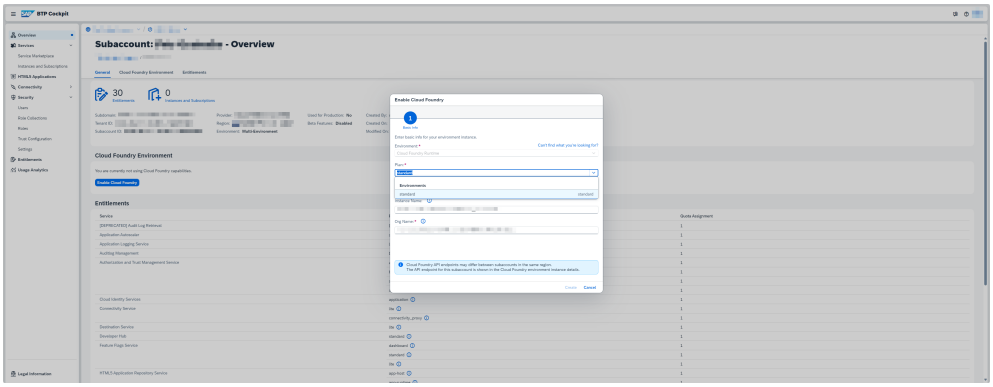
For general information about the procedure, see Configure Entitlements and Quotas for Subaccounts [page 2529] or Configure Entitlements and Quotas for Directories [page 2524].

> **ⓘ Note**
>
> The greyed out parts of the diagram indicate service plans that are automatically assigned to all subaccounts by default.

Environment Plan (SAP BTP Cockpit)

The plan that the subaccount administrator can select on the *Overview* page of a subaccount when choosing *Enable Cloud Foundry*.

> **→ Tip**
>
> Choose *Zoom In* for the full-sized version of the screenshot.



Screenshot: Selecting an environment plan for the subaccount in SAP BTP cockpit

For general information about the procedure, see Create Orgs [page 2793].

> **Note**
>
> An environment plan is a service plan associated with an environment. For more information about services and service plans, see Entitlements and Quotas [page 127].

Diagram captions: Service plans and environment plans in SAP BTP cockpit [Show/Hide]

Service Plans

The tables below provide details about the plans for SAP BTP, Cloud Foundry runtime. They can give you more context for understanding the diagram in the **Overview** section.

Table 1: Plans for Consumption-Based Commercial Model [Show/Hide]

Service Plan (SAP Discovery Center)	Service (SAP BTP cockpit: *Entity Assignments*)	Service Plan (SAP BTP cockpit: *Entity Assignments*)	Environment Plan (SAP BTP cockpit: *Enable Cloud Foundry*)	Explanation
Standard	Cloud Foundry Environment (`cloudfoundry`)	standard	standard	The plans `standard` and `MEMORY` are used together to allow you to deploy and run applications in the Cloud Foundry environment. • `standard`: this plan allows you to create a Cloud Foundry org in a subaccount. **By itself**, the action of creating a Cloud Foundry org is free. However, when you run applications in it, you pay for the runtime memory that they use. • `MEMORY`: this plan is used to assign runtime memory quota to a subaccount. The runtime memory can then be consumed by applications that run in the Cloud Foundry org created with the plan `standard`. The usage of runtime memory by applications is **bill-**

Service Plan (SAP Discovery Center)	Service (SAP BTP cockpit: *Entity Assignments*)	Service Plan (SAP BTP cockpit: *Entity Assignments*)	Environment Plan (SAP BTP cockpit: *Enable Cloud Foundry*)	Explanation
	Cloud Foundry Runtime (`APPLICATION_RUNTIME`)	MEMORY		**able** according to the commercial model of your enterprise global account. In the consumption-based commercial model, a default runtime memory quota of 200 GB is automatically assigned to all subaccounts, and the billing is based on the actual runtime memory usage. For more information, see Service Specifics [page 79]. For information on how to set up the plan, see Standard Plan for Cloud Foundry Runtime (Consumption-Based).

Service Plan (SAP Discovery Center)	Service (SAP BTP cockpit: *Entity Assignments*)	Service Plan (SAP BTP cockpit: *Entity Assignments*)	Environment Plan (SAP BTP cockpit: *Enable Cloud Foundry*)	Explanation
Free	Cloud Foundry Environment (`cloudfoundry`)	free (Environment)	free	This is a **free tier** plan that allows you to try out and evaluate the service. For more information, see Using Free Service Plans [page 118]. When you create a Cloud Foundry org with the free plan, you also get a limited free quota of runtime memory for it. The amount of such orgs that can be created in an enterprise global account is limited. For information on how to set up the plan, see Free Plan for Cloud Foundry Runtime. **Note** Only community support is available for free tier service plans and these are not subject to SLAs. Use of free tier service plans is subject to additional terms and conditions as provided in the Business Technology Platform Supplemental Terms and Conditions.

Service Plan (SAP Discovery Center)	Service (SAP BTP cockpit: *Entity Assignments*)	Service Plan (SAP BTP cockpit: *Entity Assignments*)	Environment Plan (SAP BTP cockpit: *Enable Cloud Foundry*)	Explanation
-	Cloud Foundry Environment (`cloudfoundry`)	build-runtime	build-runtime	This plan is only for using the SAP BTP, Cloud Foundry runtime as part of SAP Build. For more information, see What Is SAP Build and Service Plans and Metering. **Note** The plan is not depicted on the diagram in the **Overview** section.
See SAP Build Code	Cloud Foundry Environment (`cloudfoundry`)	build-code	build-code	This plan is only for using the SAP BTP, Cloud Foundry runtime as part of SAP Build Code. For more information, see What Is SAP Build Code. **Note** The plan is not depicted on the diagram in the **Overview** section.

Table 2: Plans for Subscription-Based Commercial Model [Show/Hide]

Service Plan (SAP Discovery Center)	Service (SAP BTP Cockpit: *Entity Assignments*)	Service Plan (SAP BTP Cockpit: *Entity Assignments*)	Environment Plan (SAP BTP Cockpit: *Enable Cloud Foundry*)	Explanation
Standard	Cloud Foundry Environment (`cloudfoundry`)	standard	standard	The plans `standard` and `MEMORY` are used together to allow you to deploy and run applications in the Cloud Foundry environment. • `standard`: this plan allows you to create a Cloud Foundry org in a

Service Plan (SAP Discovery Center)	Service (SAP BTP Cockpit: *Entity Assignments*)	Service Plan (SAP BTP Cockpit: *Entity Assignments*)	Environment Plan (SAP BTP Cockpit: *Enable Cloud Foundry*)	Explanation
	Cloud Foundry Runtime (`APPLICATION_RUNTIME`)	MEMORY		subaccount. **By itself**, the action of creating a Cloud Foundry org is free. However, when you run applications in it, you pay for the runtime memory that they use. • `MEMORY`: this plan is used to assign runtime memory quota to a subaccount. The runtime memory can then be consumed by applications that run in the Cloud Foundry org created with the plan `standard`. The usage of runtime memory by applications is **billable** according to the commercial model of your enterprise global account. In the subscription-based commercial model, a predefined amount of runtime memory is paid for in advance and then gets used. For more information, see Service Specifics [page 79]. For information on how to set up the plan, see Standard Plan for Cloud Foundry Runtime (Subscription-Based).

Service Plan (SAP Discovery Center)	Service (SAP BTP Cockpit: *Entity Assignments*)	Service Plan (SAP BTP Cockpit: *Entity Assignments*)	Environment Plan (SAP BTP Cockpit: *Enable Cloud Foundry*)	Explanation
-	Cloud Foundry Environment (`cloudfoundry`)	build-runtime	build-runtime	This plan is only for using the SAP BTP, Cloud Foundry runtime as part of SAP Build. For more information, see What Is SAP Build and Service Plans and Metering. **Note** The plan is not depicted on the diagram in the **Overview** section.
See SAP Build Code	Cloud Foundry Environment (`cloudfoundry`)	build-code	build-code	This plan is only for using the SAP BTP, Cloud Foundry runtime as part of SAP Build Code. For more information, see What Is SAP Build Code. **Note** The plan is not depicted on the diagram in the **Overview** section.

Metrics

The table below provides details about the metrics for SAP BTP, Cloud Foundry runtime. It includes the names of services, with which the metrics are associated on the pages *Usage* or *Costs and Usage* (accessed from the global account level) and *Usage Analytics* (accessed from the subaccount level) in the SAP BTP cockpit.

Metric	Service (SAP BTP cockpit)	Metric (SAP BTP cockpit)	Definition	Additional Information
GB Memory	Cloud Foundry Runtime	GB Memory	Temporary memory bank where computers store data that needs to be retrieved and processed quickly. The memory represents the size of the data that can be processed and CPU represents the speed at which the data can be retrieved.	For billing purposes, the GB Memory metric for the Cloud Foundry Runtime service is calculated as the total hourly usage of Cloud Foundry runtime memory across all spaces in the global account over a calendar month, divided by 730 hours and rounded up to the next full GB. For an example of such calculation, see Consumption Monitoring. **Note** In the Cloud Foundry environment, applications get a guaranteed CPU share of ¼ core per GB of runtime memory reserved for an application instance. For more information, see SAP BTP-Specific Configurations [page 65].
Capacity Units	SAP Build	GB Memory		In SAP Build, the usage of SAP BTP, Cloud Foundry runtime is measured in capacity units. The GB Memory metric for the SAP Build service in the SAP BTP cockpit is intended only for monitoring the approximate usage of SAP BTP, Cloud Foundry runtime in GB when it's consumed as part of SAP Build. For more information, see What Is SAP Build. **Note** The metric is not depicted on the diagram in Service [page 69].

Metric	Service (SAP BTP cockpit)	Metric (SAP BTP cockpit)	Definition	Additional Information
	SAP Build Code	CF Runtime		In SAP Build Code, the usage of SAP BTP, Cloud Foundry runtime is measured in capacity units. The CF Runtime metric for the SAP Build Code service in the SAP BTP cockpit is intended only for monitoring the approximate usage of SAP BTP, Cloud Foundry runtime in GB when it's consumed as part of SAP Build Code. For more information, see What Is SAP Build Code and SAP Build Code Capacity Unit Calculator. **Note** The metric is not depicted on the diagram in Service [page 69].

Service Specifics

In the context of SAP BTP, Cloud Foundry runtime, the terms **consumption** (or **consume**) and **usage** (or **use**) refer to the runtime memory quota **reserved** by the platform for each application instance. This quota serves as the basis for calculating the billable consumption, as explained in Consumption Monitoring.

Caution

From a billing standpoint, it doesn't matter how much of the reserved runtime memory quota is utilized when the application is running. Billable consumption is calculated based on the full amount of runtime memory reserved, regardless of how much runtime memory actually gets utilized.

Note

If an application is stopped, it doesn't reserve any runtime memory and, therefore, doesn't contribute to runtime memory consumption.

There are two application settings that define how much runtime memory an application uses at any given time:

- Runtime memory quota reserved for each application instance (default: 1024 MB)
- Number of running application instances (default: 1)

Note

The total amount of runtime memory used by the application is displayed on the *Application Overview* page under *Current Application Memory Usage*.

You can specify custom values for these settings when deploying an application in the Cloud Foundry environment. You can also change them for an application that has already been deployed without having to redeploy it.

How to set runtime memory quota and number of instances when deploying applications

- **Option 1:** Create a manifest YAML file and specify the following application attributes:
  - `memory` : runtime memory quota per application instance
  - `instances` : number of application instances

  For more information about the manifest formatting, see https://docs.cloudfoundry.org/devguide/deploy-apps/manifest-attributes.html .

  You can then deploy your application with the manifest in one of two ways:
  - Using the SAP BTP cockpit: Deploy an Application [page 2862]
  - Using the Cloud Foundry CLI: https://docs.cloudfoundry.org/devguide/deploy-apps/manifest.html
- **Option 2:** In the SAP BTP cockpit, choose *Deploy Application*, select *Custom Settings* and specify the following:
  - *Memory per Instance (MB)*: runtime memory quota per application instance
  - *Number of Instances*: number of application instances

  For more information about the procedure, see Deploy an Application [page 2862].
- **Option 3:** In the Cloud Foundry CLI, use the command `cf push` with the following flags:
  - `-m`: runtime memory quota per application instance
  - `-i`: number of application instances

  For more information about the procedure, see https://docs.cloudfoundry.org/devguide/deploy-apps/deploy-app.html#custom-push .

How to change runtime memory quota and number of instances for already deployed applications

- **Option 1:** In the SAP BTP cockpit, on the *Application Overview* page you can:
  - Change the runtime memory quota per application instance under *Instances Configuration* > *Change Configuration* > *Memory per Instance (MB)*
  - Change the number of application instances under *Instances Configuration* > *Change Configuration* > *App Instances* (or as described in Manage Application Instances [page 2867])
- **Option 2:** In the Cloud Foundry CLI, you can use the command `cf scale` with the following flags:
  - `-m`: change the runtime memory quota per application instance
  - `-i`: change the number of application instances

  For more information about the procedure, see https://docs.cloudfoundry.org/devguide/deploy-apps/cf-scale.html .

You can use the **Application Autoscaler** to automatically increase or decrease the number of application instances based on the policies you have defined. For more information, see What Is Application Autoscaler.

> → Tip
>
> For tips on how to optimize the consumption of runtime memory, see the blog post Optimise your SAP BTP, Cloud Foundry runtime costs. Note that while the general principles outlined in the blog post still apply, some of the UI texts and parameter names may have changed.

Supplemental Terms and Conditions

For more information, see the section **SAP BTP, Cloud Foundry Runtime** in the SAP Business Technology Platform Service Description Guide.

Glossary

Commercial Information Glossary

2.3.2 ABAP Environment

The ABAP environment allows you to create extensions for ABAP-based products, such as SAP S/4HANA Cloud, and develop new cloud applications. You can transform existing ABAP-based custom code or extensions to the cloud.

The ABAP environment is based on the latest ABAP platform cloud release that is also used for SAP S/4HANA Cloud. It leverages the innovations provided by SAP HANA. The software stack contains standard technology components that are familiar from the standalone Application Server ABAP. The ABAP environment supports the ABAP RESTful Application Programming Model including SAP Fiori and Core Data Services (CDS). SAP services and APIs are offered according to the new approach of released objects. The ABAP environment can be integrated with other SAP BTP services, such as SAP Destination service, SAP Build Work Zone, standard edition, SAP Build Process Automation, and SAP Interactive Forms by Adobe.

Each ABAP system in the ABAP environment utilizes a dedicated SAP HANA database, which is provided by the SAP HANA Cloud service and managed by the ABAP environment. The database is linked 1:1 to the ABAP system.

For information about regional availability, see Regions and API Endpoints for the ABAP Environment [page 43].

Related Information

Getting Started in the ABAP Environment [page 183]
Development in the ABAP Environment [page 848]
Administration and Operations in the ABAP Environment [page 2914]
Using Free Service Plans
Discovery Center
SAP Community
SAP Road Map Explorer
Learning Journey
Tutorials

2.3.2.1 ABAP Development Tools for Eclipse

ABAP development tools for Eclipse is SAP's state-of-the-art integrated development environment (IDE) for ABAP development on the open Eclipse platform.

Features

ABAP development tools for Eclipse enables you to perform ABAP-based development tasks, when you want to build, to extend, and to run ABAP applications based on SAP products, such as SAP S/4HANA and SAP S/4HANA Cloud Private Edition for classic ABAP development as well as SAP Business AI Platform, ABAP environment and SAP S/4HANA Cloud Public Edition for Cloud development.

It supports ABAP developers with the following possibilities:

- Highly flexible, customizable Eclipse UI for ABAP development tools
- High-performance, failover-safe online development in multiple ABAP systems simultaneously
- Display and edit functionality for multiple ABAP objects in parallel
- Advanced and efficient source code editing including refactoring support
- Optimum support of task-oriented and test-driven development
- Robust and reliable quality assurance and supportability tools.
- Built-in extensibility of the IDE using the established Eclipse plug-in technology

In particular, ABAP development tools for Eclipse is a modern development toolset where ABAP developers can use, for example the following features:

- Syntax check
- Code completion
- Syntax highlighting
- Pretty printing
- Navigation
- Search
- Quick fixes
- and many more

Help Content

The ABAP Development Tools for Eclipse Guide describes the following functionalities and usage of ADT ...

- SAP Joule for developers, ABAP AI capabilities supports you in the end-to-end development using generative AI and can be integrated into ABAP development tools for Eclipse. It's currently only available for customers using SAP Business AI Platform, ABAP environment or SAP S/4HANA Cloud Public Edition. To leverage Joule for developers, ABAP AI capabilities, you need to purchase an additional license. See Getting Started.

- within the ABAP core development scenario. In particular, it focuses on use cases for creating, editing, testing, debugging, and profiling development objects like ABAP classes, ABAP Dictionary objects, function modules, and so on.
- for Core Data Services (CDS) in ABAP development for SAP HANA scenarios. In particular, it focuses on use cases for creating, editing, testing, and analyzing **ABAP CDS entities** like data definitions, and so on.
- when developing all types of SAP Fiori applications as well as publishing Web APIs in the context of the ABAP RESTful Application Programming Model (in short RAP).
- when creating lifecycle-stable and cloud-ready business apps, services, and extensions in the context of the ABAP Cloud development model.
- for integration of native SAP HANA objects (in the context of SAP HANA repository content) within the ABAP layer.

Release Notes

ABAP development tools for Eclipse is released to customers in combination with the SAP Business AI Platform, ABAP environment shipments. Documentation in the context of ABAP Platform will be shipped in accordance with the relevant SAP product shipments.

The release notes are a general overview of the most significant changes relating to features of ABAP development tools for Eclipse in the context of ABAP development and the ABAP programming models.

For more information, see Release Notes of ABAP Development Tools for Eclipse.

FAQs

If you are an `SE80` expert and new to ABAP development tools for Eclipse, the frequently asked questions (FAQs) enable you to skim the features you want to perform in ADT.

For more information, see FAQs for SE80 Experts Using ADT.

Installation

To install ABAP development tools for Eclipse, follow the Installation Guide. Alternatively, you may get ABAP development tools for Eclipse from the SAP Development Tools page under the terms of the SAP DEVELOPER LICENSE AGREEMENT.

Support

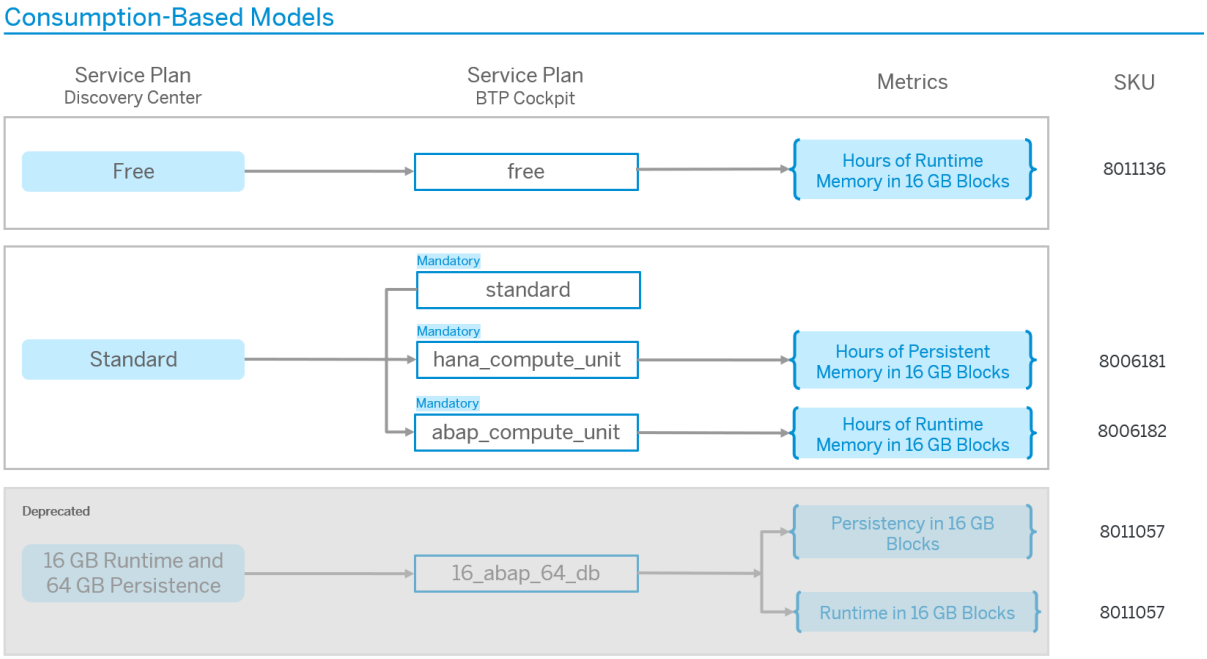
The use of ABAP development tools for Eclipse is subject to the terms and conditions of your license agreement with SAP which is directly related to the SAP shipment channel from which ABAP development tools for Eclipse was initially downloaded and installed.

2.3.2.2 Service Plans and Metering for SAP Business AI Platform, ABAP environment

This page explains the relationship between the service plans of the SAP Discovery Center and the service plans of the SAP BTP cockpit and provides information to help you understand how the service is billed.

Service

Overview



Discovery Center: SAP BTP ABAP environment

> ### Note
>
> For more information about consumption-based models, please check What is the Consumption-Based Commercial Model?

SAP BTP Cockpit: Service Plans

Name	Service Plan	Description
free	Free	Allows you to create small proof-of-concept development projects. Also can be used for the piloting of a remote ABAP test cockpit scenario against on-premise systems in the Custom Code Migration app. The instance will be stopped each night automatically and needs to be started again via the Landscape Portal. Only community support is available for free tier service plans and these are not subject to SLAs.
standard	Standard	Allows you to create projects for development, test, and productive ABAP systems. The minimum system size is 2 HANA Compute Units and 1 ABAP Compute Unit. It is mandatory to assign a combination of three entitlement service plans: ABAP Compute Unit, HANA Compute Unit, and either the Free, Standard, or SaaS OEM plan.
abap_compute_unit	Standard	Allows you to configure runtime memory, which refers to volatile memory used during the execution of applications. This memory provides fast read and write access but loses all data when the system is restarted or shut down. The metric measures the amount of runtime memory used in 16 GB blocks on an hourly basis. It is mandatory to assign a combination of three entitlement service plans: ABAP Compute Unit, HANA Compute Unit, and either the Free, Standard, or SaaS OEM plan.
hana_compute_unit	Standard	Allows you to configure persistent memory, which refers to storage that retains data even after the system is restarted or shut down. The metric measures the amount of persistent storage used in 16 GB blocks on an hourly basis. It is mandatory to assign a combination of three entitlement service plans: ABAP Compute Unit, HANA Compute Unit, and either the Free, Standard, or SaaS OEM plan.
saas_oem	SaaS OEM	Allows you to create projects for ABAP systems to run multitenancy SaaS applications. The minimum system size is 2 HANA Compute Units and 1 ABAP Compute Unit. It is mandatory to assign a combination of three entitlement service plans: ABAP Compute Unit, HANA Compute Unit, and either the Free, Standard, or SaaS OEM plan.

Name	Service Plan	Description
16_abap_64_db (Deprecated)	16 GB Runtime and 64 GB Persistence (Deprecated)	This ABAP runtime service plan provides a 16 GB ABAP runtime with a 64 GB database. No additional sizing is possible. This service plan is deprecated. Please change the service plan to “standard”.

Metrics

Metric	Definition
Memory	Temporary memory bank where computers store data that needs to be retrieve and processed quickly. The memory represents the size of the data that can be processed , and CPU represents the speed at which the data can be retrieved.

Supplemental Terms and Conditions

For more information, see the SAP Business Technology Platform Service Description Guide.

2.3.3 Kyma Environment

SAP BTP, Kyma runtime provides a fully managed cloud-native Kubernetes application runtime based on the open-source project "Kyma". Based on modular building blocks, Kyma runtime includes all the necessary capabilities to simplify the development and to run enterprise-grade cloud-native applications.

Kyma as a Managed Service

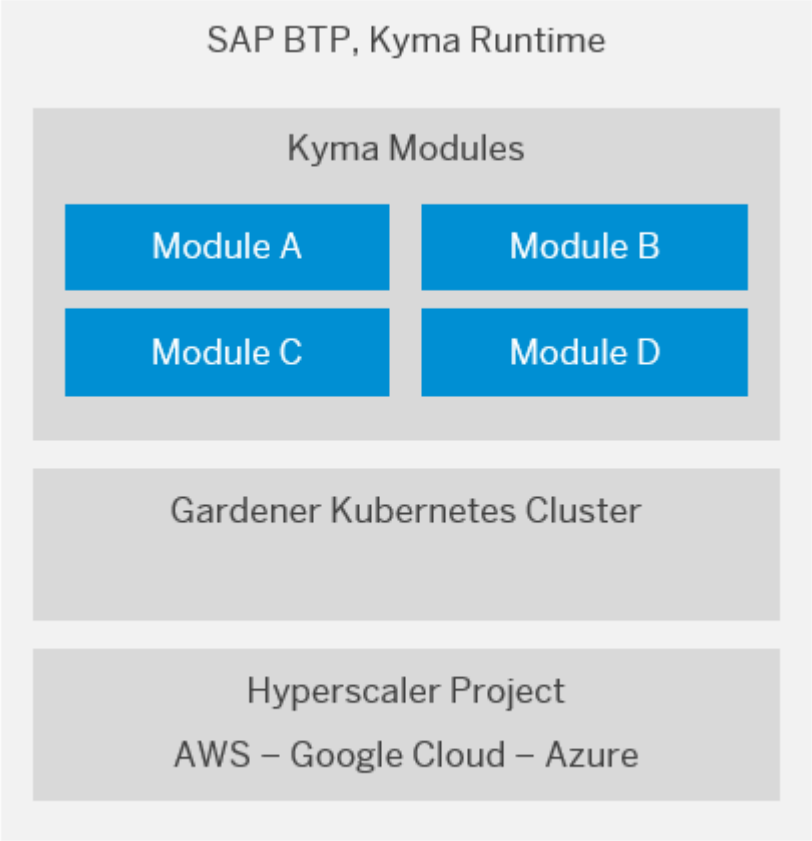
Kyma environment permits a native consumption of the Multi-Cloud Foundation Services (What Is the Multi-Cloud Foundation?) and smooth consumption of SAP and non-SAP applications. It also supports out-of-the-box Cloud Application Programming Model (CAP), SAP Cloud SDK, application router, and HTML5 deployer.

Kyma implements a dedicated application runtime to deploy highly scalable, robust, and secure containerized microservices.

> ### Note
>
> Kyma as a managed service automatically checks all Kyma-managed resources. Any unexpected modifications are discarded, and the resource is reverted to the original state.

Every Kyma environment consists of:

- One or more Kubernetes clusters based on project "Gardener" on a cloud provider and region (data center) of your choice. To find out the available regions and providers, see Regions for the Kyma Environment [page 51].
- A set of Kyma modules picked by a user in the release channel of their choice installed on the provisioned cluster.



Integration

Kyma environment runs on one or more Kubernetes clusters created in a specific subaccount. The configuration of the Kyma environment enables you to connect it to a multitude of SAP systems. This way, you can build various formations that aggregate the SAP systems and environment according to your business use cases.

SAP systems connected to a Kyma environment expose APIs and events. To extend the existing logic of these SAP services, you can build serverless applications called “Functions”, and trigger them to react to particular

events or calls to your application's API. You can also use the Kyma environment to deploy microservices or even build full-stack applications.

Related Information

Getting Started in the Kyma Environment [page 247]
Development in the Kyma Environment [page 1882]
Administration and Operations in the Kyma Environment [page 3450]
Security in the Kyma Environment [page 3644]
Discovery Center
What's New for SAP Business Technology Platform

2.3.3.1 Kyma Modules

With Kyma's modular approach, you can install just the modules you need, instead of a predefined set of components.

Pick the modules that best meet your business needs. You can add or delete modules on demand and configure them, thus determining the costs generated by the infrastructure that your modules use. To learn how, see Adding and Deleting a Kyma Module [page 3511].

You can decide which modules are needed for your workload, although some features may still require the presence of other modules or related capabilities. For example, the API Gateway module depends on Istio capabilities, but it doesn't require the Istio module from Kyma. Instead, the API Gateway module needs Istio custom resources, such as `Gateway` and `VirtualService`, to be present in the cluster. If you want to use your own Istio installation, the API Gateway module will use it as soon as the required resources are available.

> → Tip
>
> A release of a new module's version is announced with a release note in What's New for SAP Business Technology Platform for both, the fast and regular channels:
>
> - On the day of the release in the fast channel, a release note is published with the *Preview* label.
> - When the module version becomes available in the regular channel (after approximately two weeks), the *Preview* label is removed.
>
> For details, see Kyma Module Releases [page 91].
>
> To find out which module version is running in your cluster, go to Kyma dashboard.

Default Kyma Modules

When you create Kyma runtime in SAP BTP cockpit, it is provisioned with the default modules added. The default modules are not mandatory. If you don't need them, you can delete them in Kyma dashboard.

Default Kyma Modules

Module Technical Name	Purpose	Documentation
Istio `istio`	Istio is a service mesh with Kyma-specific configuration.	Istio Module [page 1887]
API Gateway `api-gateway`	API Gateway provides functionalities that allow you to expose and secure APIs.	API Gateway Module [page 1944]
SAP BTP Operator `btp-operator`	Within the SAP BTP Operator module, BTP Manager installs the SAP BTP service operator that allows you to consume SAP BTP services from your Kubernetes cluster using Kubernetes-native tools.	SAP BTP Operator Module [page 2070] What's New for SAP BTP Operator GitHub repository: BTP Manager

Optional Kyma Modules

When you create Kyma runtime in SAP BTP cockpit, the following modules are not added by default, but you can choose to add and delete them anytime in Kyma dashboard.

Optional Kyma Modules

Module Technical Name	Purpose	Documentation
Application Connector `application-connector`	Application Connector allows you to connect with external solutions. No matter if you want to integrate an on-premise or a cloud system, the integration process doesn't change, which allows you to avoid any configuration or network-related problems.	Application Connector Module [page 2116] GitHub repository: Application Connector Manager
Cloud Manager `cloud-manager`	Cloud Manager brings cloud provider products and resources into the Kyma cluster in a secure way.	Cloud Manager Module [page 2106] GitHub repository: Cloud Manager
Eventing `eventing`	Eventing provides functionality to publish and subscribe to CloudEvents. At the moment, the SAP Event Mesh default plan and NATS are supported. If you choose NATS, add the NATS module.	Eventing Module [page 2136] GitHub repository: Eventing
Keda `keda`	The Keda module comes with Keda Manager, an extension to Kyma that allows you to install KEDA (Kubernetes Event-Driven Autoscaler).	Keda Module [page 2120]

Module Technical Name	Purpose	Documentation
NATS `nats`	NATS deploys a NATS cluster within the Kyma cluster. You can use it as a backend for Kyma Eventing.	NATS Module [page 2152] GitHub repository: NATS
Registry Cache `registry-cache`	With the Registry Cache module, you can add a caching layer for container image registries, reducing outbound traffic to upstream registries and improving image pull performance.	Registry Cache Module [page 2160] GitHub repository: Registry Cache
Serverless `serverless`	With the Serverless module, you can define simple code snippets (Functions) with minimal implementation effort.	Serverless Module [page 2169] Configuring Serverless [page 2175] Deploy Workloads in the Kyma Environment to Extend SAP Systems [page 2267] GitHub repository: Serverless
Telemetry `telemetry`	The Telemetry module collects application logs and distributed traces for your application, and dispatches them to your preferred backends.	Telemetry Module [page 2184] What's New for Telemetry GitHub repository: Telemetry

Other SAP Modules

These modules are developed and maintained by SAP teams outside of Kyma. To get help or request a feature, contact the module provider directly.

Other SAP Modules

Module Technical Name	Purpose	Documentation
Transparent Proxy `transparent-proxy`	Use the transparent proxy for Kubernetes to connect workloads in a Kubernetes cluster to Internet and on-premise applications.	Transparent Proxy in the Kyma Environment
Connectivity Proxy `connectivity-proxy`	Use the connectivity proxy for Kubernetes to connect workloads in a Kubernetes cluster to on-premise systems, exposed via the Cloud Connector	On-Premise Connectivity in the Kyma Environment

Community Modules

Community modules are modules provided by the Kyma community. Use them to enhance your Kyma experience with additional features and capabilities. You can install community modules alongside managed modules. Unlike managed modules, community modules aren't automatically updated or maintained. For more information, see kyma-project.io: Community Modules .

> ⚠ Caution
>
> Community modules aren't subject to the Service Level Agreement (SLA).

Related Information

Adding and Deleting a Kyma Module [page 3511]

2.3.3.2 Kyma Module Releases

For each of your Kyma modules, you can choose if you want the regular release, or an earlier preview so you can test new features and provide early feedback.

Kyma Release Channels

Kyma is released in a modular approach. To learn more about the available Kyma modules, read: Kyma Modules [page 88].

Kyma modules are deployed in the following release channels:

- *Regular channel* is the default release channel.
- *Fast channel* provides more frequent releases. It offers early previews of all new features and changes before they are promoted to the regular channel. It also allows you to test and provide feedback on the new features sooner.

According to the Kyma modules` release cycle, we first release a new module's major or minor version in the fast channel. After approximately two weeks, we promote the release to the regular channel.

> ⓘ Note
>
> In case of important functionality fixes or critical vulnerabilities identified by our security organization, the timeline doesn't apply, as we provide hotfixes between regular releases.

You can use one or both release channels in your Kyma cluster, but you can define only one release channel per module. For example, you can mix different modules from the regular and fast channels in your development cluster, but you cannot deploy the same module in the regular and fast versions in one cluster.

> → Tip
>
> You can upgrade module versions, but you cannot downgrade them. To test the upstream versions, you can switch a module or an entire cluster from the regular channel to the fast one.
>
> To return to the regular channel, you must wait until the version you are using in the fast channel is promoted to the regular channel. Once the versions in both the fast and regular channels are the same, you can switch back to regular. Alternatively, you can delete and add your module from the regular channel.

To find out which module version is running in your cluster, go to Kyma dashboard.

Release Notes

A release of a new module's version is announced with a release note in What's New for SAP Business Technology Platform for both, the fast and regular channels:

- On the day of the release in the fast channel, a release note is published with the *Preview* label.
- After approximately two weeks, the module version becomes available in the regular channel and the *Preview* label is removed.

For details on the Kyma release schedule, see 3459911.

> **Note**
>
> For any changes that require user action, we notify users at least three months in advance in the "What's New for SAP Business Technology Platform". You get information on the required or recommended actions, as well as links to more detailed guidance, including details on the opt-in solutions and migration timelines.
>
> To get notifications on such changes and other updates, subscribe to What's New for SAP Business Technology Platform.

Related Information

Kyma Functionalities [page 92]

2.3.3.3 Kyma Functionalities

SAP BTP, Kyma runtime and open source project "Kyma" offer slightly different functionalities and install a different set of components.

For all functionalities that the Kyma environment offers, see the official project "Kyma" documentation .

Functionality Comparison

Functionality	open source project "Kyma"	SAP BTP, Kyma runtime
Service Level Agreements	✕	✓
Managed Kubernetes	✕	✓
Managed Kyma	✕	✓
Kyma CLI	✓	limited (SAP BTP, Kyma runtime supports commands for serverless Functions, not the commands related to installation.)

Functionality	open source project "Kyma"	SAP BTP, Kyma runtime
Centrally hosted Kyma dashboard	✕	✓
System landscape management in SAP BTP cockpit	✕	✓
In-cluster system landscape management (Application Connector)	✓	✕

2.3.3.4 Kyma Dashboard

Use Kyma dashboard to access various features and functionalities of SAP BTP, Kyma runtime.

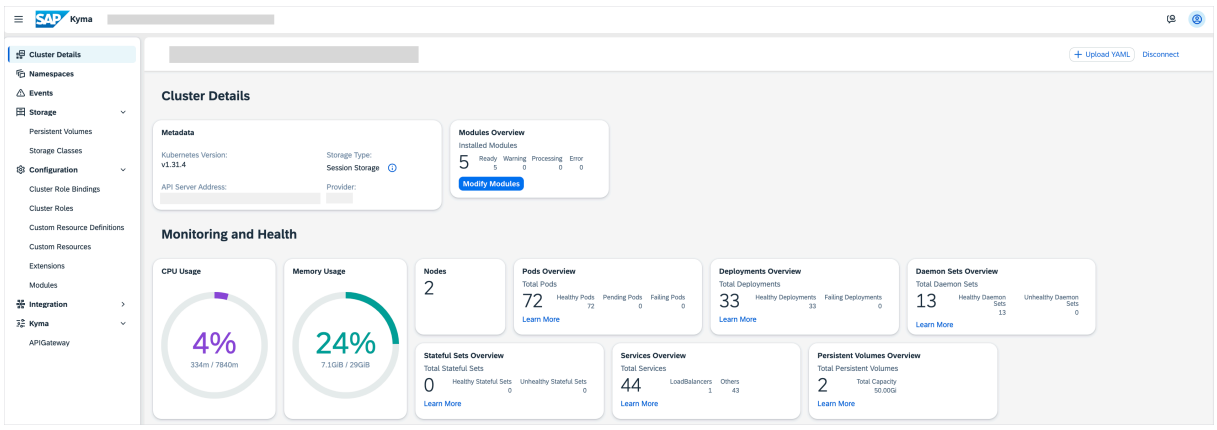
Kyma dashboard is a web-based UI utilizing the React library and the SAP Fiori Horizon design system. Kyma uses Kyma dashboard as a central administration dashboard, which provides a graphical overview of your cluster and all its resources.

To access Kyma dashboard, enable Kyma in your subaccount. The link to the dashboard will be visible in the **Kyma Environment** section.

Features

You can use Kyma dashboard to perform the following actions:

- Add, manage, and delete Kyma modules.
- Manage your namespaces.
- Manage your Kubernetes and Kyma resources.
- Generate access tokens based on service accounts and Kubernetes RoleBindings and ClusterRoleBindings, which regulate access using Role-based access control (RBAC).
- Have a quick look at the basic cluster condition.



Related Information

Creating Kyma Instances [page 3452]
Adding and Deleting a Kyma Module [page 3511]
Setting Your Module to the Unmanaged and Managed State [page 3513]
Using RBAC Authorization

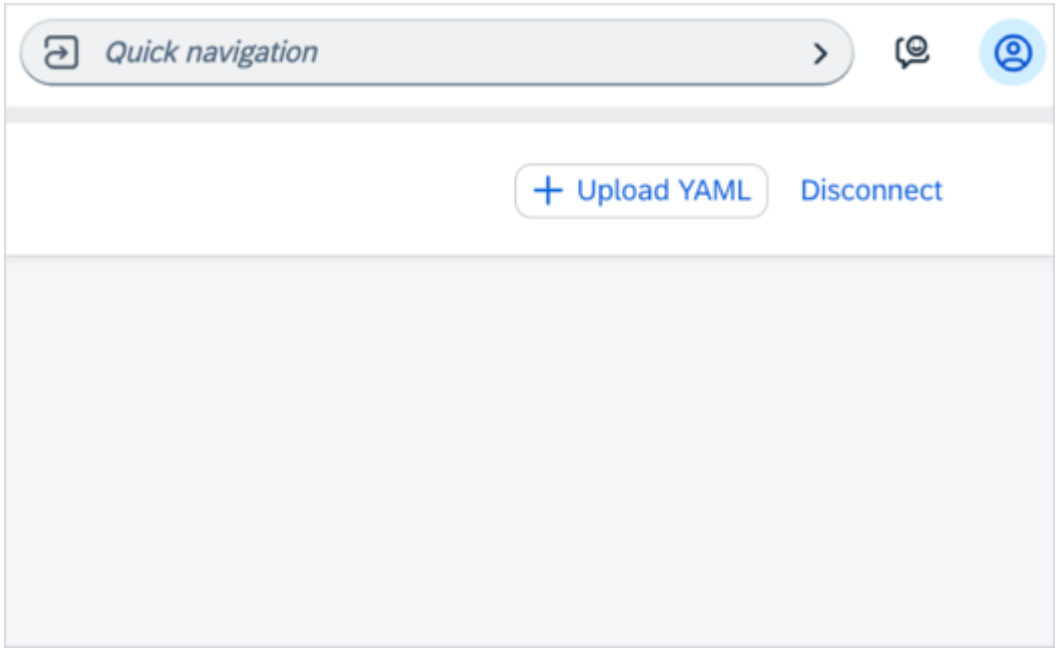
2.3.3.4.1 Kyma Dashboard Command Palette

With the Kyma dashboard command palette feature, you can easily navigate through all your dashboard resources, such as workloads, deployments, and more. You can perform actions such as listing or accessing your namespaces, or preferences, or uploading YAML files.

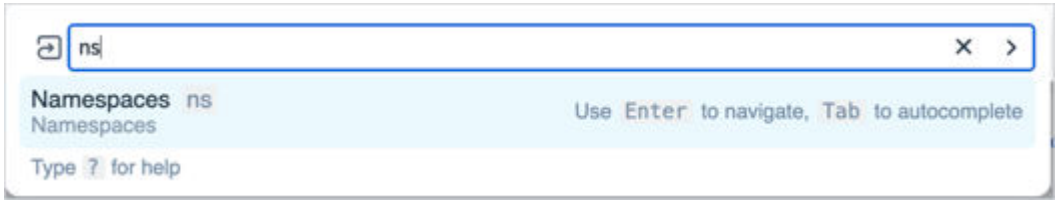
Features

You can use the command palette to perform the following actions:

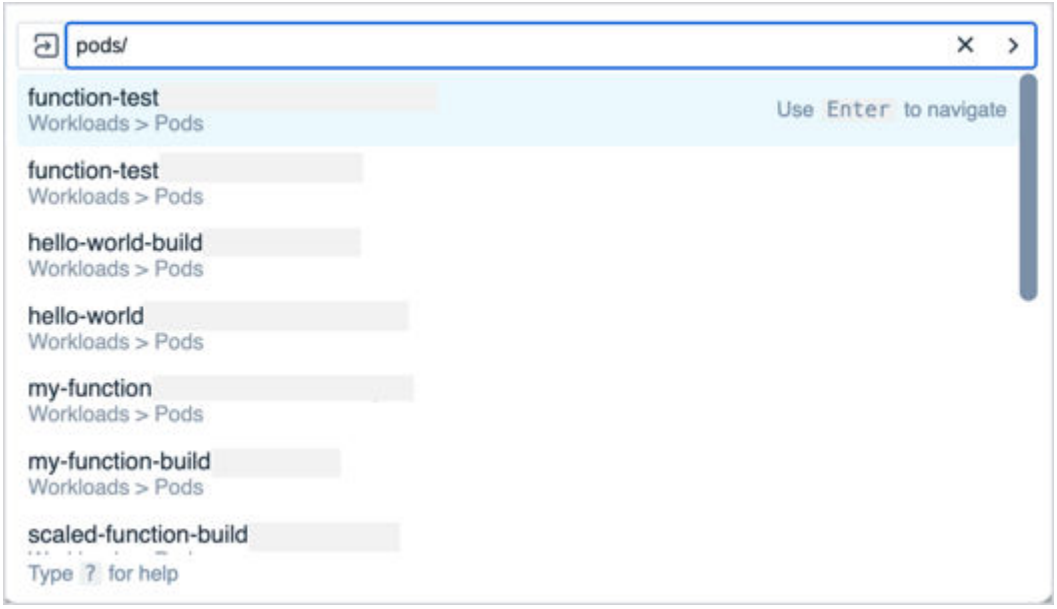
- To access the command palette, press `cmd+k` on your keyboard or choose **Quick Navigation**.



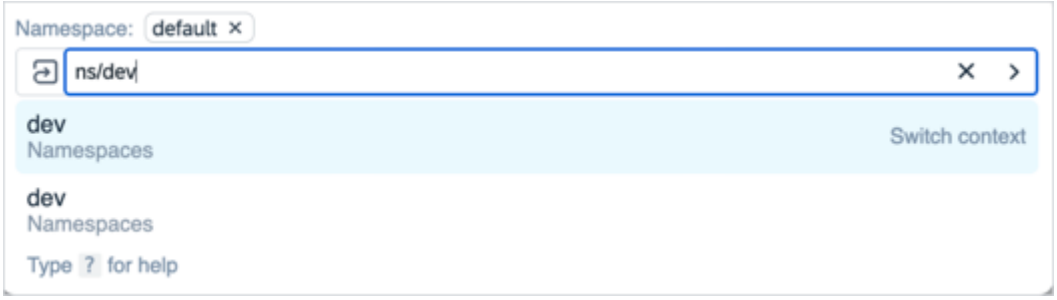
- To see all navigation options and the resources' aliases, enter the question mark sign `(?)`. For example, if you want to see your namespaces, it's enough to type `ns`.



- To see the history of your resources, use the up or down arrow.
- To see the list of all resources available in the given namespace, enter the resource name and press the tab key.



- Switch contexts between your resources. For example, if you're viewing your available Pods in the `default` namespace, and you want to check the Pods in a different one, type `ns/{NAME_OF_YOUR_NAMESPACE}` and choose the `Switch Context` option. It takes you to the list of Pods in the selected namespace.



2.3.3.4.2 Kyma Dashboard Terminal

The Kyma dashboard terminal gives you an interactive shell directly in your browser, without installing any local tools. When you open the terminal, Kyma dashboard creates a lightweight container `Pod` in your cluster and connects your browser to its shell through a WebSocket proxy.

Open the Terminal

To open the terminal, choose the Terminal icon in the top navigation bar. The terminal panel opens at the bottom of the screen.

Kyma dashboard creates the `busola-terminal` namespace in your cluster (if it does not exist yet) and starts a `Pod` using the `busola-dev-toolbox` container image.

> ⚠ Caution
>
> When you close the terminal, the `Pod` is deleted. Any tools, files, or configurations you installed or created during the session are lost. The next time you open the terminal, the `Pod` starts fresh.

Available Commands

The terminal provides a full interactive Bash shell (`/bin/bash`) inside the `busola-dev-toolbox` container. The following are available by default:

- Standard Linux commands (`ls`, `cat`, `grep`, `curl`, `wget`, and more)
- Additional tools bundled in the `busola-dev-toolbox` image

> ⓘ Note
>
> `kubectl` is not yet available. The terminal `Pod` runs without a Kubernetes `ServiceAccount`, which means it has no access to the Kubernetes API. The toolset is actively being expanded — future versions will include `kubectl` and Kyma CLI.

Limitations

The following table summarizes the key limitations of the terminal feature:

Limitation	Details
No `kubectl` access	The terminal `Pod` has no Kubernetes `ServiceAccount`. `kubectl` commands are not available.
Ephemeral session	Closing the terminal deletes the `Pod`. Files, tools, and settings are not preserved between sessions.
Single `Pod` per cluster	Each user has one terminal `Pod` per cluster connection.

Related Information

Tutorial: Check Services with curl in the Kyma Dashboard Terminal

2.3.3.4.3 Troubleshooting for Kyma Dashboard

Troubleshoot problems related to Kyma dashboard.

Cannot Find Kyma Resource in Kyma Dashboard [page 97]

2.3.3.4.3.1 Cannot Find Kyma Resource in Kyma Dashboard

Symptom

When I access Kyma dashboard, I cannot find some Kyma resources in the left panel.

Cause

The relevant Kyma module is not added. For example, if you're missing the Function resource, you must add the Serverless module first. In the case of Subscription, you must add the Eventing module. For the list of all available modules, see Kyma Modules [page 88].

Solution

See Adding and Deleting a Kyma Module [page 3511].

2.3.3.5 Joule in Kyma Dashboard

Use Joule to get AI-assisted help with Kyma and Kubernetes tasks directly in your dashboard.

Joule in Kyma dashboard is a conversational AI assistant that inspects your live cluster resources, retrieves Pod logs, searches Kyma documentation, and guides you through structured troubleshooting, all within the dashboard. You can use it to diagnose issues, understand the state of your workloads, get guidance on creating, updating, and scaling application resources, or ask how-to questions about Kyma features.

To access Joule, go to your Kyma dashboard, and click the diamond icon in the top navigation bar.

> **Note**
>
> Joule isn't available in Kyma clusters that are provisioned in regions with EU restricted access or configured with a custom OpenID Connect (OIDC) client. For a list of all EU Access regions, refer to Regions for the Kyma Environment.

When you send a message, Joule analyzes your query and the current dashboard context to decide what information to retrieve. It prioritizes live cluster data, such as resource status, configuration, and logs, over documentation. It asks for clarification if your request is too broad to answer accurately.

Answers Based on the Active Resource

An active resource in Kyma dashboard refers to the specific Kyma or Kubernetes resource, such as a Function, APIRule, or Pod, that you're currently viewing in the dashboard.

Joule uses the active resource as context for your queries. This means you can ask questions like "Why is my Function failing?" or "Show me the logs for this Pod" without specifying which resource you're referring to. Joule resolves possessive references such as "my Function" or "my Pod" to the resource you're currently viewing, making your interactions faster and more precise.

Features

Joule has the following capabilities in Kyma dashboard:

- Cluster resource inspection: Fetches the current status and configuration of any Kubernetes or Kyma resource in your cluster.
- Pod log retrieval: Retrieves current and previous container logs and `Pod` events when a workload is in an error state to help you diagnose the issue.
- Documentation search: Performs a semantic search over Kyma documentation to find relevant guidance for how-to questions or Kyma-specific errors.
- Cluster overview: Provides a high-level summary of resources in a namespace or cluster-level resources.
- Scope guard: Asks for clarification if your request is too broad or ambiguous, rather than performing unscoped queries to ensure you receive accurate and focused answers.
- Rate limiting: Allows up to 3,000 requests per minute to ensure fair usage. Once you reach the rate limit, subsequent requests return an error code until enough time passes for the call count to drop below the limit.

Related Information

Joule in SAP BTP, Kyma Runtime

2.3.3.6 Kyma CLI

Kyma CLI is an essential tool for application developers who want to get started quickly and efficiently with SAP BTP, Kyma runtime. Designed to streamline workflows, it simplifies complex tasks, enabling developers to deploy and manage applications easily.

> ⚠ Caution
>
> The `alpha` group commands are still in development, which means their functions and API may be modified over time. We encourage you to explore them, but keep in mind that changes may occur.

What is Kyma CLI

With a single command, you can push a simple application to a Kyma cluster. Kyma CLI builds the container image from source code or Dockerfile, pushes it to the in-cluster registry, and applies the necessary Kubernetes resources, eliminating the manual setup process and accelerating your development.

Kyma CLI provides a hands-on experience when interacting with Kyma's extensions to Kubernetes API, making it easier to explore, test, and fine-tune Kyma's custom resources directly from the command line.

Kyma CLI also provides a set of commands to manage Kyma modules efficiently. You can manage, deploy, and configure modules seamlessly. With the Kyma CLI module commands, you can list available and installed modules, and add or delete them. Modules can be deployed with the default or custom configuration.

Features

Kyma CLI provides the following features:

- Simplified module management.
- Automated deployments.
- Simplified SAP BTP service instance bindings.
- Commands providing useful automation.

Install Kyma CLI

- Using Homebrew:

  ```
  brew install kyma-cli
  ```

- Using GitHub releases:

  ```
  curl -sL "https://raw.githubusercontent.com/kyma-project/cli/refs/heads/main/
  hack/install_cli_stable.sh" | sh -
  kyma
  ```

→ Tip

You can add the autocomplete formula to your shell (`bash`, `fish`, `PowerShell`, or `zsh`). Run the following command to import the Kyma CLI autocomplete for the `zsh` and enable autocomplete with the tab key hit: `source <(kyma completion zsh`.

Related Information

Adding and Deleting a Kyma Module [page 3511]
Setting Your Module to the Unmanaged and Managed State [page 3513]

2.3.3.7 Availability Zones in the Kyma Environment

The Kyma environment uses availability zones to provide high availability and fault tolerance for your applications and workloads.

Availability zones are isolated failure domains within a single geographical region, connected through low-latency networks, and operating independently with their own power, network, and cooling infrastructure. This separation minimizes the risk that a failure in one zone affects the others, and a localized issue causes widespread disruption to applications and services.

High Availability

The Kubernetes and SAP BTP, Kyma runtime configurations are optimized for production use cases. By operating on a three-availability-zone architecture, Kyma offers the high availability feature within the following enterprise plans:

- Standard: Amazon Web Services, Google Cloud, Microsoft Azure, and Alibaba Cloud [page 3460]
- Build Runtime: Amazon Web Services, Google Cloud, and Microsoft Azure [page 3464]

This means that the default worker nodes are deployed in three availability zones of the respective cloud region [page 51], and thus can provide zone-level failure tolerance for Kyma and applications deployed on Kyma runtime. The Kubernetes control plane is also hosted in three availability zones. Thus, the runtime automatically manages node and zone failures for all managed components, including the Kubernetes API server.

High Availability in Custom Worker Node Pools

We recommend enabling high availability for additional worker node pools [page 3473] in production environments to enhance resilience. To ensure optimized deployment, Kyma runtime automatically selects specific availability zones during provisioning, so you cannot choose them yourself.

If you disable high availability, your additional worker node pool lacks zone-level failure tolerance because it's deployed in a single, randomly selected availability zone.

> ⚠ Caution
>
> You cannot change the high availability setting of an existing additional worker node pool. To alter this setting, you must delete and recreate the pool with the desired configuration.

High availability for additional worker node pools depends on your choice of virtual machine types:

- For general-purpose machines, three availability zones are always available in all supported regions.
- For other machine types, for example, compute-intensive machines, the number of availability zones varies by region. For more information on machine types and regional availability, see Machine Type: Machine Type in Additional Worker Node Pools [page 3481].

High Availability in User Applications

While high availability is guaranteed for Kubernetes and native Kyma components, it is not automatically provided for your own applications deployed on Kyma. To ensure your applications are resilient to zone failures, you must manually configure high availability by following these guidelines:

- Deploy multiple replicas across different availability zones.
- Use appropriate resource requests and limits.
- Configure health checks and readiness probes.
- Implement proper service discovery and load balancing.

If you deploy with topology spread constraints to distribute the replicas across multiple availability zones, the Kubernetes scheduler distributes them across zones, ensuring that if one zone becomes unavailable, your applications continue to run on replicas in the remaining zones.

For more information on building resilient applications, see Develop Resilient Applications in the Kyma Runtime [page 2278].

Updates and Upgrades

To maintain high availability and reliability of the Kyma environment during updates and upgrades, the following processes are performed:

- Regular updates: Kyma modules are updated regularly across all availability zones to ensure no downtime. By distributing updates across zones, Kyma maintains application and workload availability during routine maintenance.
- Major upgrades and maintenance schedules: For critical upgrades, such as Kubernetes version changes, Kyma manages the process within the multi-zone architecture. Updates occur one node at a time across different availability zones to minimize the risk of simultaneous failures and ensure that essential services remain operational within other zones. In Kyma, necessary downtimes are planned and managed during designated periods, following the BTP maintenance window schedule. The sequential update process ensures continuous operation within unaffected zones, maintaining service integrity and platform availability.

2.3.3.8 Kyma Runtime: Basic Concepts

This table explains basic concepts relating to the Kyma environment. It aims to give you an understanding of the Kyma environment before you actually start using it to build extensions for your SAP solutions.

> **Note**
>
> For an overview of the basic Kubernetes concepts that the Kyma environment heavily relies on, see the official Kubernetes documentation.

Concept	Description
Kyma cluster	A Kubernetes cluster provisioned with the latest version of the open-source project "Kyma". You can enable such a cluster on a given subaccount through the SAP BTP cockpit. After creating a Kyma environment instance on your subaccount, the cluster is provisioned automatically through Gardener on your chosen cloud service provider. To access the cluster, you must have appropriate roles assigned to your subaccount.
Kyma module	An extension to the Kyma environment that can be added, deleted, or re-configured at runtime.
Role	Access to every cluster is managed by the roles assigned. Roles give the assigned users a different level of permissions suitable for different purposes. For more information, read Assign Roles in the Kyma Environment [page 3666].
Namespace	Namespaces are used to organize objects in a cluster and provide a way to divide cluster resources. This way, several users can share a cluster but have access only to resources within the namespace they have permissions for. This allows for increasing the security and organization of your cluster by dividing it into smaller units. Access to namespaces in the Kyma environment depends on your Kubernetes RBAC permissions.
Service operator	A service operator is a piece of software that provides a set of all necessary resources (such as CustomResourceDefinitions and controllers) needed to provision third-party services in your Kubernetes cluster.
Binding	The connection you create between a service instance and an SAP solution so that they can communicate with each other. You can also bind a service instance to any workload running in the Kyma environment, such as a Function or a microservice.
CustomResourceDefinition (CRD)	A Kubernetes API resource that allows you to define and manage custom resources (CRs), enabling the extension of Kubernetes API to cover additional use cases.
Custom resource (CR)	An object that extends the Kubernetes API and is defined by a CustomResourceDefintion (CRD), allowing for the management of use cases that are not directly covered by core Kubernetes.
Credentials / Secrets	Sensitive data necessary for an SAP solution to call the service, connect to it, and authenticate it. Depending on whether you use Kyma dashboard or kubectl to create the binding between a service instance and an SAP solution, the Kubernetes Secret object that contains these credentials

Concept	Description
	is either created automatically or you need to create it manually.
Function	A simple code snippet that you can run without provisioning or managing servers. It implements the exact business logic you define. A Function is based on the Function custom resource and can be written in either Node.js or Python. A Function can perform a business logic of its own. You can also bind it to an instance of a service and configure it to be triggered whenever it receives a particular event type from the service or a call is made to the service's API. Functions are executed only if they are triggered by an event or an API call.
Microservice	An architectural variant for extensions or applications, where you separate the tasks into smaller pieces that interact with each other as loosely coupled, independently deployable units of code. A failing microservice should not cause your whole application to fail. Microservices are packed in a container that is always running; it's idling if there is no load. The microservice should always be reachable even when the Pods move around. Microservices typically communicate through APIs.

2.3.3.9 Service Plans and Metering for Kyma Runtime

This page explains the relationship between the service plans of the SAP Discovery Center and the service plans of the SAP BTP cockpit and provides information to help you understand how the service is billed.

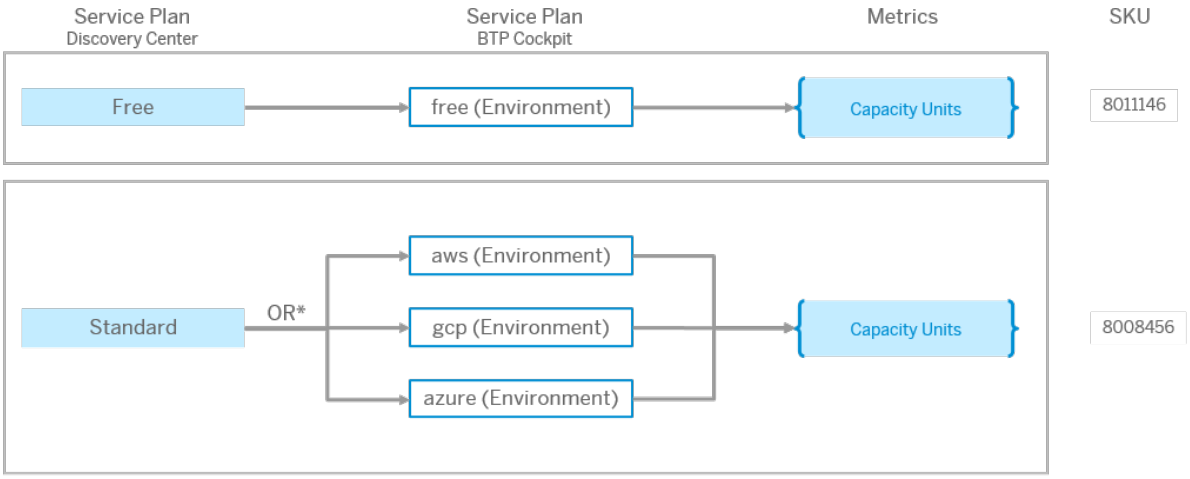
Service

Overview

The following diagram shows how the service plans listed in the SAP Discovery Center correspond to the plans you choose in the SAP BTP cockpit. For more information about the commercial model, see What is the Consumption-Based Commercial Model?

Note that there's no subscription-based commercial model for SAP BTP, Kyma runtime.

Consumption-Based Models



* Depending on which hyperscaler the subaccount is hosted on

Subscription

No Subscription available for SAP BTP, Kyma runtime

SAP BTP Cockpit: Service Plans

Name	Service Plan (Discovery Center)	Description
free	Free	Subscribe to the 30-day free plan provided on Amazon Web Services. This plan uses Kyma on a limited-size cluster (4 CPU—16 GB RAM). You can use the free plan only once in a global account for up to 30 days. The upgrade to the paid plan is not yet supported. Only best-effort support is available for free tier service plans, and these are not subject to SLAs. The services you plan to use must be available in the same region as the subaccount for the Kyma runtime.
aws	Standard	Select Amazon Web Services as the cloud provider where your Kyma cluster is deployed.
gcp	Standard	Select Google Cloud as the cloud provider where your Kyma cluster is deployed.
azure	Standard	Select Microsoft Azure as the cloud provider where your Kyma cluster is deployed.

Metrics

The usage metric for SAP BTP, Kyma runtime is a Capacity Unit (CU) per hour.

Metric	Definition
Capacity Unit	Number of units consumed by the usage of the services as outlined in the solution-specific product supplement. For SAP BTP, Kyma runtime, CUs are calculated for **nodes** and for **storage**.

Backward Calculation

> → Tip
>
> If you use the Cloud Manager module, also see Calculation with the Cloud Manager Module [page 106].

Formula

The formula is a mapping between consumed nodes/storage and the respective CUs. The details are provided in the following tables.

However, as Kyma supports flexible scaling, cost per hour may vary because it depends on the size of the nodes that were used.

Underlying Metrics: Nodes

The nodes calculation takes the number of virtual CPUs (vCPUs) and provides the number of CUs per hour. To better understand the relationship between vCPUs and nodes, note that one node is sized with 2 vCPUs.

The Calculation for vCPU table presents data for general-purpose machine types.

Additionally, the calculation is used for compute-intensive machine types.

Calculation for vCPU

Number of vCPUs	Number of CUs per Hour
2	0.24 * 1.5
4	0.24 * 2
8	0.24 * 4
16	0.24 * 8
32	0.24 * 16
...	...

Underlying Metrics: Storage

SAP BTP, Kyma runtime is provisioned with 32 GB of storage dedicated to operational data at no additional cost. However, certain Kyma modules may require extra storage. In such cases, they create additional volumes, which incur charges for the extra storage used.

You can calculate CUs for the following storage types:

- PersistentVolumes: Storage is provided in blocks of 32 GB. If you use 33 GB, you're charged for 64 GB (2 blocks of 32 GB).

- VolumeSnapshotContents: You pay based on the actual amount of storage used.
- Additional volume size: Each worker node in a Kyma cluster comes with a default disk volume. The size depends on the machine type. This default volume stores Docker images and container logs for operational purposes and isn't charged separately. If the default volume size isn't sufficient (for example, when running many containers that produce large log volumes or pulling many large Docker images), you can request additional volume size beyond the default. You configure this using service instance parameters. Any storage beyond the default volume size is an additional node volume and is billed accordingly. For example, if you use 4 nodes of 10 GB additional volume, totaling 40 GB, you're charged for 64 GB (2 blocks of 32 GB).

Calculation for Storage

Amount of Storage	Number of CUs per Hour
32 GB	0.02

Examples

- Your consumption report for nodes shows 1800 capacity units charged for 4 vCPU nodes for 30 days.
  This means that you were running roughly 1800 / (0.24 * 2) = 3750 hours worth of 4 vCPU nodes. Given that 30 days have 720 hours, this means you were running 3750/720 = 5.21 nodes cluster for the 30 days. Due to Kyma's automatic scaling, 5.21 does not mean actual size. It means that during some hours in the 30 days, the cluster size was 4 nodes. At other times, there was more load, so the cluster autoscaled to 5 or 6 nodes, based on your configurations. So your average cluster size for the 30 days was 5.21 nodes.
- Your consumption report for storage shows 200 capacity units charged for storage (32 GB block) for 30 days.
  This means that you used 200 / 0.02 = 10000 hours of 32 GB storage blocks. Given that 30 days have 720 hours, this means that you have used 10000/720 = 13.89 blocks of 32 GB over the 30 days. So, you used 13.89 * 32 = 444.44 GB of storage continuously during the 30 days.
  This implies that as you started using Kyma, you deployed more applications that used storage of various sizes, such as 4 GB or 8 GB.

Calculation with the Cloud Manager Module

Calculation with the Cloud Manager Module

Using the Cloud Manager module and enabling Redis or NFS storage or both, introduces additional costs.

- The cost of NFS storage introduces factor 3 to the calculation. Note that NFS storage is provided in blocks of 32 GB, so if you use 33 GB, you're charged for 64 GB (2 blocks of 32 GB).

  Calculation for NFS Storage

Amount of NFS Storage	Number of CUs per Hour
32 GB	0.06 (= 0.02 * 3)

- Using Redis adds costs depending on the tier you choose.

> ### Note
>
> Even though the actual available size (GiB) might vary depending on the cloud provider, the tiers for Redis are standardized across cloud providers. Because of that, also the pricing is standardized across all available cloud providers.

Calculation for Redis Tiers

Tier	Approx. Available Size (GiB)	Cost in Storage Size per Hour	Number of CUs per Hour
S1	1	0.25277778	0.1027777778
S2	3	0.50555556	0.2055555556
S3	6	1.32361111	0.5375000000
S4	12	2.65972222	1.0805555556
S5	24	5.26805556	2.1402777778
S6	48	10.54305556	4.2833333333
S7	101	21.08333333	8.5652777778
S8	202	42.15694444	17.1263888889
P1	5	2.64305556	1.0736111111
P2	12	5.31666667	2.1597222222
P3	24	10.53611111	4.2805555556
P4	48	21.08333333	8.5652777778
P5	101	42.17083333	17.1319444444
P6	200	84.31111111	34.2513888889

Examples for Cloud Manager

- Your consumption report for the NFS storage shows 200 capacity units charged for storage (32 GB block) for 30 days.
  This means that you used 200 / 0.06 = 3333.33 hours of 32 GB storage blocks. Given that 30 days have 720 hours, this means that you have used 3333.33/720 = 4.63 blocks of 32 GB over the 30 days. So, you used 4.63 * 32 = 148.15 GB of NFS storage continuously during the 30 days.
- Your consumption report for Redis shows 200 capacity units for storage for 30 days.
  This means that you used 200 / 0.1027777778 = 1945.946 hours of the Redis S1 tier. Given that 30 days have 720 hours, this means that you have used 1945.946/720 = 2.703 tier S1 Redis instances continuously during the 30 days.

Calculator

Find the Kyma price calculator at https://kyma-project.github.io/price-calculator/ and use it to estimate the costs for your SAP BTP, Kyma runtime.

1. Choose the base configuration for the size and minimum number virtual machines (VM) you need.

2. Optionally, add more storage to the calculation.
3. If needed, adjust the conversion rate.

Note that the result is an estimate, and the consumption report may vary depending on the actual hours and size of the Kyma cluster (nodes and storage) that was running in 30 days.

Supplemental Terms And Conditions

For more information, see SAP Business Technology Platform Service Description Guide, section "SAP BTP, KYMA RUNTIME".

Glossary

Commercial Information Glossary

Service Specifics

As soon as you instantiate a Kyma cluster, the basic cost for the empty cluster incurs.

Cost may then vary depending on the actual nodes and storage you consume.

2.3.4 Neo Environment

The Neo environment lets you develop HTML5, Java, and SAP HANA extended application services (SAP HANA XS) applications. You can also use the UI Development Toolkit for HTML5 (SAPUI5) to develop rich user interfaces for modern web-based business applications.

See also:

- What Is SAP BTP, Neo Environment
- Development, Neo Environment
- Extensions, Neo Environment
- Administration and Operations, Neo Environment
- Security, Neo Environment
- Getting Support, Neo Environment

Migration from the Neo to the Multi-Cloud Foundation

To learn more about why and how to migrate your scenarios on SAP BTP, see Migrating from the SAP BTP Neo Environment to the Multi-Cloud Foundation.

2.4 Trial Accounts and Free Tier

Explore the different options for trying out SAP BTP.

Trial Account or Free Tier Offering: When to Use Which?

Before setting up your account, you need to decide which free offering for SAP BTP is suitable for your needs:

- **Pay-As-You-Go or CPEA accounts with free tier service plans** are open to customers, partners, and let you try out SAP BTP in a per service defined time span for free. These account types enable you to test your scenarios and generally offer the option to upgrade to paid service plans. These accounts also allow you to store data long-term and move projects to production. You also get access to our community, including free technical resources such as tutorials and blog posts. For more information, see Enterprise Accounts [page 111] and Using Free Service Plans [page 118].

  > ### Note
  >
  > When using Kyma, the free plan is available only once in a global account for up to 30 days. For more information, see Available Plans in the Kyma Environment.

  You can self-register for an enterprise account with free tier service plans. For more information, see Get an Account on SAP BTP to Try Out Free Tier Service Plans.

  > ### Note
  >
  > Only community support is available for free tier service plans and these are not subject to SLAs.

  > ### Note
  >
  > The option to upgrade from free tier service plans to paid service plans is not yet available for all services and runtimes, such as Kyma. See Using Free Service Plans to read more about upgrading from free tier service plans to paid service plans.

- A **trial account** lets you try out SAP BTP for free for 90 days. The services provided for the trial account allow restricted use of the platform resources and services. Access is open to everyone. Trial accounts are intended for personal exploration, your own non-productive testing, and evaluation of the services in accordance with SAP BTP trial terms and conditions. A SAP BTP trial account must not be used for production use or team development. You are not permitted to use the trial account in any productive or commercial manner.
  You can self-register for a trial account. For more information on how to do that, see Get a Free Account on SAP BTP Trial

Free Tier and Always Free Tags in the SAP Discovery Center

In the SAP Discovery Center service catalog overview, some services display the tag *Free Tier*, which indicates the service is offering a free tier service plan. Some services display the tag *Always Free*, which indicates the service is offering a service plan that comes free of additional charges, as it is already included in your overall SAP BTP contract. You can use the filter function of the SAP Discovery Center to filter for services that offer *Free Tier* and *Always Free* plans. The *Always Free* service plans include the following note in their service plan description: "This service plan is included in the overall SAP Business Technology Platform contract".

> **Note**
>
> *Always Free* service plans might not be available in all regions or for all providers.

Trial Lifecycle

- **Use your SAP BTP trial account:**
  Familiarize yourself with the Trial Scope [page 110] or try out one of our Starter Scenarios on the Tutorial Navigator.
- **Extend your SAP BTP trial account in intervals:**
  - Your 90-day trial period is divided in intervals. If you sign in to your trial account regularly, the intervals are extended automatically for you by up to 30 days or until your overall 90-day trial is finished.
  - If you don't sign in to your trial account for 30 days or more, your account will be suspended. All suspended accounts will be deleted and you won't be able to use applications or services.
  - **Delete your SAP BTP trial account:**
    After 90 days, your trial account is automatically deleted. But if you want to proactively delete your SAP BTP trial account, you can navigate to the global accounts scope and select the Account Explorer page, then click the *Delete Trial Account* button.
    - If you want to continue to use an SAP BTP trial account after deletion, you need to set up a new account.
    - If you want to explore SAP BTP without time limit, create an enterprise account with free tier service plans allowing you to test your scenarios. See: Get an Account on SAP BTP to Try Out Free Tier Service Plans.

> **Caution**
>
> Environments enabled on your trial account may not be valid for your whole trial period. To learn about the validity period for the Kyma environment, see About the Trial Kyma Instance [page 252].

Trial Scope

- A trial account enables you to explore the basic functionality of SAP BTP.
- SAP BTP trial accounts are available in several regions. For more information, see Regions [page 17].

- You can create directories in your trial account. For more information, see Managing Directories Using the Cockpit [page 2501]
- You can use productive and beta services. To consume beta services, you must enable the subaccount for beta features during the subaccount creation or when you edit the subaccount details.
- You can manage platform users by assigning them role collections. For more information, see Working with Role Collections.
- A trial account includes 4 GB of memory for applications.
- You can use 8 GB of instance memory.
- There are 10 total routes and 40 total services available.
- You can use 2 configured on-premise systems with the Cloud connector.
- There's no service level agreement with regards to the availability of the platform.
- You can use HDI containers in a shared SAP HANA database (only available on cf-us10).
- For cleanup purposes, applications stop automatically on a daily basis. You need to manually restart them when needed.

  > **Note**
  >
  > Applications are stopped at midnight (or some time later depending on server load) relative to the region in which you created your trial account. If you're working in a time zone that is far from the region where your trial account was created, then your applications may stop during business hours.

- You can have a trial tenant of SAP Cloud identity Services, see Get Your Tenant.

Related Information

Getting Started with a Trial Account in the Cloud Foundry Environment [page 165]
Getting Started with a Trial Kyma Instance [page 249]

2.5 Enterprise Accounts

Enterprise accounts are usually associated with SAP customer or partner contracts and contain their purchased entitlements to platform resources and services. However, it's also possible to create an enterprise account for personal exploration.

SAP BTP provides different types of global accounts, **enterprise** and **trial**. The type you choose determines pricing, conditions of use, resources, available services, and hosts.

If you want to start out using services for free, you can create an enterprise account and only use free tier service plans: The major benefit of using such an account for exploration purposes is that you can upgrade your services to productive use without losing any of your data. Check out the Free Tier Services in SAP Discovery Center and learn the details about Using Free Service Plans [page 118] in your enterprise account.

Another way to start out using services for free is to create a trial account. But keep in mind that when you want to use your global account productively, there's no way to migrate your data from the trial account to a productive enterprise account. For more information, see Trial Accounts and Free Tier [page 109].

Enterprise Accounts

The main features of enterprise accounts are described for use by customers and by partners in the following table:

	Customer Account	Partner Account
Use case	A global account that enables you to host productive, business-critical applications with 24/7 support. You can purchase a global account just like any other SAP software. You can upgrade and refine your resources later on. You can also contact your SAP sales representative and opt for a configuration, tailored to your needs. If you want to start out exploring services for free, you can sign up for a Pay-As-You-Go for SAP BTP global account and make use of the free tier services only. See Using Free Service Plans [page 118].	A global account that enables you to build applications and to sell them to your customers. If you want to start out exploring services for free, you can sign up for a Pay-As-You-Go for SAP BTP global account and make use of the free tier services only. See Using Free Service Plans [page 118].
Benefits	Support for productive applications.	• Includes SAP Application Development licenses that enable you to get started with scenarios across cloud and on-premise applications. • Offers the opportunity to certify applications and receive SAP partner logo package with usage policies. • Advertise and sell applications via the SAP Store
Services available	Productive services.	Productive and beta services.
Limitations	Resources according to your contract.	Predefined resources according to your partner package. You can purchase additional resources if necessary.

	Customer Account	Partner Account
Registration	For more information about SAP Integration Suite and SAP Extension Suite pricing, see https://www.sap.com/products/extension-suite/pricing.html and https://www.sap.com/products/integration-suite/pricing.html. Contact us on SAP BTP or via an SAP sales representative. If you're located in China and want to buy a global account on SAP BTP you need to contact an SAP sales representative: https://www.sap.cn/registration/contact.html .	Sign up to join the SAP Partner Program.
Available Regions	See Regions [page 17].	See Regions [page 17].

Cancelling a SAP BTP Enterprise Account

As a business user, you can cancel your contract via your account executive. As an individual user with a trial account, you can cancel your account in your SAP for Me account. You can also contact the SAP store via their "Contact Us" function.

For the commercial models available for enterprise accounts, see Commercial Models [page 113].

Related Information

Getting Started in the Kyma Environment [page 247]
Getting Started with a Customer Account in the ABAP Environment [page 184]
Getting Started with an Enterprise Account in the Cloud Foundry Environment [page 176]

2.5.1 Commercial Models

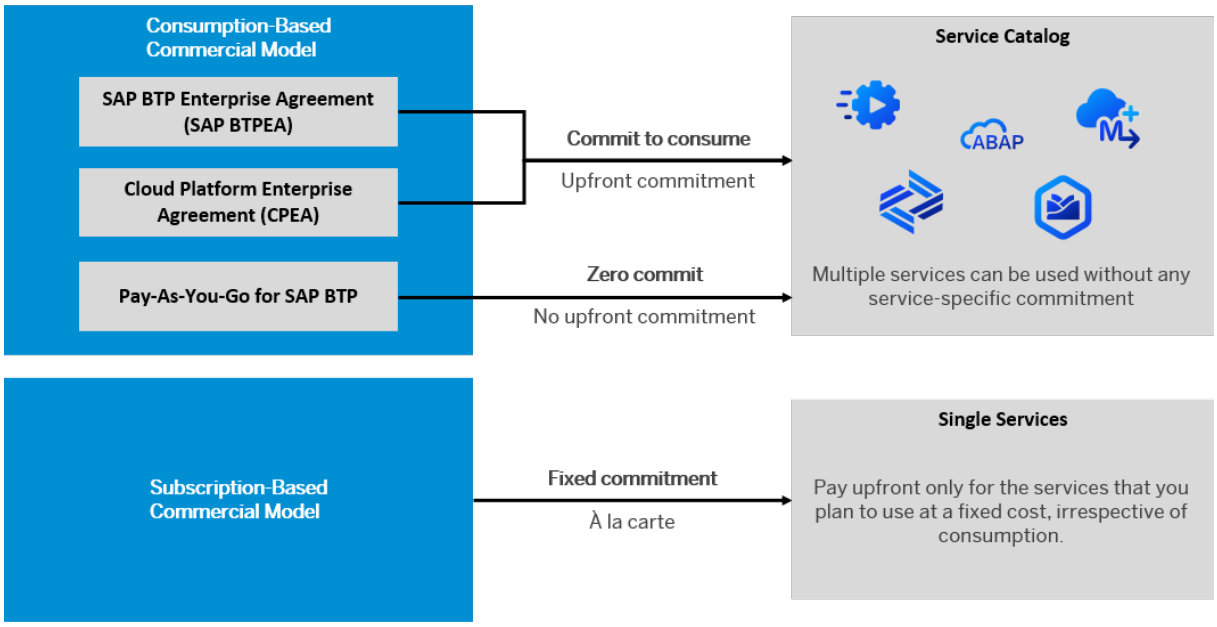
SAP BTP offers two different commercial models for enterprise accounts.

- **Consumption-based commercial model:** Your organization receives access to all current and future services that are eligible for this model. You have complete flexibility to turn services on and off and to switch between services as your business requires throughout the duration of your contract. This commercial model is available in the following flavors: SAP BTP Enterprise Agreement (SAP BTPEA), Cloud Platform Enterprise Agreement (CPEA), and Pay-As-You-Go for SAP BTP.
  For more information, see What Is the Consumption-Based Commercial Model? [page 114]

- **Subscription-based commercial model:** Your organization subscribes only to the services that you plan to use. You can then use these services at a fixed cost, irrespective of consumption.
  For more information, see What Is the Subscription-Based Commercial Model? [page 117]

> **Note**
>
> You can use both commercial models, either in separate global accounts or in the same global account depending on your business needs. Contact your SAP account executive or sales representative for more information. Note however that you cannot mix more than one flavor of the consumption-based commercial model in the same global account.

For information about service availability, prices, and estimators, see https://www.sap.com/products/technology-platform/solutions.html and https://www.sap.com/products/technology-platform/integration-suite/pricing.html. You can also view the service catalog via the SAP Discovery Center.



2.5.1.1 What Is the Consumption-Based Commercial Model?

> **Note**
>
> The use of the consumption-based commercial model is subject to its availability in your country or region.

With the consumption-based model, your organization purchases an entitlement to all current and future SAP BTP services that are eligible for this model. Throughout the duration of your contract, you have complete flexibility to turn services on and off and to switch between services as your business requires.

The consumption-based commercial model is available in the following flavors: the **SAP BTPEA (SAP BTP Enterprise Agreement)**, the **CPEA** (**Cloud Platform Enterprise Agreement**), and **Pay-As-You-Go for SAP BTP**. Each option is suited to different business situations and levels of financial commitment, as described in the table below. For additional information and clarifications, please contact your account executive.

SAP BTPEA	• Your organization makes a prepaid investment in cloud credits for the contract duration with an annual commitment to consume SAP BTP services. • This model is suitable for customers that have well-established and planned use cases, and who want the flexibility of turning services on and off, and switching between services, without the commitment of being tied to a single service throughout the duration of the contract. • You receive a monthly balance statement that provides information about the usage consumption of each service and the corresponding costs. The total monthly cost is deducted from your cloud credits balance. • This model has a minimum investment entry, and volume-based discounts are available. • You are billed annually in advance. Any overages are billed in arrears at list price. • You can top up your cloud credits at any time to prevent overages. • You get access to new SAP BTP services that are added to the consumption-based service catalog.
CPEA **Note** Certain restrictions apply. See the *Frequently Asked Questions* section in https://www.sap.com/products/technology-platform/pricing.html.	• Your organization makes a prepaid investment in cloud credits for the contract duration with an annual commitment to consume SAP BTP services. • This model is suitable for customers that have well-established and planned use cases, and who want the flexibility of turning services on and off, and switching between services, without the commitment of being tied to a single service throughout the duration of the contract. • You receive a monthly balance statement that provides information about the usage consumption of each service and the corresponding costs. The total monthly cost is deducted from your cloud credits balance. • This model has a minimum investment entry, and volume-based discounts are available. • You are billed annually in advance. Any overages are billed in arrears at list price. • You can top up your cloud credits at any time to prevent overages.

Pay-As-You-Go for SAP BTP	• You have the same access to all the services that are available in SAP BTPEA, but with a highly flexible zero-commitment model – you pay nothing upfront and there is no minimum usage requirement or annual commitment. • You pay only for the SAP BTP services that you want, when you use them. • You are billed monthly in arrears. • Service charges are non-discountable. • This low-risk model is suitable for customers with use cases that are not well defined, and are interested in running a proof-of-concept in a productive environment. This model provides the flexibility of turning services on and off, and switching between services, as needed throughout the duration of the contract. • A seamless transition to the CPEA model is available, on the condition that you have no other CPEA-based global accounts. If you want information about the **Pay-As-You-Go for SAP BTP for cloud test, demo, and development** commercial model , go to the SAP Partner Portal.

> **Note**
>
> In global accounts that use the consumption-based commercial model, a default runtime memory quota of 200 GB for the standard plan is automatically assigned to every subaccount. This quota represents a limit on how much runtime memory the applications in the subaccount can use at any given time. When you use the standard plan for SAP BTP, Cloud Foundry runtime in the consumption-based commercial model, you're billed based on the amount of runtime memory that has actually been consumed. For more information, see Service Plans and Metering for Cloud Foundry Runtime and Consumption Monitoring.
>
> A global account administrator can change the default quota as described in (optional) Adjusting Runtime Memory Quota for Cloud Foundry Runtime (Consumption-Based).

> **→ Tip**
>
> You can monitor costs and service usage throughout the contract period. See Monitoring Usage and Consumption Costs in Your Global Account [page 2471].

For information about eligible services and pricing, see https://www.sap.com/products/technology-platform/pricing.html, or access the SAP BTP service catalog via the SAP Discovery Center. The SAP BTP service catalog allows you to identify service availability per data center and to determine licensing model compatibility per available service plan.

As part of the free tier model for SAP BTP, some services offer free service plans that allow you to try out these services without any additional charge within your SAP BTPEA, CPEA, or Pay-As-You-Go for SAP BTP global account. For more information, see Using Free Service Plans [page 118].

To find frequently asked questions about the consumption-based commercial model, see the *Frequently Asked Questions* section in https://www.sap.com/products/technology-platform/pricing.html .

> **Note**
>
> Depending on your business needs, you can combine the consumption-based commercial model with the subscription-based commercial model in the same global account, or you can use both commercial models in the same global account. Contact your SAP account executive or sales representative for more information. Note however that you cannot mix more than one flavor of the consumption-based commercial model in the same global account.

Related Information

What Is the Subscription-Based Commercial Model? [page 117]
Monitoring Usage and Consumption Costs in Your Global Account [page 2471]
View Subaccount Usage Analytics [page 2518]
Managing Global Accounts Using the Cockpit [page 2464]
Managing Directories Using the Cockpit [page 2501]
Managing Subaccounts Using the Cockpit [page 2509]
Managing Entitlements and Quotas Using the Cockpit [page 2521]

2.5.1.2 What Is the Subscription-Based Commercial Model?

Your organization receives a fixed price and period (typically a 1 to 3-year period) for access to your subscribed SAP BTP services.

Under this commercial agreement:

- You are entitled to use only the subscribed services.
- To access additional services, at an extra cost, you can modify your contract via your sales representative or account executive.
- You pay at a fixed cost, regardless of consumption of subscribed services.
- You pay in advance when the contract period starts.
- Your organization can renew the subscription at the end of the contract period.

For information about available services and pricing, see SAP Store.

You can also access the SAP BTP service catalog via the SAP Discovery Center to identify the availability of services by data center, and also to determine licensing model compatibility per service plan.

Note that some services can be subscribed based on a user metric or a resource metric. For example, a Portal service can be based on the number of site visits or user metrics. A resource-based metric is more common when dealing with a large number of users; for example, suppliers accessing a portal to interact with your organization. Since it isn't always possible to predict how many resources would be required upfront for a three-year period, you can increase your original order if resource usage exceeds your subscribed quota. Using SAP BTP cockpit, you can view resource consumption within your global account on a monthly basis.

In the subscription-based model, you also get access to bundles or packages that comprise several related services and apps. Most of the time, this works out to be more cost effective when compared to subscribing to individual SAP BTP services.

Can an existing global account under a consumption-based contract be transformed to the subscription-based model?

You can change your existing contract from the consumption-based commercial model to a subscription license. Keep in mind that not all services that are eligible for the consumption-based model are compatible with the subscription-based model. We recommend that you contact your SAP account executive or sales representative to discuss feasibility and terms of transforming your contract.

Note that you can use both consumption and subscription-based commercial models either in separate global accounts or in the same global acccount depending on your business needs. Contact your SAP account executive or sales representative for more information.

Related Information

What Is the Consumption-Based Commercial Model? [page 114]
Monitoring Usage and Consumption Costs in Your Global Account [page 2471]
View Subaccount Usage Analytics [page 2518]
Managing Global Accounts Using the Cockpit [page 2464]
Managing Directories Using the Cockpit [page 2501]
Managing Subaccounts Using the Cockpit [page 2509]
Managing Entitlements and Quotas Using the Cockpit [page 2521]

2.5.2 Using Free Service Plans

The free tier model for SAP BTP lets you try out services in global accounts without any additional cost using the consumption-based commercial model and an enterprise account.

We offer free service plans for many services within SAP BTP. Free service plans are limited in scope and capacity. They are designed to allow you and your team to explore new SAP BTP capabilities before committing to subscribing to a full capacity paid service.

The free tier model for SAP BTP is added automatically to new and existing contracts using the consumption-based commercial model. There, the free service plans are visible.

> **Note**
>
> The use of the consumption-based commercial model is subject to its availability in your country or region.

The consumption-based commercial model is available in the following flavors: the **SAP BTPEA** (SAP BTP Enterprise Agreement), the **CPEA** (**Cloud Platform Enterprise Agreement**) and **Pay-As-You-Go for SAP BTP**. For more information, see What Is the Consumption-Based Commercial Model? [page 114].

To try out services that participate in the free tier model for SAP BTP, create a service instance using the **Free** service plan offered by the service.

> → Tip
>
> To find out which services offer the free plans, see Discover Free Services [page 119]. Here, you also find information on how to identify the technical limitations of the free plan of each service. Access Free Services [page 120] by assigning entitlements to subaccounts and creating service instances.

Once you reach the technical limits of a service and want to continue using it, you must move to a paid service plan. Upgrading an existing service instance from free tier to paid usually doesn't require creating a new service instance. Find out how to upgrade a service plan at Upgrade to a Paid Service Plan [page 120].

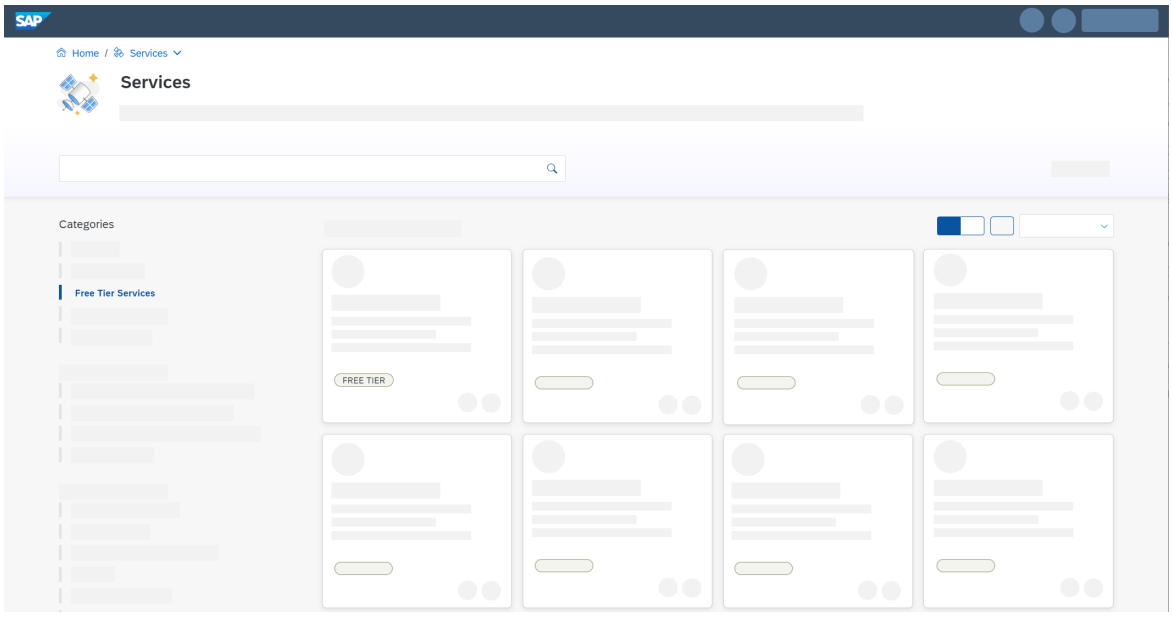
Use of free tier service plans is subject to additional terms and conditions as provided in the Business Technology Platform Supplemental Terms and Conditions. Only community support is available for Free service plans and these aren't subject to SLAs.

Discover Free Services

Find out which services offer Free plans and what the limitations are.

Free Tier services can be explored in the service catalog of SAP Discovery Center. A free tier service appears as a "Free" plan offering for a service.

1. Access the *Service Catalog* in the SAP Discovery Center.
2. In the *Categories* section, select *Free Tier Services*.



3. Select one of the services displayed in the catalog.
4. Navigate to the *Pricing* tab.

On this page, you can see all available plans for this service, including the limitations of each plan. Plans that participate in the free tier model for SAP BTP are labeled **Free**.

Access Free Service Plans

To try out the free tier model for SAP BTP, you must entitle your subaccount for the free plan and create a service instance using this plan.

> ⓘ Note
>
> The free tier model for SAP BTP is available for accounts that use the consumption-based commercial model.

1. Enter your global account.
   For more information, see Access the Cockpit.
2. Configure entitlements for the subaccount in which you want to use the free service plans. Select the service and choose the free plan.
   For more information, see Managing Entitlements and Quotas Using the Cockpit [page 2521].
3. Navigate into the subaccount and go to the *Service Marketplace*.
4. Find the service you want to try out and select it.
   You find a description of the available service plans, including the free service plan. By clicking *More*, you find the scope and limitations of the free plan for this service.
5. Create an instance of the service and select the free service plan.
   For more information, see View and Manage Services from the Service Marketplace.

You can now start using this service.

Update to a Paid Service Plan

Once you've reached the limits of the free plan, you can upgrade the free service plan to a paid plan.

To upgrade, first configure the required entitlements and quotas. Make sure the appropriate paid service plan and quotas are assigned to your subaccount.

> ⓘ Note
>
> For subaccounts that are located under a directory that is configured to manage entitlements, the global account admin must first distribute the required quota to the managed directory. For more information, see Configure Entitlements and Quotas for Subaccounts [page 2529].

> ⓘ Note
>
> The option to upgrade from free tier service plans to paid service plans is not yet available for all services and runtimes, such as Kyma. To use a paid service plan for Kyma runtime, you will need a new Kyma cluster. It is possible to have both - free and paid clusters - in parallel.

> **Note**
>
> It's not possible to switch from a paid plan to a free plan.

1. If the service instance has been created in the Cloud Foundry environment, make sure that you have the Space Developer role for the space of the service instance (otherwise, ignore this step).
    1. In your Cloud Foundry subaccount, navigate to the space, in which you have created your service instance.
    2. Navigate to *Space Members* and check if your user is listed there.
    3. If your user is not listed there, add your user and assign it the Space Developer role. For more information, see **Add Space Members**.
2. In your subaccount, navigate to the *Instances and Subscriptions* page.
3. Find the instance of the service and click ••• *(Actions)* and *Update*.
4. In the *Update Instance* dialog, change the plan to a paid service plan.
5. Click *Update Instance* to save your changes.

You can now use the service without the limitations of the free service plan.

Scope

- The free tier model for SAP BTP is available for CPEA and Pay-As-You-Go for SAP BTP contracts. For more information, see What Is the Consumption-Based Commercial Model? [page 114].
- Services using the free tier service plans run on the same platform as paid services.
- Services are technically limited. This limit depends on the service. You can find a description of the service plans in the *Service Marketplace* in the cockpit or in the SAP Discovery Center.

2.6 Account Model

Learn more about the different types of accounts on SAP BTP and how they relate to each other.

Accounts are structured according to global accounts, subaccounts, and directories.

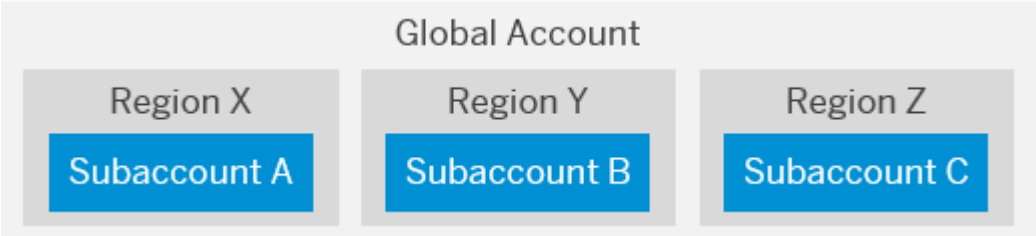
To learn more about managing your account model, see Account Administration [page 2457].

2.6.1 Global Accounts

A **global account** is the realization of a contract you or your company has made with SAP.

A global account is used to manage subaccounts, members, entitlements and quotas. You receive entitlements and quotas to use platform resources per global account and then distribute the entitlements and quotas to the subaccount for actual consumption. There are two types of commercial models for global accounts: consumption-based model and subscription-based model. See Commercial Models [page 113] .

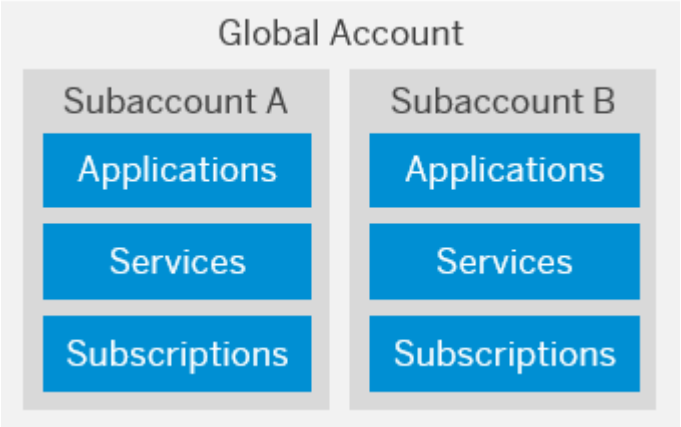
Global accounts are region- and environment-independent. Within a global account, you manage all of your subaccounts, which in turn are specific to one region.



2.6.2 Subaccounts

In **subaccounts**, you deploy applications, use services, and manage your subscriptions. You can use them to manage members, authorizations, and entitlements.

A global account can contain one or more subaccounts in which you deploy applications, use services, and manage your subscriptions. Subaccounts in a global account are independent from each other. This is important to consider with respect to security, member management, data management, data migration, integration, and so on, when you plan your landscape and overall architecture.



Each subaccount is associated with a region, which is the physical location where applications, data, or services are hosted. The specific region is relevant when you deploy applications and access the SAP BTP cockpit using the corresponding cockpit URL. The region assigned to your subaccount doesn't have to be directly related to your location. You could be located in the United States, for example, but operate your subaccount in Europe.

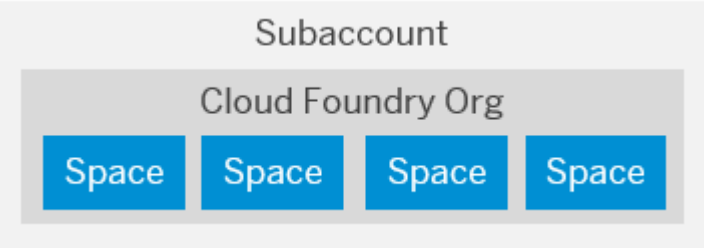
The entitlements and quotas that have been purchased for a global account have to be assigned to the individual subaccounts.

Global accounts and subaccounts are completely independent of user accounts. For more information, see User and Member Management [page 133].

Subaccounts, Cloud Foundry Orgs and Kyma Clusters

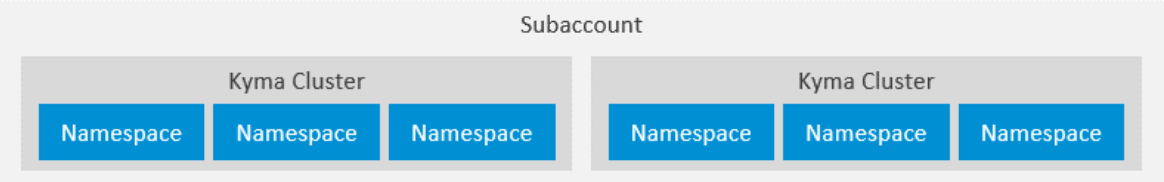
- When you enable the Cloud Foundry environment in one of your subaccounts, the system automatically creates a Cloud Foundry org for you. The subaccount and the org have a 1:1 relationship and the same navigation level in the cockpit (even though they may have different names). You can create spaces within

that Cloud Foundry org. Spaces let you further break down your account model and use services and functions in the Cloud Foundry environment.



For more information about Cloud Foundry orgs and spaces, see the Cloud Foundry documentation at https://docs.cloudfoundry.org/concepts/roles.html .

- When you enable the Kyma environment in one of your subaccounts, you can provision one or more Kubernetes clusters equipped with the latest version of the open-source project "Kyma". The subaccount and the Kyma cluster have a 1:n relationship - you can choose to create more than one cluster right away, or add clusters later.
  Within each Kyma cluster, you can create namespaces to organize your workloads and control access to resources. Namespaces let you further break down your account model and use services and functions in the Kyma environment. For more information, see the Kubernetes documentation at https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/ .

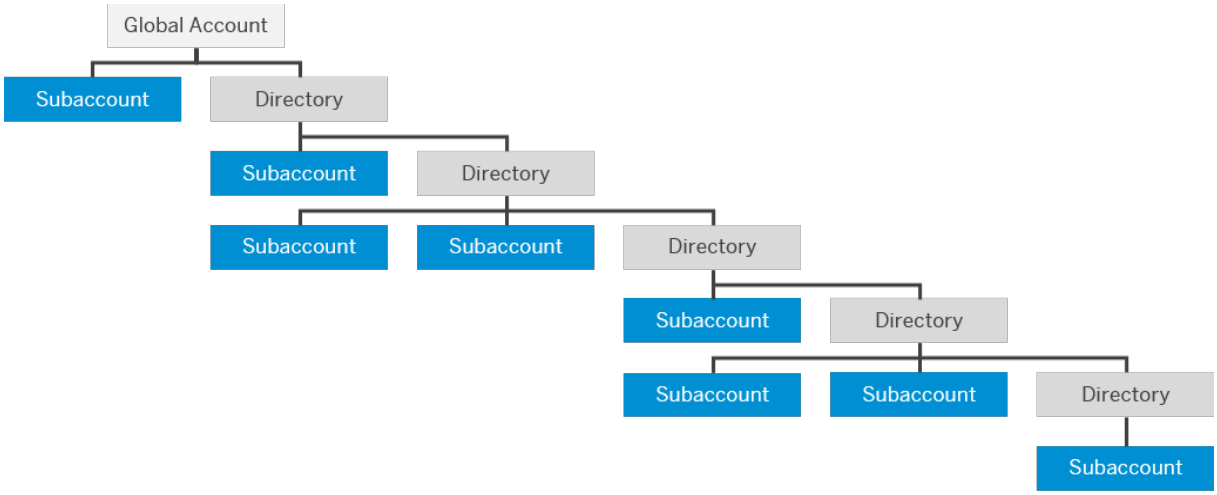


2.6.3 Directories

With **directories**, you can organize and manage your subaccounts according to your technical and business needs.

A directory can contain directories and subaccounts to create a hierarchy. Using directories to group other directories and subaccounts is optional - you can still create subaccounts directly under your global account.

You can create a hierarchical structure that is up to 7 levels deep. The highest level of a given path is always the global account and the lowest is a subaccount, which means that you can have up to 5 levels of directories between the global account and the lowest level subaccount.



Directories enable you to:

- Group and filter directories and subaccounts
- Monitor usage and costs for contracts that use the consumption-based commercial model

Optionally, you can also enable the following features in your directories:

- Manage Entitlements: Enables the assignment of a quota for services and applications to the directory from the global account quota, for distribution to the directory's subaccounts.
  When you assign entitlements to a directory, you express the entitlements and maximum quota that can be distributed across its children subaccounts. You also have the option to choose the auto-assignment of a set amount of quota to all subaccounts created or moved to that directory. Subaccounts that are already in the directory when you select that option will not be auto-assigned quota.

  > ### Note
  >
  > If you've enabled the Manage Entitlements feature for a given directory, you must first assign the necessary entitlements and maximum allowed quota from the global account to that directory. Then you can distribute this "reserved" quota to any of the directory's child subaccounts.

- Manage Authorizations: Enables authorization management for the directory. For example, it allows certain users to manage directory entitlements. You can only use this feature in combination with the *Manage Entitlements* feature.

Related Information

Getting a Global Account [page 162]
Setting Up Your Account Model
Managing Global Accounts Using the Cockpit [page 2464]
Managing Directories Using the Cockpit [page 2501]
Managing Subaccounts Using the Cockpit [page 2509]
Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
Account Administration Using APIs of the SAP Cloud Management Service [page 2730]
Manage the Account Explorer Hierarchy [page 2504]

Configure Entitlements and Quotas for Directories [page 2524]
Configure Entitlements and Quotas for Subaccounts [page 2529]

2.6.4 Relationship Between Global Accounts, Subaccounts, and Directories

A global account can group together different directories and subaccounts that an administrator makes available to users. Administrators can assign the available entitlements and quotas of a global account to its different subaccounts and move it between subaccounts that belong to the same global account.

The hierarchical structure of global accounts, directories, and subaccounts lets you define an account model that accurately fits your business and development needs. For example, if you want to separate development, testing, and productive usage for different departments in your organization, you can create a directory for each department, and within each directory, you group subaccounts for development, testing, and production.

2.6.5 Labels

Labels are user-defined words or phrases that you can assign to various entities in SAP BTP to categorize them in your global account, to identify them more easily.

For example, in the *Account Explorer* page in the cockpit, you can quickly filter for directories and subaccounts by label.

You can assign labels to these entities when you create or edit them using the SAP BTP cockpit, command line interface (btp CLI), or REST APIs:

- Directories
- Subaccounts
- Multitenant application subscriptions
- Service instances
- Environment instances

> **Note**
>
> For environment instances, these custom labels are user defined and apply only to SAP BTP. They are not the same labels that might be defined by your environment broker.

Labels are made up of a *label name* (also referred to as a *key*) and up to 10 *values* associated with the label. You can apply label names and values in any way that suits your business and technical needs.

Types of labels and examples

Types of Labels	Examples
Single-value labels are useful for labels that have an identifier, or for labels with fixed lists.	• Label Name: **`Cost Object`** Value: The ID of a cost center number or internal order that is associated with the entity, such as: **`000001134789`** • Label Name: **`Status`** Value: **`Active`** or **`Inactive`** • Label Name: **`Landscape`** Value: **`Dev`** or **`Test`** or **`Production`**
Multi-value labels make them useful for labels that typically have more than one value assigned to them.	• Label Name: **`Contacts`** Value: The e-mail addresses or names of users that are responsible for the entity, such as: **`admin-1@example.com`** and **`admin-2@example.com`**
Labels can also be assigned with a name only and no value. In such cases, the label behaves like tag. Note that currently for service instances, labels must have at least value.	• Label Name: **`For demo only`** • Label Name: **`Audited`** • Label Name: **`Flagged for deletion`**

> **Note**
>
> Labels replace what were previously called "custom properties". Custom properties supported only single values per label and were available only to directories and subaccounts. As a result of the move to labels, all relevant commands in the SAP BTP command line interface (btp CLI) and in the relevant REST APIs in the SAP Cloud Management service have been updated accordingly. The `custom-properties` parameter in the btp CLI and the `customProperties` field in the relevant REST APIs are deprecated.

You can assign labels when you create an entity, and then later add, change, or remove labels by editing the entity.

> **→ Tip**
>
> - In the *Account Explorer* and *Instances and Subscriptions* pages in the SAP BTP cockpit, assigned labels are shown in the *Labels* column. To display the column if it is not shown, click ⚙ *(Configure Table Columns)*.
> - In the *Account Explorer* page, you can view the labels that are assigned to a directory or subaccount by choosing the *More Info* option of each directory and subaccount.
> Assigned labels are also listed under the *Labels* tab when you display the *Overview* page of every directory and subaccount.
> - In the *Account Explorer* and *Instances and Subscriptions* pages, you can filter the displayed entities by their assigned labels in the *Search* field.
> - In the *Instances and Subscriptions* page, you can also view the labels that are assigned to a subscription or instance by expanding its details panel.

When working with labels, consider the following aspects:

- In the cockpit, each entity can have up to 10 labels assigned to it.
- You cannot add the same label name more than once to the same entity.
- Existing label names and values are offered as suggestions when you or anyone else assigns a label to other entities of the same type in your global account.
  When you view a subaccount in the cockpit, the subaccount also shows the labels that are assigned to its parent directory and to other directories that are above it in the same path in your account structure. And in the *Account Explorer*, when you filter by labels that are assigned to a directory, the subaccounts in that directory path are also listed. We refer to these as inherited labels.
- Label names and values are case-sensitive, which means you can create variants of the same label name with a different casing; for example, `My Label` and `My label` can coexist as separate labels. We recommend that you avoid using different casing or styling to create variants of the same names or values.

Related Links

- Create a Subaccount [page 2509]
- Change Subaccount Details [page 2516]
- Create a Directory [page 2502]
- Commands in the btp CLI [page 2698]
- Account Administration Using APIs of the SAP Cloud Management Service [page 2730]
- Managing Service Resources Using the APIs of the SAP Service Manager [page 2777]
- Creating Service Instances
- Working with Environment Instances
- Subscribe to Multitenant Applications Using the Cockpit [page 2534]

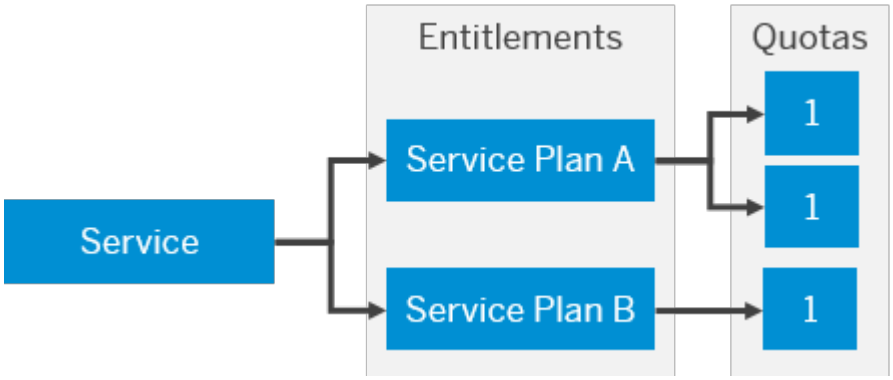
2.7 Entitlements and Quotas

When you purchase an enterprise account, you're entitled to use a specific set of resources, such as the amount of memory that can be allocated to your applications.

Services and Service Plans

On SAP BTP, all external dependencies such as databases, messaging systems, files systems, and so on, are **services**. In this context, multitenant applications and environments are considered services.

Each service has one or more **service plans** available. A service plan is the representation of the costs and benefits for a given variant of a particular service. For instance, a database may be configured with various "T-shirt sizes", each of which is a different service plan.



Entitlements

An **entitlement** is your right to provision and consume a resource. In other words, entitlements are **the service plans** that you're entitled to use.

The entitlements that are available in your enterprise account are determined by the commercial model used by your global account:

- In the consumption-based commercial model, your organization buys an entitlement to all current and future SAP BTP services eligible under this model. During your contract period, you have full flexibility to switch services on and off and change between them as your business needs. For more information, see What Is the Consumption-Based Commercial Model? [page 114].
- In the subscription-based commercial model, your organization is authorized to use only the services you have subscribed to, as per your commercial agreement. To access additional services at an extra cost, you can modify your contract through your sales representative or account executive. For more information, see What Is the Subscription-Based Commercial Model? [page 117].

Quotas

A **quota** represents the numeric quantity of a service plan that you're entitled to consume in your global account and its subaccounts.

From the perspective of your SAP BTP global account contract, the global quota of a service plan can be defined as either fixed or unlimited:

- **Fixed quota:** The quota or allowance of the service plan is the upper limit at which the plan can be consumed collectively across your global account. This is typical of global accounts that have the **subscription-based** commercial model agreement.
- **Unlimited quota:** There is no upper limit to how much the service plan can be used collectively across your global account. This is typical of most services that are eligible to global accounts with a **consumption-based** commercial model agreement (Cloud Platform Enterprise Agreement (CPEA) and Pay-As-You-Go for SAP BTP). At the end of the month, the number of units used determines how much you are charged.

Regardless of whether the global quota of a plan on the global account level is fixed or unlimited, the specifics and properties of each plan determines how its quota can be distributed by the global account admin to the subaccounts in your global account. There are two types of quota assignments:

- **Numeric assignments:** The global account admin assigns the plan to a subaccount and specifies the maximum quantity of units from the global quota that can be consumed by the subaccount. Admins can use this assignment to limit users from exceeding a specific quota for cost-controlling purposes. Typically, but not always, the assigned quota is directly related to the number of instances that can be created in subaccounts for the assigned plan.
- **Non-numeric assignments:** The global account admin does not assign a specific quantity but simply grants access to the specific plan by assigning the plan to the subaccount. When the plan is assigned, it's available for use by the subaccount. SaaS application assignments are typically non-numeric, whereby a subaccount can either subscribe to the app or not. For services that fall into this category of non-numeric assignments, there is usually no limit to the number of instances that can be created.

See also the sections below and Managing Entitlements and Quotas Using the Cockpit [page 2521].

Distribution and Usage of Entitlements and Quotas

Entitlements and quotas are purchased and managed at global account level, from where they're distributed to subaccounts, which consume them.

> → Tip
>
> You can also distribute entitlements to directories, and then redistribute to the subaccounts under the directories; however, this is optional. By default, directories are not enabled to manage entitlements. This feature needs to be enabled per directory by the global account administrator.

When assigning entitlements and quotas to directories, you also have the option to automatically assign a set amount of quota to each new subaccount added to the directory. This option doesn't apply to subaccounts that are already in the directory when you select this option. The quota you assigned to the directory is then gradually distributed to all subaccounts that you add to that directory, until it runs out. Once the directory quota runs out, if you add a new subaccount to that directory it won't get any quota automatically anymore.

Since directories are only a way of grouping subaccounts, you can't consume a service at directory level. However, when you assign entitlements and quotas from the global account to a directory, the quota you assigned is shown as used, even if there are no subaccounts in that directory to consume the quota. You can think of it as a way to "reserve" quota and make sure it's not assigned to other subaccounts or directories.

When you remove quotas or assignments from a directory or subaccount, they become available again at global account level and can be assigned to other directories or subaccounts; unless the quota is reserved for a given directory then the freed quota remains available only to that directory and its subaccounts.

> ⓘ Note
>
> Before a subaccount admin can enable a quota-based environment, such as Kyma, the subaccount admin must first assign the environment as an entitlement to the subaccount. Other environments, such as Cloud Foundry, are available by default to all subaccounts, and therefore are not available as entitlements.

For more information, see:

- Configure Entitlements and Quotas for Subaccounts [page 2529]
- Configure Entitlements and Quotas for Directories [page 2524]

Quota Plans

In the Cloud Foundry environment, you can further distribute the quotas that are allocated to a subaccount. This is done by creating space quota plans and assigning them to the spaces.

Space quota plans are optional and are used to limit how much each space can use, not to enable. If you don't create any space quota plans, all spaces in a subaccount using the Cloud Foundry environment have access to all entitlements and quotas in that subaccount. This means that one space could use up all the quota in a subaccount. If you want to prevent that from happening and set a limit to how much each space can use from the total quota available in the subaccount, you can use quota plans and assign them to spaces.

For more information on space quota plans in the Cloud Foundry environment, see:

- https://docs.cloudfoundry.org/adminguide/quota-plans.html
- Managing Space Quotas [page 2806]

Resource Providers

SAP BTP allows you to connect your global account in the SAP BTP cockpit to your provider account from a non-SAP cloud vendor, and consume remote service resources that you already own and which are supported by SAP through this channel.

For example, if you're subscribed to Amazon Web Services (AWS) and have already purchased services, such as PostgreSQL, you can register the vendor as a resource provider in SAP BTP and consume this service across your subaccounts together with other services offered by SAP.

SAP BTP currently supports the following vendors and their consumable services:

Cloud Vendor	Supported Services
Amazon Web Services	Amazon Relational Database Service (RDS) - PostgreSQL
Microsoft Azure	Azure Database for PostgreSQL.

After you configure a new resource provider, its supported services are added as entitlements in your global account. In the *Entitlements* page in the cockpit, you can then allocate the required services and quotas to the relevant directories and subaccounts in your global account.

For more information, see Managing Resource Providers [page 2468].

Related Information

Managing Entitlements and Quotas Using the Cockpit [page 2521]

Tutorial: Manage Entitlements on SAP BTP Trial

Troubleshooting: Entitlement and Quota Problems

2.8 Introduction to Identity and Access Management on SAP BTP

Identity and Access Management (IAM) is a fundamental capability of SAP Business Technology Platform that controls how you access applications and the platform itself. IAM ensures that the right users have access to the right systems with the appropriate authorizations.

When you work with SAP BTP, understanding Identity and Access Management is essential for securing your landscape and managing user access effectively. The IAM process for users begins in the system of origin where user records are created, continues with granting authorizations to those records, and ensures secure authentication when you access SAP BTP.

Cloud Identity Services: Your Central IAM Component

Cloud Identity Services play a central role in the IAM reference architecture for SAP software. By establishing these services as a core IAM component, SAP ensures that SAP cloud applications offer a consistent set of integration capabilities for identity lifecycle management and single sign-on. You can also continue using your existing third-party or legacy components, such as a central user store.

Cloud Identity Services are the default integration component for SAP cloud solutions, including SAP BTP. You can use them to protect access to the platform itself and to applications running on SAP BTP. As an SAP BTP customer, you're entitled to at least two tenants of Cloud Identity Services, one for testing and one for production. We recommend using these tenants instead of the default identity provider. If you're using an SAP BTP trial account, you receive one trial tenant.

The Four Components of Cloud Identity Services

Cloud Identity Services consist of four key components that work together to provide comprehensive identity and access management.

Identity Authentication

Identity Authentication provides central capabilities for authentication and single sign-on (SSO). It offers convenient user self-services such as registration and password resetting for employees and partners. Security features include protecting access to applications, defining risk-based authentication rules, using two-factor authentication, and delegating authentication to other identity providers.

Some key capabilities:

- Secure authentication for browser-based applications

- SSO and single logout functionality from anywhere on any device
- Strong authentication with configurable multi-factor authentication enforcement
- Risk-based authentication applied to applications, user group assignments, and IP address ranges
- Support for SAP and third-party applications
- Password policies at the service-provider application level
- Customizable look and feel, including company branding
- User self-services, including registration and password resetting
- Delegated authentication through integration with corporate identity providers and on-premises user stores
- Identity federation with corporate identity providers and applications based on SAML 2.0 and OpenID Connect

Identity Provisioning

Identity Provisioning is a cloud service for managing identity lifecycle processes. It automates these processes and helps you provision identities and related authorization assignments to various cloud and on-premises business applications.

Some key capabilities:

- Rapid scenario extensions using optimized connectors for provisioning users and groups among multiple supported SAP and non-SAP cloud and on-premises systems
- Connectors for source, target, or proxy systems in your provisioning scenario, including connectors for SAP BTP global accounts, directories, subaccounts, for the Cloud Foundry and ABAP environments
- Flexible data transformations, where you can customize connectors using either the graphical editor or the JSON editor
- Automatic delivery of default provisioning systems for specific SAP cloud solutions
- Real-time provisioning with dedicated source systems
- Consumability either directly through built-in APIs or from the user interface, which is integrated into the administration console in Cloud Identity Services
- SCIM-compliant integration with identity management solutions
- Support for cloud and on-premises user stores as sources, relying on dedicated integration with local identity stores
- Comprehensive job scheduling for provisioning processes

Identity Directory

Identity Directory is the persistency layer inside Cloud Identity Services. This component facilitates identity lifecycle flows and enables important features, such as a central user store for newer SAP BTP applications, which helps you avoid replicating users to every single application.

Identity Directory acts as the central point of truth regarding users of SAP cloud solutions. Users are typically replicated into Identity Directory from your company's user source and central point of truth, either an HR system or a corporate identity management solution.

Some key capabilities:

- SCIM 2.0 REST API for management of users, groups, and custom schemas
- Accessibility from the Cloud Identity Services admin console or through its SCIM 2.0 REST API or Identity Provisioning services

- Custom schema creation
- Generation of global user IDs
- Integration with the Authorization Management service and various SAP solutions that rely on user persistence in Identity Directory

Authorization Management

Authorization Management Service (AMS) enables you as an Cloud Identity Services administrator to customize authorization policies of cloud applications and assign them to users.

Application developers declare the detailed authorization model of their application. They group related authorizations into predefined authorization policies (base policies), which serve as a foundation for you to design custom policies.

Some key capabilities:

- Authorization design and assignment in the same user interface
- Powerful instance-based authorizations (through configuration of restrictions in the authorization policies)
- Easy maintenance of restrictions through value help based on application data
- Client libraries for Java, Node.js, and Go
- Tight integration into the Cloud Application Programming model (CAP), which supports fully declarative authorization enforcement in typical use cases

Using Cloud Identity Services with SAP BTP

Cloud Identity Services are the default integration component for SAP cloud solutions and for SAP BTP. You can use them to protect access to the platform, the SAP BTP cockpit itself, or the applications running on top of the platform. For more information about Cloud Identity Services, refer to the Cloud Identity Services documentation or explore the learning journey.

2.9 User and Member Management

On SAP BTP, user management takes place at all levels from global account to environment. There are different types of users, such as depending on their roles in the company.

User Accounts

A user account corresponds to a particular user in an identity provider. The user is always stored in an external identity provider, such as a custom tenant of SAP Cloud Identity Services - Identity Authentication or the default identity provider.

→ Recommendation

We strongly recommend that you configure your custom tenant of SAP Cloud Identity Services as the identity provider and connect SAP Cloud Identity Services to your own corporate identity provider.

For more information, see Trust and Federation with Identity Providers [page 2538].

User accounts enable users to log on to SAP BTP, access subaccounts, and to use applications according to the permissions granted to them.

ⓘ Note

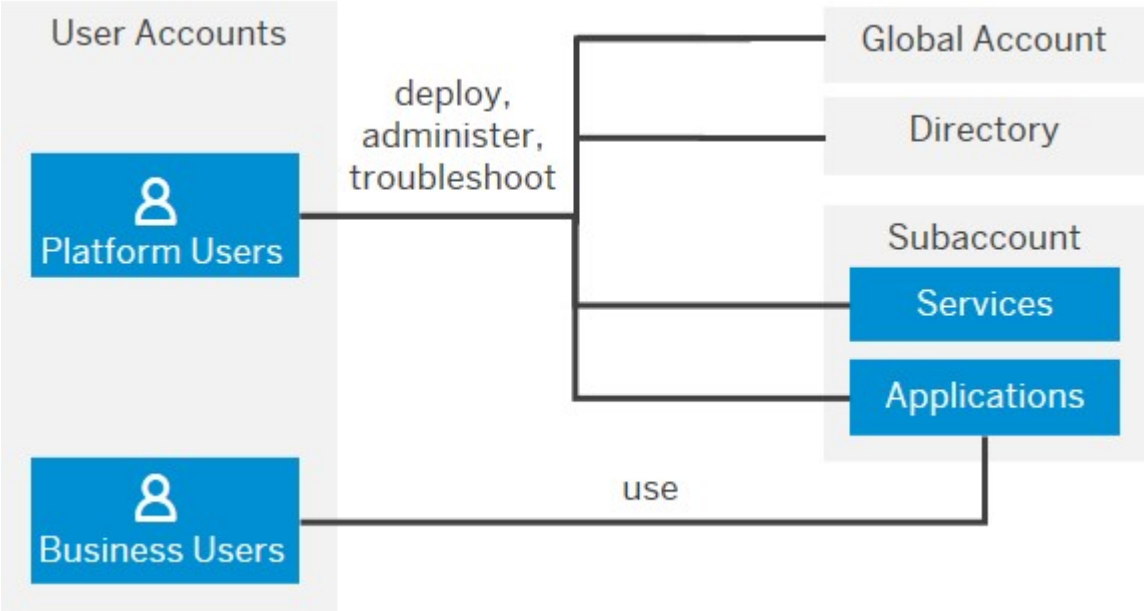
A user name alone doesn't determine a concrete user account with associated authorizations, as you can have users with the same user name in different identity providers. Accessible data and allowed operations also depend on the identity provider. The concrete user is identified by the combination of user name and identity provider.

Example

There are two users with the same user name, which is the email address here. The two users with different identity providers have different authorizations and can access different applications.

- julia.moore@acme.com from the custom identity provider has authorizations to access her favorite industrial applications. She needs the logon with the custom identity provider for her actual work.
- julia.moore@acme.com from the default identity provider has no authorizations.

Before diving into the different user and member management concepts, it's important to understand the difference between the two types of users we're referring to: **platform users** and **business users**.



For more information, see Platform Users [page 135] and Business Users [page 149].

Exceptions

There are some exceptions from the separation of platform and business users. Typical cases are the ABAP environment, which doesn't distinguish between these two types, and SAP Build Code or SAP Build applications, which developers currently access as business users.

ABAP Environment

Typical cases are the ABAP environment, which doesn't distinguish between these two types, and SAP Build Code or SAP Build applications, which developers currently access as business users.

The user and authorization concept used in the ABAP environment is completely decoupled and independent from the authorization concept of SAP BTP or within Cloud Foundry. The ABAP environment runs within the SAP BTP, Cloud Foundry environment. Therefore, administrators or developers need the right authorizations in Cloud Foundry to create the ABAP environment or other services, which get consumed in an application running in the ABAP environment.

For more information, see Administration and Operations in the ABAP Environment [page 2914].

Kyma Environment

To enable a Kyma environment, you need the subaccount administrator role collection (see Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138]). All other authorizations and trust are managed in the Kyma cluster.

Kyma uses Kubernetes Role-Based Access Control (RBAC) and assures during provisioning that a user who creates and owns a particular runtime is given the cluster-admin role. Users with the cluster-admin role can define any additional cluster roles or use those defined in Kyma and bind them to other users from Kyma dashboard or with the kubectl CLI tool. See Assign Roles in the Kyma Environment [page 3666]. For recommendations on setting up roles and permissions in Kyma, see Authorization in the Kyma Environment.

Related Information

Working with Users [page 2603]
Roles and Role Collections [page 2624]
Attributes [page 2628]
Trust and Federation with Identity Providers [page 2538]

2.9.1 Platform Users

Platform users are usually developers, administrators or operators who deploy, administer, and troubleshoot accounts, applications and services on SAP BTP. They also manage entitlements and environments. They're the users that have full access and give certain permissions, for instance, at global account, directory, or subaccount level. Due to their powerful authorizations, we recommend that you protect them with strong authentication (for example, two-factor authentication). Members of subaccounts only have basic access.

Platform users who have administrative permissions can view or manage the list of global accounts, subaccounts, and environments, such as Cloud Foundry orgs and spaces. Platform users have access to

them using the SAP BTP cockpit, infrastructure as code, the SAP BTP command-line interface (BTP CLI), or environment-specific CLI, such as the Cloud Foundry (CF) CLI. Using Joule is bound to the SAP BTP cockpit authorization.

For platform users, there's a default identity provider [page 2594]. We expect that you have your own identity provider.

> → Recommendation
>
> We strongly recommend that you configure your custom tenant of SAP Cloud Identity Services as the identity provider and connect SAP Cloud Identity Services to your own corporate identity provider.
>
> For more information, see Trust and Federation with Identity Providers [page 2538].

Member Management

Member management refers to managing permissions for platform users. Members have only basic access to SAP BTP.

Member management happens at global account, directory, subaccount, and environment level. Members' permissions apply to all operations that are associated with the global account, the organization, or the space, irrespective of the tool used. Depending on the scope and the cloud management tools you're using, you manage members in different ways:

Global Accounts	Directories	Subaccounts
You manage global account members by assigning role collections to platform users. Use the following predefined role collections: • Global Account Administrator • Global Account Viewer Assign these role collections from the SAP BTP cockpit or the BTP CLI. See: Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138] Add Members to Your Global Account [page 2467] Create Users [page 2608]	You manage directory members by assigning role collections to platform users. Use the following predefined role collections: • Directory Administrator • Directory Viewer Assign these role collections from the SAP BTP cockpit or the BTP CLI. See: Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138] Create Users [page 2608] Manage Users in Directories [page 2505]	You manage subaccount members by assigning role collections to platform users. **Note** Neo subaccounts don't use role collections. For more information, see Managing Member Authorizations in the Neo Environment. Use the predefined role collections, such as: • Subaccount Administrator • Subaccount Viewer Assign these role collections from the SAP BTP cockpit or the BTP CLI. See: Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138] Add Members to Your Subaccount [page 2513] Create Users [page 2608]

Member Management in the Cloud Foundry Environment

Orgs	Spaces
Manage org members on the *Org Members* page at environment level in the subaccount using the SAP BTP cockpit or with the Cloud Foundry CLI. A platform user added as an org member can be either an **Org Manager** or an **Org Auditor** or implicitly an **Org User**. See: Org Administration Using the Cockpit [page 2792] About Roles in the Cloud Foundry Environment [page 2788] https://docs.cloudfoundry.org/concepts/roles.html#roles Add Org Members [page 2795]	Manage space members on the *Org Members* page at space level in the SAP BTP cockpit or with the Cloud Foundry CLI. A platform user added as a space member can be either a **Space Manager**, **Space Developer**, **Space Auditor**, or **Space Supporter**. See: About Roles in the Cloud Foundry Environment [page 2788] https://docs.cloudfoundry.org/concepts/roles.html#roles Add Space Members [page 2803]

For more information on the Kyma environment, see Assign Roles in the Kyma Environment [page 3666].

See also About User Management in the Cloud Foundry Environment [page 2787].

2.9.1.1 Role Collections and Roles in Global Accounts, Directories, and Subaccounts

SAP BTP provides a set of role collections to set up administrator access to your global account and subaccounts.

Role collections group authorizations for resources and services. Your administrators assign these role collections to other platform users to create new administrators. Role collections consist of individual roles. For more information on role collections, roles, see the related link.

Role collections are account-specific. Role collections that exist in the global account don't exist in the subaccounts. Likewise, role collections in subaccounts aren't available in the global account.

> **Note**
>
> Neo subaccounts don't use role collections.
>
> For more information, see Managing Member Authorizations in the Neo Environment.

Role Collections

You can use the default role collections, but you can't change or delete them. SAP BTP provides the following administrator role collections:

- *Global Account Administrator*
- *Subaccount Administrator*
- *Directory Administrator*
- *Cloud Connector Administrator*
- *Connectivity and Destination Administrator*
- *Destination Administrator*
- *Destination Fragment Administrator*
- *Subaccount Service Administrator*

SAP BTP provides also viewer role collections for the global account and for subaccounts. In contrast to the administrator role collections, viewer role collections only grant read access.

- *Global Account Viewer*
- *Subaccount Viewer*
- *Directory Viewer*

Administrator Role Collections

If you assign the *Global Account Administrator* role collection to a user, this user can perform administration tasks for subaccounts, role collections, identity providers, entitlements, and regions on the level of the global account. If you assign the *Global Account Viewer* role collection, this user can view subaccounts, role collections, identity providers, entitlements, and regions on the level of the global account.

> ### Note
>
> You can also use the command-line interface (CLI) for SAP BTP to assign authorizations. For more information on managing authorizations, see the related link.

Global Account Administrator Role Collection

Roles Included	Description
Global Account Admin	Role for global account members with read-write authorizations for core commercialization operations, such as updating global accounts, setting entitlements, and creating, updating, and deleting subaccounts. **Note** Users with this authorization don't automatically gain administrator authorizations in all subaccounts, but can grant themselves administrator-level access. You can opt out of this feature. For more information, see Add Members to Your Subaccount [page 2513]. The *GlobalAccount_Admin* role template contains this role. You find the role template in the SAP BTP Cockpit if you choose the *cis-central!*`<suffix>` application identifier. **Example** *cis-central!b1*
Global Account Usage Reporting Viewer	Role for global account members with read-only authorizations for core commercialization operations, such as viewing global account usage information. The *GlobalAccount_Usage_Reporting_Viewer* role template provides this role. You find the role template in the SAP BTP Cockpit if you chose the *uas!*`<suffix>` application identifier

Roles Included	Description
User and Role Administrator	Manage authorizations, trusted identity providers, and users. ⚠ : Includes security authorizations. See *Scopes with Critical Security Authorizations* under Scopes. The *xsuaa_admin* role template provides this role. You find the role template in the SAP BTP Cockpit if you choose the *xsuaa!*`<suffix>` application identifier. **Example** *xsuaa!tl*
Auditlog_Auditor	Read access to audit logs
Auditlog_Auditor	View audit logs
System Landscape Administrator	Administrative access to systems and scenario-related resources. The *GlobalAccount_System_Landscape_Administrator* role template provides this role. You find the role template in the SAP BTP Cockpit if you choose the *cmp!*`<suffix>` application identifier. **Example** *cmp!b7*

If you assign the *Subaccount Administrator* role collection to a user, you grant a user administration permission for a subaccount.

Subaccount Administrator Role Collection

Roles Included	Description
Auditlog_Auditor	Read access to audit logs
Auditlog_Auditor	View audit logs
Cloud Connector Administrator	Operate the data transmission tunnels used by the Cloud connector.
Destination Administrator	Manage destination configurations, certificates and subaccount trust via the Destination editor in the SAP BTP cockpit.
Destination Fragment Administrator	Manage destination fragments

Roles Included	Description
Subaccount Admin	Role for subaccount members with read-write authorizations for core commercialization operations, such as viewing subaccount entitlements, and creating and deleting environment instances.
Subaccount Service Administrator	Administrative access to service brokers and environments on a subaccount level.
User and Role Administrator	Manage authorizations, trusted identity providers, and users. ⚠ : Includes security authorizations. See *Scopes with Critical Security Authorizations* under Scopes.

If you assign the *Cloud Connector Administrator* role collection to a user, you grant the user administration permissions for the Cloud Connector in a subaccount.

Cloud Connector Administrator Role Collection

Roles Included	Description
Cloud Connector Administrator	Operate the data transmission tunnels used by the Cloud connector.

If you assign the *Connectivity and Destination Administrator* role collection to a user, you grant the user administration permissions for the Cloud Connector and SAP Destination service in a subaccount.

Connectivity and Destination Administrator Role Collection

Roles Included	Description
Cloud Connector Administrator	Operate the data transmission tunnels used by the Cloud connector.
Destination Administrator	Manage destination configurations, certificates and subaccount trust via the Destination editor in the SAP BTP cockpit.

If you assign the *Destination Administrator* role collection to a user, you grant the user administration permissions for the SAP Destination service in a subaccount.

Destination Administrator Role Collection

Roles Included	Description
Destination Administrator	Manage destination configurations, certificates and subaccount trust via the Destination editor in the SAP BTP cockpit.

If you assign the *Destination Fragment Administrator* role collection to a user, you grant the user administration permissions for the management of destination fragments for the SAP Destination service in a subaccount.

Destination Fragment Administrator Role Collection

Roles Included	Description
Destination Fragment Administrator	Manage destination fragments

If you assign the *Subaccount Service Administrator* role collection to a user, you grant the user administration permissions for the Service Manager in a subaccount.

Subaccount Service Administrator Role Collection

Roles Included	Description
Subaccount Service Administrator	Administrative access to service brokers and environments on a subaccount level.

Viewer Role Collections

If you assign the *Global Account Viewer* role collection to a user, you grant read access to the same information as the Global Account Administrator role collection.

Global Account Viewer Role Collection

Roles Included	Description
Global Account Viewer	Role for global account members with read-only authorizations for core commercialization operations, such as viewing global accounts, subaccounts, entitlements, and regions.
Global Account Usage Reporting Viewer	Role for global account members with read-only authorizations for core commercialization operations, such as viewing global account usage information.
Auditlog_Auditor	Read access to audit logs
Auditlog_Auditor	View audit logs
User and Role Auditor	Read-only access for authorizations, trusted identity providers, and users. ⚠ : Includes security authorizations. See *Scopes with Critical Security Authorizations* under Scopes.
System Landscape Viewer	Viewer access to systems and scenario-related resources.

If you assign the *Subaccount Viewer* role collection to a user, you restrict a user's viewer permission to the subaccounts.

Subaccount Viewer Role Collection

Roles Included	Description
Auditlog_Auditor	Read access to audit logs

Roles Included	Description
Auditlog_Auditor	View audit logs
Cloud Connector Auditor	View the data transmission tunnels used by the Cloud connector to communicate with back-end systems.
Destination Fragment Viewer	View destination fragments
Destination Viewer	View destination configurations, certificates and subaccount trust via the Destination editor in the SAP BTP cockpit.
Subaccount Service Auditor	Read-only access to service brokers and environments on a subaccount level
Subaccount Viewer	Role for subaccount members with read-only authorizations for core commercialization operations, such as viewing subaccount entitlements, details of environment instances, and job results.
User and Role Auditor	Read-only access for authorizations, trusted identity providers, and users. ⚠ : Includes security authorizations. See *Scopes with Critical Security Authorizations* under Scopes.

> **Note**
>
> To obtain permissions to view the service binding credentials, ask an administrator to create a new role collection that includes the `Subaccount Viewer` role collection and the `Service Credentials Viewer` role (see Define a Role Collection [page 2614]).

Directory Role Collections

The role collections *Directory Administrator* and *Directory Viewer* can be assigned after the creation of a directory. If you select the *Enable user management* checkbox, you receive the *Directory Administrator* role collection by default. You can't create custom role collections for directories.

The *Directory Administrator* role collection grants a user administration permission for directories.

Directory Administrator Role Collection

Roles Included	Description
Directory Admin	Role for directory members with read-write authorizations for core commercialization operations, such as updating directories, setting entitlements, and creating, updating, and deleting subaccounts.

Roles Included	Description
Directory Usage Reporting Viewer	Role for directory members with read-only authorizations for core commercialization operations, such as viewing directory usage information.
User and Role Administrator	Manage authorizations, trusted identity providers, and users. ⚠ : Includes security authorizations. See *Scopes with Critical Security Authorizations* under Scopes.

The *Directory Viewer* role collection grants a user read access to the same information as the Directory Administrator role collection.

Directory Viewer Role Collection

Roles Included	Description
Directory Usage Reporting Viewer	Role for directory members with read-only authorizations for core commercialization operations, such as viewing directory usage information.
Directory Viewer	Role for directory members with read-only authorizations for core commercialization operations, such as viewing directories, subaccounts, entitlements, and regions.
User and Role Auditor	Read-only access for authorizations, trusted identity providers, and users. ⚠ : Includes security authorizations. See *Scopes with Critical Security Authorizations* under Scopes.

Roles

Create your own role collections by using the roles of the default role collections. The roles are based on role templates, which are provided by applications. The application identifier refers to the application, which provides the role templates.

> ### ✤ Example
>
> The *GlobalAccount_Admin* role template provides the *Global Account Admin* role. You find the role template in the cockpit if you choose the *cis-central!*`<suffix>` application identifier.
>
> > ### ✤ Example
> >
> > `cis-central!b1`

The following table provides the information about the roles that are available.

Role Details

Roles	Available in	Role Templates	Application Identifier
Application Subscriptions Viewer	Subaccount	Application_Subscriptions_Viewer	cis-local!<suffix>
Application Destination Administrator	Subaccount	Application_Destination_Administrator	destination-xsappname!<suffix>
Application Destination Viewer	Subaccount	Application_Destination_Viewer	destination-xsappname!<suffix>
Auditlog_Auditor	Global account and subaccount	Auditlog_Auditor	auditlog-management!<suffix>
Auditlog_Auditor	Global account and subaccount	Auditlog_Auditor	auditlog-viewer-initial-content!<suffix>
Cloud Connector Administrator	Subaccount	Cloud_Connector_Administrator	connectivity!<suffix>
Cloud Connector Auditor	Subaccount	Cloud_Connector_Auditor	connectivity!<suffix>
Destination Administrator	Subaccount	Destination_Administrator	destination-xsappname!<suffix>
Destination Administrator Instance	Subaccount	Destination_Administrator_Instance	destination-xsappname!<suffix>
Destination Certificate Administrator	Subaccount	Destination_Certificate_Administrator	destination-xsappname!<suffix>
Destination Certificate Instance Administrator	Subaccount	Destination_Certificate_Instance_Administrator	destination-xsappname!<suffix>
Destination Certificate Instance Viewer	Subaccount	Destination_Certificate_Instance_Viewer	destination-xsappname!<suffix>
Destination Certificate Viewer	Subaccount	Destination_Certificate_Viewer	destination-xsappname!<suffix>
Destination Configuration Administrator	Subaccount	Destination_Configuration_Administrator	destination-xsappname!<suffix>
Destination Configuration Instance Administrator	Subaccount	Destination_Configuration_Instance_Administrator	destination-xsappname!<suffix>
Destination Configuration Instance Viewer	Subaccount	Destination_Configuration_Instance_Viewer	destination-xsappname!<suffix>
Destination Configuration Viewer	Subaccount	Destination_Configuration_Viewer	destination-xsappname!<suffix>

Roles	Available in	Role Templates	Application Identifier
Destination Fragment Administrator	Subaccount	Destination_Fragment_Administrator	destination-xsappname!<suffix>
Destination Fragment All Properties Administrator	Subaccount	Destination_Fragment_All_Properties_Administrator	destination-xsappname!<suffix>
Destination Fragment Instance Administrator	Subaccount	Destination_Fragment_Instance_Administrator	destination-xsappname!<suffix>
Destination Fragment Instance Viewer	Subaccount	Destination_Fragment_Instance_Viewer	destination-xsappname!<suffix>
Destination Fragment Viewer	Subaccount	Destination_Fragment_Viewer	destination-xsappname!<suffix>
Destination Subaccount Trust Administrator	Subaccount	Destination_Subaccount_Trust_Administrator	destination-xsappname!<suffix>
Destination Subaccount Trust Viewer	Subaccount	Destination_Subaccount_Trust_Viewer	destination-xsappname!<suffix>
Destination Viewer	Subaccount	Destination_Viewer	destination-xsappname!<suffix>
Destination Viewer Instance	Subaccount	Destination_Viewer_Instance	destination-xsappname!<suffix>
Directory Admin	Directory	Directory_Viewer	cis-central!<suffix>
Directory Usage Reporting Viewer	Directory	Directory_Usage_Reporting_Viewer	uas!<suffix>
Directory Viewer	Directory	Directory_Admin	cis-central!<suffix>
Global Account Admin	Global account	GlobalAccount_Admin	cis-central!<suffix>
Global Account Viewer	Global account	GlobalAccount_Viewer	cis-central!<suffix>
Global Account Usage Reporting Viewer	Global account	GlobalAccount_Usage_Reporting_Viewer	uas!<suffix>
Service Credentials Viewer	Subaccount	Service_Credentials_Viewer	service-manager!<suffix>
Service Instances Viewer	Subaccount	Service_Instances_Viewer	service-manager!<suffix>
Subaccount Admin	Subaccount	Subaccount_Admin	cis-local!<suffix>
Subaccount Service Admin	Subaccount	Subaccount_Service_Admin	service-manager!<suffix>

Roles	Available in	Role Templates	Application Identifier
Subaccount Service Administrator	Subaccount	Subaccount_Service_Administrator	service-manager!<suffix>
Subaccount Service Auditor	Subaccount	Subaccount_Service_Auditor	service-manager!<suffix>
Subaccount Service Viewer	Subaccount	Subaccount_Service_Viewer	service-manager!<suffix>
Subaccount Viewer	Subaccount	Subaccount_Viewer	cis-local!<suffix>
System Landscape Administrator	Global account	GlobalAccount_System_Landscape_Administrator	cmp!<suffix>
System Landscape Viewer	Global account	GlobalAccount_System_Landscape_Viewer	extension-service-cmp!<suffix>
User and Role Administrator	Global account and subaccount	xsuaa_admin	xsuaa!<suffix>
User and Role Auditor	Global account and subaccount	xsuaa_auditor	xsuaa!<suffix>

Roles in Environments

If you've enabled environments in your subaccount, manage the roles for those environments. For more information, see:

- About Roles in the Cloud Foundry Environment [page 2788]
- Assign Roles in the Kyma Environment [page 3666]

> **Note**
>
> The ABAP environment uses the roles of the SAP BTP, Cloud Foundry Environment.

Related Information

User and Member Management [page 133]
Working with Role Collections [page 2612]
Building Roles and Role Collections for Applications [page 2623]
Mapping Role Collections [page 2620]
Managing Users and Their Authorizations Using the btp CLI [page 2709]

2.9.1.2 Bringing Your Corporate Identity Provider for Platform Users

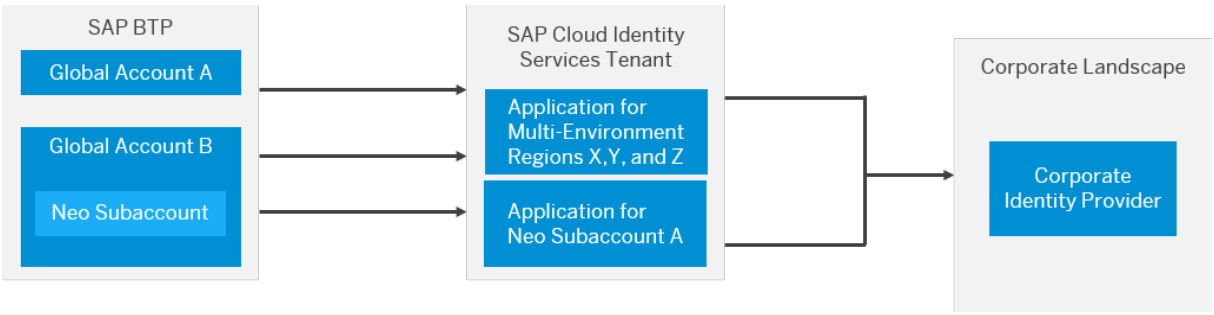
SAP BTP supports the use of your own identity provider for platform users.

Platform users perform technical development, operations, and administration tasks. They manage global accounts, directories, and subaccounts using the SAP BTP cockpit and the SAP BTP CLI. They also develop and operate custom applications. By hosting these users in your own identity provider, you gain a number of advantages over hosting them in SAP ID service.

- Integrate the management of these users with your broader identity management strategy, hosted on your own identity providers. You control your own user lifecycle and single sign-on strategies throughout the entire landscape.
- Enforce your own password and authentication policies, such as stronger passwords or multifactor authentication.

The following figure illustrates the architecture required for platform users. This configuration is independent of the default configuration with SAP ID service. You can continue to use SAP ID service in parallel.

Authentication Architecture for Platform Users with a Corporate Identity Provider



In the preceding figure, you enable trust between the SAP BTP global account and your corporate identity provider over your tenant of SAP Cloud Identity Services - Identity Authentication. Trust is configured differently in the Kyma environment (see the related link). For each global account, you choose the SAP Cloud Identity Services tenant to use as the identity provider for platform users. For the platform identify provider, you can have up to three SAP Cloud Identity Services tenants per global account. Multiple global accounts can share the same SAP Cloud Identity Services tenant. When you log on to a platform resource, such as the cockpit, you indicate the SAP Cloud Identity Services tenant that you want to log on with. For example, to log on to the cockpit, copy a URL parameter from the cockpit to identify the tenant:

`https://cockpit.btp.cloud.sap/cockpit/?idp=<tenant>.accounts.ondemand.com`

For example: `https://cockpit.btp.cloud.sap/cockpit/?idp=cidppuxhm.accounts.ondemand.com`

Once you've logged on, the cockpit displays any global accounts and subaccounts of which your platform user is a member.

A user identifier alone isn't enough to identify a user. Additionally, you also need an identifier for an identity provider. The reason is that the system treats users with the same name but from different identity providers as separate users.

For global accounts, directories, multi-environment subaccounts, and the Cloud Foundry environment, the user identifier is always an email address, and the identifier of the identity provider is the origin key (alias for the

trust configuration). For example, a platform user in the default identity provider, SAP ID service, and another user in your corporate identity provider with the same email address might have different authorizations.

> **Example**
>
> There are two users with the same email address. It is common practise that the two users with different identity providers have different authorizations and can access different applications.
>
> - julia.moore@acme.com with the default identity provider isn't authorized for any subaccount.
> - julia.moore@acme.com with the `acme-platform` custom identity provider is the administrator for multiple subaccounts. The subaccounts are only visible when she logs on with the custom identity provider.

You can log on to the cockpit with both, but the cockpit displays different user information. This difference is because you've logged on with different identity providers.

For Neo subaccounts, it is, by default, the P user ID for local SAP Cloud Identity Services users or whatever the corporate identity provider connected to the SAP Cloud Identity Services tenant sends as subject.
The identifier of the identity provider is the default domain of the SAP Cloud Identity Services tenant, for example `acme.accounts.ondemand.com`. It is called "user base". For more information, see Platform Identity Provider.

You also see this difference when assigning roles. You must provide the origin or user base in addition to the email address or user ID of the user. When platform users use the Cloud Foundry command-line interface or service dashboards, they need to remember the origin. You can choose your own origin, but the origin must be unique across all customers. We recommend that you use a meaningful name that helps identify the target it points to.

In SAP Cloud Identity Services - Identity Authentication in Neo subaccounts, there's one application that represents SAP BTP overall. So, if you have multiple global accounts with the same SAP Cloud Identity Services tenant, they all share the same application in your SAP Cloud Identity Services tenant. Here customers typically configure settings such as the corporate identity provider used for authentication and user attribute mapping between systems. For more information, see Map User Attributes from a Corporate Identity Provider for Platform Users [page 2581].

Related Information

Log On with a Custom Identity Provider to the Cloud Foundry Environment Using the Cloud Foundry Command-Line Interface [page 2587]
Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576]
Configuring a Custom Identity Provider for Kyma [page 3660]

2.9.2 Business Users

Business users use the applications that are deployed to SAP BTP. For example, the end users of SaaS apps or services, such as SAP Build Work Zone, or end users of your custom applications are business users.

Application developers (platform users) create and deploy application-specific security artifacts for business users, such as scopes. Administrators use these artifacts to assign roles, build role collections, and assign

these role collections to business users or user groups. In this way, they control the users' permissions in the application.

For business users, there's a default identity provider [page 2594], too. We expect that you have your own identity provider.

> → Recommendation
>
> We strongly recommend that you configure your custom tenant of SAP Cloud Identity Services as the identity provider and connect SAP Cloud Identity Services to your own corporate identity provider.
>
> For more information, see Trust and Federation with Identity Providers [page 2538].

User Management

User management refers to managing authentication and authorization for your business users.

To manage your business users:

1. Configure trust to an application identity provider in your subaccount.
2. Create shadow users in your subaccount for your business users in your identity provider.
   When a user accesses a resource, SAP BTP redirects the user to the identity provider for authentication.
   You assign authorizations to shadow users in SAP BTP.
3. Assign role collections either directly to users or map them to user groups.
   The role collections were either delivered from the applications to which you subscribed or custom developed by your team.

To learn more about user management, see Security Administration: Managing Authentication and Authorization [page 2537].

2.10 Tools, Programming Models, Programming Languages, and APIs

SAP BTP provides various programming languages and tools for your development project.

Tools [page 151]
: SAP BTP includes many tools to help you develop and manage applications, and connect them to your on-premise systems.

Programming Languages [page 153]
: SAP BTP supports many different programming languages; the availability of each depends on the development environment you're using.

Programming Models [page 154]

Continuous Integration and Delivery (CI/CD) [page 155]
: Configure and run predefined continuous integration and delivery (CI/CD) pipelines that automatically build, test, and deploy your code changes to speed up your development and delivery cycles.

APIs [page 158]
Discover and consume APIs to manage, build, and extend the core capabilities of SAP BTP.

Cloud Management Tools [page 159]
Cloud management tools represent the group of technologies designed for managing SAP BTP.

Prerequisites and Restrictions [page 159]
Find a list of the product prerequisites and restrictions for SAP BTP.

2.10.1 Tools

SAP BTP includes many tools to help you develop and manage applications, and connect them to your on-premise systems.

For more information, see Tools [page 151].

Tool	Description
Account Administration in the Cockpit [page 2458]	The SAP BTP cockpit is the web-based administration interface of SAP BTP and provides access to a number of functions for configuring and managing applications, services, and subaccounts. Use the cockpit to manage resources, services, security, monitor application metrics, and perform actions on cloud applications. Used for account administration in SAP BTP.
SAP Business Application Studio	Designed and optimized for business application development in SAP ecosystems, SAP Business Application Studio enhances productivity by offering specialized tools for various scenarios, including SAP Fiori application development, SAP HANA native extensions, full-stack and mobile application development, and more. Central to the development environment is Code-OSS, the open-source foundation of Visual Studio Code, ensuring a familiar experience for developers when creating SAP-centric applications. SAP Business Application Studio streamlines the building, testing, and deployment of applications with integrated features for source control and testing. Furthermore, its Full-Stack Application Productivity Toolkit offers intuitive visual tools covering the entire development process, guaranteeing seamless integration with various SAP services and solutions. Used for: • SAP Fiori development for ABAP and non-ABAP • CAP full-stack application development

Tool	Description
SAP Build	SAP Build enables everyone, no matter the skill level, to create and augment enterprise applications, process automations, and business sites with drag-and-drop simplicity. Used for low-code, no-code development.
Command Line Interface for Cloud Foundry [page 2813]	The Cloud Foundry command line interface enables you to work with the Cloud Foundry runtime to deploy and manage your applications. Used for managing subaccounts in the SAP BTP, Cloud Foundry runtime.
SAP BTP Command Line Interface [page 2664]	The SAP BTP command line interface (btp CLI) is the command line tool for convenient account management, such as managing global accounts, directories, subaccounts, entitlements, environment instances, multitenant application subscriptions, and users and their authorizations. Used for managing global accounts in SAP BTP.
Cloud Connector	The Cloud Connector serves as the link between on-demand applications in SAP BTP. This is the browser-based and existing on-premise systems. You can control the resources available for the cloud applications in those systems. Used for connectivity between SAP BTP applications and on-premise systems.
Terraform provider for SAP BTP	The Terraform provider for SAP BTP enables you to automate the provisioning, management, and configuration of resources on SAP BTP.
kubectl	The Kubernetes command line tool to communicate with a Kubernetes cluster's control plane, using the Kubernetes API. Used as a Kubernetes tool connected with the SAP BTP, Kyma runtime.
kubelogin	A `kubectl` plugin for Kubernetes OpenID Connect (OIDC) authentication. Used as a Kubernetes tool connected with the SAP BTP, Kyma runtime.
Helm	The package manager for Kubernetes, used for installing and managing Kubernetes applications in form of Helm charts. Used as a Kubernetes tool connected with the SAP BTP, Kyma runtime.

Tool	Description
Paketo (Pack)	Pack is a tool maintained by the Cloud Native Buildpacks project to support the use of buildpacks. Pack lets you build container images, which are collaboratively maintained making it easier to maintain and update. Used as a Kubernetes tool connected with the SAP BTP, Kyma runtime.
Docker Desktop	Docker Desktop is an application that enables you to manage (build, push, pull, and run) container images on your desktop and a docker-compatible command line interface. Used as a Kubernetes tool connected with the SAP BTP, Kyma runtime.
SAP Cloud SDK	SAP Cloud SDK provides a layer of abstractions for features of SAP BTP such as logging, multitenancy, and connectivity. It also includes project templates for different execution environments and implementations. Used for programming in Java and JavaScript.
Eclipse Tool for the Cloud Foundry Runtime	The Eclipse plug-in for the Cloud Foundry runtime is a Java-based toolkit for Eclipse IDE that enables you to develop and deploy Java and Spring applications in the Cloud Foundry runtime from Eclipse or Spring Tool Suite, as well as perform operations such as logging, managing user roles, creating connectivity destinations, and so on. Used for programming in Java in the SAP BTP, Cloud Foundry runtime.
Service-Specific Tools	The services that run on SAP BTP can come with service-specific tools. For an overview of the services and their tools, see the SAP Discovery Center.

2.10.2 Programming Languages

SAP BTP supports many different programming languages; the availability of each depends on the development environment you're using.

The following programming languages are available in the environments:

Supported Environments and Programming Languages

Environment	Programming Language
Cloud Foundry environment	Java [page 284]

Environment	Programming Language
	Node.js [page 384]
	Python [page 394]
ABAP environment	ABAP [page 848]
Kyma environment	Serverless Functions: Node.js and Python [page 2267]
	Microservices: Any cross-platform programming language

2.10.3 Programming Models

ABAP Cloud

ABAP Cloud reflects the modern way to develop ABAP. It allows you to build lifecycle-stable and cloud-ready business applications, services, and extensions.

ABAP Cloud provides tools and techniques that ensure cloud qualities, promotes new technologies, contains a cloud-optimized subset of the ABAP language, and makes upgrade cycles easier by a clear separation between custom code and SAP code by only using released APIs and objects. See ABAP Cloud in SAP BTP Developer's Guide.

SAP Cloud Application Programming Model (CAP)

The SAP Cloud Application Programming Model (CAP) is a framework of languages, libraries, and tools for building enterprise-grade services and applications. It supports Java (with Spring Boot), JavaScript, and TypeScript (with Node.js), which are some of the most widely adopted languages. CAP guides developers along a path of proven best practices and a great wealth of out-of-the-box solutions to recurring tasks.

CAP-based projects benefit from a primary focus on the domain. Instead of delving into overly technical disciplines, CAP focuses on accelerated development and safeguarding investments in a world of rapidly changing cloud technologies.

Here are some of the benefits that come with the SAP Cloud Application Programming Model (CAP):

- Built-in best practices
- Support for Visual Studio Code and SAP Business Application Studio tools
- Safeguarded application development investments
- No runtimes lock-in
- Reuse and integration of SAP BTP application services

- Latest UX and themes

See SAP Cloud Application Programming Model (CAP) in SAP BTP Developer's Guide.

2.10.4 Continuous Integration and Delivery (CI/CD)

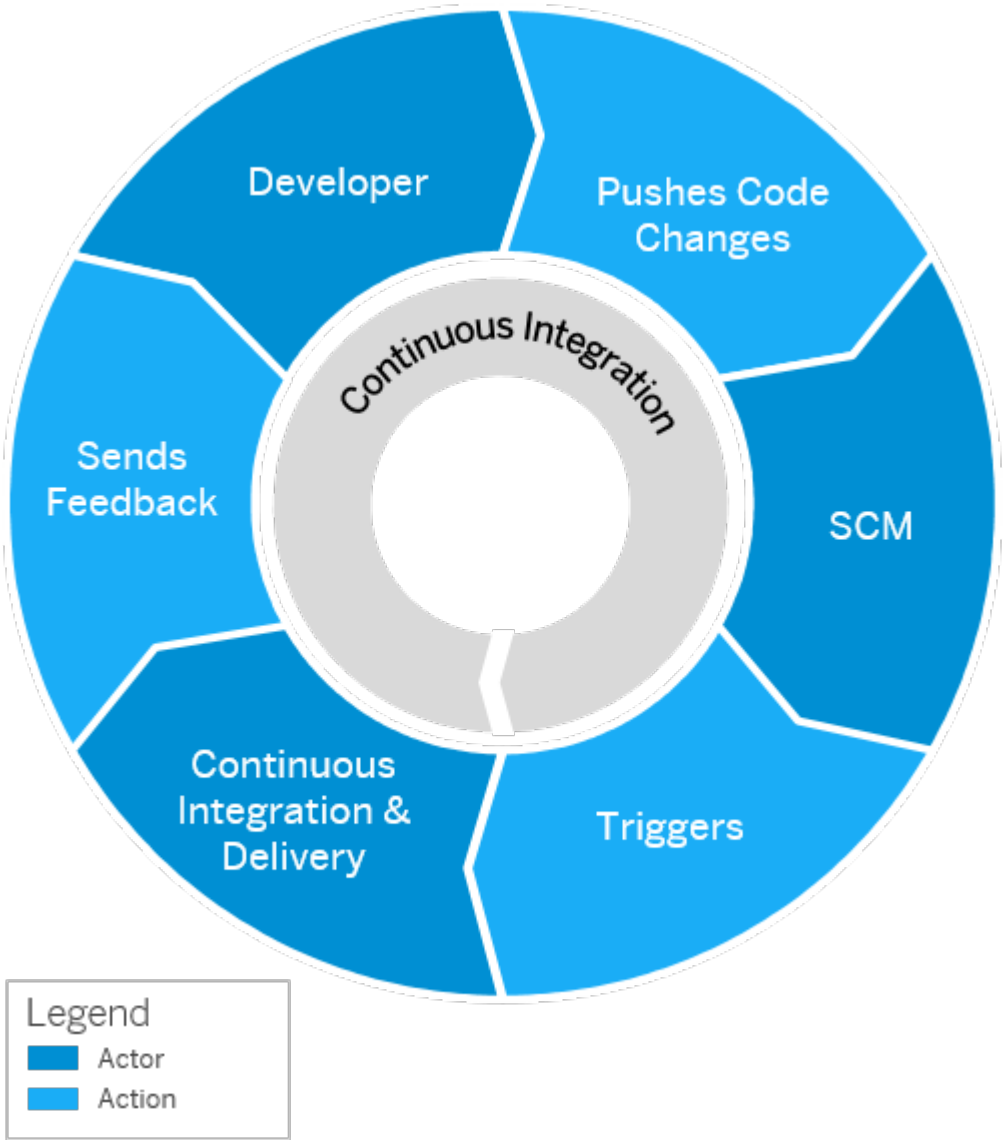
Configure and run predefined continuous integration and delivery (CI/CD) pipelines that automatically build, test, and deploy your code changes to speed up your development and delivery cycles.

> **Note**
>
> For links to all SAP solutions for CI/CD, blog posts, presentations, and tutorials, have a look at our Continuous Integration and Delivery by SAP overview.

Continuous integration (CI) describes a software development process, in which various team members integrate their contributions frequently into a single main line. Before each integration, the changes are verified through builds and automated testing. Thereby, you can detect errors as quickly as possible and prevent integration problems before completing the development.

The **continuous delivery (CD)** concept expands on the one of continuous integration. It adds the aspect that any change that has successfully passed the tests is immediately ready to be deployed to production, both from a technical and a qualitative point of view.

The following graphic shows the basic flow for continuous integration and delivery:



Continuous Integration Basic Flow

For more information about the continuous integration and continuous delivery concepts, see What Are Continuous Integration and Continuous Delivery?.

Use

Depending on your use case, you can choose between different CI/CD pipelines to help you implement continuous integration and delivery in your software development.

SAP Continuous Integration and Delivery lets you configure and run predefined pipelines for the development of the following applications:

- Configure a Cloud Foundry Environment Job

Create an SAP Continuous Integration and Delivery job for SAP Fiori and SAP Cloud Application Programming Model projects in the Cloud Foundry environment.

- Configure an SAP Fiori for the ABAP Platform Job
  Create an SAP Continuous Integration and Delivery job for SAP Fiori projects for the ABAP platform.
- Configure an SAP Integration Suite Artifacts Job
  Create an SAP Continuous Integration and Delivery job for SAP Integration Suite artifacts.

To learn more about the CI/CD pipelines supported by SAP Continuous Integration and Delivery and the stages each pipeline can comprise, see Configuring Jobs.

Get Started with CI/CD

SAP Continuous Integration and Delivery provides an easy, UI-guided way to set up the service and configure and run your pipelines, without hosting your own Jenkins instance.

To set up SAP Continuous Integration and Delivery:

1. Enable the service in the SAP BTP cockpit.
2. Assign either the *Administrator* or *Developer* role to your user.
3. Enable the API usage to connect SAP Continuous Integration and Delivery to other services, if necessary.

To configure SAP Continuous Integration and Delivery:

> ⓘ Note
>
> Only administrators of SAP Continuous Integration and Delivery can configure the service.

1. Configure credentials for connecting SAP Continuous Integration and Delivery to other services (for example, GitHub, GitLab, Bitbucket Server, or Azure Repos to clone your sources, and SAP BTP to deploy your built application).
2. Add your repository.

Now you can create and modify your CI/CD jobs and monitor their outcome. If you want to automate your builds, you can configure a webhook between your repository and the service. You can create and modify timed triggers for your jobs, if necessary.

For more information, see Initial Setup and Configuration.

Automate the Provisioning of Resources Needed to Create a Kyma Cluster

Integrate the Terraform module, which uses the Terraform provider for SAP BTP, into CI/CD pipelines, so you can automate the provisioning of resources needed to create a Kyma cluster, deploy applications, run tests, and tear down the resources when they are no longer required. As there is no need to have resources running continuously, this approach reduces operational costs.

See Terraforming Kyma Runtimes.

Learn and Get Certified

Depending on your learning goals and level of expertise, you can choose from the following offerings:

- **Exploring DevOps with SAP BTP**
  Learn how to efficiently apply DevOps practices to your development projects on SAP BTP.
- **Efficient DevOps with SAP**
  This openSAP course introduces general DevOps approaches and key principles. Learn about the basic CI/CD principles (week 1) and how to deliver cloud applications using CI/CD (week 3).
- **Continuous Integration and Delivery Introduction Guide**
  The CI/CD Introduction Guide provides you with basic knowledge for setting up and implementing continuous integration and delivery processes. It gives an overview of the concepts and principles of CI/CD, explains both procedures and their relation, and helps you plan your own CI/CD process.

2.10.5 APIs

Discover and consume APIs to manage, build, and extend the core capabilities of SAP BTP.

An Application Programming Interface or API is an **interface provided by an application for interacting with other applications**. APIs specify how software programs are able to exchange information with each other, even if designed and run by different organizations. They facilitate interaction by selectively exposing certain functionality, allowing different apps, websites, and devices to communicate effectively with each other. More importantly, APIs enable businesses to reach beyond regular business channels and share data, content, and services directly to both B2B (business to business) and B2C (business to consumer) clients, making UI development easy.

API Offerings on SAP BTP

SAP BTP enables you to consume APIs and publish your own ones through the following offerings:

Offering	Description
SAP BTP on SAP Business Accelerator Hub	The SAP Business Accelerator Hub provides you with one central repository for browsing and accessing APIs from SAP and selected partners. Test APIs and try out mock data in your systems. It is also the official place where REST and OData REST API references are published.
API Management Capability within SAP Integration Suite	With the API Management capability, you can build, manage, publish, and monetize your own APIs within one secure and scalable environment.

Offering	Description
Software Development Kits (SDKs) For an overview of the available SDKs, see Tools [page 151].	The SDKs available for SAP BTP offer APIs to, for example, accelerate enterprise app development.

Related Information

SAP API Style Guide

2.10.6 Cloud Management Tools

Cloud management tools represent the group of technologies designed for managing SAP BTP.

Related Information

Tools [page 151]

2.10.7 Prerequisites and Restrictions

Find a list of the product prerequisites and restrictions for SAP BTP.

General Constraints

- For information on constraints and default settings to consider when you deploy an application in the Cloud Foundry environment, see http://docs.cloudfoundry.org/devguide/deploy-apps/large-app-deploy.html#limits_table .
- SAP BTP exposes applications only via HTTPS. For security reasons, applications can't be accessed via HTTP.

SAP BTP Tools

- SAP BTP Tools for Java and SDK have been tested with Java 7, and Java 8.
- SAP BTP Tools for Java and SDK run in many operating environments with Java 7, and Java 8 that are supported by Eclipse. However, SAP doesn't systematically test all platforms.

- SAP BTP Tools for Java must be installed on Eclipse IDE for Java EE developers.

For the platform development tools, SDK, Cloud connector, and SAP JVM, see https://tools.hana.ondemand.com/#cloud .

Browser Support

For a list of supported browsers for the SAP BTP cockpit, see Feature Scope Description.

For a list of supported browsers for developing SAPUI5 applications, see Browser and Platform Support.

To find out the browser support for a specific service, refer to the corresponding feature scope description.

For security reasons, SAP BTP doesn't support TLS1.1 and older, SSL 3.0 and older, and RC4-based cipher suites. Make sure that your browser supports at least TLS1.2 and modern ciphers (for example, AES).

3 Getting Started

Once you're familiar with the basic concepts of SAP BTP, you can start your first workflows or check out further resources that help you get started quickly.

Onboarding Guides

The SAP BTP Administrator's Guide helps you plan your development projects on SAP BTP from onboarding to SAP Cloud Identity Services, through setting up the correct organizational structure, to creating an account and security model, to developing and operating applications.

The SAP BTP Developer's Guide helps you define the correct methodologies and tools for your development project.

Getting Started Workflows

Learn how to get a global account and get started with a trial or an enterprise account on SAP BTP. A trial account lets you try out SAP BTP for free, and within an enterprise account, you can use so-called free tier service plans for free. See Trial Accounts and Free Tier [page 109].

Depending on your use case, follow the appropriate workflow:

- Getting a Global Account [page 162]
- Getting Started in the Cloud Foundry Environment [page 165]
- Getting Started in the ABAP Environment [page 183]
- Getting Started in the Kyma Environment [page 247]

Additional Resources

The SAP Developer Center provides further resources to get you started:

- SAP Discovery Center
- SAP BTP in SAP Developer Center
- Start Developing on SAP BTP Tutorials

Related Information

SAP BTP Administrator's Guide

SAP BTP Developer's Guide
Access the Cockpit [page 2459]

3.1 Getting a Global Account

SAP BTP offers two types of global accounts: Trial accounts (completely free of charge) and enterprise accounts. Within an enterprise account, you can use both free and paid plans.

A **global account** is the realization of a contract you or your company has made with SAP. A global account is used to manage subaccounts, members, entitlements and quotas. You receive entitlements and quotas to use platform resources per global account and then distribute the entitlements and quotas to the subaccount for actual consumption. There are two types of commercial models for global accounts: consumption-based model and subscription-based model. See Commercial Models [page 113]

- Trial Accounts and Free Tier [page 109]
- Enterprise Accounts [page 111]
- Try Out SAP BTP for Free [page 162]
- Get an Enterprise Account [page 163]

3.1.1 Try Out SAP BTP for Free

There are two ways to try out SAP BTP services and applications for free.

Context

A **free trial** account is intended for individuals, for example students, who want to explore SAP BTP for free. It is an isolated account and subaccount, with preallocated service entitlements that can be used for a limited number of days.

Free tier is not an account type, but it's a set of free services that can be used within an enterprise account.

To learn which is best for your needs, see the Set Up an SAP BTP Account for Tutorials group in the SAP Developer Center.

3.1.2 Get an Enterprise Account

To use an enterprise account, you can either purchase a customer account, join the partner program to purchase a partner account, or self-register for an enterprise account to try out free tier service plans.

For more information about the scope of our enterprise offering, see Enterprise Accounts [page 111]. For more information about free tier within an enterprise account, see Trial Accounts and Free Tier [page 109] and Using Free Service Plans [page 118].

Related Information

Get an Account on SAP BTP to Try Out Free Tier Service Plans

3.1.2.1 Sign up for a Customer Account

A customer account is an enterprise account that allows you to host productive, business-critical applications with 24x7 support.

When you want to purchase a customer account, you can select from a set of predefined packages. For information about service availability, prices, and estimators, seehttps://www.sap.com/products/technology-platform/pricing.html . You can also view the service catalog via the SAP Discovery Center. Contact us on SAP BTP or via an SAP sales representative.

Free Tier

If you want to try out services for free, with the option of easily upgrading them later, you can get an enterprise account and use free tier service plans only. See Get an Account on SAP BTP to Try Out Free Tier Service Plans. When you sign up for a global account to try out free tier service plans, you need to select either an existing global account of your company, or create a new global account. If you choose an existing global account, make sure to contact the global account admin of this global account, as they'll receive the communication emails. In this case, we recommend to have them add you as Global Account Administrator. See Assign Users to Role Collections [page 2617] and SAP BTP Onboarding Resource Center.

You can upgrade and refine your resources later on. You can also contact your SAP sales representative and opt for a configuration, tailored to your needs.

Onboarding

After you have purchased a new customer account, you will receive an email confirming the provisioning of resources from the platform services team and a second email with the URL and login ID to obtain access. Note, only the person who receives the access email has initial access to SAP BTP.

If you chose an add-on to an existing global account, the initial access email will not be sent to you. You will need to contact the Global Account Administrator from the initial BTP order. If you're unsure who this person is, please contact your SAP Sales representative. If you need to add a new global admin because the person with initial access is no longer available to grant you access, then you can contact the SAP Support Portal, by creating a ticket at component BC-NEO-CIS-OPS.

> → Tip
>
> Check out the Live Onboarding Webinars.

Getting a Customer Account in China

If you are located in China and want to buy a global account on SAP BTP, you need to contact an SAP sales representative: https://www.sap.cn/registration/contact.html .

Related Information

Commercial Models [page 113]
Using Free Service Plans [page 118]
Access the Cockpit [page 2459]

3.1.2.2 Join the Partner Program

A partner account is an enterprise account that enables you to build applications and to sell them to your customers.

To become a partner, you need to fill in an application form and then sign your partner contract. You will be assigned to a partner account with the respective resources. To apply for the partner program, visit https://partneredge.sap.com/content/partnerregistration/en_us/registration.html?partnertype=BLD&engagement=0002&build=1. You will receive a welcome mail with further information afterwards.

General information about the partner program is available onhttps://www.sap.com/partners/partner-program.html .

Next Steps

- Getting Started in the Cloud Foundry Environment [page 165]
- Getting Started in the ABAP Environment [page 183]

- Getting Started in the Kyma Environment [page 247]

3.2 Getting Started in the Cloud Foundry Environment

Get onboarded in the Cloud Foundry environment of SAP BTP. Follow the workflows for trial or customer accounts or subscribe to business applications.

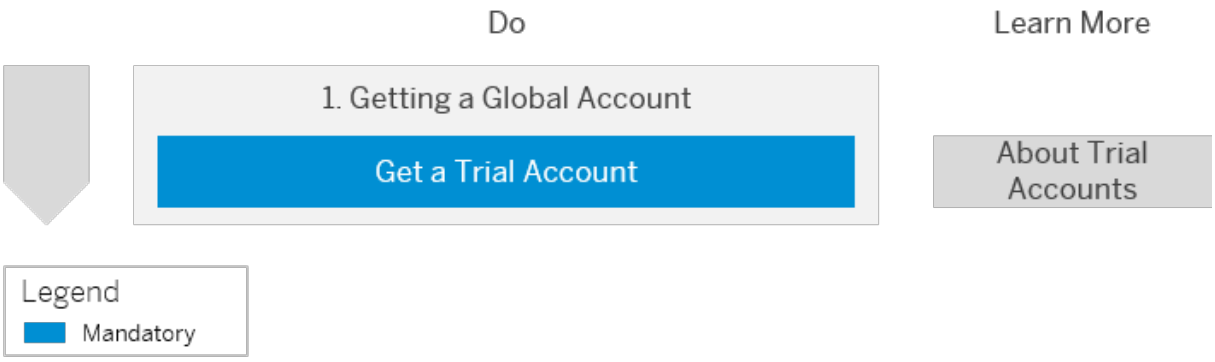
Getting Started with a Trial Account in the Cloud Foundry Environment [page 165]
: Quickly get started with a trial account.

Getting Started with an Enterprise Account in the Cloud Foundry Environment [page 176]
: Quickly get started with an enterprise account in the Cloud Foundry Environment.

3.2.1 Getting Started with a Trial Account in the Cloud Foundry Environment

Quickly get started with a trial account.

1. Getting a Global Account

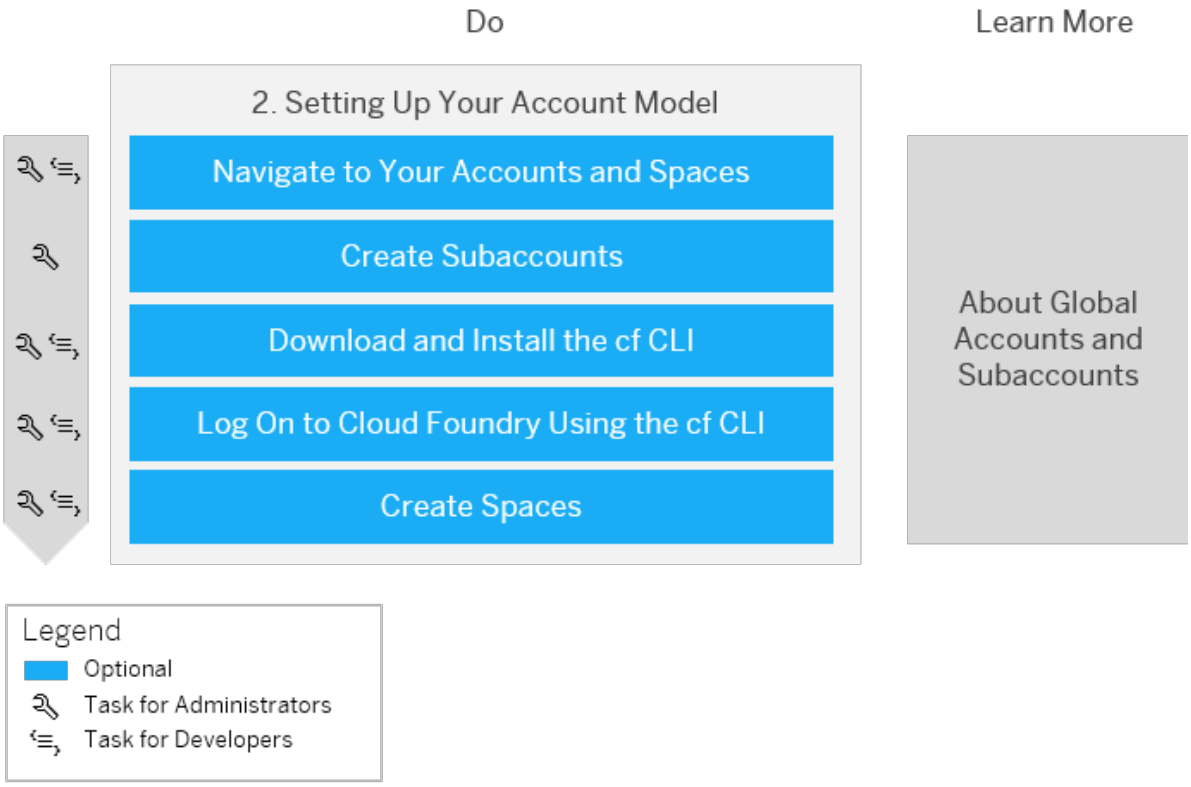


- Try Out SAP BTP for Free [page 162]
- Trial Accounts and Free Tier [page 109]

Before you begin, sign up for a free trial account. See Get a Free Trial Account [page 162]. For more information about the scope of our trial offering, see Trial Accounts and Free Tier [page 109].

If you want to familiarize yourself with the Cloud Foundry environment, see Cloud Foundry Environment [page 63].

2. Setting Up Your Account Model

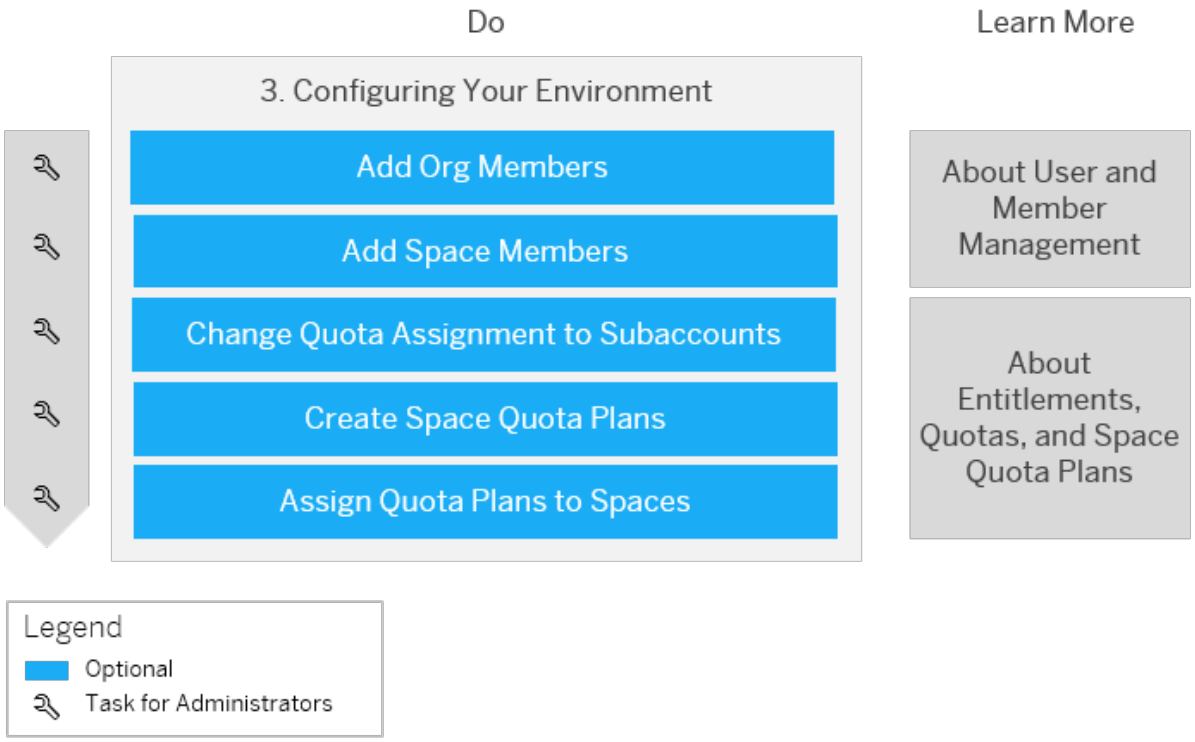


- Navigate to Orgs and Spaces [page 2792]
- Create a Subaccount [page 2509]
- Download and Install the Cloud Foundry Command Line Interface [page 2813]
- Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814]
- Create Spaces [page 2800]
- Account Model [page 121]

1. When you register for a trial account, a subaccount and a space are created for you. You can create additional subaccounts and spaces, thereby further breaking down your account model and structuring it according to your development scenario, but first it's important you understand how to navigate to your accounts and spaces using the SAP BTP cockpit. See Navigate to Orgs and Spaces [page 2792].
2. If you like, create further subaccounts. See Create a Subaccount [page 2509]. You can also download and use the CLI forSAP BTP to create new subaccounts. See Download and Start Using the BTP CLI Client [page 2665] and Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699].
3. If you haven't done so already, now is a good time to download and install the Cloud Foundry Command Line Interface (cf CLI). This tool allows you to administer and configure your environment,enable services, and deploy applications. See Download and Install the Cloud Foundry Command Line Interface [page 2813]. But don't worry, you can also perform all the necessary task using the SAP BTP cockpit, which you don't need to install.

4. If you'd like to use the cf CLI, log on to your environment. See Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].
5. If you like, create further spaces. See Create Spaces [page 2800]. If you want to learn more about subaccounts, orgs, and spaces, and how they relate to each other, see Account Model [page 121].

3. Configuring Your Environment

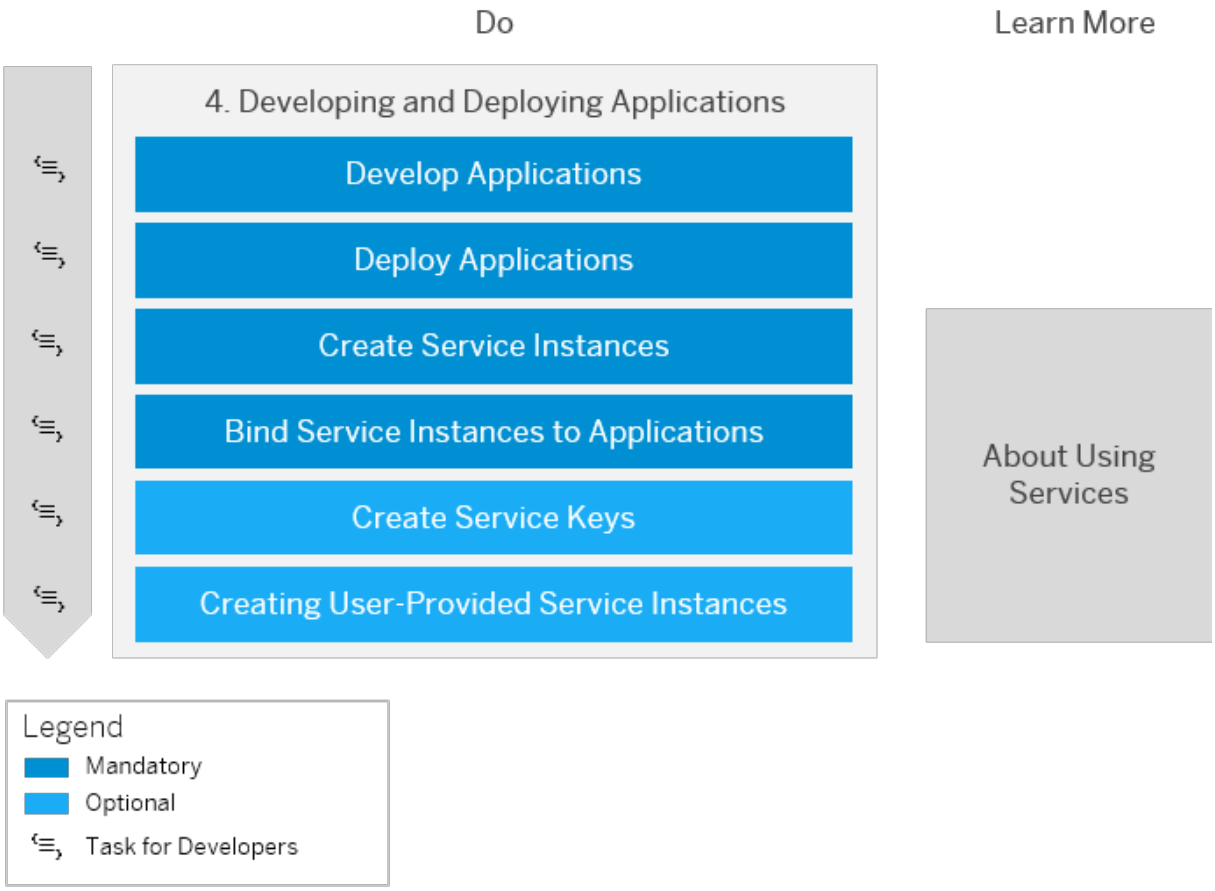


- Add Org Members [page 2795]
- Add Space Members [page 2803]
- Configure Entitlements and Quotas for Subaccounts [page 2529]
- Create Space Quotas [page 2808]
- Create Space Quota Plans Using the Cloud Foundry Command Line Interface [page 2857]
- User and Member Management [page 133]
- Entitlements and Quotas [page 127]

1. Now that you've set up your account model, it's time to think about member management. You can add members at different levels. For example, you can add members at the org level. See Add Org Members [page 2795]. For more information about the roles that are available on the different levels, see User and Member Management [page 133].
2. You can also add members at the space level. See Add Space Members [page 2803].

3. In a trial account, quotas are automatically assigned to your subaccounts, but you can change that assignment. See Configure Entitlements and Quotas for Subaccounts [page 2529]. To learn more about entitlements and quotas, see Entitlements and Quotas [page 127].
4. You can also assign quotas to different spaces within a subaccount. To do so, first create a space quota plan. See Create Space Quotas [page 2808] or Create Space Quota Plans Using the Cloud Foundry Command Line Interface [page 2857].
5. Then assign the quota plan to your space. See Assign Space Quotas to Spaces [page 2810] or Assign Quota Plans to Spaces Using the Cloud Foundry Command Line Interface [page 2858].

4. Developing and Deploying Applications



- Development [page 254]
- Deploy Business Applications in the Cloud Foundry Environment [page 845]
- Creating Service Instances [page 273]
- Binding Service Instances to Applications [page 276]
- Creating Service Keys [page 278]

- About Services [page 272]
- Creating User-Provided Service Instances [page 275]

1. Develop your application. Check out the Developer Guide for tutorials and more information. See Development [page 254].
2. Deploy your application. See Deploy Business Applications in the Cloud Foundry Environment [page 845].
3. Integrate your application with a service. To do so, first create a service instance. See Creating Service Instances [page 273].
4. Bind the service instance to your application. See Binding Service Instances to Applications [page 276].
5. Alternatively, you can also create and use service keys. See Creating Service Keys [page 278]. For more information on using services and creating service keys, see About Services [page 272].
6. You can also create instances of user-provided services. See Creating User-Provided Service Instances [page 275].

> → Tip
>
> Also check out the tutorial Create Your First App on Cloud Foundry to see how you can deploy a pre-bundled set of artifacts using the SAP BTP cockpit, access the app from your web browser, and create an instance of a service available on Cloud Foundry and bind it to your app.

Related Information

Cloud Foundry Environment [page 63]
Trial Accounts and Free Tier [page 109]
Account Model [page 121]
Entitlements and Quotas [page 127]
User and Member Management [page 133]
About Services [page 272]

3.2.1.1 Setting Up Your Trial Account

Your trial account is set up automatically, so that you can start using it right away. However, if one or more of the automatic steps fail, you can also finalize the setup manually, by following the steps below.

3.2.1.1.1 Create Your Trial Subaccount

The first thing that is needed in the setup of your trial account is the creation of a subaccount. If this step was successful in the SAP BTP cockpit, you can directly skip to the next section.

Procedure

1. Navigate into your global account by choosing *Enter Your Trial Account*.
2. Choose *New Subaccount*.
3. Configure it as follows:

Field	Input
Display Name	`trial`
Description	(Optional) Any text that describes your subaccount
Provider	Desired infrastructure provider
Region	Desired region
Subdomain	`<your_id>trial` Example: **P0123456789trial**
[Advanced] Enable beta features	(Optional) Enables the use of beta services and applications.
[Advanced] Labels	(Optional) You can assign labels to help identify and organize the subaccounts in your global account. For example, you can filter subaccounts by custom property in the *Account Explorer* page. See Labels [page 125].

Results

You have successfully set up your trial subaccount.

3.2.1.1.2 Create Your Trial Org and Configure Entitlements

Once you have a subaccount (whether it was created automatically or you followed the steps described above), you need an org and entitlements.

Procedure

1. Navigate to your subaccount by choosing its tile.
2. Choose *Enable Cloud Foundry* to create your org.
3. Once your org is created, choose *Entitlements* from the left hand-side navigation.
4. Choose *Configure Entitlements* to enter edit mode.
5. Choose *Add Service Plans* and select all the service plans available for your subaccount.

   > **Note**
   >
   > To select a service plan, choose a service from the left and tick all the service plans that appear on the right. Do that for all services.

6. Once you've added all the service plans, you see them all in a table. Before you choose Save, for all the plans with numerical quota, choose + to increase the amount to the maximum (until the icon becomes disabled).
7. Finally, choose *Save* to save all your changes and exit edit mode.

Results

You now have an org and all the entitlements for your subaccount. The last thing you need is a space where you can use the services you've configured entitlements for and deploy applications.

3.2.1.1.3 Create Your Trial Space

Procedure

1. In your *trial* subaccount, navigate to *Cloud Foundry* > *Spaces* using the left hand-side navigation.
2. Choose *Create Space* and name it **dev**.
3. Choose *Create*.

Results

You now have your trial set up all done and ready to go.

To get some guidance on how you can get started, navigate back to your Trial Home by choosing the first item in your breadcrumbs at the top. There, you can find several guided tours to walk you through the basics of SAP Business Technology Platform and the cockpit, as well as some more complex starter scenarios.

3.2.1.2 Setting Up a Trial Account From the Command Line

You can use the command-line interface to set up a trial account. For all tasks on global account and subaccount levels, you use the **SAP BTP command-line interface** (**btp CLI**). Once you've created a Cloud Foundry environment instance (a Cloud Foundry org), you use the Cloud Foundry CLI (cf CLI). This procedure works without the SAP BTP cockpit (except that you need the global account subdomain to log in, which you may have to look up in the cockpit).

For all tasks on global account and subaccount level, you can use the **the btp CLI** instead of the SAP BTP cockpit. Once you've created a Cloud Foundry environment instance (a Cloud Foundry org), use the Cloud Foundry CLI (cf CLI).

Prerequisites

- You have created a trial account.
- You have downloaded and extracted the following command-line tools:
  - SAP BTP command-line interface (btp CLI). See Download and Start Using the BTP CLI Client [page 2665]
  - Cloud Foundry CLI (cf CLI). See Download and Install the Cloud Foundry Command Line Interface [page 2813]

Procedure

> → Tip
>
> In the btp CLI, you can view the command help of each command to get information about how to use the command, its syntax, and input parameters. See Get Help [page 2678].

Step No.	Task	Performed By	Do This	More Information
1.	Log in to your global account using the subdomain of your global account.	Global account administrator or viewer	Run this command in the btp CLI: `btp login`	See Log in [page 2679].
2.	View details of your global account.	Global account admin or viewer	Run this command in the btp CLI: `btp get accounts/global-account`	See Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699].
4.	View all the regions that are available to your global account and subaccounts.	Global account admin or viewer	Run this command in the btp CLI: `btp list accounts/available-region`	This command also provides information about the environments and infrastructure provider of each region.
5.	Create subaccounts in your global account.	Global account admin	Run this command in the btp CLI: `btp create accounts/subaccount --display-name <my-subaccount> --region <my-region> --subdomain <my-subaccount-subdomain>`	See Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699] and Account Model.
6.	View the details of the subaccounts in your global account.	Global account admin or viewer	Run this command in the btp CLI: `btp get accounts/subaccount <ID of new subaccount>`	See Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699].
7.	Add admins to your subaccounts.	Subaccount admin	Assign the role collection `Subaccount Administrator` to the user by running the following command in the btp CLI: `btp assign security/role-collection "Subaccount Administrator" --to-user <user> --create-user-if-missing`	See Managing Users and Their Authorizations Using the btp CLI [page 2709] and Security Administration: Managing Authentication and Authorization [page 2537].
8.	View all the services and applications that are entitled to your global account, including quota information per service plan.	Global account admin or viewer	Run this command in the btp CLI: `btp list accounts/entitlement`	See Setting Entitlements Using the btp CLI [page 2702].

Step No.	Task	Performed By	Do This	More Information

9.	Assign quotas to your subaccounts.	Global account admin	Run this command in the btp CLI:
			`btp assign accounts/entitlement --to-subaccount <my-subaccount-id> --for-service <my-service> --plan <my-service-plan> --amount <number>`
			Validate with this command:
			`btp list accounts/entitlement`
				See Setting Entitlements Using the btp CLI [page 2702].

10.	View all the entitlements in your subaccounts.	Subaccount admin or viewer	Run this command in the btp CLI: `btp list accounts/entitlement --subaccount <my-subaccount-id>`	See Setting Entitlements Using the btp CLI [page 2702].
11.	Create a Cloud Foundry org (environment instance) in your subaccounts.	Subaccount admin	Run this command in the btp CLI: `btp create accounts/environment-instance --subaccount <my-subaccount-id> --display-name <my-environment> --environment <cloudfoundry>`	See Working with Environments Using the BTP CLI [page 2703].
12.	View details of the environment instances in your subaccounts.	Subaccount admin or viewer	Run this command in the btp CLI: `btp list accounts/environment-instance --subaccount <my-subaccount-id>`	See Working with Environments Using the BTP CLI [page 2703].

13.	Create a Cloud Foundry space.	Org manager	Run these cf CLI commands:
			`cf login`
			`cf create-space`
				See Create Spaces Using the Cloud Foundry Command Line Interface [page 2854].

Step No.	Task	Performed By	Do This	More Information

14.	Add Cloud Foundry org and space members.	Org manager	Run these cf CLI commands:
			`cf set-org-role <USERNAME> <ORG> <ROLE>`
			`cf set-space-role <USERNAME><ORG><SPACE><ROLE>`
				See Add Organization Members Using the Cloud Foundry Command Line Interface [page 2853] and Add Space Members Using the Cloud Foundry Command Line Interface [page 2855].

15.	Display all available services in the Cloud Foundry marketplace.	Org manager	Run this cf CLI command: `cf marketplace`	See Using Services in the Cloud Foundry Environment [page 271].

Using the btp CLI, you can perform account maintenance tasks, such as updating global account and subaccount details, deleting subaccounts, and deleting environment instances.

Subaccount members can also use the btp CLI to work with multitenant applications. See Working with Multitenant Applications Using the BTP CLI [page 2704].

Next Steps

Org/space members can create service instances, which are entitled to the subaccount, using also the `cf create-service <allowed-service-plan>` command in the cf CLI. Use the cf CLI command cf services to verify that the service instances exist.

For more information about developer tasks, see Development in the Cloud Foundry Environment [page 256].

Related Information

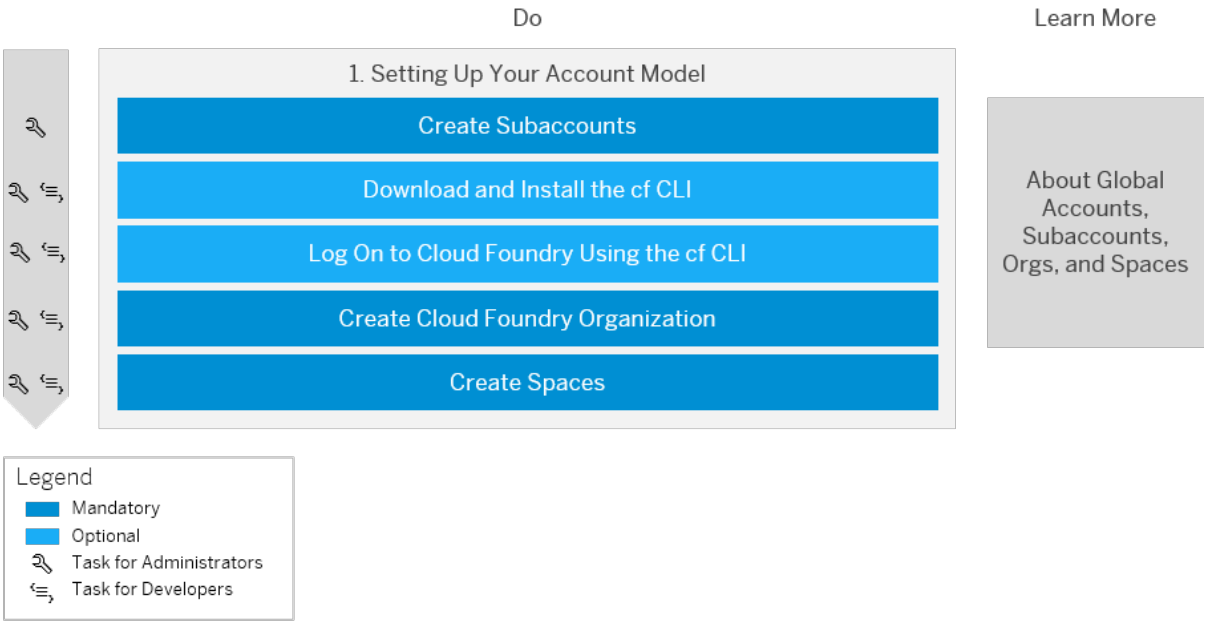
Command Syntax of the btp CLI [page 2671]
Download and Start Using the BTP CLI Client [page 2665]
Commands in the btp CLI [page 2698]

3.2.2 Getting Started with an Enterprise Account in the Cloud Foundry Environment

Quickly get started with an enterprise account in the Cloud Foundry Environment.

This topic focuses on how to get started with a customer or partner account using the SAP BTP cockpit. However, you can also perform these tasks using the CLI. See Setting Up a Global Account via the Command Line [page 180].

1. Setting Up Your Account Model



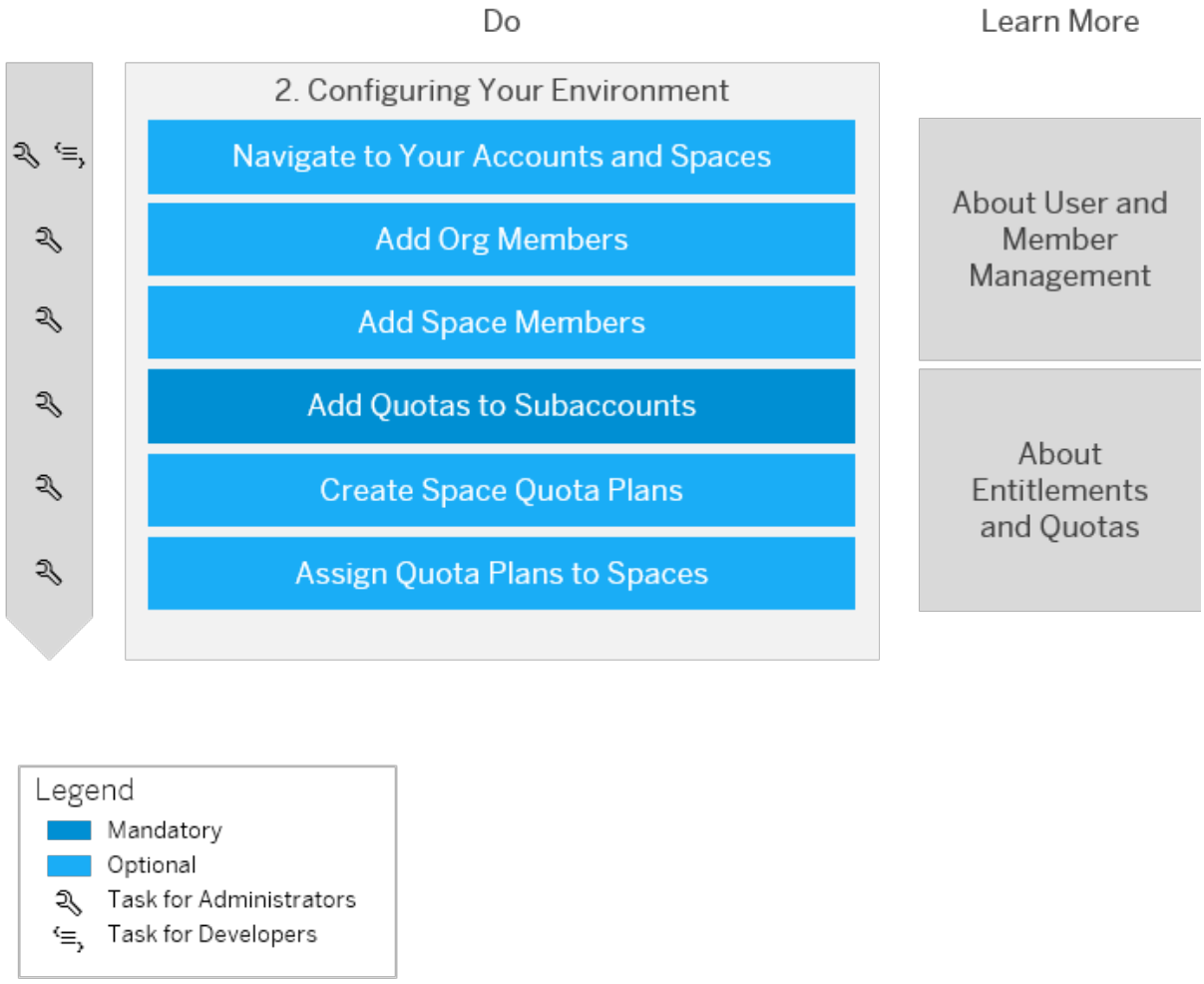
- Download and Install the Cloud Foundry Command Line Interface [page 2813]
- Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814]
- Create Spaces [page 2800]
- Global Accounts [page 121]
- Create a Subaccount [page 2509]
- Create Orgs [page 2793]

1. After you've received your logon data by email, create subaccounts in your global account. This allows you to further break down your account model and structure it according to your business needs. See Create a Subaccount [page 2509].
2. If you haven't done so already, now is a good time to download and install the Cloud Foundry Command Line Interface (cf CLI). This tool allows you to administer and configure your environment, enable services, and deploy applications. See Download and Install the Cloud Foundry Command Line Interface [page 2813].

But don't worry, you can also perform all the necessary task using the SAP BTP cockpit, which you don't need to install.

3. If you'd like to use the cf CLI, log on to your environment. See Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].
4. Create a Cloud Foundry organization in each of your subaccounts. See Create Orgs [page 2793]
5. Create spaces. See Create Spaces [page 2800].

2. Configuring Your Environment

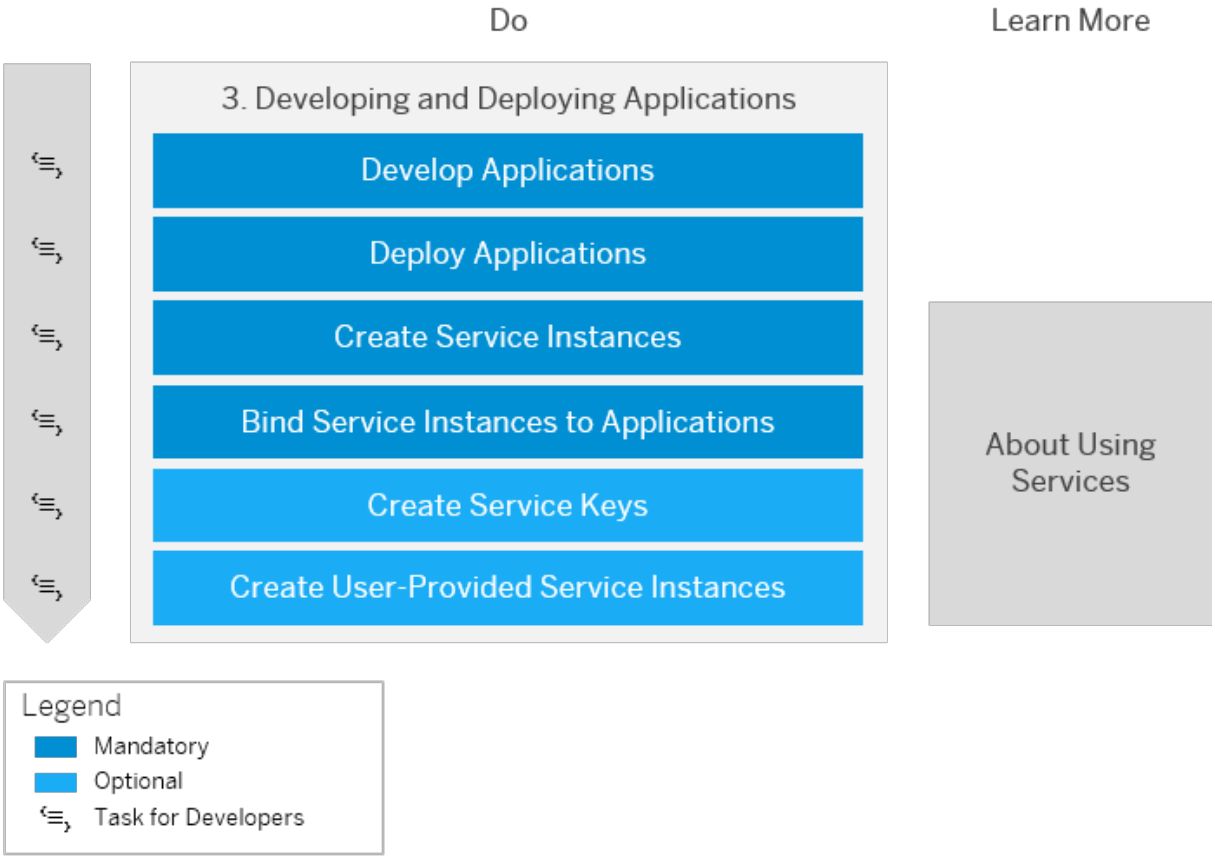


- User and Member Management [page 133]
- Navigate to Orgs and Spaces [page 2792]
- Add Org Members [page 2795]
- Add Space Members [page 2803]

- Configure Entitlements and Quotas for Subaccounts [page 2529]
- Create Space Quotas [page 2808]
- Assign Space Quotas to Spaces [page 2810]
- Managing Entitlements and Quotas Using the Cockpit [page 2521]

1. You can either use the cockpit or the cf CLI to configure your environment. If you'd like to use the cockpit, it's important you understand how you can navigate to your accounts and spaces. See Navigate to Orgs and Spaces [page 2792].
2. It's time to think about member management. You can add members at different levels. For example, you can add members at an org level. See Add Org Members [page 2795]. For more information about roles, see User and Member Management [page 133].
3. You can also add members at a space level. See Add Space Members [page 2803].
4. Before you can start using resources such as services or application runtimes, you need to manage your entitlements and add quotas to your subaccounts. See Configure Entitlements and Quotas for Subaccounts [page 2529]. To learn more about entitlements and quotas, see Managing Entitlements and Quotas Using the Cockpit [page 2521]. Note that if you want to try out services for free, you need to select free tier service plans if available. For a list of free services, check the SAP Discovery Center.
5. You can also assign quotas to different spaces within a subaccount. To do so, first create a space quota plan. See Create Space Quotas [page 2808] or Create Space Quota Plans Using the Cloud Foundry Command Line Interface [page 2857].
6. Then assign the quota plan to your space. See Assign Space Quotas to Spaces [page 2810] or Assign Quota Plans to Spaces Using the Cloud Foundry Command Line Interface [page 2858].

3. Developing and Deploying Applications



- Creating Service Instances [page 273]
- Binding Service Instances to Applications [page 276]
- Creating Service Keys [page 278]
- Using Services in the Cloud Foundry Environment [page 271]
- Deploy Business Applications in the Cloud Foundry Environment [page 845]
- Development [page 254]
- Creating User-Provided Service Instances [page 275]

1. Develop your application. Check out the Developer Guide for tutorials and more information. See Development [page 254].
2. Deploy your application. See Deploy Business Applications in the Cloud Foundry Environment [page 845].
3. Integrate your application with a service. To do so, first create a service instance. See Creating Service Instances [page 273]
4. Bind the service instance to your application. See Binding Service Instances to Applications [page 276].
5. Alternatively, you can also create and use service keys. See Creating Service Keys [page 278]. For more information on using services and creating service keys, see About Services [page 272].

6. You can also create instances of user-provided services. See Creating User-Provided Service Instances [page 275].

3.2.2.1 Setting Up a Global Account via the Command Line

Your global account is the entry point for managing the resources, landscape, and entitlements for your departments and projects in a self-service manner in SAP BTP. You can use the command-line tool **btp CLI** to set it up, and the **cf CLI** to manage Cloud Foundry instances.

Set up your account model using the btp CLI by creating subaccounts in your enterprise account. You can create any number of subaccounts in the Cloud Foundry environment and region.

To manage Cloud Foundry, i.e. for managing service instances and members in orgs and spaces, creating spaces, as well as assigning quota to orgs and spaces, you use the cf CLI.

Prerequisites

- You have purchased an enterprise global account.
- You have downloaded and extracted the following command-line tools:
  - SAP BTP command line interface (btp CLI). See Download and Start Using the BTP CLI Client [page 2665]
  - Cloud Foundry CLI (cf CLI). See Download and Install the Cloud Foundry Command Line Interface [page 2813]

Procedure

> → Tip
>
> In the btp CLI, you can view the command help of each command to get information about how to use the command, its syntax, and input parameters. See Get Help [page 2678].

Step No.	Task	Performed By	Do This	More Information
1.	Log in to your global account using the URL of the btp CLI server and the subdomain of your global account.	Global account administrator or viewer	Run this command in the btp CLI: `btp login`	See Log in [page 2679].

Step No.	Task	Performed By	Do This	More Information
2.	View details of your global account.	Global account admin or viewer	Run this command in the btp CLI: `btp get accounts/global-account`	See Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699].
3.	Add admins to your global account.	Global account admin	Assign the role collection `Global Account Administrator` to a user by running the following command in the btp CLI: `btp assign security/role-collection "Global Account Administrator" --to-user <user> --create-user-if-missing`	See Managing Users and Their Authorizations Using the btp CLI [page 2709] and Security Administration: Managing Authentication and Authorization [page 2537].
4.	View all the regions that are available to your global account and subaccounts.	Global account admin or viewer	Run this command in the btp CLI: `btp list accounts/available-region`	This command also provides information about the environments and infrastructure provider of each region.
5.	Create subaccounts in your global account.	Global account admin	Run this command in the btp CLI: `btp create accounts/subaccount --display-name <my-subaccount> --region <my-region> --subdomain <my-subaccount-subdomain>`	See Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699].
6.	View the details of the subaccounts in your global account.	Global account admin or viewer	Run this command in the btp CLI: `btp get accounts/subaccount <ID of new subaccount>`	See Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699].
7.	Add admins to your subaccounts.	Subaccount admin	Assign the role collection `Subaccount Administrator` to the user by running the following command in the btp CLI: `btp assign security/role-collection "Subaccount Administrator" --to-user <user> --create-user-if-missing`	See Managing Users and Their Authorizations Using the btp CLI [page 2709] and Security Administration: Managing Authentication and Authorization [page 2537].

Step No.	Task	Performed By	Do This	More Information
8.	View all the services and applications that are entitled to your global account, including quota information per service plan.	Global account admin or viewer	Run this command in the btp CLI: `btp list accounts/entitlement`	See Setting Entitlements Using the btp CLI [page 2702].

9.	Assign quotas to your subaccounts.	Global account admin	Run this command in the btp CLI:
			`btp assign accounts/entitlement --to-subaccount <my-subaccount-id> --for-service <my-service> --plan <my-service-plan> --amount <number>`
			Validate with this command:
			`btp list accounts/entitlement`
				See Setting Entitlements Using the btp CLI [page 2702].

10.	View all the entitlements in your subaccounts.	Subaccount admin or viewer	Run this command in the btp CLI: `btp list accounts/entitlement --subaccount <my-subaccount-id>`	See Setting Entitlements Using the btp CLI [page 2702].
11.	Create a Cloud Foundry org (environment instance) in your subaccounts.	Subaccount admin	Run this command in the btp CLI: `btp create accounts/environment-instance --subaccount <my-subaccount-id> --display-name <my-environment> --environment <cloudfoundry> --plan standard --parameters "{\"instance_name\":\"myOrg\"}"`	See Working with Environments Using the BTP CLI [page 2703] andOrg Management Using the SAP BTP Command Line Interface (BTP CLI) [page 2780] .
12.	View details of the environment instances in your subaccounts.	Subaccount admin or viewer	Run this command in the btp CLI: `btp list accounts/environment-instance --subaccount <my-subaccount-id>`	See Working with Environments Using the BTP CLI [page 2703].

13.	Create a Cloud Foundry space.	Org manager	Run these cf CLI commands:
			`cf login`
			`cf create-space`
				See Create Spaces Using the Cloud Foundry Command Line Interface [page 2854].

Step No.	Task	Performed By	Do This	More Information

14.	Add Cloud Foundry org and space members.	Org manager	Run these cf CLI commands:
			`cf set-org-role <USERNAME> <ORG> <ROLE>`
			`cf set-space-role <USERNAME><ORG><SPACE><ROLE>`
				See Add Organization Members Using the Cloud Foundry Command Line Interface [page 2853] and Add Space Members Using the Cloud Foundry Command Line Interface [page 2855].

15.	Display all available services in the Cloud Foundry marketplace.	Org manager	Run this cf CLI command: `cf marketplace`	See Using Services in the Cloud Foundry Environment [page 271].

Using the btp CLI, you can perform additional account maintenance tasks, such as updating global account and subaccount details, deleting subaccounts, and deleting environment instances.

Subaccount members can also use the btp CLI to work with multitenant applications. See Working with Multitenant Applications Using the BTP CLI [page 2704].

Next Steps

Org/space members can create service instances, which are entitled to the subaccount, using also the `cf create-service <allowed-service-plan>` command in the cf CLI. Use the cf CLI command cf services to verify that the service instances exist.

For further documentation about developer tasks, see Development in the Cloud Foundry Environment [page 256].

Related Information

Command Syntax of the btp CLI [page 2671]
Download and Start Using the BTP CLI Client [page 2665]
btp CLI Command Reference

3.3 Getting Started in the ABAP Environment

Get onboarded in the SAP Business AI Platform, ABAP environment. Follow the workflows for trial or customer accounts.

Getting Started with a Customer Account in the ABAP Environment [page 184]

After you have purchased a customer account, learn how to get started in the ABAP environment.

Getting Started with Custom Code Analysis in the ABAP Environment [page 246]
Learn how to get started with Custom Code Analysis in the SAP Business AI Platform, ABAP environment.

3.3.1 Getting Started with a Customer Account in the ABAP Environment

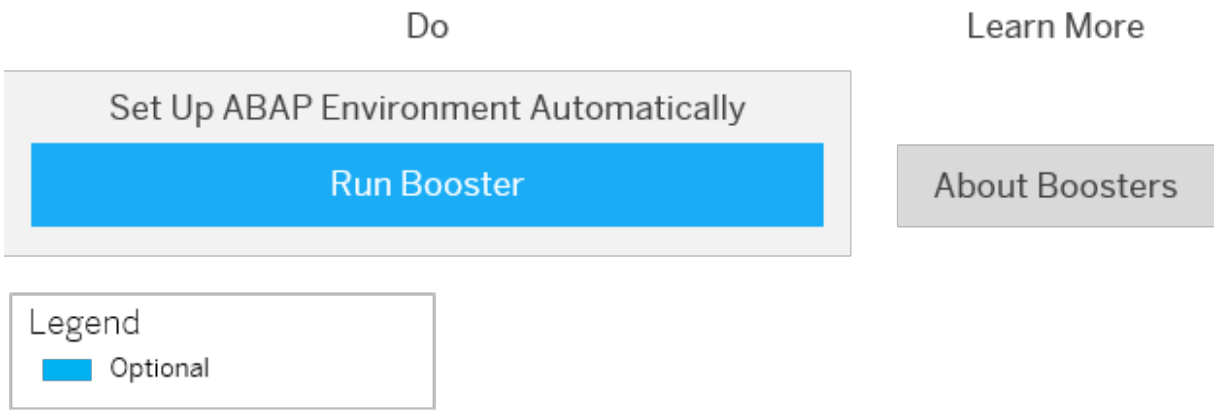
After you have purchased a customer account, learn how to get started in the ABAP environment.

> **Note**
>
> This documentation informs you about the first steps as an administrator in the ABAP environment. For more information about getting started as a developer, see Getting Started as a Developer in the ABAP Environment [page 241].

1. Logging on to SAP BTP

1. After you have purchased your customer account, you will receive an e-mail with a link to the home page of SAP BTP and the logon data as administrator for the global account.
2. Log on to SAP BTP.
   You are now in the SAP BTP cockpit.
3. Navigate to your global account.

2. Automated Initial Setup Using a Booster (Optional)



- Using a Booster to Automate the Setup of the ABAP Environment (Optional) [page 191]
- https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/fb1b56148f834749a2bf51127421610b.html [https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/fb1b56148f834749a2bf51127421610b.html]

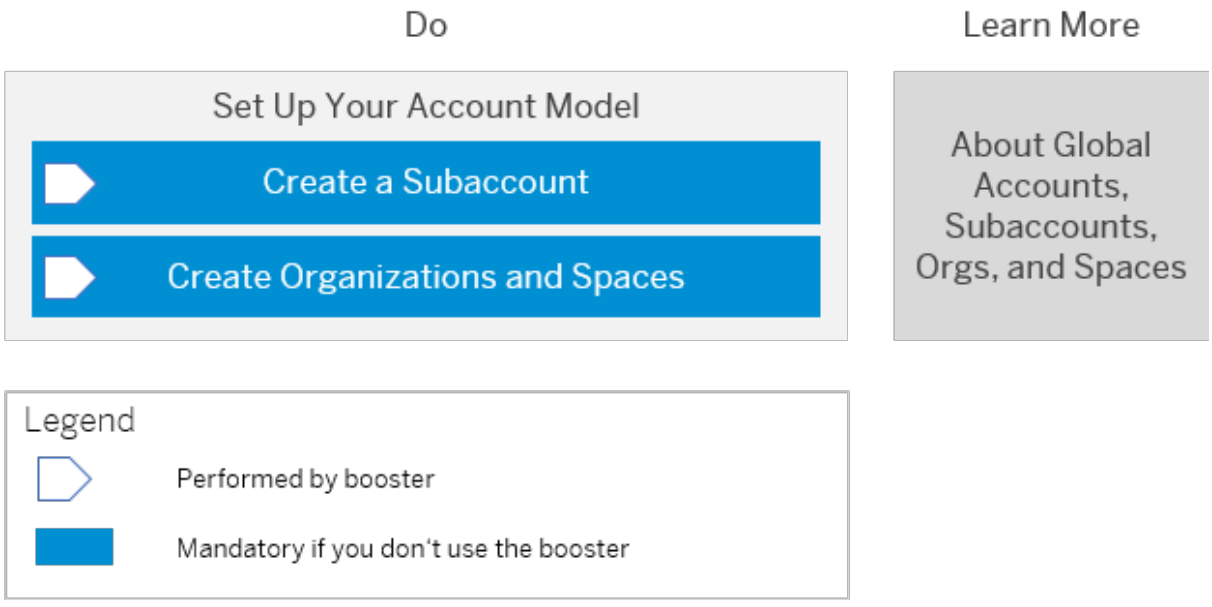
To set up a development system quickly, you can use the booster *Prepare an Account for ABAP Development*. The interactive booster guides you through the process of setting up your subaccounts, configuring entitlements, assigning members, and so on. For more information, see Using a Booster to Automate the Setup of the ABAP Environment (Optional) [page 191].

Using a booster is optional. If you don't use the booster, you must perform more steps manually.

3. Manual Setup

Even if you have used the booster, some manual setup steps are still required. In the following, steps that are performed by the booster are indicated.

a. Setting Up Your Account Model



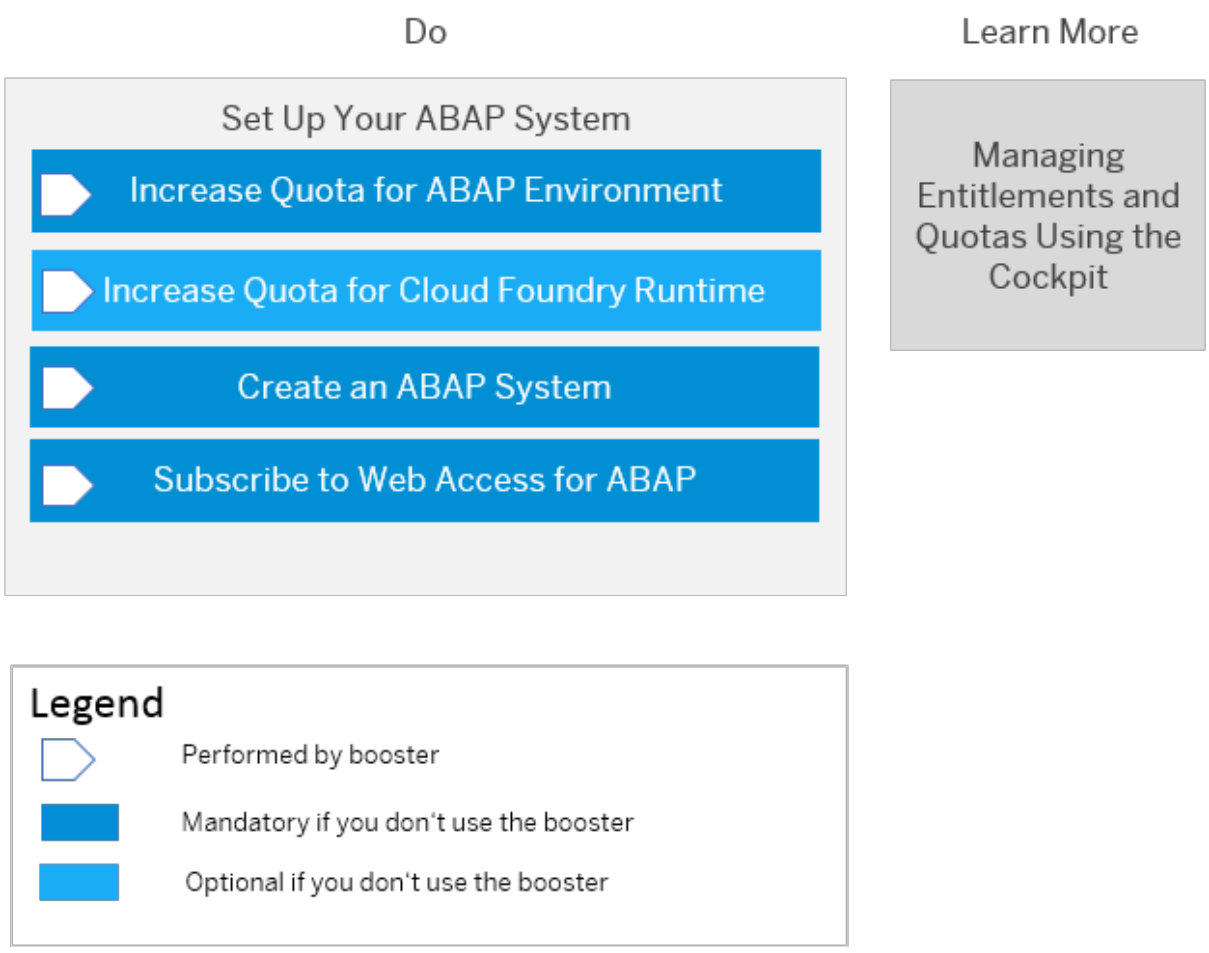
- Creating a Cloud Foundry Subaccount for the ABAP Environment [page 193]
- Creating a Cloud Foundry Organization and Space [page 194]
- https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/8ed4a705efa0431b910056c0acdbf377.html [https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/8ed4a705efa0431b910056c0acdbf377.html]

1. (Not required if you have used the booster):
   Create a subaccount for ABAP environment in your global account (see Creating a Cloud Foundry Subaccount for the ABAP Environment [page 193]).

This allows you to further break down your account model and structure it according to your business needs.

2. (Not required if you have used the booster):
   Create organizations and spaces (see Creating a Cloud Foundry Organization and Space [page 194]).
   If you want to learn more about subaccounts, orgs, and spaces, and how they relate to each other, see Account Model.

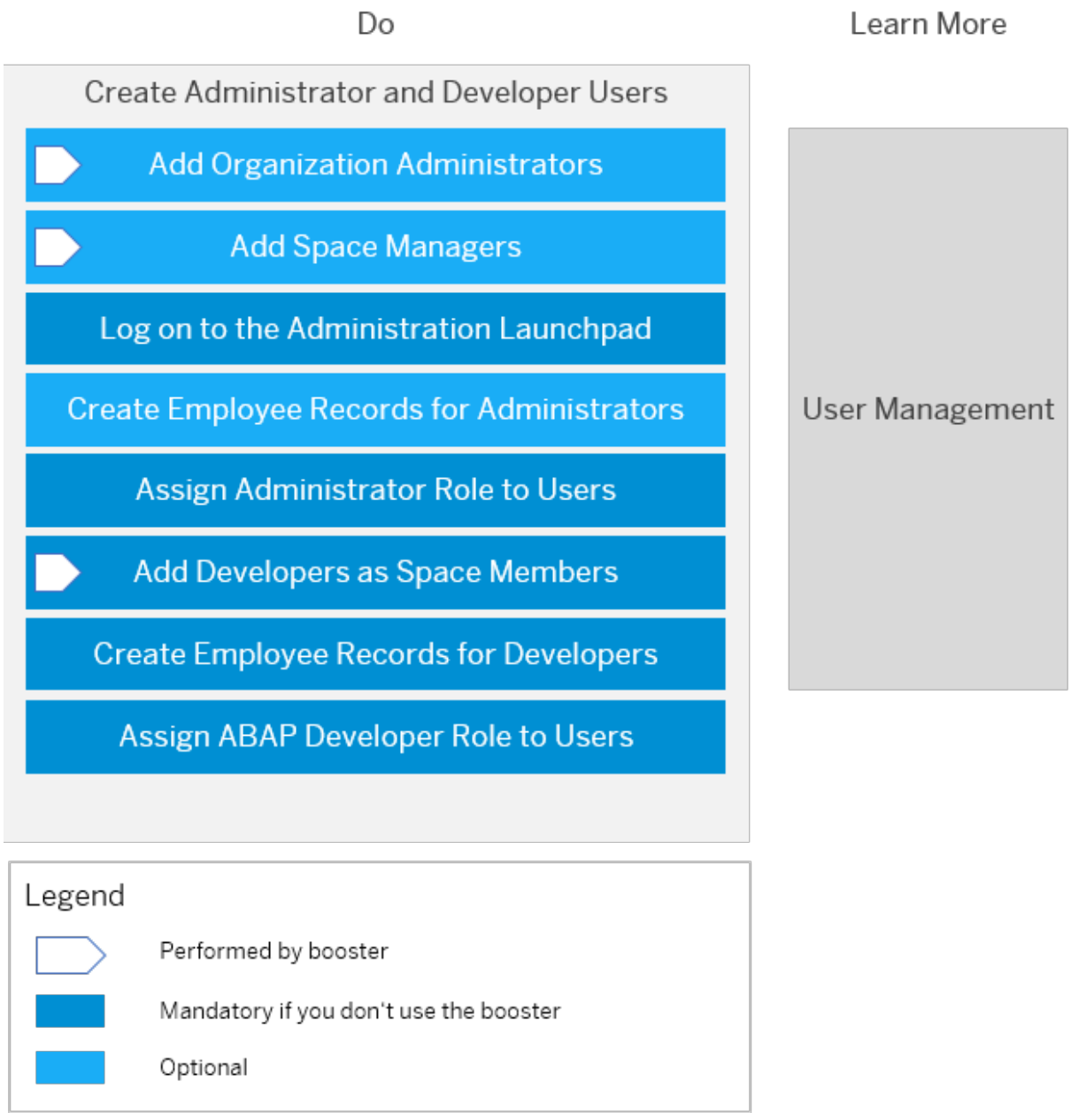
b. Setting Up the ABAP System



- Increasing the Quota for the ABAP Environment [page 195]
- Increasing the Quota for the Cloud Foundry Runtime (Optional) [page 196]
- Creating an ABAP System [page 198]
- Subscribing to the Web Access for ABAP [page 211]
- https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/c8248745dde24afb91479361de336111.html [https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/c8248745dde24afb91479361de336111.html]

1. (Not required if you have used the booster):
   Before you can start using resources such as services or application runtimes, you must manage your entitlements and add quotas to your subaccount (see Increasing the Quota for the ABAP Environment [page 195]).
2. (Not required if you have used the booster):
   Optionally, if your developers want to deploy their own apps in Cloud Foundry, increase the quota for the Cloud Foundry runtime (see Increasing the Quota for the Cloud Foundry Runtime (Optional) [page 196]).
3. (Not required if you have used the booster):
   Create your ABAP system (see Creating an ABAP System [page 198]).
4. (Not required if you have used the booster):
   Subscribe to the Web access for ABAP SaaS application to get direct browser access to your instances in the ABAP environment (see Subscribing to the Web Access for ABAP [page 211]).
   This also allows you to access the administration launchpad including your own SAP Fiori applications. You only have to subscribe once for each subaccount.

c. Creating Administrator and Developer Users



- Adding a User as Org Manager for the Cloud Foundry Organization [page 212]
- Adding a User as Space Manager for the Cloud Foundry Space [page 213]
- Logging on to the Administration Launchpad of the ABAP Environment [page 213]
- Creating an Employee Record for a New Administrator [page 214]
- Assigning the ABAP Environment Administrator Role to the New Administrator User [page 215]
- Creating New Space Members and Assigning Space Developer Roles to Them [page 216]
- Creating an Employee Record for a New Developer [page 216]

- https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/cc1c676b43904066abb2a4838cbd0c37.html [https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/cc1c676b43904066abb2a4838cbd0c37.html]
- Assigning the ABAP Developer User to the ABAP Developer Role [page 217]

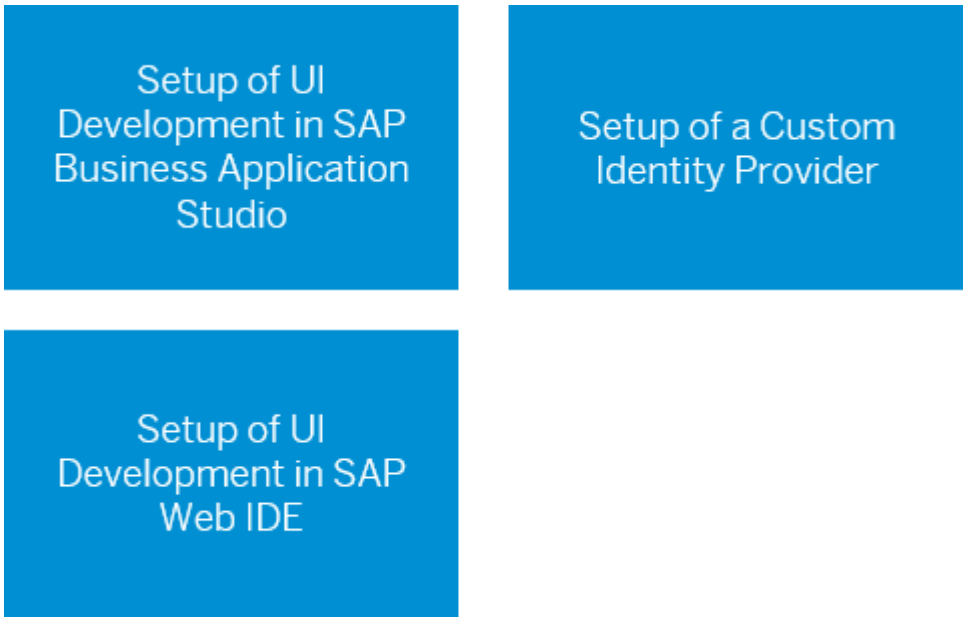
1. Optionally, you can create additional administrator users for the ABAP environment as follows:
    1. (Not required if you have used the booster):
    Add users as administrators of the Cloud Foundry organization that you created for the ABAP environment (see Adding a User as Org Manager for the Cloud Foundry Organization [page 212]).
    2. (Not required if you have used the booster):
    Add the users as managers for the Cloud Foundry space that you created for the ABAP environment (see Adding a User as Space Manager for the Cloud Foundry Space [page 213]).
    3. Log on to the administration launchpad (see Logging on to the Administration Launchpad of the ABAP Environment [page 213]).
    4. Create employee records for the new administrators (see Creating an Employee Record for a New Administrator [page 214]).
    5. Assign the administrator role for the ABAP environment to the new users (see Assigning the ABAP Environment Administrator Role to the New Administrator User [page 215]).
2. For your developers, create business users and assign them to the developer business role as follows:
    1. (Not required if you have used the booster):
    Create new members for the space that you created for the ABAP environment and assign developer roles to the new members (see Creating New Space Members and Assigning Space Developer Roles to Them [page 216]).
    2. Create employee records for the new developer users (see Creating an Employee Record for a New Developer [page 216]).
    3. Assign the ABAP developer users to the ABAP developer role (see Assigning the ABAP Developer User to the ABAP Developer Role [page 217]).

4. What's Next

After the initial setup of the ABAP environment, your developers can now develop applications using ABAP Development Tools. You can tell your developers to get started (see Getting Started as a Developer in the ABAP Environment [page 241]).

As next steps, you can optionally set up additional tools for your developers or ensure the integration with other systems:

- If your developers need a UI development tool, set up SAP Business Application Studio (see Setup of UI Development in SAP Business Application Studio (Optional) [page 218]).
- As a default identity service in the ABAP environment, you get SAP ID. You might want to set up a custom identity service (see Setup of a Custom Identity Service [page 226]).
- As an alternative to SAP Business Application Studio, you can also set up SAP Web IDE (see Setup of UI Development in SAP Web IDE (Optional) [page 232]).



- Setup of UI Development in SAP Business Application Studio (Optional) [page 218]
- Setup of a Custom Identity Service [page 226]
- Setup of UI Development in SAP Web IDE (Optional) [page 232]

Related Information

Learning Journey
Discovery Center
SAP Community
SAP Road Map Explorer
Video Tutorials

3.3.1.1 Using a Booster to Automate the Setup of the ABAP Environment (Optional)

You can use a booster to automate some of the required steps for setting up the ABAP environment. Automation includes creating a subaccount and space, configuring the required entitlements, and assigning administrators and developers to the subaccount.

Context

A booster is a set of guided interactive steps that enable you to select, configure, and consume services on SAP BTP. For more information, see Boosters.

> ⚠ Caution
>
> The booster is only intended to create development systems. For non-development systems, follow the steps to set up the ABAP environment manually (see Getting Started with a Customer Account in the ABAP Environment [page 184]).

Procedure

1. Log on to the SAP BTP cockpit and choose the global account for the Cloud Foundry environment as administrator.
2. From the navigation menu, choose *Boosters*.
3. Choose the booster *Prepare an Account for ABAP Development*.

   The tab pages *Overview*, *Components*, and *Additional Resources* are displayed, where you get more information about the booster.
4. Choose *Start*.
5. After the booster wizard has successfully checked the prerequisites, choose *Next* to continue.
6. On the *Select Scenario* dialog, choose whether you'd like the booster to create your ABAP environment service instance in a new, or in an existing subaccount.
7. On the *Configure Subaccount* dialog, entitlements and quotas for the new subaccount are proposed, but you can change them if needed:
   - For the *ABAP environment* service, the service plan *standard* enables you to size ABAP server and persistence independently from each other in 16 GB units. These units are represented in the quota plans *abap_compute_unit* and *hana_compute_unit*.
     As part of the default quota assignment, you get at least 1 compute unit in the *abap_compute_unit* service plan and at least 2 compute units in the *hana_compute_unit* service plan. This corresponds to the minimum configuration for an instance of the ABAP Environment service. You can also choose higher quotas in the dialog of the booster.
   - For the *Web Access to ABAP* service, a default plan is chosen that you cannot change. You need the service and the quota for direct browser access to your instances in the ABAP environment, including access to the administration launchpad for ABAP.

- Depending on what you have ordered for your account, additional entitlements for the services *Cloud Foundry Runtime*, **SAP Build Work Zone, standard edition**, and *SAP Business Application Studio* might be shown. They are optional; you can remove the entitlements if you don't need them right now and want to add them later:
  - A quota for the Cloud Foundry runtime is only needed for the ABAP environment if your developers want to deploy their own apps in Cloud Foundry.
  - The SAP Build Work Zone, standard edition enables organizations to establish a central point of access to SAP, custom-built, and third party applications and extensions, both in the cloud and on premise.
  - SAP Business Application Studio is a UI development environment.

8. On the *Configure Subaccount* dialog, follow the instructions on the screen to enter subaccount name, provider, region, and so on.
9. On the *Add Users* dialog, add the e-mail addresses of new users in the ABAP environment.

   This step allows you to quickly create additional administration users for the ABAP environment, if needed, and developer users. In the *Origin* field, you can see the identity provider for the ABAP environment. By default, this is SAP ID service (*sap.ids*), but if you have set up a custom identity provider, you can choose this provider from the dropdown list.

   > ⓘ Note
   >
   > If the optional entitlement for SAP Build Work Zone, standard edition is selected in the *Configure Subaccounts* step, please select a custom identity provider for applications. If you choose the default identity provider, the subscription to the SAP Build Work Zone, standard edition application will fail. Refer to Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services for more information.

10. On the *Configure ABAP Environment Instance* dialog, enter a 3-character ID for your ABAP system.
11. Review your settings and finish the booster.

Results

After the booster has run successfully, the following tasks have been performed automatically:

- A new Cloud Foundry subaccount for the ABAP environment is created and enabled.
- A space and an organization for the ABAP environment are available.
- A service instance for the ABAP environment has been created.
- A system ID of your choice has been assigned to the ABAP system.
- An identity provider (SAP ID service or a custom identity provider) has been set up for the ABAP environment.
- For the *ABAP environment* service, quotas have been distributed for the ABAP server and the SAP HANA persistence. The quotas that you have chosen are deducted from the available quota for each service in your global account..
- If you have entitlements for SAP Build Work Zone, standard edition and SAP Business Application Studio, then you have subscribed to these services and standard quotas have been assigned accordingly. Entitlements for subscription services are assigned to the subaccount.
- You're subscribed to the Web access for ABAP SaaS application and get direct browser access to your instances in the ABAP environment. Entitlements for subscription services are assigned to the subaccount.

- Additional users with Cloud Foundry organization manager and space manager roles have been added. If you have ordered the SAP Build Work Zone, standard edition as a service, the additional administration users have also been assigned to the role collection *Launchpad_Admin*.
- Additional users for developers with space developer role have been added. If you have ordered SAP Business Application Studio as a service, the additional developer users have also been assigned to the role collection *Business_Application_Studio_Developer*.

> ### Note
>
> The booster speeds up the initial setup, but there are still some tasks that you need to perform after the booster has run.
>
> In the following, you can find documentation for all necessary intial tasks, including the tasks that you don't need to perform if you have run the booster successfully. A note at the beginning of each task indicates that you can skip it if you've used the booster.

3.3.1.2 ABAP Environment: Initial Settings

With the initial settings, you as an administrator and your ABAP developers can get started in the ABAP environment.

After the initial settings are done, the ABAP environment is ready for your ABAP developers so they can implement business services using ABAP Development Tools (ADT).

3.3.1.2.1 Creating a Cloud Foundry Subaccount for the ABAP Environment

Create a Cloud Foundry subaccount for the ABAP environment in your global account using the cockpit.

Context

> ### Note
>
> If you have run the booster *Prepare an Account for ABAP Development*, you can skip these steps.

For the ABAP environment, you need a Cloud Foundry subaccount as a technical environment. If you want to reuse an existing Cloud Foundry subaccount, make sure that it uses the providers and regions that are available for provisioning an ABAP environment. For more information about the providers and regions for ABAP environment, see Regions.

Procedure

1. From your global account, choose *New Subaccount*.
2. Specify a display name, for example, **ABAP**.
3. **Optional:** Enter a description.
4. Leave the *Neo Environment* checkbox deselected.
5. Choose the desired infrastructure provider and region for your subaccount.

   You can find the available providers and regions for the ABAP environment in Regions.
6. Enter a subdomain for your subaccount. The subdomain will become part of the URL for accessing applications that you subscribe to from this subaccount.

   > ⓘ Note
   >
   > You can choose any string of your choice. However, note that the subdomain can contain only letters, digits and hyphens (not allowed in the beginning or at the end), and must be unique across all subaccounts in the same region. Uppercase and lowercase letters can be used, however that alone does not qualify as a means to differentiate subdomains (for example, **`SUBDOMAIN`** and **`subdomain`** are considered to be the same).
7. If your subaccount is to be used for production purposes, select the *Used for production* option.
8. Choose *Create*.

3.3.1.2.2 Creating a Cloud Foundry Organization and Space

Create a Cloud Foundry organization and Cloud Foundry space that will correspond to the instance of the ABAP environment.

Context

> ⓘ Note
>
> If you have already created an organization and space for the ABAP environment, you can skip these steps. If you have run the booster *Prepare an Account for ABAP Development*, you can also skip these steps.

For more information about creating Cloud Foundry spaces, see Create Spaces.

Procedure

1. Log on to the SAP BTP cockpit in the global account for the Cloud Foundry environment as administrator.
2. Choose the tile of the Cloud Foundry subaccount.

3. Check whether Cloud Foundry is enabled: If it's enabled, the Cloud Foundry organization name appears as organization on the screen area *Cloud Foundry*.
4. If Cloud Foundry is not enabled yet, proceed as follows:
    a. Choose *Enable Cloud Foundry*.
    b. On the *Create Cloud Foundry Organization* dialog, enter a name for the Cloud Foundry organization and choose *Create*.
    c. Choose the free or standard plan for the Cloud Foundry runtime according to your needs.
5. On the *Cloud Foundry* screen area, choose the link for spaces.
6. If the Cloud Foundry space in the Cloud Foundry organization does not exist yet, proceed as follows:
    a. Choose *New Space*.
    b. On the *New Space* dialog, enter a name for the new Cloud Foundry space, for example, **dev**, and leave the checkboxes for space manager and space developer checked.
    c. Choose *Create*.

3.3.1.2.3 Increasing the Quota for the ABAP Environment

Before you can create a service instance for the ABAP environment, you must assign some of the available quota to the subaccount for the ABAP environment.

Context

> ⓘ Note
>
> If you have run the booster *Prepare an Account for ABAP Development*, you can skip this step.

If you are working in an enterprise account, you need to add quotas to the services that you purchased in your subaccount before they can appear in the service marketplace. For more information, see Configure Entitlements and Quotas for Subaccounts.

Procedure

1. Log on to the SAP BTP cockpit as administrator.
2. Go to your global account.
3. From the navigation area, choose *Entitlements* > *Entity Assignments*.
4. Enter the subaccount for the ABAP environment and choose *Go*.
5. Choose *Configure Entitlements*.
6. Choose *Add Service Plans*.

    In the following popup, the available entitlements for this subaccount are shown.

7. For the *ABAP environment* entitlement, select the plans *abap_compute_unit*, *standard*, and *hana_compute_unit*.

   With the selection of the *standard* service plan, you can size the ABAP runtime and the SAP HANA memory independently from each other. To be able to do your sizing, you must also select the quota plans *abap_compute_unit* and *hana_compute_unit*.
8. Choose *Add 3 Service Plans*.
9. On the following screen, increase the quotas in the *abap_compute_unit* service plan at least by 1 and in the *hana_compute_unit* service plan at least by 2.

   The minimum configuration of 1 ABAP compute unit and 2 HANA compute units is sufficient in many cases to create one service instance for the ABAP environment. If you need more resources, choose higher quotas. Keep in mind that increasing the SAP HANA Cloud storage using the parameter `size_of_persistence_disk` consumes 0.002 HANA compute units for each GB that exceeds the minimal size of the persistence disk.
10. Choose *Save*.

Results

After you have assigned an entitlement for the three plans with respective quotas to your subaccount, you can create an ABAP system. This is the point where you can then decide how many blocks of ABAP compute units and HANA compute units you want to assign to your new system. See Creating an ABAP System [page 198].

Related Information

ABAP Compute Units [page 3438]

3.3.1.2.4 Increasing the Quota for the Cloud Foundry Runtime (Optional)

Check if your subaccount has a quota for the Cloud Foundry runtime and increase it, if needed.

Context

> **Note**
>
> If you have run the booster *Prepare an Account for ABAP Development*, you can skip this step.

A quota for the Cloud Foundry runtime is optional. It's only needed for the ABAP environment if your developers want to deploy their own apps in Cloud Foundry.

Procedure

1. Log on to the SAP BTP cockpit as Cloud Foundry administrator.
2. Go to your global account.
3. From the navigation area, choose *Entitlements* > *Entity Assignments*.
4. If there is no entry for the Cloud Foundry runtime, choose *Configure Entitlements* and *Add Service Plans*.
5. In the following popup, proceed as follows:
   a. Choose *Cloud Foundry Runtime*.
   b. Under *Available Service Plans*, select the checkbox *MEMORY*.
   c. Choose *Add 1 Service Plan*.
6. Choose + to add at least 1 to the subaccount.
7. Choose *Save*.

3.3.1.2.5 Selecting a Service Plan for the Web Access for ABAP

Before you can subscribe to the Web Access for the ABAP environment, you must select the relevant service plan.

Context

> ⓘ Note
>
> If you have run the booster *Prepare an Account for ABAP Development*, you can skip this step.

If you are working in an enterprise account, you need to add quotas to the services that you purchased in your subaccount before they can appear in the service marketplace.

Procedure

1. Log on to the SAP BTP cockpit as administrator.
2. Go to your global account.
3. From the navigation area, choose *Entitlements* > *Entity Assignments*.
4. Enter the subaccount for the ABAP environment and choose *Go*.
5. Choose *Configure Entitlements*.
6. Choose *Add Service Plans*.

   In the following popup, the available entitlements for this subaccount are shown.

7. For the *Web Access for ABAP* entitlement, select the plan *default (Application)*.
8. Choose *Add 1 Service Plan*.
9. Choose *Save*.

3.3.1.2.6 Creating an ABAP System

Create a service instance for the ABAP environment from the Service Marketplace.

Prerequisites

You have increased the quota for the ABAP environment. See Increasing the Quota for the ABAP Environment [page 195].

Context

> ⓘ Note
>
> If you have run the booster *Prepare an Account for ABAP Development*, you can also skip this step.

For more information about creating service instances, see Create Service Instances Using the Cockpit.

Procedure

1. Log on to the SAP BTP cockpit and navigate to the Cloud Foundry subaccount. See Navigate in the Cockpit .
2. From the navigation area, choose *Services* > *Service Marketplace.*

   You see a list of all services that are available to you.
3. Choose *ABAP environment*.
4. Choose *Create*.

   A wizard opens that helps you create your instance.

   > → Tip
   >
   > To see a list of all instances that have already been created in your subaccount, choose *Services* > *Instances and Subscriptions* from the navigation area. Here, you can also create a new ABAP environment instance.
5. Select the `standard` service plan.

6. Choose *Cloud Foundry* as your runtime environment.
7. Select a space.
8. Enter a CLI-friendly instance name, and choose *Next*.

   The instance name identifies the service instance in the Cloud Foundry environment. Specify an instance name that is unique among all the service instances in a space and only contains alphanumeric characters (A-Z, a-z), periods, underscores, and hyphens.
9. Provide additional instance parameters for the configuration by either using the default **form** or by uploading a **JSON file** from your computer or by specifying the parameters in JSON format. Here is a list of all parameters:

Parameter `(technical name in parenthesis)`	**Description**	**Available in Service Plans**
ABAP Runtime Size per Application Server `(size_of_application_server)`	The *ABAP Runtime Size per Application Server* refers to the size of a single ABAP application server in an ABAP environment service instance. It's specified in ABAP compute units (ACUs), with one ACU representing 16 GB. This parameter doesn't consume additional `abap_compute_unit` quota. Instead, it determines the distribution of the quota defined via the total ABAP runtime size among the application servers. Possible values are 0.5, 2, or 'auto', which is the default. A higher value results in fewer, but larger, application servers. Conversely, a lower number results in more, but smaller, application servers for the same total ABAP runtime size. If 'auto' is chosen, then 0.5 ACU is assigned per application server if `size_of_runtime` is < 4 for systems without elastic scaling, or `size_of_runtime` < 8 for systems with elastic scaling. Otherwise, 2 ACUs are assigned per application server. Every ABAP environment instance has between 2 and 32 application servers of the same size each (except for instances with a single application server, see the note below). Therefore, the possible value for the ABAP runtime size per application server depends on the total ABAP runtime size. For instance, `size_of_application_server` = 2 can only be set if the `size_of_runtime` is at least 4 ACUs. On the other hand, `size_of_application_server` = 0.5 can only be set if the `size_of_runtime` isn't higher than 8 ACUs.	• standard • build-runtime • saas_oem

Parameter (technical name in parenthesis)	Description	Available in Service Plans
	To decide what value to choose, consider the following: • Servers of size 0.5 ACUs allow more fine-granular scaling and are suitable for most use cases. • Larger servers allow slightly more user sessions per ACU. This might be more efficient in systems with very high ACU consumption. They're also a precondition for some exceptional use cases, such as ATC checks for very large ABAP programs. For more information, see ABAP Compute Units. **⚠ Caution** Instances with a *Total ABAP Runtime Size* of 0.5 ACU always have only one application server with an *ABAP Runtime Size per Application Server* of 0.5 ACU. This is an exception to the standard setup in which every service instance runs with at least two application servers. While having instances with a total ABAP runtime size of 0.5 ACU lowers costs, it also increases the likelihood of short outages and shouldn't be used for production systems.	
ABAP System Description **(description)**	The **ABAP system description** is optional.	• standard • build-runtime • saas_oem • free
ABAP System ID **(sapsystemname)**	The **ABAP system ID** must consist of exactly three alphanumeric characters. Only uppercase letters are allowed. The first character must be a letter (not a digit). The ID does not have to be technically unique.	• standard • build-runtime • saas_oem • free
Admin Email Address **(admin_email)**	The **admin email address** is used to create the initial user for the ABAP system automatically, including the assignment of the administrator role to this user. You can access the ABAP environment system only with this specified user. By default, the email address is used as subject name identifier.	• standard • build-runtime • saas_oem • free

Parameter (`technical name in parenthesis`)	Description	Available in Service Plans
Admin Employee ID (`admin_employee_id`)	The **admin employee ID** is an optional parameter used to provide the employee ID for the initial user. You can either set it directly by using this parameter during instance creation or after instance creation in the Maintain Employees Fiori application of the ABAP environment service instance. Once set, it cannot be changed anymore. The employee ID must only contain uppercase letters, digits, underscores, hyphens and periods, and must not be longer than 60 characters.	• standard • build-runtime • saas_oem • free
Admin User Name (`admin_user_name`)	The **admin user name** is an optional parameter used to provide the user name for the initial user. It must be provided if the **login attribute** is set to `user_name` (see below).	• standard • build-runtime • saas_oem •
Customer-Managed Keys (`customer_managed_keys`)	The **customer-managed keys** parameter defines whether or not to integrate the ABAP environment instance and its SAP HANA Cloud database with the SAP Key Management Service (KMS). The enablement of this integration allows you to manage the encryption keys via SAP KMS. The integration with SAP KMS is established on the level of the SAP BTP subaccount where the ABAP environment instance is created. For more information, see Integration with SAP Key Management Service [page 210]. ⚠ Caution Once the integration is enabled, it can't be disabled anymore.	• standard • build-runtime
Development System (`is_development_allowed`)	The **development system** checkbox is checked by default. By using this setting, you can control the changeability of development objects in the system. If you want to protect all your customer-related software components and ABAP namespaces against manual changes via ABAP development tools for Eclipse, uncheck the box. This setting is used for test and productive systems, where changes must be imported only. For information about which business catalogs are available in development systems only, see Business Catalogs for Development Tasks [page	• standard • build-runtime • saas_oem

Parameter (`technical name in parenthesis`)	Description	Available in Service Plans
	2928]. This setting determines the automatic selection of the respective system upgrade approach by SAP. Non-development systems are automatically chosen for the downtime-optimized release upgrade and hotfix import procedures. Please refer to this blog post for further information: Downtime-Optimized Upgrades and Hotfix Collection Imports for Production.	
Elastic Scaling of the ABAP Application Server (`elastic`)	The **elastic scaling of the ABAP application server** parameter defines whether or not to adapt the number of ABAP application servers dynamically, depending on the system load. By default, the checkbox is not checked (`elastic = false`), and the service instance will have a fixed number of ABAP application servers with a total ABAP runtime size as defined by the parameter **Total ABAP Runtime Size** (see above). If you enable elastic scaling by checking the checkbox (`elastic = true`), application servers will be automatically added to the service instance when the load increases and removed when the load decreases. More specifically, the number of application servers will scale between the number of ACUs (ABAP compute units) needed for two application servers as a minimum (1 ACU if `size_of_application_server` = 0.5 or 4 ACU if `size_of_application_server` = 2), and the number of ACUs configured in the **Total ABAP Runtime Size** parameter (see above) as a maximum. This feature considerably helps in cost-saving as charges are only levied based on the actually allocated ACUs, and not on the configured maximum. Still, to ensure that defined quotas are not exceeded, the maximum number of ACUs that is set in the **Total ABAP Runtime Size** parameter is allocated from the available ACU quota for the entire duration that the service instance is provisioned.	• standard • build-runtime • saas_oem

Parameter (`technical name in parenthesis`)	Description	Available in Service Plans
	The scaling typically occurs within a couple of minutes. Especially when a service instance is scaled down, there is a grace period of a few minutes to allow the completion of short-lived requests. Before enabling elastic scaling in production, we advise to test your typical workload patterns and monitor the embedded *Health Monitoring* app to verify the suitability of this cost-saving measure. Maintaining a higher number of statically configured ACUs might be better for handling sudden large spikes in the system load.	
HANA Cloud Disk Size **(`size_of_persistence_disc`)**	The **HANA Cloud disk size** refers to the disk size in GB of the SAP HANA Cloud database used by the ABAP environment service instance. If the parameter is set to `auto`, the SAP HANA Cloud storage size is set to the minimal value `40 * size_of_persistence + 40`. The maximum allowed value is `120 * size_of_persistence + 40`. If you set a higher value, it will consume 0.002 HANA compute units (HCU) for any GB exceeding the minimal default size of the persistence disk. Therefore, the HCU ratio of additional storage disk to RAM is 1 : 31.25 per GB (as 1 HCU = 16 GB).	• standard • build-runtime • saas_oem
HANA Cloud Memory Size **(`size_of_persistence`)**	The **HANA Cloud memory size** refers to the memory size of the SAP HANA Cloud database used by the ABAP environment service instance. The size is specified in number of HANA compute units that should be used from the quota plan *hana_compute_unit*, with one HANA compute unit representing a block size of 16 GB for the underlying SAP HANA Cloud instance. The supported number of hana_compute_unit per HANA instance is 2, 4, 8, 16, 32, or 64.	• standard • build-runtime • saas_oem
Login Attribute **(`login_attribute`)**	Using an email address as subject name identifier might not be possible if the e-mail address is ambiguous across users, or if the trusted identity provider configured for authentication in the subaccount of the ABAP environment instance is already configured with the subject name identifier `Login Name`. In	• standard • build-runtime • saas_oem • free

Parameter (`technical name in parenthesis`)	Description	Available in Service Plans
	this case, you can change the **login attribute** to `user_name`. In addition, provide the user name for the initial user in the (`admin_user_name`) **admin user name** (see above). **⚠ Caution** Currently, it's only possible to change the login attribute **when creating a new ABAP system**. To change the settings **afterwards**, please open a ticket on component BC-CP-ABA.	
Multi-Availability Zones **(`multi_availability_zones`)**	The **Multi-Availability Zones** define the level of resilience of the ABAP environment service instance. When enabled, the ABAP application servers are distributed across multiple availability zones to increase resilience, and two SAP HANA Cloud database instances are created in different zones with synchronous replication and automatic takeover in a case of failure. Additionally, system components are restarted with higher priority than for single-zone systems. Enabling this option doesn't change the ABAP compute unit (ACU) consumption, but doubles the HANA compute unit (HCU) consumption due to the second database instance. For more information, see Resilience of an ABAP Environment System [page 208].	• standard • build-runtime • saas_oem
Total ABAP Runtime Size **(`size_of_runtime`)**	The **Total ABAP runtime size** refers to the runtime size of the ABAP environment service instance. This is the sum of the runtime size of all ABAP application servers of an ABAP environment service instance. The size is specified in number of ABAP compute units that should be used from the quota plan *abap_compute_unit*, with one ABAP compute unit representing 16 GB. The supported number of abap_compute_unit is 0.5, 1, 1.5, 2, 3, 4, 4.5, 6, 8, 16, 24, 32, or 64. The total ABAP runtime size has to be less or equal to twice the size of the	• standard • build-runtime • saas_oem

Parameter (`technical name in parenthesis`)	Description	Available in Service Plans
	SAP HANA Cloud memory size (size_of_runtime ≤ 2 * size_of_persistence). For more information, see ABAP Compute Units [page 3438]. The total ABAP runtime size should be a whole-number multiple, ranging from 2 to 32 times the size of the ABAP runtime size per application server. This is because each ABAP environment instance contains between 2 and 32 application servers, all of the same size (except for instances with a single application server, see the note below). For example, if the `size_of_application_server` is 2, the `size_of_runtime` should be at least 4 ACUs. Conversely, if the `size_of_application_server` is 0.5, the `size_of_runtime` shouldn't exceed 8 ACUs. Another example: if the `size_of_application_server` is 2, setting the `size_of_runtime` to 5 won't work. This is because a total runtime size of 5 ACUs can't be split into three application servers with 2 ACUs each. **⚠ Caution** Instances with a *Total ABAP Runtime Size* of 0.5 ACU always have only one application server with an *ABAP Runtime Size per Application Server* of 0.5 ACU. This is an exception to the standard setup in which every service instance runs with at least two application servers. While having instances with a total ABAP runtime size of 0.5 ACU lowers costs, it also increases the likelihood of short outages and shouldn't be used for production systems.	

ⓘ Note

Make sure that you don't choose more compute units than you have assigned to your subaccount for the ABAP environment (see Increasing the Quota for the ABAP Environment [page 195]).

For more information about ABAP system sizing and Native Storage Extension for selected SAP HANA tables, see ABAP System Sizing [page 3426] and Native Storage Extension for the ABAP Environment [page 3430].

> ⚠ Caution
>
> Be aware that not all parameters can be changed if you update your ABAP system. For a list of updatable parameters, see Updating an ABAP System [page 206].

10. Choose *Next* to review and verify your instance details.
11. Choose *Create*.

    The ABAP environment instance is being set up, which might take up to 60 minutes. Wait for an email that is sent when the setup is completed and the system up and running. The email is sent to the email address that you specified as admin email in the previous steps.

3.3.1.2.6.1 Updating an ABAP System

Learn how to update your ABAP environment instance or change its service plan.

Context

Update the Properties of Your ABAP environment Instance

After you have created an ABAP environment instance, you can change the following properties:

- ABAP System Description (description)
- Total ABAP Runtime Size (size_of_runtime)
- ABAP Runtime Size per Application Server (application_server_size)
- Elastic Scaling of the ABAP Application Server (elastic)
- HANA Cloud Memory Size (size_of_persistence)
- HANA Cloud Disk Size (size_of_persistence_disk)
- Customer-Managed Keys (customer_managed_keys). For more information, see Integration with SAP Key Management Service [page 210]
- Multi-Availability Zones (multi_availability_zones). For more information, see Resilience of an ABAP Environment System [page 208]

> ⓘ Note
>
> The updates are done with zero downtime, except for the HANA database resizing, which is performed with near zero downtime. This means that the database is inaccessible for only a few seconds.

> → Tip
>
> To **change the initial admin**, you have to use the *Maintain Employees* SAP Fiori app. See Maintain Employees [page 3016].

⚠ Restriction

The following restrictions apply when resizing the HANA Cloud database:

- Memory downsizing is limited to the minimum amount of memory required for a stable performance of the database. Before the memory size is decreased, an automatic SQL query (as described in SAP HANA Database Downsizing) verifies that sufficient memory is available before downsizing.
- The memory size cannot be decreased while a system is stopped. To decrease the memory size of a stopped instance, the instance has to be started via the `Landscape Portal` first.
- The disk size can only be increased, not decreased.
- Both memory and disk size can only be resized once every 24 hours.

Interdependencies Between HANA Cloud Memory and Disk Sizes

	When increasing memory	When decreasing memory
Disk size = auto	If the current disk size is below the new minimum, it's automatically increased.	The disk size stays at its current value, even if it exceeds the new maximum disk size.
Disk size unchanged (fixed value)	The disk size must be at least equal to the new minimum disk size.	
Disk size increased	The new disk size must be within the new minimum-maximum range.	

ⓘ Note

The minimum and maximum disk sizes (`size_of_persistence_disk`) are defined by the memory size (`size_of_persistence`) as follows:

- `min(size_of_persistence_disk) = 40 * size_of_persistence + 40`
- `max(size_of_persistence_disk) = 240 * size_of_persistence + 40`

Each GB above the (new) minimum disk size consumes 0.002 HANA compute units (HCU) of your quota.

Change the Service Plan of Your ABAP environment Instance

You can update the service plan from one ABAP environment service plan to another. For example: once you've reached the limits of the free plan, you can upgrade the free service plan to a paid plan (standard plan or build-runtime plan). You could also update your standard plan to a build-runtime plan if you have a SAP Build subscription that includes quota for the ABAP environment service.

ⓘ Note

To update the ABAP environment to a paid service plan, you first need to configure entitlements and quotas for the ABAP environment. You need at least the following entitlements assigned to your subaccount:

- service plan: standard or build-runtime (service ABAP Environment)
- quota plans:
  - 1 abap_compute_unit
  - 2 hana_compute_unit

For more information, see Increasing the Quota for the ABAP Environment [page 195].

> ⚠ Restriction
>
> It's not possible to switch from a paid plan to a free plan.

Procedure

1. In the SAP BTP cockpit, choose your Cloud Foundry subaccount and navigate to the space in which you have created your ABAP environment instance. See Navigate in the Cockpit.
2. Make sure that you have the Space Developer role in the service instance's space:
   - Go to *Space Members* and check if your user is listed there
   - If you're not listed as a member, add yourself as a *Space Developer*. For more information, see Add Space Members
3. From the navigation area, choose *Services* > *Instances*.

   You see a list of all instances that have been created.
4. Select your *ABAP environment* instance and choose *Update* by clicking on ••• *(Actions)* at the end of the row. If you'd like to check the values that are currently configured first, please select *View Parameters* instead.
5. In the *Update Instance* wizard, you can either
   - update the parameters listed above. See Increasing the Quota for the ABAP Environment [page 195]. The currently configured values are prefilled in the *Update Wizard*, so you can keep the values that you don't want to change as they are
   - or change the plan to a new service plan. The following changes are possible:
     - From free to standard
     - From free to build-runtime
     - From standard to build-runtime
6. Choose *Update Instance* to save your changes.

Your ABAP environment instance is being updated. This might take a while.

3.3.1.2.6.2 Resilience of an ABAP Environment System

Learn about the resilience features that are provided to all ABAP Cloud applications built on the SAP Business AI Platform, ABAP environment.

Instances deployed to the SAP Business AI Platform, ABAP environment without the multi-availability zone option receive the following resilience features:

- The components of a single ABAP system and the corresponding SAP HANA Cloud database preferably run together inside the same availability zone of a region.
- Automated monitoring and restart of services in case of failures is ensured for the ABAP system components and the SAP HANA Cloud database.
- A restart usually happens within minutes if hardware is available. A restarted ABAP application server still needs some more minutes to warm up before it can serve traffic.

- In case of failures, components are restarted on other hardware (subject to availability).
- Enqueue locks are stored redundantly.

With the multi-availability zone option for an ABAP environment system inside a region, the following additional features apply:

- The application servers for ABAP are distributed across the availability zones inside the region, in a round-robin fashion.
- Two instances of the SAP HANA Cloud database will be set up in different availability zones and configured with synchronous and transaction-safe data replication. This is covered with the multizone synchronous replication feature of the SAP HANA Cloud service.
- In case of failures, the ABAP system components are restarted on other hardware with higher priority than components of systems without the multi-availability zone option.
- In most cases, the underlying infrastructure handles outages of components automatically and restarts them on functional hardware within minutes.
- In severe cases, like complete availability zone outages, a fully automatic outage handling can't be guaranteed. For such cases, a manual declaration of a zone outage by SAP's operations team will lead to the evacuation of the defective zone and will bring up the system with the multi-availability zone option in the remaining zones, within the limits of hardware availability.
- In case of database-related failures, a takeover to the secondary database instance takes place automatically, and the application servers for ABAP reconnect to the secondary database instance.
- The multi-zone option doesn't affect the costs for the ABAP runtime: the consumed ABAP Compute Units (ACUs) don't change compared to a system without the multi-availability zone option.
- The second database instance affects the costs for the ABAP persistence: the consumed HANA Compute Units (HCUs) increase by factor 2.

To switch on the multi-availability zone option, set the parameter `multi_availability_zones` to `true`. See Updating an ABAP System [page 206].

> ⓘ Note
>
> The multi-availability zone option is based on the principles of the multi-availability zone architecture for SAP BTP *In-Metro Disaster Recovery* as described in Resilience, High Availability, and Disaster Recovery.

> → Tip
>
> The distribution of ABAP application servers across multiple availability zones requires at least two application servers. For example: with application servers of the size 0.5 ACU, the minimum *Total ABAP runtime size* is 1 ACU. A more resilient setup requires a minimum of three ABAP application servers. This configuration ensures that at least two servers remain running if one availability zone fails. For example: with 0.5 ACU servers, in this case, the minimum *Total ABAP runtime size* would be 1.5 ACU, and with 2 ACU servers, the minimum *Total ABAP runtime size* would be 6 ACU.
>
> An exception is the SAP Cloud Infrastructure region `eu01`, which currently has only two availability zones. If you want at least two servers to remain running in the event of an availability zone failure, you need at least four application servers. For example, with 0.5 ACU servers, this corresponds to a minimum *Total ABAP runtime size* of 2 ACUs.

> **Note**
>
> The multi-availability zone option does not reduce the technical downtime for upgrades or updates and hotfix collection imports.

> **Note**
>
> SAP Business AI Platform, ABAP environment provides protection against denial-of-service attacks via various defense mechanisms. These are continuously improved to keep a decent protection level. However, a full protection against DoS attacks can't be guaranteed.

Related Information

SAP HANA Database Availability Zone and Replicas
Resilience, High Availability, and Disaster Recovery

3.3.1.2.6.3 Integration with SAP Key Management Service

SAP Business AI Platform, ABAP environment provides an optional integration of an ABAP environment instance and its SAP HANA Cloud database with the SAP Key Management Service (KMS).

The integration with SAP KMS is established on the level of the SAP BTP subaccount where the ABAP environment instance is created. Once this integration is enabled, the encryption keys of the SAP HANA Cloud database are manageable via SAP KMS. Key activities will affect the SAP HANA Cloud database and the ABAP environment instance.

For more information on SAP KMS, see SAP Key Management Service.

> **Caution**
>
> Once the integration is enabled, it can't be disabled anymore.
>
> If access to the encryption keys is revoked, the linked ABAP environment instances and their SAP HANA Cloud databases will be stopped and the instances are no longer accessible. An ABAP environment instance can remain in this state for up to seven days only. Grant access to the encryption keys in SAP KMS again to bring the instances back online. Open a service request to restart the instances on component `BC-CP-ABA` within seven days after the revocation of key access. There is no automatic restart once access to the keys is given again. If no restart was requested via a service request within seven days after the revocation of key access, the affected ABAP environment instances and their SAP HANA Cloud databases are no longer supported. The only supported operation at this time will be the deletion of the instances, which can be done using SAP BTP Cockpit, for example.

> **Restriction**
>
> The *Customer-Managed Keys* parameter is available in the **standard** and **build-runtime** service plans only. It's not available in the **free** and **saas-oem** service plans.

The new procedures for a reduced technical downtime of release upgrades and the import of hotfix collections will not be applied to an ABAP environment instance which is integrated with SAP KMS.

3.3.1.2.7 Subscribing to the Web Access for ABAP

You need to subscribe to the *Web access for ABAP* SaaS application to get direct browser access to your instances in the ABAP environment, including access to the administration launchpad for ABAP.

Context

> **Note**
>
> If you have run the booster *Prepare an Account for ABAP Development*, you can skip this step.

This subscription is required to access the SAP Fiori launchpad for the administrator and to access developed SAP Fiori applications. Subscription is required only once per subaccount (for all systems to be created in all spaces of this subaccount). After subscribing to the application, you have access to the administration launchpad, where you can configure application roles and assign those roles to your users.

Procedure

1. Log on to the SAP BTP cockpit in the Cloud Foundry subaccount of the global account for Cloud Foundry as administrator.
2. In the navigation area, choose *Services* > *Service Marketplace*.
3. Choose the SaaS application tile *Web access for ABAP*.
4. If the application is not shown as active, choose *Create* and wait for the subscription to be provisioned.

3.3.1.2.8 Creation of Additional Administrator Users (Optional)

After you have set up the ABAP environment for administration, add more administrator users for the ABAP environment.

After you have completed the administrative setup, only your user is available as administrator in the ABAP environment. You will probably want to create more administration users for your ABAP environment. Creating more administration users also helps you verify that your administrative setup is correct and complete.

Related Information

Adding a User as Org Manager for the Cloud Foundry Organization [page 212]
Adding a User as Space Manager for the Cloud Foundry Space [page 213]
Logging on to the Administration Launchpad of the ABAP Environment [page 213]
Creating an Employee Record for a New Administrator [page 214]
Assigning the ABAP Environment Administrator Role to the New Administrator User [page 215]

3.3.1.2.8.1 Adding a User as Org Manager for the Cloud Foundry Organization

After you have created a Cloud Foundry organization for the ABAP environment, you can add more users as organization managers.

Context

> ⓘ Note
>
> If you have run the booster *Prepare an Account for ABAP Development* and entered additional administrators, you can skip these steps.

Procedure

1. Log on to the SAP BTP cockpit as administration user for Cloud Foundry.
2. Go to your subaccount for the ABAP environment.
3. From the navigation area, choose *Org Members*.
4. Choose *Add Members*.
5. Enter the user's e-mail address in the *E-mails* field.
6. Select the checkbox for org manager.
7. Choose *OK*.

3.3.1.2.8.2 Adding a User as Space Manager for the Cloud Foundry Space

After you have created a Cloud Foundry space for the ABAP environment, you can add more users as space managers.

Context

> **Note**
>
> If you have run the booster *Prepare an Account for ABAP Development* and entered users as additional administrators, you can skip these steps.

Procedure

1. Log on to the SAP BTP cockpit as administration user for Cloud Foundry.
2. Go to your subaccount for the ABAP environment.
3. Choose *Spaces*.
4. Choose the Cloud Foundry space in the Cloud Foundry org.
5. In the navigation area, choose *Members*.
6. Choose *Add Members*.
7. Enter the user's e-mail address in the *E-mails* field.
8. Select the checkbox for space manager.
9. Choose *OK*.

3.3.1.2.8.3 Logging on to the Administration Launchpad of the ABAP Environment

To add another user as administrator for the ABAP instance, log on to the administration launchpad of the ABAP environment.

Prerequisites

You are the administrator who has created the ABAP system (see Creating an ABAP System [page 198]).

Procedure

1. In the SAP BTP cockpit, go to the Cloud Foundry subaccount for the ABAP environment as administrator.
2. Choose the space in the Cloud Foundry subaccount.
3. In the navigation area, choose *Services* > *Service Instance*.
4. Choose the service instance for the ABAP environment.
5. Choose *View Dashboard*.

   The logon window for the administration launchpad of the ABAP environment opens.
6. If you are using the custom Identity Authentication service for SAP BTP, you don't need to enter your user and password, but choose the available link of the custom Identity Authentication service and log on.
7. If you need to enter your user and password, remember to log on with the right user that was defined before (see Creating an ABAP System [page 198]).

Results

The administration launchpad opens. You can now proceed with creating an employee record for the administrator.

3.3.1.2.8.4 Creating an Employee Record for a New Administrator

To be able to create a new administration user, you first need to create an employee record for the new administrator.

Prerequisites

You have logged on to the administration launchpad of the ABAP environment as administrator user (see Logging on to the Administration Launchpad of the ABAP Environment [page 213]).

Procedure

1. From the SAP Fiori launchpad, open the *Maintain Employees* app.
2. Choose *Create*.
3. Enter the data for the new administrator, including employee ID, name, and e-mail address. For more information, see Creating an ABAP System.

The entered e-mail is important because it is the default approach to authorization and represents the mapping between the user in the identity provider and in the ABAP environment.

4. Choose *Create*.

Results

You can now create a business user and assign administration roles to the business user.

3.3.1.2.8.5 Assigning the ABAP Environment Administrator Role to the New Administrator User

Add the new administration user as administrator to the ABAP instance.

You have logged on to the administration launchpad of the ABAP environment as administrator (see Logging on to the Administration Launchpad of the ABAP Environment [page 213]). You have created an employee record for another, new administrator.

1. On the SAP Fiori launchpad, choose the *Maintain Business Users* tile.
2. Choose *New*.
3. In the following dialog, choose the employee record of the new administrator from the list.
4. Confirm the system message that you want to create a new user.
5. Add the time zone to the business user data.
6. On the *Assigned Business Roles* tab, choose *Add*.
7. Select the checkbox for the predefined administrator business role `SAP_BR_ADMINISTRATOR`.
8. Select the checkbox for other administrator roles that are available for your organization and choose *OK*.
9. Choose *OK*.
10. Choose *Save*.

3.3.1.2.9 Creation of Developer Users

After you have set up the ABAP environment for development, create developer users for the ABAP environment.

Creating developer users also helps you verify that your development setup is correct and complete.

3.3.1.2.9.1 Creating New Space Members and Assigning Space Developer Roles to Them

For your developers, create new members in the space for the ABAP environment and assign developer roles to them.

Context

> **Note**
>
> If you have run the booster *Prepare an Account for ABAP Development*, you can skip this step.

Procedure

1. Log on to the SAP BTP cockpit as Cloud Foundry administrator.
2. Choose the tile for the Cloud Foundry subaccount.
3. In the navigation area, choose *Spaces*.
4. Choose the space for the ABAP environment.
5. In the navigation area, choose *Members*.

   All members currently assigned to the space are shown in a list.
6. Choose *Add Members*.
7. Enter e-mail addresses of your developer users.
8. You can use commas, spaces, semicolons, or line breaks to separate members.
9. Select the space developer role for the users and save your changes.

3.3.1.2.9.2 Creating an Employee Record for a New Developer

Create an employee record for a new developer user for the ABAP environment.

Procedure

1. Log on to the administration launchpad of the ABAP environment as administrator (see Logging on to the Administration Launchpad of the ABAP Environment [page 213]).
2. Choose the *Maintain Employees* tile.
3. Choose *Create*.

4. Enter the data for the new developer, including employee ID, name, and e-mail address.

   The entered e-mail is important because it represents the mapping between the user in the identity provider and in the ABAP environment.
5. Choose *Save*.

3.3.1.2.9.3 Assigning the ABAP Developer User to the ABAP Developer Role

Add the developer user as business user to the ABAP instance.

You have created an employee record for the new developer user (see Creating an Employee Record for a New Developer [page 216]).

> ⚠ Caution
>
> The SAP_BR_DEVELOPER role is pre-defined by SAP and only intended for the initial set-up of a system. Please create your own business roles based on the required business role template.

For information about creating a business role based on a development-related business catalog, see Business Catalogs for Development Tasks [page 2928].

1. Log on to the administration launchpad of the ABAP environment as administrator (see Logging on to the Administration Launchpad of the ABAP Environment [page 213]).
2. On the SAP Fiori launchpad, choose the *Maintain Business Users* tile.
3. Choose *New*.
4. In the following dialog, choose the employee record of the new developer user from the list.
5. Confirm the system message that you want to create a new user.
6. On the *Assigned Business Roles* tab, choose *Add*.
7. Select the checkbox for the ABAP developer role that you have created before (see *Prerequisites*).
8. Choose *OK* and save your entries.

3.3.1.2.10 Creating a Service Key (Optional)

In the SAP BTP cockpit, you can create a service key for the ABAP system. You can later use this service key, for example, to log on to the ABAP system from ABAP development tools for Eclipse for test purposes.

Context

You can use service keys to generate credentials to communicate directly with a service instance. After you have configured the service keys for your service, local clients, apps in other spaces, or entities outside your deployment can access your service with these keys. Using a service key is one option how you can access the service instance of the ABAP environment, for example, as administrator for test purposes. Note, however,

that there's also another option for users of ABAP development tools for Eclipse: instead of using an existing service key, developers and other users with a user and password for the ABAP system can also use the project creation wizard in ABAP development tools for Eclipse to log on to the ABAP service instance and use the service key that is automatically provided by the service instance. You can also create service keys for other use cases, such as: Creating an Inbound Communication Arrangement with Service Key Type Basic and Configuring the Timeout of the @sap/approuter Component.

> ⓘ Note
>
> The process for connecting to an ABAP service instance URL when creating a new ABAP Cloud project in ABAP development tools for Eclipse has changed. For more information, see ABAP Service Instance URL [page 1881].

Procedure

1. As Cloud Foundry administration user, log on to the SAP BTP cockpit.
2. Go to your Cloud Foundry subaccount for the ABAP environment.
3. In the navigation area, choose *Spaces*.
4. Choose the tile for the ABAP environment space.
5. From the navigation area, choose *Services* > *Instances and Subscriptions*.
6. Choose the service instance of the ABAP system.
7. In the *Service Keys* section of the details area, choose *Create*.
8. Enter a name for the service key, for example, `ABAP Development`.
9. Choose *Save*.
10. Copy the created service key for later reuse.

3.3.1.3 Setup of UI Development in SAP Business Application Studio (Optional)

If developers want to use SAP Business Application Studio to create and deploy SAP Fiori application UIs, you need to perform some administrative activities to set it up.

In the Cloud Foundry environment, SAP Business Application Studio is a service that provides a modular development environment, including the development of SAP Fiori applications. As an alternative to SAP Business Application Studio, you can also use SAP Web IDE. We recommend that you use SAP Business Application Studio.

> ⓘ Note
>
> The process for connecting to an ABAP service instance URL when creating a new ABAP Cloud project in ABAP development tools for Eclipse has changed. For more information, see ABAP Service Instance URL [page 1881].

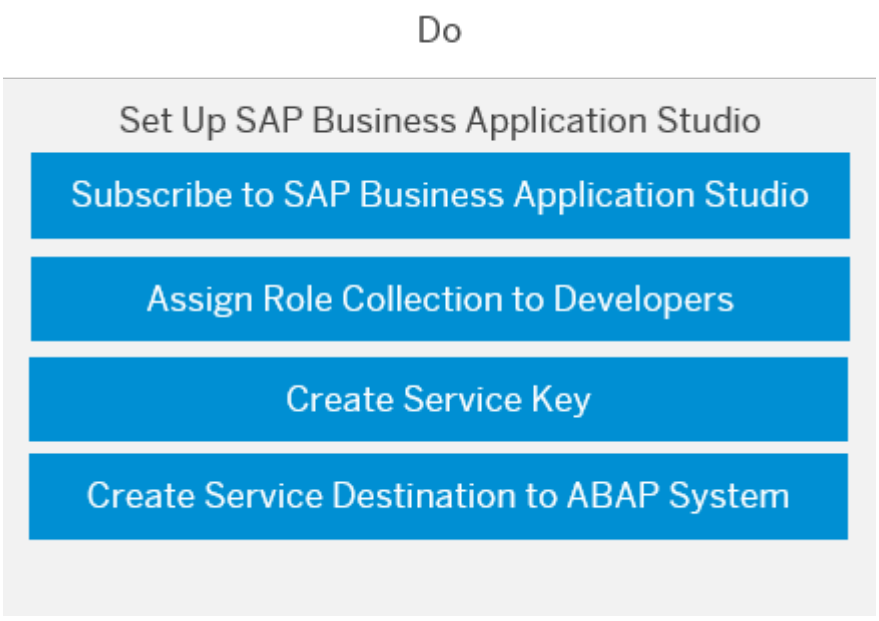
Prerequisites

For setting up SAP Business Application Studio, you need the following:

- You are a member of the global account.
- You are an organization manager in your Cloud Foundry subaccount (see also Adding a User as Org Manager for the Cloud Foundry Organization [page 212]).
- You have created an ABAP service instance (see also Creating an ABAP System [page 198]).
- You are a security administrator. If you have created a subaccount for the ABAP environment in the Cloud Foundry environment, your user automatically has the security administration role.

Setting Up SAP Business Application Studio

> **Note**
>
> If you have run the booster *Prepare an Account for ABAP Development*, you can skip the first two steps.



- Subscribing to SAP Business Application Studio [page 220]
- Assigning Permissions for SAP Business Application Studio [page 221]
- Creating a Service Key for the ABAP System [page 221]
- Creating a Destination to the ABAP System for SAP Business Application Studio [page 222]

1. To enable the creation and deployment of SAP Fiori application UIs, subscribe to SAP Business Application Studio in the same subaccount in which you've created your ABAP service instance (see Subscribing to SAP Business Application Studio [page 220]).

2. To allow developers to develop applications using SAP Business Application Studio, assign developers to the `Business_Application_Studio_Developer` role collection (see Assigning Permissions for SAP Business Application Studio [page 221]).
3. Create a service key for your ABAP system, which you will need to set up a service destination (see Creating a Service Key for the ABAP System [page 221]).
4. To connect your ABAP system with SAP Business Application Studio, set up a destination in the same subaccount in which you have subscribed to SAP Business Application Studio (see Creating a Destination to the ABAP System for SAP Business Application Studio [page 222]).

3.3.1.3.1 Subscribing to SAP Business Application Studio

If developers want to use SAP Business Application Studio as development environment for UIs, you need to subscribe to SAP Business Application Studio in your subaccount.

Context

> **Note**
>
> If you have run the booster *Prepare an Account for ABAP Development*, you can skip this step.

For more information about SAP Business Application Studio and how to subscribe to it, see also Subscribe to SAP Business Application Studio.

Procedure

1. Log on to the SAP BTP cockpit as Cloud Foundry administrator.
2. Choose the tile of the global account of the Cloud Foundry environment.
3. From the global account, navigate to the same subaccount where the ABAP instance is present.
4. From the navigation area, choose *Services* > *Service Marketplace*.
5. Search for **`Studio`**.

   The *SAP Business Application Studio* tile is displayed.
6. Choose the application name.
7. Choose *Create*.

3.3.1.3.2 Assigning Permissions for SAP Business Application Studio

Use the available role collection `Business_Application_Studio_Developer` to assign the relevant permissions to your developers to work in SAP Business Application Studio.

Context

Developers using SAP Business Application Studio must be assigned to developer roles based on the `Business_Application_Studio_Developer` role template. When you subscribe to SAP Business Application Studio, a role collection containing the relevant developer permissions is automatically created. You must assign the role collection to your developers.

> **Note**
>
> In the following steps, you can assign a role collection to an individual user. If you have run the booster *Prepare an Account for ABAP Development*, you can skip this step.
>
> As an alternative, if you are using a custom identity service and identity provider groups, you can also assign the role collection to the identity provider group for your developers. For more information, see Setup of a Custom Identity Service [page 226] and Mapping a Role Collection to the Identity Provider Group for Developers [page 231].

Procedure

1. In the SAP BTP cockpit, navigate to your subaccount for the ABAP environment.
2. From the navigation area, choose *Security* > *Role Collections*.
3. Choose the role collection `Business_Application_Studio_Developer`.
4. Go to the *Users* section and choose *Edit*.
5. Enter the user ID of the user that you want to assign to the role collection.
6. Save your changes.

3.3.1.3.3 Creating a Service Key for the ABAP System

Create a service key for the ABAP system, which will later be needed to configure a destination from SAP Business Application Studio to the ABAP system.

> **Note**
>
> The process for connecting to an ABAP service instance URL when creating a new ABAP Cloud project in ABAP development tools for Eclipse has changed. For more information, see ABAP Service Instance URL [page 1881].

You can use service keys to generate credentials to communicate directly with a service instance. After you have configured the service keys for your service, local clients, apps in other spaces, or entities outside your deployment can access your service with these keys. For more information about service keys, see Creating Service Keys.

1. As Cloud Foundry administration user, log on to the SAP BTP cockpit.
2. Go to your Cloud Foundry subaccount for the ABAP environment.
3. In the navigation area, choose *Spaces*.
4. Choose the tile for the ABAP environment space.
5. From the navigation area, choose *Services* > *Instances*.
6. Choose the service instance for the ABAP system.
7. In the *Service Keys* section of the details area, choose *Create*.
8. Enter a service key name, for example, **`ABAP Destination`**, and save.
9. From the service key, copy the following for later reuse:
   - The URL of the ABAP system from the `url` element
   - The client ID from the `clientid` element in the `uaa` section
   - The content of the `clientsecret` element in the `uaa` section
   - The URL from the `url` element in the `uaa` section

Related Information

SAP BTP Cockpit

3.3.1.3.4 Creating a Destination to the ABAP System for SAP Business Application Studio

Learn how to set up a destination in the same Cloud Foundry subaccount in which you have subscribed to SAP Business Application Studio to establish communication between the ABAP environment and SAP Business Application Studio.

Context

The process for connecting to an ABAP service instance URL when creating a new ABAP Cloud project in ABAP development tools for Eclipse has changed. For more information, see ABAP Service Instance URL [page 1881].

Procedure

1. Log on to the SAP BTP cockpit as administrator.

2. From your global account, navigate to your Cloud Foundry subaccount.
3. In the navigation area, choose *Destinations*.
4. Choose *New Destination*.
5. Enter the following data:

Field	User Input
Name	Enter a name for the destination, for example, **`SAP_Business_Application_Studio`**.
Type	*HTTP*
Description	
URL	Enter the URL of the ABAP system that you copied from the `<url>` element in the service key (see Creating a Service Key for the ABAP System [page 221]).
Proxy Type	*Internet*
Authentication	*OAuth2UserTokenExchange*
Client ID	Enter the content of the `<clientID>` element that you copied from the `uaa` section of the service key (see Creating a Service Key for the ABAP System [page 221]).
Client Secret	Enter the content of the `<clientsecret>` element that you copied from the `uaa` section of the service key (see Creating a Service Key for the ABAP System [page 221]).
Token Service URL Type	Choose *Dedicated*.
Token Service URL	Enter `<uaa-url>/oauth/token`, where `<uaa-url>` is the content of the `<url>` element that you copied from the `uaa` section of the service key (see Creating a Service Key for the ABAP System [page 221]).

6. Choose *New Property* and add the following properties:

Property	Value
HTML5.DynamicDestination	**`true`**
HTML5.Timeout	**`60000`**
WebIDEEnabled	**`true`**
WebIDEUsage	**`odata_abap,dev_abap,abap_cloud`**

7. Make sure that the *Use default JDK truststore* checkbox is checked.
8. Choose *Save*.

Related Information

Creating a Destination for Cross-Subaccount Communication [page 224]

3.3.1.3.5 Creating a Destination for Cross-Subaccount Communication

Learn how to create a destination with SAML assertion authentication to establish communication between an ABAP environment system and SAP Business Application Studio when they are set up in different subaccounts.

Procedure

1. Log on to the SAP BTP cockpit as an administrator.
2. From your global account, navigate to your Cloud Foundry subaccount.
3. In the navigation area, choose *Connectivity* > *Destinations*.
4. Choose *New Destination*.
5. Enter the following data:

Field	User Input
Name	Enter a name for the destination, for example **`SAP_Business_Application_Studio`**.
Type	*HTTP*
Description	Optionally, enter a description.
URL	Enter the URL of your system by copying the *Host Name* from the *Communications Systems* app, for example **`1a354373-d200-46f6-9d5c-daab9a65d9b6.abap.eu10.hana.ondemand.com`**
Proxy Type	*Internet*
Authentication	*SAMLAssertion*
Audience	Enter the URL of your system and add **`-web`** as follows **`1a354373-d200-46f6-9d5c-daab9a65d9b6.abap-web.eu10.hana.ondemand.com`**
AuthnContextClassRef	*urn:oasis:names:tc:SAML:2.0:ac:classes:PreviousSession*

6. Choose *New Property* and add the following properties:

Property	Value
HTML5.DynamicDestination	**`true`**
HTML5.Timeout	**`60000`**
WebIDEEnabled	**`true`**
WebIDEUsage	**`odata_abap,dev_abap`**
nameIdFormat	**`urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress`** or urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified

Property	Value
	depending on the login_attribute configuration chosen during ABAP system provisioning (see Creating ABAP System)

7. Make sure that the *Use default JDK truststore* checkbox is ticked.
8. Choose *Save*.

Related Information

Creating a Destination to the ABAP System for SAP Business Application Studio [page 222]
Creating a Communication System for SAP Business Application Studio

3.3.1.3.5.1 Creating a Communication System for SAP Business Application Studio

Create a communication system to set up communication between your ABAP environment system and SAP Business Application Studio.

Procedure

1. In the SAP BTP cockpit, download your subaccount-specific trust certificate by navigating to *Connectivity* > *Destinations* > *Download Trust*.
2. In the SAP Fiori launchpad of your ABAP environment system, create a communication system by using the *Communication Systems* app. See How to Create Communication Systems.
3. Navigate to section *General* > *SAML Bearer Assertion Provider* and switch the toggle button to *ON*.
4. Upload the *Signing Certificate* first and then enter the following data:

> **Note**
>
> The signing certificate is the trust certificate that you have downloaded from your destination.

Field	User Input
User ID Mapping Mode	User Name
SAML Bearer Issuer	From the *Signing Certificate Subject*, copy the common name (the string after *CN=*) and paste it in the *SAML Bearer Issuer* field.

3.3.1.4 Setup of a Custom Identity Service

For your business users in the ABAP environment, you set up a custom identity service.

Custom Identity Service for the ABAP Environment

For platform users such as the administrator, the default identity provider is SAP ID Service, which is set up by default. For the business users in the ABAP environment, you must set up a custom identity service. For example, the users of your deployed application or users of subscribed apps or services, such as SAP Business Application Studio, are business users. For more information about platform and business users in the ABAP environment, see User Types.

In this documentation, a custom trust configuration is described using SAP Cloud Identity Services - Identity Authentication. The Identity Authentication service is a cloud solution for identity lifecycle management for SAP BTP applications. You can set up SAP BTP as a service provider, and Identity Authentication service as an identity provider.

Identity Authentication Service as Proxy

If you have your own custom identity providers at your company, you can use SAP Cloud Identity Services - Identity Authentication as a proxy. For more information about setting up the Identity Authentication service as a proxy, see Establish Trust and Federation Between UAA and Identity Authentication and Configure Trust with Corporate Identity Provider.

More Information

This documentation describes the basic trust concepts and helps you to get started with a simple landscape. To learn more about trust setup with custom identity providers, check the following, more detailed documentation:

- For more information about trust in SAP BTP, see Trust and Federation with Identity Providers.
- For more information about the Identity Authentication service, see Overview and Initial Setup.

3.3.1.4.1 Establishing Trust of Type OpenID Connect

If you want to use a custom identity provider, you must set up trust between the subaccount and the SAP Cloud Identity Services - Identity Authentication service.

Context

> **Note**
>
> If you have run the booster *Prepare an Account for ABAP Development*, you can skip this step.

If you have custom identity provider, you can use a function in SAP BTP cockpit to set up trust between your subaccount and the Identity Authentication service for SAP BTP automatically. The trust configuration is of type *OpenID Connect*.

For more information, see Establish Trust and Federation Between SAP Authorization and Trust Management Service and Identity Authentication.

Procedure

1. In the SAP BTP cockpit, go to the subaccount for your ABAP system.
2. From the navigation area, choose *Security* > *Trust Configuration*.
3. Choose *Establish Trust*.
4. In the following popup, select a identity provider from the dropdown list.

   Only identity providers that are associated with your customer ID are shown.
5. Choose *Establish Trust*.

   Trust of type *OpenID Connect* between your subaccount and the identity provider is generated.
6. Log on to the Identity Authentication service.
7. From the navigation area, choose *Applications & Resources* > *Applications*.
8. Search for the application that has been created as part of the trust setup.

   The name of the application has the format *SAP BTP subaccount* `<Subaccount Name>`, but you can change it if needed.

   > **Note**
   >
   > Older application names have the format *XSUAA_*`<Subaccount Name>`.
9. Verify that the subject name identifier matches the login_attribute chosen during ABAP system provisioning.

ABAP login_attribute	SAP Cloud Identity Services Subject Name Identifier
email	email
user_name	login name

3.3.1.4.2 Creating Identity Provider Users and Groups

Learn how you create identity provider users and groups in the Identity Authentication service.

Context and Use Cases

This configuration is needed for easier developer authentication using Identity Authentication service. With the creation of identity provider groups, you can also more easily map role collections to entire user groups instead of single users, or map identity provider groups to groups in the Neo environment, for example.

This documentation illustrates the basic concepts and helps you to get started with users and groups in a simple landscape for the ABAP environment.

Prerequisites

You have already created developer users in the ABAP environment (see Creation of Developer Users [page 215]).

3.3.1.4.2.1 Creating an Identity Provider Group for Developers

In the administration console of the Identity Authentication service, you can create an identity provider group for the developers.

Context

An identity provider group for developers is useful, for example, if you want to assign a role collection to all developers in one step, such as the *Application_Studio* role collection for SAP Business Application Studio.

You also use identity provider groups for developers if you want to assign all developers to a corresponding group in the Neo environment, which is necessary if your developers want to use SAP Web IDE as UI development tool.

For more information about creating user groups in the Identity Authentication service, see Create a New User Group.

Procedure

1. Log on to the tenant's administration console for the Identity Authentication service at https://`<tenant ID>`.accounts.ondemand.com/admin as administration user.
2. In the navigation area, choose *Users & Authorizations* > *User Groups*.
3. If no user group for developers exists, choose *+ Add*.
4. Enter a name, for example, **`Developers`**.
5. Enter a display name and a description.
6. Choose *Save*.
7. Note down the group name for later reference.

3.3.1.4.2.2 Creating Identity Provider Users for Developers

In the administration console of the Identity Authentication service, you can create identity provider users for the developers working in the ABAP environment.

Context

You create identity provider users if you work with your own custom identity provider, such as the Identity Authentication service for SAP BTP. You can use the identity provider users, for example, for developers who develop UIs and services using SAP Web IDE or SAP Business Application Studio.

For more information about creating users in the Identity Authentication service, see Create a New User.

Procedure

1. Log on to the tenant's administration console for the Identity Authentication service at https://`<tenant ID>`.accounts.ondemand.com/admin as administration user.
2. In the navigation area, choose *Users & Authorizations* > *User Management*.
3. Choose *+ Add User*.
4. Enter the name, e-mail address and login name of the developer.
5. Leave the account type set to *Employee*.
6. Make sure that the account activation is set to *Send activation e-mail*.
7. Choose *Save*.

3.3.1.4.2.3 Assigning the Identity Provider User Group to the Identity Provider Users for Developers

Assign the identity provider user group for developers to the identity provider users that you have created for the developers.

Prerequisites

You have already created identity provider users. You have also already created identity provider user groups, for example, for SAP Web IDE, for SAP Business Application Studio, or for both.

Context

You now assign these identity provider user groups to the identity provider users for the developers.

For more information about assigning user groups, see Assign Groups to a User.

Procedure

1. Log on to the tenant's administration console for the Identity Authentication service at https://<tenant ID>.accounts.ondemand.com/admin as administration user.
2. In the navigation area, choose *Users & Authorizations* > *User Management*.
3. Choose the identity provider user that you have created for the developer.
4. Choose the *User Group* tab.
5. Choose *Assign Groups*.
6. Choose the identity provider group for developers that you have created (for example, *Developers* for SAP Web IDE, or *BAS_DEVELOPERS* for SAP Business Application Studio).
7. Choose *Save*.

3.3.1.4.2.4 Mapping a Role Collection to the Identity Provider Group for Developers

For developers to be able to work with SAP Business Application Studio, you can use a predefined role collection and assign it to the identity provider group that you created for the developers.

Prerequisites

> **Note**
>
> If you have run the booster *Prepare an Account for ABAP Development*, you can skip this step.

You have subscribed to SAP Business Application Studio (see Subscribing to SAP Business Application Studio [page 220]).

Context

> **Note**
>
> These steps are only relevant if developers want to use SAP Business Application Studio as development environment for UIs.

Developers who want to load, develop, and deploy applications in SAP Application Studio must be assigned to developer roles based on the *Business_Application_Studio_Developer* role template. There's no need to create these roles for SAP Business Application Studio. When you subscribe to SAP Business Application Studio, role collections together with these business role templates are created automatically.

This procedure briefly describes how to assign the role collections to the developer user group in your custom identity provider. For more information about managing authorizations, see Manage Authorizations.

Procedure

1. Log on to the SAP BTP cockpit as Cloud Foundry administrator.
2. Choose the tile of the global account of the Cloud Foundry environment.
3. In the SAP BTP cockpit, navigate to your subaccount for the ABAP environment.
4. From the navigation area, choose *Security* > *Role Collections*.
5. Choose the *Business_Application_Studio_Developer* role collection.
6. Go to the *User Groups* section and choose *Edit*.
7. Select the identity provider where the user group is stored.
8. Enter the name of the user group that you created before, for example, `Developers`.

9. Save your entries.

3.3.1.5 Setup of UI Development in SAP Web IDE (Optional)

If your developers want to use SAP Web IDE as a development tool for creating UIs, you as an administrator need to perform some additional settings.

If your developers want to use SAP Web IDE, which is part of the Neo environment, they need to work in the Cloud Foundry and the Neo environments. To enable an easier authentication of the developers in the Cloud Foundry and Neo environments, set up SAP BTP as a service provider, and Identity Authentication service as an identity provider. For the integration, you must set the trust on both sides and map the identity provider group for developers to its corresponding Neo group.

> → Recommendation
>
> We recommend that you use SAP Business Application Studio as a UI development environment instead of SAP Web IDE.

3.3.1.5.1 Neo Environment: Trust Setup

To enable an easier authentication of the developers in the Cloud Foundry and Neo environments, set up SAP BTP as a service provider, and Identity Authentication service as an identity provider.

Before you follow the steps outlined here for setting up trust between the Neo application and the Identity Authentication service, make sure that you have already set up the Identity Authentication service as custom identity provider for the Cloud Foundry account. For more information, see Setup of a Custom Identity Service [page 226].

3.3.1.5.1.1 Exporting SAML Identity Provider Metadata

Export the SAML metadata of the identity provider for the SAML trust setup on the service provider side.

Context

You need this step only if the SAML metadata file for the Identity Authentication service tenant has not been downloaded already. For more information about tenant SAML 2.0 configuration, see Tenant SAML 2.0 Configuration.

Procedure

1. Log on to the tenant's administration console for the Identity Authentication service at https://`<Your custom Identity Authentication service tenant>`.accounts.ondemand.com/admin as administration user.
2. In the navigation area, choose *Applications & Resources* > *Tenant Settings*.
3. Choose *SAML 2.0 Configuration*.
4. Choose *Download Metadata File* (or download it directly from https://`<Your custom Identity Authentication service tenant>`.accounts.ondemand.com/saml2/metadata).
5. Specify a file name and save it locally to your hard drive.

> **Note**
>
> Specify a file name that you can easily recognize later, for example, `metadataIDP<tentant-id>.xml`, because you need to download and upload multiple SAML metadata files later.

3.3.1.5.1.2 Configuring the Neo Account as SAML Service Provider and Export SAML Metadata

Configure the Neo account as local SAML service provider.

Context

> **Note**
>
> The following steps are only relevant if your developers will use SAP Web IDE in the Neo environment for UI development.

This configuration is needed for developer authentication in the Neo environment (Web IDE runs in the Neo environment).

In the SAML 2.0 communication, each account acts as a service provider. In this documentation, we use the term “local service provider” to describe the account in SAP BTP as a service provider in the SAML 2.0 communication. You need to configure how the local service provider communicates with the identity provider. This includes, for example, setting a signing key and certificate to verify the service provider’s identity and encrypt data.

For more information, see Configure SAP BTP as a Local Service Provider.

Procedure

1. Log on to the SAP BTP cockpit as Neo administrator.
2. Go to the subaccount for the Neo environment.
3. In the SAP BTP cockpit, choose *Security* > *Trust*.
4. On the *Local Service Provider* tab, choose *Edit*.
5. Change the configuration type from *Default* to *Custom*.
6. If the fields for the signing key and the signing certificate are already filled, leave them as they are and don't generate a new key pair.
7. If the fields for the signing key and the signing certificate are empty, choose *Generate Key Pair*.
8. Set principal propagation to *Enabled*.
9. Leave the settings for force authentication as they are.
10. Choose *Save* and confirm the alert message.
11. If you haven't downloaded the SAML metadata file yet: Choose *Get Metadata*.

    The metadata file is downloaded automatically.
12. Rename the downloaded file so that you can easily recognize it later on, for example, to `metadataNeo<tentant-id>.xml`,

    You must download and upload multiple SAML metadata files. Renaming the file helps you distinguishing the metadata files from each other.

Results

You can now switch to the *Application Identity Provider* tab to import the SAML identity provider metadata.

3.3.1.5.1.3 Establishing Trust to the SAML Identity Provider for the Neo Account

Import the SAML identity provider metadata to the Neo account to configure SAML trust to the identity provider (that is, the Identity Authentication service tenant).

Prerequisites

You have downloaded the SAML identity provider metadata file from the Identity Authentication service tenant (see Exporting SAML Identity Provider Metadata [page 232]).

Context

> **Note**
>
> The following steps are only relevant if your developers will use SAP Web IDE in the Neo environment for UI development.

You want to use Identity Authentication service as an SAML 2.0 identity provider. For more information, see Configure Trust to the SAML Identity Provider.

Procedure

1. Log on to the SAP BTP cockpit as Neo administrator.
2. In the SAP BTP cockpit, navigate to the subaccount for the Neo environment.
3. In the navigation area, choose *Security* > *Trust*.
4. Choose the *Application Identity Provider* tab.
5. Choose *Add Trusted Identity Provider*.
6. Choose *Browse*.
7. Upload the metadata file of the custom Identity Authentication service tenant.

   > **Note**
   >
   > During the setup of the ABAP environment, you need to upload multiple SAML metadata files. Make sure that you upload the correct file. If you have followed the naming conventions suggested in this documentation, the file needed here is `metadataIDP<tentant-id>.xml`.

8. Enter a description.
9. Choose *Save*.

3.3.1.5.1.4 Creating a Neo Application as Trusted Service Provider

Create and configure an application to represent the Neo account in the Identity Authentication Service tenant as SAML 2.0 service provider.

> **Note**
>
> The following steps are only relevant if your developers will use SAP Web IDE in the Neo environment for UI development.

Creating an SAML Application for the Neo Account

As a first step, you need to create the application for the Neo account.

1. Log on to the tenant's administration console for the Identity Authentication service at `https://<tenant ID>.accounts.ondemand.com/admin` as administration user.
2. In the navigation area, choose *Application & Resources* > *Applications*.
3. Choose *+ Add*.
4. Enter an application name for the Neo account (the service provider) in the Identity Authentication service.
5. Choose *Save*.

Configuring the Neo Application as Trusted Service Provider

Import the SAML service provider metadata from the Neo environment to the Identity Authentication service tenant (that is, the SAML identity provider). This configures SAML trust to the Neo account as SAML service provider.

1. Under *Applications*, choose the entry for the Neo application that you have just created.
2. Choose *SAML 2.0 Configuration*.
3. Choose *Browse*.
4. Upload the metadata file of the SAML service provider from the Neo account that you have downloaded before (see also Configuring the Neo Account as SAML Service Provider and Export SAML Metadata [page 233]).

   > ### Note
   >
   > During the setup of the ABAP environment, you need to upload multiple SAML metadata files. Make sure that you upload the correct file. If you have followed the naming conventions suggested in this documentation, the file needed here is `metadataNeo<tentant-id>.xml`.

5. Choose *Save*.

Configuring the Subject Name Identifier Sent to Cloud Foundry

Configure the subject name identifier that the Identity Authentication service (as identity provider) sends to the Neo application. Use the email address as user identifier for the Neo account.

1. Under *Applications*, choose the entry for the Neo application.
2. Choose *Subject Name Identifier*.
3. Choose *Basic Configuration* and select *E-Mail* from the dropdown list.
4. Choose *Save*.

Configuring the Default Name ID Format for the Neo Application (Optional)

Optionally, you can configure the default name ID attribute of the identity provider for Neo. Use the email address as user identifier for the Neo account.

1. Under *Applications*, choose the entry for the Neo application in the Identity Authentication service.
2. Choose *Default Name ID Format*.
3. Choose the *E-Mail* radio button.
4. Choose *Save*.

Configuring the SAML Assertion Attributes for Cloud Foundry

Include the attributes `first_name`, `last_name`, `mail`, and `Groups` for the SAML assertion.

1. Under *Applications*, choose the entry for the Neo application.
2. Choose *Attributes*.
3. Choose *Add* to enter the following attributes (if they don't already exist):

Name	Value
`first_name`	*First Name*
`last_name`	*Last Name*
`mail`	*E-Mail*
`Groups`	*Groups*

> ### Note
>
> Make sure that you enter **`Groups`** (with upper case **`G`**), not **`groups`**, in the *Name* field, so that it matches the assertation attribute used for the Neo environment.

4. Choose *Save*.

More Information

Configuring Applications

Configure Trust

Configure the Subject Name Identifier Sent to the Application

3.3.1.5.2 Enabling SAP Web IDE Full-Stack in the Neo Account

Enable the SAP Web IDE Full-Stack service in the Neo account if it is not yet enabled.

Context

> **Note**
>
> You only need this step if your developers want to use SAP Web IDE as environment for UI development. As an alternative, you can also use SAP Business Application Studio.

SAP Web IDE Full-Stack is a development environment to create new or extend existing SAP Fiori, SAPUI5, or hybrid applications. For more information about the SAP Web IDE Full-Stack, see SAP Web IDE Full-Stack.

Procedure

1. Log on to the SAP BTP cockpit for the Neo environment as Neo administration user.
2. In the navigation area, choose *Services*.
3. Scroll down to the *Web IDE Full-Stack* tile.
4. If the service is not yet enabled, choose the *Web IDE Full-Stack* tile.
5. On the *Service: Web IDE Full-Stack: Overview* screen, choose *Enable*.
6. Wait for the status to change to the green *Enabled* status.

   This may take a few minutes.

3.3.1.5.3 Creation of Developer Users for SAP Web IDE

If your developers want to use SAP Web IDE, use identity provider users for them to ease log on to the Neo enviroment.

If you have followed the steps for setting up a custom identity provider, you have already created a developer group in the tenant for the Identity Authentication service (see also Setup of a Custom Identity Service [page 226]). Now you need to create a developer group in the Neo enviroment and map this group to the developer group in the tenant for the Identity Authentication service.

3.3.1.5.3.1 Creating a Neo Group for Developers

Define a Neo group to map the developer group of the identity provider to the `DiDeveloper` role of SAP Web IDE.

Context

> **Note**
>
> The following steps are only relevant if your developers will use SAP Web IDE in the Neo environment for UI development.

For more information about roles, see Managing Roles.

Procedure

1. Log on to the SAP BTP cockpit (Neo account) as Neo administration user.
2. In the navigation area, choose *Security* > *Authorizations*.
3. Choose the *Groups* tab.
4. Choose *New Group*.
5. Enter a group name, for example, **`Developers`**, and choose *Save*.
6. In the *Roles* screen area, choose *Assign*.
7. As subaccount, select the service provider account of the Web IDE, for example, **`sapwebide EU1`**.
8. As application, select the Web IDE application (*di*).
9. As role, select the predefined Web IDE developer role *DiDeveloper*.
10. Choose *Save*.

3.3.1.5.3.2 Mapping the Identity Provider Group Developers to the Neo Group

Map the developer group of the identity provider to the Neo developer group with the Web IDE `DiDeveloper` role.

Context

> **Note**
>
> The following steps are only relevant if your developers will use SAP Web IDE in the Neo environment for UI development.

The mapping is part of the trust configuration to the SAML identity provider. For more information, see Configure Trust to the SAML Identity Provider.

Procedure

1. Log on to the SAP BTP cockpit (Neo account) as Neo administration user.
2. In the navigation area, choose *Security* > *Trust*.
3. Choose the *Application Identity Provider* tab.
4. Choose the link for the custom identity authentication service tenant.
5. Choose the *Groups* tab.
6. If no group for Neo developers exists, choose *Add Assertion-Based Group*.
7. Enter the following data:

Option	Description
Group	Select *Developers*.
Mapping Rules **(part 1)**	Enter *Groups*.
Mapping Rules **(part 2)**	Select *equals*.
Mapping Rules **(part 3)**	Enter the developer group name from the Identity Authentication service that was created by the administrator for the Identity Authentication service, for example, *Developers*. If you are not the responsible administrator for the Identity Authentication service, make sure that you are informed about the group name (see also Creating an Identity Provider Group for Developers [page 228]).

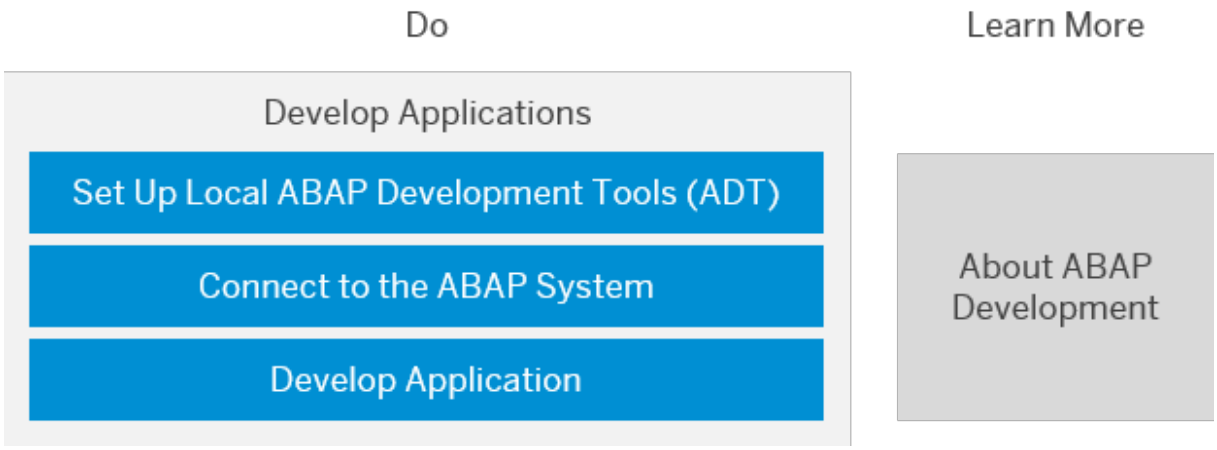
8. Choose *Save*.

3.3.1.6 Getting Started as a Developer in the ABAP Environment

Learn about your first steps to get started as a developer in the ABAP environment.

> **Note**
>
> This documentation informs you about the first steps as a developer and assumes that the necessary steps for setting up the ABAP environment are done. For more information about how to set up the ABAP environment and getting started as an administrator, see Getting Started with a Customer Account in the ABAP Environment [page 184].

Developing Applications: Overview



- https://tools.hana.ondemand.com/#abap [https://tools.hana.ondemand.com/#abap]
- Connect to the ABAP System [page 851]
- https://developers.sap.com/tutorials/abap-environment-gcts.html#b1afe95c-9acb-4f75-8f80-1abe93cb524f [https://developers.sap.com/tutorials/abap-environment-gcts.html#b1afe95c-9acb-4f75-8f80-1abe93cb524f]
- https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/31367ef6c3e947059e0d7c1cbfcaae93.html [https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/31367ef6c3e947059e0d7c1cbfcaae93.html]
- https://help.sap.com/viewer/DRAFT/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/59656c2f858748fe976456248d390c5c.html [https://help.sap.com/viewer/DRAFT/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/59656c2f858748fe976456248d390c5c.html]
- https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/31367ef6c3e947059e0d7c1cbfcaae93.html [https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/31367ef6c3e947059e0d7c1cbfcaae93.html]

1. Download and install ABAP Development Tools (ADT) from https://tools.hana.ondemand.com/#abap. See Video Tutorial: Configure ABAP Development Tools .

   > **Note**
   >
   > SAP GUI is not supported in the ABAP environment. You can only use ADT as your development environment.

2. Create an ABAP cloud project with ADT to connect to the ABAP system in the ABAP environment (see Connect to the ABAP System [page 851]).
3. Develop your application (see Development in the ABAP Environment). To learn more about how to develop applications, see Tutorial: Create Your First ABAP Console Application and Video Tutorial: Create Application .

More Information About Development in the ABAP Environment

For more information about ABAP development, see Development in the ABAP Environment.

3.3.1.7 Tutorial Overview

Getting Started

Mission

Build an SAP Fiori app Using the ABAP RESTful Application Programming Model

- Create Database Table and Generate UI Service
- Enhance the Business Data Model and Enable OData Streams
- Enhance the Business Object Behavior With Unmanaged Internal Numbering
- Enhance the Business Object Behavior With Determinations
- Enhance the Business Object Behavior With Validations
- Enhance the Business Object Behavior With Instance Action
- Enhance the Business Object Behavior With Factory Action
- Enhance the Business Object Behavior With Dynamic Feature Control
- Write an ABAP Unit Test for the RAP Business Object

Group: Create an ABAP Core Data Services (CDS) View in SAP BTP ABAP Environment

- Enhance an ABAP Core Data Services (CDS) View in SAP Business Technology Platform (BTP) ABAP Environment

ABAP Dictionary and ABAP Language

Group: Start Developing ABAP Tables

- Create an ABAP Database Table and Relevant ABAP Dictionary Objects
- Create ABAPDoc Comments in Your Class in ABAP Environment
- Create an ABAP Managed Database Procedure (AMDP) and Analyze Its Performance

Analytics

- Develop and Consume Queries on SAP Analytics Cloud
- Develop Queries Based on Booking Supplement and Consume on SAP Analytics Cloud

Authorization Model

Group: Create Authorization Model with SAP BTP ABAP Environment

- Create Data Model and Lock Object in SAP BTP ABAPEnvironment
- Create Behavior and Service in SAP BTP ABAP Environment
- Create Authorization Model and App in SAP BTP ABAPEnvironment
- Maintain Business Roles and Access Restrictions in SAP BTP ABAPEnvironment

Business Configuration

Group: Create a SAP Fiori based Table Maintenance App

- Create Business Configuration Maintenance Object
- Providing Authorization Control for a Business Configuration Maintenance Object
- Use Custom Business Configurations app
- Explore additional features of the Business Configurations app

Lifecycle Management and CI/CD

- Push Your ABAP Source Code With SAP Cloud Platform ABAP Environment via abapGit
- Transport ABAP Objects Using git-based Change and Transport System (gCTS)
- Use Checkout Functionality of Branches to Switch Between Different Versions of Code

- Transport a Software Component Between two Systems
- Use abapGit to Transfer ABAP Source Code to the Cloud
- Transport a Software Component Between two Systems Using Any Git Repository (BYOG)

Cloud Operations

- Monitor an SAP BTP ABAPEnvironment Service Using SAP Focused Run (FRUN)
- Monitor an SAP BTP ABAPEnvironment Service Using SAP Cloud ALM (CALM)

Eventing

- How to Create RAP Business Events in SAP BTP ABAPEnvironment
- Event Consumption in ABAP Development Tools for Eclipse
- How to Create RAP Business Events in an On-Premise System (In Progress)

Integration

Group: Expose an ABAP CDS View Externally Using a Communication Scenario

- Expose a Standard Core Data Service for ABAP Environment
- Maintain a Communication Arrangement for Inbound Communication
- Get Data from an On-Premise System Using a Remote Function Call (RFC) Scenario
- Remote Function Call (RFC) - Connect to Your On-Premise System from SAP BTP ABAPEnvironment
- Test the Business API (BAPI) with an ABAP Console App
- Get Data from an On-Premise System Using a Custom Entity
- RFC: Inspect Your Class in the ABAP Debugger
- RFC: Enhance the Handler Class With Filtering
- Publish an OData Service for Remote Consumption as an API
- Create Remote Client Proxy and ABAP Console App
- Implement Custom Entity and Query Implementation Class

without group/mission:

- Call an External API and Parse the Response in SAP BTP ABAPEnvironment
- Generate RFC Proxy Class
- Using ODBC driver for SQL Service
- Consume SOAP Based Web Services with SAP BTP ABAP Environment

Tools, User, Identity Management

without group/mission:

- Create an SAP Cloud Platform ABAP Environment Trial User
- Install ABAP Development Tools (ADT) and abapGit Plugin
- Create a Developer User in SAP Cloud ABAP Environment
- Provision Users into your SAP BTP ABAP Environment

UI Development

- Develop and Run SAP Fiori Application With SAP Business Application Studio
- Add the Shopping Cart Fiori Application to FLP
- Add SAP BTP ABAP environment App to Portal Site
- Developing and extending SAP Fiori Elements App
- RAP Feature Showcase App
- Create a Travel App with SAP Fiori Elements Based on OData V4 RAP Service

without group/mission:

- Develop and Run SAP Fiori Application With Visual Studio Code
- Develop SAP Fiori App & Deploy to Cloud Foundry Using SAP-managed App Router
- Add Cloud Foundry to Portal Site
- Set SAP BTP ABAP Environment as a ContentProvider for SAP Build Workzone, standard edition
- Use Custom Themes in the SAP BTP ABAP Environment

Custom Code Analysis

Connect ABAP On-Premise to a Central Check System for Custom Code Migration Using ABAP Test Cockpit

SAP S/4HANA Cloud, ABAP Environment

Group: Develop an SAP Fiori App to Trigger Purchase Requisitions API

- Create a Shopping Cart Business Object
- Create Value Help, Enhance the Behavior Definition and Behavior Implementation of the Shopping Cart Business Object
- Integrate released Purchase Requisition API into Online Shop Business Object

Without group/mission

- Implement a Business Add-In (BAdI) to Check a Purchase Requisition

- Implement a Field Control Using a Business Add-In (BAdI)
- Create Custom Logic Using Key User Extensibility and Perform Trace
- Create and Debug a BAdI Implementation in ABAP Development Tools (ADT)

Badges

Taster: Get Started with SAP BTP ABAP Environment

Level 1: Get to know the ABAP RESTfulApplication Programming Model

Level 2: Level up with SAP BTP ABAP Environment

Level 3: SAP BTP ABAP Environment: Intermediate Topics

3.3.2 Getting Started with Custom Code Analysis in the ABAP Environment

Learn how to get started with Custom Code Analysis in the SAP Business AI Platform, ABAP environment.

You want to use the SAP Business AI Platform, ABAP environment to analyze your custom code remotely in your SAP ECC or SAP S/4HANA systems? This documentation will help you gain insight into all the options to work with the ABAP Test Cockpit (ATC) in the SAP Business AI Platform, ABAP environment.

The following list covers all possible use cases and will guide you to the respective documentation chapters providing more details on each use case.

1. **Configure ABAP Test Cockpit:** Use this app to maintain ATC configurations using the SAP Fiori launchpad.
   For more information on this app, see ABAP Test Cockpit Configurator.
   To learn how to enable the app, see Enable Usage of the ABAP Test Cockpit Configurator App.
2. **Custom Code Analysis:** If you want to check your custom code in your on-premise system remotely using the SAP Business AI Platform, ABAP environment as the central check system, you need to set up the Analyze Custom Code app on the SAP Fiori launchpad.
   To enable the Analyze Custom Code app, see Enable Usage of the Custom Code Migration App.
   To learn all about custom code migration, see the Custom Code Migration Guide for SAP S/4HANA 2023.
3. **ATC Developer Scenario:** You can use a system in the SAP Business AI Platform, ABAP environment as a central check system to run ATC checks from an on-premise system against this system. To do this, a number of configurations will be necessary.
   To learn how to use the ATC as a central check system, see Configuring Remote ATC Using a Central Check System.
   To learn how to set up the ATC developer scenario, see Configuring the ATC Developer Scenario.
4. **Working with Exemptions:** Learn about your options to exempt your ATC findings.
   To learn how to enable the Approve ATC Exemptions app to be able to exempt the findings from your custom code analysis, see Enable Usage of the Approve ATC Exemptions App.
   For more information on the Approve ATC Exemptions app, see Approve ATC Exemptions.
5. **Schedule Custom Code Analysis:** The Schedule Custom Code Analysis app enables you to schedule projects created in the Analyze Custom Code app as application jobs. With this functionality, the analyses can be performed once or periodically.

To learn more about the app and how to enable it, see Schedule Custom Code Analysis.

6. **Maintain the Baseline:** In the Analyze Custom Code app, it is now possible to maintain the ATC baseline. This means that you can add/remove custom code project check results to/from the baseline and choose whether you would like to exempt your findings, suppress your findings or reduce their priority. After adding findings to the baseline, they won't appear in the subsequent check results of the custom code project.
   To learn more about the baseline functionality in the Analyze Custom Code app, see Custom Code Migration.

Related Information

SAP Business AI Platform, ABAP environment
Custom Code Analysis [page 2946]

3.4 Getting Started in the Kyma Environment

As an administrator, you must perform several steps to set up a fully operational Kyma environment to which you can connect the chosen SAP solutions.

Prerequisites

- To perform administrative and development tasks, you need a global account and one or several subaccounts for which you can provision the Kyma environment. For details, see Creating Kyma Instances [page 3452].
- The subaccount must have the following entitlements assigned:
  - Kyma environment (added by the subaccount manager)
  - Service Manager (added by default). However, if you removed the entitlement, you must add it again in your subaccount

  For details, see Configure Entitlements and Quotas for Subaccounts [page 2529].

Procedure

1. Set up a Kubernetes cluster with the project "Kyma" to connect and extend SAP systems: Creating Kyma Instances [page 3452]
2. Add the modules you'd like to use: Kyma Modules [page 88].
3. Assign the roles to users to allow the administrators to manage Kyma and the developers to create Functions: Assign Roles in the Kyma Environment [page 3666]

> ⚠ Caution
>
> Assign the roles before the users start using the Kyma dashboard. Not granting the roles results in an error.

4. Use the Kyma environment to integrate external systems: Register an SAP Customer Experience System

   You have the following options:

   - Integrate a CX system and the Kyma environment, so that the Functions you develop can use the system's API and receive business events.
   - Integrate an SAP S/4 HANA Cloud system to use the services it provides to extend your applications.

   After you've integrated externals systems, the developers can start working on their Functions.
5. Extend SAP S/4HANA Cloud by developing event-driven extensions and applications: Extending SAP S/4HANA Cloud in the Cloud Foundry and Kyma Environment
6. Group several solutions into one formation to meet the requirements of your business scenario: Including SAP Systems in a Formation

Results

You have set up the Kyma environment and connected it as required.

Next Steps

After the administrator has set up the Kyma environment, the developers can access it through the Kyma dashboard. After logging on, developers can start creating extensions for the SAP systems either from the Kyma dashboard or from the terminal after downloading the kubeconfig file with the cluster configuration. For details, see Development in the Kyma Environment [page 1882].

3.4.1 Getting Started with an Enterprise Account in the Kyma Environment

Quickly get started with an enterprise account in the Kyma environment using the SAP BTP cockpit.

Procedure

1. Get a global account.

   After you've signed up for your enterprise account, you get an e-mail with a link to SAP BTP and the logon data as administrator for the global account.

Log on to SAP BTP and navigate to your global account.

> **Note**
>
> For more information on available plans, including free tier, see Available Plans in the Kyma Environment [page 3459].

2. Set up your subaccount. For more information, see Create a Subaccount [page 2509].

   You can create more subaccounts to break down your account model and structure it according to your development scenario. But first it's important you understand how to navigate to your accounts using the SAP BTP cockpit.

   You can also download and use the SAP BTP command line interface (btp CLI) to create new subaccounts. See Download and Start Using the BTP CLI Client [page 2665] and Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699].
3. Create your Kyma environment. For more information, see Creating Kyma Instances [page 3452].
4. Configure your Kyma environment.

   Now that you've set up your account model, it's time to think about member management. You can add members at different levels:
   - For Kyma-specific role assignment, see Assign Roles in the Kyma Environment [page 3666].
   - To learn more about how user roles are assigned in SAP BTP, have a look at User and Member Management [page 133].
5. Develop your applications and extend SAP solutions.

   Check out Development in the Kyma Environment [page 1882] to learn more.

Related Information

Available Plans in the Kyma Environment [page 3459]

3.4.2 Getting Started with a Trial Kyma Instance

Get started with a trial SAP BTP, Kyma runtime instance, and explore its features and capabilities.

Procedure

1. Get a global account.

   Sign up for an SAP BTP free trial account. See Get an Account on SAP BTP Trial.
2. Set up your subaccount.

   When you register for the SAP BTP trial account, a subaccount is created for you. If your subaccount was successfully created in the SAP BTP cockpit, skip this step and proceed to adding entitlements.

If your subaccount was not created automatically, create it manually by following these steps:

a. To navigate to your global account, choose *Enter Your Trial Account*.

b. In the *Account Explorer* view, choose *Create* > *Subaccount*.

c. Configure your trial subaccount:

Field	Input
Display Name	**`trial`** (sample value; you can provide a name of your choice)
Description	Optional
Region	Desired region
Subdomain	**`<your_id>trial`** — Example: **`P0123456789trial`**
Enable beta features	Optional — Enables the use of beta services and applications.
Add labels to this subaccount	Optional — You can use labels to identify and organize the subaccounts in your global account.

If you want to create more subaccounts to break down your account model and structure it according to your development scenario, see Create a Subaccount.

You can also download and use the SAP BTP command line interface (btp CLI) to create new subaccounts. See Download and Start Using the btp CLI Client and Working with Global Accounts, Directories, and Subaccounts Using the btp CLI.

You have successfully set up your trial subaccount.

3. Add entitlements.

When you have a subaccount (whether it was created automatically or you followed the steps described above), you need entitlements to add service plans.

a. Navigate to your subaccount.

b. Choose *Entitlements* from the left-hand side navigation.

c. Choose *Edit*.

d. Choose *Add Service Plans* and select all the service plans available for your subaccount.

> **Note**
>
> To select a service plan, choose a service from the left and tick all the service plans that appear on the right. Do that for all services.

e. After adding all the service plans, you see them all in a table. Before you choose *Save*, increase the amount to the maximum for all the plans with a numerical quota. Choose + repeatedly until it becomes inactive.

f. Save all your changes.

You have successfully set up your entitlements.

4. Request a trial Kyma instance.

To request a trial Kyma instance, contact SAP using one of the following methods:

- Open a support ticket for the `BC-CP-XF-KYMA` component. See Getting Support.

- Send an email to kyma@sap.com with the subject **`SAP BTP, Kyma Runtime Trial Request`**, using the following template:

```
I would like to request access to the SAP BTP, Kyma runtime trial.
Please find the required details below:
- Customer Global Account ID: <your trial SAP BTP global account ID>
- Subaccount ID (for SAP BTP service consumption): <your trial SAP BTP
subaccount ID>
(Note: You can also use a SAP BTP trial account)
- List of Administrators:
    - <email 1>
    - <email 2>
    - <email 3>
- Reason for Request:
    <Briefly describe the evaluation use case>
```

> ### Note
>
> Ensure your request is complete and compliant with SAP requirements. SAP reviews all requests on a case-by-case basis within one month and may decline those that are incomplete, non-compliant, or not aligned with the applicable Terms and Conditions. If your request is declined, SAP is not required to provide any response, notification, explanation, or justification regarding its decision.

5. Wait for SAP to provision your trial Kyma environment.

   After SAP approves your request, your Kyma environment is provisioned automatically. You receive a confirmation message when the environment is ready to use. The SAP BTP Operator module used for consuming SAP BTP services is configured so that your service instances are created within the account and subaccount you specified in your request.

Next Steps

Develop your applications and extend SAP solutions.

For more information, see Development in the Kyma Environment [page 1882].

You can also provide extensions for the SAP Commerce Cloud, SAP Cloud for Customer, and SAP Field Service Management systems. For more information, see Extending SAP Solutions.

Related Information

Trial Accounts and Free Tier [page 109]

3.4.2.1 About the Trial Kyma Instance

You can try out the Kyma environment for free with limited use of platform resources and services. A trial Kyma instance is available upon request and is intended only for personal exploration, non-productive testing, and evaluation purposes, not for production use or team development.

Trial Period

While the SAP BTP trial account is valid for 90 days, a Kyma trial instance is valid for 30 days counting from the date SAP provisions it. After 30 days, your cluster is automatically deleted, and you can no longer access your data. To continue evaluating, you can submit a new trial request. For more information, see Getting Started with a Trial Kyma Instance [page 249].

> ⚠ Caution
>
> After your trial Kyma instance has expired, your cluster is deleted permanently and all cluster resources are removed. We recommend that you back up your cluster configuration before it expires so you can restore it quickly if you request a new trial Kyma instance.

Scope and Limitations

The trial instance has the following scope and limitations:

- The trial is available upon request. For more information, see Getting Started with a Trial Kyma Instance [page 249].
- The maximum trial duration is 30 days.
- You can request only one Kyma environment per global account.
- After the first provisioning, you cannot modify the list of administrators. To grant access to additional users, add and configure an additional OIDC through a custom resource.
- A trial account provides you with a 1-node cluster with 4 vCPU and 16 GB of memory. An idle cluster without any customer workload uses around 33% of vCPU and 33% of memory.
- There is no service level agreement with regards to the availability of the platform.

Trial Expiry

You cannot extend your trial Kyma environment. After your trial Kyma instance has expired, it is automatically removed, but you can request another trial instance.

Related Information

Trial Accounts and Free Tier [page 109]

4 Development

Develop and run business applications on SAP Business Technology Platform (SAP BTP) using our cloud application programming model, APIs, services, tools, and capabilities.

SAP BTP supports multiple environments. It features a number of tools and programming languages that increase your flexibility and freedom when developing applications. You can choose the environment that best suits your use cases and skill set and get to work. You also have the possibility to integrate applications you've built with other solutions by SAP.

Environment options

	Cloud Foundry	Kyma	ABAP
Benefits	• Simplified developer experience for business application development • Large choice of programming languages • Intuitive "code-to-container" packaging and deployment, managed by the platform • Platform-managed application security patching and updates • Automatic application routing, load balancing, health checks, and multilevel self-healing • Support for CAP – an opinionated business app development framework	• Take full advantage of the advanced features and rich ecosystem of Kubernetes • Free choice of programming languages and models (containerized deployments) • Combines microservices and serverless functions • Built-in, managed service mesh based on Istio, and other cloud-native open-source modules to reduce the development effort • Built-in, managed event mesh • Managed infrastructure: day-2 operations, security patches, and updates • Full administrator access • Refined horizontal and vertical automatic scalability • Dedicated application runtime • Zero downtime infrastructure setup by default • Support for CAP – an opinionated business app development framework • Support for on-premise connectivity	• ABAP programming language • Fast prototyping with ABAP RESTful Programming Model (RAP) • Integrated development lifecycle • Reuse existing on-prem ABAP assets

	Cloud Foundry	Kyma	ABAP
Additional Information	Comparison: SAP BTP, Kyma Runtime and SAP BTP, Cloud Foundry Runtime	Comparison: SAP BTP, Kyma Runtime and SAP BTP, Cloud Foundry Runtime	Development in the ABAP Environment
Shared Benefits	• No infrastructure vendor lock-in • Build scalable multitenancy business applications (SaaS) • Out-of-the-box consumption of SAP and hyperscaler services • Built on industry standards and open technology		
Good For	• Managed build-on approach • Enterprise-grade business applications and services • Cloud-native web applications and services • Scalable, microservice-based applications • Small to medium extensions built with CAP/low-code tooling	• Open build-on approach • Enterprise-grade applications • Cloud-native development of apps and services • Low latency infra-services communication • Reduced infrastructure management effort • Highly scalable, microservice-based applications • Applications built with CAP	• User-centric process extensions • Robust, transactional cloud applications • Migrating and adapting add-ons to the cloud • Reusing existing on-premise ABAP code • Enabling ABAP developers to go to the cloud

	Cloud Foundry	Kyma	ABAP
Skills	• Any major programming languages • SAP Fiori/UI5 and SAP HANA	• Kubernetes knowledge • Docker • NodeJS or Python for serverless functions • Any major programming language • SAP Fiori/UI5 and SAP HANA	• Ability to write modern ABAP code • Core data services • SAP Fiori/UI5 and SAP HANA

We provide detailed information about developing, configuring, and deploying your applications depending on your preferred environment and development approach.

Development in the Cloud Foundry Environment [page 256]
Learn more about developing applications on the SAP BTP, Cloud Foundry environment.

Development in the ABAP Environment [page 848]
Learn more about developing applications in the ABAP environment.

Development in the Kyma Environment [page 1882]
Learn more about developing applications in SAP BTP, Kyma runtime.

Related Information

SAP BTP Administrator's Guide
Developing with the SAP Cloud Application Programming Model [page 266]
Consuming Services in SAP BTP

4.1 Development in the Cloud Foundry Environment

Learn more about developing applications on the SAP BTP, Cloud Foundry environment.

Overview

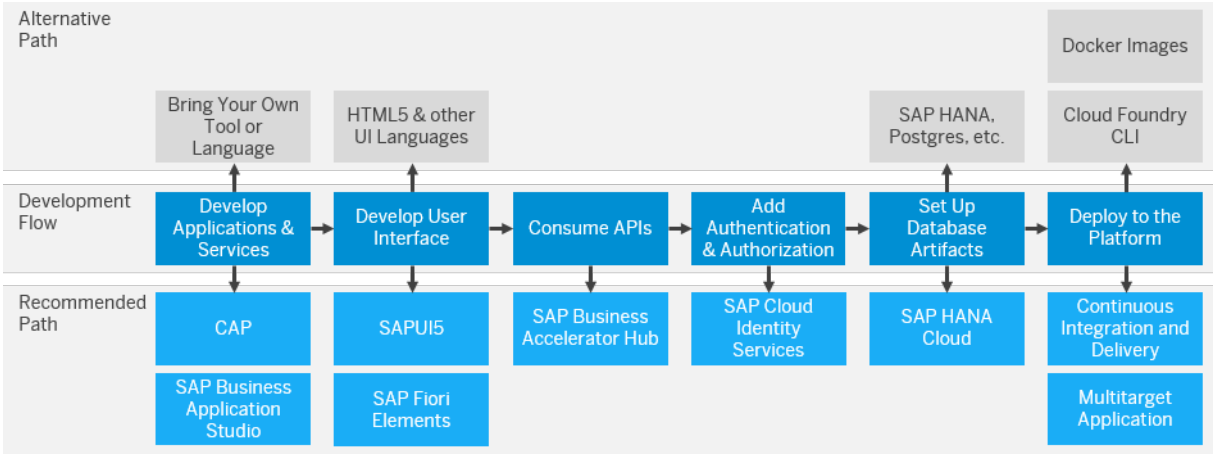
SAP BTP, Cloud Foundry environment is an open Platform-as-a-Service (PaaS) targeted at microservice development and orchestration.

Develop polyglot applications	Build on open standards with SAP Java, Node.js, and Python buildpacks or bring your own language with community buildpacks for PHP, Ruby, Go.
Manage the lifecycle of applications	Start, stop, scale, and configure distributed cloud applications using standard Cloud Foundry tools, our web-based administration user interface for SAP BTP, and dev-ops capabilities.

Optimize development and operations	Use the rich set of SAP BTP services including messaging, persistence, and many other capabilities, such as built-in security, compliance, elastic scale, and high-availability setup.
Use the application programming model	Use programming languages, libraries, and APIs tailored for full-stack application development.
Manage Cloud Foundry orgs and spaces	Create and delete Cloud Foundry orgs and spaces and add members. Create, assign, and change space quota plans.
Use built-in cloud-native capabilities	Utilize cloud-native principles built into the platform, such as containerization and multitenancy, to create resilient, portable, and efficient applications.

The following graphic is designed to help you find the information you need for your programming purposes. The bottom row represents the tools, frameworks, services, and deployment options recommended by SAP. If you want full flexibility you can also bring your own development tools and languages, as shown in the top row.

Development Options Overview



- Development Languages [page 283]
- Developing HTML5 Applications in the Cloud Foundry Environment [page 433]
- Developing SAP HANA in the Cloud Foundry Environment [page 612]
- Deploy Business Applications in the Cloud Foundry Environment [page 845]
- Deploy Docker Images in the Cloud Foundry Environment [page 846]
- Developing Applications and Services [page 261]
- Developing User Interface [page 430]
- Consuming APIs [page 546]
- Adding Authentication and Authorization [page 546]
- Setting Up Database Artifacts [page 612]

- Deploying to the Cloud Foundry Environment [page 614]
- Developing with the SAP Cloud Application Programming Model [page 266]
- SAP Business Application Studio [page 262]
- Developing SAPUI5 [page 431]
- https://api.sap.com/ [https://api.sap.com/]
- Protecting Your Application [page 549]
- SAP HANA Cloud [page 613]
- Continuous Integration and Delivery [page 842]
- Multitarget Applications in the Cloud Foundry Environment [page 615]

The Recommended Path

This development approach offers guidance for important development decisions and features proven best practices recommended by SAP. You can follow a model path for application and service development that is based on the Cloud Application Programming Model (CAP). When working with CAP, we recommend using Java and Node.js because they receive the highest level of tool support and are well suited for most use cases. This path provides you with a list of key aspects to consider, but the order shown in these steps isn't mandatory. You can adapt the steps as you wish to better fit your use case.

Choose Your Own Path

Because of the polyglot nature of the Cloud Foundry environment, you're also free to choose your own approach. You're not forced to use one language exclusively, but can choose between Java, Node.js, and Python. Concerning tools you can either use the Cloud Foundry Command Line Interface (CF CLI) or other tools to develop and deploy your application. You're also free to decide if you want to develop and deploy your applications in the multitarget format (MTA).

For more information on the supported programming languages, see:

- Developing Java in the Cloud Foundry Environment [page 284]
- Developing Node.js in the Cloud Foundry Environment [page 384]
- Developing Python in the Cloud Foundry Environment [page 394]

If you already have monolithic applications running on SAP BTP and are looking for a way to run them in the Cloud Foundry environment, read our migration best practice guide. See Migrating from the Neo Environment to the Multi-Cloud Foundation for SAP BTP (Cloud Foundry and Kyma).

Related Information

Cloud Foundry Environment [page 63]
Troubleshooting App Deployment and Health
SAP BTP, Cloud Foundry Runtime

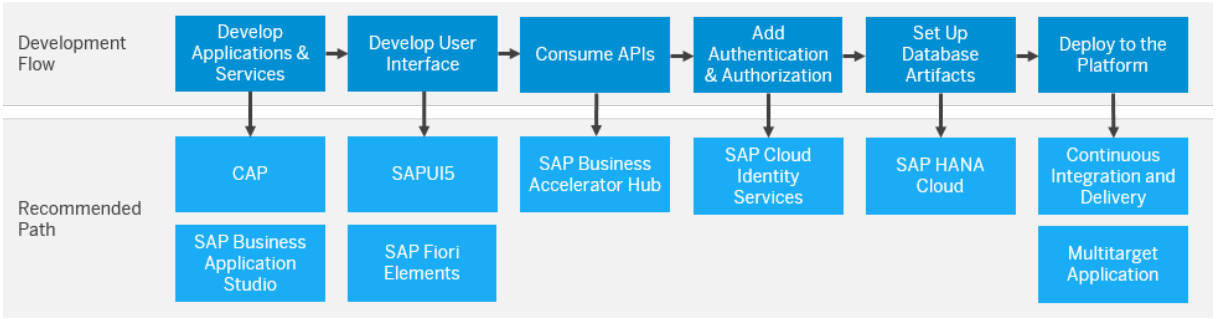
4.1.1 Best Practices

Choose the development environment, tools, APIs, and programming model that best suit your needs with recommendations from SAP.

Navigating the large portfolio of solutions, services, tools, and frameworks can be difficult. That's why we want to provide you with recommendations based on our years of experience in helping customers achieve their goals. The best practices listed here are targeted at developers. If you're looking for recommendations on general development planning and setting up SAP BTP, see SAP BTP Administrator's Guide. If you're interested in extending existing applications, see Extensions and Integrations [page 2281].

The following graphic offers an overview of the most important considerations to take when starting development on SAP BTP for Cloud Foundry. The steps are recommendations, you're free to complete them in a different order if you feel it suits your needs better.

Development Options



- Developing Applications and Services [page 261]
- Developing User Interface [page 430]
- Consuming APIs [page 546]
- Adding Authentication and Authorization [page 546]
- Setting Up Database Artifacts [page 612]
- Deploying to the Cloud Foundry Environment [page 614]
- Developing with the SAP Cloud Application Programming Model [page 266]
- Developing SAPUI5 [page 431]
- https://api.sap.com/ [https://api.sap.com/]
- Protecting Your Application [page 549]
- SAP HANA Cloud [page 613]
- SAP Business Application Studio [page 262]
- Continuous Integration and Delivery [page 842]
- Multitarget Applications in the Cloud Foundry Environment [page 615]

The Recommended Path: The SAP Cloud Application Programming Model

We recommend using the SAP Cloud Application Programming Model (CAP) for full-stack development. CAP is a framework of languages, libraries, APIs, and tools that guide developers along a proven path of best practices. It was designed with a business domain focus in mind, relying on common patterns and reuse models for programming. With CAP, you can develop multitarget applications or automate tasks such as authorization, integration, or localization to make applications and services easier to fix and maintain. CAP is compatible with any development environment, but we recommend SAP Business Application Studio.

For more information, see Developing with the SAP Cloud Application Programming Model [page 266] and SAP Business Application Studio.

Multitarget Applications

One of the challenges of programming in a cloud environment is the deployment and management of applications consisting of multiple, interdependent components. The agility, flexibility, and resilience of cloud applications brings with it an increased complexity. For example, your application could be targeted at multiple runtimes or consist of interconnected modules created in different tools and programming languages.

To reduce this complexity, we recommend programming multitarget applications (MTA). That means packaging all the components of your application into a single archive file. Doing so makes managing the application's lifecycle easier and enables you to automate processes, for example through the Continuous Integration and Delivery service.

For more information, see Multitarget Applications in the Cloud Foundry Environment [page 615].

SAP Alert Notification service

Applications running in Cloud Foundry are constantly monitored through health checks. If an application doesn't respond, for example because it crashed, it fails the health check and is automatically restarted. Because the system restarts applications quickly to avoid major disruptions, it can be difficult to identify underlying problems. For more information about configuring Health Checks, see https://docs.cloudfoundry.org/devguide/deploy-apps/healthchecks.html

To support the smooth operation of your applications and services, we recommend using the SAP Alert Notification service. Using a standardized environment-agnostic model, the service collects any update from the application checks as well as crucial technical information from other services on SAP BTP. It also handles custom scenarios that only occur in your specific application and environment. Each piece of information is translated into a common event model.

You can subscribe to events that are of interest to you and use a delivery channel of your choice, for example email or a custom webhook allowing you to send events to any REST API endpoint in the public internet. SAP Alert Notification service also natively supports integration with external systems, such as Slack, Microsoft Teams, VictorOps, ServiceNow. For more information on events, see SAP Alert Notification Service Events.

Related Information

What Is SAP Alert Notification Service for SAP BTP

4.1.2 Developing Your First Application

Follow any of these tutorials to develop your first application on SAP BTP. Choose your preferred programming model and technology.

Name	Technology
Build a Business Application Using CAP for Java	SAP Cloud Application Programming Model (CDS and Java)
Build a Business Application Using CAP for Node.js	SAP Cloud Application Programming Model (CDS and Node.js)
Develop a Simple Hello World Application Using SAPUI5	SAPUI5
Create an Application with SAP Java Buildpack	Java
Create an Application with Cloud Foundry Python Buildpack	Python
Create an Application with Cloud Foundry Node.js Buildpack	Node.js

4.1.3 Developing Applications and Services

Get started with developing applications and services in the Cloud Foundry environment.

Basic Considerations

At the outset of every development project stand questions about the goal you want to achieve, which skills, tools, and programming languages are required, and how the finished application or service is deployed and maintained. This topic provides an overview of the programming models, development languages, tools, and services you can leverage in the SAP BTP, Cloud Foundry environment to help you address those questions.

SAP Business Application Studio

Learn about the SAP Business Application Studio, the environment tailored for efficient development of business applications. For more information, see SAP Business Application Studio [page 262].

Business Application Pattern

To find information about the recommended development architecture for business applications in the SAP BTP, Cloud Foundry environment, see Business Application Pattern [page 263].

SAP Cloud Application Programming Model

The SAP Cloud Application Programming Model (CAP) is the recommended framework for application and service development in the Cloud Foundry environment. To learn more, see Developing with the SAP Cloud Application Programming Model [page 266].

Services in the Cloud Foundry Environment

In a PaaS environment, external dependencies such as databases, messaging systems, and file systems are services. To use a service, you need to create a service instance and bind it to your application, using either the cockpit or the Cloud Foundry Command Line Interface (CF CLI). To learn more, see Using Services in the Cloud Foundry Environment [page 271].

Development Languages

Find information and tutorials for the main development languages supported in the SAP BTP, Cloud Foundry environment: Java, Node.js, and Python. See Development Languages [page 283].

Multitenant Applications

If you want to make your applications and services available for use to other SAP BTP users, you can design them as multitenant applications. This allows consumers, who are called tenants in this scenario, to access what you've created through dedicated URLs. For more information, see Developing Multitenant Applications in the Cloud Foundry Environment [page 401].

4.1.3.1 SAP Business Application Studio

SAP Business Application Studio is a new cloud service in the SAP BTP, Cloud Foundry environment offering a modern development environment tailored for efficient development of business applications.

SAP Business Application Studio provides a desktop-grade IDE experience in the cloud, including command line functionalities and optimized editors.

It features dev spaces that function like isolated virtual machines. They contain tailored tools and pre-installed runtimes that can save you time and effort when setting up your development environment and that allow you to efficiently develop, test, build, and run your solutions both locally or in the cloud.

SAP Business Application Studio is the recommended environment when working with the Cloud Application Programming Model (CAP).

For more information, see SAP Business Application Studio.

4.1.3.2 Business Application Pattern

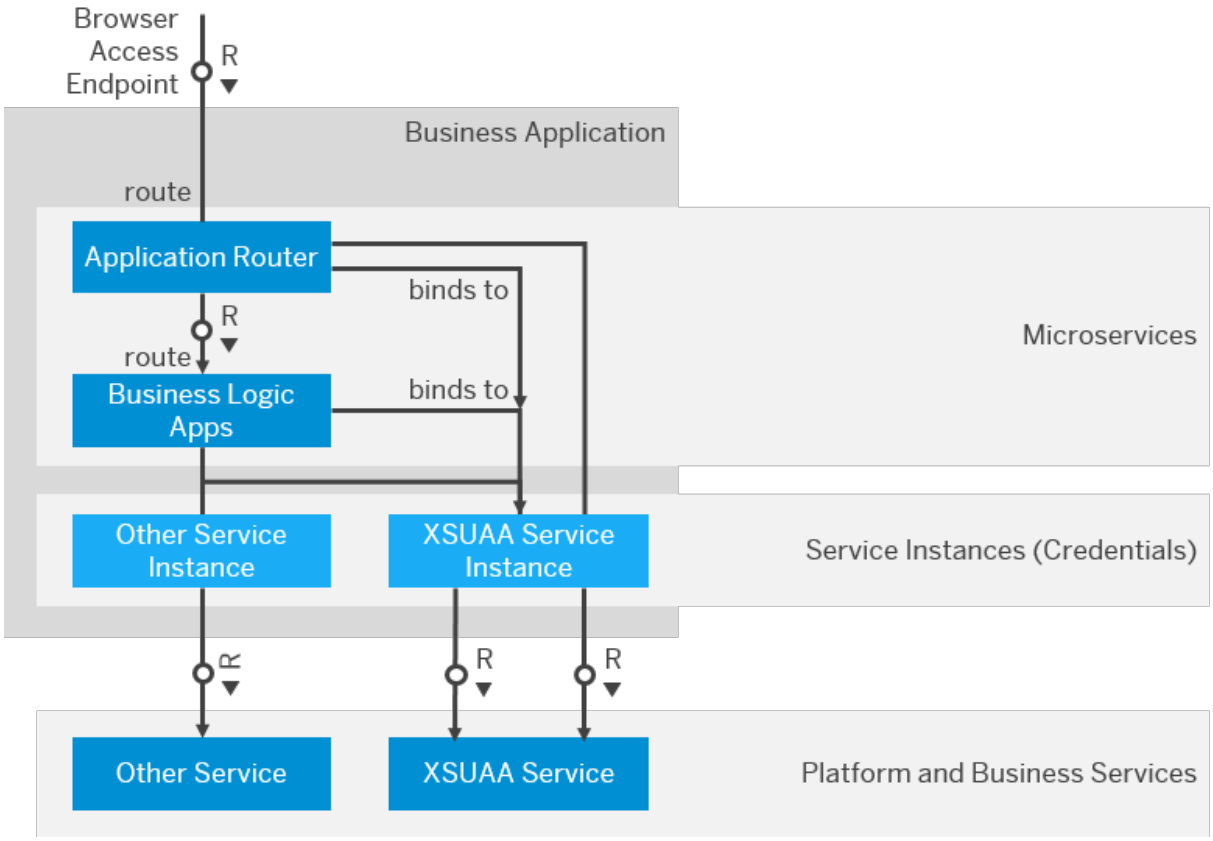
In the Cloud Foundry environment, SAP is promoting a pattern for building applications. We use the term **Business Application** to distinguish from single applications in the context of the Cloud Foundry environment.

This topic covers the following:

- Application Patterns [page 263]
- Application Router [page 264]
- UAA Service [page 265]
- Platform and Business Services [page 265]
- Application Deployment [page 265]

Application Patterns

The diagram below is a logical view, abstracting from numerous details like the CF router, CF controller, and represents architecture of a business application. In general, a business application consists of multiple microservices which are deployed as separate applications to SAP BTP, Cloud Foundry environment.



Microservices, service instances, bindings, services, and routes are entities known to the platform.

Create Microservices from "pushing" code and binaries to the platform resulting in a number of application instances each running in a separate container.

Services are exposed to apps by injecting access credentials into the environment of the applications via service binding. Applications are bound to service instances where a service instance represents the required configuration and credentials to consume a service. Services instances are managed by a service broker which has to be provided for each service (or for a collection of services).

Routes are mapped to applications and provide the actual application access points / URLs.

A business application is a collection of microservices, service instances, bindings, and routes, which together represent a usable web application from an end user point of view. These microservices, services instances, bindings, and routes are created by communicating with the CF / XSA Controller (for example, using a command line interface).

SAP provides a set of libraries, services, and component communication principles, which are used to implement (multi-tenant) business applications according this pattern.

Application Router

Business applications embracing a microservice architecture, consist of multiple services that can be managed independently. Still this approach brings some challenges for application developers, like handling security in a consistent way and dealing with same origin policy.

Application router is a separate component that addresses some of these challenges. It provides three major functions:

- Reverse proxy - provides a single entry point to a business application and forwards user requests to respective backend services
- Serves static content from the file system
- Security – provides security-related functionality like logon, logout, user authentication, authorization, and CSRF protection in a central place

The application router exposes the endpoint accessed by a browser to access the application.

UAA Service

The User Account and Authentication (UAA) service is a multi-tenant identity management service, used in the SAP BTP, Cloud Foundry environment. Its primary role is as an OAuth2 provider, issuing tokens for client applications to use when they act on behalf of the users of the Cloud Foundry environment. It can also authenticate users with their credentials for the Cloud Foundry environment, and can act as an SSO service using those credentials (or others). It has endpoints for managing user accounts and for registering OAuth2 clients, as well as various other management functions.

Platform and Business Services

The platform provides a number of backing services like SAP HANA, MongoDB, PostgreSQL, Redis, RabbitMQ, Audit Log, Application Log, and so on. Also, the platform provides various business services, like retrieving currency exchange rates. In addition, applications can use user-provided services, which are not managed by the platform.

In all these cases applications can bind and consume required services in a similar way. See Services Overview documentation for general information about services and their consumption.

Application Deployment

There are two options for a deployer (human or software agent), how to deploy and update a business application:

- Native deployment: Based on the native controller API of the Cloud Foundry environment, the `deployer` deploys individual applications and creates service instances, bindings, and routes. The `deployer` is responsible for performing all these tasks in an orchestrated way to manage the lifecycle of the entire business application.
- Multitarget Applications in the Cloud Foundry Environment [page 615] (MTA): Based on a declarative model the `deployer` creates an MTA archive and hands over the model description together with all application artifacts to the SAP Deploy Service. This service is performing and orchestrating all individual (native) steps to deploy and update the entire business application (including blue-green deployments).

4.1.3.3 Developing with the SAP Cloud Application Programming Model

The SAP Cloud Application Programming Model (CAP) is a framework of languages, libraries, and tools for building enterprise-grade services and applications. It guides developers along a 'golden path' of proven best practices and a great wealth of out-of-the-box solutions to recurring tasks.

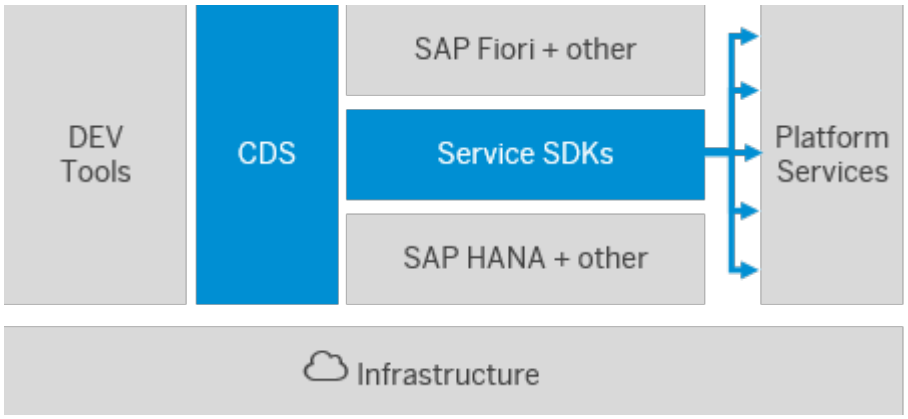
Use Core Data Services (CDS) to build data models and service definitions on a conceptual level. These CDS models are used as inputs for the data, service, and UI layers. They're then translated to native artifacts, for example SQL database schemas, and interpreted to automatically serve requests at runtime.

In summary, CDS is used as a business level data definition source, and to generate the artifacts at the persistence layer. It's used to define visual aspects relating to the data, with those definitions (annotations) defining the UI layer. And it's used to generate the application service layer.

We provide seamless integration with the Cloud Foundry environment on SAP Business Technology Platform. This makes it easier for you to deploy your application and consume platform services.

The programming model is compatible with any development environment, but we recommend using the SAP Business Application Studio.

The following graphic shows the relationship between CAP, SAP Business Technology Platform, platform services, and development tools:



Overview of the SAP Cloud Application Programming Model

Best Practices [page 267]
: To help you create concise and comprehensible models with CDS, we have put together a list of best practices.

Core Data Services (CDS) Language Reference Documentation [page 267]
: CDS is the backbone of the SAP Cloud Application Programming Model (CAP). It provides the means to declaratively capture service definitions and data models, queries, and expressions in plain (JavaScript) object notations. CDS features to parse from a variety of source languages and to compile them into various target languages.

Developing Business Applications Using Java [page 268]
: The CAP Java SDK enables developing SAP Cloud Application Programming Model (CAP) applications in Java. While the SAP Business Application Studio provides excellent support to develop CAP Java applications, you can also develop locally with your tool of choice, for example Eclipse.

Developing Business Applications Using Node.js [page 269]

Develop a sample business service using Core Data Services (CDS), Node.js, and SQLite, by using the SAP Cloud Application Programming Model (CAP). Develop on your local environment and deploy to the Cloud.

References [page 270]
Find additional references in the following sections.

4.1.3.3.1 Best Practices

To help you create concise and comprehensible models with CDS, we have put together a list of best practices.

- Domain Modeling - Find here an introduction to the basics of domain modeling with CDS, complemented with recommended best practices.
- Providing Services - Learn in this guide how to define services exposed through REST and OData APIs, including an overview of generic features to serve metadata as well as most CRUD requests out of the box, input validations and more.
- Consuming Services - CAP provides manifold generic handlers for common, recurring tasks, which serve many requests out-of-the-box, thereby capturing proven best practices collected from a wealth of successful SAP applications.
- Using Databases - This guide is about defining, providing, implementing, deploying, and publishing services — so about Service Providers in general.
- Localization - Guides you through the steps to internationalize your application to provide localized versions with respect to both Localized Models as well as Localized Data.
- Localized Data - Guides you through the steps to internationalize your application to provide localized versions.
- Temporal Data - Temporal data allow maintaining information relating to past, present, and future application time.
- Authorization - About restricting access to data by adding respective declarations to CDS models, which are then enforced in service implementations.
- SaaS Extensibility - How SaaS subscribers can extend SaaS applications on a tenant level.
- Multitenancy - How to implement multitenant aware applications (SaaS applications) with CAP.
- Serving OData APIs - All about OData Annotations, OData Vocabularies, Data Aggregation, and OData V2 Support.
- Serving Fiori UIs - Explains how to add one or more SAP Fiori Elements apps to a CAP project and how to add SAP Fiori Elements annotations to respective service definitions.
- Deploy to Cloud - How to deploy your services to the cloud.
- Native SAP HANA - How to use existing database objects with CDS.

4.1.3.3.2 Core Data Services (CDS) Language Reference Documentation

CDS is the backbone of the SAP Cloud Application Programming Model (CAP). It provides the means to declaratively capture service definitions and data models, queries, and expressions in plain (JavaScript) object

notations. CDS features to parse from a variety of source languages and to compile them into various target languages.

CDS models are plain JavaScript objects complying to the Core Schema Notation (CSN), an open specification derived from JSON Schema. You can easily create or interpret these models, which foster extensions by 3rd-party contributions. Models are processed dynamically at runtime and can also be created dynamically.

For further information, please refer to the following sections:

- Upgrade to Compiler v2 - CDS compiler version 2 (cv2) brings numerous improvements, which allow us to significantly streamline model processing going forward.
- Definition Language (CDL) - A reference and overview of all CDS concepts and features in the form of compact examples.
- Schema Notation (CSN) - Specification of CSN, CDS' canonical format for representing CDS models as plain JavaScript objects, similar to JSON Schema.
- Query Language (CQL) - Documents the CDS Query Language (aka CQL) which is an extension of the standard SQL SELECT statement.
- Query Notation (CQN) - Specification of the Core Query Notation (CQN) format that is used to capture queries as plain JavaScript objects.
- Expression Notation (CXN) - Specification of the Core Expression Notation (CXN) used to capture expressions as plain JavaScript objects.
- Built-in Types - List of provided built-in types.
- Common Types & Aspects - Introduces `@sap/cds/common` a prebuilt CDS model shipped with `@sap/cds` that provides common types and aspects.
- Common Annotations - A reference and glossary of common annotations intrinsically supported by the CDS compiler and runtimes.
- Compiler Messages - This page lists selected error messages and explanations on how to fix them.
- The Nature of Models - Introduces the fundamental principles of CDS models.
- Node.js API - The CDS compiler is implemented in Node.js. Find here the reference documentation for the respective APIs which can be used in CLI tools as well at runtime in CDS service implementations.

4.1.3.3.3 Developing Business Applications Using Java

The CAP Java SDK enables developing SAP Cloud Application Programming Model (CAP) applications in Java. While the SAP Business Application Studio provides excellent support to develop CAP Java applications, you can also develop locally with your tool of choice, for example Eclipse.

The CAP Java SDK supports lean application design by its modular architecture, that means you pick the required features and add them to your application dependencies on demand.

It enables local development by supporting in-memory or file-based SQLite databases. At the same time, the CAP Java SDK enables switching to a productive environment, using, for example, SAP HANA as a database, easily by simply switching the application deployment configuration.

The following sections help you get started.

- Getting Started
- Using Local Development

- Starting a New Project
- Stack Architecture
- Services
- Event Handlers
- Working with Data
- Building CQL Statements
- Executing CDS QL Statements
- Introspecting CQL Statements
- Working with CDS Models
- Application Services
- Fiori Drafts
- Indicating Errors
- Request Contexts
- ChangeSet Contexts
- Security
- Remote Services
- Messaging
- Multitenancy
- Persistence Services
- Audit Logging
- Development
- Observability
- Migration

4.1.3.3.4 Developing Business Applications Using Node.js

Develop a sample business service using Core Data Services (CDS), Node.js, and SQLite, by using the SAP Cloud Application Programming Model (CAP). Develop on your local environment and deploy to the Cloud.

The following sections describe how to set up a development environment to get you started.

- Getting Started in a Nutshell
- Jumpstarting Projects
- Defining Domain Models
- Defining Services
- Using Databases
- Adding/Serving UIs
- Adding Custom Logic
- Deploying to Cloud
- Model Processing in Node.js
- The `cds` Facade Object
- Node.js Core Services API

- Authentication
- Application Service Providers
- Remote Services
- Transaction Management
- Databases
- Messaging
- Best Practices
- Using TypeScript
- CDS Design Time APIs

4.1.3.3.5 References

Find additional references in the following sections.

4.1.3.3.5.1 What's New

Find out what is new and what has changed in CAP since the last release.

- Release Notes
- Changelog

4.1.3.3.5.2 Learning Content

Learn about the golden path that CAP gives users to create service-based applications. Get to know the principles that enable you to focus on your domain and avoid boilerplate code. With CAP you focus on what you can do best: Solving business problems.

Try out the Learning Sources we collected on this page.

4.1.3.3.5.3 Sample Projects

CAP enables you to quickly create business applications by allowing you to focus on your domain logic. It offers a consistent end-to-end programming model that includes languages, libraries, and tools tailored for full-stack development on SAP BTP.

Get started with samples and reusable packages created based on CAP for Node.js and Java provided can be run in a local setup on, for example, a SQLite database.

4.1.3.3.5.4 Tutorials

- Learn how to create a business service using CAP for Java and deploy it to SAP BTP: Build a Business Application Using CAP for Java.
- Learn how to create a business service with CDS using Node.js and deploy it to SAP BTP: Create a Business Service with Node.js using Visual Studio Code.
- Find more tutorials using the SAP Cloud Application Programming Model (CAP) in the Tutorial Navigator.

4.1.3.3.5.5 Community, Blogs & Q&A

Learn more about CAP by visiting SAP Community.

- Find more samples, openSAP course material, podcasts, and other related resources in the SAP Cloud Application Programming Model community.
- Read the blogs including the original introduction by Daniel Hutzel and a great collection of starting points by DJ Adams.
- Ask questions related to CAP to get help from developers and other community members.

4.1.3.3.5.6 Troubleshooting

If you run into issues working with CAP samples or tutorials, please refer to the Troubleshooting guide. There you can find a collection of frequently asked questions and provided solutions.

Furthermore, you can get support as mentioned in section Resources.

4.1.3.4 Using Services in the Cloud Foundry Environment

Learn more about using services in the Cloud Foundry environment, how to create (user-provided) service instances and bind them to applications, and how to create service keys.

- About Services [page 272]
- Binding Service Instances to Applications [page 276]
- Creating Service Instances [page 273]
- Creating Service Keys [page 278]
- Creating User-Provided Service Instances [page 275]
- Deleting Service Instances [page 279]
- Updating Service Instances [page 281]

SAP Service Manager

You can also access services for the Cloud Foundry environment through SAP Service Manager, the central registry for service brokers and platforms. It's tightly integrated with SAP BTP and allows you to consume services in connected runtime environments and to manage platforms, service brokers, service instances, and service bindings.

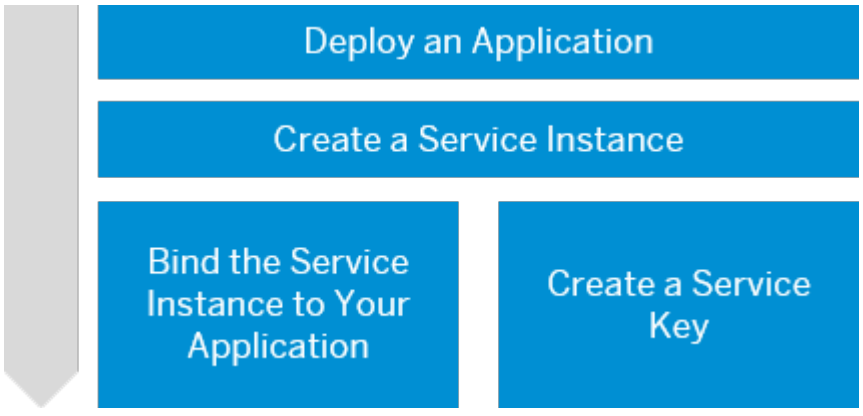
- For more information, see SAP Service Manager.
- To find out if SAP Service Manager is available in your desired region, search for it in the SAP Discovery Center - Service Catalog. In the *Pricing* tab, choose the globe icon to display a map of available regions for this service.

4.1.3.4.1 About Services

In the Cloud Foundry environment, you usually enable services by creating a service instance using either the SAP BTP cockpit or the Cloud Foundry command line interface (cf CLI), and binding that instance to your application.

In a PaaS environment, all external dependencies, such as databases, messaging systems, files systems, and so on, are services. In the Cloud Foundry environment, services are offered in a marketplace, from which users can create service instances on-demand. A service instances is a single instantiation of a service running on SAP BTP. Service instances are created using a specific service plan. A service plan is a configuration variant of a service. For example, a database may be configured with various "t-shirt sizes", each of which is a different service plan.

To integrate services with applications, the service credentials must be delivered to the application. To do so, you can bind service instances to your application to automatically deliver these credentials to your application. Or you can use service keys to generate credentials to communicate directly with a service instance. As shown in the figure below, you can deploy an application first and then bind it to a service instance:



Using Services in the Cloud Foundry Environment

Alternatively, you can also bind the service instance to your application as part of the application push via the application manifest. For more information, see https://docs.cloudfoundry.org/devguide/deploy-apps/manifest.html#services-block .

> ### Note
>
> Have in mind that you need to create a service instance first before you integrate it into your application manifest.

The Cloud Foundry environment also allows you to work with user-provided service instances. User-provided service instances enable developers to use services that are not available in the marketplace with their applications running in the Cloud Foundry environment. Once created, user-provided service instances behave in the same manner as service instances created through the marketplace. For more information, see Creating User-Provided Service Instances [page 275].

For more conceptual information about using services in the Cloud Foundry environment, see https://docs.cloudfoundry.org/devguide/services/ .

Related Information

Creating Service Instances [page 273]
Binding Service Instances to Applications [page 276]
Creating Service Keys [page 278]
Creating User-Provided Service Instances [page 275]
Solutions and Services [page 16]

4.1.3.4.2 Creating Service Instances

Use the SAP BTP cockpit or the Cloud Foundry Command Line Interface to create service instances:

- Create Service Instances Using the Cockpit [page 273]
- Create Service Instances Using the Cloud Foundry Command Line Interface [page 274]

You can also create service instances by declaring them as part of your multitarget application (MTA). To learn how to do that, have a look at the service creation parameters.

- Service Instance Parameters [page 756]

4.1.3.4.2.1 Create Service Instances Using the Cockpit

> ### Note
>
> The content of this page has moved to a new location.
>
> To learn how to create service instances using the cockpit, see
>
> Creating Service Instances.

4.1.3.4.2.2 Create Service Instances Using the Cloud Foundry Command Line Interface

You can use the Cloud Foundry Command Line Interface (cf CLI) to create service instances.

Prerequisites

- Download and install the cf CLI and log on to Cloud Foundry.
  For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813] and Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].
- If you are working in an enterprise account, you need to add quotas to the services you purchased in your subaccount before they appear in the service marketplace. Otherwise, only default free-of-charge services are listed. Quotas are automatically assigned to the resources available in trial accounts.
  For more information, see Configure Entitlements and Quotas for Subaccounts [page 2529].

Procedure

1. (Optional) Open a command line and enter the following string to list all services and service plans that are available in your org:

   ```
   cf marketplace
   ```

2. Run the following command to create a service instance:

   ```
   cf create-service SERVICE PLAN SERVICE_INSTANCE
   ```

   Specify the following parameters:
   - `SERVICE`: The name of the service you want to create an instance of.
   - `PLAN`: The name of the service plan you want to use.
   - `SERVICE_INSTANCE`: The new name for your service instance. Use only alphanumeric characters, hyphens, and underscores.

Related Information

Binding Service Instances to Applications [page 276]
Creating User-Provided Service Instances [page 275]
About Services [page 272]

4.1.3.4.3 Creating User-Provided Service Instances

User-provided service instances enable you to use services that are not available in the marketplace with your applications running in the Cloud Foundry environment.

You can create user-provided service instances using the SAP BTP cockpit or the Cloud Foundry Command Line Interface:

- Create User-Provided Service Instances Using the Cockpit [page 275]
- Create User-Provided Service Instances Using the Cloud Foundry Command Line Interface [page 275]

Fore more information on user-provided service instances, see https://docs.cloudfoundry.org/devguide/services/user-provided.html .

4.1.3.4.3.1 Create User-Provided Service Instances Using the Cockpit

> **Note**
>
> The content of this page has moved to a new location.
>
> To learn how to create user-provided service instances using the cockpit, see Creating User-Provided Service Instances in Cloud Foundry.

4.1.3.4.3.2 Create User-Provided Service Instances Using the Cloud Foundry Command Line Interface

Use the Cloud Foundry Command Line Interface to make a user-provided service instance available to Cloud Foundry applications.

Prerequisites

- Download and install the cf CLI and log on to Cloud Foundry. For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813] and Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].
- Obtain a URL, port, user name, and password for communicating with a service that is not available in the marketplace.

Context

For more informaton on user-provided service instances, see https://docs.cloudfoundry.org/devguide/services/user-provided.html .

Procedure

Open a command line and enter the following string to create a user-provided service instance:

```
cf create-user-provided-service SERVICE_INSTANCE [-p CREDENTIALS]
```

Specify the following parameters:

- `SERVICE_INSTANCE`: The new name for your service instance.
- `CREDENTIALS`: Credentials as JSON

Next Steps

To bind your application to the user-provided service instance, follow the steps described in Bind Service Instances to Applications Using the Cloud Foundry Command Line Interface [page 277].

Related Information

Creating Service Instances [page 273]
Creating Service Keys [page 278]
About Services [page 272]

4.1.3.4.4 Binding Service Instances to Applications

Use the SAP BTP cockpit or the Cloud Foundry Command Line Interface to bind service instances to applications:

- Bind Service Instances to Applications Using the Cockpit [page 277]
- Bind Service Instances to Applications Using the Cloud Foundry Command Line Interface [page 277]

You can also bind service instances by declaring them as part of your multitarget application (MTA). To learn how to do that, have a look at the service binding parameters.

- Service Binding Parameters [page 759]

4.1.3.4.4.1 Bind Service Instances to Applications Using the Cockpit

> **Note**
>
> The content of this page has moved to a new location.
>
> To learn how to bind service instances to applications using the cockpit, see Binding Service Instances to Cloud Foundry Applications.

4.1.3.4.4.2 Bind Service Instances to Applications Using the Cloud Foundry Command Line Interface

You can bind service instances to applications using the Cloud Foundry Command Line Interface (cf CLI).

Prerequisites

- Download and install the cf CLI and log on to Cloud Foundry. For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813] and Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].
- Deploy an application in the same space in which you plan to create the service instance. For more information, see Deploy Business Applications in the Cloud Foundry Environment [page 845].
- Create a service instance. For more information, see Create Service Instances Using the Cloud Foundry Command Line Interface [page 274].

Procedure

Open a command line and enter the following string:

```
cf bind-service APP-NAME SERVICE_INSTANCE [-c PARAMETERS_AS_JSON]
```

Specify the following parameters:

- `APP_NAME`: Name of the application.
- `SERVICE_INSTANCE`: Name of the service instance.
- `-c`: (Optional) Provide service-specific configuration parameters either in a valid JSON object in-line or in a parameters file. This file reference can be an absolute or relative path to a file.

Related Information

Creating Service Instances [page 273]
Creating Service Keys [page 278]
About Services [page 272]

4.1.3.4.5 Creating Service Keys

You can use service keys to generate credentials to communicate directly with a service instance. Once you configure them for your service, local clients, apps in other spaces, or entities outside your deployment can access your service with these keys.

You can use the SAP BTP cockpit or the Cloud Foundry Command Line Interface to create service keys:

- Create Service Keys Using the Cockpit [page 278]
- Create Service Keys Using the Cloud Foundry Command Line Interface [page 278]

For more information on service keys, see https://docs.cloudfoundry.org/devguide/services/service-keys.html .

4.1.3.4.5.1 Create Service Keys Using the Cockpit

> **Note**
>
> The content of this page has moved to a new location.
>
> To learn how to create service keys using the cockpit, see Creating Service Keys in Cloud Foundry.

4.1.3.4.5.2 Create Service Keys Using the Cloud Foundry Command Line Interface

Use the Cloud Foundry Command Line Interface to create a service key.

Prerequisites

- Download and install the cf CLI and log on to Cloud Foundry. For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813] and Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].
- Create a service instance. For more information, see Create Service Instances Using the Cloud Foundry Command Line Interface [page 274].

Procedure

Run the following command to create a service key:

```
cf create-service-key SERVICE_INSTANCE SERVICE_KEY [-c PARAMETERS_AS_JSON]
```

Specify the following parameters:

- `SERVICE_INSTANCE`: Name of the service instance.
- `SERVICE_KEY`: Name for the service key.
- `-c`: (Optional) Provide service-specific configuration parameters either in a valid JSON object in-line or in a parameters file. This file reference can be an absolute or relative path to a file.

Results

Local clients, apps in other spaces, or entities outside your deployment can now access your service instance with this key.

Related Information

Creating Service Instances [page 273]
Binding Service Instances to Applications [page 276]
About Services [page 272]

4.1.3.4.6 Deleting Service Instances

Use the SAP BTP cockpit or the Cloud Foundry Command Line Interface to delete service instances:

> ⚠ Caution
>
> Be aware that instances will be deleted ultimately. There is no way to revoke this step.

- Delete Service Instances Using the Cockpit [page 280]
- Delete Service Instances Using the Cloud Foundry Command Line Interface [page 280]

You can also delete service instances using the Multitarget Application plug-in for the Cloud Foundry command line interface. This works with the command `deploy`, `bg-deploy`, and `undeploy` with specifying the --delete-services option. To learn how to do that, have a look at the multitarget application commands.

- Multitarget Application Commands for the Cloud Foundry Environment [page 2834]

4.1.3.4.6.1 Delete Service Instances Using the Cockpit

> **Note**
>
> The content of this page has moved to a new location.
>
> To learn how to delete service instances using the cockpit, see Deleting Service Instances.

4.1.3.4.6.2 Delete Service Instances Using the Cloud Foundry Command Line Interface

You can use the Cloud Foundry Command Line Interface (cf CLI) to delete service instances.

Prerequisites

- Download and install the cf CLI and log on to Cloud Foundry.
  For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813] and Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].

Procedure

1. Open a command line and log in.

   ```
   cf l -a <API endpoint>
   ```

2. (Optional) List all services and bound apps in your org:

   ```
   cf services
   ```

3. Unbind the service from any application.

   ```
   cf unbind-service APP_NAME SERVICE_INSTANCE
   ```

4. (Optional) List all service keys for your service instance.

   ```
   cf service-keys SERVICE_INSTANCE
   ```

5. Delete any service key from the service instance.

   ```
   cf delete-service-key SERVICE_INSTANCE SERVICE_KEY
   ```

6. Run the following command to delete a service instance:

   ```
   cf delete-service SERVICE_INSTANCE
   ```

Specify the following parameters:

- `SERVICE_INSTANCE`: The name of your service instance.
- `APP_NAME`: The name of an application your service instance is connected to.
- `SERVICE_KEY`: The name of the service key of your service instance.

Related Information

About Services [page 272]
Deleting Service Instances [page 279]

4.1.3.4.7 Updating Service Instances

Use the Cloud Foundry Command Line Interface to update service instances:

- Update Service Instances Using the SAP BTP Cockpit or Cloud Foundry Command Line Interface [page 281]

You can also update service instances inside a multitarget application if the service broker supports updates. Change the deployment descriptor file or a configuration file for a service instance inside your multitarget application and deploy your application to trigger an update of the respective service instance.

4.1.3.4.7.1 Update Service Instances Using the SAP BTP Cockpit or Cloud Foundry Command Line Interface

> **Note**
>
> We have redesigned SAP BTP cockpit screens to view and manage service instances.
>
> For more information about the procedure to update service instances using the SAP BTP cockpit, see Updating Service Instances.
>
> To learn how to update service instances using the Cloud Foundry Command Line Interface, continue reading this document.

Use Cloud Foundry Command Line Interface to update service instances:

Prerequisites

- Download and install the cf CLI and log on to Cloud Foundry.

For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813] and Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].

Context

You are using a service instance, for which you want to change the plan or the service-specific configuration.

Procedure

1. (Optional) Open a command line and enter the following string to list all services in your space:

   ```
   cf services
   ```

2. (Optional) Enter the following string to list the details of your service:

   ```
   cf service SERVICE_INSTANCE
   ```

3. Run the following command to update your service instance:

   ```
   cf update-service SERVICE_INSTANCE [-p NEW_PLAN] [-c PARAMETERS_AS_JSON] [-t
   TAGS]
   ```

   Specify the following parameters:

   - `SERVICE_INSTANCE`: The name of your service instance as shown when executing `cf services`.

4.1.3.4.8 Boosters

Boosters are a set of guided interactive steps that enable you to select, configure, and consume services on SAP BTP to achieve a specific technical goal.

You can access them directly from your desired global account in the SAP BTP cockpit, by choosing *Boosters* in the navigation menu. This leads to a page where you can find an overview of all available boosters, grouped by capability. From this overview page, you can get quick information about a booster, start a booster, or choose a tile to access the booster details.

Booster Details

The details of boosters are organized in 3 tabs:

- *Overview*
  Here, you can get information about what the booster does, its key features and how that particular booster can help you.

- *Components*
  Here, you can see all the components that are required for the booster to run.
- *Additional Resources*
  Here, you find a list of additional information resources where you can learn more about the concepts and components mentioned in the booster.

Boosters automate processes and always achieve a technical goal, often in the form of an artifact. Artifacts are entities that you develop which may consume technical components (for example, services). Examples of artifacts include applications, content for integration and workflows, or even documents.

When you start a booster, a wizard opens up which guides you through a set of steps. Following these steps enables you to reach the result described in the booster overview.

Related Information

Create a Subaccount [page 2509]
Navigate in the Cockpit [page 2461]
Create Spaces [page 2800]

4.1.3.5 Development Languages

Learn about the development languages and buildpacks supported in the SAP BTP, Cloud Foundry environment.

The polylglot nature of SAP BTP means that you have the freedom to choose between various development languages. While we do recommend using Java and Node.js for most use cases, you're free to use the language that best suits your development needs.

To find more information on the three main languages supported by SAP, follow the links below:

- Developing Java in the Cloud Foundry Environment [page 284]
- Developing Node.js in the Cloud Foundry Environment [page 384]
- Developing Python in the Cloud Foundry Environment [page 394]

> **Note**
>
> Please be informed that the SAP BTP, Cloud Foundry runtime Cloud Service is an open Platform-as-a-Service targeted at microservice development and orchestration. SAP is a member of the Cloud Foundry Foundation, which provides Cloud Foundry, a model for cloud-native application delivery. SAP allows use of certain, publicly available, system buildpacks that are supported by the Cloud Service. However, SAP is not in possession or control of, and cannot be responsible for the content, operation, or use of system buildpacks that are not part of the Cloud Service (including SAP Business Technology Platform).
>
> SAP may, without notice, remove any system buildpacks at any time and at SAP's own discretion.

Tutorials

Tutorial Navigator: Build an Application in the Cloud Foundry Runtime

Guided Answers

Find solutions to some Java, Python and Node.js issues in the troubleshooting section: Troubleshooting [page 400]

Related Information

Best Practices [page 259]

4.1.3.5.1 Developing Java in the Cloud Foundry Environment

Find selected information for Java development on SAP BTP, Cloud Foundry and references to more detailed sources.

Usage

To use a buildpack, specify its name when deploying a Java application to SAP BTP, Cloud Foundry. For example:

```
cf push -f <PATH_TO_APP_MANIFEST> -b sap_java_buildpack_jakarta
```

```
cf push -f <PATH_TO_APP_MANIFEST> -b java_buildpack
```

You can also use the *buildpack* or *buildpacks* attribute to specify it in the `manifest.yml` file:

```
---
applications:
- name: <APP_NAME>
  buildpacks:
  - sap_java_buildpack_jakarta
  ...
```

or in the `mtad.yml` file of your archive:

```
...
modules:
  - name: <APP_NAME>
```

```
    type: java.tomcat
    path: <path_to_archive>
    properties:
      ...
    parameters:
      ...
      memory: 512M
      buildpack: sap_java_buildpack_jakarta
...
```

What's New

To check the latest news and updates about SAP Java Buildpack, go to its release notes: What's New for SAP Java Buildpack

Troubleshooting

If you encounter an issue while using SAP Java Buildpack, you can create an incident for your specific problem, using support component **BC-CP-CF-BLDP**.

Related Information

Buildpacks [page 285]
Customizing the JVM Settings [page 305]
Logging and Tracing [page 329]
Debugging Java Applications [page 334]
Bill of Materials (BOM) [page 364]

4.1.3.5.1.1 Buildpacks

You can choose to deploy applications by using the following Java buildpacks:

- SAP Java Buildpack [page 286] – an SAP-managed buildpack that supports Java 17, 21, 25, Tomcat 10, 11 and TomEE 10
- Community Java Buildpack [page 289] – an open-source product maintained by the java-buildpack community

4.1.3.5.1.1.1 SAP Java Buildpack

SAP Java Buildpack is a Cloud Foundry buildpack for running SapMachine-based applications.

This buildpack supports Java 21 and 25, as well as the following runtimes:

- TomEE 10 [page 290]
- Tomcat 10 [page 295]
- Tomcat 11 [page 298]
- Java Main [page 302]

Usage

To use this buildpack, specify its name when deploying a Jakarta-based application to the SAP BTP, Cloud Foundry environment. You can do it the following ways:

- Specify it directly in the `cf push` command:

  ```
  cf push -f <PATH_TO_APP_MANIFEST> -b sap_java_buildpack_jakarta
  ```

- Specify it in the `manifest.yml` file of your application by using the `buildpacks` attribute:

  ```
  ---
  applications:
  - name: <APP_NAME>
    memory: 512M
    buildpacks:
    - sap_java_buildpack_jakarta
    ...
    env:
      TARGET_RUNTIME: tomcat
  ```

  Then, you can deploy the application like this:

  ```
  cf push <app_name>
  ```

- (MTA only) For multi-target applications, specify it in the `mtad.yaml` deployment descriptor file by using the `buildpack` attribute:

  ```
  ...
  modules:
    - name: <APP_NAME>
      type: java.tomcat
      path: <path_to_archive>
      properties:
        ...
      parameters:
        ...
        memory: 512M
        buildpack: sap_java_buildpack_jakarta
  ...
  ```

  Then, you can deploy the application like this:

  ```
  cf push <app_name>
  ```

Buildpack Versioning

The SAP BTP, Cloud Foundry environment provides four versions of SAP Java Buildpack as part of its system buildpacks:

- *sap_java_buildpack_jakarta* – Holds the latest available version of SAP Java Buildpack. All new features and fixes are provided with this version.
- *sap_java_buildpack_jakarta_<version_latest>* – Holds the latest available version of SAP Java Buildpack. It's available for a limited timeframe (4 to 6 weeks).
- *sap_java_buildpack_jakarta_<version_previous>* – This version used to be latest in the previous update of the SAP BTP, Cloud Foundry environment. It's available for a limited timeframe (4 to 6 weeks).
- *sap_java_buildpack_jakarta_<version_before_previous>* – This version used to be latest before two updates of the SAP BTP, Cloud Foundry environment. It's available for a limited timeframe (4 to 6 weeks).

To check these versions, proceed as follows:

1. Log in to a particular SAP BTP region and subaccount. For example, if your region is **eu10**, run:

   ```
   cf api https://api.cf.eu10.hana.ondemand.com
   ```

2. Then run:

   ```
   cf buildpacks
   ```

How to use versions of SAP Java Buildpack?

- **Option 1:** Use the default one – *sap_java_buildpack_jakarta*
  You take advantage of all latest features and fixes in SAP Java Buildpack. This way, it's guaranteed that the buildpack is always available. The drawback in this case is the limited time for adoption, if it's needed. In such a scenario, applications can fall back to an older version temporarily to avoid any downtime.
- **Option 2:** Set a particular version – *sap_java_buildpack_jakarta_<version_suffix>*
  Bear in mind that this version will only exist for a limited amount of time. This may lead to the situation where application restage is failing because the used version of the buildpack is no longer available. To avoid this, we recommend that you follow the updates of the buildpack and test your applications with its latest version. Developers should never allow their applications to run on an outdated buildpack version.

Example:

Let's say that the latest version of SAP Java Buildpack is **2.69.0**. Then, the output of the `cf buildpacks` command would be:

```
buildpack                          position    enabled     locked    filename
sap_java_buildpack_jakarta            1          true      false
sap_java_buildpack_jakarta-v2.69.0.zip
sap_java_buildpack_jakarta_2_69       2          true      false
sap_java_buildpack_jakarta-v2.69.0.zip
sap_java_buildpack_jakarta_2_68       3          true      false
sap_java_buildpack_jakarta-v2.68.0.zip
sap_java_buildpack_jakarta_2_67       4          true      false
sap_java_buildpack_jakarta-v2.67.0.zip
```

When SAP Java Buildpack is updated on the SAP BTP, Cloud Foundry environment from version **2.69.0** to **2.70.0**, the list will change to:

```
buildpack                          position    enabled     locked    filename
```

```
sap_java_buildpack_jakarta           1         true      false
sap_java_buildpack_jakarta-v2.70.0.zip
sap_java_buildpack_jakarta_2_70      2         true      false
sap_java_buildpack_jakarta-v2.70.0.zip
sap_java_buildpack_jakarta_2_69      3         true      false
sap_java_buildpack_jakarta-v2.69.0.zip
sap_java_buildpack_jakarta_2_68      4         true      false
sap_java_buildpack_jakarta-v2.68.0.zip
```

This means that *sap_java_buildpack_jakarta_2_67* will no longer be available for applications.

> **Note**
>
> No fixes will be provided to older versions of the buildpack. Fixes, including security ones, will be part of the latest version.

How to switch to a specific buildpack version?

To use *sap_java_buildpack_jakarta_<version_suffix>*, specify its name when pushing an application to the SAP BTP, Cloud Foundry environment:

```
cf push -f <PATH_TO_APP_MANIFEST> -b sap_java_buildpack_jakarta_2_70
```

Alternatively, you can specify the buildpack in the `manifest.yml` file.

For example:

```
---
applications:
- name: myapp
  memory: 128M
  path: ./target/myapp.war
  instances: 1
  buildpacks:
  - sap_java_buildpack_jakarta_2_70
```

You can do the same in the `mtad.yml` of your **mtar** archive:

```
...
modules:
  - name: myapp
    type: java.tomcat
    path: ./target/myapp.war
    properties:
      ...
    parameters:
      ...
      memory: 512M
      buildpack: sap_java_buildpack_jakarta_2_70
...
```

Supported Java Versions

SAP Java Buildpack (`sap_java_buildpack_jakarta`) supports the following Java versions:

- Java **21** – **default version**. You can obtain it by using SapMachine 21 (*it provides a JRE with Java 21*)
- Java **25** – you can obtain it by using SapMachine 25 (*it provides a JRE with Java 25*)

To learn how to configure your application to use SapMachine JRE and JDK, see: SapMachine [page 314]

Components

SAP Java Buildpack provides the following components in the application container (`<APP_ROOT_DIR>/app/META-INF/.sap_java_buildpack`):

- Runtimes – Tomcat 10 [page 295], Tomcat 11 [page 298], TomEE 10 [page 290], and Java Main [page 302]
- SapMachine [page 314] – version 21, 25
- Memory Calculator [page 317]
- SAP BTP Security Services Integration Libraries – version 3.x

To check all the components regularly updated in the SAP Java Buildpack releases, see: SAP Java Buildpack BOM

Libraries

To see the latest updates about SAP Java Buildpack libraries provided on Maven Repository, regularly check page: SAP Java Buildpack: Libraries [page 366]

What's New

To see the latest news and updates about SAP Java Buildpack, regularly check the release notes on the What's New portal.

Troubleshooting

If you encounter an issue while using SAP Java Buildpack, you can:

- Create an incident for your specific problem, using support component **BC-CP-CF-BLDP**. To provide the necessary details, use the following template: Initial Problem-Related Data [page 400]

4.1.3.5.1.1.2 Community Java Buildpack

Find SAP-managed Cloud Foundry Java components and updates that belong to the community java-buildpack .

Buildpack Versioning

The SAP BTP, Cloud Foundry environment provides one recent version of `java_buildpack` as part of its system buildpacks. To check this version, proceed as follows:

1. Log in to a particular SAP BTP region and subaccount. For example, if your region is **eu10**, run:

   ```
   cf api https://api.cf.eu10.hana.ondemand.com
   ```

2. Then run:

   ```
   cf buildpacks
   ```

To learn about changes in the Java buildpack's versions and features, regularly check the latest buildpack releases in the GitHub community page.

Updates

- **SapMachine** – as of v4.55 of the `java-buildpack`, SAP provides SapMachine 11 and 17 instead of OpenJDK 11 and 17 in the offline `java_buildpack`.
  See SAP Note: 3261748 *SapMachine is replacing OpenJDK 11 & 17 in java_buildpack*

4.1.3.5.1.2 Runtimes and Containers

Find out the available application runtimes and containers you can use for your Java applications.

- TomEE 10 [page 290]
- Tomcat 10 [page 295]
- Tomcat 11 [page 298]
- Java Main [page 302]

Related Information

Buildpacks [page 285]

4.1.3.5.1.2.1 TomEE 10

Web applications deployed with SAP Java Buildpack can run in an Apache TomEE 10 container, which is compatible with Apache Tomcat 10.1.x.

Applications can explicitly define the target application container by using the TARGET_RUNTIME environment variable in the application's `manifest.yml` file

Example

```
---
applications:
- name: myapp
  ...
  env:
    TARGET_RUNTIME: tomee
```

Provided APIs

The **tomee** application runtime container provides the following standard APIs:

Runtime	Full Name	Supported Specification Version
tomee	**Apache TomEE 10.0** (*Jakarta EE 10, Plume distribution*)	Jakarta Annotations 2.1 Jakarta Authentication (JASPIC) 3.0 Jakarta Debugging Support for Other Languages 2.0 Jakarta Servlet 6.0 Jakarta Server Pages (JSP) 3.1 Jakarta Expression Language (EL) 5.0 Jakarta WebSocket 2.1 Jakarta Standard Tag Library (JSTL) 3.0 Jakarta Enterprise Beans (EJB) 4.0 Jakarta Persistence API (JPA) 3.1 Jakarta Bean Validation 3.0 Jakarta Managed Beans 2.0 Jakarta Interceptors 2.1 Jakarta Dependency Injection 2.0 Jakarta Contexts and Dependency Injection (CDI) 4.0 . . . For a full list of specification versions, see: Apache TomEE 10.0

→ Tip

TomEE 10 supports `.ear` deployments. You can apply resource configuration for `.ear` deployments as far as TomEE 10 configuration files are considered.

Resource configuration for custom files in the child `.ear` modules is **not** supported.

Customize the buildpack defaults

SAP Java Buildpack provides some default configurations for the Apache TomEE 10 application container. They can be customized by the application with the Resource Configuration [page 323] feature.

Below is a list of all the placeholders that can be customized by the application, along with their default values:

Placeholder	Description	Default Value
`connector.maxHttpHeaderSize`	The maximum size of the request and response HTTP header, specified in bytes	**8192**
`connector.maxPartHeaderSize`	The maximum number of header bytes permitted per part in a request where the content type is `multipart/form-data`	**512**
`connector.maxPartCount`	The maximum total number of parts permitted in a request where the content type is `multipart/form-data`	**50**
`connector.maxPostSize`	The maximum size (in bytes) of the POST request, which will be handled by the container FORM URL parameter parsing. The limit can be disabled by setting the attribute's value to any integer number less than zero (0). **Note:** You can use the Failed Request Filter to reject requests that exceed the default limit.	**2097152** (2 MiB)
`connector.maxThreads`	The maximum number of request processing threads to be created by this Connector, which therefore determines the maximum number of simultaneous requests that can be handled	**200**

Placeholder	Description	Default Value
`connector.encodedSolidusHandling`	A string value configuring request paths containing a `%2f` sequence. • When set to **reject**, these request paths will be rejected with a 400 response. • When set to **decode**, these request paths will have that sequence decoded to `/` at the same time other `%nn` sequences are decoded. • When set to **passthrough**, these request paths will be processed with the `%2f` sequence unchanged.	**reject**
`connector.allowTrace`	A Boolean value that enables or disables the TRACE HTTP method	**false**

Configurations in the **manifest.yml** file:

- To configure the maximum size of the HTTP header, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/conf/server.xml':
{'connector.maxHttpHeaderSize':1024}]"
```

- To configure the maximum size of a header part, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/conf/server.xml':
{'connector.maxPartHeaderSize':256}]"
```

- To configure the maximum number of parts in a request, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcee/conf/server.xml':
{'connector.maxPartCount':40}]"
```

- To configure the maximum size of the POST request, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/conf/server.xml':
{'connector.maxPostSize':800000}]"
```

- To configure the maximum number of request processing threads, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/conf/server.xml':
{'connector.maxThreads':800}]"
```

- To decode the `%2f` sequence of the request paths to `/`, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/conf/server.xml':
{'connector.encodedSolidusHandling':'decode'}]"
```

- To enable the TRACE HTTP method, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/conf/server.xml':
{'connector.allowTrace':true}]"
```

Configure the maximum number of active sessions

SAP Java Buildpack provides the default configurations for unlimited sessions for the Apache TomEE 10 application container. They can be customized by the application with the Resource Configuration [page 323] feature. To limit the number of active sessions, set the `maxActiveSessions` attribute of the `Manager` element in the application's `context.xml` file:

Example

```
<Context>
  <Manager maxActiveSessions="500" />
</Context>
```

Configure the session timeout value

To set session timeout value of active sessions, set the `<session-config>` tag in the application's `web.xml` file:

Example

```
<session-config>
    <session-timeout>1</session-timeout>
</session-config>
```

Configure the context path attribute

The default value of context path in the `server.xml` file is `""` (Empty String). You can override this default value by using `app_context_root` in the application's `manifest.yml` file:

Example

```
...
  env:
    JBP_CONFIG_TOMEE: "[tomee:{app_context_root: test_context_path}]"
...
```

4.1.3.5.1.2.2 Tomcat 10

By default, web applications pushed with SAP Java Buildpack are running in an Apache Tomcat 10 container.

Applications can explicitly define the target application container by using the TARGET_RUNTIME environment variable in the application's `manifest.yml` file.

Example

```
---
applications:
- name: myapp
  ...
  env:
    TARGET_RUNTIME: tomcat
```

Provided APIs

The **tomcat** application runtime container provides the following standard APIs:

Runtime	Full Name	Supported Specification Version
tomcat	**Apache Tomcat 10.1.x**	Java 11 and later Java Servlets 6.0 Java Server Pages (JSP) 3.1 Expression Language (EL) 5.0 Debugging Support for Other Languages 2.0 Java API for WebSocket 2.1 Java Authentication Service Provider Interface for Containers (JASPIC) 3.0 . . . For a full list of specification versions, see: Apache Tomcat Versions

Customize the buildpack defaults

SAP Java Buildpack provides some default configurations for the Apache Tomcat 10 application container. They can be customized by the application with the Resource Configuration [page 323] feature.

Below is a list of all the placeholders than can be customized by the application, along with their default values:

Placeholder	Description	Default Value
`connector.maxHttpHeaderSize`	The maximum size of the request and response HTTP header, specified in bytes	**8192**
`connector.maxPartHeaderSize`	The maximum number of header bytes permitted per part in a request where the content type is `multipart/form-data`	**512**
`connector.maxPartCount`	The maximum total number of parts permitted in a request where the content type is `multipart/form-data`	**50**
`connector.maxPostSize`	The maximum size (in bytes) of the POST request, which will be handled by the container FORM URL parameter parsing. The limit can be disabled by setting the attribute's value to any integer number less than zero (0). **Note:** You can use the Failed Request Filter to reject requests that exceed the default limit.	**2097152** (2 MiB)
`connector.maxThreads`	The maximum number of request processing threads to be created by this Connector, which therefore determines the maximum number of simultaneous requests that can be handled	**200**
`connector.encodedSolidusHandling`	A string value configuring request paths containing a `%2f` sequence. • When set to **reject**, these request paths will be rejected with a 400 response. • When set to **decode**, these request paths will have that sequence decoded to `/` at the same time other `%nn` sequences are decoded. • When set to **passthrough**, these request paths will be processed with the `%2f` sequence unchanged.	**reject**

Placeholder	Description	Default Value
`connector.allowTrace`	A Boolean value that enables or disables the TRACE HTTP method	**false**

Configurations in the **manifest.yml** file:

- To configure the maximum size of the HTTP header, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.maxHttpHeaderSize':1024}]"
```

- To configure the maximum size of a header part, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.maxPartHeaderSize':256}]"
```

- To configure the maximum number of parts in a request, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.maxPartCount':40}]"
```

- To configure the maximum size of the POST request, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.maxPostSize':800000}]"
```

- To configure the maximum number of request processing threads, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.maxThreads':800}]"
```

- To decode the `%2f` sequence of the request paths to `/`, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.encodedSolidusHandling':'decode'}]"
```

- To enable the TRACE HTTP method, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.allowTrace':true}]"
```

Configure the maximum number of active sessions

SAP Java Buildpack provides the default configurations for unlimited sessions for the Apache Tomcat 10 application container. They can be customized by the application with the Resource Configuration [page 323]

feature. To limit the number of active sessions, set the `maxActiveSessions` attribute of the `Manager` element in the application's `context.xml` file:

Example

```
<Context>
  <Manager maxActiveSessions="500" />
</Context>
```

Configure the session timeout value

To set session timeout value of active sessions, set the `<session-config>` tag in the application's `web.xml` file:

Example

```
<session-config>
    <session-timeout>1</session-timeout>
</session-config>
```

Configure the context path attribute

The default value of context path in the `server.xml` file is `""` (Empty String). You can override this default value by using `app_context_root` in the application's `manifest.yml` file:

Example

```
...
  env:
    JBP_CONFIG_TOMCAT: "[tomcat:{app_context_root: test_context_path}]"
...
```

4.1.3.5.1.2.3 Tomcat 11

You can run your web applications on the Apache Tomcat 11 container.

The current default Tomcat version is 10. If you want your applications to run on Tomcat 11, you have to explicitly define it by using the TARGET_RUNTIME environment variable in the application's `manifest.yml` file.

Example

```
---
applications:
```

```
- name: myapp
  ...
  env:
    TARGET_RUNTIME: tomcat11
```

Provided APIs

The **tomcat11** application runtime container provides the following standard APIs:

Runtime	Full Name	Supported Specification Version
tomcat11	**Apache Tomcat 11.0.x**	Java 17 and later Java Servlets 6.1 Java Server Pages (JSP) 4.0 Expression Language (EL) 6.0 Debugging Support for Other Languages 2.0 Java API for WebSocket 2.2 Java Authentication Service Provider Interface for Containers (JASPIC) 3.1 . . . For a full list of specification versions, see: Apache Tomcat Versions

Customize the buildpack defaults

SAP Java Buildpack provides some default configurations for the Apache Tomcat 11 application container. They can be customized by the application with the Resource Configuration [page 323] feature.

Below is a list of all the placeholders than can be customized by the application, along with their default values:

Placeholder	Description	Default Value
`connector.maxHttpHeaderSize`	The maximum size of the request and response HTTP header, specified in bytes	**8192**

Placeholder	Description	Default Value
`connector.maxPartHeaderSize`	The maximum number of header bytes permitted per part in a request where the content type is `multipart/form-data`	**512**
`connector.maxPartCount`	The maximum total number of parts permitted in a request where the content type is `multipart/form-data`	**50**
`connector.maxPostSize`	The maximum size (in bytes) of the POST request, which will be handled by the container FORM URL parameter parsing. The limit can be disabled by setting the attribute's value to any integer number less than zero (0). **Note:** You can use the Failed Request Filter to reject requests that exceed the default limit.	**2097152** (2 MiB)
`connector.maxThreads`	The maximum number of request processing threads to be created by this Connector, which therefore determines the maximum number of simultaneous requests that can be handled	**200**
`connector.encodedSolidusHandling`	A string value configuring request paths containing a `%2f` sequence. • When set to **reject**, these request paths will be rejected with a 400 response. • When set to **decode**, these request paths will have that sequence decoded to `/` at the same time other `%nn` sequences are decoded. • When set to **passthrough**, these request paths will be processed with the `%2f` sequence unchanged.	**reject**
`connector.allowTrace`	A Boolean value that enables or disables the TRACE HTTP method	**false**

Configurations in the **manifest.yml** file:

- To configure the maximum size of the HTTP header, use:

```
env:
```

```
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.maxHttpHeaderSize':1024}]"
```

- To configure the maximum size of a header part, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.maxPartHeaderSize':256}]"
```

- To configure the maximum number of parts in a request, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.maxPartCount':40}]"
```

- To configure the maximum size of the POST request, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.maxPostSize':800000}]"
```

- To configure the maximum number of request processing threads, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.maxThreads':800}]"
```

- To decode the `%2f` sequence of the request paths to `/`, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.encodedSolidusHandling':'decode'}]"
```

- To enable the TRACE HTTP method, use:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml':
{'connector.allowTrace':true}]"
```

Configure the maximum number of active sessions

SAP Java Buildpack provides the default configurations for unlimited sessions for the Apache Tomcat 11 application container. They can be customized by the application with the Resource Configuration [page 323] feature. To limit the number of active sessions, set the `maxActiveSessions` attribute of the `Manager` element in the application's `context.xml` file:

Example

```
<Context>
  <Manager maxActiveSessions="500" />
</Context>
```

Configure the session timeout value

To set session timeout value of active sessions, set the `<session-config>` tag in the application's `web.xml` file:

> ✤ Example

```
<session-config>
    <session-timeout>1</session-timeout>
</session-config>
```

Configure the context path attribute

The default value of context path in the `server.xml` file is `""` (Empty String). You can override this default value by using `app_context_root` in the application's `manifest.yml` file:

> ✤ Example

```
...
  env:
    JBP_CONFIG_TOMCAT11: "[tomcat11:{app_context_root: test_context_path}]"
...
```

4.1.3.5.1.2.4 Java Main

You can create a Java application that starts its own runtime. This allows the usage of frameworks and Java runtimes, such as Spring Boot, Jetty, Undertow, or Netty.

Prerequisites

- You are **not** using the Resource Configuration [page 323] feature of the buildpack.

> ⓘ Note
>
> The resource configurations needed for the database connection are not applicable for Java Main applications. For more information about database connections, see: Configuring a Database Connection [page 340]

Context

In this section, applications like this are referred as *Java Main applications*. The application container provided by SAP Java/Jakarta Buildpack for running Java Main applications is referred as *Java Main container*.

Procedure

1. Make sure your built JAR archive is configured properly.

   Regardless of the tool you use to build your Java application, you have to make sure that the following tasks are performed:

   a. You have built a JAR archive.
   b. You have specified a main class in the `META-INF/MANIFEST.MF` file of the JAR archive.

   Sample Code

   Manifest.MF

   ```
   Manifest-Version: 1.0
   Archiver-Version: Plexus Archiver
   Built-By: p1234567
   Created-By: Apache Maven 3.3.3
   Build-Jdk: 1.8.0_45
   Main-Class: com.company.xs.java.main.Controller
   ```

   c. You have packaged all your dependent libraries in the JAR file, also known as creating an uber JAR or a *flat JAR*.

   If you are using Maven as your build tool, you can use the `maven-shade-plugin` to perform the above tasks.

   Sample Code

   Sample configuration for the `maven-shade-plugin`

   ```
   <build>
   <finalName>java-main-sample</finalName>
   <plugins>
     <plugin>
       <groupId>org.apache.maven.plugins</groupId>
       <artifactId>maven-shade-plugin</artifactId>
       <version>2.4.3</version>
       <configuration>
         <createDependencyReducedPom>false</createDependencyReducedPom>
         <filters>
           <filter>
             <artifact>*:*</artifact>
             <excludes>
               <exclude>META-INF/*.SF</exclude>
               <exclude>META-INF/*.DSA</exclude>
               <exclude>META-INF/*.RSA</exclude>
             </excludes>
           </filter>
         </filters>
       </configuration>
       <executions>
         <execution>
   ```

```
        <phase>package</phase>
        <goals>
          <goal>shade</goal>
        </goals>
        <configuration>
          <transformers>
            <transformer
implementation="org.apache.maven.plugins.shade.resource.ServicesResourceTra
nsformer" />
            <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTra
nsformer">
              <manifestEntries>
                <Main-Class>com.sap.xs.java.main.Controller</Main-Class>
              </manifestEntries>
            </transformer>
          </transformers>
        </configuration>
      </execution>
    </executions>
  </plugin>
</plugins>
</build>
```

2. Configure the `manifest.yml`.
   To be able to push the Java Main application, you need to specify the path to the `.jar` archive in the `manifest.yml` file.

   Sample Code

   manifest.yml

   ```
   ---
   applications:
   - name: java-main
     memory: 128M
     path: ./target/java-main-sample.jar
     instances: 1
   ```

3. (Optional) If you use SAP HANA JDBC, we recommend that you include the dependent JAR files in the uber JAR. Then refer these files, as a space separated list, in the `Class-Path` header field of the MANIFEST.MF file. For example:

   Sample Code

   ```
   Class-Path: jar1-name jar2-name directory-name/jar3-name
   ```

4. Deploy the application on Cloud Foundry. Execute:

   ```
   cf push
   ```

Example

To create a Java Main application that is using its own runtime, proceed as follows:

1. Create an application, named *sample_main*. Use Spring Boot for that purpose.

2. Navigate to the `sample_main` directory of the application, using the command line tool, and build it. To do that, execute:

```
mvn clean install
```

3. After the successful build, check that the `sample_main` directory of the application contains a **sample_main.jar** file.
4. Open the **sample_main.jar** file and check that the `META-INF/MANIFEST.MF` file contains the `Main-Class` header, whose value is the name of the main class.
5. Add the path to the JAR file in the **manifest.yml** file.
   You need to do this to make sure that the application can be pushed to the Cloud Foundry environment. For that purpose, add the following line in the **manifest.yml** file:

```
path: ./target/sample_main.jar
```

6. Finally, deploy the Java Main application. Execute:

```
cf push sample_main
```

4.1.3.5.1.3 Customizing the Java Virtual Machine (JVM) Settings

- Java Options [page 305]
- Java Memory Assistant [AWS, Azure, or GCP Regions] [page 306]
- Out-Of-Memory Behavior [page 307]
- Using an Agent [page 313]
- SapMachine [page 314]
- Memory Calculator [page 317]

4.1.3.5.1.3.1 Java Options

You can configure the Java properties by defining the JBP_CONFIG_JAVA_OPTS environment variable.

You can define JBP_CONFIG_JAVA_OPTS in two ways:

- In the `manifest.yml` file of the application:

  Sample Code

  manifest.yml

```
---
applications:
- name: <app-name>
  memory: 512M
...
  env:
```

```
      JBP_CONFIG_JAVA_OPTS: 'java_opts: -DtestJBPConfig=\$PATH
-DtestJBPConfig1=''test test'' -DtestJBPConfig2="$PATH"'
```

- Using the `cf set-env` command in the Cloud Foundry Command Line Interface (cf CLI):

```
cf set-env myapp JBP_CONFIG_JAVA_OPTS "[java_opts: '-DtestJBPConfig=^%PATH^%
-DtestJBPConfig1=''test test'' -DtestJBPConfig2="\$PATH"']"
```

Escape Strings when Defining JBP_CONFIG_JAVA_OPTS

In the `manifest.yml` File

A single quote `'` is used to enclose the JBP_CONFIG_JAVA_OPTS environment variable in the manifest.yml file. Strings containing the following characters must be quoted: `:`, `{`, `}`, `[`, `]`, `,`, `&`, `*`, `#`, `?`, `|`, `-`, `<`, `>`, `=`, `!`, `%`, `@`, `\`. When a single quote `'` is used, other single quotes `'` in the string must be escaped by doubling it `''`.

Using the `cf set-env` Command

The string must be enclosed in the double quotes `"`. If there are other double quotes in the string, you have escape them using the backslash `\`.

Using Values Containing Spaces

In case you need to specify an option value that has blank spaces in it, you need to surround it with two single quotes `''`. If the value contains special characters, for example `$`, you have to escape them by using the escape character specific for the operating system shell where the application is pushed. In the example below, the `$` in the $PATH string is escaped by using the `\` escape character for Linux:

Sample Code

manifest.yml

```
 ---
 applications:
 - name: <app-name>
   memory: 512M
...
   env:
     JBP_CONFIG_JAVA_OPTS: 'java_opts: -DtestJBPConfig=\$PATH
-DtestJBPConfig1=''test test'' -DtestJBPConfig2="$PATH"'
```

NOTE: If the `$` sign is not escaped, the value of the PATH variable is substituted in the property value.

4.1.3.5.1.3.2 Java Memory Assistant [AWS, Azure, or GCP Regions]

Note

The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

Java Memory Assistant is a Java agent that generates heap dumps based on preconfigured conditions in the memory usage of the application. See: Java Memory Assistant

> **Note**
>
> This public repository went out of maintenance on **Sept 21, 2023**. Nevertheless, its latest version 0.5.0 is working fine so you can keep using it.

When enabled in the buildpack, the agent generates two files – ***.hprof** (heap dump) and ***.addons**, when the configured memory limits are met.

The ***.addons** file contains:

- Command-line parameters
- Implemented interfaces for the classes
- Information (name) about transient fields for the classes
- Class and metaspace statistics
- Stack traces of the last OOM errors

See also: Java Memory Assistant Framework

4.1.3.5.1.3.3 Out-Of-Memory Behavior

SAP Java Buildpack prints a histogram of the heap memory to the logs when the JVM encounters a terminal failure.

> **Output Code**

```
ERR Stopping VM due to OutOfMemoryError
ERR Resource exhaustion event: the JVM was unable to allocate memory from the
heap.
ERR ResourceExhausted! (1/0)
OUT | Instance Count | Total Bytes | Class
Name                                    |
OUT | 30130          | 5556616     |
[C                                              |
OUT | 866            | 2485600     |
[B                                              |
OUT | 29215          | 701160      | Ljava/lang/
String;                                         |
OUT | 3971           | 449528      | Ljava/lang/
Class;                                          |
OUT | 9998           | 319936      | Ljava/util/
HashMap$Node;                                   |
OUT | 3624           | 318912      | Ljava/lang/reflect/
Method;                    |
OUT | 6429           | 205728      | Ljava/util/concurrent/
ConcurrentHashMap$Node; |
OUT | 2821           | 171472      | [Ljava/lang/
Object;                                       |
OUT | 40             | 117328      |
[J                                              |
OUT | 750            | 111096      | [Ljava/util/
HashMap$Node;                                 |
OUT | 5618           | 89888       | Ljava/lang/
Object;                                        |
```

```
OUT | 1054           | 74048       | [Ljava/lang/
String;                          |
OUT | 696            | 62552       |
[I                                          |
OUT | 59             | 51920       | [Ljava/util/concurrent/
ConcurrentHashMap$Node;|
OUT | 1063           | 51024       | Ljava/util/
HashMap;                         |
OUT | 854            | 40992       | Lorg/apache/tomcat/util/modeler/
AttributeInfo;|
...
```

This functionality is **activated** by default.

If you want to deactivate it, add property `-XX:+ExitVMOnOutOfMemoryError` through the `JBP_CONFIG_JAVA_OPTS` property in the application's *manifest.yml* file, like this:

```
---
applications:
- name: <APP_NAME>
  buildpacks:
  - sap_java_buildpack_jakarta
  env:
    JBP_CONFIG_JAVA_OPTS: 'false, java_opts: ''-XX:+ExitVMOnOutOfMemoryError'''
  ...
```

Killing a process with jvmkill

SAP Java Buildpack allows you to set *jvmkill* agent when staging your application. A *jvmkill* agent is added by default to the properties when starting the Java application process. The following configuration is added:

```
-agentpath:META-INF/.sap_java_buildpack/jvm_kill/jvmkill-
<jvmkill_version>.RELEASE-trusty.so=printHeapHistogram=1
```

This is sufficient to print a histogram and kill the process when an out-of-memory (OOM) error occurs.

If you want to disable *jvmkill* and omit its agent being added to the startup properties, use the SJB_NO_JVMKILL environment variable, like this:

```
env:
  SJB_NO_JVMKILL: true
```

Generate Heap Dumps

It's a good practice to generate an `.hprof` heap dump file on the event of *OutOfMemoryError* to analyze and find the root cause of the issue. You can do this the following ways:

- Generate with Java Properties [page 309]
- Generate with Object Store [page 310]

Related Information

ActiveProcessorCount [page 312]
Cloud Foundry: jvmkill

4.1.3.5.1.3.3.1 Generate Heap Dumps with Java Properties

Context

It's a good practice to generate a `.hprof` heap dump file on the event of *OutOfMemoryError* to analyze and find the root cause of the issue.

Various distributions including *SapMachineJre* and *SapMachineJdk* provide standard mechanism for generating a heap dump in the event of an OOM error. Proceed as follows:

Procedure

1. Add the following properties on startup:

```
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=<heap_dump_path>
-XX:OnOutOfMemoryError='kill -9 %p'
```

   - **-XX:+HeapDumpOnOutOfMemoryError** triggers heap dump generation under a default path.
   - **-XX:HeapDumpPath** specifies *where* to specify the heap dump file.
   - **-XX:OnOutOfMemoryError** specifies *what command or script* to be run if OOM occurs. This command usually kills the process.

   > ### Note
   >
   > Killing the process will trigger a restart. Therefore, if you need to investigate further in the heap dump file stored in the application container, the process should not be killed.

2. The `.hprof` heap dump file is generated and stored in an application container location.

   It will be removed once the application instance container is restarted.

4.1.3.5.1.3.3.2 Generate Heap Dumps and Store Them Remotely

If an application is bound to an Object Store service instance, heap dumps can be uploaded automatically to a cloud storage when an OutOfMemoryError occurs.

Prerequisites

1. Create an Object Store service instance. For example, the service name is `myobjectstore`, and its plan is `s3-standard`:

   ```
   cf create-service objectstore s3-standard myobjectstore
   ```

2. In the `manifest.yml` file, bind this service instance to your Java application (`myapp`) and enable remote heap dump load:

   ```
   ---
   applications:
   - name: myapp
     ...
     buildpacks:
     - sap_java_buildpack_jakarta
     env:
       TARGET_TUNTIME: tomcat
       SJB_HEAPDUMP_TO_OBJECTSTORE: "true"
     services:
     - myobjectstore
   ```

3. Deploy your application:

   ```
   cf push myapp
   ```

Your application is bound to an Object Store service instance. Object Store provides the secure cloud storage location (bucket or container) where heap dumps will be uploaded. For more information, see: What Is Object Store?

Context

Use this way when you need to capture and analyze heap dumps from Java applications experiencing *OutOfMemoryError* in environments where local disk space is limited or heap dumps need to be preserved after container restarts.

When this feature is enabled, heap dumps generated during *OutOfMemoryError* are uploaded to a protected and isolated cloud provider environment (AWS S3 or Azure). The respective storage location is automatically provided by the Object Store service, ensuring your heap dumps are stored securely within your organization's cloud infrastructure with the appropriate access control and encryption.

Configuration Properties:

Category	Property	Description	Values
Required	`SJB_HEAPDUMP_TO_OBJECTSTORE`	Enables automatic heap dump uploads to the Object Store service in the event of *OutOfMemoryError*. The uploaded heap dump file is compressed (GZIP).	• **true** • **false** (default)
Optional	`HEAPDUMP_UPLOADER_PART_MB`	Enables multipart upload chunks. Their sizes are in MiB. • For faster networks, increase the default value. • For slower networks, decrease the default value.	Default: **64**
	`HEAPDUMP_UPLOADER_DEBUG`	Enables verbose logging to troubleshoot upload issues.	• **true** • **false** (default)
	`HEAPDUMP_UPLOADER_KILL_JAVA_PROCESS`	Kills the Java process after the heap dump upload completes to trigger container restart.	• **true** (default) • **false**

Once the heap dump is fully uploaded to the storage bucket/container created by the Object Store service, you can acquire it for analysis by using your cloud provider's command line or console. To do that, follow the steps:

Procedure

1. Get the service binding details from your bound Object Store service instance (bucket/container name, access credentials, endpoint URL). To do that, run:

   ```
   cf env myapp
   ```

2. Look for the VCAP_SERVICES section containing your **objectstore** credentials. Example output:

   ```
   System-Provided:
    VCAP_SERVICES: {
      "objectstore": [
        {
          "credentials": {
            "access_key_id": "<access-key>",
            "secret_access_key": "<secret-key>",
            "bucket": "hcp-12345678-8888-abcd-aaa-12345678678",
            "region": "eu-01"
          },
          "instance_name": "myobjectstore",
          "label": "objectstore",
          "plan": "s3-standard"
        }
      ]
    }
   ```

NOTE: Pay attention to the values of the following credentials (you'll need them to download the heap dump):

- `access_key_id`
- `bucket`
- `region`

3. Configure the cloud provider credentials. To do that, set up your cloud provider CLI with the credentials from **Step 2**.

   For example: If you use AWS S3, run the following commands:

   - `export AWS_ACCESS_KEY_ID=<access-key>`
   - `export AWS_SECRET_ACCESS_KEY=<secret-key>`
   - `export AWS_DEFAULT_REGION=<region>`

4. List the available heap dumps. For AWS S3, run:

   ```
   aws s3 ls s3://<bucket>/heapdumps/
   ```

5. Download the heap dump. The heap dump filename follows the pattern: `sjb-<org>-<space>-<app>-<instance>-<timestamp>.hprof`

   For AWS S3, run:

   ```
   aws s3 cp s3://<bucket>/heapdumps/<heapdump-filename>.hprof ./<heapdump-filename>.hprof.gz
   ```

6. Decompress the heap dump. The downloaded file is compressed as `.gzip` and must be unzipped before analysis. To do that, run:

   ```
   gunzip <heapdump-filename>.hprof.gz
   ```

Results

The `.hprof` heap dump file is generated and ready to be analyzed.

4.1.3.5.1.3.4 ActiveProcessorCount

This JVM option overrides the number of CPUs which the virtual machine uses to calculate the size of thread pools for operations (such as garbage collection).

> ⓘ Note
>
> Previously, SAP Java Buildpack used to set JVM option `"-XX:+UseContainerCpuShares"` but it's now replaced by **"-XX:ActiveProcessorCount=<value>"**.

The `ActiveProcessorCount` flag prevents out-of-memory (OOM) errors in case a GC algorithm with high internal memory usage has been activated by the JVM in relatively small containers (< 4GB). Larger containers (>4GB) benefit from a higher `ActiveProcessorCount` value.

SAP Java Buildpack automatically sets the JVM option `"-XX:ActiveProcessorCount=<value>"`, where **<value>** is defined as follows:

- For container instances < 4GB, the `ActiveProcessorCount` is set to **1**.
- For container instances >= 4GB, the `ActiveProcessorCount` is set to 1 CPU per 1GB of container memory. For example, if a container instance is 7GB, the value will be set to **7**.

If you want to explicitly set the value of `ActiveProcessorCount`, you can do that through the JBP_CONFIG_JAVA_OPTS property in the application's *manifest.yml* file:

```
---
applications:
- name: <APP_NAME>
  buildpacks:
  - sap_java_buildpack_jakarta
  env:
    JBP_CONFIG_JAVA_OPTS: 'java_opts: ''-XX:ActiveProcessorCount=7'''
  ...
```

Related Information

Out-Of-Memory Behavior [page 307]

4.1.3.5.1.3.5 Using an Agent

SAP Java Buildpack can work with any Java agent as long as it's included in the `.jar` or `.war` archive of your application.

The buildpack extracts the agent when the application is deployed. To check if the agent is extracted to the expected location, run:

```
cf ssh
```

To use an agent with SAP Java Buildpack, set the `<JBP_CONFIG_JAVA_OPTS>` environment variable as follows:

```
env:
    JBP_CONFIG_JAVA_OPTS: 'java_opts: "-javaagent:<PathToYourAgent>"'
```

The Java agent is platform-agnostic, but must be compatible with the version of the JVM you are using (SAP JVM or SapMachine).

Native Agents

A native agent is a dynamic library. If you want to use one, it must be compatible with the architecture of the platform. For Cloud Foundry, this is Linux x86_64. As an application developer, make sure that the correct agent is used, and regularly apply updates whenever they're needed.

To use a native agent, set the `<JBP_CONFIG_JAVA_OPTS>` environment variable as follows:

```
env:
   JBP_CONFIG_JAVA_OPTS: 'java_opts: "-agentpath:<PathToYourAgent>"'
```

4.1.3.5.1.3.6 SapMachine

SapMachine provides a Java Runtime Environment (JRE) with Java 21 and 25.

It works with the following application containers:

- TomEE 10 [page 290]
- Tomcat 10 [page 295]
- Tomcat 11 [page 298]
- Java Main [page 302]

Activation Using JRE

To activate SapMachine **JRE** in SAP Java Buildpack, add the following environment variable:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jre.SAPMachineJRE']"
  ...
```

By default, the `use_offline_repository` parameter is set to *true*. That means, your application will use the offline SapMachine JRE provided by the buildpack. Default version: **SapMachine JRE 21**

If you set this parameter to *false*, the buildpack will attempt to download SapMachine JRE of a particular version from the GitHub asset repository. This will only work if your Cloud Foundry instance has access to GitHub: SapMachine .

To specify a particular JRE version, use environment variable JBP_CONFIG_SAP_MACHINE_JRE.

SapMachine 25

> → Tip
>
> SAP Java Buildpack provides a customized SapMachine JRE 25 that contains a jdk.compiler module.

We recommend that you use the latest stable version, which is available in the major version of SapMachine JRE 25. Set the JBP_CONFIG_SAP_MACHINE_JRE variable like this:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jre.SAPMachineJRE']"
    JBP_CONFIG_SAP_MACHINE_JRE: '{ version: 25.+ }'
```

In some cases, it can be helpful to pin a particular published version of SapMachine JRE 25. To make the buildpack download this JRE version (for example, 25.0.4), specify it the following way:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jre.SAPMachineJRE']"
    JBP_CONFIG_SAP_MACHINE_JRE: '{use_offline_repository: false, version:
25.0.4 }'
  ...
```

NOTE: To stay secure, you need to update this version string on your own. For this reason, SAP does **not** recommend this approach.

SapMachine 21

> → Tip
>
> SAP Java Buildpack provides a customized SapMachine JRE 21 that contains a jdk.compiler module.

We recommend that you use the latest stable version, which is available in the major version of SapMachine JRE 21. Set the JBP_CONFIG_SAP_MACHINE_JRE variable like this:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jre.SAPMachineJRE']"
    JBP_CONFIG_SAP_MACHINE_JRE: '{ version: 21.+ }'
```

In some cases, it can be helpful to pin a particular published version of SapMachine JRE 21. To make the buildpack download this JRE version (for example, 21.0.12), specify it the following way:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jre.SAPMachineJRE']"
    JBP_CONFIG_SAP_MACHINE_JRE: '{use_offline_repository: false, version:
21.0.12 }'
  ...
```

NOTE: To stay secure, you need to update this version string on your own. For this reason, SAP does **not** recommend this approach.

Activation Using JDK

To use SapMachine **JDK**, add the following environment variable:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jdk.SAPMachineJDK']"
  ...
```

> ### Note
>
> SapMachine JDK is **not bundled** within the buildpack. Therefore, if you want to use SapMachine JDK 21 or 25, you have to download it from the GitHub project as an online component.
>
> However, you can use the jdk.compiler module, which is **provided** by the buildpack as part of the customized SapMachine JRE, v. 21 and 25. To learn more, see section Activation Using JRE [page 314].

To specify an online JDK version (21 or 25), use environment variable JBP_CONFIG_SAP_MACHINE_JDK.

SapMachine 25

We recommend that you use the latest stable version, which is available in the major version of SapMachine JDK 25. Set the JBP_CONFIG_SAP_MACHINE_JDK variable like this:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jdk.SAPMachineJDK']"
    JBP_CONFIG_SAP_MACHINE_JDK: '{ version: 25.+ }'
```

In some cases, it can be helpful to pin a particular published version of SapMachine JDK 25. To make the buildpack download this JDK version (for example, 25.0.4), specify it the following way:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jdk.SAPMachineJDK']"
    JBP_CONFIG_SAP_MACHINE_JDK: '{ version: 25.0.4 }'
  ...
```

NOTE: To stay secure, you need to update this version string on your own. For this reason, SAP does **not** recommend this approach.

SapMachine 21

We recommend that you use the latest stable version, which is available in the major version of SapMachine JDK 21. Set the JBP_CONFIG_SAP_MACHINE_JDK variable like this:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jdk.SAPMachineJDK']"
    JBP_CONFIG_SAP_MACHINE_JDK: '{ version: 21.+ }'
```

In some cases, it can be helpful to pin a particular published version of SapMachine JDK 21. To make the buildpack download this JDK version (for example, 21.0.12), specify it the following way:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jdk.SAPMachineJDK']"
    JBP_CONFIG_SAP_MACHINE_JDK: '{ version: 21.0.12 }'
  ...
```

NOTE: To stay secure, you need to update this version string on your own. For this reason, SAP does **not** recommend this approach.

Related Information

https://sapmachine.io/
SAP Java Buildpack [page 286]
Runtimes and Containers [page 290]
Debug an Application Running on SapMachine [page 334]
Configure a Java Application for Logs and Traces [page 330]

4.1.3.5.1.3.7 Memory Calculator

The memory calculator provides a mechanism to fine-tune the Java Virtual Machine (JVM) memory for an application. Its goal is to ensure that applications perform well while not exceeding a container's memory limit.

Both SAP Java Buildpack and the Community Java Buildpack use (GitHub) Java Buildpack Memory Calculator , also known as *Memory Calculator v2*.

There are three memory types, which can be sized - **stack_threads**, **class_count**, and **headroom**.

Customize the memory options by using the `JBP_CONFIG_JAVA_OPTS` environment variable:

```
---
applications:
```

```
- name: <app-name>
  ...
  env:
    JBP_CONFIG_JAVA_OPTS: "[java_opts: '-Xms144M -Xss3M -Xmx444444K
-XX:MetaspaceSize=66666K -XX:MaxMetaspaceSize=88888K']"
```

Default Settings

SAP Java Buildpack is delivered with a default built-in configuration of the memory sizing options in YML format - see the configuration files in the table below. These configuration files are parsed during application staging, and the memory configuration specified in them is used for calculating the memory sizes of `stack_threads`, `class_count`, and `headroom`.

Default structures of the relevant configuration files:

`config/`
`sap_machine_jre.yml`
	Sample Code
	`# Configuration for JRE repository`
	`---`
	`repository_root: "https://sapmachine.io/`
	`assets/cf/jre/{platform}/{architecture}"`
	`version: 21.+`
	`use_offline_repository: true`
	`repository_root_offline: "{default.repository.root}/`
	`sap_machine_jre/{platform}/{architecture}"`
	`memory_calculator_v2:`
	`  version: 1.+`
	`  repository_root: "{default.repository.root}/`
	`memory-calculator/{platform}/{architecture}"`
	`  class_count:`
	`  headroom: 0`
	`  stack_threads: 250`
	`jvmkill_agent:`
	`  version: 1.+`
	`  repository_root: "{default.repository.root}/`
	`jvmkill/{platform}/{architecture}"`

`config/sap_machine_jdk.yml`

Sample Code

```
# Configuration for JDK repository
---
repository_root: "https://sapmachine.io/
assets/cf/jdk/{platform}/{architecture}"
version: 21.+
repository_root_offline: "{default.repository.root}/
sap_machine_jre/{platform}/{architecture}"
memory_calculator_v2:
  version: 1.+
  repository_root: "{default.repository.root}/
memory-calculator/{platform}/{architecture}"
  class_count:
  headroom: 7
  stack_threads: 250
jvmkill_agent:
  version: 1.+
  repository_root: "{default.repository.root}/
jvmkill/{platform}/{architecture}"
```

The **memory_calculator_v2** section encloses the input data for the memory calculation techniques utilized in determining the JVM memory sizing options.

- `stack_threads` – the number of threads to be used by the application. Default value: **250**
- `headroom` – the percentage of the container's total memory (available to the application) that can be excluded from the memory calculator's algorithm and used for covering JVM overhead (non-JVM operations). The maximum recommended value for headroom is **10**. Default value: **0**
- `class_count` – the number of classes to be loaded. Default behavior: Estimating the number of class names in the application, adding a constant `42,215` to it, and then multiplying the final result by **0.35**. If you set a particular number, for example 500, only 500 classes will be loaded.

You can customize two memory options, by using the relevant environment variables:

- JBP_CONFIG_SAP_MACHINE_JRE
- JBP_CONFIG_SAP_MACHINE_JDK

Java 25

Note

If you need JRE with Java 25, you use SapMachine 25. Customize your memory options as follows:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jre.SAPMachineJRE']"
    JBP_CONFIG_SAP_MACHINE_JRE: "[version: 25.+, memory_calculator_v2:
{stack_threads: 266, class_count: 1001, headroom: 5}]"
```

If you want to point to the SapMachine JDK component, you need to provide a specific version as follows:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jdk.SAPMachineJDK']"
    JBP_CONFIG_SAP_MACHINE_JDK: "[ version: 25.0.4, memory_calculator_v2:
{stack_threads: 266, class_count: 1001, headroom: 5}]"
```

You can also point to the major version of the SapMachine JDK, in order to always get the latest patch versions. In this case, specify it the following way:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jdk.SAPMachineJDK']"
    JBP_CONFIG_SAP_MACHINE_JDK: "[ version: 25.+, memory_calculator_v2:
{stack_threads: 266, class_count: 1001, headroom: 5}]"
```

Java 21

> ⓘ Note

If you need JRE with Java 21, you use SapMachine 21. Customize your memory options as follows:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jre.SAPMachineJRE']"
    JBP_CONFIG_SAP_MACHINE_JRE: "[version: 21.+, memory_calculator_v2:
{stack_threads: 266, class_count: 1001, headroom: 5}]"
```

If you want to point to the SapMachine JDK component, you need to provide a specific version as follows:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jdk.SAPMachineJDK']"
    JBP_CONFIG_SAP_MACHINE_JDK: "[ version: 21.0.12, memory_calculator_v2:
{stack_threads: 266, class_count: 1001, headroom: 5}]"
```

You can also point to the major version of the SapMachine JDK, in order to always get the latest patch versions. In this case, specify it the following way:

```
---
```

```
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jdk.SAPMachineJDK']"
    JBP_CONFIG_SAP_MACHINE_JDK: "[ version: 21.+, memory_calculator_v2:
{stack_threads: 266, class_count: 1001, headroom: 5}]"
```

Java 17

> Note

If you need JRE with Java 17, you use SapMachine 17. Customize your memory options as follows:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jre.SAPMachineJRE']"
    JBP_CONFIG_SAP_MACHINE_JRE: "[version: 17.+, memory_calculator_v2:
{stack_threads: 266, class_count: 1001, headroom: 5}]"
```

If you want to point to the SapMachine JDK component, you need to provide a specific version as follows:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jdk.SAPMachineJDK']"
    JBP_CONFIG_SAP_MACHINE_JDK: "[ version: 17.0.20, memory_calculator_v2:
{stack_threads: 266, class_count: 1001, headroom: 5}]"
```

You can also point to the major version of the SapMachine JDK, in order to always get the latest patch versions. In this case, specify it the following way:

```
---
applications:
- name: <app-name>
  ...
  env:
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jdk.SAPMachineJDK']"
    JBP_CONFIG_SAP_MACHINE_JDK: "[ version: 17.+, memory_calculator_v2:
{stack_threads: 266, class_count: 1001, headroom: 5}]"
```

Related Information

GitHub: Java Buildpack Memory Calculator

SapMachine [page 314]

4.1.3.5.1.4 Overriding Resources in Droplet

Applications can override resources in the droplet by placing directory files that have the same relative path as the droplet root.

The following example shows how an application can override the default Tomcat's **server.xml** file in directory `META-INF/sap_java_buildpack/resources/`.

Example

1. You have the following file structure in your Java application:

```
META-INF/
- sap_java_buildpack/
  - resources/
    - tomcat/
      - conf/
        - server.xml/
WEB-INF/
```

2. After the application is built and deployed, this will result in the following file structure in the droplet:

```
META-INF/
- .sap_java_buildpack/
  - tomcat
    - conf/
      - server.xml
WEB-INF/
```

3. When the Tomcat container is started, the content of the **server.xml** file will come from the application.

This mechanism also allows adding new resources inside the droplet in their respective places, based on where the files are located under `META-INF/sap_java_buildpack/resources/`.

> → Tip
>
> The overriding principle is applicable for both **sap_java_buildpack** and **sap_java_buildpack_jakarta**.

Related Information

Resource Configuration [page 323]
Cloud Foundry component glossary

4.1.3.5.1.5 Resource Configuration

You can configure your application container [page 290] by using text configuration files.

For example:

- `conf/tomee.xml`
- `conf/server.xml`
- `conf/logging.properties`

Resource configuration feature of SAP Java Buildpack provides means for changing parameterized values in all text files (part of the application container or part of the application files) during staging.

Files can contain multiple key-value pairs containing placeholders, marked as **${propname}**. To prevent undesired modifications, all files that can be changed must be explicitly listed by the application in a corresponding `resource_configuration.yml` file, along with the default values for all placeholders.

Template for `resource_configuration.yml`

```
---
filepath1:
  key1: value1
  key2: value2
filepath2:
  key1: value1
```

The `filepath1` must start with either **tomcat** or **tomee** to match the used application container, and should contain a subpath to a resource.

- When changing parameterized values inside application files, the file path should look like this:
  `<application_container>/webapps/ROOT/<path_to_file_relative_to_application_root>`
- When changing parameterized values in the configuration of an application container, the file path should look like this:
  `<application_container>/conf/<path_to_config_file>`

Example 1

You can create a `resource_configuration.yml` with the following content:

```
tomcat/webapps/ROOT/WEB-INF/mytextfile.txt:
  placeholder: "defaultValue"
tomcat/conf/server.xml:
  connector.maxThreads: 800
```

Example 2

1. You have a given application with the following application files:

```
src/
    main/
        java/
        resources/
        webapp/
            META-INF/
                sap_java_buildpack/
                    config/
                        resource_configuration.yml
            WEB-INF/
                mytextfile.txt
```

2. The content of `mytextfile.txt` is:

```
My hometown is ${hometown}.
```

3. The `resource_configuration.yml` looks like this:

```
---
tomcat/webapps/ROOT/WEB-INF/mytextfile.txt:
  hometown: "London"
```

4. After the application is staged, the content of `mytextfile.txt` will be:

```
My hometown is London.
```

Example 3

The following example demonstrates how to provide values for placeholders during staging.

1. Use the data from **Example 2** (steps 1-3).
2. Then, in the `manifest.yml` file of the application, add the following environment variable:

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/webapps/ROOT/WEB-INF/
mytextfile.txt': {'hometown':"Paris"}]"
```

   This gives the possibility to provide values for placeholders that differ from the default ones provided in the example above.
3. After staging of the application, the content of `mytextfile.txt` will be modified to:

```
My hometown is Paris.
```

Related Information

Overriding Resources in Droplet [page 322]

Apache Tomcat 10: Configuration Reference

Apache TomEE: Server Configuration

4.1.3.5.1.6 Context Root Redirect

SAP Java Buildpack provides a context root redirect functionality.

When you call a Web application without adding its runtime [page 290] context path to the URL, it's automatically appended.

Example

If you configure `/my_context_path` as a context path, and the Web application is available on `/my_app`, then:

When you call:	You'll be redirected to:
`<HOST>:<PORT>/my_app`	`<HOST>:<PORT>/my_context_path/my_app`

The default context path value for Tomcat and TomEE is `""` (empty string).

For more information on how to change the default value, see:

- TomEE 10 [page 290]
- Tomcat 10 [page 295]
- Tomcat 11 [page 298]

4.1.3.5.1.7 Multitenant Java Applications

In the Cloud Foundry environment, you can develop and run multitenant Java applications, and share them with multiple consumers simultaneously on SAP BTP.

What is Multitenancy?

SAP BTP provides a multitenant functionality that allows application providers to own, deploy, and operate tenant-aware applications for multiple consumers, with reduced costs. For example, the application provider can upgrade the application for all your consumers instead of performing each update individually, or can share resources across multiple consumers. The application consumers launch the applications using consumer-specific URLs, and can configure certain application features.

To learn more, see: Developing Multitenant Applications in the Cloud Foundry Environment [page 401]

Multitenancy in the SAP Java Buildpack

SAP Java Buildpack provides the possibility for multitenant applications running on Cloud Foundry to consume tenant-aware data sources out of the box. This is achieved by integrating the SAP Service Manager capabilities in the buildpack. SAP Java Buildpack provides out of the box tenant-aware data sources for:

- All tenants that have already been onboarded to an SAP Service Manager service instance.
- Newly onboarded tenants at runtime. No restart of your Java application is needed.

Configure the Application Multitenancy

To achieve multitenant support in your application deployed with SAP Java Buildpack, your application should meet the following requirements:

1. Multitenancy support for Cloud Foundry is set up in advance. See: Developing Multitenant Applications in the Cloud Foundry Environment [page 401]
2. The application should be bound to a managed database service. Your application should also use one of the following data sources that your relevant SAP Java Buildpack version provides:
   - `com.sap.xs.jdbc.datasource.tomcat.TomcatDataSourceFactory`
   - `com.sap.xs.jdbc.datasource.tomee.TomEEDataSourceFactory`
   - `com.sap.xs.jdbc.datasource.tomee7.TomEE7DataSourceFactory`

   See section: Configure Tenant-Aware Data Source [page 326]
3. The security concept that the application uses should be **XSUAA**. See section: Configure the XSUAA Authentication Method [page 327]

Once these requirements are fulfilled, the application takes care of the onboarding of tenants in the SAP Service Manager service instance they use. For each request that comes from the application, SAP Java Buildpack acquires (obtains) the tenant ID from the request through the JWT token provided by the Authorization and Trust Management (XSUAA) service. The buildpack provides a different database instance object for each tenant.

Configure Tenant-Aware Data Source

Once a multitenant application is bound to a managed database service instance, provisioning of the tenant-aware data source comes out of the box.

In the following example of **context.xml**, the `jpa-db-managed` service instance is used to configure the data source. Provided that the `jpa-db-managed` service instance is of type *managed-hana* or *service-manager*, it will make `jdbc/DatasourceOne` tenant-aware.

```
<?xml version='1.0' encoding='utf-8'?>
<Context>
 <Resource name="jdbc/DatasourceOne"
    auth="Container"
    type="javax.sql.DataSource"
    factory="com.sap.xs.jdbc.datasource.tomcat.TomcatDataSourceFactory"
    service="jpa-db-managed"
    personalize="true"/>
```

```
</Context>
```

If a custom tenant provider is used, define the class in the `Resource` configuration as a `tenantProvider`. For example:

```
<?xml version='1.0' encoding='utf-8'?>
<Context>
  <Resource name="jdbc/DatasourceOne"
    auth="Container"
    type="javax.sql.DataSource"
    factory="com.sap.xs.jdbc.datasource.tomcat.TomcatDataSourceFactory"
    service="jpa-db-managed"
    personalize="true"/>
    tenantProvider="com.sap.test.custom.provider.UserDefinedTenantProvider"/>
</Context>
```

Tip: If you want a **non-JPA** application to consume `jdbc/DatasourceOne`, use the following code:

```
public class TestServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    @Resource(name = "jdbc/DatasourceOne")
    private DataSource tenantAwareDataSource;
 ...
}
```

Configure the XSUAA Authentication Method

The configuration is done in the `login-config` section of the **web.xml** file:

```
<?xml version=“1.0” encoding=“UTF-8”?>
 <web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance xmlns="http://
java.sun.com/xml/ns/javaee xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd version=“3.0”>
  <display-name>sample</display-name>
  <login-config>
   <auth-method>XSUAA</auth-method>
  </login-config>
 </web-app>
```

Control Determination of the Current Tenant for Tenant-Aware Data Source

As described above, by default the tenant-aware data source determines which tenant to be used for a specific request by getting the current log user from the JWT token provided by the Authorization and Trust Management (XSUAA) service.

For specific scenarios, the application can get control over the tenant determination with the following steps:

1. Implement and bundle a class which will handle the tenant provisioning. For example:

```
package com.sap.test.custom.provider;
public class UserDefinedTenantProvider implements Supplier<String> {
    private static ThreadLocal<String> currentTenant = new
InheritableThreadLocal<>();
    public static ThreadLocal<String> getCurrentTenant() {
```

```
        currentTenant.set("tenant1");
        return currentTenant;
    }
    @Override
    public String get() {
        return getCurrentTenant().get();
    }
}
```

> **Note**
>
> This class is **required** in order to implement the `Supplier` interface.

2. Define the application class as `tenantProvider`. For example:

```
<?xml version='1.0' encoding='utf-8'?>
<Context>
  <Resource name="jdbc/DatasourceOne"
    auth="Container"
    type="javax.sql.DataSource"
    factory="com.sap.xs.jdbc.datasource.tomcat.TomcatDataSourceFactory"
    service="jpa-db-managed"
    personalize="true"/>
    tenantProvider="com.sap.test.custom.provider.UserDefinedTenantProvider"/>
</Context>
```

Related Information

Providing Multitenant Applications to Consumers in the Cloud Foundry Environment [page 419]

4.1.3.5.1.8 Multiple Buildpack Framework

The Multiple Buildpack Framework enables SAP Java Buildpack to act as the final buildpack in a multiple buildpack deployment. It reads the contributions of other, earlier buildpacks and incorporates them into its standard staging.

> **Note**
>
> If you try to use SAP Java Buildpack as a non-final buildpack, an error will be thrown.

When SAP Java Buildpack acts as the final buildpack in a multiple buildpack deployment, it honors the following core contract integration points with non-final buildpacks:

Integration Point	Buildpack Usage
`/bin`	An existing `/bin` directory contributed by a non-final buildpack will be added to the `<$PATH>` of the application as it executes.

Integration Point	Buildpack Usage
`/lib`	An existing `/lib` directory contributed by a non-final buildpack will be added to the `<$LD_LIBRARY_PATH>` of the application as it executes.

4.1.3.5.1.9 Logging and Tracing

Context

SAP Java Buildpack integrates the Cloud Foundry Java Logging Support libraries, allowing applications to produce logs in JSON format that can be parsed by the SAP Application Logging service for SAP BTP.

Procedure

1. Bind your application to an SAP Application Logging service service instance. This way, you can produce application logs and forward them to a central application logging stack.
2. If you want to generate request metrics for applications based on Tomcat or TomEE, add *com.sap.hcp.cf.logging.servlet.filter.RequestLoggingFilter* to the `web.xml` file.

   ```
   <filter-name>request-logging</filter-name>
      <filter-class>com.sap.hcp.cf.logging.servlet.filter.RequestLoggingFilter</
   filter-class>
   </filter>
   <filter-mapping>
      <filter-name>request-logging</filter-name>
      <url-pattern>/*</url-pattern>
   </filter-mapping>
   ```

   > ### Note
   >
   > The default log level of *com.sap.hcp.cf.logging.servlet.filter.RequestLoggingFilter* is set to INFO.

3. Check the logs of the application.

   If the binding of your application to the SAP Application Logging service is successful, the application logs can be found on `https://logs.cf.<SAP_BTP_region>`. To request the logs of your application, run:

   ```
   cf logs <app_name> --recent
   ```

4. (Optional) Change the log level of a logging location.

   You can do this by using a file. You need to restart your application after that. Use cases:

   - Standard applications based on Tomcat or TomEE – configure the SET_LOGGING_LEVEL environment variable in the `manifest.yml` file. For example:

     ```
     env:
       SET_LOGGING_LEVEL: '{com.sap.sample.java.LogInfo: INFO,
     com.sap.sample.java.LogWarn: WARN}'
     ```

- Standard applications based on Java Main – create a `logback.xml` file in directory `src/main/resources`. For example:

```
<configuration>
  <!-- Console appender -->
  <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</
pattern>
    </encoder>
  </appender>
  <!-- Set root log level -->
  <root level="DEBUG">
    <appender-ref ref="STDOUT" />
  </root>
  <!-- Optional: package-specific logging -->
  <logger name="org.example.Main" level="ERROR"/>
</configuration>
```

- Spring Boot applications based on Java Main. Their default log configuration provides messages of levels ERROR, WARN, and INFO.
  - To enable other log levels, for example DEBUG, create a file `application.properties` in directory `/src/main/resources/` and add the following line:

    ```
    logging.level.root=DEBUG
    ```

    Note: If you use a Spring Boot generator (for example Spring Initializr), the file directory `/src/main/resources/application.properties` is automatically created for you.
  - To enable DEBUG logging for a particular class only, you can do so by specifying the fully qualified class name:

    ```
    logging.level.com.mypackage.MySpecificClass=DEBUG
    ```

Related Information

Configure a Java Application for Logs and Traces [page 330]
SAP Application Logging Service for Cloud Foundry
Spring Boot: Logging

4.1.3.5.1.9.1 Configure a Java Application for Logs and Traces

Configure the collection of log and trace messages generated by a Java application in SAP BTP, Cloud Foundry.

Prerequisites

- You have bound an application to the SAP Application Logging service. See: Logging and Tracing [page 329]

- You have installed and configured Maven.
- For **SAP Java Buildpack**: You use SapMachine 21 or 25, and your <JAVA_HOME> environment variable points to this location.
- You do **not** have any SLF4J and logback JAR files in the application.

> ⚠ Caution
>
> The JARs for the *SLF4J* and *logback* are included as part of the Tomcat and TomEE runtimes provided by SAP Java Buildpack. Packing them in the application can cause problems during class loading.

Context

The recommended framework for logging is *Simple Logging Facade for Java* (SLF4J). To use this framework, you can create an instance of the **org.slf4j.Logger** and **org.slf4j.LoggerFactory** classes. You can use SLF4J to configure your Java application to generate logs and traces, and if appropriate – set the logging and tracing levels.

> ⓘ Note
>
> It is the application's responsibility to ensure that the logback is configured in a secure way in the case when the application overrides the default logback configuration included in SAP Java Buildpack. See step 2 below.

Procedure

1. Instruct Maven that the application should not package the SLF4J dependency, since it's already provided by the runtime.

   To do that, go to the `pom.xml` file and set scope **provided** for the following dependency:

   ```
   <dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>2.0.13</version>
    <scope>provided</scope>
   </dependency>
   ```

2. (Optional) Configure the logback in case the application overrides the default logback configuration. To do that, you need to perform the following steps:

   1. Under path `META-INF/sap_java_buildpack/resources/tomcat/conf`, create a custom `logback.xml` file. For example:

      > ✤ Example
      >
      > ```
      > <?xml version="1.0" encoding="UTF-8"?>
      > <configuration debug="false" scan="false">
      >     <contextListener
      > class="ch.qos.logback.classic.jul.LevelChangePropagator"/>
      > ```

```
    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
        <encoder class="${encoder}">
            ${pattern}
        </encoder>
    </appender>
    <root level="INFO">
        <appender-ref ref="STDOUT"/>
    </root>
</configuration>
```

2. Create a `resource_configuration.yml` file or add the following entries to your existing `resource_configuration.yml` file (if using Tomcat):

   Example

   ```
   ---
   tomcat/conf/logback.xml:
     pattern: ''
     encoder: ''
   ```

   NOTE: You can also add default values for `pattern` and `encoder`.

3. Add the following property in your `manifest.yml` file. For example (if using Tomcat):

   Example

   ```
   ---
   applications:
   - name: <APP_NAME>
     ...
     env:
       JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/logback.xml':
   {'encoder': 'com.sap.hcp.cf.logback.encoder.JsonEncoder', 'pattern':
   ''}]"
   ```

Related Information

Configure the Access Logs of an Application [page 332]

4.1.3.5.1.9.2 Configure the Access Logs of an Application

SAP Java Buildpack uses the *logback-access* module to provide HTTP-access log functionality.

Prerequisites

- Download and Install the Cloud Foundry Command Line Interface [page 2813]
- Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814]

Context

The access logs differ slightly from the other logs and traces. There are many standard tools available that read access logs from any server, display them, calculate statistics and so on. That's why the format of the access log produced by Tomcat and TomEE is not SAP-specific. The default pattern defined in the configuration file is `"%date "%r" %s %b"`.

To find the access log files of your application, you have to connect via SSH to the SAP BTP, Cloud Foundry container and go to the `logs` directory.

To change the default access log settings, follow the steps below.

Procedure

1. Modify the default configuration.
   SAP Java Buildpack uses `LogbackValve` to configure access logs in Tomcat and TomEE. The following environment variables are used in the configuration file and can be overwritten:
   - `<ACCESS_LOG_FILE>`
   - `<ACCESS_LOG_FILE_COUNT>`
   - `<ACCESS_LOG_FILE_MAX_SIZE>`
   - `<logback.access.log.xs.pattern>`
2. (Optional) Change the name of the access log files.
   By default, access logs are placed in the application container in the `logs/access.log` file. You can configure the directory containing this file, as well as the name of the files by using environment variable `ACCESS_LOG_FILE`. To do that, define the full path to the desired directory and the file name. For example:

   ```
   env:
       ACCESS_LOG_FILE: "/home/vcap/logs/access-log/access.log"
   ```

3. (Optional) Change the number of access log files.
   You can change the number of stored files with the environment variable `ACCESS_LOG_FILE_COUNT`. The default value is **3**. For example:

   ```
   env:
       ACCESS_LOG_FILE_COUNT: 5
   ```

4. (Optional) Change the size of the access log files.
   You can modify the size of every access log file with the environment variable `ACCESS_LOG_FILE_MAX_SIZE`. The default value is **8** MB. For example:

   ```
   env:
       ACCESS_LOG_FILE_MAX_SIZE: 10
   ```

5. (Optional) Change the format of the HTTP access log

   You can change the default format and have your access logs written differently. SAP Java Buildpack uses the *resource configuration* functionality to allow this. Changing the `pattern` attribute in the configuration file of Tomcat and TomEE is supported. The application has to provide the new value via the JBP_CONFIG_RESOURCE_CONFIGURATION mechanism.

- For SAP Java Buildpack 1:

```
env:
JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/logback-access-
localhost.xml':{'logback.access.log.xs.pattern' : 'combined'}]"
```

- For SAP Java Buildpack 2:

```
env:
JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/logback-access-
localhost.xml':{'logback.access.log.pattern' : 'combined'}]"
```

NOTE: If you use an Apache TomEE container, the configuration is the same – just replace `tomcat` with **tomee** or **tomee7**.

4.1.3.5.1.10 Debugging Java Applications

Debugging an application helps you detect and diagnose errors in your code.

You can control the execution of your program by setting breakpoints, suspending threads, stepping through the code, and examining the contents of the variables. See:

- Debug an Application Running on SapMachine [page 334]
- Debug a Community Java Buildpack Application [page 335]

4.1.3.5.1.10.1 Debug an Application Running on SapMachine

Debug a Java web application running on a Cloud Foundry container that uses SapMachine.

Prerequisites

- Download and Install the Cloud Foundry Command Line Interface [page 2813]
- Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814]

Context

To debug an application using SapMachine , you need to open a debugging port on your Cloud Foundry container, and then open an SSH tunnel to connect to that port.

Procedure

1. To open the debugging port, you need to configure the `JAVA_OPTS` parameter in your JVM. By default, the *agentlib* (which enables the debugging) is pre-configured. It could be turned off by setting the following property in the environment:

   ```
   env
   ...
     JBP_CONFIG_SAP_MACHINE_JDK: "[default_debug_agent_active: false]"
   ```

   You could also disable the default configuration by specifying it manually. Set the following option in the application `manifest.yml` file:

   ```
   JBP_CONFIG_JAVA_OPTS: "[java_opts: '-
   agentlib:jdwp=transport=dt_socket,address=8000,server=y,suspend=n']"
   ```

   You can change the default port **8000** in the command.
2. To enable SSH tunneling in your application, run:

   ```
   cf enable-ssh <app_name>
   ```
3. To open the SSH tunnel, run:

   ```
   cf ssh <app name> -N -T -L 8000:127.0.0.1:8000
   ```

   Your local port 8000 is connected to the debugging port 8000 of the JVM, which is running in the Cloud Foundry container.

   > ### Note
   >
   > The connection is active until you close the SSH tunnel. When you finish debugging, close the SSH tunnel by pressing `Ctrl` + `C`.
4. Connect a Java debugger to your application. For example, use the standard Java debugger provided by Eclipse IDE and connect to `localhost:8000`.

4.1.3.5.1.10.2 Debug a Community Java Buildpack Application

You can debug an application running on a Cloud Foundry container that uses the community Java buildpack.

Prerequisites

- Download and Install the Cloud Foundry Command Line Interface [page 2813]
- Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814]

Context

To debug an application using the community Java buildpack , you need to open a debugging port on your Cloud Foundry container and open an SSH tunnel that will connect to that port.

Procedure

1. To open the debugging port, you need to configure the `JAVA_OPTS` parameter in your JVM. From the cf CLI, run:

   ```
   cf set-env <app name> JAVA_OPTS '-
   Xrunjdwp:transport=dt_socket,server=y,suspend=n,address=8000'
   ```

2. To enable SSH tunneling for your application, run:

   ```
   cf enable-ssh <app name>
   ```

3. To activate the previous changes, restart the application by running:

   ```
   cf restart <app name>
   ```

4. To open the SSH tunnel, run:

   ```
   cf ssh <app name> -N -T -L 8000:127.0.0.1:8000
   ```

   Your local port 8000 is connected to the debugging port 8000 of the JVM running in the Cloud Foundry container. Also, 8000 is the default port.

   > ### Note
   >
   > The connection is active until you close the SSH tunnel. When you finish debugging, close the SSH tunnel by pressing `Ctrl` + `C`.

5. Connect a Java debugger to your application. For example, use the standard Java debugger provided by Eclipse IDE and connect to `localhost:8000`.

4.1.3.5.1.11 Profiling Java Applications

Application profiling helps you analyze the resource consumption of your Java applications running on a virtual machine (SapMachine).

Related Information

Profile an Application Running on SapMachine with JFR/JMC [page 337]
Profile an Application Running on SapMachine with async-profiler [page 339]

4.1.3.5.1.11.1 Profile an Application Running on SapMachine with JFR/JMC

You can use Java Flight Recorder (JFR) to profile your Java application on SapMachine, and Java Mission Control (JMC) to do remote profiling and analysis.

Prerequisites

- You have downloaded and installed a JDK Mission Control console. To do that, go to SapMachine → *Download* → *JMC 9*, select your operating system, and then choose *Download*.
- You have installed the Cloud Foundry command line interface. See: Download and Install the cf CLI
- You are logged on to a SAP BTP, Cloud Foundry space. See: Log On to the Cloud Foundry Environment Using cf CLI
- The version of your buildpack is at least **1.110.0** (for SAP Java Buildpack 1) or **2.24.0** (for SAP Java Buildpack 2).
- The version of your SapMachine is at least **17.0.14** or **21.0.6**.
- You have a Java application that is up and running on SAP BTP, Cloud Foundry.
- You have enabled SSH for your application. See: Configuring SSH access at the app level

Context

The commands below use async-profiler , which is a lightweight command-line tool that uses low memory resources. You can use it to run *jcmd* commands without worrying about high memory consumption or potential Out-Of-Memory errors during profiling. The `async-profiler` tool is integrated in SapMachine, thus no need to download it.

To profile with JMC, you need to start the Management Agent on a RMI port and then open an SSH tunnel to connect to that port.

> **Note**
>
> In the steps below, we use **myapp** and **5555** as *exemplary* application name and port. Replace them with your actual app name and a free port number on your localhost.

Procedure

1. Set the RMI server to `localhost`. To do that, set the following JBP_CONFIG_JAVA_OPTS option in the `manifest.yml` file of your application:

```
---
applications:
- name: myapp
```

```
  buildpack: sap_java_buildpack_jakarta
...
  env:
    TARGET_RUNTIME: tomcat
    JBP_CONFIG_COMPONENTS: "jres:
['com.sap.xs.java.buildpack.jre.SAPMachineJRE']"
    JBP_CONFIG_SAP_MACHINE_JRE: '{version: 21.+}'
    JBP_CONFIG_JAVA_OPTS: "[java_opts: '-
Djava.rmi.server.hostname=127.0.0.1']"
```

2. Start the Management Agent by running `jcmd` in your Cloud Foundry container. Run one of the following commands:

   - ```
     cf ssh myapp -c "app/META-INF/.sap_java_buildpack/sap_machine_jre/bin/
     asprof jcmd $(pgrep java) ManagementAgent.start
     jmxremote.authenticate=false jmxremote.ssl=false jmxremote.port=5555
     jmxremote.rmi.port=5555"
     ```
   - ```
     cf ssh myapp -c "app/META-INF/.sap_java_buildpack/sap_machine_jre/bin/
     asprof jcmd 7 ManagementAgent.start jmxremote.authenticate=false
     jmxremote.ssl=false jmxremote.port=5555 jmxremote.rmi.port=5555"
     ```

   Depending on what your SapMachine is using (JRE or JDK), specify the path accordingly (`sap_machine_jre` or `sap_machine_jdk`). To learn more, see: SapMachine [page 314]

   Result:

   ```
   Command executed successfully
   ```

   Management Agent is started and is listening to port **5555**.

3. (Optional) You can check the status of the Management Agent. Run:

   ```
   cf ssh myapp -c "app/META-INF/.sap_java_buildpack/sap_machine_jre/bin/asprof
   jcmd $(pgrep java) ManagementAgent.status"
   ```

4. Enable an SSH tunnel for this port. Run:

   ```
   cf ssh myapp -N -T -L 5555:127.0.0.1:5555
   ```

   Your *local* port 5555 is connected to the *remote* port 5555 of the JVM, which is running in the Cloud Foundry container.

   > ### Note
   >
   > The connection is active until you close the SSH tunnel. When you finish profiling, close the SSH tunnel by pressing `Ctrl` + `C`.

5. Connect with JMC to `localhost:5555`. To do that:
   1. Open your JDK Mission Control (JMC) console.
   2. From, the *JVM Browser* tab, choose icon *Create a new custom JVM connection*.
   3. For *Port*, enter **5555** (or your actual port number), and then choose *Finish*.
6. In the left-side menu, a new node *localhost:5555* appears.
7. Double-click on *MBean Server*.
8. To start profiling, go to *Flight Recorder* and from its context menu, choose *Start Flight Recording*.
9. Set up the event settings. From the drop-down menu, choose one of the following options (templates):
   - *Continuous* – Low overhead configuration, safe for continuous use in production environments

- *Profiling* – Low overhead configuration for profiling
- *gc* – GC related events. Generates a small recording size.
- *gc_details* – GC related events. Gets heap statistic and generates a large recording size.
- Create your own configuration. To do that, choose *Template Manager*, then select a predefined template and choose *Duplicate*. Rename the new template and edit its configuration parameters according to your needs and preferences.

10. After profiling is done, go to *localhost:5555* and from its context menu, choose *Disconnect*.
11. Then go back to your command line and close the SSH tunnel by pressing `Ctrl` + `C`.
12. Stop the Management Agent. Run:

```
cf ssh myapp -c "app/META-INF/.sap_java_buildpack/sap_machine_jre/bin/asprof
jcmd $(pgrep java) ManagementAgent.stop"
```

4.1.3.5.1.11.2 Profile an Application Running on SapMachine with async-profiler

You can use the async-profiler tool to profile your Java application on SapMachine.

Prerequisites

- You have installed the Cloud Foundry command line interface. See: Download and Install the cf CLI
- You are logged on to a SAP BTP, Cloud Foundry space. See: Log On to the Cloud Foundry Environment Using cf CLI
- The version of your buildpack is at least **1.110.0** (for SAP Java Buildpack 1) or **2.24.0** (for SAP Java Buildpack 2).
- The version of your SapMachine is at least **17.0.14** or **21.0.6**.
- You have a Java application that is up and running on SAP BTP, Cloud Foundry.
- You have enabled SSH for your application. See: Configuring SSH access at the app level

Context

To profile Java applications with the `async-profiler` tool, you can use the cf CLI Java plugin [page 2831], which provides convenience commands for profiling.

> **Note**
>
> In the steps below, we use **myapp** as an *exemplary* application name. Replace it with your actual application name.

Procedure

1. Start the CPU profiling. Run:

   ```
   cf java asprof-start-cpu myapp
   ```

   Result: `Profiling started`
2. (Optional) You can check the status of the profiling. Run:

   ```
   cf java asprof-status myapp
   ```

   Result: `Profiling is running for xx seconds`
3. Stop profiling and download the profiling data to the given local directory. Run:

   ```
   cf java asprof-stop myapp
   ```

   Result:

   ```
   --- Execution profile ---
   Total samples       : 15
   ...
   Successfully created JFR recording in application container at: /tmp/myapp-
   <GUID>.jfr
   Jfr recording file saved to: ./myapp-asprof-<GUID>.jfr
   Jfr recording file deleted in app container
   ```
4. Check the additional convenience commands. Run:

   ```
   cf java
   ```
5. To profile other events, you can start the *async-profiler* by running the `start-asprof` command with arguments:

   ```
   cf java asprof-start --args "-e nativemem" myapp
   ```

Related Information

Profiling Modes
Profiler Options
Cloud Foundry Command Line Java plugin

4.1.3.5.1.12 Configuring a Database Connection

You can configure your application to use a database connection so that the application can persist its data. This configuration is applicable for the TomEE 10 [page 290], Tomcat 10 [page 295], and Tomcat 11 [page 298] application containers. See:

- Database Connection Configuration Details [page 341]

- Connect an Application to PostgreSQL via TLS Certificate [page 344]
- Configure a Database Connection for the TomEE Application Container [page 346]
- Configure a Database Connection for the Tomcat Application Container [page 348]
- SAP HANA HDI Data Source Usage [page 349]

4.1.3.5.1.12.1 Database Connection Configuration Details

Define details of the database connection used by your Java Web Application running on SAP BTP, Cloud Foundry with SAP Java Buildpack.

Configuration Files

To configure your application so as to establish a connection to the SAP HANA database, you must specify details of the connection in a configuration file. For example, you must define the data source that the application will use to discover and look up data. The application then uses a Java Naming and Directory Interface (JNDI) to look up the specified data in the file.

The easiest way to define the required data source is to declare the keys for the data source in a resource file.

- For Tomcat application containers, you can create a `context.xml` file in the `META-INF/` directory with the following content:

  Sample Code

  context.xml

  ```
  <?xml version='1.0' encoding='utf-8'?>
  <Context>
   <Resource name="jdbc/DefaultDB"
      auth="Container"
      type="javax.sql.DataSource"
      factory="com.sap.xs.jdbc.datasource.tomcat.TomcatDataSourceFactory"
      service="di-core-hdi"/>
  </Context>
  ```

- For TomEE application containers, you can create a `resources.xml` file in the `WEB-INF/` directory with the following content:

  Sample Code

  resources.xml

  ```
  <?xml version='1.0' encoding='utf-8'?>
  <resources>
   <Resource id="jdbc/DefaultDB" provider="xs.openejb:XS Default JDBC
  Database" type="javax.sql.DataSource" >
      service=di-core-hdi
   </Resource>
  </resources>
  ```

NOTE: This simple approach is not always applicable as your WAR file cannot be signed, and any modifications can only be made in the WAR file.

For production environments, we recommend that you take advantage of the Resource Configuration [page 323] feature of SAP Java Buildpack. Use modification settings in files `resource_configuration.yml` and `manifest.yml` as illustrated in the following examples.

Specify a default service name in `resource_configuration.yml`:

Sample Code

resource_configuration.yml

```
---
tomcat/webapps/ROOT/META-INF/context.xml:
  service_name_for_DefaultDB: di-core-hdi
```

NOTE: Specifying a default service name (for example, for automation purposes) is only useful if you're sure there will be no conflicts with other names. For this reason, we recommend that you include a helpful, descriptive error message instead of a default value. This way, the error message will be part of an exception triggered when the data source is initialized, which helps troubleshooting. For example:

Sample Code

resource_configuration.yml

```
---
tomcat/webapps/ROOT/META-INF/context.xml:
  service_name_for_DefaultDB: Specify the service name for Default DB in
manifest.yml via "JBP_CONFIG_RESOURCE_CONFIGURATION".
```

Placeholders

Using the JBP_CONFIG_RESOURCE_CONFIGURATION parameter is a generic mechanism that basically replaces the key values in the specified files. For this reason, if you use placeholders in the configuration files, it is important to ensure that you use unique names for the placeholders (see Resource Configuration [page 323]). For example:

Sample Code

context.xml

```
<?xml version='1.0' encoding='utf-8'?>
<Context>
 <Resource name="jdbc/DefaultDB"
  auth="Container"
  type="javax.sql.DataSource"
  description="Datasource for general functionality"
  factory="com.sap.xs.jdbc.datasource.tomcat.TomcatDataSourceFactory"
  service="${service_name_for_DefaultDB}"
  maxActive="${max_Active_Connections_For_DefaultDB}"  />
 <Resource name="jdbc/DefaultXADB"
  auth="Container"
  type="javax.sql.XADataSource"
  description="Datasource for functionality requiring more than one
transactional resource"
```

```
  factory="com.sap.xs.jdbc.datasource.tomcat.TomcatDataSourceFactory"
  service="${service_name_for_DefaultXADB}"
  maxActive="${max_Active_Connections_For_DefaultXADB}" />
</Context>
```

NOTE: This sample code uses just an exemplary set of attributes. You can use as many as you want, depending on your scenario and database. To check the full list, see Tomcat: Common Attributes

The names of the defined placeholders are also used in the other resource files:

Sample Code

resource_configuration.yml

```
---
tomcat/webapps/ROOT/META-INF/context.xml:
  service_name_for_DefaultDB: di-core-hdi
  max_Active_Connections_For_DefaultDB: 100
  service_name_for_DefaultDB: di-core-hdi-xa
  max_Active_Connections_For_DefaultXADB: 100

env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/webapps/ROOT/META-
INF/context.xml': {'service_name_for_DefaultDB' : 'my-local-di-
core-hdi' , 'max_Active_Connections_For_DefaultDB' : '30',
'service_name_for_DefaultXADB' : 'my-local-xa-di-core-hdi' ,
'max_Active_Connections_For_DefaultXADB' : '20'  }]"
```

Sample Code

manifest.yml

```
env:
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/webapps/ROOT/META-
INF/context.xml': {'service_name_for_DefaultDB' : 'my-local-di-
core-hdi' , 'max_Active_Connections_For_DefaultDB' : '30',
'service_name_for_DefaultXADB' : 'my-local-xa-di-core-hdi' ,
'max_Active_Connections_For_DefaultXADB' : '20'  }]"
```

Related Information

Configure a Database Connection for the TomEE Application Container [page 346]
Configure a Database Connection for the Tomcat Application Container [page 348]
Resource Configuration [page 323]

4.1.3.5.1.12.2 Connect an Application to PostgreSQL via TLS Certificate

Prerequisites

You have a Java application created and deployed in an enterprise (productive) subaccount, in the SAP BTP, Cloud Foundry environment.

> **Note**
>
> This scenario is not applicable for SAP BTP trial accounts.

Context

Use TLS certificates to provide extra security for your Java applications when connecting them to a PostgreSQL database.

The certificate names described below start with "ssl", which is correct as Transport Layer Security (TLS) is the advanced security successor of Secure Sockets Layer (SSL).

Procedure

1. Create a service instance of *PostgreSQL, Hyperscaler Option* with plan `development`. For example, let's create a service instance with name **mydb**. Run:

   ```
   cf create-service postgresql-db development mydb
   ```

2. Bind the new service instance to your application. To do that, add **mydb** in the `manifest.yml` file of your application:

   ```
   ---
   applications:
   - name: myapp
     memory: 512M
     buildpacks:
     - sap_java_buildpack_jakarta
     services:
     - mydb
   ...
     env:
       TARGET_RUNTIME: tomcat
   ```

3. Add environment parameter `USE_CERTS_WITH_POSTGRES` in the `manifest.yml` file. For example, set it to **verify-ca**:

   ```
   ---
   ```

```
applications:
- name: myapp
  memory: 512M
  buildpacks:
  - sap_java_buildpack_jakarta
  services:
  - mydb
...
  env:
    USE_CERTS_WITH_POSTGRES: verify-ca
    TARGET_RUNTIME: tomcat
```

To learn about all possible values, see PostgreSQL: Protection Provided in Different Modes .

4. Before deploying your Java application, build your project again. Run:

```
mvn clean install
```

5. Deploy your application. Run:

```
cf push myapp
```

6. Check the contents of VCAP_SERVICES. Run:

```
cf env myapp
```

From the output, go to the "`postgresql-db`" part of the VCAP_SERVICES section and check which of the following SSL certificates are available:

- `sslcert`
- `sslkey`
- `sslrootcert`

Example Output:

```
<path>\java-tutorial>cf env myapp
...
System-Provided:
VCAP_SERVICES: {
  "postgresql-db": [
    {
      "binding_guid": "123456789-aaaa-1111-cccc-987654321",
      "binding_name": null,
      "credentials": {
        "dbname": "...",
        "hostname": "...",
        "password": "...",
        "port": "1234",
        "sslcert": "-----BEGIN CERTIFICATE-----\nMIID/zCCAuegAwIBAg...
\...UbBiKPA7lVVhre\n-----END CERTIFICATE-----",
        "sslrootcert": "-----BEGIN CERTIFICATE-----\nMIID/zCCAuegAwIBAg...
\...FDQnKF5OxwbAgip42CD
```

In this example, certificates `sslcert` and `sslrootcert` are displayed.

7. Check the contents of directory `./postgresql`. To do that:
   1. Run: `cf ssh myapp`
   2. Display a detailed directory listing. Run: `ll`
   3. List the contents of the `.postgresql` directory. Run: `cd .postgresql`
8. Check which of the corresponding certificate names are listed in the `.postgresql` directory:
   - `postgresql.crt`

- `postgresql.key`
- `root.crt`

> ⚠ Restriction
>
> If you're using GCP, all these certificates are installed by default.
>
> If you're using a different IaaS provider (for example, AWS), only the `root.crt` certificate will be available.

Related Information

Using Secure Socket Layer (SSL) Encryption for PostgreSQL DB Instances

4.1.3.5.1.12.3 Configure a Database Connection for the TomEE Application Container

Procedure

1. Create a `resources.xml` file.
   - If the data source is to be used from a Web application, you have to create the file inside the `WEB-INF/` directory.
   - If the data source is to be used from Enterprise JavaBeans (EJBs), you have to create the file inside the `META-INF/` directory.

   The `resources.xml` file has to be inside the application's WAR file, and has to contain information about the data source to be used.

   > {·} Sample Code
   >
   > *resources.xml*

   ```
   <?xml version='1.0' encoding='utf-8'?>
   <resources>
    <Resource id="jdbc/DefaultDB" provider="xs.openejb:XS Default JDBC
   Database" type="javax.sql.DataSource" >
       service=${service_name_for_DefaultDB}
     </Resource>
   </resources>
   ```

2. Add the default keys and their values. You need to include the data source information in directory `META-INF/sap_java_buildpack/config/resource_configuration.yml` of the WAR file.
   - Define a default service name for a Web application:

Sample Code

resource_configuration.yml

```
---
tomee/webapps/ROOT/WEB-INF/resources.xml:
  service_name_for_DefaultDB: di-core-hdi
```

- Define a default service name for an EJB:

Sample Code

resource_configuration.yml

```
---
tomee/webapps/ROOT/META-INF/resources.xml:
  service_name_for_DefaultDB: di-core-hdi
```

3. Add the key values to be updated. You need to include the data source information to be updated in the `manifest.yml` file.
   - Define the new service name for the look-up of the data source in a Web application:

Sample Code

manifest.yml

```
env:
  TARGET_RUNTIME: tomee
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/webapps/ROOT/WEB-INF/
resources.xml': {'service_name_for_DefaultDB' : 'my-local-di-core-
hdi'}]"
```

   - Define the new service name for the look-up of the data source in an EJB:

Sample Code

manifest.yml

```
env:
  TARGET_RUNTIME: tomee
  JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/webapps/ROOT/META-INF/
resources.xml':{'service_name_for_DefaultDB' : 'my-local-di-core-hdi'}]"
```

Results

As a result of this configuration, when the application starts, the *com.sap.xs.jdbc.datasource.TomEEDataSourceFactory* factory takes the parameters bound to the **my-local-di-core-hdi** service from the environment, creates a data source, and binds it under `jdbc/DefaultDB`. The application then uses the Java Naming and Directory Interface (JNDI) to look up how to connect with the database.

Related Information

SAP HANA HDI Data Source Usage [page 349]
Database Connection Configuration Details [page 341]

4.1.3.5.1.12.4 Configure a Database Connection for the Tomcat Application Container

Procedure

1. Create a `context.xml` file.

   This file has to be inside the `META_INF/` folder of the application WAR file, and has to contain information about the data source to be used.

   Sample Code

   context.xml

   ```
   <?xml version='1.0' encoding='utf-8'?>
   <Context>
    <Resource name="jdbc/DefaultDB"
       auth="Container"
       type="javax.sql.DataSource"
       factory="com.sap.xs.jdbc.datasource.tomcat.TomcatDataSourceFactory"
       service="${service_name_for_DefaultDB}"/>
   </Context>
   ```

2. Add the default keys and their values. You need to include the data source information in directory `META-INF/sap_java_buildpack/config/resource_configuration.yml` of the WAR file.

   Define a default service name:

   Sample Code

   resource_configuration.yml

   ```
   ---
   tomcat/webapps/ROOT/META-INF/context.xml:
     service_name_for_DefaultDB: di-core-hdi
   ```

3. Add the key values to be updated. You need to include the data source information to be updated in the `manifest.yml` file.

   Define the new service name for the look-up of the data source:

   Sample Code

   manifest.yml

   ```
   env:
     JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/webapps/ROOT/META-INF/
   context.xml': {'service_name_for_DefaultDB' : 'my-local-core-hdi'}]"
   ```

Results

When the application starts, the factory named `com.sap.xs.jdbc.datasource.TomcatDataSourceFactory` takes the parameters bound for service **my-local-core-hdi** from the environment, creates a data source, and binds it under `jdbc/DefaultDB`. The application then uses the Java Naming and Directory Interface (JNDI) to look up how to connect with the database.

Related Information

Database Connection Configuration Details [page 341]

4.1.3.5.1.12.5 SAP HANA HDI Data Source Usage

If you want your Java application to consume SAP HANA Database (for example, to use an HDI container), follow the steps below.

Procedure

1. Create a service instance for the HDI container, using the `cf create-service` command.

   ```
   cf create-service hana hdi-shared my-hdi-container
   ```

2. Bind the service instance to a Java application.

   Specify the service instance in the Java application's deployment *manifest.yml* file.

   Sample Code

   ```
   services:
   - my-hdi-container
   ```

3. Add a resource reference XML block. Its technical name must be the same as the name of the service instance.

   Sample Code

   ```
   <resource-ref>
     <res-ref-name>jdbc/my-hdi-container</res-ref-name>
     <res-type>javax.sql.DataSource</res-type>
   </resource-ref>
   ```

   Add this resource reference to the following files, respectively:

   - For Tomcat: `web.xml` and `context.xml`

- For TomEE: `web.xml` and `resource.xml`

4. Look up the data source in your code.

   You can find the reference to the data source in two ways – by using annotations or with JNDI look-up. Add the relevant code to your Java servlet:

   - Annotations

     ```
     @Resource(name = "jdbc/my-hdi-container")
     private DataSource ds;
     ```

   - Java Naming and Directory Interface (JNDI) look-up

     ```
     Context ctx = new InitialContext();
     return (DataSource) ctx.lookup("java:comp/env/jdbc/my-hdi-container");
     ```

Related Information

SAP HDI Containers
Set Up a Schema or an HDI Container (Cloud Foundry)

4.1.3.5.1.13 Dynatrace Integration

Dynatrace OneAgent is a Java agent that sends all captured monitoring data to your Dynatrace monitoring environment for analysis.

Context

Dynatrace supports full-stack monitoring for Cloud Foundry – from the application to the infrastructure layer. To integrate it with your Java application, follow the steps below.

Procedure

1. Obtain an environment ID and an API access token.
2. Create a user-provided service.

   The service name should contain the string "dynatrace" (for example, **dynatraceservice**). The `environmentid` and `apitoken` parameters have been generated in the previous step. You also need to set up the `apiurl`, which depends on the Dynatrace type you want to integrate.

   - **SaaS Dynatrace**

     ```
     "apiurl": "https://<environmentid>.live.dynatrace.com/api"
     ```

- **Managed Dynatrace**

  ```
  "apiurl": "https://<your_domain>/e/<environmentid>/api"
  ```

Then, in cf CLI, run the following command:

```
cf cups dynatraceservice -p "environmentid, apitoken, apiurl"
```

For more information, see:

- Dynatrace Docs: Create a user-provided service in your SAP BTP, Cloud Foundry Environment
- Cloud Foundry Docs: User-Provided Service Instances

3. Bind the application to the service instance created in the previous step, by using the *manifest.yml* file.

   {·} Sample Code

   ```
   ---
   applications:
   - name: myapp
     memory: 1G
     instances: 1
     buildpack: sap_java_buildpack_jakarta
     services:
     - dynatraceservice
   ```

   NOTE: If there are more than one Dynatrace services bound to the application, the service that's going to be used is the one that was chronologically bound first.

4. Deploy the application to the SAP BTP, Cloud Foundry environment with the updated *manifest.yml* file. Run:

   ```
   cf push myapp
   ```

   Once the application receives Web traffic, monitoring data will show up in Dynatrace.

Related Information

Dynatrace Docs: Access tokens API
Dynatrace Docs: Cloud Foundry Monitoring

4.1.3.5.1.14 SAP Connectivity ApiExt

Use the SAP Connectivity ApiExt feature (provided by SAP Java Buildpack) to retrieve destination configurations or authentication headers ready to be used against the remote target system.

⚠ Restriction

SAP Connectivity ApiExt is only available for Tomcat application containers included in SAP Java Buildpack.

Also, Spring Boot works with SAP Connectivity ApiExt only when using WAR deployment.

> ### Note
>
> SAP Connectivity ApiExt is **not** the recommended way for consuming the Destination service as it only covers a subset of the service's APIs. It's primarily meant to be used internally by SAP Java Connector (SAP JCo) and for Neo migration scenarios, where there are limited possibilities to modify the code.
>
> The **recommended** consumption mechanism for Java applications is SAP Cloud SDK for Java or using the Destination service REST API.

Prerequisites

To use SAP Connectivity ApiExt during runtime, you first need to create:

- A service instance for the Destination service
- A destination entity in the Destination service that you want to retrieve from the application. See: Managing Destinations
- (Optional) A service instance for the Connectivity service – only needed if you plan to use it for destinations of type `ProxyType`=**OnPremise**

Activate the API

1. Open the `pom.xml` file of your Tomcat-based Java application and add the following dependency:

```
<dependency>
    <groupId>com.sap.cloud.connectivity.apiext</groupId>
    <artifactId>com.sap.cloud.connectivity.apiext</artifactId>
    <version>${connectivity-apiext.version}</version>
    <scope>provided</scope>
</dependency>
```

2. Now open the `manifest.yml` file and set environment variable USE_CONNECTIVITY_APIEXT to **true**:

```
---
applications:
- name: <APP_NAME>
  ...
  env:
    TARGET_RUNTIME: tomcat
    USE_CONNECTIVITY_APIEXT: true
```

3. Bind the previously created service instances to your application. To do that, add their names at the end of the `manifest.yml` file. For example:

```
---
applications:
- name: <APP_NAME>
  ...
  env:
    TARGET_RUNTIME: tomcat
    USE_CONNECTIVITY_APIEXT: true
  services:
    - my_destination
    - my_connectivity
```

```
        ...
```

For more information, see ConnectivityConfiguration API: Procedure

Related Information

Create Destinations from Scratch
SAP Java Connector [page 353]

4.1.3.5.1.15 SAP Java Connector

You can use the SAP Java Connector (SAP JCo) through SAP Java Buildpack.

Activation

SAP Java Buildpack provides an option to use SAP Java Connector (SAP JCo).

To activate SAP JCo in the buildpack, set the environment variable `USE_JCO` to *true*.

Sample Code

manifest.yml

```
---
applications:
- name: <app_name>
  ...
  env:
    USE_JCO: true
```

The previous method of activating the SAP JCo feature, by defining a service instance for both the connectivity and destination services in the `manifest.yml` file, has been deprecated and stops working at the end of a transition phase.

The activation of SAP JCo provides all relevant libraries in the application container, so that SAP JCo API can be used. To use SAP JCo during runtime, a destination and an SAP Authorization and Trust Management (XSUAA) service instance are mandatory. Additionally, if you want to set up on-premise connectivity, a connectivity service instance is required.

Limitations

- SAP JCo is only available for the Tomcat-based application containers that are included in SAP Java Buildpack.

- Spring Boot applications work with SAP JCo only if you're using WAR deployment.

Related Information

Invoking ABAP Function Modules via RFC
Invoke ABAP Function Modules in On-Premise ABAP Systems

4.1.3.5.1.16 Adding Security Providers

You can add custom security providers to the `java.security` file at deploy time.

The SECURITY_PROVIDERS mechanism supports adding one or more providers, with an explicit priority for each of them. Provider JARs can be supplied in two ways:

- As standard project dependencies
- Bundled inside the application

Prerequisites

You have created and deployed a Jakarta-based application.

Procedure

1. Open your `manifest.yml` file.
2. Add the SECURITY_PROVIDERS entries by using the "`<index>|<value>`" format. You can add single or multiple ones, like this:
   - Single provider: `SECURITY_PROVIDERS: "1|org.security.ProviderA"`
   - Multiple providers: `SECURITY_PROVIDERS: "1|org.security.ProviderA 2| org.security.ProviderB"`

Example

```
---
applications:
- name: <APP_NAME>
  memory: 512M
  buildpacks:
  - sap_java_buildpack_jakarta
  ...
  env:
    TARGET_RUNTIME: tomcat
    SECURITY_PROVIDERS: "1|org.security.ProviderA 2|org.security.ProviderB"
```

3. Add the provider JARs. You can do this in two ways:
   - Provided as normal application dependencies (for example, via the `pom.xml` file). SAP Java Buildpack will pick up those JARs from the application classpath.

     ```
     <dependencies>
       <dependency>
         <groupId>org.providerA</groupId>
         <artifactId>providerA</artifactId>
         <version>${providerA.version}</version>
       </dependency>
       <dependency>
         <groupId>org.providerB</groupId>
         <artifactId>providerB</artifactId>
         <version>${providerB.version}</version>
       </dependency>
     </dependencies>
     ```

   - Bundled in the app resources. To do that, place the provider JARs under directory `webapp/META-INF/.sap_java_buildpack/custom_security_providers`, like this:

     ```
     ├── webap
     └──── META-INF
     │     └──── .sap_java_buildpack
     │           └──── custom_security_providers
     │                 └──── providerA.jar
     │                 └──── providerB.jar
     └──── WEB-INF
           └──── ...
     ```

4. Deploy your application. To do that, run:

   ```
   cf push <APP_NAME>
   ```

Result

The mechanism parses the SECURITY_PROVIDERS value from the `manifest.yml` file and adds the listed providers to the `java.security` file in the specified priority order. It supports multiple providers and avoids duplicate entries due to unique provider class names.

4.1.3.5.1.17 Application-Provided Library Components

SAP Java Buildpack can automatically provide JDBC drivers for SAP HANA and PostgreSQL in the classpath if the respective service instance is bound.

The application can also provide its own version of these drivers in the relevant directory:

- `WEB-INF/lib` – for standard Java applications. It's loaded for all Java buildpacks
- `BOOT-INF/lib` – for Spring Boot applications. It's only loaded for SAP Java Buildpack 2.

In this case, the application-provided version of the driver will be set in the classpath, not the one that comes with SAP Java Buildpack.

> ### Note
>
> When using this mechanism, the application developer is responsible to keep the JDBC drivers up-to-date. Also, note that SAP Java Buildpack will move the JDBC driver from its original location (`WEB-INF/lib` or `BOOT-INF/lib`) to one of the following:
>
> - `META-INF/.sap_java_buildpack/hana_jdbc`
> - `META-INF/.sap_java_buildpack/postgresql_jdbc`
>
> If you want to keep the driver in its original location, set up environment variable JBP_SKIP_PROVIDED_LIBRARY_COMPONENTS. To learn how, see the section below.

The drivers have to follow the respective file name pattern:

- **ngdbc*.jar** – for SAP HANA Driver
- **postgresql*.jar** – for PostgreSQL JDBC Driver

JBP_SKIP_PROVIDED_LIBRARY_COMPONENTS

If you use environment variable `JBP_SKIP_PROVIDED_LIBRARY_COMPONENTS` and set it to **true**, SAP Java Buildpack will keep the library components in their original location (`WEB-INF/lib` or `BOOT-INF/lib`).

This way, the new location (`META-INF/.sap_java_buildpack/hana_jdbc` or `META-INF/.sap_java_buildpack/postgresql_jdbc`) will not be created by the buildpack, nor will be added to the classpath.

Auxiliary Library Components

The application developer can provide additional components alongside the aforementioned libraries. For example, this can be a custom SSL factory for the PostgreSQL JDBC Driver, which has to be accessible to the class loader that loads the actual library.

To do so, the additional libraries have to be placed in a JAR file in the `WEB-INF/lib` or `BOOT-INF/lib` directory. The JAR file has to follow the respective file name pattern:

- **sjb-ngdbc-aux*.jar** – for SAP HANA Driver
- **sjb-postgresql-aux*.jar** – for PostgreSQL JDBC Driver

Any number of files following these patterns can be placed in the `WEB-INF/lib` or `BOOT-INF/lib` directory and will be handled by SAP Java Buildpack.

> ### → Tip
>
> This mechanism works with both application-provided and buildpack-provided libraries.

4.1.3.5.1.18 Properties, Parameters, and Variables

On this page you can find the properties, parameters and environment variables to use in SAP Java Buildpack. You can filter by name, value, a word, or part of a word.

Properties & Parameters

Name	Description	Values	Example

`connector.maxHttpHeaderSize`	The maximum size of the request and response HTTP header, specified in bytes	Default value: **8192**	To configure the maximum size of the HTTP header, use:
			`env:`
			`JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/conf/server.xml': {'connector.maxHttpHeaderSize':1024}]"`

`connector.maxPartHeaderSize`	The maximum number of header bytes permitted per part in a request where the content type is `multipart/form-data`	Default value: **512**	To configure the maximum size of a header part, use:
			`env:`
			`JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/conf/server.xml': {'connector.maxPartHeaderSize':256}]"`

Name	Description	Values	Example

`connector.maxPartCount`	The maximum total number of parts permitted in a request where the content type is `multipart/form-data`	Default value: **50**	To configure the maximum number of parts in a request, use:
			`env:`
			`JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcee/conf/server.xml': {'connector.maxPartCount':40}]"`

`connector.maxPostSize`	The maximum size (in bytes) of the POST request, which will be handled by the container FORM URL parameter parsing. The limit can be disabled by setting the attribute's value to any integer number less than zero (0).
	Note: You can use the Failed Request Filter to reject requests that exceed the default limit.
		Default value:
		2097152 (2 MiB)
			To configure the maximum size of the POST request, use:
			`env:`
			`JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/conf/server.xml': {'connector.maxPostSize':800000}]"`

`connector.maxThreads`	The maximum number of request-processing threads to be created by this Connector. It therefore determines the maximum number of simultaneous requests that can be handled.	Default value: **200**	To configure the maximum number of request-processing threads, use:
			`env:`
			`JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/conf/server.xml': {'connector.maxThreads':800}]"`

Name	Description	Values	Example

`connector.encodedSolidusHandling`	A string value that configures request paths containing a `%2f` sequence.	Default value: **reject**
		• When set to **reject**, these request paths will be rejected with a 400 response.
		• When set to **decode**, these request paths will have that sequence decoded to `/` at the same time other `%nn` sequences are decoded.
		• When set to **pass-through**, these request paths will be processed with the `%2f` sequence unchanged.
			To decode the `%2f` sequence of the request paths to `/`, use:
			`env:`
			`JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/conf/server.xml': {'connector.encodedSolidusHandling':'decode'}]"`

`connector.allowTrace`	A Boolean value that enables or disables the TRACE HTTP method	Default value: **false**	To enable the TRACE HTTP method, use:
			`env:`
			`JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/conf/server.xml': {'connector.allowTrace':true}]"`

Environment Variables

Name	Description	Values	Example

VCAP_SERVICES	When you bind a service instance to your app and then restart the app, Cloud Foundry adds connection details to the VCAP_SERVICES environment variable.
	
	See: VCAP_SERVICES
		See column **Example**	`$ cf env my-app`
			`Getting env variables for app my-app in org my-org / space my-space as admin...`
			`OK`
			`System-Provided:`
			`{`
			
			`"VCAP_APPLICATION": {`
			
			`"application_id": "abcd-1234-0000-aaaa789-eeeeeeeeee",`
			
			`"application_name": "my-app",`
			
			`"application_uris": [`
			`"my-app.example.com"`
			`],`
			
			`"application_version": "xyzxyz-3333-1234567890-aaaaaaaaaa",`
			`"cf_api": "https://api.example.com",`
			`"limits": {`
			`"disk": 1024,`
			`"memory": 256`
			`},`
			`"name": "my-app",`
			
			`"organization_id": "...",`
			
			`"organization_name": "my-org",`
			`"space_id": "...",`
			`"space_name": "my-space",`
			`"uris": [`
			`"my-app.example.com"`
			`]`
			`}`
			`}`

Name	Description	Values	Example

VCAP_SERVICES_FILE_PATH	It sets the path to the file containing the service binding information in JSON format. It must be enabled by an application feature flag.
	
	See: VCAP_SERVICES_FILE_PATH (experimental)
		See column **Example**	• Set the variable:
			`VCAP_SERVICES_FILE_PATH=/etc/cf-service-bindings/vcap_services`
			• To view the binding data, run:
			`cf ssh <app_name> -c 'cat $VCAP_SERVICES_FILE_PATH'`

JBP_CONFIG_RESOURCE_CONFIGURATION		See column **Example**	`env:`
			
			`JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomcat/conf/server.xml': {'connector.maxHttpHeaderSize':1024}]"`

JBP_CONFIG_COMPONENTS		See column **Example**	`env:`
			
			`JBP_CONFIG_COMPONENTS: "jres: ['com.sap.xs.java.buildpack.jre.SAPMachineJRE']"`

JBP_CONFIG_SAP_MACHINE_JRE		See column **Example**	`env:`
			
			`JBP_CONFIG_COMPONENTS: "jres: ['com.sap.xs.java.buildpack.jre.SAPMachineJRE']"`
			
			`JBP_CONFIG_SAP_MACHINE_JRE: "[version: 11.+, memory_calculator_v2: {stack_threads: 266, class_count: 1001, headroom: 5}]"`

Name	Description	Values	Example

JBP_CONFIG_SAP_MA-CHINE_JDK		See column **Example**	`env:`
			
			`JBP_CONFIG_COMPONENTS: "jres: ['com.sap.xs.java.buildpack.jdk.SAPMachineJDK']"`
			
			`JBP_CONFIG_SAP_MACHINE_JDK: "[ version: 11.0.22, memory_calculator_v2: {stack_threads: 266, class_count: 1001, headroom: 5}]"`

JBP_CONFIG_JAVA_OPTS		See column **Example**	`env:`
			
			`JBP_CONFIG_JAVA_OPTS: "[java_opts: '-Xms144M -Xss3M -Xmx444444K -XX:MetaspaceSize=66666K -XX:MaxMetaspaceSize=88888K']"`

JBP_CONFIG_SAPJVM		See column **Example**	`env:`
			
			`JBP_CONFIG_SAPJVM: "[memory_calculator_v2: {stack_threads: 266, class_count: 1001, headroom: 5}]"`

Name	Description	Values	Example

MALLOC_ARENA_MAX	It's used by the *glibc* memory allocator (in the default `malloc()` function in Linux). It controls the maximum number of memory arenas (heaps) that a process can create.
	
	An arena is a private memory pool used by threads to reduce lock contention when many threads allocate memory at the same time.
		Default value: **2**
		
		This is also the *recommended* value for SAP Java Buildpack.
			`env:`
			
			`MALLOC_ARENA_MAX: 4`

SET_LOGGING_LEVEL	It sets the log level of a logging location. You can do this before or during application deploy. You can set this variable in three ways:
	
	• In the `manifest.yml` file.
	• In a `logback.xml` file in directory `src/main/resources`.
	• In an `application.properties` file in directory `/src/main/resources/`.
	
	See: Logging and Tracing [page 329]
		Default log level: **INFO**	`env:`
			
			`SET_LOGGING_LEVEL: '{com.sap.sample.java.LogInfo: INFO, com.sap.sample.java.LogWarn: WARN}'`

Name	Description	Values	Example

XSUAA_SERVICE_PLAN	It specifies which service plan of the Authorization and Trust Management (`xsuaa`) service binding to use for token validation.
	
	Note
	XSUAA_SERVICE_PLAN is intended to be only used in a specific use case of token validation. Typically, you won't need or use this variable with your regular plans (for example, *application*).
	
	When this variable is set, the SAP Java Buildpack's authenticator searches through all `xsuaa` service bindings to find one with a matching `plan` property, and then uses this configuration for the token validation.
		See column **Example**	`env:`
			
			`XSUAA_SERVICE_PLAN: reference-instance`

4.1.3.5.1.19 Bill of Materials (BOM)

For Maven projects, the versions of SAP Java Buildpack dependencies and the APIs provided by supported runtime containers can be consumed through a Bill of Materials (BOM).

The BOM contains the dependencies you can rely on in a runtime environment provided by SAP Java Buildpack. It can also be used to control the versions of a project's dependencies without the need to manually maintain the version of each and every artifact.

→ Remember

Any dependency not included in the BOM must be explicitly declared and bundled by the application.

The BOMs of SAP Java Buildpack are provided through Maven Central .

Usage

There are two runtimes for which a BOM is provided: Tomcat and TomEE.

The version of the BOM must match the version of the `sap_java_buildpack` or `sap_java_buildpack_jakarta` you're using. For example, if your application uses `sap_java_buildpack_jakarta` version 2.24.0, the BOM must be in version 2.24.0 as well.

If you want to use version control, add the desired BOM artifact to your project's `pom.xml` file in the section `dependencyManagement`. The following codes are exemplary.

- For Tomcat 10:

```
...
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>com.sap.cloud.sjb.cf</groupId>
            <artifactId>cf-tomcat-bom</artifactId>
            <version>2.24.0</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
...
```

- For TomEE 10:

```
...
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>com.sap.cloud.sjb.cf</groupId>
            <artifactId>cf-tomee-bom</artifactId>
            <version>2.24.0</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
...
```

To add the dependencies you want, use *groupId* and *artifactId*:

```
...
<dependency>
    <groupId>com.sap.cloud.sjb</groupId>
    <artifactId>xs-env</artifactId>
</dependency>
<dependency>
    <groupId>com.sap.cloud.sjb</groupId>
    <artifactId>xs-user-holder</artifactId>
</dependency>
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-annotations</artifactId>
</dependency>
<dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>jul-to-slf4j</artifactId>
</dependency>
<dependency>
    <groupId>ch.qos.logback</groupId>
```

```
    <artifactId>logback-access</artifactId>
</dependency>
...
```

If you want to have all dependencies in your project, you can add the BOM to your project's section as well. By doing so, you don't need to enumerate single dependency artifacts.

Related Information

Maven Repository: SAP Java Buildpack BOM

4.1.3.5.1.20 SAP Java Buildpack: Libraries

On this page you can find changelogs about the latest updates made in the main five libraries of SAP Java Buildpack, provided on Maven Repository.

XS Env

Use this library to set up and access environment variables and services.

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-env

Technical name: `xs-env`

(It's compiled to Java 17 bytecode.)

Changelog

Release	Changes
`(01 October 2026)` `Version 2.34.0`	• No changes
`(20 August 2026)` `Version 2.33.0`	• *com.fasterxml.jackson.annotations* is updated from version 2.21 to 2.22 • *com.fasterxml.jackson.databind* is updated from version 2.21.2 to 2.22.1 • *com.fasterxml.jackson* is updated from version 2.21.2 to 2.22.1
`(6 August 2026)` `Version 2.32.0`	• No changes
`(27 May 2026)` `Version 2.31.0`	• Minor changes

Release	Changes
(14 May 2026) Version 2.30.0	• *com.fasterxml.jackson.core:jackson-core* is updated from version 2.21.0 to 2.21.2 • *com.faster.jackson.core:jackson-databind* is updated from version 2.21.0 to 2.21.2
(5 May 2026) Version 2.29.0	• *org.jacoco.agent* is added again and updated to version 0.8.14
(16 February 2026) Version 2.27.0	• *com.fasterxml.jackson.core:jackson-core* is updated from version 2.20.1 to 2.21.0 • *com.faster.jackson.core:jackson-databind* is updated from version 2.20.1 to 2.21.0 • *com.faster.core:jackson-annotations* is updated from version 2.20 to 2.21
(20 January 2026) Version 2.26.0	• *org.jacoco.agent* is removed, version 0.8.10 • *com.fasterxml.jackson.core:jackson-core* is updated from version 2.20.0 to 2.20.1 • *com.faster.jackson.core:jackson-databind* is updated from version 2.20.0 to 2.20.1

XS User Holder

Use this library to access user information during authentication and authorization when using SAP BTP Security Services Integration Libraries .

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-user-holder

Technical name: `xs-user-holder`

(It's compiled to Java 17 bytecode.)

Changelog

Release	Changes
(October 15 2026) Version 2.29.0	• No changes
(September 3 2026) Version 2.24.0 - 2.28.0	• No changes
(May 27 2026) Version 2.22.0 - 2.23.0	• No changes

Release	Changes
`(4 March 2026)` `Version 2.21.0`	• *org.junit.jupiter:junit-jupiter* is updated from version 6.0.2 to 6.0.3
`(23 January 2026)` `Version 2.20.0`	• *org.junit.jupiter:junit-jupiter* is updated from version 6.0.1 to 6.0.2

XS HDB Conn Options

Use this library to handle SAP HANA DB connection options.

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-hdb-conn-options

Technical name: `xs-hdb-conn-options`

(It's compiled to Java 17 bytecode.)

Changelog

Release	Changes
`(01 October 2026)` `Version 2.35.0`	• No changes
`(August 20 2026)` `Version 2.34.0`	• No changes
`(August 6 2026)` `Version 2.33.0`	• *com.sap.cloud.sjb.xs-env* is updated from version 2.31.0 to 2.32.0
`(27 May 2026)` `Version 2.32.0`	• *com.sap.cloud.sjb:xs-env* is updated from version 2.30.0 to 2.31.0
`(14 May 2026)` `Version 2.31.0`	• *com.sap.cloud.sjb:xs-env* is updated from version 2.29.0 to 2.30.0
`(5 May 2026)` `Version 2.30.0`	• *com.sap.cloud.sjb:xs-env* is updated from version 2.28.0 to 2.29.0
`(4 March 2026)` `Version 2.29.0`	• *com.sap.cloud.sjb:xs-env* is updated from version 2.27.0 to 2.28.0

Release	Changes
`(16 February 2026)` `Version 2.28.0`	• *com.sap.cloud.sjb:xs-env* is updated from version 2.26.0 to 2.27.0
`(20 January 2026)` `Version 2.27.0`	• *com.sap.cloud.sjb:xs-env* is updated from version 2.25.0 to 2.26.0
`(16 January 2026)` `Version 2.26.0`	• *com.sap.cloud.sjb:xs-env* is updated from version 2.24.0 to 2.25.0

XS Java Parent

Use this library to consume common dependencies and Maven plug-in configurations.

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-java-parent

Technical name: `xs-java-parent`

(It's compiled to Java 17 bytecode.)

Changelog

Release	Changes
`(August 20 2026)` `Version 2.11.0 - 2.12.0`	• No changes
`(27 May 2026)` `Version 2.10.0`	• No changes

XS JDBC Routing Datasource

This wrapper library handles datasource creation in single or multitenancy mode while working with JDBC.

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-jdbc-routing-datasource

Technical name: `xs-jdbc-routing-datasource`

(It's compiled to Java 17 bytecode.)

Changelog

Release	Changes
(October 29 2026) Version 2.28.0	• *com.sap.cloud.servicemanager* is updated from version 1.10.1 to 1.10.2
(October 15 2026) Version 2.27.0	• *com.sap.cloud.servicemanager* is updated from version 1.9.0 to 1.10.1 • *org.slf4j* is updated from version 2.0.19 to 2.0.20
(October 1 2026) Version 2.26.0	• *org.slf4j* is updated from version 2.0.18 to 2.0.19
(September 17 2026) Version 2.25.0	• *com.sap.cloud.servicemanager* is updated from version 1.8.0 to 1.9.0
(September 3 2026) Version 2.24.0	• *com.sap.cloud.servicemanager* is updated from version 1.6.0 to 1.8.0
(August 20 2026) Version 2.23.0	• *com.sap.cloud.servicemanager* is updated from version 1.1.0 to 1.6.0
(August 6 2026) Version 2.22.0	• No changes
(24 June 2026) Version 2.21.0	• *com.sap.cloud.instancemanager:client* is removed, version 3.14.0 • *com.sap.cloud.servicemanager:client* is added, version 1.1.0
(27 May 2026) Version 2.20.0	• *org.slf4j:slf4j-api* is updated from version 2.0.17 to 2.0.18
(8 May 2026) Version 2.17.0 - 2.19.0	• No changes

Related Information

Maven Repository: SAP Cloud SJB

SAP Java Buildpack: Libraries (2025) [page 371]
SAP Java Buildpack: Libraries (2024) [page 377]

4.1.3.5.1.20.1 SAP Java Buildpack: Libraries (2025)

On this page you can find changelogs about the 2025 updates in the main five libraries of SAP Java Buildpack 1 and 2, provided on Maven Repository.

XS Env

Use this library to set up and access environment variables and services.

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-env

Technical name: `xs-env`

> **Note**
>
> - In SAP Java Buildpack 1, `xs-env` is compiled to Java 8 bytecode.
> - In SAP Java Buildpack 2, `xs-env` is compiled to Java 17 bytecode.

Changelog

SAP Java Buildpack 1	SAP Java Buildpack 2

	`(15 October 2025)`
	`Version 2.24.0`
	• *org.jacoco.agent* is removed, version 0.8.10

	`(1 October 2025)`
	`Version 2.23.0`
	• *org.jacoco.agent* is added, version 0.8.10
	• *com.fasterxml.jackson.core:jackson-core* is updated from version 2.19.2 to 2.20.0
	• *com.faster.jackson.core:jackson-databind* is updated from version 2.19.2 to 2.20.0
	• *com.faster.jackson.core:jackson-annotations* is updated from version 2.19.2 to 2.20

	`(2 September 2025)`
	`Version 2.22.0`
	• *org.jacoco.agent* is added, version 0.8.10
	• *com.fasterxml.jackson.core:jackson-core* is updated from version 2.18.3 to 2.19.2
	• *com.faster.jackson.core:jackson-databind* is updated from version 2.18.3 to 2.19.2
	• *com.faster.jackson.core:jackson-annotations* is updated from version 2.18.3 to 2.19.2

SAP Java Buildpack 1	SAP Java Buildpack 2
(28 July 2025) Version 1.64.0 • Last available version • Minor changes	(19 August 2025) Version 2.21.0 • *org.jacoco.agent* is added, version 0.8.10
(23 June 2025) Version 1.63.0 • *com.fasterxml.jackson.core:jackson-core* is updated from version 2.19.0 to 2.19.1 • *com.faster.jackson.core:jackson-databind* is updated from version 2.19.0 to 2.19.1 • *com.faster.jackson.core:jackson-annotations* is updated from version 2.19.0 to 2.19.1	(23 May 2025) Version 2.18.0 • *com.fasterxml.jackson.core:jackson-core* is downported from version 2.19.0 to 2.18.3 • *com.faster.jackson.core:jackson-databind* is downported from version 2.19.0 to 2.18.3 • *com.faster.jackson.core:jackson-annotations* is downported from version 2.19.0 to 2.18.3
(9 June 2025) Version 1.62.0 • *com.fasterxml.jackson.core:jackson-core* is updated from version 2.18.2 to 2.19.0 • *com.faster.jackson.core:jackson-databind* is updated from version 2.18.2 to 2.19.0 • *com.faster.jackson.core:jackson-annotations* is updated from version 2.18.2 to 2.19.0	(12 May 2025) Version 2.17.0 • *com.fasterxml.jackson.core:jackson-core* is updated from version 2.18.2 to 2.19.0 • *com.faster.jackson.core:jackson-databind* is updated from version 2.18.2 to 2.19.0 • *com.faster.jackson.core:jackson-annotations* is updated from version 2.18.2 to 2.19.0
(25 March 2025) Version 1.61.0 • *com.fasterxml.jackson.core:jackson-core* is updated from version 2.18.2 to 2.18.3 • *com.faster.jackson.core:jackson-databind* is updated from version 2.18.2 to 2.18.3 • *com.faster.jackson.core:jackson-annotations* is updated from version 2.18.2 to 2.18.3	(17 March 2025) Version 2.16.0 • *com.fasterxml.jackson.core:jackson-core* is updated from version 2.18.2 to 2.18.3 • *com.faster.jackson.core:jackson-databind* is updated from version 2.18.2 to 2.18.3 • *com.faster.jackson.core:jackson-annotations* is updated from version 2.18.2 to 2.18.3
(7 January 2025) Version 1.59.0 • *com.fasterxml.jackson.core:jackson-core* is updated from version 2.18.0 to 2.18.2 • *com.faster.jackson.core:jackson-databind* is updated from version 2.18.0 to 2.18.2 • *com.faster.jackson.core:jackson-annotations* is updated from version 2.18.0 to 2.18.2	(9 December 2024) Version 2.12.0 • *com.fasterxml.jackson.core:jackson-core* is updated from version 2.18.0 to 2.18.2 • *com.faster.jackson.core:jackson-databind* is updated from version 2.18.0 to 2.18.2 • *com.faster.jackson.core:jackson-annotations* is updated from version 2.18.0 to 2.18.2

XS User Holder

Use this library to access user information during authentication and authorization when using SAP BTP Security Services Integration Libraries .

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-user-holder

Technical name: `xs-user-holder`

> **Note**
>
> - In SAP Java Buildpack 1, `xs-user-holder` is compiled to Java 8 bytecode.
> - In SAP Java Buildpack 2, `xs-user-holder` is compiled to Java 17 bytecode.

Changelog

SAP Java Buildpack 1	SAP Java Buildpack 2

	`(12 November 2025)`
	`Version 2.18.0`
	• *org.junit.jupiter:junit-jupiter* is updated from version 6.0.0 to 6.0.1

	`(15 October 2025)`
	`Version 2.17.0`
	• *org.junit.jupiter:junit-jupiter* is updated from version 5.13.4 to 6.0.0

	`(1 August 2025)`
	`Version 2.16.0`
	• *org.junit.jupiter:junit-jupiter* is updated from version 5.13.3 to 5.13.4

	`(28 July 2025)`
	`Version 2.15.0`
	• *org.junit.jupiter:junit-jupiter* is updated from version 5.13.1 to 5.13.3

	`(23 June 2025)`
	`Version 2.14.0`
	• *org.junit.jupiter:junit-jupiter* is updated from version 5.10.2 to 5.13.1

SAP Java Buildpack 1	SAP Java Buildpack 2

	`(6 June 2025)`
	`Version 2.13.0`
	• *junit:junit* is added, version 4.13.2
	• *org.junit.jupiter:junit-jupiter* is added, version 5.10.2
	• *org.mockito:mockito-core* is added, version 3.11.2

`(7 January 2025)`
`Version 1.57.0 – 1.63.0`
• Minor changes
• Integration tests
	`(7 January 2025)`
	`Version 2.6.0 – 2.12.0`
	• Minor changes
	• Integration tests

XS HDB Conn Options

Use this library to handle SAP HANA DB connection options.

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-hdb-conn-options

Technical name: `xs-hdb-conn-options`

> **Note**
>
> - In SAP Java Buildpack 1, `xs-hdb-conn-options` is compiled to Java 8 bytecode.
> - In SAP Java Buildpack 2, `xs-hdb-conn-options` is compiled to Java 17 bytecode.

Changelog

SAP Java Buildpack 1	SAP Java Buildpack 2

	`(15 October 2025)`
	`Version 2.25.0`
	• *com.sap.cloud.sjb:xs-env* is updated from 2.23.0 to 2.24.0

	`(1 October 2025)`
	`Version 2.24.0`
	• *com.sap.cloud.sjb:xs-env* is updated from 2.21.0 to 2.23.0

SAP Java Buildpack 1	SAP Java Buildpack 2
	(19 August 2025) Version 2.22.0 • *com.sap.cloud.sjb:xs-env* is updated from 2.20.0 to 2.21.0
	(29 July 2025) Version 2.21.0 • *com.sap.cloud.sjb:xs-env* is updated from 2.19.0 to 2.20.0
	(23 June 2025) Version 2.20.0 • *com.sap.cloud.sjb:xs-env* is updated from 2.18.0 to 2.19.0
	(23 May 2025) Version 2.19.0 • *com.sap.cloud.sjb:xs-env* is updated from 2.17.0 to 2.18.0
(29 July 2025) Version 1.65.0 • *com.sap.cloud.sjb:xs-env* is updated from version 1.63.0 to 1.64.0	(12 May 2025) Version 2.18.0 • *com.sap.cloud.sjb:xs-env* is updated from 2.16.0 to 2.17.0
(23 June 2025) Version 1.64.0 • *com.sap.cloud.sjb:xs-env* is updated from version 1.61.0 to 1.63.0	(17 March 2025) Version 2.17.0 • *com.sap.cloud.sjb:xs-env* is updated from 2.15.0 to 2.16.0
(25 March 2025) Version 1.62.0 • *com.sap.cloud.sjb:xs-env* is updated from version 1.60.0 to 1.61.0	(17 February 2025) Version 2.16.0 • *com.sap.cloud.sjb:xs-env* is updated from 2.14.0 to 2.15.0
(3 February 2025) Version 1.61.0 • *com.sap.cloud.sjb:xs-env* is updated from version 1.59.0 to 1.60.0	(3 February 2025) Version 2.15.0 • *com.sap.cloud.sjb:xs-env* is updated from 2.13.0 to 2.14.0

SAP Java Buildpack 1	SAP Java Buildpack 2

`(7 January 2025)`
`Version 1.60.0`
• *com.sap.cloud.sjb:xs-env* is updated from version 1.58.0 to 1.59.0
	`(7 January 2025)`
	`Version 2.14.0`
	• *com.sap.cloud.sjb:xs-env* is updated from 2.12.0 to 2.13.0

XS Java Parent

Use this library to consume common dependencies and Maven plug-in configurations.

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-java-parent

Technical name: `xs-java-parent`

> ### Note
> - In SAP Java Buildpack 1, `xs-java-parent` is compiled to Java 8 bytecode.
> - In SAP Java Buildpack 2, `xs-java-parent` is compiled to Java 17 bytecode.

Changelog

SAP Java Buildpack 1	SAP Java Buildpack 2

`(28 July 2025)`
`Version 1.21`
• No changes
	`(28 July 2025)`
	`Version 2.6.0 - 2.8.0`
	• No changes

`(7 January 2025)`
`Version 1.20.0`
• Class *com.sap.xs.java:xs-diagnostics* has been added to the `pom.xml` file.
	`(7 January 2025)`
	`Version 2.5.0`
	• Class *com.sap.xs.java:xs-diagnostics* has been added to the `pom.xml` file.

XS JDBC Routing Datasource

This wrapper library handles datasource creation in single or multitenancy mode while working with JDBC.

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-jdbc-routing-datasource

Technical name: `xs-jdbc-routing-datasource`

> ### Note
> - In SAP Java Buildpack 1, `xs-jdbc-routing-datasource` is compiled to Java 8 bytecode.

- In SAP Java Buildpack 2, `xs-jdbc-routing-datasource` is compiled to Java 17 bytecode.

Changelog

SAP Java Buildpack 1	SAP Java Buildpack 2

	`(28 July 2025)`
	`Version 2.14.0 - 2.16.0`
	• No changes

	`(17 March 2025)`
	`Version 2.13.0`
	• *org.slf4j:slf4j-api* is updated from version 2.0.16 to 2.0.17

`(28 July 2025)`
`Version 1.52.0 - 1.54.0`
• No changes
	`(17 February 2025)`
	`Version 2.11.0 - 2.12.0`
	• No changes

`(7 January 2025)`
`Version 1.48.0 – 1.51.0`
• Minor changes
• Integration tests
	`(7 January 2025)`
	`Version 2.6.0 – 2.10.0`
	• Minor changes
	• Integration tests

Related Information

Maven Repository: SAP Cloud SJB
SAP Java Buildpack: Libraries (2024) [page 377]

4.1.3.5.1.20.2 SAP Java Buildpack: Libraries (2024)

On this page you can find changelogs about the 2024 updates in the main five libraries of SAP Java Buildpack 1 and 2, provided on Maven Repository.

XS Env

Use this library to set up and access environment variables and services.

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-env

Technical name: `xs-env`

> **Note**
>
> - In SAP Java Buildpack 1, `xs-env` is compiled to Java 8 bytecode.
> - In SAP Java Buildpack 2, `xs-env` is compiled to Java 17 bytecode.

Changelog

SAP Java Buildpack 1	SAP Java Buildpack 2

`(7 January 2025)`
`Version 1.59.0`
• *com.fasterxml.jackson.core:jackson-core* is updated from version 2.18.0 to 2.18.2
• *com.faster.jackson.core:jackson-databind*
• *com.faster.jackson.core:jackson-annotations* is updated from version 2.18.0 to 2.18.2
	`(9 December 2024)`
	`Version 2.12.0`
	• *com.fasterxml.jackson.core:jackson-core* is updated from version 2.18.0 to 2.18.2
	• *com.faster.jackson.core:jackson-databind* is updated from version 2.18.0 to 2.18.2
	• *com.faster.jackson.core:jackson-annotations* is updated from version 2.18.0 to 2.18.2

`(15 October 2024)`
`Version 1.58.0`
• *com.fasterxml.jackson.core:jackson-core* is updated from version 2.17.2 to 2.18.0
• *com.faster.jackson.core:jackson-databind* is updated from version 2.17.2 to 2.18.0
• *com.faster.jackson.core:jackson-annotations* is updated from version 2.17.2 to 2.18.0
	`(15 October 2024)`
	`Version 2.11.0`
	• *com.fasterxml.jackson.core:jackson-core* is updated from version 2.17.2 to 2.18.0
	• *com.faster.jackson.core:jackson-databind* is updated from version 2.17.2 to 2.18.0
	• *com.faster.jackson.core:jackson-databind* is updated from version 2.17.2 to 2.18.0

`(1 October 2024)`
`Version 1.55.0 – 1.57.0`
• Minor changes
• Integration tests
	`(1 October 2024)`
	`Version 2.8.0 – 2.10.0`
	• Minor changes
	• Integration tests

`(6 August 2024)`
`Version 1.54.0`
• *com.fasterxml.jackson.core:jackson-core* is updated from version 2.17.1 to 2.17.2
• *com.faster.jackson.core:jackson-databind* is updated from version 2.17.1 to 2.17.2
• *com.faster.jackson.core:jackson-annotations* is updated from version 2.17.1 to 2.17.2
	`(6 August 2024)`
	`Version 2.7.0`
	• *com.fasterxml.jackson.core:jackson-core* is updated from version 2.17.1 to 2.17.2
	• *com.faster.jackson.core:jackson-databind* is updated from version 2.17.1 to 2.17.2
	• *com.faster.jackson.core:jackson-annotations* is updated from version 2.17.1 to 2.17.2

SAP Java Buildpack 1	SAP Java Buildpack 2
(29 May 2024) Version 1.53.0 • A new *findServices* functionality is introduced. It provides a list with Cloud Foundry services that match certain required conditions.	(29 May 2024) Version 2.6.0 • A new *findServices* functionality is introduced. It provides a list with Cloud Foundry services that match certain required conditions.
(14 May 2024) Version 1.52.0 • *com.fasterxml.jackson.core:jackson-core* is updated from version 2.17.0 to 2.17.1 • *com.faster.jackson.core:jackson-databind* is updated from version 2.17.0 to 2.17.1 • *com.faster.jackson.core:jackson-annotations* is updated from version 2.17.0 to 2.17.1 • The URL and description of the xs-env library have been improved.	(14 May 2024) Version 2.5.0 • *com.fasterxml.jackson.core:jackson-core* is updated from version 2.17.0 to 2.17.1 • *com.faster.jackson.core:jackson-databind* is updated from version 2.17.0 to 2.17.1 • *com.faster.jackson.core:jackson-annotations* is updated from version 2.17.0 to 2.17.1 • The URL and description of the xs-env library have been improved.
(2 April 2024) Version 1.51.0 • Minor changes • *xs-java-parent* is updated from version 1.16.0 to 1.17.0	(20 March 2024) Version 2.4.0 • *com.fasterxml.jackson.core:jackson-core* is updated from version 2.16.1 to 2.17.0 • *com.faster.jackson.core:jackson-databind* is updated from version 2.16.1 to 2.17.0 • *com.faster.jackson.core:jackson-annotations* is updated from version 2.16.1 to 2.17.0
(20 March 2024) Version 1.50.0 • *com.fasterxml.jackson.core:jackson-core* is updated from version 2.16.1 to 2.17.0 • *com.faster.jackson.core:jackson-databind* is updated from version 2.16.1 to 2.17.0 • *com.faster.jackson.core:jackson-annotations* is updated from version 2.16.1 to 2.17.0	(16 February 2024) Version 2.2.0 • This is the first publicly released version. • *com.fasterxml.jackson.core:jackson-core* is updated from version 2.16.0 to 2.16.1 • *com.faster.jackson.core:jackson-databind* is updated from version 2.16.0 to 2.16.1 • *com.faster.jackson.core:jackson-annotations* is updated from version 2.16.0 to 2.16.1

XS User Holder

Use this library to access user information during authentication and authorization when using SAP BTP Security Services Integration Libraries .

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-user-holder

Technical name: `xs-user-holder`

> **Note**
>
> - In SAP Java Buildpack 1, `xs-user-holder` is compiled to Java 8 bytecode.
> - In SAP Java Buildpack 2, `xs-user-holder` is compiled to Java 17 bytecode.

Changelog

SAP Java Buildpack 1	SAP Java Buildpack 2

`(14 May 2024)`
`Version 1.56.0`
• *sap.cloud.security.version* is updated from 2.17.3 to 2.17.5
• The URL and description of the xs-user-holder library have been improved.
	`(14 May 2024)`
	`Version 2.5.0`
	• The URL and description of the xs-user-holder library have been improved.

`(2 April 2024)`
`Version 1.55.0`
• *com.sap.cloud.sjb.xs-env.version* is updated from 1.50.0 to 1.51.0
	`(2 April 2024)`
	`Version 2.4.0`
	• For *sap.cloud.security.version* 3.3.5, an obsolete **java-api** dependency has been removed.

`(23 January 2024)`
`Version 1.54.0`
• *sap.cloud.security.version* is updated from 2.17.2 to 2.17.3
	`(5 March 2024)`
	`Version 2.3.0`
	• *sap.cloud.security.version* is updated from 3.3.1 to 3.3.5

XS HDB Conn Options

Use this library to handle SAP HANA DB connection options.

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-hdb-conn-options

Technical name: `xs-hdb-conn-options`

> **Note**
>
> - In SAP Java Buildpack 1, `xs-hdb-conn-options` is compiled to Java 8 bytecode.
> - In SAP Java Buildpack 2, `xs-hdb-conn-options` is compiled to Java 17 bytecode.

SAP Java Buildpack 1	SAP Java Buildpack 2
	(9 December 2024) Version 2.13.0 • *com.sap.cloud.sjb.xs-env.version* is updated from 2.11.0 to 2.12.0
(15 October 2024) Version 1.58.0 • *com.sap.cloud.sjb.xs-env.version* is updated from 1.57.0 to 1.58.0	(15 October 2024) Version 2.12.0 • *com.sap.cloud.sjb.xs-env.version* is updated from 2.10.0 to 2.11.0
(1 October 2024) Version 1.57.0 • *com.sap.cloud.sjb.xs-env.version* is updated from 1.56.0 to 1.57.0	(1 October 2024) Version 2.11.0 • *com.sap.cloud.sjb.xs-env.version* is updated from 2.9.0 to 2.10.0
(17 September 2024) Version 1.56.0 • *com.sap.cloud.sjb.xs-env.version* is updated from 1.55.0 to 1.56.0	(17 September 2024) Version 2.10.0 • *com.sap.cloud.sjb.xs-env.version* is updated from 2.8.0 to 2.9.0
(9 September 2024) Version 1.55.0 • *com.sap.cloud.sjb.xs-env.version* is updated from 1.54.0 to 1.55.0	(9 September 2024) Version 2.9.0 • *com.sap.cloud.sjb.xs-env.version* is updated from 2.7.0 to 2.8.0
(6 August 2024) Version 1.54.0 • *com.sap.cloud.sjb.xs-env.version* is updated from 1.53.0 to 1.54.0	(6 August 2024) Version 2.8.0 • *com.sap.cloud.sjb.xs-env.version* is updated from 2.6.0 to 2.7.0
(29 May 2024) Version 1.53.0 • *com.sap.cloud.sjb.xs-env.version* is updated from 1.52.0 to 1.53.0	(29 May 2024) Version 2.7.0 • *com.sap.cloud.sjb.xs-env.version* is updated from 2.5.0 to 2.6.0

SAP Java Buildpack 1	SAP Java Buildpack 2

`(14 May 2024)`
`Version 1.52.0`
• *com.sap.cloud.sjb.xs-env.version* is updated from 1.51.0 to 1.52.0
• The URL and description of the xs-hdb-conn-options library have been improved.
	`(14 May 2024)`
	`Version 2.6.0`
	• *com.sap.cloud.sjb.xs-env.version* is updated from 2.4.0 to 2.5.0
	• The URL and description of the xs-hdb-conn-options library have been improved.

`(2 April 2024)`
`Version 1.51.0`
• *com.sap.cloud.sjb.xs-env.version* is updated from 1.50.0 to 1.51.0
	`(2 April 2024)`
	`Version 2.5.0`
	• Minor changes

`(20 March 2024)`
`Version 1.50.0`
• *com.sap.cloud.sjb.xs-env.version* is updated from 1.49.0 to 1.50.0
	`(20 March 2024)`
	`Version 2.4.0`
	• *com.sap.cloud.sjb.xs-env.version* is updated from 2.3.0 to 2.4.0

XS Java Parent

Use this library to consume common dependencies and Maven plug-in configurations.

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-java-parent

Technical name: `xs-java-parent`

> **Note**
>
> - In SAP Java Buildpack 1, `xs-java-parent` is compiled to Java 8 bytecode.
> - In SAP Java Buildpack 2, `xs-java-parent` is compiled to Java 17 bytecode.

Changelog

SAP Java Buildpack 1	SAP Java Buildpack 2

`(9 September 2024)`
`Version 1.19.0`
• Plugin *acoco-maven-plugin*, version 0.8.10, has been added to the `pom.xml` file.
• The following classes have been removed from the `pom.xml` file:
  • *com.sap.xs.java:xs-sapjvm-statistics-provider*
  • *com.sap.xs.java:xs-statistics-factory*
  • *com.sap.xs.java:xs-hystrix-hook*
  • All *com.sap.xs.java:xs-jdsr** classes
	`(9 September 2024)`
	`Version 2.4.0`
	• Plugin *acoco-maven-plugin*, version 0.8.10, has been added to the `pom.xml` file.
	• The following classes have been removed from the `pom.xml` file:
	  • *com.sap.xs.java:xs-statistics-factory*
	  • All *com.sap.xs.java:xs-jdsr** classes

`(14 May 2024)`
`Version 1.18.0`
• The URL and description of the xs-java-parent library have been improved.
	`(14 May 2024)`
	`Version 2.3.0`
	• The URL and description of the xs-java-parent library have been improved.

`(2 April 2024)`
`Version 1.17.0`
• Minor changes
	`(16 February 2024)`
	`Version 2.2.0`
	• This is the first publicly released version.
	• *xs-memory-calculator* has been removed from the `pom.xml` file.
	• *xs-sapjvm-statistics-provider* has been removed from the `pom.xml` file.

XS JDBC Routing Datasource

This wrapper library handles datasource creation in single or multitenancy mode while working with JDBC.

Repository: https://mvnrepository.com/artifact/com.sap.cloud.sjb/xs-jdbc-routing-datasource

Technical name: `xs-jdbc-routing-datasource`

> **Note**
>
> - In SAP Java Buildpack 1, `xs-jdbc-routing-datasource` is compiled to Java 8 bytecode.
> - In SAP Java Buildpack 2, `xs-jdbc-routing-datasource` is compiled to Java 17 bytecode.

Changelog

SAP Java Buildpack 1	SAP Java Buildpack 2

`(14 May 2024)`
`Version 1.47.0`
• The URL and description of the xs-jdbc-routing-data-source library have been improved.
• The eager loading of data sources has been deleted for all managed instances.
	`(14 May 2024)`
	`Version 2.5.0`
	• The URL and description of the xs-jdbc-routing-data-source library have been improved.

`(2 April 2024)`
`Version 1.46.0`
• Minor changes
	`(2 April 2024)`
	`Version 2.4.0`
	• Minor changes

`(20 February 2024)`
`Version 1.45.0`
• *com.sap.cloud.instancemanager.version* is updated from 2.2.1 to 3.14.0. Also, its **client-api** dependency has been changed to just **client**.
	`(20 March 2024)`
	`Version 2.3.0`
	• Minor changes

Related Information

Maven Repository: SAP Cloud SJB
SAP Java Buildpack: Libraries (2025) [page 371]

4.1.3.5.2 Developing Node.js in the Cloud Foundry Environment

This section offers selected information for Node.js development on SAP BTP, Cloud Foundry and references to more detailed sources.

You'll get information about the buildpack supported by SAP, the Node.js packages, and how to consume them in your application.

There is also a tutorial with an introduction to securing your application, and some tips and tricks for developing and running Node.js applications on SAP BTP, Cloud Foundry.

Node.js Community Buildpack

SAP BTP uses the standard Node.js buildpack provided by the Cloud Foundry community to deploy Node.js applications.

To get familiar with the buildpack and how to deploy applications with it, take a look at the Cloud Foundry Node.js Buildpack documentation .

SAP Node.js Packages

You can download and consume SAP-developed Node.js packages via the SAP NPM Registry. There is an overview of Node.js packages developed by SAP, what they are meant for, and where they are included in the SAP HANA Developer Guide for XS Advanced Model. See:

- Download and Consume SAP Node.js Packages
- Standard Node.js Packages
- The SAP NPM Registry

Buildpack Versioning

The SAP BTP, Cloud Foundry environment provides one recent version of `nodejs_buildpack` as part of its system buildpacks. To check this version, proceed as follows:

1. Log in to a particular SAP BTP region and subaccount. For example, if your region is **eu10**, run:

   ```
   cf api https://api.cf.eu10.hana.ondemand.com
   ```

2. Then run:

   ```
   cf buildpacks
   ```

To learn about changes in the Node.js buildpack's versions and features, regularly check the latest buildpack releases in the GitHub community page.

Usage

To use this buildpack, specify its name when deploying a Node.js application to the SAP BTP, Cloud Foundry environment. You can do it the following ways:

- Specify it directly in the `cf push` command:

  ```
  cf push -f <PATH_TO_APP_MANIFEST> -b nodejs_buildpack
  ```

- Specify it in the `manifest.yml` file of your application by using the `buildpacks` attribute:

  ```
  ---
  applications:
  - name: <APP_NAME>
    memory: 512M
    buildpacks:
    - nodejs_buildpack
    ...
  ```

Then, you can deploy the application like this:

```
cf push <app_name>
```

- Specify it in the `mtad.yaml` deployment descriptor file (for multi-target applications) by using the `buildpack` attribute:

```
...
modules:
  - name: <APP_NAME>
    type: nodejs
    path: <path_to_archive>
    properties:
      ...
    parameters:
      ...
      memory: 512M
      buildpack: nodejs_buildpack
...
```

Then, you can deploy the application like this:

```
cf push <app_name>
```

Supported Versions

The `nodejs_buildpack` running on SAP BTP, Cloud Foundry environment supports the following versions:

- Node.js **20** - this version reached end-of-life on April 30, 2026.
- Node.js **22**
- Node.js **24**

Deprecated Versions

Node.js 20

Node.js 20 reached end of life on **April 30, 2026** according to the Node.js Roadmap . It was removed from SAP BTP, Cloud Foundry environment as well. This means that restaging and redeployment of Cloud Foundry applications running on Node.js 20 will fail.

Action: We strongly recommend that you migrate your applications to Node.js 22 as soon as possible.

In exceptional cases (if you haven't managed to switch to Node.js 22 in time), to avoid application failures during redeployment, you can pin the last buildpack version that contains Node.js 20 (v1.9.1), as provided by the nodejs-buildpack community. To learn how, see: Specify a buildpack version in manifest.yml [page 390]

If you are using MTA deployment descriptors, in your *mtad.yaml* file you need to define module type **javascript.nodejs** and set parameter `buildpack` to **nodejs_buildpack**. For example:

```
modules:
- name: myapp
```

```
  type: javascript.nodejs
  parameters:
    memory: 512M
    buildpack: nodejs_buildpack
```

If you want to pin a particular buildpack version (for example, **1.9.1**), you can do it the following way:

```
modules:
- name: myapp
  type: javascript.nodejs
  parameters:
    memory: 512M
    buildpack: https://github.com/cloudfoundry/nodejs-buildpack.git#v1.9.1
```

To learn more, see MTA Module Types.

> → Remember
>
> SAP does **not** recommend use of deprecated Node.js versions, as support and security fixes are no longer provided for them.

What's New

To check the latest news and updates about the Node.js buildpack, go to its release notes: What's New for Node.js Buildpack

> ⓘ Note
>
> In May 2023, SAP migrated the root file system used in SAP BTP, Cloud Foundry runtime from the deprecated `cflinuxfs3` stack to **cflinuxfs4**. If you are running Node.js applications on SAP BTP, Cloud Foundry using the Node.js buildpack, we recommend that you update and migrate your applications, as well as the Node.js buildpack. For more information about migration timelines, risks, and consequences, see:
>
> - Deprecation of Cloud Foundry Stack cflinuxfs3 and Migration to cflinuxfs4
> - What's New: cflinuxfs4 becomes the default stack

Troubleshooting

If you encounter an issue while using the Node.js buildpack, you can:

- Search for your problem in our Troubleshooting [page 391] section.
- Create an incident for your specific problem, using support component **BC-CP-CF-BLDP**. To provide the necessary details, use the following template: Initial Problem-Related Data [page 400]

Node.js Tutorial

The following tutorial will guide you through creating a Node.js application in Cloud Foundry Command Line Interface (cf CLI), consuming a Cloud Foundry service, and setting up authentication and authorization checks. See: Create an Application with Cloud Foundry Node.js Buildpack

Tips and Tricks

For selected tips and tricks for your Node.js development, see Tips and Tricks for Node.js Applications [page 388].

4.1.3.5.2.1 Tips and Tricks for Node.js Applications

Get to know the Node.js buildpack for the SAP BTP, Cloud Foundry environment.

Check the following Cloud Foundry documentation:

- Node.js Buildpack
- Tips for Node.js Developers

NPM Version

When deploying an application in the SAP BTP, Cloud Foundry environment without specifying the *npm* version in the **package.json** file, it will install the default *npm* version (depending on the Node.js version) during the build step. A version mismatch can cause the build step to fail.

To check your Node.js version, run:

```
node --version
```

To check your *npm* version, run:

```
npm --version
```

Tip: We recommend that you use the same *npm* version as the one on the SAP BTP, Cloud Foundry environment.

Alternatively, you can specify the *npm* version in the **package.json** file. For example, if you use Node.js 20, you can set:

```
"engines": {
"npm": "^8.4.1"
},
```

> ### Note
>
> Bear in mind that **Node.js** and **npm** are different products, with their own independent versions. To check the version mapping between them, see NodeJS: Previous Releases .

Vendor Application Dependencies

Sometimes, productive Cloud Foundry applications might need to be deployed as self-contained. That means, they need to carry all of their dependencies so that the staging process does not require any network calls. For more information, see: Vendoring App Dependencies

Depending on the region where the application is deployed:

- Running on the China (Shanghai) region: it is **mandatory** for the application to be deployed as self-contained.
- Running on the AWS, Azure, or GCP regions: it is only **recommended** but not mandatory for the application to be deployed as self-contained.

There are various reasons to use vendoring. For example, productive applications usually rely on security scans and because *npm* doesn't provide reliable dependency fetch, it's possible that your Node.js application ends up with different dependencies in case such are installed during deployment. Additionally, *npm* downloads any missing dependencies from its registry, so if this registry isn't accessible for some reason, the deployment can fail.

> ### Note
>
> Bear in mind when using dependencies containing native code that you need to reinstall in the same environment as the Cloud Foundry container, or make sure that the package has built-in support for it.

To ensure that prepackaged dependencies are pushed to the SAP BTP, Cloud Foundry environment and on-premise runtime, make sure that the `node_modules` directory isn't listed in the `.cfignore` file.

It's also preferable that development dependencies are not deployed for productive deployments. To ensure that, run one of the following commands:

- `npm prune --production`
- `npm prune --omit=dev`

Out of Memory at Runtime

For performance reasons, Node.js (powered by V8 engine) has lazy garbage collection. Even if there are no memory leaks in your application, this can cause occasional restarts, as explained in the official Cloud Foundry documentation – Tips for Node.js Applications .

Enforce the garbage collector to run before the memory is consumed by limiting the V8 application's heap size at about ~75% of the available memory. To do that, you can either use the `OPTIMIZE_MEMORY` environment

variable supported by the Node.js buildpack, or specify the V8 heap size directly in your application *start* command (recommended).

Example for an application started with 256M of RAM:

> Sample Code
>
> ```
> node --max_old_space_size=192 server.js
> ```

You can optimize V8 behavior using additional options. To list these options, run:

```
node --v8-options
```

Specify a buildpack version in `manifest.yml`

At some point, you might need (or decide) to deploy your application with a particular buildpack version from the community nodejs-buildpack repository. For example, if this buildpack contains a Node.js version that is no longer supported by the SAP BTP, Cloud Foundry environment.

Let's say, you want to pin version 1.8.34. To do that, proceed as follows:

1. Open the `manifest.yml` file of your Node.js application.
2. For the `buildpack` attribute, add the URL to version 1.8.34, like this:

   ```
   ---
   applications:
   - name: myapp
     random-route: true
     buildpack: https://github.com/cloudfoundry/nodejs-buildpack.git#v1.8.34
     memory: 128M
   ```

3. Redeploy your application. Run:

   ```
   cf push myapp
   ```

Alternative way:

If you don't want to make changes in your `manifest.yml` file, you can include the buildpack version in the `cf push` command.

- To deploy just a **single** application with this particular buildpack version, run:

  ```
  cf push myapp -b https://github.com/cloudfoundry/nodejs-buildpack.git#v1.8.34
  ```

- To pin this buildpack version for **all** applications running in your SAP BTP, Cloud Foundry environment subaccount, run:

  ```
  cf push -b https://github.com/cloudfoundry/nodejs-buildpack.git#v1.8.34
  ```

Specify a buildpack version in `mtad.yaml`

If you have a multi-target application, you can pin a specific buildpack version for it as well. Let's say, you need version 1.8.34 . To do that, proceed as follows:

1. Open the `mtad.yaml` file of your Node.js application.
2. Set the module type to **javascript.nodejs**, for the `buildpack` attribute, add the URL to version 1.8.34, like this:

```
modules:
- name: myapp
  type: javascript.nodejs
  parameters:
    buildpack: https://github.com/cloudfoundry/nodejs-buildpack.git#v1.8.34
```

3. Redeploy your application. Run:

```
cf push myapp
```

For more information, see: MTA Module Types

Specify application memory in `manifest.yml`

When deploying an application in the Cloud Foundry environment without specifying the application memory requirements, the Cloud Foundry controller assigns the default (1G of RAM currently) for your application. Many Node.js applications require less memory and assigning the default is a waste of resources.

To save memory from your quota, specify the memory size in the deployment descriptor of your Cloud Foundry application – `manifest.yml`. For example:

```
---
applications:
- name: myapp
  memory: 128M
  buildpack: nodejs_buildpack
...
```

4.1.3.5.2.2 Node.js Buildpack: Troubleshooting

This page provides solutions on known issues related to the Node.js buildpack.

If you can't find a solution to your problem, create an incident to component **BC-CP-CF-BLDP**.

To learn how to start, see: Troubleshooting [page 400]

XSJS application stopped working after upgrade to Node v.16

Problem

You have an XSJS application, which was working fine on Node.js 14.

After an upgrade to Node 16, it fails to rebuild or redeploy and is no longer working. In the logs, you see the following error messages:

```
npm ERR! gyp ERR! command "/opt/nodejs/node-v16.15.0-linux-x64/bin/node"
"/home/user/.node_modules_global/lib/node_modules/npm/node_modules/node-gyp/bin/
node-gyp.js"
"rebuild"npm ERR! gyp ERR! cwd /home/user/projects/DataCentral/dc_node/
node_modules/@sap/xsjs/node_modules/@sap/fibers
```

You may also encounter the following error messages:

```
npm WARN EBADENGINE   required: { node: '^10 || ^12 || ^14' },
npm WARN EBADENGINE   current: { node: 'v16.19.0', npm: '8.19.3' }
npm WARN EBADENGINE Unsupported engine {
npm WARN EBADENGINE   package: '@sap/xsjs@6.7.5'
```

Reason

XSJS applications use the @sap/fibers library (on which the @sap/xsjs package depends), which is **not supported** on Node.js 16 and later versions.

Solution

To keep your XSJS applications up and running on latest Node.js versions, you need to migrate them to the new XSJS layer with an asynchronous API. See:

- SAP HANA 2 - ASYNC XS JavaScript API Reference
- https://www.npmjs.com/package/@sap/async-xsjs

> ⓘ Note
>
> Bear in mind that Node.js 14 was removed from the community buildpack on **April 30, 2023**.

We recommend migration to Node.js 18 or 20, and respectively migration to Async-XSJS, as soon as possible.

In exceptional cases (if you haven't managed to complete the migration to Async-XSJS), to avoid application failures during redeployment, you can pin the last buildpack version that contains Node.js 14, as provided by the nodejs-buildpack community. To learn how to do this, see: Specify a buildpack version in manifest.yml

Nevertheless, this approach can only be used as a temporary solution, until you complete the migration!

> → Remember
>
> SAP does **not** recommend use of deprecated Node.js versions, as support and security fixes are no longer provided for them.

Node.js 16 reached end of life

Problem

Node.js 16 reached end of life on **September 11, 2023** (instead of April 2024, as originally planned), and shortly after was removed from the community buildpack. This means that, after this date, deployment and redeployment of applications running on Node.js 16 would fail. For more information, see:

- Node.js Roadmap
- Bringing forward the End-of-Life Date for Node.js 16

Solution

To avoid such issues, please migrate to Node.js 18 or higher as soon as possible!

In exceptional cases (if you haven't managed to switch to Node.js 18 or 20 yet), to avoid application failures during redeployment, you can pin the last buildpack version that contains Node.js 16, as provided by the nodejs-buildpack community. To learn how to do this, see: Specify a buildpack version in manifest.yml

Nevertheless, this approach can only be used as a temporary solution, until you complete migration to a later version of Node.js!

> → Remember
>
> SAP does **not** recommend use of deprecated Node.js versions, as support and security fixes are no longer provided for them.

Application running on Node.js 18 fails after restage or redeploy

Problem

You have a working Node.js application, which runs on Node.js 18. After restage or redeploy, it fails and is no longer working. In the logs, you see the following error messages:

```
npm WARN EBADENGINE   required: { node: '^20 || ^22 || ^14' },
npm WARN EBADENGINE   current: { node: 'v18.19.0', npm: '8.19.3' }
npm WARN EBADENGINE Unsupported engine {
```

Reason

Node.js 18 reached end of life on **April 30, 2025** (according to the Node.js Roadmap), and was removed from the SAP BTP, Cloud Foundry runtime with `nodejs_buildpack` version **v1.8.41**.

Solution

To keep your Node.js applications up and running, please migrate to Node.js 20 or 22 as soon as possible.

In exceptional cases (if you haven't managed to migrate to a latest Node.js version), to avoid application failures you can pin the last buildpack version that contains Node.js 18, for example, v1.8.39. To learn how to do this, see: Specify a buildpack version in manifest.yml

If you are using MTA deployment descriptors, in your *mtad.yaml* file you need to define module type **javascript.nodejs** and set parameter `buildpack` to **nodejs_buildpack**. For example:

```
modules:
- name: myapp
  type: javascript.nodejs
  parameters:
    memory: 512M
    buildpack: nodejs_buildpack
```

If you want to pin a particular buildpack version (for example, **1.8.39**), you can do it the following way:

```
modules:
- name: myapp
  type: javascript.nodejs
  parameters:
    memory: 512M
    buildpack: https://github.com/cloudfoundry/nodejs-buildpack.git#v1.8.39
```

To learn more, see MTA Module Types.

> **Note**
>
> This approach can only be used as a temporary solution, until you complete the migration.

4.1.3.5.3 Developing Python in the Cloud Foundry Environment

This section offers selected information for Python development on the SAP BTP, Cloud Foundry environment and references to more detailed sources.

You'll get information about the buildpack supported by SAP, about the Python packages, and how to consume them in your application.

There is also a tutorial with an introduction to securing your application, and some tips and tricks for developing and running Python applications on the SAP BTP, Cloud Foundry environment.

Python Community Buildpack

SAP BTP uses the standard Python buildpack provided by the Cloud Foundry community to deploy Python applications.

To get familiar with the buildpack and how to deploy applications with it, take a look at the Cloud Foundry Python Buildpack documentation .

SAP Python Packages

SAP includes a selection of Python packages, which are available for download and use for customers and partners who have the appropriate access authorization. To download them, log on to SAP Software Download Center and search for software component **XS_PYTHON**, which is an archive that contains the SAP packages.

The following table lists the SAP Python packages that are currently available. For more details about the contents of each Python package, as well as any configuration tips, see the README file in the corresponding package.

Package	Description
`sap_instance_manager`	Python package for creating and deleting service instances per tenant within an application at runtime.
`sap_audit_logging`	Provides audit logging functionalities for Python applications.
`sap_xssec`	SAP XS Advanced Container Security API for Python.
`sap_cf_logging`	This is a collection of support libraries for Python applications running on Cloud Foundry that: • provide means to emit structured application log messages • instrument web applications of your application stack to collect request metrics
`hdbcli`	The SAP HANA Database Client provides means for database connectivity.

Buildpack Versioning

The SAP BTP, Cloud Foundry environment provides one recent version of `python_buildpack` as part of its system buildpacks. To check this version, proceed as follows:

1. Log in to a particular SAP BTP region and subaccount. For example, if your region is **eu10**, run:

   ```
   cf api https://api.cf.eu10.hana.ondemand.com
   ```

2. Then run:

   ```
   cf buildpacks
   ```

To learn about changes in the Python buildpack's versions and features, regularly check the latest buildpack releases in the GitHub community page.

Usage

To use this buildpack, specify its name when deploying a Python application to the SAP BTP, Cloud Foundry environment. You can do that in two ways:

- Specify it directly in the `cf push` command:

  ```
  cf push -f <PATH_TO_APP_MANIFEST> -b python_buildpack
  ```

- Specify it in the `manifest.yml` file of your application by using the `buildpacks` attribute:

  ```
  ---
  applications:
  - name: <APP_NAME>
    memory: 512M
    buildpacks:
    - python_buildpack
    ...
  ```

  Then, you can deploy the application like this:

  ```
  cf push <app_name>
  ```

Supported Versions

The `python_buildpack` supports the following versions:

- Python **3.9**
- Python **3.10**
- Python **3.11**
- Python **3.12**
- Python **3.13**

> ⓘ Note
>
> Version **3.8** is out of maintenance, as per Python release roadmap . We strongly recommend that you switch to version 3.9 or higher.

You can also decide to deploy your application with a particular buildpack version from the community python-buildpack repository. To learn how, see: Specify a buildpack version in manifest.yml [page 398]

> → Remember
>
> SAP does **not** recommend use of deprecated Python versions, as support and security fixes are no longer provided for them.

What's New

To check the latest news and updates about the Python buildpack, go to its buildpack releases on the GitHub community page.

> **Note**
>
> In May 2023, SAP migrated the root file system used in SAP BTP, Cloud Foundry runtime from the deprecated `cflinuxfs3` stack to **cflinuxfs4**. If you are running Python applications on SAP BTP, Cloud Foundry using the Python buildpack, we recommend that you update and migrate your applications, as well as the Python buildpack. For more information about migration timelines, risks, and consequences, see:
>
> - Deprecation of Cloud Foundry Stack cflinuxfs3 and Migration to cflinuxfs4
> - What's New for SAP BTP, Cloud Foundry Runtime (cflinuxfs)

Troubleshooting

If you encounter an issue while using the Python buildpack, you can:

- Search for your problem in our Troubleshooting [page 399] section.
- Create an incident for your specific problem, using support component **BC-CP-CF-BLDP**. To provide the necessary details, use the following template: Initial Problem-Related Data [page 400]

Python Tutorial

The following tutorial will guide you through creating a Python application in Cloud Foundry Command Line Interface (cf CLI), consuming Cloud Foundry services, and setting up authentication and authorization checks. See: Create an Application with Cloud Foundry Python Buildpack

Tips and Tricks

Selected tips and tricks for your Python development. See Tips and Tricks for Python Applications [page 397].

4.1.3.5.3.1 Tips and Tricks for Python Applications

Get to know the Python buildpack for the SAP BTP, Cloud Foundry environment.

Check the following Cloud Foundry documentation: Python Buildpack

Vendor Application Dependencies

Sometimes, productive Cloud Foundry applications might need to be deployed as self-contained. That means, they need to carry all of their dependencies so that the staging process does not require any network calls. For more information, see: Vendor App Dependencies

Depending on the region where the application is deployed:

- Running on the China (Shanghai) region: it is **mandatory** for the application to be deployed as self-contained
- Running on the AWS, Azure, or GCP regions: it is only **recommended** but not mandatory for the application to be deployed as self-contained.

Specify a buildpack version in `manifest.yml`

At some point, you might need (or decide) to deploy your application with a particular buildpack version from the community python-buildpack repository. For example, if this buildpack contains a Python version that is no longer supported by the SAP BTP, Cloud Foundry environment.

Let's say, you want to pin version 1.8.4 . To do that, proceed as follows:

1. Open the **manifest.yml** file of your Python application.
2. For the `buildpack` attribute, add the URL to version 1.8.4, like this:

```
---
applications:
- name: myapp
  random-route: true
  buildpack: https://github.com/cloudfoundry/python-buildpack.git#v1.8.4
  memory: 128M
```

3. Redeploy your application. Run:

```
cf push myapp
```

Alternative way:

If you don't want to make changes in your **manifest.yml** file, you can include the buildpack version in the `cf push` command.

- To deploy just a single application with this particular buildpack version, run:

```
cf push myapp -b https://github.com/cloudfoundry/python-buildpack.git#v1.8.4
```

- To pin this buildpack version for all applications running in your SAP BTP, Cloud Foundry environment subaccount, run:

```
cf push -b https://github.com/cloudfoundry/python-buildpack.git#v1.8.4
```

Specify application memory in `manifest.yml`

When deploying an application in the SAP BTP, Cloud Foundry environment without specifying the application memory requirements, the Cloud Foundry controller assigns the default (1G of RAM currently) for your application. Many Python applications require less memory and assigning the default is a waste of resources.

To save memory from your quota, specify the memory size in the deployment descriptor of your Cloud Foundry application – `manifest.yml`. For example:

```
---
applications:
- name: myapp
  memory: 128M
  buildpack: python_buildpack
...
```

Additional Tips

- The `cfenv` package provides access to the Cloud Foundry application environment settings by parsing all the relevant environment variables. The settings are returned as a class instance. See: https://github.com/jmcarp/py-cfenv
- When developing Python applications, it's a good practice to use virtual environments. The most popular Python package providing such a functionality is `virtualenv`. See: https://virtualenv.pypa.io/en/stable/
- The PEP 8 Style Guide for Python will help you improve your applications.

4.1.3.5.3.2 Python Buildpack: Troubleshooting

This page provides solutions on known issues related to the Python buildpack.

If you can't find a solution to your problem, create an incident to component **BC-CP-CF-BLDP**.

To learn how to start, see: Troubleshooting [page 400]

Error: Could not install python: no match for 3.x.x

Problem

You are trying to deploy or restage a working Python application but it suddenly fails.

Reason

If an existing application was working fine, and after being redeployed or restaged it fails for no logical reason, take a look at the logs. It might be that the Python version you're using has been recently removed from the latest releases in the community buildpack .

For example: You are using Python v **3.10.1**, and in the logs you see:

```
Could not install python: no match found for 3.10.1 in [3.8.19 3.9.19 3.10.14
3.11.9 3.12.5]
```

Solution

To make your application work again, go to the `runtime.txt` file and update the Python version like this:

```
python-3.10.x
```

You can also upgrade to a higher major version (11, 12 or 13). In this case, set the runtime analogically:

- `python-3.11.x`
- `python-3.12.x`
- `python-3.13.x`

4.1.3.5.4 Troubleshooting

While using the buildpacks, you might encounter issues or have questions regarding specific settings, limitations, or other aspects.

The following troubleshooting pages are related to the relevant development languages and buildpacks:

- Node.js Buildpack: Troubleshooting [page 391]
- Python Buildpack: Troubleshooting [page 399]

If you cannot find a solution to your problem among these pages, you can create a support incident.

Create a Support Incident

1. Open SAP for Me and log on.
2. Open the *Services & Support* dashboard.
3. (Optional) You can use the *Knowledge Search* tab to look for existing solutions. They come from different knowledge sources, such as:
   - SAP Notes & KBAs
   - SAP Support Portal
   - SAP Community
   - SAP Help Portal
   - Guided Answers
4. In the *Overview* tab, choose *Get Support* to create an incident (issue).
5. Select the relevant customer name.
6. For *Component*, enter: **`BC-CP-CF-BLDP`**

7. Fill in the mandatory fields.
8. Explain your problem. See section **Initial Problem-Related Data**.

Initial Problem-Related Data

When creating an incident, please provide the following information, which will help the support team to investigate and resolve your issue faster:

1. Type of buildpack – **Java**, **Python**, or **Node.js**
2. Environment – **Cloud Foundry** or **XSA**
3. Description of the issue
4. Steps to reproduce the issue
5. Expected result
6. Actual result
7. Logs in a **.txt** file. To collect the latest logs, run the respective command.
   - For Cloud Foundry, run:

     ```
     cf logs --recent <app_name>
     ```

   - For XSA, run:

     ```
     xs logs --recent <app_name>
     ```

   > ⓘ Note
   >
   > If possible, collect logs shortly after failure.

8. Specific files per buildpack (language), respectively:
   - Java: `manifest.yml` or `MTA` (if applicable)
   - Node.js: `manifest.yml` **and** `package.json`
   - Python: `manifest.yml`, `runtime.txt`, and `requirements.txt`

See also: Get Support Application

4.1.3.6 Developing Multitenant Applications in the Cloud Foundry Environment

In the Cloud Foundry environment, you can develop and run multitenant applications, and share them with multiple consumers simultaneously on SAP BTP.

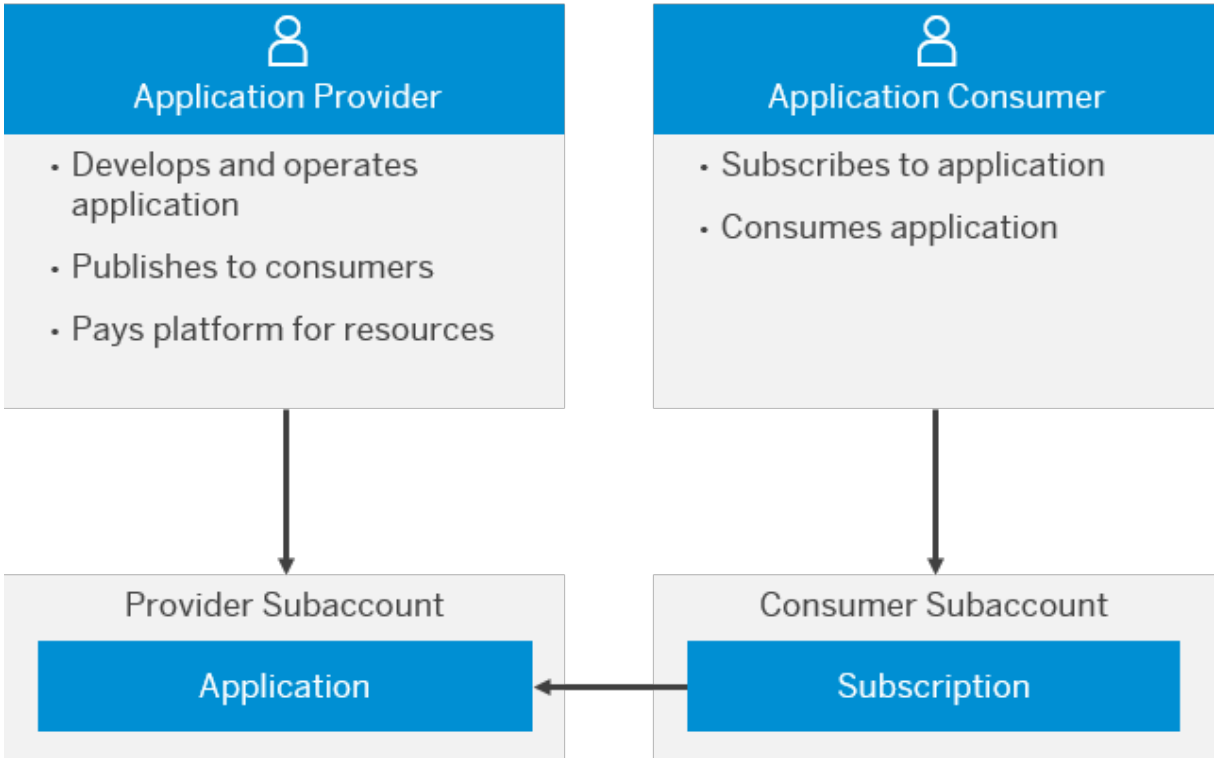
What is Multitenancy?

SAP BTP provides a multitenant functionality that allows application providers to own, deploy, and operate tenant-aware applications for multiple consumers, with reduced costs. For example, the application provider

can upgrade the application for all your consumers instead of performing each update individually, or can share resources across multiple consumers. The application consumers launch the applications using consumer-specific URLs, and can configure certain application features.

With tenant-aware applications, you can:

- Separate data securely for each tenant
- Save resources by sharing them among tenants
- Update applications efficiently, in a single step



Consumers cannot see the data of other consumers; the multitenant application maintains data separation between tenants, and SAP Customer Identity and Access Management (SAP CIAM) is kept isolated. Each consumer accesses the multitenant application through a dedicated URL.

Multitenancy Roles

The multitenancy concept involves two main user roles:

Application provider	An SAP global account owner that uses SAP BTP to own, build, run, and offer custom-developed applications to its consumers.
Application consumer	A consumer of the application provider, such as a department or organizational unit, whose users use the multitenant application.

How Does Multitenancy Work for the Application Consumer?

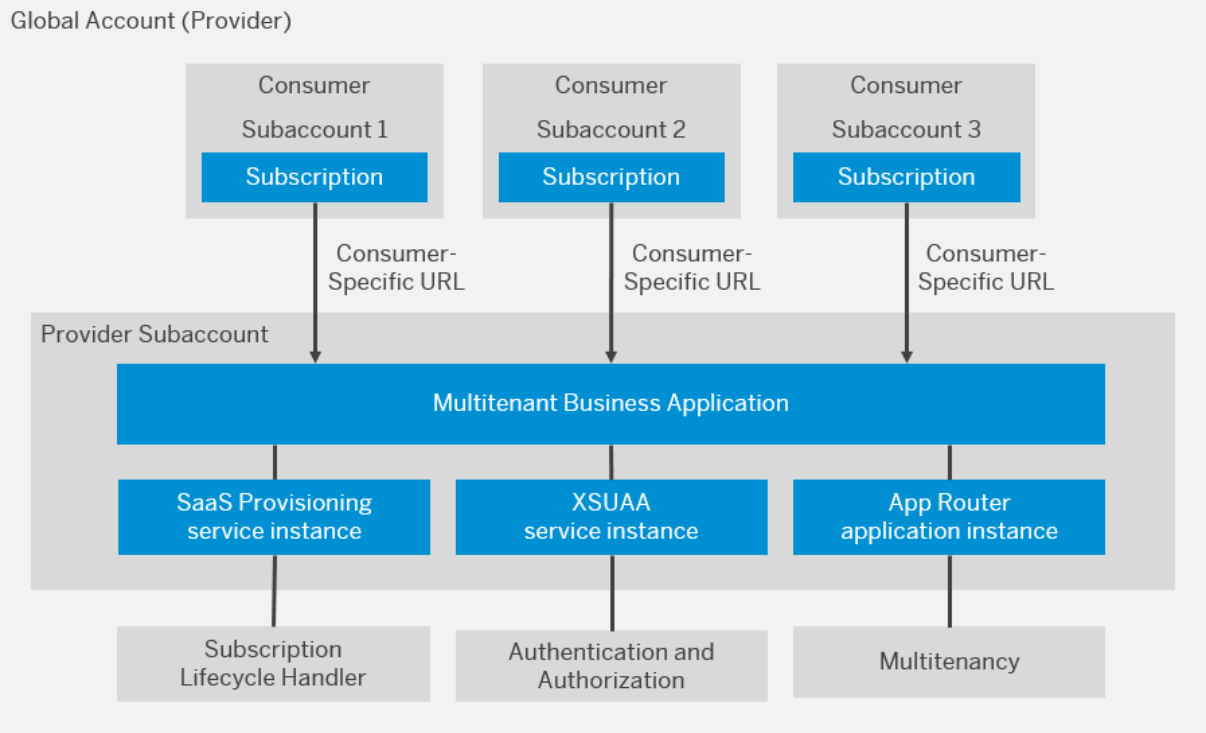
For a consumer to use a tenant-aware application on SAP BTP, the application owner must ensure that each consumer:

1. Has a dedicated subaccount in the application provider's global account.
2. Subscribes to the application using either the SAP BTP cockpit, SAP BTP command-line interface, or a dedicated REST API.
   A subscription means that there is a direct relationship between an application provider and the consumer's tenant. The application provider authorizes the consumer tenant to use the application.
3. Receives a dedicated URL so that its business users can access the application
   As with any application running in SAP BTP, these multitenant applications consume platform resources, such as compute units, structured and unstructured storage, and outgoing bandwidth. The costs for these consumed resources, and those of the application consumer, are billed to the provider of the multitenant application.

For more information about these consumer-related steps, see Providing Multitenant Applications to Consumers in the Cloud Foundry Environment [page 419] and Subscribe to Multitenant Applications Using the Cockpit [page 2534].

When a consumer accesses the application, the application environment identifies them by their unique tenant ID. The application distinguishes between requests from different consumer tenants based on the tenant ID, thus ensuring data isolation.

The following figure illustrates the relationship between the application provider's subaccount and consumer subaccounts (tenants) in the provider's global account. You deploy the multitenant application to the provider subaccount, and subsequently the consumer subaccounts subscribe to the deployed application. The application uses the tenant-aware `approuter` application and `xsuaa` service (with the `application` plan) to authenticate business users of the application at runtime. The application is then registered with the SAP Software-as-a-Service Provisioning service (technical name: `saas-registry`) with the `application` plan, which makes the application available for subscription to consumers.



Workflow

This is the workflow and requirements for developing and deploying a multitenant application in the Cloud Foundry environment:

1. Develop the Multitenant Application [page 405]
   In the Cloud Foundry environment of SAP BTP, you can develop and run multitenant applications that can be accessed by multiple consumers (tenants) through a dedicated URL.
2. Deploy the Multitenant Application to the Provider Subaccount [page 408]
   After you have developed your multitenant application and authorizations, you need to deploy the application in a Cloud Foundry space in the provider's subaccount.
3. Configure the approuter Application [page 409]
   To authenticate business users of the application at runtime, use the tenant-aware approuter application and `xsuaa` service in SAP BTP.
4. Bind the Multitenant Application and Application Router to the xsuaa Service Instance [page 412]
   Bind your multitenant application and the approuter application to the `xsuaa` service instance, which acts as an OAuth 2.0 client to your application.
5. Register the Multitenant Application to the SAP SaaS Provisioning Service [page 413]
   To make a multitenant application available for subscription to SaaS consumer tenants, you (the application provider) must register the application in the Cloud Foundry environment via the SAP Software-as-a-Service Provisioning service (`saas-registry`).
6. Providing Multitenant Applications to Consumers in the Cloud Foundry Environment [page 419]
   Create a subaccount in your global account for each application consumer, subscribe each consumer subaccount to the hosted application deployed in the provider account, and then set up application roles and assign users.

Related Information

Using SAP SaaS Provisioning Service APIs to Manage Multitenant Applications [page 421]
Data Protection and Privacy [page 3633]
Using the SAP BTP Subscription Management Dashboard [page 427]

4.1.3.6.1 Develop the Multitenant Application

In the Cloud Foundry environment of SAP BTP, you can develop and run multitenant applications that can be accessed by multiple consumers (tenants) through a dedicated URL.

Context

When developing tenant-aware applications in the Cloud Foundry environment, keep in mind the following general programming guidelines:

- Shared in-memory data such as Java static fields will be available to all tenants.
- Avoid any possibility that an application user can execute custom code in the application JVM, as this may give them access to other tenants' data.
- Avoid any possibility that an application user can access a file system, as this may give them access to other tenants' data.
- To perform internal tenant onboarding activities, such as creating a database schema for tenants, you must implement the `Subscription` callbacks of the SAP Software-as-a-Service Provisioning service (`saas-registry`) and use the information provided in the subscription event. You can also implement the `getDependencies` callback to obtain the dependencies of any SAP reuse services by your application. See details in the procedure below.

Procedure

1. Develop the application as for any other Java and Node.js application on the SAP BTP.

   For more information, see Developing Java in the Cloud Foundry Environment [page 284].
2. In your application, you must implement a REST API that will be called by the SAP SaaS Provisioning service (`saas-registry`) on a subscription event. Optionally, you can implement the `getDependencies` REST API to obtain the dependencies of any SAP reuse services consumed by your application.

   When a consumer tenant creates or revokes a subscription to your multitenant application via the cockpit, the SAP SaaS Provisioning service calls the application with two `Subscription` callbacks. The first callback gets the dependencies of any reuse services used by your application. The second callback informs the application about the subscription event (`PUT` for subscribe and `DELETE` for unsubscribe) and provides details about the subscriber (the consumer tenant).

- To inform the application that a new consumer tenant wants to subscribe to the application, implement the callback service with the `PUT` method. The callback must return a 200 response code and a string in the body, which is the access URL of the tenant to the application subscription (the URL contains the subdomain). This URL is generated by the multitenant application based on the approuter configuration (see Configure the approuter Application [page 409])

  Sample Code

  Callback response (`PUT`).

  ```
  https://subscriber_subdomain-
  approuter_saas_app.cfapps.eu10.hana.ondemand.com
  ```

  The URL must be in the format: `https://<SUBSCRIBER_TENANT_SUBDOMAIN >-<APPROUTER_APPLICATION_HOST>.<CF_DOMAIN>`

  Note

  The first segment of the URL must not exceed 63 characters (DNS host label limit). Consider using shorter names or explicitly setting a custom route to avoid truncation. For details, see Routes [page 765].

- To inform the application that a tenant has revoked its subscription to the multitenant application and is no longer allowed to use it, implement the same callback with the `DELETE` method.

  Sample Code

  Payload of subscription `PUT` and `DELETE` methods:

  ```
  {
      "subscriptionAppId": "<value>",                    # The
  application ID of the main subscribed application.
                                                         # Generated by
  Authorization and Trust Management service(xsuaa)-based on the
  xsappname.
      "subscriptionAppName": "<value>"                   # The
  application name of the main subscribed application.
      "subscribedTenantId": "<value>"                    # ID of the
  subscription tenant
      "subscribedSubdomain": "<value>"                   # The subdomain
  of the subscription tenant (hostname for
                                                         # the
  identityzone).
      "globalAccountGUID": "<value>"                     # ID of the
  global account
      "subscribedLicenseType": "<value>"                 # The license
  type of the subscription tenant
  }
  ```

- If your application consumes any reuse services provided by SAP, you must implement the `getDependencies` callback to return the service dependencies of the application. The callback must return a 200 response code and a JSON file with the dependent services `appName` and `appId`, or just the `xsappname`.

  Note

  The JSON response of the callback must be encoded as either UTF8, UTF16, or UTF32, otherwise an error is returned.

Sample Code

Sample dependency response:

```
[
   {
      "xsappname" : "<value>"   # The xsappname of the reuse service
which the application consumes.
                                # Can be found in the environment
variables of the application:
                                #
VCAP_SERVICES.<service>.credentials.xsappname
   }
]
```

For more information about the subscription callbacks and dependencies, see Register the Multitenant Application to the SAP SaaS Provisioning Service [page 413].

3. Configure the application's authentication and authorization for the SAP HANA XS advanced Java run time.

   For general instructions, see SAP Authorization and Trust Management Service [page 3538].

4. Create a security descriptor file in JSON format that specifies the functional authorization scopes for the application:

   - Define the application provider tenant as a shared tenant:

     ```
     "tenant-mode":"shared"
     ```

   - Provide access to the SAP SaaS Provisioning service SAP Authorization and Trust Management service (technical name: `saas-registry`) for calling callbacks and getting the dependencies API by granting scopes:

     ```
     "grant-as-authority-to-apps" : [ "$XSAPPNAME(application,sap-
     provisioning,tenant-onboarding)"]
     ```

Sample Code

Security descriptor file in JSON format:

```
{
   "xsappname":"saas-app",
   "tenant-mode":"shared",
   "scopes":[
      {
         "name":"$XSAPPNAME.Callback",
         "description":"With this scope set, the callbacks for subscribe,
unsubscribe and getDependencies can be called.",
         "grant-as-authority-to-apps":[
            "$XSAPPNAME(application,sap-provisioning,tenant-onboarding)"
         ]
      }
      ....
   ],
   ....
}
```

5. In the Cloud Foundry space in the provider's subaccount where your application is going to be deployed (see Deploy the Multitenant Application to the Provider Subaccount [page 408]), create an `xsuaa` service

instance with the security configurations, which you defined in the security descriptor file in the previous step, by executing this command in the Cloud Foundry command line interface (cf CLI):

```
cf create-service xsuaa application <XSUAA_INSTANCE_NAME> -c
<XS_SECURITY_JSON>
```

Specify the following parameters:

Parameter	Description
`XSUAA_INSTANCE_NAME`	The new name for the `xsuaa` service instance. Use only alphanumeric characters, hyphens, and underscores.
`<XS_SECURITY_JSON>`	Name of the security descriptor file from the previous step.

{·} Sample Code

```
cf create-service xsuaa application xsuaa-application -c xs-security.json
```

→ Tip

You can also create the service instance directly in the cockpit.

For general instructions, see Create Spaces Using the Cloud Foundry Command Line Interface [page 2854].

4.1.3.6.2 Deploy the Multitenant Application to the Provider Subaccount

After you have developed your multitenant application and authorizations, you need to deploy the application in a Cloud Foundry space in the provider's subaccount.

Context

To make the application available to subaccounts in multiple regions, you need to deploy the application to a provider subaccount in each of the regions.

Procedure

1. Create a subaccount in the Cloud Foundry environment in the global account of the application provider. This subaccount will host the multitenant application for the provider tenant.

   For general instructions, see Create a Subaccount [page 2509].

2. In the subaccount, create a space in a Cloud Foundry organization.

   For general instructions, see Create Spaces [page 2800].
3. Assign the appropriate quota for the application runtime memory to the provider's subaccount.

   For general instructions, see Configure Entitlements and Quotas for Subaccounts [page 2529].
4. Deploy the application to the Cloud Foundry space and start it by executing the `push` command in the command line interface (cf CLI).

   For more information, see Deploy Business Applications in the Cloud Foundry Environment [page 845].

4.1.3.6.3 Configure the approuter Application

To authenticate business users of the application at runtime, use the tenant-aware `approuter` application and SAP Authorization and Trust Management service (technical name: `xsuaa`) in SAP BTP.

Context

You deploy the `approuter` application as a Cloud Foundry application and as a logical part of the multitenant application. Then you configure `approuter` application as an external access point of the application. You need to deploy a separate application router for each multitenant application.

How does the application router and multitenancy work?

When a consumer accesses the application, their consumer tenant calls the multitenant application via the application router with their tenant-specific URL (`cf` route).

- **During a development phase** of your multitenant application, the following URL structure applies:
  `<SUBSCRIBER_TENANT_SUBDOMAIN>-<APPROUTER_APPLICATION_HOST>.<SAP-PROVIDED_STANDARD_DOMAIN>`
  In each development landscape, SAP provides a different standard domain you can use to create URLs. Using that domain, and following the specified URL structure, a created URL would be:
  `tenant1-myapprouter.cfapps.sap.hana.ondemand.com`
  Where: `tenant1-myapprouter` is the hostname, and `cfapps.sap.hana.ondemand.com` is the SAP-provided standard domain for this development landscape.
  In this format, create a new URL for each new tenant.
- **During a production phase** of your multitenant application, apply for a custom domain via the custom domain service, then create a URL using the following structure:
  `*.<YOUR_CUSTOM_DOMAIN>`.
  Where: * is the wildcard hostname.
  In this format, since the wildcard is replaced by an actual tenant during runtime, there is no need to create a new URL for each new tenant.
  For more information, see Using Custom Domains [page 2878]

In both cases, the application router then derives the tenant from the URL and calls the tenant-aware `xsuaa` (containing the user account and authentication service), which is responsible for the authentication of the business user. The `xsuaa` reads the tenant and gets the customer-specific identity provider (IdP) from the tenant-specific identity zone. Then, the `xsuaa` delegates the authentication to the configured IdP, and creates

a JSON Web Token (JWT) that contains the tenant, the current user, and the authorization scopes of the user. The JWT is then sent back to the application router, and from there to the application.

To read and store tenant-specific data, the multitenant application needs to know the tenant ID. To read the tenant ID, use the Container Security API to enable the multitenant application to read the tenant, user, and scopes from the given JWT. The API also validates whether the JWT is valid. The tenant information is contained in the `identityZone` user attribute.

Sample Code

Java code for retrieving the identity zone

```
UserInfo userInfo = SecurityContext.getUserInfo();
String identityZone = userInfo.getIdentityZone();
```

For more information, see:

- Web Access Control [page 3548]
- What Is the SAP Authorization and Trust Management Service? [page 3545]

Procedure

1. Create the application structure of the application router and configure it accordingly.

   For general instructions, see Application Router [page 453].
2. Configure the application router with the destination of your multitenant application.

   If you are defining the destination as an environment variable for the `approuter` application, set the router information in the `env: destinations` section of the `manifest.yml`.

   - `url`: Specify the URL of the multitenant application.
   - `forwardAuthToken`: Set to `true`

   ### Sample Code

   manifest.yml for a development phase:

   ```
   ---
   applications:
   - name: approuter-saas-app
     host: approuter_saas_app
     path: approuter
     buildpack: nodejs_buildpack
     memory: 256M
     env:
       TENANT_HOST_PATTERN: "^(.*)-cfapps.eu10.hana.ondemand.com
       destinations: >
         [
           {"name":"saas-application",
            "url":"https://backend-saas-app.cfapps.sap.hana.ondemand.com",
            "forwardAuthToken": true}
         ]
   ```

Sample Code

manifest.yml for a production phase:

```
---
applications:
- name: approuter-saas-app
  host: approuter_saas_app
  path: approuter
  buildpack: nodejs_buildpack
  memory: 256M
  env:
    TENANT_HOST_PATTERN: "^(.*).mydomain.com
    destinations: >
      [
        {"name":"saas-application",
         "url":"https:// backend-saas-app.mydomain.com",
         "forwardAuthToken": true}
      ]
```

For more information, see Application Routes and Destinations [page 496].

3. Configure the routes in the application router security descriptor file (`xs-app.json`) so that application requests are forwarded to the multitenant application destination.

Sample Code

xs-app.json:

```
{
  "routes": [{
    "source": "/",
    "target": "/",
    "destination": "saas-application"
  }]
}
```

For more information, see Routing Configuration File [page 464].

4. Use the `push` command in the cf CLI to deploy the `approuter` application to the Cloud Foundry space where your multitenant is deployed.

4.1.3.6.4 Bind the Multitenant Application and Application Router to the xsuaa Service Instance

Bind your multitenant application and the `approuter` application to the SAP Authorization and Trust Management service (technical name: `xsuaa`) instance, which acts as an OAuth 2.0 client to your application.

Procedure

Execute the following commands in the Cloud Foundry command line interface to bind both your multitenant application and the `approuter` application to the `xsuaa` service instance.

```
cf bind-service <APP_NAME> <XSUAA_INSTANCE_NAME>
```

```
cf bind-service <APPROUTER_APP_NAME> <XSUAA_INSTANCE_NAME>
```

Specify the following parameters:

Parameter	Description
APP_NAME	The name of the deployed multitenant application.
XSUAA_INSTANCE_NAME	The name of the `xsuaa` service instance you created in Develop the Multitenant Application [page 405].
APPROUTER_APP_NAME	The name of the `xsuaa` application you deployed in Configure the approuter Application [page 409].

Sample Code

```
cf bind-service saas-app xsuaa-application
cf bind-service approuter-saas-app xsuaa-application
```

→ Tip

You can also bind the application to the `xsuaa` service instances directly in the SAP BTP cockpit.

4.1.3.6.5 Register the Multitenant Application to the SAP SaaS Provisioning Service

To make a multitenant application available for subscription to SaaS consumer tenants, you (the application provider) must register the application in the Cloud Foundry environment via the SaaS Provisioning Service (technical name: `saas-registry`).

Context

The SaaS Provisioning Service allows application providers to register multitenant applications and services in the Cloud Foundry environment in SAP BTP. This activity is a one-time procedure per multitenant application.

Procedure

1. Create a service instance of the SaaS Provisioning Service with a configuration JSON file in the Cloud Foundry space where the multitenant application is deployed.

   > → Tip
   >
   > You can create the SaaS Provisioning Service instance directly in the SAP BTP cockpit.

   For general instructions, see Creating Service Instances.

   a. The configuration JSON file must follow the following format and set these properties (note that instead of `xsappname` you can use `appId`):

```
{
   "xsappname" : "<xsappname>",
   "appUrls": {
      "getDependencies" : "<getDependenciesUrl>",
      "onSubscription" : "<onSubscriptionUrl>/{tenantId}"
   },
    "appPlans": [
      {
        "name": "...",
        "description": "..."
       "allowedTargetPlans": "..."
      },{
        ...
      }
    ],
   "displayName" : "<displayName>",
   "description" : "<description>",
   "category" : "<category>",
   "onSubscriptionAsync": true/false,
   "callbackTimeoutMillis": <int>
   "allowContextUpdates": true/false,
}
```

Specify the following parameters:

Parameters	Description
`xsappname`	The `xsappname` configured in the security descriptor file used to create the `xsuaa` instance (see Develop the Multitenant Application [page 405]).

`getDependencies`	(Optional) Implement a callback API that is executed by the SaaS Provisioning Service to retrieve your application's dependencies.
	
	Note
	Implement only if your application has dependencies.
	
	API Structure
	• Request:
	  • GET
	  • URL: Define the endpoint to which SaaS Provisioning Service executes the request. For example: `https://<app url>/callback/v1.0/dependencies`
	• Response:
	  • HTTP Status Code: 200 (Ok)
	  • Header: `content-type: application/json`
	  • Response Body:
	  Specify the `xsappname` of each of your dependencies.
	
	`{ "xsappname": "..." # The xsappname of the reusable service instance that the application consumes. Find it in the following environment variable of your application: <VCAP_SERVICES.<service>.credentials.xsappname> }, ....`
	
	Note
	The JSON response of the callback must be encoded as either UTF8, UTF16, or UTF32, otherwise an error is returned.

`onSubscription`	Any URL that the application exposes via **PUT** and **DELETE** subscription. It must end with `/{tenantId}`. The tenant for the subscription is passed to this callback as a path parameter. You must keep `{tenantId}` as a parameter in the URL so that it's replaced at real time with the tenant calling the subscription. This callback URL is called when a subscription between a multitenant application and a consumer tenant is created (**PUT**) and when the subscription is removed (**DELETE**).

Parameters	Description
`appPlans`	(Optional) Specify the plan for your application. Name it and provide a short description. With the help of plans, you can try out your application in different modes or variations. For example, an app for staging or development purposes. Note that the plans are only visible in the subaccounts that belong to the provider global account. With **`allowedTargetPlans`**, you can specifiy a list of plans to which the existing subscription can switch. This includes plans for upgrades, downgrades, and other types of transitions.
`displayName`	(Optional) The display name of the application when viewed in the cockpit. For example, in the application's tile. If left empty, takes the application's technical name.
`description`	(Optional) The description of the application when viewed in the cockpit. For example, in the application's tile. If left empty, takes the application's display name.
`category`	(Optional) The category to which the application is grouped in the *Subscriptions* page in the cockpit. If left empty, gets assigned to the default category.
`onSubscriptionAsync`	Whether the subscription callback is asynchronous. If set to true, `callbackTimeoutMillis` is mandatory.
`callbackTimeoutMillis`	The number of milliseconds the SaaS Provisioning Service waits for the application's subscription asynchronous callback to execute, before it changes the subscription status to `FAILED`.
`allowContextUpdates`	Whether to send updates about the changes in contextual data for the service instance. For example, when a subaccount with which the instance is associated is moved to a different global account. Defaut value is false.

Sample Code

Configuration JSON file (example: config.json):

```
{
   "xsappname":"saas-app",
   "appUrls": {
      "getDependencies" : "https://saas-
app.cfapps.eu10.hana.ondemand.com/callback/v1.0/dependencies",
      "onSubscription" : "https://saas-
app.cfapps.eu10.hana.ondemand.com/callback/v1.0/tenants/{tenantId}"
   },
   "displayName" : "Hello World",
   "description" : "My multitenant biz app",
   "category" : "Test"
}
```

b. Execute the following cf CLI command to create the service instance with the JSON configuration file:

```
cf create-service saas-registry application
<SAAS_REGISTRY_SERVICE_INSTANCE> -c <JSON_CONFIG_FILE>
```

Specify the following parameters:

Parameters	Description
SAAS_REGISTRY_SERVICE_INSTANCE	The new name for your service instance. Use only alphanumeric characters, hyphens, and underscores.
JSON_CONFIG_FILE	The file name of the service-specific configuration parameters, in a valid JSON object (described above).

Sample Code

```
cf create-service saas-registry application saas-registry-application
-c config.json
```

2. To ensure that the application trusts the JWT issued for another identity zone (`sap-provisioning`), add the environment variable: `SAP_JWT_TRUST_ACL`

```
SAP_JWT_TRUST_ACL:'[{"clientid":"*","identityzone":"sap-provisioning"}]'
```

Note

The client libraries (java-security, spring-xsuaa, container security api for node.js >= 3.0.6, approuter >= 8.x, and sap_xssec for Python >= 2.1.0) have been updated. When using these libraries, setting the parameter `SAP_JWT_TRUST_ACL` has become obsolete.

Instead, the `xsuaa` service adds audiences to the issued JSON Web Token audience field (`aud`). Client libraries provide an audience validator to check whether the token is issued for your service or application. Technically, this information is implicitly derived from the audience field (`aud`) or from the scopes.

Therefore, this update comes with a change regarding scopes. For a business application A that wants to call an application B, it's now mandatory that application B grants at least one scope to the calling business application A. Furthermore, business application A has to accept these granted scopes or authorities as part of the application security descriptor. You can grant scopes with the `xs-security.json` file.

For more information, see Application Security Descriptor Configuration Syntax [page 592].

3. Bind the SaaS Provisioning Service instance that you created to the multitenant application that you deployed in your Cloud Foundry space using the following cf CLI command:

```
cf bind-service <APP_NAME> <SAAS_REGISTRY_SERVICE_INSTANCE>
```

Specify the following parameters:

Parameters	Description
APP_NAME	The ID of your deployed multitenant application.
SAAS_REGISTRY_SERVICE_INSTANCE	The name of the SaaS Provisioning Service instance you created in step 1 above.

{·} Sample Code

```
cf bind-service saas-app saas-registry-application
```

→ Tip

You can bind your application to the SaaS Provisioning Service instance directly in the SAP BTP cockpit.

For general instructions, see Binding Service Instances to Applications [page 276].

4. To ensure that the environment variables in the application take effect, execute the following cf CLI command:

```
cf restage <APP_NAME>
```

Specify the following parameter:

Parameters	Description
APP_NAME	The ID of your deployed multitenant application.

{·} Sample Code

```
cf restage saas-app
```

Next Steps

1. In your global account, create a subaccount for each consumer tenant.
2. Subscribe your consumer tenants to the multitenant applications using the cockpit.
3. Test your multitenant application from consumer tenants to ensure availability, subscription lifecycle, and optional application configurations.
4. Supply your customers or users of the multitenant application with their consumer-specific URL to access the application.

For more information, see Providing Multitenant Applications to Consumers in the Cloud Foundry Environment [page 419].

4.1.3.6.5.1 Unregister a Multitenant Application from the SAP SaaS Provisioning Service

Unbind and delete the service instance you have created during the registration of your multitenant application to complete the unregistration process.

Prerequisites

Make sure that no tenant is subscribed to your application before you initiate the deletion of the service instance.

> → Tip
>
> Use the `Get Application Subscriptions` API to get the application subscriptions.
>
> In case there are subscribed tenants, unsubscribe them first by calling the `Unsubscribe the tenant from an application` API.
>
> For more information, see Using SAP SaaS Provisioning Service APIs to Manage Multitenant Applications [page 421].

Context

The procedure described in this section uses the cf CLI.

> → Tip
>
> You can delete a service instance directly in the SAP BTP cockpit, in the *Service Instances* page.

Procedure

1. Use cf CLI to remove any existing service keys and app bindings:
   a. Run the following command to delete the service key:

      ```
      cf delete-service-key SERVICE-INSTANCE-NAME KEY-NAME
      ```

      > ⓘ Note
      >
      > Service key is an optional parameter.
      >
      > In case you haven't created a service key during the registration process, skip to substep b.

b. Run the following command to unbind your application from the service instance:

```
cf unbind-service APP-NAME SERVICE-INSTANCE-NAME
```

c. Run the following command to delete the service instance:

```
cf delete-service SERVICE-INSTANCE-NAME
```

2. The deletion process is asynchronous. Run `cf services` to view the current status of the process.

4.1.3.6.6 Providing Multitenant Applications to Consumers in the Cloud Foundry Environment

Once you have built a multitenant application in the Cloud Foundry environment using SAP BTP, you can then share the application with multiple consumers, such as business units in your organization.

Consumer Subaccounts and Subaccount Members

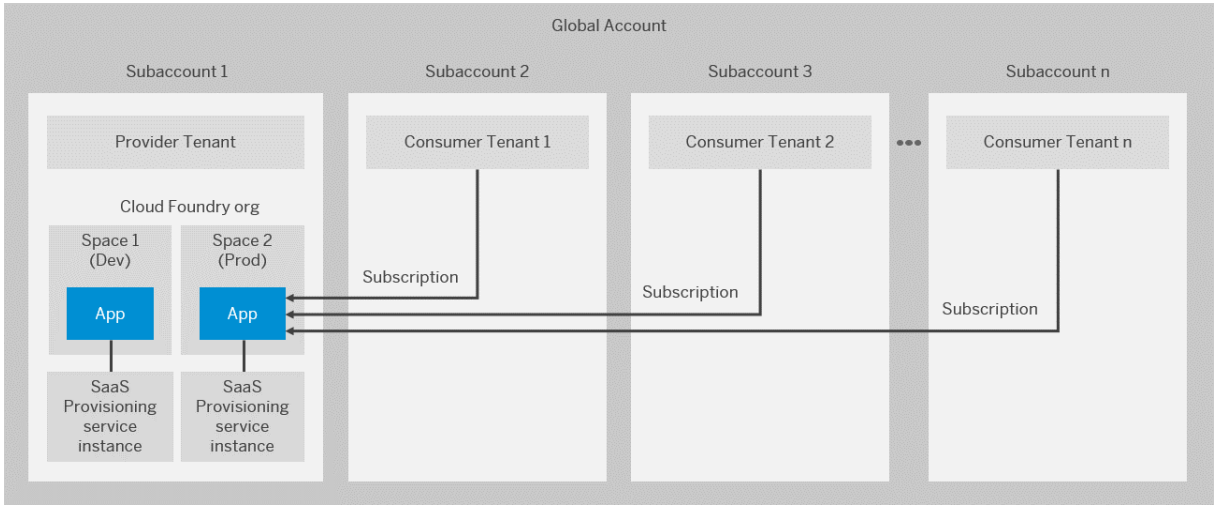
To use multitenancy in the Cloud Foundry environment in SAP BTP, the application provider must create a subaccount for each consumer in the same global account where the multitenant application is deployed.

The multitenant application is deployed to one subaccount only, which is the hosting subaccount of the application provider. Consumers simply subscribe to the provider application from their subaccount through the SAP BTP cockpit, the SAP BTP command line interface, or dedicated REST APIs.

The application provider is responsible for:

- Creating and managing the tenant subaccounts.
- Setting up the necessary role access authorizations.
- Subscribing each tenant to the application.

Typically, consumers of multitenant applications provided by a non-SAP application owner, such as an IT department of an organization, do not own or have access to their tenant subaccount.



Account setup for an application owner provisioning a multitenant application to its consumers.

Subaccount members are users who are registered via the SAP ID service. Subaccount members may have different privileges regarding the operations that are possible for a subaccount (for example, subaccount administration, deploy, start, and stop applications).

The subaccount-specific configuration allows application providers to adapt the consumer subaccounts to their specific needs.

What Do You Need to Do, and How?

Once you have developed, deployed, configured, and registered your multitenant application, your account administrator is ready to do the following.

1. Create a subaccount in your global account for each application consumer.
   See Create a Subaccount [page 2509].
2. Subscribe each consumer subaccount to the hosted application deployed in the provider account.
   See:
   - Subscribe to Multitenant Applications Using the Cockpit [page 2534]
   - Working with Multitenant Applications Using the BTP CLI [page 2704]
   - Subscribe the tenant to an application using the REST API. For more information, see Using SAP SaaS Provisioning Service APIs to Manage Multitenant Applications [page 421]
3. Set up application roles and assign users.
   See Working with Role Collections [page 2612].

Viewing and Managing of Your Multitenant Application's Subscriptions

Once you've provided your multitenant application to consumers to subscribe to it, you can view and manage all the subscriptions by using a dedicated interface called SAP BTP subscription management dashboard.

For more information, see Using the SAP BTP Subscription Management Dashboard [page 427].

4.1.3.6.7 Using SAP SaaS Provisioning Service APIs to Manage Multitenant Applications

Use the SaaS Provisioning Service (technical name: `saas-registry`) APIs to work with multitenant applications.

Background

Most of the SaaS Provisioning Service APIs are concentrated into two main groups:

1. APIs for Application Providers
2. APIs for Application Consumers

Application providers can use both groups to either manage their application subscriptions from the global account in which they deployed their multitenant application when using the first group, or mimic their consumers in working with the application from a different global account than the one in which their application is deployed, when using the second group.

Consumers can use the second group to work with the multitenant applications.

All SaaS Provisioning Service APIs adhere to the rate limiting rules as described in Rate Limiting [page 2759].

APIs for Application Providers

> → Tip
>
> Instead of APIs, can use a dedicated user interface with the same capabilities to manage your multitenant application's subscriptions.
>
> For more information, see Using the SAP BTP Subscription Management Dashboard [page 427].

Prerequisites

- You have a global account and have deployed a multitenant application in it.
- You've registered your multitenant application to the SaaS Provisioning Service. For more information, see Register the Multitenant Application to the SAP SaaS Provisioning Service.

Preparation

1. For an existing instance of `saas-registry` service with the `application` plan, create a service key or service binding if you don't already have one. For more information, see Service Bindings.
2. From the created key or service binding, extract the values of the following fields:
   - `url`
   - `clientid`
   - `clientsecret`
   - `saas_registry_url`

You need the first three values to authenticate your SaaS Provisioning Service API requests and the last one as base URL in each API call.

Authentication

To authenticate, obtain an OAuth token by calling a POST API with the acquired parameters in any REST API tool or directly in your code. For example:

```
curl -XPOST '<url>/oauth/token?
grant_type=client_credentials&client_id=<clientid>' -H 'Content-Type:
application/x-www-form-urlencoded' -H 'Authorization: Basic <encodedString>'
```

where:

`<encodedString>` is the result of base64 encoding `<clientid>:<clientsecret>`.

Available APIs

Go to SAP Business Accelerator Hub and select the **Application Operations for App Providers** section to get information about all the available APIs and their details.

You can also try out the APIs in the SAP Business Accelerator Hub.

For more information, see Trying Out APIs.

> **Note**
>
> To authenticate to try out the APIs in SAP Business Accelerator Hub, configure an environment.
>
> During the environment configuration, select the *OAuth 2.0 Application Flow* option from the dropdown menu. You must do so for the Application Provider API group because these specific APIs only support the Client Credentials authentication type.
>
> For more information about configuring an environment in general, see Trying Out APIs in a Configured Environment.

APIs for Application Consumers

Prerequisites

- You've created an instance of Cloud Management (technical name: `cis`) service with `local` plan.

Authentication

To learn how to authenticate, see Getting an Access Token for SAP Cloud Management Service APIs.

Available APIs

Go to SAP Business Accelerator Hub and select the **Subscription Operations for App Consumers** section to get information about all the available APIs and their details.

You can also try out the APIs in the SAP Business Accelerator Hub. For more information, see Trying Out APIs.

To authenticate to try out the APIs in SAP Business Accelerator Hub, configure an environment.

For more information about configuring an environment in general, see Trying Out APIs in a Configured Environment.

4.1.3.6.7.1 Getting an Application Access Token

Use this API to get the application access token from the SAP Software-as-a-Service Provisioning service instance.

Prerequisites

You've registered your multitenant application to the SAP SaaS Provisioning service.

For more information, see Register the Multitenant Application to the SAP SaaS Provisioning Service [page 413].

Obtaining API Request Parameters

To get the token, you call the API with the parameters obtained from the service binding object you created during the registration of your multitenant application to the SAP SaaS Provisioning service.

Use the following cf CLI command to get the service binding object:

```
cf env <APP_NAME>
```

See the step 3 in the Register the Multitenant Application to the SAP SaaS Provisioning Service [page 413] to find the `<APP_NAME>`.

The example of the binding object you get after executing the cf CLI command, with the needed values for the API (`clientid`, `clientsecret`, and `url`) marked in bold:

Sample Code

```
"saas-registry": [
{
"credentials": {
"apiurl": "https://api.authenticat********avlab.ondemand.com",
"appName": "sample-saas-ap********-45",
"appUrls": "{\"getDependencies\":\"http**********.cf.stag**":0}",
"clientid": "sb-sample-saas-********************-broker!b4",
"clientsecret": "riH************0=",
"description": "Sample multitenant application",
"display_name": "Sample multitenant application",
"identityzone": "cfs******44",
"saas_registry_url": "https://saa********ab.ondemand.com",
"sburl": "https://internal-xsu********ndemand.com",
"subaccountid": "3358efc9-*********10b456",
"tenantid": "34584*******711",
"url": "https://cfs-3035-7**********avlab.ondemand.com",
"xsappname": "sample-saa******istry-broker!b4",
```

```
"zoneid": "34584*******f499711"
},
"instance_name": "saas-app-saas-registry",
"label": "saas-registry",
"name": "saas-app-saas-registry",
"plan": "application",
}
]
```

The token you get after executing the API is a JSON Web Token (JWT).

For more information, see JSON Web Token (JWT) .

You use this token to manage the SAP SaaS Provisioning service APIs.

> **⚠ Restriction**
>
> Only the POST method is supported.

Request

URI: `<THE URL OBTAINED FROM THE BINDING OBJECT "URL" FIELD>/oauth/token`

HTTP Method: *POST*

Request Headers

Header	Required	Values
`Content-Type`	Yes	`<application/x-www-form-urlencoded>`
`Authorization`	Yes	Basic `<encodedString>` where `<encodedString>` is the result of base64 encoding the OAuth client's values as `clientId:clientSecret` that you obtained from the binding object as described in the previous section. For more information about the base64 encoding, see Base64

Request Parameters

Parameter	Required	Data Type	Description	Parameter Type
`grant_type`	Yes	String	The type of the authorization that is supported by the authorization server.	Authorization protocol

Parameter	Required	Data Type	Description	Parameter Type
			Set it to `client_credentials`.	
`client_id`	No	String	The ID of the client associated with the `SaaS Provisioning` service instance. Obtained from the service binding object. See the section Obtaining API Request Parameters of this document for details.	Relative URL path or JavaScript source code

Request Example (curl for Mac OS)

Sample Code

```
curl --location --request POST '<url>/oauth/token' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--header 'Authorization: Basic <base64.encoded(client_id:client_secret)>' \
--data-urlencode 'client_id=<client_id>' \
--data-urlencode 'grant_type=client_credentials'
```

Request Example (curl for Windows OS)

Sample Code

```
curl --location --request POST "<url>/oauth/token" ^
--header "Content-Type: application/x-www-form-urlencoded" ^
--header "Authorization: Basic <base64.encoded(client_id:client_secret)>" ^
--data-urlencode "client_id=<client_id>" ^
--data-urlencode "grant_type=client_credentials"
```

Response

Generates the access token for a multitenant application.

Content Type: *JSON*

Response Headers

Header	Values
`Content-Type`	<application/json;charset=UTF-8>

Response Status and Error Codes

Code	Description
200	Access token created successfully.

Response Properties

Property Name	Property Type	Description
`access_token`	JWT	Access token for the multitenant application. This is the value for which you call the API .
`token_type`	String	The type of access token issued.
`expires_in`	Number	The number of seconds until the access token expires.
`scope`	String	A space-delimited list of scopes that you authorized for the client.
`jti`	String	A globally unique identifier for JWT.

Response Example

Sample Code

```
HTTP/1.1 200 OK
Content-Type: application/json;charset=UTF-8
{
    "access_token": "eyJ***mh0d…",
    "token_type": "bearer",
    "expires_in": 43199,
    "scope": "uaa.re***.subscription.read",
    "jti": "df6cb84439a541fab33d5b7c298debe1"
}
```

Related Information

Using SAP SaaS Provisioning Service APIs to Manage Multitenant Applications [page 421]

4.1.3.6.8 Using the SAP BTP Subscription Management Dashboard

Learn how to use the SAP BTP subscription management dashboard to manage your multitenant applications through a user interface.

Context

SaaS and IAS application providers can now use the new subscription management dashboard to manage their multitenant application's subscriptions.

The tool is based on the SAP Software-as-a-Service Provisioning service (SaaS Provisioning) APIs that you can now work with using a user interface to simplify your operation and maintenance work.

Main Capabilities

You can do the following in the SAP BTP subscription management dashboard:

- View all subscriptions to your application and the information about them, such as their state and tenant ID.
- View each subscription's dependencies and status.
- Perform operations on subscriptions, for example update the list of dependencies.
- Control access to the dashboard using dedicated roles.

The dashboard also displays detailed information about any errors that arise during the subscription process. This gives you means to identify the source of issues and fix them.

Accessing the Dashboard in the SAP BTP Cockpit

Assign Roles

To access the subscription management dashboard, you must be assigned to one of these roles:

- **Subscription Management Dashboard Administrator** - Provides access to the dashboard with full read and write capabilities.
- **Subscription Management Dashboard Viewer** - Provides read-only access to the dashboard.

Add the relevant role to a role collection and assign yourself or another user to it.

> ⓘ Note
>
> You assign roles per tenants. To assign roles in a global account or subaccount tenant, you must be a global account or subaccount admin respectively.

For more information about roles, see Configure Application Roles and Assign Roles to Users.

> ### Note
>
> If you don't see the roles, you need to update the instance of the SaaS Provisioning Service (technical name: `saas-registry`) associated with your application (`application` plan). This enables the new roles.

Open the Dashboard

1. In the left-hand navigation bar of the SAP BTP cockpit, select *Services* > *Instances and Subscriptions*.

   > ### Note
   >
   > Only users who are assigned to one of the roles specified in the previous section can see the link to the dashboard and open it.

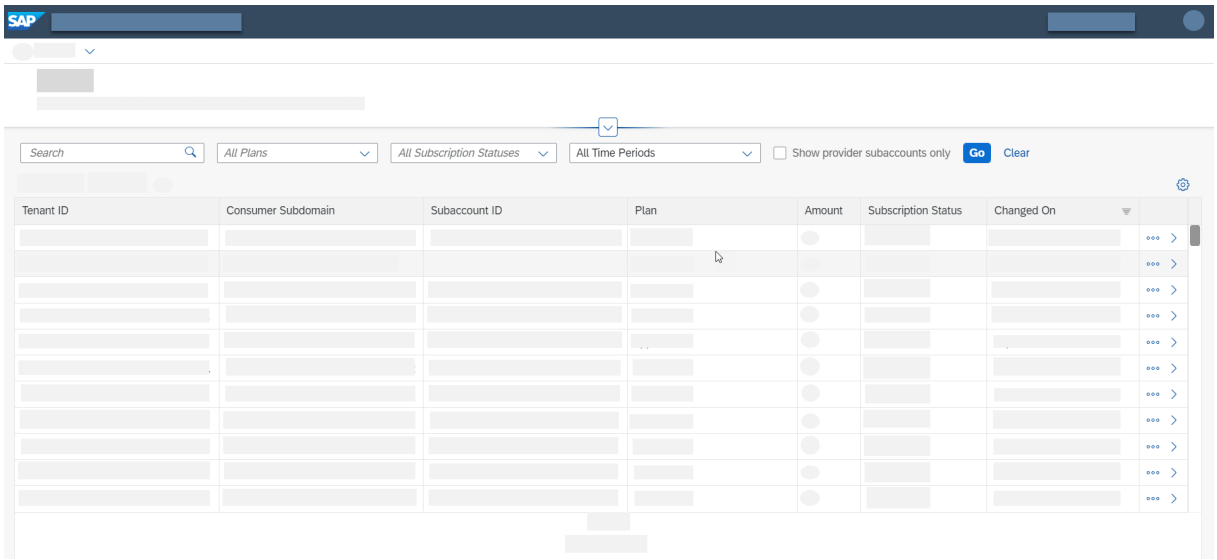
2. In the *Instances* section, find the instance of the SaaS Provisioning Service(technical name: `saas-registry`) with the `application` plan and click on it.
   Details about the instance open to the right.
3. In the top-right corner, select ••• and from the dropdown menu, choose *View Dashboard*.
   A new tab with the subscription management dashboard opens.

Using the Dashboard

The main screen shows a table with details about all of your application's subscriptions.

You can apply various search and filtering criteria such as plan, subscription status, and time periods to get a more specific list of subscriptions.

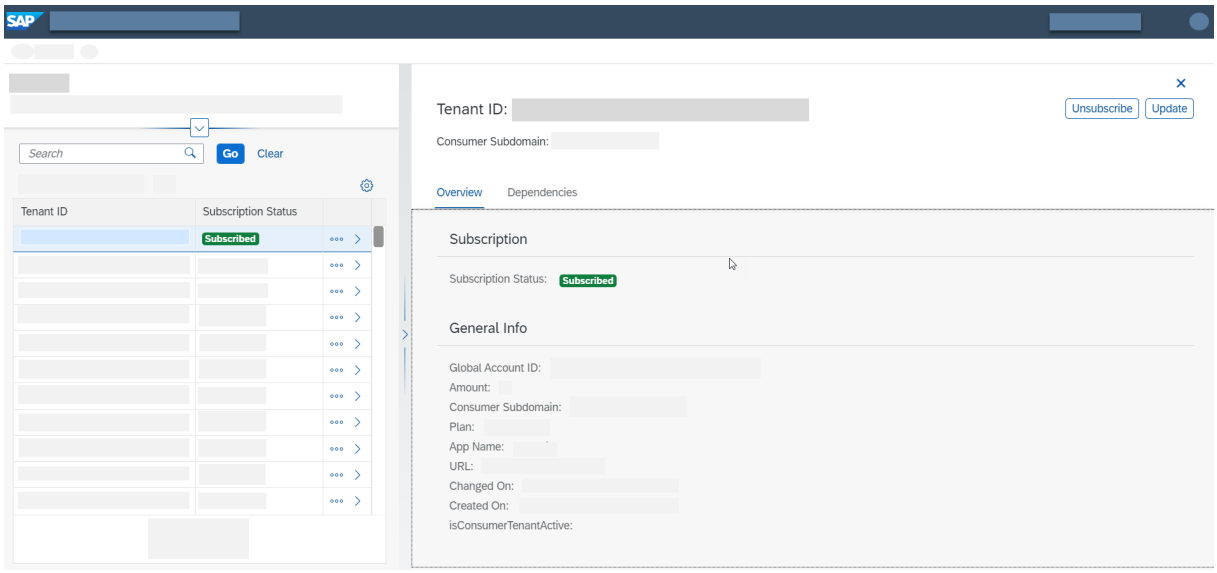
> ### → Tip
>
> Select the *Show provider subaccounts only* checkbox to see only the subaccounts that are in the global account in which your application is deployed. This can be useful because you can only create new subscriptions for these subaccounts.



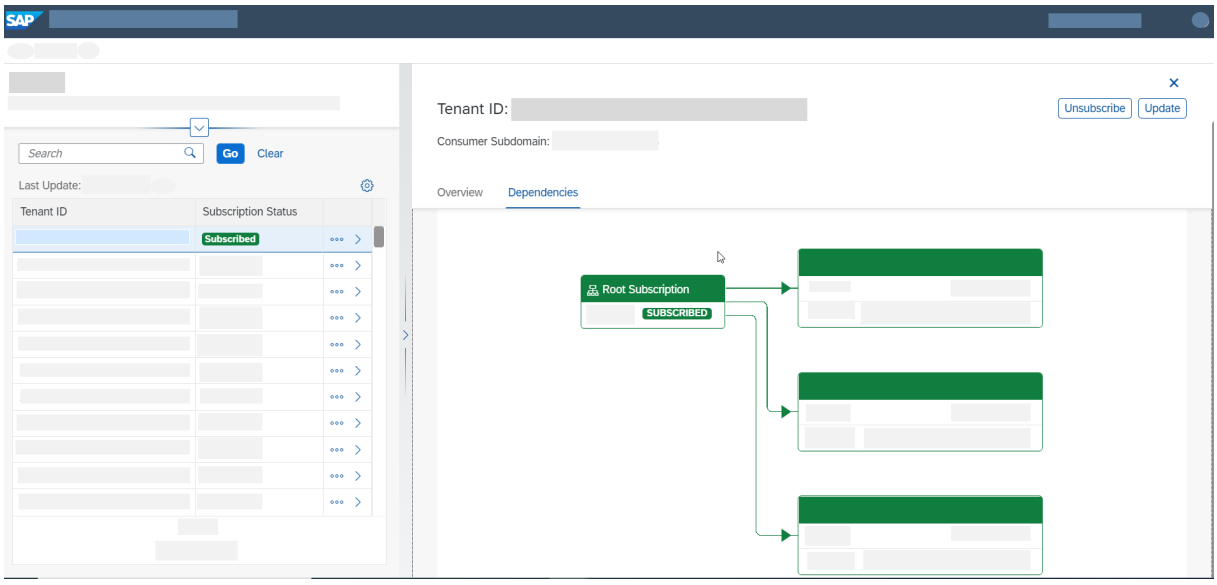
Clicking on a subscription row opens details about the subscription to the right where you can find more information about the subscription and analyze errors if the subscription status is `Failed`.

You can also perform additional operations, such as update subscription dependencies, if you're assigned to the `Subscription Management Dashboard Administrator` role. This operation updates the subscription with the latest list of dependencies (if an app provider has added or removed dependencies of a particular subscription, for example.)

In the main screen, administrators can also update multiple subscriptions at once. Select specific subscriptions to update, or update all of them.



You can choose to see your subscription dependencies in a tree or table view. See the following image for the tree view representation:



→ Tip

If you've more than one multitenant application, you can switch between them in the dashboard. Click on the dropdown menu in the upper-left corner and select the application for which you want to view or manage subscriptions.

4.1.4 Developing User Interface

Get information on user interface development in the SAP BTP, Cloud Foundry environment.

The key to a modern user interface is pairing great performance with a consistent and recognizable design across applications, platforms, and devices. To keep the user experience homogenous, you can establish a flexible implementation paradigm based on reusable components. Doing so enables extensibility and easier maintainability.

We know that it can be difficult to decide this up front. You can find information about UI development options in the SAP BTP, Cloud Foundry environment in the following sections.

Fiori Design Guidelines

The SAP Fiori Design Guidelines define design solutions that are easy to use, robust, scalable, and of enterprise-grade quality. The guidelines are independent of the implementation technology.

The modular design approach focuses on user tasks and workflows. It enables seamless transitions between applications and offers clear guidance for extension and integration design. To ensure a consistent user experience, we recommend following the Fiori Design Guidelines independent of the UI technology you plan on using.

SAPUI5

SAPUI5 is a JavaScript framework for building cross-platform, enterprise-grade web apps with modern, responsive, and rich user interfaces. For more information, see Developing SAPUI5 [page 431].

HTML5

On SAP BTP, you can access and run HTML5 applications in a cloud environment without the need to maintain your own runtime infrastructure. For more information, see Developing HTML5 Applications in the Cloud Foundry Environment [page 433].

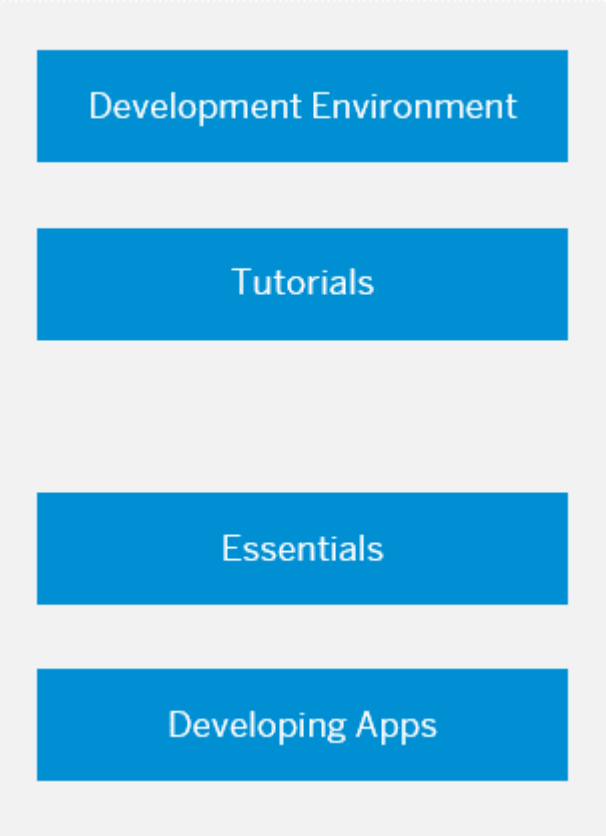
4.1.4.1 Developing SAPUI5

Get to know certain aspects of SAPUI5 development, to get up and running quickly.

If you are about to decide which UI technology to use , read everything you need to know about supported library combinations, the supported browsers and platforms, and so on, at the Read Me First section that contains the following topics and more:

- Supported Library Combinations
- Supported Combinations of Themes and Libraries
- Browser and Platform Support

Quick Start



- #unique_286/unique_286_Connect_42_subsection-im1 [page 432]
- #unique_286/unique_286_Connect_42_subsection-im2 [page 432]
- #unique_286/unique_286_Connect_42_subsection-im3 [page 432]
- #unique_286/unique_286_Connect_42_subsection-im4 [page 432]

Select the tiles to discover SAPUI5 Development. The references guide you to the documentation of the SAPUI5 Demo Kit. Besides the entry points we provide here on each tile, start exploring the demo kit on your own whenever you feel comfortable enough.

SAP Business Application Studio

Use the SAP Business Application Studio to develop on Cloud Foundry. You just need to create a dev space, which is a preconfigured development environment with the tools, capabilities, and resources needed for developing your application.

1. Register for an SAP BTP trial account at https://account.hanatrial.ondemand.com/ and log on afterwards.
2. Set Up SAP Business Application Studio in a trial account.
3. Create a SAP Fiori dev space.
4. Create a Project - either using the wizard or the terminal.

Learn how you can use SAP Business Application Studio together with UI5 Tooling to develop state-of-the-art UI5 apps.

Tutorials

Depending on the project template you're using you have a project that is more advanced than what you see in the following tutorials. Those tutorials show you the basics and you can start a new and empty project to follow along those SAPUI5 tutorials from the demo kit.

Have a look at the Get Started: Setup and Tutorials section that contains the following topics and more:

- "Hello World!"
- Data Binding
- Navigation and Routing
- Mock Server
- Worklist App
- Ice Cream Machine

Essentials

This is reference information that describes the development concepts of SAPUI5 , such as the Model View Controller, data binding, and components.

The following topics are excerpts from the Essentials section:

- Bootstrapping: Loading and Initializing
- Structuring: Components and Descriptor
- Model View Controller (MVC)
- Data Binding
- Reusing UI Parts: Fragments

Developing Apps

Create apps with rich user interfaces for modern Web business applications, responsive across browsers and devices, based on HTML5.

The following topics are excerpts from the Developing Apps section:

- App Templates: Kick Start Your App Development
- App Overview: The Basic Files of Your App
- App Initialization: What Happens When an App Is Started?
- Folder Structure: Where to Put Your Files
- Coding Issues to Avoid
- Securing Apps

4.1.4.2 Developing HTML5 Applications in the Cloud Foundry Environment

SAP BTP enables you to access and run HTML5 applications in a cloud environment without the need to maintain your own runtime infrastructure.

HTML5 applications consist of static content that runs on a browser. Then you develop your applications - either in SAP Business Application Studio, or in your own IDE (integrated development environment) - and deploy them to the HTML5 application repository [page 434].

Depending on your backend application setup, you either configure the destinations during development, or define them after deploying the application. Finally, to consume the applications, you can create a site using SAP Cloud Portal service or SAP Build Work Zone, build the URL, and define custom domains.

On the Cloud Foundry environment of SAP BTP you can run an application that was uploaded to HTML5 application repository using one of the following options:

- Use the application router that is managed by SAP.
  For more information, see Managed Application Router [page 534].
- Set up and maintain your own application router in your own space.
  For more information, see Application Router [page 453].

Both options allow you to serve static content from HTML5 application repository, authenticate users, rewrite URLs, and forward or proxy requests to other micro services while propagating user information. However, the managed application router [page 534] brings many benefits, such as:

- Simplify and speed up your development and deployment experience
- Save resources by running a serverless HTML5 application, which doesn't require any application runtime
- Lower maintenance efforts by leveraging the most up-to-date routing capabilities
- Meet the changing demand for HTML5 applications by automatically adjusting the service to maintain consistent and predictable performance
- Monitor and run your HTML5 applications from the *HTML5 Applications* UI in the SAP BTP cockpit

Therefore, we recommend running your own application router [page 453] only in advanced cases, for example when application router extensibility is required.

> **Note**
>
> When using the managed application router [page 534], you can monitor and run your HTML5 applications from the *HTML5 Applications* UI for your subaccount in the SAP BTP cockpit.

For monitoring and running HTML5 applications served by a standalone application router [page 453], you can use the CF HTML5 applications repository CLI plugin .

Related Information

HTML5 Application Repository [page 434]
Application Router [page 453]
Managed Application Router [page 534]

4.1.4.2.1 HTML5 Application Repository

HTML5 application repository enables central storage of HTML5 applications' static content on the SAP BTP, Cloud Foundry environment. This service can be consumed from the SAP BTP, Cloud Foundry runtime, the SAP BTP, Kyma runtime, Kubernetes, or by creating a service instance on the subaccount level regardless of an environment.

HTML5 applications consist of static content such as HTML, CSS, JavaScript, and other files, that run on a browser. For more information, see Basic Template and openui5-basic-template-app .

HTML5 application repository allows application developers to manage the lifecycle of their HTML5 applications. In runtime, the repository enables the consuming application, typically the application router, to access HTML5 application static content in a secure and efficient manner. For more information, see Configure Application Router.

Features

Zero Down-Time Enablement

- The HTML5 applications are decoupled from the consuming application router. This enables updating the static content of the HTML5 applications without restarting the application router.

Versioning and Authorization

- Exploration of HTML5 application content by version.
- Access control that is based on private or public authorization.
  When the HTML5 application is public, the service enables sharing this content with consuming application routers from different spaces.

Availability and Performance

- During runtime, the HTML5 application content is cached and optimized to provide high performance with minimal network load.
- The service provides several instances for a runtime to serve a high load of application requests.

Environment

This service is available in the following environments:

- Cloud Foundry
- Kyma

Multitenancy Support

This service supports multitenancy. It can be used by tenant-aware applications.

Technical Constraints

- The size of an application deployed to the repository is limited to 100 MB per service instance of the *app-host* service plan.
- Since the applications stored in HTML5 application repository can be shared, it is advised not to add personal data to them.
- You can perform up to 50 deployments per minute at design time per SAP BTP subaccount.
  If you exceed this rate limit, the HTTP error code 429 is returned.
- You can make up to 300 requests per second at runtime per SAP BTP subaccount.
  If you exceed this rate limit, the HTTP error code 429 is returned.

See Also

If you want to learn more about services in the SAP BTP, Cloud Foundry environment, see Using Services in the Cloud Foundry Environment [page 271].

4.1.4.2.1.1 Service Plans

The HTML5 Application Repository service comprises the following service plans:

- *app-host*
  Use this service plan to deploy HTML5 applications to the repository. For more information, see Deploying Content [page 436].
- *app-runtime*
  Use this service plan to consume HTML5 applications stored in the repository. For more information, see Consuming Content [page 446].

4.1.4.2.1.2 Deploying Content

Learn how to deploy, redeploy, and undeploy content from the HTML5 Application Repository.

In the Cloud Foundry environment, the Generic Application Content Deployer (GACD) is the preferred module for the application deployment. GACD deployment enables asynchronous uploads, making it more suitable for handling large content. This method helps mitigate the risk of network-related timeouts due to long response times. Additionally, with GACD, you do not need to deploy a Cloud Foundry application within your space. See Deploy Content Using the Generic Application Content Deployer [page 437].

In the Kyma environment, you use the HTML5 Application Deployer. See Deploy Content Using HTML5 Application Deployer [page 439].

The following applies to deployment with both modules:

> ### Note
>
> The HTML5 Application Repository service does not provide the technical capabilities to support the collection, processing, and storage of personal data. The file names used for deploying the HTML5 applications are not intended for personal data.

You use a service instance of the app-host service plan to deploy your applications to the HTML5 Application Repository.

> ### Note
>
> If you delete an app-host service instance, the apps that are deployed with this app-host service instance are deleted from the HTML5 Application Repository.

> ### Restriction
>
> Delta deployments are not supported.
>
> If you deploy new application content using an existing app-host service instance, the new application content will completely replace the old application content that had been previously deployed with this service instance in the HTML5 Application Repository.
>
> For example, if you have already deployed the applications a, b, and c with the instance 123 and now want to deploy the new applications d and e, you can do one of the following to avoid overwriting the applications a, b, and c:
>
> - Redeploy instance 123 with applications a, b, c, d, and e (with the new and the old applications).
> - Create a new instance 456 and use this instance to deploy the applications d and e.

Related Information

Deploy Content Using the Generic Application Content Deployer [page 437]
Deploy Content Using HTML5 Application Deployer [page 439]
Redeploy Content [page 444]
Undeploy Content [page 445]

4.1.4.2.1.2.1 Deploy Content Using the Generic Application Content Deployer

Deploy content from the HTML5 Application Repository using the Generic Application Content Deployer (GACD).

Prerequisites

- `cf CLI` is installed locally, see Download and Install the Cloud Foundry Command Line Interface [page 2813].
- The multitarget application (MTA) plug-in for the Cloud Foundry command-line interface to deploy MTAs is installed locally. For more information, see Install the MultiApps CLI Plugin in the Cloud Foundry Environment [page 2833].
- The Cloud MTA Build Tool (MBT) is installed, see Cloud MTA Build Tool: Download

Context

You can deploy your content to the HTML5 Application Repository using the GACD (Generic Application Content Deployer) module. The GACD module has the module type `com.sap.application.content`. This module type enables the deploy plug-in generic application content deploy support. It means that when a module is processed in the cf deploy flow, the deploy service locates the service resource that is required as a target for the deploy and deploys the corresponding `content.zip` file.

Procedure

1. Create an MTA development descriptor (mta.yaml file).

   See Defining Multitarget Application Development Descriptors [page 621].
2. In the MTA development descriptor (mta.yaml file), define the deployer module:

   This is an example for an MTA that contain a folder for an application "app1" and an application "app2":

   Sample Code

```
ID: testdeployer
_schema-version: '3.1'
modules:
# GACD Deployer Module
 name: html5-apps-content
  type: com.sap.application.content
  path: .
  requires:
    - name: html5-apps-xsuaa
    - name: html5-apps-html5-repo-host
    parameters:
```

```
      content-target: true
  build-parameters:
    build-result: resources
    requires:
    - name: app1
      artifacts:
      - app1.zip
      target-path: resources/
    - name: app2
      artifacts:
      - app2.zip
      target-path: resources/
# HTML5 Modules
- name: app1
  type: html5
  path: app1
  build-parameters:
    build-result: dist
    builder: custom
    commands:
    - npm install
    - npm run build:cf
    supported-platforms: []
- name: app2
  type: html5
  path: app2
  build-parameters:
    build-result: dist
    builder: custom
    commands:
    - npm install
    - npm run build:cf
    supported-platforms: []
resources:
# HTML5 Application Repository service instance
- name: html5-apps-html5-repo-host
  type: org.cloudfoundry.managed-service
  parameters:
    service: html5-apps-repo
    service-name: html5-apps-html5-repo-host-service
    service-plan: app-host
# SAP Authorization and Trust Management service (technical name: xsuaa)
- name: html5-apps-xsuaa
  type: org.cloudfoundry.managed-service
  parameters:
    path: ./xs-security.json
    service: xsuaa
    service-name: html5-apps-xsuaa
    service-plan: application
```

3. To create an MTA archive (mtar file), build your content with the MTA Build Tool (MBT) .

   Run the following command:

   Sample Code

   ```
   mbt build
   ```

   For more information on MBT build options, see: How to build an MTA archive from the project sources

4. Deploy the MTA archive (mtar file) using the CLI command cf deploy.

Run the following command:

Sample Code

```
cf deploy <path-to-mtar-file>
```

For example, from the directory where your mtar file is located, run `cf deploy myapp_0.0.1.mtar`.

Related Information

Deploying Content with Generic Application Content Deployment [page 770]

4.1.4.2.1.2.2 Deploy Content Using HTML5 Application Deployer

Use the HTML5 application deployer module to deploy the content of the HTML5 applications to the HTML5 Application Repository.

Prerequisites

- `cf CLI` is installed locally, see Download and Install the Cloud Foundry Command Line Interface [page 2813].
- The multi-target application (MTA) plug-in for the Cloud Foundry command line interface to deploy MTAs is installed locally. For more information, see Install the MultiApps CLI Plugin in the Cloud Foundry Environment [page 2833].

> **Note**
>
> It is not recommended to use the HTML5 application deployer to deploy application content that is larger than 10 MB.
>
> For larger application content, please use the Generic Application Content Deployer (GACD) [page 437] or configure the asynchronous content upload [page 442].

Context

The HTML5 application deployer is available on npmjs.com (NPM), see @sap/html5-app-deployer .

You can deploy your content to the HTML5 Application Repository using the HTML5 application deployer npm module.

Procedure

1. Add the `html5-app-deployer` module as a dependency to your `package.json` file. To do so, navigate to your `package.json` file and execute `npm install` to download the `html5-app-deployer module` from the SAP npm registry.

   The basic `package.json` file should look similar to the following example:

   Sample Code

   ```
   {
     "name": "myAppDeployer",
     "engines": {
       "node": ">=6.0.0"
     },
     "dependencies": {
       "@sap/html5-app-deployer": "2.0.1"
     },
     "scripts": {
       "start": "node node_modules/@sap/html5-app-deployer/index.js"
     }
   }
   ```

   The `start` script is mandatory as it is executed after the deployment of the application.

2. In the `html5-app-deployer` structure, create a `resources` folder and add the static content that you want to deploy. In the `resources` folder, add one folder for each application you want to deploy. For each application you want to deploy, provide a `manifest.json` and `xs-app.json` file at root level.

   If you want to deploy more than one application to the same app host instance, you can add multiple zip archives to the resources folder.

   Sample Code

   ```
   myAppsDeployer
     + node_modules
     - resources
       - app1
         index.html
         manifest.json
         xs-app.json
       - app2
         ...
     package.json
     manifest.yaml
   ```

   a. The `manifest.json` file contains `sap.app.id` and `sap.app.applicationVersion.version`, which are used in the HTML5 Application Repository as `applicationName` and `applicationVersion`.

      Note

      The format of the application version is `xx.xx.xx`.

      Sample Code

      ```
      manifest.json
      ```

```
{
  "_version": "1.7.0",
  "sap.app": {
    "id": "app1",
    "type": "application",
    "i18n": "i18n/i18n.properties",
    "applicationVersion": {
      "version": "1.0.0"
    }
  }
}
```

b. The `xs-app.json` file is used to support application routing.

Sample Code

```
xs-app.json
{
 "welcomeFile": "index.html",
 "authenticationMethod": "route",
 "routes": [
  {
   "source": "^/be$",
   "destination": "simpleui_be",
   "authenticationType": "xsuaa"
  },
  {
   "source": "^/ui(/.*)",
   "target": "$1",
   "service": "html5-apps-repo-rt",
   "authenticationType": "xsuaa"
  }
 ]
}
```

3. Create an `mta.yaml` (MTA development descriptor) file:
   a. Under `resources`, add a resource definition to create an `html5-apps-repo` service instance of the app-host service plan.
   b. Under `modules`, add your app.
   c. Add a dependency to the `html5-apps-repo` service and the app-host service plan instance using the required statement.

Sample Code

```
ID: html5.repo.deployer.myHTML5App
_schema-version: '2.0'
version: 0.0.3
modules:
- name: myHTML5App_app-deployer
   type: com.sap.html5.application-content
   path: deployer/
   requires:
    - name: myHTML5App_app-host

resources:
- name: myHTML5App_app-host            //Resource name
   type: org.cloudfoundry.managed-service
   parameters:
     service: html5-apps-repo            //Service name
     service-plan: app-host              //Service plan
     service-name: myHTML5App_app-host   //Service instance name
```

For a general description of MTA descriptors, see Multitarget Applications in the Cloud Foundry Environment [page 615].

4. Build and deploy the mta.yaml (MTA development descriptor).

   For more information about the build and deployment of MTA descriptors, see Multitarget Applications in the Cloud Foundry Environment [page 615].

   The deployment starts the following:

   - Create an HTML5 Application Repository service instance of the `app-host` service plan.
   - Create an HTML5 application deployer application, which uses the HTML5 application deployer npm module.
   - Bind the `app-host` service instance to the HTML5 application deployer application.
   - Start the HTML5 application deployer application:
     - Create a zip archive for each application in resources folder.
     - Create a client credential token from the `app-host` service instance credentials.
     - Deploy the content to the HTML5 application repository: passing on the zip archives and the client credential token.
   - Stop the HTML5 application deployer application.

4.1.4.2.1.2.2.1 Asynchronous Upload Using the HTML5 Application Deployer

You can specify that the upload of content is performed asynchronously by adding the environment variable `ASYNC_UPLOAD` to the `manifest.yaml` or `mta.yaml` files.

The asynchronous upload is especially useful, if you want to trigger the upload of service instances with a large content (more than 10 MB). A synchronous upload of service instances with a large content can result in health check errors or a connection timeout during the upload. You can avoid this by using the asynchronous upload.

During an asynchronous upload, the html5 applications content is handled synchronously to the HTML5 Application Repository, while the internal file validation and processing is performed asynchronously. To verify that the upload has completed successfully, you have to check the HTML5 Application Deployer logs.

4.1.4.2.1.2.2.2 Automatic Creation of Destination Configurations

If you use the HTML5 application deployer together with an application router managed by SAP for the Kyma runtime, you can enable that the required destination configurations pointing to the service instances are created automatically.

To enable the automatic creation of destination configurations, add the environment variable SAP_CLOUD_SERVICE in the deployment.yaml or Helm chart of the Kyma project and provide the value of the sap.cloud.service property from the manifest.json file of the HTML5 application that you want to deploy. The destination configurations can point to instances of the following services:

- An HTML5 Application Repository service instance of the app-host service plan (mandatory)
- An SAP Authorization and Trust Management (XSUAA) service instance (optional)
  To create a destination that points to an SAP Authorization and Trust Management (XSUAA) service instance, the SAP Authorization and Trust Management (XSUAA) service instance and a destination service instance must be bound to the HTML5 application deployer.
- A business service instance (optional)
- One or more backend destinations that point to cloud or on-premise backend applications.
  In addition to the SAP_CLOUD_SERVICE environment variable, also add the environment variable BACKEND_DESTINATIONS to create backend destinations.

> ### Note
>
> To enable app-to-app navigation, you also have to add the environment variable IAS_DEPENDENCY_NAME and provide the name of the dependency that has been configured for the Identity Authentication token exchange that is required for app-to-app navigation. For more information about how to configure the dependency for app-to-app navigation, see Consume an API from Another Application.

You would typically use the automatic creation of destination configurations in Kubernetes if the HTML5 application deployer application has been previously uploaded as a docker image to Artifactory or Docker Hub. See, for example, this Kubernetes deployment:

> ### Sample Code

```
--
apiVersion: apps/v1
kind: Deployment
metadata:
  name: html5appdeployer
  namespace: default
  labels:
    app: html5appdeployer
spec:
  replicas: 1
  selector:
    matchLabels:
      app: html5appdeployer
  template:
    metadata:
      labels:
        app: html5appdeployer
    spec:
      containers:
        - image: html5-apps-repo.docker.repositories.sap.ondemand.com/myapp-
html5-app-deployer:1.0
          name: html5appdeployer
          volumeMounts:
            - name: html5-repo-app-host-volume
              mountPath: "/etc/secrets/sapcp/html5-apps-repo/myapp-app-host-
instance"
              readOnly: true
            - name: xsuaa-volume
              mountPath: "/etc/secrets/sapcp/xsuaa/myapp-xsuaa-instance"
              readOnly: true
            - name: destination-volume
              mountPath: "/etc/secrets/sapcp/destination/myapp-destination-
instance"
              readOnly: true
          env:
            - name: PORT
              value: "5000"
```

```
            - name: SAP_CLOUD_SERVICE
              value: "com.sap.test.service"
            - name: BACKEND_DESTINATIONS
              value: "[{
              \"Name\":\"myapp-backend\",
              \"Description\":\"My application backend\",
              \"Type\":\"HTTP\",
              \"ProxyType\":\"Internet\",
              \"URL\":\"https://<backendApplicationHost>/\",
              \"Authentication\":\"NoAuthentication\",
              \"HTML5.ForwardAuthToken\": true}]"
      imagePullSecrets:
        - name: backend-dockersecret
      volumes:
        - name: html5-repo-app-host-volume
          secret:
            secretName: myapp-app-host-binding
        - name: xsuaa-volume
          secret:
            secretName: myapp-xsuaa-binding
        - name: destination-volume
          secret:
            secretName: myapp-destination-binding
```

4.1.4.2.1.2.3 Redeploy Content

You can redeploy changed content to the existing app-host service instance.

After making changes to the static content files of HTML5 applications, you can redeploy the new content to the already existing app-host service instance of the HTML5 application repository. All content referenced by the app-host service instance ID is replaced by the new content. If you want to keep more than one version of your HTML5 application on the repository, you deploy all versions, the new versions and the old versions that you want to keep, to the same app-host service instance ID.

Example: Deploying More Than One Version to the Repository

- Application **app1** with version 1.0.0 has been deployed to the repository. This app matches a back-end application with version 3.0.0
- For the app, a new back-end version 4.0.0 is available. Therefore, the application with version 2.0.0 is developed that matches the new back-end version.
- As customers may still have a backend with version 3.0.0, **app1** with version 1.0.0 cannot be dropped. On the repository, both versions shall be available so the customer can decide which one to use depending on their back-end version.

This example shows how to deploy two versions of an application with the HTML5 Application Deployer. You can use a similar structure when you deploy MTA applications with the Generic Application Content Deployer (GACD): /

Sample Code

```
myAppsDeployer
  + node_modules
```

```
- resources
  - app1-old-backend
    index.html
    manifest.json (app.id: 1.0.0)
    xs-app.json
  - app1-new-backend
    index.html
   manifest.json (app.id: 2.0.0)
   xs-app.json
package.json
manifest.yaml
```

4.1.4.2.1.2.4 Undeploy Content

To undeploy content you need to delete the content from the repository and delete the `app-host` service plan instance.

Procedure

1. Open the Cloud Foundry command line interface (CLI).
2. Undeploy the `*.mtar` file using the CLI command: `cf undeploy` and add the `--delete-services --delete-service-keys` option.

   Example: `cf undeploy html5.repo.deployer.myHTML5App --delete-services --delete-service-keys`

   - Use the `–delete-services` option, to delete the `app-host` service plan instance and the application content from HTML5 application repository.
   - For the `cf undeploy` command, you need the `mta id`. To get the `mta id`, do one of the following:
     - Call `cf mtas`.
     - Check the `mtad.yaml ID`.

   > ⓘ Note
   >
   > If you do not want to use the `–delete-services` option, you can delete the `app-host` service plan instance manually using the CLI command: `cf delete-service SERVICE-NAME`.

4.1.4.2.1.3 Consuming Content

Use the `app-runtime` service plan to consume HTML5 applications from the HTML5 Application Repository.

Context

Application router can consume content from the HTML5 Application Repository by binding an HTML5 Application Repository service instance with `app-runtime` plan to the application router. In addition, the routing to the HTML5 Application Repository service is configured in the `xs-app.json` file.

Procedure

1. Create a service instance of `app-runtime` service plan.

   Example: `cf create-service html5-apps-repo app-runtime MyHtml5AppsRepoRuntimeInstance`
2. Bind the HTML5 Application Repository runtime instance to the application router in the `manifest.yaml` file.

   Sample Code

   ```
   applications:
   - name: myCRMApp
     memory: 256M
     services:
       - MyHtml5AppsRepoRuntimeInstance
   ```
3. Push the application with the following command: `cf push`
4. To redirect requests to the HTML5 Application Repository runtime, configure the routing to the HTML5 Application Repository service in your `xs-app.json` file.

   Sample Code

   ```
   {
        "source": "^(/.*)",
        "target": "$1",
        "service": "html5-apps-repo-rt",
        "authenticationType": "xsuaa"
    }
   ```
5. HTML5 application content can be consumed from the application router using an URL in the following format: `https://<host>.<domain>/<appName>-<appVersion>/<resourcePath>`

> **Note**
>
> `appName` - the `manifest.json app.id` **without the dots**
>
> `appVersion` - the `manifest.json applicationVersion.version`. The appVersion is optional. If not provided, the latest version will be used.

4.1.4.2.1.4 Security and Data Privacy Configuration

This section provides information about security and data privacy.

- Identity Provider and Identity Management
  You can use the User Account and Authentication (UAA) server for user authentication. In Cloud Foundry, a service is created for this configuration. By using the standard service binding mechanism, the content of this configuration is available in the `VCAP_SERVICES` environment variable.
  As an alternative to the User Account and Authentication (UAA) service , you can also use the Identity Authentication service of the SAP Cloud Identity Services for authentication.
- Authorization Configuration
  To obtain the access token to authorize API requests, the client credentials authorization flow is used.
- Data Protection and Data Privacy
  The HTML5 application repository service applies the legally required data protection and privacy measures to data that is stored on behalf of customers. These measures include, for example, the need to know principle during support processes, or data isolation between spaces.
  The HTML5 application repository runtime is a microservice responsible for providing read access to HTML5 applications during runtime. When the HTML5 application is public, the service enables sharing this content with consuming application routers from different spaces.

> **Note**
>
> The HTML5 application repository service does not provide the technical capabilities to support the collection, processing, and storage of personal data. The file names used for deploying the HTML5 applications are not intended for personal data.

4.1.4.2.1.4.1 Configure the Credential Type for HTML5 Application Repository Service Instances

When you create an HTML5 application repository service instance key or bind a service instance to an application, values for `client_id` and `client_secret` are generated. The `client_secret` is the equivalent to a password.

As of January 28, 2021, the credential type `binding_secret` is automatically used for the `client_ secret` value of new HTML5 application repository service instance keys. This increases the security because the credential type `binding_secret` creates a new `client_secret` for each binding and service key.

If you have older HTML5 application repository service instances, the `client_secret` value is the same for all bindings and service keys because the HTML5 application repository still uses the credentials type

`instance_secret` for older HTML5 application repository service instances. To configure the more secure credential type `binding_secret` for your older HTML5 application repository service instances, please follow the instructions in Configure the Credential Type binding_secret for HTML5 Application Repository Service Instances [page 448].

As an alternative to the credential type `binding_secret`, you can also use the credential type `x.509` to authenticate your apps with X.509 certificates. For more information, see Credential Type x509 for HTML5 Application Repository Service Instances [page 449].

Configure the Credential Type `binding_secret` for HTML5 Application Repository Service Instances

1. Extract the UUID part of the `xsappname` from the of the existing instance. For example:
    1. Sample Code

        ```
        cf create-service-key <serviceInstanceName> <serviceInstanceKeyName>
        ```

    2. Sample Code

        ```
        cf service-key <serviceInstanceName> <serviceInstanceKeyName>
        ……
        "clientsecret": "HksDYy9RxYFUQeOHIzprqBam3vQ=", //Old instance secret
        …
          "xsappname": "fec39a25-a570-46b0-8ac9-1691f8d663cc!b6711|html5-apps-repo-uaa!b6711",
        ……
        ```

    3. Copy the UUID. In this example: **fec39a25-a570-46b0-8ac9-1691f8d663cc**
2. Create a file `instance-xs-security.json`, such as the following:

    Sample Code

    ```
    {
     "xs-security": {
       "xsappname": "fec39a25-a570-46b0-8ac9-1691f8d663cc",
       "oauth2-configuration": {
         "credential-types": ["binding-secret"]
       }
     }
    ```

3. Update the service instance:

    Sample Code

    ```
    cf update-service <serviceInstanceName>  -c instance-xs-security.json
    ```

4. Check that the instance now creates binding secrets as follows:
    1. Sample Code

        ```
        cf create-service-key <serviceInstanceName>
        <serviceInstanceKeyNameNew>
        ```

2. Sample Code

```
cf service-key <serviceInstanceName>  <serviceInstanceKeyNameNew>
 ……..
"clientid": "sb-fec39a25-a570-46b0-8ac9-1691f8d663cc!b6711|html5-apps-
repo-uaa!b6711",
  "clientsecret":
"8c902673-00d4-471f-9ebb-4463d4073fda$7NIo5batSFZCF1RNGcpNkRfo4mDj6vq-
cElDFIcvcwg=", //New binding secret (long)
  "identityzone": …..,
  "identityzoneid": …..,
                                  …….
```

Credential Type x509 for HTML5 Application Repository Service Instances

To use the HTML5 application repository service and authenticate with X.509 certificate, you need to acquire the required key, certificate, and base URLs. These are provided when you create an instance of the HTML5 application repository service and a binding or a service key for the instance.

When you bind an HTML5 applications repository service instance to an application, you have to explicitly provide the required configuration parameters for the authentication with x.509:

```
cf bind-service <application> <service-instance> -c parameters.json
```

The same applies if you create service keys:

```
cf create-service-key <service-instance> <key-name> -c parameters.json
```

The configuration for the credential type x509 must be wrapped with xsuaa. You have to use exactly the following format for the configuration:

```
{
  "xsuaa": {
    "credential-type": "x509",
    "x509": {
      "key-length": 2048, // specifies the byte length of the generated private
key,
                          // defaults to 2048
      "validity": 7, // specifies the number of time units in validity-type,
                     // defaults to 7, thus the complete validity defaults to 7
days
      "validity-type": "DAYS" // specifies the validity time unit,
                              // only DAYS, MONTHS and YEARS are supported,
                              // defaults to DAYS
    }
  }
}
```

> **Note**
>
> The following applies to existing HTML5 application repository service instances:
>
> Before you create a binding to a service instance or a service key as described above, you have to update of the service instance of your existing HTML5 application repository service instance. Use parameters to provide the configuration during the update of the existing service instance:
>
> ```
> cf update-service <service-instance> -c update-instance-xs-security.json
> ```

To configure the credential type x509 for existing HTML5 application repository service Instances, please follow the instructions below:

Configuring Credential Type x509 for Existing HTML5 Application Repository Service Instances

To configure the credential type x509 for your existing HTML5 application repository service instances, perform the following steps:

1. Extract the UUID part of the xsappname from the service key or the binding for the service instance. For example:
    1. Sample Code

        ```
        cf service-key <serviceInstanceName> <existingServiceInstanceKeyName>
        ……
        "clientsecret": "HksDYy9RxYFUQeOHIzprqBam3vQ=", //Old instance secret
        …
        "xsappname": "fec39a25-a570-46b0-8ac9-1691f8d663cc!b6711|html5-apps-
        repo-uaa!b6711",
        ……
        ```

    2. Copy the UUID. In this example: **fec39a25-a570-46b0-8ac9-1691f8d663cc**
2. Create a file `update-instance-xs-security.json` with the following format:

    Sample Code

    ```
    {
     "xs-security": {
       "xsappname": "fec39a25-a570-46b0-8ac9-1691f8d663cc", //The UUID that
    was extracted before
       "oauth2-configuration": {
         "credential-types": ["x509"]
       }
     }
    ```

3. Update the service instance:
    `cf update-service <serviceInstanceName> -c update-instance-xs-security.json`
4. Create a file `parameters.json`, such as the following: :

    Sample Code

    ```
    {
      "xsuaa": {
        "credential-type": "x509",
        "x509": {
          "key-length": 2048,
          "validity": 7,
          "validity-type": "DAYS"
        }
      }
    }
    ```

    (Note that the values of the "x509" properties used here are just examples.)
5. Create a binding or a service key with the credential type x509 configuration (`parameters.json` file). For example:
    `cf create-service-key <serviceInstanceName> <serviceInstanceKeyNameNew> -c parameters.json`

6. Check that the instance now creates a certificate and that the credential-type is "x509", as follows:
   `cf service-key <serviceInstanceName> <serviceInstanceKeyNameNew>`
   The following binding is generated as a result:

```
{
  ......
  "certificate": "-----BEGIN CERTIFICATE-----..-----END CERTIFICATE-----
\n-----BEGIN CERTIFICATE-----..-----END CERTIFICATE-----\n-----BEGIN
CERTIFICATE-----..-----END CERTIFICATE-----\n",
  "clientid": "........",
  "credential-type": "x509",
  ......
}
```

4.1.4.2.1.4.2 Encryption Using Customer-Managed Keys

You can secure your SAP data with your own customer-managed key (CMK) using SAP Data Custodian.

Integrating SAP Data Custodian with the HTML5 Application Repository service allows you to encrypt HTML5 application files and credentials stored in the HTML5 application repository using your own customer-managed key (CMK). For more information about customer-managed keys, see .What is Key Management Service?.

Enabling CMK Encryption for a New Instance of the HTML5 Application Repository Service

To enable the CMK for the HTML5 Application Repository, include a configuration file (for example, `config.json`) with the configuration `"enableCmk": true` when you create or update an HTML5 Application Repository service instance of the app-host service plan. For example:

Sample Code

```
cf create-service html5-apps-repo app-host -c config.json
```

The `config.json` file must contain:

Sample Code

```
{
    "enableCmk": true
}
```

Enabling CMK Encryption for an Existing HTML5 Application Repository Service Instance

To enable CMK for an existing HTML5 Application Repository service instance, update the configuration file for the service instance to include the parameter `"enableCmk": true`. Then redeploy the application content to the app-host service instance.

Disabling CMK Encryption for an HTML5 Application Repository Service Instance

To disable the CMK for your HMTL5 Application Repository service instance, set the enableCmk parameter in the instance configuration file to false (`"enableCmk": false`). Then redeploy the application content to the app-host service instance.

4.1.4.2.1.4.3 Auditing and Logging Information

Here you can find a list of the security events that are logged by the HTML5 application repository.

Events written in audit logs

Event grouping	What events are logged	How to identify related log events	Additional information
Authentication failure	Authentication failed because of an invalid token	Message: *Authentication failure status: 401*	
	Authentication failed because the token has expired	Message: *Authentication failure status: 403*	
Attempt to access an application with an invalid app-host ID	Attempt to access an application with an invalid app-host ID	Message: *Invalid App Host ID* If there was an attempt to get an application that was deployed by a developer in another space.	
Attempt to access a private application from another space	Attempt to access a private application from another space	Message: *Attempt to access private application. application name :*`'+ appKey + ', app-host : ' + dtServiceInstanceId`	

Event grouping	What events are logged	How to identify related log events	Additional information
Deletion of an application (Deletion of an app-host service instance from the HTML5 application repository)	Deletion of data started	Message: *Deletion of data started for SPACE <space> in ORG <org>*	
	Deletion of data completed	Message: *Deletion of data completed for SPACE <space> in ORG <org>*	
	Deletion of data failed	Message: *Deletion of data failed for SPACE <space> in ORG <org>*	

The following information is described in the table columns:

- *Event grouping* - Events that are logged with a similar format or are related to the same entities.
- *What events are logged* - Description of the security or data protection and privacy related event that is logged.
- *How to identify related log events* - Search criteria or key words, that are specific for a log event that is created along with the logged event.
- *Additional information* - Any related information that can be helpful.

Related Information

Audit Logging in the Cloud Foundry Environment
Audit Logging in the Neo Environment

4.1.4.2.2 Application Router

The application router is the single point-of-entry for an application running in the Cloud Foundry environment on SAP BTP. The application router is used to serve static content, authenticate users, rewrite URLs, and forward or proxy requests to other micro services while propagating user information.

You can set up and run your own application router or you can use the application router that is managed by SAP (for more information see Managed Application Router [page 534]).

We recommend running your own application router only in advanced cases, for example when application router extensibility is required. The application router is available on the following sites:

- As a library on npmjs.com (NPM), see @sap/approuter
- As a container image on Docker Hub, see https://hub.docker.com/r/sapse/approuter

Related Information

Developing HTML5 Applications in the Cloud Foundry Environment [page 433]
Setting Up Your Own Application Router [page 454]
Resource Files [page 457]
Application Router Configuration [page 458]
Routing Configuration File [page 464]
Application Routes and Destinations [page 496]
Environment Variables [page 510]
Multitenancy [page 527]
Extending the Application Router [page 529]
Extension API of the Application Router [page 532]

4.1.4.2.2.1 Setting Up Your Own Application Router

This section describes how you can set up your own application router.

Prerequisites

- cf CLI is installed locally, see Download and Install the Cloud Foundry Command Line Interface [page 2813].
- Node.js is installed and configured locally, see npm documentation .
- The SAP npm registry, which contains the Node.js package for the application router, is configured:
  `npm config set @sap:registry https://registry.npmjs.org`

Context

The application router is configured in the `xs-app.json` file. The `package.json` file contains the start command for the application router, a list of the package dependencies, and the supported Node.js versions.

Procedure

1. Create the application resource-file structure.

   For example, `/path/<myAppName>/`
2. Create a subfolder for the static Web resources module.

   The subfolder for the Web-resources module must be located in the application root folder, for example, `/path/<myAppName>/web`.

→ Tip

The Web-resource module uses `@sap/approuter` as a dependency; the Web-resources module also contains the configuration and static resources for the application.

3. Create the subfolder for the application's static resources.

   Static resources can, for example, include the following file and components: `index.html`, style sheets (`.css` files), images and icons. Typically, static resouces for a Web application are placed in a subfolder of the Web module, for example, `/path/<myAppName>/web/resources`.

4. Create the application-router configuration file.

   The application router configuration file `xs-app.json` must be located in the application's Web-resources folder, for example, `/path/<MyAppName>/web`.

   {·} Sample Code

   ```
   <myAppName>
   |- web/                          # Application descriptors
   |  |- xs-app.json                # Application routes configuration
   |  |- package.json               # Application router details/dependencies
   |  \- resources/
   ```

5. Define details of the application's routes, destinations, and security scopes.

   The contents of the `xs-app.json` file must use the required JSON syntax. For more information, see Routing Configuration File [page 464].

   a. Create the required destinations configuration.

      {·} Sample Code

      `/path/<myAppName>/web/xs-app.json`.

      ```
      {
          "welcomeFile": "index.html",
          "routes": [
              {
                  "source": "/sap/ui5/1(.*)",
                  "target": "$1",
                  "localDir": "sapui5"
              },
              {
                  "source": "/rest/addressbook/testdataDestructor",
                  "destination": "backend",
                  "scope": "node-hello-world.Delete"
              },
              {
                  "source": "/rest/.*",
                  "destination": "backend"
              },
              {
                  "source": "^/(.*)",
                  "localDir": "resources"
              }
          ]
      }
      ```

   b. Add the routes (destinations) for the specific application (for example, `node-hello-world`) to the `env` section of the application's deployment manifest (`manifest.yml`).

Every route configuration that forwards requests to a micro service has property destination. The destination is a name that refers to the same name in the destinations configuration. The destinations configuration is specified in an environment variable passed to the approuter application.

Sample Code

`<myAppName>/manifest.yml`

```
- name: node-hello-world
    routes:
    - route: myHost-node-hello-world.xsapps.acme.ondemand.com
    memory: 100M
    path: web
    env:
        destinations: >
        [
            {
                "name":"backend",
                "url":"http://myHost-node-hello-world-
backend.xsapps.acme.ondemand.com",
                "forwardAuthToken": true
            }
        ]
```

6. Add a package descriptor (`package.json`) for the application router to the root folder of your application's Web resources module (`web/`) and execute `npm install` to download the approuter npm module from the SAP npm registry.

   The package descriptor describes the prerequisites and dependencies that apply to the application router and starts the application router, too.

   ### Sample Code

   ```
   <myAppName>
   |- web/                          # Application descriptors
   |  |- xs-app.json                # Application routes configuration
   |  |- package.json               # Application router details/dependencies
   |  \- resources/
   ```

   The basic `package.json` file for your Web-resources module (`web/`) should look similar to the following example:

   ### Sample Code

   ```
   {
     "name": "node-hello-world-approuter",
     "dependencies": {
        "@sap/approuter": "5.1.0"
     },
     "scripts": {
        "start": "node node_modules/@sap/approuter/approuter.js"
     }
   }
   ```

   ### → Tip

   The start script (for example, `approuter.js`) is mandatory; the start script is executed after application deployment.

Related Information

Managed Application Router [page 534]

4.1.4.2.2.2 Resource Files

The routing configuration for an application is defined in one or more destinations.

The application router is used to serve static content, authenticate users, rewrite URLs, and forward or proxy requests to other micro services while propagating user information. The following table lists the resource files used to define routes for multi-target applications:

Application-Router Resource Files Overview

File	Description	Mandatory
`package.json`	The package descriptor is used by the Node.js package manager (`npm`) to start the application router; in the `"dependencies": {}` section	Yes
`xs-app.json`	The application descriptor contains the configuration used by the application router (for example, destinations for request forwarding)	Yes
`resources/`	A folder that contains all static resources which should be served by the application router. If no `resources/` folder is present, the application router will not serve any static content. However, it still forwards requests to the configured destinations. → Tip If you have a static resources folder name in the `xs-app.json` file, we recommend that you use `localDir` as default.	No
`local-destinations.json`	Provides the required destinations information for local development	No
`default-services.json`	Defines the configuration for one or more special User Account and Authentication (UAA) services for local development	-

{·} Sample Code

`xs-app.json`

```
{
"source":"^/web-pages/(.*)$",
"localDir":"my-static-resources"
```

```
}
```

4.1.4.2.2.3 Application Router Configuration

A file that contains the configuration information used by the application router.

The application router configuration file is named `xs-app.json` and its content is formatted according to JavaScript Object Notation (JSON) rules.

When a business application consists of several different apps (microservices), the application router is used to provide a single entry point to that business application. The application router is responsible for the following tasks:

- Dispatch requests to back-end microservices (reverse proxy)
- Authenticate users
- Serve static content
- In multitenancy scenarios, derive the tenant information from the URL and forward the information to the XS UAA service so that the authentication request is redirected to the appropriate tenant-specific Identity Provider (IdP), for example,using SAML “Bearer Assertions”.

→ Tip

The different applications (microservices) are the destinations to which the incoming requests are forwarded. The rules that determine which request should be forwarded to which destination are called routes. For every destination there can be more than one route.

User Authentication Services

For the user authentication, you can use User Account and Authentication (UAA) service or the Identity Authentication service.

User Account and Authentication (UAA)

User authentication is performed by the User Account and Authentication (UAA) server. In the run-time environment (on-premise and in the Cloud Foundry environment), a service is created for the UAA configuration; by using the standard service-binding mechanism, the content of this configuration is available in the `<VCAP_SERVICES>` environment variable, which the application router can access to read the configuration details.

ⓘ Note

The UAA service should have `xsuaa` in its tags or the environment variable `<UAA_SERVICE_NAME>` should be defined, for example, to specify the **exact** name of the UAA service to use.

A calling component accesses a target service by means of the application router only if there is no JSON Web Token (JWT) available, for example, if a user invokes the application from a Web browser. If a JWT token is already available, for example, because the user has already been authenticated, or the calling component uses

a JWT token for its own OAuth client, the calling component calls the target service directly; it does not need to use the application router.

> **Note**
>
> The application router does not “hide” the back-end microservices in any way; they remain directly accessible when bypassing the application router. So the back-end microservices must protect all their end points by validating the JWT token and implementing proper authorization scope checks.

The application router supports the use of the `$XSAPPNAME` placeholder, which you can use in your route configuration, for example, in the `scope` property for the specified route. The value of `$XSAPPNAME` is taken from the UAA configuration (for example, the `xsappname` property). For more information, see Routing Configuration File [page 464].

Identity Authentication Service

As an alternative to the User Account and Authentication service (UAA), you can also use Identity Authentication. This service provides authentication and single sign-on for users in the cloud. For more information, see the product page for SAP Cloud Identity Services - Identity Authentication on the SAP Help Portal.

Headers

If back end nodes respond to client requests with URLs, these URLs need to be accessible for the client. For this reason, the application router passes the following `x-forwarding-*` headers to the client:

- `x-forwarded-host`
  Contains the **host** header that was sent by the client to the application router
- `x-forwarded-proto`
  Contains the **protocol** that was used by the client to connect to the application router
- `x-forwarded-for`
  Contains the **address** of the client which connects to the application router
- `x-forwarded-path`
  Contains the original **path** which was requested by the client from the approuter

> **Caution**
>
> If the application router forwards a request to a destination, it blocks the header `host`.

“Hop-by-hop” headers are meaningful only for a single transport-level connection; these headers are not forwarded by the application router. The following headers are classified as “Hop-By-Hop” headers:

- Connection
- Keep-Alive
- Public
- Proxy-Authenticate
- Transfer-Encoding
- Upgrade

You can configure the application router to send additional HTTP headers, for example, either by setting it in the `httpHeaders` environment variable or in a `local-http.headers.json` file.

Sample Code

`local-http.headers.json`

```
[
  {
    "X-Frame-Options": "ALLOW-FROM http://localhost"
  },
  {
    "Test-Additional-Header": "1"
  }
]
```

Caution

For security reasons, the following headers must not be configured:

- Authorization
- Set-Cookie
- Cookie

Sessions

The application router establishes a session with the client (browser) using a session cookie. The application router intercepts all session cookies sent by back-end services and stores them in its own session. To prevent collisions between the various session cookies, back-end session cookies are not sent to the client. On request, the application router sends the cookies back to the respective back-end services so the services can establish their own sessions.

Note

Non-session cookies from back-end services **are** forwarded to the client, which might cause collisions between cookies. Applications should be able to handle cookie collisions.

Restriction

You must not use multiple destinations with the same URL in an application router user session!

The application router stores the session cookies from the backend services in a user session according to the destination URLs for the backend services. If there are multiple destinations with the same URL, the application router cannot correctly send the cookies back to the backend services.

To avoid conflicts, the destination URLs must have different domains.

If there is no session in the application router, either because there has been a session timeout or because no session has been created yet, and if the incoming request matches a non-public route, the application router triggers a redirect to the authentication service (UAA or IAS).

After a successful login, a redirect back to application router takes place using the login callback endpoint, which triggers the creation of a new session. If the incoming request is an AJAX request (has the request

header `X-Requested-With: XMLHttpRequest`) or if the HTTP verb is not `GET` and no session exists (there is no session cookie and the request doesn't have an x-approuter-authorization header), the application router returns the response code `401 - Unauthorized`. This enables the client application to handle the 401 response before it navigates to the authentication service. For example, the application can store data entered by the user and prevent data loss. When the handling of the 401 response is completed, the client application should send a request without an `xmlhttprequest` object to trigger the application router authentication flow.

Session Contents

A session established by the application router typically contains the following elements:

- Redirect location
  The location to redirect to after logon; if the request is redirected to a UAA logon form, the original request URL is stored in the session so that, after successful authentication, the user is redirected back to it.
- CSRF token
  The CSRF token value if it was requested by the clients. For more information about protection against Cross Site Request Forgery see CSRF Protection [page 462] below.
- OAuth token
  The JSON Web Token (JWT) fetched from the User Account and Authentication service (UAA) and forwarded to back-end services in the `Authorization` header. The client never receives this token. The application router refreshes the JWT automatically before it expires (if the session is still valid). By default, this routine is triggered 5 minutes before the expiration of the JWT, but it can also be configured with the `<JWT_REFRESH>` environment variable (the value is set in minutes). If `<JWT_REFRESH>` is set to 0, the refresh action is disabled.
- OAuth scopes
  The scopes owned by the current user, which are used to check if the user has the authorizations required for each request.
- Back-end session cookies
  All session cookies sent by back-end services.

Scaling

The application router keeps all established sessions in local memory, and if multiple instances of the application router are running, there is no synchronization between the sessions. To scale the application router for multiple instances, session stickiness is used so that each HTTP session is handled by the same application router instance.

Sizing and Memory Configuration

The application-router process should run with at least 256MB memory. The amount of memory actually required depends on the application the router is serving. The following aspects have an influence on the application's memory usage:

- Concurrent connections
- Active sessions

- Size of the Java Web Token
- Back-end session cookies

You can use the start-up parameter `max-old-space-size` to restrict the amount of memory used by the JavaScript heap. The default value for `max-old-space-size` is less than 2GB. To enable the application to use all available resources, the value of `max-old-space-size` should be set to a number equal to the memory limit for the whole application. For example, if the application memory is limited to 2GB, set the heap limit as follows, in the application's `package.json` file:

Sample Code

```
"scripts": {
  "start": "node --max-old-space-size=2048 node_modules/@sap/approuter/
approuter.js"
}
```

If the application router is running in an environment with limited memory, set the heap limit to about 75% of available memory. For example, if the application router memory is limited to 256MB, add the following command to your `package.json`:

Sample Code

```
"scripts": {
  "start": "node --max-old-space-size=192 node_modules/@sap/approuter/
approuter.js"
}
```

Note

For detailed information about memory consumption in different scenarios, see the Sizing Guide for the Application Router located in `approuter/approuter.js/doc/sizingGuide.md`.

CSRF Protection

The application router enables CSRF protection for any HTTP method that is not `GET` or `HEAD` and the route is not public. A path is considered public, if it does not require authentication. This is the case for routes with `authenticationType: none` or if authentication is disabled completely via the top level property `authenticationMethod: none`.

To obtain a CSRF token one must send a `GET` or `HEAD` request with a `x-csrf-token: fetch` header to the application router. The application router will return the created token in a `x-csrf-token: <token>` header, where `<token>` will be the value of the CSRF token.

If a CSRF protected route is requested with any of the above mentioned methods, `x-csrf-token: <token>` header should be present in the request with the previously obtained token. This request must use the same session as the fetch token request. If the `x-csrf-token` header is not present or is invalid, the application router will return status code “403 - Forbidden”.

Cloud Connectivity

The application router supports integration with SAP Connectivity service. The connectivity service enables you to manage proxy access to Cloud Connector, which you can use to create tunnels for connections to systems located in private networks, for example, on-premise. To use the connectivity feature, you must create an instance of the connectivity service and bind it to the `Approuter` application.

In addition, the relevant destination configurations in the `<destinations>` environment variable must have the proxy type `"onPremise"`, for example, `"proxyType": "onPremise"`. You must also ensure that you obtain a valid XSUAA logon token for the XS advanced User Account and Authentication service.

Troubleshooting

The application router uses the `cf-nodejs-logging-support` package, giving you access to all its logging control features. For example, you can set all logging and tracing to the finest level by setting the `CF_NODEJS_LOGGING_LEVEL` environment variable to `debug`.

If you've deployed the application on Cloud Foundry, you can change the log level and restart the application using the following command:

> ### {·} Sample Code
>
> ```
> cf set-env <application-name> CF_NODEJS_LOGGING_LEVEL debug
> ```

You can enable additional traces of incoming and outgoing requests by setting the `<REQUEST_TRACE>` environment variable to `true`. When enabled, basic information will be logged for every incoming and outgoing request of the application router. Note that this could impact performance.

Some libraries used by the `cf-nodejs-logging-support` package use other tracing mechanisms. For example, you can use the `debug` package. By setting the `DEBUG` environment variable, you can enable additional traces.

> ### ⓘ Note
>
> Enabling the debug log level could lead to a very large amount of data being written to the application logs and trace files. The asterisk wild card (*) enables options that trace sensitive data that is then written to the logs and traces.

In addition, you can enable internal Node.js traces via the `NODE_DEBUG` environment variable.

> ### ⓘ Note
>
> Be cautious when enabling some of these options as they may trace security-sensitive data.

The `cf-nodejs-logging-support` package sets the `x-request-id` header in the application router's responses. This is useful if you want to search entries related to a particular request execution in the logs and traces of the application router. Note that the application router doesn't change the headers it receives from the backend and forwards to the client. If the backend is a `Node.js` application using the `cf-nodejs-logging-support` package (and also sets the `x-request-id` header), the header value that the client receives will be the one from the backend, not the one from the application router.

> ### Note
>
> You can also use the `XS_APP_LOG_LEVEL` environment variable for backward compatibility in SAP HANA extended application services, advanced model (XS advanced). If you've deployed the application on the XS advanced runtime for SAP HANA Platform 2.0, you can change the log level and restart the application using the following command:
>
> #### Sample Code
>
> ```
> xs set-env <application-name> XS_APP_LOG_LEVEL debug
> ```

> ### Tip
>
> Logging levels are application-specific and case-sensitive; they can be defined with lower-case characters (for example, “debug”) or upper-case characters (for example, “DEBUG”). An error occurs if you set a logging level incorrectly, for example, using lower-case characters “debug” where the application defines the logging level as “DEBUG”.

Related Information

Routing Configuration File [page 464]
Application Router [page 453]
Resource Files [page 457]

4.1.4.2.2.4 Routing Configuration File

The routing configuration defined in the `xs-app.json` file contains the properties used by the application router.

The following example of an `xs-app.json` application descriptor shows the JSON-compliant syntax required and the properties that either must be set or can be specified as an additional option.

Code Syntax

```
{
  "welcomeFile": "index.html",
  "authenticationMethod": "route",
  "sessionTimeout": 10,
  "pluginMetadataEndpoint": "/metadata",
  "routes": [
    {
      "source": "^/sap/ui5/1(.*)$",
      "target": "$1",
      "destination": "ui5",
      "csrfProtection": false
    },
    {
      "source": "/employeeData/(.*)",
      "target": "/services/employeeService/$1",
```

```
      "destination": "employeeServices",
      "authenticationType": "xsuaa",
      "scope": ["$XSAPPNAME.viewer", "$XSAPPNAME.writer"],
      "csrfProtection": true
    },
    {
      "source": "^/(.*)$",
      "target": "/web/$1",
      "localDir": "static-content",
      "replace": {
        "pathSuffixes": ["/abc/index.html"],
        "vars": ["NAME"]
     },
     {
        "source": "^/user-api(.*)",
        "target": "$1",
        "service": "sap-approuter-userapi"
      }
    }
  ],
  "login": {
     "callbackEndpoint": "/custom/login/callback"
  },
  "logout": {
     "logoutEndpoint": "/my/logout",
     "logoutPage": "/logout-page.html"
  },
  "destinations": {
     "employeeServices": {
       "logoutPath": "/services/employeeService/logout",
       "logoutMethod": "GET"
     }
  },
  "responseHeaders" : [
    {"name": "Content-Security-Policy", "value": "default-src 'self'"}
  ],
  "compression": {
     "minSize": 2048
  },
  "whitelistService": {
     "endpoint": "/whitelist/service"
  },
  "websockets": {
    "enabled": true
  },
  "errorPage": [
    {"status": [400,401,402], "file": "/custom-err-4xx.html"},
    {"status": 501, "file": "/custom-err-501.html"}
  ]
}
```

The following table lists the properties that either must be set or can be specified as an additional option. Click on the links for information for each property:

Property	Type	Description
welcomeFile [page 467]	String	The Web page served by default if the HTTP request does not include a specific path, for example, index.html.
authenticationMethod [page 468]	String	The method used to authenticate user requests, for example: “route” or “none” (no authentication).

Property	Type	Description
sessionTimeout [page 468]	Number	Define the amount of time (in minutes) for which a session can remain inactive before it closes automatically (times out); the default time out is 15 minutes. **Note** We recommend using the environment variable SESSION_TIMEOUT [page 517] to configure the session timeout. If the environment variable SESSION_TIMEOUT [page 517] is set this property will be overwritten. **Note** You can configure `sessionTimeout` only in the central routing configuration file (xs-app.json) that belongs to the application router, not in an xs-app.json file that is part of an HTML5 application.
routes [page 469]	Array	Defines all route objects, for example: `source`, `target`, and, `destination`.
login [page 482]	Object	A redirect to the application router at a specific endpoint takes place during OAuth2 authentication with the User Account and Authentication service (UAA).
logout [page 482]	Object	You can define any options that apply if you want your application to have central log out end point.
destinations [page 488]	Object	Specify any additional options for your destinations.
services [page 488]	Object	Specify options for a service in your application.
responseHeaders [page 489]	Array	Add custom response headers to your application.
compression [page 490]	Object	The `compression` keyword enables you to define if the application router compresses text resources before sending them.

Property	Type	Description
pluginMetadataEndpoint [page 491]	String	Adds an endpoint that serves a JSON string representing all configured plugins.
whitelistService [page 491]	Object	Enable the allowlist service to help preventing click-jacking attacks.
websockets [page 493]	Object	The application router can forward web-socket communication. Web-socket communication must be enabled in the application router configuration.
errorPage [page 494]	Array	Errors originating in the application router show the HTTP status code of the error. It is possible to display a custom error page using the `errorPage` property.

Related Information

Application Router [page 453]
Application Router Configuration [page 458]

4.1.4.2.2.4.1 welcomeFile

The Web page served by default if the HTTP request does not include a specific path, for example, `index.html`.

XS Advanced Application-Router Parameters

Property	Type	Mandatory	Values
`welcomeFile`	String	No	HTML page, for example, `index.html`

Code Syntax

```
"welcomeFile": "index.html"
```

4.1.4.2.2.4.2 authenticationMethod

The method used to authenticate user requests, for example: “route” or “none” (no authentication).

Code Syntax

```
"authenticationMethod" : "route"
```

Property	Type	Mandatory	Values
`authenticationMethod`	String	No	• `route` (default) Authentication type is defined in the routes configuration • `none` Disables authentication for all routes

Caution

If `authenticationMethod` is set to “`none`”, logon with User Account and Authentication (UAA) is disabled.

4.1.4.2.2.4.3 sessionTimeout

Define the amount of time (in minutes) for which a session can remain inactive before it closes automatically (times out); the default time out is 15 minutes.

Note

We recommend using the environment variable SESSION_TIMEOUT [page 517] to configure the session timeout.

If the environment variable SESSION_TIMEOUT [page 517] is set this property will be overwritten.

Sample Code

```
{
  "sessionTimeout": 40
}
```

With the configuration in the example above, a session timeout will be triggered after 40 minutes and involves central log out.

A session timeout triggers a central log out with the following consequences:

- Deletes the user session
- Requests the log out paths for all your back-end services (if you provided these paths in the `destinations` and `service` properties).

> **Note**
>
> `sessionTimeout` can only be configured in the central routing configuration file (xs-app.json) that belongs to the application router, not in an xs-app.json file that is part of an HTML5 application.

4.1.4.2.2.4.4 routes

Defines all route objects, for example: `source`, `target`, and, `destination`.

- Routes Properties [page 469]
- Route Configuration Examples [page 476]

For more information about Basic authentication options, see SAP Note 3015211.

Application Router: Routes Properties

Property	Type	Mandatory	Description
`source`	RegEx	Yes	Describes a regular expression that matches the incoming request URL. A request matches a particular route when its path contains the given pattern. To ensure the RegExp matches the complete path, use the following form:^$`. **Note** Be aware that RegExp is applied to the full URL, including query parameters.
`httpMethods` [page 473]	Array of uppercase HTTP methods	No	HTTP methods that are served by this route; the supported methods are: `DELETE`, `GET`, `HEAD`, `OPTIONS`, `POST`, `PUT`, `TRACE`, and `PATCH`. **→ Tip** If this option isn't specified, the route serves any HTTP method.
`target`	String	No	Defines how the incoming request path is rewritten for the corresponding destination or static resource.

Property	Type	Mandatory	Description
`destination`	String	No	The name of the destination to which the incoming request is forwarded. The destination name can be a static string or a regular expression that defines how to dynamically fetch the destination name from the source property or from the host. For more information about additional destination properties, see Application Routes and Destinations [page 496].
`service`	String	No	The name of the service to which the incoming request is forwarded.
`endpoint`	String	No	The name of the endpoint within the service to which the incoming request is forwarded. It must only be used in a route containing a service attribute.
`localDir` [page 474]	String	No	The directory from which application router serves static content (for example, from the application's `web/` module).
`preferLocal`	Boolean	No	Defines from which subaccount the destination is retrieved. If `preferLocal` is true, the destination is retrieved from the provider subaccount. If `preferLocal` is false or undefined, the destination is retrieved from the subscriber subaccount.
`replace` [page 475]	Object	No	An object that contains the configuration for replacing placeholders with values from the environment. **Note** It is only relevant for static content.

Property	Type	Mandatory	Description
`authenticationType`	String	No	Specifies the authentication method for the route. You can use these values: • `ias`: Uses the Identity Authentication service (IAS). Users are redirected to an IAS login. • `xsuaa`: Uses the SAP Authorization and Trust Management service (XSUAA). Users are redirected to an XSUAA login. • `basic`: Uses HTTP Basic authentication. This option supports SAP HANA users, the SAP ID service, and the Identity Authentication service (IAS). • `none`: No authentication is required for the route. If you do not define the `authenticationType`, the application router selects the authentication method based on its service bindings: • If bound only to the Identity Authentication service (IAS), it uses `ias`. • If bound only to the SAP Authorization and Trust Management service (XSUAA), it uses `xsuaa`. • If the application router is bound to both IAS and XSUAA, it automatically determines which authentication service to use by calling the SAP BTP security endpoint., In this case, IAS is preferred if both services are available in the current subaccount. **Note** This automatic detection requires an additional call to the SAP BTP security endpoint, which may impact response time. For more information about Basic authentication options, see SAP Note 3015211.
`csrfProtection`	Boolean	No	Toggle whether this route needs CSRF token protection. The default value is “true”. The application router enforces CSRF protection for any HTTP request that changes state on the server side, for example: PUT, POST, or DELETE.

Property	Type	Mandatory	Description
`scope`	Array/String/Object	No	The authorization scope required to access the target path. The scope itself is defined in the application's security descriptor (`xs-security.json`), for example, `"$XSAPPNAME.Display"` or `"$XSAPPNAME.Create"`.
`cacheControl`	String	No	A string representing the value of the Cache-Control header, which is set on the response when serving static resources. By default the Cache-Control header isn't set. The cacheControl property is only effective when one of the following settings is performed: • localDir property is set • Service pointing to HTML5 Application Repository ("service": "html5-apps-repo-rt") is set
`identityProvider`	String	No	The name of the identity provider you use if it's provided in the definition of the route. If it isn't provided, the route is authenticated with the default identity provider. **Note** If `authenticationType` is set to *Basic Authentication* or *None*, don't define the `identityProvider` property.
`dynamicIdentityProvider`	Boolean	No	Enables dynamic identity provider provisioning. If `dynamicIdentityProvider` is `true`, the end user can set the identity provider (IDP) for the application's login process by filling the request query parameter `sap_idp` with the IDP origin key. If the `IdentityProvider` property is defined in the route, its value is overwritten by the `sap_idp` query parameter value. The default value for `dynamicIdentityProvider` is `false`.

Note

Route order is important. The first matching route will be used. Therefore, it is recommended to sort the routes from the most specific source to the more generic one. For example:

Sample Code

```
"routes": [
  {
```

```
    "source": "^/sap/backend/employees(.*)$",
    "target": "$1",
    "destination": "sfsf"
  } ,
  {
    "source": "^/sap/backend/(.*)$",
    "target": "$1",
    "destination": "erp",
  }

]
```

Code Syntax

```
"routes": [
  {
    "source": "^/sap/ui5/1(.*)$",
    "target": "$1",
    "destination": "ui5",
    "scope": "$XSAPPNAME.viewer",
    "authenticationType": "xsuaa",
    "csrfProtection": true
  }
]
```

Note

The properties `service`, `destination`, and `localDir` are optional. However, at least one of them **must** be defined.

`httpMethods`

The `httpMethods` option allows you to split the same path across different targets depending on the HTTP method. For example:

Sample Code

```
"routes": [
  {
    "source": "^/app1/(.*)$",
    "target": "/before/$1/after",
    "httpMethods": ["GET", "POST"]
  }
]
```

This route only serves GET and POST requests. Any other method (including extension ones) gets a `405 Method Not Allowed` response. The same endpoint can be split across multiple destinations depending on the HTTP method of the requests:

Sample Code

```
"routes": [
{
```

```
  "source": "^/app1/(.*)$",
  "destination" : "dest-1",
  "httpMethods": ["GET"]
},
{
  "source": "^/app1/(.*)$",
  "destination" : "dest-2",
  "httpMethods": ["DELETE", "POST", "PUT"]
}
]
```

This sample code routes GET requests to the target `dest-1`, DELETE, POST and PUT to `dest-2`, and any other method receives a `405 Method Not Allowed` response. It's also possible to specify `catchAll` routes, namely routes that don't specify `httpMethods` restrictions:

Sample Code

```
"routes": [
{
  "source": "^/app1/(.*)$",
  "destination" : "dest-1",
  "httpMethods": ["GET"]
},
{
  "source": "^/app1/(.*)$",
  "destination" : "dest-2"
}}
]
```

In this sample code, GET requests are routed to `dest-1`, and all of the rest are routed to `dest-2`.

`localDir`

If there's no route defined for serving static content via `localDir`, a default route is created for "`resources`" directory as follows:

Sample Code

```
{
  "routes": [
    {
      "source": "^/(.*)$",
      "localDir": "resources"
    }
  ]
}
```

Note

If there is at least one route using `localDir`, the default route isn't added.

`replace`

The `replace` object configures the placeholder replacement in static text resources.

Sample Code

```
{
 "replace": {
   "pathSuffixes": ["index.html"],
   "vars": ["escaped_text", "NOT_ESCAPED"]
 }
}
```

The `replace` keyword requires the following properties:

Replacement Properties for Static Resource URLs

Property	Type	Description
`pathSuffixes`	Array	An array defining the path suffixes that are relative to `localDir`. Only files with a path ending with any of these suffixes are processed.
`vars`	Array	A whitelist with the environment variables that are replaced in the files matching the suffix specified in `pathSuffixes`.

The supported tags for replacing environment variables are: `{{ENV_VAR}}` and `{{{ENV_VAR}}}` . If such an environment variable is defined, it's replaced; otherwise, it's just an empty string.

Note

Any variable that is replaced using two-brackets syntax `{{ENV_VAR}}` is HTML-escaped; the triple brackets syntax `{{{ENV_VAR}}}` is used when the replaced values don't need to be escaped and all values remain unchanged. For example, if the value of the environment variable is `ab"cd` the result is `ab"cd`.

If your application descriptor `xs-app.json` contains a route like the one illustrated in the following example,

```
{
 "source": "^/get/home(.*)",
 "target": "$1",
 "localDir": "resources",
 "replace": {
   "pathSuffixes": ["index.html"],
   "vars": ["escaped_text", "NOT_ESCAPED"]
 }
}
```

And your application uses the following `index.html` start file:

Sample Code

```
<html>
 <head>
  <title>{{escaped_text}}</title>
  <script src="{{{NOT_ESCAPED}}}/index.js"/>
 </head>
```

```
</html
```

Then, in the `index.html`, `{{escaped_text}}` and `{{{NOT_ESCAPED}}}` are replaced with the value defined in the environment variables `<escaped_text>` and `<NOT_ESCAPED>`.

> ### Note
>
> **All** `index.html` files are processed; if you want to apply the replacement only to specific files, you must set the path relative to `localDir`. In addition, all files must comply with the UTF-8 encoding rules.

The content type returned by a request is based on the file extension specified in the route. The application router supports the following file types:

- .json (application/json)
- .txt (text/plain)
- .html (text/html) **default**
- .js (application/javascript)
- .css (test/css)

The following table illustrates some examples of the `pathSuffixes` properties:

Examples of the pathSuffixes Property

Example	Result
`{ "pathSuffixes": [".html"] }`	All files with the extension `.html` under `localDir` and its subfolders are processed .
`{ "pathSuffixes": ["/abc/main.html", "some.html"] }`	For the suffix `/abc/main.html`, all files named `main.html` that are inside a folder named `abc` are processed. For the suffix `some.html`, all files with a name that ends with "`some.html`" are processed. For example: `some.html`, `awesome.html`.
`{ "pathSuffixes": [".html"] }`	All files with the name "`some.html`" are processed. For example: `some.html` , `/abc/some.html`.

Route Configuration Examples

Route with a `destination` and no `target`

> ### Sample Code
>
> ```
> {
>     "source": "^/app1/(.*)$",
>     "destination": "app-1"
> }
> ```

Since there is no target property for that route, no path rewriting will take place. If `/app1/a/b` is received as a path, then a request to `http://localhost:3001/app1/a/b` is sent. The source path is appended to the destination URL.

Route with case-insensitive matching

Sample Code

```
{
    "source": {
      "path": "^/app1/(.*)$",
      "matchCase": false
    },
    "destination": "app-1"
}
```

Note

The property `matchCase` must be boolean. It is optional and has a default value of `true`.

Route with a `destination` and a `target`

Sample Code

```
{
    "source": "^/app1/(.*)$",
    "target": "/before/$1/after",
    "destination": "app-1"
}
```

Route with a `service`, a `target` and an `endpoint`

Sample Code

```
{
     "source": "^/odata/v2/(.*)$",
     "target": "$1",
     "service": "com.sap.appbasic.country",
     "endpoint": "countryservice"
}
```

When a request with path `/app1/a/b` is received, the path rewriting is done according to the rules in the target property. The request will be forwarded to `http://localhost:3001/before/a/b/after`.

Note

In regular expressions there is the term capturing group. If a part of a regular expression is surrounded with parenthesis, then what has been matched can be accessed using $ + the number of the group (starting from 1). In code sample, $1 is mapped to the (.*) part of the regular expression in the source property.

Route with dynamic `destination` and `target`

Sample Code

```
{
      "source": "^/destination/([^/]+)/(.*)$",
      "target": "$2",
      "destination": "$1",
      "authenticationType": "xsuaa"
```

```
    }
```

If you have another destination configured, use this:

Sample Code

```
[
    {
    "name" : "myDestination",
    "url" : "http://localhost:3002"
    }
]
```

When a request with the path `/destination/myDestination/myTarget` is received, the destination will be replaced with the URL from `"myDestination"`, the target will get `"myTarget"` and the request will be redirected to `http://localhost:3002/myTarget`.

Note

You can use a dynamic value (regex) or a static string for `destination` and `target` values.

The Application Router first looks for the destination name in the `mainfest.yaml` file, and if it is not found, it looks for it in the destination service.

Destination in Host

For legacy applications that do not support relative URL paths, you need to define your URL in the following way to enable the destination to be extracted from the host:

`https://<tenant>-<destination>.<customdomain>/<pathofile>`

To enable the application router to determine the destination of the URL host, a `DESTINATION_HOST_PATTERN` attribute must be provided as an environment variable. For example, When a request with the path `https://myDestination.some-approuter.someDomain.com/app1/myTarget` is received, the following route is used:

Sample Code

```
{
      "source": "^/app1/([^/]+)/",
      "target": "$1",
      "destination": "*",
      "authenticationType": "xsuaa"
 }
```

In this example, the target will be extracted from the source and the `'$1'` value is replaced with `"myTarget"`. The destination value is extracted from the host and the `"*"` value is replaced with `"myDestination"`.

Route with a `localDir` and no `target`

Sample Code

```
{
    "source": "^/web-pages/(.*)$",
    "target": "$1",
    "localDir": "my-static-resources"
```

```
}
```

If we receive a request with a path `/web-pages/welcome-page.html`, the local file at `my-static-resources/welcome-page.html` under the working directory will be served.

> ### Note
>
> The capturing group used in the `target` property.

Route with `localDir` and `cacheControl`

> ### Sample Code
>
> ```
> {
>   "source": "^/web-pages/",
>   "localDir": "my-static-resources",
>   "cacheControl": "public, max-age=1000,must-revalidate"
> }
> ```

Route with service `"html5-apps-repo-rt"` and `cacheControl`

> ### Sample Code
>
> ```
> {
>   "source": "^/index.html$",
>   "service": "html5-apps-repo-rt",
>   "authenticationType": "xsuaa",
>   "cacheControl":"public,max-age=1000,must-revalidate"
> }
> ```

Route with `httpMethods` restrictions

This option allows you to split the same path across different targets depending on the HTTP method. For example:

> ### Sample Code
>
> ```
> {
>   "source": "^/app1/(.*)$",
>   "target": "/before/$1/after",
>   "httpMethods": ["GET", "POST"]
> }
> ```

This route only supports `GET` and `POST` requests. Any other method (including extensions) will receive a *405 Method Not Allowed* response. The same endpoint can be split across multiple destinations depending on the HTTP method of the requests:

> ### Sample Code
>
> ```
> {
>   "source": "^/app1/(.*)$",
>   "destination" : "dest-1",
>   "httpMethods": ["GET"]
> },
> {
> ```

```
  "source": "^/app1/(.*)$",
  "destination" : "dest-2",
  "httpMethods": ["DELETE", "POST", "PUT"]
}
```

In the setup above, `GET` requests will be routed to `"dest-1"`, and all the rest to `"dest-2"`.

> ### Note
>
> `localDir` and `httpMethods` are not compatible.

Route with a `scope`

An application specific scope uses the following format:

`<application-name>.<scope-name>`

It is possible to configure what scope the user needs to possess in order to access a specific resource. Those configurations are per route. The user should have at least one of the scopes in order to access the corresponding resource.

> ### Sample Code
>
> ```
> {
>     "source": "^/web-pages/(.*)$",
>     "target": "$1",
>     "scope": ["$XSAPPNAME.viewer", "$XSAPPNAME.reader", "$XSAPPNAME.writer"]
> }
> ```

For convenience if your route only requires one scope, the scope property can be a string instead of an array. The following configuration is also valid:

> ### Sample Code
>
> ```
> {
>     "source": "^/web-pages/(.*)$",
>     "target": "$1",
>     "scope": "$XSAPPNAME.viewer"
> }
> ```

You can configure scopes for different HTTP methods, such as `GET`, `POST`, `PUT`, `HEAD`, `DELETE`, `CONNECT`, `TRACE`, `PATCH`, and`OPTIONS`. If some of the HTTP methods are not explicitly set, the behaviour for them is defined by the default property. In case there is no default property specified and the HTTP method is also not specified, the request is rejected by default.

> ### Sample Code
>
> ```
> {
>     "source": "^/web-pages/(.*)$",
>     "target": "$1",
>     "scope": {
>       "GET": "$XSAPPNAME.viewer",
>       "POST": ["$XSAPPNAME.reader", "$XSAPPNAME.writer"],
>       "default": "$XSAPPNAME.guest"
>     }
> }
> ```

The application router supports the `$XSAPPNAME` placeholder. Its value is taken (and then substituted in the routes) from the UAA configuration.

> **Note**
>
> The substitution is case sensitive.

You can use the name of the business application directly instead of using the $XSAPPNAME placeholder:

> **Sample Code**

```
{
    "source": "^/backend/(.*)$",
    "scope": "my-business-application.viewer"
}
```

Routes with an `identityProvider`

You can define several identity providers for different types of users. In this code example, there are two categories: hospital patients and hospital personnel:

- patientsIDP – use for authenticating patients.
- hospitalIDP – use for authenticating all hospital personnel (doctors, nurses etc..).

> **Sample Code**

```
[
    {
    "source": "^/patients/sap/opu/odata/(.*)",
    "target": "/sap/opu/odata$1",
    "destination": "backend",
    "authenticationType": "xsuaa",
    "identityProvider": "patientsIDP"
    },
    {
        "source": "^/hospital/sap/opu/odata/(.*)",
    "target": "/sap/opu/odata$1",
    "destination": "backend", "authenticationType": "xsuaa",
    "identityProvider": "hospitalIDP"
    }
]
```

A patient who tries to log into the system will be authenticated by patientIDP, and a doctor who tries to log in will be authenticated by hospitalIDP.

> **Note**
>
> If a user logs in as one identity and then wants to perform tasks for another identity type, the user must log out and log back in to the system.
>
> Dynamic provisioning of the subscriber account identity provider is not supported.
>
> Identity provider configuration is only supported in the client side logon redirect flow.

Route with a `dynamicIdentityProvider`

This is an example of a route where the value of `identityProvider` is `patientsIDP` and where dynamic identity provider provisioning is enabled by setting `dynamicIdentityProvider` to true:

Sample Code

```
[
    {
        "source": "^/patients/index.html",
        "target": "/patients-index.html",
        "service": "html5-apps-repo-rt",
        "identityProvider": "patientsIDP",
        "dynamicIdentityProvider": true
    }
]
```

In this example, the `patientsIDP` value for the `identityProvider` is replaced by `hospitalIDP` if a request with `sap_idp=hospitalIDP` is executed, for example, if the request is `https://shiva.health-center-approuter.cfapps.hana.ondemand.com/healthreport/patients/index.html?sap_idp=hospitalIDP`.

4.1.4.2.2.4.5 login

A redirect to the application router at a specific endpoint takes place during OAuth2 authentication with the service that you are using for the authentication (User Account and Authentication service (UAA) or Identity Authentication service).

This endpoint can be configured in order to avoid possible collisions, as illustrated in the following example:

Sample Code

Application Router “login” Property

```
"login": {
  "callbackEndpoint": "/custom/login/callback"
}
```

→ Tip

The default endpoint is “`/login/callback`”.

4.1.4.2.2.4.6 logout

You can define a central logout end point for your application.

In this object you can define an application's central logout end point by using the `logoutEndpoint` property. The value of logout property should be an object with the following properties:

Property	Type	Mandatory	Description
`backChannelLogoutEndpoint`	String	No	The path to be used when logging out from the application router via Identity Authentication back channel logout
`logoutEndpoint`	String	Yes	The path to be used when logging out from the application router
`logoutPage`	String	No	The URL path of the logout page
`logoutMethod`	String	No	The logout method for the logoutEndpoint. This can be `POST` or `GET`. The default value is `GET`. If no logout method is defined for an HTML5 application in the xs-app.json file of the application, the application router uses the logout method that is defined in the central xs-app.json file of the application router. The logout method for the managed application router has been defined as POST. Therefore, if you use the managed application router and have not defined a logout method in the xs-app.json of your HTML5 applications, the POST method is used for your applications. **Note** For security reasons, it is recommended to use the `POST` method and to enable CSRF protection.

Property	Type	Mandatory	Description
`csrfProtection`	Boolean	No	Can only be defined if the `logoutMethod` is `POST`. If the `logoutMethod` is `POST` and this property is not defined, the default value is `true`. You can set it to `false` – for example if csrfProtection is implemented in back-end application

> ### Note
>
> When an HTML5 application is served using the HTML5 application repository service (`html5-apps-repo`), each HTML5 application inherits the logout configuration from the central approuter xs-app.json file. This means that if a logout property, such as `logoutMethod`, `logoutPage`, or `csrfProtection`, isn't explicitly set in the HTML5 application's own xs-app.json file, the value from the central application router configuration is used. If there's no central application router logout configuration either, the system defaults apply. For example, in this case, GET is used as the `logoutMethod`. To override the central application router's logout configuration for a specific HTML5 application, set the desired logout properties explicitly in that application's xs-app.json file

This is an example:

> ### Sample Code
>
```
"logout" {
  "logoutEndpoint": "/my/logout"
}
```

Making a `GET` or `POST` request to "`/my/logout`" triggers a client-initiated central logout with the following consequences:

- Deletes the user session
- Requests the logout paths for all your back-end services (if you provided these paths in the `destinations` and `service` properties).
- Redirects to the authentication service (UAA or Indentity Authentication - depending on which service you are using for the authentication), if such a service is provided, and logs out from there.

Logout Page

You can use the `logoutPage` property to specify the Web page in one of the following ways:

- **URL path**

The authentication service (UAA or Indentity Authentication - depending on which service you are using for the authentication) redirects the user back to the application router, and the path is interpreted according to the configured routes.
The `logoutEndpoint` can be called with the `skip-redirect` or `skip-redirect=true` query parameter to prevent the application router from redirecting to the Identity Authentication service (IAS) for logout after ending the application router user session. Use this configuration if the logout process is initiated by Identity Authentication . In this case, the end user initiates the single logout flow of Identity Authentication, and subsequently, the application router logout endpoint is called. The application router logout endpoint must not redirect to the Identity Authentication logout endpoint in this case. For more details, see Handle Single Logout Request from Identity Authentication
For example:

Sample Code

```
window.location.replace('/my/logout?skip-redirect')
```

The `logoutEndpoint` can be called with query parameters. For example:

Sample Code

```
window.location.replace('/my/logout?siteId=3');
```

These parameters will be appended as is to the redirect URL set by the `logoutPage` property. For example, if the logout section is:

Sample Code

```
"logout": {
    "logoutEndpoint": "/logout",
    "logoutPage": "/logoff.html"
  },
```

The redirect URL will end with:

Sample Code

```
/logoff.html?siteId=3
```

Note

The resource that matches the URL path specified in the property `logoutPage` should not require authentication; for this route, the property `authenticationType` must be set to `none`.

In the following example, `my-static-resources` is a folder in the working directory of the application router; the folder contains the file `logout-page.html` along with other static resources.

Sample Code

```
{
  "authenticationMethod": "route",
  "logout": {
    "logoutEndpoint": "/my/logout",
    "logoutPage": "/logout-page.html"
```

```
  },
  "routes": [
    {
      "source": "^/logout-page.html",
      "localDir": "my-static-resources",
      "authenticationType": "none"
    }
  ]
}
```

- **Absolute HTTP(S) path**
  The authentication service (UAA or Indentity Authentication - depending on which service you are using for the authentication) redirects the user to a page (or application) different from the application router.

  Sample Code

```
"logout": {
  "logoutEndpoint": "/my/logout",
  "logoutPage": "http://acme.com/employees.portal"
}
```

Using the POST Method for Logout

For security reasons, it is recommended to use *POST* method for logout and to enable CSRF protection.

The `logoutMethod` and `csrfProtection` properties are included in the logout property:

Sample Code

```
"logout": {
   "logoutEndpoint": "/my/logout",
   "logoutPage": "/logout-page.html",
   "logoutMethod": "POST",
   "csrfProtection": true
}
```

Note

For backward compatibility reasons, the default value of `logoutMethod` is `GET`. The `csrfProtection` property can only be set if the `logoutMethod` is `POST`. If the `logoutMethod` is `POST` and the `csrfProtection` property is not set, the `csrfProtection` is enabled by default.

In this example, the `POST` request is an AJAX request and includes a CSRF token:

Sample Code

```
async function getToken() {
  return new Promise((resolve) => {
  jQuery.ajax({
    type: "GET",
    url: 'my/logout',
    headers: {
      "X-CSRF-Token": 'fetch',
      contentType: "application/json",
    },
```

```
    success: function(data, textStatus, request){
      resolve(request.getResponseHeader('X-CSRF-Token'));
    },
   });
 });
};
```

This is an example for the `POST` request:

Sample Code

```
const token = await getToken();
jQuery.ajax({
  type: "POST",
  url: "my/logout",
  headers: {
    "X-CSRF-Token": token,
    contentType: "application/json",
  },
  success: function (data) {
    window.location.href = data;
  }
});
```

Note

Make sure that the `url` field matches the `logoutEndpoint`.

Back-Channel Logout Endpoint

If you use Identity Authentication for authentication, you can configure the back-channel logout endpoint using the `backChannelLogoutEndpoint` property. Configure this in the central `xs-app.json` file of the application router.

In the back-channel logout process, the Identity Authentication service initiates the logout. In this scenario, the application router logout endpoint must not redirect to the Identity Authentication logout endpoint. The application router back-channel logout endpoint terminates the user session, calls the destination or service logout path, and returns a response to Identity Authentication. For more information, see Handle Single Logout Request from Identity Authentication.

For multitenancy flows, configure the back-channel logout URL in the provider Identity Authentication application. This URL can't be changed in the subscribed application because it's read-only. Therefore, the back-channel logout URL must contain the provider subdomain.

Note

The `backChannelLogoutEndpoint` URL can't be the same as the `logoutEndpoint` URL.

If you use application routers with multiple instances, a configuration for session stickiness is required. In Cloud Foundry, session stickiness is enabled by default. In the Kyma runtime, it must be defined manually. To enable session stickiness in Kyma, define the `PLATFORM_COOKIE_NAME` environment variable and use it in the destination rule.

4.1.4.2.2.4.7 destinations

Specify any additional options for your destinations.

The destinations section in `xs-app.json` extends the destination configuration in the deployment manifest (`manifest.yml`), for example, with some static properties such as a logout path.

Sample Code

```
{
  "destinations": {
    "node-backend": {
      "logoutPath": "/ui5logout",
      "logoutMethod": "GET"
    }
  }
}
```

The following syntax rules apply:

Application Router: Destination Properties

Property	Type	Mandatory	Description
`logoutPath`	String	No	The log out end point for your destination. The `logoutPath` will be called when central log out is triggered or a session is deleted due to a time out. The request to `logoutPath` contains additional headers, including the JWT token.
`logoutMethod`	String	No	The `logoutMethod` property specifies the HTTP method with which the `logoutPath` will be requested, for example, POST, PUT, GET; the default value is POST

4.1.4.2.2.4.8 services

Specify options for a service in your application.

The services section in `xs-app.json` extends the services configuration in the deployment manifest (`manifest.yml`), for example, with some static properties such as an end point.

Sample Code

```
{
  "services": {
    "com.sap.appbasic.country": {
      "endpoint": "countryservice",
      "logoutPath": "/countrieslogout",
      "logoutMethod": "GET"
    }
  }
```

```
}
```

The following syntax rules apply:

Application Router: Services Properties

Property	Type	Mandatory	Description
`endpoint`	String	No	The name of the attribute in the VCAP_SERVICES that contains the URL of the service.
`logoutPath`	String	No	The log out end point for your destination. The `logoutPath` will be called when central log out is triggered or a session is deleted due to a time out. The request to `logoutPath` contains additional headers, including the JWT token.
`logoutMethod`	String	No	The `logoutMethod` property specifies the HTTP method with which the `logoutPath` will be requested, for example, POST, PUT, GET; the default value is POST

Related Information

Integration with Business Services [page 505]

4.1.4.2.2.4.9 responseHeaders

You can add headers that the application router returns to the client in its responses.

You can add response headers to your application, for example, to comply with security standards.

The `responseHeaders` property is an array of objects, each object having the following properties:

Property	Type	Description
name	string	response header name
value	string	response header value

```
{ "responseHeaders" : [
    {"name": "header1", "value": "value1"},
    {"name": "header2", "value": "value2"}
  ]
}
```

Example:

Sample Code

```
{ "responseHeaders" : [
    {"name": "Content-Security-Policy", "value": "default-src 'self'"}
  ]
}
```

Note

The response headers are added to the list of additional http headers that you have configured in the `<httpHeaders>` environment variable [page 510]. If the name of a response header configured in the routing configuration is identical with name of an additional http header in the list, the value of the response header overrides the value of the additional http header.

Related Information

Environment Variables [page 510]

4.1.4.2.2.4.10 compression

The `compression` keyword enables you to define if the application router compresses text resources before sending them.

By default, resources larger than 1KB are compressed. If you need to change the compression size threshold, for example, to “2048 bytes”, you can add the optional property `“minSize”`: `<size_in_KB>`, as illustrated in the following example.

Sample Code

```
{
  "compression": {
      "minSize": 2048
  }
}
```

You can disable compression in the following ways:

- Global
  Within the compression section add "enabled": false
- Front end
  The client sends a header “`Accept-Encoding`” which does not include “`gzip`”.
- Back end
  The application sends a header “`Cache-Control`” with the “`no-transform`” directive.

Application Router: Compression Properties

Property	Type	Mandatory	Description
`minSize`	Number	No	Text resources larger than this size will be compressed.
`enabled`	Boolean	No	Globally disables or enables compression. The default value is true.
`compressResponseMixedTypeContent`	Boolean	No	Determines whether response content of the multipart / mixed content type should be compressed. The default value is `false`.

> ### Note
>
> If the `<COMPRESSION>` environment variable is set it will overwrite any existing values.

4.1.4.2.2.4.11 pluginMetadataEndpoint

Adds an endpoint that serves a JSON string representing all configured plugins.

> ### Sample Code
>
> ```
> {
>   "pluginMetadataEndpoint": "/metadata"
> }
> ```

> ### Note
>
> If you request the relative path `/metadata` of your application, a JSON string is returned with the configured plug-ins.

4.1.4.2.2.4.12 whitelistService

Enable the allowlist service to help prevent against click-jacking attacks.

Enabling the allowlist service (which is called `whitelistService` in the code) opens an endpoint accepting GET requests at the relative path configured in the `endpoint` property, as illustrated in the following example:

> ### Sample Code
>
> ```
> {
>   "whitelistService": {
>     "endpoint": "/whitelist/service"
>   }
> }
> ```

If the allowlist service is enabled in the application router, each time an HTML page needs to be rendered in a frame, the allowlist service is used check if the parent frame is allowed to render the content in a frame.

Host Names and Domain Names

The allowlist service reads a list of allowed host names and domains defined in the environment variable `<CJ_PROTECT_WHITELIST>`; the content is a JSON list of objects with the following properties:

Allowlist of Host and Domain Names

Property	Type	Mandatory	Description
`protocol`	String	No	URI scheme, for example “HTTP”.
`host`	String	Yes	A valid host name, for example, `acme.com.hostname`, or a domain name defined with an asterisk (*) `*.acme.com`.
`port`	String/Number	No	Port string or number containing a valid port.

The following snippet shows an example of the resulting JSON array:

Sample Code

```
[
 {
  "protocol": "http",
  "host": "*.acme.com",
  "port": 12345
 },
 {
  "host": "hostname.acme.com"
 }
]
```

Note

Matching is done against the properties provided. For example, if only host name is provided, the allowlist service returns “`framing: true`” for all, and matching will be for all schemata and protocols.

Return Value

The allowlist service accepts only `GET` requests, and returns a JSON object as the response. The allowlist service call uses the parent origin as URI parameter (URL encoded) as follows:

Sample Code

```
GET url/to/whitelist/service?parentOrigin=https://parent.domain.acme.com
```

The response is a JSON object with the following properties; property `“active”` has the value `false` only if `<CJ_PROTECT_WHITELIST>` is not provided:

Sample Code

```
{
```

```
  "version" : "1.0",
  "active" : true | false,
  "origin" : "<same as passed to service>",
  "framing" : true | false
}
```

The `"active"` property enables framing control; the `"framing"` property specifies if framing should be allowed. By default, the application router (`approuter.js`) sends the `X-Frame-Options` header with value the `SAMEORIGIN`.

> → Tip
>
> If the allowlist service is enabled, the header value probably needs to be changed, see the *X-Frame-Options* header section for details about how to change it.

4.1.4.2.2.4.13 websockets

The application router can forward web-socket communication. Web-socket communication must be enabled in the application router configuration.

If the back-end service requires authentication, the upgrade request should contain a valid session cookie. The application router supports the destination schemata "`ws`", "`wss`", "`http`", and "`https`".

> {·} Sample Code

```
{
  "websockets": {
    "enabled": true
  }
}
```

To use web-sockets when the application router is integrated with the HTML5 Application Repository, the `websockets` property should be added to the xs-app.json of the deployed HTML5 application. When an incoming request for an application in the repository goes through the application router, it retrieves the application's configuration from the repository. If this flag is set, the application router creates a web-socket connection to the back end (the target url of the request) and acts as a proxy which delivers messages on top of the `ws` protocol from the back end to the user, and vice versa.

> ⚠ Restriction
>
> A web-socket ping is not forwarded to the back-end service.

4.1.4.2.2.4.14 errorPage

Errors originating in the application router show the HTTP status code of the error. It is possible to display a custom error page using the `errorPage` property.

The property is an array of objects, each object can have the following properties:

Application Router: errorPage Properties

Property	Type	Mandatory	Description
`status`	Number/Array	Yes	HTTP status code.
`file`	String	Yes	File path relative to the working directory of the application router.
`path`	String	Yes	URL path relative to the HTML application root folder

Here is an example:

Sample Code

```
{ "errorPage" : [
    {"status": [400,401,402], "file": "./custom-err-40x.html"},
    {"status": 501, "file": "./http_resources/custom-err-501.html"}
    {"status": 403, "path": "/forbidden.html"}
  ]
}
```

In this code example, errors with status code “400”, “401” and “402” will show the content of `./custom-err-4xx.html`. Errors with the status code “501” will display the content of `./http_resources/custom-err-501.html` For errors with status code 403, the user is redirected to "/forbidden.html".

To avoid errors when redirecting to an error page that is stored in the HTML5 Application Repository service, define a route without authentication to the error page. For example:

Sample Code

```
{
  "source": "^/forbidden.html$",
  "service": "html5-apps-repo-rt",
  "authenticationType": "none"
}
```

Note

The contents of the `errorPage` configuration section have no effect on errors that are not generated by the application router.

4.1.4.2.2.5 Headers

Forwarding Header

The application router sends the following `x-forwarding-` headers to the route targets:

Header Name	Description
`x-forwarded-host`	Contains the host header that is sent from the client to the application router.
`x-forwarded-proto`	Contains the protocol that is used by the client to connect to the application router.
`x-forwarded-for`	Contains the address of the client that connects to the application router.
`x-forwarded-path`	Contains the original path that the client requested.

If a client performs a path rewriting, it sends the `x-forwarded-proto`, `x-forwarded-host`, and the `x-forwarded-path` headers to the application router. The values of these headers are forwarded to the route targets without modifications instead of being generated from the application router request URL. The `x-forwarded-path` header of a request does not impact the source pattern of routes in the `xs-app.json`.

Hop-by-Hop Headers

Hop-by-hop headers are only for a single transport-level connection and are not forwarded by the application router. The headers are:

- `Connection`
- `Keep-Alive`
- `Public`
- `Proxy-Authenticate`
- `Transfer-Encoding`
- `Upgrade`

Custom Header

`x-custom-host` is used to support the application router behind an external reverse proxy. The x-custom-host header must contain the internal reverse proxy host.

> **Note**
>
> `EXTERNAL_REVERSE_PROXY` environment variable is set to true.

In a multi-tenancy landscape, application router can be called from multiple tenants. During the authentication flow, application router uses the tenant ID to fetch the authentication token from XSUAA. Application router extracts the tenant ID from the corresponding host using the tenant host pattern configuration.

In an external reverse proxy flow, the application router uses the `x-custom-host` to extract the tenant ID using the tenant host pattern configuration.

If the `x-custom-host` is not provided, the application router uses the host header to extract the tenant ID.

Authorization Header for Service to Application Router

The `x-approuter-authorization` header contains the JWT token or Open ID Connect (OIDC) access token to support the service-to-application-router scenario.

The application router can receive a JWT token created by the SAP Authorization and Trust Management service (XSUAA) service or an OIDC access token created by Identity Authentication and use it to access the UI and the data. The JWT token is passed to the application router in the `x-approuter-authorization` header of a request.

> **Note**
>
> The JWT token or the OIDC token are generated with the same SAP Authorization and Trust Management service (XSUAA) service instance or the same Identity Authentication service instance that is bound to the application router.

4.1.4.2.2.6 Application Routes and Destinations

The application router is the single point of entry for an application.

The application router is used to serve static content, propagates user information, and acts as a proxy to forward requests to other microservices. The routing configuration for an application is defined in one or more destinations. The application router configuration is responsible for the following tasks:

- Act as the main user-entry point to application service
- Serve static content
- Serve routes and destinations
- Rewrite URLs
- Forward requests to other services
- Propagate user information
- Handle authentication
- Manage HTML5 application/browser sessions

> **Note**
>
> The application router doesn't manage server caching. The server cache must be set (with e-tags) in the server itself. The cache must contain not only static content, but container resources too. For example, relating to OData metadata.

Destinations

A destination defines the back-end connectivity. In its simplest form, a destination is a URL to which requests are forwarded. There has to be a destination for every single app (microservice) that is a part of the business application.

> **Note**
>
> You must not use multiple destinations with the same URL in an application router user session!
>
> The application router stores the session cookies from the backend services in a user session according to the destination URLs for the backend services. If there are multiple destinations with the same URL, the application router cannot correctly send back the cookies to the backend services. Note that to avoid conflicts, the destination URLs must have different domains. For more information, see Sessions [page 460].

The destinations configuration can be provided by the destinations environment variable or by the destination service.

Destinations Environment Variable

Destinations can be defined in an environment variable for the `approuter` application. The destinations are typically specified in the application manifest file (`manifest.yml`). The router information is added to the `env: destinations` section of the `manifest.yml` file, as illustrated in the following example:

> **Sample Code**
>
> Destinations Defined in the Application Manifest (`manifest.yml`)

```
---
applications:
- name: node-hello-world
  port: <approuter-port>  #the port used for the approuter
  memory: 100M
  path: web
  env:
    destinations: >
      [
        {
          "name":"backend",
          "url":"http://<hostname>:<node-port>",
          "forwardAuthToken": true
        }
      ]
  services:
    - node-uaa
```

> **Note**
>
> The "`name`" and "`url`" properties are mandatory.

The value of the destination `"name":"backend"` must match the value of the `destinations` property configured for a route in the corresponding application-router configuration file (`xs-app.json`). It's also possible to define a logout path and method for the `destinations` property in the `xs-app.json` file.

Destination Service

Destination configuration can be provided by `destination service`. The destination service can be consumed by the application router after binding a destination service instance to it.

The following guidelines apply to the configuration of the mandatory properties of a destination:

Mandatory Properties of a Destination

Property	Description
Type	Only HTTP is supported.
Authentication	All authentication types are supported. **Note** • When using `basic authentication`, `User` and `Password` are mandatory. • When using `principal propagation`, the proxy type is `on-premise`. • When using `OAuth2SAMLBearerAssertion`, the `uaa.user` scope in the `xs-security.json` file is required.
ProxyType	Supported types: • `on-premise` If set, binding to SAP Connectivity service is required. • `internet` • `private-link` To check the availability of the `private-link` proxy type, see SAP Private Link Service in the SAP Discovery Center.

The following guidelines apply to the configuration of the additional properties of a destination:

Additional Properties of a Destination

Property	Description
HTML5.ForwardAuthToken	If `true`, the OAuth token is sent to the destination. The default value is `false`. This token contains the user identity, scopes, and other attributes. It's signed by the UAA so it can be used for user authentication and authorization with back-end services. **Note** If the `ProxyType` is set to `on-premise`, don't set the `ForwardAuthToken` property to `true`. If the `Authentication` is other than `NoAuthentication`, don't set the `ForwardAuthToken` property to `true`.

Property	Description
HTML5.Timeout	Positive integer representing the maximum time to wait for a response (in milliseconds) from the destination. The default value is 30000 ms. **Note** The timeout value specified also applies to the destination's log out path (if defined), which belongs to the `destination` property.
HTML5.PreserveHostHeader	If `true`, the application router preserves the host header in the back-end request. This is expected by some back-end systems like AS ABAP, which don't process `x-forwarded-*` headers.
HTML5.DynamicDestination	If `true`, the application router allows this destination to be used dynamically on the host or path level.
HTML5.SetXForwardedHeaders	If true , the application router adds X-Forwarded-(Host, Path, Proto) headers to the back-end request. The default value is `true`.
sap-client	If `true`, the application router propagates the `sap-client` and its value as a header in the back-end request. This is expected by ABAP back-end systems.
HTML5.IASDependencyName	Configures the name of the Identity Authentication dependency that is used to exchange the Identity Authentication token used for the user authentication at login. If configured, the exchanged token is then also forwarded to the backend application.
HTML5.ForwardAuthCertificates	If `true`, the certificates and key of the authentication service are added to the HTTP connection to the destination. The default value is `false`. For more information see: Mutual TLS Authentication (mTLS) and Certificates Handling [page 525].
HTML5.StrictConnectionMode	If `true`, the application router uses strict HTTP agents with conservative connection pooling settings for this destination. These settings include shorter socket timeouts and FIFO (First-In, First-Out) scheduling. This option is recommended for mTLS and certificate-based authentication scenarios. The default value is `false`.
URL.headers.<header-name>	If you provide this property, the application router propagates the specified attribute as a header in the destination.The application router retrieves the list of headers from the destination API. Existing request headers are not overwritten.

- If a destination with the same name is defined both in the environment destination and the destination service, the destination configuration loads the settings from the environment.
- If the configuration of a destination is updated in runtime, the changes are reflected automatically to the AppRouter. There's no need to restart the AppRouter.
- The destination service is only available in the Cloud Foundry environment.

- You can define destinations at the service instance level. This destination type has a higher priority than the same destination defined on the subaccount level. This enables exposing a specific destination to a specific application instead of to the entire subaccount.

Connectivity

Application Router supports integration with the SAP Connectivity service. The Connectivity service handles proxy access to the Cloud Connector, which tunnels connections to private network systems. In order to use connectivity, a connectivity service instance must be created and bound to the Application Router application. In addition, the relevant destination configurations must contain `proxyType=OnPremise` and a valid XSUAA login token must be obtained from the login flow.

Configure the IASDependencyName Property to Enable App-to-App Navigation

You can enable an application to consume the APIs of another application. For more information, see: Configure Integration Between Applications and Consume APIs from Other Applications

To enable the technical communication between the applications, the application router must exchange the Identity Authentication token that was used for the user authentication at login to the application with the Identity Authentication token used for the other application. The exchanged token is then forwarded by the application router to the backend applications.

For this, you must configure either a new destination with an `HTML5.IASDependencyName` property that points to the backend application or a destination environment variable with the `IASDependencyName` property (see Environment Variables [page 510]). When the application router has to send a request to the backend destination, the application router uses the `IASDependencyName` property to trigger the exchange of the token and the exchanged token is then forwarded to the backend application in the `Authorization` header.

Related Information

Application Router [page 453]
Resource Files [page 457]
Consuming the Destination Service
Consuming the Connectivity Service

4.1.4.2.2.7 User API Service

The application router exposes a user API that returns the details of the users who are logged in to the application.

You implement the user API by modelling an xs-app.json route [page 469].

The user API supports two endpoints:

- `/currentUser` returns all details of logged in users.
- `/attributes` returns the main user properties.

The `/currentUser` endpoint response has the following format:

Sample Code

```
{
   "firstname": "John",
   "lastname": "Doe",
   "email": "john.doe@sap.com",
   "name": "john.doe@sap.com",
   "displayName": "John Doe (john.doe@sap.com)" (The user ID in the identity
provider),
   "scopes": ["openid","user_attributes","uaa.user"] (Only if the
authentication type is “xsuaa")
}
```

The `/attributes` endpoint response has the following format:

Sample Code

```
{
   "firstname": "John",
   "lastname": "Doe",
   "email": "john.doe@sap.com",
   "name": "john.doe@sap.com" (The user ID in the identity provider),
   "scopes": ["openid","user_attributes","uaa.user"] (Only if the
authentication type is “xsuaa"),
   < user attributes including custom attributes > (Only if the
authentication type is “xsuaa")
}
```

Note

The `"name"` property is the user ID in the identity provider, which in many cases is also the email address.

Note

If you specify “xsuaa” as the authentication type for the route, the following applies:

- User scopes from the xsuaa access token are added to the response of both endpoints (`/currentUser` and `/attributes`).
- User attributes from the identity provider (IdP) chosen for the authentication are added to the response of the `/attributes` endpoint. If a custom IdP is configured for SAP Cloud Identity

Services – Identity Authentication, the custom user attributes are also added to the response of the `/attributes` endpoint. For more information about the definition of user attributes in Identity Authentication, see .User Attributes .

- To get the user attributes from the custom IdP, add the following property to xs-security.json file of the application router: `"foreign-scope-references": ["user_attributes"]`

Configuring the User API Service in the Routing Configuration File

You implement the user API by modelling an xs-app.json route [page 469] using the `sap-approuter-userapi` service .

The following example handles both endpoints:

Sample Code

```
{
    "source": "^/user-api(.*)",
    "target": "$1",
    "service": "sap-approuter-userapi"
}
```

The following example uses only the `/currentUser` endpoint:

Sample Code

```
{
    "source": "^/user-api/currentUser$",
    "target": "/currentUser",
    "service": "sap-approuter-userapi"
}
```

Related Information

routes [page 469]

4.1.4.2.2.8 Integration with HTML5 Application Repository

The application router is integrated with the HTML5 Application Repository service, to retrieve all the static content and routes (`xs-app.json`) of the HTML5 applications stored in the repository.

To integrate HTML5 Application Repository with an application router, create an instance of the `html5-apps-repo` service of the `app-runtime` plan, and bind it to the application router.

Model the `xs-app.json` routes that are used to retrieve static content from HTML5 Application Repository in the following way:

Sample Code

```
{
     "source": "^(/.*)",
     "target": "$1",
     "service": "html5-apps-repo-rt",
     "authenticationType": "xsuaa"
 }
```

In case the application router needs to serve HTML5 applications not stored in HTML5 Application Repository, it is possible to model that in the `xs-app.json` file of the application router.

When the application router is bound to HTML5 Application Repository, the following restrictions apply:

- It is not possible to implement the "first" middleware slot to provide routes dynamically.
- Only the `workingDir` option can be provided in the start of the application router.
- A mixed scenario of modeling part of the static content in a local resources folder and also retrieving static content from HTML5 Application Repository is not supported.

Related Information

4.1.4.2.2.8.1 Processing a Request at Runtime

At runtime, the application router tries to fetch the `xs-app.json` file from the HTML5 application in the HTML5 Application Repository, and to use it for routing the request.

A valid request to an application router that uses the HTML5 Application Repository must have the following format:

Sample Code

```
https://<tenantId>.<appRouterHost>.<domain>/<bsPrefix>.<appName-appVersion>/
<resourcePath>
```

For example:

Sample Code

```
https://tenant1.myapprouter.cf.sap.hana.com/comsapappcountry.countrylist/
index.html
```

Placeholder	Mandatory	Description
bsPrefix	No	Used when the application is provided by a business service bound to this approuter.
appName	Yes	Used to uniquely identify the application in HTML5 Application Repository. **Note** Must not contain dots or special characters.
appVersion	No	Used to uniquely identify a specific application version in HTML5 Application Repository. If no version is provided, the default application version will be used.
resourcePath	Yes	The path to the file as it was stored in HTML5 Application Repository.

When processing a request at runtime, the following algorithm is applied:

- If no HTML5 application is found in HTML5 Application Repository for the current request, the central application router `xs-app.json` will be used for routing.
- If the HTML5 application exists in HTML5 Application Repository but no `xs-app.json` file is returned, an error message will be issued and the request processing will be stopped.

4.1.4.2.2.8.2 Multitenancy of HTML5 Application Repository

The HTML5 Application Repository is a multitenant service. Non-public HTML5 Applications are visible only to the application providers (with provider subaccounts) and the consumers subscribed to the applications (with consumer subaccounts).

When a multitenant application router is subscribed to a subaccount, the HTML5 Application Repository app-runtime instance that is bound to the application router is returned as a dependency, which triggers the subscription to the app-runtime instance. You can also bind HTML5 Application Repository app-host service instances to the application router to enable the subscription of the corresponding HTML5 applications. During runtime, the application router creates an HTML5 Application Repository app-runtime `client_credentials` token using the tenant URL that the application router determines from the `<TENANT_HOST_PATTERN>` environment variable.

Note

The creation of the token can fail if the app-runtime instance is not subscribed to the subaccount, which happens if, for example, the application router was subscribed to the subaccount before the HTML5 Application Repository became a multitenant service. In this case the application router will create the token using the provider subaccount.

You can trigger the subscription to the HTML5 Application Repository app-runtime instance by using the SAAS Provisioning API: https://api.sap.com/api/APISaasManagerService/resource.

```
PATCH /saas-manager/v1/application/tenants/{tenantId}/subscriptions
```

> ### Note
>
> If you have an old application router version, the HTML5 Application Repository app-runtime `client_credentials` token is created by using HTML5 application provider subaccount.

Related Information

Developing Multitenant Applications in the Cloud Foundry Environment [page 401]

4.1.4.2.2.9 Integration with Business Services

Application router supports integration with Business Services, which are a flavor of reuse-services.

An SAP Business Service exposes in its binding information in a set of attributes in the VCAP_SERVICES credentials block that enable application router to serve Business Service UI and/or data.

To access business services, the following applies:

- The Business Service UI must be stored in HTML5 Application Repository to be accessible from an application router.
- The Business Service UI must be defined as "public" to be accessible from an application router in a different space than the one from which the UI was uploaded.
- The Business Service data can be served using two grant types:

Grant Type	Description
`user_token`	The application router performs a token exchange between the login JWT token and the Business Service token, and uses it to trigger a request to the Business Service endpoint.
`client_credentials`	The application router generates a `client_credentials` token and uses it to trigger a request to the Business Service endpoint.

To bind a Business Service instance to the application router, provide the following information in the VCAP_SERVICES credentials:

Information	Mandatory	Description
`sap.cloud.service`	Yes	Service name as referenced from `xs-app.json` route and business service prefix, if provided by the Business Service UI.
`sap.cloud.service.alias`	No	Short service name alias for user friendly URL business service prefix. Make sure the alias is unique in the context of the application router.
`endpoints`	No	One or more endpoints that can be used to access Business Service data.

Information	Mandatory	Description
`html5-apps-repo`	No	The `html5-apps-repo.app_host_id` contains one or more `html5-apps-repo` service instance GUIDs that can be used to retrieve Business Service UIs.
`saasregistryenabled`	No	Indicates that this Business Service supports SaaS Registry subscription. If provided, the application router returns this Business Service xsappname in the SaaS Registry `getDependencies` callback.
`grant_type`	No	The grant type that should be used to trigger requests to the Business Service. Allowed values: `user_token` (default) or `client_credentials`.
`forwardiastoken`	No	This flag that indicates if, in addition to the exchanged JWT token created by the SAP Authorization and Trust Management Service (xsuaa), the OIDC access token created by Identity Authentication should be forwarded as well. The Identity Authentication token is forwarded in the request header `x-ias-token`.
`forwardiasauthentication`	No	This flag indicates that if the login was performed using Identity Authentication, the application router will not exchange the JWT token created by the SAP Authorization and Trust Management Service (XSUAA). Instead, the OIDC access token created by Identity Authentication will be forwarded in the authorization header.
`URL.headers.<header-name>:header-value`	No	If you provide this information, the application router propagates this attribute as the header to the business service backend. Existing request headers will not be overwritten.

Information	Mandatory	Description
`IASDependencyName`	No	The name of the dependency configured between the Identity Authentication application of the application router and the Identity Authentication application of the business service. The dependency enables the application router to consume the API of the business service. If the login process uses Identity Authentication and the business service binding contains an IASDependencyName configuration, the application router uses the dependency name to perform an Identity Authentication token exchange. It then forwards the Identity Authentication token to the business service backend. In cases of multitenancy, configure the dependency between the subscribed Identity Authentication application of the application router and the subscribed Identity Authentication application of the business service. For more details, see Integrating Applications in [SAP Cloud Identity Services. If you use IASDependencyName here, you can't use the `HTML5.ForwardAuthToken` parameter in the configuration of the destination service (seeApplication Routes and Destinations [page 496]).

The value of the `endpoints` is an object containing the following properties:

Property	Type	Optional	Default	Description
url	String			URL to access the Business Service data.
timeout	Number	X	30000ms	Positive integer representing the maximum wait time for a response (in milliseconds) from the Business Service.

The following example shows how to provide the required information. This information should be provided via the `onBind` hook in the service-broker implementation:

Sample Code

```
"country": [
   {
    ...
    "credentials": {
     "sap.cloud.service": "com.sap.appbasic.country",
     "sap.cloud.service.alias": "country",
     "endpoints": {
      "countryservice": { "url": https://icf-countriesapp-test-
service.cfapps.sap.hana.ondemand.com/odata/v2/countryservice"},
      "countryconfig":  {
            "url": https://icf-countriesapp-test-
service.cfapps.sap.hana.ondemand.com/rest/v1/countryconfig",
```

```
            "timeout": 120000
        }
      },
      "html5-apps-repo": {
        "app_host_id": "1bd7c044-6cf4-4c5a-b904-2d3f44cd5569,
1cd7c044-6cf4-4c5a-b904-2d3f44cd54445"
      },
      "saasregistryenabled": true,
      "grant_type": "user_token"
    ....
```

Related Information

Accessing Business Service Data [page 508]
Accessing Business Service UI [page 509]
services [page 488]

4.1.4.2.2.9.1 Accessing Business Service Data

This section describes how the application router accesses the Business Service data.

To access Business Service data, the `xs-app.json` file should have a route referencing a specific `sap.cloud.service` or `sap.cloud.service.alias` via the service attribute. If an endpoint attribute is also modeled, it will be used to get the service URL; otherwise the fallback URL or URI attribute will be used.

Sample Code

```
"routes": [
    {
      "source": "^/odata/v2/(.*)$",
      "target": "$1",
      "service": "com.sap.appbasic.country",
      "endpoint": "countryservice"
    },
```

In order to support JWT token exchange, the login JWT token should contain the `uaa.user` scope. This requires that the `xs-security` configuration file contain a role template that references the `uaa.user` scope.

Sample Code

```
{
    "xsappname"   : "simple-approuter",
    "tenant-mode" : "shared",
    "scopes": [
        {
            "name": "uaa.user",
            "description": "UAA"
        },
        {
            "name": "$XSAPPNAME.simple-approuter.admin",
            "description": "Simple approuter administrator"
        }
```

```
    ],
    "role-templates": [
        {
            "name": "Token_Exchange",
            "description": "UAA",
            "scope-references": [
                "uaa.user"
            ]
        },
        {
            "name": "simple-approuter-admin",
            "description": "Simple approuter administrator",
            "scope-references": [
                "$XSAPPNAME.simple-approuter.admin"
            ]
        }
    ]
}
```

Related Information

Integration with Business Services [page 505]
Accessing Business Service UI [page 509]

4.1.4.2.2.9.2 Accessing Business Service UI

This section provides information about accessing Business Services UIs that are stored in HTML5 Application Repository.

Business Service UI's must be stored in HTML5 Application Repository and defined in their `manifest.json` files as `public: true` in order to be accessible from an application router application that is typically running in a different space than the Business Service space. In addition, dataSource URIs must be relative to the base URL, which means there is no need for a slash as the first character.

The following is an example `manifest.json` file of a Business Service:

Sample Code

```
{
  "sap.app": {
    "id":"com.sap.appbasic.country.list",
    "applicationVersion: {
    "version": "1.0.0"
  },
  "dataSources": {
    "mainService":{
      "uri": "odata/v2/countryservice",
      "type": "OData"
    }
  },
  "sap.cloud": {
    "public": true,
    "service": "com.sap.appbasic.country"
  }
```

```
}
```

A Business Service that exposes UI, must provide one or more `app-host` GUIDs in an `html5-apps-repo` block in VCAP_SERVICES credentials.

To access the Business Service UI, the URL request to the application router must contain a business service prefix, as in the following example of a request URL:

Sample Code

```
https://tenant1.approuter-repo-examples.cfapps.sap.hana.ondemand.com/
comsapappbasiccountry.comsapappbasicscountrylist/test/flpSandbox.html
```

In this example, `comsapappbasiccountry` is the business service prefix which matches the `sap.cloud.service` attribute in the country service VCAP_SERVICES credentials (without dots). The `comsapappbasicscountrylist` is the name of the HTML5 application as defined in the `app.id` attribute in the `manifest.json` file (without dots).

Related Information

Integration with Business Services [page 505]
Accessing Business Service Data [page 508]

4.1.4.2.2.10 Environment Variables

A list of environment variables that can be used to configure the application router.

The following table lists the environment variables that you can use to configure the application router. The table also provides a short description of each variable and, where appropriate, an example of the configuration data.

Application Router Configuration Variables

Variable	Description
`httpHeaders`	Configures the application router to return additional HTTP headers in its responses to client requests
`destinations`	Provides information about the available application (microservice) destinations.

Variable	Description
`cookies` [page 516]	The application router generates the following third-party cookies and sends them to the client: • `locationafterLogin` • `fragmentAfterLogin` • `signature` • `JSESSION ID` To control the behavior and usage of the third-party cookies, you can configure the attributes `SameSite` and `Partitioned`. For more information about third-party cookies, please see the KBA 3409306.
`SESSION_TIMEOUT`	Sets the time to trigger an automatic central log out from the User Account and Authentication (UAA) server.
`SEND_XFRAMEOPTIONS`	Sets or changes the `X-Frame-Options` header
`CJ_PROTECT_WHITELIST`	A list of allowed server or domain origins to use when checking for click-jacking attacks.
`WS_ALLOWED_ORIGINS`	A list of the allowed server (or domain) origins that the application router uses to verify requests.
`JWT_REFRESH`	Configures the automatic refresh of the JSON Web Token (JWT) provided by the User Account and Authentication (UAA) service to prevent expiry (default is 5 minutes).
`UAA_SERVICE_NAME`	Specifies the **exact** name of the UAA service to bind to an application.
`INCOMING_CONNECTION_TIMEOUT`	Specifies the maximum time (in milliseconds) for a client connection. If the specified time is exceeded, the connection is closed.
`TENANT_HOST_PATTERN`	Defines a regular expression to use when resolving tenant host names in the request's host name.
`COMPRESSION`	Configures the compression of resources before a response to the client.
`SECURE_SESSION_COOKIE`	Configures the enforcement of the `Secure` flag of the application router's session cookie.
`REQUEST_TRACE`	Enables additional traces of the incoming and outgoing requests.
EXTERNAL_REVERSE_PROXY	Indicates the use of the application router behind an external reverse proxy outside of the Cloud Foundry domain. For more information, see **Customer Header** under Headers [page 495].
`CORS`	Provides support for cross-origin requests, for example, by allowing the modification of the request header.

Variable	Description
`DIRECT_ROUTING_URI_PATTERNS`	Defines a list of URIs that are directed to the routing configuration file (xs-app.json file) of the application router instead of to a specific applicaion's xs-app.json file that is stored in the HTML5 Application Repository. This configuration improves the application loading time and monitoring options.
`EXT_SESSION_MGT`	You can configure external session management. See External Session Management [page 525].
`CF_NODEJS_LOGGING_LEVEL`	Sets the minimal logging level of the `cf-nodejs-logging-support` library of the application router.
`STATE_PARAMETER_SECRET`	Enables the use of state parameters to prevent CRFS attacks. If this environment variable is set, the application router creates a state parameter for each initial authorization request. By validating that the authentication server returns the same state parameter in its response, the application server can verify that the response did not originate from a third party.
`HTTP2_SUPPORT`	Enables the application router to start as an HTTP/2 server. **Note** To configure HTTP/2 support, you must use Cloud Foundry routes with an HTTP/2 destination protocol. See Configuring HTTP/2 Support in the Cloud Foundry Documentation. As connection-specific header fields aren't supported by the HTTP/2 protocol (see https://datatracker.ietf.org/doc/html/rfc9113), the application router removes such headers automatically when they are returned from a backend to prevent a failure of the HTTP/2 response.
`FULL_CERTIFICATE_CHAIN`	Enables the application router to send the entire chain of certificates provided in the binding of the authentication service (XSUAA or Indentity Authentication) to the backend applications. The certificates are required for mutual TLS authentication (mTLS handshake). See Mutual TLS Authentication (mTLS) and Certificates Handling [page 525].
`OWN_SAP_CLOUD_SERVICE`	An array that contains the business solutions ("SAP cloud services") that your HTML5 applications are associated with as values. This configuration enables the standalone application router to use the same standardized format for runtime URLs (`/<sap.cloud.service>.<appId>-<versionId>/`) that is also used by the managed application router. If a runtime URL contains one of the defined values in the `<sap.cloud.service>` section, the application router will recognize the value during the processing of a request.

Variable	Description
`DYNAMIC_IDENTITY_PROVIDER`	Configures a dynamic identity provider. If `DYNAMIC_IDENTITY_PROVIDER` is set to true, the end user can set the identity provider (IDP) for the application's login process. If SAP Authorization and Trust Management is used, the `sap_idp`request query parameter must be filled with the IDP origin key. If Identity Authentication is used, the `sap_idp` request query parameter must be filled with the name of the corporate identity provider configured in the administration console for SAP Cloud Identity services. Note that this is not the *display name*, but the *name* of the corporate identity provider (see Creating URL To Access Application with Specific Identity Provider). If the `identityProvider` property is defined in the route [page 469], its value will be overwritten by the `sap_idp` query parameter value. The default value for `DYNAMIC_IDENTITY_PROVIDER` is `false`.

httpHeaders

If configured, the application router sends additional HTTP headers in its responses to a client request. You can set the additional HTTP headers in the `<httpHeaders>` environment variable. The following example configuration shows how to configure the application router to send two additional headers in the responses to the client requests from the application `<myApp>`:

```
cf set-env <myApp> httpHeaders "[ { \"X-Frame-Options\": \"ALLOW-FROM http://
example.com\" }, { \"Test-Additional-Header\": \"1\" } ]"
```

or

```
cf set-env <myApp> httpHeaders '[{ "X-Frame-Options": "ALLOW-FROM http://
example.com" }, { "Test-Additional-Header": "1" }]'
```

> → Tip
>
> To ensure better security of your application set the `Content-Security-Policy` header. This is a response header which informs browsers (capable of interpreting it) about the trusted sources from which an application expects to load resources. This mechanism allows the client to detect and block malicious scripts injected into the application. A value can be set via the `<httpHeaders>` environment variable in the additional headers configuration. The value represents a security policy which contains directive-value pairs. The value of a directive is an allowlist of trusted sources.
>
> Usage of the Content-Security-Policy header is considered second line of defense. An application should always provide proper input validation and output encoding.

destinations

The destinations configuration is an array of objects that is defined in the `destinations` environment variable. A destination is required for every application (microservice) that is part of the business application. The following table lists the properties that can be used to describe the destination:

Destination Environment Variable Properties

Property	Type	Mandatory	Description
`name`	String	Yes	A unique indentifier for the destination
`url`	String	Yes	The Unique Resource Locator for the application (microservice)
`proxyHost`	String	No	The host of the proxy server used in case the request should go through a proxy to reach the destination. → Tip Mandatory if `proxyPort` is defined.
`proxyPort`	String	No	The port of the proxy server used in case the request should go through a proxy to reach the destination. → Tip Mandatory if `proxyHost` is defined.
`forwardAuthToken`	Boolean	No	If true, the OAuth token will be sent to the destination. The default value is “false”. This token contains the user identity, scopes, and some other attributes. The token is signed by the User Account and Authorization (UAA) service so that the token can be used for user-authentication and authorization purposed by backend services.
`strictSSL`	Boolean	No	Configures whether the application router should reject untrusted certificates. The default value is “true”. ⚠ Caution For testing purposes only. Do **not** use this property in production environments!

Property	Type	Mandatory	Description
`timeout`	Number	No	A positive integer representing the maximum amount of time to wait for a response (in milliseconds) from the specified destination. The default is 30000ms. → Tip The timeout specified here also applies to the logout path, `logoutPath`, if the logout path is defined, for example, in the application's descriptor file `xs-app.json`.
`proxyType`	String	No	Indicates if the destination is used to access applications in private networks (for example, on-premise) or publicly available on the Internet. Possible value: `onPremise`. If `proxyType` is not specified, the default (Internet access) is assumed. → Tip In Cloud environments, if you set the application's destination `proxyType` to `onPremise`, a binding to the SAP BTP connectivity service is required, and the `forwardAuthToken` property must not be set.
`IASDependencyName`	String	No	Configures the name of the Identity Authentication dependency that is used to exchange the Identity Authentication token used for the user authentication at login. If configured, the exchanged token is then also forwarded to the backend applications.
forwardAuthCertificates	Boolean	No	If `true`, the certificates and key of the authentication service are added to the HTTP connection to the destination. The default value is `false`. For more information see: Mutual TLS Authentication (mTLS) and Certificates Handling [page 525]

If `true`, the certificates and key of the authentication service are added to the HTTP connection to the destination. The default value is `false`. For more information see: Mutual TLS Authentication (mTLS) and Certificates Handling [page 525]

The following example shows a simple configuration for the `<destinations>` environment variable:

Sample Code

```
[
 {
   "name" : "ui5",
   "url" : "https://sapui5.example.com",
   "proxyHost" : "proxy",
   "proxyPort" : "8080",
   "forwardAuthToken" : false,
   "timeout" : 1200
 }
]
```

It is also possible to include the destinations in the `manifest.yml` file, as illustrated in the following example:

Sample Code

```
- name: node-hello-world
  memory: 100M
  path: web
  env: destinations: >
       [
        {"name":"ui5", "url":"https://sapui5.example.com"}
       ]
```

Additional Cookie Attributes

If configured, the application router sends additional cookie attributes in its responses to the client. Additional cookies attributes can be set in the `<COOKIES>` environment variable.

Example of configuration for cookies in the `manifest.yml`:

Sample Code

```
env:
   COOKIES: >
     {
       "SameSite":"None",
       "Partitioned":
            {
               "supportedPartitionAgents":"^Mozilla.*(Chrome|Chromium|)/
((109)|(1[1-9][0-9])|([2-9][0-9][0-9]))",
               "unsupportedPartitionAgents":"PostmanRuntime/7.29.2"
            }
     }
```

In this example, the application router sets the `SameSite attribute` of the cookie to *None* and the specifies a `Partitioned` attribute that is sent in the responses to the client.

Note

Currently, only the values `None` and `Lax` are supported for the `SameSite` attribute. The value `Strict` is not supported.

The `Partitioned` attribute contains two required regular expressions:

- `supportedPartitionAgents` for supported agents
- `unsupportedPartitionAgents` for unsupported agents

You can use a wildcard '(.*)' in supportedPartitionAgents to allow all agents to use the `Partitioned` attribute.

> ### Note
>
> `unsupportedPartionAgents` overwrites the configurations in `supportedPartitionAgents`. If an agent is allowed in `supportedPartitionAgents` but disallowed in `unsupportedPartitionAgents`, the `Partitioned` attribute will not be returned.

SESSION_TIMEOUT

You can configure the triggering of an automatic central log-out from the User Account and Authentication (UAA) service if an application session is inactive for a specified time. A session is considered to be inactive if no requests are sent to the application router. The following command shows how to set the environment variable `<SESSION_TIMEOUT>` to 40 (forty) minutes for the application `<myApp1>`:

```
cf set-env <myApp1> SESSION_TIMEOUT 40
```

> ### Note
>
> You can also set the session timeout value in the application's `manifest.yml` file, as illustrated in the following example:

> ### Sample Code
>
```
- name: myApp1
  memory: 100M
  path: web
  env:
    SESSION_TIMEOUT: 40
```

> ### → Tip
>
> If the authentication type for a route is set to `"xsuaa"` (for example, `"authenticationType": "xsuaa"`), the application router depends on the UAA server for user authentication, and the UAA server might have its own session timeout defined. To avoid problems caused by unexpected timeouts, it is recommended that the session timeout values configured for the application router and the UAA are identical."

SEND_XFRAMEOPTIONS

By default, the application router sends the `X-Frame-Options` header with the value “`SAMEORIGIN`”. You can change this behavior either by disabling the sending of the default header value (for example, by setting

SEND_XFRAMEOPTIONS environment variable to false) or by overriding the value, for example, by configuring additional headers (with the `<httpHeaders>` environment variable).

The following example shows how to disable the sending of the `X-Frame-Options` for a specific application, `myApp1`:

```
cf set-env <myApp1> SEND_XFRAMEOPTIONS false
```

CJ_PROTECT_WHITELIST

The `<CJ_PROTECT_WHITELIST>` specifies a list of origins (for example, host or domain names) that do not need to be protected against click-jacking attacks. This list of allowed host names and domains are used by the application router's allowlist service to protect XS advanced applications against click-jacking attacks. When an HTML page needs to be rendered in a frame, a check is done by calling the allowlist service to validate if the parent frame is allowed to render the requested content in a frame. The check itself is provided by the allowlist service

The following example shows how to add a host name to the click-jacking protection allowlist for the application, `myApp1`:

```
cf set-env <myApp1> CJ_PROTECT_WHITELIST {<protocol>, <hostname>, <portNr>}
```

The content is a JSON list of objects with the properties listed in the following table:

Allowlist of Host and Domain Names

Property	Type	Mandatory	Description
`protocol`	String	No	URI scheme, for example “HTTP”.
`host`	String	Yes	A valid host name, for example, `example.com.hostname`, or a domain name defined with an asterisk (*) `*.example.com`.
`port`	String/Number	No	Port string or number containing a valid port.

> ### Note
>
> Matching is done against the properties provided. For example, if only the host name is provided, the allowlist service matches all schemata and protocols.

```
xs set-env <myApp1> CJ_PROTECT_WHITELIST {<*.example.com>}
```

WS_ALLOWED_ORIGINS

When the application router receives an upgrade request, it verifies that the `origin` header includes the URL of the application router. If this is not the case, then an HTTP response with status 403 is returned to the client. This origin verification can be further configured with the environment variable `<WS_ALLOWED_ORIGINS>`, which defines a list of the allowed origins the application router uses in the verification process.

> ### Note
>
> The structure of the <WS_ALLOWED_ORIGINS> variable is the same as the variable <CJ_PROTECT_WHITELIST>.

```
cf set-env <myApp1> WS_ALLOWED_ORIGINS{<*.example.com>}
```

JWT_REFRESH

The JWT_REFRESH environment variable is used to configure the application router to refresh a JSON Web Token (JWT) for an application, by default, 5 minutes before the JWT expires, if the session is active.

```
cf set-env <myApp1> JWT_REFRESH 1
```

If the JWT is close to expiration and the session is still active, a JWT refresh will be triggered <JWT_REFRESH> minutes before expiration. The default value is 5 minutes. To disable the automatic refresh, set the value of <JWT_REFRESH> to 0 (zero).

UAA_SERVICE_NAME

The UAA_SERVICE_NAME environment variable enables you to configure an instance of the User Account and Authorization service for a specific application, as illustrated in the following example:

```
cf set-env <myApp1> UAA_SERVICE_NAME <myUAAServiceName>
```

> ### Note
>
> The details of the service configuration are defined in the <VCAP_SERVICES> environment variable, which is not configured by the user.

INCOMING_CONNECTION_TIMEOUT

The INCOMING_CONNECTION_TIMEOUT environment variable enables you to set the maximum time (in milliseconds) allowed for a client connection, as illustrated in the following example:

```
cf set-env <myApp1> INCOMING_CONNECTION_TIMEOUT 60000
```

If the specified time is exceeded, the connection is closed. If INCOMING_CONNECTION_TIMEOUT is set to zero (0), the connection-timeout feature is disabled. The default value for INCOMING_CONNECTION_TIMEOUT is 120000 ms (2 min).

TENANT_HOST_PATTERN

The `TENANT_HOST_PATTERN` environment variable enables you to specify a string containing a regular expression with a capturing group. The requested host is matched against this regular expression. The value of the first capturing group is used as the tenant Id. as illustrated in the following example:

```
cf set-env <myApp1> TENANT_HOST_PATTERN
```

COMPRESSION

The `COMPRESSION` environment variable enables you to configure the compression of resources before a response to the client, as illustrated in the following example:

```
cf set-env <myApp1> COMPRESSION
```

Here is a complete example of the compression environment variable:

Sample Code

```
env:
  COMPRESSION: >
      {
    "enabled": true,
    "minSize": 2048,
    "compressResponseMixedTypeContent": true
    }
```

SECURE_SESSION_COOKIE

The `SECURE_SESSION_COOKIE` can be set to *true* or *false*. By default, the `Secure` flag of the session cookie is set depending on the environment the application router runs in. For example, when the application router is running behind a router that is configured to serve HTTPS traffic, then this flag will be present. During local development the flag is not set.

Note

If the `Secure` flag is enforced, the application router will reject requests sent over unencrypted connections.

The following example illustrates how to set the `SECURE_SESSION_COOKIE` environment variable:

```
cf set-env <myApp1> SECURE_SESSION_COOKIE true
```

REQUEST_TRACE

You can enable additional traces of the incoming and outgoing requests by setting the environment variable <REQUEST_TRACE>"`true`". If enabled, basic information is logged for every incoming and outgoing request to the application router.

The following example illustrates how to set the `REQUEST_TRACE` environment variable:

```
cf set-env <myApp1> REQUEST_TRACE true
```

> → Tip
>
> This is in addition to the information generated by the Node.js package `@sap/logging` that is used by the XS advanced application router.

CORS

The `CORS` environment variable enables you to provide support for cross-origin requests, for example, by allowing the modification of the request header. Cross-origin resource sharing (CORS) permits Web pages from other domains to make HTTP requests to your application domain, where normally such requests would automatically be refused by the Web browser's security policy.

Cross-origin resource sharing (CORS) is a mechanism that allows restricted resources on a Web page to be requested from another domain (protocol and port) outside the domain (protocol and port) from which the first resource was served. The CORS configuration enables you to define details to control access to your application resource from other Web browsers. For example, you can specify where requests can originate from or what is allowed in the request and response headers. The following example illustrates a basic CORS configuration:

```
[
  {
      "uriPattern": "^\route1$",
      "allowedMethods": [
        "GET"
      ],
      "allowedOrigin": [
        {
          "host": "host.example.com",
          "protocol": "https",
          "port": 345
        }
      ],
      "maxAge": 3600,
      "allowedHeaders": [
        "Authorization",
        "Content-Type"
      ],
      "exposeHeaders": [
        "customHeader1",
        "customHeader2"
      ],
      "allowedCredentials": true
    }
]
```

The CORS configuration includes an array of objects with the following properties, some of which are mandatory:

Available Settings for CORS Options

CORS Property	Type	Mandatory	Description
`uriPattern`	String	Yes	A regular expression (RegExp) representing the source routes to which the CORS configuration applies. To ensure that the RegExp matches the complete path, surround it with ^ and $,f or example, `"uriPattern": "^\route1$"`. Defaults: none
`allowedOrigin`	Array	Yes	A comma-separated list of objects each of which contains a host name, port and protocol that are allowed by the server, for example: `[{"host": "www.example.com"}]` or `[{"host": ".example.com"}]`. **Note** Matching is case-sensitive. In addition, if no port or protocol is specified, the default is `"*"`. The default configuration is: `[{"host": "*"}]`, which means that the server allows **any** origin to access the resource.
`allowedMethods`	Array	No	A comma-separated list of HTTP methods that are allowed by the server, for example, `"GET", "POST"`. If `allowMethods` is defined but no method is specified, the default `"GET", "POST", "HEAD", "OPTIONS"` (all) applies. **→ Tip** The specified methods must be upper-case, for example, `GET`. Matching of the method type is case-sensitive.
`allowedHeaders`	Array	No	A comma-separated list of request headers that are allowed by the server. the default values are as follows: `["Origin", "Accept", "X-Requested-With", "Content-Type", "Access-Control-Request-Method", "Access-Control-Request-Headers"]`.
`maxAge`	String	No	A single value specifying the length of time (in seconds) a preflight request should be cached for. A negative value that prevents CORS filter from adding this response header to the pre-flight response. If `maxAge` is defined but no value is specified, the default time of “1800” seconds applies.

CORS Property	Type	Mandatory	Description
`exposeHeaders`	Array	No	A comma-separated list of **response** headers that are allowed to be exposed. If `exposeHeaders` is defined but no response header is specified for exposure, no default value is supplied.
`allowedCredentials`	Boolean	No	A Boolean flag that indicates whether the specified resource supports user credentials. The default setting is “true”.

It is also possible to include the CORS configuration in either the `manifest.yml` or the `manifest-op.yml` file. The code in the following example enables the CORS configuration for any route with a source URI pattern that matches the RegExp “`^\route1$`”:

Sample Code

CORS Configuration in the `manifest.yml` File

```
- name: node-hello-world
  memory: 100M
  path: web
  env:
    CORS: >
      [
        {
          "allowedOrigin":[
                           {
                               "host":"my_host",
                               "protocol":"https"
                           }
                          ],
          "uriPattern":"^/route1$"
        }
```

DIRECT_ROUTING_URI_PATTERNS

With the direct routing URI patterns configuration, you can define a list of URIs that are directed to the routing configuration file (xs-app.json file) of the application router instead of to a specific application's xs-app.json file that is stored in the HTML5 Application Repository. This configuration improves the application loading time and monitoring options because it prevents unnecessary calls to the HTML5 Application Repository.

The configuration is an array of strings or regular expressions. You have to provide only the first segment in the URL, after the approuter host. For example, for the URL `https://<approuter-host>/route1/index.html`, you enter `route1` in the direct routing URI patterns array.

Sample Code

```
env:
   DIRECT_ROUTING_URI_PATTERNS: >
     ["route1", "^route2$", "route3"]
```

CF_NODEJS_LOGGING_LEVEL

With the `CF_NODEJS_LOGGING_LEVEL` variable, you can set the minimal logging level of the `cf-nodejs-logging-support` library of the application router. The following levels are available:

- `off`
- `error`
- `warn`
- `info`
- `verbose`
- `debug`
- `silly`

The default value is `error`.

Here is a sample content for the `CF_NODEJS_LOGGING_LEVEL` environment variable:

Sample Code

```
env:
    CF_NODEJS_LOGGING_LEVEL: "debug"
```

Related Information

Application Router [page 453]
Routing Configuration File [page 464]

4.1.4.2.2.10.1 Performance Statistics

The application router provides performance statistics in an HTTP response header if the HTTP request contains an HTTP query parameter (URL parameter) `sap-statistics=true` or if the HTTP request contains an HTTP header field `sap-statistics:true`.

If an HTTP request that contains a header field or query parameter with `sap-statistics=true` reaches the application router, the application router forwards an `sap-statistics` header to the corresponding backend. If SAP statistics is implemented for the backend, the backend returns to the application router a response header containing the statistics information from the backend.

The application router returns the following statistics information in an `sap-statistics` response header:

- `total`: The time that has passed between the moment when the request entered into the application router and the moment when the application router started writing the response
- `ext` (in case of destination forwarding): The time spent in the backend

Each backend sub-component can add its own response header with duration measurements when it receives the HTTP header `sap-statistics:true`.

4.1.4.2.2.10.2 Mutual TLS Authentication (mTLS) and Certificates Handling

Mutual TLS (mTLS) is a mutual authentication mechanism. The application router supports the use of certificates for the creation of tokens and an mTLS handshake in backend connections.

To enable the mTLS authentication for backend connections, the following prerequisites must be met:

- The authentication service instance that is bound to the application router, Authorization and Trust Management service (XSUAA) or Identity Authentication (IAS), must provide a certificate chain and a private key in its credentials.
  If the private key isn't provided in the credentials, you can also configure the environment variables XSUAA_PRIVATE_KEY (Authorization and Trust Management) and IAS_PRIVATE_KEY (Identity Authentication) in the application router to provide the private key.
- The destinations for the backend applications must contain either the `HTML5.ForwardAuthCertificates` property in the configuration provided by the destination service (see Application Routes and Destinations [page 496]) or the `forwardAuthCertificates` property in the destinations environment variable (see Environment Variables [page 510]).
- In Cloud Foundry, the client certificate is propagated using the `x-forwarded-client-cert` header. To enable this, the backend URL must contain a `.cert` segment in its domain.

The application router gets the XSUAA or IAS tokens that provide the certificates chain and private key. The application router uses the certificates chain and private key from the credentials of the authentication service instance for the following:

- The application router creates the HTTP connection to backend using the private key and a chain of intermediate and client certificates to enable the mTLS handshake. If the `FULL_CERTIFICATE_CHAIN` environment variable is enabled, the application router sends the entire chain of certificates provided in the binding of the authentication service (XSUAA or IAS) to the backend applications. If this environment variable is not enabled, the application router sends only the root and intermediate certificates from the chain.
- When forwarding a request to business services, the application router uses these certificates also to create a `client_credentials` token or to exchange the login token.

4.1.4.2.2.11 External Session Management

The application router supports the backup of user sessions in an external session store. This enables a session recovery if the application router instance that stores a session crashes and another application router instance needs to continue handling the running user session.

To enable this capability the you must bind a service instance of a service that supports a fast persistence store such as Redis. When such a service is bound, the application router backs up the in memory session information into the external persistency store. If, in subsequent requests, the session information is not found in the in memory session store, the application router tries to rebuild the in memory session store session information from external persistency store.

The sessions are stored encrypted and compressed. For capacity planning, you can assume 50 Kb per session storage in fast persistence store.

Configuration of External Session Management

To use external session management, you have to the `EXT_SESSION_MGT` environment variable. The variable value is defined in JSON format, providing the following properties:

- `instanceName` (mandatory): the name of the service instance of the storage service
- `storageType` (mandatory): the type of the storage, for example - "redis".
  Note that if no custom storage driver is used, only `redis` is allowed.
- `sessionSecret` (mandatory): Since the application router stores encrypted sessions in a persistence store, a shared secret shall be provided. Please generate a unique string with at least 64 characters.

For example:

Sample Code

```
{
    "instanceName": "approuter-redis",
    "storageType": "redis",
    "sessionSecret": "someuniquesessionsecret"
}
```

Note

Currently, the application router supports only a Redis store.

Configuration of a Custom Storage Driver

You can use your own driver. In order to do that, the user injects its own implementation of a store.

The class has to implement the following interface:

Sample Code

```
interface UserCustomStore {
  // delete all sessions
  clear(): Promise<void>;
  // remove <sessionId> session
  destroy(sessionId : string): Promise<void>;
  // retrieve <sessionId> session
  get(sessionId : string): Promise<object | null>;
  // number of sessions
  length(): Promise<number>;
  // get <sessionId> expiration
  ttl(sessionId : string): Promise<number>;
  // set <sessionId> data to <session> with <timeout> expiration
  set(sessionId: string, session: object, timeout: number): Promise<void>;
  // check if session <sessionId> exists
  exists(sessionId: string): boolean;
  // update existing session <sessionId> expiration to <timeout>
  resetTimer(sessionId: string, timeout: number);
}
```

In addition, the file must include a method to get an instance of the store, for example:

Sample Code

```
let store;
module.exports.getStore = () => {
  if (!store) {
    store = new UserCustomStore();
  }
  return store;
};
```

In order for application router to use it, user has to set the `externalStoreFilePath` property in the `EXT_SESSION_MGT` environment variable with the path to the storage. The application router uses this path to require your storage For example:

Sample Code

```
{
    "instanceName": "approuter-redis",
    "storageType": "redis",
    "sessionSecret": "someuniquesessionsecret",
    "externalStoreFilePath": "./src/storage/my-special-storage"
}
```

Related Information

Environment Variables [page 510]

4.1.4.2.2.12 Multitenancy

Each multitenant application has to deploy its own application router, and the application router handles requests of all tenants to the application. The application router is able to determine the tenant identifier out of the URL and then forwards the authentication request to the tenant User Account and Authentication (UAA) service and the related identity zone.

To use a multitenant application router, you must have a shared UAA service and the version of the application router has to be greater than 2.3.1.

The application router must determine the tenant-specific subdomain for the UAA that in turn determines the identity zone, used for authentication. This determination is done by using a regular expression defined in the environment variable `TENANT_HOST_PATTERN`.

`TENANT_HOST_PATTERN` is a string containing a regular expression with a capturing group. The request host is matched against this regular expression. The value of the first capturing group is used as the tenant subdomain.

If you have multiple routes to the same application, for example:

`tenant1.<application domain>` and `tenant2.<application domain>`

The `TENANT_HOST_PATTERN` could be:

```
TENANT_HOST_PATTERN: "^(.*).<application domain>"
```

With this configuration, the application router extracts the tenant subdomain, which is used for authentication against a multitenant UAA.

Register an Application (SaaS Registry Configuration)

Create a service instance of the SaaS Provisioning service with a configuration JSON file in the Cloud Foundry sub-account space where the multitenant business application is deployed.

The configuration JSON file must use the following format and set these properties:

```
{
   "xsappname" : "<xsappname>",
   "appUrls": {
      "getDependencies" : "<approuter-host>/<getDependenciesPath>/{tenantId}",
      "onSubscription" : "<approuter-host>/<onSubscriptionPath>/{tenantId}"
   },
   "displayName" : "<displayName>",
   "description" : "<description>",
   "category" : "<category>"
}
```

Specify the following parameters:

Parameters	Description
`xsappname`	The `xsappname` configured in the security descriptor file used to create the XSUAA instance (see Develop the Multitenant Application [page 405]).
`getDependenciesPath`	(Optional) Any URL that the application exposes for `GET` dependencies. If the application doesn't have dependencies and the callback isn't implemented, it mustn't be declared.
`onSubscriptionPath`	This parameter must end with `/{tenantId}`. The tenant for the subscription is passed to this callback as a path parameter. You must keep `{tenantId}` as a parameter in the URL so that it's replaced at runtime with the tenant calling the subscription. This callback URL is called when a subscription between a multitenant application and a consumer tenant is created `(PUT)` and when the subscription is removed `(DELETE)`.
`displayName`	(Optional) The display name of the application when viewed in the cockpit. For example, in the application's tile. If left empty, takes the application's technical name.
`description`	(Optional) The description of the application when viewed in the cockpit. For example, in the application's tile. If left empty, takes the application's display name.
`category`	(Optional) The category to which the application is grouped in the *Subscriptions* page in the cockpit. If this parameter is left empty, it's assigned to the default category.

> ⓘ Note
>
> For information about how to model the security settings for activating the `saas-registry` callbacks, see Develop the Multitenant Application [page 405].

Related Information

Developing Multitenant Applications in the Cloud Foundry Environment [page 401]
Register the Multitenant Application to the SAP SaaS Provisioning Service [page 413]

4.1.4.2.2.13 Extending the Application Router

Configure application-specific extensions for the application router.

Instead of starting the application router directly, you can configure your XS advanced application to use its own start script. You can also use the application router as a regular Node.js package.

> {·} Sample Code

```
var approuter = require('@sap/approuter');
var ar = approuter();
ar.start();
```

Custom Middleware Injection

The application router uses the connect framework, for more information, see *Connect framework* in the *Related Links* below. You can reuse all injected "connect" middleware within the application router, for example, directly in the application start script:

> {·} Sample Code

```
var approuter = require('@sap/approuter');
var ar = approuter();
ar.beforeRequestHandler.use('/my-ext', function myMiddleware(req, res, next) {
  res.end('Request handled by my extension!');
});
ar.start();
```

> → Tip
>
> To facilitate troubleshooting, always provide a name for your custom middleware.

The `path` argument is optional. You can also chain `use` calls.

Sample Code

```
var approuter = require('@sap/approuter');
var morgan = require('morgan');
var ar = approuter();
ar.beforeRequestHandler
  .use(morgan('combined'))
  .use('/my-ext', function myMiddleware(req, res, next) {
    res.end('Request handled by my extension!');
  });
ar.start();
```

The application router defines the following slots where you can insert custom middleware:

- `first` - right after the connect application is created, and before any application router middleware. At this point security checks are not performed yet.

  > → Tip
  >
  > This is good place for infrastructure logic, for example, logging and monitoring.

- `beforeRequestHandler` - before standard application router request handling, that is static resource serving or forwarding to destinations.

  > → Tip
  >
  > This is a good place to handle custom REST API requests.

- `beforeErrorHandler` - before standard application router error handling.

  > → Tip
  >
  > This is a good place to capture or customize error handling.

If your middleware does not complete the request processing, call `next` to return control to the application router middleware, as illustrated in the following example:

Sample Code

```
ar.beforeRequestHandler.use('/my-ext', function myMiddleware(req, res, next) {
  res.setHeader('x-my-ext', 'passed');
  next();
});
```

Application Router Extensions

An extension is defined by an object with the following properties:

- `insertMiddleware` - describes the middleware provided by this extension
  - `first`, `beforeRequestHandler`, and `beforeErrorHandler`
    An array of middleware, where each one can be either of the following elements:
    - A middleware function (invoked on all requests)

- An object with the following properties:
  - `path`
    Handle requests only for this path
  - `handler`
    The middleware function to invoke

Sample Code

Example Approuter Extension Configuration (`my-ext.js`)

```
module.exports = {
  insertMiddleware: {
    first: [
      function logRequest(req, res, next) {
        console.log('Got request %s %s', req.method, req.url);
      }
    ],
    beforeRequestHandler: [
      {
        path: '/my-ext',
        handler: function myMiddleware(req, res, next) {
          res.end('Request handled by my extension!');
        }
      }
    ]
  }
};
```

The extension configuration can be referenced in the corresponding application's start script, as illustrated in the following example:

Sample Code

```
var approuter = require('@sap/approuter');
var ar = approuter();
ar.start({
  extensions: [
    require('./my-ext.js')
  ]
});
```

Customize Command Line

By default the application router handles its command-line parameters, but that can be customized as well.

An `<approuter>` instance provides the property `cmdParser` that is a commander instance. It is configured with the standard application router command line options. You can add custom options like this:

Sample Code

```
var approuter = require('@sap/approuter');
var ar = approuter();
var params = ar.cmdParser
  // add here custom command line options if needed
  .option('-d, --dummy', 'A dummy option')
```

```
  .parse(process.argv);
console.log('Dummy option:', params.dummy);
```

To disable the handling of command-line options in the application router, reset the `ar.cmdParser` property to "`false`", as illustrated in the following example:

```
ar.cmdParser = false;
```

Related Information

Middleware Connect Framework
Extension API of the Application Router [page 532]

4.1.4.2.2.14 Extension API of the Application Router

A detailed list of the features and functions provided by the application router extension API.

The application router extension API enables you to create new instances of the application router, manage the approuter instance, and insert middleware using the Node.js "connect" framework. This section contains detailed information about the following areas:

- Approuter Extension API Reference
- Middleware Slot

Approuter Extension API Reference

The application router uses the "Connect" framework for the insertion of middleware components. You can reuse all connect middleware within the application router directly.

Approuter Extension API Functions

`approuter()`	Creates a new instance of the application router
`first`	Defines a "middleware slot" (a slot for the insertion of middleware) immediately after the `connect` application is created, and before any application router middleware → Tip This is a good place to insert infrastructure logic, for example, logging and monitoring.

`beforeRequestHandler`	Defines a “middleware slot” before the standard application router request handling, that is; static resource serving or forwarding to destinations. → Tip This is a good place to handle custom REST API requests.
`beforeErrorHandler`	Defines a “middleware slot” before the standard application router error handling → Tip This is a good place to capture or customize error handling.

`start(options, callback)`	Starts the application router with the given options.
	• `options` this argument is optional. If provided, it should be an object which can have any of the following properties:
	  • `port` - a TCP port the application router will listen to (string, optional)
	  • `workingDir` - the working directory for the application router, should contain the `xs-app.json` file (string, optional)
	  • `extensions` - an array of extensions, each one is an object as defined in Application Router Extensions (optional)
	  • `xsappConfig` - An object representing the content which is usually put in `xs-app.json` file. If this property is present it will take precedence over the content of `xs-app.json`.
	• `callback` - optional function with signature `callback(err)`. It is invoked when the application router has started or an error has occurred. If not provided and an error occurs (for example the port is busy), the application will abort.

`close(callback)`	Stops the application router. • `callback` - optional function with signature `callback(err)`. It is invoked when the application router has stopped or an error has occurred.

Middleware Slot

Manage the insertion of middleware slots with the application router.

`use(path, handler)`	Inserts a request handling middleware in the current slot. • `path` - handle only requests starting with this path (string, optional) • `handler` - a middleware function to invoke (function, mandatory) Returns `this` for chaining.

Related Information

Middleware Connect Framework
Extending the Application Router [page 529]

4.1.4.2.3 Managed Application Router

The managed application router enables you to access and run HTML5 applications in a cloud environment without the need to maintain your own runtime infrastructure.

The managed application router is the HTML5 applications runtime capability that is provided by the following products:

- SAP Build Work Zone, standard edition
- SAP Build Work Zone, advanced edition
- SAP Cloud Portal

To use the managed application router, you must be subscribed to one of these services.

For information how to develop HTML5 applications and run them using the managed application router, see the following product-specific guides:

- Developing HTML5 Applications for **SAP Build Work Zone, standard edition**
- Developing HTML5 Applications for **SAP Build Work Zone, advanced edition**
- Developing HTML5 Applications for **SAP Cloud Portal**

4.1.4.2.4 Sizing Guide

This guide explains how much application runtime you must purchase to run HTML5 applications.

To develop and run HTML5 Applications using a standalone application router on Cloud Foundry, customers must purchase the Cloud Foundry application runtime.

The memory consumption for an idle application router is around 50 MB. An application router process should run with at least 256MB memory. It may require more memory depending on the application. These aspects influence the memory usage:

- Number of concurrent users. The number of users that log on to the applications at the same time has the greatest impact on the required runtime.
- Number of active sessions
- JWT token size
- Backend session cookies

Using the following table, estimate how much application runtime allocation you need, based on t-shirt sizes and the expected number of concurrent users.

T-Shirt Size	Concurrent Users	Required Application Runtime (in GB)
S	1000	1
M	10000	4
L	30000	8
XL	80000	16

Testing the Required Memory

The following tests provide measurements for different application router scenarios. You can use the measurements to approximately calculate the amount of memory that will be required by the application router. The tables contain the exact results from the measurements with Node.js v6.9.1. We recommend assuming higher numbers for productive use.

All measurements were done with authentication. If you have additional session content and want to count the session memory consumption, take a look at what is stored in the session (see "Sessions" in Application Router Configuration [page 458]). You must add the calculated session size taking into account the number of different users and the session timeout. In our tests,the JWT token alone required approximately 4 KB.

Testing the HTTP Traffic

We tested two different test scenarios

- Scenario 1: A 'Hello World' static resource is served.
- Scenario 2:
  - A 'Hello World' static resource is served.
  - A static resource of 84.78 KB (compressed by application router to 28.36 KB) is served.
  - A backend application which returns a payload of 80 kb (compressed by application router to 58 KB) is called.
  - Another backend which returns a payload of 160 KB (compressed by application router to 116 KB) is called.

Results for HTTP Traffic Tests

Memory Limit	Max. Number of Sessions - Scenario 1	Max. Number of Sessions - Scenario 2
256 MB	5300	800

Memory Limit	Max. Number of Sessions - Scenario 1	Max. Number of Sessions - Scenario 2
512 MB	13300	2300
1 GB	30100	8400
2 GB	65500	19500
4 GB	134900	46400
8 GB	275500	102300

Testing Web Socket Traffic

We tested two different test scenarios:

- Scenario 1:
  - A 'Hello World' static resource is served.
  - A single 'Hello' message is sent and then received through a web socket connection.
- Scenario 2:
  - A 'Hello World' static resource is served.
  - A static resource of 84.78 KB (compressed by the application router to 28.36 KB) is served.
  - A backend which returns a payload of 80 KB over a web socket is called.
  - Another backend which returns a payload of 160 KB over a web socket is called.

> **Note**
>
> Web sockets require a certain number of file handles to be available for the process. This number is approximately two times the number of the sessions. In Cloud Foundry, the default value is 16384.

Results for Web Socket Traffic Tests

Memory Limit	Max. Number of Sessions - Scenario 1	Max. Number of Sessions - Scenario 1
256 MB	600	300
512 MB	1100	500
1 GB	3100	800
2 GB	6500	1400
4 GB	13300	2900
8 GB	20700	6100

> **Note**
>
> `--max-old-space-size` restricts the amount of memory used in the JavaScript heap. Its default value is below 2 GB. To use the full resources that has been provided to the application, the value of this restriction should be set to a number equal to the memory limit of the whole application.
>
> For example, if the application memory is limited to 2 GB, set the V8 heap limit in the `package.json` file as follows:
>
> ```
> "scripts": {
>            "start": "node --max-old-space-size=2048 node_modules/@sap/
> approuter/approuter.js"
>       }
> ```

4.1.4.2.5 Getting Support

If you have questions or encounter an issue while working with HTML5 Application Repository, there are various ways to address them.

Create an incident in the SAP Support Portal using one of the following components:

Service	Component
HTML5 Application Repository	BC-CP-CF-HTML5
Application Router	BC-CP-APR

Provide the following incident details:

HTML5 Application Repository

Enter and attach the following to the incident;

- Subaccount HTML5 Application Repository/app-host entitlement
- Requested app-host size limit
- Upload your mtad.yaml file
- Upload your html5-app-deployer application logs (cf logs <html5-app-deployerAppName> --recent)

Application Router

Enter and attach the following to the incident;

- Version
- Working with or without HTML5 Apps Repo
- Upload your Application Router xs-app.json file
- Upload your Application Router xs-security.json file or configuration
- Application Router environment (cf env <approuterApp> or from cockpit).

  > ### Note
  >
  > The credentials(clientid/clientsecret) should be removed.

- Upload your Application Router logs

Related Information

Getting Support for SAP BTP [page 3683]
Troubleshooting [page 538]

4.1.4.2.6 Troubleshooting

A troubleshooting guide for HTML5 application repository.

Uploading Applications

400: Application Metadata Already Exists When Uploading HTML5 Applications

Term	Description
Issue	The HTML5 application deployment fails with the error "Application metadata for application xyz already exists."
Cause	There is already an `app-host` service instance that contains a `manifest.json` file with `app.id = xyz` in this space. It is not possible to have multiple `app-host` instances containing the same `app.id`.
Solution	Delete or do not deploy the old `app-host` instance or use another `app.id` in the resources folder for a new `app-host` instance.

400: Failed to Run the Application Router After Subscription; Route Not Found

Term	Description
Issue	After subscribing to the application router using the <subdomain>-<myapprouter>.<scp domain> format, calling the application router fails with the error "route not found".
Cause	To support subscriptions out of the box, a route with format *.<custom-domain> should be created and mapped to the application router. The * host represents a wildcard that is replaced by the actual subscriber subdomain during runtime. Without a custom domain, this type of route (wildcard host) cannot be created; only a fixed host can be used. Without a custom domain, the route format is <subdomain>-<myapprouter>.<scp domain>. It means that you have to create a route with the expected subdomain for each new subscriber.

Term	Description
Solution	In development setups, if you don't have a custom domain before subscription, then create a route with the expected subscriber subdomain.

400: Uploading Application Content Failed

Term	Description
Issue	When trying to upload content, it fails with error: "Upload failed".
Cause	One or more of the input validations performed by HTML5 application repository failed. The possible validation failures are: 1. Missing manifest.json file on the root level. 2. manifest.json app.id has invalid characters (hyphens, @, %, &, etc.). 3. manifest.json app.version is not using the following format: *xx.xx.xx*, where x must be an integer (e.g.: -snapshot is not supported). 4. app.id of one or more of the applications already exists in another html5-apps-repo/app-host service instance in the same space.
Solution	Check if one of the causes is your issue. For example, check the size of your html5-app-deployer resources folder, check manifest.json. If you are not sure if another service instance already uses your app.id, try making a small change to your app.id and deploy it again.

403: app-host Modified by Another Process

Term	Description
Issue	HTML5 application repository deployment fails with error "app-host is being modified by another process."
Cause	The HTML5 application repository deployer attempts to upload content while another deployer is also uploading content using the same app-host.
Solution	Try again after the first HTML5 application repository deployer has completed its upload.

409: app-host Deploy or Redeploy Remains in Progress

Term	Description
Issue	HTML5 application repository deployment fails with error "Deploy in progress" or "Redeploy in progress" and response type 409 for a long time or any other issue.

Term	Description
Cause	If something happens during the upload, it might cause some inconsistencies.
Solution	Delete the content of one or more multiple app-hosts, and reset the state to initial without deleting the service instances. Use the following command line in the CF CLI HTML5 application repository plug-in: `cf html5-delete --content <app-host-id> [...]`

Failed to Upload Content; Maximum File Length Exeeded

Term	Description
Issue	When trying to upload content, it fails with error: "Error while parsing request; Error: maximum file length exceeded".
Cause	The zipped application file length exceeds the size limit of the app-host service instance.
Solution	Increase the app-host size limit using the update service. For example: `cf update-service my-app-host -c '{"sizeLimit":100}'`.

Timeout While Uploading Content

Term	Description
Issue	When trying to upload content using the `cf push` command, it fails with error: "Failed to make TCP connection to port 8080: connection refused. Timed out after 1m0s: health check never passed."
Cause	The `cf push` plugin checks if the start process is finished. If the process times out, then it tries to start it again.
Solution	Set `health-check-type` to `none` in the manifest.yaml of the HTML5 Application Deployer.

Uploading Application Content Failed; Application Size Exceeds the Maximum Size Limit of 100 MB

Term	Description
Issue	When trying to upload content, you receive the following error: "Uploading application content failed: application's size exceeds the maximum size limit of 100 MB".
Cause	The unzipped applications content exceeds the size limit (deprecated) of the app-host service instance.
Solution	Deploy an application that is less than 100 MB or remove some of your applications and move them to another app-host.

Running Applications

400: Failed to Retrieve xs-app.json; Invalid app-host ID

Term	Description
Issue	Serving content from the HTML5 application repository fails with error "Invalid App Host ID. Please check with business service provider if the requested App Host ID is valid".
Caution	HTML5 application repository belongs to a business service and the app host ID is invalid or incompatible.
Solution	Ask the business service owner to define `"public" : true` in the app manifest.json or wait until the HTML5 application repository is restarted.

403: Failed to Retrieve xs-app.json; Unauthorized

Term	Description
Issue	Serving content from the application router fails with error "Unauthorized. Please check with the business service UI provider if the requested UI is defined as public".
Cause	HTML5 application repository belongs to a business service and it is not public.
Solution	Ask the business service owner to define `"public": true` in the app manifest.json.

404: Resource Not Found - Backend Data Resource Not Found

Term	Description
Issue	A backend endpoint cannot be found. Your browser displays the message: “404 resource not found.”
Cause	The UI sends a request with a non-relative URL. As a result, the application key section in the URL is dropped. Without the application key, the application router cannot retrieve the xs-app.json file for the HTML5 application. Instead, it uses the central xs-app.json file.

Term	Description
Solution	Check if the request URL to the application router includes the application key. If the URL does not include the application key, review how you configured backend application data retrieval in your HTML5 application. Make sure you send a relative URL to the application router: • If you use SAP Fiori tools, such as SAP Business Application Studio, check the `dataSources.uri` property in the `manifest.json` file. The value for `dataSources.uri` property must **not** start with a slash ("/"). For example: `northwind/V2/Northwind.svc` is correct, but `/northwind/V2/Northwind.svc` is incorrect. A leading slash creates an absolute path, which prevents the browser from adding the application key to the request URL. If there is a leading slash, remove it. • If you use a JQueryAjax call for the request, make sure the URL provided to the JQueryAjax call is a relative path and does not start with a slash ("/").

404: Resource Not Found - Static Application Content Is Not Retrieved

Term	Description
Issue	The application router cannot retrieve the static application content. Your browser displays the message: “404 resource not found.”
Cause	The application name in the request URL used to consume the application content is incorrect. Often, the application name includes periods (“.”) as separators, which is not allowed.
Solution	Check the URL you use to consume the application content. Make sure you provide the correct application name in the URL. Application names in this URL must **not** contain periods (“.”) as separators. For example: If the `app.id` in the `manifest.json` file is `country.list`, use `countrylist` as the application name in the URL

404: Calls to Service Endpoints Specified in an Application's xs-app.json File Fails

Term	Description
Issue	You have defined routes in a UI app's local xs-app.json but calls do not get routed and instead return a 404 error.

Term	Description
Cause	The routes in the xs-app.json file are processed top to bottom. If a route maps the pattern, it is picked even if a route below it is a better match. The route for the HTML5 application repository typically is a "catch all" route, and if any routes are defined below it, then they are never reached.
Solution	Move the route leading to the html5-apps-repo-rt to be the last entry in the xs-app.json file.

500: Failed to Retrieve xs-app.json File

Term	Description
Issue	Serving content fails with error "Error while retrieving xsApp configuration".
Cause	The HTML5 application repository is not available.
Solution	Wait until HTML5 application repository is restarted.

500: Failed to Use Dynamic Destination

Term	Description
Issue	Serving content from the application router failed with an internal server error. In the application router application log, an error: "Destination <destinationName> is not defined as a dynamic destination in destination service, configure additional property HTML5.DynamicDestination true" appears.
Cause	The destination name provided on the host or path level is not defined as a dynamic destination in destination service.
Solution	In the additional properties section of the destination section of the SAP BTP cockpit, add the `HTML5.DynamicDestination` property and set the value to *true*.

500: Missing xs-app.json File in HTML5 Application Repository When Reading an HTML5 application File from the Application Router

Issue	Serving content from the application router fails with error "Application does not have xs-app.json".
Cause	xs-app.json file is missing in HTML5 Application.
Solution	Redeploy the HTML5 Application with xs-app.json file or ask the business service owner to add the xs-app.json.

Bearer Token Invalid

Term	Description
Issue	The `app-router` token exchange with a business service token fails. XSUAA returns the following error: "Bearer token invalid, requesting client does not have grant_type=user_token or no scopes were granted."
Cause	The uaa.user scope in XSUAA instance configuration is missing or the target business service is not subscribed.
Solution	1. If missing, add the following to the `n xs-security.json` file: "scopes": [{ "name": "uaa.user", "description": "UAA" }], "role-templates": [{"name": "Token_Exchange","description": "UAA","scope-references": ["uaa.user"] }] 2. Redeploy or update the XSUAA service instance. 3. Make sure that the users have the new role_template, *Token_Exchange* added to their role_collections in the SAP BTP cockpit ▶ *Global Account* ▶ *Subaccount* ▶ *Security* ▶ *Role Collections* and verify that the created role's *Application Identifier* is the same as the `xsappname` from the xs-security.json. 4. If the roles of the target business service does not appear on the subscriber tenant UI, it means that the target business service subscription failed or it was bound to the application router after the subscription took place. In this case, try to unsubscribe and subscribe again. Make sure that the target business service roles appear on subscriber tenant UI.

Caching Issue in Browser

Term	Description
Issue	Your application does not work properly after logging out and when you try to log back in, for example, click anywhere on the application screen, nothing happens.
Cause	The main page of your application, that appears after you log in, is cached by the browser. Clicking links does not reach the backend (application router) and the log in process does not work.

Term	Description
Solution	Check that the main page is not configured to be cached by the browser in your xs-app.json file. The best practice is to model the cacheControl as follows: `{ "routes": [ { "source": "^/ui/index.html", "target": "index.html", "service": "html5-apps-repo-rt", "authenticationType": "xsuaa" "cacheControl": "no-cache, no-store, must-revalidate" } ] }`

Running the Application Router After Subscribing to it Fails

Term	Description
Issue	After subscribing to the application router using `<subdomain>-<myapprouter>.<scp domain>` format, it fails and returns the following error, "route not found."
Cause	In order to support subscriptions out-of-the-box, create a route using `*.<custom-domain>` format and map it to the application router. The `* host` represents a wildcard that during runtime should be replaced by the actual subscriber sub-domain. Without a custom domain, this type of route (wildcard host) cannot be created, only a fixed host can be used. Without a custom domain, the route format will be `<subdomain>-<myapprouter>.<scp domain>`. It means that for each new subscriber you need to create a new route with the expected sub-domain.
Solution	If you do not have a custom domain before subscribing to the application router, create a route with the expected subscriber sub-domain.

If the lists don't not contain a solution for your issue, you can create an incident in the SAP Support Portal. For more information, see Getting Support [page 537].

Related Information

Getting Support [page 537]

4.1.5 Consuming APIs

Discover APIs available on SAP BTP.

SAP BTP supports APIs that your applications can consume to interact, communicate, and exchange information with other applications, websites, and devices.

SAP Business Accelerator Hub

The central repository for APIs from SAP and selected partners is the SAP Business Accelerator Hub. It contains reference documentation for a variety of REST and ODATA API packages that enable you to leverage the capabilities of SAP BTP.

For an overview of all API offerings for SAP BTP, see SAP BTPon SAP Business Accelerator Hub.

If you're looking for information on APIs for a specific service, you can also use the service catalog in the SAP Discovery Center. Click on a service tile to display the available information. If the service has APIs, you can find links to their documentation under *Resources*. You can also check if a service is available in your region by opening the *Service Plan* tab.

Related Information

APIs [page 158]

https://answers.sap.com/tags/86cfb43d-204e-485e-a52d-41d9e73a5cd8

https://help.sap.com/docs/business-accelerator-hub/sap-business-accelerator-hub/what-is-sap-business-accelerator-hub?version=Cloud

https://blogs.sap.com/tags/86cfb43d-204e-485e-a52d-41d9e73a5cd8/

4.1.6 Adding Authentication and Authorization

Developers create authorization information for business users in their environment and deploy this information in an application. They make this available to administrators, who complete the authorization setup and assign the authorizations to business users.

Developers store authorization information as design-time role templates in the `xs-security.json` security descriptor file. Using the cockpit, administrators of the environment assign the authorizations to business users.

The following sections contain the process of adding authentication and authorization checks for protecting your applications, links to a number of associated tutorials, extended tasks for creating authorization artifacts, as well as reference information, including the syntax required to set the properties and values defined in the application security descriptor file.

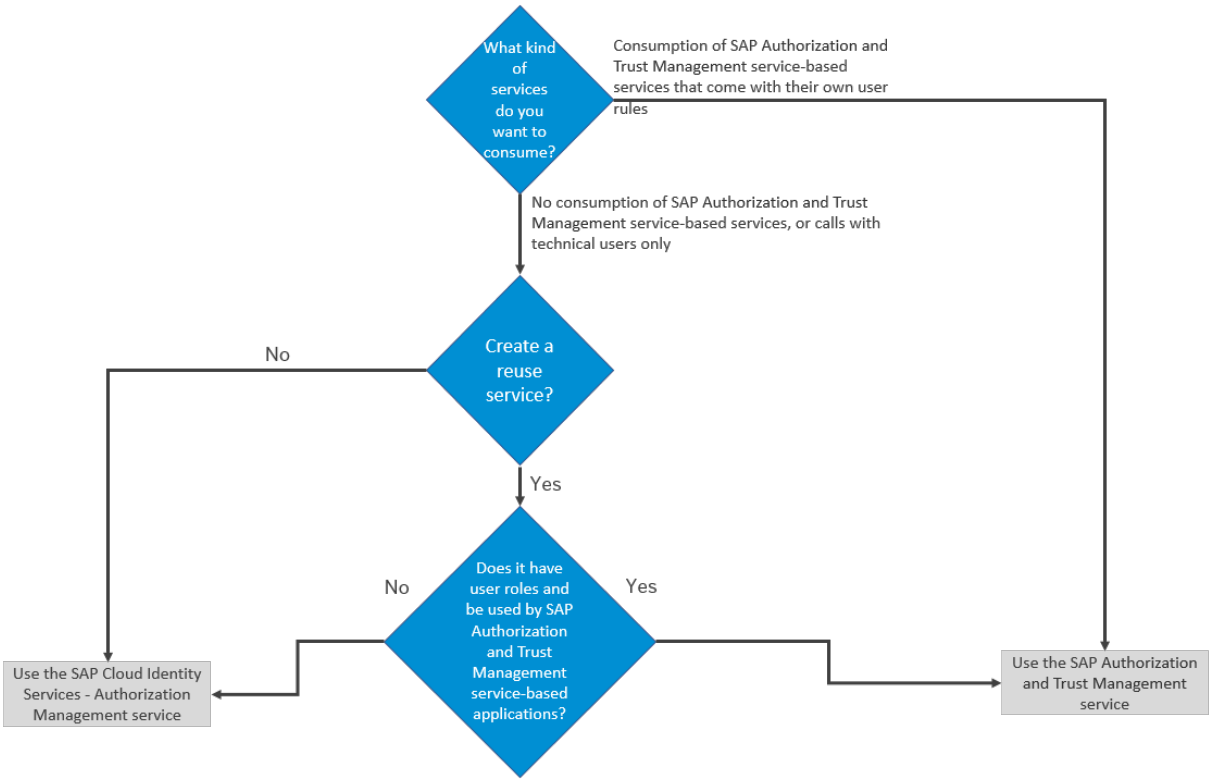
Using the SAP Authorization and Trust Management service or the SAP Cloud Identity Services - Identity Provisioning

SAP BTP is currently replacing the authorization management done by the SAP Authorization and Trust Management service, with an integrated solution with the SAP Cloud Identity Services – Authorization Management. This service is integrated into the SAP Cloud Identity Services, which offers authentication, authorization, user provisioning and management in one place.

> → Recommendation
>
> For applications, which need instance-based authorizations, we recommend using SAP Cloud Identity Services – Authorization Management on SAP BTP. Developers use the Cloud Application Programming model (CAP) for authorization policies. For more information, see Authorization and Access Control.

Currently, there are still use cases where the usage of the old model with the SAP Authorization and Trust Management service is simpler when it comes to the technical setup, especially in the administration for business users.

For decision considerations on when to use the SAP Authorization and Trust Management service and when to use SAP Cloud Identity Services – Authorization Management as the authorization management system on SAP BTP, you can follow this decision tree that will be regularly updated:



Developers use the Cloud Application Programming model for authorization policies. For more information, see Authorization and Access Control.

Protecting Your Application

Application security is maintained in the application security descriptor file (`xs-security.json`). You can use this file to define application-based security artifacts, which are the building blocks for authorizations. In this section, you can learn how to create this file and use it to create a service instance of the SAP Authorization and Trust Management service. Following this single tenant scenario, move on in increasing complexity to procedures that depict how to propagate user information between applications or services, add multitenancy to your application, and set up instance-based authorizations.

Tutorials for Authentication and Authorizations

Use tutorials to get familiar with the SAP Authorization and Trust Management service in the Cloud Foundry environment of SAP BTP.

For more information, see Tutorials for the SAP Authorization and Trust Management Service [page 574].

Tasks On Demand

Here you can find information regarding accessing the UAA admin APIs for creating XSUAA artifacts, as well as the process of configuring redirect URLs.

In addition, as an application developer, you may want to create role collections for immediate use. For example, to deliver role collections that administrators can use in the SAP BTP cockpit and easily assign to users in an onboarding process.

For more information, see Create Role Collections with Predefined Roles [page 585].

Reference Information

For the syntax required to set the properties and values defined in the application security descriptor file (`xs-security.json`), see Application Security Descriptor Configuration Syntax [page 592].

Related Information

SAP Authorization and Trust Management Service [page 3538]
Configuring Authorization Policies

4.1.6.1 Protecting Your Application

Developers create authorization information for business users in their environment; this information is deployed in an application and made available to administrators who complete the authorization setup and assign the authorizations to business users.

Lifecycle of Security Artifacts for Developers and Administrators

Developers store authorization information as design-time role templates in the application security descriptor file (`xs-security.json`). Using the `xsuaa` service broker, they deploy the security information to the `xsuaa` service. The administrators view the authorization information in role templates, which they use as part of the run-time configuration. They use the role templates to build roles, which are aggregated in role collections. The role collections are then assigned, in turn, to business users.

The tasks required to set up authorization artifacts are performed by two distinct user roles: the application developer and the administrator of the Cloud Foundry environment. After the deployment of the authorization artifacts as role templates, the administrator of the application uses the role templates provided by the developers for building role collections and assigning them to business users.

> ⓘ Note
>
> To test authorization artifacts after deployment, developers can use the role templates to build role collections and assign authorization to business users in an authorization tool based on a REST API.

Setting Up Authorization Artifacts (Developers)

Step	Task	User Role	Tool
1	Specify the application security descriptor file containing the functional authorization scopes for your application.	Application developer	Text editor
	(If applicable) If you want to create an OAuth 2.0 client in an application-related subaccount, you must use a separate security descriptor file where you specify the subaccount.	Application developer	CF command line interface
2	Create role templates for the application using the application security descriptor file.	Application developer	Text editor
3	Create a service instance from the `xsuaa` service using the service broker	Application developer	CF command line interface

Step	Task	User Role	Tool
4	Bind the service instance to the application by including it into the manifest file	Application developer	Text editor
5	Deploy the application	Application developer	CF command line interface or SAP BTP cockpit

> **Note**
>
> For more information about the different steps in setting up authorization artifacts as an application developer, refer to the associated procedures in this section.
>
> Using an Application Security Descriptor File [page 551]
>
> Propagate User Information Between Applications or Services [page 555]
>
> Set Up Your Application for Multitenancy [page 563]

Setting Up Authorization Artifacts (Account Administrators)

Task	Links
Assign the role collection to the users provided by an identity provider	Working with Role Collections [page 2612]
(If you do use a custom identity provider) Assign the role collections to user groups	Map Role Collections to User Groups [page 2621]
Assign the role collections to users and user groups, manage attribute mappings	Mapping Role Collections [page 2620]
Create a role collection and assign roles to it	Define a Role Collection [page 2614]
Use an existing role or create a new one using role templates	Add Roles to Role Collections on the Application Level [page 2632]

Related Information

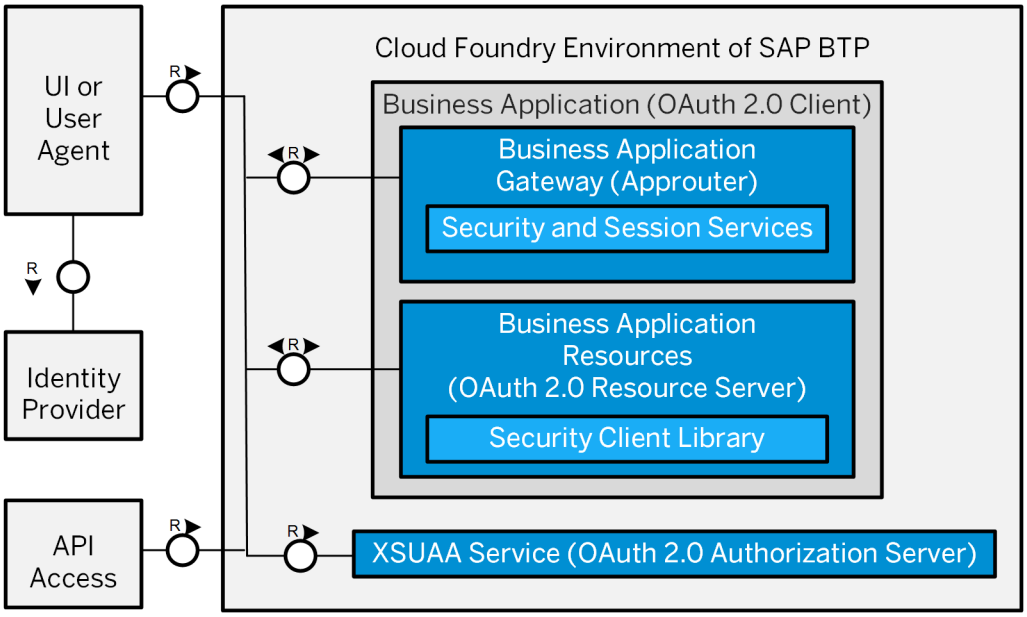
Application Security Descriptor Configuration Syntax [page 592]

4.1.6.1.1 Using an Application Security Descriptor File

Learn how to create a security application descriptor file and use it to create a service instance of the SAP Authorization and Trust Management service.

Context

You want to protect your application resources (functionality and data) so that only authenticated and authorized business users are able to access these resources.



You declare your application security descriptor with JSON syntax and store it in a flat file on the filesystem. The standard name for this file is `xs-security.json`.

Procedure

1. Create your application security descriptor (`xs-security.json`) and set the `xsappname` element.

   The element xsappname defines a prefix for the runtime name of the application. For each tenant (subaccount), which has subscribed to the application, the XSUAA service supplies a unique tenant index for the application subscription and internally concatenates the tenant index with `xsappname` at runtime (see also next step).

   Sample Code

   ```
   {
     "xsappname" : "hello-world",
     ...
   }
   ```

2. Set the `scopes` element.

   Scopes represent the API endpoints - or functions - for which access should be restricted. They're required if you want to define the functions a user is authorized to process. A scope has an arbitrary name and must be prefixed with the runtime application name to distinguish the user scopes between tenants (subaccounts) which have subscribed to the application and equally named scopes between different applications. The `$XSAPPNAME` dummy value is a wildcard for the application runtime name and is used to prefix the arbitrary scope names. The XSUAA service substitutes the `$XSAPPNAME` dummy value with the application runtime name for each of the subscribed tenants (see also step 1).

   {·} Sample Code

   ```
   {
     ...
     "scopes"    : [
       {
         "name"        : "$XSAPPNAME.Display",
         "description" : "display" },
       {
         "name"        : "$XSAPPNAME.Edit",
         "description" : "edit" },
       {
         "name"        : "$XSAPPNAME.Delete",
         "description" : "delete" }
     ],
     ...
   }
   ```

3. Set the `attributes` element.

   → Recommendation

   For applications, which need instance-based authorizations, we recommend to use SAP Cloud Identity Services – Authorization Management on SAP BTP.

   Attributes represent the data entities for which access should be restricted. They're required if you want to define which data a user is authorized to process.

   Attributes provide instance-based, fine-granular authorization checks. See Setting Up Instance-Based Authorizations [page 568].

   {·} Sample Code

   ```
   {
     ...
     "attributes" : [
       {
         "name"        : "Country",
         "description" : "Country",
         "valueType"   : "string" },
       {
         "name"        : "CostCenter",
         "description" : "CostCenter",
         "valueType"   : "int" }
     ],
     ...
   }
   ```

4. Set the `role-templates` element.

Role templates combine scopes with attributes and serve as templates from which roles are created later by the administrator.

Sample Code

```
{
  ...
  "role-templates" : [
    {
      "name"                 : "Viewer",
      "description"          : "View all books",
      "scope-references"     : [ "$XSAPPNAME.Display" ],
      "attribute-references" : [ "Country" ] },
    {
      "name"                 : "Editor",
      "description"          : "Edit, delete books",
      "scope-references"     : [ "$XSAPPNAME.Edit", "$XSAPPNAME.Delete" ],
      "attribute-references" : [ "Country", "CostCenter"] }
  ]
}
```

5. Deploy the application security descriptor (`xs-security.json`).

   Example:

   Sample Code

   ```
   cf create-service xsuaa application my_xsuaa_service_instance -c xs-
   security.json
   ```

6. Check the scopes in the application.

   Sample Code

   ```
   app.get('/books', checkReadScope, getBooks);
   // Scope check
   function checkReadScope(req, res, next) {
       if (req.authInfo.checkLocalScope('display')) {
           return next();
       } else {
           console.log('Missing the expected scope');
           res.status(403).end('Forbidden');
       }
   }
   ```

   The `checkReadScope` function ensures that only a user with the correct authorizations can look at the books.

   See Tutorials for the SAP Authorization and Trust Management Service [page 574] for ways to check scopes in Java and other frameworks, like Spring.

7. In the manifest file, add the binding for the service instance to your application.

   Sample Code

   ```
   ...
   services:
     - my_xsuaa_service_instance
   ```

8. Deploy the application.

Sample Code

```
cf push
```

Results

You've declared and deployed your application security descriptor (`xs-security.json`) with scopes (functional authorizations), attributes (data authorizations), and role templates (to combine scopes with attributes). The `xsappname` element is mandatory. The `scopes`, `attributes`, and `role-templates` elements are optional. You've also bound your service instance to your application, and then deployed it.

Sample Code

```
{
  "xsappname" : "hello-world",
  "scopes"    : [
    {
      "name"        : "$XSAPPNAME.Display",
      "description" : "display" },
    {
      "name"        : "$XSAPPNAME.Edit",
      "description" : "edit" },
    {
      "name"        : "$XSAPPNAME.Delete",
      "description" : "delete" }
  ],
  "attributes" : [
    {
      "name"        : "Country",
      "description" : "Country",
      "valueType"   : "string" },
    {
      "name"        : "CostCenter",
      "description" : "CostCenter",
      "valueType"   : "int" }
  ],
  "role-templates" : [
    {
      "name"                : "Viewer",
      "description"         : "View all books",
      "scope-references"    : [ "$XSAPPNAME.Display" ],
      "attribute-references" : [ "Country" ] },
    {
      "name"                : "Editor",
      "description"         : "Edit, delete books",
      "scope-references"    : [ "$XSAPPNAME.Edit", "$XSAPPNAME.Delete" ],
      "attribute-references" : [ "Country", "CostCenter"] }
  ]
}
```

Next Steps

To implement web access using a browser or a browser-based user interface, you can use an application router to enable you to create a secure route to your application, or an open source UI framework, such as Spring (Boot). For more information, see the related links.

Related Information

Application Router [page 453]
Open Source UI Framework
Limits for Technical Artifacts of the SAP Authorization and Trust Management Service [page 3605]
Application Security Descriptor Configuration Syntax [page 592]

4.1.6.1.2 Propagate User Information Between Applications or Services

When a business application communicates with a service, you must decide whether you want to propagate the identity of the user that called the business application, or if a call from machine-to-machine is sufficient.

Principal Propagation Versus Technical Communication

If the business application triggers an action of the service that should be auditable or requires that the identity of the user be known, use principal propagation. Principal propagation enables the identity of the user to be propagated from the business application to the service.

If the business application triggers an action of the service for which the identity of the user is unimportant, such as a regular clean-up task or the checking of a queue, then you can use technical communication. In technical communication, the service performs the action for the business application without knowing the identity of the user.

Cross-Application API Calls from an External System to Applications in the Cloud Foundry Environment

If applications from an external system must make API calls to applications running in the Cloud Foundry environment, administrators must make sure that these applications can communicate with the relevant applications in the external system. In this case, the bearer assertion flow or client credentials identify the external application at the UAA, which can then issue a JSON web token. The external application can use this JSON web token when it makes the API calls to applications in the Cloud Foundry environment.

No browser is involved here. Users are propagated in the following ways:

- Using technical communication, for example, propagating scopes and authorities.
- Using a bearer assertion or client credentials (JSON web tokens) to propagate named users.

Related Information

Principal Propagation with Tightly Coupled Developments [page 559]
Technical Communication with Tightly Coupled Developments [page 556]
Principal Propagation [page 3617]

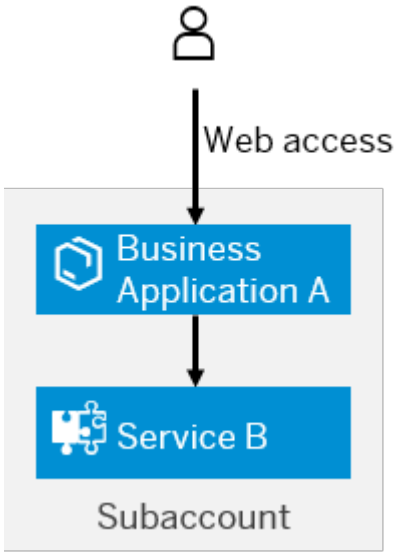
4.1.6.1.2.1 Technical Communication with Tightly Coupled Developments

When a business application and a service are developed for the same subaccount, the two developments are tightly coupled together. The service is designed to be used with this particular application.

Context

In this example, the call between the application and the service uses technical communication instead of the current user. The figure that follows illustrates this scenario.

> **Note**
>
> With service, we do not mean a development that has been registered as a service with the Cloud Controller, but rather the role one application takes in relationship to another. Both developments can be applications, but one application takes on the role of a business application enabling business users to perform tasks. The other application plays the role of a service, enabling the development of the business application.
>
> We also assume that both applications have their own application router. Having separate application routers means that the applications can define their own authorization models.



Two Developments Tightly Coupled Using Technical Communication

Like a trust arrangement, you must configure service B to grant the authority to business application A to use the `$XSAPPNAME.Create` scope. Likewise, you must configure business application A to accept such declarations of authority. This configuration is part of the `xs-security.json` of the UAA service instances of the two developments.

Procedure

1. As the service developer, declare the grant of authority to business application A in the `scopes` section of the `xs-security.json` for service B.

   Sample Code

   ```
   {
       "xsappname"     : "servb",
       "tenant-mode"   : "shared",
       "scopes"        : [
                           {
                             "name"                 : "$XSAPPNAME.Create",
                             "description"          : "create",
                             "grant-as-authority-to-apps" :
   ["$XSAPPNAME(application,appa)"]
                           }
                         ]
   }
   ```

   Under `scopes`, the scope `$XSAPPNAME.Create` grants authority to business application A to create data. Here the granted app is business application A and has been given the authorization to create data. When you create the UAA service instance for business application A, the system creates an OAuth client for business application A to call over to service B. When you bind the service, you make business application A aware of the client. In the security descriptor for the UAA service instance of business application A, accept this grant.

> ### Note
>
> `$XSAPPNAME` is replaced at runtime with the application name. The parameters that follow `$XSAPPNAME` are the service plan for the XSUAA service and `xsappname` as defined in the `xs-security.json`. These parameters help uniquely identify other applications in the subaccount.

2. Create the UAA service instance for service B.

   For example:

   ```
   cf create-service xsuaa application servb-uaa -c /servb/security/xs-security.json
   ```

3. As the business application developer, accept the authority of other applications in the `authorities` section of the `xs-security.json` in business application A.

   > ### Sample Code
   >
   > ```
   > {
   >     "xsappname"     : "appa",
   >     "tenant-mode"   : "shared",
   >     "authorities":["$ACCEPT_GRANTED_AUTHORITIES"],
   > }
   > ```

   In this example, business application A accepts all authorities granted by any other application. You can explicitly define which authorities you want business application to accept. For example:

   `"authorities":["$XSAPPNAME(application,servb).Create]"`.

   > ### Note
   >
   > For the sake of this example, we've removed any scopes and role templates for application A. Naturally, we recommend that you protect application A with appropriate scopes and deliver role templates.

4. Create the UAA service instance for business application A.

   For example:

   ```
   cf create-service xsuaa application appa-uaa -c /appa/security/xs-security.json
   ```

5. Deploy the developments.
6. As an adminístrator, create a role from the role templates for business application A and assign the role to a user.

   In the example, we created here, there are no role template to create roles from.

   For more information, see Administration: Managing Authentication and Authorization.

7. As an end user, access business application A.

   When you access business application A, business application A accesses service B on your behalf. Because service B granted the `"$XSAPPNAME.create` scope to business application A, business application A is allowed to access the resource.

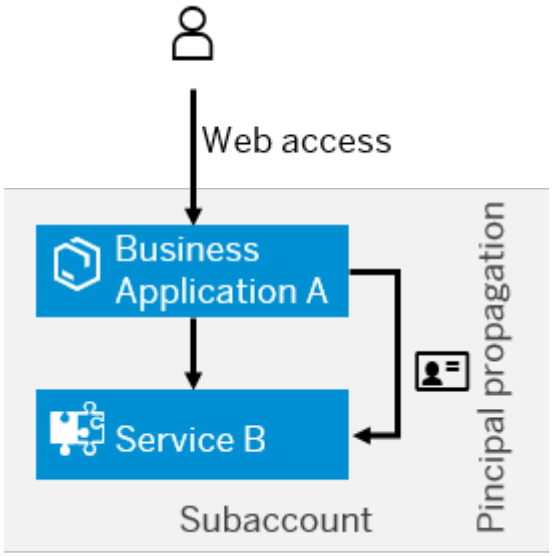
4.1.6.1.2.2 Principal Propagation with Tightly Coupled Developments

A scenario is tightly coupled when a business application calls a service within the same subaccount in the Cloud Foundry environment. The business application calls the service with principal propagation, meaning information about the current user is carried over with the service call.

Context

The figure that follows illustrates this scenario.

> **Note**
>
> With service, we do not mean a development that has been registered as a service with the Cloud Controller, but rather the role one application takes in relationship to another. Both developments can be applications, but one application takes on the role of a business application enabling business users to perform tasks. The other application plays the role of a service, enabling the development of the business application.
>
> We also assume that both applications have their own application router. Having separate application routers means that the applications can define their own authorization models.



Two Applications, Tightly Coupled Using Principal Propagation

In this scenario, we enable the service to accept the JSON Web Token (JWT) of the business application. The JWT of the business application includes the scopes of both developments. When the service receives a JWT of the business application with the scopes of the service in it, the service implicitly accepts the JWT of the business application. This configuration is part of the `xs-security.json` of the UAA service instances of the two developments.

Procedure

1. As the service developer, add the attribute `granted-apps` to scopes you want to share with other business applications in the `scopes` section of the `xs-security.json` for this service.

   In this attribute, list the names of the applications (`xsappname`) you want to share the scope with.

   Sample Code

   ```
   "scopes": [
       {
           "name"                : "$XSAPPNAME.Viewer",
           "description"         : "View log",
           "granted-apps"        : ["$XSAPPNAME(application,appa)" ]
       },
       {
           "name"                : "$XSAPPNAME.Editor",
           "description"         : "view and write logs",
       }
              ]
   ```

   In this example, we have defined two scopes, `Viewer` and `Editor`. Under `granted-apps`, we list the application names with which we want to share the `Viewer` scope. All apps listed here can consume this scope in their role templates.

   Note

   `$XSAPPNAME` is replaced at runtime with the application name. The parameters that follow `$XSAPPNAME` are the service plan for the XSUAA service and `xsappname` as defined in the `xs-security.json`. These parameters help uniquely identify other applications in the subaccount.

   You might reserve the `Editor` scope for a role template for the service. The editor role is only meant for administrators.

2. Create the UAA service instance for service B.

   For example:

   ```
   cf create-service xsuaa application servb-uaa -c /servb/security/xs-security.json
   ```

3. As the business application developer, add scope references to the scope of the service in the `role-templates` section of the `xs-security.json` in business application A.

   Sample Code

   ```
   "role-templates": [
       {
           "name"             : "Viewer",
           "description"      : "View all data of the service",
           "scope-references" : [
                                  "$XSAPPNAME.Display",
                                  "$XSAPPNAME(application,servB).Viewer"
                                ]
       }
                    ]
   ```

Under `role-templates`, the `Viewer` template includes a reference to the `Viewer` scope of service B. This reference enables administrators to assign a single role to a user to grant authorizations for both applications.

4. Create the UAA service instance for application A.

   For example:

   ```
   cf create-service xsuaa application appa-uaa -c /appa/security/xs-security.json
   ```

5. Deploy the developments.
6. As an adminístrator, create a role from the role template from business application A and assign the role to a user.

   Only assign the `Viewer` template from business application A to your test user and no templates from service B.

   For more information, see Administration: Managing Authentication and Authorization.
7. As an end user, test access business application A.

   Your test user can use the scopes delivered from the `Viewer` template to access business application A or service B directly or indirectly.

4.1.6.1.2.2.1 Use Foreign Scope Option for Principal Propagation with Tightly Coupled Developments

With foreign scope reference, configure the business application to accept new scopes for the service without having to modify the business application.

Procedure

1. As the service developer, add the attribute `granted-apps` to scopes you want to share with other business applications in the `scopes` section of the `xs-security.json` for this service.

   In this attribute, list the names of the applications (`xsappname`) you want to share the scope with.

   Sample Code

   ```
   "scopes": [
       {
           "name"                 : "$XSAPPNAME.Read",
           "description"          : "read",
           "granted-apps"         : ["$XSAPPNAME(application,appa)" ]
       },
       {
           "name"                 : "$XSAPPNAME.Approve",
           "description"          : "approval",
       }
              ]
   ```

In this example, we've defined two scopes, `Read` and `Approve`. Under `granted-apps`, we list the application names with which we want to share the `Read` scope. All apps listed here can consume this scope in their role templates.

> **Note**
>
> `$XSAPPNAME` is replaced at runtime with the application name. The parameters that follow `$XSAPPNAME` are the service plan for the XSUAA service and `xsappname` as defined in the `xs-security.json`. These parameters help uniquely identify other applications in the subaccount.

You might reserve the `Approve` scope for a role template for the service. The approver role is only meant for administrators.

2. Create the UAA service instance for service B.

   For example:

   ```
   cf create-service xsuaa application servb-uaa -c /servb/security/xs-security.json
   ```

3. In the business application, accept all foreign scope references.

   Sample Code

   ```
   {
    "xsappname": "appa",
    "tenant-mode": "shared",
    "description": "Application ecurity descriptor for application A",
    "foreign-scope-references": ["$ACCEPT_GRANTED_SCOPES"],
    "scopes"        : [
                          {
                            "name"                 : "$XSAPPNAME.View",
                            "description"          : "View data"
                          }
                        ],
       "role-templates": [
                          {
                            "name"                 : "View",
                            "description"          : "View data",
                            "scope-references"     : [
                                                         "$XSAPPNAME.View"
                                                       ]

                          }
       ]
   }
   ```

   Under `foreign-scope-references`, the application security descriptor accepts any and all foreign scopes that were granted to it. You can limit the references to just those applications you want to accept. For example, `"$XSAPPNAME(application,servb).Read`

4. Create the UAA service instance for application A.

   For example:

   ```
   cf create-service xsuaa application appa-uaa -c /appa/security/xs-security.json
   ```

5. Deploy the developments.
6. As an administrator, create a role from the role template from business application A and assign the role to a user.

Only assign the `Viewer` template from business application A to your test user and no templates from service B.

For more information, see Administration: Managing Authentication and Authorization.

7. As a business user, test business application A.

   Your test user can use the scopes delivered from the `Viewer` template to access business application A or service B directly or indirectly.

Related Information

Principal Propagation with Tightly Coupled Developments [page 559]

4.1.6.1.3 Set Up Your Application for Multitenancy

Learn how to add multitenancy to your application and make it available for other subaccounts using the SaaS Provisioning service and the SAP Authorization and Trust Management service.

Context

You've created an application in your subaccount that is secured by the SAP Authorization and Trust Management service. You now want to make that application available to other subaccounts (tenants). You'll use the SaaS Provisioning service to make your application available to a consumer subaccount within your global trial account.

You declare your application security descriptor with JSON syntax and store it in a flat file on the filesystem. The standard name for this file is `xs-security.json`.

Procedure

1. Enable multitenancy in the application security descriptor file.
   a. Go to the folder where the `xs-security.json` file is stored.
   b. Change the value of the `tenant-mode` parameter to `shared`.
   c. Under the `scopes` element, add access to the SaaS Provisioning service to call the product list callback API directly. You'll implement the callbacks in Step 3.

   Sample Code

```
"scopes": [
                {
                   "name": "$XSAPPNAME.read",
                   "description": "With this scope, USER can read
products."
```

```
          },
          {
              "name": "$XSAPPNAME.Callback",
              "description": "With this scope set, the callbacks for
tenant onboarding, offboarding and getDependencies can be called.",
              "grant-as-authority-to-apps": [
                  "$XSAPPNAME(application,sap-provisioning,tenant-
onboarding)"
              ]
          }
       ],
```

d. Save the file.

2. Update the manifest.

   In this step, you need to complete the following tasks:

   - Add a new routing pattern.
   - Add the service binding for the SaaS Provisioning service.

   a. Go to your application folder and open the `manifest.yml` file.
   b. For the application router, add the `TENANT_HOST_PATTERN` parameter under the `env` parameter.

      This parameter specifies a generic route for all tenants to call the application over the approuter.

      Sample Code

      ```
      env:
        destinations: >
          [
            {"name":"hw-dest",
             "url":"https://product-list-ap25.cfapps.eu10.hana.ondemand.com",
             "forwardAuthToken": true}
          ]
        TENANT_HOST_PATTERN: "^(.*)-approuter-product-list-
      ap25.cfapps.eu10.hana.ondemand.com"
      ```

      Restriction

      The value of the `TENANT_HOST_PATTERN` parameter must be in lowercase.

   c. Add the service binding for the SaaS Provisioning service to your application.

      Adding the service binding of the SaaS Provisioning service in the `manifest.yml` file will automatically bind the service instance to your application, when deploying it.

      Sample Code

      ```
      services:
        - xsuaa-service-tutorial
        - saas-registry-tutorial
      ```

3. Implement the subscribe/unsubscribe endpoints.

   To enable other subaccounts to subscribe to your application, you need to implement an endpoint for the SaaS registration manager to subscribe/unsubscribe.

   a. Go to the `myapp` folder and open the `index.js` file.

b. Add the following lines of code after the `checkReadScope` function. (Replace the `ap25` string with the string that you used when deploying your application security model. Adapt the region code if your trial isn't in the eu10 region.)

Sample Code

```
  app.put('/callback/v1.0/tenants/*', function (req, res) {
      var consumerSubdomain = req.body.subscribedSubdomain;
      var tenantAppURL = "https:\/\/" + consumerSubdomain + "-approuter-
product-list-ap25." + "cfapps.eu10.hana.ondemand.com/products";
      res.status(200).send(tenantAppURL);
    });
  app.delete('/callback/v1.0/tenants/*', function (req, res) {

      // Implement here any offboarding proces that's triggered when
user deletes a subscription to the application
      res.sendStatus(200);
  });
```

Return the HTTP `200 OK` success status response if the offboarding process was successful.

If the offboarding failed, return the relevant error code and an accompanying message.

Note

It's highly recommended to use a response object that contains both the error code and a user-friendly, customer-oriented error message and technical details about the error in the following format:

```
{
  "code":<The relevant response code, e.g., 500, 400>,
  "messageForCustomer":"<A customer-facing message. Please write in a
user-friendly format, as clearly as possible>",
  "errorDetails":"<Specify here additional technical details about the
error>"
}
```

c. To be able to read the body of those calls, add the body parser module at line 9 of the `index.js` file.

Sample Code

```
const bodyParser = require('body-parser')
app.use(bodyParser.json())
```

d. Add the body parser module as a dependency to the `product list/myapp/package.json` file.

Sample Code

```
"dependencies": {
  "express": "^4.17.1",
  "@sap/xsenv": "^2.2.0",
  "@sap/xssec": "^3.0.0",
  "passport": "^0.4.1",
  "body-parser": "^1.19.0"
}
```

4. Create a SaaS configuration file.

To make your multitenant application endpoints available for subscription to consumer subaccounts, you need to register the application in the Cloud Foundry environment by using the SaaS Provisioning service.

To register your application, you need a configuration file called `config.json`. In this file, you specify the subscription URL, the name, and description of your application. The `xsappname` must be the same as the `xsappname` in the `xs-security.json` file.

a. Go to your application folder and create the `config.json` file.
b. Insert the following lines.

Sample Code

```
{
  "xsappname":"product-list",
  "appUrls": {
    "onSubscription" : "https://product-list-
ap25.cfapps.eu10.hana.ondemand.com/callback/v1.0/tenants/{tenantId}"
  },
  "displayName" : "Product List MTA",
  "description" : "Product list MTA sample application",
  "category" : "Custom SaaS Applications"
}
```

5. Delete the old service instance of the SAP Authorization and Trust Management service.

When you change the tenant mode from `dedicated` to `shared` like you did in step 1, it's not enough to update the service instance. You have to first unbind and delete the old service instance, to be able to later recreate it with the updated tenant mode settings.

a. Unbind the existing service instance from your application.

```
cf unbind-service <APP_NAME> <SERVICE_INSTANCE>
```

Sample Code

```
cf unbind-service product-list xsuaa-service-tutorial
```

b. Unbind the existing service instance from the application router.

```
cf unbind-service <APP_NAME> <SERVICE_INSTANCE>
```

Sample Code

```
cf unbind-service approuter xsuaa-service-tutorial
```

c. Delete the existing service instance.

```
cf delete-service <SERVICE_INSTANCE>
```

Sample Code

```
cf delete-service xsuaa-service-tutorial
```

6. Create service instances and redeploy your application.

a. Log in to your Cloud Foundry account with the Cloud Foundry CLI.

b. Go to your application folder and create the service instance with the `xs-security.json` security descriptor file.

```
cf create-service <SERVICE> <PLAN> <SERVICE_INSTANCE> -c xs-security.json
```

> **Sample Code**
>
> ```
> cf create-service xsuaa application xsuaa-service-tutorial -c xs-security.json
> ```

c. Create the SaaS Provisioning service instance with the `config.json` file.

> **Sample Code**
>
> ```
> cf create-service <SERVICE> <PLAN> <SERVICE_INSTANCE> -c config.json
> ```
>
> ```
> cf create-service saas-registry application saas-registry-tutorial -c config.json
> ```

d. Redeploy the application with the updated `manifest.yml` file.

> **Sample Code**
>
> ```
> cf push
> ```

7. Create a route for a consumer subaccount.

   Make your application reachable for consumer subaccounts by adding a new route in the Cloud Foundry CLI. The route is composed of the subdomain of the subscribing subaccount and the `TENANT_HOST_PATTERN` of the application router that you defined in the `manifest.yml` file. You have to create a new route for every subaccount (tenant) that subscribes to the application.

   a. Log in to the Cloud Foundry account where the application is deployed with the Cloud Foundry CLI.
   b. Create a route for the consumer subaccount.

   ```
   cf map-route <APP_NAME> <DOMAIN> --hostname <APPLICATION_HOSTNAME>
   ```

   > **Note**
   >
   > The `APPLICATION_HOSTNAME` is the combined string of the subdomain ID of the consumer subaccount and the `TENANT_HOST_PATTERN` from the `manifest.yml` file.

   > **Sample Code**
   >
   > ```
   > cf map-route approuter cfapps.eu10.hana.ondemand.com --hostname consumer-tenant-ap25-approuter-product-list-ap25
   > ```

8. Access the application with the consumer subaccount.

   To access the application you need to subscribe to it. Follow these steps to subscribe to the SaaS application with the consumer subaccount and call the application URL.

   a. Open the SAP BTP trial.

b. Navigate to your consumer subaccount.
c. Choose *Subscriptions*.
d. Choose *Product List MTA*.
e. Choose *Subscribe*.
f. Choose *Go to Application*.

You'll now see the application with the message `no data` because you have to assign the role collection to your user in the consumer subaccount.

9. Assign the role collection.

Assign the `ProductListViewer` role collection to your user. This role collection contains the necessary role to view the products in your application.

a. Open the SAP BTP cockpit.
b. Navigate to your consumer subaccount.
c. Choose *Security* > *Role Collections*.
d. Choose the `ProductListViewer` role collection.
e. Use the chevron at the right side to expand the role collection.
f. Go to the *Users* section and choose *Edit*.
g. Enter the e-mail address of the user that you want to assign to the role collection. Take care that your user's identity provider is SAP ID service.
h. Save your changes.
i. Clear your cache and reload the application URL.

Sample Code

```
https://consumer-tenant-ap25-approuter-product-list-
ap25.cfapps.eu10.hana.ondemand.com/products
```

The application will now show you the products.

Related Information

Add Multitenancy to a Node.js Application Secured by the SAP Authorization and Trust Management service

4.1.6.1.4 Setting Up Instance-Based Authorizations

Instance-based authorizations are authorization checks based on the value of some authorization-relevant attribute defined by the application. Functional authorizations define whether you've permission to edit or view some function in the application, while instance-based authorizations define what data you can act upon.

As a developer, you develop your application as you did in our previous examples for functional authorization checks. In that example, we defined attributes in the application security descriptor (`xs-security.json`) even though we didn't use them. In that example, we defined *CostCenter* as an attribute. Let's look at an example.

Your application is used by a controller to view the expenses of their organization. So, your application shows the controller all invoices, which belong to the cost center of the controller. So, your application takes the dataset of invoices and shows the instance of that dataset where the cost center equals the cost center of the controller. Control access to this dataset by creating a role collection that includes the *CostCenter* attribute and the value required by the controller. Then assign that role collection to the user of the controller.

For more information about defining attributes in the application security descriptor, see Using an Application Security Descriptor File [page 551].

For instance-based authorizations, you build checks against these attributes into your coding. To help you understand authorization checks against attributes, check out our sample bulletin board application on Github.

For more information, see https://github.com/SAP-samples/cloud-application-security-sample/tree/master/spring-security-basis .

Once you've built in your authorization checks, you've a few ways to populate the attributes with values, for your code to check.

How Attribute Values Are Set

Once you've defined attributes in your role templates, how are attribute values filled? There are a couple different ways.

- Predefine the attribute values in role collections you deliver with your app.
  As a developer, you can define preconfigured role collections in your application security descriptor. When the role administrator assigns these role collections to users, the attribute values are hard-coded with the default values according to the role templates you define. These role collections enable a role administrator to get their users started quickly.
- The administrator builds static roles from your role templates.
  The administrator builds the roles they need, based on the role templates you deliver. The administrator builds each role, setting the values for the attributes you defined in the role templates. The administrator can't add or subtract any attributes. The administrator can only set the values. The administrator then bundles these roles into role collections and assigns them to users.
- The administrator builds dynamic roles from your role templates.
  The administrator builds the roles they need, based on the role templates you deliver. The administrator can decide to populate role values dynamically from the identity provider. When users log on, they receive a token with attributes defined by the identity provider. The manager of the identity provider controls the values a user has for given attributes, not the role administrator of the subaccount. This setup means that you as an application developer need to understand what kinds of attributes are available to the role administrator from the identity provider.

Related Information

Attributes [page 2628]

4.1.6.1.5 Retrieving Access Tokens with Mutual Transport Layer Security (mTLS)

Mutual Transport Layer Security (mTLS) is considered more secure than the combination of client ID and client secret. Unlike retrieving the access token with client ID and client secret, no secret is shared between calling application and the service instance of SAP Authorization and Trust Management service (XSUAA).

This configuration enables your application to retrieve or exchange access tokens from an instance of the SAP Authorization and Trust Management service with mTLS. When using TLS, the client verifies the identity of the OAuth server during their handshake. By using mTLS, the OAuth server also verifies the identity of the client. Calls to other services and applications with the access token use the standard OAuth protocols.

You, as a developer, have the option to have the service provide the X.509 certificate for the binding or service key or if you already have your own public key infrastructure (PKI), you can provide your own.

> ⚠ Restriction
>
> If you use your own PKI, your certificates must be issued by certificate authorities trusted by the service.
>
> For more information, see Binding Parameters of SAP Authorization and Trust Management Service [page 2640].

Related Information

Enable mTLS Authentication to SAP Authorization and Trust Management Service for Your Application [page 570]
Implementing Custom Token Retrieval from SAP Authorization and Trust Management Service with mTLS [page 573]

4.1.6.1.5.1 Enable mTLS Authentication to SAP Authorization and Trust Management Service for Your Application

Enabling your application to retrieve access tokens from SAP Authorization and Trust Management service(XSUAA) over mutual transport layer security (mTLS) requires you to enable mTLS token retrieval for the service instance. Then ensure the binding includes the X.509 credentials.

Prerequisites

To handle tokens, use:

- Application router version 10.7.1 or later
  For more information, see https://www.npmjs.com/package/@sap/approuter .

- Our security libraries
  - For Java or Spring, see XSUAA Token Client and Token Flow API on *Github*.
  - For Python, see SAP Python Security Library on *Python Package Index*.
  - For Node.js, see @sap/xssec on *Node Package Manager (NPM)*.
- Your own integration
  For more information, see Implementing Custom Token Retrieval from SAP Authorization and Trust Management Service with mTLS [page 573].

> → Recommendation
>
> Build automation into your deployment or CI/CD pipeline. Client certificates have a relatively short lifetime. The default lifetime for certificates managed by the service is 7 days. Only by automating the process, can you ensure that credentials are rotated and distributed in a timely manner. Otherwise, you risk authentication failures from expired certificates.

Context

Use the same procedure as outlined in Using an Application Security Descriptor File [page 551]. Modify this procedure with the following steps.

Be mindful of whether you're updating an existing application or starting with a brand new development.

- If you're adding mTLS to an existing application, add mTLS support to the credential types of the service instance. Then update all your bindings and service keys before removing the support for other credential types.

  > → Tip
  >
  > For an example of how to migrate from one secret type to another, see Migrate from Instance Secrets to Binding Secrets [page 2638].

- If you're starting from scratch, you can build in mTLS support from the beginning, without worrying about disrupting existing bindings or service keys.

Procedure

1. Enable mTLS for the service instance.

   Add the credential type `x509` to the OAuth configuration of the service instance.

   - The following is an example of the OAuth configuration from an application security descriptor (`xs-security.json`).

     ```
     "oauth2-configuration": {
       "credential-types": ["binding-secret","x509"]
     }
     ```

     In this example, binding secrets are the default credential type, but X.509 secrets are supported.

- The following is an example of the service as a resource in a multitarget application (MTA) descriptor (`mta.yaml`).

```
resources:
  - name: myXSUAA
    type: com.sap.xs.uaa
    parameters:
      service-plan: application
      config:
        xsappname: mAapp
        oauth2-configuration:
          credential-types:
            - x509
            - binding-secret
```

  In this example, X.509 secrets are the default credential type. When deployed, any preconfigured bindings get service-managed client certificates. After deployment, you can create additional bindings with binding secrets.

2. Deploy the application or update the service instance.
3. Create bindings or service keys with the X.509 credentials.

   You create the binding from the Cloud Foundry command-line interface (cf CLI),

   - From the command-line interface, provide a `parameters.json` to configure the certificates managed by the service or to provide your own X.509 certificate.
     The following is an example of a `parameters.json` with the certificates managed by the service.

```
{
  "credential-type": "x509",
  "x509": {
    "key-length": 2048,
    "validity": 8,
    "validity-type": "DAYS"
  }
}
```

     The following is an example of a `parameters.json` with certificates you provide.

```
{
  "credential-type": "x509",
  "x509": {
    "certificate": "-----BEGIN CERTIFICATE-----...-----END
CERTIFICATE-----",
    "certificate-pinning": true
  }
}
```

     Reference the `parameters.json` when you bind the service. For example:

```
cf bind-service myApp myXSUAA -c parameters.json
```

Related Information

Parameters for X.509 Certificates Managed by SAP Authorization and Trust Management Service [page 2642]
Parameters for Self-Managed X.509 Certificates [page 2643]
Rotate X.509 Secrets [page 2637]

4.1.6.1.5.2 Implementing Custom Token Retrieval from SAP Authorization and Trust Management Service with mTLS

The application router and other token client libraries take care of getting tokens from the SAP Authorization and Trust Management service for you. If you build your own integration, you must handle token retrieval yourself.

> **Note**
>
> The service doesn't check for certificate revocation. To stop the service from accepting a certificate that is still valid, delete the relevant bindings or service keys. As soon as the binding is deleted, the service stops accepting the certificate.
>
> For bindings with self-managed certificates and the `certificate-pinning` parameter set to `false`, you can rotate the secrets without deleting bindings. Just use a new certificate with the same subject and issuer distinguished name (DN). The service saves the new validity date of the new certificate.
>
> For more information, see Parameters for Self-Managed X.509 Certificates [page 2643].

> **→ Recommendation**
>
> Build automation into your deployment or CI/CD pipeline. Client certificates have a relatively short lifetime. The default lifetime for certificates managed by the service is 7 days. Only by automating the process, can you ensure that credentials are rotated and distributed in a timely manner. Otherwise, you risk authentication failures from expired certificates.

When called with mTLS, the token endpoint is a little different from the standard endpoint. The path includes `authentication.cert` instead of just `authentication`.

`https://<subdomain>.authentication.cert.<landscape>/oauth/token`

For example:

```
https://test-me.authentication.cert.eu20.hana.ondemand.com/oauth/token
```

Use the `/.well-known/openid-configuration` endpoint to find the endpoints for your subaccount. The endpoints appear under the `mtls_endpoint_aliases` parameter.

For example: `https://test-me.authentication.eu20.hana.ondemand.com/.well-known/openid-configuration`

The following is an example of part of the JSON returned by the endpoint.

```
{
    …,
    "mtls_endpoint_aliases": {
        "token_endpoint": "https://test-
me.authentication.cert.eu20.hana.ondemand.com/oauth/token",
        "authorization_endpoint": "https://test-
me.authentication.cert.eu20.hana.ondemand.com/oauth/authorize"
    },
…
}
```

For X.509 authentication, the `grant_type` are as with client secret authentication:

- For UI user flows: Authorization code
- For API user flows: JWT bearer, refresh token, and password grant
- For technical flows: Client credentials

> **Note**
>
> Leave out the `client_secret` parameter. The `client_id` is still required.

Finally, when building the call, include the client certificate (public key) and private key for signing in your request. The following syntax is for CURL:

```
curl --cert <Path_Client_Cert> --key <Path_Private_Key>
-XPOST https://<subdomain>.authentication.cert.<landscape>/oauth/token -d
'grant_type=<Grant_Type>&client_id=<Client_ID>'
```

The following is an example in CURL:

```
curl --cert x509certificate.pem --key x509privatekey.pem
-XPOST https://test-me.authentication.cert.eu20.hana.ondemand.com/oauth/
token -d 'grant_type=client_credentials&client_id=sb-na-
edb111d1-2c22-3eca-444-869fde307ac7!a5017'
```

Related Information

Enable mTLS Authentication to SAP Authorization and Trust Management Service for Your Application [page 570]

4.1.6.2 Tutorials for the SAP Authorization and Trust Management Service

Follow the tutorials below to get familiar with the SAP Authorization and Trust Management service in the Cloud Foundry environment of SAP BTP.

Tutorials for the SAP Authorization and Trust Management service in the Cloud Foundry environment	Language / Framework	Link
Learn how to secure a basic single-tenant Node.js application. Start with a Node.js application that uses the express framework and SAPUI5 to display a list of products and add the security components step by step.	Node.js	SAP Developers

Tutorials for the SAP Authorization and Trust Management service in the Cloud Foundry environment	Language / Framework	Link
Learn how to secure a basic java application. This tutorial starts with a Hello World Java application built with SAP Cloud SDK.	Java, SAP Cloud SDK	SAP Developers
Learn how to build a cloud-native Node.js application that features secured service-to-service communication. The application shows you two different ways of securing service-to-service-communication (by propagating a business user or using a technical user).	Node.js	GitHub
Learn how to secure microservices in SAP BTP using `spring-xsuaa` and Spring security. Furthermore, learn how to test the secured application using the `java-security-test` utilities.	Spring (Boot)	GitHub
Learn how to add multitenancy to a node.js application and make it available for other subaccounts using the SaaS Provisioning service and the XSUAA.	Node.js	SAP Developers
Learn how to secure microservices in SAP BTP. This sample provides J2EE Configuration using web.xml and uses the SAP Java Buildpack.	J2EE, SAP Java Buildpack	GitHub (SAP Java Buildpack version >=1.26.1) GitHub (SAP Java Buildpack version <= 1.26.0)
Learn how to use the `java-security` library to perform JWT Validation as part of your Java application. Furthermore, learn how to test the secured application using the `java-security-test` utilities.	Java	GitHub

Tutorials for the SAP Authorization and Trust Management service in the Cloud Foundry environment	Language / Framework	Link
Learn how to validate OAuth tokens using a Python library. Use this library to add authentication in your Python application. **Note** This library isn't part of an SAP BTP license. However, it belongs to a related open source project.	Python	GitHub
Learn in this reference application how the service fits into a complete architecture of microservices that interact with each other propagating user information.	Java	GitHub

4.1.6.3 Tasks On Demand

Here you can find information regarding the available extended tasks for creating authorization artifacts, as well as the process of configuring redirect URLs.

In addition, as an application developer, you may want to create role collections for immediate use. For example, to deliver role collections that administrators can use in the SAP BTP cockpit and easily assign to users in an onboarding process.

Update a Service Instance [page 577]
: You can update a service instance from the `xsuaa` service using the service broker.

Sharing and Unsharing Service Instances [page 578]
: You can share service instances of the SAP Authorization and Trust Management service across multiple spaces or environments. This has the advantage that you don't need to create separate service instances in multiple spaces or environments.

Compatible Changes in the Security Descriptor File [page 579]
: If you update a service instance, you can add, change, and/or delete scopes, attributes, and role templates. Whenever you change role templates, you adapt roles that are derived from the changed role template.

Retrieve Credentials for Remote Applications [page 580]
: You need to get the credentials of a service instance for an application that runs outside SAP BTP.

Configure Redirect URLs for Browser Logout [page 582]
: To avoid open redirect attacks, direct users to a safe and valid URL when they log out.

Create Role Collections with Predefined Roles [page 585]

As an application developer, you want to create role collections for immediate use. You want to deliver role collections that administrators can use in the cockpit, and easily assign to users, for example in an onboarding process.

Validation and Revocation of Access Tokens [page 586]
To confirm that access tokens were truly issued by the SAP Authorization and Trust Management service and are still valid, applications check the tokens for validity.

Include Tokens from Corporate Identity Providers or SAP Cloud Identity Services in Tokens of the SAP Authorization and Trust Management Service [page 587]
In scenarios where your SAP BTP application interacts with applications that don't support tokens issued by the SAP Authorization and Trust Management service, but require a token from a corporate identity provider or SAP Cloud Identity Services, you can configure SAP Authorization and Trust Management service to include this token.

Best Practices When Handling Access Tokens [page 590]
Calls to SAP Authorization and Trust Management service can fail for many reasons, including network latency, connection errors, service downtime, idle connections closed by routing components and so on. The following sections provide recommendations on when to retry connections and the caching of tokens.

4.1.6.3.1 Update a Service Instance

You can update a service instance from the `xsuaa` service using the service broker.

Context

You are running a service instance that grants user access to an application. It uses the security descriptor file `xs-security.json`. If you change properties, for example, you want to reflect the compatible changes you made in the `xs-security.json` file in an existing service instance.

> ⚠ Restriction
>
> If you define or update role templates and attributes in the `xs-security.json` file, observe the following configuration restrictions in **Relationship Between `default-values` of `attribute-references` and `valueRequired`**.

Procedure

1. Edit the `xs-security.json` file and make your changes in the security descriptors.
2. Update the service instance. During the update, you use the security descriptors you changed in the `xs-security.json` file.

   ```
   cf update-service <service_instance_name> -c xs-security.json
   ```

> ✤ Example
>
> ```
> cf update-service authorizationtest-uaa -c xs-security.json
> ```

4.1.6.3.2 Sharing and Unsharing Service Instances

You can share service instances of the SAP Authorization and Trust Management service across multiple spaces or environments. This has the advantage that you don't need to create separate service instances in multiple spaces or environments.

- You must have the *Subaccount Service Administrator* role collection in the specified subaccount. Use the SAP BTP cockpit to assign this role collection (see Assigning Role Collections to Users or User Groups [page 2616].)

Using the SAP BTP command line interface (BTP CLI), you can share service instances across spaces or environments (for example, Cloud Foundry and Kubernetes). Shared service instances don't have the same credential. The client ID can be the same or different. Sharing service instances is possible with service instances of the `application` and `broker` plan.

> ⓘ Note
>
> It depends on the specific service whether sharing and unsharing servivc instances is available.

> ⚠ Restriction
>
> You can only share service instances in the same subaccount.

For more information, see btp share services/instance and btp unshare services/instance.

4.1.6.3.3 Compatible Changes in the Security Descriptor File

If you update a service instance, you can add, change, and/or delete scopes, attributes, and role templates. Whenever you change role templates, you adapt roles that are derived from the changed role template.

Compatible Changes in xs-security.json

Security Artifacts in the xs-security.json File	Action

Scope	• Add
	• Delete
	• Change
	  • `description`
	  • `granted-apps`

Attribute	• Add • Delete • Change • `description` **Note** Do not change the `valueType` of the attribute.

Role template	• Add
	• Delete
	• Change
	  • `description`
	  • `scope-references`
	  • `attribute-references` (can be deleted only)
	
	Restriction
	When you define or update role templates and attributes, observe the following configuration restrictions in **Relationship Between `default-values` of `attribute-references` and `valueRequired`**.

4.1.6.3.4 Retrieve Credentials for Remote Applications

You need to get the credentials of a service instance for an application that runs outside SAP BTP.

Prerequisites

- A service instance for your application is available.

  > ⓘ Note
  >
  > For mutual transport layer security (mTLS), the application security descriptor supports the `binding-secret` and `x509` credential types by default. For the creation of a service key that enables the `x509` credential type, you need to pass a parameter file. See Binding Parameters of SAP Authorization and Trust Management Service [page 2640].

Context

> ⓘ Note
>
> Applications that run inside the service manager instance at SAP BTP get their credentials after the respective service has been bound to the application. However, you can't use binding for an application that runs outside of the service manager instance at SAP BTP.

Instead, you use a service key that is created in the service instance of the remote application. You need to get the credentials of the service instance for the remote application. The UAA service broker manages all credentials, including those of the remote applications. The credentials you need are the OAuth 2.0 client ID and the client secret, or X.509 certificates.

First you generate a service key for the service instance of the remote application to enable access to the UAA service broker. Then you retrieve the generated service key with the credentials, including the OAuth 2.0 client ID and the client secret or X.509 certificates, from the UAA service broker. The remote application stores the service key. The application can now use this service key with the credentials (OAuth 2.0 client ID and the client secret of the remote application).

> → Remember
>
> Rotate credentials before they expire or, if they don't expire, rotate them regularly (see Security Recommendations for SAP Authorization and Trust Management Service [page 3599]).

Procedure

1. Create a service key for the service instance of the remote application. If you want to use mutual transport layer security (mTLS), you may need to request it in a separate binding configuration file (see Enable mTLS

Authentication to SAP Authorization and Trust Management Service for Your Application [page 570] and Binding Parameters of SAP Authorization and Trust Management Service [page 2640]).

```
cf create-service-key <service_instance_name> <service-key-name>
```

> ### Example
>
> ```
> cf create-service-key rem-app-service rem-app-sk
> ```

(For mTLS) `cf create-service-key <service_instance_name> <service-key-name> -c <parameter_file>`

> ### Example
>
> ```
> cf create-service-key rem-app-service rem-app-sk -c parameters.json
> ```

The `parameters.json` file can have the following content for the usage of XSUAA-managed certificates:

> ### Sample Code
>
> ```
> {
>  "credential-type": "x509"
> }
> ```

2. You want to retrieve the credentials including the OAuth 2.0 client ID and the client secret for the service instance of your remote application. Use the following command:

```
cf service-key <service_instance_name> <service_key_name>
```

> ### Example
>
> ```
> cf service-key rem-app-service rem-app-sk
> ```

Output with default `binding-secret` credential type:

> ### Output Code
>
> ```
> {
>   "apiurl": "https://api.authentication.corpdcen.acme.ondemand.com",
>   "clientid" : "sb-sample-app!t1",
>   "clientsecret" : "<client_secret>",
>   "credential-type": "binding-secret",
>   "identityzone" : "uaa",
>   "serviceInstanceId": "cbb63529-d725-4326-880c-92cbcdd92",
>   "subaccountid": "194e6a6d-c590-5030-98b3-1baa6d8fcda4",
>   "tenantid": "095e5a7f-c480-5030-98b3-1cbb6e8fdfb4",
>   "uaadomain": "authentication.corpdcen.acme.ondemand.com",
>   "url" : "https://host.acme.com:40654/uaa-security",
>   "xsappname" : "sample-app!t1",
>   "..."
> }
> ```

(For mTLS) Output with `x.509` credential type:

> ### Sample Code
>
> ```
> {
>   "credentials": {
> ```

```
    "apiurl": "https://api.authentication.corpdcen.acme.ondemand.com",
    "certificate": "-----BEGIN CERTIFICATE-----
\nMIIFvDCCA6...KJu+8fcIaUp7MVBIVZ\n>-----END CERTIFICATE-----\n",
    "certificate-pinning": true,
    "certurl": "https://mysubaccount-
hmq7frea.authentication.cert.corpdcen.acme.ondemand.com",
    "clientid": "sb-x5app!t75135",
    "credential-type": "x509",
    "key": "-----BEGIN RSA PRIVATE KEY-----\<private_key>\n-----END RSA
PRIVATE KEY-----\n",
    "serviceInstanceId": "cbb52529-d614-4326-880c-92ba937bcc92",
    "subaccountid": "<subaccount_ID>",
    "tenantid": "194e6b7e-c370-4941-98b3-1cbb6e8fdfb4",
    "uaadomain": "authentication.corpdcen.acme.ondemand.com",
    "url": "https://<subaccount>-
hmq7frea.authentication.corpdcen.acme.ondemand.com",
    "xsappname": "x5app!t75135",
    "..."
  }
}
```

If your service needs an X.509 certificate and private key for mutual transport layer security (mTLS) in a single privacy enhanced mail (PEM) file, see Extract Certificates for Mutual Transport Layer Security.

4.1.6.3.5 Configure Redirect URLs for Browser Logout

To avoid open redirect attacks, direct users to a safe and valid URL when they log out.

Prerequisites

- You've an application that is using the application router.
- Use application router 5.7.0 or higher. For more information, see the related link.

Context

You’ve deployed an application in the Cloud Foundry environment of SAP BTP. Business users log in and out of the application. As an application developer, you want to set a redirect URI, which directs users to a specific page once they've logged out. This page can be a logout page or your organization’s employee portal. For security reasons, there’s a check of this redirect URI.

- In the application descriptor (`xs-app.json`), you define a public logout page.
- To avoid redirects, for example to malicious web sites, `xsuaa` compares this public URL with a list of allowed redirect URLs. You define them in the OAuth 2.0 configuration property of the application security descriptor (`xs-security.json`).

Procedure

1. Define `xs-app.json` in the application router folder of your application. Include a logout endpoint and define a logout page (here **`logout.html`**) which can be accessed without authentication.

   Sample Code

   ```
   {
       "welcomeFile": "index.html",
       "authenticationMethod": "route",
       "logout": {
           "logoutEndpoint": "/my/logout",
           "logoutPage": "logout.html"
       },
       "routes": [{
           "source": "^/hw/",
           "target": "/",
           "destination": "hw-dest",
           "scope": {
               "GET": [
                   "$XSAPPNAME.Display",
                   "$XSAPPNAME.Update"
               ],
               "default": "$XSAPPNAME.Update"
           }
       },
       {
           "source": "/index.html",
           "localDir": "resources",
           "cacheControl": "no-cache, no-store, must-revalidate"
       },
       {
           "source": "/logout.html",
           "localDir": "resources",
           "authenticationType": "none"
       },
       {
           "source": "^/(.*)",
           "localDir": "resources"
       }]
   }
   ```

2. Open a command prompt.
3. Log in to your Cloud Foundry environment using the Cloud Foundry command-line interface (cf CLI).
4. Choose your subaccount.

   > Note
   >
   > The cf CLI prompts you to choose an org. To find the org of your subaccount, use the cockpit to go to your subaccount. You can find the org in the Cloud Foundry tile under *Organization* (see Navigate to Orgs and Spaces [page 2792]).

5. Choose the space where your application is located.
6. Deploy the application to your space.
7. To define the redirect URIs, go to the folder where `xs-security.json` is stored.
8. Define the redirect URIs in the `oauth2-configuration` property.
9. Add a URL that contains the hostname of your application.

Sample Code

```
{
 "xsappname": "<application_name>",
 "tenant-mode": "dedicated",
 "description": "My Sample Application with Application Router",
 "oauth2-configuration": {
                                          "token-validity": 900,
                                          "redirect-uris": ["https://
<application_hostname>.<landscape_domain>/**"
                                                           ],
                                          "autoapprove": "true"
}
}
```

Note

The application domain is the first part of the application URL: For more information, see the related link.

Example

myapplication.cfapps.hana.ondemand.com/**

Example

For China (Shanghai) region:

myapplication.mycustomdomain.cn/**

10. Update the `xsuaa` service instance that is bound to your application.

```
cf update-service <service_instance> -c xs-security.json
```

Note

If the service instance isn't available, create it, bind your application to the service instance (see the related links), and restage the application.

You've implemented a valid redirect after logout, and `xsuaa` checks the redirect URL against the list of allowed URIs in the OAuth 2.0 configuration.

Related Information

Application Router [page 453]
Configuring Application URLs
Update a Service Instance [page 577]
Cloud Foundry Command Line Interface
Application Security Descriptor Configuration Syntax [page 592]
Security Considerations for the SAP Authorization and Trust Management Service [page 3591]

4.1.6.3.6 Create Role Collections with Predefined Roles

As an application developer, you want to create role collections for immediate use. You want to deliver role collections that administrators can use in the cockpit, and easily assign to users, for example in an onboarding process.

Prerequisites

- You have the `Space Developer` role in this subaccount (see the related link).

Context

You define the role collections in the application security descriptor file (`xs-security.json`). These role collections reference role templates. As soon as you've deployed your application, the cockpit displays the role collections. They contain predefined roles.

Procedure

1. Deploy an application you want to use for creating security artifacts.
2. Edit the application descriptor file (`xs-security.json`) and add the `role-collections` property. For more information, see Application Security Descriptor Configuration Syntax [page 592].

Sample Code

```
{
"role-templates": [
      {
        "name": "Viewer",
        "description": "View Users",
        "scope-references": [
          "$XSAPPNAME.Display"
        ]
      },
      {
        "name": "Manager",
        "description": "Maintain Users",
        "scope-references": [
          "$XSAPPNAME.Display",
          "$XSAPPNAME.Update"
        ]
     }
    ],
"role-collections": [
        {
        "name": "UserManagerRC",
        "description": "User Manager Role Collection",
        "role-template-references": [
          "$XSAPPNAME.Viewer",
```

```
            "$XSAPPNAME.Manager"
              ]
          }
    ]
  }
```

3. Go to the folder where the application security descriptor (`xs-security.json`) file is stored.
4. To deploy the security information, create a service using your `xs-security.json` file.

   ```
   cf create-service xsuaa application <service_name> -c xs-security.json
   ```

   > ✤ Example
   >
   > ```
   > cf create-service xsuaa application rolecoll-serv -c xs-security.json
   > ```

5. (If you do not use a manifest file) Bind your application to the service.

   ```
   cf bind-service <application_name> <service_name>
   ```

   > ✤ Example
   >
   > ```
   > cf bind-service rcpropertyapp rolecoll-serv
   > ```

   You have created a role collection that is visible in the cockpit. It contains predefined roles. Using the cockpit, administrators can assign this role collection to users.

Related Information

About Roles in the Cloud Foundry Environment [page 2788]
Adding Authentication and Authorization [page 546]
Deploy Business Applications in the Cloud Foundry Environment [page 845]
Mapping Role Collections [page 2620]
Tutorials for the SAP Authorization and Trust Management Service [page 574]
Application Security Descriptor Configuration Syntax [page 592]

4.1.6.3.7 Validation and Revocation of Access Tokens

To confirm that access tokens were truly issued by the SAP Authorization and Trust Management service and are still valid, applications check the tokens for validity.

Access tokens are JSON web tokens (JWT). When the SAP Authorization and Trust Management service issues access tokens for an application, the service sets a validity and digitally signs the token. The default validity for a token is 12 hours, but you can set the validity as part of the OAuth configuration in the application security descriptor (`xs-security.json`).

Applications must reject the access token in the following cases:

- The token is no longer valid.
  The token could have expired or has been revoked.

- The signature on the token doesn't match.

Online or Offline Validation

As an application developer, you have the design decision, whether to perform validation of the token online or offline.

In online validation, you send a request to the SAP Authorization and Trust Management service to determine that the access token is still valid and hasn't been revoked. To check the validity, use the `introspect` endpoint of the standard Cloud Foundry API.

Your application can perform offline validation with the help of our client libraries. You must get the public key from the SAP Authorization and Trust Management service and use the key to check the validity. While validating a token offline can ensure that your application can continue to operate if the SAP Authorization and Trust Management service is unavailable, your application continues to accept revoked tokens until they reach the end of their validity. A malicious user with a revoked token still has access as long as the token is still valid.

To revoke a token, use the `revoke` endpoint of the standard Cloud Foundry API.

Related Information

Rotate Signing Keys of Access Tokens [page 2646]
Application Security Descriptor Configuration Syntax [page 592]
https://docs.cloudfoundry.org/api/uaa/version/74.4.0/index.html#introspect-token
https://docs.cloudfoundry.org/api/uaa/version/74.4.0/index.html#revoke-tokens
XSUAA authentication and authorization service integration libraries and samples for authenticating users and services on GitHub

4.1.6.3.8 Include Tokens from Corporate Identity Providers or SAP Cloud Identity Services in Tokens of the SAP Authorization and Trust Management Service

In scenarios where your SAP BTP application interacts with applications that don't support tokens issued by the SAP Authorization and Trust Management service, but require a token from a corporate identity provider or SAP Cloud Identity Services, you can configure SAP Authorization and Trust Management service to include this token.

Prerequisites

- Trust with an SAP Cloud Identity Services tenant is configured with OpenID Connect (OIDC).

For more information, see Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2540].

- Trust with a corporate identity provider by your SAP Cloud Identity Services is configured with OIDC. For more information, see Configure Trust with OpenID Connect Corporate Identity Provider in the SAP Cloud Identity Services documentation.
- The initial logon of your application uses the authorization code flow.

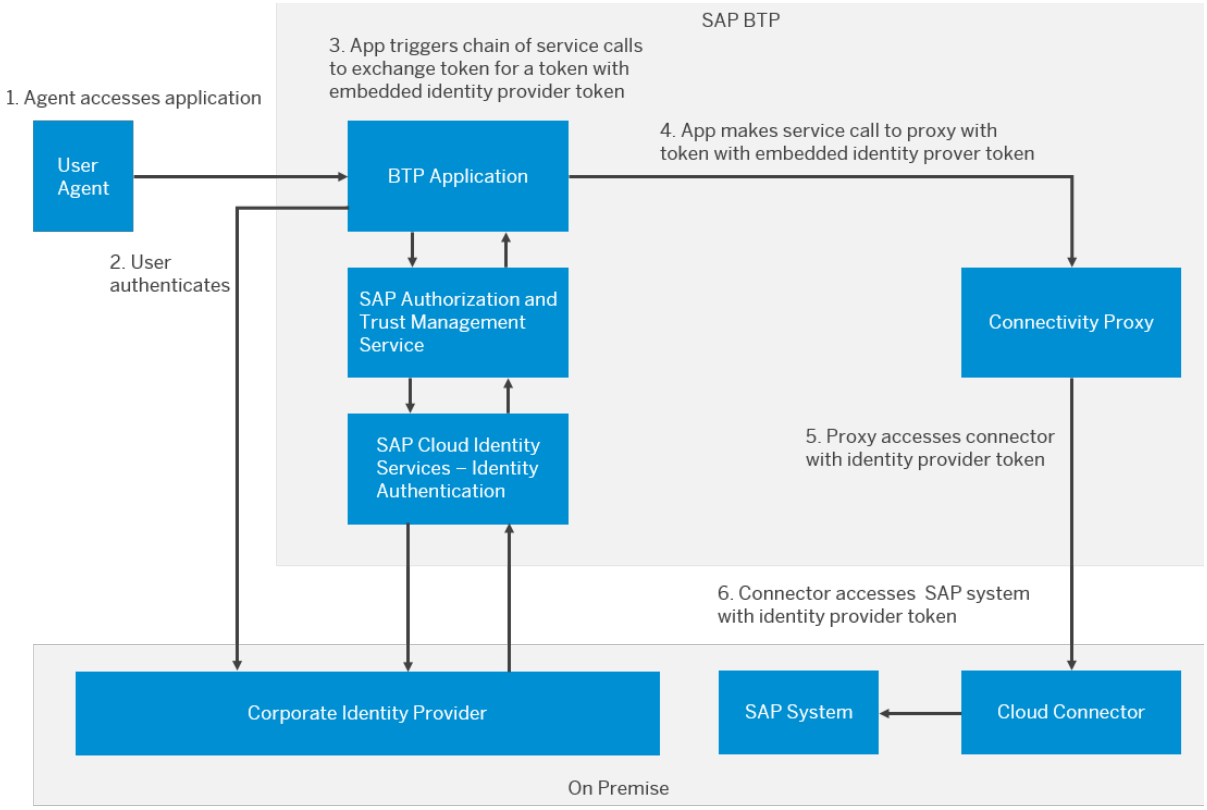
Context

Imagine you have an on-premise SAP system that trusts your corporate provider. You want to access data in that system from an application on SAP BTP. Your application can get a token from the SAP Authorization and Trust Management service, but you don't want to or can't configure your SAP system to trust that token.

In this scenario, configure the SAP Authorization and Trust Management service to embed the token of a corporate identity provider trusted by the trusted SAP Cloud Identity Services tenant. In this example, before the SAP BTP application makes a service call to the Cloud Connector proxy, the application calls the SAP Authorization and Trust Management service to exchange its current token for a new token that embeds a token from the identity provider. Cloud Connector presents the identity provider token to the on-premise SAP system.

The following figure illustrates this scenario.

Cloud Connector Using Embedded Tokens



> ⚠ Caution
>
> Token embedding is provided as a best-effort solution. The BTP application must be designed to fail gracefully when the SAP Authorization and Trust Management service is unable to embed tokens from SAP Cloud Identity Services or the corporate identity provider. SAP Authorization and Trust Management service records such events in the audit log.

Procedure

1. When deploying your application, ensure that the application security descriptor (`xs-security.json`) sets the `system-attributes` with one of the following options:

Value	Description
`ias-corporate-idp-token`	Embeds the token of the corporate identity provider trusted by your SAP Cloud Identity Services tenant in the token issued by the service.
`ias-token`	Embeds the token of your trusted SAP Cloud Identity Services tenant in the token issued by the service.

   For more information about the application security descriptor, see Application Security Descriptor Configuration Syntax [page 592].

2. In the SAP Cloud Identity Services application that represents your SAP BTP application, add the self-defined attribute **`xsuaa-persist-corporate-idp-token`** with the **`Expression`** source and **`true`** value.

   This attribute ensures that the service knows to embed the tokens.

   For more information, see Configuring Attributes Based on Flexible Expressions in the SAP Cloud Identity Services documentation.

Results

The service can embed tokens of trusted identity providers in its tokens.

> ⚠ Caution
>
> Your deployed application might not receive a token from the corporate identity provider even if it is configured to recognize the token from the corporate identity provider.
>
> We recommend that applications implement one retry for the token retrieval if the received token does not contain the corporate identity provider token. Additionally, applications should react appropriately if the corporate identity provider token is still missing after the retry.
>
> To analyze the issue, refer to the auditing and logging information. See Auditing and Logging Information for SAP Authorization and Trust Management Service [page 3555].

Next Steps

If you use Cloud Connector, ensure that the service doesn't trust the SAP Authorization and Trust Management service. To use the token of the corporate identity provider, ensure that the service doesn't trust the SAP Cloud Identity Services tenant, too.

For more information, see Configure Trusted Entities in the Cloud Connector in the documentation of SAP BTP Connectivity.

4.1.6.3.9 Best Practices When Handling Access Tokens

Calls to SAP Authorization and Trust Management service can fail for many reasons, including network latency, connection errors, service downtime, idle connections closed by routing components and so on. The following sections provide recommendations on when to retry connections and the caching of tokens.

How your application handles access tokens has an impact on its resiliency. Cache tokens to avoid running into the rate limiting restrictions of the SAP Authorization and Trust Management service. Retry with cached tokens instead of always getting a new one.

For recommended configuration settings, see Security Considerations for the SAP Authorization and Trust Management Service [page 3591].

Retry Mechanism

When your application or service tries to get a token and fails, retry the connection. A retry mechanism helps your application handle temporary issues. We recommend retrying token retrieval three times by default. The following table lists when to retry and how often.

Retry and Backoff for Different HTTP Responses

Event Type	HTTP Status Code	Retry	Backoff Policy
Connection error	None	Yes, retry up to three times.	Initial retry after 300 ms with a multiplier of 2.
Timeout	• 408: Request timeout • 502: Bad gateway For example, request timeout from a Cloud Foundry Gorouter. • 504: Gateway timeout	Yes, retry up to three times.	
Rate Limiting	429: Too many requests	Yes, retry up to three times.	Base retry on the `Retry-After` header.

Event Type	HTTP Status Code	Retry	Backoff Policy
Other Events	• 500: Internal server error • 503: Service unavailable	Yes, retry up to three times.	Initial retry after 1000 ms with a multiplier of 3.
Access Control	• 401: Unauthorized • 403: Forbidden	No, don't retry.	None.

Token Cache

We recommend that you cache access tokens in your application as long as they're valid, instead of always getting a new token. If you constantly get a new token, you risk running up against the rate limits in the service and downtime for your application.

For more information, see Rate Limiting [page 3600].

However, renewing tokens before they actually expire improves the resiliency of your applications. If your application only renews tokens at their expiration and the service isn't available at that moment, your users experience a downtime. For example, if your token validity is set to 12 hours, try to renew the token after 6 hours.

Having a local cache also improves the performance of your applications.

Implement a method to remove invalid tokens from your cache. When your application calls with a token from your cache and receives the return codes in the following table, remove the token from your cache and request a new token.

Triggers for Removing Tokens from Your Cache

HTTP Status Code	Potential Causes	Application Response
401: Unauthorized	• The signing key of the token was rotated. For more information, see Rotate Signing Keys of Access Tokens [page 2646]. • Token expired and your application hasn't removed it from your cache yet.	Remove the token from the cache and request a new token.
403: Forbidden	Occurs if the token is valid but a scope is missing. **Note** Scopes of the client or user can have changed during the caching period.	

→ Tip

SAP Authorization and Trust Management service ensures that keys remain valid for 2 hours after key rotations. Exception: When subaccount administrators force a key rotation, the key is immediately invalid. We recommend that you configure your cache with a 2-hour time-to-live (TTL). If you use a longer TTL, your application must be able to invalidate cache entries when the service responds with 401. Have your application only retry once on failure.

⚠ Restriction

When administrators force key rotation, applications that use cached tokens must accept a downtime until their caches have been cleared.

Related Information

https://github.com/SAP/cloud-security-services-integration-library/tree/main/token-client#cache-configuration

4.1.6.4 Reference Information

Use additional reference information such as the syntax required to set the properties and values defined in the application security descriptor file.

You can also authenticate Node.js applications. A collection of Node.js packages developed by SAP is provided as part of the Cloud Foundry environment at SAP BTP

Application Security Descriptor Configuration Syntax [page 592]
The syntax required to set the properties and values defined in the `xs-security.json` application security descriptor file.

Client Libraries for SAP BTP Security Services [page 611]
We provide client libraries to simplify your consumption of our services.

4.1.6.4.1 Application Security Descriptor Configuration Syntax

The syntax required to set the properties and values defined in the `xs-security.json` application security descriptor file.

```
{
  "xsappname" : "node-hello-world",
  "scopes"     : [ {
                    "name" : "$XSAPPNAME.Display",
                    "description" : "display" },
                  {
```

```
                    "name" : "$XSAPPNAME.Edit",
                    "description" : "edit" },
                  {
                    "name" : "$XSAPPNAME.Delete",
                    "description": "delete",
                    "granted-apps": ["$XSAPPNAME(application,business-partner)"]
                  }
],
 "attributes" : [ {
                    "name" : "Country",
                    "description" : "Country",
                    "valueType" : "string" },
                  {
                    "name" : "CostCenter",
                    "description" : "CostCenter",
                    "valueType" : "string" }
                ],
 "role-templates": [ {
                    "name"                : "Viewer",
                    "description"         : "View all books",
                    "default-role-name": "Viewer: Authorized to Read All Books",
                    "scope-references"    : [
                                           "$XSAPPNAME.Display" ],
                    "attribute-references": [
                                             {
                                             "name" : "Country",
                                             "default-values" : [
                                                                  "USA", "Germany"
                                                                ]
                                             }
                                             ]
                  },
                  {
                    "name"               : "Editor",
                    "description"        : "Edit, delete books",
                    "scope-references"   : [
                                            "$XSAPPNAME.Edit",
                                            "$XSAPPNAME.Delete" ],
                    "attribute-references" : [
                                            "Country",
                                            "CostCenter"]
                  }
                  ],
 "role-collections": [
                     {
                    "name": "UserViewerRC",
                    "description": "User Viewer Role Collection",
                    "role-template-references": ["$XSAPPNAME.Viewer"]
                     }
                  ],
 "authorities":["$ACCEPT_GRANTED_AUTHORITIES"],
 "oauth2-configuration": {
                    "token-validity": 900,
                    "redirect-uris": ["https://
myapp.cfapps.eu10.hana.ondemand.com","https://myapp.mydomain.com/my/logout"],
                    "credential-types": ["binding-secret","x509"]
 },
 "xsenableasyncservice":"true"
}
```

→ Tip

Try out the tutorials for the SAP Authorization and Trust Management service to get familiar with using the application security descriptor in the Cloud Foundry environment of SAP BTP.

See Tutorials for the SAP Authorization and Trust Management Service [page 574].

xsappname

Use the `xsappname` property to specify the name of the application that the security description applies to.

```
"xsappname" : "<app-name>",
```

Naming Conventions

Bear in mind the following restrictions regarding the length and content of an application name in the `xs-security.json` file:

- The following characters can be used in an application name of the Cloud Foundry environment at SAP BTP: `aA–zZ`, `0–9`, `-` (hyphen), `_` (underscore), `/` (forward slash), and \ (backslash).
- The maximum length of an application name is 100 characters.

tenant-mode (Custom Option)

Use the custom `tenant-mode` property to define the way the tenant's OAuth clients get their client secrets.

During the binding of the `xsuaa` service, the UAA service broker writes the tenant mode into the credential section. The application router uses the tenant mode information for the implementation of multitenancy with the application service plan.

Sample Code

```
{
     "xsappname"     : "<application_name>",
     "tenant-mode"   : "shared",
     "scopes"        : [
                         {
                           "name"                 : "$XSAPPNAME.Display",
                           "description"         : "display"
```

Syntax

```
"tenant-mode" : "shared"
```

The following tenant modes are available:

Tenant Modes

Value	Description
`dedicated`	Default value. An OAuth client gets a separate client secret for each subaccount.
`shared`	An OAuth client always gets the same client secret. It's valid in all subaccounts. The application service plan uses this tenant mode.

Value	Description
`external`	A tenant has multiple subscriptions to applications. For each subscription to an application, the tenant gets an OAuth client with a client secret.

scopes

In the application security file (`xs-security.json`), the `scopes` property defines an array of one or more security scopes that apply for an application. You can define multiple `scopes`; each scope has a name and a short description. The list of scopes defined in the `xs-security.json` file is used to authorize the OAuth client of the application with the correct set of **local** and **foreign** scopes; that is, the permissions the application requires to be able to respond to all requests.

```
"scopes": [
            {
              "name" : "$XSAPPNAME.Display",
              "description" : "display"
            },
            {
              "name" : "$XSAPPNAME.Edit",
              "description" : "edit"
            },
            {
              "name" : "$XSAPPNAME.Delete",
              "description" : "delete",
              "granted-apps" : [ "$XSAPPNAME(application,business-partner)"]
            }
          ]
```

Local Scopes

All scopes in the `scopes` file is used to authorize the OAuth client of the application with the correct set section are local, that is, application specific. Local scopes are checked by the application's own application router or checked programmatically within the application's runtime container. In the event that an application needs access to other services of the Cloud Foundry environment on behalf of the current user, the provided access token must contain the required foreign scopes. Foreign scopes are not provided by the application itself; they're checked by other sources outside the context of the application.

In the `xs-security.json` file, “local” scopes must be prefixed with the variable `<$XSAPPNAME>` at run time. The variable is replaced with the name of the corresponding local application name.

> → Tip
>
> The variable `<$XSAPPNAME>` is defined in the application's deployment manifest description (`manifest.yml`).

Scopes with Critical Security Authorizations

Critical security authorizations, which are scopes prefixed with `xs_`, become transparent for custom roles collections and SAP-delivered roles. These role collections have either been created by administrators, or they include roles with critical authorizations configured in the application security descriptor file (`xs-security.json`).

The following critical security authorizations are available as scopes:

Scopes with Critical Authorizations

Scope	Description
`xs_authorization.read`	Read authorizations
`xs_authorization.write`	Manage authorizations
`xs_user.read`	Read users
`xs_user.write`	Manage users
`xs_idp.read`	Read trust configurations
`xs_idp.write`	Manage trust configurations
`xs_apicredential.read`	Read API credentials
`xs_apicredential.write`	Manage API credentials

Administrators manage role collections with critical security authorizations or add roles with such authorizations to role collections. They must be aware of the criticality of the security role collections for existing or new assignments to users, user groups, and attributes.

Foreign Scopes

Usually, “foreign” scopes include the service plan and the name of the application to which the scope belongs. For more information, see Referencing the Application Use the following syntax:

```
$XSAPPNAME(<service_plan>,<xsappname>).<local_scope_name>
```

> Example
>
> ```
> "$XSAPPNAME(application,business-partner).Create"
> ```

Granting Scopes to Another Application

If you want to grant a scope from this application to another application for a user scenario, this application needs to grant access to the scope for the application that wants to use this scope. Using the `granted-apps` property in the `scopes` section, you can specify the application you want to grant your scope to. The other application (referenced as `<service_plan>,<foreign_xsappname>`) receives the scope as a “foreign”scope. For more information, see the related link.

Here is the syntax in the security descriptor of the application that grants the scope. For more information, see Referencing the Application.

```
"granted-apps" : [ "$XSAPPNAME(<service_plan>,<foreign_xsappname>)"]
```

> Example
>
> ```
> "granted-apps" : [ "$XSAPPNAME(application,business-partner)"]
> ```

If you want to grant a scope from this application to another application for a client credentials scenario, use the `grant-as-authority-to-apps` property in the `scopes` section. In this case, the scopes are granted as authorities. Specify the other application by name. For more information, see the related link.

Here is the syntax in the security descriptor of the application that grants the scope:

```
"grant-as-authority-to-apps" : [ "$XSAPPNAME(<service_plan>,<foreign_xsappname>)"]
```

> Example
>
> ```
> "grant-as-authority-to-apps" : [ "$XSAPPNAME(application,business-partner)"]
> ```

Naming Conventions

Bear in mind the following restrictions regarding the length and content of a scope name:

- The following characters can be used in a scope name: `aA–zZ`, `0–9`, `-` (hyphen), _ (underscore), `/` (forward slash), \ (backslash), and `:` (colon).
- Scope names can't start with a leading `.` (period). For example, `.myScopeName1`.
- The maximum length of a scope name, including the fully qualified application name, is 193 characters.

For other limits when using the properties, see Limits for the Application Security Descriptor [page 3606].

attributes

In the application security descriptor (`xs-security.json`), the `attributes` property enables you to define an array, listing one or more attributes that are applied to the role templates also defined in the `xs-security.json` element is only relevant for user scenarios file. You can define multiple `attributes`.

```
"attributes" : [
     {
      "name" : "Country",
      "description" : "Country",
      "valueType" : "s" },
     {
      "name" : "CostCenter",
      "description" : "CostCenter",
      "valueType" : "string" }
],
```

The `attributes` element is only relevant for a user scenario. Each element of the attributes array defines an attribute. These attributes can be referenced by role templates. There are multiple sources of attributes:

- Static attributes
  Use the SAP BTP cockpit to assign the value of the attributes. You can use the static value.
- Attributes from an identity provider
  You can reference the attributes provided by an identity provider. The token is issued by the configured identity provider during authentication. You find the attribute value in the configuration of your identity provider. The attributes provided by the identity provider, appear as an attribute in the JSON web token, if the user has assigned the corresponding roles. You can use the attributes to achieve instance-based authorization checks.
- Unrestricted attributes

In this case, you want to express that it's not necessary to set a specific value for this attribute. The behavior is the same as if the attribute would not exist for this role.

For more information, see the related link about attributes.

The `attributes` definition can take the following properties:

attributes Properties

Key	Description	Example
`name`	The name of the attribute with a value to apply when building the role template	`Country`
`description`	A short summary of the attribute defined	`Country`
`valueType`	The type of value expected for the defined attribute; possible values are: `string` (or `s`), `int` (integer), or `date`	`int`
`valueRequired`	By default, every attribute needs dedicated attribute values. The default value is `true`. For more information, see Relationship Between `default-values` of `attribute-references` and `valueRequired`.	`true`

Naming Conventions

Bear in mind the following restrictions regarding the length and content of attribute names in the `xs-security.json` file:

- The following characters can be used to declare an xs-security attribute name in the Cloud Foundry environment: “aA”-“zZ”, “0”-“9”, “_” (underscore)
- The maximum length of a security attribute name is 64 characters.

For other limits when using the properties, see Limits for the Application Security Descriptor [page 3606].

role-templates

In the application-security file (`xs-security.json`), the `role-templates` property enables you to define an array listing one or more roles (with corresponding scopes and any required attributes), which are required to access a particular application module. You can define multiple `role-templates`, each with its own scopes and attributes.

Role templates can be delivered with default values for each attribute reference. When you deploy role templates with default values for each attribute reference, you create default roles.

`attribute-references` can contain a JSON array of multiple objects.

```
"role-templates": [
     {
      "name"                : "Viewer",
      "description"         : "View all books",
```

```
      "default-role-name"   : "Viewer: Authorized to Read All Books",
      "scope-references"    : [
                              "$XSAPPNAME.Display"
                              ],
      "attribute-references": [
                              {
                               "name" : "Country",
                               "default-values" : [
                                                   "USA", "Germany"
                                                  ]
                              }
  ]
```

`attribute-references` can contain a JSON array of string.

A role template must be instantiated. This is especially true with regard to any attributes defined in the role template and the specific attribute values, which are subject to customization and, as a result, can't be provided automatically. Role templates that only contain "role-templates": [{ "name" : "Viewer", "description" : "View all books", "default-role-name" : "Viewer: Authorized to Read All Books", "scope-references" : ["$XSAPPNAME.Display"], "attribute-references": ["Country"] },] “local”"role-templates": [scopes can be instantiated without user interaction. The same is also true for “foreign”scopes where the scope **owner** has declared consent in a kind of allowlist (for example, either for “public” use or for known “friends”).

> ### Note
>
> The resulting (application-specific) role instances need to be assigned to the appropriate user groups.

role-template Properties

Key	Description	Example
`name`	The name of the role to build from the role template	`Viewer`
`description`	A short summary of the role to build	`View all books`
`default-role-name`	The name of the role in Unicode The name can be up to 255 characters long. The naming conventions of `role-templates` don't apply here.	`Viewer: Authorized to Read All Books`
`scope-references`	The security **scope** to apply to the application-related role	`$XSAPPNAME.Display`

Key	Description	Example
`attribute-references`	One or more attributes to apply to the built role. You can use a JSON array of objects or of string. If you use an array of objects, `name` is the name of the attribute. Use the attribute name specified in the `attributes` section. `default-values`, for example `"default-values": ["LeaveRequestWorkflow"]`. If you don't want to configure default values, don't specify the `default-values` element. For more information, see Relationship between default-values of attribute-references and valueRequired. **→ Tip** If you want to deliver attributes that aren't restricted by attribute values, set `"valueRequired": false`.	**Sample Code** `{ "name" : "Country", "default-values" : ["USA", "Germany"] }` (an array of objects) `["Country"]` (an array of string)

Naming Conventions

Bear in mind the following restrictions regarding the length and content of role-template names in the `xs-security.json` file:

- The following characters can be used to declare an xs-security role-template name: "aA"-"zZ", "0"-"9", "." (period), "-" (hyphen), "_" (underscore).
- The maximum length of a role-template name is 64 characters.

For other limits when using the properties, see Limits for the Application Security Descriptor [page 3606].

role-collections

The optional `role-collections` property enables you to define role collections with predefined roles. Administrators use these predefined role collections. They can assign them to users during the initial setup of SAP BTP.

The `role-collections` property only makes sense if application developers reference role templates that can create default roles at deployment time.

Note

The `role-collections` element can only reference role templates of the same subaccount.

Sample Code

```
"role-collections": [
    {
     "name": "Employee",
     "description": "Employee roles",
     "role-template-references": [
                                  "$XSAPPNAME.Employee"
                                   ]
    }
]
```

role-collections Properties

Key	Description	Example
`name`	The name of the role collection to build **Caution** If you change the name of a role collection and update the service instance, you delete the role collection with the old name and create a role collection with the new name. This means that the newly created role collection doesn't have any user assignments.	`Employee`
`description`	A short summary of the role collection to build	`Employee roles`

`role-template-references`	The role templates referenced by the `role collections` property	`$XSAPPNAME.Employee`
		Syntax`$XSAPPNAME(<service_plan>,<xsappname>)`
		Example
		`$XSAPPNAME(application,myapp2)`

Naming Conventions

Bear in mind the following restrictions regarding the length and content of `role-collections` names in the `xs-security.json` file:

- The maximum length of a role-collection name is 64 characters.
- The maximum length of a role-collection description is 1000 characters.
- The `role-template-references` (mandatory) element is an array with references to role template definitions from the Cloud Foundry application or from other Cloud Foundry applications.

→ Tip

We recommend that you reference all role templates using either the `$XSAPPNAME` or `$XSAPPNAME(<service_plan>,<XSAPPNAME>)` prefix to link to the correct application where the role template is defined (see Referencing the Application).

Conditions

The role templates must fulfill one of the following conditions regarding attributes:

- Either they don't have attribute references.
- Or default attribute values are defined. It's also possible to add the `"valueRequired" : false` property to an attribute.

This makes sure that a default role is automatically created for the role-templates after developers have deployed the application. The role collections use the predefined roles, which have been created automatically.

For other limits when using the properties, see Limits for the Application Security Descriptor [page 3606].

authorities

To enable one (sending) application to accept and use the authorization scopes granted by another application, each application must be bound to its own instance of the `xsuaa` service. This is required to ensure that the applications are registered as OAuth 2.0 clients at the UAA. You must also add an `authorities` property to the security descriptor file of the sending application that is requesting an authorization scope. In the `authorities` property of the sending application's security descriptor, you can either specify that **all** scope authorities configured as grantable in the receiving application should be accepted by the application requesting the scopes, or alternatively, only individual, named, scope authorities:

- Request and accept **all** authorities flagged as "grantable" in the receiving application's security descriptor:

  {·} Sample Code

  Specifying Scope Authorities in the Sending Application's Security Descriptor

  ```
  "authorities":["$ACCEPT_GRANTED_AUTHORITIES"]
  ```

- Request and accept only the "specified" scope authority that is flagged as grantable in the specified receiving application's security descriptor. For more information, see Referencing the Application.

  {·} Sample Code

  Specifying Scope Authorities in the Sending Application's Security Descriptor

  ```
  "authorities":["<ReceivingApp>.ForeignCall",
  "<ReceivingApp2>.ForeignCall",]
  ```

Since both the sending application and the receiving application are bound to UAA services using the information specified in the respective application's security descriptor, the sending application can accept and use the specified authority from the receiving application. The sending application is now authorized to access the receiving application and perform some tasks.

> ### Note
>
> The granted authority always includes the prefixed name of the application that granted it. This information tells the sending application, which receiving application has granted which set of authorities.

The scope authorities listed in the sending application's security descriptor must be specified as "grantable" in the receiving application's security descriptor.

> ### → Tip
>
> To flag a scope authority as "grantable", add the `grant-as-authority-to-apps` property to the respective scope in the receiving application's security descriptor.

For more information, see the related link.

oauth2-configuration (Custom Option)

Use the custom `oauth2-configuration` property to define custom values for the OAuth 2.0 clients, such as the token validity and redirect URIs.

The `xsuaa` service broker registers and uses these values for the configuration of the OAuth 2.0 clients.

> ### {·} Sample Code

```
"oauth2-configuration": {
     "token-validity": 43200,
     "redirect-uris": [
         "https://myapp.cfapps.eu10-004.hana.ondemand.com",
          "https://myapp.mydomain.com/my/content"],
     "refresh-token-validity": 1800,
     "credential-types": ["binding-secret","x509"],
     "system-attributes": ["groups","rolecollections"],
     "allowedproviders ": ["orgin_key1","origin_key2"]
     }
```

The following configuration keys are available:

oauth2-configuration Properties

Key	Description	Example
`token-validity`	Sets the token lifetime in seconds for access tokens issued by SAP Authorization and Trust Management service. The value ranges from 60 seconds to 86400 seconds, in other words, from 1 minute to 24 hours. Default: 43200 seconds (12 hours) **→ Recommendation** Relaxing the token policy means that users reauthenticate less. However, increasing the token validity also means that if a malicious user manages to steal a token, that malicious user has access until the token expires. Keep token validity in the subaccount as short as possible, but not less than 30 minutes. **Note** This change applies to applications, which consume or subscribe to this instance. These values override the values set for the subaccount. For more information, see Configure Token Policy for SAP Authorization and Trust Management Service [page 2656].	`1800` (30 minutes)
`refresh-token-validity`	Sets the token lifetime in seconds for refresh tokens issued by SAP Authorization and Trust Management service. The value ranges from 60 seconds to 31536000 seconds, in other words, from 1 minute to 1 year. Default: 604800 seconds (7 days)	`604800` (7 days)

Key	Description	Example
`redirect-uris`	This key contains a list of the redirect URIs that SAP BTP checks for when redirecting. If your landscape domain or custom domain aren't on this list, including wildcards, the Authorization and Trust Management Service won't redirect users there. We support explicit wildcards, namely domain relaxing and arbitrary paths. For example: `"https://*.mydomain.com/callback/**"` **⚠ Caution** If you use wildcards, we recommend that you make your URIs as specific as possible. By using wildcards, you open up the redirect for multiple web sites. Wildcards increase the risk of redirecting to malicious web sites. For more information, see Listing Allowed Redirect URIs [page 3593].	`["https://<application_hostname1>.<landscape_domain><path>","https://<application_hostname2>.<custom_domain><path>"]`
`credential-types`	Specifies the types of secrets available for binding applications to the service instance. The first type listed is the default type of secret. Otherwise, specify the type of secret when you create the binding or service key. For more information, see Service Instance Secrets [page 2634]. The default value is `binding-secret`. **→ Recommendation** Use `binding-secret` or `x509` so you can rotate the secret of a single binding without affecting other bindings of the service instance. For more information, see Rotating Secrets [page 3594].	`["binding-secret"]`

Key	Description	Example
`system-attributes`	Includes the system attributes in the JWT. If you don't define a value, the system includes the attributes for groups and role collections by default. If the size of the JWT becomes an issue for you, you can explicitly remove them. For example: `"system-attributes": [],` You can also have the service include a refresh token of a trusted identity provider. For more information, see Include Tokens from Corporate Identity Providers or SAP Cloud Identity Services in Tokens of the SAP Authorization and Trust Management Service [page 587]. **Values:** • **`"groups"`** includes the `xs.saml.groups` attribute. • **`"rolecollections"`** includes the `xs-rolecollections` attribute. • **`"ias-corporate-idp-token"`** includes the token of the corporate identity provider trusted by your SAP Cloud Identity Services tenant. • **`"ias-token"`** includes the token of the trusted SAP Cloud Identity Services tenant.	`["groups","rolecollections"]`

Key	Description	Example
`allowedproviders`	Includes an array of allowed identity providers. The default is that all identity providers are allowed. As an option, developers can configure on application level which identity providers can be used for business users to log on to a certain application. The value of `allowedproviders` is the origin key of your identity provider. You find the origin key in your subaccount in the *Security* > *Trust Configuration* page of your identity providers. → Recommendation We recommend that you configure allowed identity providers on subaccount level. For more information, see Using Multiple Identity Providers from the Same Subaccount [page 2556].	`["origin_key1","origin_key2"]`

xsenableasyncservice (Custom Option)

The `xsenableasyncservice` element controls whether the **`cf create-service`** and **`cf update-service`** commands of the Cloud Foundry command line interface (cf CLI) are executed synchronously (false) or asynchronously (true).

→ Remember

The Cloud Controller of Cloud Foundry times out after 60 seconds for the synchronous execution of the commands. For this reason, we recommend that you use asynchronous execution for multitenant applications or reuse services.

ⓘ Note

You usually need to wait for the completion of **`cf create-service`** and **`cf update-service`** even if you use asynchronous mode. Otherwise you might run into conflicts due to parallel update operations on the OAuth 2.0 clients.

{·} Sample Code

```
"xsenableasyncservice":"true"
```

Value	Description
`false`	Cloud Foundry command line interface commands are executed synchronously.
`true` (default)	Cloud Foundry command line interface commands are executed asynchronously.

Referencing the Application

If you want to grant scopes to an application for example, you must reference this application. Depending on where the application is located, you can reference an application in multiple ways:

- Referencing the application of the current `xs-security.json` file
- Referencing a foreign application that is located in the same subaccount

Application References

Description	Syntax
Application in the current `xs-security.json` file	`$XSAPPNAME`

Application in the same subaccount	`$XSAPPNAME(<service_plan>,<xsappname>)`
	Note
	Currently, you can only use the `application` service plan.
	Example
	`$XSAPPNAME(application,business-partner)`

Reference to any service instance in the same sub-account and space	`$XSSERVICENAME(<service_instance_name>)` This is the service instance name you used when you created it.

You can use these references with the following properties:

Properties

Property	Description	Example

`granted-apps`	Granting scopes to other applications.	Sample Code
		`"granted-apps" : [ "$XSAPPNAME(application,business-partner)"]`
		
		`"granted-apps" : [ "$XSAPPNAME(application,business-partner1)","$XSAPPNAME(application,business-partner2)"]`

`grant-as-authority-to-apps`	Use this property if you want to grant a scope to other applications for a client credential scenario.	Sample Code
		`"grant-as-authority-to-apps" : [ "$XSAPPNAME(application,business-partner)"]`
		
		`"grant-as-authority-to-apps" : [ "$XSAPPNAME(application,business-partner1)","$XSAPPNAME(application,business-partner2)"]`

`authorities`	Granting authorities (for a client credentials scenario)	Sample Code `"authorities": ["$XSAPPNAME(application,business-partner)"]`
`scope-references`	Referencing scopes in role templates	Sample Code `$XSAPPNAME.Display`

`foreign-scope-references`	Using this property, you can reference scopes in foreign applications (for a user scenario).	Sample Code
		`"foreign-scope-references": ["$XSAPPNAME(application,business-partner)"]`
		
		`"foreign-scope-references": ["$XSAPPNAME(application,business-partner1)","$XSAPPNAME(application,business-partner2)"]`

Relationship Between `default-values` of `attribute-references` and `valueRequired`

When you define role templates and attributes in the `xs-security.json` file, you need to know that there's a relationship between `default-values` of `attribute-references` in the role template and `valueRequired` in the attribute. What is important to know is that you configure `valueRequired` in the current attribute definition independently from any role template. The attributes are later referenced in the role templates using the `attribute-references` element.

This means that an attribute can be referenced in multiple role templates. You can set `default-values` for the first role template and not for the second one. If you define an attribute with `"valueRequired": true` (see rows 1 and 3), a default role can be generated automatically for the first role template only, but not for the second one.

Configuration Examples

Use Case	Role Template	Attribute
A default role with default values is generated automatically. Administrators must set attribute values for their roles.	`"default-values": ["<attribute_value>"]`	`"valueRequired" : true` or no `valueRequired` key at all
A default role with default values is generated automatically. Administrators don't have to set attribute values for their role. A role that isn't restricted by attributes (unrestricted) is created.	`"default-values": ["<attribute_value>"]`	`"valueRequired" : false`
A default role is generated automatically. Administrators don't have to set attribute values for their roles. A role that isn't restricted by attributes (unrestricted) is created.	No `default-values` element	`"valueRequired" : false`

⚠ Restriction

When you define role templates and attributes, observe the following configuration restriction:

- Do not add `attribute-references` without `default-values`, where `attribute` is defined with `"valueRequired" : true`.

When you update role templates and attributes, observe the following configuration restrictions:

- Do not change `default-values` of `attribute-references`.
- Do not change the `"valueRequired"` property from `false` to `true`.

Related Information

Attributes [page 2628]

Tutorials for the SAP Authorization and Trust Management Service [page 574]
Security Considerations for the SAP Authorization and Trust Management Service [page 3591]
Configuration Options for the SAP Authorization and Trust Management Service [page 3596]

4.1.6.4.2 Client Libraries for SAP BTP Security Services

We provide client libraries to simplify your consumption of our services.

List of Client Libraries

Client Libraries for SAP BTP Security Services

Library	Description	Link
SAP BTP Security Services Node.js Integration Library	This module allows Node.js applications to authenticate users with JWT tokens issued by SAP BTP security services (SAP Cloud Identity Services and SAP Authorization and Trust Management Service). It also provides an API for fetching tokens from these services.	https://npmjs.com/package/@sap/xssec
@sap/ams-dev	This is the Node.Js runtime library used to perform authorization checks in applications which authenticate users via the SAP Cloud Identity Services.	https://www.npmjs.com/package/@sap/ams
SAP BTP Security Services Java Integration Libraries	This repository offers a comprehensive set of libraries designed to simplify the integration of SAP BTP security services (SAP Cloud Identity Services and SAP Authorization and Trust Management Service).	https://github.com/SAP/cloud-security-services-integration-library/tree/main
sap-xssec	Learn how to validate OAuth tokens using a Python library. Use this library to add authentication in your Python application. **Note** This library isn't part of an SAP BTP license. However, it belongs to a related open source project.	https://github.com/SAP/cloud-py-sec

4.1.7 Setting Up Database Artifacts

Learn how to set up database artifacts using SAP HANA.

SAP HANA is an innovative in-memory relational database management system that you can leverage when developing in the SAP BTP, Cloud Foundry environment. It minimizes data movements by making it possible for you to work with the data directly in memory.

With SAP HANA, you can leverage current hardware capabilities to increase performance and reduce costs, for example by building applications that integrate the business control logic and database layer.

> **Note**
>
> In the Cloud Foundry environment, the SAP HANA database supports up to 1,000 simultaneous connections per database.

We recommend using SAP HANA Cloud when following a multitarget application (MTA) approach.

To find out more about the tools, data modeling strategy, and development scenarios, see Developing SAP HANA in the Cloud Foundry Environment [page 612].

4.1.7.1 Developing SAP HANA in the Cloud Foundry Environment

Find here selected information for SAP HANA database development and references to more detailed sources.

This section gives you information about database development and its surrounding tasks especially if you want to explore the SAP BTP, Cloud Foundry environment. To get more into detail, we have references to other guides and the SAP HANA Cloud service documentation. The context we're looking at is multitarget application (MTA) development, whereby SAP HANA is the database module and you develop all artifacts in that module.

There are multiple scenarios when you start using SAP HANA Cloud in the SAP BTP, Cloud Foundry environment. For each of these scenarios, we give you a recommendation:

Scenario	Recommendation
You're starting a new project and want to leverage SAP HANA capabilities in the cloud.	We recommend using the SAP Cloud Application Programming Model and SAP HANA Cloud. See SAP HANA Cloud [page 613]
You're using SAP HANA service for SAP BTP and SAP HANA Platform 2.0 (XS advanced) on-premise.	Get to know the differences. Read about features that have been supported by other SAP HANA versions but aren't supported by SAP HANA Cloud: SAP HANA Cloud Compatibility Reference

Scenario	Recommendation
You're using SAP HANA Platform 2.0 (XS advanced) on-premise.	SAP HANA extended application services, advanced model, (XS advanced) as the runtime environment for SAP HANA Platform was built similar to the Cloud Foundry environment. It has been developed further and therefore contains features that aren't available in Cloud Foundry. Some features aren't supported from the SAP HANA versions in the cloud. So, there's always some effort, at least when you reach a decent amount of complexity, to make an XS advanced application run in the cloud. The new SAP HANA Cloud service made significant moves to reduce footprint and be cloud-ready. Therefore, previously deprecated features have been removed. Read about SAP HANA Cloud Compatibility and Migration Information: • High Level Feature Compatibility • Design-time Content Compatibility You can find another collection of these features and elements in the following SAP Note:2868742
You're using SAP HANA Platform 1.0 (XS classic) on-premise. This scenario also includes any usage of XS classic artifacts even if you use XS advanced (compatibility) or the Neo environment	This scenario is presumably a bigger project. The task isn't only to use a different version of SAP HANA but also to make architectural changes: from monolithic applications to microservice-based applications. To give you an overview on things you need to consider, have a look at our guide: Migrating from the Neo Environment to the Multi-Cloud Foundation for SAP BTP (Cloud Foundry and Kyma)

4.1.7.1.1 SAP HANA Cloud

This section is a quick introduction to the latest SAP HANA offering on SAP BTP, Cloud Foundry environment. In essence, you get to know the most important tools and information about data modeling. This content is meant to provide an overview. Please follow the links to the content you're interested in and start your learning journey.

Tools

There are two tools you need to know, SAP Business Application Studio and SAP HANA Cockpit.

- SAP Business Application Studio
  SAP Business Application Studio is the recommended tool to develop your SAP HANA database artifacts. You can use an SAP HANA Native Application dev space, which is preconfigured to support the creation of database artifacts.
  For SAP Cloud Application Programming Model projects, we recommend to create a Full Stack Cloud Application dev space and add the following extensions:
  - SAP HANA Calculation View Editor
  - SAP HANA Tools
  - SAP HANA Smart Data Integration Tools

With that configuration you have the editors and tools for the programming model and SAP HANA database artifacts.

- SAP HANA Cockpit
  The SAP HANA Cockpit is used to monitor and administer your SAP HANA instances, see Open the SAP HANA Cockpit
  To learn how to access the SAP HANA Cockpit, see Accessing SAP HANA Cockpit for SAP HANA Cloud

Data Modeling

To develop your data model, we recommend the SAP Cloud Application Programming Model.

- The SAP Cloud Application Programming Model (CAP) provides a guide explaining the concepts and best practices of domain modeling using CAP CDS, see Domain Modeling
- To define your data models, queries and expressions, there's a CAP CDS reference guide. It contains samples and best practices that support your learning journey. See Core Data Services (CDS) - Language Reference Documentation for the application programming model.
- You have existing database objects and want to use them with CAP CDS. Learn about facade entities in CAP CDS and how to design them. See Native SAP HANA.

When you've created your data model with the SAP Cloud Application Programming Model, you want to leverage more of the SAP HANA power. The SAP HANA Cloud documentation provides an extensive library of guides. Find here selected links that make an easy start.

- SAP HANA Cloud Modeling Guide for SAP Web IDE Full-Stack
- SAP HANA Cloud product page, development overview

References

- SAP HANA Cloud product page on the Help Portal
- Feature Scope Description for SAP HANA Cloud

4.1.8 Deploying to the Cloud Foundry Environment

Get an overview of available deployment options.

When deploying applications and services to the cloud, you can choose from a number of different approaches and tools, depending on your development setup.

Multitarget Applications

If you've decided to develop your application or service with the multitarget approach, you can deploy it to SAP BTP using a single `.mtar` archive file. For more information, see Multitarget Applications in the Cloud Foundry Environment [page 615].

Continuous Integration and Delivery

Continuous Integration and Delivery (CI/CD) are two interrelated concepts that can enhance your development process by adding frequent integration, automated testing, and faster delivery. For more information, see Continuous Integration and Delivery [page 842] .

Cloud Foundry Command Line Interface

If you prefer to develop on your local machine, you can deploy your application from the folder on your computer directly to SAP BTP using the Cloud Foundry Command Line Interface (CF CLI). For more information, see Deploy Business Applications in the Cloud Foundry Environment [page 845].

SAP BTP Cockpit

You can also deploy your application with or without a `manifest.yml` file using the SAP BTP cockpit. For more information, see Deploy an Application [page 2862].

Docker Images

If you want a higher degree of freedom when realizing your project, you can also deploy your applications and services using Docker images and the Cloud Foundry Command Line Interface (CF CLI). Note that Docker images can't be used in combination with buildpacks. For more information, see Deploy Docker Images in the Cloud Foundry Environment [page 846].

4.1.8.1 Multitarget Applications in the Cloud Foundry Environment

A multitarget application (MTA) is essentially a single application that consists of multiple parts. These parts are created using various technologies and share the same lifecycle.

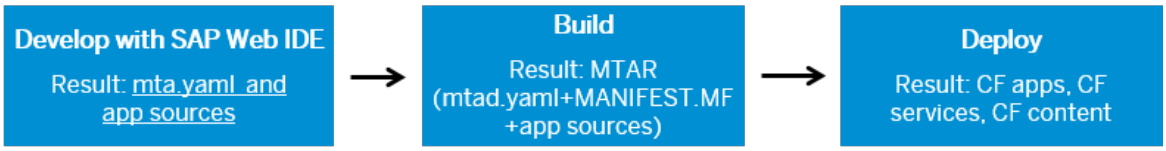
The MTA developers outline the intended outcomes using the MTA model. This model consists of MTA modules, MTA resources, and the interdependencies between them. Afterward, the SAP Cloud Deployment service validates it, orchestrates the necessary actions, and automates the MTA deployment. The outcome is the formation of Cloud Foundry applications, services, and the generation of SAP-specific content. For more information about the Multitarget Application model, see the official specification documents The Multitarget Application Model v.2 and The Multitarget Application Model v.3.

You can create and deploy a multitarget application in the Cloud Foundry environment as described below by following different approaches that can yield the same result:

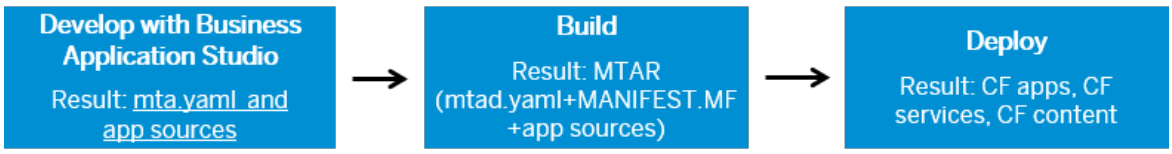
- Using SAP Web IDE Full-Stack as described in Developing Multitarget Applications - both the development descriptor `mta.yaml` and the deployment descriptor `mtad.yaml` are created automatically. The `mta.yaml` is generated when you create the application project, and the `mtad.yaml` file is created when you build the project.

  > ### Note
  >
  > You may still need to edit the development descriptor.

  Development descriptors are used to generate MTA deployment descriptors, which define the required deployment data. That is, the MTA development descriptor data specifies what you want to build, how to build it, while the deployment descriptor data specifies as what and how to deploy it.

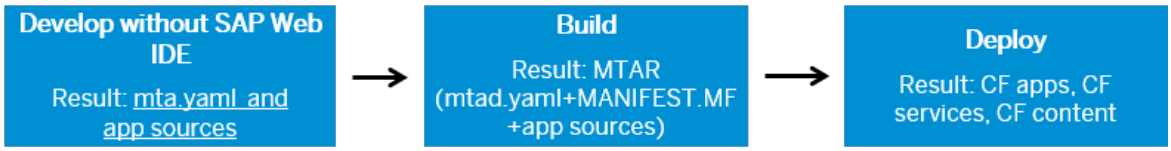
  

  - https://help.sap.com/viewer/825270ffffe74d9f988a0f0066ad59f0/CF/en-US/a71bf8281254489ea8be6e323199b304.html [https://help.sap.com/viewer/825270ffffe74d9f988a0f0066ad59f0/CF/en-US/a71bf8281254489ea8be6e323199b304.html]
  - https://help.sap.com/viewer/825270ffffe74d9f988a0f0066ad59f0/CF/en-US/3b533e3723674fad90f94510b92f10af.html [https://help.sap.com/viewer/825270ffffe74d9f988a0f0066ad59f0/CF/en-US/3b533e3723674fad90f94510b92f10af.html]
  - https://help.sap.com/viewer/825270ffffe74d9f988a0f0066ad59f0/CF/en-US/1b0a7a0938944c7fac978d4b8e23a63f.html [https://help.sap.com/viewer/825270ffffe74d9f988a0f0066ad59f0/CF/en-US/1b0a7a0938944c7fac978d4b8e23a63f.html]
- Using Business Application Studio - the flow is similar to WebIDE but the development is done from BAS.

  - https://help.sap.com/docs/bas/sap-business-application-studio/mta-development/ [https://help.sap.com/docs/bas/sap-business-application-studio/mta-development/]
  - https://help.sap.com/docs/bas/sap-business-application-studio/building-and-deploying-multitarget-applications [https://help.sap.com/docs/bas/sap-business-application-studio/building-and-deploying-multitarget-applications]
  - https://help.sap.com/docs/bas/sap-business-application-studio/building-and-deploying-multitarget-applications [https://help.sap.com/docs/bas/sap-business-application-studio/building-and-deploying-multitarget-applications]
- Using the Cloud MTA Build Tool . Afterward, you deploy the MTA using the Cloud Foundry Command Line Interface.

  > ### Note
  >
  > An MTA development descriptor `mta.yaml` is required. You have to create it manually.



- https://sap.github.io/cloud-mta-build-tool/ [https://sap.github.io/cloud-mta-build-tool/]
- https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/65ddb1b51a0642148c6b468a759a8a2e.html [https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/65ddb1b51a0642148c6b468a759a8a2e.html]
- https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/c2d31e70a86440a19e47ead0cb349fdb.html [https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/c2d31e70a86440a19e47ead0cb349fdb.html]

- Manually - create the required files manually and deploy them using the Cloud Foundry Command Line Interface

> **Note**
>
> An MTA development descriptor `mta.yaml` is not explicitly required. In this case, a deployment descriptor is maintained instead.

1. Create the required files using an IDE of your choice.
2. To produce your custom artifacts, use a build technology of your choice.
3. Maintain your deployment descriptor document (`mtad.yaml`).
    1. (Optional) If you want to have an MTA archive package, use the **`mbt assemble`**. For more information, see Cloud MTA Build Tool (MBT) .
    2. Use the `cf deploy` command to deploy the MTA package or directly from the build result directory.

To learn more about	See
Multitarget Application deployment descriptor	Defining Multitarget Application Deployment Descriptors for Cloud Foundry [page 623]
Multitarget Application archive	Defining Multitarget Application Archives [page 622]
Multitarget Application extension descriptor	Defining MTA Extension Descriptors [page 745]
Multitarget Application structure	Multitarget Application Structure [page 629]
How to deploy the Multitarget Application	Multitarget Application Plugin for Cloud Foundry Command Line Interface [page 2832]

Terms and Concepts

Terms and Concepts

Term	Description
Multitarget application (MTA)	An application comprised of multiple software modules, which are created with different technologies and deployed to different runtimes.

Term	Description
Development descriptor	A `YAML` file named `mta.yaml` that contains a list of all entities, such as modules, resources, and properties that belong to an application or are used by it at runtime, and the dependencies between them. It is automatically generated when an MTA project is created or modified, or when a module is added or removed. The developer needs to edit the descriptor manually to define resources, properties, and dependencies, as well as fill in missing information.
Deployment descriptor	A `YAML` file named `mtad.yaml` that contains a list of all entities which is created from either SAP Web IDE Full-Stack, SAP Business Application Studio, Cloud MTA Build Tool or manually. This file is similar to Development Descriptor but is used from the SAP Cloud Deployment service.
Module	A self-contained application of a certain type, which is developed, packaged and deployed.
Module type	A type that defines the structure and the development technology of a module. You can see a list of the module types at Modules [page 643].
Resource	Any resource, such as an external service that is required by a module at runtime but not provided by the module itself.
Property	A property (key-value pair) of an application, module, or resource, that is used during deployment or at runtime.
Parameter	A reserved variable belonging to a module or resource, whose value is used during deployment or at runtime.
Dependency	A relationship between a module and another module, resource, or property, such as provides and requires. • `provides`: indicates the properties or parameters that are provided by a module or resource to other modules. • `requires`: indicates other modules or resources that are required by a module in order to run.
MTA archive (MTAR)	Archive containing a deployment descriptor, the module and resource binaries, and configuration files. The archive follows the JAR file specification.

Prerequisites and Restrictions

Prerequisites

You need the **Space Developer** role to deploy multitarget applications in the Cloud Foundry environment. For more information, see About Roles in the Cloud Foundry Environment [page 2788].

Restrictions

You have to consider the following limits for the MTA artifacts, which can be handled by the SAP Cloud Deployment service:

- Maximum size of the MTA archive: 4 GB
- Maximum size of MTA module content: 1 GB
- Maximum size of MTA resource content: 1 MB
- Maximum size of MTA descriptors (`mtad.yaml` and `MANIFEST.MF`): 1 MB
- Limitations for transport scenarios via SAP Cloud Transport Management, where one transport import with multiple transport requests results into a single deployment process in SAP Cloud Deployment service with multiple MTAs:
  - The total size of all MTA archives in a single import request cannot exceed 20GB.
  - The maximum number of MTA archives used in a single import request cannot exceed 100.
- Limitations for transport scenarios via the Enhanced Change and Transport System (CTS+), where one transport import with multiple transport requests results into a single deployment process in SAP Cloud Deployment service with multiple MTAs:
  - The total size of all MTA archives in multiple transport requests per one import cannot exceed 4GB.
  - The maximum number of MTA archives used in a single import request cannot exceed 500.

Unsupported Cloud Foundry Features

In addition to the features that are not supported by the Cloud Foundry environment on SAP BTP (See Supported and Unsupported Cloud Foundry Features), MTA deployment in Cloud Foundry currently does not support these features as well:

- Sidecars. See https://docs.cloudfoundry.org/devguide/sidecars.html .
- Route services. See https://docs.cloudfoundry.org/services/route-services.html .
- Metadata. See https://docs.cloudfoundry.org/adminguide/metadata.html .
- Per-route options. See https://docs.cloudfoundry.org/devguide/custom-per-route-options.html .

> **Note**
>
> These features might be implemented in the future. For the latest information, follow the release notes regarding Multitarget Applications for Cloud Foundry in What's New for SAP Business Technology Platform.

Related Information

JAR File Specification
Cloud MTA Build Tool
Release Notes for Multitarget Applications for Cloud Foundry

4.1.8.1.1 Benefits of the Multitarget Application Model for the Cloud Foundry Environment

The benefits of the multitarget application (MTA) approach can be related to the three “A”s that the MTA offers - Abstraction, Automation, and Assembly.

- Abstraction - Simply put, the MTA model uses abstraction to handle the diversity and complexity of different parts of an application, which may increase over time. This method offers a standardized way of modeling dependencies and environment-specific configurations. Without the MTA model you would have to individually handle all this heterogeneity, using custom scripts in a CI/CD tool to manage all underlying components, service dependencies and configurations. Achieving the needed automation through such custom scripts may be labor-intensive. It also demands higher effort as the technologies and services used by the application change over time. Modeling your application as an MTA highly simplifies the lifecycle of such heterogeneous applications.
- Automation - By defining your application's structure and dependencies (the “What”), you can use the SAP Cloud Deployment Service to manage automated tasks like deployment, service creation, updates and binding (the “How”). This reduces the work required in CI/CD pipelines to oversee all these deployment-related tasks.
- Assembly - By offering a standard packaging format for your applications, the MTA serves as a release artifact. This is known as an assembly. The assembly, constituting the application and all its metadata, can be useful in different contexts. One such context is audited environments, where a signed artifact is mandatory. Another use case is moving deployable artifacts through a firewall. The assembly can also be valuable in situations where different policies are applicable based on the deployment target. Furthermore, this artifact can be distributed to other consumers. For instance, a partner distributing a business application to several customers may find this useful. These customers can then deploy this application in their own accounts or tenants.

Deciding on the Right Size for a Multitarget Application

Your business application may grow over time. When this happens, you should decide on the volume of data for your MTA or multiple MTAs by analyzing which pieces make sense to be managed together as a separate lifecycle management unit.

A popular example is the case for a “core” or “framework” application parts that could work on their own, offering the basic business capabilities. Then there might be “extension” or “plugin” applications that extend the basic business capabilities, but are not mandatory and could be released and updated separately.

Another possibility is to model just one single Cloud Foundry application in a separate MTA. This makes sense if the application can be considered a self-contained microservice. However, even one Cloud Foundry app in one MTA modeling bears the benefits of dependency management (for example, backing service creation and external API lookup), content management (Fiori Launchpad configurations, workflow definitions, and so on) and configuration management (utilizing system placeholders, default and deployment specific configuration).

Other Benefits

- Parallel deployment - This feature enables asynchronous deployment of multiple applications, this way the deployment of the archive is faster.
- Asynchronous service instance creation - The MTA deployer creates services asynchronously, this way decreasing the deployment time.
- Parallel undeployment - This feature enables asynchronous undeployment of applications, this way decreasing the undeployment time.

4.1.8.1.2 Getting Started

To learn how to deploy a simple multitarget application (MTA) to SAP BTP, Cloud Foundry using the MultiApps CF CLI Plugin, follow the steps in the tutorial Deploy Your First Multitarget Application.

If you want to see more MTA examples, see CF MTA Examples on GitHub.

4.1.8.1.3 Defining Multitarget Application Development Descriptors

Multitarget applications are defined in a development descriptor required for design-time purposes.

The development descriptor (`mta.yaml`) defines the elements and dependencies of a Multitarget application (MTA) compliant with the Cloud Foundry environment.

> **Note**
>
> The MTA development descriptor (`mta.yaml`) is used to generate the deployment descriptor (`mtad.yaml`), which is required for deploying an MTA to the target runtime. If you use command-line tools to deploy an MTA, you do not need an `mta.yaml` file. However, in these cases you have to manually create the deployment descriptor (`mtad.yaml`) file.

For more information about the MTA deployment descriptor, see Defining Multitarget Application Deployment Descriptors for Cloud Foundry.

Related Information

Developing Multitarget Applications

4.1.8.1.4 Defining Multitarget Application Archives

You package the MTA deployment descriptor and module binaries in an MTA archive. You can manually do so as described below, or alternatively use the Cloud MTA Build Tool .

The multitarget application (MTA) archive is created in a way compatible with the JAR file specification. This allows us to use common tools for creating, modifying, and signing such types of archives. According to the JAR specification, there must be an empty line at the end of the file.

An MTA archive consists of the following:

- The `MANIFEST.MF` file
- The `mtad.yaml` MTA deployment descriptor file
- Application binaries, content, or configuration files. You can store one application in two or more application binaries contained in the MTA archive.

> **Note**
>
> The MTA extension descriptor is not part of the MTA archive. During deployment, you provide it as a separate file, or as parameters that you enter manually when SAP BTP requests them.

> **Caution**
>
> The maximum size of an MTA archive is limited to 500 MB. Deployment is denied for archives with larger size.

The following example shows the basic structure of an MTA archive. It contains a Java application `.war` file and a `META-INF` directory, which contains an MTA deployment descriptor with a module and a `MANIFEST.MF` file.

> **Example**

```
/example.war
/META-INF
/META-INF/mtad.yaml
/META-INF/MANIFEST.MF
```

The `MANIFEST.MF` file has to contain a name section for each MTA module part of the archive that has a file content. In the name section, the following information has to be added:

- `Name` - the path within the MTA archive, where the corresponding module is located. If it leads to a directory, add a forward slash (`/`) at the end.
- `Content-Type` - the type of the file that is used to deploy the corresponding module
- `MTA-module` - the name of the module as it has been defined in the deployment descriptor
- > **Note**
  >
  > There could be more than one module of the same type in an MTA archive.

Sample content of the `META-INF/MANIFEST.MF` file:

> **Example**

```
Manifest-Version: 1.0
Created-By: example.com
```

```
Name: example.war
Content-Type: application/zip
MTA-Module: example-java-app
```

The example above instructs SAP BTP to:

- Look for the `example.war` file within the root of the MTA archive when working with module `example-java-app`
- Interpret the content of the `example.war` file as an `application/zip`

> **Note**
>
> The example above is incomplete. To deploy a solution, you have to create an MTA deployment descriptor. Then you have to create the MTA archive.

Related Information

Cloud MTA Build Tool
The Multitarget Application Model v.2
The Multitarget Application Model v.3
JAR File Specification
Defining MTA Extension Descriptors [page 745]

4.1.8.1.5 Defining Multitarget Application Deployment Descriptors for Cloud Foundry

The Multitarget Application (MTA) deployment descriptor is a YAML file that defines the relations between you as a provider of deployable artifacts and the SAP Cloud Deployment service in SAP BTP as a deployer tool.

Using the YAML data serialization language you describe the MTA in an MTA deployment descriptor (`mtad.yaml`) file following the Multitarget Application Structure [page 629].

> **Note**
>
> As there are technical similarities between SAP HANA XS Advanced and Cloud Foundry, you can adapt application parameter for operation in either platform. Note that each environment supports its own set of module types, resource types, and configuration parameters. For more information, see the official specification documents The Multitarget Application Model v.2 and The Multitarget Application Model v.3.

Schema Store

Writing `YAML` descriptors in plain text is often hard, so we have contributed a schema to the public Schema Store. This would provide you with an almost out-of-the-box support when writing MTA deployment descriptors

in some of the more popular IDEs. The schema would provide you with auto-completion as well as suggestions and syntax checking.

For more information, see Editor Schema Support .

4.1.8.1.5.1 MTA Deployment Descriptor Examples

Examples of MTA deployment descriptors.

> **Note**
>
> The format and available options in the MTA deployment descriptor could change with the newer versions of the MTA specification. Always specify the schema version when defining an MTA deployment descriptor, so that SAP BTP is aware against which MTA specification version you are deploying.

Example 1

Deployment descriptor with a resource (managed service)

> **Example**

```
_schema-version: "3.1.0"
ID: simple-mta
version: 1.0.0
modules:
- name: anatz
  type: javascript.nodejs
  requires:
    - name: hdi-service
resources:
- name: hdi-service
  type: org.cloudfoundry.managed-service
  parameters:
    service: hana
    service-plan: hdi-shared
```

Example 2

Deployment descriptor with the `keep-existing` parameter (will keep the existing: environment, service-bindings and routes)

> **Example**

```
_schema-version: 3.1.0
ID: anatz-keep-existing
version: 4.0.0
modules:
```

```
- name: anatz
  type: staticfile
  path: hello-world.zip
  parameters:
    memory: 64M
    route: new-custom-route-${space}
    keep-existing:
      env: true
      service-bindings: true
      routes: true
```

Example 3

Deployment descriptor with enabled parallel deployment of modules (will deploy modules in parallel)

Example

```
_schema-version: 3.1.0
ID: hello-world
version: 1.0.0
parameters:
  enable-parallel-deployments: true
modules:
- name: hello-world
  type: staticfile
  path: content/hello_world.zip
- name: hello-world-first
  type: staticfile
  path: content/hello_world.zip

- name: hello-world-second
  type: staticfile
  path: content/hello_world.zip
- name: hello-world-third
  type: staticfile
  path: content/hello_world.zip
```

Example 4

Deployment descriptor with docker image module

Example

```
_schema-version: "3.1.0"
ID: docker-mtar
version: 2.0.1
modules:
- name: docker-image
  type: application
  parameters:
    docker:
      image: cloudfoundry/test-app
```

Example 5

Deployment descriptor with optional resource

Example

```
_schema-version: "3.1.0"
ID: ztana1
version: 1.1.0
modules:
  - name: ztana
    type: javascript.nodejs
    requires:
      - name: test-resource

resources:
  - name: test-resource
    type: org.cloudfoundry.existing-service
    optional: true
    parameters:
      service: non-required-service
```

Example 6

A basic MTA deployment descriptor that is defined in an `mtad.yaml` file:

Example

```
_schema-version: "3.1"
ID: com.sap.xs2.samples.javahelloworld
version: 0.1.0


modules:
  - name: java-hello-world
    type: javascript.nodejs
    path: web/
    requires:
      - name: java-uaa
      - name: java
        group: destinations
        properties:
          name: java
          url: ~{url}
          forwardAuthToken: true
  - name: java-hello-world-backend
    type: java.tomee
    path: java/target/java-hello-world.war
    provides:
      - name: java
        properties:
```

```
          url: ${default-url}
    properties:
      JBP_CONFIG_RESOURCE_CONFIGURATION: "['tomee/webapps/ROOT/WEB-INF/
resources.xml': {'service_name_for_DefaultDB' : 'java-hdi-container'}]"
    requires:
      - name: java-uaa
      - name: java-hdi-container
      - name: java-hello-world-db
  - name: java-hello-world-db
    type: com.sap.xs.hdi
    path: db/
    requires:
      - name: java-hdi-container

resources:
  - name: java-hdi-container
    type: com.sap.xs.hdi-container

  - name: java-uaa
    type: com.sap.xs.uaa-space
    parameters:
      config-path: xs-security.json
```

Note

The example above is incomplete. To deploy a solution, you have to create an MTA extension descriptor with the user and password added there. You also have to create the MTA archive.

Example 7

Another basic example of an MTA deployment descriptor.

Sample Code

```
_schema-version: "3.1"
ID: com.sap.xs2.samples.nodehelloworld
version: 0.1.0
modules:
  - name: node-hello-world
    type: javascript.nodejs
    path: web/
    requires:
      name: nodejs-uaa
      - name: nodejs
        group: destinations
        properties:
          name: backend
          url: ~{url}
          forwardAuthToken: true
        properties-metadata:
          name:
           optional: false
           overwritable: false
          url:
           overwritable: false
    parameters:
      host: !sensitive ${user}-node-hello-world
      memory: 128MB
    parameters-metadata:
      memory:
```

```
        optional: true
        overwritable: true
  - name: node-hello-world-backend
    type: javascript.nodejs
    path: js/
    provides:
      - name: nodejs
        properties:
          url: "${default-url}"
    requires:
      - name: nodejs-uaa
      - name: nodejs-hdi-container
      - name: node-hello-world-db
    parameters:
      host: ${user}-node-hello-world-backend
  - name: node-hello-world-db
    type: com.sap.xs.hdi
    path: db/
    requires:
      - name: nodejs-hdi-container
    parameters:
      tasks:
        - name: task-1                    #You can model Cloud Foundry tasks
as described here. For additional information, check the Cloud Foundry
"Information for Developers" document.
          command: node deploy.js
          disk-quota: 1GB
          memory: 1GB
resources:
  - name: nodejs-hdi-container
    type: com.sap.xs.hdi-container
    parameters:
      config:
        schema: ${default-container-name}

  - name: nodejs-uaa
    type: com.sap.xs.uaa
    parameters:
      config-path: xs-security.json
  - name: log
    type: application-logs
    optional: true
```

Example 8

A more complex example, which shows an MTA deployment description with the following modules:

- A database model
- An SAP UI5 application (hello world)
- An application written in node.js

The UI5 application “hello-world” uses the environment variable `<ui5_library>` as a logical reference to some version of UI5 on a public Website.

Sample Code

```
ID: com.acme.xs2.samples.javahelloworld
version: 0.1.0
modules:
  - name: hello-world
```

```
    type: javascript.nodejs
    requires:
      - name: uaa
      - name: java_details
        properties:
          backend_url: ~{url3}/
    properties:
      ui5_library: "https://sapui5.hana.acme.com/"

  - name: java-hello-world-backend
    type: java.tomee
    requires:
      - name: uaa
      - name: java-hello-world-db        # deploy ordering
      - name: java-hdi-container
    provides:
      - name: java_details
        properties:
          url3: ${url}                   # the value of the place-holder $
{url}
                                         # will be made known to the deployer
  - name: java-hello-world-db
    type: com.sap.xs.hdi
    requires:
      - name: java-hdi-container
resources:
  - name: java-hdi-container
    type: com.sap.xs.hdi-container

  - name: java-uaa
    type: com.sap.xs.uaa
    parameters:
      name: java-uaa                          # the name of the actual service
```

Related Information

Defining Multitarget Application Deployment Descriptors for Cloud Foundry [page 623]
The Multitarget Application Model v.2
The Multitarget Application Model v.3

4.1.8.1.6 Multitarget Application Structure

The following chapter contains information about:

- Global elements - an identifier and version that uniquely identify the MTA, including additional optional information such as a description, the providing organization, and a copyright notice for the author.
- Modules [page 643] - the deployable parts contained in the MTA deployment archive, most commonly Cloud Foundry applications or content.
- Resources [page 630] - they contain properties of resource types, which are entities not part of an MTA, but required by the modules at runtime or at deployment time.
- Dependencies between modules and resources.
- Parameters - variables which belong to a module or a resource, whose value is used during the deployment or at runtime. For more information, see Parameters and Properties [page 675].

- Properties - these result in the application environment variables that have to be available to the respective module at runtime. For more information, see Parameters and Properties [page 675].
- Technical configuration parameters, such as URLs, and application configuration parameters such as environment variables. For more information, see Parameters and Properties [page 675].
- Metadata - provide additional information about the declared parameters and properties. For more information, see Metadata for Properties and Parameters [page 744].
- Module Hooks [page 668] - use hooks to change the typical deployment process, for example to set them to be executed before or after the actual deployment steps for a module.
- If needed, use alternative arrangement of the properties' syntax. For more information see Parameters and Properties [page 675].

4.1.8.1.6.1 Resources

The application modules defined in the “modules” section of the deployment descriptor may depend on **resources**.

The resources may be used as:

- platform services managed by the deployer or only used by the applications
- configuration entries used by the applications and services

In the `resources` section, the following elements are mandatory:

- `name` - Must be unique within the MTA it identifies

Optional resource attributes include:

- `type` - the resource **type** is one of a reserved list of resource types supported by the MTA-aware deployment tools, for example: `com.sap.xs.uaa`, `com.sap.xs.hdi-container`, `com.sap.xs.job-scheduler`; the **type** indicates to the deployer how to discover, allocate, or provision the resource, for example, a managed service such as a database, or a user-provided service. When `type` is not defined, `resource` is used for configurations only for other modules and resources.
- `description` - non-translatable, free-text string; the string is not meant to be presented on application user interfaces (UI)
- `properties` - a structured set of name-value pairs
- `parameters` - reserved variables that affect the behavior of the MTA-aware tools, such as the deployer
- `active` - its value can be `true` or `false` and the default value is `true`. If set to `false`, the resource is not processed and it is ignored in the `requires` list of the application that requires it.
- `processed-after` - the attribute is used to create an order, in which resources should be processed. If a resource has this attribute, it will be processed after the other resources in a higher position are processed. The attribute value is a structured set of a list comprised of other resource names of the same MTA.

  > → Tip
  >
  > By default recourses process in parallel, but if you want to enable an order of resources, see Sequential Resource Processing [page 805].

- `optional`- its value can be `true` or `false` and the default value is `false`. If set to `true`, the resource processing is fail-safe.

Active and Inactive Resources

If you deploy an application with a resource type attribute `active` set to `false`, the resource is not provisioned and no binding is created. If you have already deployed the application with the resource type attribute `active` set to `true`, the binding to the resource is removed.

Deployment with managed-service, existing-service and user-provided service

No binding is created and in case of `managed-service`, no service is created.

Deployment with org.cloudfoundry.existing-service-key

No binding to an application environment will be done in the following cases:

- If the `org.cloudfoundry.existing-service-key` resource itself is set to `active: false`
- If the `org.cloudfoundry.existing-service-key` resource is set to `active: true`, but refers to a resource, which is set to `active: false`

Deployment with configuration resources (cross MTA dependencies)

If the configuration resource is set to `active: false`, then:

- If the requires section of the module type expects a list, then the environment variable assigned to this list is empty and subscriptions between the modules is not created
- If the requires section does not expect a list, then no environment variable is created

See section “Optional resources” below for more information.

> ⚠ Restriction
>
> System-specific parameters for the deployment descriptor must be included in a so-called MTA deployment extension descriptor.

Optional Resources

To describe resources that are not mission-critical for the operation of your Multitarget Application, proceed as described below.

> ⓘ Note
>
> This option is available with schema version `3.1`.

You can declare some resources as optional, which mitigates the cases when they are not listed, not available, or have failed to be created or updated. This means that when the deployer cannot allocate the required resource due to any of these reasons, it generates a warning and continues processing. Alternatively, if a resource is not declared as optional, the deployer generates an error and stops processing.

The following excerpt is a code example for the `MANIFEST.MF` file.

> {·} Sample Code

```
...
resources:
```

```
...
- name: log
  type: com.sap.xs.auditlog
  optional: true
...
```

In the above example:

- If the `log` managed resource is not provided by the platform or landscape, a warning is logged and traced and the MTA deployment continues while ignoring the error.
- The available values for the `optional` parameter are `true` and `false`, with the latter being the default.

Generic MTA Resource Types

- `org.cloudfoundry.managed-service`
  In cases you have to choose a managed service and/or service plan that is not listed in Predefined MTA Resource Types [page 634], you define it using the `org.cloudfoundry.managed-service` resource type with the following parameters:
  - (Required) `service` - Name of the service to create.
  - (Optional) `service-name` - Service instance name. Default value is the resource name.

    > **Note**
    >
    > Service names that do not comply with the Cloud Foundry limitation of 50 symbols are automatically corrected. In such cases, the name is shortened and its end is replaced with a hash code.

  - (Required) `service-plan` - Name of the service plan.

  For example:

  Sample Code

  ```
  resources:
    - name: my-postgre-service
      type: org.cloudfoundry.managed-service
      parameters:
        service: postgresql
        service-plan: v9.6-dev
  ```

  > **Note**
  >
  > To choose a different service plan for a predefined MTA resource type, for example to change the service plan for `PostgreSQL` service, you define it using:
  >
  > **Sample Code**
  >
  > ```
  > resources:
  >   - name: my-postgre-service
  >     type: org.postgresql
  >     parameters:
  >       service-plan: v9.6-dev
  > ```

Updating managed service instances is fail-safe by default in certain scenarios, specifically when the service broker is involved. This behavior exists for legacy compatibility with service brokers that did not support plan updates in the Cloud Controller v2 API. If this fail-safe behavior is not desired, you can control it by using the `fail-on-service-update` parameter. For more information, see Resource-Specific Parameters [page 637], Updating Service Plans [page 755], Updating Service Instance Parameters [page 758], Updating Service Tags [page 761].

> ### Note
>
> The `optional` parameter takes precedence over the `fail-on-service-update` parameter. If `optional: true` is specified, it overrides the behavior of the `fail-on-service-update` setting.

- `org.cloudfoundry.existing-service`
  To indicate that the (named) service exists, without managing its lifecycle, you define the service name by using the `org.cloudfoundry.existing-service` resource type with the following parameters:
  - (Optional) `service-name` - Service instance name. Default value is the resource name.
- `org.cloudfoundry.existing-service-key`
  Existing service keys can be modeled as a resource of type `org.cloudfoundry.existing-service-key`, which checks and uses their credentials. For more information, see Service Keys [page 763].
- `org.cloudfoundry.user-provided-service`
  Create or update a user-provided service configured with the following resource parameters:
  - (Optional) `service-name` - Name of the service to create. Default value is the resource name.

    > ### Note
    >
    > Service names that do not comply with the Cloud Foundry limitation of 50 symbols are automatically corrected. In such cases, the name is shortened and its end is replaced with a hash code.

  - (Required) `config` - Map value, containing the service creation configuration, for example url and user credentials (user and password)

  > ### Example
  >
  > #### Sample Code
  >
  > ```
  > resources:
  >   - name: my-destination-service
  >     type: org.cloudfoundry.user-provided-service
  >     parameters:
  >       config:
  >         <credential1>: <value1>
  >         <credential2>: <value2>
  > ```

- `org.cloudfoundry.cloud-logging-service`
  Export the operation logs of your multitarget application to an SAP Cloud Logging instance by defining a resource of type `org.cloudfoundry.cloud-logging-service` with the following parameters:
  - (Optional) `service-name` - Name of the SAP Cloud Logging service instance. Default value is the resource name.
  - (Optional) `service-key-name` - Name of the service key that holds the ingestion credentials. Default value is the resource name.
  - (Optional) `log-level` - Verbosity of the exported operation logs. Supported values are INFO, WARN, DEBUG, ERROR, and TRACE. Default value is INFO.

- (Optional) `destination` - Map value specifying the `org-name` and `space-name` of the organization and space in which the SAP Cloud Logging instance resides. By default, the organization and space of the current deployment are used.

Example

Sample Code

```
resources:
  - name: my-cloud-logging
    type: org.cloudfoundry.cloud-logging-service
    parameters:
      service-name: my-cloud-logging-instance
      service-key-name: my-service-key
      log-level: INFO
```

For more information, see (Experimental) Exporting MTA Operation Logs to SAP Cloud Logging [page 823].

- `configuration`
  For more information, see Cross-MTA Dependencies [page 806].

Predefined MTA Resource Types

Modify the default MTA resource types by providing specific properties or parameters in the MTA deployment descriptor.

Predefined MTA Resource Types and Mapped Services

Resource Type	Service	Service Plan	Created Service
`com.sap.xs.hana-schema`	`hana`	`schema`	Plain schema

Example

Sample Code

```
name: hcsp-sch
type:
com.sap.xs.hana-
schema
parameters:
  service-name:
HCSP_SCH
  config:
    database_id:
9bf1f23a-123c-456e
-b789-ff12ea34a5dc
    schema:
HCSP_DB
```

Resource Type	Service	Service Plan	Created Service
`com.sap.xs.hana-securestore`	`hana`	`securestore`	SAP HANA secure store

`com.sap.xs.hdi-container`
 	
Note
When using a free trial subaccount, modify the default service:
Sample Code
`resources:`
`  - name: my-hdi-service`
`    type: com.sap.xs.hdi-container`
`    parameters:`
`      service: hanatrial`
	`hana`	`hdi-shared`	HDI container

`com.sap.xs.jobscheduler`	`jobscheduler`	`lite`	Job Scheduler
`com.sap.xs.uaa`	`xsuaa`	`applicaton`	Application UAA
`com.sap.xs.uaa-application`	`xsuaa`	`application`	Application UAA
`com.sap.xs.uaa-devuser`	`xsuaa`	`application`	Application UAA
`com.sap.xs.uaa-space`	`xsuaa`	`application`	Application UAA
`com.sap.xs.application-logs` **Restriction** This resource type is now deprecated. Use `application-logs` instead.	`application-logs`	`lite`	Streams logs of bound applications to a central application logging stack
`com.sap.portal.site-content` **Restriction** Use only with the SAP Node.js module `@sap/site-content-deployer`	`portal-services`	`site-content`	Portal services

Resource Type	Service	Service Plan	Created Service
`com.sap.portal.site-host` **⚠ Restriction** Only for use with the SAP Node.js module `@sap/site-entry`	`portal-services`	`site-host`	Portal services
`com.sap.xs.auditlog`	`auditlog`	`standard`	Audit log service
`auditlog`	`auditlog`	`standard`	Audit log service
`autoscaler`	`autoscaler`	`standard`	Automatically increase or decrease the number of application instances based on a policy you define.
`application-logs`	`application-logs`	`lite`	Streams logs of bound applications to a central application logging stack
`connectivity`	`connectivity`	`lite`	Establishes a secure and reliable connectivity between cloud applications and on-premise systems
`destination`	`destination`	`lite`	Provides a secure and a reliable access to destination configurations
`feature-flags`	`feature-flags`	`lite`	Feature Flags service for controlling feature rollout
`ml-foundation-services`	`ml-foundation-services`	`lite`	
`objectstore`	`objectstore`	`s3-standard`	Highly available and distributed consistent object store
`org.mongodb` **⚠ Restriction** This resource type is now deprecated. Use `mongodb` instead.	`mongodb`	`v3.0-container`	MongoDB document-oriented database system.
`mongodb`	`mongodb`	`v3.0-container`	MongoDB document-oriented database system

Resource Type	Service	Service Plan	Created Service
`org.postgresql` **⚠ Restriction** This resource type is now deprecated. Use `postgresql` instead.	`postgresql`	`v9.4-container`	PostgreSQL object-relational database system
`postgresql`	`postgresql`	`v9.4-container`	PostgreSQL object-relational database system
`com.rabbitmq` **⚠ Restriction** This resource type is now deprecated. Use `rabbitmq` instead.	`rabbitmq`	`v3.6-container`	RabbitMQ messaging
`rabbitmq`	`rabbitmq`	`v3.6-container`	RabbitMQ messaging
`io.redis` **⚠ Restriction** This resource type is now deprecated. Use `redis` instead.	`redis`	`v3.0-container`	Redis in-memory data structure store
`redis`	`redis`	`v3.0-container`	Redis in-memory data structure store

Resource-Specific Parameters

Resource parameters have platform-specific semantics. To reference a parameter value, use the placeholder notation `${<parameter>}`, for example, `${default-host}`.

> **→ Tip**
>
> It is also possible to declare metadata for parameters and properties defined in the MTA deployment descriptor; the mapping is based on the parameter or property keys. For example, you can specify if a parameter is **required** (`optional: false`) or can be modified (`overwritable: true`).

The following parameters are supported:

> **ⓘ Note**
>
> If you can't find a specific parameter from the native Cloud Foundry manifest here, refer to Prerequisites and Restrictions [page 618] to see which Cloud Foundry features are currently not supported.

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`apply-namespace`	Resource	Write	Applies namespace to the service name. If the namespace value is not provided in the CLI options, it is not applied. For more information, see Fine-Grained Configuration [page 798].		`resources:`
					`- name: java`
					`...`
					`parameters:`
					`apply-namespace: true`

`config`	Resource	Write	Defines service creation parameters. More information in Service Instance Parameters [page 756].	n/a	See Service Instance Parameters [page 756].
`default-container-name`	Resource	Read-Only	Default value for the `container-name` parameter that is used during HDI creation. It is based on the organization, space and resource name, which are combined in a way that conforms to the container-name restrictions for length and legal characters. All dash (-) symbols are replaced with an underscore (_). **Note** It is highly recommended to avoid using this parameter with HANA services because in some cases the generated name cannot comply with the HANA naming limitations' requirements. Forward compatability is not guaranteed.	Generated as described in the description.	`<org_name>_<space_name>_<resource_name>`
`default-service-name`	Resource	Read-Only	The name of the service in the Cloud Foundry environment to be created for this resource, based on the resource name with or without a name-space prefix.	The resource name with or without a name-space prefix	`nodejs-hdi-container` `com.sap.xs2.samples.xsjshelloworld.nodejs-hdi-container`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`default-xsappname`	Resource	Read-Only	Default value for the `xsappname` parameter that is used during UAA creation. It is based on the service name, which is modified in a way that conforms to the `xsappname` restrictions for length and legal characters.	Generated as described in the description.	`xs_-deploy-service-database` (if the service name is "`xs@-deploy-service-database`")

`destination`	Resource	Write	Map value specifying the organization and space in which the SAP Cloud Logging instance resides. Applies to a resource of type `org.cloudfoundry.cloud-logging-service`. Contains the `org-name` and `space-name` parameters. By default, the organization and space of the current deployment are used. For more information, see (Experimental) Exporting MTA Operation Logs to SAP Cloud Logging [page 823].	The organization and space of the current deployment	`destination:`
					`  org-name: my-org`
					`  space-name: my-space`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`fail-on-service-update`	Resource	Write	When this parameter is set to true, the deployment will fail on every service update failure. By default, service updates are fail-safe for asynchronous calls (Cloud Foundry API calls that result in a call to the service broker).
			
			When the parameter is set to false, deployments will not fail on service update failures, as the updates will be made fail-safe even for calls that do not result in a call to the respective service broker.
			
			For more information, see:
			
			• Updating Service Plans [page 755]
			• Updating Service Instance Parameters [page 758]
			• Updating Service Tags [page 761]
				n/a (fails only on asynchronous calls)	`fail-on-service-update:`
					`  parameters: true`
					`  plan: true`
					`  tags: true`

`log-level`	Resource	Write	Verbosity of the operation logs that are exported to the SAP Cloud Logging instance. Applies to a resource of type `org.cloudfoundry.cloud-logging-service`. Supported values are INFO, WARN, DEBUG, ERROR, and TRACE. The values are hierarchical. For more information, see (Experimental) Exporting MTA Operation Logs to SAP Cloud Logging [page 823].	INFO	`log-level: DEBUG`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`org-name`	Resource	Write	Name of the organization in which the SAP Cloud Logging instance resides. Specified within the destination parameter of a resource of type `org.cloudfoundry.cloud-logging-service`. For more information, see (Experimental) Exporting MTA Operation Logs to SAP Cloud Logging [page 823].	The organization of the current deployment	`org-name: my-org`
`service`	Resource	Write	The type of the created service	Empty, or as specified in resource-type	`service: hana`
`service-key-name`	Resource	Write	Used when consuming an existing service key. Specifies the name of the service key. See Consumption of Service Keys [page 764] for more information.	The name of the resource.	`service-key-name: my-service-key`
`service-name`	Resource	Write	The name of the service in the Cloud Foundry environment to be created for this resource, based on the resource name with or without a name-space prefix. **Note** Service names that do not comply with the Cloud Foundry limitation of 50 symbols are automatically corrected. In such cases, the name is shortened and its end is replaced with a hash code.	`${default-service-name}`	`nodejs-hdi-container` `com.sap.xs2.samples.xsjshelloworld.nodejs-hdi-container`
`service-plan`	Resource	Write	The plan of the created service	Empty, or as specified in resource-type	`service-plan: hdi-shared`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`service-tags`	Resource	Write	Some services employ a list of custom tags, which provide an easier way for applications to parse <VCAP_SERVICES> for credentials. You can provide custom tags when creating a service instance. For more information, see Service Tags [page 761].	n/a	`resources:`
					`  - name: nodejs-uaa`
					`    type: com.sap.xs.uaa`
					`    parameters:`
					`      service-tags: ["custom-tag-A", "custom-tag-B"]`

`service-broker`	Resource	Write	Use this parameter to specify the service broker you want to employ when you create your service. This can be useful for testing purposes, among others.	Name of service broker you want to be used.	`resources:`
					`  - name: test-service`
					`    type: org.cloudfoundry.managed-service`
					`    parameters:`
					`      service: foo`
					`      service-plan: bar-a`
					`      service-broker: test-broker-1`

`skip-service-updates`	Resource	Write	This parameter allows you to specify the service configuration changes to `ignore`, when deciding if you want to update a service instance – in particular changes of service `parameters`, `plan`, or `tags`.	The default value for all is `false`. The service instance would be updated on change in any of the configurations.	`resources:`
					`  - name: service-name`
					`    type: org.cloudfoundry.managed-service`
					`    parameters:`
					`      skip-service-updates:`
					`        plan: true`
					
					`parameters: true`
					`        tags: true`
					
					In the example above, `skip-service-updates` modifies the update strategy as follows:
					• allows users to specify which configurations should not be updated
					
					Note that these 3 key-value pairs can be in any order.

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`space-name`	Resource	Write	Name of the space in which the SAP Cloud Logging instance resides. Specified within the destination parameter of a resource of type `org.cloudfoundry.cloud-logging-service`. For more information, see (Experimental) Exporting MTA Operation Logs to SAP Cloud Logging [page 823].	The space of the current deployment	`space-name: my-space`

`syslog-drain-url`	Resource	Write	The URL to which logs for bound applications are streamed.
			Note
			This feature only works for user-provided services.
				n/a	`resources:`
					`  - name: service-name`
					`  type: org.cloudfoundry.user-provided-service`
					`  parameters:`
					`syslog-drain-url: syslog://example.log-aggregator.com`

→ Tip

For a better understanding of the interactions among the service, service broker, and service instances, see Provisioning and integrating service instances .

Related Information

Services Overview

Provisioning and integrating service instances

4.1.8.1.6.2 Modules

The `modules` section of the deployment descriptor lists the deployable parts contained in the MTA deployment archive.

The following elements are mandatory:

- `name` - must be unique within the MTA it identifies

- `type` - defines which deployment mechanism to be used for this module

Optional module attributes include:

- `path` - the file-system path relative to the root of the MTA directory. The content of the file is used to create or update the CF app or content, depending on the module target. The `path` is used only during MTA build. In an already built MTA archive (MTAR), path is ignored and only corresponding entry in `MANIFEST.MF` for the module is processed.
  The path is applicable only when MTA is assembled based on deployment descriptor (mtad.yaml), and not on development descriptor (mta.yaml)
- `description` - non-translatable, free-text string; the string is not meant to be presented on application user interfaces (UI)
- `properties` - a structured set of name-value pairs; if a module, which requires the resource, represents a CF application, the resource properties are injected into the environment of the application
- `parameters` - reserved variables that affect the behavior of the MTA-aware tools, such as the deployer
- `deployed-after` - the attribute is used to specify the order in which modules should be processed. Its value is a list comprised of the names of other modules within the same MTA. When this attribute is assigned to a module, it indicates that the module should be processed after the listed modules have been processed.

  > **Note**
  >
  > When using the `deployed-after` parameter in Blue-Green Deployment of Multitarget Applications [page 785], keep in mind that it cannot be used to alter the deployment order of the modules even though its name suggests otherwise. Instead, it can be used to specify the following:
  >
  > - In the first phase of the blue-green deployment, the `deployed-after` attribute is used to determine the order in which all idle applications are created.
  > - In the final phase of the blue-green deployment, the `deployed-after` attribute is used to determine the order in which all idle applications are restarted.
  > - It does not affect the testing phase, where both the live and idle versions of the applications are running in parallel.

  > **Note**
  >
  > When `bg-dependency-aware-stop-order` is enabled, the `deployed-after` attribute is used to determine the order in which all idle applications are restarted, helping prevent transient failures caused by dependencies being temporarily unavailable. For more information on `bg-dependency-aware-stop-order`, see the Top-Level Parameters table in Parameters and Properties [page 675].
- `requires` - specifies the names of `requires` sections provided in `resource` that have been declared for the same MTA. Tools check if all required names are provided within the MTA.
- `provides` - specifies the names of `provides` sections, each containing configuration data; the data provided can be `required` by other `modules` in the same MTA

> **→ Tip**
>
> Modules can be deployed in parallel. See Parallel Module Deployment [page 803].

MTA Module Types

Modify the following MTA module types by providing specific properties or parameters in the MTA deployment descriptor (`mtad.yaml`).

MTA Default Modules Types

Module Type	Default Parameter Values and Description	Module Properties	Result
`javascript.nodejs`	No default parameter. **Note** We strongly recommend that you specify the appropriate buildpack for your application.modules: parameters: buildpack: binary_buildpack To do so, use the `buildpack` - name: my-binary-app module parameter.	None	CF application with automatic buildpack detection
`nodejs`	`buildpack`(`nodejs_buildpack`)	None	CF application with Node.js runtime
`custom`	No default parameter. **Note** We strongly recommend that you specify the appropriate buildpack for your application. To do so, use the `buildpack` module parameter.	None	CF application with automatic buildpack detection
`application`	No default parameter. **Note** We strongly recommend that you specify the appropriate buildpack for your application. To do so, use the `buildpack` module parameter.	None	CF application with automatic buildpack detection
`java`	`buildpack` (`sap_java_buildpack_jakarta`)	None	CF application with Automatic runtime detection by `sap_java_buildpack_jakarta`
`java.tomcat`	`buildpack` (`sap_java_buildpack_jakarta`)	None	CF application with Tomcat runtime of `sap_java_buildpack_jakarta`

Module Type	Default Parameter Values and Description	Module Properties	Result
`java.tomee`	`buildpack` (`sap_java_buildpack_jakarta`)	None	CF application with Automatic runtime detection by `sap_java_buildpack_jakarta`
`com.sap.xs.hdi`	• `no-route` (`true`) Do not assign a route to the application. • `memory` (`256MB`) • `health-check-type` - (`none`) • `no-start (true)` • `cf-task` • `command (npm start)` • `memory (256MB)` • `name (deploy)` • `dependency-type` (`hard`) In circular module-dependencies, deploy modules with dependency type "hard" first • `buildpack(nodejs_buildpack)`	`EXIT (1)`	CF application with HDI content activation
`com.sap.xs.hdi-dynamic`	`buildpack(nodejs_buildpack)`	None	CF application with Node.js runtime
`com.sap.portal.content`	**Note** Before using this module type, update the content deployer applications to their latest version. • `no-route` (`true`). Defines if a route should be assigned to the application. • `no-start` - (`true`). Only the one-off tasks will be executed, that is, without triggering the start of the application. • `memory` (`256M`). Defines the memory allocated to the application. • `tasks` (`name:deploy, memory:256M, command:npm start`) For more information, see Tasks [page 753]. • `dependency-type`(`hard`). Defines if this module should be deployed first, if it takes part in circular module dependency cycles. If `hard` means that this module is deployed first. • `buildpack(nodejs_buildpack)`	None	CF application with SAP Fiori launchpad content

Module Type	Default Parameter Values and Description	Module Properties	Result
`com.sap.portal.site-content`	**Note** This module type is **deprecated**. You have to use `com.sap.portal.content` instead.	None	CF application with SAP Fiori launchpad content
`com.sap.application.content`	Required dependency parameters: • `content-target(false)` Specify that the resource would be used as a target for the module content deployment.	None	Direct content deployment to backing services

Module Type	Default Parameter Values and Description	Module Properties	Result
`com.sap.html5.application-content`	**Note** This module type is **deprecated**. Please use `com.sap.html5.application.content` instead. **Note** Keep the dependency to `@sap/html5-app-deployer` updated to the latest version. If you are using an older version, you might encounter some issues. • `no-route` (`true`). Defines if a route should be assigned to the application. • `memory` (`256M`). Defines the memory allocated to the application. • `execute-app` - (`true`). Defines whether the application is executed. If yes, the application performs certain amount of work and upon completion sets a `success-marker` or `failure-marker` by means of a log message. • `success-marker` (`STDOUT:The deployment of html5 application content done.*`) • `failure-marker`(`STDERR:The deployment of html5 application content failed.*`) • `stop-app` (`true`). Defines if the application should be stopped after execution. • `check-deploy-id` (`true`) - Defines if the deployment (process) ID should also be checked when checking the application execution status. • `dependency-type`(`hard`). Defines if this module should be deployed first, if it takes part in circular module dependency cycles. If `hard` means that this module is deployed first. • `health-check-type`(none). Defines if the module should be monitored for availability.	None	Deploys a content deployment application, and creates a task that performs the content deployment.

Module Type	Default Parameter Values and Description	Module Properties	Result
	• `buildpack(nodejs_buildpack)`		
`com.sap.html5.application.content`	**Note** Keep the dependency to `@sap/html5-app-deployer` updated to the latest version. If you are using an older version, you might encounter some issues. • `no-route` (`true`). Defines if a route should be assigned to the application. • `no-start` - (`true`). Only the one-off tasks will be executed, that is, without triggering the start of the application. • `memory` (`256M`). Defines the memory allocated to the application. • `tasks` (`name:deploy, memory:256M, command:npm start`) For more information, see Tasks [page 753]. • `dependency-type`(`hard`). Defines if this module should be deployed first, if it takes part in circular module dependency cycles. If `hard` means that this module is deployed first. • `buildpack(nodejs_buildpack)`	`EXIT (1)`	CF application with SAP HTML5 content
`staticfile`	`buildpack` (`staticfile_buildpack`)	None	CF application with static file runtime
`go`	`buildpack`(`go_buildpack`)	None	CF application with Go runtime
`python`	`buildpack`(`python_buildpack`)	None	CF application with Python runtime
`php`	`buildpack`(`php_buildpack`)	None	CF application with PHP runtime
`binary`	`buildpack`(`binary_buildpack`)	None	CF application with Binary runtime

To choose a `binary_buildpack`, define it by using the following:

Sample Code

```
modules:
  - name: my-binary-app
    type: custom
    parameters:
      buildpack: binary_buildpack
```

Module-Specific Parameters

Module parameters have platform-specific semantics. To reference a parameter value, use the placeholder notation `${<parameter>}`, for example, `${default-host}`.

> → Tip
>
> It is also possible to declare metadata for parameters and properties defined in the MTA deployment descriptor. The mapping is based on the parameter or property keys. For example, you can specify if a parameter is **required** (`optional: false`) or can be modified `overwritable: true`. See Metadata for Properties and Parameters [page 744].

The following parameters are supported:

> ⓘ Note
>
> If you can't find a specific parameter from the native Cloud Foundry manifest here, refer to Prerequisites and Restrictions [page 618] to see which Cloud Foundry features are currently not supported.

MTA Development and Deployment Module Parameters

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`app-features`	Module	Write	Allows enabling, disabling, and querying specific features for individual applications. These features typically control behaviors or capabilities related to application execution.
			
			The parameter accepts a map as a value. All parameters in the map are passed directly to the CF API. This mechanism ensures that all future app features will be automatically supported through MTA deployment.
			
			The parameter is especially useful when many services are bound to an application, and all credentials inside the `VCAP_SERVICES` environment variable exceed the maximum size of 130 KB (see Cloud Foundry environment variables). In this case, the application cannot be staged successfully. To solve that issue, you can switch to the "file-based VCAP services" method by enabling the respective app feature, and adapt your application coding accordingly to use a file instead of the `VCAP_SERVICES` environment variable. See Delivering service credentials to an app .
			
			For more information about the supported features, see Supported app features .
			
			Note
			If you want to configure SSH enablement, use this parameter instead of the deprecated `enable-ssh` parameter.
				n/a	`modules:`
					`- name: foo`
					`  type: application`
					`  parameters:`
					`  app-features:`
					`  file-based-vcap-services: true`
					`  ssh: true`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`app-name`	Module	Write	The name of the application in the Cloud Foundry environment to be deployed for this module, based on the module name	`${default-app-name}`	`node-hello-world` `com.sap.xs2.samples.xsjshelloworld.node-hello-world`

`apply-namespace`	Module	Write	Applies namespace to the application name. When you set `apply-namespace` to the application name and do not specify `apply-namespace` for its route, the namespace is applied to both the application name and its route. If the `namespace` value is not provided in the CLI options, it is not applied. For more information, see Fine-Grained Configuration [page 798].		`modules:`
					`- name: java`
					`...`
					`parameters:`
					`apply-namespace: true`

`buildpack`	Module	Write	The name or the URL of a custom buildpack required by the application	Empty, or as specified in the deploy service configuration	`buildpack: git://github.acme.com/xs2-java/xs2javabuildpack`
`buildpacks`	Module	Write	An array of buildpacks. If a buildpack parameter already exists, it will be overwritten by the buildpacks listed in the buildpacks parameter, so that you have to include it in the array.	Empty, or as specified in the deploy service configuration	`buildpacks: [java_buildpack, nodejs_buildpack, staticfile_buildpack]`
`command`	Module	Write	A custom command required to start the application	Empty, or as specified in the deploy service configuration	`command: node index.js`
`create-service-broker`	Module	Write	Specifies whether [true|false] a service broker should be registered for the application module	`false`	`create-service-broker: true`
`default-app-name`	Module	Read-Only	The name of the application in the Cloud Foundry environment to be deployed for this module, based on the module name	The module name with or without a namespace prefix	`node-hello-world` `com.sap.xs2.samples.xsjshelloworld.node-hello-world`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`default-host`	Module	Read-Only	The default host name, which is composed based on the module name to ensure uniqueness. Used with host-based routing to compose the default URI, see below. It follows the convention `${org}-${space}-<module_name>`. **→ Tip** We recommend you explicitly use the `routes` parameter. See Routes [page 765]. **Note** Host names that do not comply with the Cloud Foundry naming limitations are automatically corrected: • Any characters that are not within the allowed `a-z` and `0-9` ranges are replaced by dashes (-) to prevent deployment issues. • If the host name length exceeds 63 symbols, the name is shortened and its end is replaced with a hash code. **Note** When used in blue-green deployment the value is resolved depending on the phase. During the testing phase the "idle" suffix will be appended to the host. If you want to use the live version only, refer to `${default-live-host}`.	Generated as described in the description	`trial-a007007-node-hello-world`
`default-instances`	Module	Read-Only	The number of application instances that are started during the deployment	1	`default-instances: 1`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`default-live-app-name`	Module	Read-Only	Valid for blue-green deployment. Specify this parameter if you want the name of the Cloud Foundry application that is to be deployed with this module to be based on the name of the module. For standard deployment the parameter will be the same as `default-app-name` regardless the phase.	The module name with or without a namespace prefix	`node-hello-world`
`default-live-domain`	Module	Read-Only	Valid for blue-green deployment. The value of this parameter is the default domain for the current organization.		
`default-live-host`	Module	Read-Only	Valid for blue-green deployment. Specify this parameter if you want to use `${default-host}` without the "idle" suffix during the testing phase of the blue-green deployment.	Generated as described in the description	`trial-a007007-node-hello-world`
`default-live-uri`	Module	Read-Only	Valid for blue-green deployment. Specify this parameter if you want to use `${default-uri}` without the "idle" suffix during the testing phase of the blue-green deployment.	Generated as described in the description	`trial-a007007-node-hello-world.cfapps.acme.ondemand.com`
`default-live-url`	Module	Read-Only	Valid for blue-green deployment. Specify this parameter if you want to use `${default-url}` without the "idle" suffix during the testing phase of the blue-green deployment.	Generated as described in the description	`${protocol}://${default-live-uri}`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`default-uri`	Module	Read-Only	The default URI, composed as `${host}.${domain}` (host-based routing). Note that `${host}` will be the same as `${default-host}`, unless specified explicitly as a parameter. Similarly, `${domain}` would be the same as `${default-domain}`, unless specified explicitly. **Note** When used in blue-green deployment the value is resolved depending on the phase. During the testing phase the "idle" suffix will be appended to the host of the URI. If you want to use the live version only, refer to `${default-live-uri}`.	Generated as described in the description.	`trial-a007007-node-hello-world.cfapps.acme.ondemand.com`
`default-url`	Module	Read-Only	The default URL, composed as `${protocol}://${default-uri}`. Note that the `${default-uri}` placeholder is resolved as `${host}.${domain}` (host-based routing) **Note** When used in blue-green deployment the value is resolved depending on the phase. During the testing phase the "idle" suffix will be appended to the host of the URL. If you want to use the live version only, refer to `${default-live-url}`.	Generated as described in the description.	`${protocol}://${default-uri}`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`dependency-type`	Module	Write	Deployment order of modules with circular dependencies	soft	`dependency-type: hard` `dependency-type: soft`
`disk-quota`	Module	Write	The disk space that will be available to the application. This parameter requires a unit of measurement M, MB, G, or GB in upper or lower case.	1, or as specified in module-type	`disk-quota: 1G`

`docker`	Module	Write	Creates a module from a docker image. When using a docker image parameter, we do not need to specify the module in the `MANIFEST.mf` file. An image parameter is a docker image from the Docker Hub or somewhere else. The username and the password are optional, but if a Docker image from a private repository is uploaded, then they are mandatory.
			When uploading a docker image, the content of a module is not needed.
				n/a	Sample Code
					`modules:`
					`  name: foo`
					`  type: application`
					`  parameters:`
					`  docker:`
					`  image: cloudfoundry/test-app`
					`username: <optional username>`
					`password: <optional password>`

`domain`	Module	Write	The domain on which the application is available later	`${default-domain}`	`domain: ${default-domain}.acme.com`

`domains`	Module	Write	The domains on which the application is available later. The resulting application routes are the Cartesian product of the domains and hosts. That is, a separate route for each host is constructed on each domain.	`domains: - ${default}`	`domains:`
					`- ${default-domain}.acme.com`
					`- test-${default-domain}.acme.com`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`enable-ssh`	Module	Write	Enables use of SSH within an application. Supported for the Diego container runtime environment only. **Note** This parameter is deprecated. To enable SSH within an application, use the `app-features` parameter instead.	n/a	`"enable-ssh": true` `"enable-ssh": false`

`enable-parallel-service-bindings`	Module	Write	Enables or disables the parallel binding or unbinding of services during deployment. If disabled, the services are bound and unbound sequentially in the order provided in the deployment descriptor.	`true`	**Sample Code**
					`- name: <module name>`
					`  type: <module type>`
					`  parameters:`
					`  enable-parallel-service-bindings: false`

`health-check-http-endpoint`	Module	Write	If the `health-check-type` parameter is set to `http`, the controller will do a `GET` request to this endpoint. The application will be considered as healthy if the response is 200 OK.	If `health-check-type` is set to `http`, the default value is `/`, otherwise there is no default value.	`health-check-type: http` `health-check-http-endpoint: /health`
`health-check-timeout`	Module	Write	The application health check timeout in seconds	n/a	`health-check-timeout:120`
`health-check-invocation-timeout`	Module	Write	The time period in seconds, within which the application health check should be automatically started. If the health check does not start within the defined time period, it is omitted, and the deployment process continues.	n/a	`health-check-invocation-timeout:16`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`health-check-type`	Module	Write	The application health check type	`port`	`health-check-type: port`
					`health-check-type: http`
					`health-check-type: process`

`health-check-interval`	Module	Write	Specifies the interval, in seconds, between successive liveness health check invocations that the Cloud Controller performs against the application.	n/a	`health-check-interval: 15`
`host`	Module	Write	The hostname or subdomain where an application is available later. If you want to create a wildcard hostname, use an asterisk in quotes ("*").	`${default-host}`	`host: ${space}-node-hello-world` `host: "*"`

`hosts`	Module	Write	The hostnames or subdomain where an application is available later.
			If you want to create a wildcard hostname, use an asterisk in quotes ("*").
				`hosts: - ${host}`	`modules:`
					`- name: my-app`
					`type: application`
					`parameters:`
					`hosts:`
					`- ${space}-node-hello-world`
					`- "*"`

`idle-domain`	Module	Write	Valid for a blue-green deployment when a new application version is started on a temporary route.
			Specify this parameter if you want to use another domain for temporary routes.
				`${default-domain}`	`modules:`
					`- name: app`
					`type: nodejs`
					`parameters:`
					`idle-domain: "<some.domain>"`

`idle-domains`	Module	Write	Valid for a blue-green deployment when a new application version is started on several temporary routes.
			Specify this parameter if you want to use other domains for temporary routes by listing them in an array.
				`${default-domain}`	`modules:`
					`- name: app`
					`type: nodejs`
					`parameters:`
					`idle-domains: ["<some.domain>", "<some.other.domain>"]`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`idle-host`	Module	Write	Valid for a blue-green deployment when a new application version is started on a temporary route.
			
			Specify this parameter if you want to use another host for a temporary route.
			
			Note
			The new application will start on a route comprised of the specified host and the default domain.
				`${default-host}-idle`	`modules:`
					`  - name: app`
					`    type: nodejs`
					`    parameters:`
					`      idle-host: "<some-hostname>"`

`idle-hosts`	Module	Write	Valid for a blue-green deployment when a new application version is started on temporary routes.
			
			Specify this parameter if you want to use other hosts for temporary routes by listing them in an array.
			
			Note
			The new application will start on routes comprised of the specified hosts and default domain.
				`${default-host}-idle`	`modules:`
					`  - name: app`
					`    type: nodejs`
					`    parameters:`
					`      idle-hosts: ["<some-hostname>", "<some-other-hostname>"]`

`idle-routes`	Module	Write	Valid for a blue-green deployment when a new application version is started on temporary routes.
			
			Specify this parameter if you want to use other routes for the application.
				`${default-uri}-idle`	`modules:`
					`- name: app`
					`  parameters:`
					`    idle-routes:`
					`     - idle-route: "<your-first-idle-hostname.your.first.idle.domain>"`
					`     - idle-route: "<your-second-idle-hostname.your.second.idle.domain>"`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`instances`	Module	Write	The number of application instances that will be started during the deployment	`${default-instances}`	`instances: 2`

`keep-existing`	Module	Write	Defines the application attributes which will be kept after the deployment or blue-green deployment has finished. The supported attributes which could be kept are application environment, application bindings and application routes. If not specified, the default values are false, which indicates that each application attribute will be updated with the new values presented in the deployment descriptor.	`false`	Sample Code
					`keep-existing:`
					`  env: true`
					`  service-bindings: true`
					`  routes: false`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`keep-existing-routes`	Module	Write	When specified on module level, it indicates if the existing routes of the module's corresponding application should be kept even if they are not defined within the deployment and/or extension descriptors.
			
			When specified on top level, under the `parameters` section of the descriptor, it indicates if the existing routes of all applications within that MTA should be kept.
			
			Note
			- The module-level variant of the parameter has priority over the top-level parameter.
			- This parameter is typically used when users want to keep the routes they have mapped manually by using the `cf map-route` command. We discourage this approach, as manual operations could lead to inconsistent deployment results and difficult troubleshooting. We recommend you to define all routes in the deployment and/or extension descriptors, which allows for their automatic management.
				`false`	**Sample Code**
					`parameters:`
					
					`keep-existing-routes: true`
					`modules:`
					`  - name: foo`
					`  type: nodejs`
					`  parameters:`
					
					`keep-existing-routes: false`
					`  - name: bar`
					`  type: nodejs`
					`  - name: baz`
					`  type: nodejs`

`memory`	Module	Write	The memory limit for all instances of an application. This parameter requires a unit of measurement M, MB, G, or GB in upper or lower case.	256M, or as specified in module-type	`memory: 128M`
`no-route`	Module	Write	Configures the deployer to create or skip the creation of a route for the application described by the module	false	`no-route: true`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`no-start`	Module	Write	Start/do not start the application during deployment. **→ Tip** This parameter setting overrides the command-line option `--no-start`. If you explicitly set the `no-start` to `false` for the module `foo` in the example provided, then the module `foo` **is** started on deployment, even if you also specify the command-line option `--no-start` with the `cf deploy` command.	Depends on the command line option `--no-start`	`no-start: true`

`readiness-health-check-http-endpoint`	Module	Write	The endpoint called to determine if the app is ready.
			The parameter can be applied if the `readiness-health-check-http-endpoint` parameter is set to `http`. The controller will make a `GET` request to this endpoint, and the application will be considered healthy if the response is 200 OK.
				n/a	**{·} Sample Code**
					`readiness-health-check-type: http`
					`readiness-health-check-http-endpoint: /health`

`readiness-health-check-type`	Module	Write	The type of health check to be performed on the module: `http`, `port`, or `process`	`process`	`readiness-health-check-type: port`
					`readiness-health-check-type: http`
					`readiness-health-check-type: process`

`readiness-health-check-invocation-timeout`	Module	Write	The timeout in seconds for individual readiness health check requests. The parameter can be used for the `http` and `port` health check types.	n/a	`readiness-health-check-invocation-timeout: 20`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`readiness-health-check-interval`	Module	Write	The amount of time in seconds between individual readiness health check requests	n/a	`readiness-health-check-interval: 20`

`restart-on-env-change`	Module	Write	Specifies whether an application should be restarted if an environment variable has been changed in one of the following categories:
			• vcap-application
			• vcap-services
			• user-provided
			Note
			If you set these parameters to `false`, the changes in environment are not consumable by a running instances of the application. If your application depends on the latest environment, it might become outdated.
				Sample Code
				`restart-on-env-change:`
				`  vcap-application: true`
				`  vcap-services: true`
				`  user-provided: true`
					Sample Code
					`restart-on-env-change:`
					`  vcap-application: false`
					`  vcap-services: true`
					`  user-provided: true`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`routes`	Module	Write	A parameter that lists multiple HTTP routes. For more information, see Routes [page 765].	`${default-uri}`	Sample Code
					`modules:`
					`- name: my-app`
					`type: application`
					`parameters:`
					`routes:`
					`- route: "*foo.my.custom.domain/path"`
					`- route: foo.my.custom.domain/path`
					`- route: foo.${default-domain}/path`

`route-path`	Module	Write	The context “route-path” which is part of the application default URI. Context path routing is routing based not only on domain names (host header) but also the path specified in the URL.	n/a	`route-path: /myapp`
`service-broker-name`	Module	Write	The name of the service broker in the Cloud Foundry environment to be created and registered for the specified application module	`${app-name}`	`service-broker-name: jobscheduler` `service-broker-name: ${app-name}`
`service-broker-password`	Module	Write	The password used for authentication by the Cloud Controller at the service broker when performing service-related requests. The parameter is mandatory if `create-service-broker: true`.		`service-broker-password: ${generated-password}`
`service-broker-space-scoped`	Module	Write	Makes the service plans of the broker visible only within the targeted space.	`false`	`service-broker-space-scoped: true`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`service-broker-url`	Module	Write	Specifies the value of the service broker universal resource locator (URL) to register; service requests are sent to this URL. The parameter is mandatory if `create-service-broker: true`.		`service-broker-url: ${default-url}`
`service-broker-user`	Module	Write	The name of the user required for authentication by the Cloud Controller at the service broker when performing service-related requests. The parameter is mandatory if `create-service-broker: true`.		`service-broker-user: ${generated-user}`

`skip-deploy`	Module	Write	Use this parameter to skip the deployment of a specified module even if it is present in the descriptor. If you add this parameter for an already deployed module, this module will be deleted in the next operation.	`false`	Sample Code
					`modules:`
					`  - name: foo`
					`  type: application`
					`  parameters:`
					`  skip-deploy: true`

`stack`	Module	Write	Use this parameter to define which prebuilt root file system (`rootfs`) you want to use.	n/a	Sample Code
					`modules:`
					`  - name: foo`
					`  type: application`
					`  parameters:`
					`  stack: cflinuxfs3`

`stage-timeout`	Module	Write	Defines how long, in seconds, your application can take during staging before the MTA operation times out.
			See Application-Specific Timeouts [page 750].
				1h	`modules:`
					`  - name: java`
					`  .....`
					`  parameters:`
					`  stage-timeout: 100`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`start-timeout`	Module	Write	Defines how long, in seconds, your application can take to start before the MTA operation times out.
			See Application-Specific Timeouts [page 750].
				1h	`modules:`
					`  - name: java`
					`  ..........`
					`  parameters:`
					`  start-timeout: 100`

`task-execution-timeout`	Module	Write	Defines how long, in seconds, your application can take to execute a task before the MTA operation times out.
			See Application-Specific Timeouts [page 750].
				12h	`modules:`
					`  - name: java`
					`  ..........`
					`  parameters:`
					`task-execution-timeout: 100`

`tasks`	Module	Write	Specify tasks, which are available for execution in the current droplet of the application. Also provide use of environment variables which are specified with the `env` scope.	n/a	`tasks:`
					`  - name: task-1`
					`command: some-script.sh`
					`env:`
					`  env1: value1`
					`  env2: value2`

`upload-timeout`	Module	Write	Defines how long, in seconds, you can upload your application binary before the MTA operation times out.
			See Application-Specific Timeouts [page 750] .
				1h	`modules:`
					`  - name: java`
					`  .........`
					`  parameters:`
					`  upload-timeout: 100`

`timestamp`	Module	Read-Only	Current timestamp in milliseconds	Generated as described in the description.	

Module-Specific Properties

The following properties are supported:

MTA Development and Deployment Module Properties

Property	Description	Default Value	Example

`MTA_WAIT_AFTER_APP_STOP`	Adds configurable delay in seconds, after stopping the application.
	
	Note
	Reserved environment variable for subsequent MTA operations, which is used in blue-green deployment and undeployment.
	
	Tip
	Can be used to introduce a faux delay between stopping an application and unbinding its services, in order to avoid errors when unbinding from an application that is still in the process of stopping.
		n/a	**Sample Code**
			`modules:`
			`- name: my-app`
			`  type: application`
			`  properties:`
			
			`MTA_WAIT_AFTER_APP_STOP: 30`

Shared Module Binaries

By default every module has its own binary that is uploaded and deployed in specific manner. It is possible for multiple MTA modules to reference a single deployable binary, for example, an application archive. This means that during deployment, the same application archive is executed separately in multiple applications or application instances, but with different parameters and properties. This results in multiple running applications based on the same source code, which have different configurations and setup. A development project can have one source folder, which is referenced by multiple module entries in the MTA deployment descriptor `mtad.yaml`, as illustrated in the following example:

Sample Code

Multiple MTA Module Entries in the Deployment Descriptor (`mtad.yaml`)

```
_schema-version: "3.1.0"
ID: hello
version: 0.1.0
modules:
   - name: hello-router
```

```
      type: java.tomee
      path: web/router.war
      requires:
      - name: backend
        properties:
          backend: ~{url}/content
          name: backend
          url: ~{url}

    - name: hello-backend
      type: java.tomee
      path: web/router.war
      provides:
        - name: backend
          properties:
            url: "${default-url}"
```

If deployment is based on an MTA archive, it is not necessary to duplicate the code to have two different deployable modules; the specification for the MTA-module entry in `MANIFEST.MF` is extended, instead. The following (incomplete) example of a `MANIFEST.MF` shows how to use a comma-separated list of module names to associate one set of deployment artifacts with all listed modules:

Code Syntax

Multiple MTA Modules Listed in the `MANIFEST.MF` Deployment Manifest

```
Name: web/router.war
MTA-Module: hello-router,hello-backend
Content-Type: application/zip
```

Related Information

Managing Service Brokers
Parameters and Properties [page 675]
Metadata for Properties and Parameters [page 744]

4.1.8.1.6.3 Module Hooks

Define and execute hooks at specific phases of module deployment.

Note

The module hooks are supported from major schema version 3 onwards. If they are specified but schema version is below 3, they are ignored.

You can use hooks to change the typical deployment process, in this case to enable tasks to be executed during a specific moment of the application deployment. For example, you can set hooks to be executed before or after the actual deployment steps for a module, depending on the applications' needs.

> ### Note
>
> Currently, module hooks are only Cloud Foundry app tasks. Therefore, the timeout of a hook is the task execution timeout defined for the respective module. For more information, see Application-Specific Timeouts [page 750].

When added to the deployment descriptor, module hooks are modeled as follows:

> ### Sample Code

```
_schema-version: "3.3"
ID: foo
version: 3.0.0
modules:
  - name: foo
    type: javascript.nodejs
    hooks:
      - name: hook
        type: task
        phases:
          - blue-green.application.before-stop.live
          - blue-green.application.before-stop.idle
        parameters:
          name: foo-task
          command: 'sleep 5m'
```

A module hook is defined with the following attributes:

- `name` – defines the name of the hook.
- `type` – defines the type of the hook. Currently, the only supported type is `task`.
- `phases` – defines the specific moment when a hook is executed. In the example above, the `phases` element denotes that the hook is executed before the application is stopped. The suffixes `live` and `idle` are used during blue-green deployments and indicate when the `before-stop` phase is executed.

  > ### Example
  >
  > In the example above, the `blue-green.application.before-stop.idle` phase executes the hook when the new idle applications are redirected to the new live routes, and the `blue-green.application.before-stop.live` is used just before the deletion of the live application.

- `parameters` – defines the parameters of the hook. For the hooks of type `task`, the parameters must define a one-off task. See Module Hooks - Specific Parameters [page 672].

Deployment Phases

Deployment hook phases are specific to the deployment strategy. Phases defined for a standard deployment are only executed during a standard deployment, while phases defined for a blue-green deployment are only executed during a blue-green deployment. Phases that do not match the deployment type are ignored.

Depending on the deployment strategy you use, the `phases` values are:

- **For standard deployment**

Phase	Supported for types	Description
`deploy.application.before-stop`	`task`	Executed before the application corresponding to the module is stopped.
`deploy.application.after-stop`	`task`	Executed after the application corresponding to the module is stopped.
`deploy.application.before-start`	`task`	Executed before the application corresponding to the module is started.

- **For blue-green deployment**
  In a blue-green deployment context, **live** refers to the previously deployed application version, while **idle** refers to the newly deployed application version. For more information, see Blue-Green Deployment of Multitarget Applications [page 785].
  The **Default Target App** column shows the application on which the hook task is executed by default. The override the target application for a specific phase, see Overriding the Default Target App for Hook Execution [page 671].

Phase	Supported for types	Default Target App	Description
`blue-green.application.before-stop.idle`	`task`	idle	Executed before the idle application corresponding to the module is stopped. Idle applications are created as part of blue-green deployments and are restarted (stopped and started again) after the deployment validation phase of their productization. That is after the live routes are mapped to them and their environments are rebuilt to contain only live routes.
`blue-green.application.before-stop.live`	`task`	live	Executed before the live application corresponding to the module is stopped. Live applications are the ones that are already up and running before the deployment starts.
`blue-green.application.after-stop.idle`	`task`	idle	Executed after the idle application corresponding to the module is stopped.
`blue-green.application.after-stop.live`	`task`	live	Executed after the live application corresponding to the module is stopped.
`blue-green.application.before-unmap-routes.live`	`task`	live	Executed before unmapping the route of the live application that corresponds to the module.

Phase	Supported for types	Default Target App	Description
`blue-green.application.before-start.idle`	`task`	idle	Executed before the idle application corresponding to the module is started.
`blue-green.application.before-start.live`	`task`	idle	Executed before the live application corresponding to the module is started. **Note** Although this phase name carries the `.live` suffix, the task runs on the idle application by default, because at this point of the deployment it is the idle application that is being restarted with its live route configuration as part of being promoted to the live role.

Overriding the Default Target App for Hook Execution

By default, each blue-green deployment phase executes its hook task on the application that corresponds to the phase suffix. Phases with the `.live` suffix run on the live application, and phases with the `.idle` suffix run on the idle application. The exception is `blue-green.application.before-start.live`, which runs on the idle application by default (see the table above for more information).

You can use the `phases-config` parameter to override the default target application for one or more phases within a single hook definition. This makes it possible to define whether a hook should be executed on the live or idle application for a specific phase.

Example

In the following example, the hook is triggered during the `blue-green.application.after-stop.live` phase, but the task is executed on the idle application instead of the default live application:

```
hooks:
  - name: target-app-test
    type: task
    phases:
      - blue-green.application.after-stop.live
    parameters:
      command: "echo Executing hook"
      phases-config:
        - phase: blue-green.application.after-stop.live
          target-app: idle
```

You can also configure this for multiple phases at once:

```
hooks:
  - name: target-app-test
```

```
    type: task
    phases:
      - blue-green.application.after-stop.live
      - blue-green.application.before-start.idle
    parameters:
      command: "echo Executing hook"
      phases-config:
        - phase: blue-green.application.after-stop.live
          target-app: idle
        - phase: blue-green.application.before-start.idle
          target-app: live
```

Phases listed in the hook's `phases` array but not referenced in `phases-config` continue to run on their default target application.

> ### Note
>
> Only one `target-app` entry can be defined per phase. Multiple entries for the same phase are not supported.

Module Hooks - Specific Parameters

The table below contains the parameters of the supported module hook types:

Module-Hooks-Specific Parameters

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`name`	Module Hook	Write	Defines the name of the Cloud Foundry task that should be executed	The name of the hook	`name: db_migration`
`command` **Note** This is a mandatory parameter.	Module Hook	Write	Defines the actual command that is executed as a Cloud Foundry task		`command: "bin/rails db:migrate"`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`memory`	Module Hook	Write	Defines the memory that is available to the Cloud Foundry task	Landscape-specific value that is equal to the default application memory → Remember Do not rely on the default value, as it will probably be much higher than you need and may not fit into the limitations of your quota.	`memory: 256M` `memory: 1G`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`disk-quota`	Module Hook	Write	Defines the disk space that is available to the Cloud Foundry task	Landscape-specific value that is equal to the default application disk quota → Remember Do not rely on the default value, as it will probably be much higher than you need and may not fit into the limitations of your quota.	`disk-quota: 256M` `disk-quota: 1G`

`phases-config`	Module Hook	Write	Overrides the default target application (`live` or `idle`) on which a hook task is executed for a specific blue-green deployment phase. Valid values for `target-app` are `live` and `idle`. For more information, see Overriding the Default Target App for Hook Execution [page 671].		`phases-config:`
					`  - phase: blue-green.application.after-stop.live`
					`  target-app: idle`
					
					This hook will be executed on the idle application, after the stop of the live application.

Extending Module Hooks Through an Extension Descriptor

You can also extend module hooks through the extension descriptor. To do so, add the code with your specific parameters, similarly to the following example:

Sample Code

```
_schema-version: "3.3"
ID: foo-change-command
extends: foo
modules:
  - name: foo
    hooks:
      - name: hook
        parameters:
          command: 'sleep 1m'
```

Related Information

MTA Deployment Descriptor Examples [page 624]
Defining Multitarget Application Deployment Descriptors for Cloud Foundry [page 623]
Defining MTA Extension Descriptors [page 745]
Legacy Blue-Green Deployment [page 785]

4.1.8.1.6.4 Parameters and Properties

This section contains information about the parameters and properties of a multitarget application (MTA).

The values of parameters and properties can be specified at design time, in the MTA development description (`mta.yaml`) or in the MTA deployment descriptor (`mtad.yaml`). In some cases, it is better to declare certain values depending on the deployment, for example, in an extension descriptor file (`myDeployExtension.mtaext`).

The values of parameters and properties might be literals. This way, the result of each deployment will be the same every time. However, by using the placeholders described below, the values might also be set as substitution variables and therefore each deployment might end in a different result depending on the environment or other configurations.

Regardless of the defined value - literal or substitution variable, in most cases the end value for each parameter or property is definitive and well-known before the deployment. This is because the substitution variable values represent a certain template and template resolving is done in the beginning of the MTA deployment.

The module-level parameter `${default-host}` follows the template `${org}-${space}-<module_name>`. For example, when it is used in a module `my-module` and the deployment is done within org `my-org`, space `my-space`, the end value of `${default-host}` will be `my-org-my-space-my-module`.

However, in some cases the parameter or property value is dynamic and cannot be predicted before the deployment. In these cases, the substitution variable is dynamic and is done in a later phase of the MTA deployment.

If you want to learn more about dynamically resolved parameters see the Dynamic parameters [page 689] section below.

The values of properties and parameters are used during the deployment or at runtime of the MTA.

> ⓘ Note
>
> Both parameters and properties may have literal values, such as strings, integers, etc. This also applies to deeply nested structured values, such as arrays or maps.

> → Tip
>
> - You can declare metadata for parameters and properties defined in the MTA deployment descriptor. The mapping is based on the parameter or property keys. For example, you can specify if a parameter is **required** (`optional: false`) or can be modified `overwritable: true`. For more information, see Metadata for Properties and Parameters [page 744].
> - Descriptors can contain so-called placeholders (also known as substitution variables), which can be used as sub-strings within property and parameter values. Placeholder names are enclosed by the dollar sign (`$`) and curly brackets (`{}`). For example: `${host}` and `${domain}`. For each parameter "`P`", there is a corresponding placeholder `${P}`. The value of <P> can be defined either by a descriptor used for deployment, or by the deploy service itself. Placeholders can also be used without any corresponding parameters; in this scenario, their value cannot be overridden in a descriptor. Such placeholders are read-only.

Parameters

Parameters are reserved variables that affect the behavior of the MTA-aware tools, such as the Cloud MTA Build Tool (MBT) or SAP Cloud Deployment service.

Parameters might be used at various levels in the MTA descriptor — top level, module level, resource level, and dependency level. Based on the parameter applicability it might be used in combination in several places, for example, both at the resource and module levels.

> ✤ Example
>
> The example below shows the parameter `` `memory` `` at the module level which defines the amount of memory used by the Cloud Foundry application represented by the module `` `node-hello-world` `` during application runtime.
>
> > {·} Sample Code
> >
> > ```
> > modules:
> >   - name: node-hello-world
> >     type: javascript.nodejs
> >     path: web/
> >     parameters:
> >        memory: 128M
> > ```

Parameters can be "Read-Only" (also known as "System") or "Write" (default value can be overwritten). All parameter values can be referenced as part of other property or parameter value strings. The value

of a “Read-Only” parameter cannot be changed in descriptors. Only its value can be referenced using the placeholder notation. To reference a parameter value, use the placeholder notation `${<parameter>}`, for example `${org}`.

SAP Cloud Deployment service supports a list of parameters and their (default) values:

- Module-Specific Parameters [page 650]
- Resource-Specific Parameters [page 637]
- Module Hooks - Specific Parameters [page 672]
- Top-Level Parameters (see table below) - Top-level parameters are generally applicable parameters that cover various scenarios and can be used in different parts of the MTA descriptors. They can be grouped into to categories - "Read-Only" and "Write":
- - Top-level "Write" parameters are designed to impact the behavior of the whole MTA deployment or many parts of it. It is wise to use them in order to avoid duplication. Their values can be defined and controlled by the customer. These parameters can be referenced throughout the whole document.
  - Top-level "Read-Only" parameters are commonly used parameters that can only be referenced in different parts of the MTA descriptors.

- Top-Level Parameters

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`apply-namespace:`
 	
`app-names: true/false`
 	
`service-names: true/false`
 	
`app-routes: true/false`
`as-suffix: true/false`
	Top-Level	Write	Applies namespace to application names (app-names), service names (service-names), application routes (app-routes) as suffix or prefix (as-suffix). If the namespace value is not provided in the CLI options, it is not applied.
			
			See Fine-Grained Configuration [page 798].
			
			Note
			This applies to all applications.
				true	`parameters:`
					`apply-namespace:`
					`app-names: true`
					`service-names: true`
					`app-routes: false`
					`as-suffix: true`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`apps-stage-timeout`	Top-Level	Write	Defines how long, in seconds, your application can take during staging before the MTA operation times out. See Application-Specific Timeouts [page 750] . **Note** This applies to all applications.	1h	`parameters:` `  apps-stage-timeout: 100`
`apps-start-timeout`	Top-Level	Write	Defines how long, in seconds, your application can take to start before the MTA operation times out. See Application-Specific Timeouts [page 750] . **Note** This applies to all applications.	1h	`parameters:` `  apps-start-timeout: 100`
`apps-task-execution-timeout`	Top-Level	Write	Defines how long, in seconds, your application can take to execute a task before the MTA operation times out. See Application-Specific Timeouts [page 750] . **Note** This applies to all applications.	12h	`parameters:` `  apps-task-execution-timeout: 100`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`apps-upload-timeout`	Top-Level	Write	Defines how long, in seconds, you can upload your application binary before the MTA operation times out. See Application-Specific Timeouts [page 750] . **Note** This applies to all applications.	1h	`parameters:` `  apps-upload-timeout: 100`
`authorization-url`	Top-Level	Read-Only	The authorization URL as specified in the cloud controller's `/v2/info` endpoint.	Generated as described in the description.	`https://login.cf.eu10-004.hana.ondemand.com`
`bg-dependency-aware-stop-order` **Note** This parameter is currently experimental.	Top-Level	Write	Ensures that during the final phase of a blue-green deployment, idle applications are stopped and restarted in a dependency-aware order based on the dependencies defined in the `deployed-after` attribute. This prevents transient failures caused by short-lived unavailability of dependent modules. First, dependent modules are stopped, then their dependencies are restarted, and finally the dependent modules are updated and started. **Note** When both are specified, the command-line option `--dependency-aware-stop-order` overrides the `bg-dependency-aware-stop-order` descriptor parameter.	false	`parameters:` `  bg-dependency-aware-stop-order: true`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`controller-url`	Top-Level	Read-Only	The URL of the cloud controller	Generated as described in the description.	`https://api.cf.eu10-004.hana.ondemand.com`
`default-domain`	Top-Level	Read-Only	The default domain of the target organization (configured in the Cloud Foundry environment). The value is coming from CF API endpoint Get default domain , and normally, it depends on the shared domains in the current organization.	Generated as described in the description.	**Example** If MTA deployment is done in a target org/space with CF API endpoint that is equal to `api.eu10-004.hana.ondemand.com`, the value of `default-domain` would be `cfapps.eu10-004.hana.ondemand.com`. **Example** If MTA deployment is done in a target org/space with CF API endpoint that is equal to `api.eu10.hana.ondemand.com`, the value of `default-domain` would be `cfapps.eu10.hana.ondemand.com`.

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`region-domain`	Top-Level	Read-Only	The domain of the current region. For more information, refer to the **Domain** column in Regions and API Endpoints Available for the Cloud Foundry Environment [page 19]. When a region has multiple CF API endpoints, it will point to the domain of the respective endpoint used for the deployment. **Note** The parameters `region-primary-domain` and `region-domain` are equal when used in most of the regions. They are different when MTA deployments are done in a target org/space in different CF APIs in one region.	Generated as described in the description.	**Example** If MTA deployment is done in a target org/space with CF API endpoint that is equal to `api.eu10.hana.ondemand.com`, the value of `region-domain` would be `eu10.hana.ondemand.com`. **Example** If MTA deployment is done in a target org/space with CF API endpoint that is equal to `api.eu10-004.hana.ondemand.com`, the value of `region-domain` would be `eu10-004.hana.ondemand.com`.

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`region-primary-domain`	Top-Level	Read-Only	The primary domain of a region. For more information, refer to the **Domain** column in Regions and API Endpoints Available for the Cloud Foundry Environment [page 19]. When a region has multiple CF API endpoints, it will point to the primary domain (the first one). **Note** The parameters `region-primary-domain` and `region-domain` are equal when used in most of the regions. They are different when MTA deployments are done in a target org/space in different CF APIs in one region.	Generated as described in the description.	**Example** If MTA deployment is done in a target org/space with CF API endpoint that is equal to `api.eu10.hana.ondemand.com`, the value of `region-primary-domain` would be `eu10.hana.ondemand.com`. **Example** If MTA deployment is done in a target org/space with CF API endpoint that is equal to `api.eu10-004.hana.ondemand.com`, the value of `region-primary-domain` would be `eu10.hana.ondemand.com`.
`deploy-url`	Top-Level	Read-Only	The deploy service URL for the Cloud Foundry environment	Generated as described in the description.	

`enable-parallel-deployments`	Top-Level	Write	Defines if parallel deployment of modules is enabled. When enabled, modules without explicitly set `deployed-after` parameter are deployed in parallel. For more information, see Parallel Module Deployment [page 803].	no	`parameters:`
					
					`enable-parallel-deployments: true`

`generated-password`	Top-Level	Read-Only	Randomly generated string value that is composed of 16 characters that may contain upper and lower case letters, digits and special characters (_, -, @, $, &, #, *).	Generated as described in the description.	`IG@zGg#2g-cvMvsW`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`generated-user`	Top-Level	Read-Only	A generated user id that is composed of 16 characters that may contain upper and lower case letters, digits and special characters (_, -, @, $, &, #, *).	Generated as described in the description.	`uYi$d41TzM1-Dm6f`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`keep-existing-routes`	Top-Level	Write	When specified on module level, it indicates if the existing routes of the module's corresponding application should be kept even if they are not defined within the deployment and/or extension descriptors.
			
			When specified on top level, under the `parameters` section of the descriptor, it indicates if the existing routes of all applications within that MTA should be kept.
			
			Note
			• The module-level variant of the parameter has priority over the top-level parameter.
			• This parameter is typically used when users want to keep the routes they have mapped manually by using the `cf map-route` command. We discourage this approach, as manual operations could lead to inconsistent deployment results and difficult troubleshooting. We recommend you to define all routes in the deployment and/or extension descriptors, which allows for their automatic management.
				`false`	`parameters:`
					`keep-existing-routes: true`
					`modules:`
					`- name: foo`
					`type: nodejs`
					`parameters:`
					`keep-existing-routes: false`
					`- name: bar`
					`type: nodejs`
					`- name: baz`
					`type: nodejs`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`org`	Top-Level	Read-Only	Name of the target organization	The current name of the target organization	`initial, trial`
`protocol`	Top-Level	Read-Only	The protocol used by the Cloud Foundry environment.	`http` or `https`	`http, https`
`space`	Top-Level	Read-Only	Name of the target organizational space	Generated as described in the description.	`initial, a007007`
`user`	Top-Level	Read-Only	Name of the current user	Generated as described in the description.	
`xs-type`	Top-Level	Read-Only	The XS type, Cloud Foundry or XS advanced	CF	`CF, XSA`
`org-guid`	Top-Level	Read-Only	GUID (Globally Unique Identifier) of the target organization	N/A	`06564ad5-1b38-458d-8c85-a2e0bcd990a9`
`space-guid`	Top-Level	Read-Only	GUID (Globally Unique Identifier) of the target space	N/A	`06564ad5-1b38-458d-8c85-a2e0bcd990a9`
`mta-version`	Top-Level	Write	The version of the MTA	N/A	`1.0.0`
`mta-id`	Top-Level	Write	The ID of the MTA	N/A	`com.sap.sample`

These parameters can be used with the `provides` or `requires` dependencies:

Dependency-Specific Parameters

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`binding-name`	Module-Required Dependency	Write	Provide a binding name for the association between an application and a service instance		`modules:`
					`  - name: java`
					`  .........`
					`  requires:`
					`  - name:test`
					`  parameters:`
					`  binding-name: java-test`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`content-target`	Module-Required Dependency	Write	This parameter defines the target for the content deployment. It is usually a service instance or a service key toward the content backend that processes the content. The parameter is valid only for modules with direct content deployment:
			• `com.sap.application.content`
			• `com.sap.integration`
			• `com.sap.api.management`
			For more information, see Deploying Content with Generic Application Content Deployment [page 770].
				false	`modules:`
					` - name: java`
					` .........`
					` requires:`
					`  - name: test`
					`  parameters:`
					`  content-target: true`

`delete-service-key-after-deployment`	Module-Required Dependency	Write	If this parameter is set to true, the service keys used during the content deployment are deleted once the deployment is completed. The parameter is valid only for modules with direct content deployment:
			• `com.sap.application.content`
			• `com.sap.integration`
			• `com.sap.api.management`
				false	`modules:`
					` - name: java`
					` .........`
					` requires:`
					`  - name: test`
					`  parameters:`
					`  delete-service-key-after-deployment: true`

`env-var-name`	Module-Required Dependency	Write	Used when consuming an existing service key. Specifies the name of the environment variable that will contain the service key's credentials. See Consumption of existing service keys for more information.	The name of the service key.	`env-var-name: SERVICE_KEY_CREDENTIALS`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`visibility`	Module-Provided Dependency	Write	Specifies the organizations and spaces in which public provided dependencies are visible. See Visibility of cross-MTA configuration for more information.	In all spaces of the current organization.
				`visibility:`
				`  - org: ${org}`
				`  space: "*"`
					`visibility:`
					`  - org: foo`
					`  space: "*"`
					`  - org: bar`
					`  space: "*"`
					`  - org: baz`
					`  space: qux`

`use-live-routes`	Module-Provided Dependency	Write	Valid for blue-green deployment. Specify this parameter if you want to provide the routes specified in the descriptor.	false	`provides:`
					`  - name: example-name`
					`  properties:`
					`  provided-route: ${routes/0/route}`
					`  parameters:`
					`  use-live-routes: true`

As an alternative, you can also externalize such configurations in a file. See Service Instance Parameters [page 756].

Properties

MTA properties are Cloud Foundry application environment variables that are used during application runtime. When an MTA property is set at the module level, the SAP Cloud Deployment service injects its key as the environment variable key and its value as the variable value in the application, represented by the corresponding MTA module.

MTA properties can be declared at different levels — module level, resource level, and dependency level.

Cross-References to Properties

To enable resource properties to resolve values from a property in another resource or module, a resource must declare a dependency. However, these “requires” declarations do not affect the order of the application deployment.

> ⚠ Restriction
>
> It is not possible to reference **list** configuration entries either from resources or “subscription” functionalities (deployment features that are available to subscriber applications).

Code Syntax

Cross-references between properties in the MTA deployment descriptor .yaml file

```
modules:
  - name: java
    ...
    provides:
      - name: backend
        properties:
          url: ${default-live-url}/foo
resources:
  - name: example
    type: example-type
    properties:
      example-prop: my-example-prop
  - name: uaa
    type: uaa-type
    requires:
      - name: example
      - name: backend
        properties:
          prop: ~{url}
        parameters:
          param: ~{url}
    properties:
      pro1: ~{example/example-prop}
    parameters:
      config:
        app-router-url: ~{backend/url}
        example-prop: ~{example/example-prop}
```

Note the following about the example above:

- The `java` module provides a dependency named `backend`, which exposes a `url` property. Its value is built from `${default-live-url}`, which always resolves to the live (non-idle) application URL. This is the recommended choice when the property is consumed by a resource, because resource parameters are resolved during the initial phase of deployment (before idle applications are started).
- The `uaa` resource requires `backend` and declares `prop: ~{url}` in the `properties` block of that dependency. This creates an environment variable named `prop` in the `uaa` resource whose value is taken from the `url` property provided by `backend`. This is independent of the `parameters` block in the same dependency, where `param: ~{url}` uses the same referenced value as a parameter input instead of as an environment variable.
- References to provided properties can also be used in resource parameter values, as shown in the `config` block of `uaa`, where both `~{backend/url}` and `~{example/example-prop}` use fully qualified cross-reference syntax.

Note

The choice between `${default-url}` and `${default-live-url}` in `provides` properties depends on who consumes the provided value:

- **Resource consumer**: Use `${default-live-url}`. Resource parameters are resolved during the initial phase of a blue-green deployment, before idle applications are started. Using `${default-url}` here causes the resource to target the idle URL (with the `-idle` suffix) and misdirect traffic.
- **Module consumer**: `${default-url}` can be the right choice if you want new modules to communicate with each other during the testing phase of a blue-green deployment. For example, if a backend module provides `${default-url}` and a frontend module consumes it, the idle frontend

will point to the idle backend during testing. At the end of the blue-green deployment, both resolve to the correct live URLs.

Fully qualified references to properties

The SAP Cloud Deployment service supports the extension of the standard syntax for references in module properties. This extension enables you to specify the name of the `requires` section inside the property reference. You can use this syntax extension to declare implicit groups, as illustrated in the following example:

Sample Code

Syntax Extension: Alternative Grouping of MTA Properties

```
modules:
  - name: pricing-ui
    type: javascript.nodejs
    properties:
      API: # equivalent to group, but defined in the module properties
       - key: internal1
         protocol: ~{price_opt/protocol} #reference to value of protocol
defined in price_opt of module pricing-backend
       - key: external
         url: ~{competitor_data/url} # reference to string value of property
'url' in required resource 'competitor_data'
         api_keys: ~{competitor_data/creds} # reference to list value of
property 'creds' in 'competitor_data'
    requires:
     - name: competitor_data
     - name: price_opt
  - name: pricing-backend
    type: java.tomcat
    provides:
     - name: price_opt
       properties:
          protocol: http ...
  resources:
    - name: competitor_data
      properties:
        url: "https://marketwatch.acme.com/"
        creds:
          app_key: 25892e17-80f6
          secret_key: cd171f7c-560d
```

Dynamic parameters

In some cases, it is required to use a value that is unknown before deployment. For example, such entity could be the GUID of another service instance. The GUID of a newly created service instance is not known before the creation of said service but it might be consumed during the MTA deployment.

The feature enables an existing Cloud Foundry entity parameter value to be used inside an MTA descriptor.

Note

Currently, the feature is limited to resolving service GUID for the following resource types:

- `org.cloudfoundry.managed-service`
- `org.cloudfoundry.user-provided-service`
- `org.cloudfoundry.existing-service`

Resolved parameters can only be referenced from other resources or from a cross-MTA dependency. If a dynamic parameter is referenced from other places, like modules, the reference won't be resolved.

Sample Code

MTA Deployment Descriptor (`mtad.yaml`)

```
_schema-version: 3
ID: sample-dynamic-service-guid
version: 1.0.0
modules:
  - name: sample-app
    type: staticfile
    path: content.zip
    requires:
      - name: test-service
      - name: hana-service
resources:
  - name: test-service
    type: org.cloudfoundry.user-provided-service
    parameters:
      config:
        service-guid-of-db: ~{hana-service/my-db-service-guid}
    requires:
      - name: hana-service
    processed-after: [hana-service]
  - name: hana-service
    type: org.cloudfoundry.managed-service
    parameters:
      service: hana
      service-plan: schema
    properties:
      my-db-service-guid: ${service-guid}
```

Note

In order to use this feature, it is necessary to specify `${service-guid}` inside the `properties` section of the resource. Consumer of service GUID value must add respective resource name in the `requires` section and specify the `processed-after` parameter because the order of resource processing must be explicitly described.

Dynamic parameter as a cross-MTA dependency

Resolved parameters can also be referenced from a cross-MTA dependency, allowing the service instance to be referenced from another MTA.

The following example shows the dependency declaration referencing a dynamic parameter in the deployment descriptor of the "provider" MTA:

Sample Code

```
_schema-version: 3
ID: dynamic-service-guid-provider
version: 1.0.0
modules:
```

```
 - name: app-provider
   type: staticfile
   path: appBits.zip
   provides:
      - name: db-guid
        public: true
        parameters:
           visibility:
           - org: ${org}
             space: ${space}
        properties:
            db-instanceid: ~{hana-service/my-db-service-guid}
   requires:
     - name: hana-service
resources:
  - name: hana-service
    type: org.cloudfoundry.managed-service
    parameters:
      service: hana
      service-plan: schema
    properties:
      my-db-service-guid: ${service-guid}
```

The following example shows the dependency declaration in the deployment descriptor of the “consumer” MTA:

Sample Code

```
_schema-version: 3
ID: dynamic-service-guid-consumer
version: 1.0.0
modules:
  - name: app-consumer
    type: staticfile
    path: content.zip
    requires:
      - name: db-config
        properties:
          reference_instance: ~{db-instanceid}
resources:
  - name: db-config
    type: configuration
    parameters:
      provider-id: "dynamic-service-guid-provider:db-guid"
      version: ">=1.0.0"
      target:
        org: ${org}
        space: ${space}
```

For more information about cross-MTA configurations see Cross-MTA Dependencies [page 806].

4.1.8.1.6.5 List of Supported MTA Parameters

A list of all the supported parameters for MTA deployment in the Cloud Foundry environment.

How the table works

This flexible table allows you to:

- Set the pagination length to certain predefined numbers of rows, defaulting to "all".
- Enter text in the *Search* box to filter out all rows that do not include that text in any column.
- Enter text in one or more "search within the column" boxes to filter out all rows that do not include those texts in those columns.
- For certain columns, click *Filter* to select from a dropdown list whether to display all rows, or only rows containing a selected text in that column.
- Sort the rows by certain columns by clicking on the cells in the table header (marked with an up or down arrow).
- Show or hide columns of choice by using the *Columns: <n> selected* dropdown.

The available columns are:

- Parameter - the name of the MTA parameter
- Scope - the scope of the MTA parameter. Parameters can have the following scope:
  - Module
  - Module Hook
  - Module-Required Dependency
  - Module-Provided Dependency
  - Resource
  - Top-Level
- Read-Only / Write:
  - Read-Only: The parameter can be referenced but its value cannot be changed.
  - Write: The value of the parameter can be defined or overwritten by the user.
- Description - a description explaining briefly how the parameter works
- Default Value - the default value of the parameter. Not all parameters have a default value.
- Example - an example showing how to use the parameter in the descriptor

> **Note**
>
> If you can't find a specific parameter from the native Cloud Foundry manifest here, refer to Prerequisites and Restrictions [page 618] to see which Cloud Foundry features are currently not supported.

Parameters for MTA Deployment

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`app-features`	Module	Write	Allows enabling, disabling, and querying specific features for individual applications. These features typically control behaviors or capabilities related to application execution.
			
			The parameter accepts a map as a value. All parameters in the map are passed directly to the CF API. This mechanism ensures that all future app features will be automatically supported through MTA deployment.
			
			The parameter is especially useful when many services are bound to an application, and all credentials inside the `VCAP_SERVICES` environment variable exceed the maximum size of 130 KB (see Cloud Foundry environment variables). In this case, the application cannot be staged successfully. To solve that issue, you can switch to
				n/a	`modules:`
					`- name: foo`
					`  type: application`
					
					`parameters:`
					`  app-features:`
					
					`file-based-vcap-services: true`
					
					`ssh: true`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
			the "file-based VCAP services" method by enabling the respective app feature, and adapt your application coding accordingly to use a file instead of the `VCAP_SERVICES` environment variable. See Delivering service credentials to an app . For more information about the supported features, see Supported app features . **Note** If you want to configure SSH enablement, use this parameter instead of the deprecated `enable-ssh` parameter.		

`app-name`	Module	Write	The name of the application in the Cloud Foundry environment to be deployed for this module, based on the module name	`${default-app-name}`	`node-hello-world`
					
					`com.sap.xs2.samples.xsjshelloworld.node-hello-world`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`apply-namespace`	Module	Write	Applies namespace to the application name. When you set `apply-namespace` to the application name and do not specify `apply-namespace` for its route, the namespace is applied to both the application name and its route. If the `namespace` value is not provided in the CLI options, it is not applied. For more information, see Fine-Grained Configuration [page 798].		`modules:`
					`- name: java`
					`...`
					`parameters:`
					`apply-namespace: true`

`buildpack`	Module	Write	The name or the URL of a custom buildpack required by the application	Empty, or as specified in the deploy service configuration	`buildpack: git://github.acme.com/xs2-java/xs2javabuildpack`
`buildpacks`	Module	Write	An array of buildpacks. If a buildpack parameter already exists, it will be overwritten by the buildpacks listed in the buildpacks parameter, so that you have to include it in the array.	Empty, or as specified in the deploy service configuration	`buildpacks: [java_buildpack, nodejs_buildpack, staticfile_buildpack]`
`command`	Module	Write	A custom command required to start the application	Empty, or as specified in the deploy service configuration	`command: node index.js`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`create-service-broker`	Module	Write	Specifies whether [true|false] a service broker should be registered for the application module	`false`	`create-service-broker: true`
`default-app-name`	Module	Read-Only	The name of the application in the Cloud Foundry environment to be deployed for this module, based on the module name	The module name with or without a namespace prefix	`node-hello-world` `com.sap.xs2.samples.xsjshelloworld.node-hello-world`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`default-host`	Module	Read-Only	The default host name, which is composed based on the module name to ensure uniqueness. Used with host-based routing to compose the default URI, see below. It follows the convention `${org}-${space}-<module_name>`. → Tip We recommend you explicitly use the `routes` parameter. See Routes [page 765]. ⓘ Note Host names that do not comply with the Cloud Foundry naming limitations are automatically corrected: • Any characters that are not within the allowed `a-z` and `0-9` ranges are replaced by dashes (-) to pre-	Generated as described in the description	`trial-a007007-node-hello-world`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
			vent deployment issues. • If the host name length exceeds 63 symbols, the name is shortened and its end is replaced with a hash code. **Note** When used in blue-green deployment the value is resolved depending on the phase. During the testing phase the "idle" suffix will be appended to the host. If you want to use the live version only, refer to `${default-live-host}`.		
`default-instances`	Module	Read-Only	The number of application instances that are started during the deployment	1	`default-instances: 1`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`default-live-app-name`	Module	Read-Only	Valid for blue-green deployment. Specify this parameter if you want the name of the Cloud Foundry application that is to be deployed with this module to be based on the name of the module. For standard deployment the parameter will be the same as `default-app-name` regardless the phase.	The module name with or without a namespace prefix	`node-hello-world`
`default-live-domain`	Module	Read-Only	Valid for blue-green deployment. The value of this parameter is the default domain for the current organization.		
`default-live-host`	Module	Read-Only	Valid for blue-green deployment. Specify this parameter if you want to use `${default-host}` without the "idle" suffix during the testing phase of the blue-green deployment.	Generated as described in the description	`trial-a007007-node-hello-world`
`default-live-uri`	Module	Read-Only	Valid for blue-green deployment. Specify this parameter if you want to use `${default-uri}` without the "idle" suffix during the testing phase of the blue-green deployment.	Generated as described in the description	`trial-a007007-node-hello-world.cfapps.acme.ondemand.com`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`default-live-url`	Module	Read-Only	Valid for blue-green deployment. Specify this parameter if you want to use `${default-url}` without the "idle" suffix during the testing phase of the blue-green deployment.	Generated as described in the description	`${protocol}://${default-live-uri}`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`default-uri`	Module	Read-Only	The default URI, composed as `${host}.${domain}` (host-based routing). Note that `${host}` will be the same as `${default-host}`, unless specified explicitly as a parameter. Similarly, `${domain}` would be the same as `${default-domain}`, unless specified explicitly. **Note** When used in blue-green deployment the value is resolved depending on the phase. During the testing phase the "idle" suffix will be appended to the host of the URI. If you want to use the live version only, refer to `${default-live-uri}`.	Generated as described in the description.	`trial-a007007-node-hello-world.cfapps.acme.ondemand.com`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`default-url`	Module	Read-Only	The default URL, composed as `${protocol}://${default-uri}`. Note that the `${default-uri}` placeholder is resolved as `${host}.${domain}` (host-based routing) **Note** When used in blue-green deployment the value is resolved depending on the phase. During the testing phase the "idle" suffix will be appended to the host of the URL. If you want to use the live version only, refer to `${default-live-url}`.	Generated as described in the description.	`${protocol}://${default-uri}`

`dependency-type`	Module	Write	Deployment order of modules with circular dependencies	soft	`dependency-type: hard`
					
					`dependency-type: soft`

`disk-quota`	Module	Write	The disk space that will be available to the application. This parameter requires a unit of measurement M, MB, G, or GB in upper or lower case.	1, or as specified in module-type	`disk-quota: 1G`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`docker`	Module	Write	Creates a module from a docker image. When using a docker image parameter, we do not need to specify the module in the `MANIFEST.mf` file. An image parameter is a docker image from the Docker Hub or somewhere else. The username and the password are optional, but if a Docker image from a private repository is uploaded, then they are mandatory.
			
			When uploading a docker image, the content of a module is not needed.
				n/a	Sample Code
					`modules:`
					`name: foo`
					`type: application`
					`parameters:`
					`docker:`
					`image: cloudfoundry/test-app`
					`username: <optional username>`
					`password: <optional password>`

`domain`	Module	Write	The domain on which the application is available later	`${default-domain}`	`domain: ${default-domain}.acme.com`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`domains`	Module	Write	The domains on which the application is available later. The resulting application routes are the Cartesian product of the domains and hosts. That is, a separate route for each host is constructed on each domain.	`domains: - ${default}`	`domains: - ${default-domain}.acme.com - test-${default-domain}.acme.com`

`enable-ssh`	Module	Write	Enables use of SSH within an application. Supported for the Diego container runtime environment only.
			
			Note
			This parameter is deprecated. To enable SSH within an application, use the `app-features` parameter instead.
				n/a	`"enable-ssh": true`
					
					`"enable-ssh": false`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`enable-parallel-service-bindings`	Module	Write	Enables or disables the parallel binding or unbinding of services during deployment. If disabled, the services are bound and unbound sequentially in the order provided in the deployment descriptor.	`true`	Sample Code
					`- name: <module name>`
					`type: <module type>`
					`parameters:`
					`enable-parallel-service-bindings: false`

`health-check-http-endpoint`	Module	Write	If the `health-check-type` parameter is set to `http`, the controller will do a `GET` request to this endpoint. The application will be considered as healthy if the response is 200 OK.	If `health-check-type` is set to `http`, the default value is `/`, otherwise there is no default value.	`health-check-type: http` `health-check-http-endpoint: /health`
`health-check-timeout`	Module	Write	The application health check timeout in seconds	n/a	`health-check-timeout:120`
`health-check-invocation-timeout`	Module	Write	The time period in seconds, within which the application health check should be automatically started. If the health check does not start within the defined time period, it is omitted, and the deployment process continues.	n/a	`health-check-invocation-timeout:16`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`health-check-type`	Module	Write	The application health check type	`port`	`health-check-type: port`
					`health-check-type: http`
					`health-check-type: process`

`health-check-interval`	Module	Write	Specifies the interval, in seconds, between successive liveness health check invocations that the Cloud Controller performs against the application.	*n/a*	`health-check-interval: 15`
`host`	Module	Write	The hostname or subdomain where an application is available later. If you want to create a wildcard hostname, use an asterisk in quotes ("*").	`${default-host}`	`host: ${space}-node-hello-world` `host: "*"`
`hosts`	Module	Write	The hostnames or subdomain where an application is available later. If you want to create a wildcard hostname, use an asterisk in quotes ("*").	`hosts: - ${host}`	`modules: - name: my-app type: application parameters: hosts: - ${space}-node-hello-world - "*"`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`idle-domain`	Module	Write	Valid for a blue-green deployment when a new application version is started on a temporary route.
			
			Specify this parameter if you want to use another domain for temporary routes.
				`${default-domain}`	`modules:`
					`  - name: app`
					`  type: nodejs`
					
					`parameters:`
					
					`idle-domain: "<some.domain>"`

`idle-domains`	Module	Write	Valid for a blue-green deployment when a new application version is started on several temporary routes.
			
			Specify this parameter if you want to use other domains for temporary routes by listing them in an array.
				`${default-domain}`	`modules:`
					`  - name: app`
					`  type: nodejs`
					
					`parameters:`
					
					`idle-domains: ["<some.domain>", "<some.other.domain>"]`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`idle-host`	Module	Write	Valid for a blue-green deployment when a new application version is started on a temporary route.
			
			Specify this parameter if you want to use another host for a temporary route.
			
			Note
			The new application will start on a route comprised of the specified host and the default domain.
				`${default-host}-idle`	`modules:`
					`  - name: app`
					`    type: nodejs`
					
					`parameters:`
					
					`idle-host: "<some-hostname>"`

`idle-hosts`	Module	Write	Valid for a blue-green deployment when a new application version is started on temporary routes.
			
			Specify this parameter if you want to use other hosts for temporary routes by listing them in an array.
			
			Note
			The new application will start on routes comprised of the specified hosts and default domain.
				`${default-host}-idle`	`modules:`
					`  - name: app`
					`    type: nodejs`
					
					`parameters:`
					
					`idle-hosts: ["<some-hostname>", "<some-other-hostname>"]`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`idle-routes`	Module	Write	Valid for a blue-green deployment when a new application version is started on temporary routes.
			
			Specify this parameter if you want to use other routes for the application.
				`${default-uri}-idle`	`modules:`
					`- name: app`
					
					`parameters:`
					`  idle-routes:`
					`  - idle-route: "<your-first-idle-hostname.your.first.idle.domain>"`
					`  - idle-route: "<your-second-idle-hostname.your.second.idle.domain>"`

`instances`	Module	Write	The number of application instances that will be started during the deployment	`${default-instances}`	`instances: 2`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`keep-existing`	Module	Write	Defines the application attributes which will be kept after the deployment or blue-green deployment has finished. The supported attributes which could be kept are application environment, application bindings and application routes. If not specified, the default values are false, which indicates that each application attribute will be updated with the new values presented in the deployment descriptor.	`false`	Sample Code
					`keep-existing:`
					`  env: true`
					`service-bindings: true`
					`routes: false`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`keep-existing-routes`	Module	Write	When specified on module level, it indicates if the existing routes of the module's corresponding application should be kept even if they are not defined within the deployment and/or extension descriptors.
			
			When specified on top level, under the `parameters` section of the descriptor, it indicates if the existing routes of all applications within that MTA should be kept.
			
			Note
			• The module-level variant of the parameter has priority over the top-level parameter.
			• This parameter is typically used when users want to keep the routes they have mapped manually by using
				`false`	**Sample Code**
					`parameters:`
					`  keep-existing-routes: true`
					`modules:`
					`  - name: foo`
					`  type: nodejs`
					`parameters:`
					`keep-existing-routes: false`
					`  - name: bar`
					`  type: nodejs`
					`  - name: baz`
					`  type: nodejs`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
			the `cf map-route` command. We discourage this approach, as manual operations could lead to inconsistent deployment results and difficult troubleshooting. We recommend you to define all routes in the deployment and/or extension descriptors, which allows for their automatic management.		
`memory`	Module	Write	The memory limit for all instances of an application. This parameter requires a unit of measurement M, MB, G, or GB in upper or lower case.	256M, or as specified in module-type	`memory: 128M`
`no-route`	Module	Write	Configures the deployer to create or skip the creation of a route for the application described by the module	false	`no-route: true`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`no-start`	Module	Write	Start/do not start the application during deployment. **→ Tip** This parameter setting overrides the command-line option `--no-start`. If you explicitly set the `no-start` to `false` for the module `foo` in the example provided, then the module `foo` **is** started on deployment, even if you also specify the command-line option `--no-start` with the `cf deploy` command.	Depends on the command line option `--no-start`	`no-start: true`

`readiness-health-check-http-endpoint`	Module	Write	The endpoint called to determine if the app is ready. The parameter can be applied if the `readiness-health-check-http-endpoint` parameter is set to `http`. The controller will make a `GET` request to this endpoint, and the application will be considered healthy if the response is 200 OK.	n/a	**{·} Sample Code**
					`readiness-health-check-type: http`
					`readiness-health-check-http-endpoint: /health`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`readiness-health-check-type`	Module	Write	The type of health check to be performed on the module: `http`, `port`, or `process`	`process`	`readiness-health-check-type: port`
					`readiness-health-check-type: http`
					`readiness-health-check-type: process`

`readiness-health-check-invocation-timeout`	Module	Write	The timeout in seconds for individual readiness health check requests. The parameter can be used for the `http` and `port` health check types.	n/a	`readiness-health-check-invocation-timeout: 20`
`readiness-health-check-interval`	Module	Write	The amount of time in seconds between individual readiness health check requests	n/a	`readiness-health-check-interval: 20`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`restart-on-env-change`	Module	Write	Specifies whether an application should be re-started if an environment variable has been changed in one of the following categories:
			• vcap-application
			• vcap-services
			• user-provided
			Note
			If you set these parameters to `false`, the changes in environment are not consumable by a running instances of the application. If your application depends on the latest environment, it might become outdated.
				Sample Code
				`restart-on-env-change:`
				`  vcap-application: true`
				`  vcap-services: true`
				`  user-provided: true`
					Sample Code
					`restart-on-env-change:`
					`  vcap-application: false`
					`  vcap-services: true`
					`  user-provided: true`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`routes`	Module	Write	A parameter that lists multiple HTTP routes. For more information, see Routes [page 765].	`${default-uri}`	Sample Code
					`modules:`
					`- name: my-app`
					`type: application`
					`parameters:`
					`routes:`
					`- route: "*foo.my.custom.domain/path"`
					`- route: foo.my.custom.domain/path`
					`- route: foo.${default-domain}/path`

`route-path`	Module	Write	The context “route-path” which is part of the application default URI. Context path routing is routing based not only on domain names (host header) but also the path specified in the URL.	n/a	`route-path: /myapp`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`service-broker-name`	Module	Write	The name of the service broker in the Cloud Foundry environment to be created and registered for the specified application module	`${app-name}`	`service-broker-name: jobscheduler` `service-broker-name: ${app-name}`
`service-broker-password`	Module	Write	The password used for authentication by the Cloud Controller at the service broker when performing service-related requests. The parameter is mandatory if `create-service-broker: true`.		`service-broker-password: ${generated-password}`
`service-broker-space-scoped`	Module	Write	Makes the service plans of the broker visible only within the targeted space.	`false`	`service-broker-space-scoped: true`
`service-broker-url`	Module	Write	Specifies the value of the service broker universal resource locator (URL) to register; service requests are sent to this URL. The parameter is mandatory if `create-service-broker: true`.		`service-broker-url: ${default-url}`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`service-broker-user`	Module	Write	The name of the user required for authentication by the Cloud Controller at the service broker when performing service-related requests. The parameter is mandatory if `create-service-broker: true`.		`service-broker-user: ${generated-user}`
`skip-deploy`	Module	Write	Use this parameter to skip the deployment of a specified module even if it is present in the descriptor. If you add this parameter for an already deployed module, this module will be deleted in the next operation.	`false`	Sample Code `modules: - name: foo type: application parameters: skip-deploy: true`
`stack`	Module	Write	Use this parameter to define which prebuilt root file system (`rootfs`) you want to use.	n/a	Sample Code `modules: - name: foo type: application parameters: stack: cflinuxfs3`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`stage-timeout`	Module	Write	Defines how long, in seconds, your application can take during staging before the MTA operation times out.
			
			See Application-Specific Timeouts [page 750].
				1h	`modules:`
					`  - name: java`
					`  .....`
					`....`
					`parameters:`
					`  stage-timeout: 100`

`start-timeout`	Module	Write	Defines how long, in seconds, your application can take to start before the MTA operation times out.
			
			See Application-Specific Timeouts [page 750].
				1h	`modules:`
					`  - name: java`
					`  .....`
					`....`
					`parameters:`
					`  start-timeout: 100`

`task-execution-timeout`	Module	Write	Defines how long, in seconds, your application can take to execute a task before the MTA operation times out.
			
			See Application-Specific Timeouts [page 750].
				12h	`modules:`
					`  - name: java`
					`  .....`
					`....`
					`parameters:`
					`  task-execution-timeout: 100`

`tasks`	Module	Write	Specify tasks, which are available for execution in the current droplet of the application. Also provide use of environment variables which are specified with the `env` scope.	n/a	`tasks:`
					` - name: task-1`
					`command: some-script.sh`
					`env:`
					`  env1: value1`
					`  env2: value2`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`upload-timeout`	Module	Write	Defines how long, in seconds, you can upload your application binary before the MTA operation times out.
			See Application-Specific Timeouts [page 750] .
				1h	`modules:`
					`  -`
					`name: java`
					`  .....`
					`.....`
					`parameters:`
					`upload-timeout: 100`

`timestamp`	Module	Read-Only	Current timestamp in milliseconds	Generated as described in the description.	

`apply-namespace`	Resource	Write	Applies namespace to the service name. If the namespace value is not provided in the CLI options, it is not applied. For more information, see Fine-Grained Configuration [page 798].		`resources:`
					`- name: java`
					`  ...`
					`parameters:`
					`  apply-namespace: true`

`config`	Resource	Write	Defines service creation parameters. More information in Service Instance Parameters [page 756].	n/a	See Service Instance Parameters [page 756].

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`default-container-name`	Resource	Read-Only	Default value for the `container-name` parameter that is used during HDI creation. It is based on the organization, space and resource name, which are combined in a way that conforms to the container-name restrictions for length and legal characters. All dash (-) symbols are replaced with an underscore (_). **Note** It is highly recommended to avoid using this parameter with HANA services because in some cases the generated name cannot comply with the HANA naming limitations' requirements. Forward compatability is not guaranteed.	Generated as described in the description.	`<org_name>_<space_name>_<resource_name>`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`default-service-name`	Resource	Read-Only	The name of the service in the Cloud Foundry environment to be created for this resource, based on the resource name with or without a name-space prefix.	The resource name with or without a name-space prefix	`nodejs-hdi-container` `com.sap.xs2.samples.xsjshelloworld.nodejs-hdi-container`
`default-xsappname`	Resource	Read-Only	Default value for the `xsappname` parameter that is used during UAA creation. It is based on the service name, which is modified in a way that conforms to the `xsappname` restrictions for length and legal characters.	Generated as described in the description.	`xs_-deploy-service-database` (if the service name is "`xs@-deploy-service-database`")

`destination`	Resource	Write	Map value specifying the organization and space in which the SAP Cloud Logging instance resides. Applies to a resource of type `org.cloudfoundry.cloud-logging-service`. Contains the `org-name` and `space-name` parameters. By default, the organization and space of the current deployment are used. For more information, see (Experimental) Exporting MTA Operation Logs to SAP Cloud Logging [page 823].	The organization and space of the current deployment	`destination:`
					`  org-name: my-org`
					`  space-name: my-space`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`fail-on-service-update`	Resource	Write	When this parameter is set to true, the deployment will fail on every service update failure. By default, service updates are fail-safe for asynchronous calls (Cloud Foundry API calls that result in a call to the service broker).
			
			When the parameter is set to false, deployments will not fail on service update failures, as the updates will be made fail-safe even for calls that do not result in a call to the respective service broker.
			
			For more information, see:
			
			• Updating Service Plans [page 755]
			• Updating Service Instance Parameters [page 758]
			• Updating Service Tags [page 761]
				n/a (fails only on asynchronous calls)	`fail-on-service-update:`
					
					`parameters: true`
					`  plan: true`
					`  tags: true`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`log-level`	Resource	Write	Verbosity of the operation logs that are exported to the SAP Cloud Logging instance. Applies to a resource of type `org.cloudfoundry.cloud-logging-service`. Supported values are INFO, WARN, DEBUG, ERROR, and TRACE. The values are hierarchical. For more information, see (Experimental) Exporting MTA Operation Logs to SAP Cloud Logging [page 823].	INFO	`log-level: DEBUG`
`org-name`	Resource	Write	Name of the organization in which the SAP Cloud Logging instance resides. Specified within the destination parameter of a resource of type `org.cloudfoundry.cloud-logging-service`. For more information, see (Experimental) Exporting MTA Operation Logs to SAP Cloud Logging [page 823].	The organization of the current deployment	`org-name: my-org`
`service`	Resource	Write	The type of the created service	Empty, or as specified in resource-type	`service: hana`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`service-key-name`	Resource	Write	Used when consuming an existing service key. Specifies the name of the service key. See Consumption of Service Keys [page 764] for more information.	The name of the resource.	`service-key-name: my-service-key`
`service-name`	Resource	Write	The name of the service in the Cloud Foundry environment to be created for this resource, based on the resource name with or without a name-space prefix. **Note** Service names that do not comply with the Cloud Foundry limitation of 50 symbols are automatically corrected. In such cases, the name is shortened and its end is replaced with a hash code.	`${default-service-name}`	`nodejs-hdi-container` `com.sap.xs2.samples.xsjshelloworld.nodejs-hdi-container`
`service-plan`	Resource	Write	The plan of the created service	Empty, or as specified in resource-type	`service-plan: hdi-shared`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`service-tags`	Resource	Write	Some services employ a list of custom tags, which provide an easier way for applications to parse <VCAP_SERVICES> for credentials. You can provide custom tags when creating a service instance. For more information, see Service Tags [page 761].	n/a	`resources:`
					`  - name: nodejs-uaa`
					`  type: com.sap.xs.uaa`
					
					`parameters:`
					
					`service-tags: ["custom-tag-A", "custom-tag-B"]`

`service-broker`	Resource	Write	Use this parameter to specify the service broker you want to employ when you create your service. This can be useful for testing purposes, among others.	Name of service broker you want to be used.	`resources:`
					`  - name: test-service`
					`  type: org.cloudfoundry.managed-service`
					
					`parameters:`
					
					`service: foo`
					
					`service-plan: bar-a`
					
					`service-broker: test-broker-1`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`skip-service-updates`	Resource	Write	This parameter allows you to specify the service configuration changes to `ignore`, when deciding if you want to update a service instance – in particular changes of service `parameters`, `plan`, or `tags`.	The default value for all is `false`. The service instance would be updated on change in any of the configurations.	`resources:`
					`  - name: service-name`
					`    type: org.cloudfoundry.managed-service`
					
					`parameters:`
					
					`skip-service-updates:`
					
					`plan: true`
					
					`parameters: true`
					
					`tags: true`
					
					In the example above, `skip-service-updates` modifies the update strategy as follows:
					• allows users to specify which configurations should not be updated
					
					Note that these 3 key-value pairs can be in any order.

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`space-name`	Resource	Write	Name of the space in which the SAP Cloud Logging instance resides. Specified within the destination parameter of a resource of type `org.cloudfoundry.cloud-logging-service`. For more information, see (Experimental) Exporting MTA Operation Logs to SAP Cloud Logging [page 823].	The space of the current deployment	`space-name: my-space`

`syslog-drain-url`	Resource	Write	The URL to which logs for bound applications are streamed.
			Note
			This feature only works for user-provided services.
				n/a	`resources:`
					`  - name: service-name`
					`  type: org.cloudfoundry.user-provided-service`
					`parameters:`
					`syslog-drain-url: syslog://example.log-aggregator.com`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`apply-namespace:`
`  app-names: true/false`
`  service-names: true/false`
`  app-routes: true/false`
`  as-suffix: true/false`
	Top-Level	Write	Applies namespace to application names (app-names), service names (service-names), application routes (app-routes) as suffix or prefix (as-suffix). If the namespace value is not provided in the CLI options, it is not applied.
			See Fine-Grained Configuration [page 798].
			Note
			This applies to all applications.
				true	`parameters:`
					`  apply-namespace:`
					`  app-names: true`
					`  service-names: true`
					`  app-routes: false`
					`  as-suffix: true`

`apps-stage-timeout`	Top-Level	Write	Defines how long, in seconds, your application can take during staging before the MTA operation times out. See Application-Specific Timeouts [page 750] . **Note** This applies to all applications.	1h	`parameters:` `  apps-stage-timeout: 100`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`apps-start-timeout`	Top-Level	Write	Defines how long, in seconds, your application can take to start before the MTA operation times out. See Application-Specific Timeouts [page 750] . **Note** This applies to all applications.	1h	`parameters:` `  apps-start-timeout: 100`
`apps-task-execution-timeout`	Top-Level	Write	Defines how long, in seconds, your application can take to execute a task before the MTA operation times out. See Application-Specific Timeouts [page 750] . **Note** This applies to all applications.	12h	`parameters:` `  apps-task-execution-timeout: 100`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`apps-upload-timeout`	Top-Level	Write	Defines how long, in seconds, you can upload your application binary before the MTA operation times out. See Application-Specific Timeouts [page 750] . **Note** This applies to all applications.	1h	`parameters:` `  apps-upload-timeout: 100`
`authorization-url`	Top-Level	Read-Only	The authorization URL as specified in the cloud controller's `/v2/info` endpoint.	Generated as described in the description.	`https://login.cf.eu10-004.hana.ondemand.com`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`bg-dependency-aware-stop-order` **Note** This parameter is currently experimental.	Top-Level	Write	Ensures that during the final phase of a blue-green deployment, idle applications are stopped and restarted in a dependency-aware order based on the dependencies defined in the `deployed-after` attribute. This prevents transient failures caused by short-lived unavailability of dependent modules. First, dependent modules are stopped, then their dependencies are restarted, and finally the dependent modules are updated and started. **Note** When both are specified, the command-line option `--dependency-aware-stop-order` overrides the `bg-dependency-aware-stop-order` descriptor parameter.	false	`parameters:` `  bg-dependency-aware-stop-order: true`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`controller-url`	Top-Level	Read-Only	The URL of the cloud controller	Generated as described in the description.	`https://api.cf.eu10-004.hana.ondemand.com`
`default-domain`	Top-Level	Read-Only	The default domain of the target organization (configured in the Cloud Foundry environment). The value is coming from CF API endpoint Get default domain, and normally, it depends on the shared domains in the current organization.	Generated as described in the description.	**Example** If MTA deployment is done in a target org/space with CF API endpoint that is equal to `api.eu10-004.hana.ondemand.com`, the value of `default-domain` would be `cfapps.eu10-004.hana.ondemand.com`. **Example** If MTA deployment is done in a target org/space with CF API endpoint that is equal to `api.eu10.hana.ondemand.com`, the value of `default-domain` would be `cfapps.eu10.hana.ondemand.com`.

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`region-domain`	Top-Level	Read-Only	The domain of the current region. For more information, refer to the **Domain** column in Regions and API Endpoints Available for the Cloud Foundry Environment [page 19]. When a region has multiple CF API endpoints, it will point to the domain of the respective endpoint used for the deployment. **Note** The parameters `region-primary-domain` and `region-domain` are equal when used in most of the regions. They are different when MTA deployments are done in a target org/space in different CF APIs in one region.	Generated as described in the description.	**Example** If MTA deployment is done in a target org/space with CF API endpoint that is equal to `api.eu10.hana.ondemand.com`, the value of `region-domain` would be `eu10.hana.ondemand.com`. **Example** If MTA deployment is done in a target org/space with CF API endpoint that is equal to `api.eu10-004.hana.ondemand.com`, the value of `region-domain` would be `eu10-004.hana.ondemand.com`.

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`region-primary-domain`	Top-Level	Read-Only	The primary domain of a region. For more information, refer to the **Domain** column in Regions and API Endpoints Available for the Cloud Foundry Environment [page 19]. When a region has multiple CF API endpoints, it will point to the primary domain (the first one). **Note** The parameters `region-primary-domain` and `region-domain` are equal when used in most of the regions. They are different when MTA deployments are done in a target org/space in different CF APIs in one region.	Generated as described in the description.	**Example** If MTA deployment is done in a target org/space with CF API endpoint that is equal to `api.eu10.hana.ondemand.com`, the value of `region-primary-domain` would be `eu10.hana.ondemand.com`. **Example** If MTA deployment is done in a target org/space with CF API endpoint that is equal to `api.eu10-004.hana.ondemand.com`, the value of `region-primary-domain` would be `eu10.hana.ondemand.com`.
`deploy-url`	Top-Level	Read-Only	The deploy service URL for the Cloud Foundry environment	Generated as described in the description.	

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`enable-parallel-deployments`	Top-Level	Write	Defines if parallel deployment of modules is enabled. When enabled, modules without explicitly set `deployed-after` parameter are deployed in parallel. For more information, see Parallel Module Deployment [page 803].	no	`parameters:` `  enable-parallel-deployments: true`
`generated-password`	Top-Level	Read-Only	Randomly generated string value that is composed of 16 characters that may contain upper and lower case letters, digits and special characters (_, -, @, $, &, #, *).	Generated as described in the description.	`IG@zGg#2g-cvMvsW`
`generated-user`	Top-Level	Read-Only	A generated user id that is composed of 16 characters that may contain upper and lower case letters, digits and special characters (_, -, @, $, &, #, *).	Generated as described in the description.	`uYi$d41TzM1-Dm6f`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`keep-existing-routes`	Top-Level	Write	When specified on module level, it indicates if the existing routes of the module's corresponding application should be kept even if they are not defined within the deployment and/or extension descriptors.
			
			When specified on top level, under the `parameters` section of the descriptor, it indicates if the existing routes of all applications within that MTA should be kept.
			
			Note
			• The module-level variant of the parameter has priority over the top-level parameter.
			• This parameter is typically used when users want to keep the routes they have mapped manually by using
				`false`	`parameters:`
					`  keep-existing-routes: true`
					`modules:`
					`  - name: foo`
					`  type: nodejs`
					
					`parameters:`
					
					`keep-existing-routes: false`
					`  - name: bar`
					`  type: nodejs`
					`  - name: baz`
					`  type: nodejs`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
			the `cf map-route` command. We discourage this approach, as manual operations could lead to inconsistent deployment results and difficult troubleshooting. We recommend you to define all routes in the deployment and/or extension descriptors, which allows for their automatic management.		
`org`	Top-Level	Read-Only	Name of the target organization	The current name of the target organization	`initial`, `trial`
`protocol`	Top-Level	Read-Only	The protocol used by the Cloud Foundry environment.	`http` or `https`	`http, https`
`space`	Top-Level	Read-Only	Name of the target organizational space	Generated as described in the description.	`initial`, `a007007`
`user`	Top-Level	Read-Only	Name of the current user	Generated as described in the description.	

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`xs-type`	Top-Level	Read-Only	The XS type, Cloud Foundry or XS advanced	CF	`CF, XSA`
`org-guid`	Top-Level	Read-Only	GUID (Globally Unique Identifier) of the target organization	N/A	`06564ad5-1b38-458d-8c85-a2e0bcd990a9`
`space-guid`	Top-Level	Read-Only	GUID (Globally Unique Identifier) of the target space	N/A	`06564ad5-1b38-458d-8c85-a2e0bcd990a9`
`mta-version`	Top-Level	Write	The version of the MTA	N/A	`1.0.0`
`mta-id`	Top-Level	Write	The ID of the MTA	N/A	`com.sap.sample`

`binding-name`	Module-Required Dependency	Write	Provide a binding name for the association between an application and a service instance		`modules:`
					`  - name:`
					`java`
					`  .........`
					`  requires:`
					`  -`
					`name:test`
					
					`parameters`
					`:`
					
					`binding-`
					`name:`
					`java-test`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`content-target`	Module-Required Dependency	Write	This parameter defines the target for the content deployment. It is usually a service instance or a service key toward the content backend that processes the content. The parameter is valid only for modules with direct content deployment:
			• `com.sap.application.content`
			• `com.sap.integration`
			• `com.sap.api.management`
			For more information, see Deploying Content with Generic Application Content Deployment [page 770].
				false	`modules:`
					`  - name: java`
					`  .........`
					`  requires:`
					`  - name: test`
					
					`parameters:`
					
					`content-target: true`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`delete-service-key-after-deployment`	Module-Required Dependency	Write	If this parameter is set to true, the service keys used during the content deployment are deleted once the deployment is completed. The parameter is valid only for modules with direct content deployment:
			• `com.sap.application.content`
			• `com.sap.integration`
			• `com.sap.api.management`
				false	`modules:`
					`  - name: java`
					`  .........`
					`  requires:`
					`  - name: test`
					
					`parameters:`
					
					`delete-service-key-after-deployment: true`

`env-var-name`	Module-Required Dependency	Write	Used when consuming an existing service key. Specifies the name of the environment variable that will contain the service key's credentials. See Consumption of existing service keys for more information.	The name of the service key.	`env-var-name: SERVICE_KEY_CREDENTIALS`

`visibility`	Module-Provided Dependency	Write	Specifies the organizations and spaces in which public provided dependencies are visible. See Visibility of cross-MTA configuration for more information.	In all spaces of the current organization.
				`visibility:`
				`  - org: ${org}`
				
				`space: "*"`
					`visibility:`
					`  - org: foo`
					
					`space: "*"`
					`  - org: bar`
					
					`space: "*"`
					`  - org: baz`
					
					`space: qux`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`use-live-routes`	Module-Provided Dependency	Write	Valid for blue-green deployment. Specify this parameter if you want to provide the routes specified in the descriptor.	false	`provides:`
					`  - name: example-name`
					`properties:`
					`provided-route: ${routes/0/route}`
					`parameters:`
					`  use-live-routes: true`

`name`	Module Hook	Write	Defines the name of the Cloud Foundry task that should be executed	The name of the hook	`name: db_migration`
`command` **Note** This is a mandatory parameter.	Module Hook	Write	Defines the actual command that is executed as a Cloud Foundry task		`command: "bin/rails db:migrate"`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example
`memory`	Module Hook	Write	Defines the memory that is available to the Cloud Foundry task	Landscape-specific value that is equal to the default application memory → Remember Do not rely on the default value, as it will probably be much higher than you need and may not fit into the limitations of your quota.	`memory: 256M` `memory: 1G`
`disk-quota`	Module Hook	Write	Defines the disk space that is available to the Cloud Foundry task	Landscape-specific value that is equal to the default application disk quota → Remember Do not rely on the default value, as it will probably be much higher than you need and may not fit into the limitations of your quota.	`disk-quota: 256M` `disk-quota: 1G`

Parameter	Scope	Read-Only / Write	Description	Default Value	Example

`phases-config`	Module Hook	Write	Overrides the default target application (`live` or `idle`) on which a hook task is executed for a specific blue-green deployment phase. Valid values for `target-app` are `live` and `idle`. For more information, see Overriding the Default Target App for Hook Execution [page 671].		`phases-config:`
					`  -`
					`phase: blue-green.application.after-stop.live`
					
					`target-app: idle`
					
					This hook will be executed on the idle application, after the stop of the live application.

4.1.8.1.6.6 Metadata for Properties and Parameters

It is possible to declare metadata for parameters and properties defined in the MTA deployment description, for example, using the "`parameters-metadata:`" or "`properties-metadata:`" keys, respectively; the mapping is based on the keys defined for a parameter or property.

You can specify if a property is required (`optional; false`) or can be modified (`overwritable: true`), as illustrated in the following (incomplete) example:

The `overwritable:` and `optional` keywords are intended for use in extension scenarios, for example, where a value for a parameter or property is supplied at deployment time and declared in a deployment-extension descriptor file (`myMTADeploymentExtension.mtaext`).

You can declare metadata for the parameters and properties that are already defined in the MTA deployment description. However, any parameters or properties defined in the `mtad.yaml` deployment descriptor with the metadata value `overwritable: false` cannot be overwritten by a value supplied from the extension descriptor. In this case, an error would occur in the deployment.

→ Tip

Parameters or properties can be declared as sensitive. Information about properties or parameters flagged as sensitive is not written as plain text in log files; it is masked, for example, using a string of asterisks (`********`). Note the `secret_token:` element in the example.

{·} Code Syntax

Metadata for MTA Deployment Parameters and Properties

```
modules:
```

```
  - name: frontend
    type: javascript.nodejs
    parameters:
      memory: 128M
      domain: ${default-domain}
    parameters-metadata:
      memory:
        optional: true
        overwritable: true
      domain:
        overwritable: false
    properties:
      secret_token: ${generated-password}
      backend_types:
        order_management: sap-erp
        data_warehouse: sap-bw
```

4.1.8.1.7 Defining MTA Extension Descriptors

MTA extension descriptors are YAML files that complement deployment descriptors by providing environment-specific or deployment-specific data such as scaling configurations and credentials. They enable deploying the same MTA archive with different configurations by adding or overwriting parameters and properties without modifying the original archive.

The multitarget application (MTA) extension descriptor has a structure similar to the one of the deployment descriptor, and you will normally find it placed outside of the MTA archive.

> ⓘ Note
>
> The format and available options within the extension descriptor may change with newer versions of the MTA specification. You must always specify the schema version option when defining an extension descriptor to inform the SAP BTP which MTA specification version should be used. Furthermore, the schema version used within the extension descriptor and the deployment descriptor should always be the same.

When you are deploying an MTA, you can provide several extension descriptors. The descriptors in the chain are considered in order, with each descriptor having a higher priority than the one it extends. Each extension descriptor is defined in a separate file with an extension `.mtaext`.

In the examples below, we have a deployment descriptor, which has already been defined, and several extension descriptors.

MTA deployment descriptor:

> ✤ Example

```
----MTA Deployment descriptor-----
_schema-version: '3.1'
ID: sample.app
version: 0.1.0

modules:
  - name: my-app
    type: application
    parameters:
```

```
      memory: 2GB
```

When an MTA with the above deployment descriptor is being deployed, the following steps are performed:

- Validates the deployment descriptor against the MTA specification version `3.1`. If it is not compliant, the deployment will fail at the beginning
- Defines an MTA module with type `application` that will end up into a Cloud Foundry application
- Defines a parameter `memory` that equals to 2GB
- .....
- Creates a Cloud Foundry application that runs with 2GB memory

The following is a basic example of an extension descriptor that adds and overwrites data:

Example

```
----prod.mtaext-----
_schema-version: '3.1'
ID: sample.app.prod
extends: sample.app

modules:
  - name: my-app
    parameter:
      memory: 4GB
      instances: 2
```

When the same MTA is deployed in combination with the above MTA extension descriptor, using the command **`cf deploy <mtar> -e prod.mtaext`**, the following steps are performed:

- Validates the deployment descriptor and the extension descriptor against the MTA specification version **3.1**
- Extends the deployment descriptor with an extension descriptor with ID `sample.app.prod` and merges the result:
  - Overwrites the value of the parameter `memory`. It becomes equals to 4GB
  - Adds a new parameter `instances` that equals to 2
- .....
- Creates a Cloud Foundry application which runs on 2 instances with 4GB memory each

In this scenario, the descriptor chain will consist of two descriptors. They will be arranged in this order: `sample.app` followed by `sample.app.prod`.

The following is an example of a second extension descriptor that builds upon the extension descriptor from the previous example:

Example

```
----largemtaext-----
_schema-version: '3.1'
ID: sample.app.large
extends: sample.app.prod

modules:
  - name: my-app
    parameter:
      instances: 5
      disk-quota: 10GB
```

When you deploy the same MTA in combination with the two MTA extension descriptors, using the command **`cf deploy <mtar> -e prod.mtaext,large.mtaext`**, the following steps are performed:

- Validates the deployment descriptor and the extension descriptors against the MTA specification version **3.1**
- Extends the deployment descriptor with an extension descriptor with ID `sample.app.prod` and merges the result:
  - Overwrites the value of the parameter `memory`. It becomes equals to 4GB
  - Adds a new parameter `instances` that equals to 2
- Extends the merged descriptor with an extension descriptor with ID `sample.app.large` and merges the result:
  - Overwrites the value of the parameter `instances`. It becomes equals to 5
  - Adds a new parameter `disk-quota` that equals to 10GB
- ...
- Creates a Cloud Foundry application, which runs on 5 instances, each with 4GB of memory and 10GB of disk space

In this scenario, the descriptor chain will consist of three descriptors: `sample.app`, followed by `sample.app.prod`, and then `sample.app.large`.

Extension descriptors can be used to add or override all data types of parameters and properties, following the specified rules.

- Simple data types, such as Boolean, integer and String, are overwritten directly
- Maps (objects) are overwritten and merged in depth

```
----MTA Deployment descriptor-----
ID: sample.postgre

resources:
  - name: my-postgre
    type: org.cloudfoundry.managed-service
    parameters:
      service-plan: standard
      service: postgresql-db
      config:
        memory: 4
        storage: 200
```

```
----prod-postgre.mtaext-----
ID: sample.postgre.dev
extends: sample.postgre

resources:
  - name: my-postgre
    parameters:
      config:
        storage: 10
```

When using a combination of the MTA deployment descriptor and MTA extension descriptor, the MTA deployment will result in the creation of a service instance with the service offering `postgresql-db`, plan **standard**, running on 4GB memory, and 10GB storage. This example demonstrates how the parameter `config`, a YAML map (JSON Object), is processed. The value from the extension descriptor is merged with the value from the deployment descriptor while retaining the element `memory`. Without using the extension

descriptor, the result of the MTA deployment would be a Postgre service instance with 4GB memory and 200GB storage.

Note that lists (arrays) are overwritten by replacing all elements. This means that if you want to preserve any of the elements from the MTA deployment descriptor you need to define them again in the extension descriptor.

```
----MTA Deployment descriptor-----
ID: sample.app

modules:
  - name: my-app
    type: application
    properties:
      my-list: [1,2]
```

```
----app-list.mtaext-----
ID: sample.app.ext
extends: sample.app

modules:
  - name: my-app
    type: application
    properties:
      my-list: [2,3]
```

When you use a combination of the MTA deployment descriptor and an MTA extension descriptor, the resulting MTA deployment will create an application with the environment variable `my-list` set to [2,3]. This example demonstrates how the parameter `my-list`, which is a YAML list (JSON Array), is processed. The value from the extension descriptor completely overwrites the value from the deployment descriptor without retaining element "1". If the extension descriptor is not used, the environment variable `my-list` would be [1,2].

In some cases, there are operation-level parameters (command-line arguments) that take the highest priority over all descriptors. See Application-Specific Timeouts [page 750].

Purpose and Scope of the Extension Descriptor

You can do the following using an extension descriptor:

- Add new data in properties and parameters on module level, resource level, `provides` and `requires` section level
- Overwrite existing data (in depth) in modules, resources, parameters, properties, `provides` and `requires` sections. This depends on the parameter or property metadata `overwritable`. See Metadata for Properties and Parameters [page 744].
- As of schema version 3.xx, by default parameters and properties are overwritable and optional. If you want to make a certain parameter or property non-overwritable or required, you need to add specific metadata. See Metadata for Properties and Parameters [page 744].

You cannot use an extension descriptor to:

- Add new entities such as modules or resources
- Change module or resource type
- Alter read-only (system) parameters
- Add new provided or required dependencies

- Change the processing order of modules and resources with `deployed-after` and `processed-after` parameters

Related Information

Defining Multitarget Application Archives [page 622]
The Multitarget Application Model v.2
The Multitarget Application Model v.3

4.1.8.1.8 SAP Business Technology Platform Capabilities

This section contains information about how to manage standard Cloud Foundry entities with MTA modelling.

- Applications [page 749]
- Tasks [page 753]
- Services [page 754]
- Routes [page 765]

4.1.8.1.8.1 Applications

If you want to create an application in Cloud Foundry and use the SAP Cloud Deployment service, you must create an MTA module first.

In most of cases, an MTA module represents a Cloud Foundry application, but there are cases where they represent an entirely different type of entity, for example content. Modules can have different types and parameters that modify their behavior. For more information, see Modules [page 643].

> ⓘ Note
>
> We strongly recommend that you specify the appropriate buildpack for your application. This ensures that the application is handled by the buildpack for which it is intended and tested. It also speeds up application deployment, because only the specified buildpack is downloaded for use during the staging process.
>
> You have to use the `buildpack` module parameter, if you are using any of the following MTA module types:
>
> - `application`
> - `custom`
> - `javascript.nodejs`

MTA Deployment Optimizations

The MTA deployment is an incremental process. This means that the state of the artifacts in the Cloud Foundry environment is compared to the desired state described in the `mtad.yaml`. If there's a difference, a set of

operations are computed in order to make the MTA reach the desired state. The following optimizations are possible during the MTA redeployment:

- The application bits are uploaded again if the following is true:
  - The application content is changed and there is no cache of the same content for a different application, even in a different space. The Cloud Foundry environment has global caching of application bits without organizational and space isolation.
- The application is restaged and restarted if the following is true:
  - The application attributes, such as commands, buildpacks, memory, `health-check-url`, and so on, are changed.
  - The application environment is changed.
  - There are new services to which the application is bound.
  - There are services the application is bound to, which are discontinued (not available) in the new version of the MTA.

    > ### Note
    >
    > Currently, Cloud Foundry environment applications are restaged if they are bound to any service with parameters. This is necessary, because it's not possible to get the current Cloud Foundry service parameters and compare them with the desired ones.

  - The application is bound to service whose configuration parameters have been changed. This requires an update of the service instance and rebind, restage, and restart of the application.
  - The service binding parameters are changed. This requires an update of the service binding and restage of the application.
  - The MTA version is changed, which requires a change of special application environment variables, managed by the deploy service.

Application-Specific Timeouts

When creating Cloud Foundry applications as part of the MTA deployment, you have the option to set specific timeouts for different phases of the application's creation or execution. When any of these timeouts occur, the MTA operation fails with a relevant error message to prevent the MTA deployment from running indefinitely.

Currently the following timeouts are supported:

- **Upload timeout** – the time it takes to upload the application binary to the Cloud Controller. Once the upload is complete, the application is ready to use.
- **Stage timeout** – the time it takes to stage the application.
- **Start timeout** – the time it takes to start all application instances.
- **Task execution timeout** – the time between the initiation of a Cloud Foundry task and its final state. This timeout also applies to module hooks, as they use Cloud Foundry tasks. For more information, see Module Hooks [page 668].

Priority of Timeout Parameters

You can configure application timeouts at different levels according to your needs. When the same timeout (for example, start timeout) is configured in several places for one MTA deployment, a certain order of priority is applied.

The order provides flexibility when you want to define common timeouts for all applications which are part of an MTA but at the same time, you need to define application-specific timeouts.

The timeout parameters follow the following descending order of priority:

1. **Operation parameters / Command-Line Options (Highest Priority):**
   The highest priority is given to operation parameters passed during the MTA deployment. If you provide a timeout value as a command-line option, it overrides any other parameters defined into the MTA descriptor. These operation parameters apply to all applications.

   > ### Example
   >
   > ```
   > cf deploy … --apps-upload-timeout 40 –apps-upload-timeout 50 --apps-start-timeout 30 –apps-task-execution-timeout 100
   > ```

2. **Module-Level Parameters:**
   If you do not provide any command-line arguments, the module-level parameters are used instead.

   > ### Example
   >
   > ```
   > modules:
   >    - name: java
   >      ..........
   >      parameters:
   >         upload-timeout: 100
   >         stage-timeout: 50
   >         start-timeout: 60
   >         task-execution-timeout:120
   > ```

3. **Top-Level Parameters:**
   If neither a command-line option nor a module-level parameter are provided, the top-level parameters are used. Top-level parameters are applicable to all applications.

   > ### Example
   >
   > ```
   > parameters:
   >      apps-upload-timeout: 50
   >      apps-stage-timeout: 60
   >      apps-start-timeout: 70
   >      apps-task-execution-timeout:110
   > ```

4. **Default Values:**
   If you do not provide any of the above parameters, the application uses the predefined default timeout values. This ensures that there is always a time limit in place to prevent MTA operations from running indefinitely.
   Default values:
   - Start timeout: 1h
   - Upload timeout: 1h
   - Stage timeout: 1h
   - Task execution timeout: 12h

Maximum Allowed Values

- Upload timeout: 3h
- Stage timeout: 3h

- Start timeout: 3h
- Task execution timeout: 24h

Related Information

Modules [page 643]

4.1.8.1.8.2 Docker Images as Part of an MTA Deployment

Deploy Docker images as part of a Multitarget application.

You can deploy your own or 3rd party Docker images in the Cloud Foundry environment by referencing them in an application module.

This type of application deployment is faster, as images are already built. Thus, staging is not required, and also all dependencies are statically included in the image.

> **Note**
>
> When you reference docker images, MTA packages are not self-sustained - they depend on external repositories hosting images for the deployment.

You can deploy Docker images as MTA modules using the Cloud Foundry command line interface.

To deploy a Docker image as a Cloud Foundry application, your deployment descriptor should reference the image in a module as described in the following example:

Sample Code

```
…
modules:
  name: foo
  type: application
    parameters:
      docker:
        image: cloudfoundry/test-app
        username: <username>
        password: <password>
    build-parameters: #only required for mta.yaml
        no-source: true
…
```

Using the parameters above:

- `image` - you reference the location of the image so that it can be scheduled for download by the platform. You have to provide the location of the image using the format `<registry.domain><:port>/repo/image:<tag>`. If you don't do so, images are referenced from Docker Hub. See more about deploying an application with Docker in the provided link at the end of the section.
- (Optional) `username` and `password` - you provide credentials in the descriptor only when the image repository requires them.

- `no-source` - set this parameter to `true` only when creating an MTA development descriptor (`mta.yaml`) that is meant to be used with the Cloud MTA Build Tool.

Related Information

Cloud Foundry Documentation: Deploying an App with Docker
CF MTA Examples in GitHub: Deploying Docker Images as CF Apps with an MTA
https://hub.docker.com/
Cloud MTA Build Tool: Configuring a module that does not have source code to build and package

4.1.8.1.8.3 Tasks

Create one-off administration tasks or scripts.

During the deployment process, these tasks can be executed against staged applications. The platform creates a new container for them, where they are performed for a specific period until they are completed, after which the container is deleted.

One-off tasks are modeled in accordance to the following structure:

Sample Code

```
_schema-version: "3.1"
ID: foo
version: 3.0.0
modules:
  - name: foo
    type: javascript.nodejs
    parameters:
      no-route: true
      no-start: true
      disk-quota: 2GB
      tasks:
      - name: example_task
        command: npm start
        memory: 1GB
```

In the structure above, the parameters stand for the following:

- When the task is triggered, the `npm start` command is executed.
- Entering a specific value in the `memory` parameter is optional. A platform-derived default memory size is used for the execution of the task. You can specify a different value if required.
- The `no-route` parameter specifies if a route is required. Applications whose only purpose is to perform a task and then be stopped usually do not require a route. The default value is `false`.
- Use the `no-start` parameter if you want only the one-off tasks to be executed, that is, without triggering the start of the application. If you do not define this parameter, tasks are automatically executed after the applications are started.

> ### Note
>
> The values for application memory and task memory do not have a dependency. This is also valid for the allowed disk quota.

> ### → Tip
>
> For more information about one-off tasks, see MTA Deployment Descriptor Examples [page 624].

The following codeblock contains an example for a database migration task:

> ### {·} Sample Code

```
_schema-version: "3.1"
ID: foo
version: 3.0.0
modules:
  - name: foo
    type: javascript.nodejs
    parameters:
      no-route: true
      no-start: true
      memory: 1GB
      disk-quota: 2GB
      tasks:
        - name: db_migration
          command: "bin/rails db:migrate"
          memory: 256M                    #This parameter is optional.
          disk-quota: 128M                #This parameter is optional.
```

Related Information

Using Tasks

4.1.8.1.8.4 Services

If you want to create a service instance in the Cloud Foundry environment using the SAP Cloud Deployment service, you must define an MTA resource first. These resources can have different types and parameters which modify their behavior. There is a predefined set of supported resource types, that in most cases represents a certain combination of service offering and plan. See Predefined MTA Resource Types [page 634].

> ### {·} Sample Code

```
resources:
  - name: my-xsuaa-service
    type: com.sap.xs.uaa
```

The result will be a service instance called `my-xsuaa-service` with service offering `xsuaa` and service plan `application`.

For a more flexible approach, see Generic MTA Resource Types [page 632] and Resource-Specific Parameters [page 637].

> **Note**
>
> In most cases, MTA resources represent a Cloud Foundry service, but there are cases where they represent a different platform entity, for example a service key. They can even serve only to a group of configurations. See Resources [page 630] for more information.

Service Plans

> **Note**
>
> This information applies only to service instances that are managed by service brokers.

A service plan defines a specific configuration of a service offered on SAP BTP. While the service itself provides the functionality (e.g., database, messaging, etc.), the service plan determines the level or variant of that functionality - such as free, standard, or enterprise editions.

Each service offered on the marketplace has one or more available service plans. Choosing the appropriate plan is essential for resource allocation, cost, and feature set.

When you create or update a service instance, you need to specify the desired service with the required `service-plan` parameter:

> **Sample Code**

```
resources:
  - name: my-database
    type: org.cloudfoundry.managed-service
    parameters:
      service: hana
      service-plan: hdi-shared
```

Updating Service Plans

Updating service plans is, by default, **fail-safe** for those operations which result in a call to the service brokers. You can control this behavior by using the `fail-on-service-update` parameter.

Possible values for the parameter are:

- **null** (default value): If the update triggers an asynchronous operation (when the service brokers allow plan updates), the deployment will not fail even if the operation fails.
- **true**: If the update of the service plan fails (even asynchronously), the deployment will also fail.
- **false**: The update becomes even more permissive - even if the initial call to update the service plan fails, the deployment will still continue.

> **Example**

```
parameters:
  fail-on-service-update:
```

```
    plan: true
```

4.1.8.1.8.4.1 Service Instance Parameters

Some services support additional configuration parameters in the `create-service` or `update-service` request. These parameters are parsed in a valid JSON object containing the service-specific configuration parameters.

The deploy service supports the following methods for the specification of service instance parameters:

- Method A - an entry in the MTA deployment descriptor or the extension
- Method B - a JSON file with the required service configuration parameters

> ### Note
>
> If service instance information is supplied both in the deployment (or extension) descriptor **and** in a supporting JSON file, the parameters specified directly in the deployment (or extension) descriptor override the parameters specified in the JSON file.

Service instance parameters - method comparison

Method A	Method B	Combination of the two methods

Sample Code
MTA deployment descriptor or extension
`resources:`
`  - name: java-uaa`
`  type:`
`com.sap.xs.uaa`
`  parameters:`
`  config:`
`  xsappname:`
`java-hello-world`
	Sample Code
	`MTA deployment descriptor or extension`
	`resources:`
	`  - name: java-uaa`
	`  type:`
	`com.sap.xs.uaa`
	
	Sample Code
	`xs-security.json`
	`{`
	`  "xsappname": "java-hello-world"`
	`}`
	
	Sample Code
	Additional entry in `MANIFEST.MF`
	`Name: xs-security.json`
	`MTA-Resource: java-uaa`
	`Content-Type: application/json`
		Sample Code
		`resources:`
		`  - name: java-uaa`
		`  type:`
		`com.sap.xs.uaa`
		`  parameters:`
		`  path: <path to directory>`
		`  config:`
		`  xsappname:`
		`java-hello-world`
		
		Sample Code
		`xs-security.json`
		`{`
		`  "xsappname": "java-hello-world"`
		`}`
		
		Sample Code
		Additional entry in `MANIFEST.MF`
		`Name: xs-security.json`
		`MTA-Resource: java-uaa`
		`Content-Type: application/json`

Using method A, all parameters under the special `config` parameter are used for the service creation or the service update request. This parameter is optional.

→ Tip

You can use the optional `path` parameter to navigate to a JSON file containing service-specific parameters and configurations. The content of the file is used for the creation or update of the service instance.

Note that if you are deploying from a local directory using an `mtad.yaml`, it needs to contain the path to the JSON file, so that the relevant `MANIFEST.MF` entry is generated.

ⓘ Note

The `path` parameter only works when deploying from directory or when it is specified in the development descriptor `mta.yaml`. It is used only in build time and not during runtime (deployment).

Using method B, there are dependencies on further configuration entries in other configuration files. For example, if you use this JSON method, an additional entry must be included in the `MANIFEST.MF` file which

defines the path to the JSON file containing the parameters as well as the name of the resource for which the parameters should be used.

Note that when you use the combination of the two methods, the parameters defined in the descriptor have higher priority than the ones defined in the JSON file.

Updating Service Instance Parameters

Only valid for service instances managed by service brokers

Updating service instance parameters is, by default, **fail-safe** for those operations which result in a call to the service brokers. You can control this behavior by using the `fail-on-service-update` parameter.

Possible values for the parameter are:

- **null** (default value): If the update triggers an asynchronous operation (Cloud Controller sends the parameters to the service broker), the deployment will not fail even if the operation fails.
- **true**: If the update of the service parameters fails (even asynchronously), the deployment will also fail.
- **false**: The update becomes even more permissive - even if the initial call to update the service parameters fails, the deployment will still continue.

Example

```
parameters:
  fail-on-service-update:
    parameters: true
```

Only valid for user-provided service instances

The update of the user-provided service instance parameters is **not fail-safe by default** (because no service brokers are involved). All update operations for user-provided service instances are **synchronous**. You can control this behavior by using the `fail-on-service-update` parameter.

Possible values for the parameter are:

- **null** (default value): If the update of the service parameters fails, the deployment will also fail (same behavior as when set to true)
- **true**: If the update of the service parameters fails, the deployment will also fail.
- **false**: The update becomes permissive – even if the call to the Cloud Controller fails, the deployment will still continue.

Example

```
parameters:
  fail-on-service-update:
    parameters: true
```

Related Information

Standalone Application Router - HTML5 Application Repository
Managing Service Instances with the cf CLI

4.1.8.1.8.4.2 Service Binding Parameters

Some services support additional configuration parameters with the bind request. These parameters are passed in a valid JSON object containing service-specific configuration parameters, provided either in-line or in a file. For a list of supported configuration parameters, see the documentation for the particular service offering.

> **Note**
>
> At the time being user-provided services do not support arbitrary binding parameters.

The SAP Cloud Deployment service supports the following methods for the specification of service-binding parameters:

- Method A - In-line parameters into MTA descriptors (deployment or extension descriptors)
- Method B - JSON file with external configuration, containing the required service-configuration parameters
- A combination of the first two methods, as described in the table below

> **Note**
>
> If service-binding information is supplied both in the MTA's deployment (or extension) descriptor **and** in a supporting JSON file, the parameters specified directly in the deployment (or extension) descriptor override the parameters specified in the JSON file.

In the MTA deployment descriptor, the `requires` dependency between a module and a resource represents the binding between the corresponding application and the service created from them (if the service has a `type`). For this reason, the `config` parameter is nested in the `requires` dependency parameters, and a distinction must be made between the `config` parameter in the `modules` section and the `config` parameter used in the `resources` section (for example, when used for service-creation parameters).

Service Binding Parameters - method comparison

Method A	Method B	Combination of the two methods

Sample Code
MTA deployment descriptor or extension
`modules:`
`  - name: node-hello-world-backend`
`  type: javascript.nodejs`
`  requires:`
`  - name: node-hdi-container`
`  parameters:`
`  config:`
`permissions: debugging`
	Sample Code
	A JSON file with the required service-configuration parameters
	`modules:`
	`  - name: node-hello-world-backend`
	`  type: javascript.nodejs`
	`  requires:`
	`  - name: node-hdi-container`
		Sample Code
		`modules:`
		`  - name: node-hello-world-backend`
		`  type: javascript.nodejs`
		`  requires:`
		`  - name: node-hdi-container`
		`  parameters:`
		`  config:`
		`permissions: debugging`

	Sample Code
	Content of `xs-hdi.json`
	`{`
	`  "permissions": "debugging"`
	`}`
		Sample Code
		Content of `xs-hdi.json`
		`{`
		`  "permissions": "debugging"`
		`}`

	Sample Code
	Additional entry in `MANIFEST.MF`
	`Name: xs-hdi.json`
	`MTA-Requires: node-hello-world-backend/node-hdi-container`
	`Content-Type: application/json`
		Sample Code
		Additional entry in `MANIFEST.MF`
		`Name: xs-hdi.json`
		`MTA-Requires: node-hello-world-backend/node-hdi-container`
		`Content-Type: application/json`

> **Note**
>
> The examples are only illustrative for service-binding parameters. Each service offering supports its own set of parameters.

Method A shows how to define the service-binding parameters in the MTA deployment descriptor (`mtad.yaml`). If you use this method, all parameters under the special `config` parameter are used for the service-bind request. This parameter is optional.

Method B shows how to define the service-binding parameters for a service-bind request in a JSON file. Using this method, there are dependencies on entries in other configuration files. For example, when using the JSON method, an additional entry must be included in the `MANIFEST.MF` file. This new entry has to define the path to the JSON file that contains the parameters and the name of the resource for which the parameters should be used.

Note that when you use the combination of the two methods, the parameters defined in the descriptor have higher priority than the ones defined in the JSON file.

> **Note**
>
> To avoid ambiguities, the name of the module is added as a prefix to the name of the `requires` dependency; the name of the manifest attribute uses the following format: `<module-name>#<requires-dependency-name>`.

4.1.8.1.8.4.3 Service Tags

Some services provide a list of tags that are later added to the `<VCAP_SERVICES>` environment variable. These tags provide a more generic way for applications to parse `<VCAP_SERVICES>` for credentials.

You can also provide custom tags when creating a service instance. To inform the SAP Cloud Deployment service about custom tags, you can use the special `service-tags` parameter. This parameter must be located in the `resources` definition that represent the managed services, as illustrated in the following example:

> **Sample Code**
>
> Defining Service Tags in the MTA Deployment Descriptor

```
resources:
  - name: nodejs-uaa
    type: com.sap.xs.uaa
    parameters:
      service-tags: ["custom-tag-A", "custom-tag-B"]
```

> **Note**
>
> Some service tags are inserted by default, for example, `xsuaa`, for the XS User Account and Authentication (UAA) service.

Updating Service Tags

Updating service tags is, by default, **not fail-safe** because the update does not result in a call to the service broker and can also be used with user-provided service instances. You can control this behavior by using the `fail-on-service-update` parameter.

Possible values for the parameter are:

- **null** (default value): If the update fails, the deployment will also fail.
- **true**: If the update of the service tags fails, the deployment will also fail.
- **false**: The update becomes permissive - even if the call to update the service tags fails, the deployment will still continue.

Example

```
parameters:
  fail-on-service-update:
    tags: true
```

4.1.8.1.8.4.4 Service Broker Creation

Service brokers are applications that advertise a catalog of service offerings and service plans, as well as interpreting calls for creation, binding, unbinding, and deletion. The deploy service supports automatic creation (and update) of service brokers as part of an application deployment process.

An application can declare that a service broker should be created as part of its deployment process, by using the following parameters in its corresponding module in the MTA deployment (or extension) descriptor:

→ Tip

You can use placeholders `${}` in the service-URL declaration.

Sample Code

```
- name: jobscheduler-broker
  properties:
    user: ${generated-user}
    password: ${generated-password}
  parameters:
    create-service-broker: true
    service-broker-space-scoped: true
    service-broker-name: jobscheduler
    service-broker-user: ${generated-user}
    service-broker-password: ${generated-password}
    service-broker-url: ${default-url}
```

Note

By default, defined service brokers in MTA are attempted to be registered as standard service broker which is available in all orgs and spaces. In practice, however, this is forbidden, as the registration requires CF admin access, which is not provided to anyone.

For test purposes, space-scoped brokers can be registered. In order to do that, parameter `service-broker-space-scoped` needs to be equal to `true`.

The value of the "`${generated-user}`" and "`${generated-password}`" placeholders in the `properties` section is the same as in the `parameters` section. The service-broker application uses this mechanism to inject the user-password credentials.

The `create-service-broker` parameter should be set to true if a service broker must be created for the specified application module. You can specify the name of the service broker with the `service-broker-name` parameter; the default value is `${app-name}`The `service-broker-user` and `service-broker-password` are the credentials that will be used by the controller to authenticate itself to the service broker in order to make requests for creation, binding, unbinding and deletion of services. The `service-broker-url` parameter specifies the URL on which the controller will send these requests.

> **Note**
>
> During the creation of the service broker, the Cloud Controller makes a call to the service-broker API to inquire about the services and plans the service broker provides. For this reason, an application that declares itself as a service broker must implement the service-broker application-programming interface (API). Failure to do so might cause the deployment process to fail.

> **Note**
>
> Normally, the registration of a space-scoped broker is successful, because it requires SpaceDeveloper privileges of the user. However, for a global registration of the service broker, global admin privileges are needed, which the platform developer usually does not have. In such cases, the MTA deployment would fail. To solve this, do not use the `--do-not-fail-on-missing-permissions` option, which will result in skipping the step with a warning.

4.1.8.1.8.4.5 Service Keys

The consumption of existing service keys from applications is an alternative of service bindings. The application can use the service key credentials and consume the service.

Creation and Update of Service Keys

A `service-keys` parameter can be created or updated when a service instance is being created or updated. It has to be defined under the resources that represent services which support service keys.

> **Sample Code**

```
resources:
- name: my-service-instance
  type: org.cloudfoundry.managed-service
  parameters:
    service-keys: # Specifies the service keys that should be created for the
respective service instance. Optional parameter.
    - name: tool-access-key # Specified the service key name. Mandatory
element.
      config: # Specified the service key configuration. All entries under
this map element are used for the service key creation request. Optional
element.
        permissions: read-write
    - name: reader-endpoint
      config:
        permissions: read-only
```

As shown in the example above, every service key entry under the `service-keys` parameter supports optional configuration parameters that can be defined under the `config` parameter.

Consumption of Service Keys

To be consumed, the existing service keys are modeled as a resource of type `org.cloudfoundry.existing-service-key`. MTA modules might be set to depend on these resources by using a configuration in the `requires` section, which results in an injection of the service key credentials in the application environment.

Sample Code

```
modules:
  - name: app123
    type: javascript.nodejs
    requires:
      - name: service-key-1
        parameters:
          env-var-name: keycredentials
...
resources:
  - name: service-key-1
    type: org.cloudfoundry.existing-service-key
    parameters:
      service-name: service-a
      service-key-name: test-key-1
```

As a result, the application `app123` has the environment variable `keycredentials`, with value the credentials of the existing service key `test-key-1` of service `service-a`.

Note

Note that only the parameter `service-name` is mandatory. It defines which service is used by the application.

The following parameters are optional:

- `service-key-name`- resource parameter, which defines which service key of the defined service is used. The default value is the name of the resource.
- `env-var-name` - required dependency parameter, which defines what is the name of the new environment variable of the application. The default value is the service key name.

Automatic Service Key Renewal

It's a common use case to want an automatic service key renewal (e.g. if you want to rotate generated service credentials or certificates).

Service key renewal on MTA redeployment is supported by using changed key names. You can do this either by manually changing the service key name in the descriptor or by using a dynamic parameter as part of it - `${timestamp}`. This parameter will always resolve to the current timestamp at the time of deployment. This means that the key will have a different name on each subsequent deployment. This ensures that it will be considered as new and created. Afterwards the old key will be deleted. This functionality applies to service

keys modelled under a resource using the `service-keys:` parameter. Example descriptor for automated key renewal/rotation:

Sample Code

```
...
resources:
  - name: my-service
    type: org.cloudfoundry.managed-service
    parameters:
      service-plan: plan
      service: offering
      service-keys:
        - name: my-rotating-key-${timestamp}
...
```

To consume a service key with a changing name in an application environment, make sure to use the parameter `env-var-name` when defining the service key `requires` section in the consuming module. This will ensure that the service key details are persisted under a static alias in the application environment. e.g.

Sample Code

```
modules:
- name: my-app
...
  requires:
    - name: rotating-key
      parameters:
        env-var-name: *keycredentials*
...
resources:
  - name: my-service
    type: org.cloudfoundry.managed-service
    parameters:
      service-keys:
        - name: rotating-key-${timestamp}
  - name: rotating-key
    type: org.cloudfoundry.existing-service-key
    parameters:
      service-name: my-service
      service-key-name: rotating-key-${timestamp}
```

4.1.8.1.8.5 Routes

Configure application routes using the MTA modeling.

The `routes` parameter defines the HTTP routes that are created (if they don't already exist) and bound to an application module during deployment. It accepts a list of one or more route entries and replaces the older `host`, `domain`, `port`, and `route-path` parameters. If both the `routes` parameter and the older parameters are present in a descriptor, the `routes` value takes precedence and the older parameters are ignored.

Each `route` entry in the list is a map of key-value pairs. The key `route` is required and specifies the Cloud Foundry route (cf route) to be created and bound to the application. The deployer reads the `route` entry in the same way as a Cloud Foundry app Manifest. It interprets everything up to the first dot `'.'` as the host, then everything between the first dot and the first forward slash `'/'` as the domain, and the rest (if present) as the path. Multiple optional keys are supported, including `apply-namespace` and `no-hostname`.

The host and domain parts of a route must meet the following requirements:

Part	Maximum Length	Allowed Characters	General Requirements
Host	63 characters	Lowercase letters (a–z), digits (0–9), and hyphens (-)	Must both start and end with a letter or a digit
Domain	253 characters	Lowercase letters (a–z), digits (0–9), hyphens (-), and dots (.)	

If you do not define the route using the `routes` parameter or the `host` and `domain` parameters, a placeholder `default-uri` is automatically applied. This placeholder is resolved to `${default-host}.${default-domain}`. The `default-host`, as part of the URI, is resolved to `${org}-${space}-<module_name>`.

> **Note**
>
> Concatenating the org, space, and module names can exceed the 63-character DNS host label limit and cause deployment failures. For this reason, we recommend that you explicitly set the `routes` parameter instead of using the automatically set default route.

For more information about `default-` parameters, see Module-Specific Parameters [page 650].

If you want to create a route with a wildcard host name, use an asterisk in quotes ("*").

```
modules:
- name: my-app
  type: application
  parameters:
    routes:
      - route: "*foo.my.custom.domain/path"
      - route: "foo.my.custom.domain/path"
      - route: "foo.${default-domain}/path"
```

To reference a specific route inside the deployment descriptor (for example for a `provides` section), use the following syntax. The index 0 retrieves the first route in the list:

> **Example**
>
> ```
> provides:
>   - name: foo
>     properties:
>       url: "${protocol}://${routes/0/route}"
> ```

Deploy applications to a route without a host name

To bind an app to a route without a host name, you can use the `routes` and `no-hostname` parameters. Such a route is a subdomain of an existing domain with no host attached at the front. Note that `no-hostname` is a boolean parameter, and must follow every route definition. For example, if you have two or more routes without host names in your MTA, then you need to add that parameter under each of those routes. If for a given route the `no-hostname` parameter is missing, its value defaults to `false`.

Sample Code

```
routes:
        - route: host1.some-domain.com
        - route: host2.some-domain.com
          no-hostname: false
        - route: subdomain.some-domain.com
          no-hostname: true
```

In the example above, `no-hostname: false` has no effect on the second route, since that is the default value, while `host2` is registered as a host to the route. The `no-hostname: true` parameter, however conveys to the deployer that `subdomain` is not a separate host, and the whole route is used as a Cloud Foundry domain (subdomain of `some-domain.com`, if present).

Caution

The MTA deployer attempts to create the subdomain if it does not exist, but cannot resolve any specific corner cases about subdomain/domain ownership permissions. If the subdomain has already been created, the deployer maps the application to it.

Deploy applications to http2 routes

If you want to enable http2 routing traffic to MTA modules (applications) you can use the `protocol: http2` parameter on route level.

Note that the protocol value must be `http1` or `http2` and must be included in every route in order for it to take effect. If the `protocol` parameter is not specified, the value will be set to `http1` by default.

Sample Code

```
routes:
  - route: http1-route.some-domain.com
  - route: http1- route.some-domain.com
    protocol: http1 # The default value if not provided
  - route: http2- route.some-domain.com
    protocol: http2
```

Deploy applications to a route with a namespace

To bind an application to a route with a namespace, use the `routes` and `apply-namespace` parameters. When the value of the `apply-namespace` parameter is set to `true`, a prefix with the namespace value will be added to the route.

Sample Code

```
routes:
        - route: host1.some-domain.com
          apply-namespace: true
        - route: host2.some-domain.com
```

```
            apply-namespace: false
```

If you don't explicitly set the `apply-namespace` parameter for a specific route, its value will be determined by the application's `apply-namespace` parameter. This means that if the application has a namespace defined, that namespace will also be applied to the bound route. Conversely, if the application does not have a namespace, the route will not have one either.

For more information, see Fine-Grained Configuration [page 798].

Route validation and automatic correction

When a route defined in the MTA deployment descriptor contains characters that are not allowed, or exceeds the allowed length, SAP Cloud Deployment service attempts to automatically correct it before deployment:

- Uppercase letters are converted to lowercase.
- Disallowed characters (for example, underscores, @ signs, or spaces) are replaced with a hyphen (-).
- Leading and trailing hyphens are removed.
- Parts that exceed the maximum length are shortened to fit.

> **Note**
>
> These corrections are applied silently. Your application may be assigned a route that differs from the one you specified in the descriptor. If the exact route matters, define a route that already meets the requirements above.

The deployment fails in the following cases:

- The route cannot be corrected into a valid route — for example, when a host or domain contains no usable characters after normalization. The deployment fails with a similar error: `Could not create a valid host from "…"`, `Could not create a valid domain from "…"`, or `Could not create a valid route from "…"`.
- The `routes` parameter is malformed — for example, it is not a valid list of route entries or contains an unsupported attribute. The deployment fails with a similar error: `Cannot parse 'routes' - check documentation for correct format.`
- Cloud Foundry rejects the corrected route — for example, because the domain does not exist or the route conflicts with an existing one. The deployment fails with the corresponding Cloud Foundry level.

> **Note**
>
> - Automatic route correction cannot be disabled. To ensure a route is used exactly as written, define it so that it already meets the host and domain requirements. A route that is already valid is left unchanged.
> - To detect unintended corrections, compare the routes in your descriptor with the resolved routes reported in the deployment output.

4.1.8.1.9 Features

The section describes the features of MTA deployment in the Cloud Foundry environment of the SAP Business Technology Platform.

- Content Deployment [page 769]
  - Deploying Content with Generic Application Content Deployment [page 770]
  - Deploying Content with CF Task Execution [page 778]
  - Deploying Content with Simulated App Execution [page 781]
  - CPI Content Deployment [page 784]
- Blue-Green Deployment of Multitarget Applications [page 785]
  - Legacy Blue-Green Deployment [page 785]
  - Blue-Green Deployment Strategy [page 788]
  - Incremental Blue-Green Deployment Strategy [page 791]
- (Experimental) Rollback of Multitarget Applications [page 794]
- Namespaces [page 798]
- Parallel Module Deployment [page 803]
- Sequential Resource Processing [page 805]
- Cross-MTA Dependencies [page 806]
- Sensitive Data Handling During MTA Deployment [page 811]
- Features Related to SAP Alert Notification Service for SAP BTP [page 820]

4.1.8.1.9.1 Content Deployment

SAP Business Technology Platform Cloud Foundry supports the lifecycle of all standard Cloud Foundry entities, such as applications and services. To satisfy all business requirements, SAP Business Technology Platform Cloud Foundry also supports the lifecycle of other entities that do not match 1-to-1 to CF native entities.

SAP traditionally provides its customers with complex business solutions, which work with their own entities and content. Every service might have its own content with a specific format that is developed, provided, or consumed by end customers.

SAP Workflow Management provides the possibility for SAP partners to develop their own BPM workflows (for example, Leave Request flow) and provide them to end customers. In turn, end customers can easily import and use the developed BPM workflows.

With multitarget applications, you can manage the lifecycle of content provided by different SAP applications. This way, by using one standard approach via an MTA, you can deploy various kinds of SAP application content in your accounts, normally done on the CF space level.

> ⚠ Restriction
>
> Content deployment used with blue-green deployment will be done during the first phase when idle suffixes and temporary routes are used. The same rule is applicable for all content deployment mechanisms – either with direct content deployment or with content deployers (`hdi-deploy`, `html5-app-deployer`, `portal deployer`). HTTP Destinations, created with direct content deployment, will not be updated after the validation phase, so avoid using default resolvable parameters `${default-host}`, `${default-uri}`,

`${default-url}`. Configurations that include idle suffixes and temporary routes may persist after the MTA deployment finishes. Content that was already deployed during the first phase cannot be reverted or removed. It will be necessary to trigger a new MTA deployment.

Supported Content Deployment Mechanisms

- Using content applications – there is a dedicated SAP provisioned content deployer for each content type that communicates with the dedicated content backend
  - [DEPRECATED] Deploying Content with Simulated App Execution
  - Deploying Content with CF task execution
- Using direct content deployment – there is a direct communication between the SAP Cloud Deployment service and the content backends
  - Deploying Content with Generic Application Content Deployment
  - CPI content deployment

Supported MTA Module Types for Content Deployment

Module Type	Content Deployment Mechanism
`com.sap.portal.site-content`	Simulated App Execution
`com.sap.html5.application-content`	Simulated App Execution
`com.sap.portal.content`	CF task execution
`com.sap.html5.application.content`	CF task execution
`com.sap.xs.hdi`	CF task execution
`com.sap.application.content`	Generic Application Content Deployment
`com.sap.integration`	CPI content deployment
`com.sap.api.management`	CPI content deployment

For all predefined parameters and properties for each MTA module type, see MTA Module Types [page 645].

4.1.8.1.9.1.1 Deploying Content with Generic Application Content Deployment

This approach provides a mechanism for direct content deployment from SAP Cloud Deployment service to the content backend without the need for an intermediate Cloud Foundry application.

> **Note**
>
> This approach is supported only for MTA schema version 3.1 and higher.

> ### Note
>
> This approach is recommended for its light-weight nature, minimal footprint in your Cloud Foundry account and avoided overhead of transient content applications.

> ### Note
>
> The examples below are partial and they intend to show the different capabilities of content deployment. For specific application content, see the dedicated documentation linked below.

The mechanism utilizes the Generic Application Content Deployment protocol (GACD). It allows you to deploy content represented by an MTA module of type `com.sap.application.content` to a certain list of supported services.

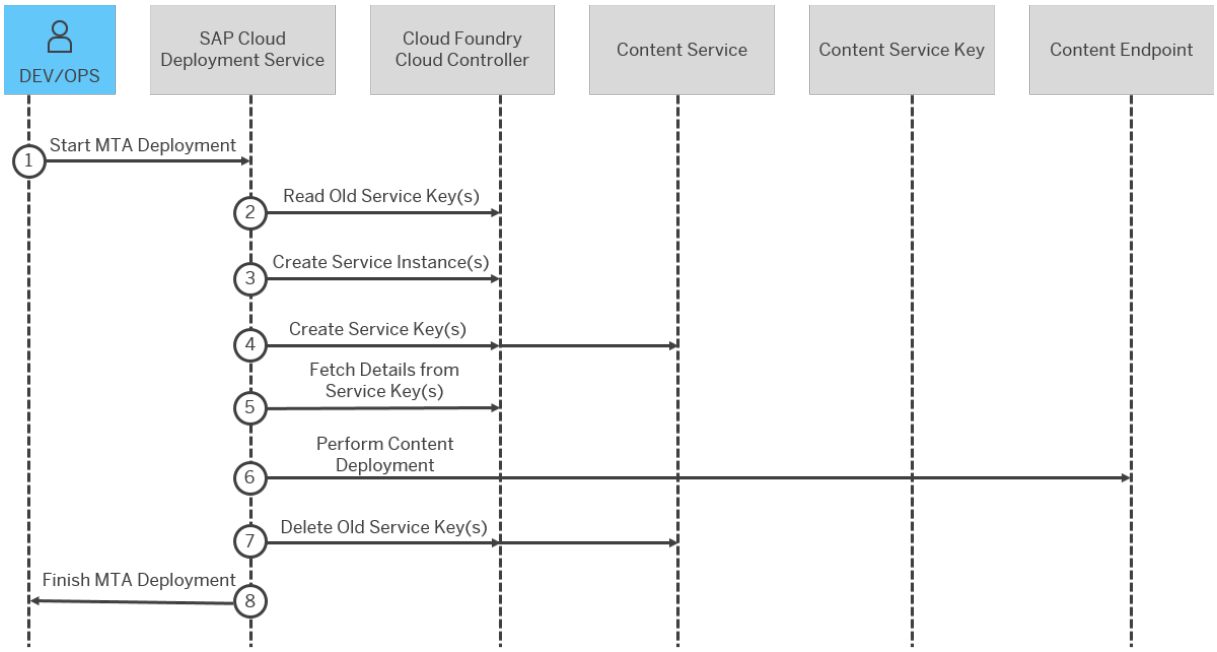
Supported service offerings

Cloud Foundry service offerings that currently support GACD are:

- `html5-apps-repo` (For more information, see Deploy Content Using Generic Application Content Deployer)
- `portal` (For more information, see Add an SAP Fiori Launchpad Module)
- `workflow` (For more information, see Create a Workflow Module)
- `destination` (For more information, see Create Destinations Using the MTA Descriptor)

How Content is Deployed with Generic Application Content Deployment

The diagram below illustrates the steps and SAP BTP components involved in the process of content deployment with Generic Application Content Deployment:



The following steps describe the process of content deployment with Generic Application Content Deployment:

1. A developer/operator runs MTA deployment – the flow demonstrates only modules of type `com.sap.application.content` indicating content (for example - a single MTA module and multiple required services, only one of them is marked with parameter `content-target: true`).
2. SAP Cloud Deployment service prepares for the next steps. This step includes:
    1. SAP Cloud Deployment service gets the existing services and service keys which are related to previous MTA deployment. The list comprises of a content service instance, a content service key and eventually additional services keys.
    2. Based on the name, SAP Cloud Deployment service decides which of the existing service keys will be deleted, which will be kept, and whether new ones will be created. In the end, only one of them will be used as a content target.
3. SAP Cloud Deployment service creates or reuses a service instance or instances that are relevant for the content deployment, not only the one that serves as a content target.
4. SAP Cloud Deployment service creates or reuses a service key. This step might include the creation of several service keys needed for successful content deployment, not only the one that serves as a content target.
5. SAP Cloud Deployment service fetches the credentials of the required service key(s). In this step content endpoint is detected from content target service key. The credentials of all remaining service keys serve as configurations for the next step.
6. SAP Cloud Deployment service executes the content deployment to content endpoint and passes previously collected data as content configurations.
7. SAP Cloud Deployment service deletes the discontinued service keys calculated in step 2. Normally these are service keys that are not part of the current MTA and were used in the previous deployment.
8. MTA deployment completes and SAP Cloud Deployment service reports the status to the developer/operator.

Content Target

When an MTA module of type `com.sap.application.content` is present in your MTA, SAP Cloud Deployment service processes it and delivers its module content to the content target.

The content target is explicitly set with a flag in the MTA and it is unknown to the SAP Cloud Deployment service without it.

There should be only one content target defined per MTA module of type `com.sap.application.content`. This guarantees that the content will be deployed to only one place.

The content target is defined when a content module requires a service or service key, and in that required dependency the parameter `content-target` is set to `true`. In case of a missing or duplicated `content-target` parameter in more than one required dependency, the deployment will not proceed.

Normally, content modules require MTA resources that represent Cloud Foundry service instances or service keys. When multiple service instances or keys are required, they are used by default for content deployment configuration. For more information see the Configuration from other service instances/keys [page 777] section.

MTA Modelling

Deliver file content

You can use a file or directory that is treated as content. The file or directory is part of the MTA archive, and its path is defined in the `MANIFEST.MF` file.

The following code block contains a snippet from an MTA deployment descriptor (mtad.yaml):

Sample Code

```
_schema-version: '3.1'
ID: com.sap.example.content.deployment
version: 0.0.1
modules:
 - name: content-module
   type: com.sap.application.content
   requires:
    - name: workflow_service
      parameters:
         content-target: true
resources:
 - name: workflow_service
   type: org.cloudfoundry.managed-service
   parameters:
      service-plan: standard
      service: workflow
```

The following code block contains the accompanying `MANIFEST.MF` file:

Sample Code

```
Name: content-module/data
MTA-Module: content-module
Content-Type: application/zip
```

Delivery text / Inline content

You can enter content definitions inline in the MTA descriptor as part of module parameter `content`. In the example below the template for new destinations (`foo-api` and `bar-api`) will be deployed to `destination-service`:

Sample Code

```
modules:
- name: destination-content-deployer
  type: com.sap.application.content
  parameters:
    content:
      subaccount:
        existing_destinations_policy: update
        destinations:
        - Name: workflow-destination
          ServiceInstanceName: workflow-service
  requires:
  - name: destination-service
    parameters:
      content-target: true
  - name: workflow-service
resources:
```

```
- name: destination-service
  type: org.cloudfoundry.managed-service
  parameters:
    service: destination
    service-name: destination-service
    service-plan: lite
- name: workflow-service
  type: org.cloudfoundry.managed-service
  parameters:
    service: workflow
    service-plan: standard
```

> **Note**
>
> For more information about destination creation with MTA, see Create Destinations Using the MTA Descriptor.

Service Keys in Content Deployment

When an MTA module of type `com.sap.application.content` requires an MTA resource, the SAP Cloud Deployment service will create or reuse a service key.

Service keys in content deployment are an alternative of service bindings between applications and services. When there is an intermediate content application, it might be bound to different services that provide specific capabilities. In direct content deployment, the required service keys have the same role. For example, a content module might require an SAP Authorization and Trust Management service (XSUAA) instance, which might be used for secure communication at a later stage.

By default, service keys to all required services are created with the name that follows the template `<module_name>-<resource_name>-credentials`. For example, the result of the sample below will be a service instance with name `workflow_service` which will have a service key named `content-module-workflow_service-credentials`:

> **Sample Code**

```
modules:
 - name: content-module
   type: com.sap.application.content
   requires:
    - name: workflow_service
      parameters:
         content-target: true
resources:
 - name: workflow_service
   type: org.cloudfoundry.managed-service
```

By default, the existing service keys are reused in future deployments. This is done to save time from their re-creation and avoid potential issues if they are used outside of the MTA and their credentials are invalidated.

Customizing service keys

You can customize the service keys created as part of the content deployment by using the required dependency parameter `service-key`. This way, you can define the service key name or pass service key creation parameters:

Sample Code

```
modules:
- name: content-module
   type: com.sap.application.content
   requires:
    - name: workflow_service
      parameters:
         content-target: true
         service-key:
           name: workflow_service-key
           config:
             <map entries for creation parameters>
```

Re-create custom service keys for deployment

This option extends the customization of service keys explained above. Based on its name, an existing service key may be reused or deleted, as described in How Content is Deployed with Generic Application Content Deployment [page 771].

The option is particularly useful if you want to rotate and renew the credentials used for content deployment, especially those based on user/password credentials or certificates.

You can set the service keys to automatically rotate for each new deployment by using the existing deployment parameter `${timestamp}` as part of the name of the defined custom service key. This ensures that the key will have a new name for each deployment, resulting in SAP Cloud Deployment service creating and using a new key for the related content deployment. The previously used service key will be deleted after the new one is in place.

Sample Code

```
modules:
- name: content-module
   type: com.sap.application.content
   requires:
    - name: workflow_service
      parameters:
         content-target: true
         service-key:
           name: workflow_service-key-${timestamp}
```

→ Tip

To see complete examples of service key re-creation, visit the following pages from the MTA examples repository:

- Automatic Service Key Renewal
- Managing CF Apps and SAP BTP Service Instance Destination with an MTA

Delete service keys after deployment

This option is similar to the **Re-create custom service keys for deployment** option explained above. As the title suggests, the used service keys are deleted after the successful content deployment.

Deleting service keys after deployment could be particularly useful for maintaining a minimal footprint in your environment or if you want to use new credentials for content deployment, especially those based on user/password credentials or certificates. This option cannot be used if the service key credentials are used at a later stage after the content deployment, such as during runtime, which is the case in HTML5 application content deployment. A drawback of this option is the missing possibility for troubleshooting in case of issues related to wrong credentials.

Sample Code

```
modules:
- name: content-module
   type: com.sap.application.content
   requires:
    - name: workflow_service
      parameters:
         content-target: true
         service-key:
           name: workflow_service-key-${timestamp}
         delete-service-key-after-deployment: true
```

Using existing service keys

You can deploy content directly to an existing service key with a descriptor modeled appropriately. This option might be useful if you want to manage the lifecycle of the service key outside of the MTA:

Sample Code

```
modules:
 - name: content-module
   type: com.sap.application.content
   requires:
    - name: workflow_service_key
      parameters:
         content-target: true
resources:
 - name: workflow_service_key #this service key needs to exist beforehand
   type: org.cloudfoundry.existing-service-key
   parameters:
      service-name: workflow_service #this service must be created and exist
beforehand
```

Using user-provided services

Sometimes, the content endpoint might not be exposed as a standard service broker in SAP BTP. In these cases, neither service instance nor service key can be created. The only option to utilize content deployment to such content target is via user-provided services.

Sample Code

```
modules:
 - name: content-module
   type: com.sap.application.content
   requires:
    - name: content-endpoint
```

```
      parameters:
         content-target: true
resources:
 - name: content-endpoint
   type: org.cloudfoundry.user-provided-service
   parameters:
      content_endpoint: https://my-content-endpoint.com
```

Content Deployment Configuration

Sometimes, the content deployment needs an environment-specific or context-specific configuration, which cannot be part of the content itself.

There are two supported ways to pass additional configuration for content deployment:

- Content Deployment Configuration from other service instances/keys
- Inline Content Deployment Configuration

Configuration from other service instances/keys

By default, all required MTA resources from content modules are processed and used for content deployment configuration. This includes the MTA resources of types `org.cloudfoundry.managed-service`, `org.cloudfoundry.existing-service`, `org.cloudfoundry.existing-service-key` and `org.cloudfoundry.user-provided-service`. A service key is used for each of those resources, except for `org.cloudfoundry.user-provided-service`.

In the example below, the content-module requires two services and only one of them is marked as content-target. The other required service is XSUAA, which will be used for content configuration and will guarantee more secure communication during the end-to-end flow:

Sample Code

```
_schema-version: '3.1'
ID: com.sap.example.content.deployment
version: 0.0.1
modules:
 - name: content-module
   type: com.sap.application.content
   requires:
    - name: xsuaa_service
    - name: html5_service
      parameters:
         content-target: true
resources:
 - name: html5_service
   type: org.cloudfoundry.managed-service
   parameters:
      service-plan: app-host
      service: html5-apps-repo
 - name: xsuaa_service
   type: com.sap.xs.uaa
```

When multiple service instances or keys are required, they are all used for content deployment configuration by default.

Inline Content Deployment Configuration

You can provide content deployment configuration as part of the module parameter `config`:

Sample Code

```
modules:
 - name: content-module
   type: com.sap.application.content
   requires:
    - name: workflow_service
      parameters:
         content-target: true
   parameters:
      config: # the content deployment configuration
         env: dev
resources:
 - name: workflow_service
   type: org.cloudfoundry.managed-service
   parameters:
      service-plan: standard
      service: workflow
```

Related Information

Deploy Content Using the Generic Application Content Deployer [page 437]
Create Destinations Using the MTA Descriptor

4.1.8.1.9.1.2 Deploying Content with CF Task Execution

This approach for content deployment relies on an intermediate Cloud Foundry application that communicates with the content backend. This application includes both the content and the content specific deployer.

The content specific deployer is defined as an application dependency during design time. It is then included in the MTA archive. During deployment, it serves as a client for the actual content deployment to the content backend.

Note

This approach is a natural successor to Deploying Content with Simulated App Execution [page 781]. When possible, it is recommended to use the current approach instead of the legacy one. Normally, there is only one dedicated content deployer for each content type that handles both approaches.

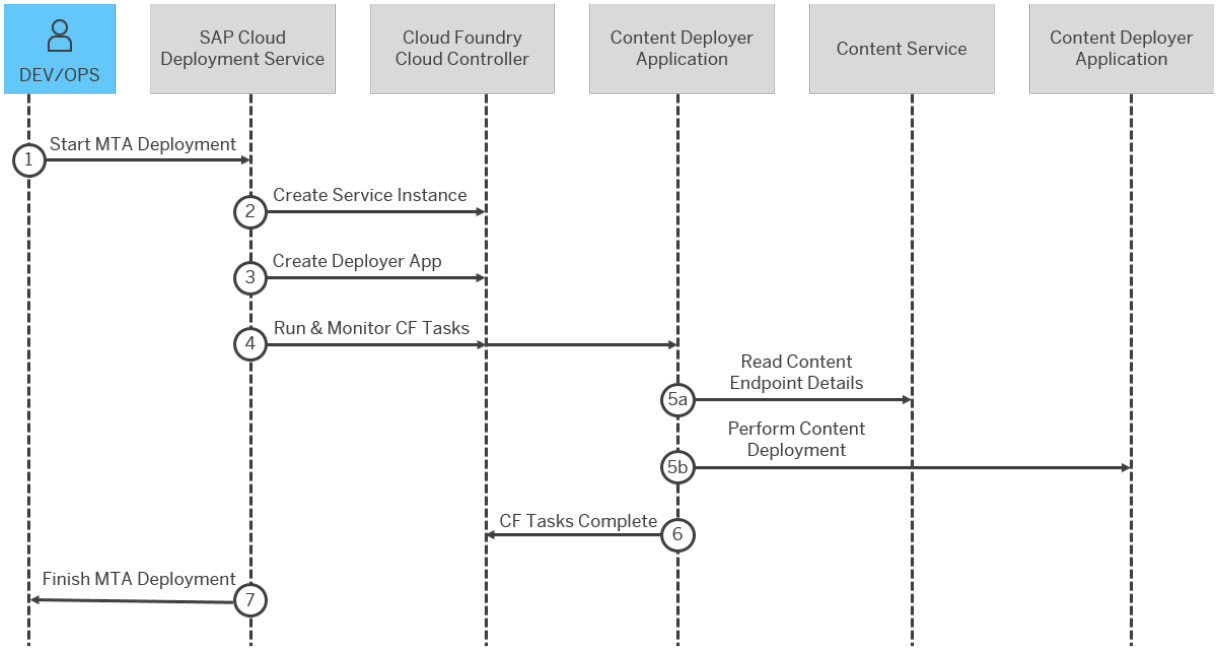
Supported Content Types

Supported content types that currently allow deployment with CF Task execution:

- HDI content - MTA module type `com.sap.xs.hdi`. For more information, see The SAP HDI Deployer
- FLP content - MTA module type `com.sap.portal.content`
- HTML5 Content - MTA module type `com.sap.html5.application.content`. For more information, see Deploy Content Using HTML5 Application Deployer

How Content is Deployed with CF Task Execution

The following diagram illustrates the steps and BTP components involved in the process of content deployment with CF task execution:



The following steps describe the process of content deployment with CF Task execution:

1. A developer/operator runs MTA deployment
2. SAP Cloud Deployment service creates or reuses a service instance provided by an SAP application
3. During this step:
    1. SAP Cloud Deployment service creates a Cloud Foundry application that serves as a content application
    2. SAP Cloud Deployment service binds the service instance to the content application
    3. SAP Cloud Deployment service uploads the combination of content and content deployer as application bits
4. SAP Cloud Deployment service runs CF tasks, including the one responsible for actual content deployment

5. The dedicated CF task of the content application triggers the content deployer, which in turn performs the actual content deployment. During this step:
    1. The CF task reads content details from the bound service instance and identifies the content endpoint and the required authentication
    2. The CF task executes the content deployment to the content endpoint
6. The CF task completed successfully
7. The MTA deployment completes and SAP Cloud Deployment service reports the status to the developer/ operator

Content Deployer and Service Instance

The Content Deployer and Service Instance follow the same rules as the ones from Deploying Content with Simulated App Execution [page 781].

CF Tasks

Tasks are native to Cloud Foundry and are valuable in various scenarios, which demand one-off jobs. For more information, see Tasks.

CF tasks, dedicated for content deployment, are predefined from the SAP Cloud Deployment service and must not be changed by the end customer. They are represented by the module level parameter `tasks`.

MTA Modeling

Below you can find the modeling for supporting the content deployment scenario for MTA module type `com.sap.xs.hdi`:

Sample Code

```
_schema-version: "3.1"
ID: my.app
version: "1.1"
- name: module-db
    type: com.sap.xs.hdi
    parameters:
      buildpack: nodejs_buildpack # Predefined parameter
      no-route: true  # Predefined parameter - skipping route mapping to the
app
      no-start: true # Predefined parameter - skipping start of the app
because only tasks are important in the scenario
      memory: 256M # Predefined parameter
      tasks: # Predefined parameter – all details below define the CF tasks
        - name: deploy
          memory: 256M
          command: npm start # Shell script that will be used for triggering
actual content deployment
    properties:
      EXIT: 1 # Predefined property – instructs the HDI deployer to exit when
done
```

```
    requires:
      - name: hdi-service

resources:
  - name: hdi-service # Provides content endpoint for the actual content
deployment. Content endpoint could be different based on the used database
    type: com.sap.xs.hdi-container
```

4.1.8.1.9.1.3 Deploying Content with Simulated App Execution

This approach for content deployment relies on an intermediate Cloud Foundry application that communicates with the content backend. This application includes both the content and the content specific deployer.

The content specific deployer is defined as an application dependency during design time. It is then included in the MTA archive. During deployment it serves as a client for the actual content deployment to the content backend.

> ⓘ Note
>
> This approach is a legacy one. Over time it has proven to have some major drawbacks. If possible, migrate to Deploying Content with Generic Application Content Deployment [page 770] or at least to Deploying Content with CF Task Execution [page 778] of the corresponding module type.

> ⓘ Note
>
> All supported MTA module types for this approach will either be discontinued or will be transformed to use CF Task Execution [page 778].

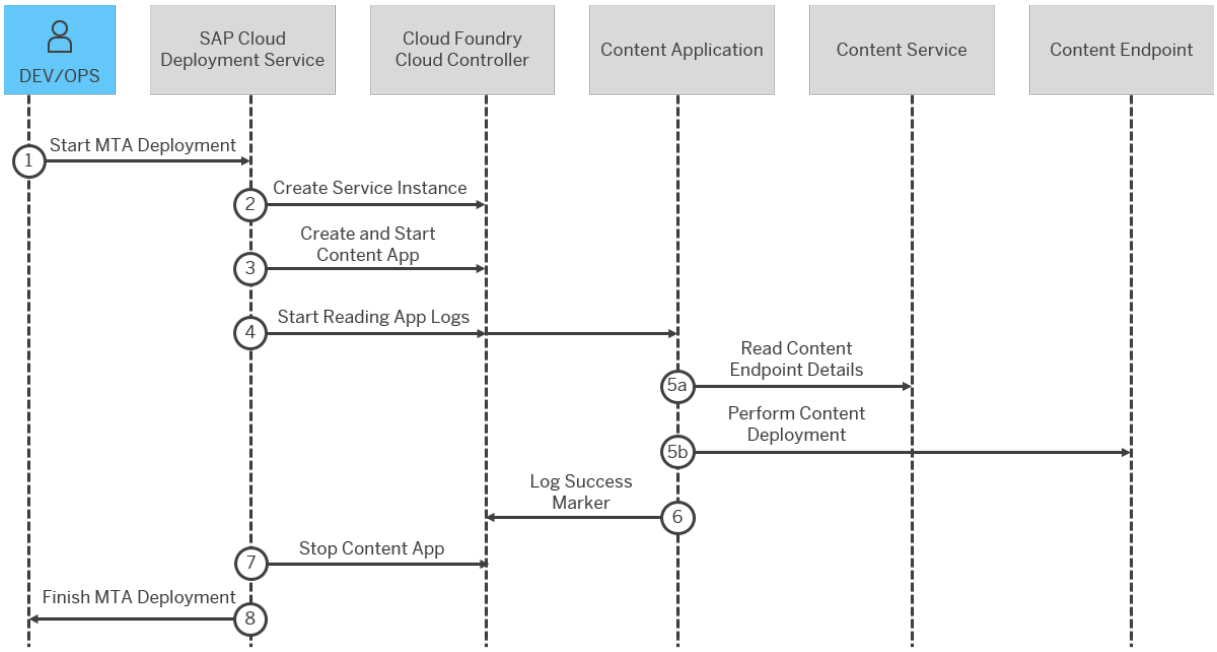
Supported Content Types

Supported content types that currently allow deployment with simulated app execution:

- FLP content - MTA module type `com.sap.portal.site-content`
- HTML5 Application Content – MTA module type `com.sap.html5.application-content`. For more information see Deploy Content Using HTML5 Application Deployer.

How Content is Deployed with Simulated App Execution

The following diagram illustrates the steps and BTP components involved in the process of content deployment with simulated app execution:



The steps below describe the process of content deployment with simulated app execution:

1. A developer/operator runs MTA deployment.
2. SAP Cloud Deployment service creates or reuses a service instance provided by an SAP application.
3. The third step includes:
    1. SAP Cloud Deployment service creates a Cloud Foundry application that serves as a content application.
    2. SAP Cloud Deployment service binds the service instance to the content application.
    3. SAP Cloud Deployment service uploads the combination of content and content deployer as application bits.
    4. SAP Cloud Deployment service starts the application, which in turn starts the content deployer.
4. SAP Cloud Deployment service starts to read the application logs and searches for the success-marker and/or failure-marker values.
5. Content deployer performs the actual content deployment. This step includes:
    1. Content deployer reads content details from the bound service instance and identifies the content endpoint and required authentication.
    2. Content deployer executes the content deployment to content endpoint.
6. Via certain success-marker or failure-marker, content deployer reports the status as application logs.
7. SAP Cloud Deployment service stops the content application.
8. MTA deployment completes and SAP Cloud Deployment service reports the status to the developer/operator.

Content Deployer and Service Instance

The content deployer and service instance are provided by the SAP business application that uses the business content. This way, the content deployer will know how to communicate to the backend on which the content will be deployed. Thus the SAP Cloud Deployment service is isolated from the actual content deployment protocol and implementation.

Normally, the content deployers are simple clients that implement specific REST APIs to pass the content. When the content application is started it invokes a content deployer which calls certain endpoints and executes the actual content deployment.

The content deployer identifies the content endpoint based on the service binding. Content endpoint could be any URL either provided by an application in BTP Cloud Foundry or one outside of the BTP entirely.

It is possible that a content application will have bindings to multiple service instances to complete the content deployment. For example, there could be a reference to an XSUAA service instance, which will ensure secure communication between the content deployer and content endpoint.

Success Marker and Failure Marker

Success and failure markers are part of the protocol between SAP Cloud Deployment service and the content deployers.

They are used to indicate when the content deployment has reached a final state. Content deployers must include the markers in its logs. Then SAP Cloud Deployment service observes the application logs (STDOUT or STDERR) and decides if the content deployment has finished successfully or failed.

Success and failure markers are predefined and must not be changed by the end customer. They are represented by the module level parameters `success-marker` and `failure-marker`.

For instance, module type `com.sap.portal.site-content` has a success-marker `STDOUT:Deployment of site content to .* done.*`

MTA Modelling

Below you can find the modelling for supporting the content deployment scenario for MTA module type `com.sap.portal.site-content`:

Sample Code

```
_schema-version: "3.1"
ID: my-portal
version: "1.1"
modules:
   - name: site-content
    type: com.sap.portal.site-content
    path: site-content/
    parameters:
      buildpack: nodejs_buildpack # Predefined parameter
      no-route: true # Predefined parameter - avoid route mapping to the
content app. It will not serve any HTTP requests
      memory: 256M # Predefined parameter – used memory
      execute-app: true # Predefined parameter – flag that marks the app to
be used for simulated app execution
      success-marker: STDOUT:Deployment of site content to .* done.* #
Predefined parameter – marker for success
      failure-marker: STDERR:Deployment of site content to .* failed.* #
Predefined parameter – marker for failure
      stop-app: true # Predefined parameter – flag that specifies if the app
will be stopped after the actual deployment
```

```
      check-deploy-id: true # Predefined parameter – flag that specifies that
certain and unique id will be checked for that content deployment
      dependency-type: hard
      health-check-type: none # Predefined parameter – skips the CF health-
check, so content app will not implement specific endpoint
    requires:
       - name: sap-portal-services-client
       - name: portal-uaa


resources:
    - name: sap-portal-services-client # Provides content endpoint for the
actual content deployment. Content endpoint could be different based on the
account
    type: com.sap.portal.site-content
    parameters:
      config:
        siteId : 1234

  - name: portal-uaa # Used for secure communication between content app and
content endpoint
    type: com.sap.xs.uaa-space
    parameters:
      config-path: security/xs-security.json
```

4.1.8.1.9.1.4 CPI Content Deployment

This approach provides a mechanism for direct content deployment from the SAP Cloud Deployment service to content backend without the need for an intermediate Cloud Foundry application.

> ⓘ Note
>
> This approach is only supported for MTA schema version 3.1 and higher.

This mechanism utilizes the CPI provided protocol. The end-to-end flow and MTA modelling are very similar to Deploying Content with Generic Application Content Deployment [page 770].

Supported content type offerings

Cloud Foundry service offerings that currently support GACD are:

- API Management content - MTA module type `com.sap.api.management`
- CPI content – MTA module type `com.sap.integration`

4.1.8.1.9.2 Blue-Green Deployment of Multitarget Applications

Use the blue-green deployment technique by running two identical production environments, allowing seamless updates without downtime for Cloud Foundry multitarget applications.

> ⚠ Restriction
>
> Blue-green deployment is supported only for Cloud Foundry applications. It is not supported for bound services, such as service instances and their configuration, workflow content, and HTML5 repository content, among others. Live and idle applications are bound to the same service instances.

By using the blue-green deployment technique, you can update the system without downtime and with reduced risk. During the process, you can perform tests and validation of the new application version using productive data. A classical blue-green deployment of stateless applications, which are modeled as an MTA, is supported.

In the context of multitarget applications, you have the following options for using blue-green deployment:

- Legacy Blue-Green Deployment [page 785] - where the production environments are called “blue” and “green”
- Blue-Green Deployment Strategy [page 788] - where the production environments are called “live” and “idle”
- Incremental Blue-Green Deployment Strategy [page 791] - an alternative to the standard blue-green deployment strategy, where the "idle" application is incrementally scaled up to the desired instance count while the "live" application instances are gradually stopped

Related Information

Using Blue-Green Deployment To Reduce Downtime

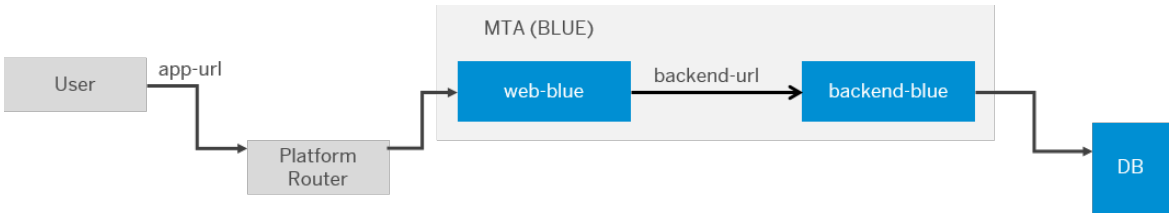
4.1.8.1.9.2.1 Legacy Blue-Green Deployment

Use the legacy blue-green deployment strategy of multitarget applications.

Prerequisites

> ⚠ Restriction
>
> Blue-green deployment is supported only for Cloud Foundry applications. It is not supported for bound services, such as service instances and their configuration, workflow content, and HTML5 repository content, among others. Live and idle applications are bound to the same service instances.

- You have a previously deployed MTA, with functional productive applications and routes:



- You have ensured that the changes in the new version of the application are compatible with the old version. This includes changes in the database tables, UAA configurations, and so on.

Procedure

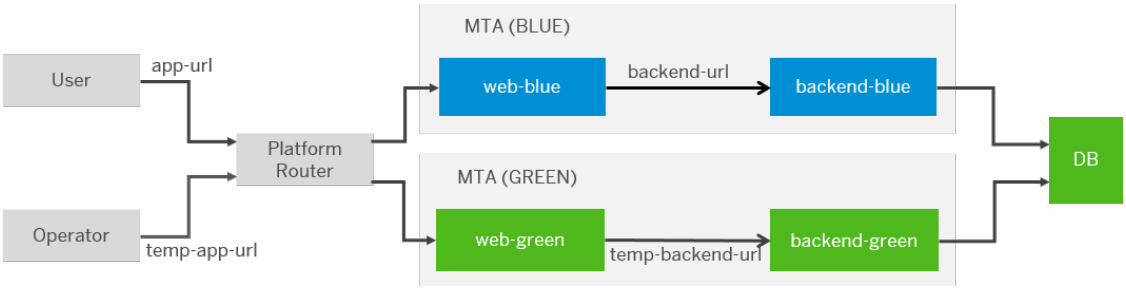
1. Deploy your updated MTA (the green version) by executing the `cf bg-deploy <your-mta-archive-v2>` command.

   > **Note**
   >
   > If you want to have the opportunity to perform a rollback after the blue-green deployment, you need to back up your MTA by adding the command line option `--backup-previous-version`. For more information, see (Experimental) Rollback of Multitarget Applications [page 794].

   The first action is that all MTA services are updated.

   This action:

   - creates new applications adding “green” to their existing application names
   - creates temporary routes to the “green” applications.



   - interrupts the process showing a message similar to the following:

   > **Output Code**
   >
   ```
   Process needs additional user input
   Use "cf bg-deploy -i 469520 -a resume" to resume the process
   Use "cf bg-deploy -i 469520 -a abort" to abort the process
   ```

2. Optionally, test the “green” version using the temporary routes.

   > **Note**
   >
   > You can skip this step by using one of the following command line options:
   >
   > - `--no-confirm` - you have to use it when starting the process.

- `--skip-idle-start` - this option will also skip the start of the newly deployed applications on temporary routes.

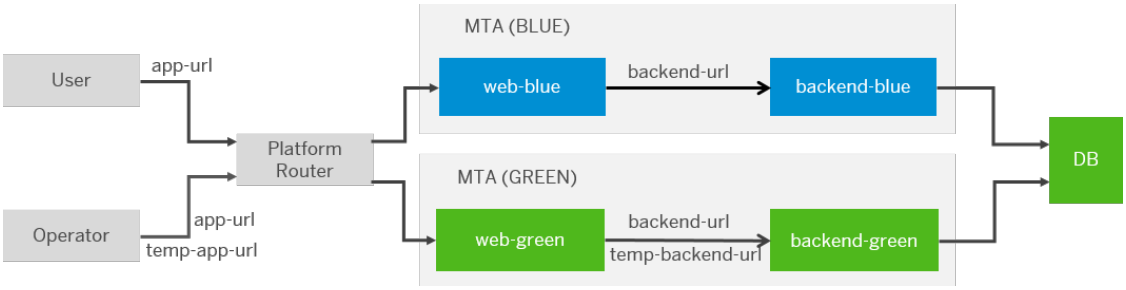
3. If you do not want to make the “green” version available, abort the process using `cf deploy -i <operation ID> -a abort`.

> ### Note
>
> This action does not perform a rollback and the state of apps, routes and services remains unchaged. Depending on your needs, you might want to remove the new app versions and the temporary routes.

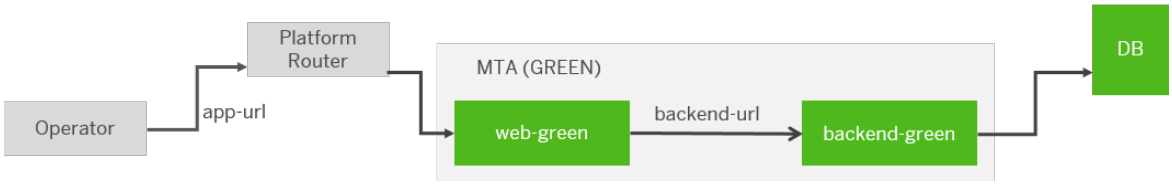
4. To make the “green” version productive, choose *Resume*.

   This action:

   - maps the productive routes to your green versions



   - deletes the temporary routes
   - restarts “green” apps with productive route configurations
   - restarts the applications, so that the environment URLs are updated
   - deletes the “blue” applications, which were productive before



Results

For more detailed information and example results of the legacy blue-green deployment process, see Legacy Blue-Green Deployment .

Related Information

Blue-Green Deployment of Multitarget Applications [page 785]
Blue-Green Deployment Strategy [page 788]

Incremental Blue-Green Deployment Strategy [page 791]

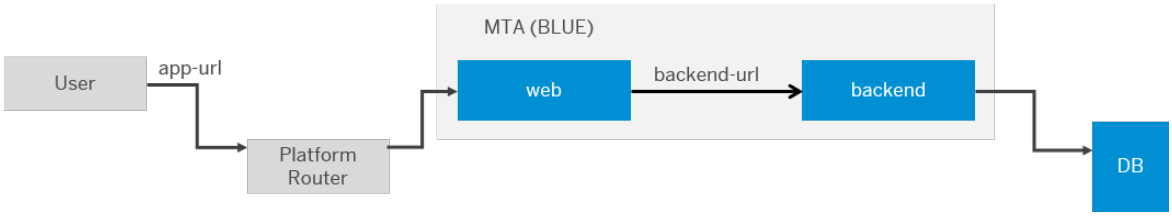
4.1.8.1.9.2.2 Blue-Green Deployment Strategy

Use the current blue-green deployment of multitarget applications.

Prerequisites

> ⚠ Restriction
>
> Blue-green deployment is supported only for Cloud Foundry applications. It is not supported for bound services, such as service instances and their configuration, workflow content, and HTML5 repository content, among others. Live and idle applications are bound to the same service instances.

- You have a previously deployed MTA, with functional productive applications and routes:



- You have ensured that the changes in the new version of the application are compatible with the old version. This includes changes in the database tables, UAA configurations, and so on.

Context

If you have already executed successful deployments using the Legacy Blue-Green Deployment [page 785] method ('`cf bg-deploy`'), you should have no problems when executing deployments with the blue-green deployment strategy ('`cf deploy --strategy blue-green`'). However, this does not apply the other way around and it is generally not recommended to go back to the legacy blue-green deployment method.

> ⚠ Caution
>
> Do not begin a blue-green deployment using '`cf bg-deploy`' and continue with '`cf deploy --strategy blue-green`', or vice versa, as this might result in downtime.

Procedure

1. Deploy your updated MTA in idle state by executing the command `cf deploy <your-mta-archive-v2> --strategy blue-green`.

> ### Note
>
> If you want to have the opportunity to perform a rollback after the blue-green deployment, you need to back up your MTA by adding the command line option `--backup-previous-version`. For more information, see (Experimental) Rollback of Multitarget Applications [page 794].

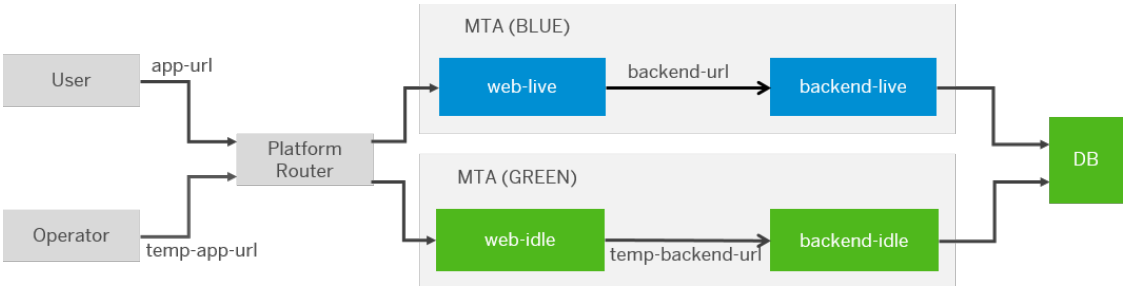
The first action is that all MTA services are updated.

This creates:

- new applications adding “idle” to the original application names

  > ### Note
  >
  > The new application version is launched on all instances defined in the MTA deployment descriptor. During the testing phase or when the deployment is paused for user verification, the application temporarily requires double the usual resources (instances). This is because both the old and the new versions of the application are running concurrently. If the testing phase is skipped or if the deployment continues without delay, the resource usage will be optimized sooner.
  >
  > If you are looking for a more resource-efficient deployment approach, see Incremental Blue-Green Deployment Strategy [page 791]. However, keep in mind that this approach requires more time than the standard blue-green deployment strategy.

- temporary routes to the idle applications



- an interrupt to the process showing a message similar to the following:

  > ### Output Code
  >
  > ```
  > Process has entered testing phase. After testing your new deployment
  > you can resume or abort the process.
  > Use "cf deploy -i 7e6233b9-6001-11ea-8959-eeee0a98a87d -a abort" to
  > abort the process.
  > Use "cf deploy -i 7e6233b9-6001-11ea-8959-eeee0a98a87d -a resume" to
  > resume the process.
  > Hint: Use the '--skip-testing-phase' option of the deploy command to
  > skip this phase
  > ```

2. Optionally, test the idle version of the application using the temporary routes.

> ### Note
>
> You can skip this step by using one of the following command line options:
>
> - --skip-testing-phase - you have to use it when starting the process.
> - --skip-idle-start - this option will also skip the start of the newly deployed applications on idle routes.

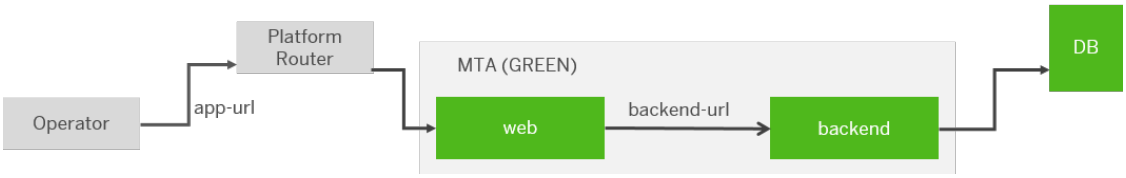
3. If you do not want to make the “idle” version available, abort the process using cf deploy **`-i <operation ID> -a abort`**.

   > **Note**
   >
   > This action does not perform a rollback and the state of apps, routes and services remains unchanged. Depending on your needs, you might want to remove the new app versions and the temporary routes.

4. If you want to make the “idle” version productive, manually resume the process, using **`cf deploy -i <operation ID> -a resume`**.

   This does the following:

   - maps the productive routes to your idle versions
   - deletes the temporary routes
   - restarts “idle” apps with productive route configurations
   - deletes the “live” applications, which were productive before



Results

For more detailed information and example results of the blue-green deployment process, see Blue-Green Deployment .

Related Information

Blue-Green Deployment of Multitarget Applications [page 785]
Legacy Blue-Green Deployment [page 785]
Incremental Blue-Green Deployment Strategy [page 791]
Multitarget Application Commands for the Cloud Foundry Environment [page 2834]

4.1.8.1.9.2.3 Incremental Blue-Green Deployment Strategy

Use the incremental blue-green deployment strategy to save resources by incrementally deploying the new version of your multitarget application.

General Information

> ⚠ Restriction
>
> Blue-green deployment is supported only for Cloud Foundry applications. It is not supported for bound services, such as service instances and their configuration, workflow content, and HTML5 repository content, among others. Live and idle applications are bound to the same service instances.

During the incremental blue-green deployment, one instance of the live application is stopped and one new instance of the new application is started at a time, until the new application is started on all instances. This strategy ensures a smooth transition between the old and new versions of an application. The process consists of the following steps:

1. **Initial Setup**: The new version of the application is deployed to only one instance.
2. **Testing Phase**: The deployment process enters the validation phase, where users or automated systems can test the new application.
3. **User Acceptance**: Customers or designated testers evaluate the new application.
4. **Deployment Continuation**: Once testing results are satisfactory, the deployment process resumes.
5. **Incremental Scaling**:
   - One instance of the old application is stopped.
   - One instance of the new application is started.

   This step is repeated until the new application reaches the desired number of instances.

   > ⓘ Note
   >
   > During the incremental blue-green deployment, the total number of instances (old + new) will not exceed **N + 1**, where **N** is the desired instance count for the new application.

   If an instance crashes during this step, a "rollback" procedure is initiated:
   - The old application is scaled back up to its initial instance count.
   - The new application is scaled down to one instance.

   This rollback ensures system stability by restoring the previous state of the applications.
6. **Completion**: After the new application is fully scaled up, the remaining instances of the old application are stopped and removed.

> ⓘ Note
>
> If you are using the Application Autoscaler service, note the following:
>
> - The new application will not be autoscaled during the incremental blue-green deployment process. The autoscaling will start after the deployment process completes.
> - Once the deployment process resumes and enters the **Incremental Scaling** phase, autoscaling will also be disabled for the live application.

This ensures that neither the idle nor the live applications are rescaled by the Autoscaler, preventing potential inconsistencies.

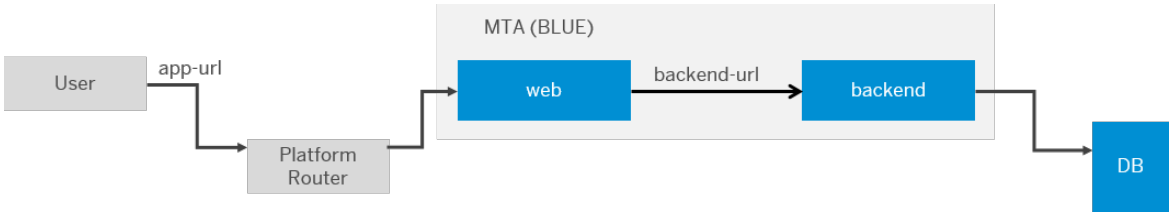
The incremental blue-green deployment has the following advantages and disadvantages:

Pros	Cons
The incremental blue-green deployment requires less memory resource than the standard blue-green deployment, which means that it could save memory quota and reduce costs. In certain scenarios, services may calculate monthly costs based on the maximum number of instances associated with an application. In a standard blue-green deployment, where both versions of the application run simultaneously, this can temporarily double the number of instances, potentially leading to higher charges. It is advisable to consult with the service provider, to understand the specific pricing implications for end customers.	The incremental blue-green deployment requires more time to complete than the normal blue-green deployment, as the new application instances are started sequentially, which means that it waits for every new instance to completely start before starting the next one.

To perform an incremental blue-green deployment, follow the steps in the sections below.

Prerequisites

- You have a previously deployed MTA, with functional productive applications and routes:



- You have ensured that the changes in the new version of the application are compatible with the old version. This includes changes in the database tables, UAA configurations, and so on.

Context

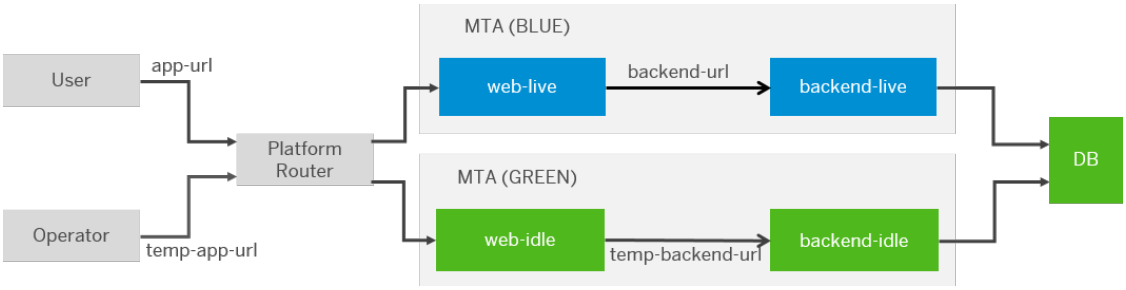
If you have already executed successful deployments using the Blue-Green Deployment Strategy [page 788] (`cf deploy --strategy blue-green`), you should have no problems when executing deployments with the incremental blue-green deployment strategy (`cf deploy --strategy incremental-blue-green`). This also applies if you decide to go back to the blue-green deployment strategy.

However, this does not apply for the Legacy Blue-Green Deployment [page 785] (`cf bg-deploy`). It is not recommended to go back to this deployment method.

> ⚠ Caution
>
> Do not begin a blue-green deployment using '`cf bg-deploy`' and continue with '`cf deploy --strategy blue-green / incremental-blue-green`', or vice versa, as this might result in downtime.

Procedure

1. Deploy your updated MTA in idle state by executing the command `cf deploy <your-mta-archive-v2> --strategy incremental-blue-green`.
   The first action is that all MTA services are updated.
   This creates:
   - one instance of the new application with the suffix “idle” added to the original application name
   - temporary routes to this idle application instance



   - an interrupt to the process showing a message similar to the following:

> {·} Output Code
>
> ```
> Process has entered testing phase. After testing your new deployment
> you can resume or abort the process.
> Use "cf deploy -i 7e6233b9-6001-11ea-8959-eeee0a98a87d -a abort" to
> abort the process.
> Use "cf deploy -i 7e6233b9-6001-11ea-8959-eeee0a98a87d -a resume" to
> resume the process.
> Hint: Use the '--skip-testing-phase' option of the deploy command to
> skip this phase
> ```

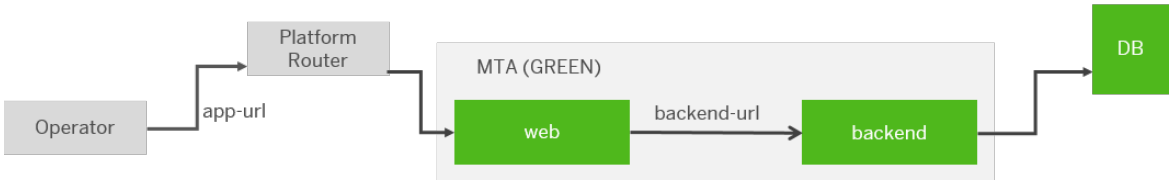
2. Optionally, test the idle version of the application using the temporary routes.

> → Tip
>
> You can skip this step by using one of the following command line options:
>
> - `--skip-testing-phase` - this option will not wait for user confirmation after testing.
> - `--skip-idle-start` - this option will also skip the start of the newly deployed application on idle routes and will not wait for user confirmation.

3. If you do not want to make the “idle” version available, abort the process by using `cf deploy -i <operation ID> -a abort`.

> ⓘ Note
>
> This action does not perform a rollback and the state of apps, routes, and services remains unchanged. Depending on your needs, you might want to remove the new app versions and the temporary routes.

4. If you want to make the “idle” version productive, manually resume the process by using `cf deploy -i <operation ID> -a resume`.
   This does the following:
   - maps the productive routes to your idle application versions
   - deletes the temporary routes of the idle application
   - restarts idle apps with productive route configurations
   - incrementally stops one instance of the live application and starts a new instance of the new application until it is started on all required instances
   - deletes the live applications which were productive before



Examples

For more detailed information and example results of the incremental blue-green deployment process, see Blue-Green Deployment with incremental-blue-green strategy .

Related Information

Blue-Green Deployment Strategy [page 788]
Legacy Blue-Green Deployment [page 785]
Multitarget Application Commands for the Cloud Foundry Environment [page 2834]

4.1.8.1.9.3 (Experimental) Rollback of Multitarget Applications

Use the rollback feature if you need to switch the productive traffic to a previously deployed backed-up version of your multitarget application.

> **Note**
>
> This is an experimental feature. Experimental features aren't part of the officially delivered scope that SAP guarantees for future releases. For more information, see Important Disclaimers and Legal Information.
>
> Please use the *Feedback* button in this topic to let us know what you like and don't, and how we can improve it to make the experience more enjoyable for you.

General Information

The rollback feature is Cloud Foundry application-centric, and it aims to ensure smooth transition of all productive traffic from the currently deployed version to the previously deployed version of the MTA. To achieve this with reduced risk and without downtime, the rollback represents a simplified blue-green deployment based on:

- the MTA descriptors used in the deployment of the previous MTA version.
- the backed-up Cloud Foundry applications from the previously deployed MTA version.

> ⚠ Restriction
>
> - Rollback is supported only for Cloud Foundry applications and bound user-provided services. It is not supported for managed service instances and their configuration, workflow content, and HTML5 repository content, among others.
> - Rollback (and consequently, backup) is supported only when an MTA is updated with blue-green deployment (Legacy Blue-Green Deployment [page 785] or Blue-Green Deployment Strategy [page 788]). It is not supported when the update is done with standard deployment where downtime is expected anyway.

Backup of Multitarget Applications

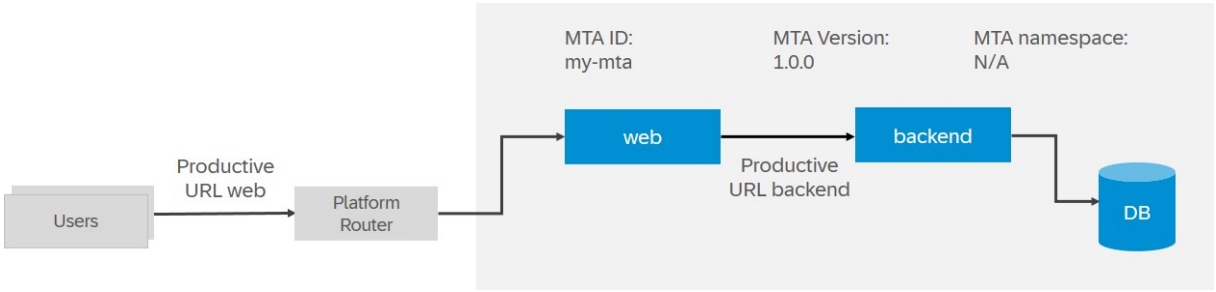
To be able to perform a rollback to a previously deployed version of your MTA, this version needs to be backed up beforehand during the blue-green deployment.

The backup is based on the namespaces feature (see Namespaces [page 798]). The backed-up MTA is put in a dedicated namespace with the default name "`mta-backup`", which means that your MTA will receive the prefix "`mta-backup`". The Cloud Foundry applications and service instances in your MTA will also receive the "`mta-backup`" prefix. However, if you use a namespace "`<your-namespace>`" to deploy your productive apps, the backup namespace will be "`mta-backup-<your-namespace>`", and accordingly, this will be the prefix.

> ⓘ Note
>
> It is important that you are aware of the following notes and restrictions:
>
> - The data needed to perform a rollback is backed up for 3 days.
> - The backup is based on the MTA version. It is done only if the version is changed, so make sure you use a new version for every release of the MTA.
> - There might be only one backup per MTA which corresponds to its previously deployed MTA version. If the same MTA version is being re-deployed, the preserved backup will not be overwritten. This ensures that triggering a rollback will revert the traffic to a different previous version and not to the same one.

Procedure

This is the initial state of your previously deployed MTA with productive applications and routes before the backup:



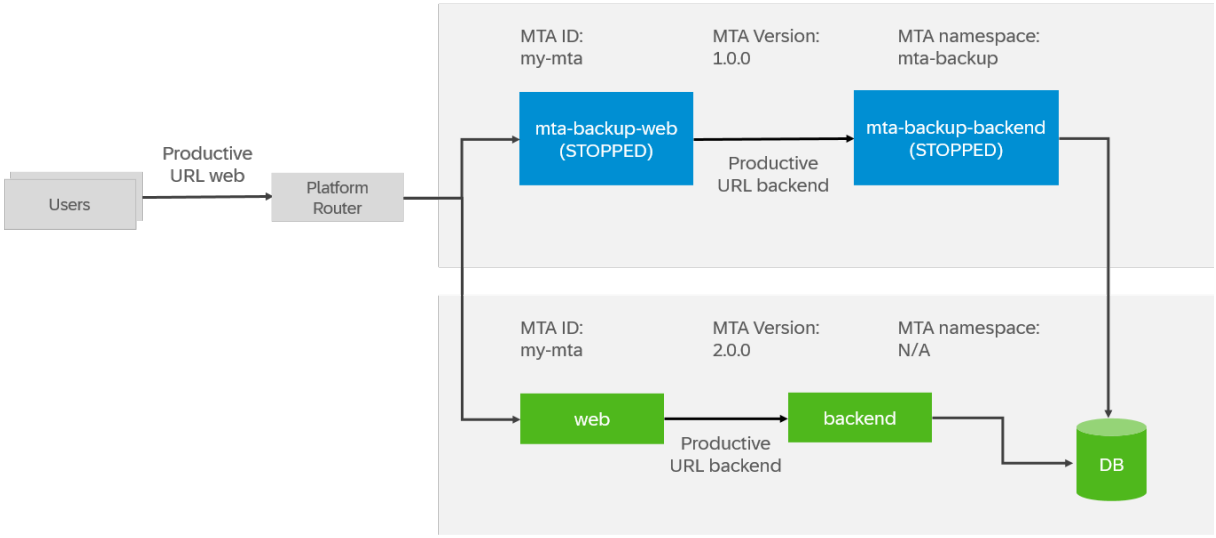
To back up your MTA, you have to add the command-line option `--backup-previous-version` when initiating the blue-green deployment:

```
cf deploy <your-mta-archive-v2> --strategy blue-green --backup-previous-version
```

The result of the operation includes all steps of blue-green deployment and the following additional ones:

- The previously deployed MTA version is moved to the dedicated namespace with name “mta-backup” and the “mta-backup” prefix is added to the names of all Cloud Foundry applications.
- The Cloud Foundry applications remain in STOPPED state. In this way, they are ready to be reused in an eventual rollback but will not inquire any additional costs in the meantime.

This is the state of the MTA after the backup:



Rollback of Multitarget Applications

The MTA rollback mechanism switches the productive traffic back to the previously deployed MTA. The rollback processes the combination of the MTA descriptors which were used for deployment of the previous MTA version. This results in processing the different MTA modules and resources in the order defined in the previous

descriptors. However, the rollback does not include any application-specific or service-specific processing, such as reverting of a database schema.

> → Remember
>
> You can perform a rollback 3 days after backing up and deploying your MTA. If you try to perform a rollback after this period, the operation will fail. However, the backed-up applications will still be available in your space.

Procedure

To initiate the rollback, you need to execute the command:

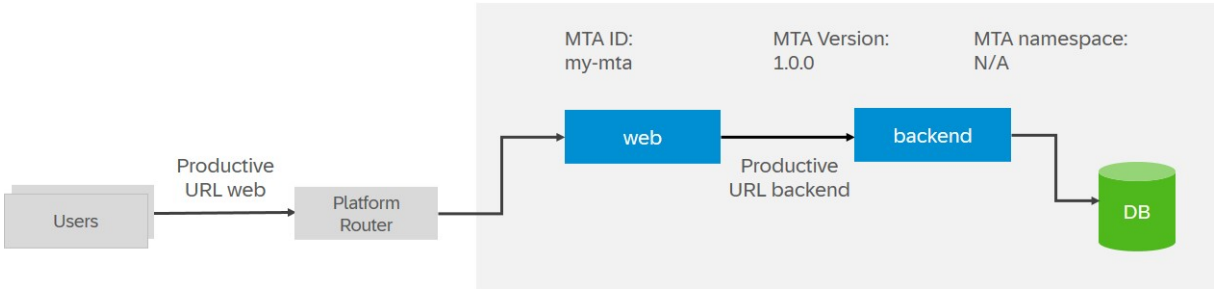
```
cf rollback-mta <MTA_ID>
```

> ⓘ Note
>
> To process user-provided services as well, pass the command-line option `--process-user-provided-services`. For more information about the available command-line options, see rollback-mta [page 2843].

The results of this operation are:

- The previously deployed MTA version is moved out of dedicated namespace "`mta-backup`".
- The applications are rebound to the service instances.
- The MTA modules and resources are processed in the order defined in the previous version of the MTA descriptors
- The newly deployed MTA version is removed, including the Cloud Foundry applications
- (Optional) User-provided services are processed.

This is the state of the MTA after the rollback:



Related Information

Namespaces [page 798]
Blue-Green Deployment of Multitarget Applications [page 785]
Multitarget Application Commands for the Cloud Foundry Environment [page 2834]

4.1.8.1.9.4 Namespaces

Use this feature to prevent conflicts for applications deriving from the same MTA, but with different version, features, and configuration.

> ⚠ Restriction
>
> - The namespace can only be passed as an argument to the deployment call, and cannot be modelled directly in the descriptor.
> - The namespace cannot be longer than 36 characters.

Prerequisites

- Make sure the version of your MultiApps Plug-in is 2.5.0 or higher. If required, use the procedure described in Install the MultiApps CLI Plugin in the Cloud Foundry Environment [page 2833] to update it.

Feature Description

By using the namespaces feature, you can deploy and operate a single multitarget applicaton multiple times within a single space. This is useful when you have to avoid conflicts among applications or entities. It modifies the modules and resources by adding a namespace at the beginning of the application names, service names, and routes.

The feature is employed by adding the parameter `--namespace`, when you input the deployment command in the Cloud Foundry Command Line Interface (CF CLI), and by modifying the deployment descriptor as described below. Using a more detailed configuration, you can include or exclude specific resources from having the namespace feature applied to them, namely:

- resources
- modules
- routes.

When set so, the excluded resources are shared among all namespaces of this MTA archive.

Fine-Grained Configuration

You can configure the application of namespaces at different levels according to your needs. When the same `apply-namespace` parameter is configured in several places for one MTA deployment, a certain order of priority applies. This order provides flexibility if you want to set a namespace for all applications, application routes, and services that are part of the MTA, but at the same time, you need to define a specific namespace for a particular application, application route, or service.

The priority for `apply-namespace` parameters in descending order is the following:

1. **Operation parameters / Command-Line Options (Highest Priority)**
   Operation parameters passed during the MTA deployment have the highest priority. If you provide an `apply-namespace` parameter as a command-line option, it overrides any other parameters defined in the MTA descriptor. Operation parameters apply to all applications.
   There are four available options for applying a namespace:
   - `--apply-namespace-app-names true/false`: Applies namespace to application names.
   - `--apply-namespace-service-names true/false`: Applies namespace to service names.
   - `--apply-namespace-app-routes true/false`: Applies namespace to application routes.
   - `--apply-namespace-as-suffix true/false`: Applies namespace either as prefix or suffix.

   Example:
   When you deploy using the command `cf deploy … --namespace test --apply-namespace-app-names true --apply-namespace-service-names false --apply-namespace-app-routes true`, all application names and application routes will have the prefix '`test-`'. Service names will remain unchanged, as specified by the `--apply-namespace-service-names false` option. Other `apply-namespace` parameters will not affect this configuration.
   If you use the command `cf deploy … --apply-namespace-app-names true --apply-namespace-service-names false --apply-namespace-app-routes true` without the '`--namespace`' option, the `apply-namespace` options will be ignored.
   For more information about `apply-namespace` options, refer to the **Command Options Overview** table in deploy.
2. **Module-Level and Resource-Level Parameters**
   If no command-line argument is provided, the module-level and resource-level parameters are considered next.
   Example:

   Sample Code

   ```
   modules:
     - name: java
       ...........
       parameters:
         apply-namespace: true
         routes:
            - route: host.some-domain.com
              apply-namespace: true
   resources:
     - name: my-service
       ...........
       parameters:
         apply-namespace: false
   ```

3. **Top-Level Parameters**
   If neither command-line options nor module-level parameter are provided, the top-level parameters are used. Top-level parameters apply to all applications.

   Sample Code

   ```
   parameters:
     apply-namespace:
       app-names: true
       service-names: false
       app-routes: true
   ```

4. **Default Values**
   If you don't provide any of the above mentioned parameters, the application will use the predefined default values. When you set the '`--namespace`' option, the default value is `true`, otherwise, it's `false`.

Examples

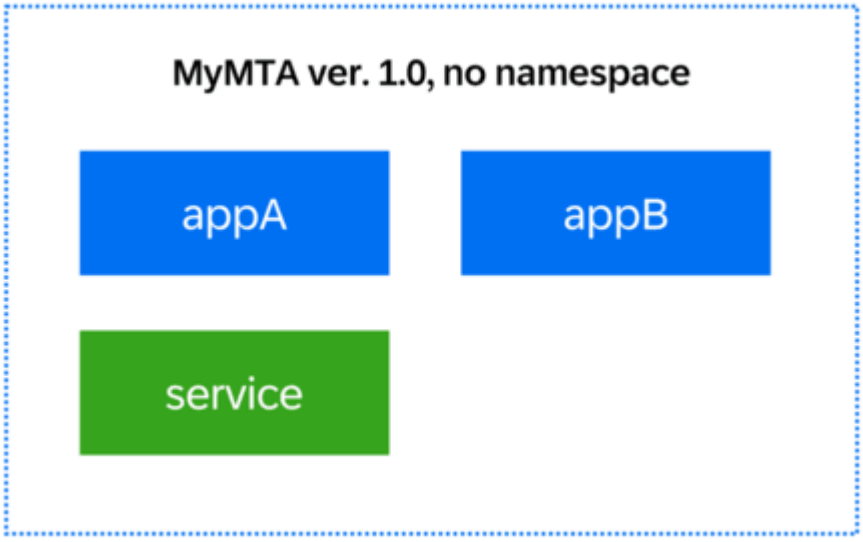
To deploy several entities of a multitarget application using namespaces, follow these steps:

1. Using the CF CLI, deploy the initial MTA archive by using the command `cf deploy ./MyMTA.mtar` with the descriptor given below:

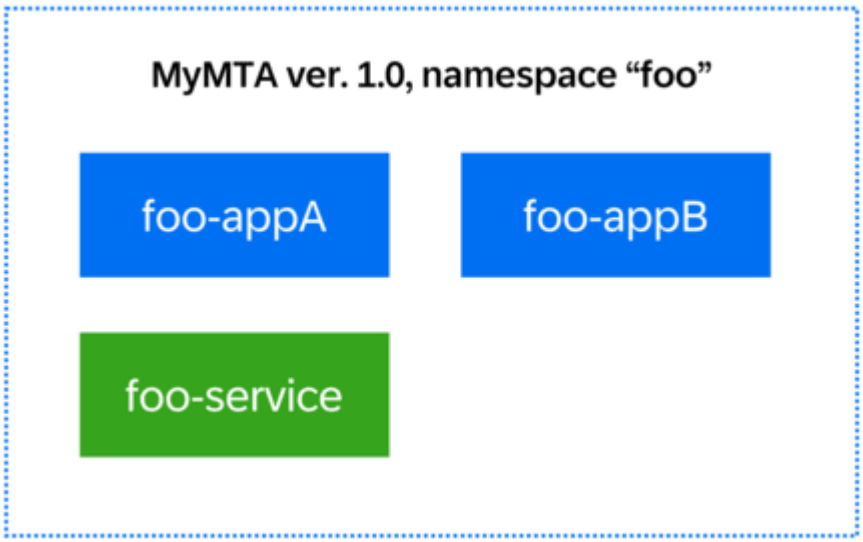
   Sample Code

   ```
   modules:
     - name: appA
       type: java.tomee
       path: application.war
       requires:
         - name: service
       parameters:
         routes:
         - route: appA.${default-domain}
         - route: route-without-namespace.${default-domain}
     - name: appB
       type: java.tomee
       path: application.war
   resources:
     - name: service
   ```

   The result should match the image below:

2. Deploy again the MTA archive by using the command `cf deploy ./MyMTA.mtar --namespace foo` with the same descriptor. The result should match the image shown below:



If you bind 'appA' to 'appA.my-domain.com', the system automatically binds 'foo-appA' to 'foo-appA.my-domain.com' upon deployment. If 'appA' requires a service instance called 'service', 'foo-appA' binds to a different service instance named 'foo-service'.
There are also other ways to achieve this result, for example:

- Deploy using the `cf deploy` command with the following options:

  ```
  cf deploy ./MyMTA.mtar --namespace foo --apply-namespace-app-names true --
  apply-namespace-service-names true --apply-namespace-app-routes true
  ```

  This command deploys the MTA archive 'MyMTA.mtar' and applies the namespace 'foo' to the application names, service names, and application routes.
- Set top-level parameters in the deployment descriptor:
  If you don't have operation parameters or module-level and resource-level parameters, you can set the top-level parameters in the deployment descriptor to `true`. This approach produces the same result as using the `cf deploy` command with the namespace options.

Both methods ensure that the namespace is applied consistently across application names, service names, and application routes during the deployment process.

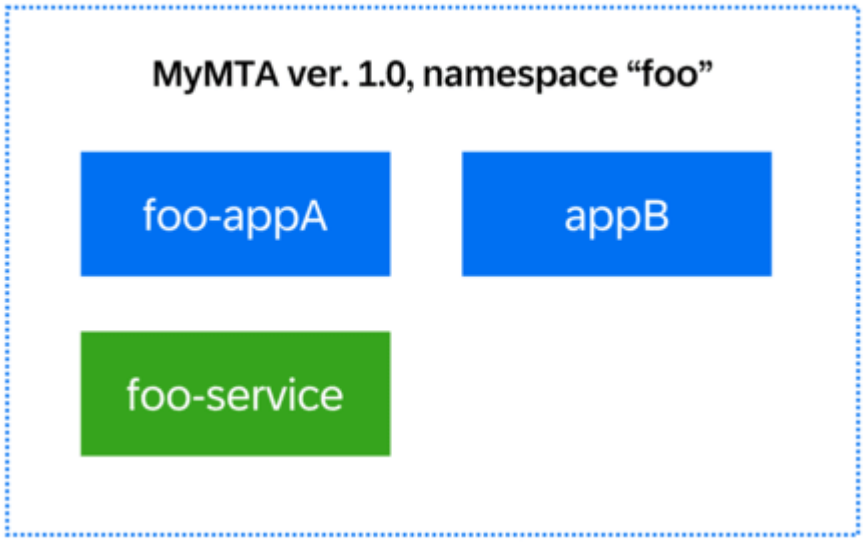
3. Deploy again the MTA archive by using the command `cf deploy ./MyMTA.mtar --namespace foo` with the given descriptor below:

{·} Sample Code

```
modules:
  - name: appA
    type: java.tomee
    path: application.war
    requires:
      - name: service
    parameters:
      routes:
      - route: appA.${default-domain}
        apply-namespace: true
  - name: appB
    type: java.tomee
    path: application.war
    parameters:
      apply-namespace: false
resources:
  - name: service
```

```
    parameters:
      apply-namespace: true
```

The result matches the image below:



The deployment automatically binds 'foo-appA' to 'foo-appA.my-domain.com'.
If the descriptor contains:

```
routes:
  - route: appA.${default-domain}
    apply-namespace: false
```

then 'foo-appA' automatically binds to 'appA.my-domain.com' upon deployment instead.

Special Use: Sharing One Resource Among Several MTAs Using Namespaces

You can use deployment with namespaces in a way that only one of the resources receives a namespace. To do so, you use the parameter `apply-namespace` in the MTA deployment descriptor follows, and then deploy by using the procedure above. The deployment result should be as follows:

For the example below we use a service, but this special use can be applied to any module or resource in the descriptor.

Sample Code

```
...
  - name: serviceA
    type: auditlog
    parameters:
      apply-namespace: false
...
```

In the snippet above, the `apply-namespace: false` overrides the default value `true`, thus prohibiting the creation of additional instances of the service. This means that the service is unique in the space across all deployments of the MTA, and shared among them.

To see a practical example of the implementation above, go to Deploying MTA in a Namespace .

4.1.8.1.9.5 Parallel Module Deployment

In some cases, it is crucial that modules and therefore applications are deployed in a predictable and consistent order. To ensure this, the module-level attribute `deployed-after` has been introduced. It contains a list of module names. If a module has this attribute, it will be deployed only after the modules specified in the attribute have already been deployed.

The relations described through this attribute are transitive, so if module A should be deployed after module B, and module B should be deployed after module C, then it means that module A should be deployed after module C.

Sample Code

MTA Deployment Descriptor (`mtad.yaml`)

```
ID: com.sap.sample
version: 0.1.0
_schema-version: "3.2.0"
parameters:
  enable-parallel-deployments: true

modules:
  - name: ui
    type: javascript
    deployed-after: [ backend, metrics ]

  - name: backend
    type: java
    deployed-after: [ hdi-content ]
    requires:
      - name: metrics
        properties:
          METRICS_URL: ~{url}

  - name: metrics
    type: javascript
    deployed-after: [ hdi-content ]
    provides:
      - name: metrics
        properties:
          METRICS_URL: ~{url}

  - name: hdi-content
    type: hdi
```

In the example above, the `deployed-after` attributes guarantee that the `ui` module is deployed after the `backend` and the `metrics` modules, and they in turn are deployed after the `hdi-content` module. Note that the deployent order of the `backend` and the `metrics` modules is not specified in the attributes. This means that they can be deployed in any order.

Parallel Deployment

> **Note**
>
> The `deployed-after` parameter is supported from major schema version 3 onwards.

In the example above, we have also specified the top-level MTA parameter `enable-parallel-deployments` with value `true`. It activates the parallel deployment of MTA modules that do not have any deployment order dependencies between each other. If the parameter is missing or its value is `false`, the module deployment will be sequential.

> **Note**
>
> The `enable-parallel-deployments` parameter has influence only on modules deployment, see section Sequential Resource Processing for the resources process.

> **Note**
>
> The `enable-parallel-deployments` parameter is supported from major schema version 3 onwards.

Circular Dependencies

Due to a modelling error, the user can introduce direct or transitive circular deployment order dependencies between modules. In such cases, this will be reported as a deployment error.

Compatibility with Previous Deployment Order

The previous deployment order algorithm was based on the `requires` module-level attribute, which contains a list of module names or provided dependencies from other modules. Since it is also the way to model configuration dependencies, this mechanism was not explicit enough to model a deployment order .

There are many applications that are still depending on the old deployment order algorithm. To support them until they adapt to the new modeling, the new deployment order is introduced in a backward compatible manner. This means that if there are no `deployed-after` module-level elements in the MTA descriptor and the top-level MTA parameter `enable-parallel-deployments` is set to `false` or is missing, the old ordering algorithm is enabled by default.

4.1.8.1.9.6 Sequential Resource Processing

In various cases, some resources depend on other resources to already exist in order to process. Whenever an order of resource processing is necessary, the `processed-after` attribute can be utilized.

This attribute contains a list of resource names. It creates a sequence of services, an order, in which the services should be processed. If a resource has this attribute, it will be processed after the other resources in a higher position are processed.

> ### Note
>
> If the parameter `processed-after` is not specified, the resources are processed in parallel.

> ### Note
>
> The `processed-after` parameter is supported from schema version 3 onwards. If you attempt to use `processed-after` with schema version lower than 3, the deployment will ignore the attribute.

Example: Service A requires service B and service C to be processed first. On the other hand, both service B and service C require service D to be processed first. Service D does not require any other service to be proccessed. So the sequence for processing is - D → B, C → A.

> ### Sample Code

```
MTA Deployment Descriptor (mtad.yaml)

_schema-version: 3
ID: multiple-anatz
version: 3.0.0

modules:
  - name: multiple-anatz
    type: staticfile
resources:
  - name: serviceA
    type: org.cloudfoundry.managed-service
    parameters:
      service: application-logs
      service-plan: lite
    processed-after: [ serviceB, serviceC ]

  - name: serviceB
    type: org.cloudfoundry.managed-service
    parameters:
      service: application-logs
      service-plan: lite
    processed-after: [ serviceD ]

  - name: serviceC
    type: org.cloudfoundry.managed-service
    parameters:
      service: application-logs
      service-plan: lite
    processed-after: [ serviceD ]

  - name: serviceD
    type: org.cloudfoundry.managed-service
    parameters:
      service: application-logs
      service-plan: lite
```

In the example, the `processed-after` attribute ensures that serviceD is processed first, then serviceC and serviceB are processed in parallel, and finally serviceA is processed.

Circular Dependencies

Due to a modelling error, you can create direct or transitive circular deployment order dependencies between resources. In such cases, it is reported as a deployment error.

4.1.8.1.9.7 Cross-MTA Dependencies

In addition to having dependencies to one or multiple modules or resources in the same MTA, modules can have dependencies with modules from other MTAs as well. For these so-called cross-MTA dependencies to work, the MTA that **provides** the dependencies must be set to declare them as "public". Use the `configuration` resource to declare when one module consumes configurations from other MTAs.

> ⚠ Caution
>
> Do not use this feature for storing sensitive content, for example credentials, as this data is stored unencrypted.

> ⓘ Note
>
> The declaration method requires adding a resource in the deployment descriptor; the additional resource defines the provided dependency from the other MTA.

Cross-MTA Dependency Method: Resource Type "configuration"

This method can be used to access any entry that is present in the configuration registry. The parameters used in this cross-MTA declaration method are `provider-nid`, `provider-id`, `version`, and `target`. The parameters are all optional and are used to filter the entries in the configuration registry based on their respective fields. If any of these parameters is not present, the entries will not be filtered based on their value for that field. The `version` parameter can accept any valid Semver ranges.

When used for cross-MTA dependency resolution the `provider-nid` is always "`mta`", the `provider-id` follows the format `<mta-id>:<mta-provides-dependency-name>` and the `version` parameter is the version of the provider MTA. In addition, as illustrated in the following example, the `target` parameter is structured and contains the name of the organization and space in which the provider MTA is deployed. In the following example, the placeholders `${org}` and `${space}` are used, which are resolved to the name of the organization and space of the consumer MTA. In this way, the provider MTA is deployed in the same space as the consumer MTA.

> ⓘ Note
>
> As of version 3.0 of the MTA specification, the provided dependencies are no longer public by default. They have to be explicitly declared public for cross-MTA dependencies to work.

The following example shows the dependency declaration in the deployment descriptor of the “consumer” MTA :

{·} Sample Code

Consumer MTA Deployment Descriptor (`mtad.yaml`)

```
_schema-version: "3.1"
ID: com.sap.sample.mta.consumer
version: 0.1.0
modules:
  - name: consumer
    type: java.tomee
    requires:
      - name: message-provider
        properties:
          message: ~{message}
resources:
  - name: message-provider
    type: configuration
    parameters:
      provider-nid: mta
      provider-id: com.sap.sample.mta.provider:message-provider
      version: ">=1.0.0"
      target:
        org: ${org}     # Specifies the org of the provider MTA
        space: ${space}
```

→ Tip

If no target organization or space is specified by the consumer, then the current organization and space are used to deploy the provider MTA.

The following example shows the dependency declaration in the deployment descriptor of the “provider” MTA :

{·} Sample Code

Provider MTA Deployment Descriptor (`mtad.yaml`)

```
_schema-version: "3.1"
ID: com.sap.sample.mta.provider
version: 2.3.0
modules:
  - name: provider
    type: javascript.nodejs
    provides:
      - name: message-provider
        public: true
        properties:
          message: "Hello! This is a message provided by application \"${app-
name}\", deployed in org \"${org}\" and space \"${space}\"!"
```

Cross-MTA Configuration Visibility

A “consumer” module must explicitly declare the organizations and spaces in which a “provider” is expected to be deployed, except if it is the same space as the consumer. The “provider” can define a white list that specifies

the organizations and spaces from which the consumption of configuration data is permitted. It is not required to white list all the provider's spaces in its own organization.

> ### Note
>
> Previously, registry entries were visible from all organizations by default. Now, the default visibility setting is “visible within the current organization and all the organization's spaces”.

White lists can be defined on various levels. For example, a visibility white list could be used to ensure that a provider's configuration data is visible in the local space only, in all organizations and spaces, in a defined list of organizations, or in a specific list of organization and space combinations.

The options for white lists and the visibility of configuration data are similar to the options available for managed services. However, for visibility white lists, space developers are authorized to extend the visibility of configuration data beyond the space in which they work, without the involvement of an administrator. An administrator is required to release service plans to certain organizations.

Visibility is declared on the provider side by setting the parameter `visibility:` (of type 'sequence'), containing entries for the specified organization (`org:`) and space (`space:`). If no `visibility:` parameter is set, the default visibility value `org: ${org}, space: '*'` is used, which restricts visibility to consumers deployed into all spaces of the provider's organization. Alternatively, the value `org: '*'` can be set, which allows to bindings from all organizations and spaces. The white list can contain entries at the organization level only. This, however, releases configuration data for consumption from all spaces within the specified organizations, as illustrated in the following (annotated) example.

> ### → Tip
>
> Since applications deployed in the same space are always considered “friends”, visibility of configuration data in the local space is always preserved, no matter which visibility conditions are set.

> ### {·} Sample Code

```
provides:
  - name: backend
    public: true
    parameters:
      visibility:          # a list of possible settings:
       - org: ${org}       # for local org
         space: ${space}   # and local space
       - org: org1         # for all spaces in org1
       - org: org2         # for the specified combination (org2,space2)
         space: space2
       - org: ${org}       # default: all spaces in local org
       - org: '*'          # all orgs and spaces
       - org: '*'
         space: space3     # every space3 in every org
```

The authorization model ensures the following rules apply:

- Only those users in the white-listed spaces can read or consume the provided configuration data.
- Only users with the role “SpaceDeveloper” in the configuration-data provider's space can modify (edit or delete) configuration data.

4.1.8.1.9.7.1 Plug-ins

The SAP Cloud Deployment service supports a method that allows a multitarget applicaton to consume multiple configuration entries for each `requires` dependency.

The following is an example for multiple `requires` dependencies in the MTA Deployment Descriptor (`mtad.yaml`):

Sample Code

```
_schema-version: "2.1"
ID: com.acme.framework
version: "1.0" modules:
  - name: framework
    type: javascript.nodejs
    requires:
      - name: plugins
        list: plugin_configs
        properties:
          plugin_name: ~{name}
          plugin_url: ~{url}/sources
        parameters:
          managed: true # true | false. Default is false
resources:
  - name: plugins
    type: configuration
    parameters:
      target:
        org: ${org}
        space: ${space}
      filter:
        type: com.acme.plugin
```

The MTA deployment descriptor shown in the example above contains a module that specifies a `requires` dependency to a configuration resource. Since the `requires` dependency has a `list` property, the deploy service attempts to find multiple configuration entries that match the criteria specified in the configuration resource.

→ Tip

It is possible to create a subscription for a **single** configuration entry, for example, where no "`list:`" element is defined in the required dependency.

ⓘ Note

The filter parameter can be used in combination with other configuration resource specific parameters, for example: `provider-nid`, `provider-id`, `target`, and `version`.

The resource itself contains a `filter` parameter that is used to filter entries from the configuration registry based on their content. In the example shown above, the filter only matches entries that are provided by an MTA deployed in the current space, which have a `type` property in their content with a value of `com.acme.plugin`.

If the `list` element is missing, the values matched by the resources filter are **single** configuration entries – not the usual list of multiple configuration entries. In addition, if either no value or multiple values are found during the deployment of the consuming (subscribing) MTA, the deployment operation fails. If a provider (plug-in) adds more configuration details after the deployment of subscriber applications, these applications won't get

the new information right away. They will only receive these details when they get updated. However, this update will not be successful because there will be several configuration entries available at that point.

The XML document in the following example shows some sample configuration entries, which would be matched by the filter if they were present in the registry.

Sample Code

MTA Configuration Entries Matched in the Registry

```
<configuration-entry>
  <id>8</id>
  <provider-nid>mta</provider-nid>
  <provider-id>com.sap.sample.mta.plugin-1:plugin-1</provider-id>
  <provider-version>0.1.0</provider-version>
  <target-space>2172121c-1d32-441b-b7e2-53ae30947ad5</target-space>
  <content>{"name":"plugin-1","type":"com.acme.plugin","url":"https://
xxx.mo.sap.corp:51008"}</content>
</configuration-entry>
<configuration-entry>
  <id>10</id>
  <provider-nid>mta</provider-nid>
  <provider-id>com.sap.sample.mta.plugin-2:plugin-2</provider-id>
  <provider-version>0.1.0</provider-version>
  <target-space>2172121c-1d32-441b-b7e2-53ae30947ad5</target-space>
  <content>{"name":"plugin-2","type":"com.acme.plugin"}</content>
</configuration-entry>
```

The JSON document in the following example shows the environment variable that is created from the `requires` dependency defined in the example deployment descriptor above, assuming that the two configuration entries shown in the XML document were matched by the filter specified in the configuration resource.

Note

References to non-existing configuration entry content properties are resolved to "`null`". In the example above, the configuration entry published for `plugin-2` does not contain a `url` property in its content. As a result, the environment variable created from that entry is set to "`null`" for `plugin_url`.

Sample Code

Application Environment Variable

```
plugin_configs: [
  {
    "plugin_name": "plugin-1",
    "plugin_url": "https://xxx.mo.sap.corp:51008/sources"
  },
  {
    "plugin_name": "plugin-2",
    "plugin_url": null
  }
]
```

`Requires` dependencies support a special parameter named "`managed`", which registers as a "subscriber" the application created from the module containing the `requires` dependency. This registration causes the application's environment to be updated if any new configuration entries are published in the configuration registry during the deployment of another MTA. Those new entries must also match the subscription filter of the application. This will cause the application to restart so it can recognize the updated environment state.

→ Tip

When starting the deployment of an MTA (with the `cf deploy` command), you can use the special option `--no-restart-subscribed-apps` to specify that, if the publishing of configuration entries created for that MTA result in the update of a subscribing application's environment, then that application should **not** be restarted.

4.1.8.1.9.8 Sensitive Data Handling During MTA Deployment

Securely manage credentials and other sensitive values during MTA deployment.

Feature Overview

When deploying applications, developers often need to specify some sensitive data during deployment, such as passwords, certificates, or credentials to external services, that will later be used during application runtime. The SAP Cloud Deployment service provides a secure mechanism for handling such sensitive data during multitarget application deployment. This eliminates the need to put credentials and secret values in MTA descriptors, which are normally stored in source control repositories or file systems.

The feature is enabled by adding the `--require-secure-parameters` flag to the `cf deploy` command. After that, sensitive values (declared as specially-formatted environment variables before the deployment) are:

1. Collected by the MultiApps CF CLI plugin and validated
2. Transmitted securely to the SAP Cloud Deployment service over HTTPS using a special MTA extension description that is built in memory
3. Encrypted for the duration of the deployment using a key stored in a Cloud Foundry user-provided service instance. The service instance is either persisted or deleted afterwards, depending on the chosen approach (see Encryption Key Handling [page 812]).
4. Decrypted only when needed during deployment
5. Automatically excluded from all deployment logs and outputs

ⓘ Note

- The feature can be used only with the MultiApps CF CLI plugin and by the services and tools that utilize it, such as SAP Continuous Integration and Delivery. For more information, see Multitarget Application Plugin for Cloud Foundry Command Line Interface [page 2832].
- Avoid using the command-line option `--keep-files` while using this feature, in order for file removal to take place after the process is finished.

Encryption Key Handling

The key used for encrypting your sensitive data is stored in a Cloud Foundry user-provided service instance located in the current Cloud Foundry space of the customer. You can choose between using a persistent or disposable user-provided service instance approach:

Persistent (default): Uses a user-provided service instance that you create and manage manually. This service instance contains the encryption key and can persist across multiple deployments. You are responsible for creating, updating, and deleting the instance, as well as managing key rotation between deployments. For more information, see Using a Persistent User-Provided Service Instance [page 812].

> ### Note
>
> - For each new deployment, we recommend creating a new user-provided service instance or updating the already existing one.
> - If key rotation is mandatory, then it must be done between MTA deployments and not during an active MTA deployment. Before changing or deleting the encryption key, ensure that all processes using this key have finished completely.
>
> Using a disposable user-provided service instance is recommended in this case, as it allows automatic key rotation (a new key is generated for each deployment).

Disposable: Uses a user-provided service instance with a randomly generated encryption key that is automatically created at the start of each deployment and deleted when the operation completes. This mode is activated with the `--disposable-user-provided-service` flag and provides automatic key rotation per deployment. For more information, see Using a Disposable User-Provided Service Instance [page 815].

Related Information

Using a Persistent User-Provided Service Instance [page 812]
Using a Disposable User-Provided Service Instance [page 815]
Environment Variables and User-Provided Service Instance Specifics [page 818]

4.1.8.1.9.8.1 Using a Persistent User-Provided Service Instance

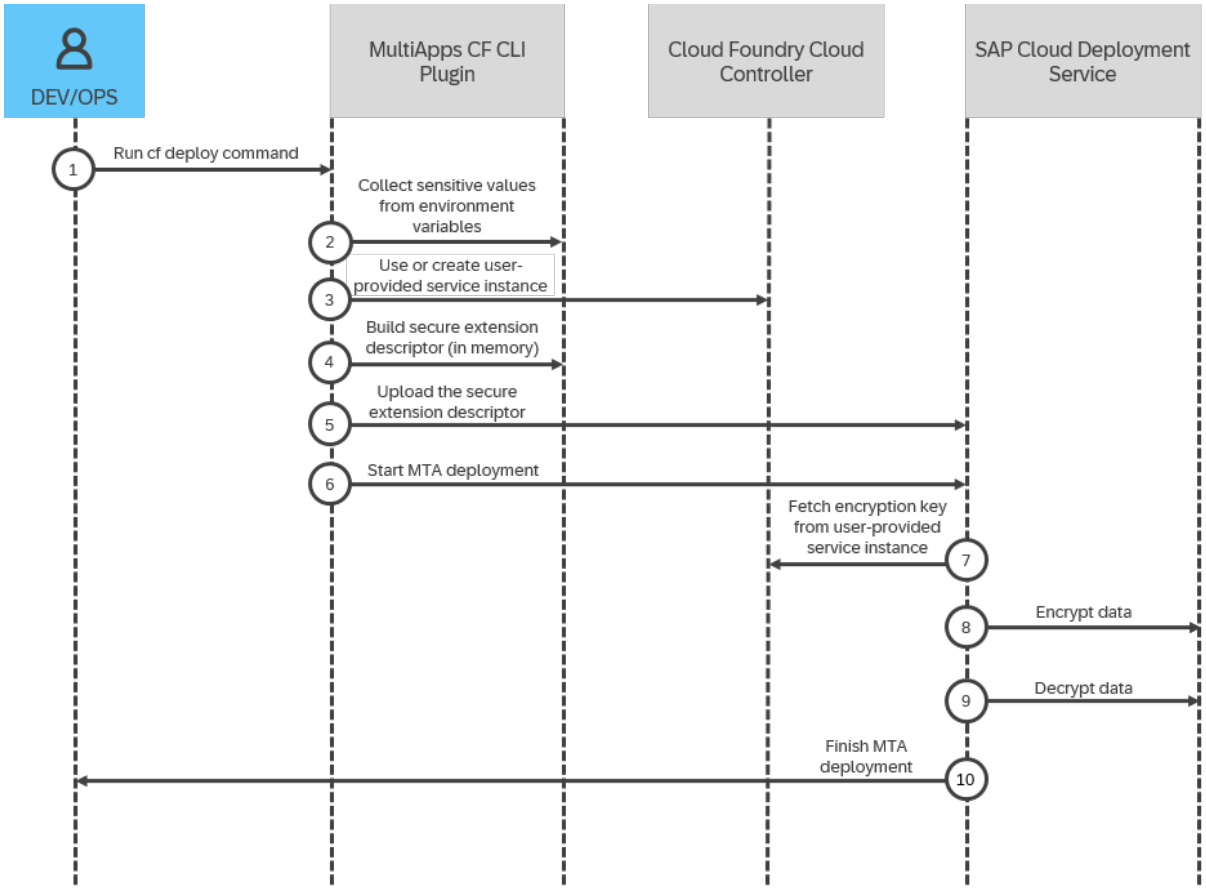
Pass sensitive values during MTA deployment by creating or using a persistent user-provided service instance.

Context

When you need to use sensitive values during MTA deployment, you can use a persistent user-provided service instance to manage the encryption key. In this approach, you are responsible for creating, maintaining, and

deleting the Cloud Foundry user-provided service instance that holds the encryption key used to secure your data during deployment. If the instance doesn't exist when you start the deployment, it is created automatically and holds a randomly generated encryption key. However, you remain responsible for managing the service instance throughout its lifecycle, including updating the encryption key if needed and deleting the instance when it's no longer required. The whole process is visualized in the diagram below and the actual step-by-step procedure that you have to follow is available in the next section.

This image is interactive. Hover over the circles for more information.



- A developer or operator initiates an MTA deployment. [page 812]
- The MultiApps CF CLI plugin collects the sensitive values declared as special environment variables, validating their correctness. [page 812]
- The MultiApps plugin checks for an already created user-provided service instance that holds the encryption key. If it doesn't exist, the instance is automatically created on behalf of the user in their CF space. An encryption key is generated and added to the instance. The user remains responsible for the user-provided service instance. [page 812]
- The MultiApps plugin builds an extension descriptor with a special ID, called the secure extension descriptor, which holds the sensitive values as its parameters. [page 812]
- The MultiApps plugin sends the secure extension descriptor to the SAP Cloud Deployment service over the HTTPS protocol. [page 812]
- The SAP Cloud Deployment service starts the MTA deployment. [page 812]
- The SAP Cloud Deployment service fetches the encryption key using the user-provided service instance. [page 812]

- The SAP Cloud Deployment service encrypts the sensitive values. [page 812]
- The SAP Cloud Deployment service decrypts sensitive values only when needed, keeping them raw in memory for a very short time during runtime. [page 812]
- The MTA deployment finishes and the SAP Cloud Deployment service reports the status to the developer or operator. [page 812]

Procedure

1. Declare environment variables locally that will store your sensitive values. These environment values must follow the naming conventions described in Environment Variables and User-Provided Service Instance Specifics [page 818].

   Sample Code

   ```
   __MTA___configSecret="confidentialInformation"
   ```

2. Reference the environment variable in your deployment or extension descriptor.

   Sample Code

   ```
   _schema-version: "3.1"
   ID: example-services.extension
   extends: example-services

   modules:
   - name: myApp
     type: staticfile
     path: content/archive.zip
     parameters:
       app-name: example-app
       memory: 299M
       disk-quota: 107M
     requires:
       - name: my-resource
         parameters:
           config:
             importantParameter: ${configSecret}

   resources:
   - name: my-resource
     type: org.cloudfoundry.managed-service
     parameters:
       service: workflow
       service-plan: standard
       service-name: my-resource-name
   ```

3. Create a user-provided service instance that holds a 32-character-long encryption key in your Cloud Foundry space. The name of the user-provided service instance must follow the naming conventions described in Environment Variables and User-Provided Service Instance Specifics [page 818].

   Sample Code

   ```
   cf cups __mta-secure-<mtaId> -p '{"encryptionKey":
   "abdfgtresghytiothewqprtimgnhdrwp"}'
   ```

For more information, see Creating User-Provided Service Instances [page 275].

> **Note**
>
> If the user-provided service instance is not present before the deployment, it will be created automatically and you will be responsible for its management and deletion at the end of the deployment operation.

4. Start a deployment by adding the `--require-secure-parameters` flag to the `cf deploy` command.

> **Sample Code**
>
> ```
> cf deploy ./ -f -e extension.mtaext,extension-after.mtaext --require-secure-parameters
> ```

5. **(Recommended)** After the deployment finishes, you can delete the user-provided service instance.

Results

The values of the parameters referencing the `configSecret` environment variable are replaced with the actual sensitive value during the MTA deployment.

Related Information

Sensitive Data Handling During MTA Deployment [page 811]
Environment Variables and User-Provided Service Instance Specifics [page 818]
Creating User-Provided Service Instances [page 275]

4.1.8.1.9.8.2 Using a Disposable User-Provided Service Instance

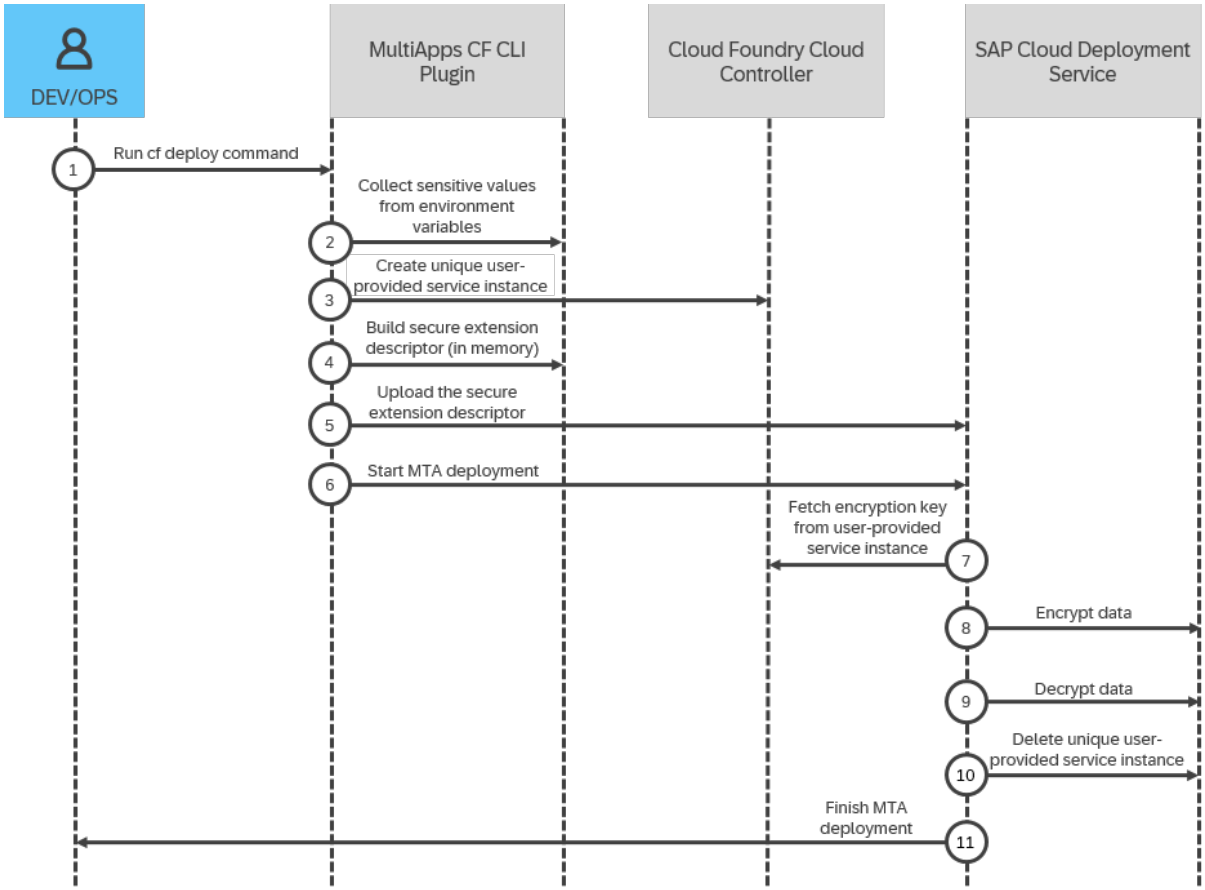
Pass sensitive values during MTA deployment by using a disposable user-provided service instance.

Context

When you need to use sensitive values during MTA deployment, you can use a disposable user-provided service instance to manage the encryption key. In this approach, the lifecycle of the Cloud Foundry user-provided service instance is handled automatically. When you start the deployment with the `--disposable-user-provided-service` flag, a unique user-provided service instance is automatically created with a randomly generated encryption key. This instance is used only for that specific deployment and is automatically deleted

when it finished. The whole process is visualized in the diagram below and the actual step-by-step procedure that you have to follow is available in the next section.

This image is interactive. Hover over the circles for more information.



- A developer or operator initiates an MTA deployment. [page 815]
- The MultiApps CF CLI plugin collects the sensitive values declared as special environment variables, validating their correctness. [page 815]
- The MultiApps plugin automatically creates a user-provided service instance on behalf of the user in their CF space. An encryption key is generated and added to the service instance. [page 815]
- The MultiApps plugin builds an extension descriptor with a special ID, called the secure extension descriptor, which holds the sensitive values as its parameters. [page 815]
- The MultiApps plugin sends the secure extension descriptor to the SAP Cloud Deployment service over the HTTPS protocol. [page 815]
- The SAP Cloud Deployment service starts the MTA deployment. [page 815]
- The SAP Cloud Deployment service fetches the encryption key using the user-provided service instance. [page 815]
- The SAP Cloud Deployment service encrypts the sensitive values. [page 815]
- The SAP Cloud Deployment service decrypts sensitive values only when needed, keeping them raw in memory for a very short time during runtime. [page 815]
- The SAP Cloud Deployment service automatically deletes the user-provided service instance. [page 815]
- The MTA deployment finishes and the SAP Cloud Deployment service reports the status to the developer or operator. [page 815]

Procedure

1. Declare environment variables locally that will store your sensitive values. These environment values must follow the naming conventions described in Environment Variables and User-Provided Service Instance Specifics [page 818].

   Sample Code

   ```
   __MTA___configSecret="confidentialInformation"
   ```

2. Reference the environment variable in your deployment or extension descriptor.

   Sample Code

   ```
   _schema-version: "3.1"
   ID: example-services.extension
   extends: example-services

   modules:
   - name: myApp
     type: staticfile
     path: content/archive.zip
     parameters:
       app-name: example-app
       memory: 299M
       disk-quota: 107M
     requires:
       - name: my-resource
         parameters:
           config:
             importantParameter: ${configSecret}

   resources:
   - name: my-resource
     type: org.cloudfoundry.managed-service
     parameters:
       service: workflow
       service-plan: standard
       service-name: my-resource-name
   ```

3. Start a deployment by adding the `--require-secure-parameters` and `--disposable-user-provided-service` flags to the `cf deploy` command.

   Sample Code

   ```
   cf deploy ./ -f -e extension.mtaext,extension-after.mtaext --require-
   secure-parameters --disposable-user-provided-service
   ```

   A unique user-provided service instance is created on your behalf in your Cloud Foundry space. A 32-character-long encryption key is generated and added to the service. At the end of the deployment, the user-provided service instance is automatically deleted.

Results

The values of the parameters referencing the `configSecret` environment variable are replaced with the actual sensitive value during the MTA deployment.

Related Information

Sensitive Data Handling During MTA Deployment [page 811]
Environment Variables and User-Provided Service Instance Specifics [page 818]
Creating User-Provided Service Instances [page 275]

4.1.8.1.9.8.3 Environment Variables and User-Provided Service Instance Specifics

Naming conventions and other specifics regarding the environment variables and user-provided service instances used for sensitive data handling during MTA deployment.

Environment Variables

Sensitive values that need to be passed during deployment must be declared as local environment variables before the deployment is initialized. Their names must contain a specific prefix depending on the data type. The supported types are strings, numbers, JSON objects, and Base64-encoded certificates.

Naming	Used For
`__MTA___<NAME>`	Strings **Example** `__MTA___secret="plainStringSecret"`

Naming	Used For

`__MTA_JSON___<NAME>`	• JSON objects
	Example
	`__MTA_JSON___secretObj='{"secretKey":"secretValue"}'`
	• Numbers
	Example
	`__MTA_JSON___secretNumber=675`

`_MTA_CERT___<NAME>`	Certificates

Note

Limitations regarding the configuration of these specialized environment variables depend on the specific shell or client environment being used. Users should check the documentation for their respective command-line interpreters to determine the correct syntax, capabilities, and limitations for setting environment variables.

User-Provided Service Instance

Persistent User-Provided Service Instance

When using the persistent user-provided service instance approach, you must adhere to the following format for the name of the user-provided service instance:

- `__mta-secure-<mtaId>`
- `__mta-secure-<mtaId>-<namespace>` (if you are using Namespaces [page 798])

You must also insert an encryption key in the user-provided service instance that will be used to encrypt and decrypt your data. The key must be 32 characters long and can contain only alphanumeric characters, hyphens, and underscores.

Example

```
cf cups __mta-secure-<mtaId> -p '{"encryptionKey":
"abdfgtresghytiothewqprtimgnhdrwp"}'
```

For more information, see Using a Persistent User-Provided Service Instance [page 812].

Disposable User-Provided Service Instance

When using the disposable user-provided service instance approach, a user-provided service instance is automatically created on your behalf. The following format is used for its name:

- `__mta-secure-<mtaId>-<random suffix>`

- `__mta-secure-<mtaId>-<namespace>-<random suffix>` (if you are using Namespaces [page 798])

For more information, see Using a Disposable User-Provided Service Instance [page 815].

Related Information

Sensitive Data Handling During MTA Deployment [page 811]
Using a Persistent User-Provided Service Instance [page 812]
Using a Disposable User-Provided Service Instance [page 815]

4.1.8.1.9.9 Features Related to SAP Alert Notification Service for SAP BTP

Deploy and update SAP Alert Notification service for SAP BTP using the multitarget application concept, or get notified by the service about the status of multitarget application operations.

Deploying and Updating the Service

As an alternative to the manual procedure, you can use a deployment descriptor to automate the initial deployment of SAP Availability service. You can use the same procedure to update the service parameters. See the procedure described in Configuration Management Using Multitarget Application Descriptors.

Receive Alerts During Deployment and Undeployment of MTAs

You can set SAP Alert Notification service for SAP BTP to dispatch alerts to a communication channel of your choice, so that you are informed about the status of MTA deployment and undeployment operations. For example, if setup to do so, upon a successful deployment it can send you an e-mail or a Slack message. You can also get notified for problems in MTA operations, as you can also receive alerts if the deployment has encountered issues.

The integration between the services is out-of-the-box, that is, no special steps are required for its activation. If you have a service instance in the space where the MTA deployment takes place, it would automatically receive alerts from the MTA deployments in that space.

See how you can match events using multitarget operations as described in Multitarget Application Events.

To see additional details, see Receive instant notifications...with the SAP Cloud Deployment service.

4.1.8.1.9.10 MTA Operations History and Logs

When the SAP Cloud Deployment service processes a multitarget application (MTA), it keeps a history of the operations and produces operation logs that record the progress and outcome of each operation.

Every time an MTA is deployed, undeployed, or otherwise processed, the SAP Cloud Deployment service creates an operation and writes operation logs for it. You can view the history of the recent MTA operations, including their unique operation IDs, status, and other useful information. If you need to inspect a deployment or perform troubleshooting, you can retrieve this history and the associated operation logs to understand the progress of an operation and investigate failures.

You can work with the history and logs of an MTA operation in the following ways:

- List the recent operations and download their logs by using multitarget application commands, as described below.
- Export the operation logs to an SAP Cloud Logging service instance, so that you can store, visualize, and analyze them together with the rest of your observability data. For more information, see (Experimental) Exporting MTA Operation Logs to SAP Cloud Logging [page 823].

> ⓘ Note
>
> The output of the commands shown on this page is provided as an example and might differ over time.

Retention

The operation logs are available for the retention period of the operation. The expiration time for all MTA operations in the Cloud Foundry environment is 3 days. When this time limit is reached, the operation and its logs are no longer available, and an operation that is still active is automatically aborted.

If you want to keep the operation logs beyond the retention period, download them before the operation expires, or export them to an SAP Cloud Logging service instance, where they are subject to the retention of that service.

Viewing the Operations History

Use the `mta-ops` command of the MultiApps CF CLI plugin to list the recent operations of a multitarget application. The command shows the operations together with their unique operation IDs, status, and other useful information, which you can use to inspect a deployment or perform troubleshooting.

> {·} Sample Code

```
cf mta-ops --mta my-mta
```

You can use the operation ID from the history to download the logs of a specific operation, as described below.

Downloading the Operation Logs

Use the `download-mta-op-logs` command (alias `dmol`) of the MultiApps CF CLI plugin to download the log files of one or more MTA operations. If you do not know the ID of the operation whose logs you want, first list the operations with the `mta-ops` command, as described above.

Download the operation logs by operation ID:

Sample Code

```
cf download-mta-op-logs -i <OPERATION_ID>
```

By default, the logs are saved to the `./mta-op-<OPERATION_ID>/` directory. To save them to a different location, use the `-d` option.

Sample Code

```
cf download-mta-op-logs -i <OPERATION_ID> -d ./my-logs
```

Alternatively, download the logs of the last operations of a multitarget application without specifying an operation ID.

Sample Code

```
cf download-mta-op-logs --mta my-mta --last 1
```

For the full list of options, see download-mta-op-logs [page 2847].

Related Information

Multitarget Application Commands for the Cloud Foundry Environment [page 2834]
(Experimental) Exporting MTA Operation Logs to SAP Cloud Logging [page 823]

4.1.8.1.9.10.1 (Experimental) Exporting MTA Operation Logs to SAP Cloud Logging

Export the operation logs of your multitarget application to SAP Cloud Logging, so that you can store, visualize, and analyze them together with the rest of your observability data.

Prerequisites

> ### Note
>
> This is an experimental feature. Experimental features aren't part of the officially delivered scope that SAP guarantees for future releases. For more information, see Important Disclaimers and Legal Information.
>
> Please use the *Feedback* button in this topic to let us know what you like and don't, and how we can improve it to make the experience more enjoyable for you.

- You have an SAP Cloud Logging service instance. For more information, see SAP Cloud Logging.
- The service instance is reachable from the same Cloud Foundry API endpoint that you use to deploy the multitarget application. A multitarget application can only export its operation logs to an SAP Cloud Logging service instance that is available under the same Cloud Foundry API endpoint as the deployment. For the available Cloud Foundry API endpoints, see Regions and API Endpoints Available for the Cloud Foundry Environment.
- You have created a service key for the SAP Cloud Logging service instance that provides mutual TLS ingestion credentials. See Create a Service Key.
- The user who starts the multitarget application deployment has access to the SAP Cloud Logging service instance. If the instance resides in a different space, ensure that this user has the **Space Developer** role in that space as well. For more information, see About Roles in the Cloud Foundry Environment.

Context

When your multitarget application (MTA) is deployed, undeployed, or otherwise processed, the SAP Cloud Deployment service produces operation logs that describe the progress and outcome of the operation. By default, these logs are available for the retention period of the operation and can be retrieved with the multitarget application commands, for example by listing operations with `mta-ops` and downloading their logs with `download-mta-op-logs`. For more information, see MTA Operations History and Logs [page 821].

By adding a resource of type `org.cloudfoundry.cloud-logging-service` to your deployment descriptor, you instruct the SAP Cloud Deployment service to also forward these operation logs to an SAP Cloud Logging service instance. The SAP Cloud Deployment service reads the credentials of a service key that belongs to the instance and uses them to establish a mutual TLS connection to the ingestion endpoint. You control which service key is used, the verbosity of the exported logs, and, optionally, the organization and space in which the instance resides.

Procedure

1. In your deployment descriptor (`mtad.yaml`), add a resource of type `org.cloudfoundry.cloud-logging-service`.

   > → Tip
   >
   > You can mark the resource as optional by setting `optional: true`. In this case, a missing or invalid SAP Cloud Logging configuration does not cause the deployment to fail. The same applies to runtime problems while exporting the logs, such as connectivity issues or rate limiting on the side of the SAP Cloud Logging service instance. As a result, some operation logs might not appear in your SAP Cloud Logging service instance, but the operation logs remain available through the multitarget application commands, such as `mta-ops` and `dmol`.

2. Set the `service-name` parameter to the name of your SAP Cloud Logging service instance.
3. Set the `service-key-name` parameter to the name of the service key that holds the ingestion credentials.
4. **Optional:** Set the `log-level` parameter to control the verbosity of the exported operation logs.

   The supported values are `INFO`, `WARN`, `DEBUG`, `ERROR`, and `TRACE`. If you do not specify a value, `INFO` is used.

   > ⓘ Note
   >
   > The log levels are hierarchical. For example, `TRACE` exports logs of all levels, whereas `ERROR` exports only error-level logs. A more verbose log level increases the volume of data that is ingested into your SAP Cloud Logging service instance, which affects its consumption and eventual costs.

5. **Optional:** If the SAP Cloud Logging service instance resides in a different organization or space than the one you deploy to, set the `destination` parameter with the `org-name` and `space-name` of the target organization and space.

   If you omit `destination`, the organization and space of the current deployment are used.

   > {·} Sample Code

   ```
   resources:
     - name: my-cloud-logging
       type: org.cloudfoundry.cloud-logging-service
       parameters:
         service-name: my-cloud-logging-instance
         service-key-name: my-service-key
         log-level: INFO
         destination:
           org-name: my-org
           space-name: my-space
   ```

6. Deploy your multitarget application.

   For more information, see Multitarget Application Commands for the Cloud Foundry Environment [page 2834].

7. Verify that the operation logs of the deployment appear in your SAP Cloud Logging service instance.

Next Steps

Each operation has a unique operation ID, which is exported to your SAP Cloud Logging service instance as the correlation ID of the operation logs. You can use this operation ID to filter the logs in SAP Cloud Logging and view only the entries that belong to a specific deployment operation.

Related Information

Multitarget Application Commands for the Cloud Foundry Environment [page 2834]
List of Supported MTA Parameters [page 692]
Resources [page 630]

4.1.8.1.10 Integration

Find information on how to integrate with SAP Cloud Deployment service and learn how other services leverage its capabilities.

To learn more about	See
Implementing the SAP Cloud Deployment service API to provide capabilities for deployment and management of multitarget applications in your service	Integrating with SAP Cloud Deployment Service [page 825]
The URLs and Cloud Foundry regions on which the SAP Cloud Deployment service is available	Service Availability and URLs [page 826]
How other tools, projects, and services utilize the SAP Cloud Deployment service	Services Integrated with SAP Cloud Deployment Service [page 829]
Transporting applications or application content packaged as MTAs between subaccounts	Integration with Transport Management Tools [page 830]

4.1.8.1.10.1 Integrating with SAP Cloud Deployment Service

Implement the SAP Cloud Deployment service API to provide capabilities for deployment and management of multitarget applications in your service.

SAP Cloud Deployment Service REST API

The SAP Cloud Deployment service provides a REST API for managing multitarget applications in the Cloud Foundry environment. The API enables programmatic control over the entire MTA lifecycle, from uploading MTA archives to monitoring deployment operations and managing existing applications.

The API provides endpoints for:

- uploading files, such as MTA archives and extension descriptors,
- performing operations on MTAs, including deploying and undeploying applications, and performing blue-green deployments,
- performing actions on the operations, including aborting ongoing operations, retrying failed operations, and resuming operations that require manual intervention,
- retrieving information, such as the status of specific operations, logs generated during execution, and accessing operation metadata
- retrieving information about deployed MTAs within a Cloud Foundry space.

The complete Swagger definitions can be found at MTA REST API .

Authentication and Authorization

To interact with the API, you must authenticate using an OAuth 2.0 token obtained from the SAP Authorization and Trust Management service.

> **Note**
>
> Users must have at least a **Space Developer** role in the target space to perform deployment operations. For more information, see About Roles in the Cloud Foundry Environment [page 2788].

Determining the SAP Cloud Deployment Service Base URL

To make API calls, you need to determine the base URL of the SAP Cloud Deployment service based on the Cloud Foundry region. For more information, see Service Availability and URLs [page 826].

4.1.8.1.10.1.1 Service Availability and URLs

Find information about the URLs and Cloud Foundry regions on which the SAP Cloud Deployment service is available.

The SAP Cloud Deployment service is currently available on the following URLs:

- `https://deploy-service.cf.<domain>`
- `https://deploy-service.cfapps.<domain>` - legacy route

> **Note**
>
> The legacy route `https://deploy-service.cfapps.<domain>` is deprecated and will be removed by the end of 2026. For more information, see SAP Note 3695458.

The `<domain>` is derived from the Cloud Foundry API endpoint, which you can find in the *Overview* section of your subaccount in the SAP BTP cockpit.

The SAP Cloud Deployment service is available in all Cloud Foundry regions (see Regions and API Endpoints Available for the Cloud Foundry Environment [page 19]). For some Cloud Foundry regions, there are multiple CF API endpoints, and the SAP Cloud Deployment service is available on each of them:

> ### Example
>
> Example for region cf-eu10
>
> | CF API Endpoint | Domain | SAP Cloud Deployment service URL | SAP Cloud Deployment service Legacy URL |
> |---|---|---|---|
> | `api.cf.eu10.hana.ondemand.com` | `eu10.hana.ondemand.com` | `https://deploy-service.cf.eu10.hana.ondemand.com` | `https://deploy-service.cfapps.eu10.hana.ondemand.com` |
> | `api.cf.eu10-002.hana.ondemand.com` | `eu10-002.hana.ondemand.com` | `https://deploy-service.cf.eu10-002.hana.ondemand.com` | `https://deploy-service.cfapps.eu10-002.hana.ondemand.com` |
> | `api.cf.eu10-003.hana.ondemand.com` | `eu10-003.hana.ondemand.com` | `https://deploy-service.cf.eu10-003.hana.ondemand.com` | `https://deploy-service.cfapps.eu10-003.hana.ondemand.com` |
>
> The same pattern applies for the other API endpoints for region cf-eu10 and for the other regions with multiple API endpoints.

Related Information

Regions and API Endpoints Available for the Cloud Foundry Environment [page 19]

4.1.8.1.10.1.2 Rate Limiting

The SAP Cloud Deployment service limits how frequently operations can be started to protect service availability and ensure consistent performance across environments.

Rate limits apply when you **start** a new operation. Requests that exceed a limit are rejected before the operation begins. Operations already running are not affected.

> ### → Remember
>
> An operation is any action that modifies the state of a multitarget application (MTA) in your Cloud Foundry space, such as deploying, undeploying, or updating an MTA.

Limits are enforced at two levels simultaneously:

- **Per space**: All users in the same Cloud Foundry space share a single allowance. Heavy usage by one user can exhaust the shared allowance.

- **Per user**: Each user has an individual allowance within the space, independent of other users.

These limits are checked every time you start an operation. If your request exceeds any limit, the service returns `HTTP 429 Too Many Requests`.

Limits

Concurrent Operation Limit

The maximum number of operations that can be active at the same time:

Scope	Concurrent Operations Limit
Per space	500 operations
Per user	200 operations

Operation Start Rate

The maximum number of new operations that can be started within a rolling time window:

Scope	New Operations Limit
Per space	Approximately 300 operations
Per user	Approximately 150 operations

> ⓘ Note
>
> Each operation start consumes one unit from both the space and the user allowances. Allowances refill automatically over the rolling time window.

Related Information

Integrating with SAP Cloud Deployment Service [page 825]

4.1.8.1.10.2 Services Integrated with SAP Cloud Deployment Service

Find information on how other tools and services utilize the SAP Cloud Deployment service.

Tool / Service	How It Uses SAP Cloud Deployment Service	Additional Information
SAP Business Application Studio	SAP Business Application Studio uses the SAP Cloud Deployment service to provide capabilities for multi-target application deployment in the Cloud Foundry environment. When you deploy an MTA from SAP Business Application Studio using any of the available methods (context menu, command palette, Task Explorer, or CLI), SAP Business Application Studio initiates a deployment operation using the Cloud Foundry CLI MTA plugin and your application is deployed in the target Cloud Foundry space by SAP Cloud Deployment service.	• What Is SAP Business Application Studio? • Building and Deploying Multitarget Applications • Multitarget Application Plugin for Cloud Foundry Command Line Interface [page 2832]
SAP Continuous Integration and Delivery	SAP Continuous Integration and Delivery uses the SAP Cloud Deployment service to deploy multitarget applications across different stages of the CI/CD pipelines. When you configure a deployment step in your CI/CD job (either in the Acceptance or Release stage), SAP Continuous Integration and Delivery initiates a deployment operation and your application is deployed to the target Cloud Foundry space by the SAP Cloud Deployment service.	• What Is SAP Continuous Integration and Delivery? • Configure a Cloud Foundry Environment Job
Project "Piper"	Project "Piper" is an open-source project that provides pre-configured Jenkins pipelines for continuous delivery scenarios. It uses the SAP Cloud Deployment service to enable automated multitarget application deployment as part of CI/CD workflows. When you use the `cloudFoundryDeploy` step in your Jenkins pipeline, project "Piper" leverages the Cloud Foundry CLI MTA plugin to initiate deployment operations, and your application is deployed to the target Cloud Foundry space by the SAP Cloud Deployment service.	• Project "Piper" User Documentation • cloudFoundryDeploy

Tool / Service	How It Uses SAP Cloud Deployment Service	Additional Information
SAP Build Process Automation	SAP Build Process Automation uses the SAP Cloud Deployment service to facilitate the import of live process objects from the *Store*. If users want to import and work on live process objects, they need to configure a destination to SAP Cloud Deployment service.	• What Is SAP Build Process Automation? • Explore the Store • Configure Destination for Live Process Projects **Note** When configuring the destination, you need to provide the URL to SAP Cloud Deployment service. For information on how to determine the correct URL, see Service Availability and URLs [page 826].
Terraform Provider for Cloud Foundry	The Terraform Provider for Cloud Foundry uses the SAP Cloud Deployment service through its `cloudfoundry_mta` resource to enable infrastructure-as-code deployment of multitarget applications. When you define an MTA deployment in your Terraform configuration using the `cloudfoundry_mta` resource, Terraform initiates a deployment operation via the SAP Cloud Deployment service API, and your application is deployed to the target Cloud Foundry space. This integration allows MTA deployments to be managed declaratively alongside other infrastructure resources in Terraform workflows, supporting automated provisioning, updates, and lifecycle management of multitarget applications.	• Terraform Provider for Cloud Foundry • cloudfoundry_mta (Resource)
Transport management tools (cTMS, CTS+)	The SAP Cloud Transport Management service and the Enhanced Change and Transport System (CTS+) use the SAP Cloud Deployment service to transport applications or application content packaged as a multitarget application between subaccounts.	• Integration with Transport Management Tools [page 830]

4.1.8.1.10.2.1 Integration with Transport Management Tools

To transport an application or application content to other subaccounts, you can use SAP Cloud Transport Management service, or the Enhanced Change and Transport System (CTS+).

Using SAP Cloud Transport Management service

Note

This is the recommended approach for customers with a cloud-based landscape.

To learn more about	See
How to use SAP Cloud Transport Management service in general	What Is SAP Cloud Transport Management
What you need to do to enable the direct upload of MTA archives to a transport request that will be used by SAP Cloud Transport Management service	Create Destinations to SAP Cloud Transport Management Service
How to configure destinations to the target endpoints of the deployment that are required as part of the setup of your transport landscapes in SAP Cloud Transport Management service	Creating Destinations for MTA Deployment on Cloud Foundry

Using the Enhanced Change and Transport System (CTS+)

Use this option if you already use CTS+ for other applications, or if you have a hybrid landscape where you want the ABAP system to be leading the transport environment. If your setup is entirely cloud-based, see the section above - Using SAP Cloud Transport Management service [page 830].

To learn more about	See
How to configure and use CTS+ to transport Cloud Foundry applications from one subaccount to another	Transporting Multitarget Applications in Cloud Foundry using CTS+ [page 832]

→ Recommendation

As a future-proof way forward, to be able to cover also more and more application-specific content with one central approach, SAP recommends SAP Cloud Transport Management.

ⓘ Note

Limitations related to the integration with SAP Cloud Transport Management and the Enhanced Change and Transport System (CTS+) are listed in the section Prerequisites and Restrictions [page 618].

Related Information

Delivery Options

4.1.8.1.10.2.1.1 Transporting Multitarget Applications in Cloud Foundry using CTS+

You can enable transport of SAP BTP applications and application content that is available as Multitarget Applications (MTA) using the Enhanced Change and Transport System (CTS+).

Prerequisites

- You have configured your SAP BTP subaccounts for transport with CTS+ as described in the document How to...Configure SAP BTP, Cloud Foundry environment for CTS.
- The content that you want to transport can be made available as a Multitarget Application (MTA) archive as described in Multitarget Applications in the Cloud Foundry Environment [page 615].

Context

You use the Change and Transport System (CTS) of ABAP to transport and deploy your applications running on SAP BTP in the form of MTAs, for example, from development to a test or production subaccount. Proceed as follows to be able to transport an SAP BTP application:

Procedure

1. Package the application in a Multitarget Application (MTA) archive using the Archive Builder Tool as described in Defining Multitarget Application Archives [page 622].
2. Attach the MTA archive to a CTS+ transport request as described in *How to...Configure SAP BTP, Cloud Foundry environment for CTS*.
3. Trigger the import of an SAP BTP application as described in *How to...Configure SAP BTP, Cloud Foundry environment for CTS*.

Related Information

Resources on CTS+

Setting up a CTS+ enabled transport landscape in SAP BTP

4.1.8.1.11 MTA Deployment in Regions China (Shanghai) and China (North 3)

Find information on region-specific deployment configurations for regions China (Shanghai) and China (North 3).

Configuring Domains for Cloud Foundry Applications

Most of the Cloud Foundry applications deployed as part of an MTA have assigned routes at the end of the MTA deployment. You can specify the route by using various module-level parameters as described in Routes [page 765].

If you do not specify the route by using one of these parameters, a placeholder `default-uri` is automatically applied. This placeholder is resolved to `${default-host}.${default-domain}`, where `${default-domain}` is resolved to the default domain for the current Cloud Foundry organization. This value is coming from the Cloud Foundry API and it cannot be configured by the end user. Usually, the default domain is resolved to `cfapps.<region>.hana.ondemand.com`. However, due to missing shared domains in regions China (Shanghai) and China (North 3), the default domain value is not resolved properly, and so the MTA parameter `${default-domain}` is not resolved properly as well.

You may encounter this problem in one of the following cases:

- You rely on the default implicit logic during MTA deployment and the parameters `domain` or `routes` are not configured at all.
- You explicitly use `${default-domain}` somewhere in the routes configuration:

  Example

  Sample Code

  ```
  modules:
  - name: my-app
    type: application
    parameters:
      routes:
        - route: "my-route.${default-domain}"
  ```

To avoid problems when deploying MTAs in regions China (Shanghai) and China (North 3), please avoid using the `${default-domain}` parameter. Instead, configure the domains for your Cloud Foundry applications by following these steps:

1. Configure a custom domain for your applications. When a custom domain is used, both the domain name and the TLS/SSL certificate of the server are owned by the customer. You can configure custom domains using the Cloud Foundry command-line interface with the plugin for custom domains. For more information, see What Is Custom Domain and Custom Domain Plugin for the Cloud Foundry Environment [page 2822].
2. Configure your MTA descriptor to use the custom domain. You can do this in one of the following ways:

Using the `routes` parameter (recommended)

```
modules:
- name: my-app
  type: application
  parameters:
    routes:
      - route: "my-route.my.custom.domain"
```

When you update your MTA by using Blue-Green Deployment of Multitarget Applications [page 785], you also need to configure `idle-routes` respectively. Otherwise, in the first phase of blue-green deployment, newly created applications will use `${default-domain}`, which might lead to problems.

```
modules:
- name: my-app
  type: application
  parameters:
    routes:
      - route: "my-route.my.custom.domain"
    idle-routes:
      - idle-route: "my-route-idle.my.custom.domain"
```

Using the `domain` or `domains` parameter

```
modules:
- name: my-app
  type: application
  parameters:
    domain: my.custom.domain
```

When you update your MTA by using Blue-Green Deployment of Multitarget Applications [page 785], you also need to configure `idle-domain` or `idle-domains` respectively. Otherwise, in the first phase of blue-green deployment, newly created applications will use `${default-domain}`, which might lead to problems.

```
modules:
- name: my-app
  type: application
  parameters:
    domain: my.custom.domain
    idle-domain: my.custom.domain
```

> **Note**
>
> The `domain` and `domains` parameters are deprecated, so their usage is not recommended. Use the `routes` parameter instead.

Specifying the SAP Cloud Deployment Service URL

> **Note**
>
> This section is relevant only for multitarget application developers who are using a version of the MultiApps CF CLI plugin that is older than 3.0.0. If you are using a version of the MultiApps CF CLI plugin that is newer than 3.0.0, you do not need to do the additional settings described below. It is recommended to update the MultiApps CF CLI plugin regularly to the latest release - see Releases .

Due to the missing shared domains in regions China (Shanghai) and China (North 3), you need to specify the URL of the Cloud Deployment service in one of the following ways:

- by setting the MULTIAPPS_CONTROLLER_URL environment variable. For more information, see Configuration .
- by using the `-u <URL>` command option of the Multitarget Application Commands for the Cloud Foundry Environment [page 2834].

> ### Note
>
> If you are using this option, make sure you have the MultiApps CLI plugin version 2.1.3 or higher.

The deploy-service URL needs to be in the following format: `deploy-service.cf.<domain>`. You can derive the domain from the Cloud Foundry API endpoint which is visible in the *Overview* of your subaccount in the SAP BTP cockpit. See also Regions and API Endpoints Available for the Cloud Foundry Environment [page 19].

> ### Note
>
> For region China (Shanghai) with API endpoint `api.cf.cn40.platform.sapcloud.cn`, you need to specify the URL as follows: `export MULTIAPPS_CONTROLLER_URL=deploy-service.cf.cn40.platform.sapcloud.cn`.

Optimizing MTA Deployment Time

In most cases, MTA deployment is done from one single CI/CD towards different target regions, including European, American and China regions. In the beginning of one MTA deployment, an MTA archive is uploaded from the CI/CD or from a local machine towards the Cloud Deployment service located in the specific region. Sometimes, connection problems (connection drops or is lost completely) might occur between other regions and regions China (Shanghai) and China (North 3). For example, if the CI/CD is in a European region and the MTA deployment is towards China (Shanghai) or China (North 3), it is very likely to observe problems while uploading the MTA archive to Cloud Deployment service.

To avoid such connection problems, you can try following these approaches:

- If you are using the MultiApps CF CLI plugin, which provides the `cf deploy` command, you can decrease the chunk size of the MTA archive – in this way, the whole MTA archive will be split in parts, where each part has smaller size. It is expected that the smaller parts will be uploaded more easily. For more information, see Configuration .
- Instead of deploying a local MTA archive, pass the URL of the archive and Cloud Deployment service will download it. For more information, see **Deployment using a URL to the MTA archive** in Multitarget Application Commands for the Cloud Foundry Environment [page 2834].
  This approach is suitable for adoption for other regions as well. It brings optimization in MTA archive upload and overall MTA deployment time, especially for MTA archives with larger size.

Related Information

MultiApps CF CLI Plugin

Multitarget Application Commands for the Cloud Foundry Environment [page 2834]

4.1.8.1.12 Frequently Asked Questions

Find the answers to the most frequently asked questions about MTA deployment in the Cloud Foundry environment.

My MTA deployment failed. What could be done?

Check the MTA Operation Failure [page 839] section.

What to do when my application start-up/staging failed?

Check the MTA Operation Failure [page 839] section.

How can I find the logs of an application from a failed/succeeded deployment?

There are several ways to find the application logs:

- `OPERATION.LOG` – contains the whole deployment log.
- `<application-name>.log` – there is a separate file for each application. It holds the logs related to the application during staging and starting.

> ### Note
>
> Note that the operation logs are kept for 3 days. Make sure that you download them before the retention period expires.

To download the logs, execute the following command:
`cf dmol -i <process-id>`

> ### Sample Code
>
> ```
> cf dmol -i cbe58aeb-0c40-4a3e-972d-82a499815745
> ```

Whom to contact in case of problems with deployment?

When an error occurs during deployment, there is a message which indicates whether the problem is in one of the components of the Cloud Foundry Platform. In such cases, an incident to the corresponding component could be created.

> ### Example
>
> “Controller operation failed: 400 Bad Request: Cannot bind application to service”

For more information, you can also check the Troubleshooting [page 838].

How to check the state of my deployment?

Use the command `cf mta-ops` and find the id of the operation related to the desired deployment. There will be information about the status of the operation.

How can I list the MTAs that I have deployed in my space?

Use the `cf mtas` command and locate the ID of the desired MTA. After locating the correct MTA ID, execute the command `cf mta <located-mta-id>` to get detailed information about the MTA with the provided ID.

What can I do with my deployment?

- If the deployment fails, see MTA Operation Failure [page 839].
- If the deployment finishes successfully, you can check the deployment logs.
- If the deployment is still running, you can abort or monitor it.
  The **Abort** action is described in more details in the MTA Operation Failure [page 839] section.
  The **Monitor** action can be executed with the following command:
  `cf <operation> -i <operation-id> -a monitor. Example: cf deploy -i 12355 -a monitor`

How can I redeploy my MTA when a deployment is already running?

You can abort the currently running deployment, using the command `cf <operation> -i <operation-id> -a abort` (Example: `cf deploy -i 12353 -a abort`) or you can execute the command for starting the deployment by providing the option -f as described in the deploy [page 2835] section.

> ### Example
>
> ```
> cf deploy <path-to-mtar>.mtar -f
> ```

What are the size limits of an MTA?

4GB for the whole MTA and 1GB for a single module. For more information, see Prerequisites and Restrictions [page 618].

What to do in case of service creation failure?

See MTA Operation Failure [page 839].

How to make my deployment faster?

You can use a parallel deployment as described in section Parallel Deployment [page 804].

How can I resolve problems with the MTA descriptor modeling?

For more information, see Defining Multitarget Application Deployment Descriptors for Cloud Foundry [page 623].

What is the deployment order of services, applications and other content?

Services are always deployed in parallel.

The applications and content deployment order is determined based on the `deployed-after` module parameter. If `deployed-after` is not used and parallel deployments are not enabled for the MTA, the `requires/provides` module sections define the order. For more information, see Parallel Module Deployment [page 803].

How to reuse one backing service in two different MTAs?

If you want one MTA to "own" the lifecycle of the service and another to use it only as an "existing service", use the `org.cloudfoundry.managed-service` and `org.cloudfoundry.existing-service` resource types respectively.

How to define service instance and bindings parameters?

For more information, see Service Binding Parameters [page 759] as well as Service Instance Parameters [page 756].

4.1.8.1.13 Troubleshooting

This topic contains information that is essential for troubleshooting issues when deploying multitarget applications in the Cloud Foundry environment.

Before you start investigating the problem, download the logs for the current deployment. The logs contain the following structure of files:

- `OPERATION.LOG` - contains the whole deployment log
- `<application-name>.log` – there is a separate file for each application. It holds the logs related to the application during staging and starting

> ⓘ Note
>
> Note that the operation logs are kept for 3 days. Make sure that you download them before the retention period expires.

To download the logs, execute the following command:

```
cf dmol -i <process-id>
```

> ✤ Example
>
> ```
> cf dmol -i cbe58aeb-0c40-4a3e-972d-82a499815745
> ```

For more information, see MTA Operations History and Logs [page 821].

Troubleshooting an MTA Operation Failure

To troubleshoot issues, see the list of problems that may occur during the Multitarget Application deployment that is available in MTA Operation Failure [page 839].

Creating an Incident

If the list does not contain a solution for your issue, you can contact the service support team by reporting an incident through the SAP Support Portal. Here is a list of what to provide in an incident:

1. Select the component *BC-CP-CF-DS* from the *Component* dropdown list.
2. In the description field, provide as much information as possible about the Cloud Foundry region, Cloud Foundry organization and space names and/or Global Unique Identifiers.
3. Provide information about scenario with the exact steps to be reproduced, expected result, actual result or the error that appears, relevant logs and screenshots. In all cases, the starting point for the investigation are the multitarget application operation logs.

4.1.8.1.13.1 MTA Operation Failure

Find information on how to troubleshoot a failed MTA deployment.

There might be different reasons for an MTA deployment failure. Retrying the last failed steps of an operation is the first basic step you can take to recover from a failed deployment. This approach is useful for temporary issues with the underlying infrastructure but not helpful for configuration or functional problems.

To do this, enter the following command:

```
cf <deployment-action> -i <process-id> -a retry
```

> Example
>
> ```
> cf undeploy -i cbe58aeb-0c40-4a3e-972d-82a499815745 -a retry
> ```

If retrying the last failed steps does not solve the problem, you can either try redeploying your MTA or troubleshoot the problem on your own.

To redeploy your MTA, proceed as follows:

1. Terminate the deployment process. To do this, enter the following command:
   ```
   cf <deployment-action> -i <operation-id> -a abort
   ```

   > Example
   >
   > ```
   > cf deploy -i cbe58aeb-0c40-4a3e-972d-82a499815745 -a abort
   > ```

   > Note
   >
   > This action doesn't roll back all applied changes to the previous stable version. It allows the next deployments of that MTA to proceed without confirmation. The current process ends without any possibility to retry from the failed step.
2. Start a new deployment.

To troubleshoot the problem on your own, check the list of possible problems that may occur during the multitarget application deployment. If the problem cannot be categorized into any of the sections below, you need to create an incident [page 838].

Service Create Failure

If the creation of the service fails, an error message with the following format will be displayed:

```
...Error creating service "<service-name>" from offering "<service-offering>"
and plan "<service-plan>": <cause-of-failure>
```

Such an error usually occurs due to a problem with the service provisioning infrastructure or the declared service creation parameters. To check if the issue is within the service provisioning infrastructure or the configuration outside the MTA, try creating the service using any of the listed methods in Creating Service Instances.

> ### Note
>
> If any `create` parameters are used, make sure that they are the same as those that are used in the MTA. Note that the service creation parameters within the MTA may be located in different places, and their combination is required. For more information, see Service Creation Parameters.

Depending on the result, proceed in one of the following ways:

- If the same error occurs, there is a problem with the service broker rather than the deploy service. Check the following:
  - Check your entitlement and quotas. Make sure that the quota is enough.
  - Check if the service parameters are correct.

  If none of the above solves the problem, create an incident to the corresponding service broker component.
- If the above error is not reproduced, download the logs as described in Troubleshooting [page 838]. Afterward,create an incident [page 838] or contact the SAP support.

Service Update Failure

If the update of the service fails, an error message with the following format will be displayed:

```
Error updating service "<service-name>" from offering "<service-offering>" and
plan "<service-plan>": <cause-of-failure>
```

Usually, such an error is produced when there is a problem with the services provisioning infrastructure. To check if the issue is within the service provisioning infrastructure or the configuration outside the MTA deployment, try updating the service using any of the listed methods in Updating Service Instances.

> ### Note
>
> If any `update` parameters are used, make sure that they are the same as those that are used in the MTA. Note that the service update parameters within the MTA may be located in different places, and their combination is important. For more information, see Service Creation Parameters.

Depending on the result, proceed in one of the following ways:

- If the same error occurs, there is a problem with the service broker rather than the deploy service. Check the following:
  - Check your entitlement and quotas. Make sure that the quota is enough.
  - Check if the service parameters are correct.

  If none of the above solves the problem, create an incident to the corresponding service broker component.
- If the above error is not reproduced, download the logs as described in Troubleshooting [page 838]. Afterward,create an incident [page 838] or contact the SAP support.

Service Delete Failure

If the deletion of the service fails, an error message with the following format will be displayed:

```
Error deleting service "<service-name>" from offering "<service-offering>" and
plan "<service-plan>": <cause-of-failure>
```

Usually, such an error is produced when there is a problem with the services provisioning infrastructure. To check if there is a problem with the service itself, execute the following command:

```
cf delete-service <service-name> -f [-c PARAMETERS_AS_JSON]
```

Depending on the result, proceed in one of the following ways:

- If the same error occurs, there is a problem with the service broker rather than the deploy service. Create an incident to the corresponding service broker component.
- If the above error is not reproduced, download the logs as described in Troubleshooting [page 838]. Afterward,create an incident [page 838] or contact the SAP support.

Application Binding to Service Failure

This error can occur when the services are created and the deployment is in phase, in which the application is being bound to the services. The error has the following format:

```
Could not bind application "<application-name>" to service "<service-name>":
<cause-of-the-error>
```

Usually, this error happens when the service provider for the service fails to initiate the binding. The problem might also occur when the Cloud Foundry Cloud Controller component has internal issues.

Try to resolve the issue by unbinding/binding the application to the service manually, using the commands:

- **`cf bind-service <application-name> <service-name> [ADDITIONAL OPTIONS]`** - binds the application with the given name to service.
- **`cf unbind-service <application-name> <service-name>`** - unbinds the application with the given name from service.

If the above error is not reproduced, download the logs as described in Troubleshooting [page 838]. Afterward,create an incident [page 838] or contact the SAP support.

Application Content Upload Failure

The following error occurs:

```
Error uploading application <specific description>
```

To check if the issue is related to the MTA deployment, execute **`cf push`** instead of **`cf deploy`** using only the application with the problematic content.

If the command fails, there is a problem with the application, or the native Cloud Foundry, such as Cloud Controller or used buildpack(s). Troubleshoot your application and/or contact your application developer.

If the command is executed successfully, the problem is not related to the application, but there is a problem with the MTA deployment. Download the logs as described in Troubleshooting [page 838]. Afterward,create an incident [page 838] or contact the SAP support.

Application Tasks Execution Failure

To check if the issue is related to the MTA deployment, execute **`cf run-task APPNAME "TASK"`** instead of **`cf deploy`** using only the problematic task of the application.

If the command fails, there is a problem with the application, or the native Cloud Foundry, such as Cloud Controller. Troubleshoot your application and/or contact your application developer.

If the command is executed successfully, the problem is not related to the application or the task, but there is a problem with the MTA deployment. Download the logs as described in Troubleshooting [page 838]. Afterward,create an incident [page 838] or contact the SAP support.

Application Starting Failure

In order to locate the problem in the application starting, the logs of the application should be checked.

Download the logs as described in Troubleshooting [page 838]. There should be a dedicated file for the relevant application, which contains more traces for the starting and staging of the application. For instance, if the application is called `my-cf-app`, there would be a file called `my-cf-app.log`.

Normally, the failures during application starting indicate coding or configuration mistakes and self-troubleshooting is a must. Rarely, they might be related to problems in the Cloud Controller or the MTA deployment itself.

The following errors in the console output indicate problems during application starting:

```
Error starting application "<app_name>": Some instances have crashed. Check the
logs of your application for more information.
```

```
Error starting application "<app_name>": Some instances are down. Check the logs
of your application for more information.
```

Application Staging Failure

In order to locate the problem in the application staging, the logs of the application should be checked.

Download the logs as described in Troubleshooting [page 838]. There should be a dedicated file for the relevant application, which contains more traces for the starting and staging of the application. For instance, if the application is called `my-cf-app`, there would be a file called `my-cf-app.log`.

Normally, the failures during application staging indicate coding or configuration mistakes and self-troubleshooting is a must. Rarely, they might be related to problems in the Cloud Controller, buildpack or the MTA deployment itself.

The following error in the console output indicate problems during application staging:

```
Error staging application "<app_name>": …
```

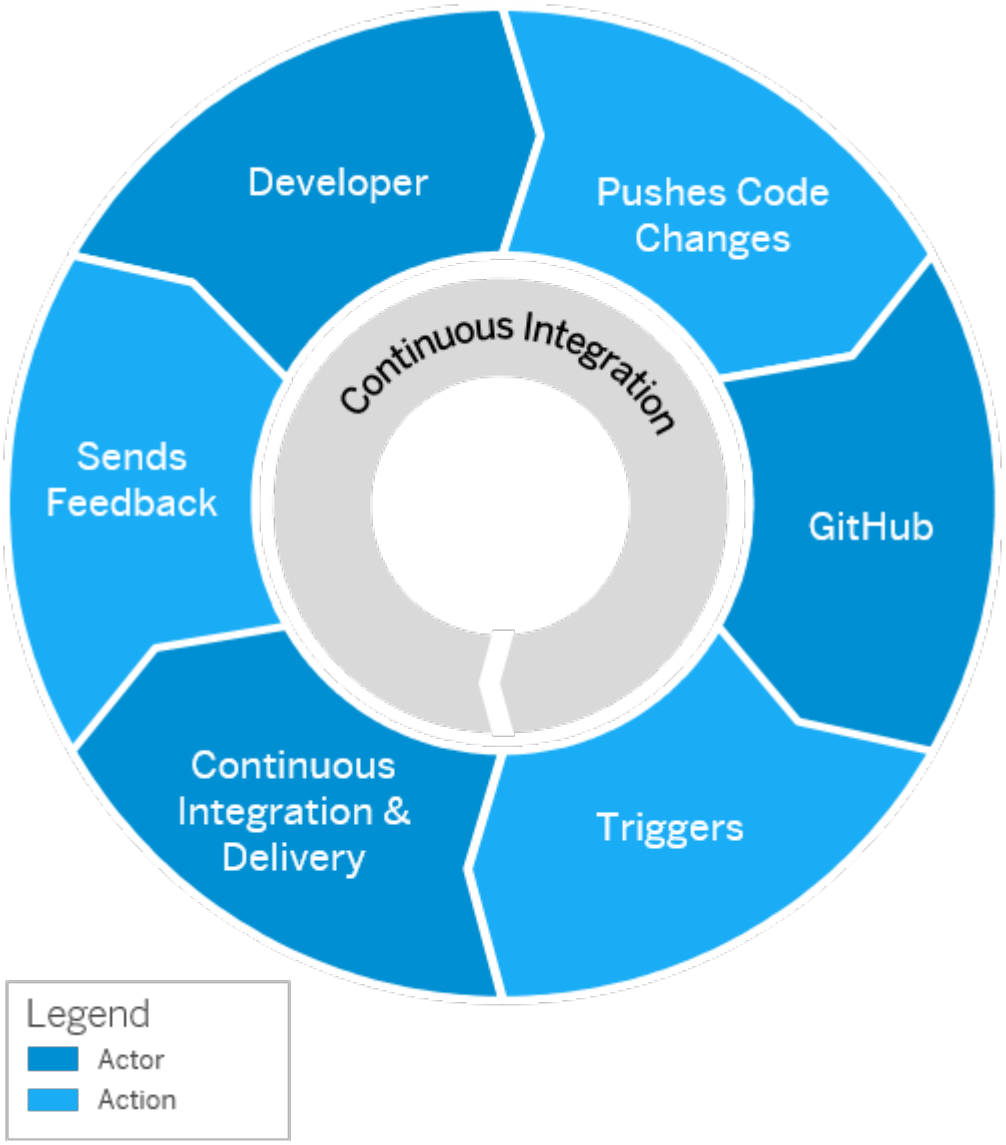
4.1.8.2 Continuous Integration and Delivery

Learn how to integrate CI/CD into your development with the SAP Cloud Application Programming Model (CAP).

4.1.8.2.1 What Are Continuous Integration and Delivery?

Continuous Integration (CI) describes a software development process in which various team members frequently integrate their contributions into a single main line. Before each integration, the changes are verified

through builds and automated testing. This allows you to detect errors as quickly as possible and prevent integration problems before completing the development.



Continuous Integration Flow

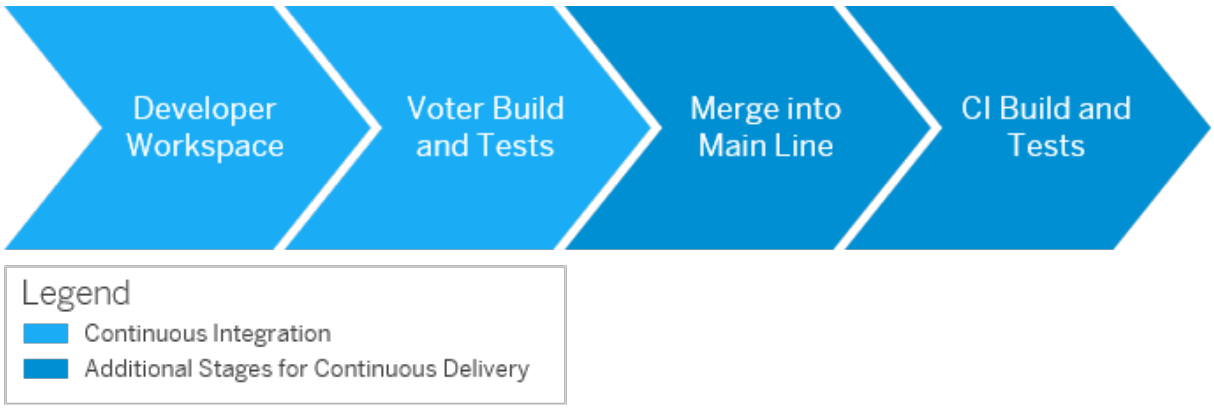
The basic flow of Continuous Integration comprises the following steps:

1. The Developer writes code and pushes the code changes into a repository on a central source code management system (SCM).
2. The Source Code Management System triggers the Continuous Integration (CI) server.
3. The Continuous Integration server runs automated builds and tests and sends feedback about their outcome to the Developer.

As you can see from the figure above, Continuous Integration is a cycle: as soon as the CI server has sent its feedback to the developer, the flow starts over again. The developers either correct their previous code change, which must then be built and tested again, or start working on an entirely new one.

The **Continuous Delivery (CD)** concept expands on the one of continuous integration. It adds the aspect that any change that has successfully passed the tests is immediately ready to be deployed to production, both from a technical and qualitative point of view.

The following graphic shows the relation between Continuous Integration and Continuous Delivery:



Relation Between CI and CD

The Continuous Delivery process ensures that the most current version is successfully built, tested, and provided in a shippable format. Based on the release decision by the development team or delivery manager, it can be shipped to customers or deployed to production at any time.

For more information about the concepts and principles of Continuous Integration and Continuous Delivery, see the SAP Continuous Integration and Delivery.

4.1.8.2.2 SAP Continuous Integration and Delivery

SAP Continuous Integration and Delivery is a service on SAP BTP that helps development teams collaborate more effectively, improve the quality of their software, and speed up their delivery process. It provides ready-made CI/CD pipelines for SAP-specific use cases that can be connected to a source repository to automatically build, test, and deploy code changes. This helps identify issues and prevent integration problems at an early stage in the development process.

The Continuous Integration & Delivery service focuses on two key benefits: **Simplicity** and **Flexibility**.

Simplicity

SAP Continuous Integration and Delivery offers predefined CI/CD pipelines that can be used out of the box. Depending on the development scenario they are used for, they only require minimal configuration. This means that even users without expertise in CI/CD can easily leverage the service for their needs. Additionally, the service includes the necessary infrastructure, freeing you from having to manage it yourself.

Flexibility

By offering the ability to customize pipelines with additional commands, credentials, and variables, SAP Continuous Integration and Delivery provides a high degree of flexibility. With this option, the service does not only cover standard use cases, but also addresses more complex scenarios. Additional commands enable you to expand your pipelines with various tasks that are executed before or after a certain stage. Additionally, extra credentials and variables allow you to further customize your CI/CD jobs.

SAP Continuous Integration and Delivery for CAP

For how to configure a CI/CD pipeline for the development of applications that follow the SAP Cloud Application Programming Model, see Get Started with an SAP Cloud Application Programming Model Project in SAP Continuous Integration and Delivery (tutorial) and Configure a Cloud Foundry Environment Job (documentation).

4.1.8.3 Deploy Business Applications in the Cloud Foundry Environment

When an application for the Cloud Foundry environment resides in a folder on your local machine, you can deploy it and start it by executing the command line interface (CLI) command `push`. To deploy business applications bundled in a multitarget application archive, you have to use the command `deploy-mta`.

- For more information about developing and deploying applications in the Cloud Foundry environment, see http://docs.cloudfoundry.org/devguide/deploy-apps/deploy-app.html .
- For more information developing Multitarget Applications, see Multitarget Applications in the Cloud Foundry Environment [page 615]. See Multitarget Application Commands for the Cloud Foundry Environment [page 2834] for information about MTA deployment.

Related Information

Download and Install the Cloud Foundry Command Line Interface [page 2813]
Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814]
Multitarget Application Commands for the Cloud Foundry Environment [page 2834]

4.1.8.4 Deploy Docker Images in the Cloud Foundry Environment

The high-level steps to application deployment using Docker images.

Prerequisites

- You are aware how `cf push` works when you deploy a regular Cloud Foundry application.
- You need a higher degree of freedom and know that this freedom comes with higher responsibility for application operation.
- Your Docker image resides in a highly available container registry.

Context

A Docker image deployed to the Cloud Foundry environment has to adhere to the same boundary conditions as regular Cloud Foundry applications. Cloud Foundry applications deployed as Docker images are running in Linux containers with user namespaces enabled, which prevents certain functionality like mounting FuseFS devices. Another difference is the usage of buildpacks. Docker images can't be used in combination with buildpacks. If you want to use certain functionality provided by the buildpack, you have to build it into your Docker image.

Compared to regular Cloud Foundry applications, using a Docker image means more effort for you. If the following factors are fulfilled, using Docker on Cloud Foundry might be a viable option.

1. Managing a whole cluster on Kubernetes is too much overhead for your scenario.
2. The usual approach doesn't let you realize what you want to achieve.

Here are some features of the Cloud Foundry environment you have to implement yourself in your Docker image. The comparison is made with the SAP Java buildpack.

Supported with SAP Java buildpack	Description
Dynatrace	Enable applications for Dynatrace based monitoring.
XSUAA	Automatic validation of OAuth token and mapping of OAuth scopes to JEE roles, if `auth-method` is set to XSUAA in `web.xml`.
Dynamic Logging	Set of log level via CLI without restart.
Switch to debug via CLI	
OS & JVM management	Consume newer versions (especially patches) of the underlying OS and JVM regularly..

Procedure

1. Your Docker image adheres to the 12 factor principles.

   https://12factor.net/

2. You only use supported features in the SAP BTP, Cloud Foundry environment, see Supported and Unsupported Cloud Foundry Features [page 64]
3. Read what the open source documentation says about deploying an application with Docker.

   Pay special attention to the mentioned requirements. Deploy an App with Docker

   Requires traffic routing to the lowest numbered port. How Diego Runs and Monitors Processes

4. Use `cf push` with the `--docker-image` parameter.

   This is an example if you want to push from Docker Hub:

   ```
   cf push <APP-NAME> --docker-image <REPO>/<IMAGE>:<TAG>
   ```

   Also pay attention to the section on private registries and Google Container Registry. Push a Docker Image From Docker Hub

5. Operate and patch your Docker images.

   Docker images contain everything that is necessary to run a process, including the operating system and libraries into one binary blob. Consequently, staging a Docker image in Cloud Foundry does not provide the same separation of droplet and stack that is available for a Cloud Foundry application staged using a buildpack.

   If there is an operating system and library security vulnerability, the developer is responsible to `cf push` a new version of the Docker images . The developer using the platform has the responsibility for watching these security vulnerabilities and for reacting accordingly.

   For scale or restart operations, the image is pulled again from the container registry. The container registry has to be highly available and reachable from network side for proper operations.

Results

Was this topic helpful? Did you miss something? Let us know and use the feedback function *(feedback)*.

4.2 Development in the ABAP Environment

Learn more about developing applications in the ABAP environment.

Overview

The ABAP environment is a platform as a service that allows you to extend existing ABAP-based applications and develop ABAP cloud apps decoupled from the digital core. You can leverage your ABAP know-how in the cloud and reuse existing ABAP assets by writing your source code with ABAP Development Tools for Eclipse.

The ABAP environment enables you to **expose**:

- OData services. See ABAP RESTful Application Programming Model.
- HTTP services. See Working with the HTTP Service Editor.
- SQL services. See Data Federation Using the SQL Service.
- RFC function modules. See Providing an RFC Service.
- SOAP provider model. See Providing a SOAP Service.

With your ABAP applications, you can **consume**:

- HTTP services (HTTP client). See Outbound HTTP Administration Tasks.
- OData services (service consumption model). See Developing a UI Service with Access to a Remote Service.
- Remote Function Calls (RFC). See Outbound RFC Administration Tasks.
- On-premise systems via Cloud Connector. See Integrating On-Premise Systems [page 1214].
- SOAP-based Web services. See Outbound SOAP Administration Tasks.

Technical Limits and Boundary Conditions for ABAP Applications

For elasticity, operability and protection reasons, the runtime of API or UI requests to ABAP applications is limited. After 10 minutes, running requests are automatically canceled. Tasks requiring longer runtimes have to be executed in Application Jobs, as background processes via the background processing framework (bgPF), or need to be split into smaller work packages. For an optimal user experience, all ABAP applications, including application jobs, have to be prepared to be interrupted and restarted even earlier than the maximum runtime. This allows to provide a continuous application service, unimpacted from scaling operations, maintenance activities or even infrastructure failures.

Development Resources

ABAP RESTful Application Programming Model

ABAP Development Tools for Eclipse: User Guide

ABAP CDS Development User Guide

Generative AI in ABAP Cloud

ABAP Keyword Documentation [page 855]

ABAP Lifecycle Management [page 861]

Connect to the ABAP System [page 851]

Providing an HTTP Service

Integrating On-Premise Systems [page 1214]

Supported Protocols and Authentication Methods

Inbound Communication

Working with abapGit [page 870]

Related Information

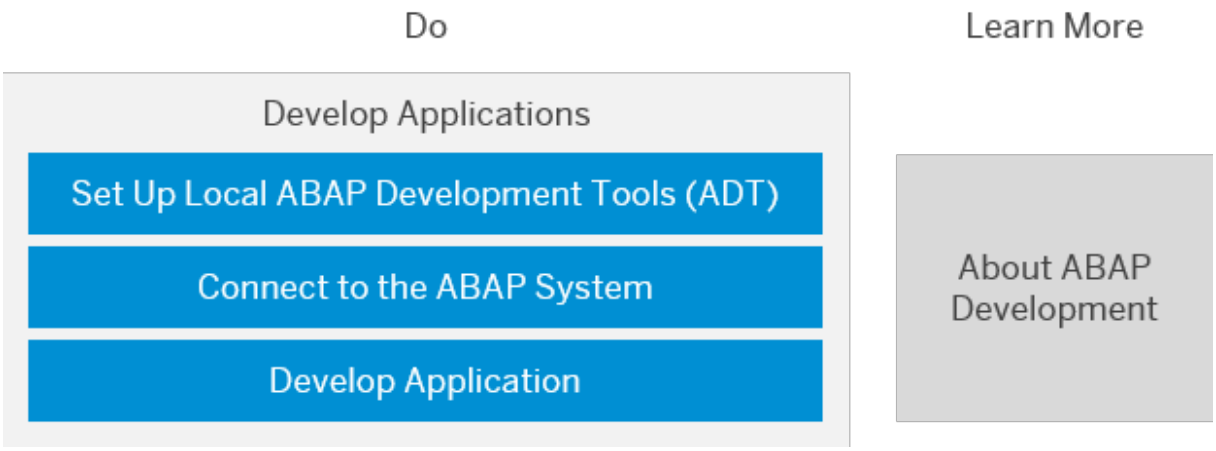
Eclipse Tool for the ABAP Environment
Learning Journey
Manage Software Components [page 3222]
Video Tutorials

4.2.1 Getting Started as a Developer in the ABAP Environment

Learn about your first steps to get started as a developer in the ABAP environment.

> **Note**
>
> This documentation informs you about the first steps as a developer and assumes that the necessary steps for setting up the ABAP environment are done. For more information about how to set up the ABAP environment and getting started as an administrator, see Getting Started with a Customer Account in the ABAP Environment [page 184].

Developing Applications: Overview



- https://tools.hana.ondemand.com/#abap [https://tools.hana.ondemand.com/#abap]
- Connect to the ABAP System [page 851]
- https://developers.sap.com/tutorials/abap-environment-gcts.html#b1afe95c-9acb-4f75-8f80-1abe93cb524f [https://developers.sap.com/tutorials/abap-environment-gcts.html#b1afe95c-9acb-4f75-8f80-1abe93cb524f]
- https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/31367ef6c3e947059e0d7c1cbfcaae93.html [https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/31367ef6c3e947059e0d7c1cbfcaae93.html]
- https://help.sap.com/viewer/DRAFT/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/59656c2f858748fe976456248d390c5c.html [https://help.sap.com/viewer/DRAFT/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/59656c2f858748fe976456248d390c5c.html]
- https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/31367ef6c3e947059e0d7c1cbfcaae93.html [https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/31367ef6c3e947059e0d7c1cbfcaae93.html]

1. Download and install ABAP Development Tools (ADT) from https://tools.hana.ondemand.com/#abap. See Video Tutorial: Configure ABAP Development Tools .

   > **Note**
   >
   > SAP GUI is not supported in the ABAP environment. You can only use ADT as your development environment.

2. Create an ABAP cloud project with ADT to connect to the ABAP system in the ABAP environment (see Connect to the ABAP System [page 851]).
3. Develop your application (see Development in the ABAP Environment). To learn more about how to develop applications, see Tutorial: Create Your First ABAP Console Application and Video Tutorial: Create Application .

More Information About Development in the ABAP Environment

For more information about ABAP development, see Development in the ABAP Environment.

4.2.1.1 Connect to the ABAP System

Set up your ABAP cloud project to connect to the ABAP system.

Prerequisites

- You are assigned the developer role. See Assigning the ABAP Developer User to the ABAP Developer Role [page 217] and Creating Service Keys.

  > **Note**
  >
  > The process for connecting to an ABAP service instance URL when creating a new ABAP Cloud project in ABAP development tools for Eclipse has changed. For more information, see ABAP Service Instance URL [page 1881].

- You have downloaded and installed the front-end components of ABAP Development Tools (ADT). See Video Tutorial: Configure ABAP Development Tools.
- You are a member of the relevant space in the Cloud Foundry environment, see Add Space Members Using the Cockpit, **or** you have a service key in JSON format, see Creating Service Keys.

Procedure

1. Open Eclipse and select *File* > *New* > *ABAP Cloud Project* from the menu.
2. To establish a service instance connection, you have the following options:
   a. Insert the URL from the SAP Fiori launchpad of your system.
   b. If you still have a service key available, you can extract the URL from it by clicking the link shown in the description of the window you're in: *Do you have a service key? Extract the URL from the service key.* In this case, a new window opens up where you can paste in your service key. Next, choose *Copy to Clipboard*. Now, you're in the *Connect to an ABAP Service Instance* window again and you can paste in the service instance URL. Choose *Copy logon URL to clipboard* and paste the URL into your browser to log on to the system.
3. To create your ABAP cloud project, select *Finish*.

Results

You have created an ABAP cloud project that is added to the *Project Explorer*.

> **Note**
>
> To verify your result, expand the first level of the project structure. Make sure that the following nodes are included:
>
> - *Favorite Packages*: Contains the local packages and the packages that you have added to your favorites.
> - *Released Objects*: Contains all released SAP objects that are available to use or reuse.

Related Information

Video Tutorial: Create ABAP Cloud Project
Video Tutorial: Create Package & Persistence
Tutorial: Create Your First ABAP Console Application
ABAP Cloud Projects

4.2.2 ABAP Development Tools: User Guides

Get an overview of all available user guides for ABAP development tools.

> **Note**
>
> Certain features of this product require an additional license. For more information, see SAP Joule for Developers, ABAP AI Capabilities.

ABAP Development Tools for Eclipse

This guide describes the tool set and functionality of ABAP development tools for Eclipse:

- SAP Joule for Developers, ABAP AI capabilities supports you in the end-to-end development using AI and can be integrated into ABAP development tools for Eclipse. It's currently available for customers using SAP Business AI Platform, ABAP environment, SAP S/4HANA Cloud Public Edition or SAP S/4HANA Cloud Private Edition. To leverage SAP Joule for Developers, ABAP AI capabilities, you need to purchase an additional license. See Getting Started.
- Within the ABAP core development scenario: In particular, it focuses on use cases for creating, editing, testing and debugging development objects like ABAP classes, ABAP Dictionary objects, function modules and so forth.
- For Core Data Services (CDS) in ABAP development for SAP HANA scenarios: In particular, it focuses on use cases for creating, editing, testing, and analyzing ABAP CDS entities like data definitions and so forth.
- When developing all types of SAP Fiori applications as well as publishing Web APIs in the context of SAP Joule for Developers, ABAP AI capabilities.
- When creating lifecycle-stable and cloud-ready business apps, services, and extensions in the context of the ABAP Cloud development model.

- For the integration of native SAP HANA objects (in the context of SAP HANA repository content) within the ABAP layer.

ABAP Development Tools for Visual Studio Code

This guide describes the tool set and functionality of ABAP development tools for Visual Studio Code:

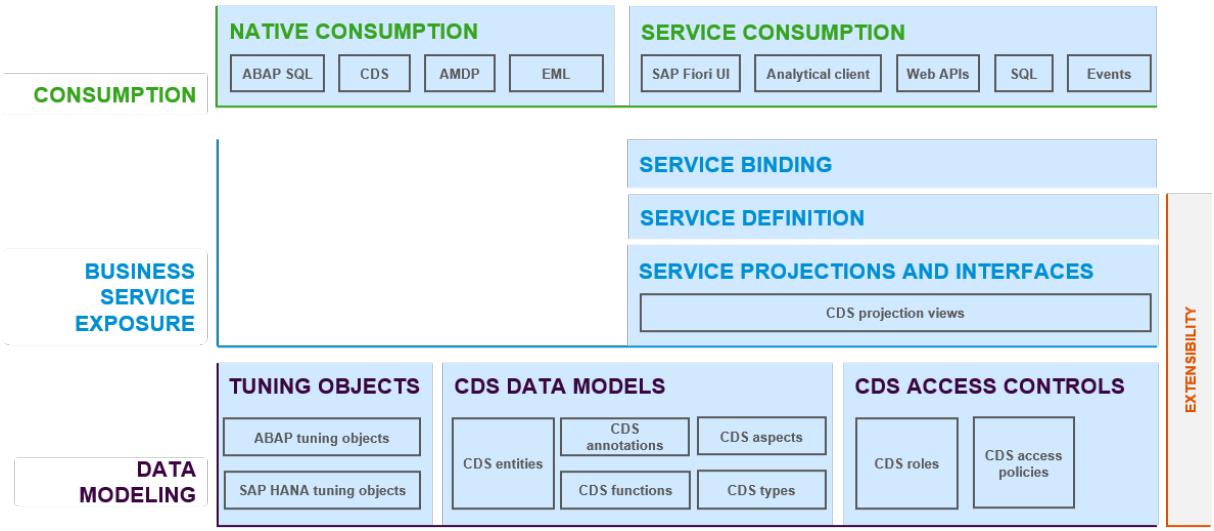
- Within the ABAP Cloud development model: In particular, it focuses on use cases for creating, editing, testing, and maintaining ABAP development objects in a modern development environment.
- For quality assurance and testing: Integrated tools such as the ABAP test cockpit (ATC) and ABAP Unit tests help to ensure code quality.
- Core Data Services (CDS) focuses on use cases for creating, editing, and maintaining ABAP CDS entities for data modeling.
- ABAP RESTful Application Programming Model (RAP) supports end-to-end application development, including the implementation of business logic, services, and user interfaces.
- SAP Joule for Developers, ABAI AI capabilities extends ABAP development tools for Visual Studio Code with AI-powered development assistance that supports developers across the software development process. Its Agentic AI capabilities based on Model Context Protocol (MCP) tools enable AI agents to support and accelerate development tasks.

AI in ABAP Cloud

This guide describes how AI supports you in the end-to-end development for ABAP Cloud: SAP Joule for Developers, ABAP AI capabilities is an integrated set of AI-based features that supports you in the end-to-end development using AI to enhance your developer experience. Agentic AI focuses on building goal-driven ABAP applications by using AI agents that execute development tasks with minimal human intervention through the open Model Context Protocol (MCP) standard. This enables complex development workflows to be automated and executed faster, increase your efficiency and reduce the total cost of development.

4.2.3 ABAP Data Models Guide

In the ABAP Cloud development model, data models used for ABAP applications are based on ABAP Core Data Services (ABAP CDS). ABAP CDS is used for defining and consuming semantically-rich data models optimized for SAP HANA. ABAP data models can be used as the foundation for analytical and transactional ABAP applications, as well as enterprise search models. The ABAP Data Models Guide provides information on the different aspects of ABAP data models and their components.



- https://help.sap.com/docs/abap-cloud/abap-data-models/abap-tuning-objects?locale=en-US [https://help.sap.com/docs/abap-cloud/abap-data-models/abap-tuning-objects?locale=en-US]
- https://help.sap.com/docs/abap-cloud/abap-data-models/tuning-objects?locale=en-US [https://help.sap.com/docs/abap-cloud/abap-data-models/tuning-objects?locale=en-US]
- https://help.sap.com/docs/abap-cloud/abap-data-models/annotations-in-abap-cds-b22c173af13f426689e977ecff7ccf2b?locale=en-US [https://help.sap.com/docs/abap-cloud/abap-data-models/annotations-in-abap-cds-b22c173af13f426689e977ecff7ccf2b?locale=en-US]
- https://help.sap.com/docs/abap-cloud/abap-data-models/data-types-in-abap-cds-f04708ca02d345e18fecbbd9a1eb9eb7?locale=en-US [https://help.sap.com/docs/abap-cloud/abap-data-models/data-types-in-abap-cds-f04708ca02d345e18fecbbd9a1eb9eb7?locale=en-US]
- https://help.sap.com/docs/abap-cloud/abap-data-models/functions-in-abap-cds?locale=en-US [https://help.sap.com/docs/abap-cloud/abap-data-models/functions-in-abap-cds?locale=en-US]
- https://help.sap.com/docs/abap-cloud/abap-data-models/entity-modeling?locale=en-US [https://help.sap.com/docs/abap-cloud/abap-data-models/entity-modeling?locale=en-US]
- https://help.sap.com/docs/abap-cloud/abap-data-models/cds-authorization-concept?locale=en-US [https://help.sap.com/docs/abap-cloud/abap-data-models/cds-authorization-concept?locale=en-US]
- https://help.sap.com/docs/abap-cloud/abap-data-models/cds-projection-views?locale=en-US [https://help.sap.com/docs/abap-cloud/abap-data-models/cds-projection-views?locale=en-US]
- https://help.sap.com/docs/abap-cloud/abap-data-models/cds-aspects?locale=en-US [https://help.sap.com/docs/abap-cloud/abap-data-models/cds-aspects?locale=en-US]

4.2.4 ABAP Keyword Documentation

The ABAP keyword documentation describes the syntax and meaning of the keywords of the ABAP language and its object-oriented part – ABAP objects.

Context

The ABAP keyword documentation provides you with context-sensitive information for your ABAP source code.

Accessing ABAP Language Help

To access the ABAP language help from the source code editor, position your cursor on an ABAP statement for which you need help, and press `F1`. The language help is displayed in a separate window.

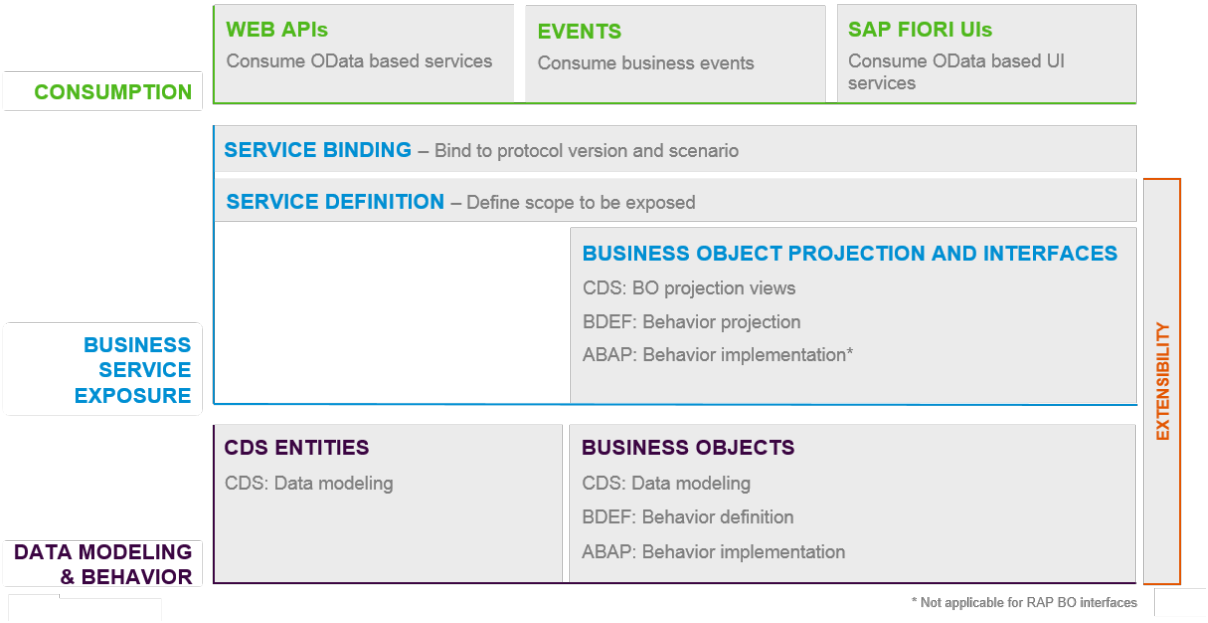
To view the entire ABAP keyword documentation, see ABAP - Keyword Documentation.

4.2.5 ABAP RESTful Application Programming Model

The ABAP RESTful Application Programming Model defines the architecture for efficient end-to-end development of intrinsically SAP HANA-optimized OData services (such as SAP Fiori apps) in the ABAP environment. It supports the development of all types of Fiori applications as well as publishing Web APIs. It is based on technologies and frameworks such as Core Data Services (CDS) for defining semantically rich data models and a service model infrastructure for creating OData services with bindings to an OData protocol and ABAP-based application services for custom logic and SAPUI5-based user interfaces.

Architecture Overview

RAP - The big picture



- https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/cc80e974d128426d9d876d5a8fc9e4aa.html#loiocc80e974d128426d9d876d5a8fc9e4aa__Design_Time [https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/cc80e974d128426d9d876d5a8fc9e4aa.html#loiocc80e974d128426d9d876d5a8fc9e4aa__Design_Time]
- https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/f2cbcacaf8b74540b0708fc143875bc3.html#loiof2cbcacaf8b74540b0708fc143875bc3__Web_API [https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/f2cbcacaf8b74540b0708fc143875bc3.html#loiof2cbcacaf8b74540b0708fc143875bc3__Web_API]
- https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/b58a3c27df4e406f9335d4b346f6be04.html [https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/b58a3c27df4e406f9335d4b346f6be04.html]
- https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/b09e4d53bfca4544a9f8910bcc2cd9d6.html [https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/b09e4d53bfca4544a9f8910bcc2cd9d6.html]
- https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/6e7a10d30b74412a9482a80b0b88e005.html [https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/6e7a10d30b74412a9482a80b0b88e005.html]
- https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/a3ff9dcdb25a4f1a9408422b8ba5fa00.html [https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/a3ff9dcdb25a4f1a9408422b8ba5fa00.html]
- https://help.sap.com/docs/btp/sap-abap-restful-application-programming-model/query?version=Cloud [https://help.sap.com/docs/btp/sap-abap-restful-application-programming-model/query?version=Cloud]

- https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/f2cbcacaf8b74540b0708fc143875bc3.html [https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/f2cbcacaf8b74540b0708fc143875bc3.html]
- https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/1913ad9f52e64ab5858df00a8d20c4d6.html [https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/1913ad9f52e64ab5858df00a8d20c4d6.html]
- https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/c4aaecac48294ef1a39ef13de0706a4b.html [https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/c4aaecac48294ef1a39ef13de0706a4b.html]
- https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/0b925bc556d4491aad395b21ec2566ff.html [https://help.sap.com/viewer/923180ddb98240829d935862025004d6/Cloud/en-US/0b925bc556d4491aad395b21ec2566ff.html]

Related Information

Tutorial: Get to Know the ABAP RESTful Application Programming Model

4.2.6 Generative AI in ABAP Cloud

The Generative AI in ABAP Cloud guide provides information on various AI use cases for ABAP Cloud.

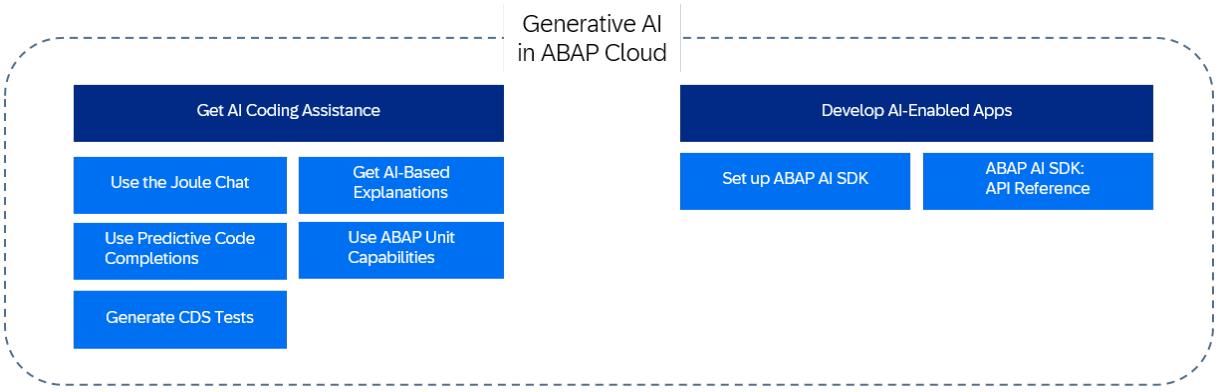
Use Cases

Generative AI in ABAP Cloud is designed to support you with the following use cases:

- Support you with your day-to-day tasks when working with ABAP code, to increase your efficiency and to lower total cost of development
- Include generative AI in your custom business applications
  The Generative AI in ABAP Cloud guide provides information on the different use cases for generative AI in ABAP Cloud.

> **Note**
>
> To leverage Joule's capabilities, you need to purchase an additional license: For SAP Business AI Platform, ABAP environment, SAP S/4HANA Cloud Public Edition, and SAP S/4HANA Cloud Private Edition, see SAP Note 3571857 .

Generative AI in ABAP - The Big Picture



- SAP Joule for developers, ABAP AI capabilities consists of a set of AI-based features that can be integrated into ABAP development tools for Eclipse. [https://help.sap.com/docs/abap-ai/generative-ai-in-abap-cloud/getting-ai-coding-assistance-with-joule-for-developers-abap-ai-capabilities]
- Get support while developing in ABAP development tools for Eclipse by using the Joule chat for questions and tasks. [https://help.sap.com/docs/abap-ai/generative-ai-in-abap-cloud/use-joule-chat]
- Get detailed explanations while developing in ABAP development tools for Eclipse by using the Explain feature. [https://help.sap.com/docs/abap-ai/generative-ai-in-abap-cloud/get-ai-based-explanations]
- Learn how to use the predictive code completion in ABAP development tools for Eclipse. [https://help.sap.com/docs/abap-ai/generative-ai-in-abap-cloud/use-predictive-code-completion]
- Learn how AI-based ABAP Unit features can help you generate unit test methods and improve your existing test code. [https://help.sap.com/docs/abap-ai/generative-ai-in-abap-cloud/use-abap-unit-capabilities]
- Use SAP Joule for developers, ABAP AI capabilities to boost your efficiency in generating unit test class for ABAP CDS entity. [https://help.sap.com/docs/abap-ai/generative-ai-in-abap-cloud/use-cds-test-class-generator]
- Using the ABAP AI SDK powered by Intelligent Scenario Lifecycle Management, you can enhance your own custom ABAP-based applications with integrated generative AI features. [https://help.sap.com/docs/abap-ai/generative-ai-in-abap-cloud/developing-your-own-ai-enabled-applications]
- To be able to use the APIs in the ABAP AI SDK powered by Intelligent Scenario Lifecycle Management (ISLM), your system administrator must perform some setup steps, including setting up system connections with the generative AI hub, where the LLM access service resides. As a developer, you create intelligent scenarios and intelligent scenario models to store information such as the relevant large language model. [https://help.sap.com/docs/abap-ai/generative-ai-in-abap-cloud/set-up-abap-ai-sdk-powered-by-intelligent-scenario-lifecycle-management]
- The ABAP AI SDK is a reuse library in an ABAP system that provides access to generative AI functionality via Intelligent Scenario Lifecycle Management (ISLM). Use it to interact with large language models and to call released ABAP classes and interfaces directly in your application code through application programming interfaces (APIs). [https://help.sap.com/docs/abap-ai/generative-ai-in-abap-cloud/api-reference-guide-for-abap-ai-sdk]

4.2.7 Assuring Quality of ABAP Code

This is an overview of ABAP quality tools that help you to keep your custom code clean and performant.

The two most important quality tools for your custom ABAP code are:

- *ABAP Unit* for dynamic tests
- *ABAP Test Cockpit (ATC)* for static code analysis

ABAP Unit

Writing ABAP Unit tests is the way to provide high-quality software, which can be easily evolved over time without introducing regressions. In methodologies, like extreme programming and test-driven development, the role of unit testing is even more important.

ABAP unit is the state-of-the-art unit testing framework for ABAP. It's embedded into the ABAP programming language, which supports you in writing unit tests. In the ABAP Development Tools for Eclipse (ADT), you have various possibilities to execute the unit tests and to evaluate the results concerning functional correctness and code coverage.

We recommend measuring the test coverage of all objects in a transport before releasing the transport. A high coverage (above 80%, for example) and a wide variety of test methods help to identify issues and make the code more robust.

For more information, see Unit Testing with ABAP Unit.

ABAP Test Cockpit (ATC)

ATC is the tool of choice for all kinds of static code analysis in ABAP. As an ABAP developer, you can use ATC in the ABAP Development Tools for Eclipse to find potential bugs already during the development phase.

In particular, you can check ABAP development objects for many types of problems, including syntax errors, potential problems in performance, suboptimal or potentially faulty programming, adherence to standards, errors in ABAP Unit testing, among others.

For more information, see Checking Quality of ABAP Code with ATC.

ATC Check Variant

The basic configuration of the ATC is the so-called ATC check variant, which includes the set of checks that should be executed. The default variant is set to ABAP_CLOUD_DEVELOPMENT_DEFAULT. It includes all recommended checks to fulfill the ABAP cloud code quality standard incl. the execution of ABAP Unit tests.

You can create a copy of this variant and add additional checks. You can also add custom created checks to your variant.

For more information, see Working with ATC Checks.

Transports

The basic quality gate is the release of transport tasks and transport requests.

For findings with priority level 1 and 2, the release of tasks and requests is blocked by the system in the default setting. For findings with priority level 3, the developer only gets a notification. This default setting can be individually sharpened based on your needs.

For more information, see Working with ATC During Transport Release.

Test Runs

You can either set up scheduled automated test runs for the ATC, or trigger it manually via ADT.

For more information, see Working with ATC During Development.

ATC Exemptions

Sometimes ATC findings can't be corrected. For example, it's possible that a correction is planned for an upcoming development cycle or only a test program is affected. It could also happen, that checks return false positives.

In this case, you can still process the issue by requesting an exemption. Once approved, it masks an ATC error or warning message. The finding then isn't displayed as an open issue in the ATC results anymore.

In the ABAP environment, exemptions are transportable objects of the type CHKE. The full workflow for the request and approval of exemption is integrated in ADT.

For more information, see: Working with ATC Exemptions.

ABAP Test Cockpit Configurator App

You can use the ATC Configuration app to set up the default variant. You can also change the priorities of check messages, set priority levels that block or interrupt transport releases, and set the handling of pseudo comments and pragmas.

For more information, see ABAP Test Cockpit Configurator.

4.2.8 Background Processing Framework

The background processing framework (bgPF) is a framework that asynchronously and reliably executes methods of applications that develop background processes.

The background processing framework (bgPF) is designed to execute methods of applications that develop background processes asynchronously and reliably. It is beneficial in applications that require time-consuming processes, enabling users to continue with other tasks while bgPF handles the processes in the background. bgPF is built on the background remote function call (bgRFC). It supports both transactional and queued background processes. In conjunction with the ABAP RESTful Programming Model (RAP), bgPF provides transactional control to maintain data consistency. The framework offers several advantages, including transactional consistency and integration with various ABAP tools. It is part of the standard functionality in ABAP Cloud and supports easy code testing.

Find more information in the Background Processing Framework documentation.

4.2.9 ABAP Lifecycle Management

This chapter helps you to plan and set up your landscape and lifecycle management by describing how software components can be used and which possibilities you have for transportation of development objects and business configuration.

4.2.9.1 Basic Concepts and Terms

To better understand how software lifecycle management with gCTS in the ABAP environment can be implemented, let's start off with some basic concepts and terms.

Classical Git

Git

Git is a distributed version-control system.

Distributed version control is a form in which development objects and all their versions and metadata are mirrored from a central place to every developer's computer. The developer can switch locally and offline between different versions and change all of them before updating the central originals later.

Repository

A repository is a collection of objects, their directory structure, and metadata, for example, a historical record of changes in the repository, a set of commit objects, and a set of references to commit objects.

The central place that all objects are mirrored from and saved to, after having finished changes on a developer's computer, is the so-called remote repository. Every developer's computer holds a so-called local repository to work on.

Branch

Git repository branches can be used to control the flow of changes through the test and production landscape. They are used to separate changes from each other, which shall or might not be delivered together (i.e. development vs. correction, feature A vs. feature B). A branch is created on the current state of another branch, the parent branch, and reflected by a new copy of all included objects, that is, the commit history of the parent branch. Depending on which branch is supposed to be changed, the corresponding copy is worked on.

Pull

Pulling means getting the code that is currently in the remote repository into the local repository.

Push

Pushing means getting the code that is currently in the local repository into the remote repository.

Commit

A commit records the state and content changes of a file. It bundles multiple changes and saves it to the local repository. When you perform a git push, all commits are transferred to the remote git.

Check Out

Checking out means that, if several branches are available, the one that is selected is added to the working tree. The working tree consists of the objects of the branch you are currently working on. It contains the state of these objects since they were last pulled and the changes that were made locally but are not yet pushed to the central remote repository. This checked out branch/working tree is what is displayed in the application to edit the objects.

ABAP and ABAP with gCTS

ABAP System/Instance

SAP Business Technology Platform offers a service called ABAP System (abap). If you want to develop with ABAP in the cloud, you have to create an instance of that service, see Create an ABAP System. This service instance provides the ABAP system that you are developing in. The terms ABAP system and ABAP instance can and are often used synonymously, however, we refer to the term ABAP system.

Transport Request

A transport request records all the changes in your ABAP development system. Once a transport request is released, the changes are pushed into your central Git repository, to be more precise a branch of it, which is by default the main branch, in the cloud as a commit represented by a commit ID. During the import (pull or checkout), the delta of the Git repository branch content between the ABAP system and central Git repository is imported.

> ⓘ Note
>
> Transport requests cannot be imported selectively into subsequent ABAP systems but as part of software component branches their changes were added to.

Release

A release, for example with the format YYYY-<nn>, is a set of changes that are tested successfully in your quality ABAP system(s) and imported into your production ABAP systems after a release decision. It is represented by a release branch in the Git repository. Once the release decision has been made, you have to create and pull (import) the new release branch into the production ABAP system.

4.2.9.2 Transport Mechanisms

There are two git-based transport mechanisms in the ABAP environment.

Git-based CTS (gCTS) is the evolution of the classical Change and Transport Management System (CTS). It is the recommended approach for transporting objects between ABAP systems in your global account. It offers built-in and easy to use functionalities provided by the Manage Software Components app in your SAP Fiori launchpad.

abapGit is an open-source Git client that allows you to import existing code into your ABAP system. You should use it for the following use cases:

- Migrate on-premise code to the cloud. See Use abapGit to Transform ABAP Source Code to the Cloud.
- Transfer your code from the cloud to on premise. See Transfer Your ABAP Source Code With SAP BTP ABAP Environment via abapGit.
- Export your code when the development ABAP system is decommissioned.
- Transfer your code from one cloud to another to share it with others, for example as open source, or from a partner to a customer account.
- Implement mechanisms for distributed development and testing in dedicated development/test ABAP systems. This can be useful for special projects, such as proof of concepts or features that are dependent on regular development of a solution but shall run independent from its lifecycle.
- To learn more about abapGit, see Working with abapGit [page 870].

For a quick start on transportation, see Transport a Software Component Between Two ABAP Systems. For more information about the according delivery process, see Delivery via Add-On or gCTS.

Transports within one or several global accounts

Transports in the ABAP environment are limited to a single global account. Use gCTS (Manage Software Components app) to transport ABAP development objects between systems within the same global account, following the standard ABAP transport lifecycle. For example, you can transport objects from DEV to TEST to PROD. You can't use gCTS (Manage Software Components app) to transport objects across global accounts. To move ABAP code between systems that belong to different global accounts, use abapGit to import and export the source code.

gCTS vs. abapGit

Issue	gCTS (transport)	abapGit (code import/export)
purpose	regularly scheduled transport in system landscape	distribution of source code in development systems
scope	within a global account	extending beyond global accounts as well
target lifecycle	3 system landscape: DEV → TEST → PROD or 5 system landscape	only DEV to DEV is possible

4.2.9.2.1 Software Components

A software component is the container that organizes and transports your ABAP development. It bundles all development objects of at least one runnable application under a top-level structure package, with sub-packages of type *Development* holding the actual objects.

Concept

Your development is organized and managed in software components. In most cases, a software component is designed to contain all development objects of at least one application and should be runnable itself. However,

you can distribute your development across multiple software components. This approach works well when development parts have independent lifecycles or when you need to separate basic reuse functionality from application-specific functionality.

The software component `ZLOCAL` is available by default. It works like `$TMP` in a classic ABAP system: use it for non-transportable, local development. You can't create development objects directly in the structure package. First, create a sub-package of type *Development*. Then, start developing in it. The project structure looks as follows:

> ### Example
>
> `Z_BONUS_MANAGEMENT` (top-level package, type Structure - created automatically with the software component of the same name)
>
> `Z_BONUS_CALCULATION` (sub-package, type Development)
>
> `Z_BONUS_CONFIGURATION` (sub-package, type Development)

> ### Example
>
> with a registered namespace:
>
> `/ACME/BONUS_MANAGEMENT` (top-level package, type Structure - created automatically with the software component of the same name)
>
> `/ACME/BONUS_CALCULATION` (sub-package, type Development)
>
> `/ACME_BONUS_CONFIG` (sub-package, type Development)

If you have registered a namespace at SAP, it is automatically provided during provisioning of your ABAP environment. The developer key and repair key are created and assigned automatically by the ABAP system, you do not need to enter them manually. In the namespace application, you can search for your key assignments by filtering the installation number `CLOUDSYSTEM`.

For more information, see

- SAP Note 105132 - How to reserve a namespace
- ONE Support Launchpad: Namespace Application - Manage Your Namespaces

Creating a Software Component

You create your software components in the development system using the SAP Fiori app Manage Software Components. Here, you choose the business catalog Lifecycle Management - Software Components with catalog ID `SAP_A4C_BC_MSCL_PC`. For an exact step-by-step guide, please follow Manage Software Components. After cloning the software component, you will have the corresponding structure package with the same name available in your system instance.

Using Objects Across Software Components

Objects from one software component cannot be used in another one by default. Software components expose their functionality only through explicitly released application programming interfaces (APIs). If you only need

to permit access without the quality contract of a released API, you can define access permissions as part of a software component relation.

Mechanism	When to use	Quality Contract
Released APIs	stable consumptions across software components (recommended default)	consistency and stability guranteed
Software Component Relations	access without full API quality (e.g. internal reuse)	access permission only, no stability guranteed

Software components should *not have cyclic dependencies* (objects in software component A use objects in software component B and vice versa). Design dependencies in a layered way. For example, group basic reuse functions in a dedicated foundation software componant. Lifecycle processes for software components can run independently as long as no development dependencies exist between them. Where dependencies do exist, declare them explicitly via Software Component Relations so that they become transparent.

For more information, see

- Released APIs
- Finding Released APIs and Deprecated Objects
- Software Component Relations

Restrictions

> ⚠ Restriction
>
> Plan your software component layout carefully. The following operations are not supported:
>
> - You can't move development objects from one software component to another.
> - You can't move objects between packages that reside in different transportable software components.
> - You can't move objects between packages that reside in a transportable software component to `ZLOCAL`.

If you need to relocate objects despite these constraints, see Changing the Package Assignment of Development Objects for the supported workflows (e.g. recreation or abapGit transfer). In addition, gCTS supports only specific object types. For the complete list of restricted object types, see Supported ABAP Object Types.

Transporting Business Configuration

If you want to transport business configuration (BC) content across your ABAP systems, you have two options. Choose based on how your projects are coupled.

Option A: Dedicated Business Configuration Software Component

Create a software component of type Business Configuration and transport all your configuration content through it. Applications can then automatically select or create a customizing transport request for this software component.

> ⚠ Caution
>
> You can import only *one* software component of the type Business Configuration per ABAP system.

Option B: Use the same Software Components as Development Objects

Transport configuration in the same software components you use for development objects, typically it is the same software component for tables and table content. This is recommended when you run multiple, well-seperated projects on the same landscape with decoupled schedules.

Criteria	Option A: Dedicated BC-Software Component	Option B: Software Component per Development
seperation of concerns	full (configuration isolated)	mixed (configuration is inherited with the code)
number of BC- Software Components	max. one per system	unlimited (one per development software component)
Best for	single configuration stream	parallel projects with independent schedules

For more information, see Transport of Business Configuration.

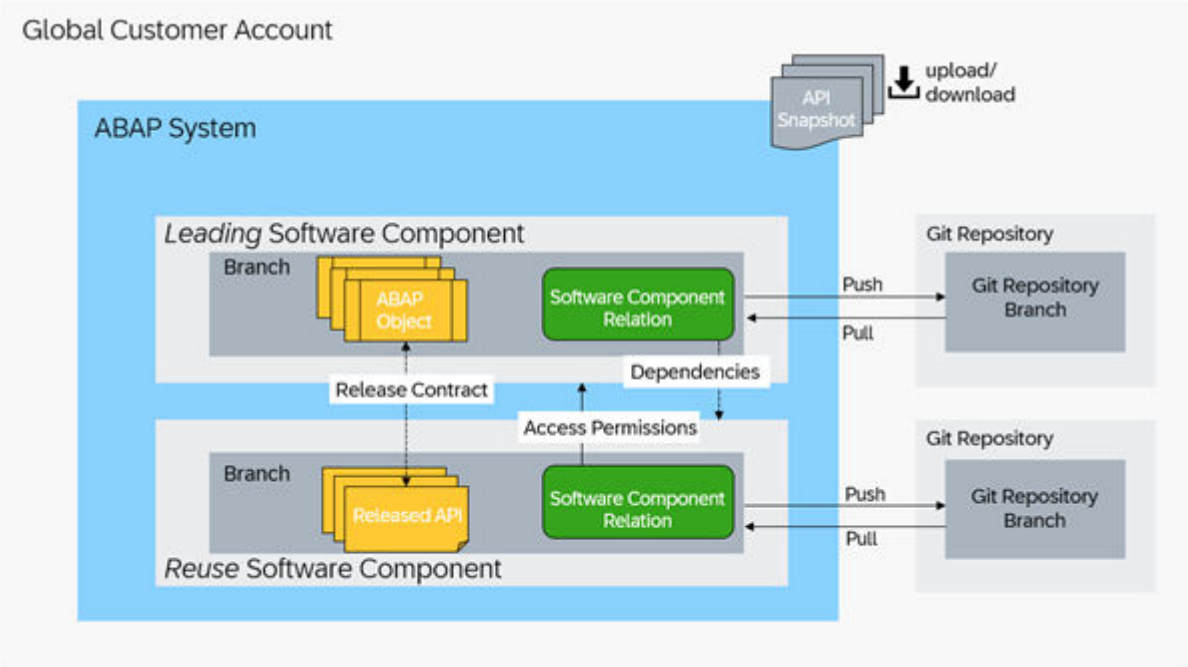
System Landscape and Hosting

A single system landscape (development, test, production) can host multiple software components. You don't need a sperate landscape per software component.

All software components in your landscape are linked to the same global account, regardless of region or infrastructure provider. Ensure that all systems involved are created within the same global account. Software components are only visible and usable in the globale account they were created in.

> → Remember
>
> SAP-managed software components created via the Manage Software Components Fiori app are *always stored on AWS Frankfurt (EU Access)*. This is independent of the hyperscaler you chose for your ABAP environment instance and independent of the data center where the instance was created. Take this into account for data residency and compliance assessments.

4.2.9.2.2 Software Component Dependencies



The leading software component is the one that is dependent on objects in the reuse software component. To make sure that the software components are imported in the correct order, the reuse software component should be the first, whereas a leading software component would be the last in order.

Software component relations are one option to grant access permissions and to define dependencies for a whole software component.

Released APIs and Snapshots

Development objects come with an API state that can be set for a certain release contract. A released API will remain stable, with different consistency and stability criteria depending on the release contract. You will get a warning in ABAP development tools for Eclipse whenever an attempt is made to change an already released API.

Additionally, for some release contracts (e.g. C1) a visibility is defined which corresponds to a use in developer extensibility in an ABAP language version like ABAP for Cloud Development. For more information, see Released APIs.

In contrast to software component relations, release contracts are mostly used to guarantee the stability and consistency of an API. See Providing Released APIs.

To detect and prevent incompatible changes in released APIs, corresponding API facades need to be extracted in a generated API snapshot. Refer to Checking the Compatibility of released APIs.

The ATC check `API_COMPATIBILITY` compares the current state of released APIs against this snapshot to find compatibility issues. An SAP default API snapshot will be created automatically whenever an API is released, but compatibility checks should be done against a check-relevant snapshot of the previous

deliverable state of a software component. For the initial release of a software component no previous state exists, and therefore comparison against a snapshot is not possible.

This means that API snapshots should be used as “image” of a finalized delivery: Instead of frequently regenerating already existing snapshots, generate a snapshot once at the end of a release cycle and try to keep existing released APIs stable across the whole release cycle. The assumption here is that the APIs already released in a software component usually do not change over the course of a release. You may use the Notes feature of the Manage API Snapshots app to document any changes to the snapshots. The system creates API snapshots locally on each system. You can download a snapshot to use it on another system. After you enable an uploaded snapshot for compatibility checks, set any existing SAP default snapshot of the affected software component to 'not check-relevant. For more information, see Manage API Snapshots.

> ### Note
>
> Usually, positive incompatible change findings should be removed. Use ATC exemptions to mitigate false-positive `API_COMPATIBILITY` check findings. In case you cannot identify an actual incompatible change in the corresponding development object, please contact the SAP support.
>
> You may also create ATC exemptions to temporarily mitigate findings if they cannot be resolved in time for an urgent correction. Make sure to remove exemptions after finding a resolution. See Working with ATC Exemptions. Although technically possible, regenerating or deleting API Snapshots should not be used as a mitigation for ATC findings. Instead, create an ATC exemption in this case.

The additional visibility in other software components that comes with certain release contracts is not targeted to a specific software component but rather bound to a certain ABAP language version.

> ### Note
>
> Do not create release contracts if you cannot guarantee the stability of the released APIs. For such cases you can use software component relations to access custom development objects across Cloud software components. For more information, see Software Component Relations. If on the other hand you release APIs in your software components, make sure to create API snapshots accordingly in your test and development systems.

Software Component Relations

With Software Component Relations you can define access permissions and specify the software components which are permitted to use content of a reuse software component. At the same time dependencies can be defined to specify whose content is used by a software component.

In contrast to using release contracts for cloud development, software component relations permit usage of APIs included in a software component from within a specific other component. No stability or consistency commitment is defined for the included APIs since only the software component relation with its access permissions is included in API snapshots. The API facades themselves are not included. Both the defined access permissions and dependencies target specific software components.

Undeclared dependencies can be detected using the `SCR_CONSISTENCY ATC` check and the incompatible removal of access permissions using `API_COMPATIBILITY`. For more information, see Software Component Relations .

> **Note**
>
> If your solution consists of (multiple) leading and reuse software components, we strongly recommend to explicitly declare these dependencies. The leading software component is the one that does not have parent components, as opposed to a reuse software component, which has other software components depending on it.
>
> When the software components are delivered as part of an add-on product, declaring software component dependencies is mandatory. The `SCR_CONSISTENCY` check is used implicitly during the abapEnvironmentAssemblePackages step in the Build stage of the ABAP environment pipeline.

Software component relations can be created in cases where all software components are originating from the same development system. Additionally, access permissions can be granted to software components with a namespace that is registered as producer namespace in the development system. In case of other distributed development scenarios, you can use release contracts instead of access permissions.

4.2.9.3 ABAP Environment Specifics

gCTS and the SAP Fiori app Manage Software Components provide a simplified git UI adapted to the possibilities of ABAP. This means:

- There is a remote Git repository for each software component managed by SAP in the cloud. Using the SAP Fiori app Manage Software Components, a software component can be created from any ABAP system in the same global account. Alongside with the software component creation, the corresponding remote repository is created. Afterwards, the software component needs to be cloned and pulled into the ABAP system. With this step, a corresponding structure package is created in the ABAP system that acts as a local repository so that you can start developing (software component = repository in gCTS).
- The creation of a Git repository branch is also performed in the Git repository. It does not matter in which ABAP system you create the branch. However, you have to check it out in each ABAP system where you want to use it, be it for development, testing or productive use.
- In an ABAP system
  - there is only one branch of a software component at a time
  - all developers work on the same object versions
  - an object can be edited by one developer at once
- Pulling a software component does not simply copy object files, but, depending on the object type, also activates the objects, for example a data element, or even generates additional invisible objects in Eclipse, for example authorization artifacts for business catalogs used in business roles.
- You have no merge functionality. If you pull the current branch while changes have been added from another ABAP system, all the changes in the ABAP system the branch is pulled into are discarded. Consequently, you should not work on the same branch in two different ABAP systems.
- Before being able to pull a software component again or before checking out a different branch, you have to release all open transport requests. There is no way of saving them without releasing, like a git stash.
- Releasing a transport request combines the git actions of committing and pushing. The Transport Request Id and description are displayed in the commit message.
- A new branch is created remotely in the Git repository. This means that if the new branch shall be derived from the one currently checked out, changes in the ABAP system that are not yet committed and pushed by releasing the transport that contains these changes are not part of the new branch. The `Last Commit`

`Message` text consists of the Transport Request ID and description, and helps you to ensure that the expected changes are part of the new branch. If you don't want to hold back any new changes, you must release all open transports before creating a new branch.

- If you need to discard changes, you have to do this manually. This can be done with the help of the *History* view in Eclipse, which offers a compare editor to revert to the last transported version. Afterwards, you have to remove all the discarded objects from their modifiable transport request in the *Transport Organizer* view.
- Branches cannot be deleted or set to *obsolete*, therefore consumers have to be informed about a branch that they shall not use, for example, if you skipped a release branch that could not be finished in time.
- Basic change logging using the "traditional" ABAP server capabilities is available in the development ABAP system: e.g. the *Revision History* for ABAP objects and the *Responsible* and *Last Changed By* contacts. However, if you de-provision the development ABAP system, this information is lost. The Git repository still stores the version (see Information for Audit [page 921]) and the person who released the transport request(s) but the information on the *Last Changed By* contact per object is only available in the development ABAP system.
- You cannot work offline because you need access to the ABAP system.

4.2.9.4 Working with abapGit

abapGit is an open source developed Git client for the ABAP server to import and export ABAP objects between ABAP systems.

With abapGit, you can export your ABAP objects from any system to another, typically from on-premise to cloud or from one cloud system to another. See Use abapGit to Transfer Your On-Premise ABAP Source Code to the Cloud and Push Your ABAP Source Code from SAP BTP ABAP Environment to a GitHub Repository Using abapGit.

> **Note**
>
> While the ABAP environment comes with a preinstalled cloud version of abapGit, you can only use abapGit in an on-premise system if you have installed the official community version of abapGit. However, the official community version is not supported by SAP.

With the official SAP distribution of abapGit, you can use ABAP Development Tools (ADT) in combination with the abapGit feature for ADT, which allows easier access to the import/export functionality provided by abapGit. For more information on how to install and work with ABAP Development Tools, see Installing ABAP Development Tools and abapGit Repositories ADT Plug-In .

Use Cases

Side-by-Side-Development

abapGit can assist you in side-by-side development by enabling the transfer of source code between on-premise systems and SAP Business AI Platform, ABAP environment.

Account Transfer

abapGit can support you in transferring your development code assets from one Steampunk system to another, particularly across different BTP global accounts.

Partner Code Transfer

As a partner, you can utilize abapGit to export your development code assets from your development landscape and import them into your customers' development system.

Demo Samples and Open Source Offerings

You can use abapGit to distribute code as open source, e.g., Code Pal for ABAP - Cloud Edition or demo samples, such as the ABAP Flight Reference Scenario for the ABAP RESTful Application Programming Model.

Restrictions and Errors

There are certain restrictions to the export and import functionality when using abapGit in the ABAP environment. This is due to the cloud-optimized set of supported ABAP object types that change with each release as new ABAP object types are added continuously. See Released ABAP Object Types [page 886]. Depending on your application, you may face the following issues when using abapGit in cloud systems:

- Export errors when pushing unsupported objects
- Import errors when pulling unsupported objects
- Activation errors when activating imported objects

Export Errors

If you try to export objects that are not supported, you receive a warning during the push operation, and the unreleased object is skipped.

The only work-around for this is to manually copy the missing ABAP objects to the target system. First, you have to create these objects using the same name in the target system. Afterwards, you either have to manually copy the code or reconfigure the objects in the target system.

Some objects are deliberately not exported because they are considered as compiled or generated so that they can be regenerated when activating their originating objects in the target system. For example, certain service definitions (SRVD) are generated from service consumption models (SRVC) so that they are not exported to git.

Import Errors

When importing objects with abapGit for ADT, you may get error messages in the import log. Possible reasons can be:

- You are importing object types (for example from an on-premise system) that are not supported in the ABAP environment and the programming model of the ABAP environment. You may have to change your application in the cloud environment to work without these objects.
- You are importing objects with missing dependencies. These dependent objects can't be exported because they are not on the list of Released ABAP Object Types. You must copy/recreate these missing dependent objects and restart the import.
- You are importing objects using ABAP namespaces. Make sure these namespaces have been imported into the target system before importing your application with abapGit.

- You are importing a software component relations object. Make sure that a software component with the same name has been created in the target system already. Please note that this object can be imported into the top-level structure package of the corresponding software component only.

> ⓘ Note
>
> Local changes to ABAP objects that have not been saved are overwritten by the import. Open transport requests for any of the imported ABAP objects must be released before they can be changed. Otherwise, they get locked.

Activation Errors

All objects imported with abapGit are imported in an inactive state, except the ones that do not offer an inactive state, to ensure that their consistency can be properly checked during activation. Hence, you have to activate all imported objects after importing them so that ABAP activation can verify that only released APIs have been used.

During the activation process, some objects may depend on other objects, which need to be activated beforehand. However, not all object types consider the objects they depend on during mass activation. For example:

- Data dictionary (DDic) and core data service (CDS) objects do not always resolve their dependencies in the first mass activation attempt so that you have to activate them in multiple cycles.
- Service bindings need to be activated after service definitions.

If activation issues occur, repeatedly try to activate objects in the right sequence. If that does not work, try reimporting and reactivating objects multiple times until all objects of your application are active.

> → Tip
>
> If you face any issues with the official abapGit distribution of SAP, create an incident using component `BC-CP-ABA`.
>
> For issues related to the community open-source distribution of abapGit, create an issue at https://github.com/abapGit/abapGit/issues .

> ⓘ Note
>
> It is possible to define the external systems, the ABAP system is allowed to communicate with, by maintaining a list of trusted certificate authorities. Please use the Maintain Certificate Trust List app to do so. This app can be used by administrators, users with the SAP_BR_ADMINISTRATOR role. For further information, please go to Maintain Certificate Trust List [page 3186]

Working with UI Artifacts

As described in Restrictions and Errors, the originating objects of the UI/SAP Fiori artifacts (SMIM, UIAD, WAPA) have their own lifecycle in a separate Git repository. The deployment to the target ABAP system will be done via SAP Business Application Studio / Visual Studio Code using SAP Fiori Tools.

For the case that one of the above-described use cases applies and you must transfer your application, remember that the UI artifacts have their own Git repository. This is important because abapGit does not take the UI objects into account.

Nevertheless, you have two different Git repositories. The Git repository with ABAP artefacts and the Git repository with the UI artefacts.

Important to Know

Understanding the unique usage of the UI Git repository compared to the abapGit repository is crucial. The main difference lies in the fact that the user interface (UI) maintains its own lifecycle. This means any modifications or updates to the UI must be made within the Business Application Studio or Visual Studio Code and then pushed to the dedicated UI repository.

The abapGit repository is usually only used once to (additionally) save and share your code. The lifecycle happens in your ABAP system.

Procedure

Depending on your needs, you might have to duplicate the repositories, create branches, or fork branches. For example, to transfer an application from a partner development system to a customer development system, you can use the export and import functionality of your Git provider. Alternatively, you can utilize Git functionality like branching or forking to "duplicate" the repository.

For additional information on how to duplicate the Git repositories, it may be helpful to consult the documentation provided by your Git provider.

Import Steps

- You have read access to both Git repositories (abapGit and UI)
- Link and pull the abapGit repository into your ABAP system.
- Click onto the button *Activate All* to activate all objects.

> **Note**
>
> The activation might fail due to missing dependencies, e.g., missing UI artifacts.

- Clone your UI repositories to Business Application StudioAS/Visual Studio and deploy the UI to your ABAP system.
- Pull the abapGit repository again because some abapGit objects rely on the UI objects.
- Click onto the button *Activate All* to activate all objects.

> **Note**
>
> Please be aware that it might be necessary to repeat the last two steps.

Related Information

Released ABAP Object Types [page 886]

4.2.9.4.1 Install and Set Up abapGit

Learn how to install and set up abapGit.

- You have signed up for a Git account of your choice, for example GitHub.
- You have downloaded and installed the front-end components of ABAP Development Tools (ADT).

> **⚠ Restriction**
>
> Please be aware that abapGit is an open-source project maintained by the open source community and is not owned by SAP. Therefore, SAP does not provide support for the open source abapGit version (ZABAPGIT). Consequently, we ask you to refrain from creating incidents related to the use of the ZABAPGIT report. However, for the SAP Business AI Platform, ABAP environment, a separate SAP-owned version (fork) of abapGit has been established. SAP does provide standard support for this version.

Part 1: Create a Git repository

1. Log on to your github.com account.
2. Create an abapGit repository by clicking on *New repository*.
3. Enter a repository name, tick the *Initialize this repository with a README* checkbox, and select *Create repository*.

Part 2: Install zabapGit for the on-premise use case

1. Copy the content of the latest build from program `zabapgit` to your clipboard. You can find the content in the abapGit repository https://github.com/abapGit/abapGit .

   > **⚠ Caution**
   >
   > Check with your system administrator before installing `zabapgit`.

2. Log on to an on-premise system of your choice, create a new program, and paste the content from your clipboard.

   > **ⓘ Note**
   >
   > Select `EN` as your logon language.

3. Activate and execute the program.

   abapGit is installed and launched.

Part 3: Install an ABAP Development Tools for Eclipse Plugin for abapGit

1. Log on to ABAP development tools for Eclipse.
2. Navigate to *Help* > *Install New Software...* .
3. To install the abapGit repositories ABAP development tools for Eclipse plug-in, add the following URL: **`https://eclipse.abapgit.org/updatesite/`** and provide a name for the repository.
4. To display all the available features, press enter, and select *abapGit for ABAP development tools for Eclipse*.
5. Select *Next* to finish the installation.

Related Information

Create Content in an On-Premise System and Push it to abapGit Repository

4.2.9.4.2 Create Content in an On-Premise System and Push it to abapGit Repository

Prerequisites

You have installed and set up abapGit. See Install and Set Up abapGit [page 874].

> **⚠ Restriction**
>
> Please be aware that abapGit is an open-source project owned by the community. Therefore, we do not provide support for abapGIT. We only support the Git integration in the ABAP environment.

Procedure

1. After installing and launching abapGit, select *Clone or download*, and copy the URL of your repository.
2. Call up transaction `ZABAPGIT`, and select *+ Online*.
3. Paste the URL of your repository.
4. Select *Create package*.

   > **ⓘ Note**
   >
   > If you have already created a software component with gCTS, you can skip steps 4 to 6 and use this software component in step 7.

5. Add a package name and short description, and select *Continue*.
6. Confirm with *OK*.

   The cloned abapGit repository is displayed in abapGit.
7. Log on to ABAP Development Tools in Eclipse, and navigate to your newly created package.
8. Add ABAP development objects to your package. See Released ABAP Object Types [page 886].
9. Navigate back to the abapGit UI, and select *Refresh* to display the development objects that you have created in Eclipse.
10. Choose *Stage*.
11. Select single objects or choose *Add all and commit*.

> **Note**
>
> You can also select structure packages to be exported.

12. Enter a *committer name*, *committer e-mail*, and *comment*.
13. Select *Commit*.
14. In the Login popup, enter your abapGit repository server credentials, and select *Execute*.

    The pushed ABAP objects are displayed in your abapGit repository.

Next Steps

Import Content from abapGit Repository into the ABAP Environment [page 876]

4.2.9.4.3 Import Content from abapGit Repository into the ABAP Environment

Learn how to import content from your abapGit repository into your ABAP environment, and transfer it across multiple instances.

Prerequisites

- You have installed the abapGit repositories ADT plug-in. See https://eclipse.abapgit.org/updatesite/ .
- You have created content in your on-premise system and pushed it to your abapGit repository. See Create Content in an On-Premise System and Push it to abapGit Repository [page 875].

> **Note**
>
> The repository has to be accessible from the Internet. Cloud Connector is not supported.

- You have access to an ABAP cloud system. See Creating an ABAP System [page 198].
- You have assigned a developer user in the ABAP environment to the developer role. See Assigning the ABAP Developer User to the ABAP Developer Role [page 217].

> **Restriction**
>
> Please note that abapGit is an open-source project owned by the community. Therefore, we do not provide support for abapGit. We only support the abapGit integration in the ABAP environment.

Procedure

1. Log on to ABAP Development Tools in Eclipse.
2. In the *Project Explorer*, select your cloud project system, and navigate to *Window* > *Show View* > *Other..* to open the abapGit repositories view.
3. Search for *ABAP*, choose *abapGit Repositories*, and select *Open*.
4. In the *abapGit repositories view*, select the clone button (green + icon).
5. Enter your abapGit repository URL, and select *Next*.
6. Select a *Branch* and *Package*, where you want your abapGit repository to be cloned, and confirm with *Next*.

   > **Note**
   >
   > If there are no packages, you have to create a structure package and add a development package.

7. Select the default transport request, and choose *Finish*.
   The imported objects are displayed in your package. See Released ABAP Object Types [page 886].

   > **Note**
   >
   > The number of imported objects can differ from the number of exported objects because only released ABAP object types are considered during the import.

4.2.9.4.4 Folder Logic in abapGit

Folder Logic

AbapGit in Steampunk offers two folder logics, either “Prefix” or “Full”. The folder logic has to be selected when linking a repository in ABAP Developer Tools in the popup. Afterward, the information can also be found in the abapGit Repositories ADT plugin. The folder logic information defines the way how subpackages of the root package that needs to be linked will be converted to folders in the linked repository. Therefore, this setting affects only the folder structure in the remote repository and has no implication to the object structure in the ABAP system.

Prefix

The folder logic PREFIX allows the installation of a repository into a different parent package (in different systems). This can even be a local package (ZLOCAL), in which case no transport request is required.
A package name must contain its parent package name as a prefix. This means that the names of the subpackages will start with the name of the parent package, for example:

- TESTMYTEST01 (parent)
- TESTMYTEST01_RUN_01 (child1)
- TESTMYTEST01_RUN_02 (child2)

> **Note**
>
> The prefix folder logic will not work correctly if you do not follow the convention.

Other examples, including valid package prefix:

- ZFOO
- ZFOO_BAR
- ZFOO_BAR_QUX

This will produce the following folder structure:

/src

/src/bar

/src/bar/qux

Invalid package prefix:

- ZFOO
- ZBAR

Full

The folder logic FULL forces the installation of a repository into packages with exactly the same name. Any package name is accepted. Folders will be named like packages. Any package name is accepted

- ZBASE
- ZSOMETHING
- ZHELLO

This will produce the following folder structure:

/src

/src/zsomething

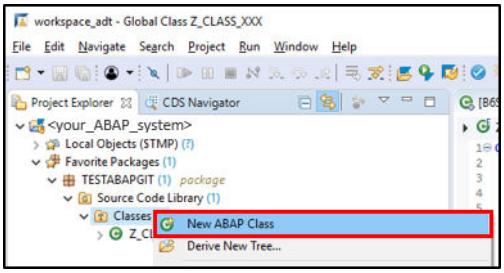
/src/zsomething/zhello

In general, the folder logic setting is stored in the *.abapgit.xml file* in the remote repository, too. It is highly recommended to take over this information during the linkage process, in case the file already exists. Otherwise, a mixture of different folder logic information on the local and remote side would lead to inconsistencies.

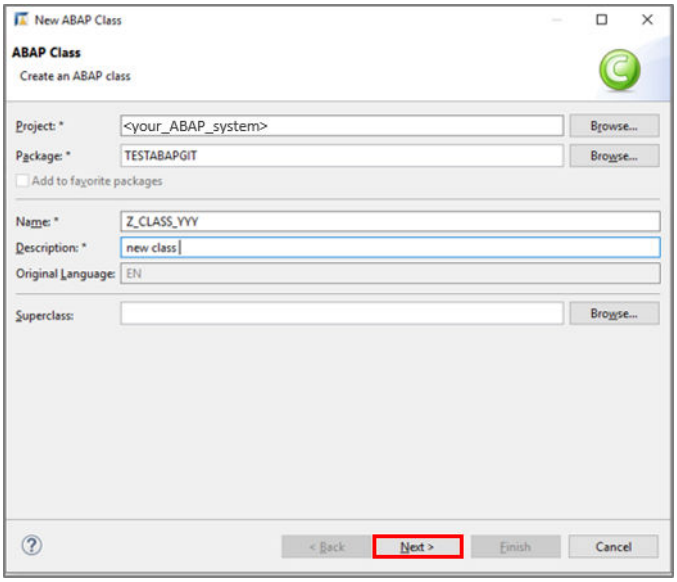
4.2.9.4.5 Transfer your ABAP Source Code via abapGit

Use abapGit to transfer your ABAP source code from an ABAP environment instance back to your repository.

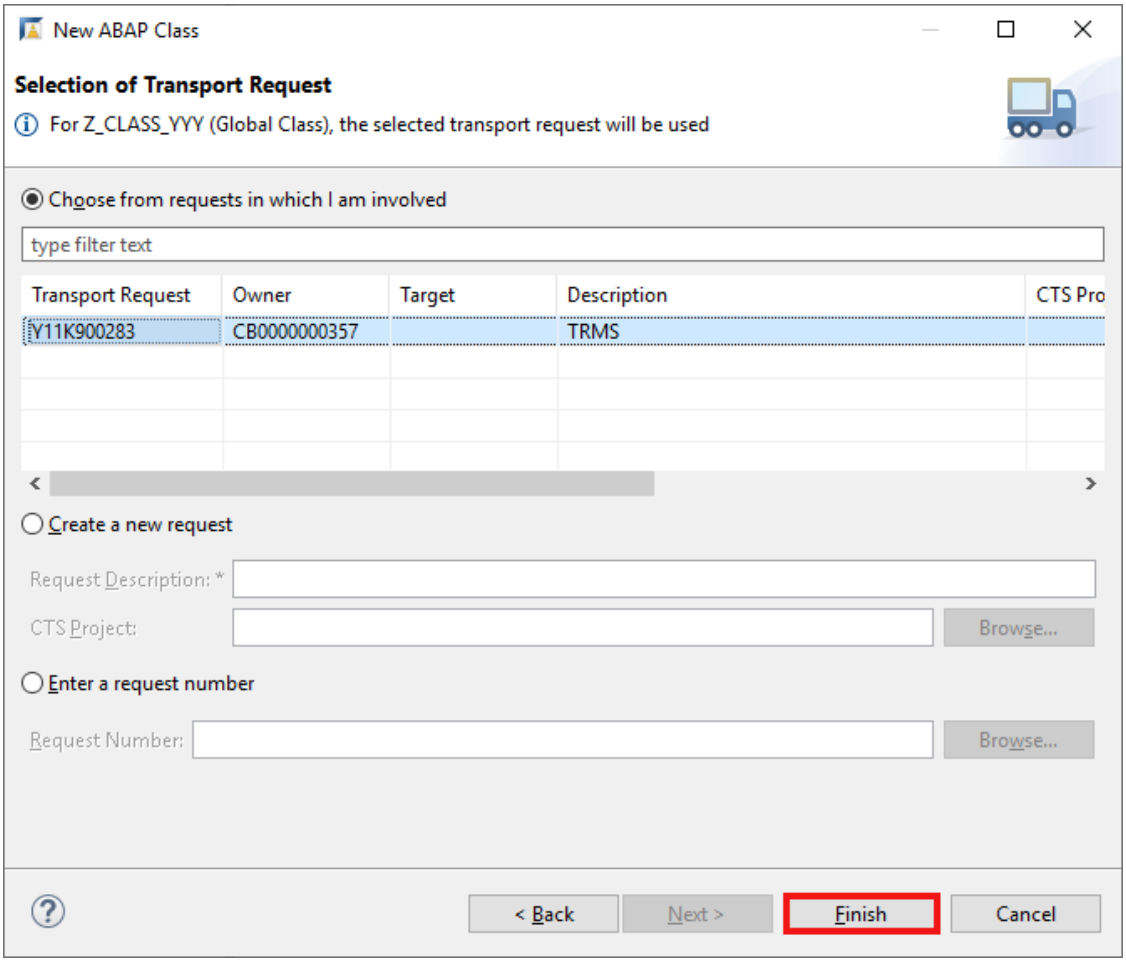
1. Create or adapt your source code in the ABAP environment instance:
    1. Open **ABAP Development Tools** and logon to your ABAP system.
    2. Add an ABAP development object, e.g. an ABAP class, to your already existing *TESTABAPGIT* package. To do this, click on *Classes* and select *New ABAP Class*.



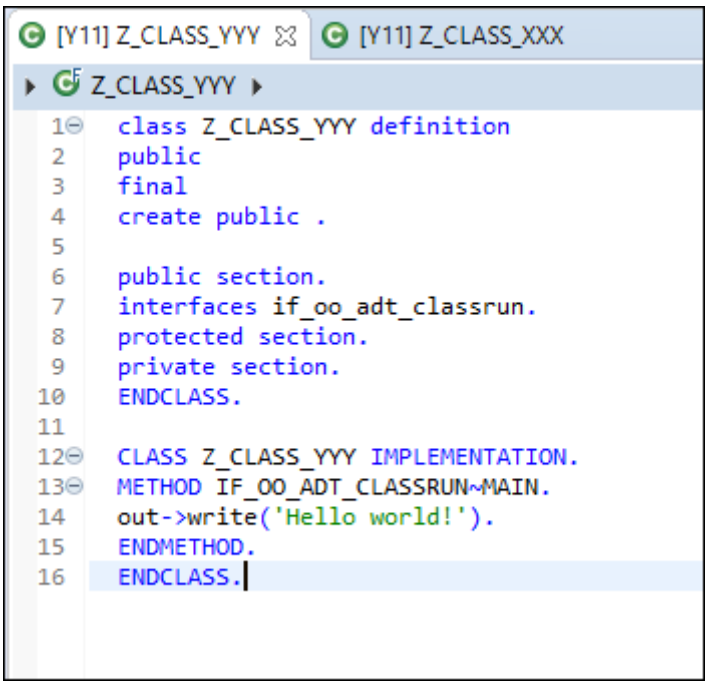
3. Enter a name and description for your new ABAP class and click *Next*.



4. Click *Finish*.

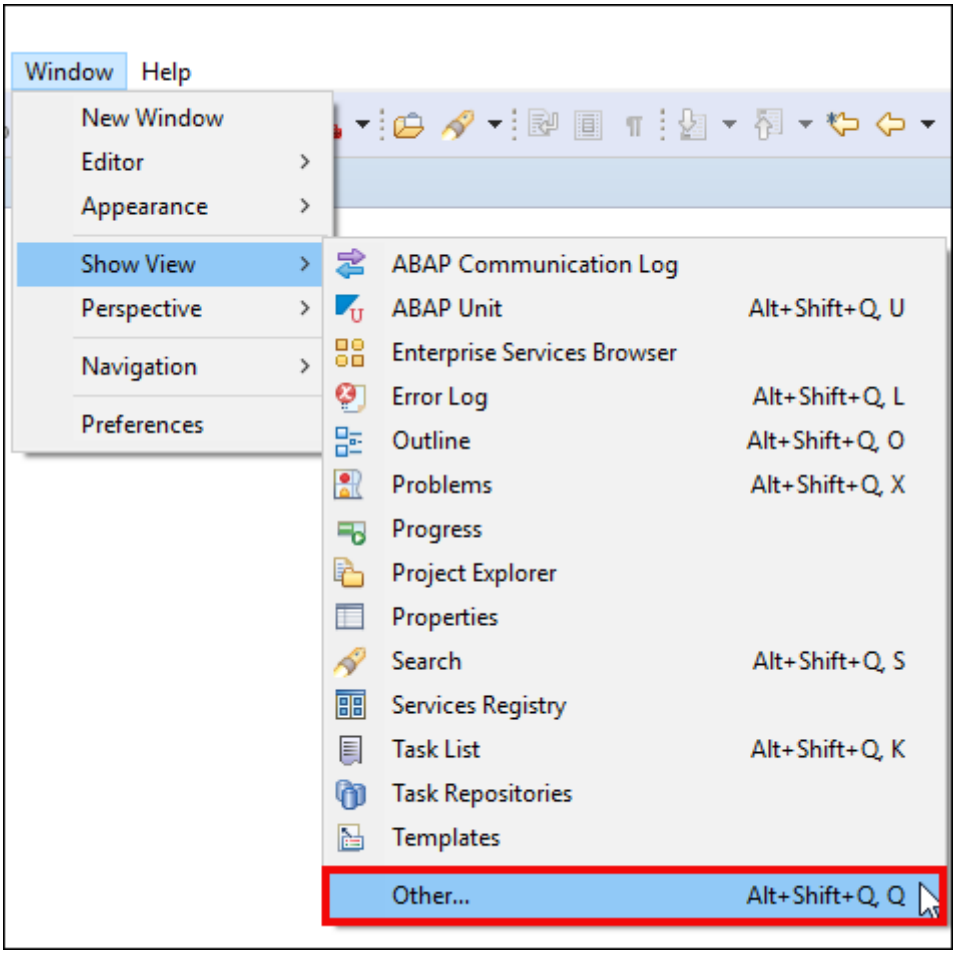


5. Now you can implement your newly created class.

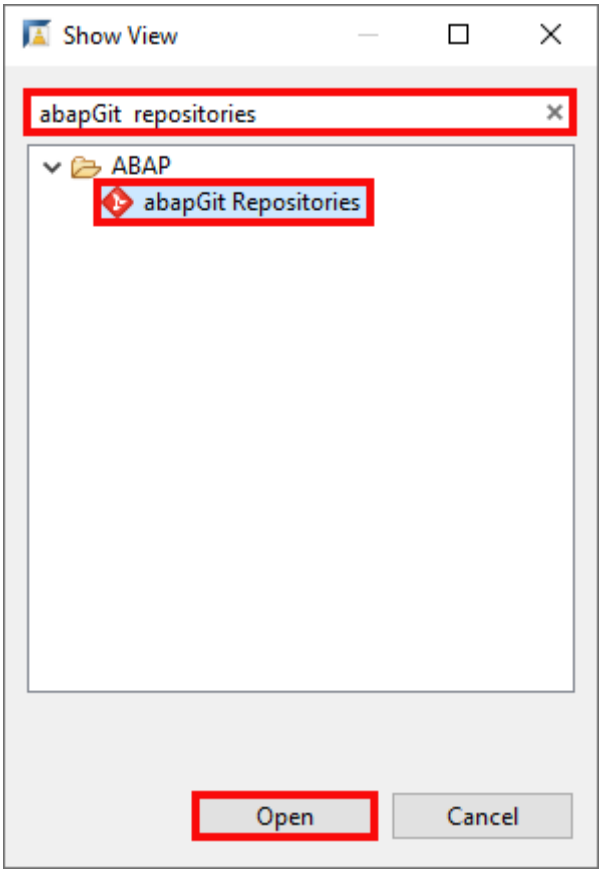


6. Save and activate.

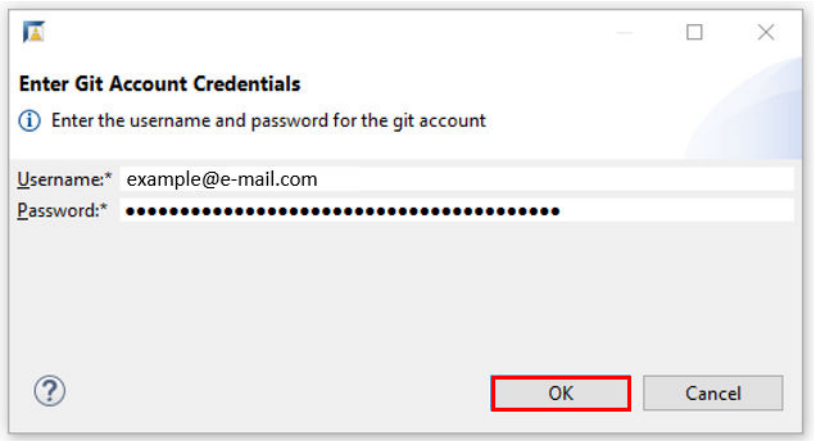
2. Open abapGit repositories:
    1. Select your **ABAP system** in the Project Explorer and select *Windows* > *Show View* > *Other...* to open the *abapGit* repositories.



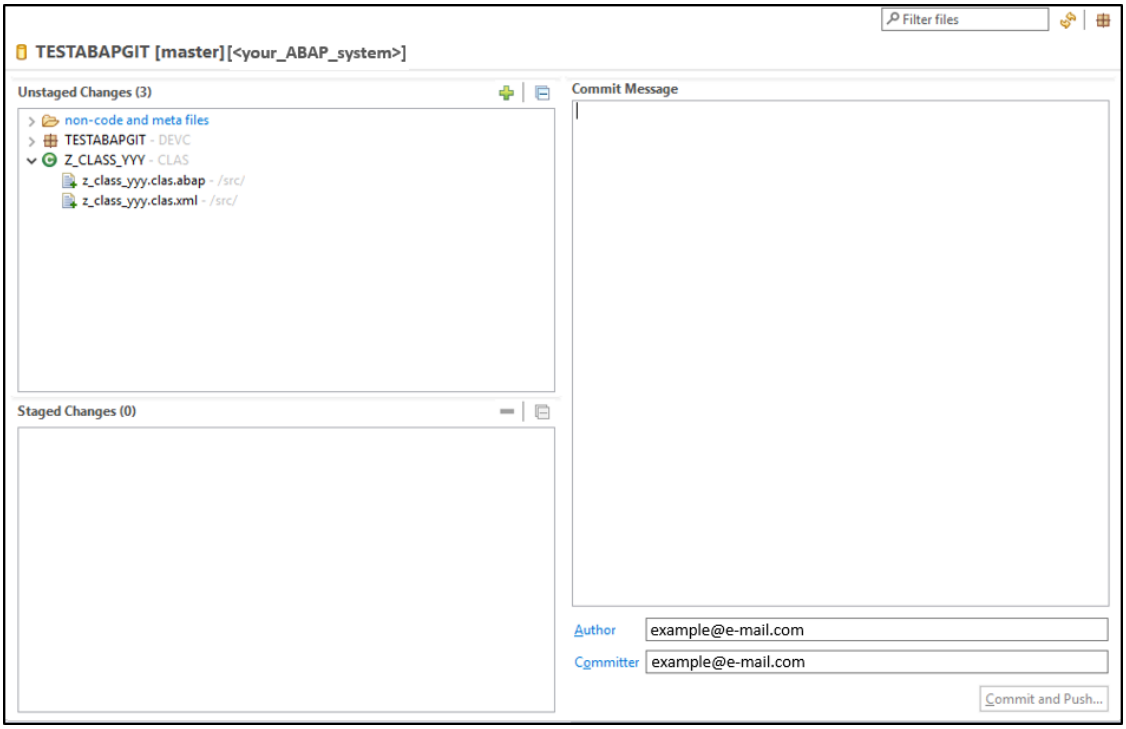
    2. Search for *abapGit repositories*, select it and click *Open*.



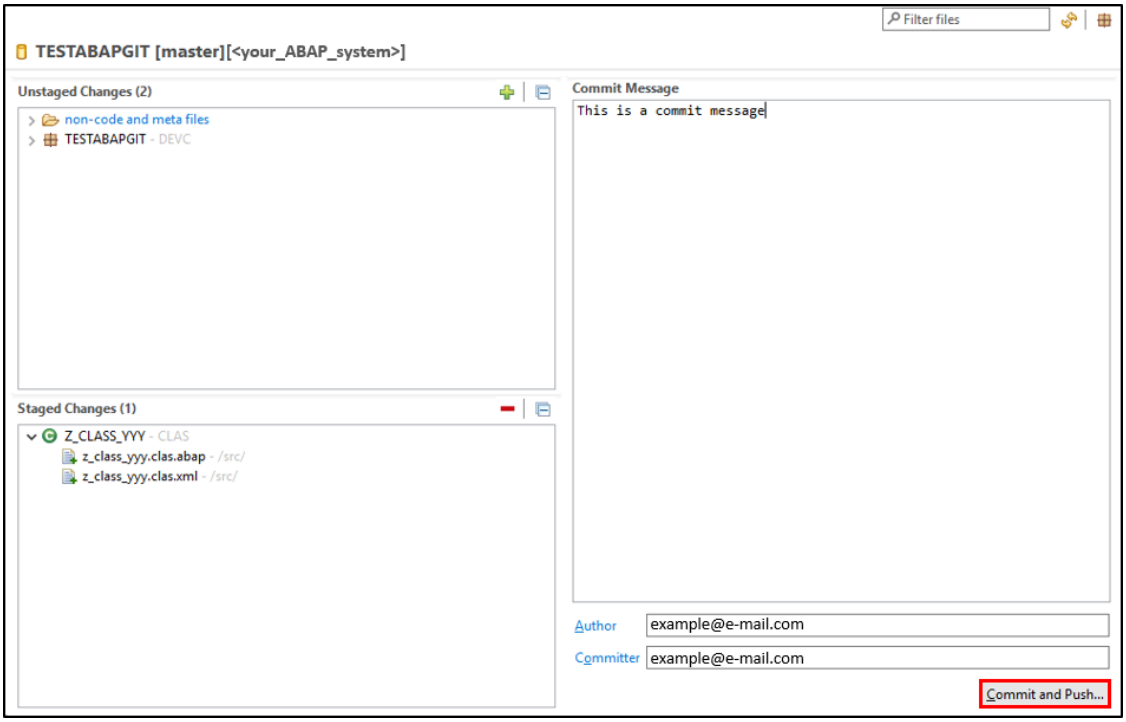
3. Stage and commit your ABAP development objects.
    1. Right-click your *TESTABAPGIT* package and click *Stage and Push*.
    2. Enter your repository credentials in the popup and click *OK*.



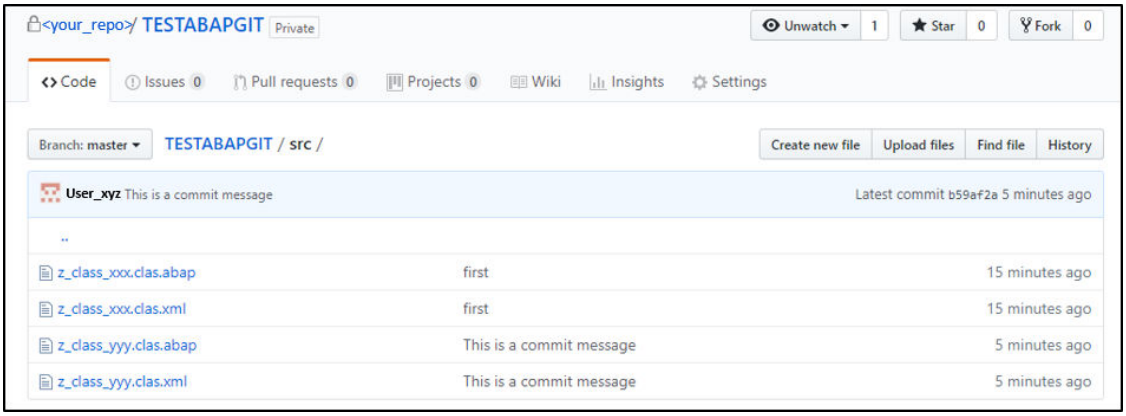
    3. In the staging view you will see a list of changed, created or deleted ABAP development objects which can be transferred to your repository. The icon of the respective file indicates whether a file was created, modified, or deleted.



4. ABAP development objects which should become staged can be selected by drag and drop from the *unstaged* box to the *staged* box or by clicking on the + button. Enter your commit message and click **Commit and Push**.



5. You'll see a notification telling you that your push has started. Click *OK*.
6. The staged objects have now been transferred to your repository.



4.2.9.4.6 Object Deletions with abapGit

Initially, abapGit as a Git client has been integrated into SAP Business AI Platform, ABAP environment to enable customers to facilitate the import of code assets from customers' on-premise systems to SAP Business AI Platform, ABAP environment. Over time, additional use cases came up, so that the focus is no longer on a one-time transfer but also onrepeated export and import of development objects across different landscapes, including handling of deletions.

In general, there are two basic types of deletions.

Deletion Type	Git Action	Leading Repository	Determination logic
Local	Pull	Remote	If an object exists locally but not remotely, it will be marked for deletion during the pull process.
Remote	Stage & Push	Local	If an object exists remotely but not locally, it will be marked for deletion during the staging and push process.

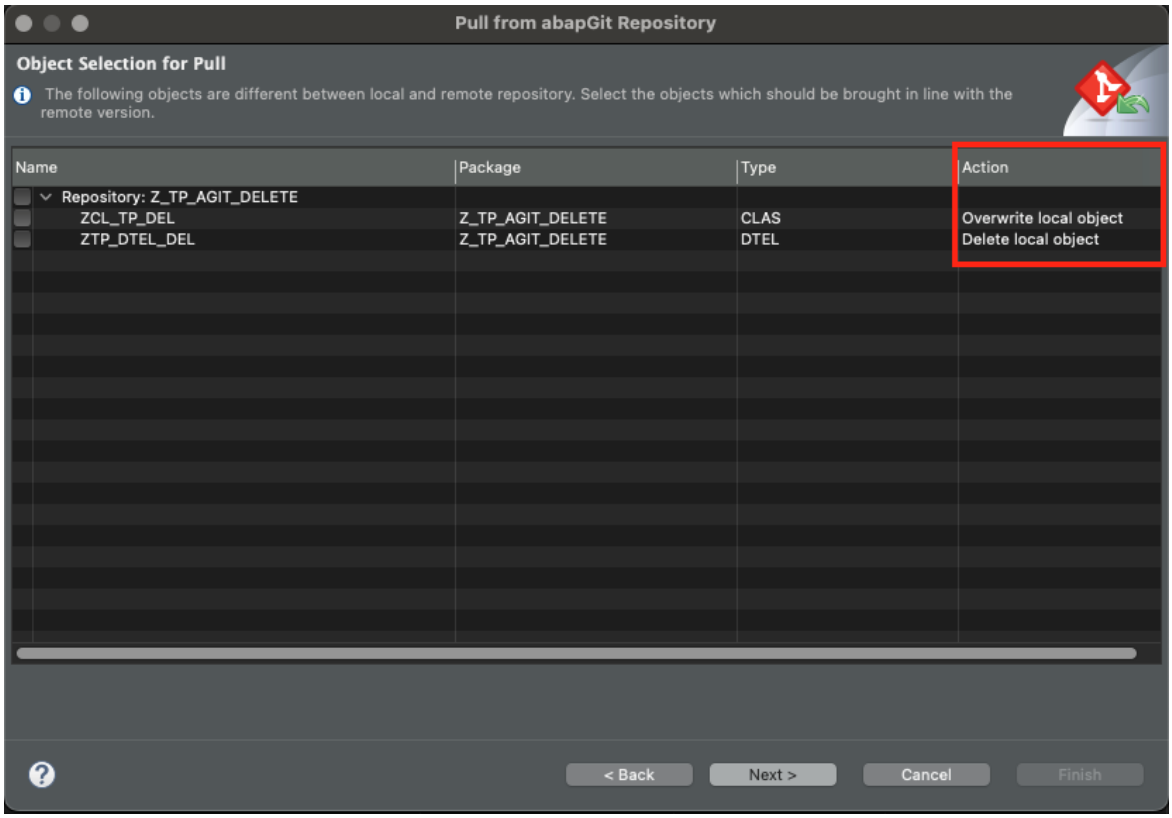
Local Deletion

Prerequisites

Objects from the remote repository have been pulled into your linked ABAP package.

1. Start by deleting the file(s) of your objects in your abapGit remote repository.
2. In ABAP Development Tools for Eclipse, you initiate a pull run.
   After that, the system performs a delta calculation to identify the differences between the package and the remote repository.
3. The object previously deleted in your remote repository will now be shown as the action *delete local object*.
   Here, you can select the object to be deleted.

Afterwards, click on *Next* and then *Finish*.



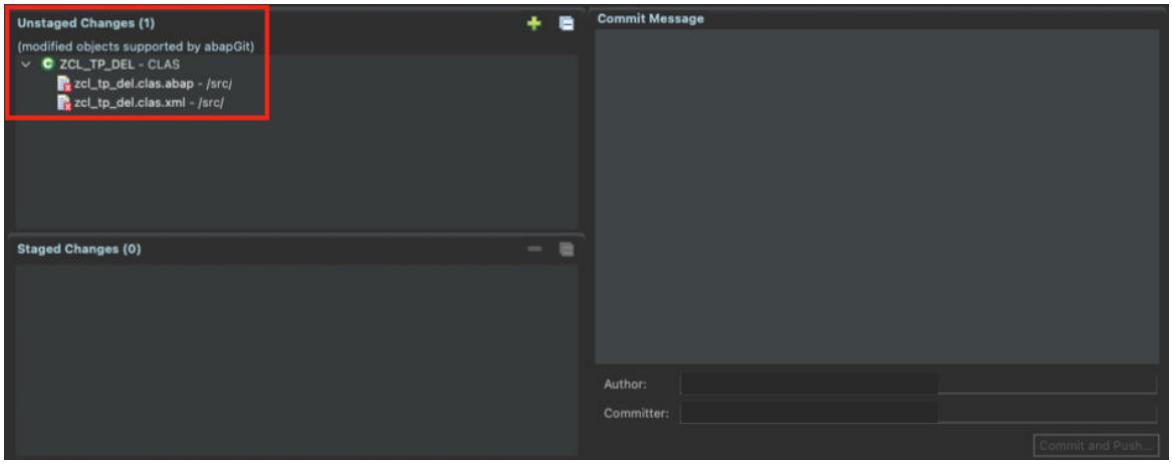
4. After the pull run, you can check the *ABAP Object Log* for the successful deletion.
5. Finally, reload the *Project Explorer* to see that the object has dissapeared in the overview as well.

Remote Deletion

Prerequisites

Objects from the remote repository have been pulled into your linked ABAP package.

1. Start by deleting your object locally in ABAP Development Tools for Eclipse first.
   In the *Project Explorer*, select the according object and delete it. This opens up a window called *Select Objects*. Here, you select the object to be deleted and then *Finish*.
2. Next, initiate a stage and pull of the emerging changes.
   To do this, right-click your abapGit package in ABAP Development Tools for Eclipse and select *Stage and Push*. The following picture shows an example overview of unstaged and staged changes.



3. In the changes overview, the previously deleted object in your package will now be shown with a deleted icon.
   You can drag and drop the object from *Unstaged Changes* to *Staged Changes*.
   Afterwards, select *Commit & Push*.
   In the tab *abapGit Repositories*, you can now see that your linked repository will show the Status *Pushed Successfully*.

4.2.9.4.7 Released ABAP Object Types

The following table contains all the released ABAP object types that can be imported into your ABAP environment tenant.

> **Note**
>
> If you try to import or export objects that are not supported, you receive a warning during the transport operation. The objects that are not released are skipped and not transported. To avoid this, you have to manually copy the ABAP objects to the target system. Note that you have to create these objects using the same name in the target system, unless they already exist. Afterwards, you either have to manually copy the code or reconfigure the objects identically in the target system.

APIS	API Release State of Objects
APLO	Application Log Object
AUTH	Authorization Check Fields
BDEF	Behavior Definition
BGQC	Background Processing Context
CDBO	Customer Data Browser Object
CFDF	Custom Field
CHDO	Change Document
CHKC	ATC Check Category

CHKO	ATC Check Object
CHKV	ATC Check Variant
CLAS	Class
COTA	Connection Target Transport Object
DCLS	ABAP Data Control Language Sources
DDLS	Data Definition Language Source
DDLX	Core Data Services Metadata Extensions
DEVC	Package
DESD	Logical External Schema
DOMA	Domain
DRAS	CDS Aspects
DRTY	Dictionary: CDS Type Definitions
DSFD	CDS Scalar Function Definition
DSFI	CDS Scalar Function Implementation Reference
DTEB	Dictionary Tuning: Entities Buffer
DTIX	Entity Index
DTEL	Data Element
EEEC	Event Consumption Modell
ENHO	Enhancement Implementation Object
ENHS	Enhancement Spot Object
EVTB	Event Binding
ENQU	Lock Object
FUGR	Function Group
FUNC	Function Module
GSMP	Metric Provider
HTTP	HTTP Service
INTF	Interface
INTM	Intelligent Scenario Model
INTS	Intelligent Scenario
MSAG	Message Class
NONT	SAP Object Node Type
NTTY	Note Type
NROB	Number Range Object
RONT	SAP Object Type
RVBC	Review Booklet Configuration Model
SAJC	Application Job Catalog Entry
SAJT	Application Job Template

SCO1	Communication: Communication Scenario
SCO2	Communication: Inbound Service
SCO3	Outbound Service
SIA1	IAM: Business Catalog
SIA2	IAM: Restriction Type
SIA3	IAM: Authorization Object Extension
SIA5	IAM: Restriction Field
SIA6	IAM: App
SIA7	IAM: Business Catalog App Assignment
SIA8	IAM: Business Role Template
SIA9	IAM: Business Role Templ. Bus. Catalog Assignment
SIAD	Business Role Template Fiori Space Assignment
SKTD	Knowledge Transfer Document
SMBC	Maintainable Business Configuration
SMTG	OM: Email Template
SOD1	API Package
SOD2	API Package Assignment
SRVD	Service Definition
SUCO	Authorization Default Variant
SUSH	Assignment: Service --> Authorization Objects
SUSO	Authorization Object
SWCR	Software Component Relations
TABL	Table Definition
TRAN	Transaction
TTYP	Table Type
UIAD	Application Descriptors for the Fiori Launchpad Enviorn-ment
UIPG	Fiori Launchpad Page Template
UIST	Fiori Launchpad Space Template
XSLT	Transformation

Restrictions

- All objects imported with abapGit are imported as inactive, except the ones that don't have an inactive state. Therefore, you have to activate all the imported objects after the import.

> **Note**
>
> During the activation process, some objects may depend on other objects which need to be activated beforehand. For instance, the newly enabled DSFI objects depend on a corresponding DSFD, that needs to be activated beforehand.
>
> Also note that mass activation is not supported.

- All local changes to ABAP objects that are not saved are overwritten by the import.
- Open transport requests for any of the imported ABAP objects must be released before they can be changed. Otherwise, they get locked.

4.2.9.5 Setting Up and Working with Your Landscape

Depending on your requirements, you need to adjust the setup of your software lifecycle management and the underlying landscape. The following sections give you examples for the most common setups.

In general, the number of ABAP systems in your landscape is the result of the processes (development (DEV), quality assurance (QAS), and production (PRD)) that need to run in parallel and the number of branches of your biggest software component that is worked on or tested in parallel.

The number of ABAP systems = (number of branches x DEV systems) + (number of branches x QAS systems) + (number of PRD systems).

This is due to the fact that an ABAP system always checks out only one branch of a software component. Yet, several software components represented by one branch can run in one ABAP system in parallel.

4.2.9.5.1 Required Tools

Different actions require different tools. In the process descriptions, the following tools are used:

- SAP BTP cockpit (web-based administration interface)
- SAP Fiori app Maintain Business Roles [page 3068]
- SAP Fiori app Maintain Business Users [page 3044]
- SAP Fiori app Manage Software Components [page 3222]
- SAP Fiori app Export Customizing Transports [page 2978]
- SAP Fiori app Maintain Business Configurations [page 2970]
- SAP Fiori app Manage API Snapshots
- ABAP development tools for Eclipse is an Eclipse installation with ABAP Development Tools plugin (see ABAP Development Tools for Eclipse)
- Custom SAP Fiori apps: these apps are the result/aim of your development
- External test tools, for example for OData tests
- External documentation tool to document test results and release decision if required

4.2.9.5.2 Required Business Roles

Different actions require different roles as well. In the process description, the following role proposals are used:

- System Administrator
  - Creates ABAP systems in the SAP BTP cockpit
  - Needs authorization for space/organization management in the Cloud Foundry environment subaccount
- User Administrator
  - Maintains business roles and assigns users to them
  - Needs authorization for business catalogs

Catalog ID	Catalog Description	Needed For
SAP_CORE_BC_IAM_UM	Identity and Access Management – User Management	Creating and maintaining business users with *Maintain Business Users* app
SAP_CORE_BC_IAM_RM	Identity and Access Management – Role Management	Maintaining business roles with *Maintain Business Roles* app

- Release Manager
  - Performs tasks that are executed centrally for a release, such as software component creation and import, making the release decision, approving corrections, creating and releasing transports, creating API Snapshots etc.
  - Needs authorization for business catalogs

Catalog ID	Catalog Description	Needed For
SAP_A4C_BC_MSCL_PC	Lifecycle Management – Software Components	Creating, importing software components with *Manage Software Components* app
SAP_A4C_BC_TRN_REL_PC	Development Transport Release Management	Releasing transport requests in ABAP development tools for Eclipse
SAP_A4C_BC_DEV_OBJ_DIS_PC	Development Objects – Display	Show development objects in transport organizer in ABAP development tools for Eclipse
SAP_CORE_BC_IAM_UM	Identity and Access Management - User Management	Un-/locking developers for corrections in correction ABAP systems with *Maintain Business Users* app
SAP_CORE_BC_IAM_RA	Identity and Access Management – Role Assignment	En-/disabling developers for corrections in correction ABAP systems by role assignment with *Maintain Business Users* app
SAP_CORE_BC_BCT_TRN_REL_PC	Business Configuration – Transport Release Management	Creating, editing and releasing customizing transports with *Export Customizing Transports* app

Catalog ID	Catalog Description	Needed For
SAP_CORE_BC_ARS_PC	Lifecycle Management	Creating API Snapshots using the *Manage API Snapshots* app to allow running API compability checks for released objects. As well as downloading and uploading the snapshots for a reuse across different systems.

- Developer
  - Develops the code and performs tests
    - Creates software component relations to declare access permissions and dependencies between software components. For more information, see Software Component Relations.
    - Releases APIs based on consistency and stability requirements and defines the visibility for use in different ABAP language versions. For more information, see Released APIs.
    - Runs ATC Checks (Checking Quality of ABAP Code with ATC) and requests ATC exemptions for false-positive findings.
  - Needs authorization for business catalogs

Catalog ID	Catalog Description	Needed For
SAP_A4C_BC_DEV_PC	Development – ABAP Development Tools	Developing in ADT for Eclipse
SAP_A4C_BC_TRN_MNG_PC	Development – Transport Management	Creating transport requests in ADT for Eclipse

- Quality Manager
  - Approves or rejects ATC Exemptions
  - Needs authorization for business catalogs

Catalog ID	Catalog Description	Needed for
SAP_A4C_BC_DEV_CHKE_APR_PC	Quality Management - ABAP Test Cockpit Exemption Approval	Approving or rejecting ATC Exemptions

- Tester
  - Tests the software

    > ⓘ Note
    >
    > Depending on your organization, the tasks of this role can be performed by automatic test tools, developers, or dedicated persons that are responsible for executing manual tests.

  - Needs authorizations for custom business catalogs created via IAM business catalogs for custom apps.

    > ⚠ Caution
    >
    > Do not assign catalog SAP_CORE_BC_EXT_TST (Extensibility - Custom Apps and Services) to a tester.
    >
    > As this catalog is used to provide an easy test of custom apps or their underlying services in a development ABAP system only by adding them automatically to it during activation, real

> service tests with own catalogs might not be tested securely. In non-development systems, automatic authorization via business catalog `SAP_CORE_BC_EXT_TST` is not enabled, instead custom authorization is required.

- Business Configuration Expert
  - Maintains and tests business configuration

Catalog ID	Catalog Description	Needed For
`SAP_CA_BC_IC_LND_PC`	Customizing - Business Configuration	General business configuration tasks, such as uploading business configuration or viewing the change log for business configuration
`SAP_CORE_BC_BCT_TRN_MNG_PC`	Business Configuration - Transport Management	Creating and editing customizing transports
Application-specific catalog with configuration apps, for example `SAP_CA_BC_IC_LND_NUM_PC`	Example: Number Range Management - Configuration	Configuring number ranges

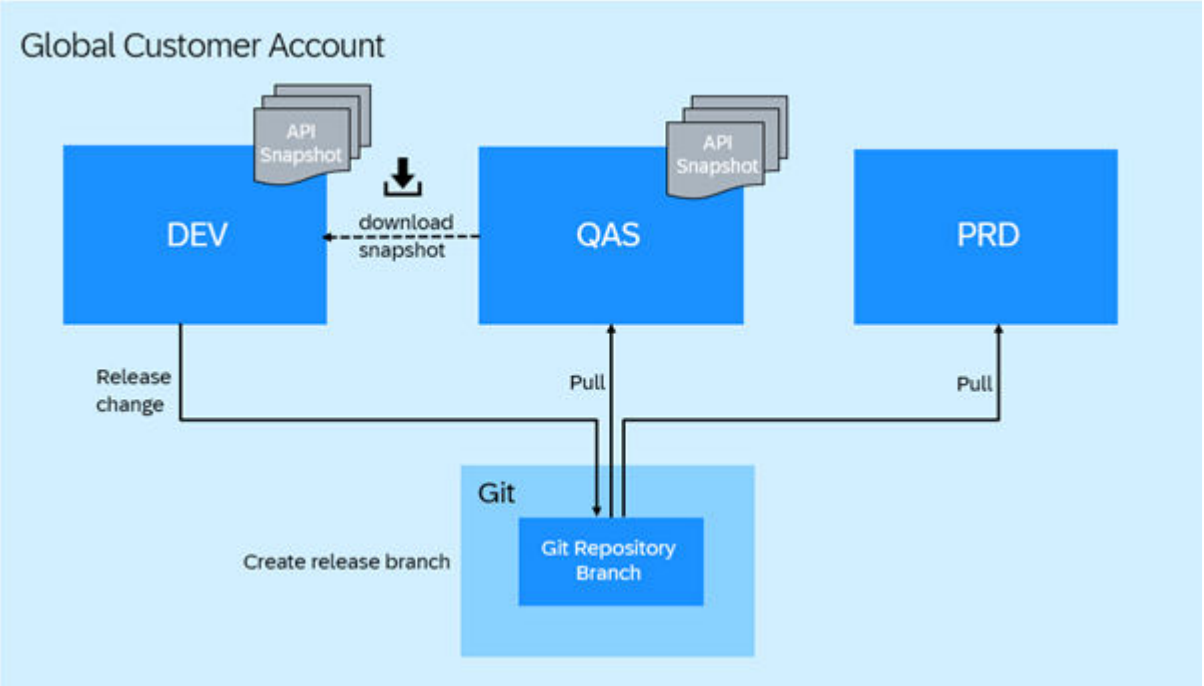
4.2.9.5.3 Important Notes

- The creation of a Git repository branch creates a clone of the parent branch. If you create a branch immediately after the release decision, you have a “frozen” state. You can import this state safely into one or multiple production systems, “pre-production” systems, etc., or you may use it later for auditing.
- In the different system setups, the production systems can either be provisioned once development starts or later on, when needed for the first time for the Go Live, which is the point in time, when the first release is used productively in the production system.
- A classical 3-system setup comprising a development, quality assurance, and production system can be enhanced by pre-production systems, multiple test systems, etc. Test systems can be provisioned on demand (temporary) or permanently.

4.2.9.5.4 Use Case 1: One Codeline in a 3-System Landscape

You can apply this setup if you have occasional development activities for larger applications where testing needs to run in parallel to development or should take place in a non-development system to ensure the solution also runs in a non-development system. In this setup, you either need to be able to pause development for a fix that has to be delivered before the next release or you have to deliver fixes as part of the next possible release.

This landscape consists of a development, quality assurance, and production system. Software component branches are provided remotely in a Git repository branch and checked out locally in such systems. In case of released APIs in the involved software components, API Snapshots are generated locally after release decisions in the quality assurance system and dowloaded for use in the development system..



For Go Live/Development After Go Live (Including Deferrable Corrections)

Starting situation for Go Live

The Go Live process is characterized by creating different systems only when needed for the first time, however, you can provision the systems already beforehand. Furthermore, the software component of a planned solution does not exist from the beginning. The resulting release branch is YYYY-01. Apart from this, the Go Live process does not differ from the release development processes afterwards.

Starting situation after Go Live

- Development system DEV is based on the main branch
- Quality Assurance system QAS and production system PRD are based on the latest release branch YYYY-<nn>. In case of a first release after the Go Live, YYYY-<nn> is YYYY-01
- Software component relations are defined for dependencies between leading- and reuse software components in the YYYY-<nn> release
- In case of released APIs: In the quality assurance system QAS, a check-relevant API snapshot named YYYY-<nn> was generated manually with all released APIs extracted as per the current release. Furthermore, the generated snapshot was downloaded from system QAS and uploaded to development system DEV. Finally the API snapshot was set to check-relevant in system DEV.

System QAS has always the same software state as the PRD system, unless a new change is tested and released. This means, transport requests are released in development ABAP systems only if development is completed and it is planned to import the changes to the production system.

This process can also be used for deferrable corrections, which do not need to reach production before the next development release. These corrections are handled like regular development.

Step	System	Role	Task	Tool
0	DEV	Release Manager	At Go Live only: Create the software component and clone it initially	*Manage Software Components* app
			If required, create a customizing transport request and tasks for the relevant business configuration expert(s).	*Export Customizing Transports* app
1a	DEV	Developer	Develop new functionality or a deferrable correction. All changes are collected in workbench transport requests. During development ATC checks are running and exemptions requested for false-positive findings (Requesting ATC Exemptions). Make sure to maintain new dependencies between software components in software component relations (Editing Software Component Relations).	ABAP Development Tools for Eclipse
1b	DEV	Business Configuration Expert	Maintain business configuration. All changes are collected in customizing transport requests	*Maintain Business Configurations* app
2	DEV	Quality Manager	Approve or reject ATC exemptions	ABAP development tools for Eclipse: ATC Exemptions
3	DEV	Release Manager	Release the transport request(s). During transport release ATC checks are running (Working with ATC During Transport Release). The changes are now in the main branch.	ABAP Development Tools for Eclipse: Transport Organizer

Step	System	Role	Task	Tool
4	QAS	Release Manager	At Go Live: Pull software component(s) into system QAS. After Go Live: Check out main branch of software component(s)	*Manage Software Components* app
5	QAS	Tester	Test the change and report test result. During testing ATC checks are running, see Working with ATC During Development. If changes are required, repeat steps 1-4	ABAP development tools for Eclipse and custom SAP Fiori apps as well as external test tools, see Automate the Software Lifecycle Process External documentation tool
6	QAS	Release Manager	Release decision: the changes are successfully tested and approved	External documentation tool
7	QAS	Tester	Test the change and report the test result. During testing ATC checks are running, see Working with ATC During Transport Release.	ABAP development tools for Eclipse and custom SAP Fiori apps as well as external test tools. For more information, see Automate the Software Lifecycle Management Process External documentation tool
8	QAS	Release Manager	Check out the new release branch YYYY-<nn+1>	*Manage Software Components* app

Step	System	Role	Task	Tool
9	QAS/DEV	Release Manager	In case of Released APIs: Create and generate a new API snapshot YYYY-<nn+1> for the new release in system QAS. Set the new API snapshot as check-relevant so that it will be used as reference for API compatibility checks. Afterwards download the generated snapshot from system QAS and upload it to system DEV. Finally, set the snapshot as check-relevant in system DEV as well.	Manage API Snapshots app
10	DEV	Release Manager	Check out the new release branch YYYY<nn+1> for each software component (at Go Live: YYYY-01) into system PRD	*Manage Software Components*

Urgent Corrections

- Development system DEV is based on the main branch
- Quality assurance system QAS and production system PRD are based on the latest release branch YYYY-<nn>. In case of a correction after the Go Live before the second release, YYYY-<nn> is YYYY-01
- In case of released APIs: In the quality assurance system QAS and development system DEV, a check-relevant API snapshot named YYYY-<nn> is generated with all released APIs extracted

This process differs from the previous one in the branch it is developed in. As the correction is too urgent to release it with the next development release only, it is done in the release branch. To achieve this separation, all current development activities need to be paused because the DEV system needs to check out the latest release branch instead of the main branch. That means all open transports in DEV need to be released first to save the work in progress to the main branch, so that feature development can be resumed later after the correction.

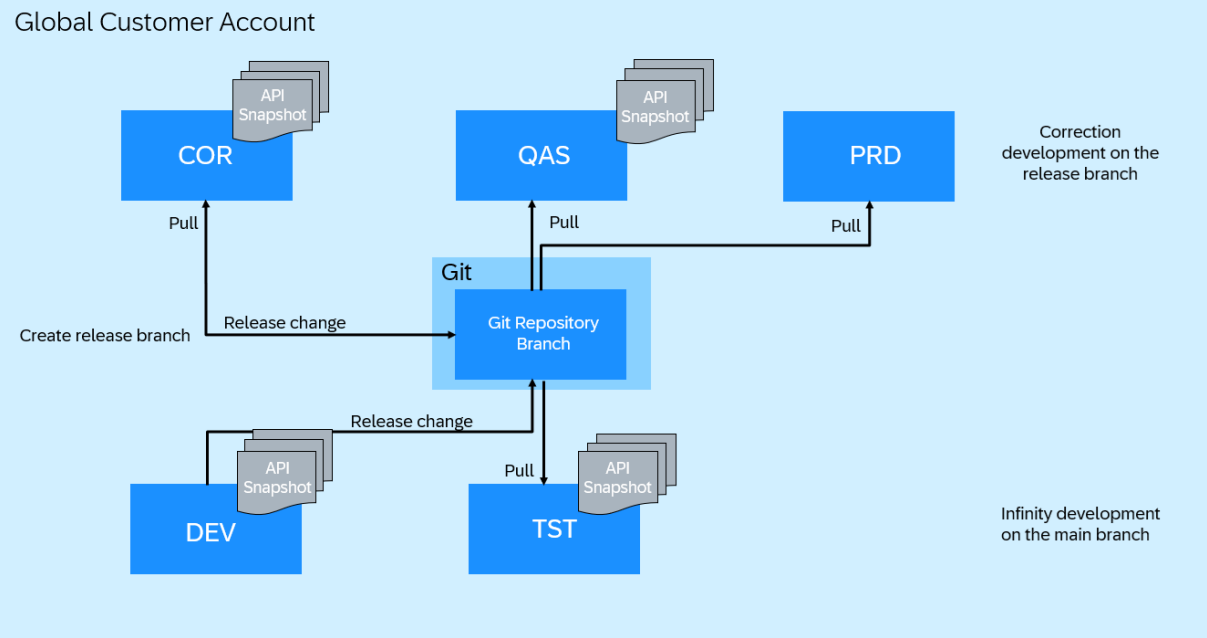
Step	System	Role	Task	Tool
1	DEV	Release Manager	Check out the release branch YYYY-<nn> for each software component	*Manage Software Components* app

Step	System	Role	Task	Tool
			If required, create a customizing transport request and tasks for the relevant business configuration expert(s)	*Export Customizing Transports* app
2	DEV	Release Manager	In case of Released APIs: Set API Snapshot created for the release YYYY-<nn> as check-relevant. For release YYYY-01 all API snapshots will be set to not check-relevant as no API snapshot is available for comparison.	Manage API Snapshots app
3a	DEV	Developer	Fix existing functionality. All changes are collected in workbench transport requests. During Development ATC checks are running and exemptions requested for false-positive findings, see Requesting ATC Exemptions.	Manage API Snapshots app
3b	DEV	Business Configuration Expert	Maintain business configuration. All changes are collected in customizing transport requests	Custom Business Configurations app
4	DEV	Quality Manager	Approve or reject ATC exemptions	ABAP Development Tools for Eclipse: ATC Exemptions
5	DEV	Release Manager	Releases the transport requests. During transport release ATC checks) are running.	ABAP Development Tools for Eclipse: Transport Organizer or *Export Customizing Transports* app
6	QAS	Release Manager	Pull the software component(s) to get the correction into the already checked out release branch YYYY-<nn>	*Manage Software Components* app

Step	System	Role	Task	Tool
7	QAS	Tester	Test the change and report the test result. During Testing ATC checks are running, see Working with ATC During Transport Release If changes are required, repeat steps 2-7	ABAP Development Tools for Eclipse and custom SAP Fiori apps as well as external test tools, see Automate the Software Lifecycle Management Process. External documentation tool
8	QAS	Release Manager	Fix is successfully tested and approved	External documentation tool
9	PRD	Release Manager	Pull the software component(s) to get the correction into the already checked out release branch YYYY-<nn>	*Manage Software Components* app
10	DEV	Release Manager	Check out the main branch in system DEV for each software component	*Manage Software Components* app
11	DEV	Release Manager	In case of Released APIs: Set API Snapshot created for the latest release as check relevant. For release YYYY-01 all API snapshots shall be set to not check-relevant as no API snapshot is available for comparison.	Manage API Snapshots app
12	DEV	Developer, Business Configuration Expert	Perform the same changes as for the correction in the main branch and release them	ABAP Development Tools for Eclipse *Maintain Business Configurations* app *Export Customizing Transports* app

4.2.9.5.5 Use Case 2: One Development and Correction Codeline in a 5-System Landscape

You can apply this setup if you have permanent/infinite development activities for large applications with many developers, where development cannot be paused to implement an urgent correction. Software component branches are provided remotely in a Git repository branch and checked out locally in such systems. In case of released APIs in the involved software components, API snapshots are generated locally after release decisions in the test system and are then distributed to other systems using the download or upload option.. Corrections need to run in parallel to development and on a released state. You need to separate testing from development to ensure the solutions also runs in a non-development system before being delivered to production.



General considerations:

- Systems COR and QAS have the same software state as production system PRD, unless a new change is tested and released. This means transport requests are released in the DEV system only if development is completed and if it is planned to import the changes to the production system.
- Releases are planned and communicated to development in advance:
  - Upon cut-off date, development is finished. All development that is released at this time must be tested and be of good quality. From then on, you have to fix defects in the COR system and maintain them in parallel in the DEV system.
  - Upon release date, all defects must be fixed. If you make the decision during testing in the QAS system that a complete functionality is not delivered, developers must delete, revert, or disable the functionality in the COR system and release the corresponding transport requests. You cannot remove objects from the release branch, e.g. by deselecting transport requests. To revert objects to an older transported state, use the compare editor of the Eclipse *History* view. If the withdrawal of the functionality should be performed in the DEV system as well, it is considered as a correction and you have to perform double maintenance of corrections into the DEV system. The released software state from the COR system is imported into the production system(s) PRD.
- System COR is usually locked for development. First, this means developers cannot do changes by default and there are two approaches how to handle this:

	User Locking	Read-Only + Write Developer Role
How-to Details	Unlock user on demand	Assign write role on demand
Pros	No additional role needed	No generic read user needed
		No logon with different user for read access needed
		User-specific auditing
Cons	Generic read user needed if you want to provide read access	Additional role needed

Second, developers are also not allowed to create transport requests and tasks on their own, but it's the release manager who creates them for all developers. This separation is achieved by giving business catalog `SAP_A4C_BC_TRN_MNG_PC` (Development - Transport Management) to the release manager instead of the developer role in correction system COR.

For Go Live/Development after Go Live (Including Deferrable Corrections)

Starting Situation for Go Live:

Recently created development system DEV and other already existing ABAP systems cannot be based on some branch yet, as the software component does not exist yet.

The Go Live process is characterized by creating different systems only when needed for the first time, however, you can provision the systems already beforehand. Additionally, the future solution's software component does not exist from the start. The resulting release branch is YYYY-01. Apart from this, the Go Live process does not differ from the release development processes.

Starting Situation after Go Live:

- Development system DEV and test system TST are on the main branch.
- Correction system COR, quality assurance system QAS, and production system PRD are on release branch YYYY-<nn>.
- Software component relations are defined for dependencies between leading- and reuse software components in the YYYY-<nn> release.
- In case of released APIs: In the test system TST, you manually generate a check-relevant API snapshot named YYYY-<nn> that extracts all released APIs. You then download this snapshot and upload it to the remaining systems: DEV, COR, and QAS. Moreover, you need to set the uploaded snapshot to check-relevant in all these systems.

This process can also be used for deferrable corrections, which do not need to reach production before the next development release. These corrections are handled like normal development.

Step	System	Role	Task	Tool
0	DEV	Release Manager	At Go Live only: create a software component of type *Development* and, if required, a software component of type *Business Configuration*. Pull the software component(s) into the system.	*Manage Software Components* app
			If required, create a customizing transport request and create tasks for the relevant business configuration expert(s)	*Export Customizing Transports* app
1a	DEV	Developer	Develop a new functionality or a deferrable correction. All changes are collected in Workbench transport requests. During Development ATC checks are running (Working with ATC During Development) and exemptions requested for false-positive findings (Requesting ATC Exemptions). Make sure to maintain new dependencies between software components in software component relations, see Editing Software Component Relations.	ABAP Development Tools for Eclipse
1b	DEV	Business Configuration Expert	Maintain business configuration. All changes are collected in customizing transport requests	*Maintain Business Configurations* app
2	DEV	Quality Manager	Approve or reject ATC exemptions	ABAP Development Tools for Eclipse: ATC Exemptions

Step	System	Role	Task	Tool
3	DEV	Release Manager	Once development is finished, release the transport request(s). During transport release, ATC checks are running, see Working with ATC During Transport Release. The changes are now in the main branch	ABAP Development Tools for Eclipse: Transport Organizer or Export Customizing Transports app
4	TST	Release Manager	Pull the software component(s) into system TST	*Manage Software Components* app
5	TST	Tester	Test the change and report the test result. During Testing ATC checks are running.	ABAP Development Tools for Eclipse and custom SAP Fiori apps as well as external test tools. Refer to Automate the Software Lifecycle Management Process External documentation tool
			If changes are required, repeat steps 1-5	
6	TST/DEV	Release Manager	Before Cutoff: In case of Released APIs, Create and generate a new API snapshot YYYY-<nn+1> for the new release in system TST. Set the new API snapshot as check-relevant so that it will be used as reference for API compatibility checks. Finally, download the generated API snapshot, upload it to the DEV system and set it to check-relevant.	Manage API Snapshots app

Step	System	Role	Task	Tool
7	QAS or any other	Release Manager	Cutoff: at cutoff date, create a release branch YYYY-<nn+1> for each software component (at Go Live: YYYY-01) (release candidate)	*Manage Software Components* app
8	QAS	Release Manager	Check out the release branch YYYY-<nn+1> of each software component (at Go Live: YYYY-01) into system QAS	*Manage Software Components* app
9	QAS	Tester	Test the release candidate and report the test result. During testing ATC checks are running.	ABAP Development Tools for Eclipse with ADT and custom SAP Fiori apps as well as external test tools. Refer to Automate the Software Lifecycle Management Process. External documentation tool
10	COR	Release Manager	Check out the branch YYYY-<nn+1> of each software component (at Go Live: YYYY-01) into system COR	*Manage Software Components* app
11	COR	Release Manager	Enable the respective development users for development in system COR, depending on the process you decided for, either by unlocking or assigning a different role	*Maintain Business Users* app
			If required, create a customizing transport request and create tasks for the relevant business configuration expert(s)	*Export Customizing Transports* app

Step	System	Role	Task	Tool
12a	COR	Developer	Implement development-related corrections. During testing ATC checks are running. and exemptions requested for false-positive findings (Requesting ATC Exemptions).	ABAP Development Tools for Eclipse
12b	COR	Business Configuration Expert	Implement corrections to business configuration content	*Maintain Business Configurations* app
13	COR	Quality Manager	Approve or reject ATC exemptions	ABAP Development Tools for Eclipse: ATC Exemptions
14	COR	Release Manager	Release the development transport request(s). During testing ATC checks are running. The changes are now in the release candidate.	ABAP Development Tools for Eclipse: Transport Organizer or *Export Customizing Transports* app
15	QAS	Release Manager	Pull the software component(s) to get the correction into the already checked out release branch YYYY-<nn+1> (@Go Live: YYYY-01)	*Manage Software Components* app
16	QAS	Tester	Test the change and report the test result. During testing ATC checks are running. If changes are required, repeat steps 12-16	ABAP Development Tools for Eclipse and custom SAP Fiori apps as well as external test tools. Refer to Automate the Software Lifecycle Management Process External documentation tool
17	QAS	Release Manager	Release decision: the changes are successfully tested and approved	External documentation tool

Step	System	Role	Task	Tool
18	QAS/COR	Release Manager	In case of Released APIs: Upload the API snapshot YYYY-<nn+1> for the new release in the system of the correction code line. Set the new API snapshot as check-relevant so that it will be used as reference for API compatibility checks.	Manage API Snapshots app
19	PRD	Release Manager	Check out the release branch YYYY-<nn+1> of each software component (at GoLive: YYYY-01) into system PRD	*Manage Software Components* app
20	COR	Release Manager	Disable the respective development users for development in system COR	*Maintain Business Users* app
	DEV		If required, create a customizing transport request for the double maintenance of the correction, and create tasks for the relevant business configuration expert(s)	*Export Customizing Transports* app
21a	DEV	Developer	Perform double maintenance of your development-related corrections in the main branch. During Development ATC checks are running During testing ATC checks are running. and exemptions requested for false-positive findings (Requesting ATC Exemptions).	ABAP Development Tools for Eclipse
21b	DEV	Business Configuration Expert	Perform double maintenance of your business configuration corrections in the main branch.	*Maintain Business Configurations* app

Step	System	Role	Task	Tool
21c	DEV	Quality Manager	Approve or reject ATC exemptions	ABAP Development Tools for Eclipse: ATC Exemptions
21d	DEV	Release Manager	Release the transport request(s). During transport release ATC checks are running. The corrections are now implemented both in the main branch and in the current release branch.	ABAP Development Tools for Eclipse: Transport Organizer or *Export Customizing Transports* app
22	TST	Release Manager	Pull the software component(s) to get the correction into the already checked out main branch	*Manage Software Components* app

Skipping a Release

If issues during the test phase of YYYY-<nn+1> cannot be fixed in a reasonable time frame until the next release date, you can skip that release, especially if you have a tight release schedule (“continuous delivery” model). In that case, you have to perform double maintenance for the unfinished corrections from YYYY-<nn+1> in the main branch of the development ABAP system, release them, and create the new release branch YYYY-<nn+2> derived from that main branch. That way, branch YYYY-<nn+2> contains finished new development as well as the unfinished corrections from branch YYYY-<nn+1>. Before cutoff for the new release branch YYYY-<nn+2>, perform the usual API snapshot handling steps. Afterwards, you can bring system COR and QAS to branch YYYY-<nn+2> and continue with that.

> ⓘ Note
>
> Branches cannot be deleted or marked as obsolete. Therefore, it’s important to use other tools to inform consumers about not using branch YYYY-<nn+1>.

Urgent Corrections

The starting situation:

- Development system DEV and test system TST are on the main branch
- Correction system COR, quality assurance system QAS, and production system PRD are on release branch YYYY-<nn>
- In case of released APIs: In the test system TST, you manually generate a check-relevant API snapshot named YYYY-<nn> that extracts all released APIs. You then download this snapshot and upload it to the

remaining systems: DEV, COR, and QAS. You set the uploaded snapshot to check-relevant in all these systems.

This process is a subset of the previous development process and can be applied to corrections that are too urgent to release with the next development release.

Step	System	Role	Task	Tool
1	COR	Release Manager	Enable the respective development users for development in system COR, depending on the process you decided for, either by unlocking or assigning a different role.	*Maintain Business Users* app
			If required, create a customizing transport request, and create tasks for the relevant Business Configuration Expert(s)	*Export Customizing Transports* app
2a	COR	Developer	Create a Workbench transport request and implement the correction. During Development ATC checks are running and exemptions requested for false-positive findings (Requesting ATC Exemptions).	ABAP Development Tools for Eclipse
2b	COR	Business Configuration Expert	Maintain business configuration. All changes are collected in customizing transport requests	*Maintain Business Configurations* app
3	COR	Quality Manager	Approve or reject ATC exemptions	ABAP Development Tools for Eclipse: ATC Exemptions
4	COR	Release Manager	Release the transport request(s). During transport release ATC checks are running The changes are now in the release candidate	ABAP Development Tools for Eclipse: Transport Organizer or *Export Customizing Transports* app

Step	System	Role	Task	Tool
5	QAS	Release Manager	Pull the already checked out branch YYYY-<nn+1> of each software component into system QAS	*Manage Software Components* app or external tool calling the pull service of communication scenario Test Integration (SAP_COM_0510)
6	QAS	Tester	Test the change and report the test result. During transport release ATC checks are running.	ABAP Development Tools for Eclipse and custom SAP Fiori apps as well as external test tools, see Automate the Software Lifecycle Management Process. External documentation tool
			If changes are required, repeat steps 2-6	
7	QAS	Release Manager	As the correction was successfully tested before it gets approved now.	External documentation tool
8	PRD	Release Manager	Pull the already checked out release branch YYYY-<nn+1> of each software component into system PRD	*Manage Software Components* app
9	COR	Release Manager	Disable the respective development users for development in system COR	*Maintain Business Users* app
10	DEV	Release Manager	If required, create a customizing transport request for the double maintenance of the corrections, and create tasks for the relevant business configuration expert(s)	*Export Customizing Transports*

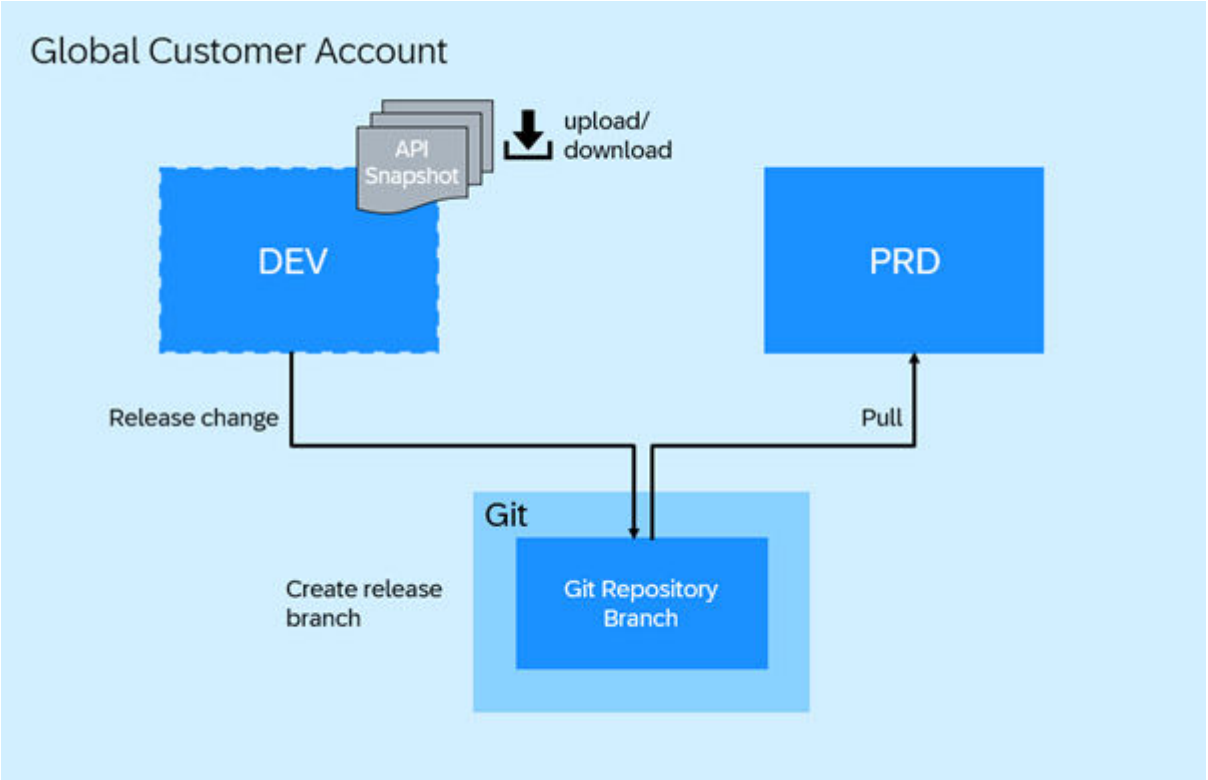
Step	System	Role	Task	Tool
11a	DEV	Developer	Perform double maintenance of your development-related corrections in the main branch. During transport release ATC checks are running and exemptions requested for false-positive findings (Requesting ATC Exemptions).	ABAP Development Tools for Eclipse
11b	DEV	Business Configuration Expert	Perform double maintenance of your business configuration corrections in the main branch	*Maintain Business Configurations* app
11c	DEV	Quality Manager	Approve or reject ATC exemptions	ABAP Development Tools for Eclipse: ATC Exemptions
11d	DEV	Release Manager	Release the transport request(s). During transport release ATC checks are running . The corrections are now implemented both in the main branch and in the current release branch.	ABAP Development Tools for Eclipse: Transport Organizer or *Export Customizing Transports* app
12	TST	Release Manager	Pull the software component to get correction into already checked out main branch	*Manage Software Components* app

4.2.9.5.6 Use Case 3: One Codeline with On-Demand Development Systems

You can apply this setup if you have:

- Small development projects (for example one SAP Fiori application, one developer)
- Occasional development activities (for example after an initial development phase, it is expected to only implement new features on a yearly basis)

The ABAP system landscape consists of a permanent production system and an on-demand development system.



The advantage of this setup is that you have to administer few systems and only pay for the development system during the development periods. The payment model according to the lifetime of a system requires an SAP BTP enterprise agreement contract.

The disadvantages of this setup are that de-commissioning the development system means losing the change history as well as the configuration and application data.

The user base and authorizations have to be set up each time the development system is provisioned.

Not testing with a transported version of the solution always comes with the risk of forgetting to transport an object. This can be avoided by testing in the development system on the release branch.

> ⓘ Note
>
> A much more production-like test can only be ensured in a non-development system, where the objects of the solution are initially created and changed by pulls, and where there is no authorization automatism for testing, see tester role in Required Business Roles [page 890].
>
> API compatibility checks are difficult to use without a permanent system. The system doesn't transport API snapshots that require ATC checks and it deletes them during system deprovisioning.. For more information, see Checking the Compatibility of Released APIs. We thus strongly recommend developing with Use Case 1: One Codeline in a 3-System Landscape [page 892] instead of use case 3. You can then commission or decommission the quality assurance system on demand. If you want to use API compatibility checks with use case 3, download API snapshots from system DEV as a backup. You can upload them whenever you create a new DEV system..

For the Go Live

Starting situation:

- A (temporary) development system is set up
- The production system is set up
- Both systems are based on the main branch
- Software Component Relations are defined for dependencies between leading- and reuse software components

Step	System	Role	Task	Tool
1	DEV, PRD	System Administrator	Provision a development and a production system	SAP BTP cockpit
2	DEV, PRD	User Administrator	Create users and maintain roles	*Maintain Business Users* and *Maintain Business Roles* app
3	DEV	Release Manager	Create a software component of type *Development* and, if required, a software component of type *Business Configuration*. Pull the software component(s) into the development system.	*Manage Software Components* app
4	PRD	Release Manager	Pull the (empty) software component(s) into the production system	*Manage Software Components* app

Step	System	Role	Task	Tool
5a	DEV	Developer	Develop new functional requirements or correct existing functionalities. All required changes are collected in Workbench transport requests. During development ATC checks are running and exemptions requested for false-positive findings Requesting ATC Exemptions . Make sure to maintain new dependencies between software components in software component relations (Editing Software Component Relations).	ABAP Development Tools for Eclipse
5b	DEV	Business Configuration Expert	Maintain business configuration. All changes are collected in customizing transport requests.	*Maintain Business Configurations* app
6	DEV	Tester	Test the change and report the test results. During testing ATC checks are running.	ABAP Development Tools for Eclipse with and custom SAP Fiori apps as well as external test tools. See Automate the Software Lifecycle Management Process Process. External documentation tool
7	DEV	Quality Manager	Approve or reject ATC exemptions	ABAP Development Tools for Eclipse: ATC Exemptions
		Release Manager	If changes are required, repeat steps 5-7	
8	DEV	Release Manager	Release decision: the changes are successfully tested and approved.	External documentation tool

Step	System	Role	Task	Tool
9	DEV	Release Manager	Release the transport request(s). During transport release ATC checks are running.	ABAP Development Tools for Eclipse: Transport Organizer or *Export Customizing Transports* app
10	DEV	Release Manager	Create a release branch YYYY-<nn> for each software component	*Manage Software Components* app
11	DEV	Release Manager	In case of released APIs: For released APIs, create a new API snapshot YYYY-nn for the new release in the DEV system. Set the new API snapshot as check-relevant so the system uses it as a reference for API compatibility checks. Then download the snapshot from the DEV system as a backup for when you provision a new DEV system.	
12	PRD	Release Manager	Check out the release branch YYYY-<nn> of each software component	*Manage Software Components* app
13	DEV	System Administrator	De-provision system DEV	SAP BTP cockpit

Productive Usage

- The production ABAP system is based on the main branch
- Since no development can be performed in the production ABAP system, a temporary development ABAP system must be set up if occasional development is required

Occasional Development

- The production ABAP system is based on the main branch
- A temporary development ABAP system is set up
  - All software components are imported

Step	System	Role	Task	Tool
1	DEV	Release Manager	Provision a new development system	SAP BTP cockpit
2	DEV	Release Manager	Create users, maintain and assign roles	*Manage Business Users* and *Manage Business Roles* app
3	DEV	Release Manager	Import main branch of each software component into the newly provisioned development system	*Manage Software Components* app
4	DEV	Release Manager	In case of Released APIs: Upload the API snapshot YYYY-nn for the latest release in the new Development system. Set the new API snapshot as the reference for API compatibility checks.	*Manage API snapshots* app
5a	DEV	Developer	Develop features. All required changes are collected in Workbench transport requests. Working with ATC during development and exemptions requested for false-positive findings Requesting ATC Exemptions . Make sure to maintain new dependencies between software components in software component relations (Editing Software Component Relations).	ABAP Development Tools for Eclipse
5b	DEV	Business Configuration Expert	Maintain business configuration. All changes are collected in customizing transport requests.	*Maintain Business Configurations* app
6	DEV	Release Manager	Release all transport requests. During transport release ATC checks are running Working with ATC during transport release	ADT for Eclipse: Transport Organizer or *Export Customizing Transports* app

Step	System	Role	Task	Tool
7	DEV	Tester	Test new developments/fixes. During development ATC checks are running	ADT for Eclipse and custom SAP Fiori apps as well as external test tools. See Automate the Software Lifecycle Management Process
8	DEV	Quality Manager	Approve or reject ATC exemptions If changes are required, repeat steps 4-7	ABAP Development Tools for Eclipse: ATC Exemptions
9	PRD	Release Manager	Pull the main branch of each software component into system PRD	Manage Software Components app
10	DEV	Release Manager	Delete the development system	SAP BTP cockpit

4.2.9.5.7 Double Maintenance of Corrections into Development

For corrections, double maintenance is necessary so that everything that had to be fixed during release testing or usage can be retrofitted into the development system.

> ⚠ Caution
>
> Missing double maintenance can lead to data loss in production systems.

As there is no selective picking of transport requests to merge corrections back into the main branch, this is manual effort.

You can use the *Compare Editor* in ABAP Development Tools for Eclipse to merge corrections back to the main branch.

The process would be as follows:

You can compare the latest version of an object in your ABAP Development Tools project in a hotfix system with the latest version of the same object in the development system. If there are newly created objects, you have to create an object manually in the development ABAP system.

To make a comparison, from the context menu of the object in the *Project Explorer* or in the source code editor, choose *Compare With* > *ABAP project*, where ABAP project is one of the ABAP projects defined in ABAP Development Tools.

Step	System	Role	Task	Tool
1	COR	Developer	Select the corrected objects in the transport request and open them. This can be done selectively object by object.	ABAP Development Tools for Eclipse: Transport Organizer View
2	COR, DEV	Developer	1. Select the resources in the Project Explorer in correction system COR 2. From the resource's pop-up menu, select Compare With 3. Select the ABAP project that points to your development system DEV	ABAP Development Tools for Eclipse: Project Explorer
3	DEV	Developer	Merge the changes into the DEV system	ABAP Development Tools for Eclipse

4.2.9.5.8 Transport of Business Configuration

Business configuration is client-dependent, just like master or transactional data, and after being imported, it is only available in the client where you have executed the import.

You usually maintain business configuration content in a non-productive environment and import it into test and productive environments. To do so, you record business configuration changes on transport requests of type *Customizing*. Once you release the request, all the changes are pushed into the remote repository associated with the used software component, just as for ABAP Workbench transport requests. Therefore, you can import business configuration content by pulling that same software component in the target system/client 100.

Depending on your scenario, you can use one of the following transport patterns:

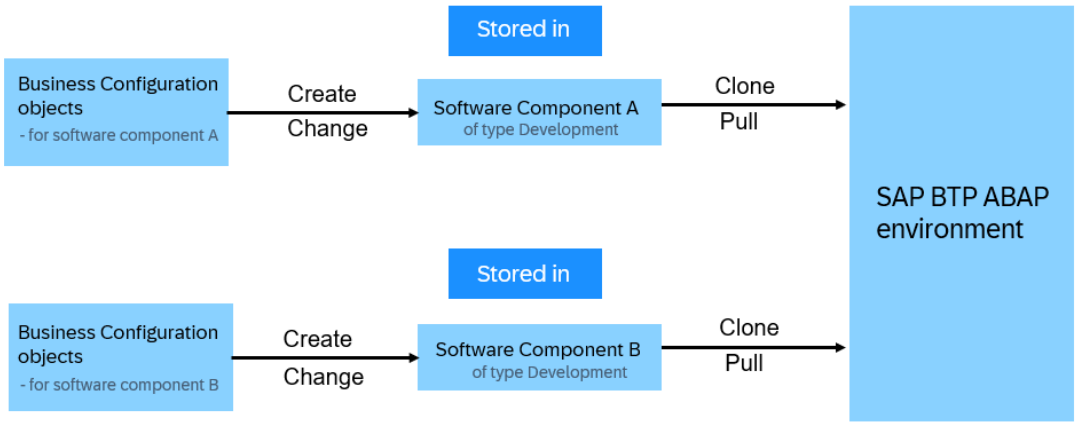
- Isolated tenants – no transport
  If you use an isolated tenant, you don't have to clone and transport a software component of type *Business Configuration*. You can still create content, but it is not imported or exported. Since a customizing transport request is a technical prerequisite for business configuration changes, the changes are written to a local customizing transport request. That means, you just have to create a transport request. The system then generates a local request and sets its category to *Default*. See Working in the Export Customizing Transports App [page 2980].

> **Note**
>
> All consumer tenants (≥200) in multitenant systems are isolated tenants because the administration of software components is not supported in these tenants.

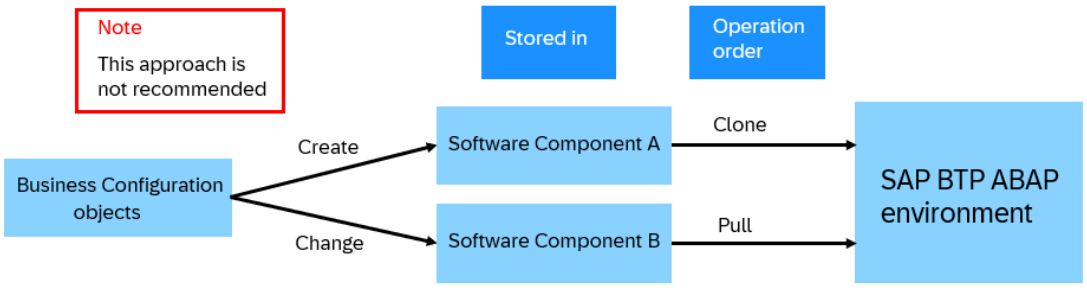
- Simple – transports via software component of type *Business Configuration*. See Transport Business Configuration via Software Component of Type Business Configuration [page 919].
  You can transport business configuration for simple solutions via a software component of type *Business Configuration*. Since you can only clone one software component of this type in each system, all business configuration content is pushed to the same remote repository.
  In this scenario, you have to create and manage customizing transport requests with the *Export Customizing Transports* SAP Fiori app. See Export Customizing Transports [page 2978].
- Multiple software components/Git repositories – transports via software components/Git repositores of type *Development*. See Transport Business Configuration via Software Components of Type Development [page 920].
  If you want to work on multiple separated development projects in parallel in one development system with decoupled release schedules and in multiple software components, it is not advisable to use a software component of type *Business Configuration* because you may need to transport different sets of business configuration content independently from one another. Instead, transport the configuration together with the development via the corresponding software components of type *Development*.

> **Note**
>
> We recommend storing business configuration objects in one software component. If you need different software components for the business configuration, you need to make sure to always store single business configuration objects in the same software component.

Explanation



In the picture below the business configuration object creation and changes are stored on different software components, which is not recommended. This results in the necessity to clone the software components in the right order to get the latest changes correctly imported into the system. For the case that you forgot which software component stores a business configuration object, you must determine the software collection. You will use this software collection to also store additional changes.



There are two ways to determine the software collection:

1. In the development system, where you created the business configuration objects, you can use the filter in the Transport Organizer View in ABAP development tools for Eclipse.
   - Logon to system.
   - Access the **Transport Organizer** view.
   - Adjust the tree configuration to make sure all relevant transport requests are selected.
   - In the Filter section, type the name of the object, or the description of the type (e.g. Business Role):

2. In every system the relevant software components are cloned: Search in the Manage Software Components app:
   - Navigate into the different software components one by one.
   - Choose the relevant branch.
   - Go over the list of commits added to the software component.
   - You must find the commit, with which the business configuration object was stored on the software component. There are two ways:
     - Identify commits from the description, where you assume the business configuration object was committed with.
     - Search the commits one by one.
   - Search inside the list of **Committed Objects** for the business configuration object.
     - You must provide parts of the object name in the search field in capital letters.

> → Remember
>
> Commits of systems that are already deleted will not show the **Committed Objects** list. Therefore, they cannot be searched.

Recommendations

1. Provide a well-defined text for transport requests. This text is used as a commit description along with the transport request number. Thus, it's easier to find commits containing business configuration objects during a search.
2. If you store an existing business configuration object on a transport request, follow the four-eyes principle. Let another person double-check the chosen software component to ensure correctness.

In all of these scenarios you may create and manage customizing transport requests with the SAP Fiori app **Export Customizing Transports**. See Export Customizing Transports.

To use the *Manage Software Components* app, business role `SAP_BR_ADMINISTRATOR` has to be assigned.

To use the *Export Customizing Transports* app, business role `SAP_BR_BPC_EXPERT` or `SAP_BR_ADMINISTRATOR` have to be assigned.

1. In the *Manage Software Components* app, create a software component of type *Business Configuration or Development* according to your scenario if transport of business configuration is required. Clone it to the tenant where you want the business configuration to be created. See How to Create Software Components and How to Clone Software Components.

   > ⓘ Note
   >
   > If you have already started development, such components may already be available.

2. In the *Export Customizing Transports* app, create a transport request. See Working in the Export Customizing Transports app. The system automatically sets its category to *Default*. Any user that is assigned to one of the transport request's tasks can use the request for change recording. In case a transport is required, assign the *transport target* according to the software component that has been created. An empty transport target results in a local transport.

   > ⚠ Restriction
   >
   > There can only be one open transport request of type *Default* at a time.

3. Release the transport request. See Working in the Export Customizing Transports app.

   > ⓘ Note
   >
   > In exceptional cases, for example for an urgent preimport that must be performed in parallel to the default request, you can create additional customizing transport requests. To create an additional customizing transport request in the *Export Customizing Transports* app, change the transport category from *Default* (`SAP_CUS_TRANSPORT_CATEGORY = DEFAULT_CUST`) to *Manual* (`SAP_CUS_TRANSPORT_CATEGORY = MANUAL_CUST`).

4.2.9.5.8.1 Transport Business Configuration via Software Component of Type Business Configuration

Learn how to transport business configuration via a software component of type *Business Configuration*.

Prerequisites

To use the *Manage Software Components* app, business role `SAP_BR_ADMINISTRATOR` has to be assigned.

To use the *Export Customizing Transports* app, business role `SAP_BR_BPC_EXPERT` has to be assigned.

Procedure

1. In the *Manage Software Components* app, create a software component of type *Business Configuration* and clone it to the tenant where you want the business configuration to be created. See Create Software Components [page 3224] and Clone Software Components [page 3226].
2. In the *Export Customizing Transports* app, create a transport request. See Working in the Export Customizing Transports App [page 2980]. The system automatically associates the request to the previously cloned software component and sets its category to *Default*. Any user that is assigned to one of the transport request's tasks can use the request for change recording.

   > ⚠ Restriction
   >
   > There can only be one open transport request of type *Default* at a time.

3. Release the transport request. See Working in the Export Customizing Transports App [page 2980].

   > ⓘ Note
   >
   > In exceptional cases, for example for an urgent preimport that must be performed in parallel to the default request, you can create additional customizing transport requests. To create an additional customizing transport request in the *Export Customizing Transports* app, change the transport category from *Default* to *Manual*. Alternatively, you can use ABAP Development Tools for Eclipse. See Transport Business Configuration via Software Components of Type Development [page 920].

4.2.9.5.8.2 Transport Business Configuration via Software Components of Type Development

Learn how to transport business configuration via a software component of type *Development*.

Prerequisites

To use the *Manage Software Components* app, business role `SAP_BR_ADMINISTRATOR` has to be assigned.

To use the *Export Customizing Transports* app or to create a customizing request in ABAP Development Tools for Eclipse, business role `SAP_BR_BPC_EXPERT` or administrator role `SAP_BR_ADMINISTRATOR`need to be assigned.

To use the Transport Organizer view in ABAP Development for Eclipse, business role `SAP_BR_DEVELOPER` has to be assigned. See Transport Organizer.

Procedure

1. In the *Manage Software Components* app, create software components of type *Development* and clone them to the tenant where you want the business configuration to be created. See How to Create Software Components and How to Clone Software Components.

   > **Note**
   >
   > If you have already started development, such components may already be available.

2. In ABAP Development Tools for Eclipse, use the *Transport Organizer* view to create customizing transport requests for each software component, making sure to assign the correct target layer for each of the components used.
3. In the Transport Editor, assign the following attributes to all the created customizing transport requests so that they are displayed in the *Export Customizing Transports* app:
   - `SAP_ATO_TRANSPORT_TYPE = BC`
   - `SAP_CUS_TRANSPORT_CATEGORY = MANUAL_CUST`
4. In the *Export Customizing Transports* app, release the transport request. See Working in the Export Customizing Transports App.

   > **Note**
   >
   > When developing custom business configuration objects, you can decide if you want to provide a transport selection screen for change recording. If you are using this transport pattern, we recommend autofilling the transport request based on the software component of the business configuration object.

4.2.9.6 Information for Audit

The following information is relevant for audit:

- Git repository: The creation of a Git repository branch creates a clone of the parent branch (including all development objects in the branch)
  - During a pull or checkout, all objects that differ from the previously checked out commit and the pulled commit are imported into the ABAP system. Transport requests cannot be imported selectively into subsequent ABAP systems. Together with the information which branch and commit ID was pulled into an ABAP system, you can reproduce which version of an object was in an ABAP system at which point in time.
  - The content of the Git repository with all its commits and objects is not directly accessible to users of an ABAP system. Only SAP employees have access to it for support purposes because the information is typically not required for an audit. In exceptional cases, the content of the Git repository can be requested via a service request.
- In the Manage Software Components SAP Fiori app and in ABAP Development Tools (ADT), the following information is available for a regular audit.
  - Manage Software Components app:

- Action history of imports (pulls and branch checkouts), which also contains an execution log and a transport log per import. As these two logs are only saved temporarily in the ABAP system, we recommend to save them to an excel if they are needed for error analysis or audit.
- Information on the last commit (commit ID, commit message consisting of the transport request number and description, commit author and last commit timestamp) for each branch

- ADT
  - In the *Transport Organizer*, you can find the transport requests together with the piece list
  - In the *Revision History*, you can find the history of an object

4.2.9.7 Automate the Software Lifecycle Management Process

To facilitate your software lifecycle management process, you can automate it by using communication scenario `SAP_COM_0948` and `SAP_COM_0735`. See API for Managing Software Components [page 3252] and Running ABAP Unit Test Runs [page 923].

These communication scenarios contain several tools that can be used to craft a continuous integration process.

To make the setup of continuous integration pipelines as easy as possible, the open source project "Piper" was created. In general, there are various functions (Library steps) as well as whole pipelines available for reuse. You can find various steps that implement the functionality of the APIs of `SAP_COM_0948` and `SAP_COM_0735`.

The reusable continuous integration pipeline contains the following steps:

- Creating an SAP Business AI Platform, ABAP environment system
- Setting up a communication arrangement for the APIs
- Cloning a software component / Git repository
- Running ABAP Test Cockpit checks
- Running ABAP unit tests
- Deprovisioning the SAP Business AI Platform, ABAP environment system

> **Note**
>
> You can only dynamically create SAP Business AI Platform, ABAP environment systems during the pipeline execution if you are using a consumption-based model. See What Is the Consumption-Based Commercial Model?

Related Information

Continuous Testing on SAP BTP ABAP Environment

4.2.9.7.1 Running ABAP Unit Test Runs

Prerequisites

- You have an SAP Business AI Platform, ABAP environment system.
- You've created a *Communication User* as described in How to Create Communication Users [page 2995].
- You've created a *Communication System* as described in How to Create Communication Systems.
- You've created a *Communication Arrangement* as described in How to Create a Communication Arrangement [page 2998].
- You've selected the communication scenario **SAP_COM_0735** for your communication arrangement and have mapped it to your communication system.

Context

ABAP Unit is the standard tool for running unit tests. In this help topic, you'll learn how to start an ABAP Unit Test Run via REST Service.

For more information about ABAP Unit, see Ensuring Quality of ABAP Code.

Procedure

1. **Get CSRF token:** The first step serves for the authentication on the server. The response header contains a CSRF token, which is used as authentication for the POST request following in step 2.

   Request

   Authentication Type: Basic Authentication

   GET https://<host.com>:<port>/sap/bc/adt/api/abapunit/runs/00000000000000000000000000000000

   Headers

KEY	VALUE
accept	application/vnd.sap.adt.api.abapunit.run-status.v1+xml
x-csrf-token	fetch

   Response

   Headers

KEY	VALUE
x-csrf-token	<token>
location	/sap/bc/adt/api/abapunit/runs/ 00000000000000000000000000000000

2. **Start an ABAP Unit Run:** To start an ABAP Unit run, insert the **CSRF token** that was retrieved in the first request in the header parameters.

Request

POST https://<host.com>:<port>/sap/bc/adt/api/abapunit/runs

Headers

KEY	VALUE
x-csrf-token	<token>
content type	application/vnd.sap.adt.api.abapunit.run.v1+xml

Body

You can specify arbitrary object sets. See also Object Sets and XML Representations of Object Sets for more details.

You can provide additional parameters to control the selection of tests. You can restrict the amount of tests to certain risk level and durations. Furthermore, you can decide to run foreign tests, which are connected via test relations.

```
<?xml version="1.0" encoding="UTF-8"?>
<aunit:run title="My Run" context="AIE Integration Test" xmlns:aunit="http://
www.sap.com/adt/api/aunit">
  <aunit:options>
    <aunit:measurements type="none"/>
    <aunit:scope ownTests="true" foreignTests="true"/>
    <aunit:riskLevel harmless="true" dangerous="true" critical="true"/>
    <aunit:duration short="true" medium="true" long="true"/>
  </aunit:options>
  <osl:objectSet xsi:type="unionSet" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance" xmlns:osl="http://www.sap.com/api/osl">
   <osl:set xsi:type="osl:packageSet">
     <osl:package includeSubpackages="true" name="PACKAGE_ONE"/>
   </osl:set>
   <osl:set xsi:type="osl:flatObjectSet">
     <osl:object name="ZCL_TEST_ONE" type="CLAS"/>
     <osl:object name="ZIF_TEST_TWO" type="INTF"/>
   </osl:set>
  </osl:objectSet>
</aunit:run>
```

Response

Headers

KEY	VALUE
location	/sap/bc/adt/api/abapunit/runs/{runId}

3. **Tracking the Status of the Test Run:** To track the status of the test run, you can make a GET request using the URI contained in the location header.

   Request

   GET https://<host.com>:<port>/sap/bc/adt/api/abapunit/runs/{runId}

   Headers

KEY	VALUE
accept	application/vnd.sap.adt.api.abapunit.run-status.v1+xml

   Response

   While the tests are still running, the returned status will be "In Process".

   If all tests have been run, the status "Completed" is returned. You'll find the link /sap/bc/adt/api/atc/results/<UUID> at the bottom of the body. Use this link to retrieve the ABAP Unit test results in the next step.

   Body

```
<?xml version="1.0" encoding="UTF-8"?>
<aunit:run xmlns:aunit="http://www.sap.com/adt/api/aunit" title="MyTitle"
context="MyContext">
  <aunit:progress status="FINISHED" percentage="100" />
  <aunit:executedBy user="JOHNDOE" />
  <aunit:time started="2021-02-12T17:09:19Z" ended="2021-02-12T17:09:22Z" />
  <atom:link xmlns:atom="http://www.w3.org/2005/Atom"
href="/sap/bc/adt/api/abapunit/results/0123821392189313" rel="http://
www.sap.com/adt/relations/api/abapunit/run-result" type="application/
vnd.sap.adt.api.junit.run-result.v1+xml"/>
</aunit:run>
```

4. **Retrieve Results:** To get the ABAP Unit results, use the following request.

   Request

   GEThttps://<host.com>:<port>/sap/bc/adt/api/abapunit/results/{resultId}

   Headers

KEY	VALUE
Accept	application/vnd.sap.adt.api.junit.run-result.v1+xml

   Response

   The results are submitted in the JUnit format.

   Body

   As an example, the result may look like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<testsuites tests="3" asserts="2" skipped="1" errors="0" failures="1"
timestamp="2021-01-22T19:17:30Z" time="0.36" executedBy="JOHNDOE"
client="000" system="UIA">
```

```
    <testsuite tests="2" asserts="2" skipped="1" errors="0" failures="1"
timestamp="2021-01-22T19:17:30Z" time="0.28 " hostname="ldai3uia"
package="test_cars" name="">
        <testcase asserts="1" time="0.01 " name="order"
classname="fugr:zmg_cars.ltc_finder">
            <failure type="Assert Failure" message="Critical Assertion Error:
'Cds_View: ASSERT_EQUALS'">
            Character string different as of position 4Expected
[CL_AUNIT_TESTABLE_OBJECT======CP]Actual [CL_CAR=======================CP]
Test 'LTC_FINDER->ORDER' in Main Program 'SAPLZMG_CARS'
            Stack: Include: <LZMG_CARST99> Line: <38> (ORDER)
            </failure>
        </testcase>
        <testcase asserts="1" time="0.01 " name="preview"
classname="fugr:zmg_cars.ltc_finder">
            <skipped message="Missing Prerequisites - Dummy: ABORT">
            Test execution skipped due to missing prerequisites Test
'LTC_FINDER->PREVIEW' in Main Program 'SAPLZMG_CARS'
            Stack: Include: <LZMG_CARST99> Line: <44> (PREVIEW)
            </skipped>
        </testcase>
        <testcase asserts="0" time="0.01 " name="drive"
classname="fugr:zmg_cars.ltc_finder"/>
    </testsuite>
</testsuites>
```

4.2.9.7.2 Reading and Deploying ATC Configurations

Prerequisites

- You have an SAP Business AI Platform, ABAP environment system.
- You've created a *Communication User* and generated a **password** as described in How to Create Communication Users [page 2995].
- You've created a *Communication System* as described in How to Create Communication Systems.
- You've created a *Communication Arrangement* as described in How to Create a Communication Arrangement [page 2998].

  > **Note**
  >
  > The URL displayed for the inbound service of the communication arrangement will be needed for the GET requests described below.

- You have selected the communication scenario **SAP_COM_0763** for your communication arrangement and have mapped it to your communication system.

Context

The communication scenario **SAP_COM_0763** offers an OData V4 API for maintenance of ABAP Test Cockpit (ATC) configurations on the business objects `SATC_CI_CF_RT_E`, `SATC_CI_CF_E`, and `SATC_CI_CF_P_V`.

Therefore, you have to create a valid communication arrangement as described in the prerequisites before performing the actual test steps.

Procedure

1. **Get CSRF token:** The first step serves for the authentication on the server. Use the URL you've obtained from the *Communication Arrangement*'s inbound service for the **GET** request. The response header contains an CSRF token, which is used as authentication for the GET request following in step 2.

   Request

   Authentication Type: Basic Authentication. Enter the name of your *Communication User* (not the technical user ID) and the generated password.

   GET <URL_from_Communication_Arrangement>

   Headers

KEY	VALUE
x-csrf-token	fetch

   Response

   Headers

KEY	VALUE
x-csrf-token	<token>

2. **Read ATC configurations:** To read ATC configurations, prepare a GET request using the following parameters:

   Request

   Header

KEY	VALUE
x-csrf-token	<token>

   - All configurations:
     GET <URL_from_Communication_Arrangement>/configuration
   - By name:
     GET <URL_from_Communication_Arrangement>/configuration?$filter=conf_name eq '<CONFNAME>'
   - Default configuration:
     GET <URL_from_Communication_Arrangement/configuration?$filter=is_default eq true

   If you want to look deeper into configuration data (e.g. the priorities of the configuration), you can use expands by adding:

   - to all priorities: &$expand=_priorities

- filtered only on priorities changed from default_priority: &$expand=_priorities($filter=priority ne default_priority)

3. **Deploy ATC configurations:** To deploy an ATC configuration, prepare a POST request and use the body to provide a json (or XML) representing the configuration you want to deploy.

   Request

   Header

KEY	VALUE
content-type	application/json
x-csrf-token	<token>

   POST <URL_from_Communication_Arrangement>/configuration

   - Flat Insert of changed priorities:
     When you pass the json from the `get_configuration` with no expand by creating a valid configuration, the default priorities will also be created.
   - Deep Insert of changed priorities:
     You can pass the json from the `get_configuration` with expand to filtered priorities and potentially change the `confname` if desired.

4.2.9.7.3 Running ATC Check Runs

Prerequisites

- You have an SAP Business AI Platform, ABAP environment system.
- You've created a *Communication User* as described in How to Create Communication Users [page 2995].
- You've created a *Communication System* as described in How to Create Communication Systems.
- You've created a *Communication Arrangement* as described in How to Create a Communication Arrangement [page 2998].
- You've selected the communication scenario **SAP_COM_0901** for your communication arrangement and have mapped it to your communication system.

  > ⚠ Restriction
  >
  > The communication scenario **SAP_COM_0510** is deprecated and should no longer be used.

- You've defined a business user in the additional properties in the communication arrangement. To do so, choose the *Additional Properties* link for ATC Check Run in the *Inbound Services* section. Specify a business user (CB) or select a user from the value help. ATC runs started via the service will be scheduled and run with the specified business user.

> ### Note
>
> The assigned business user needs development authorization for ADT, which is contained in the business catalog `SAP_A4C_BC_DEV_ATC_RUN_PC`. In case the ATC is used to run ABAP Unit Tests, further specific authorizations might be required.
>
> When you assign the catalog to a dedicated role, do not forget to remove the restriction to read-only access before saving the role.

Context

The ABAP Test Cockpit (ATC) is the standard tool for checking the quality of ABAP development objects using static checks and ABAP unit tests. In this help topic, you'll learn how to start an ATC Check Run via REST Service.

For more information about the ABAP Test Cockpit, see Checking Quality of ABAP Code with ATC.

Procedure

1. **(Get CSRF token)** The first step serves for the authentication on the server. The response header contains an CSRF token, which is used as authentication for the POST request following in step 2.

   Request

   Authentication Type: Basic Authentication

   GET https://<host.com>:<port>/sap/bc/adt/api/atc/runs/00000000000000000000000000000000

   Headers

KEY	VALUE
accept	application/vnd.sap.atc.run.v1+xml
x-csrf-token	fetch

   Response

   Body

   ```
   <?xml version="1.0" encoding="utf-8"?>
   <atc:run status="Not Created" xmlns:atc="http://www.sap.com/adt/atc">
       <atc:progress description="No run was created, no objects had to be
   checked"/>
       <atc:phases/>
   </atc:run>
   ```

   Headers

KEY	VALUE
x-csrf-token	<token>
location	/sap/bc/adt/atc/runs/ 00000000000000000000000000000000

2. **(Start an ATC Check Run)** To start an ATC check run, insert the **CSRF token** that was retrieved in the first request in the header parameters.

   Request

   POST https://<host.com>:<port>/sap/bc/adt/api/atc/runs?clientWait=false

   Headers

KEY	VALUE
x-csrf-token	<token>
content type	application/vnd.sap.atc.run.parameters.v1+xml

   Body

   You can specify components, packages or both. If you specify both components and packages, only those objects will be checked that are assigned to packages that belong to the specified software components.

   For the package value, enter the name of the package you want to have checked.

   You can provide an ATC configuration as additional parameter that is used instead of the default configuration of your system.

   You can also provide an ATC check variant as additional parameter. This is used instead of the default check variant of your ATC configuration, regardless of whether an ATC configuration was explicitly provided via parameter or not.

   You can specify arbitrary object sets. See also Object Sets and XML Representations of Object Sets for more details.

```
<?xml version="1.0" encoding="UTF-8"?>
<atc:runparameters xmlns:atc="http://www.sap.com/adt/atc"
         xmlns:obj="http://www.sap.com/adt/objectset"
         checkVariant="CHECKVARIANT_NAME"
         objectProvider="OBJECTPROVIDER_NAME"
         configuration="CONFIGURATION_NAME">
    <osl:objectSet xsi:type="unionSet" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance" xmlns:osl="http://www.sap.com/api/osl">
      <osl:set xsi:type="osl:multiPropertySet">
        <osl:property key="softwareComponent" value="Z_MY_COMPONENT"/>
      </osl:set>
      <osl:set xsi:type="osl:packageSet">
        <osl:package includeSubpackages="false" name="Z_MY_PACKAGE"/>
      </osl:set>
    </osl:objectSet>
</atc:runparameters>
```

> ### Note
>
> If you want to use the ATC REST service in BTP for an object provider, the function module `OSL_GET_REMOTE_OBJECTS` needs to be exposed in the cloud connector in addition to following SAP note 2861842.

Response

Headers

KEY	VALUE
location	/sap/bc/adt/api/atc/runs/<UUID>

3. **(Tracking the Status of the Check Run)** To track the status of the check run, you can make a GET request using the URI contained in the location header.

   Request

   GET https://<host.com>:<port>/sap/bc/adt/api/atc/runs/<UUID>

   Headers

KEY	VALUE
accept	application/vnd.sap.atc.run.v1+xml

   Response

   While the check is still running, the ATC phases will show the status "In Process" or "Undefined" in the body.

   The check run is finished when all ATC phases show the status "Completed". You'll find the link /sap/bc/adt/api/atc/results/<UUID> at the bottom of the body. Use this link to retrieve the ATC check run results in the next step.

   Body

   ```
   <?xml version="1.0" encoding="utf-8"?>
   <atc:run status="Completed" xmlns:atc="http://www.sap.com/adt/atc">
       <atc:progress description="Run Completed"/>
       <atc:phases>
           <atc:phase title="Determine Object Keys" status="Completed"
   number="1"/>
           ...
       </atc:phases>
       <atom:link href="/sap/bc/adt/api/atc/results/<UUID>" rel="http://
   www.sap.com/abap/checks/atc/relations/results/displayid" type="application/
   xml" title="Result" xmlns:atom="http://www.w3.org/2005/Atom"/>
   </atc:run>
   ```

4. **(Retrieve Results)** To get the ATC results, use the following request.

   Request

   GET /sap/bc/adt/api/atc/results/<UUID>

   Headers

KEY	VALUE
Accept	application/vnd.sap.atc.checkstyle.v1+xml

Response

The results are submitted in the checkstyle format. You can find the checkstyle XSD here: https://github.com/linkedin/pygradle/blob/master/pygradle-plugin/src/test/resources/checkstyle/checkstyle.xsd .

Body

As an example, the result may look like this:

```
<?xml version="1.0" encoding="utf-8"?>
<checkstyle version="1.0">
    <file name="PACKAGE/OBJECT_NAME.OBJECT_TYPE">
    <error message="Hard-coded user name (@User's formatted name)"
source="CL_CI_TEST_EXTENDED_CHECK_SEC#0821" line="2" severity="error"/>
</checkstyle>
```

In this example, the error message shows a hard-coded username. Ideally, the formatted user name corresponds to the developer that edited the object for the last time. In case "last changed by" shouldn't be available for the object, the author is used instead. In case the formatted user name can't be determined, the business user id is displayed as fallback.

> **Note**
>
> In the output, the namespace slashes will be replaced by dashes. That means `/NAMESPACE/OBJECT_NAME` will be output as `-NAMESPACE-OBJECT_NAME`.

4.2.9.7.4 Reading ATC Results

Prerequisites

- You have an SAP Business AI Platform, ABAP environment system.
- You've created a *Communication User* and generated a **password** as described in How to Create Communication Users [page 2995].
- You've created a *Communication System* as described in How to Create Communication Systems.
- You've created a *Communication Arrangement* as described in How to Create a Communication Arrangement [page 2998].

> **Note**
>
> The URL displayed for the inbound service of the communication arrangement will be needed for the GET requests described below.

- You have selected the communication scenario **SAP_COM_0A59** for your communication arrangement and have mapped it to your communication system.

Context

The communication scenario `SAP_COM_0A59` provides an OData V4 API for reading ABAP Test Cockpit (ATC) results, including their findings, potential run failures, and descriptions. It also offers navigation information for findings.

Procedure

1. **Get CSRF token:** The first step serves for the authentication on the server. Use the URL you've obtained from the *Communication Arrangement*'s inbound service for the **GET** request. The response header contains an CSRF token, which is used as authentication for the GET request following in step 2.

   Request

   Authentication Type: Basic Authentication. Enter the name of your *Communication User* (not the technical user ID) and the generated password.

   GET <URL_from_Communication_Arrangement>

   Headers

KEY	VALUE
x-csrf-token	fetch

   Response

   Headers

KEY	VALUE
x-csrf-token	<token>

2. **Read ATC results:** To read ATC results, prepare a GET request using the following parameters:

   Request

   Header

KEY	VALUE
x-csrf-token	<token>

   - ATC Findings:
     GET <URL_from_Communication_Arrangement>/SATC_API_FINDINGS_C
   - ATC Item Description Data:

GET <URL_from_Communication_Arrangement>/SATC_API_RAW_DESCR_C

- ATC Item Navigation Data:
  GET <URL_from_Communication_Arrangement>/SATC_API_RAW_NAV_C
- ATC Result Headers:
  GET <URL_from_Communication_Arrangement>/SATC_API_RESULT_HEADERS_C
- ATC Run Failures:
  GET <URL_from_Communication_Arrangement>/SATC_API_RUN_FAILURES_C

4.2.9.7.5 Configuring Remote ATC Using a Central Check System

You can perform remote code analysis in ATC, allowing you to analyze custom code remotely with the latest checks.

When using SAP BTP ABAP environment as the central check system, you will use the **Analyze Custom Code/ Custom Code Migration** app to scope and analyze custom code in an on-premise SAP system.

To use this SAP Fiori app, your SAP administrator must enable the app to provide access for the relevant users. In addition, you also need to establish an RFC connection for the Cloud Connector. This enables you to access the on-premise system as the checked system from the ABAP environment as central check system remotely.

Related Information

Configuring the Checked System [page 934]
Configuring and Administrating the Cloud Connector [page 938]
Maintaining Communication Arrangements [page 942]
Configuring the ATC Developer Scenario [page 943]
Custom Code Analysis

4.2.9.7.5.1 Configuring the Checked System

You need to configure an RFC destination from your on-premise system (checked system) to the SAP BTP ABAP environment as central check system. This enables you to run ATC checks in the central check system for your custom code in the checked cystem.

Related Information

Configuring the User [page 935]

Installing Remote Stubs [page 937]

4.2.9.7.5.1.1 Configuring the User

You can either apply SAP Note 3548772 in the **checked system** to get all relevant authorizations for your RFC user or, alternatively, check the table below.

Authorization Object Value	Parameter	Value
S_RFC	ACTVT	16 (Execute)
	RFC_TYPE	FUGR
	RFC_NAME	SCA_REMOTE_DATA_ACCESS
		SABP_COMP_PROCS_E
		SYCM_APS_REMOTE
		SYST
		S_CODE_INSPECTOR_TESTS
		SUSG_API
S_RFC	ACTVT	16 (Execute)
	RFC_TYPE	FUNC

Authorization Object Value	Parameter	Value
	RFC_NAME	FUNCTION_EXISTS REPOSITORY_ENVIRONMENT_ALL RFC_GET_NAMETAB SVRS_GET_VERSION_DIRECTORY_46 RFCPING SLINRMT_RUN TRINT_PROGRESS_INDICATOR TRINT_TP_UPDATE_TPSTAT SLDAG_GET_SYSTEM_NAME CVA_BSP_RUN IDOCTYPE_COLLECT_ATC_INFOS IQAPI_QUERY_CONTENT IQAPI_INFOSET_CONTENT IQAPI_CHECK_INFOSET FM_CCM_DYNPRO_READ SCI_GET_DDLS
S_DEVELOP	ACTVT	03 (Display) **Note** To launch **ABAP unit tests**, you need the following authorizations: • S_DEVELOP with ACTVT = 'EU' (Automated Tests) or ACTVT = '16' (Execute). • If a language version check is active in your system, you also need: S_ABPLNGVS with ABP_LNG_VS = '5' and ACTVT = 'EU' or ACTVT = '16'.
	DEVCLASS	*
	OBJNAME	*
	OBJTYPE	*
	P_GROUP	*
S_SYS_RWBO	ACTVT	01 (Create or generate) 02 (Change) 03 (Display)

Authorization Object Value	Parameter	Value
	DOMAIN	*
	DESTSYS	*
	TTYPE	TRAN
S_TRANSPRT	ACTVT	01 (Create or generate) 02 (Change) 03 (Display)
	TTYPE	TRAN
S_DATASET	ACTVT	34 (Write)
	FILENAME	*
	PROGRAM	SAPLSABC SAPLSTRF
S_ADMI_FCD	S_ADMI_FCD	SCMD (Write)

4.2.9.7.5.1.2 Installing Remote Stubs

During ATC execution, the central check system accesses the systems in your landscape remotely through so-called remote stubs using RFC connection. Thus, remote stubs serve as an interface between the central check system and on-premise systems; they return a model of custom code that needs to be checked.

Prerequisites

In the on-premise system, implement SAP Note 2599695 (Custom Code Migration Fiori App: Remote Stubs for the Checked System) in advance.

Context

You want to analyze your custom code in your on-premise system using the `Custom Code Migration` app.

Procedure

1. In the on-premise system, perform transaction `SNOTE`.
2. Download SAP Note 2599695.

   The *Note Assistant: Note Download* dialog is opened.
3. To implement the remote stubs, select and confirm the displayed SAP Note.
4. It is also recommended to apply SAP Notes in checked systems. Update your check system by following these steps:
   a. Perform transaction `SNOTE`.
   b. In the menu bar, select *Goto* > *Other tools* > *Launch Note Analyzer*.
   c. Upload the file *SAPNote_2436688_Checked_System.xml* available as attachment in SAP Note 2436688. You can now process the notes using SAP Note Analyzer.

> **Note**
>
> In case you don't want to use the Note Analyzer, please also apply the following SAP Notes:
>
> - 2485231 - Remote ATC Checks of Modifications and Enhancements
> - 2270689 - RFC Extractor for performing static checks
> - 2190065 - ATC/CI: Remote Code Analysis - Object Provider Stub

Results

The `Custom Code Migration` app now has access to the on-premise system. You can now start the custom code analysis.

4.2.9.7.5.2 Configuring and Administrating the Cloud Connector

The *Cloud Connector* enables the connection between the SAP Business AI Platform, ABAP environment and on-premise systems in a secured network.

4.2.9.7.5.2.1 Connecting the Cloud Foundry Environment Subaccount to the Cloud Connector

Prerequisites

- You have created a subaccount for the Cloud Foundry environment.
- You have installed the Cloud Connector.

Context

You want to add and connect your subaccount for the Cloud Foundry environment to the Cloud Connector.

For more information, see

- Managing Subaccounts
- Cloud Connector

Procedure

1. Go to your *Cloud Connector* and select the *Connector* navigation pane.
2. Choose the *Add Subaccount* button in the title toolbar.
3. In the *Add Subaccount* wizard, enter the *Region* of your server. Make sure that you choose the exact same region as in your Cloud Foundry subaccount.
4. Enter the ID of the *Subaccount* that you have created for the Cloud Foundry environment in the *SAP BTP Cockpit*. You will find the subaccount ID under *SAP BTP Cockpit* > *your Global Account* > *your Cloud Foundry Subaccount* > *Subaccount Details*.
5. *Cloud Connector*, enter the [Optional:] To label the subaccount in the *Display Name*.
6. Enter the login email address of the *Subaccount User*.
7. Enter the *Password* of the subaccount user.
8. [Optional: To overwrite the default location ID of the data center in the Cloud Foundry environment, enter the relevant *Location ID*.
9. [Optional:] Provide a *Description* for further details.
10. To add your subaccount to the *Cloud Connector*, choose *Save*.

Results

A new entry for the Cloud Foundry subaccount is added in the *Subaccount Dashboard* table in the Cloud Connector.

4.2.9.7.5.2.2 Configuring the Access Control (RFC) for the Remote Connection

The resource accessible represents the on-premise system that can be accessed through the *Cloud Connector*.

Context

You want to enable the connection for SAP Business AI Platform, ABAP environment to an on-premise system.

For more information, see Configure Access Control (RFC)

Procedure

1. In the *Cloud Connector*, select the *[Display Name of the Cloud Foundry Subaccount]* > *Cloud to On-Premise* navigation pane.
2. To add a new system mapping, choose + from the section toolbar of the *Mapping Virtual To Internal System* table.
3. In the *Add System Mapping* wizard, provide the following data:
   a. Choose *ABAP System* from the *Back-end Type* drop-down list box.
   b. Choose *Next*.
   c. Choose *RFC* as *Protocol* from the drop-down listbox.
   d. Choose *Next*.
4. > ⓘ Note
   >
   > To use a connection **with** load balancing, skip this step and proceed with the next step.

   To use a connection **without** load balancing, proceed as follows:
   a. Choose the *Without load balancing (application server and instance number)* radio button.
   b. Choose *Next*.
   c. Enter the following connection details of the on-premise system:

      > ⓘ Note
      >
      > You will find these details in the SAP Logon of your On-Premise system.

- *Application Server*.
- *Instance Number* of your Application Server ABAP.
- Name of the *SAProuter*, if applicable.

d. Enter a virtual name for the *Application Server*.
e. Enter a virtual *Instance Number*.
f. Choose *Next*.
g. Optional: Enter a description.
h. Choose *Next*.

5. > **Note**
   >
   > If you use a connection **without** load balancing, skip this step. Follow the instructions in the previous step instead.

   To use a connection **with** load balancing, proceed as follows:

   a. Choose the *With load balancing (system ID and message server)* radio button.
   b. Choose *Next*.
   c. Enter the following connection details of the on-premise system:

      > **Note**
      >
      > You will find these details in the SAP Logon of your On-Premise system.

      - *Message Server*.
      - *System ID*.
      - Name of the *SAProuter*, if applicable.

   d. Choose *Next*.
   e. Define and enter a virtual name for the *Message Server*.
   f. Define and enter a virtual *System ID*.
   g. Optional: Enter a description.
   h. Choose *Next*.

6. Optional: You can mark the checkbox *Check Internal Host* to immediately check the connection during creation.
7. To trigger creation, choose *Finish*.

   You have added the relevant on-premise system as resource accessible.

   Now, you want to define the permitted function modules as resources for a specific back-end system.

   To do this, proceed as follows:

8. In the *Cloud To On-Premise* navigation pane, select the relevant system mapping in the *Mapping Virtual to Internal System* section.
9. To define which function modules are executable in the on-premise, choose ↥ *Import resources belonging to a scenario* from the section toolbar in the *Resources Of ...* section.

   > **Note**
   >
   > To get the relevant import file for the Custom Code Migration scenario, see SAP Note 2861842 - Custom Code Migration in SAP Business AI Platform, ABAP environment: Set up SAP Cloud Connector.

10. *Browse* the *Scenario File* and choose *Select*.

Results

You have defined the on-premise system(s) that are accessible from the ABAP environment.

Now, you can call the defined function modules in the on-premise system.

4.2.9.7.5.3 Maintaining Communication Arrangements

A communication arrangement describes a communication scenario with a remote system during configuration time. It provides the required metadata for the service configuration.

Prerequisites

To create these communication arrangements, you need the `Administrator` role.

You have created a destination to your on-premise system in the *Cloud Connector* that is connected to your SAP BTP subaccount.

Context

You want to enable communication from your ABAP environment to your on-premise systems using Remote Function Calls (RFC).

To do this, you need to use the communication arrangement `SAP_COM_0464` (SAP Custom Code Migration Integration).

Procedure

1. Log on to the *SAP Fiori launchpad* of your ABAP environment.
2. In the *Communication Management* section, select the *Communication Arrangements* tile.
3. Choose *New*.
4. Use the value help to choose the `SAP_COM_0464` scenario.
5. Define and enter an *Arrangement Name*.

   If not yet available or defined, create a new communication system for the communication arrangement to define an endpoint for your checked system.

a. For the *Communication System*, choose *New*.
b. In the *New Communication System* dialog, enter the *System ID* and *System Name* of the checked system.
c. Choose *Create*.
d. In the *Technical Data* tab under *General*, enter the virtual host as specified in your *Cloud Connector* as *Host Name*. The field *Port* has already been filled in automatically with the default *443*.
e. Turn on the slider for *Cloud Connector*.
f. Filling in the field *SCC Location ID* is optional.
g. Under *RFC Settings*, fill in the fields *Client*, *Instance Number* and *Target Host* as specified as virtual host in your *Cloud Connector*.
h. Under *User for Outbound Communication*, choose + to assign the RFC user you created in the checked system to the communication system.
i. Choose *Create* and then *Save* to save the communication system and to be navigated back to the Communication Arrangement creation page.

6. Under *Additional Properties*, fill out the fields *Object Provider* and *System Group* (both freely selectable). This is the name you then select as your *Object Provider to Remote System* for your custom code migration project in the `Analyze Custom Code` app.
7. Under *Outbound Services* > *Retrieve Custom Code*, ensure that the *Service Status* is set to *Active*.
8. Choose *Save* to save the communication arrangement. A message should now pop up at the bottom of the screen informing you that the activation was successful.

Results

You can now select the communication arrangement in the *Object Provider to Remote System* field in the `Analyze Custom Code` app to establish the connection to your on-premise system.

4.2.9.7.5.4 Configuring the ATC Developer Scenario

You can use a system in SAP BTP ABAP environment as a central check system to run ATC checks from an on-premise system against this system (ATC Developer Scenario).

Prerequisites

- You've defined communication arrangement `SAP_COM_0464` (SAP Custom Code Migration Integration) to enable communication from your ABAP environment to your on-premise systems using remote function calls (RFC). For more information, see Maintaining Communication Arrangements [page 942].
- You've implemented SAP Note 3358660 - Developer Scenario Cloud in your checked system.

Context

To enable the ATC Developer Scenario, you've to define communication arrangement `SAP_COM_0936` for your cloud system.

Procedure

1. Log on to the *SAP Fiori launchpad* of your ABAP environment.
2. In the *Communication Management* section, select the *Communication Arrangements* tile.
3. Choose *New*.
4. Use the value help to choose the `SAP_COM_0936` scenario.
5. Define and enter an *Arrangement Name*.
6. If not yet available or defined, create a new communication system for the communication arrangement to define an endpoint for your checked system.
   a. For the *Communication System*, choose *New*.
   b. In the *New Communication System* dialog, enter the *System ID* and *System Name* of the checked system.
   c. Choose *Create*.
   d. In the *Technical Data* tab under *General*, select the *Inbound Only* check box.

      Be aware that you should only select the *Inbound Only* check box if you are using a new communication system. Please do not check this box when you are re-using a communication scenario.
   e. In the *Users for Inbound Communication* tab, choose + to add a new inbound communication user.
   f. Choose *User Name and Password* as *Authentication Method*, and enter a user name, description, and password for the user.
   g. Choose *Create*.
7. In the Communication Arrangement in the *Inbound Communication* tab, enter the *User Name* you've previously defined as described in step 6.
8. In the *Additional Properties* tab, select the *Object Provider* you've defined in `SAP_COM_0464`.
9. Choose *Save*.
10. In the *Code Inspector* section, enter the *Reference Check System*. This is the RFC connection to your cloud system. If not yet available or defined, create a new RFC destination in transaction `SM59`. To do this, follow these steps (see also Maintaining Remote Destinations):

    > **Note**
    >
    > In order to setup your RFC connection, you first need to open a **service channel** (on-premise to cloud connection) in your respective Cloud Connector. See Configure a Service Channel for RFC to learn how to create one.

    a. Choose connection type "3" (ABAP Connection) or connection type "W" (WebSocket RFC) to connect to your cloud system via Cloud Connector.
    b. In the *Technical Settings* tab, enter the host of your Cloud Connector as *Target Host*. As instance number, choose the instance number that you have defined for your service channel.

c. In the *Logon & Security* tab, enter the correct client of your cloud system. The user to be entered here is the inbound communication user that you have assigned to communication scenario `SAP_COM_0936`. Here you need to enter the **alias name** of this user. However, be aware that it cannot exceed 12 characters.
d. As an optional step, you can run a connection test, which should be successful.
e. Save your destination.

11. In your checked system, run transaction `ATC`.

    The *ABAP Test Cockpit* is opened.

12. You can now proceed and maintain your destination in *ATC* > *Basic Settings* as *Reference Check System*.
13. Choose *ATC Administration* > *Setup* > *Basic Settings*.
14. Go to transaction `SCI` and create a proxy variant (as described in step 2 in Configuring the Checked System).

    You can reference one of the SAP delivered ATC variants in your cloud system, for example`SLIN_SEC`, or one of your own variants that you've created in the cloud system. Go to Creating ATC Check Variants for guidance on how to create ATC check variants.

15. Go back to transaction `ATC`. In the *Basic Settings*, enter the name of this proxy variant as *Global Check Variant*.
16. Confirm.

Results

You've enabled a cloud system as the central check system for remote ATC checks from an on-premise system against the cloud system. In SAP GUI or ABAP development tools for Eclipse, you can now run ATC checks in your checked system. In ABAP development tools for Eclipse in the *ATC Problems* view, you can create exemptions that you can then manage in the Approve ATC Exemptions app.

4.2.9.7.6 (Deprecated) Test Integration (SAP_COM_0510)

Communication scenario `SAP_COM_0510` enables you to create continuous integration pipelines for SAP Business AI Platform, ABAP environment systems.

You can use this communication scenario to create continuous integration processes. SeeAutomate the Software Lifecycle Management Process.

> **Note**
>
> Please be aware that this communication scenario is deprecated. From now on, please refer to the new communication scenario API for Managing Git repositories (0948).

4.2.9.7.6.1 API to Manage Git Repositories

You can use the `MANAGE_GIT_REPOSITORY` API to pull or clone Git repositories to ABAP environment systems and to create and checkout branches. The `MANAGE_GIT_REPOSITORY` API belongs to the Communication Scenario SAP_COM_0510.

> ⓘ Note
>
> In ABAP environment, Git repositories are wrapped in software components. These are currently managed in the **Manage Software Components** app. The technical parameter `sc_name` used in this API needs to be the name of the Git repository on the ABAP environment system – the name of the software component.

> ⓘ Note
>
> It is currently not possible to execute/trigger multiple actions (clones/pulls/deletes) in parallel on one system, regardless of the fact that the calls run asynchronously.

Related Information

Cloning Git Repositories to an ABAP Environment System [page 946]
Pulling Git Repositories to an ABAP Environment System [page 948]
Working with Branches on a Git Repository [page 952]

4.2.9.7.6.1.1 Cloning Git Repositories to an ABAP Environment System

Prerequisites

- You have an ABAP environment system.
- You've created a *Communication User* as described in How to Create Communication Users [page 2995].
- You've created a *Communication System* as described in How to Create Communication Systems.
- You've created a *Communication Arrangement* as described in How to Create a Communication Arrangement [page 2998].
- You have selected the communication scenario **SAP_COM_0510** for your communication arrangement and have mapped it to your communication system.

Context

If a software component is not in your system yet, you will first need to clone it once to import it into your system. Please note that you can clone a software component only once into your system.

> **Note**
>
> In ABAP environment, Git repositories are wrapped in software components. These are currently managed in the *Manage Software Components* app. The parameter `sc_name` passed to this API needs to be the name of the Git repository on the ABAP environment system – the name of the software component.

Procedure

1. **(Authentication on the server)**The first step serves the authentication on the server. The response header contains an x-csrf-token, which is used as authentication for the POST request following in *step 2*.

   Request

   ```
   GET/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY HTTP/1.1
   Host: host.com
   Authentication: basicAuthentication
   X-csrf-token: fetch
   ```

   Response

   ```
   HTTP/1.1 200 OK
   X-csrf-token: xCsrfToken
   ```

2. **(Cloning a Git repository)** To trigger the clone, you need to insert the x-csrf-token that was retrieved in the first request in the header parameters. The Git repository and the branch you want to clone are passed in the body of the request. If you don't enter a branch name, the main branch is automatically selected. Alternatively you can clone a specific tag by entering the tag name. Make sure that the tag exists on the branch. If that's not the case, no clone is executed.

   Request

   ```
   POST/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Clones HTTP/1.1
   Host: host.com
   Authentication: basicAuthentication
   X-csrf-token: xCsrfToken
   Content-Type: application/json
   Accept: application/json

   {
           "sc_name" : "/DMO/GIT_REPOSITORY",
           "branch_name": "main",
           "tag_name": "my-release-tag"
   }
   ```

3. **(View all clones)**To view all clones that have been made, you can make a GET request.

   Request

   ```
   GET /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Clones HTTP/1.1
   Host: host.com
   ```

```
Authentication: basicAuthentication
Accept: application/json
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
    "d": {
        "results": [
            {
                "__metadata": {
                    "id":
"host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Clones(uuid=guid'02ceab7d-
ff04-1eda-b1ea-eb7fdbf28bc4',sc_name='ZHM_INITIAL_TEST',branch_name='')",
                    "uri":
"host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Clones(uuid=guid'02ceab7d-
ff04-1eda-b1ea-eb7fdbf28bc4',sc_name='ZHM_INITIAL_TEST',branch_name='')",
                    "type": "cds_sd_a4c_a2g_gha_sc_web_api.ClonesType"
                },
                "uuid": "02ceab7d-ff04-1eda-b1ea-eb7fdbf28bc4",
                "sc_name": "/DMO/GIT_REPOSITORY",
                "branch_name": "",
                "import_type": "Clone",
                "namespace": "",
                "status": "S",
                "status_descr": "Success",
                "criticality": 3,
                "user_name": "administrator@example.com",
                "start_time": "/Date(1594898863000+0000)/",
                "change_time": "/Date(1594898891000+0000)/"
            },
               // all clones for this system instance will be listed here …
        ]
    }
}
```

4.2.9.7.6.1.2 Pulling Git Repositories to an ABAP Environment System

Prerequisites

- You have an ABAP environment system.
- You've created a *Communication User* as described in How to Create Communication Users [page 2995].
- You've created a *Communication System* as described in How to Create Communication Systems.
- You've created a *Communication Arrangement* as described in How to Create a Communication Arrangement [page 2998].
- You have selected the communication scenario **SAP_COM_0510** for your communication arrangement and have mapped it to your communication system.
- You have cloned the Git repository to the ABAP Environment system. See Cloning Git Repositories to an ABAP Environment System [page 946].

Context

You can use the communication scenario *SAP_COM_0510* to pull Git repositories to an ABAP environment system.

> **Note**
>
> In ABAP environment, Git repositories are wrapped in *Software Components*. These are currently managed in the App *Manage Software Components*. The parameter passed to this API needs to be the name of the Git repository on the ABAP environment system – the name of the Software Component.

Procedure

1. **(Authentication on the Server)**The first step serves the authentication on the server. The response header contains an x-csrf-token, which is used as authentication for the POST request following in *step 2*.

   Request

   ```
   GET/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY HTTP/1.1
   Host: host.com
   Authentication: basicAuthentication
   X-csrf-token: fetch
   ```

   Response

   ```
   HTTP/1.1 200 OK
   X-csrf-token: xCsrfToken
   ```

2. **(Pull a Git Repository)** To trigger the pull of a Git repository, you have to insert the *x-csrf-token* that was retrieved in the first request in the header parameters. The Git repository you want to pull is passed in the body of the request.

   Request

   ```
   POST/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Pull HTTP/1.1
   Host: host.com
   Authentication: basicAuthentication
   X-csrf-token: xCsrfToken
   Content-Type: application/json
   Accept: application/json
   {
       “sc_name” : “/DMO/GIT_REPOSITORY”
   }
   ```

   Response

   ```
   HTTP/1.1 200 OK
   Content-Type: application/json
   ```

   ```
   {
       "d": {
           "__metadata": {
               "id": "https://host.com/sap/opu/odata/sap/
   MANAGE_GIT_REPOSITORY/Pull(guid’UUID’)", "uri": "https://host.com/sap/opu/
   ```

```
odata/sap/MANAGE_GIT_REPOSITORY/Pull(guid'UUID')", "type":
"cds_sd_a4c_a2g_gha_sc_web_api.PullType"
        }
        ,
        "uuid": "UUID ",
        "sc_name": "/DMO/GIT_REPOSITORY ",
        "namespace": "",
        "status": "R",
        "status_descr": "Running",
        "start_time": "/Date(1571227437000+0000)/",
        "change_time": "/Date(1571227472000+0000)/",
        "criticality": 2,
        "user_name": "CC0000000001",
        "to_Execution_log": {
            "__deferred": {
                "uri": "https://host.com/sap/opu/odata/sap/
MANAGE_GIT_REPOSITORY/Pull(guid'UUID')/to_Execution_log"
            }
        }
        ,
        "to_Transport_log": {
            "__deferred": {
                "uri": "https://host.com/sap/opu/odata/sap/
MANAGE_GIT_REPOSITORY/Pull(guid'UUID')/to_Transport_log"
            }
        }
    }
}
```

3. **(Tracking the Status of the Pull)** To track the status of the pull, you can make a GET request using the uuid contained in the response. You can also read the URI directly from the "__metadata" of the response.

   Request

   ```
   GET/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Pull(guid'UUID') HTTP/1.1
   Host: host.com
   Authentication: basicAuthentication
   Accept: application/json
   ```

   The response contains the same entity as the second request. The Pull is successful, when the "status" in the response body has the value **S**. The status description "status_descr" will then return **Successful**. In case of an error "status" will have the value **E** and "status_descr" the value **Error**.

4. **(Retrieving Logs)** To get the Execution Log and the Transport Log after the Pull is finished, you can use the following requests. Alternatively, you can use the URIs from the response of the POST request. You can also check both logs in the *Manage Software Components* app for better readability.

   For the Execution Log:

   Request

   ```
   GET /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Pull(guid'UUID')/
   to_Execution_log HTTP/1.1
   Host: host.com
   Authentication: basicAuthentication
   Accept: application/json
   ```

   Response

   ```
   HTTP/1.1 200 OK
   Content-Type: application/json
   ```

   ```
   {
   ```

```
    "d": {
        "results": [ {
            "__metadata": {
                "id": "host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
ExecutionLogs(index_no=1m,uuid=guid'UUID')", "uri": "host.com/sap/opu/
odata/sap/MANAGE_GIT_REPOSITORY/ExecutionLogs(index_no=1m,uuid=guid'UUID')",
"type": "cds_sd_a4c_a2g_gha_sc_web_api.ExecutionLogsType"
            }
            ,
            "index_no": "1",
            "uuid": "UUID",
            "type": "Information",
            "descr": "Step X: MESSAGE",
            "timestamp": "/Date(1571227438000+0000)/",
            "criticality": 0,
        }
        ]
    }
}
```

For the Transport Log:

Request

```
GET /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Pull(guid'UUID')/
to_Transport_log HTTP/1.1
Host: host.com
Authentication: basicAuthentication
Accept: application/json
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
    "d": {
        "results": [ {
            "__metadata": {
                "id": "host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
ExecutionLogs(index_no=1m,uuid=guid'UUID')", "uri": "host.com/sap/opu/
odata/sap/MANAGE_GIT_REPOSITORY/ExecutionLogs(index_no=1m,uuid=guid'UUID')",
"type": "cds_sd_a4c_a2g_gha_sc_web_api.ExecutionLogsType"
            }
            ,
            "index_no": "1",
            "uuid": "UUID",
            "type": "Information",
            "descr": "Step X: Message",
            "timestamp": "/Date(1571227438000+0000)/",
            "criticality": 0,
        }
        ]
    }
}
```

4.2.9.7.6.1.3 Working with Branches on a Git Repository

Prerequisites

- You have an ABAP environment system.
- You've created a *Communication User* as described in How to Create Communication Users [page 2995].
- You've created a *Communication System* as described in How to Create Communication Systems.
- You've created a *Communication Arrangement* as described in How to Create a Communication Arrangement [page 2998].
- You have selected the communication scenario **SAP_COM_0510** for your communication arrangement and have mapped it to your communication system.
- You have a Git repository/ software component with a main branch. A main branch is created when you push content to the repository for the first time, i.e. you release a transport request for the repository/ software component.

Context

You can use the communication scenario `SAP_COM_0510` to work with Git repositories on an SAP Business AI Platform, ABAP environment system.

> **Note**
>
> In ABAP Environment, Git repositories are wrapped in software components. These are currently managed in the *Manage Software Components* app. The technical parameter `sc_name` used in this API needs to be the name of the Git repository on the ABAP Environment system – the name of the software component.

Procedure

1. **(Authentication on the Server)** The first step serves the authentication on the server. The response header contains an x-csrf-token, which is used as authentication for the `POST` request following in *step 2*.

   Request

   ```
   GET/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY HTTP/1.1
   Host: host.com
   Authentication: basicAuthentication
   X-csrf-token: fetch
   ```

   Response

   ```
   HTTP/1.1 200 OK
   X-csrf-token: xCsrfToken
   ```

2. **(Clone a Git Repository)** If a software component is not yet in your system, you first need to clone it once to import it into your system. For more information on how to do this, see Cloning Git Repositories to an ABAP Environment System [page 946].
3. **(Pull a Git Repository)** In order to work with branches, the Git repository needs to be pulled to the system. For more information on how to do this, see Pulling Git Repositories to an ABAP Environment System [page 948].
4. **(Create a Branch)** To create a branch, you perform a `POST` request on the `Branches` entity. In the header parameters, you should include the x-csrf-token that was retrieved in the first request. Fill in the following data in your request body:

    1. "sc_name": the name of your Git repository
    2. "branch_name": the name of your new branch
    3. "derived_from": the branch that your new branch should derive from

    Request

    ```
    POST/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Branches HTTP/1.1
    Host: host.com
    Authentication: basicAuthentication
    X-csrf-token: xCsrfToken
    Content-Type: application/json
    Accept: application/json
        {
    "sc_name" : "/DMO/GIT_REPOSITORY",
    "derived_from" : "main",
    "branch_name" : "newBranch"
        }
    ```

    Response

    ```
    HTTP/1.1 200 OK
    Content-Type: application/json
    {
        "d": {
            "__metadata": {
                "id": "https://host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
    Branches(branch_name='newBranch',sc_name='/DMO/GIT_REPOSITORY')",
                "uri": "https://host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
    Branches(branch_name='newBranch',sc_name='/DMO/GIT_REPOSITORY')",
                "type": "cds_sd_a4c_a2g_gha_sc_web_api.BranchesType"
            },
            "branch_name": "newBranch",
            "sc_name": "/DMO/GIT_REPOSITORY",
            "is_active": false,
            "criticality": 0,
            "derived_from": "main",
            "created_by": "CC0000000001",
            "created_on": "/Date(1584967657000+0000)/",
            "last_commit_on": "/Date(1584634658000+0000)/"
        }
    }
    ```

5. **(GET the Branch)** To read the branch entity, you can use the following request:

    Request

    ```
    GET /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
    Branches(branch_name='newBranch',sc_name='/DMO/GIT_REPOSITORY' HTTP/1.1
    Host: host.com
    Authentication: basicAuthentication
    Accept: application/json
    ```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
    "d": {
        "__metadata": {
            "id": "https://host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
Branches(branch_name='newBranch',sc_name='/DMO/GIT_REPOSITORY')",
            "uri": "https://host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
Branches(branch_name='newBranch',sc_name='/DMO/GIT_REPOSITORY')",
            "type": "cds_sd_a4c_a2g_gha_sc_web_api.BranchesType"
        },
        "branch_name": "newBranch",
        "sc_name": "/DMO/GIT_REPOSITORY",
        "is_active": false,
        "criticality": 0,
        "derived_from": "main",
        "created_by": "CC0000000001",
        "created_on": "/Date(1584967657000+0000)/",
        "last_commit_on": "/Date(1584634658000+0000)/"
    }
}
```

> **Note**
>
> Please note that the newly created branch is not yet active.

6. **(Checkout a Branch)** Simply creating a branch did not change the state of the system. In order to work with the newly created branch, you need to import the branch to the system. This action is called "checkout branch". You need to pass the Git repository and the name of the branch as query parameters.

Request

```
POST/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/checkout_branch?
branch_name='newBranch'&sc_name="'/DMO/GIT_REPOSITORY' HTTP/1.1
Host: host.com
Authentication: basicAuthentication
X-csrf-token: xCsrfToken
Content-Type: application/json
Accept: application/json
```

Response

```
HTTP/1.1 200 OK
Content-Type: application/json
{
    "d": {
        "__metadata": {
            "id": "https://host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
Pull(guid'UUID')",
            "uri": "https://host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
Pull(guid'UUID')",
            "type": "cds_sd_a4c_a2g_gha_sc_web_api.PullType"
        },
        "uuid": "UUID ",
        "sc_name": "/DMO/GIT_REPOSITORY ",
        "namespace": "",
        "status": "R",
        "status_descr": "Running",
        "start_time": "/Date(1571227437000+0000)/",
        "change_time": "/Date(1571227472000+0000)/",
        "criticality": 2,
        "user_name": "CC0000000001",
```

```
        "to_Execution_log": {
            "__deferred": {
                "uri": "https://host.com/sap/opu/odata/sap/
MANAGE_GIT_REPOSITORY/Pull(guid'UUID')/to_Execution_log"
            }
        },
        "to_Transport_log": {
            "__deferred": {
                "uri": "https://host.com/sap/opu/odata/sap/
MANAGE_GIT_REPOSITORY/Pull(guid'UUID')/to_Transport_log"
            }
        }
    }
}
```

Behind the scenes, a `Pull` entity is created. It can be used to track the status of the checkout (see steps 3 and 4 in Pulling Git Repositories to an ABAP Environment System [page 948]). This entity is specified in the `__metadata` of the response body.

7. **(Delete a Branch)** You can delete existing branches. Keep in mind, however, that neither the current active branch nor the main branch can be deleted. Both parameters are mandatory.

   Request

   ```
   DELETE /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Branches(branch_name='my-to-
   be-deleted-branch',sc_name='/DMO/GIT_REPOSITORY') HTTP/1.1
   Host: host.com
   Authentication: basicAuthentication
   Accept: application/json
   ```

   Response

   Since you're using a DELETE request, an HTTP 204 code is returned as confirmation that your branch has successfully been deleted.

   ```
   HTTP/1.1 204 No Content
   ```

4.2.9.7.7 Software Assembly Integration (SAP_COM_0582)

The communication scenario SAP_COM_0582 enables you to assemble software products via integration into build pipelines. With the APIs belonging to this communication scenario, you can check software components, assemble software packages, and upload the assembly as well as the export logs to the build pipeline. The pipeline then will upload the software package for registration in the SAP Add-on Assembly Kit as a Service.

Software Component Assembly

The assembly of software packages happens on software component level. The build pipeline has to orchestrate the API calls in case the software product consists out of multiple software components. Depending on the kind of package to be assembled (installation, support or patch) different steps are executed including the following:

- Check for a valid producer license of the used namespace(s)

- The object list is calculated. For installation packages these are all objects currently existing in the Git repository and optionally in the list of deleted objects compared to the predecessor release. For maintenance packages (component support package or correction package) this is the list of changed (including added and deleted) objects since the predecessor package. Comparisons are done by evaluating the current Git repository state with the one of the predecessor packages (indicated by the commit ID used for assembling the predecessor package)
- The delivery transport is created, objects are inserted, object list checks are executed and finally the delivery transport is exported
- Further metadata is gathered (e.g. current system software levels, DDIC extensibility quotas, software component relations) and combined with the delivery transport
- Logs are gathered and compressed

  > ⚠ Caution
  >
  > The logs might be needed in case of support inquiries or for compliance requirements. The build system automatically deletes the `DELIVERY_LOGS.ZIP` archive, like other build results, after a grace period. We recommend you transfer this file to long-term storage.

- The API snapshot of the predecessor semantic version is selected and set to check relevant (if found)
  - For add-on installation (AOI) software packages the following snapshot is selected: `X.*.*` with X representing the previous release version
  - For Component Support Package (CSP) software packages the following snapshot is selected: `Y.X.*` with X representing the current release version and Y representing the previous support package level
  - For Correction Package (CPK) software packages the following snapshot is selected: `Y.Y.X` with Y representing the current release and support package level and X representing the previous patch level
  - For the initial version 1.0.0 there will never be a predecessor version, and since no snapshot will be found in this case, no snapshot will be set to check-relevant.
  - If no semantic version API snapshot can be found in the assembly system but a different API snapshot is set to check-relevant, the API compatibility check will be done based on such a snapshot instead.
- Mandatory ABAP Test Cockpit Checks are run, including
  - Consistency of software component dependencies - Virus Check of MIME objects using SAP Malware Scanning Service. See What Is SAP Malware Scanning Service
  - Released API Compatibility. See Compatibility Checks for Released APIs

  > ⓘ Note
  >
  > The Released API Compatibility ABAP Test Cockpit check can only function properly if there is an API snapshot available in the assembly system. Therefore, make sure to use a permanent system for the assembly if you need to fulfill consistency and stability criteria for your released APIs. For more information, see Checking the Compatibility of Released APIs.
  >
  > In some cases where the assembly of a software package version needs to be repeated with a different commit ID in the software component, you should delete any API snapshot that was already created in the assembly system for the semantic version, so that the snapshot is recreated during the reassembly.

- A new API snapshot for the current assembled semantic version is created. See Creating API Snapshots .
- Generation and Syntax Check of the complete software component

Build Pipeline

The build pipeline used to call this communication scenario has to orchestrate the assembly of the individual software components of the build software product and has to integrate with the SAP Add-On Assembly Kit as a Service as well as SAP Business Technology Platform and several other communication scenarios of the build and the acceptance test / integration test system to achieve a meaningful continuous integration and delivery (CI/CD) level.

We strongly recommend using the SAP Landscape Portals Build Product Version app as build pipeline and for SAP S/4 HANA Cloud Public Edition based software products it is mandatory. For more information, see Access to Landscape Portal and Build.

To make the setup of build pipelines as easy as possible, the open source project "Piper" has been created. In general, there are various functions ("Library steps"), as well as whole pipelines, available to reuse. There, you can find the step `abapEnvironmentAssemblePackages` (see abapEnvironmentAssemblePackages), that implements the functionality of the APIs belonging to SAP_COM_0582. The step `abapEnvironmentAssemblePackages` is embedded in a sequence of steps which are needed to produce and distribute software packages, see the Library steps on Project "Piper" and Build and Publish Add-on Products on SAP BTP ABAP Environment .

> **Note**
>
> Project 'Piper' is expected to be archived on December 31st 2026. No further maintenance will be done as of this date.

For more information, see

- Developing and Operating SaaS Applications in the Landscape Portal for SAP BTP ABAP Environment.
- Scalable Delivery in the Landscape Portal for SAP S/4HANA Cloud Public Edition.

4.2.9.7.7.1 Assembling a Software Package

Prerequisites

- You have fulfilled all prerequisites mentioned in Build and Publish Add-on Products on SAP Cloud Platform ABAP Environment
- Ideally, you have set up and configured a pipeline with stages and steps as recommended in ABAP Environment Pipeline , and Configuration .

In doing so, the following activities are performed automatically:

- You have an ABAP environment system.
- You've created a *Communication User* as described in How to Create Communication Users [page 2995].
- You've created a *Communication System* as described in How to Create Communication Systems.
- You've created a *Communication Arrangement* as described in How to Create a Communication Arrangement [page 2998].

- You have selected the communication scenario *SAP_COM_0582* for your communication arrangement and have mapped it to your communication system.
- You have pulled a repository to the SAP Cloud Platform ABAP Environment system using communication scenario *SAP_COM_0948*.
- You have provided information on build essentials in an addon.yml file which has already been validated by the Add-On Assembly Kit as a Service (AAKaaS).

Procedure

1. During the build pipeline run, the step **abapEnvironmentAssemblePackages** is executed (see abapEnvironmentAssemblePackages). This step runs the assembly of a list of provided installations, support packages or patches in an SAP Cloud Platform ABAP Environment system and saves the corresponding SAR archive to the pipeline.
2. An example for the configuration in config.yml as well as input arguments and values can be found here: Examples .

4.2.10 Intelligent Scenario Lifecycle Management

Intelligent Scenario Lifecycle Management (ISLM) is a self-service framework that allows you to perform lifecycle management operations on machine learning scenarios.

This includes training, deploying, activating, and fetching an accurate inference result based on the trained intelligent scenarios. An intelligent scenario is an ABAP representation of machine learning business-specific use case.

4.2.10.1 Generative AI for ISLM

Intelligent Scenario Lifecycle Management (ISLM) is a standardized framework that facilitates end-to-end lifecycle operations and the consumption of business AI scenarios.

You can use ISLM to seamlessly adopt a Generative AI scenario into your business application by creating a Gen AI Intelligent Scenario. The framework also offers additional features like automatic enablement (Turnkey) and Usage Types for faster integration to Business Apps.

ISLM integrates with the Generative AI Hub in SAP AI Core that offers easy access to LLM models hosted by hyperscalers. This integration is made possible via a communication scenario SAP_COM_0A69, which is a Customer Outbound Integration. This is a customer-managed communication scenario, which requires manual connection setup.

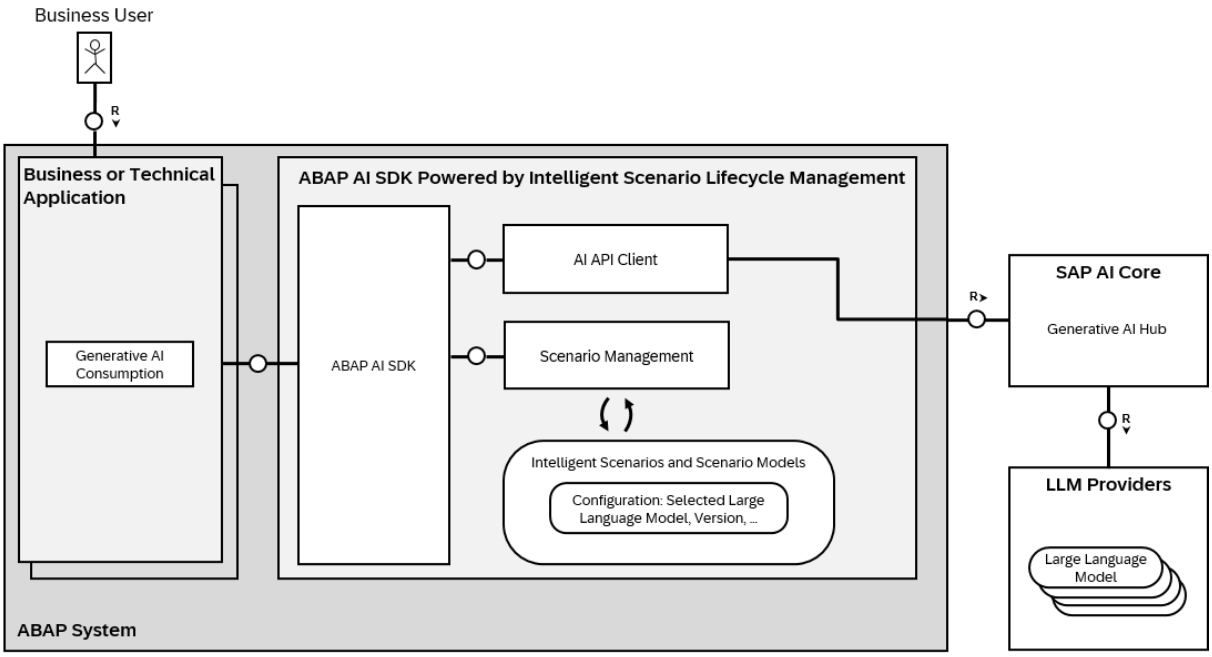
With the connection to Gen AI Hub established, ISLM can be used to create and activate deployments – Turnkey or Manually. You can then use the ABAP AI SDK APIs for prompt execution. The ABAP AI SDK is a reuse library in an ABAP system that provides access to generative AI functionality via Intelligent Scenario Lifecycle Management (ISLM).

The intelligent scenarios are development objects that can be created using the ABAP development tools for Eclipse in ABAP systems.

The Intelligent Scenario Management app should be used for operating intelligent scenarios, for example activation and deployments.

For SAP delivered Gen AI and custom-developed intelligent scenarios that support turnkey, the creation and activation of deployments can be automated using the Turnkey [page 968] feature.

Following is the architectural representation of ISLM integration with ABAP AI SDK.



The following sections shows get step-by-step instructions on how to set up the integration of ISLM with the generative AI hub in SAP AI Core:

- Download Certificate [page 959]
- Create SAP BTP Service Instance and Key [page 960]
- How to Configure the Communication System for SAP_COM_0A69 [page 962]
- How to Create the Communication Arrangement for SAP_COM_0A69 [page 964]
- Manage Gen AI Scenarios [page 964]

4.2.10.1.1 Download Certificate

Prerequisites

You already have the *SAP_BR_ADMINISTRATOR* business role assigned.

Context

In this task, download the client default certificate of the ABAP system to create the X.509 service key in AI Core instance.

Procedure

1. Log on to the SAP Fiori launchpad.
2. Enter user credentials for administrator.
3. Choose *Maintain Client Certificates*.

   You can also search for *Maintain Client Certificates* and select it.
4. On the *Maintain Client Certificates* screen, search for *Client Default*.
5. Choose the *Client Default* from the certificates list.
6. Choose *Export* to export the file to your local file folder.
7. Choose the *Base-64-encoded X.509 certificate (.crt)* format to save the certificate.
8. Choose *Export*.

Next Steps

Create SAP BTP Service Instance and Key [page 960]

4.2.10.1.2 Create SAP BTP Service Instance and Key

Create a service instance and service key for accessing SAP AI Core, where the large language models in the generative AI hub are hosted. The X.509 service key is created with the downloaded client default certificate of the ABAP system passed as parameter.

Prerequisites

- You have purchased the license for SAP AI Core.
- You have an active global account on SAP BTP with access to the SAP BTP cockpit.
  For more information, see Setting up a Global Account.
- You have subscribed to SAP AI Core.
- You have already downloaded the client default certificate from the ABAP system. See, Download Certificate [page 959]

Create Service Instance

Context

Once you have an active SAP BTP global account, create a service instance in your SAP BTP subaccount.

> ⓘ Note
>
> This procedure is based on default content and UIs. Depending on your configuration settings, the procedure may be slightly different.

Procedure

To create service instance for SAP AI Core, see Create a Service Instance.

> → Remember
>
> While updating the basic information for your instance, ensure that you choose *Plan* as *Extended*. For more information, see Service Plans

Create Service Key

Context

Create service key for OAuth 2.0-based authentication.

Procedure

1. On the *Instances and Subscriptions* page, find your new instance that you created and choose *Create Service Key* from the drop-down.
2. Enter a name for your service key.
3. Configure binding properties.
   a. Enter *Binding Name* as SAP_COM_0A69_Skey (any name of your choice).
   b. The parameters you need to set when you create your service key or service binding are the following in JSON format:

> {·} Sample Code

```
{
  "xsuaa": {
    "credential-type": "x509",
```

```
    "x509": {
      "certificate": "<Client-Default-Certificate>",
      "ensure-uniqueness": false,
      "certificate-pinning": false,
      "hide-certificate": true
    }
  }
}
```

To rotate a certificate, see: How to Handle Default Client Certificate Renewal [page 3188]

For more information, see Parameters for Self-Managed X.509 Certificates

4. Click *Create*.
5. Download your service key to save it.

Next Steps

Save the service key details to use while creating the communication system and communication arrangement for the integration between ISLM and Generative AI Hub.

4.2.10.1.3 How to Configure the Communication System for SAP_COM_0A69

Configure communication system for SAP_COM_0A69.

Currently, you can create one instance of the communication scenario per client.

Prerequisites

Make sure you have followed all steps as described in Create SAP BTP Service Instance and Key [page 960], so the X.509 service key is created from the default client certificate of the ABAP system.

Sample Code

```
{
"serviceurls": {
"AI_API_URL": "xxx"
},
"appname": "xxx",
"clientid": "xxx",
"identityzone": "xxx",
"identityzoneid": "xxx",
"url": "https://xxx.authentication.xxx.hana.ondemand.com",
"certurl": "https://xxx.authentication.cert.xxx.hana.ondemand.com",
"certificate": "-----BEGIN CERTIFICATE-----\n...\n-----END CERTIFICATE-----\n"
}
```

Procedure

1. Log on to the SAP Fiori launchpad.
2. Navigate to Communication Management.
3. Choose *Communication Systems* tile.
4. Choose *New*.
5. Enter *System ID* and *System Name*.
   System Name is auto populated. We recommend providing the same name for both *System ID* and *System Name*.
6. Choose *Create*.
7. Navigate to the *Technical Data* tab.
   Update the *Technical Data* details as per the service key details.
8. In the *General* section, for *Host Name*, use the *AI_API_URL* value without protocol.
9. In the *Outbound OAuth 2.0 Client Settings*, enter the following details:
   - For *Authorization Endpoint* use the url value without protocol.
   - For *Token Endpoint*, use the *url* value without protocol and add the path: /oauth/token.
     Add the *"/oauth/token"* string to the URL value.
   - For *mTLS Endpoint*, use the *certurl* value without protocol and add the path: /oauth/token.

   > ### Note
   >
   > For the steps 8 and 9, ensure that you remove the *"https://"* string from all URL values.

10. On the *Users for Outbound Communication* tab, choose the *Add* button.
11. Create an authentication method, enter the following details:
    - *Authentication Method* - Select *OAuth 2.0*
    - *OAuth 2.0 Client ID* - Use the *clientid* value from the service key.
    - *Client Authentication* - Select *mTLS*
    - For *Client Certificate*, select *Client Default* from the value help.
12. Choose *Create*.
13. Choose *Save*.

Next Steps

How to Create the Communication Arrangement for SAP_COM_0A69 [page 964]

4.2.10.1.4 How to Create the Communication Arrangement for SAP_COM_0A69

Procedure

1. Log on to the SAP Fiori launchpad.
2. Navigate to *Communication Management*.
3. Choose *Communication Arrangements* tile.
4. Choose *New*.
5. In the *Scenario* field, select the predefined communication scenario *SAP_COM_0A69*.
6. Enter *Arrangement Name*.
7. Choose *Create*.
8. Navigate to the *Outbound Communication* area.
9. In the *Communication System* field, enter the communication system that you created. See, How to Configure the Communication System for SAP_COM_0A69 [page 962]
10. Choose *Save*.

Next Steps

Once the communication system and arrangement are created, you can continue with creating an intelligent scenario of type Generative AI in the ABAP development tools for Eclipse.

You are required to create an intelligent scenario for your Gen AI use cases. To create an intelligent scenario, see Creating Intelligent Scenarios and Creating Intelligent Scenario Models .

With the intelligent scenario, you can perform all the lifecycle management operations. See, Manage Gen AI Scenarios [page 964]

4.2.10.1.5 Manage Gen AI Scenarios

After communication system and communication arrangement are configured, create an intelligent scenario and continue managing the lifecycle of the intelligent scenario. You are required to create an intelligent scenario for your custom owned Gen AI use cases.

With this intelligent scenario you can perform all lifecycle management operations to get an inference result. You can perform all the operations using the Intelligent Scenario Management App [page 965]. You can also choose to automate all the operations by using turnkey feature. Once turnkey is enabled for an intelligent scenario, all the operations from pre-requisites check to activation are run automatically. Or, you can perform all these operations manually too.

The following is a list of operations that you can perform on an intelligent scenario:

- Create - You can create an intelligent scenario of type Gen AI in ABAP development tools for Eclipse. See, Creating Intelligent Scenarios and Creating Intelligent Scenario Models
- Deployment: You can deploy the trained machine learning models for inference consumption. This functionality applies to Side-by-Side scenarios only. See, Deploy and Undeploy
- Activation: You can activate the trained machine learning model for which you want to get the inference results. See, Activate and Deactivate
- Inference Results - You can get the predictions that are generated from the trained machine learning models. See, Inference Request
- Turnkey - You can automate all ML operations of an intelligent scenario. Which means, once turnkey is enabled for an intelligent scenario, all the operations, such as pre-requisites check, train, deployment, and activation are run automatically. See, Turnkey [page 968]

4.2.10.1.5.1 Intelligent Scenario Management App

Manage the lifecycle of your intelligent scenarios, including training, deployment, and activation.

Context

With this app you can manage the lifecycle of your intelligent scenarios, including training, deploying, and activating machine learning models. You can access this app from the SAP Fiori launchpad.

To use this app, you must have the `SAP_BR_ANALYTICS_SPECIALIST` business role assigned. See: Business Catalogs and Business Roles

Key Features

You can use this app to:

- Deploy and Undeploy [page 965] a trained model for inference consumption.
- Activate and Deactivate [page 967] a deployed model to generate predictions.
- Monitor deployment and activation status.
- Use Turnkey [page 968] to automate all ML operations.

4.2.10.1.5.2 Deploy and Undeploy

Context

Once training has successfully completed for a model, it can then be deployed or undeployed.

Procedure

1. Launch the *SAP Fiori launchpad*.
2. Open the *Intelligent Scenario Management* app.

   The *Intelligent Scenarios Management* app opens with all the available intelligent scenarios.
3. Click the intelligent scenario that you want to work with.

   A list of all the available versions for the selected intelligent scenario are displayed in the *Version* section.
4. Click the version that you want.

   A list of all the training for the selected version is displayed in the *Training* section.
5. Select the training that you want to deploy.

   Ensure that the selected training is in the completed status.
6. Click *Deploy*.
7. Alternatively, you can click *Deploy and Monitor*.

   With this option, you create deployment and navigate to the *Deployment* tab for monitoring purpose.
8. In the *Configuration* section, enter the following:

Field	Description
Description	Displays a short description for the deployment. You can either change the description for the deployment.
Parameter Name	Enter the parameter value.
Artifact Name	Enter the artifact value.

> ### Note
>
> The parameters and artifacts are derived from the Inference Pipeline in the remote machine learning provider. So, the parameter, artifact names, or the number of parameters and artifacts are displayed as configured in the remote machine learning provider.

9. Click *Deploy*.

   To undeploy, click the *Deployments* tab. Select the deployment that you want to undeploy and click *Undeploy*. Before you undeploy, ensure that you deactivate deployment for your user, other users, and all users.

4.2.10.1.5.3 Activate and Deactivate

Prerequisites

Before you begin:

- For activation, ensure that the deployment is in the deployed status.
- For deactivation, ensure that the deployment is in the active status.

Context

You activate the deployment to consume the inference in your business application. You can activate deployment for your user and all users by selecting the *For Me* and *For All* options respectively. You can activate the deployment only for your user, test the inference in the Business Application, and then decide whether to activate the deployment for all user in the system. For each intelligent scenario, only one active deployment is supported for your user and all users. Once activated, the inference is returned on the active deployment.

Consider the following possible scenarios while activating a deployment:

- You can activate the deployment for your user or all users in the system.
- If you activate the deployment for your user, then the previous activation for your user is deactivated automatically.
- If you activated deployment for you and all users, then the active deployment for your users is considered for inference.
- If you activated deployment for you and all users, and you deactivate deployment for all users, then the deployment for your user is activated.

To activate or deactivate the deployment, follow these steps:

Procedure

1. Launch the *SAP Fiori launchpad*.
2. Open the *Intelligent Scenario Management* app.

   The *Intelligent Scenarios Management* app opens with all the published intelligent scenarios.
3. Click the intelligent scenario that you want to work with.

   A list of all the available versions for the selected intelligent scenario is displayed in the *Version* section.
4. Click the version that you want.
5. Click the *Deployments* tab.

   A list of all the Deployments for the selected version are displayed.

6. Select the deployment that you want to activate.
7. Select one of the following from the *Activate* drop-down list:
    - *For Me*
    - *For All*

    Once activated, the status changes to *Active*. Now, this active deployment is used to return the inference result in your business application. You can also click the *Active* status icon to see the list of users for which the deployment is active. Additionally, check the *User-Specific Activation Details* section in the *Deployment Report* page to view the activation details.

    The selected deployment is now activated. As a result, any previously activated deployment is deactivated.

    For deactivating, select an active deployment that you no longer require and selects from the *Deactivate* drop-down list. You can either choose *For Me* or *For All*. Once the deployment is deactivated for your user and for all user, no inference result is returned.

    > ⓘ Note
    >
    > You can view the active deployment only in the relevant deployment list. Also, in the *Active* column, you can check whether the deployment is active for all users, your user, or other users.

4.2.10.1.5.4 Turnkey

The turnkey feature enables you to automate all end-to-end lifecycle operations of an intelligent scenario. Which means, once turnkey is enabled for an intelligent scenario, all the operations, such as pre-requisites check, train, deployment, and activation are run automatically. By automating these operations, turnkey feature significantly reduces manual effort during the initial activation of a scenario and throughout its lifecycle management. You can schedule turnkey and monitor the schedule in the *Turnkey* tab. Further details are about the schedule and runs are provided in the *Turnkey Schedule* and *Turnkey Runs* tabs.

You can select the intelligent scenario that supports the turnkey and switch it on. Once switched on, you can continue with the predefined schedule for turnkey. Else, modify the schedule details and create a new turnkey schedule as needed.

Before using the turnkey option, review the following conditions:

- **Side-by-Side**:
    - Undeploys already deployed deployments. Another deployment may be deployed
    - Deactivates already activated deployment. Another deployment may be activated.
- An admin can override the deployment and activation triggered by the turnkey.

4.2.10.1.5.4.1 Create a Turnkey Schedule

Create a turnkey schedule for an intelligent scenario with predefined schedule pattern.

Prerequisites

- Intelligent scenario is not in an obsolete state.
- Intelligent scenario already supports turnkey automation feature.

Context

Any intelligent scenario that supports turnkey automation has predefined recurrence schedules. You can either choose to create a schedule with the same details or modify the existing schedule to create a new turnkey schedule for the scenario.

Procedure

1. Launch the SAP Fiori launchpad.
2. Open the Intelligent Scenario Management app.

   The *Intelligent Scenarios Management* page opens with a list of published intelligent scenarios that are in *Published*, *Deprecated*, and *Obsolete* status.

   > ⓘ Note
   >
   > If the intelligent scenario is in a deprecated state, you can continue to schedule turnkey automation. However, it is strongly recommended to use the successor intelligent scenario as this scenario might be obsolete in future releases.
   >
   > If the intelligent scenario is in an obsolete state, no further operations are allowed. However, you can use the successor intelligent scenario, if available.

3. Choose any model or version for which you want to create a turnkey schedule.

   You can also use the search functionality to filter the intelligent scenario that you want to use and supports turnkey automation.
4. Choose the turnkey switch corresponding to the intelligent scenario from the *Turnkey* column.

   > ⓘ Note
   >
   > - This option is available for intelligent scenario that supports turnkey automation only.
   > - By default, the turnkey option is set to *OFF*. By choosing this switch, the turnkey option changes to *ON* status.

The *Schedule Turnkey* dialog box opens.

5. Review the following details:

Name	Description
Description	A short description about the schedule.
Recurrence Pattern	Predefined schedule patter for the selected intelligent scenario.
Execute Turnkey Run Immediately	Choose whether you want to start the turnkey schedule immediately once created.

6. Choose *Create Turnkey Schedule*.

   Turnkey schedule is on.

Next Steps

Depending on your requirement, you can either choose to modify, monitor the schedule progress or cancel the turnkey schedule.

- Modify a Turnkey Schedule [page 970]
- Additional Turnkey Actions [page 973]
- Turnkey Schedule Details [page 975]

4.2.10.1.5.4.2 Modify a Turnkey Schedule

Create a turnkey schedule for an intelligent scenario with new schedule pattern.

Prerequisites

- Intelligent scenario is not in an obsolete state.
- Intelligent scenario already supports turnkey automation feature.

Context

Any intelligent scenario that supports turnkey automation has predefined recurrence schedules. You can either choose to create a schedule with the same details or modify the existing schedule to create a new turnkey schedule for the scenario.

Procedure

1. Launch the SAP Fiori launchpad.
2. Open the Intelligent Scenario Management app.

   The *Intelligent Scenarios Management* page opens with a list of published intelligent scenarios that are in *Published*, *Deprecated*, and *Obsolete* status.

   > ⓘ Note
   >
   > If the intelligent scenario is in a deprecated state, you can continue to schedule turnkey automation. However, it is strongly recommended to use the successor intelligent scenario as this scenario might be obsolete in future releases.
   >
   > If the intelligent scenario is in an obsolete state, no further operations are allowed. However, you can use the successor intelligent scenario, if available.

3. Choose any model or version for which you want to create a turnkey schedule.

   You can also use the search functionality to filter the intelligent scenario that you want to use and supports turnkey automation.

4. Choose the turnkey switch corresponding to the intelligent scenario from the *Turnkey* column.

   > ⓘ Note
   >
   > - This option is available for intelligent scenario that supports turnkey automation only.
   > - By default, the turnkey option is set to *OFF*. By choosing this switch, the turnkey option changes to *ON* status.

   The *Schedule Turnkey* dialog box opens.

5. Review the following details:

Name	Description
Description	A short description about the schedule.
Recurrence Pattern	Predefined schedule patter for the selected intelligent scenario.
Execute Turnkey Run Immediately	Choose whether you want to start the turnkey schedule immediately once created.

6. Choose the *Edit Turnkey Schedule* icon.

   The *Schedule Turnkey* dialog box opens.

7. Enter the following details:

Name	Description
Description	A short description about the schedule.
Time Zone	Select the time zone that you want to follow for scheduling turnkey.

Name	Description
Start Time	Select the earliest date and time to start with turnkey.
Frequency	Select whether you want to schedule turnkey weekly or monthly.
Recur for Every	Select the number of turnkeys that you want to schedule per month or week.
On Days	Select the day that you want to start the turnkey. For weekly, select the day or day on which you want to start the turnkey. For monthly, select one of the following: • Beginning of the month: Select on which day of the month you want to schedule turnkey. • End of the month: Select the last working day for turnkey. • A certain weekday: Select a specific day for turnkey.
End Schedule	Select when to end your schedule for turnkey. This schedule can be one of the following: • On a specific day: Select the date and time on which the turnkey schedule must end. • After a specific number of occurrences: Enter the number of occurrences after which the turnkey schedule must end.
On Non-Working Day	Select whether or not you want to schedule turnkey for your intelligent scenario during a non-working day. This option can be one of the following: • No Restriction • Run on the Next Working Day • Run on the Previous Working Day • Don't Run
Calendar	Select the calendar of the country/region that you want to follow for scheduling turnkey.
Execute Turnkey Run Immediately	Choose whether or not you want to start the turnkey run immediately.

8. Choose *Create Turnkey Schedule*.

   Turnkey schedule is on.

Next Steps

Depending on your requirement, you can either choose to monitor the schedule progress or cancel the turnkey schedule.

- Turnkey Schedule Details [page 975]

- Additional Turnkey Actions [page 973]

4.2.10.1.5.4.3 Additional Turnkey Actions

Context

In addition to creating and editing the turnkey schedule for an intelligent scenario, you can choose whether you want to cancel the running schedule or restart any failed turnkey schedule.

Restart Turnkey Run

Prerequisites

Ensure that the turnkey run is in *Failed* state.

Context

After a turnkey operation is completed, it is possible that due to some error the turnkey run has failed. In such case, you can choose to restart the turnkey run.

Procedure

1. Launch the SAP Fiori launchpad.
2. Open the Intelligent Scenario Management app.
3. Choose any model or version that you want.
4. Choose *Turnkey* > *Turnkey Runs* tab.
5. Choose the *Run ID* for which you want to restart the turnkey run.
6. Choose *Restart*.
7. On prompt, choose *Ok*.

Cancel Turnkey Run

Prerequisites

Ensure that the turnkey run is in *Running* state.

Context

You can cancel any turnkey run that is in progress.

> **Note**
>
> Once canceled, all subsequent operations are canceled, and you can no longer continue with that turnkey run.

Procedure

1. Launch the SAP Fiori launchpad.
2. Open the *Intelligent Scenario Management* app.
3. Choose any model or version that you want.
4. Choose *Turnkey* > *Turnkey Runs* tab.
5. Choose the *Run ID* for which you want to cancel the turnkey run.
6. Choose *Cancel*.
7. On prompt, choose *Ok*.

Switch Off Turnkey

Context

You can choose to switch off the turnkey. Only the user who switched on the turnkey for that intelligent scenario can switch it off. However, you can grant access to another user to switch off the turnkey for an intelligent scenario. To provide access, see Maintaining Authorizations.

4.2.10.1.5.4.4 Turnkey Schedule Details

The following tables list the overall turnkey schedule details for the intelligent scenario:

Scheduling Options Tab

Name	Description
Planned Start	Displays when the turnkey schedule starts.
Scheduled in Time Zone	Displays the timezone selected while creating schedule.
Recurrence Pattern	Displays summary of the schedule pattern.
Planned End	Displays the end condition of the schedule as defined by the user, which can be: • No end condition • A fixed date and time • Number of occurrences
On Non-Working Day	Displays schedule restrictions selected for a non-working day. As defined, the schedule adapts if the next run starts on a non-working day as per the calendar. The possible values are: • No restriction • Run on the next working day • Run on the previous working day • Don't run
Calendar	Displays calendar selected for schedule.

Turnkey Runs Tab

Name	Description
Run ID	Displays the ID for the turnkey run.
Status	Displays the turnkey run status, which can be: • *Running* - Turnkey run is in progress. • *Success* - Turnkey run completed. • *Failed* - Turnkey run failed due to errors. • *Canceled* - Turnkey run canceled by user.
Run Type	Displays the turnkey run type, which can be: • *Immediate* - Turnkey run that runs immediately once created. • *Recurrent* - Turnkey runs based on the periodic schedule. • *Restart* - Turnkey run that is restarted after a failed run.
Created At	Displays start time of the run.

Name	Description
Ended at	Displays end time of the run.

4.2.10.1.5.4.5 Turnkey Run Details

Name	Description
Status	Displays the turnkey run status, which can be: • *Running* - Turnkey run is in progress. • *Success* - Turnkey run completed. • *Failed* - Turnkey run failed due to errors. • *Canceled* - Turnkey run canceled by user.
Created At	Displays start time of the turnkey run.
Created By	Displays the user ID of the scheduler.
Changed By	Displays the user ID who modified the turnkey run.
Duration	Displays turnkey run duration.
Additional Information	Displays information about parent schedule in which the run is created.

Turnkey Operations

Name	Description
Type	Displays ML operation type, which can be: • Prerequisite Checks • Train • Activate • Deactivate • Deploy (only for Side-by-Side) • Undeploy (only for Side-by-Side)
Created At	Displays start time of the operation.
Message	Displays any message that are logged. You can view by clicking the *i* button for that operation.

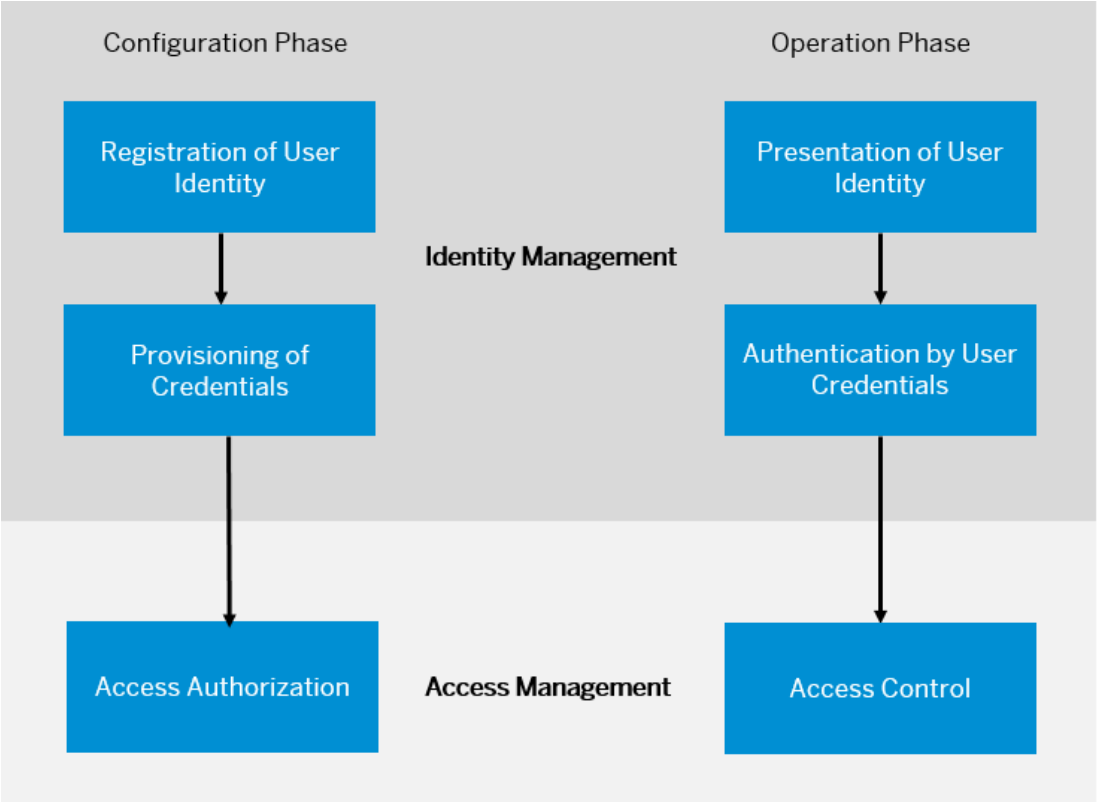
4.2.11 Identity and Access Management (IAM)

Learn more about the basic concepts and procedures for managing users and defining authorizations for business services and SAP Fiori applications.

What Is Identity and Access Management?

The main components of Identity and Access Management are:

- User and Identity Management
- Roles and Authorization
- Authentication



In the configuration phase, an administrator registers and authorizes access rights.

In the operation phase, business users identify and authenticate themselves. The access to applications is based on previously authorized access rights.

In a business scenario, this means that an administrator manages the authentication and authorization of business users, including configuring trust and identity federation in the subaccount usually by configuring a custom identity authentication service tenant [page 226]. As a fallback, SAP ID service is configured as the default identity provider. Additionally, an administrator also creates roles using templates, adds roles to role collections, and assigns role collections to business users. The business users can then log on to SAP Fiori applications and get authenticated.

Who Is This Guide For?

If you're an SAP customer **developer** or partner developer and you create business services in the ABAP environment using ABAP Development Tools for Eclipse, this guide is for you. It guides you through an example of a business service based on a service binding. Note that http services work in a similar way.

In addition, the guide also covers activities that need to be performed by an **administrator**, such as assigning users to business roles.

What Is This Guide About?

The guide starts with an overview of the basic concepts of identity and access management. It then leads you step by step through the design and implementation of your authorization model using an example.

Why should you care about access control and authorizations? You must know that every new service is automatically available to all developers in a development system, provided the system administrator has assigned them a role that is derived from the developer role SAP_BR_DEVELOPER (which is typically the case). However, business users in nondevelopment systems can only access the business service if a developer has defined the appropriate authorizations and an administrator grants the business users access for each relevant system. The same is true for communication users that are needed in scenarios where you work with an API business service (inbound or outbound) from or into the ABAP environment.

The **Identity Management** section gives you an overview about Identity and User Management, which includes identity federation, user types, and user provisioning.

The **Access Management** section includes the following main scenarios about how you can implement and provide authorizations for your services:

- In chapter Providing (Unrestricted) Access for Business Users [page 996], you learn how to provide users in a nondevelopment system access to the service.
  Providing unrestricted access is the simplest scenario with only a few steps. The access is unrestricted, though, which means, for example, that you can't differentiate between write and read access.
- In chapter Providing Access Based on Activities for Business Users [page 1000], you learn how to implement access for different activities, for example, separate read and write authorizations.
- In chapter Providing Access Based on Field Values for Business Users [page 1011], you learn how to fine-tune your access restrictions depending on field values.

OData services that you develop using ADT can also be consumed in SAP Fiori applications (developed using the Business Application Studio). In this guide, you also learn how to provide access to an SAP Fiori application that consumes a business service.

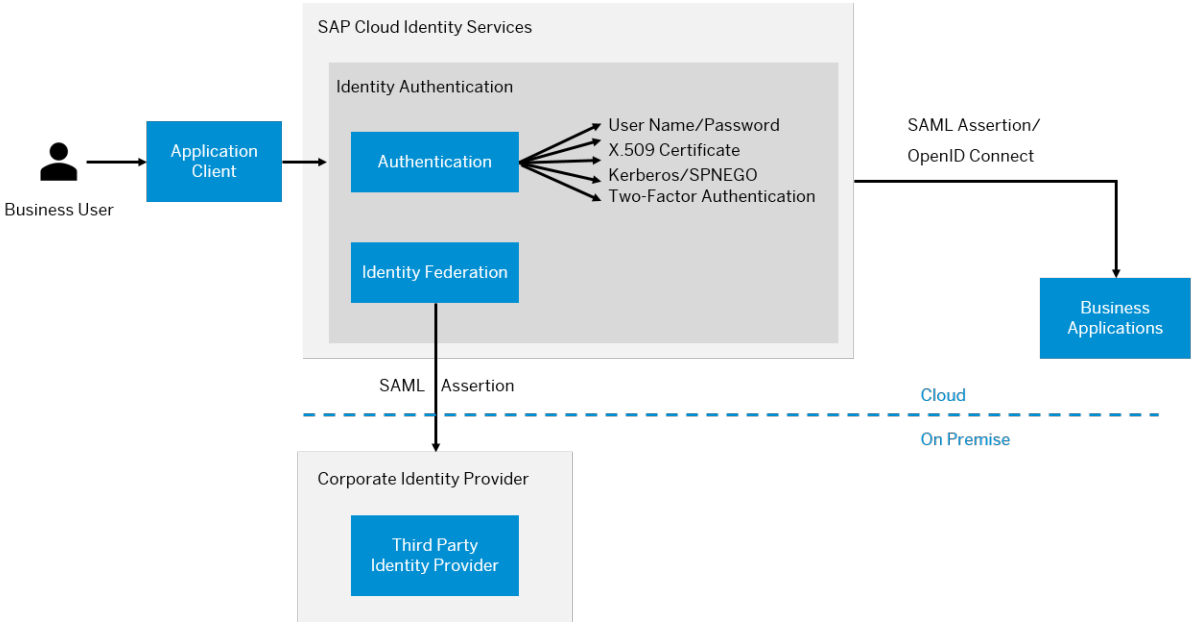
Custom Development and Testing

In SAP Business Technology Platform (SAP BTP), customers have the flexibility to develop their own code and services using ABAP. To test newly created services and applications in the development tenant, it is crucial that the developer role SAP_A4C_BC_DEV_TST_PC is extended with every newly created custom Authorization Object (AO), ensuring that full authorizations are granted for testing purposes. This ensures that developers

can comprehensively test their applications without facing authorization issues, enhancing the development and deployment experience.

4.2.11.1 Identity Management

Identity management comprises setting up trust and providing credentials for business users.



When setting up your identity management architecture, the following uses cases are possible:

- Using the default identity provider SAP ID service, an instance of Identity Authentication service. It is used to set up trust for business users and supports identity federation, which enables user access across various systems by exchanging identity information based on a trust relationship. See Identity Federation [page 980].
- Establishing trust to a custom identity authentication service tenant. See Setup of a Custom Identity Service [page 226].
- Using a corporate identity provider in a hybrid landscape. In this case, we recommend using the Identity Authentication service as a proxy for the corporate identity provider. See Corporate Identity Providers. Note that there is no direct trust relationship between the authenticating identity provider and the service provider that the business user is trying to access.

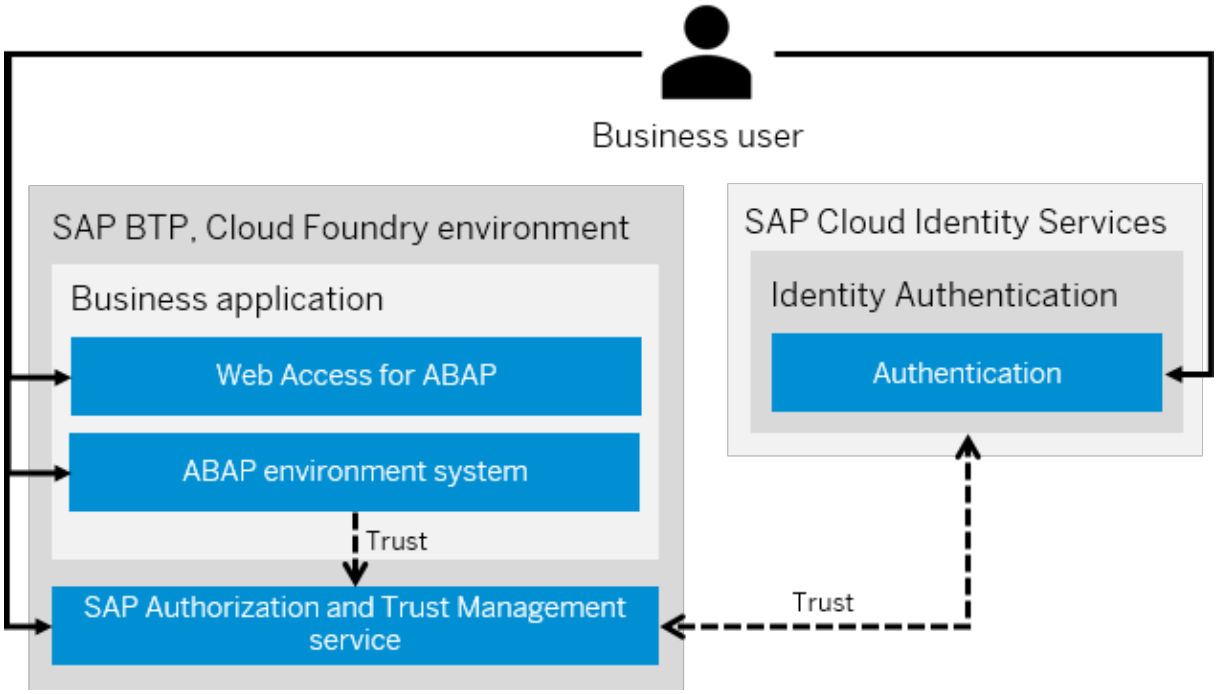
Related Information

Trust and Federation with Identity Providers
SAP Cloud Identity Services - Identity Authentication

4.2.11.1.1 Identity Federation

Identity federation is the process of sharing identity information between two parties by delegating the authentication responsibility to a trusted external party through the use of a common token.

In the ABAP environment, the authentication of business users is based on OAuth 2.0 authorization code flow, using either the SAML 2.0 or OpenID Connect protocol and JSON web token (JWT) as a common access token.



The business application (OAuth 2.0 client) is on one hand represented by the Web Access for ABAP as the gateway component or a standalone approuter application deployed to the SAP BTP, Cloud Foundry environment. See Subscribing to the Web Access for ABAP and Application Router.

On the other hand, an ABAP environment system is serving the business application resources (OAuth 2.0 resource server). See Creating an ABAP System.

The SAP Authorization and Trust Management service in the SAP BTP, Cloud Foundry environment is providing endpoints to retrieve and exchange the authorization code for an access token (OAuth 2.0 authorization/ authentication server). See SAP Authorization and Trust Management Service.

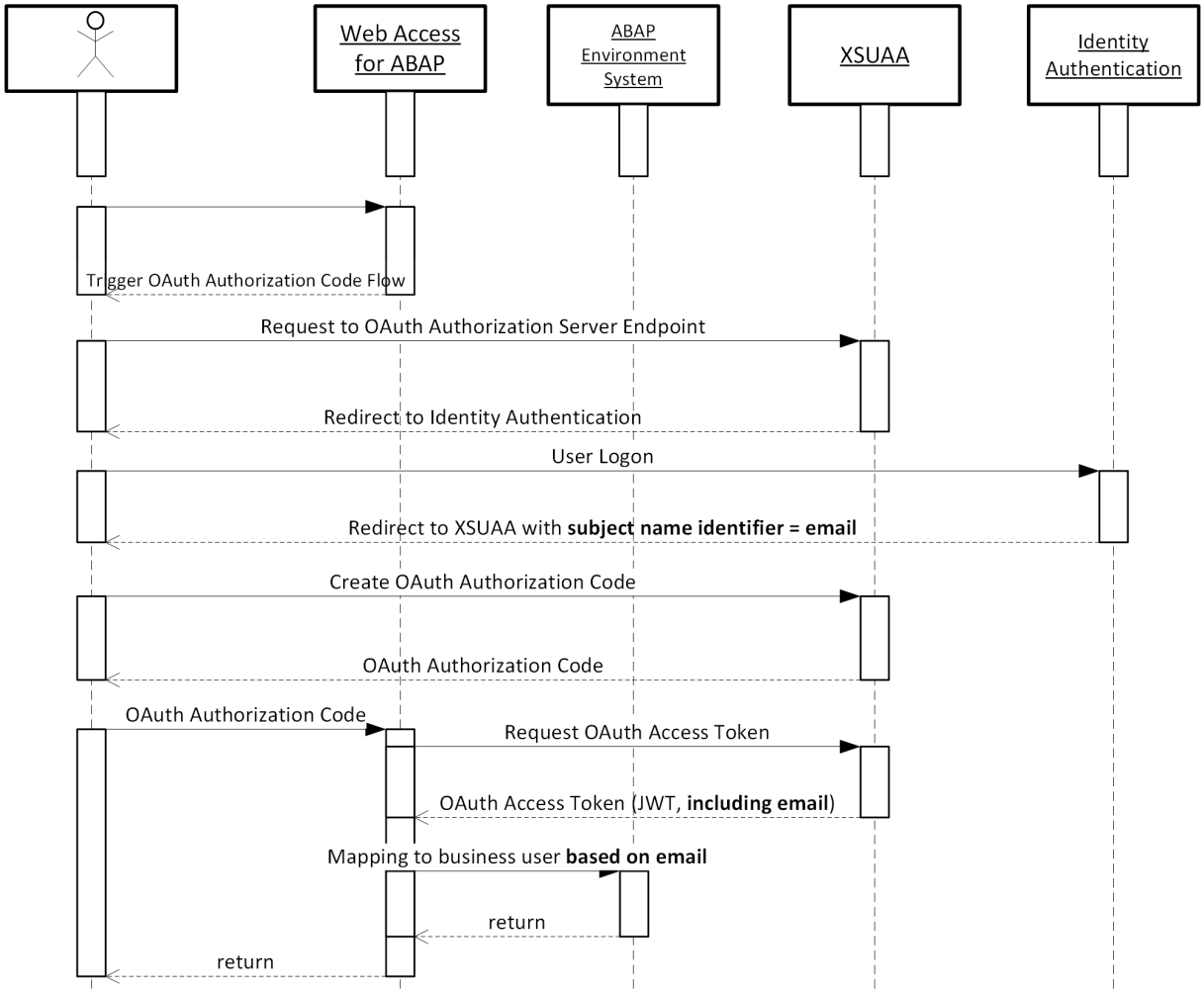
The Identity Authentication service is configured as an identity provider in the SAP BTP subaccount where the ABAP environment system has been created. The same subaccount is created as an application in the Identity Authentication service. See Configuring Applications.

This configuration establishes a mutual trust relationship between SAP Authorization and Trust Management service (XSUAA) and the Identity Authentication service as identity provider. See Trust and Federation with Identity Providers.

The ABAP environment system in turn trusts SAP Authorization and Trust Management service.

The business user is federated based on its e-mail address or user name, depending on the configuration of the login_attribute during system provisioning (See Creating ABAP System) and the subject name identifier in the Identity Authentication application (see Configure the Subject Name Identifier Sent to the Application).

If a resource in the ABAP environment system is requested to be accessed by a business user, the steps to acquire the required access token are as follows:



1. The business user accesses the ABAP environment system through the Web Access for ABAP or a standalone approuter application. If no access token is available, the OAuth 2.0 authorization code flow is triggered.
2. A request to the authorization server endpoint in the SAP Authorization and Trust Management service (XSUAA) checks if the business user is already logged on. If the business user is not logged on, the business user is redirected to the Identity Authentication service as the identity provider used for logon.
3. After logging on, the response of the Identity Authentication service, including the e-mail address of the business user as the subject name identifier, is redirected to the authorization server endpoint of SAP Authorization and Trust Management service (XSUAA) to retrieve the authorization code.
   The authorization code is necessary to retrieve the access token in the following steps. Finally, the authorization code is redirected again to the Web Access.
4. Based on the authorization code, a JSON web token (JWT) is retrieved from the access token endpoint in SAP Authorization and Trust Management service (XSUAA). This OAuth access token includes the e-mail address or login name of the business user that serves as a subject name identifier.
5. To consume resources, such as business services in the ABAP environment system, the JSON web token is forwarded as part of such requests and mapped internally to the business user.

> **Note**
>
> Authentication of users other than communication users does not require identity federation as these technical users are authenticated locally in the ABAP environment system.

4.2.11.1.2 User Types

SAP Business AI Platform, ABAP environment Users

In the ABAP environment, there are different types of users such as business users, communication users, and support users.

	Business User	Communication User	SAP Support User	Provider Support User
Owner	Customer	Customer	SAP	Provider
Purpose	Accessing business applications (SAP Fiori Applications) and using ABAP Development Tools	System to system communication	Analyzing and troubleshooting	Analyzing and troubleshooting consumer tenants
Authentication	Identity Provider	Local	Local	Local
Authorization	Business Role	Communication Scenario Role	Support Role	Access to Identity and Access Management and Communication Management apps
User ID	CB*	CC*	_SAP*	PS*
Access	`<host>.abap-web.<region>...`	`<host>.abap.<region>...`	`<host>.abap-support.<region>...`	<host>.abap-provider-web.<region>

Business users are customer-owned end users that use SAP Fiori applications as well as applications and services in SAP BTP, such as SAP Business Application Studio. They are authorized by being assigned a specific business role. See User Provisioning [page 983].

> **Note**
>
> Developers using ABAP Development Tools are also considered business users.

Communication users are customer-owned technical users that are assigned to a communication system to enable integration with other solutions. For authentication, they can either be assigned a password or X.509 client certificate. See Communication User and Maintain Communication Users [page 2993].

Support users are SAP-owned technical users that can access customer systems. See SAP Support User Request Log [page 3056].

Provider support users are technical users of the provider of a SaaS solution that can access consumer tenants in customer systems. See Create Support Users.

SAP BTP Users

In SAP BTP, there are platform and business users.

Platform users operate in the SAP BTP cockpit by deploying, administering, and troubleshooting applications and services. They are authorized by using role collections or by user assignment in the Cloud Foundry space as space or org members, such as space developers.

Business users use the applications that are deployed to SAP BTP and are authorized by using role collections.

To learn more about platform and business users in SAP BTP, see User and Member Management.

Related Information

Business Catalogs for IAM Administration Apps [page 3115]

4.2.11.1.3 User Provisioning

User provisioning is the automated process of creating users, that are later granted permissions to access business services and applications, to allow system to system communication, or to perform troubleshooting.

> ⓘ Note
>
> Please make sure that employee and business user data is maintained in accordance with the `login_attribute` system provisioning parameter (see Creating ABAP System) and the subject name identifier configured in the Identity Authentication application (see Configure the Subject Name Identifier Sent to the Application): If `login_attribute = email` is used, the email address at employee level needs to be maintained for identity federation to work, if login_attribute = user_name is configured, the username on business user level needs to match the login name provided in the identity authentication user.

Creating SAP Business AI Platform, ABAP environment Users

Each business user created in the ABAP environment system has a unique and stable employee ID. Employees are maintained in the SAP Fiori app *Maintain Employees*. See Maintain Employees [page 3016].

> → Recommendation
>
> We recommend setting the employee ID by an HR system, which allows you to update master data in the *Maintain Employees* app, for example, when a business user has a new email address because of a name

change, or if you want to edit the user information of the initial admin of an ABAP environment service instance.

The following provisioning methods to automate identity lifecycle processes for business users are available:

- SAP Fiori app *Maintain Employees* with CSV file upload. See Maintain Employees [page 3016].
- SOAP service using communication scenario Identity Management Integration `SAP_COM_0093`. See Inbound Service: Business User [page 1070] and Inbound Service: Business User - Read [page 1087].
- Identity Provisioning service using communication scenario SAP Cloud Identity Services - Identity Provisioning `SAP_COM_0193`. See Identity Provisioning - SAP BTP ABAP Environment as a Target System.

→ Recommendation

To create new business users in the ABAP environment, we recommend using the Identity Provisioning service.

This requires setting up the Identity Authentication service as a source system for identity provisioning and managing employee information in the user store. The Identity Provisioning service can then read this information and map the Identity Authentication logon name to the ABAP environment employee ID. See Identity Authentication as a Source System and Tutorial: Provision Users into your SAP BTP ABAP Environment.

To create **communication users**, see How to Create Communication Users [page 2995].

SAP support users are owned and created by SAP. However, in the *Display Technical Users* SAP Fiori app, you can check when and why SAP support users accessed your customer system in the past 12 months. See SAP Support User Request Log [page 3056].

Creating SAP BTP Users

To create **platform users** and **space members**, see User and Member Management and Creating New Space Members and Assigning Space Developer Roles to Them.

Related Information

What Is Identity Provisioning?

4.2.11.2 Access Management

Access management comprises the process of implementing and providing authorizations for your services.

Authorization Basics [page 985]
: If you haven't worked with an ABAP system before, you might find this introduction to the basic authorization concepts useful before you proceed.

Example: Authorizations for a Bonus Calculation Service [page 991]
Let's assume that you have a bonus calculation service, which, in the example used here, is represented by an active service binding. You want to model authorizations for this service.

Developer-Centric View on System Behavior [page 992]

Providing Access to a Business Service for Business Users [page 993]
For a newly created business service, you must define how business users can access it and which activities are allowed.

Providing Access to the SAP Fiori Application [page 1028]
Learn what developers and administrators need to do so that an SAP Fiori application appears in the SAP Fiori launchpad of an authorized business user.

Providing Access to a Business Service for Communication Users [page 1031]
For a newly created business service, you must define how communication users can access it and which activities are allowed.

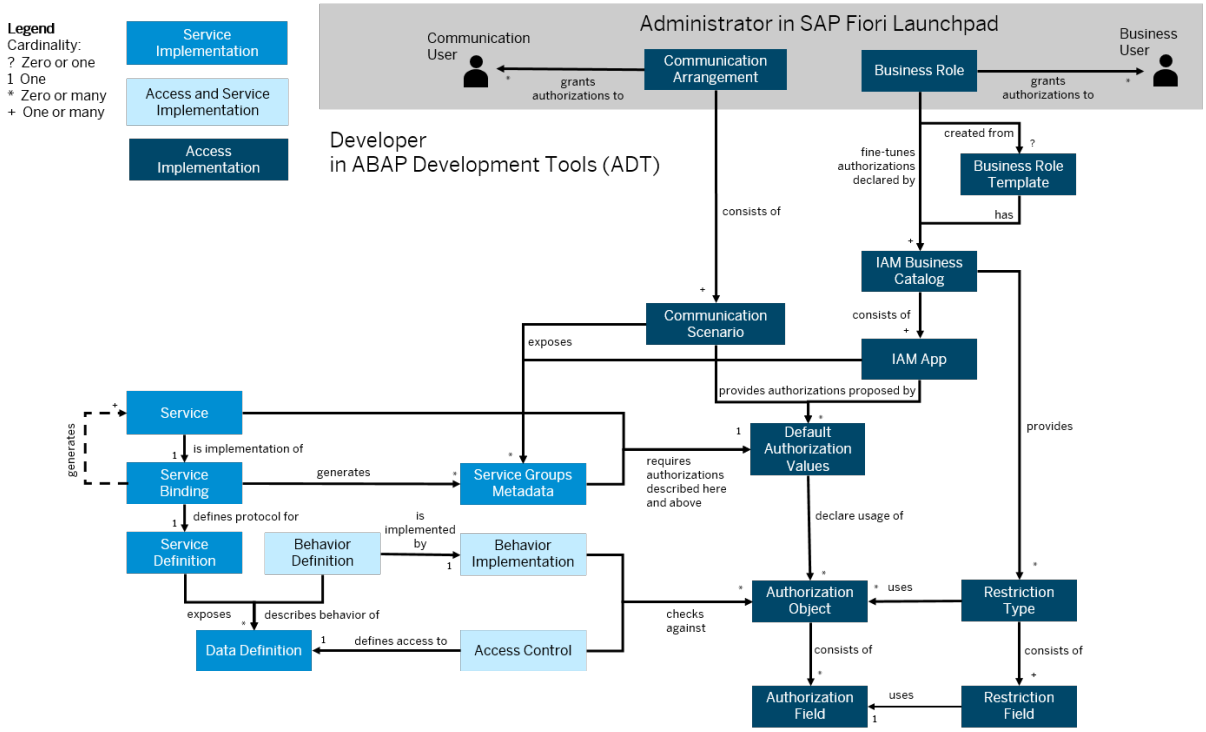
Providing Read or Write Access to a Business Service Using Privileged Mode [page 1042]
With enablement of privileged mode for a business object, it's possible to consume the business object without the need for additional business authorizations.

Troubleshooting [page 1044]
Learn more about apps and tools that help you identify and solve authorization issues.

4.2.11.2.1 Authorization Basics

If you haven't worked with an ABAP system before, you might find this introduction to the basic authorization concepts useful before you proceed.

Let's assume that you create business services. During the service implementation, you create a service definition, which exposes a data definition. In the ABAP RESTful Application Programming Model (RAP), you use behaviors to specify the operations and field properties of an individual business object. You can later enhance the behavior definition and implementation to include an authorization control. For more information about the ABAP RESTful Application Programming Model (RAP), see the documentation for the ABAP RESTful Application Programming Model. The documentation of the ABAP RESTful Application Programming Model also describes the service implementation, that is, the creation of all objects highlighted in medium blue in the following graphic.



To provide access to your business service, you also create additional development objects in ABAP Development Tools (ADT). These additional objects help define which users can eventually access which data with their authorizations. Such objects can be authorization objects, authorization fields, IAM apps, IAM catalogs, and so on (highlighted in light and dark blue in the graphic above).

Let's have a look at each object.

Authorization Field

An authorization field is the smallest unit in the area of authorizations. It's defined by a data element and its values; it's an attribute of a business object. An example of an authorization field is *Product Type*, with values such as *Material* or *Service* and the corresponding business object *Product*. In the example of a bonus management solution that is used in this guide, a possible authorization field is *Bonus Variant* with the values *Rate* and *Amount*.

A standard authorization field, which is already predefined in the system and which you can reuse to protect custom applications, is *Activity*. Like every authorization field, *Activity* has multiple values, such as create, update, delete, release, and so on, for which you can check the user's authorization.

You can assign a check table to each authorization field. This table will serve as a value help when the field is consumed during the configuration of authorizations in the business role maintenance.

Authorization Object

An authorization object groups up to 10 authorization fields that are related by `AND`. An authorization object ensures that complex authorization checks can run for multiple conditions. For example, you can create an

authorization object that defines which products users can access, such as products of type *Material*, of type *Service*, or both. Typically, you have the authorization field *Activity* in an authorization object because you want to define what kind of activities users can perform or not. Depending on the business context, you add more authorization fields, such as a purchase order type. In an authorization object, you also define which values of the *Activity* field you think are relevant.

After you have created authorization fields and authorization objects, you must implement checks against the authorization object to make sure that only authorized users can perform certain activities. During this check, the values specified in the program are compared with the values contained in the user's authorizations.

Authorization Check

An authorization check determines whether the current user of a program has a certain authorization. The check compares programmed (fix or actual runtime) values that are required for a specific authorization field within a specific authorization object against the granted authorizations of the current user. As a developer, you can implement authorization checks in ABAP coding using the statement `AUTHORITY-CHECK` or using access controls.

Access Control

ABAP Core Data Services (CDS) have their own concept for data protection and authorization: access controls. Access controls provide an easy way to implement protection against unauthorized read access based on field values.

For more information about access control, see Access Controls in the ABAP CDS Development User Guide.

Authorization Control

Authorization control in the RESTful Application Programming Model protects your business object against unauthorized access to data. You can use global and instance-based authorization control. Global authorization is used for all authorization checks that only depend on the user. You can define global authorization to check if users are allowed to execute an operation in general. Instance authorization is used for all authorization checks that additionally depend on the state of the entity instance in question. With instance authorization, you can define authorization that depends on a field value of the instance.

Authorization control is part of the behavior implementation of a business service.

For more information about authorization control, see the documentation for the ABAP RESTful Application Programming Model.

Default Authorization Values

You can define default authorization values for each authorization object and business service. These default authorization values are then automatically assigned when the business service is added to an IAM app or to a

communication scenario. Note, however, that the default authorization values can be overwritten by values set in the IAM app and by restrictions in the business role or communication scenario.

Restriction Field

A restriction field is based on an authorization field. It's used to restrict the access to instances of a business object, for example, a product. Restrictions are only available when access to business users is provided.

With restriction fields, you can empower an administrator to create multiple business roles with different values in the restriction field. Without restriction fields, you, as a developer, need to create a separate IAM app and a business catalog for each business role.

For example, an administrator wants to create different business roles for product master specialists who maintain products in the system. With a restriction field that contains values for the different available product types, you can fine-tune business roles in such a way that some product master specialists only get the authorizations for maintaining products of type *Materials* and others also of type *Service*.

Restriction Type

A restriction type is an authorization entity that bundles the available restriction fields according to a logical definition, for example, product type and product category. In a restriction type, you also define for which authorization object the restrictions are relevant. You can add restriction types to business catalogs.

For example, you can create a number of restriction fields that are relevant for creating business roles in product master data. You can, for example, create restriction fields such as *Product Type* and *Product Category*, and then bundle them using the restriction type *Product*.

IAM App

An IAM app is a development artifact that you can use to define necessary authorizations for business users for one or more services that you created in ADT. You create the IAM app in ADT, add the services, and define the necessary authorizations needed for the services. After that, you can include the IAM app in a business catalog, which then allows you to include the IAM app and its related authorization definitions into a business role.

IAM Business Catalog

In an IAM business catalog, you bundle multiple IAM apps and their predefined authorizations, for example, for a specific business area. The business catalog links what has been defined by development with settings done by administration. Business catalogs consist of IAM apps with their predefined authorizations, which can be reused in business roles. If you define an IAM business catalog with restriction types, you allow the administrator later to fine-tune the authorizations during business role maintenance.

You create an IAM business catalog in ADT. After you, as a developer, have published the business catalog in ADT, it's available in the SAP Fiori apps for business catalog and business role maintenance of the development system and can be used for user tests.

Business Role Template

You can define business role templates to make it easier for administrators to find the business catalogs that are relevant for the corresponding role in your company.

For SAP Business Technology Platform, role templates for administrators and developers are available: The business role template `SAP_BR_DEVELOPER` includes permissions for using ABAP Development Tools (ADT) and for creating the development objects mentioned here. The business role template `SAP_BR_ADMINISTRATOR` includes permissions for creating business roles.

Business Role

A business role is the administrative object that administrators use to group and fine-tune authorizations and assign them to business users. To create a business role, an administrator adds one or multiple IAM business catalogs to it. These IAM business catalogs contain predefined authorizations that allow users to access IAM apps. In business roles, administrators can set the authorizations for each access category, including available restrictions for different instances of a business object. If the IAM business catalog contains restrictions, administrators can also override authorizations for these restriction fields during business role maintenance.

Access Categories

An access category defines what kind of access is granted to the business users assigned to a business role, for example, read, write, or value help.

Each activity from the corresponding standard authorization field is typed with an access category, for example, the activity *Display* has the access category *Read* and the activity *Create* has the access category *Write*.

The activities for which permissions are assigned using a business role are defined in the IAM apps. The activities that a business user is allowed are the result of restriction settings for each access category in a business role and the corresponding activities defined in the IAM apps in the business role.

If the business role doesn't give write access, the *Create* activity, for example, isn't allowed. While it's possible to block write access, it's not possible to forbid read access completely in a business role because a *No Access* option isn't available. Therefore, read access is either allowed or, with the use of restriction fields, it's restricted by the allowed values of the restriction fields. In this case, display is only allowed for business object instances matching the allowed restriction field values.

Authorization Context

A CDS behavior definition can define authorization contexts that list multiple authorization objects. These authorization objects are used for the ABAP statement `AUTHORITY-CHECK OBJECT` in the implementation methods of the ABAP behavior pool of the business object.

The authorization contexts can be defined as follows:

- Own authorization context: Such an authorization context is defined using the statement `define own authorization context`.

  > → Recommendation
  >
  > We recommend that you define the own authorization context for all your custom business objects to document which authorization objects are checked in the logic of each business object.

- An authorization context with a specific name: Such an authorization context is defined using the statement `define authorization context`. With the authorization context with a specific name, you can define an authorization context that contains only a subset of the authorization objects from the own authorization context.

An authorization context can be useful, for example, to enable privileged mode for a business object (see Providing Read or Write Access to a Business Service Using Privileged Mode [page 1042]).

For more information, see the ABAP keyword documentation that you can find in the ABAP Development Tools.

Privileged Mode

With privileged mode in the behavior definition, RAP business object consumers can circumvent authorization checks, such as RAP authorization control or CDS access control. After enabling privileged mode, privileged read and write access to the business object is possible. In this case, the authorization checks for all listed authorization objects always return the value `authorized`.

For more information, see the ABAP keyword documentation in the ABAP Development Tools and Authorization Control in the guide to the ABAP RESTful Application Programming Model (RAP).

Communication Scenario

A communication scenario in the ABAP environment is a development artifact that describes how two communication partners communicate with each other. It provides technical information, such as the used inbound and outbound services and their service type, for example OData or SOAP, and the number of allowed communication arrangement instances. It is also the place where you define the required authorizations that a communication user must have to use the services of the communication scenario.

For more information, see Communication Scenario.

Communication Arrangement

A communication arrangement in the ABAP environment is a runtime instance of a specific communication scenario. It grants the authorizations for the inbound and outbound services of the communication scenario to communication users. It grants the authoriziations only from and to a specified partner, the so-called communication system. The administrator of the ABAP environment creates the communication arrangement in the SAP Fiori launchpad.

For more information, see Communication Arrangement.

Putting It All Together: Authorization

A user's authorization is influenced by the combination of permissible values in each authorization field of an authorization object. An authorization enables a user to perform an activity in the system, based on a set of authorization field values. However, the authorization object isn't the only entity in the system that influences what a user is allowed to do. IAM apps, communication scenarios, business roles, and restrictions in business roles can still change what was defined in the authorization object.

4.2.11.2.2 Example: Authorizations for a Bonus Calculation Service

Let's assume that you have a bonus calculation service, which, in the example used here, is represented by an active service binding. You want to model authorizations for this service.

The Bonus Calculation Service

The bonus calculation service is part of a sales bonus program where the sales revenues that sales reps were able to generate are used as the basis for their bonus plan. In the ABAP RESTful Application Programming Model (RAP), the bonus plan is modeled as a business object *Bonus Calculation* of type managed.

With the bonus calculation service, users, for example, managers in sales, can create, update, display, and delete bonus calculations as standard operations. As part of a bonus calculation, users enter an employee name, a bonus variant and validity period can be set, and the bonus be calculated, which is a non-standard, business object-specific activity. In the example used in this guide, the available bonus variants are based on *Rate* or *Amount*, that is, the bonus recipient either gets a bonus based on a rate or a fixed amount.

The service can be consumed either by business users via a user interface (UI) or by technical communication users as a Web API. For both consumption types access has to be provided at least for nondevelopment systems because this access isn't automatically available in such systems.

Consumption in a UI for Business Users

If the business service is consumed by a UI, the available UI enables business users easily, depending on their authorizations, to set data or to execute the actions the service is offering. For example, a UI for managers lets them perform activities on bonus calculations. The UI wraps the underlying service calls for the business users into intuitive UI elements. Changes that the business user isn't authorized for can either be made unavailable on the UI or their corresponding service call is done and an appropriate error returned from the service. The UI can act as a kind of authorization filter. This kind of service consumption always requires business user action.

Consumption as API with a Communication User

If the business service is consumed as API, no UI wraps the allowed changes to the business service. Instead, the service caller can try to change all data and execute all actions directly and always gets errors if there are missing authorizations to do it.

In the example used here, the bonus calculation is an inbound business service that business users use to create new bonus calculations. There's no dedicated bonus calculation in the HR system, but instead, the HR business service calls the bonus calculation business service in the ABAP environment to set data and execute actions.

Where you work with an API business service (inbound or outbound) from or into the ABAP environment, you need to provide authorizations to technical users, so-called communication users, instead of business users. This service consumption can be performed automatically without any user action.

4.2.11.2.3 Developer-Centric View on System Behavior

If you have the developer role `SAP_BR_DEVELOPER`, you can e.g. test your new OData V4 service instantly without needing to create an IAM app and link it to a business catalog and role. By publishing the service binding, an IAM app is created and automatically added to the `SAP_CORE_BC_EXT_TST` business catalog, which is provided to customer developers for testing. This catalog is associated with the `SAP_BR_DEVELOPER` role template. Consequently, when a new service is developed in a system and its service binding published, users with the developer role gain immediate access to the service. Additionally, you can gain immediate access to the service with the developer role. However, this access is not available in non-development systems, where this test catalog feature isn't enabled.

Using Custom Authorization Objects

When implementing a custom authorization check in your own service as a developer, you face the challenge of being unable to test the service due to insufficient permissions. To overcome this issue, you must include the created authorization object (AO) in the automatically generated authorization default values. An overview of when the authorization default values are created can be found here. In this context, there is a restriction: Not every service can be published, which impacts when an authorization object is added to the developer role and how you can ultimately test the service.

The following services can be published:

- SAP Gateway OData V2/V4
- HTTP service
- RAP event binding

The following services can not be published:

- Application job catalog entry & application job template
- SQL service binding
- InA service function module

If you have a service that can be published, republish it to make it available. If your service cannot be published, use another service that can be published to assign the authorization object to the generated IAM app.

Deleting Authorization Objects

To remove an authorization object that is already part of the authorization default values, you must first delete the associated service, which will result in the deletion of the authorization default values. It is not possible to directly delete the authorization object from the default values. After the relevant service is deleted, a local publish of another service must be performed. The publishing of another service results in the removal of the authorization object assignment from the IAM app that was previously linked to the service. Once this is done, the authorization object can be deleted.

Creation of Authorization Default Values

Authorization Default Values Created by the Creation of the Service	Authorization Default Values Created by the Activation of the Service
• HTTP service • application job catalog entry / application job template • SQL service binding	• SAP gateway OData V2/V4 • RAP event binding • function module

4.2.11.2.4 Providing Access to a Business Service for Business Users

For a newly created business service, you must define how business users can access it and which activities are allowed.

Motivation

If you have the developer role SAP_BR_DEVELOPER, for example, you can test your new service immediately without having to create an IAM app and assign it to a business catalog and a business role. Activating the service binding, for example, creates the IAM app and assigns it to the business

catalog SAP_CORE_BC_EXT_TST that is shipped to customers for testing purposes. This business catalog is assigned to the developer role template SAP_BR_DEVELOPER. Therefore, when you create a new service in a development system and activate the service binding, all users with this developer role can automatically access and use the service. Of course, all users with another role containing this catalog are then also able to access and use the service.

This kind of access isn't available anymore when you pull the business service into nondevelopment systems, for example, where this test catalog feature isn't enabled.

For business users, you must provide access to a newly created service because access isn't automatically available. However, all activities are allowed as long as you don't protect your business service. Therefore, you must develop identity and access management artifacts, which is explained in detail in this guide. They ensure that you can define the necessary authorizations and enable business users to consume the business service.

Overview of Development Options

Now, let's assume you would like to achieve that certain business users are able to view and edit bonus calculations:

- Under *Providing Unrestricted Access*, you learn how to provide users read and write access to all bonus calculations.
  This scenario is simple to implement. It's enough if you have one user group that is allowed to make full use of the business service while all other users don't need access, not even read access. In this case, if managers have access to the business service, they can create, update, display, and delete all available bonus calculations.
- Under *Providing Access Based on Activities*, you learn how to create two business roles for separate read and write authorizations.
  With this scenario, you can keep business users with read access only from changing bonus calculations.
- Under *Providing Access Based on Field Values*, you learn how to fine-tune your access restrictions to particular fields, that is, on the level of data elements, such as the bonus variant.
  This is the most complex scenario, but it allows administrators to define business roles that authorize users to view and change only particular data; other data is filtered. For the bonus calculation service, managers can only create, view, update, and delete bonus calculations that have a specific bonus variant. The advantage of this scenario is that you, as a developer, can delegate some decisions about the business role design to the administrator. The administrator can decide which data is visible to business users, based on restriction fields in the business role.

After you've implemented access based on one of the scenarios mentioned, you can then also go on and implement access to the corresponding SAP Fiori application for the business service.

Tutorial

In addition to this documentation, the following tutorial is also available:

Create Authorization Model with SAP BTP ABAP Environment

4.2.11.2.4.1 Considerations for Implementing Access and Authorizations

You can implement data access and authorizations for a service differently, depending on how you want to implement differences on the UI for different target groups. It also matters how you want to design business roles. Therefore, before you start with the actual implementation of data access and authorizations, you need to consider how you want to proceed.

There are different points in application development where you can start separating data access for different user groups. As a result, you get different business roles that are assigned to business users.

Different Services for Different User Groups

With this approach, you create different services that grant different data access, depending on the user group and its corresponding role. Here, you already consider the required data access for the various user groups when you implement the services. As a result, you also implement different UIs, authorization defaults, IAM apps, and business catalogs, at least one per service. That way, you invest more effort in service development, but less in UI development because it's already clear from the data exposed by the service what the UI will provide.

Since you create different services for the user groups, system administrators would then also assign different business roles to business users that authorize them for one of the services, depending on the user's tasks.

Different UIs for Different User Groups

If different user groups require a different structure or layout, another option is to implement different UIs for different user groups. While you need to invest more time to implement many separate UIs, especially if you need many of them, the effort for each UI is lower in comparison to dynamic UI handling.

A typical use case would be to use service bindings on different projections for the same business object. For example, a business object *Leave Request* has *Create*, *Delete*, and *Approve* activities. There could be two projections: one for the employees that allows them to create and delete leave requests, whereas another projection is for the manager who approves them.

If you decide to create different UIs for different user groups, this means you need to create an IAM app for each UI. Each IAM app needs to be included into a separate business catalog, which in turn is then included into a separate business role. As a result, an administrator creates a different business role for each UI.

Dynamic UI Handling

With dynamic UI handling, you implement a UI that changes dynamically, depending on the user group. For example, while one user group has full access to all data, another user group can only see data in display mode and actions are disabled, or data and possible actions aren't even shown. Dynamic UI handling is useful if there are only small UI differences for different user groups.

For example, you implement an app that calculates bonuses for sales representatives. With dynamic UI handling, you can define a business role for a user group where users can view all bonus calculations and create, change, and delete bonus variants. Other user groups can only display the bonus calculations (possibly only for one bonus variant), but buttons for creating, changing, or deleting bonus variants are grayed out.

The development of a dynamic UI can result in more effort because you need to model and implement authorization access, for example. However, you also save efforts because you only implement one UI for all user groups and thus you avoid redundancies that arise when you maintain multiple UIs for different user groups.

In the ABAP RESTful Application Programming Model (RAP), you realize dynamic UI handling using dynamic behavior implementation. For more information about dynamic behavior implementation, see Implementing a Protection Against Unauthorized Write Activities [page 1019].

Business Role Design

In addition to the choices you make for the implementation of different UIs for different user groups, you also need to consider how business roles need to be implemented for your services. Depending on the use case of your service or services, you want the administrator to create different business roles. It's crucial to understand that the possible design of business roles depends on the choices you make when you implement the service, its UI, and the authorizations.

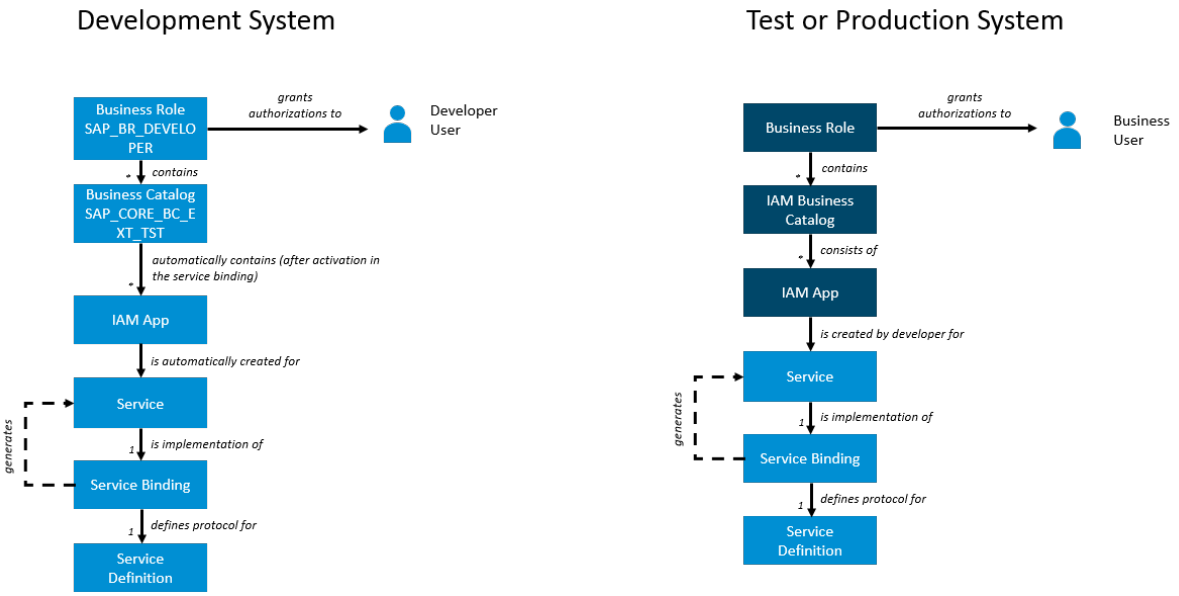
One possible approach is that you envisage business roles that can differentiate between the access categories of read, write, and value help (F4). This is possible using different business roles and doesn't require different authorization implementations.

The exception is when you need a finer distinction of activity authorizations, for example, between create/update and delete. In this case, you must implement different IAM apps for your service. On the basis of these IAM apps, you create different business catalogs, which the administrator uses to create different roles.

4.2.11.2.4.2 Providing (Unrestricted) Access for Business Users

In this scenario, you get information about how to provide unrestricted access with a few steps.

In this scenario, to provide access to the business service, you must create an IAM app and assign it to a business catalog. Administrators then create a business role that contains the business catalog and assigns the role to business users. Note that all objects that need to be created are highlighted in dark blue in the graphic.



This scenario requires only a few steps to provide authorizations, and it's the minimum that you must do to make your service available to business users. However, with this scenario, you always provide unrestricted write **and** read access. If you want to fine-tune the authorizations in more detail, check the scenarios *Providing Access Based on Activities* and *Providing Access Based on Field Values*.

Related Information

Providing Access Based on Activities for Business Users [page 1000]
Providing Access Based on Field Values for Business Users [page 1011]

4.2.11.2.4.2.1 Developing Authorizations for Business Users (Developer)

For providing unrestricted access to a business service for business users, as a developer, you first create an IAM app, a business catalog, and, optionally, a business role template.

4.2.11.2.4.2.1.1 Creating an IAM App for the Business Service

As a developer, you need to define an IAM app for the business service that you created in ABAP development tools (ADT). Creating an IAM app is the first step to be able to create a business role for your business service.

Context

You create the IAM app in ADT. After the IAM app has been created, you can include the IAM app in a business catalog, which, in turn, can be included in a business role. This business role then can be used to control access to the app. For more information about IAM apps, see the user guide for ABAP development tools.

Instead of creating a new app, you can also use an existing IAM app for the service.

Procedure

1. Follow the procedure in the user guide for ABAP development tools.

   > ⓘ Note
   >
   > In this scenario, there are no authorizations that you need to change on the *Authorization* tab page.

2. Save and publish the newly created IAM app locally.

4.2.11.2.4.2.1.2 Creating a Business Catalog

To be able to assign an IAM app to a business role, you first need to include the IAM app in a business catalog.

Context

In a business catalog, you can bundle several IAM apps for which users should get authorized. You can assign a business catalog to a business role template.

In this simple scenario, no predefined, detailed authorizations are available for the IAM app, but you still need to create a business catalog. By including the IAM app in the business catalog, you simply create the prerequisites for allowing users a full access to your business service once they have the business role with the business catalog assigned.

Instead of creating a new business catalog, you can also reuse an existing business catalog.

> → Recommendation
>
> We recommend that you add the information to the business catalog description that the IAM app belongs to an unprotected service, so that this important information is available to administrators.

Procedure

To create a business catalog, follow the instructions in the user guide for ABAP development tools (ADT)..

Also follow the instructions to assign your IAM app to the business catalog.

4.2.11.2.4.2.1.3 Creating a Business Role Template (Optional)

In addition to a business catalog, you can also create a business role template with ABAP Development Tools (ADT). Creating a business role template is optional, but it can ease the business role creation for the administrator.

Context

Creating a business role template especially makes sense if you have more than one business catalog to assign to a business role.

Procedure

1. Create a business role template as described in Creating a Business Role Template.
2. After the business role template has been created, add the business catalog to it.
3. Choose *Publish Locally* to make the business role template available for use in the development system.

4.2.11.2.4.2.2 Creating a Business Role and Assigning Business Users (Administrator)

As an administrator, to be able to grant authorizations to business users, you need to create a business role that you then assign to the business users.

Context

Perform this task in every system. Alternatively, you can also create a business role once, download it, and upload it to other systems. For more information, see How to Download and Upload Business Roles.

Procedure

Follow the instructions of the documentation to create a business role from a template or from scratch (but omit the steps for maintaining instance-based restrictions and for managing launchpad space):

- How to Create a Business Role from Scratch
- How to Create a Business Role from a Template

> ⚠ Caution
>
> Even if you leave the write restrictions to their default *No Access*, business users still have write access with this business role in this scenario. Users always have full access because you haven't implemented an authorization check in the service behavior.
>
> We recommend that you set the restrictions in the business role to unrestricted write authorizations to make it transparent to other administrators that this business role provides users with full read and write access.

Results

Users in the system have full read and write access to the service.

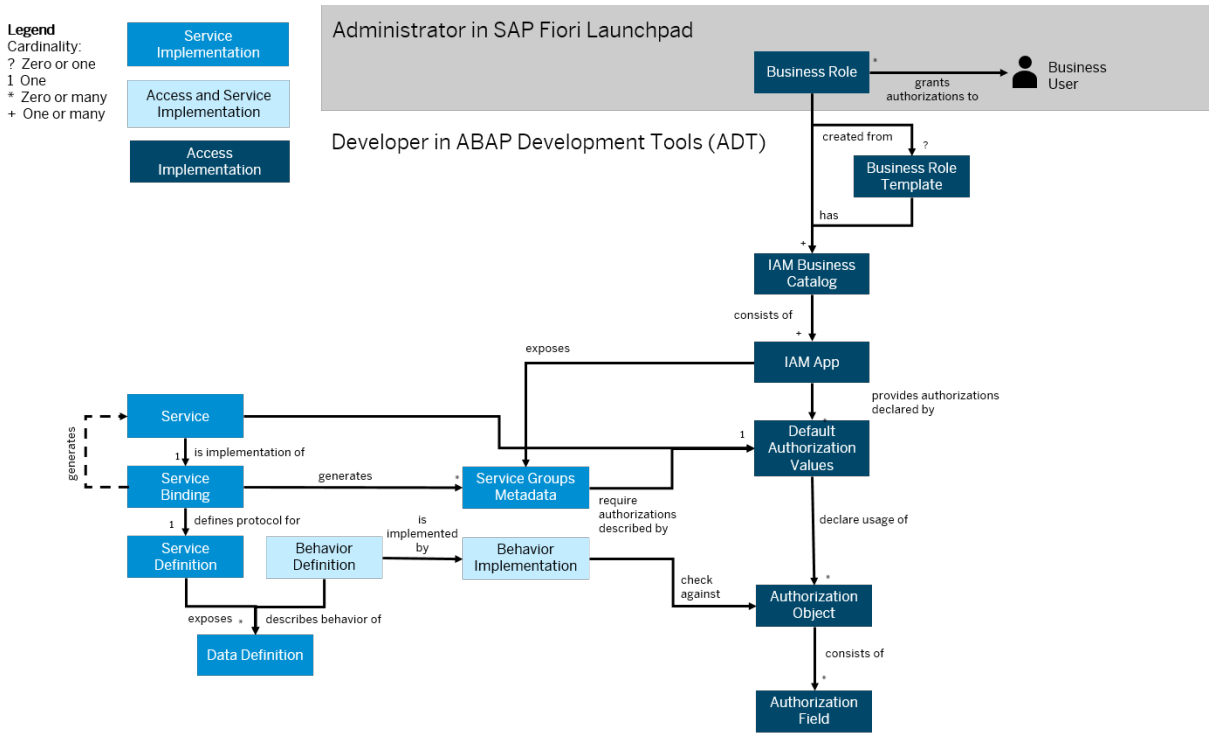
4.2.11.2.4.3 Providing Access Based on Activities for Business Users

In this scenario, you provide access depending on what the user should be allowed to do, for example, read or write access.

This scenario is more complex than a simple unrestricted access implementation because you must also create an authorization object and implement authority checks in the behavior implementation. However, you

benefit from the extra efforts because you can enable the administrator to create separate business roles for different activities, that is, based on the authorization field ACTVT (*Activity*) for create, read, update, and delete.

Overview



In the following, you get a more detailed overview of how to implement a separate read and write access. You can achieve such a separation by creating an authorization object with the standard authorization field ACTVT (*Activities*), an authority check to protect the service, an IAM app, and a business catalog. The administrator can then create two business roles for read and for write access each. If you want to enable the administrator to create more fine-tuned business roles, for example, to differentiate between the activities of create and delete, you proceed in a similar fashion, but you need to create two IAM apps (see also Considerations for Implementing Access and Authorizations [page 995]).

Process

To create access protection and provide authorizations, you perform the following steps:

Protecting the Service from Unauthorized Use

As a developer, you perform the following steps in ABAP Development Tools (ADT):

1. Create an authorization object with standard authorization field *Activity* (ACTVT) and permitted values 01,02,03,06.
2. Implement authority checks in the behavior implementation of the service to allow create, update, and delete activities only to users who have the authorizations of the authorization object.

As a result, users cannot create, update, or delete the relevant business objects anymore, for example, bonus calculations.

Developing Authorizations for the Service

As a developer, you perform the following steps in ADT:

1. Add the required authorization objects to the list of default authorization values.
2. Create an IAM app and assign the service binding to it. Under *Authorizations*, select all activities (*01 Add or Create*, *02 Change*, *03 Display*, *06 Delete*) to allow them for business users.
3. Create a business catalog and assign the IAM app to it.
4. Optional: Create a business role template and assign the catalog to it.

Granting Authorizations to Business Users

As an administrator in the ABAP environment, you can now grant authorizations to business users for this service:

1. Create a business role. For business roles with write access, set unrestricted write authorization. For read-only business roles, keep the default, which is no access.
2. Assign the business role to the business users.

As a result, business users can view, create, update, or delete bonus calculations again, depending on their business roles.

4.2.11.2.4.3.1 Protecting the Service (Developer)

With creating an authorization object for your service and with the implementation of a protection against unauthorized creation and change activities, you implement a basic protection of your service.

After this, you can implement how authorizations are granted to business users.

Related Information

Developing Authorizations for Business Users (Developer) [page 1006]

4.2.11.2.4.3.1.1 Defining Permitted Activities in an Authorization Object

To define the permitted activities for a service, create an authorization object with the standard authorization field *Activity* (ACTVT) for the permitted activities.

Procedure

To define the permitted activities, follow the procedure for defining authorization objects in the user guide for ABAP development tools (ADT).

> **Note**
>
> In our example, in the newly created authorization object, under *Authorization Fields*, you only need the predefined *Activity* authorization field to define the permitted activities for the business users of your service. In our example, these activities would be *01 Add or Create*, *02 Change*, *03 Display*, and *06 Delete* for standard activities.In addition, you need *93 Calculate* for the business object-specific activity *Calculate Bonus*. If you click on an empty row and choose ENTER, the list of available values is shown.
>
> In theory, by using the syntax `authorization: update` on an operation in the entity's behavior definition, you could use the authorization check that is implemented for the update operation for the annotated operation. This is also referred to as “authorization delegation”. However, for security reasons, we recommend that you carefully consider whether you want to use authorization delegation because in this case, a user who is allowed to update a business object is then also allowed to perform all other activities on the business object.

Results

In the following, the newly created authorization object for the bonus calculation example is called `ZBNSCLC_AO`.

4.2.11.2.4.3.1.2 Modeling a Protection Against Unauthorized Write Activities

To protect your service from activities such as create, update or delete by unauthorized users, you can use the authorization controls that are available for services based on managed business objects.

Context

In the example used here, the service is based on a managed business object *Bonus Calculation*. The use of a managed business object allows you to benefit from the global authorization for managed business objects,

which is part of the ABAP RESTful Application Programming model. For more information about authorization control, see the documentation for the ABAP RESTful Application Programming Model.

Procedure

1. In ABAP development tools, call up the behavior definition of your service.
2. In the behavior definition, define authorization controls for your service.

   In the bonus calculation example, they can be as follows:

   Sample Code

   ```
   managed;
   define own authorization context
   {
     'ZBNSCLC_AO';
   }
   define behavior for z_i_bonus_calculation alias calculation
   implementation in class zbp_bonus_calculation unique
   persistent table zbonusclc
   lock master
   // Identity and Access Management:
   // Enable RAP managed authorization check at create, update, and delete
   // -> requires implementation of two methods, one with the addition "FOR
   INSTANCE AUTHORIZATION"
   //  and one with the addition "FOR GLOBAL AUTHORIZATION"
   authorization master ( instance, global )
   ```

   With the line `authorization master ( global )`, an authorization check for the standard operations `create`, `update`, and `delete` and for non-standard operations, for example, `calculate bonus`, is defined. You must implement the authorization check using a method with addition `FOR GLOBAL AUTHORIZATION` in the behavior implementation.

   The sample code also contains the definition of your own authorization context. The own authorization context documents all authorization objects that are used by the business object implementation. In the bonus calculation example used in this documentation, this is the authorization object `ZBNSCLC_AO` because it's checked in the authorization control. The own authorization context is also used to prefill authorization defaults.

4.2.11.2.4.3.1.3 Implementing a Protection Against Unauthorized Write Activities

To protect a service such as the example of bonus calculation against unauthorized bonus creations, updates, or deletions, enhance the behavior implementation of the service in ABAP development tools (ADT). The implementation is required by what has been defined in the behavior definition.

Context

Among the methods of the behavior implementation, you add methods that check the authorizations for creation and for update and delete.

Procedure

1. In the *Source Code Library* folder of your package, under *Classes*, choose the behavior implementation of your service.
2. In the `PRIVATE SECTION` in the handler class of the behavior implementation, define the methods for authorization checks for creation, update, and deletion.

   Sample Code

   ```
   METHODS get_global_authorizations FOR GLOBAL AUTHORIZATION
     IMPORTING REQUEST requested_authorizations FOR <behavior definition or
   its alias> RESULT result.
   ```

   In the bonus calculation example, these method definitions could be the following:

   Sample Code

   ```
   METHODS get_global_authorizations FOR GLOBAL AUTHORIZATION
     IMPORTING REQUEST requested_authorizations FOR calculation RESULT
   result.
   ```

3. Add a method implementation that checks the authorizations for creations, updates, and deletions.

   Note

   For more information about authorization control, see the documentation for the ABAP RESTful Application Programming Model.

   In the bonus calculation example, this method could be the following:

   Sample Code

   ```
   METHOD get_global_authorizations.

     IF requested_authorizations-%create EQ if_abap_behv=>mk-on.
   ```

```
*     check create authorization
      AUTHORITY-CHECK OBJECT 'ZBNSCLC_AO' ID 'ACTVT' FIELD '01'.
      result-%create = COND #( WHEN sy-subrc = 0 THEN
      if_abap_behv=>auth-allowed ELSE
      if_abap_behv=>auth-unauthorized ).
    ENDIF.
    IF requested_authorizations-%update EQ if_abap_behv=>mk-on.
*     check update authorization
      AUTHORITY-CHECK OBJECT 'ZBNSCLC_AO' ID 'ACTVT' FIELD '02'.
      result-%update = COND #( WHEN sy-subrc = 0 THEN
      if_abap_behv=>auth-allowed ELSE
      if_abap_behv=>auth-unauthorized ).
    ENDIF.
    IF requested_authorizations-%delete EQ if_abap_behv=>mk-on.
*     check delete authorization
      AUTHORITY-CHECK OBJECT 'ZBNSCLC_AO' ID 'ACTVT' FIELD '06'.
      result-%delete = COND #( WHEN sy-subrc = 0 THEN
      if_abap_behv=>auth-allowed ELSE
      if_abap_behv=>auth-unauthorized ).
    ENDIF.
  ENDMETHOD.
```

Results

The implemented methods provide the desired protection from unauthorized uses:

- With the implementation of the method defined with the addition `FOR GLOBAL AUTHORIZATION`, an unauthorized business user gets an error message when trying to create a business object instance (in the example, a bonus calculation) or when trying to update or delete it via service call.
- Unauthorized business users can't use the *Delete* button in the list view on the service UI. In addition, the *Edit* and *Delete* buttons are hidden in the detail view on the service UI for a business object instance.

Now, the service is secured against unauthorized operations. As a next step, you must make sure that business users who need to use the service get the corresponding authorizations.

4.2.11.2.4.3.2 Developing Authorizations for Business Users (Developer)

After you've protected your service against unauthorized use, you can now create the objects that are needed to grant authorizations for business users: authorization default values, an IAM app, a business catalog, and a business role.

Follow the steps for editing authorization default values and for creating an IAM app, a business catalog, and a business role template. You should define `own authorization context` in the behavior definition first, which can be used to prefill authorization defaults.

> **Note**
>
> For developing authorizations for communication users, the procedure is different. For more information, see Developing Authorizations for Communication Users (Developer) [page 1039].

4.2.11.2.4.3.2.1 Editing Authorization Default Values

Authorization default values are automatically created when you create a service binding. You can add authorization objects and change the default authorization values.

Context

The authorization default values are the authorizations that are assigned to business users by default if they have a business role based on the service. Note, however, that the authorization default values can be overwritten by values set in the IAM app. Therefore, if you want to avoid double maintenance, try and define the authorization default values in such a way that they can be used by all derived IAM apps.

For more information about authorization default values, see the user guide for ABAP development tools.

Procedure

1. Open the service binding and choose the *Maintain Authorization Default Values* link.
2. Choose *Synchronize* to add the latest authorization objects from the own context.

   > ⓘ Note
   >
   > The *Synchronize* function only **adds** authorization objects that are not yet part of the list of authorization objects; it does not remove objects.

3. To check or change the authorization default values that are assigned automatically, select the authorization objects in the list.

   You can now specify what activities you want to authorize.
4. Choose the authorization object from the list and choose *Default With Field Values* from the dropdown list.

   In our example, we authorize for the standard activities *Create*, *Display*, *Change*, and *Delete* as well as the business object-specific activity *Calculate*.

Results

Now, the standard authorizations for the service are defined and will be applied to an IAM app based on the service. You still can change them later in the IAM app itself.

Related Information

Defining an IAM App for the Business Service [page 1008]

4.2.11.2.4.3.2.2 Defining an IAM App for the Business Service

To assign a business user to a business role for your service, you need to create an IAM app, which can then be included into a business catalog, which, in turn, can be assigned to a business role.

Context

Since you want to make your service available to users with a certain business role, you need to define an IAM app for the service that you created in ADT. The IAM app is only relevant for identity and access management (IAM), and you create the IAM app in ADT. You can use an IAM app to define authorizations for one or more business services.

Instead of creating a new IAM app, you can also use an existing IAM app for the service.

For more information, see the information about IAM apps in the user guide for ABAP development tools (ADT).

Procedure

To create an IAM app, follow the instructions in the user guide for ABAP development tools.

For the bonus calculation IAM app used as an example here, you have already specified before in the authorization default values what you want to authorize. If needed, you can overwrite these defaults again here.

> ⓘ Note
>
> In this example scenario, we assume that you create only one IAM app for access using all permitted activities and their access categories (write, read, value help). Be aware at this point that in the business role Fiori app, the administrator can later separate authorizations depending on the access categories (write, read, or value help). Therefore, there's no need to develop different types of access if you just want to separate write, read, and value help.
>
> However, it's different if you want to have different business roles for activities within one access category, for example, one for the create and update activities and another for the delete activity, which are all write activities. In this case, you must start here to separate authorizations: Create separate IAM apps, one for each write activities group that should be separated, go on with separate business catalogs, and finally enable the administrator to create separate business roles based on the business catalogs.

4.2.11.2.4.3.2.3 Creating a Business Catalog

To make predefined authorizations for your business service available to administrators to use, create a business catalog.

Context

In the business catalog, you can bundle multiple IAM apps and their predefined authorizations. You assign a business catalog to a business role. If you have more than one business catalog for the same business role, you might also consider creating a business role template.

Instead of creating a new business catalog, you can also reuse an existing business catalog for the app.

Procedure

To create a business catalog, follow the instructions of the ABAP development user guide: Creating a Business Catalog.

Also follow the instructions to assign your IAM app to the business catalog.

4.2.11.2.4.3.2.4 Creating a Business Role Template (Optional)

In addition to a business catalog, you can also create a business role template with ABAP Development Tools (ADT). Creating a business role template is optional, but it can ease the business role creation for the administrator.

Context

Creating a business role template especially makes sense if you have more than one business catalog to assign to a business role.

Procedure

1. Create a business role template as described in Creating a Business Role Template.
2. After the business role template has been created, add the business catalog to it.
3. Choose *Publish Locally* to make the business role template available for use in the development system.

4.2.11.2.4.3.3 Creating a Business Role and Assigning Business Users (Administrator)

To be able to grant authorizations to business users, you need to create a business role that you then assign to the users.

Context

You can create a business role from a business role template or from scratch.

Perform this task in every system. Alternatively, you can also create a business role once, download it, and upload it to other systems.

Procedure

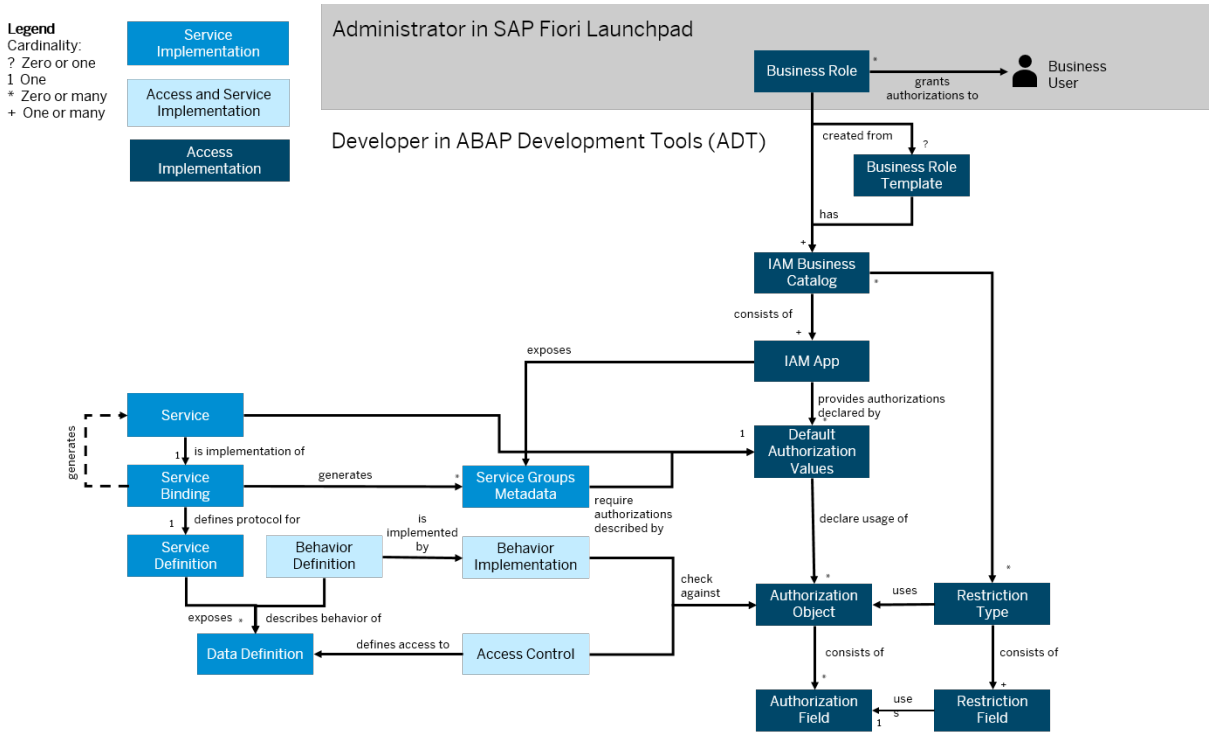
1. To create a business role, proceed as follows:
   - To create a business role from scratch and assign business users, follow the instructions in How to Create a Business Role from Scratch.
   - To create a business role from a business role template and assign business users, follow the instructions in How to Create a Business Role from a Template.
2. For roles with write access, set unrestricted write authorization. For read-only roles, leave the write restrictions as they are (the default is no access).

   For more information about restrictions, see How to Maintain Restrictions.

4.2.11.2.4.4 Providing Access Based on Field Values for Business Users

You can enable access to a service in such a way that it's dependent on the field values of a business object. As a result, business users can view or change only business object instances where the field values match their authorizations.

Overview



The central concept in this scenario is the restriction field: In a restriction field, you can expose an existing authorization field as a restrictable field in a business role. You first create an authorization object and implement an access control and authority checks in the behavior implementation to protect your service from unauthorized use. Then you create an IAM app and a business catalog to make it available to administrators, who want to include the service (or, rather, the corresponding business catalog) into a business role. With the restriction field, you as a developer, can define which authorization fields should be available for the administrator during business role maintenance. Administrators can then decide which values of the restriction fields are relevant for some business roles. In the example of the bonus calculation service, administrators can create business roles to restrict the access for bonus calculations to selected bonus variants only.

Process

To create access protection and provide authorizations, you perform the following steps:

Protecting the Service from Unauthorized Use

As a developer, you perform the following steps in ABAP Development Tools (ADT):

1. Create an authorization field. For example, create an authorization field `ZBNS_VARN` for the bonus variant.
2. Create an authorization object with the standard authorization field `ACTVT` and other authorization fields if needed, for example, bonus variant.
3. Implement an access control as a read protection for the service.
4. Model authority checks in the behavior definition of the service.
5. Implement authority checks in the behavior implementation of the service to allow create, update, and delete activities only to users who have the authorizations of the authorization object.

As a result, no user can access the service to display bonus calculations or create, update, or delete bonus calculations anymore.

Developing Authorizations for the Service

As a developer, you perform the following steps in ADT:

1. Add the required authorization objects to the list of default authorization values.
2. Create an IAM app and assign the service binding to it.
3. For each authorization field that you want to expose and consider in a business role, create a corresponding restriction field.
4. Assign the restriction fields to a restriction type.
5. Create a business catalog and assign the IAM app to it. Add the restriction types to the business catalog.
6. Optional: Create a business role template and assign the catalog to it.

Now, administrators can start granting access to business users. Development is finished and testing can start.

Granting Authorizations to Business Users

As an administrator in the ABAP environment, you can now grant authorizations to business users for this service:

1. Create business roles and assign them to business users.
   For each business role, set the access category *Write* to restricted. Select the values for which you want to authorize the users. For example, create a business role with the authorization to create, update, and delete only bonus calculations that are based on specific bonus variants.

As a result, business users with the business role assigned can access the service to display bonus calculations or create, update, or delete bonus calculations again.

4.2.11.2.4.4.1 Protecting the Service (Developer)

With creating authorization fields and an authorization object for your service and with the implementation of a protection against unauthorized read and creation, you can implement a protection of your service against unauthorized use on field level.

After this, you can implement authorizations on field level that can be granted to business users.

4.2.11.2.4.4.1.1 Creating a Check Table for Authorization Fields

Create a check table that you can provide to the administrator as value help during business role maintenance.

Context

The advantage of having a check table is that the administrator has a value help when maintaining business roles with restricted access. As a developer, you provide access to a table where the field values are stored. When administrators create business roles and assign them to business users, they can call up the check table in the format of a value help and select the relevant values from the table.

In the bonus calculation example, the check table contains the bonus variants.

Procedure

1. In ABAP Development Tools, in the project explorer, right-click on the relevant package node.
2. In the context menu, choose *New* > *Other ABAP Repository Object*.
3. In the following popup, choose *Core Data Services* > *Data Definition* and *Next*.
4. Enter a name, description, and, as referenced object, enter the table where the values are stored that you want to provide for the administrator. In the bonus calculation example, this table is the table where the bonus variants are stored for the app. Choose *Next*.
5. Select a transport request and choose *Next*.
6. From the list of templates, choose *Define View* and choose *Finish*.
7. In the coding, choose the fields that you need to expose.

   For the CDS view, it's sufficient that you use only those fields that are needed for a field help.
8. To provide some UI texts for the field help, add `@EndUserText.label` with a good label for each field that is exposed as part of the CDS view. The labels will be displayed as column headings in the field help of the business role maintenance.
9. Make sure that the annotation `@AbapCatalog.sqlViewName` is added at the beginning of the CDS view code.

For example, the code of such a CDS view can look as follows:

Sample Code

```
@AbapCatalog.sqlViewName: 'ZVARCHECK'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Check Table for bonus variants'
define view Z_VARIANTS_CHECK_TABLE as select from zbonusvar {
  @EndUserText.label: 'Bonus Variant'
  key variant as BonusVariant,

  @EndUserText.label: 'Category'
  category as Category,

  @EndUserText.label: 'Description'
  description as Description,

  @EndUserText.label: 'Created By'
  createdby as Createdby,

  @EndUserText.label: 'Created At'
  createdat as Createdat
}
```

10. Copy the SQL view name from `@AbapCatalog.sqlViewName` for later use (see Defining an Authorization Field [page 1014]).

4.2.11.2.4.4.1.2 Defining an Authorization Field

As in the scenario for granting access based on activities, you define an authorization object. However, for this scenario, you first define authorization fields to fine-tune access on field level.

Context

In the following, we assume that the bonus calculation service has a bonus variant field of type domain. You create an authorization field based on this data element.

Procedure

1. To create an authorization field ZBNS_VARN for the bonus variant, following the procedure for defining authorization fields in the user guide for ABAP development tools (ADT).
2. In the *Data Element* field, enter the data element for the bonus variant.
3. In the *Check Table* field, enter the SQL view name that you have created as a check table for all possible bonus variants (see Creating a Check Table for Authorization Fields [page 1013]).

> ⓘ Note
>
> You can use the *Search Help* button to simulate the field help as it's shown to the administrator during the business role maintenance.

4.2.11.2.4.4.1.3 Defining an Authorization Object

As in the scenario for granting access based on activities, you define an authorization object. However, for this scenario, you also add the authorization field `ZBNS_VARN` for the bonus variant.

Procedure

Follow the instructions for defining authorization objects in the user guide for ABAP development tools (ADT).

> → Tip
>
> In our example, in the newly created authorization object, under *Authorization Fields*, you need the following:
>
> - You need the predefined *Activity (ACTVT)* authorization field to define the permitted activities for the business users of your business service: *01 Add or Create*, *02 Change*, *03 Display*, and *06 Delete* as standard activities.
> - If you have non-standard activities such as *Calculate Bonus* that you need to protect (and to permit later), check the list of existing values. If you click on an empty row and choose ENTER, the list of available values is shown. For the bonus calculation example, let's choose *93 Calculate* for the business object-specific activity *Calculate Bonus*.
> - You need the newly created authorization field `ZBNS_VARN` for the bonus variant.

4.2.11.2.4.4.1.4 Implementing a Read Protection

Prevent unauthorized read access to the data exposed by your service using access control. For this purpose, you create an access control and then implement it for the CDS view on which your service is based.

Defining the Use of Access Control in the Data Definition

Context

For the data definitions that you want to protect, you need to define the use of access control.

> **Note**
>
> If you're using additional data definitions of type projection view, you need to define the use of an access control in them.

Procedure

1. Add the following line to the data definition of the interface:

   ```
   @AccessControl.authorizationCheck: #CHECK
   ```

   For more information, see Access Control Annotations in the documentation of the ABAP RESTful Application Programming Model.
2. If you're using additional data definitions of type projection view, you also need to define the use of an access control in them.

Creating an Access Control

Context

Even if you have modeled necessary authorizations for your service by creating a new authorization object, the authorization object itself doesn't protect your service from being accessed by a business user in the ABAP environment. Therefore, as a next step, you implement a read protection for your service. In our example, this means that business users should only be allowed to view bonus calculations whose bonus variant matches with the users' authorization. You implement a read protection using access control, which uses the authorization object that you created.

Procedure

1. Go to the core data service (CDS) view on which your service is based.

   You can find this CDS view under the data definitions of the core data services of your package.
2. To create an access control for the CDS view, follow the instructions of the ABAP CDS Development User Guide: Creating Access Controls and use the template *Define Role with PFCG Aspect*.
3. If you're using additional data definitions of type projection view, you also need to create additional access controls for them using the template *Define Role with Inherited Conditions*.

Implementing the Access Control

Context

You must edit the access control code to implement access control for the CDS view on which your service is based. Access control is based on the authorization object that you created before.

Procedure

1. Choose the access control that you've created, which currently looks like the following:

   Sample Code

   ```
   @EndUserText.label: '<your access control description>'
   @MappingRole: true
   define role <your access control name> {
       grant
           select
               on
                   ${cds_entity}
                       where
                           (${entity_element_1}, ${entity_element_2}) =
   aspect pfcg_auth(${authorization_object}, ${authorization_field_1}, $
   {authorization_field_2}, ${filter_field_1} = '${filter_value_1}');
   }
   ```

2. Adapt the code so that it grants access if the requesting business user has authorizations that contain authorization object `ZBNSCLC_AO` (*Bonus Calculation*) with allowed activity (`ACTVT`) *03 Display* and bonus variant (`ZBNS_VARNT`) values that match the bonus variant (`bonusvariant`) of the existing bonus calculation:

   Sample Code

   ```
   @EndUserText.label: 'Access Control for CDS Z_I_BONUS_CALCULATION'
   @MappingRole: true
   define role Z_I_BONUS_CALCULATION {
       grant
           select
               on
                   z_i_bonus_calculation
                       where
                           (bonusvariant) = aspect pfcg_auth(ZBNSCLC_AO,
   ZBNS_VARNT, ACTVT = '03');

   }
   ```

   In our example, `cds_entity` is the `z_i_bonus_calculation` CDS view.

3. If you're using additional data definitions of type projection view, enhance the coding as follows:

   Sample Code

   ```
   @EndUserText.label: '<your access control description>'
   @MappingRole: true
   ```

```
define role <your access control name> {
    grant
        select
            on
                <your projection view name>
                    where
                        inheriting conditions from entity
z_i_bonus_calculation;
}
```

Results

As a result, only business users with the corresponding authorizations get bonus calculations from the service, and they only get bonus calculations with allowed bonus variant values.

4.2.11.2.4.4.1.5 Modeling a Protection Against Unauthorized Write Activities

To protect your service from activities such as create, update or delete by unauthorized business users, you can use the authorization controls that are available for services based on managed business objects.

Context

In the example used here, the service is based on a managed business object *Bonus Calculation*. The use of a managed business object allows you to benefit from the global and instance-based authorizations for managed business objects, which are part of the ABAP RESTful Application Programming model. For more information about authorization control, see the documentation for the ABAP RESTful Application Programming Model.

Procedure

1. In ABAP development tools, call up the behavior definition of your service.
2. In the behavior definition, define authorization controls for your service.

   In the bonus calculation example, they can look as follows:

Sample Code

```
managed;
define own authorization context
{
  'ZBNSCLC_AO';
}
define behavior for z_i_bonus_calculation alias calculation
```

```
implementation in class zbp_bonus_calculation unique
persistent table zbonusclc
lock master
// Identity and Access Management:
// Enable RAP managed authorization check at create, update, and delete
// -> requires implementation of two methods, one with the addition "FOR
INSTANCE AUTHORIZATION"
//  and one with the addition "FOR GLOBAL AUTHORIZATION"
authorization master ( instance, global )
```

With the line `authorization master ( instance, global )`, an authorization check for the standard operations `create`, `update`, and `delete` and for non-standard operations, for example, `calculate bonus`, is defined. You must implement the authorization check using methods with addition `FOR GLOBAL AUTHORIZATION` (for `create`) and `FOR INSTANCE AUTHORIZATION` (for `update`, `delete`) in the behavior implementation.

The sample code also contains the definition of your own authorization context. The own authorization context documents all authorization objects that are used by the business object implementation. In the bonus calculation example used in this documentation, this is the authorization object `ZBNSCLC_AO` because it's checked in the RAP authorization control and the CDS access control. The own authorization context can also be used to prefill authorization defaults.

4.2.11.2.4.4.1.6 Implementing a Protection Against Unauthorized Write Activities

To protect a service, such as the example of bonus calculation from unauthorized creations, updates, or deletions, enhance the behavior implementation of the service in ABAP development tools (ADT). The implementation is required by what has been defined as authorization control in the behavior definition.

Context

Among the methods of the behavior implementation, you add methods that check the authorizations for creation and for update and delete.

For more information about authorization control, see see the documentation for the ABAP RESTful Application Programming Model.

Procedure

1. In the *Source Code Library* folder of your package, under *Classes*, choose the behavior implementation of your service.
2. In the `PRIVATE SECTION` in the handler class of the behavior implementation, define the method for authorization checks for update and deletion:

Sample Code

```
  METHODS get_instance_authorizations FOR INSTANCE AUTHORIZATION
    IMPORTING it_entity_keys REQUEST requested_authorizations FOR
<behavior definition or its alias> RESULT result.
```

In the bonus calculation example, this method definition can look as follows:

Sample Code

```
  METHODS get_instance_authorizations FOR INSTANCE AUTHORIZATION
    IMPORTING keys REQUEST requested_authorizations FOR calculation RESULT
result.
```

3. In the `PRIVATE SECTION` in the handler class of the behavior implementation, also define the method for authorization checks for creation:

Sample Code

```
  METHODS get_global_authorizations FOR GLOBAL AUTHORIZATION
    IMPORTING REQUEST requested_authorizations FOR <behavior definition or
its alias> RESULT result.
```

In the bonus calculation example, this method definition can look as follows:

Sample Code

```
  METHODS get_global_authorizations FOR GLOBAL AUTHORIZATION
    IMPORTING REQUEST requested_authorizations FOR calculation RESULT
result.
```

4. Add a method implementation that checks the authorizations for updates and deletions.

In the bonus calculation example, this method can look like this:

Sample Code

```
  METHOD get_instance_authorizations.
*     As import table contains only keys, we need to get complete bonus
calculation data for bonus variant value
    READ ENTITIES OF z_i_bonus_calculation IN LOCAL MODE
             ENTITY calculation
              ALL FIELDS
                 WITH CORRESPONDING #( it_bonus_calc_keys )
             RESULT DATA(lt_bonus_calcs).
*     fill result list with authorizations per bonus calculation
    LOOP AT lt_bonus_calcs INTO DATA(ls_bonus_calc).
*       check update authorization for bonus calculation, incl. bonus
variant dependency
      AUTHORITY-CHECK OBJECT 'ZBNSCLC_AO' ID 'ACTVT' FIELD '02' ID
'ZBNS_VARNT' FIELD ls_bonus_calc-bonusvariant.
*       set variable for update authorization
      DATA(lv_update_allowed) = COND #( WHEN sy-subrc = 0 THEN
if_abap_behv=>auth-allowed ELSE if_abap_behv=>auth-unauthorized ).
*       check calculate bonus authorization for bonus calculation, incl.
bonus variant dependency
      AUTHORITY-CHECK OBJECT 'ZBNSCLC_AO' ID 'ACTVT' FIELD '93' ID
'ZBNS_VARNT' FIELD ls_bonus_calc-bonusvariant.
*       set variable for calculate bonus authorization
```

```
      DATA(lv_calculate_bonus_allowed) = COND #( WHEN sy-subrc = 0 THEN
if_abap_behv=>auth-allowed ELSE if_abap_behv=>auth-unauthorized ).
*       check delete authorization for bonus calculation, incl. bonus
variant dependency
      AUTHORITY-CHECK OBJECT 'ZBNSCLC_AO' ID 'ACTVT' FIELD '06' ID
'ZBNS_VARNT' FIELD ls_bonus_calc-bonusvariant.
*       set variable for delete authorization
      DATA(lv_delete_allowed) = COND #( WHEN sy-subrc = 0 THEN
if_abap_behv=>auth-allowed ELSE if_abap_behv=>auth-unauthorized ).
*       fill result list
      APPEND VALUE #( id = ls_bonus_calc-id %update = lv_update_allowed
%delete = lv_delete_allowed ) TO result.
    ENDLOOP.
  ENDMETHOD.
```

5. Add a method that checks the authorizations for creation.

   In the bonus calculation example, such a method could look like this:

{·} Sample Code

```
  METHOD get_global_authorizations.

    IF requested_authorizations-%create EQ if_abap_behv=>mk-on.
*     check create authorization
      AUTHORITY-CHECK OBJECT 'ZBNSCLC_AO' ID 'ACTVT' FIELD '01'.
      result-%create = COND #( WHEN sy-subrc = 0 THEN
      if_abap_behv=>auth-allowed ELSE
      if_abap_behv=>auth-unauthorized ).
    ENDIF.
  ENDMETHOD.
```

Results

The implemented methods provide the desired protection from unauthorized uses:

- With the implementation of the method `get_instance_authorizations`, an unauthorized business user gets an error message when trying to update or delete a business object instance (in the example, a bonus calculation) via service call.
- With the implementation of the method `get_global_authorizations`, an unauthorized business user gets an error message when trying to save the creation of a new business object instance.
- Unauthorized business users can't use the *Delete* button in the list view on the service UI. In addition, the *Edit* and *Delete* buttons are hidden in the detail view on the service UI for a business object instance.

Now, the service is secured against unauthorized operations. As a next step, you must make sure that business users who need to use service get the corresponding authorizations.

4.2.11.2.4.4.2 Developing Authorizations (Developer)

After you've protected your service against unauthorized use, you create the objects that are needed for granting authorizations to business users to use your service.

For granting authorizations, follow the steps for restriction fields and types and for creating an IAM app, a business catalog, and a business role template. You should define your own authorization context in the behavior definition first, which can be used to prefill authorization defaults.

4.2.11.2.4.4.2.1 Editing Authorization Default Values

Authorization default values are automatically created when you create a service binding. You can add authorization objects and change the authorization default values.

Context

The authorization default values are the authorizations that are assigned to business users by default if they have a business role based on the service. Note, however, that the authorization default values can be overwritten by values set in the IAM app or by restrictions in the business role. Therefore, if you want to avoid double maintenance, try and define the authorization default values in such a way that they can be used by all derived IAM apps.

For more information about authorization default values, see the user guide for ABAP development tools (ADT).

Procedure

1. Open the service binding and choose the *Maintain Authorization Default Values* link.
2. Choose *Synchronize* to add the latest authorization objects from the own context.

   > ⓘ Note
   >
   > The *Synchronize* function only **adds** authorization objects that are not yet part of the list of authorization objects; it does not remove objects.

3. To check or change the authorization default values that are assigned automatically, select the authorization objects in the list.

   An authorization object that is used in the read-protecting access control automatically gets the settings *Default with Field Values* and the required activity value for *Display*. Other authorization objects get *Default Without Field Values*.
4. For the bonus calculation, you can now specify what you want to authorize, which are, in the example used here, the standard activities *Create*, *Change*, and *Delete* as well as the business object-specific activity *Calculate*.

Results

Now, the standard authorizations for the service are defined and will be applied to an IAM app based on the service. You still can change them later in the IAM app itself or delegate the value definition to the administrator during business role maintenance (only if restrictions are used).

Related Information

Defining an IAM App for the Business Service [page 1008]
Creating Restriction Fields Based on Authorization Fields [page 1024]

4.2.11.2.4.4.2.2 Defining an IAM App for the Business Service

To assign a business user to a business role for your service, you need to create an IAM app, which can then be included into a business catalog, which, in turn, can be assigned to a business role.

Context

Since you want to make your service available to users with a certain business role, you need to define an IAM app for the service that you created in ADT. The IAM app is only relevant for identity and access management (IAM), and you create the IAM app in ADT. You can use an IAM app to define authorizations for one or more business services.

Instead of creating a new IAM app, you can also use an existing IAM app for the service.

For more information, see the information about IAM apps in the user guide for ABAP development tools (ADT).

Procedure

To create an IAM app, follow the instructions in the user guide for ABAP development tools.

For the bonus calculation IAM app used as an example here, you have already specified before in the authorization default values what you want to authorize. If needed, you can overwrite these defaults again here.

> **Note**
>
> In this example scenario, we assume that you create only one IAM app for access using all permitted activities and their access categories (write, read, value help). Be aware at this point that in the business role Fiori app, the administrator can later separate authorizations depending on the access categories

(write, read, or value help). Therefore, there's no need to develop different types of access if you just want to separate write, read, and value help.

However, it's different if you want to have different business roles for activities within one access category, for example, one for the create and update activities and another for the delete activity, which are all write activities. In this case, you must start here to separate authorizations: Create separate IAM apps, one for each write activities group that should be separated, go on with separate business catalogs, and finally enable the administrator to create separate business roles based on the business catalogs.

4.2.11.2.4.4.2.3 Creating Restriction Fields Based on Authorization Fields

For each authorization field that you want to expose and consider in a business role, you must create a corresponding restriction field and assign it to a restriction type.

Context

In the example used here, create a restriction field for the bonus variant field.

Procedure

For the procedure, see the ABAP development user guide: Defining Restriction Fields.

4.2.11.2.4.4.2.4 Creating Restriction Types

Create a restriction type containing a restriction field and authorization object.

Context

Restriction types bundle restriction fields. In the example used here, there's one restriction type for the bonus calculation.

Procedure

For the procedure, see the ABAP development user guide: Defining Restriction Types.

4.2.11.2.4.4.2.5 Creating a Business Catalog with Restriction Types

Create a business catalog and add the restriction types to it.

Context

In the business catalog, you can bundle multiple IAM apps and their predefined authorizations. You assign a business catalog to a business role. If you have more than one business catalog for the same business role, you might also consider creating a business role template.

In the business catalog, you can also define for which fields and access categories an administrator can maintain restriction values during business role maintenance.

Instead of creating a new business catalog, you can also reuse an existing business catalog for the app.

Procedure

1. Create a business catalog and assign the IAM app to it.

   For more information about creating business catalogs, see Creating a Business Catalog in the ABAP development user guide.
2. On the *Restriction Type* tab, enter the restriction type that you created.

   In our example, add the restriction type *Bonus Calculation*.
3. For each field, choose the appropriate checkboxes for write access, read access, and value help (F4).

   > ⓘ Note
   >
   > Even if you defined fewer values for the authorization field on which the restriction field is based in the IAM app (for example, one bonus variant only),the checkboxes here enable the administrator to allow all bonus variants again during business role maintenance.
4. Save the business catalog and publish it locally.

4.2.11.2.4.4.2.6 Creating a Business Role Template (Optional)

In addition to a business catalog, you can also create a business role template with ABAP Development Tools (ADT). Creating a business role template is optional, but it can ease the business role creation for the administrator.

Context

Creating a business role template especially makes sense if you have more than one business catalog to assign to a business role.

Procedure

1. Create a business role template as described in Creating a Business Role Template.
2. After the business role template has been created, add the business catalog to it.
3. Choose *Publish Locally* to make the business role template available for use in the development system.

4.2.11.2.4.4.3 Granting Authorizations to Business Users (Administrator)

As an administrator, create business roles to grant authorizations to business users.

After a business catalog and optionally a business role template have been created by the developer, administrators take over. They grant authorizations to business users by creating and assigning business roles. The business role needs to contain the business catalog created by the developer to get all the predefined authorizations needed for the business service. If a business role template is available, the administrator can also use the business role template to create a new business role.

4.2.11.2.4.4.3.1 Creating a Business Role with Restrictions and Assigning It to Users

You can create a new business role where you can benefit from the predefined values and restrictions that were defined before.

Context

The default is always no write access and unrestricted read access. You can have dedicated business roles that use these default settings. In the bonus calculation example, you can create multiple roles for changing only bonus calculation instances of a bonus variant.

For more information about maintaining restrictions, see How to Maintain Restrictions.

Perform this task in every system. Alternatively, you can also create a business role once, download it, and upload it to other systems.

Procedure

1. Create a business role from scratch or from a business role template, add the business catalog with the restrictions that the developer has created for the service.
2. Choose *Maintain Restrictions*.
3. On the *Write, Read, Value Help* tab page, choose *Restricted*.

   The available restriction types and their fields are now shown, which is, in our example, the bonus calculation and its bonus variant.
4. Choose the edit button next to the restriction type.

   In the following popup, the list of available values from the restriction field is shown.
5. Select the values for which you want to authorize the users.

   In our example, the value would be one or multiple bonus variants that have been saved to the check table that the developer has assigned to the authorization field for the bonus variant (see Defining an Authorization Field [page 1014]). As a result, business users with the business role assigned can create, update, and delete bonus calculations with the selected bonus variants only.
6. On the *Read, Value Help* tab page, choose *Unrestricted*.

   This means that business users with this business role can still view all bonus calculations independently of their bonus variant, even if they can only change bonus calculations with a particular bonus variant.
7. On the *Assigned Business Users* tab, assign the business users to your new business role

   These business users get the authorizations as defined in the business role.
8. Save the business role to activate it.

4.2.11.2.5 Providing Access to the SAP Fiori Application

Learn what developers and administrators need to do so that an SAP Fiori application appears in the SAP Fiori launchpad of an authorized business user.

Overview

For business services created in ABAP Development Tools, developers can also create SAP Fiori application UIs. Let's assume that they created UIs using, for example, SAP Business Application Studio or Visual Studio Code and deployed them back to the ABAP environment. To ensure that business users have access to the SAP Fiori application in their SAP Fiori launchpad, developers and administrators need to perform a few steps.

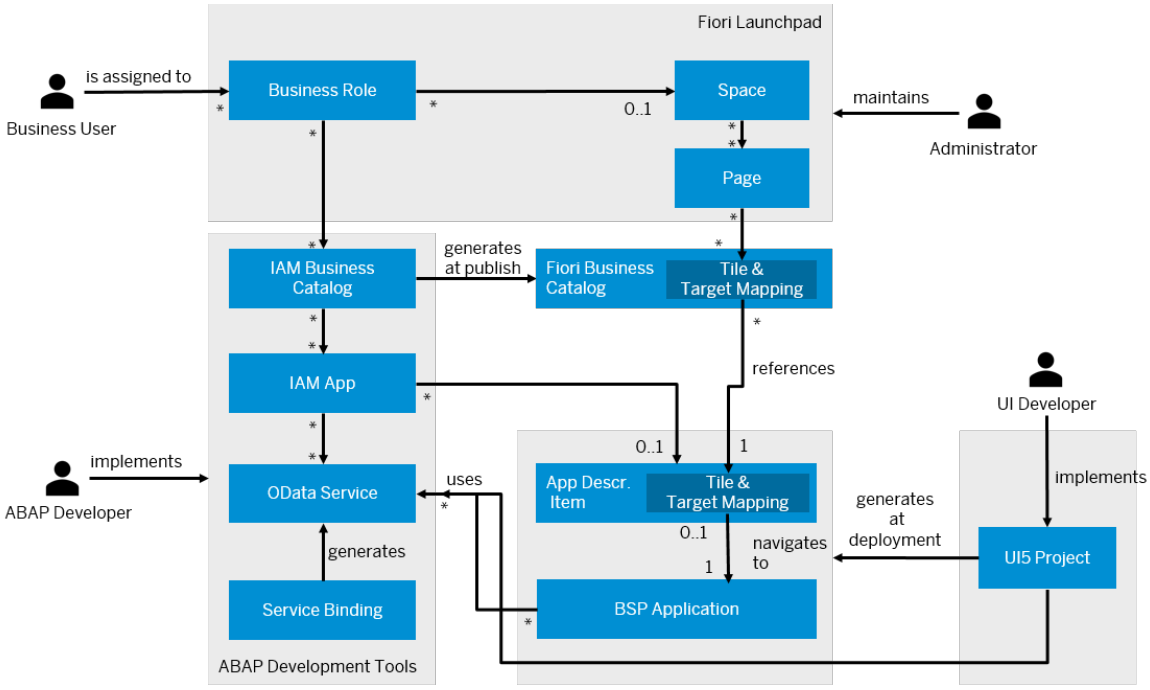
When you deploy an SAP Fiori application to the ABAP environment, the following objects, among others, are created automatically:

- A business server page (BSP) application, which stores the code of the SAP Fiori application
- An SAP Fiori launchpad app descriptor item, which contains the tile and navigation target for the SAP Fiori launchpad integration
  The SAP Fiori launchpad app descriptor item is created from the `manifest.json` file of the UI5 app, where the tile and navigation parameters are configured.

You can find the BSP application and the SAP Fiori launchpad app descriptor item as objects in your deployment package in ABAP Development Tools.

After an SAP Fiori application has been deployed, it can't be immediately consumed in the SAP Fiori launchpad. You have already created an IAM app and added the business service for which you've created your UI to the IAM app. You must now also assign the automatically created SAP Fiori launchpad app descriptor item to this IAM app.

As an administrator, you must create and enable an SAP Fiori launchpad space and page for business users.



This guide outlines the steps that are needed after you've followed the procedures outlined in one of the scenarios *Providing (Unrestricted) Access*, *Providing Access Based on Activities*, or *Providing Access Based on Field Values*. For a general overview of all steps for developing an SAP Fiori application UI in the ABAP environment, see Develop an SAP Fiori Application UI and Deploy it to ABAP Using SAP Business Application Studio [page 1859].

Tutorial

In addition to this documentation, the following tutorial is available:

Integrate List Report into ABAP Fiori Launchpad

4.2.11.2.5.1 Assign the SAP Fiori Application to the IAM App (Developer)

To ensure consistent authorizations for the business service and its UI, you need to assign the SAP Fiori launchpad app descriptor item and the business service to the same IAM app.

Context

After an SAP Fiori application has been deployed, it can't be immediately consumed in the SAP Fiori launchpad. You have already created an IAM app and added your business service to it. You must now also assign the SAP Fiori launchpad app descriptor item, which was automatically created during deployment, to this IAM app.

You can find the SAP Fiori launchpad app descriptor item as an object in ABAP Development Tools in the package to which you deployed the SAP Fiori application.

Procedure

1. In ABAP Development Tools, open your IAM app.
2. In the *UI5 Application ID* field, enter the technical name of the SAP Fiori launchpad app descriptor item or use the field value help.
3. Save and publish locally.

Results

If you haven't already done so, assign the IAM app to an IAM business catalog, and publish the IAM business catalog in ABAP Development Tools. Publishing generates a corresponding SAP Fiori business catalog that is listed in the SAP Fiori launchpad app *Business Catalogs*.

4.2.11.2.5.2 Integrating the SAP Fiori Application into the SAP Fiori Launchpad (Administrator)

To ensure that an SAP Fiori application appears in the SAP Fiori launchpad of the business users, you, as an administrator, need to perform a few configuration steps.

Prerequisites

You, as an administrator, have already created one or multiple business roles and assigned them to business users.

Context

You must integrate the SAP Fiori app into the IAM structure of the SAP Fiori launchpad of SAP Business Technology Platform.

Procedure

1. Create a space for the business role.
2. Add a page to this space and add the app tile to the page.

   For more information about how to proceed, see Create a New Space and Page for a Business Role in the SAP Fiori launchpad documentation on SAP Help Portal.

Results

You've now enhanced the business roles for the business service with authorizations for the UI. All users that have been assigned to the business roles before can now view the page and SAP Fiori application in their SAP Fiori launchpad.

> **Note**
>
> The SAP Fiori application only appears in the spaces mode for the user. The spaces mode was developed to offer more flexibility to influence the launchpad layout for specific user groups. The business role defines which users see a specific space. For more information, see Managing Launchpad Spaces and Pages on SAP Help Portal.

4.2.11.2.6 Providing Access to a Business Service for Communication Users

For a newly created business service, you must define how communication users can access it and which activities are allowed.

Instead of accessing a business service using a business user, it is also possible to access a business service using a communication user. Communication users are relevant in scenarios where you work with an API business service (inbound or outbound) from or into the ABAP environment.

For communication users, you must provide access to a newly created service because access isn't automatically available. However, on instance level, all activities are allowed as long as you don't protect your business service. Therefore, you must develop identity and access management artifacts.

The main difference to providing access to a business user is that, as a developer, you do not create an IAM app and a business catalog but a communication scenario. Authorizations are maintained in the communication scenario. Administrators cannot restrict the authorizations defined in the communication scenario later on when they create communication arrangements based on the communication scenario. Therefore, you as a developer decide which authorizations are granted for the communication user by the administrator.

Overview of Development Options

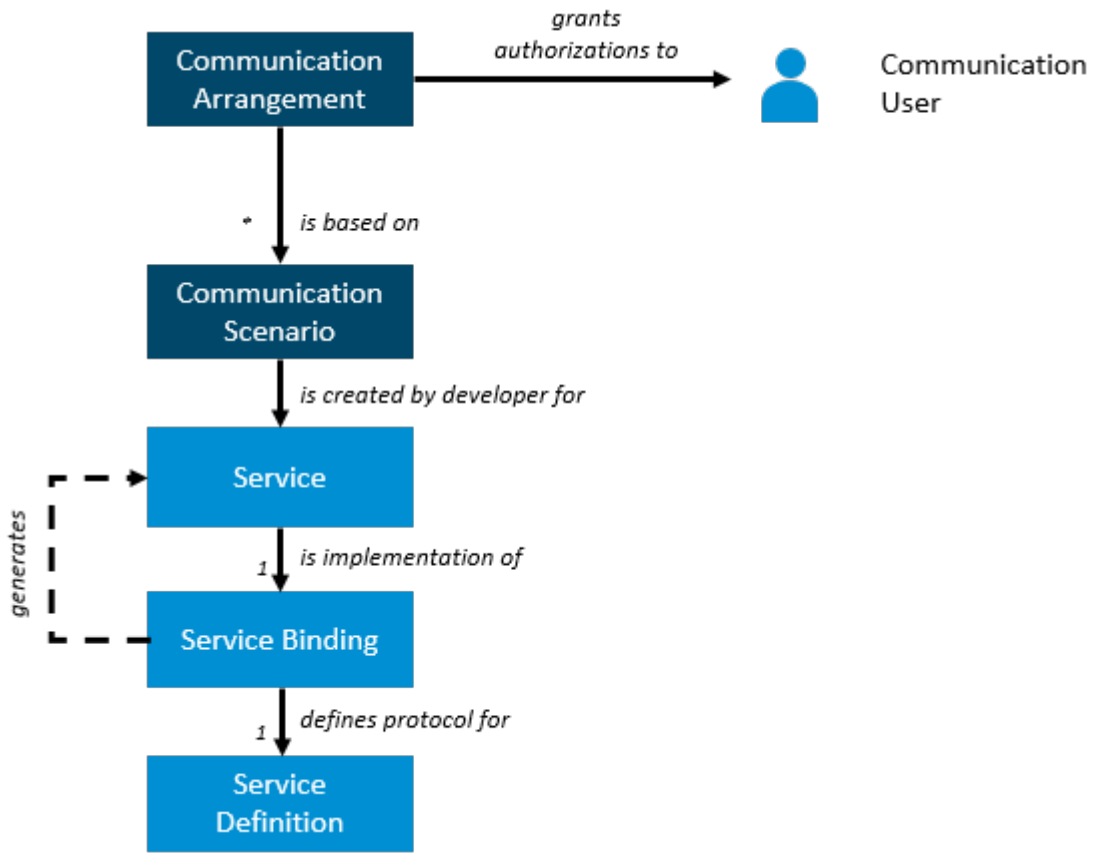
You can provide access the following ways:

- Unrestricted access
- Access based on activties from an authorization object

4.2.11.2.6.1 Granting (Unrestricted) Access for Communication Users

In this scenario, you get information about how to grant unrestricted access for communication users with a few steps.

In this scenario, to provide access to the business service, you must create a communication scenario. Administrators then create a communication arrangement based on this communication scenario. Note that all objects that need to be created are highlighted in dark blue in the graphic.



This scenario requires only a few steps to grant authorizations, and it's the minimum that you must do to make your service available to communication users. However, with this scenario, you always grant unrestricted write **and** read access. If you want to fine-tune the authorizations in more detail, check the scenario *Granting Access Based on Activities*.

4.2.11.2.6.1.1 Creating a Communication Scenario for Unrestriced Access (Developer)

In ABAP Development Tools, you can define communication scenarios, which can be used to access business services using a technical communication user.

Context

In ABAP Development Tools, you define a communication scenario and add the required services to the scenario. An administrator can then use this communication scenario and create a communication arrangement based on it. This communication arrangement is the basis for accessing the business service using a communication user. If you don't include an authorization object to the communication scenario, the communication user will have unrestricted access to the business service.

For more information, see the information about consuming services in the context of APIs in the user guide for ABAP development tools (ADT).

Procedure

1. Choose the relevant package in the project explorer.
2. Right-click to open the context menu and choose *New* > *Other* > *ABAP Repository Object* to launch the creation wizard.
3. Choose *Communication Management* > *Communication Scenario* and choose *Next*.
4. Enter a name and a description and choose *Next*.
5. Choose *Finish* to create a transport request.
6. On the *Inbound* or the *Outbound* tab, add your inbound or an outbound business service to the communication scenario.
7. If you want to test your communication scenario directly in the development system, publish it locally to make sure that the required services have been generated.

4.2.11.2.6.1.2 Creating a Communication Arrangement (Administrator)

To grant authorizations to a communication user, as admnistrator, you create a communication arrangement. In this example, the communication arrangement is based on the communication scenario created by the developer.

Prerequisites

You have created a communication user that is used for data exchange with other systems. To create a communication user, you use the *Maintain Communication Users* app on the SAP Fiori launchpad (see also How to Create Communication Users).

You have created a communication system, which represents the communication partner within a communication. To create a communication system, you use the *Maintain Communication Systems* app (see also How to Create Communication Systems).

Procedure

1. Open the SAP Fiori Launchpad.
2. Choose *Communication Arrangements*.
3. Create a new communication arrangement based on the communication scenario, using the communication user and system that you have already created.

For more information, see How to Create a Communication Arrangement.

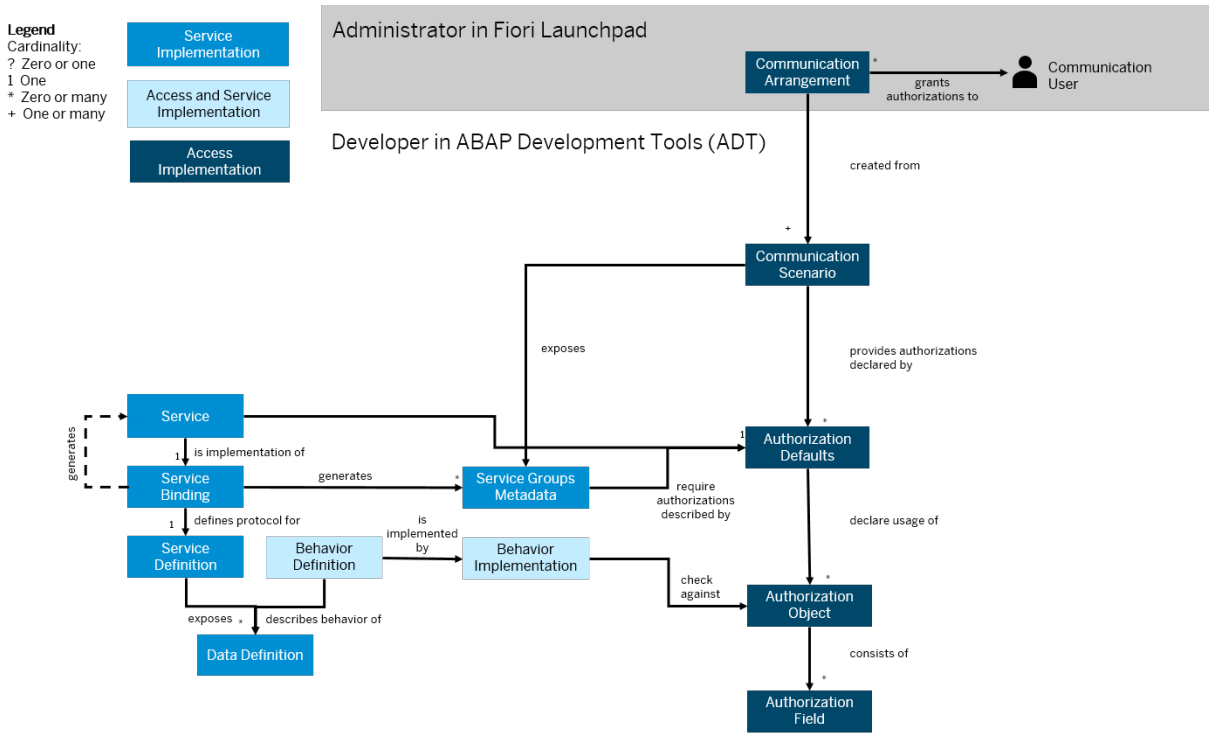
Results

The communication user in the communication arrangement gets the authorizations that have been defined as part of the communication scenario by the developer. In the example of unrestricted access, no authorization object or fields have been defined, so the communication user gets full access to the business service.

4.2.11.2.6.2 Granting Access Based on Activities for Communication Users

In this scenario, you grant access depending on what the communication user should be allowed to do, for example, read or write access.

This scenario is more complex than a simple unrestricted access grant because you also need to create an authorization object and implement authority checks in the behavior implementation.



In this documentation, you get a more detailed overview of how to implement a separate read and write access. You can achieve such a separation by creating an authorization object with the standard authorization field for activities, an authority check to protect the service, and a communication scenario with authorization fields and activities defined. The administrator can then create two communication arrangements for read and write access each, for example.

4.2.11.2.6.2.1 Protecting the Service (Developer)

With creating an authorization object for your service and with the implementation of a protection against unauthorized creation and change activities, you implement a basic protection of your service.

After this, you can implement how authorizations are granted to communication users.

Related Information

Developing Authorizations for Communication Users (Developer) [page 1039]

4.2.11.2.6.2.1.1 Defining Permitted Activities in an Authorization Object

To define the permitted activities for a service, create an authorization object with the standard authorization field *Activity* (ACTVT) for the permitted activities.

Procedure

To define the permitted activities, follow the procedure for defining authorization objects in the user guide for ABAP development tools (ADT).

> ⓘ Note
>
> In our example, in the newly created authorization object, under *Authorization Fields*, you only need the predefined *Activity* authorization field to define the permitted activities for the business users of your service. In our example, these activities would be *01 Add or Create*, *02 Change*, *03 Display*, and *06 Delete* for standard activities.In addition, you need *93 Calculate* for the business object-specific activity *Calculate Bonus*. If you click on an empty row and choose ENTER, the list of available values is shown.
>
> In theory, by using the syntax `authorization: update` on an operation in the entity's behavior definition, you could use the authorization check that is implemented for the update operation for the annotated operation. This is also referred to as “authorization delegation”. However, for security reasons, we recommend that you carefully consider whether you want to use authorization delegation because in this case, a user who is allowed to update a business object is then also allowed to perform all other activities on the business object.

Results

In the following, the newly created authorization object for the bonus calculation example is called `ZBNSCLC_AO`.

4.2.11.2.6.2.1.2 Modeling a Protection Against Unauthorized Write Activities

To protect your service from activities such as create, update or delete by unauthorized users, you can use the authorization controls that are available for services based on managed business objects.

Context

In the example used here, the service is based on a managed business object *Bonus Calculation*. The use of a managed business object allows you to benefit from the global authorization for managed business objects, which is part of the ABAP RESTful Application Programming model. For more information about authorization control, see the documentation for the ABAP RESTful Application Programming Model.

Procedure

1. In ABAP development tools, call up the behavior definition of your service.
2. In the behavior definition, define authorization controls for your service.

   In the bonus calculation example, they can be as follows:

   Sample Code

   ```
   managed;
   define own authorization context
   {
     'ZBNSCLC_AO';
   }
   define behavior for z_i_bonus_calculation alias calculation
   implementation in class zbp_bonus_calculation unique
   persistent table zbonusclc
   lock master
   // Identity and Access Management:
   // Enable RAP managed authorization check at create, update, and delete
   // -> requires implementation of two methods, one with the addition "FOR
   INSTANCE AUTHORIZATION"
   //  and one with the addition "FOR GLOBAL AUTHORIZATION"
   authorization master ( instance, global )
   ```

   With the line `authorization master ( global )`, an authorization check for the standard operations `create`, `update`, and `delete` and for non-standard operations, for example, `calculate bonus`, is

defined. You must implement the authorization check using a method with addition `FOR GLOBAL AUTHORIZATION` in the behavior implementation.

The sample code also contains the definition of your own authorization context. The own authorization context documents all authorization objects that are used by the business object implementation. In the bonus calculation example used in this documentation, this is the authorization object `ZBNSCLC_AO` because it's checked in the authorization control. The own authorization context is also used to prefill authorization defaults.

4.2.11.2.6.2.1.3 Implementing a Protection Against Unauthorized Write Activities

To protect a service such as the example of bonus calculation against unauthorized bonus creations, updates, or deletions, enhance the behavior implementation of the service in ABAP development tools (ADT). The implementation is required by what has been defined in the behavior definition.

Context

Among the methods of the behavior implementation, you add methods that check the authorizations for creation and for update and delete.

Procedure

1. In the *Source Code Library* folder of your package, under *Classes*, choose the behavior implementation of your service.
2. In the `PRIVATE SECTION` in the handler class of the behavior implementation, define the methods for authorization checks for creation, update, and deletion.

   Sample Code

   ```
   METHODS get_global_authorizations FOR GLOBAL AUTHORIZATION
     IMPORTING REQUEST requested_authorizations FOR <behavior definition or
   its alias> RESULT result.
   ```

   In the bonus calculation example, these method definitions could be the following:

   Sample Code

   ```
   METHODS get_global_authorizations FOR GLOBAL AUTHORIZATION
     IMPORTING REQUEST requested_authorizations FOR calculation RESULT
   result.
   ```

3. Add a method implementation that checks the authorizations for creations, updates, and deletions.

> **Note**
>
> For more information about authorization control, see the documentation for the ABAP RESTful Application Programming Model.

In the bonus calculation example, this method could be the following:

> **Sample Code**

```
  METHOD get_global_authorizations.

    IF requested_authorizations-%create EQ if_abap_behv=>mk-on.
*     check create authorization
      AUTHORITY-CHECK OBJECT 'ZBNSCLC_AO' ID 'ACTVT' FIELD '01'.
      result-%create = COND #( WHEN sy-subrc = 0 THEN
      if_abap_behv=>auth-allowed ELSE
      if_abap_behv=>auth-unauthorized ).
    ENDIF.
    IF requested_authorizations-%update EQ if_abap_behv=>mk-on.
*     check update authorization
      AUTHORITY-CHECK OBJECT 'ZBNSCLC_AO' ID 'ACTVT' FIELD '02'.
      result-%update = COND #( WHEN sy-subrc = 0 THEN
      if_abap_behv=>auth-allowed ELSE
      if_abap_behv=>auth-unauthorized ).
    ENDIF.
    IF requested_authorizations-%delete EQ if_abap_behv=>mk-on.
*     check delete authorization
      AUTHORITY-CHECK OBJECT 'ZBNSCLC_AO' ID 'ACTVT' FIELD '06'.
      result-%delete = COND #( WHEN sy-subrc = 0 THEN
      if_abap_behv=>auth-allowed ELSE
      if_abap_behv=>auth-unauthorized ).
    ENDIF.
  ENDMETHOD.
```

Results

The implemented methods provide the desired protection from unauthorized uses:

- With the implementation of the method defined with the addition `FOR GLOBAL AUTHORIZATION`, an unauthorized business user gets an error message when trying to create a business object instance (in the example, a bonus calculation) or when trying to update or delete it via service call.
- Unauthorized business users can't use the *Delete* button in the list view on the service UI. In addition, the *Edit* and *Delete* buttons are hidden in the detail view on the service UI for a business object instance.

Now, the service is secured against unauthorized operations. As a next step, you must make sure that business users who need to use the service get the corresponding authorizations.

4.2.11.2.6.2.2 Developing Authorizations for Communication Users (Developer)

After you've protected your service against unauthorized use, you can now create the objects that are needed to grant authorizations for communication users: authorization default values and a communication scenario.

Follow the steps for editing authorization default values and for creating a communication scenario in ABAP Development Tools. You should define `own authorization context` in the behavior definition first, which can be used to prefill authorization defaults.

> ⓘ Note
>
> For developing authorizations for business users, the procedure is different. For more information, see Developing Authorizations for Business Users (Developer) [page 1006].

4.2.11.2.6.2.2.1 Editing Authorization Default Values

Authorization default values are automatically created when you create a service binding. You can add authorization objects and change the authorization default values.

Context

The authorization default values are shown when you include an authorization object into a communication scenario. These values are assigned to communication user if you leave them unchanged. Note, however, you can overwrite the authorization default values by values set in the communication scenario.

For more information about authorization default values, see the user guide for ABAP development tools.

Procedure

1. Open the service binding and choose the *Maintain Authorization Default Values* link.
2. Choose *Synchronize* to add the latest authorization objects from the own context.

   > ⓘ Note
   >
   > The *Synchronize* function only **adds** authorization objects that are not yet part of the list of authorization objects; it does not remove objects.

3. To check or change the authorization default values that are assigned automatically, select the authorization objects in the list.

   You can now specify what activities you want to authorize.
4. Choose the authorization object from the list and choose *Default With Field Values* from the dropdown list.

In our example, we authorize for the standard activities *Create*, *Display*, *Change*, and *Delete* as well as the business object-specific activity *Calculate*.

4.2.11.2.6.2.2.2 Defining a Communication Scenario Including Authorization Values

In ABAP Development Tools, you can define communication scenarios, which can be used to access business services using a technical communication user.

Context

In ABAP Development Tools, you define a communication scenario and add the required services to the scenario. As part of the communication scenario definition, you also define the relevant authorization fields and activities. An administrator can then use this communication scenario and create a communication arrangement based on it.

> **Note**
>
> Administrators can't change the authorization activities and values that you define as part of the communication scenario when they create their communication arrangements.

For more information, see the information about consuming services in the context of APIs in the user guide for ABAP development tools (ADT).

Procedure

1. Choose the relevant package in the project explorer.
2. Right-click to open the context menu and choose *New* > *Other* > *ABAP Repository Object* to launch the creation wizard.
3. Choose *Communication Management* > *Communication Scenario* and choose *Next*.
4. Enter a name and a description and choose *Next*.
5. Choose *Finish* to create a transport request.
6. On the *Inbound* or the *Outbound* tab, add your inbound or an outbound business service to the communication scenario.
7. On the *Authorizations* tab, add the authorization object that you created for the business service.
8. On the *Authorizations* tab, also add the relevant authorization fields and activities. For each authorization field, you can now enter the values that you allow for the communication user. Similarly, you can select which activities are allowed for the communication user.

   For example, you could define that only bonus variant V0 can be created using the communication user, but you would allow all possible activities.

The communication user will be defined by the administrator as part of the communication arrangement that is based on this communication scenario.

9. If you want to test your communication scenario directly in the development system, publish it locally to make sure that the required services have been generated.

4.2.11.2.6.2.3 Creating a Communication Arrangement (Administrator)

To grant authorizations to a communication user, as admnistrator, you create a communication arrangement. In this example, the communication arrangement is based on the communication scenario created by the developer, including all authorization restrictions.

Prerequisites

You have created a communication user that is used for data exchange with other systems. To create a communication user, you use the *Maintain Communication Users* app on the SAP Fiori launchpad (see also How to Create Communication Users).

You have created a communication system, which represents the communication partner within a communication. To create a communication system, you use the *Maintain Communication Systems* app (see also How to Create Communication Systems).

Procedure

1. Open the SAP Fiori Launchpad.
2. Choose *Communication Arrangements*.
3. Create a new communication arrangement based on the communication scenario, using the communication user and system that you have already created.

   For more information, see How to Create a Communication Arrangement.

Results

The communication user in the communication arrangement gets the authorizations that have been defined as part of the communication scenario by the developer. If any authorizations based on authorization fields or activities have been defined or values set in the communication scenario, these also apply.

4.2.11.2.7 Providing Read or Write Access to a Business Service Using Privileged Mode

With enablement of privileged mode for a business object, it's possible to consume the business object without the need for additional business authorizations.

Use Case for Business Users

A typical use case for privileged mode is the execution of an application job. In the bonus calculation example used in this documentation, such an application job could be required to schedule calculations of the target bonus amounts of valid bonus calculations that haven't been calculated yet. For this purpose, you can enable privileged access for the *Bonus Calculation* business object.

As a result, the business user for which the bonus calculation job is scheduled doesn't require all the authorizations that a typical business user of the business service would need. So, there's no need to define an IAM app, a business catalog, and a business role for the authorization to calculate the bonus.

Use Case for Communication Users

During the execution of instance A of a business object X, write access might be needed to instance B of business object B. This is where privileged mode comes into play. Let's turn to a concrete example: You've created a custom business object for employees and a bonus business object. The employee business object has a performance indicator, such as *Exceeds Expectations*, *Consistently Exceeds Expectations*, and so on.

Now, when the bonus of the employee is calculated using the action *Calculate Bonus*, then the performance indicator of the employee is also updated at the same time, for example, using the action *Update Performance Indicator*. At this point, it makes sense to check whether the user (human or machine) has the authority to calculate the bonus. If so, then the system could also automatically update the performance indicator even if the user doesn't have a dedicated authority for it. What's more, the user might not even be allowed to display the information.

4.2.11.2.7.1 Modeling the Authorization Context for Privileged Mode

To enable privileged mode, you need another authorization context named `NoCheckWhenPrivileged`.

Context

Let's assume that you want to enable privileged mode for the bonus calculation object that's used as an example in this documentation. First, you define an authorization context called `NoCheckWhenPrivileged`

in the behavior definition. This authorization context is needed in addition to the `own authorization` context of the business object that you've already defined in the behavior definition. Then, you enable privileged mode for the business object by disabling `NoCheckWhenPrivileged` using the clause `with privileged mode disabling NoCheckWhenPrivileged`. As a result, authorization objects in the context `NoCheckWhenPrivileged` aren't checked during privileged access.

Procedure

In the behavior definition, add the following code, for example:

Sample Code

```
managed;
define authorization context NoCheckWhenPrivileged
{
  'ZBNSCLC_AO';
}
with privileged mode disabling NoCheckWhenPrivileged;
define own authorization context
{
  'ZBNSCLC_AO';
}
define behavior for z_i_bonus_calculation alias calculation
implementation in class zbp_bonus_calculation unique
persistent table zbonusclc
lock master
// Identity and Access Management:
// Enable RAP managed authorization check at create, update, and delete
// -> requires implementation of two methods, one with the addition "FOR
INSTANCE AUTHORIZATION"
//  and one with the addition "FOR GLOBAL AUTHORIZATION"
authorization master ( instance, global )
```

4.2.11.2.7.2 Providing Read Access Using Privileged Mode

Use the keyword `WITH PRIVILEGED ACCESS` to provide read access using privileged mode.

Context

For the bonus calculation job in the example used in this documentation, all valid bonus calculations with empty bonus amount are retrieved for calculation in the job. By using the keyword `WITH PRIVILEGED ACCESS` after the entity `z_i_bonus_calculation`, the CDS access controls with checks for authorization objects defined in authorization context `NoCheckWhenPrivileged` are disabled. As a result, unfiltered access to the `z_i_bonus_calculation` entity is possible.

Procedure

In the example used in this documentation, you can use the following code for selecting bonus calculations:

Sample Code

```
SELECT * FROM z_i_bonus_calculation
        WITH PRIVILEGED ACCESS
        WHERE z_i_bonus_calculation~validityenddate >= @sy-datum
        AND z_i_bonus_calculation~bonusamount_v = 0
        INTO TABLE @DATA(lt_bonus_calculation).
```

4.2.11.2.7.3 Providing Write Access Using Privileged Mode

Use the keyword `PRIVILEGED` to provide write access using privileged mode.

Context

After open bonus calculations have been retrieved, the bonus calculation action `calculatebonus` is triggered in the *Bonus Calculation* business object with privileged access. By using the keyword `PRIVILEGED` after the behavior definition `z_i_bonus_calculation`, you enable privileged mode. As a result, the bonus calculation is executed without further authorization checks in the RAP authorization control.

Procedure

In the example used in this documentation, you can use the following code:

Sample Code

```
MODIFY ENTITIES OF z_i_bonus_calculation
        PRIVILEGED
        ENTITY calculation
        EXECUTE calculatebonus FROM CORRESPONDING  #( lt_bonus_calculation )
        RESULT DATA(result)
        REPORTED DATA(reported)
        FAILED DATA(failed).
```

4.2.11.2.8 Troubleshooting

Learn more about apps and tools that help you identify and solve authorization issues.

4.2.11.2.8.1 Authorization Trace

There's an app for troubleshooting available: With the *Display Authorization Trace* app you can enable an authorization trace for a business user, which helps you to find out if the user's authorizations are insufficient.

More Information

Display Authorization Trace

4.2.11.2.8.2 Debugging

You can use the ABAP Debugger in ABAP development tools to gain more insights about authorization checks. You can do so by activating a breakpoint next to the performed `AUTHORITY-CHECK` as well as by adjusting the breakpoint activation to be effective for the relevant developer user.

More Information

For more information about the ABAP debugger, see the user guide for ABAP development tools (ADT).

4.2.12 Business Configuration

Find out how to create custom business configuration objects to enrich your extensibility solutions.

Context

An important part of a business application is business configuration. Business configuration in enterprise software refers to a predefined set of configuration options that affect its functionality and behavior. You can use your `SAP ABAP Environment` to create custom business configuration objects. Find out which tasks a developer (business role `SAP_BR_DEVELOPER`) needs to perform to implement business configuration objects. Mind that lifecycle management is usually performed by a separate user. See Business Configuration Change Logs for more information.

Procedure

Consider the following to implement your custom business configuration objects:

- Business configuration objects are based on database tables with the delivery type C (`@AbapCatalog.deliveryClass : #C`). This indicates that the content in these tables can be transported between tenants.
- Business configuration content is usually not directly maintained in a productive system but created in a development system and then transported to the test and productive system. As a result, the corresponding transport handling must also be implemented.
- Business users need a way to maintain content. The number of maintenance objects often exceeds the number of master data or transactional apps, yet at the same time, they are used less frequently. This suggests using a highly standardized way of providing business configuration maintenance apps.
- Business configuration administration often involves the maintenance of language-dependent texts.

To enable the maintenance of content, a full business object is built on top of the table and exposed via an OData service, following the RAP (Restful ABAP Programming) model. The resulting service can be consumed by a frontend application, allowing business users to maintain business configuration content.

SAP offers the *Custom Business Configurations* app as a standardized tool for business configuration maintenance. Please refer to Creating Business Configuration Apps with ABAP RESTful Application Programming Model and Custom Business Configurations App [page 1046].

As an alternative, you can also enable your business configuration object for upload or download, allowing you to quickly and reliably maintain large amounts of content. In this case, you're not required to develop a full business object. The generic file upload app supports the file format that can be downloaded from SAP ERP or SAP S/4HANA systems, thus supporting an easy migration of data from SAP on-premise systems. For more information, see here.

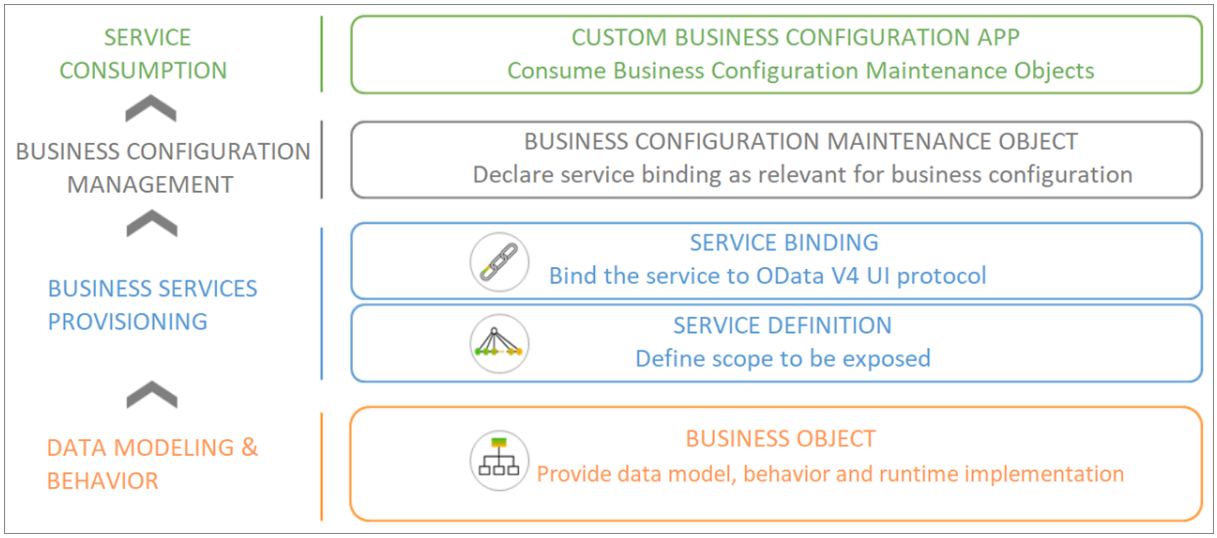
Should you have special requirements to your maintenance UI, you can also create and deploy your own custom Fiori application. For more information, seeDevelop an SAP Fiori Application UI and Deploy it ABAP Using Visual Studio Code.

> **Note**
>
> Although the management of customizing transport requests is usually done in the *Export stomizing Transports* app by the business process configuration expert, developers can also create such transport requests using the transport organizer view in `ABAP Development Tools`.

4.2.12.1 Creating Business Configuration Apps with ABAP RESTful Application Programming Model and Custom Business Configurations App

As an ABAP Developer you can leverage the *ABAP RESTful Application Programming Model* (RAP) and the *Custom Business Configurations* (CUBCO) app to create a SAP Fiori UI to maintain customizing table data. You don't need to create your own Fiori app. Everything can be implemented with ABAP repository objects.

For more information, see this tutorial.



The development of a business configuration maintenance Fiori UI requires developers to perform the following fundamental activities:

1. Developing Business Object

Based on the customizing tables you define a data model and business object structure. Check the prerequisites of the business configuration maintenance object to see which kind of business object is supported. Using model-driven development approach, you develop actions, validations, and determinations for the entities of the business object to change, maintain and validate configuration entries.

You have the option to use the Generate ABAP Repository Objects Wizard [page 1052] to generate all required development objects based on a customizng table. If required, you can then add further functions or enhance the generated business object structure.

For more information, see Developing a Ready-to-Run Business Object and Developing Business Logic.

2. Developing Business Service

Define the scope by exposing the entities of the business object in a service definition and bind the service to OData V4 UI protocol with a service binding. For more information on this, see Working with Business Services.

3. Developing Business Configuration Maintenance Object

Declare the service binding as relevant for business configuration by creating a Business Configuration Maintenance Object [page 1048].

4. Custom Business Configurations App

Based on the Business configuration maintenance object, the custom business configurations app renders a Fiori Elements List Report Floorplan for the entities exposed by the service binding to maintain the configuration entries. For more information, see Custom Business Configurations App [page 2970].

4.2.12.1.1 Business Configuration Maintenance Object

Purpose

A business configuration maintenance object declares a service binding as relevant for business configuration. It will be shown on the list of all maintainable business configurations in the Fiori app Custom Business Configurations App [page 2970]. The business configuration maintenance object can be maintained via API or ADT. For more information, see also Creating Business Configuration Apps with ABAP RESTful Application Programming Model and Custom Business Configurations App [page 1046].

Prerequisites

- The service binding must be of type OData V4 - UI and exposing CDS root entity with draft enabled behavior definition.
- The following two business object composition tree structures are supported:
  - the height of the tree is less or equal than 2.
    Example: Root entity A has two child entities B and C. Entity C has one child entity D. D can't have further child entities
  - the height of the tree is less equal than 3 and the option skip root entity list report is enabled. The root entity must be a singleton and have exactly one child entity.
    Example: Root entity S has one child entity A. Entity A has two child entities B and C. Entity C has one child entity D. D can't have further child entities.
- The data model must consist only of client-dependent tables.

Provide Authorizations for a Business Configuration

To grant frontend users the rights to use a business configuration maintenance object, first create an *Identity and Access Management (IAM) App* and assign it to an *IAM Business Catalog*. Follow this user guide but make sure to select the IAM app type *Business Configurations* app. In the service tab of the IAM app, select OData V4 as service type and the service binding name of the business configuration maintenance object as service name. In the authorizations tab maintain the authorization object used by the RAP BO for authorization control.

Once you have created the IAM business catalog, assign the catalog to a business role and finally assign the business role to business users to grant the rights to use your business configuration inside the *Custom Business Configurations* Fiori app. For more information, see Set Up Authorizations for Business Users in SAP S/4HANA Cloud Public Edition.

Related Information

Creating a Business Configuration Maintenance Object in ADT [page 1051]
Business Configuration Maintenance Object Cloud Platform API [page 1049]
Generating a Business Configuration Maintenance Object with the Generate ABAP Repository Objects Wizard [page 1052]

4.2.12.1.1.1 Business Configuration Maintenance Object Cloud Platform API

Use the ABAP API *mbc_cp_api* to create, update, delete, and read business configuration maintenance objects.

After a business configuration maintenance object has been created, it will be shown on the list of all maintainable business configurations in the Fiori app *Custom Business Configurations* if you have all necessary authorizations regarding the service of the business configuration. You can get the necessary authorizations by following the instructions in Business Configuration Maintenance Object [page 1048] > **Provide Authorizations for a Business Configuration**. The end user can then maintain data for the registered business configuration from the frontend.

Before using any of the following methods, first obtain an interface handle by calling `mbc_cp_api=>business_configuration_api` with an identifier for your business configuration.

If an error occurs while calling the following methods, an exception of type `cx_mbc_api_exception` will be raised. Use the method `if_xco_news~get_messages` to retrieve all messages that the exception carries.

Create a Business Configuration Maintenance Object

Use the `create` method of the obtained business configuration interface handle to create a new object. Supply the following mandatory parameters, as illustrated in the code sample below.

- A name that is shown on the frontend list of all registered business configurations.
- A description for the business configuration that is shown on the frontend.

  > ⓘ Note
  >
  > The name and description are language dependent. The translations can be maintained using the *Maintain Translations* app.

- The OData V4 - UI Service Binding that is used to maintain the business configuration data. The service must be draft enabled.
- The name of the service to be used (as defined in the Service Binding).

- The name of an entity set as exposed by the service definition. This entity set is used as the root node for the UI. Only for this root entity set and its associations a UI is shown. Keep in mind that only one level of sub nodes is supported.
- A transport request of type `Workbench` to write the business configuration registration to. An entry of type "SMBC" (Business Configuration Object) will be written to that transport request.

Sample Code

```
CLASS zcl_ys_register_bc DEFINITION PUBLIC FINAL CREATE PUBLIC
    INHERITING FROM cl_xco_cp_adt_simple_classrun.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.
ENDCLASS.
CLASS zcl_ys_register_bc IMPLEMENTATION.
  METHOD main.
    DATA(lo_business_configuration) =
mbc_cp_api=>business_configuration_api('/Y123456/HOLIDAY_CALENDAR').
    TRY.
        lo_business_configuration->create(
          iv_name            = 'Holiday Calendar'
          iv_description     = 'A calendar for holidays'
          iv_service_binding = '/Y123456/CAL_I_HOLIDAY_SB'
          iv_service_name    = 'Holiday'
          iv_service_version = 0001
          iv_root_entity_set = 'HolidayRoot'
          iv_transport       = 'X02K900025'
        ).
      CATCH cx_mbc_api_exception INTO DATA(lx_mbc_api_exception).
        DATA(lt_messages) = lx_mbc_api_exception->if_xco_news~get_messages( ).
        LOOP AT lt_messages INTO DATA(lo_message).
          " Use lo_message->get_text( ) to get the error message.
        ENDLOOP.
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

The supplied information and an application component are stored in the registration table. The application component is derived from the package that contains the registered service binding. If an error occurs while using the business configuration, it will be shown in an error message on the frontend.

The `create` method also has two optional parameters:

- skip root entity list report: If `abap_true`, the UI automatically navigates to the object page of the root entity skipping the list report. Exactly one root entity must exist.
- app configuration: Configuration of root entity list report and object pages. It is sufficient to maintain only those attributes that should deviate from the standard behavior.

Update a Business Configuration Maintenance Object

You can only update the name and the description of your business configuration by using the respective methods `update_name` and `update_decription`. A transport request of type Workbench must be supplied to transport the updated object.

Delete a Business Configuration Maintenance Object

To delete a business configuration maintenance object, use the method `delete`. A transport request of type `Workbench` must be supplied to transport the deletion.

Check if a Business Configuration Maintenance Object Exists

To check the existence of a business configuration maintenance object, use the method `exists`. The method has no parameters. It returns `abap_true` if the object exists, `abap_false` if it doesn't.

For an overview of business configuration maintenance objects, use CDS view `I_CustABAPObjDirectoryEntry` with `ABAPObjectType = 'SMBC'`.

Read a Business Configuration Maintenance Object

To retrieve the details of a business configuration maintenance object, use the method `read`. The method has no parameters. The returned structure contains all the metadata of the business configuration maintenance object.

4.2.12.1.1.2 Creating a Business Configuration Maintenance Object in ADT

Find out how to create a business configuration maintenance object using the ABAP Development Tools (ADT).

Procedure

1. In the *Project Explorer*, select the relevant package node.
2. Open the context menu and choose *New>Other ABAP Repository Object>Business Configuration Management>Business Configuration Maintenance Object>Next* to launch the creation wizard.
3. Enter a name and a description and choose *Next*.

Related Information

Working with Business Configuration Maintenance Objects

4.2.12.1.1.3 Generating a Business Configuration Maintenance Object with the Generate ABAP Repository Objects Wizard

You can create a business configuration maintenance object together with all related development objects on the basis of a database table by using the *Generate ABAP Repository Objects Wizard*.

Context

Creating a Fiori app to maintain customizing tables involves many different objects that need to be created manually. On the basis of a database table, this wizard creates all the development objects that are required to maintain the content of this table and, optionally, the content of foreign key associated tables and text tables using the *Custom Business Configurations* app. If required, you can then, for example, add further functions or enhance the generated business object structure. As an alternative to using the app, you can also build your own custom SAP Fiori elements app based on the generated objects. This is described in this blog entry.

For more information, see Creating Business Configuration Apps with ABAP RESTful Application Programming Model and Custom Business Configurations App. A tutorial on how to use this wizard and the *Custom Business Configurations* app is available here.

Prerequisites

The wizard distinguishes between three types of tables:

- The basis database table where you run the wizard with
- (Optional) Additional tables with a foreign key relation of type `#KEY` to the basis database table
- (Optional) Text tables with a foreign key relationship of type `#TEXT_KEY` to one of the tables mentioned above

All tables must fulfill the following requirements:

- have a client key field
- have delivery class C
- data maintenance must be allowed or restricted
- (optional) have a timestamp field with data element `ABP_LOCINST_LASTCHANGE_TSTMPL`. If the table doesn't contain this field, the concurrency control is not active

The basis databse table must also meet the following requirements:

- have a timestamp field with data element `ABP_LASTCHANGE_TSTMPL`. For more information, see RAP Reuse Data Elements. If the table doesn't contain this field, the entire ETag is handled by CDS entity `I_CstmBizConfignLastChgd`

The additional tables must also meet the following requirements:

- Have a foreign key of type `#KEY` that fully matches the primary key of the base table

The text tables must also meet the following requirements:

- have a language key field with type `ABAP.LANG`
- have a foreign key of type `#TEXT_KEY` that fully matches the primary key of one of the other tables

The software component of the target package and the table package must be changeable.

Procedure

The wizard can't overwrite existing objects or create only a subset of the objects. The generated objects will have the same ABAP language version as the target package.

1. In your ABAP project, open the context menu for a database table and choose *Generate ABAP Repository Objects*.
2. In the folder *Business Configuration Management*, select *Maintenance Object*. Choose *Next*. Enter the target package and choose *Next* again.
3. In the *Configure Generator* page you can define the names and options for the generated objects. The wizard fills all required fields as a proposal based on the description of the basis table. Select *Next.*
4. In the *Preview Generator Output* page, a preview list is available with the objects to be generated. Choose *Next*.
5. Assign a transport request.
6. Choose *Finish* to start the generation of the objects.
7. After the generation process is complete, the generated *Business Configuration Maintenance Object* is shown.

Procedure Results

The generated RAP business object has the following properties:

- draft enabled, managed
- authorization control based on authorization object `S_TABU_NAM` and the CDS view entity name of the basis table
- concurrency control based on the table timestamp fields
- integration of the BC transport API to validate and record changes on a customizing transport request
- To enable mass editing and manual transport selection, a singleton root entity is part of the generated data model. For more information, see here.
- internal numbering is used for key fields with `ABAP` type `raw(16) (UUID)`

Scenario Options

Add Copy Action

Select this to include a *Copy* action in the generated app. With this factory action a copy of the selected entry will be created.

Add Deprecate Actions

Select to include a Deprecate and Invalidate action in the generated app. The table must have a field `CONFIGDEPRECATIONCODE` with the data element `CONFIG_DEPRECATION_CODE`. With this action a selected

entry can be deprecated or invalidated. If a foreign key check is executed on a deprecated or invalidated check table entry, the check returns a warning (deprecated) or error (invalidated). If this option is selected, the user can delete a table entry only if it is in draft mode and the entry was never active.

Add Data Consistency Check

Select this option to include a *Validation* for the *Prepare* draft action in the generated app, which checks the consistency of field inputs with:

- regular type checks for all fields of the entity and, if possible, the fixed value and foreign key checks specified in the ABAP Dictionary
- Foreign key checks are performed if the annotation `@AbapCatalog.foreignKey.screenCheck : true` is used and the check table is not part of the RAP business object. The foreign key table must be the table itself in which the foreign key relation is defined, SY or SYST.
- The end of character-like key fields is checked for invalid characters like vertical or horizontal tab.

Copy Parameter Entity

If the option *Add Copy Action* is selected, an abstract entity for the user input is generated. Provide a name for the abstract entity. This is not necessary if all key fields except the client field of the basis table have type ABAP `type raw(16)`.

Enable Transport Selection Strip

Select to enable a transport selection strip in the header toolbar with the following properties:

- The transport request selection action is only displayed in edit mode.
- The transport request information is displayed in the header toolbar.
- If the save action is executed without a selected transport request and the transport is mandatory, the action to select a transport request is triggered automatically. After selecting a transport request, the save action continues.

If this option is not selected, a transport selection is made possible using a action button. This option is selected by default.

Transport Selection

Select *Manual* to include the action *Select Transport* in the generated app. With this action, you can select an existing customizing transport request before saving the configuration changes.

Select *Manual with preselection* to include the action *Select Transport* in the generated app. With this action, you can select an existing customizing transport request before saving the configuration changes. When the *Edit* action is performed, a customizing transport request is determined automatically. You can find the determination logic in the ABAP documentation of method `get_transport_request` of interface `if_mbc_cp_rap_tdat_cts`.

Select *No Transport* for an app without a transport option. Suitable for configurations that are to be adjusted in the productive client.

BC Management

BC Maintenance Object

Enter the name of the maintenance object.

Transport Object

You must specify the name of a transport object. A transport object of type `Individual Transaction` is generated. Configuration changes are recorded under this transport object.

Data Model

Tables

This list must contain the basis database table. You can add any additional tables that fulfill the requirements that should be part of the generated RAP business object.

Text Tables

You can add any text table that fulfills the requirements that are to be part of the generated RAP business object.

Example

Our example scenario consists of four editable entities with a 1:N cardinality at each level:

- Status Class
- Status Class Text
- Status Code
- Status Code Text

The corresponding database table definitions are as follows:

Status Class:

Sample Code

```
@EndUserText.label : 'Status Class'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #C
@AbapCatalog.dataMaintenance : #ALLOWED
define table /dmo/statusclass {
  key client            : abap.clnt not null;
  key status_class_id   : /dmo/status_class_id not null;
  last_changed_at       : abp_lastchange_tstmpl;
  local_last_changed_at : abp_locinst_lastchange_tstmpl;
}
```

Status Class Text

Sample Code

```
@EndUserText.label : 'Status Class Text'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #C
@AbapCatalog.dataMaintenance : #ALLOWED
define table /dmo/stclasstext {
  key client            : abap.clnt not null;
  key lang              : abap.lang not null;
  @AbapCatalog.foreignKey.keyType : #TEXT_KEY
  key status_class_id   : /dmo/status_class_id not null
    with foreign key /dmo/statusclass where client = /dmo/stclasstext.client
and status_class_id = /dmo/stclasstext.status_class_id;
  description           : /dmo/description;
  local_last_changed_at : abp_locinst_lastchange_tstmpl;
```

Status Code:

Sample Code

```
@EndUserText.label : 'Status Code'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #C
@AbapCatalog.dataMaintenance : #ALLOWED
define table /dmo/statuscode {
  key client            : abap.clnt not null;
  @AbapCatalog.foreignKey.keyType : #KEY
  key status_class_id   : /dmo/status_class_id not null with foreign key /dmo/
statusclass where client = /dmo/statuscode.client and status_class_id = /dmo/
statuscode.status_class_id;
  key status_code_id    : /dmo/status_code_id not null;
  local_last_changed_at : abp_locinst_lastchange_tstmpl;
}
```

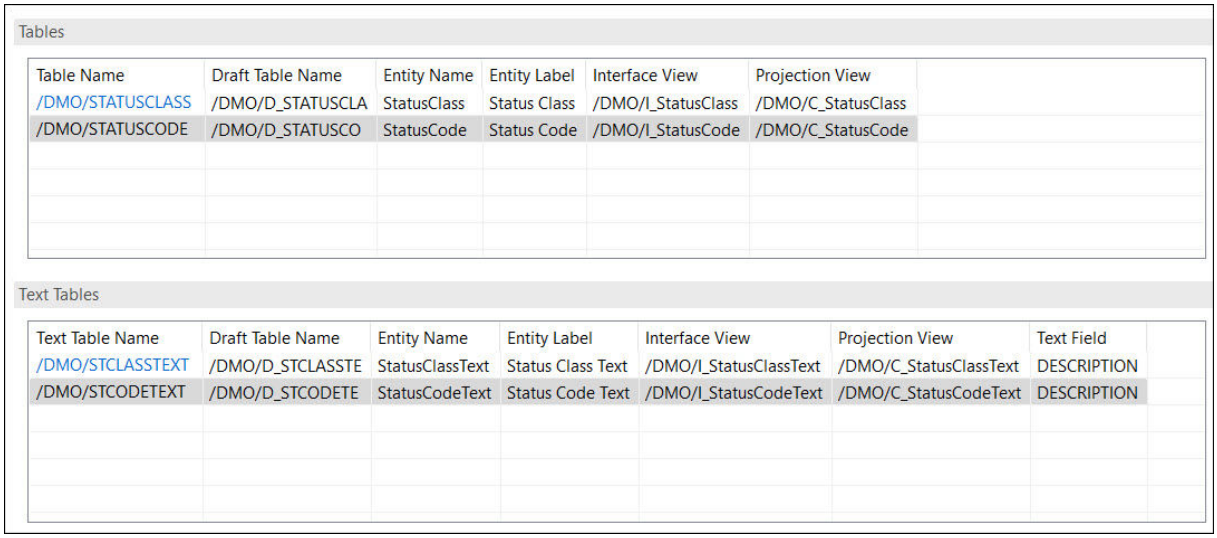
Status Code Text:

Sample Code

```
@EndUserText.label : 'Status Code Text'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #C
@AbapCatalog.dataMaintenance : #ALLOWED
define table /dmo/stcodetext {
  key client            : abap.clnt not null;
  key lang              : abap.lang not null;
  key status_class_id   : /dmo/status_class_id not null;
  @AbapCatalog.foreignKey.keyType : #TEXT_KEY
  key status_code_id    : /dmo/status_code_id not null with
foreign key /dmo/statuscode where client = /dmo/stcodetext.client and
status_class_id = /dmo/stcodetext.status_class_id and status_code_id = /dmo/
stcodetext.status_code_id;
  description           : /dmo/description;
  local_last_changed_at : abp_locinst_lastchange_tstmpl;
}
```

If you execute the wizard in the context of the table /DMO/STATUSCLASS, the text table /DMO/STCLASSTEXT is automatically added to the list of text tables in the data model section.

Due to the foreign key definition, you can also add the Status Code and Status Code Text table manually.



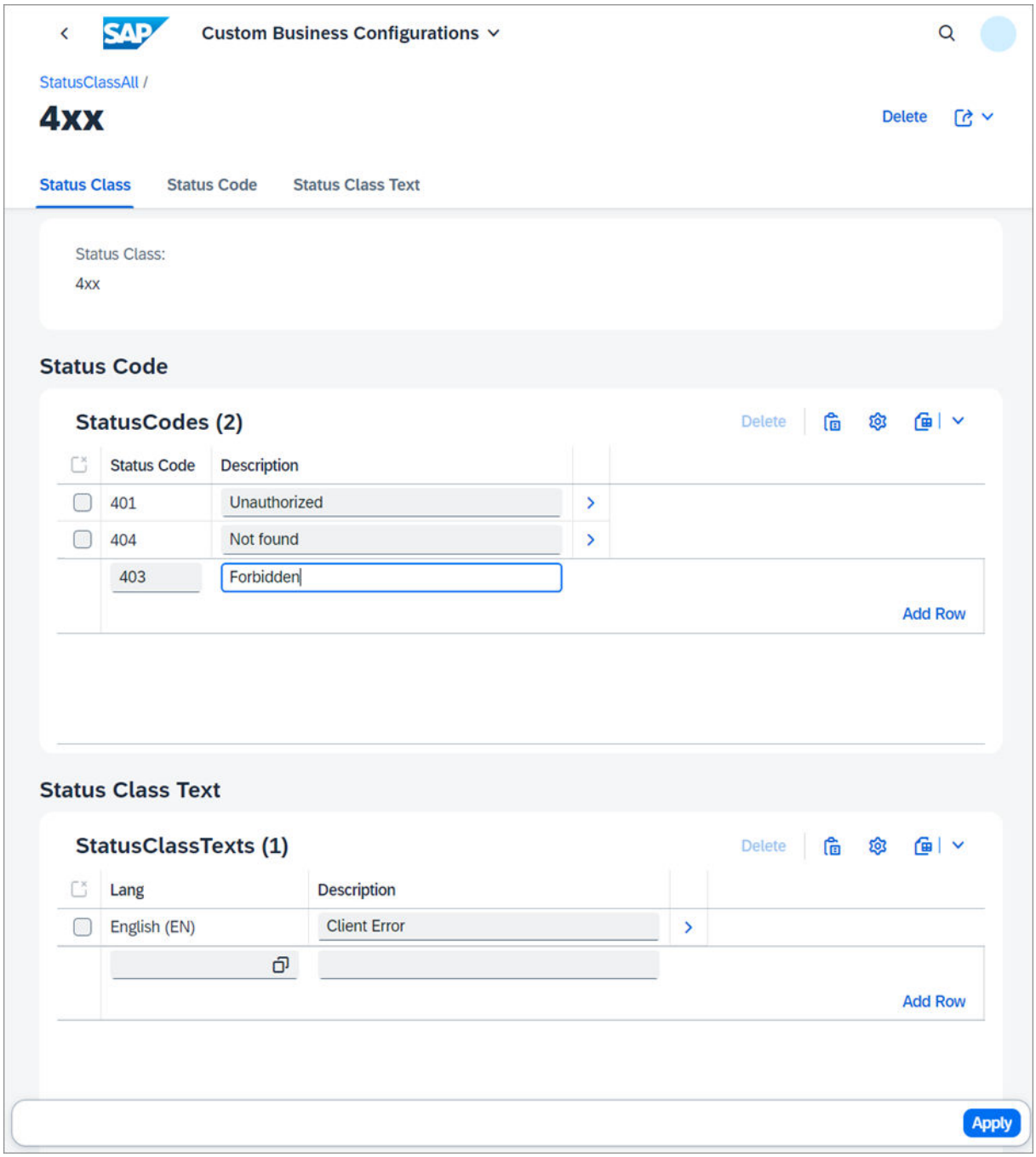
Tables

Table Name	Draft Table Name	Entity Name	Entity Label	Interface View	Projection View
/DMO/STATUSCLASS	/DMO/D_STATUSCLA	StatusClass	Status Class	/DMO/I_StatusClass	/DMO/C_StatusClass
/DMO/STATUSCODE	/DMO/D_STATUSCO	StatusCode	Status Code	/DMO/I_StatusCode	/DMO/C_StatusCode

Text Tables

Text Table Name	Draft Table Name	Entity Name	Entity Label	Interface View	Projection View	Text Field
/DMO/STCLASSTEXT	/DMO/D_STCLASSTE	StatusClassText	Status Class Text	/DMO/I_StatusClassText	/DMO/C_StatusClassText	DESCRIPTION
/DMO/STCODETEXT	/DMO/D_STCODETE	StatusCodeText	Status Code Text	/DMO/I_StatusCodeText	/DMO/C_StatusCodeText	DESCRIPTION

The resulting table maintenance application then looks like this:



4.2.12.1.2 Develop Common Capabilities

The guides in this section focus on specific development tasks common to a business configuration application. Whether or not you want to follow these guidelines depends on your initial situation and the desired result of your development.

> ### Note
>
> The Business Configuration Maintenance Object wizard [page 1052] takes these concepts into account. You can therefore use the wizard to generate an executable example which illustrates the implementation of these concepts.

- Authorization Control [page 1059]
- Transport Handling [page 1060]

4.2.12.1.2.1 Authorization Control

Context

Authorization control in RAP protects your business configuration object against unauthorized access to customizing data.

It's recommend to use the authorization `S_TABU_NAM` together with the CDS root entity name of the business object as table name to perform the read and modify operation. However, you can also check against the physical table names or use a different authorization object instead. Follow this blog on how to implement additional authorization objects.

Procedure

The advantage of using the CDS root entity name is that if the business object is extended by a new table you don't need to adjust the authorization check.

See also Provide Authorization on how to create the IAM app and add the authorization object.

Example

The business configuration CDS root entity name is `/ITAPC1/I_Status`.

The data control language (DCL) can be defined as follows:

> ### Sample Code
>
```
@MappingRole: true
define role /ITAPC1/I_Status {
  grant select on /ITAPC1/I_Status
  where ( ) = ASPECT pfcg_auth ( 'S_TABU_NAM', ACTVT = '03', TABLE = '/ITAPC1/
I_STATUS' );
}
```

The global authorization check in the behavior implementation can be defined as follows:

Sample Code

```
 METHOD GET_GLOBAL_AUTHORIZATIONS.
    AUTHORITY-CHECK OBJECT 'S_TABU_NAM' ID 'TABLE' FIELD '/ITAPC1/I_STATUS'
ID 'ACTVT' FIELD '02'.
    DATA(is_authorized) = COND #( WHEN sy-subrc = 0 THEN if_abap_behv=>auth-
allowed
                                  ELSE if_abap_behv=>auth-unauthorized ).
    result-%UPDATE      = is_authorized.
    result-%ACTION-Edit = is_authorized.
  ENDMETHOD.
```

The authorization instance of *S_TABU_NAM* in the IAM app can be defined as follows. Whether or not the user has write or read authorization is finally defined in the General Role Details section of the Business Role the Business Catalog with this IAM app is assigned to.

Sample Code

```
ACTVT = Change, Display
TABLE = /ITAPC1/I_STATUS
```

4.2.12.1.2.2 Transport Handling

Context

Business configuration content is usually not directly maintained in a productive system, but created in a development system, and then transported to the test system and productive system. As the developer of a business configuration app, you need to ensure that changes to the business configuration content is recorded onto a transport request of type *Customizing* and the user has the option to select the correct transport request.

Recording Changes on Transport Request

With the factory class *MBC_CP_API* you can instantiate a transport API to validate and record entity keys of a RAP business object on a transport request.

Sample Code

```
  DATA(api) = mbc_cp_api=>rap_tdat_cts(
              tdat_name = '/ITAPC1/CODE'
              table_entity_relations = VALUE #( ( entity = '/ITAPC1/
I_Status' table = '/ITAPC1/STATUS' )
                                                ( entity = '/ITAPC1/
I_StatusText' table = '/ITAPC1/STATUST' ) )
             ).
```

In the behavior implementation define an Additional Save Implementation in the header directly after the keyword managed. In the `save_modified` method you can record the changes on the transport request with

the method `record_changes` of this API. Check the ABAP Doc of `record_changes` for more information and example implementation.

For each entity of the RAP BO define a Validation on save for create, update and delete. In the validation implementation call the method `validate_changes` of the API. Check the ABAP Doc of validate_changes for more information and example implementation.

Transport Selection

The before mentioned API methods require a transport request to be passed. For the transport selection define an action with Abstract CDS Entity *D_SelectCustomizingTransptReqP* as the parameter. This CDS Entity returns all modifiable transport request of type *Customizing* where the user either owns the request or owns at least one open task. The transport request must have the attribute `SAP_CUS_TRANSPORT_CATEGORY` set to `MANUAL_CUST` or `DEFAULT_CUST`.

Save the result of the action in a virtual field of the draft entity. When recording or validating the changes the previously selected transport request can then be retrieved from the draft entity.

No Transport

When using the ADT wizard [page 1052] to generate a business configuration maintenance object, you can choose *No Transport* for *Transport Selection* for an app without transport option. Suitable for configurations that are to be adjusted in the production client. For more information read this blog post.

For more information read this blog post.

4.2.12.1.3 Develop UI-Specifics

The UI layout of a *Business Configuration Maintenance Object* in the *Custom Business Configurations* app depends on the following layers.

- CDS Annotations

You can use CDS annotations to define, among other things, the field labels and description, value helps, facets and actions.

- Fiori Elements Settings in the Business Configuration Maintenance Objects

You can configure the list report, object pages and table settings of the *Fiori Elements List Report Floor Plan* rendered by the *Custom Business Configurations* app.

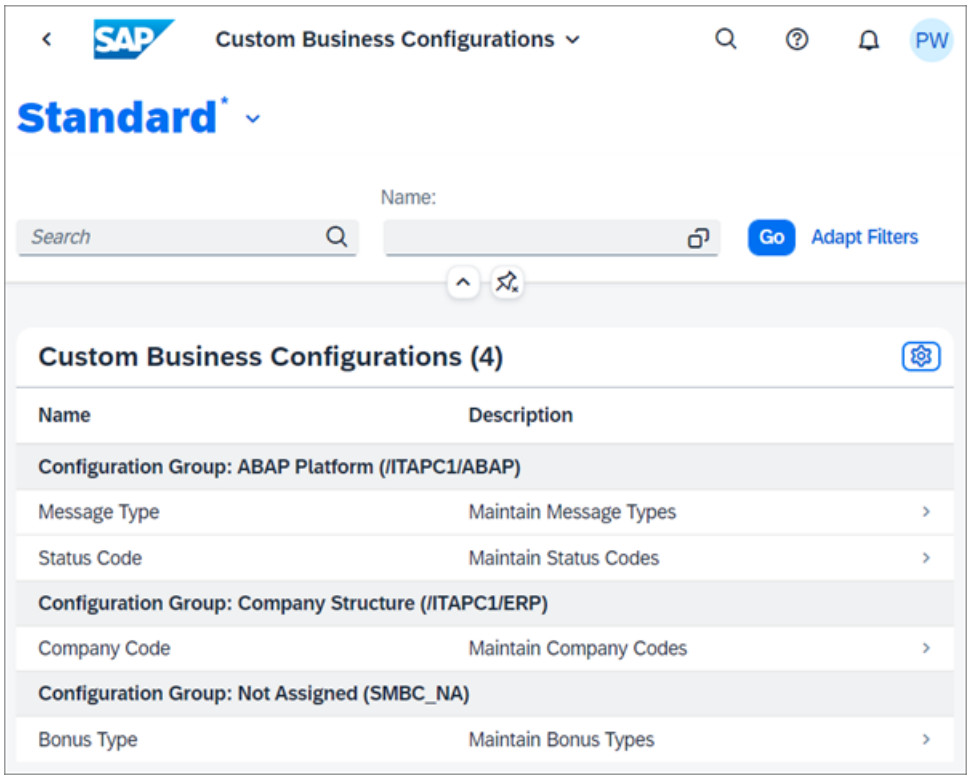
- Generic UI functions of the Custom Business Configurations App [page 2970].

Grouping of business configuration maintenance objects, intent navigation, display change logs and show documentation are generic UI functions of the *Custom Business Configurations*app.

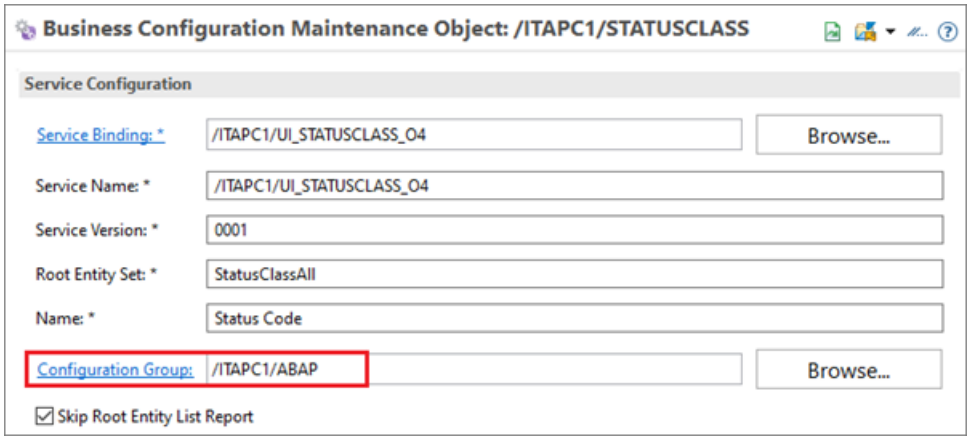
4.2.12.1.3.1 Grouping

The standard variant of the *Custom Business Configurations* app shows you the list of business configuration maintenance objects.

If the list needs to be structured in a simple hierarchical manner, you have the option to group the business configuration maintenance objects semantically.



As the developer of the business configuration maintenance object, you specify a data element of the same software component as configuration group.



The name of the data element can then be used by the custom business configurations app user as the *Group By* element in the table settings.

By clicking on settings you can navigate to the *View Settings* window.

The description of the configuration group data element is shown as the group label. Other attributes of the data element are not relevant. If the configuration group of a business configuration maintenance object is not maintained, the object is assigned to the generic group *Not Assigned*.

The *Custom Business Configurations* app user can save and share the variant with grouping in the My Views Dialog.

4.2.12.1.3.2 Table Creation Mode

If a new entry is added to a table, the *Custom Business Configurations* navigates to the object page of the new entry by default. This can be changed so that a `navigation is not triggered`.

This configuration is done with the *Table Creation Mode* attribute in the *Table Settings* section of the definition.

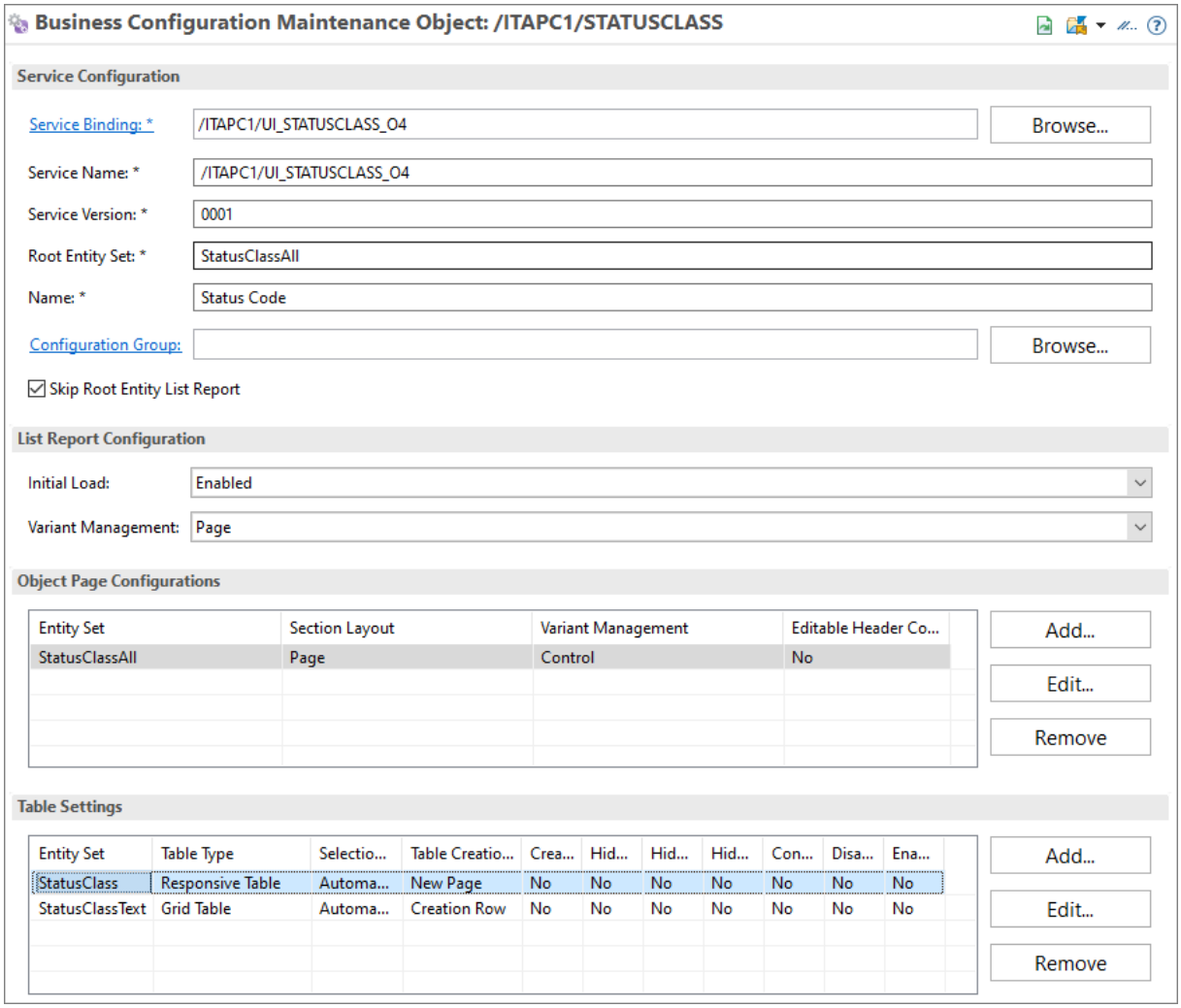
In the following example the business object consists of five entities:

[X11] /ITAPC1/UI_STATUSCLASS

```
1 define service /ITAPC1/UI_STATUSCLASS {
2   expose /ITAPC1/C_StatusClass_S as StatusClassAll;
3   expose /ITAPC1/C_StatusClass as StatusClass;
4   expose /ITAPC1/C_StatusClassText as StatusClassText;
5   expose /ITAPC1/C_StatusCodeText as StatusCodeText;
6   expose /ITAPC1/C_StatusCode as StatusCode;
7 }
```

For entity `StatusClass` the app shall navigate to the object page when creating a new table entry because the entity has many attributes which the user must maintain. Therefore *Table Creation Mode* is set to *New Page*.

The app shall not navigate to the object page when creating a new table entry for the entity `StatusClassText` because the user only needs to specify language key and text. Therefore *Table Creation Mode* is set to *Creation Row* and *Table Type* to *Grid Table*.



4.2.13 Integration and Connectivity

The ABAP environment enables you to connect to other systems using different protocols. It is possible to consume services from other systems and provide services for consumption in other systems.

Integration and Connectivity Development Guide [page 1065]
Get an overview of the ABAP integration and connectivity development guide.

SAP-Provided APIs [page 1068]

Integrating On-Premise Systems [page 1214]
Set up the Cloud Connector to enable communication from the ABAP environment to your on-premise systems using Remote Function Calls (RFC) and HTTP calls.

Managing the SAP Business AI Platform, ABAP environment Service Instance [page 1218]

Integrating the ABAP Environment with SAP S/4HANA Cloud [page 1222]
You can integrate the SAP Business AI Platform, ABAP environment with an SAP S/4HANA Cloud Public Edition system. This allows ABAP developers to implement outbound service calls from the ABAP environment to the SAP S/4HANA Cloud Public Edition system.

Developing Metrics for Health Monitoring [page 1223]
: Learn how you can add your own metrics to Health Monitoring in SAP Cloud ALM or SAP Focused Run.

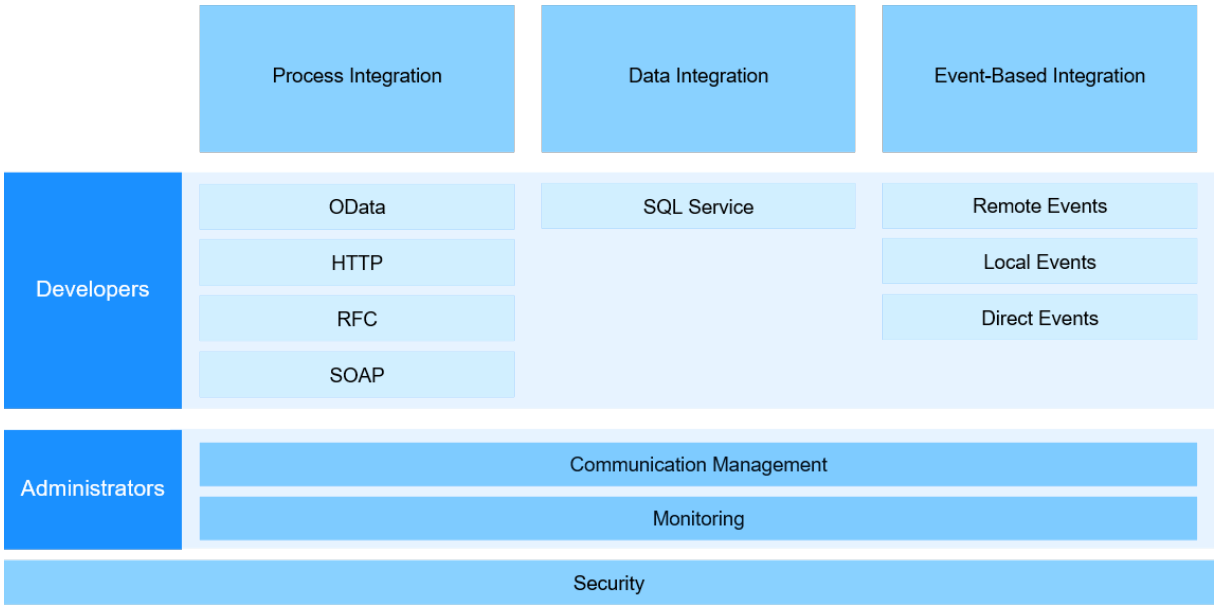
Accessing ABAP-Managed Data Using SQL Services for Data Integration Scenarios [page 1232]
: As a developer in the ABAP environment, you can expose CDS view entities in an ABAP system for external consumption using SQL services.

4.2.13.1 Integration and Connectivity Development Guide

Get an overview of the ABAP integration and connectivity development guide.

ABAP integration and connectivity allows different systems and applications to communicate and work together. In the context of the ABAP environment, this means enabling ABAP applications to exchange data and functionality with other ABAP and non-ABAP systems. Key technologies make this possible, such as APIs, events, web services, and various communication protocols. The goal is to create an integrated IT landscape where applications can seamlessly share information and processes.

These technologies are described in detail with corresponding examples in the ABAP integration and connectivity development guide.



- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/process-integration [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/process-integration]
- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/data-integration [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/data-integration]
- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/about-business-events [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/about-business-events]
- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/odata [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/odata]

- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/http [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/http]
- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/rfc [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/rfc]
- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/soap [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/soap]
- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/inbound-data-integration-using-sql [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/inbound-data-integration-using-sql]
- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/remote-events [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/remote-events]
- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/remote-events [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/remote-events]
- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/remote-events [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/remote-events]
- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/communication-management [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/communication-management]
- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/monitoring [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/monitoring]
- https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/security-for-abap-integration-and-connectivity [https://help.sap.com/docs/abap-cloud/abap-integration-connectivity/security-for-abap-integration-and-connectivity]

4.2.13.1.1 Enhanced WSDL Document

The enhanced Web Services Description Language (WSDL) document is a WSDL document enhanced by ABAP information that is relevant for design time.

When you use a regular WSDL as input to create a Service Consumption Model (SRVC), the design time relevant information, such as the types, messages, and port type sections, are analyzed and interpreted by the ABAP Web service framework. For all the ABAP properties like the names and types, it will make suggestions and generate a WSDL similar to the input WSDL enhanced with this ABAP information. If needed, you can change these ABAP properties within the enhanced WSDL document to adjust the ABAP artifacts accordingly (upon activation). The mapping between XSD and ABAP types in particular can be adjusted this way. Of course, an enhanced WSDL document can be used as input as well.

To encode ABAP properties into the WSDL, the namespace `http://sap.com/abap/proxy` with the prefix `abap` was introduced: `xmlns:abap=http://sap.com/abap/proxy`. For any applicable node of a WSDL, the corresponding enhanced WSDL document contains additional nodes within this namespace to accommodate its ABAP properties. Wherever you see the `abap` prefix in the WSDL, you can adjust the ABAP property. These additional nodes can be XSD attributes or elements, depending on where the node to be enhanced is located. This is necessary to maintain WSDL validity. Schema nodes such as `xmlns:xsd=http://www.w3.org/2001/XMLSchema` are enhanced via additional XSD attributes. WSDL nodes `xmlns:wsdl=http://schemas.xmlsoap.org/wsdl/`, on the other hand, are enhanced via additional XSD (sub)elements. Regular ABAP properties of any node contain the following:

- Prefix
- Name
- Description
- ABAP type

With the ABAP type, you can control the mapping between XSD and ABAP types. The property `availableTypes` displays all applicable ABAP types in this case. To change the ABAP type, copy and paste one of the listed types into the type attribute of the given element. Afterwars, reactivate the SRVC to apply the changes.

Summary of Additional Nodes and Their Technical Types

Node name	Technical Type	Description
`abap:prefix`	Up to 20 characters, case insensitive	ABAP prefix (only used if the object is represented by an ABAP object such as `CLAS`, `INTF`, `TABL`, `DTEL`)
`abap:name`	Up to 30 characters, case insensitive	ABAP name
`abap:type`	String	Default or set ABAP type
`abap:availableTypes`	Comma-separated list of 30 characters	List of all available ABAP types
`abap:description`	Up to 60 characters	Description
`abap:prefixT`	See `abap:prefix`	Prefix for corresponding table type (if `maxOccurs` > 1 and additional table type is generated and a prefix is required)
`abap:nameT`	See `abap:name`	Name for corresponding table type (if `maxOccurs` > 1 and additional table type is generated)
`abap:descriptionT`	See `abap:description`	Description for corresponding table type (if maxOccurs > 1 and additional table type is generated)

Example

The following code snippet displays a types section of an enhanced WSDL with a schema node (prefix `xsd`), where all ABAP properties are encoded as XSD attributes:

Sample Code

```
   <xsd:element name="NumberToWords">
    <xsd:complexType abap:prefix="ZZZ_"
abap:name="ZZZ_NUMBER_TO_WORDS_SOAP_REQUE" abap:description="Proxy
Structure (generated)">
     <xsd:sequence>
      <xsd:element name="ubiNum" form="qualified" type="xsd:unsignedLong"
abap:name="UBI_NUM" abap:type="INT8" abap:availableTypes="INT8, INT4,
DEC-20"/>
     </xsd:sequence>
    </xsd:complexType>
   </xsd:element>
```

The element `NumberToWords` being a `complexType` is represented by the dictionary structure `ZZZ_NUMBER_TO_WORDS_SOAP_REQUE` in ABAP. The element within this `complexType` `ubiNum` of XSD type `xsd:unsignedLong` corresponds to structure entry `UBI_NUM` of ABAP type `INT8`, which you could change to `INT4` or `DEC-20` (`availableTypes`).

The second snippet represents an exemplary `portType` section, where all ABAP properties are encoded as additional XSD elements (prefix `wsdl`) rather than attributes:

Sample Code

```
<wsdl:portType name="NumberConversionSoapType">
  <abap:prefix>ZZZ_</abap:prefix>
  <abap:name>ZZZ_CO_NUMBER_CONVERSION_SOAP</abap:name>
  <abap:description>Proxy Class (generated)</abap:description>
  <wsdl:operation name="NumberToWords">
   <abap:name>NUMBER_TO_WORDS</abap:name>
   <abap:description>Returns the word corresponding to the positive number
passed</abap:description>
   <wsdl:input message="tns:NumberToWordsSoapRequest"/>
   <wsdl:output message="tns:NumberToWordsSoapResponse"/>
  </wsdl:operation>
 </wsdl:portType>
```

In this code snippet, the generated ABAP class is called `ZZZ_CO_NUMBER_CONVERSION_SOAP` and its method is `NUMBER_TO_WORDS`, which corresponds to the operation `NumberToWords` of the service.

4.2.13.2 SAP-Provided APIs

Integrating Business Users from an External Identity Management System [page 1069]
: You can use this communication scenario for the business user.

Integrating Business Role Change Documents [page 1109]
: You can use this scenario to read the change documents of business roles.

Integrating Business User Change Documents [page 1112]
: You can use this scenario to read the change documents of business users.

Integrating Business Role Data (Read) [page 1114]
: You can use this communication scenario to read business role data.

Integrating Business User Logon Details (Read) [page 1122]
: You can use this communication scenario to read business user logon details.

Integrating Support User Data (Read) [page 1127]
: You can use this communication scenario to read support user data.

Integrating Communication Arrangement Data (Read) [page 1130]
: You can use this communication scenario to read communication arrangement data.

Integrating Communication Management Data (Read and Write) [page 1137]
: You can use this communication scenario to read and write communication arrangement, communication system and communication user data.

Integrating Communication System Data (Read) [page 1147]

You can use this communication scenario to read communication system data.

Integrating Communication User Data (Read) [page 1155]
You can use this communication scenario to read communication user data.

Integrating Content Security Policy Data (Read) [page 1160]
You can use this communication scenario to read Content Security Policy (CSP) data.

Integrating Protection Allowlist Data (Read) [page 1163]
You can use this communication scenario to read protection allowlist data.

Integrating Certificate Data (Read) [page 1170]

Integrating System for Cross-Domain Identity Management (SCIM) [page 1180]
You can use this communication scenario to maintain data via SCIM.

Manage Git Repository [page 1187]
The Manage Git Repository API allows you to manage software components (Git repositories) on an SAP Business AI Platform, ABAP environment system.

4.2.13.2.1 Integrating Business Users from an External Identity Management System

You can use this communication scenario for the business user.

The communication scenario `SAP_COM_0093` allows you to use and provision business users from an external data source, for example an identity management system, in SAP Integrated Business Planning.

The communication scenario gives you access to two SOAP services:

- `MANAGEBUSINESSUSERIN` lets you create business users in your system
- `QUERYBUSINESSUSERIN` lets you read existing business users from your external data source into your own identity management system.
- `QueryBusinessUserMetadaIn` lets you read metadata information for the service `MANAGEBUSINESSUSERIN` from an external data source such as an identity management system.
  The service provides the search parameter `RoleCategory`, which restricts the result set of the metadata information of the service. The response includes the metadata information based on the given criteria.
  If errors occur, the log contains information about the severity code, the message number, and message texts.

> **Note**
>
> This scenario processes personal data. For more information on how to handle data protection and privacy, see Data Protection and Privacy.

For more information, see the service documentation on the SAP Business Accelerator Hub and search for *SAP IBP - Identity Management*.

Related Information

Inbound Service: Business User [page 1070]
Inbound Service: Business User - Read [page 1087]
Business User - Read Metadata [page 1102]

4.2.13.2.1.1 Inbound Service: Business User

Technical name: `MANAGEBUSINESSUSERIN`

This synchronous inbound SOAP service enables you to create, update, and delete business users from your external data sources, such as an identity management system. Deleting business users doesn't mean you've actually deleted them yet. The user assigned to the business user is deleted and the `MarkedForArchivingIndicator` has been set. Only during the ILM process the business user is physically deleted.

You can assign business role IDs to the users at the node `Role`.

We recommend processing blocks of 10 users to a maximum of 100 users. Otherwise, the target system may time out.

This service supports the business users Employee (BUP003).

> ⚠ Caution
>
> This service directly influences the data and authorizations of business users. Changes are effective immediately in the target system.
>
> Make sure to maintain only those authorizations that are intended for what a user needs to do in the system. Not doing so can cause security issues.

Service Request

The service is structured into the following two top-level nodes:

Message Header (`MessageHeader`)

The service message header is not in use in this service.

Business User (`BusinessUser`)

The service nodes contain the service's business data.

> ⓘ Note
>
> In the following table, attributes are marked in blue.

Nodes and Fields for the BusinessUser Node

Node or Field	Description	Maximum Field Length	Cardinality	Necessity
`PersonExternalID`	The Person External ID is the ID of the employee.	60	0..1	Mandatory for create for business partner category (`BUP003` - Employee). Optional for update (at least one of the person IDs is mandatory.)
`PersonID`	Person ID is the ID of the business partner.	10	0..1	Optional (At least one of the person IDs is mandatory.)
`PersonUUID`	Person UUID is the unique ID of the business partner.	36	0..1	Optional (At least one of the person IDs is mandatory.)
`BusinessPartnerRoleCode`	Business Partner Role Code	6	0..1	Mandatory for create for Business partner category (`BUP003` - Employee). Optional for update.

Node or Field		Description	Maximum Field Length	Cardinality	Necessity
`MarkedForArchivingIndicator`		Mark for Archiving Set to **True**: • The business user will be archived • The `actionCode` [1] for `User` must be set to 02 Set to **False**: • The business user will be reactivated (Undo Archive) • The `actionCode` [1] for `User` must be set to 02		0..1	Optional
`ValidityPeriod` Cardinality: 0..1	`StartDate`	Format: YYYY-MM-DD By default, the system date is set.		0..1	Optional
	`EndDate`	Format: YYYY-MM-DD By default, 9999-12-31 is set.		0..1	Optional
`PersonalInformation`	`FormOfAddress`	Form of address	4	0..1	Optional

Node or Field		Description	Maximum Field Length	Cardinality	Necessity
Cardinality: 0..1	`FirstName`	First name	40	0..1	Optional
	`LastName`	Last name This field is mandatory.	40	0..1	Mandatory
	`PersonFullName`	Person full name	80	0..1	Optional
	`AcademicTitle`	Academic title	4	0..1	Optional
	`CorrespondenceLanguage`	Correspondence language	9	0..1	Optional
	`MiddleName`	Middle name	40	0..1	Optional
	`AdditionalLastName`	Additional last name	40	0..1	Optional
	`BirthName`	Birth name	40	0..1	Optional
	`NickName`	Nick name	40	0..1	Optional
	`Initials`	Initials	10	0..1	Optional
	`AcademicSecondTitle`	Academic second title	4	0..1	Optional
	`LastNamePrefix`	Last name prefix	4	0..1	Optional
	`LastNameSecondPrefix`	Last name second prefix	4	0..1	Optional
	`NameSupplement`	Name supplement	4	0..1	Optional

Node or Field		Description	Maximum Field Length	Cardinality	Necessity
	`actionCode`	You can use the following values: • 01 - Create • 02 - Update • 03 - Delete Mandatory if [2] is not set and personal information data are given.	2	optional	Optional
`User` **(only for Cloud)** Cardinality: 0..1 **Note** For more information about default values, see Defaulting of User Attributes.	`UserName`	User name/ Alias	40	0..1	Optional
	`LogonLanguageCode`	Logon language	9	0..1	Optional

Node or Field	Description	Maximum Field Length	Cardinality	Necessity
`DateFormatCode`	You can use the following values: • 1 - DD.MM.YYYY (Gregorian Date) • 2 - MM/DD/YYYY (Gregorian Date) • 3 - MM-DD-YYYY (Gregorian Date) • 4 - YYYY.MM.DD (Gregorian Date) • 5 - YYYY/MM/DD (Gregorian Date) • 6 - YYYY-MM-DD (Gregorian Date, ISO 8601) • 7 - GYY.MM.DD (Japanese Date) • 8 - GYY/MM/DD (Japanese Date) • 9 - GYY-MM-DD (Japanese Date)	2	0..1	Optional

Node or Field		Description	Maximum Field Length	Cardinality	Necessity
		• A - YYYY/MM/DD (Islamic Date 1) • B - YYYY/MM/DD (Islamic Date 2) • C - YYYY/MM/DD (Iranian Date)			
	`DecimalFormatCode`	You can use the following values: • 1.234.567,89 • X - 1,234,567.89 • Y - 1 234 567,89	2	0..1	Optional
	`TimeZoneCode`	Time zone	10	0..1	Optional

Node or Field		Description	Maximum Field Length	Cardinality	Necessity
	`TimeFormatCode`	You can use the following values: • 0 - 24 Hour Format (Example: 12:05:10) • 1 - 12 Hour Format (Example: 12:05:10 PM) • 2 - 12 Hour Format (Example: 12:05:10 pm) • 3 - Hours from 0 to 11 (Example: 00:05:10 PM) • 4 - Hours from 0 to 11 (Example: 00:05:10 pm)	2	0..1	Optional
	`LockedIndicator`	Locked indicator	5	0..1	Optional

	`ValidityPeri od`
	`StartDate`
	Cardinality: 1
		Format:
		YYYY-MM-DD
		If the start date is not maintained for the `User`, the `StartDate` for the `BusinessUser` is entered.
				1	Optional

Node or Field			Description	Maximum Field Length	Cardinality	Necessity
		`EndDate`	Format: YYYY-MM-DD If the `EndDate` is not maintained, it is set to 9999-12-31.		1	Optional
	`Role` Cardinality: 0..unbounded	`RoleName`	Role name	40	1	Optional
		`actionCode`	You can use the following values: • 01 - Create • 03 - Delete Mandatory if [6] is not set and role name data is given.	2	Optional	Optional
	GlobalUserID		Used to uniquely identify your business user across different systems.	36		Optional
	UserGroupCode		Assign a business user to an existing user group.	12	0..1	Optional

Node or Field		Description	Maximum Field Length	Cardinality	Necessity
	`actionCode`	You can use the following values: • 01 - Create • 02 - Update • 03 - Delete Mandatory if [3] is not set and user data (`UserName` and `Role`) are given.	2	Optional	Optional
	[6] `roleListCompleteTransmissionIndicator`	CTI for the `Role` node See CTI note [page 1083] for more information.		Optional	Optional
`UserAssignment` **(only for on-premise)** Cardinality: 0..1	`UserID`	User ID	12	1	N/A
	`actionCode`	You can use the following values: • 01 - Create • 02 - Update • 03 - Delete Mandatory if [4] is not set and User ID data are given.	2	Optional	N/A
`WorkplaceInformation`	`EmailAddress`	Email address	241	0..1	N/A
	`PhoneInformation` `PhoneType` Cardinality: 0..2 One set of phone informa-	Phone type • B - Business • C - Cell	1	1	N/A

Node or Field			Description	Maximum Field Length	Cardinality	Necessity
	tion per phone type supported.	`CountryDialingCode`	Country dialing code Used for both phone types.	10	0..1	N/A
		`PhoneNumberAreaID`	Phone number area code Used only for phone type B.	10	0..1	N/A
		`PhoneNumberSubscriberID`	Phone number subscriber ID Used for both phone types.	30	0..1	N/A
		`PhoneNumberExtension`	Phone number extension Used only for phone type B.	10	0..1	N/A
		`actionCode`	You can use the following values: • 01 - Create • 02 - Update • 03 - Delete Mandatory if [7] is not set and phone data is given.	2	Optional	N/A
	`FunctionalTitleName`		Functional title name	40	0..1	N/A
	`Department`		Department name	40	0..1	N/A
	`RoomNumber`		Room number	10	0..1	N/A
	`Building`		Building name	10	0..1	N/A

Node or Field			Description	Maximum Field Length	Cardinality	Necessity
	`actionCode`		You can use the following values: • 01 - Create • 02 - Update • 03 - Delete Mandatory if [5] is not set and workplace information data is given.	2	Optional	N/A
	[7] `phoneInformationListCompleteTransmissionIndicator`		CTI for the `PhoneInformation` node See CTI note [page 1083] for more information.		Optional	N/A
`Relationship` (only for on-premise)	`Partner1`	`BusinessPartnerID`				N/A
		`BusinessPartnerUUID`				N/A
		`BusinessPartnerExternalID`				N/A
		`BusinessPartnerExternalIDCategoryCode`				N/A
	`Partner2`	`BusinessPartnerID`				N/A
		`BusinessPartnerUUID`			0..1	N/A
		`BusinessPartnerExternalID`			0..1	N/A

Node or Field	Description	Maximum Field Length	Cardinality	Necessity
`BusinessPartnerExternalIDCategoryCode`			0..1	N/A
`RelationshipCategoryCode`			0..1	N/A
`actionCode`			Optional	N/A
[1] `actionCode`	You can use the following values: • 01 - Create • 02 - Update • 03 - Delete This attribute is mandatory.	2	Optional	Optional
[2] `personalInformationListCompleteTransmissionIndicator`	CTI for the `PersonalInformation` node See CTI note [page 1083] for more information.		Optional	Optional
[3] `userListCompleteTransmissionIndicator`	CTI for the `User` node See CTI note [page 1083] for more information.		Optional	Optional
[4] `userAssignmentListCompleteTransmissionIndicator`	CTI for the `UserAssignment` node See CTI note [page 1083] for more information.		Optional	Optional

Node or Field	Description	Maximum Field Length	Cardinality	Necessity
[5] `workplaceInformationListCompleteTransmissionIndicator`	CTI for the `WorkplaceInformation` node See CTI note [page 1083] for more information.		Optional	Optional
[5] `relationshipListCompleteTransmissionIndicator`			Optional	Optional

> **Note**
>
> **Complete Transmission Indicator (CTI)**
>
> The complete transmission indicator represents the state of the data in the inbound XML There are multiple transmission indicators that represents the state of the corresponding segment. The transmission indicator is interpreted by the web service:
>
> - If a complete transmission indicator is set to true, the existing data is overwritten by the newly entered data.
> - If a complete transmission indicator is set to false, the value of the action code of the corresponding node applies.
> - If an action code and the complete transmission indicator are set, then the action code of the corresponding node applies.

Sample Payload

> **Sample Code**

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:aba="http://sap.com/xi/ABA">
   <soapenv:Header/>
   <soapenv:Body>
      <aba:BusinessUserBundleMaintainRequest_sync>
          <!--1 or more repetitions:-->
         <BusinessUser actionCode="01"
personalInformationListCompleteTransmissionIndicator="false"
userListCompleteTransmissionIndicator="false"
userAssignmentListCompleteTransmissionIndicator="false"
workplaceInformationListCompleteTransmissionIndicator="false">
            <PersonExternalID>Muster01</PersonExternalID>
            <BusinessPartnerRoleCode>BUP003</BusinessPartnerRoleCode>
            <PersonalInformation actionCode="01">
               <FormOfAddress>0002</FormOfAddress>
               <FirstName>Max</FirstName>
               <LastName>Muster</LastName>
               <PersonFullName>Prof. Dr. Max Muster</PersonFullName>
               <AcademicTitle>0002</AcademicTitle>

               <CorrespondenceLanguage>D</CorrespondenceLanguage>
```

```
               <MiddleName>Michael</MiddleName>
               <AcademicSecondTitle>0001</AcademicSecondTitle>
               <BirthName>Milli</BirthName>
               <NickName>Maxi</NickName>
               <LastNamePrefix>0001</LastNamePrefix>
            </PersonalInformation>
            <User actionCode="01"
roleListCompleteTransmissionIndicator="false">
               <!--Optional:-->
               <UserName>MAXMUSTER01</UserName>
               <LogonLanguageCode>DE</LogonLanguageCode>
               <LockedIndicator>false</LockedIndicator>
               <Role actionCode="01">
                  <RoleName>SAP_BR_MANAGER</RoleName>
               </Role>
               <Role actionCode="01">
                  <RoleName>SAP_BR_BPC_EXPERT</RoleName>
               </Role>
            </User>
            <WorkplaceInformation actionCode="01"
phoneInformationListCompleteTransmissionIndicator="true">
               <EmailAddress>Max.Muster01@Test.com</EmailAddress>
               <PhoneInformation actionCode="01">
                  <PhoneType>C</PhoneType>
                  <CountryDialingCode>+49</CountryDialingCode>
                  <PhoneNumberSubscriberID>0160123456</
PhoneNumberSubscriberID>
               </PhoneInformation>
               <PhoneInformation actionCode="01">
                  <PhoneType>B</PhoneType>
                  <CountryDialingCode>+49</CountryDialingCode>
                  <PhoneNumberAreaID>06227</PhoneNumberAreaID>
                  <PhoneNumberSubscriberID>7</PhoneNumberSubscriberID>
                  <PhoneNumberExtension>12345</PhoneNumberExtension>
               </PhoneInformation>
               <FunctionalTitleName>TESTER</FunctionalTitleName>
               <Department>QUALITY</Department>
               <RoomNumber>C1.23</RoomNumber>
               <Building>WDF01</Building>
            </WorkplaceInformation>
         </BusinessUser>
         <BusinessUser actionCode="01"
personalInformationListCompleteTransmissionIndicator="false"
userListCompleteTransmissionIndicator="false"
userAssignmentListCompleteTransmissionIndicator="false"
workplaceInformationListCompleteTransmissionIndicator="false">
            <PersonExternalID>MINIMUSTER01</PersonExternalID>
            <BusinessPartnerRoleCode>BUP003</BusinessPartnerRoleCode>
            <PersonalInformation actionCode="01">
               <FormOfAddress>0001</FormOfAddress>
               <FirstName>Mini</FirstName>
               <LastName>Muster</LastName>
               <PersonFullName>Prof. Dr. Mini Muster</PersonFullName>
               <AcademicTitle>0002</AcademicTitle>
               <CorrespondenceLanguage>D</CorrespondenceLanguage>
               <AcademicSecondTitle>0001</AcademicSecondTitle>
               <LastNamePrefix>0001</LastNamePrefix>
            </PersonalInformation>
            <User actionCode="01"
roleListCompleteTransmissionIndicator="false">
               <!--Optional:-->
               <UserName>MINIMUSTER01</UserName>
               <LogonLanguageCode>DE</LogonLanguageCode>
               <LockedIndicator>false</LockedIndicator>
               <Role actionCode="01">
                  <RoleName>SAP_BR_MANAGER</RoleName>
               </Role>
               <Role actionCode="01">
```

```
                  <RoleName>SAP_BR_BPC_EXPERT</RoleName>
               </Role>
            </User>
            <WorkplaceInformation actionCode="01"
phoneInformationListCompleteTransmissionIndicator="true">
               <EmailAddress>Mini.Muster01@Test.com</EmailAddress>
               <PhoneInformation actionCode="01">
                  <PhoneType>C</PhoneType>
                  <CountryDialingCode>+49</CountryDialingCode>
                  <PhoneNumberSubscriberID>0160123456</
PhoneNumberSubscriberID>
               </PhoneInformation>
               <PhoneInformation actionCode="01">
                  <PhoneType>B</PhoneType>
                  <CountryDialingCode>+49</CountryDialingCode>
                  <PhoneNumberAreaID>06227</PhoneNumberAreaID>
                  <PhoneNumberSubscriberID>7</PhoneNumberSubscriberID>
                  <PhoneNumberExtension>12345</PhoneNumberExtension>
               </PhoneInformation>
               <FunctionalTitleName>TESTER</FunctionalTitleName>
               <Department>QUALITY</Department>
               <RoomNumber>C1.23</RoomNumber>
               <Building>WDF01</Building>
            </WorkplaceInformation>
         </BusinessUser>
      </aba:BusinessUserBundleMaintainRequest_sync>
   </soapenv:Body>
</soapenv:Envelope>
```

Service Response

You receive a confirmation message response for each bundle of business users you send. If the service request is processed, a confirmation message is sent. This contain crucial information provided by the fields `PersonExternalID`, `PersonID`, and `PersonUUID` for each business user of the bundle.

The following table provides an overview of the response structure for the `BusinessUser` service node.

Field or Node		Description	Maximum Field Length	Cardinality
`PersonExternalID`		Person External ID	60	0..1
`PersonID`		Person ID	10	0..1
`PersonUUID`		Person UUID	36	0..1
`Log` Cardinality: 1	`BusinessDocumentProcessingResultCode`	Not in use	2	0..1

Field or Node			Description	Maximum Field Length	Cardinality
	`MaximumLogItemSeverityCode`		If several messages are stored for a business user, the maximum of all received severity codes the most severe level will be shown.	1	0..1
	`Item` Cardinality: 0..unbounded	`TypeID`	Message number	40	0..1
		`CategoryCode`	Not in use	15	0..1
		`SeverityCode`	Severity code definition: • 1 - Information • 2 - Warning • 3 - Error	1	0..1
		`Note`	Contains the message texts.	200	1
		`WebURI`	Not in use		0..1

Error Codes

Error Code	Description
104	Combination of Ext. ID &1 and ID &2 inconsistent. Processing cancelled. `PersonExternalID` and `PersonID` have a 1:1 relationship. Enter the`PersonID` that corresponds with the `PersonExternalID`.
105	Combination of Ext. ID &1 and UUID &2 inconsistent. `Person ExternalID` and `PersonUUID` have a 1:1 relationship. Enter the`PersonUUID` that corresponds with the `PersonExternalID`

Constraints

This service does not support:

- Freelancer (BBP010) business users

Additional Information

> Note
>
> For more details about Communication Management, see Communication Management [page 2992].

4.2.13.2.1.2 Inbound Service: Business User - Read

Technical name: `QUERYBUSINESSUSERIN`

This synchronous inbound SOAP service enables you to read users from your external data source such as an identity management system in SAP S/4HANA Cloud.

Service Request

The service is structured into the following two top-level nodes:

Business User (`BusinessUser`)

The service node contains the search parameters.

Nodes and Fields for the BusinessUser Node

Field or Node		Description	Maximum Field Length	Cardinality
`PersonExternalIDInterval` Cardinality: 0..unbounded	`IntervalBoundaryTypeCode`	The following values exist: • 1- Equal No upper boundary value must be set. • 3 - Between Upper boundary value is mandatory. • 6 - Lower than Upper boundary value is optional. • 7 - Lower equal Upper boundary value is optional. • 8 - Greater than Upper boundary value is optional. • 9 - Greater equal Upper boundary value is optional. This field is mandatory if `LowerBoundaryPersonExtID` is set.	1	1
	`LowerBoundaryPersonExtID`	Employee name	60	0..1
	`UpperBoundaryPersonExtID`		60	0..1

Field or Node		Description	Maximum Field Length	Cardinality
`PersonIDInterval` Cardinality: 0..unbounded	`IntervalBoundaryTypeCode`	The following values exist: • 1- Equal No upper boundary value must be set. • 3 - Between Upper boundary value is mandatory. • 6 - Lower than Upper boundary value is optional. • 7 - Lower equal Upper boundary value is optional. • 8 - Greater than Upper boundary value is optional. • 9 - Greater equal Upper boundary value is optional. This field is mandatory if `LowerBoundaryPersonID` is set.	1	1
	`LowerBoundaryPersonID`		10	0..1
	`UpperBoundaryPersonID`		10	0..1

Field or Node		Description	Maximum Field Length	Cardinality
`BusinessPartnerRoleCodeInterval` Cardinality: 0..unbounded	`IntervalBoundaryTypeCode`	The following values exist: • 1- Equal No upper boundary value must be set. • 3 - Between Upper boundary value is mandatory. • 6 - Lower than Upper boundary value is optional. • 7 - Lower equal Upper boundary value is optional. • 8 - Greater than Upper boundary value is optional. • 9 - Greater equal Upper boundary value is optional. This field is mandatory if `LowerBoundaryBusinessPartnerRoleCode` is set.	1	1
	`LowerBoundaryBusinessPartnerRoleCode`	Only business partner role code BUP003 (Employee) is supported.	6	0..1
`MarkedForArchivingIndicator` Cardinality: 0..unbounded	`IntervalBoundaryTypeCode`	The following values exist: • True • False		1
	`LowerBoundaryMarkedForArchivingIndicator`		1	0..1

Field or Node		Description	Maximum Field Length	Cardinality
`UserIDInterval` Cardinality: 0..unbounded	`IntervalBoundaryTypeCode`	The following values exist: • 1- Equal No upper boundary value must be set. • 3 - Between Upper boundary value is mandatory. • 6 - Lower than Upper boundary value is optional. • 7 - Lower equal Upper boundary value is optional. • 8 - Greater than Upper boundary value is optional. • 9 - Greater equal Upper boundary value is optional. This field is mandatory if `LowerBoundaryUserID` is set.	1	1
	`LowerBoundaryUserID`		12	0..1
	`UpperBoundaryUserID`		12	0..1

Field or Node		Description	Maximum Field Length	Cardinality
`UserNameInterval` Cardinality: 0..unbounded	`IntervalBoundaryTypeCode`	The following values exist: • 1- Equal No upper boundary value must be set. • 3 - Between Upper boundary value is mandatory. • 6 - Lower than Upper boundary value is optional. • 7 - Lower equal Upper boundary value is optional. • 8 - Greater than Upper boundary value is optional. • 9 - Greater equal Upper boundary value is optional. This field is mandatory if `LowerBoundaryUserName` is set.	1	1
	`LowerBoundaryUserName`		40	0..1
	`UpperBoundaryUserName`		40	0..1

Field or Node		Description	Maximum Field Length	Cardinality
`FirstNameInterval` Cardinality: 0..unbounded	`IntervalBoundaryTypeCode`	The following values exist: • 1- Equal No upper boundary value must be set. • 3 - Between Upper boundary value is mandatory. • 6 - Lower than Upper boundary value is optional. • 7 - Lower equal Upper boundary value is optional. • 8 - Greater than Upper boundary value is optional. • 9 - Greater equal Upper boundary value is optional. This field is mandatory if `LowerBoundaryFirstName` is set.	1	1
	`LowerBoundaryFirstName`		35	0..1
	`UpperBoundaryFirstName`		35	0..1

Field or Node		Description	Maximum Field Length	Cardinality
`LastNameInterval` Cardinality: 0..unbounded	`IntervalBoundaryTypeCode`	The following values exist: • 1- Equal No upper boundary value must be set. • 3 - Between Upper boundary value is mandatory. • 6 - Lower than Upper boundary value is optional. • 7 - Lower equal Upper boundary value is optional. • 8 - Greater than Upper boundary value is optional. • 9 - Greater equal Upper boundary value is optional. This field is mandatory if `LowerBoundaryLastName` is set.	1	1
	`LowerBoundaryLastName`		40	0..1
	`UpperBoundaryLastName`		40	0..1
`EmailAddressInterval` Cardinality: 0..unbounded	`IntervalBoundaryTypeCode`		1	1
	`LowerBoundaryEmailAddress`		241	0..1
	`UpperBoundaryEmailAddress`		241	0..1

Query Processing Conditions (`QueryProcessingConditions`)

The service nodes contain the service's business data.

Fields for the QueryProcessingConditions Node

Field	Description	Maximum Field Length	Cardinality
`QueryHitsTotalNumberIndicator`	The following values exist: • True • False (default)		1
`QueryHitsMaximumNumberValue`	Enter the maximum number of hits. If no value is entered, the default is automatically set to 1000.	999999999	0..1
`QueryHitsUnlimitedIndicator`	The following values exist: • True • False (default) Set **True** to get all data based on selection criteria.		1
`QueryLastReturnedObjectID`	The following values exist: • True • False (default) If `QueryHitsMaximumNumberValue` is set and more data is available, you can set this value to **True**.		0..1

Sample Payload

Sample Code

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:aba="http://sap.com/xi/ABA">
   <soapenv:Header/>
   <soapenv:Body>
      <aba:BusinessUserSimpleByElementsQuery_sync>
         <BusinessUser>
            <PersonIDInterval>
               <IntervalBoundaryTypeCode>1</IntervalBoundaryTypeCode>
               <!--Optional:-->
               <LowerBoundaryPersonID>9980035943</LowerBoundaryPersonID>
               <!--Optional:-->
            </PersonIDInterval>
            <BusinessPartnerRoleCodeInterval>
               <IntervalBoundaryTypeCode>1</IntervalBoundaryTypeCode>
               <!--Optional:-->
               <LowerBoundaryBusinessPartnerRoleCode>bup003</
LowerBoundaryBusinessPartnerRoleCode>
            </BusinessPartnerRoleCodeInterval>
         </BusinessUser>
         <QueryProcessingConditions>
            <!--Optional:-->
            <QueryHitsMaximumNumberValue>1</QueryHitsMaximumNumberValue>
            <QueryHitsUnlimitedIndicator>false</QueryHitsUnlimitedIndicator>
         </QueryProcessingConditions>
      </aba:BusinessUserSimpleByElementsQuery_sync>
```

```
    </soapenv:Body>
</soapenv:Envelope>
```

Service Response

Business User (`BusinessUser`)

> **Note**
>
> The fields below the node `User` will be filled.

Node or Field		Description	Maximum Field Length	Cardinality
`PersonExternalID`		Person External ID	60	0..1
`PersonID`		Person ID	10	1
`PersonUUID`		Person UUID	36	1
`BusinessPartnerRoleCode`		Business Partner Role Code	6	1
`MarkedForArchivingIndicator`		• True • False		1
`ValidityPeriod` Cardinality: 0..1	`StartDate`	Format: YYYY-MM-DD		1
	`EndDate`	Format: YYYY-MM-DD		1
`PersonalInformation` Cardinality: 0..1	`FormOfAddress`	Form of address	4	0..1
	`FirstName`	First name	40	0..1
	`LastName`	Last name	40	0..1
	`PersonFullName`	Person full name	80	0..1
	`AcademicTitle`	Academic title	4	0..1
	`CorrespondenceLanguage`	Correspondence language	9	0..1
	`MiddleName`	Middle name	40	0..1

Node or Field		Description	Maximum Field Length	Cardinality
	`AdditionalLastName`	Additional last name	40	0..1
	`BirthName`	Birth name	40	0..1
	`NickName`	Nick name	40	0..1
	`Initials`	Initials	10	0..1
	`AcademicSecondTitle`	Academic second title	4	0..1
	`LastNamePrefix`	Last name prefix	4	0..1
	`LastNameSecondPrefix`	Last name second prefix	4	0..1
	`NameSupplement`	Name supplement	4	0..1
`User` **(only for Cloud)** Cardinality: 0..1	`UserID`	User ID	12	1
	`UserName`	User name/Alias	40	1
	`LogonLanguageCode`	Logon language	9	0..1

Node or Field	Description	Maximum Field Length	Cardinality
`DateFormatCode`	The following values exist: • 1 - DD.MM.YYYY (Gregorian Date) • 2 - MM/DD/YYYY (Gregorian Date) • 3 - MM-DD-YYYY (Gregorian Date) • 4 - YYYY.MM.DD (Gregorian Date) • 5 - YYYY/MM/DD (Gregorian Date) • 6 - YYYY-MM-DD (Gregorian Date, ISO 8601) • 7 - GYY.MM.DD (Japanese Date) • 8 - GYY/MM/DD (Japanese Date) • 9 - GYY-MM-DD (Japanese Date) • A - YYYY/MM/DD (Islamic Date 1) • B - YYYY/MM/DD (Islamic Date 2)	2	0..1

Node or Field		Description	Maximum Field Length	Cardinality
		• C - YYYY/MM/DD (Iranian Date)		
	`DecimalFormatCode`	The following values exist: • 1.234.567,89 • X - 1,234,567.89 • Y - 1 234 567,89	2	0..1
	`TimeZoneCode`	Time zone	10	0..1
	`TimeFormatCode`	The following values exist: • 0 - 24 Hour Format (Example: 12:05:10) • 1 - 12 Hour Format (Example: 12:05:10 PM) • 2 - 12 Hour Format (Example: 12:05:10 pm) • 3 - Hours from 0 to 11 (Example: 00:05:10 PM) • 4 - Hours from 0 to 11 (Example: 00:05:10 pm)	2	0..1
	`LockedIndicator`	Locked indicator	5	0..1

Node or Field			Description	Maximum Field Length	Cardinality
	`ValidityPeriod` Cardinality: 1	`StartDate`	Format: YYYY-MM-DD If no start date is maintained for the `User`, the `StartDate` for the `BusinessUser` is entered.		1
		`EndDate`	Format: YYYY-MM-DD If no `EndDate` is maintained, it is set to 9999-12-31.		1
	`Role` Cardinality: 0..unbounded	`RoleName`	Role name	40	1
`UserAssignment` **(only for on-premise)** Cardinality: 0..1	`UserID`		User ID	12	1
	`UserName`			40	0..1
	`UserAssignmentStatusCode`		The following values exist: • 1 - is reserved • 2 - is assigned	1	1
`WorkplaceInformation` Cardinality: 0..1	`EmailAddress`		Email address	241	0..1
	`PhoneInformation` Cardinality: 0..2 One set of phone information per phone type supported.	`PhoneType`	Phone type	1	1
		`CountryDialingCode`	Country dialing code	10	0..1
		`PhoneNumberAreaID`	Phone number area code	10	0..1
		`PhoneNumberSubscriberID`	Phone number subscriber ID	30	0..1
		`PhoneNumberExtension`	Phone number extension	10	0..1

Node or Field		Description	Maximum Field Length	Cardinality
	`FunctionalTitleName`	Functional title name	40	0..1
	`Department`	Department name	40	0..1
	`RoomNumber`	Room number	10	0..1
	`Building`	Building name	10	0..1

Response Processing Conditions (`ResponseProcessingConditions`)

Field	Description	Maximum Field Length	Cardinality
`HitsTotalNumberValue`	Contains the number of users based on given criteria.	999999999	1
`ReturnedQueryHitsNumberValue`	Contains the number of found data sets for business users.	999999999	1
`MoreHitsAvailableIndiactor`	The indicator is set if the query was limited to a number of hits, but more business user data sets are available based on the query.		1
`LastReturnedObjectID`	Displays the last row of the results list limited by the found hits or by the value given for `QueryHitsMaximumNumberValue`.	999999999	0..1

Log (`Log`)

If errors occur, the log contains the information shown in the table below:

Field or Node	Description	Maximum Field Length	Cardinality
`BusinessDocumentProcessingResultCode`		2	0..1
`MaximumLogItemSeverityCode`	If several messages are stored for a business user, the most severe level is shown from the maximum of all received severity codes.	1	0..1

Field or Node		Description	Maximum Field Length	Cardinality
`Item`	TypeID	Message number	40	0..1
Cardinality: 0..unbounded	CateoryCode	Not in use	15	0..1
	SeverityCode	Severity code definition: • 1 - Information • 2 - Warning • 3 - Error	1	0..1
	Note	Contains the message texts.	200	1
	WebURI	Not in use		0..1

Constraints

This service does not support:

- Freelancer (BBP010) business users

Additional Information

> ⓘ Note
>
> For more details about Communication Management, see Communication Management [page 2992].

4.2.13.2.1.3 Business User - Read Metadata

Technical name: `QueryBusinessUserMetadataIn`

This service enables you to read metadata information from your external data source such as an identity management system for service Business User with this synchronous inbound service.

This service provides the search parameter `RoleCategory`, which restricts the result set of the metadata information of the service. The response includes the metadata information based on the given criteria. If errors occur, the log contains information about the severity code, the message number and message texts.

This service is available on the SAP Business Accelerator Hub, for more information see APIs on SAP Business Accelerator Hub.

Service Request

Service Nodes

The service nodes contain the service's business data.

Node or Field		Description	Cardinality
`BusinessPartnerRoleCategoryInterval` Cardinality: 0..unbounded	`IntervalBoundaryTypeCode`	You can only use the following value: • 1 - Equal	1..1
	`LowerBoundaryBusinessPartnerRoleCategoryCode`	For example: `BUP003`	0..1

Sample Payload

Sample Code

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:aba="http://sap.com/xi/ABA">
   <soapenv:Header/>
   <soapenv:Body>
      <aba:BusinessUserMetaDataQuery_sync>
         <!--Zero or more repetitions:-->
         <BusinessPartnerRoleCategoryInterval>
            <IntervalBoundaryTypeCode>1</IntervalBoundaryTypeCode>
            <LowerBoundaryBusinessPartnerRoleCategoryCode>BUP003</
LowerBoundaryBusinessPartnerRoleCategoryCode>
         </BusinessPartnerRoleCategoryInterval>
      </aba:BusinessUserMetaDataQuery_sync>
   </soapenv:Body>
</soapenv:Envelope>
```

Service Response

Service Node		Description	Link to Details
`BusinessUserMetaData`	`RoleCategoryDependentMetaData`	This node contains all metadata, which depends on the role category of a business user, such as the role category with its role, external ID category with its external ID type or relationship category.	RoleCategoryDependentMetaData [page 1105]

Service Node		Description	Link to Details
	`CodeList`	This node provides the available code lists for SAP specific codes. For example country/region code, academic title.	CodeList [page 1107]
`Log`		This nodes displays occurred messages.	Log [page 1108]

Error Codes

Error Code	Message	Description
112	Interval Boundary Type Code &1 is not supported for BusinessPartnerRoleCategoryInterval.	You can only use the following value: • 1 - Equal
118	No data found by given search criteria.	To display all data, don't enter any value.

Authentication Method

You can use the following authentication methods: User ID/password (Username Token), X.509 certificate (X509 Token) or Single Sign On using SAML (SAML Token).

Constraints

Currently this SOAP service is only enabled for English.

Additional Information

If you have any issues, report an incident for component `CA-GTF-BUM`.

> ### Note
>
> For more information about the API, choose the *Details* tab on the SAP Business Accelerator Hub.

4.2.13.2.1.3.1 RoleCategoryDependentMetaData

Nodes and Fields

> **Note**
>
> In the following table, field attributes are marked in blue.

RoleCategoryDependentMetaData

Node or Field				Description	Cardinality
`BusinessPartnerRoleCategory` Cardinality: 1..1	`BusinessPartnerRoleCategoryCode`			For example: `BUP003`	1..1
	`Description`	`languageCode`		For example: Employee	0..unbounded
	`BusinessPartnerRole` Cardinality: 1..1	`BusinessPartnerRoleCode`		For example: `BBP005`	1..1
		`Description`	`languageCode`		0..unbounded
		`DefaultIndicator`		Can be *true* or *false*	
`BusinessPartnerExternalIDCategory` Cardinality: 0..1	`BusinessPartnerExternalIDCategoryCode`			For example: `HCM030`	1..1
	`Description`	`languageCode`			0..unbounded
	`BusinessPartnerExternalID` Cardinality: 0..unbounded	`BusinessPartnerExternalIDCode`		For example: `HCM030`	1..1
		`Description`	`languageCode`		0..unbounded
		`DefaultIndicator`		The value can be either*true* or *false*	
`BusinessPartnerRelationshipCategory` Cardinality: 0..1	`BusinessPartnerRelationshipCategoryCode`			For example: `BUR025`	1..1
	`Description`	`languageCode`			0..unbounded
	`Partner1_BusinessPartnerCategory` Cardinality: 0..unbounded	`BusinessPartnerCategoryCode`		Can be: • 2 - Organi-zation • 3 - Group	1..1
		`Description`	`languageCode`		0..unbounded
	`Partner2_BusinessPartnerCategory`	`BusinessPartnerCategoryCode`		Can be: • 1 - Person	1..1

Node or Field				Description	Cardinality
	Cardinality: 0..unbounded	`Description`	`languageCode`		0..unbounded
`NodeProperties` Cardinality: 0..unbounded	`NodePath`			For example: `BUSINESS_USER`	1..1
	`NodeProperty` Cardinality: 0..unbounded	`NodePropertyCode`		Node property codes are: • 01 - enabled • 02 - disabled • 03 - read only	1..1
		`Description`	`languageCode`		0..unbounded
`FieldProperties` 0..unbounded	`NodePath`			For example: • `BUSINESS_USER-USER-ROLE` • `BUSINESS_USER-USER-VALIDITY_PERIOD` • `BUSINESS_USER-USER_ASSIGNMENT`	1..1
	`Fieldname`			Name of a field.	1..1

Node or Field			Description	Cardinality
	`FieldProperty` Cardinality: 0..unbounded	`FieldPropertyCode`	Node property codes are: • 01 - enabled • 02 - disabled • 03 - read only • 04 - mandatory • 05 - enabled, read only for update • 06 - mandatory, read only for update	1..1
		`Description` `languageCode`		0..unbounded

> **Note**
>
> To display more information about the service nodes on the SAP Business Accelerator Hub, select the *API Reference* tab and choose the *View Details* button in the table.

4.2.13.2.1.3.2 CodeList

Nodes and Fields

> **Note**
>
> In the following table, field attributes are marked in blue.

CodeList

Node or Field		Description	Cardinality
`FieldName`		Name of the field.	1..1
`FieldCodeList` Cardinality: 0..unbounded	`FieldValue`	This node provides the available code lists for SAP specific codes. For example coun-	1..1

Node or Field			Description	Cardinality
	FieldDescription	languageCode	try/region code, academic title.	0..unbounded

> **Note**
>
> To display more information about the service node on the SAP Business Accelerator Hub, select the operation.

4.2.13.2.1.3.3 Log

Nodes and Fields

> **Note**
>
> In the following table, field attributes are marked in blue.

Log

Node or Field		Description	Cardinality
BusinessDocumentProcessingResultCode			0..1
MaximumLogItemSeverityCode		If several messages are stored for a business user, the most severe level is shown from the maximum of all received severity codes.	0..1
Item Cardinality: 0..unbounded	TypeID	Message number.	0..1
	CategoryCode	Not in use.	0..1
	SeverityCode	Severity code definition: • 1 - Information • 2 - Warning • 3 - Error	0..1
	Note	Contains the message texts.	1..1
	WebURI	Not in use.	0..1

> **Note**
>
> To display more information about the service node on the SAP Business Accelerator Hub, select the operation.

4.2.13.2.2 Integrating Business Role Change Documents

You can use this scenario to read the change documents of business roles.

Context

This scenario allows you to read the change documents of business roles. We recommend that you limit the number of change documents according to the *Changed On* and the *Business Role ID* fields to avoid a high volume of data.

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Business Role Change Document Integration* scenario (SAP_COM_0366) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

Related Information

How to Create a Communication Arrangement [page 2998]
How to Create Communication Users [page 2995]

4.2.13.2.2.1 Business Role Changes - Read

Technical name: `APS_IAM_API_BROLE_CDOC_0001_IWSG`

This service enables you to to read the change documents of business roles.

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the **Related Information** section.

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the service's business data.

Entity	Description	Necessity
`BusinessRoleID`	Business Role ID	Optional
`ChangedOn`	Change on Date/Time	Optional
`ChangeCategory`	Category of change The following values are available: • *GE* = General changes to the business role attributes, such as the description • *BU* = Assigned business users • *BC* = Assigned business catalogs • *FS* = Assigned launchpad spaces • *RA* = Access restrictions • *RV* = Restriction values • *RS* = Restrictions	Optional
`ChangeCategoryText`	Category of change description (such as *Business catalog added*)	Optional
`Attribute`	Name of changed attribute	Optional
`ValueChangedFrom`	Old value	Optional
`ValueChangedTo`	New value	Optional
`ChangedByUserName`	Changed by user identified by user name	Optional
`ChangedByUserID`	Changed by user identified by user ID	Optional
`ChangedByUserEmailAddress`	Changed by user identified by email address	Optional
`ChangedByGlobalUserID`	Changed by user identified by *Global User ID*	Optional
`Action`	Action	Optional

Service Response

The details of an API response are according to the operation types that are supported. For more information, see Operation for Business Role Changes - Read [page 1111].

Entity	Description
200	OK
400	Bad Request
403	Forbidden

Additional Information

For more information about Communication Management, see the **Related Information** section.

Related Information

APIs on SAP Business Accelerator Hub
Communication Management [page 2992]

4.2.13.2.2.1.1 Operation for Business Role Changes - Read

The *Business Role Changes - Read* inbound service offers this operation:

Operation	HTTP Method	Sample URL
BusinessRoleChangeDocument	GET	GET<host>/sap/opu/odata/sap/APS_IAM_API_BROLE_CDOC/BusinessRoleChanges

4.2.13.2.3 Integrating Business User Change Documents

You can use this scenario to read the change documents of business users.

Context

This scenario allows you to read the change documents of business users. We recommend that you limit the number of change documents according to the *Changed On* and the *Business User ID* fields to avoid a high volume of data.

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.

   For more information, see the *Related information* section.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Business User Change Document Integration* scenario (SAP_COM_0327) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

Related Information

How to Create a Communication Arrangement [page 2998]
How to Create Communication Users [page 2995]

4.2.13.2.3.1 Business User Changes - Read

Technical name: `APS_IAM_API_BUSER_CDOC_0001_IWSG`

This service enables you to read the change documents of business users.

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the *Related Information* section.

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the service's business data.

Entity	Description	Necessity
`BusinessUserID`	Business User ID	Optional
`ChangedOn`	Change on Date/Time	Optional
`ChangeCategory`	Category of change The following values are available • *GE* = General changes to the business user attributes, such as the description • *BR* = Business role assignments	Optional
`ChangeCategoryText`	Category of change description (such as *Business catalog added*)	Optional
`Attribute`	Name of changed attribute	Optional
`ValueChangedFrom`	Old value	Optional
`ValueChangedTo`	New value	Optional
`ChangedByUserName`	Changed by user identified by user name	Optional
`ChangedByUserID`	Changed by user identified by user ID	Optional
`ChangedByUserEmailAddress`	Changed by user identified by email address	Optional
`ChangedByGlobalUserID`	Changed by user identified by *Global User ID*	Optional
`Action`	Action	Optional

Service Response

The details of an API response are according to the operation types that are supported. For more information, see Operation for Business User Changes - Read [page 1114].

Entity	Description
200	OK

Entity	Description
400	Bad Request
403	Forbidden

Related Information

APIs on SAP Business Accelerator Hub
Communication Management [page 2992]

4.2.13.2.3.1.1 Operation for Business User Changes - Read

The *Business User Changes - Read* inbound service offers the following operation:

Operation	HTTP Method	Sample URL
BusinessUserChangeDocument	GET	GET<host>/sap/opu/odata/sap/APS_IAM_API_BUSER_CDOC/BusinessUserChanges

4.2.13.2.4 Integrating Business Role Data (Read)

You can use this communication scenario to read business role data.

Context

The communication scenario SAP_COM_0A04 allows you to read business role data, such as business role ID or description, read business roles, read assigned business users, launchpad spaces and business catalogs, or filter for specific combinations.

Authentication Methods

This communication scenario can be configured with the following inbound authentication methods:

- *Basic Authentication* (user and password)
- *X.509* (certificates)

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Business Role Read Integration* scenario (SAP_COM_0A04) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

Related Information

Communication Management [page 2992]

4.2.13.2.4.1 Business Role - Read Integration

This service enables you to to do the following:

- Read business roles
- Read assigned business users, launchpad spaces and business catalogs
- Filter for specific combinations

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the *Related Information* section.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

Service Group (incl. Namespace if It Exists)	Repository ID	Service Name (incl. Namespace if It Exists)	Version
`aps_iam_api_brole_read`	`srvd_a2x`	`BusinessRole`	`0001`

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Necessity	Link to Details
BusinessRoles	Contains business roles.	Mandatory	Please refer to the *Related Information* section.

Additional Information

> ⓘ Note
>
> For more information about Communication Management, see the *Related Information* section.

Related Information

APIs on SAP Business Accelerator Hub
Communication Management [page 2992]

4.2.13.2.4.1.1 Business Role Entity

Technical name: `BusinessRoles`

Properties

Property	Description	Necessity
BusinessRoleID	Business role ID	Mandatory (Key)
Description	Description	Optional

Property	Description	Necessity
LongText	Long Text	Optional
BusinessRoleTemplateID	Business role template ID	Optional
IsLeading	Leading business role	Optional
LeadingBusinessRoleID	Leading business role ID	Optional
IsDerived	Derived business role	Optional
BusinessRoleGroup	Business role group	Optional Not available in SAP IBP
ExposedToSAPBTP	Exposed to SAP BTP	Optional
PriceCategoryCode	Price category code	Optional
PriceCategory	Price category	Optional
WriteAccessRestrictionCode	• *1: Unresticted* • *2: Restricted* • *3: No access*	Optional
ReadAccessRestrictionCode	• *1: Unresticted* • *2: Restricted* • *3: No access*	Optional
F4AccessRestrictionCode	• *1: Unresticted* • *2: Restricted* • *3: No access*	Optional
Imported	Imported	Optional
Exported	Exported	Optional
LastChangedDateTime	*Changed On:* <date>, <time>	Optional
LastChangedByUserID	*Changed By:* <user ID>	Optional
LastChangedByUserName	*Changed By:* <user name>	Optional
LastChangedByUserDescription	*Changed By:* <user description>	Optional
LastChangedByEMailAddress	*Changed By:* <email address>	Optional
LastChangedByGlobalUserID	*Changed By:* <global user ID>	

To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

Expandables

This entity is expandable with the following:

- Business Users
- Launchpad Spaces

- Business Catalogs
- Derived Business Roles
- Restrictions
  - RestrictionFields
    - RestrictionValues

Related Information

Business Role – Business User Entity [page 1118]
Business Role – Launchpad Space Entity [page 1118]
Business Role – Business Catalog Entity [page 1119]
Business Role – Derived Business Role Entity [page 1119]
Business Role - Restriction Entity [page 1120]
Business Role – Restriction Field Entity [page 1120]
Business Role – Restriction Value Entity [page 1121]

4.2.13.2.4.1.1.1 Business Role – Business User Entity

Technical name: `BusinessRoleBusinessUsers`

Properties

Property	Description	Necessity
BusinessRoleID	Business role ID	Mandatory (Key)
UserID	Business user ID	Mandatory (Key)

To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.4.1.1.2 Business Role – Launchpad Space Entity

Technical name: `BusinessRoleLaunchpadSpaces`

Properties

Property	Description	Necessity
BusinessRoleID	Business role ID	Mandatory (Key)
SpaceID	Space ID	Mandatory (Key)

To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.4.1.1.3 Business Role – Business Catalog Entity

Technical name: `BusinessRoleBusinessCatalogs`

Properties

Property	Description	Necessity
BusinessRoleID	Business role ID	Mandatory (Key)
BusinessCatalogID	Business catalog ID	Mandatory (Key)

To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.4.1.1.4 Business Role – Derived Business Role Entity

Technical name: `BusinessRoleDerivedRoles`

Properties

Property	Description	Necessity
BusinessRoleID	Business role ID	Mandatory (Key)
DerivedBusinessRoleID	Derived business role ID	Mandatory (Key)

> ### Note
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.4.1.1.5 Business Role - Restriction Entity

Technical name: `BusinessRoleRestrictions`

Properties

Property	Description	Necessity
BusinessRoleID	Business role ID	Mandatory (Key)
RestrictionTypeAssignmentUUID	Restriction type assignment UUID	Mandatory (Key)
AccessCategory	*W: Write* *R: Read* *F: Value Help*	Optional
RestrictionType	Restriction type	Optional

To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.4.1.1.6 Business Role – Restriction Field Entity

Technical name: `BusinessRoleRestrictionFields`

Properties

Property	Description	Necessity
BusinessRoleID	Business role ID	Mandatory (Key)
RestrictionTypeAssignmentUUID	Restriction type assignment UUID	Mandatory (Key)

Property	Description	Necessity
FieldName	Field name	Mandatory (Key)
MaintainStatus	<empty>: No maintenance status set. not_maint: Maintenance status set to *Not Maintained*. maint_derived: Maintenance Status set to *Maintained in Derived Role*.	Optional
IsLeading	Leading	Optional

To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.4.1.1.7 Business Role – Restriction Value Entity

Technical name: `BusinessRoleRestrictionValues`

Properties

Property	Description	Necessity
BusinessRoleID	Business role ID	Mandatory (Key)
RestrictionTypeAssignmentUUID	Restriction type assignment UUID	Mandatory (Key)
FieldName	Field name	Mandatory (Key)
RestrictionValueAssignmentUUID	Restriction value assignment UUID	Mandatory (Key)
LowValue	Low value	Optional
HighValue	High value	Optional

To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.5 Integrating Business User Logon Details (Read)

You can use this communication scenario to read business user logon details.

Context

The communication scenario SAP_COM_0889 allows you to read business user logon details, such as user name, validity, assigned business roles, read business users, read assigned business catalogs, business roles and application jobs, or filter for specific combinations.

Authentication Methods

This communication scenario can be configured with the following inbound authentication methods:

- *Basic Authentication* (user and password)
- *X.509* (certificates)

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Business User Logon Details Read Integration* scenario (SAP_COM_0889) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

Related Information

Communication Management [page 2992]

4.2.13.2.5.1 Business User Logon Details - Read Integration

This service enables you to

- Read business users

- Read assigned business catalogs, business roles and application jobs
- Filter for specific combinations

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the *Related Information* section.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

Service Group (incl. Namespace if It Exists)	Repository ID	Service Name (incl. Namespace if It Exists)	Version
`aps_iam_api_buser_read`	`srvd_a2x`	`BusinessUser`	`0001`

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Necessity	Link to Details
BusinessUsers	Contains Business Users	Mandatory	For more information, see the *Related Information* section.

Additional Information

> **Note**
>
> For more details about Communication Management, see Communication Management [page 2992].

Related Information

Business User Entity [page 1124]

APIs on SAP Business Accelerator Hub

4.2.13.2.5.1.1 Business User Entity

Technical name: `BusinessUsers`

Properties

Property	Description	Necessity
UserID	User ID	Mandatory (Key)
UserName	User name	Optional
Description	Description	Optional
FirstName	First name	Optional
LastName	Last name	Optional
EMailAddress	Email address	Optional
GlobalUserID	Global user ID	Optional
BusinessUserGroup	Business user group	Optional Not available in SAP IBP
PriceCategoryCode	Price category code	Optional
PriceCategory	Price category	Optional
LockStatus	Lock status	Optional
Validity	Validity	Optional
IsLoggedOn	Is logged on	Optional
NeverLoggedOn	Never logged on	Optional
ValidFromDate	*Valid From* date	Optional
ValidToDate	*Valid To* date	Optional
LastLogonDateTime	Date and time of last logon	Optional
LastChangedDateTime	Date and time of last change	Optional
LastChangedByUserID	*Last Changed By* <user ID>	Optional
LastChangedByUserName	*Last Changed By* <user name>	Optional
LastChangedByUserDescription	*Last Changed By* <user description>	Optional
LastChangedByEMailAddress	*Last Changed By* <email address>	Optional
LastChangedByGlobalUserID	*Last Changed By* <global user ID>	Optional

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

Expandables

This entity is expandable with the following:

- Business Catalogs
- Business Roles
- Application Jobs

Related Information

Business User – Business Catalog Entity [page 1125]
Business User – Business Role Entity [page 1126]
Business User – Application Job Entity [page 1126]

4.2.13.2.5.1.1.1 Business User – Business Catalog Entity

Technical name: `BusinessUserBusinessCatalogs`

Properties

Property	Description	Necessity
UserID	User ID	Mandatory (Key)
BusinessCatalogID	Business catalog ID	Mandatory (Key)

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.5.1.1.2 Business User – Business Role Entity

Technical name: `BusinessUserBusinessRoles`

Properties

Property	Description	Necessity
UserID	User ID	Mandatory (Key)
BusinessRoleID	Business role ID	Mandatory (Key)

> ⓘ Note
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.5.1.1.3 Business User – Application Job Entity

Technical name: `BusinessUserApplicationJobs`

Properties

Property	Description	Necessity
UserID	User ID	Mandatory (Key)
ApplicationJob	Application job	Mandatory (Key)

> ⓘ Note
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.6 Integrating Support User Data (Read)

You can use this communication scenario to read support user data.

Context

The communication scenario SAP_COM_0347 allows you to read support user request logs.

Authentication Methods

This communication scenario can be configured with the following inbound authentication methods:

- *Basic Authentication* (user and password)
- *X.509* (certificates)

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Support User Request Log Integration* scenario (SAP_COM_0347) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

4.2.13.2.6.1 Support User - Read Integration

This service enables you to to do the following:

- Read support users
- Read associated user information

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the *Related Information* section.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

Service Group (incl. Namespace if It Exists)	Repository ID	Service Name (incl. Namespace if It Exists)	Version
`api_iam_susr_read`	`srvd_a2x`	`SupportUser`	`0001`

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Necessity	Link to Details
SupportUsers	Contains support users.	Mandatory	Please refer to the *Related Information* section.

Related Information

APIs on SAP Business Accelerator Hub
Communication Management [page 2992]

4.2.13.2.6.1.1 Support User Entity

Technical name: `SupportUsers`

Properties

Property	Description	Necessity
CAMSupportUserRequestUUID	Support user request UUID for SAP Cloud Access Manager.	Mandatory (Key)

Property	Description	Necessity
CAMSuppUserValdtyStartDateTime	Support user validity start datetime for SAP Cloud Access Manager.	Optional
CAMSupportUser	Support user for SAP Cloud Access Manager.	Optional
CAMSupportAccessLevel	Support access level for SAP Cloud Access Manager.	Optional
CAMSupportAccessCategory	Support access category for SAP Cloud Access Manager.	Optional
CAMSupportAccessCategoryText	Support access category text for SAP Cloud Access Manager.	Optional
CAMSuppAddedAuthorizationMode	Support added authorization mode for SAP Cloud Access Manager.	Optional
CAMSuppAddedAuthznModeText	Support added authorization mode text for SAP Cloud Access Manager.	Optional
CAMSuppAddedAuthorizationUser	Support added authorization user for SAP Cloud Access Manager.	Optional
CAMSupportIncident	Support incident for SAP Cloud Access Manager.	Optional
CAMSupportIncidentSystem	Support incident system for SAP Cloud Access Manager.	Optional
CAMSupportIncidentSystemText	Support incident system text for SAP Cloud Access Manager.	Optional
CAMSupportIncidentSystemNumber	Support incident system number for SAP Cloud Access Manager.	Optional
CAMSupportIncidentTicketNumber	Support incident ticket number for SAP Cloud Access Manager.	Optional
CAMSupportIncidentTicketYear	Support incident ticket year for SAP Cloud Access Manager.	Optional
CAMSuppUserValdtyEndDateTime	Support user validity end date and time for SAP Cloud Access Manager.	Optional

To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

Expandables

This entity is expandable with the following:

- SupportUser

Related Information

Support User - User Entity [page 1130]

4.2.13.2.6.1.1.1 Support User - User Entity

Technical name: SupportUserUsers

Properties

Property	Description	Necessity
UserID	Contains the user ID.	Mandatory (Key)
UserDescription	Contains the user description.	Optional
IsTechnicalUser	Technical user.	Optional
AddressPersonID	Contains the person ID.	Optional
AddressID	Contains the address ID.	Optional

4.2.13.2.7 Integrating Communication Arrangement Data (Read)

You can use this communication scenario to read communication arrangement data.

Context

This communication scenario allows you to read communication arrangements, read assigned inbound users, outbound users, inbound services and outbound services.

Authentication Methods

This communication scenario can be configured with the following inbound authentication methods:

- *Basic Authentication* (user and password)
- *X.509* (certificates)

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Communication Arrangement Read Integration* scenario (SAP_COM_0A07) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

Related Information

Communication Management [page 2992]

4.2.13.2.7.1 Communication Arrangement - Read Integration

This service enables you to:

- Read communication arrangements
- Read assigned inbound users, outbound users, inbound services and outbound services

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the *Related Information* section.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

Service Group (incl. Namespace if It Exists)	Repository ID	Service Name (incl. Namespace if It Exists)	Version
`aps_com_api_ca_read`	`srvd_a2x`	`CommunicationArrangement`	`0001`

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Necessity	Link to Details
CommunicationArrangements	Contains communication arrangements	Mandatory	For more information, see the *Related Information* section.

Additional Information

> **Note**
>
> For more information about Communication Management, see the *Related Information* section.

Related Information

Communication Arrangements - Entity [page 1132]
APIs on SAP Business Accelerator Hub
Communication Management [page 2992]

4.2.13.2.7.1.1 Communication Arrangements - Entity

Technical name: `CommunicationArrangements`

Properties

Property	Description	Necessity
CommunicationArrangementUUID	Communication arrangement UUID	Mandatory (Key)

Property	Description	Necessity
CommunicationArrangementName	Communicaton arrangement name	Optional
CommunicationSystemID	Communication system ID	Optional
CommunicationSystemName	Communication system name	Optional
CommunicationScenarioID	Communication scenario ID	Optional
CommunicationScenarioName	Communication scenario name	Optional
LastChangedDateTime	Date and time of last change	Optional
LastChangedByUserID	Last changed by <user ID>	Optional
LastChangedByUserName	Last changed by <user name>	Optional
LastChangedByUserDescription	Last changed by <user description>	Optional
LastChangedByEMailAddress	Last changed by <email address>	Optional
LastChangedByGlobalUserID	Last changed by <global user ID>	Optional

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

Expandables

This entity is expandable with the following:

- *Inbound User*
- *Outbound User*
- *Inbound Services*
- *Outbound Services*

Related Information

Communication Arrangement – Inbound User Entity [page 1134]
Communication Arrangement – Outbound User Entity [page 1134]
Communication Arrangement – Inbound Service Entity [page 1135]
Communication Arrangement – Outbound Service Entity [page 1136]

4.2.13.2.7.1.1.1 Communication Arrangement – Inbound User Entity

Technical name: `CommArrangementsInboundUsers`

Properties

Property	Description	Necessity
CommunicationArrangementUUID	Communication arrangement UUID	Mandatory (Key)
UserID	User ID	Mandatory (Key)
UserName	User name	Optional
OAuth2ClientID	OAuth 2.0 client ID	Optional
OAuth2GrantType	OAuth 2.0 grant type	Optional
AuthMethod	**`x509`**: SSL Client Certificate **`basic`**: User ID and Password **`none`**: None **`oauth2`**: OAuth 2.0 **`oauth1`**: OAuth 1.0 princ_prop: Principal Propagation oauth2_mtls: OAuth 2.0 (mTLS)	Optional

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.7.1.1.2 Communication Arrangement – Outbound User Entity

Technical name: `CommArrangementsOutboundUsers`

Properties

Property	Description	Necessity
CommunicationArrangementUUID	Communication arrangement UUID	Mandatory (Key)
UserName	User name	Mandatory (Key)
OAuth2ClientID	OAuth 2.0 client ID	Mandatory (Key)
CertificateID	Certificate ID	Optional
AuthMethod	**`x509`**: SSL Client Certificate **`basic`**: User ID and Password **`none`**: None **`oauth2`**: OAuth 2.0 - **`oauth1`**: OAuth 1.0 **`princ_prop`**: Principal Propagation **`oauth2_mtls`**: OAuth 2.0 (mTLS)	Optional

> ### Note
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.7.1.1.3 Communication Arrangement – Inbound Service Entity

Technical name: `CommArrangementsInboundServ`

Properties

Property	Description	Necessity
CommunicationArrangementUUID	Communication arrangement UUID	Mandatory (Key)
ServiceUUID	Service UUID	Mandatory (Key)
ServiceNumber	Service number	Optional

Property	Description	Necessity
ServiceID	Service ID	Optional
ServiceType	Service type	Optional
IsHidden	Hidden	Optional

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.7.1.1.4 Communication Arrangement – Outbound Service Entity

Technical name: `CommArrangementsOutboundServ`

Properties

Property	Description	Necessity
CommunicationArrangementUUID	Communication arrangement UUID	Mandatory (Key)
ServiceUUID	Service UUID	Mandatory (Key)
ServiceNumber	Service number	Optional
ServiceID	Service ID	Optional
ServiceType	Service type	Optional
Status	Status	Optional
IsActive	Active	Optional
URL	URL	Optional
TCPPort	TCP port	Optional
SOAPWSRMVersion	SOAPWSRM version	Optional
JobTimezoneCode	Job time zone code	Optional

Property	Description	Necessity
JobStartTime	Job start time	Optional
JobPackageSize	Job package size	Optional
JobRecurrenceValue	Job recurrence value	Optional
JobFrequency	Job frequency	Optional

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.8 Integrating Communication Management Data (Read and Write)

You can use this communication scenario to read and write communication arrangement, communication system and communication user data.

Context

This communication scenario allows you to read and write communication arrangements, communication systems and communication users.

Authentication Methods

This communication scenario can be configured with the following inbound authentication methods:

- *Basic Authentication* (user and password)
- *X.509* (certificates)

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.

3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Communication Management Integration* scenario (SAP_COM_0A48) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

Related Information

Communication Management [page 2992]

4.2.13.2.8.1 Communication Arrangement - Read and Write Integration

This service enables you to:

- Read and write communication arrangements
- Read and write assigned inbound users, outbound users, inbound services and outbound services

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the *Related Information* section.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

Service Group (incl. Namespace if It Exists)	Repository ID	Service Name (incl. Namespace if It Exists)	Version
`aps_com_ca_a4c_odata`	`srvd_a2x`	`aps_com_ca_a4c_odata`	`0001`

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Constants	Necessity
CommunicationArrangements	Contains communication arrangement root data		Mandatory
Properties	Contains communication arrangement root property names		Optional
PropertyValues	Contains communication arrangement root property values		Optional
InboundUsers	Contains the reference to the communication system inbound user		Optional
InboundServices	Contains the communication arrangement inbound service data		Optional
InboundServiceProperties	Contains communication arrangement inbound service property names		Optional
InboundServicePropertyValues	Contains communication arrangement inbound service property values		Optional

OutboundUsers	Contains the reference to the communication system outbound user	Contains the following attributes and values:
		• OAuth2SAML2Identifier
		  • user_email
		  • user_uuid
		  • user_name
			Optional

OutboundUserAuthScopes	Contains additional scopes for outbound users		Optional

OutboundServices	Contains the communication arrangement outbound service data	Contains the following attributes and values:
		• Status
		  • active
		  • inactive
			Optional

OutboundServiceProperties	Contains communication arrangement outbound service property names		Optional
OutboundServicePropertyValues	Contains communication arrangement outbound service property values		Optional

Additional Information

> **Note**
>
> For more information about Communication Management, see the *Related Information* section.

Related Information

Communication Arrangements - Entity [page 1132]
APIs on SAP Business Accelerator Hub
Communication Management [page 2992]

4.2.13.2.8.2 Communication System - Read and Write Integration

This service enables you to:

- Read and write communication systems
- Read and write assigned inbound users, outbound users, communication arrangements as well as channels for Event Exchange Infrastructure and OpenID Connect configurations

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the *Related Information* section.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

Service Group (incl. Name-space if It Exists)	Repository ID	Service Name (incl. Name-space if It Exists)	Version
`aps_com_cs_a4c_odata`	`srvd_a2x`	`aps_com_cs_a4c_odata`	`0001`

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Constants	Necessity

CommunicationSystems	Contains communication system data	Contains the following attributes and values:
		• Type
		  • customer_managed
		  • provider_managed
		• CipherSuites
		  • TLS_1_0_to_1_2
		  • TLS_1_0_to_1_3
		  • TLS_1_2_only
		  • TLS_1_2_to_1_3
		Please note that these values can also be empty.
		• IsCipherSuiteDefault
		  • TLS_1_0_to_1_2
		  • TLS_1_0_to_1_3
		  • TLS_1_2_only
		  • TLS_1_2_to_1_3
		Please note that these values can also be empty.
		• OAuth2IdentityProvider-UserLogonType
		  • user_uuid
		  • user_name
		• SAMLBearerAssertion-ProviderUserLogonType
		  • user_uuid
		  • user_name
			Mandatory

Entity	Description	Constants	Necessity

InboundUsers	Contains references to the communication user ID	Contains the following attributes and values:
		• OAuth2GrantType
		  • authorization_code
		  • saml2-bearer
		  • ropc
		  • client_credentials
		• OAuth2RefreshTokenExpiryUnit
		  • days
		  • months
		  • years
		• AuthenticationMethod
		  • x509
		  • basic
		  • none
		  • oauth2
		  • oauth2_mtls
		  • sso2
		  • oauth1
			Optional

Event Channels	Contains Event Exchange Infrastructure channel information		Optional

Entity	Description	Constants	Necessity

OpenIDConnectProviders	Contains OpenID Connect configurations	Contains the following attributes and values:
		• UserMapping
		  • global_user_id
		  • email
		  • user_name
		• UserMappingClaim
		  • sub_claim
		  • email_claim_verified
		  • email_claim_unverified
		  • oid_claim
		  • user_name_claim
		  • custom_claim
		• AudienceEvaluationMode
		  • own_client_id_only
		  • own_and_selected_other_client_ids
		Note
		Please note that an *OpenIDConnet Audiences* entity is required for creating or changing this value.
		  • own_and_all_other_client_ids
		• IsImportWebKeysActive
		Note
		If this value is false, you must specify at least one *OpenIDConnectWebKey* entity. If this value is true, the *WebKeysURL* value or the *Provider* value is used to import the data of the
			Optional

Entity	Description	Constants	Necessity
		OpenIDConnectWebKeys entities.	

OutboundUsers	Contains outbound user data	Contains the following attributes and values:
		• OAuth2ClientAuthenticationMethod
		  • basic
		  • mtls
		  • jwt
		• AuthenticationMethod
		  • x509
		  • basic
		  • none
		  • oauth2
		  • oauth2_mtls
		  • sso2
		  • oauth1
			Optional

BusinessPartners	Contains references to the business partner IDs		Optional

Additional Information

> **Note**
>
> For more information about Communication Management, see the *Related Information* section.

Related Information

Communication Arrangements - Entity [page 1132]

APIs on SAP Business Accelerator Hub

Communication Management [page 2992]

Creating Communication Arrangements for SAP Event Mesh

Supported Protocols and Authentication Methods

4.2.13.2.8.3 Communication User - Read and Write Integration

This service enables you to:

- Read and write communication users
- Read and write assigned certificates, communication systems and communication arrangements

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the *Related Information* section.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

Service Group (incl. Namespace if It Exists)	Repository ID	Service Name (incl. Namespace if It Exists)	Version
`aps_com_cu_a4c_odata`	`srvd_a2x`	`aps_com_cu_a4c_odata`	`0001`

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Constants	Necessity

CommunicationUsers	Contains communication user root data	Contains the following attributes and values:
		• Type
		  • customer_communication
		  • provider_communication
		• LockStatus
		  • not_locked
		  • locked_global_by_admin
		  • locked_local_by_admin
		  • locked_by_system
		• PasswordStatus
		  • active
		  • inactive
		  • productive
		  • initial
			Mandatory

Certificates	Contains the assigned certificates		Optional

Additional Information

> **Note**
>
> For more information about Communication Management, see the *Related Information* section.

Related Information

Communication Arrangements - Entity [page 1132]
APIs on SAP Business Accelerator Hub
Communication Management [page 2992]

4.2.13.2.9 Integrating Communication System Data (Read)

You can use this communication scenario to read communication system data.

Context

This communication scenario allows you to read communication system data, such as read communication systems, read assigned inbound users, outbound users and communication arrangements.

Authentication Methods

This communication scenario can be configured with the following inbound authentication methods:

- *Basic Authentication* (user and password)
- *X.509* (certificates)

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Communication System Read Integration* scenario (SAP_COM_0A06) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

Related Information

Communication Management [page 2992]

4.2.13.2.9.1 Communication System - Read Integration

This service enables you to:

- Read communication systems
- Read assigned inbound users, outbound users and communication arrangements

- Read change documents about changes of communication systems and their sub entities

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the *Related Information* section.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

Service Group (incl. Namespace if It Exists)	Repository ID	Service Name (incl. Namespace if It Exists)	Version
`aps_com_api_cs_read`	`srvd_a2`	`CommunicationSystem`	`0001`

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Necessity	Link to Details
CommunicationSystems	Contains communication systems	Mandatory	For more information, see the *Related Information* section.
CommSysChangeDocuments	Contains change document data for communication systems	Optional	For more information, see the *Related Information* section.

Additional Information

> **Note**
>
> For more information about Communication Management, see the *Related Information* section.

Related Information

Communication Systems - Entity [page 1149]
APIs on SAP Business Accelerator Hub
Communication Management [page 2992]
Communication Systems Change Documents - Entity [page 1154]

4.2.13.2.9.1.1 Communication Systems - Entity

Technical name: `CommunicationSystems`

Properties

Property	Description	Necessity
CommunicationSystemID	Communication system ID	Mandatory (Key)
CommunicationSystemName	Communication system name	Optional
HostName	Host name	Optional
UIHostName	UI host name	Optional
TCPPort	TCP port	Optional
LogicalSystemID	Logical system ID	Optional
BusinessSystemID	Business system ID	Optional
InboundOnly	Inbound only	Optional
HubSystem	Hub system	Optional
SAPClient	SAP client	Optional
RFCLoadBalancing	RFC load balancing	Optional
RFCSAPSystemID	RFC SAP system ID	Optional
RFCSAPSystemNumber	RFC SAP system ID number	Optional
RFCLogonGroup	RFC logon group	Optional
RFCMessageServerTargetHost	RFC message server target host	Optional

Property	Description	Necessity
OwnSystem	Own system	Optional
CipherSuites	Cipher suites	Optional
IsCipherSuiteDefault	Default system for cipher suites	Optional
OAuth2IdentityProviderActive	OAuth 2.0 identity provider is active	Optional
OAuth2IdentityProviderName	OAuth 2.0 identity provider name	Optional
OAuth2IDPUserLogonType	OAuth 2.0 IDP user logon type	Optional
OAuth2AuthEndpoint	OAuth 2.0 authorization endpoint	Optional
OAuth2TokenEndpoint	OAuth 2.0 token endpoint	Optional
OAuth2Audience	OAuth 2.0 audience	Optional
SAPCloudConnectorActive	SAP Cloud Connector is active	Optional
SAPCloudConnectorLocationID	SAP Cloud Connector location ID	Optional
SAMLBearerAssertionProvActive	SAML Bearer Assertion Provider is active	Optional
SAMLBearerAssertionProvName	SAML Bearer Assertion Provider name	Optional
SAMLBearerAssProvUserLogonType	SAML Bearer Assertion Provider user logon type	Optional
OwnerContactPersonName	Owner contact person name	Optional
OwnerContactPersonPhone	Owner contact person phone	Optional
OwnerContactPersonEMail	Owner contact person email	Optional
LastChangedDateTime	Date and time of last change	Optional
LastChangedByUserID	Last changed by <user ID>	Optional
LastChangedByUserName	Last changed by <user name>	Optional
LastChangedByUserDescription	Last changed by <user description>	Optional
LastChangedByEMailAddress	Last changed by <email address>	Optional
LastChangedByGlobalUserID	Last changed by <global user ID>	Optional

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

Expandables

This entity is expandable with the following:

- Inbound Users
- Outbound Users
- Communication Arrangements

Related Information

Communication System – Inbound User Entity [page 1151]
Communication System – Outbound User Entity [page 1152]
Communication System – Communication Arrangement Entity [page 1153]

4.2.13.2.9.1.1.1 Communication System – Inbound User Entity

Technical name: `CommSystemInboundUsers`

Properties

Property	Description	Necessity
CommunicationSystemID	Communication system ID	Mandatory (Key)
InboundUUID	Inbound UUID	Mandatory (Key)
UserID	User ID	Mandatory (Key)
UserName	User name	Optional
OAuth2ClientID	OAuth 2.0 client ID	Optional
OAuth2GrantType	OAuth 2.0 grant type	Optional

Property	Description	Necessity
AuthMethod	**`x509`**: SSL Client Certificate **`basic`**: User ID and Password **`none`**: None **`oauth2`**: OAuth 2.0 **`oauth1`**: OAuth 1.0 **`princ_prop`**: Principal Propagation **`oauth2_mtls`**: OAuth 2.0 (mTLS)	Optional

> ### Note
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.9.1.1.2 Communication System – Outbound User Entity

Technical name: `CommSystemOutboundUsers`

Properties

Property	Description	Necessity
CommunicationSystemID	Communication system ID	Mandatory (Key)
OutboundUUID	Outbound UUID	Mandatory (Key)
UserName	User name	Mandatory (Key)
OAuth2ClientID	OAuth 2.0 client ID	Mandatory (Key)
CertificateID	Certificate ID	Optional

Property	Description	Necessity
AuthMethod	**`x509`**: SSL Client Certificate **`basic`**: User ID and Password **`none`**: None **`oauth2`**: OAuth 2.0 **`oauth1`**: OAuth 1.0 **`princ_prop`**: Principal Propagation **`oauth2_mtls`**: OAuth 2.0 (mTLS)	Optional

> ⓘ Note
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.9.1.1.3 Communication System – Communication Arrangement Entity

Technical name: `CommSystemCommArrangements`

Properties

Property	Description	Necessity
CommunicationSystemID	Communication system ID	Mandatory (Key)
CommunicationArrangementUUID	Communication arrangement UUID	Mandatory (Key)
CommunicationArrangementName	Communication arrangment name	Optional
CommunicationScenarioID	Communication scenario ID	Optional

> ⓘ Note
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.9.1.2 Communication Systems Change Documents - Entity

Technical name: `CommSysChangeDocuments`

Properties

Property	Description	Necessity
CDocClass	Object class	Mandatory (Key)
CDocKey	Document object	Mandatory (Key)
CDoc	Change document number	Mandatory (Key)
DBTableKey	Table key	Mandatory (Key)
Table Field	Table field	Mandatory (Key)
FieldName	Field name	Mandatory (Key)
DBKey	Table key	Mandatory (Key)
CommunicationSystemID	System ID	Mandatory (Key)
CommunicationSystemName	System name	Mandatory (Key)
ChangeCategory	Change category	Mandatory (Key)
Action	Action	Mandatory (Key)
ChangedAt	Changed at <date and time>	Mandatory (Key)
ChangedBy	Changed by <name>	Mandatory (Key)
Attribute	Attribute	Mandatory (Key)
ChangedFrom	Changed from	Mandatory (Key)
ChangedTo	Changed to	Mandatory (Key)
ChangeCategoryText	Change category text	Optional
ChangedByName	Changed by <name>	Optional
AttributeText	Attribute text	Optional
ActionText	Action text	Optional
ChangedFromText	'Changed From' text	Optional
ChangedToText	'Changed To' text	Optional

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.10 Integrating Communication User Data (Read)

You can use this communication scenario to read communication user data.

Context

This communication scenario allows you to read communication user data, such as read communication users, read assigned certificates, communication systems and communication arrangements.

Authentication Methods

This communication scenario can be configured with the following inbound authentication methods:

- *Basic Authentication* (user and password)
- *X.509* (certificates)

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Communication User Read Integration* scenario (SAP_COM_0A05) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

Related Information

Communication Management [page 2992]

4.2.13.2.10.1 Communication User - Read Integration

This service enables you to:

- Read communication users
- Read assigned certificates, communication systems and communication arrangements

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the *Related Information* section.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

Service Group (incl. Namespace if It Exists)	Repository ID	Service Name (incl. Namespace if It Exists)	Version
`aps_com_api_cusr_read`	`srvd_a2`	`CommunicationUser`	`0001`

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Necessity	Link to Details
CommunicationUsers	Contains communication users	Mandatory	For more information, see the *Related Information* section.

Additional Information

> **Note**
>
> For more information about Communication Management, see the *Related Information* section.

Related Information

Communication Users - Entity [page 1157]
APIs on SAP Business Accelerator Hub
Communication Management [page 2992]

4.2.13.2.10.2 Communication Users - Entity

Technical name: `CommunicationUsers`

Properties

Property	Description	Necessity
UserID	User ID	Mandatory (Key)
UserName	User Name	Optional
Description	Description	Optional
LastLogonDateTime	Date and time of last logon	Optional
LockStatus	Lock status	Optional
PasswordStatus	Password status	Optional
LastChangedDateTime	Date and time of last change	Optional
LastChangedByUserID	Last changed by <user ID>	Optional
LastChangedByUserName	Last changed by <user name>	Optional
LastChangedByUserDescription	Last changed by <user description>	Optional
LastChangedByEMailAddress	Last changed by <email address>	Optional
LastChangedByGlobalUserID	Last changed by <global user ID>	Optional

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

Expandables

This entity is expandable with the following:

- Certificates
- Communication Systems
- Communication Arrangements

Related Information

Communication User - Certificate Entity [page 1158]
Communication User – Communication Arrangement Entity [page 1158]
Communication User – Communication System Entity [page 1159]

4.2.13.2.10.2.1 Communication User - Certificate Entity

Technical name: `CommunicationUserCertificates`

Properties

Property	Description	Necessity
UserID	User ID	Mandatory (Key)
SubjectHash	Subject Hash	Mandatory (Key)
IssuerHash	Issuer Hash	Mandatory (Key)
Issuer	Issuer	Optional
Subject	Subject	Optional

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.10.2.2 Communication User – Communication Arrangement Entity

Technical name: `CommUserCommArrangements`

Properties

Property	Description	Necessity
UserID	User ID	Mandatory (Key)
CommunicationArrangementUUID	Communication arrangement UUID	Mandatory (Key)
CommunicationArrangementName	Communication arrangement name	Optional
CommunicationSystemID	Communication system ID	Optional
CommunicationScenarioID	Communication scenario ID	Optional

> ### Note
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.10.2.3 Communication User – Communication System Entity

Technical name: `CommUserCommSystems`

Properties

Property	Description	Necessity
UserID	User ID	Mandatory (Key)
CommunicationSystemID	Communication system ID	Mandatory (Key)

> ### Note
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose *Schema View*.

4.2.13.2.11 Integrating Content Security Policy Data (Read)

You can use this communication scenario to read Content Security Policy (CSP) data.

Context

This communication scenario allows you to read Content Security Policy (CSP) data, such as read Content Security Policy statuses, or read trusted sites.

Authentication Methods

This communication scenario can be configured with the following inbound authentication methods:

- *Basic Authentication* (user and password)
- *X.509* (certificates)

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Content Security Policy Read Integration* scenario (SAP_COM_0A08) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

Related Information

Communication Management [page 2992]

4.2.13.2.11.1 Content Security Policy - Read Integration

This service enables you to

- Read Content Security Policy statuses
- Read trusted sites

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the *Related Information* section.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

Service Group (incl. Namespace if It Exists)	Repository ID	Service Name (incl. Namespace if It Exists)	Version
`aps_sec_api_csp_read`	`srvd_a2x`	`ContentSecurityPolicy`	`0001`

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Necessity	Link to Details
ContentSecurityPolicyStatus	Contains the Content Security Policy statuses	Mandatory	For more information, see the *Related Information* section.
TrustedSites	Contains trusted sites	Mandatory	For more information, see the *Related Information* section.

Additional Information

> **Note**
>
> For more information about Communication Management, see the *Related Information* section.

Related Information

Content Security Policy Status - Entity [page 1162]
Trusted Sites - Entity [page 1162]
APIs on SAP Business Accelerator Hub
Communication Management [page 2992]

4.2.13.2.11.1.1 Content Security Policy Status - Entity

Technical name: `ContentSecurityPolicyStatus`

Properties

Property	Description	Necessity
ContentSecurityPolicyStatus	Content Security Policy Status	Mandatory (Key)

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.11.1.2 Trusted Sites - Entity

Technical name: `TrustedSites`

Properties

Property	Description	Necessity
ListName	Trusted Sites	Mandatory (Key)

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

Expandables

This entity is expandable with:

- TrustedContent

Related Information

Trusted Sites – Content Entity [page 1163]

4.2.13.2.11.1.2.1 Trusted Sites – Content Entity

Technical name: `TrustedSiteContent`

Properties

Property	Description	Necessity
ListName	Trusted Sites	Mandatory (Key)
TrustedSite	Trusted Site	Mandatory (Key)

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.12 Integrating Protection Allowlist Data (Read)

You can use this communication scenario to read protection allowlist data.

Context

This communication scenario allows you to read protection allowlist data, such as read clickjacking protection data, read trusted network zones, read trusted CSS Style Sheets, read Cross-Origin Resource Sharing.

Authentication Methods

This communication scenario can be configured with the following inbound authentication methods:

- *Basic Authentication* (user and password)
- *X.509* (certificates)

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Protection Allowlist Read Integration* scenario (SAP_COM_0A09) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

Related Information

Communication Management [page 2992]

4.2.13.2.12.1 Protection Allowlist - Read Integration

This service enables you to:

- Read clickjacking protection data
- Read trusted network zones
- Read trusted CSS Style Sheets
- Read Cross-Origin Resource Sharing

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the *Related Information* section.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

Service Group (incl. Namespace if It Exists)	Repository ID	Service Name (incl. Namespace if It Exists)	Version
`aps_sec_api_pal_read`	`srvd_a2`	`ProtectionAllowlist`	`0001`

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Necessity	Link to Details
ClickjackingProtection	Contains trusted hosts for Clickjacking Protection	Mandatory	For more information, see the *Related Information* section.
TrustedNetworkZones	Contains trusted hosts for trusted network zones	Mandatory	For more information, see the *Related Information* section.
TrustedCSSStyleSheets	Contains trusted hosts CSS Style Sheets	Mandatory	For more information, see the *Related Information* section.
CrossOriginResourceSharing	Contains trusted hosts for Cross-Origin Resource sharing	Mandatory	For more information, see the *Related Information* section.

Additional Information

> **Note**
>
> For more information about Communication Management, see the *Related Information* section.

Related Information

Clickjacking Protection - Entity [page 1166]
CORS - Entity [page 1166]
APIs on SAP Business Accelerator Hub
Communication Management [page 2992]

4.2.13.2.12.1.1 Clickjacking Protection - Entity

Technical name: `ClickjackingProtection`

Properties

Property	Description	Necessity
HostName	Host name	Mandatory: Key
SchemeCode	Scheme code	Optional
Port	Port	Optional

To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.12.1.2 CORS - Entity

Technical name: `CrossOriginResourceSharing`

Properties

Property	Description	Necessity
HostName	Host Name	Mandatory: Key
ServicePath	Service Path	Optional
HTTPGet	HTTP Get	Optional
HTTPPost	HTTP Post	Optional
HTTPHead	HTTP head	Optional

Property	Description	Necessity
HTTPPut	HTTP Put	Optional
HTTPPatch	HTTP Patch	Optional
HTTPDelete	HTTP Delete	Optional
HTTPTrace	HTTP Trace	Optional
HTTPOptions	HTTP Options	Optional
HTTPConnect	HTTP Connect	Optional
AllowCredentials	Allow Credentials	Optional
AllowPrivateNetworkAccess	Allow Private Network Access	Optional
MaximumAge	MaximumAge	Optional

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

Expandables

This entity is expandable with the following:

- Allowed Headers
- Exposed Headers

Related Information

CORS – Allowed Header Entity [page 1167]
CORS – Exposed Header Entity [page 1168]

4.2.13.2.12.1.2.1 CORS – Allowed Header Entity

Technical name: `CORSAllowedHeaders`

Properties

Property	Description	Necessity
HostName	Host Name	Mandatory: Key
AllowedHeader	Allowed header	Mandatory (Key)

> ⓘ Note
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.12.1.2.2 CORS – Exposed Header Entity

Technical name: `CORSExposedHeaders`

Properties

Property	Description	Necessity
HostName	Host Name	Mandatory: Key
ExposedHeader	Exposed header	Mandatory (Key)

> ⓘ Note
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.12.1.3 Trusted CSS Style Sheets - Entity

Technical name: `TrustedCSSStyleSheets`

Properties

Property	Description	Necessity
HostName	Host name	Mandatory (Key)
SchemeCode	Scheme code	Optional
Port	Port	Optional

> ⓘ Note
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.12.1.4 Trusted Network Zones - Entity

Technical name: `TrustedNetworkZones`

Properties

Property	Description	Necessity
HostName	Host name	Mandatory (Key)
SchemeCode	Scheme code	Optional
Port	Port	Optional

> ⓘ Note
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.13 Integrating Certificate Data (Read)

Context

You can use this communication scenario to read certificate data.

The communication scenario SAP_COM_0A13 allows you to read certificate data, such as client certificates and certificate trust lists.

Authentication Methods

This communication scenario can be configured with the following inbound authentication methods:

- *Basic Authentication* (user and password)
- *X.509* (certificates)

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related Information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Certificate Read Integration* scenario (SAP_COM_0A13) in the *Communication Arrangements* app.

   For more information, see the *Related Information* section.

Related Information

Communication Management [page 2992]

4.2.13.2.13.1 Certificates - Read Integration

This service enables you to:

- Read client certificates
- Read certificate trust lists

- Read signing certificates

This service is published on the SAP Business Accelerator Hub. For more information about APIs, see the *Related Information* section.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

Service Group (incl. Namespace If It Exists)	Repository ID	Service Name (incl. Namespace If It Exists)	Version
`aps_sec_api_cert_read`	`srvd_a2x`	`Certificates`	`0001`

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Necessity	Link to Details
ClientCertificates	Contains client certificates	Mandatory	For more information, see the *Related Information* section.
CertificateTrustList	Contains certificate trust lists	Mandatory	For more information, see the *Related Information* section.
SigningCertificates	Contains signing certificates	Mandatory	For more information, see the *Related Information* section.

> **Note**
>
> For more information about Communication Management, see the *Related Information* section.

Related Information

APIs on SAP Business Accelerator Hub
Communication Management [page 2992]

4.2.13.2.13.1.1 Client Certificates - Entity

Technical name: `ClientCertificates`

Properties

Property	Description	Necessity
Certificate	Certificate	Mandatory (Key)
Description	Description	Optional
ManagedBy	Managed by	Optional
IssuedTo	Issued to	Optional
IssuedBy	Issue by	Optional
PublicKey	Public key	Optional
PublicKeyAlgorithm	Public key algorithm	Optional
FingerPrint	Fingerprint	Optional
SerialNumber	Serial number	Optional
SignatureAlgorithm	Signature algorithm	Optional
Status	Certificate status	Optional
StatusText	Certificate status text	Optional
IsValid	Valid	Optional
ValidFromDateTime	**Valid From** date and time	Optional
ValidToDateTime	**Valid To** date and time	Optional
LastChangedDateTime	Date and time of last change	Optional
LastChangedByUserID	*Last Changed By* <user ID>	Optional

Property	Description	Necessity
LastChangedByUserName	*Last Changed By* <user name>	Optional
LastChangedByUserDescription	*Last Changed By* <user description>	Optional
LastChangedByEMailAddress	*Last Changed By* <email address>	Optional
LastChangedByGlobalUserID	*Last Changed By* <global user ID>	Optional

Note

To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

Expandables

This entity is expandable with the following:

- Subject
- Issuer
- CommunicationSystems

Related Information

Client Certificates - Subject [page 1173]
Client Certificates - Issuer [page 1174]
Client Certificates - Communication Systems [page 1174]

4.2.13.2.13.1.1.1 Client Certificates - Subject

Technical name: `ClientCertificatesSubject`

Properties

Property	Description	Necessity
Certificate	Certificate	Mandatory (Key)
IssuedTo	Issued to	Mandatory (Key)

Property	Description	Necessity
OrganizationUnit	Organizational unit	Optional
Organization	Organization	Optional
Country	Country	Optional
City	City	Optional
State	State	Optional

Note

To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.13.1.1.2 Client Certificates - Issuer

Technical name: `ClientCertificatesIssuer`

Properties

Property	Description	Necessity
Certificate	Certificate	Mandatory (Key)
IssuedBy	Issued by	Mandatory (Key)
OrganizationUnit	Organizational unit	Optional
Organization	Organization	Optional
Country	Country	Optional
City	City	Optional
State	State	Optional

Note

To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.13.1.1.3 Client Certificates - Communication Systems

Technical name: `ClientCertificatesCommSystems`

Properties

Property	Description	Necessity
Certificate	Certificate	Mandatory (Key)
CommunicationSystemID	Communication system ID	Mandatory (Key)

Note

To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.13.1.2 Certificate Trust List - Entity

Technical name: `CertificateTrustList`

Properties

Property	Description	Necessity
TrustListKey	Trust list key	Mandatory (Key)
ManagedBy	Managed by	Optional
IssuedTo	Issued to	Optional
IssuedBy	Issued by	Optional
PublicKey	Public key	Optional
PublicKeyAlgorithm	Public key algorithm	Optional
FingerPrint	Fingerprint	Optional
SerialNumber	Serial number	Optional
SignatureAlgorithm	Signature algorithm	Optional
IsValid	Valid	Optional
ValidFromDateTime	**Valid From** date and time	Optional
ValidToDateTime	**Valid To** date and time	Optional

Note

To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

Expandable

This entity is expandable with the following:

- Subject
- Issuer

Related Information

Certificate Trust List - Subject [page 1176]
Certificate Trust List - Issuer [page 1177]

4.2.13.2.13.1.2.1 Certificate Trust List - Subject

Technical name: `CertificateTrustListSubject`

Properties

Property	Description	Necessity
TrustListKey	Trust list key	Mandatory (Key)
IssuedTo	Issued to	Mandatory (Key)
OrganizationUnit	Organizational unit	Optional
Organization	Organization	Optional
Country	Country	Optional
City	City	Optional
State	State	Optional

> **Note**
>
> To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.13.1.2.2 Certificate Trust List - Issuer

Technical name: CertificateTrustListIssuer

Properties

Property	Description	Necessity
TrustListKey	Trust list key	Mandatory (Key)
IssuedBy	Issued by	Mandatory (Key)
OrganizationUnit	Organizational unit	Optional
Organization	Organization	Optional
Country	Country	Optional
City	City	Optional
State	State	Optional

Note

To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.13.1.3 Signing Certificates - Entity

Technical name: `SigningCertificates`

Properties

Property	Description	Necessity
SigningKey	Signing key	Mandatory (Key)
CertificateType	Certifcate type	Optional
IssuedTo	Issued to	Optional
IssuedBy	Issued by	Optional

Property	Description	Necessity
PublicKey	Public key	Optional
PublicKeyAlgorithm	Public key algorithm	Optional
FingerPrint	Fingerprint	Optional
SerialNumber	Serial number	Optional
SignatureAlgorithm	Signature algorithm	Optional
IsValid	Valid	Optional
ValidFromDateTime	**Valid From** date and time	Optional
ValidToDateTime	**Valid To** date and time	Optional
CertificateTypeName	Certificate type name	Optional

Note

To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

Expandables

This entity is expandable with the following:

- Subject
- Issuer
- CommunicationSystems

Related Information

Signing Certificates - Subject [page 1178]
Signing Certificates - Issuer [page 1179]
Signing Certificates - Communication Systems [page 1180]

4.2.13.2.13.1.3.1 Signing Certificates - Subject

Technical name: `SigningCertificatesSubject`

Properties

Property	Description	Necessity
SigningKey	Signing key	Mandatory (Key)
IssuedTo	Issued to	Mandatory (Key)
OrganizationUnit	Organizational unit	Optional
Organization	Organization	Optional
Country	Country	Optional
City	City	Optional
State	State	Optional

Note

To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.13.1.3.2 Signing Certificates - Issuer

Technical name: `SigningCertificatesIssuer`

Properties

Property	Description	Necessity
IssuedBy	Issued by	Mandatory (Key)
OrganizationUnit	Organizational unit	Optional
Organization	Organization	Optional
Country	Country	Optional
City	City	Optional
State	State	Optional

Note

To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.13.1.3.3 Signing Certificates - Communication Systems

Technical name: `SigningCertificatesCommSystems`

Properties

Property	Description	Necessity
SigningKey	Signing key	Mandatory (Key)
CommunicationSystemID	Communication system ID	Mandatory (Key)

Note

To display more information about the properties on the SAP Business Accelerator Hub, choose **Schema View**.

4.2.13.2.14 Integrating System for Cross-Domain Identity Management (SCIM)

You can use this communication scenario to maintain data via SCIM.

Context

The communication scenario SAP_COM_0465 allows you to maintain data via SCIM (System for Cross-domain Identity Management).

Authentication Methods

This communication scenario can be configured with the following inbound authentication methods:

- *Basic Authentication* (user and password)
- *X.509* (certificates)

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.

2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *System for Cross-domain Identity Management Integration* scenario (SAP_COM_0465) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

Related Information

Communication Management [page 2992]
SCIM Interface for IAM [page 1181]
User Management (System for Cross-domain Identity Management (SCIM))

4.2.13.2.14.1 SCIM Interface for IAM

SCIM stands for System for Cross-domain Identity Management. It is an open standard that allows for the automation of user provisioning and management in cloud-based applications and services.

You can use this interface provided by the communication scenario `SAP_COM_0465` to maintain and retrieve business users in your Cloud system and assign business roles and business user groups to them.

> ⚠ Caution
>
> The *userType* attribute in the SCIM User payload supports only a predefined set of values.
>
> If the *userType* attribute is not provided, it will default to *Employee*.
>
> Any value other than the supported ones will result in a processing error.
>
> Do not send unsupported user type values. If required, map existing user type values from your source system to one of the supported values before calling the SCIM interface.

For more information about SCIM in general, see System for cross-domain Identity Management .

Useful links can also be found in the *Related Information* section at the bottom of this document.

> ⓘ Note
>
> The write operations are protected with a X-CSRF-Token.

SCIM Extensions

In addition, your Cloud system provides the following extensions to extend the endpoints of users and groups:

Extension name: *urn:ietf:params:scim:schemas:extension:sap:2.0:User*

The following singular attributes are defined:

- *validFrom*: Valid From date of a business user
- *validTo*: Valid To date of a business user
- *loginTime*: Last login time of a business user
- *userUuid*: Defines the global user ID
- *userUuidHistory*: All global user IDs that were assigned to the business user
  - *value*: Global user ID assigned to a business user
  - *active*: Global user ID currently actively in use
- *language*: Contains different languages
  - *correspondence*: The worker's correspondence language
- *localeSettings*: Describes the local settings
  - *numericalRepresentation*: defines how numbers are represented
  - *date*: defines how dates are represented
  - *time*: defines how times are represented

Example

Sample Code

```
      "urn:ietf:params:scim:schemas:extension:sap:2.0:User" : {
        "validFrom" : "2023-05-01T00:00:00Z",
        "validTo" : "2123-05-01T00:00:00Z",
        "loginTime" : "2024-01-01T11:22:33Z",
        "userUuid" : "<GLOBAL_USER_ID>",
        "userUuidHistory" : [
          {
            "value" : "<GLOBAL_USER_ID>",
            "active" : true
          },
          {
            "value" : "<GLOBAL_USER_ID_OLD>",
            "active" : false
          }
        ]
      }
        ],
        "language": {
          "correspondence": "EN"
        },
        "localeSettings": {
          "numericalRepresentation": "1.234.567,89",
          "date": "DD.MM.YYYY (Gregorian Date)",
          "time": "24 Hour Format (Example: 12:05:10)"
        }
      }
```

Extension name: *urn:ietf:params:scim:schemas:extension:sap.erp.worker:2.0:User*

The following singular attributes are defined:

- *personalInformation*: Contains the personal information of a worker
  - *additionalLastName*: Contains the additional last name
  - *birthName*: Contains the birth name
  - *initials*: Contains the initials
  - *lastNamePrefix*: Contains the last name prefix

    - *lastNameSecondPrefix*: Contains the second prefix of the last name
    - *nameSupplement*: Contains the name supplement
    - *academicSecondTitle*: Contains the academic second title
- *validity*: Contains the validity of the worker
    - *startDate*: Contains the start date
    - *endDate*: Contains the end date
- *workplace*: Contains the work place
    - *functionalTitleName*: Contains the name of the functional title

Sample Code

```
"urn:ietf:params:scim:schemas:extension:sap.erp.worker:2.0:User": {
        "personalInformation": {
          "additionalLastName": "Musterextra",
          "birthName": "Geburtsname",
          "initials": "MM",
          "lastNamePrefix": "von",
          "lastNameSecondPrefix": "von",
          "nameSupplement": "Graf",
          "academicSecondTitle": "Dr."
        },
        "validity": {
          "startDate": "2025-11-14T09:02:49.044Z",
          "endDate": "2025-11-14T09:02:49.044Z"
        },
        "workplace": {
          "functionalTitleName": "Functional Title Name"
        }
      }
```

Extension name: *urn:ietf:params:scim:schemas:extension:sap:2.0:Group*

The following singular attributes are defined:

- *type*: Describes whether the group is a business role (*authorization*) or a business user group (*userGroup*)
- *supportedOperations*: Describes the possible operations that can be performed either with the group *membership* or *readWrite*

Example

Sample Code

```
"urn:ietf:params:scim:schemas:extension:sap:2.0:Group": {
  "type": "authorization"
  "supportedOperations": "membership"
}
```

Note

For groups of the *userGroup* type, a user can have only one assignment at a time. If a user is assigned to a new group of the *userGroup* type, the previous assignment of the same type will be automatically removed, and the user will be exclusively assigned to the newly selected group.

Parameter Equivalents in your Cloud system

User

Parameter in SCIM Standard	Equivalent in your Cloud System
id	Unique *business user ID*
externalId	*worker/employee ID*
userName	*user name*
groups["type"]	Determines whether it is a business role or business user group.

Group

Parameter in SCIM Standard	Equivalent in your Cloud System
id	Unique internal ID for SCIM containing a prefix (*BROL_* or *BUGR_*) and the business role or business user group ID to ensure that no duplicates exist. This ID has no corresponding Cloud equivalent.
externalId	Business role or business user group ID

Monitoring SCIM Interface Content in the SAP Application Interface Framework (AIF)

To view messages and errors related to SCIM, call up AIF and enter the following data:

Namespace: **`/IAM`**

Interface Name: **`SCIM`**

Interface Version: **`1`**

We recommend that you use the body of the payload from the *Message Dashboards* app because it is easier to read and export the body in this app. For more information, see Message Dashboard.

For more information about AIF, see SAP Application Interface Framework.

Related Information

System for cross-domain Identity Management

Example: User Endpoint [page 1185]
Example: Group Endpoint [page 1186]

4.2.13.2.14.2 Example: User Endpoint

Please find an example of a user endpoint below:

Sample Code

```
{
  "schemas" : [
    "urn:ietf:params:scim:schemas:core:2.0:User",
    "urn:ietf:params:scim:schemas:extension:sap:2.0:User"
  ],
  "id" : "CB0000000001",
  "externalId" : "WORKER_ID",
  "userName" : "JOHN_DOE",
  "name" : {
    "formatted" : "John Doe",
    "familyName" : "Doe",
    "givenName" : "John"
  },
  "displayName" : "John Doe",
  "emails" : [
    {
      "value" : "john.doe@sap.com",
      "type" : "work",
      "primary" : true
    }
  ],
  "phoneNumbers" : [
    {
      "value" : "+123456789",
      "type" : "work"
    },
    {
      "value" : "+987654321",
      "type" : "mobile"
    }
  ],
  "userType" : "Employee",
  "preferredLanguage" : "E",
  "timezone" : "CET",
  "active" : true,
  "groups" : [
    {
      "value" : "BUGR_ZCBUSERGROUP",
      "$ref" : "https://system.com/sap/bc/http/sap/aps_iam_api_scim/Groups/
BUGR_ZCBUSERGROUP",
      "display" : "business user group",
      "type" : "userGroup"
    },
    {
      "value" : "BROL_BUSINESSROLE1",
      "$ref" : "https://system.com/sap/bc/http/sap/aps_iam_api_scim/Groups/
BROL_BUSINESSROLE1",
      "display" : "business role 1",
      "type" : "authorization"
    },
    {
      "value" : "BROL_BUSINESSROLE2",
      "$ref" : "https://system.com/sap/bc/http/sap/aps_iam_api_scim/Groups/
BROL_BUSINESSROLE2",
      "display" : "business role 2",
      "type" : "authorization"
    }
  ],
  "urn:ietf:params:scim:schemas:extension:sap:2.0:User" : {
```

```
    "userUuid" : "GLOBAL_USER_ID"
  },
  "meta" : {
    "resourceType" : "User",
    "created" : "2015-05-22T12:53:56Z",
    "lastModified" : "2015-05-22T12:53:56Z",
    "version" : "20150522125356",
    "location" : "https://system.com/sap/bc/http/sap/aps_iam_api_scim/Users/
CB0000000001"
  }
}
```

4.2.13.2.14.3 Example: Group Endpoint

Please find an example of a group endpoint below:

Sample Code

```
{
  "schemas" : [
    "urn:ietf:params:scim:schemas:core:2.0:Group",
    "urn:ietf:params:scim:schemas:extension:sap:2.0:Group"
  ],
  "id" : "BROL_BUSINESSROLE1",
  "externalId" : "BUSINESSROLE1",
  "displayName" : "business role 1",
  "members" : [
    {
      "value" : "CB0000000001",
      "$ref" : "https://system.com/sap/bc/http/sap/aps_iam_api_scim/Users/
CB0000000001",
      "display" : "John Doe"
    }
  ],
  "urn:ietf:params:scim:schemas:extension:sap:2.0:Group" : {
    "type" : "authorization",
    "supportedOperations" : "membership"
  },
  "meta" : {
    "resourceType" : "Group",
    "created" : "2018-12-11T19:18:05Z",
    "lastModified" : "2021-09-08T12:38:32Z",
    "version" : "20210908123832",
    "location" : "https://system.com/sap/bc/http/sap/aps_iam_api_scim/Groups/
BROL_BUSINESSROLE1
  }
}
```

4.2.13.2.15 Manage Git Repository

The Manage Git Repository API allows you to manage software components (Git repositories) on an SAP Business AI Platform, ABAP environment system.

Overview

On an ABAP Environment system, software components, which represent Git repositories, can be controlled with Git operations. This includes cloning to an ABAP Environment system, pulling updates and working with branches (create, delete, checkout). These operations can be used with this API.

Technical Name: MANAGE_GIT_REPOSITORY

OData Version: 2.0

Root URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY

Permissions

In order to access this service, you need to:

- Create a *Communication User* as described in How to Create Communication Users [page 2995].
- Create a *Communication System* as described in How to Create Communication Systems.
- Create a *Communication Arrangement* as described in How to Create a Communication Arrangement [page 2998].
- Select the communication scenario **SAP_COM_0510** for your communication arrangement and have mapped it to your communication system.

Service Structure

Service Metadata URI: sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/$metadata

Service Entities

Service Entity	Description
Branches [page 1187]	Branches of a software component
Clones [page 1195]	Clone job of a software component
Execution Log [page 1199]	Execution Log of a software component
Pull [page 1201]	Pull job of a software component
Transport Log [page 1205]	Transport log of an impored transport

4.2.13.2.15.1 Branches

Resource path: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Branches

Operations

CRUD Operations

HTTP Method	Operation
GET	Read Branches [page 1188]
POST	Create Branch [page 1190]
DELETE	Delete Branches [page 1192]

Custom Operations

HTTP Method	Operation Type	Operation	URI
POST	Action	Checkout Branches [page 1193]	/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/checkout_branch

Parameters

Property	Description
branch_name	name of the branch
sc_name	name of the software component
is_active	indicates if the branch is active in the system
critivality	
derived_from	name of the original branch from which the branch was derived
created_by	user who created the branch
created_on	date when the branch was created
commit_id	latest commit ID on the branch
commit_message	commit message of the latest commit
last_commit_by	user who commited the last commit
last_commit_on	date of the latest commit

4.2.13.2.15.1.1 Read Branches

Read one or multiple branches .

Request

URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Branches

Operation Type: CRUD

HTTP Method: GET

Request Headers

Header	Required	Values
Accept	no	application/json application/xml
x-csrf-token	no	fetch

Request Parameters

Parameter	Required	Data Type	Description	Parameter Type
sc_name	yes	string	name of the software component	query string
branch_name	yes	string	name of the branch	query string

Request Example

Sample Code

```
GET /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
Branches(branch_name='newBranch',sc_name='/DMO/GIT_REPOSITORY' HTTP/1.1
Host: host.com
Authentication: basicAuthentication
Accept: application/json
```

Response

Response Headers

Header	Description
x-csrf-token	Token, which can be used for POST requests

Response Status and Error Codes

Code	Reason	Description
200	OK	Branch was read successfully
404	Not foumd	Could not read branch; software component or branch may not exist..

Request Payload Example

Sample Code

```
{
```

```
    "d": {
        "__metadata": {
            "id": "https://host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
Branches(branch_name='newBranch',sc_name='/DMO/GIT_REPOSITORY')",
            "uri": "https://host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
Branches(branch_name='newBranch',sc_name='/DMO/GIT_REPOSITORY')",
            "type": "cds_sd_a4c_a2g_gha_sc_web_api.BranchesType"
        },
        "branch_name": "newBranch",
        "sc_name": "/DMO/GIT_REPOSITORY",
        "is_active": false,
        "criticality": 0,
        "derived_from": "main",
        "created_by": "CC0000000001",
        "created_on": "/Date(1584967657000+0000)/",
        "last_commit_on": "/Date(1584634658000+0000)/"
    }
}
```


4.2.13.2.15.1.2 Create Branch

Create a new branch.

Request

URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Branches

Operation Type: CRUD

HTTP Method: POST

Request Headers

Header	Required	Values
Accept	no	application/json application/xml
Content-Type	no	application/json application/xml
x-csrf-token	yes	Value of x-csrf-token

Request Parameters

Parameter	Required	Data Type	Description	Parameter Type
sc_name	yes	string	name of the software component	Request body

Parameter	Required	Data Type	Description	Parameter Type
branch_name	yes	string	name of the branch	Request body
derived_from	yes	string	name of the original branch form which the branch was derived	Request body

Request Example

Sample Code

```
POST/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Branches HTTP/1.1
Host: host.com
Authentication: basicAuthentication
X-csrf-token: xCsrfToken
Content-Type: application/json
Accept: application/json
    {
"sc_name" : "/DMO/GIT_REPOSITORY",
"derived_from" : "main",
"branch_name" : "newBranch"
    }
```

Response

Response Status and Error Codes

Code	Reason	Description
201	Created	Branch was created successfully
400	Bad Request	Could not create a branch due to the values passed in the request body.

Request Payload Example

Sample Code

```
{
    "d": {
        "__metadata": {
            "id": "https://host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
Branches(branch_name='newBranch',sc_name='/DMO/GIT_REPOSITORY')",
            "uri": "https://host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
Branches(branch_name='newBranch',sc_name='/DMO/GIT_REPOSITORY')",
            "type": "cds_sd_a4c_a2g_gha_sc_web_api.BranchesType"
        },
        "branch_name": "newBranch",
        "sc_name": "/DMO/GIT_REPOSITORY",
        "is_active": false,
        "criticality": 0,
        "derived_from": "master",
        "created_by": "CC0000000001",
        "created_on": "/Date(1584967657000+0000)/",
        "last_commit_on": "/Date(1584634658000+0000)/"
    }
```

```
}
```

4.2.13.2.15.1.3 Delete Branches

Delete a branch.

Request

URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Branches

Operation Type: CRUD

HTTP Method: DELETE

Request Headers

Request Parameters

Parameter	Required	Data Type	Description	Parameter Type
sc_name	yes	string	name of the software component	query string
branch_name	yes	string	name of the branch	query string

Request Example

Sample Code

```
DELETE /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Branches(branch_name='my-to-
be-deleted-branch',sc_name='/DMO/GIT_REPOSITORY') HTTP/1.1
Host: host.com
Authentication: basicAuthentication
Accept: application/json
```

Response

Response Headers

Sample Code

```
HTTP/1.1 204 No Content
```

Response Status and Error Codes

Code	Reason	Description
204	No content	Branch was deleted successfully
400	Bad Request	Could not delete branch due to the values passed in the request body.

4.2.13.2.15.1.4 Checkout Branches

Checkout a branch.

Request

URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Branches

Operation Type: CRUD

HTTP Method: POST

Request Headers

Header	Required	Values
Content-Type	no	application/json application/xml
Accept	no	application/json application/xml
x-csrf-token	yes	Value of x-csrf-token

Request Parameters

Parameter	Required	Data Type	Description	Parameter Type
sc_name	yes	string	name of the software component	query string
branch_name	yes	string	name of the branch	query string

Request Example

Sample Code

```
POST/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/checkout_branch?
branch_name='newBranch'&sc_name="'/DMO/GIT_REPOSITORY' HTTP/1.1
Host: host.com
Authentication: basicAuthentication
```

```
X-csrf-token: xCsrfToken
Content-Type: application/json
Accept: application/json
```

Response

Response Status and Error Codes

Code	Reason	Description
200	OK	Branch was checked out successfully
400	Bad Request	Could not check out a branch due to the values passed in the request body

Response Payload Example

Sample Code

```
{
    "d": {
        "__metadata": {
            "id": "https://host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
Pull(guid'UUID')",
            "uri": "https://host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
Pull(guid'UUID')",
            "type": "cds_sd_a4c_a2g_gha_sc_web_api.PullType"
        },
        "uuid": "UUID ",
        "sc_name": "/DMO/GIT_REPOSITORY ",
        "namespace": "",
        "status": "R",
        "status_descr": "Running",
        "start_time": "/Date(1571227437000+0000)/",
        "change_time": "/Date(1571227472000+0000)/",
        "criticality": 2,
        "user_name": "CC0000000001",
        "to_Execution_log": {
            "__deferred": {
                "uri": "https://host.com/sap/opu/odata/sap/
MANAGE_GIT_REPOSITORY/Pull(guid'UUID')/to_Execution_log"
            }
        },
        "to_Transport_log": {
            "__deferred": {
                "uri": "https://host.com/sap/opu/odata/sap/
MANAGE_GIT_REPOSITORY/Pull(guid'UUID')/to_Transport_log"
            }
        }
    }
}
```

4.2.13.2.15.2 Clones

Resource path: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Clones

Operations

CRUD Operations

HTTP Method	Description
GET	Read Clones [page 1195]
POST	Create Clone [page 1197]

Parameterss

Property	Description
uuid	ID of the job
sc_name	Name of the software component
branch_name	Name of the branch
commit_id	Commit ID
import_type	Type of import
status	Status of the job
status_descr	Status description
user_name	Name of the user who triggered the action
start_time	Start time of the job
change_time	Last update of the job

4.2.13.2.15.2.1 Read Clones

Read a clone job. The entity is only available after the job has finished. To track the status of the job, you can use the GET /Pull(guid'UUID') request.

Request

URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Clones

Operation Type: CRUD

HTTP Method: GET

Request Headers

Header	Required	Values
Accept	no	application/json application/xml
x-csrf-token	no	fetch

Request Parameters

Parameter	Required	Data Type	Description	Parameter Type
sc_name	yes	string	name of the software component	query string
branch_name	yes	string	name of the branch	query string
uuid	yes	string	ID of the job	query string

Request Example

Sample Code

```
GET /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Clones(uuid=02409ac8-
dc6a-1edb-908e-31e4b44596d4',branch_name='main',sc_name='/DMO/
GIT_REPOSITORY') HTTP/1.1
Host: host.com
Authentication: basicAuthentication
Accept: application/json
```

Response

Response Headers

Header	Description
x-csrf-token	Token, which can be used for POST requests

Response Status and Error Codes

Code	Reason	Description
200	OK	Read clone succesfully
400	Bad Request	Could not read the clone entity. Please check the parameters.

Request Payload Example

Sample Code

```
{
    "d": {
        "__metadata": {
```

```
            "id": "https://4c7c0f11-
da7c-43cf-b60c-e6e308144458.abap.stagingaws.hanavlab.ondemand.com/sap/opu/
odata/sap/MANAGE_GIT_REPOSITORY/Clones(uuid=guid'02409ac8-
dc6a-1edb-908e-31e4b44596d4',sc_name='%2FDMO%2F
GIT_REPOSITORY,branch_name='main')",
            "uri": "https://4c7c0f11-
da7c-43cf-b60c-e6e308144458.abap.stagingaws.hanavlab.ondemand.com/sap/opu/
odata/sap/MANAGE_GIT_REPOSITORY/Clones(uuid=guid'02409ac8-
dc6a-1edb-908e-31e4b44596d4',sc_name='%2FDMO%2F
GIT_REPOSITORY,branch_name='main')",
            "type": "cds_sd_a4c_a2g_gha_sc_web_api.ClonesType"
        },
        "uuid": "02409ac8-dc6a-1edb-908e-31e4b44596d4",
        "sc_name": "/DMO/GIT_REPOSITORY",
        "branch_name": "main",
        "commit_id": "",
        "import_type": "Clone",
        "namespace": "",
        "status": "S",
        "status_descr": "Success",
        "criticality": 3,
        "user_name": "CC0000000001",
        "start_time": "/Date(1608214205000+0000)/",
        "change_time": "/Date(1608214243000+0000)/"
    }
}
```


4.2.13.2.15.2.2 Create Clone

Create a clone job.

Request

URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Clones

Operation Type: CRUD

HTTP Method: POST

Request Headers

Header	Required	Values
Accept	no	application/json application/xml
Contentt-Type	no	application/json application/xml
x-csrf-token	no	fetch

Request Parameters

Parameter	Required	Data Type	Description	Parameter Type
sc_name	yes	string	name of the software component	Request body
branch_name	no	string	name of the branch	Request body
commit_id	no	string	commit ID	Request body

Request Example

Sample Code

```
POST/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Clones HTTP/1.1
Host: host.com
Authentication: basicAuthentication
X-csrf-token: xCsrfToken
Content-Type: application/json
Accept: application/json

{
        "sc_name" : "/DMO/GIT_REPOSITORY"
        "branch_name": "main"
}
```

Response

Response Status and Error Codes

Code	Reason	Description
201	Created	Created a clone job
400	Bad Request	Could not read the clone job. Please check the parameters.

Request Payload Example

Sample Code

```
{
    "d": {
            {
                "__metadata": {
                    "id":
"host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Clones(uuid=guid'02ceab7d-
ff04-1eda-b1ea-eb7fdbf28bc4',sc_name='ZHM_INITIAL_TEST',branch_name='')",
                    "uri":
"host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Clones(uuid=guid'02ceab7d-
ff04-1eda-b1ea-eb7fdbf28bc4',sc_name='ZHM_INITIAL_TEST',branch_name='')",
                    "type": "cds_sd_a4c_a2g_gha_sc_web_api.ClonesType"
                },
                "uuid": "02ceab7d-ff04-1eda-b1ea-eb7fdbf28bc4",
                "sc_name": "/DMO/GIT_REPOSITORY",
                "branch_name": "",
                "import_type": "Clone",
```

```
                "namespace": "",
                "status": "S",
                "status_descr": "Success",
                "criticality": 3,
                "user_name": "administrator@example.com",
                "start_time": "/Date(1594898863000+0000)/",
                "change_time": "/Date(1594898891000+0000)/"
            }

    }
}
```


4.2.13.2.15.3 Execution Log

Resource path: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/ExecutionLogs

Operations

CRUD Operations

HTTP Method	Description
GET	Read Execution Log [page 1199]

Parameters

Property	Description
uuid	ID of the job
index_no	Index number of the log entry
type	Type of the log (information, warning, etc...)
descry	Description text of this log entry
timestamp	Timestamp of the log entry

4.2.13.2.15.3.1 Read Execution Log

Read the execution log of a specific job.

Request

URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/to_Execution_log

Operation Type: CRUD

HTTP Method: GET

Request Headers

Header	Required	Values
Accept	no	application/json application/xml
x-csrf-token	no	fetch

Request Parameters

Parameter	Required	Data Type	Description	Parameter Type
uuid	yes	string	ID of the job	query string

Request Example

Sample Code

```
GET /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Pull(uuid=02409ac8-
dc6a-1edb-908e-31e4b44596d4')/to_Execution_log HTTP/1.1
Host: host.com
Authentication: basicAuthentication
Accept: application/json
```

Response

Response Headers

Header	Description
x-csrf-token	Token, which can be used for POST requests

Response Status and Error Codes

Code	Reason	Description
200	OK	Read execution log succesfully
400	Bad Request	Could not find an execution log with the specified UUID.

Request Payload Example

Sample Code

```
{
    "d": {
        "results": [ {
            "__metadata": {
```

```
                "id": "host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
ExecutionLogs(index_no=1m,uuid=guid'UUID')", "uri": "host.com/sap/opu/
odata/sap/MANAGE_GIT_REPOSITORY/ExecutionLogs(index_no=1m,uuid=guid'UUID')",
"type": "cds_sd_a4c_a2g_gha_sc_web_api.ExecutionLogsType"
            }
            ,
            "index_no": "1",
            "uuid": "UUID",
            "type": "Information",
            "descr": "Step X: MESSAGE",
            "timestamp": "/Date(1571227438000+0000)/",
            "criticality": 0,
        }
        ]
    }
}
```


4.2.13.2.15.4 Pull

Resource path: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Pull

Operations

CRUD Operations

HTTP Method	Description	Operation
GET	Read pull	
POST	Trigger a pull job	

Custom Operations

HTTP Method	Description	Operation	URI
GET	Associationl		/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY /Pull(guid'UUID')/to_Executiong_log
GET	Associationl		/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY /Pull(guid'UUID')/to_Transport_log

Parameters

Property	Description
uuid	ID of the job
sc_name	Name of the software component

Property	Description
branch_name	Name of the branch
commit_id	Commit ID
import_type	Type of import
status	Status of the job
status_descr	Status description
user_name	Name of the user who triggered the action
start_time	Start time of the job
change_time	Last update of the job

4.2.13.2.15.4.1 Read Pull

Read a pull jon.

Request

URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Pull

Operation Type: CRUD

HTTP Method: GET

Request Headers

Header	Required	Values
Accept	no	application/json application/xml
x-csrf-token	no	fetch

Request Parameters

Parameter	Required	Data Type	Description	Parameter Type
uuid	yes	string	ID of the job	query string

Request Example

Sample Code

```
GET /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Pull(uuid=02409ac8-
dc6a-1edb-908e-31e4b44596d4') HTTP/1.1
Host: host.com
```

```
Authentication: basicAuthentication
Accept: application/json
```

Response

Response Headers

Header	Description
x-csrf-token	Token, which can be used for POST requests

Response Status and Error Codes

Code	Reason	Description
200	OK	Read pull succesfully
400	Bad Request	Could not find a pull entity with the specified UUID.

Request Payload Example

Sample Code

```
{
    "d": {
        "__metadata": {
            "id": "https://host.com/sap/opu/odata/sap/
MANAGE_GIT_REPOSITORY/Pull(guid'UUID')", "uri": "https://host.com/sap/opu/
odata/sap/MANAGE_GIT_REPOSITORY/Pull(guid'UUID')", "type":
"cds_sd_a4c_a2g_gha_sc_web_api.PullType"
        }
        ,
        "uuid": "UUID ",
        "sc_name": "/DMO/GIT_REPOSITORY ",
        "namespace": "",
        "status": "R",
        "status_descr": "Running",
        "start_time": "/Date(1571227437000+0000)/",
        "change_time": "/Date(1571227472000+0000)/",
        "criticality": 2,
        "user_name": "CC0000000001",
        "to_Execution_log": {
            "__deferred": {
                "uri": "https://host.com/sap/opu/odata/sap/
MANAGE_GIT_REPOSITORY/Pull(guid'UUID')/to_Execution_log"
            }
        }
        ,
        "to_Transport_log": {
            "__deferred": {
                "uri": "https://host.com/sap/opu/odata/sap/
MANAGE_GIT_REPOSITORY/Pull(guid'UUID')/to_Transport_log"
            }
        }
    }
}
```

4.2.13.2.15.4.2 Create Pull

Create a pull job.

Request

URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Pull

Operation Type: CRUD

HTTP Method: POST

Request Headers

Header	Required	Values
Accept	no	application/json application/xml
Content-Type	no	application/json application/xml
x-csrf-token	yes	Value of x-csrf-token

Request Parameters

Parameter	Required	Data Type	Description	Parameter Type
sc_name	yes	string	name of the software component	Request body
commit_id	no	string	commit ID	Request body

Request Example

Sample Code

```
POST/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Pull HTTP/1.1
Host: host.com
Authentication: basicAuthentication
X-csrf-token: xCsrfToken
Content-Type: application/json
Accept: application/json
{
    "sc_name" : "/DMO/GIT_REPOSITORY"
}
```

Response

Response Status and Error Codes

Code	Reason	Description
201	Created	Created a pull job
400	Bad Request	Could not read the pull job. Please check the parameters.

Request Payload Example

Sample Code

```
{
    "d": {
        "__metadata": {
            "id": "https://host.com/sap/opu/odata/sap/
MANAGE_GIT_REPOSITORY/Pull(guid'UUID')", "uri": "https://host.com/sap/opu/
odata/sap/MANAGE_GIT_REPOSITORY/Pull(guid'UUID')", "type":
"cds_sd_a4c_a2g_gha_sc_web_api.PullType"
        }
        ,
        "uuid": "UUID ",
        "sc_name": "/DMO/GIT_REPOSITORY ",
        "namespace": "",
        "status": "R",
        "status_descr": "Running",
        "start_time": "/Date(1571227437000+0000)/",
        "change_time": "/Date(1571227472000+0000)/",
        "criticality": 2,
        "user_name": "CC0000000001",
        "to_Execution_log": {
            "__deferred": {
                "uri": "https://host.com/sap/opu/odata/sap/
MANAGE_GIT_REPOSITORY/Pull(guid'UUID')/to_Execution_log"
            }
        }
        ,
        "to_Transport_log": {
            "__deferred": {
                "uri": "https://host.com/sap/opu/odata/sap/
MANAGE_GIT_REPOSITORY/Pull(guid'UUID')/to_Transport_log"
            }
        }
    }
}
```

4.2.13.2.15.5 Transport Log

Resource path: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/TransportLogs

Operations

CRUD Operations

HTTP Method	Operation
GET	Read Transport Log [page 1206]

Parameters

Property	Description
uuid	ID of the job
index_no	Index number of the log entry
type	Type of the log (information, warning, etc...)
descry	Description text of this log entry
timestamp	Timestamp of the log entry

4.2.13.2.15.5.1 Read Transport Log

Request

URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/to_Transport_log

Operation Type: CRUD

HTTP Method: GET

Request Headers

Header	Required	Values
Accept	no	application/json application/xml
x-csrf-token	no	fetch

Request Parameters

Parameter	Required	Data Type	Description	Parameter Type
uuid	yes	string	ID of the job	query string

Request Example

Sample Code

```
GET /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Pull(uuid=02409ac8-
dc6a-1edb-908e-31e4b44596d4')/to_Transport_log HTTP/1.1
Host: host.com
Authentication: basicAuthentication
Accept: application/json
```

Response

Response Headers

Header	Description
x-csrf-token	Token, which can be used for POST requests

Response Status and Error Codes

Code	Reason	Description
200	OK	Read transport log succesfully
400	Bad Request	Could not find a transport log with the specified UUID.

Request Payload Example

Sample Code

```
{ {
    "d": {
        "results": [ {
            "__metadata": {
                "id": "host.com/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
ExecutionLogs(index_no=1m,uuid=guid'UUID')", "uri": "host.com/sap/opu/
odata/sap/MANAGE_GIT_REPOSITORY/ExecutionLogs(index_no=1m,uuid=guid'UUID')",
"type": "cds_sd_a4c_a2g_gha_sc_web_api.ExecutionLogsType"
            }
            ,
            "index_no": "1",
            "uuid": "UUID",
            "type": "Information",
            "descr": "Step X: Message",
            "timestamp": "/Date(1571227438000+0000)/",
            "criticality": 0,
        }
        ]
    }
}
```

4.2.13.2.15.6 Tags

Resource path: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Tags

Operations

CRUD Operations

HTTP Method	Operation
GET	Read Entities from Tag [page 1208]
POST	Create Tag [page 1209]
DELETE	Delete Tag [page 1212]

Custom Operations

HTTP Method	Operation Type	Operation	URI
POST	Action	Read Tags by Key [page 1211]	/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Tags(sc_name='{sc_name}',commit_id='{commit_id}',tag_name='{tag_name}')

4.2.13.2.15.6.1 Read Entities from Tag

Read entities from tags.

Request

URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Tags

Operation Type: CRUD

HTTP Method: GET

Request Headers

Header	Required	Values
Accept	no	application/json application/xml
x-csrf-token	no	fetch

Request Parameters

No parameters.

Request Example

Sample Code

```
GET /sap/opu/odata/sap/Tags
Host: host.com
Authentication: basicAuthentication
Accept: application/json
```

Response

Response Headers

Header	Description
x-csrf-token	Token, which can be used for POST requests

Response Status and Error Codes

Code	Reason	Description
200	OK	Branch was read successfully
404	Not foumd	Could not read branch; software component or branch may not exist..

Request Payload Example

Sample Code

4.2.13.2.15.6.2 Create Tag

Create a new tag.

Request

URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Tags

Operation Type: CRUD

HTTP Method: POST

Request Headers

Header	Required	Values
Accept	no	application/json application/xml
Content-Type	no	application/json application/xml
x-csrf-token	yes	Value of x-csrf-token

Request Parameters

No parameters.

Request Body

Parameter	Required	Data Type	Description	Parameter Type
sc_name	yes	string	name of the software component	Request body
commit_id	yes	string	long commit id	Request body
tag_name	yes	string	name of the tag	Request body

Sample Code

```
POST/sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Tags
HTTP/1.1
Host: host.com
Authentication: basicAuthentication
X-csrf-token: xCsrfToken
Content-Type: application/json
Accept: application/json
    {
  "sc_name": "string",
  "commit_id": "string",
  "tag_name": "string",
  "tag_description": "string",
  "tag_created_by": "string",
  "tag_created_on": "/Date(1492098664000)/",
  "short_commit_id": "string",
  "message": "string",
  "author": "string",
  "author_mail": "string"
}
```

Response

Response Status and Error Codes

Code	Reason	Description
201	Created	Tag was created successfully
400	Bad Request	Could not create a tag due to the values passed in the request body.

Request Payload Example

Sample Code

4.2.13.2.15.6.3 Read Tags by Key

Read entities from tags.

Request

URI: : /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Tags(sc_name='{sc_name}',commit_id='{commit_id}',tag_name='{tag_name}')

Operation Type: CRUD

HTTP Method: GET

Request Headers

Header	Required	Values
Accept	no	application/json application/xml
x-csrf-token	no	fetch

Request Parameters

Parameter	Required	Data Type	Description	Parameter Type
sc_name	yes	string	name of the software component	query string
commit_id	yes	string	long commit id	query string

Parameter	Required	Data Type	Description	Parameter Type
tag_name	yes	string	name of the tag	query string

Request Example

Sample Code

```
GET /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
Tags(sc_name='{sc_name}',commit_id='{commit_id}',tag_name='{tag_name}')
HTTP/1.1
Host: host.com
Authentication: basicAuthentication
Accept: application/json
```

Response

Response Headers

Header	Description
x-csrf-token	Token, which can be used for POST requests

Response Status and Error Codes

Code	Reason	Description
200	OK	Tag was read successfully
404	Not foumd	Could not read tag; software component or commit id may not exist..

Request Payload Example

Sample Code

4.2.13.2.15.6.4 Delete Tag

Delete a branch.

Request

URI: /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/
Tags(sc_name='{sc_name}',commit_id='{commit_id}',tag_name='{tag_name}')

Operation Type: CRUD

HTTP Method: DELETE

Request Headers

Header	Required	Values
Accept	no	application/json application/xml
x-csrf-token	yes	Value of x-csrf-token

Request Body

Parameter	Required	Data Type	Description	Parameter Type
sc_name	yes	string	name of the software component	query string
commid_id	yes	string	long commit id	query string
tag_name	yes	string	name of the branch	query string

Request Example

Sample Code

```
DELETE /sap/opu/odata/sap/MANAGE_GIT_REPOSITORY/Branches(branch_name='my-to-be-deleted-branch',sc_name='/DMO/GIT_REPOSITORY') HTTP/1.1
Host: host.com
Authentication: basicAuthentication
Accept: application/json
```

Response

Response Headers

Sample Code

```
HTTP/1.1 204 No Content
```

Response Status and Error Codes

Code	Reason	Description
204	No content	Tag was deleted successfully
400	Bad Request	Could not delete tag due to the values passed in the request body.

4.2.13.3 Integrating On-Premise Systems

Set up the Cloud Connector to enable communication from the ABAP environment to your on-premise systems using Remote Function Calls (RFC) and HTTP calls.

Concept

For each subaccount, a Connectivity service instance is set up automatically in the SAP provider account. To use this instance, activate the Cloud Connector in your communication system.

In this scenario, you must connect the Cloud Connector to and configure trust for your Cloud Foundry subaccount as described in SAP BTP Connectivity: Cloud Connector.

> ⚠ Restriction
>
> This scenario requires a Cloud Connector as of version 2.12.3 or higher.

> ⓘ Note
>
> For more information on the deprecated scenario that uses the Connectivity service in the Neo environment, see Create a Communication Arrangement for Cloud Connector Integration (Deprecated) [page 1216].

After you have completed the setup, a connection from the ABAP environment tenant to an on-premise system is established in the following order:

1. The ABAP environment tenant requests to open the tunnel connection through the Connectivity service.
2. The Connectivity service tells the Cloud Connector to open the connection to this specific ABAP environment tenant using the admin connection.
3. The Cloud Connector opens a tunnel connection to the ABAP environment tenant using its public tenant URL.
4. After the tunnel is established, it can be used for actual data connection using RFC and HTTP(S) protocols.

Next Steps

To set up your actual data connections between your ABAP environment and on-premise systems, you must enable the required Cloud Connector in your communication system. See Enable On-Premise Connectivity [page 1215].

Troubleshooting

In case of issues using the Cloud Connector, see Cloud Connector Troubleshooting for a troubleshooting guide.

4.2.13.3.1 Enable On-Premise Connectivity

Create an HTTP and an RFC destination to enable communication from the ABAP environment to your on-premise systems.

Prerequisites

- You have installed Cloud Connector version 2.12.3 or higher, see Cloud Connector: Installation.
- You have done the initial configuration for the Cloud Connector, see Cloud Connector: Initial Configuration.
- If you want to use principal propagation as authentication type, the Cloud Connector must be configured to support this authentication type. See Cloud Connector: Configuring Principal Propagation.
  Also, make sure that no communication arrangement to the deprecated communication scenario `SAP_COM_0200` exists.
- If you use more than one Cloud Connector in your subaccount, you have assigned a location ID to each of these Cloud Connectors. See Managing Subaccounts.

Procedure

To enable on-premise connectivity, you must use the Cloud Connector in your communication system:

- Creating a Communication System for Outbound HTTP
- Creating a Communication System for Outbound RFC

Related Information

Outbound Communication
Communication System

4.2.13.3.2 Create a Communication Arrangement for Cloud Connector Integration (Deprecated)

Learn how to create a communication arrangement for communication scenario `SAP_COM_0200` to integrate the Cloud Connector.

Prerequisites

> **Note**
>
> This procedure is deprecated and is only available for systems provisioned before May 2022. It was replaced with a scenario that does not require a Neo subaccount. To learn about the steps to migrate to a Cloud Foundry-only usage, see section **Next Steps** below.

To set up your ABAP environment, you have to create a communication arrangement for communication scenario `SAP_COM_0200 - Cloud Connector Integration`, that requires the following:

- An administrative user in the ABAP environment tenant.
- Increased quota for the ABAP runtime.
- An ABAP service instance set up in Cloud Foundry.
- Your Neo subaccount name.
- The name of the region host of your Neo subaccount

  > **Note**
  >
  > Please check out the supported Neo subaccounts for on-premise connectivity of the ABAP environment. See SAP Note 2765161.

- Installation of Cloud Connector version 2.11 or higher. See Cloud Connector: Installation
- Initial configuration for Cloud Connector and the Neo subaccount. See Cloud Connector: Initial Configuration

  > **Note**
  >
  > The Cloud Connector is connected to the Neo subaccount and is displayed in the Cloud Cockpit, section *Connectivity* under *Cloud Connectors*.

- User credentials for the Cloud Connector admin role in the Neo subaccount

Procedure

1. To create a communication system that represents your Neo subaccount, open the *Communication Systems* app from the SAP Fiori launchpad in your Cloud Foundry subbaccount and select *New*.
2. Provide a meaningful system name and choose *Create* to proceed.

> **Note**
>
> You have to disable the destination service first by using the *on/off* slider button.

Enter the following information:

Field	Action
Host Name	Enter the name of the region host of your Neo subaccount. If your Neo URL is https://account.xyz.hana.ondemand.com, you need to enter **`xyz.hana.ondemand.com`**, where **`xyz`** represents the region name. See Regions and Hosts Available for the Neo Environment.
HTTPS Port	Enter **`443`**.
Checkbox *Use Cloud Connector*	Make sure this checkbox is **not checked** .
User for Outbound Communication	a) Scroll down to this section. b) Add a new user. c) In the dropdown list for *Authentication Method*, select a user name and password. d) Enter the user name and password of the Cloud Connector admin user of the Neo subaccount.

For more information on how to create a communication system, see How to Create Communication Systems.

3. Save the communication system to activate it.
4. To create a communication arrangement that activates the connection to the Neo subaccount, open the *Maintain Communication Arrangements* app from the SAP Fiori Launchpad and click *New*.
5. Select scenario `SAP_COM_0200` and enter a meaningful arrangement name.
6. Click *Create* to proceed.

   Enter the following information:

Field	Action
Communication System	Enter the name of the communication system created in steps 1-3, or select it from the value help.
Additional Properties > *Account Name*	Enter the name of your Neo subaccount.
Outbound Communication > *User Name*	

   For more information on how to create a communication arrangement, see How to Create a Communication Arrangement [page 2998].

7. Save the communication arrangement to activate it.

8. Carry out the initial configuration for Cloud Connector and the Neo subaccount to make it operational for connections between your Neo subaccount and the on-premise systems.

Results

You have associated the ABAP environment tenant with the Neo subaccount. This enables the ABAP environment tenant to request the Neo environment to open a tunnel connection.

Next Steps

After completing the setup of communication scenario `SAP_COM_0200`, you are ready to set up your actual data connections between your ABAP environment and on-premise systems. This requires the configuration of other communication systems and communication arrangements in the ABAP environment tenant as well as the configuration of Cloud Connector. See Enable On-Premise Connectivity [page 1215].

To **migrate** to the usage of the **Cloud Foundry Connectivity service**, proceed as follows:

1. Configure trust in the Cloud Connector for the Cloud Foundry subaccount in which the ABAP instance resides.
2. Delete communication arrangement `SAP_COM_0200`.
3. Destinations do not need to be modified if the same `LocationID` is used.

4.2.13.4 Managing the SAP Business AI Platform, ABAP environment Service Instance

In this chapter, you learn how to create configurations as an administrator for different integration use cases.

Create a Destination [page 1219]
: If your business application uses external services, you have to set up a destination for outbound communication either in your subaccount, which is recommended, or in your space.

Create a Communication Arrangement for Inbound Communication with Service Key Type Basic [page 1220]
: Learn how to quickly create a communication user and communication arrangement for an inbound communication scenario by using a service key of type basic.

Configure Timeout of @sap/approuter Component of a Communication Arrangement for Inbound Communication with Service Key OAuth [page 1221]
: By default, communication between the approuter and connected services is timed out after 30 seconds. This default value is suitable for most applications. However, for long-lasting data transfer, communication lasts longer. If you experience timeouts, you can set the service key parameter `abap_endpoint_timeout` to a value higher than 30 seconds.

4.2.13.4.1 Create a Destination

If your business application uses external services, you have to set up a destination for outbound communication either in your subaccount, which is recommended, or in your space.

Creating a Destination in Your Subaccount

1. Log on to the SAP BTP cockpit and navigate to your subaccount.
2. In the navigation area, go to *Connectivity* > *Destinations* and select *New Destination*.
3. Set up your destination and select *Save*. For more information, see Create Destinations from Scratch.

Creating a Destination in Your Space

> **Note**
>
> - You have created an ABAP service instance. See Creating an ABAP System.
> - You have created a destination service instance. See Creating Service Instances in Cloud Foundry.
> - You have created a service key. See Creating Service Keys in Cloud Foundry.

1. Log on to the SAP BTP and go to the subaccount that contains the space you'd like to navigate to. See Navigate in the Cockpit.
2. In the navigation area, go to *Cloud Foundry* > *Spaces* and select the space that contains the ABAP system service instance.
3. Select *Services* > *Instances* and navigate to your ABAP system service instance.
4. To open the administration launchpad, click on ••• and select *View Dashboard*.
5. If necessary, provide your logon credentials to access the administration launchpad.
6. In the *Communication Management* section, select the *Communication Arrangements* tile.
7. On the *Communication Arrangements* page, select *New*.
8. In the *New Communication Arrangement* dialog, use the value help to select scenario `SAP_COM_0276` and give the arrangement a meaningful name (e.g. the name of the destination service instance).
9. Enter the service key of your destination service instance and select *Create*.

> **Note**
>
> The process for connecting to an ABAP service instance URL when creating a new ABAP Cloud project in ABAP development tools for Eclipse has changed. For more information, see ABAP Service Instance URL [page 1881].

4.2.13.4.2 Create a Communication Arrangement for Inbound Communication with Service Key Type Basic

Learn how to quickly create a communication user and communication arrangement for an inbound communication scenario by using a service key of type basic.

Prerequisites

- You have created a communication scenario.
- You have created a space. See Create Spaces.
- You have the space developer role. See Assigning the Space Developer Role to the Developer Users.

Procedure

1. Log on to the cockpit and go to the subaccount that contains the space you'd like to navigate to. See Navigate in the Cockpit.
2. In the navigation area, go to *Cloud Foundry* > *Spaces* and choose your space.
3. Select *Services* > *Instances* from the navigation area and find your ABAP system service instance.
4. Click on ••• and select *Create Service Key*.
5. In the *New Service Key* dialog, enter a name for your service key and specify the parameters in JSON format as follows:

```
{
   "scenario_id":"SAP_COM_XYZ",
   "type":"basic"
}
```

> **Note**
>
> `SAP_COM_XYZ` is the ID of the communication scenario in your ABAP system.
>
> `basic` is the type of service key that is needed to generate a communication user and communication arrangement for an inbound communication scenario.

6. **(Optional to step 5)** If your communication scenario has additional custom properties, you can automatically assign values to them by using the optional parameter `additional_properties`.

```
{
   "scenario_id":"SAP_COM_XYZ",
   "type":"basic",
   "additional_properties": [
            {
                  "technical_name": "CUSTOM_PROPERTY_1",
                  "value": "VALUE_1"
            },
            {
                  "technical_name": "CUSTOM_PROPERTY_2",
```

```
                "value": "VALUE_2"
            }
    ]
}
```

> **Note**
>
> `CUSTOM_PROPERTY_1` and `CUSTOM_PROPERTY_2` are the technical names of the properties as specified in your communication scenario. You can add any number of additional properties to the array `additional_properties` (`CUSTOM_PROPERTY_3`, `CUSTOM_PROPERTY_4`, and so on).
>
> `VALUE_1`, `VALUE_2`, and so on are the values that should be assigned to the properties.

Results

The communication user is generated and the credentials, depending on the selected type, are returned in the service key. The communication user receives authorizations for all services included in the communication scenario.

4.2.13.4.3 Configure Timeout of @sap/approuter Component of a Communication Arrangement for Inbound Communication with Service Key OAuth

By default, communication between the approuter and connected services is timed out after 30 seconds. This default value is suitable for most applications. However, for long-lasting data transfer, communication lasts longer. If you experience timeouts, you can set the service key parameter `abap_endpoint_timeout` to a value higher than 30 seconds.

Prerequisites

- You have created a space. See Create Spaces.
- You have the space developer role. See Assigning the Space Developer Role to the Developer Users.

Procedure

1. Log on to the cockpit and go to the subaccount that contains the space you'd like to navigate to. See Navigate in the Cockpit.
2. In the navigation area, go to *Cloud Foundry* > *Spaces* and choose your space.
3. Select *Services* > *Instances* from the navigation area and navigate to your ABAP system service instance.

4. Click on ••• and select *Create Service Key*.
5. In the *New Service Key* dialog, enter a name for your service key and specify the parameter in JSON format as follows:

```
{
      "abap_endpoint_timeout": 600000
}
```

> ⓘ Note
>
> You can use s (seconds) or m (minutes) to define the value.
>
> If no unit is defined, the value is interpreted as seconds.
>
> The maximum value is 600s or 10m.

Related Information

Integration with Business Services

4.2.13.5 Integrating the ABAP Environment with SAP S/4HANA Cloud

You can integrate the SAP Business AI Platform, ABAP environment with an SAP S/4HANA Cloud Public Edition system. This allows ABAP developers to implement outbound service calls from the ABAP environment to the SAP S/4HANA Cloud Public Edition system.

Context

Developers in SAP Business AI Platform, ABAP environment would like to access data from business services in SAP S/4HANA Cloud Public Edition as part of their applications.

This documentation provides you with a high-level description of how to integrate the two systems and how to implement service calls from SAP Business AI Platform, ABAP environment to SAP S/4HANA Cloud Public Edition. This is complemented by a tutorial group that provides a detailed, step-by-step description based on an example.

Overview of Implementation Activities

To set up remote connectivity it is necessary to perform some steps in both involved systems, as the connection must be enabled by both communication partners.

In the scenario that we are considering, an SAP S/4HANA Cloud Public Edition service is called from SAP Business AI Platform, ABAP environment. In other words, SAP S/4HANA Cloud Public Edition is the *inbound* communication partner, while SAP Business AI Platform, ABAP environment is the *outbound* communication partner.

Following, you can find an overview of necessary steps, grouped by the executing persona.

Developer in SAP Business AI Platform, ABAP environment

- *Create a communication scenario*
  As a developer you create an outbound service that represents the service of an SAP S/4HANA Cloud Public Edition system. You add this outbound service to a communication scenario and maintain the supported authentication method.
- *Create a service consumption model*
  As a developer you can use the metadata of a given service to create a service consumption model. Although not strictly necessary, this greatly simplifies the consumption of the remote service from your ABAP code.
- *Implement the application*
  The actual service call itself needs to be implemented. This can be part of a frontend application, API service, application job, console application, etc. The communication artifacts and the service consumption model are reused to create compact and understandable code.

Administrator in SAP S/4HANA Cloud Public Edition

- *Expose the service*
  As administrator you need to make the required service available for remote access. To do so, you create a communication arrangement for a scenario containing the API as an inbound service. Along with the arrangement, you define a communication system and an inbound communication user that shall be used to establish the connection.

Configure the outbound communication

As administrator you create a communication arrangement based on the scenario implemented by the developer. The arrangement is tied to a communication system that identifies the SAP S/4HANA Cloud Public Edition system as the target host.

Tutorial group Integrate the SAP BTP ABAP environment with SAP S/4HANA Cloud, public edition shows how to consume the business partner OData API service in SAP S/4HANA Cloud Public Edition from a console application in SAP Business AI Platform, ABAP environment using basic authentication (communication user) and OAuth 2.0 SAML Bearer Assertion (propagation of business user).

4.2.13.6 Developing Metrics for Health Monitoring

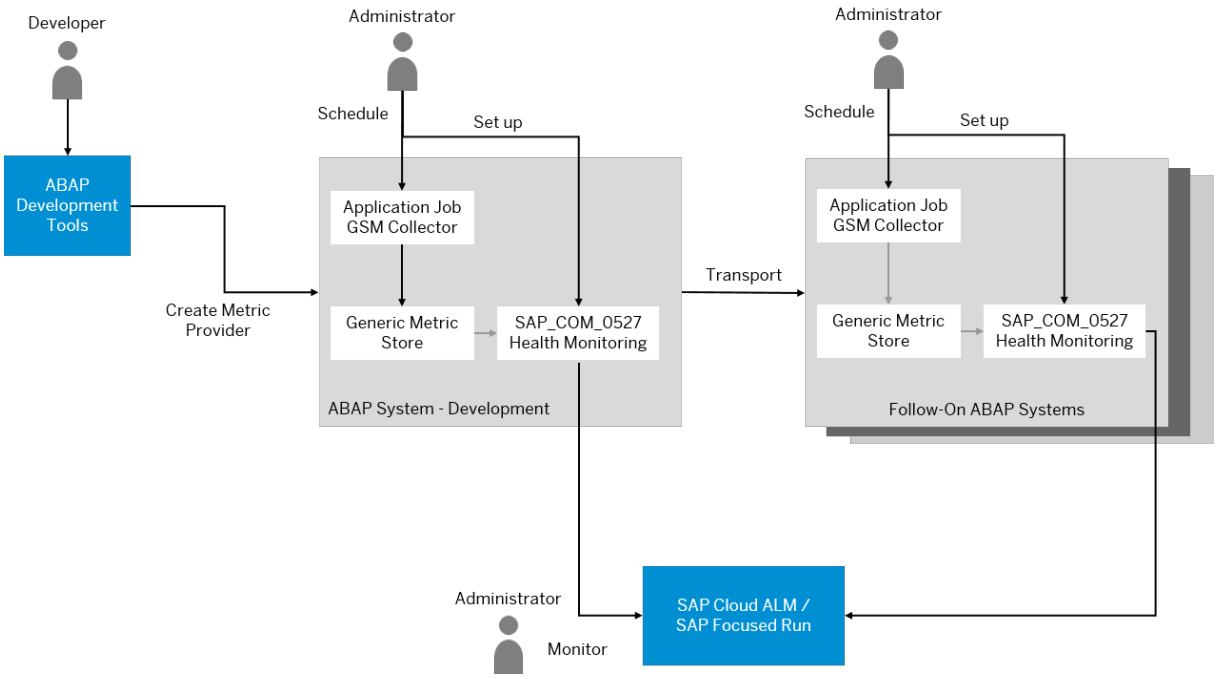
Learn how you can add your own metrics to Health Monitoring in SAP Cloud ALM or SAP Focused Run.

Use Case

SAP Cloud ALM provides you with insights regarding the health of Cloud services. In the overview dashboard, all health monitoring metrics are exposed, and for each metric, a detailed history is available. While a number

of metrics defined by SAP are available for checking customer-initiated operational issues, you might want to add your own application-specific metrics. These metrics serve to monitor the health of your own customer or partner solutions running on the ABAP environment. Similarly, you might want to have your own metrics in Health Monitoring of SAP Focused Run.

Architecture Overview



The central object in this context is the metric provider, an ABAP repository object that references an implementing class. The class defines a metric model for your application and implements the metric measurement. The metric model defines, for example, what kind of metrics are available in the system, how often a metric is measured, its tags, or its unit of measurement.

An application job (the generic simple metric collector) runs regularly. It calls up the `GET_METRIC_VALUES( )` method of all metric providers and stores all measured values as metrics in a generic format in the metric store. After an administrator has set up a communication arrangement based on the communication scenario `SAP_COM_0527`, the metrics are automatically pushed from the generic metric store to SAP Cloud ALM and SAP Focused Run.

About This Documentation

In this documentation, you get information about creating your own metric providers as a developer. In addition, there's also a short information for administrators about creating an application job that calls up all created metric providers and transfers their measured values to the generic metric store.

For more information about setting up SAP Cloud ALM and SAP Focused Run, see Central Health Monitoring Using SAP Focused Run and SAP Cloud ALM [page 3411].

4.2.13.6.1 Creating a Class for Metric Providers

To create a metric model for your customer or partner application, create a class in ABAP Development Tools (ADT) that implements the interface `IF_GSM_API_PROVIDER`.

Prerequisites

You have a developer role for creating objects with ABAP for Cloud Development.

You have created an ABAP Cloud project and a package in ABAP Development Tools.

Context

With the class, you define the metric model and measurement.

> ⓘ Note
>
> For code samples, see https://github.com/SAP-samples/abap-platform-ops .

Procedure

1. In your ABAP Cloud project and package, create a class implementing the interface `IF_GSM_API_PROVIDER`.

   As a result, you get a class with the following methods: `INITIALIZE( )`, `DEFINE_MODEL( )`, and `GET_METRIC_VALUES( )`.
2. When you edit this class, we recommend that you define constants for the metric names and values that are exactly the same for methods `DEFINE_MODEL( )` and `GET_METRIC_VALUES( )`. All metric names that do not match are ignored at runtime.

   For example, your code might look as follows:

   > {·} Sample Code

```
    CONSTANTS:
      BEGIN OF e_metric_group_id,
        demo_group TYPE if_gsm_api_types=>tv_metric_group_id VALUE
'zdemo_group_01',
      END OF e_metric_group_id .
    CONSTANTS:
      BEGIN OF e_metric_id,
        demo_metric_id TYPE if_gsm_api_types=>tv_metric_id VALUE
'zdemo_metric_01',
      END OF e_metric_id .
    CONSTANTS:
      BEGIN OF e_attribute_id,
```

```
        demo_attr_01 TYPE if_gsm_api_types=>tv_attribute_id VALUE
'zdemo_attr_01',
      END OF e_attribute_id .
```

You create one metric group and assign your metrics to the group. The metric group is just for semantical grouping and holder for meta data. It has no physical impact on the metric store or the data collection for metrics later.

3. Redefine the methods `DEFINE_MODEL( )` and `GET_METRIC_VALUES( )` according to your business needs.

Related Information

Creating a Metric Model [page 1226]
Defining the Metric Measurement [page 1227]

4.2.13.6.1.1 Creating a Metric Model

In the `DEFINE_MODEL( )` method, you specify your metric model, including the metric IDs, their data types, or their units of measurement.

Prerequisites

You've created a class for metric providers (see Creating a Class for Metric Providers [page 1225]).

Procedure

In the class that you've created, change the `DEFINE_MODEL( )` method according to your business needs.

For example, your method might be as follows:

Sample Code

```
METHOD if_gsm_api_provider~define_model.
    DATA: lo_api_metric_group TYPE REF TO if_gsm_api_metric_group.
    lo_api_metric_group = io_model->add_metric_group(
      iv_metric_group_id   = me->e_metric_group_id-demo_group
      iv_metric_group_name = `Demo Client API Provider` ##NO_TEXT
      iv_category          = if_gsm_api_constants=>e_category-performance
      iv_severity          = if_gsm_api_constants=>e_severity-_2_low ).
    lo_api_metric_group->add_target(
      iv_target_type       = if_gsm_api_constants=>e_target_type-health ).
    lo_api_metric_group->add_metric(
      iv_metric_id         = me->e_metric_id-demo_metric_id
      iv_metric_name       = `Demo Client API Metric` ##NO_TEXT
```

```
      iv_unit              = if_gsm_api_constants=>e_unit-seconds
      iv_data_type         = if_gsm_api_constants=>e_data_type-integer
      iv_period            = if_gsm_api_constants=>e_period-every_05_minutes
      iv_description       = `Just a demo`  ##NO_TEXT
      iv_metric_type       = if_gsm_api_constants=>e_metric_type-counter )-
>add_target(
      iv_target_type       = if_gsm_api_constants=>e_target_type-health ).
  ENDMETHOD.
```

Notes: For all input parameters, you can find the relevant enumerations in the released interface `IF_GSM_API_CONSTANTS`. Note, however, that there are a few additional considerations:

- The category of the metric group expresses the semantic nature of the metric, for example, whether the availability or performance of the system is affected.
- The attributes of a metric refer to the consumption of the metric:
  - **Period**: The period specifies the frequency with which the metric measurement is triggered. It implicitly derives a runtime quota. Metric providers that are executed more often must return the metrics faster. For example, metric providers running each minute must return values within 5 seconds. Otherwise, they will be banned by the framework in the future.
    The `period` attribute is the only control attribute. All other attributes don't have any runtime impacts.
  - **Metric type**: Metrics can be collected in different ways. Some metrics are of type *Counter*. A counter always goes up and is only reset at special events like system restart. For example, the number of processed requests is typically expressed as a counter. In contrast to a counter, a gauge metric can go up or down. The temperature or current memory usage are common examples of gauge metrics.
  - **Target type**: The target type is used to control the metric flow for different usage scenarios. Only the available target type `HEALTH` is supported. This target type registers the metric to be exported to Health Monitoring in SAP Cloud ALM and SAP Focused Run.
  - **Severity**: The severity is available, but not pushed to Health Monitoring.

4.2.13.6.1.2 Defining the Metric Measurement

In the `GET_METRIC_VALUES( )` method, you specify how the metrics are measured, including the requested metrics and the time frame for processing.

Prerequisites

You've created a class for metric providers (see Creating a Class for Metric Providers [page 1225]).

Context

At runtime, the generic metric provider framework calls the method `GET_METRIC_VALUES( )`. The method expects multiple input parameters like the requested metrics and the time frame for processing. If you include multiple metrics with different collection frequencies, the framework calls this method accordingly with the

corresponding metric IDs for which it requests to return data. When you provide the correct input in the `GET_METRIC_VALUES( )` implementation, you avoid wasting work for metrics not being requested.

The measurement should be fast. If a measurement takes too much time, it blocks all the other measurements. Therefore, the metric provider framework automatically monitors the runtime of each metric provider. If a metric provider exceeds the runtime quota, the framework will ban the metric provider so that all the other metric providers can be collected.

Procedure

In the class that you've created, change the `GET_METRIC_VALUES( )` method according to your business needs.

For example, your method might be as follows:

Sample Code

```
METHOD if_gsm_api_provider~get_metric_values.
    DATA: ls_metric_value    TYPE if_gsm_api_types=>ts_metric_value.
    DATA: lv_timestamp       TYPE timestamp.
    GET TIME STAMP FIELD lv_timestamp.
    ls_metric_value-metric_id = me->e_metric_id-demo_metric_id.
    IF iv_timestamp_start IS SUPPLIED AND iv_timestamp_start <> 0.
      ls_metric_value-value_timestamp = iv_timestamp_start.
    ELSEIF iv_timestamp_end IS SUPPLIED AND iv_timestamp_end <> 0.
      ls_metric_value-value_timestamp = iv_timestamp_end.
    ELSE.
      ls_metric_value-value_timestamp = lv_timestamp.
    ENDIF.
    ls_metric_value-value_count  = 1.
    ls_metric_value-value_text   = ''.
    INSERT VALUE #(
      attribute_id = me->e_attribute_id-demo_attr_01
      value        = 'None' ##NO_TEXT
    ) INTO TABLE ls_metric_value-attributes
      REFERENCE INTO DATA(lsr_mp_attr_01).
    " metric data
    "
------------------------------------------------------------------------
    " Retrieval of the metric values at this point in time.
    DATA(lt_metrics) = me->mo_api_access->get_metrics( ).
    " transformation
    "
------------------------------------------------------------------------
    " Transform the metric values format to the GSM data structures.
    LOOP AT lt_metrics REFERENCE INTO DATA(lrs_metric).
      lsr_mp_attr_01->value     = lrs_metric->attribute.
      ls_metric_value-value_sum = lrs_metric->value.
      ls_metric_value-value_max = ls_metric_value-value_sum.
      ls_metric_value-value_min = ls_metric_value-value_sum.
      IF ls_metric_value-value_sum > 10.
        ls_metric_value-value_rating = if_gsm_api_constants=>e_rating-
_2_warning.
      ELSE.
        ls_metric_value-value_rating = if_gsm_api_constants=>e_rating-_1_ok.
      ENDIF.
      INSERT ls_metric_value INTO TABLE et_metric_values.
    ENDLOOP.
  ENDMETHOD.
```

4.2.13.6.2 Creating a Metric Provider

You can create a metric provider in ABAP Development Tools to make your own metrics available for health monitoring of your ABAP system.

Prerequisites

You've created a class implementing the interface `IF_GSM_API_PROVIDER`. This class defines your metric model.

Context

With the creation of a metric provider, you register your metrics that are defined in the class for a periodic collection of values.

> **Note**
>
> For more information about creating metric providers, see the documentation in ABAP Development Tools.

Procedure

1. To create a metric provider in ABAP Development Tools, choose *New* > *Other* > *ABAP Repository Object* > *Observability* > *Metric Provider* .
2. In the wizard, enter a package, name, and description.

   A metric provider is created.
3. In the metric provider, as implementing object, enter the class that defines your metric model for this metric provider (see Creating a Class for Metric Providers [page 1225]).
4. Under *Execution*, select the priority with which the values of this metric provider must be collected.

   > **Note**
   >
   > The execution mode is set to *Job*, which you can't change. The collection of metric values is executed by an application job that your administrator must schedule (see Collect Metric Provider Values (Administrator) [page 1231]). The application job runs regularly and collects the values of all metric providers using their `GET_METRIC_VALUES( )` methods.

   Set a higher priority for metric providers that you consider as more important than others. A higher priority ensures that the values for these metric providers are collected first by the application job. Make sure that metric providers that are dependent on the values of other metric providers have a lower priority than the metric providers on which they depend.

5. Leave the field *Scope Dependent* set to *No*.

4.2.13.6.3 Testing the Metric Providers

You can use class `CL_GSM_API_TEST` to create unit tests for your metric providers.

Context

With unit tests, you can simulate the API that retrieves the lower-level raw data. You focus here on the transformation logic to the structures of the generic metric store.

Procedure

1. In ABAP Development Tools (ADT), you can find the class `CL_GSM_API_TEST` as released object in the package `S_GSM_API`.
2. To create unit tests, implement the methods of the class `CL_GSM_API_TEST`.

4.2.13.6.4 Considerations for Metric Provider Updates

When you update an existing metric provider, make sure that these updates are synchronized across the system landscape.

Background

With the metric provider, you define a metric model using the method `DEFINE_MODEL( )`. This code is executed each time when you activate the metric provider and when the metric provider is imported into target systems. It's important to understand these synchronization events to transport metric providers correctly.

Things to Consider

Please keep the following considerations in mind to ensure a consistent metric model for your applications:

- Whenever you add a metric provider to a transport request (because you changed or activated it), also add the implementing class to the transport.

- Whenever you change a metric model in an implementing class, also add the corresponding metric provider to the transport. If you forget to add the metric provider, the metric model synchronization won't be triggered when the transport is imported into another system.
- When you activate a metric provider, the metric model is synchronized. Let's assume you change the behavior of the implementing class: For example, you change the definition of a metric, add a new metric, or you change something else in the metric model of your metric provider. Then you must not only activate the implementing class, but also the metric provider again to trigger a metric model synchronization. After activating the implementing class, log off and on again to the ABAP Development Tools and then activate the metric provider.

4.2.13.6.5 Collect Metric Provider Values (Administrator)

As an administrator, you must create an application job to collect metric providers and their measured values for the generic metric store. From the generic metric store, they can be pushed to SAP Cloud ALM or SAP Focused Run.

Prerequisites

You need an administration role to be able to create application jobs.

You must set up a communication arrangement based on communication scenario `SAP_COM_0527`, so that metrics stored in the generic metric store are automatically pushed to Health Monitoring in SAP Cloud ALM and SAP Focused Run.

Procedure

1. In the SAP Fiori launchpad, open the *Schedule Metric Provider Collection* app.
2. Choose *Create* to create a new application job.

   The job template for collecting metric provider values is already selected.
3. To change job settings, follow the instructions of the in-built documentation of SAP Companion.

   > ⓘ Note
   >
   > Enter a recurrence that is a multiple of 5 minutes. **Don't** use a recurrence pattern that's shorter than 5 minutes.
4. Schedule the job.

   > ⓘ Note
   >
   > Make sure that you schedule this job for the development system and all follow-up systems.

Results

The metric provider values are collected and saved to the generic metric store.

4.2.13.6.6 Troubleshooting

If your own metrics for health monitoring aren't collected as expected, check for the possible error sources.

You can check the following:

- Display the log of the application job *Collect Metric Provider Values*. In the log, search for `*GSM*` or the name of the metric provider.
  In the log, you might find error messages saying that metric providers have been banned and the reasons for the bans.
  Background: If the metric provider framework detects any kind of runtime violation such as, for example, too many consecutive dumps, an exceeded runtime limit, or too much memory consumption during the metric value collection, the relevant metric provider is banned for 24 hours. The ban protects the metric measurement of other metric providers. Fix the issues mentioned in the log. After 24 hours, the metric provider framework attempts to execute the provider once more in a tentative mode. If the execution passes without any issues, the ban is lifted, otherwise it's extended by 24 hours.
- If the metric provider is executed, but a metric is missing, you might need to retransport the metric provider to trigger an implicit metric model synchronization.

Related Information

Collect Metric Provider Values (Administrator) [page 1231]
Considerations for Metric Provider Updates [page 1230]

4.2.13.7 Accessing ABAP-Managed Data Using SQL Services for Data Integration Scenarios

As a developer in the ABAP environment, you can expose CDS view entities in an ABAP system for external consumption using SQL services.

Using the open database connectivity (ODBC) as a standard API for accessing databases, you can use SQL statements in external analytical tools to access the data of these CDS view entities from your ABAP system and formulate ad-hoc SQL queries on your data. You can also use the exposed CDS view entities from SQL services to set up data replication for further processing of the data and combining it with data from other data sources.

Why SQL Services?

There are situations where you would like to have external SQL read access to CDS objects owned by the ABAP system. Direct SQL read access to the underlying SAP HANA database of an ABAP system isn't recommended because, for example, ABAP-level security concepts are bypassed and typecasts might not be performed as expected (see also SAP Note 2511210).

Security concerns and other aspects such as authentication and authorization don't arise if you treat the ABAP system itself as a database by accessing the ABAP system directly using the SQL service exposure. In this case, authentication and authorization are done using an ABAP user. Full ABAP SQL semantics apply and even buffering on application server level can be used as well as ABAP-level access control and read access logging.

Compared to the OData interface, the SQL service exposure has the advantage that it allows for unrestricted SQL access to all exposed ABAP CDS view entities. Data from different entities can be joined in an ad-hoc fashion and data can be aggregated for analytical queries.

Data Integration Patterns and Scenarios

Data access patterns for ABAP-managed data can be categorized into data federation and data replication, as well as privileged access and business user access. Data federation allows for live access to the source system without replicating the data, while data replication involves replicating the data to a second system for local execution. Privileged access enables communication between an SQL-based client and the ABAP system using a communication user. (The communication user is a non-restricted technical power user or represents a technical system.) Business user access applies access controls to limit data access based on authorizations of the business user.

Examples of data consumption scenarios include data consumption using external ODBC clients like Microsoft Excel and LibreOffice, which can work with both privileged and business user access. Another example is data consumption using SAP Datasphere, where data is replicated from an ABAP system to SAP Datasphere for analytical scenarios.

More Information

For more information and a detailed end-to-end setup description, see the ABAP integration and connectivity guide on SAP Help Portal:

Data Integration (concept and architecture overview)

Data Integration (developer guide)

4.2.14 SaaS Applications and Multitenancy

Find out how to develop and operate SaaS applications by using add-ons in the ABAP environment, see Developing and Operating SaaS Applications [page 1234].

Learn more about multitenancy in the ABAP environment, the multitenant application, tenant business types and lifecycle, and see the multitenancy development guideline: Multitenancy in the ABAP Environment [page 1349].

Find out how to use the Landscape Portal, the central tool that allows providers to perform system administration tasks and lifecycle management operations.

Read about how SaaS applications can be extended, see Extending SaaS Applications [page 1361].

4.2.14.1 Developing and Operating SaaS Applications

Learn how to develop and operate SaaS applications by using add-ons in the ABAP environment.

Introduction

The ABAP environment allows you to leverage existing ABAP know-how to create custom add-ons in the cloud. This add-on build process is orchestrated by using the ABAP environment. Delivery tools, such as the Add-on Assembly Kit as a Service for the registration and publishing of the add-on as a software product, and deployment tools to carry out test installations and to make the add-on available for productive use, simplify this process. See Delivery Tools and Deployment Tools .

> **Note**
>
> You should only consider using add-on delivery if you intend to offer a Software as a Service (SaaS) solution.

> **Note**
>
> Note that only one add-on can be installed per system.

Following the add-on build process, you can develop a multitenant application to share the add-on with multiple consumers (tenants) simultaneously. See Multitenant Applications [page 1317]. The application is registered with the Software-as-a-Service provisioning service (saas-registry) to make the application available for subscription to consumers. See Register the Multitenant Application to the SaaS Provisioning Service.

In this context, the ABAP Solution Service [page 1320] accounts for the provisioning of new ABAP service instances, including the installation of the add-on, and the creation of tenants whenever required for a new consumer subscription.

The way ABAP service instances and tenants are used for consumer subscription applications can be configured as follows:

- **Single-tenant enabled**: With each new consumer, a new ABAP service instance is created
- **Multi-tenant enabled**: For multiple consumers, the same ABAP service instance is used

For more information, see Multitenancy in the ABAP Environment.

What Is This Guide About?

This guide describes the end-to-end process of developing and offering a SaaS solution using the ABAP environment. The following chapters cover all aspects of this scenario, from the initial setup and development tasks to the final productive use by consumers.

The Concepts [page 1260] chapter gives you an overview about important concepts that are relevant throughout the end-to-end process.

The Glossary [page 1255] contains a list of all the technical terms and phrases used in this scenario.

Chapter Develop, Test, Build [page 1288] describes the actual development activities. This includes the setup of the required account structure and system landscape, as well as the recommended testing process and subsequent build of the add-on.

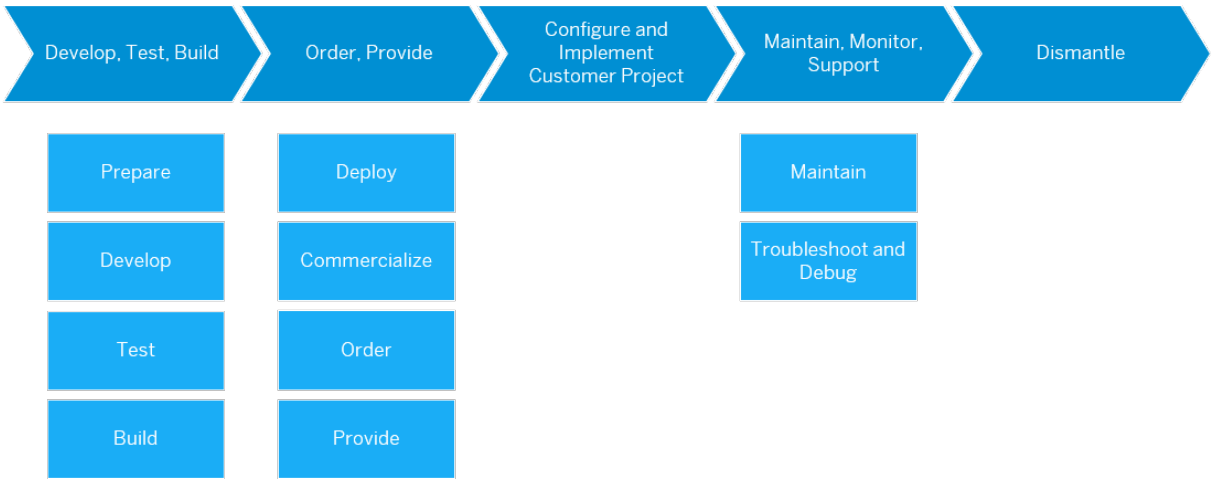
Chapter Order and Provide [page 1313] illustrates how a developed add-on can be deployed and made available to consumers. It also shows how consumers order solutions and what steps you, as the provider, need to perform to provision the solution.

Chapter Configure and Implement a Customer Project [page 1336] describes how to configure and implement a customer project in the consumer tenant, including the setup of identity and authentication management, integration with other systems or services, and adjustment of business configuration.

Chapter Maintain, Monitor, Support [page 1339] gives you an overview about the activities involved in maintaining an already commercialized product, as you usually continue to support the add-on, fix bugs, and potentially offer new functionalities as time goes on.

Chapter Dismantle [page 1348] provides information about consumer offboarding, in particular about deleting and restoring tenants.

> **Note**
>
> If you decide to use gCTS transport instead of add-on delivery, please refer to the gCTS Delivery boxes across the guide. See Delivery via Add-On or gCTS [page 1282].



- Prepare [page 1289]

- Develop [page 1301]
- Test [page 1304]
- Build [page 1307]
- Deploy [page 1313]
- Commercialize [page 1333]
- Subscribe [page 1334]
- Subscribe [page 1334]
- Maintain, Monitor, Support [page 1339]
- Order and Provide [page 1313]
- Develop, Test, Build [page 1288]
- Configure and Implement a Customer Project [page 1336]
- Maintain [page 1339]
- Troubleshoot and Debug [page 1347]
- Dismantle [page 1348]

For more information, see Build ABAP Code into an Add-on Product and Make it Available as a Tenant-Aware SaaS Solution.

> ⚠ Caution
>
> The tutorial group "Convert Your Add-on Product into a Tenant-aware SaaS Solution" is now replaced by the functionality of the Maintain Solution app.

4.2.14.1.1 Overview

Develop, Test, Build

Phase	Step	Description	User/Role
Prepare [page 1289]	Register a namespace. See Register a Namespace [page 1290].	A namespace is mandatory and must be reserved at https://support.sap.com/namespaces.	S-user

> **Note**
>
> You have to register a namespace before the first ABAP system is provisioned. To create namespaces after system provisioning, see Maintain Namespaces.
>
> Namespaces should have 5–8 characters due to length restrictions of certain objects. See SAP note 105132 and 395083.

Phase	Step	Description	User/Role

	Set up a global account for development. See Set Up a Global Account for Development [page 1291].	Get a partner development license for discounted systems used for development, test, or demo purposes. These systems are provisioned in the global account for development.
		
		Required entitlements:
		
		• Services:
		  • `abap/standard` and `abap/saas_oem`
		  • Application runtime for approuter
		  • abap-solution (ABAP Solution Provider)
		• Applications:
		  • Landscape Portal
		  • Web access for ABAP
		  • SAP Business Application Studio for UI development
		
		The `saas-registry` and `xsuaa` service are available without entitlement.
		
		See SAPPartnerEdge Test, Demo and Development Price List.
			Operator

Phase	Step	Description	User/Role

	Set up a global account for production. See Set Up a Global Account for Production [page 1293].	Get a partner production license for systems used for production purposes. These systems are provisioned in the global account for production.
		
		Required entitlements:
		
		• Services:
		  • abap/saas_oem
		  • Application runtime for approuter
		  • abap-solution (ABAP Solution Provider)
		  • saas-registry (without entitlement)
		  • xsuaa (without entitlement)
		• Applications:
		  • Landscape Portal
		  • Web access for ABAP
		
		See SAPPartnerEdge: Resources for OEM Partners.
			Operator

Phase	Step	Description	User/Role

	Create ABAP instance(s). See Create ABAP Instances [page 1295].	Set up your development/test subaccounts and ABAP system and create the following:
		• Development system DEV with parameter `is_development_allowed = true`
		  • abap/abap_compute_unit (standard: 1)
		  • abap/hana_compute_unit (standard: 4)
		• Test system TST with parameter `is_development_allowed = false`
		  • abap/abap_compute_unit (standard: 1)
		  • abap/hana_compute_unit (standard: 4)
		• Subscriptions in development/test subaccount to ABAP Web Access
		See Getting Started with a Customer Account: Workflow in the ABAP Environment and Create an ABAP System.
			Operator

	Set up UI development. See Set Up UI Development [page 1296].	Configure SAP Business Application Studio for UI development.	Operator
	Set up add-on development. Set Up Add-On Development [page 1297].	Create a new software component for add-on development and clone the main branch of the software component to the development and test system.	Add-on administrator

Phase	Step	Description	User/Role
	Transport from development to test system. See Set Up Transport from Development to Test System [page 1298].	Plan and set up the transport of software components from development to test systems. Tutorial: Transport a Software Component Between two Systems **Note** A CI/CD server that is running the ABAP Environment Pipeline is required. See ABAP Environment Pipeline [page 1266].	Add-on administrator
Develop [page 1301]	ABAP development. See ABAP Development [page 1301].	Follow the guidelines to enable multitenancy for applications built on the ABAP environment. See Multitenancy Development Guideline [page 1353] and Tutorials.	Developer
	UI development. See UI Development [page 1304].	Develop UIs using SAP Business Application Studio. See Develop an SAP Fiori Application UI and Deploy it to ABAP Using SAP Business Application Studio [page 1859].	Developer
	[Optional] Code migration. See (Optional) Code Migration from On-Premise [page 1304].	Migrate your ABAP code from on-premise. See Blog: How to bring your ABAP custom to the ABAP Environment and Blog: How to check your custom ABAP code for the ABAP Environment.	
Test [page 1304]	Test in cloud system. See Test in the ABAP Environment SAP Fiori Launchpad [page 1305].	Import the ABAP service and the UI into your test system. Create a role and assign it to users, then test the service and UI.	Test user

Phase	Step	Description	User/Role
	Test in the ABAP Test Cockpit. See Test in the ABAP Test Cockpit [page 1307].	Run ABAP Test Cockpit checks in DEV and/or TST systems. See ABAP Test Cockpit in the Cloud – What is already possible and Configure ABAP Test Cockpit [page 2933]. Request exemptions for false-positives and approve or reject the same.	Developer runs ATC checks and requests exemptions. Quality manager approves or rejects exemptions
Build [page 1307]	Prepare your add-on build. See Set Up Add-On Build [page 1309].	Set up a CI/CD server and pipeline for the add-on build. Create a technical communication user to access AAKaaS. See Build and Publish Add-on Products on SAP BTP ABAP Environment . Register the add-on product in the global account for development and production as described in Register Add-on Product for a Global Account and Register Product.	• Jenkins administrator for access to CI/CD Server • S-user to create technical communication user • Operator to assign technical platform user as space developer • DevOps engineer to configure the pipeline
	Build your first add-on version. See Build the First Add-On Version [page 1311].	Create a maintenance branch. Download API snapshot from add-on build system to upload in other systems. Configure the addon.yml file. Trigger the add-on product build/test/release.	Add-on administrator: create maintenance branch, configure addon.yml file, and trigger build pipeline

Order and Provide

Phase	Step	Description	User/Role
Deploy [page 1313]	Configure the sizing of the SaaS application. See Sizing [page 1313].	Decide on the metric of your offering (for example users, documents, space) and map the metric of your offering to the service plan of the ABAP environment service. **Note** For multitenancy offerings, there is no sizing/quota per customer (client). You must decide on an overall sizing depending on the expected load in a region (for example Europe/Frankfurt). As a DevOps engineer using parameter `tenant_mode` in the ABAP Solution service, you can define whether a customer gets a tenant in a dedicated system (single) or a shared system (multi). Determine the number of ABAP compute units and HANA compute units for the creation of an ABAP system by using parameters `size_of_runtime` and `size_of_persistence`.	DevOps engineer
	Implement a multitenant application and deploy it to the provider subaccount in the global account for development. See Multitenant Applications [page 1317].	Create the multitenant application with SaaS registry, XSUAA, and ABAP Solution Provider. Deploy to the provider subaccount with parameters used for development phase.	DevOps engineer

Phase	Step	Description	User/Role
	Access the Landscape Portal. See Access to Landscape Portal [page 1289].	Subscribe to the Landscape Portal application in the global accounts for development and production. See Accessing the Landscape Portal.	Operator
	Test the multitenant application deployed to the global account for development.	Subscribe to the SaaS solution from the consumer subaccount.	DevOps engineer
	Deploy the multitenant application to the provider subaccount in the global account for production.	Deploy to the provider subaccount with parameters used for production phase.	DevOps engineer
Commercialize [page 1333]	[Optional] SAP Store registration. See (Optional) Register SaaS Solution in SAP Store [page 1333].	Register your SaaS solution in SAP Store. See https://store.sap.com/dcp/en/.	
	[Optional] Certification. See (Optional) Certification [page 1334].	Get an optional certification for your product. See Certification and Certified Solutions.	
Subscribe [page 1334]	Create a consumer subaccount. See Create Consumer Subaccount [page 1334].	Set the account structure for the consumer subaccount.	Operator
	Subscribe to the SaaS solution. See Subscribe to SaaS Solution [page 1334].		Operator
	Configure consumer subaccount. See Configure Consumer Subaccount [page 1335].	Configure customer subaccount. • Option A: provider configures the subaccount • Option B: customer configures subaccount	Consumer subaccount administrator
	Initial user onboarding. See Initial User Onboarding [page 1336]	Create an initial administrator user for the customer.	Consumer subaccount administrator

Configure and Implement a Customer Project

Phase	Step	Description	User/Role
Configure and Implement a Customer Project [page 1336]	Set up Identity and Access Management in the consumer tenant of the customer.	Create and assign business roles. Create spaces and pages for business roles.	Business user
	Integrate the customer project in the consumer tenant of the customer.	Create communication arrangements.	Business user
	Configure the customer project in the consumer tenant of the customer.	Create business configuration. Configure key user extemsibility.	Business user

Maintain, Monitor, Support

Phase	Step	Description	User/Role
Maintain [page 1339]	Set up your test and maintenance system landscape and processes. See Set Up Maintenance System Landscape [page 1340].		Operator
	Create an update. See Create Add-On Update [page 1341].	Create a new patch version, support package version, or release/product version. SeeDeploy Product.	• Developer: implement update and double-maintenance • Test user: verify changes in test systems • Add-on administrator: check out maintenance branch, download API snapshots from the add-on build system as well as upload them to other systems, set the uploaded API snapshot as check-relevant, import software components into test systems, and configure the addon.yml file

Phase	Step	Description	User/Role
	Trigger the add-on product build. See Trigger Add-On Build Pipeline [page 1345].	Trigger the execution of the configured ABAP environment pipeline for an add-on build. SeeOperations Dashboard.	Add-on administrator
	Apply update for SaaS solution (=add-on product). See Deploy Add-On Update [page 1346].	Deploy your add-on update by using the Landscape Portal. See Landscape Portal.	Operator
Troubleshoot and debug. See Troubleshoot and Debug [page 1347].	• Identify ABAP system and consumer tenant • Access the consumer tenant of the ABAP system • Troubleshoot		Operator

Dismantle

Phase	Step	Description	User/Role
Dismantle [page 1348]	Delete tenants.	Unsubscribe the SaaS solution. See Dismantle [page 1348].	Consumer subaccount administrator or operator using Subscription Management Dashboard
	(Optional) Restore deleted tenants.	Restore the deleted tenants within a grace period of 30 days. See Dismantle [page 1348].	Operator

4.2.14.1.2 Users and Roles

User	Role	Description	Tasks/Usage
Provider	Add-On Administrator	The add-on administrator is responsible for everything related to the add-on production, for example, releasing an add-on product version.	• Creating software component • Importing changes in software component to test systems • Adjusting add-on descriptor file `addon.yml` for new versions • Registering add-on product for installation in global account for development and production. See Register Product. • Executing add-on build pipeline in Jenkins instance • Confirming Integration Tests stage in add-on build pipeline • Trigger publishing in Confirm stage during add-on build pipeline • Creating maintenance branches (for example v1.0.0) per support package level in *Manage Software Components* app. See Manage Software Components [page 3222]. • Checking out maintenance branch in correction system COR and quality assurance system QAS using the *Manage Software Components* app. • Using the Manage API Snapshots app: Downloading API snapshots from the add-on build

User	Role	Description	Tasks/Usage
			system, uploading these snapshots to other systems, and setting the snapshots as check-relevant to allow running compatibility checks for released objects
	DevOps Engineer	The DevOps engineer is responsible for the configuration of the pipeline and implementation of the multitenant application that needs to be deployed to enable the add-on as a SaaS solution.	• Initial configuration of add-on pipeline, especially `config.yml` file in git repository • Implementing pipeline extensions if necessary • Implementing a multitenant application. See Multitenant Applications [page 1317]. • Deploy multitenant application to provider subaccount in global account for development • Subscribe to multitenant application from consumer subaccount in global account for development • Deploy multitenant application to provider subaccount in global account for production

User	Role	Description	Tasks/Usage
	Developer	Developer users use ABAP Development Tools (ADT) to create backend service artifacts and UI developers create and deploy SAP Fiori apps in SAP Business Application Studio. See What is SAP Business Application Studio.	• Developing backend services in ABAP Development Tools (ADT) using ABAP RESTful Application Programming Model. See Eclipse Tool for the ABAP Environment and ABAP RESTful Application Programming Model [page 855]. • Developing SAP Fiori based UI in SAP Business Application Studio and deploying back to ABAP system. See Develop an SAP Fiori Application UI and Deploy it to ABAP Using SAP Business Application Studio [page 1859]. • Supporting and troubleshooting customer issues via provider support access in the Landscape Portal. See Landscape Portal. • Creates software component relations to declare access permissions and dependencies between software components. See Software Component Relations . • Releases APIs based on consistency and stability requirements and defines the visibility for use in different ABAP language versions. See Released APIs . • Runs ATC checks (Checking Quality of ABAP Code with ATC) and requests ATC exemptions for false-positive findings (Requesting Exemptions for ATC Findings)

User	Role	Description	Tasks/Usage
	Quality Manager	Evaluates requested ATC exemptions in ABAP development tools for Eclipse.	• Approving or rejecting ATC exemptions
	Test User	Test users are business users in test systems TST and QAS that validate the correct implementation of the add-on. This involves, among others, maintaining business roles and communication arrangements delivered as part of the add-on.	• Maintaining business roles. See Maintain Business Roles. • Creating spaces and pages for business roles. See How to Create and Assign Spaces and Pages. • Maintaining communication arrangements. See Communication Arrangements. • Creating business configuration • Creating key user extensibility • Testing ABAP backend services • Testing SAP Fiori UIs
	Provider Identity Authentication Administrator	An Identity Authentication service tenant can be configured for authentication in development/test/assembly systems. The Identity Authentication administrator configures the trust setup between the subaccount and Identity Authentication application. See SAP Cloud Identity Services - Identity Authentication.	• Configuring Identity Authentication application for trust setup in development/test/assembly subaccounts. See Create New Application. • Configuring corporate identity provider in Identity Authentication tenant. See Corporate Identity Providers.

User	Role	Description	Tasks/Usage

	Operator	The SaaS solution operator is responsible for creating the account model on the provider side (creating consumer subaccounts), adding subaccount members etc.	• Setting up global accounts for development and production including entitlements, subaccounts, Cloud Foundry org, spaces, services, and apps, as well as trust configuration and connectivity per subaccount. See System Landscape/Account Model [page 1260].
			• Consumer tenant onboarding: creating consumer subaccount and subscription. See **Subscribe to Multitenant Applications Using the Cockpit**.
			• Creating users in ABAP systems. See Creation of Additional Administrator Users.
			  • Developer users in DEV/COR systems. See Creation of Developer Users.
			  • Tester users in TST/QAS systems
			• Add-on update of existing systems in the Update Product Version app. See Deploy Product.
			• Monitoring of system/tenant provisioning and user onboarding. See Operations Dashboard.
			• **Note**
			If you use gCTS for delivery to customer production systems instead of using add-ons, you

User	Role	Description	Tasks/Usage
			are responsible for cloning software components and pulling the latest changes into the system using the *Manage Software Components* app. See Delivery via Add-On or gCTS [page 1282].

	Jenkins Administrator	A Jenkins CI/CD server is used as the infrastructure for automation purposes. Pipeline configuration, user management, credentials management etc. is handled by the Jenkins administrator. See Operations Dashboard	• Creating Jenkins instance
			• Maintaining Jenkins credentials for:
			  • Cloud Foundry technical platform user
			  • Technical communication user
			  • Credentials of service user for git repository of pipeline configuration
			• Maintaining Jenkins pipeline with reference to git repository of pipeline configuration
			• Adding additional Jenkins users, for example for add-on administrator user

User	Role	Description	Tasks/Usage
	S-User	S-users are used by partners and customers to log on to SAP ONE Support Launchpad to reserve development namespaces or to create technical communication users. User management is taken care of by user administrators in the *User Management* app. If customer numbers are assigned to a Partner ID, S-users are registered via PartnerEdge. See User Management and SAP note 2261006.	• Reserving development namespace in Namespace Application See https://launchpad.support.sap.com/#/namespaces. • Activating new namespaces in already existing systems using Landscape Portal. See Maintain Namespaces. • Creating technical communication user. See SAP note 2532813. • test
Technical Users of Provider	Technical Cloud Foundry Platform User	A platform user is added as a member to subaccounts in the Cloud Foundry environment to create or delete ABAP service instances, create service keys etc. This user should be a non-personalized user, for example based on a distribution list. The user does not need authorizations in SAP ONE Support Launchpad. The user does not need authorizations in SAP ONE Support Launchpad. Platform users are created by default in SAP ID Service. See User and Member Management and Default Identity Provider.	• Creating ABAP system AMT via ABAP solution service for consumer tenants. See Define Your ABAP Solution. • Creating ABAP system BLD via add-on build pipeline for add-on assembly • Creating ABAP system ATI via add-on build pipeline for add-on installation test • Deleting ABAP systems
	Service User Git	Jenkins pipeline definitions are source-code-based and stored in a source code repository, usually a Git repository. To retrieve the pipeline definition from the repository, a (technical) service user is configured.	• Pulling changes in git repository of pipeline configuration • Pulling changes in SAP/Jenkins library to project "Piper" library steps

User	Role	Description	Tasks/Usage
	Technical Communication User	A technical communication user (created via SAP ONE Support Launchpad) is used for system-to-system connections with the SAP Support backbone. For information on how technical communication users are different from S-users, see SAP note 2668288.	Used for communication with the AAKaaS. See Add-on Assembly Kit as a Service (=AAKaaS).
Consumer	Consumer Identity Authentication Administrator	An Identity Authentication Service tenant can be configured for authentication at consumer tenants in the productive ABAP systems. The Identity Authentication admin configures the trust setup between consumer subaccount and Identity Authentication application. See Identity Authentication Service.	• Configuring Identity Authentication application for trust setup in consumer subaccount • Optional: Configuring corporate identity provider in Identity Authentication tenant
	Consumer Subaccount Administrator	The consumer subaccount administrator is responsible for the configuration of: • Trust settings (custom identity provider), see Trust and Federation with Identity Providers. • Connectivity via Cloud Connector, see Connectivity in the Cloud Foundry Environment • Destinations, see Managing Destinations in the consumer subaccount created for the customer.	• Setting up connectivity via Cloud Connector • Adding destinations in consumer subaccount • Setting up trust configuration

User	Role	Description	Tasks/Usage
	Business User	Business users make use of the SaaS solution in the consumer tenant for their daily work.	• Initial administrator onboarding • Creating additional business users • Creating business roles • Creating spaces and pages for business roles. See How to Create and Assign Spaces and Pages. • Creating communication arrangements • Creating business configuration • Creating key user extensibility • Using SAP Fiori UIs
	Cloud Connector Administrator	To connect on-premise systems on the customer side to the consumer tenant with the SaaS solution, Cloud Connector can be configured in the consumer subaccount. The Cloud Connector administrator maintains the consumer subaccount, cloud-to-on-premise system mappings etc. See Cloud Connector.	• Adding a consumer subaccount to Cloud Connector. See Managing Subaccounts. • Configuring cloud to on-premise system mappings. See Configure Access Control. • Synchronizing trust configuration for principal propagation. See Set Up Trust for Principal Propagation.

4.2.14.1.3 Glossary

Add-on Assembly Kit as a Service (AAKaaS)	The Add-on Assembly Kit as a Service registers and publishes the software product. It is accessible via APIs with a technical communication user.
ABAP environment pipeline	The ABAP environment pipeline enables continuous integration for the ABAP environment. The pipeline contains several stages and supports different scenarios. See ABAP Environment Pipeline .
ABAP Solution (Provider)	The ABAP Solution service maintains ABAP systems and tenants to provide an add-on product as a SaaS solution.

ABAP Test Cockpit	ABAP Test Cockpit checks can be executed using `abapEnvironmentRunATCCheck`. The step can receive software components or packages configured in a YML file. The results are returned in checkstyle format. With the use of a pipeline extension, you can configure quality gates. See ATC and Configure ABAP Test Cockpit [page 2933].
ABAP System	The ABAP environment service (abap/standard) is used to create development and test systems. See Creating an ABAP System.
ABAP System (SaaS, OEM)	The ABAP environment (`abap/saas_oem`) service is used to create the add-on installation test system and production systems, where a specific add-on product needs to be installed in the system.
Add-on build pipeline	A pipeline configured for the add-on build scenario of the ABAP environment pipeline. See Build and Publish Add-on Products on SAP BTP ABAP Environment .
Add-on descriptor	The build process is controlled by an add-on descriptor file called `addon.yml`. This file must be created manually and stored in the Git repository of the pipeline. It must contain information about the software product version to be delivered and the containing software component versions. See Add-On Descriptor File .
Add-on installation test system	To verify that the delivery packages included in the add-on product version being built are installable, a target vector is published in "test" scope. In the integration tests stage, an ABAP system of service plan `saas_oem` is created. This ABAP OEM service allows you to install a specific add-on product version into an ABAP system that is provisioned.
Add-on product version (software product version)	To install and maintain ABAP software, software product versions are used. A software product version is a bundle of software component versions made available at the same time for implementing a well-defined scope of functionality. See Software Product Version .
API Snapshot	To be able to run API compatibility checks for your released objects using the ATC check `API_COMPATIBILITY`, you need to create API snapshots manually or download and upload them from your add-on build system. See Creating API Snapshots
API (Release) State	The API release state of a development object defines whether the object can be used for stable custom development. For some release contracts the visibility in certain ABAP language versions can be set. See Released APIs.
Assembly System	The ABAP system responsible for the add-on build. It is created during the pipeline and eventually deleted. All actions related to the ABAP source code are executed on this system, e.g. running checks with the ABAP Test Cockpit or the build of the software components.

Branch	Git repository branches can be used to control the flow of changes through the test and production landscape. They separate changes from each other, which shall or might not be delivered together (i.e. development vs. correction, feature A vs. feature B). A branch is created on the current state of another branch, the parent branch, and reflected by a new copy of all included objects. Depending on which branch is supposed to be changed, the corresponding copy is worked on.
Business role	A business role provides users with authorizations to access apps.
CI/CD server	The pipeline for building ABAP add-ons has been created specifically for Jenkins. Therefore, a Jenkins server is required. The project "Piper" provides a Jenkins image, which already includes the necessary configurations. Note that you can also configure an existing server.
Cloud Foundry environment	The Cloud Foundry environment enables you to develop new business applications and business services, supporting multiple runtimes, programming languages, libraries, and services. See Cloud Foundry Environment [page 63].
Communication arrangement	A communication arrangement describes a communication scenario with a remote system during configuration time. It provides the necessary metadata for service configuration. See Communication Arrangement.
Communication scenario	A communication scenario in the ABAP environment is a design time description of how two communication partners communicate with each other. It provides technical information, such as the used inbound and outbound services and their service type, for example OData or SOAP, and the number of allowed communication arrangement instances. See Communication Scenario.
Consumer	The service consumer is an end-customer of the service provider and has access to the SaaS application in a specific tenant
Correction codeline	Corrections need to run in parallel to development and on a released state. This is performed in the correction codeline and on a maintenance branch.
Destination service	The Destination service lets you find the destination information that is required to access a remote service or system from your Cloud Foundry application. See Consuming the Destination Service.
Development codeline	Infinity development (infinity codeline) is done based on the latest software component state in the main branch.
Development namespace	Naming conflicts can be avoided through agreements on naming conventions or by carrying out development in separate namespaces. See SAP note 84282 and Maintain Namespaces.

Global account	A global account is the realization of a contract you made with SAP. A global account is used to manage subaccounts, members, entitlements, and quotas. You receive entitlements and quotas to use platform resources per global account and then distribute the entitlements and quotas to the subaccount for actual consumption. See Getting a Global Account [page 162].
Identity Provider	SAP Business Technology Platform supports identity federation, a concept of linking and reusing digital identities of a user base across loosely coupled systems. Identity federation frees applications on SAP Business Technology Platform from the need to obtain and store the credentials of users to authenticate them. Instead, the application user base is reused from identity providers, which support the administration of digital user identities, authentication, and authorizations in a centralized and decoupled manner. See Trust and Federation with Identity Providers [page 2538].
Landscape Portal	The Landscape Portal app acts as a central tool to allow service providers to perform lifecycle management operations such as add-on updates, provisioning new consumers as new tenants, and more. SeeLandscape Portal .
Multitenancy	A multitenant service/application serves requests from different customers - the tenants - and processes their data strictly isolated from one another.
Patch level	Patch deliveries shall only contain small corrections. They are shipped with delivery packages of type *Correction Package*. For both the add-on product version and the software component version, the patch level is denoted in the third digit.
Project "Piper"	SAP implements tooling for continuous delivery in project "Piper". The goal of project "Piper" is to substantially ease setting up continuous delivery in your project using SAP technologies. See Project Piper .
Provider	The provider is responsible for the development and maintenance of the SaaS application. This is typically an independent software vendor or development partner.
Release	Release deliveries contain the whole software component and deliver new features and enhancements of existing functionalities. For both the add-on product version and the software component version, the release is denoted in the first digit.

SAP Business Application Studio	Available as a cloud service, SAP Business Application Studio provides a desktop-like experience similar to leading IDEs, with command line and optimized editors. At the heart of SAP Business Application Studio are the dev spaces, which are similar to isolated virtual machines in the cloud containing tailored tools and pre-installed runtimes per business scenario, such as SAP Fiori, SAP S/4HANA extensions, Workflow, Mobile and more. This simplifies and saves time in setting up your development environment and allows you to efficiently develop, test, build, and run your solution locally or in the cloud. See What is SAP Business Application Studio.
SAP Cloud Identity Services - Identity Authentication	The Identity Authentication service provides you with controlled cloud-based access to business processes, applications, and data. It simplifies your user experience through authentication mechanisms, single sign-on, on-premise integration, and convenient self-service options. See What Is Identity Authentication?.
SAP Cloud Identity Services - Identity Provisioning	The Identity Provisioning service automates identity lifecycle processes. It helps you provision identities and their authorizations to various cloud and on-premise business applications. See What is Identity Provisioning.
SAP ID service	The default platform identity provider and application identity provider of SAP Business Technology Platform is SAP ID service. See Default Identity Provider [page 2594].
SAP ONE Support launchpad	SAP ONE Support Launchpad provides you access to task-driven support resources in an intuitive interface. By using customizable role profiles, it displays only the relevant applications and insights to you. See SAP ONE Support Launchpad.
SAP PartnerEdge	Get information, training, tools, and resources regarding partner licensing at SAP Partner Portal.
SAP Store	SAP Store is the enterprise marketplace where we bring together customers and partners on a single, easy-to-use, global online platform. Here, customers can discover, try, and buy SAP-validated partner applications that are built on or extend their existing SAP technology and solutions. For partners, it's the only place they can market and deliver their apps, add-ons, and integration kits to SAP's global customers — solutions that help customers grow their business. See SAP Store.
Software component	In ABAP environment systems, you develop within software components (also called repositories). The add-ons being built in this scenario are made up by one or multiple software components combined to an add-on product. See Software Components [page 863].
Software Component Relation	Software component relations define access permissions and dependencies between software components. For more information, see Software Component Relations

Software component version	A software component version is a technically distinguishable unit of software and is installed and patched as a whole. See Software Component Version .
Subaccount	Subaccounts allow you to structure a global account according to your organization's and project's requirements with regard to members, authorizations, and entitlements. See Subaccounts.
Support Package (Stack) Level	Support package deliveries contain a larger collection of corrections and may contain smaller functional enhancements. For both the add-on product version and the software component version, the support package level is denoted in the second digit.
Web Access for ABAP	Subscribe to the Web access for ABAP to get direct browser access to your instances in the ABAP environment, including access to the administration launchpad for ABAP. See Subscribing to the Web Access for ABAP [page 211].

4.2.14.1.4 Concepts

Learn more about the system landscape/account model, ABAP environment pipeline, as well as versioning and branches.

4.2.14.1.4.1 System Landscape/Account Model

> **Note**
>
> If you use gCTS transport delivery instead of add-on delivery, no discounted development licenses are available: ABAP systems created for development purposes (development, test, demo) cost the same as ABAP systems created for production purposes. For creation on system AMT instead of `saas_oem`, the standard service plan is used since no add-on is installed.
>
> Additionally, the same global account is used for development/production purposes because gCTS repositories are only available across the same global account.

ABAP systems for development and for production purposes are created based on different partner contracts and licenses.

Discounted development licenses can be used for development, test, and demo purposes.

> **Note**
>
> You have to acquire a development license for partners. See SAP PartnerEdge Test, Demo & Development Price List.

Production licenses are used whenever one of your customers is consuming the solution provided by you in one of your systems.

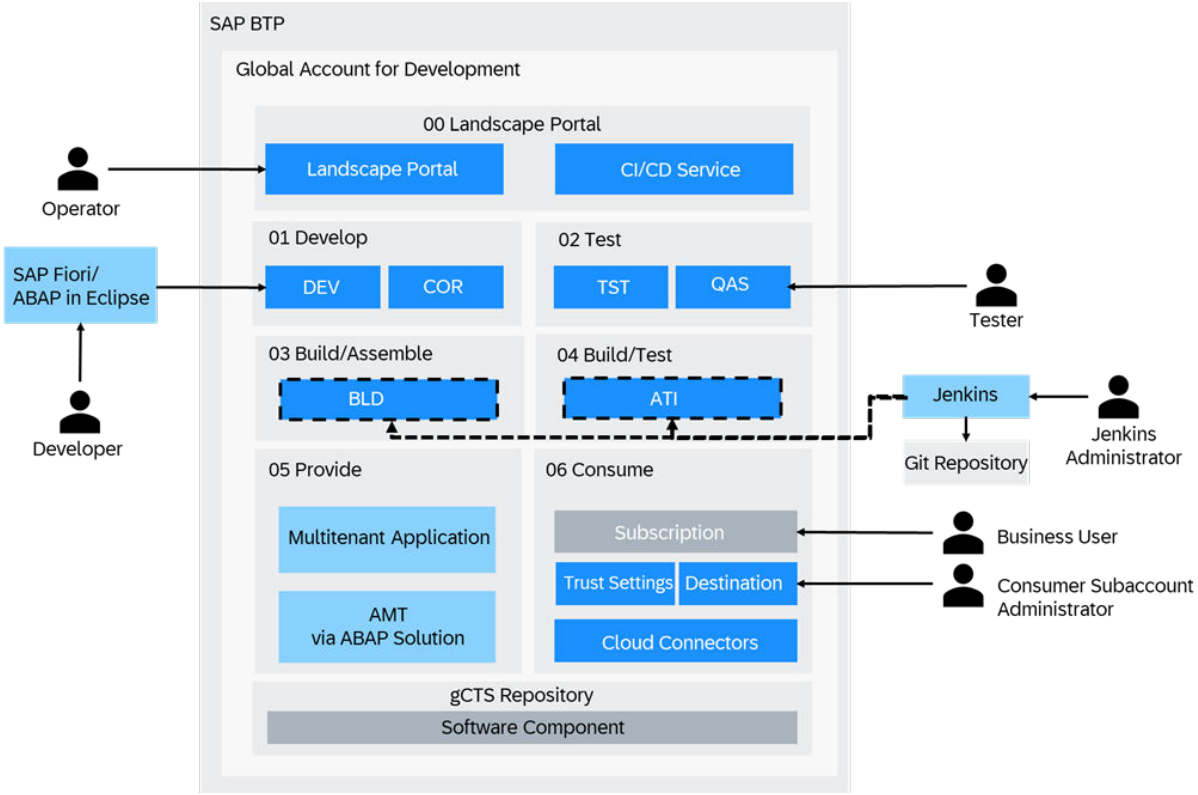
> ### Note
>
> You have to acquire a production license for partners. See Resources for OEM Partners.

To separate development and production purposes, you have to create different global accounts:

- **Global account for development**
  The global account for development is used for add-on/UI development and testing. Also, add-on assembly and add-on test installation triggered by the add-on build pipeline are performed in this global account.
  All ABAP systems created in this global account are used for development, test, or demo purposes.
- **Global account for production**
  All ABAP systems created in this global account are used for production purposes. That means, the systems are used to provide the SaaS solution to your customers. Consumer subaccounts are also created in the global account for production.

> ### → Recommendation
>
> We recommend creating subaccounts for the different stages of the SaaS solution enablement.
>
> Using different subaccounts for the different phases has multiple advantages: Trust settings and destination/connectivity settings can be adjusted for each subaccount. That means, you can connect the development subaccount to the developer user identity provider and the test subaccount to the test user identity provider. In terms of connectivity, the development subaccount can be connected to the development on-premise system and the test subaccount can be connected to the test on-premise system.

Global Account for Development



1. **00 Landscape Portal**
   We recommend creating a dedicated subaccount for the Landscape Portal. This subaccount will contain your Landscape Portal & CI/CD service subscriptions for you to freely choose between regions for your other subaccounts.
2. **01 Develop: Development subaccount including development space**
   ABAP development system DEV and correction system COR are created in this subaccount. You need to subscribe to SAP Business Application Studio to develop SAP Fiori UIs and implement multitenant applications. See Setup of UI Development in SAP Business Application Studio.
3. **02 Test: Test subaccount including test space**
   ABAP test system TST and quality assurance system QAS are created in this subaccount. See Use Case 2: One Development and Correction Codeline in a 5-System Landscape [page 899]. Since development is structured with software components that are stored in a repository for each global account, these software components can automatically be imported to the test system on a regular basis automated by the CI/CD server. See (Deprecated) Test Integration (SAP_COM_0510) [page 945]. The CI/CD server uses a Git repository to read the pipeline definition and configuration.
4. **03 Build/Assemble: Subaccount for add-on assembly including build/assemble space**

> ### Note
>
> The ABAP environment platform version of the assembly system is used to determine the minimum platform version for the add-on product version that is created.
>
> Such a system should not be nominated for the pre-upgrade option of the ABAP environment because this would lead to the add-on product only being able to installed in systems with the pre-upgrade

release. When using Build Product Version (recommended), the add-on definition is configured directly in the app.

For the add-on build process, an assembly system BLD is provisioned in this subaccount by the CI/CD server. See Software Assembly Integration (SAP_COM_0582) [page 955] When using the Build Product Version app (recommended), the add-on definition is configured directly in the app.

5. **04 Build/Test: Subaccount for add-on installation test including build/test space**
   After the add-on has been assembled during the build of an add-on version, an installation test is required to verify that the add-on can be installed without errors into a system. As part of the add-on build pipeline, an `abap/saas_oem` system ATI is provisioned in this subaccount and the add-on is installed. When using the Build Product Version app (recommended), the add-on definition is configured directly in the app.
6. **05 Provide: Provider subaccount including provider space**
   During the development phase, the multitenant application is deployed to this space for testing purposes. The ABAP solution service can then provision `abap/saas_oem` systems AMT, tenants, and users in this account, once a consumer subscribes to the provided SaaS solution.
7. **06 Consume: Consumer subaccount**
   To test the subscription to the SaaS solution during the development phase of the multitenant application, a consumer subaccount is created in the global account for development. This allows your consumers to have their own configuration of:
   - Trust settings (custom identity provider)

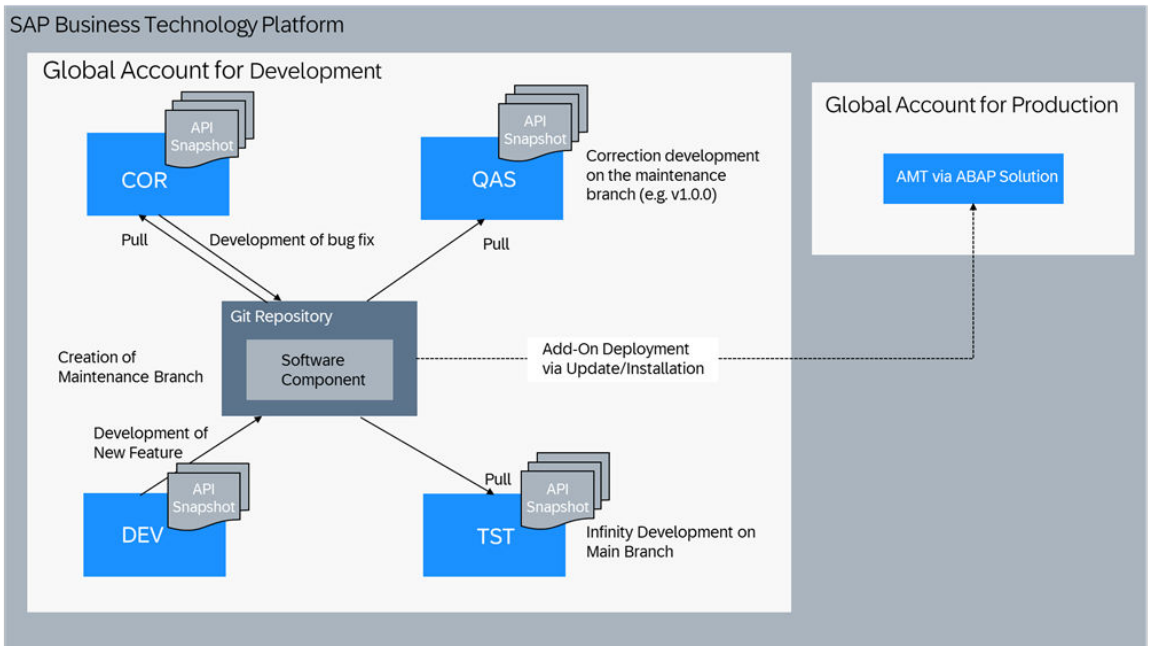
     > ⓘ Note
     >
     > If you want to integrate an existing corporate identity provider for authentication/authorization in subaccounts of the global account for development, see Trust and Federation with Identity Providers. To restrict access based on certain criteria, such as the IP address, you need to use the Identity Authentication service. See SAP Cloud Identity Services - Identity Authentication.

   - Connectivity via SAP Cloud Connector. See Connectivity in the Cloud Foundry Environment.
   - Destinations. See Managing Destinations.
   - Subscriptions

> ⓘ Note
>
> You can use a booster (see Booster for Landscape Portal [page 1298]) to automatically perform the setup of subaccounts 00 Landscape Portal, 03 Build/Assemble and 04 Build/Test.

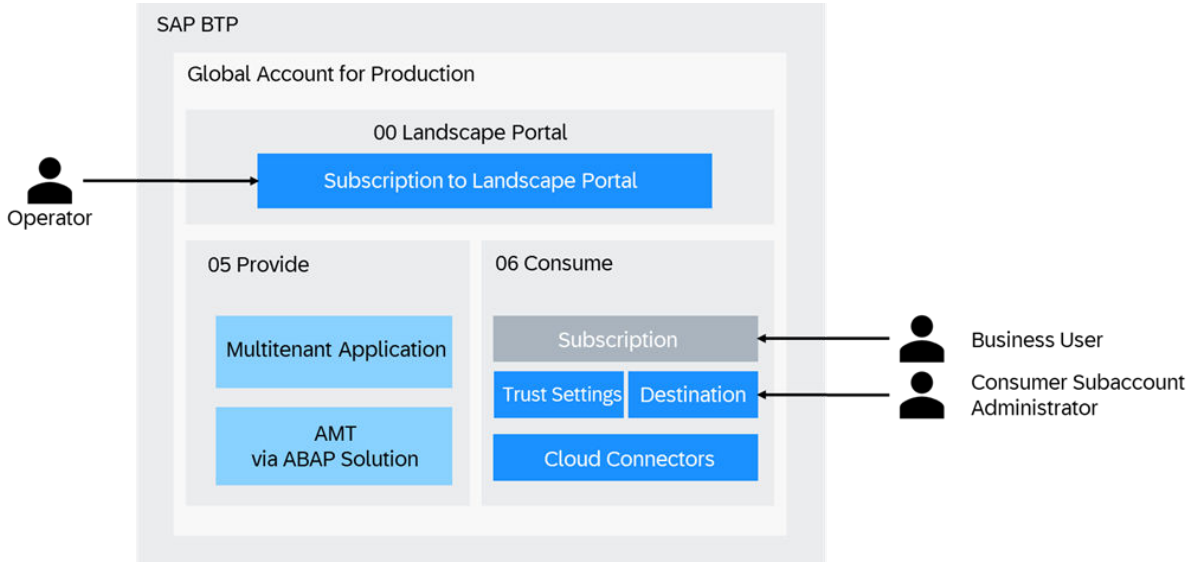
Development Flow

For development and maintenance processes, the steps mentioned below, that are similar to the ones described in Use Case 2: One Development and Correction Codeline in a 5-System Landscape [page 899], are performed.



- ABAP system COR and QAS have the same software state, unless a new change is tested and released. This means, transport requests are released in ABAP system DEV only if development is completed and it's planned to import the changes to the production ABAP system.
- Upon cutoff date, development is finished. All development that is released at this time must be tested and be of good quality. From then on, you must fix defects in the COR system and maintain them in parallel in the DEV system. In case of released APIs in the involved software components, API snapshots should be created at this point. This is required for API compatibility checks to function properly. Checking the Compatibility of Released APIs.
  The API snapshots can be generated manually in system TST or they are generated automatically in an add-on build system. The snapshots should be downloaded and then be uploaded into the DEV system.
- Upon release date, all defects must be fixed. If, during testing in the QAS system, you make the decision that a complete functionality isn't delivered, developers must delete, revert, or disable the functionality in the COR system and release the corresponding transport requests. You can't remove objects from the release branch, e.g. by deselecting transport requests. To revert objects to a previous transported state, use the Compare editor of the Eclipse History view. If you want to withdraw the functionality in the DEV system as well, it's considered a correction and you have to perform double maintenance of corrections into the DEV system. See Double Maintenance of Corrections into Development [page 915].
- Once correction system COR and quality assurance system QAS are used for maintenance development, you can also upload the API snapshots created for the latest release or support package version in these systems.
- Users in ABAP system COR are locked during ongoing development and only unlocked when a correction has to be implemented
- For the consumption as a SaaS solution, instead of importing a release branch into a productive system, software components are installed via add-on delivery packages into multitenancy-enabled production systems AMT provisioned via the ABAP Solution service. See ABAP Solution Service [page 1320].

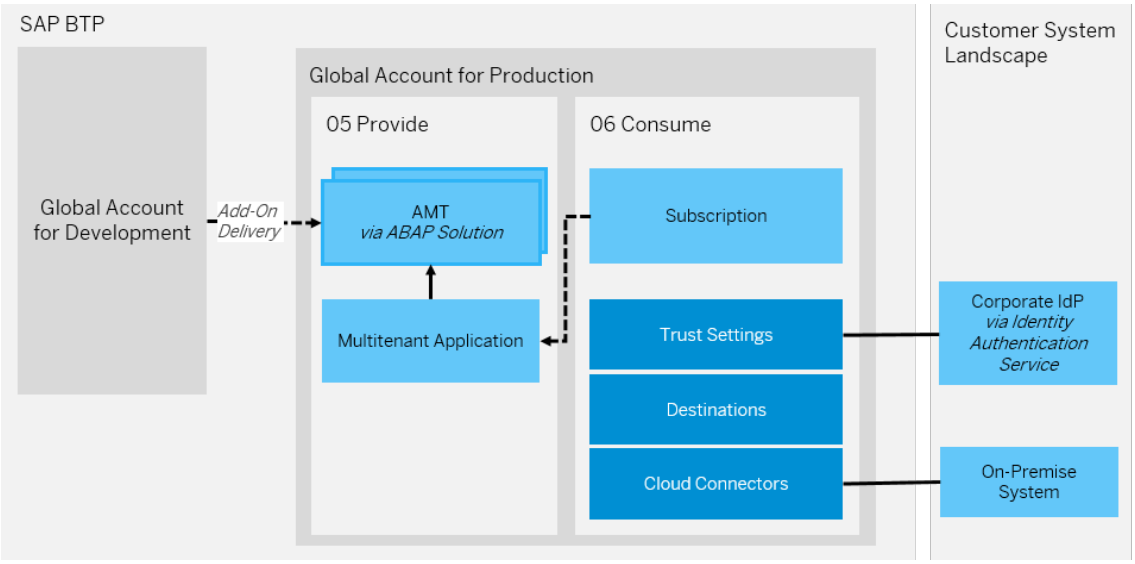
Global Account for Production



1. **00 Landscape Portal**
   Since the Landscape Portal service is only available in region eu10, we recommend creating a dedicated subaccount in this region. This subaccount will contain your Landscape Portal & CI/CD service subscriptions for you to freely choose other regions for your other subaccounts.

   > ### Note
   >
   > If you plan to create any of the other subaccounts in eu10, then you can simply reuse one of those subaccounts.

2. **05 Provide: Provider subaccount including provider space**
   The multitenant application is deployed to the provider subaccount in the global account for production. The ABAP solution service can then provision `abap/saas_oem` systems (AMT), tenants, and users in this account once a consumer subscribes to the provided SaaS solution.
3. **06 Consume: Consumer subaccount**
   For each customer, a consumer subaccount is created in the global account for production.



This allows consumers to have their own configuration of:

- Trust settings (custom identity provider).

> **Note**
>
> If you want to integrate an existing corporate identity provider for authentication/authorization in subaccounts of the global account for production, see Trust and Federation with Identity Providers. To restrict access based on certain criteria, such as the IP address, you need to use the Identity Authentication service. See SAP Cloud Identity Services - Identity Authentication.

- Connectivity via SAP Cloud Connector. See Connectivity in the Cloud Foundry Environment.
- Destinations. See Managing Destinations.
- Subscriptions

> **Restriction**
>
> As of now, you can only expose SaaS applications for subscription from consumer subaccounts in the provider global accounts.

4.2.14.1.4.2 ABAP Environment Pipeline

The goal of the ABAP environment pipeline is to enable continuous integration for the ABAP environment.

The pipeline contains several stages and currently supports two relevant scenarios:

- Add-on build using AAKaaS
- Continuous testing via gCTS

A pipeline definition, containing all configuration, and a running CI/CD server are required to execute the ABAP environment pipeline. You can consume the different scenarios by enabling different pipeline stages in the configuration. For more information on how to configure the ABAP environment steps and stages, see Configuration and ABAP Environment Pipeline .

The Landscape Portal offers the option to configure and execute the add-on build scenario using the Build Product Version app, without the need for external repositories or Jenkins servers.

> **Note**
>
> The *Continuous Testing* scenario is currently not supported by the Landscape Portal. This means that a running Jenkins server is required, even when using the Build Product Version app. See Add-On or gCTS.

Jenkins Setup

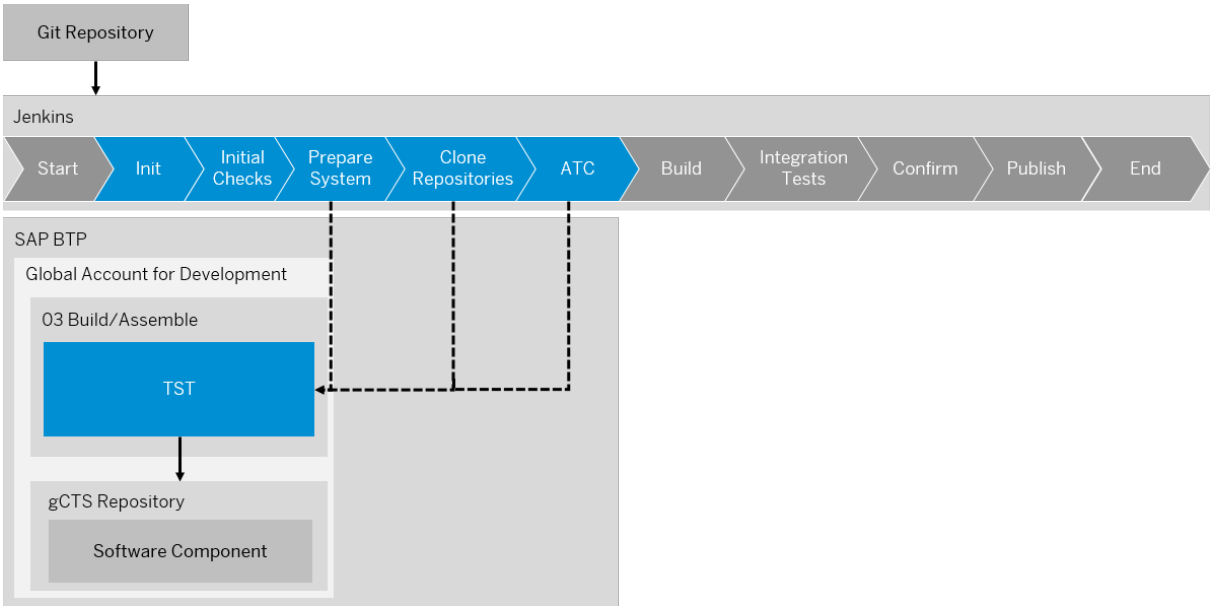
> **Note**
>
> This is only required for running the Continuous Testing scenario. The add-on build scenario can be realized using the Build Product Version app in the Landscape Portal.

The general configuration of the ABAP environment pipeline always follows the same approach regardless of the scenario that you want to configure:

1. **Prepare Git Repository**
   As a DevOps engineer, you need to prepare a Git repository by including the Jenkins file, initializing the pipeline, and the pipeline configuration file `.pipeline/config.yml`. See Jenkins File and Technical Pipeline Configuration.
2. **Create Service User for Git Repository**
   To enable read access to the Git repository, you need to create a service user and assign it to the repository. Later, this user's access credentials are stored in the Jenkins credentials by the Jenkins administrator. See Using Credentials.
3. **Create Jenkins Instance**
   As a Jenkins administrator, you need to set up a new Jenkins instance. See Custom Jenkins Setup and Docker Hub.
4. **Configure Jenkins Instance**
   As a Jenkins administrator, you need to add technical Git user credentials and platform user credentials to the integrated secure store. Make sure that a shared library piper-lib-os is configured pointing to the project "Piper" library. See Piper Library.

For more information on how to configure the ABAP environment steps and stages, see Configuration.

Transport from DEV to TST System via gCTS



Development system DEV and test system TST stay on the same main branch. To make the latest implementations available in the test system, the developed software components can be imported on a regular basis to test system TST and tested using the ABAP Test Cockpit by scheduling a planned execution of the ABAP environment pipeline. See Continuous Testing on SAP BTP ABAP Environment and Running ATC Checks and ABAP Unit Tests on a Static ABAP Environment System .

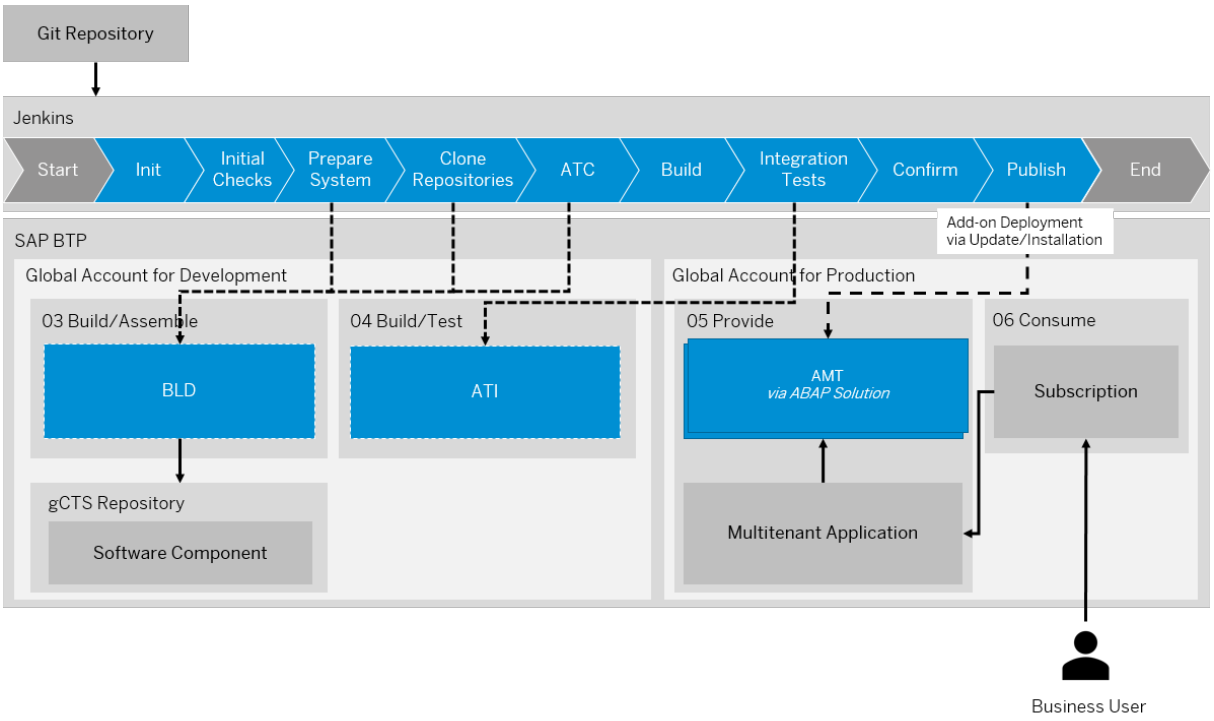
The ABAP environment pipeline is executed in a Jenkins server that is connected to subaccount *02 Test* in the global account for development by authenticating via a technical Cloud Foundry platform user. The permanent test system TST is used to import software components and run ABAP Test Cockpit checks.

Build Add-On Version

The ABAP environment pipeline can automate the build and assembly process of ABAP add-ons. From creating the delivery packages in the assembly system to publishing the add-on release, all steps are part of this pipeline.

This scenario can be run in the Landscape Portal using the Build Product Version app, or it can be configured manually and run on a local Jenkins server. In both cases, the same pipeline stages will be executed.

See Build and Publish Add-on Products on SAP BTP ABAP Environment .



- https://www.project-piper.io/pipelines/abapEnvironment/stages/initialChecks/ [https://www.project-piper.io/pipelines/abapEnvironment/stages/initialChecks/]
- https://www.project-piper.io/pipelines/abapEnvironment/stages/prepareSystem/ [https://www.project-piper.io/pipelines/abapEnvironment/stages/prepareSystem/]
- https://www.project-piper.io/pipelines/abapEnvironment/stages/cloneRepositories/ [https://www.project-piper.io/pipelines/abapEnvironment/stages/cloneRepositories/]
- https://www.project-piper.io/pipelines/abapEnvironment/stages/ATC/ [https://www.project-piper.io/pipelines/abapEnvironment/stages/ATC/]
- https://www.project-piper.io/pipelines/abapEnvironment/stages/build/ [https://www.project-piper.io/pipelines/abapEnvironment/stages/build/]
- https://www.project-piper.io/pipelines/abapEnvironment/stages/integrationTest/ [https://www.project-piper.io/pipelines/abapEnvironment/stages/integrationTest/]
- https://www.project-piper.io/pipelines/abapEnvironment/stages/confirm/ [https://www.project-piper.io/pipelines/abapEnvironment/stages/confirm/]
- https://www.project-piper.io/pipelines/abapEnvironment/stages/publish/ [https://www.project-piper.io/pipelines/abapEnvironment/stages/publish/]

The ABAP environment pipeline is executed in a Jenkins server that is connected to subaccount *03 Build/Assemble* and *04 Build/Test* in the global account for development by authenticating via a technical Cloud Foundry platform user. Transient systems are then provisioned for add-on assembly (BLD) and installation tests (ATI).

Add-On Assembly System

Add-on assembly system BLD includes an instance of communication scenario `SAP_COM_0948` and an instance of communication scenario `SAP_COM_0582`. See SAP_COM_0948 and Software Assembly Integration (SAP_COM_0582) [page 955]. Service keys with parameters referring to the communication scenarios are created in the BLD system, which leads to the creation of communication arrangements that can be used by the pipeline for inbound communication.

In the **Prepare System** pipeline stage, a new transient system BLD is provisioned for the add-on assembly. See Prepare System . ABAP Test Cockpit checks, software component imports, and the local build of deliveries are performed in this stage. By default, the system is deleted again in the **Post** pipeline stage. See Post .

The communication with remote components is enabled via communication scenario SAP_COM_0948 and communication scenario Software Assembly Integration (SAP COM 0582).

By default, the system is created from scratch for each new add-on version. You can choose to reuse a permanent BLD system instead of provisioning a new one each time. This has the advantage of reducing the pipeline execution time, as system provisioning is one of the most time-consuming tasks. See Build Add-Ons on a Permanent ABAP Environment System .

To reduce costs, it is recommended that you shut down the permanent system when no builds are scheduled. See Manage System Hibernation. Please note that hibernated systems still incur costs.

You can decide in detail whether to use a permanent or transient assembly system based on following criteria:

	Transient Assembly System	Permanent Assembly System
Pros	• If additional dependencies to objects outside the modeled product exist, they will be transparent as import errors	• API Snapshots are automatically created locally, can be downloaded for use in other systems, and remain in the system for continuous API compatibility checks • Reduced time consumed for add-on build as no new assembly system needs to be provisioned • Local history of ABAP objects, transport piece lists, deliveries, transport logs etc. remains available in the system for trouble shooting and error analysis

	Transient Assembly System	Permanent Assembly System
Contras	• API Snapshots are only created locally and thus deleted once a system is deprovisioned. Therefore, API compatibility checks cannot be used. Snapshots can be downloaded for use in other systems, but only while the system exists. • A new assembly system needs to be provisioned for each new add-on build, software components need to be cloned from scratch – resulting in longer runtime • Local history of ABAP objects, transport piece lists, deliveries, transport logs etc. is not available after the assembly system is deprovisioned	• Import errors during gCTS operations must be resolved to always guarantee the software component state in the system to be consistent with the remote repository • Dependencies to objects outside the modeled product might exist, so there is no guarantee to reveal unwanted dependencies

> ### Note
>
> In case of dependencies between software components that are facilitated trough the creation of released APIs, it is recommended to use a permanent assembly system. Only then the API compatibility checks can function properly based on the API snapshots that are required to remain in the system. See Released APIs and API Snapshots.

> ### → Tip
>
> You may reuse existing permanent test systems like TST or quality assurance system QAS as alternative for the assembly system BLD. If you choose to do so, make sure that there are no ongoing activities like software component lifecycle actions. See System Landscape/Account Model.
>
> In such a case also the manual creation of API snapshots in those systems becomes obsolete as the add-on build pipeline will automatically create such snapshots and set them as check-relevant. See Released APIs and API Snapshots.

Add-on Installation Test System

To verify that the delivery packages included in the add-on product version are installable, a target vector is published in "test" scope during the **Build** stage.

In the **Integration Tests** pipeline stage, a new transient system ATI is then provisioned for an add-on deployment test that installs the add-on product version. See Integration Tests . After the successful add-on installation is confirmed, the system is deleted, unless otherwise specified.

For generation and syntax check, please take a look at communication scenario Software Assembly Integration (SAP_COM 0582).

By default the ATI system is not created from scratch for each new add-on version. If you configure your pipeline manually, you can choose not to dismantle the ATI system automatically after the import test. This is intended to be able to analyze issues in this system in case they cannot be solved by the pipeline log output. Do not forget to delete the system manually prior to the next implementation of the pipeline. Otherwise it is

reused. Reusing the system would save runtime as no new system would need to be provisioned but not all issues which should be detected by this installation test would be detectable anymore.

Add-on Assembly Kit as a Service

The Add-on Assembly Kit as a Service (AAKaaS) is used for registering and publishing the software product.

The service is offered in the SAP Service and Support systems, which means that access is granted via a technical communication user that packs the delivery into an importable package format. It is similar to the Software Delivery Assembler (SDA, transaction SSDA) as a part of the SAP Add-On Assembly Kit. See SAP Add-On Assembly Kit and Add-On Assembly Kit as a Service .

Add-on Consumption

For the consumption of the add-on as a SaaS solution, software components are installed via add-on delivery packages into multitenancy-enabled production systems (AMT) provisioned via the ABAP Solution service. This is orchestrated via a multi-tenant application deployed to the Cloud Foundry environment on SAP BTP. See Multitenant Application and Deploy.

> **Note**
>
> If you need support or experience issues during the add-on build, see Troubleshooting .

4.2.14.1.4.3 Multitenancy

The ABAP environment provides capabilities for multitenancy, meaning to host different customers on a single ABAP system. See Multitenancy in the ABAP Environment [page 1349].

> **→ Tip**
>
> You should align your decision whether to enable multitenancy mode for your solution based on whether the Multitenancy Development Guideline [page 1353] can be followed during ABAP development.

4.2.14.1.4.4 Versioning and Branches

4.2.14.1.4.4.1 Add-On Versioning

For the add-on delivery process, certain versioning principles apply. Learn how software components are combined to an add-on product and what versioning rules are used for both parts. Software components and add-on products follow a similar versioning pattern.

Add-on Product Version

Instead of importing software components into productive systems, when providing a SaaS solution in the ABAP environment, software components are packaged and installed into ABAP systems based on so-called add-on products/software products.

You can only install one add-on product for each system using the ABAP environment `saas_oem` service plan instead of the `standard` service. During provisioning of this service, parameters `addon_product_name` and `addon_product_version` are used to install a certain add-on product version into the system, after the system has been provisioned. See ABAP Solution Service [page 1320].

The add-on product name includes a reserved development namespace to separate the name from other customers/partners. See SAP note 84282.

The versioning pattern `<Release>.<Support Package Stack Level>.<Patch Level>` applies to the add-on product version.

> **Note**
>
> Only in a new release, you can change the bundled software components of the add-on product.

Technically, the allowed number range is 1.0.0 to 9999.9999.9999.

For the release, there are no technical restrictions regarding the number of new versions. You only have to consider that the version has to be ascending across builds and there must be no gaps between versions. The initial release is always 1.0.0.

The support package stack level count starts again from 0 with each release.

The patch level count starts again from 0 with each support package.

While providing a specific add-on version, leading zeroes should be omitted (1.2.3 instead of 0001.0002.0003).

The add-on product version should be increased analogous to the version of the leading software component. The leading software component is the software component that is, as opposed to a reuse software component, exclusively used as part of one add-on product.

An exception to this recommendation is the patch level in the add-on production version: In case of an add-on product with a reuse software component, the patch level of the add-on product version might be higher than the patch level of the leading software component version.

For more information about software product versioning, see Add-On Product Version .

Software Component Version

Software components are self-contained and include packages as well as development objects. They are used in combination with a reserved development namespace to separate the name from other customers/partners. See Software Components [page 863].

The versioning pattern `<Release>.<Support Package Level>.<Patch Level>` also applies to the software component version.

> **Note**
>
> Software component versions are delivered with delivery packages. However, software component versions are not individual shipment entities. They can only be delivered to customers as part of a software product version.

Software component versions are only built once, independent from add-on product versions where the software component version was referenced. If an add-on product version is referring to a software component version that has already been created as part of a different add-on build, the created delivery package is reused.

Technically, the allowed number range is 1.0.0 to 9999.9999.9999.

For the release, there are no technical restrictions regarding the number of new versions. You only have to consider that the release has to be ascending across builds and there must be no gaps between versions. The initial release is always 1.0.0.

The support package level can only go up until 369 because of technical limitations. The support package level count starts again from 0 with each new release.

For the patch level, there is a technical limit of 36^3, limited to 9999. The patch level count starts again from 0 with each support package.

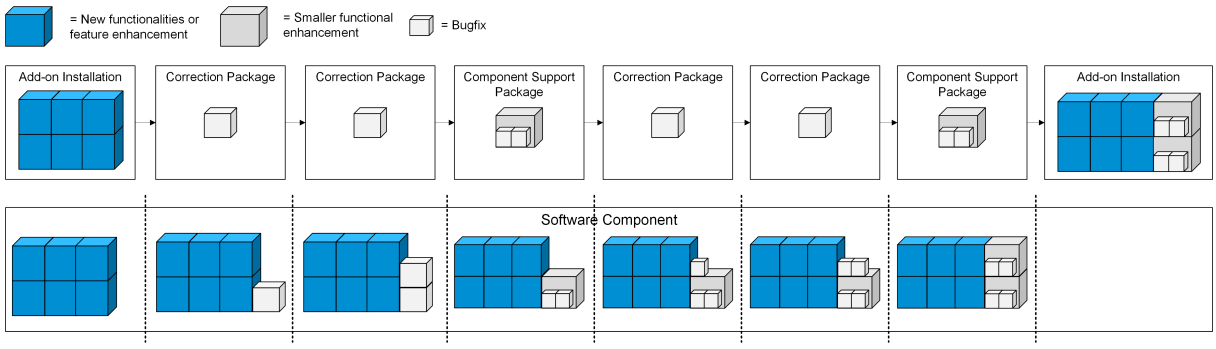
For more information on software component versioning, see Software Component Version .

4.2.14.1.4.4.2 Add-On Package Types

For the delivery of add-on packages, there are three package types that serve different purposes: AOI, CSP, and CPK.

The different add-on package types are used for different purposes. The set of objects to be included depends on the package type.

> **Note**
>
> The required type of delivery is automatically determined by the delivery production tools based on how the version number of software components is changed.



AOI (Add-On Installation)

Delivery packages of type Add-on Installation (AOI) are created for all new release versions, e.g. version 1.0.0, and should be used to deliver new functionalities or feature enhancements.

These delivery packages include all the objects in the software component and are usually created on a regular basis (e.g. quarterly).

CSP (Component Support Package)

Delivery packages of type Component Support Package (CSP) are created for all new support package deliveries, e.g. version 1.1.0, and should be used to deliver a collection of patch deliveries or to deliver smaller functional enhancements.

These delivery packages include either the objects that were changed since the previous release delivery or since the previous support package delivery. They are usually created on a regular basis (e.g. bi-weekly).

CPK (Correction Package)

Delivery packages of type Correction Package (CPK) are created for all new patch deliveries, e.g. version 1.0.1, and should be used to deliver bugfixes.

These delivery packages include only those objects that were changed since the previous patch delivery and are only created when necessary (e.g. emergency patch).

For more information on the add-on package types and how these are determined based on changes to the software component version, see Software Component Version .

4.2.14.1.4.4.3 Branching

> **Note**
>
> Keep in mind that there are some restrictions for using branches in your development processes, for example the merging of branches is not supported and has to be done manually. See ABAP Environment Specifics [page 869].

Development in the ABAP environment is done in software components. For each software component, different branches can be created that include a different list of commits. For each system, only one branch can be active at a time. Branches are used to separate changes from each other that shall or might not be delivered together (development vs. correction).

To separate development of new features from the development of bug fixes for the active version of a software component, you can use branches for different purposes:

- **Main Branch**: Implementation of new planned features that are to be delivered in the future (development codeline)
- **Maintenance Branch**: Implementation of unplanned bug fixes for the active version of a software component that is deployed to customer systems (maintenance codeline)

The main branch is the default branch of each software component and therefore already exists when the software component is created. This branch is not deleted throughout delivery of different software component versions.

A maintenance branch should be created for each support package level of a software component and is named accordingly, for example v1.1.0 for support package level 1 in release 1. Maintenance branches can be deleted once the successor support package level is the currently deployed support package level in customer systems.

→ Recommendation

Create a new maintenance branch with each new support package level, and thus with each new AOI and CSP package delivered for a software component.

For the first release, the main branch of software components is used to create a maintenance branch for the add-on build process. In subsequent versions, the development and correction code line are separated by using the main branch and the maintenance branch. See Use Case 2: One Development and Correction Codeline in a 5-System Landscape [page 899].

All corrections to a software component based on a certain support package level are released in the corresponding maintenance branch and need to be maintained as new commits in the main branch (double maintenance). See Double Maintenance of Corrections into Development [page 915].

For each ABAP system, only one branch can be active at a time. Therefore, in development system DEV and test system TST, the main branch of a software component is checked out. Whereas in correction system COR and quality assurance system QAS, the currently relevant maintenance branch is checked out.

4.2.14.1.4.4.4 Delivery Models

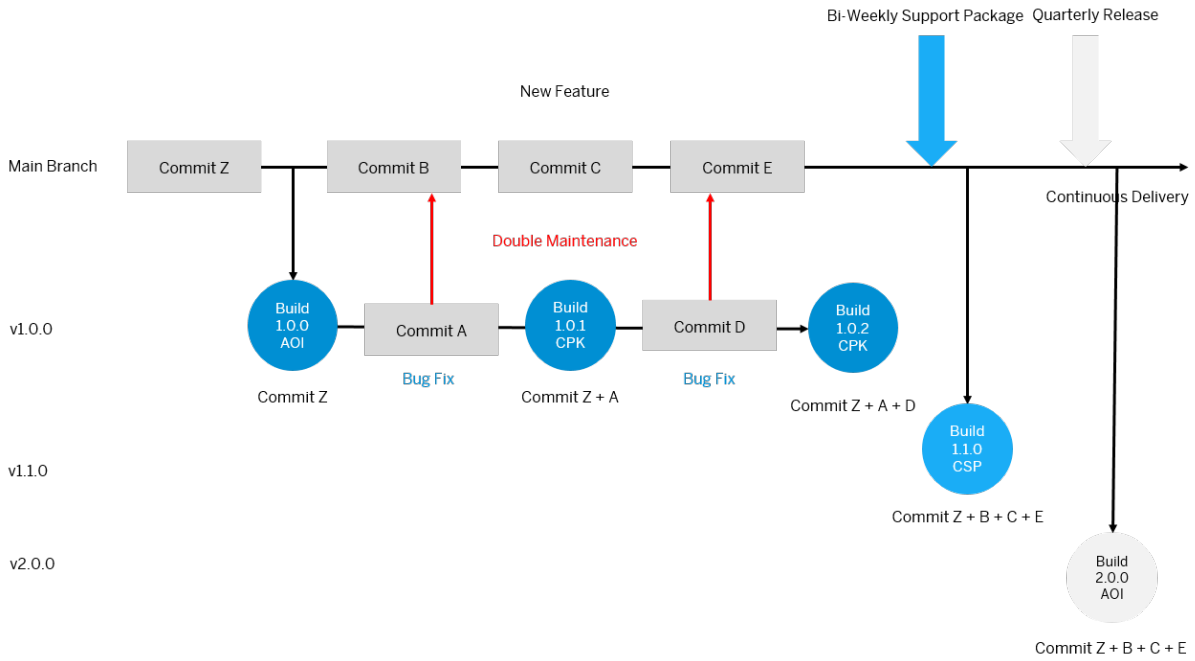
Based on the mentioned software component versioning principles, the add-on package types, and branching approach, you can use different delivery models.

→ Recommendation

You should choose the delivery model based on the existing processes in your development/shipment environment. Shorter delivery cycles have the advantage that new features are delivered more frequently. Therefore, we recommend following the continuous delivery approach whenever possible.

Continuous Delivery

With the continuous delivery approach, new features should be delivered as soon as possible: In this case, new features can be shipped with support packages more often than new releases of a software component. These short development cycles ensure that changes can be deployed on short notice and in smaller increments. See Continuous Integration and Delivery (CI/CD).



In this example, commit Z represents all the commits prior to the first build.

The maintenance branch v1.0.0 is used for bug fixes that must be manually maintained in the main branch, which requires double maintenance to make sure that the development code line is up to date.

Commit Z and commit C are new features implemented in the development code line on the main branch. Development of these features is done in the DEV system, testing in the TST system.

Commit A and commit D are small corrections that are implemented in the correction code line on the maintenance branch for support package level zero of software component release one.

Development of these bug fixes is done in the COR system, testing in the QAS system.

After releasing these corrections in the maintenance branch, you have to maintain them as well in the main branch in the DEV system to secure consistency of changes in the correction and development code lines. As a result, commit B and E are created in the main branch to reflect the corrections made in the maintenance branch (commit A and commit D).

The new support package build for version 1.1.0 is derived from the main branch and includes new features and bug fixes since the last support package level 1.0.0 has been released. With the new support package build for version 1.1.0 of the software component, a new maintenance branch v1.1.0 is created. All corrections for this support package level are developed on this branch.

For a new release, add-on version 2.0.0 is released. The resulting AOI package includes all features and bug fixes since the last AOI build for version 1.0.0 has been released.

Branch v1.1.0, that is based on the main branch, includes same commits as the main branch (Z, B, C, and E). For the build of CPK v1.0.2 and CSP v1.1.0, however, only the objects that have been changed since the previous CSP/AOI are included in the object list.

> ### Note
>
> With this delivery model, support packages not only include the corrections for the current support package level but also new features allowing shorter development cycles.

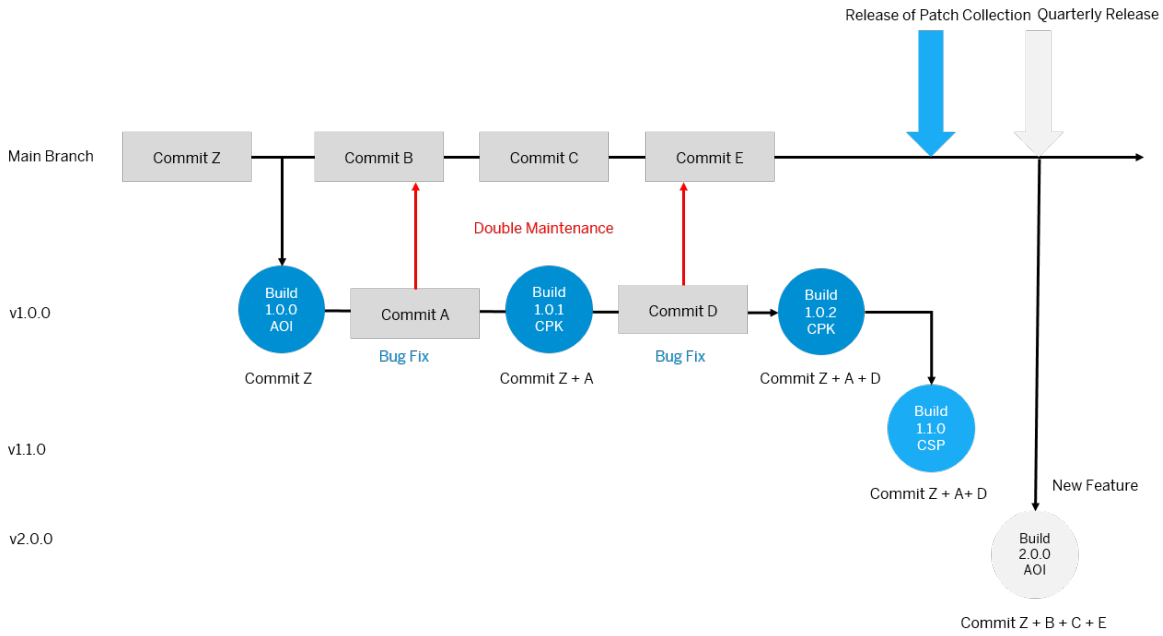
→ Recommendation

In a continuous delivery model, we recommend:

- Building and deploying a new CSP on a weekly or biweekly basis.
- Shipping new software component releases on a quarterly basis. These AOI packages contain every object that has been built and delivered with the support packages, including all the patches.

Quarterly Shipment

The quarterly delivery approach is recommended whenever longer test cycles are necessary and faster incremental development is not feasible. This might be required due to industry requirements, legal requirement etc.



Compared to the continuous delivery model, the maintenance branches for each support package level are not created based on the main branch but on the previous maintenance branch: e.g. branch v1.1.0 is based on v1.0.0.

That way, support packages are only used for the purpose of patch collections: only the corrections in commit A and D that were already delivered as patches in version 1.0.1 and 1.0.2 are included in support package 1.1.0.

ⓘ Note

With this delivery model, new features are only delivered with new release versions (AOI packages). The build for version 1.1.0 does not include commit C. As late as with the build for version 2.0.0, this feature is shipped. Consequently, new features take longer until they are available in production systems.

This delivery model is similar to processes commonly used to ship on-premise solutions where upgrades are much more complex and usually performed on a quarterly basis. Instead of shipping more often but in smaller packages, this approach promotes building larger shipment packages with a lot of new features.

Branch v1.1.0, that is based on branch v1.0.0, includes the same commits as branch v1.0.0 (Z, A, and D). For the build of CPK v1.0.2 and CSP v1.1.0, however, only the objects that have been changed since the previous CSP/AOI are included in the object list.

4.2.14.1.4.4.5 Reuse Software Components

Released APIs and API snapshots

Add-On products can consist of multiple software components. However, objects of one software component can't be used in another software component by default. Software components provide their functionality to other software components via explicitly released APIs. This means, you must set the API state of an object to *Released* if you want to use it from another software component.

Once you have released an object, we recommend not changing it incompatibly because this can lead to errors for its consumers. Released APIs need to fulfill certain stability and consistency criteria depending on the underlying release contract. To fulfill these criteria, any changes done to already released APIs need to be checked for compatibility. This is done using so called API snapshots that are created locally in development and test systems. For more information, see Released APIs and API Snapshots.

> ### Note
>
> The `abapEnvironmentAssemblePackages` step in the *Build* stage of the ABAP environment pipeline, which is used to assemble delivery packages while following the add-on delivery approach, will automatically create a new API snapshot for each semantic version. The created snapshot will be searched for at the beginning of the build of the next version and will then be set as check-relevant. Finally, the `API_COMPATIBILITY` check will be used to look for any incompatible changes on the piece list of the delivery, comparing the latest state of the released APIs in a software component to the state of the API facades in the API snapshot. With this, there is no need to manually create API snapshots in an add-on assembly system. For more information, see abapEnvironmentAssemblePackages and Software Assembly Integration (SAP_COM_0582).
>
> You also have the option to download the API snapshot for use in other systems. See Downloading API Snapshots.
>
> Performing manual actions (e.g. regenerate or delete) on the API snapshots created in the build system is not recommended. Instead, use ATC exemptions to mitigate compatibility check findings that are false-positive or cannot be resolved in time. For more information, see Working with ATC Exemptions. If your add-on build system is transient, the semantic version API snapshots won't be carried over from system to system and the compatibility checks will be skipped.

Software Component Relations

Software components must not have cyclic dependencies. For more information on software component restrictions, see Software Components.

You should however use software component relations to define access permissions and dependencies between specific software components. This approach of permitting usage across software components does not come with the same stability and consistency requirements as is the case with the release state on object level (which is also not targeted to specific software components). For more information, see Software Component Relations.

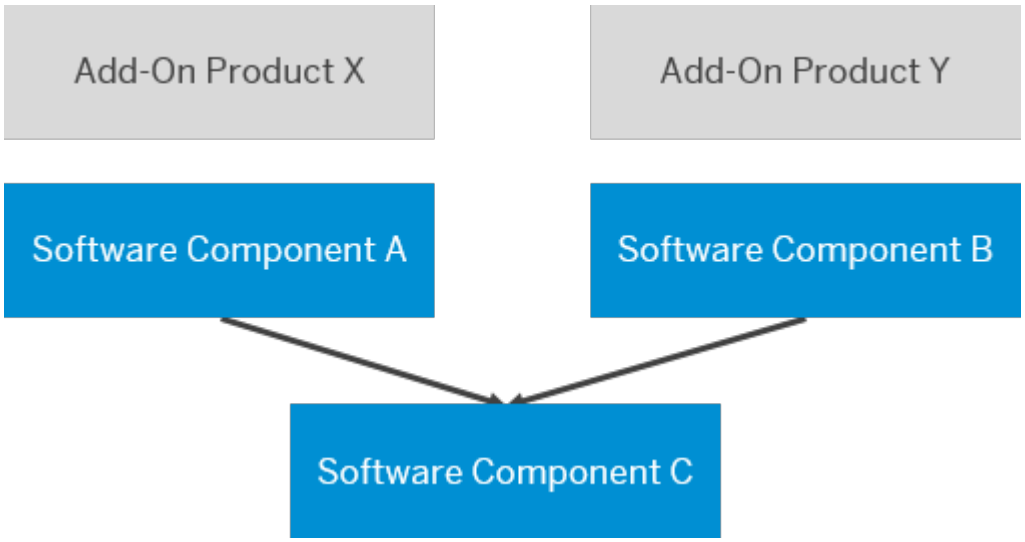
> ### Note
>
> The abapEnvironmentAssemblePackages step in the Build stage of the ABAP Environment Pipeline, which is used to assemble delivery packages while following the add-on delivery approach, will automatically

run the `SCR_CONSISTENCY ATC` Check. Refer to abapEnvironmentAssemblePackages and Software Assembly Integration (SAP_COM_0582).

The `SCR_CONSISTENCY` check will fail if undeclared dependencies between add-on software components are found. In such cases resolve the findings by defining the missing dependencies as part of a software component relation or removing the dependencies if unintended. For more information, see Software Component Relations. Use ATC exemptions to mitigate dependency consistency check findings that are false-positives or cannot be resolved in time. See Working with ATC Exemptions.

Software Component Dependencies in Add-ons

Having this in mind, you can structure development into reuse components that are used across several components.



In this example, one add-on product consists of a leading software component A and software component C, whereas another add-on product consists of a leading software component B and C. Software component A and B use released objects of software component C, and thereby depend on this software component.

In the add-on descriptor file, dependencies are reflected by the order of the components in the repositories list:

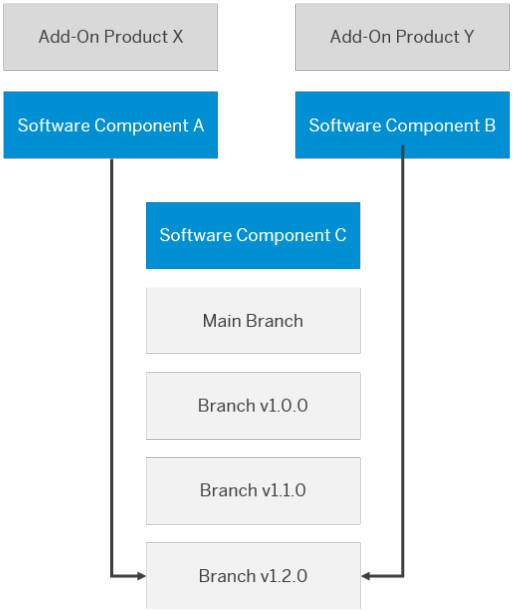
```
---
addonProduct: "/NAMESPC/PRODUCT_X"
addonVersion: "1.2.0"
repositories:
  - name: "/NAMESPC/COMPONENT_C"
    branch: "v1.2.0"
    version: "1.2.0"
    commitID: "7d4516e9"
  - name: "/NAMESPC/COMPONENT_A"
    branch: "v2.0.0"
    version: "2.0.0"
    commitID: "9f102ffb"
```

In this case, component `/NAMESPC/COMPONENT_C` is the reuse component. It has no dependencies to any other top components and is therefore listed first. This is important for the right order during software component import. `/NAMESPC/COMPONENT_C` needs to be imported before `/NAMESPC/COMPONENT_A` to avoid import errors. In addition to that, software components in the bundle can use different namespaces in the software component name.

→ Recommendation

In a software component bundle with a reuse component, this component is assembled whenever the software component version is defined for the first time in the add-on descriptor file. Therefore, make sure to define the correct software component version to prevent building the wrong software component version for `/NAMESPC/COMPONENT_C`.

We recommend following the same delivery model (identical shipment cycles) for all involved software components in a delivery scenario with a reuse component.



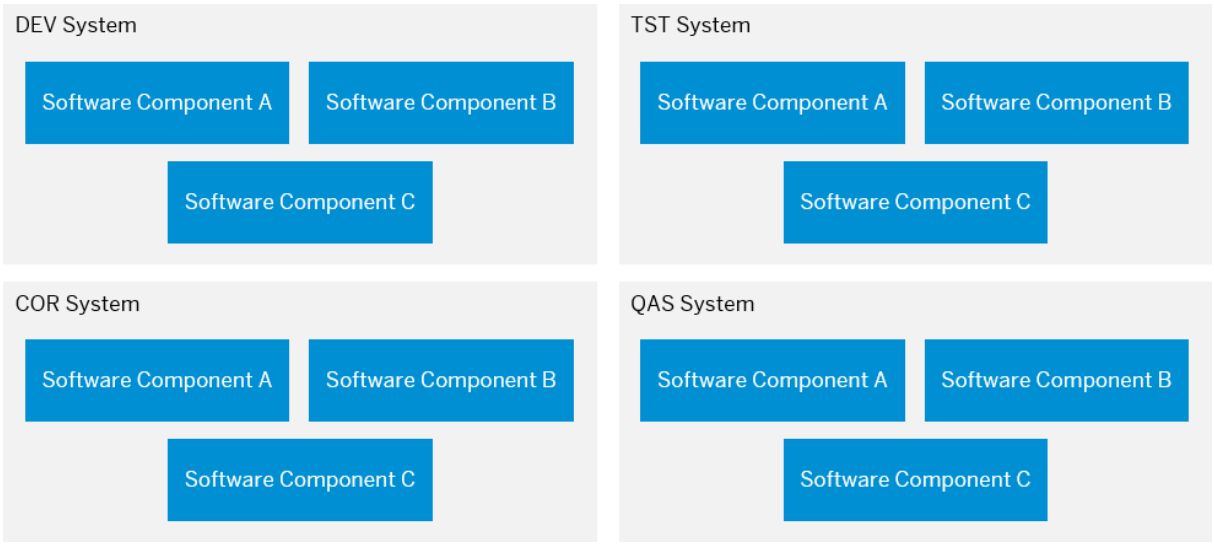
In this scenario, add-on product X and Y are based on the continuous delivery model and still undergo development. That means, for both add-on products, a new version can be shipped on a bi-weekly basis.

Consequently, the leading software components A and B refer to objects in the same branch of the reuse software component C, for example branch v1.2.0

Let's assume that development and correction of both software components is performed in one DEV and COR system. If there has to be a bug fix in reuse software component C, corrections to the corresponding software components A and B are still possible. This is due to the fact that branch v1.2.0 can remain checked out in software component C, while these changes are made.

The same applies to the development process: A new feature can be developed in leading software component A using the main branch of reuse software component C. Since only the main branch of reuse software component C is used in the DEV system, the development in the corresponding software components A and B can continue in parrallel.

This scenario is resource-efficient: only one system is required for development in system DEV and one for corrections in COR. With respect to the developer experience, it is more convenient to have all implementation components in one development system because everything is in one place and the whole system landscape is easily understandable.



The system setup regarding the testing and quality assurance for corrections depends on how you want to test the add-on products:

- Do you want to provide a test environment that is identical to a production system with the add-on product installed? Then you should establish one TST and one QAS system for each add-on product in the sense that only those software components used for each add-on product are imported into these systems.
- Do you want to reduce the number of systems without restricting the test environment? Then you only need one TST and one QAS system, where all software components are imported.

> → Recommendation
>
> The number of required systems for DEV, COR, TST, and QAS depends on the number of branches that are actively used in dev, test, and build processes. Whenever more than one branch is actively used, one ABAP system is required per branch because an ABAP system always checks out only one branch of a software component.

4.2.14.1.4.5 Delivery via Add-On or gCTS

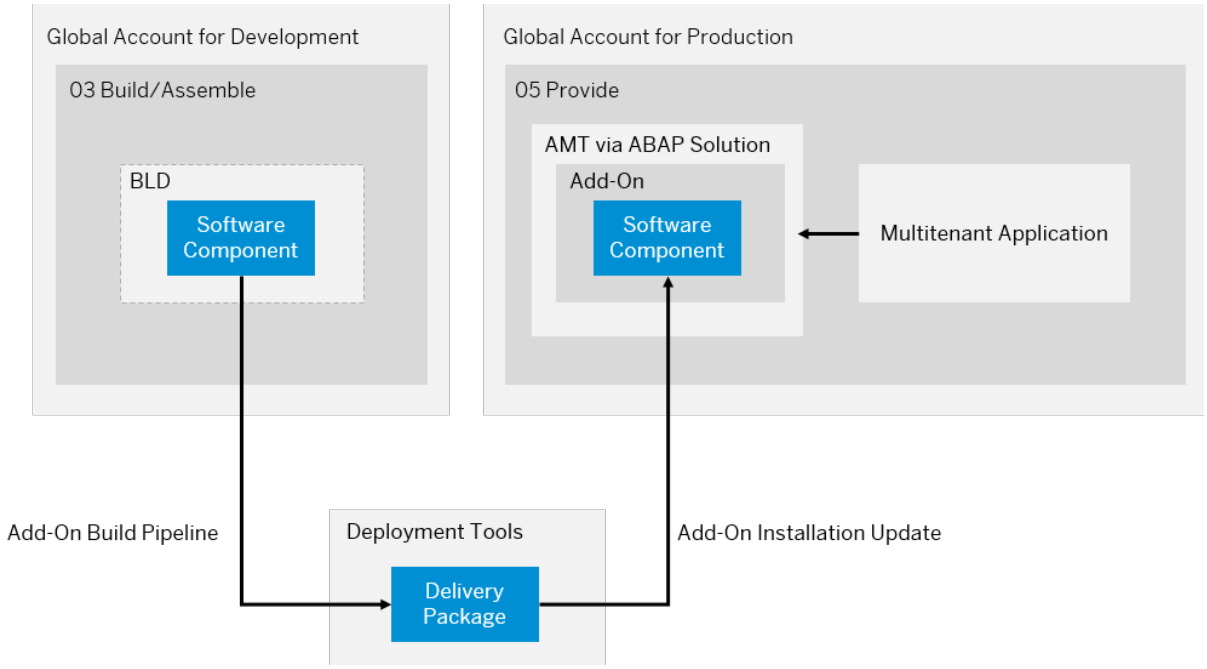
For SaaS solutions in the ABAP environment, software components including ABAP development objects have to be delivered to your production system. This is referred to as upstream process.

Afterwards, the SaaS solution is provided by offering a multitenant application that facilitates a subscription mechanism. This is referred to as downstream process.

There are multiple options to deliver software components to the systems. The delivery process described in this documentation mainly focuses on using add-on products as a means to deliver software components by installing or updating delivery packages in a system. See The Add-On Product .

As an alternative, you can choose a delivery process using the gCTS-transport-based import of software components. See Transport Mechanisms [page 862].

Add-On Build Delivery Process



An add-on is a bundle of software components. To create an add-on, a build process orchestrated by an add-on build pipeline has to be triggered.

With delivery tools, the software components are packaged into deliveries that are then further processed. Eventually, the deliveries are stored as delivery packages using the deployment tools provided by SAP.

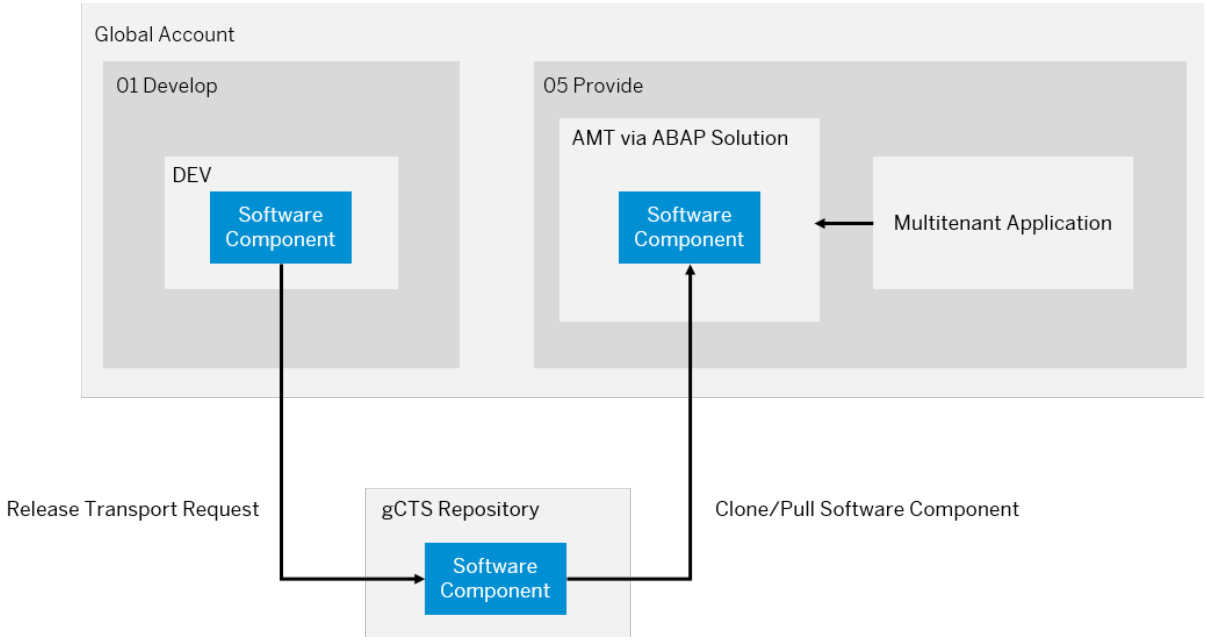
Add-on deployment is either performed by using an add-on installation during system provisioning or as an add-on update triggered in the *Landscape Portal* application.

Delivery packages are of different types, which means that the build and deployment of these packages differ accordingly. For example, the import of a software component for an add-on installation package or delta-import for correction packages.

BLD and AMT systems are based on different service plans: `abap/standard` for a plain ABAP system and `abap/saas_oem` for an ABAP system with installed add-on. The usage of these service plans is separated in two different global accounts for development and production purposes.

Making software components available across different global accounts is only possible by using the add-on delivery process. You can't use software components imported via add-on in the *Manage Software Components* app.

gCTS Transport Delivery Process



The gCTS-transport-based delivery process takes place in one global development account only. This is due to the fact that the underlying gCTS repository is only shared among systems in the same region in the same global account.

ABAP development objects are released with a transport request in a particular software component in the development system. The transport request release automatically exports the changes into an SAP-managed gCTS repository by creating a new commit. Once the changes are exported into the gCTS repository, they are centrally available and can be imported into any other systems in the same region in the same global account. By cloning a software component into a customer production system using the *Manage Software Components* app, implemented ABAP objects become available to customers subscribed to the multitenant application.

The customer production system AMT is provisioned during the initial subscription to the multitenant application. Since the SaaS solution needs to include the implemented software, software components must be cloned right after the first subscription.

	Add-On Build Delivery (Productive Build Pipeline)	gCTS Transport Delivery (Transport Mechanism)

System Landscape/Account Model	• ABAP systems for development purposes (development, test, demo) are of type `abap/standard` or `abap/abap_saas_oem`, and are provisioned in the global account for development based on a discounted development license • ABAP systems for production purposes (used by partner customers) are of type `abap/standard` or `abap/ abap_saas_oem`, and are provisioned in the global account for production based on a production license • Software components are made available across these different global accounts by using add-ons • For execution of the add-on build pipeline, an add-on assembly system BLD and add-on installation test system ATI are (temporarily) required	• All ABAP systems are of type `abap/standard` and are provisioned in the same global account • No additional ABAP systems, apart from already existing development and test systems, are required to facilitate the delivery • Single global account because gCTS repositories are only shared among ABAP systems in the same region in the same global account
Ease of Use	• Initial efforts to set up ABAP environment pipeline for add-on build • ABAP environment pipeline needs to be executed in a Jenkins CI/CD Server • Initial configuration effort and requires to set up Jenkins server infrastructure. See Infrastructure . • Resulting delivery packages are either installed as part of an add-on during system provisioning or updated centrally in the systems using the *Landscape Portal* application. See Perform Add-on Updates.	• No build process required; deployment triggered locally using *Manage Software Components* app • Changes to software components are released and exported to a central remote repository using transport requests • The *Manage Software Components* app, that is part of every ABAP system, is used to trigger the import of new changes locally for each system • As an alternative, software component import can be triggered by using the ABAP environment pipeline. See ABAP Environment Pipeline .

Quality Measures/Resilience	Add-on build pipeline for delivery to production provides a guided process with strict checks: • ABAP Test Cockpit checks before transport release • Optional: ABAP Test Cockpit checks as part of ABAP environment pipeline for continuous testing • ABAP Test Cockpit checks as part of ABAP environment pipeline for add-on build • Checks during add-on build • Add-on installation test • Check for minimum platform version during add-on installation/update • Checks for import conditions during add-on installation/update that prevent import of unwanted software component states	gCTS transport includes basic checks, for strict checks additional processes need to be established: • ABAP Test Cockpit checks before transport release • Optional: ABAP Test Cockpit checks as part of ABAP environment pipeline for continuous testing
Auditability/Traceability	• Reliable, secure, traceable, and automated process based on add-on build using ABAP environment pipeline • Resulting delivery packages created as part of add-on build are signed and stored permanently so that they can't be altered or deleted after a release • Import conditions are assigned to delivery packages so that it's technically verified, during add-on update/installation, that all prerequisite package requirements are met • Delivery packages can be traced back to a specific commit ID of a software component, also dependencies between delivery packages can be translated into dependencies between different commit IDs • Build logs are archived as part of the add-on build pipeline execution • Anonymization of build results (*Created by* information) • Add-on build pipeline guarantees that only tested changes are delivered to consumers	• Logs of software component import are not archived, only locally available in ABAP environment system • Commit IDs in software component branches can be deleted by accident if not contained in any other existing branch • Import dependencies between software components need to be taken care of manually, a specific import order must be adhered to • Software component states are only identified by a specific commit ID instead of a readable version number

Scalability	• Online deployment only possible with add-on-based delivery • With the *Landscape Portal* as a central tool for orchestration of software lifecycle activities, the add-on version can be updated centrally in multiple systems at the same time • Add-ons are automatically installed during system provisioning of ABAP systems and application content is immediately available in systems after subscription to the multitenant application	• Application content is initially not available: Software components need to be cloned locally into the provisioned systems using the *Manage Software Components* app. See Clone Software Components [page 3226]. **Note** You can use a separate test subscription that is not related to a consumer to trigger the creation of the system before the consumer subscribes to the solution. This initial system creation and import of the software components needs to be performed for each consumer production system. • Latest changes can be pulled locally into a system by using the *Manage Software Components* app. See Pull Software Components [page 3227]. • Software component states installed in a system can only be identified locally in the *Manage Software Components* app. **Note** Especially for SaaS solutions deployed using multiple ABAP environment systems, a gCTS-transport-based delivery process results in a lot of manual effort.
SAP Software Certification	Possible, check details here: • SAP Extension Suite Certification • https://www.sap.com/icc	Not possible
Certifications and Compliance of Solution	SAP Compliance Offerings available in SAP Trust Center. See https://www.sap.com/about/trust-center/certification-compliance.html.	
	Requires less effort to be compliant because a strict process, supported by SAP delivery/deployment tools, is provided.	Requires additional effort to to be compliant because a strict process needs to be established without SAP tooling support.

→ Recommendation

We recommend using the **add-on build delivery** process because it offers a bundling that can be used to implement a sophisticated delivery approach with separate codelines, versioning, and, as a result, different delivery package types. See Delivery Models [page 1276]. It also comes with a separation of ABAP system licenses for development and production purposes, with the development systems being available at a discounted rate.

Example scenarios: Long-term projects with frequent maintenance deliveries, expected growth of consumers over time, and adherence to specific standards/certifications.

The **gCTS transport delivery** approach is a more lightweight approach that might be used especially if a simple setup and quick readiness are important. Ease-of-use of the approach comes with disadvantages in auditability, traceability, scalability, and resilience. These trade-offs might be resolved by establishing additional manual process steps or documentation requirements.

If the multitenant application is configured with `tenant_mode = single`, a dedicated system is created per subscription. See ABAP Solution Service [page 1320]. In that case, gCTS delivery is only recommended if the SaaS solution is expecting a small set of customers or if the subscription is performed together with the customer. Otherwise, the effort for executing manual tasks, such as importing software components would be too high.

Example scenarios: Prototyping projects, quick time-to-market launch, in-house solutions, predictable number of consumers, and no expected exceptional growth rate

Once a first customer is using the SaaS solution and a first production system is provisioned, there is no migration path available to switch from one approach to the other, which means that systems would have to be deleted. However, it is likely is to start out with gCTS transport delivery in the prototyping phase of a project and establish an add-on build delivery approach once the initial version needs to be delivered and customer systems need to be created.

4.2.14.1.5 Develop, Test, Build

To develop and provide add-ons, you have to set up a suitable system landscape and Cloud Foundry environment account structure.

You need a partner contract to create the account structure. The accounts for the upstream (add-on development, test, build) and downstream (providing add-on as SaaS solution) parts of the process need to be separated from each other and therefore are handled in dedicated global accounts. We refer to these as global accounts for development and production account. For more information regarding the development license, see SAP PartnerEdge Test, Demo & Development Price List. To find out more about production licenses, see SAP PartnerEdge: Resources for OEM Partners.

→ Tip

For in-depth information about the system landscape/account model, check out System Landscape/Account Model [page 1260].

For the initial add-on delivery, you should create a development and test landscape following the guidelines mentioned in Concepts [page 1260]. Once backend and frontend development is completed, you can test

the add-on implementation in a persistent test system. Upon successful completion of this step, set up a CI/CD server along with a Jenkins pipeline to execute the add-on build allowing for an automated and efficient assembly. This automated pipeline can then be reused transforming all upgrade and update activities into a straightforward process.

Once the add-on build/assembly is finished, you can release the add-on for installation/update. First, the add-on build is installed in a test system in the global account for development to verify that it works correctly. If successful, you can provide the add-on to customers.

> ⓘ Note
>
> Besides using add-ons for delivering software components to production systems, gCTS transports can be used as an alternative approach. It is used for transporting software components between different ABAP systems.
>
> See Delivery via Add-On or gCTS [page 1282].

4.2.14.1.5.1 Access to Landscape Portal

The *Landscape Portal* acts as a central tool that allows you to perform lifecycle management operations, such as provisioning new consumers tenants, users, and more. See Landscape Portal.

As an operator, subscribe to the *Landscape Portal* application in the *05 Provide* subaccount in both the global account for development and global account for production. The `LandscapePortalAdminRoleCollection` needs to be assigned to the users in the default identity provider of the subaccount that you want to provide access to the *Landscape Portal* app. See Accessing the Landscape Portal.

> ⓘ Note
>
> Note: In the global account for development, you can use a booster (see Booster for Landscape Portal [page 1298]) to automatically set up a subaccount with a working Landscape Portal subscription.

> ⓘ Note
>
> In the Landscape Portal, you can access the provider system and choose the provider tenant (client 100). Via the provider tenant, you get access to Fiori Apps, for example the *Manage Software Components* app, which is used for importing software components via gCTS.
>
> See Delivery via Add-On or gCTS [page 1282].

4.2.14.1.5.2 Prepare

Before you can proceed with the development of the add-on, you have to perform several preliminary steps.

- Register the development namespaces before any system is set up. To start the actual add-on development process, you have to set up the system landscape and global account structure for development. For pipeline automation, set up a CI/CD server, in this case a Jenkins server.

- Create one or more software components that make up the add-on in the namespaces registered for you. In this scenario, namespaces are required for both the add-on product name and software components. You can register these namespaces with the development namespace/license keys tool.
- Purchase the SAP BTP, Cloud Foundry environment entitlements necessary for the account setup, which includes the service ABAP environment for development. The ABAP environment (service plan `saas_oem`), Application runtime, SaaS provisioning, ABAP Solution service as well as the Landscape Portal are used for providing the SaaS app.

  > → Recommendation
  >
  > We recommend using the browser-based IDE SAP Business Application Studio for UI development. In this case, additional configuration is required to enable developers to use this service.

Once you've completed these steps, you can start developing an add-on.

Prerequisites

- To register a namespace, you need an S-user in SAP One Support Launchpad with authorization to reserve a development namespace. See SAP note 1271482.
- To set up global accounts for development and poduction, you need two global accounts on the SAP Business Technology Platform with the corresponding entitlements for services and applications. See Entitlements and Quotas.
- To create ABAP instances, you have to set up the account structure in the global account for development, and assign entitlements for each subaccount. See Configure Entitlements and Quotas for Subaccounts.
- To develop UIs, you need an entitlement for SAP Business Application Studio and an assignment to a subaccount for development. See SAP Business Application Studio.
- To set up transports from the development to the test system, you need a test system and optionally a pipeline in a Jenkins CI/CD server that is provisioned using a Cx Server to automatically import new changes. See Jenkins and Cx Server .
- To set up add-on development, you need a development system.

4.2.14.1.5.2.1 Register a Namespace

Using a reserved namespace for add-on development and build is necessary for a unique add-on product and software component name.

> ⓘ Note
>
> You need to register a namespace before the first ABAP system is provisioned. To activate namespaces in already existing systems after system provisioning, see Maintain Namespaces.

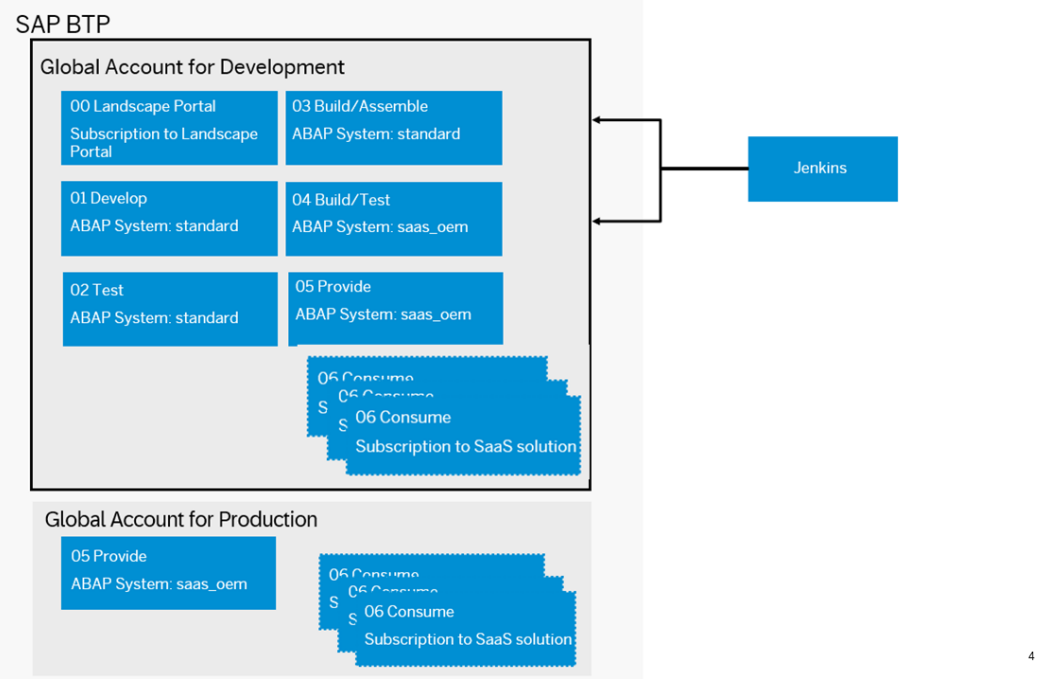
If you need a new S-user, get in touch with a user administrator in SAP ONE Support Launchpad.

As an S-user with authorization *Reserve Namespaces*, you have to reserve a namespace for your partner customer ID to register a namespace with the *Namespace* app. See Namespace Application.

Due to length restrictions of some objects, namespaces should have 5–8 characters. See SAP note 105132 and 395083.

4.2.14.1.5.2.2 Set Up a Global Account for Development

As a SaaS solution operator, you have to configure the global account for development (used for development, testing, and demo purposes).



→ Recommendation

We recommend the following subaccount structure:

- In the *00 Landscape Portal* subaccount, your subscription to the Landscape Portal is created. If required, the CI/CD service should also be subscribed in this subaccount. No Cloud Foundry environment is required.
- In the *01 Develop* subaccount, add-on development is performed in a permanent development system. See Development in the ABAP Environment [page 848].
- In the *02 Test* subaccount, the developed software components are tested after a successful import into a permanent test system.
- In the *03 Build/Assemble* subaccount, the add-on package assembly is performed in a transient assembly system that is created and deleted automatically by the build pipeline.
- In the *04 Build/Test* subaccount, the add-on product is installed and tested again.
- In the *05 Provide* subaccount, the add-on product is provided as a SaaS solution for testing purposes in the development phase.

> **Note**
>
> You can use a booster (see Booster for Landscape Portal [page 1298]) to automatically perform the setup of subaccounts 00 Landscape Portal, 03 Build/Assemble and 04 Build/Test. This includes the assignment of entitlements, creation of subscriptions and trust setup as described below.

> **Note**
>
> If you use gCTS instead of add-ons for delivering software components to production systems, the setup and usage of the following subaccounts is redundant:
>
> - *03 Build/Assemble* (used for the add-on assembly)
> - *04 Build/Test* (used for add-on installation test)
>
> Additionally, considering the availability of software components only in the same global accounts, you have to create the production systems as well as development and test systems in the same global account (only one global account is used).
>
> In the provider subaccount, the entitlement for an ABAP instance of service plan type `abap/standard` instead of `abap/saas_oem` needs to be available.
>
> See Delivery via Add-On or gCTS [page 1282].

You should configure a Cloud Foundry space in each subaccount. Dividing the development, testing, and assembling activities into different subaccounts allows for maximum flexibility. For instance, you may want to use different identity providers or consume different connectivity services during testing and development.

The ABAP systems that you use for development, testing, and add-on assembly are of type `abap/standard` and made available via entitlements. ABAP systems for add-on installation are of type `abap/saas_oem`. These service entitlements must be assigned by an administrator to different subaccounts, according to the following structure:

Global Account	Subaccount	Space	Services
Global Account for Development	01 Develop	Develop	1x abap/standard abap/hana_compute_unit (standard: 2) abap/abap_compute_unit (standard: 1)
	02 Test	Test	1x abap/standard abap/hana_compute_unit (standard: 2) abap/abap_compute_unit (standard: 1)

Global Account	Subaccount	Space	Services
	03 Build/Assemble	Build/Assemble	1x abap/standard abap/hana_compute_unit (standard: 2) abap/abap_compute_unit (standard: 1)
	04 Build/Test	Build/Test	1x abap/saas_oem abap/hana_compute_unit (standard: 2) abap/abap_compute_unit (standard: 1)
	05 Provide	Provide	1x abap/saas_oem abap/hana_compute_unit (standard: 2) abap/abap_compute_unit (standard: 1) Application Runtime abap-solution saas-registry xsuaa

Additionally, the following entitlements for SaaS application subscriptions are required:

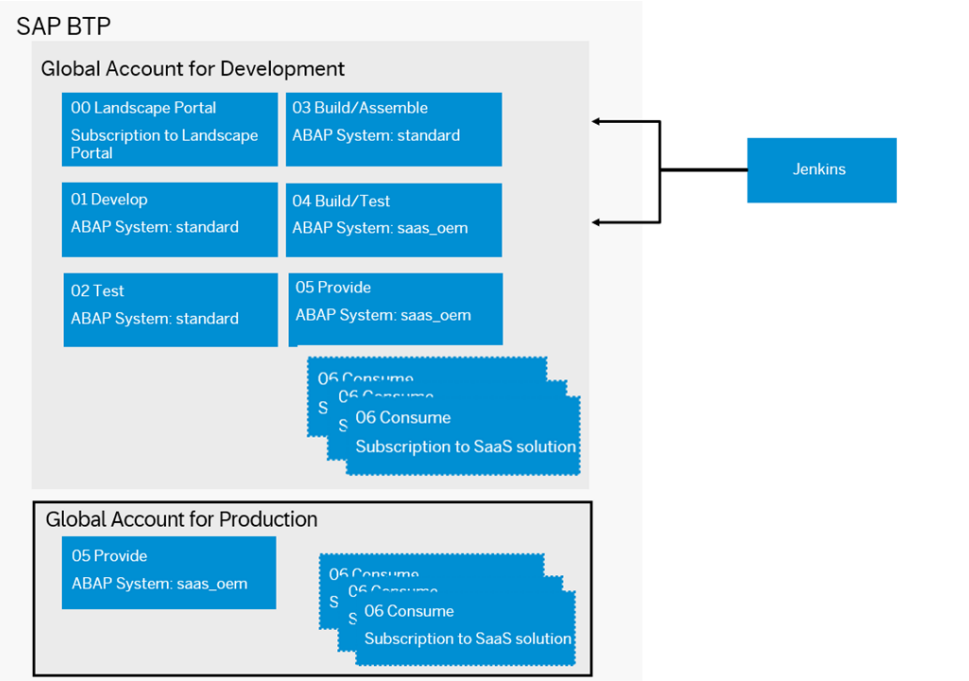
- SAP Business Application Studio for UI development. See SAP Business Application Studio [page 262].
- Web access for ABAP for access to systems during development phase. See Subscribing to the Web Access for ABAP [page 211].
- Landscape Portal to manage systems and tenants in the provider subaccount. See Landscape Portal.

If you want to integrate an existing corporate identity provider in the subaccounts of the global account for development for authentication/authorization, see Trust and Federation with Identity Providers [page 2538]. To restrict access based on certain criteria such as the IP address, you need to use the SAP Cloud Identity Services - Identity Authentication.

> → Tip
>
> For in-depth information about the system landscape/account model, check out System Landscape/ Account Model [page 1260].

4.2.14.1.5.2.3 Set Up a Global Account for Production

As a SaaS solution operator, you have to configure the global account for production.



→ Recommendation

We recommend the following subaccount structure:

In the *00 Landscape Portal* subaccount, your subscription to the Landscape Portal is created. If required, the CI/CD service should also be subscribed in this subaccount. No Cloud Foundry environment is required.

In the *05 Provide* subaccount, the add-on product is provided as a SaaS solution for production purposes in the production phase. The solution is consumed by your customers from consumer subaccounts.

In the provider context, the `ABAP environment (saas_oem)` service plan is used.

ⓘ Note

In the provider subaccount, the entitlement for an ABAP instance of service plan type `abap/standard` needs to be available.

Additionally, considering the availability of software components only in the same global accounts, you have to create the production systems as well as development and test systems in the same global account.

See Delivery via Add-On or gCTS [page 1282].

These provider ABAP instances allow flexible sizing, multitenancy, and the possibility to install an add-on product during provisioning.

For the provisioning of these ABAP systems of service plan `saas_oem`, another set of services comes into play: With the ABAP Solution Provider and the saas-registry service, you can provide your add-ons as SaaS solution offerings. See ABAP Solution Service [page 1320].

Note that these service entitlements must be assigned to different subaccounts, according to the following structure:

Global Account	Subaccount	Space	Services
Global Account for Production	05 Provide	Provide	abap/saas_oem abap/hana_compute_unit (standard: 4) abap/abap_compute_unit (standard: 1) Application Runtime abap-solution saas-registry xsuaa

Additionally, the following entitlements for SaaS application subscriptions are required:

- Web access for ABAP for access to systems during the development phase. See Subscribing to the Web Access for ABAP.
- Landscape Portal to manage systems and tenants in the provider subaccount. See Landscape Portal.

If you want to integrate an existing corporate identity provider in the subaccounts of the global account for production for authentication/authorization, see Trust and Federation with Identity Providers [page 2538]. To restrict access based on certain criteria such as the IP address, you need to use the SAP Cloud Identity Services - Identity Authentication.

> → Tip
>
> For in-depth information about the system landscape/account model, check out System Landscape/Account Model [page 1260].

4.2.14.1.5.2.4 Create ABAP Instances

As a SaaS solution operator, you have to create ABAP instances.

For development and testing in the development codeline, one system is provisioned in each of the development and test subaccounts. Use service parameter `is_development_allowed` to differentiate between development and production systems. The test and assembly systems must be productive to avoid changes being made to the add-on outside of the development system.

Global Account	Subaccount	Space	ABAP Instances
Global Account for Development	01 Develop	Develop	Create an ABAP instance (abap/standard) DEV Set parameter `is_development_allowed = true`

Global Account	Subaccount	Space	ABAP Instances
	02 Test	Test	Create an ABAP instance (abap/standard) TST Set parameter `is_development_allowed = false`

> **Note**
>
> You don't have to create an ABAP instance in the *03 Build/Assemble*, *04 Build/Test*, and *05 Provide* subaccount because the system is created automatically.

In the *03 Build/Assemble* account, assembly systems are provisioned by the add-on build pipeline. These systems are used to import the desired software components that are then packaged for add-on delivery (see Software Assembly Integration (SAP_COM_0582) [page 955]). As an operator, you don't have to provision these systems manually.

In the *04 Build/Test* subaccount of the global account for production, an add-on installation test system is automatically created. This ABAP environment service instance of plan `saas_oem` is provisioned with parameter `is_development_allowed = false`.

In the *05 Provide* subaccount of the global accounts for development and production, a customer production system is created automatically by the ABAP Solution Service during the first subscription. For more information on how to get started with your customer account, see Getting Started with a Customer Account in the ABAP Environment [page 184] and Creating an ABAP System.

Use service parameter `is_development_allowed` to differentiate between development and production systems. The test and assembly systems must be productive to avoid changes being made to the add-on outside of the development system.

Subscribe to the Web Access for ABAP to gain access to the SAP Fiori launchpad in all subaccounts of the global production account.

4.2.14.1.5.2.5 Set Up UI Development

As a SaaS solution operator, you have to set up SAP Business Application Studio for development.

As a developer user, you can then create an SAP Fiori dev space and generate UI projects.

> **→ Recommendation**
>
> For frontend development, we recommend using SAP Business Application Studio. See Develop an SAP Fiori Application UI and Deploy it to ABAP Using SAP Business Application Studio [page 1859].

4.2.14.1.5.2.6 Set Up Add-On Development

Learn how to set up add-on development by creating and importing software components. Furthermore, read about configuring ABAP Test Cockpit checks and check variants, as well as enabling transport blocking to fix issues early on during development.

Create and Import Software Components

To transport new developments from system to system, the add-on development is structured by software components. Software components are independent development containers.

To create a new software component for add-on development, as an add-on admin user, use the *Manage Software Components* app and create a new component of type `Development`. The name of the software component begins with a namespace that was activated in the development system. By default, all namespaces of the global account owner are enabled in the systems. See How to Create Software Components.

Clone the main branch of the software component to the development and test system.

Software components in the development and test system should always stay on the main branch. If you want to work on maintenance branches to develop bug fixes and other maintenance deliveries, create a dedicated hotfix development and test system (maintenance codeline).

> → Tip
>
> For in-depth information about versioning and branches, check out Versioning and Branches. In case you are creating software components with dependencies to each other, make sure to declare these dependencies in software component relations. Refer to Software Component Relations.

Configure ABAP Test Cockpit Checks, Check Variants

By default, a check variant `ABAP_CLOUD_DEVELOPMENT_PARTNER` is generated in ABAP environment systems. You should use this variant for ABAP Test Cockpit check runs or create a custom check variant based on it. See Creating ATC Check Variants. You can also create custom ABAP Test Cockpit checks. See Creating ATC Checks.

Custom ABAP Test Cockpit checks and check variants are created as part of a software component, whereas the default check variant is generated locally.

> ⓘ Note
>
> We recommend running ATC checks using at least those checks included in the `ABAP_CLOUD_DEVELOPMENT_PARTNER` check variant. Depending on your use case, it makes sense to add additional ATC checks to a custom check variant instead.

Configure ABAP Test Cockpit to Interrupt Transport Releases

We recommend enabling the blocking of transport releases in case of priority 1 (error) and priority 2 (warning) findings using the default check variant `ABAP_CLOUD_DEVELOPMENT_DEFAULT` or a custom check variant. You can enable transport blocking in the ABAP Test Cockpit Configurator app. See ABAP Test Cockpit Configurator.

Interrupting a transport release in case of severe findings can help to fix issues early on during development. The later errors are detected in the development process, the more costly it is to resolve them. See Launching ATC Check Implicitly.

4.2.14.1.5.2.7 Set Up Transport from Development to Test System

You can perform the import of your software components into a test system either manually by using the *Manage Software Components* app or in an automated way using the ABAP Environment Pipeline [page 1266].

Manual Import into Test System

As an add-on admin user, you can pull the latest released changes of a software component to the test system by using the main branch. You can test these changes in the test system independent from ongoing development. See Pull Software Components [page 3227] and ABAP Lifecycle Management [page 861].

Automatic Import into Test System with ABAP Environment Pipeline

> → Tip
>
> For in-depth information about the ABAP environment pipeline used for automatic import into a test system, see ABAP Environment Pipeline [page 1266].

As a DevOps engineer, you can configure the ABAP environment pipeline for an automated import of software components to test system TST. With the pipeline, the pulling of specified software components is automated and performed on a regular schedule.

The continuous testing scenario of the ABAP environment pipeline is described in detail in Continuous Testing on SAP BTP ABAP Environment .

4.2.14.1.5.2.8 Booster for Landscape Portal

One of the main use cases of the Landscape Portal is the building of product (add-on) versions. While the Landscape Portal hides much of the complexity involved in the process, it is required that a partner set up a suitable account model before starting.

The “Landscape Portal for SAP BTP ABAP Environment” booster is an automated process that automates the setup required to use the *Build Product Version* app in the Landscape Portal. Upon execution, it creates three new subaccounts corresponding to 00 Landscape Portal, 03 Build/Assemble and 04 Build/Test as described in Set Up a Global Account for Development [page 1291].

The three subaccounts fulfil the following roles:

- **00 Landscape Portal**: This subaccount will host your Landscape Portal subscription, which can be used for a wide range of lifecycle management tasks.
- **03 Build/Assemble**: This subaccount will host ABAP environment instances that are used to build new add-on product versions.
- **04 Build/Test**: This subaccount will host ABAP environment instances that are used to test the installation of new add-on product versions.

It is recommended to use a transient build system during the add-on build process. This means that with every build that is triggered, a new ABAP environment instance is provisioned and, after the process is finished,

it is deleted again. The booster also offers the possibility to provision such an instance during the booster execution, which can be later used during the build process.

You can start the booster from your global account for development. A wizard queries the user for the necessary data for the configuration of the subaccounts.

Prerequisites

- Your global account for development should be assigned the necessary entitlements:
  - Landscape Portal (1x standard)
  - Continuous Integration & Delivery (1x standard)
  - abap (2x standard, 2x abap_compute_unit, 4x hana_compute_unit)
  - Web Access for ABAP (1x default)

Procedure

1. Navigate to your global account for development in the BTP Cockpit.
2. Navigate to the *Boosters* tab.
3. Select the "Landscape Portal for SAP Business AI Platform, ABAP environment" booster and click *Start*.
4. Configure the *00 Landscape Portal* subaccount.
   1. Choose a subaccount name and subdomain.
5. Configure the 03 Build/Assemble subaccount.
   1. Choose a subaccount name and subdomain.
   2. Choose a Cloud Foundry organization and space name.
   3. Specify a technical user from your identity provider that shall be used for the provisioning of build systems.
   4. Specify whether to provision a build system.
   5. If a system shall be provisioned, choose a system ID.
6. Configure the *04 Build/Test* subaccount.
   1. Choose a subaccount name and subdomain.
   2. Choose a Cloud Foundry organization and space name.
   3. Specify a technical user from your identity provider that shall be used for the provisioning of installation test systems.
7. Configure the user groups.
   1. Specify a list of administrator users. These users will be assigned authorizations as described in *Results* down below.
   2. Specify a list of viewer users. These users will be assigned authorizations as described in *Results* down below.

SAP ID Service is configured as the identity provider in all three subaccounts. If you wish to use different trust settings for different subaccounts, you can adjust the configuration as necessary after the booster execution.

> **Note**
>
> Currently, the Landscape Portal can only be used with SAP ID service as the application identity provider.

Results

After successful execution, your global account for development should contain three new subaccounts with the following properties:

00 Landscape Portal

- Entitlements: Landscape Portal (1x standard), Continuous Integration and Delivery (1x default)
- Subscriptions: Landscape Portal, Continuous Integration and Delivery
- Administrator users have role collections *Subaccount Administrator* and *LandscapePortalAdminRoleCollection*.
- Viewer users have role collections *Subaccount Viewer*.

03 Build/Assemble

- Entitlements: abap (1x standard, 2x abap_compute_unit, 2x hana_compute_unit)
- Subscriptions: -
- Cloud Foundry is enabled
- 1 Cloud Foundry space is created
- If configured, an ABAP environment instance is provisioned
- Administrator users have role collection *Subaccount Administrator*, are *CF Org Managers* and *Space Managers*
- Viewer users have role collection *Subaccount Viewer*, are *CF Org Members* and *Space Developers*
- The specified technical user is granted the same authorizations as the viewer user group.

04 Build/Test

- Entitlements: abap (1x standard, 2x abap_compute_unit, 2x hana_compute_unit), Web Access for ABAP (1x default)
- Subscriptions: Web Access for ABAP
- Cloud Foundry is enabled
- 1 Cloud Foundry space is created
- Administrator users have role collection *Subaccount Administrator,* are *CF Org Managers* and *Space Managers*
- Viewer users have role collection *Subaccount Viewer*, are *CF Org Members* and *Space Developers*
- The specified technical user is granted the same authorizations as the viewer user group.

After executing the booster, you maintain the same technical user(s) in the *Credentials* section of the *Build Product Version* app.

You are then ready to configure one or more pipeline templates, see Configure a Pipeline Template.

When configuring a template, reuse the resources created by the booster as follows:

For the "Prepare System" stage, reuse the subaccount/CF space from the **03 Build/Assemble** subaccount. If you provisioned a system during the booster execution, specify its instance name in the template so that it will be used.

For the "Integration Tests" stage, reuse the subaccount/CF space from the **04 Build/Test** subaccount. If you provisioned a system during the booster execution, specify its instance name in the template so that it will be used.

4.2.14.1.5.3 Develop

Both the initial implementation and any further development of the add-on are performed in the development system of the development subaccount.

You can use various SAP technologies at this point, such as the ABAP RESTful application programming model (RAP) for modeling and implementing business objects and the browser-based IDE SAP Business Application Studio for developing SAP Fiori UIs. Depending on the business use case, some additional development effort may be necessary to implement suitable launchpad content, Identity & Access Management artifacts, or communication management artifacts.

Once you've completed these development activities, the solution is ready to be tested in a suitable test system.

Prerequisites

- For ABAP development, you need a developer user using ABAP development tools for Eclipse. See Getting Started as a Developer in the ABAP Environment [page 241].
- For UI development, you need a developer user using SAP Business Application Studio. See Develop an SAP Fiori Application UI and Deploy it to ABAP Using SAP Business Application Studio [page 1859].
- For custom code migration, you need a business user that is assigned the business role based on business role template `SAP_BR_IT_PROJECT_MANAGER`, and a communication arrangement instance for `SAP_COM_0464`. See Custom Code Analysis [page 2941].

4.2.14.1.5.3.1 ABAP Development

As a developer user, implement your custom business services with the ABAP RESTful application programming model. See ABAP RESTful Appliation Programming Model. Maintain business catalogs (see Identity and Access Management (IAM) [page 977]) and communication scenarios (see Communication Management) to expose services to business users and communication users. See SAP Business Technology Platform, ABAP Environment Learning Journey, and ABAP Environment Community. Also consider starting out with a free tier option for the ABAP environment to get hands-on development experience. See Trial Accounts and Free Tier.

Identity and Access Management

SAP Fiori applications and business services are represented by IAM apps and can be used to define the necessary authorizations. In an IAM business catalog, you bundle multiple IAM apps and their predefined authorizations, for example, for a specific business area.

Additionally, you can define business role templates to make it easier for administrators to find the relevant business catalogs to create business roles. See Identity and Access Management (IAM) [page 977].

Communication Management

> **Note**
>
> Use customer-managed communication scenarios as a design time description to store technical information, such as inbound and outbound services and their service type, for example OData or SOAP by using the `create_by_comm_arrangement` method.
>
> Instead of coding against destination names, using communication scenarios allows consumers of the SaaS solution to create communication arrangements based on provided communication scenarios. Therefore, you don't have to create specific destinations in the consumer subaccount.

> **Restriction**
>
> The ABAP environment only supports one communication configuration layer intended to be used by the consumer. It's currently not possible to enforce a communication configuration exclusively managed by the service provider within the consumer tenant.

Services with the following protocols supported for **inbound communication** can be included as part of customer-managed communication scenarios:

- HTTP
- SOAP
- RFC Internet/Cloud Connector

> **Restriction**
>
> RFC is currently not supported for inbound communication in consumer tenants.

For **outbound communication**, the following protocols are supported:

- HTTP (Internet)
- HTTP (Cloud Connector)
- SOAP (Internet)
- SOAP (Cloud Connector)
- RFC (Internet)
- RFC (Cloud Connector)

See Developing External Service Consumption.

Business Configuration

Business configuration plays a major role in SaaS solutions. It refers to a predefined set of configuration options that affect its functionality and behavior. See Business Configuration in SAP BTP ABAP Environment (1): Overview and BC Maintenance Apps.

To maintain these configuration options, you have to create dedicated apps using the ABAP RESTful Application Programming Model. See Create a Business Configuration App for Factory Calendar Using the ABAP RESTful Application Programming Model.

Using the business configurations API, you can register business configurations. These business configurations are then displayed in the list of all maintainable business configurations in the SAP Fiori App Maintain Business Configurations, if the user has the necessary authorizations for the service of the business configuration. See Business Configuration Maintenance Object Cloud Platform API [page 1049].

Providing Released APIs

APIs can be released for stable custom development based on different release contracts. Refer to Released APIs.

Developer Extensibility Enablement

Developer extensibility can be provided for APIs by defining the corresponding API state for system-internal use (contract C1) and use in ABAP for Cloud Development. This allows the APIs to be used across ABAP for Cloud Development software components and objects. Refer to Releasing Development Objects.

Key User Extensibility Enablement

Using SaaS solutions, you can provide key user extensibility for system-internal use (contract C1) and use in key user apps. This allows customers who use the solution to extend it to their specific requirements.

As a developer user, you have to implement key user extensibility in the development system in the Partner Development tenant (client 100) using ABAP Development Tools.

See Providing Business Add-Ins [page 1361] for guidance on how to prepare business add-ins (BAdIs) so that customers can add their own business logic in the solution by using the *Custom Logic* app.

See Configuring Predefined Custom Fields [page 3031] for guidance on how to equip your SaaS solution with support for customer-specific extension fields.

Stability of Released APIs

Released APIs, such as BAdIs or predefined custom fields, must only be changed compatibly. Otherwise, for example, runtime errors might occur or an upgrade might fail. To ensure that released APIs are not changed incompatibly, run an ATC compatibility check. This check uses snapshots that are created using the *Manage API Snapshots* app and which are set to check-relevant, to check whether incompatible changes have been made to the released API. For more information, see Manage API Snapshots [page 2936].

Using Released API

You can use the object-level API state to release APIs for use in cloud development, without targeting a specific software component. With software component relations on the other hand, you can also provide access permissions, without stability requirements. In any case, such dependencies between software component should always be declared – especially in the case of add-on components. See Software Component Relations.

Multitenancy

With the ABAP environment, you can build multitenancy-enabled SaaS solutions. To do so, the add-on implementation has to follow certain guidelines. See Multitenancy Development Guideline [page 1353].

→ Recommendation

If the add-on implementation is aligned with the development guideline for multitenancy, we recommend configuring your solution by setting `tenant_mode = multi` so that the same ABAP service instance is used for multiple consumers.

→ Tip

For in-depth information about multitenancy, check out Multitenancy [page 1272].

4.2.14.1.5.3.2 UI Development

SAP Fiori applications are developed in SAP Business Application Studio on top of business services in the ABAP development system and then deployed to the ABAP development system to be part of the same software component as backend artifacts.

As a developer user, once business services are implemented as UI services, you can create SAP Fiori elements apps using SAP Business Application Studio. See Develop an SAP Fiori Application UI and Deploy it to ABAP Using SAP Business Application Studio [page 1859].

ⓘ Note

To grant business users access to an app, the consumer's administrator creates corresponding business roles. Moreover, to influence the layout for specific user groups. the administrator can maintain corresponding space and page templates. It's recommended to provide business role templates with business catalogs and corresponding space and page templates as part of an add-on. This simplifies the process and speeds up the configuration of the Fiori launchpad for each consumer.

As described in Scoping Space and Page Templates [page 1870], space and page templates need to be scoped manually in the development system. However, these are scoped automatically in consumer tenants.

4.2.14.1.5.3.3 (Optional) Code Migration from On-Premise

Optionally, you can migrate existing custom ABAP code for add-on development purposes. This custom code migration process analyzes your existing code for cloud-readiness. See How to Check your Custom ABAP Code for SAP BTP ABAP Environment and Custom Code Analysis [page 2941].

After adapting the code and making necessary changes, you can migrate the code via abapGit. See How to Bring your ABAP Custom Code to SAP BTP ABAP Environment.

4.2.14.1.5.4 Test

Once development has been completed, you can test the solution in the test subaccount that you have previously set up. The software components that make up the solution are imported into a provisioned test

system, while any additionally required configuration has to be carried out by you. This may entail creating suitable business roles, maintaining communication arrangements, or other administrative tasks performed via the SAP Fiori launchpad of your ABAP system.

You can also use a CI/CD server and a Jenkins pipeline to automate the test process. This allows you to schedule regular tests and to be notified as soon as issues arise within the solution.

If the solution is successfully tested and works correctly, you can proceed with the add-on build.

Prerequisites

- For testing in SAP Fiori launchpad in your ABAP environment, you need a business user in the test system that has the required authorizations to use the *Manage Software Components* app as well as authorizations that are required as a test user.
- For testing in the ABAP Test Cockpit, you need a developer user using ABAP Development Tools. See Getting Started as a Developer in the ABAP Environment [page 241].
- (Optional) For running ATC checks as part of the ABAP environment pipeline, you have to create a pipeline in a Jenkins CI/CD server. See Jenkins and Custom Jenkins Setup .

4.2.14.1.5.4.1 Test in the ABAP Environment SAP Fiori Launchpad

Import Software Components

Before testing new developments in a software component in the test system, as an add-on admin, you have to import the latest changes from the remote repository. After you have cloned a software component into the test system TST, you can import the latest changes by pulling the software component in the *Manage Software Components* app. See Pull Software Components [page 3227].

Create and Assign Business Roles

Before creating and assigning business roles, you have to make sure that business users are already available. They can be created manually or automatically. See User Provisioning [page 983].

To test the developed business services, as a test user, create business roles from the role templates in the test system and assign them to your user. See Maintain Business Roles [page 3068].

Create Launchpad Space and Pages for Business Roles

To enable navigation to custom UIs via tiles, enable launchpad spaces and pages. Add spaces to the relevant business roles and add the needed SAP Fiori launchpad tiles to those spaces. Finally, enable spaces for business users in the system launchpad settings. See How to Create Spaces and Pages for a Business Role [page 3083].

Create Communication Arrangements

To test inbound and outbound communication, create communication arrangements in the test system TST based on the implemented communication scenarios. See How to Create a Communication Arrangement [page 2998].

> **Note**
>
> Depending on whether you want to use an authentication method for outbound communication that requires a business user context (Oauth2 SAML Bearer Assertion, Oauth2 User Token Exchange, JWT Principal Propagation), you need to configure a destination in a communication arrangement instead of maintaining credentials by using an outbound communication user. See Supported Protocols and Authentication Methods and Creating a Destination.

You can integrate the test system TST with on-premise systems. See Integrating On-Premise Systems [page 1214].

In the subaccount of test system TST, the subaccount for testing, you can assign the Cloud Connector administrator role collection to the Cloud Connector administrator user to operate the data transmission tunnels used by the Cloud Connector.

Create Business Configuration

As a test user, you can adjust business configuration objects in the *Maintain Business Configurations* app to change and influence the system behavior. See Custom Business Configurations App [page 2970].

Configure Key User Extensibility

Key user extensibility that is enabled in the SaaS solution can be configured and consumed in test systems.

> **Note**
>
> Key user extensibility provided in a SaaS solution can only be configured in tenants of particular types, for testing purposes in tenants of type Test Tenant.
>
> These tenant types are provisioned in non-development systems, such as test system TST or quality assurance system QAS, where development is not allowed (`is_development_allowed = false`). The tenants are created independently from a subscription to the SaaS solution by using the *Landscape Portal* application. See Manage Test Tenants.

As a test user in a Test Tenant (client >= 200), you configure key user extensibility in a test system.

- See Custom Logic (Deprecated) [page 3025] for guidance on how to use the *Custom Logic* app to create and maintain custom logic for business add-ins (BAdIs).
- See Configuring Predefined Custom Fields [page 3031] for guidance on how to configure predefined custom fields to customize applications and their UIs.

4.2.14.1.5.4.2 Test in the ABAP Test Cockpit

Manual Testing During Development

With the ABAP Test Cockpit, you can run a set of checks (check variants) on software component or package level. See Checking Quality of ABAP Code with ATC and ABAP Test Cockpit in the Cloud – What is already possible.

To fix and revalidate ABAP Test Cockpit findings, as a developer user, you can run ABAP Test Cockpit checks on developed software components explicitly via ABAP Development Tools in the DEV system. See Launching ATC Check Run from the Project Explorer and Launching ATC Check Run Explicitly.

If transport blocking in case of ABAP Test Cockpit findings is configured (see Set Up Add-On Development), ABAP Test Cockpit checks are executed implicitly. See Launching ATC Check Implicitly.

Continuous Testing Using ABAp Environment Pipeline

> → Tip
>
> For in-depth information about the ABAP environment pipeline used for continuous testing, see ABAP Environment Pipeline [page 1266].

To schedule a regular execution of ABAP Test Cockpit checks for a software component in the test system, you can use a CI server and pipeline to automate this process. You can reuse the previously described setup of a transport from development to test system using the ABAP environment pipeline. See Set Up Transport from Development to Test System.

As a DevOps engineer, configure the ABAP environment pipeline by using a static and preconfigured system. With the pipeline, the following steps are then automated, triggered by the pipeline execution of a Jenkins administrator:

- Pulling the specified software components/Git repositories
- Running the configured ABAP Test Cockpit checks

The continuous testing scenario of the ABAP environment pipeline is described in detail in Continuous Testing on SAP BTP ABAP Environment .

4.2.14.1.5.5 Build

As an add-on administrator, you have two options for triggering the add-on build process. The recommended approach is to use the Build Product Version app in the Landscape Portal to configure and execute an add-on build pipeline, leveraging the CI/CD service on SAP BTP. Alternatively, you can implement your own pipeline manually and run it on an existing CI/CD server. In both cases, a successful pipeline execution will result in a new product version that can be installed.

The add-on build pipeline runs through multiple steps:

- For the add-on assembly, an assembly system is created in the 03 Build/Assemble subaccount. All software components that are part of the add-on product are imported into the system.

  > ### Note
  >
  > The import in the assembly system is enabled by communication scenario API for Managing Software Components (SAP_COM_0948).

- Before the add-on build, it is possible to execute ATC checks and AUnit test runs in the assembly system.

  > ### Note
  >
  > The ATC checks in the assembly system are enabled by communication scenario ABAP Test Cockpit - ATC Run Integration (SAP_COM_0901). The AUnit test runs in the assembly system are enabled by communication scenario ABAP Unit Test Integration (SAP_COM_0735).

- During the add-on build, delivery packages corresponding to included software component versions are created. For the add-on product version, a target vector is created and published in test scope. See Target Vector

  > ### Note
  >
  > The build process is enabled by communication scenario Software Assembly Integration (SAP COM 0582).

- To ensure that the add-on provisioning works correctly, an installation test system is provisioned in the 04 Build/Test subaccount. The new product version is installed using the target vector published in the build stage.

  > ### Note
  >
  > The installation test system uses the service plan `saas_oem`, enabling the add-on installation.

- Once the build and test installation have been completed successfully, the target vector is published in production scope.
- After a successful build, all ABAP systems used are deprovisioned (unless otherwise specified) and the add-on is technically available for deployment to the ABAP environment.

With any new add-on, it is required that you first register the add-on product and any global accounts you wish to use before building your initial product version.

> ### → Tip
>
> For in-depth information about the ABAP environment pipeline used for add-on build, check out ABAP Environment Pipeline.

Prerequisites

- Using the Build Product Version app requires an existing CI/CD subscription in the same subaccount as the Landscape Portal subscription (recommended option). If you used the Booster for Landscape Portal [page 1298] to set up your landscape, then this is already configured.
- Using a custom pipeline configuration and Jenkins server requires the following (not recommended):
  - To execute the pipeline, you have to set up a Jenkins CI/CD server and you need an S-user in SAP One Support Launchpad with user management authorization. See Jenkins , Custom Jenkins Setup , and SAP note 1271482.
  - To configure the pipeline, you need to create a configuration repository and implement the add-on build scenario. See Build and Publish Add-on Products on SAP BTP ABAP Environment .

4.2.14.1.5.5.1 Set Up Add-On Build

The following section explains how to build the initial version of the add-on product. The result is a delivery of type AOI (Add-on Installation) for the initial release 1.0.0 of both the software component and the add-on product.

The add-on build process is orchestrated by an automated pipeline. Before the pipeline can be configured, the following steps need to be performed.

Add Technical Platform User to Space in Subaccount *03 Build/Assemble*

As an operator, assign a technical Cloud Foundry platform user as a space developer in subaccount 03 Build/Assemble. This technical user will be used to provision the assembly system. See Creating New Space Members and Assigning Space Developer Roles to Them.

> **Note**
>
> If you used the Booster for Landscape Portal [page 1298] to set up your landscape, then this is already configured.

Add Technical Platform User to Space in Subaccount *04 Build/Test*

As an operator, assign a technical Cloud Foundry platform user as a space developer in the space in subaccount 04 Build/Test. This technical user will be used to provision the installation test system. See Creating New Space Members and Assigning Space Developer Roles to Them.

> **Note**
>
> If you used the Booster for Landscape Portal [page 1298] to set up your landscape, then this is already configured.

Create a Technical Communication User for Access to AAKaaS

As part of the add-on build process, the Add-on Assembly Kit is used. With AAKaaS, the on-premise tooling is now available as a cloud service that is used by the build pipeline.

To access the service that is offered via SAP Support Launchpad, a technical communication user is used. See SAP note 2532813.

In the *Support User Management* app in SAP ONE Support Launchpad, as an S-user, request a new technical communication user. After the technical communication user has been generated, activate the user.

> **Note**
>
> To create technical communication users in SAP Support Launchpad, you have to assign the user administrator function to the logged on user.
>
> Make sure that this technical communication user is assigned to the customer number under which the ABAP environment tenants are licensed and for which the development namespace was reserved.

See SAP note 2174416 for more information on how to create and activate technical communication users.

Add credentials to credential store

The credentials of the three technical communication users need to be maintained in the Build Product Version app. See Maintain Credentials.

If you are using your own Jenkins server, the Jenkins administrator needs to add the credentials of the three technical communication users to the Jenkins Credentials store.

Register Add-On Product for Installation in Global Accounts

The registration of a new add-on product is a manual step. Your add-on product should only be installed in ABAP systems in your global accounts for development and production. Therefore, the product needs to be created and global accounts need to be registered with SAP using Landscape Portal. See Register Product.

Configure ABAP Environment Pipeline

As a DevOps engineer, you have the choice between using the Build Product Version app in the Landscape Portal or creating a pipeline configuration for the add-on build scenario from scratch.

We strongly recommend using the Build Product Version app, as it greatly reduces both the initial configuration and the maintenance effort:

- the user-friendly tooling hides the complexity of configuring a pipeline definition and removes the need for an external repository,
- the app automatically delegates the execution of the pipeline to the CI/CD service on SAP BTP, removing the need for a partner-owned Jenkins server.

Creating a custom pipeline configuration allows you to modify parameters of the ABAP Environment pipeline that are not exposed in the Build Product Version app; however, the app should cover all common use cases.

> **Note**
>
> Currently there are two features available in the pipeline that the app does not support:
>
> It is not possible to define a manual confirmation step before the target vector is published productively. This option allows you to perform integration tests in the ATI system before your final release decision. The pipeline executed by the app will simply check if the new version was installed successfully and will then automatically proceed to the publishing step.

Build Product Version app (recommended)

The Build Product Version app allows you to configure the pipeline directly in a Fiori UI. See Build Product Version.

As a Landscape Portal Administrator, configure the Release Delivery pipeline template to be able to build your initial version. You can also configure the Test Release Delivery template in case you wish to perform a test execution, without releasing the new version.

Manual Configuration

As a Jenkins administrator, create a new pipeline in Jenkins pointing to the Jenkins file of the pipeline configuration for the add-on build scenario. See Configuration .

The add-on build scenario of the ABAP environment pipeline is described in detail in Build and Publish Add-on Products on SAP BTP ABAP Environment .

4.2.14.1.5.5.2 Build the First Add-On Version

Create Maintenance Branch

> → Recommendation
>
> For each release and support package level, as an add-on admin, create a maintenance branch that is used for development of bug fixes and maintenance deliveries. See Versioning and Branches [page 1272].

In test system TST, create branch `v1.0.0` that is based on the main branch.

Build New Product Version

Each version contains an add-on version number – 1.0.0 for the initial delivery – as well as the included software components, such as the corresponding names, branches, and software component versions to be used for the add-on build. You may also choose specific software component commit IDs (as seen in the Manage Software Components app) and select which languages should be included in the new product version. Add-on version numbers may not be used more than once.

Build Using Product Version App (recommended)

New product versions for your registered add-on products can be de ned directly in the Build Product Version app. The initial product version will always be of type “Release Delivery”. See Create a New Product Version.

The corresponding pipeline is executed automatically when the Build Product Version button is clicked on in the app. If the execution is successful, the initial version of the add-on will then be available for installation.

Build Using Manual Configuration

If using a custom pipeline configuration and your own Jenkins server, create an add-on configuration file (`addon.yml`) in the corresponding Git repository. This file includes the metadata of the add-on product that

is being built, such as versioning information and the included software component versions. Please follow the best practices on how to define the `addon.yml` file. See Add-on Descriptor File .

Once the repository is updated, trigger the pipeline execution on your Jenkins server. If your pipeline includes manual confirmation steps, you will have to follow its progress and react accordingly. You may use the add-on installation test system ATI to confirm the success of the add-on installation before confirming its release. You can also use the system for additional tests, similar to the steps described in Test in the ABAP Environment SAP Fiori Launchpad. For testing in a consumer-like environment, you can create tenants of type Test Tenant using the Landscape Portal application. See Use Test Tenants.

> → Tip
>
> For in-depth information about versioning and branches, check out Versioning and Branches [page 1272].
>
> To learn how software lifecycle management in the ABAP environment works with software components, see Basic Concepts and Terms [page 861].

> ⓘ Note
>
> Please make sure that the add-on product version to be published is properly tested before confirming the release decision. This includes testing in SAP Fiori launchpad and the ABAP Test Cockpit. See Test in the ABAP Environment SAP Fiori Launchpad [page 1305] and Test in the ABAP Test Cockpit [page 1307].
>
> During add-on build a new semantic version API snapshot will be automatically created for each add-on software component. In the assembly system BLD semantic version snapshots are automatically created and set to check-relevant during add-on build. Do not perform manual actions on API snapshots in such systems. For the initial software component version 1.0.0 no predecessor version is available and thus compatibility checks will not be performed due to the missing API snapshot. See Released APIs and Snapshots.

Check Add-on Build Result

Use the *Check Product Version* app in Landscape Portal to check whether the product version, its components, and respective packages are ready for delivery. See Check Product Version.

Download API Snapshot for use in other systems

In case of released APIs, download the API snapshot created for the initial version 1.0.0 from the add-on build system BLD. The snapshot is created automatically during the add-on build. Afterwards, upload the snapshot into the test system TST and the development System DEV. Set the new API snapshot as check-relevant so that it will be used as a reference for API compatibility checks.

> ⓘ Note
>
> The API snapshot will also be uploaded into the correction system COR and quality assurance system QAS once the maintenance branch is checked out for the first time, usually during the first patch version creation.

4.2.14.1.6 Order and Provide

After the initial add-on version has been built, you must deploy and provide it as a SaaS solution so that the add-on can be consumed by customers via a subscription in the SAP BTP cockpit.

4.2.14.1.6.1 Deploy

In order to offer a multitenant application based on an ABAP add-on product, a multi-target application (MTA) is developed and deployed to the Cloud Foundry environment on SAP BTP. The MTA integrates the different services required for this scenario, such as the ABAP Solution service, responsible for provisioning ABAP systems, tenants, and users on demand. For more in-depth information, see Multitenant Applications [page 1317].

Once you've configured your multitenant application, you can deploy it once for testing purposes in the global account for development. After testing the subscription process during this development phase, the multitenant application can be deployed to the global account for production. See System Landscape/Account Model.

After the multitenant application has been deployed for production purposes, the SaaS solution is ready for commercialization.

You have two options for implementing and deploying the application. The recommended approach is to use the Maintain Solution app in the Landscape Portal. See Maintain Solution.

Alternatively, you can create your own multitenant application. This allows you to modify parameters that are not exposed in the Maintain Solution app; however, the app should cover all common use cases. See Developing Multitenant Applications in the Abap Environment.

gCTS Delivery

If you use gCTS instead of add-ons for delivering software components to production systems, you have to configure the multitenant application manually. In the configuration of the ABAP solution service the value for the add-on product name must be left empty. This is currently not supported by the Maintain Solution app. See Delivery via Add-On or gCTS.

Prerequisites

• To configure the sizing of a SaaS solution, you have to determine the expected load per region by using the Technical Monitoring Cockpit. See Technical Monitoring Cockpit (Cloud Version).

• To implement and deploy a multitenant application for a SaaS solution, you have to assign the necessary entitlements in the provider subaccount, for example for the ABAP Solution service. See Multitenant Applications [page 1317] and Entitlements and Quotas.

4.2.14.1.6.1.1 Sizing

A multitenant application has some central sizing properties that dictate the scope and metric of your offering, or in other words, how many resources will be available for each consumer.

Tenant Limit: Before deploying your application, you must decide on a tenant limit. Once that number of consumer tenants is reached in a system, a new system will be provisioned for the next consumer. If you choose a limit of 1, you will be defining a single tenant offering with one consumer per system.

> → Tip
>
> For in-depth information about multitenancy, see this section.

Additionally, you need to define the sizing of each provisioned ABAP system. The sizing is defined by the number of ABAP compute units (runtime) and HANA compute units (persistence) each system is assigned. These values are reflected by the parameters size_of_runtime and size_of_persistence. For more information, see Creating an ABAP System [page 198].

For multitenancy offerings, there's no sizing/quota per customer. You must decide on an overall sizing depending on the expected load in a region. You can use the Technical Monitoring Cockpit to assist you in this.

> ⓘ Note
>
> If the quota for a system is exceeded, you can request a resizing of the ABAP runtime or persistency depending on the needs of the SaaS application.

4.2.14.1.6.1.2 Using the Maintain Solution App

Configure the solution

As a Landscape Portal administrator, you can implement the multitenant application using the *Maintain Solution* app in the Landscape Portal. Create a new solution using:

- the previously registered product,
- the product version that shall be installed initially ("latest" will always install the newest available product version),
- the usage mode "prod", unless you do not intend to deploy a productive application at the end,
- the consumer tenant limit and system sizing values that you have decided on.

For more information regarding solutions, seeCreate Solution.

Deploy the solution for test purposes

After creating the solution, it is recommended to perform a test deployment to validate the subscription process. Use the *Maintain Solution* app to create a deployment configuration for your solution:

- Target the Provide space in the 05 Provide subaccount in your global account for development. You need to specify the corresponding CF API Endpoint, CF Organization Name and CF Space Name.
- Choose the domain type "shared". Since this is not a productive deployment, it is sufficient to use one of the available app domains provided by SAP.

- Since you are deploying to a shared app domain, a unique route prefix is needed to uniquely identify your application endpoint.
- Maintain the subdomain of the subaccount where you intend to test the subscription process.

For more information regarding deployment configurations, seeCreate Deployment Configuration.

Test the solution

You can test the multitenant application by subscribing from a consumer subaccount created in the global account for development. See Subscribe to Multitenant Applications Using the Cockpit.

Once the subscription is successful, you may also want to test the initial user onboarding process. First, assign the onboarding role collection, which includes the SolutionAdmin role, to your user. Afterwards, access the application via the consumer subaccount and trigger the user onboarding. For more details, please see the section on the productive tenant onboarding.

Deploy the solution for production purposes

After successfully testing the subscription process you can proceed to a productive deployment of your solution. Use the *Maintain Solution* app to create a second deployment configuration for your solution:

- Target the Provide space in the 05 Provide subaccount in the global account for production. You need to specify the corresponding CF API Endpoint, CF Organization Name and CF Space Name.
- Choose the domain type “custom”. It is recommended to use a custom domain for your productive application, such that consumers can access it via a recognizable URL.
- It is recommended to use a wildcard route in conjunction with custom domains. This will route all requests towards that domain to your application.

To set up a custom domain, please follow the steps outlined in Configuring Custom Domains or Get Started with the Custom Domain Manager. After registering the custom domain in your provider subaccount, it becomes available as a private domain (see Private domains) for route creation in any of the Cloud Foundry spaces of the subaccount. Once you have created the production deployment configuration, you can trigger it using the *Deploy* action, seeDeploy Solution. Once deployed, your solution should appear in the service marketplace on SAP BTP Cockpit for subaccounts within your global account for production.

4.2.14.1.6.1.3 Configuring a Solution Manually

You can configure the multitenant application manually, which allows you to finetune your solution beyond the options offered in the *Maintain Solution* app. In this case, you must configure the individual project files and deploy the project using the corresponding command line tools.

In the following sections, the general steps are described. For a description of multitenant applications in general and more detailed configuration options, see Multitenant Applications [page 1317].

Configure the solution

It is recommended that you initially configure your solution using the Maintain Solution app and then use the Download functionality to obtain the application descriptor file. Since the download is available on deployment configuration level, it is necessary to create both a solution and a deployment configuration.

Follow the steps detailed in Using the Maintain Solution app (see Order and Provide [page 1313]), creating both the solution and the deployment configuration for the test phase. Once configured, use the Download function to obtain the application descriptor. Finally, clone the reference solution linked below and replace the standard descriptor with the generated one. This results in a project that is already configured, making it the perfect starting point to implement your modifications.

The reference solution provided by SAP can be found here: GitHub - sap-software/abap-saas-reference-solution: Template of an ABAP SaaS Reference Solution allowing the SAP Partner to deploy a solution containing approuter and XSUAA dependencies .

> **Note**
>
> gCTS Delivery: If you use gCTS instead of add-ons for the delivery of software components to production systems, the value for the add-on product name in the application descriptor has to be empty. See Delivery via Add-On or gCTS [page 1282].

Deploy the solution for test purposes

After configuring the multitenant application, it is recommended to perform a test deployment to validate the subscription process.

Your project should already have the correct deployment configuration, as you maintained this in the Maintain Solution app before generating the application descriptor. As a result, you can directly build and deploy your project.

Use the Cloud MTA Build Tool and the Cloud Foundry CLI .

Once deployed, your solution should appear in the service marketplace on SAP BTP Cockpit for subaccounts within your global account for development.

Test the solution

You can test the multitenant application by subscribing from a consumer subaccount created in the global account for development. See Subscribe to Multitenant Applications Using the Cockpit.

Once the subscription is successful, you may also want to test the initial user onboarding process. First, assign the onboarding role collection, which includes the SolutionAdmin role, to your user. Afterwards, access the application via the consumer subaccount and trigger the user onboarding. For more details, please see the section on the Initial User Onboarding [page 1336].

Deploy the solution for production purposes

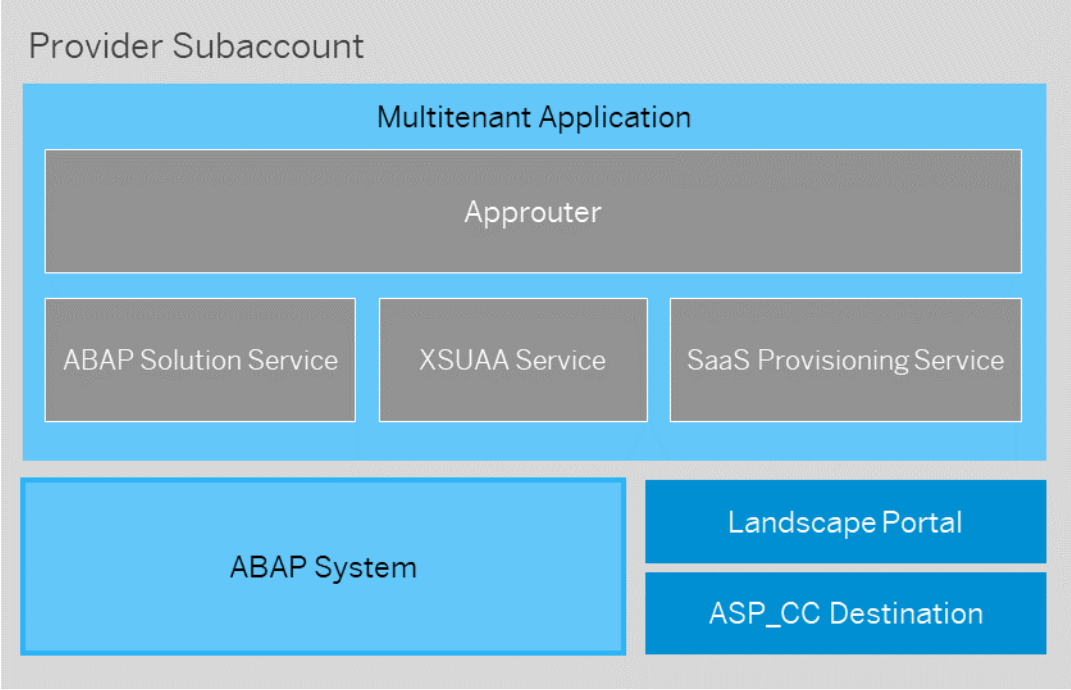
After successfully testing the subscription process, you can proceed to a production deployment of your solution.

You can use MTA extension descriptors to configure the required values for deployment to different targets. This allows you to deploy the same project to different subaccounts. See Multitenant Applications [page 1317].

Build and then deploy your project. Once deployed, your solution should appear in the service marketplace on SAP BTP Cockpit for subaccounts within your global account for production.

4.2.14.1.6.1.4 Multitenant Applications

To make the software components in the ABAP System available for multiple different consumers, a multitenant application is deployed into the provider subaccount. This application can then be accessed by different tenants through a dedicated URL.



For the multitenant application, different services and apps are used:

- **Approuter application**: The application router is the single point-of-entry for the multitenant application and used to forward tenant-specific requests to the corresponding tenant in the ABAP system. See Application Router.
- **ABAP Solution Service**: ABAP Solution service is required to create multitenant-enabled, ABAP environment-based SaaS solutions. Instead of a service binding from the approuter application to the ABAP environment business service, the approuter is bound to an ABAP Solution service instance. The service orchestrates the creation of ABAP systems upon consumer subscription as well as tenant provisioning/decommissioning. Requests routed via the approuter application to the ABAP solution service instance are automatically proxied to the corresponding ABAP tenant.

- **SAP Authorization and Trust Management service (XSUAA)**: The `xsuaa` service instance acts as an OAuth 2.0 client to the multitenant application, and to the `ABAP Solution` service instance. The SAP Authorization and Trust Management service lets you manage user authorizations and trust to identity providers used for the multitenant application. The service instance is created with a corresponding security configuration. See SAP Authorization and Trust Management Service in the Cloud Foundry Environment.
- **SaaS Provisioning Service**: The `SaaS Provisioning` service allows application providers to register multitenant applications and services in the Cloud Foundry environment in SAP Business Technology Platform. After a SaaS Provisioning service instance is created, the multitenant application becomes available for subscription in a consumer subaccount.
- **ABAP System**: ABAP systems and tenants used to provide the multitenant application are not created manually but are managed by the ABAP Solution service: Upon first subscription the ABAP system is created as well the consumer tenant in the system. To be able to create the system, the ABAP solution service uses credentials of a technical platform user that is defined in the ASP_CC destination. Depending on the configuration of the ABAP solution, for every subsequent subscription either a new tenant is created in the same ABAP system (`consumer_tenant_limit` > 1) or a new system is created for every new subscription (`consumer_tenant_limit` = 1). A new ABAP system is also created if all the existing ones are "full" because the consumer tenant limit that you specified was reached. If you're using the *Manage Consumer Subscription* app, please be aware that a consumer subdomain reservation counts toward that limit, too. Note that an ABAP system can only be deleted if no consumer tenants exist.
  Consumer subscription-based multitenancy is only possible in those systems in which you provide your software as a service (SaaS) solution to consumers, i.e. your production and test systems. It is not available in development systems.
  See Creating an ABAP System.
- **Landscape Portal**: The Landscape Portal acts as a central tool to allow service providers to perform lifecycle management operations such as add-on updates, provisioning new consumers as new tenants, and more. A subscription to the Landscape Portal in the globalaccount where the multitenant application is deployed is required for the subscription flow to work.
  See Landscape Portal.
- **ASP_CC Destination**: An `OAuth2Password` destination pointing to the Cloud Foundry Cloud Controller API is required for the ABAP Solution service to create and access ABAP service instances. Credentials of a technical platform user, that is assigned as a space developer in the space where the multitenant application is deployed, are maintained.

Multitarget Application

The multitenant application is deployed as a multitarget application (MTA), which is logically a single application comprised of multiple parts created with different technologies.

The MTA describes the desired result in an mta.yaml file which may be extended with mta extensions (.mtaext) using the MTA model which contains MTA modules, MTA resources and interdependencies between them. Afterwards, the MTA deployment service validates, orchestrates, and automates the deployment of the MTA, which results in Cloud Foundry (CF) applications, services and SAP-specific contents. For more information about the Multitarget Application model, see the official The Multitarget Application Model specification. For more details on how to use it in the CF Environment, which is the case here, see Multitarget Applications in the Cloud Foundry Environment.

A complete reference example of the multitenant application MTA is provided in GitHub/SAPSamples: ABAP SaaS Reference Solution for ABAP solution provider

MTA Parameters

Parameters are defined at the beginning of the mta.yaml file, so that the actual structure of the MTA model does not have to be changed but can be extended via extension files. The parameters are referenced in the subsequent sections of the MTA using the syntax `${parameter-name}`. For more details on parameters, see Parameters and Properties.

- **Freely defined parameters:**
  - app-domain: The domain that should be used for the approuter and therefore the SaaS application. In our example, this is the ${default-domain} (equals to the cfapps-domain of the current landscapes) for a development deployment; for production it should be the custom domain (so here: `mydomain.com`).
  - appname: The value that should be used as xsappname and solution name (here: `abap-saas-reference-solution`).
  - route-prefix: The value that should be used as a prefix in the route of the approuter, allowing it to deploy the same MTA to the same landscape muliple times using the default landscape domain.
  - addon-product-name: Name of add-on product to be installed
  - provider-admin-email: Provider administrator e-mail address used for provisioning of ABAP systems
  - saas-display-name: Display name of SaaS application shown in subscription tile
  - saas-description: Description of SaaS application shown in subscription tile
  - tenant-mode: Whether to provide a `'single'` or `'multi'` tenancy solution
- **Parameter with semantics to MTA deployer:**
  - enable-parallel-deployments: If set to `true`, deploys multiple modules at the same time.

MTA Extension Descriptors

The parameters defined in the MTA can be adapted for the corresponding landscape using MTA extension descriptor files. These files extend the original MTA descriptor file (mta.yaml) and provide certain deployment specific configurations/parameters/values. The descriptor files have the file extension *.mtaext and are used during the deployment using CF CLI with MultiApps plugin, see MultiApps CF CLI Plugin .

For more information on MTA extension descriptors, see: Defining MTA Extension Descriptors.

The extension descriptor files are usually in the folder “extensions” next to the mta.yaml file.

Depending on the stage of implementation of the multitenant application, a different approuter configuration is used. This is controlled via different extension descriptor files used during MTA deployment:

- Configuration for **Development Phase**:
  For development, test, and demo purposes, MTA deployment can be done using the `${default-domain}` as `app-domain` and a `route-prefix`, which will avoid collisions due to the default domain. We recommend defining the specific routes that shall be used later for consumer subscriptions in the approuter MTA module and as redirect-uris in the XSUAA oauth2-configuration (refer to oauth2-configuration (Custom Option)). Additionally, we suggest setting the property `XS_APP_LOG_LEVEL` on the approuter module to `debug` to ensure that there is more detailed logging output.
- Configuration for **Production Phase**:
  To provide a solution to applications consumers in a production scenario, the MTA deployment should be done using a custom domain configured with the app-domain parameter (`app-domain`) and an empty route-prefix (`route-prefix`). While using the custom domain, we recommend defining a wildcard route ("`*.${app-domain}`") as part of the approuter MTA module (refer to oauth2-configuration (Custom Option)). In this case you can make use of the same wildcard for defining redirect-uris in the XSUAA oauth2-configuration

MTA Build and Deploy

Before MTA deployment, the MTA project is built using the Cloud MTA Build Tool (MBT) . Afterwards the deployment-ready multitarget application archive (*.mtar file) is deployed in combination with the corresponding MTA extension descriptor file using the Cloud Foundry CLI with the MultiApps plugin: `cf deploy <path to .mtar file> -e <path to MTA extension descriptor>`

Related Information

Installing the cf CLI

MultiApps CF CLI Plugin - Download and Installation

MultiApps CF CLI Plugin - Usage

4.2.14.1.6.1.4.1 ABAP Solution Service

Sample Code

```
- name: abap-solution
  type: org.cloudfoundry.managed-service
  parameters:
    service: abap-solution
    service-plan: standard
    service-name: ${appname}-abap-solution
    config:
      name: ${appname}
      consumer_id_pattern: ${consumer-id-pattern}
      plans:
        - plan_name: ${plan-name}
          addon_product_name: ${addon-product-name}
          size_of_runtime: ${size-of-runtime}
          size_of_persistence: ${size-of-persistence}
          provider_admin_email: ${provider-admin-email}
          consumer_tenant_limit: ${consumer-tenant-limit}
          sap_system_name: ${sap-system-name}
          usage: ${usage-mode}
      xs-security:
        xsappname: xs-app-${appname}
```

ABAP Solution Service is used to enable multitenancy for ABAP Environment systems: Instead of creating the ABAP system manually it is created by the ABAP Solution service during the first subscription of an application consumer or once the consumer tenant limit for all viable systems has been reached.

Credentials provided in the ASP_CC destination are used by ABAP Solution Service to authenticate at the Cloud Foundry Cloud Controller API endpoint and to create required ABAP service instances.

The service plan used for created ABAP service instances is depending on the `addon_product_name` parameter: `abap/standard` is used for a system without add-on, `abap/saas_oem` is used for a system with installed add-on. Provisioning parameters like `sap_system_name`, `provider_admin_email` and sizing-relevant parameters can be defined.

After a new system is created, a tenant for the application consumer is provisioned. The business type (see Tenant Business Types) used for provisioning of the tenant depends on the `usage` parameter: In case of `usage = test`, the tenant is created as `Partner Customer Test` tenant. Whereas with `usage = prod`, the tenant

is created as `Partner Customer Production` tenant. Parameter `tenant_mode` controls if per subscription a new tenant is created in the same system and with `consumer_tenant_limit` the number of tenants per system is configured.

ABAP System: saas_oem plan

This service plan is like the standard service plan of the ABAP service (see Create an ABAP System), but it is used as "backing service" for a partner-provided SaaS solution. In comparison to a standard service plan of the ABAP service, there is one main differences: An add-on product (see The Add-on product) which is installed during system provisioning can be specified.

- Parameter: `addon_product_name`
  (specifies which ABAP add-on to install during provisioning)
- Parameter: `addon_product_version`
  (if - for testing purpose - another add-on version instead of the most recent released version shall be installed)

ABAP Solution Service Parameters

Name	Data Type	Description	Purpose	Updateable
name	string	Name of the solution as defined by the provider. Must be unique in the scope of the Global Account (or globally if this information is not available)	Used to identify the solution and will be passed through to ABAP System (saas_oem plan)	No
sap_system_name (optional)	string	Name of the system as defined by the provider. If a value is supplied, the new system(s) will be created with this parameter.	Passed through ABAP System	No
addon_product_name (optional)	string	Registered name of the add-on product	Passed through ABAP System (saas_oem plan)	No
addon_product_version (optional)	string	Version of the add-on product to be installed. A released product version needs to be defined. If you do not specify the parameter, the latest released product version will be used during the add-on installation. If you specify other versions than allowed, the add-on installation and therefore also the system provisioning and consumer subscription will be unsuccessful.	Passed through ABAP System (saas_oem plan)	Yes (updated value applies for new systems only)

Name	Data Type	Description	Purpose	Updateable
consumer_tenant_limit	int	Maximum number of consumer tenants in the multitenant ABAP system. If the consumer tenant limit is exceeded, a new multitenant system will be created. consumer_tenant_limit >= 1 **Note** A recently deleted tenant will be counted towards the limit, in case it is restored during the grace period. After the 30-day grace period is up, it will no longer be considered for the limit.	Passed through ABAP System (saas_oem plan)	Yes
size_of_runtime	int	Default sizing (ABAP runtime) of abap system(s) created/provisioned by solution	Passed through to ABAP System	Yes (updated value applies for new systems only)
size_of_persistence	int	Default sizing (ABAP persistence [HANA]) of abap system(s) created/provisioned by solution	Passed through to ABAP System	Yes (updated value applies for new systems only)

Name	Data Type	Description	Purpose	Updateable
tenant_mode (deprecated)	enum (string): - single - multi	Tenant Mode of the solution	Decides whether a customer will have a tenant in - a dedicated system (single) - a shared system (multi) **single:** will trigger a system creation and a tenant onboarding **multi** will trigger a system creation (only if necessary) and a tenant onboarding The parameter will be passed through to ABAP System (saas_oem plan).	No
consumer_id_pattern	string (regex)	String containing a regular expression with a capturing group. The **subdomain of the consumer** is matched against this regular expression. The value of the first capturing group is used as consumer id.	To allow the provider to group his consumer subaccounts based on a self-chosen consumer identifier: e.g. the subdomains of consumer subaccounts always include a static part that identifies the application consumer, then event if there are multiple test & production consumer subaccounts, the application consumer can be identified in Landscape Portal.	No
provider_admin_email	email	Email address of initial provider user	Passed through to ABAP System (saas_oem plan).	Yes

Name	Data Type	Description	Purpose	Updateable
xs-security/xsapp-name (optional)	tring	xsappname used for the OAuth clone during instance creation. Will be visible in the security section of SAP BTP Cockpit when assigning the initial onboarder role. Will be the "Application Name" shown in the Roles UI. **Default:** Service Instance GUID **Recommendation:** Provide the same value as defined in the xs-security.json of the XSUAA Instance of your application	Allows providing a meaningful application name to assign the onboarding roles.	No
usage	enum (string): - test - prod	Specifies whether it is a test or production solution.	In case of usage = test a Partner Customer Test tenant is created, whereas with usage = prod, a Partner Customer Production tenant is provisioned.	No

4.2.14.1.6.1.4.2 SAP Authorization and Trust Management Service

Sample Code

```
  - name: xsuaa
    type: com.sap.xs.uaa
    requires:
      - name: abap-solution
    parameters:
      service-plan: application
      service-name: xsuaa
      config:
        oauth2-configuration:
          redirect-uris:
            - "https://*.${app-domain}/**"
        xsappname: ${appname}
        tenant-mode: shared
        scopes:
          - name: $XSAPPNAME.Callback
            description: With this scope set, the callbacks for tenant
onboarding, offboarding and getDependencies can be called.
            grant-as-authority-to-apps:
```

```
            - $XSAPPNAME(application,sap-provisioning,tenant-onboarding)
        foreign-scope-references:
          - uaa.user
        role-collections:
          - name: ${appname}-admin
            role-template-references:
              - $XSSERVICENAME(${appname}-abap-solution).SolutionAdmin
```

The `xsuaa` service instance, acts as an OAuth 2.0 client to the multitenant application, and to the ABAP `Solution` service instance. It provides access to the SaaS Provisioning service for calling callbacks and getting the dependencies API by granting corresponding scopes.

A role collection is defined as part of the xsuaa configuration which provides authorization for the initial user onboarding in the new ABAP tenant after a subscription was done.

For this purpose, the “SolutionAdmin” role is referenced. See Quick Start: Create Role Collections (with Predefined Roles).

Finally, a foreign scope reference is defined to ensure a token exchange will be possible and the ABAP systems can be called.

4.2.14.1.6.1.4.3 Saas Provisioning Service

Sample Code

```
  - name: saas-registry
    type: org.cloudfoundry.managed-service
    parameters:
      service: saas-registry
      service-plan: application
      service-name: saas-registry
      config:
        xsappname: ${appname}
        appName: ${appname}
        appUrls:
          getDependencies: https://cis${route-prefix}.${app-domain}/callback/
v1.0/dependencies
          onSubscription: https://cis${route-prefix}.${app-domain}/callback/
v1.0/tenants/{tenantId}
        displayName: ${saas-display-name}
        description: ${saas-description}
```

The `SaaS Provisioning` service allows application providers to register multitenant applications and services in the Cloud Foundry environment in SAP Business Technology Platform.

SaaS Provisioning Service Paramters

xsappname	The xsappname configured in the security descriptor file used to create the XSUAA instance.

getDependencies	**(Optional)** Any URL that the application exposes for GET dependencies. If the application does not have dependencies and the callback is not implemented, it should not be declared. **Note:** The JSON response of the callback must be encoded as either UTF8, UTF16, or UTF32, otherwise an error is returned. Important: You can either provide your own getDependencies Callback or use the default implementation of the AppRouter (recommended if no special logic is needed). **But:** If an own implementation is provided you have to make sure that the ABAP Solution instance is returned as a dependency. The path is: /callback/v1.0/dependencies
onSubscription	Any URL that the application exposes via PUT and DELETE subscription. It must end with /{tenantId}. The tenant for the subscription is passed to this callback as a path parameter. You must keep {tenantId} as a parameter in the URL so that it is replaced at real time with the tenant calling the subscription. This callback URL is called when a subscription between a multitenant application and a consumer tenant is created (PUT) and when the subscription is removed (DELETE). **Important:** You can either provide your own onSubscription Callback or use the default implementation of the approuter (recommended if no special logic is needed). The path is: /callback/v1.0/tenants/{tenantId}
displayName	**(Optional)** The display name of the application when viewed in the cockpit. For example, in the application's tile. If left empty, takes the application's technical name.
description	**(Optional)** The description of the application when viewed in the cockpit. For example, in the application's tile. If left empty, takes the application's display name.
category	**(Optional)** The category to which the application is grouped in the Subscriptions page in the cockpit. If left empty, gets assigned to the default category.

Callback APIs

Callback URLs defined in parameters *getDependencies* and *onSubscription* are called during the subscription flow and need to be callable via a consumer-independent URL. For this purpose, a dedicated URL that matches the `TENANT_HOST_PATTERN` of the approuter application should be defined as part for the MTA project.

Depending on the phase of the multitenant application, a different `TENANT_HOST_PATTERN` is defined:

- **During development phase:**
  `TENANT_HOST_PATTERN: ^(.*)-abap-saas-app.cfapps.eu10.hana.ondemand.com`
  Define a route cis-abap-saas-app.cfapps.eu10.hana.ondemand.com for SaaS Provisioning service callback URLs
- **During production phase:**
  `TENANT_HOST_PATTERN: ^(.*).mydomain.com`

Define a route cis.mydomain.com for SaaS Provisioning service callback URLs

SAP BTP Subscription Management Dashboard

With SAP BTP subscription management dashboard as the dashboard UI of SaaS Provisioning Service, the application provider can centrally manage subscriptions, e.g. unsubscribe existing application consumer subscriptions. See Using the Subscription Management Dashboard.

4.2.14.1.6.1.4.4 Approuter Application

To authenticate business users of the application at runtime, a tenant-aware approuter application and the `xsuaa` service in SAP Business Technology Platform are used. This will also provide the necessary callbacks for the SaaS Provisioning Service.

The enhanced approuter application is as a logical part of the multitenant application and configured as an external access point of the application.

For more details on the application router, see Application Router.

With the middleware `@sap/asp-middleware` component a library (npm module) is provided that enhances the approuter to also take care of automatically routing the consumer to their ABAP tenants. See @sap/asp-middleware.

Approuter Configuration

To make sure that the approuter routes all relevant requests to the ABAP Solution, a routing configuration is configured:

Sample Code

```
{
                                "authenticationMethod": "route",
                                "welcomeFile": "/ui",
                                "logout": {
                                "logoutEndpoint": "/sap/public/bc/icf/logoff",
                                "logoutPage": "/ui"
                                },
                                "routes": [
                                {
                                "source": "^/sap/(.*)$",
                                "target": "/sap/$1",
                                "authenticationType": "xsuaa",
                                "service": "com.sap.cloud.abap.solution",
                                "csrfProtection": false
                                },
                                {
                                "source": "^/ui(.*)$",
                                "target": "/ui$1",
                                "authenticationType": "xsuaa",
                                "service": "com.sap.cloud.abap.solution",
                                "csrfProtection": false
                                }
                                ]
                                }
```

- **Requests to the .../sap/... path** are backend requests and will be routed to the ABAP Solution service after successful authentication. CSRF token protection is disabled for this route as CSRF protection is already enforced via the underlying services in the ABAP system.
- **Requests to the .../ui/...** are requests to the Fiori Launchpad / End User UI and will be routed to the ABAP Solution service after successful authentication. CSRF token protection is disabled for this route as CSRF protection is already enforced via the underlying services in the ABAP system.

For more details on the parameters, see Routing Configuration File.

Approuter Multitenancy

The consumer accesses their multitenant application via the tenant-specific url which is resolved by the app router and "forwarded" (calls) to the corresponding abap tenant (client). The URL follows the same pattern for all application consumers and differs depending on the implementation phase of the multitenant application:

- **During development phase:**
  `<SUBSCRIBER_TENANT_SUBDOMAIN>-<APPROUTER_APPLICATION_HOST>.<SAP-PROVIDED_STANDARD_DOMAIN>`
  In each development landscape, SAP provides a different standard domain you can use to create URLs. Using that domain, and following the specified URL structure, an example for a created URL could be: tenant1-myapprouter.cfapps.eu10.hana.ondemand.com
  Here `tenant1-myapprouter` is the hostname, and `cfapps.sap.hana.ondemand.com` is the SAP-provided standard domain for this development landscape. In this format, a new route must be created manually for each new tenant.
  The common pattern identifying tenant-specific URLs is defined with `TENANT_HOST_PATTERN: ^(.*)->-<APPROUTER_APPLICATION_HOST>.<SAP-PROVIDED_STANDARD_DOMAIN>`
- **During production phase:**
  `*.<YOUR_CUSTOM_DOMAIN>`
  A custom domain is registered via the custom domain service, then a pattern with * as the wildcard hostname can be established.
  In this format, since the wildcard is replaced by an actual tenant during runtime, there is no need to create a new route for each new tenant manually.
  The common pattern identifying tenant-specific URLs is defined with TENANT_HOST_PATTERN: ^(.*)-<YOUR_CUSTOM_DOMAIN>
  For more information, see Using Custom Domains.

The application router derives the tenant from the URL and calls the tenant-aware XSUAA, which is responsible for the authentication of the business user. The UAA reads the tenant and gets the customer-specific identity provider (IdP) from the tenant-specific identity zone. Then, the XSUAA delegates the authentication to the configured IdP, and creates a JSON Web Token (JWT) that contains the tenant, the current user, and the authorization scopes of the user. The JWT is then sent back to the application router, and from there to the application.

To read and store tenant-specific data, the multitenant application needs to know the tenant ID. Using the Container Security API, the multitenant application is enabled to read the tenant, user, and scopes from the given JWT. The API also validates whether the JWT is valid. The tenant information is contained in the `identityZone` user attribute.

For more information, see:

Web Access in the Cloud Foundry Environment

What Is Authorization and Trust Management

4.2.14.1.6.1.4.5 Cloud Controller Access Destination

A destination called ASP_CC needs to be created for ABAP Solution Service to be able to access the Cloud Foundry Cloud Controller API for creation and access to ABAP service instances.

Prerequisites

- You've created a user in SAP ID service.
- The user has been added as a Cloud Foundry org member in the provider subaccount where the Cloud Foundy environment is enabled.
- The User has been added with the role of a Space Developer, in the space where the multitenant application (MTA) is deployed.

> ⓘ Note
>
> Please note that you should never set personal credentials (e.g. for your account) in the User and Password fields. Always use a technical user instead.

Process Steps

1. In the provider subaccount navigate to Connectivity -> Destinations
2. Create a new destination as follows:

Property	Value
Name	ASP_CC
Type	HTTP
Description	Access the Cloud Foundry Cloud Controller API
URL	< API Endpoint domain of Cloud Foundry landscape> You can find it under your subaccount > Overview. **Example** https://api.cf.eu10.hana.ondemand.com
Proxy Type	Internet
Authentication	OAuth2Password
User	<User ID/E-mail>
Password	<Password>
Client ID	cf
Client Secret	<needs to be left blank>

Property	Value
Token Service URL	https:// uaa.<landscape-host>/oauth/token **Example** https://uaa.cf.eu10.hana.ondemand.com/oauth/token

For more information on landscapes/regions, see Regions and API Endpoints for the ABAP Environment.

Instead of creating the ASP_CC destination manually, you can also save the following as a text file instead and then import the destination as a template:

Sample Code

```
Type=HTTP
                        clientId=cf
                        Authentication=OAuth2Password
                        Name=ASP_CC
                        tokenServiceURL=https\://
uaa.cf.eu10.hana.ondemand.com/oauth/token
                        ProxyType=Internet
                        URL=https\://api.cf.<e.g.eu10>.hana.ondemand.com
```

4.2.14.1.6.1.4.6 Regions

As a provider, you can deploy your SaaS solution in a subaccount that is located in one of the regions available for the ABAP environment, see Regions and API Endpoints for the ABAP Environment [page 43].

A consumer can then subscribe to and access your SaaS solution from a subaccount. The consumer subaccount needs to be located in the same region as the subaccount which you deployed the SaaS solution in.

Keep in mind that regions are chosen on the subaccount level: For each subaccount, you select exactly one region. This is done when creating the subaccount. The **Landscape Portal** can be accessed from subaccounts located in regions that are available for the ABAP environment. You thus need to make sure that the subaccount from which you want to access the **Landscape Portal** has the correct region attribute (that is, the technical key, such as cf-eu10). For more information on regions, see Regions and Accessing the Landscape Portal.

4.2.14.1.6.1.4.7 Troubleshooting

Multitenant application cannot be undeployed

- Always use MTA Undeployment (cf undeploy <MTA ID> --delete-services) instead of deleting service instances individually.

- Check if the saas-registry service instance still contains active subscriptions: the saas-registry instance can only be deleted once there are no more active subscriptions. Unsubscribe using the Subscription Management Dashboard Note, however that if the subscription is forced to be deleted through any additional channel (using the SAP BTP subscription management dashboard or by subaccount deletion), it will lead to an orphan tenant in the ABAP Solution. Providers need to be aware of this before deleting subscriptions with the 'force' option.

Cannot create consumer-specific route according to `TENANT_HOST_PATTERN` (in the development phase)

Sample Code

```
TENANT_HOST_PATTERN: (.*)-my-consumer-subdomain-productabcdefghijklmnopq-saas-
solution-dev.cfapps.eu10.hana.ondemand.com
Consumer Subaccount Subdomain: my-consumer-subdomain
Route Domain: cfapps.eu10.hana.ondemand.com
Route Hostname: my-consumer-subdomain-productabcdefghijklmnopq-saas-solution-
dev (64 characters)
```

After deployment of MTA for multitenant application, no abap system is created

- ABAP systems are automatically created by the ABAP Solution Provider (ASP) during the first subscription and not after the solution deployment.

Issues with parameter consumer_tenant_limit

- Keep in mind that recently deleted consumer tenants are counted towards the tenant limit as long as they are still in retention time. Systems are automatically created. Non-consumer tenants (provider tenant, test tenants) do not count towards this limit.

4.2.14.1.6.1.4.8 Using a Custom Domain for the Multitenant Application

The SAP Custom Domain Service (see SAP Custom Domain service) lets you configure your own custom domain (see What Is Custom Domain?) to publicly expose your application instead of using an SAP-provided default/shared/standard domain like cfapps.eu10.hana.ondemand.com for your application URLs.

By using this service, subaccount owners can make their SAP BTP applications accessible via a custom domain that is different from the default one; for example, myshop.com.

In case of multitenant applications (see Multitenant Applications [page 1317]), using a custom domain comes with another advantage: For each subscription there needs to be a CF route (see Routes) that is mapped to the approuter module of the multitenant application, so that the application is accessible via a URL.

While using a default domain, a new route like `<SUBSCRIBER_TENANT_SUBDOMAIN>-<APPROUTER_APPLICATION_HOST>.<SAP-PROVIDED_STANDARD_DOMAIN>` must be created manually for each new tenant.

By configuring the multitenant application with a custom domain, a route with a wildcard hostname like `*.<YOUR_CUSTOM_DOMAIN>` can be mapped, so that there is no need to create a new route for each new tenant manually. For more information, refer to Approuter Application [page 1327].

It is recommended to use a custom domain for your multitenant application only during the production phase; during the development phase using a default domain is sufficient.

Please follow the steps outlined in Configuring Custom Domains or Get Started with the Custom Domain Manager (refer to PaaS-User) to register your custom domain in the provider subaccount's CF org (see Multitenancy in the ABAP Environment [page 1349]). After registering the custom domain successfully, the domain becomes available as private domain (see Private domains) for route creation in any space of the provider subaccount's CF org.

To create a multitenant application for use with a custom domain, following configurations are adjusted accordingly:

- `TENANT_HOST_PATTERN` configuration of the approuter application: String containing a regular expression with a capturing group. The request host is matched against this regular expression. The value of the first capturing group is used as tenant id. Please refer to TENANT_HOST_PATTERN.
- SaaS Registry Callback URLs getDependencies and onSubscription. Please refer to Saas Provisioning Service [page 1325].
- XSUAA OAuth 2.0 Client configuration redirect-uris. Please refer to Listing Allowed Redirect URIs. We recommend to tighten these configurations as much as possible.
- CF routes assigned to approuter application

For the subscriptions to the multitenant application to work correctly, configured SaaS Registry callback URLs need to be reachable (cis... route). For Application Routing to work, the approuter needs to be able to resolve the application URL based on current `TENANT_HOST_PATTERN` configuration.

You can use the Maintain Solution app in the Landscape Portal to easily create such a multitenant application with custom domain, please refer to Create Deployment Configuration → **Step 3: Routing Settings**.

Or, if you are following the MTA-based approach to create your multitenant application, in the MTA Extension descriptor for production phase these configurations are changed using following MTA parameters:

- app-domain: the registered custom domain, e.g. mydomain.com
- route-prefix: needs to be empty because application URLs are mapped using following structure: `*.<YOUR_CUSTOM_DOMAIN>` (dot separator) and therefore do not include a route prefix

A route with wildcard hostname for tenant access is added to the approuter module, on top of the route used for SaaS Registry callback URLs:

Sample Code

```
modules:
  - name: approuter
    parameters:
      routes:
```

```
        - route: "cis${route-prefix}.${app-domain}"
        - route: "*${route-prefix}.${app-domain}"
```

To reduce risks use Blue-Green Deployment of Multitarget Applications while deploying multitenant application MTA during production phase.

After MTA build and deployment, the approuter application is created with a route cis.mydomain.com that is used for the SaaS Registry callback URLs `getDependencies` and `onSubscription`. On top of that a route with wild card hostname `*.mydomain.com` is mapped to the approuter. In this format, since the wildcard is replaced by an actual tenant during runtime, there is no need to create a new route for each new tenant manually.

If you change the domain type of your multitenant application from shared to custom domain, you need to update the application URL of existing subscriptions. Please use the option to update the application URL of an active subscription in the Subscription Management Dashboard. In such case, make sure to inform application consumers beforehand about temporary downtimes that might occur during this change.

4.2.14.1.6.2 Commercialize

To offer your SaaS solution to customers, register the SaaS solution in the SAP Store. See SAP Store.

Additionally, you can request an SAP Extension Suite certification for your product so that it gets listed in the SAP Certified Solutions Directory. See SAP Extension Suite Certification and SAP Certified Solutions Directory.

After finishing this process, your customers can order the SaaS solution, which is the preliminary step to actual subscription.

Prerequisites

Before registering a SaaS solution in SAP Store or certifying the solution, you have to deploy the first add-on version and multitenant application. See Multitenant Applications [page 1317].

4.2.14.1.6.2.1 (Optional) Register SaaS Solution in SAP Store

SAP Store is the enterprise marketplace where we bring together customers and partners on a single, easy-to-use, global online platform. See SAP Store.

To learn more about how you can offer your SaaS solution to SAP customers via SAP Store, see Getting Started with SAP Store.

4.2.14.1.6.2.2 (Optional) Certification

With an SAP Extension Suite certification, your SaaS solution is featured in the SAP Certified Solution Directory, which allows for more growth and customer participation. See SAP ICC Integration Scenario - SAP Extension Suite Certification and SAP Certified Solution Directory.

> **Note**
>
> Certification of SaaS solutions is only available if you have used add-on build delivery.

4.2.14.1.6.3 Subscribe

Once the SaaS solution has been fully deployed and commercialized, customers can order the offered product. After the buying process has been completed, you, as a SaaS solution operator, are responsible for onboarding the customer.

4.2.14.1.6.3.1 Create Consumer Subaccount

Since your solution is only available within your global accounts for production, you as the solution operator must create a consumer subaccount for your customer. After a customer has ordered the SaaS solution, create a 06 Consume subaccount in the global account for production, which will be used to create the subscription. See Set Up a Global Account for Production [page 1293].

While creating the subaccount, you must provide the following details:

- Name: Give the subaccount a unique display name. This can be the name of the customer, a customer number or a specific naming pattern.
- Provider: Choose the IaaS provider according to the IaaS provider selected for the provider subaccount.
- Region: Choose the subaccount region according to the region where the provider subaccount was created. See Regions and API Endpoints for the ABAP Environment.
- Subdomain: The subdomain plays an important role, as it becomes the consumer-specific part of the tenant-specific application URL.

4.2.14.1.6.3.2 Subscribe to SaaS Solution

After you as the solution operator have created the consumer subaccount, the subscription to the SaaS solution is enabled for the consumer.

In the consumer subaccount, navigate to the Service Marketplace. To subscribe to the SaaS solution, select the solution and the desired service plan and choose Create. The subscribed application is now displayed in Services Instances and Subscriptions.

The subscription process triggers the provisioning of:

- a new ABAP system in case all other systems have reached the consumer tenant limit.
- a new consumer tenant that is represented by an additional client in the ABAP system.

In both cases, a confirmation mail will be sent to the initial administrator maintained in the solution configuration. You can also monitor the provisioning using the *Systems Overview* and *Operations Dashboard* apps in the Landscape Portal.

The business type of the consumer tenant that is created depends on the usage parameter configured for your solution. If you have set `usage = prod`, a tenant of business type *Partner Customer Production Tenant* is created. If you have set `usage = test`, a tenant of business type *Partner Customer Test Tenant* is created. For more information, see Tenant Business Types [page 1350].

> **Note**
>
> gCTS Delivery: In case of delivery via gCTS, you, as a SaaS solution operator, can use a separate test subscription that is not related to a customer to trigger the creation of the system before the actual customer subscription. This initial system creation and import of the software components needs to be performed for each customer production system.

4.2.14.1.6.3.3 Configure Consumer Subaccount

Depending on the scope of the SaaS solution and customer requirements, you can make the following configurations in the consumer subaccount:

- Trust Configuration: You can configure a custom identity provider for the authentication of subscribed applications in the consumer subaccount. See Trust and Federation with Identity Providers. To restrict access based on certain criteria, such as the IP address, use the SAP Cloud Identity Services - Identity Authentication.

  > **Note**
  >
  > With new subaccounts created after September 24th 2020, automatic creation of shadow users is switched off by default for the default identity provider, SAP ID service. If the creation of shadow users is disabled, any additional business users need to be added manually to the users of the consumer subaccount so that authentication on subaccount level is possible. Business users can only log on to the ABAP tenant if there is an existing employee/business user in the ABAP system.

- Cloud Connector: Using Cloud Connector, you can create a link between SAP BTP applications and on-premise systems. See Managing Subaccounts on how to add and connect a consumer subaccount to Cloud Connector.
- Destination: Configure destinations, including technical information about the target resource, to connect your application to a remote service or system. See Managing Destinations.

These configurations can be done directly by you, the SaaS solution operator, or by the customer. In the latter case you provide the necessary authorizations by assigning the role collection Subaccount Administrator to a consumer subaccount administrator. The consumer subaccount administrator can then access the consumer subaccount in the SAP BTP cockpit including subscriptions, trust configuration, Cloud Connectors and destinations without having access to other consumer subaccounts in the global account for production.

The consumer subaccount administrator can assign role collection Cloud Connector Administrator to a user to operate the data transmission tunnels used by the Cloud Connector. While adding the consumer subaccount in Cloud Connector, the same Cloud Connector administrator user is being configured.

See Role Collections and Roles in Global Accounts, Directories, and Subaccounts [Feature Set B] for more information on available role collections in the consumer subaccount.

4.2.14.1.6.3.4 Initial User Onboarding

In order to be able to start using the solution, your customer needs to receive an initial administrator user in their consumer tenant. This is done automatically via the initial tenant onboarding of your solution.

To trigger this onboarding process, you as the provider need to be assigned to the SolutionAdmin role collection in the consumer subaccount:

1. Log on to the consumer subaccount and navigate to Security Role Collections.
2. Select the role collection SolutionAdmin, choose Edit, scroll down to Users, and select +. Enter the email address of your user, select SAP ID Service as your identity provider or use your preferred custom identity provider, and save your changes.

Now you are ready to access the solution's new tenant, where you will see the initial onboarding screen. In the confirmation dialog, you must confirm details such as email, subdomain, and subaccount ID to onboard an initial user for the customer:

1. Open the URL your provider has sent you to get access to their SaaS application.
2. On the initial administrator onboarding screen, your email address, subdomain, and subaccount ID are displayed. Select Onboard User to start the onboarding process. This might take a few minutes. Once the process is finished, you are redirected to the system. This onboarding process only needs to be performed once.

You can also monitor the tenant user provisioning using the Systems Overview and Operations Dashboard apps in the Landscape Portal. Ongoing tenant user provisionings are displayed in the Requests section of the Systems Overview app. See Systems Overview.

4.2.14.1.7 Configure and Implement a Customer Project

After you have set up the consumer subaccount, tenant, and tenant user, a remaining customer implementation project in the consumer tenant can start. This includes the setup of identity and authentication management, integration with other systems or services, and adjustment of business configuration.

Once you have configured and implemented the customer project, the SaaS solution matches the requirements of the customer and you can provide the SaaS application URL to the customer.

> → Tip
>
> We recommend providing documentation for your solution that includes specific configuration steps, for example to describe which business roles users have to create. In the following, we only describe the generic configuration steps.

> **Note**
>
> If you use gCTS for delivery to a customer production system AMT instead of add-ons, the system is initially created without application content. Therefore, you have to clone the required software components manually.
>
> For the initial import of a software component, as a SaaS solution operator on the provider side, you have to clone the software component in the customer production system AMT. The system can be accessed via the *Landscape Portal* application. In the *Landscape Portal*, the *Systems Overview* app provides an overview about the provisioned systems. Open the system that has been created during subscription to the multitenant application. In the *Tenants Overview*, the production tenant (client 100) can then be opened for provider access.
>
> In the *Manage Software Components* app, select a software component and clone the branch of the software component with the product version, for example v1.0.0.
>
> See Delivery via Add-On or gCTS [page 1282].

Prerequisites

To start with the customer project, you have to know the requirements of the customer, and need to have a consumer tenant subscription and a user for the initial consumer access. See **Subscribe to Multitenant Applications Using the Cockpit** and Initial User Onboarding [page 1336].

As the consumer subaccount administrator, access the SAP Fiori launchpad of the SaaS Solution subscription from the consumer subaccount account by navigating to *Services* > *Instances and Subscriptions*.

Set Up IAM

Create and Assign Business Roles

Before creating and assigning business roles, you have to make sure that business users are already available. They can be created manually or automatically. See User Provisioning [page 983].

As a business user, you configure the authorization to access the applications that are part of the SaaS solution via business roles. With the *Maintain Business Roles* app, you define business roles by combining predefined business catalogs and, if necessary, define value help, read and write access by maintaining the allowed values for fields.

Instead of creating a business role from scratch, you can also use business role templates. See How to Create a Business Role from a Template [page 3072].

Create Spaces and Pages for Business Roles

The spaces mode offers more flexibility to adjust the SAP Fiori launchpad layout for specific user groups.

As a business user, to create launchpad spaces and pages, see How to Create Spaces and Pages for a Business Role [page 3083].

Integrate

Create Communication Arrangements

A communication arrangement is a runtime description of a specific communication scenario created by you as a business user. It describes which communication partners communicate with each other in the scenario and how they communicate. See How to Create a Communication Arrangement [page 2998].

> **Note**
>
> Depending on whether you want to use an authentication method for outbound communication that requires a business user context (OAuth 2.0 SAML Bearer Assertion, OAuth 2.0 User Token Exchange, JWT Principal Propagation), you need to configure a destination for communication arrangements in the system instead of maintaining credentials by using an outbound communication user. See Supported Protocols and Authentication Methods and Creating a Destination.

See Integrating On-Premise Systems [page 1214] on how to integrate the SaaS solution with on-premise systems.

Configure

Create Business Configuration

As a business user, you can use the *Custom Business Configurations* app to adjust business configuration objects to change and influence the system behavior.

See Custom Business Configurations App [page 2970] on how to adjust configuration objects provided in the SaaS solution.

Configure Key User Extensibility

Key user extensibility that is enabled in the SaaS solution can be configured and consumed in consumer tenants.

> **Note**
>
> Key user extensibility provided in a SaaS solution can only be configured in tenants of particular types, for consumption purposes in customer systems, such as AMT, in tenants of type *Partner Customer Test* or *Partner Customer Production* depending on parameter `usage` in the configuration of the ABAP solution. See ABAP Solution Service [page 1320].
>
> These tenant types are provisioned in non-development systems, such as customer system AMT, where development is not allowed (`is_development_allowed = false`). The tenants are created dependent on a subscription to the SaaS solution. See **Subscribe to Multitenant Applications Using the Cockpit**.
>
> For a complete list of tenant business types that are enabled for key user extensibility, see Tenant Business Types [page 1350].

As a business user in a *Partner Customer Test* or *Partner Customer Production* tenant (client >= 200), you configure key user extensibility in a customer production system AMT.

- See Custom Logic (Deprecated) [page 3025] for guidance on how to use the *Custom Logic* app to create and maintain custom logic for business add-ins (BAdIs).
- See Configuring Predefined Custom Fields [page 3031] for guidance on how to configure predefined custom fields to customize applications and their UIs.
- See Adapting SAP Fiori UIs at Runtime - Key User Adaptation for guidance on how to adapt the user interface of an app separately for specific user roles.

4.2.14.1.8 Maintain, Monitor, Support

After the go-live of the SaaS solution and the deployment of its updates, consumers using the solution might run into issues that you have to troubleshoot and debug.

4.2.14.1.8.1 Maintain

After the initial add-on release has been shipped as a SaaS solution offering, the development of maintenance patches, support packages, and consequent add-on releases comes into play.

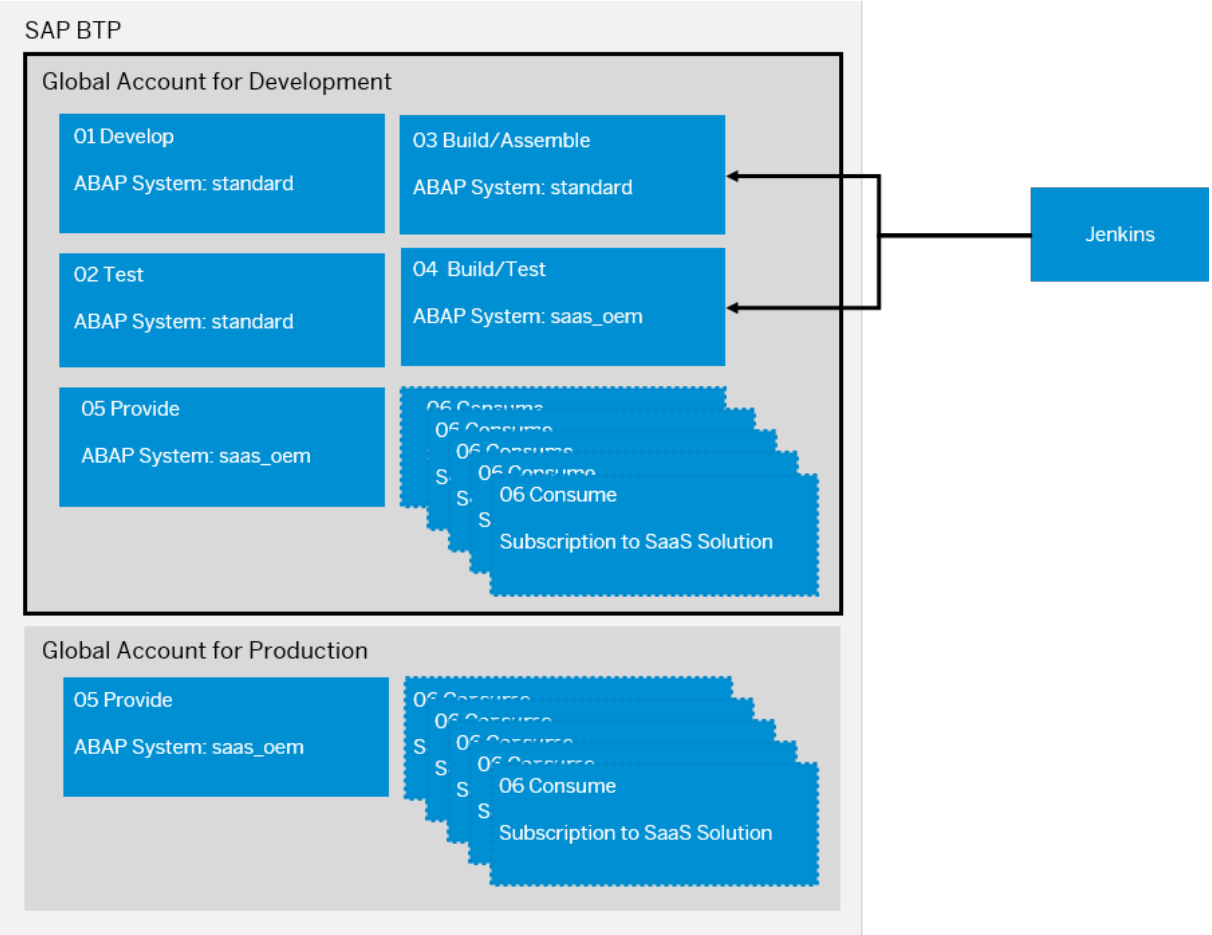
Once everything is implemented, built, and the maintenance delivery is deployed, the corresponding changes become available in the customer production system AMT.

> **Note**
>
> New features are developed on different code lines (branches). If you create a so-called maintenance branch to implement patches while new features are implemented in the main branch of a software component, you can implement new features and provide bug fixes at the same time. For more information, see Versioning and Branches.

Prerequisites

- To set up the maintenance system landscape, you need the relevant entitlements in the global account for development. See Entitlements and Quotas.
- When using a maintenance system landscape, you require business users with authorization to use the Manage Software Components app and developer users using ABAP Development Tools must be available in the systems. See Manage Software Components [page 3222] and Getting Started as a Developer in the ABAP Environment [page 241].
- To configure new add-on versions, you need the existing pipeline configuration for an add-on build pipeline. These can be the pipeline templates in the Build Product Version app or a manual configuration. See Build Product Version and Build and Publish Add-on Products on SAP BTP ABAP Environment .
- If you have configured the add-on build pipeline manually, you need a Jenkins server where you can execute it.
- To use the Landscape Portal, you need a subscription to the Landscape Portal application and a user assigned to role collection `LandscapePortalAdminRoleCollection`.

4.2.14.1.8.1.1 Set Up Maintenance System Landscape



Bug fixes aren't implemented as part of the development code line, but in a separate correction code line. In this manner, corrections can be delivered to consumers even while regular development is ongoing.

We recommend setting up a maintenance system landscape that runs in parallel to the development system landscape. This maintenance landscape consists of:

- A correction system COR where corrections are developed. This system is provisioned in subaccount *01 Develop* in the development space. Set parameter `is_development_allowed = true`.
- A quality assurance system QAS for testing the corrections. This system is provisioned in subaccount *02 Test* in the test space. Set parameter `is_development_allowed = false`.

The required entitlements and system-related details are listed in the table below:

Global Account	Subaccount	Space	Entitlements	ABAP System
Global Account for Development	01 Develop	Develop	1x abap/standard abap/hana_compute_unit (standard: 2) abap/abap_compute_unit (standard: 1)	COR
	02 Test	Test	1x abap/standard abap/hana_compute_unit (standard: 2) abap/abap_compute_unit (standard: 1)	QAS

The Manage System Hibernation app in the Landscape Portal allows you to shut down the maintenance systems when no corrections are being developed, significantly reducing the costs incurred. See Manage System Hibernation. For more information on working with two codelines, see Use Case 2: One Development and Correction Codeline in a 5-System Landscape.

It is not strictly necessary to set up a separate maintenance system landscape. You can also implement corrections in the existing development and test systems to further reduce costs. In this case, ongoing development must be stopped while the release branch is checked out and worked on. See Use Case 1: One Codeline in a 3-System Landscape.

4.2.14.1.8.1.2 Create Add-On Update

As an add-on administrator, you can provide different kinds of updates. The decision regarding which one to use depends on the motivation behind a given update.

- Small and urgent corrections are delivered as patches
- Collections of less urgent corrections and other functional enhancements should be delivered as support packages
- New release versions are released with updates containing significant enhancements and new features

> → Tip
>
> Use the Check Product Version app in the Landscape Portal to find out which product version has already been built.

> ⓘ Note
>
> If you use gCTS for delivery, the process of creating an update for SaaS solutions is grouped into urgent corrections (patch version) and new releases (release version). See Use Case 2: One Development and Correction Codeline in a 5-System Landscape and Delivery via Add-On or gCTS.

For delivering an emergency patch via gCTS, see section *Urgent Corrections* in Use Case 2: One Development and Correction Codeline in a 5-System Landscape.

For delivering a new major version via gCTS instead of using add-ons, see section *For Go Live/Development after Go Live (Including Deferrable Corrections)* in Use Case 2: One Development and Correction Codeline in a 5-System Landscape.

For general information regarding gCTS delivery, see Delivery via Add-On or gCTS.

Create new patch version (emergency patch)

Patch versions are used to deliver unplanned and most likely urgent corrections that are required to keep the application up and running. These changes could be required for a service consumption model of an OData Client Proxy in case of changes to the structure of the underlying OData service. See Consuming an OData Service.

Bug fixes are delivered as a new patch version of a software component and are implemented on the correction code line. Therefore, the dedicated ABAP correction system COR is used. For more information on the different software component version numbers, see Software Component Version .

> **Note**
>
> While creating patch versions for the same support package level or release version, the same maintenance branch should be used.

To deliver a new patch version, see the following steps:

- **Develop**
  - As an add-on admin, check out the maintenance branch, which is the branch that is created for bug fixes and maintenance deliveries in correction system COR and quality assurance system QAS. See Work with Branches [page 3238].
  - In case of released APIs: If not already done, upload the API snapshot that was created for the latest support package or release version into quality assurance system QAS and correction system COR. Set the API snapshot to check-relevant if not already enabled. See Manage API Snapshots.

    > **Note**
    >
    > While creating new patch versions, the maintenance branch is usually checked out for the first time in correction system COR and quality assurance system QAS. Thus, the corresponding snapshot must uploaded and set to check-relevant. See Released APIs and API Snapshots.

  - Implement the bug fix in the ABAP correction system
    As a developer, implement the required changes to the software component in the ABAP correction system COR. These changes shouldn't introduce new features because patch versions should only contain small emergency corrections. Release the transport tasks and requests.
- **Test**
  - Import the maintenance branch in the ABAP quality assurance system
    You need to test the implemented corrections by importing the maintenance branch for the current support package level, where the transports for necessary bug fixes were released, in the dedicated test system, which is the ABAP quality assurance system QAS. See Pull Software Components [page 3227].
  - Test bugfix in ABAP quality assurance systemAs a tester, test the corrections in the QAS system to validate that the provided bug fixes are solving the problem and are working properly. Required testing

can involve anything from maintaining business roles, creating communication arrangements to more complex testing scenarios where connectivity is involved.

- **Double maintenance of corrections into development**
  As a developer, you have to maintain the corrections that have been developed in the ABAP correction system COR as separate transports in development system DEV (so called double-maintenance). See Double Maintenance of Corrections into Development [page 915].
- **Configure new product version**
  As an add-on admin, you can use the Build Product Version app in the Landscape Portal to create new patches:
- Make sure that the pipeline template for Patch Delivery is configured. If you want to validate the add-on build beforehand, configure the template for Test Patch Delivery as well. See Configure Pipeline Template.
- Choose the parent version that shall be patched and create a new product version of type Patch Delivery. See Create a New Product Version.

If you are configuring the add-on build pipeline manually, adjust your add-on descriptor file to build the new patch version. See Build and Publish Add-on Products on SAP BTP ABAP Environment .

Create new support package version (hotfix collection)

Support package versions are used to deliver planned functional enhancements outside of new major releases. They are often used to bundle multiple patch versions to a hotfix collection.

A support package version could be used for example to deliver a new simple business configuration UI. See Business Configuration in SAP BTP ABAP Environment (1): Overview and BC Maintenance Apps.

> **Note**
>
> Support package deliveries aren't limited to changes to only one software component. But only software component support packages and patches should be included in a support package delivery. The software components that are part of an add-on product can only be changed as part of a release delivery.

> **Note**
>
> If you use a quarterly delivery approach, new features are not delivered using support package versions. Instead, a new maintenance branch is created from the previous maintenance branch, using systems COR and QAS instead. See Delivery Models.

To deliver a new support package stack, see the following steps:

- **Develop**
  - As a developer, implement the new feature in the main branch in the ABAP development system DEV. Bug fixes should be delivered via patch deliveries if they need to be available before the next support package delivery cycle. See The Add-On Product .
- **Test**
  As an add-on admin, use the *Manage Software Components* app to import the changes in the main branch into test system TST.
- As a tester, test the new implemented features in the TST system to validate the functionality. Required testing can involve anything from maintaining business roles, creating communication arrangements to more complex testing scenarios where connectivity is involved.
- **Create maintenance branches**
  As an add-on admin, create a new maintenance branch for each new support package level of a software component. These maintenance branches are used for developing bug fixes and maintaining deliveries.

This assures that these bug fixes can be implemented in the correction code line to not block ongoing development on the main branch.

- **Create new product version**
  As an add-on admin, you can use the Build Product Version app in the Landscape Portal to create new support package stacks:
  1. Make sure that the pipeline template for Suport Package Stack is configured. If you want to validate the add-on build beforehand, configure the template for Test Support Package Stack as well. See Configure a Pipeline Template.
  2. Choose the parent version that shall be patched and create a new product version of type Patch Delivery. See Create a New Product Version.

  If you are configuring the add-on build pipeline manually, adjust your add-on descriptor file to build the new patch version. See Build and Publish Add-on Products on SAP BTP ABAP Environment .

- **Download API Snapshot for use in other systems**
  In case of released APIs, download the API snapshot created for the new support package version from the add-on build system BLD. The snapshot is created automatically during the add-on build. Afterwards, upload the snapshot into the test system TST and the development System DEV. Set the new API snapshot as check-relevant so that it will be used as a reference for API compatibility checks.

Create new release version (product version)

Release versions are used to deliver new major, planned functional enhancements. Typically, they include multiple new implemented features. For example, multiple new apps could be introduced with such a major release.

> **Note**
>
> Only with new release deliveries, you can change the software component version bundle in a new release.

To deliver a new release version, see the following steps:

- **Develop**
  As a developer, implement the new feature in the main branch in the ABAP development system DEV. Bug fixes should be delivered via patches or support packages instead. Release the transport tasks and requests.
- **Test**
  As an add-on admin, use the *Manage Software Components* app to import the main branch into the test system TST so that new developments can be tested.
  As a tester, test the new implemented features in system TST to validate the functionality.
  Required testing can involve anything from maintaining business roles, creating communication arrangements to more complex testing scenarios where connectivity is involved.
- **Create maintenance branches**
  As an add-on admin, create a new maintenance branch for each new release version of a software component. These maintenance branches are used for developing bug fixes and maintaining deliveries. This assures that these bug fixes can be implemented in the correction code line to not block ongoing development on the main branch.
- **Configure new product version**
  As an add-on admin, you can use the Build Product Version app in the Landscape Portal to create new release versions:

1. Make sure that the pipeline template for Release Delivery is configured. If you want to validate the add-on build beforehand, configure the template for *Test Release Delivery* as well. See Configure Pipeline Template.
2. Choose the relevant add-on product and create a new product version of type *Release Delivery*. See Create a New Product Version.

- **Download API Snapshot for use in other systems**
  In case of released APIs, download the API snapshot created for the new release version from the add-on build system BLD. The snapshot is created automatically during the add-on build. Afterwards, upload the snapshot into the test system TST and the development System DEV. Set the new API snapshot as check-relevant so that it will be used as a reference for API compatibility checks.

If you are configuring the add-on build pipeline manually, adjust your add-on descriptor file to build the new patch version. See Build and Publish Add-on Products on SAP BTP ABAP Environment .

4.2.14.1.8.1.3 Trigger Add-On Build Pipeline

Similar to the build of the initial add-on version, as an add-on administrator, you need to trigger the execution of the configured ABAP environment pipeline for an add-on build. You can do this using the Build Product Version button in the corresponding app. After a successful build, the new add-on version is technically available for deployment to the ABAP environment. See Build Product Version. If you are configuring the add-on build pipeline manually, then you need to trigger its execution in your Jenkins server. For in-depth information about the ABAP environment pipeline, check out ABAP Environment Pipeline.

> **Note**
>
> If you are using gCTS instead of add-ons for delivering software components into production systems, an add-on build is not required.
>
> Please ensure that the add-on product version to be published is properly tested before confirming the release decision. This includes testing in SAP Fiori launchpad and the ABAP Test Cockpit. See Test in the ABAP Environment SAP Fiori Launchpad [page 1305] and Test in the ABAP Test Cockpit [page 1307].
>
> During add-on build a new semantic version API snapshot will automatically be created for each add-on software component. In the assembly system BLD semantic version snapshots are automatically created and set to check-relevant during add-on build. Do not perform manual actions on API snapshots in these systems. You also have the option to download a created API snapshot for use in other systems. See Downloading API snapshots.

4.2.14.1.8.1.4 Check Add-on Build Result

Use the *Check Product Version* app in Landscape Portal to check whether the product version, its components, and respective packages are ready for deployment. See Check Product Version.

4.2.14.1.8.1.5 Deploy Add-On Update

As a SaaS solution operator, you can apply add-on updates to existing systems via theDeploy Product app in the Landscape Portal.

You can select the target add-on product and version for the update. Afterwards, you can select the systems where the update should be applied and schedule the update.

> **Note**
>
> If you are using gCTS instead of add-ons for delivering software components into production systems, you need to pull the changes of all relevant software components into all productive systems using the *Manage Software Components* app. An update deployment via the Landscape Portal is not required.

4.2.14.1.8.2 Manage Incidents

Incident Management System

As a provider of a SaaS solution, you should establish an incident management system, such as SAP Resolve, so that consumers can report issues.

If consumers of your SaaS solution run into problems, they need to be able to contact the provider of the software in a standardized way.

Information to Identify ABAP System and Consumer Tenant for Incident Processing

To process an incident, you have to identify the ABAP system and tenant, where the consumer issue has occurred. Therefore, the following information is required:

- SAP Fiori launchpad URL
  In the SAP Fiori launchpad user actions menu, the SAP Fiori launchpad URL of a consumer tenant is displayed as *Server* in the *Settings* dialog. See Settings.
- System Details
  In the user actions menu, information about the ABAP system and tenant is displayed in section *System* in the *About* dialog. See About.
  The system name and description are of particular interest.

Information to Reproduce the Customer Issue

To reproduce the consumer issue, the following information should be provided:

- Application details
  In the user actions menu, information about the SAP Fiori application is displayed in the *About* dialog in the *Application* section. See About.
  The application title and technical component ID are of particular interest.
- Error details
  The error message is displayed in the SAP Fiori application or as a status code of the HTTP request that is failing.
- Business user details

In the user actions menu, the name and email address identifying the business user of the consumer are displayed in the *Settings* dialog in the *User Account* section. See Settings.

- Step-by-Step Description of the User Actions and Input
  To reproduce the customer issue, you need detailed information about the user actions that lead to the error, and, if possible, user input that was performed during these steps.

4.2.14.1.8.3 Troubleshoot and Debug

If a customer faces an issue, you have to provide troubleshooting based on a reported consumer incident. You can access the SAP Fiori launchpad of the consumer tenant or the backend using ABAP Development Tools to further analyze the issue.

Identify ABAP System and Consumer Tenant

The ABAP system can be identified in the *Systems Overview* in the Landscape Portal application. See Systems Overview.

The system ID is defined in the ABAP Solution service via configuration parameter `sap_system_name`. If there are multiple solutions, the system description identifies the corresponding ABAP system as it always follows the same pattern: *ABAP Solution System for <parameter "name" in ABAP Solution service>*. See ABAP Solution Service [page 1320].

Based on the SAP Fiori launchpad URL of a consumer tenant, the consumer subaccount subdomain, that uniquely identifies the tenant, can be derived. The pattern to extract the subdomain is defined in `TENANT_HOST_PATTERN`. This is defined as environment variable of the deployed approuter, which is part of the multitenant application. See Approuter Application [page 1327].

> **Example**
>
> Identifying the subdomain with a multitenant application created based on the MTA-based approach, see Multitenant Applications [page 1317].
>
> - `TENANT_HOST_PATTERN = (.*)${route-prefix}.${app-domain}`
> - `${route-prefix} = <empty>`
> - `${app-domain} = mydomain.com`
> - SAP Fiori Launchpad URL = `myconsumer.mydomain.com`
>
> In this example, `myconsumer` is the subdomain identifying the consumer tenant.

The subdomain is displayed in the *Tenants View* as *Subaccount Domain* when selecting the system in the Landscape Portal app and thus can be used to find the consumer tenant (business type *Partner Customer Test* and *Partner Customer Production*).

Access the Consumer Tenant of the ABAP System

A consumer tenant in client >= 200 (business type *Partner Customer Test* or *Partner Customer Production*) can be accessed with a provider support user created in the Landscape Portal. SeeCreate Support Users.

- **Frontend Access to SAP Fiori launchpad**
  The SAP Fiori launchpad of a consumer tenant in client >= 200 can be accessed via the link listed in the tenant overview in the Landscape Portal. A provider support user must be requested for the tenant

beforehand. Such a user can only access a limited set of SAP-provided apps, for example for Identity and Access Management and Communication Management.

- **Backend Access using ABAP Development Tools**
  Access to a consumer tenant in client >= 200 using ABAP Development Tools requires an adapted service key in the service instance in the SAP BTP cockpit: The service key SAP_ASP displayed in the cockpit needs to be adjusted with the GUID of the consumer tenant. See Create Support Users.

ABAP Development Tools access with a provider support user created in the Landscape Portal grants access to troubleshooting capabilities, such as those granted with business role `Application Support Engineer - Development Support (BR_APPL_SUP_ENG_DEV_SUP)`.

Troubleshoot

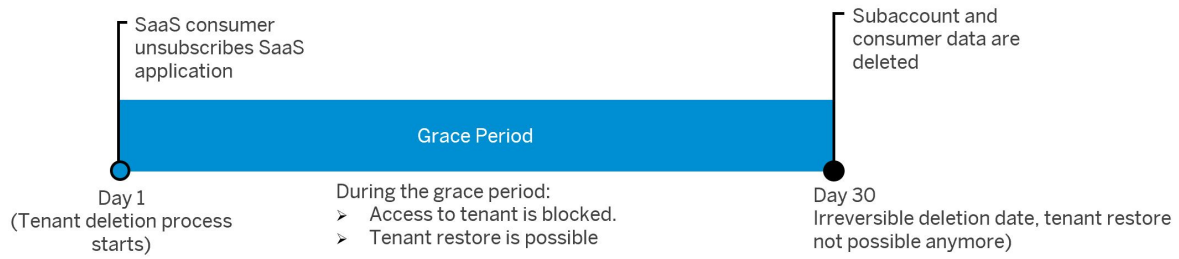
The most common use case for troubleshooting is to debug a business user or communication user executing the implemented business logic that is causing an issue in the consumer tenant. See Troubleshoot Custom Apps [page 3446].

With a provider support user, you need to connect with ABAP Development Tools to the system and consumer tenant, set a breakpoint in the logic, and afterwards the business user of the consumer must reproduce the error.

Apart from debugging, you can also use other troubleshooting tools in ABAP Development Tools to analyze issues in a consumer tenant. See Troubleshooting Tools [page 3449].

4.2.14.1.9 Dismantle

As soon as a consumer unsubscribes from the SaaS solution, an immediate tenant decommissioning process takes place while taking a “grace period” of 30 days into account, according to the following legal obligations:



During the grace period

- The tenant access is blocked for the consumer and for the SaaS provider as well in the support use case.
- The provider can restore the tenant in the *Landscape Portal*. See Restore Consumer Tenants.

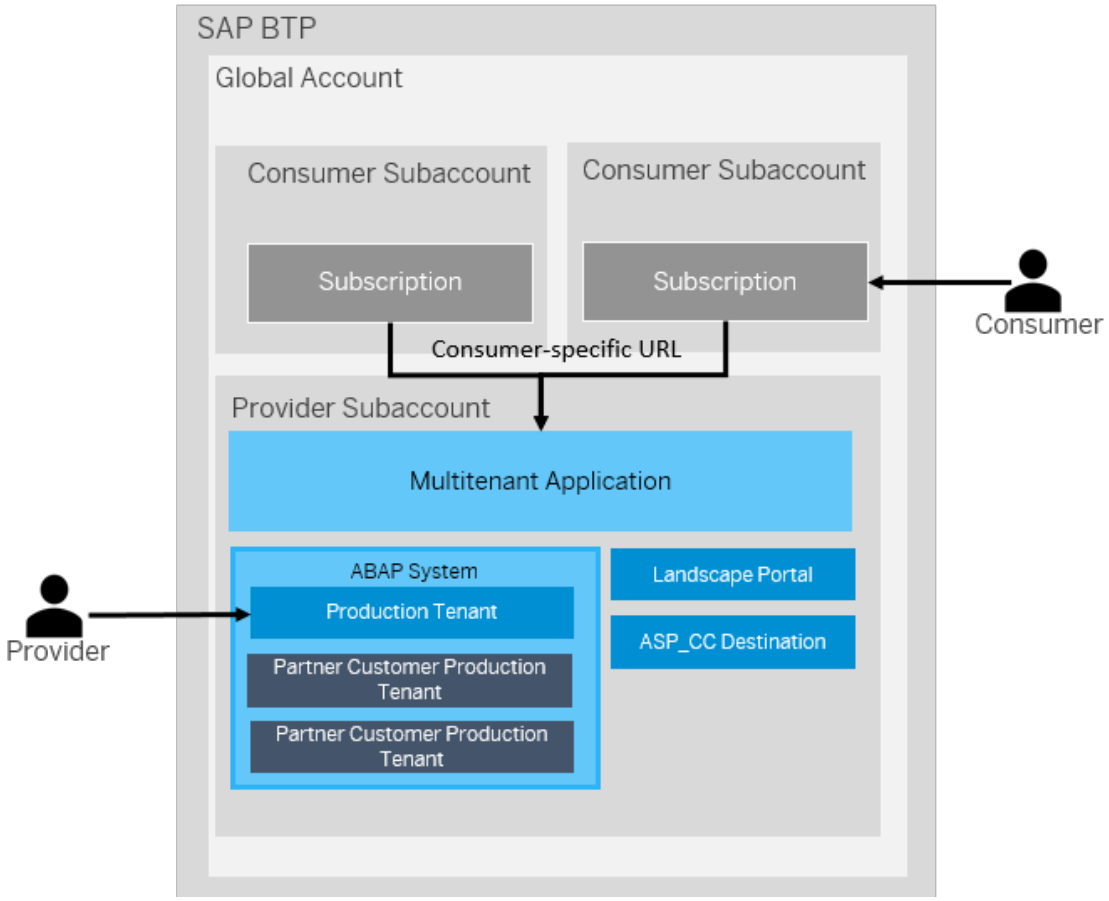
After the end of the grace period, the tenant and all related data are irreversibly deleted and cannot be restored anymore.

Once the last consumer unsubscribes from the SaaS solution, the provider needs to delete the empty system in the BTP cockpit.

Alternatively, as a SaaS solution operator, your can use the *Subscription Management Dashboard* in the provider subaccount to unsubscribe tenants centrally without triggering unsubscription in the consumer subaccount. See Using the Subscription Management Dashboard.

4.2.14.2 Multitenancy in the ABAP Environment

Multitenancy defines the capability to host different customers (tenants) on a single, shared computing infrastructure in order to optimize administration and significantly reduce TCO. A tenant is an organizationally independent unit whose IT business is entirely or partly operated together with the businesses of other tenants by a hosting provider. Multitenancy is especially relevant in a “Software as a Service” (SaaS) business model where customers subscribe to hosted software solutions rather than buying and installing them. For hosting providers, the operation costs per user per tenant are a decisive factor - particularly in the SME market, where typically each tenant only has a small number of users (10 – 50) accessing the hosted solution. Hosting providers can only operate profitably if they succeed in keeping the administration and maintenance costs for the hosted solution as low as possible. This cost pressure also limits the degree of individualization and customization a hosting provider is willing to accept for a hosted solution provided to many tenants.



Multitenancy in the ABAP environment enables independent software vendors or partners (referred to as application **provider**) to develop and operate ABAP solutions as software as a service (SaaS) leveraging SAP BTP infrastructure while hosting several consumers on the same ABAP system. The resources on SAP BTP platform consumed by the solution are paid for by the application provider.

Application **consumers** (= end customers of the provider) subscribe to a provider’s multitenant application and use it in a specific consumer subaccount (= tenant). Consumers access the provider’s SaaS application via a consumer-specific URL. Consumers cannot see the data of other consumers and Identity and Access Management is kept isolated between different tenants.

The **Multitenant Application** is deployed to the provider subaccount and serves as the entry point for the consumer-specific URLs, so that requests are routed to the corresponding consumer tenant in the ABAP system. Only after the multitenant application has been deployed, the application will be available for subscription to consumers. See Multitenant Applications [page 1317].

The **ABAP system** used to serve the application to the consumers is provisioned in the provider subaccount during the first subscription as abap/standard or abap/saas_oem system (depending on whether add-on is used for delivery). See Creating an ABAP System [page 198]. Different tenants are created as separate clients in the system. Tenants in the ABAP system have different capabilities represented by the tenant business type and a lifecycle status. The ABAP system contains by default a tenant used by the application provider (client 100) for system-level operations like the import of software components to the system. For each subscription to the multitenant application, a tenant used by the consumer (client >= 200) is created. If any consumer tenants still exist in the ABAP system, the system cannot be deleted.

The **Landscape Portal** functions as a central plane for tenant management that allows providers to perform lifecycle management operations such as add-on updates, creating test tenants or support users and more. For more information on how to access and use the Landscape Portal, seeLandscape Portal.

The **ASP_CC Destination** is a destination created on subaccount-level in the provider subaccounts. It points to the Cloud Foundry Cloud Controller API and is utilized by the multitenant application to create the ABAP system in the Cloud Foundry space/org where the multitenant application is deployed. See Cloud Controller Access Destination [page 1329].

> **Note**
>
> When building tenant-aware applications on top of the ABAP environment, providers need to follow specific ABAP implementation rules to ensure a content separation between different consumers. To view these guidelines, see Multitenancy Development Guideline [page 1353].

4.2.14.2.1 Tenant Business Types

The capabilities of a tenant used by the application provider or application consumer depend on the tenant business type. Business types of tenants that reflect how the tenant is used are derived from the system business type (partner development or production system via parameter `is_development_allowed`).

- **Target Group**: Whether the tenant is primarily used by the application provider or application consumer.
- **ABAP System Client**: Consumer/Provider tenants are created as a different ABAP system client for content separation.
- **Business User Logon**: Business users are authenticated in an ABAP tenant using trusted identity providers configured in either the provider subaccount or consumer subaccount. See Trust and Federation with Identity Providers.
- **Provider Support Access**: In the Landscape Portal the application provider can request support user access for certain tenants in the ABAP system to troubleshoot & debug issues. See Create Support Users .
- **Key-User Extensibility Configuration**: Key-user extensibility features of a solution like custom logic and predefined custom fields can only be configured in tenants of particular types. See Providing Business Add-Ins [page 1361] and Configuring Predefined Custom Fields [page 1366].
- **Software Component Import**: The Manage Software Components app is only available in tenants of particular type. See Manage Software Components [page 3222].

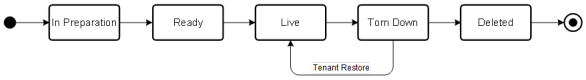
- **Tenant Onboarding**: When and how tenants are created differs depending on the business type. See **Subscribe to Multitenant Applications Using the Cockpit**.
- **Initial Administrator Onboarding**: In each tenant an initial user is created based on an e-mail address to get access to the tenant after it has been created.
- **Tenant Offboarding**: When and how tenants are deleted differs depending on the business type. See Dismantle [page 1348].
- **Tenant Restore**: Only tenants of particular types can be restored during the retention period. SeeRestore Consumer Tenants.

	Partner Development System		Production System			
	Partner Development Tenant	**Partner Test Tenant**	**Production Tenant**	**Test Tenant**	**Partner Customer Test Tenant**	**Partner Customer Production Tenant**
Target Group	Application Provider	Application Provider	Application Provider	Application Provider	Application Consumer	Application Consumer
ABAP System Client	100	>=200	100	>=200	>=200	>=200
Business User Logon	Logon via Identity Provider configured in provider subaccount	Logon via Identity Provider configured in provider subaccount	Logon via Identity Provider configured in provider subaccount	Logon via Identity Provider configured in provider subaccount	Logon via Identity Provider configured in consumer subaccount	Logon via Identity Provider configured in consumer subaccount
Provider Support User Access	Not available	Not available	Not available	Not available	Available via Landscape Portal	Available via Landscape Portal
Key-User Extensibility Configuration	Not available	Not available	Not available	Available	Available	Available
Software Component Import	Manage Software Components app available	Not available	Manage Software Components app available	Not available	Not available	Not available
Tenant Onboarding	Created during system provisioning	Test Tenant created in Landscape Portal	Created during system provisioning	Test Tenant created in Landscape Portal	Created during subscription to solution with parameter `usage = test`	Created during subscription to solution with parameter `usage = prod`
Initial Administrator Onboarding	Created during system provisioning based on `provider_admin_email` parameter	Created during test tenant creation in Landscape Portal	Created during system provisioning based on `provider_admin_email` paramete	Created during test tenant creation in Landscape Portal	Created using initial user onboarding form after subscription	Created using initial user onboarding form after subscription
Tenant Offboarding	Deleted during system decommissioning	Deleted in Landscape Portal	Deleted during system decommissioning	Deleted in Landscape Portal	Deleted automatically after retention period	Deleted automatically after retention period

	Partner Development System		Production System			
Tenant Restore	Not available	Not available	Not available	Not available	Consumer Tenant restore using Landscape Portal	Consumer Tenant restore using Landscape Portal

4.2.14.2.2 Tenant Lifecycle

Each consumer tenant goes through a certain lifecycle starting from the provisioning to the deprovisioning of the tenant and possibly tenant restoration. The current lifecycle status of a tenant in the ABAP system is shown in the Landscape Portal.



Request Type in Landscape Portal	Changes to Lifecycle Status	Description
New Tenant	In Preparation Ready Live	The creation of a new tenant starts with status **In Preparation**. After the creation of the new tenant in the system, the registration of client and tenant in the system, and the import of initial tenant data, the tenant is **Ready**. Finally, after preparation of logon to the new tenant, technical configuration, business configuration, activation of tenant features, setup of monitoring, the tenant is finalized, and the lifecycle status is set to **Live**.
Delete Tenant	Live Torn Down Deleted	Tenant deletion is triggered when the consumer subscription is deleted. The deletion request changes the tenant lifecycle status from **Live** to **Torn Down**. During a retention period of 30 days, the tenant remains in status **Torn Down**, reflecting the deletion in progress. After 30 days, and if the tenant is not restored, the tenant deletion is automatically completed, and the status is changed to **Deleted**.
Restore Tenant	Live Torn Down Live	After a tenant deletion was requested by deleting the corresponding consumer subscription, the tenant can be restored in the Landscape Portal during a retention period of 30 days. Doing so, the lifecycle status is changed back from **Torn Down** during the retention period, to **Live** after the completed tenant restoration.

4.2.14.2.3 Multitenancy Development Guideline

Multitenancy is required if you want to run several customers on the same ABAP system. When building tenant-aware applications on top of the ABAP environment, you must follow dedicated rules to ensure, for example, a content separation between different customers.

Architecture Overview and Basic Concept

The following sections explain how to host multiple tenants in one ABAP system by building a multitenant enabled solution. We use multiple clients in the ABAP environment as a technical means to implement multitenancy.

There are the following basic principles regarding multitenancy:

- for the **tenant**: Strict data isolation is guaranteed between tenants on all levels of the system. It must be ensured that the tenants' data as well as their tenant-specific application extensions are under no circumstances mingled with the data or applications of any other tenant being operated in the same system.
- for the **hosting provider**: The costs for operating multiple tenants in the same system must be significantly lower compared to single tenant systems. Multitenant systems must reduce the TCO of a hosted solution in such a way that the business case is met.

  > ### Note
  >
  > Multitenancy is only one means to reduce the marginal costs of operations addressing system and hardware costs.

To some extent, these principles are contradictory. The first principle could be most easily fulfilled by single tenant systems, whereas the second principle imposes the need for sharing resources on each system level, for example, hardware, database, application server. Multitenancy can thus be described as sharing as much system resources as possible without violating the principle of tenant data isolation.

> ### Note
>
> Tenant isolation isn't completely ensured by the ABAP environment infrastructure. However, the following sections describe the design principles how multitenancy can be achieved in the ABAP environment by an evolution of the present multiclient capability of an ABAP system.

The ABAP environment is a Platform as a Service (PaaS) offering for ABAP. In a typical scenario, an independent software vendor (ISV) develops and runs a Software as a Service (SaaS) application in the ABAP environment and sells this application to end-customers. In order to avoid confusion with the term “customer”, as the ISV is also a customer of the ABAP environment, the following terminology is used in this document:

- The **service provider** of the SaaS application is responsible for developing, maintaining, and operating the application on top of the ABAP environment. The service provider is typically an ISV or a development partner.
- The **provider system** is an ABAP system in the ABAP environment that runs the SaaS application. The provider system is operated by the service provider and offers multitenancy capabilities.
- The **service consumer** of the SaaS application is an end-customer of the service provider who has subscribed a tenant in a provider system.

Assumptions and Limitations

- Multitenancy is only supported in provider systems operated by the same service provider to run a single SaaS application.
- Multitenancy isn't supported for the development scenario using the ABAP Development Tools (ADT). This means that several service providers can't share the same system for development. Furthermore, a consumer can't further develop or extend the provider's SaaS application using ADT.
- You as a service provider can build a multitenancy application following the respective guidelines.
- Transport of objects isn't supported in a multitenant environment.
  Business configuration content is usually transported to a productive environment by using the transport API `IF_A4C_BC_HANDLER`, in this case, the user interface doesn't have to be implemented. To transport business configurations between systems, you can use the download/upload functionality, see Upload Business Configuration [page 2975].
- There is no system-controlled process to ensure that you as a service provider build your application in a multitenancy-compliant way. If strict data isolation is a legal requirement, we propose an external audit to ensure that your code doesn't break data isolation.
- There are options to share data between tenants. Furthermore, native SAP HANA database access via ABAP-managed database procedures (AMDP) requires special considerations. It's only documented which functionality does have which impact on client isolation. There is no check ensuring that you as a service provider use these features in a multitenancy-compliant way.
- To ensure tenant lifecycle procedures, the applications must be built in such a way that the procedures for tenant copy, tenant move, and tenant delete can deal with it.

  > ⓘ Note
  >
  > If you don't comply with these aspects of the guideline, this causes issues where SAP, you as a provider, and the consumer are involved.

- Security and system logs store data in a cross-client persistence together.
- Any multitenancy-specific certifications from SAP in addition to the already existing ABAP environment product certifications are out of scope.

Guidelines for Multitenancy Applications in the ABAP environment

The following sections cover the relevant guidelines for content separation in multiclient systems. We use the multiclient architecture of the ABAP system for multitenancy enablement and list the design principles to reach multitenancy.

Prerequisites

- Store tenant-related data in client-dependent tables of type A, C, or L.
- Store system-related data in client-independent tables of type S.
- Always add the selection of the client to ABAP database procedures (AMDPs).

Make sure consumers cannot modify the client parameter or any other part of the AMDP using the application or by tampering requests.

- Don't generate development objects or other client-independent data system-locally in the provider system.
- Don't evaluate the actual value of the 3-digit client field (IF sy-mandt = 'nnn'. .... ENDIF).

Database Table Design

You have to classify database tables according to their content. There are the following types:

- Tenant Content (client-dependent)
  - Tenant configuration data – tables with delivery class "C"
  - Tenant application data – tables with delivery class "A"
  - Tenant temporary data – tables with delivery class "L"

Database tables for tenant content must be client-dependent. This means that the first field of the table must be of datatype "CLNT". We recommend using the inline declaration „abap.clnt".

Only the content of client-dependent "C" and "A" tables is considered during tenant copy and tenant move. Content of client-independent tables which are not delivered from the development system and "L" tables are lost during tenant lifecycle processes such as tenant move.

During tenant delete, the content of all client-dependent tables is removed.

The delivery class must be "C", "A", or "L".

The delivery classes "E", "G" and "W" are not supported in the ABAP environment at all.

- System Content (client-independent)
  - System configuration data – tables with delivery class "S"

Store data that is defined by the service provider and not specific for any tenant in a client-independent "S" table. Define the content in the respective development system and export it as TABU entries via a development transport request. The content is considered as code and imported like other development artifacts into subsequent systems such as the provider system.

Database Table Access

- Read Access Using ABAP SQL
  Whenever you access table content, ensure that you don't violate tenant isolation. ABAP SQL does not support additions such as CLIENT SPECIFIED or USING CLIENT in the ABAP environment. As a result, access to other clients than SY-MANDT is technically not possible. But you have to define client-dependent tables properly so that the ABAP runtime applies this role accordingly.
- Write Access Using ABAP SQL
  Do not write "S" tables in the provider system.

> **Note**
>
> If you write “S” tables in the provider system, this content is not considered during tenant lifecycle management events, such as tenant move, and you lose content. Furthermore, you violate tenant isolation, since every tenant has immediately access to the changes.

- Access Using AMDP
  Access to SAP HANA database using AMDP is not controlled via the ABAP database interface. The client is just handed over as a parameter. This can easily violate the content separation and access a table data for another client. Make sure that the client is always passed via SY-MANDT.
  Consumers must never be able to modify the client parameter or any other part of the AMDP using the application or by tampering requests.

  > **Note**
  >
  > In the future, it is highly recommended to use the AMDP option CDS SESSION CLIENT DEPENDENT or CLIENT INDEPENDENT.

Code Generation

Development artifacts such as Data Dictionary objects, classes, or interfaces, which are typically created via ABAP Development Tools (ADT), are always client-independent. Changes are always recorded in transport requests.

> **Note**
>
> It is not allowed to generate development artifacts in a non-development system, such as a provider system. This generation violates the content separation and is not be considered during tenant move and copy procedures. You are only allowed to generate development artifacts in **development systems** in the ABAP environment. You can use the XCO library for that purpose. If you generate development artifacts, the service provider is responsible for ensuring that the generated artifacts are transported into all required provider systems. Standard lifecycle events such as a tenant move don't consider such client-independent artifacts.

Multitenant-Aware Programming

- Cross-Client Locking (Enqueue)
  Lock Objects are associated to database tables or structures. The runtime API in the ABAP environment is `CL_ABAP_LOCK_OBJECT_FACTORY` and the returned instance of type `IF_ABAP_LOCK_OBJECT`.

  > **Note**
  >
  > Only lock client-dependent database tables or structures having a `CLIENT` field.

- CL_ABAP_CONTEXT_INFO=> GET_SYSTEM_DATE and TIME
  The system date/time (methods `CL_ABAP_CONTEXT_INFO=>GET_SYSTEM_DATE`, `CL_ABAP_CONTEXT_INFO=>GET_SYSTEM_TIME`) is always based on UTC time zone. The system time

zone can't be changed tenant-specifically. As a usual approach in cloud environments, it needs to be considered in your code, as it gives the same date and time for all tenants.

- Usage of System Information (sy-sysid and sy-mandt)
  Storing SY-SYSID and SY-MANDT explicitly in database tables is not allowed. This would lead to issues after tenant copy or tenant move, because both values are not unique and not stable in the system landscape. A dedicated API is provided to get a unique and stable identifier for the tenant. See Tenant [page 1812].

Connectivity

Outbound communication management in the ABAP environment differs from the on-premise usage of SM59 destinations (see Outbound Communication). It is recommended to connect tenants and systems via communication arrangements, see Communication Management.

> ⚠ Restriction
>
> The ABAP environment only supports one communication configuration layer that is intended for the consumer. It is currently not possible to enforce a communication configuration exclusively managed by the service provider within the consumer tenant.

Limitations

There might be some use cases where the guidelines provided are not applicable, for example, when sharing data across tenants or sharing aggregated customer data.

For such use cases, the operations concept needs to be documented and an alignment with SAP is required. We recommend checking the operational impact on tenant move and tenant copy procedures. In addition, we recommend a review for multiclient content separation.

Usage of SAP-Delivered SAP Fiori Apps

SAP has delivered several SAP Fiori apps for the administration of an ABAP environment system. These SAP Fiori apps are targeting different roles, such as administrator, project manager, and developer. Regarding multitenancy, some of these apps are covering client-independent objects and configuration according to their purpose. For example, the SAP Fiori app Manage Software Components allows you to import software components, which has an immediate impact on all tenants. Therefore, you need to carefully choose which SAP-delivered business catalogs to include in the business roles of your SaaS application.

4.2.14.2.4 Troubleshooting

Find out which steps you can take to identify and resolve specific issues that may come up when developing a multitenant application.

SaaS subscription error

- The *Landscape Portal* subscription might be missing. Make sure the *Landscape Portal* is subscribed to in the provider subaccount. This needs to be done before the SaaS subscription.
- The ASP_CC (CF Cloud Controller) destination might have failed: An ASP_CC destination with the correct credentials/URLs must exist in the provider subaccount (see Cloud Controller Access Destination [page 1329]). You can use a connection check in the destination for a basic connectivity test. You can also test if the user that was used in the ASP_CC destination is able to authenticate with the following cf command: cf login -a <cf-api-endpoint e.g. https://api.cf.eu10.hana.ondemand.com> -u <destination-user-name> -p <password> -s <provider-space> -o <provider-org>. Make sure the following is the case:
  - You have the credentials you require (space developer user). This user is a CF Platform User, i.e. it is either an S-User or a P-User, usually created using a distribution list so that it is a non-personalized e-mail address. A technical communication user that can also be created similar to normal S-Users cannot be used here.
  - You are using the correct URL: the URL needs to be the same as the CF API URL in the provider subaccount, depending on the region, see Region.
  - You are using the right Token Service URL for the region.
  - You are using a technical user as recommended, not a personal user. Personal user accounts can cause issues due to password changes etc.
- The space developer user might be missing: The user used in the credentials of the ASP_CC destination must be added as space developer in the space of the multitenant application.
- There might be missing entitlements for ABAP systems: During the subscription, the multitenant application/ASP automatically provisions ABAP systems. Make sure that the necessary entitlements are added to the provider subaccount:
  - abap/standard (for delivery via gCTS) or abap/saas_oem (for add-on based delivery), depending on the configuration of ASP parameter addon_product_name. See Multitenancy in the ABAP Environment [page 1349] and ABAP Solution Service [page 1320].
  - abap_compute_unit and hana_compute_unit depending on the configuration of ASP parameter size_of_runtime and size_of_persistence. See Multitenancy in the ABAP Environment [page 1349] and ABAP Solution Service [page 1320].
- There might be missing entitlements for multitenant app deployment. Make sure that the necessary entitlements are added to the provider subaccount:
  - Cloud Foundry Runtime entitlement for the approuter in the multitenant app.
  - ABAP Solution Service.
- In case of error "Illegal given domain name" during subscription:
  - The subscription fails because in the saas-registry service instance the URL defined for onSubscription and getDependencies callbacks does not match the pattern of the cis-... route assigned to the approuter application. Please make sure that the route defined in your saas-registry configuration for callbacks has the same pattern as defined in the cis-... route.

Cannot delete ABAP service instance

- If you cannot delete an ABAP service instance, the ABAP service instance might still contain *Test Tenant*, *Partner Customer Test*, or *Partner Customer Production* tenants. Make sure you delete these tenants first by unsubscribing from them (for tenants created via consumer subscription) or deleting the tenants in the *Landscape Portal* (for test tenants created directly in the *Landscape Portal*).
- The ABAP service instance might still contain a SAP_ASP service key: The service key needs to be deleted before the service instance can be deleted.

Multitenant application cannot be undeployed

- Always use MTA Undeployment (cf undeploy <MTA ID> --delete-services) instead of deleting service instances individually.
- Check if the saas-registry service instance still contains active subscriptions: the saas-registry instance can only be deleted once there are no more active subscriptions. Unsubscribe using the SAP BTP subscription management dashboard.

Cannot create consumer-specific route according to `TENANT_HOST_PATTERN` (in the development phase)

Sample Code

```
TENANT_HOST_PATTERN: (.*)-my-consumer-subdomain-productabcdefghijklmnopq-saas-
solution-dev.cfapps.eu10.hana.ondemand.com
Consumer Subaccount Subdomain: my-consumer-subdomain
Route Domain: cfapps.eu10.hana.ondemand.com
Route Hostname: my-consumer-subdomain-productabcdefghijklmnopq-saas-solution-
dev (64 characters)
```

- Keep in mind that the hostname of the route in CF is limited to 63 characters.
- In the development phase, the hostname part of the route should consist of the consumer subdomain, separator (“-“), and the appName.
- Either shorten the subdomain of the consumer subaccount or shorten the appName used in TENANT_HOST_PATTERN.

After deployment of MTA for multitenant application, no abap system is created

- ABAP systems are automatically created by the ABAP Solution Provider (ASP) during the first subscription.

Issues with parameter consumer_tenant_limit

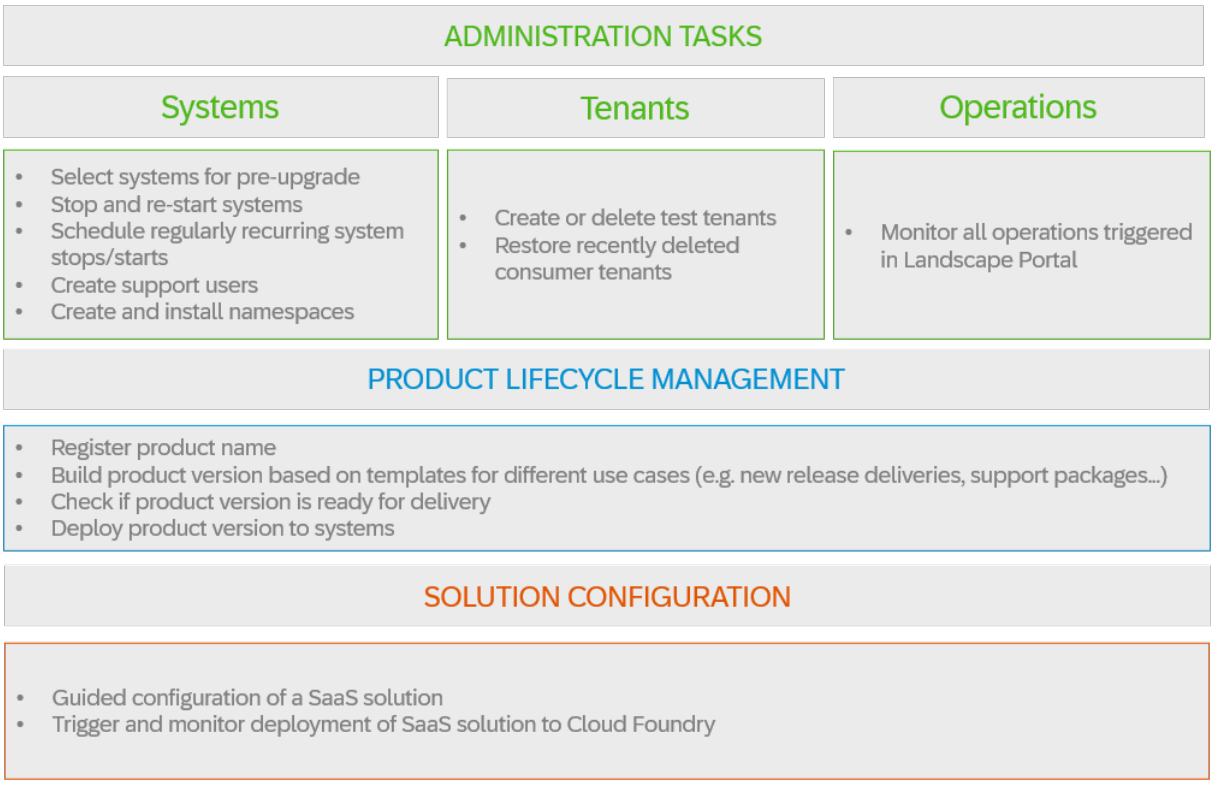
- Keep in mind that recently deleted consumer tenants are counted towards the tenant limit. Systems are automatically created.

First consumer tenant is missing application content after subscription

- Only in case of delivery via add-on, the application content is installed before the consumer tenant is accessible. In case of delivery via gCTS, the software components need to be imported manually using the *Manage Software Components* app in client 100 of the system.

4.2.14.3 Landscape Portal

The Landscape Portal acts as a central tool to allow service providers to perform administration tasks such as hibernating systems, creating test tenants and support users, selecting systems for a pre-upgrade, but also lifecycle management operations such as building, deploying and updating product versions, and even guides you through the configuration and deployment of multitenant SaaS solutions.



For more information about how to access and use the Landscape Portal, see Landscape Portal.

4.2.14.4 Extending SaaS Applications

Learn more about extending SaaS applications.

Providing Business Add-Ins [page 1361]
You, as an SAP partner, want to provide your customers with business add-ins (hereafter called BAdIs) so that customers can add their own business logic to your business processes by using the *Custom Logic* app.

Configuring Predefined Custom Fields [page 1366]

4.2.14.4.1 Providing Business Add-Ins

You, as an SAP partner, want to provide your customers with business add-ins (hereafter called BAdIs) so that customers can add their own business logic to your business processes by using the *Custom Logic* app.

This document describes the steps required for releasing a BAdI that can be used by key users of the *Custom Logic* app. The following steps are required:

- Create an enhancement spot for your BAdIs.
- Create the BAdI definition. This includes the interface definition and filter definitions.
- Release the BAdI definition.
- Provide an example implementation for your BAdI.
- Register the BAdI definition for the *Custom Logic* app.

After following these steps, your customers will be able to create BAdI implementations using the *Custom Logic* app.

4.2.14.4.1.1 Creating an Enhancement Spot for BAdIs

Context

Creating an enhancement spot for your BAdIs allows you to group BAdI definitions that you want to register for the *Custom Logic* app together in one place.

Procedure

Create a BAdI enhancement spot in ABAP Development Tools (ADT) using the creation wizard, as described in Creating BAdI Enhancement Spots.

4.2.14.4.1.2 Creating the BAdI Definition

Create a BAdI definition to provide your business add-in to the customer.

When creating a BAdI definition, please pay attention that some features may not be used, as indicated in the table below. When registering your BAdI for use within the *Custom Logic* app, make sure to perform a compatability check to verify that all restrictions are met.

BAdI Option Restrictions

Option	Description	Restriction	Comment
Description	The short description of the BAdI definition	Mandatory	Please keep the short description clear and intuitive for frontend display.
Instantiation	BAdI implementations are objects that may contain a state. Therefore, it's important whether subsequent similar requests for BAdI instances return the same or different instances of the BAdI implementation. For the instantiation of BAdIs, three different modes are supported.	Set to `Creating New Instances`	This option simplifies the instantiation process.
Multiple Use Indicator	If this indicator is set, several BAdI implementations can be active at the same time.	Set to `TRUE`	
Use Fallback Indicator	If this indicator is set, the fallback class will be executed in case no other implementation is called.	No restrictions	
Limited Filter Use Indicator	If this checkbox is marked, at most only one filter must be defined in the BAdI definition.	Set to `FALSE`	It's not allowed to prevent interference with the mandatory filter `TENANT_PREFIX`.
AMDP BAdI	Option to use a BAdI definition in an ABAP-Managed Database Procedure (AMDP)	Set to `FALSE`	This option can't be set due to the limitations of our app.
Example Classes	In order to demonstrate how to implement the BAdI method, you may use implementation example classes to provide a template for your customers.	Mandatory	Exactly one example class must be implemented for your customers to refer to. The class should contain meaningful restricted ABAP coding.
BAdI Interface	Defines the methods for this BAdI.	For restrictions, see the next section below	

Naming Conventions

The description of the BAdI should relate to, or be, a noun. For example, the description of the BAdI that converts A to B should rather be “A To B Conversion” than “Convert A To B”. The technical (ABAP) name should be derived from the description (shortened to 30 characters maximum, upper case only). The description and the technical name of the BAdI are visible to the customer.

Do not use technical terms like “Enhancement Spot”, “Extensibility”, or “Customer” in the description and in the technical name.

For the interface parameter names, use the following naming convention: Use `BUSINESSPARTNER` as importing parameter instead of `IV_BP_ID`. Do not use technical names, abbreviations, or the hungarian notation.

BAdI Interface

Define the method for the BAdI. Note that the BAdI interface must include the interface `IF_BADI_INTERFACE`, and no other interfaces should be included. Choose a name for the method, and remember to use exactly one method for the interface. Note the following restrictions for the interface **method signature**:

- Use only `IMPORTING` and `CHANGING` parameters. Use `CHANGING` parameters only for the output. This requirement is due to the BAdI framework and limitations of the app.
- Consider `CHANGING` parameters as returning parameters from the key users' point of view, which means that you shouldn't pass values through the parameters.
- Use `TYPE` as the typing method.
- The use of released CDS entities as parameters is allowed.
- The method must have the exception `CX_BLE_RUNTIME_ERROR` in its `RAISING` clause. No other exceptions must be raised. The framework catches all catchable exceptions, logs them and re-throws them as `CX_BLE_RUNTIME_ERROR`.
- The table type is allowed for `IMPORTING` and `CHANGING` parameters. Please use a limited number of fields to reduce the complexity.
- Nested tables are technically allowed for both `IMPORTING` and `CHANGING` parameters, but they should only be used in exceptional cases where their usage simplifies the process.
- Please use only a limited number of `IMPORTING` or `CHANGING` parameters.

> ⓘ Note
>
> Whenever possible, make the method signatures self-contained by using parameter types which you define directly in ABAP within the BAdI interface. Example: `TYPES ty_name TYPE c LENGTH 30.`

Additional Notes

All ABAP Dictionary types that are used in the parameter typing must be C1-released with the visibility *Use in Key User Apps*. It's often useful to create wrapper classes to control the set of released data types. Furthermore, any catchable exception raised from the key user coding (directly or indirectly by called methods) will be caught and re-raised in the previous field of `CX_BLE_RUNTIME_ERROR`. This is done by a try-catch

statement that is automatically wrapped around the key user's coding. Variable `IMPLEMENTATION_NAME` of exception `CX_BLE_RUNTIME_ERROR` contains the name of the implementation where the error occurred. If you show messages to the user, remember to always mention this implementation, so that the key user has a chance to find the error and fix it. Adding to that, non-catchable exceptions will still cause a backend dump, as in for example:

`ITAB_DUPLICATE_KEY`: Updating the unique table key `PRIMARY_KEY` resulted in a duplicate entry.

BAdI Filter Definition

Define a filter for your BAdI definition. Note that filter usage in BAdI implementations is restricted: Filters shouldn't overlap. To make sure that this is not the case, the filters of two distinct implementations of the same BAdI definition should not be `true` at the same time. See the table below for more restrictions and requirements when defining filters:

Filter Definitions

Option	Description	Recommendation/Restriction
Type	Data type of BAdI filter attribute.	The following types are available: • I - Integers • C - Character type • S - String • N - Numeric • P - Packed (not allowed)
Description	The short description of the filter.	This field is mandatory. Please keep the short description clear and intuitive for frontend display.
Only constant filter values	The field indicates filters which must be specified as constants when calling them. Therefore, you can already make sure during the compilation of your BAdI whether or not a corresponding BAdI implementation is called. This improves the runtime.	This option can be set depending on business use case. The default is that the indicator is not set.
Value check	It's possible to check the filter values maintained for BAdI implementations.	Set the value check to FALSE. Note that filter value checks are not supported.

> ### Note
>
> When implementing the BAdI, the key user maintains filter values to specify under which conditions the implementation will be called at runtime. The *Custom Logic* app only allows the definition of conditions in disjunctive normal form, and only allows comparisons containing a single operator. For example: `(area =1 AND category > 0 AND volume > 0 AND volume <= 100) OR (area = 1 AND category > 1 AND volume > 100 AND volumn <= 199) OR ...`
>
> Key users can maintain the conditions in a form-based editor. At runtime, at most one customer-provided implementation will be executed

Tenant Prefix Filter

Add a filter called `TENANT_PREFIX` with the type `CHARACTER` to your BAdI definition. This filter ensures that only BAdI implementations for the current tenant are executed at runtime. In your code, obtain the current tenant's prefix by using the ABAP API `cl_ble_api_mt_tenant`. The method `GET_PREFIX` returns the tenant prefix.

In the following example, the demo class `zcl_badi_demo` is used to illustrate the usage of `cl_ble_api_mt_tenant`. `z_badi` could be any BAdI that you want to register for use within the *Custom Logic* app.

```
CLASS zcl_badi_demo DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      my_method.
ENDCLASS.

CLASS zcl_badi_demo IMPLEMENTATION.
  METHOD my_method.
    DATA(lo_tenant_api) = cl_ble_api_mt_tenant=>create_instance( ).
    DATA(lv_tenant_prefix) = lo_tenant_api->get_prefix( ).

    DATA lo_badi TYPE REF TO z_badi.
    GET BADI lo_badi FILTERS tenant_prefix = lv_tenant_prefix.
    CALL BADI lo_badi->execute.
  ENDMETHOD.
ENDCLASS.
```

4.2.14.4.1.3 Providing an Example Implementation

Provide an example implementation of your BAdI for your customers. Note that only one example implementation is allowed. The example code must be written in restricted ABAP because this is the template for the key users, and the key users are only allowed to write restricted ABAP code.

As you can only use the ABAP language version *ABAP for Cloud Development* in your example class, it could happen that you write code that isn't allowed to be used by key users. To prevent this from happening, please create an implementation of your BAdI in the *Custom Logic* app on your test system. If you don't see syntax errors on the UI, your example code is written in restricted ABAP. Have a look at the code reference for key users here: Statements in ABAP for Key Users [page 3027].

4.2.14.4.1.4 Releasing the BAdI Definition

Context

Maintain the release status for development objects.

Procedure

1. Right-click the development object in the *Project Explorer* and choose *API State*. Select *Add Use System-Internally (Contract C1)* .
2. Set the visibility to *Use in Key User Apps*.
3. Maintain the release status in a similar way for the BAdI definition, the BAdI interface, and for all data types that are used in the BAdI method signature.

4.2.14.4.1.5 Registering the BAdI Definition

Context

To make your BAdI available in the *Custom Logic* app, follow the steps below.

Procedure

1. Right-click on your BAdI definition in the enhancement spot detail screen and choose *Custom Logic*.
2. Select *Check* to verify that your BAdI was implemented correctly. In case error messages appear, fix the root cause of the errors. Then, select *Register*.
3. Choose a transport request and select *Finish*. Your BAdI definition has now been registered.
4. To deregister your BAdI, right-click on the BAdI definition and select *Custom Logic*. Then, choose *Deregister*. Click *Next*, choose a transport request and select *Finish*. Now, the BAdI has been deregistered.

> ⓘ Note
>
> Do not deregister a BAdI that was already shipped to customers. There could be existing implementations that are no longer visible when the BAdI is gone.
>
> Remember to deregister the BAdI that is registered for *Custom Logic* first before deleting it. If you delete the BAdI without deregistering it, you'll receive an error message preventing you from saving the enhancement spot. You can go back to the last active version by closing the enhancement spot without saving.

4.2.14.4.2 Configuring Predefined Custom Fields

The predefined extension field framework allows you, as an SAP partner, to equip your SaaS applications with the support of customer-specific extension fields. Customers can extend partner applications with

customer-specific extension fields once the application has been deployed on SAP Business AI Platform, ABAP environment multitenancy.

An extensible partner application contains fields which are marked as predefined extension fields. Those fields are part of application but invisible in OData Services by default. You can enable the predefined custom fields in ABAP development tools for Eclipse and configure the extension fields using the Fiori app *Configure Predefined Custom Fields*.

Related Information

Working with Predefined Field Enablings
Configure Predefined Custom Fields [page 3030]
Manage API Snapshots [page 2936]

4.2.15 Released Components and Objects

To establish lifecycle independence of SAP code and your own applications as well as extensions built in the ABAP environment, you can only use released components and reuse services, together with a cloud-optimized scope of the ABAP language. Those components and reuse services form the public, upgrade-stable interface of the ABAP environment. Any other object or service that is not released can't be used for your own development.

In this chapter, you can learn more about released APIs from the ABAP language, technical APIs, and released services.

4.2.15.1 ABAP Language

Get an overview of all the objects and services of the ABAP programming language that were released for use in the ABAP environment.

You can use them for basic programming techniques such as data handling and database access. Note that not all objects and services that you know from the ABAP on-premise environment are available (or available in the same way).

More Information and Help

For more detailed information about the objects and services listed here, go to the ABAP Keyword Documentation (ABAP language help) or, in the relevant development object in ABAP Development Tools (ADT), directly to ABAP Doc.

To access the ABAP language help from the source code editor, position your cursor on an ABAP statement for which you need help, and press F1.

ADT Class Execution [page 1368]
Easily execute ABAP classes directly in ABAP Development Tools (ADT) without having to launch a service.

ABAP Managed Database Procedures (AMDP) [page 1369]
Use this class-based framework for managing and calling stored procedures or database procedures as AMDP procedures.

Arithmetic [page 1369]
Get an overview of the service classes that support arithmetic features such as conversions and random number generators.

Compression/Decompression [page 1369]
You can compress and decompress data using the `gzip` format.

Database Access [page 1369]
You can use locators and streams to access large binary or large character objects (BLOB and CLOB) in the database.

Date/Time and Time Stamp Processing [page 1370]

Parallel Processing [page 1370]
To start parallel tasks, use class `CL_ABAP_PARALLEL`.

Runtime Type Information [page 1372]
Implement the Runtime Type Services (RTTS) using a hierarchy of type description classes that contain the methods for Runtime Type Creation (RTTC) and Runtime Type Identification (RTTI).

Security [page 1372]
To support secure dynamic programming in ABAP, use class `CL_ABAP_DYN_PRG`.

String Processing [page 1373]

Structure Processing [page 1373]

System Information [page 1373]
To get technical information such as the current user, language, or time zone, use class `CL_ABAP_CONTEXT_INFO`.

XML/XSLT/ST [page 1373]
You can handle and process XML data using XSLT or ST programs.

Access to System Structure SY [page 1374]

4.2.15.1.1 ADT Class Execution

Easily execute ABAP classes directly in ABAP Development Tools (ADT) without having to launch a service.

Sample Code

```
CLASS zcl_example_class DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
```

```
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_example_class IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    out->write( 'Hello World!' ).
  ENDMETHOD.
ENDCLASS.
```

For more information, see the ABAP Doc comments of the `IF_OO_ADT_CLASSRUN` interface in ABAP Development Tools (ADT).

4.2.15.1.2 ABAP Managed Database Procedures (AMDP)

Use this class-based framework for managing and calling stored procedures or database procedures as AMDP procedures.

For more information, see the ABAP Keyword Documentation.

4.2.15.1.3 Arithmetic

Get an overview of the service classes that support arithmetic features such as conversions and random number generators.

For more information about numeric calculations, see the ABAP Keyword Documentation.

4.2.15.1.4 Compression/Decompression

You can compress and decompress data using the `gzip` format.

For more information, see the ABAP Doc comments of the classes in package `SABP_GZIP` in ABAP Development Tools (ADT).

4.2.15.1.5 Database Access

You can use locators and streams to access large binary or large character objects (BLOB and CLOB) in the database.

For more information, see the ABAP Keyword Documentation.

4.2.15.1.6 Date/Time and Time Stamp Processing

To handle and convert time stamps, use the following classes:

- `CL_ABAP_DATFM`
- `CL_ABAP_TIMEFM`
- `CL_ABAP_UTCLONG`
- `CL_ABAP_TSTMP`

For more information, see the ABAP Doc comments for these classes.

For more information about handling time stamps, see System Class for Time Stamps in Packed Numbers and System Class for Time Stamp Fields in the ABAP Keyword Documentation.

4.2.15.1.7 Parallel Processing

To start parallel tasks, use class `CL_ABAP_PARALLEL`.

Context

In an ABAP system, dialog processes are intended for processing dialog requests. In most cases, a large number of dialog processes is unused and you want to assign these free processes to parallel processing.

Procedure

Specify how many and which of the free dialog processes are used for parallel tasks when you create an instance of `CL_ABAP_PARALLEL`:

> **Note**
>
> Keep in mind that two of the free dialog processes on a server are always retained by the system for dialog requests.

- If you set parameter `P_LOCAL_SERVER` to value *'X'*, only processes of the current server are used. By default, the dialog processes of all servers of the current SAP system are used.
- With parameter `P_PERCENTAGE`, you can specify the share of dialog work processes that are reserved and available for the processing of asynchronous RFC tasks (aRFC) only. Keep in mind that this number does not correspond to the number of all generally available dialog work processes. If nothing is specified here, a value of 50 percent is used.

Transporting the data into a free dialog process may take some time. Therefore, it's not efficient to parallelize very short-running operations. The time limit is around 200 milliseconds.

→ Recommendation

Shorter operations should be bundled, that is, several short operations should be executed in one task.

Error Handling

When the framework is called, an internal table with the information on the tasks to be executed is transferred for each variant.

The result is a table that has the same number of entries as the input table, regardless of whether the tasks were actually executed. The output table also has the same order. A structure is returned as the result for each task. If the task was successful, the results of the processing are returned. In the case of the `RUNT_INST` method, for example, a result instance is returned in the `INST` component. If the processing was not successful for any reason, a message is returned instead. The corresponding field is the `MESSAGE` component in the result structure.

Messages from the framework are returned in this message field. For example, if a task is canceled due to a timeout, the following message is returned:

```
Time limit exceeded
```

If a task was aborted with a runtime error, the short text of the corresponding runtime error is provided as a message.

Example

In this example, method `RUN_INST` of class `CL_ABAP_PARALLEL` is used.

First, you have to define a processing class that implements the interface `IF_ABAP_PARALLEL`. The parallel processing starts with method `IF_ABAP_PARALLEL~DO`. The method in this example computes the square of a value.

{·} Sample Code

```
class LCL_PARALLEL_TASK definition.
  public section.
    interfaces IF_ABAP_PARALLEL.
    methods CONSTRUCTOR importing P_INPUT type I.
    data INPUT type I.
    data RESULT type I.
endclass.
class LCL_PARALLEL_TASK implementation.
  method CONSTRUCTOR.
    SUPER->CONSTRUCTOR( ).
    INPUT = P_INPUT.
  endmethod.
  method IF_ABAP_PARALLEL~DO.
    RESULT = INPUT * INPUT.
  endmethod.
endclass.
```

Afterwards, you have to start the parallel processing using method `RUN_INST`. The input for this method is a list of instances of the processsing class. The output is also a list of instances of the processsing class plus additional information.

Sample Code

```
class LCL_TEST_INST implementation.
   method START.
     data:
       L_IN_TAB type CL_ABAP_PARALLEL=>T_IN_INST_TAB,
       L_INST type ref to LCL_PARALLEL_TASK.
     data(L_REF) = new CL_ABAP_PARALLEL( ).
     do 1000 times.
       append new LCL_PARALLEL_TASK( SY-INDEX ) to L_IN_TAB.
     enddo.
     L_REF->RUN_INST( exporting P_IN_TAB = L_IN_TAB importing P_OUT_TAB =
data(L_OUT_TAB) ).
     loop at L_OUT_TAB assigning field-symbol(<L_OUT>) where INST is not
initial.
       L_INST ?= <L_OUT>-INST.
       " use result in L_INST->RESULT.
     endloop.
   endmethod.
endclass.
```

Related Information

For more information and examples, see the ABAP Doc comments of the `CL_ABAP_PARALLEL` class in the ABAP development tools for Eclipse (ADT).

4.2.15.1.8 Runtime Type Information

Implement the Runtime Type Services (RTTS) using a hierarchy of type description classes that contain the methods for Runtime Type Creation (RTTC) and Runtime Type Identification (RTTI).

Using these system classes, you can do the following:

- Determine type information of existing instances and type names in the ABAP type system at runtime.
- Define new data types at runtime.

For more information, see the ABAP Doc comments of class `CL_ABAP_TYPEDESCR` and its subclasses in the ABAP Development Tools (ADT).

4.2.15.1.9 Security

To support secure dynamic programming in ABAP, use class `CL_ABAP_DYN_PRG`.

For more information, see the ABAP Doc comments of this class in ABAP Development Tools (ADT).

4.2.15.1.10 String Processing

Use the following classes to process strings:

- `CL_ABAP_[STRING|CHAR]_UTILITIES`
  Provides auxiliary functions for string/character processing
- `CL_ABAP_CONV_CODEPAGE`
  For code page conversion of strings/characters
- `CL_ABAP_[REGEX|MATCHER]`
  For regular expression processing
  For more information about regular expressions, see the ABAP Keyword Documentation.

For more information about these classes, see their ABAP Doc comments in ABAP Development Tools (ADT).

4.2.15.1.11 Structure Processing

You can use the system class `CL_ABAP_CORRESPONDING` to assign components between structures or between internal tables with dynamically specified mapping rules. Methods are available for simple assignments, assignments of default values, and for using a lookup table.

For more information, see the ABAP Keyword Documentation.

4.2.15.1.12 System Information

To get technical information such as the current user, language, or time zone, use class `CL_ABAP_CONTEXT_INFO`.

To query for released objects, use the CDS view `I_APIsForSAPCloudPlatform`.

For more information, see the relevant ABAP Doc comments in ABAP Development Tools (ADT).

4.2.15.1.13 XML/XSLT/ST

You can handle and process XML data using XSLT or ST programs.

For more information about how XML data can be created and read in ABAP, see the ABAP Keyword Documentation.

4.2.15.1.14 Access to System Structure SY

Access to the system structure `SY` is restricted to read access to the following components:

`BATCH`, `DBCNT`, `FDPOS`, `INDEX`, `LANGU`, `MANDT`, `MSGID`, `MSGNO`, `MSGTY`, `MSGV1`, `MSGV2`, `MSGV3`, `MSGV4`, `SYSID`, `SUBRC`, `TABIX`, and `UNAME`.

> ⓘ Note
>
> Access to all other components is **not** allowed because they are related to either obsolete or unsupported features.

For more information, see ABAP System Fields in the ABAP Keyword Documentation.

You can use class `CL_ABAP_CONTEXT_INFO` to retrieve information about the user session, for example, technical user name, business user name, time zone, and so on. The built-in function `utclong_current` generates a UTC time stamp from the current system time and the current system date in accordance with POSIX standards.

The `XCO` time library also offers `SY` representations, especially when it comes to time and date retrieval. For more information about `XCO`, see Time Library [page 1813].

4.2.15.2 Change Document Solution

You can document changes made to a commercial object, such as the time, the content and the way changes are made, by logging these changes in a change document.

> ✤ Example
>
> You can use the change document to simplify the change history analysis for auditing in *Financial Accounting*.

Every application object type has its own change document object type, which is called the *Change Document Object*. To log changes to a commercial object in a change document, you must define the *Change Document Object* for the commercial object type. The *Change Document Object* definition contains the tables which represent a commercial object in the system.

> ⓘ Note
>
> - Specifiy for each table, whether a commercial object contains only one (single case) or several (multiple case) records. For example, an order contains an order header and several order items. In general, one record for the order header and several records for the order items are passed to the change document creation when an order is changed.
> - If a table contains fields with values referring to units and currency fields, the associated table, containing these units and currencies, can also be specified.
> - The object ID identifies a given commercial object. You can retrieve all changes made to a commercial object using this key.

Change Document Structure

- Change Document Header
  The change document header is the part of the change document which contains general information about the changes made. The document header is identified by object class name, the object ID and a change document number (automatically provided by the runtime system). The change document header contains information about who made the changes, when they were made, what type of changes were made (update, insert or delete operation) and if they were real or planned changes. Starting with Release 6.20 information about the system language and the length of the table key of the documented table are saved here as well. The header data is stored in table `CDHDR`.
- Change Document Item
  The change document item contains the old and new values of a field for a particular change, and a change flag, that presents the kind of change. The change flag can take the following values:
  - U = Update
    Changed data. (Log entry for each changed field which was flagged in the Dictionary as *change document relevant*.
  - I = Insert
    Data Inserted.
    Changes: Log entry for the whole table record.
    Planned changes: Log entry for each table record field.
  - D = Delete
    Data were deleted (log entry for the whole table record).
  - E = Individual field documentation at Delete
    Delete a table record with field documentation.
    One log entry per field of the deleted table entry.
  - J = Individual field documentation at Insert
    Insert a table record with field documentation.
    One log entry per field of the inserted table entry.

4.2.15.2.1 Authorization Checks

Authorization Check for Change Document Object Maintenance

Use method `IF_CHDO_OBJECT_TOOLS_REL~CHECK_AUTHORIZATION` to run an additional authorization check.

`IV_OBJECT. IV_DEVCLASS` and `IV_ACTIVITY` get passed as import parameters. The return parameter `RV_IS_AUTHORIZED` must be set to `ABAP_TRUE` if the check is successful.

Example

Sample Code

```
CLASS zcl_chdo_test_auth DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
```

```
  PUBLIC SECTION.
   INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_chdo_test_auth IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    DATA: lv_is_authorized type abap_bool.
    TRY.
      " iv_object : Change document object name
      " it_activity : Activity to be checked. Possible values '01' = create,
      "                                                       '02' = change,
      "                                                       '03' = read,
      "                                                       '06' = delete
      " it_devclass : development class of change document object

cl_chdo_object_tools_rel=>if_chdo_object_tools_rel~check_authorization(
        EXPORTING
          iv_object        = 'ZCHDO_TEST'
          it_activity      = '03'
          it_devclass      = 'ZLOCAL'
        RECEIVING
          rv_is_authorized = lv_is_authorized
      ).
    ENDTRY.
    IF lv_is_authorized IS INITIAL.
      out->write( |Exception occurred: authorization error.| ).
    ELSE.
      out->write( |Activity can be performed on the change document
object.| ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

Authorization Check for Reading Change Documents

When a change document object is generated, you receive the `CL_<change document object name>_CHDO` class with method `IF_CHDO_ENHANCEMENTS~AUTHORITY_CHECK` without implementation. You can create your own authority check for reading change documents written for this change document object. The authority check for reading change documents are successful if parameter `RV_IS_AUTHORIZED = 'X'` is returned.

4.2.15.2.2 Managing Change Document Objects

To maintain a change document object during design time, the `ABAP Development Tools` UI editor for change document objects should be used. For more information, see Working with Change Document Objects.

You can use class `CL_CHDO_OBJECT_TOOLS_REL` if you want or have to maintain change document objects programmatically.

For more information, see

- Creating a Change Document Object [page 1377]

- Updating a Change Document Object [page 1382]
- Deleting a Change Document Object [page 1387]
- Reading a Change Document Object Definition [page 1388]

Related Information

Working with Change Document Objects

4.2.15.2.2.1 Creating a Change Document Object

When creating a change document object, the following naming-rules apply:

- For customer objects, start the name with a 'Z' or a 'Y'. For more information, see SAP Note 16466.
- If a name space is requested, use an object starting with namespace as object name.
- Keep in mind that the change document object name including the name space has a maximum length of 15 characters. That means that if the name space has 10 characters, only 5 characters are left for the change document object name.

Process

Use method `IF_CHDO_OBJECT_TOOLS_REL~CREATE_AND_GENERATE_OBJECT` to create and generate change document objects.

The name of the object is assigned using the import parameter `IV_OBJECT`. Object details and generation information are passed using the internal tables `IT_CD_OBJECT_DEF` (the object definition), `IT_CDOBJECT_TEXT` (object texts) and `IS_CD_OBJECT_GEN` (generation information).

Once generated, the class `(<name space>CL_<change document object name>_CHDO)` with methods `WRITE` and `IF_CHDO_ENHANCEMENTS~AUTHORITY_CHECK` are created. `IV_CL_OVERWRITE` can be used to specify whether an existing class with the specified name can be overwritten or not. If a class already exists and can't be overwritten, no changes will be made. Changes are saved in the transport request (`IV_CORRNR`).

Method `AUTHORITY_CHECK` of interface `'IF_CHDO_ENHANCEMENTS'` must be implemented later on in order to enable an authorization check for reading change documents.

The export parameter `ET_ERRORS` is used to return all generation messages (messages from message class `CD`). Any syntax errors in the generated class are provided using `ET_SYNT_ERROR` (with long text if applicable (`ET_SYNT_ERROR_LONG`)).

Import Parameters

Parameter Name	Field Name	Value Help
IV_OBJECT		Name of change document object
IT_CD_OBJECT_DEF		Change document object definition

Parameter Name	Field Name	Value Help
	TABNAME	Name of the table as defined in the dictionary
	MULTCASE	If more than one record for a particular table is to be documented during a single `CREATE/UPDATE/DELETE` operation, the value `MULTCASE` should be `'ABAP_TRUE'` or `'X'` (multiple case). If no value is provided a single record can be documented and passed in a work area (single case).
	DOCUDEL	DOCUDEL = SPACE The deletion of a table entry will be documented in one change document item using the table key to identify the table entry deleted. The field FNAME will be filled with `'KEY'`. DOCUDEL = 'X' Each change document relevant field value of the table entry will be documented individually in its own change document item. The change indicator is 'E' instead of 'D'.
	DOCUINS	DOCUINS = SPACE The insert of a table entry will be documented in one change document item using the table key to identify the inserted table entry. The field FNAME will be filled with 'KEY'. DOCUINS = 'X' Each change document relevant field value of the table entry will be documented individually in its own change document item. The change indicator is 'J' instead of 'I'.

Parameter Name	Field Name	Value Help
	REFNAME	If fields of table TABNAME reference unit or currency field values from another table, the name of that table must be passed here to document its values as well. Only one reference table can be used for table TABNAME. In single case, the referenced entry from table REFNAME is passed as two additional work areas (old, new). In multiple case, the import tables (old, new) are enhanced to include the referenced structure of table REFNAME.
	DOCUD_IF	If you want to document the value of a field even though it is initial when data is deleted, mark this field. If it is not marked, log entries will only be written if the field values are not initial when they are deleted. Be aware that a lot of additional change documents may be written, if you choose this option. Only mark this flag if it is required.
	DOCUI_IF	If you want to document the value of a field even though it is initial when data is inserted, mark this field. If it is not marked, log entries will only be written if the field values are not initial when they are inserted. Be aware that a lot of additional change documents may be written, if you choose this option. Only mark this flag, if it is required.
IT_CD_OBJECT_TEXT		Object texts for change document object
	LANG_KEY	Language key of the text
	OBJECT_TEXT	Descriptive short text for the change document object
IS_CD_OBJECT_GEN		Change document object generation information
	AUTHOR	User who performs the generation

Parameter Name	Field Name	Value Help
	UPDNAME	User who performs the change
	CHANGE_DATE	Date of change
	CHANGE_TIME	Time of change
	TEXTCASE	Special Text Handling flag Select this field to log long text changes. The old and new status of long texts is not logged. Only the fact that they have been changed is noted.
	DEVCLASS	Change document object package
IV_CL_OVERWRITE		Whether generated class should overwrite an already existing class. Value 'X' means an existing class will be overwritten.
IV_CORRNR		Transport request where changes should be logged

Export Parameters

Parameter Name	Field Name	Value Help
ET_ERRORS		
	kind	Message type (emtpy means information message, ,E-, means error)
	msgid	Message class (CD)
	msgnr	Message ID
	v1	Variable to message
	v2	Variable to message
	v3	Variable to message
	v4	Variable to message
	text	Short text of the message
ET_SYNT_ERRORS		Syntax errors raised during generation of class. Check syntax of generated class directly.

Parameter Name	Field Name	Value Help
ET_SYNT_ERROR_LONG		Syntax errors raised during generation of class. Check syntax of generated class directly.

Sample Code

```
CLASS zcl_chdo_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
   INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_chdo_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    DATA:
      ls_error       TYPE abap_bool,
      rt_errors      TYPE if_chdo_object_tools_rel=>ty_tr_error_tab,
      lt_errors_err  TYPE LINE OF if_chdo_object_tools_rel=>ty_tr_error_tab,
      p_it_tcdob     TYPE if_chdo_object_tools_rel=>ty_tcdobdef_tab,
      p_it_tcdobt    TYPE if_chdo_object_tools_rel=>ty_tcdobtext_tab,
      p_it_tcdrp     TYPE if_chdo_object_tools_rel=>ty_tcdgen,
      lt_tcdobt      TYPE if_chdo_object_tools_rel=>ty_tcdobt_tabtyp,
      ls_tcdob       TYPE LINE OF if_chdo_object_tools_rel=>ty_tcdobdef_tab,
      ls_tcdobt      TYPE LINE OF if_chdo_object_tools_rel=>ty_tcdobtext_tab.
    data: lr_err                TYPE REF TO cx_chdo_generation_error.
    ls_tcdob-tabname = 'ZCHDO'.
    ls_tcdob-multcase = ' '.
    ls_tcdob-docudel = ' '.
    ls_tcdob-docuins = ' '.
    ls_tcdob-docud_if = ' '.
    APPEND ls_tcdob TO p_it_tcdob.
    ls_tcdobt-lang_key = 'D'.
    ls_tcdobt-object_text = 'Single Case'.
    APPEND ls_tcdobt TO p_it_tcdobt.
    p_it_tcdrp-author = 'X11'.
    p_it_tcdrp-textcase = 'X'.
    p_it_tcdrp-devclass = '<development class>'.
    CLEAR: rt_errors, lr_err.
    TRY.

cl_chdo_object_tools_rel=>if_chdo_object_tools_rel~create_and_generate_object(
        EXPORTING
          iv_object          = 'ZCHDO_TEST' " change document object name
          it_cd_object_def   = p_it_tcdob   " change document object
definition
          it_cd_object_text  = p_it_tcdobt  " change document object text
          is_cd_object_gen   = p_it_tcdrp   " change document object
generation info
          iv_cl_overwrite    = 'X'          " class overwrite flag
          iv_corrnr          = '<transport_request>' " transport request
number
        IMPORTING
          et_errors          = rt_errors    " generation message table
*          et_synt_errors     =
*          et_synt_error_long =
      ).
      CATCH cx_chdo_generation_error into lr_err.
        out->write( |Exception occurred: { lr_err->get_text( ) }| ).
        ls_error = 'X'.
```

```
    ENDTRY.
      IF ls_error IS INITIAL.
        READ TABLE rt_errors WITH KEY kind = 'E-'
                                  INTO lt_errors_err.
        IF sy-subrc IS INITIAL.
          out->write( |Exception occurred: { lt_errors_err-text } | ).
        ELSE.
          out->write( |Change document object created and generated | ).
        ENDIF.
      ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.2.2.2 Updating a Change Document Object

Use method `IF_CHDO_OBJECT_TOOLS_REL~UPDATE_OBJECT` to modify and regenerate change document objects.

The name of the object is assigned using the import parameter `IV_OBJECT`. The object details and generation information are passed using the internal tables `IT_CD_OBJECT_DEF` (the object definition), IT_CD_OBJECT_TEXT (the object texts) and IS_CD_OBJECT_GEN (the generation information).

If the internal tables are not filled, the object definition is read directly from the database tables `TCDOB` and `TADIR` and the generation information is read directly from database table `TCDRP`. If only parameter `IS_CD_OBJECT_GEN` is passed, the change document object will be newly generated without changing the change document object definition. Use this method when the structure of a table, that belongs to the change document object, was changed.

Once generated, a class (<name space>CL_<change document object name>_CHDO) with methods `WRITE` and `IF_CHDO_ENHANCEMENTS~AUTHORITY_CHECK` are created. You can use `IV_CL_OVERWRITE` to specify whether an existing class can be overwritten with the specified name or not. If a class already exists and can't be overwritten, no changes will be made. Changes made to the class are saved in the transport request (`IV_CORRNR`).

Method `AUTHORITY_CHECK` of interface `'IF_CHDO_ENHANCEMENTS'` must be implemented later on in order to enable an authorization check for reading change documents.

The export parameter `ET_ERRORS` is used to return all generation messages (messages from message class `CD`) . Any syntax errors in the generated class are provided using `ET_SYNT_ERROR` (with long text if applicable (`ET_SYNT_ERROR_LONG`)).

Import Parameters

Parameter Name	Field Name	Value Help
IV_OBJECT		Name of change document object
IT_CD_OBJECT_DEF		Change document object definition
	TABNAME	Name of the table as defined in the dictionary

Parameter Name	Field Name	Value Help
	MULTCASE	If more than one record for a particular table is to be documented during a single `CREATE/UPDATE/DELETE` operation, the value `MULTICASE` should be `'ABAP_TRUE'` or `'X'` (multiple case). If no value is provided a single record can be documented and passed in a work area (single case).
	DOCUDEL	DOCUDEL = SPACE The deletion of a table entry will be documented in one change document item using the table key to identify the table entry deleted. The field FNAME will be filled with 'KEY'. DOCUDEL = 'X' Each change document relevant field value of the table entry will be documented individually in its own change document item. The change indicator is 'E' instead of 'D'.
	DOCUINS	DOCUINS = SPACE The insert of a table entry will be documented in one change document item using the table key to identify the inserted table entry. The field FNAME will be filled with 'KEY'. DOCUINS = 'X' Each change document relevant field value of the table entry will be documented individually in its own change document item. The change indicator is 'J' instead of 'I'.

Parameter Name	Field Name	Value Help
	REFNAME	If fields of table TABNAME reference unit or currency field values from another table, the name of that table has to be passed here to document its values as well. Only one reference table can be used for table `TABNAME`. In single case, the referenced entry from table `REFNAME` is passed as two additional work areas (old, new). In multiple case, the import tables (old, new) are enhanced to include the referenced structure of table `REFNAME`.
	DOCUD_IF	If you want to document the value of a field even though it is initial when data is deleted, mark this field. If it is not marked, log entries will only be written if the field values are not initial when they are deleted. Be aware that a lot of additional change documents may be written, if you choose this option. Only mark this flag if it is required.
	DOCUI_IF	If you want to document the value of a field even though it is initial when data is inserted, mark this field. If it is not marked, log entries will only be written if the field values are not initial when they are inserted. Be aware that a lot of additional change documents may be written, if you choose this option. Only mark this flag, if it is required.
IT_CD_OBJECT_TEXT		Object texts for change document object
	LANG_KEY	Language key of the text
	OBJECT_TEXT	Descriptive short text for the change document object
IS_CD_OBJECT_GEN		Change document object generation information

Parameter Name	Field Name	Value Help
	AUTHOR	User who performs the generation
	UPDNAME	User who performs the change
	CHANGE_DATE	Date of change
	CHANGE_TIME	Time of change
	TEXTCASE	Special Text Handling flag Select this field to log long text changes. The old and new status of long texts is not logged. Only the fact that they have been changed is noted.
	DEVCLASS	Change document object package
IV_CL_OVERWRITE		Whether generated class should overwrite an already existing class. Value 'X' means an existing class will be overwritten.
IV_CORRNR		Transport request where changes should be logged

Export Parameters

Parameter Name	Field Name	Value Help
ET_ERRORS		
	kind	Message type (emtpy means information message, ,E-, means error)
	msgid	Message class (CD)
	msgnr	Message ID
	v1	Variable to message
	v2	Variable to message
	v3	Variable to message
	v4	Variable to message
	text	Short text of the message
ET_SYNT_ERRORS		Syntax errors raised during generation of class. Check syntax of generated class directly.

Parameter Name	Field Name	Value Help
ET_SYNT_ERROR_LONG		Syntax errors raised during generation of class. Check syntax of generated class directly.

Sample Code

```
CLASS zcl_chdo_update_object DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
   INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_chdo_update_object IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
  DATA:
        rt_errors             TYPE if_chdo_object_tools_rel=>TY_TR_ERROR_TAB,
        lt_errors_err         TYPE LINE OF
if_chdo_object_tools_rel=>TY_TR_ERROR_TAB,
        lt_tcdob              TYPE iF_CHDO_OBJECT_TOOLS_REL=>TY_TCDOBDEF_TAB,
        P_IT_TCDOB            TYPE iF_CHDO_OBJECT_TOOLS_REL=>TY_TCDOBDEF_TAB,
        P_IT_TCDOBT           TYPE IF_CHDO_OBJECT_TOOLS_REL=>TY_TCDOBTEXT_TAB,
        P_IT_TCDRP            TYPE IF_CHDO_OBJECT_TOOLS_REL=>TY_TCDGEN,
        lt_tcdobt             TYPE IF_CHDO_OBJECT_TOOLS_REL=>TY_TCDOBTEXT_TAB,
        ls_tcdob              TYPE LINE OF
iF_CHDO_OBJECT_TOOLS_REL=>TY_TCDOBDEF_TAB,
        ls_tcdobt             TYPE LINE OF
IF_CHDO_OBJECT_TOOLS_REL=>TY_TCDOBTEXT_TAB.
  data: lr_err                TYPE REF TO cx_chdo_generation_error.
  ls_tcdob-tabname = 'ZCHDO'.
  ls_tcdob-multcase = 'X'.
  ls_tcdob-docudel = ' '.
  ls_tcdob-docuins = ' '.
  ls_tcdob-docud_if = ' '.
  APPEND ls_tcdob TO p_it_tcdob.
  ls_tcdobt-lang_key = 'D'.
  ls_tcdobt-object_text = 'Single Case: Update'.
  APPEND ls_tcdobt TO p_it_tcdobt.
  p_it_tcdrp-author = sy-uname.
  p_it_tcdrp-textcase = 'X'.
  p_it_tcdrp-devclass = <development class>'.
  CLEAR: rt_errors, lr_err.
  TRY.
    cl_chdo_object_tools_rel=>if_chdo_object_tools_rel~update_object(
      EXPORTING
        iv_object          = 'ZCHDO_TEST'
        it_cd_object_def   = p_it_tcdob
        it_cd_object_text  = p_it_tcdobt
        is_cd_object_gen   = p_it_tcdrp
        iv_cl_overwrite    = 'X'
        iv_corrnr          = '<transport request>'
      IMPORTING
        et_errors          = rt_errors
    ).
    CATCH cx_chdo_generation_error INTO lr_err.
  ENDTRY.
      IF lr_err IS INITIAL.
        READ TABLE rt_errors WITH KEY kind = 'E-'
                                  INTO lt_errors_err.
        IF sy-subrc IS INITIAL.
```

```
          out->write( |Exception occurred: { lt_errors_err-text } | ).
        ELSE.
          out->write( |Change document object updated | ).
        ENDIF.
      ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.2.2.3 Deleting a Change Document Object

Use method `IF_CHDO_OBJECT_TOOLS_REL~DELETE_OBJECT` to delete change document objects. The name of the object is assigned using the import parameter `IV_OBJECT`.

Furthermore, the import parameter `IV_DEL_CL_WHEN_USED` determines, if the class of the change document object should be deleted (if value passed is `ABAP_TRUE`) or not (value `ABAP_FALSE`) when it is still being used. The changes made to the class are saved in the transport request (`IV_CORRNR`).

The export parameter `ET_ERRORS` is used to return all deletion messages (messages from message class `CD`).

The object will be deleted if no exception and no error messages of kind ‘E-‘ were raised.

Import Parameters

Parameter Name	Field Name	Value Help
IV_OBJECT		Change document object name
IV_CORRNR		Transport request where deletion should be logged
IV_DEL_CL_WHEN_USED		Delete generated class "CL_<change document object name>_CHDO" even if it is currently used.

Export Parameters

Parameter Name	Field Name	Value Help
ET_ERRORS		
	kind	Message type (emtpy means information message, ‚E-, means error)
	msgid	Message class (CD)
	msgnr	Message ID
	v1	Variable to message
	v2	Variable to message
	v3	Variable to message

Parameter Name	Field Name	Value Help
	v4	Variable to message
	text	Short text of the message

Sample Code

```
CLASS zcl_chdo_delete_object DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
   INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_chdo_delete_object IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    data: ls_error              TYPE abap_bool,
          lr_err                TYPE REF TO cx_chdo_generation_error,
          lt_errors_err         TYPE LINE OF
if_chdo_object_tools_rel=>ty_tr_error_tab,
          rt_errors             TYPE
if_chdo_object_tools_rel=>ty_tr_error_tab.
    TRY.
      cl_chdo_object_tools_rel=>if_chdo_object_tools_rel~delete_object(
        EXPORTING
          iv_object           = 'ZCHDO_TEST'    " change document object name
          iv_corrnr           = '<transport_request>'   " transport request
number
          iv_del_cl_when_used = 'X'             " delete class even when it
is used
        IMPORTING
          et_errors           = rt_errors       " messages from deletion
      ).
      CATCH cx_chdo_generation_error into lr_err.
        out->write( |Exception occurred: { lr_err->get_text( ) }| ).
        ls_error ='X'.
    ENDTRY.
    IF ls_error IS INITIAL.
      READ TABLE rt_errors WITH KEY kind = 'E-'
                             INTO lt_errors_err.
      IF sy-subrc IS INITIAL.
        out->write( |Exception occurred: { lt_errors_err-text } | ).
      ELSE.
        out->write( |Change document object deleted | ).
      ENDIF.
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.2.2.4 Reading a Change Document Object Definition

Use method `IF_CHDO_OBJECT_TOOLS_REL~READ_OBJECT` to read a change document object definition.

The name of the object is assigned using the import parameter `IV_OBJECT`. The information is returned using the export parameter `ET_OBJECT_INFO`.

Import Parameter

Parameter Name	Field Name	Value Help
IV_OBJECT		Change document object name

Export Parameters

Parameter Name	Field Name	Value Help
ET_OBJECT_INFO		
	ET_OBJECT_INFO	Name of the change document object
	ET_OBJECT_INFO	Generation information counter for field gen_type
	ET_OBJECT_INFO	Generation information type technical name (DEFINITION, GENERATION, CLASS)
	ET_OBJECT_INFO	Generation information type text (Definition, Generate, Class details)
	ET_OBJECT_INFO	Line counter for field name
	ET_OBJECT_INFO	Information long text
	ET_OBJECT_INFO	Information technical name
	ET_OBJECT_INFO	Value of the change document object
	OBJECTCLASS	Name of the change document object
	GNR	Generation information counter for field gen_type
	GEN_TYPE	Generation information type technical name (DEFINITION, GENERATION, CLASS)
	GROUP	Generation information type text (Definition, Generate, Class details)
	ZNR	Line counter
	INFOTEXT	Information long text
	OBJ_TYPE	Information technical name
	NAME	Information value (e.g. table name, package, user name, class name)

Sample Code

```
CLASS zcl_chdo_read_object DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
   INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_chdo_read_object IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    DATA: pt_object_info TYPE if_chdo_object_tools_rel=>tty_object_info.
    data: lr_err               TYPE REF TO cx_chdo_generation_error.
      TRY.
      cl_chdo_object_tools_rel=>if_chdo_object_tools_rel~read_object(
        EXPORTING
          iv_object      = 'ZCHDO_TEST'     " change document object name
        IMPORTING
          et_object_info = pt_object_info   " change document object details
      ).
      CATCH cx_chdo_generation_error INTO lr_err.
        out->write( |Exception occurred: { lr_err->get_text( ) }| ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.2.3 Writing Change Documents

The `WRITE` method in the generated class `CL_<change document object name>_CHDO` creates change documents from the object-specific update for an object ID.

If the *Fiori Reuse Library for Change Document Solution* will be used to display the change documents, time of change (UTIME) and date of change (UDATE) need to be provided in system time zone.

Import Parameters

Parameter Name	Value Help
OBJECTID	Identifier of the changed application object
UTIME	Time of change (If the *Fiori Reuse Library for Change Document Solution* is used provide information in system time zone.)
UDATE	Date of change (If the *Fiori Reuse Library for Change Document Solution* is used provide information in system time zone.)
USERNAME	User name of the person responsible in change document
PLANNED_CHANGE_NUMBER	Planned change number (only for writing planned changes)

Parameter Name	Value Help
OBJECT_CHANGE_INDICATOR	Type of Change to Application Object. Possible values are: • U - Change • I - Insert • D - Delete
PLANNED_OR_REAL_CHANGES	You can use this parameter to control whether the changes to be logged are actual or planned changes. Available values: • "R" actual (real) changes -"P" planned changes • if there is no plan number: actual change if there is a plan number: planned change
NO_CHANGE_POINTERS	'X': no change pointers will be written. Change pointers are used for Application Link Enabling (ALE) master data scenarios. Currently, this parameter is obsolete for SAP Business Technology Platform (SAP BTP), ABAP Environment.
ICDTXT_<change document object name>	In this structure, the change document-relevant texts are collected with the corresponding specifications: • TEILOBJID: Key of changed table row • TEXTART: Text type of changed text • TEXTSPR: Language Key • UPDKZ: Change flag for table row: D(elete), I(nsert) or U(pdate)
UPD_ICDTXT_<change document object name>	Change flag for text table: • " " (Space): Table is not considered. • "U": Table is considered
O_<table name>	Object-specific parameter for single case tables with the same table structure as <table name>. The workarea must contain the original data.
N_<table name>	Object-specific parameter for single case tables with the same table structure as <table name>. The workarea must contain the original data.
Y<table name>	Object-specific parameter for multiple case tables. The table must contain the original version of the changed or deleted records. The structure consists of the table, as specified in the change document object definition under table name, a processing flag (TYPE C, length 1)

Parameter Name	Value Help
X<table name>	The table must contain the current version of the changed or created records. The structure is the same as Y<table name> (see above). Is is not necessary to set the processing flag to values <> space for standard behavior. The following values for the processing flag exist: • "I" (INSERT)Records were created, or table records were deleted, then a record with the same key was created in the same transaction, and this is to be documented as "Delete" and "Create" (special case), not as "Change". • "U" or " " (space) (UPDATE) The parameter UPD_<table name > (see below) initially determines whether the record is new or changed. The processing flag is only checked when - with the parameter value "U" - the following key comparison between the two tables x<table name> and y<table name> finds two records with the same key.

Parameter Name	Value Help

UPD_<table name >	With this flag, you determine the processing logic. The following values exist:
	• "D" (DELETE) A change document item is to be created for each record in Y<table name >. X<table name> is not processed.
	• "I" (INSERT)A change document item is to be created for each record in X<table name>. Y<table name> is not processed.
	• "U" (UPDATE)The keys of Y<table name> and X<table name> are compared. The following cases are distinguished:
	  • Record exists in Y<table name> but not in X<table name>: Change document items are to be created for the record in Y<table name> which is to be deleted.
	  • Record exists in X<table name> but not in Y<table name>: A change document item is to be created for the records in X<table name> which are to be flagged as created.
	  • Record exists in both Y<table name> and X<table name> and processing flag in X<table name> record is "U" or space: A change document item is created for each changed field which is defined as change document-relevant in the dictionary.
	  • Record exists in both Y<table name> and X<table name> and processing flag in X<table name> record is "I": Chage document item is to be created for the record in Y<table name> which is to be deleted and change document item is to be created for the record in X<table name> which is to be flagged as created.
	  • " " (space, no processing) X<table name> and Y<table name> are not processed by the `WRITE` method. (If no changes have been made, the processing can be skipped to save time.)

Export Parameter

Parameter Name	Value Help
CHANGENUMBER	Change number of the document

Sample Code

```
" example for change document object ZTEST_1
  " with tables
  " ZTEST_1 single case and
```

```
  " ZTEST_2 multiple case
  " Start of default parameter part
  DATA: objectid        TYPE if_chdo_object_tools_rel=>ty_cdobjectv,
        utime           TYPE if_chdo_object_tools_rel=>ty_cduzeit,
        udate           TYPE if_chdo_object_tools_rel=>ty_cddatum,
        username        TYPE if_chdo_object_tools_rel=>ty_cdusername,
        cdoc_upd_object TYPE if_chdo_object_tools_rel=>ty_cdchngindh VALUE
'U'.
  DATA: cdchangenumber  TYPE if_chdo_object_tools_rel=>ty_cdchangenr.
  " End of default parameter part
  " Begin of dynamic DATA part for class ZCL_ZTEST_1_CHDO
  " declaration for the long text : ICDTXT_ZTEST_1
  DATA icdtxt_ztest_1 TYPE if_chdo_object_tools_rel=>ty_cdtxt_tab.
  " update indicator for the long text
  DATA upd_icdtxt_ztest_1 TYPE if_chdo_object_tools_rel=>ty_cdchngindh.
  " workaera_old of ZTABLE_1
  DATA os_ztable_1 TYPE ztable_1.
  " workaera_new of ZTABLE1
  DATA ns_ztable_1 TYPE ztable_1.
  " change indicator for ZTABLE_1
  DATA upd_ztable_1 TYPE if_chdo_object_tools_rel=>ty_cdchngindh.
  " table with the NEW content of: ZTABLE_2
  DATA xztable_2 TYPE zcl_ztest_1_chdo=>tt_ztable_2.
  " table with the OLD content of: ZTABLE_2
  DATA yztable_2 TYPE zcl_ztest_1_chdo=>tt_ztable_2.
  " change indicator for table: ZTABLE_2
  DATA upd_ztable_2 TYPE if_chdo_object_tools_rel=>ty_cdchngindh.
  " Change Number of Document
  DATA changenumber TYPE if_chdo_object_tools_rel=>ty_cdchangenr.
* save old values before change
* do some application changes to ZTABLE_1 and ZTABLE_2.
" set the change indicator for tables ('U', 'D', 'I')
" upd_ztable_1 = 'U'.
" upd_ztable_2 = 'U'.
" set change document application object identifier (objectid, date, time and
user)
" objectid                = 'Testobject'.
" utime                   = 'hhmmss'.
" udate                   = 'ddmmyyyy'.
" username                = 'Testusername'.
" set change indicator for change document header
  cdoc_upd_object = 'U'.
" prepare old and new tables
  SORT yztable_2.
  DELETE ADJACENT DUPLICATES FROM yztable_2.
  SORT xztable_2.
  DELETE ADJACENT DUPLICATES FROM xztable_2.
  " Begin of method call part
  " define needed DATA for error handling
   DATA err_ref TYPE REF TO cx_chdo_write_error.
   DATA err_action TYPE string.
    TRY.
        CALL METHOD zcl_ztest_1_chdo=>write
          EXPORTING
"  Begin of default method call part
            objectid = objectid
            utime = utime
            udate = udate
            username = username
            object_change_indicator = cdoc_upd_object
"  End of default method call part
"  Begin of dynamic part for method call
        "  declaration for the long text : ICDTXT_ZTEST_1
           icdtxt_ztest_1 = icdtxt_ztest_1
        "  update indicator for the long text
           upd_icdtxt_ztest_1 = upd_icdtxt_ztest_1
        " workaera_old of ZTABLE_1
          o_ztable_1 = os_ztable_1
```

```
        " workaera_new of ZTABLE_1
          n_ztable_1 = ns_ztable_1
        " change indicator for ZTABLE_1
          upd_ztable_1 = upd_ztable_1
        " table with the NEW content of: ZTABLE_2
          xztable_2 = xztable_2
        " table with the OLD content of: ZTABLE_2
          yztable_2 = yztable_2
        " change indicator for table: ZTABLE_2
          upd_ztable_2 = upd_ztable_2
" End of dynamic part for method call
      " Change Number of Document
        IMPORTING
          changenumber = changenumber.
     CATCH cx_chdo_write_error INTO err_ref.
         out->write( |Exception occurred: { err_ref->get_text( ) }| ).
    ENDTRY.
...
```

4.2.15.2.4 Deleting Change Documents

Use class `CL_CHDO_DELETE_TOOLS` to delete change documents.

First use `GET_INSTANCE` method to create an instance for the interface `IF_CHDO_DELETE_TOOLS`.

Parameter Name	Field Name	Value Help
RO_INSTANCE		Return parameter for instance creation

Method `IF_CHDO_DELETE_TOOLS~CHANGEDOCUMENT_DELETE` deletes the change documents for one change document object. You can restrict the selection by various parameters (such as date, change number or time).

Import Parameters

Parameter Name	Field Name	Value Help
I_OBJECTCLASS		Name of change document object
I_OBJECTID		Parameter of application objects IDs
I_UP_TO_DATE		Date by which the change documents are to be deleted All change documents until the specified date are deleted.
I_CHANGENUMBER		Change Number of Document If only one specific change document number is to be deleted, it can be transferred here.

Parameter Name	Field Name	Value Help
I_WITH_COMMIT		Flag whether there should be a `COMMIT WORK` in the method. The flag determines whether a `COMMIT WORK` should be called in the method. This can be necessary when a lot is to be deleted. In this case the delete operations are divided up to avoid Roll-back segment overflow. The division is by clusters, so that no inconsistencies occur in the case of cancellation.
COMMIT_COUNTER		The parameter is only effective when a `COMMIT WORK` should be performed in the method (see parameter `I_WITH_COMMIT`). In the method, as many object change documents are read as are passed in `COMMIT_COUNTER`. These change documents are then deleted, and `COMMIT WORK` called. This procedure is repeated until there are no more change documents for the object before the until date.

Export Parameter

Parameter Name	Field Name	Value Help
E_NUMBER_OF_DELETED_HEADERS		Number of deleted change document headers This parameter contains the number of deleted change document headers
E_NUMBER_OF_DELETED_POSITIONS		Number of deleted change document items This parameter contains the number of deleted change document items.
E_NUMBER_OF_DELETED_UIDS		Number of deleted change document item GUIDs
E_NUMBER_OF_DELETED_STRINGS		Number of deleted change document item STRINGs This parameter contains the number of deleted change document items with STRINGs.

Sample Code

```
DATA lv_objectclass   TYPE if_chdo_object_tools_rel=>ty_cdobjectcl VALUE
'ZTEST_CO'.
    DATA lv_objectid  TYPE cl_chdo_write_tools=>ty_cdobjectv VALUE '9990001'.
    DATA lv_changenr  TYPE cl_chdo_write_tools=>ty_cdchangenr VALUE
'0000000001'.
    DATA lr_err TYPE REF TO cx_chdo_delete_error.
          ).
    TRY.
      DATA(lo_instance) = cl_chdo_delete_tools=>get_instance( ).
      lo_instance->if_chdo_delete_tools~changedocument_delete(
        EXPORTING
          i_objectclass                = lv_objectclass
          it_objectid                  = lv_objectid
          i_changenumber               = lv_changenr
          i_with_commit                = abap_true
        IMPORTING
          e_number_of_deleted_headers   = DATA(ls_headers)
          e_number_of_deleted_positions = DATA(ls_positions)
          e_number_of_deleted_uids      = DATA(ls_uids)
          e_number_of_deleted_strings   = DATA(ls_strings)
      ).
      CATCH cx_chdo_delete_error INTO lr_err.
*    error handling for deletion
    ENDTRY.
```

4.2.15.2.5 Reading Change Documents

Use class `CL_CHDO_READ_TOOLS` to read change documents.

Method `CHANGEDOCUMENT_READ` reads the change documents for one change document object. You can restrict the search by various parameters (such as changed by, date, or time).

Import Parameters

Parameter Name	Field Name	Value Help
ET_CDREDADD_TAB		Table type for structure `CDREDADD`, change documents return table
I_OBJECTCLAS		Name of change document object
IT_OBJECTID		Range table of application object IDs
I_DATE_OF_CHANGE		From-change date for search. All change documents are selected written on the specified date or later are found.
I_TIME_OF_CHANGE		From-change time for search. If no change time is specified, a selection is made from the time '000000'.

Parameter Name	Field Name	Value Help
I_DATE_UNTIL		Change date up to which you want to search. All change documents are selected written up to and including this date.
I_TIME_UNTIL		Latest change time in search. Time to which change documents are read on the "To" change date. If no time is passed, all change documents on the "To" change date are read.
IT_USERNAME		Username of the person responsible in change document. Only those change documents are selected that document changes made by this user. If no user name is passed, change documents are read for all users.
IS_READ_OPTIONS		
	local_time	If it is set, the date and time information in the formatted change documents is displayed in the local time of the user.
	time_zone	It contains the time zone in which the change documents were written. If it's not set UTC applies. If it contains a time zone, this zone is used as the time zone in which the change documents were saved. If the change documents were saved in CET, the parameter must be set to CET.

Parameter Name	Field Name	Value Help
	it_changenr	Range table for change document number. Change document numbers were created internally as part of key of change documents. The key of change document is represented by Object name, Object ID of the application object and a change number. During creation of change documents using the write method of class `<name space>CL_<change document object name>_CHDO` Change documents numbers were received by export paramter `CHANGENUMBER`.
	only_headers	Only return the change document header information without the position
IV_READ_ARCHIVE		Control the interface to read change documents from the archive, too

Export Parameter

Parameter Name	Field Name	Value Help
	OBJECTCLAS	Name of Change Document Object
	OBJECTID	Object ID of application object
	OBJECTID_DB	Object value
	CHANGENR	Change Number of Document
	OBJECTTXT	Object Description
	USERNAME	Username of the person responsible in change document
	USERNAME_DB	Username of the person responsible in change document
	UDATE	Creation date of the change document
	UDATE_DB	Creation date of the change document
	UTIME	Time changed
	UTIME_DB	Time changed

Parameter Name	Field Name	Value Help
	TCODE	Transaction in which a change was made
	APPLNAME	Application Object
	APPLTYPE	Application Type
	TABNAME	Change document creation: Table name
	TABNAME_DB	Change document creation: Table name
	TABKEY	Key of Changed Table Line
	TABKEY_DB	Key of Changed Table Line
	KEYLEN	Table key length
	CHNGIND	Type of Change
	FNAME	Field Name
	FNAME_DB	Field Name
	FTEXT	Explanatory Short Text
	TEXTART	Create change document: Text type
	SPRACHE	Language Key
	TEXT_CASE	Text change flag ('X')
	OUTLEN	Output length of the old and new value
	F_OLD	Old contents of changed field
	F_NEW	New contents of changed field
	F_NEW_DB	New contents of changed field
	VALUE_OLD	Old Extended Value (Long)
	VALUE_OLD_DB	Old Extended Value (Long)
	VALUE_NEW	New Extended Value (Long)
	VALUE_NEW_DB	New Extended Value (Long)
	VALUE_RAWSTR_OLD	Old Change Document Value for RAW-STRING Variable

Parameter Name	Field Name	Value Help
	VALUE_RAWSTR_OLD_DB	Old Change Document Value for RAW-STRING Variable
	VALUE_RAWSTR_NEW	New Change Document Value for RAW-STRING Variable
	VALUE_RAWSTR_NEW_DB	New Change Document Value for RAW-STRING Variable
	VALUE_SHSTR_OLD	Old Extended Value (Short)
	VALUE_SHSTR_OLD_DB	Old Extended Value (Short)
	VALUE_SHSTR_NEW	New Extended Value (Short)
	VALUE_SHSTR_NEW_DB	New Extended Value (Short)
	KEYGUID	KEYGUID for Link to CDPOS_UID
	TABKEY254	Key of Modified Table Row
	TABKEY254_DB	Key of Modified Table Row
	EXT_KEYLEN	Table key length
	KEYGUID_STR	KEYGUID for Link to CDPOS_STR
	VERSION	3-Byte field

Sample Code

Read all change documents for object class `ZCHDO_TEST`

```
CLASS zcl_chdo_read DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
   INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_chdo_read IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    DATA: rt_cdredadd TYPE cl_chdo_read_tools=>TT_CDREDADD_TAB,
          lr_err      TYPE REF TO cx_chdo_read_error.
    TRY.
      cl_chdo_read_tools=>changedocument_read(
        EXPORTING
          i_objectclass    = 'ZCHDO_TEST'  " change document object name
*         it_objectid      =
*         i_date_of_change =
*         i_time_of_change =
*         i_date_until     =
*         i_time_until     =
*         it_username      =
```

```
*          it_read_options  =
        IMPORTING
          et_cdredadd_tab  = rt_cdredadd    " result returned in table
      ).
       CATCH cx_chdo_read_error into lr_err.
        out->write( |Exception occurred: { lr_err->get_text( ) }| ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

Sample Code

Read all change documents for object class `ZCHDO_TEST` including the archive.

```
CLASS zcl_chdo_read DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_chdo_read IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    DATA: rt_cdredadd TYPE cl_chdo_read_tools=>tt_cdredadd_tab,
          lr_err      TYPE REF TO cx_chdo_read_error.
    DATA lt_cdredadd TYPE cl_chdo_read_tools=>tt_cdredadd_tab.
    TRY.
        DATA(lt_files) =
cl_arch_read_api=>get_files_to_read( iv_archiving_object = 'ZAOBJ_TEST' ).
        SORT lt_files BY creation_date DESCENDING creation_time DESCENDING.
        READ TABLE lt_files ASSIGNING FIELD-SYMBOL(<ls_file>) INDEX 1.
        CHECK sy-subrc = 0. " no files
        DATA(lo_read) = cl_arch_read_api=>get_instance( iv_archiving_object =
'ZAOBJ_TEST'
                                                        iv_archive_key =
<ls_file>-archive_key ).
        DO.
          lo_read->get_next_data_object( IMPORTING ev_end_of_file =
DATA(lv_end_of_file)
                                                 ev_archive_key =
DATA(lv_archive_key)
                                                 ev_object_offset =
DATA(lv_offset) ).
          IF lv_end_of_file = abap_true.
            EXIT.
          ENDIF.
*          no application data
*          lo_read->get_data_records( EXPORTING iv_record_structure = 'XXX'
*                                     IMPORTING et_data_records = lt_data ).
*          APPEND LINES OF lt_data TO lt_data_all.
          TRY.
              cl_chdo_read_tools=>changedocument_read(
                EXPORTING
                  i_objectclass    = 'ZCHDO_TEST'
                  iv_read_archive  = lo_read
                IMPORTING
                  et_cdredadd_tab  = DATA(lt_cd)
              ).
            CATCH cx_chdo_read_error INTO DATA(ls_read_err).
              out->write( |Exception occurred: { ls_read_err-
>get_text( ) }| ).
          ENDTRY.
          APPEND LINES OF lt_cd TO lt_cdredadd.
        ENDDO.
```

```
        lo_read->close( ).
      CATCH cx_arch_api INTO DATA(lx_error).
        out->write( |Exception occurred: { lx_error->get_text( ) }| ).
    ENDTRY.
  ENDMETHOD.
```

4.2.15.2.6 Archiving Change Documents

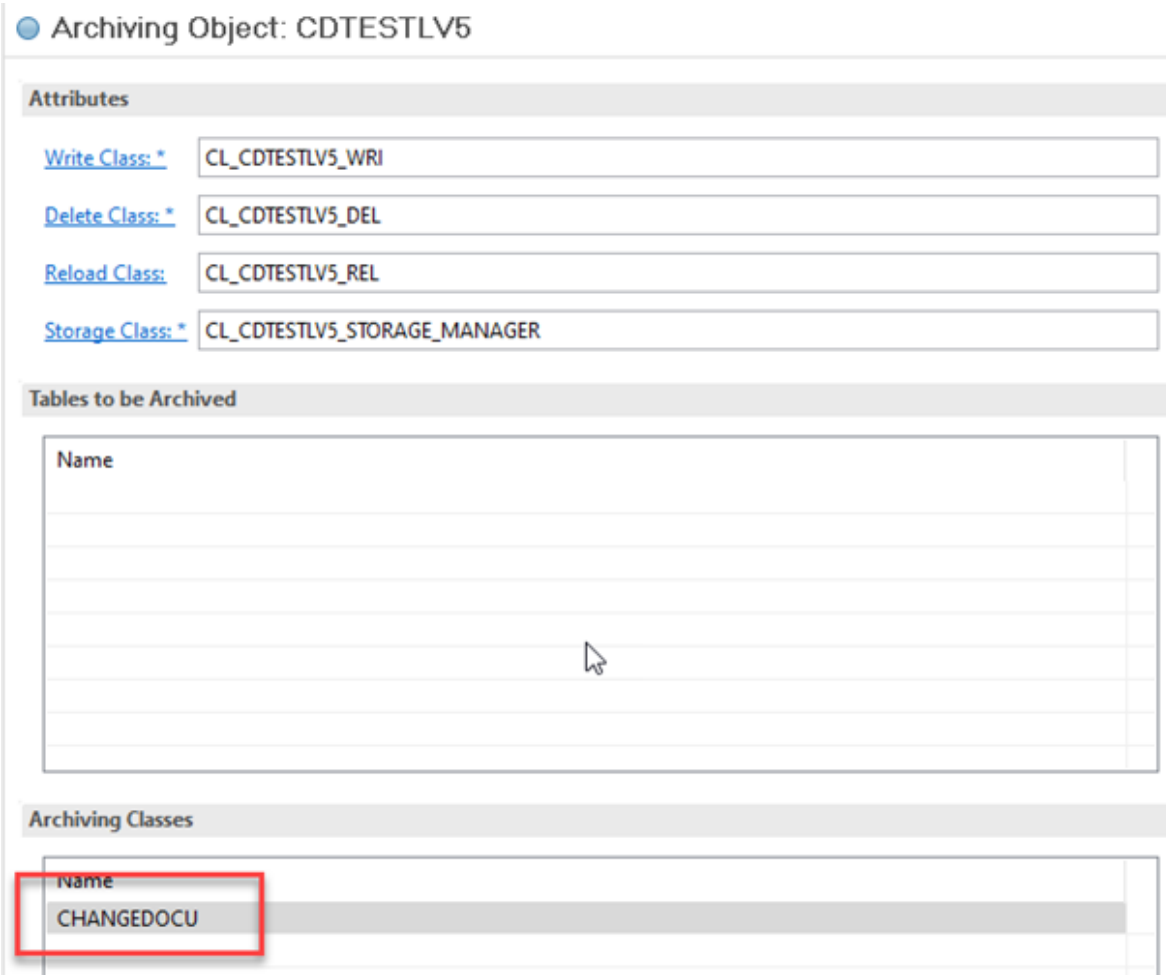
You can archive change documents together with their application documents.

Prerequisites

1. A change document object is available (see Working with Change Document Objects) and it's implemented to create change documents (see Writing Change Documents [page 1390]).
2. There is an archiving object to archive current application documents (see Working with Archiving Objects.

Procedure

1. Add the archiving class `CHANGEDOCU` to the archiving classes of your archiving object using ABAP `Development Tools` (ADT):



2. Update the write class of your archiving object to archive change documents.

Sample Code

```
CLASS ZCL_<YOUR_ARCH_OBJECT>_WRI IMPLEMENTATION.
METHOD if_apj_rt_exec_object~execute.
…
TRY.
DATA(lo_write) = cl_arch_write_api=>get_instance( iv_archiving_object =
<YOUR_ARCH_OBJECT> iv_testmode = lv_test ).
LOOP AT lt_keys_tab ASSIGNING FIELD-SYMBOL(<fs_keys>).
lo_write->open_data_object( ).
cl_changedocu_archive=>get_instance( )-
>if_changedocu_archive~changedocu_archive_object( iv_objectclass
= <YOUR_CD_OBJECTCLASS>
iv_objectid = <fs_keys>-objectid
iv_write_instance = lo_write ).
lo_write->close_data_object( ).
ENDLOOP.
lo_write->finalize( ).
CATCH cx_arch_api INTO DATA(lx_error).
CATCH cx_chdo_archive_error INTO DATA(lx_error2).
ENDTRY.
ENDMETHOD.
```

> **Note**
>
> String <YOUR_ARCH_OBJECT> is for your archiving object. String <YOUR_CD_OBJECTCLASS> is for your change document object.

> **Note**
>
> To read change documents from the archive, consider Reading Change Documents [page 1397].

4.2.15.2.7 How to Use the Fiori Reuse Library for Change Document Solution

You can use the *Reuse Library* to implement a reusable component that can display change documents.

Context

To be able to use the reusable components, you need to adapt the code that can be found in your `manifest.json` file.

> **Restriction**
>
> Mind that the Fiori reuse library is only released for the OData V2 protocol.

Procedure

1. Adapt your <`manifest.json`> file and add the `sap.nw.core.changedocs.lib.reuse` library under the `sap.ui5` section:

> **Sample Code**

```
"sap.ui5": {
        "resources": {
            "js": [],
            "css": []
        },
        "componentUsages": {
            "ChangedocReuseComponent": {
                "name":
"sap.nw.core.changedocs.lib.reuse.changedocscomponent"
            }
        },
        "dependencies": {
            "minUI5Version": "1.65.0",
            "libs": {
                "sap.ui.core": {
                    "lazy": false
```

```
                },
                "sap.ui.generic.app": {
                    "lazy": false
                },
                "sap.suite.ui.generic.template": {
                    "lazy": false
                },
                "sap.ui.comp": {
                    "lazy": false
                }
            },
            "components": {
                "sap.nw.core.changedocs.lib.reuse": {
                    "lazy": true

                }
            }
        },
```

2. Adapt your <manifest.json> file and add the component as section on your page:

Sample Code

```
"pages": {
    "ObjectPage|TravelProcessor": {
        "entitySet": "TravelProcessor",
        "defaultLayoutTypeIfExternalNavigation": "MidColumnFullScreen",
        "component": {
            "name": "sap.suite.ui.generic.template.ObjectPage"
        },
        "embeddedComponents": {
            "changedoccomponent": {
                "id": "changedoccomponent",
                "componentUsage": "ChangedocReuseComponent",
                "title": "{{reuseTitle}}",
                "settings": {
                    "objectClass": [
                        "<MY_CHANGE_DOC>"
                                    ],
                    "objectId": "{parts:
[{path: '<MY_OBJECTID>'}], formatter:
'sap.nw.core.changedocs.lib.reuse.changedocscomponent.arrayFormatter'}",
                    "startDate": "1900-01-01T00:00:00"
                }
            }
        }
    }
}
```

Note

Make sure to replace strings <MY_CHANGE_DOC> with your own change document object and <MY_OBJECTID> with your own Object id which were also used to create change documents.

3. Adapt your <i18n.properties> file and add:

Sample Code

```
…
#XTIT: Reuse component title
```

```
reuseTitle =Change Documents
…
```

> **Note**
>
> Manual tests of the reuse library using *SAP Business Application Studio* is not possible. Create and test your app finally before implementing the reuse app. Test the reuse app after deployment.
>
> After your app was deployed successfully to an SAP Business Technology Platform system, the updated BSP application and the SAP Fiori Launchpad app descriptor item will appear under your created package in Eclipse.

4. Update your IAM App. For more information, see Defining an IAM App for the Business Service [page 1008].
5. Once created, maintain the *Change Document OData Service*. Go to the *Service* tab and add the *Change Document Log OData Service* by naming the *OData V2* service type and add the following service name: `APS_CHANGE_DOCUMENTS_SRV 0001` (**Note: make sure to add all 11 spaces in between 'SRV' and '0001'**).
6. Create a new Business Catalog. For more information, see Creating a Business Catalog [page 1009]

   Having created the business catalog, you have successfully implemented a reusable component to display your change documents.

4.2.15.2.8 How to Use the Fiori Reuse Library

You can use the Reuse Library to implement a reusable component that can display change documents.

For more information on how to use the Fiori reuse library in a Fiori elements app, see Fiori Elements Integration OData V2 [page 1407](for OData V2 apps) and Fiori Elements Integration OData V4 [page 1409](for OData V4 apps).

4.2.15.2.8.1 Fiori Elements Integration OData V2

You can implement the reusable component from the Reuse Library in a Fiori Elements app in order to display change documents.

Context

To be able to use the reusable components, you need to adapt the code that can be found in your manifest.json file.

Procedure

1. Adapt your <manifest.json> file and add the sap.nw.core.changedocs.lib.reuse library under the sap.ui5 section:

   Sample Code

   ```
   sap.ui5": {
       "dependencies": {
         "libs": {
           " sap.nw.core.changedocs.lib.reuse": {
           "lazy": true
           }
       },
         "components": {}
       }
   ```

2. Adapt your <manifest.json> file and add the component as section on your page:

   Sample Code

   ```
   "pages": {
       "ObjectPage|TravelProcessor": {
           "entitySet": "TravelProcessor",
           "defaultLayoutTypeIfExternalNavigation": "MidColumnFullScreen",
           "component": {
               "name": "sap.suite.ui.generic.template.ObjectPage"
           },
           "embeddedComponents": {
               "changedoccomponent": {
                   "id": "changedoccomponent",
                   "componentUsage": "ChangedocReuseComponent",
                   "title": "{{reuseTitle}}",
                   "settings": {
                       "objectClass": [
                           "<MY_CHANGE_DOC>"
                                       ],
                       "objectId": "{parts:
   [{path: '<MY_OBJECTID>'}], formatter:
   'sap.nw.core.changedocs.lib.reuse.changedocscomponent.arrayFormatter'}",
                       "startDate": "1900-01-01T00:00:00"
                   }
               }
           }
       }
   }
   ```

   Note

   Make sure to replace strings <MY_CHANGE_DOC> with your own change document object and <MY_OBJECTID> with your own Object id which were also used to create change documents.

3. Adapt your <i18n.properties> file and add:

   Sample Code

   ```
   …
   #XTIT: Reuse component title
   reuseTitle =Change Documents
   …
   ```

> ### Note
>
> Manual tests of the reuse library using **SAP Business Application Studio** is not possible. Create and test your app finally before implementing the reuse app. Test the reuse app after deployment.
>
> After your app was deployed successfully to an SAP Business Technology Platform system, the updated BSP application and the SAP Fiori Launchpad app descriptor item will appear under your created package in Eclipse.

4. Update your IAM App. For more information, see Defining an IAM App for the Business Service.
5. Once created, maintain the **Change Document OData Service**. Go to the **Service** tab and add the **Change Document Log OData Service** by naming the **OData V2** service type and add the following service name: `APS_CHANGE_DOCUMENTS_SRV 0001`

   > ### Note
   >
   > make sure to add all 11 spaces in between 'SRV' and '0001'

6. Create a new Business Catalog. For more information, see Creating a Business Catalog
   Having created the business catalog, you have successfully implemented a reusable component to display your change documents.

4.2.15.2.8.2 Fiori Elements Integration OData V4

You can implement the reusable component from the **Reuse Library** in a Fiori Elements app in order to display an application log.

Context

To implement the reusable component in a Fiori Elements app, you first have to add the application log reuse component to your manifest.

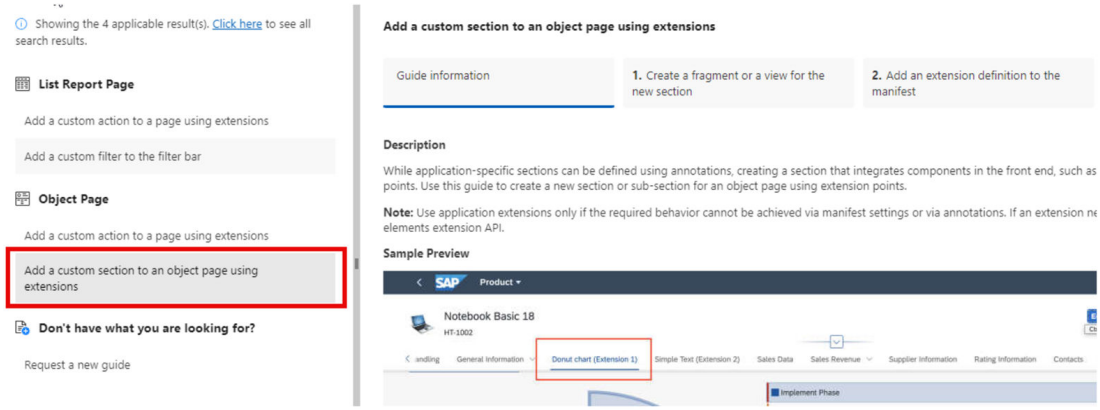
Procedure

1. Open the `manifest.json` file of the application and locate the `sap.ui5` section.
2. Add the `sap.nw.core.changedocs.lib.reuse` library under dependencies:

> ### Sample Code
>
```
"sap.ui5": {
    "dependencies": {
      "libs": {
        " sap.nw.core.changedocs.lib.reuse": {
        "lazy": true
```

```
        }
      }
    }
  }
```

3. Next, **create a view extension** by adding a custom section to your object page using the **Guided Development** tool or by manually adjusting the `manifest.json file`:
   - Right-click on your projects root folder and click **Open Guided Development**
   - Choose the guide **Add a custom section to an object page using extensions**



   - In your `manifest.json file`, you should now be able to see something similar to the following:

Sample Code

```
      "targets": {
        "ZC_BV_SZD_BOOKList": {
          "type": "Component",
          "id": "ZC_BV_SZD_BOOKList",
          "name": "sap.fe.templates.ListReport",
          "options": {
            "settings": {
              "contextPath": "/ZC_BV_SZD_BOOK",
              "variantManagement": "Page",
              "navigation": {
                "ZC_BV_SZD_BOOK": {
                  "detail": {
                    "route": "ZC_BV_SZD_BOOKObjectPage"
                  }
                }
              },
              "controlConfiguration": {
                "@com.sap.vocabularies.UI.v1.LineItem": {
                  "tableSettings": {
                    "type": "ResponsiveTable"
                  }
                }
              }
            }
          }
        },
        "ZC_BV_SZD_BOOKObjectPage": {
          "type": "Component",
          "id": "ZC_BV_SZD_BOOKObjectPage",
          "name": "sap.fe.templates.ObjectPage",
          "options": {
            "settings": {
              "editableHeaderContent": false,
              "contextPath": "/ZC_BV_SZD_BOOK",
```

```
              "content": {
                "body": {
                  "sections": {
                    "myCustomSection": {
                      "template":
"chdbookproject.custom.fragment.chdFragmentExtension",
                      "title": "chdBookExt",
                      "position": {
                        "placement": "After",
                        "anchor": "idIdentification"
                      }
                    }
                  }
                }
              }
            }
          }
        }
      }
```

4. Open the fragment that was created from the **Guided Development** tool and add a component container:

Sample Code

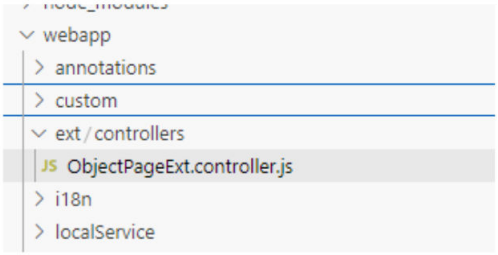
```
<core:FragmentDefinition xmlns:core="sap.ui.core" xmlns="sap.m" >
    <core:ComponentContainer id="ChdContainer"/>
</core:FragmentDefinition>
```

5. As a next step, **create a controller extension**. First, open the `manifest.json file` of the application and locate the `sap.ui5` section. Then, add an object page controller extension:

Sample Code

```
"extends": {
      "extensions": {
        "sap.ui.controllerExtensions": {
          "sap.fe.templates.ObjectPage.ObjectPageController": {
            "controllerName":
"chdbookproject.ext.controller.ChdControllerExt"
          }
        }
      }
    }
```

6. Create the controller extension file with the specified name and path from the `manifest.json` file:



7. Adjust the controller extension coding by opening the `ObjectPageExt.controller.js` file you've created and adjust the coding like this:

Note

Make sure to adjust the names and paths accordingly.

Sample Code

```
sap.ui.define(
  ['sap/ui/core/mvc/ControllerExtension', 'sap/ui/core/Core'],
  function (ControllerExtension, Core) {
    'use strict';
    return ControllerExtension.extend(
    'chdbookproject.ext.controller.ChdControllerExt', {
        _chdContainer: "chdbookproject::ZC_BV_SZD_BOOKObjectPage--
fe::CustomSubSection::myCustomSection--ChdContainer",
        // this section allows to extend lifecycle hooks or hooks provided
by Fiori elements
        override: {
      /**
       * Called when a controller is instantiated and its View controls
       * (if available) are already created.
       * Can be used to modify the View before it is displayed, to bind
       * event handlers and do other one-time initialization.
       * @memberOf chdbookproject.ext.controller.ChdControllerExt
       */
      onInit: function () {
            var that = this;
            // you can access the Fiori elements extensionAPI via
this.base.getExtensionAPI
            var oViewId = this.getView().getId();
            if (oViewId === "chdbookproject::ZC_BV_SZD_BOOKObjectPage") {
        this._oComp = Core.createComponent({
                name:
"sap.nw.core.changedocs.lib.reuse.changedocscomponent",
                id : "ChangeDocReuseComponent",
                settings: {
          "objectClass": ["ZBVSZDBOOK"],
          "startDate": "2019-01-28T00:00:00.0000",
          "stIsAreaVisible": true
                }
        });
         var oChdContainer = Core.byId(this._chdContainer);
         if (oChdContainer !== undefined) {
           oChdContainer.setComponent(this._oComp);
         }
         this._oComp.setStIsAreaVisible(true);
         this._oComp.stRefresh();
            }
      },
          editFlow: {
            onAfterSave: function () {
              var that = this;
              this._oComp.stRefresh();
            }
          },
          routing: {
            onAfterBinding: function (oBindingContext) {
              var that = this;
              oBindingContext.requestProperty("Idnr").then(function
(sObject) {
                if (sObject) {
                  that._oComp.setObjectId([sObject]);
                }
                that._oComp.stRefresh();
              });
            }
          },

    }
      }
    )
  });
```

8. To **refresh the logs**, you can use the edit flow hooks offered by the controller extension. See also UI5 Demo Kit: Edit Flow Overview. An example:

Sample Code

```
editFlow: {
                onAfterSave: function () {
                    var that = this;
                     this._oComp.stRefresh();
                }
            },
```

4.2.15.3 Currency Conversion

The ABAP Core Data Services (CDS) provide the built-in function `CURRENCY_CONVERSION` to convert different currencies to a common target currency.

For more information, see CDS DDL - CDS View Entity, Unit and Currency Conversion Functions in the ABAP Keyword Documentation.

To customize this conversion function, you can use API class `CL_EXCHANGE_RATES` to load the current exchange rates into your system for a correct conversion. Class `CL_EXCHANGE_RATES` is based on the business API function modules `BAPI_EXCHRATE_CREATEMULTIPLE` and `BAPI_EXCHANGERATE_SAVEREPLICA`.

API class `CL_EXCHANGE_RATES` provides PUT methods to write exchange rates to the corresponding customizing persistence. For more information, see the ABAP Doc comments in the class implementation.

Note that aside from use in CDS, the class additionally provides two methods to convert currencies.

Example

Sample Code

```
 @AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Currency Conversion Example'
@Metadata.ignorePropagatedAnnotations: true
define view entity Z_CURR_CONV_EXAMPLE
as select from /DMO/I_Supplement
{
  key SupplementID,
      @Semantics.amount.currencyCode: 'CurrencyCode'
      Price,
      CurrencyCode,
      @Semantics.amount.currencyCode: 'CurrencyUSD'
      currency_conversion(
                           amount => Price,
                           round => '',
                           source_currency => CurrencyCode,
                           target_currency => cast('USD' as abap.cuky),
                           exchange_rate_date => cast($session.system_date as
abap.dats)
```

```
                            ) as PriceInUSD,
      cast('USD' as abap.cuky) as CurrencyUSD
}
```

> **Note**
>
> You can find a more detailed working example in GitHub at https://github.com/SAP-samples/cloud-abap-exchange-rates to learn how to obtain and store exchange rates from an official source.

4.2.15.4 Conversion of SAP Codes

Learn how to use the methods of class `CL_CONVERSION_EXT_INT` to convert SAP codes to external codes and vice versa.

Context

Communication in heterogeneous IT landscapes uses external data formats (ISO). SAP systems use a different internal format for data such as currencies, languages, and units of measurement. Therefore, data must be converted into the appropriate format during communication.

For more information, see the following sections.

Currency Code Conversion [page 1415]
: Learn how to use the methods `CURRENCY_CODE_EXT_TO_INT` and `CURRENCY_CODE_INT_TO_EXT` to convert SAP currency codes.

Currency Amount Conversion [page 1417]
: Learn how to use the methods `CURRENCY_AMOUNT_EXT_TO_INT` and `CURRENCY_AMOUNT_INT_TO_EXT` to convert SAP currency amounts.

Language Code Conversion [page 1419]
: Learn how to use the methods `LANGUAGE_CODE_EXT_TO_INT` and `LANGUAGE_CODE_INT_TO_EXT` to convert SAP language codes.

Conversion of Units of Measurement [page 1422]
: Learn how to use the methods `UNIT_OF_MEASURE_EXT_TO_INT` and `UNIT_OF_MEASURE_INT_TO_EXT` to convert SAP units of measurement.

4.2.15.4.1 Currency Code Conversion

Learn how to use the methods `CURRENCY_CODE_EXT_TO_INT` and `CURRENCY_CODE_INT_TO_EXT` to convert SAP currency codes.

CURRENCY_CODE_EXT_TO_INT

Use method `CURRENCY_CODE_EXT_TO_INT` to convert an external currency into an SAP currency.

Import Parameters

Parameter Name	Value Help
`ISO_CODE`	ISO currency code

Export Parameters

Parameter Name	Value Help
`SAP_CODE`	SAP currency code
`IS_UNIQUE`	`ABAP_FALSE` if an ISO code is assigned to any SAP codes

> **Note**
>
> Class exception `CX_CONVERSION_EXT_INT` is raised to check the integrity of the data import parameters.

> **Sample Code**

```
CLASS zcl_conversion_ext_int_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC.
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_conversion_ext_int_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    TRY.
        DATA(lo_conv) = cl_conversion_ext_int=>get_instance( ).
      CATCH cx_conversion_ext_int.
    ENDTRY.
    "Conversion of currency codes ISO to SAP
    TRY.
        lo_conv->currency_code_ext_to_int(
          EXPORTING
            iso_code = 'USD'
          IMPORTING
            sap_code = DATA(lv_sap_code)
            is_unique = DATA(lv_unique)
        ).
        out->write( |Result: { lv_sap_code } is unique { lv_unique }| ).
```

```
      CATCH cx_conversion_ext_int INTO DATA(lo_error).
        out->write( lo_error->get_text( ) ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

CURRENCY_CODE_INT_TO_EXT

Use method `CURRENCY_CODE_INT_TO_EXT` to convert an SAP currency into an external currency.

Import Parameters

Parameter Name	Value Help
SAP_CODE	SAP currency code

Export Parameters

Parameter Name	Value Help
ISO_CODE	ISO currency code
IS_UNIQUE	ABAP_FALSE if an ISO code is assigned to any SAP codes

> **Note**
>
> Class exception `CX_CONVERSION_EXT_INT` is raised to check the integrity of the data import parameters.

Sample Code

```
CLASS zcl_conversion_ext_int_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_conversion_ext_int_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    TRY.
        DATA(lo_conv) = cl_conversion_ext_int=>get_instance( ).
      CATCH cx_conversion_ext_int.
    ENDTRY.
    "Conversion of currency codes SAP to ISO
    TRY.
        lo_conv->currency_code_int_to_ext(
          EXPORTING
            sap_code = 'USD'
          IMPORTING
            iso_code = DATA(lv_iso_code)
        ).
        out->write( |Result: { lv_iso_code }| ).
      CATCH cx_conversion_ext_int INTO DATA(lo_error).
```

```
        out->write( lo_error->get_text( ) ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.4.2 Currency Amount Conversion

Learn how to use the methods `CURRENCY_AMOUNT_EXT_TO_INT` and `CURRENCY_AMOUNT_INT_TO_EXT` to convert SAP currency amounts.

Context

In an SAP system, a currency amount field only makes sense in connection with a currency because this is the only way to correctly set the decimal point in the amounts. Therefore, each currency amount field must be assigned to a currency field. For example, two Yen are stored as 0.02 in the database in a field of data type `CURR`.

All SAP currency data types have two decimal places, despite some currencies having three decimal places.

CURRENCY_AMOUNT_EXT_TO_INT

Use method `CURRENCY_AMOUNT_EXT_TO_INT` to convert an external currency amount into an SAP currency amount.

Import Parameters

Parameter Name	Value Help
`CURRENCY`	ISO currency code
`BAPI_AMOUNT`	External currency amount

Export Parameters

Parameter Name	Value Help
`SAP_AMOUNT`	SAP currency amount

> **Note**
>
> Class exception `CX_CONVERSION_EXT_INT` is raised to check the integrity of the data import parameters.

Sample Code

```
CLASS zcl_conversion_ext_int_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_conversion_ext_int_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    TRY.
        DATA(lo_conv) = cl_conversion_ext_int=>get_instance( ).
      CATCH cx_conversion_ext_int.
    ENDTRY.
    "Conversion of currency amounts ISO to SAP
    DATA lv_amount TYPE bapicurr_d.
    lv_amount = CONV #( '20' ).
    TRY.
        lo_conv->currency_amount_ext_to_int(
          EXPORTING
            currency = 'JPY'
            bapi_amount = lv_amount
          IMPORTING
            sap_amount = DATA(lv_sap_amount)
        ).
        out->write( |Result: { lv_sap_amount } | ).
      CATCH cx_conversion_ext_int INTO DATA(lo_error).
        out->write( lo_error->get_text( ) ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

CURRENCY_AMOUNT_INT_TO_EXT

Use method `CURRENCY_AMOUNT_INT_TO_EXT` to convert an SAP currency amount into an external currency amount.

Import Parameters

Parameter Name	Value Help
CURRENCY	ISO currency code
SAP_AMOUNT	SAP currency amount

Export Parameters

Parameter Name	Value Help
BAPI_AMOUNT	External currency amount

Note

Class exception `CX_CONVERSION_EXT_INT` is raised to check the integrity of the data import parameters.

Sample Code

```
CLASS zcl_conversion_ext_int_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_conversion_ext_int_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    TRY.
        DATA(lo_conv) = cl_conversion_ext_int=>get_instance( ).
      CATCH cx_conversion_ext_int.
    ENDTRY.
    "Conversion of currency amounts SAP to ISO
    DATA lv_amount TYPE bapicurr_d.
    lv_amount = CONV #( '0.2000' ).
    TRY.
        lo_conv->currency_amount_int_to_ext(
          EXPORTING
            currency = 'JPY'
            sap_amount = lv_amount
          IMPORTING
            bapi_amount = DATA(lv_iso_amount)
        ).
        out->write( |Result: { lv_iso_amount } | ).
      CATCH cx_conversion_ext_int INTO DATA(lo_error).
        out->write( lo_error->get_text( ) ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.4.3 Language Code Conversion

Learn how to use the methods `LANGUAGE_CODE_EXT_TO_INT` and `LANGUAGE_CODE_INT_TO_EXT` to convert SAP language codes.

LANGUAGE_CODE_EXT_TO_INT

Use method `LANGUAGE_CODE_EXT_TO_INT` to convert an external language into an SAP language.

Import Parameters

Parameter Name	Value Help
ISO_CODE	ISO language code

Export Parameters

Parameter Name	Value Help
SAP_CODE	SAP language code

> **Note**
>
> Class exception CX_CONVERSION_EXT_INT is raised to check the integrity of the data import parameters.

Sample Code

```
CLASS zcl_conversion_ext_int_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_conversion_ext_int_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    TRY.
        DATA(lo_conv) = cl_conversion_ext_int=>get_instance( ).
      CATCH cx_conversion_ext_int.
    ENDTRY.
    "Conversion of languages codes ISO to SAP
    TRY.
        lo_conv->language_code_ext_to_int(
          EXPORTING
            iso_code = 'EN'
          IMPORTING
            sap_code = DATA(lv_sap_code)
        ).
        out->write( |Result: { lv_sap_code } | ).
      CATCH cx_conversion_ext_int INTO DATA(lo_error).
        out->write( lo_error->get_text( ) ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

LANGUAGE_CODE_INT_TO_EXT

Use method LANGUAGE_CODE_INT_TO_EXT to convert an SAP language into an external language.

Import Parameters

Parameter Name	Value Help
SAP_CODE	SAP language code

Export Parameters

Parameter Name	Value Help
ISO_CODE	ISO language code

> **Note**
>
> Class exception `CX_CONVERSION_EXT_INT` is raised to check the integrity of the data import parameters.

Sample Code

```
CLASS zcl_conversion_ext_int_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_conversion_ext_int_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    TRY.
        DATA(lo_conv) = cl_conversion_ext_int=>get_instance( ).
      CATCH cx_conversion_ext_int.
    ENDTRY.
    "Conversion of languages codes SAP to ISO
    TRY.
        lo_conv->language_code_int_to_ext(
          EXPORTING
            sap_code = 'E'
          IMPORTING
            iso_code = DATA(lv_iso_code)
        ).
        out->write( |Result: { lv_iso_code } | ).
      CATCH cx_conversion_ext_int INTO DATA(lo_error).
        out->write( lo_error->get_text( ) ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.4.4 Conversion of Units of Measurement

Learn how to use the methods `UNIT_OF_MEASURE_EXT_TO_INT` and `UNIT_OF_MEASURE_INT_TO_EXT` to convert SAP units of measurement.

UNIT_OF_MEASURE_EXT_TO_INT

Use method `UNIT_OF_MEASURE_EXT_TO_INT` to convert an external unit of measurement into an SAP unit of measurement.

Import Parameters

Parameter Name	Value Help
`ISO_CODE`	ISO code unit of measurement

Export Parameters

Parameter Name	Value Help
`SAP_CODE`	SAP code unit of measurement
`UNIQUE`	`ABAP_FALSE` if an ISO code is assigned to any SAP codes

> **Note**
>
> Class exception `CX_CONVERSION_EXT_INT` is raised to check the integrity of the data import parameters.

> **Sample Code**

```
CLASS zcl_conversion_ext_int_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_conversion_ext_int_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    TRY.
        DATA(lo_conv) = cl_conversion_ext_int=>get_instance( ).
      CATCH cx_conversion_ext_int.
    ENDTRY.
    "Conversion of units of measurements ISO to SAP
    TRY.
        lo_conv->unit_of_measure_ext_to_int(
          EXPORTING
            iso_code = 'MTR'
          IMPORTING
            sap_code = DATA(lv_sap_code)
        ).
        out->write( |Result: { lv_sap_code } | ).
```

```
      CATCH cx_conversion_ext_int INTO DATA(lo_error).
        out->write( lo_error->get_text( ) ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

UNIT_OF_MEASURE_INT_TO_EXT

Use method `UNIT_OF_MEASURE_INT_TO_EXT` to convert an SAP unit of measurement into an external unit of measurement.

Import Parameters

Parameter Name	Value Help
SAP_CODE	SAP code unit of measurement

Export Parameters

Parameter Name	Value Help
ISO_CODE	ISO code unit of measurement

> **Note**
>
> Class exception `CX_CONVERSION_EXT_INT` is raised to check the integrity of the data import parameters.

Sample Code

```
CLASS zcl_conversion_ext_int_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_conversion_ext_int_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    TRY.
        DATA(lo_conv) = cl_conversion_ext_int=>get_instance( ).
      CATCH cx_conversion_ext_int.
    ENDTRY.
    "Conversion of units of measurements SAP to ISO
    TRY.
        lo_conv->unit_of_measure_int_to_ext(
          EXPORTING
            sap_code = 'KM'
          IMPORTING
            iso_code = DATA(lv_iso_code)
        ).
        out->write( |Result: { lv_iso_code } | ).
      CATCH cx_conversion_ext_int INTO DATA(lo_error).
        out->write( lo_error->get_text( ) ).
```

```
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.5 Factory Calendar

Get an overview of the ABAP classes, interfaces, and methods you can use to access calendar-related information.

The calendar system comprises the definition of the following categories:

- Public holidays
- Holiday calendars (a collection of public holidays)
- Factory calendars

A factory calendar is based on a country-specific holiday calendar and defines which days are working days, for example, Monday to Friday, and location-specific rules.

Read on to learn about the set of data we deliver for these categories.

> ⓘ Note
>
> Class `CL_SCAL_API` is still available, but deprecated. We recommend that you use class `CL_FHC_CALENDAR_RUNTIME` and the related interfaces `IF_FHC_FCAL_RUNTIME`, `IF_FHC_HCAL_RUNTIME` and `IF_FHC_HOLIDAY_RUNTIME` instead. Furthermore, the CDS views from the section Accessing Holiday and Factory Calendar Tables Using VDM-Compliant CDS Views [page 1428] can be used to get an overview of existing factory- and holiday calendars. For information about the mapping between SCAL-relevant IDs and FHC-relevant IDs, see the corresponding section below.

Creating a Runtime to Access Calendar-Related Information

Use class `CL_FHC_CALENDAR_RUNTIME` that contains the following list of methods to create a runtime for holidays, holiday calendars, and factory calendar:

Method	Description
create_factorycalendar_runtime	Provides a factory calendar runtime
create_holidaycalendar_runtime	Provides a holiday calendar runtime
create_holiday_runtime	Provides a holiday runtime

> {·} Sample Code

```
try.
   data(lo_fcal_run) =
cl_fhc_calendar_runtime=>create_factorycalendar_runtime (
```

```
            iv_factorycalendar_id = 'ExampleID' ).
      catch cx_fhc_runtime into data(lx_err).
        "exception handling
endtry.
```

Reading Factory Calendar-Related Information

Use the before created runtime to access the following list of methods provided in the interface `IF_FHC_FCAL_RUNTIME` to get information about the factory calendar:

Method	Description
convert_date_to_factorydate	Converts a date to a factory date
convert_factorydate_to_date	Converts a factory date to a date
get_last_factorydate	Provides the last factory date of the calendar
calc_workingdays_between_dates	Calculates the number of working days between two dates
add_workingdays_to_date	Adds working days to a date
subtract_workingdays_from_date	Subtracts working days to a date
get_validity_start	Provides the first valid date
get_validity_end	Provides the last valid date
is_date_workingday	Check if a weekday is a working day
is_holiday_workingday	Check if holiday is a working day
get_description	Provides the description of the calendar
get_hcal_assignment	Provides the assigned holiday calendar runtime
get_id	Provides the ID of the factory calendar

Sample Code

```
try.
    data(lv_date) = lo_fcal_run->convert_factorydate_to_date(
            iv_factorydate = '123' ).
      catch cx_fhc_runtime into data(lx_run_err).
        "exception handling
endtry.
```

Reading Holiday Calendar-Related Information

Use the before created runtime to access the following list of methods provided in the interface `IF_FHC_HCAL_RUNTIME` to get information about the holiday calendar:

Method	Description
is_holiday	Check if a date is a holiday
get_holiday	Provides the assigned holiday for a date
calc_holidays_between_dates	Calculates the number of holidays between two dates
get_validity_start	Provides the first valid date
get_validity_end	Provides the last valid date
get_description	Provides the description of the calendar
get_holiday_assignments	Get the assigned holidays from current holiday calendar
get_id	Provides the ID of the holiday calendar

Reading Holiday-Related Information

Use the before created runtime to access the following list of methods provided in the interface `IF_FHC_HOLIDAY_RUNTIME` to get information about the holidays:

Method	Description
get_holiday_id	Provides the ID of the holiday
get_type	Provides the type of the holiday
get_class	Provides the class of the holiday
get_confession	Provides the confession of the holiday
get_text	Provides the title and description of the holiday in a specific language

Mapping between SCAL-relevant IDs and FHC-relevant IDs

Class `CL_FHC_CALENDAR_RUNTIME` uses longer IDs (32 digits) than the deprecated class `CL_SCAL_API`, which had only two-digit IDs for the factory calendars and holiday calendars, and three-digit IDs for the public holidays (also referred to as legacy IDs). The longer IDs make it easier to categorize calendar data.

To enable the use of the above mentioned runtimes, it may be necessary that an application establishes a connection between a legacy two-digit ID (as formerly used with `CL_SCAL_API`) and a longer ID that the newer class `CL_FHC_CALENDAR_RUNTIME` uses.

You can do such a mapping using the interface IF_FHC_ID_MAPPER and class `CL_FHC_CALENDAR_ID_MAPPER`. The mapping is available for holidays, holiday calendars, and factory calendars.

The `CL_FHC_CALENDAR_ID_MAPPER` class returns an ID mapper instance. Subsequently, you can use the methods provided by the interface `IF_FHC_ID_MAPPER` to perform the desired mappings.

> ⚠ Caution
>
> It can't be guaranteed that the 2-digit legacy ID will point to the same calendar on each system. The 2-digit legacy ID should only be used if applications have already used `CL_SCAL_API`, stored the 2-digit legacy ID and have not yet migrated to the new APIs and IDs. The 2-digit legacy IDs are only intended for the transition phase.
>
> We recommend using and saving the new 32-digit IDs on the application side. The whole functionality is optimized for using the new 32-digit ID.

Creating an ID Mapper Instance

Method	Description
create_id_mapper	Provides an ID mapper instance

> {·} Sample Code

```
data(lo_id_mapper) = cl_fhc_calendar_id_mapper=>create_id_mapper(  ).
```

Mapping the IDs

Method	Description
mapping_fcal_legacyid_to_id	Provides the 32-digit factory calendar ID for a two-digit SCAL factory calendar ID
mapping_hcal_legacyid_to_id	Provides the 32-digit holiday calendar ID for a two-digit SCAL holiday calendar ID
mapping_hol_legacyid_to_id	Provides the 32-digit holiday ID for a three-digit SCAL holiday ID
mapping_fcal_id_to_legacyid	Provides the two-digit SCAL factory calendar ID for a 32-digit factory calendar ID
mapping_hcal_id_to_legacyid	Provides the two-digit SCAL holiday calendar ID for a 32-digit holiday calendar ID
mapping_hol_id_to_legacyid	Provides the three-digit SCAL holiday calendar ID for a 32-digit holiday ID

> {·} Sample Code

```
try.
```

```
    data(lv_new_id) = lo_id_mapper->mapping_fcal_legacyid_to_id( iv_legacy_id
= '01' ).
  catch cx_fhc_runtime into data(lx_err).
    "exception handling
endtry.

"another example

try.
   data(lv_legacy_id) = lo_id_mapper-
>mapping_fcal_id_to_legacyid( iv_factorycalendar_id = 'ExampleID' ).
  catch cx_fhc_runtime into lx_err.
    "exception handling
endtry.
```

Reading Additional Factory Calendar-Related Information

Independently from the above factory calendar information, you can use class `CL_SCAL_UTILS`, which contains the following methods:

Method	Description
MONTH_NAMES_GET	Provides the names of the months
WEEK_GET_FIRST_DAY	Provides the first day of a week
DATE_GET_WEEK	Provides the year and week of a date
DATE_COMPUTE_DAY	Provides the name and number of the weekday for a specified date

Sample Code

```
try.
        cl_scal_utils=>date_get_week(
          exporting
            iv_date = '20201001'
          importing
            ev_year = data(lv_year)
            ev_week = data(lv_week) ).
      catch cx_scal into data(lx_scal).
        "exception handling
    endtry.
```

4.2.15.5.1 Accessing Holiday and Factory Calendar Tables Using VDM-Compliant CDS Views

You can access the factory calendar and holiday calendar tables using VDM-compliant CDS views.

The VDM-compliant CDS views used here are CDS views that were created in `SAP_BASIS` and belong to the component `BC-SRV-ASF-CAL`.

The following CDS views access the factory- and holiday calendar tables:

CDS View	Description
I_PublicHolidayCalendarBasic	General information about holiday calendars
I_PublHolidayCalendarBasicText	Maintained description for holiday calendars
I_PublicHolidayCalendarVH	Value help for holiday calendars containing the holiday calendar ID, legacy holiday calendar ID, the validity start and end date and description
I_FactoryCalendarBasic	General information about a factory calendar
I_FactoryCalendarBasicText	Maintained description for factory calendars
I_FactoryCalendarValueHelp	Value help for factory calendars containing the factory calendar ID, legacy factory calendar ID, the validity start and end date and description

> **Note**
>
> The CDS views mentioned here should be differentiated from the C1-released VDM CDS views created by `SAP S/4HANA`. An example for a C1-released CDS view is `I_FactoryCalendar`. C1-released VDM CDS views refer to the tables of the factory- and holiday calendar used previously. They don't belong to component `BC-SRV-ASF-CAL`. For this reason, the name addition `BASIC` was introduced for the new version of the CDS views, as can be seen below.

4.2.15.6 Lock Objects

When you activate a lock object in the ABAP Dictionary, the system automatically creates function modules for setting locks (`ENQUEUE_<lock object name>`) and releasing locks (`DEQUEUE_<lock object name>`).

Using Lock Objects

Since the direct usage of these function modules is not permitted, the function module calls are wrapped in a generic API to set and release locks.

- Use class `CL_ABAP_LOCK_OBJECT_FACTORY` to do the following:
  - Create a lock object instance using method `GET_INSTANCE`
  - Release all locks of the current LUW using method `DEQUEUE_ALL`
- Use the created lock object instance to do the following:
  - Set a lock using method `ENQUEUE`
  - Release a lock using method `DEQUEUE`

For more information, see the ABAP Doc comments of class `CL_ABAP_LOCK_OBJECT_FACTORY` and interface `IF_ABAP_LOCK_OBJECT`. The interface `IF_ABAP_LOCK_OBJECT` also contains constants to be used for parameterization of lock methods.

> **Sample Code**

```
data lv_value type <type of lock object parameter 1>.
```

```
lv_value = <value for lock parameter 1>.
data(lr_lock_object) =  cl_abap_lock_object_factory=>get_instance(
    exporting iv_name = <name of lock object> ).
try.
    lr_lock_object->enqueue(
       it_parameter =  value #( ( name = <name of lock parameter 1> value =
ref #( lv_value ) ) )
       it_table_mode = value #( ( table_name = <table name> mode =
if_abap_lock_object=>cs_mode-<constant for lock mode>) ) ).
  catch cx_abap_lock_failure into data(lr_exc).
    <exception handling code>
endtry.
<application code>
try.
    lr_lock_object->dequeue( ).
  catch cx_abap_lock_failure into lr_exc.
    <exception handling code>
endtry.
```

4.2.15.7 Number Range Solution

Many business applications require unique numbers, for example, to complete the keys of data records. In order to get numbers from an interval, a number range object must be defined which can contain different properties. In addition, intervals containing the numbers, must be assigned to the number range object. Numbers can be generated from existing number range intervals.

> **Note**
>
> Creation, change, and deletion of number range objects and intervals require developer role authorization. Changes to objects and intervals can only be performed in the same software layer.

4.2.15.7.1 Maintaining Number Range Objects

To maintain a number range object during design time, the ABAP development tools for Eclipse UI editor for number range objects can be used. For more information, see Working with Number Range Objects.

Class `CL_NUMBERRANGE_OBJECTS` provides methods for maintaining number range objects.

For more information, see

- Creating Number Range Objects [page 1431]
- Changing Number Range Objects [page 1433]
- Deleting Number Range Objects [page 1434]
- Reading Number Range Objects [page 1434]

Related Information

Working with Number Range Objects

4.2.15.7.1.1 Creating Number Range Objects

Use method CREATE to create number range objects.

When creating a number range object, the following naming-rules apply:

- For customer objects, the name must start with a 'Z' or a 'Y'.
- The maximum length of a number range object is 10 characters.

Import Parameters

Parameter Name	Field Name	Value Help
ATTRIBUTES		Number Range Object Definition.
	OBJECT	Name Range Object.
	DTELSOBJ	Data element for sub-object.
	YEARIND	Flag, whether number range object is to- year relevant.
	DOMLEN	Domain, which determines the length of the numbers.
	PERCENTAGE	Percentage of numbers remaining in an interval after having identified in which number assignment a warning is given.
	CODE	Transaction code to call interval maintenance (obsolete for ABAP CP).
	BUFFER	Buffering type for number assignment.
	NOIVBUFFER	Number of numbers in the buffer.
	NRSWAP	Selecting the flag prevents the intervals from automatically starting from the beginning at the upper limit.
	NRCHECKASCII	
	DEVCLASS	Development class of the object.
OBJ_TEXT	CORRNR	Correction number for transport.

Parameter Name	Field Name	Value Help
		Texts for objects for change document object creation.
	LANGU	Language.
	TXT	Description of object, long text.
	TXTSHORT	Description of object, short text.

Export Parameters

Parameter Name	Field Name	Value Help
ERRORS		
	MSGID	Message class (NR)
	MSGTYPE	Message type
	MSGNUMBER	Message number
	MSGVAR1	Variable to message
	MSGVAR2	Variable to message
	MSGVAR3	Variable to message
	MSGVAR4	Variable to message
	TABLENAME	Table
	FIELDNAME	Field
	CRITCHANGE	Critical change
RETURNCODE		Space: no error E: error W: warning

Sample Code

```
CLASS zcl_nr_object_create DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
   INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_nr_object_create IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
  DATA:
    lv_object   TYPE cl_numberrange_objects=>nr_attributes-object,
```

```
    lv_devclass TYPE cl_numberrange_objects=>nr_attributes-devclass,
    lv_corrnr   TYPE cl_numberrange_objects=>nr_attributes-corrnr.
    lv_object   = 'Z_TEST_03'.
    lv_devclass = 'Z_SNUM'.
    lv_corrnr   = 'SIDK123456'.
    TRY.
    cl_numberrange_objects=>create(
      EXPORTING
        attributes = VALUE #( object     = lv_object
                              domlen     = 'CHAR8'
                              percentage = 5
                              buffer     = abap_true
                              noivbuffer = 10
                              devclass   = lv_devclass
                              corrnr     = lv_corrnr )
        obj_text   = VALUE #( object     = lv_object
                              langu      = 'E'
                              txt        = 'Create object'
                              txtshort   = 'Test create' )
      IMPORTING
        errors     = DATA(lt_errors)
        returncode = DATA(lv_returncode)
        ).
      CATCH.
        …
    ENDTRY.
    …
  ENDMETHOD.
ENDCLASS.
```

4.2.15.7.1.2 Changing Number Range Objects

Use method `CHANGE` to update number range objects.

Import and export parameters are the same as in the `CREATE` method.

Sample Code

```
…
    lv_object   = 'Z_TEST_03'.
    lv_devclass = 'Z_SNUM'.
    lv_corrnr   = 'SIDK123456'.
…
  cl_numberrange_objects=>update(
    EXPORTING
      attributes = VALUE #( object     = lv_object
                            domlen     = 'CHAR8'
                            percentage = 9
                            buffer     = 'S'
                            noivbuffer = 12
                            devclass   = lv_devclass
                            corrnr     = lv_corrnr )
      obj_text   = VALUE #( object     = lv_object
                            langu      = 'E'
                            txt        = 'Update object'
                            txtshort   = 'Test update' )
    IMPORTING
      errors     = DATA(lt_errors)
      returncode = DATA(lv_returncode)
        ).
    …
```

4.2.15.7.1.3 Deleting Number Range Objects

Use method DELETE to delete number range objects.

Import Parameters

Parameter Name	Field Name	Value Help
OBJECT		Number Range Object
CORRNR		Correction number for transport

Sample Code

```
…
lv_object = 'Z_TEST_03'.
lv_corrnr = 'SIDK123456'.
…
  cl_numberrange_objects=>delete(
    EXPORTING
      object = lv_object
      corrnr = lv_corrnr
        ).
…
```

4.2.15.7.1.4 Reading Number Range Objects

Use the READ method to read the attributes of a number range object.

Import Parameters

Parameter Name	Field Name	Value Help
LANGUAGE		Language for the object texts
OBJECT		Number Range Object

Export Parameters

Parameter Name	Field Name	Value Help
ATTRIBUTES		Number Range Object Definition.
	OBJECT	Number Range Object.
	DTELSOBJ	Data element for sub-object.

Parameter Name	Field Name	Value Help
	YEARIND	Flag, whether number range object is to-year relevant.
	DOMLEN	Domain, which determines the length of the numbers.
	PERCENTAGE	Percentage of numbers remaining in an interval after having identified in which number assignment a warning is given.
	CODE	Transaction code to call interval maintenance (obsolete for ABAP CP).
	BUFFER	Buffering type for number assignment.
	NOIVBUFFER	Number of numbers in the buffer.
	NRSWAP	Selecting the flag prevents intervals from automatically starting from the beginning at the upper limit.
	NRCHECKASCII	
INTERVAL_EXISTS		
OBJ_TEXT		Texts for objects for change document object creation.
	LANGU	Language.
	TXT	Description of object, long text.
	TXTSHORT	Description of object, short text.

Sample Code

```
…
 lv_object = 'Z_TEST_03'
…
cl_numberrange_objects=>read(
          EXPORTING
            language        = sy-langu
            object          = lv_object
          IMPORTING
            attributes      = DATA(ls_attributes)
            interval_exists = DATA(lv_interval_exists)
            obj_text        = DATA(obj_text)
        ).
…
```

4.2.15.7.2 Maintaining Intervals of Number Range Objects

The class `CL_NUMBERRANGE_INTERVALS` provides methods for maintaining intervals of number range objects. Will the APIs be used by a business user, assign the `SAP_CA_BC_IC_LND_NUM_PC` (Number Range Management – Configuration) business catalog to a proper business role and - if required - restrict it to specific number range objects.

For more information, see

- Creating Intervals of Number Range Objects [page 1436]
- Changing Intervals of Number Range Objects [page 1438]
- Deleting Intervals of Number Range Objects [page 1439]
- Reading Intervals of Number Range Objects [page 1439]

4.2.15.7.2.1 Creating Intervals of Number Range Objects

Use the `CREATE` method to create number range intervals.

Import Parameters

Parameter Name	Field Name	Value Help
INTERVAL		Interval Table
	SUBOBJECT	Number Range Object Sub-object Value
	NRRANGENR	Number Range Number
	TOYEAR	To Fiscal Year
	FROMNUMBER	From Number
	TONUMBER	To Number
	NRLEVEL	Number Range Level
	EXTERNIND	Internal (' ') or external ('X') number range flag
	PROCIND	Processing flag (I=Insert, D=Delete, U=Update,' '=no changes)
OBJECT		Number Range Object
SUBOBJECT		Sub-object

Export Parameters

Parameter Name	Field Name	Value Help
ERROR		Flag showing that an error occurred during testing
ERROR_INF		Error Information
	MSGNR	Message Number
	TABLENAME	Parameter Name
	FIELDNAME	Field Name
	TABIX	Index of Row with Error
ERROR_IV		Intervals with errors
	SUBOBJECT	Number range object subobject value
	NRRANGENR	Number range number
	TOYEAR	To fiscal year
	FROMNUMBER	From number
	TONUMBER	To number
	NRLEVEL	Number range level
	EXTERNIND	Internal (' ') or external ('X') number range flag
	PROCIND	Processing flag (I=Insert, D=Delete, U=Update,' '=no changes)
WARNING		Flag: Warning after check?

Sample Code

```
…
CLASS zcl_nr_test_intervals_create DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC.
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_nr_test_intervals_create IMPLEMENTATION.
METHOD if_oo_adt_classrun~main.
DATA: lv_object   TYPE cl_numberrange_objects=>nr_attributes-object,
      lt_interval TYPE cl_numberrange_intervals=>nr_interval,
      ls_interval TYPE cl_numberrange_intervals=>nr_nriv_line.
    lv_object = 'Z_TEST_03'.
*   intervals
```

```
    ls_interval-nrrangenr  = '01'.
    ls_interval-fromnumber = '00000001'.
    ls_interval-tonumber   = '19999999'.
    ls_interval-procind    = 'I'.
    APPEND ls_interval TO lt_interval.
    ls_interval-nrrangenr  = '02'.
    ls_interval-fromnumber = '20000000'.
    ls_interval-tonumber   = '29999999'.
    APPEND ls_interval TO lt_interval.
*   create intervals
    TRY.
        out->write( |Create Intervals for Object: { lv_object } | ).
        CALL METHOD cl_numberrange_intervals=>create
          EXPORTING
            interval  = lt_interval
            object    = lv_object
            subobject = ' '
          IMPORTING
            error     = DATA(lv_error)
            error_inf = DATA(ls_error)
            error_iv  = DATA(lt_error_iv)
            warning   = DATA(lv_warning).
      ENDTRY.
  ENDMETHOD.
ENDCLASS.
…
```

4.2.15.7.2.2 Changing Intervals of Number Range Objects

Use the `UPDATE` method to change number range intervals.

Import and export parameters are the same as in the `CREATE` method.

Sample Code

```
…
    lv_object = 'Z_TEST_03'.
*   intervals
    ls_interval-nrrangenr  = '01'.
    ls_interval-fromnumber = '00000002'.
    ls_interval-tonumber   = '19999998'.
    ls_interval-procind    = 'U'.
    APPEND ls_interval TO lt_interval.
    ls_interval-nrrangenr  = '02'.
    ls_interval-fromnumber = '20000002'.
    ls_interval-tonumber   = '29999997'.
    APPEND ls_interval TO lt_interval.
…
 CALL METHOD cl_numberrange_intervals=>update
      EXPORTING
        interval  = lt_interval
        object    = lv_object
        subobject = ' '
      IMPORTING
        error     = DATA(lv_error)
        error_inf = DATA(ls_error)
        error_iv  = DATA(lt_error_iv)
        warning   = DATA(lv_warning)
…
```

4.2.15.7.2.3 Deleting Intervals of Number Range Objects

Use the `DELETE` method to delete number range intervals.

Import and export parameters are the same as in the `CREATE` method.

Sample Code

```
…
    lv_object = 'Z_TEST_03'
*   intervals
    ls_interval-nrrangenr  = '01'.
    ls_interval-fromnumber = '00000001'.
    ls_interval-tonumber   = '19999999'.
    ls_interval-procind    = 'D'.
    APPEND ls_interval TO lt_interval.
    ls_interval-nrrangenr  = '02'.
    ls_interval-fromnumber = '20000000'.
    ls_interval-tonumber   = '29999999'.
    APPEND ls_interval TO lt_interval.
 …
  CALL METHOD cl_numberrange_intervals=>delete
       EXPORTING
         interval  = lt_interval
         object    = lv_object
         subobject = ' '
       IMPORTING
         error     = DATA(lv_error)
         error_inf = DATA(ls_error)
         error_iv  = DATA(lt_error_iv)
         warning   = DATA(lv_warning).
…
```

4.2.15.7.2.4 Reading Intervals of Number Range Objects

Use the `READ` method to get the properties of number range intervals.

Import Parameters

Parameter Name	Field Name	Value Help
NR_RANGE_NR1		Interval Number (internal interval)
NR_RANGE_NR2		Interval Number (external interval)
OBJECT		Number Range Object
SUBOBJECT		Sub-object

Sample Code

```
…
  lv_object = 'Z_TEST_03'.
…
  CALL METHOD cl_numberrange_intervals=>read
```

```
        EXPORTING
          object       = lv_object
          nr_range_nr1 = ' '
          nr_range_nr2 = ' '
          subobject    = ' '
        IMPORTING
          interval     = lt_interval.
…
```

4.2.15.7.2.5 How to Create a Fiori App for a Single Number Range Object

You can maintain files in the SAP Fiori launchpad to launch the *Number Range* app with the help of a custom app.

Prerequisites

The business user will need to be assigned with suitable authorization in the SAP Business AI Platform, ABAP environment:

- the `SAP_CA_BC_IC_LND_NUM_PC` catalog is needed, as it grants access to the *Maintain Number Ranges* app
- the `SAP_CA_BC_IC_LND_PC` catalog is needed, as it grants access to the customizing changes
- if the targeted business configuration instance is protected, the relevant custom business catalog must also be assigned to the business user

Procedure

A custom tile that navigates directly to the *Number Range Object* app has been created. This is done by using the parameter `Technical Identifier` in the intent-based navigation .

The implementation contains a creation and deployment of an empty freestyle UI5 app that redirects to the `NumberRangeInterval-manage` (manage number range intervals) with the required parameter. When clicking the tile, the launchpad will first navigate to the intent of the custom app (which is maintained in the app descriptor) and will then automatically be rerouted to the intent of the *Maintain Business Configurations* app. This rerouting is quite simple to implement after generating an empty app in Business Application Studio (BAS).

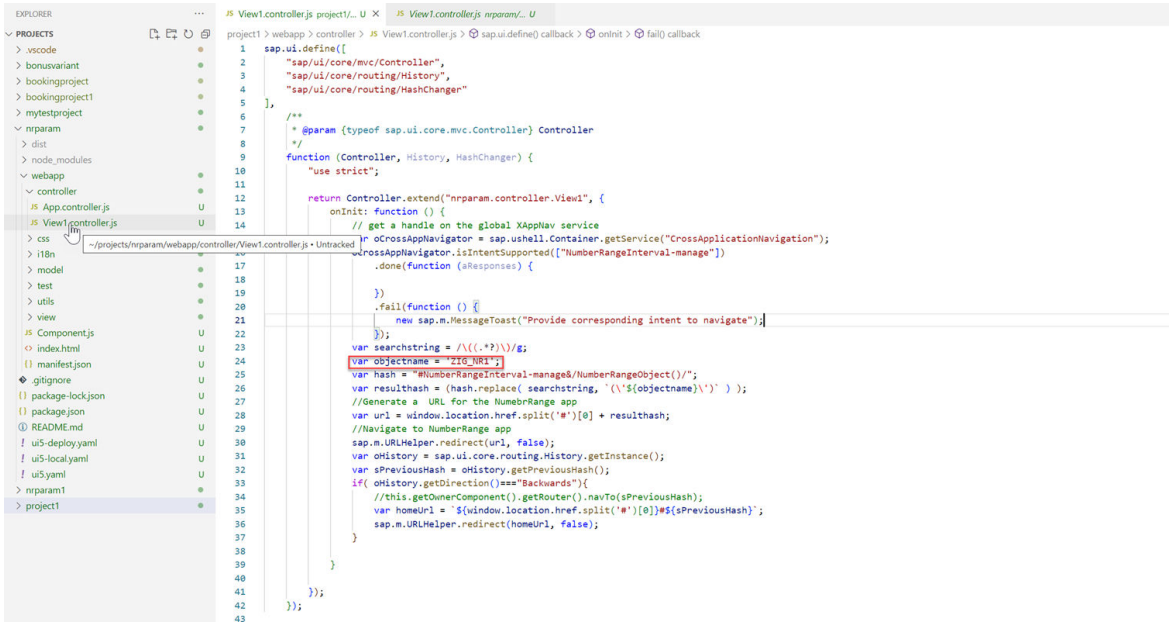
In terms of authorization control, a custom business catalog and business role needs to be created in the SAP Business AI Platform, ABAP environment and assigned to the business user.

Reproduce

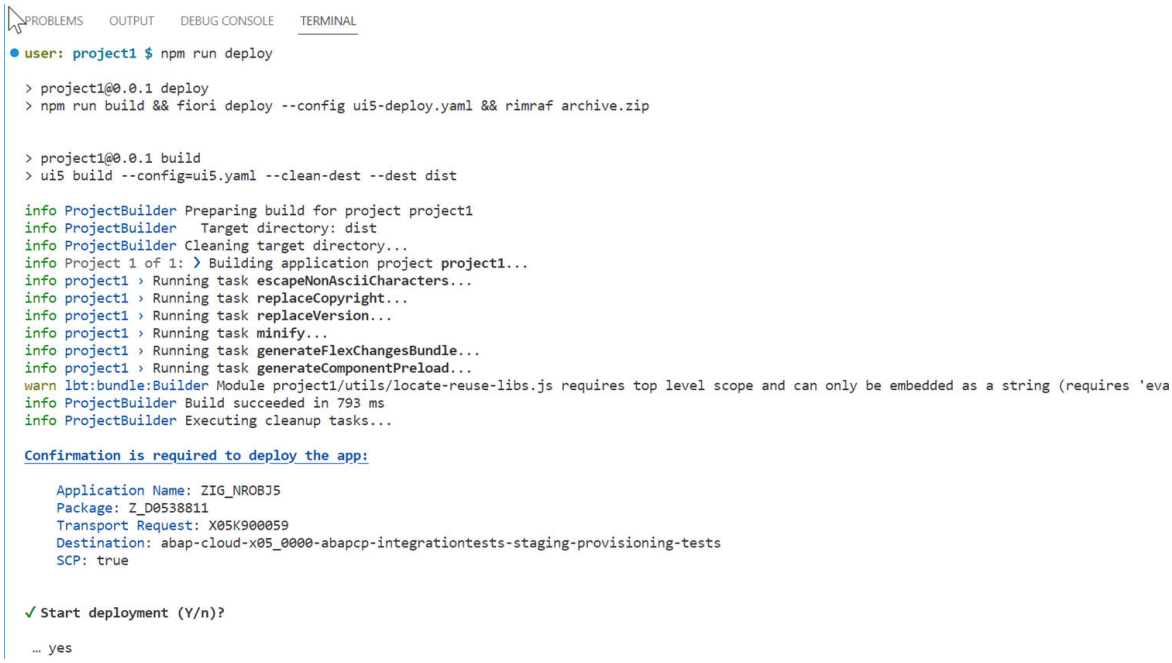
A development package and a transport request are needed to implement the solution.

Create a new SAP Fiori application in BAS

1. Log on to the Business Application Studio (BAS) Development Space to create a new *SAP Fiori application* in BAS and press *Start*.
2. Choose SAPUI5 freestyle from the drop-down menu *Application Type* and press *Next*.



3. For *Data source* choose *None* out of the drop-down menu and press *Next*. You can use the proposed view name in the next step or rename it. Press *Next*.



4. After filling in the mandatory fields of the project attributes, add your deployment configuration as well as your Fiori configuration by choosing *Yes*.

> ⓘ Note
>
> This configuration can also be added later via the terminal by running the command
>
> `npx fiori add flp-config`

Press *Next*.

5. 1. Add a target repository name and a target package, then create a workbench transport request or choose an already existing one. Press *Next*.
6. Finally, adjust the Fiori Configuration information:
   - Choose a semantic object that represents a business entity such as a customer, a sales order or a product.
   - Choose an action that describes which operation (such as `display` or `manage`) is intended to be performed on a semantic object.
   - Choose a title that will represent the tile name.

   Press *Finish*.

After creating a new SAP Fiori application in BAS, you can deploy it.

Deploy your SAP Fiori application

1. In the project folder open the created project in the integrated terminal.
2. Go to the *controller* folder which you will find in the folder *webapps* of the project. Open `View1.controller.js`. Insert the code snippet into the view. Adopt the value of the `objectname` variable to the object which you would like to call in Fiori.
3. Deploy your application by means of running the terminal command: `npm run deploy`.
4. After successful deployment a note will appear that is required for the identity application manager.

> {·} Sample Code
>
> ```
> info builder:custom deploy-to-abap Fiori launchpad App Descr Item
> ZIG_NROBJ5_UI5R was created
> ```

5. Now, a successfully deployed application will appear in ABAP development tools for Eclipse in the BSP library folder of the deployment package.

To publish the business catalog to the system and pin your final product to the home screen of your Fiori launchpad, continue reading the next topic called *IAM App Creation and Settings*.

4.2.15.7.2.5.1 IAM App Creation and Settings

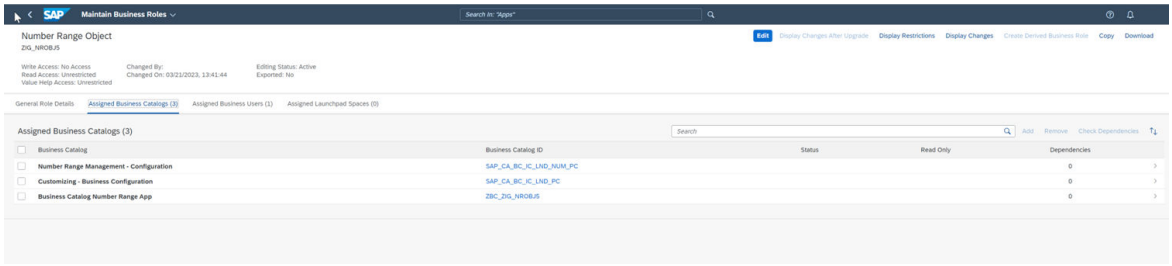
To publish a business catalog, you need to create a new IAM app first in case you do not use an already existing one.

1. Create a new IAM app under the Cloud Identity and Access Management folder.
2. Add an IAM app name as well as a description and press *Next*.
3. Choose a corresponding transport request and press *Finish*.
4. Open the created IAM app and add the previously deployed Business Application Studio app as *Fiori Launchpad Descr Item ID*. Finally, press *Publish Locally*.

5. Now assign the IAM app to a new or already existing business catalog and an according description. Press *Finish*.
6. Complete the process by publishing the business catalog to the system. Press *Publish Locally*.

Business Role Settings

1. Log onto the staging landscape of the Steampunk system and create a new business role or use an existing one.
2. Assign the previously published business catalog to the business role and press *Save*.
3. In order to run your application properly, your business role should contain the following business catalogs:



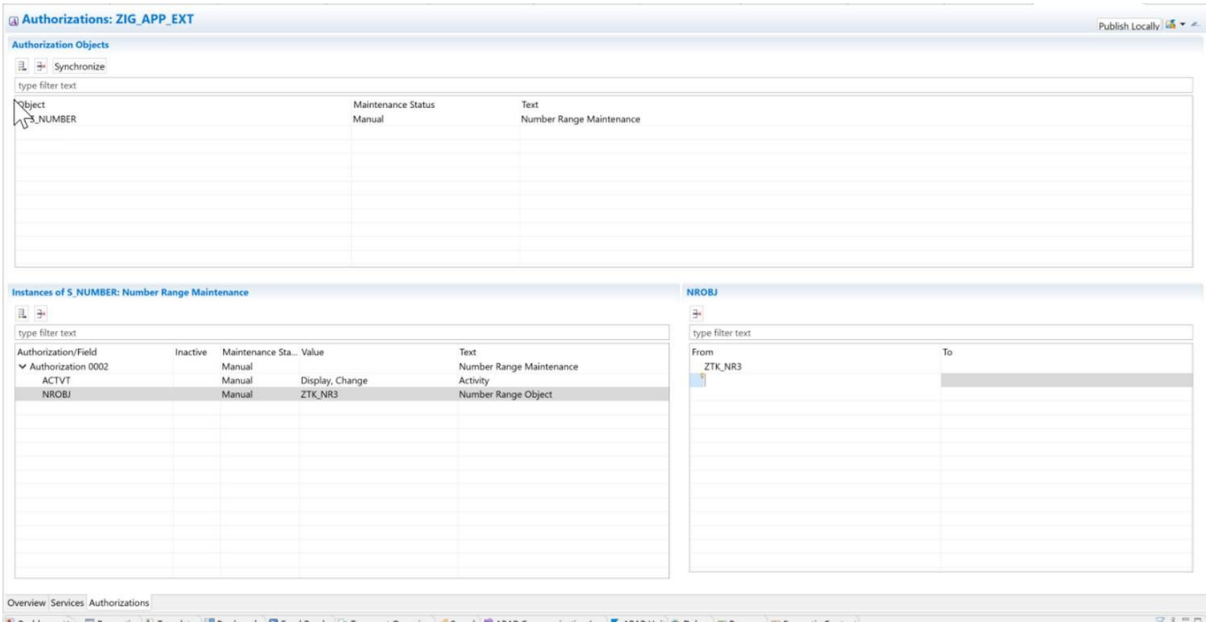
4. Go back to the Fiori Launchpad Home view and find your application by searching via App Finder in the drop-down menu of your account icon on the upper right corner of the screen. Now you can also pin the app on your home screen.

4.2.15.7.2.5.2 Authorization Object S_NUMBER

Prerequisites

- You need access to the authorization role `S_NUMBER` for the number range app.
- You need access to an authorization role of the `ZTK_NR3 object`.

To make objects available for business partners in the SAP Business AI Platform, ABAP environment, you need to create an I AM app and assign the authorization `S_NUMBER` to this app in the authorization settings. Here, all available objects will be listed, though they don't need to be released objects.



Afterwards, you create a business catalog and assign the created IAM application to it.

Next, you need to assign a user to the catalog in the I AM app. In the Fiori Identity and Access Management menu, you can navigate to the Business Catalog maintenance view. Here, you are able to assign a business user to the created business catalog. Now as a user you are able to only see objects that were assigned to this catalog and keep all other objects private.

In parallel to this setting, you still need an assignment to the number range catalog to be able to see number range objects. In the number range catalog, you can navigate to the restriction settings to edit the readability and changeability restrictions for individual objects.

4.2.15.7.3 Getting Numbers from an Interval

The `CL_NUMBERRANGE_RUNTIME` class provides methods for getting numbers from an interval at runtime.

Checking Numbers for External Intervals

Use the `NUMBER_CHECK` method to check whether a number is within an external interval.

Import Parameters

Parameter Name	Field Name	Value Help
NR_RANGE_NR		Interval number
NUMBER		Number to be checked

Parameter Name	Field Name	Value Help
NUMERIC_CHECK		Numeric check (for numeric intervals only)
OBJECT		Number range object
SUBOBJECT		Sub-object
TOYEAR		To fiscal year
NUMBER_ALPHA		Alphanumeric and optional length check
LENGTH_CHECK		Check of number length

Export Parameter

Parameter Name	Field Name	Value Help
RETURNCODE		Return code

Getting Numbers for Internal Intervals

Use the `NUMBER_GET` method to determine the next number of a number range interval.

Import Parameters

Parameter Name	Field Name	Value Help
IGNORE_BUFFER		Ignore Buffer
NR_RANGE_NR		Interval Number
OBJECT		Number Range Object
QUANTITY		Number of Numbers in Buffer
SUBOBJECT		Sub-object
TOYEAR		To Fiscal Year

Export Parameters

Parameter Name	Field Name	Value Help
NUMBER		Returned number
RETURNCODE		Return code
RETURNED_QUANTITY		Number of returned numbers

Sample Code

```
…
  lv_object = 'Z_TEST_03'.
…
  CALL METHOD cl_numberrange_runtime=>number_get
       EXPORTING
         nr_range_nr = '01'
         object      = lv_object
       IMPORTING
         number      = DATA(lv_number)
         returncode  = DATA(lv_rcode).
…
```

Getting the Number Status of an Interval

Use the `NUMBER_STATUS` method to determine the number status of a number range interval.

Import Parameters

Parameter Name	Field Name	Value Help
NR_RANGE_NR		Interval number
OBJECT		Number range object
SUBOBJECT		Sub-object
TOYEAR		To Fiscal Year

Export parameter

Parameter Name	Field Name	Value Help
NUMBER		Number status of the interval

4.2.15.7.4 Getting Numbers from an Interval in RAP environment

The CL_NUMBERRANGE_BUFFER class provides methods for getting numbers from an interval at runtime. Each method is called based on the buffer type setting of the object. In case the method caller calls a method, which doesn't coincide with the buffer type of the object, an exception CX_NUMBER_RANGES will be raised.

Getting Numbers for Internal Intervals

Based on the buffering settings of the object, you can call a specific API method.

Getting Numbers for the Object with main Memory Buffering

Use the NUMBER_GET_MAIN_MEMORY method to determine the next number of a number range interval.

Import Parameters

Parameter Name	Value Help
IV_OBJECT	Number Range Object
IV_SUBOBJECT	Sub-object
IV_INTERVAL	Interval Number
IV_QUANTITY	Number of Numbers in Buffer
IV_TOYEAR	To Fiscal Year

Export Parameters

Parameter Name	Value Help
EV_NUMBER	Returned Number
EV_RETURNCODE	Return Code
EV_RETURNED_QUANTITY	Number of Returned Numbers

Sample Code

```
lv_object = 'ZIG_NR5'.
lv_interval = '01'.
lv_quantity = 10.
TRY.
DATA(lo_get_number) = cl_numberrange_buffer=>get_instance( ).
lo_get_number->if_numberrange_buffer~number_get_main_memory( EXPORTING
iv_object    =
lv_object
iv_subobject = lv_subobject
                                              iv_interval  = lv_interval
                                                    iv_quantity  =
lv_quantity
                                            iv_toyear    = lv_year
                                                IMPORTING
ev_number    = lv_number
                                            ev_returned_quantity =
lv_returned_qunatity).
CATCH cx_number_ranges INTO DATA(lr_error).
ENDTRY.
```

Getting Numbers for the object with parallel buffering

Use NUMBER_GET_PARALLEL method to determine the next number of a number range interval.

Import Parameters

Parameter Name	Value Help
IV_OBJECT	Number Range Object
IV_SUBOBJECT	Sub-object
IV_INTERVAL	Interval Number
IV_QUANTITY	Number of Numbers in Buffer

Parameter Name	Value Help
IV_TOYEAR	To Fiscal Year

Export Parameters

Parameter Name	Value Help
EV_NUMBER	Returned Number
EV_RETURNCODE	Return Code

Sample Code

```
lv_object = 'ZIG_NR2'.
lv_interval = '01'.
TRY.
DATA(lo_get_number) = cl_numberrange_buffer=>get_instance( ).
lo_get_number->if_numberrange_buffer~number_get_parallel( EXPORTING
iv_object    = lv_object
                                                          iv_interval  =
lv_interval
                                                          iv_subobject =
lv_subobject
                                                          iv_toyear    =
lv_year
                                                          IMPORTING
ev_number    = iv_number ).
  CATCH cx_number_ranges INTO DATA(lr_error).
ENDTRY.
```

Getting Numbers for the Object without Buffering

Use the NUMBER_GET_NO_BUFFER method to determine the next number of a number range interval.

Export Parameters

Parameter Name	Value Help
EV_NUMBER	Returned Number
EV_RETURNCODE	Return Code
EV_RETURNED_QUANTITY	Number of Returned Numbers

Sample Code

```
lv_object = 'ZIG_NR9'.
lv_interval = '01'.
lv_quantity = 10.
TRY.
  DATA(lo_get_number) = cl_numberrange_buffer=>get_instance( ).
lo_get_number->if_numberrange_buffer~number_get_no_buffer( EXPORTING
iv_object            = lv_object
                                                    iv_subobject         =
lv_subobject
                                                    iv_interval          =
lv_interval
                                                    iv_toyear            =
lv_year
                                                    iv_quantity          =
lv_quantity
```

```
                                                        iv_ignore_buffer     =
abap_true
              IMPORTING
 ev_number            = lv_number
         ev_returned_quantity = lv_returned_qunatity).
CATCH cx_number_ranges INTO DATA(lr_error).
ENDTRY.
```

4.2.15.8 Customer Data Browser

With this app, you can view SAP Business AI Platform, ABAP environment data owned by the customer.

Key Features

You can use this app as a self-service to:

- View business data owned by customers.
- View the data of custom fields.
- View the technical names and business names of fields.
- Configure filter criteria and sort data.
- Configure display columns.
- Get the total number of records.
- Export data visible in the application to a spreadsheet.
- Provide controlled and restricted access to users (Admin user feature).

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-CDB`.

Supported Device Types

- Desktop
- Tablet

Related Information

Customer Data Browser Objects [page 1450]

Read Access Logging [page 1451]
Security Audit Log [page 1451]
Accessing the App [page 1455]
Data Visibility Based on Restrictions [page 1452]
Working wtih the App [page 1457]
FAQs [page 1459]

4.2.15.8.1 Customer Data Browser Objects

Customer Data Browser is a self-service application that you can use to view data that belongs to CDS views and database tables allowed for user access by SAP or by the customer.

The CDS views and database tables allowed for user access by SAP in the Customer Data Browser app are called Customer Data Browser objects.

You can create Customer Data Browser objects using the ABAP Development Tools (ADT). For more information, see Working with Customer Data Browser Objects.

CDS View Coverage

In keeping with the SAP BTP ABAP Environemnt guidelines, the Customer Data Browser application primarily displays data using published CDS views (of the type contract C1) that are relevant to this environment.

For application areas which do not have good coverage of published CDS views, SAP may provide database tables as a temporary solution to display the data. After the relevant CDS views are published, SAP may disable the corresponding tables.

As part of continuous review and update of application content, the number of allowed Customer Data Browser objects available in the application are projected to increase with every upcoming release.

Disclaimer

The use of the Customer Data Browser application may conflict with the data protection policies of your organization. The authorization concept does not support any filtering by the Data Controller or by typical attributes reflecting the purposes of processing. In addition, a download of the data is a transfer in a software/ environment that is potentially not secured . Hence, before using the Customer Data Browser application, please ensure that you follow the data protection Policies in your organization.

Related Information

Customer Data Browser [page 1449]
Read Access Logging [page 1451]

4.2.15.8.2 Read Access Logging

Use

In Read Access Logging (RAL), you can configure which read-access information to log and under which conditions.

SAP delivers sample configurations for applications.

The **BC/BC-SRV-APS-CDB/Template for Preview Access** captures information related to viewing and accessing data.

In the following configurations, fields are logged in combination with additional fields, in the following business contexts:

Channel	Configuration	Fields Logged	Business Context
SAP Gateway (OData)	`BC/BC-SRV-APS-CDB/Template for Preview Access`	Username (`SY-UNAME`) Request URL (`:Request URL`)>	Log viewing and accessing data along with the search filter criteria

> **Note**
>
> See SAP Note 3351781 for information about the template RAL configuration file for Customer Data Browser.

Related Information

Customer Data Browser [page 1449]
Security Audit Log [page 1451]

4.2.15.8.3 Security Audit Log

In Security Audit Log (SAL), you can get information about user access to the *Customer Data Browser* application.

You can check the security audit logs for the Customer Data Browser app using the Display Security Audit Log [page 3190] app. In the *Display Security Audit Log* app, choose the audit event FU5.

For more information about security audit logs, see: Security Audit Log.

Related Information

Customer Data Browser [page 1449]
Accessing the App [page 1455]
Data Visibility Based on Restrictions [page 1452]

4.2.15.8.4 Data Visibility Based on Restrictions

Restriction Fields in the Maintain Business Roles App

The administrator can set restrictions on restriction fields in the Maintain Business Roles app. The restrictions set appear in the *Read/Value Help* restriction type in the Maintain Business Roles app.

> **Note**
>
> When maintaining restrictions, set the *Write, Read, Value Help* restriction to **No Access**.

The user can view objects in the Customer Data Browser app based on the restrictions set by the administrator.

The *Read/Value Help* restriction type for the Customer Data Browser app consists of the following restriction fields:

Restriction Field	Description
Object Group	Logically groups Customer Data Browser objects. Note that this field is not active.
Object Name	Displays a list of Customer Data Browser objects and their descriptions.
Object Fields	Provides access to the following categories of fields in Customer Data Browser objects: • GE: General fields that do not contain sensitive personal information • SP: Sensitive personal information fields that contain key fields and fields with sensitive personal information • Unrestricted: All fields containing general and sensitive personal information Note that the field categories are determined by internal SAP classifications.

Restriction Field	Description
Object Records	Provides access to the following categories of data records in Customer Data Browser objects: • OV: Records with bypassed access controls (privileged access) • AC: Records that fulfill access controls (unprivileged access) • Unrestricted: Records that fulfill access controls (unprivileged access)

Display Behavior in the Customer Data Browser App

See the restriction categories and visibility scenarios below to understand what you can view in the Customer Data Browser application.

Restriction Category: Bypassing Access Controls

Scenario 1: View the data of all CDS views and all database tables

If you want to view the data of all CDS views and database tables, the administrator needs to set the following restrictions in the Maintain Business Roles app:

Object	Required Restriction
Object Name	Unrestricted
Object Fields	General fields (GE) <or> Sensitive personal fields (SP) <or> Both
Object Records	Access to records with bypassed access controls (OV)

Scenario 2: View the data of only CDS views

If you want to view the data of only CDS views, the administrator needs to set the following restrictions in the Maintain Business Roles app:

Object	Required Restriction
Object Name	Restricted - Select all CDS views
Object Fields	General fields (GE) <or> Sensitive personal fields (SP) <or> Both
Object Records	Access to records with bypassed access controls (OV)

Scenario 3: View the data of only database tables

If you want to view the data of only database tables, the administrator needs to set the following restrictions in the Maintain Business Roles app:

Object	Required Restriction
Object Name	Restricted - Select all database tables
Object Fields	General fields (GE) <or> Sensitive personal fields (SP) <or> Both
Object Records	Access to records with bypassed access controls (OV)

Scenario 4: View the data of specific CDS views and specific database tables

If you want to view the data of specific CDS views and specific database tables, the administrator needs to set the following restrictions in the Maintain Business Roles app:

Object	Required Restriction
Object Name	Restricted - Select specific CDS views and specific database tables
Object Fields	General fields (GE) <or> Sensitive personal fields (SP) <or> Both
Object Records	Access to records with bypassed access controls (OV)

Scenario 5: View the data of a group of objects created with SSCUI

If you want to view the data of objects created with SSCUI, the administrator needs to set the following restrictions in the Maintain Business Roles app:

Object	Required Restriction
Object Name	Optional - Restricted <or> Unrestricted <or> Not maintained
Object Fields	General fields (GE) <or> Sensitive personal fields (SP) <or> Both
Object Records	Access to records with bypassed access controls (OV)

Restriction Category: Retaining Access Controls

Scenario 1: View the data of only CDS views

If you want to view the data of only CDS views, the administrator needs to set the following restrictions in the Maintain Business Roles app

Object	Required Restriction
Object Name	Unrestricted <or> Restricted
Object Fields	General fields (GE) <or> Sensitive personal fields (SP) <or> Both
Object Records	Access to records that fulfill access controls (AC)

Scenario 2: View the data of only database tables

This is not possible.

Restriction Permissions by User Role

Business Users

You can access...	If...	Even if...
Data in CDS views	You have the required CDS view-specific access controls	You have Customer Data Browser-specific access to bypass access controls (Object Records: OV)
Data in database tables	You have Customer Data Browser-specific access to bypass access controls (Object Records: OV)	

Administrators

You can restrict access to Customer Data Browser objects by providing a list of object names in the *Object Name* restriction field.:

Related Information

Customer Data Browser [page 1449]
Working wtih the App [page 1457]
Accessing the App [page 1455]

4.2.15.8.5 Accessing the App

Customer Data Browser is a self-service application that you can use to view data that belongs to specific CDS views and database tables that are allowed for user access by SAP or by the customer.

> ⓘ Note
>
> The CDS views and database tables that are allowed for user access by SAP or by the customer in the Customer Data Browser app are called Customer Data Browser objects.

Viewing Fiori Tiles

This application is available on the SAP Fiori launchpad screen as a customer self-service.

Based on your assigned business role catalog, you can view the SAP Fiori launchpad tile *Customer Data Browser*.

Required Users

The required user and the respective business catalog role and business role template for the **Customer Data Browser** app are described below.

Required User	User Task	Business Catalog Role	Business Role Template	Application
Business User	Access application	SAP_CORE_BC_CDB_PC	SAP_BR_ADMINISTRATOR_DANA (Administrator - Data Analysis)	Customer Data Browser
Admin User	Maintain authorization restrictions	Not relevant	SAP_BR_ADMINISTRATOR (Administrator)	Maintain Business Roles

Role Purpose

For each role, the table below shows information about access, purpose and assignments.

User Role	Purpose	Access	Recommendations
Administrator `SAP_BR_ADMINISTRATOR`	With this role, you can maintain authorization restrictions using the Maintain Business Roles app.For more information about maintaining restrictions for Customer Data Browser, see How to Define Authorizations Based on Restrictions [page 3075]. .	Access the Maintain Business Roles app in the Fiori Apps Library using the Fiori ID: F1492.	

User Role	Purpose	Access	Recommendations
Administrator – Data Analysis SAP_BR_ADMINISTRATOR	With this role, you can access the Customer Data Browser application.	Access the Customer Data Browser application in the Fiori apps Library using the Fiori ID: F5746.	• Assign the SAP_BR_ADMINISTRATOR_DANA role to a business user who does not have the SAP_BR_ADMINISTRATOR role. • Make sure that the user with the SAP_BR_ADMINISTRATOR_DANA role is assigned to a single copy of the SAP_BR_ADMINISTRATOR_DANA role. **Note** Assigning the user to multiple copies of the SAP_BR_ADMINISTRATOR_DANA role may cause authorization issues when you access the Customer Data Browser app.

Related Information

Customer Data Browser [page 1449]
Data Visibility Based on Restrictions [page 1452]

4.2.15.8.6 Working wtih the App

When you choose the **Customer Data Browser** tile, you can see a list of Customer Data Browser objects that you are authorized to view.

The list of Customer Data Browser objects includes:

- CDS views, including custom CDS views
- Database tables

Working with the App

Navigating to the Data Details Screen

Choose a row to navigate to the *Customer Data Browser – Data Details* screen where you can view the data relevant to the object. You can sort the data and configure filter criteria to view a subset of data.

Entering Parameter Values for CDS Views

If you choose a CDS view that requires you to enter parameter values before previewing data, you can enter the values in the Maintain Parameters dialog box.

Customer Data Browser - Data Details Screen

You can see the following options on the *Customer Data Browser – Data Details* screen

- *Total Records*: Choose to display the total number of records in the selected Customer Data Browser object.

  > ⓘ Note
  >
  > If you set filters, the total number of records is calculated based on the filter criteria.

- *View Technical Name/View Business Name*: Choose this toggle button to display the technical names or the business names of the columns.
- *Parameters*: Choose this button to change parameter values for CDS views that require parameters.

  > ⓘ Note
  >
  > This button is visible if the CDS view requires parameter values before preview.

- *Maximum Rows to Be Displayed*: Enter the maximum number of rows to be displayed on the screen. By default, this value is set to 100. This means that the first 100 entries that match the filter criteria are displayed on the screen. You can reset this value to display a different maximum number.
- *Adapt Filters*: Choose this button to select additional filters or deselect existing filters.
- *Export* : From the *Export* icon, choose the export option to download data displayed on the screen to your local machine as a spreadsheet.
- *Settings*: Choose this button to select columns you want to view or deselect columns you no longer want to view.

Related Information

Customer Data Browser [page 1449]
FAQs [page 1459]

4.2.15.8.7 FAQs

Business Users

1. Why can't I see any Customer Data Browser objects in the app?

Answer: You do not have the authorization to view the objects. Please contact your administrator and ask them to grant the authorization you need to view the objects. For more information, see Data Visibility Based on Restrictions [page 1452].

2: How can I see the complete list of Customer Data Browser objects available with the app?

Answer: You can view the complete list of Customer Data Browser objects if you have admin-level authorization. Contact your administrator to get the complete set of authorizations. Please ask the administrator to select all values in the *Object Name* restriction field.

3. Why can't I find a particular CDS view or table in the list of Customer Data Browser objects?

Answer: Here are some possible reasons:

- Your administrator has not granted you authorization to access this object, or
- The particular object is not yet enabled for Customer Data Browser

Contact your administrator for the exact reason.

4. When trying to view the CDS view data, why do I get the message "You either do not have the required authorization or there is no data to be displayed"?

Answer: You get this message because:

- Your administrator has not granted the required CDS view-specific authorizations (business catalogs) to your user. This means you cannot view the data of this CDS view.
- Alternatively, your administrator has granted the required CDS view-specific authorizations (business catalogs) to your user, but there is no data to be displayed. So, the system displays the error message.

Contact your administrator to obtain the required CDS view-specific authorizations.

5. When trying to view the data of a table, why do I get the message "No authorization to view object data"?

Answer: You see this message when your administrator has not granted the Customer Data Browser-specific authorizations to your user to view the data of database tables.

Contact your administrator to obtain the required authorizations.

For more information, see Data Visibility Based on Restrictions [page 1452].

6. My administrator has maintained the required restrictions to view data of relevant tables or CDS views. However, I keep getting a message in the app that I am not authorized to view the data.

Answer: Inform your administrator that you do not have the authorization to view the specific tables or CDS views.

- The user with the `SAP_BR_ADMINISTRATOR_DANA` role is assigned to a single copy of the `SAP_BR_ADMINISTRATOR_DANA` role (either the original or a copy of original, we recommend creating a copy). You can check the assignment in the Maintain Business Users [page 3044] app.
- The *Write, Read, Value Help* restriction is maintained as **No Access**

To validate the values provided in the restrictions, see Data Visibility Based on Restrictions [page 1452].

Administrators

1. As an administrator, how can I maintain restrictions for the Customer Data Browser app?

Answer: For detailed information about restrictions, see Data Visibility Based on Restrictions [page 1452].

2. As an administrator, how can I identify the CDS view-specific authorizations so that the user can view the data of a specific CDS view?

Answer: In the SAP Business Accelerator Hub, choose *CDS Views* to see detailed information about published interface (I_) CDS views.

If you have any issues identifying the necessary CDS view-specific authorizations, report an incident under the application component relevant to the CDS view.

3 I have maintained the required restrictions for the business user to view data of relevant tables or CDS views. However, the business user keeps getting a message in the app that there is no authorization to view the data.

Answer

- Ensure that the user with the `SAP_BR_ADMINISTRATOR_DANA` role is assigned to a single copy of the `SAP_BR_ADMINISTRATOR_DANA` role (either the original or a copy of original, we recommend creating a copy). You can check the assignment in the Maintain Business Users [page 3044] app.
- Ensure that the *Write, Read, Value Help* restriction is maintained as **No Access**

To validate the values provided in the restrictions, see Data Visibility Based on Restrictions [page 1452].

4: How can I grant authorization to the business user to view the complete set of Customer Data Browser objects available with the app?

Answer: You can provide unrestricted access to the business user by choosing the *Unrestricted Access* buttons in the *Object Name* window.

Related Information

Customer Data Browser [page 1449]
Customer Data Browser Objects [page 1450]
Read Access Logging [page 1451]
Security Audit Log [page 1451]
Accessing the App [page 1455]
Data Visibility Based on Restrictions [page 1452]

Working wtih the App [page 1457]

4.2.15.9 Application Jobs

You can use this app to schedule and monitor application-related jobs. If you have manual activities that you often need to do at a specific time, the `Application Jobs` app can reduce your workload by running these tasks smoothly in the background. Planning regular jobs will keep you free to concentrate on other tasks.

The application job framework ideally defines long-running, repeated jobs. It's not recommended to use it to define short-running jobs with a high frequency. See also Technical Limits and Boundary Conditions for ABAP Applications.

Key Features

You can use this app to:

- Schedule jobs based on a job template: Select a job template, define the description, start date, start time and recurrence of the job, and enter job-specific selection criteria and parameters
- Work with personalized job templates: Create and personalize a job template, save personalized job template for later use, and share personalized template

  > **Note**
  >
  > The parameters under *Scheduling Information* can't be saved as part of a template.

- Schedule jobs with a custom factory calendar: In the `Application Jobs` app, define the scheduling information, change an existing calendar, create a new one, and define execution days
- Arrange job series that take different time zones into account
- Monitor jobs
- Display logs and results: You can navigate to job logs and job results by selecting the icon in the *Log* or the *Results* column
- Copy a job
- Cancel a job
- Restart job chains
- Navigate to the *Application Logs* reuse library. You can do this by selecting the corresponding icon in the log on the initial screen of the app.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component BC-SRV-APS-APJ.

Related Information

Application Logs [page 1498]
Working with Application Job Objects

4.2.15.9.1 Creating a Job Catalog Entry and a Job Template

Follow these steps to run your business coding in an application job.

Procedure

1. Implement the business logic. You need to create a class which implements certain interfaces so that it's usable in an application job. Implement the coding there that should run within the application job. For more information, see Implementing the Business Logic [page 1463].
2. Define the job catalog entry. Using ABAP development tools for Eclipse, you create a job catalog entry which refers to the class of step 1. In the job catalog entry, you can specify how the selection fields are rendered in the job scheduling dialog in the app. In addition, you can configure various settings which are used by the application job. For more information, see Creating the Job Catalog Entry [page 1470].
3. Define the job template. Using ABAP development tools for Eclipse, you create a job template which refers to the job catalog entry of step 2. In the job template, you define the default values of the job parameters which appear on the selection screen. For more information, see Creating the Job Template [page 1470].

   For more information on how to create a job catalog entry and a job template in ABAP development tools for Eclipse, see Working with Application Job Objects.

   If you want to implement a simple RAP (ABAP RESTful application programming model) business object, see Code Sample: How to Schedule Application Jobs from a RAP-based Business Object . This code sample allows you to schedule a class as an application job that takes the semantic key of the selected entity as a parameter.

Related Information

Setting up the Authorizations [page 1487]

4.2.15.9.1.1 Implementing the Business Logic

Business logic is ABAP coding that you want to be run within an application job.

Usually, business logic also defines the data type and some properties of the selection fields that you get on the selection screen before you schedule an application job. In this context, the business logic is implemented in an ABAP class. The following development steps require a user with the development role.

To implement the business logic, create an ABAP class that implements a certain interface:

- Create an ABAP class with interface `IF_APJ_RT_RUN`. Here, the selection fields are defined as public attributes of the class. See Creating an ABAP Class with Interface IF_APJ_RT_RUN [page 1463] for more information. It's recommended to use this interface for all new developments.
- You can still create an ABAP class with interfaces `IF_APJ_DT_EXEC_OBJECT` and `IF_APJ_RT_EXEC_OBJECT`. Here, the selection fields are defined in method `GET_PARAMETERS` of the interface `IF_APJ_DT_EXEC_OBJECT`. See Creating an ABAP Class with Interfaces IF_APJ_DT_EXEC_OBJECT and IF_APJ_RT_EXEC_OBJECT [page 1466] for more information.

  > ### Note
  >
  > Note that this is a legacy interface. Don't use it for new developments. If you use this interface, you only have limited options during the editing of the job catalog entry.

The ABAP class mentioned here is considered the main class of each application job. This class needs to be part of a customer-defined software component (not `ZLOCAL`) to be able to transport the class into subsequent systems.

Related Information

Creating the Job Catalog Entry [page 1470]
Creating the Job Template [page 1470]
Setting up the Authorizations [page 1487]
Application Logs [page 1498]

4.2.15.9.1.1.1 Creating an ABAP Class with Interface IF_APJ_RT_RUN

Learn how to create an ABAP class that implements the interface `IF_APJ_RT_RUN` to run the business logic of your application in an application job.

The interface `IF_APJ_RT_RUN` contains the following method:

- `EXECUTE`: This method is called by the application job framework when the scheduled application job is run. This means that the ABAP coding that should run within the application job is implemented here.

Before you schedule an application job, you get a selection screen where you can enter values for the application job parameters. These parameters are used to control the processing of the business logic. If

you put the business logic into an ABAP class with interface `IF_APJ_RT_RUN`, you can use the public attributes of the ABAP class as selection parameters. The parameters should fulfil the requirements described in Defining Class Attributes Used as Selection Parameters [page 1464]. Apart from the job catalog entry, the data type of the class attribute determines how the parameter is displayed and behaves on the selection screen.

Before the `EXECUTE` method is called by the application job framework during the job run, the class attributes are automatically filled with the values which were entered on the selection screen. The attributes can be used in the class coding without further preparations.

When you create a new application job template based on this class (see Creating the Job Template [page 1470]), its parameter values are prefilled with default values. To determine these default values, the class is instantiated and the values of the class attributes which have the same name as the template parameters are read. You have the following possibilities to set these default values:

- In the definition section of the class as default values of the class attributes
- In the `CONSTRUCTOR` of the class by setting values to the class attributes
- In addition, you can implement the interface `IF_APJ_DT_DEFAULTS` in the class and define default values in method `FILL_ATTRIBUTE_DEFAULTS` by setting values to the class attributes

> ### Note
>
> If the parameter is a date field, you can use a command like `@STARTDAYPLUSDAY@:03` in the value field of the template to define a dynamic date. This command can't be set as default value in method `FILL_ATTRIBUTE_DEFAULTS`, but you can enter it into the template after the template has been created.

4.2.15.9.1.1.1.1 Defining Class Attributes Used as Selection Parameters

Find out how to define class attributes that can be used as selection parameters to create your own business logic.

Before you schedule an application job, you get a selection screen where you can enter the values of the selection parameters. These values are transferred to the ABAP class before the `EXECUTE` method is run by the application job. To use a class attribute as selection parameter of the application job, it should fulfil the following requirements:

- The class attribute must be public
- Constants and read-only attributes can't be used
- The length of the attribute name is limited to 38 characters

  > ### Note
  >
  > If the attribute is part of an interface which is implemented in the class, and if the total length of the interface name and attribute name is longer than 38 characters, you may define an alias in the class and use the alias name in the job catalog entry

On the selection screen, fields either allow a single value, or multiple values or a value range to be entered. These fields have the following additional requirements:

- If the selection field allows the input of a single value, the corresponding class attribute must have an elementary data type.

> ### Note
>
> An elementary data type is a data type that contains a single value. Examples are built-in data types like C or I, or data elements which are defined in the data dictionary.
>
> The data types `STRING` and `XSTRING` are allowed. Mind that the application job framework only allows values with a maximum length of 255 characters. If a character field with a length such as 1024, or a string is used, and if you enter a value which is longer than 255 characters on the selection screen, the value is cut off.
>
> References and complex data types like structures or internal tables are not allowed because they can't be represented by a selection field with a single value.

- If the selection field allows the input of multiple values or a value range, the class attribute must be defined as ranges table.

  > ### Note
  >
  > A ranges table is an internal table with the fields `SIGN`, `OPTION`, `LOW`, and `HIGH`. These fields fulfil the following conditions:
  >
  > - `SIGN` is a character field of length 1
  > - `OPTION` is a character field of length 2
  > - `LOW` uses an elementary data type
  > - `HIGH` uses the same data type as `LOW`
  >
  > A ranges table can, for example, be defined via ABAP command `TYPE RANGES OF`.

The data type of the class attribute determines the data type and behavior of the selection parameter on the selection screen. An example: if the class attribute uses a date type, the selection field allows to enter a date, and it offers a value help for date selection.

The data type of the class attribute determines whether the selection field on the selection screen allows to enter a single value or multiple values or value ranges. If the data type is an elementary type, a single value can be entered. If it's a ranges table, multiple values or value ranges can be used.

When a parameter is added to a job catalog entry, the default value of the parameter title is taken from the description text of the class attribute. In ABAP development tools for Eclipse, the description text of the class attribute can be set using the ABAP Doc comment: `"! <p class="shorttext synchronized" lang="en">…</p>`

It's possible to define class attributes in the ABAP class which don't fulfil the above requirements if you don't want to use them as parameters of a job catalog entry.

4.2.15.9.1.1.2 Creating an ABAP Class with Interfaces IF_APJ_DT_EXEC_OBJECT and IF_APJ_RT_EXEC_OBJECT

Find out how to run business logic in an application job using an ABAP class that implements interfaces `IF_APJ_DT_EXEC_OBJECT` and `IF_APJ_RT_EXEC_OBJECT`.

> ⚠ Caution
>
> **Obsolete - Legacy Approach**
>
> These are legacy interfaces. If you use them, you have limited options when editing the application job catalog entry. Don't use these interfaces for new developments. Instead, use the interface `IF_APJ_RT_RUN`. We recommend that you adopt the new interface whenever appropriate. Enhancements and new features are implemented solely with the new interfaces. As a result, runtime APIs or UIs for application jobs built with the new interface provide better results and features compared to application jobs built with the old interfaces.
>
> Best practice → Creating an ABAP Class with Interface IF_APJ_RT_RUN [page 1463]

If you want to run the business coding of your application in an application job, you can create an ABAP class which implements the interfaces`IF_APJ_DT_EXEC_OBJECT` and `IF_APJ_RT_EXEC_OBJECT` and put the coding into this class. It's not recommended to use this possibility for new developments, because it offers limited functionality only.

The interface `IF_APJ_DT_EXEC_OBJECT` contains the following method:

- `GET_PARAMETERS`: This method is called by the application job framework to get all selection parameters. The method returns two internal tables:
  - `ET_PARAMETER_DEF`: Determines the parameter list which is used by the job catalog entry that refers to this main class. It also defines how the parameters are displayed and how they behave on the selection screen
  - `ET_PARAMETER_VAL`: Determines the default values for these parameters in a newly created job template that refers to the job catalog entry mentioned above

The structure of table `ET_PARAMETER_DEF` has the following fields:

Field Name	Definition
`SELNAME`	Name of the parameter. The length of the name is restricted to 8 characters. It defines the parameter name which is used in the job catalog entry and in the job template
`KIND`	Defines whether a single value or a value range can be entered on the selection screen for this parameter. Possible values are: • `IF_APJ_DT_EXEC_OBJECT=>PARAMETER` for a single value parameter • `IF_APJ_DT_EXEC_OBJECT=>SELECT_OPTION` for a value range parameter
`DATATYPE`	The data type of the parameter. It must be an elementary data type, like `C` or `CHAR`
`LENGTH`	The output length of the parameter if the data type has no fixed length
`DECIMALS`	Number of decimals if the data type belongs to a number with decimals

Field Name	Definition
COMPONENT_TYPE	Not supported
SECTION_TEXT	Not used
GROUP_TEXT	Not used
PARAM_TEXT	The title of the parameter which is displayed on the selection screen
LOWERCASE_IND	Defines whether lower case characters are allowed for the parameter
HIDDEN_IND	Defines whether the parameter is hidden on the selection screen
CHANGEABLE_IND	If set to ABAP_FALSE, the read-only flag is set for this parameter if the parameter is added to a catalog entry
MANDATORY_IND	Defines whether the parameter is mandatory on the selection screen
CHECKBOX_IND	If set, the parameter is displayed as checkbox on the selection screen
LIST_IND	If set, the parameter is displayed as list box on the selection screen
RADIO_GROUP_IND	If set, the parameter is displayed as radio button on the selection screen
RADIO_GROUP_ID	Name of the radio button group if the parameter is a radio button

The structure of the table ET_PARAMETER_VAL has the following fields:

Field Name	Definition
SELNAME	Name of the parameter. It's the same name that was used in table ET_PARAMETER_DEF
KIND	Defines whether a single value or a value range can be entered on the selection screen for this parameter. It should be the same value that was used in table ET_PARAMETER_DEF
SIGN, OPTION, LOW, HIGH	Entry of a ranges table which contains the parameter value, or a parameter value range

The interface IF_APJ_RT_EXEC_OBJECT contains the following method:

- EXECUTE: This method is called by the application job framework when the scheduled job is run. It receives the internal table IT_PARAMETERS as parameter. This internal table contains all parameter values that you entered on the selection screen when you scheduled the job.

The structure of table IT_PARAMETERS has the following fields:

Field Name	Definition
SELNAME	Name of the parameter. It's the same name that was used in method GET_PARAMETERS
KIND	If you've defined the parameter as a single value parameter in method GET_PARAMETERS, it's set to IF_APJ_DT_EXEC_OBJECT=>PARAMETER. Otherwise, it's set to IF_APJ_DT_EXEC_OBJECT=>SELECT_OPTION

Field Name	Definition
SIGN, OPTION, LOW, HIGH	Entry of a ranges table which contains the parameter value, or a parameter value range

> **Note**
>
> If you've selected several value ranges for a parameter on the selection screen, the table IT_PARAMETERS contains one entry for each value range.

4.2.15.9.1.1.3 Automatically Restarting an Application Job

The automatic restart of an application job ensures continuity by resuming interrupted processes without user intervention. This allows maintaining efficiency and reducing downtime in long-running jobs, especially during planned or unplanned system interruptions.

Context

The runtime of any process can be interrupted without programming errors being involved. Reasons are planned or unplanned failovers. In those cases, currently running transactions are rolled back and the session is ended. In particular, long-running application jobs can be affected by this. In such cases, in the past, the application job status was set to canceled by the application job runtime system. Afterwards, you had to find out the reason of the job cancellation and take appropriate action. However, when an application job stops running due to the reason mentioned above, there is no error in the job itself. There is now a new behavior for such situations.

Business Logic for Creating an Automatic Restart

If an application job stops running due to process interruption, the system doesn't set the application job status to canceled, but the job remains active. Another server resumes processing the application job as soon as possible, starting with the job step that was interrupted. If the application job has only one step, the job will run right from the beginning again.

Please consider the following when you implement the business logic:

- When an application job is automatically restarted, no new application job (no copy) is created.
- Within your coding that runs inside the application job, you can find out if the current execution was triggered by an automatic restart. You can use the method CL_APJ_RT_API=>GET_OWN_AUTO_RESTART_NUMBER. The method returns an integer. If it returns 0, the current execution is the original one. If it returns 1, the current execution was triggered by an automatic restart. Since an automatically restarted job can undergo the same procedure again, the method can return values greater than 1.

- Please implement your logic in a way that it's transactionally consistent and can be restarted at any time. Keep in mind that the system calls a rollback after the application job has stopped due to process interruption. Any database changes which your coding has not yet committed will be rolled back.
- The automatic restart applies for application jobs that have no recurrence, and for application jobs that have a recurrence period of 5 minutes or greater. If an application job has a small recurrence period of less than 5 minutes, it's assumed that an automatic restart is not necessary, because the successor job will start shortly and resume the work. That means that periodic application jobs with a recurrence of less than 5 minutes will be set to canceled in case of a process interruption.

Testing Job Behavior After an Automatic Restart

If you want to test if your application job behaves correctly after an automatic restart, please use the explicit restart functionality:

1. Start your application job
2. Cancel your application job using *Cancel* in the *Application Jobs* app or via the method `CL_APJ_RT_API=>CANCEL_JOB`.
3. Restart your application job using *Restart* (with the option *From Error Step*) in the *Application Jobs* app or via the method `CL_APJ_RT_API=>RESTART_JOB`.

There is a difference between the automatic restart and the explicit restart: when an application job is automatically restarted, no new job is created. The existing job is resumed from the beginning of the step that got interrupted. When you explicitly restart an application job, a copy of the original job is created and run from the beginning of the step that was canceled before. Essentially, both restart functionalities are the same, though: the application coding that's run within an application job is stopped, and afterwards starts from the beginning. To check if the current job execution was caused by an explicit restart, you can use the method `CL_APJ_RT_API=>IS_EXPLICIT_RESTART`.

4.2.15.9.1.1.4 Exception Handling

Learn how to handle the exception `CX_APJ_RT_CONTENT` of the method `EXECUTE`.

The method `EXECUTE` has the exception `CX_APJ_RT_CONTENT` in its signature. You can raise an exception of this type and specify a message text (see code snippet) . The application job framework handles exceptions of type `CX_APJ_RT_CONTENT` and creates a separate log for the application job. The log contains the message text, and information about the source position where the exception was raised. The application job gets the status *Failed*.

Sample Code

```
" the following coding simulates the application logic that causes a system
exception
" (in this case: division by zero)
DATA a TYPE i.
DATA b TYPE i.
DATA c TYPE i.
TRY.
   a = 1.
   b = 0.
```

```
   c = a / b.
   CATCH cx_root INTO DATA(exc).
" Precondition for the following statement:
" message class Z_APJ must exist, message 001 must exist in this message
class with one variable part

   RAISE EXCEPTION TYPE cx_apj_rt_content MESSAGE ID 'Z_APJ' NUMBER 001 WITH
exc->get_text(  ).
ENDTRY.
```

The application job framework doesn't handle any other exception apart from `CX_APJ_RT_CONTENT`. Any other exception which is not handled within the `EXECUTE` method itself will lead to a runtime error. The log will then contain the name of the runtime error, and further details of the runtime error have to be checked in the *ABAP Runtime Errors* app.

4.2.15.9.1.2 Creating the Job Catalog Entry

An application job catalog entry contains the information that is needed for scheduling and running an application job. The application job catalog entry contains the name of the class that implements the business logic, and the information on how selection fields are rendered in the job scheduling dialog in the app. In addition, the job catalog entry contains some further settings which are used during the scheduling and running of the application job.

To create a job catalog entry, use the ABAP development tools for Eclipse (ADT) editor. For more information, see Working with Application Job Objects. Alternatively, you could still also create job catalog entries and job templates via a released API. For more information, see Creating a Job Catalog Entry and a Job Template via API [page 1485].

Job catalog entries and job templates are created with a package assignment and the objects are assigned to a transport request. You need to make sure that the package and transport request fit together (regarding the transport layer) and that possible naming conventions are obeyed.

Related Information

Creating the Job Template [page 1470]
Setting up the Authorizations [page 1487]

4.2.15.9.1.3 Creating the Job Template

An *Application Job Template* refers to an application job catalog entry and contains values for some or all selection fields. It can be thought of as a variant of an application job catalog. In the simplest case, a job template doesn't contain any values for the selection fields. In this case, the selection fields are prefilled with their initial values (such as 0 if the selection field is of type integer). An application job catalog entry can have more than one application job template. The job template is the entity that can be scheduled in the *Application*

Jobs app. In the scheduling dialog, the selection fields appear and are prefilled with the values defined in the template. They can be overwritten by the user. Scheduling an application job template then results in an application job.

The creation of a job template follows the same technical rules as a *Job Catalog Entry* as described in Creating the Job Catalog Entry [page 1470]. To create a job template, use the ABAP development tools for Eclipse. For more information, see Creating Application Job Templates. Alternatively, you could still also create application job templates via a released API. For more information, see Creating a Job Catalog Entry and a Job Template via API [page 1485].

Related Information

Setting up the Authorizations [page 1487]

4.2.15.9.1.4 Example Implementation Using Interface IF_APJ_RT_RUN

This example demonstrates the implementation of the concept of application jobs based on a class with interface `IF_APJ_RT_RUN`.

To implement the business logic, the ABAP class will contain the following attributes that will also be visible on the selection screen when the application job is scheduled:

- The attribute `NUMBERS` which may contain value ranges
- The attribute `NAMES` which may contain multiple single values
- The attribute `TEXT` which may contain a single value only
- The boolean attributes `NUMBERS_AVAILABLE` and `NAMES_AVAILABLE`. The attribute `NUMBERS` will only be used if `NUMBERS_AVAILABLE` is set, while the attribute `NAMES` will only be used if `NAMES_AVAILABLE` is set.
- In addition, the class will contain the attribute `ANOTHER_PUBLIC_ATTRIBUTE` which will not be visible on the selection screen and is not filled by the application job framework

To implement these requirements, the ABAP class is created with the following definition part:

Sample Code

```
CLASS zcl_apj_demo_class DEFINITION PUBLIC FINAL CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_apj_rt_run.
    INTERFACES if_apj_dt_defaults.
    TYPES:
      BEGIN OF ty_name_range,
        sign   TYPE c LENGTH 1,
        option TYPE c LENGTH 2,
        low    TYPE c LENGTH 50,
        high   TYPE c LENGTH 50,
      END OF ty_name_range.
    TYPES: ty_name_ranges TYPE STANDARD TABLE OF ty_name_range WITH EMPTY KEY.
    "! <p class="shorttext synchronized" lang="en">Numbers available</p>
    DATA numbers_available TYPE abap_bool VALUE abap_true.
```

```
    "! <p class="shorttext synchronized" lang="en">Numbers</p>
    DATA numbers TYPE RANGE OF i.
    "! <p class="shorttext synchronized" lang="en">Names available</p>
    DATA names_available TYPE abap_bool VALUE abap_true.
    "! <p class="shorttext synchronized" lang="en">Names</p>
    DATA names TYPE ty_name_ranges.
    "! <p class="shorttext synchronized" lang="en">Some text</p>
    DATA text TYPE c LENGTH 255.
    "! <p class="shorttext synchronized" lang="en">Public attribute which is
not used on the selection screen</p>
    DATA another_public_attribute TYPE c LENGTH 10.
ENDCLASS.
```

The class definition contains the following parts:

- The class implements the interface `IF_APJ_RT_RUN`. This is necessary so the class can be used in an application job. In addition, it implements the interface `IF_APJ_DT_DEFAULTS` to set default values of new templates, which is optional
- It contains the following public attributes:
  - `NUMBERS` and `NAMES` are defined as ranges tables, because they may contain more than one value on the selection screen. The example implementation shows two possible ways how the ranges tables can be defined in the class
  - Because field `TEXT` should only contain a single value on the selection screen, it's defined with an elementary datatype in the class
  - The attributes `NUMBERS_AVAILABLE` and `NAMES_AVAILABLE` are defined as boolean type
  - Finally, there is attribute `ANOTHER_PUBLIC_ATTRIBUTE` which may have any data type because it will not be visible on the selection screen
- All attributes have description texts which are defined via ABAP Doc comments

In the implementation part of the class, the method `IF_APJ_RT_RUN~EXECUTE` must be implemented. This method is called by the application job framework when the application job is started. The method should contain the business logic that should be run by the application job.

Sample Code

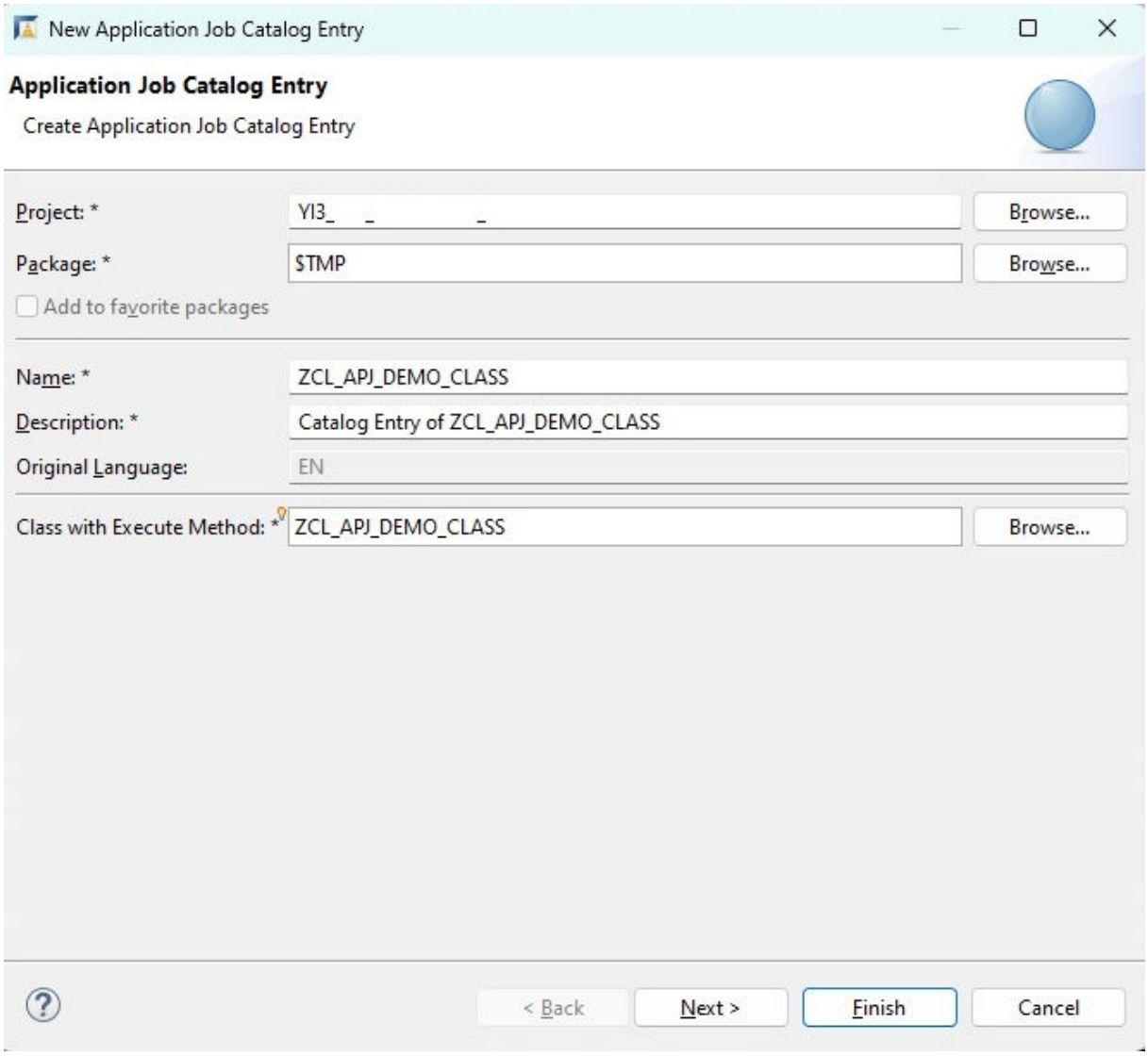
```
METHOD if_apj_rt_run~execute.
    TRY.
        DATA(l_log) = cl_bali_log=>create_with_header(
                        header = cl_bali_header_setter=>create( object =
'ZOBJECT'
                                                               subobject =
'ZSUBOBJECT' ) ).
        IF numbers_available = abap_true AND '42' IN numbers.
          l_log->add_item( item = cl_bali_free_text_setter=>create( severity
= if_bali_constants=>c_severity_information
                                                                  text =
'42 is in the number ranges' ) ).
        ENDIF.
        IF names_available = abap_true AND names IS NOT INITIAL.
          l_log->add_item( item = cl_bali_free_text_setter=>create( severity
= if_bali_constants=>c_severity_status
                                                                  text =
'Some names are available' ) ).
        ENDIF.
        l_log->add_item( item = cl_bali_free_text_setter=>create( severity =
if_bali_constants=>c_severity_status
                                                                text = CONV
#( text ) ) ).
```

```
        cl_bali_log_db=>get_instance( )->save_log_2nd_db_connection( log =
l_log

assign_to_current_appl_job = abap_true ).
      CATCH cx_bali_runtime INTO DATA(l_runtime_exception).
        " some error handling
    ENDTRY.
  ENDMETHOD.
```

In this example, the `EXECUTE` method creates an application log and assigns it to the current application job when the job is run. If the number 42 is in the ranges of field `NUMBERS`, a message is written into the log. Another message is written if field `NAMES` contains any entry. Finally, the content of field `TEXT` is put into the log. Attributes like `NUMBERS`, `NAMES` and `TEXT` can be used without any preparation, because they are automatically filled by the application job framework before the `EXECUTE` method is started.

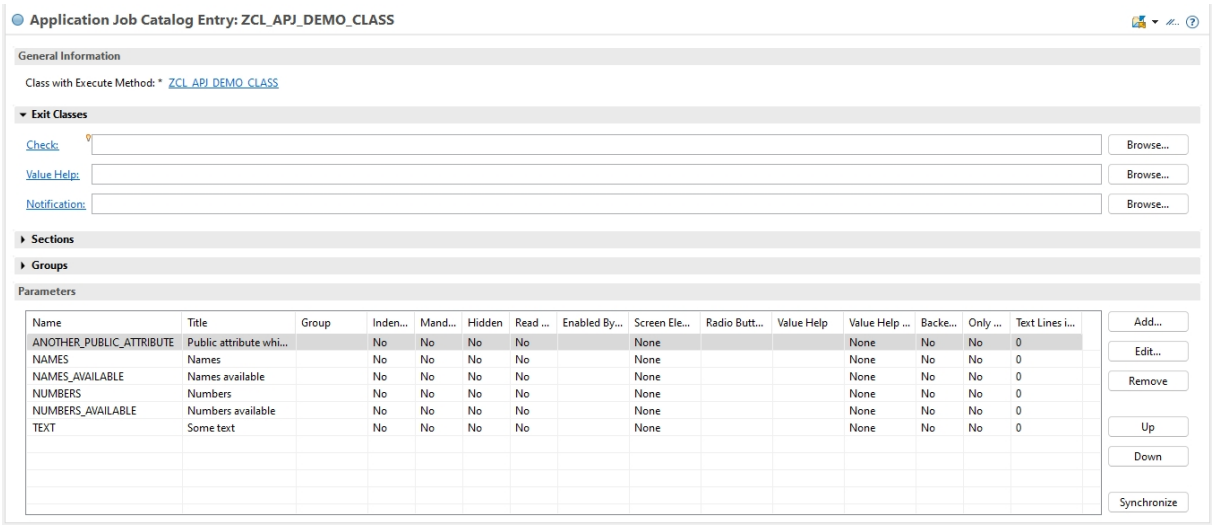
After the class was implemented, a new catalog entry should be created for this class so that it can be used in an application job. Besides other settings, the catalog entry defines which parameters will be visible on the selection screen (which is visible when you schedule the job) and where and how these parameters are displayed.

A new catalog entry can be created in the ABAP development tools for Eclipse. It can be found under *Other ABAP Repository Object → Application Jobs → Application Job Catalog Entry*.



On the first screen, you select the name and a description text of the new application job catalog entry and the name of the class which is run within the job. It's the class which you just defined.

After you have created the application job catalog entry, you can start editing it:



In this demo implementation, the table with the parameters was automatically prefilled with all public attributes of the ABAP class which fulfil the requirements to be used in a catalog entry. The titles of the parameters were automatically filled with the description texts of the class attributes (which were defined via the ABAP Doc comments in the definition part of the class).

As a next step, you can change the properties of each parameter and define what the selection screen should look like. In this example, we will meet the following requirements:

- The parameters `NUMBERS_AVAILABLE` and `NUMBERS` will be grouped. The group will get a header text. `NUMBERS_AVAILABLE` will be a checkbox, and it will only be possible to enter values into `NUMBERS` if `NUMBERS_AVAILABLE` is set.
- The parameters `NAMES_AVAILABLE` and `NAMES` will be grouped. The group will get a header text. `NAMES_AVAILABLE` will be a checkbox, and it will only be possible to enter values into `NAMES` if `NAMES_AVAILABLE` is set.
- Both groups will be displayed side by side in the same section of the screen (`NUMBERS` on the left side and `NAMES` on the right side). This section will get a header text, too.
- The parameter `TEXT` will get its own group (with group header) which is part of its own section (with section header).
- The parameter `ANOTHER_PUBLIC_ATTRIBUTE` is not relevant here and will not be displayed.

To implement these settings, you first define the sections of the selection screen:

There are two sections carrying the section header texts *Section with Checkboxes* and *Section with Text Field*. Because section `SECTION_1` is above section `SECTION_2` in this table, section `SECTION_1` is displayed above section `SECTION_2` on the selection screen.

Next, you define the groups:

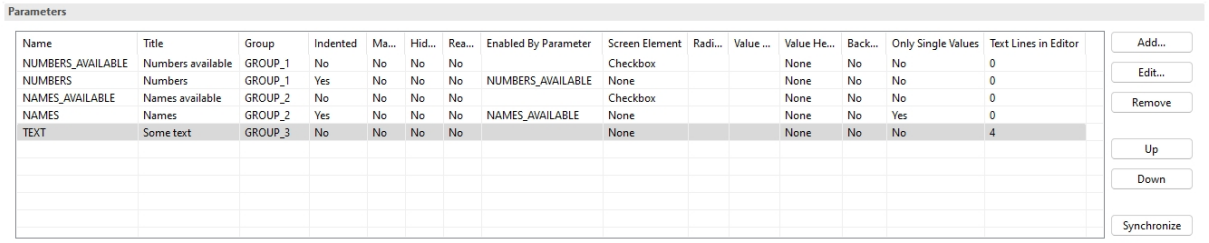
Groups

Name	Title	Section
GROUP_1	Group with Numbers	SECTION_1
GROUP_2	Group with Names	SECTION_1
GROUP_3	Group with Text Field	SECTION_2

Add... Edit... Remove Up Down

Three groups were defined: The groups with title texts *Group with Numbers* and *Group with Names* were assigned to the first section, while the group with title text *Group with Text Field* was assigned to the second section. Because group `GROUP_1` is above group `GROUP_2` in this table, group `GROUP_1` is on the left side of the selection screen, while group `GROUP_2` is on the right side.

Finally, you can adapt the properties of the parameters according to the requirements:



Parameters

Name	Title	Group	Indented	Ma...	Hid...	Rea...	Enabled By Parameter	Screen Element	Radi...	Value ...	Value He...	Back...	Only Single Values	Text Lines in Editor
NUMBERS_AVAILABLE	Numbers available	GROUP_1	No	No	No	No		Checkbox			None	No	No	0
NUMBERS	Numbers	GROUP_1	Yes	No	No	No	NUMBERS_AVAILABLE	None			None	No	No	0
NAMES_AVAILABLE	Names available	GROUP_2	No	No	No	No		Checkbox			None	No	No	0
NAMES	Names	GROUP_2	Yes	No	No	No	NAMES_AVAILABLE	None			None	No	Yes	0
TEXT	Some text	GROUP_3	No	No	No	No		None			None	No	No	4

Add... Edit... Remove Up Down Synchronize

The following parameter properties were changed in this example:

- The parameters `NUMBERS_AVAILABLE` and `NUMBERS` were put into the first group of the first section (`Group = GROUP_1`). Because parameter `NUMBERS_AVAILABLE` is above parameter `NUMBERS` in this table, it's displayed above parameter `NUMBERS` on the selection screen. Parameter `NUMBERS` is output-indented on the selection screen (`Indented = Yes`). In addition, it's only active on the screen if the parameter `NUMBERS_AVAILABLE` is set (`Enabled By Parameter = NUMBERS_AVAILABLE`). Parameter `NUMBERS_AVAILABLE` is displayed as checkbox on the selection screen (`Screen Element = Checkbox`).
- The parameters `NAMES_AVAILABLE` and `NAMES` were put into another group (`Group = GROUP_2`). The parameter `NAMES` is also output-indented (`Indented = yes`). In addition, it's only active on the selection screen if the parameter `NAMES_AVAILABLE` is set (`Enabled By Parameter = NAMES_AVAILABLE`). Parameter `NAMES_AVAILABLE` is displayed as checkbox on the selection screen (`Screen Element = Checkbox`). Parameter `NAMES` only allows to enter multiple single values, but no value ranges (`Only Single Values = Yes`).
- The parameter `TEXT` is displayed in its own group (`Group = GROUP_3`). It's displayed as text editor with four lines (`Text Lines in Editor = 4`).
- The parameter `ANOTHER_PUBLIC_ATTRIBUTE` is not relevant for the selection screen. Therefore, it was removed from the parameter list.

After you finished working on the catalog entry, don't forget to activate it.

After the application job catalog entry has been defined, you need to create an application job template. The template contains the parameter values that are displayed as default values on the selection screen. You can change these values before you schedule an application job. If you create a new template for your ABAP class, you may want that the parameters of the new template are already prefilled with default values. To determine these default values, the application job framework instantiates your class and calls the method `IF_APJ_DT_DEFAULTS~FILL_ATTRIBUTE_DEFAULTS`. It then reads the values of the class attributes and uses them as default values of the template parameters. These default values can either

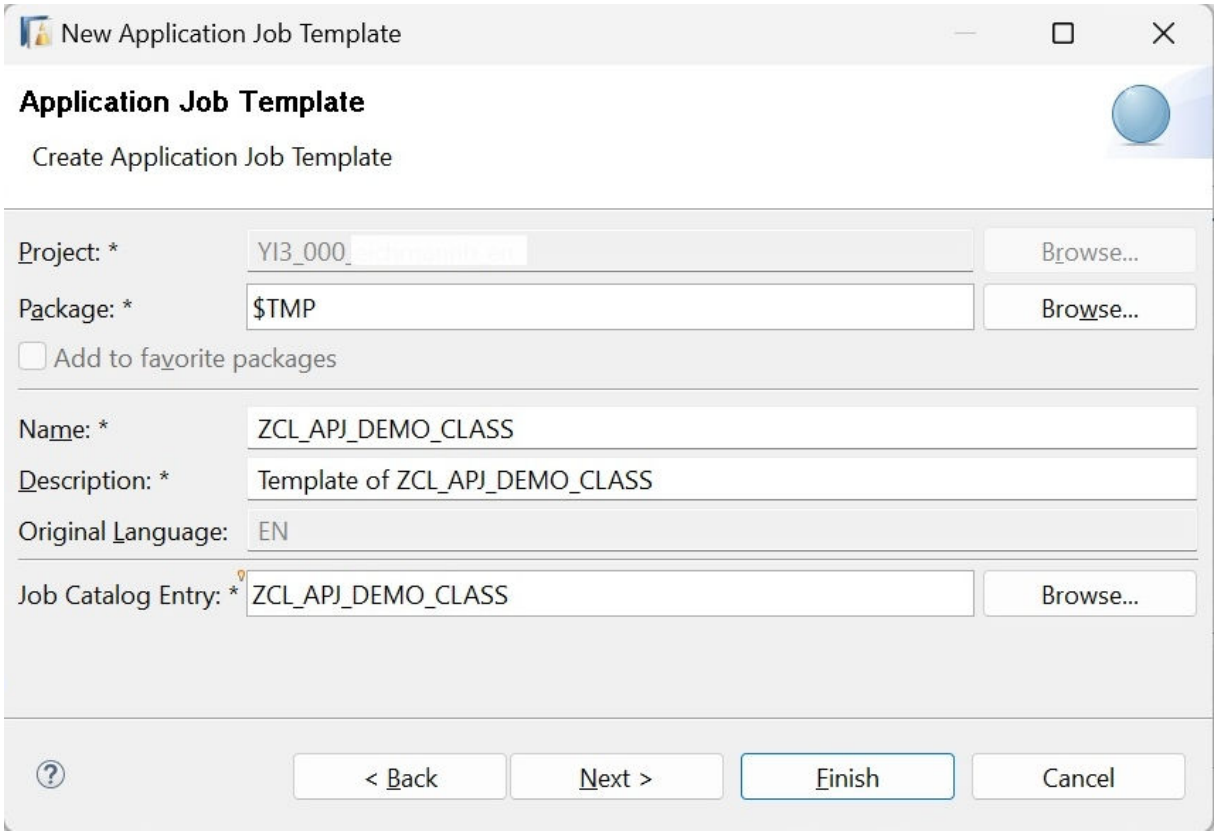
be set via the default values of the class attributes, via the `CONSTRUCTOR` of the class, or, if a more complex coding is required to determine these default values, you can also set them via the method `IF_APJ_DT_DEFAULTS~FILL_ATTRIBUTE_DEFAULTS`:

Sample Code

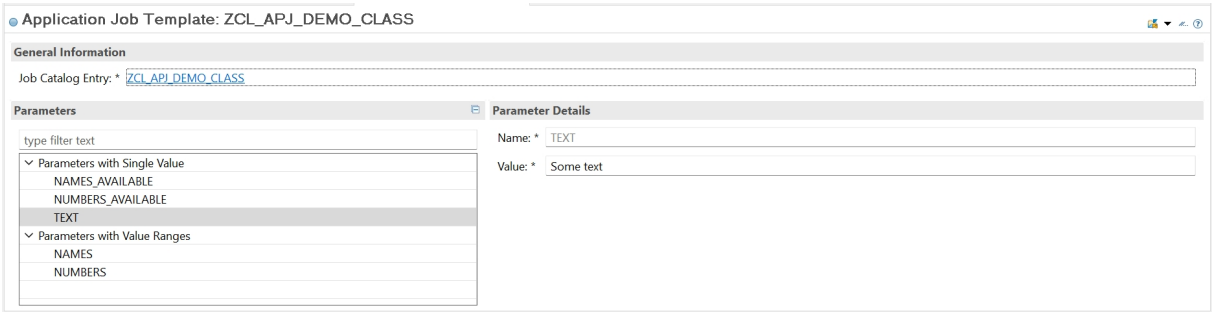
```
METHOD if_apj_dt_defaults~fill_attribute_defaults.
    numbers = VALUE #( ( sign = 'I' option = 'BT' low = '1' high = '10' )
                       ( sign = 'I' option = 'BT' low = '91' high = '100' ) ).
    text = 'Some text'.
  ENDMETHOD.
```

Here, the default values of the attributes `NUMBERS` (which is a ranges table) and `TEXT` are set.

To create the new application job template, you can use the ABAP development tools for Eclipse. The template can be found under *Other ABAP Repository Object → Application Jobs → Application Job Template*.



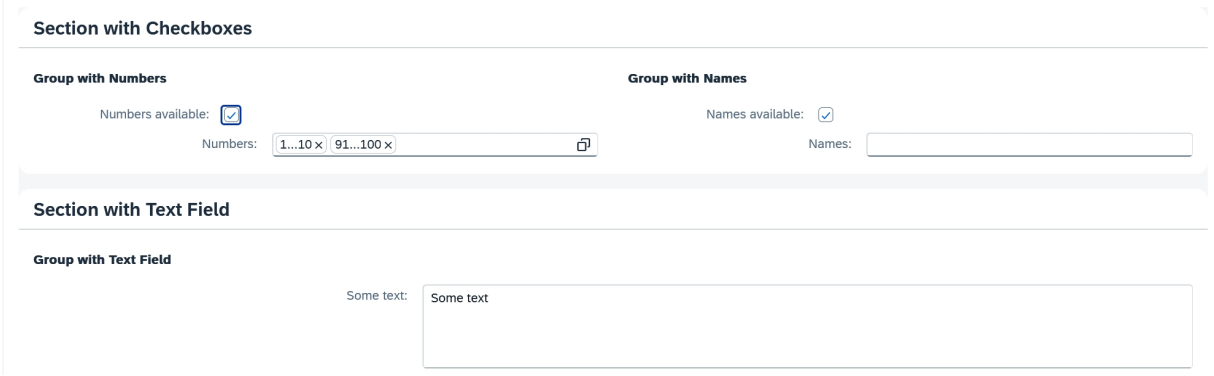
On the first screen, you select the name and a description text of the new template and the name of the catalog entry on which the template is based. It's the catalog entry which you just created.



After you've created the application job template, the parameters of the template are prefilled with the values which you set in the ABAP class. In this example, the following parameters got a default value:

- `NUMBERS_AVAILABLE` and `NAMES_AVAILABLE` are both set to `X` because these default values are set in the definition part of the class.
- `NUMBERS` is set to the ranges 1 - 10 and 91 - 100 and `TEXT` is set to the text `Some text` because of the implementation of method `FILL_ATTRIBUTE_DEFAULTS`.

Now, you can change the parameter values if you want. After you've finished this, don't forget to activate the template. With the new template, you can start to schedule an application job. If you do this, you get the following selection screen which meets the requirements set before:



4.2.15.9.1.4.1 Example Implementation Using Interfaces IF_APJ_DT_EXEC_OBJECT and IF_APJ_RT_EXEC_OBJECT

This example demonstrates the implementation of the concept of application jobs based on a class with the legacy interfaces `IF_APJ_DT_EXEC_OBJECT` and `IF_APJ_RT_EXEC_OBJECT`.

> **Note**
>
> Note that these are legacy interfaces. If you use these interfaces, you only have limited options during the editing of the job catalog entry. Don't use these interfaces for new developments, but use the interface `IF_APJ_RT_RUN` instead. For more information, see Example Implementation Using Interface IF_APJ_RT_RUN [page 1471].

To implement the concept of application jobs, the ABAP class will contain the following attributes that will also be visible on the selection screen when the application job is scheduled:

- The attributes `NUMBERS` and `NAMES` which may contain value ranges
- The attribute `TEXT` which may contain a single value only
- The boolean attributes `NUMBERS_AVAILABLE` and `NAMES_AVAILABLE`. Attribute `NUMBERS` will only be used if `NUMBERS_AVAILABLE` is set, while attribute `NAMES` will only be used if `NAMES_AVAILABLE` is set.
- The parameter which belongs to `NUMBERS_AVAILABLE` will be a checkbox.
- The parameter which belongs to `NAMES_AVAILABLE` will be a checkbox.

Only limited options are available when interfaces `IF_APJ_DT_EXEC_OBJECT` and `IF_APJ_RT_EXEC_OBJECT` are used. For example, it is not possible to use the following features:

- Grouping of parameters on the selection screen, or putting them into sections.
- Controling the enabled/disabled status of a parameter on the selection screen with another parameter.
- Restricting a field with value ranges to multiple single value, or setting the size of the text editor.

To implement these requirements, the ABAP class is created with the following definition part:

Sample Code

```
 CLASS zcl_apj_demo_class2 DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION.
    INTERFACES if_apj_rt_exec_object.
    INTERFACES if_apj_dt_exec_object.

    TYPES:
      BEGIN OF ty_name_range,
        sign   TYPE c LENGTH 1,
        option TYPE c LENGTH 2,
        low    TYPE c LENGTH 50,
        high   TYPE c LENGTH 50,
      END OF ty_name_range.
    TYPES: ty_name_ranges TYPE STANDARD TABLE OF ty_name_range WITH EMPTY KEY.
    "! <p class="shorttext synchronized" lang="en">Numbers available</p>
    DATA numbers_available TYPE abap_bool VALUE abap_true.
    "! <p class="shorttext synchronized" lang="en">Numbers</p>
    DATA numbers TYPE RANGE OF i.
    "! <p class="shorttext synchronized" lang="en">Names available</p>
    DATA names_available TYPE abap_bool VALUE abap_true.
    "! <p class="shorttext synchronized" lang="en">Names</p>
    DATA names TYPE ty_name_ranges.
    "! <p class="shorttext synchronized" lang="en">Some text</p>
    DATA text TYPE c LENGTH 255.
    "! <p class="shorttext synchronized" lang="en">Public attribute which is
not used on the selection screen</p>
    DATA another_public_attribute TYPE c LENGTH 10.
ENDCLASS.
```

The class definition contains the following parts:

- The class implements the interface `IF_APJ_RT_EXEC_OBJECT` and `IF_APJ_DT_EXEC_OBJECT`. This is necessary so that the class can be used in an application job.
- The class definition defines specific public attributes according to the requirements that will be set, these attributes are used by the logic of the class (or coding).

In the next step it is necessary to define the technical names and properties of all parameters that are used by all catalog entries and templates which are based on this class. To do this, the method `GET_PARAMETERS` of interface `IF_APJ_DT_EXEC_OBJECT` must be implemented in the implementation part of the class:

Sample Code

```
METHOD if_apj_dt_exec_object~get_parameters.
    et_parameter_def = VALUE #(
      ( selname = 'NUM_AVAI' kind = if_apj_dt_exec_object=>parameter
datatype = 'C' length = 1   param_text = 'Numbers available' changeable_ind
= abap_true checkbox_ind = abap_true )
      ( selname = 'NUMBERS'  kind = if_apj_dt_exec_object=>select_option
datatype = 'I'              param_text = 'Numbers'           changeable_ind =
abap_true )
      ( selname = 'NAM_AVAI' kind = if_apj_dt_exec_object=>parameter
datatype = 'C' length = 1   param_text = 'Names available'   changeable_ind
= abap_true checkbox_ind = abap_true )
      ( selname = 'NAMES'    kind = if_apj_dt_exec_object=>select_option
datatype = 'C' length = 50  param_text = 'Names'             changeable_ind
= abap_true )
      ( selname = 'TEXT'     kind = if_apj_dt_exec_object=>parameter
datatype = 'C' length = 255 param_text = 'Some text'         changeable_ind
= abap_true )
    ).
  ENDMETHOD.
```

In the method, you need to fill out an internal table with the list of all parameters to be used by the application job framework. The technical name of the parameters is `SELNAME`. Please notice:

- The length of the technical name is limited to 8 characters.
- The properties `KIND`, `DATATYPE`, and `LENGTH` should be defined according to the data type of the class attribute to which the parameter belongs.

The following parameter properties were set in this example:

- The titles of the parameters on the selection screen were set by property `PARAM_TEXT`.
- The parameters `NUM_AVAI` and `NAM_AVAI` are displayed as a checkbox on the selection screen (`checkbox_ind = abap_true`).
- If any of the parameters are added to a catalog entry, the read-only property is set to false (`changeable_ind = abap_true`).
- The sequence of the parameters in table `ET_PARAMETER_DEF` defines the sequence of the parameters on the selection screen. For example, the parameter `NUM_AVAI` is displayed above parameter `NUMBERS`.

In addition, in the implementation part of the class the method `IF_APJ_RT_EXEC_OBJECT~EXECUTE` must be implemented. This method is called by the application log framework when the application job is started. So, it should contain the business coding which will be run by the application job.

Sample Code

```
METHOD if_apj_rt_exec_object~execute.
    LOOP AT it_parameters INTO DATA(l_parameter).
      CASE l_parameter-selname.
        WHEN 'NUM_AVAI'. numbers_available = l_parameter-low.
        WHEN 'NUMBERS'.
          APPEND VALUE #( sign   = l_parameter-sign
                          option = l_parameter-option
                          low    = l_parameter-low
                          high   = l_parameter-high ) TO numbers.
        WHEN 'NAM_AVAI'. names_available = l_parameter-low.
```

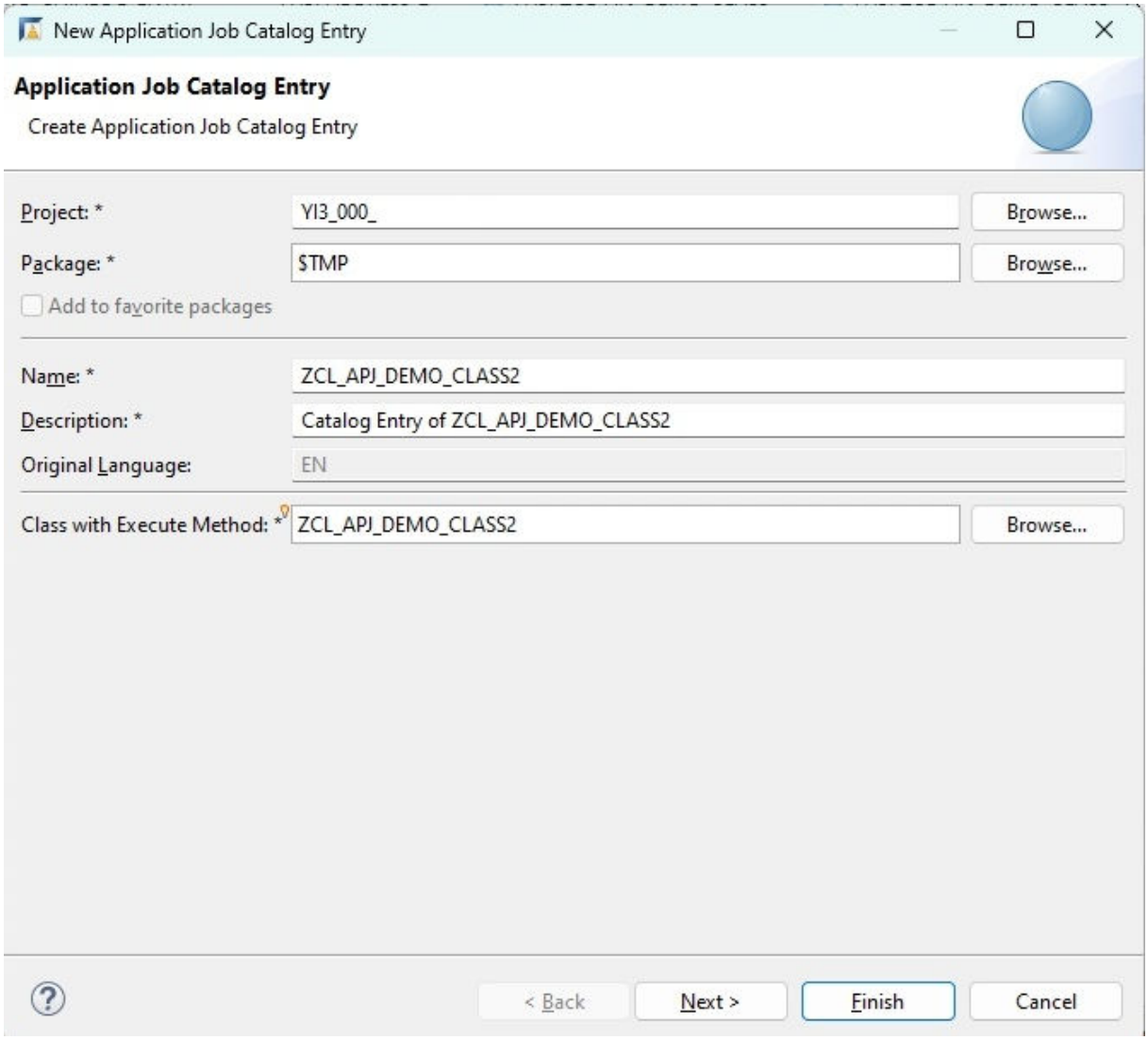
```
        WHEN 'NAMES'.
          APPEND VALUE #( sign   = l_parameter-sign
                          option = l_parameter-option
                          low    = l_parameter-low
                          high   = l_parameter-high ) TO names.
        WHEN 'TEXT'. text = l_parameter-low.
      ENDCASE.
    ENDLOOP.
    TRY.
        DATA(l_log) = cl_bali_log=>create_with_header(
                        header = cl_bali_header_setter=>create( object =
'ZOBJECT'
                                                               subobject =
'ZSUBOBJECT' ) ).
        IF numbers_available = abap_true AND '42' IN numbers.
          l_log->add_item( item = cl_bali_free_text_setter=>create( severity
= if_bali_constants=>c_severity_information
                                                                  text =
'42 is in the number ranges' ) ).
        ENDIF.
        IF names_available = abap_true AND names IS NOT INITIAL.
          l_log->add_item( item = cl_bali_free_text_setter=>create( severity
= if_bali_constants=>c_severity_status
                                                                  text =
'Some names are available' ) ).
        ENDIF.
        l_log->add_item( item = cl_bali_free_text_setter=>create( severity =
if_bali_constants=>c_severity_status
                                                                text = CONV
#( text ) ) ).
        cl_bali_log_db=>get_instance( )->save_log_2nd_db_connection( log =
l_log

assign_to_current_appl_job = abap_true ).
      CATCH cx_bali_runtime INTO DATA(l_runtime_exception).
        " some error handling
    ENDTRY.
  ENDMETHOD.
```

The values of the parameters that were set on the selection screen are available in the internal table `IT_PARAMETERS`. First, the table entries are read and the attributes of the class are filled in with the values. In this example, the global attributes of the class are filled in, you may also use local attributes of method `EXECUTE` to store the values. The field `SELNAME` of table `IT_PARAMETERS` contains the same technical names that you set in method `GET_PARAMETERS`.

In this simple example, the method `EXECUTE`creates an application log and assigns it to the current application job when the job is run. If the number 42 is in the ranges of field `NUMBERS`, a message is written into the log. Another message is written, if the field `NAMES` contains any entries. Finally, the content of the field `TEXT` is put into the log.

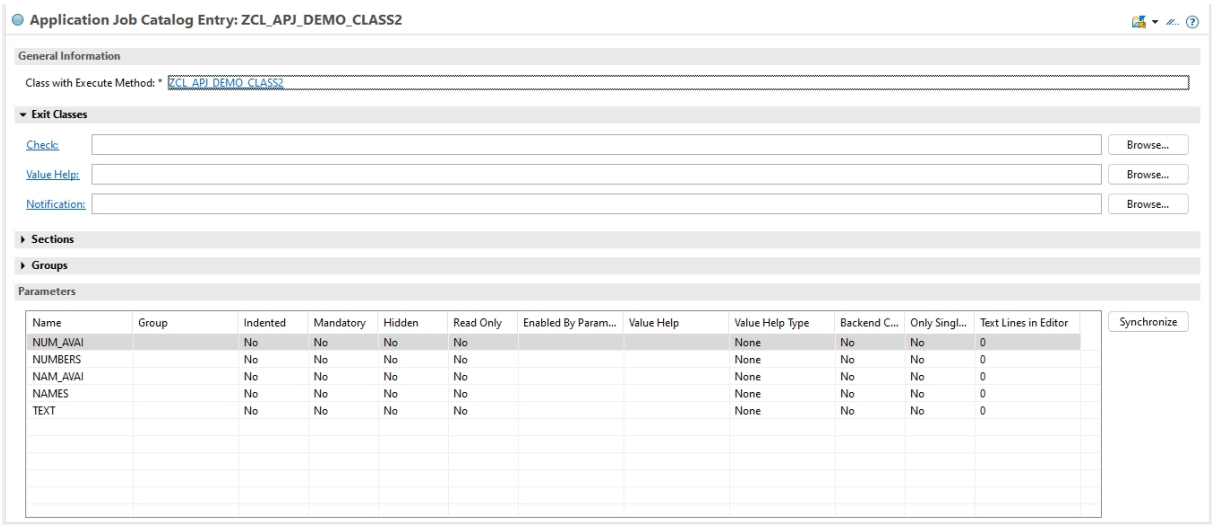
After you implemented the class, you should create a new catalog entry with ABAP development tools for Eclipse for this class so it can be used in an application job.

To do this, go to *Other ABAP Repository Object → Application Jobs → Application Job Template*.



On the first screen you select the name and a description text of the new catalog entry and the name of the class which is run within the job. It shows the class that you just previously implemented.

After you have created the catalog entry, you can start to change it:



Here, you can see that the parameter table was filled with all parameters which were defined in method `GET_PARAMETERS` automatically. Please note that it is not possible to change the parameter properties here if the parameter properties were defined in the class via interface `IF_APJ_DT_EXEC_OBJECT`.

In the next step, you can optionally set `Exit` classes. After finishing your changes of the catalog entry, don't forget to activate it.

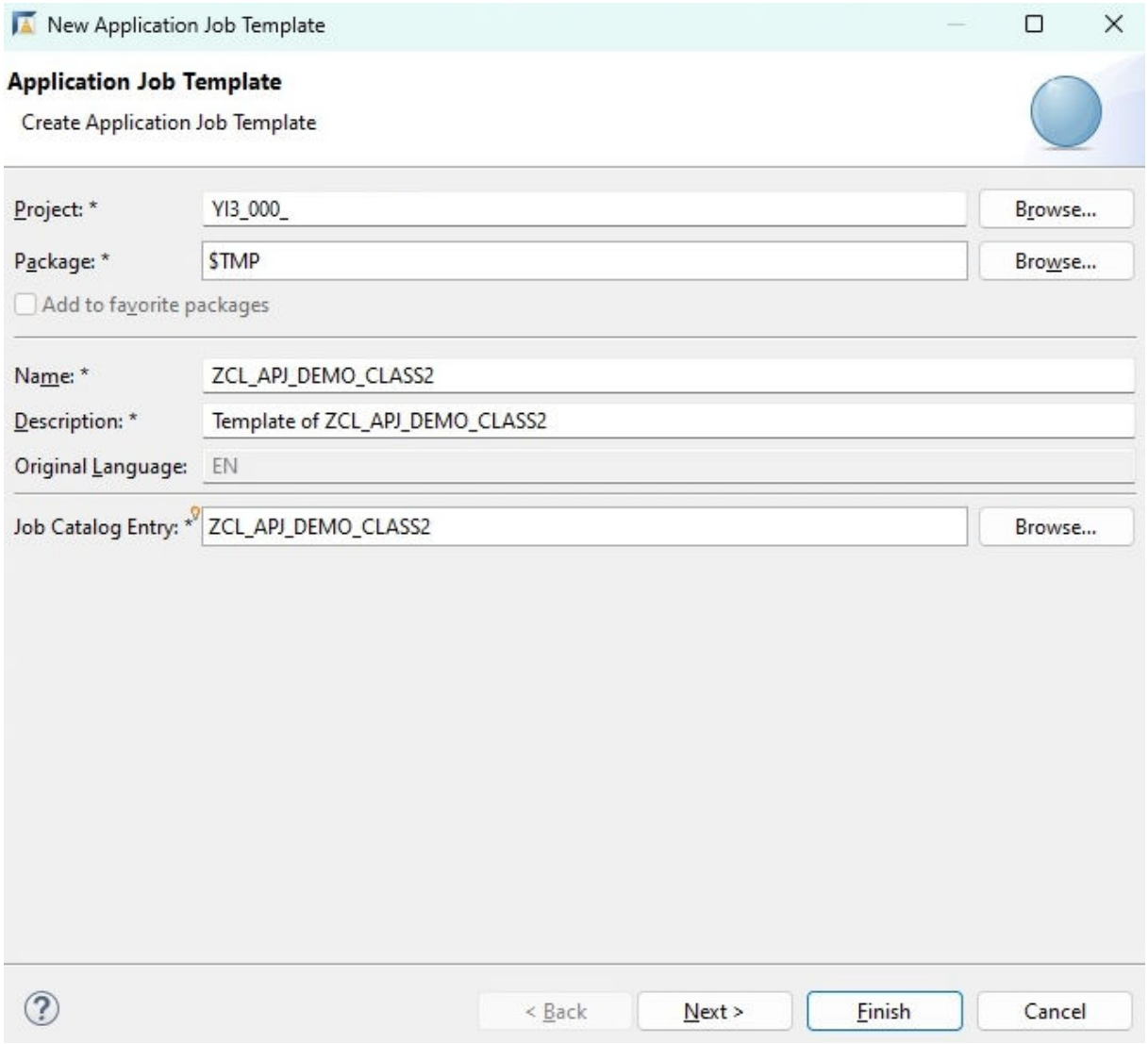
After the catalog entry has been defined, the next step is the creation of a template. The template contains the parameter values which are displayed as default values on the selection screen. As a user you may change these values before scheduling the application job. If you create a new template for your ABAP class, you might want the parameters of it to be pre-filled with default values already. You can set these default values in your ABAP class by filling out the internal table `ET_PARAMETER_VAL` in method `GET_PARAMETERS`:

Sample Code

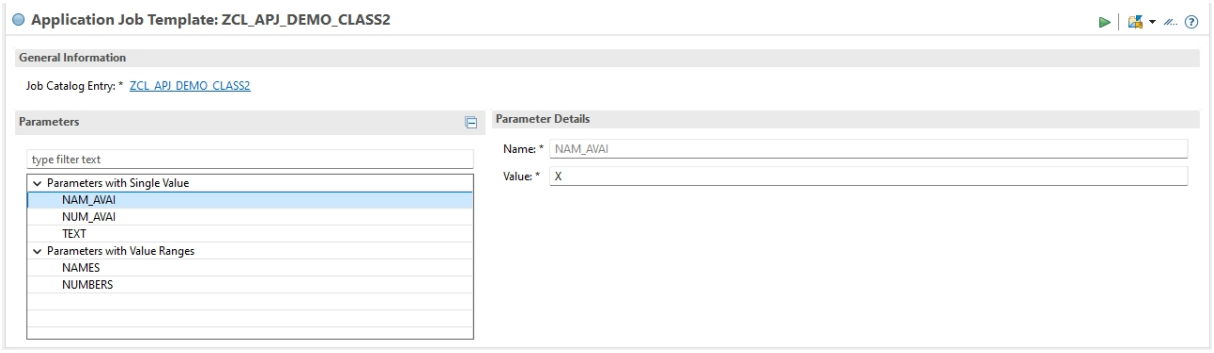
```
METHOD if_apj_dt_exec_object~get_parameters.
 ...
    et_parameter_val = VALUE #(
      ( selname = 'NUM_AVAI' kind = if_apj_dt_exec_object=>parameter     sign
= 'I' option = 'EQ' low = 'X' )
      ( selname = 'NUMBERS'  kind = if_apj_dt_exec_object=>select_option sign
= 'I' option = 'BT' low = '1' high = '10' )
      ( selname = 'NUMBERS'  kind = if_apj_dt_exec_object=>select_option sign
= 'I' option = 'BT' low = '91' high = '100' )
      ( selname = 'NAM_AVAI' kind = if_apj_dt_exec_object=>parameter     sign
= 'I' option = 'EQ' low = 'X' )
      ( selname = 'TEXT'     kind = if_apj_dt_exec_object=>parameter     sign
= 'I' option = 'EQ' low = 'Some Text' )
    ).
  ENDMETHOD.
```

Here, the default values of the attributes `NUMBERS` (which is a ranges table) and `TEXT` are set.

To create the new application job template, you can use the ABAP development tools for Eclipse. The template can be found under *Other ABAP Repository Object → Application Jobs → Application Job Template*.

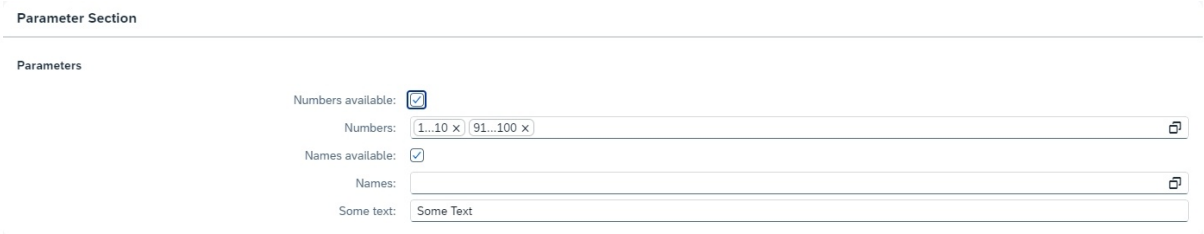


On the first screen, you select the name and a description text of the new template and the name of the catalog entry on which the template is based. It's the catalog entry which you just created.



After you've created the application job template, the parameters of the template are prefilled with the values which you set in method `GET_PARAMETERS` of the ABAP class.

Now, you can change the parameter values if you want. After you've finished this, don't forget to activate the template. With the new template, you can start to schedule an application job. If you do this, you get the following selection screen which meets the requirements set before:



4.2.15.9.1.5 Creating a Job Catalog Entry and a Job Template via API

Find out how to create job catalog entries and job templates using a released API.

If you can't use the ABAP development tools for Eclipse to create job catalog entries and job templates, or of you have the requirement to create these objects via an ABAP class, you can use a released API. To use this API, you can use the ABAP class `CL_APJ_DT_CREATE_CONTENT`. The class offers the following methods:

- `GET_INSTANCE` (static): Get an instance of class `CL_APJ_DT_CREATE_CONTENT`
- `CREATE_JOB_CAT_ENTRY`: Create an application job catalog entry
- `CREATE_JOB_TEMPLATE_ENTRY`: Create an application job template
- `DELETE_JOB_CAT_ENTRY`: Delete an application job catalog entry
- `DELETE_JOB_TEMPLATE_ENTRY`: Delete an application job template
- `EXISTS_JOB_CAT_ENTRY`: Check the existence of an application job catalog entry
- `EXISTS_JOB_TEMPLATE_ENTRY`: Check the existence of an application job template

The following code sample generates one job catalog entry and one related job template:

Sample Code

```
CLASS zcl_test_apj_simple_obj_gen DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_test_apj_simple_obj_gen IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    CONSTANTS lc_catalog_name      TYPE
cl_apj_dt_create_content=>ty_catalog_name  VALUE 'ZTEST_MY_SIMPLE_JOB'.
    CONSTANTS lc_catalog_text      TYPE
cl_apj_dt_create_content=>ty_text          VALUE 'My first simple application
job'.
    CONSTANTS lc_class_name        TYPE
cl_apj_dt_create_content=>ty_class_name    VALUE 'ZCL_TEST_APJ_SIMPLE'.
    CONSTANTS lc_template_name     TYPE
cl_apj_dt_create_content=>ty_template_name VALUE 'ZTEST_MY_SIMPLE_JOB_TEMPL'.
```

```
    CONSTANTS lc_template_text     TYPE
cl_apj_dt_create_content=>ty_text          VALUE 'My first simple job
template'.
    CONSTANTS lc_transport_request TYPE
cl_apj_dt_create_content=>ty_transport_request VALUE 'Y11K900361'.
    CONSTANTS lc_package           TYPE
cl_apj_dt_create_content=>ty_package           VALUE 'Z_D028092_APJ_MAIN'.
    DATA(lo_dt) = cl_apj_dt_create_content=>get_instance( ).
    " Create job catalog entry (corresponds to the former report incl.
selection parameters)
    " Provided implementation class iv_class_name shall implement two
interfaces:
    " - if_apj_dt_exec_object to provide the definition of all supported
selection parameters of the job
    "   (corresponds to the former report selection parameters) and to
provide the actual default values
    " - if_apj_rt_exec_object to implement the job execution
    TRY.
        lo_dt->create_job_cat_entry(
            iv_catalog_name       = lc_catalog_name
            iv_class_name         = lc_class_name
            iv_text               = lc_catalog_text
            iv_catalog_entry_type = cl_apj_dt_create_content=>class_based
            iv_transport_request  = lc_transport_request
            iv_package            = lc_package
        ).
        out->write( |Job catalog entry created successfully| ).
      CATCH cx_apj_dt_content INTO DATA(lx_apj_dt_content).
        out->write( |Creation of job catalog entry failed:
{ lx_apj_dt_content->get_text( ) }| ).
    ENDTRY.
    " Create job template (corresponds to the former system selection
variant) which is mandatory
    " to select the job later on in the Fiori app to schedule the job
    DATA lt_parameters TYPE if_apj_dt_exec_object=>tt_templ_val.
    NEW zcl_test_apj_simple( )->if_apj_dt_exec_object~get_parameters(
      IMPORTING
        et_parameter_val = lt_parameters
    ).
    TRY.
        lo_dt->create_job_template_entry(
            iv_template_name     = lc_template_name
            iv_catalog_name      = lc_catalog_name
            iv_text              = lc_template_text
            it_parameters        = lt_parameters
            iv_transport_request = lc_transport_request
            iv_package           = lc_package
        ).
        out->write( |Job template created successfully| ).
      CATCH cx_apj_dt_content INTO lx_apj_dt_content.
        out->write( |Creation of job template failed: { lx_apj_dt_content-
>get_text( ) }| ).
        RETURN.
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.9.1.6 Setting up the Authorizations

Some further activities in ADT and in the administrator's launchpad are necessary to be able to schedule the job template in the Fiori app *Application Jobs*.

1. Create an Identity and Access Management (IAM) Business Catalog.
   When you create a job catalog entry and a job template as explained in Creating a Job Catalog Entry and a Job Template [page 1462], an object of the type **IAM App** is created automatically. It has the name **<job catalog entry name>_SAJC**.
   Create an IAM business catalog via ADT:
   1. Right-click the package where the objects are located that have already been created.
   2. Select *New > Other ABAP Repository Object*.
   3. Collapse *Cloud Identity and Access Management*.
   4. Double-click *Business Catalog*.
   5. Enter a name (e.g. `ZTEST_MY_SIMPLE_JOBS`) and a description.
   6. Select *Save* and choose the appropriate transport request.
   7. Select the tab *Apps* below.
   8. Press *Add ...* on the top right corner of the screen.
   9. Enter the name of the IAM app mentioned in step **e** (`ZTEST_MY_SIMPLE_JOB_SAJC`).
   10. Select *Save*.
   11. Press *Publish Locally* on the top right corner of the screen.
2. Create a business role.
   Open the Fiori App *Maintain Business Roles* in the Fiori Launchpad of the administrator and perform the following steps:
   1. Click *New*.
   2. Choose a name (e.g. `ZTEST_MY_SIMPLE_JOBS`) and a description.
   3. Click on *Assigned Business Catalogs*.
   4. Select *Add*.
   5. Select the business role created in the previous step (e.g. `ZTEST_MY_SIMPLE_JOBS`).

      > **Note**
      >
      > A business user who shall be able to schedule application jobs needs to have access to the *Application Jobs* tile. This tile was provided by the business catalog `SAP_CORE_BC_APJ`, which is contained in the business role `SAP_BR_ADMINISTRATOR`.
      >
      > With the 2105 cloud release, the business catalog `SAP_CORE_BC_APJ` has been set to **deprecated**. Use the new business catalog `SAP_CORE_BC_APJ_JCE` instead. It doesn't only provide the *Application Jobs* app, but also contains restriction types to configure the authorization on other users' jobs (for more information, see the section *Assigning Further Authorizations* below).
      >
      > Add the business catalog `SAP_CORE_BC_APJ_JCE` to the business role created following the steps above, or assign it to a separate business role.
   6. Select *Save*.
3. Assign the business role to a user.
   For more information, see How to Maintain Business Users [page 3045].

Assigning Further Authorizations

With the 2105 cloud release, you can access only your own jobs in the *Application Jobs* app. In order to grant access to other users' objects used in the *Application Jobs* app, create a new business role assigning the business catalog `SAP_CORE_BC_APJ_JCE` to it. You can modify the provided restriction types according to your needs.

1. Create a new business role assigning the business catalog `SAP_CORE_BC_APJ_JCE` to it. The restriction type `Job Catalog Entry/Application Job Part` contains the two fields `Application Job Catalog Entry` and `Application Job Part`. For more information, see Maintain Business Roles [page 3068].

   > **Note**
   >
   > With this restriction type you'll be able to provide the necessary authorizations for certain actions on other users' application jobs, which are based on a specified job catalog entry. With this, you can display the logs of other users' jobs, which are based on this specified job catalog entry.
   >
   > Alternatively, you can also create a display only-role using this restriction type.

2. Next, define your business role with the above-mentioned restriction fields `Application Job Catalog Entry` and `Application Job Part`. To do so, select *Maintain Restrictions*. In the `Application Job Catalog Entry` field, you can choose the job catalog entries, and in the `Application Job Part` field, you can choose for which part of a job access should be granted. You can choose the values `JOB` (Job Details), `SPOOL` (Result List), and `APPLOG` (Log) by clicking the *Edit* icon next to the fields.
3. Go back and select *Save*. Now, you've maintained your business role, and you can assign the role to a business user.

Related Information

4.2.15.9.1.7 Deleting the Job Catalog Entry and the Job Template

Find out how to delete a job catalog entry and a related job template.

To delete a job catalog entry or a job template, use the ABAP development tools for Eclipse. Right-click on the object in the *Project Explorer* and select *Delete*.

Alternatively, you could also still delete job catalog entries and job templates via a released API. For more information, see Creating a Job Catalog Entry and a Job Template via API [page 1485].

Before you can delete a job catalog entry, you must first delete all related job templates: When you create a job catalog entry, an IAM app that is based on the job catalog entry is automatically created in the Cloud. If you have followed the steps in Setting up the Authorizations [page 1487], you should perform additional steps before deleting the job catalog entry, since otherwise, the IAM app can't automatically be deleted together with the job catalog entry:

- Remove the business role from the business user in the Fiori app *Maintain Business Users*
- Delete the business role in the Fiori app *Maintain Business Roles*
- Unassign the IAM app from the IAM business catalog in ABAP development tools for Eclipse
- Delete the IAM business catalog in ABAP development tools for Eclipse

Now, you can delete the job catalog entry.

If you only want to replace the job catalog entry (if you want to delete it to recreate it with the same name), it's sufficient to remove the IAM app assignment from the IAM business catalogs. After the creation of the new job catalog entry, assign the new IAM app to the IAM business catalogs.

4.2.15.9.2 Maintaining Application Jobs Using an API

You can use the `CL_APJ_RT_API` class to maintain application jobs. You can do the following:

- schedule an application job
- retrieve the status of an application job
- cancel an application job
- delete an application job

Example

Sample Code

```
class z_cl_rt_api_demo definition
  public
  final
  create public .
  public section.
    interfaces if_oo_adt_classrun.
  protected section.
  private section.
endclass.
class z_cl_rt_api_demo implementation.
"
<SIGNATURE>-----------------------------------------------------------------
----------------------+
" | Instance Public Method Z_CL_RT_API_DEMO->IF_OO_ADT_CLASSRUN~MAIN
"
+--------------------------------------------------------------------------
----------------------+
" | [--->] OUT                            TYPE REF TO IF_OO_ADT_CLASSRUN_OUT
"
+--------------------------------------------------------------------------
------------</SIGNATURE>
  method if_oo_adt_classrun~main.
    data lv_job_text type cl_apj_rt_api=>ty_job_text value 'Demo_Job'.
    data lv_template_name type cl_apj_rt_api=>ty_template_name.
    data ls_start_info type cl_apj_rt_api=>ty_start_info.
    data ls_scheduling_info type cl_apj_rt_api=>ty_scheduling_info.
    data ls_end_info type cl_apj_rt_api=>ty_end_info.
    data lt_job_parameters type cl_apj_rt_api=>tt_job_parameter_value.
    data ls_job_parameters type cl_apj_rt_api=>ty_job_parameter_value.
    data ls_value type cl_apj_rt_api=>ty_value_range.
    data lv_jobname type cl_apj_rt_api=>ty_jobname.
    data lv_jobcount type cl_apj_rt_api=>ty_jobcount.
    data lv_status type cl_apj_rt_api=>ty_job_status.
```

```
    data lv_statustext type cl_apj_rt_api=>ty_job_status_text.
    data lv_txt type string.
    data ls_ret type bapiret2.
" Choose the name of the existing job template.
    lv_template_name = 'ZTEST_MY_SIMPLE_JOB_TEMP'.
" The immediate start can't be used when being called from within a RAP
business object
" because the underlying API performs an implicit COMMIT WORK.
"   ls_start_info-start_immediately = 'X'.
" Start the job using a timestamp instead. This will start the job
immediately but can have a delay depending on the current workload.
    get time stamp field data(ls_ts1).
" Add 1 hour
    data(ls_ts2) = cl_abap_tstmp=>add( tstmp = ls_ts1
                                       secs  = 3600 ).
    ls_start_info-timestamp = ls_ts2.
" Periodicity
    ls_scheduling_info-periodic_granularity = 'D'.
    ls_scheduling_info-periodic_value = 1.
    ls_scheduling_info-test_mode = abap_false.
    ls_scheduling_info-timezone = 'CET'.
    ls_end_info-type = 'NUM'.
    ls_end_info-max_iterations = 3.
" Fill parameter table:
" Fill the table only if you want to overrule the parameter values
" which are stored in the template.
" The field names in this program must match the field names of the
template.
    ls_job_parameters-name = 'P_TEST1'.
    ls_value-sign = 'I'.
    ls_value-option = 'EQ'.
    ls_value-low = 'Test 1'.
    append ls_value to ls_job_parameters-t_value.
    append ls_job_parameters to lt_job_parameters.
    clear ls_job_parameters.
    ls_job_parameters-name = 'P_TEST2'.
    ls_value-sign = 'I'.
    ls_value-option = 'BT'.
    ls_value-low = 'ATEST'.
    ls_value-high = 'ZTEST'.
    append ls_value to ls_job_parameters-t_value.
    ls_job_parameters-name = 'P_TEST2'.
    ls_value-sign = 'I'.
    ls_value-option = 'BT'.
    ls_value-low = '11111'.
    ls_value-high = '99999'.
    append ls_value to ls_job_parameters-t_value.
    append ls_job_parameters to lt_job_parameters.
    clear ls_job_parameters.
    ls_job_parameters-name = 'P_TEST3'.
    ls_value-sign = 'I'.
    ls_value-option = 'EQ'.
    ls_value-low = '220'.
    append ls_value to ls_job_parameters-t_value.
    append ls_job_parameters to lt_job_parameters.
    clear ls_job_parameters.
    try.
" Some scenarios require that the job key ( = jobname, jobcount) is already
known
" before the job is created. The method generate_jobkey creates a valid job
key.
" This key can then be passed later on to the method schedule_job, and a
job with
" exactly this key is created.
" Optional. You need this call only if you have to know the job key in
advance.
"       cl_apj_rt_api=>generate_jobkey(
"         importing
```

```
"            ev_jobname  = lv_jobname
"            ev_jobcount = lv_jobcount ).
" If you pass the table lt_job_parameters , then the parameters
" contained in this table are used.
" If you don't pass the table, the parameters contained in the
" job template are used.
       cl_apj_rt_api=>schedule_job(
         exporting
           iv_job_template_name   = lv_template_name
           iv_job_text            = lv_job_text
           is_start_info          = ls_start_info
           is_scheduling_info     = ls_scheduling_info
           is_end_info            = ls_end_info
           it_job_parameter_value = lt_job_parameters
" The following two parameters are optional. If you pass them, they must
have been generated
" with the optional call generate_jobkey above.
"           iv_jobname             = lv_jobname
"           iv_jobcount            = lv_jobcount
         importing
           ev_jobname             = lv_jobname
           ev_jobcount            = lv_jobcount
       ).
       out->write( lv_jobname ).
       out->write( lv_jobcount ).
       cl_apj_rt_api=>get_job_status(
         exporting
           iv_jobname         = lv_jobname
           iv_jobcount        = lv_jobcount
         importing
           ev_job_status      = lv_status
           ev_job_status_text = lv_statustext
       ).
       out->write( lv_status ).
       out->write( lv_statustext ).
" Via the following method you can cancel the job
" in the application job context 'cancel' means (as in the Fiori app):
" 1. if the job is running, it will be canceled
" 2. if the job has not yet started, it will be deleted.
" In case the job is periodic, the whole periodicity chain is deleted.
       cl_apj_rt_api=>cancel_job(
         exporting
           iv_jobname  = lv_jobname
           iv_jobcount = lv_jobcount
       ).
     catch cx_apj_rt into data(exc).
       lv_txt = exc->get_longtext( ).
       ls_ret = exc->get_bapiret2( ).
       out->write( 'ERROR:' ).
       out->write( lv_txt ).
       out->write( 'msg type =' ).
       out->write( ls_ret-type ).
       out->write( 'msg id =' ).
       out->write( ls_ret-id ).
       out->write( 'msg number =' ).
       out->write( ls_ret-number ).
       out->write( 'msg message =' ).
       out->write( ls_ret-message ).
   endtry.
  endmethod.
endclass.
```

4.2.15.9.2.1 Examples of Periodic Application Jobs

You can schedule application jobs to run periodically.

Here, you'll find a few codesamples that specify various periodicities. These code samples use the new human-readable constants of the class `CL_APJ_RT_API`. Let's start with a codesample showing an application job that runs monthly, always on the second day of the month:

Sample Code

```
CLASS zcl_apj_monthly DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_apj_monthly IMPLEMENTATION.
"
<SIGNATURE>---------------------------------------------------------------------
--------------------+
" | Instance Public Method ZCL_APJ_MONTHLY->IF_OO_ADT_CLASSRUN~MAIN
"
+-------------------------------------------------------------------------------
--------------------+
" | [--->] OUT                            TYPE REF TO IF_OO_ADT_CLASSRUN_OUT
"
+-------------------------------------------------------------------------------
---------</SIGNATURE>
  METHOD if_oo_adt_classrun~main.
" the following coding creates an application job with the following start
condition:
"
" periodicity:  every month on the second day - starting September 2nd, 2024
" first start:  specified by variable  ls_start_info-timestamp
"
" Exception rule: If the execution day is a non-working day according to a
"                  specified calendar, the job should be skipped.
"
" End of series:   after 10 executions - specified by ls_end_info
    DATA job_text          TYPE cl_apj_rt_api=>ty_job_text VALUE
'Monthly_Job'.
    DATA template_name       TYPE cl_apj_rt_api=>ty_template_name.
    DATA ls_start_info       TYPE cl_apj_rt_api=>ty_start_info.
    DATA ls_scheduling_info  TYPE cl_apj_rt_api=>ty_scheduling_info.
    DATA ls_job_exception    TYPE cl_apj_rt_api=>ty_job_exception.
    DATA ls_end_info         TYPE cl_apj_rt_api=>ty_end_info.
    DATA jobname             TYPE cl_apj_rt_api=>ty_jobname.
    DATA jobcount            TYPE cl_apj_rt_api=>ty_jobcount.
    DATA status              TYPE cl_apj_rt_api=>ty_job_status.
    DATA statustext          TYPE cl_apj_rt_api=>ty_job_status_text.
    DATA: txt TYPE string.
    DATA: tz  TYPE timezone.
    DATA dat TYPE d.
    DATA tim TYPE t.
    template_name = 'ZTEST_MY_SIMPLE_JOB_TEMP'.
" the immediate start would look like this:
" ls_start_info-start_immediately = cl_apj_rt_api=>start_immediately.
    dat = '20240902'.
    tim = '040000'.
    tz = cl_abap_tstmp=>get_system_timezone(  ).
    CONVERT DATE dat TIME tim
            INTO TIME STAMP DATA(ts) TIME ZONE tz.
```

```
    ls_start_info-timestamp = ts.
" periodicity
    ls_scheduling_info-periodic_granularity   = cl_apj_rt_api=>period_months.
" monthly
    ls_scheduling_info-periodic_value         = 1.
    ls_scheduling_info-test_mode              = abap_false.
    ls_scheduling_info-timezone               = tz.
" exception rule
    ls_job_exception-calender_id = '01'.
    ls_job_exception-start_restriction_code =
cl_apj_rt_api=>dont_process_on_holiday.  "skip, if the execution day is a
holiday
    ls_scheduling_info-exception = ls_job_exception.
" the periodic job should not run anymore after the 10th execution
    ls_end_info-type = cl_apj_rt_api=>period_end_by_nr_of_executions.
    ls_end_info-max_iterations = 10.
    TRY.
        cl_apj_rt_api=>schedule_job(
          EXPORTING
            iv_job_template_name = template_name
            iv_job_text          = job_text
            is_start_info        = ls_start_info
            is_scheduling_info   = ls_scheduling_info
            is_end_info          = ls_end_info
            iv_jobname           = jobname
            iv_jobcount          = jobcount
          IMPORTING
            ev_jobname           = jobname
            ev_jobcount          = jobcount
        ).
        CONCATENATE jobname jobcount INTO txt SEPARATED BY space.
        out->write( txt ).
        cl_apj_rt_api=>get_job_status(
          EXPORTING
            iv_jobname         = jobname
            iv_jobcount        = jobcount
          IMPORTING
            ev_job_status      = status
            ev_job_status_text = statustext
        ).
        CONCATENATE 'status =' statustext INTO txt SEPARATED BY space.
        out->write( txt ).
      CATCH cx_apj_rt INTO DATA(exc).
        txt = exc->get_longtext( ).
        out->write( txt ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

You can specify more periodicities with the method `CL_APJ_RT_API=>SCHEDULE_JOB`. The code snippets below show and explain in comments how to fill the method's parameters `IS_START_INFO`, `IS_SCHEDULING_INFO` (including the substructures `EXCEPTION`, `WEEKDAY_INFO`, and `MONTH_INFO`), and `IS_END_INFO`.

Note

Note that there are hard-coded dates and times in the examples.

Each of the examples needs the following data definitions (or at least a subset):

Sample Code

```
DATA ls_start_info       TYPE cl_apj_rt_api=>ty_start_info.  "for method
parameter IS_START_INFO
```

```
DATA ls_scheduling_info  TYPE cl_apj_rt_api=>ty_scheduling_info. "for method
parameter IS_SCHEDULING_INFO
DATA ls_job_exception    TYPE cl_apj_rt_api=>ty_job_exception. "substructure
of ls_scheduling_info
DATA ls_weekday_info     TYPE cl_apj_rt_api=>ty_weekday_info. "substructure
of ls_scheduling_info
DATA ls_month_info       TYPE cl_apj_rt_api=>ty_month_info.  "substructure of
ls_scheduling_info
DATA ls_end_info         TYPE cl_apj_rt_api=>ty_end_info. "for method
parameter IS_END_INFO
DATA tz  TYPE timezone.
DATA dat TYPE d.
DATA tim TYPE t.
DATA ts TYPE timestamp.
```

Example 1

This code snippet shows how to specify the start condition of the periodicity of the fourth from the last day every month, starting from 28 January, 2024. Its first start is specified by the variable `LS_START_INFO-TIMESTAMP`. The application job stops running after ten executions, specified by `LS_END_INFO`.

Sample Code

```
" the immediate start would look like this:
" ls_start_info-start_immediately = cl_apj_rt_api=>start_immediately.
dat = '20241028'.  "should be the exact date of the first execution
tim = '040000'.
tz = cl_abap_tstmp=>get_system_timezone(  ).
CONVERT DATE dat TIME tim
        INTO TIME STAMP ts TIME ZONE tz.
ls_start_info-timestamp = ts.
" periodicity
ls_scheduling_info-periodic_granularity   = cl_apj_rt_api=>period_months.
ls_scheduling_info-periodic_value         = 1.
ls_scheduling_info-test_mode              = abap_false.
ls_scheduling_info-timezone               = tz.
ls_month_info-day = 4.
ls_month_info-shift_direction = cl_apj_rt_api=>from_end_of_month.
" there is also the constant
" cl_apj_rt_api=>FROM_BEGINNING_OF_MONTH
ls_scheduling_info-month_info   = ls_month_info.
" the periodic job should not run anymore after the 10th execution
ls_end_info-type = cl_apj_rt_api=>period_end_by_nr_of_executions.
ls_end_info-max_iterations = 10.
```

Example 2

This code snippet shows how to specify the start condition of the periodicity of the second day of every month, starting from 2 September, 2024. Its first start is specified by the variable `LS_START_INFO-TIMESTAMP`. The exception rule is that if the execution day is a non-working day according to a specified calendar, the job should be skipped. The application job stops running after ten executions, specified by `LS_END_INFO`.

Sample Code

```
" the immediate start would look like this:
" ls_start_info-start_immediately = cl_apj_rt_api=>start_immediately.
dat = '20240902'.
tim = '040000'.
tz = cl_abap_tstmp=>get_system_timezone(  ).
CONVERT DATE dat TIME tim
        INTO TIME STAMP ts TIME ZONE tz.
ls_start_info-timestamp = ts.
```

```
" periodicity
ls_scheduling_info-periodic_granularity   = cl_apj_rt_api=>period_months. "
monthly
ls_scheduling_info-periodic_value         = 1.
ls_scheduling_info-test_mode              = abap_false.
ls_scheduling_info-timezone               = tz.
" exception rule
ls_job_exception-calender_id = '01'.
ls_job_exception-start_restriction_code =
cl_apj_rt_api=>dont_process_on_holiday.  "skip if the execution day is a
holiday
" There are also the following constants for exception rules (names are self-
explanatory):
" cl_apj_rt_api=>PROCESS_BEFORE_HOLIDAY
" cl_apj_rt_api=>PROCESS_AFTER_HOLIDAY
" cl_apj_rt_api=>PROCESS_ALWAYS
ls_scheduling_info-exception = ls_job_exception.
" the periodic job should not run anymore after the 10th execution
ls_end_info-type = cl_apj_rt_api=>period_end_by_nr_of_executions.
ls_end_info-max_iterations = 10.
```

Example 3

This code snippet shows how to specify the start condition of the periodicity of every week on Monday and Thursday. Its first start is specified by the variable `LS_START_INFO-TIMESTAMP`. The application job stops running after a specific time, specified by `LS_END_INFO-TIMESTAMP`.

> **Note**
>
> The application job start in this code snippet has to match a valid start date (a Monday or a Thursday).

Sample Code

```
" the immediate start would look like this
" ls_start_info-start_immediately = cl_apj_rt_api=>start_immediately.
" In the following the assignment
"      dat
" must be a Monday or a Thursday, because the job should run
" every week on Monday and Thursday.
" Otherwise, an error will be reported.
" So the variable
"   ls_start_info-timestamp
" will be the start date and time of the series.
dat = '20240219'.
tim = '180000'.
tz = cl_abap_tstmp=>get_system_timezone(  ).
CONVERT DATE dat TIME tim
        INTO TIME STAMP ts TIME ZONE tz.
ls_start_info-timestamp = ts.
" periodicity
" the job should run every third Tuesday of the month
ls_weekday_info-on_monday      = abap_true.
ls_weekday_info-on_thursday    = abap_true.
ls_scheduling_info-periodic_granularity   = cl_apj_rt_api=>period_weeks. "
weekly
ls_scheduling_info-periodic_value         = 1.
ls_scheduling_info-test_mode              = abap_false.
ls_scheduling_info-timezone               = tz.
ls_scheduling_info-weekday_info = ls_weekday_info.
" the periodic job should not run anymore after Dec. 31st 2030, 11:00 time
zone tz (system time zone)
ls_end_info-type = cl_apj_rt_api=>period_end_by_date.
dat = '20300331'.
tim = '110000'.
```

```
CONVERT DATE dat TIME tim
        INTO TIME STAMP ls_end_info-timestamp TIME ZONE tz.
```

Example 4

This code snippet shows how to specify the start condition of the periodicity of every week on Friday. Its first start is specified by the variable `LS_START_INFO-TIMESTAMP`. The exception rule is that if a Friday is a non-working day according to a specified calendar, the job should run on the next working day. The application job stops running at a specific day, specified by `LS_END_INFO-TIMESTAMP`.

> **Note**
>
> The application job start in this code snippet has to match a valid start date (a Friday).

Sample Code

```
" the immediate start would look like this
" ls_start_info-start_immediately = cl_apj_rt_api=>start_immediately.
" In the following the assignment
"      dat
" must be a Friday, because the job should run
" every week on Friday.
" Otherwise, an error will be reported.
" So the variable
"   ls_start_info-timestamp
" will be the start date and time of the series.
dat = '20240823'.
tim = '040000'.
" alternatively, we can work with the system timezone
" tz = CL_ABAP_TSTMP=>get_system_timezone(  ).
CONVERT DATE dat TIME tim
        INTO TIME STAMP ts TIME ZONE 'CET'. "means: the first job should
start at 4:00 CET
ls_start_info-timestamp = ts.
" periodicity
ls_scheduling_info-periodic_granularity   = cl_apj_rt_api=>period_weeks. "
weekly
ls_scheduling_info-periodic_value         = 1.
ls_scheduling_info-test_mode              = abap_false.
" Below, the timezone for the periodicity is set. In combination with the
above first start time 4:00
" the periodicity time zone CET means: The job should always start at 4:00
CET - no matter if daylight savings time
" is active or not.
" If we don't set the periodicity timezone, it will be set to the system
timezone by default. Let's assume UTC.
" Since the first start date and time of the job is 20240223 040000 CET, it
is 20240223 030000 UTC.
" The job will then run at 3:00 UTC every Friday, which is 4:00 CET in
'winter' and 5:00 CET in 'summer'.
ls_scheduling_info-timezone               = 'CET'.
" the job should run every Friday
ls_weekday_info-on_friday      = abap_true.
ls_scheduling_info-weekday_info = ls_weekday_info.
" exception rule
ls_job_exception-calender_id = '01'.
ls_job_exception-start_restriction_code =
cl_apj_rt_api=>process_after_holiday.  "run on the next working day
ls_scheduling_info-exception = ls_job_exception.
" the periodic job should not run anymore after Dec. 31st 2030, 11:00 time
zone tz (system time zone)
ls_end_info-type = cl_apj_rt_api=>period_end_by_date.
dat = '20300331'.
tim = '110000'.
```

```
CONVERT DATE dat TIME tim
        INTO TIME STAMP ls_end_info-timestamp TIME ZONE tz.
```

Example 5

This code snippet shows how to specify the start condition of the periodicity of the second Thursday of every month. Its first start is specified by the variable `LS_START_INFO-TIMESTAMP`. The application job stops running after 10 executions, specified by `LS_END_INFO`.

Sample Code

```
" the immediate start would look like this:
" ls_start_info-start_immediately = cl_apj_rt_api=>start_immediately.
" start date and time of series
dat = '20240208'. " this date must match the date of the first job execution
tim = '180000'.
tz = cl_abap_tstmp=>get_system_timezone(  ).
CONVERT DATE dat TIME tim
        INTO TIME STAMP ts TIME ZONE tz.
ls_start_info-timestamp = ts.
" periodicity
ls_scheduling_info-periodic_granularity   =
cl_apj_rt_api=>period_weekday_in_month.
ls_scheduling_info-periodic_value         = 1.
ls_scheduling_info-test_mode              = abap_false.
ls_scheduling_info-timezone               = tz.
ls_weekday_info-on_thursday               = abap_true.
ls_scheduling_info-weekday_info  = ls_weekday_info.
ls_month_info-week_number = 2.
ls_scheduling_info-month_info   = ls_month_info.
" the periodic job should not run anymore after the 10th execution
ls_end_info-type = cl_apj_rt_api=>period_end_by_nr_of_executions.
ls_end_info-max_iterations = 10.
```

New Constants of the Class `CL_APJ_RT_API`:

For periodicity:

- `CL_APJ_RT_API=>PERIOD_MINUTES`
- `CL_APJ_RT_API=>PERIOD_HOURS`
- `CL_APJ_RT_API=>PERIOD_DAYS`
- `CL_APJ_RT_API=>PERIOD_WEEKS`
- `CL_APJ_RT_API=>PERIOD_MONTHS`
- `CL_APJ_RT_API=>PERIOD_WEEKDAY_IN_MONTH` (see example 5)

For immediate start:

- `CL_APJ_RT_API=>START_IMMEDIATELY` (used in all above examples, but commented out)

For end of condition series:

- `CL_APJ_RT_API=>PERIOD_END_BY_DATE` (see example 5, for example)
- `CL_APJ_RT_API=>PERIOD_END_BY_NR_OF_EXECUTIONS` (see example 2, for example)

For count direction (from the start or end of the month):

- `CL_APJ_RT_API=>FROM_BEGINNING_OF_MONTH` (see example 1)
- `CL_APJ_RT_API=>FROM_END_OF_MONTH` (see example 1)

Exception rules (how to treat an application job if the regular job execution is (or would be) on a holiday according to the specified calendar:

- `CL_APJ_RT_API=>DONT_PROCESS_ON_HOLIDAY`
- `CL_APJ_RT_API=>DONT_PROCESS_BEFORE_HOLIDAY` (for example, on the last working day before the holiday)
- `CL_APJ_RT_API=>PROCESS_AFTER_HOLIDAY` (for example, on the first working day after the holiday)
- `CL_APJ_RT_API=>PROCESS_ALWAYS`

Only relevant for the method `CL_APJ_RT_API=>RESTART_JOB`:

- `CL_APJ_RT_API=>RESTART_FROM_ERROR_STEP`
- `CL_APJ_RT_API=>RESTART_AFTER_ERROR_STEP`

4.2.15.10 Application Logs

You can use the *Application Logs* to display and check if any errors occurred during runtime.

The *Application Logs* are a reuse library that is mainly used within the *Application Jobs* app, but can also be used in other apps. This reuse library gives you a clearly structured overview of all errors that might have occurred during runtime.

Key Features

You can use this reuse library to:

- view application log details
- filter the logs by severity
- search for message texts
- display the message details
- display archived application logs

Component for Customer Incidents

BC-SRV-APS-APL

For more information, see

- Design Time API [page 1499]
- Runtime API [page 1503]
- How to Use the Fiori Reuse Library [page 1564]

Related Information

Working with Application Log Objects

4.2.15.10.1 Design Time API

You can use the *Application Log Design Time API* if you want to create, change, or delete an application log object or subobject during the development process of an application.

For information on how to create an application log object in ADT, see Working with Application Log Objects.

The *Application Log Design Time API* provides the following design time operations:

- Create a new application log object, optionally with subobjects
- Delete an application log object and all of its subobjects
- Add a new subobject to an existing application log object
- Delete a subobject from an application log object
- Read an application log object and all of its subobjects

Create a New Application Log Object, Optionally with Subobjects

Once you have created an instance of the `CL_BALI_OBJECT_HANDLER` API class using method `CREATE_OBJECT`, a new application log object is created.

Sample Code

```
DATA(lo_log_object) = cl_bali_object_handler=>get_instance( ).
TRY.
    lo_log_object->create_object( EXPORTING iv_object = 'MYOBJECT'
                                            iv_object_text = 'My Application
Log Object'
                                            iv_package = 'MYPACKAGE'
                                            iv_transport_request =
'MYTRANSPORT' ).
  CATCH cx_bali_objects INTO DATA(lx_exception).
    WRITE lx_exception->get_text( ).
ENDTRY.
```

In case the new application log object has subobjects, you can create these too, using the `CREATE_OBJECT` method.

Sample Code

```
DATA: ls_subobject TYPE LINE OF if_bali_object_handler=>ty_tab_subobject,
      lt_subobject TYPE if_bali_object_handler=>ty_tab_subobject.
ls_subobject-subobject = 'MYSUBOBJECT'.
ls_subobject-subobject_text = 'My Application Log Sub-Object'.
APPEND ls_subobject TO lt_subobject.
DATA(lo_log_object) = cl_bali_object_handler=>get_instance( ).
```

```
TRY.
    lo_log_object->create_object( EXPORTING iv_object = 'MYOBJECT'
                                            iv_object_text = 'My Application
Log Object'
                                            it_subobjects = lt_subobject
                                            iv_package = 'MYPACKAGE'
                                            iv_transport_request =
'MYTRANSPORT' ).
  CATCH cx_bali_objects INTO DATA(lx_exception).
    WRITE lx_exception->get_text( ).
ENDTRY.
```

Delete an Application Log Object and all of its Subobjects

Use method `DELETE_OBJECT` to delete an application log object and all of its subobjects.

Sample Code

```
DATA(lo_log_object) = cl_bali_object_handler=>get_instance( ).
TRY.
    lo_log_object->delete_object( EXPORTING iv_object = 'MYOBJECT'
                                            iv_transport_request =
'MYTRANSPORT' ).
  CATCH cx_bali_objects INTO DATA(lx_exception).
    WRITE lx_exception->get_text( ).
ENDTRY.
```

Add a New Subobject to an Existing Application Log Object

Use method `ADD_SUBOBJECT` if you want to add a new subobject to an existing application log object.

Sample Code

```
DATA(lo_log_object) = cl_bali_object_handler=>get_instance( ).
TRY.
    lo_log_object->add_subobject( EXPORTING iv_object = 'MYOBJECT'
                                            iv_subobject = 'MYSUBOBJECT'
                                            iv_subobject_text = 'My
Application Log Sub-Object'
                                            iv_transport_request =
'MYTRANSPORT' ).
  CATCH cx_bali_objects INTO DATA(lx_exception).
    WRITE lx_exception->get_text( ).
ENDTRY.
```

Delete a Subobject from an Application Log Object

You can delete a subobject using method `DELETE_SUBOBJECT`.

Sample Code

```
DATA(lo_log_object) = cl_bali_object_handler=>get_instance( ).
TRY.
    lo_log_object->delete_subobject( EXPORTING iv_object = 'MYOBJECT'
                                               iv_subobject = 'MYSUBOBJECT'
                                               iv_transport_request =
'MYTRANSPORT' ).
  CATCH cx_bali_objects INTO DATA(lx_exception).
    WRITE lx_exception->get_text( ).
ENDTRY.
```

Read an Application Log Object and all of its Subobjects

Use method `READ_OBJECT` to read an application log object with its subobjects.

Sample Code

```
DATA(lo_log_object) = cl_bali_object_handler=>get_instance( ).
TRY.
    lo_log_object->read_object( EXPORTING iv_object = 'MYOBJECT'
                                IMPORTING et_subobjects = DATA(lt_subobjects)
                                          ev_object_text =
DATA(lv_object_text) ).
  CATCH cx_bali_objects INTO DATA(lx_exception).
    WRITE lx_exception->get_text( ).
ENDTRY.
```

4.2.15.10.1.1 Classes and Interfaces of the Design Time API

You can use the `CL_BALI_OBJECT_HANDLER` class as *Application Log Design Time API* to create, change, read, or delete application log objects or subobjects. It uses the public interface `IF_BALI_OBJECT_HANDLER`.

Public Methods

GET_INSTANCE (static)

Parameter	Type	Description
RO_OBJ_HANDLER	Returning	Create, change or delete application log objects

Create a new application log object, optionally with subobjects.

IF_BALI_OBJECT_HANDLER~CREATE_OBJECT

Parameter	Type	Description
IV_OBJECT	Importing	New application log object
IV_OBJECT_TEXT	Importing	Description for new application log object
IT_SUBOBJECTS	Importing	Table of new application log subobjects
IV_PACKAGE	Importing	Package
IV_TRANSPORT_REQUEST	Importing	Transport request

> **Note**
>
> The input of a table of subobjects for parameter `IT_SUBOBJECTS` is optional. If parameters `IV_PACKAGE` and `IV_TRANSPORT_REQUEST` are not provided, it is assumed that a local object shall be created.

Delete an application log object and all of its subobjects.

IF_BALI_OBJECT_HANDLER~DELETE_OBJECT

Parameter	Type	Description
IV_OBJECT	Importing	Application log object
IV_TRANSPORT_REQUEST	Importing	Transport request

Add a new subobject to an existing application log object.

IF_BALI_OBJECT_HANDLER~ADD_SUBOBJECT

Parameter	Type	Description
IV_OBJECT	Importing	Application log object
IV_SUBOBJECT	Importing	New application log subobject
IV_SUBOBJECT_TEXT	Importing	Description for new application log subobject
IV_TRANSPORT_REQUEST	Importing	Transport request

Delete a subobject from an application log object.

IF_BALI_OBJECT_HANDLER~DELETE_SUBOBJECT

Parameter	Type	Description
IV_OBJECT	Importing	Application log object
IV_SUBOBJECT	Importing	Application log subobject

Parameter	Type	Description
IV_TRANSPORT_REQUEST	Importing	Transport request

Read an application log object and all of its subobjects

IF_BALI_OBJECT_HANDLER~READ_OBJECT

Parameter	Type	Description
IV_OBJECT	Importing	Application log object
EV_OBJECT_TEXT	Importing	Description for an application log object
ET_SUBOBJECTS	Importing	Table of application log subobjects

> ### Note
>
> For all methods, `IV_TRANSPORT_REQUEST` is not required for local objects.
>
> All methods of `IF_BALI_OBJECT_HANDLER` include exception `CX_BALI_OBJECTS`.

4.2.15.10.2 Runtime API

Use a class-based API to create and read application logs.

You can use a class-based API to create and read application logs. The application log is a tool for collecting messages or exceptions which are stored in a log. You can save this log in the database or read and delete it from there.

If an application wants to collect messages and exceptions which occur during runtime of an application program, the following steps are required:

- Create a New Application Log [page 1504]
- Set Header Information [page 1506]
- Add Items [page 1507]
- Writing Application Logs to the Database [page 1514]

If an application wants to read the content of an application log from the database, for example, to display its content, the following steps are required:

- Define a Filter [page 1516]
- Read Application Logs from the Database [page 1517]
- Get Header Information from the Log [page 1520]
- Get Items from the Log [page 1521]

4.2.15.10.2.1 Creating an Application Log

If an application wants to collect messages and exceptions which occur during runtime of an application program, the following steps are required:

- Create a New Application Log [page 1504]
- Set Header Information [page 1506]
- Add Items [page 1507]
- Writing Application Logs to the Database [page 1514]

4.2.15.10.2.1.1 Create a New Application Log

If you want to create a new application log, you must first create an object of class `CL_BALI_LOG`. This object manages a single application log and allows to get and set the log header and all log items, such as messages or exceptions.

To create an object of class `CL_BALI_LOG`, the class provides two methods:

- If the header information of the log, for example the external identifier, is already known when the application log is created, you can use the `CREATE_WITH_HEADER` method. In this case, you can set the log header during the call. For more information, see Set Header Information [page 1506].

Example

Sample Code

```
CLASS zcl_test_write DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_test_write IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    TRY.
        " Create a new application log
        DATA(l_log) = cl_bali_log=>create( ).
        " Add a header to the log
        l_log->set_header( header = cl_bali_header_setter=>create( object =
'ZOBJECT'

subobject = 'ZSUBOBJECT'

external_id = 'External ID' ) ).
        " Add a message as item to the log
        DATA(l_message) = cl_bali_message_setter=>create( severity =
if_bali_constants=>c_severity_error
                                                         id = 'PO'
                                                         number = '000' ).
        l_log->add_item( item = l_message ).
        " Add a second message, this time from system fields SY-MSGID, ...
        MESSAGE ID 'ZTEST' TYPE 'S' NUMBER '058' INTO DATA(l_text).
```

```
        l_log->add_item( item = cl_bali_message_setter=>create_from_sy( ) ).
        " Add a free text to the log
        DATA(l_free_text) = cl_bali_free_text_setter=>create( severity =
if_bali_constants=>c_severity_error
                                                             text = 'Some
Error Text' ).
        l_log->add_item( item = l_free_text ).
        " Add an exception to the log
        DATA: i TYPE i.
        TRY.
            i = 1 / 0.
          CATCH cx_sy_zerodivide INTO DATA(l_ref).
        ENDTRY.
        DATA(l_exception) = cl_bali_exception_setter=>create( severity =
if_bali_constants=>c_severity_error
                                                             exception =
l_ref ).
        l_log->add_item( item = l_exception ).
        " Save the log into the database
        cl_bali_log_db=>get_instance( )->save_log( log = l_log ).
      CATCH cx_bali_runtime INTO DATA(l_runtime_exception).
        out->write( l_runtime_exception->get_text(  ) ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

Sample Code

```
...
    TRY.
        DATA(l_log) = cl_bali_log=>create_with_header(
                        header = cl_bali_header_setter=>create( object =
'ZOBJECT'
                                                               subobject =
'ZSUBOBJECT'
                                                               external_id =
'External ID' ) ).
      CATCH cx_bali_runtime INTO DATA(l_exception).
        out->write( l_exception->get_text(  ) ).
    ENDTRY.
...
```

If the header information is not known when the application log is created, you can use the `CREATE` method. It creates an empty application log. In this case, the header should be set later using the `SET_HEADER` method. If no subobjects were defined for the application log object, an empty string should be used at the parameter `SUBOBJECT`.

Sample Code

```
...
    TRY.
        DATA(l_log) = cl_bali_log=>create( ).
        ...
        l_log->set_header( header = cl_bali_header_setter=>create( object =
'ZOBJECT'
                                                                  subobject
= 'ZSUBOBJECT'

external_id = 'External ID' ) ).
      CATCH cx_bali_runtime INTO DATA(l_exception).
        out->write( l_exception->get_text(  ) ).
    ENDTRY.
```

```
...
```

If the application log is no longer needed in the memory, for example because it was saved into the database, it's possible to release its memory. This can be done using the method RELEASE_MEMORY.

After calling the method RELEASE_MEMORY, the log object is invalidated and can no longer be used. The method IS_INVALIDATED can be used to check whether the memory of the log object was already released by a previous call of the method RELEASE_MEMORY.

Sample Code

```
...
 TRY.
     DATA(l_log) = cl_bali_log=>create( ).
     ...
     " Add items to the log and, for example, write the log to the database
     ...
     " Release the memory of the log
     l_log->release_memory( ).
   CATCH cx_bali_runtime INTO DATA(l_exception).
     out->write( l_exception->get_text(  ) ).
 ENDTRY.
 ...
```

Sample Code

```
 ...
 TRY.
     DATA(l_log) = cl_bali_log=>create( ).
     ...
     " Check whether the log was already invalidated
     IF l_log->is_invalidated( ) = abap_true.
       " Start some error handling
     ENDIF.
     ...
   CATCH cx_bali_runtime INTO DATA(l_exception).
     out->write( l_exception->get_text(  ) ).
 ENDTRY.
 ...
```

4.2.15.10.2.1.2 Set Header Information

An application log contains header information and attributes to identify the log. During log creation, the following header information can be set:

- Log Object (required): Identifies the application to which the log belongs.
- Log Subobject: An application can define subcategories of its log object. These subcategories are called subobjects and must be set if the application defined them. If no subobjects were defined for the log object, the field should remain empty.
- External Identifier: We recommend to enter your preferred identifier containing, for example, the application document number.
- Expiry Date: It defines the date when the log expires and can be deleted from the database. The default value is the creation date of the log + 7 days.

- A flag defining whether it should not be allowed to delete the log before the expiry date by a standard report. The application is still able to delete the log before this date.
- Context: Allows the storage of application-specific data related to the log. See Set a Context [page 1513].

To define the log header, an instance of the `IF_BALI_HEADER_SETTER` interface is required. To create this instance, you can use method `CREATE` of class `CL_BALI_HEADER_SETTER`. It allows to set the object, subobject and the external identifier of the application log.

Interface `IF_BALI_HEADER_SETTER` contains the following methods to set or change the header attributes:

- `SET_DESCRIPTOR`: Changes the object, subobject and external identifier.
- `SET_EXPIRY`: Sets the expiry date and the *keep until expiry* flag.
- `SET_CONTEXT`: Sets the context of the log.

> **Note**
>
> You have two options to transfer the header to the application log object:
>
> - Using method `CREATE_WITH_HEADER` of class `CL_BALI_LOG` when the log object is created.
> - Using method `SET_HEADER` of interface `IF_BAL_LOG`.
>
> If the header was already written to the log (for example, using method `SET_HEADER` of interface `IF_BAL_LOG`) and if the header object is changed afterwards (for example, using method `SET_DESCRIPTOR`), you must call method `SET_HEADER` to make these changes visible in the log.

> **Sample Code**

```
...
 TRY.
     DATA(l_header) = cl_bali_header_setter=>create( object = 'ZOBJECT'
                                                     subobject = 'ZSUBOBJECT'
                                                     external_id = 'External
ID' ).
     l_header->set_expiry( expiry_date = CONV
#( cl_abap_context_info=>get_system_date(  ) + 5 )
                           keep_until_expiry = abap_true ).
   CATCH cx_bali_runtime INTO DATA(l_runtime_exception).
     out->write( l_runtime_exception->get_text(  ) ).
 ENDTRY.
...
```

4.2.15.10.2.1.3 Add Items

To write items, such as messages or exceptions, to the log during runtime, the application requires an item object which contains all attributes of the item, such as the message number or the reference to the exception object.

The following categories of log items are provided:

- Message (for example, a message is displayed using ABAP command **`MESSAGE`**). For more information, see Create a Message [page 1509].
- Free Text (a text with a maximum length of 200 characters). For more information, see Create a Free Text [page 1511].

- Exception (for example, an exception is raised using ABAP command **RAISE EXCEPTION** and can be caught using ABAP command **CATCH**). For more information, see Create an Exception [page 1512].

To add a log item to an application log, the IF_BALI_LOG interface provides the following methods:

- ADD_ITEM: Append an item to the application log
- CUMULATE_ITEM: If the item is a message, it's checked whether the log already contains a message with the same message attributes (severity, message ID, message number and variables). If yes, the counter of the message is increased by 1. In all other cases, it works the same way as ADD_ITEM.
- ADD_MESSAGES_FROM_BAPIRETTAB: Append several messages to the application log. The message attributes are stored in an internal table of type BAPIRETTAB.
- ADD_ABAP_BEHAVIOR_MESSAGE: Append an ABAP behavior message, which uses the interface IF_ABAP_BEHV_MESSAGE, to the log.
- ADD_ALL_ITEMS_FROM_OTHER_LOG: Read all items from another application log and append them to this log. This method can be used if the items of different application logs are merged into one log. Sometimes it's required to filter items which are put into an application log by using the method ADD_ITEM. For example, there may be an encapsulated application class which writes error and status messages into an application log, and the caller of this application class only requires the error messages. In this case, the caller can define an item filter (see Define a Filter for the Log Items [page 1512]). The caller can then add this item filter to the log using the method SET_FILTER_FOR_ADD_ITEM. If an item filter is set for a log object, only those items that pass this filter are added to the log object.

Note

If an item was already added to the application log, its attributes can't be changed any more. Therefore, if the attributes of the item object are changed after the item was added to the log, these changes are not transferred to the log.

When an item is added to the application log, it can happen that it's converted internally. For example, an exception which is based on a message (which contains a message ID and message number) is internally converted into a message.

Sample Code

```
...
 TRY.
     DATA(l_log) = cl_bali_log=>create( ).

     MESSAGE ID 'ZTEST' TYPE 'W' NUMBER '002' INTO DATA(l_text).
     l_log->add_item( item = cl_bali_message_setter=>create_from_sy( ) ).
     l_log->cumulate_item( item = cl_bali_message_setter=>create_from_sy( ) ).
     DATA: i TYPE i.
     TRY.
         i = 1 / 0.
       CATCH cx_sy_zerodivide INTO DATA(l_ref).
     ENDTRY.
     l_log->add_item( item = cl_bali_exception_setter=>create( exception =
l_ref ) ).
     DATA(l_bapirettab) = VALUE bapirettab( ( id = 'BL' type = 'I' number =
'315'
                                            message_v1 = 'A' message_v2 =
'B' message_v3 = 'C' message_v4 = 'D' )
                                          ( id = 'BL' type = 'E' number =
'319' ) ).
     l_log->add_messages_from_bapirettab( message_table = l_bapirettab ).
   CATCH cx_bali_runtime INTO DATA(l_exception).
     out->write( l_exception->get_text(  ) ).
 ENDTRY.
```

```
...
```

Sample Code

```
...
 TRY.
     DATA(l_log_target) = cl_bali_log=>create( ).

     MESSAGE ID 'ZTEST' TYPE 'W' NUMBER '002' INTO DATA(l_text).
     l_log_target->add_item( item =
cl_bali_message_setter=>create_from_sy( ) ).
     DATA(l_log_source) = cl_bali_log=>create( ).
     DATA: i TYPE i.
     TRY.
         i = 1 / 0.
       CATCH cx_sy_zerodivide INTO DATA(l_ref).
     ENDTRY.
     l_log_source->add_item( item =
cl_bali_exception_setter=>create( exception = l_ref ) ).
     " add all items of l_log_source to l_log_target
     l_log_target->add_all_items_from_other_log( source_log = l_log_source ).
   CATCH cx_bali_runtime INTO DATA(l_exception).
     out->write( l_exception->get_text(  ) ).
 ENDTRY.
 ...
```

Sample Code

```
...
 TRY.
     DATA(l_log) = cl_bali_log=>create( ).

     " Add item filter to the log which only allows error messages
     DATA(l_filter) = cl_bali_item_filter=>create( )-
>set_severity( severity_table = VALUE #( ( 'E' ) ) ).
     l_log->set_filter_for_add_item( filter = l_filter ).
     MESSAGE ID 'ZTEST' TYPE 'W' NUMBER '002' INTO DATA(l_text).
     l_log->add_item( item = cl_bali_message_setter=>create_from_sy( ) ).  "
Message is not added, because it is a warning
     MESSAGE ID 'ZTEST' TYPE 'E' NUMBER '002' INTO l_text.
     l_log->add_item( item = cl_bali_message_setter=>create_from_sy( ) ). "
Message is added, because it is an error
     " Get the current item filter
     data(l_item_filter) = l_log->get_filter_for_add_item( ).

     " Clear the current item filter
     l_log->set_filter_for_add_item( filter = value #( ) ).
   CATCH cx_bali_runtime INTO DATA(l_exception).
     out->write( l_exception->get_text(  ) ).
 ENDTRY.
...
```

4.2.15.10.2.1.3.1 Create a Message

One item category, which can be written into an application log, is a message. A message is identified by the message ID and the message number.

When an application log message is defined, you can set the following attributes:

- Severity, such as *Error*, *Warning*, *Information*. Possible values of the severity are defined as constants in interface `IF_BALI_CONSTANTS`.
- Message ID
- Message Number
- Variables 1 - 4 of the message
- Detail Level of the item: If the application displays the items of the application log, the detail level can be used to define at which level of detail the item shall be displayed.
- Context: Allows the storage of application-specific data related to the message. See Set a Context [page 1513].

To add a message to an application log, an instance of interface `IF_BALI_MESSAGE_SETTER` is required. To create this instance, class `CL_BALI_MESSAGE_SETTER` provides the following methods:

- `CREATE`: It allows to set the severity, message ID, message number and the variables of the message.
- `CREATE_FROM_SY`: You can use this method to create an application log message from the system fields in the list below. These system fields are filled, for example, by ABAP command **`MESSAGE`**. The following system fields are available:
  - SY-MSGTY
  - SY-MSGID
  - SY-MSGNO
  - SY-MSGV1
  - SY-MSGV2
  - SY-MSGV3
  - SY-MSGV4
- `CREATE_FROM_BAPIRET2`: It allows to set all message parameters via structure `BAPIRET2`.

Interface `IF_BALI_MESSAGE_SETTER` contains the following methods to set or change the attributes of the message:

- SET_ATTRIBUTES: Changes the severity, message ID, message number and the variables of the message.
- SET_FROM_SY: Changes the severity, message ID, message number and the variables of the message to the values of the system fields, such as SY-MSGTY.
- SET_FROM_BAPIRET2: Changes the severity, message ID, message number and the variables of the message to the values of structure BAPIRET2.
- SET_DETAIL_LEVEL: Sets the detail level.
- SET_CONTEXT: Sets the context of the message.

Sample Code

```
...
DATA(l_ref) = cl_bali_message_setter=>create(
                        severity = if_bali_constants=>c_severity_error
                        id = 'BL'
                        number = '315'
                        variable_1 = 'A'
                        variable_2 = 'B'
                        variable_3 = 'C'
                        variable_4 = 'D' ).
l_ref->set_detail_level( detail_level = '7' ).
...
```

```
 MESSAGE ID 'ZTEST' TYPE 'I' NUMBER '315' WITH 'E' 'F' 'G' 'H' INTO
DATA(l_message).
 l_ref = cl_bali_message_setter=>create_from_sy(
                              )->set_detail_level( detail_level = '3' ).
 ...
 DATA(l_bapiret2) = VALUE bapiret2( id = 'BL' type = 'I' number = '315'
                                    message_v1 = 'A' message_v2 = 'B'
message_v3 = 'C' message_v4 = 'D' ).
 l_ref = cl_bali_message_setter=>create_from_bapiret2( message_data =
l_bapiret2 ).
...
```

4.2.15.10.2.1.3.2 Create a Free Text

One item category, which can be written into an application log, is *Free Text*. It is any text with up to 200 characters.

The following attributes can be set when an application log free text is defined:

- Severity, such as *Error*, *Warning*, *Information*. Possible values of the severity are defined as constants in interface `IF_BALI_CONSTANTS`.
- Text content
- Detail level of the item: If the application displays the items of the application log, the detail level can be used to define at which level of detail the item should be visible.
- Context: Allows the storage of application-specific data related to the free text. See Set a Context [page 1513].

To add a free text to an application log, an instance of interface `IF_BALI_FREE_TEXT_SETTER` is required. To create this instance, you can use method `CREATE` of class `CL_BALI_FREE_TEXT_SETTER`. It allows to set the severity and the text content.

To set or change the attributes of the free text, interface `IF_BALI_FREE_TEXT_SETTER` contains the following methods:

- SET_TEXT: Changes the text content and severity.
- SET_DETAIL_LEVEL: Sets the detail level.
- SET_CONTEXT: Sets the context of the free text.

Sample Code

```
...
 DATA(l_ref) = cl_bali_free_text_setter=>create(
                             severity = if_bali_constants=>c_severity_error
                             text = 'Some Error Text' ).
 l_ref->set_detail_level( detail_level = '4' ).
 ...
 l_ref = cl_bali_free_text_setter=>create( text = 'Some Other Text'
                               )->set_detail_level( detail_level = '1' ).
...
```

4.2.15.10.2.1.3.3 Create an Exception

One item category, which can be written into an application log, is an *Exception*. An exception is raised by the application using ABAP command **RAISE EXCEPTION** and can be caught by ABAP command **CATCH**.

To define an application log exception, you can set the following attributes:

- Severity, such as *Error*, *Warning*, *Information*. Possible values of the severity are defined as constants in interface `IF_BALI_CONSTANTS`.
- Reference to the exception object
- Detail Level of the item: If the application displays the items of the application log, the detail level can be used to define at which level of detail the item should be displayed.
- Context: Allows the storage of application-specific data related to the exception. See Set a Context [page 1513].

To add an exception to an application log, an instance of interface `IF_BALI_EXCEPTION_SETTER` is required. To create this instance, method `CREATE` of class `CL_BALI_EXCEPTION_SETTER` can be used. It allows to set the severity and the reference to the exception object.

If you want to set or change the attributes of the exception, you can use interface `IF_BALI_EXCEPTION_SETTER`. It contains the following methods:

- SET_EXCEPTION: Changes the reference to the exception object and the severity.
- SET_DETAIL_LEVEL: Sets the detail level.
- SET_CONTEXT: Sets the context of the exception.

Sample Code

```
...
 DATA: i TYPE i.
 TRY.
     i = 1 / 0.
   CATCH cx_sy_zerodivide INTO DATA(l_exception_ref).
 ENDTRY.
 DATA(l_ref) = cl_bali_exception_setter=>create( severity =
if_bali_constants=>c_severity_error
                                                exception =
l_exception_ref ).
 l_ref->set_detail_level( detail_level = '2' ).
...
```

4.2.15.10.2.1.3.4 Define a Filter for the Log Items

If items are put into an application log, there may be the requirement to use an item filter in order to, for example, keep the log small. For example, a report calls an encapsulated application class which writes different types of items into the application log. If the report doesn't need all items, but, for example, only the errors, it may define an item filter which only lets the errors pass.

To create a filter, an instance of interface `IF_BALI_ITEM_FILTER` is required. To create this instance, the method `CREATE` of class `CL_BALI_ITEM_FILTER` can be used. It creates an empty filter. The interface `IF_BALI_ITEM_FILTER` contains the following method to set the filter parameters:

- `SET_SEVERITY`: Set a filter for the severity (error, warning,...) of the item. The severity values which pass the filter are set via an internal table.

  Sample Code

  Example:

  Set a filter which lets all items pass that have the severity *Error* or *Warning*:

```
...
    TRY.
        DATA(l_filter) = cl_bali_item_filter=>create( ).
        l_filter->set_severity( severity_table = value #( ( 'E' )
( 'W' ) ) ).
      CATCH cx_bali_runtime INTO DATA(l_exception).
        out->write( l_exception->get_text(  ) ).
    ENDTRY.
    ...
```

4.2.15.10.2.1.3.5 Set a Context

There may be the requirement to store application-specific data in the log. For example, a message may only be meaningful if you have information about the context in which a message was generated. An example of the context is the number of the order which was processed when the message was written into the log.

You can set a context for the complete log in the log header, and for each individual item of the log. To do this, the following steps are necessary:

- You need a structure which can store the context data. It must meet the following requirements:
  - The structure must either use a database table type, or a structure type which is defined in the data dictionary.
  - The structure type must be flat (which means that no internal tables or nested structures are allowed).
  - The structure type must be character-like (which means that no integer fields are allowed).
  - The total length of the complete structure must not be larger than 256 characters.
- Now, you can fill the structure fields with the application data which you want to store as context in the log or item.
- To transfer the structure with the context data to the log, you can use the method `SET_CONTEXT` of the header or item object.

Sample Code

```
...
 DATA: l_context TYPE ADDRESS_1. " it can be any application structure that
fulfils the requirements
 l_context = VALUE #( name1 = 'Test' name2 = 'Data' ). " fill the structure
with the context data
 TRY.
     DATA(l_header) = cl_bali_header_setter=>create( object = 'ZOBJECT'
                                                    subobject =
'ZSUBOBJECT' ).
     l_header->set_context( l_context ).
   CATCH cx_bali_runtime INTO DATA(l_runtime_exception).
     out->write( l_runtime_exception->get_text(  ) ).
```

```
 ENDTRY.
...
```

4.2.15.10.2.1.4 Writing Application Logs to the Database

You can use interface `IF_BALI_LOG_DB` to write an application log to the database or to delete database entries. To get an instance of the interface, method `GET_INSTANCE` of class `CL_BALI_LOG_DB` is available.

To change the logs in the database, interface `IF_BALI_LOG_DB` contains the following methods:

- SAVE_LOG / SAVE_LOG_2ND_DB_CONNECTION: Save an application log in the database. The log is identified by the log object (which uses interface `IF_BAL_LOG`).
  It may be required to commit the saving of the log immediately after the saving. It ensures that the log is stored, even if the application calls *ROLLBACK WORK* later on. Whether it is required depends on the application. Some applications want to keep the log entries after the rollback, others want to remove everything by the *ROLLBACK WORK*, including the log entries.
  Therefore, there are two different methods to save an application log:
  - SAVE_LOG uses the default database connection. This means that the changes of the log are only committed if the application calls `COMMIT_WORK`.
  - SAVE_LOG_2ND_DB_CONNECTION uses a service connection to the database for the saving, and it immediately commits the saving of the log.

  In addition, the optional parameter `ASSIGN_TO_CURRENT_APPL_JOB` is available. If this parameter is set, and if the application runs in an application job, the connection between the application log and an application job is established. The application log is visible in the application job display.
- DELETE_LOG: Deletes an application log from the database. The log is identified by the log object (which uses interface `IF_BAL_LOG`).
- ENQUEUE: If there is a parallel processing of the same application report, it may happen - depending on the application - that several reports try to change the same application log at the same time. In this case, one report overwrites the application log data of the other report. To avoid this, method `ENQUEUE` can be used to set an enqueue on the application log. The log is identified by the log object (which uses interface `IF_BAL_LOG`).
- DEQUEUE: Clear the enqueue which was set via method ENQUEUE. The log is identified by the log object (which uses interface `IF_BAL_LOG`).

Note

You can quickly access a log stored in the database using the log handle. The log handle is the UUID of the log. To enable the application to store the log handle in one of the application tables, interface `IF_BAL_LOG` offers method `GET_HANDLE` which returns the log handle.

Example

Save a log in the database:

Sample Code

```
...
    TRY.
```

```
        DATA(l_log) =
cl_bali_log=>create_with_header( cl_bali_header_setter=>create( object =
'ZOBJECT'

           subobject = 'ZSUBOBJECT'

           external_id = 'External ID' ) ).
        MESSAGE ID 'ZTEST' TYPE 'I' NUMBER '315' WITH 'A' 'B' 'C' 'D' INTO
DATA(l_text).
        l_log->add_item( cl_bali_message_setter=>create_from_sy( ) ).
        cl_bali_log_db=>get_instance( )->save_log( log = l_log ).
      CATCH cx_bali_runtime INTO DATA(l_exception).
        out->write( l_exception->get_text(  ) ).
    ENDTRY.
...
```

Save the same log using the second database connection:

Sample Code

```
...
    TRY.
        cl_bali_log_db=>get_instance( )->save_log_2nd_db_connection( log =
l_log ).
      CATCH cx_bali_runtime INTO DATA(l_exception).
        out->write( l_exception->get_text(  ) ).
    ENDTRY.
...
```

Delete the same log from the database:

Sample Code

```
...
    TRY.
        cl_bali_log_db=>get_instance( )->delete_log( log = l_log ).
      CATCH cx_bali_runtime INTO DATA(l_exception).
        out->write( l_exception->get_text(  ) ).
    ENDTRY.
...
```

Delete a log from the database which uses the handle l_handle.

:

Sample Code

```
...
    DATA l_handle TYPE if_bali_log=>ty_handle.
    l_handle = ...
    TRY.
        DATA(l_log_db) = cl_bali_log_db=>get_instance( ).
        DATA(l_log) = l_log_db->load_log( handle = l_handle
                                          read_only_header = abap_true ).
        l_log_db->delete_log( log = l_log ).
      CATCH cx_bali_runtime INTO DATA(l_exception).
        out->write( l_exception->get_text(  ) ).
    ENDTRY.
...
```

The following authorization is checked before a log is deleted from the database:

Authorization object: `S_APPL_LOG`, with

- `ALG_OBJECT`: Object name of the application log
- `ALG_SUBOBJ`: Subobject of the application log
- `ACTVT`: 06

4.2.15.10.2.2 Reading an Application Log

If an application wants to read the content of an application log from the database, for example, to display its content, the following steps are required:

- Define a Filter [page 1516]
- Read Application Logs from the Database [page 1517]
- Get Header Information from the Log [page 1520]
- Get Items from the Log [page 1521]

4.2.15.10.2.2.1 Define a Filter

If one or more application logs shall be read from the database and if the log handles are not known, you can identify the logs by defining a filter.

To create a filter, an instance of interface `IF_BALI_LOG_FILTER` is required. To create this instance, you can use method `CREATE` of class `CL_BALI_LOG_FILTER`. It creates an empty filter.

> **Note**
>
> Due to performance reasons, we recommend to set at least the object and subobject in the filter.

Interface `IF_BALI_LOG_FILTER` contains the following methods to set the filter parameters:

- SET_DESCRIPTOR: Set a filter for the log object, subobject and external identifier. It offers the following parameters:
  - Select a single object
  - Select a single subobject (wildcards allowed)
  - Select a table of subobjects (wildcards allowed)
  - Select a single external identifier (wildcards allowed)
  - Select a table of external identifiers (wildcards allowed)
- SET_CREATE_INFO: Set a filter for the attribute which identifies the creator of the log. It offers the following parameters:
  - Select a single user (wildcards allowed)
  - Select a table of users (wildcards allowed)
- SET_TIME_INTERVAL: Set a time interval for the creation date and time of the log. The start and end time of the interval are set using UTC time stamps.

- SET_MAXIMUM_LOG_NUMBER: Set the maximum number of logs which shall be read from the database. If the parameter is not set, all logs are read (which fulfill the other filter criteria).

Example

Set a filter which selects up to 5 logs which were created in the last hour by the current user:

Sample Code

```
...
    TRY.
        DATA(l_filter) = cl_bali_log_filter=>create( ).
        l_filter->set_create_info( user = sy-uname ).

        DATA(l_timestamp_now) = utclong_current( ).
        DATA(l_timestamp_minus_1_hour) = utclong_add( val = l_timestamp_now
                                                      hours = '1-' ).
        l_filter->set_time_interval( start_time = l_timestamp_minus_1_hour
                                     end_time = l_timestamp_now ).
        l_filter->set_maximum_log_number( max_log_number = 5 ).
      CATCH cx_bali_runtime INTO DATA(l_exception).
        out->write( l_exception->get_text(  ) ).
    ENDTRY.
...
```

Set a filter which reads all logs of the current user with object `ZOBJECT`, subobject `ZSUBOBJECT` and an external identifier which starts with an 'E':

Sample Code

```
...
    TRY.
        DATA(l_filter) = cl_bali_log_filter=>create(
                                           )->set_create_info( user = sy-
uname
                                           )->set_descriptor( object =
'ZOBJECT'
                                                              subobject =
'ZSUBOBJECT'
                                                              external_id =
'E*' ).
      CATCH cx_bali_runtime INTO DATA(l_exception).
        out->write( l_exception->get_text(  ) ).
    ENDTRY.
...
```

4.2.15.10.2.2.2 Read Application Logs from the Database

You can use interface `IF_BALI_LOG_DB` to read one or more application logs from the database. To get an instance of the interface, method `GET_INSTANCE` of class `CL_BALI_LOG_DB` is available.

Interface `IF_BALI_LOG_DB` contains the following methods which can be used to read the logs from the database:

- LOAD_LOG / LOAD_LOG_WITH_ITEMS: Load one application log from the database.
  - The log is identified by the log handle.
  - The method returns an object of interface `IF_BALI_LOG`.
  - With method `LOAD_LOG_WITH_ITEMS`, the complete log including all items is read from the database. If you use method `LOAD_LOG` and set the optional parameter `READ_ONLY_HEADER`, only the log header is read from the database. The log items are not read yet.
- LOAD_LOGS_VIA_FILTER / LOAD_LOGS_W_ITEMS_VIA_FILTER: Load one or more logs from the database.
  - The logs are identified by a filter object of interface `IF_BALI_LOG_FILTER`.
  - The method returns an internal table of objects of interface `IF_BALI_LOG`.
  - With method `LOAD_LOG_W_ITEMS_VIA_FILTER`, the complete logs including all items are read from the database. If you use method `LOAD_LOGS_VIA_FILTER` and set the optional parameter `READ_ONLY_HEADER`, only the log headers are read from the database. The log items are not read yet.

The following authorization is checked before a log is read from the database:

Authorization object: `S_APPL_LOG`, with

- `ALG_OBJECT`: Object name of the application log
- `ALG_SUBOBJ`: Subobject of the application log
- `ACTVT`: 03

Example

Load a log from the database:

Sample Code

```
CLASS zcl_test_read DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_test_read IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    TRY.
        " Create a filter which selects all logs of the current user from
the last hour
        DATA(l_filter) = cl_bali_log_filter=>create( ).
        l_filter->set_create_info( user = sy-uname ).
        DATA(l_timestamp_now) = utclong_current( ).
        DATA(l_timestamp_minus_1_hour) = utclong_add( val = l_timestamp_now
                                                      hours = '1-' ).
        l_filter->set_time_interval( start_time = l_timestamp_minus_1_hour
                                     end_time = l_timestamp_now ).
        " Read all Application Logs from the database which fit to the
filter
        DATA(log_table) = cl_bali_log_db=>get_instance( )-
>load_logs_w_items_via_filter( filter = l_filter ).
        LOOP AT log_table INTO DATA(l_log).
          " Output log handle
          out->write( |Handle: { l_log->get_handle( ) }| ).
```

```
          " Get log header and output attributes of the header
          DATA(l_header) = l_log->get_header( ).
          out->write( |{ l_header->object } { l_header->subobject }
{ l_header->external_id } { l_header->log_user }| ).
          " Get all items and output some data which exist in all item
categories
          DATA(l_item_table) = l_log->get_all_items( ).
          LOOP AT l_item_table INTO DATA(l_item_entry).
            out->write( |{ l_item_entry-log_item_number } { l_item_entry-
item->get_message_text( ) }| ).
            " Output attributes which are specific for messages and
exceptions
            IF l_item_entry-item->category =
if_bali_constants=>c_category_message.
              DATA(l_message_ref) = CAST
if_bali_message_getter( l_item_entry-item ).
              out->write( |{ l_message_ref->id } { l_message_ref-
>number }| ).
            ELSEIF l_item_entry-item->category =
if_bali_constants=>c_category_exception.
              DATA(l_exception_ref) = CAST
if_bali_exception_getter( l_item_entry-item ).
              out->write( |{ l_exception_ref->exception_class }
{ l_exception_ref->exception_id_name }| ).
            ENDIF.
          ENDLOOP.
        ENDLOOP.
      CATCH cx_bali_runtime INTO DATA(l_runtime_exception).
        out->write( l_runtime_exception->get_text(  ) ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

Example

Load a single log from the database which uses log handle l_handle. Only read the log header:

Sample Code

```
...
    DATA l_handle TYPE if_bali_log=>ty_handle.
    ...
    TRY.
        DATA(l_log) = cl_bali_log_db=>get_instance( )->load_log( handle =
l_handle

read_only_header = abap_true ).
      CATCH cx_bali_runtime INTO DATA(l_exception).
        out->write( l_exception->get_text(  ) ).
    ENDTRY.
...
```

Load all logs from the database which were created by the current user:

Sample Code

```
...
    TRY.
        DATA(l_filter) = cl_bali_log_filter=>create( )-
>set_create_info( user = sy-uname ).
        DATA(log_table) = cl_bali_log_db=>get_instance( )-
>load_logs_w_items_via_filter( filter = l_filter ).
      CATCH cx_bali_runtime INTO DATA(l_exception).
```

```
        out->write( l_exception->get_text(  ) ).
    ENDTRY.
...
```

4.2.15.10.2.2.3 Get Header Information from the Log

An application log contains header information and several attributes which identify the log. To read the header from a log, method `GET_HEADER` of interface `IF_BALI_LOG` can be used. It returns an object which uses interface `IF_BALI_HEADER_GETTER`.

Interface `IF_BALI_HEADER_GETTER` contains the following attributes:

- OBJECT: Log object which identifies the application to which the log belongs.
- SUBOBJECT: Log subobject which is the subcategory of an application log object.
- EXTERNAL_ID: A free text which is usually used by the application to identify the log. It contains, for example, the application document number.
- LOG_TIMESTAMP: UTC time stamp of the log (usually the creation time).
- LOG_USER: Log user (usually the user who created the log).
- EXPIRY_DATE: It defines the date when the log expires and can be deleted from the database.
- KEEP_UNTIL_EXPIRY: A flag which defines whether or not it is allowed to delete the log before the expiry date. If the flag is set, the standard reports which delete application logs do not delete the log before the expiry date. The application is still able to delete the log before this date.
- CONTEXT_STRUCTURE_NAME: The name of the dictionary structure which was used to set the context. See Get the Context [page 1525].
- NUMBER_ALL_ITEMS: Total number of items which are stored in the log.
- NUMBER_ABORT_ITEMS: Number of abort items which are stored in the log.
- NUMBER_ERROR_ITEMS: Number of error items which are stored in the log.
- NUMBER_WARNING_ITEMS: Number of warning items which are stored in the log.
- NUMBER_INFORMATION_ITEMS: Number of information items which are stored in the log.
- NUMBER_STATUS_ITEMS: Number of status items which are stored in the log.

Interface `IF_BALI_HEADER_GETTER` contains the following methods:

- GET_OBJECT_DESCRIPTION: Returns the description text of the log object in the logon language.
- GET_SUBOBJECT_DESCRIPTION: Returns the description text of the log subobject in the logon language.
- GET_CONTEXT: Get the context of the log. See Get the Context [page 1525].

> **Note**
>
> The header object contains the header attributes that were valid when the method GET_HEADER was called. If the log is changed after this call (for example, when new items are added to the log), these changes are not automatically visible in the header object. To refresh the header object it's necessary to call the method GET_HEADER again.

Example

Load a single log from the database which uses log handle l_handle, and output some fields of the log header:

Sample Code

```
...
 DATA l_handle TYPE if_bali_log=>ty_handle.
 ...
 TRY.
     DATA(l_log) = cl_bali_log_db=>get_instance( )->load_log( handle =
l_handle ).
     DATA(l_header) = l_log->get_header( ).
     out->write( |{ l_header->object } { l_header-
>get_object_description( ) } {
                   l_header->subobject } { l_header->external_id }
{ l_header->log_user }| ).
   CATCH cx_bali_runtime INTO DATA(l_exception).
     out->write( l_exception->get_text(  ) ).
 ENDTRY.
...
```

4.2.15.10.2.2.4 Get Items from the Log

After an application log was read from the database, the items stored in the log must be accessed. To read one or more items from an application log, interface `IF_BALI_LOG` provides the following methods:

- GET_ITEM: Read a single item from the application log.
  - The item is identified by the *Log Item Number* which is a serial number containing the position of the item in the log.
  - The method returns the reference to the item object.
- GET_ALL_ITEMS: Read all items from the application log.
  - The method returns an internal table with the following structure:
    - The *Log Item Number* of the item
    - The reference to the item object

The item object contains all attributes of the item, for example the severity. Each item object uses interface `IF_BALI_ITEM_GETTER`.

Public attributes of all log items:

- CATEGORY: Identifies the category of the item (see below).
- LOG_ITEM_NUMBER: The log item number which is the position of the item in the log.
- SEVERITY: Severity, such as 'Error', 'Warning', 'Information'. Possible values of the severity are defined as constants in interface `IF_BALI_CONSTANTS`.
- DETAIL_LEVEL: If the application outputs the items of the application log, the detail level can be used to define at which level of detail the item shall be visible.
- TIMESTAMP: A UTC timestamp which contains the creation time of the item.

- CONTEXT_STRUCTURE_NAME: The name of the dictionary structure which was used to set the context. See Get the Context [page 1525].

Methods of all log items:

- GET_MESSAGE_TEXT: It returns the message text of the item. It is:
  - The output of ABAP command **MESSAGE** for a message item
  - The text of a free text item
  - The output of method `GET_TEXT` of the exception object for an exception item
- GET_CONTEXT: Get the context of the item. See Get the Context [page 1525].

The category of the item restricts which other attributes of the item are available. For example, the message number is only available for a message, because a free text does not have a message number. To get additional attributes, a down cast to the more specific interface of the item category is required. The following item categories are supported:

- Message
  - Attribute `CATEGORY` contains the value `IF_BALI_CONSTANTS=>C_CATEGORY_MESSAGE`.
  - Interface of a message is `IF_BALI_MESSAGE_GETTER`.

  For more information, see Get a Message [page 1523].
- Free Text
  - Attribute CATEGORY contains the value IF_BALI_CONSTANTS=>C_CATEGORY_FREE_TEXT.
  - Interface of a free text is IF_BALI_FREE_TEXT_GETTER.

  For more information, see Get a Free Text [page 1523].
- Exception
  - Attribute CATEGORY contains the value IF_BALI_CONSTANTS=>C_CATEGORY_EXCEPTION.
  - Interface of an exception is IF_BALI_EXCEPTION_GETTER.

  For more information, see Get an Exception [page 1524].

Example

Load a single log from the database which uses log handle l_handle and output all items of the log:

Sample Code

```
...
 DATA l_handle TYPE if_bali_log=>ty_handle.
 ...
 TRY.
     DATA(l_log) = cl_bali_log_db=>get_instance( )->load_log( handle =
l_handle ).
     DATA(l_item_table) = l_log->get_all_items( ).
     LOOP AT l_item_table INTO DATA(l_item_entry).
       out->write( |{ l_item_entry-log_item_number } { l_item_entry-item-
>get_message_text( ) }| ).
       IF l_item_entry-item->category =
if_bali_constants=>c_category_message.
         DATA(l_message_ref) = CAST if_bali_message_getter( l_item_entry-
item ).
         out->write( |{ l_message_ref->id } { l_message_ref->number }| ).
       ELSEIF l_item_entry-item->category =
if_bali_constants=>c_category_exception.
         DATA(l_exception_ref) = CAST
if_bali_exception_getter( l_item_entry-item ).
         out->write( |{ l_exception_ref->exception_class }
{ l_exception_ref->exception_id_name }| ).
```

```
      ENDIF.
    ENDLOOP.
  CATCH cx_bali_runtime INTO DATA(l_exception).
    out->write( l_exception->get_text(  ) ).
ENDTRY.
...
```

4.2.15.10.2.2.4.1 Get a Free Text

Similar to all other items, a free text can be read from the application log using methods `GET_ITEM` and `GET_ALL_ITEMS` of interface `IF_BALI_LOG`. The free text object, which is returned by these methods, uses interface `IF_BALI_FREE_TEXT_GETTER`.

The interface supports all attributes and methods of interface `IF_BALI_ITEM_GETTER`.

Interface `IF_BALI_FREE_TEXT_GETTER` contains no additional attributes.

Example

Load a single log from the database which uses log handle l_handle and output the text of the first item:

Sample Code

```
...
 DATA l_handle TYPE if_bali_log=>ty_handle.
 ...
 TRY.
     DATA(l_log) = cl_bali_log_db=>get_instance( )->load_log( handle =
l_handle ).
     DATA(l_item) = l_log->get_item( log_item_number = '1' ).
     out->write( |{ l_item->get_message_text( ) }| ).
   CATCH cx_bali_runtime INTO DATA(l_exception).
     out->write( l_exception->get_text(  ) ).
 ENDTRY.
...
```

4.2.15.10.2.2.4.2 Get a Message

Similar to all other items, a message can be read from the application log using methods `GET_ITEM` and `GET_ALL_ITEMS` of interface `IF_BALI_LOG`. The message object, which is returned by these methods, uses interface `IF_BALI_MESSAGE_GETTER`.

The interface supports all attributes and methods of interface `IF_BALI_ITEM_GETTER`.

Interface `IF_BALI_MESSAGE_GETTER` contains the following additional attributes:

- ID: Message ID
- NUMBER: Message Number

- VARIABLE_1: Variable 1 of the message
- VARIABLE_2: Variable 2 of the message
- VARIABLE_3: Variable 3 of the message
- VARIABLE_4: Variable 4 of the message
- COUNT: The count of the message. It is increased, if method `CUMULATE_ITEM` of interface `IF_BALI_LOG` is used to add a message to the log and if the log already contains a message with the same message attributes (severity, message ID, message number and variables).

Example

Load a single log from the database which uses log handle l_handle and output some attributes of the first item, if it is a message:

Sample Code

```
...
 DATA l_handle TYPE if_bali_log=>ty_handle.
 ...
 TRY.
     DATA(l_log) = cl_bali_log_db=>get_instance( )->load_log( handle =
l_handle ).
     DATA(l_item) = l_log->get_item( log_item_number = '1' ).
     IF l_item->category = if_bali_constants=>c_category_message.
       DATA(l_message_ref) = CAST if_bali_message_getter( l_item ).
       out->write( |{ l_message_ref->id } { l_message_ref->number }| ).
     ENDIF.
   CATCH cx_bali_runtime INTO DATA(l_exception).
     out->write( l_exception->get_text(  ) ).
 ENDTRY.
...
```

4.2.15.10.2.2.4.3 Get an Exception

Similar to all other items, an exception can be read from the application log using methods `GET_ITEM` and `GET_ALL_ITEMS` of interface `IF_BALI_LOG`. The exception object, which is returned by these methods, uses interface `IF_BALI_EXCEPTION_GETTER`.

The interface supports all attributes and methods of interface `IF_BALI_ITEM_GETTER`.

Interface `IF_BALI_EXCEPTION_GETTER` contains the following additional attributes:

- EXCEPTION_CLASS: The name of the exception class.
- EXCEPTION_ID_NAME: Name of the text ID of the ABAP exception.

Example

Load a single log from the database which uses log handle l_handle and output some attributes of the first item, if it is an exception:

Sample Code

```
...
```

```
 DATA l_handle TYPE if_bali_log=>ty_handle.
 TRY.
     DATA(l_log) = cl_bali_log_db=>get_instance( )->load_log( handle =
l_handle ).
     DATA(l_item) = l_log->get_item( log_item_number = '1' ).
     IF l_item->category = if_bali_constants=>c_category_exception.
       DATA(l_exception_ref) = CAST if_bali_exception_getter( l_item ).
       out->write( |{ l_exception_ref->exception_class } { l_exception_ref-
>exception_id_name }| ).
     ENDIF.
   CATCH cx_bali_runtime INTO DATA(l_exception).
     out->write( l_exception->get_text(  ) ).
 ENDTRY.
...
```

4.2.15.10.2.2.4.4 Get the Context

To read the context data from the log header or from the log item, the following steps are necessary:

- You must define a structure which can be used to store the context data. It's recommended that you use the same structure type that was used when the context was set.
  - To get the data dictionary name of the structure type that was used to set the context, you can use the attribute `CONTEXT_STRUCTURE_NAME` of the header or item object.
- After this, you can use the method `GET_CONTEXT` of the header or item object to fill this context structure.
  - If you use a different structure type for getting the context than the one used for setting, only the identically named components are copied to each other.

Sample Code

```
...
 DATA: l_handle TYPE if_bali_log=>ty_handle.
 DATA: l_context TYPE ADDRESS_1. " the structure type which was used to set
the context
...
 TRY.
     DATA(l_log) = cl_bali_log_db=>get_instance( )->load_log( handle =
l_handle ).
     DATA(l_header) = l_log->get_header( ).
     l_header->get_context( IMPORTING context = l_context ).
     out->write( |{ l_context-name1 } { l_context-name2 }| ).
   CATCH cx_bali_runtime INTO DATA(l_exception).
     out->write( l_exception->get_text(  ) ).
 ENDTRY.
...
```

4.2.15.10.2.3 Classes and Interfaces of the Application Log API

The following classes and interfaces are available:

Access the Database

Class Name	Public Interface	Description
CL_BALI_LOG_DB	IF_BALI_LOG_DB	Handles database access like reading or writing of logs in the database.
CL_BALI_LOG_FILTER	IF_BALI_LOG_FILTER	Defines a filter for reading of logs from the database.

Access the Content of a Log

Class Name	Public Interface	Description
CL_BALI_LOG	IF_BALI_LOG	Reads and writes the header and items of a log

Writing the Log Header

Class Name	Public Interface	Description
CL_BALI_HEADER_SETTER	IF_BALI_HEADER_SETTER	Log header which can be put into a log

Reading the Log Header

Class Name	Public Interface	Description
	IF_BALI_HEADER_GETTER	Log header which was read from the log

Writing a Log Item

Class Name	Public Interface	Description
	IF_BALI_ITEM_SETTER	Each item contains this interface
CL_BALI_MESSAGE_SETTER	IF_BALI_MESSAGE_SETTER	Message which can be put into a log
CL_BALI_FREE_TEXT_SETTER	IF_BALI_FREE_TEXT_SETTER	Free text which can be put into a log
CL_BALI_EXCEPTION_SETTER	IF_BALI_EXCEPTION_SETTER	Exception which can be put into a log

Reading a Log Item

Class Name	Public Interface	Description
	IF_BALI_ITEM_GETTER	Each item contains this interface
	IF_BALI_MESSAGE_GETTER	Message which was read from the log

Class Name	Public Interface	Description
	IF_BALI_FREE_TEXT_GETTER	Free text which was read from the log
	IF_BALI_EXCEPTION_GETTER	Exception which was read from the log

Other Classes and Interfaces

Class Name	Public Interface	Description
	IF_BALI_CONSTANTS	Some constants, such as available item categories and severities
CL_BALI_ITEM_FILTER	IF_BALI_ITEM_FILTER	Define an item filter for adding items to a log

If one of the class methods can't be processed or can't return the requested results, an exception is raised. The following exceptions are possible, each of them inherit from exception class `CX_BALI_RUNTIME`:

Exception Classes

Class Name	Description
`CX_BALI_INVALID_PARAMETER`	An input parameter of the method is invalid (e.g. the log object doesn't exist).
`CX_BALI_NOT_FOUND`	The entry which shall be read or changed was not found.
`CX_BALI_NOT_POSSIBLE`	The requested processing is not possible. Possible values of class attribute `ERROR_CODE`: • `CX_BALI_NOT_POSSIBLE=>NO_AUTHORIZATION`: No authorization to access the log • `CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED`: Access to the log object is not allowed • `CX_BALI_NOT_POSSIBLE=>TOO_MANY_ITEMS`: The maximum number 999999 of items was reached • `CX_BALI_NOT_POSSIBLE=>SAVE_NOT_ALLOWED`: Error during saving to the database (e.g. database error or object is empty) • `CX_BALI_NOT_POSSIBLE=>ENTRY_IS_LOCKED`: The enqueue cannot be set, because the log is already locked
`CX_BALI_INTERNAL_ERROR`	Internal error during processing

> ### Note
>
> Find more information about classes and interfaces of the *Application Log API* in the ABAP development tools for Eclipse (ADT).

4.2.15.10.2.3.1 CL_BALI_LOG_DB (Interface IF_BALI_LOG_DB)

Class `CL_BALI_LOG_DB` handles all database accesses of the application logs. This includes the reading of one or several logs from the database, writing a log to the database and deleting a log from the database. In addition, it offers methods to set and clear an SAP enqueue on a log. The public interface of the instance methods is `IF_BALI_LOG_DB`.

Public Methods

Get an Instance of the Database Handler:

GET_INSTANCE (static)

Name	Description
Returning parameter	
DB_HANDLER	Database handler object: A reference to interface IF_BALI_LOG_DB

Load a single log from the database into the memory:

LOAD_LOG

Name	Description
Importing parameters	
HANDLE	Handle of the Application Log which shall be read
READ_ONLY_HEADER	(Optional): If set, only the header of the log is read (no items) Default: Not set
Returning parameter	
LOG	Log which was read from the database: Reference to interface IF_BALI_LOG
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_FOUND	• The log handle is initial • The log was not found in the database
CX_BALI_NOT_POSSIBLE	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED: Access to the log object is not allowed • ERROR_CODE: CX_BALI_NOT_POSSIBLE=>NO_AUTHORIZATION: No authorization to access the log
CX_BALI_INTERNAL_ERROR	Internal error during processing

> **Note**
>
> It is recommended to use method `LOAD_LOG_WITH_ITEMS` if all items should be loaded.

The following authorization is checked:

Authorization Object: S_APPL_LOG with:

- ALG_OBJECT: Object name of the application log
- ALG_SUBOBJ: Subobject of the application log
- ACTVT: 03

Load a single log including all items from the database into the memory:

LOAD_LOG_WITH_ITEMS

Name	Description
Importing parameters	
HANDLE	Handle of the application log that should be read
Returning parameter	
LOG	Log that was read from the database: Reference to interface IF_BALI_LOG
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_FOUND	• The log handle is initial • The log was not found in the database
CX_BALI_NOT_POSSIBLE	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED: Access to the log object is not allowed • ERROR_CODE: CX_BALI_NOT_POSSIBLE=>NO_AUTHORIZATION: No authorization to access the log
CX_BALI_INTERNAL_ERROR	Internal error during processing

Note

The following authorization is checked:

Authorization Object: S_APPL_LOG with:

- ALG_OBJECT: Object name of the application log
- ALG_SUBOBJ: Subobject of the application log
- ACTVT: 03

Load several logs via a filter from the database into the memory:

LOAD_LOGS_VIA_FILTER

Name	Description
Importing parameters	
FILTER	Log filter object: Reference to interface IF_BALI_LOG_FILTER

Name	Description
READ_ONLY_HEADER	(Optional): If set, only the headers of the logs are read (no items) Default: Not set
Returning parameter	
LOG_TABLE	Table of logs which were read from the database: Table of references to interface IF_BALI_LOG
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_FOUND	No log can be returned which fits to the filter criteria
CX_BALI_INVALID_PARAMETERS	• The filter contains invalid values • The filter is initial or empty
CX_BALI_INTERNAL_ERROR	Internal error during processing

> ### Note
>
> It is recommended to use method `LOAD_LOGS_W_ITEMS_VIA_FILTER` if all items should be loaded.
>
> The following authorization is checked:
>
> Authorization Object: S_APPL_LOG with:
>
> - ALG_OBJECT: Object name of the application log
> - ALG_SUBOBJ: Subobject of the application log
> - ACTVT: 03
>
> If the object of a log is not allowed or if the log doesn't pass the authorization check, it is removed from the table of logs which is returned. An exception is only raised if the final table is empty.

Load several logs including all items via a filter from the database into the memory:

LOAD_LOGs_W_ITEMS_VIA_FILTER

Name	Description
Importing parameters	
FILTER	Log filter object: Reference to interface IF_BALI_LOG_FILTER
Returning parameter	
LOG_TABLE	Table of logs which were read from the database: Table of references to interface IF_BALI_LOG
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_FOUND	No log can be returned that fits to the filter criteria

Name	Description
CX_BALI_INVALID_PARAMETERS	• The filter contains invalid values • The filter is initial or empty
CX_BALI_INTERNAL_ERROR	Internal error during processing

> **Note**
>
> The following authorization is checked:
>
> Authorization Object: S_APPL_LOG with:
>
> - ALG_OBJECT: Object name of the application log
> - ALG_SUBOBJ: Subobject of the application log
> - ACTVT: 03
>
> If the object of a log is not allowed or if the log doesn't pass the authorization check, it's removed from the table of logs that is returned. An exception is only raised if the final table is empty.

Save a single log to the database using the default database connection:

SAVE_LOG

Name	Description
Importing parameters	
LOG	Log that should be saved: Reference to interface IF_BALI_LOG
USE_2ND_DB_CONNECTION	Deprecated: Please use method SAVE_LOG_2ND_DB_CONNECTION instead
ASSIGN_TO_CURRENT_APPL_JOB	(Optional): If set, and if the application runs in an application job, a connection between the application log and an application job is established Default: Not set
Exceptions (inherit from CX_BALI_RUNTIME)	

Name	Description

CX_BALI_NOT_POSSIBLE	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED:
	Access to the log object is not allowed
	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>SAVE_NOT_ALLOWED:
	  • Log object and log subobject are empty
	  • Locking of the log is not possible
	  • Error during saving into the database
	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>LOG_WAS_INVALIDATED:
	The memory of the log was released. The log can't be used

CX_BALI_INTERNAL_ERROR	Internal error during processing

> **Note**
>
> If parameter LOG is initial, the method returns without exception.

Save a single log to the database using a service connection to the database:

SAVE_LOG_2ND_DB_CONNECTION

Name	Description
Importing parameters	
LOG	Log that should be saved: Reference to interface IF_BALI_LOG
ASSIGN_TO_CURRENT_APPL_JOB	(Optional): If set, and if the application runs in an application job, a connection between the application log and an application job is established Default: Not set
Exceptions (inherit from CX_BALI_RUNTIME)	

Name	Description

CX_BALI_NOT_POSSIBLE	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED:
	Access to the log object is not allowed
	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>SAVE_NOT_ALLOWED:
	  • Log object and log subobject are empty
	  • Locking of the log is not possible
	  • Error during saving into the database
	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>LOG_WAS_INVALIDATED:
	The memory of the log was released. The log can't be used

CX_BALI_INTERNAL_ERROR	Internal error during processing

> ### Note
>
> A commit is executed on the service connection after the saving.
>
> If parameter LOG is initial, the method returns without exception.

Delete a single log from the database:

DELETE_LOG

Name	Description
Importing parameters	
LOG	Log which shall be saved: Reference to interface IF_BALI_LOG
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED: Access to the log object is not allowed • ERROR_CODE: CX_BALI_NOT_POSSIBLE=>NO_AUTHORIZATION: No authorization to access the log • ERROR_CODE: CX_BALI_NOT_POSSIBLE=>LOG_WAS_INVALIDATED: The memory of the log was released. The log can't be used
CX_BALI_NOT_FOUND	The log was not found in the database
CX_BALI_INTERNAL_ERROR	Internal error during processing

> **Note**
>
> - If parameter LOG is initial, the method returns without exception.
> - The following authorization is checked:
>   Authorization Object: S_APPL_LOG with:
>   - ALG_OBJECT: Object name of the application log
>   - ALG_SUBOBJ: Subobject of the application log
>   - ACTVT: 06

Set an SAP enqueue on a log:

ENQUEUE

Name	Description
Importing parameters	
LOG	Log which shall be saved: Reference to interface IF_BALI_LOG
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED: Access to the log object is not allowed • ERROR_CODE: CX_BALI_NOT_POSSIBLE=>ENTRY_IS_LOCKED: The enqueue can't be set, because the log is already locked • ERROR_CODE: CX_BALI_NOT_POSSIBLE=>LOG_WAS_INVALIDATED: The memory of the log was released. The log can't be used
CX_BALI_INTERNAL_ERROR	Internal error during processing

> **Note**
>
> If parameter LOG is initial, the method returns without exception.

Clear an SAP enqueue from a log:

DEQUEUE

Name	Description
Importing parameters	
LOG	Log which shall be saved: Reference to interface IF_BALI_LOG
Exceptions (inherit from CX_BALI_RUNTIME)	

Name	Description
CX_BALI_NOT_POSSIBLE	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED: Access to the log object is not allowed • ERROR_CODE: CX_BALI_NOT_POSSIBLE=>LOG_WAS_INVALIDATED: The memory of the log was released. The log can't be used
CX_BALI_INTERNAL_ERROR	Internal error during processing

> ⓘ Note
>
> If parameter LOG is initial, the method returns without exception.

4.2.15.10.2.3.2 CL_BALI_LOG_FILTER (Interface IF_BALI_LOG_FILTER)

Class `CL_BALI_LOG_FILTER` allows to define a filter which can be used if logs shall be read from the database and if the log handles are not known. The public interface of the instance methods is `IF_BALI_LOG_FILTER`.

Public Methods

Create an instance of the filter class:

CREATE (static)

Name	Description
Returning parameter	
FILTER	Filter object: A reference to interface IF_BALI_LOG_FILTER

Set object, subobject and external identifier of the log. It overwrites all previous filter settings of object, subobject and external identifier:

SET_DESCRIPTOR

Name	Description
Importing parameters	
OBJECT	(Optional): Object of the log (no wildcards)
SUBOBJECT	(Optional): Subobject of the log (wildcards allowed)
SUBOBJECT_TABLE	(Optional): Table with subobjects (wildcards allowed)

Name	Description
EXTERNAL_ID	(Optional): External identifier of the log (wildcards allowed)
EXTERNAL_ID_TABLE	(Optional): Table with external identifiers (wildcards allowed)
Returning parameter	
NEW_FILTER	Reference to current filter object
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED: Access to the log object is not allowed

> **Note**
>
> If parameter EXTERNAL_ID is supplied and empty, the filter searches for logs with empty external identifier.
>
> If parameters OBJECT, SUBOBJECT, SUBOBJECT_TABLE, or EXTERNAL_ID_TABLE are supplied and empty, they are ignored

Set information about the log creation like the user. It overwrites all previous filter settings about the log creation:

SET_CREATE_INFO

Name	Description
Importing parameters	
USER	(Optional): Log user (wildcards allowed)
USER_TABLE	(Optional): Table with log users (wildcards allowed)
Returning parameter	
NEW_FILTER	Reference to current filter object

> **Note**
>
> If parameters USER or USER_TABLE are supplied and empty, they are ignored.

Set the date and time interval of the log creation. It overwrites all previous filter settings of the time interval:

SET_TIME_INTERVAL

Name	Description
Importing parameters	
START_TIME	UTC time stamp of the start time of the time interval

Name	Description
END_TIME	UTC time stamp of the end time of the time interval
Returning parameter	
NEW_FILTER	Reference to current filter object

Set the maximum number of logs which are processed:

SET_MAXIMUM_LOG_NUMBER

Name	Description
Importing parameters	
MAX_LOG_NUMBER	Maximum number of logs which are processed (0 = all logs are processed which is the default)
Returning parameter	
NEW_FILTER	Reference to current filter object

Get all filter values:

GET_ALL_VALUES

Name	Description
Exporting parameters	
OBJECT_TABLE	Table of objects (no wildcards)
SUBOBJECT_TABLE	Table of subobjects (wildcards allowed)
EXTERNAL_ID_TABLE	Table of external identifiers (wildcards allowed)
USER_TABLE	
TIME_INTERVAL	Date and time interval
MAX_LOG_NUMBER	Maximum number of logs processed

4.2.15.10.2.3.3 CL_BALI_LOG (Interface IF_BALI_LOG)

Class `CL_BALI_LOG` handles all read and change operations on a single application log. It contains methods to read and change the log header. In addition, it allows to read items from the log and to add items to the log. The public interface of the instance methods is `IF_BALI_LOG`.

Public Methods

Create an instance of the log class:

CREATE (static)

Name	Description
Returning parameter	
LOG	Log object: A reference to interface IF_BALI_LOG
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_INTERNAL_ERROR	Internal error during processing

Create an instance of the log class and set the header:

CREATE_WITH_HEADER (static)

Name	Description
Importing parameter	
HEADER	Header which is put into the log: Reference to interface IF_BALI_HEADER_SETTER
Returning parameter	
LOG	Log object: A reference to interface IF_BALI_LOG
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED: Access to the log object of the header is not allowed
CX_BALI_INVALID_PARAMETER	The log object or subobject of the header don't exist
CX_BALI_INTERNAL_ERROR	Internal error during processing

Get the log handle which is the unique identifier of the log:

GET_HANDLE

Name	Description
Returning parameter	
HANDLE	Log handle

Get the log header:

GET_HEADER

Name	Description
Returning parameter	
HEADER	Log header: References to interface IF_BALI_HEADER_GETTER

Name	Description
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	ERROR_CODE: CX_BALI_NOT_POSSIBLE=>LOG_WAS_IN-VALIDATED: The memory of the log was released. The log can't be used
CX_BALI_INTERNAL_ERROR	Internal error during processing

Set the log header:

SET_HEADER

Name	Description
Importing parameter	
HEADER	Header which is put into the log: References to interface IF_BALI_HEADER_SETTER
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OB-JECT_NOT_ALLOWED: Access to the log object of the header is not allowed • ERROR_CODE: CX_BALI_NOT_POSSI-BLE=>LOG_WAS_INVALIDATED: The memory of the log was released. The log can't be used
CX_BALI_INVALID_PARAMETER	The log object or subobject of the header don't exist
CX_BALI_INTERNAL_ERROR	Internal error during processing

Add an item (e.g. a message) to the log:

ADD_ITEM

Name	Description
Importing parameter	
ITEM	Item which is added: Reference to interface IF_BALI_ITEM_SETTER
Exceptions (inherit from CX_BALI_RUNTIME)	

Name	Description
CX_BALI_NOT_POSSIBLE	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED: Access to the log object of the header is not allowed • ERROR_CODE: CX_BALI_NOT_POSSIBLE=>LOG_WAS_INVALIDATED: The memory of the log was released. The log can't be used
CX_BALI_INVALID_PARAMETER	The log object or subobject of the header don't exist
CX_BALI_INTERNAL_ERROR	Internal error during processing

> **Note**
>
> An exception may contain a message. This means that the ABAP exception class contains interface `IF_T100_MESSAGE`, or it contains at least the attributes `T100_MSGID` and `T100_MSGNO`. In this case, the exception is internally converted into a message before it is added to the log.

If the item is a message, it's checked whether the log already contains another message with identical message attributes. These are the attributes: severity, message ID, message number and message variable 1 - 4. If this message exist, the message counter of the message is increased by 1. Otherwise, a new message is added to the log. Also free texts and exceptions are always added to the log without cumulation.

CUMULATE_ITEM

Name	Description
Importing parameter	
ITEM	Item which is cumulated or added: Reference to interface IF_BALI_ITEM_SETTER
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED: Access to the log object of the header is not allowed • ERROR_CODE: CX_BALI_NOT_POSSIBLE=>LOG_WAS_INVALIDATED: The memory of the log was released. The log can't be used
CX_BALI_INVALID_PARAMETER	The log object or subobject of the header don't exist
CX_BALI_INTERNAL_ERROR	Internal error during processing

> **Note**
>
> An exception may contain a message. This means that the ABAP exception class contains interface `IF_T100_MESSAGE`, or it contains at least the attributes `T100_MSGID` and `T100_MSGNO`. In this case,

the exception is internally converted to a message before it is added to the log (but this message is not cumulated to an already existing message).

Add all messages from an internal table of type `BAPIRETTAB` table to the log:

ADD_MESSAGES_FROM_BAPIRETTAB

Name	Description
Importing parameter	
MESSAGE_TABLE	An internal table with messages which use type BAPIRET-TAB
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	• ERROR_CODE: CX_BALI_NOT_POSSI-BLE=>TOO_MANY_ITEMS: The maximum number 999999 of items was reached • ERROR_CODE: CX_BALI_NOT_POSSI-BLE=>LOG_WAS_INVALIDATED: The memory of the log was released. The log can't be used
CX_BALI_INVALID_PARAMETER	The message ID of one of the messages is initial
CX_BALI_INTERNAL_ERROR	Internal error during processing

Note

If a message of the message table can't be added to the log, it's skipped and the processing continues until the end of the table is reached. Afterwards, an exception is raised to notify the caller that some of the messages couldn't be added to the log.

Add a message from the ABAP Restful Application Programming Model (interface IF_ABAP_BEHV_MESSAGE) to the log:

ADD_ABAP_BEHAVIOR_MESSAGE

Name	Description
Importing parameter	
MESSAGE	A reference to the interface IF_ABAP_BEHV_MESSAGE
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	• ERROR_CODE: CX_BALI_NOT_POSSI-BLE=>TOO_MANY_ITEMS: The maximum number 999999 of items was reached • ERROR_CODE: CX_BALI_NOT_POSSI-BLE=>LOG_WAS_INVALIDATED: The memory of the log was released. The log can't be used
CX_BALI_INVALID_PARAMETER	The message ID of the messages is initial

Name	Description
CX_BALI_INTERNAL_ERROR	Internal error during processing

Add all items from another log to this log:

ADD_ALL_ITEMS_FROM_OTHER_LOG

Name	Description
Importing parameter	
SOURCE_LOG	Reference to the log whose items are to be copied
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	• ERROR_CODE: CX_BALI_NOT_POSSIBLE=>TOO_MANY_ITEMS: The maximum number of 999999 items was reached • ERROR_CODE: CX_BALI_NOT_POSSIBLE=>LOG_WAS_INVALIDATED: The memory of the log was released. The log can't be used
CX_BALI_INVALID_PARAMETER	The source log is invalid. It contains a log handle which doesn't exist
CX_BALI_INTERNAL_ERROR	Internal error during processing

> **Note**
>
> If the parameter `SOURCE_LOG` is initial, the processing is skipped and no exception is raised.

Add an item filter to the log. The item filter is checked before an item is added to the log using the methods ADD_ITEM, CUMULATE_ITEM, ADD_MESSAGES_FROM_BAPIRETTAB, or ADD_ABAP_BEHAVIOR_MESSAGE. If an item doesn't pass the filter, it's ignored.

SET_FILTER_FOR_ADD_ITEM

Name	Description
Importing parameter	
FILTER	A reference to the item filter
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	ERROR_CODE: CX_BALI_NOT_POSSIBLE=>LOG_WAS_INVALIDATED: The memory of the log was released. The log can't be used

> **Note**
>
> • If FILTER is initial, any item filter is removed from the log. The log then no longer uses an item filter.
> • Since the method ADD_ALL_ITEMS_FROM_OTHER_LOG is intended for the mass-processing of items, it ignores the item filter because of performance reasons.

- The filter is only used by methods of the class CL_BALI_LOG. If an application uses the application log function modules to add an item to the log, the filter is ignored.

Get the item filter which was set using SET_FILTER_FOR_ADD_ITEM.

GET_FILTER_FOR_ADD_ITEM

Name	Description
Returning parameter	
FILTER	A reference to the item filter which is currently used by the log
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	ERROR_CODE: CX_BALI_NOT_POSSIBLE=>LOG_WAS_INVALIDATED: The memory of the log was released. The log can't be used

Get a single item from the log:

GET_ITEM

Name	Description
Importing parameter	
LOG_ITEM_NUMBER	Serial number of the item which shall be read (it is the position of the item in the log)
Returning parameter	
ITEM	Item which was read: Reference to interface IF_BALI_ITEM_GETTER
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_FOUND	An item with the requested log item number does not exist
CX_BALI_NOT_POSSIBLE	ERROR_CODE: CX_BALI_NOT_POSSIBLE=>LOG_WAS_INVALIDATED: The memory of the log was released. The log can't be used
CX_BALI_INTERNAL_ERROR	Internal error during processing

Get all items from the log:

GET_ALL_ITEMS

Name	Description
Returning parameter	

Name	Description
ITEM_TABLE	Table of all items which are stored in the log. The table has the following structure: • LOG_ITEM_NUMBER: The serial number of the item in the log • ITEM: Item object: Reference to interface IF_BALI_ITEM_GETTER
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_FOUND	No item was found in the log
CX_BALI_NOT_POSSIBLE	ERROR_CODE: CX_BALI_NOT_POSSIBLE=>LOG_WAS_INVALIDATED: The memory of the log was released. The log can't be used
CX_BALI_INTERNAL_ERROR	Internal error during processing

RELEASE_MEMORY:

Remove the log from the memory. After this, the log is invalidated and can no longer be used.

Check whether the memory of the log was released in order to verify that the log is invalidated and can no longer be used.

IS_INVALIDATED

Name	Description
Returning parameter	
IS_INVALIDATED	If set, the log is invalidated and can no longer be used

4.2.15.10.2.3.4 CL_BALI_HEADER_SETTER (Interface IF_BALI_HEADER_SETTER)

Class `CL_BALI_HEADER_SETTER` is used to set the header attributes of an application log, such as the log object, subobject, and the external identifier. The public interface of the instance methods is `IF_BALI_HEADER_SETTER`.

Public Methods

Create an instance of the header class with the settings of the descriptor (object, subobject and external identifier):

CREATE (static)

Name	Description
Importing parameters	
OBJECT	Object of the log
SUBOBJECT	Subobject of the log If no subobjects were defined for the log object, the parameter should be empty
EXTERNAL_ID	(Optional): External identifier of the log Default: ' '
Returning parameter	
HEADER	Header object: A reference to interface IF_BALI_HEADER_SETTER
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED: Access to the log object is not allowed
CX_BALI_INVALID_PARAMETER	The log object or subobject of the header doesn't exist

Set object, subobject and external identifier of the log:

SET_DESCRIPTOR

Name	Description
Importing parameters	
OBJECT	Object of the log
SUBOBJECT	Subobject of the log
EXTERNAL_ID	(Optional): External identifier of the log Default: ' '
Returning parameter	
NEW_HEADER	Reference to current header object
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_NOT_POSSIBLE	ERROR_CODE: CX_BALI_NOT_POSSIBLE=>OBJECT_NOT_ALLOWED: Access to the log object is not allowed
CX_BALI_INVALID_PARAMETER	The log object or subobject of the header doesn't exist

Set the expiry date and attributes:

SET_EXPIRY

Name	Description
Importing parameters	
EXPIRY_DATE	Date when the log expires and can be deleted. If the parameter is initial, the expiry date remains unchanged; the default value of the expiry date is the creation date of the log + 7 days.
KEEP_UNTIL_EXPIRY	(Optional): If set: It is not allowed to delete the log before the expiry date Default: Not set
Returning parameter	
NEW_HEADER	Reference to current header object

Set the log context using a data dictionary structure:

SET_CONTEXT

Name	Description
Importing parameter	
CONTEXT	Structure with the context data
Returning parameters	
NEW_HEADER	Reference to current header object
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_INVALID_PARAMETER	• Field CONTEXT is not based on a database table or structure which is defined in the data dictionary • The total size of field CONTEXT is longer than 256 characters

Get all header values:

GET_ALL_VALUES

Name	Description
Exporting parameters	
OBJECT	Object of the log
SUBOBJECT	Subobject of the log
EXTERNAL_ID	External identifier of the log

Name	Description
EXPIRY_DATE	Date when the log expires and can be deleted
KEEP_UNTIL_EXPIRY	If set: It is not allowed to delete the log before the expiry date
CONTEXT_DATA	Structure and content of the context. `CONTEXT_DATA` has the following fields: • `STRUCTURE_NAME`: Name of the data dictionary structure which was used for the context data • `CONTENT`: The content of the context data, stored in a character field with a length of 256 characters

4.2.15.10.2.3.5 CL_BALI_MESSAGE_SETTER (Interface IF_BALI_MESSAGE_SETTER)

To add a new message to an application log an object of class *CL_BALI_MESSAGE_SETTER* is used. The public interface of the instance methods is *IF_BALI_MESSAGE_SETTER*. It contains the interface *IF_BALI_ITEM_SETTER*.

Public Attributes

Name	Description
CATEGORY (from IF_BALI_ITEM_SETTER)	Category of the item Contains fixed value: IF_BALI_CONSTANTS=>C_CATEGORY_MESSAGE

Public Methods

Create a message class instance and set the message attributes:

CREATE (static)

Name	Description
Importing parameters	
SEVERITY	(Optional): Severity of the message ('Error', 'Warning', etc) Default: IF_BALI_CONSTANTS=>C_SEVERITY_STATUS
ID	Message ID
NUMBER	Message number
VARIABLE_1	(Optional): Message variable 1

Name	Description
VARIABLE_2	(Optional): Message variable 2
VARIABLE_3	(Optional): Message variable 3
VARIABLE_4	(Optional): Message variable 4
Returning parameter	
MESSAGE	Message object: A reference to interface IF_BALI_MESSAGE_SETTER

Create a message class instance. The message attributes are read from the fields of structure SY (like SY-MSGID):

CREATE_FROM_SY (static)

Name	Description
Returning parameter	
MESSAGE	Message object: A reference to interface IF_BALI_MESSAGE_SETTER

Create a message class instance. The message attributes are read from a structure of type BAPIRET2:

CREATE_FROM_BAPIRET2 (static)

Name	Description
Importing parameter	
MESSAGE_DATA	Message attributes (a structure of type BAPIRET2)
Returning parameter	
MESSAGE	Message object: A reference to interface IF_BALI_MESSAGE_SETTER

Set attributes of the message:

SET_ATTRIBUTES

Name	Description
Importing parameters	
SEVERITY	(Optional): Severity of the message ('Error', 'Warning', etc) Default: IF_BALI_CONSTANTS=>C_SEVERITY_STATUS
ID	Message ID
NUMBER	Message number

Name	Description
VARIABLE_1	(Optional): Message variable 1
VARIABLE_2	(Optional): Message variable 2
VARIABLE_3	(Optional): Message variable 3
VARIABLE_4	(Optional): Message variable 4
Returning parameter	
NEW_MESSAGE	Reference to current message object

Set message attributes from the fields of structure SY (like SY-MSGID):

SET_FROM_SY

Name	Description
Returning parameter	
NEW_MESSAGE	Reference to current message object

Set message attributes from structure BAPIRET2:

SET_FROM_BAPIRET2

Name	Description
Importing parameter	
MESSAGE_DATA	Message attributes (a structure of type BAPIRET2)
Returning parameter	
NEW_MESSAGE	Reference to current message object

Set the level of detail of the item:

SET_DETAIL_LEVEL

Name	Description
Importing parameter	
DETAIL_LEVEL	Detail level of the item Allowed values: Number between '1' and '9' or ' '
Returning parameter	
NEW_MESSAGE	Reference to current message object

Set the message context using a data dictionary structure:

SET_CONTEXT

Name	Description
Importing parameters	
CONTEXT	Structure with the context data
NEW_MESSAGE	Reference to the current message object
CX_BALI_INVALID_PARAMETER	• Field `CONTEXT` is not based on a database table or structure that is defined in the data dictionary • The total size of field `CONTEXT` is longer than 256 characters

Get all message values:

GET_ALL_VALUES

Name	Description
Exporting parameters	
DETAIL_LEVEL	Detail Level of the item (number between '1' and '9' or ' ')
SEVERITY	Severity of the message ('Error', 'Warning', etc)
ID	Message ID
NUMBER	Message number
VARIABLE_1	(Optional): Message variable 1
VARIABLE_2	(Optional): Message variable 2
VARIABLE_3	(Optional): Message variable 3
VARIABLE_4	(Optional): Message variable 4
CONTEXT_DATA	Structure and content of the context. `CONTEXT_DATA` has the following fields: • `STRUCTURE_NAME`: Name of the data dictionary structure which was used for the context data • `CONTENT`: The content of the context data. The content is stored in a character field with a length of 256 characters

Check whether the message can pass an item filter:

CHECK_PASSING_ITEM_FILTER (from IF_BALI_ITEM_SETTER)

Name	Description
Importing parameter	

Name	Description
ITEM_FILTER	Reference to the item filter that is being checked
Returning parameter	
FILTER_PASSED	If set, the message can pass the item filter

> **Note**
>
> If the severity of the message contains a value which is not allowed, it is changed to `IF_BALI_CONSTANTS=>C_SEVERITY_DEFAULT`. Allowed values of the severity can be found in interface `IF_BALI_CONSTANTS`.
>
> If the detail level of the message contains a value which is not allowed, it is changed to `IF_BALI_CONSTANTS=>C_DETAIL_LEVEL_DEFAULT`. Allowed values of the detail level are a number between '1' and '9' and ' '.
>
> If the message attributes are set from structure `BAPIRET2`, only the fields TYPE, ID, NUMBER, MESSAGE_V1, MESSAGE_V2, MESSAGE_V3 and MESSAGE_V4 of the structure are considered.

4.2.15.10.2.3.6 CL_BALI_FREE_TEXT_SETTER (Interface IF_BALI_FREE_TEXT_SETTER)

To add a new free text to an application log, an object of class `CL_BALI_FREE_TEXT_SETTER` is used. The public interface of the instance methods is `IF_BALI_FREE_TEXT_SETTER`. It contains the interface `IF_BALI_ITEM_SETTER`.

Public Attributes

Name	Description
CATEGORY	Category of the item
(from IF_BALI_ITEM_SETTER)	Contains fixed value: IF_BALI_CONSTANTS=>C_CATEGORY_FREE_TEXT

Public Methods

Create an instance of the free text class and set the text and the severity:

CREATE (static)

Name	Description
Importing parameters	
SEVERITY	(Optional): Severity of the message ('Error', 'Warning', etc) Default: IF_BALI_CONSTANTS=>C_SEVERITY_STATUS

Name	Description
TEXT	Free text
Returning parameter	
FREE_TEXT	Free text object: A reference to interface IF_BALI_FREE_TEXT_SETTER

Set the free text and severity:

SET_TEXT

Name	Description
Importing parameters	
SEVERITY	(Optional): Severity of the message ('Error', 'Warning', etc) Default: IF_BALI_CONSTANTS=>C_SEVERITY_STATUS
TEXT	Free text
Returning parameter	
NEW_FREE_TEXT	Reference to current free text object

Set the level of detail of the free text:

SET_DETAIL_LEVEL

Name	Description
Importing parameter	
DETAIL_LEVEL	Detail level of the free text Allowed values: Number between '1' and '9' or ' '
Returning parameter	
NEW_FREE_TEXT	Reference to current free text object

Set the free text context using a data dictionary structure:

SET_CONTEXT

Name	Description
Importing parameter	
CONTEXT	Structure with the context data
Returning parameter	
NEW_FREE_TEXT	Reference to the current free text object
Exceptions (inherit from CX_BALI_RUNTIME)	

Name	Description
CX_BALI_INVALID_PARAMETER	• Field `CONTEXT` is not based on a database table or structure that is defined in the data dictionary • The total size of field `CONTEXT` is longer than 256 characters

Get all free text values:

GET_ALL_VALUES

Name	Description
Exporting parameters	
DETAIL_LEVEL	Detail level of the free text Number between '1' and '9' or ' '
SEVERITY	Severity of the free text ('Error', 'Warning', etc)
TEXT	Content of the free text
CONTEXT_DATA	Structure and content of the context. `CONTEXT_DATA` has the following fields: • `STRUCTURE_NAME`: Name of the data dictionary structure which was used for the context data • `CONTENT`: The content of the context data. The content is stored in a character field with a length of 256 characters

Check whether the free text can pass an item filter:

CHECK_PASSING_ITEM_FILTER (from IF_BALI_ITEM_SETTER)

Name	Description
Importing parameter	
ITEM_FILTER	Reference to the item filter that is being checked
Returning parameter	
FILTER_PASSED	If set, the free text can pass the item filter

> ### Note
>
> If the severity of the free text contains a value which is not allowed, it is changed to `IF_BALI_CONSTANTS=>C_SEVERITY_DEFAULT`. Allowed values of the severity can be found in interface `IF_BALI_CONSTANTS`.
>
> If the detail level of the free text contains a value which is not allowed, it is changed to `IF_BALI_CONSTANTS=>C_DETAIL_LEVEL_DEFAULT`. Allowed values of the detail level are a number between '1' and '9' and ' '.

4.2.15.10.2.3.7 CL_BALI_EXCEPTION_SETTER (Interface IF_BALI_EXCEPTION_SETTER)

To add a new exception to an application log, an object of class `CL_BALI_EXCEPTION_SETTER` is used. The public interface of the instance methods is `IF_BALI_EXCEPTION_SETTER`. It contains the interface `IF_BALI_ITEM_SETTER`.

Public Attributes

Name	Description
CATEGORY	Category of the item
(from IF_BALI_ITEM_SETTER)	Contains fixed value: IF_BALI_CONSTANTS=>C_CATEGORY_EXCEPTION

Public Methods

Create an instance of the exception class. Set the reference of the ABAP exception which is stored in the exception item and its severity:

CREATE (static)

Name	Description
Importing parameters	
SEVERITY	(Optional): Severity of the message ('Error', 'Warning', etc) Default: IF_BALI_CONSTANTS=>C_SEVERITY_STATUS
EXCEPTION	Reference to ABAP exception class instance
Returning parameter	
EXCEPTION_OBJ	Exception object: A reference to interface IF_BALI_EXCEPTION_SETTER

Set the ABAP exception class instance and severity:

SET_EXCEPTION

Name	Description
Importing parameters	
SEVERITY	(Optional): Severity of the message ('Error', 'Warning', etc) Default: IF_BALI_CONSTANTS=>C_SEVERITY_STATUS
EXCEPTION	Reference to ABAP exception class instance
Returning parameter	
NEW_EXCEPTION_OBJ	Reference to current application log exception object

Set the level of detail of the exception:

SET_DETAIL_LEVEL

Name	Description
Importing parameter	
DETAIL_LEVEL	Detail level of the exception Allowed values: Number between '1' and '9' or ' '
Returning parameter	
NEW_EXCEPTION_OBJ	Reference to current application log exception object

Set the exception context using a data dictionary structure:

SET_CONTEXT

Name	Description
Importing parameter	
CONTEXT	Structure with the context data
Returning parameter	
NEW_EXCEPTION_OBJ	Reference to the current application log exception object
Exceptions (inherit from CX_BALI_RUNTIME)	
CX_BALI_INVALID_PARAMETER	• Field `CONTEXT` is not based on a database table or structure that is defined in the data dictionary • The total size of field `CONTEXT` is longer than 256 characters

Get all exception values:

GET_ALL_VALUES

Name	Description
Exporting parameters	
DETAIL_LEVEL	Detail level of the exception (number between '1' and '9' or ' ')
SEVERITY	Severity of the exception ('Error', 'Warning', etc)
EXCEPTION	Reference to ABAP exception class instance

Name	Description
CONTEXT_DATA	Structure and content of the context. `CONTEXT_DATA` has the following fields: • `STRUCTURE_NAME`: Name of the data dictionary structure which was used for the context data • `CONTENT`: The content of the context data. The content is stored in a character field with a length of 256 characters

Check whether the exception can pass an item filter:

CHECK_PASSING_ITEM_FILTER (from IF_BALI_ITEM_SETTER)

Name	Description
Importing parameter	
ITEM_FILTER	Reference to the item filter that is being checked
Returning parameter	
FILTER_PASSED	If set, the exception can pass the item filter

> ### Note
>
> If the severity of the exception contains a value which is not allowed, it is changed to `IF_BALI_CONSTANTS=>C_SEVERITY_DEFAULT`. Allowed values of the severity can be found in interface `IF_BALI_CONSTANTS`.
>
> If the detail level of the exception contains a value which is not allowed, it is changed to `IF_BALI_CONSTANTS=>C_DETAIL_LEVEL_DEFAULT`. Allowed values of the detail level are a number between '1' and '9' and ' '.

4.2.15.10.2.3.8 CL_BALI_ITEM_FILTER (Interface IF_BALI_ITEM_FILTER)

The class `CL_BALI_ITEM_FILTER` allows you to define an item filter. You can use it if it's necessary to restrict which items are added to an application log. The public interface of the instance methods is `IF_BALI_ITEM_FILTER`.

Public Methods

Create an instance of the filter class:

CREATE (static)

Name	Description
Returning parameter	
FILTER	Reference to the interface `IF_BALI_ITEM_FILTER`

Set a table of all severity values that pass the filter. It overwrites all previous filter settings related to the severity:

SET_SEVERITY

Name	Description
Importing parameter	
SEVERITY_TABLE	Table with severity values (such as *Error* or *Warning*)
Returning parameter	
NEW_FILTER	Reference to the current filter object

> ### Note
>
> - Valid values of the severity can be found in the interface `IF_BALI_CONSTANTS`
> - If the parameter `SEVERITY_TABLE` doesn't contain an entry, all severity values pass the filter
> - If the parameter `SEVERITY_TABLE` only contains invalid entries, no severity passes the filter

Apply the filter to a set of log item attributes and check whether it passes the filter:

APPLY_FILTER

Name	Description
Importing parameter	
SEVERITY	Severity of the log item that is checked
Returning parameter	
FILTER_PASSED	If set, the set of attributes passes the item filter

4.2.15.10.2.3.9 IF_BALI_HEADER_GETTER

If the header attributes of an application log are read, they are returned in an object which uses interface `IF_BALI_HEADER_GETTER`.

Public Attributes

Name	Description
OBJECT	Object of the log

Name	Description
SUBOBJECT	Subobject of the log
EXTERNAL_ID	External identifier of the log
LOG_TIMESTAMP	UTC time stamp of the log (usually the creation time)
LOG_USER	Log user (usually the user who created the log)
EXPIRY_DATE	Date when the log expires and can be deleted
KEEP_UNTIL_EXPIRY	If set: It's not allowed to delete the log before the expiry date
NUMBER_ALL_ITEMS	Total number of items which are stored in the log
NUMBER_ABORT_ITEMS	Number of abort items which are stored in the log
NUMBER_ERROR_ITEMS	Number of error items which are stored in the log
NUMBER_WARNING_ITEMS	Number of warning items which are stored in the log
NUMBER_INFORMATION_ITEMS	Number of information items which are stored in the log
NUMBER_STATUS_ITEMS	Number of status items which are stored in the log
CONTEXT_STRUCTURE_NAME	Data dictionary name of the context structure

Public Methods

Get description text of the log object in the logon language:

GET_OBJECT_DESCRIPTION

Name	Description
Returning parameter	
OBJECT_DESCRIPTION	Description of the object in the logon language

Get description text of log subobject in the logon language:

GET_SUBOBJECT_DESCRIPTION

Name	Description
Returning parameter	
SUBOBJECT_DESCRIPTION	Description of the subobject in the logon language

Get the context data of the header:

GET_CONTEXT

Name	Description
Exporting parameter	
CONTEXT	Structure that is filled with context data

4.2.15.10.2.3.10 IF_BALI_EXCEPTION_GETTER

If an exception is read from an application log, an object instance of interface `IF_BALI_EXCEPTION_GETTER` is returned. It contains the interface `IF_BALI_ITEM_GETTER`.

Public Attributes

Name	Description
CATEGORY (from IF_BALI_ITEM_GETTER)	Category of the item Contains fixed value: IF_BALI_CONSTANTS=>C_CATEGORY_EXCEPTION
LOG_ITEM_NUMBER (from IF_BALI_ITEM_GETTER)	Serial number which is the position of the exception in the log
SEVERITY (from IF_BALI_ITEM_GETTER)	Severity of the exception ('Error', 'Warning', etc)
DETAIL_LEVEL (from IF_BALI_ITEM_GETTER)	Detail level of the exception (number between '1' and '9' or ' ')
TIMESTAMP (from IF_BALI_ITEM_GETTER)	UTC time stamp of the exception creation
CONTEXT_STRUCTURE_NAME (from IF_BALI_ITEM_GETTER)	Data dictionary name of the context structure
EXCEPTION_CLASS	Name of the ABAP exception class
EXCEPTION_ID_NAME	Name of the Text ID of the ABAP exception

Public Methods

Get the message short text of the exception (the output of method GET_TEXT of the ABAP exception object):

GET_MESSAGE_TEXT (from interface IF_BALI_ITEM_GETTER)

Name	Description
Returning parameter	

Name	Description
MESSAGE_TEXT	Message short text of the exception in the logon language

Get the context data of the exception:

GET_CONTEXT (from the interface IF_BALI_ITEM_GETTER)

Name	Description
Exporting parameter	
CONTEXT	Structure that is filled with context data

4.2.15.10.2.3.11 IF_BALI_FREE_TEXT_GETTER

If a free text is read from an application log, an object instance of interface `IF_BALI_FREE_TEXT_GETTER` is returned. It contains the interface `IF_BALI_ITEM_GETTER`.

Public Attributes

Name	Description
CATEGORY (from IF_BALI_ITEM_GETTER)	Category of the item Contains fixed value: IF_BALI_CONSTANTS=>C_CATEGORY_FREE_TEXT
LOG_ITEM_NUMBER (from IF_BALI_ITEM_GETTER)	Serial number which is the position of the free text in the log
SEVERITY (from IF_BALI_ITEM_GETTER)	Severity of the free text ('Error', 'Warning', etc)
DETAIL_LEVEL (from IF_BALI_ITEM_GETTER)	Detail level of the free text (number between '1' and '9' or ' ')
TIMESTAMP (from IF_BALI_ITEM_GETTER)	UTC time stamp of the free text creation
CONTEXT_STRUCTURE_NAME (from IF_BALI_ITEM_GETTER)	Data dictionary name of the context structure

Public Methods

Get the content of the free text:

GET_MESSAGE_TEXT (from interface IF_BALI_ITEM_GETTER)

Name	Description
Returning parameter	
MESSAGE_TEXT	Content of the free text

Get the context data of the free text:

GET_CONTEXT (from the interface IF_BALI_ITEM_GETTER)

Name	Description
Exporting parameter	
CONTEXT	Structure that is filled with context data

4.2.15.10.2.3.12 IF_BALI_ITEM_GETTER

If an item like a message or exception is read from an application log, its object instance contains the interface `IF_BALI_ITEM_GETTER`. So, interface `IF_BALI_ITEM_GETTER` contains all attributes and methods which are available for all items which are read from the log.

Public Attributes

Name	Description
CATEGORY	Category of the item Possible values: • IF_BALI_CONSTANTS=>C_CATEGORY_MESSAGE: Item is a message • IF_BALI_CONSTANTS=>C_CATEGORY_FREE_TEXT: Item is a free text • IF_BALI_CONSTANTS=>C_CATEGORY_EXCEPTION: Item is an exception
LOG_ITEM_NUMBER	Serial number which is the position of the item in the log
SEVERITY	Severity of the item ('Error', 'Warning', etc)
DETAIL_LEVEL	Detail level of the item (number between '1' and '9' or ' ')
TIMESTAMP	UTC time stamp of the item creation
CONTEXT_STRUCTURE_NAME	Data dictionary name of the context structure

Public Methods

Get the message text of the item. It is:

- The output of ABAP command **MESSAGE** for a message item
- The text of a free text item
- The output of method GET_TEXT of the ABAP exception object for an exception item

GET_MESSAGE_TEXT

Name	Description
Returning parameter	
MESSAGE_TEXT	ext of the item message in the logon language

Get the context data of the item:

GET_CONTEXT

Name	Description
Exporting parameter	
CONTEXT	Structure that is filled with context data

4.2.15.10.2.3.13 IF_BALI_ITEM_SETTER

Context

If a new item, such as a message or an exception, is added to an application log, its object instance contains the interface IF_BALI_ITEM_SETTER.

Public Attributes

Name	Description
CATEGORY	Category of the item Possible values: • IF_BALI_CONSTANTS=>C_CATEGORY_MESSAGE: Item is a message • IF_BALI_CONSTANTS=>C_CATEGORY_FREE_TEXT: Item is a free text • IF_BALI_CONSTANTS=>C_CATEGORY_EXCEPTION: Item is an exception

Public Methods

Check whether the item can pass an item filter:

CHECK_PASSING_ITEM_FILTER

Name	Description
Importing parameter	
ITEM_FILTER	Reference to the item filter that is being checked
Returning parameter	
FILTER_PASSED	If set, the item can pass the item filter

4.2.15.10.2.3.14 IF_BALI_MESSAGE_GETTER

If a message is read from an application log, an object instance of interface `IF_BALI_MESSAGE_GETTER` is returned. It contains the interface `IF_BALI_ITEM_GETTER`.

Public Attributes

Name	Description
CATEGORY (from IF_BALI_ITEM_GETTER)	Category of the item Contains fixed value: IF_BALI_CONSTANTS=>C_CATEGORY_MESSAGE
LOG_ITEM_NUMBER (from IF_BALI_ITEM_GETTER)	Serial number which is the position of the message in the log
SEVERITY (from IF_BALI_ITEM_GETTER)	Severity of the message ('Error', 'Warning', etc)
DETAIL_LEVEL (from IF_BALI_ITEM_GETTER)	Detail level of the message (number between '1' and '9' or ' ')
TIMESTAMP (from IF_BALI_ITEM_GETTER)	UTC time stamp of the message creation
CONTEXT_STRUCTURE_NAME (from IF_BALI_ITEM_GETTER)	Data dictionary name of the context structure
ID	Message ID
NUMBER	Message number

Name	Description
VARIABLE_1	Message variable 1
VARIABLE_2	Message variable 2
VARIABLE_3	Message variable 3
VARIABLE_4	Message variable 4
COUNT	Count of cumulated messages

Public Methods

Get the message text of the message (the output of ABAP command **MESSAGE**):

GET_MESSAGE_TEXT (from interface IF_BALI_ITEM_GETTER):

Name	Description
Returning parameter	
MESSAGE_TEXT	Text of the message in the logon language

Get the context data of the message:

GET_CONTEXT (from the interface IF_BALI_ITEM_GETTER)

Name	Description
Exporting parameter	
CONTEXT	Structure that is filled with context data

4.2.15.10.3 How to Use the Fiori Reuse Library

You can use the *Reuse Library* to implement a reusable component that can display the messages of a single application log.

For more information on how to use the Fiori reuse library in a freestyle app, see Freestyle App Integration [page 1565].

For more information on how to use the Fiori reuse library in a Fiori elements app, see Fiori Elements Integration OData V2 [page 1567] (for OData V2 apps) and Fiori Elements Integration OData V4 [page 1568] (for OData V4 apps).

4.2.15.10.3.1 Freestyle App Integration

You can implement the reusable component from the *Reuse Library* in a freestyle app in order to display the messages of a single application log.

Context

To be able to use the reusable components, you need to adapt the code that can be found in your view definition file and your controller file.

Procedure

1. In the view definition file (`<MY_LOG_DISPLAY_VIEW.view.xml>`), the log display must be positioned on the page. Prepare a container control that will hold the log display component:

   Sample Code

   ```
   <!-- xmlns:core="sap.ui.core" -->
   <core:ComponentContainer id="LogMessagesControlContainer" />
   ```

2. In the controller file (`<MY_LOG_DISPLAY_VIEW.controller.js>`), the relevant coding goes into the handler for the *onInit* event. Create the component and place it into the container control:

   Note

   Make sure to replace the string `<MY_LOG_DISPLAY_VIEW>` with your own view's name!

   Sample Code

   ```
   var that = this;

   this.getOwnerComponent().createComponent({
       usage: "ApplicationLogs",
       id: "LogMessagesControlComponent",
       settings: {
           "persistencyKey": "MY_LOG_DISPLAY_VIEW",
           "showHeader": false,
           "showFilterBar": false,
           "showAsTree": false
       }
   }).then(function (oComp) {
       that.byId("LogMessagesControlContainer").setComponent(oComp);
       oComp.setLogHandle("<LogHandle>");
       oComp.refresh();
   });
   ```

   Note

   The component you embed makes use of the *SmartFilter* and the *SmartTable* control. Both controls enable a user to configure the components interactively, and also to store a current set of configuration

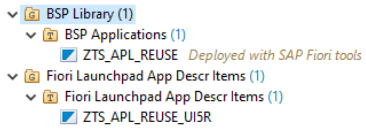
settings as a named variant. By providing a value for the `persistencyKey` parameter, you make sure that the variants that get created in your application become visible only to the users of your application, and not to all users of the component in all applications.

3. Adapt your `<manifest.json>` file and add the `sap.nw.core.applogs.lib.reuse.applogs` component:

Sample Code

```
"sap.ui5": {
    "dependencies": {
      "libs": {
        "sap.nw.core.applogs.lib.reuse": {
        "lazy": true
        }
    },
      "components": {}
    },
    "componentUsages": {
        "ApplicationLogs": {
            "name": "sap.nw.core.applogs.lib.reuse.applogs",
            "lazy": true
            }
        }
```

After your app was deployed successfully to an SAP system, the *BSP application* and the *SAP Fiori launchpad* app descriptor item will appear under your created package in Eclipse.



4. Now, you need to create a new *IAM App*. Follow the instructions described here: Defining an IAM App for the Business Service [page 1008].
5. Once created, you need to maintain the *Application Log* OData Service to call your application log data. To do this, go to the *Service* tab and add the *Application Log OData Service* by naming the service type `OData V2` and add the following service name: `APL_LOG_MANAGEMENT_SRV 0001`. Mind that all 13 spaces between `SRV` and `0001` are required. Now, go to the *Authorization* tab and maintain your *log objects / sub objects* for the authorization object `S_APPL_LOG`.
6. Finally, you need to create a new *Business Catalog*. Please follow the procedure described here: Creating a Business Catalog [page 1009]

   Having created the business catalog, you have successfully implemented a reusable component to display your application logs.

4.2.15.10.3.2 Fiori Elements Integration OData V2

You can implement the reusable component from the *Reuse Library* in a `Fiori Elements` app in order to display the messages of a single application log.

Context

In order to implement the reusable component in a Fiori Elements app, you first have to add the application log reuse component to your manifest.

Procedure

1. Open the `manifest.json` file of the application and locate the `sap.ui5` section.
2. Add the `sap.nw.core.applogs.lib.reuse.smarttemplate` component:

   Sample Code

   ```
   "sap.ui5": {
       "dependencies": {
         "libs": {
           "sap.nw.core.applogs.lib.reuse": {
           "lazy": true
           }
       },
         "components": {}
       },
       "componentUsages": {
           "ApplicationLogs": {
               "name": "sap.nw.core.applogs.lib.reuse.smarttemplate",
               "lazy": true
               }
           }
   ```

3. Next, you need to add the smarttemplate component of the application log reuse library to an `ObjectPage`. In the `manifest.json` file, you have to add the application log reuse component as an embedded component to your `ObjectPage`:

   Sample Code

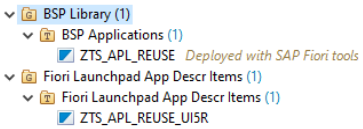
   Snippet (embeddedComponents)

   ```
   "embeddedComponents": {
       "appLog": {
           "id": "<enter an ID>",
           "componentUsage": "ApplicationLogs",
           "title": "<the name of the tab>",
           "settings": {
               "logHandle": "{LogHandle}",
               "persistencyKey": "<your content>"
           }
       }
   ```

```
}
```

4. You can set the following parameters:
   - **log handle**: The application log handle
   - **Optional persistencyKey**: Set a personalization key which is then used to set app-specific variants for `SmartTable`. This parameter is optional. The default value is defined in the *Application Log* app. Make sure not to change the key value again, since users might lose their variants.
5. Here are some examples for binding:
   - `logHandle`: `{myOdataServiceLogHandle}`
   - `logHandle`: `{path: 'myOdataServiceLogHandle'}`

   After your app was deployed successfully to an SAP system, the *BSP application* and the *SAP Fiori Launchpad* app descriptor item will appear under your created package in Eclipse.



6. Now, you need to create a new *IAM App*. Follow the instructions described here: Defining an IAM App for the Business Service [page 1008].
7. Once created, you need to maintain the *Application Log* OData Service to call your application log data. To do this, go to the *Service* tab and add the *Application Log OData Service* by naming the service type `OData V2` and add the following service name: `APL_LOG_MANAGEMENT_SRV 0001`. Please make sure to add all 13 spaces in between `SRV` and `0001`. Now, go to the *Authorization* tab and maintain your *log objects / sub objects* for the authorization object `S_APPL_LOG`.
8. Finally, you need to create a new *Business Catalog*. Please follow the procedure described here: Creating a Business Catalog [page 1009]

   Having created the business catalog, you have successfully implemented a reusable component to display your application logs.

4.2.15.10.3.3 Fiori Elements Integration OData V4

You can implement the reusable component from the *Reuse Library* in a `Fiori Elements` app in order to display the messages of a single application log.

Context

To implement the reusable component in a Fiori Elements app, you first have to add the application log reuse component to your manifest.

Procedure

1. Open the `manifest.json` file of the application and locate the `sap.ui5` section.
2. Add the `sap.nw.core.applogs.lib.reuse` library under dependencies:

   Sample Code

   ```
   "sap.ui5": {
       "dependencies": {
         "libs": {
           "sap.nw.core.applogs.lib.reuse": {
           "lazy": true
           }
       }
     }
   }
   ```

3. Next, you need to add a reuse component as embedded component to your manifest. Each reuse component instance is defined for an object page belonging to an app based on SAP Fiori elements. The reuse component is realized by a section that contains a custom subsection, which in turn contains the content of the reuse component. These sections, like any other custom sections, can be placed as described here: Extension Points for Sections on the Object Page.
4. Open the `manifest.json` file of the application and locate the object page within your `targets` section.

   Sample Code

   ```
   "ZTS_APL_C_APPL_LOG_OVERVIEWObjectPage": {
     "type": "Component",
     "id": "ZTS_APL_C_APPL_LOG_OVERVIEWObjectPage",
     "name": "sap.fe.templates.ObjectPage",
     "options": {
       "settings": {
         "editableHeaderContent": false,
         "contextPath": "/ZTS_APL_C_APPL_LOG_OVERVIEW",
         "content": {
           "body": {
               "sections": {
                   "customSectionReuse": {
                       "title": "Log Reuse Component",
                       "embeddedComponent": {
                           "name": "sap.nw.core.applogs.lib.reuse.applogs",
                           "settings": {
                               "logHandle": "{log_handle}",
                               "tableType": "ResponsiveTable"
                             }
                       },
                       "position": {
                           "placement": "After",
                           "anchor":
   "someOtherSection"
                         }
                   }
               }
           }
       }
       }
     }
   }
   ```

5. **Settings**:
    - **persistencyKey**: The component you embed uses the *SmartFilter* and *SmartTable* control. Both controls allow you to configure them interactively. You can also store a current set of configuration settings as a named variant. By providing a value for the `persistencyKey` parameter, you make sure that the variants that get created in your application become visible only to the users of your application, and not to all users of the component in all applications.
    - **showHeader**: Display or hide the header with information about the log object, the log subobject, the creator, and the creation date.
    - **showFilterBar**: Display or hide the *SmartFilter* bar.
    - **tableType**: Specifies the type of table that you can create in the *SmartTable* control. Possible values are: `Table` (default), which shows a grid table, and `ResponsiveTable`, which shows a responsive table. If you use the `tableType` setting within an object page, it's recommended to use the `ResponsiveTable` value. This value doesn't work in combination with the tree view.
    - **showAsTree**: Display log messages as a tree. Make sure the *Level of Detail* is maintained for all log messages. The table supports up to 9 expandable levels.
    - **detailLevel**: Filter out less relevant log messages. If set, only log messages of the log levels you've specified are displayed. An example: if you've specified the `detailLevel: "1,2,3"`, then only log messages with detail level 1, 2, or 3 are displayed.
    - **showContextFields**: By default, the dynamic context fields of a log are not visible in the reuse component. If this parameter is set to `true`, the context fields will also be displayed. Be aware, however, that an additional second round trip is needed to fetch the metadata of the context fields.
    - **sortOrder**: Possible values of the sort order are: *ASC* (Ascending. The oldest log is shown first. This is the default) and *DESC* (Descending. The newest log is shown first)
    - **tableTitle**: Set the table title. The default title is *Log Details*.
6. Now, you need to create a new *IAM App*. Follow the instructions described here: Defining an IAM App for the Business Service [page 1008].
7. Once created, you need to maintain the *Application Log* OData Service to call your application log data. To do this, go to the *Service* tab and add the *Application Log OData Service* by naming the service type `OData V2` and add the following service name: `APL_LOG_MANAGEMENT_SRV 0001`. Please make sure to add all 13 spaces in between `SRV` and `0001`. Now, go to the *Authorization* tab and maintain your *log objects / sub objects* for the authorization object `S_APPL_LOG`.
8. Finally, you need to create a new *Business Catalog*. Please follow the procedure described here: Creating a Business Catalog [page 1009]

    Having created the business catalog, you have successfully implemented a reusable component to display your application logs.

4.2.15.10.4 How to Create a Standalone Application Logs Tile

Learn how to provide a standalone Fiori application tile to business users to display the application logs of your application-specific business application log.

Context

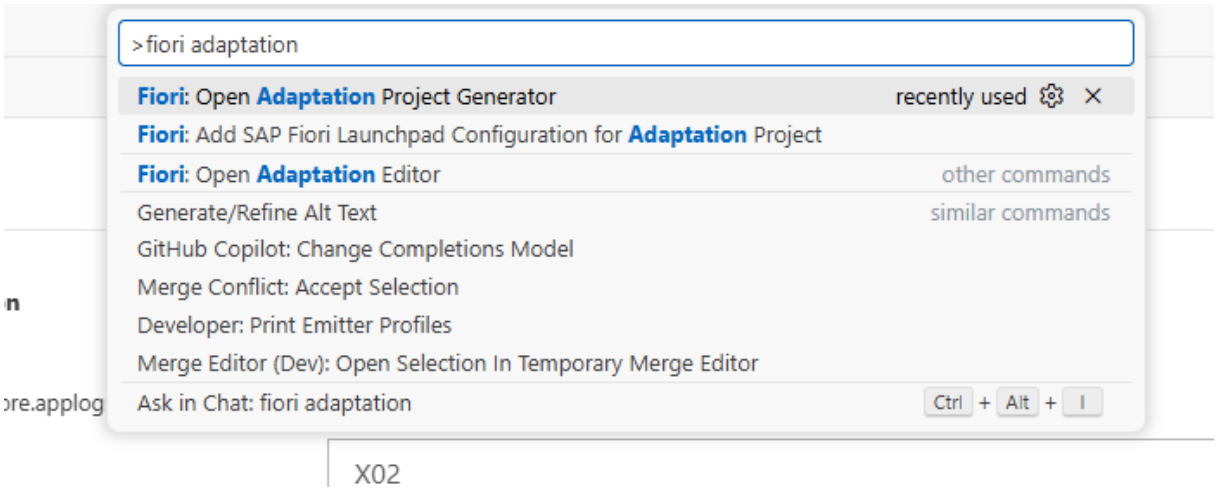
As an application developer, you want to provide a standalone Fiori application tile to business users so that they can display the application logs of your application-specific business application log. In addition, you may want to offer cross-app navigation from arbitrary business applications to this particular standalone application log tile.

Procedure

To create a standalone application tile, you can use the *Fiori Adaptation Project* mechanism that has been enabled for the *Application Logs* app (`nw.core.applogs`). You can find more information on how to work with adaptation projects here: Working with an Adaptation Project.

Create a Fiori Adaptation Project

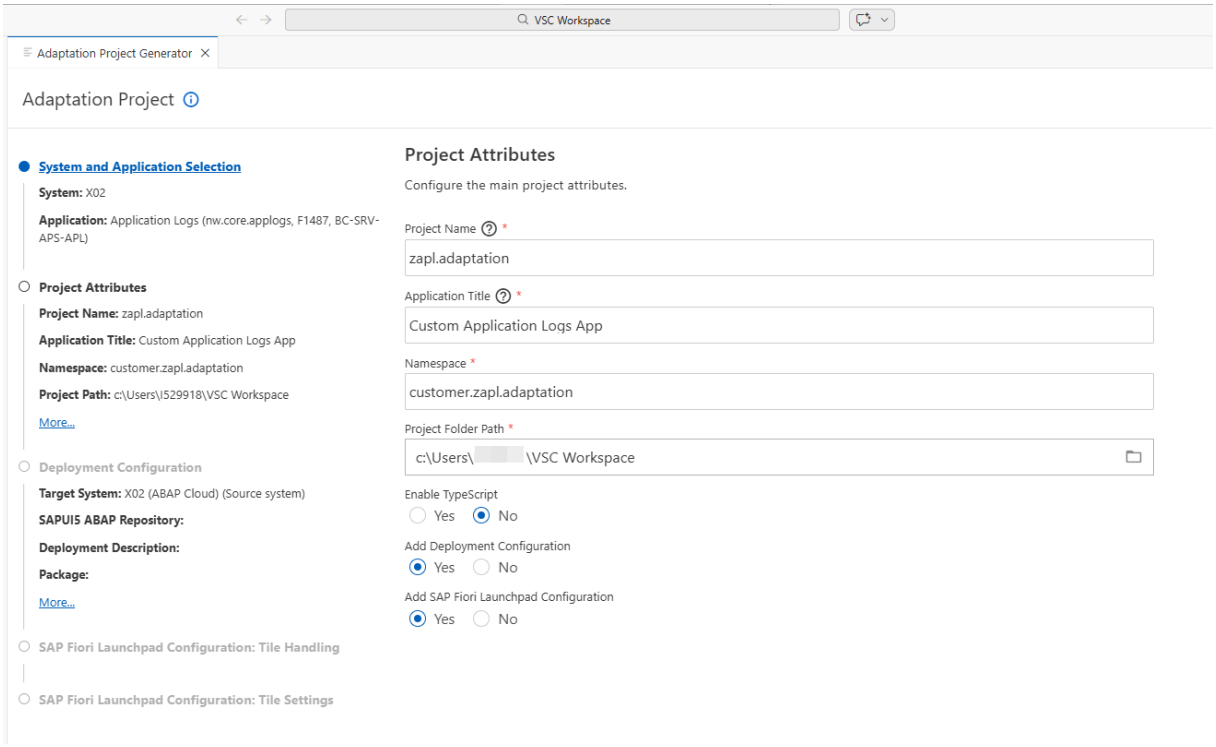
Create a Fiori adaptation project in SAP Business Application Studio or Visual Studio Code by using the *Adaptation Project Generator*.



Select the target system and the app *Application Logs* (`nw.core.applogs`, `F1487`, `BC-SRV-APS-APL`).

Project Attributes

Fill out the project attributes. At the options for adding deployment information and a SAP Fiori launchpad configuration, select *Yes*.



Deployment Information

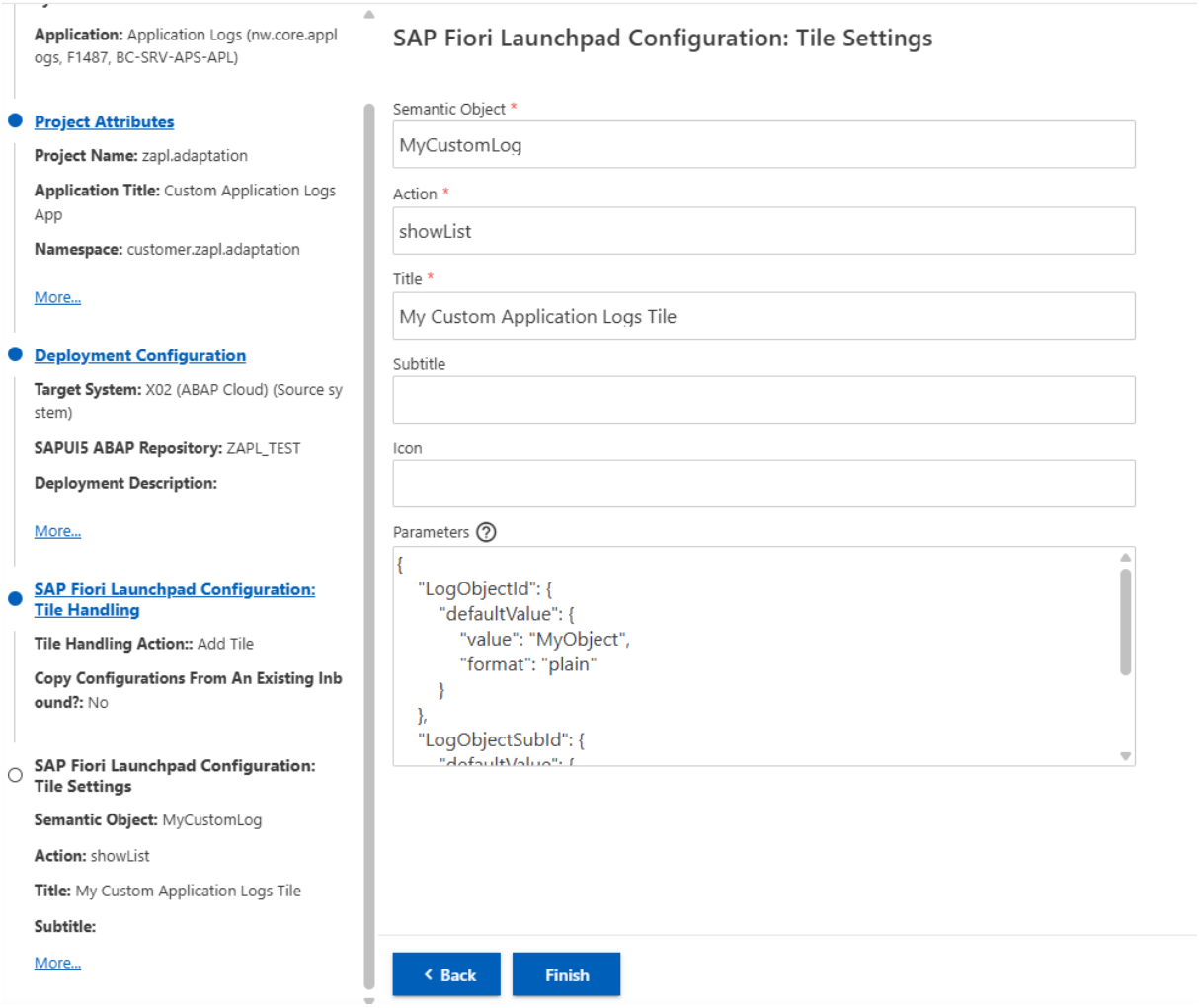
Give a repository name, a package, and a transport request.

SAP Fiori Launchpad Configuration

Select *Add Tile* and *NO* for *Copy configuration from an existing tile*.

> ⚠ Caution
>
> Do **not** use *Replace Tile*, as it would overwrite all specialized *Application Logs* tiles in the system.

In the next dialog, fill out the tile settings as needed. Since you're creating a specialized *Application Logs* tile, you must restrict the displayed logs. Therefore, it's necessary to maintain the *Log Object* and *Log Subobject* accordingly in the *Parameters* section.



The following parameters could be used:

Sample Code

```
{
    "LogObjectId": {
        "defaultValue": {
            "value": "MyObject",
            "format": "plain"
        }
    },
    "LogObjectSubId": {
        "defaultValue": {
            "value": "MySubObject",
            "format": "plain"
        }
    }
}
```

- **LogObjectId**: filters the list of logs by a log object ID (**mandatory**)
  - The parameter can contain one or more log object IDs separated by comma (without a space)
  - You can use an asterisk (like ABAP*)

- **LogObjectSubId**: filters the list of logs by a suboject ID
  - The parameter can contain one or more subobject IDs separated by comma (without a space)
  - You can use an asterisk (like ABAP*)

> ### Note
>
> Maintaining the *LogObjectId* and *LogObjectSubId* parameters in the tile settings does **not** automatically grant authorization to access those logs. You can maintain your authorizations via an IAM app (described in the next section).

After the adaptation project is generated, verify that the parameters were actually applied. In some cases, the generator doesn't write them to the manifest file. To verify,

- Open the `manifest.appdescr_variant` that is located within the webapp folder of your project.
- Search for *content* > *inbound* > *signatures* > *parameters* and verify that the entered parameters appear there. If they're missing, add them manually using the `JSON` structure above.
- Example:

> ### Sample Code
>
> **`zapl.adaptation\webapp\manifest.appdescr_variant`**

```
{
  "fileName": "manifest",
  "layer": "CUSTOMER_BASE",
  "fileType": "appdescr_variant",
  "reference": "nw.core.applogs",
  "id": "customer.zapl.adaptation",
  "namespace": "apps/nw.core.applogs/appVariants/
customer.zapl.adaptation/",
  "version": "0.1.0",
  "content": [
    {
      "changeType": "appdescr_ui5_addNewModelEnhanceWith",
      "content": {
        "modelId": "i18n",
        "bundleUrl": "i18n/i18n.properties",
        "supportedLocales": [
          ""
        ],
        "fallbackLocale": ""
      }
    },
    {
      "changeType": "appdescr_ui5_setMinUI5Version",
      "content": {
        "minUI5Version": "1.142.7"
      }
    },
    {
      "changeType": "appdescr_app_setTitle",
      "content": {},
      "texts": {
        "i18n": "i18n/i18n.properties"
      }
    },
    {
      "changeType": "appdescr_app_addNewInbound",
      "content": {
        "inbound": {
          "customer.MyCustomLog-showList": {
            "action": "showList",
```

```
            "semanticObject": "MyCustomLog",
            "icon": "",
            "title":
"{{customer.zapl.adaptation_sap.app.crossNavigation.inbounds.customer.MyCus
tomLog-showList.title}}",
            "signature": {
              "additionalParameters": "allowed",
              "parameters": {
                "LogObjectId": {
                    "defaultValue": {
                        "value": "MyObject",
                        "format": "plain"
                    }
                },
                "LogObjectSubId": {
                    "defaultValue": {
                        "value": "MySubObject",
                        "format": "plain"
                    }
                  }
              }
            }
          }
        }
      },
      "texts": {
        "i18n": "i18n/i18n.properties"
      }
    },
    {
      "changeType": "appdescr_app_removeAllInboundsExceptOne",
      "content": {
        "inboundId": "customer.MyCustomLog-showList"
      },
      "texts": {}
    }
  ]
}
```

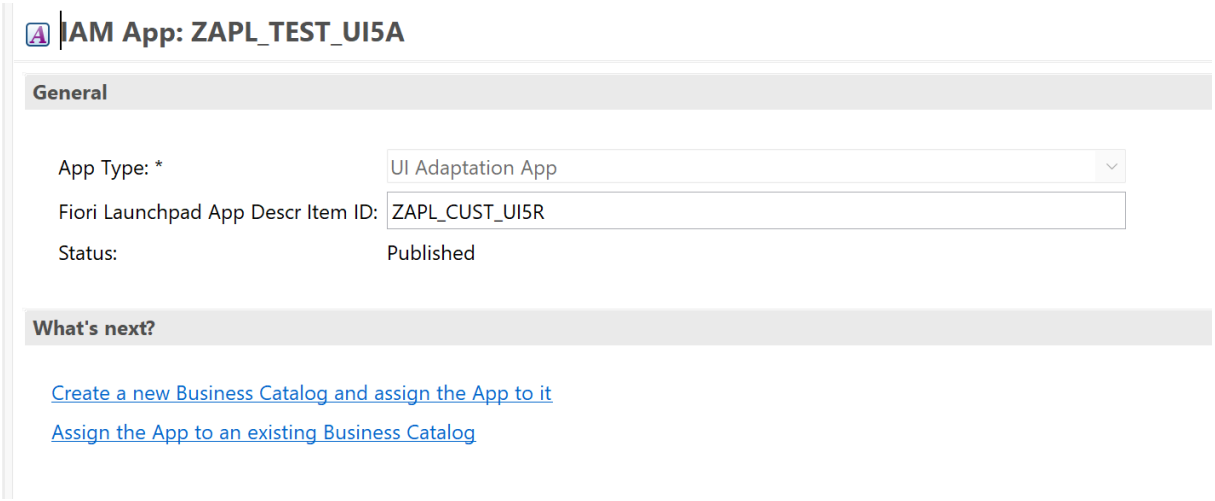
Deploy the App

Create an IAM App

After your app was deployed successfully to an SAP system, you should see the BSP application and the SAP Fiori launchpad app descriptor item under your package.

Now, you need to create a new IAM app by selecting *New* > *Other Repository Object* > *IAM App*.

Select *UI Adaptation App* as *App Type* and insert the Fiori launchpad app descriptor item of your app under *UI5 Application ID*.



Go to the *Authorization* tab and maintain your log objects or subobjects for the authorization object S_APPL_LOG.

Create a Business Catalog

Create a new business catalog by selecting *New* > *Other Repository Object* > *Business Catalog*.

Go to the *Apps* tab and add your *IAM App*.

Click *Publish Locally* to publish your business catalog.

You should now see your business catalog in the *Business Catalogs* Fiori app and you can assign it to a role.

4.2.15.11 Data Archiving

Data archiving is used to extract data from the database, write it to archive files and delete the data from the database tables.

Data archiving is needed for data volume management. Usually, you want to remove data from the database which is not needed for your daily business anymore, but needs to be retained.

Steps to implement a data archiving solution

1. Create a storage manager implementation to store the archive data to an external storage system [page 1577]
2. Select data and write it to archive files [page 1578]
3. Delete data from the database [page 1580]
4. Read archived data [page 1583]
5. Reload archived data into the database [page 1586]
6. Create a definition for archiving an object in ADT
7. Perform data archiving [page 1588]

For data archiving in ABAP Cloud the *Archive Development Kit* (ADK) provides released APIs or interfaces for all of these steps, which can be used in your implementation to archive content of your own tables.

4.2.15.11.1 Storage Manager Implementation

The storage manager integrates the data archiving runtime with the actual storage of the archiving files.

The storage manager will be processed in the different steps of the data archiving process. Within the storage manager implementation, you have the full flexibility to choose which storage will be integrated. Possible options to integrate external storages already exist in ABAP Cloud:

- Via Cloud Connector for On-Premise integration
- Outbound Communication

As a precondition for data archiving, you need to create a class which implements the interface `IF_ARCH_STORAGE_MANAGER`. This interface provides the following methods:

- The method *IF_ARCH_STORAGE_MANAGER~STORE_FILE* stores the archived data to a physical storage location outside of the SAP system. It will be called by the ADK write API and has the following parameters:

Parameter	Description
Importing: *IV_ARCHIVE_KEY*	Key of the archive file to be stored
Importing: *IV_DATA*	Content of the archive file (binary data)
Importing: *IV_LENGTH*	Length of the binary content
Returning: *RV_DOCUMENT_ID*	Document identifier which can be used to access the archived data. The document ID needs to be returned by the storage manager and will be stored in a link table in the SAP system.

- The method *IF_ARCH_STORAGE_MANAGER~GET_BYTESTREAM* retrieves parts of the binary content of an archive file via range request. It will be called by an ADK API to read the archived data and has the following parameters:

Parameter	Description
Importing: *IV_DOCUMENT_ID*	Document identifier
Importing: *IV_FROM_OFFSET*	Data object address
Importing: *IV_LENGTH*	Length of the binary content
Exporting: *EV_DATA*	Content of the archive file (binary data)
Exporting:*EV_ACT_LENGTH*	Length of the byte stream which is returned

- The method *IF_ARCH_STORAGE~RETRIEVE_FILE* retrieves the entire binary content of an archive file.

Parameter	Description
Importing: *IV_DOCUMENT_ID*	Document identifier
Returning:*RV_DATA*	Content of the archive file (binary data) which will be returned by the storage manager

- The method *IF_ARCH_STORAGE_MANAGER~DELETE FILE* deletes the archive file from the external storage system.

Parameter	Description
Importing: *IV_DOCUMENT_ID*	Document identifier

- After the method *IF_ARCH_STORAGE_MANAGER~STORE_FILE* has been called by the ADK write API, the stored file is verified by the SAP system. If the verification fails, the method *IF_ARCH_STORAGE_MANAGER~FILE_VERIFICATION_FAIL* is called by the ADK write API to inform the storage manager that the archive file is considered invalid and will be deleted from the SAP system.

Parameter	Description
Importing: *IV_ARCHIVE_KEY*	Key of the invalid archive file
Importing: *IV_DOCUMENT_ID*	Document identifier

> **Note**
>
> If any errors appear during any of the operations of the storage manager, please use the exception class `CX_ARCH_STORAGE_MANAGER`.

4.2.15.11.2 Select Data and Write It to Archive Files

The runtime to write archived data to an external storage should be implemented in a central ABAP class. This ABAP class must register the interface `IF_ARCH_WRITE` to be able to be selected in an archiving object definition in ADT. This ABAP class can be performed within an application job [page 1588].

The purpose of the ABAP class is the selection of application data which will be archived from the database. By using the ADK write API, this application data can be handed over to create a data object. A data object can contain entries of different tables that have been listed in the definition of the archiving object in ADT. Typically, these are all tables belonging to one business object instance, but there is no technical restriction. The application is free to group the tables according to their needs.

As soon as the data object is completed, the object will be transferred as an archiving file to the related storage manager implementation to store data in the related external storage.

The following table shows the write API to collect the data that will be archived:

`CL_ARCH_WRITE_API`

Method	*Parameter*	*Description*
GET_INSTANCE Static method which returns an instance of the write API via parameter `RO_WRITE_INSTANCE`	Importing: *IV_ARCHIVING_OBJECT*	Name of the archiving object which will be used
	Importing:*IV_TESTMODE*	*True*: no storage takes place *False*: Archive content is written, and storage is requested

CL_ARCH_WRITE_API

	Returning:*RO_WRITE_INSTANCE*	Instance of the write API for further processing

The following table shows the interface for writing archived data:

IF_ARCH_WRITE_API

Method	*Parameter*	*Description*
OPEN_DATA_OBJECT Opens a new data object for the data of the business object instance		
PUT_DATA_RECORDS Writes the data for a specific table to the open data object	Importing:*IV_TABLE_NAME*	Name of the table which will be archived
	Importing:*IT_RECORDS*	Records of the table
CLOSE_DATA_OBJECT Closes the data object and writes it to the archive file	Exporting:*EV_OBJECT_OFFSET*	Offset at which the data of the data object starts (can be used to build application owned indices for the archived data)
	Exporting:*EV_ARCHIVE_KEY*	Archive key of the archive file to which the data was written (can be used to build application owned indices for the archived data)
	Exporting:*EV_ARCHIVING_SESSION*	Sessions of the archiving object to be used
FINALIZE Finalizes processing and closes the archiving session		

Typical flow of the archiving write process:

1. Create an instance of the write API and open the archiving write session by using method `CL_ARCH_WRITE_API=>GET_INSTANCE`.
2. Select the data from your application DB tables.
3. For each data object you want to archive, call method `OPEN_DATA_OBJECT`.
4. For each table you want to archive entries from, call method `PUT_DATA_RECORDS`.
5. As soon as all tables of your data object are passed to the API, call method `CLOSE_DATA_OBJECT`.
6. Either start with the next data object (step 3) or finish the archiving write session by calling method `FINALIZE`.

Sample Code

```
CONSTANTS: lc_object TYPE sarch_object VALUE ‘MY_OBJECT’.
" Replace with the name of your ADT archiving object
" select flight data (leading table) from the database (replace table names
with your names of your own tables)
        SELECT * FROM my_header_table
                 WHERE carrid     IN @lt_range_carrid
                   AND connid     IN @lt_range_connid
```

```
                  AND fldate     IN @lt_range_fldate
          INTO TABLE @DATA(lt_flights).
        " open a new archiving session to archive data
        TRY.
            DATA(lo_write) =
cl_arch_write_api=>get_instance( iv_archiving_object =
lc_object                                   iv_testmode = lv_test ).
            LOOP AT lt_flights ASSIGNING FIELD-SYMBOL(<ls_flight>).
 " select data of dependent tables (replace table names with your names of
your own tables)
              INSERT <ls_flight> INTO lt_flights.
              SELECT * FROM my_item_table1
                 WHERE carrid     = @<ls_flight>-carrid
                   AND connid     = @<ls_flight>-connid
                   AND fldate     = @<ls_flight>-fldate
               INTO TABLE @DATA(lt_bookings).
              SELECT * FROM my_item_table2
                 WHERE carrid     = @<ls_flight>-carrid
                   AND connid     = @<ls_flight>-connid
                   AND fldate     = @<ls_flight>-fldate
               INTO TABLE @DATA(lt_tickets).
              SELECT * FROM my_item_table3
                 WHERE carrid     = @<ls_flight>-carrid
                   AND connid     = @<ls_flight>-connid
                   AND fldate     = @<ls_flight>-fldate
               INTO TABLE @DATA(lt_invoices).
              " open new ADK data object
              lo_write->open_data_object( ).
              " write data of header table
              lo_write->put_data_records( iv_table_name = ‘<MY_HEADER_TALBE>’
it_records = lt_sflight_arch ).
              CLEAR lt_sflight_arch.
              " write data of depending tables
              lo_write->put_data_records( iv_table_name = ‘<MY_ITEM_TABLE1>’
it_records = lt_bookings ). " bookings
              CLEAR lt_sflight_arch.
              Lo_write->put_data_records( iv_table_name = ‘<MY_ITEM_TABLE2>’
it_records = lt_tickets ). " tickets
              CLEAR lt_sflight_arch.
              Lo_write->put_data_records( iv_table_name = ‘<MY_ITEM_TABLE3>’
it_records = lt_invoices ). " invoices
              CLEAR lt_sflight_arch.
              " write data object into the archive file
              lo_write->close_data_object( ).
            ENDLOOP.
            " finalize and end writing
            lo_write->finalize( ).
```

4.2.15.11.3 Delete Data from the Database

The runtime to delete archived data from database should be implemented in a central ABAP class. This ABAP class must register interface `IF_ARCH_DELETE` to be able to be selected in an archiving object definition in ADT. This ABAP class can be performed within an application job [page 1588].

There are different ways to delete archived data from the database:

- Using a central delete API to delete all table entries of a data object
- Using a central delete API to delete specified tables of a data object
- Using a central delete API to get all table entries of a data object to perform an own deletion from the database

`CL_ARCH_DELETE_API` **(API for deleting archived data from the database)**

Method	*Parameter*	*Description*
GET_INSTANCE Returns an instance of the delete API. Reference to interface `IF_ARCH_DELETE_API`	Importing: *IV_ARCHIVING_OBJECT*	Name of the archiving object which will be used
	Importing:*IV_ARCHIVE_KEY*	Archive key for which data should be deleted
	Importing:*IV_TESTMODE*	*True*: no storage takes place *False*: Archive content is written, and storage is requested
	Returning:*RO_DELETE_INSTANCE*	Instance of the delete API
GET_FILES_TO_DELETE Returns a table with archive keys which have not yet been processed by deletion	Importing: *IV_ARCHIVING_OBJECT*	Name of the archiving object which will be used
	Importing: *IT_DATE_SELECTION*	Range of dates on which archive files have been created
	ReturnsImporting: *IT_USER_SELECTION*	Range of users which have created archive files
	Returning: *RT_ARCHIVE_FILE_ATTRIBUTES*	Table of archive keys which fulfill the selection criteria for dates and users

`IF_ARCH_DELETE_API` **(Interface for deleting archived data from the database)**

Method	*Parameter*	*Description*
GET_NEXT_OBJECT Reads the next data object for the archived data	Exporting: *EV_OBJECT_OFSET*	Offset at which the data of the data object starts. This can be used to build application-owned indices for the archived data.
	Exporting:*EV_ARCHIVE_KEY*	Archive key of the archive file to which the data was written. This can be used to build application-owned indices for the archived data.
	Exporting:*EV_ARCHIVING_SESSION*	Sessions of the archiving object which will be used
	Exporting:*EV_END_OF_FILE*	*True*: All data contained in the files is processed, and the application should finalize the deletion *False*: The application should continue processing the next data object
GET_DATA_RECORDS	Importing: *IV_RECORD_STRUCTURE*	Name of the table for which archived data should be returned
	Exporting: *ET_DATA_RECORDS*	Records of the table
DELETE_DATA_FOR_TABLE Deletes the data for a specific table from the database	Importing: *IV_TABLE_NAME*	Name of the table for which archived data should be deleted
	Importing: *IT_RECORDS*	Records of the table to be deleted

IF_ARCH_DELETE_API (Interface for deleting archived data from the database)	
DELETE_DATA_FOR_OBJECT	
Deletes all table entries contained in the data object	
FINALIZE	
Finalizes processing	

Typical flow of the archiving delete process:

1. Determine files which are still available for deletion by using the method `CL_ARCH_DELETE_API=>GET_FILES_TO_DELETE`, with filter options on creating user and creation date of the archive file.
2. Create an instance of the delete API and open the archiving delete session for one of the selected archive files by using method `CL_ARCH_DELETE_API=>GET_INSTANCE`.
3. Open the next data object with the method `GET_NEXT_DATA_OBJECT`. If the method returns `EV_END_OF_FILE = TRUE ('X')`, continue with step 6. Optional (can be used for reading archived data via index access): Update the index table with the data object archived.
4. Delete the data of the object from your database tables either by:
   - Use of the method `DELETE_DATA_FOR_OBJECT`
   - Use of the method `DELETE_DATA_FOR_TABLE`. Call this method once for each table.
   - Use of the method `GET_DATA_RECORDS`. Use the result to perform your own `DELETE` statements.
5. Continue with the next data object. See step 3.
6. Finish the archiving delete session by calling the method `FINALIZE`.

Sample Code

```
CONSTANTS: lc_object TYPE if_arch_api_types=>ty_object_name VALUE 'MY_OBJECT'.
DATA:  lv_test TYPE abap_bool VALUE abap_true,
lt_data TYPE STANDARD TABLE OF my_index_table,
lv_arkey TYPE sarch_archive_key.
DATA:   ls_index TYPE my_index_table.

 " get newest undeleted file
      DATA(lt_files) =
cl_arch_delete_api=>get_files_to_delete( iv_archiving_object =
lc_object ).
              SORT lt_files BY creation_date DESCENDING creation_time
DESCENDING.
              READ TABLE lt_files ASSIGNING FIELD-SYMBOL(<ls_file>) INDEX
1.
            DATA(lo_delete) = cl_arch_delete_api=>get_instance
( iv_archiving_object =
lc_object
            iv_archive_key = lv_arkey
                                                  iv_testmode = lv_test ).
            " read next data object from archive file into internal memory
            DO.
              lo_delete->get_next_data_object( IMPORTING ev_end_of_file =
DATA(lv_end_of_file)
                                                  ev_archive_key =
DATA(lv_archive_key)
                                                  ev_object_offset =
DATA(lv_offset) ).
              IF lv_end_of_file = abap_true.
                EXIT.
              ENDIF.
```

```
              " optional: Create customer owned index for single document
access during read
              " read archived data for table MY_HEADER_TABLE
              lo_delete->get_data_records( EXPORTING iv_record_structure =
'MY_HEADER_TABLE'
                                           IMPORTING et_data_records =
lt_data ).
              " map flight data to your index table MY_INDEX_TABLE
              IF lv_test <> abap_true.
                LOOP AT lt_data ASSIGNING FIELD-SYMBOL(<ls_data>).
                  MOVE-CORRESPONDING <ls_data> TO ls_index.
                  Ls_index-archive_key = lv_archive_key.
                  Ls_index-archive_offset = lv_offset.
                  INSERT my_index_table FROM @ls_Index.
                ENDLOOP.
              ENDIF.
              " delete the data from the data base
              lo_delete->delete_data_for_object( ).
            ENDDO.
            COMMIT WORK.
            lo_delete->finalize( ).
```

Example of a customer-owned index table for archived flights:

Sample Code

```
@EndUserText.label : 'Archiving Index: Flights'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #ALLOWED
define table my_index_table {
@EndUserText.label : 'Client'
key mandt : abap.clnt not null;
@EndUserText.label : 'Airline Code'
key carrid : abap.char(3) not null;
@EndUserText.label : 'Flight Connection Number'
key connid : abap.numc(4) not null;
@EndUserText.label : 'Flight date'
key fldate : abap.dats not null;
archive_key : sarch_archive_key;
archive_offset : sarch_offset;
```

4.2.15.11.4 Read Archived Data

Applications can use the ADK read API to read the data which is contained in an archive file stored in an external storage system.

There are two possible ways to access the data:

- Read the data for entire archive files sequentially
- Read the data by index access

To optimize the reading access on archive files, an index solution can be established in the application. For an index-based read access, the archive key and the offset of a data object are needed. ADK API provides this information within methods `IF_ARCH_DELETE_API->GET_NEXT_DATA_OBJECT` and `IF_ARCH_READ_API->GET_NEXT_DATA_OBJECT`.

A typical index-based solution needs a transparent table containing columns for application attributes, a column for the archive key and a column for the offset. The archive key is the ID of the archive file, which contains the application attribute. The offset is the offset in the archive file at which the data for the application attribute starts. Usually, the initial construction of an archive index will be established by deleting archived data from the database [page 1580]. In case the index should be constructed at a later point in time, the ADK read API can be used to determine archive key and offset (returned by method `IF_ARCH_READ_API->GET_NEXT_DATA_OBJECT`).

`CL_ARCH_READ_API` (API to read archived data)

Method	*Parameter*	*Description*
GET_FILES_TO_READ Selection of archiving sessions and files	Importing: *IT_DATE_SELECTION*	Range on dates on which archive files have been created
	Importing:*IT_USER_SELECTION*	Range on users which have created archive files
	Importing:*IT_ARCHIVING_OBJECT*	Archiving object which will be used
	Returning:*RT_ARCHIVING_FILE_ATTRIBUTES*	Archiving sessions' attributes
GET_INSTANCE Returns an instance of the delete API. Reference to interface `IF_ARCH_READ_API`	Importing: *IV_ARCHIVING_OBJECT*	Name of the archiving object which will be used
	Importing: *IV_ARCHIVING_SESSION*	Sessions of the archiving object which will be used
	Importing: *IT_ARCHIVE_KEY*	Archive key for which data should be read
	Returning: *RO_READ_INSTANCE*	Instance of the read API for further processing

`IF_ARCH_READ_API` (Interface to read archived data)

Method	*Parameter*	*Description*
GET_DATA_RECORDS Read records by structure from the current data object	Importing: *IV_RECORDS_STRUCTURE*	Name of the structure of all data records in the table
	Exporting:*EV_DATA_RECORDS*	Table containing the data records
GET_NEXT_DATA_OBJECT Read data object from the archive file	Exporting: *EV_ARCHIVE_KEY*	Archive key according to archive management
	Exporting: *EV_END_OF_FILE*	Boolean values: *TRUE* (='X') *FALSE* (= ' ')
	Exporting: *EV_ARCHIVING_SESSION*	Sessions of the archiving object which will be used
	Exporting: *EV_OBJECT_OFFSET*	Offset of data object in the archive
CLOSE Open archive files will be closed		

As mentioned before, there are two different ways to acceess the data. A typical procedure for the sequential read access would be as follows:

1. Determine files which are available for evaluation by using the method `CL_ARCH_READ_API=>GET_FILES_TO_READ`, with filter options on creating user and creation date of the archive file.
2. Create an instance of the read API and open the archiving read session for one of the selected archive files by using the method `CL_ARCH_READ_API=>GET_INSTANCE`.
3. Open the next data object with the method `GET_NEXT_DATA_OBJECT`. If the method returns `EV_END_OF_FILE = TRUE ('X')`, continue with step 6.
4. Read the data with the method `GET_DATA_RECORDS`.
5. Continue with the next data object. See step 3.
6. Finish the read operation by calling method `CLOSE`.

A typical flow of the index-based read access would be as follows:

1. Determine the archive key and the offset of the data object by selecting the relevant entry from your index table.
2. Create an instance of the read API and open the archiving read session for the data object by using the method `CL_ARCH_READ_API=>GET_INSTANCE_BY_OFFSET`.
3. Read the data by using the method `GET_DATA_RECORDS`.
4. Finish the read operation by calling method `CLOSE`.

An example code for the index-based read operation:

Sample Code

```
CONSTANTS: lc_object TYPE if_arch_api_types=>ty_object_name VALUE 'MY_OBJECT'.
DATA: lt_data TYPE STANDARD TABLE OF my_item_table.
DATA: lt_index TYPE STANDARD TABLE OF my_index_table.
    " let's assume, we want to read the archived bookings for flight AA/0040
on 20010110
    TRY.
        " get archive key and offset for the given flight from the index table
        SELECT * FROM my_index_table WHERE carrid = 'AA' AND connid = '0040'
AND fldate = '20010110'
         INTO TABLE @lt_index.
        SORT lt_index BY archive_key DESCENDING.
        READ TABLE lt_index ASSIGNING FIELD-SYMBOL(<ls_index>) INDEX 1.
        IF NOT <ls_index> IS ASSIGNED.
          RETURN.
        ENDIF.
        " open a read instance for the archive key and offset found
        DATA(lo_read) = cl_arch_read_api=>get_instance_by_offset
( iv_archiving_object = lc_object
                                                        iv_archive_key =
<ls_index>-archive_key
                                                iv_object_offset = <ls_index>-
archive_offset ).
        " get all archived booking entries for this flight
        lo_read->get_data_records( EXPORTING iv_record_structure =
'MY_ITEM_TABLE1'
                                   IMPORTING et_data_records = lt_data ).
        " close the read instance
        lo_read->close( ).
```

4.2.15.11.5 Reload Archived Data into the Database

Reloading is optional and should only be used in exceptional cases. The application must decide if a reload is necessary or not.

In the standard process, a reloading of archived data is not required, as archived data and direct access to individual data objects can be performed by using the Read API [page 1583].

There are two cases which need to be distinguished:

1. Data must be reloaded shortly after deleting data from the database within the archiving process. An example for this case would be if selection criteria were set wrongly for the archiving process.
2. Reload of archived data after a long period of time. In this case it must be ensured that references to and within the reloaded data are still available in database. Furthermore, it needs to be guaranteed that no table key conflicts exist. Since this is not a real error, it's strongly recommended not to reload the data in this case.

The runtime to reload archived data from an external storage system back into the database should be implemented in a central ABAP class. This ABAP class has to register interface `IF_ARCH_RELOAD` to be able to be selected in an archiving object definition in ADT.

`CL_ARCH_RELOAD_API` (API to reload archived data)

Method	*Parameter*	*Description*
GET_INSTANCE Returns an instance of the delete API. Reference to interface `IF_ARCH_RELOAD_API`	Importing: *IV_ARCHIVING_OBJECT*	Name of the archiving object which will be used
	Importing:*IV_ARCHIVING_SESSION*	Sessions of the archiving object which will be used
	Importing:*IV_TESTMODE*	Reload program runs in test mode
	Returning:*RO_RELOAD_INSTANCE*	Instance of the reload API for further processing
GET_SESSIONS_TO_RELOAD Read a data object from an archive file	Importing: *IT_DATA_SELECTION*	Range on dates on which archive files have been created
	Importing: *IT_USER_SELECTION*	Range on users which have created archive files
	Importing: *IV_ARCHIVING_OBJECT*	Archiving object which will be used
	Returning: *RT_SESSION_ATTRIBUTES*	Archiving sessions' attributes

`IF_ARCH_RELOAD_API` (Interface to reload archived data)

Method	*Parameter*	*Description*
GET_DATA_RECORDS Read records by structure from the current data object	Importing: *IV_RECORDS_STRUCTURE*	Name of a structure of all data records in the table
	Exporting:*ET_DATA_RECORDS*	Table containing the data records
GET_NEXT_DATA_OBJECT Read data object from the archive file	Exporting: *EV_ARCHIVE_KEY*	Archive key according to archive management

IF_ARCH_RELOAD_API (Interface to reload archived data)

	Exporting: *EV_END_OF_FILE*	Boolean values: *TRUE* (='X') *FALSE* (= ' ')
	Exporting: *EV_ARCHIVING_SESSION*	Sessions of the archiving object which will be used
	Exporting: *EV_OBJECT_OFFSET*	Offset of data object in the archive
RELOAD_DATA_FOR_OBJECT		
RELOAD_DATA_FOR_TABLE Reloading the data of an internal table	Importing: *IV_TABLE_NAME*	DDIC structure name of an internal table
	Tables: *IT_TABLE_NAME*	
FINALIZE Open archive files will be closed		

A typical flow of the archiving reload process would be as follows:

1. Determine archiving sessions which are still available for reload by using the method `CL_ARCH_RELOAD_API=> GET_SESSIONS_TO_RELOAD`, with filter options on creating user and creation date of the archive file.
2. Create an instance of the reload API and open the archiving reload session for one of the selected archiving sessions by using the method `CL_ARCH_RELOAD_API=>GET_INSTANCE`.
3. Open the next data object via method `GET_NEXT_DATA_OBJECT`. If the method returns `EV_END_OF_FILE = TRUE ('X')`, continue with step 6 to call the method `FINALIZE`.
4. Reload the data of the object from the external storage system back into your database tables, either by:
   - Use of the method `RELOAD_DATA_FOR_OBJECT`
   - Or use of the method `RELOAD_DATA_FOR_TABLE` . This method needs to be called once for each table.
   - Use of the method `GET_DATA_RECORDS`. With this method, the result needs to be used to perform your own `INSERT` statements.
5. Continue with the next data object. See step 3.
6. Finish the archiving reload session by calling the method `FINALIZE`.

An example coding snippet:

Sample Code

```
CONSTANTS: lc_object TYPE if_arch_api_types=>ty_object_name VALUE 'MY_OBJECT'.
DATA:     lv_test TYPE abap_bool VALUE abap_true,
lt_data TYPE STANDARD TABLE OF my_header_table,
lv_session TYPE sarch_session_number.

            " get newest complete archiving session
              DATA(lt_sessions) =
cl_arch_reload_api=>get_sessions_to_reload(           iv_archiving_object =
lc_object ).
              SORT lt_sessions BY creation_date DESCENDING creation_time
DESCENDING.
              READ TABLE lt_sessions ASSIGNING FIELD-SYMBOL(<ls_session>)
INDEX 1.
```

```
            DATA(lo_reload) = cl_arch_reload_api=>get_instance
( iv_archiving_object =   lc_object
                                                    iv_archiving_session =
lv_session
                                                    iv_testmode =
lv_test ).
            " read next data object from archive file into internal memory
            DO.
              lo_reload->get_next_data_object( IMPORTING ev_end_of_file =
DATA(lv_end_of_file)
                                                      ev_archive_key =
DATA(lv_archive_key)
                                                      ev_object_offset =
DATA(lv_offset) ).
              IF lv_end_of_file = abap_true.
                EXIT.
              ENDIF.
              " optional: remove
              " read archived data for table ZADK_SFLIGHT
              lo_reload->get_data_records( EXPORTING iv_record_structure =
'MY_HEADER_TABLE'
                                           IMPORTING et_data_records =
lt_data ).
              " reload the data into the data base
              lo_reload->reload_data_for_object( ).
              " optional: delete data from customer owned index
              DELETE FROM <my_index_table>
                WHERE archive_key = @lv_archive_key.
            ENDDO.
            COMMIT WORK.
            lo_reload->finalize( ).
```

4.2.15.11.6 Archiving Classes

Archiving classes are used to archive object instances of reuse objects together with the business object instances in your application to which the reuse object instance belongs. If your application includes such reuse objects, and if SAP offers a released archiving class for the reuse object, you can easily integrate the archiving class into your archiving object:

- Enhance the archiving object by adding the archiving class to your archiving object in the ADT editor
- Enhance the write class by registering the object instance of the reuse object via a released method belonging to the archiving class
- Use the released read functionality belonging to the archiving class to read the data of the reuse object

4.2.15.11.7 Perform Data Archiving

Productive data archiving is a mass data processing and should therefore always be performed in the background, decoupled from the transactional business process. For background processing, regular performing and monitoring of data archiving runs, the *Application Job* framework is recommended to use. For that, an application job catalog entry with at least one application job template is required for each data archiving step:

- Write data to external storage
- Delete data from custom database tables

> **Note**
>
> Both data archiving steps need to run in independent scheduled application jobs.

The application job catalog entry contains the implementation of the data archiving step: it uses the released corresponding ADK runtime API. For logging, the application log in the context of an application job can be used. Related monitoring is embedded in the application job apps.

All APIs for data archiving provide a test mode parameter. If this parameter is set, no changes are performed on the database and no data is stored via the storage manager. Applications themselves need to expose a test mode parameter to the user. They need to take care that no database changes are performed in their part of the write, delete, and reload ABAP classes in case the test mode option is chosen.

Related Information

Application Jobs [page 1461]
Application Logs [page 1498]

4.2.15.12 Business Configuration Sets

A Business Configuration Set is a container to group customizing data logically according to business needs. The business configuration set can be activated in a system to deploy its contents to the relevant customizing tables

You can create a business configuration set using either of the following ways:

1. Using the business configuration set creation wizard in ADT and import content from a customizing transport to the Business Configuration Set. For more information refer Working with Business Confiuration Sets
2. Using the XCO Generation APIs to create or update a business configuration set by specifying the object template, and to delete the business configuration set. See Business Configuration Sets [page 1788]

4.2.15.12.1 Runtime API

You can activate a business configuration set using a class-based API by specifying the relevant activation options. Activation options are certain parameters that can be specified pertaining to the activation of the Business Configuration Set.

The following steps are involved in activating a business configuration set:

1. Accessing the Activation Handler [page 1590]

2. Specifying the Activation Options [page 1591]
3. Activating the Business Configuration Set [page 1592]
4. Accessing the Activation Results [page 1593]

The class `CL_SCPR_CD_BCSET_ACTIVATION` enables the activation of a Business Configuration set.

Class CL_SCPR_CD_BCSET_ACTIVATION

Method	Interface	Description
`OF`		Instantiate the activation API
`OPTIONS`		Access the activation options
`GET_ACTIVATE_HANDLER`	`IF_SCPR_CD_BCSET_ACTIVATION`	Access the activation handler

Exception Class

Class Name	Description
`CX_SCPR_OBJECT_NOT_PERMITTED`	The Activation of the specified business configuration set is not permitted. Possible reasons : • `CX_SCPR_OBJECT_NOT_PERMITTED=>BC_SET_NOT_FOUND` The specified business configuration set does not exist. • `CX_SCPR_OBJECT_NOT_PERMITTED=>BC_SET_NOT_PERMITTED` The specified business configuration set is not permitted to be activated based on the object restrictions or language version check.

4.2.15.12.1.1 Accessing the Activation Handler

You can access the activation handler by specifying the name of the business configuration set.

Instantiate the class using the method `OF` and provide the name of the *Business Configuration Set* to be activated. If the specified business configuration set does not exist or the checks based on the object restrictions and language version fail, the exception `CX_SCPR_OBJECT_NOT_PERMITTED` is raised.

Method `OF`

Importing Parameter

Parameter Name	Description
`IV_BCSET`	Name of the business configuration Sst

Returning Parameter

Parameter Name	Description
RO_RESULT	Reference to instance of IF_SCPR_CD_BCSET_AVTIVATION

Use the instance method `IF_SCPR_CD_BCSET_ACTIV_START~GET_ACTIVATE_HANDLER( )` to access the activation handler.

Method `IF_SCPR_CD_BCSET_ACTIVATION~GET_ACTIVATE_HANDLER`

Returning Parameter

Parameter Name	Description
RO_RESULT	Reference to instance of IF_SCPR_CD_BCSET_ACTIV_HANDLER

Sample Code

```
TRY.
DATA(lo_bcset_activate) =
cl_scpr_cd_bcset_activation=>of( 'BUSINESS_CONFIGURATION_SET_NAME' ).
DATA(lo_activation_handler) = lo_bcset_activate->get_activate_handler( ).
CATCH cx_scpr_object_not_permitted INTO DATA(lo_exception).
" Process the exception
DATA(lv_exception_message) = lo_exception->get_text( ).
ENDTRY.
```

4.2.15.12.1.2 Specifying the Activation Options

The activation option *Activation Languages* can be customized if needed, or the default settings can be used. This indicates the languages in which the language-dependent contents of the business configuration set should be activated.

You can choose to activate the business configuration set with:

- Default activation options: All active system languages are considered for the activation languages
- Customized activation options: You can specify the languages in which the language-dependent contents are to be activated.

To customize the activation options, they can be accessed using the method `OPTIONS`, followed by calling the method `IF_SCPR_CD_BCSET_OPTNS_HANDLER~GET_ACTIVATE_OPTIONS( )`.

Method `OPTIONS`

Returning Parameter

Parameter Name	Description
RO_RESULT	Reference to instance of IF_SCPR_CD_BCSET_OPTNS_HANDLER

Method `IF_SCPR_CD_BCSET_OPTNS_HANDLER~GET_ACTIVATE_OPTIONS`

Returning Parameter

Parameter Name	Description
RO_RESULT	Reference to instance of IF_SCPR_CD_BCSET_ACTIV_OPTIONS

The method IF_SCPR_CD_BCSET_ACTIV_OPTIONS~ADD_ACTIVATION_LANGUAGES can be used to customize the activation languages.

Method IF_SCPR_CD_BCSET_ACTIV_OPTIONS~ADD_ACTIVATION_LANGUAGES

Importing Parameter

Parameter Name	Description
IT_LANGUAGES	List of activation languages (corresponding key of the language)

The default or customized activation options can be set to the activation handler by calling the method IF_SCPR_CD_BCSET_ACTIV_HANDLER~LOCAL( ) followed by a call to IF_SCPR_CD_BCSET_ACTIV_ADD_OPT~ADD_ACTIVATION_OPTIONS( ).

The call to method IF_SCPR_CD_BCSET_ACTIV_HANDLER~LOCAL( ) indicates that the contents are deployed to the tables locally during activation and are not captured in a transport request.

Sample Code

```
" Case 1) Default activation options
DATA(lo_activate_handler) = cl_scpr_cd_bcset_activation=>of( ‘BCSET_NAME’ )-
>get_activate_handler( ).
DATA(lo_activation_handler) = lo_activate_handler->local( )-
>add_activation_options( ).
" Case 2) Customized activation options
DATA(lo_activation_options) = cl_scpr_cd_bcset_activation=>options( )-
>get_activate_options( ).
lo_activation_options->add_activation_languages( VALUE #( ( langu = 'E' )
( langu = 'F' ) ) ).
DATA(lo_activate_handler) = cl_scpr_cd_bcset_activation=>of( ‘BCSET_NAME’ )-
>get_activate_handler( ).
DATA(lo_activation_handler) = lo_activate_handler->local( )-
>add_activation_options( lo_activation_options ).
```

4.2.15.12.1.3 Activating the Business Configuration Set

Once the activation options have been set, the method IF_SCPR_CD_BCSET_ACTIV_START~START( ) can be used to trigger the activation.

Method IF_SCPR_CD_BCSET_ACTIV_START~START

Returning Parameter

Parameter Name	Description
RO_RESULT	Reference to instance of IF_SCPR_CD_BCSET_ACTIV_RESULT

Sample Code

```
" Activating a business configuration set with the default activation options
TRY.
DATA(lo_activate_handler) = cl_scpr_cd_bcset_activation=>of( 'BCSET_NAME' )-
>get_activate_handler( ).
DATA(lo_activation_result) = lo_activate_handler->local( )-
>add_activation_options( )->start( ).
CATCH cx_scpr_object_not_permitted INTO DATA(lo_exception).
cc Process the exception
DATA(lv_exception_message) = lo_exception->get_text( ).
ENDTRY.
```

4.2.15.12.1.4 Accessing the Activation Results

Upon activation of the business configuration set, the instance returned from method `START( )` can be used to check if the activation was successful, using the method `IF_SCPR_CD_BCSET_ACTIV_RESULT~IS_BCSET_ACTIVATION_SUCCESSFUL( )`.

The log messages from the activation can be accessed using the method `IF_SCPR_CD_BCSET_ACTIV_RESULT~GET_MESSAGES( )`.

Method `IF_SCPR_CD_BCSET_ACTIV_RESULT~IS_BCSET_ACTIVATION_SUCCESSFUL`

Returning Parameter

Parameter Name	Description
RV_RESULT	ABAP_TRUE if the activation was successful, else ABAP_FALSE

Method `IF_SCPR_CD_BCSET_ACTIV_RESULT~GET_MESSAGES`

Returning Parameter

Parameter Name	Description
RT_MESSAGES	Log messages from the activation – of type BAPIRETTAB

Sample Code

```
" lo_result is the instance returned from calling
IF_SCPR_CD_BCSET_ACTIV_START->START( )
" Check if activation was successful
DATA(lv_status) = lo_result->is_bcset_activation_successful( ).
" Get the activation log messages
DATA(lt_activation_messages) = lo_result->get_messages( ).
```

4.2.15.13 Output Management

Printing [page 1594]
Find out how to create a print queue item API.

Print Forms [page 1595]
ADS Rendering in ABAP environment

Emailing [page 1603]
This chapter covers the latest applications for sending, monitoring, and configuring emails.

4.2.15.13.1 Printing

Find out how to create a print queue item API.

You can use the method `CREATE_QUEUE_ITEM_BY_DATA` with the class `CL_PRINT_QUEUE_UTILS` to create a print queue item of a queue. One print queue item has one main document and can have either no or a variable amount of attachments. The method `CREATE_QUEUE_ITEM_BY_DATA` has the following importing parameters:

- `IV_QNAME`: Name of the print queue
- `IV_PRINT_DATA`: Print document contained in xstring format
- `IV_NAME_OF_MAIN_DOC`: Name of the main document
- `IV_NUMBER_OF_COPIES`: Number of copies
- `IV_PAGES`: Number of pages
- `IT_ATTACHMENT_DATA`: List of attachments within the structure. This includes the name and the attachment data in xstring

If a print queue item has been created successfully, it returns the ID of the item with the parameter `RV_ITEMID`. In case of any errors, you can find the error messages in the exporting parameter `EV_ERR_MSG`.

> Example

```
CLASS zcl_pq_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun .
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_pq_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    DATA: l_print_data TYPE xstring.
    l_print_data = |…|.
    DATA(lv_qitem_id) = cl_print_queue_utils=>create_queue_item_by_data
      EXPORTING
        iv_qname           = '…'
        iv_print_data      = l_print_data
        iv_name_of_main_doc = '…'.
    out->write( lv_qitem_id ).
  ENDMETHOD.
ENDCLASS.
```

Related Information

Maintain Print Queues [page 3128]
Integrating SAP Business AI Platform, ABAP environment and Local Printers [page 3321]

4.2.15.13.2 Print Forms

ADS Rendering in ABAP environment

The *Adobe Document Services* (ADS) can render Adobe XML Forms (XFA) to PDF documents and other print formats. The application sends the data and the form template using a secure connection and the service returns the rendered document. The rendered documents can be used by your applications for the screen output or for the output into the print queue. For more information, see Maintain Print Queues [page 3128].

For more information, see Connecting the SAP Forms Service to ABAP.

4.2.15.13.2.1 Retrieve Stored Form Templates

Using ABAP development tools for Eclipse, you can upload form templates designed with the Adobe LiveCycle Designer directly to the system. See Working with Forms.

The class `CL_FP_FORM_READER` enables the usage of uploaded form templates during runtime.

Initiate the Form Template Reader

Initiate the class by providing the name of the uploaded form template.

Sample Code

```
data(lo_reader) = cl_fp_form_reader=>create_form_reader( 'MY_FORM' ).
```

Read Stored Attributes

The reader class offers the following read-only methods to retrieve the stored attributes.

Method Name	Output Type	Description
get_data_optimized	fpoptimize	Is data optimization enabled for the connected Form Data Provider?

Method Name	Output Type	Description
get_font_embed	fpfntemb	Is font embedding automatically enabled when rendering the form?
get_fdp_name	fpdpname	The name of the connected Form Data Provider

All of these methods do not support any additional import parameters.

Read Layout Content to be Used by ADS Rendering Call

The reader class also enables you to retrieve the layout.

The `GET_LAYOUT` method has the following returning parameters:

Parameter	Definition
RV_LAYOUT	Resulting XML as XSTRING

4.2.15.13.2.2 Runtime API for ADS Rendering Calls

The `CL_FP_ADS_UTIL` class provides the ABAP Runtime API for *Adobe Document Services* (ADS) rendering calls. It contains the following *Public* methods to be used for the corresponding functions:

Public Methods

Method	Description
`RENDER_PDF`	Rendering PDF
`RENDER_4_PQ`	Rendering for Print Queue

The PDF rendering process on ADS creates the PDF file to be printed (or print output format) using the following both formats:

- XDP form template file
- Data for the form filling in XML format, for example, data from the RAP Data Services for Print Forms [page 1598].

The XDP form template file is a description of the form layout design in Adobe XFA format. The supported data binding type is an XML data schema.

The *Adobe LiveCycle Designer for SAP solutions* is the tool for visual design XDP form templates from Adobe Inc. It can be downloaded using the *Install Additional Software* app, which contains the download link to the SAP ONE Support Launchpad.

The data XML file contains the data that corresponds to the used data schema in the XDP form template.

Rendering PDF

The `RENDER_PDF` method calls the ADS to render print PDF using the XDP form file and the XML data file. It has the following importing/exporting parameters:

Importing Parameters

Parameter	Description
`IV_XML_DATA`	XML data
`IV_XDP_LAYOUT`	Adobe XDP form template
`IV_LOCALE`	Locale for rendering the language: `language_COUNTRY`, for example en_US
`IS_OPTIONS`	PDF rendering parameters (optional)

Exporting Parameters

Parameter	Description
`EV_PDF`	PDF rendering result
`EV_PAGES`	
`EV_TRACE_STRING`	Trace string

Sample Code

```
TRY.
*Render PDF
    cl_fp_ads_util=>render_pdf( EXPORTING iv_xml_data      = lv_xml_data
                                          iv_xdp_layout    = lv_xdp
                                          iv_locale        = 'de_DE'
                                          is_options       = ls_options
                                IMPORTING ev_pdf           = ev_pdf
                                          ev_pages         = ev_pages
                                          ev_trace_string  = ev_trace_string
                                          ).
  CATCH cx_fp_ads_util INTO lx_fp_ads_util.

ENDTRY.
```

Rendering for Print Queue

The `RENDER_4_PQ` method calls the ADS to render in the print output format required for the target print queue. The support of the *Print Definition Language* (PDL) print queue format will be taken from print queue properties.

Importing Parameters

Parameter	Description
`IV_XML_DATA`	XML data
`IV_XDP_LAYOUT`	Adobe XDP form template
`IV_LOCALE`	Locale for rendering the language: `language_COUNTRY`, for example en_US
`IV_PQ_NAME`	Print Queue name
`IS_OPTIONS`	PDL rendering parameters (optional)

Exporting Parameters

Parameter	Description
`EV_PDL`	PDF rendering result
`EV_PAGES`	Number of pages
`EV_TRACE_STRING`	Trace string

Sample Code

```
TRY.
*Render in Print Queue Format
    cl_fp_ads_util=>render_4_pq( EXPORTING iv_xml_data      = lv_xml_data
                                           iv_xdp_layout    = lv_xdp
                                           iv_locale        = 'en_US'
                                           iv_pq_name       = 'PQ1'
                                           is_options       = value #( )

                                 IMPORTING ev_pdl           = ev_pdl
                                           ev_pages         = ev_pages
                                           ev_trace_string = ev_trace_string
                                           ev_pdf_trace     = ev_pdf_trace
).
CATCH cx_fp_ads_util INTO lx_fp_ads_util.
ENDTRY.
```

4.2.15.13.2.3 RAP Data Services for Print Forms

The XML data for print form rendering can be obtained from the RAP Business Services. The data model can be defined just as well as the *RAP Service Definition*. For more information, see Service Definition.

The `CL_FP_FDP_SERVICES` class provides the ABAP API to receive the following:

- data in xml format for the use in print forms
- XML Schema Definition (XSD) used for the design of print forms in the *Adobe LiveCycle Designer*.

Initiate Business Data Reader

To supply the rendering operation with business data, a reuse service can be applied. Beforehand, a service definition needs to be created.

> **Note**
>
> The service definition doesn't need a binding for RAP Data Services for print forms.

1. Initiate the reuse utility by specifying the service definition: `data(lo_fdp_api) = cl_fp_fdp_services=>get_instance( `FP_FDP_AUNIT_SO` )`.
2. Specify the key parameters. This is required to read the data for a specific item. The reuse utility offers the following functionality to retrieve all key fields for the service definition: `data(lt_keys) = lo_fdp_api->get_keys( ).`
3. Once you have retrieved the array with key values, assign the keys that fit to the document you want to output: `lt_keys[ name = 'UUID' ]-value = 'FA163EE47BDD1ED9A682A2E6F1ECF696'`.

> **Sample Code**

```
DATA(lo_fdp_api) = cl_fp_fdp_services=>get_instance( `FP_FDP_SERVICE_EX1` ).
DATA(lt_keys)    = lo_fdp_api->get_keys( ).
  lt_keys[ name = 'UUID' ]-value = 'FA163EE47BDD1ED9A682A2E6F1ECF696'.
```

Retrieve XML Data from Service Definition

The `READ_TO_XML_V2` method retrieves the data structure of the service definition for a specific item. It has the following Importing parameters:

Importing Parameters

Parameter	Definition
`IT_SELECT`	Key parameters
`IV_LANGUAGE` (Optional)	Overwrites the locale language (useful if you want to control how dynamic values (such as unites) are resolved

The `READ_TO_XML_V2` method has the following returning parameters:

Returning Parameters

Parameter	Definition
`RV_XML`	Resulting XML as XSTRING

> **Sample Code**

```
TRY.
    DATA(lt_keys)    = lo_fdp_api->get_keys(  ).
    lt_keys[ name = 'UUID' ]-VALUE = 'FA163EE47BDD1ED9A682A2E6F1ECF696'.
```

```
    DATA(lv_data) = lo_fdp_api->read_to_xml_v2( lt_keys ).
 CATCH cx_fp_fdp_error INTO DATA(lo_exception).
ENDTRY.
```

Retrieve XML Schema Definition from Service Definition

The `GET_XSD_V2` method retrieves the XML Schema Definition (XSD) for the service definition. It has the following returning parameters:

Returning Parameters

Parameter	Definition
`RV_XML`	Resulting XSD as XSTRING

Sample Code

```
TRY.
   DATA(lv_xml)     = lo_fdp_api->get_xsd_v2(  ).
 CATCH cx_fp_fdp_error INTO DATA(lo_exception).
ENDTRY
```

Read XML Data for a Service Definition Using Draft Functionality

By default, for draft-enabled services, the `IsActiveEntity` property is exported implicitly, which describes if the active or the draft data should be read. Because this property is always false, by default, it needs to be overwritten during a service call to read the active data from the service definition.

You can use the following sample code. Mind that you need to initiate the business data reader first (see above).

Sample Code

```
TRY.
    DATA(lt_keys)    = lo_fdp_api->get_keys(  ).
    lt_keys[ name = 'UUID' ]-VALUE = 'FA163EE47BDD1ED9A682A2E6F1ECF696'.
    DATA(lv_data) = lo_fdp_api->read_to_xml_v2(
        it_select     = lt_keys
        it_select_add = VALUE if_fp_fdp_api=>tt_select_keys( name =
'IsActiveEntity' value = 'X' data_type =
'ABAP_BOOL' )

    ).
 CATCH cx_fp_fdp_error INTO DATA(lo_exception).
ENDTRY.
```

4.2.15.13.2.4 PDF Merger API

You can use a class-based API to merge the content of different PDF files into a single PDF file. The result file contains all pages of the source files in the sequence of the original files.

If you want to merge PDF files, you require an instance of class `CL_RSPO_PDF_MERGER`. To create this instance, class `CL_RSPO_PDF_MERGER` provides the method `CREATE_INSTANCE`. After using this method, the following two steps are necessary:

- With method `ADD_DOCUMENT` you can add the data of a PDF document to the list of files you want to merge. You should call this method for each source file.
- With method `MERGE_DOCUMENTS` you can start the merge process. The method returns the data of the merged document.

Sample Code

```
...
    " Create an instance of the PDF merger class
    DATA(l_merger) = cl_rspo_pdf_merger=>create_instance( ).
    " Add the data of the first PDF document to the list of files which shall
be merged
    l_merger->add_document( l_data_of_first_pdf ).
    " Add the data of the second PDF document
    l_merger->add_document( l_data_of_second_pdf ).
    TRY.
        " Merge both documents and receive the result
          DATA(l_merged_PDF) = l_merger->merge_documents( ).
      CATCH cx_rspo_pdf_merger INTO DATA(l_exception).
        " Add a useful error handling here
    ENDTRY.
...
```

If you try to merge damaged documents, or documents that don't fulfill the requirements of the PDF merger, an exception of type `CX_RSPO_PDF_MERGER` is raised. The exception object contains the following attributes that contain more information regarding the problem:

- `DOCUMENT_INDEX`: The index of the PDF document that was responsible for the exception
- `ERROR_CODE`: The internal error code of the exception
- `ERROR_TEXT`: The description text of the error code

The PDF merger has the following restrictions:

- It can only handle PDF files.
- It can't handle encrypted PDF files. Usually, you can check whether a PDF file is encrypted by checking the *Security* section of the file properties. Another possibility is that you open the file with a text editor and search for the key word **Encrypt**, which marks an encrypted file.
- It can't handle damaged PDF files. For example, if parts of the file are missing, or if some bytes were changed, the file can't be processed. PDF files are usually binary data, so any conversion like a code page conversion damages the file.
- Further restrictions can be found in SAP note 2264208.

Related Information

Class and Interface of the PDF Merger API [page 1602]

4.2.15.13.2.4.1 Class and Interface of the PDF Merger API

Class `CL_RSPO_PDF_MERGER` uses interface `IF_RSPO_PDF_MERGER`. Find out which public methods it contains.

Context

The following public methods exist in interface `IF_RSPO_PDF_MERGER` :

CREATE_INSTANCE (static)

Create an instance of the PDF merger class.

Name	Description
Returning parameter	
`MERGER_INSTANCE`	PDF merger instance: A reference to interface `IF_RSPO_PDF_MERGER`

ADD_DOCUMENT

Add a PDF document to the list of files that you want to merge.

Name	Description
Importing parameter	
`DOCUMENT`	Data of the PDF document that you want to merge (type `XSTRING`)

MERGE_DOCUMENTS

Merge all PDF documents into a single PDF file.

Name	Description
Returning parameter	
`MERGED_DOCUMENT`	Data of the merged document (type `XSTRING`)
Exceptions	
`CX_RSPO_PDF_MERGER`	Exceptions of the PDF merger

Related Information

PDF Merger API [page 1601]

4.2.15.13.3 Emailing

This chapter covers the latest applications for sending, monitoring, and configuring emails.

It focuses exclusively on the latest email API and its related applications. The use of older mail APIs, which some SAP applications might still use, isn't recommended.

The SAP component `BC-SRV-COM` email APIs use SMTP as the sending protocol. They conform to `RFC 822` and `RFC 5322`, ensuring secure and error-free communication between email senders and recipients.

4.2.15.13.3.1 Email Templates

An email template is a development object that serves as a predefined layout for generating and automatically sending emails from SAP applications. Email templates can be created using the editor in ABAP Development Tools (ADT) for Eclipse.

During template creation, placeholders can be inserted to enable dynamic content, such as recipient names or other message-specific details. At runtime, when emails are generated from the template, these placeholders are automatically replaced with actual values. The placeholders correspond to fields in the referenced Core Data Services (CDS) view, and the actual values are determined based on the key values provided during execution. This approach enables the mass distribution of emails with personalized, recipient-specific information.

Each email template is a transportable development object and can include both HTML and plain text content. Once activated, templates are ready to be used for rendering and sending emails.

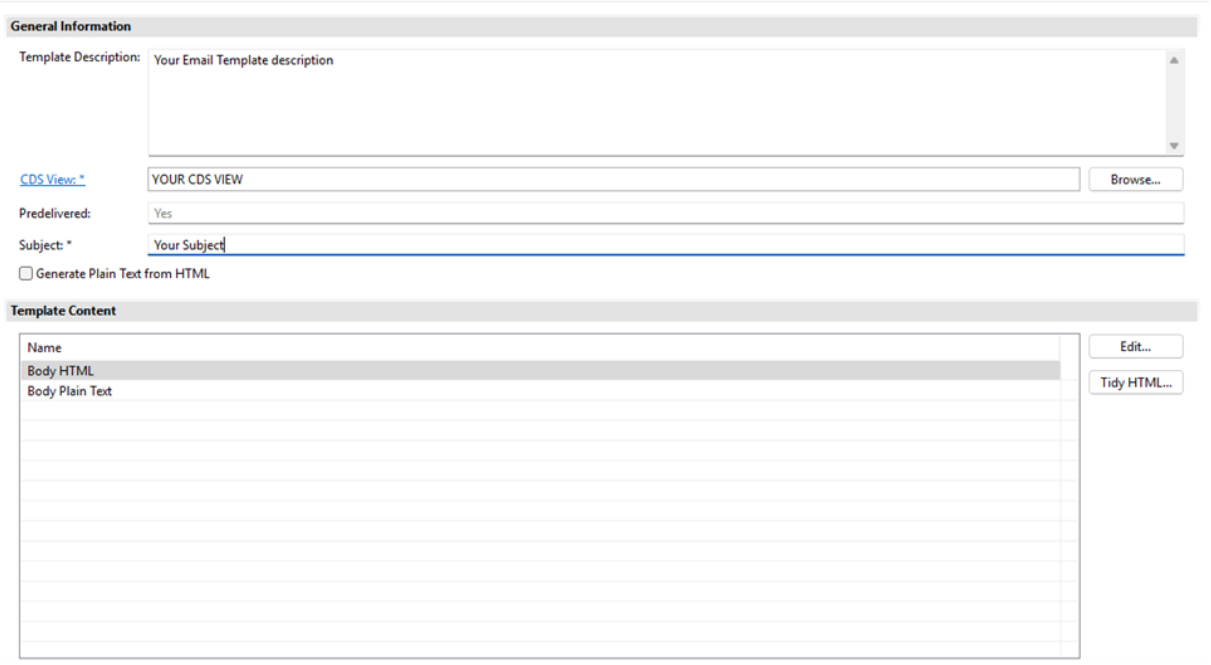
4.2.15.13.3.1.1 Email Templates Editor

You would use this to standardize and automate email generation by creating reusable templates in ADT that automatically populate placeholders with data from CDS views, eliminating manual email composition and ensuring consistency.

You can create pre-delivered email templates for use with the email API `CL_BCS_MAIL_MESSAGE` or in the Fiori app *Maintain Email Templates*.

Use the email template renderer to use the created templates for sending emails. The rendering API will select the data from the corresponding CDS view with the provided key and insert these values into the placeholders used in the template.

Create new email templates using the editor in the ABAP Development Tools (ADT), as shown in the screenshot below.



Each template needs a CDS view to supply data for the placeholders in the emails. The email template also needs a subject and can include a description to clarify its intended use. The CDS view serves as the data source for any email rendered from the email template. Implementation checks ensure that the email template and its CDS view meet the requirements for use.

Implementation checks

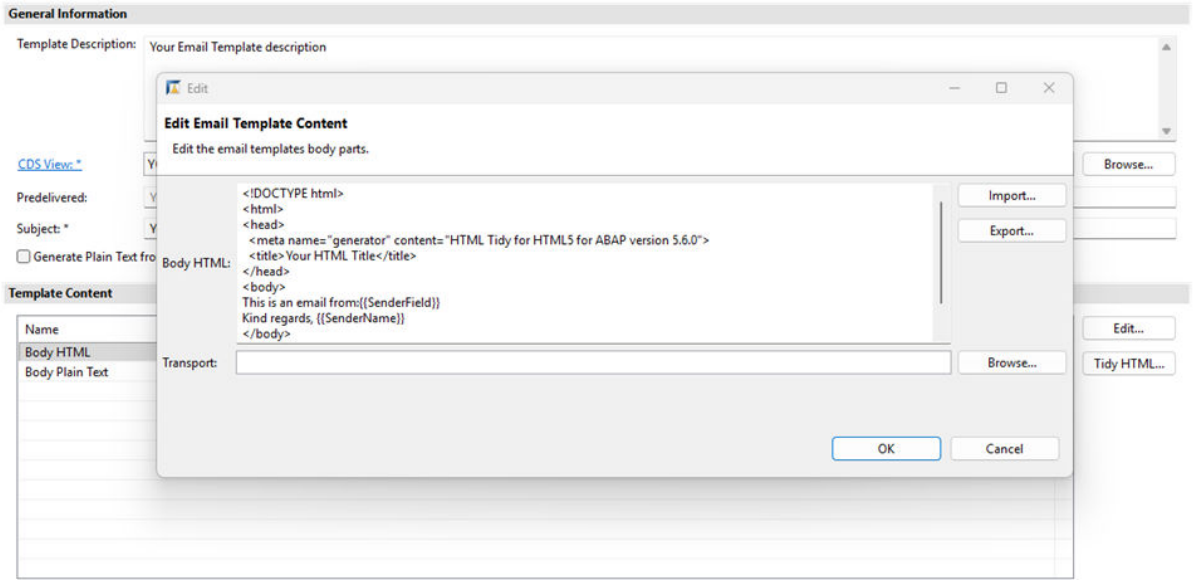
The CDS view must meet the following criteria:

- It should either be released for use in cloud development or be in the same package as the calling API.
- The CDS view must be supported and created for use in an email template. You can indicate this by adding the supported capabilities or modeling pattern annotation *OUTPUT_EMAIL_DATA_PROVIDER* to the CDS view. For more details, see the section on Supported Capabilities for CDS Views.

The email template must be in an active state, meaning it shouldn't be inactive, deprecated or obsolete.

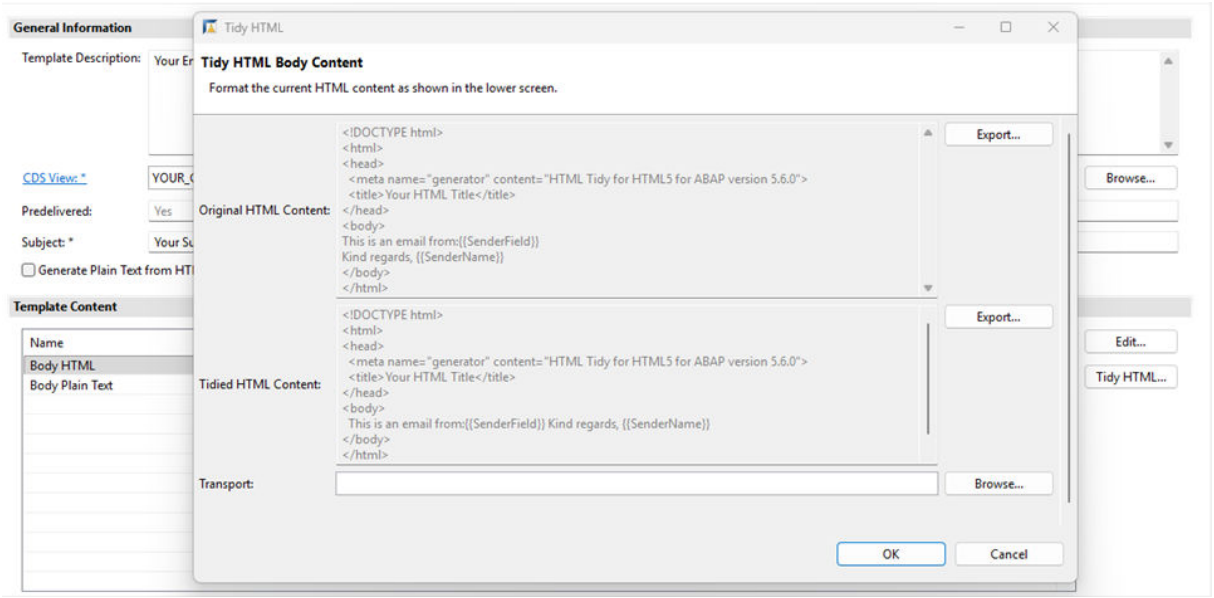
Email Template Content

You can maintain the content of an email template using the *Edit* action for both Body HTML and Body Plain Text. This action also lets you import or export the text fields. You must add any changes to the content to a transport request. If the email template is already assigned to a transport request, the *Transport* field is filled and can't be changed. If the *Transport* field is empty, you can choose a transport or use an automatically generated transport.



Tidy the HTML Content

To tidy up HTML content, use the *Tidy HTML* action button. This action provides a view of both the current HTML content and a tidied version. To keep the tidied version, click the *OK* button. If the *Transport* field is empty, choose a transport or use an automatically generated one.



Translations of template texts

Translations for each email template can be created in the Fiori app Maintain Translations. The ADT editor will only show the email template in the original object language.

4.2.15.13.3.1.2 Email Template Rendering

Send mails using the Simple Message Transfer Protocol (SMTP) and render the mail content from email templates.

Prerequisites

You must create a predelivered email template using the Email Template editor in ADT or copy an existing predelivered Email Template in the Fiori app *Maintain Email Templates*. Both methods provide an Email Template object which can be used for the rendering

Application Programming Interface (API)

Use the class `CL_BCS_MAIL_MESSAGE` to create an email request and the class `CL_SMTG_EMAIL_TEMPL_RENDERER` to render an email template. The email request will then use the mail content and subject defined by the email template.

The field values of the email template will be selected from the corresponding CDS view according to the key values which are handed over to the email template renderer.

Sample Code

```
data lt_key_table type if_smtg_email_template=>ty_gt_data_key.
data lo_template type ref to if_smtg_email_templ_renderer.
try.
    "create email request
    data(lo_message) = cl_bcs_mail_message=>create_instance( ).
    "sender
    lo_message->set_sender( 'noreply@yourcompany.com' ).
    "recipient
    lo_message->add_recipient( 'recipient@yourcompany.com' ).
    "create email template
    lo_template =
cl_smtg_email_templ_renderer=>create_instance( iv_template_id =
'Your_Template_ID' ).
    "select only the key values of the template data
    append value #(
      name = 'Key_field1'
      value = 'Key_value1'
    ) to lt_key_table.
    "render email template
    lo_template->render_email_message(
      io_message  = lo_message
      iv_language = sy-langu
      it_data_key = lt_key_table ).
    "send email
    lo_message->send( ).
  catch cx_bcs_mail cx_smtg_email_common into data(lv_mail_error).
    "handle exception here
endtry.
```

You can find more information about the mail API on the following page: Sending Mails Using SMTP .

4.2.15.13.3.2 Sending Mails Using SMTP

Send mails using the Simple Message Transfer Protocol (SMTP).

Prerequesites

It is necessary to complete the communication scenario Integrating Outbound Emails Using SMTP.

Verify if the email address checks are activated, as described in Configuration of the System Email Outbound [page 1609]. Enabling these checks helps to ensure sender and recipient address validation, improves delivery reliability, and simplifies compliance, especially in non-productive systems.

Application Programming Interface (API)

Use the `CL_BCS_MAIL_MESSAGE` class to create and send mails. You can specify sender and recipient. Furthermore, you can add textual body parts using class *CL_BCS_MAIL_TEXTPART* or binary attachments (for example, PDF documents) represented by class *CL_BCS_MAIL_BINARYPART*, respectively.

With the send method of the API, the mail is sent via the mail server configured in the destination using the cloud connector. Sending is performed synchronously.

Sample Code

```
try.
    data(lo_mail) = cl_bcs_mail_message=>create_instance( ).
    lo_mail->set_sender( 'noreply@yourcompany.com' ).
    lo_mail->add_recipient( 'recipient1@yourcompany.com' ).
    lo_mail->add_recipient( iv_address = 'recipient2@yourcompany.com' iv_copy
= cl_bcs_mail_message=>cc ).
    lo_mail->set_subject( 'Test Mail' ).
    lo_mail->set_importance( cl_bcs_mail_message=>importance-low ).
    lo_mail->set_sensitivity( cl_bcs_mail_message=>sensitivity-confidential ).
    lo_mail->set_additional_mime_headers( value #( ( field_name =
'Additional_Header' field_body = 'Header_Body' ) ) ).
    lo_mail->set_main( cl_bcs_mail_textpart=>create_text_html( '<h1>Hello</
h1><p>This is a test mail.</p>' ) ).
    lo_mail->add_attachment( cl_bcs_mail_textpart=>create_text_plain(
      iv_content      = 'This is a text attachment'
      iv_filename     = 'Text_Attachment.txt'
    ) ).
    lo_mail->add_attachment( cl_bcs_mail_textpart=>create_instance(
      iv_content      = '<note><to>John</to><from>Jane</from><body>My nice
XML!</body></note>'
      iv_content_type = 'text/xml'
      iv_filename     = 'Text_Attachment.xml'
    ) ).

    data(lo_mail_status_monitor) = lo_mail->send( ).

    lo_mail_status_monitor->get_email_status(
      importing
        es_mail_status         = data(ls_mail_status)
        et_recipients_statuses = data(lt_recipients_statuses) ).
```

```
  catch cx_bcs_mail into data(lx_mail).
    “handle exceptions here
endtry.
```

> **Note**
>
> The use of additional MIME header fields is only advisable if these fields are universally recognized. Please be aware that SAP doesn't hold accountability for any issues that may occur due to these headers' usage. Header fields that can be placed using set-methods, along with the `Date`, `Message-ID`, and `Content-Type` header fields, are not allowed.

Bodyparts in a Message

Each mail message consists of one or more body parts (MIME parts). A body part consists of the text or binary content, the type and additional attributes like the filename for an attachment. A main body part (`SET_MAIN`) must be set and additional body parts can be set as attachments (`ADD_ATTACHMENT`) or alternatives (`ADD_MAIN_ALTERNATIVE`). Attachments are usually files that are attached to the mail. Alternative body parts are different representations of the main part of the email. For example, a mail can have an HTML representation and clients that can only display text or plain text will see this alternative body part.

The factory methods `CREATE_TEXT_PLAIN`, `CREATE_TEXT_HTML` of class `CL_BCS_MAIL_TEXTPART` can be used for the most common body parts. If a body part is to be sent with a different content-type, the `CREATE_INSTANCE` method can be used and a content-type can be passed.

Sending Emails Asynchronously

In RESTful application programming (RAP) or in other transactional contexts, you may need to send an email asynchronously from time to time. This can be due to the performance, since external communication can take a while, or because you want to send an email when the `COMMIT WORK` statement is triggered at the end of a successful transaction.

An email can be sent asynchronously with the method `SEND_ASYNC`. See the following example:

> **Sample Code**

```
try.
    data(lo_mail) = cl_bcs_mail_message=>create_instance( ).
    lo_mail->set_sender( 'noreply@yourcompany.com' ).
    lo_mail->add_recipient( 'recipient1@yourcompany.com' ).
    " ...
    lo_mail->send_async( ).
  catch cx_bcs_mail into data(lx_mail).
    “handle exceptions here
endtry.
```

After calling the `SEND_ASYNC` method, a background process is triggered. The process can be monitored in the Fiori app Monitor Email Transmissions [page 3139].

> ⚠ Caution
>
> The process must call `COMMIT WORK` at the end of a transaction, otherwise the background process will not start and the email status will remain **Waiting** in the *Monitor Email Transmissions* app.

Email Status Monitoring

The status of an email can be monitored using the Fiori app *Monitor Email Transmission*. Additionally, it's possible to implement a monitoring instance directly in the backend for each email request. This monitoring instance, which is an implementation of the interface `IF_BCS_MAIL_STATUS_MONITOR`, provides information about the email status as well as the status of each recipient, including the SMTP response. The `send()` method provides the same statuses as exporting parameters.

To obtain this monitoring instance, you can either receive it as the return parameter of the methods `send()` or `send_async()`, or create a new instance using the factory method `create_mail_status_monitor()` of the class `CL_BCS_MAIL_MESSAGE`.

4.2.15.13.3.3 Configuration of the System Email Outbound

Configure the email system using class `CL_BCS_MAIL_SYSTEM_CONFIG`. This configuration provides additional functionality to an SMTP email configuration.

Configuring the Email Outbound Check

You can restrict the email outbound by checking the sender and recipient addresses. The check is executed when the email is sent. If the address doesn't match with the sender or recipient domains or addresses on the allowlist, the email will not be sent and is not displayed in the app.

In addition, you can set a default sender address.

Activating the Email Outbound Check

Use class `CL_BCS_MAIL_SYSTEM_CONFIG` and method `SET_ADDRESS_CHECK_ACTIVE()` to activate the email outbound check. The importing parameter `ABAP_TRUE` activates the check, `ABAP_FALSE` deactivates it. The functionality is inactive per default.

Configuring the Email Outbound Allow Tables

To restrict the email outbound, two tables can be filled with allowed domains or email addresses. The first table contains allowed recipients (filled with method `ADD_ALLOWED_RECIPIENT_DOMAINS()`). The second table contains allowed senders (filled with method `ADD_ALLOWED_SENDER_DOMAINS()`). The check can be used for both the sender address and recipient address(es). If either all sender or recipient addresses should be allowed, add `a '*'` to the corresponding allow domain table.

The configuration must be done as system customizing. It's not allowed to set the configuration in a session together with any email outbound.

Configuring the Default Sender

In addition to the allowed domains, you can define a default sender. This default sender will be used as sender at anytime when no sender has been defined for an outgoing email. Please keep in mind that the default sender must be a valid email address. If no default sender is maintained and no sender is set in the outgoing email, the sender will be set to `do.not.reply@(system-default-domain)`. The default sender will only be used if the Email Outbound Check is set to active. Otherwise, no email sender will be set per default and the email might fail with an error.

> **Note**
>
> You can define only one default sender per system.

Configuration with Upload Business Configuration

You can configure the following entities for the **Upload Business Configuration** app:

- SMAIL_VM_DF_ADDR
- SMAIL_VM_RCV_DOM
- SMAIL_VM_SND_DOM

Configuration with Application Programming Interface (API)

To maintain the allowed domains and the default sender, further methods are provided to read or delete the current table data:

> **Sample Code**

```
DATA(config_instance) = cl_bcs_mail_system_config=>create_instance( ).
DATA recipient_domains TYPE cl_bcs_mail_system_config=>tyt_recipient_domains.
DATA sender_domains TYPE cl_bcs_mail_system_config=>tyt_sender_domains.
recipient_domains = VALUE #( ( 'recipient1domain.com' )
( 'recipient2domain.com' ) ).
sender_domains = VALUE #( ( 'sender1domain.com' ) ( 'sender2domain.com' ) ).
"Add allowed domains
TRY.
    config_instance->set_address_check_active( abap_true ).
    config_instance->add_allowed_recipient_domains( recipient_domains ).
    config_instance->add_allowed_sender_domains( sender_domains ).
    config_instance->modify_default_sender_address( iv_default_address =
'DefaultSender@yourcompany.com'
                                                    iv_default_name = 'Default
Sender' ).
  CATCH cx_bcs_mail_config INTO DATA(write_error).
    "handle exception
ENDTRY.
"Read allowed domains
DATA(allowed_recipient_domains) = config_instance-
>read_allowed_recipient_domains( ).
DATA(allowed_sender_domains) = config_instance-
>read_allowed_sender_domains( ).
config_instance->read_default_sender_address(
  IMPORTING
    ev_default_sender_address = DATA(default_sender_address)
    ev_default_sender_name = DATA(default_sender_name) ).
"Delete allowed domains
TRY.
    config_instance->delete_allowed_rec_domains( allowed_recipient_domains ).
    config_instance->delete_allowed_sender_domains( allowed_sender_domains ).
    config_instance-
>delete_default_sender_addr( 'DefaultSender@yourcompany.com' ).
  CATCH cx_bcs_mail_config INTO DATA(deletion_error).
```

```
    "handle exception
ENDTRY.
```

Configuring the Email Expiry Date

The expiry date of email send attempts can be defined using method `SET_DAYS_UNTIL_MAIL_EXPIRES()`. The default value for mail expiry is 30 days. After this time, sent and failed emails attempts will be deleted and no longer visible within the *Monitor Email Transmission* app.

If the `DELETE_DAYS_UNTIL_MAIL_EXPIRES()` method is used, the expiry date is set back to 30 days.

4.2.15.13.3.4 Email Address Check

Email Address Validation

The class `CL_MAIL_ADDRESS` implements the interface `IF_MAIL_ADDRESS` and enables the user to validate the provided email address. This class checks whether the provided email address is syntactically valid according to the RFC 5322 standard.

Create an E-Mail Address instance

Method	Description
Create_instance	Provides an email address instance.
validate	Validates whether a string is a syntactic valid mail address (according to RFC 5322 Standard).

Methods

Sample Code

```
Try.
        Data(lo_mail_address) =
cl_mail_address=>create_instance( iv_address_string = ‘testmail@ok.com’ ).
    Catch cx_bcs_mail into data(lx_err).
        "exception handling
endtry.
```

```
Data(lv_address_valid) = lo_mail_address->validate( ).
```

4.2.15.14 Units of Measurement

Many business applications use units of measurement in their business processes. To standardize these processes, you need a central maintenance of units and related dimensions. Beside that, there's a business need for conversion between different units.

We provide a subset of common standardized units, dimensions, and ISO codes for use as predelivered content. In addition, you need to define customer-owned units and dimensions in customer applications.

> ⓘ Note
>
> We recommend that you maintain units of measurement using the following configurations of the *Custom Business Configurations* SAP Fiori app:
>
> - *Units of Measurement*
> - *Dimensions for Units of Measurement*
> - *ISO Codes for Units of Measurement*
>
> For more information, see Configure Units of Measurement [page 2990] and Custom Business Configurations App [page 2970].

How are Units and Dimensions Linked?

Units are related to a dimension. In each dimension, a unit is defined as an SI unit (International System of Units). This is the basis for the conversion from one unit to another.

Warnings

Changing a unit of measurement (UoM) may have an impact on your applications, especially on the critical fields.

UoMs are used across applications. An initial set of UoMs is delivered in the system. You can adapt UoMs as per your requirements. Be aware that by adding, changing, or deleting the customizing, you are changing the initial set of UoMs delivered in the system.

> ⚠ Caution
>
> Pay special attention if UoMs are already saved in transactional data of productive business processes. Changes of the fields marked as critical can result in serious inconsistencies.
>
> We strongly recommend that you do **not** delete any dimensions, ISO codes, and units that are already defined in the system.

Critical Fields

Critical Fields for...	Description
Dimensions	*SI Unit* The *SI Unit* is the base unit of a dimension for conversion. By changing the base unit, the conversion will be performed on a different base unit defined.
Units of Measurement	• Numerator • Denominator • Exponent • Additive Constant • ISO Code • Primary Code **⚠ Caution** Numerator, denominator, exponent, and additive constant define the conversion factor with respect to the SI units. Any changes of the conversion factor will change the output result of a conversion between units under the same dimension. **⚠ Caution** Changing the ISO code and primary code may have an impact on applications. For example, ISO codes are used as normed measurement unit IDs for electronic data interchange (EDI). If no ISO code is assigned to a unit, you may encounter issues in EDI.

Related Information

Maintaining Dimensions [page 1614]
Maintaining Units of Measurement [page 1624]
Retrieving and Resetting Standard Units of Measurement [page 1636]
Conversion Functions for Units of Measurement [page 1640]

4.2.15.14.1 Maintaining Dimensions

Class `CL_UOM_DIM_MAINTENANCE` provides methods for maintaining a dimension.

> ⚠ Caution
>
> Changing the initial set of configurations has implications. Before proceeding with any change, familiarize yourself with the warnings in Units of Measurement [page 1612].

> ⓘ Note
>
> We recommend that you maintain units of measurement using the following configurations of the *Custom Business Configurations* SAP Fiori app:
>
> - *Units of Measurement*
> - *Dimensions for Units of Measurement*
> - *ISO Codes for Units of Measurement*
>
> For more information about how to use the SAP Fiori app for units of measurement, see Configure Units of Measurement [page 2990] and Custom Business Configurations App [page 2970].

4.2.15.14.1.1 Creating a Dimension

Use method `CREATE` to create a new dimension.

> ⚠ Caution
>
> Changing the initial set of configurations has implications. Before proceeding with any change, familiarize yourself with the warnings in Units of Measurement [page 1612].

Import Parameters

Parameter Name	Field Name	Value Help
DIM_CRE_TS		Structure for creating a dimension
	DIMID	Dimension key
	TXDIM	Description of the dimension key
	LENGTH	Length exponent of the dimension
	MASS	Mass exponent of the dimension
	TIME	Time exponent of the dimension

Parameter Name	Field Name	Value Help
	CURRENT	Electric current exponent of the dimension
	TEMPERATURE	Temperature exponent of the dimension
	MOLE_QTY	Mole quantity exponent of the dimension
	LUMINOSITY	Light exponent of the dimension
	HASUNITSWITHTEMPERATURESPEC	Indicates whether the dimension has units with a temperature specification
	HASUNITSWITHPRESSURESPEC	Indicates whether the dimension has units with a pressure specification

Export Parameters

Parameter Name	Value Help
ERROR	**Space**: No error **'X'**: Save error

Note

Class exception `CX_UOM_ERROR` is raised to check the integrity of the data import parameters.

Sample Code

```
CLASS zcl_uom_dimension_create_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC.

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_uom_dimension_create_test IMPLEMENTATION.

  METHOD if_oo_adt_classrun~main.

    DATA(lo_dim) = cl_uom_dim_maintenance=>get_instance( ).
    DATA(ls_dim) = VALUE cl_uom_dim_maintenance=>ty_dim_cre_ts( dimid =
'ZNEWDI'
                                                                txdim = 'New
Dimension'
                                                                mass  = 89 ).
```

```
    TRY.
        lo_dim->create( EXPORTING dim_cre_ts = ls_dim
                        IMPORTING error      = DATA(lv_error) ).

      CATCH cx_uom_error INTO DATA(lo_error).
        out->write( |Exception raised| ).
        out->write( lo_error->get_text( ) ).
    ENDTRY.
    IF lv_error = abap_true.
      out->write( |Error occurred while saving the data in the database| ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.1.2 Changing a Dimension

Use method UPDATE to change an existing dimension.

> ⚠ Caution
>
> Changing the initial set of configurations has implications. Before proceeding with any change, familiarize yourself with the warnings in Units of Measurement [page 1612].

Import Parameters

Parameter Name	Field Name	Value Help
DIMID		Dimension key
DIM_UPD_TS		Structure for updating a dimension ⚠ Caution Be aware that all the fields belonging to the structure will be updated.
	TXDIM	Description of the dimension key
	SI_UNIT	Base unit of a dimension
	LENGTH	Length exponent of the dimension
	MASS	Mass exponent of the dimension
	TIME	Time exponent of the dimension

Parameter Name	Field Name	Value Help
	CURRENT	Electric current exponent of the dimension
	TEMPERATURE	Temperature exponent of the dimension
	MOLE_QTY	Mole quantity exponent of the dimension
	LUMINOSITY	Light exponent of the dimension
	HASUNITSWITHTEMPERATURESPEC	Indicates whether the dimension has units with a temperature specification
	HASUNITSWITHPRESSURESPEC	Indicates whether the dimension has units with a pressure specification

Export Parameters

Parameter Name	Value Help
ERROR	**Space**: No error **'X'**: Save error

Note

Class exception `CX_UOM_ERROR` is raised to check the integrity of the data import parameters.

Sample Code

```
CLASS zcl_uom_dimension_update_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.

  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_uom_dimension_update_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
   DATA(lo_dim) = cl_uom_dim_maintenance=>get_instance( ).
   TRY.
        lo_dim->read( EXPORTING dimid    = 'ZNEWDI'
              IMPORTING dim_st = DATA(ls_dim) ).
   DATA(ls_upd_dim) = VALUE cl_uom_dim_maintenance=>ty_dim_upd_ts(
                      BASE CORRESPONDING #( ls_dim )
                                          txdim = 'Update Dimension'
```

```
                                          mass  = 88
                                          length = 89 ).
    lo_dim->update( EXPORTING dimid      = 'ZNEWDI'
                              dim_upd_ts = ls_upd_dim
                    IMPORTING error      = DATA(lv_error) ).
   CATCH cx_uom_error INTO DATA(lo_error).
      out->write( |Exception raised| ).
      out->write( lo_error->get_text( ) ).
   ENDTRY.
   IF lv_error = abap_true.
     out->write( |Error occurred while updating the data in the database| ).
   ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.1.3 Deleting a Dimension

Use method `DELETE` to delete a dimension.

> ⚠ Caution
>
> Changing the initial set of configurations has implications. Before proceeding with any change, familiarize yourself with the warnings in Units of Measurement [page 1612].

Import Parameters

Parameter Name	Value Help
DIMID	Dimension key

Export Parameters

Parameter Name	Value Help
ERROR	**Space**: No error **'X'**: Save error

> ⓘ Note
>
> Class exception `CX_UOM_ERROR` is raised to check the integrity of the data import parameters.

> {·} Sample Code

```
CLASS zcl_uom_dimension_delete_test DEFINITION
  PUBLIC
```

```
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_uom_dimension_delete_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.

 DATA(lo_dim) = cl_uom_dim_maintenance=>get_instance( ).

    TRY.
        lo_dim->delete( EXPORTING dimid = 'ZNEWDI'
                        IMPORTING error = DATA(lv_error) ).
      CATCH cx_uom_error INTO DATA(lo_error).
        out->write( |Exception raised| ).
        out->write( lo_error->get_text( ) ).
    ENDTRY.
    IF lv_error = abap_true.
      out->write( |Error occurred while deleting the data from the
database| ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.1.4 Reading a Dimension

Use method READ to read a dimension.

Import Parameters

Parameter Name	Value Help
DIMID	Dimension key
LANGUAGE	Language key (optional) **Note** If no language key is provided, the default logon language is used.

Export Parameters

Parameter Name	Field Name	Value Help
DIM_ST		Structure for reading a dimension
	DIMID	Dimension key
	TXDIM	Description of the dimension key
	SI_UNIT	Base unit of a dimension
	SI_TXT	Description of the base unit
	LENGTH	Length exponent of the dimension
	MASS	Mass exponent of the dimension
	TIME	Time exponent of the dimension
	CURRENT	Electric current exponent of the dimension
	TEMPERATURE	Temperature exponent of the dimension
	MOLE_QTY	Mole quantity exponent of the dimension
	LUMINOSITY	Light exponent of the dimension
	HASUNITSWITHTEMPERATURESPEC	Indicates whether the dimension has units with a temperature specification
	HASUNITSWITHPRESSURESPEC	Indicates whether the dimension has units with a pressure specification
	LANGUAGE	Language key

Note

Class exception `CX_UOM_ERROR` is raised if no dimension is found.

Sample Code

```
CLASS zcl_uom_dimension_read_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
```

```
ENDCLASS.

CLASS zcl_uom_dimension_read_test IMPLEMENTATION.

  METHOD if_oo_adt_classrun~main.

    DATA(lo_dim) = cl_uom_dim_maintenance=>get_instance( ).

    TRY.
        lo_dim->read( EXPORTING  dimid  = 'AAAADL'
                                 language = 'E'
                      IMPORTING  dim_st = DATA(ls_dim) ).
      CATCH cx_uom_error.
    ENDTRY.

    out->write( ls_dim-DIMID ).
    out->write( ls_dim-txdim ).

  ENDMETHOD.

ENDCLASS.
```

4.2.15.14.1.5 Maintaining the Translation of a Dimension

Use method `MAINTAIN_TRANSLATION` to maintain the translation of a dimension for a particular language.

> ⚠ Caution
>
> Changing the initial set of configurations has implications. Before proceeding with any change, familiarize yourself with the warnings in Units of Measurement [page 1612].

Import Parameters

Parameter Name	Value Help
LANGUAGE	Language key
DIMID	Dimension key
TXDIM	Description of the dimension key

Export Parameters

Parameter Name	Value Help
ERROR	**Space**: No error **'X'**: Save error

> **Note**
>
> Class exception `CX_UOM_ERROR` is raised to check the integrity of the data import parameters.

> **Sample Code**

```
CLASS zcl_dim_maintain_transl_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun .
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_dim_maintain_transl_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
     DATA(lo_dim) = cl_uom_dim_maintenance=>get_instance( ).
     TRY.
        lo_dim->maintain_translation( EXPORTING language = 'F'
                                                dimid    = 'ZNEWDI'
                                                txdim    = 'Dimension FR'
                                      IMPORTING
                                            error = DATA(lv_error) ).
CATCH cx_uom_error INTO DATA(lo_error).
        out->write( |Exception raised| ).
        out->write( lo_error->get_text( ) ).
     ENDTRY.
     IF lv_error = abap_true.
        out->write( |Error occurred while updating the data in the
database| ).
     ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.1.6 Deleting the Translation of a Dimension

Use method `DELETE_TRANSLATION` to delete translation of a dimension for a particular language.

> **Caution**
>
> Changing the initial set of configurations has implications. Before proceeding with any change, familiarize yourself with the warnings in Units of Measurement [page 1612].

Import Parameters

Parameter Name	Value Help
LANGUAGE	Language key
DIMID	Dimension key

Export Parameters

Parameter Name	Value Help
ERROR	**Space**: No error **'X'**: Save error

Note

Class exception `CX_UOM_ERROR` is raised to check the integrity of the data import parameters.

Sample Code

```
CLASS zcl_dim_delete_transl_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun .
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_dim_maintain_transl_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
     DATA(lo_dim) = cl_uom_dim_maintenance=>get_instance( ).
TRY.
        lo_dim->delete_translation( EXPORTING dimid    = 'ZNEWDI'
                                              language = 'F'
                                    IMPORTING
                                          error = DATA(lv_error) ).
     CATCH cx_uom_error INTO DATA(lo_error).
        out->write( |Exception raised| ).
        out->write( lo_error->get_text( ) ).
     ENDTRY.
     IF lv_error = abap_true.
        out->write( |Error occurred while updating the data in the
database| ).
     ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.2 Maintaining Units of Measurement

Class `CL_UOM_MAINTENANCE` provides methods for maintaining units of measurement.

> ⚠ Caution
>
> Changing the initial set of configurations has implications. Before proceeding with any change, familiarize yourself with the warnings in Units of Measurement [page 1612].

> ⓘ Note
>
> We recommend that you maintain units of measurement using the following configurations of the *Custom Business Configurations* SAP Fiori app:
>
> - *Units of Measurement*
> - *Dimensions for Units of Measurement*
> - *ISO Codes for Units of Measurement*
>
> For more information about how to use the SAP Fiori app for units of measurement, see Configure Units of Measurement [page 2990] and Custom Business Configurations App [page 2970].

4.2.15.14.2.1 Creating a Unit of Measurement

Use method `CREATE` to create a unit of measurement.

> ⚠ Caution
>
> Changing the initial set of configurations has implications. Before proceeding with any change, familiarize yourself with the warnings in Units of Measurement [page 1612].

Import Parameters

Parameter Name	Field Name	Value Help
UNIT_DIMID		Dimension key
UNIT_INT		Internal unit of measurement
UNIT_CRE_TS		Structure for creating a unit of measurement
	COMMERCIAL	Commercial/external measurement unit format

Parameter Name	Field Name	Value Help
	TECHNICAL	Technical measurement unit format
	DEC_ROUND	Number of decimal places to which this measurement unit should be rounded for conversion
	NUMERATOR	Numerator for conversion to SI unit
	DENOMINATOR	Denominator for conversion into SI unit
	EXPONENT	Base ten exponent for conversion to SI unit
	CONSTANT	Additive constant for conversion to SI unit
	DEC_DISP	Number of decimal places with which this measurement unit is displayed
	ISOCODE	ISO code for measurement units. An ISO code can be assigned to several internal measurement units of a dimension.
	PRIMARY	Unit of measure flagged as a primary unit for an ISO code **Space**: Not primary **'X'**: Set as primary
	TEXT	Description of a unit of measurement
	LONG_TEXT	Long description of a unit of measurement
	ISCOMMERCIAL	Flag for commercial measurement units
	ISVALUEBASE	Value-based commitment indicator
	ISGRPBYFAM	Measurement units grouped by family
	TEMPERATURE	Temperature value
	TEMPERATUREUNIT	Temperature unit
	PRESSURE	Pressure value
	PRESSUREUNIT	Pressure unit

Export Parameters

Parameter Name	Value Help
ERROR	**Space**: No error **'X'**: Save error

Note

Class exception `CX_UOM_ERROR` is raised to check the integrity of the data import parameters.

Sample Code

```
CLASS zcl_uom_unit_create_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_uom_unit_create_test IMPLEMENTATION.

  METHOD if_oo_adt_classrun~main.

    DATA(lo_uom) = cl_uom_maintenance=>get_instance( ).
    DATA(ls_unit) = VALUE cl_uom_maintenance=>ty_uom_cre_ts( commercial  =
'ZYX'
                                                             technical   =
'ZYX'
                                                             denominator =
'1'
                                                             numerator   =
'1'
                                                             dec_disp    =
'3'
                                                             long_text   =
'Create Unit' ).

    TRY.
        lo_uom->create( EXPORTING unit_dimid  = 'AAAADL'
                                  unit_int    = 'ZYX'
                                  unit_cre_ts = ls_unit
                        IMPORTING error       = DATA(lv_error) ).

      CATCH cx_uom_error INTO DATA(lo_error).
        out->write( | Exception raised | ).
        out->write( lo_error->get_text( ) ).
    ENDTRY.
    IF lv_error = abap_true.
      out->write( |Error occurred while saving the data in the database| ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.2.2 Changing a Unit of Measurement

Use method UPDATE to change a unit of measurement.

> ⚠ Caution
>
> Changing the initial set of configurations has implications. Before proceeding with any change, familiarize yourself with the warnings in Units of Measurement [page 1612].

Import Parameters

Parameter Name	Field Name	Value Help
UNIT		Internal unit of measurement
UNIT_UPD_TS		Structure for updating a unit of measurement ⚠ Caution Be aware that all the fields belonging to the structure will be updated.
	COMMERCIAL	Commercial/external measurement unit format
	TECHNICAL	Technical measurement unit format
	DEC_ROUND	Number of decimal places to which this measurement unit should be rounded for conversion
	NUMERATOR	Numerator for conversion to SI unit
	DENOMINATOR	Denominator for conversion into SI unit
	EXPONENT	Base ten exponent for conversion to SI unit
	CONSTANT	Additive constant for conversion to SI unit
	DEC_DISP	Number of decimal places with which this measurement unit is displayed

Parameter Name	Field Name	Value Help
	ISOCODE	ISO code for measurement units. An ISO code can be assigned to several internal measurement units of a dimension.
	PRIMARY	Unit of measure flagged as a primary unit for an ISO code **Space**: Not primary **'X'**: Set as primary
	TEXT	Description of a unit of measurement
	LONG_TEXT	Long description of a unit of measurement
	ISCOMMERCIAL	Flag for commercial measurement units
	ISVALUEBASE	Value-based commitment indicator
	ISGRPBYFAM	Measurement units grouped by family
	TEMPERATURE	Temperature value
	TEMPERATUREUNIT	Temperature unit
	PRESSURE	Pressure value
	PRESSUREUNIT	Pressure unit

Export Parameters

Parameter Name	Value Help
ERROR	**Space**: No error **'X'**: Save error

> **Note**
>
> Class exception `CX_UOM_ERROR` is raised to check the integrity of the data import parameters.

> **Sample Code**

```
CLASS zcl_uom_unit_update_test DEFINITION
  PUBLIC
  FINAL
```

```
  CREATE PUBLIC.

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_uom_unit_update_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    DATA(lo_uom) = cl_uom_maintenance=>get_instance( ).
TRY.
lo_uom->read( EXPORTING  unit       = 'ZYX'
              IMPORTING unit_st = DATA(ls_unit) ).
DATA(ls_upd_unit) = VALUE cl_uom_maintenance=>ty_uom_upd_ts(
                    BASE CORRESPONDING #( ls_unit )
                                          commercial   = 'ZYA'
                                          technical    = 'ZYA'
                                          denominator  = '1'
                                          numerator    = '1'
                                          dec_disp     = '5'
                                          long_text    = 'Updates Unit'
                                          text         = 'Upd Unit' ).
lo_uom->update( EXPORTING unit        = 'ZYX'
                          unit_upd_ts = ls_upd_unit
                IMPORTING error      = DATA(lv_error) ).
      CATCH cx_uom_error INTO DATA(lo_error).
        out->write( |Exception raised| ).
        out->write( lo_error->get_text( ) ).
    ENDTRY.
    IF lv_error = abap_true.
      out->write( |Error occurred while updating the data in the database| ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.2.3 Deleting a Unit of Measurement

Use method `DELETE` to delete a unit of measurement.

> ⚠ Caution
>
> Changing the initial set of configurations has implications. Before proceeding with any change, familiarize yourself with the warnings in Units of Measurement [page 1612].

Import Parameters

Parameter Name	Value Help
UNIT	Internal unit of measurement

Export Parameters

Parameter Name	Value Help
ERROR	**Space**: No error **'X'**: Save error

Note

Class exception `CX_UOM_ERROR` is raised to check the integrity of the data import parameters.

Sample Code

```
CLASS zcl_uom_unit_delete_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC.

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_uom_unit_delete_test IMPLEMENTATION.

  METHOD if_oo_adt_classrun~main.
    DATA(lo_uom) = cl_uom_maintenance=>get_instance( ).

    TRY.
        lo_uom->delete( EXPORTING unit = 'ZYX'
                        IMPORTING error = DATA(lv_error) ).

      CATCH cx_uom_error INTO DATA(lo_error).
        out->write( | Exception raised | ).
        out->write( lo_error->get_text( ) ).
    ENDTRY.
    IF lv_error = abap_true.
      out->write( |Error occurred while deleting the data from the
database| ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.2.4 Reading a Unit of Measurement

Use method READ to read a unit of measurement.

Import Parameters

Parameter Name	Value Help
UNIT	Internal unit of measurement
LANGUAGE	Language key (optional) **Note** If no language key is provided, the default logon language is used.

Export Parameters

Parameter Name	Field Name	Value Help
UNIT_ST		Structure for reading a unit of measurement
	UNIT	Internal unit of measurement
	COMMERCIAL	Commercial/external measurement unit format
	TECHNICAL	Technical measurement unit format
	DEC_ROUND	Number of decimal places to which this measurement unit should be rounded for conversion
	DIMID	Dimension key
	NUMERATOR	Numerator for conversion to SI unit
	DENOMINATOR	Denominator for conversion into SI unit
	EXPONENT	Base ten exponent for conversion to SI unit

Parameter Name	Field Name	Value Help
	CONSTANT	Additive constant for conversion to SI unit
	DEC_DISP	Number of decimal places with which this measurement unit is displayed
	ISOCODE	ISO code for measurement units. An ISO code can be assigned to several internal measurement units of a dimension.
	PRIMARY	Unit of measure flagged as a primary unit for an ISO code **Space**: Not primary **'X'**: Set as primary
	TEXT	Description of a unit of measurement
	LONG_TEXT	Long description of a unit of measurement
	ISCOMMERCIAL	Flag for commercial measurement units
	ISVALUEBASE	Value-based commitment indicator
	ISGRPBYFAM	Measurement units grouped by family
	TEMPERATURE	Temperature value
	TEMPERATUREUNIT	Temperature unit
	PRESSURE	Pressure value
	PRESSUREUNIT	Pressure unit
	LANGUAGE	Language key

Note

Class exception `CX_UOM_ERROR` is raised if no unit is found.

Sample Code

```
CLASS zcl_uom_unit_read_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC.

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
```

```
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_uom_unit_read_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.

    DATA(lo_uom) = cl_uom_maintenance=>get_instance( ).
    TRY.
        lo_uom->read( EXPORTING unit = 'ST'
                                language = 'E'
                      IMPORTING unit_st = DATA(ls_unit) ).

      CATCH cx_uom_error.
    ENDTRY.
    out->write( ls_unit-unit ).
    out->write( ls_unit-commercial ).
    out->write( ls_unit-technical ).
    out->write( ls_unit-dimid ).
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.2.5 Maintaining the Translation of a Unit of Measurement

Use method `MAINTAIN_TRANSLATION` to maintain the translation of a unit of measurement for a particular language.

> ⚠ Caution
>
> Changing the initial set of configurations has implications. Before proceeding with any change, familiarize yourself with the warnings in Units of Measurement [page 1612].

Import Parameters

Parameter Name	Value Help
LANGUAGE	Language key
UNIT_INT	Internal unit of measurement
COMMERCIAL	Commercial/external measurement unit format
TECHNICAL	Technical measurement unit format
TEXT	Description of a unit of measurement (optional)

Parameter Name	Value Help
LONG_TEXT	Long description of a unit of measurement (optional)

Export Parameters

Parameter Name	Value Help
ERROR	**Space**: No error **'X'**: Save error

Note

Class exception `CX_UOM_ERROR` is raised to check the integrity of the data import parameters.

Sample Code

```
CLASS zcl_uom_maintain_transl_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun .
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_uom_maintain_transl_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
      DATA(lo_uom) = cl_uom_maintenance=>get_instance( ).
      TRY.
          lo_uom->maintain_translation( EXPORTING language   = 'F'
                                                  unit_int   = 'ZYX'
                                                  commercial = 'zyx'
                                                  technical  = 'zyx'
                                                  text       = 'FR transl'
                                                  long_text  = 'FR transl'
                                        IMPORTING error = DATA(lv_error) ).
          CATCH cx_uom_error INTO DATA(lo_error).
             out->write( | Exception raised | ).
             out->write( lo_error->get_text( ) ).
       ENDTRY.
       IF lv_error = abap_true.
          out->write( |Error occurred while saving the data in the
database| ).
       ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.2.6 Deleting the Translation of a Unit of Measurement

Use method `DELETE_TRANSLATION` to delete translation of a unit of measurement for a particular language.

> ⚠ Caution
>
> Changing the initial set of configurations has implications. Before proceeding with any change, familiarize yourself with the warnings in Units of Measurement [page 1612].

Import Parameters

Parameter Name	Value Help
LANGUAGE	Language key
UNIT_INT	Internal unit of measurement

Export Parameters

Parameter Name	Value Help
ERROR	**Space**: No error **'X'**: Save error

> ⓘ Note
>
> Class exception `CX_UOM_ERROR` is raised to check the integrity of the data import parameters.

> {·} Sample Code

```
CLASS zcl_uom_delete_transl_test DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun .
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.
CLASS zcl_uom_delete_transl_test IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
      DATA(lo_uom) = cl_uom_maintenance=>get_instance( ).
      TRY.
          lo_uom->delete_translation( EXPORTING language   = 'F'
                                                unit_int   = 'ZYX'
```

```
                                        IMPORTING error = DATA(lv_error) ).
          CATCH cx_uom_error INTO DATA(lo_error).
             out->write( | Exception raised | ).
             out->write( lo_error->get_text( ) ).
       ENDTRY.
       IF lv_error = abap_true.
          out->write( |Error occurred while saving the data in the
database| ).
       ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.3 Retrieving and Resetting Standard Units of Measurement

Class `CL_UOM_CONTENT_HANDLER` provides methods for reading the standard units of measurement and dimensions delivered by SAP. With this functionality, you can check the standard units of measurement provided by SAP and adjust any custom changes accordingly. For more information, see Reading Standard Dimensions and Units of Measurement [page 1636].

This API also provides methods for resetting the content to the standard units of measurement, that is, dimensions, ISO codes, and units.

> ⚠ Caution
>
> Before you use the reset methods, check that custom units of measurement are not already in use by any applications. The reset operation completely refreshes the units of measurement content and reverts to the proposed standard values provided by SAP for the current release. All changes of the standard units of measurement will be removed, and the custom ones will be deleted. This includes dimensions, ISO codes, and units.
>
> When you use this API, you take full responsibility for UoM configuration changes. Use this API only when you fully understand the impact. SAP does not take responsibility for the deletion of custom units.

For more information, see Resetting Content to Standard Units of Measurement [page 1638].

4.2.15.14.3.1 Reading Standard Dimensions and Units of Measurement

Use the `GET_DIMENSIONS` method to read the standard dimensions for a unit of measurement, and the `GET_UNITS` method to read the standard units of measurement.

These methods perform the following actions based on the parameters listed below:

- Language filter (`IV_LANGUAGE`):
  If the `IV_LANGUAGE` filter is set, the API retrieves all standard dimensions/all standard units of measurement based on the specified language.

- Dimension filter (`IV_DIM`) and unit filter (`IV_UNIT`):
  If these filters are set, the API retrieves the standard dimensions or, respectively, standard units of measurement for the specified dimension/unit across all languages available in the standard units of measurement.
- Both language and dimension or unit filters (`IV_LANGUAGE` and `IV_DIM` or `IV_UNIT`):
  If both these filters are set, the API retrieves the standard dimension/standard unit of measurement based on both the specified language **and** dimension or unit filters.
- No parameters provided:
  If no parameters are set, the API retrieves all standard dimensions and units of measurement.

Import Parameters

Parameters	Value Help
IV_LANGUAGE	Specific language to filter
IV_DIM IV_UNIT	Specific dimension or unit to filter

Returning Parameters

Parameter Name	Field Name	Value Help
RT_DIMENSIONS RT_UNITS	IF_UOM_CONTENT_HANDLER=>TT_DIM_TS IF_UOM_CONTENT_HANDLER=>tt_uom_content_ts	Table type for the read dimension or units of measurement See also the *Export Parameters* section in Reading a Dimension [page 1619] and Reading a Unit of Measurement [page 1631].

> ### Note
>
> Class exception `CX_UOM_ERROR` is raised if no unit is found.

> ### Sample Code

```
CLASS zcl_read_std_uom DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun .
  PROTECTED SECTION.
  PRIVATE SECTION.
```

```
ENDCLASS.
CLASS zcl_read_std_uom IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    CONSTANTS pieces TYPE msehi VALUE 'ST' ##NO_TEXT.
    CONSTANTS dimensionless TYPE dimid VALUE 'AAAADL' ##NO_TEXT.
    DATA(lo_uom) = cl_uom_content_handler=>get_instance( ).
    TRY.
        DATA(lt_standard_uom_content) = lo_uom->get_units(
            iv_unit     = pieces
            iv_language = sy-langu
        ).
        out->write( lt_standard_uom_content ).
        DATA(lt_standard_dim_content) = lo_uom->get_dimensions(
            iv_dim      = dimensionless
            iv_language = sy-langu
        ).
        out->write( lt_standard_dim_content ).
      CATCH cx_uom_error INTO DATA(error).
        out->write( error->get_text( ) ).
        out->write( |Error occurred while fetching standard content| ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.3.2 Resetting Content to Standard Units of Measurement

You can reset content to standard units of measurement either with or without a transport request.

Reset Without Transport Request

Use method `RESET_TO_STD_ LOCALLY` to reset the units, dimensions, and ISO codes to the standard units of measurement delivered by SAP. The changes are saved locally.

> ⓘ Note
>
> Class exception `CX_UOM_ERROR` is raised if the reset fails.

> ⚠ Caution
>
> Make sure you read and understand the risks and warnings described in Retrieving and Resetting Standard Units of Measurement [page 1636].

> {·} Sample Code

```
CLASS zcl_reset_to_std_locally DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC.

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
```

```
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_reset_to_std_locally IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    DATA(lo_uom_reset_tool) = cl_uom_content_handler=>get_instance( ).
        TRY.
           lo_uom_reset_tool->reset_to_std_locally( ).
      CATCH cx_uom_error INTO DATA(error).
        out->write( error->get_text( ) ).
        out->write( |Error occurred while resetting to standard content| ).
    ENDTRY.

  ENDMETHOD.
ENDCLASS.
```

Reset Using a Transport Request

Use method `RESET_TO_STD_USING_TRANSPORT` to reset units, dimensions, and ISO codes to the standard units of measurement delivered by SAP. The changes are transported.

Parameter Name	Value Help
`IV_TRANSPORT`	Transport Request

Note

Class exception `CX_UOM_ERROR` is raised if the reset fails.

Caution

All changes and standard units, dimensions, and ISO codes will be included in the customizing transport. Make sure you read and understand the risks and warnings described in Retrieving and Resetting Standard Units of Measurement [page 1636].

Sample Code

```
CLASS zcl_reset_to_std_transport DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC.

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_reset_to_std_transport IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    DATA(lo_uom_reset_tool) = cl_uom_content_handler=>get_instance( ).
        TRY.
           lo_uom_reset_tool->reset_to_std_using_transport( iv_transport =
'transport_request' ).
```

```
      CATCH cx_uom_error INTO DATA(error).
        out->write( error->get_text( ) ).
        out->write( |Error occurred while resetting to standard content| ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.4 Conversion Functions for Units of Measurement

Class `CL_UOM_CONVERSION` provides methods for simple conversion functions for units of measurement.

For more information about the methods on class API level, see the following sections in this documentation.

For information about the built-in function `UNIT_CONVERSION` on CDS view level, see Handling of Amounts and Quantities in the ABAP Data Models Guide, and CDS DDL - CDS View Entity, Unit and Currency Conversion Functions in the ABAP Keyword Documentation.

4.2.15.14.4.1 Simple Conversion Between Two Units

Use method `UNIT_CONVERSION_SIMPLE` to convert values from one measurement unit to another and round the result to the number of decimal places maintained in the measurement unit table `T006`, if necessary. Depending on the parameter `ROUND_SIGN`, the rounding is up (`'+'`), down (`'-'`), commercial (`'X'`), or no rounding (SPACE).

> ⓘ Note
>
> Make sure that both units are maintained in the measurement unit table and are not dimensionless, but have both the **same** dimension.

Import Parameters

Parameter Name	Value Help
INPUT	Input value
NO_TYPE_CHECK	Conversion factor type check `'X'`: No check **Space**: Type check

Parameter Name	Value Help
ROUND_SIGN	Rounding flag **'+'**: Up **'-'**: Down **'X'**: Commercial
UNIT_IN	Unit of input value

Export Parameters

Parameter Name	Value Help
ADD_CONST	Additive constant for conversion
DECIMALS	Number of decimal places for rounding
DENOMINATOR	Denominator for conversion
NUMERATOR	Numerator for conversion
OUTPUT	Output value

Exceptions

Exception Name	Value Help
CONVERSION_NOT_FOUND	Conversion factor could not be determined
DIVISION_BY_ZERO	Division by zero caught
INPUT_INVALID	Input value is not a number
OUTPUT_INVALID	Output parameter is not a number
OVERFLOW	Field overflow
TYPE_INVALID	Output parameter is not a number
UNITS_MISSING	No units specified
UNIT_IN_NOT_FOUND	UNIT_IN is not maintained
UNIT_OUT_NOT_FOUND	UNIT_OUT is not maintained

Sample Code

```
CLASS zcl_uom_conversion DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC.

  PUBLIC SECTION.
     INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_uom_conversion IMPLEMENTATION.

  METHOD if_oo_adt_classrun~main.
   DATA: lv_input  TYPE p DECIMALS 8 VALUE '1.98678923',
         lv_result TYPE p DECIMALS 8.
    DATA(lo_unit) = cl_uom_conversion=>create( ).

    lo_unit->unit_conversion_simple( EXPORTING  input                =
lv_input
                                                round_sign           = 'X'
                                                unit_in              = 'KG'
                                                unit_out             = 'G'
                                     IMPORTING  output               =
lv_result
                                     EXCEPTIONS conversion_not_found = 01
                                                division_by_zero     = 02
                                                input_invalid        = 03
                                                output_invalid       = 04
                                                overflow             = 05
                                                units_missing        = 06
                                                unit_in_not_found    = 07
                                                unit_out_not_found   = 08 ).
    IF sy-subrc = 0.
      out->write( lv_result ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

Related Information

For information about the built-in function `UNIT_CONVERSION` on CDS view level, see Handling of Amounts and Quantities in the ABAP Data Models Guide, and CDS DDL - CDS View Entity, Unit and Currency Conversion Functions in the ABAP Keyword Documentation.

4.2.15.14.4.2 Determining the SI Unit

Use method `SI_UNIT_GET` to determine the SI unit.

Import Parameters

Parameter Name	Value Help
DIMENSION	Dimension key
UNIT	Unit of measurement

Export Parameters

Parameter Name	Value Help
SI_UNIT	Base unit/SI unit of a dimension

Exceptions

Exception Name	Value Help
DIMENSION_NOT_FOUND	Dimension is not defined
UNIT_NOT_FOUND	Unit of measurement is not maintained
SI_UNIT_NOT_FOUND	SI unit is not defined

Sample Code

```
CLASS zcl_uom_conversion DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC.
  PUBLIC SECTION.
     INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_uom_conversion IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
   DATA(lo_unit) = cl_uom_conversion=>create( ).
    lo_unit->si_unit_get(
      EXPORTING
        dimension           = 'MASS'
```

```
        unit                = 'G'
      IMPORTING
        si_unit             = DATA(lv_si_unit)
      EXCEPTIONS
        dimension_not_found = 1
        unit_not_found      = 2
        si_unit_not_found   = 3
        OTHERS              = 4 ).

    IF sy-subrc = 0.
      out->write( lv_si_unit ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.4.3 Determining the Conversion Factors

Use method `UNIT_PARAMETERS_GET` to determine the data conversion factors of a unit.

Import Parameters

Parameter Name	Value Help
UNIT	Unit of measurement

Export Parameters

Parameter Name	Value Help
DECIMALS	Number of decimal places for rounding
DIMENSION	Dimension key
NUMERATOR	Numerator for conversion to SI unit
DENOMINATOR	Denominator for conversion to SI unit
EXPONENT	Base ten exponent for conversion to SI unit
ADD_CONST	Additive constant for conversion to SI unit
DECAN	Number of decimal places for number display
FAMUNIT	Unit of measurement family

Exceptions

Exception Name	Value Help
UNIT_NOT_FOUND	Unit of measurement is not maintained

Sample Code

```
CLASS zcl_uom_conversion DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC.

  PUBLIC SECTION.
     INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_uom_conversion IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
     DATA(lo_unit) = cl_uom_conversion=>create( ).
    lo_unit->unit_parameters_get(
      EXPORTING
        unit           = 'G'
      IMPORTING
        decimals       = DATA(lv_decimal)
        dimension      = DATA(lv_dimension)
        numerator      = DATA(lv_numerator)
        denominator    = DATA(lv_denominator)
      EXCEPTIONS
        unit_not_found = 1
        OTHERS         = 2 ).

    IF sy-subrc = 0.
      out->write( lv_dimension ).
      out->write( lv_decimal ).
      out->write( lv_numerator ).
      out->write( lv_denominator ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.14.4.4 Converting Fraction to Fraction with a 5-Digit Numerator/Denominator

Use method `CONVERT_TO_FRACT5` to convert a fraction to a fraction with a maximum of 5-digit numerators/denominators.

If the numerator or denominator contain more than 5 digits, the result will be an approximation as close as possible.

Import Parameters

Parameter Name	Value Help
NOMIN	Nominator input
DENOMIN	Denominator input

Export Parameters

Parameter Name	Value Help
NOMOUT	Nominator output (5-digit max.)
DENOMOUT	Denominator output (5-digit max.)

Exceptions

Exception Name	Value Help
CONVERSION_OVERFLOW	Unit of measurement is not maintained.

Sample Code

```
CLASS zcl_uom_convert_to_fract5 DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC.
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.


CLASS zcl_convert_to_fract5 IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    DATA: lv_nomin   TYPE f VALUE 999998,
          lv_denomin TYPE f VALUE 999999.

    DATA(lo_unit) = cl_uom_conversion=>create( ).
    TRY.
        lo_unit->convert_to_fract5(
          EXPORTING
            nomin    = lv_nomin
            denomin  = lv_denomin
          IMPORTING
            nomout   = DATA(lv_nomout)
            denomout = DATA(lv_denomout) ).

      CATCH cx_uom_error INTO DATA(lo_error).
```

```
        out->write( |Exception raised| ).
        out->write( lo_error->get_text( ) ).
    ENDTRY.
    out->write( lv_nomout ).
    out->write( lv_denomout ).
  ENDMETHOD.
ENDCLASS.
```

4.2.15.15 Identity and Access Management

Get an overview of the ABAP classes and methods you can use to access business roles and business users.

The following classes and methods allow you to read, create and modify business roles and business users.

These classes and methods can only be used if the authorizations are defined via standard IAM catalogs.

Business Roles

Class/Method	Description
CL_IAM_BUSINESS_ROLE_FACTORY	Factory class for IF_IAM_BUSINESS_ROLE_FACTORY
IF_IAM_BUSINESS_ROLE_FACTORY	Allows you to query and retrieve business role instances
IF_IAM_BUSINESS_ROLE	Allows you to read and modify attributes of business roles

Query for Business Roles

Sample Code

```
* Get instance
    data(lo_br_factory) = cl_iam_business_role_factory=>create_instance( ).
* Query for Business Roles
    lo_br_factory->query_business_roles(
      exporting
        ir_brole_id = value #( ( sign = 'I'
 option = 'CP'
 low = 'SAP_BR_QUALITY_ENGINEER_DWH' ) )
      importing
        et_result   = data(lt_result) ).
* get details of the Business Roles
    loop at lt_result assigning field-symbol(<fs_result>).
* Get Business Role instance
      lo_br_factory->get_business_role(
        exporting
          iv_uuid          = <fs_result>-uuid
        importing
          eo_business_role = data(lo_brole)
          et_return        = data(lt_return) ).
* Business Role ID
      data(lv_id) = lo_brole->get_id( ).
* Business Role description
      data(lv_description) = lo_brole->get_description( ).
      out->write( |{ lv_id }: { lv_description }| ).
* Get Business Role Access restrictions
      lo_brole->get_access_restriction(
        importing
```

```
          ev_write = data(lv_w)
          ev_read  = data(lv_r)
          ev_f4    = data(lv_f4) ).
      out->write( |Access-Restrictions: Write: { lv_w },
                   Read: { lv_r }, F4: { lv_f4 }| ).
* Get assigned Business Catalogs
      data(lt_bc) = lo_brole->get_business_catalogs( ).
* Get assigned Business Users
      data(lo_user_factory) =
cl_iam_business_user_factory=>create_instance( ).
      data(lt_users) = lo_brole->get_users( ).
      loop at lt_users assigning field-symbol(<fs_user>).
        out->write( <fs_user> ).
        lo_user_factory->get_business_user(
          exporting
            iv_user_id       = <fs_user>
```

Create Business Role from Business Role Template

Sample Code

```
    data: lv_id type if_iam_business_role=>ty_id.
    data(lo_br_factory) = cl_iam_business_role_factory=>create_instance( ).
* Create Business From Business Role Template
    lv_id = 'ZZ_MY_BR_ADMINISTRATOR'.
    lo_br_factory->create_brole_from_template(
      exporting
        iv_id            = lv_id
        iv_template_id   = 'SAP_BR_ADMINISTRATOR'
      importing
        eo_business_role = data(lo_brole)
        et_return        = data(lt_return) ).
* Set Business Role: Read = unrestricted, Write = Unrestricted, F4 =
unrestricted
    lo_brole->set_access_restriction(
      exporting
        iv_write  = if_iam_business_role=>co_access_restriction_code-
unrestricted
        iv_read   = if_iam_business_role=>co_access_restriction_code-
unrestricted
        iv_f4     = if_iam_business_role=>co_access_restriction_code-
unrestricted
      importing
        et_return = lt_return  ).
* Assign Business User to Business Role
    lo_brole->add_user(
      exporting
        iv_user_id = 'CB0000000051'
      importing
        et_return  = lt_return ).
    lo_brole->save(
      importing
        et_return = lt_return ).
```

Create Restricted Business Role

Sample Code

```
    data: ls_restriction type if_iam_business_role=>ty_restriction,
          lv_id          type if_iam_business_role=>ty_id.
    data(lo_br_factory) = cl_iam_business_role_factory=>create_instance( ).
* Create Business from Business Role Template
    lv_id = 'ZZ_MY_BUSINESS_ROLE'.
    lo_br_factory->create_business_role(
      exporting
        iv_id            = lv_id
      importing
        eo_business_role = data(lo_brole)
        et_return        = data(lt_return) ).
    lo_brole->set_description( 'My Business Role' ).
    lo_brole->set_long_text( 'This Business Role grant access to ....' ).
* Add Business Catalog
    lo_brole->add_business_catalog(
      exporting
        iv_bu_catalog_id = 'ZZ_MY_BUSINESS_CATALOG'
      importing
        et_return        = lt_return ).
* Set Business Role: Read = Unrestricted, Write = Restricted, F4 =
Unrestricted
    lo_brole->set_access_restriction(
      exporting
        iv_write  = if_iam_business_role=>co_access_restriction_code-
restricted
        iv_read   = if_iam_business_role=>co_access_restriction_code-
unrestricted
        iv_f4     = if_iam_business_role=>co_access_restriction_code-
unrestricted
      importing
        et_return = lt_return  ).
* Define restriction values for Write access
    ls_restriction-access_category_code =
if_iam_business_role=>co_access_category_code-write.
    ls_restriction-restriction_type = 'ZZ_SALES_AREA'.
    ls_restriction-restriction_field_value = value #(
        ( field_name = 'ZSALES_ORG'
          values = value #( ( low_value = 'A' )
                            ( low_value = 'B' ) ) )
        ( field_name = 'ZDIST_CHANNEL'
          values = value #( ( low_value = 'A*' high_value = 'F*' ) ) )
        ( field_name = 'ZDIVISION'
          values = value #( ( low_value = '*' ) ) ) ).
    lo_brole->create_restriction(
      exporting
        is_restriction = ls_restriction
      importing
        es_restriction = data(ls_restriction_set)
        et_return      = lt_return ).
* Assign Business User to Business Role
    lo_brole->add_user(
      exporting
        iv_user_id = 'CB0000000051'
      importing
        et_return  = lt_return ).
    lo_brole->save(
      importing
        et_return = lt_return ).
```

Business Users

Class/Method	Description
CL_IAM_BUSINESS_USER_FACTORY	Factory class for IF_IAM_BUSINESS_USER_FACTORY
IF_IAM_BUSINESS_USER_FACTORY	Allows you to query and retrieve business user instances
IF_IAM_BUSINESS_USER	Allows you to read and modify attributes of business users

Query for Business Users and Their Assigned Business Roles

Sample Code

```
* Get Business user by UserID
    data(lo_bu_factory) = cl_iam_business_user_factory=>create_instance( ).
    lo_bu_factory->get_business_user(
      exporting
        iv_user_id       = 'CB0000000051'
      importing
        eo_business_user = data(lo_buser)
        et_return        = data(lt_return) ).
* Get User properties
    data(lv_user) = lo_buser->get_user_name( ).
* get assigned Business Roles
    data(lo_br_factory) = cl_iam_business_role_factory=>create_instance( ).
    data(lt_brole_id) = lo_buser->get_business_roles( ).
    loop at lt_brole_id assigning field-symbol(<fs_brole_id>).
      lo_br_factory->get_business_role_by_id(
        exporting
          iv_id            = <fs_brole_id>
        importing
          eo_business_role = data(lo_brole)
          et_return        = lt_return ).
       out->write( |ID: { lo_brole->get_id(  ) }| ).
    endloop.
```

Create Business User and Add Business Role

Sample Code

```
    data(lo_bu_factory) = cl_iam_business_user_factory=>create_instance( ).
* Create Business user for Employee ID
    lo_bu_factory->create_business_user(
      exporting
        iv_bupa_id       = '0000000023'
        iv_user_name     = 'ZMY_BUSINESS_USER'
      importing
        eo_business_user = data(lo_buser)
        et_return        = data(lt_return) ).
* Set Date Format to MM-DD-YYYY (Gregorian Date)
    lo_buser->set_date_format( '3' ).
* Set logon language to English
    lo_buser->set_logon_language( 'E' ).
* Assign Business Role to the User
    lo_buser->add_business_roles(
```

```
      exporting
        it_business_role_id = value #( ( 'ZZ_MY_BUSINESS_ROLE' ) )
      importing
        et_return           = lt_return ).
* Save Business user
    lo_buser->save(
      importing
        et_return = lt_return ).
```

Lock Business User

Sample Code

```
* get business user user name
    data(lo_bu_factory) = cl_iam_business_user_factory=>create_instance( ).
    lo_bu_factory->get_business_user(
      exporting
        iv_user_name     = 'ZMY_BUSINESS_USER'
      importing
        eo_business_user = data(lo_buser)
        et_return        = data(lt_return) ).
* Get lock status
    lo_buser->get_lock_status(
      importing
        ev_locked = data(lv_locked) ).
* Lock user
    if lv_locked = abap_false.
      lo_buser->lock(
        importing
          et_return = lt_return ).
    endif.
```

Authority Checks for Business Users

Class/Method	Description
CL_AUTH_FACTORY	The entry point for using the API functions. It can be used as a factory class for the instantiation of the class for the authority checks.
GET_AUTH_CHECK_INSTANCE	This method provides an instance of the class cl_auth_check_for_bu for a specific business partner. The required business partner GUID needs to be provided in the method call as current parameter. This instance can then be used for an authority check for the related business user.
CL_AUTH_CHECK_FOR_BU	This class provides the option of performing an authority check on another business user for whom the class was instantiated.

Class/Method	Description
CHECK	This method performs an authority check against the business user represented by the business partner GUID which was provided during instance generation.

For more information, see the Knowledge Transfer Document in the package *SUSR_BU*.

4.2.15.16 Communication Management

Get an overview of the ABAP classes and methods you can use to access communication scenarios, communication systems, communication arrangements and communication users.

The following classes and methods allow you to read communication scenarios, communication systems, communication arrangements and communication users.

These classes and methods can only be used if the authorizations are defined via standard COM catalogs.

Communication Scenarios

Class/Method	Description
CL_COM_SCENARIO_FACTORY	Factory class for IF_COM_SCENARIO_FACTORY
IF_COM_SCENARIO_FACTORY	Allows you to query and retrieve communication scenarios
IF_COM_SCENARIO	Allows you to read attributes of communication scenarios

Communication Users

Class/Method	Description
CL_COM_USER_FACTORY	Factory class for IF_COM_USER_FACTORY
IF_COM_USER_FACTORY	Allows you to query and retrieve communication users
IF_COM_USER	Allows you to read attributes of communication users

Communication Systems

Class/Method	Description
CL_COM_SYSTEM_FACTORY	Factory class for IF_COM_SYSTEM_FACTORY
IF_COM_SYSTEM_FACTORY	Allows you to query and retrieve communication systems
IF_COM_SYSTEM	Allows you to read attributes of communication systems

Communication Arrangements

Class/Method	Description
CL_COM_ARRANGEMENT_FACTORY	Factory class for IF_COM_ARRANGEMENT_FACTORY
IF_COM_ARRANGEMENT_FACTORY	Allows you to query and retrieve communication arrangements
IF_COM_ ARRANGEMENT	Allows you to read attributes of communication arrangements

4.2.15.17 Proxy API for SAP Build Process Automation

With the proxy API, you can start workflows of the workflow capability within SAP Build Process Automation out of your ABAP Environment.

The overall starting point is the class CL_SWF_CPWF_API_FACTORY_A4C. Once you have the API object of interface IF_SWF_CPWF_API, see the ABAP documentation on supported actions.

Prerequisites

You've executed the integration steps in your ABAP environment. See Integrating SAP Build Process Automation [page 3365].

4.2.15.17.1 Development in Classic ABAP

The starting point is the class `CL_SWF_CPWF_API_FACTORY_A4C`. Once you have the API object of interface `IF_SWF_CPWF_API`, see the ABAP documentation on supported actions.

Starting SAP Build Process Automation process instances

Instances can be created with the following snippet:

Sample Code

Start a SAP Cloud Platform Workflow

```
TYPES: BEGIN OF ty_context,
         some_property TYPE string,
       END OF ty_context.

DATA(lo_cpwf_api) = cl_swf_cpwf_api_factory_a4c...=>get_api_instance( ... ).

DATA(ls_context) = VALUE ty_context(
  some_property = 'someValue'
).
DATA(lv_context_json) = lo_cpwf_api->get_start_context_from_data(
  iv_data = ls_context
).

DATA(lv_cpwf_handle) = lo_cpwf_api->start_workflow(
  iv_cp_workflow_def_id = '<SAP Workflow Service Process Definition
Identifier>'
  iv_context            = lv_context_json
  iv_retention_time     = 30
  iv_callback_class     = '<Callback class for final state handling>'
).
```

Registering SAP Build Process Automation process instances

If there is a running process instance in SAP Build Process automation which you would like to handle from within your S/4 Hana Application, you can register the instance in the following way:

Sample Code

Register a SAP Cloud Platform Workflow

```
" Get a Instance for the CPWF Integration API
DATA(lo_cpwf_api) = cl_swf_cpwf_api_factory_a4c...=>get_api_instance( ... ).

DATA(lv_cpwf_handle) = lo_cpwf_api->register_workflow(
    iv_cp_workflow_inst_id = '<SAP Workflow Service Instance Identifier>'
    iv_retention_time      = 30
    iv_callback_class      = '<Callback class for final state handling>'
).

COMMIT WORK.
```

Handling the result of SAP Build Process Automation process instances

As shown in the above snippets, a callback class can be provided that allows processing the result of the process. For example, it is possible to read the context of the completed instance.

Sample Code

Completion callback with reading context

```
METHOD if_swf_cpwf_callback~workflow_instance_completed.
  TYPES: BEGIN OF ty_context,
           some_result TYPE string,
         END OF ty_context.

  TRY.
      DATA(lo_cpwf_api) =
cl_swf_cpwf_api_factory_a4c...=>get_api_instance( ... ).
      data(lv_context_json) = lo_cpwf_api-
>get_workflow_context( iv_cpwf_handle = iv_cpwf_handle ).

      DATA ls_context TYPE ty_context.

      lo_cpwf_api->get_context_data_from_json(
        EXPORTING
          iv_context = lv_context_json
       IMPORTING
          ev_data = ls_context
      ).
    CATCH cx_swf_cpwf_api INTO DATA(lx_exc).
      " Exception handling
  ENDTRY.

  " your business coding
ENDMETHOD.
```

Raising an intermediate message event for a SAP Build Process Automation process instance

If SAP Build Process Automationis waiting for an intermediate message event (IME), you can trigger that message by calling the corresponding API, using the following code.

Sample Code

Raise an intermediate message event

```
CONSTANTS: lc_cp_workflow_def_id TYPE if_swf_cpwf_api=>cpwf_def_id      VALUE
'<Your Workflow Definition ID>',
             lc_api_trigger_uid    TYPE if_swf_cpwf_api=>cpwf_trigger_uid VALUE
'<Your API Trigger UID>'.
" Get a Instance for the CPWF Integration API
   DATA(lo_cpwf_api) = cl_swf_cpwf_api_factory...=>get_api_instance( ... ).
" actually raise the event
   lo_cpwf_api->api_trigger(
     EXPORTING
       iv_cpwf_handle   = lv_cpwf_handle  " the BTP process instance handle
       iv_event_def_id  = lc_api_trigger_uid
   ).

   COMMIT WORK.
```

In older releases the API trigger may not yet be supported by the API. Here intermediate message event can be raised through the following code.

Sample Code

Raise an intermediate message event (deprecated)

```
TYPES: BEGIN OF ty_example_context,
         value TYPE int4,
       END OF ty_example_context.

" Get a Instance for the CPWF Integration API
DATA(lo_cpwf_api) = cl_swf_cpwf_api_factory_a4c...=>get_api_instance( ... ).

" provide relevant event context data => in this case a meaningful integer
DATA(lv_event_context_data) = VALUE ty_example_context( value = 4711 ).
DATA(lv_event_context) = lo_cpwf_api->get_context_from_data( iv_data =
lv_event_context_data ).

" actually raise the event
lo_cpwf_api->raise_event(
  EXPORTING
    iv_cpwf_handle   = lv_cpwf_handle
    iv_event_def_id  = '<Event Definition Identifier>'
    iv_event_context = lv_event_context
).

COMMIT WORK.
```

Note

This approach works only for "old" workflow management process definitions being deployed from SAP Business Application Studio.

Retrieving the current state of SAP Build Process Automation process instances

It is also possible to synchronously retrieve data about the processes. It can be accessed through the interface `IF_SWF_CPWF_API`, which extends the core API IF_SWF_CPWF_CAPI.

For example, you can query the current lifecycle status of processes.

Sample Code

Read Workflow Instances

```
" Get a Instance for the CPWF Integration API
DATA(lo_cpwf_api) = cl_swf_cpwf_api_factory...=>get_api_instance( ... ).
DATA(lo_wf_inst) = lo_cpwf_api-
>if_swf_cpwf_capi~get_workflow_instances_api( ).

DATA(lt_workflow_instance) = lo_wf_inst->query_instances(
  iv_definition_id = '<SAP Workflow Service Process Definition Identifier>'
  iv_status = if_swf_cpwf_capi_wf_insts=>cs_status-running
).
```

Or retrieve the current context:

Sample Code

Read Workflow Instance Context

```
TYPES: BEGIN OF ty_context,
         attr1 TYPE string,
         attr2 TYPE int4,
         attr3 TYPE string,
       END OF ty_context.

" Get a Instance for the CPWF Integration API
DATA(lo_cpwf_api) = cl_swf_cpwf_api_factory...=>get_api_instance( ... ).
DATA(lo_cp_json) = lo_cpwf_api->get_json_converter( ).
DATA(lo_wf_inst_ctxt) = lo_cpwf_api-
>if_swf_cpwf_capi~get_wf_instance_context_api( ).

DATA ls_context TYPE ty_context.
lo_wf_inst_ctxt->get_instance_context( EXPORTING iv_cpwf_handle =
<cpwf_handle> io_cp_json = lo_cp_json IMPORTING data = ls_context ).
```

As shown in the code there is also a JSON converter available to simplify the processing of the context.

Restricting the retrieved process context

To retrieve only the relevant parts of the process context, the APIs `IF_SWF_CPWF_API=> GET_WORKFLOW_CONTEXT` and `IF_SWF_CPWF_CAPI_WF_INST_CTXT=> GET_INSTANCE_CONTEXT` now support a new parameter `IV_CONTEXT_PATH`. The parameter accepts any JSON path expression. This JSON path expression is forwarded to the workflow API of SAP Build Process Automation. The SAP Build Process Automation Workflow Service API uses the expression to returns then only the specific section of the context instead of the entire data.

The following examples show how the `IV_CONTEXT_PATH` parameter can be used.

Sample Code

Sample Context

```
{
    "decision_travelRequestApproved_2": {
        "outcome": "APPROVED"
    },
    "startEvent": {
        "travelid": "1",
        "createdby": "UserX",
        "createdbymail": "userx@sap.com",
        "enddate": "2025-02-28",
        "startdate": "2025-02-01",
        "destination": "Far Far Away"
    },
    "action_post_ProcessCompleted_2": {
        "result": {
            "notificationProcessed": true
        }
    },
    "custom": {
        "approver": "userx@sap.com",
        "outcome": "APPROVED",
        "destinations": [
            {
                "date": "2025-02-01",
                "location": "here"
            },
            {
                "date": "2025-02-12",
                "location": "there"
             }
        ]
    },
    "decision_determineRecipient_1": {
        "Approver": "userx@sap.com"
    },
    "form_approveTravelRequest_1": {
        "travelID": 1,
        "requestor": "UserX",
        "description_1": "Far Far Away",
        "startDate": "2025-02-01",
        "endDate": "2025-02-28"
    }
}
```

Sample Code

Get Context with JSONPath "custom"

```
{
    "approver": "userx@sap.com",
    "outcome": "APPROVED"
    "destinations": [
        {
            "date": "2025-02-01",
            "location": "here"
        },
        {
            "date": "2025-02-12",
            "location": "there"
        }
    ]
```

```
}
```

Sample Code

Get Context with JSONPath "custom.outcome"

```
"APPROVED"
```

Sample Code

Get Context with JSONPath "custom.outcome.destinations[1]"

```
{
    "date": "2025-02-12",
    "location": "there"
}
```

Sample Code

Get Context with JSONPath "custom.outcome.destinations[1].location"

```
"here"
```

4.2.15.17.2 Development in ABAP RESTful Application Programming Model (RAP)

For development in the ABAP RESTful Application Programming Model (RAP) follow the below guidance.

Early Numbering

In the RAP version of the workflow service, API workflows can be registered via the command.

Sample Code

Example: EML Create

```
MODIFY ENTITIES OF i_cpwf_inst
      ENTITY CPWFInstance
      EXECUTE registerWorkflow
      FROM VALUE #( ( %key = '<a UUID which is mandatory for mass enabled
callers>'
                      %param-RetentionTime = '<retention time in days>'
                      %param-CpWfDefId = '<SAP Workflow Service Definition
Id>'
                      %param-CallbackClass = '<Callback class for final state
handling>'
                      %param-Consumer = '<SAP Workflow Scenario Id>') ).
```

Note

It's not allowed to call the `registerWorkflow` method multiple times with an identical set of parameters. This also applies to cases where the otherwise optional %key parameter stays empty. Therefore, if you

want to use the `registerWorkflow` method in a mass-enabled context you must provide the unique %key parameter.

The parameters `RetentionTime`, `CpWfDefId`, and `Consumer` must be provided

The workflow context can be provided using the following EML statement:

Sample Code

Example: EML Create

```
MODIFY ENTITIES OF i_cpwf_inst
      ENTITY CPWFInstance
      EXECUTE setPayload
      FROM VALUE #( ( %key-CpWfHandle = '<required key (UUID) of the CPWF
instance>'
                      %param-context = '<workflow context in JSON
format>' ) ).
```

Both commands should be part of a determination on save. During save sequence the workflow is started with the provided data.

Late Numbering

As for early numbering, the `registerWorkflow` method can be called in a determination on save for late numbering, too.

The workflow context can be set even later in the `adjust_numbers` method of your business object using the `setPayload` method.

Sample Code

Example: EML Create

```
MODIFY ENTITIES OF i_cpwf_inst
      ENTITY CPWFInstance
      EXECUTE setPayload
      FROM VALUE #( ( %key-CpWfHandle = <required key (UUID) of the CPWF
instance>
                      %param-context = <workflow context in JSON format> ) ).
```

During the save sequence, the workflow is started with the provided data. The instance is removed from the draft table and is entered into the active table.

Note, that in the `adjust_numbers` method you must fill the key of each line of the mapped table to make sure the save sequence is executed successfully.

Note also that if you have multiple workflows per instance of your business object you might need to store the assignments yourself in an internal table to be able to call the `set_payload` method during `adjust_numbers` method with the correct data.

Sample Code

This coding sample shows reading the workflow instance context.

```
TYPES: BEGIN OF ty_context,
         attr1 TYPE string,
         attr2 TYPE int4,
         attr3 TYPE string,
```

```
       END OF ty_context.

DATA(lo_cpwf_api) = cl_swf_cpwf_api_factory=>get_api_instance( 'DEFAULT' ).
DATA(lo_cp_json_with_empty) = lo_cpwf_api->get_json_converter( ).
DATA(lo_wf_inst_ctxt) = lo_cpwf_api-
>if_swf_cpwf_capi~workflow_instance_context( ).

DATA ls_context TYPE ty_context.
lo_wf_inst_ctxt->get_instance_context( EXPORTING iv_cpwf_handle =
<cpwf_handle> io_cp_json = lo_cp_json IMPORTING data = ls_context ).
```

Cancel Workflow Capability Instances

To register a running workflow capability instance for cancelation, use the `registerWorkflowCancel` action using the command:

Sample Code

Example: EML Cancel

```
MODIFY ENTITIES OF i_cpwf_inst
      ENTITY CPWFInstance
      EXECUTE registerWorkflowCancel
      FROM VALUE #( ( %key = '<a UUID which is mandatory for mass enabled
callers>' ) ).
```

Note, that this action should be part of the interaction phase.

During the save sequence, the workflow instance is canceled in a background unit.

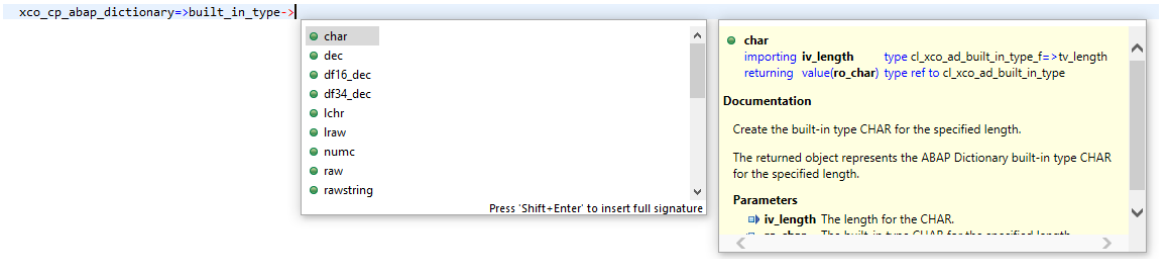
4.2.15.18 XCO Library

The XCO (“Extension Components”) library is a general-purpose development library for ABAP aimed at providing a highly efficient ABAP development experience. The Cloud Platform (CP) edition of the XCO library is specifically designed to support ABAP development scenarios in the new SAP Business AI Platform, ABAP environment.

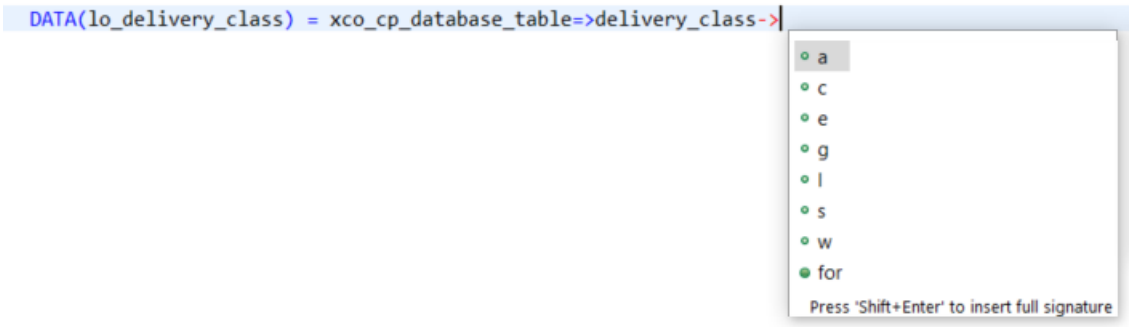
4.2.15.18.1 Core Principles

The design of the XCO library is based on the following core principles:

- **Modules**
  The XCO library is organized as a collection of (largely) independent modules (corresponding to ABAP packages). The public API of each module is exposed via a special class, called the API “class” of the module. It is distinguished from all other XCO classes and interfaces in that it starts with “XCO_CP_”. The API class acts as the single point of entry for the functionality offered by a given module.
  Starting with the API class, code completion can be used to easily discover the scope of a given module.



- **Enumerated Values**
  In the XCO library enumerated values are represented by pre-created objects which can be accessed via the API class of the module they belong to. Besides offering a strongly typed way to work with fixed values they also provide additional properties specific to the enumeration as well as access to the underlying primitive value.



- **Exception Handling**
  All XCO modules follow a uniform error propagation strategy. Unless the caller of an API can be expected to recover from an error situation (e.g. when attempting to acquire an exclusive lock), all error situations are handled through no_check exceptions.
  Any runtime exception that is raised explicitly by the XCO library is a subclass of CX_XCO_RUNTIME_EXCEPTION (cf. section Exception philosophy in Standard Library [page 1796]).

4.2.15.18.2 Overview of XCO Modules

The Cloud Platform (CP) edition of the XCO library is comprised of the following modules:

- ABAP (XCO_CP_ABAP)
  Provides access to conceptual abstractions for classes and interfaces as well as to representations of ABAP built-in and generic types.
- Application log object (XCO_CP_APPLICATION_LOG_OBJECT)
  Provides access to text attributes specific to application log objects.
- ABAP Dictionary (XCO_CP_ABAP_DICTIONARY)
  Provides access to conceptual abstractions for ABAP Dictionary elements (Database tables, data elements, structures, table types and domains) as well as to representations of ABAP Dictionary built-in types and their corresponding reference types.
- ABAP Objects (XCO_CP_ABAP_OBJECTS)
  Provides access to enumerations specific to ABAP Objects, e.g. the visibility of class or interface components, as well as to supported origins for the XCO Read APIs for classes and interfaces.

- ABAP Repository (XCO_CP_ABAP_REPOSITORY)
  Provides APIs and abstractions for retrieving and filtering development objects of the ABAP Repository in a strongly typed manner.
- ABAP SQL (XCO_CP_ABAP_SQL)
  Provides ways to build ABAP SQL constraints to be used in conjunction with the filtering offerings of the XCO ABAP Repository module.
- ABAP Unit (XCO_CP_ABAP_UNIT)
  Provides access to enumerations in the context of ABAP Unit.
- AMDP (XCO_CP_AMDP)
  Provides access to enumerations in the context of ABAP-managed database procedures.
- ARS (XCO_CP_ARS)
  Provides standard abstractions in the context of the API Release (ARS) framework (e.g. for compatibility contracts) to be used when programmatically setting or getting API states via the XCO ABAP Repository APIs.
- Business Application Log (XCO_CP_BAL)
  Provides APIs for creating, deleting and searching logs as well as standard abstractions for integrating logging functionality into custom application logic.
- Behavior definition (XCO_CP_BEHAVIOR_DEFINITION)
  Provides access to enumerations specific to behavior definitions.
- Behavior implementation (XCO_CP_BEHAVIOR_IMPLEMENTATION)
  Provides access to enumerations specific to behavior implementations.
- Binary (XCO_CP_BINARY)
  Provides access to binary text encodings, e.g. Base64.
- Business configuration object (XCO_CP_BUSINESS_CNFGRTN_OBJECT)
  Provides access to text attributes specific to business configuration objects.
- Call stack (XCO_CP_CALL_STACK)
  Provides access to abstractions for programmatically working with all stacks (e.g. supported formats)
- Core Data Services (XCO_CP_CDS)
  Provides access to enumerations specific to the field of Core Data Services (CDS) as well as conceptual abstractions for behavior definitions, data definitions, metadata extensions and CDS entities.
- CDS Annotation (XCO_CP_CDS_ANNOTATION)
  Provides ways to build CDS annotation values to be used when generating DDLS, DDLX or SRVD objects via the XCO Generation APIs.
- CDS Type Definition (XCO_CP_CDS_TYPE_DEFINITION
  Provides access to enumerations specific to CDS type definitions as well as to supported versions and origins as they can be used in the context of the XCO Read APIs for CDS type definitions.
- Character (XCO_CP_CHARACTER)
  Provides access to abstractions useful when working with character-like data, e.g. code pages which can be used to translate between STRINGs and XSTRINGs
- Correction and Transport System (XCO_CP_CTS)
  Provides abstractions for working with the Correction and Transport System (CTS), e.g. for creating and releasing Workbench transport requests.
- Database table (XCO_CP_DATABASE_TABLE)
  Provides access to database table specific enumerations, e.g. the size category of a database table, as well as to supported origins for the XCO Read APIs for database tables.
- Data definition (XCO_CP_DATA_DEFINITION)
  Provides access to enumerations and text attributes specific to CDS data definitions.

- Data element (XCO_CP_DATA_ELEMENT)
  Provides access to text attributes specific to data elements as well as to supported origins for the XCO Read APIs for data elements.
- Data control language (XCO_CP_DCL)
  Provides ways to build DCL (Data control language) expressions to be used when generating DCLS objects via the XCO Generation APIs
- Data definition language (XCO_CP_DDL)
  Provides ways to build DDL (Data definition language) expressions to be used when generating DDLS objects via the XCO Generation APIs.
- Domain (XCO_CP_DOMAIN)
  Provides access to text attributes specific to domains as well as to supported origins for the XCO Read APIs for domains.
- Event binding (XCO_CP_EVENT_BINDING)
  Provides access to supported origins and versions for the XCO Read APIs for event bindings.
- Function group (XCO_CP_FUNCTION_GROUP)
  Provides access to supported origins for the XCO Read APIs for function groups.
- Function module (XCO_CP_FUNCTION_MODULE)
  Provides access to function module-specific enumerations as well as to supported origins for the XCO Read APIs for function modules.
- Generation (XCO_CP_GENERATION)
  Provides access to the XCO Generation APIs, i.e. allows to obtain a generation environment which can be used to create POST, PUT, PATCH and DELETE operations (as supported by the respective object type).
- Hash (XCO_CP_HASH)
  Provides access to supported hash algorithms which can be used to calculate the hash value of strings with the XCO Standard Library.
- Internationalization (XCO_CP_I18N)
  Provides access to the XCO I18N APIs, i.e. allows to obtain domain, data element, data definition and message class targets which can be used to programmatically maintain language-dependent texts.
- Identify and Access Management (XCO_CP_IAM)
  Provides access to standard abstractions (e.g. for IAM business catalogs) within the area of Identify and Access Management.
- IAM Business Catalog
  Provides access to text attributes specific to IAM business catalogs.
- JSON (XCO_CP_JSON)
  Provides access to facilities used when working with JSON data in the context of the XCO standard library, such as the JSON builder or standard JSON transformations.
- Language (XCO_CP_LANGUAGE)
  Provides access to abstractions when working with languages, e.g. different formats.
- Message (XCO_CP_MESSAGE)
  Provides access to enumerations specific to messages, such as the message type.
- Message class (XCO_CP_MESSAGE_CLASS)
  Provides access to text attributes specific to message classes.
- Metadata extension (XCO_CP_METADATA_EXTENSION)
  Provides access to enumerations specific to metadata extensions.
- Name (XCO_CP_NAME)
  Provides access to IF_XCO_NAME_CHOICEs to be used in conjunction with POST operations in the context of the XCO Generation APIs.

- Package (XCO_CP_PACKAGE)
  Provides access to enumerations specific to packages.
- RESTful Application Programming (XCO_CP_RAP)
  Provides access to abstractions to integrate RAP specific types (e.g. RAP behavior messages) with standard abstractions (e.g. for business application logs) of the XCO Library.
- Regular expression (XCO_CP_REGULAR_EXPRESSION)
  Provides access to abstractions used when working with regular expressions in the context of the XCO standard library, such as different regular expression engines.
- SAP object node type (XCO_CP_SAP_OBJECT_NODE_TYPE)
  Provides access to enumerations specific to SAP object node types as well as to supported versions and origins as they can be used in the context of the XCO Read APIs for SAP object node types.
- SAP object type (XCO_CP_SAP_OBJECT_TYPE)
  Provides access to enumerations specific to SAP object types as well as to supported versions and origins as they can be used in the context of the XCO Read APIs for SAP object types.
- Service binding (XCO_CP_SERVICE_BINDING)
  Provides access to enumerations specific to service bindings as well as to supported versions and origins that can be used in the context of XCO Read APIs for service bindings.
- Service consumption model (XCO_CP_SERVICE_CONS_MODEL)
  Provides access to supported versions and origins that can be used in the context of XCO Read APIs for service comsumption models.
- Service definition (XCO_CP_SERVICE_DEFINITION)
  Provides access to supported versions and origins as they can be used in the context of the XCO Read APIs for service definitions.
- Software component (XCO_CP_SOFTWARE_COMPONENT)
  Provides access to enumerations specific to software components.
- String (XCO_CP_STRING)
  Provides access to abstractions used when working with strings in the context of the XCO standard library, such as the string builder or standard string compositions and decompositions.
- System (XCO_CP_SYSTEM)
  Provides access to abstractions for system-wide entities such as software components or application components..
- Table (XCO_CP_TABLE)
  Provides access to enumerations specific to tables (i.e. structures and database tables).
- Table type (XCO_CP_TABLE_TYPE)
  Provides access to enumerations specific to table types.
- Tenant (XCO_CP_TENANT)
  Provides access to enumerations specific to tenants.
- Time (XCO_CP_TIME)
  Provides access to the XCO time library.
- Transport (XCO_CP_TRANSPORT)
  Provides access to enumerations and abstractions specific to transports.
- UUID (XCO_CP_UUID)
  Provides access to abstractions used when working with UUIDs in the context of the XCO standard library, such as different UUID formats.
- XSLX (XCO_CP_XSLX, XCO_CP_XLSX_SELECTION, XCO_CP_XLSX_READ_ACCESS, XCO_CP_XLSX_WRITE_ACCESS
)

The XCO XLSX module supports the programmatic processing of XLSX documents. The various external APIs provide access to central abstractions required when reading data from XLSX documents.

4.2.15.18.3 ABAP Repository

4.2.15.18.3.1 Query APIs

The XCO ABAP Repository module provides Query APIs that allow to easily navigate through the ABAP Repository.

Context

To this extent, an object selection functionality is provided that is based on the following two central abstractions:

- Object source: An entity that can naturally act as a source of ABAP repository objects, e.g. a package or a transport or even the complete ABAP repository
- Object filter: An encapsulation for any kind of filter that can be applied to a selection of ABAP Repository objects. Examples are software or application components or type specific properties like foreign key attributes of database table fields

The entry point for retrieving a collection of objects from the ABAP Repository is XCO_CP_ABAP_REPOSITORY=>OBJECTS. Note that independently of any filters, the visible ABAP Repository objects are those which either have been explicitly released by SAP or which belong to customer software components. The overall usage is illustrated by two examples below.

In the first example, all repository objects of the whole ABAP Repository whose software component is ZLOCAL and whose name contains the fragment CAL are retrieved. As no explicit object type is specified the retrieved objects have the type IF_XCO_AR_OBJECT:

Sample Code

```
DATA(lo_software_component_filter) = xco_cp_system=>software_component-
>get_filter( xco_cp_abap_sql=>constraint->equal( 'ZLOCAL' ) ).
DATA(lo_name_filter) = xco_cp_abap_repository=>object_name-
>get_filter( xco_cp_abap_sql=>constraint->contains_pattern( '%CAL%' ) ).
DATA(lt_objects) = xco_cp_abap_repository=>objects->where( VALUE #(
  ( lo_software_component_filter )
  ( lo_name_filter )
) )->in( xco_cp_abap=>repository )->get( ).
LOOP AT lt_objects INTO DATA(lo_object).
  DATA(lv_object_type) = lo_object->type->value.
  DATA(lv_object_name) = lo_object->name->value.
  " ...
ENDLOOP.
```

In the second example all database tables of the package Z_MY_OBJECTS that have at least one field which has a foreign key whose check table is ZHOLIDAY are retrieved. As in this case the object type (database table) is explicitly specified the retrieved objects are of type IF_XCO_DATABASE_TABLE:

Sample Code

```
DATA(lo_package) = xco_cp_abap_repository=>package->for( 'Z_MY_OBJECTS' ).
DATA(lo_filter) = xco_cp_table=>field_property->foreign_key_check_table-
>get_filter(
  xco_cp_abap_sql=>constraint->equal( 'ZHOLIDAY' )
).

DATA(lt_database_tables) = xco_cp_abap_repository=>objects->tabl-
>database_tables->where( VALUE #(
  ( lo_filter )
) )->in( lo_package )->get( ).

LOOP AT lt_database_tables INTO DATA(lo_database_table).
  DATA(lv_name) = lo_database_table->name.

  " ...
ENDLOOP.
```

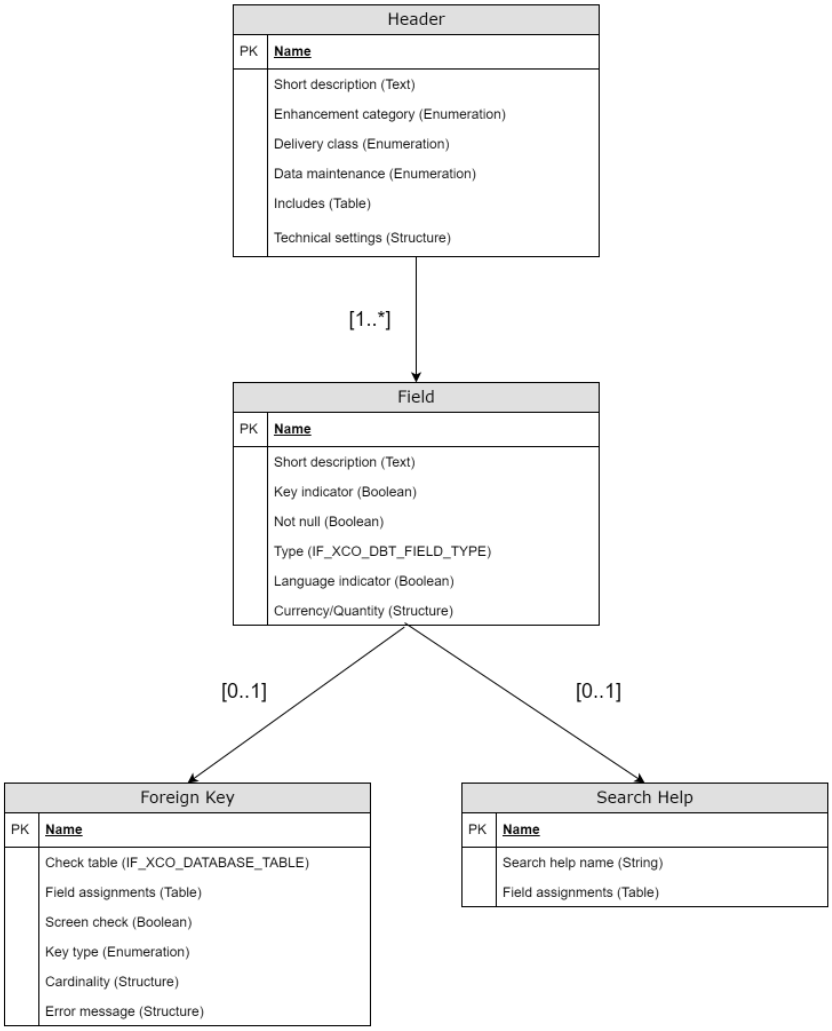
4.2.15.18.3.2 Read APIs

The XCO Read APIs provide homogenous, yet strongly-typed, APIs that allow to access the content of ABAP Repository objects in a structured way.

Hierarchical Object Model

The overall design of the XCO Read APIs (regardless of the concrete object type) is based on a natural hierarchical model for each respective object type. The hierarchical model used by the XCO Read APIs regards an object as a rooted tree, with the root node given by the header of the object and all other nodes

corresponding to parts of the object. To illustrate, the diagram below contains the hierarchical object model for a database table of the ABAP Dictionary as used by the XCO Read APIs:



For each node in the hierarchical object model for an object type, two abstractions are provided:

- Object/part handle
- Content handle

Object/Part Handles

The object/part handle provides an abstraction which encapsulates all key values (object name and keys of the parts if applicable) of an object/part. For the database table example above we have:

- IF_XCO_DATABASE_TABLE: The object handle for the database table (encapsulating the name of the database table)
- IF_XCO_DBT_FIELD: The handle for the field of a database table (encapsulating the name of the database table and the name of the field)
- IF_XCO_DBT_FOREIGN_KEY: The handle for the foreign key of a field of a database table (encapsulating the name of the database table and the name of the field for which to consider the foreign key)

- IF_XCO_DBT_SEARCH_HELP: The handle for the search help of a field of a database table (encapsulating the name of the database table and the name of the field for which to consider the search help)

The following code sample illustrates how these four handles can be obtained:

Sample Code

```
" Handle for database table ZMY_DBT (IF_XCO_DATABASE_TABLE).
DATA(lo_database_table) = xco_cp_abap_dictionary=>database_table( 'ZMY_DBT' ).

" Handle for the field KEY_FIELD of database table ZMY_DBT (IF_XCO_DBT_FIELD).
DATA(lo_field) = lo_database_table->field( 'KEY_FIELD' ).

" Handle for the foreign key of field KEY_FIELD of database table ZMY_DBT
(IF_XCO_DBT_FOREIGN_KEY).
DATA(lo_foreign_key) = lo_field->foreign_key.

" Handle for the search help of field KEY_FIELD of database table ZMY_DBT
(IF_XCO_DBT_SEARCH_HELP).
DATA(lo_search_help) = lo_field->search_help.
```

A given object/part handle allows to check the existence via the EXISTS method and provides access to the content handle of the respective object/part via the CONTENT method.

Content Handles

Content handles exist in a natural 1:1 relationship with object/part handles and can be obtained via the CONTENT method of the object/part handle. In case the object type supports different versions (e.g. active and inactive) a choice for the version needs to be made before obtaining a content handle. More generally, a content handle fixes all degrees of freedom except for the origin (see below) such that the attributes exposed by the content handle can be read out.

A given content handle provides a dedicated GET_* method for each exposed attribute as well as a method GET which will return all exposed attributes in a single structure. Note that for language-dependent attributes, e.g. short descriptions, the value will always be retrieved in the original language of the underlying ABAP Repository object.

For the database table example we have:

- IF_XCO_DBT_CONTENT: The content handle for the header of the database table
- IF_XCO_DBT_FIELD_CONTENT: The content handle for a field of a database table
- IF_XCO_DBT_FOREIGN_KEY_CONTENT: The content handle for the foreign key of a field of a database table
- IF_XCO_DBT_SEARCH_HELP_CONTENT: The content handle for the search help of a field of a database table

The code sample below illustrates how the content handle for a database table field is obtained and used to read out the attributes of the field:

Sample Code

```
" Content handle for the field KEY_FIELD of database table ZMY_DBT.
DATA(lo_field_content) = xco_cp_abap_dictionary=>database_table( 'ZMY_DBT'
  )->field( 'KEY_FIELD'
```

```
  )->content( ).

" Get the complete content (i.e. all the available attributes of the field)
" as a structure of all attributes.
DATA(ls_field_content) = lo_field_content->get( ).

" Get each attribute individually.
DATA(lv_short_description) = lo_field_content->get_short_description( ).
DATA(lv_key_indicator) = lo_field_content->get_key_indicator( ).
DATA(lv_not_null) = lo_field_content->get_not_null( ).
DATA(lo_type) = lo_field_content->get_type( ).
DATA(lv_language_indicator) = lo_field_content->get_language_indicator( ).
DATA(ls_currency_quantity) = lo_field_content->get_currency_quantity( ).
```

Origins

An origin is an abstraction that encapsulates where the content (and existence status) of an object or part thereof are read from. As explained above, object/part as well as content handles are only backed by the semantic identifiers (e.g. the name of the database table) but do not make any choices if the content is read from the local system or from elsewhere. To this extent, each GET* method on a content handle asks for an (optional) origin from which the attribute will henceforth be read. In short:

- Content handle: Encapsulates **what** shall be read
- Origin: Encapsulates **where** content shall be read from

As of now, the XCO Read APIs define the following three kinds of origins (availability depending on the implementation status of each object type):

- Default: The default origin will be used when no explicit origin is supplied. With the default origin, an object will be read exactly once per ABAP session from the local system. Once read from the local database, the content will be cached for the duration of the ABAP Session and used for any subsequent reads
- Local: The local origin reads from the local database. The caching behavior for individual or groups of objects can be configured freely
- Remote: Based on an RFC destination (given as IF_RFC_DEST) objects are read from remote systems. The caching behavior for individual or groups of objects can be configured freely

Note that not all object types support the origin concept yet (see table below). In case an object type does not yet support the origin concept the content will always be read from the local database.

Object Type Feature Matrix

Object Type		Supports Origins?	Origin Factory	Supports Local Origin	Supports Remote Origin
APLO	Application Log Object	No	-	-	-
CLAS	Class	Yes	XCO_CP_ABAP_OBJECTS=>ORIGIN	Yes	Yes

Object Type		Supports Origins?	Origin Factory	Supports Local Origin	Supports Remote Origin
DDLS	Data Definition	No	-	-	-
DOMA	Domain	Yes	XCO_CP_DOMAIN=>ORIGIN	Yes	No
DRTY	CDS Type Definition	Yes	XCO_CP_CDS_TYPE_DEFINITION	Yes	No
DTEL	Data Element	Yes	XCO_CP_DATA_ELEMENT=>ORIGIN	Yes	No
EVTB	Event Binding	Yes	XCO_CP_EVENT_BINDING=>ORIGIN	Yes	No
FUGR	Function Group	Yes	XCO_CP_FUNCTION_GROUP=>ORIGIN	Yes	No
	Function Module	Yes	XCO_CP_FUNCTION_MODULE=>ORIGIN	Yes	No
INTF	Interface	Yes	XCO_CP_ABAP_OBJECTS=>ORIGIN	Yes	Yes
MSAG	Message Class	No	-	-	-
NONT	SAP Object Node Type	Yes	XCO_CP_SAP_OBJECT_NODE_TYPE=>ORIGIN	Yes	No
RONT	SAP Object Type	Yes	XCO_CP_SAP_OBJECT_TYPE=>ORIGIN	Yes	No
SRVB	Service Binding	Yes	XCO_CP_SERVICE_BINDING=>ORIGIN	Yes	No
SRVC	Service Consumption Model	Yes	XCO_CP_SERVICE_CONS_MODEL=>ORIGIN	Yes	No
SRVD	Service Definition	Yes	XCO_CP_SERVICE_DEFINITION=>ORIGIN	Yes	No
TABL	Structure	No	-	-	-
	Database Table	Yes	XCO_CP_DATABASE_TABLE=>ORIGIN	Yes	No
	Global Temporary Table	No	-	-	-
TTYP	Table Type	No	-	-	-
XSLT	Transformation	No	-	-	-

4.2.15.18.3.2.1 Reading a Class

Given the following class

```
CLASS zcl_xco_doc_cp_cs_class DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION.
    CLASS-METHODS:
      "! <p class="shorttext synchronized" lang="en">
      "!  Calculate the factorial for the given natural number
      "! </p>
      "!
      "! @parameter iv_n |
      "!  <p class="shorttext synchronized" lang="en">
      "!    The natural number for which to calculate the factorial
      "!  </p>
      "! @parameter rv_factorial_n |
      "!  <p class="shorttext synchronized" lang="en">
      "!    The factorial for the given natural number
      "!  </p>
      "! @raising cx_abap_invalid_value |
      "!  <p class="shorttext synchronized" lang="en">
      "!    If IV_N is less than 0
      "!  </p>
      factorial
        IMPORTING
          iv_n                 TYPE i
        RETURNING
          VALUE(rv_factorial_n) TYPE i
        RAISING
          cx_abap_invalid_value.
ENDCLASS.
CLASS zcl_xco_doc_cp_cs_class IMPLEMENTATION.
  METHOD factorial.
    IF iv_n LT 0.
      RAISE EXCEPTION TYPE cx_abap_invalid_value.
    ELSEIF iv_n EQ 0.
      rv_factorial_n = 1.
    ELSE.
      rv_factorial_n = iv_n * factorial( iv_n - 1 ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

its contents can be read and written to the console as follows

Sample Code

```
CLASS zcl_xco_doc_cp_cs_read_clas DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.
  PRIVATE SECTION.
    CONSTANTS:
      co_class_name  TYPE sxco_ao_object_name VALUE 'ZCL_XCO_DOC_CP_CS_CLASS'.
    METHODS:
      write_parameters
        IMPORTING
          io_method TYPE REF TO if_xco_ao_c_method
          out       TYPE REF TO if_xco_adt_classrun_out,
      write_exceptions
```

```
        IMPORTING
          io_method TYPE REF TO if_xco_ao_c_method
          out       TYPE REF TO if_xco_adt_classrun_out,
      write_source
        IMPORTING
          io_class       TYPE REF TO if_xco_ao_class
          iv_method_name TYPE sxco_clas_method_name
          out            TYPE REF TO if_xco_adt_classrun_out.
ENDCLASS.
CLASS zcl_xco_doc_cp_cs_read_clas IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).
  ENDMETHOD.
  METHOD main.
    DATA(lo_class) = xco_cp_abap=>class( co_class_name ).
    " DEFINITION.
    DATA(ls_definition_content) = lo_class->definition->content( )->get( ).
    out->write( |Definition of { lo_class->name }:| ).
    out->write( ls_definition_content ).
    " PUBLIC class methods.
    DATA(lt_public_methods) = lo_class->definition->section-public-
>components->class_method->all->get( ).
    LOOP AT lt_public_methods INTO DATA(lo_public_method).
      out->write( |\nClass method { lo_public_method->name }:| ).
      DATA(ls_public_method) = lo_public_method->content( )->get( ).
      out->write( ls_public_method ).
      out->write( |\nParameters:| ).
      write_parameters(
        io_method = lo_public_method
        out       = out
      ).
      out->write( |\nExceptions:| ).
      write_exceptions(
        io_method = lo_public_method
        out       = out
      ).
      out->write( |\nSource:| ).
      write_source(
        io_class       = lo_class
        iv_method_name = lo_public_method->name
        out            = out
      ).
    ENDLOOP.
  ENDMETHOD.
  METHOD write_parameters.
    " IMPORTING parameters.
    DATA(lt_importing_parameters) = io_method->importing_parameters->all-
>get( ).
    LOOP AT lt_importing_parameters INTO DATA(lo_importing_parameter).
      out->write( lo_importing_parameter->name ).
      DATA(ls_importing_parameter) = lo_importing_parameter->content( )-
>get( ).
      out->write( ls_importing_parameter ).
    ENDLOOP.
    " RETURNING parameters.
    DATA(lt_returning_parameters) = io_method->returning_parameters->all-
>get( ).
    LOOP AT lt_returning_parameters INTO DATA(lo_returning_parameter).
      out->write( lo_returning_parameter->name ).
      DATA(ls_returning_parameter) = lo_returning_parameter->content( )-
>get( ).
      out->write( ls_returning_parameter ).
    ENDLOOP.
  ENDMETHOD.
  METHOD write_exceptions.
    DATA(lt_exceptions) = io_method->exceptions->all->get( ).
    LOOP AT lt_exceptions INTO DATA(lo_exception).
      out->write( lo_exception->name ).
```

```
      DATA(ls_exception) = lo_exception->content( )->get( ).
      out->write( ls_exception ).
    ENDLOOP.
  ENDMETHOD.
  METHOD write_source.
    DATA(ls_implementation) = io_class->implementation->method( iv_method_name
      )->content( )->get( ).
    DATA(lv_source) = xco_cp=>strings( ls_implementation-source
      )->join( |{ cl_abap_char_utilities=>cr_lf }|
      )->value.
    out->write( lv_source ).
  ENDMETHOD.
ENDCLASS.
```

4.2.15.18.3.2.2 Reading AMDP Attributes of a Method Implementation

Given a class with an AMDP method implementation such as

Sample Code

```
CLASS zcl_xco_doc_cp_amdp DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION.
    INTERFACES:
      if_amdp_marker_hdb.

    METHODS:
      amdp_method.
ENDCLASS.

CLASS zcl_xco_doc_cp_amdp IMPLEMENTATION.
  METHOD amdp_method BY DATABASE PROCEDURE FOR HDB LANGUAGE SQLSCRIPT
      OPTIONS
        READ-ONLY
      USING
        zxco_amdp_dbt.
    SELECT * FROM zxco_amdp_dbt WHERE id = '001';
  ENDMETHOD.
ENDCLASS.
```

the AMDP attributes of the method implementation can be read out as follows:

Sample Code

```
"! Code sample for reading the AMDP attributes of a method implementation.
"!
"! IMPORTANT: The values of the constants CO_CLASS_NAME and CO_METHOD_NAME
"! must be replaced with the name of an existing class and a method that is
"! implemented in this class.
CLASS zcl_xco_doc_cp_read_amdp DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.

  PRIVATE SECTION.
    CONSTANTS:
      co_class_name  TYPE sxco_ao_object_name VALUE 'ZCL_XCO_DOC_CP_AMDP',
```

```
      co_method_name TYPE sxco_clas_method_name VALUE 'AMDP_METHOD'.
ENDCLASS.

CLASS zcl_xco_doc_cp_read_amdp IMPLEMENTATION.
  METHOD main.
    DATA(lo_method_implementation) = xco_cp_abap=>class( co_class_name
      )->implementation->method( co_method_name ).

    " LS_METHOD_IMPLEMENTATION will contain both the AMDP attributes and the
source
    " code of the method implementation.
    DATA(ls_method_implementation) = lo_method_implementation->content( )-
>get( ).
    out->write( ls_method_implementation ).

    " The AMDP database type is represented as an object of type
CL_XCO_AMDP_DB_TYPE.
    " Supported AMDP database types are available via
XCO_CP_AMDP=>DATABASE_TYPE.
    DATA(lo_amdp_database_type) = ls_method_implementation-amdp-database_type.
    out->write( lo_amdp_database_type ).

    " The AMDP database language is represented as an object of type
CL_XCO_AMDP_DB_LANGUAGE.
    " Supported AMDP database languages are available via
XCO_CP_AMDP=>DATABASE_LANGUAGE.
    DATA(lo_amdp_database_language) = ls_method_implementation-amdp-
database_language.
    out->write( lo_amdp_database_language ).

    " The AMDP database options are represented as objects of type
CL_XCO_AMDP_DB_OPTION.
    " Supported AMDP database options are available via
XCO_CP_AMDP=>DATABASE_OPTION.
    DATA(lt_amdp_database_options) = ls_method_implementation-amdp-
database_options.
    out->write( lt_amdp_database_options ).

    " An AMDP database entity is represented as object of type
IF_XCO_AMDP_DATABASE_ENTITY.
    " The IF_XCO_PRINTABLE~GET_TEXT method can be used to obtain the string
value for a database
    " entity.
    LOOP AT ls_method_implementation-amdp-database_entities INTO
DATA(lo_amdp_database_entity).
      DATA(lv_amdp_database_entity) = lo_amdp_database_entity-
>if_xco_printable~get_text(
        )->get_lines(
        )->join(
        )->value.
      out->write( lv_amdp_database_entity ).
    ENDLOOP.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.18.3.2.3 Reading a Function Module

Given the following function module

Sample Code

```
FUNCTION z_xco_doc_cp_cs_fm
  IMPORTING
    VALUE(im_class) TYPE REF TO cl_xco_ad_built_in_type OPTIONAL
    im_interface TYPE REF TO if_xco_news
    VALUE(im_abap_built_in_type) TYPE i DEFAULT 5
    VALUE(im_abap_gt_reference) TYPE REF TO data
  EXPORTING
    ex_p1 TYPE REF TO string
    VALUE(ex_p2) TYPE c
  CHANGING
    ch_1 TYPE any
    VALUE(ch_2) TYPE i
  RAISING
    cx_sy_zerodivide
    RESUMABLE(cx_sy_assign_cast_error).



  IF im_class IS NOT INITIAL.
    ch_1 = 'ABC'.
  ENDIF.

ENDFUNCTION.
```

its contents can be read and written to the console as follows

Sample Code

```
CLASS zcl_xco_doc_cp_cs_fnctn_mdle DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.

  PRIVATE SECTION.
    METHODS:
      read_importing_parameters
        IMPORTING
          io_function_module TYPE REF TO if_xco_function_module
          out                TYPE REF TO if_xco_adt_classrun_out,

      read_exporting_parameters
        IMPORTING
          io_function_module TYPE REF TO if_xco_function_module
          out                TYPE REF TO if_xco_adt_classrun_out,

      read_changing_parameters
        IMPORTING
          io_function_module TYPE REF TO if_xco_function_module
          out                TYPE REF TO if_xco_adt_classrun_out,

      read_exceptions
        IMPORTING
          io_function_module TYPE REF TO if_xco_function_module
          out                TYPE REF TO if_xco_adt_classrun_out.
ENDCLASS.

CLASS zcl_xco_doc_cp_cs_fnctn_mdle IMPLEMENTATION.
  METHOD main.
    DATA(lo_function_module) =
xco_cp_abap=>function_module( 'Z_XCO_DOC_CP_CS_FM' ).
    out->write( lo_function_module ).
```

```
    DATA(ls_function_module) = lo_function_module->content( )->get( ).
    out->write( ls_function_module ).

    out->write( |\nIMPORTING parameters:| ) ##NO_TEXT.
    read_importing_parameters(
      io_function_module = lo_function_module
      out                = out
    ).

    out->write( |\nEXPORTING parameters:| ) ##NO_TEXT.
    read_exporting_parameters(
      io_function_module = lo_function_module
      out                = out
    ).

    out->write( |\nCHANGING parameters:| ) ##NO_TEXT.
    read_changing_parameters(
      io_function_module = lo_function_module
      out                = out
    ).

    out->write( |\nExceptions:| ) ##NO_TEXT.
    read_exceptions(
      io_function_module = lo_function_module
      out                = out
    ).
  ENDMETHOD.

  METHOD read_importing_parameters.
    DATA(lt_importing_parameters) = io_function_module->importing_parameters-
>all->get( ).

    LOOP AT lt_importing_parameters INTO DATA(lo_importing_parameter).
      out->write( lo_importing_parameter ).

      DATA(ls_type) = lo_importing_parameter->content( )->get( ).
      out->write( ls_type ).
    ENDLOOP.
  ENDMETHOD.

  METHOD read_exporting_parameters.
    DATA(lt_exporting_parameters) = io_function_module->exporting_parameters-
>all->get( ).

    LOOP AT lt_exporting_parameters INTO DATA(lo_exporting_parameter).
      out->write( lo_exporting_parameter ).

      DATA(ls_type) = lo_exporting_parameter->content( )->get( ).
      out->write( ls_type ).
    ENDLOOP.
  ENDMETHOD.

  METHOD read_changing_parameters.
    DATA(lt_changing_parameters) = io_function_module->changing_parameters-
>all->get( ).

    LOOP AT lt_changing_parameters INTO DATA(lo_changing_parameter).
      out->write( lo_changing_parameter ).

      DATA(ls_type) = lo_changing_parameter->content( )->get( ).
      out->write( ls_type ).
    ENDLOOP.
  ENDMETHOD.

  METHOD read_exceptions.
    DATA(lt_exceptions) = io_function_module->exceptions->all->get( ).

    LOOP AT lt_exceptions INTO DATA(lo_exceptions).
```

```
      out->write( lo_exceptions ).

      DATA(ls_type) = lo_exceptions->content( )->get( ).
      out->write( ls_type ).
    ENDLOOP.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.18.3.2.4 Reading a Transformation

Given a transformation (i.e. an XSLT object), its contents can be read and written to the console as follows:

Sample Code

```
"! Code sample for reading the contents of a transformation (XSLT object).
"!
"! IMPORTANT: The value of the constant CO_TRANSFORMATION_NAME must be
replaced with
"! the name of an existing transformation for which the content shall be read.
CLASS zcl_xco_doc_cp_read_trnsfrmtn DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.

  PRIVATE SECTION.
    CONSTANTS:
      co_transformation_name TYPE sxco_tf_object_name VALUE
'ZMY_TRANSFORMATION'.
ENDCLASS.

CLASS zcl_xco_doc_cp_read_trnsfrmtn IMPLEMENTATION.
  METHOD main.
    DATA(lo_transformation) = xco_cp_abap_repository=>object->xslt-
>for( co_transformation_name ).
    out->write( lo_transformation ).

    " LS_TRANSFORMATION will contain both the short description and the
source code of the
    " transformation.
    DATA(ls_transformation) = lo_transformation->content( )->get( ).
    out->write( ls_transformation ).

    " The source code of the transformation is represented as an object of
type IF_XCO_TF_SOURCE_CODE.
    " Via the IF_XCO_TEXT~GET_LINES method the source code can also be easily
obtained as a string_table.
    DATA(lt_source_code) = ls_transformation-source_code-
>if_xco_text~get_lines( )->value.
    out->write( lt_source_code ).
  ENDMETHOD.
ENDCLASS.
```

4.2.15.18.3.2.5 Reading Classes and Interfaces from a Remote Destination

Besides the default and local origin, the Read APIs for both classes and interfaces offer the option to read the content of classes and interfaces from remote systems. Currently, only reading from On-Premise systems is supported. Reading from SAP Public Cloud systems is not yet available. Remote systems are identified via an `IF_RFC_DEST` for which objects can be obtained via `CL_RFC_DESTINATION_PROVIDER`. Origins for classes and interfaces are obtained via `XCO_CP_ABAP_OBJECTS=>ORIGIN` which provides a common basis for both classes and interfaces. To obtain a remote origin, the following code can be used:

Sample Code

```
TRY.
    DATA(lo_rfc_destination) =
cl_rfc_destination_provider=>create_by_comm_arrangement( 'MY_COMM_ARRANGEMENT'
 ).
  CATCH cx_rfc_dest_provider_error INTO DATA(lx_rfc_dest_provider_error).
    " Handle exception.
ENDTRY.

DATA(lo_remote_origin) = xco_cp_abap_objects=>origin-
>remote( lo_rfc_destination ).
```

Note

Authorization requirements for remote reading

Note that the user used for the system identified by the RFC destination needs to have the following authorizations in order to support the remote reading of classes and interfaces:

- S_RFC with RFC_TYPE = Function module, RFC_NAME = `SEO_CLASS_TYPEINFO_GET_RFC`, `SEO_INTERFACE_TYPEINFO_GET_RFC` and ACTVT = Execute

or alternatively for the complete function group, meaning

- S_RFC with RFC_TYPE = Function group, RFC_NAME = SEO_REMOTE and ACTVT = Execute

Once the origin has been obtained, it can be used for any of the exposed GET_* methods as part of the Read APIs for classes and interfaces. The code sample below illustrates how the public methods of the active version of a class can be read out for a remote destination:

Sample Code

```
DATA(lt_public_methods) = xco_cp_abap=>class( 'MY_REMOTE_CLASS' )->definition-
>section-public->components->method->all->get(
  io_version = xco_cp_abap_objects=>version->active
  io_origin  = lo_remote_origin
).

LOOP AT lt_public_methods INTO DATA(lo_public_method).
  " The name of the public method.
  DATA(lv_public_method_name) = lo_public_method->name.

  " The short description of the method.
  DATA(lv_short_description) = ls_public_method-short_description.

  " The indicator whether the method is ABSTRACT.
```

```
  DATA(lv_abstract_indicator) = ls_public_method-abstract_indicator.

  " The indicator whether the method is FINAL.
  DATA(lv_final_indicator) = ls_public_method-final_indicator.

  " The indicator whether the method is a REDEFINITION.
  DATA(lv_redefinition_indicator) = ls_public_method-redefinition_indicator.

  " A structure containing the AMDP attributes of the method (definition).
  DATA(ls_amdp) = ls_public_method-amdp.
ENDLOOP.
```

Similarly, using the same LO_REMOTE_ORIGIN as in the example above, we can read the constants of the inactive version of a remote interface:

Sample Code

```
DATA(lt_constants) = xco_cp_abap=>interface( 'MY_REMOTE_INTERFACE' )-
>components->constant->all->get(
  io_version = xco_cp_abap_objects=>version->inactive
  io_origin  = lo_remote_origin
).

LOOP AT lt_constants INTO DATA(lo_constant).
  " The name of the constant.
  DATA(lv_constant_name) = lo_constant->name.

  DATA(ls_constant) = lo_constant->content( xco_cp_abap_objects=>version-
>inactive )->get( lo_remote_origin ).

  " The typing method of the constant.
  DATA(lo_typing_method) = ls_constant-typing_method.

  " The typing definition of the constant.
  DATA(lo_typing_definition) = ls_constant-typing_definition.

  " The value of the constant
  DATA(lv_value) = ls_constant-value.
ENDLOOP.
```

4.2.15.18.3.3 Generation APIs

The XCO Generation APIs are the part of the XCO library that allows the programmatic creation, update and deletion of ABAP repository objects. It consists of high-level and strongly typed APIs for the following objects types:

- BDEF (Behavior Definitions)
- BGQC (Background Processing Context)
- CLAS (Classes)
- DCLS (Access Controls)
- DDLS (Data Definitions)
- DDLX (Metadata Extensions)
- DESD (Logical External Schema)

- DEVC (Packages)
- DOMA (Domains)
- DRTY (CDS Type Definitions)
- DTEL (Data Elements)
- EVTB (Event Bindings)
- FUGR (Function Groups)
- INTF (Interfaces)
- MSAG (Message Classes)
- NONT (SAP Object Node Types)
- RONT (SAP Object Types)
- SCP1 (Business Configuration Sets)
- SRVB (Service Bindings)
- SRVD (Service Definitions)
- TABL (Structures, Database Tables and Global Temporary Tables)
- TTYP (Table Types)
- XSLT (Transformations)

The following kinds of operations are supported:

- POST Create the object according to a provided specification
- PUT Create or update the object according to a provided specification
- PATCH Update the object according to a provided specification
- DELETE Delete the object

The matrix below shows which kinds of operations are supported for which object types:

	PUT	POST	PATCH	DELETE
BDEF	X			X
BGQC	X		X	X
CLAS	X		X	X
DCLS	X			X
DDLS	X			X
DDLX	X			X
DESD	X			X
DEVC	X			X
DOMA	X		X	X
DRTY	X			X
DTEL	X		X	X
EVTB	X		X	X
FUGR		X	X	X
INTF	X		X	X
MSAG	X		X	X

	PUT	POST	PATCH	DELETE
NONT	X		X	X
RONT	X		X	X
SCP1		X	X	X
SRVB	X			X
SRVD	X			X
TABL	X			X
TTYP	X			X
XSLT	X			X

4.2.15.18.3.3.1 Design of the XCO Generation APIs

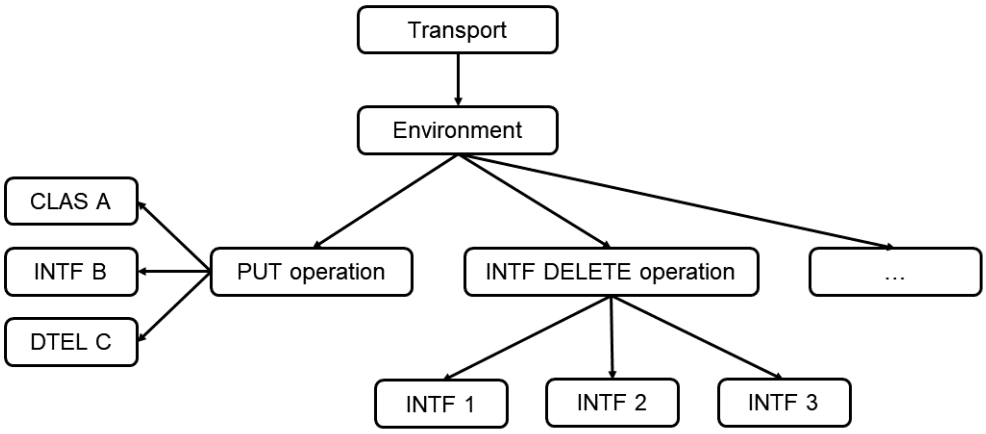
As with the whole XCO library, the design of the XCO Generation APIs is governed by core principles:

- **Environment**
  The prerequisite for building and executing operations is a generation environment. A generation environment is backed by a transport which must be in status ‘Modifiable’ and is obtained via the XCO_CP_GENERATION API.

  Sample Code

  ```
  " In this example, X08K123456 would need to be a modifiable Workbench
  " transport.
  DATA(lo_environment) = xco_cp_generation=>environment-
  >dev_system( 'X08K123456' ).
  DATA(lo_put_operation) = lo_environment->create_put_operation( ).
  lo_put_operation->for-dtel->add_object( '...' ).
  " ...
  ```

  The environment acts as the entry point to the type-specific Generation APIs which allow to build and successively execute PUT and DELETE operations. All system changes caused by an operation created for an environment will be written to the environment’s transport.



- **Operations**
  Operations consist of a set of objects (identified by their type and name) and allow to perform an action for these objects (determined by the kind of operation).
  PUT operations
  Upon execution, a PUT operation either creates or updates its objects according to provided specifications (depending on whether the object already exists in the system). Each object type defines a form-based specification tailored to the specific attributes of the object type which is used to describe the content of each object of the PUT operation. As a PUT operation potentially creates new objects a valid package must be provided for all objects without a package. This package is used when the object is newly created but is ignored when the object already exists and is only updated.

  Sample Code

```
DATA(lo_put_operation) = lo_environment->create_put_operation( ).
DATA(lo_specification) = lo_put_operation->for-dtel-
>add_object( 'ZDATA_ELEMENT'
  )->set_package( 'ZPACKAGE'
  )->create_form_specification( ).
lo_specification->set_short_description( 'My generated data element' ).
lo_specification->set_data_type( xco_cp_abap_dictionary=>built_in_type-
>char( 10 ) ).
lo_specification->field_label-short->set_text( 'ID' ).
lo_specification->field_label-medium->set_text( 'Identifier' ).
lo_specification->field_label-long->set_text( 'User identifier' ).
lo_specification->field_label-heading->set_text( 'User identifier' ).
lo_put_operation->execute( ).
```

  Object types that support inactive versions can be combined into a single PUT operation. Once the objects have either been created or updated (in inactive version) all the objects of the PUT operation will be activated together in a single mass activation. This provides support for interdependencies (even circular ones) between the objects of a PUT operation.
  It is furthermore possible to influence the execution of a PUT operation with so-called options. Currently the following options are offered:

  - Skip activation: Create or update the objects of the PUT operation but do not perform a mass activation afterwards

Using an option is as simple as supplying it to the EXECUTE method invocation of the corresponding PUT operation:

Sample Code

```
lo_put_operation->execute( VALUE
#( ( xco_cp_generation=>put_operation_option->skip_activation ) ) ).
```

POST operations

The objects on a POST operation consist of a name choice (IF_XCO_NAME_CHOICE) and a specification. In contrast to PUT operations, objects are not explicitly identified by their name, but instead a name choice is provided which will be evaluated upon execution of the POST operation.

By providing a name choice instead of a fixed name it is possible to potentially pick a fallback name in case the preferred name is already taken. However, this is not mandatory and it is also possible to provide a fixed name, e.g. for the creation of function group:

```
DATA(lo_post_operation) = lo_environment->for-fugr->create_post_operation( ).
DATA(lo_fixed_name_choice) = xco_cp_name=>choice->fixed( 'ZMY_FNCTN_GRP' ).
lo_post_operation->add_object( lo_fixed_name_choice
  )->set_package( 'ZLOCAL'
  )->create_form_specification( ).
" ...
lo_post_operation->execute( ).
```

In any case, upon execution the name choices for all the objects on the POST operation are resolved. The operation will only proceed if all name choices can be resolved, i.e. a name can be determined for which no object already exists in the ABAP repository. If all name choices were resolved successfully the semantics of a POST operation are those of a PUT operation where the objects do not exist prior to the execution.

PATCH operations

Upon execution, a PATCH operation will update its objects according to the changes provided in the specifications. The available changes depend on the object type.

For message classes (MSAG) it is e.g. possible to insert, update, modify or delete messages of the message class. The semantics are as follows:

- INSERT Add a new part to the object if it does not exist yet. If it already exists, no action will be performed.
- UPDATE Update an existing part of the object. If the part does not exist, no action will be performed.
- MODIFY Modify the specified part of the object. If the part does not exist, it will be created according to the specification, otherwise it will be updated.
- DELETE Delete the specified part of the object. If the specified part does not exist no action will be performed

DELETE operations

Upon execution, a DELETE operation will delete all its objects (in case they exist in the system). DELETE operations are always built and executed for a given type and it is the responsibility of the caller to ensure that objects of different types are deleted in the correct order (e.g. a domain still in use by a data element cannot be deleted; the data element must be deleted first).

COMMIT behavior of operations

The execution of an operation will always execute a COMMIT WORK.

Errors during operation execution

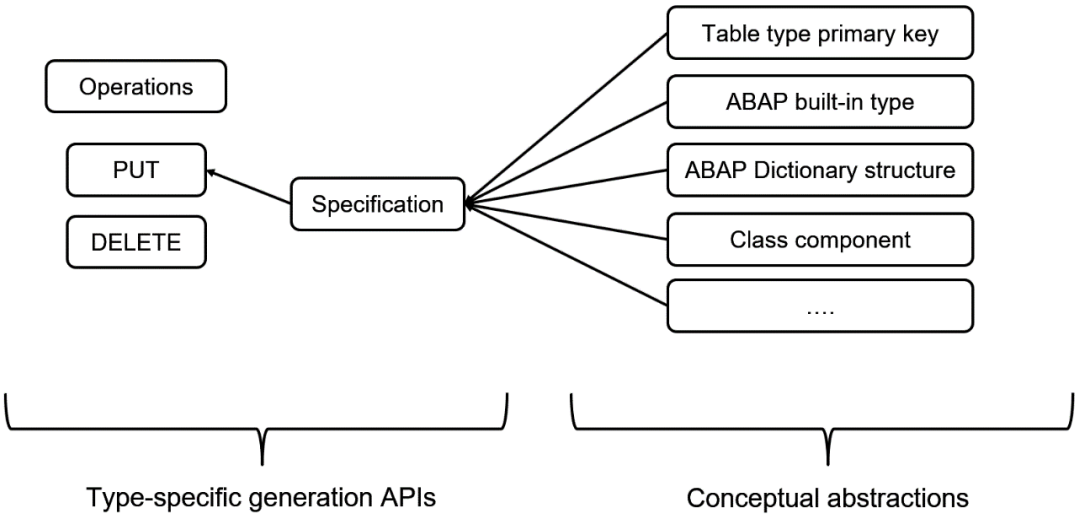
If the execution of an operation fails, the following exceptions are used to communicate the failure:

- CX_XCO_GEN_PUT_EXCEPTION for a failed PUT operation
- CX_XCO_GEN_DELETE_EXCEPTION for a failed DELETE operation

- CX_XCO_GEN_PATCH_EXCEPTION for a failed PATCH operation

Each exception provides access to the findings, i.e. all the messages and which object they belong to, that were encountered when attempting to execute the operation.
In case the mass activation of a PUT operation fails, the already existing inactive versions will not be deleted.

- **Conceptual Abstractions**
  The overall design and architecture of the XCO Generation functionality is based on the differentiation between conceptual abstractions and type-specific generation APIs.



Conceptual abstractions represent entities in the ABAP system which can be used in conjunction with different object types. For example, a built-in type of the ABAP Dictionary may be used as the format for a domain (DOMA), the data type of a data element (DTEL) or the row type of a table type (TTYP). Conceptual abstractions create a common language which can be used across different object types in the context of generation.

Sample Code

```
DATA(lo_char_10) = xco_cp_abap_dictionary=>built_in_type->char( 10 ).
lo_doma_specification->set_format( lo_char_10 ).
lo_dtel_specification->set_data_type( lo_char_10 ).
lo_ttyp_specification->set_row_type( lo_char_10 ).
```

- **4. Templates**
  Instead of manually filling a form specification, templates can be used to prefill an empty form specification from an existing ABAP Repository object. Currently the following object types support templates:
  - DOMA (Domain)
  - DTEL (Data element)
  - TABL (Database tables and structures)
  - TTYP (Table types)
  - DDLS (Data Definitions)

Using templates, it is e.g. possible to easily copy an existing object. The following code sample uses the active version of domain ZSOURCE_DOMAIN as a template and will create (resp. update) domain ZCOPIED_DOMAIN to reflect the content of ZSOURCE_DOMAIN:

Sample Code

```
DATA(lo_template) = xco_cp_generation_doma=>template->for_domain(
  iv_name       = 'ZSOURCE_DOMAIN'
  io_read_state = xco_cp_abap_dictionary=>object_read_state->active_version
).
DATA(lo_put_operation) = lo_environment->create_put_operation( ).
lo_put_operation->for-doma->add_object( 'ZCOPIED_DOMAIN'
  )->set_package( 'ZPACKAGE'
  )->set_template( lo_template ).
lo_put_operation->execute( ).
```

4.2.15.18.3.3.2 RAP BO Generation

The following code sample illustrates how a complete RAP business object, starting from domains for fixed values all the way up to a service binding, can be generated with the XCO Generation APIs:

Sample Code

```
"! Code sample for executing PUT operations to generate a full RAP business
object.
"!
"! IMPORTANT: The values of the constants CO_PACKAGE and CO_TRANSPORT must be
replaced
"! with an existing package and a modifiable Workbench transport matching the
transport
"! target of the package.
CLASS zcl_xco_doc_cp_cs_gen_put DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.
  PRIVATE SECTION.
    CONSTANTS:
      co_package              TYPE sxco_package VALUE 'ZXCO_CP_PACKAGE',
      co_transport            TYPE sxco_transport VALUE 'X08K900164',
      co_doma_status          TYPE sxco_ad_object_name VALUE
'ZXCO_CP_VR_STATUS',
      co_dtel_status          TYPE sxco_ad_object_name VALUE
'ZXCO_CP_VR_STATUS',
      co_tabl_dbt_name        TYPE sxco_dbt_object_name VALUE
'ZXCO_CP_VAC_REQ',
      co_ddls_interface_name  TYPE sxco_cds_object_name VALUE
'ZXCO_CP_I_VACATION_REQUEST',
      co_ddls_consumption_name TYPE sxco_cds_object_name VALUE
'ZXCO_CP_C_VACATION_REQUEST',
      co_ddlx_name            TYPE sxco_cds_object_name VALUE
'ZXCO_CP_VACATION_REQUEST_EXT',
      co_dcls_name            TYPE sxco_cds_object_name VALUE
'ZXCO_CP_VACTION_REQUEST_AC',
      co_clas_name            TYPE sxco_ad_object_name VALUE
'ZXCO_CP_BP_VACATION_REQUEST',
```

```
      co_srvd_name             TYPE sxco_srvd_object_name VALUE
'ZXCO_CP_VAC_REQ_SRVD',
      co_srvb_name             TYPE sxco_srvb_object_name VALUE
'ZXCO_CP_VAC_REQ_SRVB'.
    DATA:
      mo_environment TYPE REF TO if_xco_cp_gen_env_dev_system.
    METHODS:
      add_doma
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put,
      add_dtel
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put,
      add_tabl_dbt
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put,
      add_ddls
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put,
      add_ddlx
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put,
      add_dcls
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put,
      add_bdef
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put,
      add_clas
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put,
      add_srvd
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put,
      put_srvb.
ENDCLASS.
CLASS zcl_xco_doc_cp_cs_gen_put IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).
    mo_environment = xco_cp_generation=>environment-
>dev_system( co_transport ).
  ENDMETHOD.
  METHOD main.
    DATA(lo_put_operation) = mo_environment->create_put_operation( ).
    add_doma( lo_put_operation ).
    add_dtel( lo_put_operation ).
    add_tabl_dbt( lo_put_operation ).
    add_ddls( lo_put_operation ).
    add_ddlx( lo_put_operation ).
    add_dcls( lo_put_operation ).
    add_bdef( lo_put_operation ).
    add_clas( lo_put_operation ).
    add_srvd( lo_put_operation ).
    lo_put_operation->execute( ).
    put_srvb( ).
    out->write( |PUT operations executed successfully.| ).
  ENDMETHOD.
  METHOD add_doma.
    DATA(lo_specification) = io_put_operation->for-doma-
>add_object( co_doma_status
      )->set_package( co_package
      )->create_form_specification( ).
    lo_specification->set_short_description( 'Vacation request status' ).
    lo_specification->set_format( xco_cp_abap_dictionary=>built_in_type-
>char( 10 ) ).
    lo_specification->fixed_values->add_fixed_value( 'APPROVED'
      )->set_description( 'Approved' ).
    lo_specification->fixed_values->add_fixed_value( 'DECLINED'
```

```
      )->set_description( 'Declined' ).
  ENDMETHOD.
  METHOD add_dtel.
    DATA(lo_specification) = io_put_operation->for-dtel-
>add_object( co_dtel_status
      )->set_package( co_package
      )->create_form_specification( ).
    lo_specification->set_short_description( 'Vacation request status' ).
    lo_specification-
>set_data_type( xco_cp_abap_dictionary=>domain( co_doma_status ) ).
  ENDMETHOD.
  METHOD add_tabl_dbt.
    DATA(lo_specification) = io_put_operation->for-tabl-for-database_table-
>add_object( co_tabl_dbt_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_specification->set_short_description( 'Vacation request'
      )->set_delivery_class( xco_cp_database_table=>delivery_class->a
      )->set_data_maintenance( xco_cp_database_table=>data_maintenance-
>allowed ).
    lo_specification->add_field( 'CLIENT' )-
>set_type( xco_cp_abap_dictionary=>built_in_type->clnt
      )->set_key_indicator(
      )->set_not_null( ).
    lo_specification->add_field( 'IDENTIFIER' )-
>set_type( xco_cp_abap_dictionary=>built_in_type->char( 30 )
      )->set_key_indicator(
      )->set_not_null( ).
    lo_specification->add_field( 'START_DATE' )-
>set_type( xco_cp_abap_dictionary=>built_in_type->dats ).
    lo_specification->add_field( 'END_DATE' )-
>set_type( xco_cp_abap_dictionary=>built_in_type->dats ).
    lo_specification->add_field( 'STATUS' )-
>set_type( xco_cp_abap_dictionary=>data_element( co_dtel_status ) ).
    lo_specification->add_field( 'IS_ACTIVE' )-
>set_type( xco_cp_abap_dictionary=>data_element( 'ABAP_BOOLEAN' ) ).
  ENDMETHOD.
  METHOD add_ddls.
    DATA(lo_interface_specification) = io_put_operation->for-ddls-
>add_object( co_ddls_interface_name
      )->set_package( co_package
      )->create_form_specification( ).
    DATA(lo_view_entity) = lo_interface_specification-
>set_short_description( 'Vacation request'
      )->add_view_entity( ).
    lo_view_entity->set_root( )->data_source->set_view_entity( CONV
#( co_tabl_dbt_name ) ).
    lo_view_entity->set_where( xco_cp_ddl=>field( 'is_active' )-
>eq( xco_cp_ddl=>literal->character( 'X' ) ) ).
    " View annotations.
    lo_view_entity->add_annotation( 'AccessControl.authorizationCheck' )-
>value->build( )->add_enum( 'CHECK' ).
    lo_view_entity->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( 'Vacation request view entity' ).
    lo_view_entity->add_annotation( 'Metadata.allowExtensions' )->value-
>build( )->add_boolean( abap_true ).
    " Add fields.
    DATA(lo_identifier)
= lo_view_entity->add_field( xco_cp_ddl=>field( 'identifier' ) )->set_key( )-
>set_alias( 'Identifier' ).
    lo_identifier->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( 'Identifier' ).
    lo_view_entity->add_field( xco_cp_ddl=>field( 'start_date' ) )-
>set_alias( 'StartDate' ).
    lo_view_entity->add_field( xco_cp_ddl=>field( 'end_date' ) )-
>set_alias( 'EndDate' ).
    lo_view_entity->add_field( xco_cp_ddl=>field( 'status' ) )-
>set_alias( 'Status' ).
```

```
    DATA(lo_consumption_specification) = io_put_operation->for-ddls-
>add_object( co_ddls_consumption_name
      )->set_package( co_package
      )->create_form_specification( ).
    DATA(lo_projection_view) = lo_consumption_specification-
>set_short_description( 'Vacation request projection'
      )->add_projection_view( ).
    lo_projection_view->set_root( ).
    lo_projection_view->data_source-
>set_view_entity( co_ddls_interface_name ).
    lo_projection_view->add_field( xco_cp_ddl=>field( 'Identifier' ) )-
>set_key( ).
    lo_projection_view->add_field( xco_cp_ddl=>field( 'StartDate' ) ).
    lo_projection_view->add_field( xco_cp_ddl=>field( 'EndDate' ) ).
    lo_projection_view->add_field( xco_cp_ddl=>field( 'Status' ) ).
  ENDMETHOD.
  METHOD add_ddlx.
    DATA(lo_specification) = io_put_operation->for-ddlx-
>add_object( co_ddlx_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_specification->set_short_description( 'Vacation request extension'
      )->set_layer( xco_cp_metadata_extension=>layer->customer
      )->set_view( co_ddls_interface_name ).
    " UI annotations.
    TYPES:
      BEGIN OF ts_field,
        name  TYPE sxco_cds_field_name,
        label TYPE string,
      END OF ts_field,
      tt_field TYPE STANDARD TABLE OF ts_field WITH EMPTY KEY.
    DATA(lt_fields) = VALUE tt_field(
      ( name = 'Identifier' label = 'Identifier' )
      ( name = 'StartDate' label = 'Start Date' )
      ( name = 'EndDate' label = 'End Date' )
      ( name = 'Status' label = 'Status' )
    ).
    LOOP AT lt_fields INTO DATA(ls_field).
      DATA(lv_position) = sy-tabix * 10.
      DATA(lo_field) = lo_specification->add_field( ls_field-name ).
      lo_field->add_annotation( 'UI.lineItem' )->value->build(
        )->begin_array(
          )->begin_record(
            )->add_member( 'position' )->add_number( lv_position
            )->add_member( 'label' )->add_string( ls_field-label
          )->end_record(
        )->end_array( ).
      lo_field->add_annotation( 'UI.identification' )->value->build(
        )->begin_array(
          )->begin_record(
            )->add_member( 'position' )->add_number( lv_position
            )->add_member( 'label' )->add_string( ls_field-label
          )->end_record(
        )->end_array( ).
    ENDLOOP.
  ENDMETHOD.
  METHOD add_dcls.
    DATA(lo_specification) = io_put_operation->for-dcls-
>add_object( co_dcls_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_specification->set_short_description( 'Vacation request access
control' ).
    DATA(lo_role) = lo_specification->add_role( ).
    lo_role->add_annotation( 'MappingRole' )->value->build(
      )->add_boolean( abap_true ).
    lo_role->add_access_rule( co_ddls_interface_name
      )->set_where( xco_cp_dcl=>field( 'Identifier' )->is_not_initial( ) ).
```

```
  ENDMETHOD.
  METHOD add_bdef.
    DATA(lo_interface_specification) = io_put_operation->for-bdef-
>add_object( co_ddls_interface_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_interface_specification->set_short_description( 'Vacation request
behavior'
      )-
>set_implementation_type( xco_cp_behavior_definition=>implementation_type-
>managed
      )->set_implementation_class( co_clas_name ).
    DATA(lo_interface_behavior) = lo_interface_specification->add_behavior( ).
    " Characteristics.
    lo_interface_behavior->characteristics-
>set_persistent_table( co_tabl_dbt_name
      )->lock->set_master( ).
    " Fields.
    lo_interface_behavior->add_mapping_for( co_tabl_dbt_name )-
>set_field_mapping( VALUE #(
      ( cds_view_field = 'Identifier' dbtable_field = 'identifier' )
      ( cds_view_field = 'StartDate' dbtable_field = 'start_date' )
      ( cds_view_field = 'EndDate' dbtable_field = 'end_date' )
      ( cds_view_field = 'Status' dbtable_field = 'status' )
    ) ).
    " Standard operations.
    lo_interface_behavior-
>add_standard_operation( xco_cp_behavior_definition=>standard_operation-
>create ).
    lo_interface_behavior-
>add_standard_operation( xco_cp_behavior_definition=>standard_operation-
>update ).
    lo_interface_behavior-
>add_standard_operation( xco_cp_behavior_definition=>standard_operation-
>delete ).
    " Action.
    DATA(lo_action) = lo_interface_behavior->add_action( 'approve' ).
    lo_action->set_external_name( 'Approve' ).
    lo_action->result->set_cardinality( xco_cp_cds=>cardinality->one )-
>set_self( ).
    DATA(lo_consumption_specification) = io_put_operation->for-bdef-
>add_object( co_ddls_consumption_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_consumption_specification->set_short_description( 'Vacation request
behavior projection'
      )-
>set_implementation_type( xco_cp_behavior_definition=>implementation_type-
>projection ).
    DATA(lo_consumption_behavior) = lo_consumption_specification-
>add_behavior( ).
    lo_consumption_behavior-
>add_standard_operation( xco_cp_behavior_definition=>standard_operation-
>create
      )->set_use( ).
    lo_consumption_behavior-
>add_standard_operation( xco_cp_behavior_definition=>standard_operation-
>update
      )->set_use( ).
    lo_consumption_behavior-
>add_standard_operation( xco_cp_behavior_definition=>standard_operation-
>delete
      )->set_use( ).
  ENDMETHOD.
  METHOD add_clas.
    DATA(lo_specification) = io_put_operation->for-clas-
>add_object( co_clas_name
      )->set_package( co_package
```

```
      )->create_form_specification( ).
    lo_specification->set_short_description( 'Behavior implementation' ).
    lo_specification->definition->set_abstract(
      )->set_for_behavior_of( co_ddls_interface_name ).
    DATA(lo_handler) = lo_specification->add_local_class( 'LCL_HANDLER' ).
    lo_handler->definition->set_superclass( 'CL_ABAP_BEHAVIOR_HANDLER' ).
    " Action implementation.
    DATA(lo_publish) = lo_handler->definition->section-private-
>add_method( 'APPROVE' ).
    lo_publish->behavior_implementation->set_for_modify(
      )->set_result( co_ddls_interface_name ).
    lo_publish->add_importing_parameter( 'IT_VACATION_REQUESTS'
      )->behavior_implementation->set_for_action(
        iv_entity_name = co_ddls_interface_name
        iv_action_name = 'approve'
    ).
    lo_handler->implementation->add_method( 'APPROVE' ).
  ENDMETHOD.
  METHOD add_srvd.
    DATA(lo_specification) = io_put_operation->for-srvd-
>add_object( co_srvd_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_specification->set_short_description( 'Service definition' ).
    lo_specification->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( 'Vacation request service definition' ).
    lo_specification->add_exposure( co_ddls_consumption_name )-
>set_alias( 'VacationRequest' ).
  ENDMETHOD.
  METHOD put_srvb.
    DATA(lo_put_operation) = mo_environment->for-srvb-
>create_put_operation( ).
    DATA(lo_specification) = lo_put_operation->add_object( co_srvb_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_specification->set_short_description( 'Service binding' ).
    lo_specification->set_binding_type( xco_cp_service_binding=>binding_type-
>odata_v2_ui ).
    lo_specification->add_service( )->add_version( '0001' )-
>set_service_definition( co_srvd_name ).
    lo_put_operation->execute( ).
  ENDMETHOD.
ENDCLASS.
```

All objects generated with the previous code sample can also be deleted again:

Sample Code

```
"! Code sample for executing DELETE operations to delete all objects of a
full RAP business
"! object.
"!
"! IMPORTANT: The value of the constant CO_TRANSPORT must be replaced with a
modifiable Workbench
"! transport matching the transport target of the package containing the
objects.
CLASS zcl_xco_doc_cp_cs_gen_delete DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.
  PRIVATE SECTION.
    CONSTANTS:
```

```
      co_transport             TYPE sxco_transport VALUE 'X08K900164',
      co_doma_status           TYPE sxco_ad_object_name VALUE
'ZXCO_CP_VR_STATUS',
      co_dtel_status           TYPE sxco_ad_object_name VALUE
'ZXCO_CP_VR_STATUS',
      co_tabl_dbt_name         TYPE sxco_dbt_object_name VALUE
'ZXCO_CP_VAC_REQ',
      co_ddls_interface_name   TYPE sxco_cds_object_name VALUE
'ZXCO_CP_I_VACATION_REQUEST',
      co_ddls_consumption_name TYPE sxco_cds_object_name VALUE
'ZXCO_CP_C_VACATION_REQUEST',
      co_ddlx_name             TYPE sxco_cds_object_name VALUE
'ZXCO_CP_VACATION_REQUEST_EXT',
      co_dcls_name             TYPE sxco_cds_object_name VALUE
'ZXCO_CP_VACTION_REQUEST_AC',
      co_clas_name             TYPE sxco_ad_object_name VALUE
'ZXCO_CP_BP_VACATION_REQUEST',
      co_srvd_name             TYPE sxco_srvd_object_name VALUE
'ZXCO_CP_VAC_REQ_SRVD',
      co_srvb_name             TYPE sxco_srvb_object_name VALUE
'ZXCO_CP_VAC_REQ_SRVB'.
    DATA:
      mo_environment TYPE REF TO if_xco_cp_gen_env_dev_system.
    METHODS:
      delete_srvb,
      delete_srvd,
      delete_clas,
      delete_bdef,
      delete_dcls,
      delete_ddlx,
      delete_ddls,
      delete_tabl_dbt,
      delete_dtel,
      delete_doma.
ENDCLASS.
CLASS zcl_xco_doc_cp_cs_gen_delete IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).
    mo_environment = xco_cp_generation=>environment-
>dev_system( co_transport ).
  ENDMETHOD.
  METHOD main.
    DATA(lo_put_operation) = mo_environment->create_put_operation( ).
    delete_srvb( ).
    delete_srvd( ).
    delete_clas( ).
    delete_bdef( ).
    delete_dcls( ).
    delete_ddlx( ).
    delete_ddls( ).
    delete_tabl_dbt( ).
    delete_dtel( ).
    delete_doma( ).
    out->write( |DELETE operations executed successfully.| ).
  ENDMETHOD.
  METHOD delete_srvb.
    DATA(lo_delete_operation) = mo_environment->for-srvb-
>create_delete_operation( ).
    lo_delete_operation->add_object( co_srvb_name ).
    lo_delete_operation->execute( ).
  ENDMETHOD.
  METHOD delete_srvd.
    DATA(lo_delete_operation) = mo_environment->for-srvd-
>create_delete_operation( ).
    lo_delete_operation->add_object( co_srvd_name ).
    lo_delete_operation->execute( ).
  ENDMETHOD.
  METHOD delete_clas.
```

```
    DATA(lo_delete_operation) = mo_environment->for-clas-
>create_delete_operation( ).
    lo_delete_operation->add_object( co_clas_name ).
    lo_delete_operation->execute( ).
  ENDMETHOD.
  METHOD delete_bdef.
    DATA(lo_delete_operation) = mo_environment->for-bdef-
>create_delete_operation( ).
    lo_delete_operation->add_object( co_ddls_consumption_name ).
    lo_delete_operation->add_object( co_ddls_interface_name ).
    lo_delete_operation->execute( ).
  ENDMETHOD.
  METHOD delete_dcls.
    DATA(lo_delete_operation) = mo_environment->for-dcls-
>create_delete_operation( ).
    lo_delete_operation->add_object( co_dcls_name ).
    lo_delete_operation->execute( ).
  ENDMETHOD.
  METHOD delete_ddlx.
    DATA(lo_delete_operation) = mo_environment->for-ddlx-
>create_delete_operation( ).
    lo_delete_operation->add_object( co_ddlx_name ).
    lo_delete_operation->execute( ).
  ENDMETHOD.
  METHOD delete_ddls.
    DATA(lo_delete_operation) = mo_environment->for-ddls-
>create_delete_operation( ).
    lo_delete_operation->add_object( co_ddls_consumption_name ).
    lo_delete_operation->add_object( co_ddls_interface_name ).
    lo_delete_operation->execute( ).
  ENDMETHOD.
  METHOD delete_tabl_dbt.
    DATA(lo_delete_operation) = mo_environment->for-tabl-for-database_table-
>create_delete_operation( ).
    lo_delete_operation->add_object( co_tabl_dbt_name ).
    lo_delete_operation->execute( ).
  ENDMETHOD.
  METHOD delete_dtel.
    DATA(lo_delete_operation) = mo_environment->for-dtel-
>create_delete_operation( ).
    lo_delete_operation->add_object( co_dtel_status ).
    lo_delete_operation->execute( ).
  ENDMETHOD.
  METHOD delete_doma.
    DATA(lo_delete_operation) = mo_environment->for-doma-
>create_delete_operation( ).
    lo_delete_operation->add_object( co_doma_status ).
    lo_delete_operation->execute( ).
  ENDMETHOD.
ENDCLASS.
```

4.2.15.18.3.3.3 Generation Error Handling

Each type of operation within the XCO Generation APIs defines its own exception class that can be used to deal with errors that were encountered during the execution of the operation:

- CX_XCO_GEN_POST_EXCEPTION for POST operations
- CX_XCO_GEN_PUT_EXCEPTION for PUT operations
- CX_XCO_GEN_PATCH_EXCEPTION for PATCH operations

- CX_XCO_GEN_DELETE_EXCEPTION for DELETE operations

The following code sample shows how the findings of a failed PUT operation can be extracted:

Sample Code

```
"! Code sample for handling errors of a PUT operation.
"!
"! IMPORTANT: The values of the constants CO_PACKAGE and CO_TRANSPORT must be
replaced
"! with an existing package and a modifiable Workbench transport matching the
transport
"! target of the package.
CLASS zcl_xco_doc_cp_cs_gen_error DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.
  PRIVATE SECTION.
    CONSTANTS:
      co_package   TYPE sxco_package VALUE 'ZXCO_CP_PACKAGE',
      co_transport TYPE sxco_transport VALUE 'X08K900164',
      co_doma_name TYPE sxco_ad_object_name VALUE
'ZXCO_CP_NON_EXISTENT_DOMAIN',
      co_dtel_name TYPE sxco_ad_object_name VALUE 'ZXCO_CP_INVALID_DTEL',
      co_ttyp_name TYPE sxco_ad_object_name VALUE 'ZXCO_CP_INVALID_TTYP'.
    DATA:
      mo_environment TYPE REF TO if_xco_cp_gen_env_dev_system.
ENDCLASS.
CLASS zcl_xco_doc_cp_cs_gen_error IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).
    mo_environment = xco_cp_generation=>environment-
>dev_system( co_transport ).
  ENDMETHOD.
  METHOD main.
    DATA(lo_put_operation) = mo_environment->create_put_operation( ).
    " Add an invalid data element
    DATA(lo_dtel_specification) = lo_put_operation->for-dtel-
>add_object( co_dtel_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_dtel_specification->set_short_description( 'My data element' ).
    lo_dtel_specification-
>set_data_type( xco_cp_abap_dictionary=>domain( co_doma_name ) ).
    " Add an invalid table type
    DATA(lo_ttyp_specification) = lo_put_operation->for-ttyp-
>add_object( co_ttyp_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_ttyp_specification->set_short_description( 'My table type' ).
    lo_ttyp_specification-
>set_row_type( xco_cp_abap_dictionary=>data_element( co_dtel_name ) ).
    TRY.
        lo_put_operation->execute( ).
      CATCH cx_xco_gen_put_exception INTO DATA(lx_put_exception).
        " Get all findings.
        out->plain->write( |Findings:| ).
        out->write_news( lx_put_exception ).
        " Get only the TTYP findings.
        out->plain->write( |Table type findings:| ).
        out->write_news( lx_put_exception->findings->for->ttyp ).
    ENDTRY.
  ENDMETHOD.
```

```
ENDCLASS.
```

4.2.15.18.3.3.4 Generating and Reading CDS Abstract Entities

The following code sample illustrates how abstract entities can be generated and read with the XCO Library:

```
"! <p class="shorttext synchronized" lang="EN">
"!  XCO Documentation CP: Abstract entities
"! </p>
"!
"! Code sample for generating and reading CDS abstract entities.
"!
"! IMPORTANT: The values of the constants CO_PACKAGE and CO_TRANSPORT must be
replaced
"! with an existing package and a modifiable Workbench transport matching the
transport
"! target of the package.
CLASS zcl_xco_doc_cp_cs_abs_ent DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.
  PRIVATE SECTION.
    CONSTANTS:
      co_package             TYPE sxco_package VALUE 'ZXCO_CP_PACKAGE',
      co_transport           TYPE sxco_transport VALUE 'X08K900164',
      co_abstract_entity_name TYPE sxco_ao_object_name VALUE 'ZXCO_DEMO_PRODUCT'.
    DATA:
      mo_environment TYPE REF TO if_xco_cp_gen_env_dev_system.
    METHODS:
      put_ddls.
ENDCLASS.
CLASS zcl_xco_doc_cp_cs_abs_ent IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).
    mo_environment = xco_cp_generation=>environment->dev_system( co_transport ).
  ENDMETHOD.
  METHOD main.
    put_ddls( ).
    out->write( |PUT operation executed successfully.| ).
    DATA(lo_abstract_entity) =
xco_cp_cds=>abstract_entity( co_abstract_entity_name ).
    DATA(ls_abstract_entity) = lo_abstract_entity->content( )->get( ).
    out->write( ls_abstract_entity ).
    LOOP AT lo_abstract_entity->fields->all->get( ) INTO DATA(lo_field).
      out->write( |Field { lo_field->name }:| ).
      DATA(ls_field) = lo_field->content( )->get( ).
      out->write( ls_field ).
    ENDLOOP.
  ENDMETHOD.
  METHOD put_ddls.
    DATA(lo_put_operation) = mo_environment->create_put_operation( ).
    DATA(lo_form_specification) = lo_put_operation->for-ddls-
>add_object( co_abstract_entity_name
      )->set_package( co_package
      )->create_form_specification( ).
```

```
    DATA(lo_abstract_entity) = lo_form_specification-
>set_short_description( 'Product'
      )->add_abstract_entity( ).
    DATA(lo_parameter) = lo_abstract_entity->add_parameter( 'p_language'
      )->set_data_type( xco_cp_abap_dictionary=>built_in_type->lang ).
    DATA(lo_id_field) = lo_abstract_entity-
>add_field( xco_cp_ddl=>field( 'ID' ) ).
    lo_id_field->set_key( ).
    lo_id_field->set_type( xco_cp_abap_dictionary=>built_in_type->char( 30 ) ).
    lo_id_field->add_annotation( 'UI.lineItem' )->value->build(
      )->begin_array(
        )->begin_record(
          )->add_member( 'position' )->add_number( 1
          )->add_member( 'label' )->add_string( 'Product'
        )->end_record(
      )->end_array( ).
    DATA(lo_description_field) = lo_abstract_entity-
>add_field( xco_cp_ddl=>field( 'description' ) ).
    lo_description_field->set_type( xco_cp_abap_dictionary=>built_in_type-
>char( 60 ) ).
    lo_description_field->add_annotation( 'UI.lineItem' )->value->build(
      )->begin_array(
        )->begin_record(
          )->add_member( 'position' )->add_number( 2
          )->add_member( 'label' )->add_string( 'Description'
        )->end_record(
      )->end_array( ).
    lo_put_operation->execute( ).
  ENDMETHOD.
ENDCLASS.
```

4.2.15.18.3.3.5 Generating and Reading Global Temporary Tables

The following code sample illustrates how global temporary tables can be generated and read with the XCO Library:

```
"! <p class="shorttext synchronized" lang="EN">
"!  XCO Documentation CP: Global temporary tables
"! </p>
"!
"! Code sample for generating and reading global temporary tables.
"!
"! IMPORTANT: The value of the constant CO_PACKAGE  must be replaced with an
existing
"! development package.
CLASS zcl_xco_doc_cp_cs_abs_ent DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.

  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.

  PRIVATE SECTION.
    CONSTANTS:
      co_package                   TYPE sxco_package VALUE 'ZMY_PACKAGE',

      co_global_temporary_table_name TYPE sxco_gtt_object_name VALUE 'ZXCO_GTT'.
```

```
    DATA:
      mo_global_temporary_table TYPE REF TO if_xco_global_temporary_table.

    METHODS:
      put_global_temporary_table
        IMPORTING
          out TYPE REF TO if_xco_adt_classrun_out,

      get_generation_environment
        IMPORTING
          out                               TYPE REF TO if_xco_adt_classrun_out
        RETURNING
          VALUE(ro_generation_environment) TYPE REF TO
if_xco_cp_gen_env_dev_system,

      get_transport_request
        IMPORTING
          out                         TYPE REF TO if_xco_adt_classrun_out
        RETURNING
          VALUE(ro_transport_request) TYPE REF TO if_xco_cp_tr_request.
ENDCLASS.

CLASS zcl_xco_doc_cp_cs_abs_ent IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).

    mo_global_temporary_table =
xco_cp_abap_dictionary=>global_temporary_table( co_global_temporary_table_name ).
  ENDMETHOD.

  METHOD main.
    put_global_temporary_table( out ).
    out->write( |PUT operation executed successfully.| ).

    DATA(ls_global_temporary_table) = mo_global_temporary_table->content( )-
>get( ).
    out->write( ls_global_temporary_table ).

    LOOP AT mo_global_temporary_table->fields->all->get( ) INTO DATA(lo_field).
      out->write( |Field { lo_field->name }:| ).

      DATA(ls_field) = lo_field->content( )->get( ).
      out->write( ls_field ).
    ENDLOOP.
  ENDMETHOD.

  METHOD put_global_temporary_table.
    DATA(lo_put_operation) = get_generation_environment( out )-
>create_put_operation( ).

    DATA(lo_form_specification) = lo_put_operation->for-tabl-for-
global_temporary_table->add_object( mo_global_temporary_table->name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_form_specification->set_short_description( 'Generated global temporary
table' ).

    lo_form_specification->add_field( 'KEY_FIELD'
      )->set_type( xco_cp_abap_dictionary=>built_in_type->char( 10 )
      )->set_key_indicator(
      )->set_not_null( ).

    lo_form_specification->add_field( 'DATA_FIELD'
      )->set_type( xco_cp_abap_dictionary=>built_in_type->char( 60 ) ).

    lo_put_operation->execute( ).
  ENDMETHOD.
```

```
  METHOD get_generation_environment.
    DATA(lo_transport_request) = get_transport_request( out ).

    ro_generation_environment = xco_cp_generation=>environment-
>dev_system( lo_transport_request->value ).
  ENDMETHOD.

  METHOD get_transport_request.
    DATA(lo_lock) = mo_global_temporary_table->if_xco_cts_changeable~get_object(
      )->get_lock( ).

    IF lo_lock->exists( ) EQ abap_true.
      DATA(lv_transport) = lo_lock->get_transport( ).

      ro_transport_request = xco_cp_cts=>transport->for( lv_transport
        )->get_request( ).

      out->write( |Using transport request { ro_transport_request->value }
(Global temporary table is already locked).| ).
    ELSE.
      DATA(lo_transport_target) = xco_cp_abap_repository=>package-
>for( co_package
        )->read(
        )-property-transport_layer->get_transport_target( ).

      ro_transport_request = xco_cp_cts=>transports-
>workbench( lo_transport_target->value
        )->create_request( 'Generated transport request' ).

      out->write( |Using newly generated transport request
{ ro_transport_request->value }.| ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.18.3.3.6 Logic Generation

The following code sample illustrates the generation of classes and interfaces:

Sample Code

```
"! <p class="shorttext synchronized" lang="EN">
"!  XCO Documentation CP: Logic generation
"! </p>
"!
"! Code sample for generating an interface along with a class that implements
it.
"!
"! IMPORTANT: The values of the constants CO_PACKAGE and CO_TRANSPORT must be
replaced
"! with an existing package and a modifiable Workbench transport matching the
transport
"! target of the package.
CLASS zcl_xco_doc_cp_cs_gen_logic DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.
```

```
  PRIVATE SECTION.
    CONSTANTS:
      co_package        TYPE sxco_package VALUE 'ZXCO_CP_PACKAGE',
      co_transport      TYPE sxco_transport VALUE 'X08K900164',
      co_interface_name TYPE sxco_ao_object_name VALUE 'ZIF_CUSTOMER',
      co_class_name     TYPE sxco_ad_object_name VALUE 'ZCL_CUSTOMER'.
    DATA:
      mo_environment TYPE REF TO if_xco_cp_gen_env_dev_system.
    METHODS:
      add_intf
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put,
      add_clas
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put.
ENDCLASS.
CLASS zcl_xco_doc_cp_cs_gen_logic IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).
    mo_environment = xco_cp_generation=>environment-
>dev_system( co_transport ).
  ENDMETHOD.
  METHOD main.
    DATA(lo_put_operation) = mo_environment->create_put_operation( ).
    add_intf( lo_put_operation ).
    add_clas( lo_put_operation ).
    lo_put_operation->execute( ).
    out->write( |PUT operation executed successfully.| ).
  ENDMETHOD.
  METHOD add_intf.
    DATA(lo_specification) = io_put_operation->for-intf-
>add_object( co_interface_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_specification->set_short_description( 'Customer' ).
    lo_specification->add_type( 'TV_PRODUCT' )->for( xco_cp_abap=>type-
built_in->c( 10 ) ).
    lo_specification->add_type( 'TT_PRODUCTS' )->for_table_type(
      io_row_type = lo_specification->own->type( 'TV_PRODUCT' )
      io_access   = xco_cp_table_type=>access->standard_table
    ).
    lo_specification->add_type( 'TV_ID' )->for( xco_cp_abap=>type-built_in-
>c( 10 ) ).
    lo_specification->add_data( 'ID' )->set_type( lo_specification->own-
>type( 'TV_ID' )
      )->set_read_only( ).
    DATA(lo_get_favorite_products) = lo_specification-
>add_method( 'GET_FAVORITE_PRODUCTS' ).
    lo_get_favorite_products->add_returning_parameter( 'RT_FAVORITE_PRODUCTS'
      )->set_type( lo_specification->own->type( 'TT_PRODUCTS' ) ).
  ENDMETHOD.
  METHOD add_clas.
    DATA(lo_specification) = io_put_operation->for-clas-
>add_object( co_class_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_specification->set_short_description( 'Customer implementation' ).
    lo_specification->definition-
>set_create_visibility( xco_cp_abap_objects=>visibility->private ).
    lo_specification->definition->add_interface( co_interface_name ).
    lo_specification->definition->section-public->add_alias( 'TV_ID'
      )->set_component( 'TV_ID'
      )->set_interface( co_interface_name ).
    " CREATE factory method.
    DATA(lo_create) = lo_specification->definition->section-public-
>add_class_method( 'CREATE' ).
    lo_create->add_importing_parameter( 'IV_ID' )->set_type( lo_specification-
>own->type( 'TV_ID' ) ).
```

```
    lo_create->add_returning_parameter( 'RO_INSTANCE' )-
>set_type( xco_cp_abap=>interface( co_interface_name ) ).
    lo_specification->implementation->add_method( 'CREATE' )-
>set_source( VALUE #(
      ( |ro_instance = new { co_class_name }( iv_id ).| )
    ) ).
    " CONSTRUCTOR.
    DATA(lo_constructor) = lo_specification->definition->section-private-
>add_method( 'CONSTRUCTOR' ).
    lo_constructor->add_importing_parameter( 'IV_ID' )-
>set_type( lo_specification->own->type( 'TV_ID' ) ).
    lo_specification->implementation->add_method( 'CONSTRUCTOR' )-
>set_source( VALUE #(
      ( |{ co_interface_name }~id = iv_id.| )
    ) ).
    " Implementation of GET_FAVORITE_PRODUCTS interface method.
    lo_specification->implementation->add_method( |
{ co_interface_name }~GET_FAVORITE_PRODUCTS|
      )->set_source( VALUE #(
        ( |" Source code goes here...| )
    ) ).
  ENDMETHOD.
ENDCLASS.
```

Note that the XCO Generation APIs for classes also allow the specification of local types and local test classes:

- Method `ADD_LOCAL_CLASS` on `IF_XCO_CP_GEN_CLAS_S_FORM` can be used to add a local type to the generated class. The content of the local class can be fully specified via the returned local class specification `IF_XCO_GEN_CLAS_S_FO_LCL_CLASS`
- Method `ADD_TEST_CLASS` on `IF_XCO_CP_GEN_CLAS_S_FORM` can be used to add a local test class to the generated class. The content of the local test class can be fully specified via the returned test class specification `IF_XCO_GEN_CLAS_S_FO_TST_CLASS`. Note that the API `XCO_CP_ABAP_UNIT` offers XCO enumeration constants for ABAP Unit durations and risk levels

4.2.15.18.3.3.7 Patching a Message Class

The following code sample illustrates how PATCH operations can be used to change parts of an existing messages class:

Sample Code

```
"! <p class="shorttext synchronized" lang="EN">
"!  XCO Documentation CP: Generation PATCH
"! </p>
"!
"! Code sample for executing PATCH operations for messages classes (MSAGs).
"!
"! IMPORTANT: The values of the constants CO_PACKAGE and CO_TRANSPORT must be
replaced
"! with an existing package and a modifiable Workbench transport matching the
transport
"! target of the package.
CLASS zcl_xco_doc_cp_cs_gen_patch DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.
```

```
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.

  PRIVATE SECTION.
    CONSTANTS:
      co_package            TYPE sxco_package VALUE 'ZXCO_CP_PACKAGE',
      co_transport          TYPE sxco_transport VALUE 'X08K900164',

      co_message_class_name TYPE sxco_mc_object_name VALUE
'ZXCO_DEMO_MSG_CLS'.

    DATA:
      mo_environment TYPE REF TO if_xco_cp_gen_env_dev_system.

    METHODS:
      put_msag,

      patch_msag_1,

      patch_msag_2,

      read_msag
        IMPORTING
          out TYPE REF TO if_xco_adt_classrun_out.
ENDCLASS.

CLASS zcl_xco_doc_cp_cs_gen_patch IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).

    mo_environment = xco_cp_generation=>environment-
>dev_system( co_transport ).
  ENDMETHOD.

  METHOD main.
    put_msag( ).
    read_msag( out ).

    patch_msag_1( ).
    read_msag( out ).

    patch_msag_2( ).
    read_msag( out ).
  ENDMETHOD.

  METHOD put_msag.
    DATA(lo_put_operation) = mo_environment->for-msag-
>create_put_operation( ).

    DATA(lo_specification) = lo_put_operation-
>add_object( co_message_class_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_specification->set_short_description( 'Generated message class' ).

    lo_specification->add_message( '000'
      )->set_short_text( 'First message' ).

    lo_put_operation->execute( ).
  ENDMETHOD.

  METHOD patch_msag_1.
    DATA(lo_patch_operation) = mo_environment->for-msag-
>create_patch_operation( ).

    DATA(lo_object) = lo_patch_operation->add_object( co_message_class_name ).
```

```
    " Update existing message.
    lo_object->for-update->message( '000' )->set_short_text( 'First message
(UPDATED)' ).

    " Insert new message.
    lo_object->for-insert->add_message( '001' )->set_short_text( 'Second
message' ).

    " Modify non-existing message (will insert the message).
    lo_object->for-modify->add_message( '002' )->set_short_text( 'Third
message' ).

    lo_patch_operation->execute( ).
  ENDMETHOD.

  METHOD patch_msag_2.
    DATA(lo_patch_operation) = mo_environment->for-msag-
>create_patch_operation( ).

    DATA(lo_object) = lo_patch_operation->add_object( co_message_class_name ).

    " Modify non-existing message (will update the message).
    lo_object->for-modify->add_message( '002' )->set_short_text( 'Third
message (UPDATED)' ).

    " Delete message
    lo_object->for-delete->add_message( '000' ).

    lo_patch_operation->execute( ).
  ENDMETHOD.

  METHOD read_msag.
    DATA(lo_message_class) = xco_cp_abap_repository=>object->msag-
>for( co_message_class_name ).

    out->write( |Messages of message class { lo_message_class->name }:| ).

    LOOP AT lo_message_class->messages->all->get( ) INTO DATA(lo_message).
      out->write( |Message { lo_message->number }:| ).

      DATA(ls_message) = lo_message->content( )->get( ).
      out->write( ls_message ).
    ENDLOOP.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.18.3.3.8 Patching Methods of Interfaces and Classes

The XCO Generation APIs allow to incrementally modify existing interfaces and classes through `PATCH` operations.

The following two code samples illustrate how methods of interfaces and classes can be patched. In each code sample, the `MAIN` method will execute one of four phases depending on the current state of the interface or class. To see each phase in action simply execute each class several times in a row.

Here is how it looks for interfaces:

{·} Sample Code

```
"! Code sample for patching methods of an interface.
```

```
"!
"! IMPORTANT: The value of the constant CO_PACKAGE  must be replaced with an
existing
"! development package.
CLASS zcl_xco_doc_cp_cs_patch_in_me DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.

  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.

  PRIVATE SECTION.
    CONSTANTS:
      co_package       TYPE sxco_package VALUE 'ZMY_PACKAGE',

      co_interface_name TYPE sxco_ao_object_name VALUE
'ZIF_XCO_DMO_GEN_INTF_PATCH_MTD',

      co_first_method  TYPE sxco_ao_component_name VALUE 'FIRST_METHOD',
      co_second_method TYPE sxco_ao_component_name VALUE 'SECOND_METHOD',
      co_third_method  TYPE sxco_ao_component_name VALUE 'THIRD_METHOD'.

    DATA:
      mo_interface TYPE REF TO if_xco_ao_interface.

    METHODS:
      phase_1
        IMPORTING
          out TYPE REF TO if_xco_adt_classrun_out,

      phase_2
        IMPORTING
          out TYPE REF TO if_xco_adt_classrun_out,

      phase_3
        IMPORTING
          out TYPE REF TO if_xco_adt_classrun_out,

      phase_4
        IMPORTING
          out TYPE REF TO if_xco_adt_classrun_out,

      get_environment
        IMPORTING
          out                  TYPE REF TO if_xco_adt_classrun_out
        RETURNING
          VALUE(ro_environment) TYPE REF TO if_xco_cp_gen_env_dev_system,

      get_transport_request
        IMPORTING
          out                          TYPE REF TO if_xco_adt_classrun_out
        RETURNING
          VALUE(ro_transport_request) TYPE REF TO if_xco_cp_tr_request.
ENDCLASS.

CLASS zcl_xco_doc_cp_cs_patch_in_me IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).

    mo_interface = xco_cp_abap=>interface( co_interface_name ).
  ENDMETHOD.

  METHOD main.
    IF mo_interface->exists( ) EQ abap_false.
      " In the first phase the interface is created via a PUT operation.
```

```
      phase_1( out ).

      out->write( |Executed phase 1| ).
    ELSEIF mo_interface->component->method( 'FIRST_METHOD' )->exists( ) EQ
abap_false.
      " In the second phase two new methods are inserted into the interface
      " via a single PATCH operation.
      phase_2( out ).

      out->write( |Executed phase 2| ).
    ELSEIF mo_interface->component->method( 'SECOND_METHOD' )->exists( ) EQ
abap_true.
      " In the third phase one method is deleted, a new method is inserted and
      " an existing method is updated via a single PATCH operation.
      phase_3( out ).

      out->write( |Executed phase 3| ).
    ELSE.
      " In the fourth phase the interface is deleted via a DELETE operation.
      phase_4( out ).

      out->write( |Executed phase 4| ).
    ENDIF.
  ENDMETHOD.

  METHOD phase_1.
    DATA(lo_put_operation) = get_environment( out )->create_put_operation( ).

    DATA(lo_specification) = lo_put_operation->for-intf-
>add_object( mo_interface->name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_specification->set_short_description( 'My interface' ).

    " Data
    lo_specification->add_data( 'ID'
      )->set_type( xco_cp_abap=>type-built_in->i
      )->set_read_only( ).

    lo_put_operation->execute( ).
  ENDMETHOD.

  METHOD phase_2.
    DATA(lo_patch_operation) = get_environment( out )->for-intf-
>create_patch_operation( ).

    DATA(lo_patch_operation_object) = lo_patch_operation-
>add_object( mo_interface->name ).

    " Insert first method
    DATA(lo_first_method) = lo_patch_operation_object->for-insert-
>add_method( co_first_method
      )->set_default_ignore( ).

    lo_first_method->add_importing_parameter( 'IMP_1'
      )->set_type( xco_cp_abap=>type-built_in->string ).
    lo_first_method->add_importing_parameter( 'IMP_2'
      )->set_type( xco_cp_abap=>type-built_in->string ).
    lo_first_method->add_changing_parameter( 'CHA_1'
      )->set_type( xco_cp_abap=>interface( 'IF_XCO_NEWS' ) ).

    lo_first_method->add_exception( 'CX_ABAP_INVALID_VALUE' ).

    " Insert second method
    DATA(lo_second_method) = lo_patch_operation_object->for-insert-
>add_method( co_second_method
      )->set_default_fail( ).
```

```
    lo_patch_operation->execute( ).
  ENDMETHOD.

  METHOD phase_3.
    DATA(lo_patch_operation) = get_environment( out )->for-intf-
>create_patch_operation( ).

    DATA(lo_patch_operation_object) = lo_patch_operation-
>add_object( mo_interface->name ).

    " Update first method.
    DATA(lo_first_method) = lo_patch_operation_object->for-update-
>add_method( co_first_method
      )->set_default_fail( ).

    lo_first_method->for-update->add_importing_parameter( 'IMP_1'
      )->set_pass_by_value(
      )->set_optional( ).

    lo_first_method->for-delete->add_importing_parameter( 'IMP_2' ).

    lo_first_method->for-insert->add_importing_parameter( 'IMP_3'
      )-
>set_type( xco_cp_abap_dictionary=>data_element( 'SXCO_AO_OBJECT_NAME' ) ).

    lo_first_method->for-update->add_exception( 'CX_ABAP_INVALID_VALUE'
      )->set_resumable( ).

    " Delete second method.
    lo_patch_operation_object->for-delete->add_method( co_second_method ).

    " Insert third method.
    lo_patch_operation_object->for-insert->add_method( co_third_method
      )->set_default_fail( ).

    lo_patch_operation->execute( ).
  ENDMETHOD.

  METHOD phase_4.
    DATA(lo_delete_operation) = get_environment( out )->for-intf-
>create_delete_operation( ).
    lo_delete_operation->add_object( mo_interface->name ).
    lo_delete_operation->execute( ).
  ENDMETHOD.

  METHOD get_environment.
    DATA(lo_transport_request) = get_transport_request( out ).

    ro_environment = xco_cp_generation=>environment-
>dev_system( lo_transport_request->value ).
  ENDMETHOD.

  METHOD get_transport_request.
    DATA(lo_lock) = mo_interface->if_xco_cts_changeable~get_object(
      )->get_lock( ).

    IF lo_lock->exists( ) EQ abap_true.
      DATA(lv_transport) = lo_lock->get_transport( ).

      ro_transport_request = xco_cp_cts=>transport->for( lv_transport
        )->get_request( ).

      out->write( |Using transport request { ro_transport_request->value }
(Interface is already locked).| ).
    ELSE.
      DATA(lo_transport_target) = xco_cp_abap_repository=>package-
>for( co_package
        )->read(
```

```
        )->property-transport_layer->get_transport_target( ).

      ro_transport_request = xco_cp_cts=>transports-
>workbench( lo_transport_target->value
        )->create_request( 'Generated transport request' ).

      out->write( |Using newly generated transport request
{ ro_transport_request->value }.| ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

and here is how it looks for classes:

Sample Code

```
"! Code sample for patching methods of a class.
"!
"! IMPORTANT: The value of the constant CO_PACKAGE  must be replaced with an
existing
"! development package.
CLASS zcl_xco_doc_cp_cs_patch_cl_me DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.
  PRIVATE SECTION.
    CONSTANTS:
      co_package       TYPE sxco_package VALUE 'ZMY_PACKAGE',
      co_class_name    TYPE sxco_ao_object_name VALUE
'ZCL_XCO_DMO_GEN_CLAS_PATCH_MTD',
      co_first_method  TYPE sxco_clas_method_name VALUE 'FIRST_METHOD',
      co_second_method TYPE sxco_clas_method_name VALUE 'SECOND_METHOD',
      co_third_method  TYPE sxco_clas_method_name VALUE 'THIRD_METHOD'.
    DATA:
      mo_class TYPE REF TO if_xco_ao_class.
    METHODS:
      phase_1
        IMPORTING
          out TYPE REF TO if_xco_adt_classrun_out,
      phase_2
        IMPORTING
          out TYPE REF TO if_xco_adt_classrun_out,
      phase_3
        IMPORTING
          out TYPE REF TO if_xco_adt_classrun_out,
      phase_4
        IMPORTING
          out TYPE REF TO if_xco_adt_classrun_out,
      get_environment
        IMPORTING
          out                   TYPE REF TO if_xco_adt_classrun_out
        RETURNING
          VALUE(ro_environment) TYPE REF TO if_xco_cp_gen_env_dev_system,
      get_transport_request
        IMPORTING
          out                           TYPE REF TO if_xco_adt_classrun_out
        RETURNING
          VALUE(ro_transport_request) TYPE REF TO if_xco_cp_tr_request.
ENDCLASS.
CLASS zcl_xco_doc_cp_cs_patch_cl_me IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).
```

```
    mo_class = xco_cp_abap=>class( co_class_name ).
  ENDMETHOD.
  METHOD main.
    IF mo_class->exists( ) EQ abap_false.
      " In the first phase the class is created via a PUT operation.
      phase_1( out ).
      out->write( |Executed phase 1| ).
    ELSEIF mo_class->definition->section-public->component-
>method( 'FIRST_METHOD' )->exists( ) EQ abap_false.
      " In the second phase two new methods are inserted into the class, one
in the public section
      " and one in the private section, via a single PATCH operation.
      phase_2( out ).
      out->write( |Executed phase 2| ).
    ELSEIF mo_class->definition->section-private->component-
>method( 'SECOND_METHOD' )->exists( ) EQ abap_true.
      " In the third phase one method is deleted, a new method is inserted
into the protected
      " section and an existing method is updated via a single PATCH
operation.
      phase_3( out ).
      out->write( |Executed phase 3| ).
    ELSE.
      " In the fourth phase the class is deleted via a DELETE operation.
      phase_4( out ).
      out->write( |Executed phase 4| ).
    ENDIF.
  ENDMETHOD.
  METHOD phase_1.
    DATA(lo_put_operation) = get_environment( out )->create_put_operation( ).
    DATA(lo_specification) = lo_put_operation->for-clas->add_object( mo_class-
>name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_specification->set_short_description( 'My class' ).
    " Data
    lo_specification->definition->section-public->add_data( 'ID'
      )->set_type( xco_cp_abap=>type-built_in->i
      )->set_read_only( ).
    lo_put_operation->execute( ).
  ENDMETHOD.
  METHOD phase_2.
    DATA(lo_patch_operation) = get_environment( out )->for-clas-
>create_patch_operation( ).
    DATA(lo_patch_operation_object) = lo_patch_operation-
>add_object( mo_class->name ).
    " First we specify the definition part of the class.
    " Insert the first method into the public section of the class.
    DATA(lo_first_method) = lo_patch_operation_object->for-insert->definition-
>section-public->add_method( co_first_method ).
    lo_first_method->add_importing_parameter( 'IMP_1'
      )->set_type( xco_cp_abap=>type-built_in->string ).
    lo_first_method->add_importing_parameter( 'IMP_2'
      )->set_type( xco_cp_abap=>type-built_in->string ).
    lo_first_method->add_changing_parameter( 'CHA_1'
      )->set_type( xco_cp_abap=>interface( 'IF_XCO_NEWS' ) ).
    lo_first_method->add_exception( 'CX_ABAP_INVALID_VALUE' ).
    " Insert the second method into the private section of the class.
    DATA(lo_second_method) = lo_patch_operation_object->for-insert-
>definition->section-private->add_method( co_second_method ).
    " After the definition has been specified we also specify the
implementation part. If no implementation
    " is specified for an inserted method it will have an empty
implementation (as is the case for
    " the second method).
    lo_patch_operation_object->for-insert->implementation-
>add_method( co_first_method
      )->set_source( VALUE #(
```

```
        ( `  " Source code goes here` )
      ) ).
    lo_patch_operation->execute( ).
  ENDMETHOD.
  METHOD phase_3.
    DATA(lo_patch_operation) = get_environment( out )->for-clas-
>create_patch_operation( ).
    DATA(lo_patch_operation_object) = lo_patch_operation-
>add_object( mo_class->name ).
    " As in phase 2, we first specify the definition part of the class.
    " Update first method.
    DATA(lo_first_method) = lo_patch_operation_object->for-update->definition-
>section-public->add_method( co_first_method ).
    lo_first_method->for-update->add_importing_parameter( 'IMP_1'
      )->set_pass_by_value(
      )->set_optional( ).
    lo_first_method->for-delete->add_importing_parameter( 'IMP_2' ).
    lo_first_method->for-insert->add_importing_parameter( 'IMP_3'
      )-
>set_type( xco_cp_abap_dictionary=>data_element( 'SXCO_AO_OBJECT_NAME' ) ).
    lo_first_method->for-update->add_exception( 'CX_ABAP_INVALID_VALUE'
      )->set_resumable( ).
    " Delete second method.
    lo_patch_operation_object->for-delete->definition->section-private-
>add_method( co_second_method ).
    " Insert the third method into the protected section of the class.
    lo_patch_operation_object->for-insert->definition->section-protected-
>add_method( co_third_method ).
    " Now the implementation part of the class is specified.
    lo_patch_operation_object->for-update->implementation-
>add_method( co_first_method
      )->set_source( VALUE #(
        ( `  RAISE EXCEPTION TYPE cx_abap_invalid_value.` )
    ) ).
    lo_patch_operation_object->for-insert->implementation-
>add_method( co_third_method
      )->set_source( VALUE #(
        ( `  " Third method implementation` )
      ) ).
    lo_patch_operation->execute( ).
  ENDMETHOD.
  METHOD phase_4.
    DATA(lo_delete_operation) = get_environment( out )->for-clas-
>create_delete_operation( ).
    lo_delete_operation->add_object( mo_class->name ).
    lo_delete_operation->execute( ).
  ENDMETHOD.
  METHOD get_environment.
    DATA(lo_transport_request) = get_transport_request( out ).
    ro_environment = xco_cp_generation=>environment-
>dev_system( lo_transport_request->value ).
  ENDMETHOD.
  METHOD get_transport_request.
    DATA(lo_lock) = mo_class->if_xco_cts_changeable~get_object(
      )->get_lock( ).
    IF lo_lock->exists( ) EQ abap_true.
      DATA(lv_transport) = lo_lock->get_transport( ).
      ro_transport_request = xco_cp_cts=>transport->for( lv_transport
        )->get_request( ).
      out->write( |Using transport request { ro_transport_request->value }
(Class is already locked).| ).
    ELSE.
      DATA(lo_transport_target) = xco_cp_abap_repository=>package-
>for( co_package
        )->read(
        )-property-transport_layer->get_transport_target( ).
      ro_transport_request = xco_cp_cts=>transports-
>workbench( lo_transport_target->value
```

```
        )->create_request( 'Generated transport request' ).
      out->write( |Using newly generated transport request
{ ro_transport_request->value }.| ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.18.3.3.9 Patching Attributes of Interfaces and Classes

The code samples illustrate how `PATCH` operations can be used to modify attributes of interfaces and classes, with XCO Read APIs enabling conditional modifications based on the current state of the objects.

In the first example an attribute is inserted into an interface if it's not yet part of it and deleted if it's already a part of the interface. By running the class several times in a row the attribute will be continuously inserted and deleted.

Sample Code

```
"! Code sample for using the interface Read APIs together with the INTF PATCH
operation
"! functionality to "toggle" a DATA attribute on an interface.
"!
"! IMPORTANT: The interface referred to by the constant CO_INTERFACE_NAME
needs to
"! exist prior to executing this code sample.
CLASS zcl_xco_doc_cp_cs_chg_intf DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.

  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.

  PRIVATE SECTION.
    CONSTANTS:
      co_interface_name TYPE sxco_ao_object_name VALUE 'ZIF_INTERFACE',

      co_numeric_id    TYPE sxco_ao_component_name VALUE 'NUMERIC_ID'.

    METHODS:
      get_transport_request
        IMPORTING
          io_interface              TYPE REF TO if_xco_ao_interface
          out                       TYPE REF TO if_xco_adt_classrun_out
        RETURNING
          VALUE(ro_transport_request) TYPE REF TO if_xco_cp_tr_request.
ENDCLASS.

CLASS zcl_xco_doc_cp_cs_chg_intf IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).
  ENDMETHOD.

  METHOD main.
    DATA(lo_interface) = xco_cp_abap=>interface( co_interface_name ).

    DATA(lo_transport_request) = get_transport_request(
```

```
      io_interface = lo_interface
      out          = out
    ).

    DATA(lo_environment) = xco_cp_generation=>environment-
>dev_system( lo_transport_request->value ).

    DATA(lo_patch_operation) = lo_environment->for-intf-
>create_patch_operation( ).
    DATA(lo_object) = lo_patch_operation->add_object( co_interface_name ).

    IF lo_interface->component->data( co_numeric_id )->exists( ) EQ
abap_false.
      lo_object->for-insert->add_data( co_numeric_id
        )->set_type( xco_cp_abap=>type-built_in->int8
        )->set_read_only( ).

      out->write( |Inserting DATA { co_numeric_id }| ).
    ELSE.
      lo_object->for-delete->add_data( co_numeric_id ).

      out->write( |Deleting DATA { co_numeric_id }| ).
    ENDIF.

    lo_patch_operation->execute( ).
  ENDMETHOD.

  METHOD get_transport_request.
    DATA(lo_lock) = io_interface->if_xco_cts_changeable~get_object(
      )->get_lock( ).

    IF lo_lock->exists( ) EQ abap_true.
      DATA(lv_transport) = lo_lock->get_transport( ).

      ro_transport_request = xco_cp_cts=>transport->for( lv_transport
        )->get_request( ).

      out->write( |Using transport request { ro_transport_request->value }
(Interface is already locked).| ).
    ELSE.
      DATA(lo_transport_target) = io_interface->if_xco_ar_object~get_package(
        )->read(
        )-property-transport_layer->get_transport_target( ).

      ro_transport_request = xco_cp_cts=>transports-
>workbench( lo_transport_target->value
        )->create_request( 'Generated transport request' ).

      out->write( |Using newly generated transport request
{ ro_transport_request->value }.| ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

The second example "moves" an attribute through all three sections of a class. By running the below class several times in a row the attribute will first move from the public section to the protected section, then to the private section and then back to the public section to repeat the cycle:

Sample Code

```
"! Code sample for using the class Read APIs together with the CLAS PATCH
operation
"! functionality to "move" a DATA attribute across the sections of the class.
"!
"! IMPORTANT: The class referred to by the constant CO_CLASS_NAME needs to
```

```
"! exist prior to executing this code sample.
CLASS zcl_xco_doc_cp_cs_chg_clas DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.
  PRIVATE SECTION.
    TYPES:
      BEGIN OF ts_section,
        name             TYPE string,
        class_definition TYPE REF TO if_xco_clas_definition_section,
        for_delete       TYPE REF TO if_xco_gen_clas_s_de_d_section,
        for_insert       TYPE REF TO if_xco_gen_clas_s_in_d_section,
      END OF ts_section,
      tt_section TYPE STANDARD TABLE OF ts_section WITH EMPTY KEY.
    CONSTANTS:
      co_class_name TYPE sxco_ao_object_name VALUE 'ZCL_CLASS',
      co_numeric_id TYPE sxco_ao_component_name VALUE 'NUMERIC_ID'.
    DATA:
      mo_class TYPE REF TO if_xco_ao_class.
    METHODS:
      get_generation_environment
        IMPORTING
          out                                TYPE REF TO if_xco_adt_classrun_out
        RETURNING
          VALUE(ro_generation_environment) TYPE REF TO
if_xco_cp_gen_env_dev_system,
      get_transport_request
        IMPORTING
          out                          TYPE REF TO if_xco_adt_classrun_out
        RETURNING
          VALUE(ro_transport_request) TYPE REF TO if_xco_cp_tr_request,
      get_sections
        IMPORTING
          io_object          TYPE REF TO if_xco_cp_gen_clas_d_o_pat_obj
        RETURNING
          VALUE(rt_sections) TYPE tt_section.
ENDCLASS.
CLASS zcl_xco_doc_cp_cs_chg_clas IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).
    mo_class = xco_cp_abap=>class( co_class_name ).
  ENDMETHOD.
  METHOD main.
    DATA(lo_patch_operation) = get_generation_environment( out )->for-clas-
>create_patch_operation( ).
    DATA(lo_object) = lo_patch_operation->add_object( co_class_name ).
    DATA(lt_sections) = get_sections( lo_object ).
    LOOP AT lt_sections INTO DATA(ls_section).
      DATA(lv_current_section) = sy-tabix.
      IF ls_section-class_definition->component->data( co_numeric_id )-
>exists( ) EQ abap_true.
        ls_section-for_delete->add_data( co_numeric_id ).
        out->write( |Deleting DATA { co_numeric_id } from section
{ ls_section-name }.| ).
        EXIT.
      ENDIF.
    ENDLOOP.
    " Insert attribute to next section.
    DATA(ls_next_section) = lt_sections[ lv_current_section MOD 3 + 1 ].
    ls_next_section-for_insert->add_data( co_numeric_id
      )->set_type( xco_cp_abap=>type-built_in->int8 ).
    out->write( |Inserting DATA { co_numeric_id } into section
{ ls_next_section-name }.| ).
    lo_patch_operation->execute( ).
```

```
  ENDMETHOD.
  METHOD get_generation_environment.
    DATA(lo_transport_request) = get_transport_request( out ).
    ro_generation_environment = xco_cp_generation=>environment-
>dev_system( lo_transport_request->value ).
  ENDMETHOD.
  METHOD get_transport_request.
    DATA(lo_lock) = mo_class->if_xco_cts_changeable~get_object(
      )->get_lock( ).
    IF lo_lock->exists( ) EQ abap_true.
      DATA(lv_transport) = lo_lock->get_transport( ).
      ro_transport_request = xco_cp_cts=>transport->for( lv_transport
        )->get_request( ).
      out->write( |Using transport request { ro_transport_request->value }
(Class is already locked).| ).
    ELSE.
      DATA(lo_transport_target) = mo_class->if_xco_ar_object~get_package(
        )->read(
        )-property-transport_layer->get_transport_target( ).
      ro_transport_request = xco_cp_cts=>transports-
>workbench( lo_transport_target->value
        )->create_request( 'Generated transport request' ).
      out->write( |Using newly generated transport request
{ ro_transport_request->value }.| ).
    ENDIF.
  ENDMETHOD.
  METHOD get_sections.
    DATA(ls_public_section) = VALUE ts_section(
      name             = `PUBLIC`
      class_definition = mo_class->definition->section-public
      for_delete       = io_object->for-delete->definition->section-public
      for_insert       = io_object->for-insert->definition->section-public
    ).
    APPEND ls_public_section TO rt_sections.
    DATA(ls_protected_section) = VALUE ts_section(
      name             = `PROTECTED`
      class_definition = mo_class->definition->section-protected
      for_delete       = io_object->for-delete->definition->section-protected
      for_insert       = io_object->for-insert->definition->section-protected
    ).
    APPEND ls_protected_section TO rt_sections.
    DATA(ls_private_section) = VALUE ts_section(
      name             = `PRIVATE`
      class_definition = mo_class->definition->section-private
      for_delete       = io_object->for-delete->definition->section-private
      for_insert       = io_object->for-insert->definition->section-private
    ).
    APPEND ls_private_section TO rt_sections.
  ENDMETHOD.
ENDCLASS.
```

4.2.15.18.3.3.10 Function Module Generation

The following code sample illustrates the generation of function modules:

Sample Code

```
"! Code sample for generating a function module along with a function group
"! that contains it.
"!
```

```
"! IMPORTANT: The values of the constants CO_PACKAGE and CO_TRANSPORT must be
replaced
"! with an existing package and a modifiable Workbench transport request
matching the
"! transport target of the package.
CLASS zcl_xco_doc_cp_cs_gen_fm DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.
  PRIVATE SECTION.
    CONSTANTS:
      co_package              TYPE sxco_package VALUE 'ZXCO_CP_PACKAGE',
      co_transport            TYPE sxco_transport VALUE 'X08K900164',
      co_function_group_name  TYPE sxco_fg_object_name VALUE
'ZXCO_GEN_FUGR_FG',
      co_function_module_name TYPE sxco_fm_name VALUE 'ZXCO_GEN_FUGR_FM'.
    DATA:
      mo_function_group  TYPE REF TO if_xco_function_group,
      mo_function_module TYPE REF TO if_xco_function_module,
      mo_environment     TYPE REF TO if_xco_cp_gen_env_dev_system.
    METHODS:
      set_up_function_group
        IMPORTING
          out TYPE REF TO if_xco_adt_classrun_out,
      post_function_group,
      delete_function_modules
        IMPORTING
          out TYPE REF TO if_xco_adt_classrun_out,
      insert_function_modules
        IMPORTING
          out TYPE REF TO if_xco_adt_classrun_out.
ENDCLASS.
CLASS zcl_xco_doc_cp_cs_gen_fm IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).
    mo_function_group = xco_cp_abap=>function_group( co_function_group_name ).
    mo_function_module =
xco_cp_abap=>function_module( co_function_module_name ).
    mo_environment = xco_cp_generation=>environment-
>dev_system( co_transport ).
  ENDMETHOD.
  METHOD main.
    set_up_function_group( out ).
    delete_function_modules( out ).
    insert_function_modules( out ).
  ENDMETHOD.
  METHOD set_up_function_group.
    IF mo_function_group->if_xco_ar_object~exists( ) EQ abap_true.
      out->write( |Function group { mo_function_group->name } already
exists.| ).
    ELSE.
      post_function_group( ).
      out->write( |Function group { mo_function_group->name } created
successfully.| ).
    ENDIF.
  ENDMETHOD.
  METHOD post_function_group.
    DATA(lo_post_operation) = mo_environment->for-fugr-
>create_post_operation( ).
    DATA(lo_form_specification) = lo_post_operation-
>add_object( xco_cp_name=>choice->fixed( co_function_group_name )
      )->set_package( co_package
      )->create_form_specification( ).
    lo_form_specification->set_short_description( 'Generated FUGR' ).
```

```
    lo_post_operation->execute( ).
  ENDMETHOD.
  METHOD delete_function_modules.
    DATA(lo_patch_operation) = mo_environment->for-fugr-
>create_patch_operation( ).
    DATA(lt_function_module_names) = mo_function_group->function_modules->all-
>get_names( ).
    LOOP AT lt_function_module_names INTO DATA(lv_function_module_name).
      lo_patch_operation->add_object( co_function_group_name
        )->for-delete->add_function_module( lv_function_module_name ).
    ENDLOOP.
    lo_patch_operation->execute( ).
    out->write( |Function modules of function group { mo_function_group-
>name } deleted succesfully.| ).
  ENDMETHOD.
  METHOD insert_function_modules.
    DATA(lo_patch_operation) = mo_environment->for-fugr-
>create_patch_operation( ).
    DATA(lo_function_module) = lo_patch_operation-
>add_object( co_function_group_name
      )->for-insert->add_function_module( co_function_module_name ).
    " IMPORTING parameters.
    lo_function_module->add_importing_parameter( 'IM_CLASS'
      )->set_type( xco_cp_abap=>class( 'CL_XCO_AD_BUILT_IN_TYPE' )
      )->set_optional( ).
    lo_function_module->add_importing_parameter( 'IM_INTERFACE'
      )->set_type( xco_cp_abap=>interface( 'IF_XCO_NEWS' ) ).
    lo_function_module->add_importing_parameter( 'IM_ABAP_BUILT_IN_TYPE'
      )->set_type( xco_cp_abap=>type-built_in->i
      )->set_default_value( '5' ).
    lo_function_module->add_importing_parameter( 'IM_ABAP_BIT_REFERENCE'
      )->set_type( xco_cp_abap=>type-built_in->i->reference( ) ).
    lo_function_module->add_importing_parameter( 'IM_ABAP_GENERIC_TYPE'
      )->set_type( xco_cp_abap=>type-generic->c ).
    lo_function_module->add_importing_parameter( 'IM_ABAP_GT_REFERENCE'
      )->set_type( xco_cp_abap=>type-generic->data->reference( ) ).
    lo_function_module->add_importing_parameter( 'IM_DATA_ELEMENT'
      )->set_type( xco_cp_abap_dictionary=>data_element( 'SXCO_FM_NAME' ) ).
    " EXPORTING parameters.
    lo_function_module->add_exporting_parameter( 'EX_STRING'
      )->set_type( xco_cp_abap=>type-built_in->string ).
    " CHANGING parameters.
    lo_function_module->add_exporting_parameter( 'CH_ANY'
      )->set_type( xco_cp_abap=>type-generic->any ).
    " Exceptions.
    lo_function_module->add_exception( 'CX_ABAP_INVALID_NAME' ).
    lo_function_module->add_exception( 'CX_ABAP_INVALID_VALUE'
      )->set_resumable( abap_true ).
    " Source code.
    lo_function_module->set_source_code( xco_cp=>strings( VALUE #(
      ( |DATA lo_string TYPE REF TO if_xco_string.| )
    ) ) ).
    lo_patch_operation->execute( ).
    out->write( |Inserted function module { mo_function_module->name } into
function group { mo_function_group->name }.| ).
  ENDMETHOD.
ENDCLASS.
```

4.2.15.18.3.3.11 Updating the Source Code of a Function Module

The following code sample illustrates the source code of an already existing function module that can be updated:

Sample Code

```
"! Code sample for updating the source code of an existing function module.
"!
"! IMPORTANT: The function module referred to by the constant
CO_FUNCTION_MODULE_NAME needs to
"! exist prior to executing this code sample and must not already be locked
on a transport
"! request.
CLASS zcl_xco_doc_cp_cs_chg_fm DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS:
      constructor.

  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.

  PRIVATE SECTION.
    CONSTANTS:
      co_function_module_name TYPE sxco_fm_name VALUE 'ZMY_FUNCTION_MODULE'.

    DATA:
      mo_function_module TYPE REF TO if_xco_function_module.

    METHODS:
      generate_transport_request
        IMPORTING
          out                         TYPE REF TO if_xco_adt_classrun_out
        RETURNING
          VALUE(ro_transport_request) TYPE REF TO if_xco_cp_tr_request,

      release_transport_request
        IMPORTING
          io_transport_request TYPE REF TO if_xco_cp_tr_request
          out                  TYPE REF TO if_xco_adt_classrun_out.
ENDCLASS.

CLASS zcl_xco_doc_cp_cs_chg_fm IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).

    mo_function_module =
xco_cp_abap=>function_module( co_function_module_name ).
  ENDMETHOD.

  METHOD main.
    DATA(lo_transport_request) = generate_transport_request( out ).

    DATA(lo_patch_operation) = xco_cp_generation=>environment-
>dev_system( lo_transport_request->value
      )->for-fugr->create_patch_operation( ).

    DATA(lo_function_group) = mo_function_module->get_function_group( ).
    lo_patch_operation->add_object( lo_function_group->name
      )->for-update->add_function_module( mo_function_module->name
      )->set_source_code( xco_cp=>strings( VALUE #(
```

```
        ( `  " New source code goes here...` )
      ) ) ).

    lo_patch_operation->execute( ).

    release_transport_request(
      io_transport_request = lo_transport_request
      out                  = out
    ).
  ENDMETHOD.

  METHOD generate_transport_request.
    DATA(lo_transport_target) = mo_function_module->get_function_group(
      )->if_xco_ar_object~get_package(
      )->read(
      )-property-transport_layer->get_transport_target( ).

    ro_transport_request = xco_cp_cts=>transports-
>workbench( lo_transport_target->value
      )->create_request( 'Generated transport request' ).

    out->write( |Using newly generated transport request
{ ro_transport_request->value }.| ).
  ENDMETHOD.

  METHOD release_transport_request.
    DATA(lt_tasks) = io_transport_request->get_tasks( ).

    LOOP AT lt_tasks INTO DATA(lo_task).
      lo_task->release( ).
    ENDLOOP.

    io_transport_request->release( ).

    out->write( |Transport request { io_transport_request->value } has been
released.| ).
  ENDMETHOD.
ENDCLASS.
```

4.2.15.18.3.3.12 Table Functions and AMDP

The code sample below provides a self-contained example for the generation of a CDS table function. The CDS table function is generated together with the class providing the AMDP implementation and the underlying database table in a single PUT operation.

{·} Sample Code

```
"! Code sample for generating a CDS table function together with an AMDP class
"! and database table.
"!
"! IMPORTANT: The values of the constants CO_PACKAGE and CO_TRANSPORT must be
replaced
"! with an existing package and a modifiable Workbench transport request
matching the
"! transport target of the package.
CLASS zcl_xco_doc_cp_cs_gen_tbl_fnc DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.
```

```
  PRIVATE SECTION.
    CONSTANTS:
      co_package             TYPE sxco_package VALUE 'ZXCO_CP_PACKAGE',
      co_transport           TYPE sxco_transport VALUE 'X08K900164',
      co_database_table_name TYPE sxco_dbt_object_name VALUE 'ZXCO_TF_DBT',
      co_table_function_name TYPE sxco_cds_object_name VALUE
'ZXCO_TABLE_FUNCTION',
      co_amdp_class_name     TYPE sxco_ao_object_name VALUE
'ZCL_XCO_AMDP_CLASS',
      co_amdp_method_name    TYPE sxco_ao_component_name VALUE 'GET_VALUES'.
    METHODS:
      add_database_table
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put,
      add_table_function
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put,
      add_class
        IMPORTING
          io_put_operation TYPE REF TO if_xco_cp_gen_d_o_put.
ENDCLASS.
CLASS zcl_xco_doc_cp_cs_gen_tbl_fnc IMPLEMENTATION.
  METHOD main.
    DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( co_transport
      )->create_put_operation( ).
    add_database_table( lo_put_operation ).
    add_table_function( lo_put_operation ).
    add_class( lo_put_operation ).
    lo_put_operation->execute( ).
    out->write( |Table function { co_table_function_name } generated
successfully.| ).
  ENDMETHOD.
  METHOD add_database_table.
    DATA(lo_specification) = io_put_operation->for-tabl-for-database_table-
>add_object( co_database_table_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_specification->set_short_description( 'GENERATED' ).
    lo_specification->add_field( 'CLIENT'
      )->set_type( xco_cp_abap_dictionary=>built_in_type->clnt
      )->set_key_indicator(
      )->set_not_null( ).
    lo_specification->add_field( 'IDENTIFIER'
      )->set_type( xco_cp_abap_dictionary=>built_in_type->char( 30 )
      )->set_key_indicator(
      )->set_not_null( ).
    lo_specification->add_field( 'START_DATE'
      )->set_type( xco_cp_abap_dictionary=>built_in_type->dats ).
  ENDMETHOD.
  METHOD add_table_function.
    DATA(lo_form_specification) = io_put_operation->for-ddls-
>add_object( co_table_function_name
      )->set_package( co_package
      )->create_form_specification( ).
    DATA(lo_table_function) = lo_form_specification-
>set_short_description( 'GENERATED'
      )->add_table_function( ).
    lo_table_function->add_annotation( 'AccessControl.authorizationCheck'
      )->value->build( )->add_enum( 'NOT_REQUIRED' ).
    DATA(lo_parameter) = lo_table_function->add_parameter( 'p_identifier'
      )->set_data_type( xco_cp_abap_dictionary=>data_element( 'char30' ) ).
    DATA(lo_client_field) = lo_table_function-
>add_field( xco_cp_ddl=>field( 'client' ) ).
    lo_client_field->set_type( xco_cp_abap_dictionary=>built_in_type->clnt ).
    DATA(lo_val_identifier_field) = lo_table_function-
>add_field( xco_cp_ddl=>field( 'val_identifier' ) ).
```

```
    lo_val_identifier_field->set_type( xco_cp_abap_dictionary=>built_in_type-
>char( 30 ) ).
    lo_val_identifier_field->add_annotation( 'EndUserText.label'
      )->value->set( xco_cp_cds_annotation=>value->string( 'Identifier
field' ) ) ##NO_TEXT.
    lo_table_function->set_amdp_class( co_amdp_class_name
      )->set_amdp_method( co_amdp_method_name ).
  ENDMETHOD.
  METHOD add_class.
    DATA(lo_form_specification) = io_put_operation->for-clas-
>add_object( co_amdp_class_name
      )->set_package( co_package
      )->create_form_specification( ).
    lo_form_specification->set_short_description( 'GENERATED' ).
    lo_form_specification->definition->add_interface( 'IF_AMDP_MARKER_HDB' ).
    DATA(lo_class_method) = lo_form_specification->definition->section-public-
>add_class_method( CONV #( co_amdp_method_name ) ).
    lo_class_method->amdp->set_for_table_function( co_table_function_name ).
    lo_form_specification->implementation->add_method( CONV
#( co_amdp_method_name )
      )->set_source( VALUE #(
        ( |RETURN| ) ##NO_TEXT
        ( |SELECT CLIENT, IDENTIFIER AS val_identifier FROM
{ co_database_table_name }| ) ##NO_TEXT
        ( |WHERE IDENTIFIER = :p_identifier;| ) ##NO_TEXT
      ) )->amdp->mark_as_function(
      )->set_database_type( xco_cp_amdp=>database_type->hdb
      )->set_database_language( xco_cp_amdp=>database_language->sqlscript
      )->set_database_options( VALUE #( ( xco_cp_amdp=>database_option-
>read_only ) )
      )->set_database_entities( VALUE #( ( co_database_table_name ) ) ).
  ENDMETHOD.
ENDCLASS.
```

4.2.15.18.3.3.13 Generating Transformations

The XCO Generation APIs offer support for PUT and DELETE operations for transformations (XSLT objects).

The first code sample below illustrates how a transformation can be generated via a PUT operation and directly invoked dynamically to extract data from a provided XML string as well as present data as XML (making use of XCO Standard Library functionality to conveniently translate between strings and xstrings based on UTF-8).

Sample Code

```
DATA(lv_transport_request) = CONV sxco_transport( '...' ).
DATA(lv_transformation_name) = CONV
sxco_tf_object_name( 'ZMY_TRANSFORMATION' ).

DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( lv_transport_request
  )->create_put_operation( ).

DATA(lo_specification) = lo_put_operation->for-xslt-
>add_object( lv_transformation_name
  )->set_package( 'Z_MY_PACKAGE'
  )->create_form_specification( ).
lo_specification->set_short_description( 'Generated transformation'
  )->set_source( VALUE #(
    ( |<tt:transform xmlns:tt="http://www.sap.com/transformation-
templates">| )
    ( |  <tt:root name="ROOT"/>| )
```

```
    ( |  <tt:template>| )
    ( |    <record>| )
    ( |      <tt:attribute name="text" value-ref="root.text"/>| )
    ( |    </record>| )
    ( |  </tt:template>| )
    ( |</tt:transform>| )
  ) ).

lo_put_operation->execute( ).

" At this point a transformation with the name ZMY_TRANSFORMATION exists in
the system in active
" state.

TYPES:
  BEGIN OF ts_record,
    text TYPE string,
  END OF ts_record.
DATA ls_record TYPE ts_record.

" Using the generated transformation, it is now possible to dynamically apply
it to provided XML
" data.
DATA(lv_xml_string) = |<?xml version="1.0" encoding="UTF-8"?><record
text="Hello World"/>|.

" Converts 'string' to 'xstring' using 'UTF-8' code page.
DATA(lv_xml_xstring) = xco_cp=>string( lv_xml_string
  )->as_xstring( xco_cp_character=>code_page->utf_8
  )->value.

CALL TRANSFORMATION (lv_transformation_name)
  SOURCE XML lv_xml_xstring
  RESULT root = ls_record.

" At this point the TEXT component of the structure LS_RECORD will have the
value Hello World
" In the same manner, an XML string can be also be obtained from the ABAP
data object.
ls_record-text = 'Hello XCO'.

CALL TRANSFORMATION (lv_transformation_name)
 SOURCE root = ls_record
 RESULT XML lv_xml_xstring.

" Converts 'xstring' to 'string' using 'UTF-8' code page.
lv_xml_string = xco_cp=>xstring( lv_xml_xstring
  )->as_string( xco_cp_character=>code_page->utf_8
  )->value.

" Now the variable LV_XML_STRING will have the value <?xml version="1.0"
encoding="UTF-8"?><record text="Hello XCO"/>.
```

Just as with other object types, transformations can also be deleted:

Sample Code

```
DATA(lv_transport_request) = CONV sxco_transport( '...' ).
DATA(lv_transformation_name) = CONV
sxco_tf_object_name( 'ZMY_TRANSFORMATION' ).

DATA(lo_delete_operation) = xco_cp_generation=>environment-
>dev_system( lv_transport_request
  )->for-xslt->create_delete_operation( ).
lo_delete_operation->add_object( lv_transformation_name ).
```

```
lo_delete_operation->execute( ).
```

4.2.15.18.3.3.14 Changing the Content of Existing Domains

Using `PATCH` operations, it's possible to change the content of an existing domain. The following changes can be specified:

- Changes to header-level properties, such as the format of a domain or its output length
- Insertions, updates, or deletions of fixed values

The code sample below illustrates how the following changes can be performed with a single `PATCH` operation:

- Update the short description, case-sensitive indicator, and output style
- Update an existing fixed value
- Delete an existing fixed value
- Insert a new fixed value

Sample Code

```
DATA(lv_transport) = CONV sxco_transport( '...' ).
DATA(lv_domain_name) = CONV sxco_ad_object_name( 'MY_DOMAIN' ).

DATA(lo_patch_operation) = xco_cp_generation=>environment-
>dev_system( lv_transport
  )->for-doma->create_patch_operation( ).

DATA(lo_change_specification) = lo_patch_operation->add_object( lv_domain_name
  )->create_change_specification( ).

" Change header-level properties.
lo_change_specification->for-update->set_short_description( 'My new short
description'
  )->set_case_sensitive( abap_true
  )->set_output_style( xco_cp_domain=>output_style->normal ).

" Update the description of fixed value "A".
lo_change_specification->for-update->add_fixed_value( 'A'
  )->set_description( 'Fixed Value A' ).

" Delete fixed value "B".
lo_change_specification->for-delete->add_fixed_value( 'B' ).

" Insert fixed value "C".
lo_change_specification->for-insert->add_fixed_value( 'C'
  )->set_description( 'Fixed Value C' ).

lo_patch_operation->execute( ).
```

4.2.15.18.3.3.15 Changing the Content of Existing Data Elements

Using `PATCH` operations, it's possible to change the content of an existing data element. The following changes can be specified:

- Changes to header-level properties, such as the data type or the field labels

The code sample below illustrates how the following changes can be performed with a single `PATCH` operation:

- Update the short description
- Update the data type
- Update the short field label

Sample Code

```
DATA(lv_transport) = CONV sxco_transport( '...' ).
DATA(lv_data_element_name) = CONV sxco_ad_object_name( 'MY_DATA_ELEMENT' ).

DATA(lo_patch_operation) = xco_cp_generation=>environment-
>dev_system( lv_transport
  )->for-dtel->create_patch_operation( ).

DATA(lo_change_specification) = lo_patch_operation-
>add_object( lv_data_element_name
  )->create_change_specification( ).

" Change header-level properties.
lo_change_specification->for-update->set_short_description( 'My new short
description'
  )->set_data_type( xco_cp_abap_dictionary=>built_in_type->int4
  )->field_label-short->set_text( 'Sh. FL' )->set_length( 10 ).

lo_patch_operation->execute( ).
```

4.2.15.18.3.3.16 Generating CDS Type Definitions

The following code sample illustrates how you can generate a CDS simple type with the XCO library by means of running a `PUT` operation:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specifation) = lo_put_operation->for-drty-
>add_object( 'ZMY_CDS_TYPE_DEFINITION'
  )->set_package( 'ZMY_PACKAGE'
  )->create_form_specification( ).
lo_form_specifation->set_short_description( 'Generated CDS Type Definition' ).

DATA(lo_simple_type_definition) = lo_form_specifation-
>add_simple_type_definition( ).
```

```
lo_simple_type_definition->add_annotation( 'EndUserText.label' )->value-
>build(
  )->add_string( 'My CDS type definition' ).

lo_simple_type_definition->set_type( xco_cp_abap_dictionary=>built_in_type-
>char( 10 ) ).

lo_put_operation->execute( ).
```

Running this `PUT` operation will produce the CDS type definition `ZMY_CDS_TYPE_DEFINITION` with the following source:

Sample Code

```
@EndUserText.label: 'My CDS type definition'
define type  ZMY_CDS_TYPE_DEFINITION : abap.char( 10 )
```

In general, the following objects are eligible to be used as the type for a simple type definition:

- ABAP dictionary built-in type (objects of type `CL_XCO_AD_BUILT_IN_TYPE`)
- Data elements (objects of type `IF_XCO_AD_DATA_ELEMENT`)
- CDS type definitions (objects of type `IF_XCO_CDS_TYPE_DEFINITION`)

CDS type definitions can also be deleted again, as is illustrated by the following code sample:

Sample Code

```
DATA(lo_delete_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->for-drty->create_delete_operation( ).
lo_delete_operation->add_object( 'ZMY_CDS_TYPE_DEFINITION' ).
lo_delete_operation->execute( ).
```

4.2.15.18.3.3.17 Patching Structures and Database Tables

Using `PATCH` operations, it's possible to change the content of an existing structure or database table. The following changes can be specified:

- Choose to update the type and/or the currency and/or the quantity information of an existing database table field or structure component
- Update of the structure of an existing include of a database table or a structure

The following code sample illustrates how the changes described above can be applied to an existing database table:

Sample Code

```
DATA(lv_transport) = CONV sxco_transport( '...' ).
DATA(lv_database_table_name) = CONV sxco_dbt_object_name( 'ZMY_DBT' ).

DATA(lo_patch_operation) = xco_cp_generation=>environment-
>dev_system( lv_transport
  )->create_patch_operation( ).
```

```
DATA(lo_change_specification) = lo_patch_operation->for-tabl-for-
database_table->add_object( lv_database_table_name
  )->create_change_specification( ).

" Update the type of field DATA_FIELD to abap.char( 10 )
lo_change_specification->for-update->add_field( 'DATA_FIELD'
  )->set_type( xco_cp_abap_dictionary=>built_in_type->char( 10 ) ).

" Update the currency/quantity reference information for field CURR_QUAN
lo_change_specification->for-update->add_field( 'CURR_QUAN'
  )->currency_quantity->set_reference_table( 'ZMY_REF_TABLE'
  )->set_reference_field( 'ZMY_REF_FIELD' ).

" Update the structure that is used for the first include of the
" database table.
lo_change_specification->for-update->add_include( 1
  )->set_structure( 'ZMY_INCL_STR' ).

lo_patch_operation->execute( ).
```

The same changes could also be applied to an existing structure by adding a structure to the `PATCH` operation created above:

Sample Code

```
lo_patch_operation->for-tabl-for-structure->add_object( 'ZMY_STRUCTURE' ).
```

Note that the `PATCH` operation obtained via `XCO_CP_GENERATION=>ENVIRONMENT→DEV_SYSTEM( LV_TRANSPORT )→CREATE_PATCH_OPERATION( )` is a mass-enabled operation which can contain several objects at once. Upon execution, first, the specified changes will be applied to all objects that have been added to the `PATCH` operation, and afterwards, all objects will be activated together in a single mass activation. Currently, the following object types are enabled for mass `PATCH` operations:

- DOMA (Domains)
- DTEL (Data Elements)
- TABL (Database Tables and Structures)

4.2.15.18.3.3.18 Generating CDS External Entities

The following code sample shows how you can generate a CDS dynamic external entity with the XCO library by running a `PUT` operation:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-ddls-
>add_object( 'ZMY_CDS_DYN_EXT_ENTITY'
  )->set_package( 'ZMY_PACKAGE'
  )->create_form_specification( ).
```

```
DATA(lo_external_entity) = lo_form_specification-
>set_short_description( 'Generated CDS External Entity'
  )->add_external_entity( ).

lo_external_entity->set_with_federated_data(
  )->set_provided_at_runtime(
  )->set_on_unlinked_fail( ).

lo_external_entity->add_annotation( 'EndUserText.label' ##NO_TEXT
  )->value->build( )->add_string( 'My external entity' ).

lo_external_entity->data_source->set_remote_object_name( 'My remote object
name' ).

lo_external_entity->add_field( xco_cp_ddl=>field( 'key_field' )
  )->set_key(
  )->set_type( xco_cp_abap_dictionary=>built_in_type->int1 ).

lo_external_entity->add_field( xco_cp_ddl=>field( 'field' )
  )->set_type( xco_cp_abap_dictionary=>built_in_type->char( 10 )
  )->set_external_name( 'My field external name' ).

lo_put_operation->execute( ).
```

Running this `PUT` operation will produce the CDS external entity `ZMY_CDS_DYN_EXT_ENTITY` with the following source:

Sample Code

```
@EndUserText.label: 'My external entity'
define external entity ZMY_CDS_DYN_EXT_ENTITY external name "My remote object
name"
{
  key key_field : abap.int1;
  field : abap.char(10) external name "My field external name";
}
WITH FEDERATED DATA
PROVIDED AT RUNTIME
ON UNLINKED FAIL
```

To connect to an external system, you will also need a logical external schema. It can be generated like this:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-desd-
>add_object( 'ZMY_LOGICAL_EXT_SCHEMA'
  )->set_package( 'ZMY_PACKAGE'
  )->create_form_specification( ).

lo_form_specification->set_short_description( 'Generated Logical External
Schema'
  )->set_default_remote_schema_name( 'My Schema Name' ).

lo_put_operation->execute( ).
```

The XCO library also supports static external entity generation by specifying a logical external schema in the `set_provided_by_lgcl_ex_schema` method like this:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-ddls-
>add_object( 'ZMY_CDS_STATIC_EXT_ENTITY'
  )->set_package( 'ZMY_PACKAGE'
  )->create_form_specification( ).

DATA(lo_external_entity) = lo_form_specification-
>set_short_description( 'Generated CDS External Entity'
  )->add_external_entity( ).

lo_external_entity->set_with_federated_data(
  )->set_provided_by_lgcl_ex_schema( xco_cp_abap_repository=>object->desd-
>for( 'ZMY_LOGICAL_EXT_SCHEMA' ) ).

lo_external_entity->add_annotation( 'EndUserText.label' ##NO_TEXT
  )->value->build( )->add_string( 'My external entity' ).

lo_external_entity->data_source->set_remote_object_name( 'My remote object
name' ).

lo_external_entity->add_field( xco_cp_ddl=>field( 'key_field' )
  )->set_key(
  )->set_type( xco_cp_abap_dictionary=>built_in_type->int1 ).

lo_external_entity->add_field( xco_cp_ddl=>field( 'field' )
  )->set_type( xco_cp_abap_dictionary=>built_in_type->char( 10 )
  )->set_external_name( 'My field external name' ).

lo_put_operation->execute( ).
```

Running this `PUT` operation will produce the static CDS external entity `ZMY_CDS_STATIC_EXT_ENTITY` with the following source:

Sample Code

```
@EndUserText.label: 'My external entity'
define external entity ZMY_CDS_STATIC_EXT_ENTITY external name "My remote
object name"
{
  key key_field : abap.int1;
  field : abap.char(10) external name "My field external name";
}
WITH FEDERATED DATA
PROVIDED BY ZMY_LOGICAL_EXT_SCHEMA
```

Related Information

CDS External Entities

4.2.15.18.3.3.19 Generating CDS Table Entities

The XCO Generation APIs allow to generate CDS table entities through `PUT` operations.

The following code sample demonstrates how to generate a CDS table entity using the XCO library by running a `PUT` operation:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).
DATA(lo_form_specification) = lo_put_operation->for-ddls-
>add_object( 'ZMY_CDS_TABLE_ENTITY'
  )->set_package( 'ZMY_PACKAGE'
  )->create_form_specification( ).
DATA(lo_table_entity) = lo_form_specification->set_short_description( 'XCO
Generated CDS Table Entity'
  )->add_table_entity( ).
lo_table_entity->add_annotation( 'EndUserText.label'
  )->value->build( )->add_string( 'My Table Entity' ).
lo_table_entity->add_annotation( 'ClientHandling.type'
  )->value->build(
  )->add_enum( 'CLIENT_INDEPENDENT' ).
lo_table_entity->add_annotation( 'AbapCatalog.deliveryClass'
  )->value->build(
  )->add_enum( 'LOCAL_DATA' ).
lo_table_entity->add_field( xco_cp_ddl=>field( 'key_field' )
  )->set_type( xco_cp_abap_dictionary=>built_in_type->int1
  )->set_key( ).
lo_table_entity->add_field( xco_cp_ddl=>field( 'data_field' )
  )->set_type( xco_cp_abap_dictionary=>built_in_type->int1
  )->set_null( ).
lo_put_operation->execute( ).
```

Running this `PUT` operation generates the CDS table entity `ZMY_CDS_TABLE_ENTITY` with the following source code:

Sample Code

```
@EndUserText.label: 'My Table Entity'
@ClientHandling.type: #CLIENT_INDEPENDENT
@AbapCatalog.deliveryClass: #LOCAL_DATA
define table entity ZMY_CDS_TABLE_ENTITY
{
  key key_field : abap.int1;
  data_field : abap.int1 null;
}
```

4.2.15.18.3.4 I18N APIs

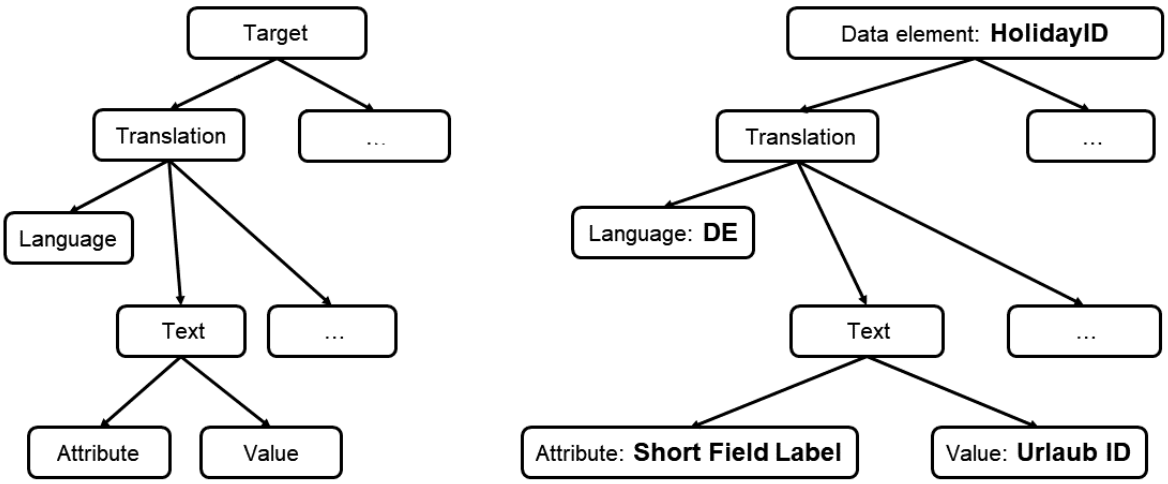
The XCO I18N APIs allow the programmatic maintenance of translations for language-dependent texts. The I18N module consists of high-level and strongly typed APIs and provides support for the following targets:

- Domains

- Data elements
- Data definitions
- Metadata extensions
- Message classes
- Application log objects
- Business configuration objects
- IAM business catalogs
- Text pools
- Text tables

Design of the XCO I18N APIs

The goal of the XCO I18N architecture is to provide a homogenous way to programmatically maintain and access translations for different kinds of language-dependent texts with a public API that is independent of the underlying concrete object type. The overall structure is illustrated in the following diagram:



The left side of the diagram shows the general abstractions while the right side shows what the general abstractions would correspond to in the case of the short field label of a given data element. The following terminology is used:

- Target: A collection of translations
- Translation: A tuple consisting of a language and a collection of texts
- Language: A language that is installed in the system and is identified by an SPRAS value
- Text: A tuple consisting of an attribute and a value

The following sections will describe and illustrate the available targets and text attributes in more detail:

Domains

For a given domain a single fixed value (identified by its lower limit) defines a target. The only available text attribute is the description of a fixed value. Here's how the translation for language German could be maintained for the fixed value DE of domain ZCOUNTRY:

Sample Code

```
DATA(lo_text_attribute) = xco_cp_domain=>text_attribute-
>fixed_value_description.
DATA(lo_language) = xco_cp=>language( 'D' ).
DATA(lo_text) = lo_text_attribute-
>create_text( xco_cp=>string( 'Deutschland' ) ).
DATA(lo_target) = xco_cp_i18n=>target->domain->fixed_value(
  iv_domain_name = 'ZCOUNTRY'
  iv_lower_limit = 'DE'
).
" Set the translation.
lo_target->set_translation(
  it_texts           = VALUE #( ( lo_text ) )
  io_language        = lo_language
  io_change_scenario = lo_transport_request
).
" Read the translation.
DATA(lo_translation) = lo_target->get_translation(
  io_language        = lo_language
  it_text_attributes = VALUE #( ( lo_text_attribute ) )
).
LOOP AT lo_translation->texts INTO DATA(lo_german_text).
  " LV_VALUE is of type STRING and contains `Deutschland`.
  DATA(lv_value) = lo_text_attribute-
>if_xco_i18n_text_attribute~get_string_for_text( lo_german_text->value ).
ENDLOOP.
```

Data elements

Unlike domains, once a data element is fixed there are no more degrees of freedom for language-dependent texts as all the different field labels are directly defined on the level of the data element itself. The following example illustrates how the short field label of data element ZFIRST_NAME can be translated to German (first name = Vorname) and how the translated value can be read back again:

Sample Code

```
DATA(lo_text_attribute) = xco_cp_data_element=>text_attribute-
>short_field_label.
DATA(lo_language) = xco_cp=>language( 'D' ).
DATA(lo_text) = lo_text_attribute->create_text( xco_cp=>string( 'Vorname' ) ).
DATA(lo_target) = xco_cp_i18n=>target->data_element->object( 'ZFIRST_NAME' ).
" Set the translation.
lo_target->set_translation(
  it_texts           = VALUE #( ( lo_text ) )
  io_language        = lo_language
  io_change_scenario = lo_transport_request
).
" Read the translation.
DATA(lo_translation) = lo_target->get_translation(
  io_language        = lo_language
```

```
  it_text_attributes = VALUE #( ( lo_text_attribute ) )
).
LOOP AT lo_translation->texts INTO DATA(lo_german_text).
  " LV_VALUE is of type STRING and contains `Vorname`.
  DATA(lv_value) = lo_text_attribute-
>if_xco_i18n_text_attribute~get_string_for_text( lo_german_text->value ).
ENDLOOP.
```

Data definitions

For data definitions all language-dependent texts are supplied via annotations provided directly in the source of the data definition. A data definition defines the following targets:

- Entity
- A parameter identified by its name
- A field identified by its name

Depending on the annotation it may be necessary to specify an array index (e.g. UI.lineItem.label) to fully specify a text attribute. The APIs in XCO_CP_DATA_DEFINITION=>TEXT_ATTRIBUTE provide an easy overview of maintainable text attributes for the three different kinds of data definition targets.

The code sample below illustrates how the EndUserText.label annotation value can be translated to German for the field startDate of the CDS view entity ZVACATION_REQUEST (start date = Startdatum):

Sample Code

```
DATA(lo_text_attribute) = xco_cp_data_definition=>text_attribute->field-
>endusertext_label.
DATA(lo_language) = xco_cp=>language( 'D' ).
DATA(lo_text) = lo_text_attribute-
>create_text( xco_cp=>string( 'Startdatum' ) ).
DATA(lo_target) = xco_cp_i18n=>target->data_definition->field(
  iv_entity_name = 'ZVACATION_REQUEST'
  iv_field_name  = 'startDate'
).
" Set the translation.
lo_target->set_translation(
  it_texts           = VALUE #( ( lo_text ) )
  io_language        = lo_language
  io_change_scenario = lo_transport_request
).
" Read the translation.
DATA(lo_translation) = lo_target->get_translation(
  io_language        = lo_language
  it_text_attributes = VALUE #( ( lo_text_attribute ) )
).
LOOP AT lo_translation->texts INTO DATA(lo_german_text).
  " LV_VALUE is of type STRING and contains `Startdatum`.
  DATA(lv_value) = lo_text_attribute-
>if_xco_i18n_text_attribute~get_string_for_text( lo_german_text->value ).
ENDLOOP.
```

Metadata extensions

Metadata extensions are structurally equivalent to data definitions in that all language-dependent texts are supplied via annotations provided directly in the source of the metadata extension and the available targets are

- Entity
- A parameter identified by its name
- A field identified by its name

The maintainable text attributes are available via the XCO_CP_METADATA_EXTENSION=>TEXT_ATTRIBUTE API.

The code sample below illustrates how the UI.lineItem.label annotation value can be translated to German for the field endDate of the metadata extension ZVACATION_REQUEST_EXT (end date = Enddatum):

Sample Code

```
DATA(lo_text_attribute) = xco_cp_metadata_extension=>text_attribute->field-
>ui_lineitem_label( 1 ).
DATA(lo_language) = xco_cp=>language( 'D' ).
DATA(lo_text) = lo_text_attribute-
>create_text( xco_cp=>string( 'Enddatum' ) ).
DATA(lo_target) = xco_cp_i18n=>target->metadata_extension->field(
  iv_metadata_extension_name  = 'ZVACATION_REQUEST_EXT'
  iv_field_name               = 'endDate'
).
" Set the translation.
lo_target->set_translation(
  it_texts           = VALUE #( ( lo_text ) )
  io_language        = lo_language
  io_change_scenario = lo_transport_request
).
" Read the translation.
DATA(lo_translation) = lo_target->get_translation(
  io_language        = lo_language
  it_text_attributes = VALUE #( ( lo_text_attribute ) )
).
LOOP AT lo_translation->texts INTO DATA(lo_german_text).
  " LV_VALUE is of type STRING and contains `Enddatum`.
  DATA(lv_value) = lo_text_attribute-
>if_xco_i18n_text_attribute~get_string_for_text( lo_german_text->value ).
ENDLOOP.
```

Message class

Messages classes follow a structure similar to that of domains: A single message of a message class (identified by its number) defines a target and the only available text attribute is the short text of a message.

In the code sample below message 005 of message class ZMESSAGES (“Record created.” = “Eintrag angelegt.”) is translated to German:

Sample Code

```
DATA(lo_text_attribute) = xco_cp_message_class=>text_attribute-
>message_short_text.
    DATA(lo_language) = xco_cp=>language( 'D' ).
```

```
    DATA(lo_text) = lo_text_attribute->create_text( xco_cp=>string( 'Eintrag
angelegt.' ) ).
    DATA(lo_target) = xco_cp_i18n=>target->message_class->message(
      iv_message_class_name = 'ZMESSAGES'
      iv_message_number     = '005'
    ).
    " Set the translation.
    lo_target->set_translation(
      it_texts           = VALUE #( ( lo_text ) )
      io_language        = lo_language
      io_change_scenario = lo_transport_request
    ).
    " Read the translation.
    DATA(lo_translation) = lo_target->get_translation(
      io_language        = lo_language
      it_text_attributes = VALUE #( ( lo_text_attribute ) )
    ).
    LOOP AT lo_translation->texts INTO DATA(lo_german_text).
      " LV_VALUE is of type STRING and contains `Eintrag angelegt.`.
      DATA(lv_value) = lo_text_attribute-
>if_xco_i18n_text_attribute~get_string_for_text( lo_german_text->value ).
    ENDLOOP.
```

Application log objects

For a given application log object both the object itself as well as each of its subobjects act as targets. The only available text attribute is the short description. The following code sample illustrates how the German translation for the short description of the application log object Z_PROGRESS_RECORDING can be set and read:

Sample Code

```
DATA(lo_text_attribute) = xco_cp_application_log_object=>text_attribute-
>object->short_description.

DATA(lo_language) = xco_cp=>language( 'D' ).
DATA(lo_text) = lo_text_attribute-
>create_text( xco_cp=>string( 'Fortschrittsaufzeichnung' ) ).

DATA(lo_target) = xco_cp_i18n=>target->application_log_object-
>object( 'Z_PROGRESS_RECORDING' ).

" Set the translation.
lo_target->set_translation(
  it_texts           = VALUE #( ( lo_text ) )
  io_language        = lo_language
  io_change_scenario = lo_transport_request
).

" Read the translation.
DATA(lo_translation) = lo_target->get_translation(
  io_language        = lo_language
  it_text_attributes = VALUE #( ( lo_text_attribute ) )
).

LOOP AT lo_translation->texts INTO DATA(lo_german_text).
  " LV_VALUE is of type STRING and contains `Fortschrittsaufzeichnung`.
  DATA(lv_value) = lo_text_attribute-
>if_xco_i18n_text_attribute~get_string_for_text( lo_german_text->value ).
ENDLOOP.
```

Setting and getting translations for subobjects works in completely analogous fashion by first obtaining the subobject target and then using text attribute XCO_CP_APPLICATION_LOG_OBJECT=>TEXT_ATTRIBUTE->SUBOBJECT->SHORT_DESCRIPTION.

Business configuration objects

Translations can also be maintained for business configuration objects which are available for maintenance via the "Maintain Business Configurations" Fiori app. Each business configuration object has a name and description which is visible in the Maintain Business Configurations app. The code sample below illustrates how the description of a business configuration object can be translated (and read out).

Sample Code

```
DATA(lo_text_attribute) = xco_cp_business_cnfgrtn_object=>text_attribute-
>description.

DATA(lo_language) = xco_cp=>language( 'D' ).
DATA(lo_text) = lo_text_attribute-
>create_text( xco_cp=>string( 'Urlaubskalender' ) ).

DATA(lo_target) = xco_cp_i18n=>target->business_configuration_object-
>object( 'Z_HOLIDAY_CALENDAR' ).

" Set the translation.
lo_target->set_translation(
  it_texts           = VALUE #( ( lo_text ) )
  io_language        = lo_language
  io_change_scenario = lo_transport_request
).

" Read the translation.
DATA(lo_translation) = lo_target->get_translation(
  io_language        = lo_language
  it_text_attributes = VALUE #( ( lo_text_attribute ) )
).

LOOP AT lo_translation->texts INTO DATA(lo_german_text).
  " LV_VALUE is of type STRING and contains `Urlaubskalender`.
  DATA(lv_value) = lo_text_attribute-
>if_xco_i18n_text_attribute~get_string_for_text( lo_german_text->value ).
ENDLOOP.
```

IAM business catalogs

Translations can also be maintained for IAM business catalogs. The only text attribute of an IAM business catalog is its description. The code sample below illustrates how the description of an IAM business catalog can be translated (and read out).

Sample Code

```
DATA(lo_text_attribute) = xco_cp_iam_business_catalog=>text_attribute-
>description.
DATA(lo_language) = xco_cp=>language( 'D' ).
```

```
DATA(lo_text) = lo_text_attribute-
>create_text( xco_cp=>string( 'Buchhaltung' ) ).
DATA(lo_target) = xco_cp_iam=>business_catalog->for( 'Z_ACCOUNTING' )-
>i18n_target->object( ).
" Set the translation.
lo_target->set_translation(
  it_texts           = VALUE #( ( lo_text ) )
  io_language        = lo_language
  io_change_scenario = lo_transport_request
).
" Read the translation.
DATA(lo_translation) = lo_target->get_translation(
  io_language        = lo_language
  it_text_attributes = VALUE #( ( lo_text_attribute ) )
).
LOOP AT lo_translation->texts INTO DATA(lo_german_text).
  " LV_VALUE is of type STRING and contains `Buchhaltung`.
  DATA(lv_value) = lo_text_attribute-
>if_xco_i18n_text_attribute~get_string_for_text( lo_german_text->value ).
ENDLOOP.
```

Text pools

Both classes and function groups allow the definition of text symbols which are stored in the text pool associated with the respective class or function group. Maintaining the translations for such text symbols can be accomplished like:

Sample Code

```
DATA(lo_text_attribute) = xco_cp_text_pool=>text_attribute->text_element_text.
DATA(lo_language) = xco_cp=>language( 'D' ).
DATA(lo_text) = lo_text_attribute->create_text( xco_cp=>string( 'Auftrag
abgeschickt.' ) ).
DATA(lo_target) = xco_cp_i18n=>target->text_pool->class_text_symbol(
  iv_class_name     = 'ZCL_MY_CLASS'
  iv_text_symbol_id = '001'
).
" Set the translation.
lo_target->set_translation(
  it_texts           = VALUE #( ( lo_text ) )
  io_language        = lo_language
  io_change_scenario = lo_transport_request
).
" Read the translation.
DATA(lo_translation) = lo_target->get_translation(
  io_language        = lo_language
  it_text_attributes = VALUE #( ( lo_text_attribute ) )
).
LOOP AT lo_translation->texts INTO DATA(lo_german_text).
  " LV_VALUE is of type STRING and contains `Auftrag abgeschickt.`.
  DATA(lv_value) = lo_text_attribute-
>if_xco_i18n_text_attribute~get_string_for_text( lo_german_text->value ).
ENDLOOP.
```

In the above example the German translation is maintained for the text of text symbol 001 (the original text is "Request sent.") of class `ZCL_MY_CLASS`. If the text symbol belonged to a function group instead of a class the code would look like:

Sample Code

```
DATA(lo_text_attribute) = xco_cp_text_pool=>text_attribute->text_element_text.
DATA(lo_language) = xco_cp=>language( 'D' ).
DATA(lo_text) = lo_text_attribute->create_text( xco_cp=>string( 'Auftrag
abgeschickt.' ) ).
DATA(lo_target) = xco_cp_i18n=>target->text_pool->function_group_text_symbol(
  iv_function_group_name = 'ZMY_FNCTN_GRP'
  iv_text_symbol_id         = '001'
).
" Set the translation.
lo_target->set_translation(
  it_texts           = VALUE #( ( lo_text ) )
  io_language        = lo_language
  io_change_scenario = lo_transport_request
).
" Read the translation.
DATA(lo_translation) = lo_target->get_translation(
  io_language        = lo_language
  it_text_attributes = VALUE #( ( lo_text_attribute ) )
).
LOOP AT lo_translation->texts INTO DATA(lo_german_text).
  " LV_VALUE is of type STRING and contains `Auftrag abgeschickt.`.
  DATA(lv_value) = lo_text_attribute-
>if_xco_i18n_text_attribute~get_string_for_text( lo_german_text->value ).
ENDLOOP.
```

Text tables

Any database table with delivery class C or S which has exactly one key field with the underlying ABAP Dictionary built-in type LANG is considered a text table and can act as a source of targets. Each key field of a text table is uniquely associated with one of the following three groups:

- Client key field
- Master key fields
- Language key field

The semantics are such that a full specification of all master key fields of a text table defines a target and that each non-key field which is character-like and has at least length 2 is considered a text attribute of that target.

As an example, consider the database table ZXCO_CAL_HOLITXT which contains the language-dependent texts for public holidays maintained in database table ZXCO_CAL_HLDY:

{ZXCO_CAL_HLDY}

{ZXCO_CAL_HOLITXT}

Here CLIENT is the client key field of the text table ZXCO_CAL_HOLITXT, HOLIDAY_ID is the master key field and LANGUAGE is the language key field. Fixing a value for the HOLIDAY_ID key field (e.g. “CHRISTMAS”) determines a target for which the text attribute DESCRIPTION can then be read and written, as is illustrated by the code sample below:

Sample Code

```
DATA(lo_language) = xco_cp=>language( 'D' ).
DATA(lo_text) = xco_cp=>string( 'Weihnachten' ).
```

```
DATA(lo_target) = xco_cp_i18n=>target->text_table->record(
  iv_database_table_name     = 'ZCAL_XCO_HOLITXT'
  it_master_key_fields       = VALUE #( ( name = 'HOLIDAY_ID' value =
xco_cp=>data_object->for( 'CHRISTMAS' ) ) )
  iv_language_key_field_name = 'LANGUAGE'
).
" Set the translation.
lo_target->set_translation(
  it_texts           = VALUE #( ( attribute = 'DESCRIPTION' value =
lo_text ) )
  io_language        = lo_language
  io_change_scenario = lo_transport_request
).
" Read the translation.
DATA(lo_translation) = lo_target->get_translation(
  io_language        = lo_language
  it_text_attributes = VALUE #( ( 'DESCRIPTION' ) )
).
LOOP AT lo_translation->texts INTO DATA(lo_german_text).
  " LV_VALUE is of type STRING and contains `Weihnachten`.
  DATA(lv_value) = lo_german_text->get_string_value( ).
ENDLOOP.
```

4.2.15.18.3.5 API Release Framework

Find out how to set the API state.

Setting and getting API states programmatically is supported for the following objects:

- Classes (IF_XCO_AO_CLASS)
- Entities of data definitions (IF_XCO_CDS_ENTITY)
- Domains (IF_XCO_DOMAIN)
- Data elements (IF_XCO_AD_DATA_ELEMENT)
- Function modules of function groups (IF_XCO_FUNCTION_MODULE)
- Interfaces (IF_XCO_AO_INTERFACE)
- Structures (IF_XCO_AD_STRUCTURE)
- Database tables (IF_XCO_DATABASE_TABLE)
- Table types (IF_XCO_AD_TABLE_TYPE)
- Transformations (IF_XCO_TRANSFORMATION)

Each of the above interfaces defines the two methods SET_API_STATE and GET_API_STATE to programmatically set or get the API state of the represented object. For data elements, for example, this looks like:

Sample Code

```
DATA(lo_transport) = xco_cp_cts=>transport->for( '...' ).
DATA(lo_api_state) = xco_cp_ars=>api_state->released( VALUE
#( ( xco_cp_ars=>visibility->sap_cloud_platform ) ) ).

DATA(lo_data_element) = xco_cp_abap_repository=>object->dtel-
>for( 'ZMY_DATA_ELEMENT' ).
lo_data_element->set_api_state(
  io_change_scenario = lo_transport
  io_api_state       = lo_api_state
```

```
).
```

4.2.15.18.3.6 Core Data Services

The XCO library provides several abstractions and APIs to simplify working with objects in the area of Core Data Services. The functionality ranges from reading the content of CDS entities to querying annotations.

Read APIs

With the CDS Read APIs, it's possible to access the content of a given CDS entity in a strongly typed manner, as is illustrated by the following example:

Sample Code

```
DATA(lo_view_entity) = xco_cp_cds=>view_entity( 'MY_VIEW_ENTITY' ).
DATA(lo_view_entity_content) = lo_view_entity->content( ).
DATA(ls_view_entity_content) = lo_view_entity_content->get( ).
DATA(lv_short_description) = ls_view_entity_content-short_description.
DATA(ls_data_source) = ls_view_entity_content-data_source.
DATA(lv_root_indicator) = ls_view_entity_content-root_indicator.
DATA(lt_name_list) = ls_view_entity_content-name_list.
DATA(lo_where) = ls_view_entity_content-where.
DATA(lt_group_by) = ls_view_entity_content-group_by.
" Attributes can be accessed directly via corresponding get methods, such as:
lt_name_list = lo_view_entity_content->get_name_list( ).
LOOP AT lo_view_entity->parameters->all->get( ) INTO DATA(lo_parameter).
  DATA(ls_parameter) = lo_parameter->content( )->get( ).
ENDLOOP.
LOOP AT lo_view_entity->associations->all->get( ) INTO DATA(lo_association).
  DATA(ls_association) = lo_association->content( )->get( ).
ENDLOOP.
LOOP AT lo_view_entity->compositions->all->get( ) INTO DATA(lo_composition).
  DATA(ls_composition) = lo_composition->content( )->get( ).
ENDLOOP.
LOOP AT lo_view_entity->fields->all->get( ) INTO DATA(lo_field).
  DATA(ls_field) = lo_field->content( )->get( ).
ENDLOOP.
" Fields/parameters/associations/compositions can be addressed directly, such
as:
lo_field = lo_view_entity->field( 'FIELD_1' ).
DATA(lv_does_field_exist) = lo_field->exists( ).
IF lv_does_field_exist EQ abap_true.
  ls_field = lo_field->content( )->get( ).
ENDIF.
```

Data Definition Language

Using the Data Definition Language (DDL) module of the XCO Library, different kinds of DDL expressions can be built and successively used in conjunction with the XCO Generation APIs when providing specifications for

DDLS objects. Complex expressions like case, cast, or conditional expressions as well as literals, field, and data source expressions can be easily specified using the `XCO_CP_DDL` API:

Sample Code

```
" Case expression
DATA(lo_case_expression_builder) = xco_cp_ddl=>expression->case->builder( ).
lo_case_expression_builder->set_operand( xco_cp_ddl=>field( 'NAME' )
  )->add_when(
    io_operand = xco_cp_ddl=>expression->for( 'JOHN' )
    io_result  = xco_cp_ddl=>literal->numeric( 1 )
  )->add_when(
    io_operand = xco_cp_ddl=>expression->for( 'MARK' )
    io_result  = xco_cp_ddl=>literal->numeric( 2 )
  )->add_when(
    io_operand = xco_cp_ddl=>expression->for( 'JULIA' )
    io_result  = xco_cp_ddl=>literal->numeric( 3 )
  )->set_else( xco_cp_ddl=>literal->numeric( 4 ) ).
" Conditional expression
DATA(lo_condition_expression) = xco_cp_ddl=>expression->for( 'CITY'
  )->ne( xco_cp_ddl=>literal->character( 'BERLIN' )
  )->or( xco_cp_ddl=>expression->for( 'CITY'
    )->is_not_initial( ) ).
" Data source expression
DATA(lo_data_source_expression) = xco_cp_ddl=>data_source-
>entity( 'XCO_TEST_ENTITY'
  )->inner_join(
    io_data_source = xco_cp_ddl=>data_source-
>database_table( 'XCO_TEST_TABLE' )
    io_condition   = xco_cp_ddl=>field( 'FIELD' )->of_projection(
      )->eq( xco_cp_ddl=>field( 'FIELD' )->of( 'XCO_TEST_TABLE' )
  )
).
```

Annotations Query APIs

The XCO CDS module may be used to conveniently retrieve the value of annotations that have been provided for CDS entities as well as their fields or parameters. As with the XCO JSON module it is possible to write the annotation value to an explicitly defined ABAP structure to enable further processing of the value:

Sample Code

```
TYPES:
  BEGIN OF ts_title,
    type  TYPE string,
    label TYPE string,
    value TYPE string,
  END OF ts_title,
  BEGIN OF ts_header_info,
    typename       TYPE string,
    typenameplural TYPE string,
    title          TYPE ts_title,
  END OF ts_header_info,
  BEGIN OF ts_ui,
    headerinfo TYPE ts_header_info,
  END OF ts_ui.
DATA ls_ui TYPE ts_ui.
" Direct annotations.
DATA(lo_view_entity) = xco_cp_cds=>view_entity( 'MY_VIEW_ENTITY' ).
```

```
xco_cp_cds=>annotations->direct->of( lo_view_entity
  )->pick( 'UI'
  )->get_value(
  )->write_to( REF #( ls_ui ) ).
```

In the example above, the value of the “UI” annotation defined directly on the provided CDS view entity is read and written to the specifically defined ABAP structure. Parts of the annotation value not specified in the ABAP structure will be ignored.

Annotations for CDS entities are aggregated at runtime from different sources. With the XCO annotation Query APIs, it's easy to specify which source should be considered when annotations are retrieved. For example, it's possible to consider only annotations defined in metadata extensions:

{·} Sample Code

```
DATA(lo_view_entity_field) = xco_cp_cds=>view_entity( 'MY_VIEW_ENTITY'
  )->field( 'MY_FIELD' ).
DATA(lv_ui_annotation_contained) = xco_cp_cds=>annotations-
>metadata_extension->of( lo_view_entity_field
  )->contain( 'UI' ).
```

The following annotation sources are available via `XCO_CP_CDS=>ANNOTATIONS`:

- Aggregated: The aggregation of all the different sources according to the preference rules reflecting the value that will be present at runtime
- Derived: Annotations that are derived from data elements
- Direct: Annotations that are defined directly in the source for the given CDS entity
- Inherited: Annotations that are inherited from a CDS entity that the given CDS entity is based on
- Metadata extension: Annotations that are defined in a metadata extension that extends the given CDS entity

Cardinality

The XCO CDS module supports the definition of cardinalities using two different syntax forms:

- Cardinality Syntax Written in Words
- Numeric Cardinality Syntax

→ Tip

SAP recommends using the cardinality syntax written in words instead of the numeric syntax, since it improves query performance in some scenarios.

{·} Sample Code

Cardinality Syntax Written in Words

```
DATA(lo_of_exact_one_to_exact_one) = xco_cp_cds=>cardinality-
>of_exact_one_to_exact_one.
DATA(lo_of_exact_one_to_many) = xco_cp_cds=>cardinality->of_exact_one_to_many.
DATA(lo_of_exact_one_to_one) = xco_cp_cds=>cardinality->of_exact_one_to_one.
DATA(lo_of_many_to_exact_one) = xco_cp_cds=>cardinality->of_many_to_exact_one.
```

```
DATA(lo_of_many_to_many) = xco_cp_cds=>cardinality->of_many_to_many.
DATA(lo_of_many_to_one) = xco_cp_cds=>cardinality->of_many_to_one.
DATA(lo_of_one_to_exact_one) = xco_cp_cds=>cardinality->of_one_to_exact_one.
DATA(lo_of_one_to_many) = xco_cp_cds=>cardinality->of_one_to_many.
DATA(lo_of_one_to_one) = xco_cp_cds=>cardinality->of_one_to_one.
```

Sample Code

Numeric Syntax

```
DATA(lo_zero_to_one) = xco_cp_cds=>cardinality->zero_to_one.
DATA(lo_one) = xco_cp_cds=>cardinality->one.
DATA(lo_one_to_one) = xco_cp_cds=>cardinality->one_to_one.
DATA(lo_zero_to_n) = xco_cp_cds=>cardinality->zero_to_n.
DATA(lo_one_to_n) = xco_cp_cds=>cardinality->one_to_n.
```

4.2.15.18.3.7 Correction and Transport System

The XCO CTS module provides APIs which support the creation and release of Workbench and Customizing transports and integrates seamlessly into the XCO ABAP Repository module to support locating objects contained on transports.

Creation and releasing transport requests programmatically is especially useful in combination with the XCO Generation APIs to build completely self-contained generation programs.

Creating and releasing Workbench transport requests

A prerequisite for creating a new Workbench transport is a valid transport target which is usually derived from the structural package for which objects are supposed to be created, changed or deleted in a corresponding development subpackage.

Using the XCO ABAP Repository APIs the transport layer and transport target associated with a given structure package can be determined easily:

Sample Code

```
DATA(ls_package) = xco_cp_abap_repository=>package-
>for( 'ZMY_STRUCTURE_PACKAGE'
  )->read( ).
DATA(lv_transport_layer) = ls_package-property-transport_layer->value.
DATA(lv_transport_target) = ls_package-property-transport_layer-
>get_transport_target( )->value.
```

Once the correct transport target has been determined a new transport request can be created like this:

Sample Code

```
DATA(lo_transport_request) = xco_cp_cts=>transports-
>workbench( lv_transport_target
```

```
  )->create_request( 'My generated Workbench transport request' ).
```

This statement will create a new workbench transport request with one unclassified task in the name of the active user. Once all required changes have been recorded on the tasks of the transport request releasing the transport request along with all open tasks is performed as follows:

Sample Code

```
DATA(lt_transport_tasks) = lo_transport_request->get_tasks( ).
LOOP AT lt_transport_tasks INTO DATA(lo_transport_task).
  CHECK lo_transport_task->get_status( ) EQ xco_cp_transport=>status-
>modifiable.
  lo_transport_task->release( ).
ENDLOOP.
lo_transport_request->release( ).
```

When releasing a transport, it's also possible to specify dedicated release options that should be considered when the transport is released. Transport release options are accessible via `XCO_CP_TRANSPORT=>RELEASE_OPTION` and as of now, the following transport release options are offered:

- `IGNORE_OBJECTS_CHECK`: If non-critical issues are detected with objects contained on the transport (such as issues due to ATC violations), this option allows you to ignore these issues and release the transport anyways. Setting this option has an effect equivalent to explicitly confirming the release of the transport when an interactive tool, such as the ADT Transport Organizer, is used to release the transport.

Releasing a transport request with the ignore objects check option can be done like this:

Sample Code

```
DATA(lo_ignore_objects_check_option) = xco_cp_transport=>release_option-
>ignore_objects_check( ).
lo_transport_request->release( VALUE
#( ( lo_ignore_objects_check_option ) ) ).
```

Besides being able to easily release transport tasks and requests, it is also possible to protect and unprotect transport requests:

Sample Code

```
" Protect a transport request.
lo_transport_request->protect( ).
" Unprotect a transport request.
lo_transport_request->unprotect( ).
```

Creating Customizing transport requests

A Customizing transport request for a given transport target can be created as follows:

Sample Code

```
DATA(lo_transport_request) = xco_cp_cts=>transports-
>customizing( '$TRANSPORT_TARGET$'
```

```
  )->create_request( 'My generated Customizing transport request' ).
```

Once created, a Customizing transport request can be further used in the context of Business Configuration, e.g. the *Export Customizing Transports* Fiori app.

Querying and reading transports

Within the XCO Library, a transport (IF_XCO_CP_TRANSPORT) represents either a transport request or a task of a transport request. One of the characteristic properties of a transport is that it always has an associated request which is either the transport itself in case it already represents a transport request or otherwise the transport request of the task represented by the transport.

In a style similar to the ABAP Repository Query APIs it is possible to efficiently obtain a list of transports matching user-defined filters. Filters are obtained via the XCO_CP_TRANSPORT=>FILTER API and currently cover the following aspects of transports:

- Kind (Request or Task)
- Owner
- Request
- Target of the request
- Type of the request
- Status
- Type
- Attributes

Specific to the CTS transport Query APIs is the concept of a resolution that is applied to a transport query. A resolution defines a mapping of the matched transports to the final list of transports. Currently, the following resolutions are offered:

- Request: Map each matched transport to its associated request
- Identity: Map each matched transport to itself

As a first example, obtaining a list of transport requests for a given target on which the current user has at least one modifiable task can be accomplished like:

Sample Code

```
DATA(lo_current_user) = xco_cp=>sy->user( ).
DATA(lo_kind_filter) = xco_cp_transport=>filter->kind( xco_cp_transport=>kind-
>task ).
DATA(lo_owner_filter) = xco_cp_transport=>filter-
>owner( xco_cp_abap_sql=>constraint->equal( lo_current_user->name ) ).
DATA(lo_request_target_filter) = xco_cp_transport=>filter-
>request_target( xco_cp_abap_sql=>constraint->equal( 'ZX11' ) ).
DATA(lo_status_filter) = xco_cp_transport=>filter-
>status( xco_cp_transport=>status->modifiable ).
DATA(lt_transports) = xco_cp_cts=>transports->where( VALUE #(
  ( lo_kind_filter )
  ( lo_owner_filter )
  ( lo_request_target_filter )
  ( lo_status_filter )
) )->resolve( xco_cp_transport=>resolution->request ).
LOOP AT lt_transports INTO DATA(lo_transport).
  DATA(lo_transport_request) = lo_transport->get_request( ).
```

```
  " LO_TRANSPORT is of type IF_XCO_CP_TRANSPORT. Based on the applied
resolution
  " it is guaranteed that every LO_TRANSPORT object already represents a
request.
ENDLOOP.
```

As a second example, obtaining a list of all transport requests which are both released and have the attribute SAP_CUS_TRANSPORT_CATEGORY with the value DEFAULT_CUS can be accomplished like:

Sample Code

```
DATA(lo_attribute_filter) = xco_cp_transport=>filter->attribute(
  io_name_constraint  = xco_cp_abap_sql=>constraint-
>equal( 'SAP_CUS_TRANSPORT_CATEGORY' )
  io_value_constraint = xco_cp_abap_sql=>constraint->equal( 'DEFAULT_CUS' )
).
DATA(lo_status_filter) = xco_cp_transport=>filter-
>status( xco_cp_transport=>status->released ).
DATA(lt_transport_requests) = xco_cp_cts=>transports->where( VALUE #(
  ( lo_attribute_filter )
  ( lo_status_filter )
) )->resolve( xco_cp_transport=>resolution->identity ).
LOOP AT lt_transport_requests INTO DATA(lo_transport_request).
  " LV_TRANSPORT_REQUEST will be a transport request which is both in status
"Released"
  " and has attribute SAP_CUS_TRANSPORT_CATEGORY set to value DEFAULT_CUS.
  DATA(lv_transport_request) = lo_transport_request->value.
ENDLOOP.
```

Integration with the ABAP Repository APIs

The XCO CTS module integrates directly into the XCO ABAP Repository module which allows to take advantage of the powerful filtering mechanisms provided by the XCO ABAP Repository module. To this extent, the type IF_XCO_CP_TRANSPORT implements the IF_XCO_AR_OBJECT_SOURCE interface.

With this integration, locating all domains on a given transport is as easy as

Sample Code

```
DATA(lo_transport) = xco_cp_cts=>transport->for( '...' ).
DATA(lt_domains) = xco_cp_abap_repository=>objects->doma->all-
>in( lo_transport )->get( ).
```

All filters that can be applied when objects are being queried in a package or the whole ABAP Repository can also be applied when a transport is used as the object source.

Conversely, it is also possible to check if a given ABAP Repository object is currently locked on a transport request:

Sample Code

```
DATA(lo_domain) = xco_cp_abap_repository=>object->doma->for( 'ZMY_DOMAIN' ).
IF lo_domain->if_xco_cts_changeable~get_object( )->is_locked( ) EQ abap_true.
  DATA(lv_transport) = lo_domain->if_xco_cts_changeable~get_object(
```

```
    )->get_lock(
    )->get_transport( ).
  " LO_TRANSPORT_REQUEST will be the transport request that ZMY_DOMAIN is
  " currently locked on (if it is locked at all).
  DATA(lo_transport_request) = xco_cp_cts=>transport->for( lv_transport
    )->get_request( ).
ENDIF.
```

4.2.15.18.3.7.1 Transport Read APIs

Reading properties of a transport

Given a transport request (represented as an object of type `IF_XCO_CP_TR_REQUEST`), its properties can be read out as follows:

Sample Code

```
DATA(ls_properties) = lo_transport_request->properties( )->get( ).

" LS_PROPERTIES is a structure providing the short description, owner, target,
" status and last changed moment of the transport request.
```

The properties of a transport task (represented as an object of type `IF_XCO_CP_TR_TASK`) can similarly be read out like this:

Sample Code

```
DATA(ls_properties) = lo_transport_task->properties( )->get( ).

" LS_PROPERTIES is a structure providing the short description, owner,
" status and last changed moment of the transport task.
```

Properties of both transport requests and transport tasks can also be retrieved individually by using one of the specialized `GET_` methods, e.g. `GET_SHORT_DESCRIPTION`, instead of the general `GET` method which retrieves all available properties.

Reading attributes of a transport request

Given a transport request (represented as an object of the type IF_XCO_CP_TR_REQUEST), its attributes can be read out as follows:

Sample Code

```
DATA(lt_attributes) = lo_transport_request->attributes->all->get( ).
LOOP AT lt_attributes INTO DATA(lo_attribute).
  " The name of the attribute (as a CHAR of length 30).
  DATA(lv_name) = lo_attribute->get_attribute( )->name.
  " The value of the attribute (as a CHAR of length 32).
  DATA(lv_value) = lo_attribute->get_value( ).
```

```
ENDLOOP.
```

Reading entries of a transport

A transport can contain two kinds of entries:

- Object entries: An object entry of a transport is identified by its values for Program ID (R3TR, LIMU or LANG), its object type and object name
- Key entries: A key entry of a transport identifies one or several rows in a database table that are to be transported once the transport is released. A key entry always has a corresponding object entry on the same transport

Both object and key entries of a transport can easily be retrieved via the XCO transport module. The starting point for retrieving entries of a transport is `XCO_CP_TRANSPORT->ENTRIES`.

Reading object entries

The following code sample shows how to read all object entries contained on a given transport:

Sample Code

```
" Get all object entries on a transport (given as an object LO_TRANSPORT of
type IF_XCO_CP_TRANSPORT).
DATA(lt_object_entries) = xco_cp_transport=>entries->object->all-
>on( lo_transport )->get( ).

LOOP AT lt_object_entries INTO DATA(lo_object_entry).
  " The Program ID of the object entry represented as an XCO enumeration
constant
  " of type CL_XCO_CTS_PROGRAM_ID. The primitive value for the program ID can
be
  " obtained via LO_PROGRAM_ID->VALUE.
  DATA(lo_program_id) = lo_object_entry->object->program_id.

  " The object type of the object entry (e.g. DOMA for a domain of the ABAP
Dictionary).
  DATA(lv_object_type) = lo_object_entry->object->type.

  " The object name of the object entry (e.g. the name of the domain if the
object entry
  " refers to a domain of the ABAP Dictionary).
  DATA(lv_object_name) = lo_object_entry->object->name.
ENDLOOP.
```

In addition to retrieving all objects it is furthermore possible to provide one or more entry filters to restrict the object entries to those matching the provided entry filters:

Sample Code

```
" The program ID filter will only match object entries where the program ID
is LIMU.
DATA(lo_program_id_filter) = xco_cp_transport=>entry_filter->object-
>program_id(
  xco_cp_cts=>program_id->limu
).
```

```
" The object type filter will only match object entries where the object type
is FUNC.
DATA(lo_object_type_filter) = xco_cp_transport=>entry_filter->object-
>object_type(
  xco_cp_abap_sql=>constraint->equal( 'FUNC' )
).

" The object name filter will only match object entries where the object name
starts with Z.
DATA(lo_object_name_filter) = xco_cp_transport=>entry_filter->object-
>object_name(
  xco_cp_abap_sql=>constraint->contains_pattern( 'Z%' )
).

" LT_OBJECT_ENTRIES will contain all LIMU FUNC object entries on LO_TRANSPORT
where the object name starts with Z
" (where LO_TRANSPORT is an object of type IF_XCO_CP_TRANSPORT).
DATA(lt_object_entries) = xco_cp_transport=>entries->object->where( VALUE #(
  ( lo_program_id_filter )
  ( lo_object_type_filter )
  ( lo_object_name_filter )
) )->on( lo_transport )->get( ).
```

Reading key entries

Reading key entries on a transport works in a similar manner as when object entries are being read. Obtaining all key entries on a given transport can be accomplished like this:

Sample Code

```
" Get all key entries on a transport (given as an object LO_TRANSPORT of type
IF_XCO_CP_TRANSPORT).
DATA(lt_key_entries) = xco_cp_transport=>entries->key->all-
>on( lo_transport )->get( ).

LOOP AT lt_key_entries INTO DATA(lo_key_entry).
  " Get the object entry to which this key entry belongs.
  DATA(lo_object_entry) = lo_key_entry->get_object_entry( ).

  " Get the table key for the entry which can be used to subsequently retrieve
  " the column values used to select the database table entries that shall be
  " transported.
  DATA(lo_table_key) = lo_key_entry->get_table_key( ).

  DATA(lt_table_key_components) = lo_table_key->get_components( ).

  LOOP AT lt_table_key_components INTO DATA(lo_table_key_component).
    " The name of the field in the underlying database table.
    DATA(lv_field_name) = lo_table_key_component->field_name.

    " The value which is used for this field. The WRITE_TO method of
    " LO_VALUE can be used to write the value for the field to an actual
    " data object.
    DATA(lo_value) = lo_table_key_component->get_value( ).

    " The type of the value depends on the type of the field in the database
table.
    " In the case where it is of type C of length 10, the actual value can be
    " obtained like this:
    DATA lv_value TYPE c LENGTH 10.

    lo_value->write_to( REF #( lv_value ) ).
  ENDLOOP.
ENDLOOP.
```

An R3TR TABU object entry always has exactly one corresponding key entry. Thanks to a direct integration of R3TR TABUs with the entry query APIs, all key entries coming from R3TR TABU object entries can be conveniently retrieved as follows:

Sample Code

```
" Get all R3TR TABU entries on a transport (given as an object LO_TRANSPORT
of type IF_XCO_CP_TRANSPORT).
DATA(lt_r3tr_tabu_entries) = xco_cp_transport=>entries->object->r3tr->tabu-
>all->on( lo_transport )->get( ).

LOOP AT lt_r3tr_tabu_entries INTO DATA(lo_r3tr_tabu_entry).
  " The key entry corresponding to this R3TR TABU object entry.
  DATA(lo_key_entry) = lo_r3tr_tabu_entry->get_key_entry( ).
ENDLOOP.
```

4.2.15.18.3.8 Object Type References

4.2.15.18.3.8.1 CDS Type Definitions

Find out how to create CDS type definitions.

CDS type definitions, meaning objects of type `DRTY`, are supported by the following API families within the XCO library:

- Query APIs
- Read APIs
- Generation APIs

Query APIs

Obtaining an object handle for a CDS type definition with a given name can be accomplished via

Sample Code

```
DATA(lo_cds_type_definition) = xco_cp_abap_repository=>object->drty-
>for( '$CDS_TYPE_DEFINITION_NAME$' ).
```

The obtained object handle can then be used to, for example, check the existence of the CDS type definition in the ABAP repository (via method `IF_XCO_AR_OBJECT~EXISTS`) or read out its content via the Read APIs (see below). Obtaining a list of all visible CDS type definitions in the ABAP repository can be done via

Sample Code

```
DATA(lt_cds_type_definitions) = xco_cp_abap_repository=>objects->drty->all-
>in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_cds_type_definitions ASSIGNING FIELD-
SYMBOL(<fs_cds_type_definition>).
  " Process the found CDS type definition.
ENDLOOP.
```

Instead of searching for CDS type definitions in the entire ABAP repository, it's also possible to only search for CDS type definitions in a dedicated subset of the ABAP repository, such as in a single package:

Sample Code

```
DATA(lo_package) = xco_cp_abap_repository=>package->for( '$PACKAGE_NAME$' ).

DATA(lt_cds_type_definitions) = xco_cp_abap_repository=>objects->drty->all-
>in( lo_package )->get( ).

LOOP AT lt_cds_type_definitions ASSIGNING FIELD-
SYMBOL(<fs_cds_type_definition>).
  " Process the found CDS type definition.
ENDLOOP.
```

Furthermore, you can refine a search for CDS type definitions by using filters to specify exactly which CDS type definitions should be matched, for example:

Sample Code

```
DATA(lo_object_name_filter) = xco_cp_abap_repository=>object_name->get_filter(
  xco_cp_abap_sql=>constraint->contains_pattern( '/ABC/%' )
).

" LT_CDS_TYPE_DEFINITIONS will be a list of all visible CDS type definitions
" in the ABAP Repository whose name starts with /ABC/.
DATA(lt_cds_type_definitions) = xco_cp_abap_repository=>objects->drty-
>where( VALUE #(
  ( lo_object_name_filter )
) )->in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_cds_type_definitions ASSIGNING FIELD-
SYMBOL(<fs_cds_type_definition>).
  " Process the found CDS type definition.
ENDLOOP.
```

Read APIs

Reading out the content and checking the existence of a CDS type definition is possible via its object handle `IF_XCO_CDS_TYPE_DEFINITION`. The Read APIs for CDS type definitions are based on the origin infrastructure with the following origins being offered currently for CDS type definitions:

- **DEFAULT**: Can be obtained via `XCO_CP_CDS_TYPE_DEFINITION=>ORIGIN→DEFAULT` and will read a given CDS type definition exactly once per ABAP session from the local system, with all subsequent reads

of the CDS type definition being fulfilled from a dedicated buffer. This origin is used by default, meaning if no origin is explicitly specified when reading the content or checking the existence of a CDS type definition.

- **LOCAL**: Can be obtained via `XCO_CP_CDS_TYPE_DEFINITION=>ORIGIN→LOCAL` and will read a given CDS type definition from the local system. By default, every read of the content as well as existence checks of a CDS type definition will read directly from the database, but the caching behavior can be configured via the methods of `IF_XCO_DRTY_CACHEABLE_ORIGIN`.

Next, when reading a CDS type definition or checking its existence, it can explicitly be specified which version of the CDS type definition should be considered. The following versions are supported:

- **ACTIVE**: Can be obtained via `XCO_CP_CDS_TYPE_DEFINITION=>VERSION→ACTIVE`. The active version is used by default, meaning if no version is explicitly specified when reading the content of a CDS type definition or checking its existence.
- **INACTIVE**: Can be obtained via `XCO_CP_CDS_TYPE_DEFINITION=>VERSION→INACTIVE`.

Following the general style of the XCO Read APIs, each part of a CDS type definition can be read out separately:

Header

Reading out the header of a CDS type definition can be done like this:

Sample Code

```
DATA(lo_cds_type_definition) = xco_cp_abap_repository=>object->drty-
>for( '$CDS_TYPE_DEFINITION_NAME$' ).

" Read out the complete header of the active version of the CDS type
" definition using the default origin:
DATA(ls_header) = lo_cds_type_definition->content( )->get( ).

" Read out only the short description of the inactive version of the CDS
" type definition using an explicitly obtained local origin (without
" caching).
DATA(lo_local_origin) = xco_cp_cds_type_definition=>origin->local( ).

DATA(lv_short_description) = lo_cds_type_definition-
>content( xco_cp_cds_type_definition=>version->inactive
  )->get_short_description( lo_local_origin ).
```

Generation APIs

CDS type definitions are integrated with the XCO Generation APIs with the following operations being offered currently for CDS type definitions:

- **PUT**: Create or update a CDS type definition based on an entire content specification so that after running the operation successfully, the CDS type definition exists and its content is exactly as specified
- **DELETE**: Delete a CDS type definition (if it exists)

PUT Operations

A form specification is used to specify the entire content of a CDS type definition, which is the basis for running a PUT operation. Upon running the PUT operation, depending on whether the CDS type definition already exists or not, it will either be newly created, or updated. This is so that after the successful completion of the PUT operation, the CDS type definition exists with the content resembling exactly what was specified in the

provided form specification. Consider the following code sample which illustrates how to run a PUT operation for a CDS type definition:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-drty-
>add_object( '$CDS_TYPE_DEFINITION_NAME$'
  )->set_package( '$PACKAGE_NAME$'
  )->create_form_specification( ).

lo_form_specification->set_short_description( 'My short description' ).

DATA(lo_simple_type_definition) = lo_form_specification-
>add_simple_type_definition( ).
lo_simple_type_definition->add_annotation( 'EndUserText.label'
  )->value->build(
  )->add_string( 'My simple type definition' ).
lo_simple_type_definition->set_type( xco_cp_abap_dictionary=>built_in_type-
>char( 30 ) ).

lo_put_operation->execute( ).
```

DELETE Operations

CDS type definitions can be deleted via DELETE operations. As per the standard semantics of DELETE operations, if the CDS type definition for which a DELETE operation is being run doesn't exist, no action will be performed (and no error will be raised). If the CDS type definition does exist, it will be deleted. The code sample below illustrates how to run a DELETE operation for a CDS type definition:

Sample Code

```
DATA(lo_delete_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->for-drty->create_delete_operation( ).

lo_delete_operation->add_object( '$CDS_TYPE_DEFINITION_NAME$' ).

lo_delete_operation->execute( ).
```

4.2.15.18.3.8.2 Data Definitions

Find out how to create data definitions.

Data definitions, also called objects of type `DDLS`, are supported by the following API families within the *XCO Library*:

- Query APIs
- Read APIs
- Generation APIs

In addition to the standard API families listed above, dedicated APIs are offered for the following aspects:

- Setting and getting the API state of a CDS entity

Query APIs

Obtaining an object handle for a data definition with a given name can be accomplished via

Sample Code

```
DATA(lo_data_definition) = xco_cp_abap_repository=>object->ddls-
>for( '$DATA_DEFINITION_NAME$' ).
```

The obtained object handle can then be used to check the existence of the data definition in the ABAP Repository (via method `IF_XCO_AR_OBJECT~EXISTS`) or read out its content via the Read APIs (see below). Obtaining a list of all visible data definitions in the ABAP Repository can simply be accomplished via

Sample Code

```
DATA(lt_data_definitions) = xco_cp_abap_repository=>objects->ddls->all-
>in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_data_definitions ASSIGNING FIELD-SYMBOL(<fs_data_definition>).
  " Process the found data definition.
ENDLOOP.
```

Instead of searching for data definitions in the entire ABAP Repository, it's also possible to only search for data definitions in a dedicated subset of the ABAP Repository, such as a single package:

Sample Code

```
DATA(lo_package) = xco_cp_abap_repository=>package->for( '$PACKAGE_NAME$' ).

DATA(lt_data_definitions) = xco_cp_abap_repository=>objects->ddls->all-
>in( lo_package )->get( ).

LOOP AT lt_data_definitions ASSIGNING FIELD-SYMBOL(<fs_data_definition>).
  " Process the found data definition.
ENDLOOP.
```

Another way to refine a search for data definitions is to use filters to specify exactly which data definitions shall be matched, for example:

Sample Code

```
DATA(lo_object_name_filter) = xco_cp_abap_repository=>object_name->get_filter(
  xco_cp_abap_sql=>constraint->contains_pattern( '/ABC/%' )
).

" LT_DATA_DEFINITIONS will be a list of all visible data definitions in the
ABAP
" Repository whose name starts with /ABC/.
DATA(lt_data_definitions) = xco_cp_abap_repository=>objects->ddls-
>where( VALUE #(
  ( lo_object_name_filter )
) )->in( xco_cp_abap=>repository )->get( ).
```

```
LOOP AT lt_data_definitions ASSIGNING FIELD-SYMBOL(<fs_data_definition>).
  " Process the found data definition.
ENDLOOP.
```

Read APIs

Reading out the content and checking the existence of a data definition is possible via its object handle `IF_XCO_DATA_DEFINITION`, respectively the corresponding interfaces for each supported data definition type:

- View entity: `IF_XCO_CDS_VIEW_ENTITY`
- Project view: `IF_XCO_CDS_PROJECTION_VIEW`
- Abstract entity: `IF_XCO_CDS_ABSTRACT_ENTITY`
- Custom entity: `IF_XCO_CDS_CUSTOM_ENTITY`
- Table function: `IF_XCO_CDS_TABLE_FUNCTION`

When reading the content of a CDS entity, the read state for which the content shall be read can explicitly be specified. The following read states are supported:

- *ACTIVE*: Can be obtained via `XCO_CP_DATA_DEFINITION=>OBJECT_READ_STATE→ACTIVE`. The active read state is used by default, so if no read state is explicitly specified when reading the content of a data definition or checking its existence
- *LATEST*: Can be obtained via `XCO_CP_DATA_DEFINITION=>OBJECT_READ_STATE→LATEST`

Following the general style of the XCO Read APIs, each part of a CDS entity can be read out separately:

View Entities

The following code sample illustrates how the (structured) content of a view entity can be read out:

Sample Code

```
DATA(lo_view_entity) = xco_cp_cds=>view_entity( '$VIEW_ENTITY_NAME$' ).

" The (structured) header content.
DATA(ls_header) = lo_view_entity->content( )->get( ).

" Read out the parameters of the view entity.
DATA(lt_parameters) = lo_view_entity->parameters->all->get( ).

LOOP AT lt_parameters ASSIGNING FIELD-SYMBOL(<fs_parameter>).
  " The (structured) content of the parameter.
  DATA(ls_parameter) = <fs_parameter>->content( )->get( ).
ENDLOOP.

" Read out the associations of the view entity.
DATA(lt_associations) = lo_view_entity->associations->all->get( ).

LOOP AT lt_associations ASSIGNING FIELD-SYMBOL(<fs_association>).
  " The (structured) content of the association.
  DATA(ls_association) = <fs_association>->content( )->get( ).
ENDLOOP.

" Read out the compositions of the view entity.
DATA(lt_compositions) = lo_view_entity->compositions->all->get( ).
```

```
LOOP AT lt_compositions ASSIGNING FIELD-SYMBOL(<fs_composition>).
  " The (structured) content of the composition.
  DATA(ls_composition) = <fs_composition>->content( )->get( ).
ENDLOOP.

" Read out the fields of the view entity.
DATA(lt_fields) = lo_view_entity->fields->all->get( ).

LOOP AT lt_fields ASSIGNING FIELD-SYMBOL(<fs_field>).
  " The (structured) content of the field.
  DATA(ls_field) = <fs_field>->content( )->get( ).
ENDLOOP.
```

Projection Views

The following code sample illustrates how the (structured) content of a projection view can be read out:

Sample Code

```
DATA(lo_projection_view) =
xco_cp_cds=>projection_view( '$PROJECTION_VIEW_NAME$' ).

" The (structured) header content.
DATA(ls_header) = lo_projection_view->content( )->get( ).

" Read out the fields of the projection view.
DATA(lt_fields) = lo_projection_view->fields->all->get( ).

LOOP AT lt_fields ASSIGNING FIELD-SYMBOL(<fs_field>).
  " The (structured) content of the field.
  DATA(ls_field) = <fs_field>->content( )->get( ).
ENDLOOP.
```

Abstract Entities

The following code sample illustrates how the (structured) content of an abstract entity can be read out:

Sample Code

```
DATA(lo_abstract_entity) =
xco_cp_cds=>abstract_entity( '$ABSTRACT_ENTITY_NAME$' ).

" The (structured) header content.
DATA(ls_header) = lo_abstract_entity->content( )->get( ).

" Read out the parameters of the abstract entity.
DATA(lt_parameters) = lo_abstract_entity->parameters->all->get( ).

LOOP AT lt_parameters ASSIGNING FIELD-SYMBOL(<fs_parameter>).
  " The (structured) content of the parameter.
  DATA(ls_parameter) = <fs_parameter>->content( )->get( ).
ENDLOOP.

" Read out the fields of the abstract entity.
DATA(lt_fields) = lo_abstract_entity->fields->all->get( ).

LOOP AT lt_fields ASSIGNING FIELD-SYMBOL(<fs_field>).
  " The (structured) content of the field.
  DATA(ls_field) = <fs_field>->content( )->get( ).
ENDLOOP.
```

Custom Entities

The following code sample illustrates how the (structured) content of a custom entity can be read out:

Sample Code

```
DATA(lo_custom_entity) = xco_cp_cds=>custom_entity( '$CUSTOM_ENTITY_NAME$' ).

" The (structured) header content.
DATA(ls_header) = lo_custom_entity->content( )->get( ).

" Read out the parameters of the custom entity.
DATA(lt_parameters) = lo_custom_entity->parameters->all->get( ).

LOOP AT lt_parameters ASSIGNING FIELD-SYMBOL(<fs_parameter>).
  " The (structured) content of the parameter.
  DATA(ls_parameter) = <fs_parameter>->content( )->get( ).
ENDLOOP.

" Read out the fields of the custom entity.
DATA(lt_fields) = lo_custom_entity->fields->all->get( ).

LOOP AT lt_fields ASSIGNING FIELD-SYMBOL(<fs_field>).
  " The (structured) content of the field.
  DATA(ls_field) = <fs_field>->content( )->get( ).
ENDLOOP.
```

Table Functions

The following code sample illustrates how the (structured) content of a table function can be read out:

Sample Code

```
DATA(lo_table_function) =
xco_cp_cds=>table_function( '$TABLE_FUNCTION_NAME$' ).

" The (structured) header content.
DATA(ls_header) = lo_table_function->content( )->get( ).

" Read out the parameters of the table_function.
DATA(lt_parameters) = lo_table_function->parameters->all->get( ).

LOOP AT lt_parameters ASSIGNING FIELD-SYMBOL(<fs_parameter>).
  " The (structured) content of the parameter.
  DATA(ls_parameter) = <fs_parameter>->content( )->get( ).
ENDLOOP.

" Read out the fields of the table function.
DATA(lt_fields) = lo_table_function->fields->all->get( ).

LOOP AT lt_fields ASSIGNING FIELD-SYMBOL(<fs_field>).
  " The (structured) content of the field.
  DATA(ls_field) = <fs_field>->content( )->get( ).
ENDLOOP.
```

Generation APIs

Data definitions are integrated with XCO Generation APIs. The following operations are currently offered:

- *PUT*: Create or update a data definition based on a complete specification of its content, so that after the successful execution of the operation, the data definition exists and its content is exactly as specified. Both form specifications as well as source templates can be used to provide a complete content specification
- *DELETE*: Delete a data definition (if it exists)

PUT Operations

Upon running the PUT operation, depending on whether the data definition already exists or not, it will either be newly created or updated. This is done so that after the successful completion of the PUT operation, the data definition exists with the content resembling exactly what was specified in the provided content specification. In addition to the traditional form specification, data definitions also support the usage of source templates to specify the source for the data definition directly.

> **Note**
>
> Note that the data definition examples given below are intended exclusively to illustrate how the respective facilities of the XCO Generation APIs can be used to programmatically create (or update) data definitions. The examples are chosen in a way that syntactically error-free data definitions are generated and are not meant to be real-world modelling examples. In particular, note that certain examples, such as those for projection views, might not correlate with general best modelling practices.

View Entity Form Specification

The content of a view entity can be specified via a form specification. The following code sample illustrates how a simple view entity (based on `I_Language` for the purposes of this example) can be generated:

> **Sample Code**

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-ddls-
>add_object( '$VIEW_ENTITY_NAME$'
  )->set_package( '$PACKAGE_NAME$'
  )->create_form_specification( ).
lo_form_specification->set_short_description( 'Generated view entity' ).

DATA(lo_view_entity) = lo_form_specification->add_view_entity( ).
lo_view_entity->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( |My view entity| ).
lo_view_entity->set_root( ).
lo_view_entity->data_source->set_view_entity( 'I_Language' ).

DATA(lo_parameter) = lo_view_entity->add_parameter( 'myParameter' ).
lo_parameter->set_data_type( xco_cp_abap_dictionary=>built_in_type-
>char( 10 ) ).
lo_parameter->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( |My parameter| ).

DATA(lo_field) = lo_view_entity->add_field( xco_cp_ddl=>field( 'Language' )
  )->set_key(
  )->set_alias( 'myKeyField' ).
lo_field->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( |My key field| ).
```

```
lo_field->add_comment( xco_cp=>string( 'This is a key field' ) ).
lo_put_operation->execute( ).
```

After successfully running the above `PUT` operation, a view entity with the name `$VIEW_ENTITY_NAME$` will exist in an active version with the following (source) content:

Sample Code

```
@EndUserText.label: 'My view entity'
define root view entity $VIEW_ENTITY_NAME$
  with parameters
    @EndUserText.label: 'My parameter'
    myParameter : abap.char( 10 )
  as select from I_Language
{
  @EndUserText.label: 'My key field'
  key Language as myKeyField

}
```

Projection View Form Specification

The content of a projection view can be specified via a form specification. The following code sample illustrates how a simple projection view (based on `I_Language` and modelled as an analytical query for the purposes of this example) can be generated:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-ddls-
>add_object( '$PROJECTION_VIEW_NAME$'
  )->set_package( '$PACKAGE_NAME$'
  )->create_form_specification( ).
lo_form_specification->set_short_description( 'Generated projection view' ).

DATA(lo_projection_view) = lo_form_specification->add_projection_view( ).
lo_projection_view->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( |My projection view| ).
lo_projection_view->add_annotation( 'AccessControl.authorizationCheck' )-
>value->build( )->add_enum( 'NOT_ALLOWED' ).
lo_projection_view->set_transient( ).
lo_projection_view->set_provider_contract( xco_cp_cds=>provider_contract-
>analytical_query ).
lo_projection_view->data_source->set_view_entity( 'I_Language' ).

DATA(lo_field) = lo_projection_view-
>add_field( xco_cp_ddl=>field( 'Language' )
  )->set_alias( 'myField' ).
lo_field->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( |My field| ).

lo_put_operation->execute( ).
```

After the successful execution of the above PUT operation, a projection view with the name $PROJECTION_VIEW_NAME$ will exist in active version with the following (source) content:

Sample Code

```
@EndUserText.label: 'My projection view'
@AccessControl.authorizationCheck: #NOT_ALLOWED
define transient view entity $PROJECTION_VIEW_NAME$
  provider contract analytical_query
  as projection on I_Language
{
  @EndUserText.label: 'My field'
  Language as myField

}
```

Abstract Entity Form Specification

The content of an abstract entity can be specified via a form specification. The following code sample illustrates how a simple abstract entity can be generated:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-ddls-
>add_object( '$ABSTRACT_ENTITY_NAME$'
  )->set_package( '$PACKAGE_NAME$'
  )->create_form_specification( ).
lo_form_specification->set_short_description( 'Generated abstract entity' ).

DATA(lo_abstract_entity) = lo_form_specification->add_abstract_entity( ).
lo_abstract_entity->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( |My abstract entity| ).

DATA(lo_parameter) = lo_abstract_entity->add_parameter( 'MY_PARAMETER' ).
lo_parameter->set_data_type( xco_cp_abap_dictionary=>built_in_type-
>char( 10 ) ).
lo_parameter->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( |My parameter| ).

DATA(lo_field) = lo_abstract_entity-
>add_field( xco_cp_ddl=>field( 'MY_FIELD' ) ).
lo_field->set_type( xco_cp_abap_dictionary=>data_element( 'SPRAS' ) ).
lo_field->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( |My field| ).

lo_put_operation->execute( ).
```

After successfully running the above `PUT` operation, an abstract entity with the name `$ABSTRACT_ENTITY_NAME$` will exist in an active version with the following (source) content:

Sample Code

```
@EndUserText.label: 'My abstract entity'
define abstract entity $ABSTRACT_ENTITY_NAME$
  with parameters
    @EndUserText.label: 'My parameter'
    MY_PARAMETER : abap.char( 10 )
{
  @EndUserText.label: 'My field'
  MY_FIELD : SPRAS;
```

```
}
```

Custom Entity Form Specification

The content of a custom entity can be specified via a form specification. The following code sample illustrates how a simple custom entity can be generated:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-ddls-
>add_object( '$CUSTOM_ENTITY_NAME$'
  )->set_package( '$PACKAGE_NAME$'
  )->create_form_specification( ).
lo_form_specification->set_short_description( 'Generated custom entity' ).

DATA(lo_custom_entity) = lo_form_specification->add_custom_entity( ).
lo_custom_entity->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( |My custom entity| ).

DATA(lo_parameter) = lo_custom_entity->add_parameter( 'myParameter' ).
lo_parameter->set_data_type( xco_cp_abap_dictionary=>built_in_type-
>char( 10 ) ).
lo_parameter->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( |My parameter| ).

DATA(lo_key_field) = lo_custom_entity-
>add_field( xco_cp_ddl=>field( 'myKeyField' ) ).
lo_key_field->set_key( ).
lo_key_field->set_type( xco_cp_abap_dictionary=>built_in_type->char( 10 ) ).
lo_key_field->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( |My key field| ).

DATA(lo_data_field) = lo_custom_entity-
>add_field( xco_cp_ddl=>field( 'myDataField' ) ).
lo_data_field->set_type( xco_cp_abap_dictionary=>data_element( 'MSEHI' ) ).
lo_data_field->add_annotation( 'EndUserText.label' )->value->build( )-
>add_string( |My data field| ).

lo_put_operation->execute( ).
```

After successful execution, a custom entity with the name `$CUSTOM_ENTITY_NAME$` will exist in an active version with the following (source) content:

Sample Code

```
@EndUserText.label: 'My abstract entity'
define custom entity $CUSTOM_ENTITY_NAME$
  with parameters
    @EndUserText.label: 'My parameter'
    myParameter : abap.char( 10 )
{
  @EndUserText.label: 'My key field'
  key myKeyField : abap.char( 10 );
  @EndUserText.label: 'My data field'
  myDataField : MSEHI;

}
```

Table Function Form Specification

The content of a table function can be specified via a form specification. The following code sample illustrates how a simple table function can be generated:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-ddls-
>add_object( '$TABLE_FUNCTION_NAME$'
  )->set_package( '$PACKAGE_NAME$'
  )->create_form_specification( ).
lo_form_specification->set_short_description( 'Generated table function' ).

DATA(lo_table_function) = lo_form_specification->add_table_function( ).
lo_table_function->add_annotation( 'AccessControl.authorizationCheck'
  )->value->build(
  )->add_enum( 'NOT_REQUIRED' ).
lo_table_function->add_annotation( 'ClientHandling.type'
  )->value->set( xco_cp_cds_annotation=>value->enum( 'CLIENT_INDEPENDENT' ) ).

DATA(lo_parameter) = lo_table_function->add_parameter( 'p_identifier'
  )->set_data_type( xco_cp_abap_dictionary=>data_element( 'syuname' ) ).

DATA(lo_field) = lo_table_function-
>add_field( xco_cp_ddl=>field( 'val_identifier' ) ).
lo_field->set_type( xco_cp_abap_dictionary=>built_in_type->char( 30 ) ).

lo_field->add_annotation( 'EndUserText.label'
  )->value->set( xco_cp_cds_annotation=>value->string( 'Identifier field' ) )
##NO_TEXT.

lo_table_function->set_amdp_class( '$AMDP_CLASS_NAME$'
  )->set_amdp_method( '$AMDP_CLASS_METHOD$' ).

lo_put_operation->execute( ).
```

After successful execution, a table function with the name `$TABLE_FUNCTION_NAME$` will exist in an active version with the following (source) content:

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@ClientHandling.type: #CLIENT_INDEPENDENT
define table function $TABLE_FUNCTION_NAME$
  with parameters
    p_identifier : syuname
returns
{
  @EndUserText.label: 'Identifier field'
  val_identifier : abap.char( 30 );

}
implemented by method
  $AMDP_CLASS_NAME$=>$AMDP_METHOD_NAME$
```

Source Templates

As an alternative to providing the content of a data definition via a form specification, it's possible to use a source template where the source of the data definition can be passed as a plain string. The following example illustrates how this can be used to generate a simple abstract entity by supplying its complete source directly:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_source_template) = xco_cp_generation_ddls=>template->source( ).

DATA(lv_source) = xco_cp=>strings( VALUE #(
  ( `define abstract entity $ABSTRACT_ENTITY_NAME$` )
  ( `{` )
  ( `  // My field` )
  ( `  MY_FIELD : spras;` )
  ( `}` )
) )->join( |{ cl_abap_char_utilities=>cr_lf }| )->value.

lo_source_template->set_short_description( 'Generated abstract entity'
  )->set_source( lv_source ).

lo_put_operation->for-ddls->add_object( '$ABSTRACT_ENTITY_NAME$'
  )->set_package( '$PACKAGE_NAME$'
  )->set_template( lo_source_template ).

lo_put_operation->execute( ).
```

Once the `PUT` operation has run successfully, an abstract entity with the name `$ABSTRACT_ENTITY_NAME$` will exist in an active version with the following (source) content:

Sample Code

```
define abstract entity $ABSTRACT_ENTITY_NAME$
{
  // My field
  MY_FIELD : spras;
}
```

DELETE Operations

Data definitions (regardless of the type of entity they define) can be deleted via `DDLS DELETE` operations. As per the standard semantics of `DELETE` operations, if the data definition for which a `DELETE` operation is run doesn't exist, no action will be performed (and no error will be raised). If the data definition does exist, it will be deleted. The code sample below illustrates how to execute a `DELETE` operation for a data definition:

Sample Code

```
DATA(lo_delete_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST'
  )->for-ddls->create_delete_operation( ).

lo_delete_operation->add_object( '$DATA_DEFINITION_NAME$' ).

lo_delete_operation->execute( ).
```

Setting and Getting the API State of a CDS Entity

A CDS entity defined by a data definition can be released via the API state mechanisms of the *API Release Framework* (ARS). The *XCO Library* allows to programmatically set and get the API state of a CDS entity (represented as `IF_XCO_CDS_ENTITY`). The following code sample illustrates how the API state of a given CDS entity can be read:

Sample Code

```
DATA(lo_cds_entity) = xco_cp_cds=>entity( '$CDS_ENTITY_NAME$' ).

" Get the (current) API state of the CDS entity for the C1 release contract.
DATA(lo_api_state) = lo_cds_entity-
>get_api_state( xco_cp_ars=>release_contract->c1 ).

" The returned LO_API_STATE object allows to inspect the individual attributes
" of the API state.
DATA(lo_release_state) = lo_api_state->get_release_state( ).

" Release states supported by the XCO Library can be obtained via
XCO_CP_ARS=>RELEASE_STATE.
" Checking whether the API state of the CDS entity has the release state
"RELEASED"
" would look like this:
IF lo_release_state EQ xco_cp_ars=>release_state->released.
  " ...
ENDIF.

DATA(lt_visibilities) = lo_api_state->get_visibilities( ).

" Visibilities supported by the XCO Library can be obtained via
XCO_CP_ARS=>VISIBILITY.
" Note that due to compatibility and historical reasons, the visibility
" CLOUD_DEVELOPMENT is called SAP_CLOUD_PLATFORM within the XCO Library.
Checking
" whether the visibilities of the API state include CLOUD_DEVELOPMENT would
" then look like this:
IF line_exists( lt_visibilities[ table_line = xco_cp_ars=>visibility-
>sap_cloud_platform ] ).
  " ...
ENDIF.
```

Setting the API state for a CDS entity can be done like this:

Sample Code

```
" The API state which shall be set for the CDS entity is built via
XCO_CP_ARS=>API_STATE:
DATA(lo_api_state) = xco_cp_ars=>api_state->released( VALUE
#( ( xco_cp_ars=>visibility->sap_cloud_platform ) ) ).

" Setting the API state for a CDS entity implies a change which must be
recorded
" on an appropriate transport.
DATA(lo_transport) = xco_cp_cts=>transport->for( '$TRANSPORT$' ).

" The API state is set via the handle for the CDS entity.
DATA(lo_cds_entity) = xco_cp_cds=>entity( '$CDS_ENTITY_NAME$' ).

" We set the previously built API State (Released for Cloud Development) using
" the C1-release contract for the CDS entity (recording the changes on the
" provided transport).
lo_cds_entity->set_api_state(
```

```
  io_release_contract = xco_cp_ars=>release_contract->c1
  io_change_scenario  = lo_transport
  io_api_state        = lo_api_state
).
```

4.2.15.18.3.8.3 Domains

Find out how to create domains.

Domains, meaning objects of type `DOMA`, are supported by the following API families within the XCO library:

- Query APIs
- Read APIs
- Generation APIs

In addition to the standard API families listed above, dedicated APIs are offered for the following aspects:

- Seting and getting the API state of a domain

Query APIs

Obtaining an object handle for a domain with a given name can be accomplished via

Sample Code

```
DATA(lo_domain) = xco_cp_abap_repository=>object->doma-
>for( '$DOMAIN_NAME$' ).
```

The obtained object handle can then be used to, for examle, check the existence of the domain in the ABAP repository (via method `IF_XCO_AR_OBJECT~EXISTS` or read out its content via the Read APIs (see below). Obtaining a list of all visible domains in the ABAP repository can be done via

Sample Code

```
DATA(lt_domains) = xco_cp_abap_repository=>objects->doma->all-
>in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_domains ASSIGNING FIELD-SYMBOL(<fs_domain>).
  " Process the found domain.
ENDLOOP.
```

Instead of searching for domains in the entire ABAP repository, it's also possible to only search for domains in a dedicated subset of the ABAP repository, such as in a single package:

Sample Code

```
DATA(lo_package) = xco_cp_abap_repository=>package->for( '$PACKAGE_NAME$' ).

DATA(lt_domains) = xco_cp_abap_repository=>objects->doma->all-
>in( lo_package )->get( ).
```

```
LOOP AT lt_domains ASSIGNING FIELD-SYMBOL(<fs_domain>).
  " Process the found domain.
ENDLOOP.
```

Furthermore, you can refine a search for domains by using filters to specify exactly which domains should be matched, for example:

Sample Code

```
DATA(lo_object_name_filter) = xco_cp_abap_repository=>object_name->get_filter(
  xco_cp_abap_sql=>constraint->contains_pattern( '/ABC/%' )
).

" LT_DOMAINS will be a list of all visible domains in the ABAP Repository
" whose name starts with /ABC/.
DATA(lt_domains) = xco_cp_abap_repository=>objects->doma->where( VALUE #(
  ( lo_object_name_filter )
) )->in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_domains ASSIGNING FIELD-SYMBOL(<fs_domain>).
  " Process the found domain.
ENDLOOP.
```

Read APIs

Reading out the content and checking the existence of a domain is possible via its object handle `IF_XCO_DOMAIN`. The Read APIs for domains are based on the origin infrastructure with the following origins being offered currently for domains:

- **DEFAULT**: Can be obtained via `XCO_CP_DOMAIN=>ORIGIN→DEFAULT` and will read a given domain exactly once per ABAP session from the local system, with all subsequent reads of the domain being fulfilled from a dedicated buffer. This origin is used by default, meaning if no origin is explicitly specified when reading the content or checking the existence of a domain.
- **LOCAL**: Can be obtained via `XCO_CP_DOMAIN=>ORIGIN→LOCAL` and will read a given domain from the local system. By default, every read of the content as well as existence checks of a domain will read directly from the database, but the caching behavior can be configured via the methods of `IF_XCO_DOMAIN_CACHEABLE_ORIGIN`.

Next, when reading a domain or checking its existence, it can explicitly be specified which read state of the domain should be considered. The following read states are supported:

- **ACTIVE VERSION**: Can be obtained via `XCO_CP_ABAP_TYPE_DICTIONARY=>OBJECT_READ_STATE→ACTIVE_VERSION`. The active version is used by default, meaning if no read state is explicitly specified when reading the content of a domain or checking its existence.
- **NEWEST VERSION**: Can be obtained via `XCO_CP_ABAP_DICTIONARY=>OBJECT_READ_STATE→NEWEST_VERSION`.

Following the general style of the XCO Read APIs, each part of a domain can be read out separately:

Header

Reading out the header of a domain can be done like this:

Sample Code

```
DATA(lo_domain) = xco_cp_abap_repository=>object->doma-
>for( '$DOMAIN_NAME$' ).

" Read out the complete header of the active version of the domain using the
" default origin:
DATA(ls_header) = lo_domain->content( )->get( ).

" Read out only the short description of the newest version of the domain
" using an explicitly obtained local origin (without caching).
DATA(lo_local_origin) = xco_cp_domain=>origin->local( ).

DATA(lv_short_description) = lo_domain-
>content( xco_cp_abap_dictionary=>object_read_state->newest_version
  )->get_short_description( lo_local_origin ).
```

Fixed Values

Obtaining a list of the fixed values of a domain and reading out an individual fixed value can be done like this:

Sample Code

```
DATA(lo_domain) = xco_cp_abap_repository=>object->doma-
>for( '$DOMAIN_NAME$' ).

" Get a list of all the fixed values of the active version of the domain
" using the default origin:
DATA(lt_fixed_values) = lo_domain->fixed_values->all->get( ).

LOOP AT lt_fixed_values ASSIGNING FIELD-SYMBOL(<fs_fixed_value>).
  " Process the fixed value.
ENDLOOP.

" Obtain the handle for an individual fixed value.
DATA(lo_fixed_value) = lo_domain->fixed_value( 'A' ).

" Read out the complete content of the active version of the fixed value
" using the default origin.
DATA(ls_fixed_value) = lo_fixed_value->content( )->get( ).

" Read out only the description of the newest version of the fixed value
" using an explicitly obtained local origin with caching enabled.
DATA(lo_local_origin) = xco_cp_domain=>origin->local( ).
lo_local_origin->if_xco_domain_cacheable_origin~enable_caching( ).

DATA(lv_description) = lo_fixed_value-
>content( xco_cp_abap_dictionary=>object_read_state->newest_version
  )->get_description( lo_local_origin ).
```

Generation APIs

Domains are integrated with the XCO Generation APIs with the following operations being offered:

- **PUT**: Create or update a domain based on an entire content specification so that after running the operation successfully, the domain exists and its content is exactly as specified
- **PATCH**: Change an existing domain by applying the changes which have been provided in a change specification

- **DELETE**: Delete a domain (if it exists)

PUT Operations

Upon running a PUT operation, depending on whether the domain already exists or not, it will either be newly created or updated. This is so that after the successful completion of the PUT operation, the domain exists with the content resembling exactly what was specified in the provided content specification. In addition to the traditional form specification, domains also support the use of object templates to use the content of an existing domain as the content for the domain of the PUT operation.

Form specification

Consider the following code sample which illustrates how to run a PUT operation for a domain using a form specification:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-doma-
>add_object( '$DOMAIN_NAME$'
  )->set_package( '$PACKAGE_NAME$'
  )->create_form_specification( ).

lo_form_specification->set_short_description( 'My short description'
  )->set_format( xco_cp_abap_dictionary=>built_in_type->numc( 2 ) ).

DATA(lo_fixed_value_01) = lo_form_specification->fixed_values-
>add_fixed_value( '01' ).
lo_fixed_value_01->set_description( 'January' ).

lo_put_operation->execute( ).
```

Object template

An object template can be used to effectively copy an already existing domain to a new domain. The following code sample illustrates the use:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_object_template) = xco_cp_generation_doma=>template->for_domain(
  iv_name       = '$SOURCE_DOMAIN_NAME$'
  io_read_state = xco_cp_abap_dictionary=>object_read_state->active_version
).

lo_put_operation->for-doma->add_object( '$DOMAIN_NAME$'
  )->set_package( '$PACKAGE_NAME$'
  )->set_template( lo_object_template ).

lo_put_operation->execute( ).
```

PATCH Operations

A change specification is used to specify the changes that should be applied to an existing domain, and is the basis for running a PATCH operation for a domain. Upon running a PATCH operation, the changes will be applied to the domain. The following kinds of changes can be specified:

- **DELETE**: If the specified part exists in the domain, it'll be deleted. If it doesn't exist, no action will be performed.
- **INSERT**: If the specified part doesn't exist in the domain, it'll be inserted according to the provided specification. If it exists, no action will be performed.
- **UPDATE**: If the specified part exists in the domain, it'll be updated according to the provided specification.

Note that the different kinds of changes are always evaluated in the order Delete, Insert, Update with the changes that are applied for one kind being directly visible to the changes that are applied for a subsequent kind. This orchestration can be exploited to, for example, implement a MODIFY by specifying both a DELETE and an INSERT for a given object part. The following code example illustrates selected changes and how they can be applied via a (single) PATCH operation:

Sample Code

```
DATA(lo_patch_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_patch_operation( ).

DATA(lo_change_specification) = lo_patch_operation->for-doma-
>add_object( '$DOMAIN_NAME$'
  )->create_change_specification( ).

" Update the short description.
lo_change_specification->for-update->set_short_description( 'New short
description' ).

" Modify fixed version 02 by specifying both a deletion and an insertion for
" it.
lo_change_specification->for-delete->add_fixed_value( '02' ).

DATA(lo_fixed_value_02) = lo_change_specification->for-insert-
>add_fixed_value( '02' ).
lo_fixed_value_02->set_description( 'February' ).

" Update the description of fixed value 01 (if it exists in the domain).
lo_change_specification->for-update->add_fixed_value( '01'
  )->set_description( 'January' ).

" Delete fixed value 03 (if it exists in the domain).
lo_change_specification->for-delete->add_fixed_value( '03' ).

lo_patch_operation->execute( ).
```

DELETE Operations

Domains can be deleted via DELETE operations. As per the standard semantics of DELETE operations, if the domain for which a DELETE operation is run doesn't exist, no action will be performed (and no error will be raised). If the domain exists, it will be deleted. The code sample below illustrates how to run a DELETE operation for a domain:

Sample Code

```
DATA(lo_delete_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
```

```
  )->for-doma->create_delete_operation( ).

lo_delete_operation->add_object( '$DOMAIN_NAME$' ).

lo_delete_operation->execute( ).
```

Setting and Getting the API State of a Domain

A domain can be released via the API state mechanisms of the API Release Framework (ARS). The XCO library allows you to programmatically set and get the API state of a domain (represented as `IF_XCO_DOMAIN`). The following code sample illustrates how the API state of a given domain can be read:

Sample Code

```
DATA(lo_domain) = xco_cp_abap_dictionary=>domain( '$DOMAIN_NAME$' ).

" Get the (current) API state of the domain for the C1 release contract.
DATA(lo_api_state) = lo_domain->get_api_state( ).

" The returned LO_API_STATE object allows to inspect the individual
" attributes of the API state.
DATA(lo_release_state) = lo_api_state->get_release_state( ).

" Release states supported by the XCO Library can be obtained via
XCO_CP_ARS=>RELEASE_STATE.
" Checking whether the API state of the domain has the release state
" "RELEASED" would look like this:
IF lo_release_state EQ xco_cp_ars=>release_state->released.
  " ...
ENDIF.

DATA(lt_visibilities) = lo_api_state->get_visibilities( ).

" Visibilities supported by the XCO Library can be obtained via
XCO_CP_ARS=>VISIBILITY.
" Note that due to compatibility and historical reasons, the visibility
" CLOUD_DEVELOPMENT is called SAP_CLOUD_PLATFORM within the XCO Library.
Checking
" whether the visibilities of the API state include CLOUD_DEVELOPMENT would
" then look like this:
IF line_exists( lt_visibilities[ table_line = xco_cp_ars=>visibility-
>sap_cloud_platform ] ).
  " ...
ENDIF.
```

Setting the API state for a domain can be done like this:

Sample Code

```
" The API state which should be set for the domain is built via
XCO_CP_ARS=>API_STATE:
DATA(lo_api_state) = xco_cp_ars=>api_state->released( VALUE
#( ( xco_cp_ars=>visibility->sap_cloud_platform ) ) ).

" Setting the API state for a domain implies a change which must be recorded
" on an appropriate transport.
DATA(lo_transport) = xco_cp_cts=>transport->for( '$TRANSPORT$' ).

" The API state is set via the handle for the domain.
```

```
DATA(lo_domain) = xco_cp_abap_dictionary=>domain( '$DOMAIN_NAME$' ).

" We set the previously built API State (Released for Cloud Development)
" using the C1-release contract for the domain (recording the changes on the
" provided transport).
lo_domain->set_api_state(
  io_change_scenario = lo_transport
  io_api_state       = lo_api_state
).
```

4.2.15.18.3.8.4 Event Bindings

Find out how to create event bindings.

Event bindings, also called objects of type `EVTB`, are supported by the following API families within the *XCO Library*:

- Query APIs
- Read APIs
- Generation APIs

Query APIs

Obtaining an object handle for an event binding with a given name can be accomplished via

Sample Code

```
DATA(lo_event_binding) = xco_cp_abap_repository=>object->evtb-
>for( '$EVENT_BINDING_NAME$' ).
```

The obtained object handle can then be used to check the existence of the event binding in the ABAP Repository (via method `IF_XCO_AR_OBJECT~EXISTS`) or read out its content via the Read APIs (see below). Obtaining a list of all visible event bindings in the ABAP Repository can simply be accomplished via

Sample Code

```
DATA(lt_event_bindings) = xco_cp_abap_repository=>objects->evtb->all-
>in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_event_bindings ASSIGNING FIELD-SYMBOL(<fs_event_binding>).
  " Process the found event binding.
ENDLOOP.
```

Instead of searching the entire ABAP Repository for service bindings, it's also possible to only search for data definitions in a dedicated subset of the ABAP Repository, such as a single package:

Sample Code

```
DATA(lo_package) = xco_cp_abap_repository=>package->for( '$PACKAGE_NAME$' ).
```

```
DATA(lt_event_bindings) = xco_cp_abap_repository=>objects->evtb->all-
>in( lo_package )->get( ).

LOOP AT lt_event_bindings ASSIGNING FIELD-SYMBOL(<fs_event_binding>).
  " Process the found event binding.
ENDLOOP.
```

Another way to refine a search for event bindings is to use filters to specify exactly which event bindings shall be matched, for example:

Sample Code

```
DATA(lo_object_name_filter) = xco_cp_abap_repository=>object_name->get_filter(
  xco_cp_abap_sql=>constraint->contains_pattern( '/ABC/%' )
).

" LT_EVENT_BINDINGS will be a list of all visible event bindings in the ABAP
" Repository whose name starts with /ABC/.
DATA(lt_event_bindings) = xco_cp_abap_repository=>objects->evtb->where( VALUE
#(
  ( lo_object_name_filter )
) )->in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_event_bindings ASSIGNING FIELD-SYMBOL(<fs_event_binding>).
  " Process the found event binding.
ENDLOOP.
```

Read APIs

Reading out the content and checking the existence of an event binding is possible via its object handle`IF_XCO_EVENT_BINDING`. The read APIs for event bindings are based on the origin infrastructure with the following origins being currently offered for event bindings:

- *DEFAULT*: Can be obtained via `XCO_CP_EVENT_BINDING=>ORIGIN→DEFAULT` and will read a given event binding exactly once per ABAP session from the local system with all subsequent reads of the event binding being fulfilled from a dedicated buffer. This origin is used by default if no origin is explicitly specified when reading the content or checking the existence of an event binding.
- *LOCAL*: Can be obtained via `XCO_CP_EVENT_BINDING=>ORIGIN→LOCAL` and will read a given event binding from the local system. By default, every read of the content as well as existence check of an event binding will read directly from the database but the caching behavior can be configured via the methods of `IF_XCO_EB_CACHEABLE_ORIGIN`.

Furthermore, when reading an event binding or checking its existence, it can explicitly be specified which version of the event binding shall be considered. The following versions are supported:

- *ACTIVE*: Can be obtained via `XCO_CP_EVENT_BINDING=>VERSION→ACTIVE`. The active version is used by default, if no version is explicitly specified when reading the content of an event binding or checking its existence.
- *INACTIVE*: Can be obtained via `XCO_CP_EVENT_BINDING=>VERSION→INACTIVE`.

Following the general style of the XCO Read APIs, each part of an event binding can be read out separately:

Header

The following code sample illustrates how the header of an event binding can be read out:

Sample Code

```
DATA(lo_event_binding) = xco_cp_abap_repository=>object->evtb-
>for( '$EVENT_BINDING_NAME$' ).

" Read out the complete header of the active version of the event binding
" using the default origin:
DATA(ls_header) = lo_event_binding->content( )->get( ).

" Read out only the short description of the inactive version of the event
" binding using an explicitly obtained local origin (without caching).
DATA(lo_local_origin) = xco_cp_event_binding=>origin->local( ).

DATA(lv_short_description) = lo_event_binding-
>content( xco_cp_event_binding=>version->inactive
  )->get_short_description( lo_local_origin ).
```

Event Versions

Obtaining a list of the event versions of an event binding and reading out an individual event version can be done like this:

```
DATA(lo_event_binding) = xco_cp_abap_repository=>object->evtb-
>for( '$EVENT_BINDING_NAME$' ).

" Get a list of all the event versions of the active version of the event
" binding using the default origin:
DATA(lt_event_versions) = lo_event_binding->event_versions->all->get( ).

LOOP AT lt_event_versions ASSIGNING FIELD-SYMBOL(<fs_event_version>).
  " Process the event version.
ENDLOOP.

" Obtain the handle for an individual event version.
DATA(lo_event_version) = lo_event_binding->event_version( 0001 ).

" Read out the complete content of the active version of the event version
" using the default origin.
DATA(ls_event_version) = lo_event_version->content( )->get( ).

" Read out only the minor version of the inactive version of the event
" version using an explicitly obtained local origin with caching enabled.
DATA(lo_local_origin) = xco_cp_event_binding=>origin->local( ).
lo_local_origin->if_xco_eb_cacheable_origin~enable_caching( ).

DATA(lv_minor_version) = lo_event_version-
>content( xco_cp_event_binding=>version->inactive
  )->get_minor_version( lo_local_origin ).
```

Generation APIs

Event bindings are integrated with XCO Generation APIs. The following operations are currently offered:

- *PUT*: Create or update an event binding based on a complete specification of its content, so that after the successful execution of the operation, the event binding exists and its content is exactly as specified
- *PATCH*; Change an existing event binding by applying the changes which have been provided in a change specification
- *DELETE* Delete an event binding (if it exists)

PUT operations

A form specification is used to specify the complete content of an event binding which is the basis for executing a PUT operation. Upon execution of the PUT operation, depending on whether the event binding already exists or not, it will either be newly created or updated. After the successful completion of the PUT operation the event binding exists with the content resembling exactly what was specified in the provided form specification. Consider the following code sample which illustrates how to execute a PUT operation for an event binding:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-evtb-
>add_object( '$EVENT_BINDING_NAME$'
  )->set_package( '$PACKAGE_NAME$'
  )->create_form_specification( ).

lo_form_specification->set_short_description( 'My short description'
  )->set_type_namespace( '$TYPE_NS$'
  )->set_sap_object_type( '$SAP_OBJECT_TYPE$'
  )->set_operation( '$OPERATION' ).

DATA(lo_event_version_0001) = lo_form_specification-
>add_event_version( 0001 ).
lo_event_version_0001->set_entity_name( '$ENTITY_NAME$'
  )->set_entity_event_name( '$ENTITY_EVENT_NAME$' ).

lo_put_operation->execute( ).
```

PATCH operations

A change specification is used to specify the changes that shall be applied to an existing event binding and is the basis for executing a PATCH operation for an event binding. Upon execution of a PATCH operation, the changes will be applied to the event binding. The following kinds of changes can be specified:

- *DELETE*: If the specified part exists in the event binding it will be deleted. If it doesn't exist no action will be performed.
- *INSERT*: If the specified part doesn't exist in the event binding it will be inserted according to the provided specification. If it exists no action will be performed.
- *UPDATE*: If the specified part exists in the event binding it will be updated according to the provided specification.

Note

Note that the different kinds of changes are always evaluated in the order delete, insert, update with the changes that are applied for one kind being directly visible to the changes that are applied for a subsequent

kind. This orchestration can be exploited to, for example, implement a *MODIFY* by specifying both a *DELETE* and an *INSERT* for a given object part. The following code example illustrates selected changes and how they can be applied via a (single) *PATCH* operation

```
DATA(lo_patch_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_patch_operation( ).

DATA(lo_change_specification) = lo_patch_operation->for-evtb-
>add_object( '$EVENT_BINDING_NAME$'
  )->create_change_specification( ).

" Update the short description.
lo_change_specification->for-update->set_short_description( 'New short
description' ).

" Modify event version 0001 by specifying both a deletion and an insertion
" for it.
lo_change_specification->for-delete->add_event_version( 0001 ).

DATA(lo_event_version_0001) = lo_change_specification->for-insert-
>add_event_version( 0001 ).
lo_event_version_0001->set_minor_version( 0000 ).
lo_event_version_0001->set_patch_version( 0000 ).
lo_event_version_0001->set_entity_name( '$ENTITY_NAME' ).
lo_event_version_0001->set_entity_event_name( '$ENTITY_EVENT_NAME$' ).

" Update the Entity Event Name of event version 0002 (if it exists in the
" event binding).
lo_change_specification->for-update->add_event_version( 0002
  )->set_entity_event_name( 'CREATED2' ).

" Delete event version 0003 (if it exists in the event binding).
lo_change_specification->for-delete->add_event_version( 0003 ).

lo_patch_operation->execute( ).
```

DELETE operations

Event bindings can be deleted via `DELETE` operations. As per the standard semantics of `DELETE` operations, if the data definition for which a `DELETE` operation is run doesn't exist, no action will be performed (and no error will be raised). If the data definition does exist, it will be deleted. The code sample below illustrates how to execute a `DELETE` operation for a data definition:

{·} Sample Code

```
DATA(lo_delete_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->for-evtb->create_delete_operation( ).

lo_delete_operation->add_object( '$EVENT_BINDING_NAME$' ).

lo_delete_operation->execute( ).
```

4.2.15.18.3.8.5 SAP Object Node Types

Find out how to create SAP object node types.

SAP object node types, meaning objects of type `NONT`, are supported by the following API families within the XCO library:

- Query APIs
- Read APIs
- Generation APIs

Query APIs

Obtaining an object handle for an SAP object node type with a given name can be accomplished via

Sample Code

```
DATA(lo_sap_object_node_type) = xco_cp_abap_repository=>object->nont-
>for( '$SAP_OBJECT_NODE_TYPE_NAME$' ).
```

The obtained object handle can then be used to, for example, check the existence of the SAP object node type in the ABAP repository (via method `IF_XCO_AR_OBJECT~EXISTS`) or read out its content via the Read APIs (see below). Obtaining a list of all visible SAP object node types in the ABAP repository can be done via

Sample Code

```
DATA(lt_sap_object_node_types) = xco_cp_abap_repository=>objects->nont->all-
>in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_sap_object_node_types ASSIGNING FIELD-
SYMBOL(<fs_sap_object_node_type>).
  " Process the found SAP object node type.
ENDLOOP.
```

Instead of searching for SAP object node types in the entire ABAP repository, it's also possible to only search for SAP object node types in a dedicated subset of the ABAP repository, such as in a single package:

Sample Code

```
DATA(lo_package) = xco_cp_abap_repository=>package->for( '$PACKAGE_NAME$' ).

DATA(lt_sap_object_node_types) = xco_cp_abap_repository=>objects->nont->all-
>in( lo_package )->get( ).

LOOP AT lt_sap_object_node_types ASSIGNING FIELD-
SYMBOL(<fs_sap_object_node_type>).
  " Process the found SAP object node type.
ENDLOOP.
```

Furthermore, you can refine a search for SAO object node types by using filters to specify exactly which SAP object node types should be matched, for example:

Sample Code

```
DATA(lo_object_name_filter) = xco_cp_abap_repository=>object_name->get_filter(
  xco_cp_abap_sql=>constraint->contains_pattern( '/ABC/%' )
).

" LT_SAP_OBJECT_NODE_TYPES will be a list of all visible SAP object node
" types in the ABAP Repository whose name starts with /ABC/.
DATA(lt_sap_object_node_types) = xco_cp_abap_repository=>objects->nont-
>where( VALUE #(
  ( lo_object_name_filter )
) )->in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_sap_object_node_types ASSIGNING FIELD-
SYMBOL(<fs_sap_object_node_type>).
  " Process the found SAP object node type.
ENDLOOP.
```

Read APIs

Reading out the content and checking the existence of an SAP object node type is possible via its object handle `IF_XCO_SAP_OBJECT_NODE_TYPE`. The Read APIs for SAP object node types are based on the origin infrastructure with the following origins being offered currently for SAP object node types:

- **DEFAULT**: Can be obtained via `XCO_CP_SAP_OBJECT_NODE_TYPE=>ORIGIN→DEFAULT` and will read a given SAP object node type exactly once per ABAP session from the local system, with all subsequent reads of the SAP object node type being fulfilled from a dedicated buffer. This origin is used by default, meaning if no origin is explicitly specified when reading the content or checking the existence of an SAP object node type.
- **LOCAL**: Can be obtained via `XCO_CP_SAP_OBJECT_NODE_TYPE=>ORIGIN→LOCAL` and will read a given SAP object node type from the local system. By default, every read of the content as well as existence checks of an SAP object node type will read directly from the database, but the caching behavior can be configured via the methods of `IF_XCO_NONT_CACHEABLE_ORIGIN`.

Next, when reading an SAP object node type or checking its existence, it can explicitly be specified which version of the SAP object node type should be considered. The following versions are supported:

- **ACTIVE**: Can be obtained via `XCO_CP_SAP_OBJECT_NODE_TYPE=>VERSION→ACTIVE`. The active version is used by default, meaning if no version is explicitly specified when reading the content of an SAP object node type or checking its existence.
- **INACTIVE**: Can be obtained via `XCO_CP_SAP_OBJECT_NODE_TYPE=>VERSION→INACTIVE`.

Following the general style of the XCO Read APIs, each part of an SAP object node type can be read out separately:

Header

Reading out the header of an SAP object node type can be done like this:

Sample Code

```
DATA(lo_sap_object_node_type) = xco_cp_abap_repository=>object->nont-
>for( '$SAP_OBJECT_NODE_TYPE_NAME$' ).
```

```
" Read out the complete header of the active version of the SAP object node
" type using the default origin:
DATA(ls_header) = lo_sap_object_node_type->content( )->get( ).

" Read out only the short description of the inactive version of the SAP
" object node type using an explicitly obtained local origin (without
" caching).
DATA(lo_local_origin) = xco_cp_sap_object_node_type=>origin->local( ).

DATA(lv_short_description) = lo_sap_object_node_type-
>content( xco_cp_sap_object_node_type=>version->inactive
  )->get_short_description( lo_local_origin ).
```

Generation APIs

SAP object node types are integrated with the XCO Generation APIs with the following operations being offered currently for SAP object node types:

- **PUT**: Create or update an SAP object node type based on an entire content specification so that after running the operation successfully, the SAP object node type exists and its content is exactly as specified
- **PATCH**: Change an existing SAP object node type by applying the changes which have been provided in a change specification
- **DELETE**: Delete an SAP object node type (if it exists)

PUT Operations

A form specification is used to specify the entire content of an SAP object node type, which is the basis for running a PUT operation. Upon running the PUT operation, depending on whether the SAP object node type already exists or not, it will either be newly created or updated. This is so that after the successful completion of the PUT operation, the SAP object node type exists with the content resembling exactly what was specified in the provided form specification. Consider the following code sample which illustrates how to run a PUT operation for an SAP object node type:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-nont-
>add_object( '$SAP_OBJECT_NODE_TYPE_NAME$'
  )->set_package( '$PACKAGE_NAME$'
  )->create_form_specification( ).

lo_form_specification->set_short_description( 'My short description'
  )->set_name( '$sapObjectNodeTypeName$'
  )->set_sap_object_type( '$SAP_OBJECT_TYPE_NAME$'
  )->set_root_node( abap_true ).

lo_put_operation->execute( ).
```

PATCH Operations

A change specification is used to specify the changes that should be applied to an existing SAP object node type, and is the basis for running a PATCH operation for an SAP object node type. Upon running a PATCH

operation, the changes will be applied to the SAP object node type. The following code example illustrates selected changes and how they can be applied via a PATCH operation:

Sample Code

```
DATA(lo_patch_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_patch_operation( ).

DATA(lo_change_specification) = lo_patch_operation->for-nont-
>add_object( '$SAP_OBJECT_NODE_TYPE_NAME$'
  )->create_change_specification( ).

" Update the short description.
lo_change_specification->for-update->set_short_description( 'New short
description' ).

lo_patch_operation->execute( ).
```

DELETE Operations

SAP object node types can be deleted via DELETE operations. As per the standard semantics of DELETE operations, if the SAP object node type for which a DELETE operation is being run doesn't exist, no action will be performed (and no error will be raised). If the SAP object node type exists, it will be deleted. The code sample below illustrates how to run a DELETE operation for an SAP object node type:

Sample Code

```
DATA(lo_delete_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->for-nont->create_delete_operation( ).

lo_delete_operation->add_object( '$SAP_OBJECT_NODE_TYPE_NAME$' ).

lo_delete_operation->execute( ).
```

4.2.15.18.3.8.6 SAP Object Types

Find out how to create SAP object types.

SAP object types, meaning objects of type `RONT`, are supported by the following API families within the XCO library:

- Query APIs
- Read APIs
- Generation APIs

Query APIs

Obtaining an object handle for an SAP object type with a given name can be accomplished via

Sample Code

```
DATA(lo_sap_object_type) = xco_cp_abap_repository=>object->ront-
>for( '$SAP_OBJECT_TYPE_NAME$' ).
```

The obtained object handle can then be used to, for example, check the existence of the SAP object type in the ABAP repository (via method `IF_XCO_AR_OBJECT~EXISTS`) or read out its content via the Read APIs (see below). Obtaining a list of all visible SAP object types in the ABAP repository can be done via

Sample Code

```
DATA(lt_sap_object_types) = xco_cp_abap_repository=>objects->ront->all-
>in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_sap_object_types ASSIGNING FIELD-SYMBOL(<fs_sap_object_type>).
  " Process the found SAP object type.
ENDLOOP.
```

Instead of searching for SAP object types in the entire ABAP repository, it's also possible to only search for SAP object types in a dedicated subset of the ABAP repository, such as in a single package:

Sample Code

```
DATA(lo_package) = xco_cp_abap_repository=>package->for( '$PACKAGE_NAME$' ).

DATA(lt_sap_object_types) = xco_cp_abap_repository=>objects->ront->all-
>in( lo_package )->get( ).

LOOP AT lt_sap_object_types ASSIGNING FIELD-SYMBOL(<fs_sap_object_type>).
  " Process the found SAP object type.
ENDLOOP.
```

Furthermore, you can refine a search for SAP object types by using filters to specify exactly which SAP object types should be matched, for example:

Sample Code

```
DATA(lo_object_name_filter) = xco_cp_abap_repository=>object_name->get_filter(
  xco_cp_abap_sql=>constraint->contains_pattern( '/ABC/%' )
).

" LT_SAP_OBJECT_TYPES will be a list of all visible SAP object types in the
" ABAP Repository whose name starts with /ABC/.
DATA(lt_sap_object_types) = xco_cp_abap_repository=>objects->ront-
>where( VALUE #(
  ( lo_object_name_filter )
) )->in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_sap_object_types ASSIGNING FIELD-SYMBOL(<fs_sap_object_type>).
  " Process the found SAP object type.
ENDLOOP.
```

Read APIs

Reading out the content and checking the existence of an SAP object type is possible via its object handle `IF_XCO_SAP_OBJECT_TYPE`. The Read APIs for SAP object types are based on the origin infrastructure with the following origins being offered currently for SAP object types:

- **DEFAULT**: Can be obtained via `XCO_CP_SAP_OBJECT_TYPE=>ORIGIN→DEFAULT` and will read a given SAP object type exactly once per ABAP session from the local system, with all subsequent reads of the SAP object type being fulfilled from a dedicated buffer. This origin is used by default, meaning if no origin is explicitly specified when reading the content or checking the existence of an SAP object type.
- **LOCAL**: Can be obtained via `XCO_CP_SAP_OBJECT_TYPE=>ORIGIN→LOCAL` and will read a given SAP object type from the local system. By default, every read of the content as well as existence checks of an SAP object type will read directly from the database, but the caching behavior can be configured via the methods of `IF_XCO_NONT_CACHEABLE_ORIGIN`.

Next, when reading an SAP object type or checking its existence, it can explicitly be specified which version of the SAP object type should be considered. The following versions are supported:

- **ACTIVE**: Can be obtained via `XCO_CP_SAP_OBJECT_TYPE=>VERSION→ACTIVE`. The active version is used by default, meaning if no version is explicitly specified when reading the content of an SAP object type or checking its existence.
- **INACTIVE**: Can be obtained via `XCO_CP_SAP_OBJECT_TYPE=>VERSION→INACTIVE`.

Following the general style of the XCO Read APIs, each part of an SAP object type can be read out separately:

Header

Reading out the header of an SAP object type can be done like this:

Sample Code

```
DATA(lo_sap_object_type) = xco_cp_abap_repository=>object->ront-
>for( '$SAP_OBJECT_TYPE_NAME$' ).

" Read out the complete header of the active version of the SAP object type
" using the default origin:
DATA(ls_header) = lo_sap_object_type->content( )->get( ).

" Read out only the short description of the inactive version of the SAP
" object type using an explicitly obtained local origin (without caching).
DATA(lo_local_origin) = xco_cp_sap_object_type=>origin->local( ).

DATA(lv_short_description) = lo_sap_object_type-
>content( xco_cp_sap_object_type=>version->inactive
  )->get_short_description( lo_local_origin ).
```

Generation APIs

SAP object types are integrated with the XCO Generation APIs with the following operations being offered currently for SAP object types:

- **PUT**: Create or update an SAP object type based on an entire content specification so that after running the operation successfully, the SAP object type exists and its content is exactly as specified

- **PATCH**: Change an existing SAP object type by applying the changes which have been provided in a change specification
- **DELETE**: Delete an SAP object type (if it exists)

PUT Operations

A form specification is used to specify the entire content of an SAP object type which is the basis for running a PUT operation. Upon running the PUT operation, depending on whether the SAP object type already exists or not, it will either be newly created or updated. This is so that after the successful completion of the PUT operation, the SAP object type exists with the content resembling exactly what was specified in the provided form specification. Consider the following code sample which illustrates how to run a PUT operation for an SAP object type:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-ront-
>add_object( '$SAP_OBJECT_TYPE_NAME$'
  )->set_package( '$PACKAGE_NAME$'
  )->create_form_specification( ).

lo_form_specification->set_short_description( 'My short description'
  )->set_type_category( xco_cp_sap_object_type=>type_category->business_object
  )->set_name( '$sapObjectTypeName$'
  )->set_object_type_code( '00000' ).

lo_put_operation->execute( ).
```

PATCH Operations

A change specification is used to specify the changes that should be applied to an existing SAP object type and is the basis for running a PATCH operation for an SAP object type. Upon running a PATCH operation, the changes will be applied to the SAP object type. The following code example illustrates selected changes and how they can be applied via a PATCH operation:

Sample Code

```
DATA(lo_patch_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_patch_operation( ).

DATA(lo_change_specification) = lo_patch_operation->for-ront-
>add_object( '$SAP_OBJECT_TYPE_NAME$'
  )->create_change_specification( ).

" Update the short description.
lo_change_specification->for-update->set_short_description( 'New short
description' ).

lo_patch_operation->execute( ).
```

DELETE Operations

SAP object types can be deleted via DELETE operations. As per the standard semantics of DELETE operations, if the SAP object type for which a DELETE operation is being run doesn't exist, no action will be performed (and

no error will be raised). If the SAP object type exists, it will be deleted. The code sample below illustrates how to run a DELETE operation for an SAP object type:

Sample Code

```
DATA(lo_delete_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->for-ront->create_delete_operation( ).

lo_delete_operation->add_object( '$SAP_OBJECT_TYPE_NAME$' ).

lo_delete_operation->execute( ).
```

4.2.15.18.3.8.7 Service Bindings

Find out how to create service bindings.

Service bindings, also called objects of type SRVB, are supported by the following API families within the *XCO Library*:

- Query APIs
- Read APIs
- Generation APIs

In addition to the standard API families listed above, dedicated APIs are offered for the following aspects:

- Publishing and unpublishing local service endpoints

Query APIs

Obtaining an object handle for a service binding with a given name can be accomplished via

Sample Code

```
DATA(lo_service_binding) = xco_cp_abap_repository=>object->srvb-
>for( '$SERVICE_BINDING_NAME$' ).
```

The obtained object handle can then be used to, for example, check the existence of the service binding in the ABAP Repository (via method `IF_XCO_AR_OBJECT~EXISTS`) or read out its content via the Read APIs (see below). Obtaining a list of all visible service bindings in the ABAP Repository can be accomplished via

Sample Code

```
DATA(lt_service_bindings) = xco_cp_abap_repository=>objects->srvb->all-
>in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_service_bindings ASSIGNING FIELD-SYMBOL(<fs_service_binding>).
  " Process the found service binding.
ENDLOOP.
```

Instead of searching the entire ABAP Repository for service bindings, it's also possible to only search for service bindings in a dedicated subset of the ABAP Repository, such as in a single package:

Sample Code

```
DATA(lo_package) = xco_cp_abap_repository=>package->for( '$PACKAGE_NAME$' ).

DATA(lt_service_bindings) = xco_cp_abap_repository=>objects->srvb->all-
>in( lo_package )->get( ).

LOOP AT lt_service_bindings ASSIGNING FIELD-SYMBOL(<fs_service_binding>).
  " Process the found service binding.
ENDLOOP.
```

Another way to refine a search for service bindings is to use filters to specify exactly which service bindings shall be matched, for example:

Sample Code

```
DATA(lo_object_name_filter) = xco_cp_abap_repository=>object_name->get_filter(
  xco_cp_abap_sql=>constraint->contains_pattern( '/ABC/%' )
).

" LT_SERVICE_BINDINGS will be a list of all visible service bindings in the
ABAP
" Repository whose name starts with /ABC/.
DATA(lt_service_bindings) = xco_cp_abap_repository=>objects->srvb-
>where( VALUE #(
  ( lo_object_name_filter )
) )->in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_service_bindings ASSIGNING FIELD-SYMBOL(<fs_service_binding>).
  " Process the found service binding.
ENDLOOP.
```

Read APIs

Reading out the content and checking the existence of a service binding is possible via its object handle `IF_XCO_SERVICE_BINDING`. The Read APIs for service bindings are based on the origin infrastructure with the following origins being currently offered for service bindings:

- *DEFAULT*: Can be obtained via `XCO_CP_SERVICE_BINDING=>ORIGIN→DEFAULT` and will read a given service binding exactly once per ABAP session from the local system with all subsequent reads of the service binding being done from a dedicated buffer. As the name suggests, this origin is used by default, so if no origin is explicitly specified when reading the content or checking the existence of a service binding.
- *LOCAL*: Can be obtained via `XCO_CP_SERVICE_BINDING=>ORIGIN→LOCAL` and will read a given service binding from the local system. By default, every read of the content and check of the existence of a service binding will read directly from the database, but the caching behavior can be configured via the methods of `IF_XCO_SRVB_CACHEABLE_ORIGIN`.

Furthermore, when reading a service binding or checking its existence, it can explicitly be specified which version of the service binding to consider. The following versions are supported:

- *ACTIVE*: Can be obtained via `XCO_CP_SERVICE_BINDING=>VERSION→ACTIVE`. The active version is used by default, so if no version is explicitly specified when reading the content or checking the existence of a service binding.
- *INACTIVE* : Can be obtained via `XCO_CP_SERVICE_BINDING=>VERSION→INACTIVE`.

Following the general style of the XCO Read APIs, each part of a service binding can be read out separately:

Header

Reading out the header of a service binding can be done like this:

Sample Code

```
DATA(lo_service_binding) = xco_cp_abap_repository=>object->srvb-
>for( '$SERVICE_BINDING_NAME$' ).

" Read out the complete header of the active version of the service binding
" using the default origin:
DATA(ls_header) = lo_service_binding->content( )->get( ).

" Read out only the short description of the inactive version of the service
" binding using an explicitly obtained local origin (without caching).
DATA(lo_local_origin) = xco_cp_service_binding=>origin->local( ).

DATA(lv_short_description) = lo_service_binding-
>content( xco_cp_service_binding=>version->inactive
  )->get_short_description( lo_local_origin ).
```

Services

Obtaining a list of the services of a service binding can be done like this:

Sample Code

```
DATA(lo_service_binding) = xco_cp_abap_repository=>object->srvb-
>for( '$SERVICE_BINDING_NAME$' ).

" Get a list of all the services of the active version of the service binding
using
" the default origin:
DATA(lt_services) = lo_service_binding->services->all->get( ).

LOOP AT lt_services ASSIGNING FIELD-SYMBOL(<fs_service>).
  " Process the service.
ENDLOOP.
```

Service Versions

Obtaining a list of the service versions of a service and reading out an individual service version can be done like this:

Sample Code

```
" Note that in this example we would be reading the content of a service
" with a binding type for which services can be added freely (e.g. an OData v4
" service binding).
```

```
DATA(lo_service) = xco_cp_abap_repository=>object->srvb-
>for( '$SERVICE_BINDING_NAME$'
  )->service( '$SERVICE_NAME$' ).

" Get a list of all the service versions of the service for the active version
" of the service binding using the default origin.
DATA(lt_service_versions) = lo_service->versions->all->get( ).

LOOP AT lt_service_versions ASSIGNING FIELD-SYMBOL(<fs_service_version>).
  " Process the service version.
ENDLOOP.

" Obtain the handle for an individual service version.
DATA(lo_service_version) = lo_service->version( 0001 ).

" Read out the complete content of the service version for the active version
of the
" service binding using the default origin.
DATA(ls_service_version) = lo_service_version->content( )->get( ).

" Read out only the service definition of the service version for the
inactive version
" of the service binding using an explicitly obtained local origin (with
caching enabled).
DATA(lo_local_origin) = xco_cp_service_binding=>origin->local( ).
lo_local_origin->if_xco_srvb_cacheable_origin~enable_caching( ).

DATA(lo_service_definition) = lo_service_version-
>content( xco_cp_service_binding=>version->inactive
  )->get_service_definition( lo_local_origin ).
```

Generation APIs

Service bindings are integrated with the XCO Generation APIs with the following operations being currently offered for service bindings:

- *PUT*: Create or update a service binding based on a complete specification of its content, so that after successfully running the operation, the service binding exists and its content is exactly as specified.
- *DELETE* : Delete a service binding (if it exists).

PUT Operations

A form specification is used to specify the entire content of a service binding, which is the foundation for running a PUT operation. Upon execution of the PUT operation, depending on whether the service binding already exists or not, it will either be newly created or updated. This is so that after the successful completion of the PUT operation, the service binding exists with the content resembling exactly what was specified in the provided form specification. Consider the following code sample which illustrates how to run a PUT operation for a service binding:

Sample Code

```
DATA(lo_put_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->create_put_operation( ).

DATA(lo_form_specification) = lo_put_operation->for-srvb-
>add_object( '$SERVICE_BINDING_NAME$'
  )->set_package( '$PACKAGE_NAME$'
```

```
  )->create_form_specification( ).

lo_form_specification->set_short_description( 'My short description'
  )->set_binding_type( xco_cp_service_binding=>binding_type->odata_v4_ui ).

DATA(lo_service) = lo_form_specification->add_service( '$SERVICE_NAME$' ).

DATA(lo_service_version_0001) = lo_service->add_version( 0001 ).
lo_service_version_0001-
>set_service_definition( '$SERVICE_DEFINITION_NAME$' ).

lo_put_operation->execute( ).
```

DELETE Operations

Service bindings can be deleted via `DELETE` operations. As per the standard semantics of `DELETE` operations, if the service binding for which a `DELETE` operation is run doesn't exist, no action will be performed (and no error will be raised). If the service binding does exist, it will be deleted. The code sample below illustrates how to run a `DELETE` operation for a service binding:

Sample Code

```
DATA(lo_delete_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST'
  )->for-srvb->create_delete_operation( ).

lo_delete_operation->add_object( '$SERVICE_BINDING_NAME$' ).

lo_delete_operation->execute( ).
```

Publishing and Unpublishing Local Service Endpoints

For service bindings with binding types OData V2 - UI/Web API or OData V4 - UI/Web API, it's possible to publish (or unpublish) a local service endpoint for the service binding. The exact granularity of a local service endpoint depends on whether the binding type belongs to the OData V2 or V4 binding type family:

Local Service Endpoints and OData V2 Service Bindings

For service bindings with binding type OData V2 - UI or OData V2 - Web API, a local service endpoint can be published (or unpublished) for each (service) version defined in the service binding. To programmatically trigger the publish (or unpublish) of a local service endpoint, the *XCO Library* provides designated operations which are available via `XCO_CP_SERVICE_BINDING=>LOCAL_SERVICE_ENDPOINT->ODATA_V2->OPERATION`, namely:

- *PUBLISH* : Triggers a publish of the local service endpoint of the provided service version
- *UNPUBLISH*: Triggers an unpublish of the local service endpoint of the provided service version

Moreover, it's possible to check whether the local service endpoint of a given service version is currently published via `XCO_CP_SERVICE_BINDING=>LOCAL_SERVICE_ENDPOINT→ODATA_V2→IS_PUBLISHED`. The following code illustrates the use of the operations as well as of the `IS_PUBLISHED` method:

Sample Code

```
DATA(lo_service_version) = xco_cp_abap_repository=>object->srvb-
>for( '$SERVICE_BINDING_NAME$'
  )->service(
  )->version( 0001 ).

" First we check whether the local service endpoint for LO_SERVICE_VERSION is
" currently published.
DATA(lv_is_published) = xco_cp_service_binding=>local_service_endpoint-
>odata_v2->is_published( lo_service_version ).

" Depending on whether the local service endpoint is currently published we
either
" trigger an unpublish or a publish of the local service endpoint.
DATA lo_operation TYPE REF TO if_xco_srvb_operation.

IF lv_is_published EQ abap_true.
  lo_operation = xco_cp_service_binding=>local_service_endpoint->odata_v2-
>operation->unpublish( lo_service_version ).
ELSE.
  lo_operation = xco_cp_service_binding=>local_service_endpoint->odata_v2-
>operation->publish( lo_service_version ).
ENDIF.

" Note that both a publish as well as an unpublish operation are concrete
" realizations of an IF_XCO_SRVB_OPERATION and can thus be treated uniformly.
" Regardless of the concrete realization of the operation, its execution can
" always be triggered via the method EXECUTE.
lo_operation->execute( ).
```

Local Service Endpoints for OData V4 Service Bindings

For service bindings with binding type OData V4 - UI or OData V4 - Web API, a local service endpoint can only be published (or unpublished) for the entire service binding. To programmatically trigger the publish (or unpublish) of a local service endpoint, the *XCO Library* provides designated operations which are available via `XCO_CP_SERVICE_BINDING=>LOCAL_SERVICE_ENDPOINT->ODATA_V4->OPERATION`, namely:

- *PUBLISH* : Triggers a publish of the local service endpoint of the provided service binding
- *UNPUBLISH*: Triggers an unpublish of the local service endpoint of the provided service binding

Moreover, it's possible to check whether the local service endpoint of a given service binding is currently published via `XCO_CP_SERVICE_BINDING=>LOCAL_SERVICE_ENDPOINT→ODATA_V4→IS_PUBLISHED`. The following code illustrates the use of the operations as well as of the `IS_PUBLISHED` method:

Sample Code

```
DATA(lo_service_binding) = xco_cp_abap_repository=>object->srvb-
>for( '$SERVICE_BINDING_NAME$' ).

" First we check whether the local service endpoint for LO_SERVICE_BINDING is
" currently published.
DATA(lv_is_published) = xco_cp_service_binding=>local_service_endpoint-
>odata_v4->is_published( lo_service_binding ).

" Depending on whether the local service endpoint is currently published we
either
" trigger an unpublish or a publish of the local service endpoint.
DATA lo_operation TYPE REF TO if_xco_srvb_operation.

IF lv_is_published EQ abap_true.
  lo_operation = xco_cp_service_binding=>local_service_endpoint->odata_v4-
>operation->unpublish( lo_service_binding ).
```

```
ELSE.
  lo_operation = xco_cp_service_binding=>local_service_endpoint->odata_v4-
>operation->publish( lo_service_binding ).
ENDIF.

" Note that both a publish as well as an unpublish operation are concrete
" realizations of an IF_XCO_SRVB_OPERATION and can thus be treated uniformly.
" Regardless of the concrete realization of the operation, its execution can
" always be triggered via the method EXECUTE.
lo_operation->execute( ).
```

4.2.15.18.3.8.8 Service Consumption Models

Find out how to create service consumption models.

Service consumption models, also called objects of type `SRVC`, are supported by the following API families within the *XCO Library*:

- Query APIs
- Read APIs

Query APIs

Obtaining an object handle for a service consumption model with a given name can be accomplished via

Sample Code

```
DATA(lo_service_consumption_model) = xco_cp_abap_repository=>object->srvc-
>for( '$SRVC_NAME$' ).
```

The obtained object handle can then be used to, for example, check the existence of the service consumption model in the ABAP Repository (via method `IF_XCO_AR_OBJECT~EXISTS`) or read out its content via the Read APIs (see below). Obtaining a list of all visible service consumption models in the ABAP Repository can be accomplished via

Sample Code

```
DATA(lt_service_consumption_models) = xco_cp_abap_repository=>objects->srvc-
>all->in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_service_consumption_models ASSIGNING FIELD-
SYMBOL(<fs_service_consumption_model>).
  " Process the found service consumption model.
ENDLOOP.
```

Instead of searching the entire ABAP Repository for service consumption models, it's also possible to only search for service consumption models in a dedicated subset of the ABAP Repository, such as in a single package:

Sample Code

```
DATA(lo_package) = xco_cp_abap_repository=>package->for( '$PACKAGE_NAME$' ).

DATA(lt_service_consumption_models) = xco_cp_abap_repository=>objects->srvc-
>all->in( lo_package )->get( ).

LOOP AT lt_service_consumption_models ASSIGNING FIELD-
SYMBOL(<fs_service_consumption_model>).
  " Process the found service consumption model.
ENDLOOP.
```

Another way to refine a search for service consumption models is to use filters to specify exactly which service consumption models shall be matched, for example:

Sample Code

```
DATA(lo_object_name_filter) = xco_cp_abap_repository=>object_name->get_filter(
  xco_cp_abap_sql=>constraint->contains_pattern( '/ABC/%' )
).

" LT_SERVICE_CONSUMPTION_MODELS will be a list of all visible service
consumption
" models in the ABAP Repository whose name starts with /ABC/.
DATA(lt_service_consumption_models) = xco_cp_abap_repository=>objects->srvc-
>where( VALUE #(
  ( lo_object_name_filter )
) )->in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_service_consumption_models ASSIGNING FIELD-
SYMBOL(<fs_service_consumption_model>).
  " Process the found service consumption model.
ENDLOOP.
```

Read APIs

Reading out the content and checking the existence of a service consumption model is possible via its object handle `IF_XCO_SERVICE_CONSUMPTION_MDL`. The Read APIs for service consumption models are based on the origin infrastructure with the following origins being currently offered for service consumption models:

- *DEFAULT*: Can be obtained via `XCO_CP_SERVICE_CONS_MODEL=>ORIGIN→DEFAULT` and will read a given service consumption model exactly once per ABAP session from the local system, with all subsequent reads of the service consumption model being done from a dedicated buffer. As the name suggests, this origin is used by default, so if no origin is explicitly specified when reading the content or checking the existence of a service consumption model.
- *LOCAL*: Can be obtained via `XCO_CP_SERVICE_CONS_MODEL=>ORIGIN→LOCAL` and will read a given service consumption model from the local system. By default, every read of a service consumption model will read directly from the database, but the caching behavior can be configured via the methods of `IF_XCO_SRVC_CACHEABLE_ORIGIN`.

Furthermore, when reading a service consumption model or checking its existence, it can explicitly be specified which version of the service consumption model shall be considered. The following versions are supported:

- *ACTIVE*: Can be obtained via `XCO_CP_SERVICE_CONS_MODEL=>VERSION→ACTIVE`. The active version is used by default, so if no version is explicitly specified when reading the content of a service consumption model or checking its existence.
- *INACTIVE*: Can be obtained via `XCO_CP_SERVICE_CONS_MODEL=>VERSION→INACTIVE`.

Following the general style of the XCO Read APIs, each part of a service consumption model can be read out separately:

Header

Reading out the header of a service consumption model can be done like this:

Sample Code

```
DATA(lo_service_consumption_model) = xco_cp_abap_repository=>object->srvc-
>for( '$SRVC_NAME$' ).

" Read out the complete header of the active version of the service
consumption model
" using the default origin:
DATA(ls_header) = lo_service_consumption_model->content( )->get( ).

" Read out only the short description of the inactive version of the service
" consumption model using an explicitly obtained local origin (without
caching).
DATA(lo_local_origin) = xco_cp_service_cons_model=>origin->local( ).

DATA(lv_short_description) = lo_service_consumption_model-
>content( xco_cp_service_cons_model=>version->inactive
  )->get_short_description( lo_local_origin ).
```

Service Entity Sets

For service consumption models with consumption type `ODATA` (the corresponding XCO enumeration constant is obtainable via `XCO_CP_SERVICE_CONS_MODEL=>CONSUMPTION_TYPE→ODATA`), it's possible to read the list of service entity sets of the service consumption model as well as the details of a given service entity set:

Sample Code

```
DATA(lo_service_consumption_model) = xco_cp_abap_repository=>object->srvc-
>for( '$SRVC_NAME$' ).

" Get a list of all the service entity sets of the active version of the
service
" consumption model using the default origin:
DATA(lt_service_entity_sets) = lo_service_consumption_model-
>service_entity_sets->all->get( ).

LOOP AT lt_service_entity_sets ASSIGNING FIELD-
SYMBOL(<fs_service_entity_set>).
  " Process the service entity sets.
ENDLOOP.

" Obtain the handle for an individual service entity set.
DATA(lo_service_entity_set) = lo_service_consumption_model-
>service_entity_set( '$SERVICE_ENTITY_SET$' ).

" Read out the complete content of the service entity set for the active
version of the service
```

```
" consumption model using the default origin.
DATA(ls_service_entity_set) = lo_service_entity_set->content( )->get( ).

" Read out only the data definition of the service entity set for the
inactive version of the service
" consumption model using an explicitly obtained local origin (with caching
enabled).
DATA(lo_local_origin) = xco_cp_service_cons_model=>origin->local( ).
lo_local_origin->if_xco_srvc_cacheable_origin~enable_caching( ).

DATA(lo_data_definition) = lo_service_entity_set-
>content( xco_cp_service_cons_model=>version->inactive
  )->get_data_definition( lo_local_origin ).
```

Service Operations

For service consumption models with consumption type `WEB_SERVICE` (the corresponding XCO enumeration constant is obtainable via `XCO_CP_SERVICE_CONS_MODEL=>CONSUMPTION_TYPE→WEB_SERVICE`), it's possible to read the list of service operations of the service consumption model:

Sample Code

```
DATA(lo_service_consumption_model) = xco_cp_abap_repository=>object->srvc-
>for( '$SRVC_NAME$' ).

" Get a list of all the service operations of the active version of the
service
" consumption model using the default origin:
DATA(lt_service_operations) = lo_service_consumption_model-
>service_operations->all->get( ).

LOOP AT lt_service_operations ASSIGNING FIELD-SYMBOL(<fs_service_operation>).
  " Process the service operations.
ENDLOOP.
```

Remote Functions

For service consumption models with consumption type `RFC` (the corresponding XCO enumeration constant is obtainable via `XCO_CP_SERVICE_CONS_MODEL=>CONSUMPTION_TYPE→RFC`), it's possible to read the list of remote functions of the service consumption model:

Sample Code

```
DATA(lo_service_consumption_model) = xco_cp_abap_repository=>object->srvc-
>for( '$SRVC_NAME$' ).

" Get a list of all the remote functions of the active version of the service
" consumption model using the default origin:
DATA(lt_remote_functions) = lo_service_consumption_model->remote_functions-
>all->get( ).

LOOP AT lt_remote_functions ASSIGNING FIELD-SYMBOL(<fs_remote_function>).
  " Process the remote functions.
ENDLOOP.
```

4.2.15.18.3.8.9 Business Configuration Sets

Business configuration sets, also called objects of type `SCP1`, are supported by query APIs, read APIs, and generation APIs.

Query APIs

Obtaining an object handle for a business configuration set with a given name can be accomplished via:

Sample Code

```
DATA(lo_business_configuration_set) = xco_cp_abap_repository=>object->scp1-
>for( '$BUSINESS_CONFIGURATION_SET_NAME$' ).
```

The obtained object handle can then be used to check the existence of the business configuration set in the ABAP repository (via method `IF_XCO_AR_OBJECT~EXISTS`) or read out its short description via the Read APIs (see below). Obtaining a list of all visible business configuration sets in the ABAP repository can be accomplished via:

Sample Code

```
DATA(lt_business_configuration_sets) = xco_cp_abap_repository=>objects->scp1-
>all->in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_business_configuration_sets ASSIGNING FIELD-
SYMBOL(<fs_business_config_set>).
  " Process the found business configuration set.
ENDLOOP.
```

Instead of searching the entire ABAP repository for business configuration sets, it's also possible to only search for business configuration sets in a dedicated subset of the ABAP repository, such as in a single package:

Sample Code

```
DATA(lo_package) = xco_cp_abap_repository=>package->for( '$PACKAGE_NAME$' ).

DATA(lt_business_configuration_sets) = xco_cp_abap_repository=>objects->scp1-
>all->in( lo_package )->get( ).

LOOP AT lt_business_configuration_sets ASSIGNING FIELD-
SYMBOL(<fs_business_config_set>).
  " Process the found business configuration set.
ENDLOOP.
```

Furthermore, you can refine a search for business configuration sets by using filters to specify exactly which business configuration sets should be matched, for example:

Sample Code

```
DATA(lo_object_name_filter) = xco_cp_abap_repository=>object_name->get_filter(
  xco_cp_abap_sql=>constraint->contains_pattern( '/ABC/%' )
).
```

```
" LT_BUSINESS_CONFIGURATION_SETS will be a list of all visible business
configuration sets in the ABAP Repository
" whose name starts with /ABC/.
DATA(lt_business_configuration_sets) = xco_cp_abap_repository=>objects->scp1-
>where( VALUE #(
  ( lo_object_name_filter )
) )->in( xco_cp_abap=>repository )->get( ).

LOOP AT lt_business_configuration_sets ASSIGNING FIELD-
SYMBOL(<fs_business_config_set>).
  " Process the found business configuration set.
ENDLOOP.
```

Read APIs

Reading out the short description and checking the existence of a business configuration set is possible via its object handle `IF_XCO_BUSINESS_CNFGRTN_SET`. The Read APIs for business configuration sets are based on the origin infrastructure with the following origins being currently offered for business configuration sets:

- **DEFAULT**: Can be obtained via `XCO_CP_BUSINESS_CNFGRTN_SET=>ORIGIN→DEFAULT` and will read a given business configuration set exactly once per ABAP session from the local system with all subsequent reads of the business configuration set being fulfilled from a dedicated buffer. As the name suggests, this origin is used by default, meaning if no origin is explicitly specified when reading the short description or checking the existence of a business configuration set.
- **LOCAL**: Can be obtained via `XCO_CP_BUSINESS_CNFGRTN_SET=>ORIGIN→LOCAL` and will read a given business configuration set from the local system. By default, every read of the short description as well as existence checks of a business configuration set will read directly from the database, but the caching behavior can be configured via the methods of `IF_XCO_BCS_CACHEABLE_ORIGIN`.

Business configuration sets do not support active or inactive version handling. The read always returns the information about the active version of the business configuration set.

Reading out the short description of a business configuration set can be done like this:

Sample Code

```
DATA(lo_business_configuration_set) = xco_cp_abap_repository=>object->scp1-
>for( '$BUSINESS_CONFIGURATION_SET_NAME$ ' ).

" Read out the short description of the business configuration set
" using the default origin:
DATA(lv_short_description) = lo_business_configuration_set->content( )-
>get( ).

" Read out the short description of the business configuration set
" using an explicitly obtained local origin (without caching).
DATA(lo_local_origin)       = xco_cp_business_cnfgrtn_set=>origin->local( ).

lv_short_description = lo_business_configuration_set->content( )-
>get_short_description( lo_local_origin ).
```

Generation APIs

Business configuration sets are integrated with the XCO generation APIs. The following operations are currently offered:

- **POST**: Create a new business configuration set based on a complete specification of its content, so that after the successful execution of the operation, the business configuration set exists and its content is exactly as specified.
- **PATCH**: Change an existing business configuration set by applying the changes which have been provided in a change specification.
- **DELETE**: Delete a business configuration set.

POST Operations

A new business configuration is created when running the `POST` operation. An object template can be used to specify the entire content of a business configuration set. Only customizing objects of type `Individual Transaction Object` are currently supported within the business configuration sets generated using the XCO API. An exception is raised if a business configuration set with the specified name already exists.

Consider the following code sample which illustrates how to run a `POST` operation for a business configuration set with a single customizing object:

Sample Code

```
DATA: lt_table1_records  TYPE STANDARD TABLE OF $TABLE1$,
      lt_table2_records  TYPE STANDARD TABLE OF $TABLE2$,
      lt_entity_records  TYPE if_xco_bcs_entity=>tt_entity_records.

" Prepare relevant data records of the individual transaction object in
" lt_table1_records, lt_table2_records
lt_table1_records[] = VALUE #( ( ... ) ).
lt_table2_records[] = VALUE #( ( ... ) ).

lt_entity_records[] = VALUE #( ( entity_name = '$TABLE1$' records = REF
#( lt_table1_records[] ) )
                               ( entity_name = '$TABLE2$' records = REF
#( lt_table2_records[] ) ) ).

DATA(lo_object_template) = xco_cp_generation_scp1=>template-
>individual_transaction_object(
  iv_customizing_object_name = '$INDIVIDUAL_TRANSACTION_OBJECT_NAME$'
  iv_short_description       = '$SHORT_DESCRIPTION$'
  it_entity_records          = lt_entity_records[] ).

DATA(lo_post_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->for-scp1->create_post_operation( ).

lo_post_operation->add_object( xco_cp_name=>choice-
>fixed( '$BUSINESS_CONFIGURATION_SET_NAME$' )
  )->set_package( '$PACKAGE_NAME$'
  )->set_template( lo_object_template ).

TRY.
    DATA(lo_result) = lo_post_operation->execute( ).

    " Process messages from a successful POST operation
    DATA(lt_findings) = lo_result->findings->get( ).

CATCH cx_xco_runtime_exception INTO DATA(lo_xco_runtime_exception).
```

```
  IF lo_xco_runtime_exception IS INSTANCE OF cx_xco_gen_post_exception.

    " Process exceptions from POST operation
    DATA(lo_xco_gen_post_exception) = CAST
cx_xco_gen_post_exception( lo_xco_runtime_exception ).
    lt_findings = lo_xco_gen_post_exception->findings->get( ).

  ELSE.

    " Process runtime exceptions such as 'Business Configuration Set already
exists'
    DATA(lv_runtime_exception) = lo_xco_runtime_exception->get_text( ).

  ENDIF.

ENDTRY.
```

Consider the following code sample which illustrates how to run a POST operation for a business configuration set with multiple customizing objects:

Sample Code

```
DATA: lt_table1_records              TYPE STANDARD TABLE OF $TABLE1$,
      lt_table2_records              TYPE STANDARD TABLE OF $TABLE2$,
      lt_individual_transaction_objs TYPE
if_xco_bcs_customizing_object=>tt_ind_trans_objects.

" Prepare relevant data records of each individual transaction object in
" lt_table1_records, lt_table2_records..
lt_table1_records[] = VALUE #( ( ... ) ).
lt_table2_records[] = VALUE #( ( ... ) ).

lt_individual_transaction_objs[] = VALUE #( ( name           =
'$INDIVIDUAL_TRANSACTION_OBJECT_1$'
                                              entity_records = REF
#( lt_table1_records[] ) )
                                            ( name           =
'$INDIVIDUAL_TRANSACTION_OBJECT_2$'
                                              entity_records = REF
#( lt_table2_records[] ) ) ).

DATA(lo_object_template) = xco_cp_generation_scp1=>template-
>individual_transaction_objects(
  it_ind_trans_objects  = lt_individual_transaction_objs
  iv_short_description  = '$SHORT_DESCRIPTION$'
 ).

DATA(lo_post_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->for-scp1->create_post_operation( ).

lo_post_operation->add_object( xco_cp_name=>choice-
>fixed( '$BUSINESS_CONFIGURATION_SET_NAME$' )
  )->set_package( '$PACKAGE_NAME$'
  )->set_template( lo_object_template ).

TRY.
    DATA(lo_result) = lo_post_operation->execute( ).

    " Process messages from a successful POST operation
    DATA(lt_findings) = lo_result->findings->get( ).

CATCH cx_xco_runtime_exception INTO DATA(lo_xco_runtime_exception).

  IF lo_xco_runtime_exception IS INSTANCE OF cx_xco_gen_post_exception.
```

```
    " Process exceptions from POST operation
    DATA(lo_xco_gen_post_exception) = CAST
cx_xco_gen_post_exception( lo_xco_runtime_exception ).
    lt_findings = lo_xco_gen_post_exception->findings->get( ).

  ELSE.

    " Process runtime exceptions such as 'Business Configuration Set already
exists'
    DATA(lv_runtime_exception) = lo_xco_runtime_exception->get_text( ).

  ENDIF.

ENDTRY.
```

PATCH Operations

A change specification is used to specify the changes that must be applied to an existing business configuration set and is the basis for running a `PATCH` operation for a business configuration set. When running a `PATCH` operation, the changes will be applied to the business configuration set. The following kinds of changes can be specified:

- **DELETE**: If the specified part exists in the business configuration set, it will be deleted. If it doesn't exist, no action will be performed.
- **INSERT**: If the specified part doesn't exist in the business configuration set, it will be inserted according to the provided specification. If it exists, no action will be performed.
- **UPDATE**: If the specified part exists in the business configuration set, it will be updated according to the provided specification.

The `PATCH` operation can be used to specify changes to the records or entities (tables) or customizing objects of a business configuration set.

> ⓘ Note
>
> Note that the different kinds of changes are always evaluated in the order Delete, Insert, Update with the changes that are applied for one kind being directly visible to the changes that are applied for a subsequent kind. You can use this by implementing a `MODIFY` by specifying both a `DELETE` and an `INSERT` for a given object part.

The following code example illustrates selected changes and how they can be applied via a (single) `PATCH` operation:

> {·} Sample Code

```
DATA: lt_remove_table1_records   TYPE STANDARD TABLE OF $TABLE1$,
      lt_remove_table2_records   TYPE STANDARD TABLE OF $TABLE2$,
      lt_insert_table3_records   TYPE STANDARD TABLE OF $TABLE3$,
      lt_insert_table4_deletions TYPE STANDARD TABLE OF $TABLE4$,
      lt_insert_table6_records   TYPE STANDARD TABLE OF $TABLE6$,
      lt_insert_table7_records   TYPE STANDARD TABLE OF $TABLE7$,
      lt_insert_table8_records   TYPE STANDARD TABLE OF $TABLE8$.

DATA(lo_patch_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->for-scp1->create_patch_operation( ).

DATA(lo_change_specification) = lo_patch_operation-
>add_object( '$BUSINESS_CONFIGURATION_SET_NAME$'
  )->create_change_specification( ).
```

```
" Remove Records from Entity => Remove specific records corresponding to
Table 1 and Table 2 of Customizing Object 1 in the business configuration set
" Consider that the entity (table) is already present in the business
configuration set and hence needs to be updated

lt_remove_table1_records[]   = VALUE #( ( ... ) ).
lt_remove_table2_records[]   = VALUE #( ( ... ) ).

DATA(lo_individual_transaction_obj1) = lo_change_specification->for-update-
>add_individual_transaction_obj(
  '$CUSTOMIZING_OBJECT_1$' ).

lo_individual_transaction_obj1->for-update->add_entity( '$TABLE1$'
  )->for-delete->add_records( REF #( lt_remove_table1_records[] ) ).

lo_individual_transaction_obj1->for-update->add_entity( '$TABLE2$'
  )->for-delete->add_records( REF #( lt_remove_table2_records[] ) ).

" Add Records to Entity => Add specific records corresponding to Table 3 of
Customizing Object 1 in the business configuration set
" These records will be upserted to the table upon activation of the business
configuration set
" Consider that the entity (table) is already present in the business
configuration set and hence needs to be updated

lt_insert_table3_records[]   = VALUE #( ( ... ) ).

lo_individual_transaction_obj1->for-update->add_entity( '$TABLE3$'
  )->for-insert->add_records( REF #( lt_insert_table3_records[] ) ).

" Add Deletion records to Entity => Add specific records (with operation at
activation as 'Delete') corresponding to Table 4 of Customizing Object 1 in
the business configuration set
" These records will be deleted from the table upon activation of the
business configuration set
" Consider that the entity (table) is already present in the business
configuration set and hence needs to be updated by adding these deletion
records

lt_insert_table4_deletions[] = VALUE #( ( ... ) ).

lo_individual_transaction_obj1->for-update->add_entity( '$TABLE4$'
  )->for-insert->add_records( iv_to_be_deleted = abap_true
                              io_records       = REF
#( lt_insert_table4_deletions[] ) ).

" Remove Entity from Customizing Object => Remove Table 5 (all
its corresponding records) of Customizing Object 2 from the business
configuration set
" The corresponding customizing object is already present in the business
configuration set and hence needs to be updated
" The entity is already present as part of the customizing object in the
business configuration set and hence needs to be deleted

DATA(lo_individual_transaction_obj2) = lo_change_specification->for-update-
>add_individual_transaction_obj(
 '$CUSTOMIZING_OBJECT_2$' ).

lo_individual_transaction_obj2->for-delete->add_entity( '$TABLE5$' ).

" Add Entity to Customizing Object => Add specific records corresponding to
Table 6 of Customizing Object 2 in the business configuration set
“ Consider that the entity (table) is not already present in the business
configuration set and hence needs to be inserted
" Consider that the corresponding customizing object is already present in
the business configuration set and hence needs to be updated
```

```
lt_insert_table6_records[] = VALUE #( ( ... ) ).

lo_individual_transaction_obj2->for-insert->add_entity( '$TABLE6$'
  )->for-insert->add_records( REF #( lt_insert_table6_records[] ) ).

" Remove Customizing Object 3 (all its Entities and corresponding records)
from the business configuration set
" The customizing object is already present in the business configuration set
and hence needs to be deleted

lo_change_specification->for-delete->add_individual_transaction_obj(
  '$CUSTOMIZING_OBJECT_3$' ).

" Add Customizing Object and its associated records of its Entities i.e.Table
7 and Table 8 to the business configuration set
" The customizing object and the associated entities are not already present
in the business configuration set and hence need to be inserted

lt_insert_table7_records[]   = VALUE #( ( ... ) ).
lt_insert_table8_records[]   = VALUE #( ( ... ) ).

DATA(lo_individual_transaction_obj4) = lo_change_specification->for-insert-
>add_individual_transaction_obj(
  '$CUSTOMIZING_OBJECT_4$' ).

lo_individual_transaction_obj4->for-insert->add_entity( '$TABLE7$'
  )->for-insert->add_records( REF #( lt_insert_table7_records[] ) ).

lo_individual_transaction_obj4->for-insert->add_entity( '$TABLE8$'
  )->for-insert->add_records( REF #( lt_insert_table8_records[] ) ).

" After preparing the change specification, execute the PATCH operation
TRY.

  DATA(lo_result) = lo_patch_operation->execute( ).

CATCH cx_xco_gen_patch_exception INTO DATA(lo_xco_gen_patch_exception).

    LOOP AT lo_xco_gen_patch_exception->findings->get( ) ASSIGNING FIELD-
SYMBOL(<fs_finding>).
      " Process errors/exceptions raised from the PATCH operation
    ENDLOOP.

ENDTRY.
```

DELETE Operations

Business configuration sets can be deleted via a `DELETE` operation. Following the standard semantics of a `DELETE` operation, if the business configuration set for which a `DELETE` operation is run doesn't exist, no action will be performed (and no error will be raised). If the business configuration set exists, it will be deleted. The code sample below illustrates how to run a `DELETE` operation for a business configuration set:

Sample Code

```
DATA(lo_delete_operation) = xco_cp_generation=>environment-
>dev_system( '$TRANSPORT_REQUEST$'
  )->for-scp1->create_delete_operation( ).

lo_delete_operation->add_object( '$BUSINESS_CONFIGURATION_SET_NAME$' ).

lo_delete_operation->execute( ).
```

4.2.15.18.3.9 Miscellaneous Code Samples

4.2.15.18.3.9.1 Getting the Text Table of a Database Table

The XCO standard abstraction for database tables (`IF_XCO_DATABASE_TABLE`) provides a convenient way to determine the text table of a given database table (if it has one):

Sample Code

```
DATA(lo_database_table) = xco_cp_abap_dictionary=>database_table( 'ZMY_DBT' ).

" From a database table handle (in this case LO_DATABASE_TABLE) we can obtain
" the handle for its text table.
DATA(lo_text_table) = lo_database_table->text_table( ).

" Using this text table handle, we can first check if a text table exists at
all
" (for the previously fixed database table) and if so, obtain information
about
" it.
DATA(lv_text_table_exists) = lo_text_table->exists( ).

IF lv_text_table_exists EQ abap_true.
  " The name of the text table that belongs to database table ZMY_DBT.
  DATA(lv_text_table_name) = lo_text_table->get( )->name.

  " We can also use the text table handle to get the name of the check field
of the
  " database table/text table relationship.
  DATA(lv_check_field_name) = lo_text_table->get_check_field( )->name.
ENDIF.
```

4.2.15.18.3.9.2 Retrieving a List of All Classes Implementing an Interface

The standard abstraction provided by the XCO library for ABAP interfaces (`IF_XCO_AO_INTERFACE`) can be used to retrieve a list of all implementations of a given interface:

Sample Code

```
DATA(lo_interface) = xco_cp_abap=>interface( 'ZMY_INTERFACE' ).

" From an interface handle (in this case LO_INTERFACE) we can easily obtain
" access to its implementations via the IMPLEMENTATIONS data member. Obtaining
" a list of all implementations for an interface can be accomplished like
this:
DATA(lt_implementations) = lo_interface->implementations->all->get( ).

" Once the previous statement has been executed, LT_IMPLEMENTATIONS will be a
" table of IF_XCO_AO_CLASS objects which can then subsequently be used to e.g.
" read the content of each class. Note that only those implementations will be
" considered which are either C1-released or part of a customer software
component.
" Additionally, if only the names of the classes implementing the interface
```

```
" shall be obtained the following statement can be used:
DATA(lt_implementation_names) = lo_interface->implementations->all-
>get_names( ).
```

4.2.15.18.3.9.3 Retrieving a List of all Subclasses of a Given Class

Find out how to retrieve a list of all subclasses of a given class.

The standard abstraction provided by the XCO library for ABAP classes (`IF_XCO_AO_CLASS`) can be used to retrieve a list of all subclasses (including both direct and indirect subclasses) of a given class:

Sample Code

```
DATA(lo_class) = xco_cp_abap=>class( 'ZMY_CLASS' ).

" From a class handle (in this case LO_CLASS) we can easily obtain access to
" its subclasses via the SUBCLASSES data member. Accessing all subclasses
" via SUBCLASSES->ALL will resolve the subclasses of the class recursively
" along the inheritance tree, i.e. all subclasses including subclasses of
" subclasses and so forth will be included.
DATA(lt_subclasses) = lo_class->subclasses->all->get( ).

" Once the previous statement has been executed, LT_SUBCLASSES will be a
" table of IF_XCO_AO_CLASS objects which can then subsequently be used to
" e.g. read the content of each class. Furthermore, note that only those
" subclasses will be considered which are either C1-released or part of a
" customer software component. If only the names of the subclasses shall be
" obtained, the following variation can be used:
DATA(lt_subclass_names) = lo_class->subclasses->all->get_names( ).
```

4.2.15.18.4 Standard Library

The XCO standard library provides lightweight abstractions to simplify everyday programming tasks and defines central abstractions, most notably IF_XCO_NEWS and IF_XCO_TEXT, that provide a common language to foster synergies between different XCO modules.

Building blocks

There are several basic building blocks defined within the XCO standard library that are used throughout the modules of the whole XCO library and may even be integrated into custom coding.

Exception philosophy

Unless a caller can be reasonably expected to recover from an error situation is always handled through NO_CHECK exceptions that are derived from CX_XCO_RUNTIME_EXCEPTION.

The class CX_XCO_RUNTIME_EXCEPTION is an abstract class that defines aspects that are common to all kinds of unexpected error situations. Most importantly, it provides access to the messages of the exceptional situation via IF_XCO_NEWS.

On top of that, every time a CX_XCO_RUNTIME_EXCEPTION is raised the active call stack is captured when the exception object is constructed. By having exception classes inherit from CX_XCO_RUNTIME_EXCEPTION stack traces can also be easily enabled for self-defined exceptions.

When such an exception is logged via the XCO BAL module the stack trace will be included along with the message of the exception (assuming the XCO_CP_BAL=>EXCEPTION_ADDITION->STACK_TRACE exception addition has been enabled).

News

The interface IF_XCO_NEWS provides a central abstraction that is applicable to any object that can naturally act as a source of messages. A message is defined as an object of type IF_XCO_MESSAGE which in turn is characterized by a SYMSG value.

As the interface IF_XCO_NEWS can be flexibly mixed in with existing class hierarchies it provides a common denominator that enables communication between independent modules both within the XCO library as well as between the XCO library and custom coding.

A good example for the synergies created by the IF_XCO_NEWS abstraction is the interplay of the XCO BAL module with the XCO ADT standard library functionality which is described below. Even though the XCO ADT functionality has no knowledge and is completely independent of the XCO BAL module writing all the messages contained in a log to the console is as simple as

Sample Code

```
out->write( lo_log->messages->all )
```

Notable implementations of IF_XCO_NEWS are:

- IF_XCO_RAP_BEHAVIOR_MESSAGE. Obtainable via XCO_CP_RAP=>BEHAVIOR MESSAGE(...), an IF_XCO_RAP_BEHAVIOR_MESSAGE encapsulates an IF_ABAP_BEHV_MESSAGE and allows it to be used anywhere where an IF_XCO_NEWS object can be used.

Text

Closely related to the IF_XCO_NEWS abstraction is the IF_XCO_TEXT interface. The main difference is that it represents an object than can naturally act as a source of strings (as compared to an object that can naturally act as a source of messages as is the case with IF_XCO_NEWS).

Like IF_XCO_NEWS it provides a common ground for many different XCO modules. It is furthermore the basis of the IF_XCO_PRINTABLE interface which allows to define a print version for any object that is used when the object is written to the console via the XCO ADT functionality (see ADT [page 1798]).

Related Information

ADT [page 1798]
JSON [page 1806]
Regular Expression [page 1808]

String [page 1809]
UUID [page 1815]

4.2.15.18.4.1 ADT

By having a class implement the interface IF_OO_ADT_CLASSRUN it is possible to easily write programs that can be run directly within ADT using the console as output.

The XCO library provides an abstract implementation of this interface via CL_XCO_CP_ADT_SIMPLE_CLASSRUN that exposes all the original functionality of IF_OO_ADT_CLASSRUN but adds several additional functionalities that enable a natural integration of XCO library abstractions with ADT classruns.

The following sample class illustrates how the OUT object provided by CL_XCO_CP_ADT_SIMPL_CLASSRUN can be used to flexibly write complex data structures or even object references to the console. Using the interface IF_XCO_PRINTABLE any object can define a text that is used when the object is written to the console.

Sample Code

```
CLASS zcl_xco_doc_cp_std_adt DEFINITION PUBLIC FINAL
    INHERITING FROM cl_xco_cp_adt_simple_classrun CREATE PUBLIC.
  PROTECTED SECTION.
    METHODS:
      main REDEFINITION.
ENDCLASS.
CLASS zcl_xco_doc_cp_std_adt IMPLEMENTATION.
  METHOD main.
    TYPES:
      BEGIN OF ts_address,
        street       TYPE string,
        house_number TYPE n LENGTH 2,
        postal_code  TYPE n LENGTH 5,
        city         TYPE string,
      END OF ts_address,
      BEGIN OF ts_member,
        first_name TYPE string,
        last_name  TYPE string,
        address    TYPE ts_address,
      END OF ts_member,
      tt_member TYPE STANDARD TABLE OF ts_member WITH EMPTY KEY.
    " Print a JSON-style serialization (works for arbitrarily deep data
structures)
    DATA(lt_members) = VALUE tt_member(
      ( first_name = 'John' last_name = 'Doe' address = VALUE #( street =
'Main street' house_number = 12 postal_code = 12345 city = 'City' ) )
    ).
    " Will print: [{FIRST_NAME: John, LAST_NAME: Doe, ADDRESS: {STREET: Main
street, HOUSE_NUMBER: 12, POSTAL_CODE: 12345, CITY: City}}]
    out->write( lt_members ).
    " IF_XCO_PRINTABLE example: Will print "WARNING"
    out->write( xco_cp_message=>type->warning ).
    " IF_XCO_PRINTABLE example: Will print "Field MY_FIELD of View entity
MY_VIEW_ENTITY"
    DATA(lo_field) = xco_cp_cds=>view_entity( 'MY_VIEW_ENTITY' )-
>field( 'MY_FIELD' ).
    out->write( lo_field ).
  ENDMETHOD.
```

```
ENDCLASS.
```

On top of the rich console writing functionalities, any NO_CHECK exception that is not caught by the MAIN method of a class implementing CL_XCO_CP_ADT_SIMPLE_CLASSRUN will be caught and written to the console with maximum information. This includes the raise position, messages and even the stack trace if the exception is carrying one (cf. explanation of CX_XCO_RUNTIME_EXCEPTION above).

4.2.15.18.4.2 Business Application Log

The XCO BAL module provides APIs and abstractions that allow a smooth integration of logging into application logic based on the standard Business Application Log (BAL). The module works seamlessly with other XCO library features.

Terminology

Within the BAL module of the XCO Library, the following terminology is used:

- **Log object**: The design time ABAP Repository object (type APLO) defining an application log object together with its subobjects.
- **Log**: A single log instance for a given log object and subobject with an optional external ID into which messages can be written and read from. Represented by an object of type `IF_XCO_CP_BAL_LOG` and uniquely identified by a log handle.
- **Log handle**: A unique identifier for a log (UUID in CHAR22 format) associated with a log (available through attribute `LOG_HANDLE` of interface `IF_XCO_CP_BAL_LOG`.

Persistence

The first and one of the central abstractions within the XCO BAL module is that of the persistence given by the interface `IF_XCO_CP_BAL_PERSISTENCE`. A persistence decides where logs are created (or loaded from) and where the messages and exceptions are written to (or read from).

The following two flavors are offered:

- Memory: Logs are created only in memory and messages as well as exceptions are not written to the database
- Database: Logs are created and loaded from the database and messages and exceptions are always saved to the database. Whenever a message or exception is added to the log a `COMMIT WORK` is performed on a secondary database connection

Searching logs

In a style like that of the XCO ABAP Repository Query APIs, it's possible to easily locate existing logs based on filters for standard log attributes, like the object, subobject or external ID of a log:

Sample Code

```
DATA(lo_external_id_filter) = xco_cp_bal=>log_filter->external_id(
  xco_cp_abap_sql=>constraint->contains_pattern( '*INVOICE*' )
).
DATA(lt_logs) = xco_cp_bal=>for->database( )->logs->where( VALUE #(
  ( lo_external_id_filter )
) )->get( ).
```

In this example, all logs in the database whose external ID contains the fragment `INVOICE` will be located.

Creating and loading logs

Once the desired persistence has been selected via `XCO_CP_BAL=>FOR`, a log can be created like

Sample Code

```
DATA(lo_log) = xco_cp_bal=>for->database( )->log->create(
  iv_object      = 'MY_LOG_OBJECT'
  iv_subobject   = 'MY_LOG_SUBOBJECT'
  iv_external_id = 'MY_EXTERNAL_ID'
).
```

If a log already exists in the database, it can be loaded like

Sample Code

```
DATA(lo_log) = xco_cp_bal=>for->database( )->log->load( 'MY_LOG_HANDLE' ).
```

for a given log handle.

Profiles

Associated with each `IF_XCO_CP_BAL_LOG` object is a profile (`IF_XCO_CP_BAL_PROFILE`) that influences how messages and exceptions are added to the log. For messages, a profile provides a default value for the level of detail that is used when messages are added to the log and no explicit level of detail is supplied.

For exceptions, the XCO BAL module offers a rich set of extra information that can be added along with the exception message to achieve maximum information that can be used for detailed error analysis.

The standard profile (`XCO_CP_BAL=>PROFILE->STANDARD`) defines a default level of detail of 4 and includes all possible exception additions as well as an automatic recursive descent for exceptions, so all the previous exceptions of an exception are added to the log as well.

Adding and reading messages and exceptions

Messages can be added to a log in number of ways. They can be added directly via an explicit `SYMSG` value like

Sample Code

```
" Logging a plain SYMSG using the level of detail given by the underlying
" profile.
DATA(ls_symsg) = VALUE symsg(
  msgty = 'W'
  msgid = 'MSG_CLASS'
  msgno = '001'
  msgv1 = 'Item 1'
).
lo_log->add_message( ls_symsg ).
" Logging the same message but this time with an explicitly specified
" level of detail.
lo_log->add_message(
  is_symsg           = ls_symsg
  io_level_of_detail = xco_cp_bal=>level_of_detail->nine
).
```

or by resorting to the standard abstractions `IF_XCO_NEWS` and `IF_XCO_TEXT` from the XCO standard library:

Sample Code

```
" Support for IF_XCO_NEWS and IF_XCO_TEXT provides a high degree of
integration with other
" parts of the XCO library.
MESSAGE w001(msg_class) WITH |Item 1| INTO DATA(lv_message).
lo_log->add_news( xco_cp=>sy ).
DATA(lv_string) = |MyStringValue|.
lo_log->add_text( xco_cp=>string( lv_string ) ).
```

Following the pattern which is also used when reading the content of an object of the ABAP repository, it's easy to access all the messages contained in a log:

Sample Code

```
DATA(lt_messages) = lo_log->messages->all->get( ).
```

Note

Note that in order to be able to read from a log, an authorization is required for authorization object `S_APPL_LOG` for the following authorization field values:

- `ALG_OBJECT`: The object of the application log that is being read
- `ALG_SUBOBJ`: The subobject of the application log that is being read
- `ACTVT`: 03

Adding and getting exceptions is identical to how messages are added to and read from the log: Exceptions are added via the `ADD_EXCEPTION` method and read via the `EXCEPTIONS` attribute of an `IF_XCO_CP_BAL_LOG` object.

Note that when an exception is added to a log, it will automatically be transformed into a message by the underlying business application log if it provides a T100 message. Only when an exception doesn't provide a T100 message will it be available via the `EXCEPTIONS` read attribute of `IF_XCO_CP_BAL_LOG`.

4.2.15.18.4.3 Call Stack

The XCO call stack module allows to programmatically capture and process ABAP call stacks during execution.

The following example call stack, captured by running a locally created ADT classrun (`ZCL_XCO_CALL_STACK`) will be used to illustrate the different functionalities.

Line	Positive position	Negative position
`ZCL_XCO_CALL_STACK` main [method]	1	-17
`CL_XCO_ADT_CR_SIMPLE_ACTION` execute [method]	2	-16
`CL_XCO_DP_ACTION_ABSTRACT` if_xco_dp_action~execute [method]	3	-15
`CL_XCO_ADT_CALL_CLASSRUN` if_xco_rt_adt_call_classrun~dispatch [method]	4	-14
`CL_XCO_RT_ADT_CLASSRUN_ABSTR` if_oo_adt_classrun~main [method]	5	-13
`CL_OO_ADT_RES_CLASSRUN` execute_clas [method]	6	-12
`CL_OO_ADT_RES_CLASSRUN` post [method]	7	-11
`CL_ADT_REST_RESOURCE` if_rest_handler~handle [method]	8	-10
`CL_REST_ROUTER` if_rest_handler~handle [method]	9	-9
`CL_REST_ROUTER` if_rest_handler~handle [method]	10	-8
`CL_REST_HTTP_HANDLER` if_http_extension~handle_request [method]	11	-7
`CL_ADT_WB_RES_APP` lif_request_handler~handle_request [method]	12	-6
`CL_ADT_WB_RES_APP` handle_request [method]	13	-5
`CL_ADT_WB_RES_APP` if_http_extension~handle_request	14	-4
`CL_HTTP_SERVER` [system] execute_request [method]	15	-3

Line	Positive position	Negative position
SAPLHTTP_RUNTIME [system] http_dispatch_request [function]	16	-2
SAPMHTTP [system] %_http_start [module (pbo)]	17	-1

In a manner analogous to how characters of a string are indexed when substrings are extracted via IF_XCO_STRING, the XCO call stack module associates both a positive and a negative index/position with each call stack entry as illustrated above.

Capturing the current call stack

The current call stack can be captured by using the API XCO_CP=>CURRENT->CALL_STACK. The following two methods are available:

- FULL: Captures the full current call stack, i.e. all available entries up the lowest entry point will be included
- UP_TO: Captures the current call stack up to the given depth, i.e. only the topmost IV_DEPTH call stack entries will be captured

Once captured, a call stack is represented as an immutable object of type IF_XCO_CP_CALL_STACK.

Extracting parts of a call stack

Given a call stack (represented as an object of type IF_XCO_CP_CALL_STACK) various ways are provided to extract specific parts of the call stack. As the result of any extraction is again an object of type IF_XCO_CP_CALL_STACK it's easy to cascade multiple extractions to form complex and highly customized extractions.

The following kinds of extractions are provided:

- POSITION: Uses a position (either positive or negative) to fix a reference call stack entry
- FIRST_OCCURRENCE_OF: Uses the first occurrence of a given call stack line pattern to fix a reference call stack entry
- LAST_OCCURRENCE_OF: Uses the last occurrence of a given call stack line pattern to fix a reference call stack entry

Each extraction is applied in either a TO or FROM manner:

- TO: Extracts that part of the call stack that starts from the top of the call stack to the determined reference call stack entry
- FROM: Extracts that part of the call stack that starts from the determined reference call stack entry to the bottom of the call stack

To illustrate these concepts, consider the following extractions where LO_CALL_STACK represents the call stack from the above table:

Sample Code

```
DATA(lo_line_pattern) = xco_cp_call_stack=>line_pattern->method(
  )->where_class_name_starts_with( 'CL_XCO_' ).

DATA(lo_extracted_call_stack) = lo_call_stack->from->position( 2
  )->to->last_occurence_of( lo_line_pattern
  )->to->position( -2 ).
```

In total three extractions are applied. The first extraction, `->FROM→POSITION( 2 )`, will produce the following call stack:

Line	Positive position	Negative position
`CL_XCO_ADT_CR_SIMPLE_ACTION` execute [method]	1	-16
`CL_XCO_DP_ACTION_ABSTRACT` if_xco_dp_action~execute [method]	2	-15
`CL_XCO_ADT_CALL_CLASSRUN` if_xco_rt_adt_call_classrun~dispatch [method]	3	-14
`CL_XCO_RT_ADT_CLASSRUN_ABSTR` if_oo_adt_classrun~main [method]	4	-13
`CL_OO_ADT_RES_CLASSRUN` execute_clas [method]	5	-12
`CL_OO_ADT_RES_CLASSRUN` post [method]	6	-11
`CL_ADT_REST_RESOURCE` if_rest_handler~handle [method]	7	-10
`CL_REST_ROUTER` if_rest_handler~handle [method]	8	-9
`CL_REST_ROUTER` if_rest_handler~handle [method]	9	-8
`CL_REST_HTTP_HANDLER` if_http_extension~handle_request [method]	10	-7
`CL_ADT_WB_RES_APP` lif_request_handler~handle_request [method]	11	-6
`CL_ADT_WB_RES_APP` handle_request [method]	12	-5
`CL_ADT_WB_RES_APP` if_http_extension~handle_request	13	-4
`CL_HTTP_SERVER` [system] execute_request [method]	14	-3
`SAPLHTTP_RUNTIME` [system] http_dispatch_request [function]	15	-2
`SAPMHTTP` [system] %_http_start [module (pbo)]	16	-1

The second extraction, `->to->last_occurence_of( lo_line_pattern )`, applied to the result of the first extraction, will then produce the following call:

Line	Positive position	Negative position
`CL_XCO_ADT_CR_SIMPLE_ACTION` execute [method]	1	-4
`CL_XCO_DP_ACTION_ABSTRACT` if_xco_dp_action~execute [method]	2	-3
`CL_XCO_ADT_CALL_CLASSRUN` if_xco_rt_adt_call_classrun~dispatch [method]	3	-2
`CL_XCO_RT_ADT_CLASSRUN_ABSTR` if_oo_adt_classrun~main [method]	4	-1

Finally, the application of the third extraction, `->to->position( -2 )`, to the result of the second extraction yields the following call stack:

Line	Positive position	Negative position
`CL_XCO_ADT_CR_SIMPLE_ACTION` execute [method]	1	-3
`CL_XCO_DP_ACTION_ABSTRACT` if_xco_dp_action~execute [method]	2	-2
`CL_XCO_ADT_CALL_CLASSRUN` if_xco_rt_adt_call_classrun~dispatch [method]	3	-1

Note that the reference call stack entries are always treated inclusively, i.e. they will be included in the extracted call stack.

Formatting a call stack

To be written to the console or into an application log (using `IF_XCO_CP_BAL_LOG`), a given call stack can be formatted which will produce an `IF_XCO_TEXT` for the call stack. As of now, the following formats are supported:

- ADT: The ADT format will render a call stack in the same style as the ADT Debugger. Optionally, the captured line numbers (if present) can be added to each resulting call stack entry line whereby source or include based line number flavors are available for selection

Using the call stack from the beginning, formatting it with source-based line numbers and writing the resulting `IF_XCO_TEXT` to the console of an ADT classrun (obtained by inheriting from `CL_XCO_CP_ADT_SIMPLE_CLASSRUN`) can be accomplished as follows:

Sample Code

```
DATA(lo_format) = xco_cp_call_stack=>format->adt(
  )->with_line_number_flavor( xco_cp_call_stack=>line_number_flavor->source ).
```

```
DATA(lo_call_stack_text) = lo_call_stack->as_text( lo_format ).
out->write( lo_call_stack_text ).
```

This will produce the following output on the console (line numbers may vary on local execution):

```
ZCL_XCO_CALL_STACK main [method] Source line: 10

CL_XCO_ADT_CR_SIMPLE_ACTION execute [method] Source line: 39

CL_XCO_DP_ACTION_ABSTRACT if_xco_dp_action~execute [method] Source line: 59

CL_XCO_ADT_CALL_CLASSRUN if_xco_rt_adt_call_classrun~dispatch [method] Source line:
30

CL_XCO_RT_ADT_CLASSRUN_ABSTR if_oo_adt_classrun~main [method] Source line: 36

CL_OO_ADT_RES_CLASSRUN execute_clas [method] Source line: 206

CL_OO_ADT_RES_CLASSRUN post [method] Source line: 395

CL_ADT_REST_RESOURCE if_rest_handler~handle [method] Source line: 150

CL_REST_ROUTER if_rest_handler~handle [method] Source line: 325

CL_REST_HTTP_HANDLER if_http_extension~handle_request [method] Source line: 190

CL_ADT_WB_RES_APP lif_request_handler~handle_request [method] Source line: 19

CL_ADT_WB_RES_APP handle_request [method] Source line: 317

CL_ADT_WB_RES_APP if_http_extension~handle_request [method] Source line: 435

CL_HTTP_SERVER [system] execute_request [method] Source line: 2840

SAPLHTTP_RUNTIME [system] http_dispatch_request [function] Source line: 1626

SAPMHTTP [system] %_http_start [module (pbo)] Source line: 12
```

4.2.15.18.4.4 JSON

The goal of the XCO JSON module is to make working with JSON data as simple as possible. This includes the creation of JSON strings (both from ABAP data structures and from scratch) as well as the translation of JSON strings to ABAP.

Reading JSON strings

The following code shows an example of how a JSON string (e.g. the response of an outbound service call) can be translated into a corresponding ABAP structure:

Sample Code

```
DATA(lv_json_string) = '{'
  && |"SessionId":"7cd44fff-036a-4155-b0d2-f5a4dfbcee92",|
  && |"UserName":"John Doe",|
  && |"IsPremiumUser":true|
  && '}'.
TYPES:
  BEGIN OF ts_input,
```

```
    session_id      TYPE sysuuid_c36,
    user_name       TYPE string,
    is_premium_user TYPE abap_bool,
  END OF ts_input.
DATA ls_input TYPE ts_input.
" After execution, the structure components will have the following values
"  session_id = 7cd44fff-036a-4155-b0d2-f5a4dfbcee92
"  user_name = John Doe
"  is_premium_user = X
xco_cp_json=>data->from_string( lv_json_string )->apply( VALUE #(
  ( xco_cp_json=>transformation->pascal_case_to_underscore )
  ( xco_cp_json=>transformation->boolean_to_abap_bool )
) )->write_to( REF #( ls_input ) ).
```

To make it easy to adjust JSON data to ABAP requirements (e.g. to switch between camel case and underscore notation) the XCO library provides built-in transformations which are accessible via the XCO_CP_JSON API. The current version of the XCO library comes with the following three built-in transformations:

- Camel case/Pascal case to underscore: Transforms the name of each object member in the JSON data from Camel case/Pascal case to underscore notation
- Underscore to Camel case/Pascal case: Transforms the name of each object member in the JSON data from underscore to Camel case/Pascal case notation
- Boolean to abap_bool: Transforms each JSON boolean value to its abap_bool equivalent, i.e. true becomes 'X' and false becomes ''

Note that the XCO library uses the following terminology:

• Camel case: The first character of each word is capitalized except for the first word.

• Pascal case: The first character of each word is capitalized including the first word.

Creating JSON strings

JSON strings can be created in two ways, either from scratch or from a corresponding ABAP data structure. Using the XCO JSON data builder, JSON data can be built like:

Sample Code

```
DATA(lo_json_builder) = xco_cp_json=>data->builder( ).
lo_json_builder->begin_object(
  )->add_member( 'SessionId' )->add_string( '7cd44fff-036a-4155-b0d2-
f5a4dfbcee92'
  )->add_member( 'UserName' )->add_string( 'John Doe'
  )->add_member( 'IsPremiumUser' )->add_boolean( abap_true
  )->end_object( ).
" After execution LV_JSON_STRING will have the value
" {"SessionId":"7cd44fff-036a-4155-b0d2-f5a4dfbcee92","UserName":"John
Doe","IsPremiumUser":true}
DATA(lv_json_string) = lo_json_builder->get_data( )->to_string( ).
```

In the same way that a JSON string can be translated to an appropriate ABAP data structure an ABAP data structure can be translated to a JSON string. It is again possible to apply transformations to e.g. transform underscore to camel case notation.

Sample Code

```
TYPES:
  BEGIN OF ts_output,
    session_id      TYPE sysuuid_c36,
```

```
    user_name       TYPE string,
    is_premium_user TYPE xsdboolean,
  END OF ts_output.
DATA(ls_output) = VALUE ts_output(
  session_id      = '7cd44fff-036a-4155-b0d2-f5a4dfbcee92'
  user_name       = |John Doe|
  is_premium_user = abap_true
).
" After execution LV_JSON_STRING will have the value
" {"SessionId":"7cd44fff-036a-4155-b0d2-f5a4dfbcee92","UserName":"John
Doe","IsPremiumUser":true}
DATA(lv_json_string) = xco_cp_json=>data->from_abap( ls_output )-
>apply( VALUE #(
  ( xco_cp_json=>transformation->underscore_to_pascal_case )
) )->to_string( ).
```

4.2.15.18.4.5 Regular Expression

Regular expressions can be used to match strings against patterns or extract substrings that match certain criteria.

The regular expression engines offered by XCO are based on the standards offered by CL_ABAP_REGEX:

- POSIX
- PCRE
- XPATH2
- XSD

In XCO, these regular expression standards are encapsulated in the IF_XCO_REGEX_ENGINE abstraction which is accessible via the XCO_CP_REGULAR_EXPRESSION API:

Sample Code

```
" Example for a POSIX regular expression engine (More configuration options
are available
" as optional parameters of the method POSIX).
DATA(lo_posix_engine) = xco_cp_regular_expression=>engine->posix(
  iv_ignore_case = abap_true
).
" Example for a PCRE regular expression engine (More configuration options
are available
" as optional parameters of the method PCRE).
DATA(lo_pcre_engine) = xco_cp_regular_expression=>engine->pcre(
  iv_ignore_case      = abap_false
  iv_enable_multiline = abap_false
).
" Example for an XPATH 2.0 regular expression engine (More configuration
options are available
" as optional parameters of the method XPATH2).
DATA(lo_xpath2_engine) = xco_cp_regular_expression=>engine->xpath2(
  iv_ignore_case      = abap_true
  iv_enable_multiline = abap_false
).
" Example for an XSD regular expression engine (More configuration options
are available
" as optional parameters of the method XSD).
DATA(lo_xsd_engine) = xco_cp_regular_expression=>engine->xsd(
  iv_ignore_case      = abap_true
  iv_enable_multiline = abap_false
```

```
).
```

The parameters of the regular expression engine factory methods can be used to configure the behavior of the engine when it is used against a string. The default engine (which is used when no explicit engine is supplied) is POSIX in the default configuration as provided by CL_ABAP_REGEX.

A regular expression can be used to easily check if a given string matches a pattern

Sample Code

```
" LV_CASE_INSENSITIVE_MATCHES will have the value abap_true as
" LO_POSIX_ENGINE is case insensitive (see above).
DATA(lv_case_insensitive_matches) = xco_cp=>string( |abc| )->matches(
  iv_regular_expression = '^a.*C$'
  io_engine             = lo_posix_engine
).
" LV_CASE_SENSITIVE_MATCHES will have the value abap_false as the
" default POSIX engine is case sensitive.
DATA(lv_case_sensitive_matches) = xco_cp=>string( |abc| )-
>matches( '^a.*C$' ).
```

or to retrieve certain parts from it

Sample Code

```
DATA(lo_name_parts) = xco_cp=>string( |John Doe| )->grep( '^(\S+)\s+(\S+)$' ).
" LV_FIRST_NAME = John
DATA(lv_first_name) = lo_name_parts->value[ 1 ].
" LV_LAST_NAME = Doe
DATA(lv_last_name) = lo_name_parts->value[ 2 ].
```

4.2.15.18.4.6 String

Besides providing a straightforward integration with regular expressions (see Regular Expression [page 1808]), the XCO string abstraction offers further simplifications when working with strings.

The methods TO and FROM

The methods TO and FROM provide flexible ways to extract a substring from a string. Their functionality is based on the following character indexing scheme (The table is based on the example string ABCDEF):

A	B	C	D	E	F
1	2	3	4	5	6
-6	-5	-4	-3	-2	-1

Both TO and FROM have an inclusive behavior, i.e. the provided index is always included in the retrieved substring. Furthermore, if the provided index is out of bounds the behavior is as if the index had pointed to the first (resp. last) character of the string.

The following sample illustrates several usages:

Sample Code

```
DATA(lo_string) = xco_cp=>string( |ABCDEF| ).
" LV_SUBSTRING_1 = DEF
DATA(lv_substring_1) = lo_string->from( 4 )->value.
" LV_SUBSTRING_2 = DEF
DATA(lv_substring_2) = lo_string->from( -3 )->value.
" LV_SUBSTRING_3 = ABC
DATA(lv_substring_3) = lo_string->to( 3 )->value.
" LV_SUBSTRING_4 = ABC
DATA(lv_substring_4) = lo_string->to( -4 )->value.
" LV_SUBSTRING_5 = CD
DATA(lv_substring_5) = lo_string->from( 3 )->to( 2 )->value.
" LV_SUBSTRING_6 = BC
DATA(lv_substring_6) = lo_string->to( -4 )->from( 2 )->value.
```

Splitting and joining

The XCO string library also provides a way to split strings and join a list of strings:

Sample Code

```
DATA(lo_name_parts) = xco_cp=>string( |John Doe| )->split( | | ).
" LV_REVERSED_NAME = Doe, John
DATA(lv_reversed_name) = lo_name_parts->reverse( )->join( |, | )->value.
```

Composing and decomposing

A generalization of split and join operations on strings are string compositions and decompositions.

String compositions and decompositions encapsulate algorithms which can transform a string to a list of strings (decomposition) and vice versa (composition) according to a certain formula. Currently, both Camel Case and Pascal Case are supported in both directions.

The following example illustrates how the camel case composition and decomposition can be used to easily translate between underscore and camel case notation.

Sample Code

```
DATA(lv_string_with_underscores) = |FIRST_NAME|.
" LV_STRING_IN_CAMEL_CASE = firstName
DATA(lv_string_in_camel_case) = xco_cp=>string( lv_string_with_underscores
  )->split( |_|
  )->compose( xco_cp_string=>composition->camel_case
  )->value.
" LV_RESTORED_STRING = FIRST_NAME
DATA(lv_restored_string) = xco_cp=>string( lv_string_in_camel_case
  )->decompose( xco_cp_string=>decomposition->camel_case
  )->join( |_|
  )->to_upper_case(
  )->value.
```

Converting strings to xstrings

Converting a given string to its xstring representation for a given code page can be easily accomplished by making use of `IF_XCO_CHAR_CODE_PAGE`s which are available via `XCO_CP_CHARACTER`:

Sample Code

```
DATA(lv_string) = |ABC|.
" The representation of LV_STRING as an XSTRING where the given character
" code page is used for the conversion.
DATA(lv_xstring) = xco_cp=>string( lv_string
  )->as_xstring( xco_cp_character=>code_page->utf_8 ).
```

Calculating Hash Codes

Calculating the hash value of a given string for a given hash algorithm can be accomplished as follows:

Sample Code

```
" Upon execution, LV_HASH_VALUE will have the value
C79C561BB2CC3D0430A54B3D014C89C3089FE089.
DATA(lv_hash_value) = xco_cp=>string( 'MY_STRING'
  )->as_xstring( xco_cp_hash=>algorithm->sha_1
  )->value.
```

Note the following details:

- Supported hash algorithms (represented as objects of type IF_XCO_HASH_ALGORITHM) can be obtained via XCO_CP_HASH=>ALGORITHM. The SHA-1 hash algorithm is statically available via the SHA_1 property while the FOR method can be used to obtain hash algorithms based on their string identifiers, e.g. XCO_CP_HASH=>ALGORITHM→FOR('MD5') for the MD5 hash algorithm
- When calculating the hash value of a string, an IF_XCO_HASH_ALGORITHM object acts as an IF_XCO_STRING_XSTRING_CNVRSN, where the XSTRING that is derived from the string corresponds to the hash value

4.2.15.18.4.7 System

With the XCO system module, you can easily request information about the current system.

Namespaces

You can get a handle for a given namespace via the API `XCO_CP_SYSTEM=>NAMESPACE`. Once obtained, you can use a namespace handle to find out whether the given namespace exists in the system and, if so, whether it's changeable:

Sample Code

```
" LO_NAMESPACE is a handle that can be used to request information
" about namespace /ABC/.
DATA(lo_namespace) = xco_cp_system=>namespace->for( '/ABC/' ).
```

```
" LV_NAMESPACE is an ABAP_BOOL value indicating whether the namespace
" represented by LO_NAMESPACE exists or not.
DATA(lv_namespace_exists) = lo_namespace->exists( ).

" LV_NAMESPACE_IS_CHANGEABLE is an ABAP_BOOL value indicating whether the
" namespace represented by LO_NAMESPACE is changeable or not. Note that,
" before invoking this method, it should be ensured that the namespace exists
" as otherwise a runtime exception will be raised.
DATA(lv_namespace_is_changeable) = lo_namespace->is_changeable( ).
```

4.2.15.18.4.8 Tenant

The XCO tenant module provides standard abstractions for querying information about tenants, in particular the tenant associated with the current ABAP session.

The central abstraction `IF_XCO_CP_TENANT` represents a tenant and is used to retrieve its public information. As of now, the following information can be retrieved:

- ID: The ID for the tenant
- GUID: The GUID for the tenant (represented as an IF_XCO_UUID)
- URL: The URL of the tenant for the given URL type (URL types are available via XCO_CP_TENANT=>URL_TYPE)
- Global Account ID: The ID of the global account associated with the tenant
- Subaccount ID: The ID of the subaccount associated with the tenant

The currently active tenant can be obtained via:

```
DATA(lo_current_tenant) = xco_cp=>current->tenant( ).
```

Note that depending on the overall system infrastructure and setup a current tenant doesn't necessarily exist, which would be indicated by an initial reference returned by the statement above. Once the current tenant has been obtained by the URL (in the example below for the type UI) along with its different components, the currently active tenant can be obtained as follows:

Sample Code

```
" UI URL of the current tenant.
DATA(lo_ui_url) = lo_current_tenant->get_url( xco_cp_tenant=>url_type->ui ).

" The protocol of the UI URL (Type string).
DATA(lv_ui_url_protocol) = lo_ui_url->get_protocol( ).

" The host (including the domain) of the UI URL (Type string).
DATA(lv_ui_url_host) = lo_ui_url->get_host( ).

" The port of the UI URL (Type i).
DATA(lv_ui_url_port) = lo_ui_url->get_port( ).
```

In a similar way it's also possible to access the information about the associated global account and subaccount:

Sample Code

```
" The string value of the global account ID of the current tenant.
DATA(lv_global_account_id) = lo_current_tenant->get_global_account_id(
  )->as_string( ).
" The string value of the subaccount ID of the current tenant.
DATA(lv_subaccount_id) = lo_current_tenant->get_subaccount_id(
  )->as_string( ).
```

4.2.15.18.4.9 Time Library

The XCO time library provides standard abstractions for working with temporal values.

- A date (IF_XCO_CP_TM_DATE) consists of values for year, month and day
- A time (IF_XCO_CP_TM_TIME) consists of values for hour, minute and second
- A moment (IF_XCO_CP_TM_MOMENT) consists of a fully specified date and time
- A UNIX timestamp (IF_XCO_CP_TM_UNIX_TIMESTAMP) consists of an INT8 value specifying the number of seconds that have elapsed since 00:00:00 UTC on Jan 1st, 1970.

Dates, times and moments

The abstractions date, time and moment correspond to the ABAP temporal types DATS, TIMS and TZNTSTMPS. Given a date, time or moment object, a new object can be derived from each of them by adding or subtracting values from their components or directly overriding specific values. Note that date, time and moment objects are always immutable, i.e. once constructed their values will never change.

Time zone information is explicitly not part of the state of a date, time or moment object.

Time formats

A time format is a unique representation of a date, time or moment. Currently, the following time formats are supported:

- ABAP: Will provide the DATS, TIMS and TZNTSTMPS value, respectively
- ISO 8601 Basic
- ISO 8601 Extended

Time formats are accessible via XCO_CP_TIME=>FORMAT and can be used either directly or passed to the AS method of a date, time or moment object to obtain the string representation of the object according to the provided time format.

The following code sample illustrates the usage of the various time formats for a moment object:

Sample Code

```
DATA(lo_moment) = xco_cp_time=>moment(
  iv_year   = '2020'
  iv_month  = '09'
  iv_day    = '16'
  iv_hour   = '22'
  iv_minute = '33'
  iv_second = '49'
).
" LV_ABAP_FORMAT will be of type STRING and have the value 20200916223349.
```

```
DATA(lv_abap_format) = lo_moment->as( xco_cp_time=>format->abap )->value.
" LV_ISO_8601_BASIC will be of type STRING and have the value 20200916T223349.
DATA(lv_iso_6801_basic) = lo_moment->as( xco_cp_time=>format-
>iso_8601_basic )->value.
" LV_ISO_8601_EXTENDED will be of type STRING and have the value
2020-09-16T22:33:49.
DATA(lv_iso_6801_extended) = lo_moment->as( xco_cp_time=>format-
>iso_8601_extended )->value.
" Restore the moment from LV_ISO_8601_EXTENDED. LO_RESTORED_MOMENT will be an
object of type
" IF_XCO_CP_TM_MOMENT with values equal to LO_MOMENT.
DATA(lo_restored_moment) = xco_cp_time=>format->iso_8601_extended-
>to_moment( lv_iso_6801_extended ).
```

Integration with XCO_CP=>SY

The XCO time library is tightly integrated with the XCO abstraction for the ABAP SY structure to provide an easy way to access the current time, either as a date, time or moment object or as a UNIX timestamp.

When a date, time or moment object is obtained, the time zone according to which the values should be calculated can be specified in addition. Note that a UNIX timestamp will always be in UTC.

Sample Code

```
" The current moment in the time zone of the active user.
DATA(lo_moment_user) = xco_cp=>sy->moment( xco_cp_time=>time_zone->user ).
" The current moment in UTC.
DATA(lo_moment_utc) = xco_cp=>sy->moment( xco_cp_time=>time_zone->utc ).
" The current UNIX timestamp.
DATA(lo_unix_timestamp) = xco_cp=>sy->unix_timestamp( ).
```

Note that if a time zone is not explicitly specified when a date, time or moment object is retrieved from XCO_CP=>SY, the time zone of the active user will be taken.

UNIX timestamps

UNIX timestamps are a common way for describing a given point in time by denoting the number of elapsed seconds since 00:00:00 UTC on Jan 1st, 1970. The XCO time library makes it easy to convert from a given UNIX timestamp to a moment and vice versa:

Sample Code

```
" The current UNIX timestamp.
DATA(lo_unix_timestamp) = xco_cp=>sy->unix_timestamp( ).
" Get the moment for the UNIX timestamp (in the time zone of the active
" user).
DATA(lo_moment) = lo_unix_timestamp->get_moment( xco_cp_time=>time_zone-
>user ).
" Restore the original UNIX timestamp. The value of
LO_RESTORED_UNIX_TIMESTAMP is
" equal to that of LO_UNIX_TIMESTAMP.
DATA(lo_restored_unix_timestamp) = lo_moment-
>get_unix_timestamp( xco_cp_time=>time_zone->user ).
```

4.2.15.18.4.10 UUID

To effectively work with UUIDs the XCO standard library provides a simple way to translate between different UUID formats:

Sample Code

```
DATA(lo_uuid) = xco_cp_uuid=>format->c36->to_uuid( '7cd44fff-036a-4155-b0d2-
f5a4dfbcee92' ).
" LV_UUID_C32 will hold the value 7CD44FFF036A4155B0D2F5A4DFBCEE92
DATA(lv_uuid_c32) =  CONV sysuuid_c32( xco_cp_uuid=>format->c32-
>from_uuid( lo_uuid ) ).
```

4.2.15.18.4.11 XLSX

The XCO XLSX module offers a set of standard abstractions and APIs to programmatically work with XLSX workbooks and their worksheets (e.g. coming form an uploaded Microsoft Excel .XLSX file).

The XCO XLSX module exposes three external APIs:

- XCO_CP_XLSX: Access to general abstractions used throughout the XCO XLSX module
- XCO_CP_XLSX_SELECTION: Access to selection pattern builders used to build selection patterns
- XCO_CP_XLSX_READ_ACCESS: Access to abstractions used when reading the content of an XLSX worksheet

Document handles

The starting point for any programmatic interaction with an XLSX workbook and its worksheets is obtaining a handle for the containing document via XCO_CP_XLSX=>DOCUMENT. As of now, document handles can be obtained for the complete file content of an .XLSX file, which must be provided as an XSTRING:

Sample Code

```
DATA lv_file_content TYPE xstring.

" LV_FILE_CONTENT must be populated with the complete file content of
the .XLSX file
" whose content shall be processed programmatically.

DATA(lo_document) = xco_cp_xlsx=>document-
>for_file_content( lv_file_content ).
```

The obtained document handle then allows to obtain a read access, which is the starting point for accessing the cell contents of the worksheets contained in the document XLSX Read Access [page 1818].

Coordinates

Within the XCO XLSX module, a worksheet is regarded as a space with two dimensions, with the first dimension being the column and the second dimension being the row. Consequently, a cell in a worksheet is uniquely determined by providing values for its column and row. A coordinate (an object of type CL_XCO_XLSX_COORDINATE) uniquely fixes a value for a given dimension, for example, for a value for a column or row. In office suites, e.g. Microsoft Excel, alphabetic values are used to identify columns and numeric values are used to identify rows. The XCO XLSX module provides support for both variants so that alphabetic as well as numeric values can be used freely when values are provided for columns and rows.

- Alphabetic coordinates: A, B, C, ... Z, AA, AB, ...ZZ, AAA, AAB, ...
- Numeric coordinates: 1, 2, 3, ...

Coordinates can be obtained via XCO_CP_XLSX=>COORDINATE for both alphabetic and numeric values and once obtained, both the alphabetic as well as the numeric value can be retrieved for a coordinate. This also provides an easy way to translate between alphabetic and numeric coordinate values, for example:

Sample Code

```
DATA(lo_coordinate_1) = xco_cp_xlsx=>coordinate->for_alphabetic_value( 'AA' ).

" LV_COORD_1_NUMERIC_VALUE will be an integer with value 27.
DATA(lv_coord_1_numeric_value) = lo_coordinate_1->get_numeric_value( ).

" LV_COORD_1_ALPHABETIC_VALUE will be a string with value AA.
DATA(lv_coord_1_alphabetic_value) = lo_coordinate_1->get_alphabetic_value( ).

DATA(lo_coordinate_2) = xco_cp_xlsx=>coordinate->for_numeric_value( 27 ).

" LV_COORD_2_NUMERIC_VALUE will be an integer with value 27.
DATA(lv_coord_2_numeric_value) = lo_coordinate_2->get_numeric_value( ).

" LV_COORD_2_ALPHABETIC_VALUE will be a string with value AA.
DATA(lv_coord_2_alphabetic_value) = lo_coordinate_2->get_alphabetic_value( ).
```

It's possible to derive new coordinates from existing coordinates such as objects of type CL_XCO_XLSX_COORDINATE, by using the SHIFT method to either increment or decrement the value of the coordinate (depending on the sign of the provided offset).

Selection patterns

Selections are a central concept within the XCO XLSX module. A selection is based on a selection pattern which encapsulates a prescription according to which individual cells are identified in a given worksheet which then form the selection. To illustrate the concepts and offered selection patterns consider the following exemplary portion of an XLSX worksheet:

	A	B	C	D	E	F	...
1							
2		1	2	3	4		

	A	B	C	D	E	F	...
3		5	6	7	8		
4		9	10	11	12		
5		13	14	15	16		
6							
...							

For the purposes of the following code samples, it's assumed that except for the cell values shown above, all other cells of the worksheet are initial. Supported selection patterns can be built via selection pattern builders which are obtainable via XCO_CP_XLSX_SELECTION=>PATTERN_BUILDER.

Simple from-to selection patterns

A simple from-to selection pattern allows to define static boundary values for columns and rows. It's not required to specify a boundary value for each dimension, hence both bounded and unbounded selections are possible. Consider the following characteristic examples:

Sample Code

```
" The selection pattern LO_PATTERN_1 will be a bounded rectangle, matching
values
" 6, 7, 10 and 11 of the above worksheet.
DATA(lo_pattern_1) = xco_cp_xlsx_selection=>pattern_builder->simple_from_to(
  )->from_column( xco_cp_xlsx=>coordinate->for_alphabetic_value( 'C' )
  )->to_column( xco_cp_xlsx=>coordinate->for_alphabetic_value( 'D' )
  )->from_row( xco_cp_xlsx=>coordinate->for_numeric_value( 3 )
  )->to_row( xco_cp_xlsx=>coordinate->for_numeric_value( 4 )
  )->get_pattern( ).

" The selection pattern LO_PATTERN_2 will be unbounded at the bottom, matching
" values 6, 7, 10, 11, 14 and 15 of the above worksheet.
DATA(lo_pattern_2) = xco_cp_xlsx_selection=>pattern_builder->simple_from_to(
  )->from_column( xco_cp_xlsx=>coordinate->for_alphabetic_value( 'C' )
  )->to_column( xco_cp_xlsx=>coordinate->for_alphabetic_value( 'D' )
  )->from_row( xco_cp_xlsx=>coordinate->for_numeric_value( 3 )
  )->get_pattern( ).

" The selection pattern LO_PATTERN_3 will be unbounded at the bottom and at
the
" right, matching values 6, 7, 8, 10, 11, 12, 14, 15 and 16 of the above
worksheet.
DATA(lo_pattern_3) = xco_cp_xlsx_selection=>pattern_builder->simple_from_to(
  )->from_column( xco_cp_xlsx=>coordinate->for_alphabetic_value( 'C' )
  )->from_row( xco_cp_xlsx=>coordinate->for_numeric_value( 3 )
  )->get_pattern( ).

" The selection pattern LO_PATTERN_4 is completely unbounded, matching all
cells of the
" worksheet, hence also all values of the above worksheet.
DATA(lo_pattern_4) = xco_cp_xlsx_selection=>pattern_builder->simple_from_to(
  )->get_pattern( ).
```

4.2.15.18.4.11.1 XLSX Read Access

The starting point for programmatically reading the content of an XLSX document is to get a read access for the document. Find out how this is done.

> ⚠ Caution
>
> Note that when data is read from a given XLSX document via the XCO XLSX module, the data is provided in exactly the way it's stored within the XLSX document. This implies that no security or any other kind of additional validations are performed against the data contained in the worksheets of the XLSX workbook when it's accessed by the means described below. Please ensure that when the read data is further processed by application logic (for example, stored in a database table or shown in a Fiori app), dedicated checks are in place to guard your application against potentially malicious content (such as string values in an XLSX worksheet containing JavaScript code).

Consider the following:

> {·} Sample Code

```
DATA lv_file_content TYPE xstring.

" LV_FILE_CONTENT must be populated with the complete file content of
the .XLSX file
" whose content will be processed programmatically.

DATA(lo_read_access) = xco_cp_xlsx=>document-
>for_file_content( lv_file_content
  )->read_access( ).
```

Once obtained, the next step is to get the read access for the worksheet which contains the data that will be read. It's possible to identify a worksheet based on its name or position:

> {·} Sample Code

```
" Read access for the worksheet at position 1, such as the first worksheet in
the workbook.
DATA(lo_first_worksheet) = lo_read_access->get_workbook(
  )->worksheet->at_position( 1 ).

" Read access for the worksheet with name INVOICES.
DATA(lo_invoices_worksheet) = lo_read_access->get_workbook(
  )->worksheet->for_name( 'INVOICES' ).
```

Accessing Data via a Stream

The first way to gain access to the data stored in a worksheet is by selecting a collection of cells based on a selection pattern (see XLSX [page 1815]=> *Selection patterns*. You can do this using the method `SELECT` on `IF_XCO_XLSX_RA_WORKSHEET`. You can access the cells contained in the selection following a stream-based approach, meaning that you'll get a dedicated stream for the selection which will provide sequential access to the individual blocks of the selection. Two kinds of streams are offered:

- Cell stream: A cell stream provides access to the selection one cell at a time, traversing the selection from left to right and top to bottom
- Row stream: A row stream provides access to the selection one row at a time, traversing the selection from top to bottom

Cell streams

Cell streams are intended for dynamic reading scenarios where it's required to process each cell individually. The following operations are offered for cell streams:

- Visit: The visit operation takes a visitor implementation as the input (an object of type `IF_XCO_XLSX_RA_CS_VISITOR`) which defines the logic that should be performed for each cell. This construction follows the `Visitor` design pattern.

Consider the following example of how a visit operation can be obtained and run:

Sample Code

```
" A selection pattern that was obtained via
XCO_CP_XLSX_SELECTION=>PATTERN_BUILDER.
DATA lo_selection_pattern TYPE REF TO if_xco_xlsx_slc_pattern.

" The read access to the worksheet.
DATA lo_worksheet TYPE REF TO if_xco_xlsx_ra_worksheet.

" The implementation of the visitor, such as a dedicated local class
containing the
" logic that will be performed for each visited cell.
DATA lo_visitor TYPE REF TO if_xco_xlsx_ra_cs_visitor.

lo_worksheet->select( lo_selection_pattern
  )->cell_stream(
  )->operation->visit( lo_visitor
  )->if_xco_xlsx_ra_operation~execute( ).
```

Row streams

Use row streams when the structure of the data that should be read is statically known. The primary use case is to read a portion of a worksheet (as identified by a selection) into an internal table. The following operations are offered for row streams:

- Write To: The write to operation takes a reference to an internal table as the input to which the selected data will be written (when running the operation)

Consider the following example of how a write to operation can be obtained and run:

Sample Code

```
" A selection pattern that was obtained via
XCO_CP_XLSX_SELECTION=>PATTERN_BUILDER.
DATA lo_selection_pattern TYPE REF TO if_xco_xlsx_slc_pattern.

" The read access to the worksheet.
DATA lo_worksheet TYPE REF TO if_xco_xlsx_ra_worksheet.

" The type definition resembling the structure of the rows in the worksheet
selection.
TYPES:
  BEGIN OF ts_row,
    first_name   TYPE string,
    last_name    TYPE string,
```

```
    day_of_birth TYPE d,
  END OF ts_row,

  tt_row TYPE STANDARD TABLE OF ts_row WITH DEFAULT KEY.

DATA lt_rows TYPE tt_row.

lo_worksheet->select( lo_selection_pattern
  )->row_stream(
  )->operation->write_to( REF #( lt_rows )
  )->if_xco_xlsx_ra_operation~execute( ).

" At this point, the internal table LT_ROWS will contain the rows from the
worksheet
" selection.
```

Accessing Data via a Cursor

An alternative to accessing the data in a worksheet via selections and streams is to get a cursor for a worksheet read access using the method `CURSOR` on `IF_XCO_XLSX_RA_WORKSHEET`. Just as with desktop office suites, you can first position a cursor on any given cell (identified by coordinate values for both the column and row of the cell). Afterwards, you can move it around the worksheet freely via the methods on `IF_XCO_XLSX_RA_CURSOR`:

- You can use the methods `MOVE_UP`, `MOVE_RIGHT`, `MOVE_DOWN` and `MOVE_LEFT` to move the cursor relative to its current position by the given number of steps
- You can use the methods `SET_COLUMN` and `SET_ROW` to set the new column or row for the cursor
- You can use the method `HAS_CELL` to determine if the underlying worksheet contains a cell for the current position of the cursor. If so, you can access it using the method `GET_CELL`

The following example illustrates how the string values of the cells in column A starting at row 5 can be read out until the first row is encountered for which the worksheet has no cell or the cell has no value:

Sample Code

```
" The read access to the worksheet.
DATA lo_worksheet TYPE REF TO if_xco_xlsx_ra_worksheet.

DATA(lo_cursor) = lo_worksheet->cursor(
  io_column = xco_cp_xlsx=>coordinate->for_alphabetic_value( 'A' )
  io_row    = xco_cp_xlsx=>coordinate->for_numeric_value( 5 )
).

WHILE lo_cursor->has_cell( ) EQ abap_true
    AND lo_cursor->get_cell( )->has_value( ) EQ abap_true.
  DATA(lo_cell) = lo_cursor->get_cell( ).

  DATA(lv_string_value) = ``.
  lo_cell->get_value(
    )->set_transformation( xco_cp_xlsx_read_access=>value_transformation-
>string_value
    )->write_to( REF #( lv_string_value ) ).

  " At this point LV_STRING_VALUE contains the string value of the cell
  " at the current position of the cursor.

  " Move the cursor down one row.
  lo_cursor->move_down( ).
```

```
ENDWHILE.
```

Reading Out Hyperlinks

When you access data using a cursor, it's also possible to read out the hyperlink associated with a given cell. Once you get a handle for a cell in the form of an object of type `IF_XCO_XLSX_RA_CELL`, it's possible to

- Check if a hyperlink is associated to the given cell via the method `HAS_HYPERLINK` on `IF_XCO_XLSX_RA_CELL`
- Get the handle for the hyperlink of the given cell via the method `GET_HYPERLINK` on `IF_XCO_XLSX_RA_CELL`

The handle for a hyperlink, `IF_XCO_XLSX_RA_HYPERLINK`, can then be used to get both the target and the location of the hyperlink via the methods `GET_TARGET` and `GET_LOCATION`. Both target and location are returned exactly as they are stored within the XLSX document.

Value Transformations

When accessing the value of an individual cell (via `IF_XCO_XLSX_RA_CELL_VALUE`) or of a complete row (as part of a row stream operation), it's possible to apply transformations to the value, which will affect how the cell value is written to an ABAP data field. Technically, a value transformation that you can get using the method `XCO_CP_XLSX_READ_ACCESS=>VALUE_TRANSFORMATION` contains a transformation routine that can be applied to

- Values of individual cells (in case the value transformation implements the interface `IF_XCO_XLSX_RA_VT_CELL_VALUE`)
- Values of rows (in case the value transformation implements the interface `IF_XCO_XLSX_RA_VT_ROW_VALUE`)

The following value transformations are currently offered:

- Identity
- String value
- Best effort

The default value transformation is the best effort value transformation. You can overwrite it using

- The method `SET_VALUE_TRANSFORMATION` of `IF_XCO_XLSX_RA_CELL_VALUE` when the value of an individual cell is read
- The method `SET_VALUE_TRANSFORMATION` of `IF_XCO_XLSX_RA_RS_OP_WRITE_TO` when row values are read and written to an internal table as part of the write to row stream operation

'Identity' value transformation

The 'identity' value transformation doesn't apply any modification to the XLSX type or value of a cell. The ABAP field that the cell value will be written to must be fully compliant with the type and value of the cell as it's stored in the XLSX file. The value is first written to a string data object, which is then written to the provided ABAP field without changes. If this write can't be performed successfully, a runtime error appears.

> **Note**
>
> For floating point numbers, please use one of the `decfloat` built-in types. Don't use packed numbers (type P) since it might lead to wrong conversions.

'String value' value transformation

The 'string value' value transformation gets the stringified value of any cell so that it can always be safely written to an ABAP field of type `STRING`.

'Best effort' value transformation

The 'best effort' value transformation is based on an inspection of the ABAP field that a given cell value will be written to. Based on the type determined for the target ABAP field, a transformation is applied to the cell value. The 'best effort' value transformation behaves just as the 'identity' value transformation except for the following types of ABAP fields:

- D: When a cell value is written to an ABAP date field, the value of the cell is interpreted as a date, and it's converted to the ABAP date format
- T: When a cell value is written to an ABAP time field, the value of the cell is interpreted as a time, and it's converted to the ABAP time format
- Data element `MSEHI`: When a cell value is written to an ABAP field typed against data element `MSEHI`, the cell value is interpreted as the external value for a unit of measurement, which is converted to the internal ABAP format using conversion routine `CUNIT`
- Data element `SPRAS`: When a cell value is written to an ABAP field typed against data element `SPRAS`, the cell value is interpreted as the external value for a language, which is converted to the internal ABAP format using conversion routine `ISOLA`

4.2.15.18.4.11.2 XLSX Write Access

The starting point for programmatically writing the content of an XLSX document is to get a write access for the document. Find out how this is done.

Context

You can create a new, empty XLSX document and get write access for it like this:

> **Sample Code**

```
DATA(lo_write_access) = xco_cp_xlsx=>document->empty( )->write_access( ).
```

An empty XLSX document consists of one worksheet named `Sheet1` which you can access via

> **Sample Code**

```
DATA(lo_worksheet) = lo_write_access->get_workbook(
```

```
  )->worksheet->at_position( 1 ).
```

You can populate it with data using the means described below. Once the worksheet has been filled as desired, you can get the corresponding file content of the document as an XSTRING via

Sample Code

```
DATA(lv_file_content) = lo_write_access->get_file_content( ).
```

Adding a new worksheet works like this:

Sample Code

```
DATA(lo_worksheet) = lo_write_access->get_workbook(
  )->add_new_sheet( ).
```

You can provide a name for the sheet by filling the optional parameter `iv_name`.

To modify an existing XSLX document, get a write access like this:

Sample Code

```
DATA(lo_write_access) = xco_cp_xlsx=>document-
>for_file_content( lv_file_content )->write_access( ).
```

⚠ Caution

Write access can also be used to update existing XLSX documents. The original documents must meet one of the following prerequisites:

- The XLSX file was originally created using the XLSX Write APIs
  or
- The XLSX file was created manually or with other tools, but its content and functional scope are fully compatible with the features supported by the XCO XLSX Write APIs (this means that the file could also be created using these APIs)

If these conditions are not fulfilled, the behavior of the update operation is undefined and correct processing can't be guaranteed. In such cases, errors may occur during processing, and the resulting XLSX documents may show unexpected behavior. Certain features, formatting, or functionalities may be altered, impaired, or lost.

Writing Data via a Stream

The first way how you can write data into a worksheet is by selecting a collection of cells based on a selection pattern (see also the section *Selection patterns* here: XLSX [page 1815]) via the method `SELECT` on `IF_XCO_XLSX_WA_WORKSHEET`. Accessing the cells contained in the selection is done following a stream-based approach, which means that a dedicated stream is obtained for the selection that will provide sequential access to the individual blocks of the selection. One kind of stream is currently offered:

- Row stream: A row stream provides access to the selection one row at a time, traversing the selection from top to bottom

Row streams

Row streams are best used when the structure of the data that should be written is statically known. The primary use case is to write the rows of an internal table into a corresponding portion of the worksheet (as identified by a selection). The internal table used to hold the data stream must only contain elementary components. The following operations are offered for row streams:

- Write From: The write from operation takes a reference to an internal table as the input whose rows will be written to the selected rows in the worksheet (upon running the operation)

Consider the following example of how a write from operation can be obtained and run:

Sample Code

```
" A selection pattern that was obtained via
XCO_CP_XLSX_SELECTION=>PATTERN_BUILDER.
DATA lo_selection_pattern TYPE REF TO if_xco_xlsx_slc_pattern.

" The write access to the worksheet.
DATA lo_worksheet TYPE REF TO if_xco_xlsx_wa_worksheet.

" The type definition for the internal table.
TYPES:
  BEGIN OF ts_row,
    first_name   TYPE string,
    last_name    TYPE string,
    day_of_birth TYPE d,
  END OF ts_row,

  tt_row TYPE STANDARD TABLE OF ts_row WITH DEFAULT KEY.

DATA lt_rows TYPE tt_row.

lo_worksheet->select( lo_selection_pattern
  )->row_stream(
  )->operation->write_from( REF #( lt_rows )
  )->execute( ).

" At this point, the rows of internal table LT_ROWS will have been written
into the
" worksheet selection.
```

Writing Data via a Cursor

An alternative to writing data into a worksheet via selections and streams is to get a cursor for a worksheet write access using the method `CURSOR` on `IF_XCO_XLSX_WA_WORKSHEET`. Just as with desktop office suites, you can position a cursor on any given cell (identified by coordinate values for both the column and row of the cell). Afterwards, you can move it around the worksheet freely via the methods on `IF_XCO_XLSX_WA_CURSOR`:

- You can use the methods `MOVE_UP`, `MOVE_RIGHT`, `MOVE_DOWN` and `MOVE_LEFT` to move the cursor relative to its current position by the given number of steps
- You can use the methods `SET_COLUMN` and `SET_ROW` to set the new column or row for the cursor
- Method `GET_CELL` provides access to the cell at the current position of the cursor

The following example illustrates how the current date and time (stored in ABAP variables `LV_DATE` and `LV_TIME`) can be written into a worksheet:

Sample Code

```
" The write access to the worksheet.
DATA lo_worksheet TYPE REF TO if_xco_xlsx_wa_worksheet.

DATA(lo_cursor) = lo_worksheet->cursor(
  io_column = xco_cp_xlsx=>coordinate->for_alphabetic_value( 'B' )
  io_row    = xco_cp_xlsx=>coordinate->for_numeric_value( 2 )
).

" Write the current date.
lo_cursor->get_cell( )->value->write_from( 'Date:' ).

DATA(lv_date) = CONV d( xco_cp=>sy->date( )->as( xco_cp_time=>format->abap )-
>value ).
lo_cursor->move_right( )->get_cell( )->value->write_from( lv_date ).

" Write the current time.
lo_cursor->move_down( )->move_left( )->get_cell( )->value-
>write_from( 'Time:' ).

DATA(lv_time) = CONV t( xco_cp=>sy->time( )->as( xco_cp_time=>format->abap )-
>value ).
lo_cursor->move_right( )->get_cell( )->value->write_from( lv_time ).
```

For an otherwise empty worksheet, this will produce the following content:

	A	B	C
1			
2		Date:	LV_DATE
3		Time:	LV_TIME

where LV_DATE is the current date and LV_TIME is the current time.

Value Transformations

When writing the value of an individual cell (via `IF_XCO_XLSX_WA_CELL_VALUE`) or of a complete row (as part of a row stream operation) it's possible to apply transformations to the value, which will affect what XLSX value is written to the cell in the worksheet. Technically, a value transformation that you can get using the method `XCO_CP_XLSX_WRITE_ACCESS=>VALUE_TRANSFORMATION` contains a transformation routine that can be applied to

- Values of individual cells (in case the value transformation implements the interface `IF_XCO_XLSX_WA_VT_CELL_VALUE`)
- Values of rows (in case the value transformation implements the interface `IF_XCO_XLSX_WA_VT_ROW_VALUE`)

The following value transformations are currently offered:

- Best effort

The default value transformation is the best effort value transformation. Value transformations can be set explicitly via the following methods:

- Method `SET_VALUE_TRANSFORMATION` of `IF_XCO_XLSX_WA_CELL_VALUE` when the value of an individual cell is written
- Method `SET_VALUE_TRANSFORMATION` of `IF_XCO_XLSX_WA_RS_OP_WRITE_FRM` when row values are written as part of the write from row stream operation

"Best effort" value transformation

The **best effort** value transformation is based on an inspection (based on ABAP runtime type services) of the ABAP field that should be written to a given cell. Based on the type determined for the ABAP field, a transformation is applied to the value before it's written to the cell of the worksheet. The **best effort** value transformation provides support for the following types of ABAP fields:

- Type `ABAP_BOOL`: When an ABAP field of type `ABAP_BOOL` is written to a cell, a boolean value will be written to the worksheet
- D: When an ABAP field of type D is written to a cell, a date value will be written to the worksheet
- T: When an ABAP field of type T is written to a cell, a time value will be written to the worksheet
- Data element `MSEHI`: When an ABAP field typed against data element `MSEHI` is written to a cell, the external value of the corresponding unit of measurement as determined by conversion routine `CUNIT` will be written to the worksheet
- Data element `SPRAS`: When an ABAP field typed against data element `SPRAS` is written to a cell, the external value of the corresponding language as determined by conversion routine `ISOLA` will be written to the worksheet
- C, N and `STRING`: When an ABAP field of type C, N or `STRING` is written to a cell, a string value will be written to the worksheet
- I, INT8 and P: When an ABAP field of type I, INT8 or P is written to a cell, a numeric value will be written to the worksheet

If an attempt is made to write an ABAP field of any other type to a cell of a worksheet using the best effort value transformation, you can expect a runtime error.

Cell Styling

XCO XLSX offers various ways to style a cell in an XSLX document.

Background Color

To change the background color of a cell, do the following:

Sample Code

```
" Get cell reference
DATA(lo_cursor) = lo_worksheet->cursor(
  io_column = xco_cp_xlsx=>coordinate->for_alphabetic_value( 'B' )
  io_row    = xco_cp_xlsx=>coordinate->for_numeric_value( 2 )
).

" Create fill style and apply to target cell
DATA(lo_fill_color) = xco_cp_xlsx=>style->fill(
    )->set_background_color( xco_cp_xlsx=>color->standard->yellow ).
```

```
lo_cursor->get_cell( )->apply_styles( VALUE #( ( lo_fill_color ) ) ).
```

Protection Configuration

You can also protect cells from modifications by locking the entire worksheet. Single cells can then be unlocked to allow selective modifications. See the following example:

Sample Code

```
" Protect a worksheet
lo_worksheet->protect( ).

" Get cell reference
DATA(lo_cursor) = lo_worksheet->cursor(
  io_column = xco_cp_xlsx=>coordinate->for_alphabetic_value( 'B' )
  io_row    = xco_cp_xlsx=>coordinate->for_numeric_value( 2 )
).

" Create and apply unlock style to target cell
DATA(lo_protection_unlock_cell) = xco_cp_xlsx=>style->protection(
    )->set_locked( abap_false ).
lo_cursor->get_cell( )->apply_styles( VALUE
#( ( lo_protection_unlock_cell ) ) ).
```

Data Validation

To show a drop-down list of values for a cell, set the data validation. Add the values that should be shown on the list by calling `add_source` with a string that contains a single value or a comma-separated list of values. See the following example:

Sample Code

```
" Get cell reference
DATA(lo_cursor) = lo_worksheet->cursor(
  io_column = xco_cp_xlsx=>coordinate->for_alphabetic_value( 'B' )
  io_row    = xco_cp_xlsx=>coordinate->for_numeric_value( 2 )
).

" Define validation list with multiple value sources
DATA(lo_data_validation) = xco_cp_xlsx=>data_validation_type-
>if_xco_xlsx_dat_val_type_f~list(
  )->add_source( '1'
  )->add_source( '2'
  )->add_source( '3,4,5' ).

" Apply validation to target cell
lo_cursor->get_cell( )->data_validation->set_type( lo_data_validation ).
```

Font Customization

You can change the font in various ways as depicted in the following example:

```
" Get cell reference
DATA(lo_cursor) = lo_worksheet->cursor(
  io_column = xco_cp_xlsx=>coordinate->for_alphabetic_value( 'B' )
  io_row    = xco_cp_xlsx=>coordinate->for_numeric_value( 2 )
).

" Configure multiple font attributes
DATA(lo_font) = xco_cp_xlsx=>style->font( ).
```

```
lo_font->set_color( xco_cp_xlsx=>color->standard->orange
  )->set_type( xco_cp_xlsx=>font_type->arial
  )->set_size( 16
  )->set_bold(
  )->set_italic(
  )->set_strikethrough(
  )->set_superscript(
  )->set_underline_type( xco_cp_xlsx=>underline_type->single ).

" Apply font style to target cell
lo_cursor->get_cell( )->apply_styles( VALUE #( ( lo_font ) ) ).
```

Text Alignment Customization

You can customize the text alignment, as shown in the following example:

```
" Get cell reference
DATA(lo_cursor) = lo_worksheet->cursor(
  io_column = xco_cp_xlsx=>coordinate->for_alphabetic_value( 'B' )
  io_row    = xco_cp_xlsx=>coordinate->for_numeric_value( 2 )
).

" Configure alignment properties
DATA(lo_alignment) = xco_cp_xlsx=>style->alignment( ).

lo_alignment->set_horizontal_alignment( xco_cp_xlsx=>horizontal_alignment-
>right ).
lo_alignment->set_indent( 2 ).
lo_alignment->set_vertical_alignment( xco_cp_xlsx=>vertical_alignment->center ).
lo_alignment->set_wrap_text( ).

" Apply alignment to target cell
lo_cursor->get_cell( )->apply_styles( VALUE #( ( lo_alignment ) ) ).
```

Cell Border Customization

You can customize the borders of individual cells, with each border side (top, bottom, left, right) having its own style and color. You can also apply an outline to a range of cells. You can customize borders for a single cell like this:

Sample Code

```
" Get cell reference
DATA(lo_cursor) = lo_worksheet->cursor(
  io_column = xco_cp_xlsx=>coordinate->for_alphabetic_value( 'B' )
  io_row    = xco_cp_xlsx=>coordinate->for_numeric_value( 2 )
).
" Configure border properties
DATA(lo_border) = xco_cp_xlsx=>style->border( ).
" You can set individual border side.
lo_border->set_top(
  io_style = xco_cp_xlsx=>border_style->dashed
  io_color = xco_cp_xlsx=>color->standard->blue
).
lo_border->set_bottom(
  io_style = xco_cp_xlsx=>border_style->thick
  io_color = xco_cp_xlsx=>color->standard->blue
).
lo_border->set_left(
  io_style = xco_cp_xlsx=>border_style->double
  io_color = xco_cp_xlsx=>color->standard->blue
).
lo_border->set_right(
  io_style = xco_cp_xlsx=>border_style->medium
  io_color = xco_cp_xlsx=>color->standard->blue
```

```
  ).
  " You can also set the cell outline.
  lo_border->set_outline(
    io_style = xco_cp_xlsx=>border_style->medium
    io_color = xco_cp_xlsx=>color->standard->blue
  ).
  " Apply alignment to target cell
  lo_cursor->get_cell( )->apply_styles( VALUE #( ( lo_border ) ) ).
```

You can apply an outline to a range of cells like this:

Sample Code

```
" Get cells pattern reference
DATA(lo_pattern) = xco_cp_xlsx_selection=>pattern_builder->simple_from_to(
  )->from_column( xco_cp_xlsx=>coordinate->for_alphabetic_value( 'A' )
  )->from_row( xco_cp_xlsx=>coordinate->for_numeric_value( 1 )
  )->to_column( xco_cp_xlsx=>coordinate->for_alphabetic_value( 'C' )
  )->to_row( xco_cp_xlsx=>coordinate->for_numeric_value( 2 )
  )->get_pattern( ).
lo_worksheet->set_outline(
  io_pattern = lo_pattern
  io_style   = xco_cp_xlsx=>border_style->thick
  io_color   = xco_cp_xlsx=>color->standard->purple
).
```

Worksheet Customizing

You can customize your worksheet with tab coloring, cell merging operations, and much more. The method `merge_cells` applies cell merging to the worksheet using a specified rectangular pattern that must represent a valid worksheet range (such as `A1:C5`). Only contiguous rectangular areas are allowed. The method `unmerge_cells` splits previously merged cells back into individual cells within the given pattern, which only affects merged ranges that are fully contained within the given pattern. An example:

```
" Set tab color
lo_worksheet->set_tab_color( xco_cp_xlsx=>color->standard->yellow ).

" Get cells pattern reference
DATA(lo_pattern) = xco_cp_xlsx_selection=>pattern_builder->simple_from_to(
  )->from_column( xco_cp_xlsx=>coordinate->for_alphabetic_value( 'A' )
  )->from_row( xco_cp_xlsx=>coordinate->for_numeric_value( 1 )
  )->to_column( xco_cp_xlsx=>coordinate->for_alphabetic_value( 'C' )
  )->to_row( xco_cp_xlsx=>coordinate->for_numeric_value( 2 )
  )->get_pattern( ).

" Merge and unmerge cells
lo_worksheet->merge_cells( lo_pattern ).
lo_worksheet->unmerge_cells( lo_pattern ).

" Set the outline of cells range
lo_worksheet->set_outline(
  io_style = xco_cp_xlsx=>border_style->medium
  io_color = xco_cp_xlsx=>color->standard->blue
).

" Protect
lo_worksheet->protect( ).
```

```
" Show or remove Gridline
lo_worksheet->show_gridlines( abap_true ).
lo_worksheet->show_gridlines( abap_false ).
```

4.2.15.18.4.12 XString

To offer a convenient way to incorporate standard operations on binary data into application logic, the XCO Library provides a standard abstraction for xstrings, `IF_XCO_XSTRING`, that is tightly integrated with the XCO string abstraction `IF_XCO_STRING`.

Using the Base64 binary-to-text encoding

Built-in conversions are provided to support working with the Base64 binary-to-text encoding. Encoding raw binary data to its Base64 representation and decoding a given Base64 representation to its underlying binary data can be accomplished as follows:

Sample Code

```
" LV_XSTRING can store arbitrary binary data.
DATA lv_xstring TYPE xstring.
" LV_BASE64_ENCODING is of type STRING and contains the Base64 encoded version
" of LV_XSTRING.
DATA(lv_base64_encoding) = xco_cp=>xstring( lv_xstring
  )->as_string( xco_cp_binary=>text_encoding->base64
  )->value.
" LV_ORIGINAL_XSTRING is of type XSTRING and contains the binary data encoded
" in LV_BASE64_ENCODING
DATA(lv_original_xstring) = xco_cp=>string( lv_base64_encoding
  )->as_xstring( xco_cp_binary=>text_encoding->base64
  )->value.
```

Using code pages

An xstring can also be converted to a string by using a code page, e.g. UTF-8:

Sample Code

```
DATA(lv_string) = xco_cp=>xstring( lv_xstring
  )->as_string( xco_cp_character=>code_page->utf_8
  )->value.
```

4.2.15.19 Metric Providers

The following objects were released for use in the ABAP environment:

Object	Description
CL_GSM_API_TEST	Convenience class to test the metric provider implementation
CX_GSM_API	Metric provider exception class
IF_GSM_API_CONSTANTS	Interface for metric provider constants
IF_GSM_API_METRIC	Interface for metric definition
IF_GSM_API_METRIC_GROUP	Interface for metric group definition
IF_GSM_API_MODEL	Interface for metric provider model definition
IF_GSM_API_PROVIDER	Interface for metric provider definition
IF_GSM_API_TEST	Interface for metric provider testing
IF_GSM_API_TYPES	Interface for metric provider type definitions

These objects are all related to the ABAP repository object called metric provider. You use metric providers as well as these released objects in ABAP development tools in Eclipse when you want to create your own metrics for health monitoring of the ABAP system.

Related Information

Developing Metrics for Health Monitoring [page 1223]

4.2.15.20 Integration to SAP BTP Services

Integrating SAP Document Management Service [page 3320]

Proxy API for SAP Build Process Automation [page 1653]

4.2.15.21 Scoping Framework

Learn how to use the scoping API.

Scoping is a mechanism that determines the visibility of certain development object types at configuration time. Depending on your decisions, some object types may be hidden or displayed in the current client.

For instance, in the Business Roles app, not all business catalogs shall be available for the administrator user, because their usage is subject to additional license constraints.

To scope the relevant objects, you can use the API `IF_APS_BC_SCOPE_CHANGE_API`. The interface provides mass-enabled methods for setting the scope state of scoping-enabled `TADIR` objects in the current client.

> **Note**
>
> The objects to be scoped must be in the same software component as the caller.

The following `TADIR` types are supported:

List of TADIR Objects That Can Be Scoped

Constant	TADIR Object
`CMHC`	Cloud Management Health Check
`ESH1`	ESH: CDS Search Model
`PCFN`	Predefined Field: Extensible Node
`SCO1`	Communication Scenario
`SIA1`	Business Catalog
`SIA6`	IAM App
`SIA8`	Business Role Template
`UIST`	Fiori Launchpad Space Template
`UIPG`	Fiori Launchpad Page Template

To scope one of these objects, you have to provide the object type, object name, and the desired scoping state (`ON` or `OFF`).

> **Note**
>
> Objects that are set to scoping state `ON` should not be set to `OFF`. The object must remain available to the customer to guarantee further access to the data maintained via the object.

For further information, see the long text documentation of the API `IF_APS_BC_SCOPE_CHANGE_API`.

4.2.15.22 API for Consumption of Business Rules

The purpose of the CL_FDT_FUNCTION_PROCESS_API is to enable consumption of BRF+ (Business Rule Framework plus) rules within the ABAP Cloud development environment. It acts as a bridge between your application and BRF+ functions(rules).

The following table shows methods in CL_FDT_FUNCTION_PROCESS_API class:

Method	Parameter	Description
`Get_Content_List`	Importing: `IV_FUNCTION_ID`	ID of the function to be processed.

Method	Parameter	Description
This method is used to get the list of context elements for BRFplus function.	Importing: V_TIMESTAMP:	The timestamp parameter specifies the data retrieval or function evaluation time. Without a timestamp, the function returns the context list at the current date and time.
	Importing: IV_TRACE_GENERATION	This Boolean indicates that the trace should be generated with a lean trace or not.
	Exporting: ET_CONTEXT_LIST	The exporting parameter is a table containing the function's context list, which includes the id, name, and data object type of each context.
GET_CONTEXT_VALUE This method is used to get the value of the context object of a BRFplus function in a referenced object.	Importing: IV_FUNCTION_ID	ID of the function to be processed.
	Importing: IV_DATA_OBJECT	A character-like parameter that specifies the name or ID of the data object involved in the function call.
	Importing: IV_TIMESTAMP	The timestamp parameter specifies the point in time for data retrieval or function evaluation. If no timestamp is provided, the function returns the context value at the current date and time.
	Importing: IV_TRACE_GENERATION	This Boolean indicates that the trace should be generated with a lean trace or not.
	Exporting: EV_DATA	This parameter exports the data object's actual value after function execution. Its type "ANY" indicates flexibility to handle any data type, expanding the function's return possibilities.
	Exporting: ER_DATA	This exporting parameter references the data object, enabling its manipulation or examination beyond the function call.
GET_DATA_OBJECT_REFERENCE This method is used to get the data reference of a context object in BRFplus function.	Importing: IV_FUNCTION_ID	ID of the function to be processed.
	Importing: IV_DATA_OBJECT:	Importing: IV_DATA_OBJECT: A character-like parameter that specifies the name or ID of the data object involved in the function call.
	Importing: IV_DATA_OBJECT	A character-like parameter that specifies the name or ID of the data object involved in the function call.

Method	Parameter	Description
	Importing: IV_TIMESTAMP	Importing: IV_TIMESTAMP: The timestamp parameter specifies a specific point in time for data retrieval or function evaluation. If no timestamp is provided, all function context in a referenced object at the current date and time is returned.
	Importing: IV_TRACE_GENERATION	Importing: IV_TRACE_GENERATION: This Boolean indicates that the trace should be generated with a lean trace or not.
	Exporting: EV_DATA_OBJECT_NAME	This parameter exports the technical name of the data object involved in the operation, providing metadata such as its name or identifier.
	Exporting ER_DATA	This exporting parameter provides a reference to the data object. It does not contain an actual value but rather a pointer to the data object.
MOVE_DATA_TO_DATA_OBJECT This method is used to pass data from external source to BRFplus function's context.	Importing: IR_DATA	This is a reference to any ABAP data object. It allows the method to directly interact with ABAP data objects provided by the calling program, offering a flexible interface for data processing.
	Importing: IV_FUNCTION_ID	ID of the function to be processed.
	Importing: IV_DATA_OBJECT:	A character-like parameter that specifies the name or ID of the data object involved in the function call.
	Importing: IV_TIMESTAMP	The timestamp parameter is used to define a specific point in time for the data retrieval or function evaluation. If no time stamp is provided, then it takes current date and time.
	Importing: IV_TRACE_GENERATION	This Boolean indicates that the trace should be generated with a lean trace or not.
	Importing: IV_HAS_DDIC_BINDING	Indicates whether the data passed to the method confirms to a DDIC (Data Dictionary) structure as defined in BRF+.
	Exporting: ER_DATA	This exporting parameter exports a reference to a BRFplus data object. This allows the method to return a data object that can be further manipulated or evaluated outside the method.
PROCESS	Importing: IV_FUNCTION_ID	ID of the function to be processed.

Method	Parameter	Description
With this method, a BRFplus function can be processed. You can specify a timestamp for the function to be processed so you can simulate the system behavior as it would have been at that point. Also, you can decide whether the trace for the function execution shall be saved in database for later reference or not.	Importing: `IV_TIMESTAMP`	The timestamp parameter is used to define a specific point in time for the data retrieval or function evaluation. If no time stamp is provided, the result of the function at the current date and time is returned.
	Importing: `IV_TRACE_MODE`	Indicates the modes of trace. By default, mode is 'L' (Lean Trace).
	Importing: `: IV_SAVE_TRACE`	Indicates whether the trace generated for current processing of the Function should be saved in database.
	Importing: `IT_ID_VALUE`	Table of ID/value pairs (ID/reference to value for each context element). From a performance perspective, this is the most efficient way of triggering the processing of BRFplus. Also, using this parameter requires that you know the IDs of the context elements that you want to pass.
	Exporting: `EA_RESULT`	Exporting: EA_RESULT: A variable of a type that is compatible with the result data object of the BRFplus function. You can get an ABAP data object of this type (a reference) by calling method `CL_FDT_FUNCTION_PROCESS_API ->IF_FDT_FUNCTION_PROCESS_API~GET_DATA_OBJECT_REFERENCE`.

4.2.15.23 Business Event Logging

This section describes all the released objects in Business Event Logging.

4.2.15.23.1 Log Custom Business Events

You can create your own business events with SAP-defined or customer-defined objects. For more information on creating events, see Business Events.

Define Events for Business Event Logging

The event binding can contain an SAP-defined or a customer-defined object and and an operation.

Business events should be declared in the correct node they refer to - the child node or the root node.

You must add the `@ObjectModel.sapObjectNodeType.name` annotation in the abstract event entity denoting the SAP Object Node Type the event refers to.

`@ObjectModel.sapObjectNodeType.name Annotation`

`@ObjectModel.sapObjectNodeType.name Annotation` : The name of the object components the event refers to.

This can be an SAP-defined object component like SalesOrderItem, or a customer defined object component. For example: Z_MyObjectItem. If the event refers to the object as a whole or the header of the object, then the object component name should be the same as the object component name in the event binding.

Sample Code

```
@ObjectModel.sapObjectNodeType.name: 'Z_MySalesOrderItem'
define abstract entity ZD_SALESORDERITEMCREATED
{ ItemNo : item_no
 Status : Status }
```

In case an event referring to a child node is exceptionally assigned to a root node to support backward compatibility, then the following `@Event.sapObjectNodeTypeKey` annotation is additionally required:

`@Event.sapObjectNodeTypeKey Annotations`

`@Event.sapObjectNodeTypeKey Annotations` : The key of the object node the event refers to needs to be defined if this is a customer defined object component, or if the key is deviating from SAP standard. The elements of the key must either be defined in the underlying behaviour definition, or the abstract entity itself.

Note

The order of keys in the annotation should be the same as they are in the node's corresponding entity.

Sample Code

```
@ObjectModel.sapObjectNodeType.name: 'Z_MySalesOrderItem'
 @Event.sapObjectNodeTypeKey : [ { element : 'SalesOrder' } , { element :
'ItemNo' } ]
define abstract entity ZD_SALESORDERITEMCREATED
 { ItemNo : item_no
Status : Status }
```

Note

For partners who implement the same scenario under a partner namespace, change documents can be logged as business events only if the event binding version is C2 released.

Reset Shared Memory

Any new RAP object or changed object will be considered for logging only when the shared memory (metadata buffer) is reset. Business Event Logging resets the memory once everyday to ensure that the new or changed objects are considered for logging within the framework.

4.2.15.23.2 Log Custom Change Documents

Change documents defined in custom RAP business objects are automatically logged as business events when the corresponding business is activated for logging in the Activate Business Event Logging configuration.

For information on defining change documents for a managed RAP business object as a customer, see Enabling Change Documents Integration.

> ⓘ Note
>
> For partners who implement the same scenario under a partner namespace, change documents can be logged as business events only if the event binding version is C2 released.

Reset Shared Memory

Any new RAP object or changed object will be considered for logging only when the shared memory (metadata buffer) is reset. Business Event Logging resets the memory once everyday to ensure that the new or changed objects are considered for logging within the framework.

4.2.15.23.3 Direct Logging API

Context

You can use the class-based **Direct Logging API** to write to the Business Event Log without creating a Business Event. This would result in adding entries for past business events to the business event log.

For an entry in the Business Event Log, the attribute ‘source of data’ will indicate whether the entry was created via Business Event or via the Direct Logging API.

Creating an instance to log business event

If you want to write past business events to the business event log, you require an instance of class CL_BEL_DIRECT_LOGGING. To create this instance, the class CL_BEL_DIRECT_LOGGING provides the following methods:

Methods

Method	Description
`IF_BEL_DIRECT_LOGGING~LOG_BUSINESS_EVENT`	Logs business events
`GET_INSTANCE`	Creates an object instance

The Class `CL_BEL_DIRECT_LOGGING` can be consumed via the interface `IF_BEL_DIRECT_LOGGING`. Create the instance of the API using `GET_INSTANCE` method of class `CL_BEL_DIRECT_LOGGING`. The method returns the object reference of type `IF_BEL_DIRECT_LOGGING`.

```
8
9   DATA : lo_log_api TYPE REF TO  if_bel_direct_logging.
10
11  lo_log_api = cl_bel_direct_logging=>get_instance( ).
12
13
```

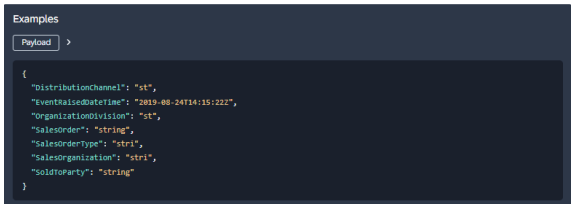
The API is invoked via the `LOG_BUSINESS_EVENT`method of interface `IF_BEL_DIRECT_LOGGING`. The method takes input as different events and returns the results in terms of return code and an internal table containing processing results of each event.

Method Parameters

The method has the following parameters:

- `IT_EVENTS`: This is an input parameter of type table and contains the list of events that need to be logged into the Business Event Logging component. The main components of this parameter are:

Parameter	Description
EVENT_GUID	Unique identifier for a given event. This has to be provided by the consumer application, and it helps us identify whether an event has already been logged. By default, an empty GUID is automatically generated for EVENT_GUID. **Note** If you want to use the direct event API to add single or multiple events with past timestamp to the business event log, you should generate the EVENT_GUID manually. You do this the same way that you generate the event ID for events. With the same EVENT_GUID, the ID will be stored only once, even if it is added multiple times. For example, you can create the event ID by concatenating and then hashing the object key, event type, and timestamp of the event.
SOURCE_SYSTEM	This identifies the system where the event was generated. It is an optional parameter. By default, it would be filled with the current logical system.
TRANSACTION_ID	An ID uniquely identifying the transaction in which the business event was run. Business events with the same IDs are considered to have been run at the same time and by default the transaction_IDS will be automatically filled with the ID of the current kernel transaction. **Note** If you want to use the direct event API to add single or multiple events with past timestamp to the business event log, then we recommend that you provide the transaction ID manually. Ensure that you provide a unique ID for each event. For example: Use the same ID that you have used for the event id.
EXEC_USER	This identifies the user who performed the business activity that created the event. Although it is an optional parameter, we recommend that you provide it. By default, it will be filled with the current logon user.
EXEC_TIMESTAMP	The timestamp at which the event was created. Although it is an optional parameter, we recommend that you provide it. By default, it would be filled with the current timestamp. The timestamp can be in the past.

Parameter	Description
EVENT_TYPE	This is a mandatory parameter. This string carries the metadata such as the object and version information about the event. It is given in a specific format. For example: **Sample Code** `sap.s4.beh.salesorder.v1.SalesOrder.Changed.v1` In the event type string, the Object is **SalesOrder**, the Event Operation is **Changed**, and the Event Version is **v1**. For a given Business Event, the EVENT_TYPE can be found in the events section on the SAP Business Accelerator Hub. SUB `ce/sap/s4/beh/salesorder/v1/SalesOrder/Changed/v1` When you copy the string, ensure that you copy from "sap" and replace the forward slash (/) with a period (.).
EVENT_DATA	This is a mandatory parameter. It carries event information such as business event data (payload) of an event. The definition will consist of the fields of event. For a given business event, the EVENT_DATA structure and data types can be found in the events section on the SAP Business Accelerator Hub. An example of Event_data is given below:  **Note** The fields don't have to be in the same order as in the SAP Business Accelerator Hub.
EV_RETCODE	The return code may consist of 3 different values indicating the result of processing. - 0 = Successful. - 1 = Successful with warnings. - 2 = Errors.

Parameter	Description
CT_RETURN	This table parameter consists of the result of processing of each event which was passed via the IT_EVENTS parameter. The components of the return table are as • EVENT_GUID: Event identifier passed by the consumer application. • EVENT_TYPE: Event type passed by the consumer application. • EXEC_TIMESTAMP: Event's timestamp passed by consumer application. • EVENT_DATA: Event-specific data passed by the consumer application. • TYPE: Message type: S Success, E Error, W Warning • ID: Message Class • NUMBER: Message Number • MESSAGE_V1: Event identifier passed by the consumer application. • MESSAGE_V2: Event type passed by the consumer application. • MESSAGE_V3: First 50 characters of event data. • MESSAGE_V4: Event's timestamp passed by consumer application.

Below is an example of how the API can be consumed. This sample contains the mandatory parameters:

Sample Code

```
TYPES : BEGIN OF ty_so_changed,
          DistributionChannel  TYPE c LENGTH 2,
          EventRaisedDateTime  TYPE tzntstmpl,
          OrganizationDivision TYPE c LENGTH 2,
          SalesOrder           TYPE c LENGTH 10,
          SalesOrderType       TYPE c LENGTH 4,
          SalesOrganization    TYPE c LENGTH 4,
          SoldToParty          TYPE c LENGTH 10,
        END OF ty_so_changed.
DATA : lr_structdescr TYPE REF TO cl_abap_structdescr,
       lr_typedescr   TYPE REF TO cl_abap_typedescr,
       lo_data        TYPE REF TO data,
       lt_event       TYPE REF TO bel_t_event_data,
       lv_retcode     TYPE bel_api_retcode,
       lt_return      TYPE REF TO bel_t_api_return.
DATA : lo_log_api TYPE REF TO if_bel_direct_logging.
cl_abap_typedescr=>describe_by_name(
  EXPORTING
    p_name         = 'ty_so_changed'
  RECEIVING
    p_descr_ref    = lr_typedescr
  EXCEPTIONS
    type_not_found = 1
    OTHERS         = 2
).
IF sy-subrc <> 0.
  MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
    WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
ENDIF.
```

```
lr_structdescr = CAST cl_abap_structdescr( lr_typedescr ).
CREATE DATA lo_data TYPE HANDLE lr_structdescr.
ASSIGN lo_data->* TO FIELD-SYMBOL(<fs_data>).
CHECK <fs_data> IS ASSIGNED.
ASSIGN COMPONENT  'DistributionChannel' OF STRUCTURE <fs_data> TO FIELD-
SYMBOL(<fs_dc>).
IF sy-subrc = 0.
  <fs_dc> = '01'.
ENDIF.
ASSIGN COMPONENT  'EventRaisedDateTime' OF STRUCTURE <fs_data> TO FIELD-
SYMBOL(<fs_tstp>).
IF sy-subrc = 0.
  GET TIME STAMP FIELD <fs_tstp>.
ENDIF.
ASSIGN COMPONENT  'OrganizationDivision' OF STRUCTURE <fs_data> TO FIELD-
SYMBOL(<fs_od>).
IF sy-subrc = 0.
  <fs_od> = 'AB'.
ENDIF.
ASSIGN COMPONENT  'SalesOrder' OF STRUCTURE <fs_data> TO FIELD-
SYMBOL(<fs_so>).
IF sy-subrc = 0.
  <fs_so> = '0000000100'.
ENDIF.
ASSIGN COMPONENT  'SalesOrderType' OF STRUCTURE <fs_data> TO FIELD-
SYMBOL(<fs_sotyp>).
IF sy-subrc = 0.
  <fs_sotyp> = 'ABCD'.
ENDIF.
ASSIGN COMPONENT  'SalesOrganization' OF STRUCTURE <fs_data> TO FIELD-
SYMBOL(<fs_soorg>).
IF sy-subrc = 0.
  <fs_soorg> = 'WXYZ'.
ENDIF.
ASSIGN COMPONENT  'SoldToParty' OF STRUCTURE <fs_data> TO FIELD-
SYMBOL(<fs_spty>).
IF sy-subrc = 0.
  <fs_spty> = '2000200001'.
ENDIF.
CREATE DATA lt_event   TYPE bel_t_event_data.
CREATE DATA lt_return  TYPE bel_t_api_return.
lt_event->* = VALUE #( (  event_type =
'sap.s4.beh.salesorder.v1.SalesOrder.Changed.v1'  event_data = lo_data   ) ).
lo_log_api = cl_bel_direct_logging=>get_instance( ).
lo_log_api->log_business_event(
  EXPORTING
    it_events  = lt_event
  IMPORTING
    ev_retcode = lv_retcode
  CHANGING
    ct_return  = lt_return
).
```

Troubleshooting issues

The API can throw a Warning(W) return code in case the event is already existing in the BEL database or the event is not activated to be logged into the BEL framework. The API can return error(E) return code in case the mandatory parameters are not passed or passed with initial values, if the authorization check fails, if the EVENT_DATA definition is passed incorrectly. The details of the error/warning can be seen in the CT_RETURN table parameter of the API. Some examples of error/warning are given below:

- Business Event is not defined (E): The supplied business event is not defined. Ensure that the business event exists in the SAP Business Accelerator Hub.

- Business event with Identical ID exists (W): The business event supplied already exists in the system. The corresponding event will be skipped.
- Business Event not enabled for logging (W): The business event is not enabled for logging in the system.
- Business event processed successfully (S): Business event is successfully logged into the system.
- Business event data not supplied (E): Event-related data is not supplied. It is mandatory to provide the event data.
- Business event keys can't be determined (E): For the given event, refer to SAP Business Accelerator Hub and verify if the fields are correct.
- Keys can't be retrieved from event data (E): For the given event, refer to SAP Business Accelerator Hub and verify if the fields are correct

4.2.15.23.4 Accessing Business Event Log Data using CDS Views

The following views are released for key user extensibility:

CDS Views	Description
C_BUSEVTLOGEVENTDEX_2	This CDS view provides business event header data.
C_BUSEVTLOGPAYLOADDEX_2	This CDS view provides a subset of event payload details.
C_BUEVLGEVTFULLPAYLOADJSONDEX	This CDS view provides full business event data (payload) in JSON format.

4.2.16 UI Development

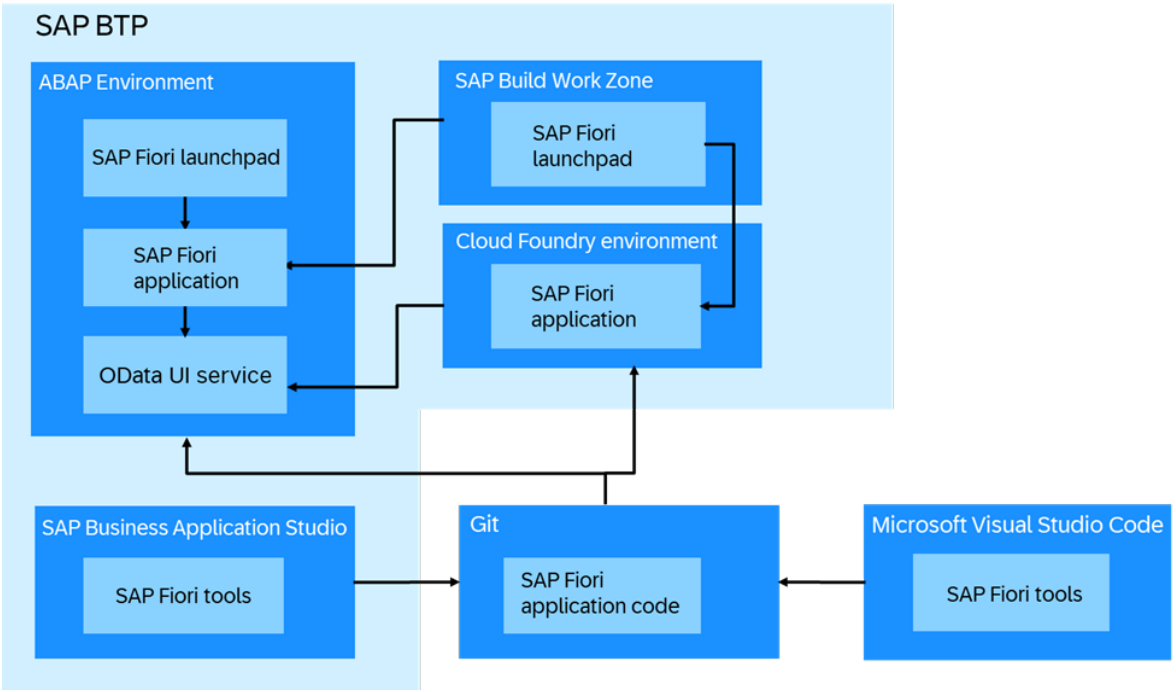
Learn more about the different options to develop SAP Fiori applications for an existing OData UI service, where to store the application, and how to enable business users to access the application from SAP Fiori launchpad.

Overview

You can develop SAP Fiori applications for OData UI services that are exposed as a UI service. See OData service Consumption.

- SAP Fiori is a design system that enables you to create business apps with a consumer-grade user experience, turning casual users into SAP experts with simple screens that run on any device. Apps that follow the Fiori design principle can be built using a combination of SAP UI5 and SAP Fiori Elements. See SAP Fiori.
- SAPUI5 is a development framework that a developer would use to actually build a front-end application that follows the Fiori design guidelines. UI5 apps are built using HTML5, JS, XML, OData and JSON, and are based on the Model, View, Controller (or MVC) design pattern. See SAPUI5.

- SAP Fiori Elements is a framework that comprises the most commonly used floorplan templates and is designed to speed up development by reducing the amount of frontend code needed to build SAP Fiori apps, and driving UX consistency and compliance with the latest SAP Fiori design guidelines. See SAP Fiori Elements.



- [https://help.sap.com/docs/btp/sap-fiori-launchpad-for-sap-btp-abap-environment/sap-fiori-launchpad-user-guide?version=Cloud]
- [https://help.sap.com/docs/btp/sap-business-technology-platform/sap-fiori-applications-in-cloud-foundry-environment?version=Cloud]
- [https://help.sap.com/docs/btp/sap-business-technology-platform/sap-business-application-studio?version=Cloud]
- [https://www.git-scm.com/]
- [https://help.sap.com/docs/SAP_FIORI_tools/17d50220bcd848aa854c9c182d65b699/f09752ebcf63473e9194ea29ca232e56.html?q=sap%20fiori%20tools]
- [https://help.sap.com/docs/SAP_FIORI_tools/17d50220bcd848aa854c9c182d65b699/17efa217f7f34a9eba53d7b209ca4280.html?q=cloud%20foundry%20environment]

An OData UI service makes it possible to consume a RAP business object with a Fiori elements UI or other UI clients. That means, a Fiori UI application reads the information in the metadata and creates the matching UI for the service.

SAP does not provide UI services for consumption but instead business object interfaces or ABAP Core Data Services that are released with **Contract C1: Use System-Internally** that can be utilized.

Not all business object interfaces and CDS views are suitable for UI consumption. Only those explicitly annotated with the `@ObjectModel.supportedCapabilities: [#UI_PROVIDER_PROJECTION_SOURCE]`

annotation are intended for UI use (see `ObjectModel.supportedCapabilities`). These business object interfaces/CDS views come with the following qualities:

- Text relations: Available for all ID fields with associated texts.
- Value helps: Provided for all ID fields that have texts.
- Additional fields: Exposes commonly used fields and associations.
- Draft enablement: Available for business object interfaces when create, update, and delete behaviors are implemented.

For a UI developer to implement an SAP Fiori application using these business object interfaces/CDS views, a custom OData UI service must be created first.

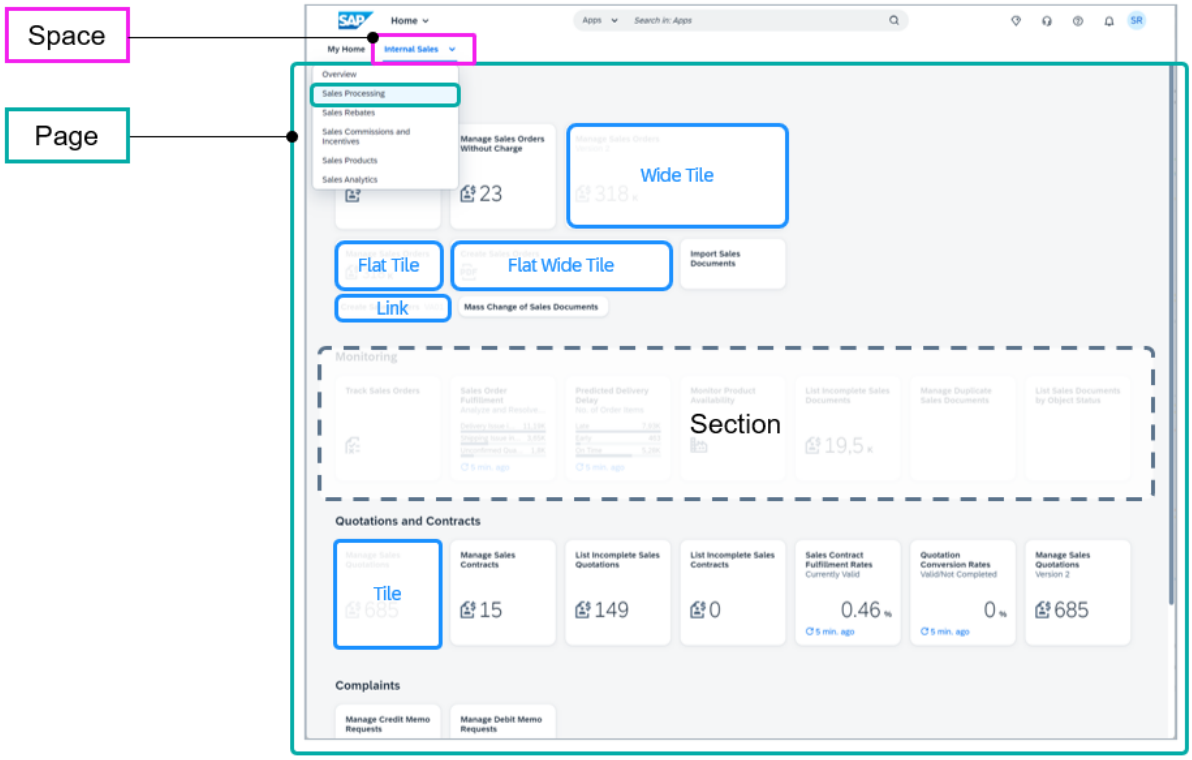
In order to develop SAP Fiori applications on top of OData UI services, you use SAP Business Application Studio or Visual Studio Code with SAP Fiori tools extensions. SAP Business Application Studio supports integration with Git so that you can use Git as source control system and store the code of SAP Fiori applications in remote git repositories. Once you have implemented the custom SAP Fiori application, you can deploy it to the ABAP environment or the Cloud Foundry environment. You can launch the deployed apps standalone or embed them into the SAP Fiori launchpad. The ABAP environment comes with an embedded Fiori Launchpad and provides access to Fiori apps that are stored on the SAPUI5 ABAP repository. Apps stored in the HTML5 application repository can be added to a launchpad provided by the SAP Build Work Zone. It is also possible to add apps stored on the SAPUI5 ABAP repository to launchpads provided by the SAP Build Work Zone. See Integration Scenario"

Spaces and pages structure the way apps and other content appears in the SAP Fiori launchpad. They offer more flexibility to influence the launchpad layout for different user groups. Each user can see one or more spaces that contain one or more pages. The pages show apps clustered in different sections.

A *Space* is assigned to a business role and offers a structured layout of the business role content. Each space must consist of at least one page.

A *Page* holds the actual layout information for the corresponding space. It groups the apps visualized by tiles into different sections from the work perspective of the end user.

The following graphic illustrates the concept of spaces and pages.



Read Managing Launchpad Spaces and Pages for an in-depth description of the concept in the SAP Business Technology Platform. For more information on how to use spaces and pages in SAP Build Work Zone, please refer to Manual Configuration of Spaces and Pages.

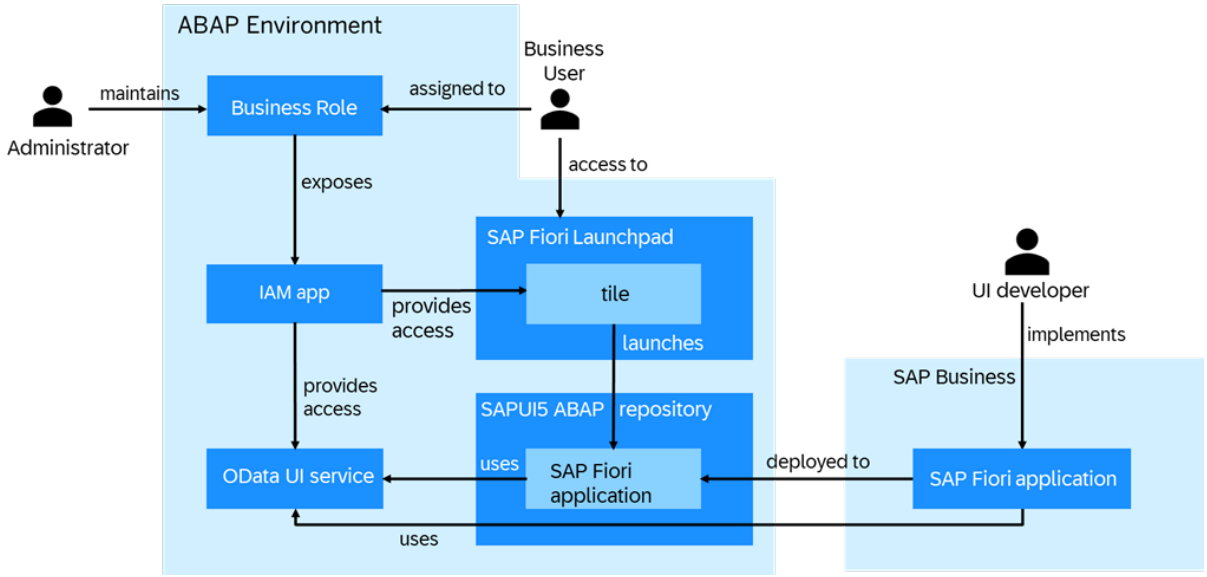
SAP Fiori Applications in the ABAP environment

If you want to deploy an SAP Fiori application to the ABAP environment, the following users are involved:

- A UI developer implements the SAP Fiori application against the OData UI service and defines the tile as part of the application. Once this is done, the developer deploys the SAP Fiori application to the SAPUI5 ABAP repository.

  > **Note**
  >
  > The SAPUI5 ABAP repository is part of the ABAP environment and is the umbrella term for the single SAPUI5 repository of each application. Technically, the SAPUI5 ABAP repository is based on the Business Server Page (BSP) repository. Each SAPUI5 repository is represented by an individual BSP application. The SAPUI5 ABAP repository is also used for delivering SAPUI5 apps of the ABAP environment.

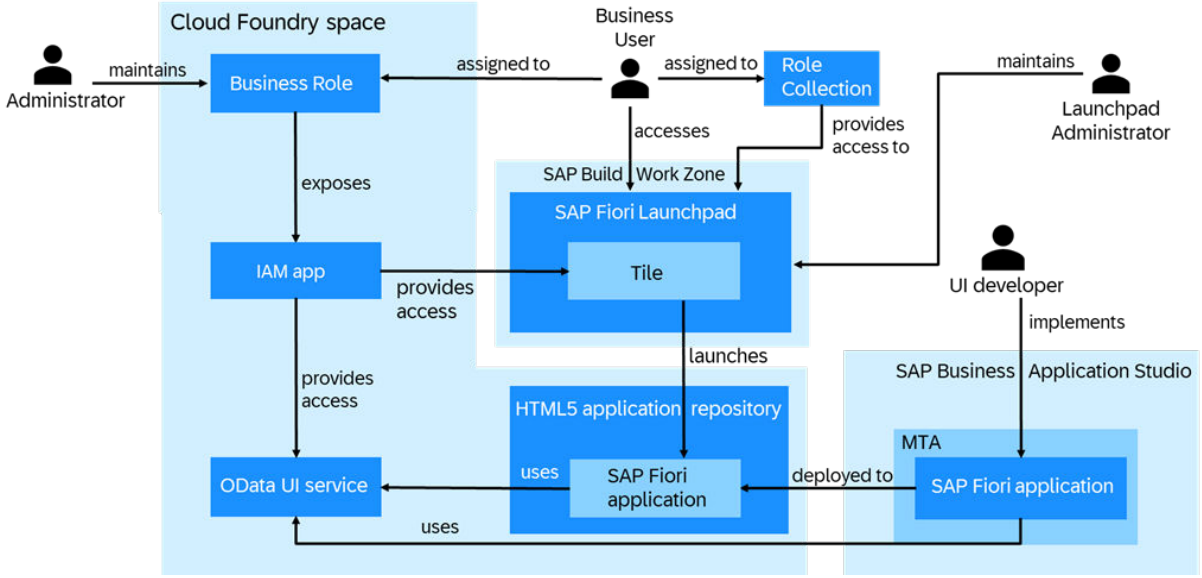
- An administrator in the ABAP environment provides access to the OData UI service and the tile via a business role.
- A business user that is assigned to the business role can access the tile from SAP Fiori launchpad and launch the application.



SAP Fiori Applications in the Cloud Foundry environment

If you want to deploy an SAP Fiori application to the Cloud Foundry environment, the following users are involved:

- A UI developer implements the SAP Fiori application against the OData UI service as part of a multi-target application (MTA).
  An MTA is logically a single application comprised of multiple parts created with different technologies, which share the same lifecycle. The developers of the MTA describe the desired result using the MTA model, which contains MTA modules, MTA resources, and interdependencies between them.
- The developer defines the tile as part of the application. Once this is done, the MTA is deployed to the Cloud Foundry space. By doing so, the SAP Fiori application of the MTA is deployed to the HTML5 application repository. The developer can deploy the MTA either manually or as part of a CI/CD pipeline. See Continuous Integration and Delivery (CI/CD).
  The HTML5 application repository is an SAP BTP service that enables central storage of HTML5 applications' static content on the SAP BTP, Cloud Foundry environment. See HTML5 Application Repository.
- An administrator in the ABAP environment provides access to the OData UI service via a business role. A launchpad administrator enables access to the tile via a role collection.
- A business user that is assigned to the business role and role collection can access the tile from SAP Fiori launchpad and launch the application.



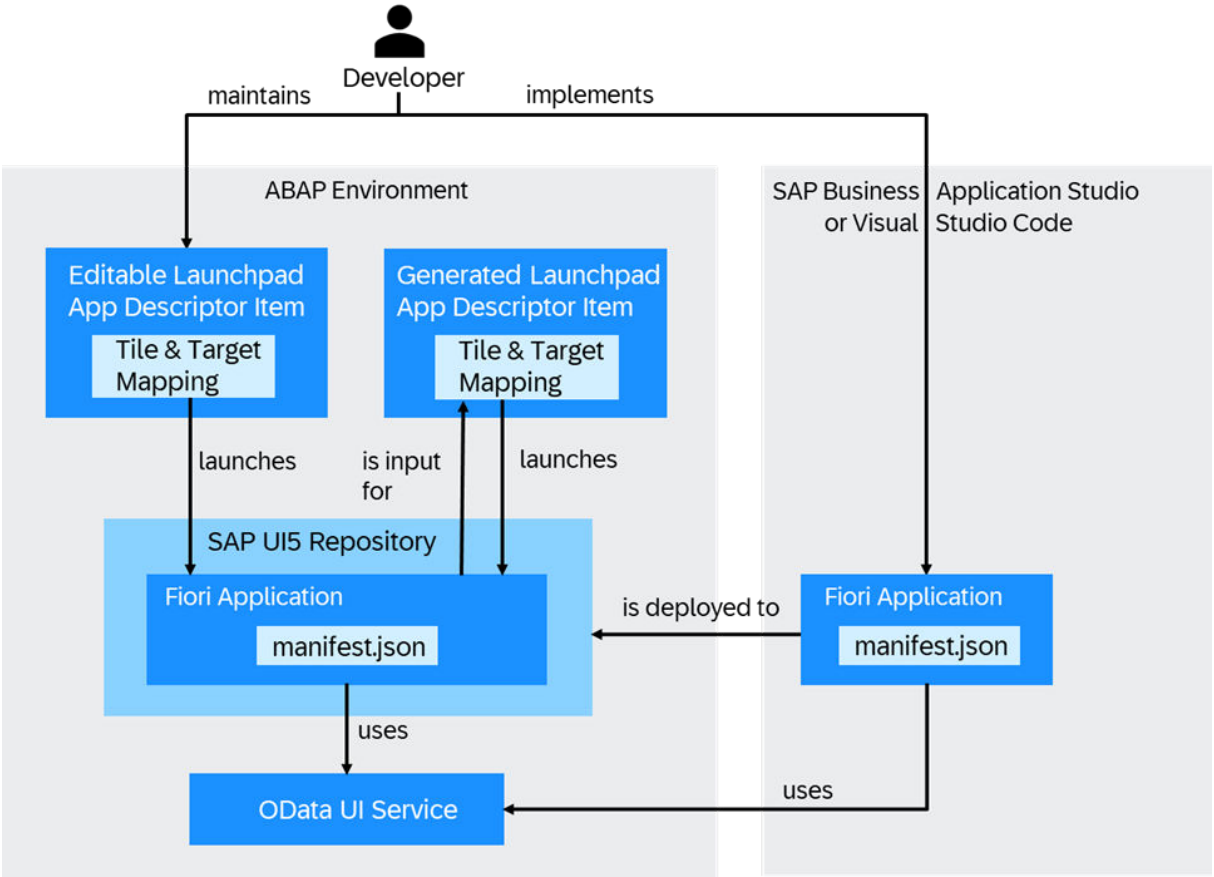
Related Information

SAP Business Application Studio
SAP Fiori Tools
SAP Fiori Overview
Visual Studio Code
Test Automation

4.2.16.1 SAP Fiori Applications in the ABAP Environment

When you plan to create an SAP Fiori application in the ABAP environment and want to provide this application to business users, you may want to get a better understanding of the required steps before getting started with the UI development.

Implementing and Deploying the UI



Implementing an SAP Fiori Application

To implement an SAP Fiori application for an OData UI service, you can use the Fiori tools application generator in SAP Business Application Studio or Visual Studio Code. When generating the application, you can select one of the available OData UI services for UI development and choose the main entity. Amongst others, the generator will create the `manifest.json` file. The manifest defines static information about the application, such as the name of the application or OData UI service used. You can add a SAP Fiori launchpad configuration to the manifest.json file, for example, a tile with an icon. The SAP Fiori launchpad configuration can be added from the application generator or from the application information page.

After generating the app, you can run the application in preview mode.

> ⚠ Restriction
>
> - Only OData UI services from your custom service binding are supported.
> - API services on SAP Business Accelerator Hub are not supported. They are released for external consumption only.

Refer to [Develop an OData UI service against a released BO interface or CDS View] for how to create OData UI services that are suitable for UI consumption in an SAP Fiori application.

To discover the available OData UI services, the `SAP_A4_BC_DEV_UID_PC` business catalog needs to be assigned. In order to run the application in preview mode, the `SAP_CORE_BC_EXT_TST` business catalog has to be assigned. This business catalog is contained in the `SAP_BR_DEVELOPER` business role template.

Deploying an SAP Fiori Application

To deploy the SAP Fiori application, you have to create a deployment configuration and provide the target package and workbench request. You can create the deployment configuration from the application generator or from the application information page.

When deploying the application to the SAPUI5 ABAP repository in the ABAP environment, the following objects are automatically created and recorded on the workbench request.

- A business server page (BSP) application, which stores the code of the SAP Fiori application
- A folder in the MIME repository, which stores all MIME objects, such as graphics or icons of the app
- A node in the Internet Communication Framework (ICF), which enables clients (browser) to access the code stored in the BSP application
- SAP Fiori launchpad app descriptor items which enable launching the app from the Fiori launchpad

You can find the BSP application and the generated launchpad app descriptor items in your deployment package in ABAP development tools for Eclipse. The folder in the MIME repository and the node in the ICF are not visible in ABAP development tools for Eclipse but are listed in the transport request. Alternatively, you can use CDS view `I_CustABAPObjDirectoryEntry` to inspect those objects in the ABAP object directory. The CDS view gives you customer object centric visibility of the classic ABAP table `TADIR`.

The launchpad app descriptor items are created from the SAP Fiori launchpad configuration in the `manifest.json` file. For more information, please navigate to the entry *crossnavigation* in table `sap.app` of Descriptor for Applications, Components, and Libraries (manifest.json). Each app descriptor item consists of a target mapping and a tile. The target mapping maps the deployed app (navigation target) to the intent (combination of semantic object and action). Moreover, the generated Fiori launchpad app descriptor items follow the application deployment lifecycle and cannot be edited in ABAP development tools for Eclipse.

> ⓘ Note
>
> If you change the ID of the SAP Fiori launchpad configuration intent in the manifest.json file of the Fiori application and deploy the latter, existing references from the SAP Fiori launchpad to the corresponding Fiori launchpad app descriptor item will break.

The Fiori UI is deployed in ABAP language version ABAP for Cloud development which restricts access to data sources. Data sources must be released for system-internal use in cloud development or must be in the same software component. For more information, see Use System-Internally (C1).

To deploy the application, the `SAP_A4C_BC_DEV_UID_PC` business catalog needs to be assigned. This business catalog is contained in the `SAP_BR_DEVELOPER` business role template.

Creating Launchpad App Descriptor Items

In addition to the launchpad app descriptor items that are generated automatically from the `manifest.json` file during deployment, you can also create launchpad app descriptor items manually in ABAP development tools for Eclipse. The launchpad app descriptor item consists of a target mapping and one or more tiles. The target mapping maps the application (navigation target) to the intent (combination of semantic object and action).

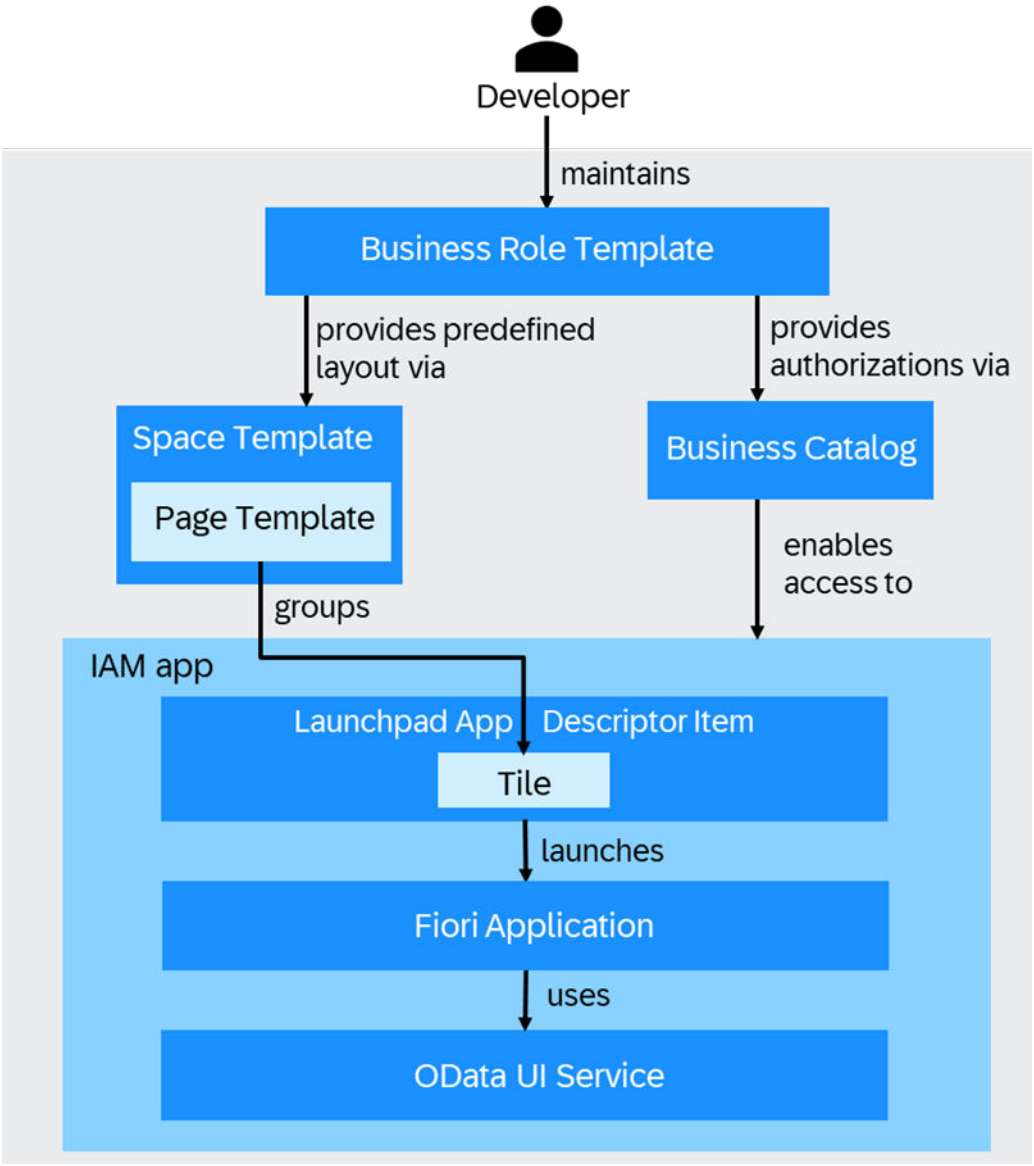
If you want to provide different tiles for the app, for example, a static tile and a dynamic tile with a counter, then you can add additional tiles to the launchpad app descriptor. In this case, users can personalize the launchpad with the tile of their choice. If you are intending to provide different tiles for different personas, then it is recommended to create distinct launchpad app descriptor items. For more information, see Working with Launchpad App Descriptor Items. The properties provided under Editing Launchpad App Descriptor Items can be maintained.

These manually created launchpad app descriptor items can be created with a freely chosen name. Keep in mind that the lifecycle of those manually created objects is decoupled from the Fiori application deployment lifecycle, so they can be edited in ABAP development tools for Eclipse independently.

It is recommended to utilize launchpad app descriptor items created from the `manifest.json` file during app deployment. This read-only app-descriptor item serves as reference and fallback tile for any other tiles that are created in addition to that. You can manually create these items in ABAP development tools for Eclipse if specialized tiles are needed for an already deployed Fiori application. For example, if you wish to create a separate tile that displays a specific subset of data and has a different title, you can manually create an additional app descriptor item with a dedicated tile. This additional app descriptor item can also be used for enabling access and defining the predefined launchpad layout.

Enabling Access and Providing Predefined Launchpad Layout

After you have deployed the SAP Fiori application and created Fiori launchpad app descriptor items, either automatically from the manifest or manually in ABAP development tools for Eclipse, you need to create an IAM app and assign the created SAP Fiori launchpad app descriptor item to the IAM app. Furthermore, you need to create a business catalog and assign the IAM app to the business catalog. The business catalog enables access to the SAP Fiori application.



Additionally, you can provide a predefined launchpad layout by creatinge a space template and a page template referring to the tile of the SAP Fiori launchpad app descriptor item.

Moreover, you can maintain a business role template to have a streamlined process for an administrator to set up business roles by providing predefined roles with predefined layouts.

This saves time and effort for an administrator but still allows for a more tailored user experience.

While the business catalog defines the access to the apps, the space and page template determine which apps are visible for the users on the Fiori Launchpad. The business role template combines the business catalogs and space templates that are typically needed for such a role.

> ⚠ Restriction
>
> Creation of Space Templates and Page Templates
>
> Space templates can't be merged with other space templates directly. To merge a space template with another space template, one of the templates must be copied to create a space first.

When you want to merge one or more customer-created spaces with a predefined space (e.g. delivered by SAP) or a template, open the general details for the customer-created space, then enter the space ID of the predefined space or template in the field *Merge ID*. The same is true for the merging of page templates.

Scoping

Business catalogs, business role templates as well as Fiori space and page templates are not visible in the development system by default and therefore cannot be used.

To use space and page templates in the development system, you need to publish it locally. After the business catalog and role template are published, they are visible in the Business Catalogs and Business Role Templates apps. To use the space and page templates in the development system, you need to programmatically scope them. See Scoping Space and Page Templates [page 1870] for more details about scoping. After the page and pace templates are scoped, they are visible as predefined spaces and pages in the Manage Launchpad Spaces and Manage Launchpad Pages apps. In non-development systems the space and page templates are scoped automatically during import into the system.

Preview

As a developer you can launch the preview of the page template in the Manage Launchpad Pages app from the ABAP development tools for Eclipse editor. To preview the created page template, you need authorization for the Manage Launchpad Pages app.

Translation

Maintain Translations

When it comes to managing translation, there are basically two areas: Texts from the ABAP development objects and texts from the SAP Fiori application, e.g., texts for the IAM app or IAM business catalog. Texts from the SAP Fiori application are usually defined in i18n (internationalization) property files, e.g. the texts for the tile used in the `manifest.json`.

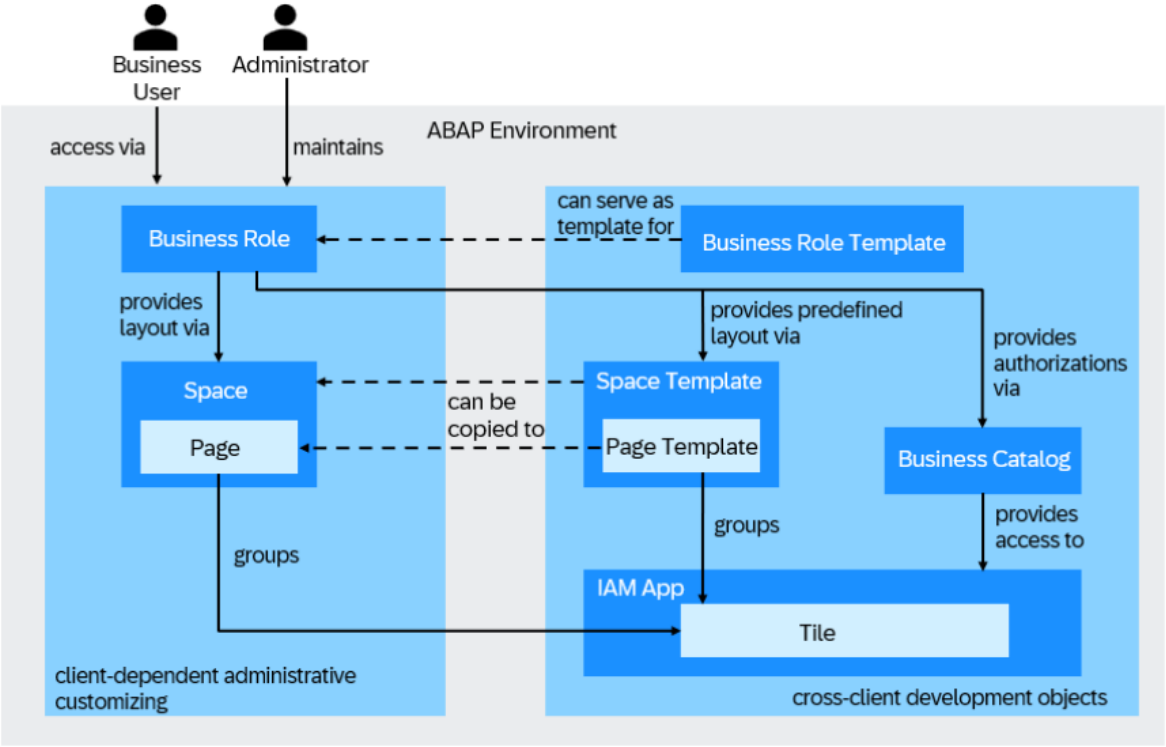
To handle texts from the ABAP development objects, you use the Maintain Translations App. This application provides a user-friendly interface for managing translations within the ABAP environment. It allows you to easily maintain and update translations used in ABAP development objects.

For non-ABAP based texts, such as those found in the SAP Fiori application, a different approach is needed. In this case, you need to follow the Fiori translation process. Please refer to Translate Texts in SAP Fiori for more details.

> **Note**
>
> Texts of launchpad app descriptor items created manually in ABAP development tools for Eclipse can be translated in the *Maintain Translations* app. Texts of launchpad app descriptor items generated from the `manifest.json` file during application deployment origin from i18n property files and follow the translation approach for non-ABAP based texts.

Providing Access and Launchpad Layout

You as an administrator create a business role, so that the application appears in the SAP Fiori launchpad for business users that are assigned to the business role. Using role templates as well as predefined space and page templates simplifies the process for an administrator of setting up business roles.



To make an SAP Fiori application accessible on the SAP Fiori launchpad home page of business users, you must perform the following steps:

- Create a business role and assign at least one business catalog to the role. The business role controls the access to your applications. The business catalog contains the actual authorizations that allow users to access apps.
- You can either assign a predefined space (space template) to the business role, copy an existing one, or create a new launchpad space and page and then add apps to the page. The space and page control the layout of the Fiori launchpad. More information and tips on creating and handling spaces and pages can be found in Best Practices for Managing Spaces and Pages.
- You can also create a business role from a business role template and adapt the business role to your needs.
- Business users that are assigned to the business role can now access the app.
- You can also create spaces, pages, and business roles for testing purposes locally in the development system. If you want to import these artifacts into the production system, you must transport the spaces, pages, and business roles into the production system via a customizing request. See Export Customizing Transports for more details on how to use it.
- If you are using a group-based layout in the SAP Fiori launchpad, you must enable SAP Fiori launchpad spaces in the Manage Launchpad Settings app.

> **Note**
>
> This is a one-time effort.

Related Information

Develop an SAP Fiori Application UI and Deploy it to ABAP Using SAP Business Application Studio [page 1859]

4.2.16.1.1 Develop an OData UI Service Against a Released BO Interface or CDS View

Get an overview about how to create an OData UI service based on a released business object interface or CDS view using ABAP development tools for Eclipse, SAP Business Application Studio or Microsoft Visual Studio Code.

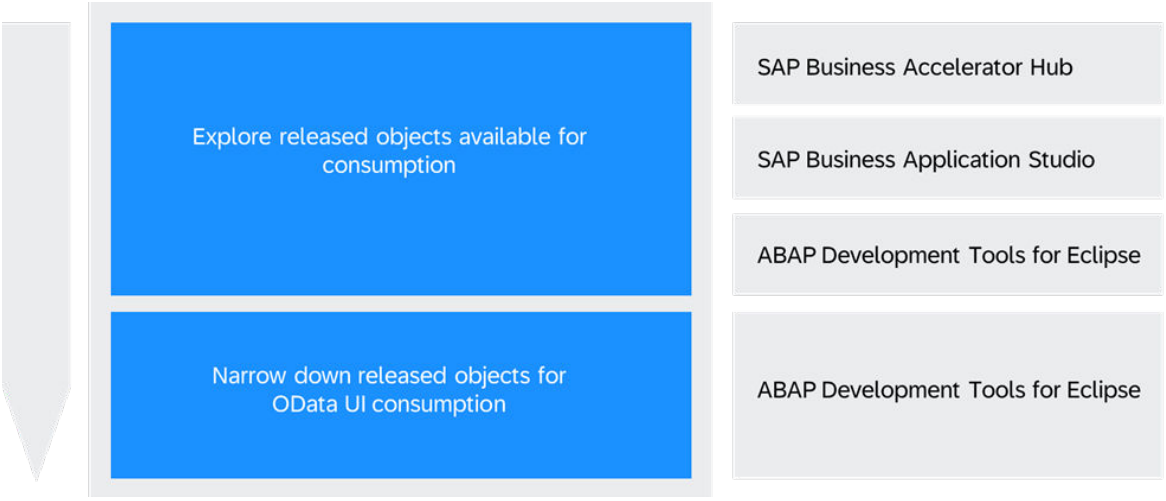
Prerequisites

- If you are using SAP Business Application Studio:
  - You have set up SAP Business Application Studio. See Setup of UI Development in SAP Business Application Studio
  - You have established trust by setting up a custom identity service. See Setup of a Custom Identity Service.
- If you are using Visual Studio Code:
  - You have installed and set up Visual Studio Code including the SAP Fiori tools extensions. See Visual Studio Code
  - To establish a connection with your ABAP environment system, you either have to be a space developer in the ABAP environment instance or have access to a service key in the ABAP environment instance. See Add Space Members Using the Cockpit and Creating Service Keys in the ABAP service instance.
  - You have established trust by setting up a custom Identity service. See Setup of a Custom Identity Service.
- If you are using ABAP development tools for Eclipse:
  - You have installed and set up ABAP development tools for Eclipse. SeeDownload the Eclipse IDE and add the ABAP development Tools (ADT) Plugin .

1. Explore Business Object Interfaces and CDS Views suitable for UI Consumption

> **Note**
>
> **Prerequisites**:
>
> - Business Catalog SAP_A4C_BC_DEV_OBJ_DIS_PC, which allows you to discover released objects. See Business Catalogs for Development Tasks.

Steps



1. As a developer user, explore business object interfaces as well as CDS views that are released with Contract C1: Use System-Internally
   1. Business Object Interfaces and CDS views listed in SAP System Service Provider in the **Service Center** within SAP Business Application Studio. See SAP System Service Provider.
   2. Core Data Services listed under the released objects tree in ABAP development tools for Eclipse, look for behavior definitions and data definitions. See ABAP Repository Trees.

      > **Note**
      >
      > Customer-owned business object interfaces and CDS views are only visible in ABAP development tools for Eclipse as well as in the UI Service Generator. The SAP System Service Provider in Service Center within SAP Business Application only shows SAP-provided objects.

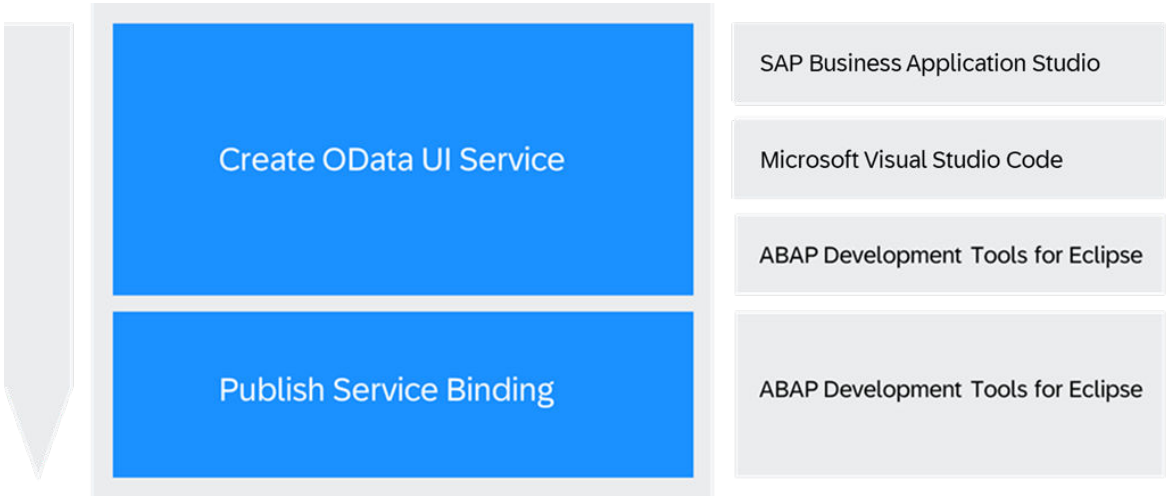
2. On top of finding released objects for consumption, you can further narrow down the selection to only those objects that support OData UI consumption.
   1. In the **Project Explorer** of ABAP development tools for Eclipse, open a released core data service or the root view entity of a released business object interface.
   2. Check the header annotations of the corresponding object. If the annotation and enumeration `@ObjectModel.supportedCapabilities: [#UI_PROVIDER_PROJECTION_SOURCE]` are included, the OData UI consumption is supported.

> **Note**
>
> To open the root view entity for a business object interface, navigate from the released behaviour definition object to the corresponding data definition via the name of the root view entity in the behaviour definition.

2. Creating and publishing an OData UI service based on BO interface or CDS View

> **Note**
>
> **Prerequisites**:
>
> - Business Catalog SAP_A4C_BC_DEV_PC is assigned to your user, which allows you to create and publish OData UI services. See Business Catalogs for Development Tasks.

Steps



1. As developer user, create an OData UI Service based on the business object interface or CDS view suitable for UI consumption:
   1. In SAP Business Application Studio and Microsoft Visual Studio Code you can use the **UI Service Generator** to generate an OData UI Service. See UI Service Generation.

      > **Note**
      >
      > When using the OData UI service generator in SAP Business Application Studio, communication with the ABAP environment is established via destinations (see Creating a Destination to the ABAP System for SAP Business Application Studio or Creating a Destination for Cross-Subaccount Communication). As part of this destination for backend connectivity, you must define an additional property, `HTML5.Timeout`. This value represents the maximum time to wait for a response (in milliseconds) from the destination (refer to Additional Properties of a Destination). Given the complexity and time-consuming nature of creating an OData UI service,

> it is recommended to configure a destination timeout of at least 300000 milliseconds to avoid potential timeouts.

2. In SAP Business Application Studio, utilize the SAP system service provider as part of the Service Center view to explore business objects or CDS views and their properties, then generate an OData UI service from an ABAP business object interface or CDS view. For more details, refer to Explore SAP System Business Objects and ABAP CDS Views. While creating an OData UI service from the Service Center view, the **UI Service Generator** will be opened for the actual generation of the service.

   > ### Note
   >
   > If the underlying business object interface/CDS view does not support UI consumption, a corresponding warning will appear before adding an OData UI service from the SAP system service provider to the project.
   >
   > This method currently only supports the creation based on SAP-provided business object interfaces and CDS views.

3. OData UI services can be created in ABAP development tools for Eclipse either manually by creating an OData UI service binding or by utilizing the **Generate ABAP Repository Objects** wizard. See Creating Service Binding and Generating ABAP Repository Objects.

   > ### Note
   >
   > If the underlying business object interface/CDS view is not suitable for UI consumption, the **Generate ABAP Repository Objects** wizard will display a relevant warning.

2. When using the UI Service Generator in Business Application Studio or Visual Studio Code (directly or opened via the Service Center view), the service binding for the OData UI service is automatically published. This allows you to continue with the SAP Fiori application implementation without additional steps.
   While creating the OData UI service in ABAP development tools for Eclipse, you have to publish the corresponding service binding so it will be available in the service catalog. For more information, see Using Service Binding Editor for OData V4 Service .

Next Step

Develop an SAP Fiori Application UI based on the OData UI Service. See Develop an SAP Fiori Application UI and Deploy it to ABAP Using SAP Business Application Studio and Develop an SAP Fiori Application UI and Deploy it to ABAP Using Visual Studio Code.

Related Information

Generate your own custom UI Service based on a Business Object Interface

Video: Create an SAP Fiori Application Based on an ABAP Business Object (BO) with SAP Build

4.2.16.1.2 Develop an SAP Fiori Application UI and Deploy it to ABAP Using SAP Business Application Studio

Get an overview about how to create and deploy an SAP Fiori application to ABAP using SAP Business Application Studio.

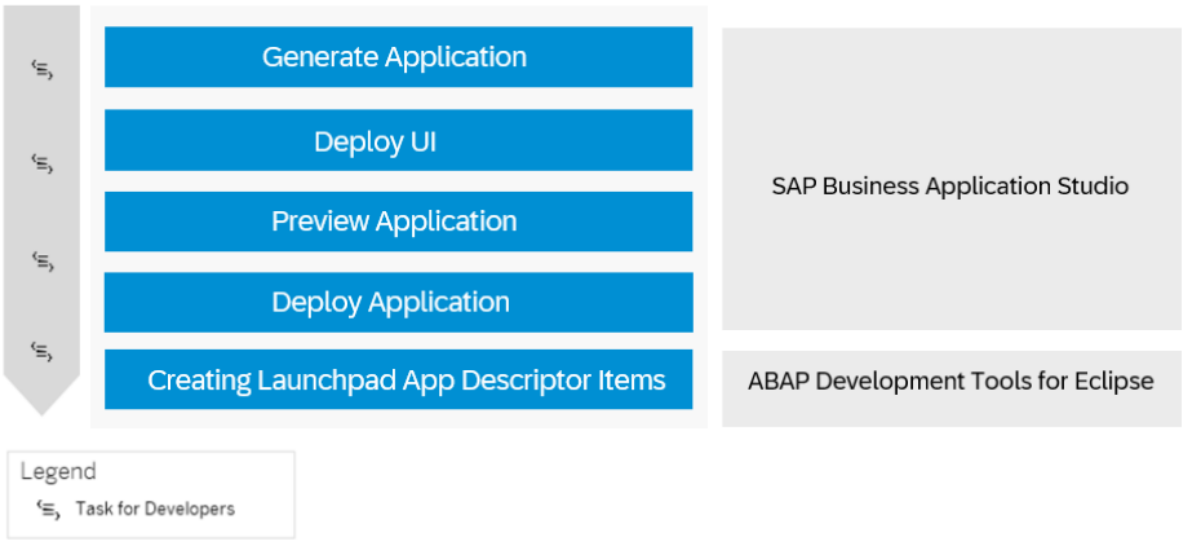
If you need further assistance with creating and deploying an application, check out the tutorial Create an SAP Fiori App and Deploy it to SAP BTP ABAP Environment.

Prerequisites

- You have set up SAP Business Application Studio. See Setup of UI Development in SAP Business Application Studio.
- You have established trust by setting up a custom Identity service. See Setup of a Custom Identity Service.

1. Generating and Deploying Your Application

> **Note**
>
> **Prerequisites**:
>
> - You have created a development package in ABAP Development Tools for Eclipse. See Creating ABAP Packages.
> - You have exposed a RAP business service as an OData service. See Using Service Binding Editor for OData Service. In case no suitable service is available yet, you may create one based on a released BO interface or CDS View. SeeDevelop an OData UI Service Against a Released BO Interface or CS View.
> - Business catalog `SAP_A4C_BC_DEV_UID_PC` is assigned to your user, which allows you to discover available services and deploy your application.
> - Business catalog `SAP_CORE_BC_EXT_TST` is assigned to your user, which allows you to preview your application. See Business Catalogs for Development Tasks.
> - You have an open transport request.



1. As a developer user in SAP Business Application Studio, generate an SAP Fiori application. See Generate an Application.
    1. In the *Data Source and Service Selection* section of the Template Wizard, select the following values:
        - Data source: *Connect to a System*.
        - System:
            - (Option 1) Select the destination that you have created for the SAP Business Application studio integration (`SAP_Business_Application_Studio`). See Creating a Destination to the ABAP System for SAP Business Application Studio.
            - (Option 2) Select *ABAP Environment on SAP Business Technology Platform*.
            From the ABAP environment drop-down menu, choose a service instance.

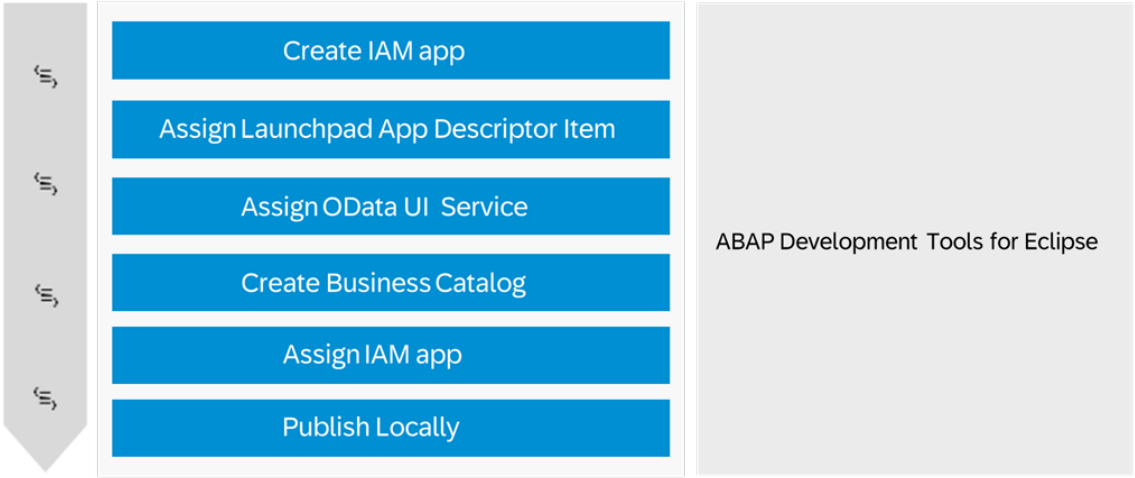
            > ### Note
            > You have to log on to your Cloud Foundry space by executing command `cf login` in the terminal or by navigating to *View* > *Find Command* > *CF: Login to Cloud Foundry*. When you're prompted to enter the API endpoint, org name, and space, you can navigate to your subaccount in the SAP BTP cockpit, where you can find this information.

        - Continue with choosing a service.
    2. Add a deployment configuration. See section *Add deployment configuration* > *ABAP system* in Additional Configuration.
    If you want to create your deployment configuration later, see Generate Deployment Configuration ABAP.
    3. Add an SAP Fiori launchpad configuration for your UI project. See section *Add FLP configuration* in Additional Configuration.
    If you want to create you FLP configuration later, see SAP Fiori Launchpad Configuration.
2. Continue with the development of the UI, for example, with the help of guided development. See Implement Features using Guided Development.
3. Now you can preview the generated SAP Fiori application. See Preview an Application.
4. Deploy the SAP Fiori UI by executing command **`npm run deploy`** in the terminal of your project. See Deployment to ABAP.

5. 4. On top of these launchpad app descriptor items that are created automatically during app deployment, you may also create such objects manually in ABAP development tools for Eclipse. Refer to Creating Launchpad App Descriptor Items.

2. Creating and Publishing Your Identity and Access Management (IAM) App

> **Note**
>
> **Prerequisites**:
>
> Business catalog SAP_A4C_BC_DEV_PC is assigned to your user, which is required for development with ABAP Development Tools. See Business Catalogs for Development Tasks [page 2928].



1. To manage access to your SAP Fiori application, you need to log on as a developer in ABAP development tools for Eclipse to create an Identity and Access Management (IAM) application, assign a launchpad app descriptor item for your UI5 application and a service, and maintain authorizations (steps 1-3 in the figure above). See Creating an IAM App for the Business Service.
2. Once you have created your IAM app, you have to create a business catalog. See Creating a Business Catalog.
3. Assign your IAM app to the business catalog.
4. Publish the IAM app and business catalog locally.

Next Step

Launch your app in SAP Fiori launchpad. See Add Your App to SAP Fiori Launchpad.

Related Information

SAP Business Application Studio
SAP Fiori Tools
SAP Fiori Overview
Tutorial: Develop and Run SAP Fiori Application With SAP Business Application Studio
https://developers.sap.com/tutorials/abap-environment-abap-flp.html
Tutorial: Integrate List Report into ABAP Fiori Launchpad

4.2.16.1.3 Develop an SAP Fiori Application UI and Deploy it to ABAP Using Visual Studio Code

Get an overview about how to create and deploy an SAP Fiori application to ABAP using Visual Studio Code.

For further assistance with creating and deploying an application into SAP Fiori launchpad, see Create a SAP Fiori App in Visual Studio Code and Deploy it to SAP BTP ABAP Environment.

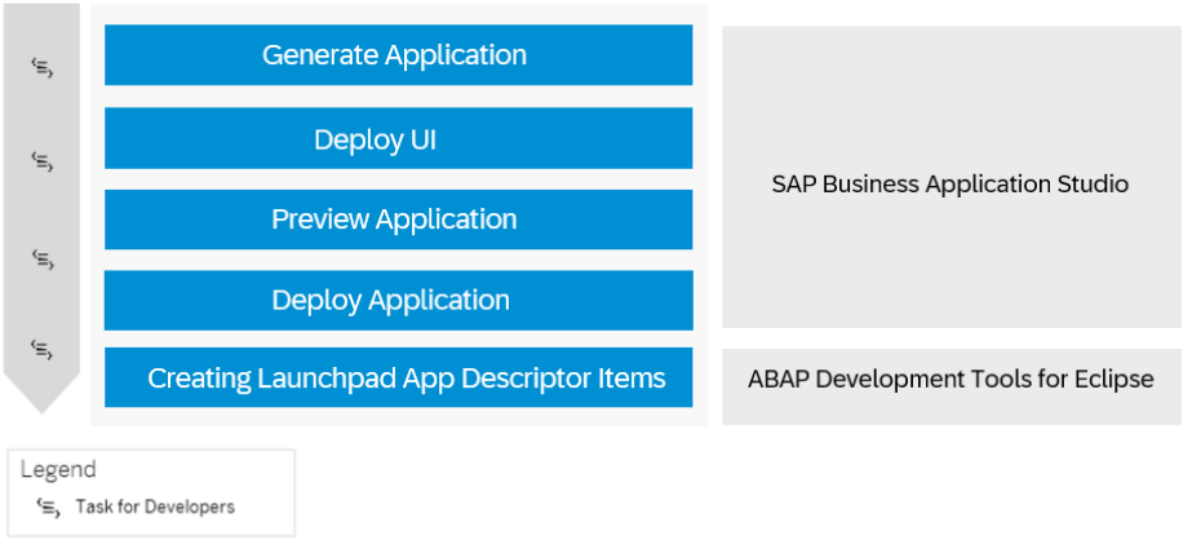
Prerequisites

- You have installed and set up Visual Studio Code including the SAP Fiori tools extensions. See Visual Studio Code.
- You have access to a RAP business service that has been exposed as an OData UI service. See Using Service Binding Editor for OData V2 Service.
- To establish a connection with your ABAP environment system, you either have to be a space developer in the ABAP environment instance or have access to a service key in the ABAP environment instance. See Add Space Members Using the Cockpit and Creating Service Keys in the ABAP service instance.
- You have established trust by setting up a custom Identity service. See Setup of a Custom Identity Service.

1. Generating and Deploying Your Application

> **Note**
>
> **Prerequisites**
>
> - You have created a development package in ABAP development tools for Eclipse. See Creating ABAP Packages.
> - You have exposed a RAP business service as an OData UI service. See Using Service Binding Editor for OData V2 Service.
> - Business catalog `SAP_A4C_BC_DEV_UID_PC` is assigned to your user, which allows you to discover available OData UI services and deploy your application.

- Business catalog `SAP_CORE_BC_EXT_TST` is assigned to your user, which allows you to preview your application. See Business Catalogs for Development Tasks [page 2928].
- You have an open transport request.



1. As a developer user in Visual Studio Code, generate an SAP Fiori application. See Generate an Application.
   In the *Data Source and Service Selection* section of the Template Wizard, select the following values:
   - Data source: *Connect to a System*.
   - System: *New System*
   - System type: *ABAP Environment on SAP Business Technology Platform*
   - ABAP environment definition source:
     - (Option 1) **Discover a Cloud Foundry Service**

       > **Note**
       >
       > You have to log on to your Cloud Foundry space by executing command `cf login` in the terminal or by navigating to *View* > *Find Command* > *CF: Login to Cloud Foundry*. When you're prompted to enter the API endpoint, org name, and space, you can navigate to your subaccount in the SAP BTP cockpit, where you can find this information.

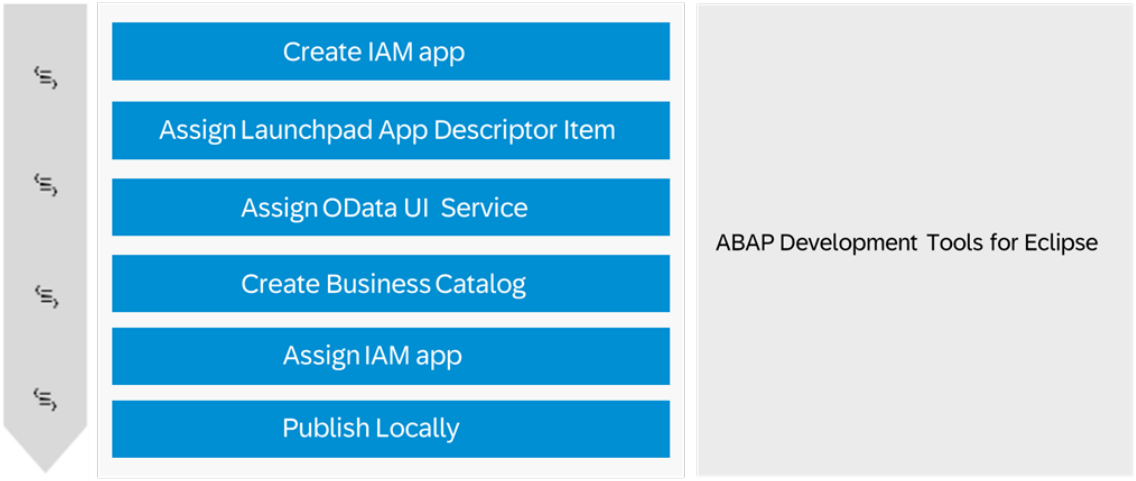
     - (Option 2) **Upload a Service Key File**
   - Continue with choosing a system name and service.

   In the *Project Attributes* section, add deployment configuration and FLP configuration to your UI project. See Additional Configuration.
   If you want to create your deployment configuration later, see Generate Deployment Configuration ABAP.
   If you want to create you FLP configuration later, see SAP Fiori Launchpad Configuration.
2. Continue with the development of the UI, for example, with the help of guided development. See Implement Features using Guided Development.
3. Now you can preview the generated SAP Fiori application. See Preview an Application.
4. Deploy the SAP Fiori UI by executing command `npm run deploy` in the terminal of your project. See Deployment to ABAP.

5. On top of these launchpad app descriptor items that are created automatically during app deployment, you may also create such objects manually in ABAP development tools for Eclipse. Refer to Creating Launchpad App Descriptor Items.

2. Creating and Publishing Your Identity and Access Management (IAM) App

> **Note**
>
> **Prerequisites**: Business catalog SAP_A4C_BC_DEV_PC is assigned to your user, which is required for development with ABAP development tools for Eclipse. See Business Catalogs for Development Tasks [page 2928].



1. To manage access to your SAP Fiori application, you have to log on as a developer in ABAP development tools for Eclipse to create an Identity and Access Management (IAM) application, assign a launchpad app descriptor item for your UI5 application and a service, and maintain authorizations (steps 1-3 in the figure above). See Creating an IAM App for the Business Service.
2. Once you have created your IAM app, you have to create a business catalog. See Creating a Business Catalog.
3. Assign your IAM app to the business catalog.
4. Publish the IAM app and business catalog locally.

Next Step

Launch your app in SAP Fiori launchpad. See Add Your App to SAP Fiori Launchpad [page 1869].

Related Information

Visual Studio Code

SAP Fiori Tools
SAP Fiori Overview
Tutorial: Integrate List Report into ABAP Fiori Launchpad
Tutorial: Create an SAP Fiori App in Visual Studio Code and Deploy it to SAP BTP ABAP Environment

4.2.16.1.4 Extend an SAP Fiori App and Deploy it to ABAP Using SAP BAS

Learn how to extend an SAP Fiori application with an application variant by creating an SAPUI5 adaptation project and deploying it to ABAP using SAP Business Application Studio.

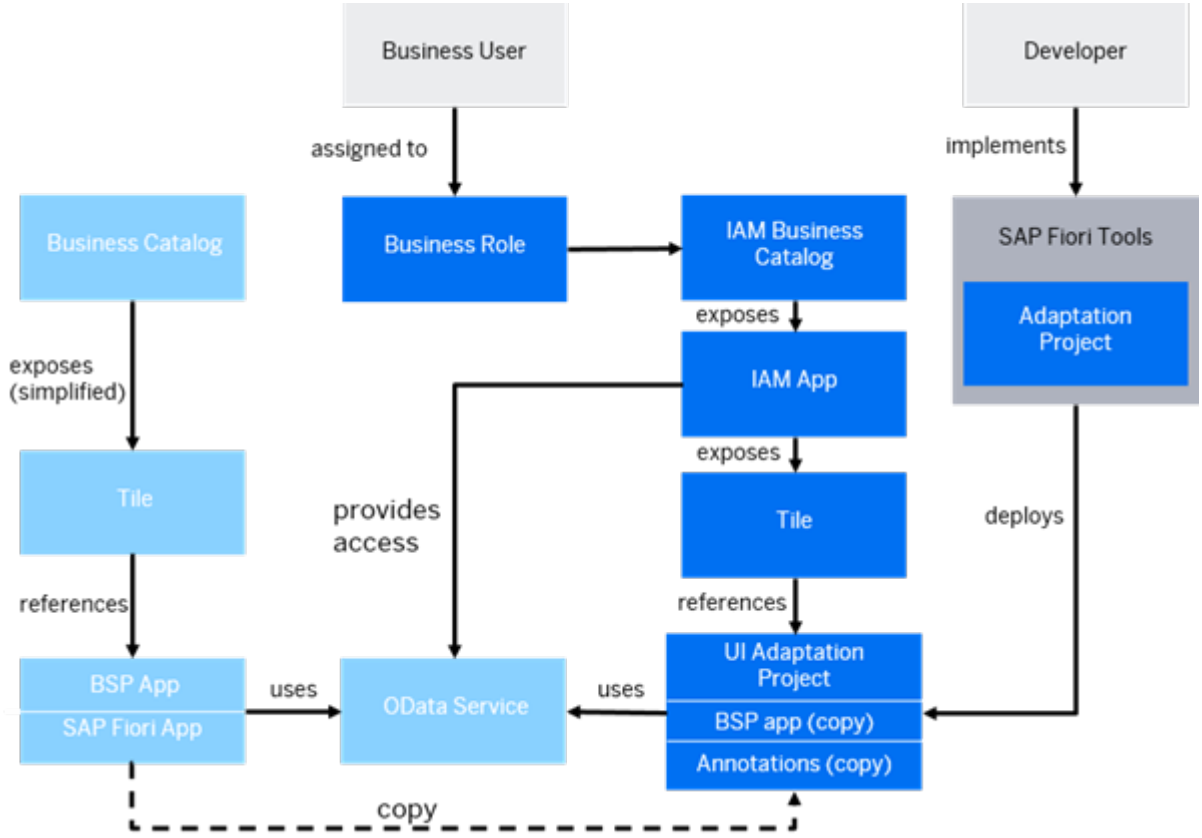
Overview

If an SAP Fiori application doesn't fully support your business process or the specific needs of some of your business users, and if the SAP Fiori application is released for extensibility, you can extend the SAP Fiori application with an applicaion variant while the original SAP Fiori application remains available and unchanged. In an application variant, you can extend the original code to define changes that fulfil the specific business requirements of different business roles, user groups, organizational units, and countries. For example, you might want to create an application variant to simplify and streamline a specific process by hiding fields not required by casual users, visualizing data as a chart, or changing a responsive table to a grid table to make information easier to process.

As a developer, you can use SAPUI5 adaptaion projects in SAP Business Application Studio to create applications variants.You can efficiently reuse the original SAP Fiori application together with its OData UI service and application logic to define changes that are specific to the application variant only.

After deployment, both the base application and the newly created application variant exist and can be made available to the relevant business users through corresponding tiles in the SAP Fiori launchpad. If the base application is changed by a new version, it does not affect the deployed application variant. You can test the application variant with the latest enhancements of the base application in SAP Business Application Studio and deploy it again to benefit from the latest enhancements. For further information, see Check Whether Your Adaptaion Project Is Up-To-Date with Base App Upgrades.

The following diagram shows the relationship between the source application (shown in blue) and the application variant created by an adaptation project (shown in dark blue).



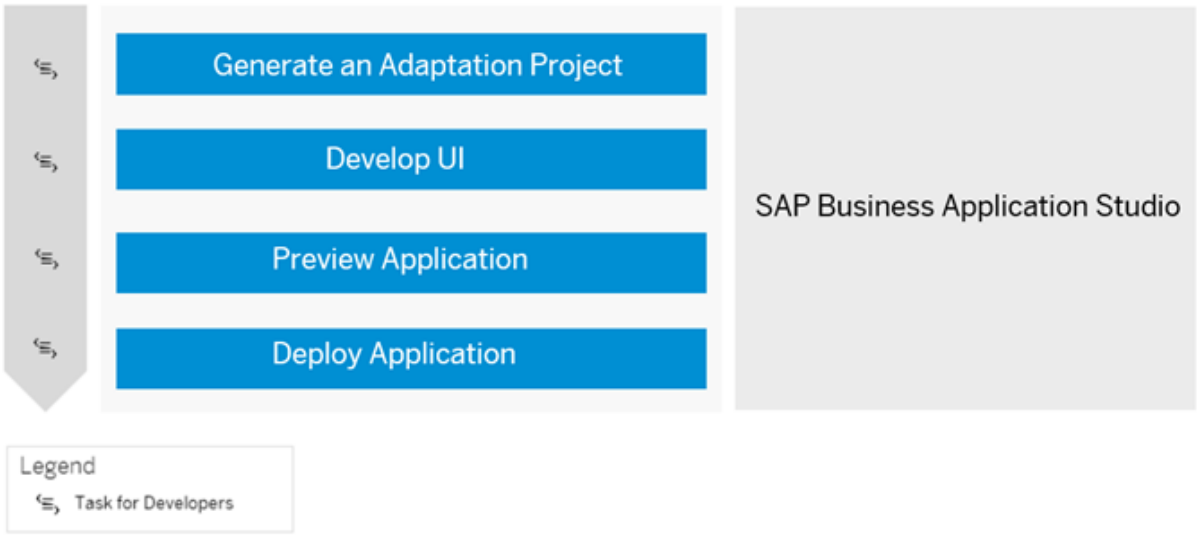
Prerequisites

You have set up and integrated SAP Business Application Studio. See Setup of UI Development in SAP Business Application Studio.

Generating and Deploying Your Adaptation Project

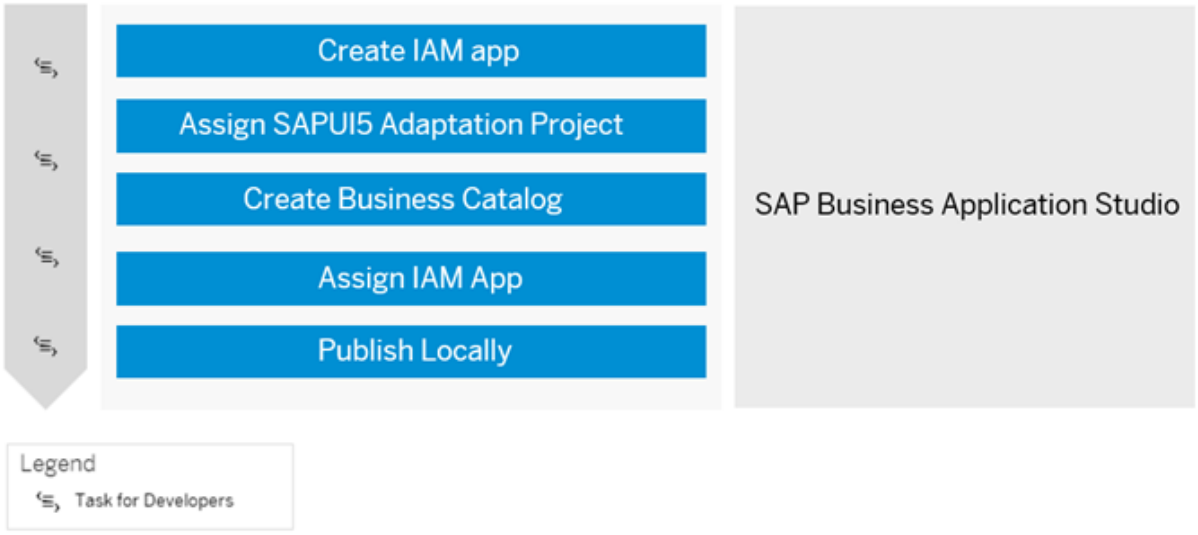
Prerequisites

- The following business catalogs are assigned to your user:
  - Business catalog `SAP_A4C_BC_DEV_OBJ_DIS_PC` (or other business catalog for development tasks), which allows you to retrieve the list of extensible base applications.
  - A business catalog that grants access to the base application, which allows you to execute the corresponding OData UI service.
  - Business catalog `SAP_A4C_BC_DEV_UID_PC`, which allows you to deploy the adaptation project.
- You have an open transport request.



1. As a developer user in SAP Business Application Studio, generate an SAPUI5 adaptation project. See Create the Adaptation Project.
2. Make adaptations to the UI. For example, change control properties, add own sections to the object page, or add controller extensions. See Make Adaptations.
3. Now, you can preview the generated SAP Fiori application. See Preview the Adaptation Project.
4. Deploy the SAP Fiori UI by running command `npm run deploy` in the terminal of your project. See Deploy or Update the Adaptation Project to the ABAP Repository .

Creating and Publishing Your Identity and Access Management (IAM) App



1. To manage access to your SAP Fiori application, you need to log on to the development tenant in ABAP development tools for Eclipse as a developer to create an Identity and Access Management (IAM) application. Moreover, you need to assign an SAPUI5 adaptation project and maintain authorizations, as shown in steps one and two in the figure above. See Creating an IAM App for the Business Service (Developer).

1. Open and log onto ABAP development tools for Eclipse (ADT).
2. In the Project Explorer, open your package and expand the BSP Library folder and then the BSP Applications folder to view the application variant you deployed in SAP Business Application Studio.
3. In the same package, you will find the folder Fiori User Interface containing the FLP App Descriptor Items folder. An FLP app descriptor item with the name `<BSP application name>_UI5R` was generated automatically when you deployed the application variant in SAP Business Application Studio.
4. Right-click on your package and choose New –> Other ABAP Repository Object
5. In the ABAP Repository Object dialog box, open theCloud Identity and Access Management folder and select IAM App.
6. Choose *Next*.
7. In the New IAM App dialog box, enter a unique name and description for your IAM app.
8. In the Application Type field, select UI5A – UI Adaptation App. The application ID suffix _UI5A is set automatically.
9. Choose *Next*
10. In the *Select Transport Request* dialog box, choose the transport request you want to use and choose *Finish*.
11. In your package, expand the folder *Cloud Identity and Access Management –> IAM Apps* and you will see your newly created IAM app with the application name you specified in step g) and the suffix UI5A.
12. Open your newly created IAM App in the *IAM Apps* folder. In the *Overview* tab, enter the *Fiori Launchpad App Description Item ID* that was generated automatically in your *Fiori User Interface folder –> FLP Descriptor Items* called `<BSP application name>_UI5R`.
13. Open the *Services* tab, which will now list the services used by your application variant. Choose *Synchronize*.
14. Open the *Authorizations* tab and maintain authorizations for your application variant and choose *Save*.
15. You need to publish your IAM app locally, to do this choose the *Publish Locally* button on the top right of the screen.

2. Once you have created your IAM app, you have to create a business catalog. See Creating a Business Catalog (Developer).
3. Assign your IAM app to the catalog.
4. Publish the IAM app and business catalog locally.

Next Steps

Add Your App to SAP Fiori Launchpad

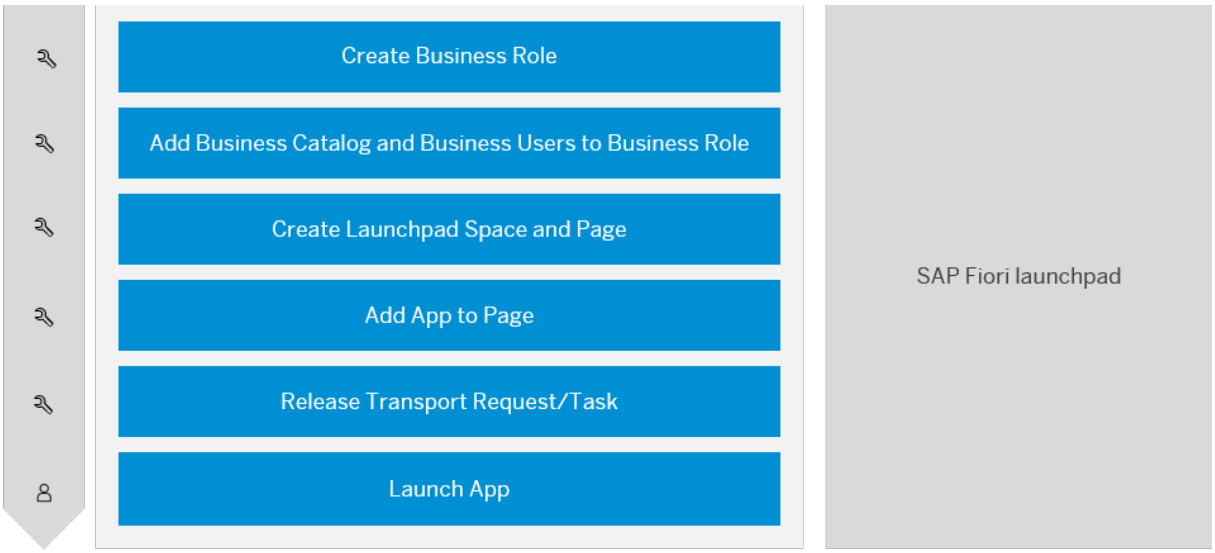
Related Information

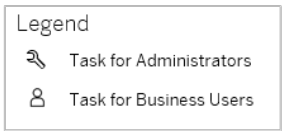
Extending an SAP Fiori Application for S/4HANA Cloud public Edition and SAP BTP ABAP Environment
SAP Business Application Studio
SAP Fiori tools
SAP Fiori Overview

4.2.16.1.5 Add Your App to SAP Fiori Launchpad

Get an overview on how to add your app to SAP Fiori launchpad.

If you need further assistance with the individual steps, check out the developer tutorial Integrate List Report into ABAP Fiori Launchpad.





1. In the SAP Fiori launchpad of your ABAP system, create a business role with the *Maintain Business Roles* app. See Maintain Business Roles.
2. Assign a business catalog and business user to the business role. See Maintain Business Users.
3. Create a launchpad space and page. See Step by Step: Create a New Space and Page for a Business Role.

> **Note**
>
> To use this app, your user must be assigned to business catalog `SAP_CORE_BC_UI_FLD`.

4. Enable spaces for your user. See Enabling Spaces. Optionally, you can personalize and adapt the user interface of your app for all users. See Personalizing and Adapting Apps.
5. As a business user, you can launch the app.

Next Step

Transport Your App

Related Information

Transporting Changes to Spaces and Pages

4.2.16.1.6 Scoping Space and Page Templates

The scoping of space and page templates refers to the process of defining the visibility or accessibility of these objects.

Space and page templates are not visible in the Manage Launchpad Spaces and Manage Launchpad Pages apps in the development system by default. Meaning, they cannot be assigned or used within a business role. The space and page templates need to be scoped.

To use the space and page templates in the development system you must implement your own scoping using the scoping API (Application Programming Interface) `CL APS BC SCOPE CHANGE API`. It's recommended to use a console application to set the space and page template in scope.

> **Note**
>
> Please be aware that the API in the following example is not available for SAP trial accounts.

Sample Code

```
    CONSTANTS obj_name_page  TYPE c LENGTH 40 VALUE 'PAGE_NAME' ##NO_TEXT.
    CONSTANTS obj_name_space TYPE c LENGTH 40 VALUE 'SPACE_NAME' ##NO_TEXT.
    DATA(lo_scope_api) = cl_aps_bc_scope_change_api=>create_instance( ).
    lo_scope_api->scope(
      EXPORTING it_object_scope  = VALUE #(
          pgmid       = if_aps_bc_scope_change_api=>gc_tadir_pgmid-R3TR
          scope_state = if_aps_bc_scope_change_api=>gc_scope_state-ON
          ( object = if_aps_bc_scope_change_api=>gc_tadir_object-UIST
obj_name = obj_name_space )
          ( object = if_aps_bc_scope_change_api=>gc_tadir_object-UIPG
obj_name = obj_name_page ) )
                iv_simulate      = abap_false
                iv_force         = abap_false
      IMPORTING et_object_result = DATA(lt_results)
                et_message       = DATA(lt_messages) ).
    LOOP AT lt_results INTO DATA(ls_result).
       ...
    ENDLOOP.
```

In non-development systems, space and page templates are scoped automatically during import into the system.

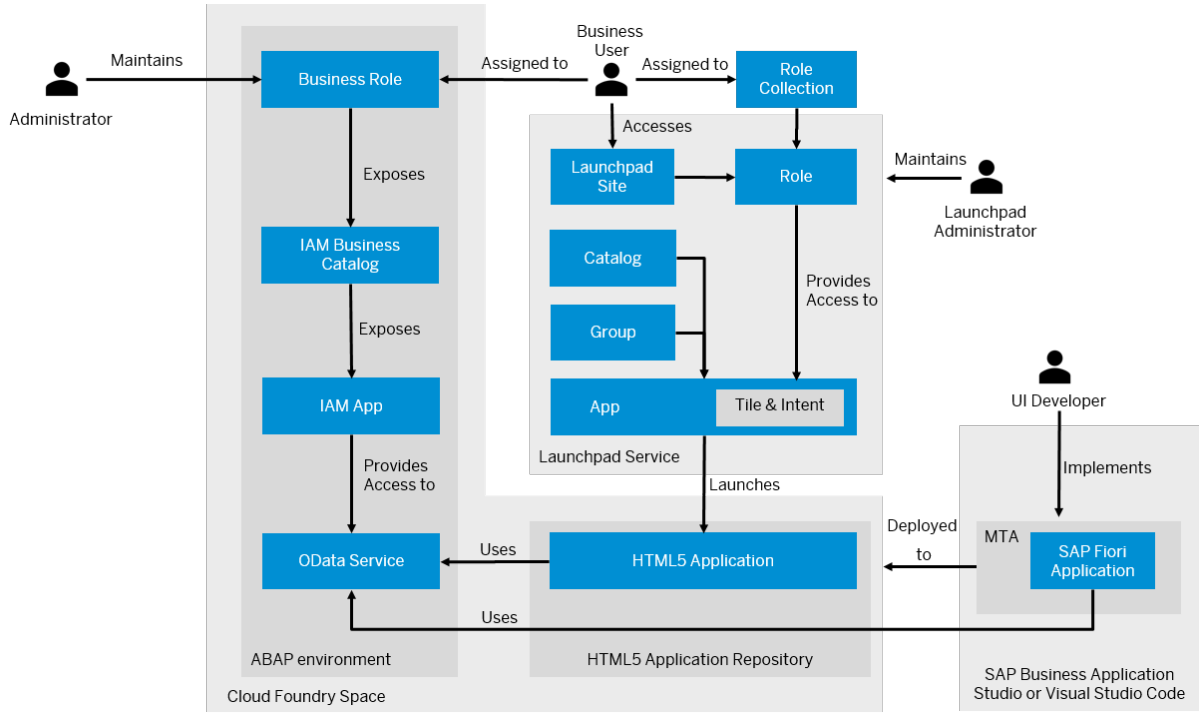
Related Information

Scoping Framework

4.2.16.2 SAP Fiori Applications in the Cloud Foundry Environment

When you plan to create an SAP Fiori application that you have deployed to the SAP BTP, Cloud Foundry environment and want to provide this application to business users, you may want to get a better understanding of the required steps before getting started with the UI development.

> → Recommendation
>
> We recommend using the same UI5 version that is being used by the ABAP environment system.
>
> To check the UI5 version in the SAP Fiori launchpad of your ABAP environment system, use command **`Alt+Ctrl+Shift+P`** or navigate to the user actions menu and select *About*. See User Actions Menu.



Implementing and Deploying the UI

To develop an SAP Fiori application, you can create a multitarget application using the Fiori Tools application router generator, where we recommended using the SAP-managed approuter. See Application Router and Developing HTML5 Applications and Extensions.

Afterwards, you can start creating your SAP Fiori application using the Fiori Tools application generator and add it to the multitarget application (MTA). When generating the application, you can browse the available OData services for UI development. The application generation process also includes adding information for the SAP Fiori app tile in the SAP Fiori launchpad, such as a title, action, and semantic object. This information has to be defined in the project attributes of your MTA by adding an FLP configuration. Your configuration is then stored in the `manifest.json` file. Note that only one intent navigation consisting of a semantic object

and action is supported. If you change the ID of the intent, existing references from the SAP Fiori launchpad to the tile break.

You can integrate the OData service of your ABAP environment into the application by specifying a corresponding route in the `xs-app.json` file. The routing can be done either by providing a destination that points to the ABAP environment instance or by specifying the service endpoint of the ABAP environment and binding the service instance. See Configure Destinations and Binding Service Instances to Applications. The Fiori application generator adds a route to the `xs-app.json` file using the destination that you specify in the generator.

> **Note**
>
> If you have used the Fiori Tools application generator to create your app, you have selected a destination that points to your development subaccount. Make sure that the destination in your target subaccount has the same name as the one maintained in the `xs-app.json` file.

The next step is to build the MTA project. This generates an `*.mtar` file and `mta_archives` folder.

Before deploying your UI, you should preview your application. To preview your application or discover available OData services, business catalog `SAP_CORE_BC_EXT_TST` has to be assigned to your user, which allows you to preview your application and . See Business Catalogs for Development Tasks [page 2928].

The final step is to create a deployment configuration and deploy the MTA to your SAP BTP, Cloud Foundry environment space, which requires the space developer role. By doing so, the SAP Fiori application that you have added to the MTA gets deployed to the HTML5 application repository.

Providing Access to Business Users

After you have deployed the SAP Fiori application, you have to create an IAM app to make your application consumable for business users. This requires assigning the business service to the IAM app. Additionally, you have to use SAP Build Work Zone, standard edition and log on as an administrator to create a role.
A corresponding role collection is automatically created in the SAP BTP cockpit. This allows you to assign business users to the role collection in the SAP BTP cockpit. Being assigned to the role collection, business users can access the launchpad site and the app.

Integrating the SAP Fiori App into SAP Fiori launchpad

To make an SAP Fiori application accessible on the SAP Fiori launchpad home page of business users, you have to assign your app to a group and catalog. See Assign Apps to a Group and to a Catalog. Afterwards, you need to add content to the HTML5 apps content provider in SAP Build Work Zone, standard edition. The HTML5 apps provider reflects all the HTML5 apps that are deployed to the HTML5 app repository that is assigned to the subaccount. See HTML5 Apps Content Provider and Integrate Apps and Shell Plugins.

Related Information

Develop an SAP Fiori Application and Deploy it to Cloud Foundry Using SAP Business Application Studio [page 1873]
Develop an SAP Fiori Application and Deploy it to Cloud Foundry Using Visual Studio Code [page 1875]

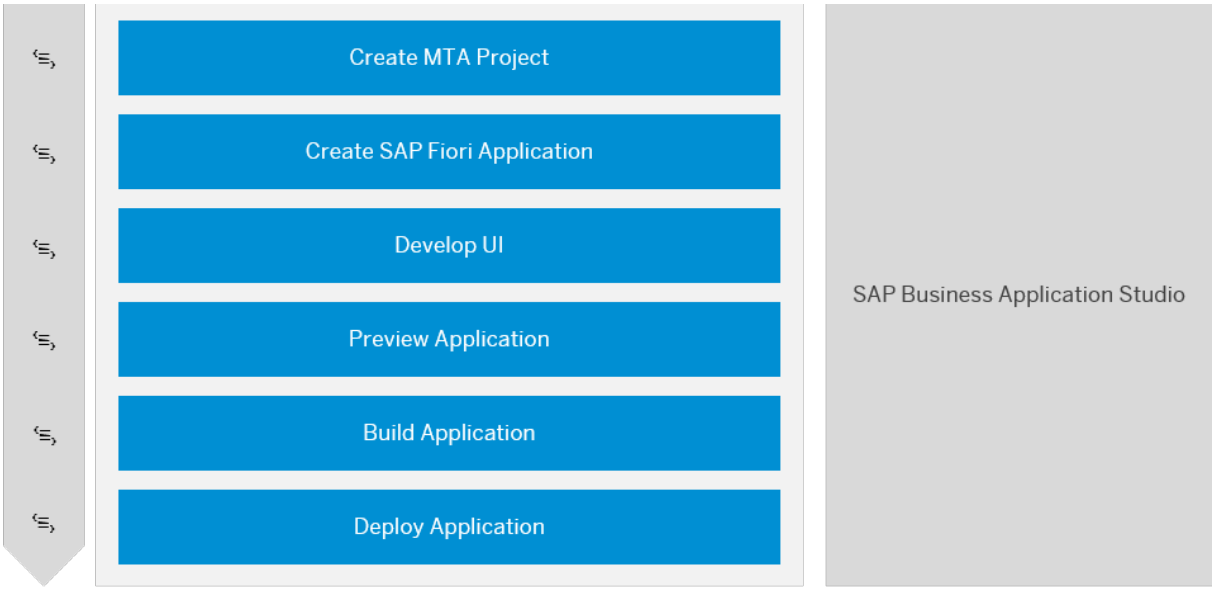
4.2.16.2.1 Develop an SAP Fiori Application and Deploy it to Cloud Foundry Using SAP Business Application Studio

Get an overview about how to create and deploy an SAP Fiori application to Cloud Foundry using SAP Business Application Studio.

Prerequisites

- A subaccount administrator has provided access to SAP Business Application Studio. See Setup of UI Development in SAP Business Application Studio (Optional) [page 218].
- A subaccount administrator has created a destination to the ABAP system to integrate SAP Business Application Studio. See Creating a Destination to the ABAP System for SAP Business Application Studio [page 222].
- You have created a dev space of type SAP Fiori. See SAP Business Application Studio.
- You have access to a RAP business service that has been exposed as an OData service. See Using Service Binding Editor for OData V2 Service.

Generating and Deploying Your Application



1. Create an MTA project with approuter configuration.
   To do so, use the *Fiori CF Application Router Generator*. See Generate an MTA Deployment File.
   This generates an MTA project and the corresponding application router configuration.
2. With the *SAP Fiori Generator*, create an SAP Fiori application in the subfolder of the MTA file location by selecting *Fiori: Open Application Generator* from the command palette. See Generate an Application and Add a Fiori Application to an MTA Deployment File with the Fiori Generator.
   - As a data source, select *Connect to a System*.
   - As a system, choose the destination to the ABAP system that has been created to integrate SAP Business Application Studio.
   - As a service, select your RAP business service that has been exposed as an OData service

   In the *Project Attributes* section, add an SAP Fiori launchpad configuration for your UI project. See section *Add FLP configuration* in Additional Configuration.
   If you want to create you FLP configuration later, see SAP Fiori Launchpad Configuration.
3. Continue with the development of the UI, for example, with the help of guided development. See Implement Features using Guided Development.
4. Now you can preview the generated SAP Fiori application. See Preview an Application.
5. Build the application by executing command **`npm run build`** in the terminal of your project. See Deployment to Cloud Foundry.
   This generates an `*.mtar` file and `mta_archives` folder.
6. Log on to Cloud Foundry and deploy the SAP Fiori UI to your development space by executing command **`npm run deploy`**. See Deployment to Cloud Foundry.

→ Tip

If you can't discover an available OData service in the generator, or preview your application, check if you user is assigned to business catalog `SAP_CORE_BC_EXT_TST`. If you aren't authorized to deploy the application, check if business catalog `SAP_A4C_BC_DEV_UID_PC` is assigned to your user.

Related Information

SAP Business Application Studio
SAP Fiori Tools
SAP Fiori Overview
Tutorial: Create an SAP Fiori App and Deploy it to SAP BTP, Cloud Foundry environment

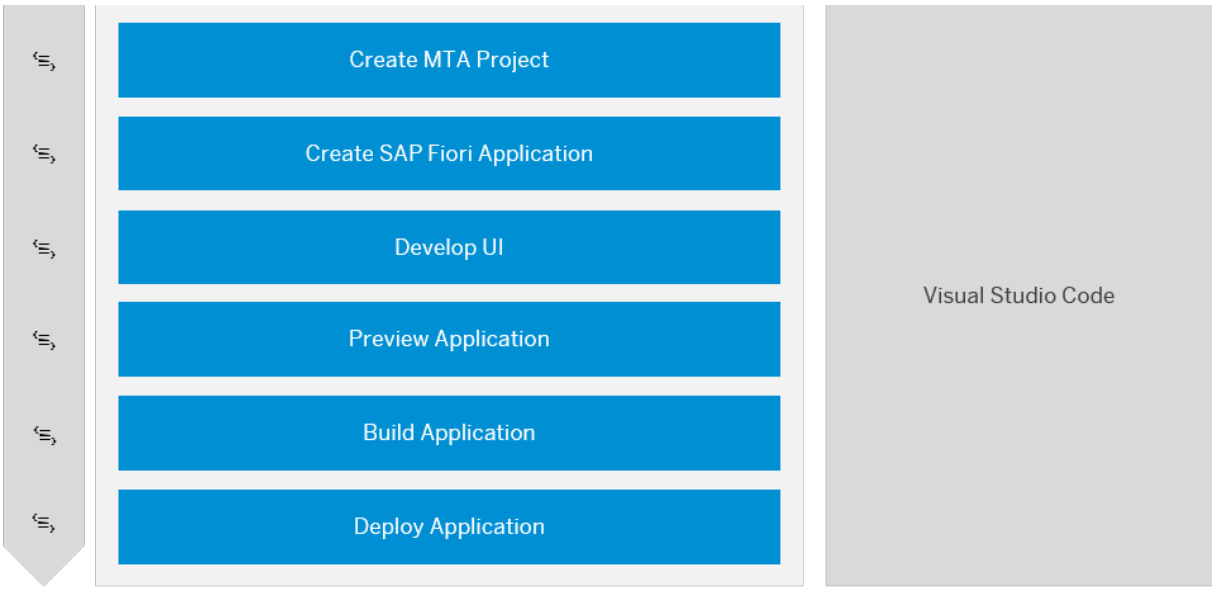
4.2.16.2.2 Develop an SAP Fiori Application and Deploy it to Cloud Foundry Using Visual Studio Code

Get an overview about how to create and deploy an SAP Fiori application to Cloud Foundry using Visual Studio Code (VS Code).

Prerequisites

- You have installed and set up Visual Studio Code including the SAP Fiori tools extensions. See Visual Studio Code.
- You have access to a RAP business service that has been exposed as an OData service. See Using Service Binding Editor for OData V2 Service.
- To establish a connection with your ABAP environment system, you either have to be a space developer in the ABAP environment instance or have access to a service key in the ABAP environment instance. See Add Space Members Using the Cockpit and Creating Service Keys in the ABAP service instance.

Generating and Deploying Your Application



1. Create an MTA project with approuter configuration.
   To do so, use the *Fiori CF Application Router Generator*. See Generate an MTA Deployment File.
   This generates an MTA project and the corresponding application router configuration.
2. With the *Fiori Application Generator*, create an SAP Fiori application in the subfolder of the MTA file location by selecting *Fiori: Open Application Generator* from the command palette. See Generate an Application and Add a Fiori Application to an MTA Deployment File with the Fiori Generator.
   In the *Data Source and Service Selection* section of the Template Wizard, select the following values:
   - Data source: *Connect to a System*.
   - System: *New System*
   - System type: *ABAP Environment on SAP Business Technology Platform*
   - ABAP environment definition source:
     - (Option 1) **Discover a Cloud Foundry Service**

       > **Note**
       >
       > You have to log on to your Cloud Foundry space by executing command `cf login`. When you're prompted to enter the API endpoint and org name, you can navigate to your subaccount in the SAP BTP cockpit, where you can find this information.

     - (Option 2) **Upload a Service Key File**
   - Continue with choosing a system name and service.

   In the *Project Attributes section*, add an SAP Fiori launchpad configuration for your UI project. See Add FLP Configuration.
   If you want to create you FLP configuration later, see SAP Fiori Launchpad Configuration.
3. Continue with the development of the UI, for example, with the help of guided development. See Implement Features using Guided Development.

4. Now you can preview the generated SAP Fiori application. See Preview an Application.
5. Build the application by executing command **`npm run build`** in the terminal of your project. See Deployment to Cloud Foundry.
   This generates an `*.mtar` file and `mta_archives` folder.
6. **`npm run deploy`**. See Deployment to Cloud Foundry.

> → Tip
>
> If you can't discover an available OData service in the generator, or preview your application, check if you user is assigned to business catalog Log on to Cloud Foundry and deploy the SAP Fiori UI to your development space by executing command `SAP_CORE_BC_EXT_TST`. If you aren't authorized to deploy the application, check if business catalog `SAP_A4C_BC_DEV_UID_PC` is assigned to your user.

Related Information

SAP Fiori Tools
SAP Fiori Overview

4.2.16.2.3 Integration with Approuter

You can integrate the OData service of your ABAP environment into the application by specifying a corresponding route in the `xs-app.json` file of the HTML5 module. The routing can be done either by providing a destination that points to the ABAP environment instance or by specifying the service endpoint of the ABAP environment and binding the service instance.

> ⓘ Note
>
> If the application and the instance of the ABAP environment are deployed to different subaccounts, use routing via destination.

For more information about the two types of routing, navigate to Routing via Service [page 1877] or Routing via Destination [page 1880].

4.2.16.2.3.1 Routing via Service

Maintain Route

In order to access the service instance of the ABAP environment directly, maintain a route with service `com.sap.cloud.abap` and endpoint `abap` for the path of the OData service in the `xs-app.json` file.

Sample Code

```
"routes": [
  {
    "source": "^/sap/(.*)$",
    "target": "/sap/$1",
    "service": "com.sap.cloud.abap",
    "endpoint": "abap",
    "authenticationType": "xsuaa",
    "csrfProtection": false
  } ]
```

Bind ABAP Service Instance

Bind the service instance to the approuter. You can bind the service instance using the SAP BTP cockpit or by supplying the information in the multitarget application.

The following service binding parameters are supported by the ABAP service:

Binding Parameter	Description
abap_endpoint_timeout	Positive integer that defines the time in milliseconds when the communication between the approuter and service instance will be timed out. The default value is 30000, which corresponds to 30 seconds. This default value is suitable for most applications. However, the communication with the service instance may take longer. If you experience timeouts, you can maintain a value higher than 30 seconds. The maximum value is 600000, which corresponds to 10 minutes.
create_x_sap_security_session	Boolean that defines if the approuter shall request a security session cookie by setting the `x-sap-security-session: true` http header. The default is true. For more information regarding security session cookies, see SAP Note 3319136 - No implicit security session creation for API callers calling an ABAP Cloud system as of 2402.

For more information, see Binding Service Instances to Applications.

Bind Service Instance Using the Cockpit

- Log on to the cockpit and go to the subaccount that contains the space you'd like to navigate to. See Navigate in the Cockpit.
- In the navigation area, go to **Cloud Foundry Spaces** and choose your space.

- Select **service instances** from the navigation area and navigate to your ABAP system service instance.
- In the service instance details section that opens to the right, select the **Actions** menu and then select **Bind Application**.
- In the **New Binding wizard**, choose the application to bind.
- Provide the parameters for your binding by specifying the parameter in JSON format. For example:

  Sample Code

  ```
  {
    "abap_endpoint_timeout": 600000,
    "create_x_sap_security_session": false
  }
  ```

For more information, see Binding Service Instances to Applications.

Bind Service Instance in Multitarget Application

To bind the service instance to the approuter, add the service instance as a required resource in the `mta.yaml` file. For example:

Sample Code

```
_schema-version: "3.2"
ID: my-mta
version: 1.0.
modules:
- name: my-app
  type: approuter.nodejs
  path: cf/router
  requires:
  - name: my-abap-service-instance
    parameters:
      config:
        create_x_sap_security_session: false
        abap_endpoint_timeout: 600000
…
resources:
- name: my-abap-service-instance
  type: org.cloudfoundry.existing-service
  parameters:
    service: abap
    service-plan: standard
    service-name: my-abap-service-instance-name
…
```

4.2.16.2.3.2 Routing via Destination

Maintain Route

To access the service instance of the ABAP environment via a destination, maintain a route for the path of the OData service with a destination that points to the ABAP environment instance in the `xs-app.json` file.

Sample Code

```
"routes": [
  {
    "source": "^/sap/(.*)$",
    "target": "/sap/$1",
    "destination": "my-destination",
    "authenticationType": "xsuaa",
    "csrfProtection": false
  } ]
```

Bind Destination Service Instance

Bind the service instance to the approuter. You can bind the service instance using the cockpit or by supplying the information in the MTA. See Binding Service Instances to Applications for more information.

To bind the destination service instance to the approuter in the multitarget application, add it as a required resource in the `mta.yaml` file. For example:

Sample Code

```
_schema-version: "3.2"
ID: my-mta
version: 1.0.
modules:
- name: my-app
  type: approuter.nodejs
  path: cf/router
  requires:
  - name: my-abap-service-instance
    parameters:
      config:
        create_x_sap_security_session: false
        abap_endpoint_timeout: 600000
…
resources:
- name: my-abap-service-instance
  type: org.cloudfoundry.existing-service
  parameters:
    service: abap
    service-plan: standard
    service-name: my-abap-service-instance-name
…
```

Maintain Destination

Create a destination with the same name as specified in the `xs-app.json` file in the SAP Destinations Service instance that is bound to your application or on subaccount level, for example, maintain the `my-destination` destination in the `my-destination-service-instance-name` SAP Destination service instance.

If the application and the instance of the ABAP environment reside in the same subaccount, use the `OAuth2UserTokenExchange` authentication type for the destination for principal propagation. If the application and the instance of the ABAP environment are deployed to different subaccounts, then use the `SAMLAssertion` authentication type.

You can add additional properties to the destination, for instance:

Property	Description
HTML5.Timeout	Positive integer representing the maximum time to wait for a response (in milliseconds) from the destination. The default value is 30000 milliseconds, which corresponds to 30 seconds.
URL.headers.<header-key>	A static key prefix used as a namespace grouping of the URL's HTTP headers whose values will be sent to the target endpoint. For instance, if you maintain the property `URL.headers.x-sap-security-session` with value `true`, then the approuter will send the `x-sap-security-session: true` http header. For more information regarding security session cookie, see 3319136.

Related Information

Application Routes and Destinations

4.2.17 ABAP Service Instance URL

Context

Previously, when creating a new ABAP Cloud project in ABAP development tools for Eclipse, a service key was necessary to connect to the SAP Business AI Platform, ABAP environment. In the window *Connection to an ABAP Service Instance*, there was a distinction between the connection to SAP S/4HANA Cloud Public Edition and SAP Business AI Platform, ABAP environment. From now on, only one option to paste in your ABAP service instance URL is available.

Connection to an ABAP Service Instance

1. Select *Create a new ABAP cloud project*. This opens up a window called *Connection to an ABAP Service Instance*.
2. Following, you can now choose two options to connect to your ABAP service instance listed below.
    1. Insert the URL from the SAP Fiori launchpad of your system.
    2. If you still have a service key available, you can extract the URL from it by clicking the link shown in the description text of the window you're in: **Do you have a service key? Extract the URL from the service key.**
        1. In this case, a new window opens up where you can paste in your service key.
        2. Next, choose *Copy to Clipboard*.
        3. Now, you are in the *Connect to an ABAP Service Instance* window again and can paste in the service instance URL.
        4. Choose *Copy logon URL to clipboard*.
        5. Finally, you can paste the URL into your browser and log on to the system.
           The message shows *You have been successfully logged on* and ABAP development tools for Eclipse will pick up your project data.

4.3 Development in the Kyma Environment

Learn more about developing applications in SAP BTP, Kyma runtime.

Overview

With SAP BTP, Kyma runtime, you can build simple Functions, develop, and deploy more complex microservices, or mixtures of those, depending on your use case complexity level. Functions and microservices can act as stand-alone applications or extensions of these SAP systems:

- SAP S/4HANA
- SAP S/4HANA Cloud
- SAP S/4HANA Cloud, public edition
- SAP Field Service Management
- SAP SuccessFactors
- SAP Customer Experience systems:
    - SAP Commerce Cloud
    - SAP Cloud for Customer
    - SAP Marketing Cloud

Language of Choice

With the Kyma runtime, you can create microservices in the language of your choice and run them as containerized applications. Additionally, you can create Functions in Node.js or Python.

Kyma Dashboard and kubectl

The Kyma runtime comes with a central administration dashboard (Kyma dashboard), which allows you to add a module, deploy microservices, create Functions, and manage their configurations. You can also use it to connect SAP BTP services to your cluster and manage them using the SAP BTP Operator module, create instances of these services, and use them in your microservices or Functions. For more information, see Kyma Dashboard [page 93].

For those who prefer to work with command-line tools, the Kyma runtime also offers the Kubernetes command-line tool, kubectl .

SAP Cloud Application Programming Model

The SAP Cloud Application Programming Model (CAP) is the recommended framework for application and service development in the Kyma runtime. To learn more, see Programming Models [page 154].

Using API

The APIs of Kyma modules and third-party modules' may vary from the least stable alpha version through a more stable beta and the most stable **vX** version (**X** being an integer). While using the alpha version, you can face issues with module upgrades, because there is no guarantee that the API will not change or won't be removed in the future. For more information, check the Kubernetes API versioning .

→ Tip

To get the list of the APIs, their kind, and version, run:

```
kubectl get crd -o custom-
columns="KIND:.spec.names.kind,GROUP:.spec.group,VERSION:.spec.versions[*].nam
e"
```

If an API is available in two versions, always use the version that is more stable.

Development Tutorials

To help you get started with the development process, go through the following missions that show how to develop applications in the Kyma runtime.

- Build an Application in SAP BTP, Kyma Runtime
- Develop a Full-Stack Application in SAP BTP, Kyma Runtime
- Deploy a Full-Stack CAP Application in SAP BTP, Kyma Runtime Following SAP BTP Developer's Guide

Related Information

- Extension Samples
  Kyma provides a ready-to-use project that contains sample applications. You can use them to build event- and API-based extensions in the Kyma runtime using your favorite technology. These samples are implemented in multiple languages and demonstrate various Kyma features and use case scenarios.
- Using SAP BTP Services in the Kyma Environment [page 2267]
  With the Kyma runtime, you can extend the SAP systems to build and deploy your own applications.
- Deploy Workloads in the Kyma Environment to Extend SAP Systems [page 2267]
  Access the Kyma runtime and start creating extensions for SAP systems using Functions.

4.3.1 Kyma Modules

With Kyma's modular approach, you can install just the modules you need, instead of a predefined set of components.

Pick the modules that best meet your business needs. You can add or delete modules on demand and configure them, thus determining the costs generated by the infrastructure that your modules use. To learn how, see Adding and Deleting a Kyma Module [page 3511].

You can decide which modules are needed for your workload, although some features may still require the presence of other modules or related capabilities. For example, the API Gateway module depends on Istio capabilities, but it doesn't require the Istio module from Kyma. Instead, the API Gateway module needs Istio custom resources, such as `Gateway` and `VirtualService`, to be present in the cluster. If you want to use your own Istio installation, the API Gateway module will use it as soon as the required resources are available.

> → Tip
>
> A release of a new module's version is announced with a release note in What's New for SAP Business Technology Platform for both, the fast and regular channels:
>
> - On the day of the release in the fast channel, a release note is published with the *Preview* label.
> - When the module version becomes available in the regular channel (after approximately two weeks), the *Preview* label is removed.
>
> For details, see Kyma Module Releases [page 91].
>
> To find out which module version is running in your cluster, go to Kyma dashboard.

Default Kyma Modules

When you create Kyma runtime in SAP BTP cockpit, it is provisioned with the default modules added. The default modules are not mandatory. If you don't need them, you can delete them in Kyma dashboard.

Default Kyma Modules

Module Technical Name	Purpose	Documentation
Istio `istio`	Istio is a service mesh with Kyma-specific configuration.	Istio Module [page 1887]
API Gateway `api-gateway`	API Gateway provides functionalities that allow you to expose and secure APIs.	API Gateway Module [page 1944]
SAP BTP Operator `btp-operator`	Within the SAP BTP Operator module, BTP Manager installs the SAP BTP service operator that allows you to consume SAP BTP services from your Kubernetes cluster using Kubernetes-native tools.	SAP BTP Operator Module [page 2070] What's New for SAP BTP Operator GitHub repository: BTP Manager

Optional Kyma Modules

When you create Kyma runtime in SAP BTP cockpit, the following modules are not added by default, but you can choose to add and delete them anytime in Kyma dashboard.

Optional Kyma Modules

Module Technical Name	Purpose	Documentation
Application Connector `application-connector`	Application Connector allows you to connect with external solutions. No matter if you want to integrate an on-premise or a cloud system, the integration process doesn't change, which allows you to avoid any configuration or network-related problems.	Application Connector Module [page 2116] GitHub repository: Application Connector Manager
Cloud Manager `cloud-manager`	Cloud Manager brings cloud provider products and resources into the Kyma cluster in a secure way.	Cloud Manager Module [page 2106] GitHub repository: Cloud Manager

Module Technical Name	Purpose	Documentation
Eventing `eventing`	Eventing provides functionality to publish and subscribe to CloudEvents. At the moment, the SAP Event Mesh default plan and NATS are supported. If you choose NATS, add the NATS module.	Eventing Module [page 2136] GitHub repository: Eventing
Keda `keda`	The Keda module comes with Keda Manager, an extension to Kyma that allows you to install KEDA (Kubernetes Event-Driven Autoscaler).	Keda Module [page 2120]
NATS `nats`	NATS deploys a NATS cluster within the Kyma cluster. You can use it as a backend for Kyma Eventing.	NATS Module [page 2152] GitHub repository: NATS
Registry Cache `registry-cache`	With the Registry Cache module, you can add a caching layer for container image registries, reducing outbound traffic to upstream registries and improving image pull performance.	Registry Cache Module [page 2160] GitHub repository: Registry Cache
Serverless `serverless`	With the Serverless module, you can define simple code snippets (Functions) with minimal implementation effort.	Serverless Module [page 2169] Configuring Serverless [page 2175] Deploy Workloads in the Kyma Environment to Extend SAP Systems [page 2267] GitHub repository: Serverless
Telemetry `telemetry`	The Telemetry module collects application logs and distributed traces for your application, and dispatches them to your preferred backends.	Telemetry Module [page 2184] What's New for Telemetry GitHub repository: Telemetry

Other SAP Modules

These modules are developed and maintained by SAP teams outside of Kyma. To get help or request a feature, contact the module provider directly.

Other SAP Modules

Module Technical Name	Purpose	Documentation
Transparent Proxy `transparent-proxy`	Use the transparent proxy for Kubernetes to connect workloads in a Kubernetes cluster to Internet and on-premise applications.	Transparent Proxy in the Kyma Environment

Module Technical Name	Purpose	Documentation
Connectivity Proxy `connectivity-proxy`	Use the connectivity proxy for Kubernetes to connect workloads in a Kubernetes cluster to on-premise systems, exposed via the Cloud Connector	On-Premise Connectivity in the Kyma Environment

Community Modules

Community modules are modules provided by the Kyma community. Use them to enhance your Kyma experience with additional features and capabilities. You can install community modules alongside managed modules. Unlike managed modules, community modules aren't automatically updated or maintained. For more information, see kyma-project.io: Community Modules .

> ⚠ Caution
>
> Community modules aren't subject to the Service Level Agreement (SLA).

Related Information

Adding and Deleting a Kyma Module [page 3511]

4.3.1.1 Istio Module

Use the Istio module to manage and configure the Istio service mesh.

What Is Istio?

Istio is an open-source service mesh that provides a uniform way to manage, connect, and secure microservices. It helps to manage traffic, enhance security capabilities, and provide telemetry data for understanding service behavior. See the open-source Istio documentation .

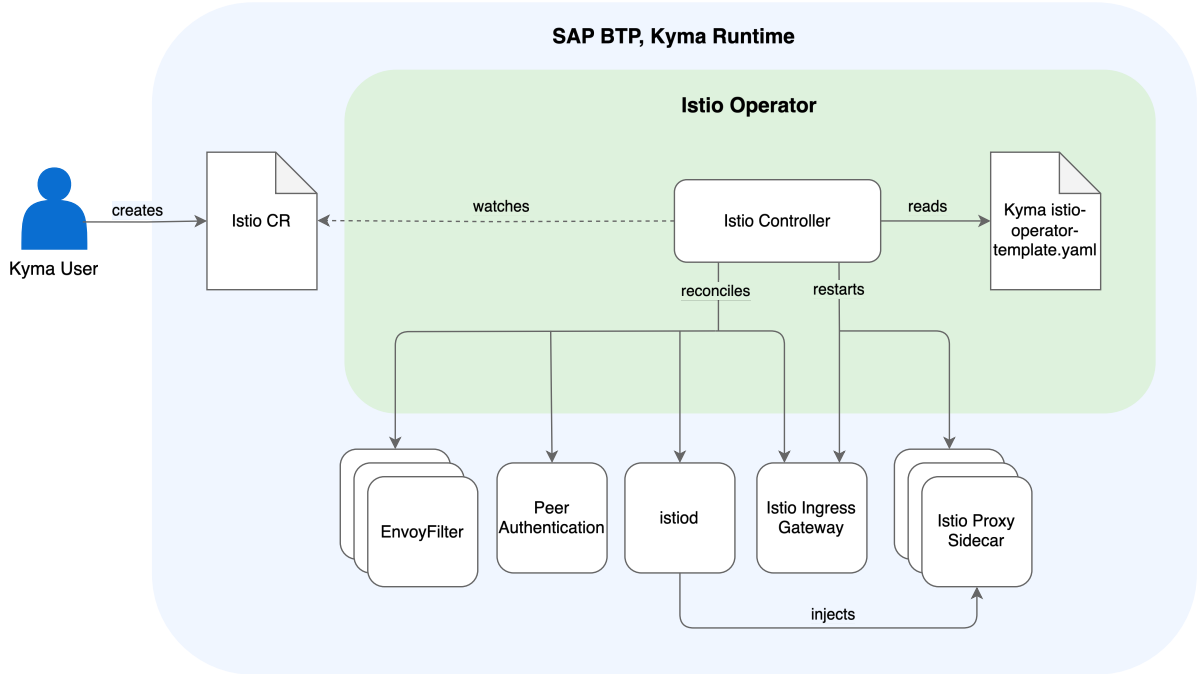
The Istio module installs and manages Istio in your Kyma cluster. By default, the Istio module is automatically added once you create a Kyma runtime instance.

Features

The Istio module offers the following features:

- Management of Istio installation and upgrades: The module installs Istio and simplifies the process of managing its installation, reducing the complexity and time required for maintenance.
- Default Istio configuration: You can quickly have Istio installed with default settings.
- Fine-tuning capabilities: You can optimize Istio and fine-tune its settings according to specific performance or operational requirements.
- Synchronization of the data plane with the Istio control plane: This ensures that changes you make to the control plane are consistently reflected in the data plane, ensuring that the network operates consistently and reliably without any discrepancies.
- Support for the X-Forwarded-For (XFF) header: You can configure the XFF header to manage and track the source of incoming requests.

Architecture



Istio Operator

Within the Istio module, Istio Operator handles the management and configuration of the Istio service mesh. It contains one controller, referred to as Istio Controller.

Istio Controller

Istio Controller manages Istio installation as defined in the Istio custom resource (CR). Istio Controller is responsible for:

- Installing, upgrading, and uninstalling Istio
- Restarting workloads that have Istio sidecar proxy injected to ensure that these workloads use the correct version of Istio

API/Custom Resource Definitions

The `istios.operator.kyma-project.io` CustomResourceDefinition (CRD) describes the Istio CR that Istio Controller uses to manage the installation of Istio. See Istio Custom Resource .

Authorization

To assign access permissions to the Istio module resources, use the following aggregated ClusterRoles :

- `kyma-istio-view` - Grants read-only access to the `Istio` custom resource.
- `kyma-istio-edit` - Grants full access to the `Istio` custom resource.
- `kyma-istio-resources-view` - Grants read-only access to resources from all API groups managed by Istio.
- `kyma-istio-resources-edit` - Grants full access to resources from all API groups managed by Istio.

Resource Consumption

To learn more about the resources used by the Istio module, see Kyma Modules' Sizing [page 3518].

4.3.1.1.1 Istio Service Mesh

Learn more about Istio sidecar proxies: what they are, what benefits they bring, how and when the Istio module restarts workloads with Istio sidecar proxy injection enabled.

What Is a Service Mesh?

A service mesh is an infrastructure layer that handles service-to-service communication, proxying, service discovery, traceability, and security, independently of the code of the services. To deliver this functionality, the Istio module uses the Istio service mesh that is customized for the specific needs of an implementation. The main principle of the Istio service mesh is to inject Pods of every service with Istio sidecar proxy, which is an extended version of the Envoy proxy. Envoy intercepts the communication between the services and regulates it by applying and enforcing the rules you create.

Purpose and Benefits of Istio Sidecar Proxies

By default, Istio installed as part of the Istio module is configured with automatic Istio sidecar proxy injection disabled. This means that none of your workloads' Pods, except those in the `kyma-system` namespace, get their own sidecar proxy container running next to the application. When Istio sidecar proxy injection is disabled for a service or for a namespace, you must manage mutual TLS (mTLS) traffic in services or at a namespace level by creating DestinationRule and PeerAuthentication resources.

With an Istio sidecar proxy injected, a resource becomes part of the Istio service mesh, which brings the following benefits that would be complex to manage otherwise.

Secure Communication

The Istio module sets peer authentication to cluster-wide STRICT mode. This ensures that your workload only accepts mutual TLS (mTLS) traffic where both client and server certificates are validated to ensure that all traffic is encrypted. This provides each service with a strong identity and a reliable system for managing keys and certificates.

Also, with Istio sidecar proxy injected, you can perform request authentication for your service. Istio enables request authentication with JSON Web Token (JWT) validation using a custom authentication provider.

Observability

Istio sidecar proxies enhance tracing capabilities by performing global tracing and forwarding the data to a tracing backend using the OTLP protocol. When you integrate your application into the Istio service mesh, you can easily access advanced observability features without needing to implement complex instrumentation within the application. See Configure Observability for the Istio Service Mesh [page 1917].

Traffic Management

If you have the sidecar injected into every workload, you can use Istio's traffic routing rules without additional configuration. See Traffic management.

Traffic shifting and request routing allows you to use techniques like canary releases and A/B testing to make your software release process faster and more reliable. To improve the resiliency of your applications, you can use mirroring and fault injection for testing and audit purposes.

Resiliency

Application resiliency is an important topic within traffic management. Traditionally, application libraries implemented resiliency features like timeouts, retries, and circuit breakers. However, you can delegate such tasks to a service mesh, and the same configuration options work regardless of the programming language of your application. See Network Resilience and Testing.

4.3.1.1.1.1 Restart of Workloads in the Istio Service Mesh

When you add workloads to the Istio service mesh, you must be aware that the Istio module handles the automatic restart of these workloads. A restart happens in two scenarios:

- During Istio updates
- When you update certain fields in the Istio configuration

To effectively monitor your workloads, learn when and why they might be restarted. The Istio module restarts both types of sidecar containers: regular ones and Kubernetes native sidecars.

> → Remember
>
> To improve resiliency and ensure continuous service operation during the Istio module's rollouts, you must properly configure Pod Disruption Budgets (PDBs) in Kubernetes. Additionally, for PDBs to function correctly, your application must be well prepared to run in the Kubernetes environment. This includes implementing Kubernetes probes. For more information, see Develop Resilient Applications in the Kyma Runtime [page 2278]. Adhering to these practices highly reduces the risk of disruptions caused by workload restarts.

Workload Restart During Istio Updates

The main function of the Istio module is to install Istio within your Kubernetes cluster and ensure it remains up-to-date. To achieve this, new versions of the Istio module often install updated versions of Istio in your Kyma runtime. The module handles the update process, saving you the time and effort of manually rolling the Istio updates.

When a new version of the Istio module installs an updated version of Istio, workloads that are part of the Istio service mesh (those that have Istio sidecar proxies injected into them) are automatically restarted.

To understand why the update is required in this scenario, it's important to grasp the architecture of the Istio service mesh. The Istio service mesh consists of two primary components: the data plane and the control plane. The data plane is a set of proxies mediating and controlling network communication between microservices, while the control plane is responsible for managing and configuring these proxies. During an Istio update, the module deploys an updated version of the control plane. Then, to maintain compatibility between the control plane and the data plane, the Istio module must restart any workloads with Istio sidecar proxies running alongside them. This step ensures that Istio sidecar proxies can properly communicate with the updated control plane. Without the Istio module managing this process, you must manually deploy the new control plane and manually restart the workloads with sidecar proxies to retrigger sidecar injection.

If you uninstall Istio from your Kyma runtime, the workloads are restarted to remove the Istio sidecar proxies.

Workload Restart During Istio Configuration Updates

Another major feature of the Istio module is managing the Istio configuration. During Istio installation, the Istio module provides opinionated Istio configuration and ensures that it is correctly applied. Then, the module provides you with access to a subset of this configuration through the Istio CR. By editing the Istio CR, you can modify specific fields and apply your changes.

If you modify the Istio CR, the Istio module ensures that these changes are effective. In some cases, to successfully apply the updated settings, the module must restart Pods that have an Istio sidecar proxy injected. The restart is necessary in the following cases:

- When you update the field `spec.config.telemetry.metrics.prometheusMerge` in the Istio CR.
- When you enable the compatibility mode (`spec.compatibilityMode`), and the compatibility version introduces any flags to the Istio proxy component.

- When you update the field `spec.config.NumTrustedProxies` in the Istio CR, only Istio sidecar proxies that are part of the `istio-ingressgateway` Deployment are restarted.

When a Workload Can't Be Restarted

Restarting the Istio sidecar proxies is possible for all resources that allow for a rolling restart. However, if a resource is a Job or a Pod that is not managed by any other resource, the restart can't be performed automatically. In such cases, a warning is logged, and you must manually restart the resources. See Incompatible Sidecar Version After the Istio Module's Update [page 1936].

The Istio module does not restart an Istio sidecar proxy if it has a custom image set. See Resource Annotations .

> ⚠ Caution
>
> Istio-injected Pods with `restartPolicy: Never` may end up in a permanently broken state due to a known issue in Istio. See issue #49210 . If you need to use this setting, you must be aware of the risk until the issue is fixed. If you don't have a specific need for setting `restartPolicy` to Never, consider using a different option.

4.3.1.1.1.2 Enabling Istio Sidecar Proxy Injection

To include your workloads into the Istio service mesh, enable the Istio sidecar proxy injection. You can do this for either an entire namespace or a single Deployment. Learn how to perform both these operations.

Enabling Injection for a Namespace

Prerequisites

- You have the Istio module added. See Adding and Deleting a Kyma Module [page 3511].
- You have access to Kyma dashboard. Alternatively, to use CLI instructions, you must install kubectl .

Context

Enabling Istio sidecar proxy injection for a namespace allows istiod to watch all Pod creation operations in this namespace and automatically inject newly created Pods with an Istio sidecar proxy.

> ### Note
>
> A Pod is not injected with an Istio sidecar proxy if:
>
> - Istio sidecar proxy injection is disabled at the namespace level
> - The `sidecar.istio.io/inject` label on the Pod is set to false
> - The Pod's `spec` contains `hostNetwork: true`

Procedure

- You can either use Kyma dashboard, or kubectl.
- Use Kyma dashboard.
  a. Select the namespace where you want to enable Istio sidecar proxy injection.
  b. Choose *Edit*.
  c. In the *UI Form* section, switch the toggle to enable Istio sidecar proxy injection.
  d. Choose *Save*.
- Use kubectl.

  Replace the placeholder and run the command:

  ```
  kubectl label namespace <namespace-name> istio-injection=enabled
  ```

Results

You've enabled Istio sidecar proxy injection for the specified namespace. The namespace is labeled with `istio-injection: enabled`, which means that all Pods created in it from now on will have the Istio sidecar proxy injected.

Enabling Injection for a Deployment

Prerequisites

- You have the Istio module added. See Adding and Deleting a Kyma Module [page 3511].
- To use CLI instructions, you must install kubectl . Alternatively, you can use Kyma dashboard.

Context

Enabling Istio sidecar proxy injection for a Deployment injects an Istio sidecar proxy into all the Deployment's Pods.

> ### Note
>
> A Pod is not injected with an Istio sidecar proxy if:
>
> - Istio sidecar proxy injection is disabled at the namespace level
> - The `sidecar.istio.io/inject` label on the Pod is set to false
> - The Pod's `spec` contains `hostNetwork: true`

Procedure

- Use Kyma dashboard.
  a. Select the namespace of the Deployment for which you want to enable Istio sidecar proxy injection.
  b. In the *Workloads* section, choose *Deployments*.
  c. Choose the Deployment.
  d. Choose *Edit*.
  e. In the *UI Form* section, switch the toggle to enable Istio sidecar proxy injection.
  f. Choose *Save*.
- Use kubectl.

  Replace the placeholders and run the following command:

  ```
  kubectl patch -n <namespace-name> deployments/<deployment-name> -p '{"spec":
  {"template":{"metadata":{"labels":{"sidecar.istio.io/inject":"true"}}}}}'
  ```

Results

You've enabled Istio sidecar proxy injection for the specified Deployment. The Deployment and all its Pods are labeled with `istio-injection: enabled`. All the Deployment's Pods are instantly injected with an Istio sidecar proxy.

4.3.1.1.1.3 Regular and Native Sidecar Containers

Understand the differences between Istio proxies running as regular containers and as native sidecars, learn about the default settings applied by the Istio module, and find out how to override these settings for specific workloads as needed.

Advantages of Native Sidecar Containers

Every Pod in the Istio mesh gets an additional `istio-proxy` container, which intercepts network traffic to provide Istio features. For Istio proxies, Istio uses two types of containers: regular containers and native

sidecars. Native sidecars are init containers that have `restartPolicy` set to Always. This approach, introduced with Kubernetes 1.28, allows for controlling the startup and shutdown sequence of containers within a Pod, which is not possible with sidecars deployed as regular containers.

Using native sidecars resolves the following problems that occur when Istio proxies run as regular containers:

- When an application container starts before the regular `istio-proxy` container is ready, the application doesn't have network access at startup.
- When `istio-proxy` running as a regular container stops before the application container stops, the application loses network access during shutdown.
- An init container runs before the `istio-proxy` regular container, so it can't access the network.
- A running regular `istio-proxy` container prevents Pods from being finished (for example, in the case of Jobs).

Native Sidecars as the Default Istio Proxy Container Type

By default, the Istio module 1.22 and later versions use `istio-proxy` native sidecars. This setting is configured globally and applied to all Pods that allow for Istio sidecar proxy injection. However, you can override this default setting for a specific Pod, allowing it to run `istio-proxy` as a regular container instead.

Furthermore, as long as the Istio module deploys Istio version 1.27, you have the option to set the `compatibilityMode` in the Istio Custom Resource (CR) to true. This setting makes Istio behave as version 1.26, which uses regular `istio-proxy` containers by default.

Istio module version	Istio version	Default Sidecar Type
1.20 or lower	1.26 or lower	Regular sidecars
1.21	1.27	Regular sidecars
1.22	1.27	Native sidecars, unless you set `compatibilityMode` in the Istio CR to `true`
One of the next versions after 1.22 with updated Istio	1.28	Native sidecars

Configuring Regular Istio Proxy Containers for a Particular Workload

You can explicitly configure a specific workload to run `istio-proxy` as a regular container. To inject `istio-proxy` as a regular container, set the `sidecar.istio.io/nativeSidecar` annotation to "false" on a given Pod or in the Pod template. If you do not set the annotation, native sidecars are used.

You must set the annotation at the Pod level. So, when a Pod is created by a parent resource (for example, Deployment, StatefulSet, ReplicaSet, DaemonSet, Job, CronJob), you must configure the annotation in the Pod template. See the example:

Sample Code

```
apiVersion: apps/v1
```

```
kind: Deployment
metadata:
  name: test-deployment
spec:
  template:
    metadata:
      annotations:
        sidecar.istio.io/nativeSidecar: "false"
...
```

Running `istio-proxy` as a regular container is only recommended when a specific workload experiences issues when the Istio proxy is a native sidecar. This might be necessary if you have implemented workarounds to address problems caused by Istio proxies running as regular containers and now need additional time to adapt to native sidecars becoming the new default.

Support in the Istio Module

The Istio module fully supports Istio proxies run as regular containers and as init containers. In particular, during Istio version's update, the module detects which workloads should be restarted and restarts both container types.

Example of Istio Proxy Running as a Native Sidecar

Follow this example to observe the differences between regular sidecar containers and native sidecar containers.

Example

1. Create a test namespace with Istio injection enabled:

   #### Sample Code

   ```
   kubectl create ns test
   kubectl label namespace test istio-injection=enabled
   ```

2. Create a workload with `istio-proxy` running as a native sidecar:
   The Pod template does not contain the annotation `sidecar.istio.io/nativeSidecar: "false"`, so the `istio-proxy` is injected as a native sidecar.

   #### Sample Code

   ```
   kubectl apply -f - <<EOF
   apiVersion: batch/v1
   kind: Job
   metadata:
     name: test-job
     namespace: test
   spec:
     template:
       spec:
         containers:
         - name: test-job
   ```

```
        image: alpine:latest
        command: [ "/bin/sleep", "10" ]
      restartPolicy: Never
EOF
```

3. To observe the progress, run:

Sample Code

```
kubectl get pod -n test -w
```

Output Code

```
NAME             READY   STATUS     RESTARTS   AGE
test-job-2tt95   0/2     Init:0/2   0          0s
test-job-2tt95   0/2     Init:1/2   0          1s
test-job-2tt95   0/2     Init:1/2   0          2s
test-job-2tt95   0/2     PodInitializing   0          10s
test-job-2tt95   1/2     PodInitializing   0          10s
test-job-2tt95   2/2     Running           0          11s
test-job-2tt95   1/2     Completed         0          21s
test-job-2tt95   0/2     Completed         0          22s
```

See the key differences when compared to the `istio-proxy` run as a regular container:

- When the Pod starts, it has the `Init` status with two containers.
- The application container only starts after the `istio-proxy` native sidecar is fully operational.
- After the main container completes its job, the Pod finishes with the status `Completed`.

4. List init containers:

Sample Code

```
kubectl get pod -n test -o
jsonpath='{.items[0].spec.initContainers[*].name}'
```

The output contains two containers, and `istio-proxy` is now an init container:

Output Code

```
istio-validation istio-proxy
```

5. List regular containers:

Sample Code

```
kubectl get pod -n test -o
jsonpath='{.items[0].spec.containers[*].name}'
```

The output includes only the application container without `istio-proxy`:

Output Code

```
test-job
```

Known Hacks that Native Sidecars Can Replace

The `/quitquitquit` call

The `istio-proxy` sidecar offers an additional endpoint `/quitquitquit` that you can use in the following way to shut down `istio-proxy`:

```
curl -sf -XPOST http://127.0.0.1:15020/quitquitquitCopy to clipboardErrorCopied
```

The solution was proposed under issue #6324 .

Using the endpoint is no longer required if `istio-proxy` runs as a native sidecar. Its lifecycle is bound to the main application container, so the `istio-proxy` sidecar shuts down automatically when the main container is finished.

`runAsUser 1337`, or `excludeOutboundIPRanges`, or `excludeOutboundPorts`

Init containers are started before regular containers. This means that the `istio-proxy` running as a regular container doesn't work when init containers are running. As a result init containers don't have network access.

By configuring `UID 1337`, `excludeOutboundIPRanges`, or `excludeOutboundPorts` you can exclude the network traffic from being captured by the `istio-proxy`. This allows init containers to access the network, but they are able to connect only to resources outside service mesh. The solution was proposed in the blog post Upcoming breaking change in SAP BTP, Kyma Runtime: Enabling the Istio CNI plugin.

This is no longer required if `istio-proxy` runs as a native sidecar because the `istio-proxy` is injected as the first init container and runs until the main application container finishes. So, init containers are able to access the network in the same way as regular application containers, and they may connect to resources inside and outside the service mesh.

Related Information

Kubernetes v1.28: Introducing native sidecar containers
Kubernetes Native Sidecars in Istio

4.3.1.1.1.4 Optimizing Sidecars

Configure Istio using the Sidecar resource to minimize the load and footprint.

With automatic Istio sidecar proxy injection enabled, all of your workloads' Pods get their own sidecar proxy container running next to the application. Using sidecar proxies is beneficial but also has some side effects.

Default Behavior

By default, Istio enables communication between all the services in the service mesh. As a result, each sidecar proxy receives all the configurations for every service. The configurations contain a lot of Istio-based

resource details. Also, when a service scales or a Pod restarts, the Istio control plane always propagates the configurations of all other services to Istio sidecar proxies.

If you have only a few services in your Kyma cluster, this is not a problem. However, as you start expanding your microservice architecture and the number of services grows, you can notice the following effects:

- Increased network traffic and CPU usage for the Istio control plane.
- Increased memory usage due to the growing number of configurations stored by each sidecar proxy.
- Slower changes propagation to the services in the mesh.

All these effects can be solved with the optimized usage of the Istio Sidecar resource.

Optimization

To select which services communicate with each other, specify the configurations the proxy should get from the Istio control plane using the Sidecar custom resource (CR).

In plain language, using the Sidecar CR, you can tell the Istio control plane the following: *There are 500 services in my Kyma runtime, but my application/scenario needs to access only 5. I need details only about those 5.*

See the following example of the Sidecar CR configuration:

{·} Sample Code

```
apiVersion: networking.istio.io/v1beta1
kind: Sidecar
metadata:
  name: default
  namespace: my-namespace
spec:
  egress:
  - hosts:
    - ./*
    - istio-system/*
    - kyma-system/eventing-event-publisher-proxy.kyma-system.svc.cluster.local
```

The following table presents the example `egress.hosts` values from the sample code and their corresponding access details.

Example egress.hosts Values

Value	Description
`./*`	Access all services in this namespace.
`istio-system/*`	Access all services in the `istio-system` namespace.
`kyma-system/eventing-event-publisher-proxy.kyma-system.svc.cluster.local`	Access the `eventing-event-publisher-proxy` service to publish events.

→ Tip

If you want to apply the configuration to specific workloads, configure the `workloadSelector`. For more information, see Sidecar in the official Istio documentation.

4.3.1.1.1.5 Istio Version

To track the changes introduced in the upstream Istio, learn which version of Istio the Istio module installs. To revert certain changes in Istio's behavior when you encounter compatibility issues with its new version, consider enabling compatibility mode.

The version of Istio depends on the version of the Istio module that you use. If a new version of the Istio module introduces a new version of Istio, an upgrade of the module causes an automatic upgrade of Istio. To learn which version of Istio is installed by the newest version of the Istio module, follow What's New notes.

Compatibility Mode

Compatibility mode allows you to revert certain changes in Istio's behavior, and it is recommended only when you encounter compatibility issues with the new version of Istio. The Istio module supports compatibility with the previous minor version of Istio. For example, for the version of the Istio module that contains Istio 1.24, you can apply a compatibility version of Istio 1.23. See Compatibility Versions .

> ⚠ Caution
>
> You can use the compatibility mode to retain the behavior of the current Istio version before a new version of the Istio module with a higher version of Istio is released. Then, the compatibility is first set to a minor version lower than the one you are currently using. If this lower version's behavior is not compatible with your current mesh setup, some configurations may be broken until the new release of the Istio module is rolled out.

To enable compatibility mode, set the `spec.compatibilityMode` field in the Istio CR to true.

When you set `spec.compatibilityMode: true`, the Istio module applies an opinionated subset of Istio `compatibilityVersion` variables. To learn more about the changes that specific compatibility versions revert, follow What's New notes.

4.3.1.1.1.6 Configure Istio Certificate Authority (CA) with Custom Certificates

For enhanced security, replace Istio's default self-signed certificates with administrator-provided certificates.

Default Istio Behavior

By default, Istio generates a self-signed root certificate and stores it in a Secret within a Kyma cluster. Istio uses this certificate to issue certificates for each workload in the service mesh.

Configuring a Custom CA

> → Recommendation
>
> For live landscapes, it is recommended to use a production-ready CA and manage your root CA on a secure, offline machine. In addition, consider using an additional intermediate CA outside of the cluster and automate the submission of the CA certificates to the clusters.
>
> Additionally, make sure to implement proper certificate rotation procedures according to your organization's security policies.

You can replace the default self-signed certificate provided by Istio and use a certificate issued by your organization. Istio supports the following public key infrastructure (PKI) hierarchy:

- **Root CA**: A self-signed certificate running outside of the cluster that issues intermediate certificates to the Istio CAs that run in each cluster. It's recommended that this certificate runs on an offline, secure machine.
- **Intermediate CAs**: Issued by the Root CA to each Kubernetes cluster running Istio. It is recommended to keep the Intermediate CA certificates as short-lived as possible.
- **Workload Certificates**: Automatically generated by Istio for each Pod and signed by the cluster's intermediate CA.

To configure a custom certificate in Kyma, you must create a Secret `cacerts` in each cluster's `istio-system` namespace. To create the Secret, you need the following input:

File	Description
`ca-cert.pem`	The intermediate CA certificate with permissions to sign server certificates. You can use any CA certificate in your organization's chain of trust.
`ca-key.pem`	The private key corresponding to `ca-cert.pem`. Istio uses it to sign new workload certificates.
`root-cert.pem`	The top-level root CA certificate in your trust chain.
`cert-chain.pem`	The complete certificate chain from your intermediate CA (`ca-cert.pem`) to the root CA (`root-cert.pem`).

To create the Secret for cluster `cluster1`, run:

```
kubectl create secret generic cacerts -n istio-system \
    --from-file=cluster1/ca-cert.pem \
    --from-file=cluster1/ca-key.pem \
    --from-file=cluster1/root-cert.pem \
    --from-file=cluster1/cert-chain.pem
```

For more information, see Plug in CA Certificates .

4.3.1.1.1.7 Istio Trust Domain

Configure a trust domain to align identities across multiple clusters, enable seamless multi-mesh mTLS communication with shared root CAs, and match your organizational naming standards.

What Is a Trust Domain?

A trust domain defines the security boundary of a service mesh. The trust domain becomes part of each proxy identity and is embedded into the Subject Alternative Name (SAN) of workload certificates.

Istio uses Secure Production Identity Framework For Everyone (SPIFFE) identities in the following format:

```
spiffe://<trust-domain>/ns/<namespace>/sa/<service-account>
```

By default, Istio uses the trust domain `cluster.local`. You can override this default to align identities across clusters or to match organizational naming standards.

Configure a Trust Domain in the Istio Custom Resource

Set the trust domain under `spec.config.trustDomain` in the Istio custom resource (CR). When you don't specify this field, the default value `cluster.local` is used.

```
apiVersion: operator.kyma-project.io/v1alpha2
kind: Istio
metadata:
  name: default
  namespace: kyma-system
spec:
  config:
    trustDomain: "example.com"
```

Where Does the Trust Domain Appear?

The trust domain appears in the following key contexts:

- Istio proxy identity certificate: The trust domain is included in the SAN of the certificate issued to every Istio proxy in the mesh.
- AuthorizationPolicies: When you define AuthorizationPolicies, you can use either the default value `cluster.local` or your custom trust domain in the SPIFFE IDs to specify the workloads to which the policy applies.

Multi-Mesh Seamless mTLS Communication

If a shared Root CA is configured to issue proxy certificates in multiple meshes (see Configure Istio Certificate Authority (CA) with Custom Certificates [page 1900]), you can use a Gateway with `tls.mode: ISTIO_MUTUAL` to seamlessly allow mTLS communication between them. In this case, you don't need to manually set up mTLS for the Gateway.

Additionally, you can use the trust domain to differentiate the source of requests in the upstream mesh (whether they come from within the same mesh or from another mesh). This is possible because the SPIFFE ID in the client certificate includes the trust domain of the originating mesh.

To configure this setup, you can use the following example Gateway configuration in the target mesh:

```
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: multi-mesh-gateway
  namespace: istio-system
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 443
      name: https
      protocol: HTTPS
    tls:
      mode: ISTIO_MUTUAL
    hosts:
    - "*.example-target.com"
```

In the source mesh, configure a ServiceEntry and DestinationRule to direct traffic to the target mesh with mTLS. See the following example:

```
apiVersion: networking.istio.io/v1
kind: ServiceEntry
metadata:
  name: external-httpbin
spec:
  hosts:
    - "httpbin.example-target.com"
  ports:
    - number: 80
      name: http
      protocol: HTTP
      targetPort: 443
  resolution: DNS
  location: MESH_EXTERNAL
---
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: external-httpbin-mtls
spec:
  host: "httpbin.example-target.com"
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL
```

Migration Considerations

Changing the trust domain is a sensitive operation. After you update the trust domain, existing workloads may fail authentication until they receive new certificates.

The default `cluster.local` value is treated as a special case that is compatible with custom trust domains. This means that setting the trust domain from `cluster.local` to a new, custom value doesn't cause authentication failures for existing workloads until they restart and receive new certificates with the updated trust domain.

However, you should still plan the change carefully and perform it during a maintenance window to minimize disruption. For a detailed migration guide, see Trust Domain Migration .

Related Information

Istio trust domain migration
Istio mesh configuration reference
SPIFFE trust domain specification

4.3.1.1.1.8 DNS Proxying

Learn how DNS proxying in Istio improves DNS resolution performance, enables external (outside of mesh) service resolution with `ServiceEntry` resources, and solves routing issues for external TCP services, which share the same port.

Overview

DNS resolution is a vital component of any application infrastructure on Kubernetes. When your application code attempts to access another service in the Kubernetes cluster, or even a service on the internet, it must first look up the IP address corresponding to the service's hostname before initiating a connection to the service.

DNS proxying intercepts DNS requests from applications and resolves them locally at the Istio proxy level. The proxy maintains a local mapping of host names to IP addresses based on the Kubernetes services and `ServiceEntry` resources in the cluster. If a hostname can be resolved locally within the mesh, the proxy responds immediately. Otherwise, it forwards the request upstream following the standard DNS resolution.

Improvements DNS Proxying Introduces

Reduced Load on Upstream DNS

Without DNS proxying, every DNS query from workloads goes to the upstream DNS server. With DNS proxying enabled, the Istio proxy resolves known service addresses locally in the mesh, reducing traffic to the upstream DNS server.

`ServiceEntry` Resolution

When you define a `ServiceEntry` with a custom hostname (for example, `address.internal`) that the upstream DNS does not know about, applications cannot resolve these addresses without DNS proxying. DNS proxying allows the Istio proxy to resolve `ServiceEntry` hostnames directly.

DNS Resolution for External TCP Services on the Same Port

In some cases, TCP traffic in Istio is routed based on destination IP and port only. Unlike HTTP traffic, which includes a Host header, TCP has no additional metadata for routing decisions.

For example, when multiple external TCP services share the same port (for example, two databases on port `3306`) and they do not have stable IP addresses, the specified `ServiceEntry` resources have resolution set to DNS, and Istio cannot distinguish between them. The proxy creates a single listener on `0.0.0.0:{port}` and forwards traffic to a single destination (an external TCP service).

DNS proxying solves this by auto-allocating virtual IPs (VIPs) from the `240.240.0.0/16` range to each `ServiceEntry`. This gives each external TCP service a unique address, enabling the proxy to route traffic correctly even when multiple services share the same port.

How to Enable DNS Proxying

DNS proxying is disabled by default. You can enable it globally for the entire mesh or locally per workload. Local per-workload configuration overrides the global mesh configuration.

Global Mesh Configuration

You can enable DNS proxying globally using the Kyma Dashboard or `kubectl`.

To enable DNS proxying using the Kyma Dashboard, perform these steps:

1. Go to *Cluster Details* in Kyma dashboard.
2. In the `kyma-system` namespace, go to the *Istio* section.
3. Choose *Edit*.
4. In the configuration, set `Enable global DNS Proxying` to true.
5. Save the changes.

With kubectl, run the following command to set `enableDNSProxying` to true in the Istio custom resource (CR):

```
apiVersion: operator.kyma-project.io/v1alpha2
kind: Istio
metadata:
  name: default
  namespace: kyma-system
```

```
spec:
  config:
    enableDNSProxying: true
```

You can also use `kubectl patch`:

```
kubectl patch istio default -n kyma-system --type=merge -p '{"spec":{"config":
{"enableDNSProxying":true}}}'
```

To verify the configuration, run:

```
kubectl get istio default -n kyma-system -o
jsonpath='{.spec.config.enableDNSProxying}'
```

This enables DNS proxying globally for all proxies in the mesh.

Local Per-Workload Configuration

Add the `proxy.istio.io/config` annotation to enable DNS proxying for a specific workload:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  template:
    metadata:
      annotations:
        proxy.istio.io/config: |
          proxyMetadata:
            ISTIO_META_DNS_CAPTURE: "true"
    spec:
      containers:
      - name: my-app
        image: my-app:latest
```

You can also use `kubectl` to add the annotation to an existing `Deployment`:

```
kubectl patch deployment my-app -p '{"spec":{"template":
{"metadata":{"annotations":{"proxy.istio.io/config":"proxyMetadata:\n
ISTIO_META_DNS_CAPTURE: \"true\"\n"}}}}}'
```

To verify the annotation is applied, run:

```
kubectl get deployment my-app -o
jsonpath='{.spec.template.metadata.annotations.proxy\.istio\.io/config}'
```

Auto-Allocation of Virtual IPs

The DNS proxy additionally supports automatically allocating addresses for `ServiceEntry` resources that do not explicitly define one. When enabled, the DNS response includes a distinct and automatically assigned address for each `ServiceEntry` from the reserved Class E range (`240.240.0.0/16`). The proxy is then configured to match requests to this IP address and forward the request to the corresponding `ServiceEntry`.

See the following example `ServiceEntry`:

```
apiVersion: networking.istio.io/v1
```

```
kind: ServiceEntry
metadata:
  name: external-db
spec:
  hosts:
  - db.example.com
  ports:
  - number: 3306
    name: tcp
    protocol: TCP
  resolution: DNS
```

Results in DNS queries for `db.example.com` return an auto-allocated IP like `240.240.0.1` instead of the actual external IP. The proxy then routes traffic for `240.240.0.1:3306` to the resolved backend.

To opt out of auto-allocation for a specific `ServiceEntry`, add the following label:

```
metadata:
  labels:
    networking.istio.io/enable-autoallocate-ip: "false"
```

You can also use `kubectl` to add this label:

```
kubectl label serviceentry external-db networking.istio.io/enable-autoallocate-
ip=false
```

`NOTE:` Auto-allocation does not work for wildcard hosts (for example, `*.example.com`).

Consequences

Benefits

The benefits of DNS proxying include:

- `Performance`: Reduced DNS query latency and lower load on upstream DNS.
- `ServiceEntry` support: Applications can resolve hostnames defined in `ServiceEntry` resources.
- TCP routing: Multiple external TCP services on the same port work correctly with auto-allocated VIPs.
- Mesh visibility: Istio gains visibility and control over DNS resolution.

Considerations

Consider the following aspects when using DNS proxying:

- Non-routable IPs: Auto-allocated addresses use the `240.240.0.0/16` range. Applications that validate or log IP addresses may see unexpected values.
- Proxy complexity: The proxy takes on DNS resolution responsibilities, slightly increasing resource usage.
- No wildcard support: Auto-allocation does not apply to `ServiceEntry` resources with wildcard hosts.

Related Information

For more details, see the following resources:

- Istio DNS Proxy documentation
- DNS Configuration in Istio
- Egress Traffic Control

4.3.1.1.2 Expose Workloads Using Gateway API

Use Gateway API to expose your workload.

Prerequisites

- You have the Istio module added. See Adding and Deleting a Kyma Module [page 3511].
- You must install kubectl and curl .
- You have a deployed workload.

> ⚠ Caution
>
> Exposing an unsecured workload to the outside world is a potential security vulnerability, so be careful. If you want to use this example in a production environment, make sure to secure your workload.

Procedure

1. Export the following values as environment variables:

```
export NAMESAPCE={service-namespace}
export BACKENDNAME={service-name}
export PORT={service-port}
```

Option	Description
NAMESAPCE	The name of the backend service.
BACKENDNAME	The name of the backend service that you want to use for routing the incoming HTTP traffic.
PORT	The port number of the backend server to which requests should be forwarded.

2. Install Gateway API CustomResourceDefinitions (CRDs) from the standard channel.

```
kubectl get crd gateways.gateway.networking.k8s.io &> /dev/null || \
{ kubectl kustomize "github.com/kubernetes-sigs/gateway-api/config/crd?
ref=v1.1.0" | kubectl apply -f -; }
```

> ### Note
>
> If you've already installed Gateway API CRDs from the experimental channel, you must delete them before installing Gateway API CRDs from the standard channel.

3. Create a Kubernetes Gateway to deploy Istio Ingress Gateway.

```
cat <<EOF | kubectl apply -f -
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: gateway
  namespace: ${NAMESPACE}
spec:
  gatewayClassName: istio
  listeners:
  - name: http
    hostname: "your-domain.kyma.example.com"
    port: 80
    protocol: HTTP
    allowedRoutes:
      namespaces:
        from: Same
EOF
```

   This command deploys the Istio Ingress service in your namespace with the corresponding Kubernetes Service of type `LoadBalanced` and an assigned external IP address.

4. Create an HTTPRoute to configure access to your workload:

```
cat <<EOF | kubectl apply -f -
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: httproute
  namespace: ${NAMESPACE}
spec:
  parentRefs:
  - name: gateway
  hostnames: ["your-domain.kyma.example.com"]
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /headers
    backendRefs:
    - name: ${BACKENDNAME}
      namespace: ${NAMESPACE}
      port: ${PORT}
EOF
```

Results

You've exposed your workload. To access it, follow the steps:

1. Discover Istio Ingress Gateway's IP and port.

```
export INGRESS_HOST=$(kubectl get gtw gateway -n $NAMESPACE -o
jsonpath='{.status.addresses[0].value}')
```

```
export INGRESS_PORT=$(kubectl get gtw gateway -n $NAMESPACE -o
jsonpath='{.spec.listeners[?(@.name=="http")].port}')
```

2. Call the service.

```
curl -s -I -HHost:your-domain.kyma.example.com "http://
$INGRESS_HOST:$INGRESS_PORT/headers"
```

> ⓘ Note
>
> This task assumes there's no DNS setup for the `httpbin.kyma.example.com` host, so the call contains the host header.

4.3.1.1.3 Expose Workloads Using oauth2-proxy

Learn how to configure oauth2-proxy external authorization provider in the Istio custom resource (CR).

Prerequisites

- You have the Istio module added.
- You have set up your custom domain. See Configure a TLS Gateway [page 1948]. Alternatively, you can use the default domain of your Kyma cluster and the default Gateway `kyma-system/kyma-gateway`. To use a Kyma domain to expose a workload, the API Gateway module must be added.

  > ⓘ Note
  >
  > Because the default Kyma domain is a wildcard domain, which uses a simple TLS Gateway, it is recommended that you set up your custom domain for use in a production environment.

  > → Tip
  >
  > To learn what the default domain of your Kyma cluster is, run the following command:
  >
  > ```
  > kubectl get gateway -n kyma-system kyma-gateway -o
  > jsonpath='{.spec.servers[0].hosts}'
  > ```

- You must install kubectl , curl , and Helm .
- You have a deployed workload.

Context

The Istio CR allows configuring external authorization providers that operate over HTTP. To set up the oauth2-proxy external authorization provider in the Istio CR and expose your workload using an Istio VirtualService, follow these steps.

Procedure

1. Export the following values as environment variables:

```
export WORKLOAD_NAME={WORKLOAD_NAME}
export NAMESPACE={WORKLOAD_NAMESPACE}
export DOMAIN_TO_EXPOSE_WORKLOADS={YOUR_DOMAIN}
export GATEWAY={YOUR_GATEWAY}
```

Option	Description
WORKLOAD_NAME	The name of your workload.
NAMESPACE	The name of your workload's namespace.
DOMAIN_TO_EXPOSE_WORKLOADS	The domain that should be used to expose the workloads.
GATEWAY	The Gateway that the VirtualService should use to route traffic to the workload.

2. Create a VirtualService to expose the workload:

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: ext-authz
  namespace: $NAMESPACE
spec:
  hosts:
  - "$WORKLOAD_NAME.$DOMAIN_TO_EXPOSE_WORKLOADS"
  gateways:
  - $GATEWAY
  http:
  - match:
    - uri:
        prefix: /
    route:
    - destination:
        port:
          number: 80
        host: $WORKLOAD_NAME.$NAMESPACE.svc.cluster.local
EOF
```

3. Export the following configuration values as environment variables:

```
export CLIENT_ID={YOUR_CLIENT_ID}
export CLIENT_SECRET={YOUR_CLIENT_SECRET}
export COOKIE_SECRET={YOUR_COOKIE_SECRET}
export OIDC_ISSUER_URL={YOUR_OIDC_ISSUER_URL}
```

Option	Description
CLIENT_ID	The unique identifier for the client application that is registered with the external authorizer.
CLIENT_SECRET	A secret key known only to the client and the external authorizer. It is used to authenticate the client when communicating with the external authorizer. To generate this value, you can use the command `openssl rand -base64 32 | head -c 32 | base64`.
CLIENT_COOKIE	A secret key used to sign and encrypt the cookies that are used for session management and user authentication. To generate this value, you can use the command `openssl rand -base64 32 | head -c 32 | base64`.

Option	Description
OIDC_ISSUER_URL	This is the URL of the OpenID Connect (OIDC) issuer. Typically, you can find the issuer at `https://{YOUR_IDENTITY_PROVIDER_INSTANCE}/.well-known/openid-configuration`.

4. Create a `values.yaml` file with the oauth2-proxy configuration for your authorization server:

> → Tip
>
> You can adapt this configuration to better suit your needs. See the additional configuration parameters .

```
cat <<EOF > values.yaml
config:
  clientID: $CLIENT_ID
  clientSecret: $CLIENT_SECRET
  cookieSecret: $COOKIE_SECRET
extraArgs:
  provider: oidc
  cookie-secure: false
  cookie-domain: "$DOMAIN_TO_EXPOSE_WORKLOADS"
  cookie-samesite: lax
  set-xauthrequest: true
  whitelist-domain: "*.$DOMAIN_TO_EXPOSE_WORKLOADS:*"
  reverse-proxy: true
  pass-access-token: true
  set-authorization-header: true
  pass-authorization-header: true
  scope: "openid email"
  upstream: static://200
  skip-provider-button: true
  redirect-url: "https://oauth2-proxy.$DOMAIN_TO_EXPOSE_WORKLOADS/oauth2/
callback"
  oidc-issuer-url: https://$OIDC_ISSUER_URL
  code-challenge-method: S256
EOF
```

5. To install oauth2-proxy with the defined configuration, use oauth2-proxy helm chart :

```
helm repo add oauth2-proxy https://oauth2-proxy.github.io/manifests
helm install custom oauth2-proxy/oauth2-proxy -f values.yaml
```

6. Register oauth2-proxy as an authorization provider in the Istio module:

```
cat <<EOF | kubectl apply -f -
apiVersion: operator.kyma-project.io/v1alpha1
kind: Istio
metadata:
  name: default
  namespace: kyma-system
spec:
  config:
    numTrustedProxies: 1
    authorizers:
    - name: "oauth2-proxy"
      service: "custom-oauth2-proxy.$NAMESPACE.svc.cluster.local"
      port: 80
      headers:
        inCheck:
          include: ["authorization", "cookie"]
          add:
            x-auth-request-redirect: "https://%REQ(:authority)%%REQ(x-envoy-
original-path?:path)%"
```

```
        toUpstream:
          onAllow: ["authorization", "path", "x-auth-request-user", "x-auth-
request-email", "x-auth-request-access-token"]
        toDownstream:
          onAllow: ["set-cookie"]
          onDeny: ["content-type", "set-cookie"]
EOF
```

7. Create an AuthorizationPolicy CR with the `CUSTOM` action and the `oauth2-proxy` provider:

```
cat <<EOF | kubectl apply -f -
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
    name: ext-authz
    namespace: test
spec:
  action: CUSTOM
  provider:
    name: oauth2-proxy
  rules:
  - to:
    - operation:
        paths:
        - /headers
  selector:
    matchLabels:
      app: httpbin
EOF
```

8. Create a `DestinationRule` resource with a traffic policy for the external authorization provider:

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: external-authz-https
  namespace: istio-system
spec:
  host: $OIDC_ISSUER_URL
  trafficPolicy:
    tls:
      mode: SIMPLE
EOF
```

Results

When you access the URL of the exposed service, you are redirected to the authorization provider's page.

4.3.1.1.4 Expose Workloads with Istio VirtualService

Learn how to expose a workload with Istio VirtualService.

Prerequisites

- You have the Istio module added to your Kyma runtime instance.
- You have a deployed workload.
- To set up a custom Gateway, see Configure a TLS Gateway [page 1948]. Alternatively, you can use the default domain of your Kyma cluster and the default Gateway `kyma-system/kyma-gateway`.

> ⚠ Caution
>
> Because the default Kyma domain is a wildcard domain, which uses a simple TLS Gateway, it is recommended that you set up your custom domain for use in a production environment.

> → Tip
>
> To learn what the default domain of your Kyma cluster is, run the following command:
>
> ```
> kubectl get gateway -n kyma-system kyma-gateway -o
> jsonpath='{.spec.servers[0].hosts}'
> ```

Context

Kyma's API Gateway module provides the APIRule custom resource (CR), which is the recommended solution for securely exposing workloads. To expose a workload using an APIRule `v2`, you must include the workload in the Istio service mesh. Including a workload in the mesh brings several benefits, such as secure service-to-service communication, tracing capabilities, or traffic management. For more information, see Purpose and Benefits of Istio Sidecar Proxies and The Istio service mesh .

However, if you do not need the capabilities provided by the Istio service mesh, you can expose an unsecured workload using Istio VirtualService only. Such an approach might be useful in the following scenarios:

- If you use Unified Gateway as an entry point for SAP Cloud solutions. In this case, you can configure Unified Gateway to manage API exposure, JWT validation, and routing capabilities, offloading these responsibilities from the service mesh.
- If you want to expose front-end Services that manage their own authentication mechanisms.
- If you require certain features or want to implement specific configurations that APIRule does not support.
- If you require full control over Istio resources and you want to manage them directly without any higher-level abstractions.

The following instructions demonstrate a simple use case where VirtualService exposes an unsecured Service, skipping the requirement to include the Service in the Istio service mesh.

Procedure

- Use Kyma dashbaord
  a. Go to the namespace in which you want to create a VirtualService resource.
  b. Go to *Istio > Virtual Services*.
  c. Choose *Create* and provide the following configuration details:

Option	Description
`Name`	The name of the VirtualService resource you're creating.
`Hosts`	The address or addresses the client uses when sending requests to the destination Service. Use the fully qualified domain name (FQDN) constructed with the subdomain and domain name in the following format: {SUBDOMAIN}.{DOMAIN_NAME}. The hosts must be defined in the referenced Gateway.
`Gateways`	The name of the Gateway you want to use and the namespace in which it is created. Use the format {GATEWAY_NAMESPACE}/{GATEWAY_NAME}.

  d. Go to *HTTP > Matches > Match* and provide the following configuration details:

Option	Description
`URI`	Add the URI prefix used to match incoming requests. This determines which requests are routed to the specified destination Service.
`Port`	This is the port number on which the destination Service is listening. It specifies where the VirtualService routes the incoming traffic. If a Service exposes only a single port it is not required to explicitly select the port.

  e. Go to *HTTP > Routes > Route > Destinations > Host* and add the name and namespace of the Service you want to expose using the following format: {SERVICE_NAME}.{SERVICE_NAMESPACE}.svc.cluster.local. The traffic is routed to this Service.

  For more configuration options, see Virtual Service.

{·} Sample Code

See a sample VirtualService configuration that directs all HTTP traffic received at `httpbin.my-domain.com` through the my-gateway Gateway to the HTTPBin Service, which is running on port 8000 in the default namespace.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: example-vs
  namespace: default
spec:
```

```
  hosts:
    - httpbin.my-domain.com
  gateways:
    - default/my-gateway
  http:
  - match:
    - uri:
        prefix: /
    route:
    - destination:
        port:
          number: 8000
        host: httpbin.default.svc.cluster.local
```

f. To verify if the Service is exposed, run the following command:

```
curl -s -I https://{SUBDOMAIN}.{DOMAIN_NAME}/
```

If successful, you get code `200` in response.

- Use kubectl

a. To expose a workload using VirtualService, replace the placeholders and run the following command:

Option	Description
`{NAME}`	The name of the VirtualService resource you're creating.
`{NAMESPACE}`	The namespace in which you want to create the VirtualService resource.
`{SUBDOMAIN}.{DOMAIN_NAME}`	The address or addresses the client uses when sending requests to the destination Service. Use the fully qualified domain name (FQDN) constructed with the subdomain and domain name in the following format: {SUBDOMAIN}.{DOMAIN_NAME}. The hosts must be defined in the referenced Gateway.
`{GATEWAY_NAMESPACE}/{GATEWAY_NAME}`	The name of the Gateway you want to use and the namespace in which it is created.
`{URI_PREFIX}`	The URI prefix used to match incoming requests. This determines which requests are routed to the specified destination Service.
`{PORT_NUMBER}`	This is the port number on which the destination Service is listening. It specifies where the VirtualService routes incoming requests. If a Service exposes only a single port it is not required to explicitly select the port.
`{SERVICE_NAME}.{SERVICE_NAMESPACE}`	The name and namespace of the Service you want to expose. The requests are routed to this Service.

For more configuration options, see Virtual Service.

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.istio.io/v1
kind: VirtualService
```

```
metadata:
  name: {VS_NAME}
  namespace: {VS_NAMESPACE}
spec:
  hosts:
    - {SUBDOMAIN}.{DOMAIN_NAME}
  gateways:
    - {GATEWAY_NAMESPACE}/{GATEWAY_NAME}
  http:
  - match:
    - uri:
        prefix: {URI_PREFIX}
    route:
    - destination:
        port:
          number: {PORT_NUMBER}
        host: {SERVICE_NAME}.{SERVICE_NAMESPACE}.svc.cluster.local
```

Sample Code

See a sample VirtualService configuration that directs all HTTP traffic received at `httpbin.my-domain.com` through the my-gateway Gateway to the HTTPBin Service, which is running on port 8000 in the default namespace.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: example-vs
  namespace: default
spec:
  hosts:
    - httpbin.my-domain.com
  gateways:
    - default/my-gateway
  http:
  - match:
    - uri:
        prefix: /
    route:
    - destination:
        port:
          number: 8000
        host: httpbin.default.svc.cluster.local
```

b. To verify if the Service is exposed, run the following command:

```
curl -s -I https://{SUBDOMAIN}.{DOMAIN_NAME}/
```

If successful, you get code `200` in response.

4.3.1.1.5 Configure Observability for the Istio Service Mesh

Learn how the Telemetry module collects logs, traces, and metrics from the Istio service mesh, and how you can configure each telemetry signal for your workloads.

The Istio service mesh generates detailed telemetry data, including access logs, traces, and metrics. The Telemetry module collects, processes, and exports this data, so you can analyze your mesh's traffic, performance, and behavior.

When you have both the Istio and Telemetry modules in your cluster, they integrate automatically. The Telemetry module's components are injected with an Istio sidecar, which adds them to the service mesh. This enables secure, mTLS-encrypted communication for all telemetry data by default. For details, see Istio Integration [page 2263].

To monitor your service mesh, configure the Telemetry module to collect the specific signals you need:

- To monitor traffic details like latency, traffic volume, and errors, configure Istio to send access logs (see Configure Istio Access Logs [page 2197]).
- To get an end-to-end view of requests, configure Istio to send trace data (see Configure Istio Tracing [page 2201]).
- To monitor the health and performance of your service mesh, enable the `istio` input in the Telemetry module. This scrapes metrics directly from Istio proxies (sidecars) and the control plan (see Collect Istio Metrics [page 2209]).

4.3.1.1.6 Manage Network Policies

Network policy support in the Istio module is disabled by default. Learn how to enable the network policy support and allow egress traffic to your workloads.

Context

To support secure-by-default configurations, the Istio module can create network policies in the `istio-system` and `kyma-system` namespaces. These policies restrict traffic to and from Istio components so that only the required baseline communication is allowed. This helps ensure that the Istio module's components continue to function even when a deny-by-default policy is applied at the cluster or namespace level.

All module-managed policies use the following labels:

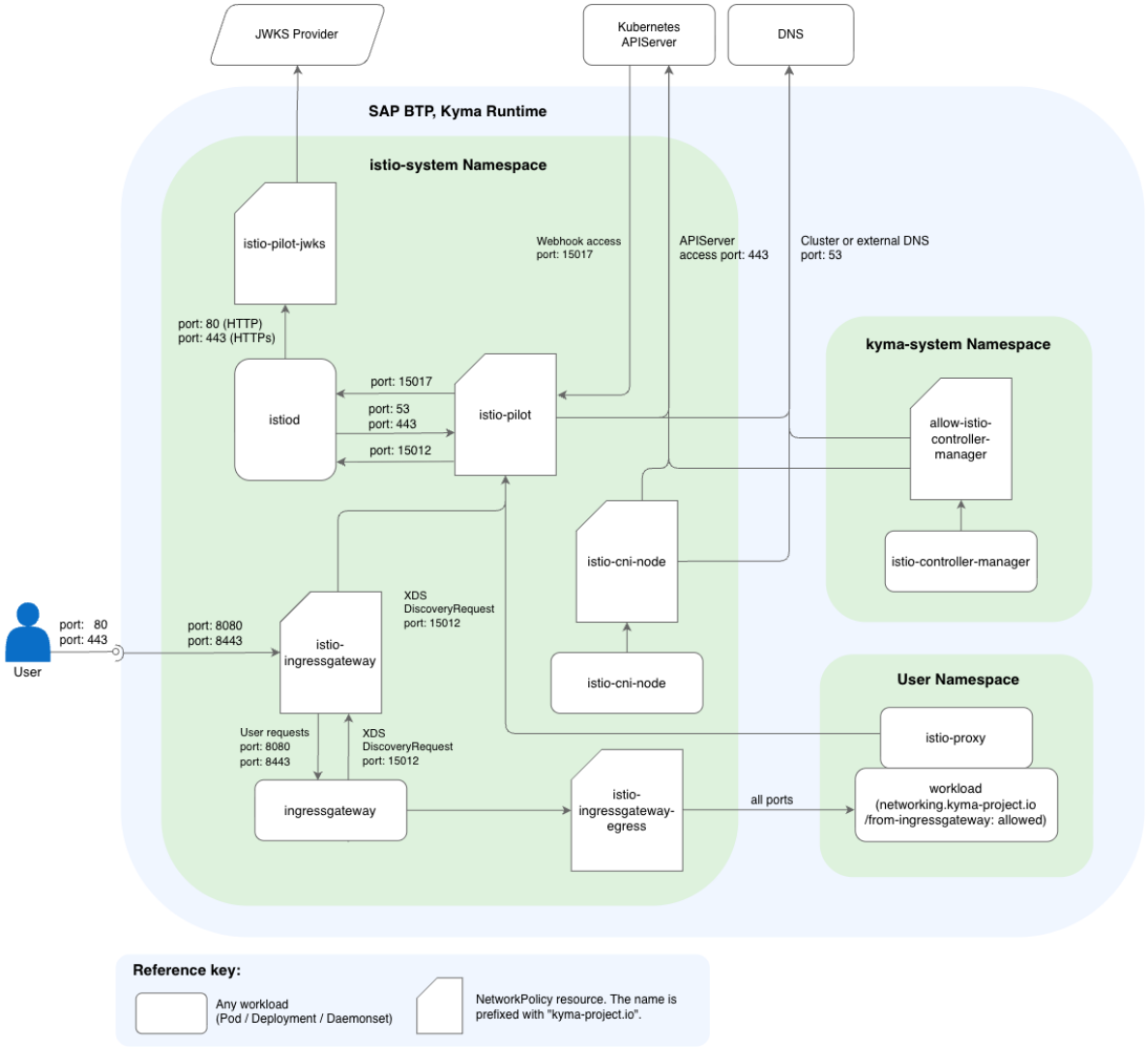
- `kyma-project.io/module: istio`
- `kyma-project.io/managed-by: kyma`

Do not modify these resources manually. The module updates them automatically and overwrites any manual changes.

Networking Diagram

The following diagram illustrates the allowed traffic flows between Istio components and user workloads when network policy support is enabled.

In the diagram, network policies appear as resources that traffic flows through. In practice, a network policy is a custom resource that defines which traffic is allowed or denied, while the Istio module's components perform the actual filtering.



List of Network Policies

See the list of network policies that the Istio module creates when network policy support is enabled.

Show table

Component	Namespace	Port	Protocol	Direction	Source/Destination	Purpose
`istio-controller-manager`	`kyma-system`	53	UDP/TCP	Egress	destination: *	DNS resolution
`istio-controller-manager`	`kyma-system`	443	TCP	Egress	destination: *	Kubernetes API server access
`istiod`	`istio-system`	53	UDP/TCP	Egress	destination: *	DNS resolution

Component	Namespace	Port	Protocol	Direction	Source/Destination	Purpose
`istiod`	`istio-system`	`80`	TCP	Egress	destination: *	Access to external JWKS endpoints for JWT validation (HTTP)
`istiod`	`istio-system`	`443`	TCP	Egress	destination: *	Access to external JWKS endpoints for JWT validation (HTTPS) / Kubernetes API server access
`istiod`	`istio-system`	`15012`	TCP/gRPC	Ingress	source (any of): podSelector `security.istio.io/tlsMode: istio` (any namespace); podSelector `istio: ingressgateway`	XDS config distribution to sidecars and gateways
`istiod`	`istio-system`	`15014`	TCP/HTTP	Ingress	source (any of): has label `kyma-project.io/module` (any namespace); podSelector `networking.kyma-project.io/istio-metrics: allowed` (any namespace); podSelector `networking.kyma-project.io/metrics-scraping: allowed` (any namespace)	Control plane metrics (Prometheus scrape)

Component	Namespace	Port	Protocol	Direction	Source/Destination	Purpose
`istiod`	`istio-system`	`15017`	TCP/HTTPS	Ingress	source: *	Webhook endpoint (defaulting/mutation/admission). It is generally only accessed by the Kubernetes API server.
`istio-egressgateway`	`istio-system`	*	UDP/TCP	Egress	destination: *	All outbound traffic from egress is allowed, as the configuration is done via Istio resources
`istio-egressgateway`	`istio-system`	*	UDP/TCP	Ingress	source: podSelector `networking.kyma-project.io/to-egressgateway: allowed` (any namespace)	Traffic from labeled user Pods (`networking.kyma-project.io/to-egressgateway: allowed`)
`istio-egressgateway`	`istio-system`	`15020`	TCP/HTTP	Ingress	source (any of): has label `kyma-project.io/module` (any namespace); podSelector `networking.kyma-project.io/istio-metrics: allowed` (any namespace); podSelector `networking.kyma-project.io/metrics-scraping: allowed` (any namespace)	Merged Prometheus metrics

Component	Namespace	Port	Protocol	Direction	Source/Destination	Purpose
istio-egressgateway	istio-system	15021	TCP/HTTP	Ingress	source (any of): has label kyma-project.io/module (any namespace); podSelector networking.kyma-project.io/istio-metrics: allowed (any namespace); podSelector networking.kyma-project.io/metrics-scraping: allowed (any namespace)	Health check endpoint
istio-egressgateway	istio-system	15090	TCP/HTTP	Ingress	source (any of): has label kyma-project.io/module (any namespace); podSelector networking.kyma-project.io/istio-metrics: allowed (any namespace); podSelector networking.kyma-project.io/metrics-scraping: allowed (any namespace)	Envoy Prometheus telemetry

Component	Namespace	Port	Protocol	Direction	Source/Destination	Purpose
`istio-ingressgateway`	`istio-system`	*	TCP	Egress	destination: podSelector `networking.kyma-project.io/from-ingressgateway: allowed` (any namespace)	Traffic to labeled user Pods (`networking.kyma-project.io/from-ingressgateway: allowed`)
`istio-ingressgateway`	`istio-system`	`53`	UDP/TCP	Egress	destination: *	DNS resolution
`istio-ingressgateway`	`istio-system`	`8080`	TCP	Ingress	source: *	HTTP traffic inbound to cluster
`istio-ingressgateway`	`istio-system`	`8443`	TCP	Ingress	source: *	HTTPS traffic inbound to cluster
`istio-ingressgateway`	`istio-system`	`15012`	TCP/gRPC	Egress	destination: podSelector `istio: pilot`	Request XDS config from istiod
`istio-ingressgateway`	`istio-system`	`15020`	TCP/HTTP	Ingress	source (any of): has label `kyma-project.io/module` (any namespace); podSelector `networking.kyma-project.io/istio-metrics: allowed` (any namespace); podSelector `networking.kyma-project.io/metrics-scraping: allowed` (any namespace)	Merged Prometheus metrics

Component	Namespace	Port	Protocol	Direction	Source/Destination	Purpose
`istio-ingressgateway`	`istio-system`	`15021`	TCP/HTTP	Ingress	source (any of): has label `kyma-project.io/module` (any namespace); podSelector `networking.kyma-project.io/istio-metrics: allowed` (any namespace); podSelector `networking.kyma-project.io/metrics-scraping: allowed` (any namespace)	Health check endpoint
`istio-ingressgateway`	`istio-system`	`15090`	TCP/HTTP	Ingress	source (any of): has label `kyma-project.io/module` (any namespace); podSelector `networking.kyma-project.io/istio-metrics: allowed` (any namespace); podSelector `networking.kyma-project.io/metrics-scraping: allowed` (any namespace)	Envoy Prometheus telemetry
`istio-cni-node`	`istio-system`	`53`	UDP/TCP	Egress	destination: *	DNS resolution
`istio-cni-node`	`istio-system`	`443`	TCP	Egress	destination: *	Kubernetes API server access

Procedure

1. To enable support for network policies, set the flag `networkPoliciesEnabled` to true in the `Istio` custom resource. This setting is disabled by default.

   ```
   apiVersion: operator.kyma-project.io/v1alpha2
   kind: Istio
   metadata:
     name: default
     namespace: kyma-system
   spec:
     networkPoliciesEnabled: true
   ```

   When the flag changes, Istio components are restarted, terminating existing TCP connections and enforcing the policies immediately.

2. Enable egress from `istio-ingressgateway` to your workloads.

   The egress traffic from `istio-ingressgateway` to user workloads is restricted by default. To allow egress traffic from `istio-ingressgateway` to your workloads, add the label `networking.kyma-project.io/from-ingressgateway: allowed` to the Pods that should be reachable from `istio-ingressgateway`.

   See the following example workload template snippet:

   ```
   spec:
     template:
       metadata:
         labels:
           networking.kyma-project.io/from-ingressgateway: allowed
   ```

3. Enable egress traffic from your workloads to `istio-egressgateway`.

   If you have `egressgateway` enabled and want to allow traffic from your workloads to `egressgateway`, add the label `networking.kyma-project.io/to-egressgateway: allowed` to the Pods.

   See the following example workload template snippet:

   ```
   spec:
     template:
       metadata:
         labels:
           networking.kyma-project.io/to-egressgateway: allowed
   ```

4. Enable access to metrics and health check endpoints.

   To allow access to the metrics and health check endpoints of `istio-ingressgateway` and `istio-egressgateway`, add either of these labels to the Pods that should be able to access those endpoints:

   - `networking.kyma-project.io/istio-metrics: allowed`
   - `networking.kyma-project.io/metrics-scraping: allowed`

   See the following example workload template snippet:

   ```
   spec:
     template:
       metadata:
         labels:
           networking.kyma-project.io/istio-metrics: allowed
           networking.kyma-project.io/metrics-scraping: allowed
   ```

5. Apply a deny-by-default policy.

   To isolate a workload's namespace with a deny-by-default policy, make sure to allow ingress from `istio-ingressgateway` in that policy. See an example `NetworkPolicy` resource allowing ingress from `istio-ingressgateway` to the workload:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
    name: allow-ingress-from-istio-ingressgateway
    namespace: my-namespace
spec:
    podSelector:
      matchLabels:
        app: my-app
    policyTypes:
    - Ingress
    ingress:
    - from:
      - namespaceSelector:
          matchLabels:
            kubernetes.io/metadata.name: istio-system
        podSelector:
          matchLabels:
            istio: ingressgateway
      ports:
        - protocol: TCP
          port: 8080 # The targetPort of the application container
```

4.3.1.1.7 Troubleshooting for the Istio Module

Troubleshoot problems related to the Istio module.

Network Connectivity Diagnostics [page 1927]
If you're having trouble with network connectivity and don't know where to begin troubleshooting, follow these steps. The issues may not be directly related to Istio, but could be due to misconfigured Istio resources or other cluster resources.

Increase Verbosity of Istio Ingress Logs [page 1929]
To diagnose connection issues, you may need more verbose logs in the Istio Ingress container. Such logs may help determine whether the request reaches the Ingress and what happens to it.

Connection Refused Errors [page 1931]

You Get 403 Forbidden if Host Header Contains Port [page 1932]

Istio Sidecar Proxy Injection Issues [page 1933]

Incompatible Sidecar Version After the Istio Module's Update [page 1936]
Learn which Pods you must restart manually after the Istio module's update.

Pods Created by Jobs Can't Finish [page 1937]

Init Containers Can't Access the Network [page 1938]

Reverting the Istio Module's Deletion [page 1940]
Follow the steps outlined in this troubleshooting guide if you unintentionally deleted the Istio module and want to restore the cluster to its normal state without losing any resources created in the cluster.

SAP HANA Database Connection Issues [page 1941]

4.3.1.1.7.1 Network Connectivity Diagnostics

If you're having trouble with network connectivity and don't know where to begin troubleshooting, follow these steps. The issues may not be directly related to Istio, but could be due to misconfigured Istio resources or other cluster resources.

Before starting the detailed analysis, perform these initial checks:

- Verify the state of the `Istio` CR. If it is in the `Warning` state, check the warning message and conditions. It might help you begin the investigation.
- Verify that no NetworkPolicies are affecting the connectivity by blocking traffic between Pods in the service mesh. To find all `NetworkPolicy` resources, run the command `kubectl get networkpolicies -A`.
- The configuration of the following kinds of resources can affect the connectivity in the service mesh. Verify that those resources are configured as intended:
  - `DestinationRule`
  - `PeerAuthentication`
  - `Gateway`
  - `AuthorizationPolicy`
  - `RequestAuthentication`

1. Global Analysis

As a starting point, run `istioctl analyze` to check the state of your mesh and point out common errors and misconfigurations.

Analyze a specific namespace:

```
istioctl analyze -n <your-namespace>
```

Analyze the entire cluster:

```
istioctl analyze --all-namespaces
```

What to look for:

- **Errors and Warnings:** Review all output carefully, especially warnings like `IST0133`, which indicate an `AuthorizationPolicy` might be blocking more traffic than intended. The analysis often includes the name and namespace of the problematic resource.

2. AuthorizationPolicies

Misconfigured `AuthorizationPolicy` resources are a common cause of access issues, such as `403` errors or unexpected connection timeouts.

List all `AuthorizationPolicy` resources in a namespace:

```
kubectl get authorizationpolicies -n <your-namespace>
```

List all `AuthorizationPolicy` resources cluster-wide:

```
kubectl get authorizationpolicies --all-namespaces
```

Describe a specific `AuthorizationPolicy` to see its details:

```
kubectl describe authorizationpolicy <policy-name> -n <namespace>
```

What to look for:

- **Errors and Warnings**: Deny policies may block more traffic than intended. A deny rule without the `ports` field on an HTTP-based `hosts` rule can block all TCP traffic, not just HTTP.
- **Allow policies**: If even a single allow policy matches a workload, all traffic that does not match any allow rule is denied. Make sure that all intended traffic is covered by allow rules.
- **Scope**: Check the `selector` to see which workloads the policy applies to. An empty selector applies the policy to all workloads in the namespace.
- **Rules**: Look at the `to`, `from`, and `when` clauses to understand what traffic is being allowed or denied.

3. RequestAuthentications

`RequestAuthentication` defines how JSON Web Tokens (JWTs) are validated. Misconfigurations often lead to `401 Unauthorized` responses.

List all request authentications in a namespace:

```
kubectl get requestauthentications -n <your-namespace>
```

Describe a specific `RequestAuthentication` to see its configuration:

```
kubectl describe requestauthentication <ra-name> -n <namespace>
```

What to look for:

- `jwksUri`: Ensure the JSON Web Key Set (JWKS) endpoint is correct and reachable from Istiod.
- `issuer`: The `iss` claim in the JWT must match this value.
- `audiences`: In case those are set, the `aud` claim in the JWT must match one of the values here.

4. EnvoyFilters

`EnvoyFilter` resources modify the Envoy configuration and can easily cause problems if misconfigured. They are often used for complex use cases.

List all `EnvoyFilter` resources cluster-wide:

```
kubectl get envoyfilters --all-namespaces
```

Examine a specific filter:

```
kubectl get envoyfilter <filter-name> -n <namespace> -o yaml
```

What to look for:

- **Workload selector**: Check which Pods the filter applies to.
- **Filters**: Look at the filter configuration. These are often complex Lua scripts or WASM filters. A bug in the script can break traffic.

5. Envoy Configuration

If the previous steps don't reveal the issue, inspect the live Envoy configuration for a specific Pod. This shows exactly which rules Envoy applies.

Get the proxy config for a specific Pod:

```
istioctl proxy-config <all|listeners|clusters|routes|endpoints|bootstrap> <pod-
name> -n <namespace>
```

For example, check the routes for a Pod:

```
istioctl proxy-config routes <pod-name>.<namespace> -o json
```

4.3.1.1.7.2 Increase Verbosity of Istio Ingress Logs

To diagnose connection issues, you may need more verbose logs in the Istio Ingress container. Such logs may help determine whether the request reaches the Ingress and what happens to it.

Enable Access Logs

An access log is the main observability feature. It contains a detailed log entry for every request that is processed.

With Istio, you can enable access logging by the `Telemetry` custom resource (CR). To enable access logging for the Istio Ingress Gateway, run:

```
cat <<EOF | kubectl apply -f -
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: ingress-access-logs-enabler
  namespace: istio-system
spec:
  selector:
    matchLabels:
      istio: ingressgateway
  accessLogging:
```

```
    - providers:
      - name: envoy
EOF
```

As a result, the Ingress log contains access logs.

Example: When you run the following command:

```
curl -k --tls-max 1.0 https://httpbin.local.kyma.dev/headers
```

The Ingress log contains the following entry:

```
{"authority":"httpbin.local.kyma.dev","bytes_received":0,"bytes_sent":442,"connec
tion_termination_details":null,"downstream_local_address":"10.42.0.9:8443","downs
tream_remote_address":"10.42.0.1:22564","duration":15,"method":"GET","path":"/
headers","protocol":"HTTP/2","request_id":"b947e4b6-2965-4cca-
af90-51e3a1f4673f","requested_server_name":"httpbin.local.kyma.dev","response_cod
e":200,"response_code_details":"via_upstream","response_flags":"-","route_name":n
ull,"start_time":"2026-06-30T09:09:58.374Z","upstream_cluster":"outbound|8000||
httpbin.test.svc.cluster.local","upstream_host":"10.42.0.13:80","upstream_local_a
ddress":"10.42.0.9:35226","upstream_service_time":"13","upstream_transport_failur
e_reason":null,"user_agent":"curl/8.7.1","x_forwarded_for":"10.42.0.1"}
```

Enable Trace Logs

If the access log is insufficient, increase the log level of the Istio Ingress. You can do it at runtime, so it doesn't require redeployment.

```
for ingress_pod in $(kubectl get pod -n istio-system --no-headers --output=name
-l app=istio-ingressgateway); do
    istioctl proxy-config log -n istio-system "${ingress_pod}" --level trace
done
```

The effect is immediate, and the Ingress log provides detailed information for every connection. It is very useful for diagnosing problems in lower layers, such as (m)TLS.

Example: The following request fails because the TLS version is too old:

```
curl -k --tls-max 1.0 https://httpbin.local.kyma.dev/headers
```

Such a request is not visible in the access log because the connection has not been established, and the HTTP protocol has not been involved yet. The Ingress log shows the `UNSUPPORTED_PROTOCOL` error.

```
2026-07-01T06:17:13.289504Z    trace    envoy filter external/envoy/source/
extensions/filters/listener/tls_inspector/tls_inspector.cc:107    tls inspector:
new connection accepted    thread=20
2026-07-01T06:17:13.289609Z    trace    envoy misc external/envoy/source/common/
network/tcp_listener_impl.cc:123    TcpListener accepted 1 new connections.
thread=20
2026-07-01T06:17:13.289623Z    trace    envoy filter external/envoy/source/
common/network/listener_filter_buffer_impl.cc:95    onFileEvent: 1    thread=20
2026-07-01T06:17:13.289637Z    trace    envoy filter external/envoy/source/
common/network/listener_filter_buffer_impl.cc:60    recv returned: 165
thread=20
2026-07-01T06:17:13.289643Z    trace    envoy filter external/envoy/source/
extensions/filters/listener/tls_inspector/tls_inspector.cc:147    tls inspector:
recv: 165    thread=20
```

```
2026-07-01T06:17:13.289676Z    trace    envoy filter external/envoy/source/
extensions/filters/listener/tls_inspector/tls_inspector.cc:129    tls:onALPN(),
ALPN: h2,http/1.1    thread=20
2026-07-01T06:17:13.289688Z    debug    envoy filter external/
envoy/source/extensions/filters/listener/tls_inspector/tls_inspector.cc:138
tls:onServerName(), requestedServerName: httpbin.local.kyma.dev    thread=20
2026-07-01T06:17:13.289797Z    trace    envoy misc external/envoy/source/
common/event/scaled_range_timer_manager_impl.cc:60    enableTimer called on
0x32633f9a7180 for 15000ms, min is 15000ms    thread=20
2026-07-01T06:17:13.289848Z    trace    envoy misc external/envoy/source/
common/event/scaled_range_timer_manager_impl.cc:60    enableTimer called on
0x32633f9a7080 for 3600000ms, min is 3600000ms    thread=20
2026-07-01T06:17:13.289880Z    debug    envoy conn_handler external/
envoy/source/common/listener_manager/active_tcp_listener.cc:162    [Tags:
"ConnectionId":"388"] new connection from 10.42.0.1:6943    thread=20
2026-07-01T06:17:13.289886Z    trace    envoy main external/envoy/source/common/
event/dispatcher_impl.cc:249    item added to deferred deletion list (size=1)
thread=20
2026-07-01T06:17:13.289889Z    trace    envoy main external/envoy/source/common/
event/dispatcher_impl.cc:123    clearing deferred deletion list (size=1)
thread=20
2026-07-01T06:17:13.289897Z    trace    envoy connection external/envoy/source/
common/network/connection_impl.cc:662    [Tags: "ConnectionId":"388"] socket
event: 3    thread=20
2026-07-01T06:17:13.289905Z    trace    envoy connection external/envoy/source/
common/network/connection_impl.cc:808    [Tags: "ConnectionId":"388"] write
ready    thread=20
2026-07-01T06:17:13.289918Z    trace    envoy config external/envoy/source/
common/tls/server_context_impl.cc:511    TLS context selection result: 0, before
selectTlsContext    thread=20
2026-07-01T06:17:13.289928Z    trace    envoy config external/envoy/source/
common/tls/server_context_impl.cc:519    TLS context selection result: 1, after
selectTlsContext, selection result status: 0    thread=20
2026-07-01T06:17:13.290005Z    trace    envoy connection external/envoy/source/
common/tls/ssl_handshaker.cc:152    [Tags: "ConnectionId":"388"] ssl error
occurred while read: SSL    thread=20
2026-07-01T06:17:13.290020Z    debug    envoy connection
external/envoy/source/common/tls/ssl_socket.cc:269    [Tags:
"ConnectionId":"388"] remote address:10.42.0.1:6943,TLS_error:|268435696:SSL
routines:OPENSSL_internal:UNSUPPORTED_PROTOCOL:TLS_error_end    thread=20
2026-07-01T06:17:13.290025Z    debug    envoy connection external/envoy/source/
common/network/connection_impl.cc:313    [Tags: "ConnectionId":"388"] closing
socket: 0    thread=20
2026-07-01T06:17:13.290068Z    trace    envoy connection external/envoy/source/
common/network/connection_impl.cc:517    [Tags: "ConnectionId":"388"] raising
connection event 0    thread=20
```

> ⚠ Caution
>
> Such verbose logs may influence the Ingress performance, so switch it back to a lower level after diagnosis is completed. By default, the Ingress uses a warn level.

4.3.1.1.7.3 Connection Refused Errors

Symptom

You get either the `Connection reset by peer` response or the `GOAWAY` response when you attempt to establish the connection between a service without a sidecar and a service with a sidecar.

Cause

By default, mutual TLS (mTLS) is enabled in the service mesh. As a result, every element of the service mesh must have an Istio sidecar with a valid TLS certificate to allow communication.

Solution

- To add a service without a sidecar to the allowlist and disable mTLS traffic for it, create a `DestinationRule` resource. See DestinationRule .
- To allow connections between a service without a sidecar and a service with a sidecar, create a `PeerAuthentication` resource in the PERMISSIVE mode. See PeerAuthentication .

4.3.1.1.7.4 You Get 403 Forbidden if Host Header Contains Port

Symptom

When you try to access a Kyma endpoint protected by an `AuthorizationPolicy` allowing a given hostname, it reports a `403 Forbidden` error.

Cause

The error might be caused by the unnecessary port number in the `Host` header. Istio checks the host as-is, so if the `Host` header contains a port number and the `AuthorizationPolicy` defines only a hostname, the request is denied.

Example:

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: ingress-allow-headers
  namespace: istio-system
spec:
  action: ALLOW
  rules:
  - to:
    - operation:
        hosts: [ "httpbin.local.kyma.dev" ]
        methods: ["GET"]
        paths: ["/headers"]
  selector:
    matchLabels:
      app: istio-ingressgateway
```

```
curl -k -H "Host: httpbin.local.kyma.dev:443" https://httpbin.local.kyma.dev/
headers
```

```
RBAC: access denied
```

Solution

The RFC 9110 and RFC 3986 documents define that HTTP clients should remove the port if it is the default port for a given protocol. The general recommendation is to fix the client implementation.

If this solution cannot be implemented, the workaround is to modify the `AuthorizationPolicy` to also include the port number.

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: ingress-allow-headers
  namespace: istio-system
spec:
  action: ALLOW
  rules:
  - to:
    - operation:
        hosts: [ "httpbin.local.kyma.dev", "httpbin.local.kyma.dev:443" ]
        methods: ["GET"]
        paths: ["/headers"]
  selector:
    matchLabels:
      app: istio-ingressgateway
```

4.3.1.1.7.5 Istio Sidecar Proxy Injection Issues

Symptom

Some Pods don't have an Istio sidecar proxy injected.

Cause

By default, the Istio module does not automatically inject an Istio sidecar proxy into any Pods you create. To inject a Pod with an Istio sidecar proxy, you must explicitly enable injection for the Pod's Deployment or for the entire namespace. If you have done this and the sidecar is still not installed, follow the solution steps to identify which settings are preventing the injection.

A Pod is not injected with an Istio sidecar proxy if:

- Istio sidecar proxy injection is disabled at the namespace level

- The `sidecar.istio.io/inject` label on the Pod is set to false
- The Pod's `spec` contains `hostNetwork: true`

Solution

Find out which Pods do not have Istio sidecar proxy injection enabled and why. You can either inspect Pods across all namespaces, focus on a specific namespace, or verify why a selective Pod is not injected.

Check Pods in All Namespaces

1. Download the sidecar analysis script .
2. Run the script.

   ```
   ./sidecar-analysis.sh
   ```

   You get an output similar to this one:

   ```
   See all Pods that are not part of the Istio service mesh:
     Pods in namespaces labeled with "istio-injection=disabled":
       - namespace/Pod
       ...
     Pods labeled with "sidecar.istio.io/inject=false" in namespaces labeled
   with "istio-injection=enabled":
       - namespace/Pod
       ...
     Pods not labeled with "sidecar.istio.io/inject=true" in unlabeled
   namespaces:
       - namespace/Pod
       ...
   ```

3. To learn how to include a Pod into the Istio service mesh, see Enabling Istio Sidecar Proxy Injection [page 1892].

Check Pods in a Selective Namespace

1. Download the sidecar analysis script .
2. Run the script passing the namespace name as a parameter.

   ```
   ./sidecar-analysis.sh {YOUR_NAMEPSACE}
   ```

   You get an output similar to this one:

   ```
   In the namespace default, the following Pods are not part of the Istio
   service mesh:
     - Pod
     - Pod
     ...
   Reason: The namespace default has Istio sidecar proxy injection disabled, so
   none of its Pods have been injected with an Istio sidecar proxy.
   ```

3. To learn how to include a Pod into the Istio service mesh, see Enabling Istio Sidecar Proxy Injection [page 1892].

Check a Selective Pod

- Use Kyma dashboard.
  1. Go to the Pod's namespace.
  2. Check if Istio sidecar proxy injection is enabled at the namespace level.
     Verify if the `Labels` section contains `istio-injection=disabled` or `istio-injection=enabled`. If Istio sidecar proxy injection is disabled at the namespace level, none of its Pods are injected with an Istio sidecar proxy.
  3. Check if Istio sidecar proxy injection is enabled for the Pod's Deployment.
     In the *Workloads* section, select *Deployments* and choose *Edit*. In the `UI Form` section, check if the `Enable Sidecar Injection` toggle is switched.
  4. Check if the label `sidecar.istio.io/inject: false` is set on a Pod.
     In the *Workloads* section, select *Pods*. Search for `sidecar.istio.io/inject: false`. If your Pod is displayed on the list, it has the label set.
  5. To learn how to include a Pod into the Istio service mesh, see Enabling Istio Sidecar Proxy Injection [page 1892].
- Use kubectl.
  1. Check if Istio sidecar proxy injection is enabled at the namespace level.
     Run the command:

     ```
     kubectl get namespaces {POD_NAMESPACE} -o
     jsonpath='{ .metadata.labels.istio-injection }'
     ```

  2. Check if Istio sidecar proxy injection is enabled for the Pod's Deployment.
     Run the command:

     ```
     kubectl get deployments {POD_DEPLOYMENT} -n {NAMESPACE} -o
     jsonpath='{ .spec.template.metadata.labels }'
     ```

  3. Check if the label `sidecar.istio.io/inject: false` is set on a Pod.

     ```
      kubectl get pod {POD} -n default
     -o=jsonpath='{.metadata.labels.sidecar\.istio\.io/inject}
     ```

  4. To learn how to include a Pod into the Istio service mesh, see Enabling Istio Sidecar Proxy Injection [page 1892].

4.3.1.1.7.6 Incompatible Sidecar Version After the Istio Module's Update

Learn which Pods you must restart manually after the Istio module's update.

Symptom

After the Istio module's update, the Istio custom resource (CR) is in the `Warning` state, and mesh connectivity is disrupted. When you click on the warning in Kyma dashboard or run `kubectl get istio default -n kyma-system -o jsonpath='{.status.description}'`, you get the following message:

```
Some Pods with Istio sidecar injection failed to restart. To learn more about
the warning, see kyma-system/istio-controller-manager logs: Istio Controller
could not restart one or more Istio-injected Pods.
```

Cause

The sidecar version in Pods must match the installed Istio version to ensure proper mesh connectivity. During an upgrade of the Istio module to a new version, Istio Operator's `ProxySidecarReconcilation` component performs a rollout for most common workload types ensuring that the injected Istio sidecar proxies are updated correctly. However, if a resource is a Job, a ReplicaSet that is not managed by any Deployment, or a Pod that is not managed by any other resource, the restart cannot be performed automatically.

You must manually restart such workloads to ensure proper functionality with the updated Istio version.

Solution

Use Kyma dashboard.

1. Choose *Modify Modules*.
2. Select the Istio module.
   The `ProxySidecarRestartSucceeded` reconciliation condition has the status `False` and the reason: `ProxySidecarManualRestartRequired`. The message contains the list of workloads that you must restart manually, for example:

   ```
   The sidecars of the following workloads could not be restarted: test/httpbin
   ```

3. Restart the listed workloads so that new Istio sidecars are injected into the Pods.
   The method for restarting resources varies depending on your specific use case. For standalone resources, one approach is to modify the configuration of the running Pod to initiate a restart and then revert it back to the previous configuration. Since restart issues might also be application-specific, it's recommended that you check your application's logs for more insights.

Use kubectl.

1. To get the Istio CR, run:

   ```
   kubectl get istio default -n kyma-system -o yaml
   ```

2. To learn if any Pods or workloads require a manual restart, see the message in the `status.conditions` section of the Istio CR, for example:

   ```
   - lastTransitionTime: "2025-01-23T10:02:08Z"
     message: 'The sidecars of the following workloads could not be restarted:
   test/httpbin'
     reason: ProxySidecarManualRestartRequired
     status: "False"
     type: ProxySidecarRestartSucceeded
   ```

3. Restart the listed workloads so that new Istio sidecars are injected into the Pods.
   The method for restarting resources varies depending on your specific use case. For standalone resources, one approach is to modify the configuration of the running Pod to initiate a restart and then revert it back to the previous configuration. Since restart issues might also be application-specific, it's recommended that you check your application's logs for more insights.

4.3.1.1.7.7 Pods Created by Jobs Can't Finish

Symptom

Pods created by Jobs remain in the `NotReady` status after the main containers have finished.

Cause

By default, the Istio module 1.22 and later versions inject `istio-proxy` containers as native sidecars. However, you can also set the annotation `sidecar.istio.io/nativeSidecar` to "false" for a specific Pod. This annotation overwrites the default setting and indicates that instead of native sidecars, the annotated Pod must be injected with a regular `istio-proxy` container.

When `istio-proxy` is a regular container, it runs independently of the application container. There is no mechanism that shuts down the `istio-proxy` regular container when the main container completes its tasks. Consequently, the Pod is also running.

Solution

Check whether `istio-proxy` is declared as `initContainer` or `container`.

- If it is an `initContainer`, it means that `istio-proxy` already runs as a native sidecar. Since native sidecars do not cause problems with incomplete Jobs, the root cause of the issue is not related to the type of containers used by Istio proxies.

- If `istio-proxy` is declared as a regular container, switch to using a native sidecar instead. To do this, apply the annotation `sidecar.istio.io/nativeSidecar="true"` on the Pod or in the Pod template. See the following example:

  Sample Code

  ```
  apiVersion: batch/v1
  kind: Job
  metadata:
    name: test-job
    namespace: test
  spec:
    template:
      metadata:
        annotations:
          sidecar.istio.io/nativeSidecar: "true"
      spec:
        containers:
        - name: test-job
          image: alpine:latest
          command: [ "/bin/sleep", "10" ]
        restartPolicy: Never
  ```

When `istio-proxy` runs as a native sidecar, its lifecycle depends on the main application container, so `istio-proxy` finishes automatically at the same time as the main application container. After this, the Pod's state changes to `Completed`.

Related Information

Regular and Native Sidecar Containers [page 1894]

4.3.1.1.7.8 Init Containers Can’t Access the Network

Symptom

Init containers can’t access the network.

Cause

If Istio injection is enabled, the `istio-proxy` container intercepts all network traffic from all containers. By default, the Istio module 1.22 and later versions inject `istio-proxy` containers as native sidecars. However, you can also set the `sidecar.istio.io/nativeSidecar` annotation to "false" for a specific Pod. This annotation overwrites the default setting and indicates that instead of native sidecars, the annotated Pod must be injected with a regular sidecar container.

Init containers are started before regular containers. If `istio-proxy` is a regular sidecar, it doesn't work when init containers are running. As a result, init containers don't have network access.

Solution

Check whether `istio-proxy` is declared as `initContainer` or `container`.

- If it is an `initContainer`, it means that `istio-proxy` already runs as a native sidecar. Since native sidecars do not cause networking problems with init containers, the root cause of the issue is not related to the type of containers used by Istio proxies.
- If `istio-proxy` is declared as a regular container, switch to using a native sidecar instead. To do this, apply the annotation `sidecar.istio.io/nativeSidecar="true"` on the Pod or in the Pod template. See the following example:

  Sample Code

  ```
  apiVersion: v1
  kind: Pod
  metadata:
    name: init-container-network-check
    namespace: test
    annotations:
      sidecar.istio.io/nativeSidecar: "true"
  spec:
    initContainers:
    - name: init
      image: curlimages/curl
      command: [ "curl", "httpbin.org/get" ]
    containers:
    - name: main
      image: alpine:latest
      command: ["/bin/sleep", "10"]
    restartPolicy: Never
  ```

When `istio-proxy` is a native sidecar, Istio injects it as the first init container, so all containers running later are able to access the network.

Related Information

Regular and Native Sidecar Containers [page 1894]

4.3.1.1.7.9 Reverting the Istio Module's Deletion

Follow the steps outlined in this troubleshooting guide if you unintentionally deleted the Istio module and want to restore the cluster to its normal state without losing any resources created in the cluster.

Symptom

The Istio custom resource (CR) is in the `Warning` state. The condition of type `Ready` is set to `false` with the reason `IstioCustomResourcesDangling`. To verify this, run the command:

```
kubectl get istio default -n kyma-system -o jsonpath='{.status.conditions[0]}'
```

You get an output similar to this:

```
{"lastTransitionTime":"2024-09-26T18:23:00Z","message":"Istio deletion
blocked because of existing Istio custom
resources","reason":"IstioCustomResourcesDangling","status":"False","type":"Ready"}
```

> ### Note
>
> If you intended to delete the Istio module, the symptoms described in this document are expected, and you must clean up the remaining resources yourself. To check which resources are blocking the deletion, see the logs of the `istio-controller-manager` container.

Cause

The Istio module wasn't completely removed because related resources still exist in the cluster.

For example, the issue occurs when you delete Istio, but there are still `VirtualService` resources either created by you or installed by another Kyma component or module. In such cases, the hooked finalizer pauses the deletion of Istio until you remove all the related resources. This blocking deletion strategy is intentionally designed and is enabled by default for the Istio module.

Solution

Use Kyma dashboard.

1. In the *Cluster Overview* section, select *Modify Modules*.
2. Select the Istio module.
3. Choose *Edit*.
4. Switch to the *YAML* section.
5. To remove the finalizers from the Istio custom resource, delete the following lines:

   ```
   finalizers:
       - istios.operator.kyma-project.io/istio-installation
   ```

When the finalizers are removed, the Istio module is deleted. All the other resources remain in the cluster.

6. Choose *Save*.
7. Add the Istio module again.

When you re-add the Istio module, its reconciliation is reinitiated. The Istio CR returns to the `Ready` state within a few seconds.

Use kubectl.

1. To edit the Istio CR, run:

```
kubectl edit istio -n kyma-system default
```

2. To remove the finalizers from the Istio custom resource, delete the following lines:

```
finalizers:
    - istios.operator.kyma-project.io/istio-installation
```

When the finalizers are removed, the Istio module is deleted. All the other resources remain in the cluster.

3. Save the changes.
4. Add the Istio module again.

When you re-add the Istio module, its reconciliation is reinitiated. The Istio CR returns to the `Ready` state within a few seconds.

4.3.1.1.7.10 SAP HANA Database Connection Issues

Symptom

You're unable to connect an application to a SAP HANA Database instance.

Cause

The Istio module's default configuration does not restrict outbound traffic. This means that the application should have no issues connecting to a SAP HANA Database instance. If you are experiencing issues, they may be related to the SAP HANA Database instance or your cluster configuration. To identify the source of the problem, follow the troubleshooting steps.

Solution

Connect to the SAP HANA Database Instance from Outside of the Cluster

1. Download SAP HANA Client for your operating system from SAP Development Tools.
2. Unpack the downloaded archive.
3. Install SAP HANA Client.

4. To connect to SAP HANA Database instance, use the following command:

   ```
   hdbsql -n <Hana_DB_instance_adress> -u <Hana_DB_user> -p <Hana_DB_password>
   ```

   For example:

   ```
   hdbsql -n aaa.bbb.ccc.ddd:30015 -u my_user -p mypassword
   ```

   If the connection is successful and you can run queries, the issue is not related to the SAP HANA Database instance.

Connect to the SAP HANA Database Instance from Inside of the Cluster

1. Build a Docker image with the SAP HANA Client installed. You can use the following Dockerfile:

   ```
   FROM eclipse-temurin:17
   WORKDIR /build
   COPY client.tar client.tar
   RUN tar -xvf client.tar
   RUN echo "/usr/local/bin" | ./client/hdbinst
   ENTRYPOINT ["sleep", "8000"]
   ```

2. Download SAP HANA Client for Linux x86 64-bit from SAP Development Tools and save it as `client.tar` in the same directory as the Dockerfile.
3. To build the image, run the following command:

   ```
   docker buildx build --platform=linux/amd64 -t hdbsql .
   ```

4. To test your image, run the following command:

   ```
   docker run --entrypoint "hdbsql" hdbsql -v
   ```

   You get an output similar to this example:

   ```
   HDBSQL version 2.20.20.1712178305, the SAP HANA Database interactive terminal.
   Copyright 2000-2024 by SAP SE.
   ```

5. Publish the image to a container registry.
6. Run the image in the Kubernetes cluster:

   ```
   kubectl create deployment hdbsql --image={PUBLISHED_IMAGE_NAME}
   ```

7. To attach to the Pod and connect to the SAP HANA Database instance, run the following command:

   ```
   hdbsql -n {HANA_DB_INSTANCE_ADDRESS} -u {HANA_DB_USER} -p {HANA_DB_PASSWORD}
   ```

   If the connection is successful and you can execute queries, the issue is not related to the setup of the cluster.
8. Check the connection from a Pod that has the Istio sidecar injected. To do this, create a Deployment in a namespace with Istio sidecar injection enabled.

4.3.1.1.8 Default Istio Configuration

Within the Istio module, Istio Operator provides baseline values for the Istio installation. To adjust these settings according to your needs, you can override them in the Istio custom resource (CR).

See the major differences in the configuration of Istio Operator compared to upstream Istio:

- The Istiod (Pilot) and Ingress Gateway components are enabled by default.
- Automatic Istio sidecar proxy injection is disabled by default.
- To enhance security and performance, both Istio control plane and data plane use the distroless version of Istio images. Those images are not Debian-based and are slimmed down to reduce any potential attack surface. To learn more, see Harden Docker Container Images .
- Resource requests and limits for Istio sidecars proxies are modified to best suit the needs of the evaluation and production profiles.
- Mutual TLS (mTLS) is enabled in the STRICT mode for workloads in the Istio service mesh.
- Egress traffic is not controlled. All applications deployed in the Kyma cluster can access outside resources without limitations.
- The CNI component, used for the installation of an Istio sidecar, is provided as a DaemonSet. This means that one replica is present on every node of the target cluster.
- The self-signed CA certificate's bit length is set to 4096 instead of the default 2048.

Configuration Based on the Cluster Size

The configuration of Istio resources depends on the cluster capabilities. If your cluster has less than 5 total virtual CPU cores or its total memory capacity is less than 10 gigabytes, the default setup for resources and autoscaling is lighter. If your cluster exceeds both of these thresholds, Istio is installed with the higher resource configuration.

Default Resource Configuration for Smaller Clusters

Component	CPU Requests	CPU Limits	Memory Requests	Memory Limits
Proxy	10 m	250 m	32 Mi	254 Mi
Ingress Gateway	10 m	1000 m	32 Mi	1024 Mi
Pilot	50 m	1000 m	128 Mi	1024 Mi
CNI	10 m	250 m	128 Mi	384 Mi

Default Resource Configuration for Larger Clusters

Component	CPU Requests	CPU Limits	Memory Requests	Memory Limits
Proxy	10 m	1000 m	192 Mi	1024 Mi
Ingress Gateway	100 m	2000 m	128 Mi	1024 Mi
Pilot	100 m	4000 m	512 Mi	2 Gi
CNI	100 m	500 m	512 Mi	1024 Mi

Default Autoscaling Configuration for Smaller Clusters

Component	minReplicas	maxReplicas
Ingress Gateway	1	1

Component	minReplicas	maxReplicas
Pilot	1	1

Default Autoscaling Configuration for Larger Clusters

Component	Minimum Number of Replicas	Maximum Number of Replicas
Ingress Gateway	3	10
Pilot	2	5

4.3.1.2 API Gateway Module

API Gateway is a Kyma module with which you can expose and secure APIs.

What Is API Gateway?

To use the API Gateway module, you must also add the Istio module. Moreover, to expose a workload using the `APIRule` custom resource (CR), the workload must be part of the Istio service mesh.

By default, both the API Gateway and Istio modules are automatically added when you create a SAP BTP, Kyma runtime instance.

Features

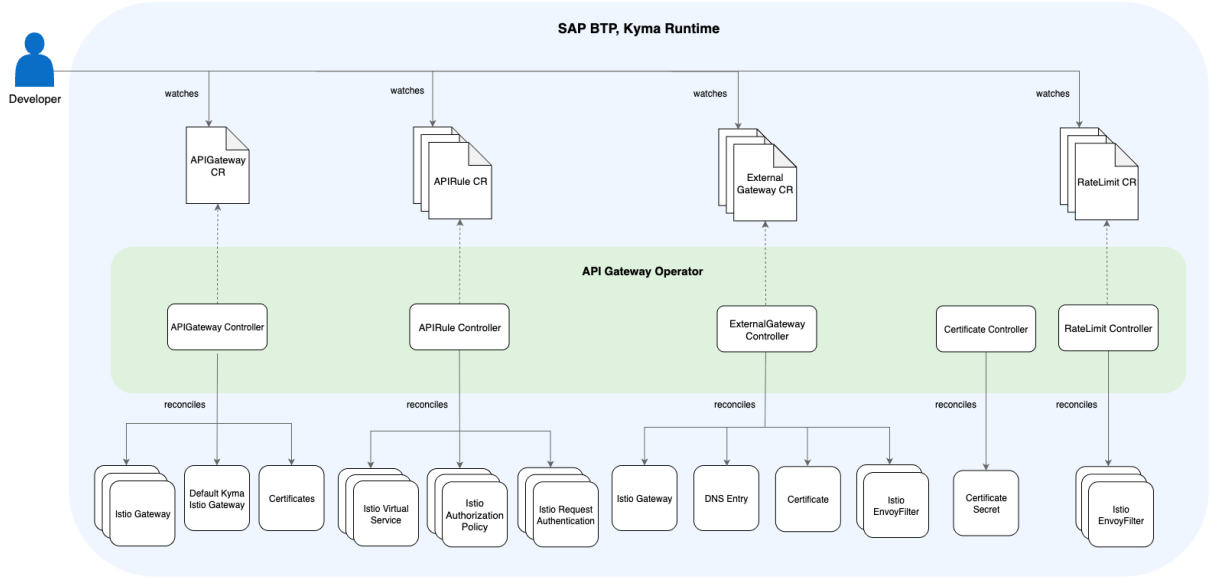
The API Gateway module offers the following features:

- API exposure: The module uses Istio features to help you easily and securely expose your workloads by creating `APIRule` CRs. With an `APIRule`, you can perform the following actions:
  - Group multiple workloads and expose them under a single host.
  - Use a short host name to simplify the migration of resources to a new cluster.
  - Configure the noAuth access strategy, which offers a simple configuration to allow access to specific HTTP methods.
  - Secure your workloads by configuring jwt or extAuth access strategies. The jwt access strategy enables you to use Istio's JWT configuration to protect your exposed services and interact with them using JSON Web Tokens. The extAuth access strategy allows you to implement custom authentication and authorization logic.
- Default Kyma gateway configuration: The module creates and manages the default Istio TLS Gateway that receives incoming traffic in your Kyma cluster. The gateway uses the cluster's default domain and a self-signed certificate.
- Rate limiting: The module simplifies local rate limiting on the Istio service mesh layer. By configuring a `RateLimit` CR, you can limit the number of requests targeting an exposed application in a unit of time, based on specific paths and headers.

- External gateway integration: Configure Istio mTLS Gateway and integrate it with an external gateway deployed outside of the Kyma cluster.

Architecture

Within the API Gateway module, API Gateway Operator manages the application of API Gateway's configuration and handles resource reconciliation. It contains the following controllers: APIGateway Controller, APIRule Controller, RateLimit Controller, and ExternalGateway Controller.



API/Custom Resource Definitions

The `apigateways.operator.kyma-project.io` CustomResourceDefinition (CRD) describes the `APIGateway` CR that APIGateway Controller uses to manage the module and its resources. See APIGateway Custom Resource .

The `apirules.operator.kyma-project.io` CRD describes the `APIRule` CR that APIRule Controller uses to expose and secure APIs. See APIRule Custom Resource .

The `externalgateways.gateway.kyma-project.io` CRD describes the kind and the format of data that ExternalGateway Controller uses to configure external gateway integration. See ExternalGateway Custom Resource .

The `ratelimits.gateway.kyma-project.io` CRD describes the kind and the format of data that RateLimit Controller uses to configure request rate limits for applications. See RateLimit Custom Resource .

Authorization

To assign access permissions to the API Gateway module resources, use the following aggregated ClusterRoles :

- `kyma-api-gateway-view` - Grants read-only access to API Gateway resources.
- `kyma-api-gateway-edit` - Grants full access to API Gateway resources.

Resource Consumption

To learn more about the resources used by the Istio module, see Kyma Modules' Sizing.

4.3.1.2.1 Istio Gateways

Learn what Istio Gateways are, why you might need a custom Gateway, and how to choose the right Gateway configuration for your use case in Kyma.

What is a Gateway?

> **Note**
>
> This document refers to Istio Gateway custom resources, which are Istio-specific and use VirtualServices for routing. This is different from the Kubernetes Gateway API, which is a vendor-neutral standard also supported by the Istio module. To learn more about Gateway API, see the Kubernetes Gateway API documentation and Exposing Workloads Using Gateway API .

A Gateway manages inbound and outbound traffic for the service mesh. It acts as a load balancer operating at the edge of the mesh, receiving incoming HTTP/HTTPS connections.

The Gateway performs the following functions:

- Decrypts incoming HTTPS traffic and encrypts responses.
- Directs requests to the appropriate services based on traffic redirection rules.
- Applies authentication, authorization, and traffic management rules in a centralized location.
- Acts as the entry point for traffic from outside the cluster to reach services inside.

Choosing Istio Ingress Gateway Configuration

To configure an Istio Ingress Gateway, choose the domain type (your cluster's default domain or your own custom domain) and the TLS mode (simple TLS or mTLS). These choices determine the setup steps in Kyma. See the following matrix:

TLS mode \ Domain name	Default Kyma domain	Custom domain
TLS	• Default Kyma Gateway `kyma-system/kyma-gateway` - pre-configured and ready to use for quick start and development. See Kyma Gateway [page 1969]. **⚠ Caution** Because the default Kyma domain is a wildcard domain, which uses a simple TLS Gateway, it is recommended that you set up your custom domain for use in a production environment. • Custom TLS Gateway on the default Kyma domain - Gateway isolation with minimal setup and no DNS configuration required. See Configure a TLS Gateway: Kyma Domain Scenario [page 1953].	Custom TLS Gateway on your custom domain - recommended for production environments, provides full control over the domain name. See Configure a TLS Gateway: Custom Domain Scenario [page 1949].
mTLS	Custom mTLS Gateway on the default Kyma domain - secure B2B APIs with client authentication, no DNS configuration required. See Configure an mTLS Gateway: Kyma Domain Scenario [page 1962].	Custom mTLS Gateway on your custom domain - recommended for production B2B APIs integrations, provides strongest security with full control over domain name. See Configure an mTLS Gateway: Custom Domain Scenario [page 1956].

Domain Name

A Gateway host is the domain name or hostname that the Gateway listens on to accept incoming traffic. It defines which domain names the Gateway responds to. As for the domain name, you can choose from the following options:

- Use your custom domain.
  To use a custom domain, you must own the DNS zone and supply credentials for a provider supported by Gardener so the ACME DNS challenge can be completed. For this, you must first register this DNS provider in your Kyma runtime cluster and create a DNS entry resource.
- Use the default domain of your Kyma cluster.

When you create an SAP BTP, Kyma runtime instance, your cluster receives a default wildcard domain that provides the endpoint for the Kubernetes API server. This is the primary access point for all cluster management operations, used by kubectl and other tools.
By default, the default Ingress Gateway `kyma-gateway` is configured under this domain. To learn what the domain is, you can check the APIServer URL in your subaccount overview, or get the domain name from the default simple TLS Gateway:

```
kubectl get gateway -n kyma-system kyma-gateway -o
jsonpath='{.spec.servers[0].hosts[0]}'
```

You can request any subdomain of the assigned default domain and use it to create a TLS or mTLS Gateway, as long as it is not used by another resource. For example, if your default domain is `*.c12345.kyma.ondemand.com` you can use such subdomains as `example.c12345.kyma.ondemand.com`, `*.example.c12345.kyma.ondemand.com`, and more. If you use the Kyma runtime default domain, Gardener's issuer can issue certificates for subdomains of that domain without additional DNS delegation.

TLS Mode

You can either use simple TLS or mTLS:

- Use simple TLS.
  In TLS mode, the server presents a certificate to prove its identity to clients, but clients don't need to provide certificates. This is the most common configuration for public-facing websites and APIs.
- Use mTLS.
  In mTLS mode, both the server and client present certificates to verify each other's identity. This provides stronger authentication by ensuring only clients with valid certificates can connect.

Additional Configuration Options

For additional configuration options, see Gateway .

4.3.1.2.1.1 Configure a TLS Gateway

Learn how to configure a TLS Gateway in SAP BTP, Kyma runtime using Gardener-managed Let's Encrypt certificates.

Context

In this procedure, you set up a TLS Gateway that secures communication between clients and your workloads. The server certificate is automatically provisioned and managed through Gardener's Certificate custom resource (CR), which requests a publicly trusted certificate from Let's Encrypt.

For setting up the TLS Gateway, you must prepare the domain name available in the public DNS zone. You can either use a custom domain (see Custom Domain Scenario [page 1949]) or the default domain of your Kyma cluster (see Default Domain Scenario [page 1962]. For more information on the domain names and Gateways, see Istio Gateways.

Custom Domain Scenario

Prerequisites

- The Istio and API Gateway modules are added to your Kyma cluster by default. See Adding and Deleting a Kyma Module [page 3511].
- You have the domain name available in the public DNS zone.
- Your DNS provider is supported by Gardener. For the list of supported providers, see External DNS Management Guidelines .

Procedure

1. Create a namespace with enabled Istio sidecar proxy injection.

```
kubectl create ns test
kubectl label namespace test istio-injection=enabled --overwrite
```

2. Export the following domain names as environment variables. Replace `my-own-domain.example.com` with the name of your domain. You can adjust these values as needed.

```
PARENT_DOMAIN="my-own-domain.example.com"
SUBDOMAIN="tls.${PARENT_DOMAIN}"
GATEWAY_DOMAIN="*.${SUBDOMAIN}"
WORKLOAD_DOMAIN="httpbin.${SUBDOMAIN}"
echo "Parent Domain: ${PARENT_DOMAIN}"
echo "Subdomain: ${SUBDOMAIN}"
echo "Gateway Domain: ${GATEWAY_DOMAIN}"
echo "Workload Domain: ${WORKLOAD_DOMAIN}"
```

Placeholder	Example domain name	Description
`PARENT_DOMAIN`	`my-own-domain.example.com`	The domain name available in your public DNS zone.
`SUBDOMAIN`	`tls.my-own-domain.example.com`	A subdomain created under the parent domain, specifically for the TLS Gateway.
`GATEWAY_DOMAIN`	`*.tls.my-own-domain.example.com`	A wildcard domain covering all possible subdomains under the TLS subdomain. When configuring the Gateway, this allows you to expose workloads on multiple hosts (for example, `httpbin.tls.my-own-domain.example.com`, `test.httpbin.tls.my-own-domain.example.com`) without creating separate Gateway rules for each one.

Placeholder	Example domain name	Description
`WORKLOAD_DOMAIN`	`httpbin.tls.my-own-domain.example.com`	The specific domain assigned to your workload.

3. Create a Secret containing credentials for your DNS cloud service provider.

   The information you provide to the data field differs depending on the DNS provider you're using. The DNS provider must be supported by Gardener. To learn how to configure the Secret for a specific provider, follow External DNS Management Guidelines and the External DNS Management examples.

   See an example Secret for AWS Route 53 DNS provider. `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` are base-64 encoded credentials.

```
apiVersion: v1
kind: Secret
metadata:
  annotations:
    dns.gardener.cloud/class: garden
  name: aws-credentials
  namespace: test
type: Opaque
data:
  AWS_ACCESS_KEY_ID: <your AWS access key>
  AWS_SECRET_ACCESS_KEY: <your AWS secret access key>
  # Optionally, specify the region
  #AWS_REGION: <your AWS region>
  # Optionally, specify the token
  #AWS_SESSION_TOKEN: <your AWS session token>
```

4. Create a DNSProvider resource that references the Secret with your DNS provider's credentials.

   See an example Secret for AWS Route 53 DNS provider:

```
apiVersion: dns.gardener.cloud/v1alpha1
kind: DNSProvider
metadata:
  name: aws
  namespace: test
spec:
  type: aws-route53
  secretRef:
    name: aws-credentials
  domains:
    include:
    - "${PARENT_DOMAIN}"
```

   For DNSProvider configuration for other providers, see the External DNS Management examples.

5. Get the external access point of the `istio-ingressgateway` service.

   The external access point is either stored in the ingress Gateway's `ip` field (for example, on GCP) or in the ingress Gateway's `hostname` field (for example, on AWS).

```
LOAD_BALANCER_ADDRESS=$(kubectl get services --namespace istio-system istio-
ingressgateway --output jsonpath='{.status.loadBalancer.ingress[0].ip}')
if [[ -z $LOAD_BALANCER_ADDRESS ]]; then
    LOAD_BALANCER_ADDRESS=$(kubectl get services --
namespace istio-system istio-ingressgateway --output
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
fi
echo "The load balancer address is ${LOAD_BALANCER_ADDRESS}"
```

6. Create a DNSEntry resource.

```
cat <<EOF | kubectl apply -f -
apiVersion: dns.gardener.cloud/v1alpha1
kind: DNSEntry
metadata:
  name: dns-entry
  namespace: test
  annotations:
    dns.gardener.cloud/class: garden
spec:
  dnsName: "${GATEWAY_DOMAIN}"
  ttl: 600
  targets:
    - "${LOAD_BALANCER_ADDRESS}"
EOF
```

7. Create the server's certificate.

   You use a Certificate CR to request and manage Let's Encrypt certificates from your Kyma cluster. When you create a Certificate CR, one of Gardener's operators detects it and creates an ACME request to Let's Encrypt requesting certificate for the specified domain names. The issued certificate is stored in an automatically created Kubernetes Secret, which name you specify in the Certificate's `secretName` field. For more information, see Manage certificates with Gardener for public domain .

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: cert.gardener.cloud/v1alpha1
   kind: Certificate
   metadata:
     name: domain-certificate
     namespace: "istio-system"
   spec:
     secretName: kyma-mtls
     commonName: "${GATEWAY_DOMAIN}"
     issuerRef:
       name: garden
   EOF
   ```

   To verify that the Secret with Gateway certificates is created, run:

   Sample Code

   ```
   kubectl get secret -n istio-system custom-tls-secret
   ```

8. Create a TLS Gateway.

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: networking.istio.io/v1alpha3
   kind: Gateway
   metadata:
     name: custom-tls-gateway
     namespace: test
   spec:
     selector:
       app: istio-ingressgateway
       istio: ingressgateway
     servers:
       - port:
           number: 443
           name: https
           protocol: HTTPS
         tls:
           mode: SIMPLE
   ```

```
        credentialName: custom-tls-secret
      hosts:
        - "${GATEWAY_DOMAIN}"
EOF
```

To verify that the TLS Gateway is created, run:

```
kubectl get gateway -n test custom-tls-gateway
```

9. To test the TLS connection, you can deploy and expose a sample HTTPBin Service.
   a. Deploy the HTTPBin Service.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ServiceAccount
metadata:
  name: httpbin
  namespace: test
---
apiVersion: v1
kind: Service
metadata:
  name: httpbin
  namespace: test
  labels:
    app: httpbin
    service: httpbin
spec:
  ports:
  - name: http
    port: 8000
    targetPort: 80
  selector:
    app: httpbin
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
  namespace: test
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
      labels:
        app: httpbin
        version: v1
    spec:
      serviceAccountName: httpbin
      containers:
      - image: docker.io/kennethreitz/httpbin
        imagePullPolicy: IfNotPresent
        name: httpbin
        ports:
        - containerPort: 80
EOF
```

   b. Expose the HTTPBin Service.

```
cat <<EOF | kubectl apply -f -
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
```

```
metadata:
  name: httpbin-tls
  namespace: test
spec:
  gateway: test/custom-tls-gateway
  hosts:
    - "${WORKLOAD_DOMAIN}"
  rules:
    - methods:
        - GET
      noAuth: true
      path: /*
      timeout: 300
  service:
    name: httpbin
    port: 8000
EOF
```

c. Test the connection.

```
curl -X GET https://${WORKLOAD_DOMAIN}/ip
```

If successful, you get code `200` in response.

Default Domain Scenario

Prerequisites

- The Istio and API Gateway modules are added to your Kyma cluster by default. See Adding and Deleting a Kyma Module [page 3511].

Procedure

1. Create a namespace with enabled Istio sidecar proxy injection.

```
kubectl create ns test
kubectl label namespace test istio-injection=enabled --overwrite
```

2. Export the following domain names as environment variables. Replace `my-own-domain.example.com` with the name of your domain. You can adjust these values as needed.

```
PARENT_DOMAIN=$(kubectl get gateway -n kyma-system kyma-gateway -o
jsonpath='{.spec.servers[0].hosts[0]}' | sed 's/\*\.//')
SUBDOMAIN="tls.${PARENT_DOMAIN}"
GATEWAY_DOMAIN="*.${SUBDOMAIN}"
WORKLOAD_DOMAIN="httpbin.${SUBDOMAIN}"
echo "Parent Domain: ${PARENT_DOMAIN}"
echo "Subdomain: ${SUBDOMAIN}"
echo "Gateway Domain: ${GATEWAY_DOMAIN}"
echo "Workload Domain: ${WORKLOAD_DOMAIN}"
```

Placeholder	Example domain name	Description
PARENT_DOMAIN	`my-default-domain.example.com`	The domain name available in your public DNS zone.
SUBDOMAIN	`tls.my-default-domain.example.com`	A subdomain created under the parent domain, specifically for the TLS Gateway.
GATEWAY_DOMAIN	`*.tls.my-default-domain.example.com`	A wildcard domain covering all possible subdomains under the TLS subdomain. When configuring the Gateway, this allows you to expose workloads on multiple hosts (for example, `httpbin.tls.my-default-domain.example.com`, `test.httpbin.tls.my-default-domain.example.com`) without creating separate Gateway rules for each one.
WORKLOAD_DOMAIN	`httpbin.tls.my-default-domain.example.com`	The specific domain assigned to your workload.

3. Create a TLS Gateway.

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: custom-tls-gateway
  namespace: test
spec:
  selector:
    app: istio-ingressgateway
    istio: ingressgateway
  servers:
    - port:
        number: 443
        name: https
        protocol: HTTPS
      tls:
        mode: SIMPLE
        credentialName: custom-tls-secret
      hosts:
        - "${GATEWAY_DOMAIN}"
EOF
```

To verify that the TLS Gateway is created, run:

```
kubectl get gateway -n test custom-tls-gateway
```

4. To test the TLS connection, you can deploy and expose a sample HTTPBin Service.
   a. Deploy the HTTPBin Service.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ServiceAccount
metadata:
  name: httpbin
```

```
  namespace: test
---
apiVersion: v1
kind: Service
metadata:
  name: httpbin
  namespace: test
  labels:
    app: httpbin
    service: httpbin
spec:
  ports:
  - name: http
    port: 8000
    targetPort: 80
  selector:
    app: httpbin
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
  namespace: test
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
      labels:
        app: httpbin
        version: v1
    spec:
      serviceAccountName: httpbin
      containers:
      - image: docker.io/kennethreitz/httpbin
        imagePullPolicy: IfNotPresent
        name: httpbin
        ports:
        - containerPort: 80
EOF
```

b. Expose the HTTPBin Service.

```
cat <<EOF | kubectl apply -f -
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: httpbin-tls
  namespace: test
spec:
  gateway: test/custom-tls-gateway
  hosts:
    - "${WORKLOAD_DOMAIN}"
  rules:
    - methods:
        - GET
      noAuth: true
      path: /*
      timeout: 300
  service:
    name: httpbin
    port: 8000
EOF
```

c. Test the connection.

```
curl -X GET https://${WORKLOAD_DOMAIN}/ip
```

If successful, you get code `200` in response.

4.3.1.2.1.2 Configure an mTLS Gateway

Learn how to configure mutual TLS (mTLS) in SAP BTP, Kyma runtime using Gardener-managed Let's Encrypt server certificates and client certificates that you supply.

Context

In this procedure, you generate certificates using the following approach:

- Gardener's Certificate custom resource (CR) requests a publicly trusted server certificate from Let's Encrypt and creates a Secret that stores the certificate and private key. Therefore, the clients must trust Let's Encrypt, which is the CA that signs the server's certificate. Most modern HTTP clients already trust Let's Encrypt.
- Client certificates are self-signed. For production use, it's advised to use certificates issued by a trusted CA instead.

For setting up the mTLS Gateway, you must prepare the domain name available in the public DNS zone. You can either use a custom domain (see Custom Domain Scenario [page 1956]) or the default domain of your Kyma cluster (see Default Domain Scenario [page 1962]. For more information on the domain names and Gateways, see Istio Gateways.

Custom Domain Scenario

Prerequisites

- The Istio and API Gateway modules are added to your Kyma cluster by default. See Adding and Deleting a Kyma Module [page 3511].
- You have the domain name available in the public DNS zone.
- Your DNS provider is supported by Gardener. For the list of supported providers, see External DNS Management Guidelines .

Procedure

1. Create a namespace with enabled Istio sidecar proxy injection.

```
kubectl create ns test
kubectl label namespace test istio-injection=enabled --overwrite
```

2. Export the following domain names as environment variables. Replace `my-own-domain.example.com` with the name of your domain. You can adjust the following values as needed.

```
PARENT_DOMAIN="my-own-domain.example.com"
SUBDOMAIN="mtls.${PARENT_DOMAIN}"
GATEWAY_DOMAIN="*.${SUBDOMAIN}"
WORKLOAD_DOMAIN="httpbin.${SUBDOMAIN}"
echo "Parent Domain: ${PARENT_DOMAIN}"
echo "Subdomain: ${SUBDOMAIN}"
echo "Gateway Domain: ${GATEWAY_DOMAIN}"
echo "Workload Domain: ${WORKLOAD_DOMAIN}"
```

Placeholder	Example domain name	Description
`PARENT_DOMAIN`	`my-own-domain.example.com`	The domain name available in your public DNS zone.
`SUBDOMAIN`	`mtls.my-own-domain.example.com`	A subdomain created under the parent domain, specifically for the mTLS Gateway.
`GATEWAY_DOMAIN`	`*.mtls.my-own-domain.example.com`	A wildcard domain covering all possible subdomains under the mTLS subdomain. When configuring the Gateway, this allows you to expose workloads on multiple hosts (for example, `httpbin.mtls.my-own-domain.example.com`, `test.httpbin.mtls.my-own-domain.example.com`) without creating separate Gateway rules for each one.
`WORKLOAD_DOMAIN`	`httpbin.mtls.my-own-domain.example.com`	The specific domain assigned to your workload.

3. Create a Secret containing credentials for your DNS cloud service provider.

   The information you provide to the data field differs depending on the DNS provider you're using. The DNS provider must be supported by Gardener. To learn how to configure the Secret for a specific provider, follow External DNS Management Guidelines and the External DNS Management examples.

   See an example Secret for AWS Route 53 DNS provider. `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` are base-64 encoded credentials.

```
apiVersion: v1
kind: Secret
metadata:
  annotations:
    dns.gardener.cloud/class: garden
  name: aws-credentials
  namespace: test
```

```
type: Opaque
data:
  AWS_ACCESS_KEY_ID: <your AWS access key>
  AWS_SECRET_ACCESS_KEY: <your AWS secret access key>
  # Optionally, specify the region
  #AWS_REGION: <your AWS region>
  # Optionally, specify the token
  #AWS_SESSION_TOKEN: <your AWS session token>
```

4. Create a DNSProvider resource that references the Secret with your DNS provider's credentials.

   See an example Secret for AWS Route 53 DNS provider:

```
apiVersion: dns.gardener.cloud/v1alpha1
kind: DNSProvider
metadata:
  name: aws
  namespace: test
spec:
  type: aws-route53
  secretRef:
    name: aws-credentials
  domains:
    include:
    - "${PARENT_DOMAIN}"
```

   For DNSProvider configuration for other providers, see the External DNS Management examples .

5. Get the external access point of the `istio-ingressgateway` service.

   The external access point is either stored in the ingress Gateway's `ip` field (for example, on GCP) or in the ingress Gateway's `hostname` field (for example, on AWS).

```
LOAD_BALANCER_ADDRESS=$(kubectl get services --namespace istio-system istio-
ingressgateway --output jsonpath='{.status.loadBalancer.ingress[0].ip}')
if [[ -z $LOAD_BALANCER_ADDRESS ]]; then
    LOAD_BALANCER_ADDRESS=$(kubectl get services --
namespace istio-system istio-ingressgateway --output
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
fi
echo "The load balancer address is ${LOAD_BALANCER_ADDRESS}"
```

6. Create a DNSEntry resource.

```
cat <<EOF | kubectl apply -f -
apiVersion: dns.gardener.cloud/v1alpha1
kind: DNSEntry
metadata:
  name: dns-entry
  namespace: test
  annotations:
    dns.gardener.cloud/class: garden
spec:
  dnsName: "${GATEWAY_DOMAIN}"
  ttl: 600
  targets:
    - "${LOAD_BALANCER_ADDRESS}"
EOF
```

7. Create the server's certificate.

   You use a Certificate CR to request and manage Let's Encrypt certificates from your Kyma cluster. When you create a Certificate CR, one of Gardener's operators detects it and creates an ACME request to Let's Encrypt requesting certificate for the specified domain names. The issued certificate is stored in an

automatically created Kubernetes Secret, which name you specify in the Certificate's `secretName` field. For more information, see Manage certificates with Gardener for public domain .

```
cat <<EOF | kubectl apply -f -
apiVersion: cert.gardener.cloud/v1alpha1
kind: Certificate
metadata:
  name: domain-certificate
  namespace: "istio-system"
spec:
  secretName: kyma-mtls
  commonName: "${GATEWAY_DOMAIN}"
  issuerRef:
    name: garden
EOF
```

To verify that the Secret with Gateway certificates is created, run:

Sample Code

```
kubectl get secret -n istio-system kyma-mtls
```

8. Prepare the client's certificates.

Caution

For production deployments, use trusted certificate authorities to ensure proper security and automatic certificate management.

a. Create the client's root CA.

```
CLIENT_ROOT_CA_KEY_FILE="client_root_ca_cn.key"
CLIENT_ROOT_CA_CRT_FILE="client_root_ca_cn.crt"
openssl req -x509 -sha256 -nodes -days 365 -newkey rsa:2048 -subj "/
O=Example Client Root CA ORG/CN=Example Client Root CA CN" -keyout "$
{CLIENT_ROOT_CA_KEY_FILE}" -out "${CLIENT_ROOT_CA_CRT_FILE}"
```

b. Create the client's certificate.

```
CLIENT_CERT_CRT_FILE="client_cert_cn.crt"
CLIENT_CERT_CSR_FILE="client_cert_cn.csr"
CLIENT_CERT_KEY_FILE="client_cert_cn.key"
openssl req -out "${CLIENT_CERT_CSR_FILE}" -newkey rsa:2048 -nodes -keyout
"${CLIENT_CERT_KEY_FILE}" -subj "/CN=Example Client Cert CN/O=Example
Client Cert Org"
```

c. Sign the client's certificate.

```
openssl x509 -req -days 365 -CA "${CLIENT_ROOT_CA_CRT_FILE}" -CAkey "$
{CLIENT_ROOT_CA_KEY_FILE}" -set_serial 0 -in "${CLIENT_CERT_CSR_FILE}"
-out "${CLIENT_CERT_CRT_FILE}"
```

9. Create a Secret with Client CA Cert for mTLS Gateway. For more information on the convention that the Secret must use, see Key Convention .

```
kubectl create secret generic -n istio-system "kyma-mtls-cacert" --from-
file=cacert="${CLIENT_ROOT_CA_CRT_FILE}"
```

10. Create an mTLS Gateway.

```
cat <<EOF | kubectl apply -f -
```

```
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: kyma-mtls-gateway
  namespace: test
spec:
  selector:
    app: istio-ingressgateway
    istio: ingressgateway
  servers:
    - port:
        number: 443
        name: mtls
        protocol: HTTPS
      tls:
        mode: MUTUAL
        credentialName: kyma-mtls
      hosts:
        - "${GATEWAY_DOMAIN}"
EOF
```

11. To expose your workload, create an APIRule custom resource.

    You can configure the APIRule to append the headers `X-CLIENT-SSL-CN: '%DOWNSTREAM_PEER_SUBJECT%'`, `X-CLIENT-SSL-ISSUER: '%DOWNSTREAM_PEER_ISSUER%'`, and `X-CLIENT-SSL-SAN: '%DOWNSTREAM_PEER_URI_SAN%'` to the request. These headers provide the upstream (your workload) with the downstream (authenticated client's) identity. This is optional configuration is commonly used in mTLS use cases. For more information about these values, see Envoy Access logging .

    See an example APIRule that exposes the following sample HTTPBin Service:

    Sample Code

    ```
    apiVersion: v1
    kind: ServiceAccount
    metadata:
      name: httpbin
      namespace: test
    ---
    apiVersion: v1
    kind: Service
    metadata:
      name: httpbin
      namespace: test
      labels:
        app: httpbin
        service: httpbin
    spec:
      ports:
      - name: http
        port: 8000
        targetPort: 80
      selector:
        app: httpbin
    ---
    apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: httpbin
      namespace: test
    spec:
      replicas: 1
      selector:
        matchLabels:
    ```

```
      app: httpbin
      version: v1
  template:
    metadata:
      labels:
        app: httpbin
        version: v1
    spec:
      serviceAccountName: httpbin
      containers:
      - image: docker.io/kennethreitz/httpbin
        imagePullPolicy: IfNotPresent
        name: httpbin
        ports:
        - containerPort: 80
```

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: httpbin-mtls
  namespace: test
spec:
  gateway: test/kyma-mtls-gateway
  hosts:
    - "${WORKLOAD_DOMAIN}"
  rules:
    - methods:
        - GET
      noAuth: true
      path: /*
      timeout: 300
      request:
        headers:
          X-CLIENT-SSL-CN: '%DOWNSTREAM_PEER_SUBJECT%'
          X-CLIENT-SSL-ISSUER: '%DOWNSTREAM_PEER_ISSUER%'
          X-CLIENT-SSL-SAN: '%DOWNSTREAM_PEER_URI_SAN%'
  service:
    name: httpbin
    port: 8000
```

12. Test the mTLS connection.
    a. Call the workload without providing client certificates.

    ```
    curl --fail --verbose "https://${WORKLOAD_DOMAIN}/headers?show_env=true"
    ```

    You get cURL 56 error, which indicates that a certificate is not provided by the client. See the following example:

    ```
    curl: (56) OpenSSL SSL_read: OpenSSL/3.5.2: error:0A00045C:SSL
    routines::tlsv13 alert certificate required, errno 0
    ```

    b. Run the following curl command, which specifies the client's certificate and public key:

    ```
    curl --fail --verbose \
      --key "${CLIENT_CERT_KEY_FILE}" \
      --cert "${CLIENT_CERT_CRT_FILE}" \
      "https://${WORKLOAD_DOMAIN}/headers?show_env=true"
    ```

    If successful, you get code `200` in response. The configured headers are also populated. See the following example:

    ```
    {
    ```

```
  "headers": {
    ...
    "X-Client-Ssl-Cn": "O=Example Client Cert Org,CN=Example Client Cert
CN",
    "X-Client-Ssl-Issuer": "CN=Example Client Root CA CN,O=Example Client
Root CA ORG",
    ...
  }
}
```

Default Domain Scenario

Prerequisites

- The Istio and API Gateway modules are added to your Kyma cluster by default. See Adding and Deleting a Kyma Module [page 3511].

Procedure

1. Create a namespace with enabled Istio sidecar proxy injection.

   ```
   kubectl create ns test
   kubectl label namespace test istio-injection=enabled --overwrite
   ```

2. Export the following domain names as environment variables. Replace `my-own-domain.example.com` with the name of your domain. You can adjust the following values as needed.

   ```
   PARENT_DOMAIN=$(kubectl get gateway -n kyma-system kyma-gateway -o
   jsonpath='{.spec.servers[0].hosts[0]}' | sed 's/\*\.//')
   SUBDOMAIN="mtls.${PARENT_DOMAIN}"
   GATEWAY_DOMAIN="*.${SUBDOMAIN}"
   WORKLOAD_DOMAIN="httpbin.${SUBDOMAIN}"
   echo "Parent Domain: ${PARENT_DOMAIN}"
   echo "Subdomain: ${SUBDOMAIN}"
   echo "Gateway Domain: ${GATEWAY_DOMAIN}"
   echo "Workload Domain: ${WORKLOAD_DOMAIN}"
   ```

Placeholder	Example domain name	Description
`PARENT_DOMAIN`	`my-default-domain.kyma.ondemand.com`	The default domain of your Kyma cluster retrieved from the default TLS Gateway `kyma-gateway`.

Placeholder	Example domain name	Description
`SUBDOMAIN`	`mtls.my-default-domain.kyma.ondemand.com`	A subdomain created under the parent domain, specifically for the mTLS Gateway. Having a separate subdomain is required if you use the default domain of your Kyma cluster, as the parent domain name is already assigned to the TLS Gateway `kyma-gateway` installed in your cluster by default.
`GATEWAY_DOMAIN`	`*.mtls.my-default-domain.kyma.ondemand.com`	A wildcard domain covering all possible subdomains under the mTLS subdomain. When configuring the Gateway, this allows you to expose workloads on multiple hosts (for example, `httpbin.mtls.my-default-domain.kyma.ondemand.com`, `test.httpbin.mtls.my-default-domain.kyma.ondemand.com`) without creating separate Gateway rules for each one.
`WORKLOAD_DOMAIN`	`httpbin.mtls.my-default-domain.kyma.ondemand.com`	The specific domain assigned to your workload.

3. Create the server's certificate.

   You use a Certificate CR to request and manage Let's Encrypt certificates from your Kyma cluster. When you create a Certificate CR, one of Gardener's operators detects it and creates an ACME request to Let's Encrypt requesting certificate for the specified domain names. The issued certificate is stored in an automatically created Kubernetes Secret, which name you specify in the Certificate's `secretName` field. For more information, see Manage certificates with Gardener for public domain .

   ```
   cat <<EOF | kubectl apply -f -
   apiVersion: cert.gardener.cloud/v1alpha1
   kind: Certificate
   metadata:
     name: domain-certificate
     namespace: "istio-system"
   spec:
     secretName: kyma-mtls
     commonName: "${GATEWAY_DOMAIN}"
     issuerRef:
       name: garden
   EOF
   ```

   To verify that the Secret with Gateway certificates is created, run:

   ```
   kubectl get secret -n istio-system kyma-mtls
   ```

4. Prepare the client's certificates.

> ⚠ Caution
>
> For production deployments, use trusted certificate authorities to ensure proper security and automatic certificate management.

a. Create the client's root CA.

```
CLIENT_ROOT_CA_KEY_FILE="client_root_ca_cn.key"
CLIENT_ROOT_CA_CRT_FILE="client_root_ca_cn.crt"
openssl req -x509 -sha256 -nodes -days 365 -newkey rsa:2048 -subj "/
O=Example Client Root CA ORG/CN=Example Client Root CA CN" -keyout "$
{CLIENT_ROOT_CA_KEY_FILE}" -out "${CLIENT_ROOT_CA_CRT_FILE}"
```

b. Create the client's certificate.

```
CLIENT_CERT_CRT_FILE="client_cert_cn.crt"
CLIENT_CERT_CSR_FILE="client_cert_cn.csr"
CLIENT_CERT_KEY_FILE="client_cert_cn.key"
openssl req -out "${CLIENT_CERT_CSR_FILE}" -newkey rsa:2048 -nodes -keyout
"${CLIENT_CERT_KEY_FILE}" -subj "/CN=Example Client Cert CN/O=Example
Client Cert Org"
```

c. Sign the client's certificate.

```
openssl x509 -req -days 365 -CA "${CLIENT_ROOT_CA_CRT_FILE}" -CAkey "$
{CLIENT_ROOT_CA_KEY_FILE}" -set_serial 0 -in "${CLIENT_CERT_CSR_FILE}"
-out "${CLIENT_CERT_CRT_FILE}"
```

5. Create a Secret with Client CA Cert for mTLS Gateway. For more information on the convention that the Secret must use, see Key Convention .

```
kubectl create secret generic -n istio-system "kyma-mtls-cacert" --from-
file=cacert="${CLIENT_ROOT_CA_CRT_FILE}"
```

6. Create an mTLS Gateway.

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: kyma-mtls-gateway
  namespace: test
spec:
  selector:
    app: istio-ingressgateway
    istio: ingressgateway
  servers:
    - port:
        number: 443
        name: mtls
        protocol: HTTPS
      tls:
        mode: MUTUAL
        credentialName: kyma-mtls
      hosts:
        - "${GATEWAY_DOMAIN}"
EOF
```

7. To expose your workload, create an APIRule custom resource.

   You can cofigure the APIRule to append the headers `X-CLIENT-SSL-CN: '%DOWNSTREAM_PEER_SUBJECT%'`, `X-CLIENT-SSL-ISSUER: '%DOWNSTREAM_PEER_ISSUER%'`, and `X-CLIENT-SSL-SAN: '%DOWNSTREAM_PEER_URI_SAN%'` to the request. These headers provide the

upstream (your workload) with the downstream (authenticated client's) identity. This is optional configuration is commonly used in mTLS use cases. For more information about these values, see Envoy Access logging .

See an example APIRule that exposes the following sample HTTPBin Service:

Sample Code

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: httpbin
  namespace: test
---
apiVersion: v1
kind: Service
metadata:
  name: httpbin
  namespace: test
  labels:
    app: httpbin
    service: httpbin
spec:
  ports:
  - name: http
    port: 8000
    targetPort: 80
  selector:
    app: httpbin
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
  namespace: test
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
      labels:
        app: httpbin
        version: v1
    spec:
      serviceAccountName: httpbin
      containers:
      - image: docker.io/kennethreitz/httpbin
        imagePullPolicy: IfNotPresent
        name: httpbin
        ports:
        - containerPort: 80
```

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: httpbin-mtls
  namespace: test
spec:
  gateway: test/kyma-mtls-gateway
  hosts:
    - "${WORKLOAD_DOMAIN}"
  rules:
```

```
    - methods:
        - GET
      noAuth: true
      path: /*
      timeout: 300
      request:
        headers:
          X-CLIENT-SSL-CN: '%DOWNSTREAM_PEER_SUBJECT%'
          X-CLIENT-SSL-ISSUER: '%DOWNSTREAM_PEER_ISSUER%'
          X-CLIENT-SSL-SAN: '%DOWNSTREAM_PEER_URI_SAN%'
  service:
    name: httpbin
    port: 8000
```

8. Test the mTLS connection.
   a. Call the workload without providing client certificates.

      ```
      curl --fail --verbose "https://${WORKLOAD_DOMAIN}/headers?show_env=true"
      ```

      You get cURL 56 error, which indicates that a certificate is not provided by the client. See the following example:

      ```
      curl: (56) OpenSSL SSL_read: OpenSSL/3.5.2: error:0A00045C:SSL
      routines::tlsv13 alert certificate required, errno 0
      ```

   b. Run the following curl command, which specifies the client's certificate and public key:

      ```
      curl --fail --verbose \
        --key "${CLIENT_CERT_KEY_FILE}" \
        --cert "${CLIENT_CERT_CRT_FILE}" \
        "https://${WORKLOAD_DOMAIN}/headers?show_env=true"
      ```

      If successful, you get code `200` in response. The configured headers are also populated. See the following example:

      ```
      {
        "headers": {
          ...
          "X-Client-Ssl-Cn": "O=Example Client Cert Org,CN=Example Client Cert
      CN",
          "X-Client-Ssl-Issuer": "CN=Example Client Root CA CN,O=Example Client
      Root CA ORG",
          ...
        }
      }
      ```

4.3.1.2.1.3 Mutual TLS Authentication

Learn what mutual TLS (mTLS) is and understand the flow of mTLS authentication in Kyma.

Benefits of Using mTLS

mTLS is a security protocol that ensures that both the client and the server authenticate each other. This two-way authentication ensures a higher level of security compared to simple TLS, where only the client checks the server's identity.

TLS brings the following benefits:

- It reduces the risk of a man-in-the-middle attacks, which occur when an attacker intercepts communication between two parties without their knowledge. The encryption ensures that even if someone retrieves the data, they cannot decrypt or modify it.
- The client is able to verify that the server is actually who they claim to be. For example, when opening SAP Help Portal (the server) your browser (the client) ensures that the certificate that the website presents is valid, confirming you are accessing the official website managed by SAP.

Additionally, with mTLS, the server is able to verify that the client is actually who they claim to be. For example, if you open an internal company website (the client), the application (the server) might request your certificate and only continue the session after it confirms your identity.

Technical Foundations of mTLS

Both TLS and mTLS protocols are based on the concept of asymmetric encryption. Asymmetric encryption involves the use of two keys:

- The private key, which is kept secret
- The public key, which is publicly available

For example, imagine a server that has both a private and a public key. A client encrypts a message using the server's public key and then sends it to the server. Only the server, with its corresponding private key, can decrypt this message. This demonstrates the server's identity because only the genuine owner of the private key could perform the decryption.

In practice, however, it's not possible to collect, manage, and verify the authenticity of many public keys presented by each application you interact with. To resolve the problem, communication protocols use the public-key infrastructure, in which one or more third-parties, known as certificate authorities (CAs), certify ownership of key pairs by issuing digital certificates. A digital certificate includes such information as:

- The public key
- The details on the owner of the certificate
- A signature of the authority that confirms that the certificate's contents are valid
- The certificate's validity period

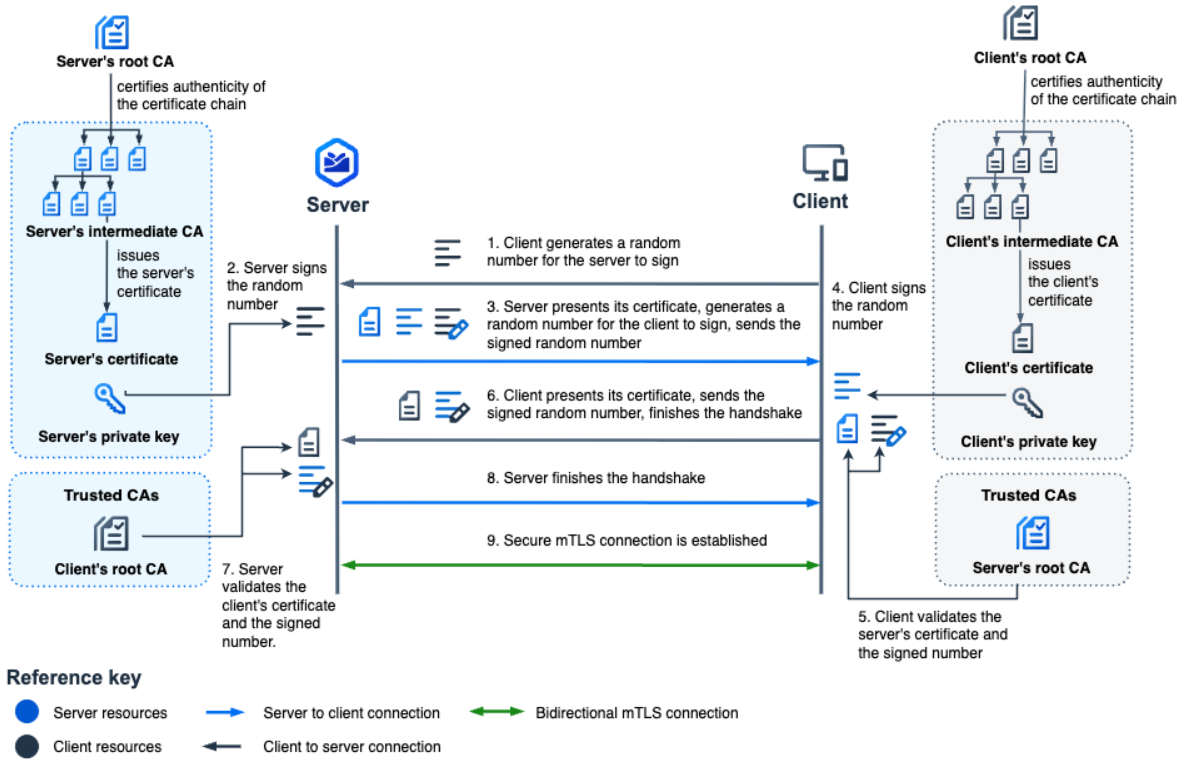
Certificates are usually issued in a hierarchical structure known as a chain of trust. This chain starts with a root Certificate Authority (CA), which is explicitly trusted (for example, by being included in your operating system

or browser). The root CA can sign intermediate CA certificates, and those intermediate CAs can, in turn, sign additional intermediate or end-entity (leaf) certificates. If you trust the root CA and each certificate in the chain is properly signed by its issuer, you can trust the end-entity certificate at the bottom of the chain.

The mTLS Authentication Flow

When the communication is two-way, like in the case of mTLS, the following prerequisites must be met:

- **Private keys**: Both the client and the server are required to have their respective private keys. The keys are kept in secret.
- **Signed certificates**: Both the client and the server are required to present signed certificates.
- **Root CAs**: Both the client and the server must trust the validity of the certificate presented by the other party.
  - The client must have the server's root CA certificate installed.
  - The server must have the client's root CA certificate installed.
- **Intermediate CAs**:
  - The server's intermediate CAs must be on the server to ensure the server sends the entire trust chain to the client (except for the server's root CA, which the client must have installed on their side). If the server does not have the intermediate CAs installed, some clients may not trust the server's certificate.
  - Similarly, the client must have its intermediate CA certificates installed (excluding the root CA, which should be installed on the server's side).



When a client attempts to connect to a server, the following steps take place:

1. The client initiates the connection and generates a random number for the server to sign. The client's hello message also includes such information as the supported TLS versions and the supported cipher suites.
2. The server uses its private key, kept in secret, to sign the random number generated by the client.
3. The server presents its certificate, generates a random number for the client to sign, and sends the signed random number. The server's hello message also contains the selected TLS version and cipher suite.
4. The client uses its private key, kept in secret, to sign the random number generated by the server.
5. The client validates the server's certificate against trusted CAs. It also checks if the random number signed by the server can be decrypted using the server's public key stored in the server's certificate. If both these checks are successful, it considers the server's identity valid.
6. The client presents its own certificate and sends the random number signed with its private key. At this point, the client finishes the mTLS handshake.
7. The server validates if the client's certificate is issued by one of the certificate authorities it trusts. It also checks if the random number signed by the client can be decrypted using the client's public key stored in the client's certificate.
8. If both these checks are successful, the server considers the client's identity valid and finishes the mTLS handshake.
9. The authenticated bidirectional mTLS connection is established.

Related Information

Configure an mTLS Gateway [page 1956]

4.3.1.2.1.4 Kyma Gateway

is a preconfigured Istio Gateway CR named `kyma-gateway` located in the `kyma-system` namespace. It describes the ports and protocols exposed for a particular domain.

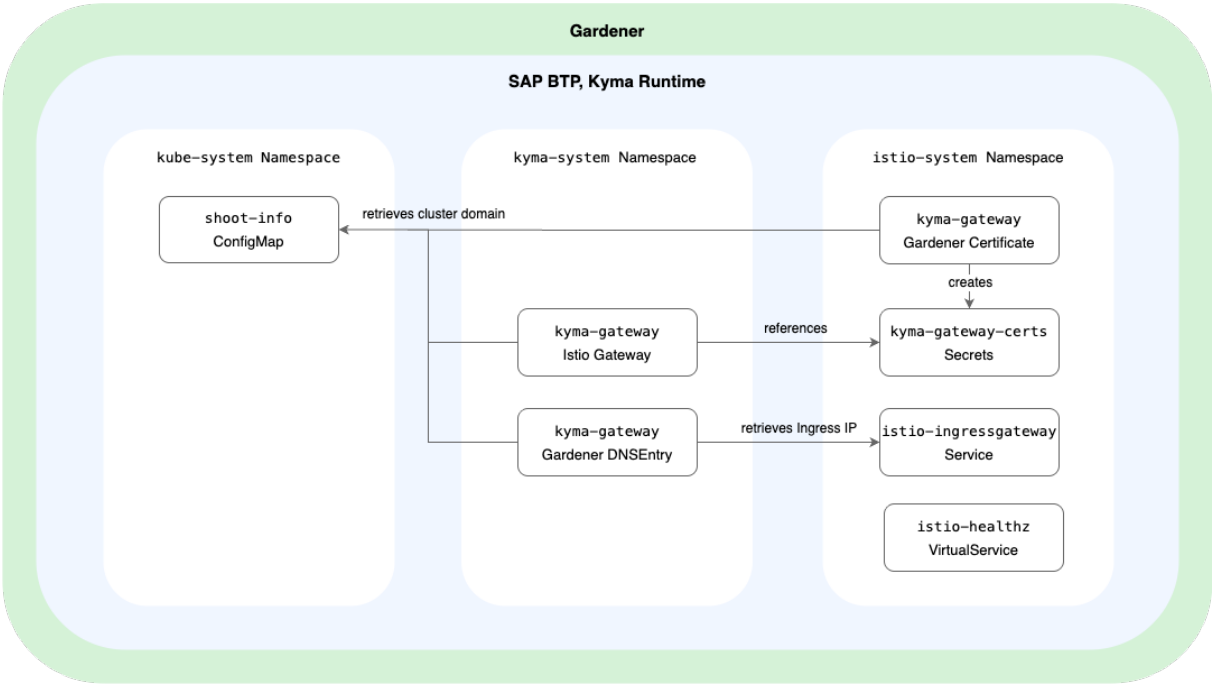
> ⚠ Caution
>
> `Kyma Gateway` is not recommended for production environments. For production use, set up a custom gateway with your own domain and certificate. For more information, see Istio Gateways [page 1969].
>
> For instructions on how to set up a custom gateway, see the following topics:
>
> - For TLS, see Configure a TLS Gateway in SAP BTP, Kyma Runtime [page 1969].
> - For mutual TLS (mTLS), see Mutual TLS Authentication [page 1969] and Configure mTLS Authentication in SAP BTP, Kyma Runtime [page 1969].

Gateway Configuration

Kyma Gateway is configured with the following settings:

- It listens on port 443 (HTTPS) using TLS mode SIMPLE, with a TLS credential supplied from the `kyma-gateway-certs` Secret in the `istio-system` namespace.

- It listens on port 80 (HTTP) and automatically redirects all HTTP requests to HTTPS (responds with a 301 status code).
- It serves all hosts matching the wildcard `*.{domain}`, where `{domain}` is the cluster domain resolved at reconciliation time. In an SAP BTP, Kyma runtime cluster, Kyma Gateway uses the Gardener Shoot domain.
- The gateway selector targets the default Istio ingress gateway (`app: istio-ingressgateway`, `istio: ingressgateway`).
- The `istio-healthz` VirtualService is reconciled in the `istio-system` namespace. It exposes the Istio readiness endpoint at `healthz.{GARDENER_SHOOT_DOMAIN}/healthz/ready` through Kyma Gateway.



DNS Resolution

The cluster domain is resolved from the Gardener `shoot-info` ConfigMap. The operator creates and manages a DNSEntry CR named `kyma-gateway` in the `kyma-system` namespace. The `DNSEntry` points to the external IP addresses or hostnames of the `istio-ingressgateway` LoadBalancer Service in the `istio-system` namespace.

The operator detects the IP stack of the `istio-ingressgateway` Service by inspecting the `spec.ipFamilies` field:

IP families detected	IP stack type	Annotation set on DNSEntry
Single entry: `IPv4`	IPv4	**(none)**
Single entry: `IPv6`	IPv6	`dns.gardener.cloud/ip-stack: ipv6`
Two entries	Dual-stack	`dns.gardener.cloud/ip-stack: dual-stack`

Certificate Management

The operator creates and manages a Gardener Certificate CR named `kyma-tls-cert` in the `istio-system` namespace. The certificate covers all subdomains of the cluster domain (`*.{domain}`) and automatically stores the TLS data in the `kyma-gateway-certs` Secret.

Enable or Disable Kyma Gateway

By default, Kyma Gateway is enabled. To disable it, remove the `enableKymaGateway` field from the APIGateway CR [page 1969] or set `enableKymaGateway` to false:

```
apiVersion: operator.kyma-project.io/v1alpha1
kind: APIGateway
metadata:
  name: default
spec:
  enableKymaGateway: false
```

When you disable Kyma Gateway or delete the APIGateway CR, the operator removes all managed resources: Gateway, DNSEntry, Certificate or certificate Secret, and VirtualService.

You can only disable Kyma Gateway if there are no APIRule or VirtualService resources in the cluster that reference `kyma-system/kyma-gateway`. If such resources exist, the operator returns a warning listing up to five of them. Remove or migrate those resources to a different gateway before disabling Kyma Gateway.

4.3.1.2.2 Expose and Secure Workloads

Use APIRules to expose and secure your workloads in Kyma.

When you expose a workload in Kyma, you make a Kubernetes Service reachable from outside the cluster. By default, workloads run only inside the cluster. To allow external clients (applications, tools, or users in a browser) to call a workload, you publish it under a host name such as `myapp.example.com`, decide which paths and HTTP methods can be called (for example, `GET /headers`), and choose whether the endpoint is open or requires authentication.

What Is an APIRule?

An APIRule is a Kyma custom resource that describes how a workload is exposed.

You use an APIRule to define the following:

- Which host names are used.
- Which Services and ports receive the traffic.
- Which paths and HTTP methods are allowed.
- Which access strategy is used (`noAuth`, `jwt`, or `extAuth`).

The API Gateway module watches APIRules and translates them into Istio configuration, such as `VirtualService` and `AuthorizationPolicy` objects. This means you don't need to create these Istio objects yourself.

For a detailed field description, see APIRule Custom Resource .

APIRule Capabilities

With an APIRule, you can configure the following options:

- Choose an access strategy per path:
  - `jwt` – Protect an endpoint with JSON Web Tokens. See JWT Validation [page 1973].
  - `extAuth` – Delegate authentication and authorization to an external provider. See External Authorization [page 1984].
  - `noAuth` – Expose an endpoint without authentication. See noAuth Configuration [page 1996].

  > → Recommendation
  >
  > For most common scenarios, it's recommended to use the **jwt** access strategy as it provides built-in, token-based authentication and authorization without requiring additional infrastructure.
  >
  > The **extAuth** access strategy allows you to provide your custom authorization and authentication logic. Use this access strategy when the built-in Istio JWT authentication and authorization mechanisms don't meet your specific requirements, and you want to offload the logic to a custom external service.
  >
  > Use **noAuth** for endpoints that must remain openly accessible. This setup is suitable for development and testing environments where security requirements are lower and quick access to services is necessary, or when the data being accessed isn't sensitive and doesn't require strict security measures.

- Use short host names instead of full domain names. The domain suffix is taken from the referenced Istio Gateway. See Short Host Name [page 2010].
- Expose multiple workloads on the same host and route traffic to different Services based on paths and HTTP methods. See Exposing Multiple Workloads on the Same Host [page 2012].
- Expose and secure workloads with mTLS by using an APIRule together with an mTLS-enabled Istio Gateway. See Configure an mTLS Gateway [page 1956].

You can combine these options in a single APIRule. For example, you can expose several Services under one host, protect some paths with `jwt` or `extAuth`, and leave public status and health check endpoints open with `noAuth`.

Exposing Workloads with Istio

In most scenarios, using APIRules to expose and secure workloads is recommended. However, you can also configure access with Istio resources directly, for example, if you need to manage Istio configuration yourself or require a feature that APIRule does not yet support.

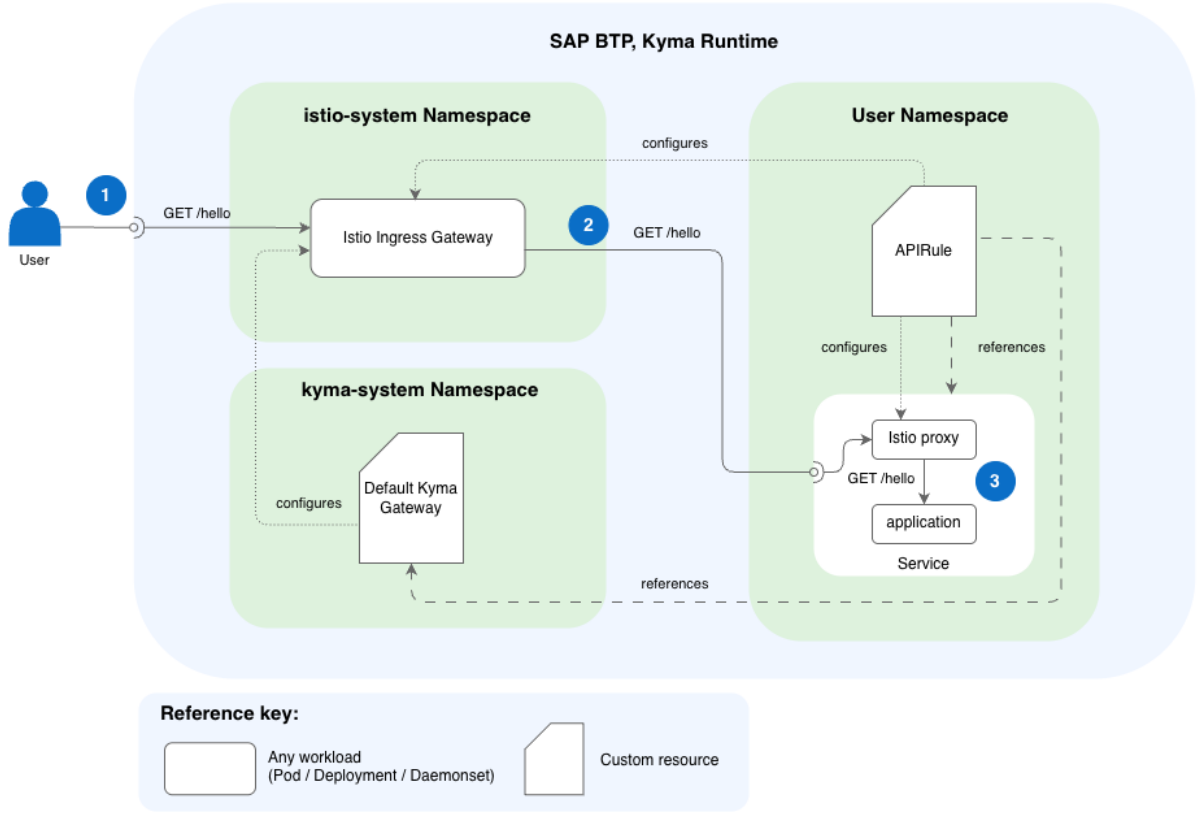
See the following topics:

- Restrict IP-Based Access [page 2015]
- Expose Workloads with Istio VirtualService [page 2020]

4.3.1.2.2.1 JWT Validation

Learn how to configure the `jwt` access strategy in an `APIRule` custom resource to secure your workload with JSON Web Tokens (JWTs).

Request Flow



To expose a workload with an APIRule and enforce JWT validation, you need the following resources:

- A Kyma Gateway that configures the Istio Ingress Gateway. You can use the default Kyma Gateway or define your own in any namespace. For details, see Istio Gateways [page 1946].
- An APIRule with the `jwt` access strategy that references:
  - The Service you want to expose.
  - The Istio Gateway (in this case, Kyma Gateway) to route traffic through.

With this setup, a request is processed as follows:

1. The client sends an HTTP request with a JWT to the exposed hostname, which enters the cluster through the Istio Ingress Gateway.

2. The Istio Ingress Gateway routes the request to the `Service` selected by the `APIRule`.
3. The Istio sidecar proxy next to your workload validates the JWT by using the issuer, JWKS URI, and other settings derived from the `APIRule`.
    - If the token is valid and passes any configured authorization checks, the proxy forwards the request to the workload.
    - If the token is missing or invalid, the proxy rejects the request, and it doesn't reach the application.

Minimal Configuration

Minimal `jwt` configuration secures a path with a single issuer and JWKS URI.

In the following example, the API Gateway module only accepts JWTs issued by `https://example.com`, and Istio uses the configured `jwksUri` to fetch the public keys and verify the token signature. Because no additional authorization rules are defined, any valid token from that issuer is treated as authorized.

```
...
rules:
  - path: /headers
    methods: ["GET"]
    jwt:
      authentications:
        - issuer: https://example.com
          jwksUri: https://example.com/.well-known/jwks.json
```

Configure Authentications

Use the `authentications` section to define where the token comes from.

By default, Istio reads the token from `Authorization: Bearer <token>`. You can override this and use custom headers or query parameters:

```
jwt:
  authentications:
    - issuer: https://example.com
      jwksUri: https://example.com/.well-known/jwks.json
      fromHeaders:
        - name: X-JWT-Assertion
          prefix: "Kyma "
      fromParameters:
        - "access_token"
```

The `authentications` section contains the following fields:

- `issuer` – The expected value of the JWT `iss` claim.
- `jwksUri` – The URL where Istio fetches the JWKS (public keys) to verify the token signature.
- (Optional) `fromHeaders` – One or more HTTP headers to read the token from. Use prefix to remove a static prefix before parsing the token.
- (Optional) `fromParameters` – One or more URL query parameter names to read the token from.

Under the hood, the `authentications` array creates a corresponding `requestPrincipals` array in Istio's `AuthorizationPolicy`. Each entry is formatted as `<ISSUER>/*`.

Configure Authorizations

Use the `authorizations` section to further define the token's scopes and audiences. If authorizations are not defined, a request is authorized as long as the JWT is valid for the configured issuer. If multiple authorization entries are defined, the request is allowed if at least one entry matches.

```
jwt:
  authorizations:
    - requiredScopes: ["read", "write"]
      audiences: ["app1"]
```

The `authorizations` section contains the following fields:

- (Optional) `requiredScopes` – The token must contain all listed scopes in one of the `scp`, `scope`, or `scopes` claims.
- (Optional) `audiences` – The token's `aud` claim must contain all listed audiences.

4.3.1.2.2.1.1 Expose and Secure a Workload with a JWT Using SAP Cloud Identity Services

This procedure explains how to expose a workload on a custom domain and secure it with JSON Web Tokens (JWTs) issued by SAP Cloud Identity Services - Identity Authentication using the Client Credentials grant.

Prerequisites

- You have the Istio and API Gateway modules in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- You have an SAP Cloud Identity Services tenant. See Initial Setup.

Context

Use this procedure to secure your workload with a short-lived JWT. To get the JWT, you must first register an OpenID Connect (OIDC) application in SAP Cloud Identity Services and enable the Client Credentials grant. This generates a client ID (public identifier) and a client secret (confidential credential). A calling system sends these credentials to the OIDC token endpoint over TLS, receiving a signed JWT.

When the client calls your exposed API, it includes the token in the Authorization header using the Bearer scheme. The API Gateway module validates the token based on the configuration you include in the APIRule custom resource (CR). If the validation fails, the Gateway returns `HTTP/2 403 RBAC: access denied` without forwarding the request to the backend Service.

If the validation is successful, the request proceeds to the Service behind the Gateway. At that point, you can implement optional, deeper authorization (examining scopes, audience, or custom claims) inside your application code.

Follow these steps:

1. *Configure a TLS Gateway for Your Custom Domain*.
2. *Create and Configure an OpenID Connect Application*.
3. *Get a JWT*.
4. *Configure JWT Authentication in Kyma*.

Configure a TLS Gateway for Your Custom Domain

To configure the flow in Kyma, you must first provide credentials for a supported DNS provider so Gardener can create and verify the necessary DNS records for your custom wildcard domain. After this, Let's Encrypt issues a trusted TLS certificate. The issued certificate is stored in a Kubernetes Secret referenced by an Istio Gateway, which terminates HTTPS at the cluster edge, so all downstream traffic enters encrypted.

Procedure

1. Create a namespace with enabled Istio sidecar proxy injection.

   ```
   kubectl create ns test
   kubectl label namespace test istio-injection=enabled --overwrite
   ```

2. Export the following domain names as environment variables. Replace `my-own-domain.example.com` with the name of your domain. You can adjust these values as needed.

   ```
   PARENT_DOMAIN="my-own-domain.example.com"
   SUBDOMAIN="tls.${PARENT_DOMAIN}"
   GATEWAY_DOMAIN="*.${SUBDOMAIN}"
   WORKLOAD_DOMAIN="httpbin.${SUBDOMAIN}"
   ```

Placeholder	Example domain name	Description
`PARENT_DOMAIN`	`my-own-domain.example.com`	The domain name available in your public DNS zone.
`SUBDOMAIN`	`tls.my-own-domain.example.com`	A subdomain created under the parent domain, specifically for the TLS Gateway.
`GATEWAY_DOMAIN`	`*.tls.my-own-domain.example.com`	A wildcard domain covering all possible subdomains under the TLS subdomain. When configuring the Gateway, this allows you to expose workloads on multiple hosts (for example, `httpbin.tls.my-own-domain.example.com`, `test.httpbin.tls.my-own-domain.example.com`) without creating separate Gateway rules for each one.

Placeholder	Example domain name	Description
`WORKLOAD_DOMAIN`	`httpbin.tls.my-own-domain.example.co`	The specific domain assigned to your workload.

3. Create a Secret containing credentials for your DNS cloud service provider.

   The information you provide to the data field differs depending on the DNS provider you're using. The DNS provider must be supported by Gardener. To learn how to configure the Secret for a specific provider, follow External DNS Management Guidelines and the External DNS Management examples.

   See an example Secret for the AWS Route 53 DNS provider. `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` are base-64 encoded credentials.

```
apiVersion: v1
kind: Secret
metadata:
  annotations:
    dns.gardener.cloud/class: garden
  name: aws-credentials
  namespace: test
type: Opaque
data:
  AWS_ACCESS_KEY_ID: <your AWS access key>
  AWS_SECRET_ACCESS_KEY: <your AWS secret access key>
  # Optionally, specify the region
  #AWS_REGION: <your AWS region>
  # Optionally, specify the token
  #AWS_SESSION_TOKEN: <your AWS session token>
```

4. Create a DNSProvider resource that references the Secret with your DNS provider's credentials.

   See an example Secret for the AWS Route 53 DNS provider:

```
apiVersion: dns.gardener.cloud/v1alpha1
kind: DNSProvider
metadata:
  name: aws
  namespace: test
spec:
  type: aws-route53
  secretRef:
    name: aws-credentials
  domains:
    include:
    - "${PARENT_DOMAIN}"
```

   For DNSProvider configuration for other providers, see the External DNS Management examples.

   To verify that the DNSProvider is ready, run:

   Sample Code

```
kubectl get DNSProvider -n test {DNSPROVIDER_NAME}
```

5. Get the external access point of the `istio-ingressgateway` service.

   The external access point is either stored in the ingress Gateway's `ip` field (for example, on GCP) or in the ingress Gateway's `hostname` field (for example, on AWS).

```
LOAD_BALANCER_ADDRESS=$(kubectl get services --namespace istio-system istio-
ingressgateway --output jsonpath='{.status.loadBalancer.ingress[0].ip}')
```

```
if [[ -z $LOAD_BALANCER_ADDRESS ]]; then
    LOAD_BALANCER_ADDRESS=$(kubectl get services --
namespace istio-system istio-ingressgateway --output
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
fi
echo "The load balancer address is ${LOAD_BALANCER_ADDRESS}"
```

6. Create a DNSEntry resource.

```
cat <<EOF | kubectl apply -f -
apiVersion: dns.gardener.cloud/v1alpha1
kind: DNSEntry
metadata:
  name: dns-entry
  namespace: test
  annotations:
    dns.gardener.cloud/class: garden
spec:
  dnsName: "${GATEWAY_DOMAIN}"
  ttl: 600
  targets:
    - "${LOAD_BALANCER_ADDRESS}"
EOF
```

To verify that the DNSEntry is ready, run:

Sample Code

```
kubectl get DNSEntry -n test dns-entry
```

7. Create the server's certificate.

You use a Certificate CR to request and manage Let's Encrypt certificates from your Kyma cluster. When you create a Certificate CR, one of Gardener's operators detects it and creates an ACME request to Let's Encrypt requesting certificate for the specified domain names. The issued certificate is stored in an automatically created Kubernetes Secret, which name you specify in the Certificate's `secretName` field. For more information, see Manage certificates with Gardener for public domain .

```
cat <<EOF | kubectl apply -f -
apiVersion: cert.gardener.cloud/v1alpha1
kind: Certificate
metadata:
  name: domain-certificate
  namespace: "istio-system"
spec:
  secretName: custom-tls-secret
  commonName: "${GATEWAY_DOMAIN}"
  issuerRef:
    name: garden
EOF
```

To verify that the Secret with Gateway certificates is ready, run:

Sample Code

```
kubectl get secret -n istio-system custom-tls-secret
```

8. Create a TLS Gateway.

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.istio.io/v1alpha3
```

```
kind: Gateway
metadata:
  name: custom-tls-gateway
  namespace: test
spec:
  selector:
    app: istio-ingressgateway
    istio: ingressgateway
  servers:
    - port:
        number: 443
        name: tls
        protocol: HTTPS
      tls:
        mode: SIMPLE
        credentialName: custom-tls-secret
      hosts:
        - "${GATEWAY_DOMAIN}"
EOF
```

To verify that the TLS Gateway is ready, run:

Sample Code

```
kubectl get gateway -n test custom-tls-gateway
```

Create and Configure an OpenID Connect Application

You need an identity provider to issue JWTs. Create an OpenID Connect application to use SAP Cloud Identity Services as your JWT issuer that manages authentication for your workloads.

Procedure

1. Sign in to the administration console for SAP Cloud Identity Services. See Access Admin Console.
2. Create OpenID Connect Application.
3. Configure OpenID Connect Application for Client Credentials Flow.
4. Configure Secrets for API Authentication.

Get a JWT

Procedure

1. Export the following values as environment variables:

```
CLOUD_IDENTITY_SERVICES_INSTANCE="my-example-tenant.accounts.ondemand.com"
CLIENT_ID="{YOUR-CLIENT-ID}"
```

```
CLIENT_SECRET="{YOUR-CLIENT-SECRET}"
```

2. Export base 64 encoded client ID and client secret.

```
export ENCODED_CREDENTIALS=$(echo -n "$CLIENT_ID:$CLIENT_SECRET" | base64)
```

3. Get `token_endpoint`, `jwks_uri`, and `issuer` from your OpenID application, and save these values as environment variables:

```
TOKEN_ENDPOINT=$(curl -s https://$CLOUD_IDENTITY_SERVICES_INSTANCE/.well-
known/openid-configuration | jq -r '.token_endpoint')
echo token_endpoint: $TOKEN_ENDPOINT
JWKS_URI=$(curl -s https://$CLOUD_IDENTITY_SERVICES_INSTANCE/.well-known/
openid-configuration | jq -r '.jwks_uri')
echo jwks_uri: $JWKS_URI
ISSUER=$(curl -s https://$CLOUD_IDENTITY_SERVICES_INSTANCE/.well-known/openid-
configuration | jq -r '.issuer')
echo issuer: $ISSUER
```

4. Get the JWT access token:

```
ACCESS_TOKEN=$(curl -s -X POST "$TOKEN_ENDPOINT" \
    -d "grant_type=client_credentials" \
    -d "client_id=$CLIENT_ID" \
    -H "Content-Type: application/x-www-form-urlencoded" \
    -H "Authorization: Basic $ENCODED_CREDENTIALS" |  jq -r '.access_token')
echo $ACCESS_TOKEN
```

Configure JWT Authentication in Kyma

To configure JWT authentication, expose your workload using APIRule custom resource (CR) and configure `jwt` as the access strategy. You can either use kubectl or Kyma dashboard.

Procedure

- Use Kyma dashboard.
  a. Go to *Discovery and Network → API Rules* and choose *Create*.
  b. Provide all the required configuration details.
  c. For each path you want to secure with a JWT, add a rule with the following configuration.

Option	Description
Path	Specify the request path.
Methods	Check the allowed access methods.
Access Strategy	JWT
JWT → Authentications	In the *JWT* section, add an authentication with your issuer and JSON Web Key Set URIs.

See the following example of a sample HTTPBin Deployment exposed by an APIRule with JWT authentication:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: httpbin
  namespace: test
---
apiVersion: v1
kind: Service
metadata:
  name: httpbin
  namespace: test
  labels:
    app: httpbin
    service: httpbin
spec:
  ports:
  - name: http
    port: 8000
    targetPort: 80
  selector:
    app: httpbin
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
  namespace: test
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
      labels:
        app: httpbin
        version: v1
    spec:
      serviceAccountName: httpbin
      containers:
      - image: docker.io/kennethreitz/httpbin
        imagePullPolicy: IfNotPresent
        name: httpbin
        ports:
        - containerPort: 80
```

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: httpbin-tls
  namespace: test
spec:
  gateway: test/custom-tls-gateway
  hosts:
    - "${WORKLOAD_DOMAIN}"
  rules:
    - jwt:
        authentications:
          - issuer: ${ISSUER}
            jwksUri: ${JWKS_URI}
```

```
      methods:
        - GET
      path: /*
  service:
    name: httpbin
    port: 8000
```

- Use kubectl.

  To expose and secure your Service, create the APIRule CR. For each path you want to secure with a JWT, add a rule with the `jwt` field and specify the `issuer` and `jwksUri`.

  ```
  rules:
  ...
    - jwt:
        authentications:
          - issuer: ${ISSUER}
            jwksUri: ${JWKS_URI}
  ...
  ```

  See the following example of a sample HTTPBin Deployment exposed by an APIRule with JWT authentication:

  ```
  apiVersion: v1
  kind: ServiceAccount
  metadata:
    name: httpbin
    namespace: test
  ---
  apiVersion: v1
  kind: Service
  metadata:
    name: httpbin
    namespace: test
    labels:
      app: httpbin
      service: httpbin
  spec:
    ports:
    - name: http
      port: 8000
      targetPort: 80
    selector:
      app: httpbin
  ---
  apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: httpbin
    namespace: test
  spec:
    replicas: 1
    selector:
      matchLabels:
        app: httpbin
        version: v1
    template:
      metadata:
        labels:
          app: httpbin
          version: v1
      spec:
        serviceAccountName: httpbin
        containers:
        - image: docker.io/kennethreitz/httpbin
  ```

```
        imagePullPolicy: IfNotPresent
        name: httpbin
        ports:
        - containerPort: 80
```

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: httpbin-tls
  namespace: test
spec:
  gateway: test/custom-tls-gateway
  hosts:
    - "${WORKLOAD_DOMAIN}"
  rules:
    - jwt:
        authentications:
          - issuer: ${ISSUER}
            jwksUri: ${JWKS_URI}
      methods:
        - GET
      path: /*
  service:
    name: httpbin
    port: 8000
```

Results

You have exposed the workload with APIRule custom resource.

1. First, test the connection without the JWT.

   ```
   curl -ik -X GET https://${WORKLOAD_DOMAIN}/headers
   ```

   You get the error `HTTP/2 403 RBAC: access denied`.
2. Now, access the secured workload using the correct JWT.

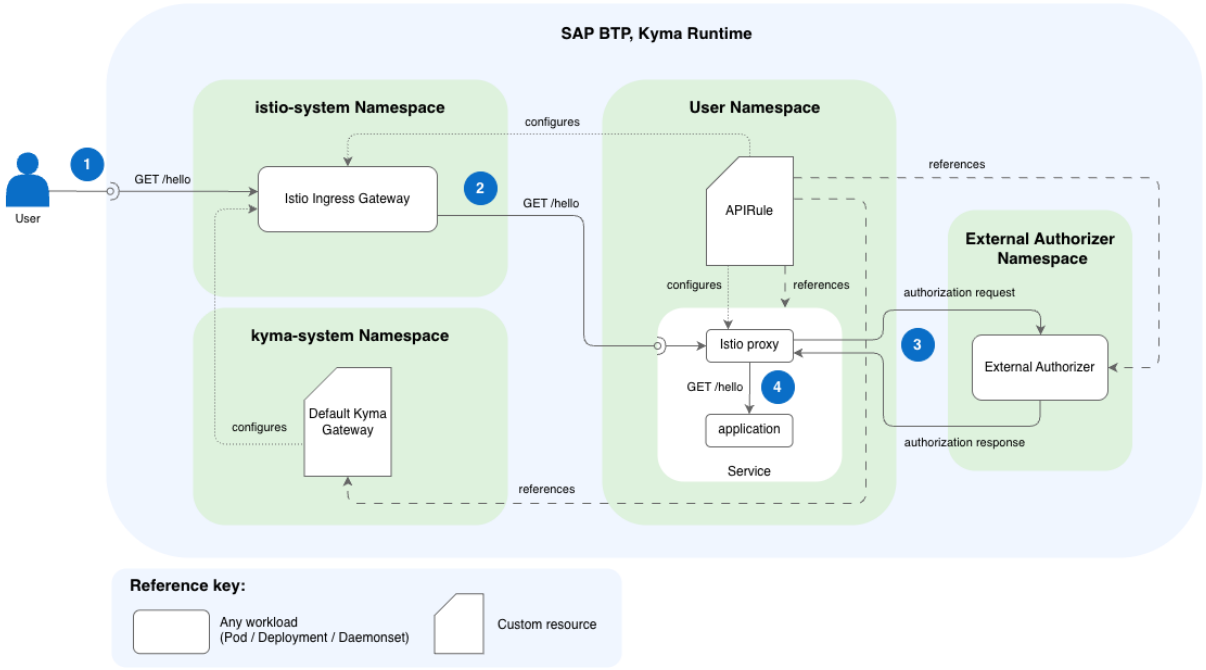
   ```
   curl -ik -X GET https://${WORKLOAD_DOMAIN}/headers --header
   "Authorization:Bearer $ACCESS_TOKEN"
   ```

   You get the `200 OK` response code.

4.3.1.2.2.2 External Authorization

Configure the `extAuth` access strategy in the `APIRule` custom resource (CR) to define custom authentication and authorization logic.

Request Flow



To expose a workload with an `APIRule` and an external authorizer, you need the following resources:

- A Kyma Gateway that configures the Istio Ingress Gateway. You can use the default Kyma Gateway or define your own in any namespace. For details, see Istio Gateways [page 1946].
- An `APIRule` with the `extAuth` access strategy that references:
  - The `Service` you want to expose.
  - The Istio Gateway (in this case, Kyma Gateway) to route traffic through.
  - An external authorization provider, configured in the `authorizers` field.

With this setup, a request is processed as follows:

1. The client sends an HTTP request with a JWT to the exposed hostname, which enters the cluster through the Istio Ingress Gateway.
2. Istio Ingress Gateway routes the request to the `Service` based on the `APIRule` configuration.
3. The Istio proxy next to your application sends the authorization request to the configured external authorization provider.
4. If the external authorization provider allows the request, the Istio proxy forwards the request to the application. If the external authorization provider denies the request, the request does not reach the application.

Minimal Configuration

To use `extAuth`, you must first define the authorization provider in the Istio configuration, most commonly in the Istio CR. For example, the following Istio CR defines a provider named `ext-auth-provider`.

```
apiVersion: operator.kyma-project.io/v1alpha2
kind: Istio
metadata:
  name: default
  namespace: kyma-system
spec:
  config:
    authorizers:
    - headers:
        inCheck:
          include:
          - x-ext-authz
      name: ext-auth-provider
      port: 8000
      service: ext-auth-provider.provider-system.svc.cluster.local
```

Once you define the provider in the Istio configuration, you can reference it in the `APIRule` CR. This configuration allows access to the `/get` path of the `user-service` service. Based on this `APIRule`, a `CUSTOM` Istio `AuthorizationPolicy` is created with the `ext-auth-provider` provider, securing access.

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: ext-authz
  namespace: user-system
spec:
  hosts:
    - na.example.com
  service:
    name: user-service
    port: 8000
  gateway: kyma-system/kyma-gateway
  rules:
    - path: "/get"
      methods: ["GET"]
      extAuth:
        authorizers:
        - ext-auth-provider
```

Securing an Endpoint with `extAuth` and JWT Restrictions

The following configuration allows access to the `/get` path of the `user-service` service, as in the example in the previous section. The access is further restricted by the JWT configuration, specified in the `restrictions` field.

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: ext-authz
  namespace: user-system
spec:
  hosts:
```

```
    - na.example.com
  service:
    name: user-service
    port: 8000
  gateway: kyma-system/kyma-gateway
  rules:
    - path: "/get"
      methods: ["GET"]
      extAuth:
        authorizers:
          - x-ext-authz
        restrictions:
          authentications:
            - issuer: https://example.com
              jwksUri: https://example.com/.well-known/jwks.json
          authorizations:
            - audiences: ["app1"]
```

4.3.1.2.2.2.1 Expose and Secure a Workload with OAuth2 Proxy External Authorizer (Authorization Code Flow)

Learn how to expose and secure a workload using the OAuth2 Proxy external authorizer and the OAuth 2.0 Authorization Code flow. SAP Cloud Identity Services acts as the OAuth2/OIDC Identity Provider (IdP) that authenticates users.

Prerequisites

- You have the Istio and API Gateway modules in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- You have an SAP Cloud Identity Services tenant. See Initial Setup.
- You have installed Helm .

Context

This procedure shows how to implement external authorization for your Kyma workloads using the OAuth 2.0 Authorization Code flow.

When the user visits a URL of your exposed workload, the following steps take place:

1. OAuth2 Proxy redirects the user's browser to the SAP Cloud Identity Services authorization endpoint. This redirection includes several parameters, such as the OAuth2 Proxy's client ID and the callback URL (`https://oauth2-proxy.{YOUR_DOMAIN}/oauth2/callback`) where SAP Cloud Identity Services returns the user after granting or denying access.
2. The user logs in to SAP Cloud Identity Services.

3. After successsful authentication, SAP Cloud Identity Services redirects the browser back to the callback URL. This redirection includes an authorization code and any local state previously supplied by OAuth2 Proxy.
4. OAuth2 Proxy requests an access token from the SAP Cloud Identity Services token endpoint using the authorization code received in the previous step. This request includes OAuth2 Proxy's client ID and client secret for authentication, as well as the callback URL.
5. SAP Cloud Identity Services authenticates OAuth2 Proxy, validates the authorization code, and verifies that the callback URL matches the one you configured in the OpenID Connect application. If everything is valid, SAP Cloud Identity Services responds with an access token, an ID token, and optionally, a refresh token. OAuth2 Proxy then establishes a session and allows for forwarding the original request to your workload.

For more information on the Authorization Code flow, see OAuth 2.0 RFC 6749 and How Authorization Code Flow works .

Follow these steps:

1. *Create and Configure OpenID Connect Application*.
2. *Deploy OAuth2 Proxy as an External Authorizer*.
3. *Expose Your Workload Using extAuth APIRule*.

Create and Configure OpenID Connect Application

In SAP Cloud Identity Services, create an OpenID Connect application and configure it for the Authorization Code flow.

Procedure

1. Sign in to the administration console for SAP Cloud Identity Services. See Access Admin Console.
2. Create OpenID Connect Application.
3. Configure OpenID Connect Application for Authorization Code Flow. When configuring the application, add the `https://oauth2-proxy.{YOUR_DOMAIN}/oauth2/callback` redirect URI and choose the *Authorization Code* grant type.

   The redirect URI is where the IdP sends the user back after a successful login. In this case, replace `{YOUR_DOMAIN}` with the name of the host on which you expose your service in Kyma.
4. Configure a secret for API authentication. See Configure Secrets for API Authentication.

Deploy OAuth2 Proxy as an External Authorizer

Procedure

1. Export the domain name:

   ```
   EXPOSE_DOMAIN=$(kubectl get gateway -n kyma-system kyma-gateway -o
   jsonpath='{.spec.servers[0].hosts[0]}')
   EXPOSE_DOMAIN=${EXPOSE_DOMAIN#*.}
   GATEWAY=kyma-system/kyma-gateway
   ```

   This procedure uses the default domain of your Kyma cluster and the default Gateway. Alternatively, you can replace these values and use your custom domain and Gateway instead.

2. Create a namespace for deploying the OAuth2 Proxy.

   ```
   kubectl create ns oauth2-proxy
   kubectl label namespace oauth2-proxy istio-injection=enabled --overwrite
   ```

3. Add the oauth2-proxy helm chart to your local Helm registry.

   ```
   helm repo add oauth2-proxy https://oauth2-proxy.github.io/manifests
   helm repo update oauth2-proxy
   ```

4. Replace the placeholders and define the OAuth2 Proxy configuration for your authorization server.

   You can adjust this configuration as needed. See the additional configuration parameters .

   ```
   helm upgrade --install oauth2-proxy --namespace oauth2-proxy oauth2-proxy/
   oauth2-proxy \
   --set config.clientID="${CLIENT_ID}" \
   --set config.clientSecret="${CLIENT_SECRET}" \
   --set config.cookieName="" \
   --set config.cookieSecret="$(openssl rand -base64 32 | head -c 32 | base64)" \
   --set extraArgs.provider=oidc \
   --set extraArgs.auth-logging="true" \
   --set extraArgs.cookie-domain="${EXPOSE_DOMAIN}" \
   --set extraArgs.cookie-samesite="lax" \
   --set extraArgs.cookie-secure="false" \
   --set extraArgs.force-json-errors="true" \
   --set extraArgs.login-url="static://401" \
   --set extraArgs.oidc-issuer-url="${TENANT_URL}" \
   --set extraArgs.pass-access-token="true" \
   --set extraArgs.pass-authorization-header="true" \
   --set extraArgs.pass-host-header="true" \
   --set extraArgs.pass-user-headers="true" \
   --set extraArgs.request-logging="true" \
   --set extraArgs.reverse-proxy="true" \
   --set extraArgs.scope="openid email" \
   --set extraArgs.set-authorization-header="true" \
   --set extraArgs.set-xauthrequest="true" \
   --set extraArgs.skip-jwt-bearer-tokens="true" \
   --set extraArgs.skip-oidc-discovery="false" \
   --set extraArgs.skip-provider-button="true" \
   --set extraArgs.standard-logging="true" \
   --set extraArgs.upstream="static://200" \
   --set extraArgs.whitelist-domain="*.${EXPOSE_DOMAIN}:*" \
   --wait
   ```

   Check if the Oauth2 Proxy Pods are running:

   ```
   kubectl --namespace=oauth2-proxy get pods -l "app=oauth2-proxy"
   ```

5. Register OAuth2 Proxy as an authorization provider in the Istio module:

```
kubectl patch istio -n kyma-system default --type merge --patch '{"spec":
{"config":{"authorizers":[{"name":"oauth2-proxy","port":80,"service":"oauth2-
proxy.oauth2-proxy.svc.cluster.local","headers":{"inCheck":{"include":["x-
forwarded-for", "cookie", "authorization"]}}}]}}}'
```

For more information about the fields set above, see the Istio Custom Resource documentation .

Expose Your Workload Using `extAuth` APIRule

To configure OAuth2 Proxy, expose your workload using the APIRule custom resource (CR). Configure `extAuth` as the access strategy and add the `oauth2-proxy` authorizer.

Procedure

- You can either use Kyma dashboard or kubectl.
- Use Kyma dashboard.
  a. Go to *Discovery and Network → API Rules* and choose *Create*.
  b. Provide all the required configuration details.
  c. Add a rule with the following configuration.
     - `Access Strategy`: `extAuth`
     - In `External Authorization`, add the `oauth2-proxy` authorizer.
     - Add allowed methods and the request path.
  d. Choose *Create*.
- Use kubectl.

  To expose and secure your Service, create the APIRule CR. In the rules section, define the `extAuth` field and add the `oauth2-proxy` authorizer.

```
...
  rules:
    - path: /*
      methods: ["GET"]
      extAuth:
        authorizers:
          - oauth2-proxy
...
```

Results

To access your workload, copy the host link defined in your APIRule and open it in a browser. You're redirected to Cloud Identity Services first, where you need to log in. If the login is successful, you can access your application.

Example

See the following example APIRule with `extAuth` authorizer that exposes the sample HTTPBin Deployment:

Example

1. Create the `httpbin-system` namespace and deploy a sample HTTPBin Deployment.

```
kubectl create ns httpbin-system
kubectl label namespace httpbin-system istio-injection=enabled
kubectl apply -f \
https://raw.githubusercontent.com/istio/istio/master/samples/httpbin/
httpbin.yaml \
-n httpbin-system
```

2. Expose the workload with an APIRule using the extAuth access strategy.

```
cat <<EOF | kubectl apply -f -
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: httpbin
  namespace: httpbin-system
spec:
  gateway: ${GATEWAY}
  hosts:
    - httpbin.${EXPOSE_DOMAIN}
  service:
    name: httpbin
    port: 8000
  rules:
    - path: /*
      methods: ["GET"]
      extAuth:
        authorizers:
          - oauth2-proxy
EOF
```

   Check if the APIRule's status is ready:

```
kubectl --namespace=oauth2-proxy get apirules httpbin -n httpbin-system
```

3. When you open `httpbin.${EXPOSE_DOMAIN}`, the page displays HTTPBin endpoints.

4.3.1.2.2.2.2 Expose and Secure a Workload with OAuth2 Proxy External Authorizer (Client Credentials Flow)

Learn how to expose and secure a workload using the OAuth2 Proxy external authorizer and the OAuth 2.0 Client Credentials flow. SAP Cloud Identity Services acts as the OAuth2/OIDC Identity Provider (IdP) that issues JSON Web Tokens (JWTs).

Prerequisites

- You have the Istio and API Gateway modules in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- You have an SAP Cloud Identity Services tenant. See Initial Setup.
- You have installed Helm .

Context

This procedure shows how to implement external authorization for your Kyma workloads using the OAuth2 Client Credentials flow. In this context, your application exchanges its credentials directly with SAP Cloud Identity Services to obtain an access token.

When the URL of your exposed workload is requested, the following steps take place:

1. The client sends its client ID and client secret to SAP Cloud Identity Services.
2. SAP Cloud Identity Services validates these credentials and responds with a JWT that contains specific scopes defining what resources the client can access.
3. The client includes this JWT in the Authorization header when making requests to your secured Kyma workload.
4. OAuth2 Proxy validates the JWT against SAP Cloud Identity Services and either allows the request to proceed to your workload or returns `401 Unauthorized` in response.

Unlike the Authorization Code flow that requires browser redirects and user consent, the Client Credentials flow enables direct server-to-server communication using only the application's own credentials. For more information, see OAuth 2.0 RFC 6749 and Client Credentials Flow .

Follow these steps:

1. *Create and Configure OpenID Connect Application*.
2. Get a JWT [page 1992].
3. *Deploy OAuth2 Proxy as an External Authorizer*.
4. *Expose Your Workload Using extAuth APIRule*.

Create and Configure OpenID Connect Application

You need an identity provider to issue JWTs. Creating an OpenID Connect application allows SAP Cloud Identity Services to act as your issuer and manage authentication for your workloads.

Procedure

1. Sign in to the administration console for SAP Cloud Identity Services. See Access Admin Console.
2. Create OpenID Connect Application.
3. Configure OpenID Connect Application for Client Credentials Flow.
4. Configure Secrets for API Authentication.

Get a JWT

Procedure

1. Export the following values as environment variables:

```
CLOUD_IDENTITY_SERVICES_INSTANCE="my-example-tenant.accounts.ondemand.com"
CLIENT_ID="{YOUR-CLIENT-ID}"
CLIENT_SECRET="{YOUR-CLIENT-SECRET}"
```

2. Export base 64 encoded client ID and client secret.

```
export ENCODED_CREDENTIALS=$(echo -n "$CLIENT_ID:$CLIENT_SECRET" | base64)
```

3. Get `token_endpoint`, `jwks_uri`, and `issuer` from your OpenID application, and save these values as environment variables:

```
TOKEN_ENDPOINT=$(curl -s https://$CLOUD_IDENTITY_SERVICES_INSTANCE/.well-
known/openid-configuration | jq -r '.token_endpoint')
echo token_endpoint: $TOKEN_ENDPOINT
JWKS_URI=$(curl -s https://$CLOUD_IDENTITY_SERVICES_INSTANCE/.well-known/
openid-configuration | jq -r '.jwks_uri')
echo jwks_uri: $JWKS_URI
ISSUER=$(curl -s https://$CLOUD_IDENTITY_SERVICES_INSTANCE/.well-known/openid-
configuration | jq -r '.issuer')
echo issuer: $ISSUER
```

4. Get the JWT access token:

```
ACCESS_TOKEN=$(curl -s -X POST "$TOKEN_ENDPOINT" \
    -d "grant_type=client_credentials" \
    -d "client_id=$CLIENT_ID" \
    -H "Content-Type: application/x-www-form-urlencoded" \
    -H "Authorization: Basic $ENCODED_CREDENTIALS" |  jq -r '.access_token')
echo $ACCESS_TOKEN
```

Deploy OAuth2 Proxy as an External Authorizer

Procedure

1. Export the domain name:

   ```
   EXPOSE_DOMAIN=$(kubectl get gateway -n kyma-system kyma-gateway -o
   jsonpath='{.spec.servers[0].hosts[0]}')
   EXPOSE_DOMAIN=${EXPOSE_DOMAIN#*.}
   GATEWAY=kyma-system/kyma-gateway
   ```

   This procedure uses the default domain of your Kyma cluster and the default Gateway. Alternatively, you can replace these values and use your custom domain and Gateway instead.

2. Create a namespace for deploying the OAuth2 Proxy.

   ```
   kubectl create ns oauth2-proxy
   kubectl label namespace oauth2-proxy istio-injection=enabled --overwrite
   ```

3. Add the oauth2-proxy helm chart to your local Helm registry.

   ```
   helm repo add oauth2-proxy https://oauth2-proxy.github.io/manifests
   helm repo update oauth2-proxy
   ```

4. Replace the placeholders and define the OAuth2 Proxy configuration for your authorization server.

   You can adjust this configuration as needed. See the additional configuration parameters .

   ```
   helm upgrade --install oauth2-proxy --namespace oauth2-proxy oauth2-proxy/
   oauth2-proxy \
   --set config.clientID="${CLIENT_ID}" \
   --set config.clientSecret="${CLIENT_SECRET}" \
   --set config.cookieName="" \
   --set config.cookieSecret="$(openssl rand -base64 32 | head -c 32 | base64)" \
   --set extraArgs.provider=oidc \
   --set extraArgs.auth-logging="true" \
   --set extraArgs.cookie-domain="${EXPOSE_DOMAIN}" \
   --set extraArgs.cookie-samesite="lax" \
   --set extraArgs.cookie-secure="false" \
   --set extraArgs.force-json-errors="true" \
   --set extraArgs.login-url="static://401" \
   --set extraArgs.oidc-issuer-url="${TENANT_URL}" \
   --set extraArgs.pass-access-token="true" \
   --set extraArgs.pass-authorization-header="true" \
   --set extraArgs.pass-host-header="true" \
   --set extraArgs.pass-user-headers="true" \
   --set extraArgs.request-logging="true" \
   --set extraArgs.reverse-proxy="true" \
   --set extraArgs.scope="openid email" \
   --set extraArgs.set-authorization-header="true" \
   --set extraArgs.set-xauthrequest="true" \
   --set extraArgs.skip-jwt-bearer-tokens="true" \
   --set extraArgs.skip-oidc-discovery="false" \
   --set extraArgs.skip-provider-button="true" \
   --set extraArgs.standard-logging="true" \
   --set extraArgs.upstream="static://200" \
   --set extraArgs.whitelist-domain="*.${EXPOSE_DOMAIN}:*" \
   --wait
   ```

   Check if the Oauth2 Proxy Pods are running:

   ```
   kubectl --namespace=oauth2-proxy get pods -l "app=oauth2-proxy"
   ```

5. Register OAuth2 Proxy as an authorization provider in the Istio module:

```
kubectl patch istio -n kyma-system default --type merge --patch '{"spec":
{"config":{"authorizers":[{"name":"oauth2-proxy","port":80,"service":"oauth2-
proxy.oauth2-proxy.svc.cluster.local","headers":{"inCheck":{"include":["x-
forwarded-for", "cookie", "authorization"]}}}]}}}'
```

For more information about the fields set above, see the Istio Custom Resource documentation .

Expose Your Workload Using `extAuth` APIRule

To configure OAuth2 Proxy, expose your workload using APIRule custom resource (CR). Configure `extAuth` as the access strategy and add the `oauth2-proxy` authorizer.

Procedure

- You can either use Kyma dashboard or kubectl.
- Use Kyma dashboard.
  a. Go to *Discovery and Network → API Rules* and choose *Create*.
  b. Provide all the required configuration details.
  c. Add a rule with the following configuration.
     - `Access Strategy`: `extAuth`
     - In `External Authorization`, add the `oauth2-proxy` authorizer.
     - Add allowed methods and the request path.
  d. Choose *Create*.
- Use kubectl.

  To expose and secure your Service, create the APIRule CR. In the rules section, define the `extAuth` field and add the `oauth2-proxy` authorizer.

```
...
  rules:
    - path: /*
      methods: ["GET"]
      extAuth:
        authorizers:
          - oauth2-proxy
...
```

Example

See the following example APIRule with `extAuth` authorizer that exposes the sample HTTPBin Deployment:

Example

1. Create the `httpbin-system` namespace and deploy a sample HTTPBin Deployment.

```
kubectl create ns httpbin-system
kubectl label namespace httpbin-system istio-injection=enabled
kubectl apply -f \
https://raw.githubusercontent.com/istio/istio/master/samples/httpbin/
httpbin.yaml \
-n httpbin-system
```

2. Expose the workload with an APIRule using the extAuth access strategy.

```
cat <<EOF | kubectl apply -f -
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: httpbin
  namespace: httpbin-system
spec:
  gateway: ${GATEWAY}
  hosts:
    - httpbin.${EXPOSE_DOMAIN}
  service:
    name: httpbin
    port: 8000
  rules:
    - path: /*
      methods: ["GET"]
      extAuth:
        authorizers:
          - oauth2-proxy
EOF
```

   Check if the APIRule's status is ready:

```
kubectl --namespace=oauth2-proxy get apirules httpbin -n httpbin-system
```

3. To test the connection, first, do not provide the JWT.

```
curl -ik -X GET https://httpbin.${EXPOSE_DOMAIN}/headers
```

   You get the error `401 Unauthorized`.

4. Now, access the secured workload using the correct JWT.

```
curl -ik -X GET https://httpbin.${EXPOSE_DOMAIN}/headers --header
"Authorization:Bearer ${ACCESS_TOKEN}"
```

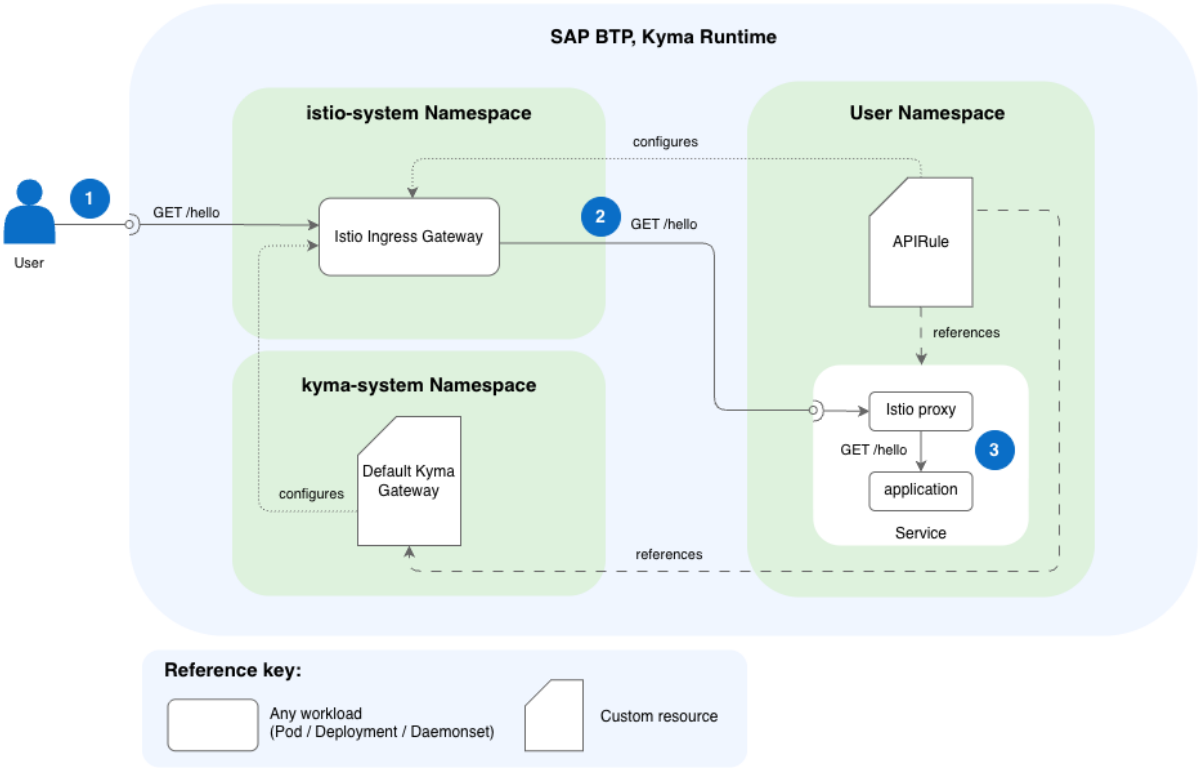
   You get code `200 OK` in response.

4.3.1.2.2.3 `noAuth` Configuration

Use the `noAuth` access strategy to expose a workload with no authorization or authentication configurations.

> ⚠ Caution
>
> Exposing a workload to the outside world is always a potential security vulnerability, so be careful. In a production environment, secure the workload you expose with `jwt` or `extAuth`.

Request Flow



To expose a workload with an `APIRule` and `noAuth`, you need the following resources:

- An Istio Gateway resource that configures the Istio Ingress Gateway. You can use the default Kyma Gateway or define your own in any namespace. For details, see Istio Gateways [page 1946].
- An `APIRule` with the `noAuth` access strategy that references:
  - The `Service` you want to expose.
  - The Istio Gateway (in this case, Kyma Gateway) to route traffic through.

With this setup, a request is processed as follows:

1. The client sends an HTTPS request to the exposed hostname, which enters the cluster's Istio Ingress Gateway.
2. Istio Ingress Gateway routes the request straight to the `Service` based on the `APIRule` configuration.

3. The Istio sidecar proxy next to your application forwards the request directly to the application, without performing any authentication or authorization checks.

Configuring `noAuth`

The `noAuth` access strategy provides a simple configuration for exposing workloads. Use it when you need to allow access to specific HTTP methods without any authentication or authorization checks. This setup is suitable for development and testing environments where security requirements are lower and quick access to services is necessary, or when the data being accessed is not sensitive and does not require strict security measures.

See the following sample configuration:

```
...
rules:
  - path: /headers
    methods: ["GET"]
    noAuth: true
```

4.3.1.2.2.3.1 Expose a Workload with noAuth

Learn how to expose an unsecured workload using the `noAuth` access strategy and call its endpoints.

Prerequisites

- You have the Istio and API Gateway modules in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- You have a deployed workload.

  > ⓘ Note
  >
  > To expose a workload using APIRule in version `v2`, the workload must be part of the Istio service mesh. See Enabling Istio Sidecar Proxy Injection [page 1892].

- To set up a custom Gateway, see Configure a TLS Gateway [page 1948]. Alternatively, you can use the default domain of your Kyma cluster and the default Gateway `kyma-system/kyma-gateway`. See Istio Gateways [page 1946].

  > → Tip
  >
  > To learn what the default domain of your Kyma cluster is, run the following command:
  >
  > ```
  > kubectl get gateway -n kyma-system kyma-gateway -o
  > jsonpath='{.spec.servers[0].hosts}'
  > ```

Context

The intended functionality of the noAuth access strategy is to provide a simple configuration for exposing workloads. It allows public access to your workload without any authentication or authorization checks. This is useful for:

- Development and testing environments
- Public APIs that don't require authentication
- Services that implement their own authentication logic

> ⚠ Caution
>
> Exposing a workload to the outside world is a potential security vulnerability, so be careful. In a production environment, always secure the workload you expose.

To expose a workload without authentication, create an APIRule with `noAuth: true` configured for each path you want to expose publicly.

Procedure

- You can use either Kyma dashboard or kubectl.
- Use Kyma dashboard.
  a. In a namespace of your choice, go to *Discovery and Network* → *API Rules* and choose *Create*.
  b. Provide all the required configuration details.
  c. For each path you want to expose with `noAuth`, add a rule with the following configuration.
     - `Access Strategy`: noAuth
     - Add allowed methods and the request path.
  d. Choose *Create*.

  You can access your Service at `https://${WORKLOAD_DOMAIN}`.
  - Send a `GET` request to the exposed workload:

    ```
    curl -ik -X GET "https://${WORKLOAD_DOMAIN}/ip"
    ```
  - Send a `POST` request to the exposed workload:

    ```
    curl -ik -X POST "https://${WORKLOAD_DOMAIN}/post" -d "test data"
    ```

  If successful, the calls return `200 OK` response codes.
- Use kubectl.

  Replace the placeholders and apply the following configuration. Adjust the rules sections as needed.

  ```
  cat <<EOF | kubectl apply -f -
  apiVersion: gateway.kyma-project.io/v2
  kind: APIRule
  metadata:
    name: ${APIRULE_NAME}
    namespace: ${NAMESPACE}
  spec:
  ```

```
  hosts:
    - ${SUBDOMAIN}.${PARENT_DOMAIN}
  service:
    name: ${SERVICE_NAME}
    namespace: ${SERVICE_NAMESPACE}
    port: ${SERVICE_PORT}
  gateway: ${GATEWAY_NAMESPACE}/${GATEWAY_NAME}
  rules:
    - path: /post
      methods: ["POST"]
      noAuth: true
    - path: /{**}
      methods: ["GET"]
      noAuth: true
EOF
```

You can access your Service at `https://${WORKLOAD_DOMAIN}`.

- Send a `GET` request to the exposed workload:

  ```
  curl -ik -X GET "https://${WORKLOAD_DOMAIN}/ip"
  ```

- Send a `POST` request to the exposed workload:

  ```
  curl -ik -X POST "https://${WORKLOAD_DOMAIN}/post" -d "test data"
  ```

If successful, the calls return `200 OK` response codes.

Example

Choose one of the following methods to create an APIRule that exposes a sample HTTPBin Deployment. You can either follow the example using the Kyma dashboard or the one using kubectl.

- Use Kyma dashboard:

  ### Example

  Go to *Namespaces* and create a namespace with enabled Istio sidecar proxy injection.

  1. Select *+ Upload YAML*, paste the following cofiguration and upload it.

     ```
     apiVersion: v1
     kind: ServiceAccount
     metadata:
       name: httpbin
     ---
     apiVersion: v1
     kind: Service
     metadata:
       name: httpbin
       labels:
         app: httpbin
         service: httpbin
     spec:
       ports:
       - name: http
         port: 8000
         targetPort: 80
       selector:
     ```

```
    app: httpbin
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
      labels:
        app: httpbin
        version: v1
    spec:
      serviceAccountName: httpbin
      containers:
      - image: docker.io/kennethreitz/httpbin
        imagePullPolicy: IfNotPresent
        name: httpbin
        ports:
        - containerPort: 80
```

2. Go to *Discovery and Network* → *API Rules* and select *Create*.
3. Provide the name of the APIRule CR.
4. In the *Service* section, add the name httpbin and port 8000.
5. Use the default Gateway kyma-system/kyma-gateway.
   Alternatively, you can replace these values and use your custom Gateway. See Configure a TLS Gateway [page 1948].
6. Add the host `httpbin.${PARENT_DOMAIN}`.
   To learn what your default parent domain is, go to the Kyma Environment section of your subaccount overview, and copy the part of the *APIServerURL* link after `https://api.`.
   For example, if your *APIServerURL* link is `https://api.c123abc.kyma.ondemand.com`, use httpbin.c123abc.kyma.ondemand.com as the host. If you use a custom Gateway, add the host configured in the Gateway.
7. Add a rule with the following configuration:

Option	Description
`Path`	/post
`Handler`	noAuth
`Methods`	POST

8. Add one more rule with the following configuration:

Option	Description
`Path`	/{**}
`Handler`	noAuth
`Methods`	GET

9. Choose *Create*.
10. Access the Service at `https://${WORKLOAD_DOMAIN}`.

- Use kubectl:

Example

Use kubectl:

1. Create a namespace with enabled Istio sidecar proxy injection.

```
NAMESPACE="test"
kubectl create ns "${NAMESPACE}"
kubectl label namespace "${NAMESPACE}" istio-injection=enabled --
overwrite
```

2. Get the default domain of your Kyma cluster.

```
PARENT_DOMAIN=$(kubectl get configmap -n kube-system shoot-info -o
jsonpath="{.data.domain}")
WORKLOAD_DOMAIN="httpbin.${PARENT_DOMAIN}"
GATEWAY="kyma-system/kyma-gateway"
echo "Parent domain: ${PARENT_DOMAIN}"
echo "Workload domain: ${WORKLOAD_DOMAIN}"
echo "Gateway namespace and name: ${GATEWAY}"
```

   This procedure uses the default domain of your Kyma cluster and the default Gateway. Alternatively, you can replace these values and use your custom domain and Gateway instead. See Istio Gateways [page 1946] and Configure a TLS Gateway [page 1948].

3. Deploy a sample instance of the HTTPBin Service.

```
cat <<EOF | kubectl -n "${NAMESPACE}" apply -f -
apiVersion: v1
kind: ServiceAccount
metadata:
  name: httpbin
---
apiVersion: v1
kind: Service
metadata:
  name: httpbin
  labels:
    app: httpbin
    service: httpbin
spec:
  ports:
  - name: http
    port: 8000
    targetPort: 80
  selector:
    app: httpbin
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
```

```
      labels:
        app: httpbin
        version: v1
    spec:
      serviceAccountName: httpbin
      containers:
      - image: docker.io/kennethreitz/httpbin
        imagePullPolicy: IfNotPresent
        name: httpbin
        ports:
        - containerPort: 80
EOF
```

To verify if an instance of the HTTPBin Service is successfully created, run:

```
kubectl get pods -l app=httpbin -n "${NAMESPACE}"
```

If successful, you get a result similar to this one:

```
NAME                 READY    STATUS     RESTARTS    AGE
httpbin-{SUFFIX}     2/2      Running    0           96s
```

4. Expose the workload with an APIRule using the `noAuth` access strategy.

```
cat <<EOF | kubectl apply -n "${NAMESPACE}" -f -
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: apirule-noauth
spec:
  hosts:
    - ${WORKLOAD_DOMAIN}
  service:
    name: httpbin
    namespace: ${NAMESPACE}
    port: 8000
  gateway: ${GATEWAY}
  rules:
    - path: /post
      methods: ["POST"]
      noAuth: true
    - path: /{**}
      methods: ["GET"]
      noAuth: true
EOF
```

5. Check if the APIRule's status is ready:

```
kubectl get apirules apirule-noauth -n "${NAMESPACE}"
```

6. Send `GET` and `POST` request to the exposed workload:

```
curl -ik -X GET "https://${WORKLOAD_DOMAIN}/ip"
curl -ik -X POST "https://${WORKLOAD_DOMAIN}/post" -d "test data"
```

If successful, the calls return `200 OK` response codes.

4.3.1.2.2.4 Ordering Rules in APIRule v2

To ensure your service requests are handled as intended, learn how to create and order rules in your APIRule custom resource (CR).

Context

APIRule allows you to define a list of rules that specify how requests to your service are handled. You can define a list of rules in the `spec.rules` field of the APIRule CR. Each rule starts with a hyphen and must contain the following details:

- The path on which the service is exposed
- The list of HTTP methods available for this path
- The specified access strategy

If your APIRule includes multiple rules, their order matters. Follow these steps when creating and ordering the rules to avoid path conflicts.

Procedure

1. Specify the paths.

   To define the paths for each rule, use one of the following approaches. You can specify exact path names or, to match multiple paths, use the operators `{*}`, `{**}`, or the `/*` wildcard.

   - Specify the exact path name.
     For example, `/example/one` specifies the exact path `/example/one`, and `/` specifies the root path.
   - Use the operators `{*}`, `{**}`, or the `/*` wildcard.

Pattern	Description	Examples
`{*}`	It matches a single path component, up to the next path separator `/`.	• `/example/{*}/one` - matches requests with the path prefix `example`, exactly one additional segment in the middle, and the path suffix `one`. For example, possible matches include `/example/anything/one`. • `/example/{*}` - matches requests with path prefix `example` and exactly one other segment. For example, possible matches include: `/example/anything`. The paths `/example/` and `/example/anything/` don't match.
`{**}`	If it is the last element of the path, it matches zero or more path segments. If it is not the last element of the path, it matches one or more path segments. If multiple operators are present, `{**}` must be the last one.	• `/example/{**}/one` - matches requests with the path prefix `example`, one or more additional segments in the middle, and the path suffix `one`. For example, possible matches include `/example/anything/two/one` and `/example/anything/one`. The paths `/example//one` and `/example/one` don't match. • `/example/{**}` - matches requests with the path prefix `example` followed by zero, one, or more additional segments. For example, possible matches include `/example/anything`, `/example/anything/more/`, and `/example/`. • `/{*}/example/{*}/{**}` - matches with any segment at the beginning, the path `example`, and at least one additional path segment. For example, possible matches include `/anything/example/anything/` and `/anything/example/anything/more`.

Pattern	Description	Examples
`/*`	It matches all paths. It is equivalent to the path `/{**}`. If defined, `/*` must be the only element in the entire path. You cannot combine the wildcard path with any other operators or path segments. It was introduced for backward compatibility.	• `/*` - matches `/` and any other path, such as `/example/anything/more/` or `/example/`.

> ### Note
>
> To be a valid path template, the path must not contain `*`, `{`, or `}` outside of the supported operators or the `/*` wildcard. No other characters are allowed in the path segment with the path template operator or the wildcard.

Using the wildcard and the operators introduces the possibility of path conflicts. A path conflict occurs when two or more APIRule `spec.rules` match the same path and share at least one common HTTP method. This is why it is important to consider the order of rules and to understand the connection between rules based on the path prefix and shared HTTP methods. Knowing the expected outcome of a configured rule helps organize and sort the rules.

2. Group paths into rules based on common HTTP methods and access strategies.

   Specify HTTP methods that you want to allow for each path. If you want to allow more than one method for a path, you can use one of these approaches:

   - Group multiple HTTP methods in a single rule. This is useful when you want to apply the same access strategy to multiple HTTP methods for the same endpoint.
     For example, to allow both GET and POST requests to all service endpoints, you specify the `/{**}` operator in the path without authentication. In such a case, you can define a single rule with both methods:

     > ### Sample Code
     >
     ```
     ...
       rules:
         - path: /{**}
           methods:
             - GET
             - POST
           noAuth: true
     ```

   - Create a separate rule for each HTTP method. Use this approach if you want to apply different access strategies or configurations to each method.
     In this example, the outcome is the same as grouping multiple HTTP methods in one rule. You still allow both GET and POST requests to all service endpoints without authentication.

     > ### Sample Code
     >
     ```
     rules:
       - path: /{**}
         methods:
           - GET
         noAuth: true
       - path: /{**}
     ```

```
      methods:
        - POST
      noAuth: true
```

> ### Note
>
> You can group multiple HTTP methods in a single rule if they share the same access strategy, or create separate rules for each method to allow for applying different configurations. Both approaches are valid.

3. Order the rules.

   Search for the paths that overlap. Overlapping occurs when two or more rules in an APIRule configuration contain paths that can match the same request and share at least one common HTTP method. This might lead to ambiguity about which rule applies to a given request.

   When defining the order of rules, remember that each request is evaluated against the list of paths from top to bottom in your APIRule, and the first matching rule is applied. If a request matches the first path, the first rule applies. This is why you must define the rules starting from the most specific path, and leave the most general path as the last one.

   > ### Note
   >
   > Rules defined earlier in the list have a higher priority than those defined later. The request searches for the first matching rule starting from the top of the APIRule `spec.rules` list. Therefore, it's recommended to order rules from the most specific path to the most general.

   See the following example of an incorrect rules' order:

   > ### Sample Code
   >
   > ```
   > ...
   >  rules:
   >    - path: /anything/{**}
   >      methods:
   >        - POST
   >        - GET
   >      noAuth: true
   >    - path: /anything/{*}/one
   >      methods:
   >        - POST
   >      jwt:
   >        authentications:
   >          - issuer: https://example.com
   >            jwksUri: https://example.com/.well-known/jwks.json
   > ```

   Applying this configuration causes an APIRule to be in `Error` state with the description:

   ```
   Validation errors: Attribute '.spec.rules': Path /anything/{*}/one with
   method POST conflicts with at least one of the previous rule paths
   ```

   In the following APIRule, the first rule specifically matches requests to the path `/anything/{*}/one` with the POST method, requiring JWT authentication. The second rule acts as a catch-all for any other paths that start with `/anything/`, allowing unauthenticated POST and GET requests. By placing the more specific rule first, you ensure that requests to `/anything/{*}/one` are handled as intended, while all

other matching paths are covered by the more general rule. This approach prevents the more general rule from overshadowing the specific one and ensures the correct access strategy is applied to each path.

Sample Code

```
...
  rules:
    - path: /anything/{*}/one
      methods:
        - POST
      jwt:
        authentications:
          - issuer: https://example.com
            jwksUri: https://example.com/.well-known/jwks.json
    - path: /anything/{**}
      methods:
        - POST
        - GET
      noAuth: true
```

You can test the above APIRule configuration using the following requests:

Request	Rule Matched	Access Strategy	Expected Outcome	HTTP Status Code
`curl -ik -X GET https://{SUBDOMAIN}.{DOMAIN_NAME}/anything/more`	Second rule (`/anything/{**}`)	`noAuth`	Unauthenticated GET to `/anything/more` allowed	`200 OK`
`curl -ik -X POST https://{SUBDOMAIN}.{DOMAIN_NAME}/anything/more`	Second rule (`/anything/{**}`)	`noAuth`	Unauthenticated POST to `/anything/more` allowed	`200 OK`
`curl -ik -X POST https://{SUBDOMAIN}.{DOMAIN_NAME}/anything/more/one --header "Authorization: Bearer <valid_jwt_token>"`	First rule (`/anything/{*}/one`)	`JWT` (required)	Authenticated POST to `/anything/more/one` allowed	`200 OK`
`curl -ik -X POST https://{SUBDOMAIN}.{DOMAIN_NAME}/anything/more/one`	First rule (`/anything/{*}/one`)	`JWT` (required)	Unauthenticated POST to `/anything/more/one` denied	`403 RBAC: access denied`

Request	Rule Matched	Access Strategy	Expected Outcome	HTTP Status Code
`curl -ik -X GET https://{SUBDOMAIN}.{DOMAIN_NAME}/anything/more/one`	No matching rule	-	GET to `/anything/more/one` not allowed (no rule matches)	`403 RBAC: access denied`

4. Check for excluding rules that share common methods.

> **Note**
>
> Understanding the relationship between paths and methods in a rule is crucial to avoid unexpected behavior. If a rule shares at least one common method with a preceding rule, then the path from preceding rule is excluded from this rule.

For example, the following APIRule configuration excludes the POST and GET methods for the path `/anything/one` with the `noAuth` access strategy. This happens because the rule with the path `/anything/{**}` shares one common HTTP method POST with the preceding rule with the path `/anything/one`.

Sample Code

```
...
  rules:
    - methods:
      - POST
      jwt:
        authentications:
          - issuer: https://example.com
            jwksUri: https://example.com/.well-known/jwks.json
      path: /anything/one
    - methods:
      - GET
      - POST
      noAuth: true
      path: /anything/{**}
```

This outcome might be unexpected if you intended to allow GET requests to `/anything/one` without authentication. To achieve that, you must specifically define separate rules for overlapping methods and paths. The following APIRule configuration allows POST requests to `/anything/one` with JWT authentication, while permitting unauthenticated POST requests to all paths starting with `/anything/` except for the `/anything/one` endpoint. Additionally, it allows unauthenticated GET requests to all paths prefixed with `/anything/`. See the following example:

Sample Code

```
...
  rules:
    - methods:
        - POST
      jwt:
        authentications:
          - issuer: https://example.com
```

```
            jwksUri: https://example.com/.well-known/jwks.json
      path: /anything/one
    - methods:
        - POST
      noAuth: true
      path: /anything/{**}
    - methods:
        - GET
      noAuth: true
      path: /anything/{**}
```

You can test the above APIRule configuration using the following requests:

Request	Rule Matched	Access Strategy	Expected Outcome	HTTP Status Code
`curl -ik -X GET https://{SUBDOMAIN}.{DOMAIN_NAME}/anything/more`	Third rule (`/anything/{**}`)	`noAuth`	Unauthenticated GET to `/anything/more` allowed	`200 OK`
`curl -ik -X POST https://{SUBDOMAIN}.{DOMAIN_NAME}/anything/more`	Second rule (`/anything/{**}`)	`noAuth`	Unauthenticated POST to `/anything/more` allowed	`200 OK`
`curl -ik -X POST https://{SUBDOMAIN}.{DOMAIN_NAME}/anything/more/one --header "Authorization: Bearer <valid_jwt_token>"`	First rule (`/anything/{*}/one`)	`JWT` (required)	Authenticated POST to `/anything/more/one` allowed	`200 OK`
`curl -ik -X POST https://{SUBDOMAIN}.{DOMAIN_NAME}/anything/more/one`	First rule (`/anything/{*}/one`)	`JWT` (required)	Unauthenticated POST to `/anything/more/one` denied	`403 RBAC: access denied`
`curl -ik -X GET https://{SUBDOMAIN}.{DOMAIN_NAME}/anything/more/one`	Third rule (`/anything/{**}`)	`noAuth`	Unauthenticated GET to `/anything/more/one` allowed)	`200 OK`

4.3.1.2.2.5 Short Host Name

Simplify your APIRule configuration by using a short host name instead of a fully qualified domain name (FQDN).

What Is a Short Host Name?

A short host name is a single lowercase RFC 1123 subdomain label without the domain suffix. For example, instead of specifying the FQDN `myapp.example.com` in APIRule, you use `myapp`. The domain `example.com` is automatically appended from the referenced Gateway resource.

Using a short host is especially useful when you move or recreate workloads in a new cluster. With a short host name, the APIRule does not hard-code the domain. Instead of updating every APIRule when the cluster's domain changes, you only update the Gateway configuration once, and all APIRules that use short hosts start using the new domain automatically. This reduces the risk of mistakes and makes cluster migration easier.

Gateway Requirements

The referenced Gateway must meet these requirements:

- Define a single wildcard host across all Server definitions.
- Use the wildcard prefix `*.` (for example, `*.example.com`).

Example

See an example Gateway resource:

```
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: kyma-gateway
  namespace: kyma-system
spec:
  selector:
    app: istio-ingressgateway
  servers:
  - hosts:
    - "*.example.com"
    port:
      name: https
      number: 443
      protocol: HTTPS
    tls:
      mode: SIMPLE
      credentialName: example-com-tls
```

The following APIRule uses a short host name and references the Gateway `kyma-gateway`:

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: httpbin-short
  namespace: default
spec:
  hosts:
    - myapp
  service:
    name: httpbin
    port: 8000
  gateway: kyma-system/kyma-gateway
  rules:
    - path: /headers
      methods: ["GET"]
      noAuth: true
```

You can call your Service at `myapp.example.com` even though the domain suffix is not explicitly specified in the APIRule. The domain suffix is retrieved from the referenced Gateway.

4.3.1.2.2.6 Wildcard Host

Use a wildcard hostname in `APIRule` to route traffic from matching subdomains to a single service.

What Is a Wildcard Host?

A wildcard hostname, such as `*.example.com`, matches any subdomain under a given domain and routes all matching traffic with a single `APIRule`.

This is especially useful for multitenant applications where each tenant is accessed through a unique URL following a predictable pattern, such as `<tenant-id>.example.com`. A wildcard hostname covers all matching subdomains without requiring a separate `APIRule` for each tenant.

Requirements

- The referenced Gateway must serve a wildcard host that covers the domain you use in the `APIRule`.
- The * wildcard character is only allowed as the leftmost label of the host, for example, `*.example.com`. Patterns like `api.*.com` are not supported.
- Full regex host matching is not supported. If you need to match a specific pattern, use explicit hostnames in `spec.hosts` instead.

Example

The following `APIRule` exposes a workload for any subdomain under `example.com`:

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: httpbin-wildcard
  namespace: default
spec:
  hosts:
    - "*.example.com"
  service:
    name: httpbin
    port: 8000
  gateway: kyma-system/kyma-gateway
  rules:
    - path: /headers
      methods: ["GET"]
      noAuth: true
```

Any request to a host matching `*.example.com` — for example `tenant1.example.com` or `tenant2.example.com` — is routed to the HTTPBin Service.

Wildcard Host and Specific Host Conflict

> ⚠ Caution
>
> If a wildcard `APIRule` (for example, `*.example.com`) uses `noAuth`, it also matches specific subdomains like `a.example.com`. In such a case, a separate `APIRule` for `a.example.com` with `jwt` does not enforce authentication, because the wildcard `noAuth` route already allows the request.

To avoid the issue, follow these rules:

- Do not combine a wildcard `noAuth` `APIRule` with specific-host `APIRule` resources that require authentication under the same domain.
- If you need different access strategies for specific subdomains, avoid using `noAuth` on the wildcard host. Instead, apply the least permissive strategy (for example, `jwt`) on the wildcard and create separate `APIRule` resources only for hosts that genuinely require `noAuth`.

4.3.1.2.2.7 Exposing Multiple Workloads on the Same Host

Expose multiple backend workloads under a single host by routing traffic to different Services.

There are two primary patterns for configuring multiple workloads on the same host: Path-Level Service Definition [page 2013] and Root-Level Service with Overrides [page 2013]. Additionally, you can expose Services deployed in different namespaces. See Cross-Namespace Service Exposure [page 2014].

Path-Level Service Definition

In this pattern, each path explicitly declares which Service it routes to. You define all Service configurations at the `spec.rules` level, with no default Service defined at the root.

- Explicit routing - each path clearly states its target Service
- No default fallback behavior
- Maximum clarity and predictability
- Ideal when Services are unrelated or follow different architectural patterns

```
spec:
  rules:
    - path: /headers
      service:
        name: service-a
    - path: /get
      service:
        name: service-b
```

Root-Level Service with Overrides

This pattern defines a default Service at the root level (`spec.service`), which applies to all paths unless explicitly overridden. Individual rules can specify alternative Services that take precedence over the root definition.

- Default Service handles most traffic (with exceptions)
- Selective overrides for specific paths
- Reduces configuration verbosity when most paths use the same Service

> ⓘ Note
>
> When both root-level and rule-level Services are defined, the rule-level Service takes precedence.

```
spec:
  service:
    name: primary-service
  rules:
    - path: /headers
    - path: /admin
      service:
        name: admin-service
```

Cross-Namespace Service Exposure

When exposing Services, each Service referenced in the rules can be in a different namespace. To expose a Service deployed in a different namespace from the `APIRule` resource itself, specify its namespace in the field `service.namespace`.

```
spec:
  rules:
    - path: /headers
      service:
        name: service-a
        namespace: ns-a
    - path: /get
      service:
        name: service-b
        namespace: ns-b
```

Example

This example demonstrates all configuration patterns in a single `APIRule`, showing how to expose multiple workloads on one host.

1. Create namespaces with enabled Istio sidecar proxy injection.

   ```
   kubectl create ns httpbin-a
   kubectl label namespace httpbin-a istio-injection=enabled --overwrite
   kubectl create ns httpbin-b
   kubectl label namespace httpbin-b istio-injection=enabled --overwrite
   kubectl create ns apirule
   kubectl label namespace apirule istio-injection=enabled --overwrite
   ```

2. Get the default domain of your project "Kyma" cluster.

   ```
   PARENT_DOMAIN=$(kubectl get gateway -n kyma-system kyma-gateway -o
   jsonpath='{.spec.servers[0].hosts[0]}' | sed 's/\*\.//')
   ```

3. Deploy two sample HTTPBin Services in different namespaces.

   ```
   kubectl apply -n httpbin-a -f https://raw.githubusercontent.com/istio/istio/
   master/samples/httpbin/httpbin.yaml
   kubectl apply -n httpbin-b -f https://raw.githubusercontent.com/istio/istio/
   master/samples/httpbin/httpbin.yaml
   ```

4. Create an `APIRule`:

   > ⚠ Caution
   >
   > Exposing a workload to the outside world is always a potential security vulnerability, so be careful. In a production environment, remember to secure the workload you expose with `jwt` or `extAuth`.

   ```
   cat <<EOF | kubectl -n apirule apply -f -
   apiVersion: gateway.kyma-project.io/v2
   kind: APIRule
   metadata:
     name: multi-workload
   spec:
     hosts:
   ```

```
    - test.${PARENT_DOMAIN}
  gateway: kyma-system/kyma-gateway
  service:
    name: httpbin
    namespace: httpbin-a
    port: 8000
  rules:
    - path: /headers    # Default: routes to httpbin-a service
      methods: ["GET"]
      noAuth: true
    - path: /ip     # Override: routes to httpbin-b service
      methods: ["GET"]
      noAuth: true
      service:
        name: httpbin
        namespace: httpbin-b
        port: 8000
EOF
```

5. Test the connection:
   - Call default service.

     ```
     curl -ik https://test.${PARENT_DOMAIN}/headers
     ```

   - Call the override service.

     ```
     curl -ik https://test.${PARENT_DOMAIN}/ip
     ```

   If successful, each request returns a `200 OK` response from its respective service.

4.3.1.2.2.8 Restrict IP-Based Access

Expose your workload and configure IP-based access using the X-Forwarded-For (XFF) header. This helps to enhance security by ensuring that only trusted IPs can interact with your application. (testing web push)

Prerequisites

- You have the Istio and API Gateway modules added. See Adding and Deleting a Kyma Module [page 3511].
- You have a deployed workload.
- You have access to Kyma dashboard. Alternatively, to use CLI instructions, you must install kubectl and curl.
- You have set up your custom domain (see Configure a TLS Gateway [page 1948]). Alternatively, you can use the default domain of your Kyma cluster and the default Gateway `kyma-system/kyma-gateway`.

> **Note**
>
> Because the default Kyma domain is a wildcard domain, which uses a simple TLS Gateway, it is recommended that you set up your custom domain for use in a production environment.

→ Tip

To learn what the default domain of your Kyma cluster is, run the following command:

```
kubectl get gateway -n kyma-system kyma-gateway -o
jsonpath='{.spec.servers[0].hosts}'
```

Context

The XFF header is a standard HTTP header that conveys the client IP address and the chain of intermediary proxies that the request traverses to reach the Istio service mesh. This is particularly useful when an application must be provided with the client IP address of an originating request, for example, for access control.

However, there are some technical limitations when using the XFF header. The header might not include all IP addresses if an intermediary proxy does not support modifying the header. Due to technical limitations of AWS Classic ELBs , when using an IPv4 connection, the header does not include the public IP of the load balancer in front of Istio Ingress Gateway. Moreover, Istio Ingress Gateway's Envoy does not append the private IP address of the load balancer to the XFF header, effectively removing this information from the request. For more information on XFF, see the IETF's RFC documentation and Envoy documentation .

To use the XFF header, you must configure the corresponding settings in the Istio custom resource (CR). Then, expose your workload using an VirtualService CR and create an AuthorizationPolicy resource with allowed IP addresses specified in the `remoteIpBlocks` field. To learn how to do this, follow the procedure.

Procedure

- Use Kyma dashboard.
  a. Go to *Cluster Overview* and choose *Modify Modules*.
  b. Select the Istio module and choose *Edit*.
  c. In the *General* section, add the number of trusted proxies.

     Due to the variety of network topologies, the Istio CR must specify the number of trusted proxies deployed in front of the Istio Ingress Gateway proxy, so that the client address can be extracted correctly.
  d. If you use a Google Cloud or Microsoft Azure cluster, navigate to the *Gateway* section and set the Gateway traffic policy to Local. If you use a different cloud service provider, skip this step.

     ⚠ Caution

     For production Deployments, it is strongly recommended to deploy Istio Ingress Gateway Pod to multiple nodes if you enable `externalTrafficPolicy : Local`. For more information, see Network Load Balancer .

     Default Istio installation profile configures `PodAntiAffinity` to ensure that Ingress Gateway Pods are evenly spread across all nodes. This guarantees that the above requirement is satisfied if your IngressGateway autoscaling configuration `minReplicas` is equal to or greater than

the number of nodes. You can configure autoscaling options in the Istio CR using the field `spec.config.components.ingressGateway.k8s.hpaSpec.minReplicas`.

> → Tip
>
> If you use a Google Cloud or Microsoft Azure cluster, you can find your load balancer's IP address in the field `status.loadBalancer.ingress` of the `ingress-gateway` Service.

e. Choose *Save*.

f. Go to *Istio > VirtualServices* and choose *Create*.

g. Switch to the *YAML* section and paste the following configuration:

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: {VIRTUALSERVICE_NAME}
  namespace: {VIRTUALSERVICE_NAMESPACE}
spec:
  hosts:
  - "{SERVICE_NAME}.{DOMAIN_NAME}"
  gateways:
  - {GATEWAY_NAME}/{GATEWAY_NAMESPACE}
  http:
  - match:
    - uri:
        prefix: /
    route:
    - destination:
        port:
          number: {SERVICE_PORT}
        host: {SERVICE_NAME}.{SERVICE_NAMESPACE}.svc.cluster.local
```

When you go to `https:/{SUBDOMAIN}.{DOMAIN}/{PATH}`, the response contains the `X-Forwarded-For` and `X-Envoy-External-Address` headers with your public IP address. See an example response for the Client IP 165.1.187.197:

```
{
  "args": {
    "show_env": "true"
  },
  "headers": {
    ...
    "X-Envoy-Attempt-Count": "...",
    "X-Envoy-External-Address": "165.1.187.197",
    "X-Forwarded-Client-Cert": "...",
    "X-Forwarded-For": "165.1.187.197",
    "X-Forwarded-Proto": "...",
    "X-Request-Id": "..."
  },
  "origin": "165.1.187.197",
  "url": "..."
}
```

> → Tip
>
> You can check your public IP address at https://api.ipify.org .

h. Go to *Istio > Authorizaiton Policies* and choose *Create*.

i. Add a selector to specify the workload for which access should be configured.

j. Add a rule with a `From` field.

k. In the `RemoteIpBlocks` field, specify the IP addresses that should be allowed access to the workload.

l. Choose *Create*.

- Use kubectl.

a. In the following command, replace the placeholder with the number of trusted proxies deployed in front of the Istio Ingress Gateway proxy. Run the command.

```
kubectl patch istios/default -n kyma-system --type merge -p '{"spec":
{"config":{"numTrustedProxies": NUM_OF_TRUSTED_PROXIES}}}'
```

Due to the variety of network topologies, the Istio CR must specify the configuration property `numTrustedProxies`, so that the client IP address can be extracted correctly.

b. If you use a Google Cloud or Microsoft Azure cluster, run the following command to set the traffic policy to Local. If you use a different cloud service provider, skip this step.

```
kubectl patch istios/default -n kyma-system --type merge -p '{"spec":
{"config":{"gatewayExternalTrafficPolicy": "Local"}}}'
```

> ⚠ Caution
>
> For production Deployments, it is strongly recommended to deploy Istio Ingress Gateway Pod to multiple nodes if you enable `externalTrafficPolicy : Local`. For more information, see Network Load Balancer .
>
> Default Istio installation profile configures `PodAntiAffinity` to ensure that Ingress Gateway Pods are evenly spread across all nodes. This guarantees that the above requirement is satisfied if your IngressGateway autoscaling configuration `minReplicas` is equal to or greater than the number of nodes. You can configure autoscaling options in the Istio CR using the field `spec.config.components.ingressGateway.k8s.hpaSpec.minReplicas`.

> → Tip
>
> If you use a Google Cloud or Microsoft Azure cluster, you can find your load balancer's IP address in the field `status.loadBalancer.ingress` of the `ingress-gateway` Service.

c. To expose your workload, apply the following VirtualService resource.

You can adjust this sample configuration and use another access strategy, according to your needs.

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: {VIRTUALSERVICE_NAME}
  namespace: {VIRTUALSERVICE_NAMESPACE}
spec:
  hosts:
  - "{SERVICE_NAME}.{DOMAIN_NAME}"
  gateways:
  - {GATEWAY_NAME}/{GATEWAY_NAMESPACE}
  http:
  - match:
    - uri:
        prefix: /
    route:
    - destination:
        port:
          number: {SERVICE_PORT}
        host: {SERVICE_NAME}.{SERVICE_NAMESPACE}.svc.cluster.local
```

```
EOF
```

When you go to `https:/{SUBDOMAIN}.{DOMAIN}/{PATH}`, the response contains the `X-Forwarded-For` and `X-Envoy-External-Address` headers with your public IP address. See an example response for the Client IP 165.1.187.197:

```
{
  "args": {
    "show_env": "true"
  },
  "headers": {
    ...
    "X-Envoy-Attempt-Count": "...",
    "X-Envoy-External-Address": "165.1.187.197",
    "X-Forwarded-Client-Cert": "...",
    "X-Forwarded-For": "165.1.187.197",
    "X-Forwarded-Proto": "...",
    "X-Request-Id": "..."
  },
  "origin": "165.1.187.197",
  "url": "..."
}
```

> → Tip
>
> You can check your public IP address at https://api.ipify.org .

d. To configure IP-based access to the exposed workload, create an AuthorizationPolicy resource.

The selector specifies the workload for which access should be configured, and the `RemoteIpBlocks` field specifies the IP addresses for which access should be allowed.

```
cat <<EOF | kubectl apply -f -
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: {AUTHORIZATIONPOLICY_NAME}
  namespace: {AUTHORIZATIONPOLICY_NAMESPACE}
spec:
  action: ALLOW
  rules:
    - from:
        - source:
            ipBlocks: []
            remoteIpBlocks:
              - {ALLOWED_IP}
  selector:
    matchLabels:
      {KEY}: {VALUE}
EOF
```

Results

You have configured the XFF header in the Istio CR and exposed your workload to the internet. Access to the workload is limited to the IP addresses that you have specified.

4.3.1.2.2.9 Expose Workloads with Istio VirtualService

Learn how to expose a workload with Istio VirtualService.

Prerequisites

- You have the Istio module added to your Kyma runtime instance.
- You have a deployed workload.
- To set up a custom Gateway, see Configure a TLS Gateway [page 1948]. Alternatively, you can use the default domain of your Kyma cluster and the default Gateway `kyma-system/kyma-gateway`.

> ⚠ Caution
>
> Because the default Kyma domain is a wildcard domain, which uses a simple TLS Gateway, it is recommended that you set up your custom domain for use in a production environment.

> → Tip
>
> To learn what the default domain of your Kyma cluster is, run the following command:
>
> ```
> kubectl get gateway -n kyma-system kyma-gateway -o
> jsonpath='{.spec.servers[0].hosts}'
> ```

Context

Kyma's API Gateway module provides the APIRule custom resource (CR), which is the recommended solution for securely exposing workloads. To expose a workload using an APIRule `v2`, you must include the workload in the Istio service mesh. Including a workload in the mesh brings several benefits, such as secure service-to-service communication, tracing capabilities, or traffic management. For more information, see Purpose and Benefits of Istio Sidecar Proxies and The Istio service mesh .

However, if you do not need the capabilities provided by the Istio service mesh, you can expose an unsecured workload using Istio VirtualService only. Such an approach might be useful in the following scenarios:

- If you use Unified Gateway as an entry point for SAP Cloud solutions. In this case, you can configure Unified Gateway to manage API exposure, JWT validation, and routing capabilities, offloading these responsibilities from the service mesh.
- If you want to expose front-end Services that manage their own authentication mechanisms.
- If you require certain features or want to implement specific configurations that APIRule does not support.
- If you require full control over Istio resources and you want to manage them directly without any higher-level abstractions.

The following instructions demonstrate a simple use case where VirtualService exposes an unsecured Service, skipping the requirement to include the Service in the Istio service mesh.

Procedure

- Use Kyma dashbaord
  a. Go to the namespace in which you want to create a VirtualService resource.
  b. Go to *Istio > Virtual Services*.
  c. Choose *Create* and provide the following configuration details:

Option	Description
`Name`	The name of the VirtualService resource you're creating.
`Hosts`	The address or addresses the client uses when sending requests to the destination Service. Use the fully qualified domain name (FQDN) constructed with the subdomain and domain name in the following format: {SUBDOMAIN}.{DOMAIN_NAME}. The hosts must be defined in the referenced Gateway.
`Gateways`	The name of the Gateway you want to use and the namespace in which it is created. Use the format {GATEWAY_NAMESPACE}/{GATEWAY_NAME}.

  d. Go to *HTTP > Matches > Match* and provide the following configuration details:

Option	Description
`URI`	Add the URI prefix used to match incoming requests. This determines which requests are routed to the specified destination Service.
`Port`	This is the port number on which the destination Service is listening. It specifies where the VirtualService routes the incoming traffic. If a Service exposes only a single port it is not required to explicitly select the port.

  e. Go to *HTTP > Routes > Route > Destinations > Host* and add the name and namespace of the Service you want to expose using the following format: {SERVICE_NAME}.{SERVICE_NAMESPACE}.svc.cluster.local. The traffic is routed to this Service.

  For more configuration options, see Virtual Service.

{·} Sample Code

See a sample VirtualService configuration that directs all HTTP traffic received at `httpbin.my-domain.com` through the my-gateway Gateway to the HTTPBin Service, which is running on port 8000 in the default namespace.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: example-vs
  namespace: default
spec:
```

```
  hosts:
    - httpbin.my-domain.com
  gateways:
    - default/my-gateway
  http:
  - match:
    - uri:
        prefix: /
    route:
    - destination:
        port:
          number: 8000
        host: httpbin.default.svc.cluster.local
```

f. To verify if the Service is exposed, run the following command:

```
curl -s -I https://{SUBDOMAIN}.{DOMAIN_NAME}/
```

If successful, you get code `200` in response.

- Use kubectl

a. To expose a workload using VirtualService, replace the placeholders and run the following command:

Option	Description
`{NAME}`	The name of the VirtualService resource you're creating.
`{NAMESPACE}`	The namespace in which you want to create the VirtualService resource.
`{SUBDOMAIN}.{DOMAIN_NAME}`	The address or addresses the client uses when sending requests to the destination Service. Use the fully qualified domain name (FQDN) constructed with the subdomain and domain name in the following format: {SUBDOMAIN}.{DOMAIN_NAME}. The hosts must be defined in the referenced Gateway.
`{GATEWAY_NAMESPACE}/{GATEWAY_NAME}`	The name of the Gateway you want to use and the namespace in which it is created.
`{URI_PREFIX}`	The URI prefix used to match incoming requests. This determines which requests are routed to the specified destination Service.
`{PORT_NUMBER}`	This is the port number on which the destination Service is listening. It specifies where the VirtualService routes incoming requests. If a Service exposes only a single port it is not required to explicitly select the port.
`{SERVICE_NAME}.{SERVICE_NAMESPACE}`	The name and namespace of the Service you want to expose. The requests are routed to this Service.

For more configuration options, see Virtual Service.

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.istio.io/v1
kind: VirtualService
```

```
metadata:
  name: {VS_NAME}
  namespace: {VS_NAMESPACE}
spec:
  hosts:
    - {SUBDOMAIN}.{DOMAIN_NAME}
  gateways:
    - {GATEWAY_NAMESPACE}/{GATEWAY_NAME}
  http:
  - match:
    - uri:
        prefix: {URI_PREFIX}
    route:
    - destination:
        port:
          number: {PORT_NUMBER}
        host: {SERVICE_NAME}.{SERVICE_NAMESPACE}.svc.cluster.local
```

Sample Code

See a sample VirtualService configuration that directs all HTTP traffic received at `httpbin.my-domain.com` through the my-gateway Gateway to the HTTPBin Service, which is running on port 8000 in the default namespace.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: example-vs
  namespace: default
spec:
  hosts:
    - httpbin.my-domain.com
  gateways:
    - default/my-gateway
  http:
  - match:
    - uri:
        prefix: /
    route:
    - destination:
        port:
          number: 8000
        host: httpbin.default.svc.cluster.local
```

b. To verify if the Service is exposed, run the following command:

```
curl -s -I https://{SUBDOMAIN}.{DOMAIN_NAME}/
```

If successful, you get code `200` in response.

4.3.1.2.3 Manage Network Policies

Network policy support in the API Gateway module is disabled by default. Learn about the network policies for the API Gateway module and how to manage them.

When enabled, the API Gateway module creates the `kyma-project.io--api-gateway-allow` network policy. This policy allows only the following connections:

- Egress to the `kube-dns` and `local-dns` workloads for in-cluster name resolution.
- Egress to the Kubernetes API server on port `443`.
- Ingress to the webhook port `9443`.
- Ingress to the metrics port `8080` from external workloads labeled with `networking.kyma-project.io/metrics-scraping=allowed` or from other Kyma modules.

> ⚠ Caution
>
> Since Oathkeeper support is deprecated, network policies don't cover Ory Oathkeeper connectivity. It's not recommended to use network policies when Ory Oathkeeper support is enabled. If you still use Ory Oathkeeper and wish to enable network policies, expose your workloads with `APIRule` resources in version v2.

Enable Network Policies

To enable network policies, set the `networkPoliciesEnabled` field to true in the `APIGateway` custom resource:

```
kubectl patch apigateways.operator.kyma-project.io default --type=merge --
patch='{"spec": {"networkPoliciesEnabled": true}}'
```

Disable Network Policies

To disable network policies, set the `networkPoliciesEnabled` field to false in the `APIGateway` custom resource:

```
kubectl patch apigateways.operator.kyma-project.io default --type=merge --
patch='{"spec": {"networkPoliciesEnabled": false}}'
```

> ⚠ Caution
>
> If you disable network policies in supported clusters, the traffic to the API server or other critical workloads may be disrupted. Always consult your cluster configuration to choose the best solution.

Verify Status

To check if the network policies are active, run:

```
kubectl get networkpolicies -n kyma-system -l api-gateway.kyma-project.io/
managed-by
```

4.3.1.2.4 Migrate `APIRule` from Version v1beta1 to Version v2

`APIRule` custom resource (CR) v1beta1 is deleted. You must migrate all your `APIRule` CRs to version v2. Learn how to perform the migration.

Context

> ⚠ Caution
>
> `APIRule` CRD v2 is the latest stable version.
>
> - You can no longer create, edit, or delete `APIRules` using v1beta1 API. Existing configurations continue to function as expected, but to make any changes, migrate to version v2.
> - `APIRules` v1beta1 are no longer visible in Kyma dashboard. You can still view them with `kubectl`, but they display in the converted v2 format.
> - Reconciliation of `APIRules` v1beta1 is disabled, and the API Gateway module no longer manages them. Migrating to version v2 may temporarily disrupt workload availability and access.
>
> For the `APIRule` deletion timeline for SAP BTP, Kyma runtime, see API Gateway What's New notes.

Procedure

- To migrate to version `v2`, follow these steps:
  a. To identify which `APIRules` must be migrated, run the following command:

     ```
     kubectl get apirules.gateway.kyma-project.io -A -o json | jq '.items[]
     | select(.metadata.annotations["gateway.kyma-project.io/original-version"]
     == "v1beta1") | {namespace: .metadata.namespace, name: .metadata.name}'
     ```

  b. If two or more of your `APIRules` target the same workload, apply an additional `AuthorizationPolicy` to avoid traffic disruption during migration. See Migrate Multiple APIRules Targeting the Same Workload from v1beta1 to v2 [page 2031].
  c. To retrieve the complete `spec` with the `rules` field of an `APIRule` in version v1beta1, see Retrieve the Complete spec of an APIRule in Version v1beta1 [page 2029].
  d. To migrate an `APIRule` from version v1beta1 to version v2, follow the relevant guide:

- Migrate APIRule v1beta1 of Type jwt to Version v2 [page 2036]
- Migrate APIRule v1beta1 of Type noop, allow, or no_auth to Version v2 [page 2040]
- Migrate APIRule v1beta1 of Type oauth2_introspection to Version v2 [page 2044]

- To delete `APIRules` v1beta1, use v2 API:

```
kubectl delete apirules.v2.gateway.kyma-project.io -n $NAMESPACE
$APIRULE_NAME -oyaml
```

Related Information

Changes Introduced in APIRule v2 [page 2026]

4.3.1.2.4.1 Changes Introduced in APIRule v2

Learn about the changes that APIRule `v2` introduces and the actions you must take to adjust your `v1beta1` resources.

A Workload Must Be in the Istio Service Mesh

To use APIRules in version `v2`, the workload that an APIRule exposes must be in the Istio service mesh. If the workload is not inside the Istio service mesh, the APIRule does not work as expected.

Required action: To add a workload to the Istio service mesh, see Enabling Istio Sidecar Proxy Injection [page 1892].

Internal Traffic to Workloads Is Blocked by Default

By default, access to the workload from internal traffic is blocked if APIRule CR in version `v2` is applied. This approach aligns with Kyma's “secure by default” principle.

CORS Policy Is Not Applied by Default

Version `v1beta1` applied the following CORS configuration by default:

```
corsPolicy:
  allowOrigins: ["*"]
  allowMethods: ["GET", "POST", "PUT", "DELETE", "PATCH"]
  allowHeaders: ["Authorization", "Content-Type", "*"]
```

Version `v2` does not apply these default values. If the `corsPolicy` field is empty, the CORS configuration is not applied. For more information, see architecture decision record #752 .

Required action: Configure CORS policy in the `corsPolicy` field.

> → Remember
>
> Since not all APIs require the same level of access, adjust your CORS policy configuration according to your application's specific needs and security requirements.

If you decide to use the default CORS values defined in the APIRule `v1beta1`, you must explicitly define them in your APIRule `v2`. For preflight requests to work, you must allow the "OPTIONS" method in the `rules.methods` field of your APIRule.

Path Specification Must Not Contain Regexp

APIRule in version `v2` does not support regexp in the `spec.rules.path` field of APIRule CR. Instead, it supports the use of the {*} and {**} operators. See the supported configurations:

- Use the exact path (for example, /abc). It matches the specified path exactly.
- Use the {*} operator (for example, /foo/{*} or /foo/{*}/bar). This operator represents any request that matches the given pattern, with exactly one path segment replacing the operator.
- Use the {**} operator (for example, /foo/{**} or /foo/{**}/bar). This operator represents any request that matches the pattern with zero or more path segments in the operator's place. It must be the last operator in the path.
- Use the wildcard path /*, which matches all paths. It's equivalent to the /{**} path. If your configuration in APIRule `v1beta1` used such a path as /foo(.*), when migrating to the new versions, you must define configurations for two separate paths: /foo and /foo/{**}.

> ⓘ Note
>
> The order of rules in the APIRule CR is important. Rules defined earlier in the list have a higher priority than those defined later. Therefore, we recommend defining rules from the most specific path to the most general.
>
> Operators allow you to define a single APIRule that matches multiple request paths. However, this also introduces the possibility of path conflicts. A path conflict occurs when two or more APIRule resources match the same path and share at least one common HTTP method. This is why the order of rules is important.

For more information on the APIRule specification, see APIRule v2 Custom Resource .

Required action: Replace regexp expressions in the `spec.rules.path` field of your APIRule CRs with the {*} and {**} operators.

JWT Configuration Requires Explicit Issuer URL

Version `v2` of APIRule introduces an additional mandatory configuration filed for JWT-based authorization - `issuer`. You must provide an explicit issuer URL in the APIRule CR. See an example configuration:

```
rules:
- jwt:
    authentications:
        -   issuer: {YOUR_ISSUER_URL}jwksUri: {YOUR_JWKS_URI}
```

If you use Cloud Identity Services, you can find the issuer URL in the OIDC well-known configuration at `https://{YOUR_TENANT}.accounts.ondemand.com/.well-known/openid-configuration`.

Required action: Add the `issuer` field to your APIRule specification. For more information, see Migrate APIRule v1beta1 of Type jwt to Version v2 [page 2036].

Removed Support for Oathkeeper OAuth2 Handlers

The APIRule CR in version `v2` does not support Oathkeeper OAuth2 handlers. Instead, it introduces the `extAuth` field, which you can use to configure an external authorizer.

Required action: Migrate your Oathkeeper-based OAuth2 handlers to use an external authorizer. To learn how to do this, see Migrate APIRule v1beta1 of Type oauth2_introspection to Version v2 [page 2044] and External Authorization .

Removed Support for Oathkeeper Mutators

The APIRule CR in version `v2` does not support Oathkeeper mutators. Request mutators are replaced with request modifiers defined in the `spec.rule.request` section of the APIRule CR. This section contains the request modification rules applied before the request is forwarded to the target workload. Token mutators are not supported in APIRule `v2`. For that, you must define your own `extAuth` configuration.

Required action: Migrate your rules that rely on Oathkeeper mutators to use request modifiers or an external authorizer. For more information, see External Authorization .

Removed Support for Opaque Tokens

The APIRule CR in version `v2` does not support the usage of Opaque tokens. Instead, it introduces the `extAuth` field, which you can use to configure an external authorizer.

Required action: Migrate your rules that use Opaque tokens to use an external authorizer. For more information, see External Authorization .

4.3.1.2.4.2 Retrieve the Complete `spec` of an `APIRule` in Version v1beta1

Retrieve the complete `spec` of an `APIRule` originally applied in version v1beta1 when the displayed `spec` does not contain the `rules` field.

Prerequisites

- You have the Istio and API Gateway modules added.
- You have a deployed workload exposed by an `APIRule` in the deprecated v1beta1 version.
- You must have `kubectl` and `yq` .

Context

`APIRule` in version v1beta1 is removed, and the API server no longer responds to requests for `APIRules` in this version.

This creates a migration challenge: If your `APIRule` was originally created using v1beta1 and you have not yet migrated to v2, the `spec` is missing the `rules` field when viewed via the `kubectl get` command. Kyma dashboard does not display `APIRules` in version v1beta1.

For example, suppose you have applied the following `APIRule` in version v1beta1:

```
apiVersion: gateway.kyma-project.io/v1beta1
kind: APIRule
metadata:
  name: httpbin
  namespace: test
spec:
  gateway: kyma-gateway.kyma-system
  host: httpbin
  rules:
  - accessStrategies:
    - handler: noop
    methods:
    - POST
    path: /anything
  - accessStrategies:
    - handler: allow
    methods:
    - HEAD
    path: /headers
  - accessStrategies:
    - handler: no_auth
    methods:
    - GET
    path: /.*
  service:
    name: httpbin
    namespace: test
    port: 8000
```

When retrieving this `APIRule`, the resulting `spec` does not include the `rules` field, as these rules cannot be converted to v2:

```
kubectl get apirules.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME -oyaml
```

```
...
spec:
  gateway: kyma-system/kyma-gateway
  hosts:
    - httpbin
  service:
    name: httpbin
    namespace: test
    port: 8000
...
```

In this case, you must access the original `APIRule` v1beta1 configuration through an annotation. To learn how to do this, follow the procedure.

Procedure

1. To get the full original `spec` of the `APIRule` created in version v1beta1, use the annotation that stores the original configuration. Run:

   ```
   kubectl get apirules.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME
   -ojsonpath='{.metadata.annotations.gateway\.kyma-project\.io/v1beta1-spec}'
   ```

   See a sample output in the JSON format:

   ```
   {"host":"httpbin","service":
   {"name":"httpbin","namespace":"test","port":8000},"gateway":"kyma-
   gateway.kyma-system","rules":[{"path":"/anything","methods":
   ["POST"],"accessStrategies":[{"handler":"noop"}]},{"path":"/
   headers","methods":["HEAD"],"accessStrategies":[{"handler":"allow"}]},
   {"path":"/.*","methods":["GET"],"accessStrategies":[{"handler":"no_auth"}]}]}
   ```

2. To format the output as YAML for better readability, use the `yq` command.

   ```
   kubectl get apirules.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME
   -ojsonpath='{.metadata.annotations.gateway\.kyma-project\.io/v1beta1-spec}' |
   yq -P
   ```

   See a sample output in the YAML format:

   ```
   host: httpbin
   service:
     name: httpbin
     namespace: test
     port: 8000
   gateway: kyma-gateway.kyma-system
   rules:
     - path: /anything
       methods:
         - POST
       accessStrategies:
         - handler: noop
     - path: /headers
       methods:
   ```

```
      - HEAD
    accessStrategies:
      - handler: allow
  - path: /.*
    methods:
      - GET
    accessStrategies:
      - handler: no_auth
```

Next Steps

Adjust the obtained configuration of the `APIRule` to migrate it to version v2. To learn how to do this, follow the relevant tutorial:

- Migrate APIRule v1beta1 of Type jwt to Version v2 [page 2036]
- Migrate APIRule v1beta1 of Type noop, allow, or no_auth to Version v2 [page 2040]
- Migrate APIRule v1beta1 of Type oauth2_introspection to Version v2 [page 2044]

4.3.1.2.4.3 Migrate Multiple `APIRule`s Targeting the Same Workload from v1beta1 to v2

Learn how to migrate multiple `APIRule`s v1beta1 that expose the same workload using different host names. To keep all endpoints available during migration, you must create an additional, temporary `AuthorizationPolicy`. This ensures that service requests to `APIRule`s v1beta1 are handled as intended, while another `APIRule` targeting the same workload has already been migrated to v2.

Context

When you have multiple `APIRule`s v1beta1 that expose the same workload but use different host values, and you migrate only one of those `APIRule`s to version v2, you may get `HTTP/2 403 RBAC: Access Denied` errors when making requests to the endpoints of the remaining v1beta1 `APIRule`s. However, requests to the endpoint of the migrated v2 `APIRule` are successful, returning `HTTP/2 200 OK` responses.

`HTTP/2 403 RBAC: Access Denied` errors are caused by Istio subresources created by the migrated `APIRule` v2. Because the other `APIRule`s exposing the same workload remain in version v1beta1 and do not use Istio subresources, requests from these `APIRule`s v1beta1 to the workload are no longer allowed. In such cases, access to the target workload is permitted only for requests that match the rules specified in the `APIRule` v2.

For example, suppose a scenario where you have applied the following two v1beta1 `APIRule`s configured to expose the same HTTPBin Service, each with a different host value, and you want to maintain uninterrupted access to service endpoints during migration.

```
apiVersion: gateway.kyma-project.io/v1beta1
kind: APIRule
metadata:
```

```
  name: example1
  namespace: test
spec:
  host: example1
  service:
    name: httpbin
    namespace: default
    port: 8000
  gateway: kyma-gateway.kyma-system
  rules:
    - path: /post
      methods: ["POST"]
      accessStrategies:
        - handler: no_auth
    - path: /.*
      methods: ["GET"]
      mutators: []
      accessStrategies:
        - handler: jwt
          config:
            trusted_issuers:
              - https://{IAS_TENANT}.accounts.ondemand.com
            jwks_urls:
              - https://{IAS_TENANT}.accounts.ondemand.com/oauth2/certs
---
apiVersion: gateway.kyma-project.io/v1beta1
kind: APIRule
metadata:
  name: example2
  namespace: test
spec:
  host: example2
  service:
    name: httpbin
    namespace: default
    port: 8000
  gateway: kyma-gateway.kyma-system
  rules:
    - path: /post
      methods: ["POST"]
      accessStrategies:
        - handler: no_auth
    - path: /.*
      methods: ["GET"]
      mutators: []
      accessStrategies:
        - handler: jwt
          config:
            trusted_issuers:
              - https://{IAS_TENANT}.accounts.ondemand.com
            jwks_urls:
              - https://{IAS_TENANT}.accounts.ondemand.com/oauth2/certs
```

To ensure a seamless migration to v2 without any downtime, you must create an additional, temporary `AuthorizationPolicy` before applying the first migrated `APIRule` v2.

> ### Note
>
> Using both versions of `APIRule`s for one target workload is not recommended. Instead, migrate all `APIRule`s targeting the same workload at once to version v2. Creating a temporary `AuthorizationPolicy` blocks internal traffic to the workload, which migration to `APIRule` v2 causes anyway. We recommend the following course of action:
>
> 1. Apply the temporary `AuthorizationPolicy` that blocks in-cluster communication and unblocks v1beta1 exposure to external traffic during the migration.

2. Migrate `APIRule`s to v2. For instructions, follow the migration guidelines.
3. Delete the temporary `AuthorizationPolicy`.

Procedure

1. For each of your `APIRule`s v1beta1 targeting the same workload, list hosts that contain at least one allow or no_auth handler. Specify those hosts in the FQDN format.

   > ### Note
   >
   > If your `APIRule`s v1beta1 don't use the allow or no_auth handlers, skip this step. Specifying these hosts is only necessary for `APIRule`s with allow or no_auth access strategies to allow traffic from Istio ingress gateway to the target workload during migration. Other handlers, such as jwt, oauth2_introspection, and noop, use the Ory Oathkeeper service, and the traffic doesn't come directly from the ingress gateway to the target workload.

   In the example, two `APIRule`s v1beta1 with no_auth handler expose the same `httpbin` Service on different hosts: `example1` and `example2`.

   To obtain the FQDN format of the hosts, get the domain of the referenced Gateway:

   ```
   kubectl get gateway <GATEWAY_NAME> -n <GATEWAY_NAMESPACE> -o
   jsonpath='{.spec.servers[0].hosts}'
   ```

   In the example scenario, the command looks like this:

   ```
   kubectl get gateway -n kyma-system kyma-gateway -o
   jsonpath='{.spec.servers[0].hosts}'
   ["*.local.kyma.dev"]%
   ```

   Assuming the default domain is `local.kyma.dev`, the FQDN format of the hosts is:

   - `example1.local.kyma.dev`
   - `example2.local.kyma.dev`

2. To identify the label key and value for the target workload, check the selector from the Service that the `APIRule`s expose.

   ```
   kubectl get service <SERVICE_NAME> -n <NAMESPACE> -o
   jsonpath='{.spec.selector}'
   ```

   In the example, the selector for the `httpbin` Service in the `default` namespace is `app: httpbin`:

   ```
   kubectl get service httpbin -n default -o jsonpath='{.spec.selector}'
   {"app":"httpbin"}%
   ```

3. Create a temporary `AuthorizationPolicy` using the gathered information in the previous steps.

Option	Description
{NAMESPACE}	The namespace to which the `AuthorizationPolicy` applies. This namespace must include the target workload for which you allow traffic. The selector matches workloads in the same namespace as the `AuthorizationPolicy`.
{HOSTNAME}	List all host names that are exposed by the v1beta1 `APIRule`s being migrated which contain handler no_auth or allow. List all relevant hostnames in FQDN format.
{LABEL_KEY}: {LABEL_VALUE}	Specify label selectors that match the target workload. For more information, see Authorization Policy .

This `AuthorizationPolicy` allows traffic from the Ory Oathkeeper service and the Istio ingress gateway to the target workload only for the specified hosts. Applying this `AuthorizationPolicy` before starting the migration process ensures uninterrupted access to service endpoints during the migration.

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-migration
  namespace: {NAMESPACE}
spec:
  action: ALLOW
  rules:
    - from:
        - source:
            principals:
              - "cluster.local/ns/kyma-system/sa/oathkeeper-maester-account"
    - from:
        - source:
            principals:
              - "cluster.local/ns/istio-system/sa/istio-ingressgateway-
service-account"
      to:
        - operation:
            hosts:
              - {HOSTNAME1}
              - {HOSTNAME2}
  selector:
    matchLabels:
      {LABEL_KEY}: {LABEL_VALUE}
```

If you don't migrate any `APIRule`s v1beta1 with allow or no_auth handlers, remove the second rule from the `AuthorizationPolicy`. The `AuthorizationPolicy` then allows all traffic from the Ory Oathkeeper service to the target workload.

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-migration
  namespace: {NAMESPACE}
spec:
  action: ALLOW
  rules:
    - from:
        - source:
            principals:
```

```
              - "cluster.local/ns/kyma-system/sa/oathkeeper-maester-account"
  selector:
    matchLabels:
      {LABEL_KEY}: {LABEL_VALUE}
```

If you migrate `APIRule`s v1beta1 that specify only the allow or no_auth handlers, remove the first rule from the `AuthorizationPolicy`:

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-migration
  namespace: {NAMESPACE}
spec:
  action: ALLOW
  rules:
    - from:
        - source:
            principals:
              - "cluster.local/ns/istio-system/sa/istio-ingressgateway-
service-account"
      to:
        - operation:
            hosts:
              - {HOSTNAME1}
              - {HOSTNAME2}
  selector:
    matchLabels:
      {LABEL_KEY}: {LABEL_VALUE}
```

In the example scenario, `APIRule`s v1beta1 specify both no_auth and jwt handlers. Therefore, the temporarily applied `AuthorizationPolicy` looks like this:

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-migration
  namespace: test
spec:
  action: ALLOW
  rules:
    - from:
        - source:
            principals:
              - "cluster.local/ns/kyma-system/sa/oathkeeper-maester-account"
    - from:
        - source:
            principals:
              - "cluster.local/ns/istio-system/sa/istio-ingressgateway-
service-account"
      to:
        - operation:
            hosts:
              - example1.local.kyma.dev
              - example2.local.kyma.dev
  selector:
    matchLabels:
      app: httpbin
```

4. To migrate `APIRule`s v1beta1 to v2, follow the steps in the migration guidelines. During this process, the temporary `AuthorizationPolicy` ensures that requests from both v1beta1 `APIRule`s to the target workload are allowed.

   See Migrate APIRule from Version v1beta1 to Version v2 [page 2025].

5. After all `APIRule`s targeting the same workload are migrated to v2, delete the temporary `AuthorizationPolicy`.

   ```
   kubectl delete authorizationpolicy <AUTHORIZATION_POLICY_NAME> -n <NAMESPACE>
   ```

   See an example:

   ```
   kubectl delete authorizationpolicy allow-migration -n default
   ```

4.3.1.2.4.4 Migrate `APIRule` v1beta1 of Type jwt to Version v2

Learn how to migrate an `APIRule` created in version v1beta1 using the jwt handler to version v2.

Prerequisites

- You have read Changes Introduced in APIRule v2 [page 2026], which details the updates implemented in the new version of `APIRule`. If any of these changes affect your setup, you must consider them when migrating to `APIRule` v2 and make the necessary adjustments.
- You have the Istio and API Gateway modules added.
- You have installed `kubectl` and `curl`.
- You have a deployed workload exposed by an `APIRule` in version v1beta1. The `APIRule` uses the jwt handler.

> ### Note
>
> The workload exposed by the `APIRule` in version v2 must be a part of the Istio service mesh. See Enable Istio Sidecar Proxy Injection.

Context

`APIRule` in version v1beta1 is removed, and the API server no longer responds to requests for `APIRule`s in this version. To maintain support for your `APIRule`s, migrate to version v2.

Procedure

1. Retrieve a configuration of the `APIRule` in version v1beta1 and save it for further modifications.

For instructions, see Retrieve the Complete spec of an APIRule in Version v1beta1 [page 2029]. See a sample of the retrieved `spec` in the YAML format. The following configuration uses the jwt handler to expose the HTTPBin service's `/anything` and `/.*` endpoints:

```
host: httpbin.local.kyma.dev
service:
  name: httpbin
  namespace: test
  port: 8000
gateway: kyma-gateway.kyma-system
rules:
  - path: /anything
    methods:
      - POST
    accessStrategies:
      - handler: jwt
        config:
          jwks_urls:
            - https://{IAS_TENANT}.accounts.ondemand.com/oauth2/certs
  - path: /.*
    methods:
      - GET
    accessStrategies:
      - handler: jwt
        config:
          jwks_urls:
            - https://{IAS_TENANT}.accounts.ondemand.com/oauth2/certs
```

> ⚠ Caution
>
> If multiple `APIRule`s target the same workload, you must perform an additional migration step to avoid traffic disruption. This step involves creating an additional, temporary `AuthorizationPolicy` before applying the first migrated `APIRule` v2. For detailed instructions, refer to Migrate Multiple APIRules Targeting the Same Workload from v1beta1 to v2 [page 2031] before proceeding.

2. Adjust the retrieved configuration to align with the jwt configuration of `APIRule` v2.

   To ensure the `APIRule` specification is compatible with version v2, you must include a mandatory field named `issuer` in the jwt handler's configuration. You can find the `issuer` URL in the OIDC well-known configuration of your tenant, located at `https://{YOUR_TENANT}.accounts.ondemand.com/.well-known/openid-configuration`. Additionally, note that the value of the `jwks_urls` field is now stored in the `jwksUri` field. Tokens do not need to be reissued unless they have expired.

   If you previously used a legacy gateway name in any of the following formats, you must update it to the new format: `namespace/gateway-name`

   - `gateway-name.namespace.svc.cluster.local`
   - `gateway-name.namespace.svc.cluster`
   - `gateway-name.namespace.svc`
   - `gateway-name.namespace`
   - `gateway-name`

   See an example of the adjusted `APIRule` configuration for version v2:

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: httpbin
  namespace: test
spec:
```

```
  hosts:
    - httpbin
  service:
    name: httpbin
    namespace: test
    port: 8000
  gateway: kyma-system/kyma-gateway
  rules:
    - jwt:
        authentications:
          - issuer: https://{YOUR_TENANT}.accounts.ondemand.com
            jwksUri: https://{YOUR_TENANT}.accounts.ondemand.com/oauth2/certs
      methods:
        - POST
      path: /anything
    - jwt:
        authentications:
          - issuer: https://{YOUR_TENANT}.accounts.ondemand.com
            jwksUri: https://{YOUR_TENANT}.accounts.ondemand.com/oauth2/certs
      methods:
        - GET
      path: /{**}
```

> ### Note
>
> The `hosts` field accepts a short host name (without a domain). Additionally, the path `/.*` has been changed to `/{**}` because `APIRule` v2 does not support regular expressions in the `spec.rules.path` field. For more information, see Changes Introduced in APIRule v2 [page 2026].

3. Update the `APIRule` to version v2 by applying the adjusted configuration.

   To verify the version of the applied `APIRule`, check the value of the `gateway.kyma-project.io/original-version` annotation in the `APIRule` spec. A value of v2 indicates that the `APIRule` has been successfully migrated. To see the value, run:

   ```
   kubectl get apirules.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME
   -oyaml
   ```

   The following output indicates that the `APIRule` has been successfully migrated to version v2:

   ```
   apiVersion: gateway.kyma-project.io/v2
   kind: APIRule
   metadata:
     annotations:
       gateway.kyma-project.io/original-version: v2
   ...
   ```

   > ### Caution
   >
   > Do not manually change the `gateway.kyma-project.io/original-version` annotation. This annotation is automatically updated when you apply your `APIRule` in version v2. Modifying the annotation's value manually causes your `APIRule` v1beta1 to be handled and configured as version v2, potentially leading to reconciliation errors.

4. To preserve the internal traffic policy from the `APIRule` v1beta1, apply the following `AuthorizationPolicy`.

   In `APIRule` v2, internal traffic is blocked by default. Without this `AuthorizationPolicy`, attempts to connect internally to the workload cause an `RBAC: access denied` error. Ensure that the selector label is updated to match the target workload.

Option	Description
`{NAMESPACE}`	The namespace to which the `AuthorizationPolicy` applies. This namespace must include the target workload for which you allow internal traffic. The selector matches workloads in the same namespace as the `AuthorizationPolicy`.
`{LABEL_KEY}: {LABEL_VALUE}`	To further restrict the scope of the `AuthorizationPolicy`, specify label selectors that match the target workload. Replace these placeholders with the actual key and value of the label. For more information, see Authorization Policy .

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-internal
  namespace: {NAMESPACE}
spec:
  selector:
    matchLabels:
      {LABEL_KEY}: {LABEL_VALUE}
  action: ALLOW
  rules:
  - from:
    - source:
        notPrincipals: ["cluster.local/ns/istio-system/sa/istio-
ingressgateway-service-account"]
```

5. To retain the CORS configuration from the `APIRule` v1beta1, update the `APIRule` in version v2 to include the same CORS settings.

   For preflight requests to work correctly, you must explicitly add the "OPTIONS" method to the `rules.methods` field of your `APIRule` v2.

   > ⚠ Caution
   >
   > If you migrated multiple `APIRules` that target the same workload, and you applied an additional `AuthorizationPolicy` to avoid traffic disruption during migration, delete it. For instructions, see the last point of the procedure in Migrate Multiple APIRules Targeting the Same Workload from v1beta1 to v2 [page 2031].

Results

Verify access to your workload:

- Send a GET request to the exposed workload using JWT authentication:

  ```
  curl -ik -X GET https://{SUBDOMAIN}.{DOMAIN_NAME}/ip --header
  "Authorization:Bearer $ACCESS_TOKEN"
  ```

  If successful, the call returns the `200 OK` response code.

- Send a POST request to the exposed workload using JWT authentication:

  ```
  curl -ik -X POST https://{SUBDOMAIN}.{DOMAIN_NAME}/anything -d "test data" --
  header "Authorization:Bearer $ACCESS_TOKEN"
  ```

  If successful, the call returns the `200 OK` response code.
- Send a POST request to the exposed workload without JWT authentication:

  ```
  curl -ik -X POST https://{SUBDOMAIN}.{DOMAIN_NAME}/anything -d "test data"
  ```

  The call returns the `403` error code.

4.3.1.2.4.5 Migrate `APIRule` v1beta1 of Type noop, allow, or no_auth to Version v2

Learn how to migrate an `APIRule` created in version v1beta1 using the noop, allow, or no_auth handlers to version v2. In `APIRule` v2, the noAuth handler replaces the noop, allow, and no_auth handlers from v1beta1.

Prerequisites

- You have read Changes Introduced in APIRule v2 [page 2026], which details the updates implemented in the new version of `APIRule`. If any of these changes affect your setup, you must consider them when migrating to `APIRule` v2 and make the necessary adjustments.
- You have the Istio and API Gateway modules added.
- You have installed `kubectl` and `curl`.
- You have a deployed workload exposed by an `APIRule` in version v1beta1. The `APIRule` uses the noop, allow, or no_auth handlers.

> ### Note
>
> The workload exposed by the `APIRule` in version v2 must be a part of the Istio service mesh. See Enable Istio Sidecar Proxy Injection.

Context

`APIRule` in version v1beta1 is removed, and the API server no longer responds to requests for `APIRules` in this version. To maintain support for your `APIRules`, migrate to version v2.

Procedure

1. Obtain a configuration of the `APIRule` in version v1beta1 and save it for further modifications.

   For instructions, see Retrieve the Complete spec of an APIRule in Version v1beta1 [page 2029]. See a sample of the retrieved `spec` in the YAML format:

   ```
   host: httpbin.local.kyma.dev
   service:
     name: httpbin
     namespace: test
     port: 8000
   gateway: kyma-gateway.kyma-system
   rules:
     - path: /anything
       methods:
         - POST
       accessStrategies:
         - handler: noop
     - path: /headers
       methods:
         - HEAD
       accessStrategies:
         - handler: allow
     - path: /.*
       methods:
         - GET
       accessStrategies:
         - handler: no_auth
   ```

   This configuration uses the following handlers to expose the HTTPBin endpoints:

   - The noop handler to expose `/anything`
   - The allow handler to expose `/headers`
   - The no_auth handler to expose `/.*`

   > ⚠ Caution
   >
   > If multiple `APIRule`s target the same workload, you must perform an additional migration step to avoid traffic disruption. This step involves creating an additional, temporary `AuthorizationPolicy` before applying the first migrated `APIRule` v2. For detailed instructions, refer to Migrate Multiple APIRules Targeting the Same Workload from v1beta1 to v2 [page 2031] before proceeding.

2. Adjust the obtained configuration to match the v2 `APIRule` specification by replacing the noop, allow, and no_auth handlers with the noAuth handler.

   To do this, you must modify the existing `APIRule spec` and ensure it is valid for the v2 version of the noAuth type.

   If you previously used a legacy gateway name in any of the following formats, you must update it to the new format: `namespace/gateway-name`

   - `gateway-name.namespace.svc.cluster.local`
   - `gateway-name.namespace.svc.cluster`
   - `gateway-name.namespace.svc`
   - `gateway-name.namespace`
   - `gateway-name`

See an example of the adjusted `APIRule`:

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: httpbin
  namespace: test
spec:
  hosts:
    - httpbin
  service:
    name: httpbin
    namespace: test
    port: 8000
  gateway: kyma-system/kyma-gateway
  rules:
    - path: /anything
      methods: ["POST"]
      noAuth: true
    - path: /headers
      methods: ["HEAD"]
      noAuth: true
    - path: /{**}
      methods: ["GET"]
      noAuth: true
```

> ### Note
>
> The `hosts` field accepts a short host name (without a domain). Additionally, the path `/.*` has been changed to `/{**}` because `APIRule` v2 does not support regular expressions in the `spec.rules.path` field. For more information, see Changes Introduced in APIRule v2 [page 2026].

3. To update the `APIRule` to version v2, apply the adjusted configuration.

   To verify the version of the applied `APIRule`, check the value of the `gateway.kyma-project.io/original-version` annotation in the `APIRule` `spec`. If the `APIRule` has been successfully migrated, you see the value v2. Use the following command:

   ```
   kubectl get apirules.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME
   -oyaml
   ```

   The following output indicates that the `APIRule` has been successfully migrated to version v2:

   ```
   apiVersion: gateway.kyma-project.io/v2
   kind: APIRule
   metadata:
     annotations:
       gateway.kyma-project.io/original-version: v2
   ...
   ```

   > ### Caution
   >
   > Do not manually change the `gateway.kyma-project.io/original-version` annotation. This annotation is automatically updated when you apply your `APIRule` in version v2. Modifying the annotation's value manually causes your `APIRule` v1beta1 to be handled and configured as version v2, potentially leading to reconciliation errors.

4. To preserve the internal traffic policy from the `APIRule` v1beta1, apply the following `AuthorizationPolicy`.

In `APIRule` v2, internal traffic is blocked by default. Without this `AuthorizationPolicy`, attempts to connect internally to the workload will result in an `RBAC: access denied` error. Ensure that the selector label is updated to match the target workload.

Option	Description
`{NAMESPACE}`	The namespace to which the `AuthorizationPolicy` applies. This namespace must include the target workload for which you allow internal traffic. The selector matches workloads in the same namespace as the `AuthorizationPolicy`.
`{LABEL_KEY}`: `{LABEL_VALUE}`	To further restrict the scope of the `AuthorizationPolicy`, specify label selectors that match the target workload. Replace these placeholders with the actual key and value of the label. For more information, see Authorization Policy .

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-internal
  namespace: {NAMESPACE}
spec:
  selector:
    matchLabels:
      {LABEL_KEY}: {LABEL_VALUE}
  action: ALLOW
  rules:
  - from:
    - source:
        notPrincipals: ["cluster.local/ns/istio-system/sa/istio-
ingressgateway-service-account"]
```

5. To retain the CORS configuration from the `APIRule` v1beta1, update the `APIRule` in version v2 to include the same CORS settings.

   For preflight requests to work correctly, you must explicitly add the "OPTIONS" method to the `rules.methods` field of your `APIRule` v2.

   > ⚠ Caution
   >
   > If you migrated multiple `APIRule`s that target the same workload, and you applied an additional `AuthorizationPolicy` to avoid traffic disruption during migration, delete it. For instructions, see the last point of the procedure in Migrate Multiple APIRules Targeting the Same Workload from v1beta1 to v2 [page 2031].

Results

Verify access to your workload:

- Send a POST request to the exposed workload:

```
curl -ik -X POST https://{SUBDOMAIN}.{DOMAIN_NAME}/anything -d "test data"
```

If successful, the call returns the `200 OK` response code.

- Send a HEAD request to the exposed workload:

  ```
  curl -ik -I https://{SUBDOMAIN}.{DOMAIN_NAME}/headers
  ```

  If successful, the call returns the `200 OK` response code.
- Send a GET request to the exposed workload:

  ```
  curl -ik -X GET https://{SUBDOMAIN}.{DOMAIN_NAME}/ip
  ```

  If successful, the call returns the `200 OK` response code.

4.3.1.2.4.6 Migrate `APIRule` v1beta1 of Type oauth2_introspection to Version v2

Learn how to migrate an `APIRule` created in version v1beta1 using the oauth2_introspection handler to version v2. In `APIRule` v2, the extAuth handler replaces all Ory Oathkeeper-based handlers used in the v1beta1 version. The instructions focus on oauth2_introspection because it is the most popular Ory Oathkeeper-based handler.

Prerequisites

- You have read Changes Introduced in APIRule v2 [page 2026], which details the updates implemented in the new version of `APIRule`. If any of these changes affect your setup, you must consider them when migrating to `APIRule` v2 and make the necessary adjustments.
- You have the Istio and API Gateway modules added.
- You have installed `kubectl` and `curl`.
- You have a deployed workload exposed by an `APIRule` in version v1beta1. The `APIRule` uses the oauth2_introspection handler.

> **Note**
>
> The workload exposed by the `APIRule` in version v2 must be a part of the Istio service mesh. See Enable Istio Sidecar Proxy Injection.

Context

`APIRule` in version v1beta1 is removed, and the API server no longer responds to requests for `APIRules` in this version. To maintain support for your `APIRules`, migrate to version v2.

Procedure

1. Retrieve a configuration of the `APIRule` in version v1beta1 and save it for further modifications.

   For instructions, see Retrieve the Complete spec of an APIRule in Version v1beta1 [page 2029]. See a sample of the retrieved `spec` in the YAML format. The following configuration uses the oauth2_introspection handler to expose the HTTPBin service's `/anything` and `/.*` endpoints:

   ```
   host: httpbin.local.kyma.dev
   service:
     name: httpbin
     namespace: test
     port: 8000
   gateway: kyma-gateway.kyma-system
   rules:
     - path: /anything
       methods:
         - POST
       accessStrategies:
         - handler: oauth2_introspection
           config:
             introspection_request_headers:
               Authorization: Basic {ENCODED_CREDENTIALS}
             introspection_url: https://{IAS_TENANT}.accounts.ondemand.com/
   oauth2/introspect
             required_scope:
               - write
     - path: /.*
       methods:
         - GET
       accessStrategies:
         - handler: oauth2_introspection
           config:
             introspection_request_headers:
               Authorization: Basic {ENCODED_CREDENTIALS}
             introspection_url: https://{IAS_TENANT}.accounts.ondemand.com/
   oauth2/introspect
             required_scope:
               - read
   ```

   > ⚠ Caution
   >
   > If multiple `APIRule`s target the same workload, you must perform an additional migration step to avoid traffic disruption. This step involves creating an additional, temporary `AuthorizationPolicy` before applying the first migrated `APIRule` v2. For detailed instructions, refer to Migrate Multiple APIRules Targeting the Same Workload from v1beta1 to v2 [page 2031] before proceeding.

2. Deploy a service that acts as an external authorizer for Istio.

   The following instructions use OAuth2 Proxy with an OAuth2.0-compliant authorization server supporting OIDC discovery.

   a. Replace the placeholders and create the `values.yaml` file with the OAuth2 Proxy configuration.

      ```
      cat <<EOF > values.yaml
      config:
        clientID: {CLIENT_ID}
        clientSecret: {CLIENT_SECRET}
        cookieName: ""
        cookieSecret: {COOKIE_SECRET}
      extraArgs:
        auth-logging: true
      ```

```
  cookie-domain: "{DOMAIN_TO_EXPOSE_WORKLOADS}"
  cookie-samesite: lax
  cookie-secure: false
  force-json-errors: true
  login-url: static://401
  oidc-issuer-url: {OIDC_ISSUER_URL}
  pass-access-token: true
  pass-authorization-header: true
  pass-host-header: true
  pass-user-headers: true
  provider: oidc
  request-logging: true
  reverse-proxy: true
  scope: "{TOKEN_SCOPES}"
  set-authorization-header: true
  set-xauthrequest: true
  skip-jwt-bearer-tokens: true
  skip-oidc-discovery: false
  skip-provider-button: true
  standard-logging: true
  upstream: static://200
  whitelist-domain: "*.{DOMAIN_TO_EXPOSE_WORKLOADS}:*"
EOF
```

The example shows the configuration of OAuth2 Proxy with the following parameters:

- `CLIENT_SECRET`, `CLIENT_ID`, and `OIDC_ISSUER_URL`: To get them, follow the guide for getting a JSON Web Token (JWT).
- `DOMAIN_TO_EXPOSE_WORKLOADS`: Refers to either a custom domain or the default domain `local.kyma.dev`.
- `COOKIE_SECRET`: Generate using the following command:

  ```
  openssl rand -base64 32 | tr -- '+/' '-_'
  ```

- `TOKEN_SCOPES`: Specifies the OAuth scopes. Each provider has a default set of scopes that are used if you haven't configured custom scopes. For a list of options and further details, refer to the OAuth2 Proxy documentation .

b. To install OAuth2 Proxy with your configuration, use the OAuth2 Proxy Helm chart .

```
kubectl create namespace oauth2-proxy
helm repo add oauth2-proxy https://oauth2-proxy.github.io/manifests
helm upgrade --install oauth2-proxy oauth2-proxy/oauth2-proxy -f
values.yaml -n oauth2-proxy
```

c. Register OAuth2 Proxy as an authorization provider in the Istio module.

```
kubectl patch istio -n kyma-system
default --type merge --patch '{"spec":{"config":
{"authorizers":[{"name":"oauth2-proxy","port":80,"service":"oauth2-
proxy.oauth2-proxy.svc.cluster.local","headers":{"inCheck":{"include":["x-
forwarded-for", "cookie", "authorization"]}}}]}}}'
```

3. Adjust the obtained configuration of the `APIRule` to use the extAuth handler in version v2.

   The following `APIRule` example delegates token validation to the previously configured OAuth2 Proxy. Existing tokens stay valid throughout the migration, ensuring that the process does not disrupt any exposed or secured workloads.

   If you previously used a legacy gateway name in any of the following formats, you must update it to the new format: `namespace/gateway-name`

   - `gateway-name.namespace.svc.cluster.local`

- `gateway-name.namespace.svc.cluster`
- `gateway-name.namespace.svc`
- `gateway-name.namespace`
- `gateway-name`

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: httpbin
  namespace: test
spec:
  hosts:
    - httpbin
  service:
    name: httpbin
    namespace: test
    port: 8000
  gateway: kyma-system/kyma-gateway
  rules:
    - extAuth:
        authorizers:
          - oauth2-proxy
      methods:
        - POST
      path: /anything
    - extAuth:
        authorizers:
          - oauth2-proxy
      methods:
        - GET
      path: /{**}
```

> ### Note
>
> The `hosts` field accepts a short host name (without a domain). Additionally, the path `/.*` has been changed to `/{**}` because `APIRule` v2 does not support regular expressions in the `spec.rules.path` field. For more information, see Changes Introduced in APIRule v2 [page 2026].

4. Update the `APIRule` to version v2 by applying the adjusted configuration.

   To verify the version of the applied `APIRule`, check the value of the `gateway.kyma-project.io/original-version` annotation in the `APIRule` `spec`. A value of v2 indicates that the `APIRule` has been successfully migrated. To see the value, run:

   ```
   kubectl get apirules.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME
   -oyaml
   ```

   The following output indicates that the `APIRule` is successfully migrated to version v2:

   ```
   apiVersion: gateway.kyma-project.io/v2
   kind: APIRule
   metadata:
     annotations:
       gateway.kyma-project.io/original-version: v2
   ...
   ```

   > ### Caution
   >
   > Do not manually change the `gateway.kyma-project.io/original-version` annotation. This annotation is automatically updated when you apply your `APIRule` in version v2.

5. To preserve the internal traffic policy from the `APIRule` v1beta1, apply the following `AuthorizationPolicy`.

   In `APIRule` v2, internal traffic is blocked by default. Without this `AuthorizationPolicy`, attempts to connect internally to the workload will result in an `RBAC: access denied` error. Ensure that the selector label is updated to match the target workload.

Option	Description
`{NAMESPACE}`	The namespace to which the `AuthorizationPolicy` applies. This namespace must include the target workload for which you allow internal traffic.
`{LABEL_KEY}`: `{LABEL_VALUE}`	Specify label selectors that match the target workload. For more information, see Authorization Policy .

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-internal
  namespace: {NAMESPACE}
spec:
  selector:
    matchLabels:
      {LABEL_KEY}: {LABEL_VALUE}
  action: ALLOW
  rules:
  - from:
    - source:
        notPrincipals: ["cluster.local/ns/istio-system/sa/istio-
ingressgateway-service-account"]
```

6. To retain the CORS configuration from the `APIRule` v1beta1, update the `APIRule` in version v2 to include the same CORS settings.

   For preflight requests to work correctly, you must explicitly add the "OPTIONS" method to the `rules.methods` field of your `APIRule` v2.

> ⚠ Caution
>
> If you migrated multiple `APIRule`s that target the same workload, and you applied an additional `AuthorizationPolicy` to avoid traffic disruption during migration, delete it. For instructions, see the last point of the procedure in Migrate Multiple APIRules Targeting the Same Workload from v1beta1 to v2 [page 2031].

Results

Verify access to your workload:

- Send a GET request to the exposed workload using JWT authentication:

```
curl -ik -X GET https://{SUBDOMAIN}.{DOMAIN_NAME}/ip --header
"Authorization:Bearer $ACCESS_TOKEN"
```

  If successful, the call returns the `200 OK` response code.

- Send a POST request to the exposed workload using JWT authentication:

  ```
  curl -ik -X POST https://{SUBDOMAIN}.{DOMAIN_NAME}/anything -d "test data" --
  header "Authorization:Bearer $ACCESS_TOKEN"
  ```

  If successful, the call returns the `200 OK` response code.
- Send a POST request to the exposed workload without JWT authentication:

  ```
  curl -ik -X POST https://{SUBDOMAIN}.{DOMAIN_NAME}/anything -d "test data"
  ```

  The call returns the `401` error code.

4.3.1.2.4.7 FAQ

`APIRule` CRD v2 is the latest stable version. See the frequently asked questions related to the migration process.

Displaying an `APIRule`'s Spec

Why doesn't my APIRule contain any rules?

This `APIRule` is not migrated to version v2. Since version v2 is now the default, when you request an `APIRule`, `kubectl` converts it to version v2. This conversion only affects the displayed resource's textual format and does not modify the resource in the cluster. If the full conversion is possible, the `rules` field is presented in the output. However, if the conversion cannot be completed, the rules are missing, and the original rules are stored in the resource's annotation `gateway.kyma-project.io/v1beta1-spec`. For more information, see Retrieve the Complete spec of an APIRule in Version v1beta1 [page 2029].

Why doesn't my APIRule contain a Gateway?

If your `APIRule` doesn't contain the Gateway when displayed using `kubectl`, this means that your `APIRule` is in version v1beta1 and uses an unsupported Gateway format. The `APIRule` v2 supports only the Gateway format `namespace/gateway-name`. When you try to display the `APIRule` v1beta1 using `kubectl`, its textual format is converted to version v2. Since the Gateway format you're using is neither available in version v2 nor v1beta1, it is not included in the output.

How do I request an APIRule in a particular version?

Specify the version you want to request in the `kubectl` command.

To get version v1beta1, run:

```
kubectl get apirules.v1beta1.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME
-o yaml
```

To get version v2alpha1, run:

```
kubectl get apirules.v2alpha1.gateway.kyma-project.io -n $NAMESPACE
$APIRULE_NAME -o yaml
```

To get version v2, run:

```
kubectl get apirules.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME -o yaml
```

Version v2 is the stored version, so `kubectl` uses it by default to display your `APIRules`, no matter if you specify version v2 in the command or do not specify any version.

Why doesn't Kyma dashboard display all my APIRules?

As part of `APIRule` v1beta1 deletion, `APIRule` v1beta1 support has been removed from Kyma dashboard. To display or migrate `APIRules` v1beta1, use `kubectl`. See Migrate APIRule from Version v1beta1 to Version v2 [page 2025].

Checking an `APIRule`'s Version

Why does the kubectl get command return my APIRule in version v2?

`APIRule` v2 is now the default version displayed by `kubectl`. This means that no matter in which version the `APIRule` was actually created in the cluster, `kubectl` converts the `APIRule`'s displayed textual format to the latest stable version v2. It does not modify the resource in the cluster.

How do I check which version of APIRule I'm using?

To check the version of your `APIRule`, run the following command:

```
kubectl get apirules.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME -o yaml
```

The annotation `gateway.kyma-project.io/original-version` specifies the version of your `APIRule`.

Migrating an `APIRule` v1beta1 to Version v2

By when must I migrate my APIRules v1beta1?

To migrate `APIRules` v1beta1 to v2 with no downtime, you must do it before release 3.10, which disables reconciliation of `APIRules` and their subresources. It is scheduled for 19 August 2026 (fast channel) and 2 September 2026 (regular channel). Migration remains possible after these dates, but zero downtime is no longer guaranteed.

How do I know which APIRules must be migrated?

You must migrate all `APIRules` v1beta1 to version v2. To list all your `APIRules` v1beta1, run the following command:

```
kubectl get apirules.gateway.kyma-project.io -A -o json | jq '.items[]
| select(.metadata.annotations["gateway.kyma-project.io/original-version"] ==
"v1beta1") | {namespace: .metadata.namespace, name: .metadata.name}'
```

If kubectl get returns an APIRule in version v2, does it mean that my APIRule is migrated to v2?

No. `APIRule` v2 is now the default version displayed by `kubectl`. `kubectl` converts the textual format of each `APIRule`, no matter what its original version is. So, if your `APIRule` is in version v1beta1, `kubectl` converts it to version v2 and displays it in the command's output. This conversion does not affect the resource itself.

To verify if your `APIRule` is migrated, check the annotation `gateway.kyma-project.io/original-version`. If it specifies version v2, your `APIRule` is migrated. If the annotation is `gateway.kyma-project.io/original-version: v1beta1`, this means that the resource is in version v1beta1 even though in the command line it is converted to match the v2 specification.

> ### Note
>
> Do not manually change the `gateway.kyma-project.io/original-version` annotation. This annotation is automatically updated when you migrate your `APIRule` to version v2.

Why do I get CORS policy errors after applying APIRule v2?

`APIRule` v1beta1 applied the default CORS configuration. `APIRule` v2 does not apply any default values, which means that by default, it is only allowed to request resources from the same origin from which the application is loaded. If you want to use a less restrictive CORS policy in `APIRule` v2, you must define it in the `spec.corsPolicy` field. For more information, see Changes Introduced in APIRule v2 [page 2026].

I used oauth2_introspection in APIRule v1beta1. How do I migrate it to v2?

The oauth2-introspection handler is removed from `APIRule` v2. To migrate your `APIRule` that uses this handler, you must first deploy a service that acts as an external authorizer for Istio and then define the extAuth access strategy in your `APIRule` CR. See Migrate APIRule v1beta1 of Type oauth2_introspection to Version v2 [page 2044].

I used regexp in the paths of APIRule v1beta1. How do I migrate it to v2?

`APIRule` v2 does not support regexp in the `spec.rules.path` field of `APIRule` CR. Instead, it supports using the {*} and {**} operators and the /* wildcard. For more information, see Changes Introduced in APIRule v2 [page 2026].

Why do I get a validation error for the legacy gateway format while trying to migrate to v2?

In `APIRule` v2, you must provide the Gateway using the format `namespace/gateway-name`. The legacy formats are not supported.

How do I migrate multiple APIRules v1beta1 targeting the same workload to version v2?

When multiple `APIRule`s v1beta1 target the same workload using different host names, you must apply an additional, temporary `AuthorizationPolicy` in order to have continuous access to endpoints exposed by `APIRule`s v1beta1 during the migration. For more information, see Migrate Multiple APIRules Targeting the Same Workload from v1beta1 to v2 [page 2031] and Migrate APIRule from Version v1beta1 to Version v2 [page 2025].

Using `APIRule`s v1beta1

Why are my APIRules v1beta1 in the `Warning` state?

When a resource is in the `Warning` state, it signifies that user action is required. All `APIRule`s v1beta1 are set to this state to indicate that you must migrate these resources to version v2.

When will APIRules v1beta1 stop being reconciled?

Release 3.10 disables migration and reconciliation of `APIRule`s v1beta1. For the complete timeline for SAP BTP, Kyma runtime, follow API Gateway what's new notes.

Once reconciliation is disabled, `APIRule`s v1beta1 will continue to function as currently configured, but the API Gateway module will no longer own or manage them. Changes you make will not be reverted, and unmanaged resources may cause disruptions in workload availability and access.

Why can't I create APIRules v1beta1?

It's no longer possible to create, modify, or delete `APIRule` CRs v1beta1 in new and existing clusters. All `APIRule` v1beta1 configurations set up prior to this restriction remain active and continue to function as expected. Reconciliation of these resources will be disabled with release 3.10. For the complete timeline for SAP BTP, Kyma runtime, follow API Gateway what's new notes.

4.3.1.2.5 Troubleshooting for the API Gateway Module

Troubleshoot problems related to the API Gateway module.

Issues when Creating APIRule CRs [page 2052]

Certificate Management: Issuer Not Created [page 2055]

Kyma Gateway Not Reachable [page 2056]

401 Unauthorized or 403 Forbidden [page 2056]

Connection Refused or Timeout [page 2057]

Blocked In-Cluster Communication (RBAC: Access Denied) [page 2058]

APIRule v2 Doesn't Contain Rules [page 2060]

APIRule v2alpha1 Doesn't Contain Rules [page 2061]

APIRule v2 Contains a Changed Status Schema [page 2062]

APIRules Are Missing from Kyma Dashboard [page 2063]

ExternalGateway Troubleshooting [page 2064]

Learn how to diagnose and resolve common `ExternalGateway` issues, including requests not reaching Kyma, rejected TLS handshake, certificate validation filter failures, not routed host, and unreachable workloads.

4.3.1.2.5.1 Issues when Creating APIRule CRs

This document offers solutions for common problems that may arise when creating an APIRule custom resource (CR) and outlines possible reasons why the CR might be in an error state after creation.

To check the state of an APIRule CR, run:

```
kubectl get apirules httpbin
```

If the CR is in the error state, you get a response similar to this one:

```
NAME      STATUS   HOST
httpbin   ERROR    httpbin.xxx.shoot.canary.k8s-hana.ondemand.com
```

The error may signify that your APIRule CR is in an inconsistent state and the service cannot be properly exposed. To check the error message of the APIRule CR, run:

```
kubectl get apirules -n {NAMESPACE} {APIRULE_NAME}
-o=jsonpath='{.status.description}'
```

The `issuer` Configuration of the `jwt` Access Strategy is Missing

Symptom

You get the following error:

```
{"code":"ERROR","description":"Validation error: Attribute
\".spec.rules[0].jwt\": supplied config cannot be empty"}
```

Cause

Your APIRule is missing the `issuer` configuration for the `jwt` access strategy. See an example:

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  ...
spec:
  ...
  rules:
    - path: /.*
      jwt:
```

Solution

Add JWT configuration for the **issuer** or use the `` ` `` symbols. Here's an example of a valid configuration:

```
spec:
  ...
  rules:
    - path: /.*
      methods: ["GET"]
      jwt:
        authentications:
          - issuer: "https://dev.kyma.local"
            jwksUri: "https://example.com/.well-known/jwks.json"
```

Invalid `issuer` Configuration for the `jwt` Access Strategy

Symptom

You get the following error and the APIRule CR is not created:

```
The APIRule "httpbin" is invalid: .spec.rules[0].jwt.authentications[0].issuer:
value is empty or not a valid URI
```

Cause

If the `issuer` contains a colon '`:`', it must be a valid URI. Here's an example of the APIRule CR with an invalid `issuer` URL configuration:

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  ...
spec:
  ...
  rules:
    - path: /.*
      jwt:
        authentications:
          - issuer: ://unsecured.or.not.valid.url
            jwksUri: https://example.com/.well-known/jwks.json
```

Solution

The JWT **issuer** must not be empty and must be a valid URI, for example:

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  ...
spec:
  ...
  rules:
    - path: /.*
      jwt:
        authentications:
          - issuer: https://dev.kyma.local
            jwksUri: https://dev.kyma.local/.well-known/jwks.json
```

Both `noAuth` and `jwt` Access Strategies Defined on the Same Path

Symptom

You get the following error:

```
{"code":"ERROR","description":"Validation error: Attribute
\".spec.rules[0].noAuth\": noAuth access strategy is not allowed in combination
with other access strategies"}
```

Cause

You must not set the `noAuth` access strategy to noAuth and define the `jwt` configuration on the same path. See an example of APIRule CR with invalid configuration:

```
spec:
  ...
  rules:
    - path: /.*
      noAuth: true
      jwt:
        authentications:
          - issuer: https://dev.kyma.local
            jwksUri: https://dev.kyma.local/.well-known/jwks.json
```

Solution

Decide on one configuration you want to use. You can either use `noAuth` access to the specific path or restrict it using a JWT security token.

Occupied Host

Symptom

You get the following error:

```
{"code":"ERROR","description":"Validation error: Attribute \".spec.host\": This
host is occupied by another Virtual Service"}
```

Cause

Your APIRule CR specifies a host that is already used by another APIRule or Virtual Service. See an example of two APIRule CRs that use the same host:

```
spec:
  host: httpbin.xxx.shoot.canary.k8s-hana.ondemand.com
```

```
spec:
  host: httpbin.xxx.shoot.canary.k8s-hana.ondemand.com
```

Solution

Use a different host for one of the APIRule CRs. See an example:

```
spec:
  httpbin-new.xxx.shoot.canary.k8s-hana.ondemand.com
```

```
spec:
  host: httpbin.xxx.shoot.canary.k8s-hana.ondemand.com
```

4.3.1.2.5.2 Certificate Management: Issuer Not Created

Symptom

When you try to create an Issuer custom resource (CR) using `cert.gardener.cloud/v1alpha1`, the resource is not created. There are no logs in the `cert-management` controller.

Cause

The `cert-management` controller does not watch the namespace in which you created the Issuesr CR. By default, `cert-management` watches the `default` namespace for all Issuer CRs.

Solution

To make sure that you created the Issuer CR in the `default` namespace, run: `kubectl get issuers -A`. If you want to create the Issuer CR in a different namespace, adjust `cert-management` settings during the installation.

4.3.1.2.5.3 Kyma Gateway Not Reachable

Symptom

You cannot access Services or Functions exposed using APIRules. Kyma Gateway refuses the connection.

Cause

The issue occurs when the default `kyma-gateway` is either renamed or duplicated. Once you have multiple Gateway resources pointing to the same host, the Gateway CR created first takes precedence over the others.

Solution

Having two Gateway resources pointing to the same host is not recommended. To resolve the issue, make sure the default `kyma-gateway` exists and is not renamed or duplicated. If there are multiple Gateway resources pointing to the same host, delete the duplicated Gateway. To delete the Gateway resource, run: `kubectl -n kyma-system delete gateway {DUPLICATED_GATEWAY_NAME}`.

4.3.1.2.5.4 401 Unauthorized or 403 Forbidden

Symptom

When you try to reach a service, you get `401 Unauthorized` or `403 Forbidden` in response.

Cause

The error `401 Unauthorized` occurs when you try to access a Service that requires authentication, and the appropriate credentials were not provided or were incorrect. You get the error `403 Forbidden` when you try to access a Service or perform an action for which you lack permission.

Solution

Make sure that you are using an active JSON Web Token (JWT) with proper scopes.

1. Decode the JWT.
2. Check the validity and scopes of the JWT:

```
{
   "sub": ********,
   "scp": "test",
   "aud": ********,
   "iss": ******,
   "exp": 1697120462,
   "iat": ******,
   "jti": ******,
}
```

3. Get a JWT if needed.

4.3.1.2.5.5 Connection Refused or Timeout

Symptom

After you have finished all the steps required to set up your custom domain, you receive the `connection refused` or `connection timeout` error when you try to expose a Service. It shows up when you call the Service endpoint by sending a `GET` request. The error looks as follows:

```
curl: (7) Failed to connect to httpbin.mydomain.com port 443: Connection refused
```

Cause

DNS resolves to an incorrect IP address.

Solution

Check if the IP address provided as the value of the `spec.targets` parameter of the DNSEntry custom resource is the IP address of the Ingress Gateway you are using. To check the Ingress Gateway IP, run:

```
kubectl -n istio-system get service istio-ingressgateway -o
jsonpath='{.status.loadBalancer.ingress[0].ip}'
```

In addition, ensure that your OS resolves the target host name to the same Ingress Gateway IP address. Run:

```
host {YOUR_SUBDOMAIN}
```

> ⓘ Note
>
> **YOUR_SUBDOMAIN** specifies the name of your subdomain, for example, `httpbin.mydomain.com`.

4.3.1.2.5.6 Blocked In-Cluster Communication (RBAC: Access Denied)

Symptom

After switching from APIRule custom resource (CR) `v1beta1` to version `v2`, in-cluster communication is blocked. Attempts to connect internally to the workload result in the error `RBAC: access denied`.

Cause

By default, access to the workload from internal traffic is blocked if an APIRule CR in version `v2` or `v2alpha1` is applied. This approach aligns with Kyma's "secure by default" principle.

Solution

If APIRule is applied, internal traffic is blocked by default. To allow it, you must create an ALLOW-type AuthorizationPolicy.

See the following example of an `AuthorizationPolicy` that allows internal traffic to the given workload. Note that it excludes traffic coming from `istio-ingressgateway` not to interfere with policies applied by APIRule to external traffic.

To use this code sample, replace `{NAMESPACE}`, `{LABEL_KEY}`, and `{LABEL_VALUE}` with the values appropriate for your environment.

Option	Description
{NAMESPACE}	The namespace to which the AuthorizationPolicy applies. This namespace must include the target workload for which you allow internal traffic. The selector matches workloads in the same namespace as the AuthorizationPolicy.
{LABEL_KEY}: {LABEL_VALUE}	To further restrict the scope of the AuthorizationPolicy, specify label selectors that match the target workload. Replace these placeholders with the actual key and value of the label. The label indicates a specific set of Pods to which a policy should be applied. The scope of the label search is restricted to the configuration namespace in which the AuthorizationPolicy is present. For more information, see Authorization Policy .

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-internal
  namespace: {NAMESPACE}
spec:
  selector:
    matchLabels:
      {LABEL_KEY}: {LABEL_VALUE}
  action: ALLOW
  rules:
  - from:
    - source:
        notPrincipals: ["cluster.local/ns/istio-system/sa/istio-ingressgateway-
service-account"]
```

For example, let's suppose you have deployed the HTTPBin Service and exposed it using an APIRule `v2` or `v2alpha1`. To apply this policy to the HTTPBin Service labeled with `app: httpbin` deployed in the `default` namespace, you must set `{NAMESPACE}` to default, `{LABEL_KEY}` to app, and `{LABEL_VALUE}` to httpbin.

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-internal
  namespace: default
spec:
  selector:
    matchLabels:
      app: httpbin
  action: ALLOW
  rules:
  - from:
    - source:
        notPrincipals: ["cluster.local/ns/istio-system/sa/istio-ingressgateway-
service-account"]
```

Verification

To test if the internal traffic reaches the exposed workload, follow the steps:

1. Create a sample namespace with enabled Istio sidecar proxy injection:

```
kubectl create ns curl-test
kubectl label namespace curl-test istio-injection=enabled --overwrite
```

2. Run a Pod inside the Istio service mesh:

```
kubectl run -n curl-test --image=curlimages/curl test-internal -- /bin/sleep
3600
```

3. To test the internal communication, replace the placeholders and run the following command:

Option	Description
{SERVICE_NAME}	The name of the exposed Service.
{NAMESPACE}	The namespace in which the Service is deployed.
{PORT}	The port number on which the Service is listening for incoming traffic. Defined in the `port` field of the Service's configuration.
{EXPOSED_PATH}	A path exposed by the Service that you want to access.

```
kubectl exec -ti -n curl-test test-internal -- curl http://{SERVICE_NAME}.
{NAMESPACE}.svc.cluster.local:{PORT}/{EXPOSED_PATH}
```

For example, to test the connection to the HTTPBin Service exposed using an APIRule `v2` or `v2alpha1` on the /get path, you can use the following command:

```
kubectl exec -ti -n curl-test test-internal -- curl http://
httpbin.default.svc.cluster.local:8000/get
```

If successful, the command returns the contents of the specified path.

4. To clean up the resources created for testing, delete the namespace `curl-test` from the cluster. Deleting the namespace removes all resources within it.

```
kubectl delete namespace curl-test
```

4.3.1.2.5.7 APIRule v2 Doesn't Contain Rules

Symptom

An APIRule custom resource (CR) does not contain the `rules` field, for example:

```
kubectl get apirules.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME -o yaml
```

```
apiVersion: gateway.kyma-project.io/v2
```

```
kind: APIRule
metadata:
  name: httpbin
  namespace: test
spec:
  gateway: kyma-system/kyma-gateway
  hosts:
    - httpbin.local.kyma.dev
  service:
    name: httpbin
    namespace: test
    port: 8000
status:
  lastProcessedTime: "2025-04-25T11:16:11Z"
  state: Ready
```

Cause

The APIRule was originally created using version `v1beta1`, and you haven't yet migrated it to version `v2`. Since the latest stable version of the APIRule in the Kubernetes API is now `v2`, running the `kubectl get` command without specifying a version of APIRule assumes version `v2`.

To display the resource in version `v2`, a conversion from `v1beta1` to `v2` is performed. This conversion only affects the displayed resource's textual format and does not modify the resource in the cluster. If the full conversion is possible, the `rules` field is presented in the output. However, if the conversion cannot be completed, the rules are missing, and the original rules are stored in the resource's annotations.

Solution

Specify explicitly `v1beta1` version when requesting the APIRule resource:

```
kubectl get apirules.v1beta1.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME
-o yaml
```

4.3.1.2.5.8 APIRule `v2alpha1` Doesn't Contain Rules

Symptom

An APIRule custom resource (CR) does not contain the `rules` field, for example:

```
kubectl get apirules.v2alpha1.gateway.kyma-project.io -n $NAMESPACE
$APIRULE_NAME -o yaml
```

```
apiVersion: gateway.kyma-project.io/v2alpha1
kind: APIRule
metadata:
  name: httpbin
  namespace: test
```

```
spec:
  gateway: kyma-system/kyma-gateway
  hosts:
    - httpbin.local.kyma.dev
  service:
    name: httpbin
    namespace: test
    port: 8000
status:
  lastProcessedTime: "2025-04-25T11:16:11Z"
  state: Ready
```

Cause

The APIRule was originally created using version `v1beta1`, and you haven't yet migrated it to version `v2`.

To display the resource in version `v2alpha1`, a conversion from `v1beta1` to `v2alpha1` is performed. This conversion only affects the displayed resource's textual format and does not modify the resource in the cluster. If the full conversion is possible, the `rules` field is presented in the output. However, if the conversion cannot be completed, the rules are missing, and the original rules are stored in the resource's annotations.

Solution

Specify explicitly `v1beta1` version when requesting the APIRule resource:

```
kubectl get apirules.v1beta1.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME
-o yaml
```

4.3.1.2.5.9 APIRule v2 Contains a Changed Status Schema

Symptom

There is a changed `status` schema in an APIRule custom resource (CR), for example:

```
kubectl get apirules.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME -o yaml
```

```
...
status:
  lastProcessedTime: "2025-04-25T11:16v:11Z"
  state: Ready
```

Cause

The schema of the **status.state** field in the `v2` APIRule CR introduces a unified approach, similar to the one used in the API Gateway CR. The possible states of the **status.state** field are `Ready`, `Warning`, `Error`, `Processing`, or `Deleting`.

Solution

To get the APIRule in its original version, run:

```
kubectl get apirules.v1beta1.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME
-o yaml
```

You get an output similar to this one:

```
...
status:
APIRuleStatus:
  code: OK
accessRuleStatus:
  code: OK
lastProcessedTime: "2025-04-25T11:16:11Z"
observedGeneration: 1
virtualServiceStatus:
  code: OK
```

4.3.1.2.5.10 APIRules Are Missing from Kyma Dashboard

If you created APIRules using version `v1beta1`, and you have not yet migrated them to version `v2`, they are not displayed in Kyma dashboard's *API Rules* view. To display APIRules in Kyma dashboard, you must migrate them to version `v2`.

Symptom

Kyma dashboard's *API Rules* view does not display APIRules created in version `v1beta1`. To check if the APIRules are present in the cluster, run the following command:

```
kubectl get apirules.v2.gateway.kyma-project.io -A
```

This command lists all APIRules available in your Kyma cluster, regardless of their original version. To get a specific APIRule and check the version in which it was created, run the following command:

```
kubectl get apirules.v2.gateway.kyma-project.io -n $NAMESPACE $APIRULE_NAME -o
yaml
```

If the APIRule was created using version `v1beta1`, the output contains the annotation `gateway.kyma-project.io/original-version: v1beta1`.

See the following example:

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  annotations:
    gateway.kyma-project.io/original-version: v1beta1
  name: httpbin
  namespace: test
spec:
    gateway: kyma-system/kyma-gateway
    hosts:
        - httpbin.local.kyma.dev
    service:
        name: httpbin
        namespace: test
        port: 8000
status:
    lastProcessedTime: "2025-04-25T11:16:11Z"
    state: Warning
```

Cause

APIRules that are not displayed in Kyma dashboard were originally created using version `v1beta1`, and you haven't yet migrated them to version `v2`. The APIRule `v1beta1` API is no longer available via Kyma dashboard.

Solution

To make sure that support for your APIRules is maintained, you must migrate them to version `v2`. To learn how to do this, see Migrate APIRule from Version v1beta1 to Version v2 [page 2025].

4.3.1.2.5.11 ExternalGateway Troubleshooting

Learn how to diagnose and resolve common `ExternalGateway` issues, including requests not reaching Kyma, rejected TLS handshake, certificate validation filter failures, not routed host, and unreachable workloads.

If your `ExternalGateway` or the traffic that flows through it is not behaving as expected, start troubleshooting by running the following command:

```
kubectl describe externalgateway <name> -n <namespace>
```

If the `ExternalGateway` status is `Ready=False`, jump to ExternalGateway Readiness Conditions [page 2068].

Otherwise, match your symptom to one of the following failure modes.

Request Doesn't Reach Kyma

Symptom

You send a request to your `ExternalGateway` host, but the connection is terminated without any Envoy response headers.

Cause

The request never reaches the Kyma Istio ingress gateway. See the common root causes:

- The `DNSEntry` for the internal domain has not been provisioned yet, or its target LoadBalancer address is empty (Gardener has not populated `istio-ingressgateway`).
- The external gateway (the appliance outside the cluster) is not configured to route to the Kyma internal domain.
- The Kyma cluster's LoadBalancer Service is not reachable from where the external gateway resolves it.

Solution

1. Check the `ExternalGateway` status first. If `Ready=False`, see ExternalGateway Readiness Conditions [page 2068].
2. Verify that the `DNSEntry` for the internal domain exists and is ready:

   ```
   kubectl get dnsentry -n istio-system | grep <externalgateway-name>
   kubectl describe dnsentry -n istio-system <dnsentry-name>
   ```

3. Verify that the Istio ingress gateway Service has a LoadBalancer address:

   ```
   kubectl get svc istio-ingressgateway -n istio-system
   ```

   If the `EXTERNAL-IP` column is empty or `<pending>`, the request cannot reach the cluster.

TLS Handshake Rejected

Symptom

Your request reaches the Kyma Istio ingress gateway, but the TLS handshake fails with one of the following errors:

- `tls: unknown certificate authority`
- `alert: bad certificate`
- `alert: unknown ca`
- The client certificate is rejected.

Cause

- The external gateway's client certificate is not signed by the CA in the referenced `spec.caSecretRef` Secret.
- The `spec.caSecretRef` Secret is missing, empty, or does not contain a valid PEM-encoded CA.
- The Istio Gateway is configured for mTLS MUTUAL, but the external gateway is not sending a client certificate.
- The SNI in the connection does not match either the external or the internal domain configured on the Istio Gateway.

Solution

1. Check the `ExternalGateway` status. Look for `Ready=False` with the reason `CASecretNotFound`, `CASecretKeyAmbiguous`, or `CASecretInvalid`. See ExternalGateway Readiness Conditions [page 2068].
2. Inspect the CA Secret content:

   ```
   kubectl get secret <ca-secret-name> -n <namespace> -o
   jsonpath='{.data.ca\.crt}' | base64 -d | openssl x509 -noout -subject -issuer
   -dates
   ```

   Confirm it is a valid, unexpired CA certificate.

Certificate Validation Filter Failed

Symptom

TLS handshake completes successfully, but the request is answered with HTTP `403 Forbidden` and a body of `Forbidden` (no JSON envelope). Requests never reach your workload.

Cause

The Kyma-side certificate validation `EnvoyFilter` rejected the external gateway's client certificate because its `Subject` does not match any entry in the RegionsConfigMap for the configured region.

A common root cause is that the `spec.region` of the `ExternalGateway` points to the wrong region entry.

Solution

1. Check the `ExternalGateway` status. Look for `Ready=False` with the reason `RegionNotFound` or `RegionHasNoSubjects`. See ExternalGateway Readiness Conditions [page 2068].
2. Print the RegionsConfigMap and confirm your region has the expected subjects:

   ```
   kubectl get configmap <regions-configmap> -n <namespace> -o yaml
   ```

Host Not Routed

Symptom

Both TLS and certificate validation succeed, but you receive HTTP `404 Not Found` or `502 Bad Gateway` error with an Envoy-shaped body. The request never reaches your workload.

Cause

The `Host` header on the request does not match any `VirtualService` or `APIRule` `hosts`.

- Your `APIRule` `spec.hosts` does not include the domain in the `Host` header.
- The external gateway sends a different `Host` header than you expect (for example, a `.internal` variant when the `APIRule` expects the customer-facing domain).
- The `APIRule` references the wrong `ExternalGateway` via `spec.externalGateway: ns/name`.

Solution

1. Verify the `APIRule` and its host list:

   ```
   kubectl get apirule <name> -n <namespace> -o yaml
   ```

   Confirm `spec.hosts` contains the exact domain you send in the `Host` header, and that `spec.externalGateway` matches the `ExternalGateway` name and namespace.
2. Inspect the corresponding `VirtualService`:

   ```
   kubectl get virtualservice -n <namespace> -o yaml
   ```

   Confirm that `spec.hosts` matches the domain you send in the `Host` header.

Workload Unreachable

Symptom

Routing succeeds, but the response is HTTP `503 Service Unavailable`, `504 Gateway Timeout`, or an Envoy `no_healthy_upstream` body.

Cause

- The `APIRule` references a Service that does not exist in the target namespace.
- The Service exists but has no endpoints (no matching Pods, or the Pods are not ready).
- The Service port does not match the workload's container port.
- Istio sidecar injection is disabled on the workload namespace and the workload does not participate in the mesh.

Solution

1. Check the target Service:

   ```
   kubectl get svc <service-name> -n <namespace>
   ```
2. Check the Pods behind the Service:

   ```
   kubectl get pods -n <namespace> -l <selector-from-svc>
   ```

   The target Pods should be in `Running` state.
3. Confirm the port in the `APIRule` matches the Service port:

   ```
   kubectl get apirule <name> -n <namespace> -o jsonpath='{.spec.service.port}'
   kubectl get svc <service-name> -n <namespace> -o
   jsonpath='{.spec.ports[*].port}'
   ```
4. Confirm sidecar injection on the namespace:

   ```
   kubectl get namespace <namespace> -o jsonpath='{.metadata.labels}'
   ```

   Look for `istio-injection=enabled`.

ExternalGateway Readiness Conditions

Run `kubectl describe externalgateway <name> -n <namespace>` and look at the `Ready` condition. Match its `reason` field against this table.

Reason	Meaning
`InternalDomainTooLong`	The assembled internal domain (`{kymaSubdomain}.{gateway-domain}`) is longer than 64 characters — the X.509 CommonName limit.
`CASecretNotFound`	The Secret referenced by `spec.caSecretRef` does not exist in the referenced namespace (defaults to the `ExternalGateway`'s namespace).
`CASecretKeyAmbiguous`	The referenced Secret has more than one data key and none of them is `ca.crt`.
`CASecretInvalid`	The Secret exists, and the key is present, but the value is empty.
`RegionsConfigMapNotFound`	The ConfigMap named by `spec.regionsConfigMap` does not exist in the `ExternalGateway`'s namespace.
`RegionsConfigMapKeyAmbiguous`	The ConfigMap has more than one data key and none of them is `regions.yaml`.
`RegionsConfigMapInvalid`	The ConfigMap value does not parse as YAML, or the parsed structure has zero regions.
`RegionNotFound`	`spec.region` does not match any region entry in the ConfigMap.
`RegionHasNoSubjects`	The region exists, but its `subjects` list is empty.
`ExternalDomainConflict`	Another `ExternalGateway` in the cluster already uses the same `spec.externalDomain`. The message names the conflicting `namespace/name`.
`ReconciliationFailed`	Fallback for any error not classified above.

4.3.1.2.6 Default Configuration and Limitations of the API Gateway Module

APIRule Controller Limitations

APIRule Controller relies on Istio and Ory custom resources (CRs) to provide routing capabilities. In terms of persistence, the controller depends only on APIRules stored in the Kubernetes cluster. In terms of the resource configuration, the following requirements are set on APIGateway Controller:

- CPU Requests: 10 m
- CPU Limits: 100 m
- Memory Requests: 64 Mi
- Memory Limits: 128 Mi

You can create an unlimited number of APIRule CRs.

Ory Resources' Configuration

The configuration of Ory resources depends on the cluster capabilities. If your cluster has fewer than 5 total virtual CPU cores or its total memory capacity is less than 10 gigabytes, the default setup for resources is lighter. If your cluster exceeds both of these thresholds, the higher resource configuration is applied.

The default configuration for larger clusters includes the following settings for the Ory components' resources:

Ory Resources' Configuration

Component	CPU Requests	CPU Limits	Memory Requests	Memory Limits
Oathkeeper	100 m	10000 m	64 Mi	512 Mi
Oathkeeper Maester	10 m	400 m	32 Mi	1 Gi

The default configuration for smaller clusters includes the following settings for the Ory components' resources:

Ory Resources' Configuration

Component	CPU Requests	CPU Limits	Memory Requests	Memory Limits
Oathkeeper	10 m	100 m	64 Mi	128 Mi
Oathkeeper Maester	10 m	100 m	20 Mi	50 Mi

Autoscaling Configuration

The default configuration in terms of autoscaling of Ory components is as follows:

Ory Resources' Configuration

Component	Min Replicas	Max Replicas
Oathkeeper	3	10
Oathkeeper Maester	3	10

Oathkeeper Maester is a separate container running in the same Pod as Oathkeeper. Because of that, the autoscaling configuration of the Oathkeeper and Oathkeeper Master components is similar. The autoscaling configuration is based on CPU utilization, with HorizontalPodAutoscaler set up for 80% average CPU request utilization.

4.3.1.3 SAP BTP Operator Module

Use the SAP BTP Operator module to enable service management and consume SAP BTP services from your Kyma cluster.

What Is SAP BTP Operator?

The SAP BTP Operator module provides service management, which allows you to consume SAP BTP services from your Kyma cluster using Kubernetes-native tools. Within the SAP BTP Operator module, BTP Manager installs the SAP BTP service operator. The SAP BTP service operator enables provisioning and managing service instances and service bindings of SAP BTP services. Consequently, your Kubernetes-native applications can access and use the services from your cluster.

Features

The SAP BTP Operator module provides the following features:

- Credentials and access preconfiguration: Your Secret is readily available on Kyma runtime creation. See Preconfigured Credentials and Access [page 2072].
- Customization of the default credentials and access: You can customize the default `sap-btp-manager` Secret. See Customizing the Default Credentials and Access [page 2075].
- Multitenancy: You can configure multiple subaccounts in a single cluster. See Working with Multiple Subaccounts [page 2087].
- Lifecycle management of service instances and service bindings: You can create, update, and delete service instances and service bindings. See Creating Service Instances and Service Bindings [page 2077], Updating Service Instances [page 2082], and Deleting Service Bindings and Service Instances [page 2083].
- Service binding rotation: You can enhance security by automatically rotating the credentials associated with your service bindings. See Rotating Service Bindings [page 2095].
- Service binding Secret formatting: You can use different attributes in your `ServiceBinding` resource to generate different formats of your Secret resources. See Formatting Service Binding Secrets [page 2097].

Scope

By default, the SAP BTP Operator module consumes SAP BTP services from the subaccount in which you created it. To consume the services from multiple subaccounts in a single Kyma cluster, configure a dedicated Secret for each additional subaccount.

Architecture

The SAP BTP Operator module provides and retrieves the initial credentials that your application needs to use an SAP BTP service.

SAP BTP Operator Architecture

Depending on the number of configured Secrets, SAP BTP Operator can have access to multiple subaccounts within your cluster.

Access Configuration

For more information on multitenancy, see Working with Multiple Subaccounts [page 2087].

SAP BTP, Kyma Runtime

SAP BTP, Kyma runtime runs on a Kubernetes cluster and wraps the SAP BTP Operator module, API server, and one or more applications. The application or multiple applications are the actual consumers of SAP BTP services.

BTP Manager

BTP Manager is an operator based on the Kubebuilder framework. It extends the Kubernetes API by providing the BtpOperator Custom Resource Definition (CRD).

BTP Manager performs the following operations:

- Manages the lifecycle of the SAP BTP service operator, including provisioning of the latest version, updating, and deprovisioning.
- Configures the SAP BTP service operator.

SAP BTP Service Operator

The SAP BTP service operator is an open-source component that allows you to connect SAP BTP services to your cluster and manage them using Kubernetes-native tools. It is responsible for communicating with SAP Service Manager. The operator's resources, service instances and service bindings, come from the `services.cloud.sap.com` API group.

SAP Service Manager

SAP Service Manager is the central registry for service brokers and platforms in SAP BTP, which enables you to do the following:

- Consume platform services in any connected runtime environment.
- Track the creation and management of service instances.
- Share services and service instances between different runtimes.

SAP Service Manager uses Open Service Broker API (OSB API) to communicate with service brokers.

Service Brokers

Service brokers manage the lifecycle of services. SAP Service Manager interacts with service brokers using OSB API to provision and manage service instances and service bindings.

API/Custom Resource Definitions

The `btpoperators.operator.kyma-project.io` Custom Resource Definition (CRD) describes the kind and the format of data that SAP BTP Operator uses to configure resources.

See the documentation related to the BtpOperator custom resource (CR):

- SAP BTP Operator Custom Resource
- Service Instance Custom Resource
- Service Binding Custom Resource

Authorization

To assign access permissions to the SAP BTP Operator module resources, use the following aggregated ClusterRoles :

- `kyma-btp-operator-view` - Grants read-only access to BtpOperator, ServiceInstance, and ServiceBinding CRs and their status.
- `kyma-btp-operator-edit` - Grants full access to BtpOperator, ServiceInstance, and ServiceBinding CRs and read-only access to their status.

Resource Consumption

To learn more about the resources used by the SAP BTP Operator module, see SAP BTP Operator [page 3519].

4.3.1.3.1 Preconfigured Credentials and Access

When you create SAP BTP, Kyma runtime, all necessary resources for consuming SAP BTP services are created, and the basic cluster access is configured.

Credentials

When you create a Kyma instance in the SAP BTP cockpit, the following events happen in your subaccount:

1. An SAP Service Manager service instance with the `service-operator-access` plan is created.
2. An SAP Service Manager service binding with access credentials for the SAP BTP Operator is created.
3. The credentials from the service binding are passed on to the Kyma service instance in the creation process.
4. The `sap-btp-manager` Secret is created and managed in the `kyma-system` namespace.
5. By default, the SAP BTP Operator module is installed together with the following:

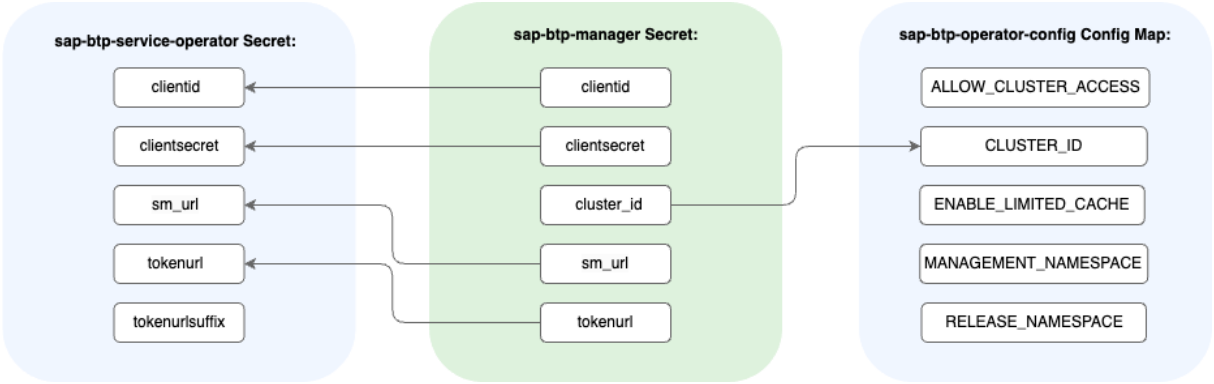
- The `sap-btp-manager` Secret.
- The `sap-btp-service-operator` Secret with the access credentials for the SAP BTP service operator. You can view the credentials in the `kyma-system` namespace.
- The `sap-btp-operator-config` ConfigMap.

The `sap-btp-manager` Secret provides the following credentials:

- `clientid`
- `clientsecret`
- `cluster_id`
- `sm_url`
- `tokenurl`

> ### Note
>
> If you modify or delete the `sap-btp-manager` Secret, it is modified back to its previous settings or regenerated within 24 hours.
>
> To prevent your changes from being reverted, label the Secret with `kyma-project.io/skip-reconciliation: "true"`. For more information, see Customizing the Default Credentials and Access [page 2075].

When the SAP BTP Operator module is added to your cluster by default, the `sap-btp-manager` Secret populates the SAP BTP service operator's resources as shown in the following diagram:



The cluster ID represents and identifies a Kyma service instance created in a particular subaccount. In Kyma dashboard, you can view the cluster ID in the following resources:

- The `sap-btp-manager` Secret.
- The `sap-btp-service-operator` Secret.
- The `sap-btp-operator-config` ConfigMap.

Cluster Access

By default, SAP BTP Operator has cluster-wide permissions. The following parameters manage cluster access:

Cluster Access Parameters

Parameter	Description
`CLUSTER_ID`	Generated when Kyma runtime is created.
`MANAGEMENT_NAMESPACE`	Indicates the namespace for the Secrets with credentials to communicate with the SAP Service Manager. By default, set to `kyma-system`.
`RELEASE_NAMESPACE`	Indicates the namespace for the `sap-btp-service-operator` and `sap-btp-operator-clusterid` Secrets. By default, set to `kyma-system`.
`ALLOW_CLUSTER_ACCESS`	You can use every namespace for your operations. The parameter is always set to true. If you change it to false, your setting is automatically reverted, and SAP BTP Operator cluster-wide permissions remain unchanged.

Default Credentials and Kyma Runtime Deletion

The preconfigured credentials described in the Credentials [page 2072] section may affect the deletion of your Kyma cluster.

If any non-deleted service instances in your Kyma cluster use the credentials from the SAP Service Manager resources created automatically, you can't delete the cluster. In this case, the existing service instances block the cluster's deletion. Before you attempt to delete the cluster from the SAP BTP cockpit, delete your service instances and bindings in Kyma dashboard. See Deleting Service Bindings and Service Instances [page 2083].

However, if all existing service instances in your Kyma cluster use your custom SAP Service Manager credentials, the non-deleted service instances do not block the cluster's deletion. See Customizing the Default Credentials and Access [page 2075].

If you have not deleted service instances and bindings connected to your expired free tier service, you can still find the service binding credentials in the SAP Service Manager instance details in the SAP BTP cockpit. Use them to delete the leftover service instances and bindings.

Related Information

Customizing the Default Credentials and Access [page 2075]

4.3.1.3.2 Customizing the Default Credentials and Access

You can customize the `sap-btp-manager` Secret and manage your own default configuration of the SAP BTP Operator module.

Prerequisites

You have the SAP BTP Operator module added. See Adding and Deleting a Kyma Module [page 3511].

Context

When you create SAP BTP, Kyma runtime, the `sap-btp-manager` Secret is automatically created as the default Secret managing the SAP BTP Operator's resources. Because of Kyma's automatic reconciliation, any changes to the Secret are reverted, and the previous settings are restored within 24 hours. See Preconfigured Credentials and Access [page 2072].

To customize the `sap-btp-manager` Secret and prevent your changes from being reverted, you must stop the Secret's reconciliation. With the customized Secret, you can perform the following actions:

- Manage your own default configuration of SAP BTP Operator
- Migrate the service instances you created outside of the Kyma environment to your Kyma cluster

Procedure

To customize the `sap-btp-manager` Secret, modify the credentials in the following way:

- Label the Secret with `kyma-project.io/skip-reconciliation: 'true'`.
- Provide the following credentials from your SAP Service Manager instance: `clientid`, `clientsecret`, `sm_url`, and `tokenurl`.
- Optionally, provide your `cluster_id`. Otherwise, the default generated one is used.
- Optionally, add the `credentials_namespace` parameter and provide the name of your existing custom namespace for Secrets with credentials to communicate with the SAP Service Manager.

  > **Note**
  >
  > Setting the `credentials_namespace` parameter changes the values of `MANAGEMENT_NAMESPACE` and `RELEASE_NAMESPACE` keys in the `sap-btp-operator-config` ConfigMap. The custom namespace replaces the default managed `kyma-system` namespace for `sap-btp-service-operator` and `sap-btp-operator-clusterid` Secrets. BTP Manager deletes the Secrets from the default `kyma-system` namespace and creates them in the custom namespace.

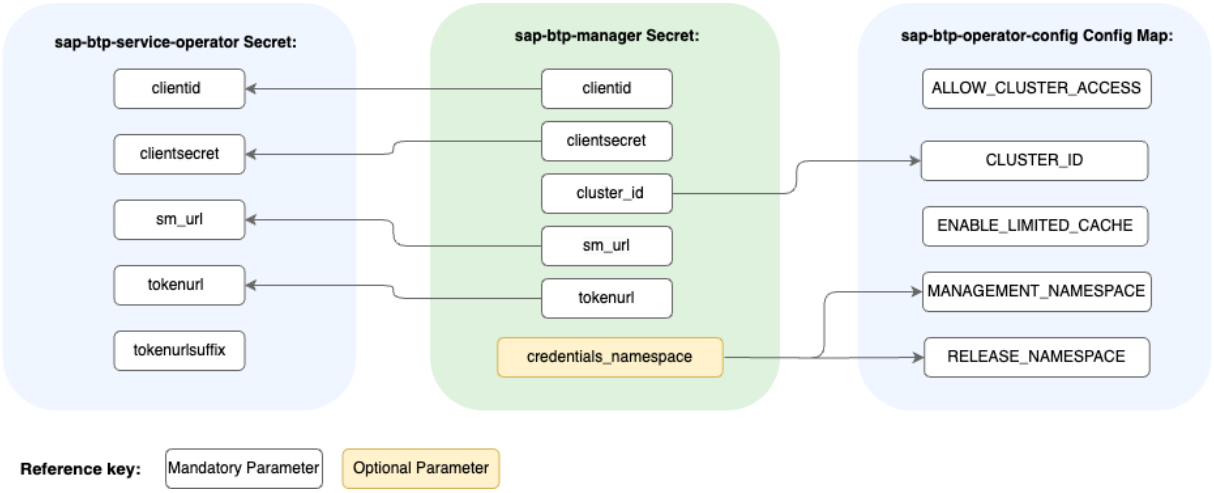
See the following example:

```
apiVersion: v1
```

```
kind: Secret
type: Opaque
metadata:
  name: sap-btp-manager
  namespace: kyma-system
  labels:
    app.kubernetes.io/managed-by: kcp-kyma-environment-broker
    kyma-project.io/skip-reconciliation: "true"
data:
  clientid: {CLIENT_ID}
  clientsecret: {CLIENT_SECRET}
  sm_url: {SM_URL}
  tokenurl: {AUTH_URL}
  cluster_id: {CLUSTER_ID}
  credentials_namespace: {CREDENTIALS_NAMESPACE}
```

Results

Your customized `sap-btp-manager` Secret is now the default Secret of the SAP BTP Operator module. It generates the SAP BTP service operator's resources, as shown in the following diagram:



The reconciliation of the Secret stops and your changes are not reverted.

> ⚠ Caution
>
> If you delete the customized `sap-btp-manager` Secret or remove the `kyma-project.io/skip-reconciliation: 'true'` label from it, the reconciliation starts again, and the preconfigured default `sap-btp-manager` Secret is recreated for your Kyma instance within 24 hours. See Preconfigured Credentials and Access [page 2072].

Related Information

Working with Multiple Subaccounts [page 2087]
Instance-Level Mapping [page 2089]

Namespace-Level Mapping [page 2092]

4.3.1.3.3 Creating Service Instances and Service Bindings

To use an SAP BTP service in your Kyma cluster, create its service instance and service binding using Kyma dashboard or kubectl.

Prerequisites

- You have the SAP BTP Operator module in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- For CLI interactions: kubectl configured to communicate with your Kyma instance. See Access a Kyma Instance Using kubectl [page 2265].
- For an enterprise account, you have added quotas to the services you purchased in your subaccount. Otherwise, only default free-of-charge services are listed in the service marketplace. Quotas are automatically assigned to the resources available in trial accounts.
  For more information, see Configure Entitlements and Quotas for Subaccounts.
- You know the service offering name and service plan[1] [page 2077] name for the SAP BTP service you want to connect to your Kyma cluster.

  → Tip

  [1] To find the service offering and service plan names, go to the SAP BTP cockpit, and then to *Services* > *Service Marketplace*. Click on the service tile and find its *name* and *Plan*.

4.3.1.3.3.1 Creating a Service Instance

To create a service instance, use either Kyma dashboard or kubectl.

Use Kyma Dashboard

Kyma dashboard is a web-based UI providing a graphical overview of your cluster and all its resources. To access Kyma dashboard, use the link available in the **Kyma Environment** section of your subaccount *Overview*.

Procedure

1. In the navigation area, choose *Namespaces*, and go to the namespace you want to work in.
2. Go to *Service Management* > *Service Instances*, and choose *Create*.
3. Provide the required service details[1] [page 2077] in the *Form*. Alternatively, you can switch to the *YAML* tab, and edit or upload your file.
4. Choose *Create*.
   You see the status `PROVISIONED`.

Use kubectl

Procedure

1. To create a `ServiceInstance` custom resource (CR), replace the placeholders[1] [page 2077] and run the following command:

   ```
   kubectl create -f - <<EOF
   apiVersion: services.cloud.sap.com/v1
   kind: ServiceInstance
   metadata:
       name: {SERVICE_INSTANCE_NAME}
       namespace: {NAMESPACE}
   spec:
       serviceOfferingName: {SERVICE_OFFERING_NAME}
       servicePlanName: {SERVICE_PLAN_NAME}
       externalName: {SERVICE_INSTANCE_NAME}
       parameters:
         key1: val1
         key2: val2
   EOF
   ```

2. To check the service's status in your cluster, run:

   ```
   kubectl get serviceinstances.services.cloud.sap.com {SERVICE_INSTANCE_NAME}
   -n {NAMESPACE}
   ```

   You get an output similar to this one:

   ```
   NAME                      OFFERING
   PLAN                     STATUS    AGE
   {SERVICE_INSTANCE_NAME}   {SERVICE_OFFERING_NAME}
   {SERVICE_PLAN_NAME}      Created   44s
   ```

4.3.1.3.3.2 Creating a Service Binding

To create a service binding, use either Kyma dashboard or kubectl.

With a `ServiceBinding` custom resource (CR), your application can get access credentials for communicating with an SAP BTP service. These access credentials are available to applications through a Secret resource generated in your cluster.

Use Kyma Dashboard

Kyma dashboard is a web-based UI providing a graphical overview of your cluster and all its resources. To access Kyma dashboard, use the link available in the **Kyma Environment** section of your subaccount *Overview*.

Procedure

1. In the navigation area, choose *Namespaces*, and go to the namespace you want to work in.
2. Go to *Service Management* > *Service Bindings*, and choose *Create*.
3. Provide the required details, and choose your service instance name from the dropdown list. Alternatively, you can provide the required details by switching from the *Form* to *YAML* tab, and editing or uploading your file.
4. Choose *Create*.

   You see the status `PROVISIONED`.

Use kubectl

Procedure

1. To create a `ServiceBinding` CR, replace the placeholders and run the following command:

```
kubectl create -f - <<EOF
apiVersion: services.cloud.sap.com/v1
kind: ServiceBinding
metadata:
  name: {BINDING_NAME}
spec:
  serviceInstanceName: {SERVICE_INSTANCE_NAME}
  externalName: {EXTERNAL_NAME}
  secretName: {SECRET_NAME}
  parameters:
    key1: val1
    key2: val2
EOF
```

> → Remember
>
> In the `serviceInstanceName` field of the service binding, enter the name of the `ServiceInstance` resource you previously created.

2. To check your service binding status, run:

```
kubectl get servicebindings {BINDING_NAME} -n {NAMESPACE}
```

   You see the status `Created`.

3. Verify the Secret is created with the name specified in the `spec.secretName` field of the `ServiceBinding` CR. The Secret contains access credentials that the applications need to use the service. Run:

```
kubectl get secrets {SECRET_NAME} -n {NAMESPACE}
```

   You see the same Secret name as in the `spec.secretName` field of the `ServiceBinding` CR.

Results

You can use a given service in your Kyma cluster.

4.3.1.3.3.3 Creating a Service Binding for a Service Instance in a Different Namespace

You can create a service binding that references a service instance provisioned in a different namespace.

Procedure

1. Set the `serviceInstanceNamespace` field in the `ServiceBinding` `spec` to the namespace where the service instance resides. Replace the placeholders and run:

```
kubectl create -f - <<EOF
apiVersion: services.cloud.sap.com/v1
kind: ServiceBinding
metadata:
  name: {BINDING_NAME}
  namespace: {BINDING_NAMESPACE}
spec:
  serviceInstanceName: {SERVICE_INSTANCE_NAME}
  serviceInstanceNamespace: {INSTANCE_NAMESPACE}
  secretName: {SECRET_NAME}
EOF
```

2. Check the service binding status:

```
kubectl get servicebindings.services.cloud.sap.com {BINDING_NAME} -n
{BINDING_NAMESPACE}
```

   You see the status `Created`.

Results

The application in the binding's namespace can use the Secret referenced in the `spec.secretName` field to access the service instance provisioned in the namespace specified in the `spec.serviceInstanceNamespace` field.

Next Steps

Restricting Cross-Namespace Bindings

To restrict which namespaces can create cross-namespace bindings, add one of the following annotations to the service instance:

- `services.cloud.sap.com/allowCrossNamespaceBinding: "false"` — to block cross-namespace bindings from all other namespaces
- `services.cloud.sap.com/allowedNamespacesForBinding` — to limit cross-namespace bindings to specific namespaces, add the annotation with a comma-separated list of allowed namespaces (with no spaces), for example: `services.cloud.sap.com/allowedNamespacesForBinding: "ns1,ns2"`

Without these annotations, any namespace can create cross-namespace bindings for this instance.

If the binding's namespace is not permitted by the service instance's annotations, the `Succeeded` condition of the binding is set to false with reason Blocked. The binding automatically retries when the service instance's annotations change.

When a cross-namespace binding is blocked, the binding status shows a message indicating that the service instance couldn't be found or doesn't allow cross-namespace binding. You see the same message when the instance doesn't exist. If you see it and you're sure the instance name and namespace are correct, check whether the service instance has cross-namespace binding restrictions set.

To resolve a blocked binding, either remove the restriction from the service instance or update the `services.cloud.sap.com/allowedNamespacesForBinding` annotation to include the binding's namespace.

4.3.1.3.4 Updating Service Instances

Use Kyma dashboard or kubectl to update service instances.

Prerequisites

- You have the SAP BTP Operator module in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- For CLI interactions: kubectl configured to communicate with your Kyma instance. See Access a Kyma Instance Using kubectl [page 2265].

Context

You are using a service instance in the Kyma environment and want to update the service's plan or other service-specific parameters.

> **Note**
>
> You can only update service instances in Kyma using Kyma dashboard or kubectl. You can't perform the operation using the SAP BTP cockpit.

Procedure

- To update a service instance, use either Kyma dashboard or kubectl.
  Kyma dashboard is a web-based UI providing a graphical overview of your cluster and all its resources. To access Kyma dashboard, use the link in the **Kyma Environment** section of your subaccount *Overview*.
- Use Kyma dashboard.
  a. In the navigation area, choose *Namespaces*, and go to the namespace with the service instance you want to update.
  b. Go to *Service Management* > *Service Instances*, and choose the service instance from the list.
  c. Choose *Edit*.
  d. Update the required service details in *Form* and save your changes. Alternatively, you can switch to the *YAML* tab to edit or upload your file, and save your changes.

  You see the message confirming the service instance update.
- Use kubectl.
  a. To update a ServiceInstance custom resource (CR), replace the placeholders with the following:
     - The name of the service instance you want to update
     - The namespace in which the service instance resides
     - The name of the service offering you are using

- The name of the service plan associated with the service offering
- Updated or new parameters in the `parameters` section

Then, run:

```
kubectl apply -f - <<EOF
apiVersion: services.cloud.sap.com/v1
kind: ServiceInstance
metadata:
    name: {SERVICE_INSTANCE_NAME}
    namespace: {NAMESPACE}
spec:
    serviceOfferingName: {SERVICE_OFFERING_NAME}
    servicePlanName: {SERVICE_PLAN_NAME}
    externalName: {SERVICE_INSTANCE_NAME}
    parameters:
        key1: val1
        key2: val2
EOF
```

b. To check the service's status in your cluster, run:

```
kubectl get serviceinstances.services.cloud.sap.com
{SERVICE_INSTANCE_NAME} -n {NAMESPACE}
```

You get an output similar to this one:

```
NAME                        OFFERING                    PLAN
STATUS    AGE
{SERVICE_INSTANCE_NAME}   {SERVICE_OFFERING_NAME}   {SERVICE_PLAN_NAME}
UPDATED   30m27s
```

4.3.1.3.5 Deleting Service Bindings and Service Instances

Delete service bindings and service instances using Kyma dashboard or kubectl.

Prerequisites

- You have the SAP BTP Operator module in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- For CLI interactions: kubectl configured to communicate with your Kyma instance. See Access a Kyma Instance Using kubectl [page 2265].

Context

You can only delete service instances or service bindings created in Kyma using Kyma dashboard or kubectl. You can't perform these operations using the SAP BTP cockpit. To delete a service instance, first delete its service bindings.

> ⚠ Caution
>
> Once you delete your service instances and service bindings, you cannot revert the operation.

Procedure

- Use either Kyma dashboard or kubectl to delete a service binding or a service instance.
  Kyma dashboard is a web-based UI providing a graphical overview of your cluster and all its resources.
  To access Kyma dashboard, use the link available in the **Kyma Environment** section of your subaccount *Overview*.
- **Use Kyma dashboard**
  a. In the navigation area, choose *Namespaces*, and go to the namespace you want to delete a service binding/instance from.
  b. Go to *Service Management*, and then to *Service Bindings* or *Service Instances*.
  c. Choose the service binding/instance and delete it.
- **Use kubectl**
  - To delete a service binding, run:

    ```
    kubectl delete servicebindings.services.cloud.sap.com {BINDING_NAME}
    ```

  - To delete a service instance, run:

    ```
    kubectl delete serviceinstances.services.cloud.sap.com
    {SERVICE_INSTANCE_NAME}
    ```

4.3.1.3.6 Migrating Service Instances and Service Bindings from a Custom SAP BTP Service Operator

Learn how to migrate service instances and service bindings from your custom SAP BTP service operator to a Kyma cluster.

Prerequisites

- You have a new Kyma cluster deployed without any service instances in it.
- You have the SAP BTP Operator module in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- For CLI interactions: kubectl configured to communicate with your Kyma instance. See Access a Kyma Instance Using kubectl [page 2265].
- For processing JSON data output from kubectl commands: jq .

Context

If you have service instances and service bindings created outside the Kyma environment, and you want to continue using them in the Kyma environment, you must migrate them into your Kyma cluster.

Preparing Migration Data from Your Custom SAP BTP Service Operator

Procedure

1. To connect kubectl to the cluster with your custom SAP BTP service operator, set either the `KUBECONFIG` environment variable or the cluster context with the `kubectl config use-context` command.
2. Find the `sap-btp-service-operator` Secret by running the `kubectl get secrets -A` command.
3. Save the Secret name and its namespace in the `SAP_BTP_OPERATOR_SECRET_NAME` and `SAP_BTP_OPERATOR_SECRET_NAMESPACE` environment variables.
4. Save the SAP BTP service operator credentials in the following environment variables:

   ```
   CLIENT_ID=$(kubectl get secret -n $SAP_BTP_OPERATOR_SECRET_NAMESPACE
   $SAP_BTP_OPERATOR_SECRET_NAME -o jsonpath={.data.clientid})
   CLIENT_SECRET=$(kubectl get secret -n $SAP_BTP_OPERATOR_SECRET_NAMESPACE
   $SAP_BTP_OPERATOR_SECRET_NAME -o jsonpath={.data.clientsecret})
   SM_URL=$(kubectl get secret -n $SAP_BTP_OPERATOR_SECRET_NAMESPACE
   $SAP_BTP_OPERATOR_SECRET_NAME -o jsonpath={.data.sm_url})
   TOKEN_URL=$(kubectl get secret -n $SAP_BTP_OPERATOR_SECRET_NAMESPACE
   $SAP_BTP_OPERATOR_SECRET_NAME -o jsonpath={.data.tokenurl})
   ```

5. To find the `sap-btp-operator-config` ConfigMap in the cluster, run the `kubectl get configmaps -A` command.
6. Save the ConfigMap name and its namespace in the `SAP_BTP_OPERATOR_CONFIGMAP_NAME` and `SAP_BTP_OPERATOR_CONFIGMAP_NAMESPACE` environment variables.
7. Save the cluster ID in the environment variable:

   ```
   CLUSTER_ID=$(kubectl get configmap -n $SAP_BTP_OPERATOR_CONFIGMAP_NAMESPACE
   $SAP_BTP_OPERATOR_CONFIGMAP_NAME -o jsonpath={.data.CLUSTER_ID} | base64)
   ```

8. List all service instances with the `kubectl get serviceinstances -A` command. Take note of the namespaces that must be present in the Kyma cluster.

   When you migrate service instances into the Kyma cluster, you must use these noted namespaces to recreate the necessary structure within the new environment.

9. Save each service instance you want to migrate as a manifest in a JSON file. To do that, replace the placeholders with the actual service instance name and its namespace, and run:

   ```
   kubectl get serviceinstance -n <SERVICE_INSTANCE_NAMESPACE>
   <SERVICE_INSTANCE_NAME> -o json \
   | jq
   'del(.metadata.annotations, .metadata.creationTimestamp, .metadata.finalizers,
    .metadata.generation, .metadata.resourceVersion, .metadata.uid, .status)' \
   > <SERVICE_INSTANCE_NAME>-si.json
   ```

10. List all service bindings with the `kubectl get servicebindings -A` command. Take note of namespaces that must be present in the Kyma cluster.

    When you migrate service bindings into the Kyma cluster, you must use these noted namespaces to recreate the necessary structure within the new environment.

11. Save each service binding you want to migrate as a manifest in a JSON file. To do that, replace the placeholders with the actual service binding name and its namespace, and run:

```
kubectl get servicebinding -n <SERVICE_BINDING_NAMESPACE>
<SERVICE_BINDING_NAME> -o json \
| jq
'del(.metadata.annotations, .metadata.creationTimestamp, .metadata.finalizers,
 .metadata.generation, .metadata.ownerReferences, .metadata.resourceVersion, .
metadata.uid, .status)' \
> <SERVICE_BINDING_NAME>-sb.json
```

Migrating Resources to a Kyma Cluster

Procedure

To migrate your resources to a Kyma cluster, you must first customize the `sap-btp-manager` Secret. See Customizing the Default Credentials and Access [page 2075]. To prevent automatic reversion of your custom changes, add the `kyma-project.io/skip-reconciliation: 'true'` label to the Secret. Then, perform the following steps:

1. To connect kubectl to your Kyma cluster, set either the `KUBECONFIG` environment variable or the cluster context with the `kubectl config use-context` command.
2. To find the `sap-btp-manager` Secret in the Kyma cluster, run the `kubectl get secrets -A` command.
3. Save the Secret name and its namespace in the `BTP_MANAGER_SECRET_NAME` and `BTP_MANAGER_SECRET_NAMESPACE` environment variables.
4. When preparing resources for migration, you noted the namespaces that must be present in the Kyma cluster. To avoid errors and maintain structures based on specific namespace assignments, recreate these namespaces required for the service instances and service bindings you are migrating.
5. To override the SAP BTP service operator credentials and cluster ID, patch the `sap-btp-manager` Secret:

```
kubectl patch secret -n $
{BTP_MANAGER_SECRET_NAMESPACE} ${BTP_MANAGER_SECRET_NAME} -p
"{\"data\":{\"clientid\":\"${CLIENT_ID}\",\"clientsecret\":\"${CLIENT_SECRET}
\",\"sm_url\":\"${SM_URL}\",\"tokenurl\":\"${TOKEN_URL}\",\"cluster_id\":\"$
{CLUSTER_ID}\"}}"
```

6. To create service instances and service bindings from the saved manifests in JSON format, replace the placeholder with the actual service instance or service binding JSON manifest, and use the `kubectl apply -f <JSON_MANIFEST>` command.

Deleting Migrated Resources

Procedure

To limit service instances and service bindings management to the SAP BTP service operator in the Kyma cluster, perform the following steps:

1. To avoid deleting the resources from your Kyma cluster, ensure that you connect kubectl to the cluster with your custom SAP BTP service operator. Set either the `KUBECONFIG` environment variable or the cluster context with the `kubectl config use-context` command.
2. To find the `sap-btp-operator-controller-manager` deployment in the cluster with your custom SAP BTP service operator, run the `kubectl get deployments -A` command.
3. Save the deployment name and its namespace in the `SAP_BTP_OPERATOR_DEPLOYMENT_NAME` and `SAP_BTP_OPERATOR_DEPLOYMENT_NAMESPACE` environment variables.
4. To scale the deployment to 0 replicas, run:

   ```
   kubectl scale deployment -n $SAP_BTP_OPERATOR_DEPLOYMENT_NAMESPACE
   $SAP_BTP_OPERATOR_DEPLOYMENT_NAME --replicas=0
   ```

5. To delete the SAP BTP service operator webhooks, run:

   ```
   kubectl delete mutatingwebhookconfigurations sap-btp-operator-mutating-
   webhook-configuration && kubectl delete validatingwebhookconfigurations sap-
   btp-operator-validating-webhook-configuration
   ```

6. Delete finalizers from each migrated service instance and service binding.
7. Delete migrated service bindings.
8. Delete migrated service instances.

Results

Your service instances and service bindings are now migrated from another environment to the Kyma environment, and you manage them from the SAP BTP service operator in your Kyma cluster.

4.3.1.3.7 Working with Multiple Subaccounts

With the SAP BTP Operator module, you can create configurations for several subaccounts in a single Kyma cluster.

Context

By default, a Kyma cluster is associated with one subaccount. Consequently, any service instance created within any namespace is provisioned in the associated subaccount. See Preconfigured Credentials and Access

[page 2072]. However, with SAP BTP Operator, you can create configurations in a single Kyma cluster that are applied to several subaccounts.

To apply the multitenancy feature, choose the method that suits your needs and application architecture:

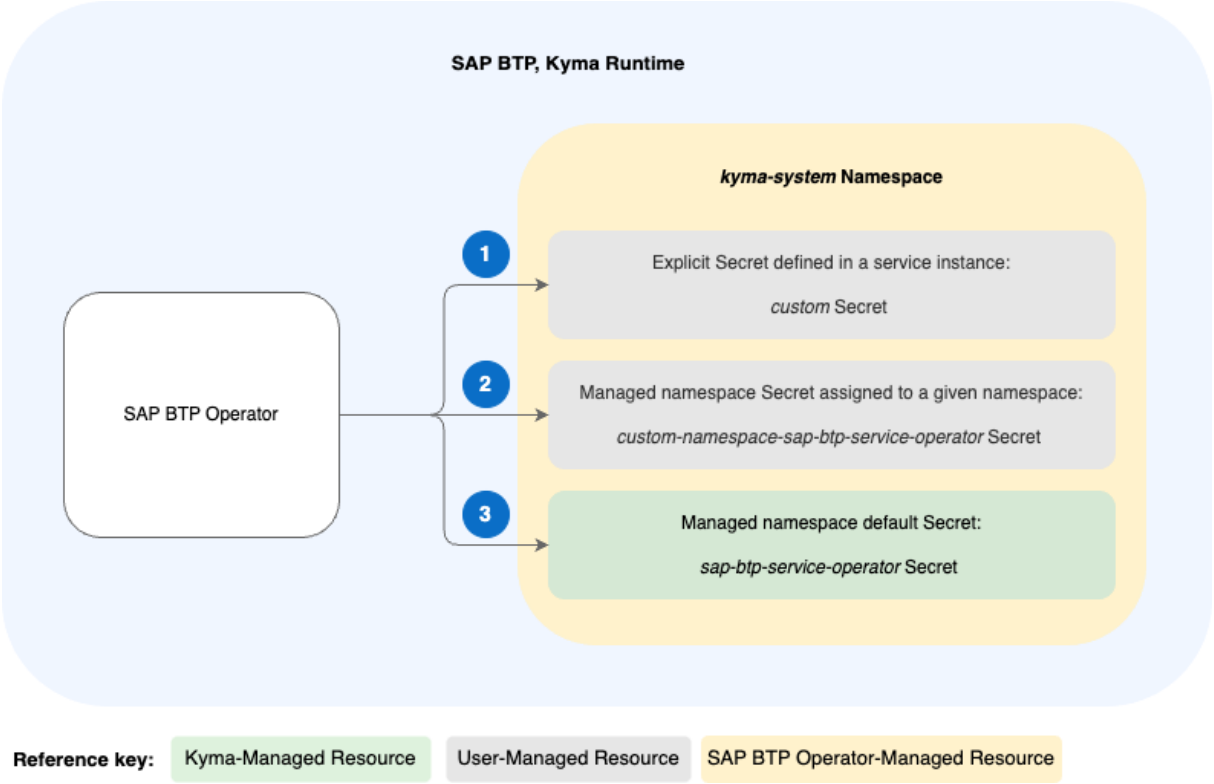
- Namespace-level mapping [page 2092]: Connect namespaces to separate subaccounts by configuring dedicated credentials for each namespace.
- Instance-level mapping [page 2089]: Define a specific subaccount for each service instance, regardless of the namespace context.

Regardless of the method, you must create Secrets in the managed namespace. By default, it is the `kyma-system` namespace. If you want to customize the managed namespace, follow the instructions in Customizing the Default Credentials and Access [page 2075].

Secrets Precedence

SAP BTP Operator searches for the credentials in the following order:

1. Explicit Secret defined in a service instance
2. Managed namespace Secret assigned for a given namespace
3. Managed namespace default Secret



Procedure

- To connect a namespace to a specific subaccount, see Namespace-Level Mapping [page 2092].
- To deploy service instances belonging to different subaccounts within the same namespace, see Instance-Level Mapping [page 2089].

4.3.1.3.7.1 Instance-Level Mapping

You can map a Kubernetes service instance to an SAP Service Manager instance in a given subaccount. The Service Manager instance is then used to provision that service instance.

Prerequisites

- A subaccount in the SAP BTP cockpit.
- You have the SAP BTP Operator module in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- For CLI interactions: kubectl configured to communicate with your Kyma instance. See Access a Kyma Instance Using kubectl [page 2265].

Context

To have multiple service instances from different subaccounts associated with one namespace, you must store access credentials for each subaccount in a custom Secret in the managed namespace, which is by default the `kyma-system` namespace. To create a service instance with a custom Secret, you must use the `btpAccessCredentialsSecret` field in the `spec` of the service instance. In it, you pass the Secret from the `kyma-system` namespace to create your service instance. You can use different Secrets for different service instances.

Creating Your Custom Secret

Procedure

1. In the SAP BTP cockpit, create an SAP Service Manager service instance with the `service-operator-access` plan. See Creating Instances in Other Environments.
2. Create a service binding to the SAP Service Manager service instance you have created. See Creating Service Bindings in Other Environments.
3. Get the access credentials of the SAP Service Manager instance from its service binding. Copy them from the SAP BTP cockpit as a JSON file.
4. Create the `creds.json` file in your working directory and save the credentials there.
5. In the same working directory, generate the Secret by calling the `create-secret-file.sh` script with the `operator` option as the first parameter and `your-secret-name` as the second parameter:

```
curl https://raw.githubusercontent.com/kyma-project/btp-manager/main/hack/
create-secret-file.sh | bash -s operator {YOUR_SECRET_NAME}
```

The expected result is the file `btp-access-credentials-secret.yaml` created in your working directory:

```
apiVersion: v1
kind: Secret
type: Opaque
metadata:
  name: {YOUR_SECRET_NAME}
  namespace: kyma-system
stringData:
  clientid: {CLIENT_ID}
  clientsecret: {CLIENT_SECRET}
  sm_url: {SM_URL}
  tokenurl: {AUTH_URL}
  tokenurlsuffix: "/oauth/token"
```

6. To create the Secret, run:

```
kubectl create -f ./btp-access-credentials-secret.yaml
```

7. To verify if the Secret has been successfully created, run:

```
kubectl get secret -n kyma-system {YOUR_SECRET_NAME}
```

You see the status `Created`.

> ### Note
>
> You can also view the Secret in Kyma dashboard. In the `kyma-system` namespace, go to *Configuration* > *Secrets*, and check the list of Secrets.

Creating a Service Instance with the Custom Secret

Context

To create the service instance, use either Kyma dashboard or kubectl.

Kyma dashboard is a web-based UI providing a graphical overview of your cluster and all its resources. To access Kyma dashboard, use the link available in the **Kyma Environment** section of your subaccount *Overview*.

Procedure

- Use Kyma dashboard.
  a. In the *Namespaces* view, go to the namespace you want to work in.
  b. Go to *Service Management* > *Service Instances*.
  c. In the *BTP Access Credentials Secret* field, add the name of the custom Secret you have created.
  d. Provide other required service details and create a service instance.

> ⚠ Caution
>
> Once you set a Secret name in the service instance, you cannot change it in the future.

You see the status `PROVISIONED`.

- Use kubectl.
  a. Create your service instance with the following:
     - The `btpAccessCredentialsSecret` field in the `spec` pointing to the custom Secret you have created.
     - Other parameters as needed

     > ⚠ Caution
     >
     > Once you set a Secret name in the service instance, you cannot change it in the future.

     See an example of a `ServiceInstance` custom resource (CR):

     ```
     kubectl create -f - <<EOF
     apiVersion: services.cloud.sap.com/v1
     kind: ServiceInstance
     metadata:
       name: {SERVICE_INSTANCE_NAME}
       namespace: {NAMESPACE_NAME}
     spec:
       serviceOfferingName: {SERVICE_OFFERING_NAME}
       servicePlanName: {SERVICE_PLAN_NAME}
       btpAccessCredentialsSecret: {YOUR_SECRET_NAME}
     EOF
     ```

  b. To verify that your service instance has been created successfully, run:

     ```
     kubectl get serviceinstances.services.cloud.sap.com
     {SERVICE_INSTANCE_NAME} -n {NAMESPACE}
     ```

     You see the status `Created` and the message that your service instance has been created successfully. You also see your Secret name in the `btpAccessCredentialsSecret` field of the `spec`.

  c. To verify that you've correctly added the access credentials of the SAP Service Manager instance in your service instance, go to the CR `status` section, and make sure the subaccount ID to which the instance belongs is provided in the `subaccountID` field. The field must not be empty.

Related Information

Working with Multiple Subaccounts [page 2087]
Namespace-Level Mapping [page 2092]
Customizing the Default Credentials and Access [page 2075]

4.3.1.3.7.2 Namespace-Level Mapping

You can map a Kubernetes namespace to an SAP Service Manager instance in a given subaccount. The Service Manager instance is then used to provision all service instances in that namespace.

Prerequisites

- A subaccount in the SAP BTP cockpit.
- You have the SAP BTP Operator module in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- For CLI interactions: kubectl configured to communicate with your Kyma instance. See Access a Kyma Instance Using kubectl [page 2265].

Context

To connect a namespace to a specific subaccount, maintain the access credentials to the subaccount in a Secret dedicated to a specific namespace. Create the `{NAMESPACE-NAME}-sap-btp-service-operator` Secret in the managed namespace, which is by default the `kyma-system` namespace.

Creating a Namespace-Based Secret

Procedure

1. In the SAP BTP cockpit, create a new SAP Service Manager service instance with the `service-operator-access` plan. See Creating Instances in Other Environments.
2. Create a service binding to the SAP Service Manager service instance you have created. See Creating Service Bindings in Other Environments.
3. Get the access credentials of the SAP Service Manager instance from its service binding. Copy them from the SAP BTP cockpit as a JSON file.
4. Create the `creds.json` file in your working directory and save the credentials there.
5. In the same working directory, call the `create-secret-file.sh` script with the `operator` option as the first parameter and `{namespace-name}-sap-btp-service-operator` Secret as the second parameter:

```
curl https://raw.githubusercontent.com/kyma-project/btp-manager/main/hack/
create-secret-file.sh | bash -s operator {NAMESPACE_NAME}-sap-btp-service-
operator
```

The expected result is the `btp-access-credentials-secret.yaml` file created in your working directory:

```
apiVersion: v1
kind: Secret
type: Opaque
metadata:
  name: {NAMESPACE_NAME}-sap-btp-service-operator
  namespace: kyma-system
stringData:
  clientid: {CLIENT_ID}
  clientsecret: {CLIENT_SECRET}
  sm_url: {SM_URL}
  tokenurl: {AUTH_URL}
  tokenurlsuffix: "/oauth/token"
```

6. To create the Secret, run:

```
kubectl create -f ./btp-access-credentials-secret.yaml
```

You see the status `Created`.

Creating a Service Instance with a Namespace-Based Secret

Procedure

1. To create a service instance with a namespace-based Secret, follow the instructions in Creating Service Instances and Service Bindings [page 2077].
2. To verify that you've correctly added the access credentials of the SAP Service Manager instance in your service instance, go to the custom resource (CR) `status` section, and make sure the subaccount ID to which the instance belongs is provided in the `subaccountID` field. The field must not be empty.

Related Information

Working with Multiple Subaccounts [page 2087]
Instance-Level Mapping [page 2089]
Customizing the Default Credentials and Access [page 2075]

4.3.1.3.8 Passing Parameters

You can set input parameters for your resources.

Procedure

To pass additional parameters stored outside your resource `spec`, create a Kubernetes Secret manually in the same namespace as your service instance or service binding, and reference it using the `parametersFrom` field. To set input parameters, go to the `spec` of the service instance or service binding resource, and use one or both of the following fields:

- `parameters`: Specifies a set of properties sent to the service broker. The specified data is passed to the service broker without any modifications aside from converting it to JSON for transmission to the broker if the `spec` field is specified as YAML. All valid YAML or JSON constructs are supported.
- `parametersFrom`: Specifies which Secret, together with the key in it, to include in the set of parameters sent to the service broker. The key contains a `string` that represents a JSON file. The `parametersFrom` field is a list that supports multiple sources referenced per `spec`. The service instance resource can specify multiple related Secrets.

For service instance resources, you can also use the following parameter:

- `watchParametersFromChanges`: Use this field together with `parametersFrom`. This field is only relevant for service instance resources because you cannot update service binding resources. Set it to `true` to trigger an automatic update of the service instance resource with the changes to the Secret values listed in `parametersFrom`. By default, it is set to `false`.

> ⚠ Caution
>
> When using `watchParametersFromChanges`, the referenced Secret must have the label `services.cloud.sap.com/managed-by-sap-btp-operator: "true"`. Without the label, the SAP BTP service operator does not watch the Secret, and changes to it are not detected.

If you specify multiple sources in the `parameters` and `parametersFrom` fields, the final payload merges all of them at the top level.

To avoid errors, do not use the same top-level parameter name in multiple sources in the `parameters` and `parametersFrom` fields. Otherwise, the specification is invalid, and further processing of the service instance or service binding resources stops with the status `Error`.

Examples

The following example shows a service instance `spec` that uses `parameters`, `parametersFrom`, and `watchParametersFromChanges`:

```
spec:
  ...
  parameters:
    name: value
```

```
  parametersFrom:
    - secretKeyRef:
        name: {SECRET_NAME}
        key: secret-parameter
  watchParametersFromChanges: true
```

You must create the Secret referenced by `parametersFrom` in the same namespace as the service instance:

```
apiVersion: v1
kind: Secret
metadata:
  name: {SECRET_NAME}
  namespace: {NAMESPACE}
  labels:
    services.cloud.sap.com/managed-by-sap-btp-operator: "true"
type: Opaque
stringData:
  secret-parameter:
    '{
      "password": "password",
      "key2": "value2",
      "key3": "value3"
    }'
```

The values from `parameters` and `parametersFrom` are merged into a single JSON payload sent to the service broker:

```
{
  "name": "value",
  "password": "password",
  "key2": "value2",
  "key3": "value3"
}
```

4.3.1.3.9 Rotating Service Bindings

Enhance security by automatically rotating the credentials associated with your service bindings. This process involves generating a new service binding while keeping the old credentials active for a specified period to ensure a smooth transition.

Enabling Automatic Rotation

To enable automatic service binding rotation, use the `credentialsRotationPolicy` field within the `spec` section of the `ServiceBinding` resource. You can configure the following parameters:

Automatic Rotation Parameters

Parameter	Type	Description	Valid Values
`enabled`	bool	Turns automatic rotation on or off.	true or false

Parameter	Type	Description	Valid Values
`rotationFrequency`	string	Defines the desired interval between binding rotations.	m (minutes) or h (hours)
`rotatedBindingTTL`	string	Determines how long to keep the old `ServiceBinding` resource after rotation and before deletion. The actual TTL may be slightly longer.	m (minutes) or h (hours)

> ### Note
>
> The `credentialsRotationPolicy` does not manage the validity or expiration of the credentials themselves. This is determined by the service you are using.

The `credentialsRotationPolicy` is evaluated periodically during a control loop on every service binding update or during a complete reconciliation process. This means the actual rotation occurs in the closest upcoming reconciliation loop.

Enabling Immediate Rotation

To trigger an immediate rotation regardless of the configured rotation frequency, add the `services.cloud.sap.com/forceRotate: "true"` annotation to the `ServiceBinding` resource. The immediate rotation only works if automatic rotation is already enabled.

The following example shows the configuration of a `ServiceBinding` resource for rotating credentials every 25 days (600 hours) and keeping the old `ServiceBinding` resource for 2 days (48 hours) before deleting it:

```
apiVersion: services.cloud.sap.com/v1
kind: ServiceBinding
metadata:
  name: {BINDING_NAME}
spec:
  serviceInstanceName: {SERVICE_INSTANCE_NAME}
  credentialsRotationPolicy:
    enabled: true
    rotatedBindingTTL: 48h
    rotationFrequency: 600h
```

Result

Rotating the service binding has the following results:

- The Secret is updated with the latest credentials.
- The old credentials are kept in a newly-created Secret named `{ORIGINAL_SECRET_NAME}-{GUID}`. This temporary Secret is marked with the services.cloud.sap.com/stale label and kept until the configured deletion time (TTL).
  To see the timestamp of the last service binding rotation, go to the `status.lastCredentialsRotationTime` field.

Limitations

You cannot enable automatic credential rotation for a backup service binding (named: `{ORIGINAL_BINDING_NAME}-{GUID}`) marked with the services.cloud.sap.com/stale label. This backup service binding is created during the credentials rotation process to facilitate the process.

4.3.1.3.10 Formatting Service Binding Secrets

Use different attributes in your `ServiceBinding` resource to generate different formats of your Secret resources.

Context

Secret resources share a common set of basic parameters that can be divided into two categories:

- Credentials returned from the service broker that allow your application to access and consume an SAP BTP service.
- Attributes of the associated service instance: The details of the service instance itself.

However, the Secret resources can come in various formats:

- Default key-value pairs
- A JSON object
- One JSON object with credentials and service information
- Custom formats

Key-Value Pairs

If you do not use any of the attributes, the generated Secret is by default in the key-value pair format, as in the following examples:

- Service binding

```
apiVersion: services.cloud.sap.com/v1
kind: ServiceBinding
metadata:
  name: {BINDING_NAME}
spec:
  serviceInstanceName: {SERVICE_INSTANCE_NAME}
```

- Secret

```
apiVersion: v1
metadata:
  name: {BINDING_NAME}
kind: Secret
data:
```

```
  url: {URL}
  client_id: {CLIENT_ID}
  client_secret: {CLIENT_SECRET}
  instance_guid: {SERVICE_INSTANCE_ID}
  instance_name: {SERVICE_INSTANCE_NAME}
  plan: {SERVICE_PLAN_NAME}
  type: {SERVICE_OFFERING_NAME}
```

Credentials as a JSON Object

To show credentials that the service broker returns within the Secret resource as a JSON object, use the `secretKey` attribute in the service binding `spec`. The value of the `secretKey` is the name of the key that stores the credentials. The credentials are represented in both formats: YAML or JSON.

See the following examples:

- Service binding

```
apiVersion: services.cloud.sap.com/v1
kind: ServiceBinding
metadata:
  name: {BINDING_NAME}
spec:
  serviceInstanceName: {SERVICE_INSTANCE_NAME}
  secretKey: myCredentials
```

- Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: {BINDING_NAME}
data:
    myCredentials:
      url: {URL}
      client_id: {CLIENT_ID},
      client_secret: {CLIENT_SECRET}
    instance_guid: {SERVICE_INSTANCE_ID}
    instance_name: {SERVICE_INSTANCE_NAME}
    plan: {SERVICE_PLAN_NAME}
    type: {SERVICE_OFFERING_NAME}
```

Credentials and Service Information as One JSON Object

To show both credentials returned from the service broker and additional service instance attributes as a JSON object, use the `secretRootKey` attribute in the service binding `spec`.

The `secretRootKey` value is the name of the key that stores both credentials and service instance info. The credentials are represented in both formats: YAML or JSON.

See the following examples:

- Service binding

```
apiVersion: services.cloud.sap.com/v1
```

```
kind: ServiceBinding
metadata:
  name: {BINDING_NAME}
spec:
  serviceInstanceName: {SERVICE_INSTANCE_NAME}
  secretRootKey: myCredentialsAndInstance
```

- Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: {BINDING_NAME}
data:
    myCredentialsAndInstance:
        url: {URL}
        client_id: {CLIENT_ID}
        client_secret: {CLIENT_SECRET}
        instance_guid: {SERVICE_INSTANCE_ID}
        instance_name: {SERVICE_INSTANCE_NAME}
        plan: {SERVICE_PLAN_NAME}
        type: {SERVICE_OFFERING_NAME}
```

Custom Formats

For additional flexibility, model the Secret resources according to your needs. To generate a custom-formatted Secret, use the `secretTemplate` attribute in the service binding `spec`. This attribute expects a Go template as its value. For more information, see Go Templates .

Ensure the template is in the YAML format and has the structure of a Kubernetes Secret.

In the provided Secret, you can customize the `metadata` and `data` sections with the following options:

- `metadata`: labels and annotations
- `data`: customize or utilize one of the available formatting options listed in the Context [page 2097] section

> **Note**
>
> If you customize `data`, it takes precedence over the provided predefined formats.

The provided templates are executed on a map with the following available attributes:

Custom Formatting Parameters

Reference	Description
`instance.instance_guid`	The service instance ID.
`instance.instance_name`	The service instance name.
`instance.plan`	The name of the service plan used to create this service instance.
`instance.type`	The name of the associated service offering.
`credentials.attributes(var)`	The content of the credentials depends on a service. For more details, refer to the documentation of the service you're using.
`instance.label`	The service offering name.

The following examples demonstrate the service binding and generated Secret resources:

- In a service binding with customized `metadata` and `data` sections, you specify both metadata and data in the `secretTemplate`:
  - Service binding

```
apiVersion: services.cloud.sap.com/v1
kind: ServiceBinding
metadata:
  name: {BINDING_NAME}
spec:
  serviceInstanceName: {SERVICE_INSTANCE_NAME}
  secretTemplate: |
    apiVersion: v1
    kind: Secret
    metadata:
      labels:
        service_plan: {{ .instance.plan }}
      annotations:
        instance: {{ .instance.instance_name }}
    data:
      USERNAME: {{ .credentials.client_id }}
      PASSWORD: {{ .credentials.client_secret }}
```

  - Secret

```
apiVersion: v1
kind: Secret
metadata:
  labels:
    service_plan: {SERVICE_PLAN_NAME}
  annotations:
    instance: {SERVICE_INSTANCE_NAME}
data:
  USERNAME: {CLIENT_ID}
  PASSWORD: {CLIENT_SECRET}
```

- In a binding with a customized `metadata` section and applied preexisting formatting option for `data` with credentials as a JSON object, you omit `data` from the `secretTemplate` and use the `secretKey` to format your `data` instead:
  - Service binding

```
apiVersion: services.cloud.sap.com/v1
kind: ServiceBinding
metadata:
  name: {BINDING_NAME}
spec:
  serviceInstanceName: {SERVICE_INSTANCE_NAME}
  secretKey: myCredentials
  secretTemplate: |
    apiVersion: v1
    kind: Secret
    metadata:
      labels:
        service_plan: {{ .instance.plan }}
      annotations:
        instance: {{ .instance.instance_name }}
```

  - Secret

```
apiVersion: v1
kind: Secret
metadata:
  labels:
```

```
    service_plan: {SERVICE_PLAN_NAME}
  annotations:
    instance: {SERVICE_INSTANCE_NAME}
data:
  myCredentials:
    url: {URL}
    client_id: {CLIENT_ID}
    client_secret: {CLIENT_SECRET}
  instance_guid: {SERVICE_INSTANCE_ID}
  instance_name: {SERVICE_INSTANCE_NAME}
  plan: {SERVICE_PLAN_NAME}
  type: {SERVICE_OFFERING_NAME}
```

4.3.1.3.11 Troubleshooting for the SAP BTP Operator Module

Troubleshoot problems related to the SAP BTP Operator module.

You Cannot Delete Leftover Service Instances and Bindings [page 2101]

Resource CR Missing from the Cluster [page 2102]

Service Instance Not Updated When Watched Parameters Secret Is Modified [page 2104]

Creating Service Instances or Bindings Fails After Deleting the Service Manager Binding [page 2105]

4.3.1.3.11.1 You Cannot Delete Leftover Service Instances and Bindings

Symptom

After your Kyma cluster has become unavailable, some orphaned service instances and service bindings are still present in the SAP Business Technology Platform (BTP).

Cause

You cannot access your Kyma cluster where the service instances and bindings were created. Without access to the cluster, you cannot use standard deletion methods.

Solution

> ⚠ Caution
>
> This cleanup procedure deletes all service instances and bindings associated with the inaccessible cluster.

1. To access the SAP Service Manager API, perform the following actions:
    1. In the SAP BTP cockpit, create an SAP Service Manager service instance with the `subaccount-admin` plan and its service binding. The `service-operator-access` plan does not have the necessary access level for performing the cleanup operation. See Creating Instances in Other Environments and Creating Service Bindings in Other Environments.
    2. To get an access token, run the following command. Replace the placeholders with the `url`, `clientid`, and `clientsecret` from the service binding you have created.

        ```
        curl '{url}/oauth/token' -X POST \
           -H 'Accept: application/json' \
           -d
        'grant_type=client_credentials&client_id={clientid}&client_secret={clientse
        cret}'
        ```

2. To prepare the DELETE request, identify the following parameters:
    - `platformID` - the ID of the Service Manager instance with the `service-operator-access` plan.
    - `clusterID` - the ID of your cluster. To get `it`, in the SAP BTP cockpit, go to *Services* > *Instances and Subscriptions*. In the *Instances* section, copy the cluster ID from the `Scope` column in the row that provides information about the service instance(s) you want to delete.
3. Send the following request:

    ```
    curl -X DELETE '{sm_url}v1/platforms/{platformID}/clusters/{clusterID}' \
       -H 'Authorization: Bearer {ACCESS_TOKEN}'
    ```

4. Monitor the response. You can get one of the following options:
    - `202 Accepted` - the request is accepted for processing.
    - `404 Resource Not Found` - platform or cluster not found.
    - `429 Too Many Requests` - the rate limit exceeded.
5. For the operation status, review the following headers:
    - `Location` - the path provided to monitor the status of the operation. For more information about operations, see Service Manager operation API.
    - `Retry_After` - indicates the time in seconds after which you can retry the request after hitting rate limits.

4.3.1.3.11.2 Resource CR Missing from the Cluster

Symptom

Your service instance or binding is visible in SAP Business Technology Platform (BTP), but its corresponding custom resource (CR) is missing from your Kyma cluster.

Cause

The CR representing an SAP BTP service instance or service binding is absent from the Kyma cluster. This happens if the CR was accidentally deleted or if there are issues preventing its synchronization or visibility in

the cluster. While the resource still exists, the connection to your Kyma cluster is broken due to the absence of the CR.

Solution

To address this issue, you can manually recreate the service instance CR or service binding CR using details from SAP BTP to restore the connection with the existing SAP BTP resource. For successful reconnection, ensure that the new CR matches the name, resides in the same namespace, and is linked to the same cluster ID as the original CR. Since the SAP BTP resource maintains its configuration, matching these attributes allows the new CR to reconnect to the existing SAP BTP resource without provisioning a new one.

To restore the CR, follow these steps:

1. Retrieve the following CR details from the service instance or binding:
   - The CR name
   - The name of the namespace where the CR should reside
2. Create your YAML manifest for the CR, including the exact name and namespace you retrieved from the SAP BTP service instance or binding.
3. To recreate the CR in your Kyma cluster, run:

   ```
   kubectl apply -f {YAML_MANIFEST}
   ```

4. To verify recreation of the CR in your Kyma cluster, replace the placeholders and run:

   ```
   kubectl get serviceinstances.services.cloud.sap.com {SERVICE_INSTANCE_NAME}
   -n {NAMESPACE}
   ```

   Or

   ```
   kubectl get servicebindings.services.cloud.sap.com {BINDING_NAME} -n
   {NAMESPACE}
   ```

5. Review the service instance or binding in SAP BTP to confirm it recognizes the re-established connection with the CR in your Kyma cluster.

If the connection is not re-established, ensure that your Kyma cluster's ID matches the cluster ID associated with the SAP BTP service instance or binding. To view the cluster ID in the SAP BTP cockpit, go to *Services* > *Instances and Subscriptions*. It's under *Instances*, in the `Scope` column, in the row that provides information about the service instance. Alternatively, you can view the cluster ID using the SAP BTP command line interface (btp CLI). If you discover mismatched IDs, reconfigure your Kyma cluster with the correct cluster ID.

4.3.1.3.11.3 Service Instance Not Updated When Watched Parameters Secret Is Modified

Symptom

You have a service instance configured with `parametersFrom` referencing your parameters Secret (see Passing Parameters [page 2094]) and `watchParametersFromChanges` set to true, but updating the Secret does not trigger an update of the service instance.

Cause

The SAP BTP service operator uses limited cache mode, which is enabled by default in the SAP BTP Operator module. In this mode, the operator only watches Secrets that carry the label `services.cloud.sap.com/managed-by-sap-btp-operator: "true"`. A Secret without this label is not tracked, so changes to it go undetected — no update is triggered and no error is reported.

Solution

Add the label `services.cloud.sap.com/managed-by-sap-btp-operator: "true"` to the parameters Secret. Choose one of the following methods:

- Run the following kubectl command:

```
kubectl label secret <SECRET_NAME> -n <NAMESPACE> services.cloud.sap.com/
managed-by-sap-btp-operator=true
```

- Include the label in your Secret manifest:

```
apiVersion: v1
kind: Secret
metadata:
  name: <SECRET_NAME>
  namespace: <NAMESPACE>
  labels:
    services.cloud.sap.com/managed-by-sap-btp-operator: "true"
type: Opaque
stringData:
  secret-parameter: |
    {
      "key": "value"
    }
```

When the label is present, the SAP BTP service operator detects it immediately and reconciles the service instance on the next Secret change. You don't need to wait or restart the operator.

4.3.1.3.11.4 Creating Service Instances or Bindings Fails After Deleting the Service Manager Binding

Symptom

You deleted the SAP Service Manager service binding in the SAP BTP cockpit and now cannot create new SAP BTP service instances or bindings. The SAP BTP Operator module may be failing.

Cause

When your Kyma runtime was provisioned, a Service Manager service instance with the `service-operator-access` plan and a corresponding service binding were created automatically. The credentials from that binding are used to populate the `sap-btp-manager` Secret in your cluster, which the SAP BTP Operator module relies on to function.

When you delete the binding, the Kyma infrastructure still holds the old credentials and keeps refreshing the `sap-btp-manager` Secret with them. If you recreate the binding, the new binding has different credentials, but the Secret is still not updated. You can't create new service instances because the SAP BTP Operator module can't authenticate. The support team must manually update the credentials on your behalf.

> ⓘ Note
>
> Alternatively, if the `sap-btp-manager` Secret itself was deleted from the cluster, Kyma's automatic reconciliation restores it within 24 hours — no support ticket is needed. To restore the Secret immediately, use the credentials from the existing binding and follow Customizing the Default Credentials and Access [page 2075].

Each Kyma runtime has a dedicated Service Manager instance in a one-to-one relationship. If you have multiple Kyma runtimes, you have multiple Service Manager instances, each with its own binding. The Service Manager instance name in the SAP BTP cockpit matches the instance ID of the corresponding Kyma runtime. You need this mapping to identify the correct instance when raising a support ticket.

Solution

1. In the SAP BTP cockpit, open your Kyma runtime and note its instance ID.
2. In the list of Service Manager service instances, find the one whose name matches your Kyma instance ID. This is the dedicated Service Manager instance for your Kyma runtime.
3. In that Service Manager instance, check whether a binding exists.
   - If no binding exists, recreate it, then proceed to the next step.
   - If a binding already exists (whether original or recreated), proceed to the next step.
4. Check the binding's creation timestamp and compare it with the Service Manager instance's creation timestamp:
   - If the binding was created later than the Service Manager instance, it proves the binding was deleted and then recreated. This confirms you're in the scenario described by this guide.

- If the timestamps match, the binding has not been recreated and this guide does not apply. Instead, the issue may be caused by deletion of the `sap-btp-manager` Secret from the cluster. See Cause [page 2105].

5. Download the binding credentials as a JSON file.
6. Create a support ticket and attach the credentials JSON file. Include the following information in the ticket:
   - Your subaccount ID
   - Your Kyma instance ID
   - The Service Manager instance name (which equals your Kyma instance ID)

   For more information on creating a ticket, see Getting Support.
7. Wait for the support team to confirm that the `sap-btp-manager` Secret in your cluster is updated with the new credentials.
8. To verify the fix, create a service instance. If it's created without errors, the issue is resolved.

4.3.1.4 Cloud Manager Module

Use the Cloud Manager module to manage infrastructure providers' resources from SAP BTP, Kyma runtime.

What Is Cloud Manager?

The Cloud Manager module manages access to cloud providers' chosen resources and products. Once you add Cloud Manager to your Kyma cluster, the module securely brings the offered resources. Cloud Manager is strictly coupled with the cloud provider where your Kyma cluster is deployed.

> → Remember
>
> Using certain features of the Cloud Manager module introduces additional costs. For more information, see Calculation with the Cloud Manager Module [page 106].

Scope

The Cloud Manager module is available on Kyma clusters running on Amazon Web Services, Google Cloud, and Microsoft Azure.

> ⚠ Restriction
>
> Cloud Manager supports the NFS feature of Amazon Web Services and Google Cloud only.

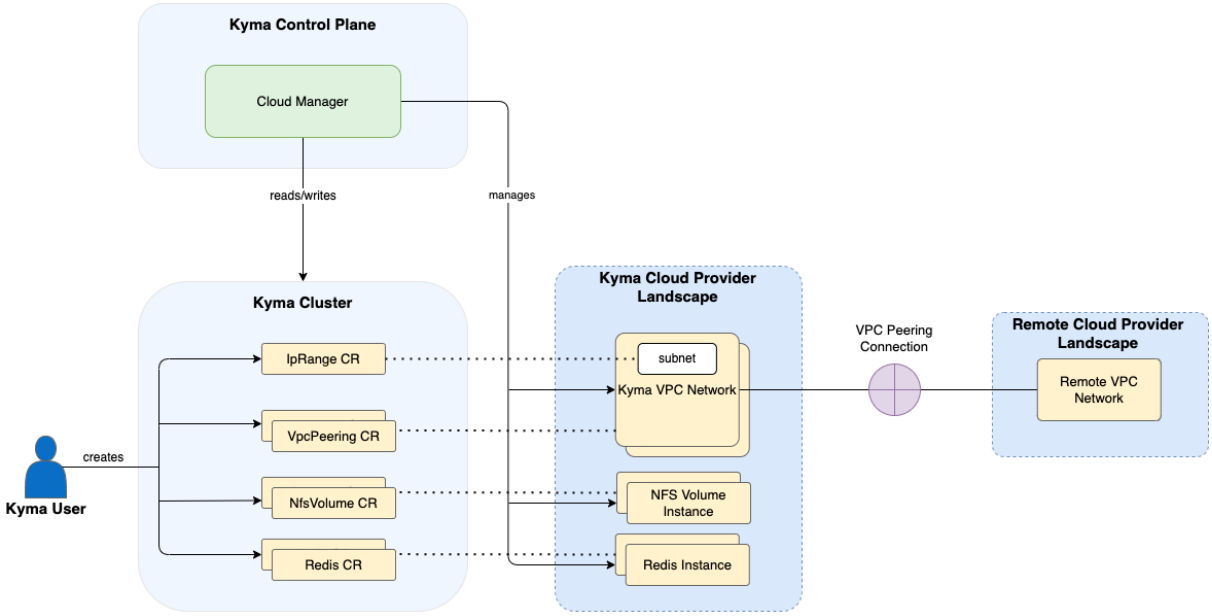
Features

The Cloud Manager module provides the following features tailored for each of the cloud providers.

- NFS [page 2108]: Network File System (NFS) server that can be used as a ReadWriteMany (RWX) volume in the Kyma cluster.
- VPC Peering [page 2109]: Virtual Private Cloud (VPC) peering between your SAP BTP, Kyma runtime and remote cloud provider's project, account, or subscription.
- Redis [page 2113]: cloud provider-flavored cache that can be used in your Kyma cluster.

Architecture

Cloud Manager has read and write access to your IpRange, VpcPeering, NfsVolume, and Redis custom resources (CRs) in the Kyma cluster. The module also manages Kyma VPC networks, NFS Volume instances, and Redis instances in your cloud provider landscape in Kyma.



API/Custom Resource Definitions

The `cloud-resources.kyma-project.io` Custom Resource Definition (CRD) describes the kind and the format of data that Cloud Manager uses to configure resources. For more information, see Cloud Manager Resources [page 2115].

Related Information

kyma-project.io: Cloud Manager module: Tutorials
Calculation with the Cloud Manager Module [page 106]

4.3.1.4.1 NFS

Cloud Manager NFS feature enables creation of RWX volumes, allowing multiple workloads to use the same storage.

The Cloud Manager module enables a Network File System (NFS) feature which allows you to create ReadWriteMany (RWX) volumes. While regular persistent disk volumes can't be shared and can be used by only a single workload, RWX volumes can be used by multiple workloads.

> → Remember
>
> Using the Cloud Manager module and enabling NFS storage, introduces additional costs. For more information, see Calculation with the Cloud Manager Module [page 106].

Cloud Providers

When you create an NFS instance in SAP BTP, Kyma runtime, you depend on the cloud provider of your Kyma cluster. The cloud provider in use determines the exact implementation.

The Cloud Manager module supports the NFS feature of the following cloud providers:

- Amazon Web Services Amazon Elastic File System
- Google Cloud Filestore

To create and configure a Cloud Manager's NFS instance, use a dedicated NFS custom resource (CR) corresponding with the cloud provider for your Kyma cluster, namely AwsNfsVolume CR or GcpNfsVolume CR. For more information, see NFS Resources [page 2115].

Prerequisites

To instantiate NFS, an IpRange CR must exist in the Kyma cluster. IpRange defines network address space reserved for your cloud provider's NFS resources. If you don't create the IpRange CR manually, Cloud Manager creates a default IpRange CR with the default address space and Classless Inter-Domain Routing (CIDR) selected. For more information, see IP Range [page 2115].

Lifecycle

AwsNfsVolume and GcpNfsVolume are namespace-level CRs. Once you create a GcpNfsVolume or AwsNfsVolume resource, the following resources are also created automatically:

- IpRange CR
  - IpRange is a cluster-level CR.
  - Only one IpRange CR can exist per cluster.

    - If you don't want the default IpRange to be used, create one manually.
- Persistent Volume (PV)
    - PV is a cluster-level piece of storage.
    - The PV uses a unique name based on the NFS instance status ID.
    - The PV references the instantiated NFS in the respective cloud provider.
- Persistent Volume Claim (PVC)
    - PVC is a namespace-level resource created in the same namespace as the IpRange CR.
    - By default, the PVC's name is the same as the name of your NFS instance unless you specify a different name.
    - The PVC references the automatically created PV.

Related Information

kyma-project.io: Using NFS in Amazon Web Services
kyma-project.io: Using NFS in Google Cloud
NFS Resources [page 2115]
Calculation with the Cloud Manager Module [page 106]

4.3.1.4.2 VPC Peering

VPC Peering in the Cloud Manager module enables peering with remote VPC networks of the same cloud provider.

The Cloud Manager module enables managed Virtual Private Cloud (VPC) peering functionality that allows you to peer the Kyma VPC network with a remote VPC network. Virtual network peering is possible only between networks of the same cloud providers. VPC peering in Kyma is fully automated. It means that Cloud Manager configures the peering on both Kyma's and cloud provider's side.

Cloud Providers

When you configure VPC peering in SAP BTP, Kyma runtime, you depend on the cloud provider of your Kyma cluster. The cloud provider in use determines the exact implementation.

The Cloud Manager module supports the VPC Peering feature of the following cloud providers:

- Amazon Web Services VPC peering
- Google Cloud VPC Network Peering
- Microsoft Azure Virtual network peering

You can configure Cloud Manager's VPC peering using a dedicated custom resource (CR) corresponding with the cloud provider for your Kyma cluster, namely:

- AwsVpcPeering CR
- GcpVpcPeering CR
- AzureVpcPeering CR

For more information, see VPC Peering Resources [page 2115].

Prerequisites

Before you initiate VPC peering from a Kyma cluster, you must perform the following actions:

- Authorize Cloud Manager in the remote cloud provider landscape. For more information, see Authorizing Cloud Manager in the Remote Cloud Provider [page 2111].
- Tag the remote network with the Kyma shoot name. For more information, see the following tutorials:
  - Allow SAP BTP, Kyma Runtime to Peer with Your Remote Network in Creating Virtual Private Cloud Peering in Amazon Web Services.
  - Allow SAP BTP, Kyma Runtime to Peer with Your Remote Network in Creating Virtual Private Cloud Peering in Google Cloud.
  - Allow SAP BTP, Kyma Runtime to Peer with Your Remote Network in Creating Virtual Private Cloud Peering in Microsoft Azure.

Lifecycle

VPC peering CRs are cluster-level resources. Once one of the VPC peering resources is applied, the status of the VPC peering connection is reflected in that CR. At the same time, Cloud Manager creates a VPC peering connection in the Kyma cluster underlying cloud provider landscape and accepts the VPC peering connection in the remote cloud provider landscape.

When you delete a VPC peering CR, the VPC peering connection in the Kyma cloud provider landscape is deleted automatically. However, the remote VPC peering connection is left hanging, and you must delete it manually.

Limitations

The limit on the number of VPC peering CRs per Kyma cluster depends on the quotas for each cloud provider.

For Microsoft Azure, changes to the address space are not synced between the Kyma and remote VPC networks. If you resize the remote VPC network address space, routes are out of sync. In this case, you must delete the existing AzureVpcPeering CR and create a new one.

Related Information

kyma-project.io: Creating VPC Peering in Amazon Web Services

kyma-project.io: Creating VPC Peering in Google Cloud

kyma-project.io: Creating VPC Peering in Microsoft Azure

VPC Peering Resources [page 2115]

4.3.1.4.2.1 Authorizing Cloud Manager in the Remote Cloud Provider

Learn about the entitlements for the Cloud Manager module in the remote cloud provider required to use the VPC peering feature.

To create VPC peering in SAP BTP, Kyma runtime, you must authorize the Cloud Manager module in the remote cloud provider to accept the connection.

Amazon Web Services

For cross-account access in Amazon Web Services, Cloud Manager uses `AssumeRole`. `AssumeRole` requires specifying the trusted principle. For more information, see the official Amazon Web Services documentation .

Use the following table to identify the Cloud Manager principal. Then, perform the required actions.

SAP BTP Cockpit URL	Kyma Dashboard URL	Cloud Manager Principal
https://emea.cockpit.btp.cloud.sap	https://dashboard.kyma.cloud.sap	`arn:aws:iam::194230256199:user/cloud-manager-peering-prod`

[1]Relevant only for the Government Cloud (US) region.

1. Create a new role named **CloudManagerPeeringRole** with a trust policy that allows the Cloud Manager principal to assume the role:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": {
                "AWS": "{CLOUD_MANAGER_PRINCIPAL}"
            },
            "Action": "sts:AssumeRole"
        }
    ]
}
```

2. Create a new **CloudManagerPeeringAccess** managed policy with the following permissions:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "Statement1",
            "Effect": "Allow",
            "Action": [
                "ec2:AcceptVpcPeeringConnection",
                "ec2:DescribeVpcs",
                "ec2:DescribeVpcPeeringConnections",
```

```
                "ec2:DescribeRouteTables",
                "ec2:CreateRoute",
                "ec2:CreateTags"
            ],
            "Resource": "*"
        }
    ]
}
```

3. Attach the **CloudManagerPeeringAccess** policy to the **CloudManagerPeeringRole**.

Google Cloud

Grant the following permissions to the Kyma service account in your GCP project:

Permission	Description
`compute.networks.addPeering`	Required to create the peering request in the remote project and VPC network.
`compute.networks.get`	Required to fetch the list of existing VPC peerings from the remote VPC network.
`compute.networks.ListEffectiveTags`	Required to check if the remote VPC network is tagged with the Kyma shoot name tag.

It is recommended to create an Identity and Access Management (IAM) custom role with the permissions listed above. For more information, see the official Google Cloud documentation on how to create a custom role or the Authorize Cloud Manager in the Remote Project section in the Creating VPC Peering in Google Cloud tutorial.

Service Account

Use the following table to identify the Cloud Manager service account:

SAP BTP Cockpit URL	Kyma Dashboard URL	Cloud Manager Service Account
https://emea.cockpit.btp.cloud.sap	https://dashboard.kyma.cloud.sap	`cloud-manager-peering@sap-ti-dx-kyma-mps-prod.iam.gserviceaccount.com`

With the service account, you can authorize the Cloud Manager module in the remote project. For more information, see the official Google Cloud documentation on how to grant a single role to a service account or the Authorize Cloud Manager in the Remote Project section in the Creating VPC Peering in Google Cloud tutorial.

Microsoft Azure

To authorize Cloud Manager in the remote subscription, Microsoft Azure requires specifying the service principal. Use the following table to identify the Cloud Manager service principal. Then, perform the required actions.

SAP BTP Cockpit URL	Kyma Dashboard URL	Cloud Manager Service Principal
https://emea.cockpit.btp.cloud.sap	https://dashboard.kyma.cloud.sap	`kyma-cloud-manager-peering-prod`

1. Verify if the Cloud Manager service principal exists in your tenant.
2. **Optional:** If the service principal doesn't exist, create one for the Cloud Manager application in your tenant.
3. Assign the `Classic Network Contributor` and `Network Contributor` roles to the Cloud Manager service principal.

For more information, see the official Microsoft Azure documentation on how to Assign Azure roles using the Azure portal and how to Manage service principals or the Authorize Cloud Manager in the Remote Subscription section in the Creating VPC Peering in Microsoft Azure tutorial.

4.3.1.4.3 Redis

Use the Cloud Manager module to provision Redis instances.

The Cloud Manager module allows you to provision a cloud provider-managed Redis instance within your cluster network.

> → Remember
>
> Using the Cloud Manager module and enabling Redis, introduces additional costs. For more information, see Calculation with the Cloud Manager Module [page 106].

Cloud Providers

When you create a Redis instance in SAP BTP, Kyma runtime, you depend on the cloud provider of your Kyma cluster. The cloud provider in use determines the exact implementation.

The Cloud Manager module supports the Redis feature of the following cloud providers:

- Amazon Web Services Amazon ElastiCashe for Redis OSS
- Google Cloud Memorystore
- Microsoft Azure Azure Cache for Redis

You can configure Cloud Manager's Redis instances using a dedicated Redis instance custom resource (CR) corresponding with the cloud provider for your Kyma cluster, namely AwsRedisInstance CR, GcpRedisInstance CR, or AzureRedisInstance CR. For more information, see Redis Resources [page 2116].

Tiers

When you provision a Redis instance, you can choose between Standard and Premium Tier.

- Standard Tier offers one instance.
- Premium Tier offers high availability with automatic failover by provisioning an additional read replica of your instance.

Prerequisites

To instantiate Redis, an IpRange CR must exist in the Kyma cluster. IpRange defines network address space reserved for your cloud provider's Redis resources. If you don't create the IpRange CR manually, Cloud Manager creates a default IpRange CR with the default address space and Classless Inter-Domain Routing (CIDR) selected. For more information, see IP Range [page 2115].

Lifecycle

AwsRedisInstance, GcpRedisInstance, and AzureRedisInstance are namespace-level CRs. Once you create any of the Redis resources, the following resources are also created automatically:

- IpRange CR
  - IpRange is a cluster-level CR.
  - Only one IpRange CR can exist per cluster.
  - If you don't want the default IpRange to be used, create one manually.
- Secret CR
  - The Secret is a namespace-level CR.
  - The Secret's name is the same as the name of the respective Redis instance CR.
  - The Secret holds values and information used to access the Redis instance.

Related Information

kyma-project.io: Using AwsRedisInstance Custom Resources
kyma-project.io: Using GcpRedisInstance Custom Resources
kyma-project.io: Using AzureRedisInstance Custom Resources
Redis Resources [page 2116]
Calculation with the Cloud Manager Module [page 106]

4.3.1.4.4 Cloud Manager Resources

Learn about Cloud Manager's custom resources.

The API of the Cloud Manager module is based on Kubernetes Custom Resource Definitions (CRDs), which extend the Kubernetes API with custom additions. To inspect the specification of the Cloud Manager module API, see the following custom resources (CRs):

IP Range

IpRange CR

The `iprange.cloud-resources.kyma-project.io` CRD describes the Virtual Private Cloud (VPC) network IP range used for IP address allocation for cloud resources that require an IP address. For more information, see IpRange Custom Resource .

NFS Resources

AwsNfsVolume CR

The `awsnfsvolume.cloud-resources.kyma-project.io` CRD describes the Amazon Web Services Elastic File System (Amazon EFS) instance that can be used as a ReadWriteMany (RWX) volume in the cluster. For more information, see AwsNfsVolume Custom Resource .

GcpNfsVolume CR

The `gcpnfsvolume.cloud-resources.kyma-project.io` CRD describes the Google Cloud Filestore instance that can be used as an RWX volume in the cluster. For more information, see GcpNfsVolume Custom Resource .

VPC Peering Resources

AwsVpcPeering CR

The `awsvpcpeering.cloud-resources.kyma-project.io` CRD describes the AWS peering connection between Kyma and the remote AWS Virtual Network. For more information, see AwsVpcPeering Custom Resource .

GcpVpcPeering CR

The `gcpvpcpeering.cloud-resources.kyma-project.io` CRD describes the VPC peering that you can use to peer your Kyma cluster with your Google Cloud project VPC. For more information, see GcpVpcPeering Custom Resource .

Redis Resources

AwsRedisInstance CR

The `awsredisinstance.cloud-resources.kyma-project.io` CRD describes the Amazon ElastiCache Redis instance. For more information, see AwsRedisInstance Custom Resource .

GcpRedisInstance CR

The `gcpredisinstance.cloud-resources.kyma-project.io` CRD describes the Google Cloud Memorystore for Redis instance. For more information, see GcpRedisInstance Custom Resource .

AzureRedisInstance CR

The `azureredisinstance.cloud-resources.kyma-project.io` CRD describes the Microsoft Azure Cache for Redis instance. For more information, see AzureRedisInstance Custom Resource .

4.3.1.5 Application Connector Module

With the Application Connector module, you can connect your workflows deployed on Kyma with external solutions. No matter if you want to integrate an on-premise or a cloud system, the integration process does not change, which prevents any configuration or network-related problems.

What is Application Connectivity in Kyma?

Application Connectivity in Kyma simplifies the interaction between external systems and your Kyma workloads. The main benefits are:

- Smooth and loosely coupled integration of external systems with Kyma workloads using Kyma Eventing
- Easy consumption of SAP BTP services by supporting the SAP BTP Extensibility approach
- Establishing high-security standards for any interaction between systems by using trusted communication channels and authentication methods
- Reducing configuration changes for Kyma workloads through encapsulating configuration details of external API endpoints
- Monitoring and tracing capabilities to facilitate operational aspects

The Application Connector module bundles all features of Application Connectivity in Kyma. You can install and manage the module using Kyma dashboard [page 3511].

The module includes Kubernetes operators and is fully configurable over its own Kubernetes custom resources (CRs). For each external system, a dedicated configuration is used. This allows for individual configuration of security and aspects (like encryption and authentication) per system.

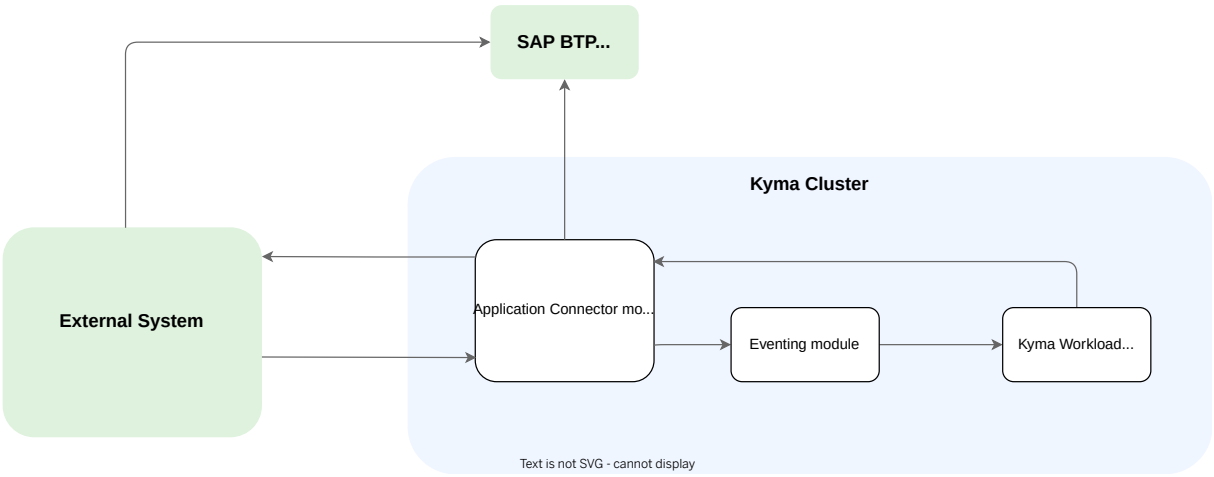
Besides proxying any ingress and egress requests to external systems and dealing with security concerns, it also includes full integration with SAP BTP Unified Customer Landscape (UCL) to simplify the consumption of SAP BTP services.

Features

The Application Connector module provides the following features:

- Easy installation of Kyma's Application Connectivity capabilities by enabling the Application Connector module in your SAP BTP, Kyma runtime.
- Simple configuration using Kubernetes CRs and easy management with Kyma dashboard.
- Full integration of BTP's UCL service, which implements the SAP BTP Extensibility concept. This allows for the automated integration of external systems registered in the UCL service.
- Dispatching of incoming requests from external systems to Kyma workloads (for example, a Kyma Serverless Function) by using an Istio Gateway with mTLS and the Kyma Eventing module .
- Proxying outgoing requests to external APIs and transparently covering security requirements like encryption and authentication (like OAuth 2.0 + mTLS, Basic Auth, and Client Certificates).
- Metering of throughput and exposing monitoring metrics.

Architecture



Application Connector Manager

When you add the Application Connector module, Application Connector Manager takes care of installation and configuration of all the Application Connector module components in your cluster. It manages the lifecycle of the Application Connector module based on the dedicated `ApplicationConnector` custom resource (CR).

API/Custom Resource Definitions

The API of the Applicaton Connector module is based on Kubernetes Custom Resource Definitions (CRD), which extend the Kubernetes API with custom additions. To inspect the specification of the Application Connector module API, see:

- Application CRD
- CompassConnection CRD
- ApplicationConnector CRD

Resource Consumption

To learn more about the resources used by the Application Connector module, see Application Connector [page 3519].

Useful Links

To learn more how to work with the Application Connector module, see the Application Connectivity tutorials .

To learn more about the architecture, the configuration parameters, and any other references, see Technical Reference .

4.3.1.5.1 Enable and Disable Network Policies for the Application Connector Module

Learn about the network policies for the Application Connector module and how to enable and disable them.

Context

To increase security, the Application Connector module creates network policies that control traffic to and from its Pods. All the policies are disabled by default. You can enable these policies using Kyma dashboard or kubectl.

Network Policies for the Application Connector Module

Policy Name	Description
`kyma-project.io--acm-module-to-api-server`	Allows egress from the Application Connector module Pods to any destination on TCP port 443, such as the Kubernetes API server.
`kyma-project.io--acm-module-to-dns`	Allows egress from the Application Connector module Pods to DNS services on UDP/TCP ports 53 and 8053 for cluster and external DNS resolution.
`kyma-project.io--acm-module-allow-to-sidecar`	Allows egress from the Application Connector module Pods to Istio's istiod on TCP port 15012 for sidecar configuration.
`kyma-project.io--acm-module-allow-metrics`	Allows ingress to the Application Connector Manager Pods on TCP port 8080 from Pods in the `kyma-system` namespace labeled `networking.kyma-project.io/metrics-scraping: allowed` for metrics scraping.
`kyma-project.io--acm-module-allow-to-external-system`	Allows egress from the Compass Runtime Agent and Central Application Gateway Pods to external systems at 0.0.0.0/0, excluding link-local addresses.
`kyma-project.io--acm-gateway-allow-from-cluster-workload`	Allows ingress to the Central Application Gateway Pods on TCP ports 8080 and 8082 from any source within the cluster.
`kyma-project.io--acm-gateway-allow-from-health-check`	Allows ingress to the Central Application Gateway Pods on TCP port 8081 for health checks.
`kyma-project.io--acm-gateway-allow-to-external-system`	Allows egress from the Central Application Gateway Pods to external systems on TCP port 443.
`kyma-project.io--acm-validator-allow-from-istio-ingressgateway`	Allows ingress to the Validator Pods on TCP port 8080 from Istio Ingress Gateway in the `istio-system` namespace.
`kyma-project.io--acm-validator-allow-from-health-check`	Allows ingress to the Validator Pods on TCP port 8081 for health checks.
`kyma-project.io--acm-validator-allow-to-eventing`	Allows egress from the Validator Pods to the Eventing Publisher Proxy on TCP port 8080.
`kyma-project.io--acm-compass-allow-from-health-check`	Allows ingress to the Compass Runtime Agent Pods on TCP port 8090 for health checks.

Procedure

- Use Kyma dashboard
  a. Go to Kyma dashboard. The URL is in the *Overview* section of your subaccount.
  b. Go to *Configuration* > *Modules*.
  c. Choose the *application-connector* module from the list, and switch to the *Edit* tab.
  d. In the ApplicationConnector custom resource specification, find the `networkPoliciesEnabled` parameter. To enable all the network policies, change the parameter value to `true`. If the value is `false`, the network policies are disabled.
- Use kubectl

a. To enable all the network policies for the Application Connector module, run the following command:

```
kubectl patch applicationconnector applicationconnector-sample -n kyma-
system \
  --type=merge \
  -p '{"spec":{"networkPoliciesEnabled": true}}'
```

b. To disable the network policies, run the following command:

```
kubectl patch applicationconnector applicationconnector-sample -n kyma-
system \
  --type=merge \
  -p '{"spec":{"networkPoliciesEnabled": false}}'
```

c. To check if the network policies are active, run the following command:

```
kubectl get networkpolicies -n kyma-system -l kyma-project.io/
module=application-connector
```

4.3.1.6 Keda Module

Learn more about the Keda module. Use it to install and manage the KEDA autoscaler in your Kubernetes cluster.

What Is KEDA?

Kubernetes-based Event Driven Autoscaler (KEDA) is an autoscaler that allows you to easily scale your Kubernetes-based resources. You can scale your applications based on the data of your choice.

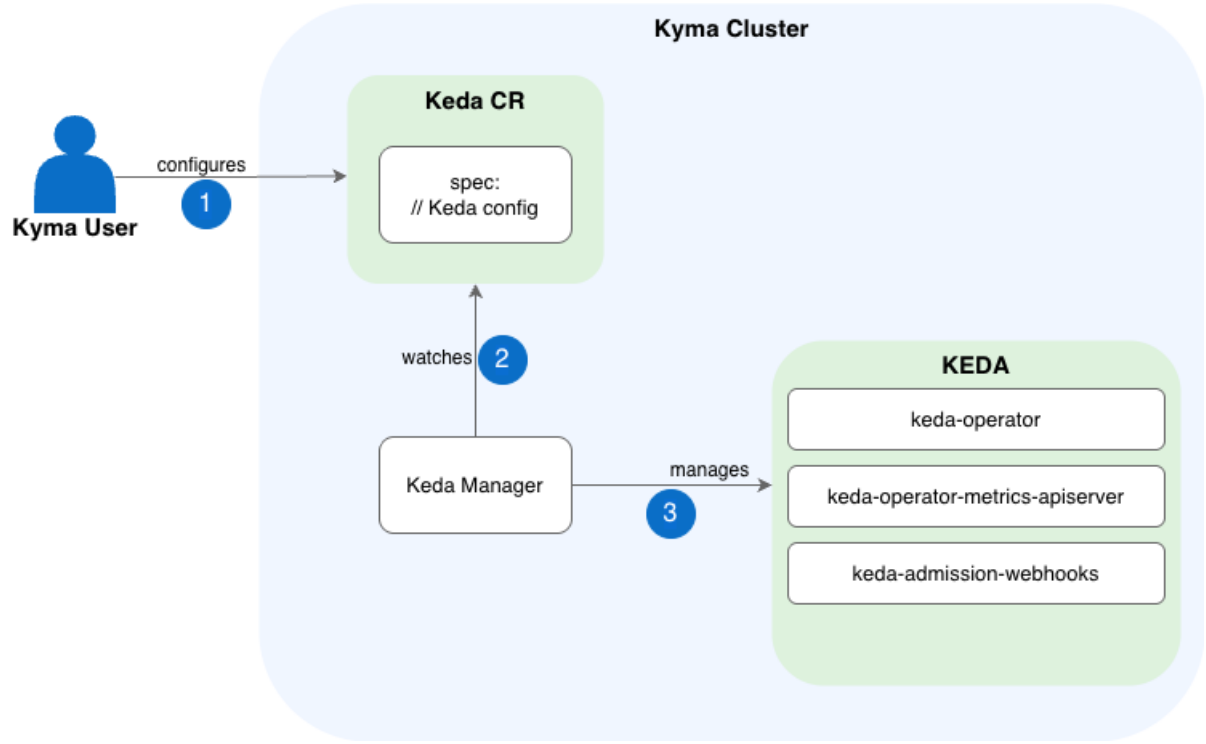
KEDA supports a great number of scalers that help you manage your deployments. For the complete list, see the KEDA Scalers documentation .

For more information about KEDA features, see the KEDA documentation .

Features

With the Keda module, you can have a custom event-driven autoscaling for Kubernetes workloads.

Architecture



1. User configures the Keda custom resource (CR). With the CR you can determine the way the Keda module runs.
2. Keda Manager watches the Keda CR.
3. Keda Manager manages the KEDA workloads.

To learn more about the KEDA architecture, see the KEDA architecture diagram .

Keda Manager

Keda Manager helps you to install and manage KEDA in your cluster. It manages the lifecycle of KEDA based on the dedicated Keda CR.

API/Custom Resource Definitions

The `kedas.operator.kyma-project.io` CustomResourceDefinition (CRD) is a detailed description of the Keda module configuration that you want to install on your cluster. See Keda .

To learn more about the KEDA CR, see KEDA Custom Resources .

Authorization

To assign access permissions to the Keda module resources, use the following aggregated ClusterRoles :

- `kyma-keda-view` - Grants read-only access to the Keda custom resource and resources from all API groups managed by KEDA.
- `kyma-keda-edit` - Grants full access to the Keda custom resource and resources from all API groups managed by KEDA.

Resource Consumption

To learn more about the resources used by the Keda module, see Keda Module's Sizing [page 3519].

Keda Module Demo Applications

To learn how to scale the Kubernetes workloads using the KEDA API based on a simple CPU consumption case, and how the Keda module can complement other Kyma components, see Demo Applications .

4.3.1.6.1 Configuring Keda Module

By default, the Keda module comes with the default configuration. You can change the configuration using the Keda CustomResourceDefinition (CRD). See how to configure the `logging.level` attribute, enable the Istio sidecar injection, change resource consumption, define custom annotations, override the minimum TLS version, or enable the KEDA HTTP Add-on.

Prerequisites

- You have added the Keda module. To learn how to do it, see Adding and Deleting a Kyma Module [page 3511].

Procedure

1. Go to Kyma dashboard. The URL is in the Overview section of your subaccount.
2. Choose *Modify Modules*, and in the *View* tab, choose *keda*.
3. Go to *Edit*, and provide your configuration changes. You can use the *Form* or *YAML* tab.

- To define the level of detail of your logs, set the `logging.level` attribute to one of the following values:
  - `debug` - is the most detailed option. Useful for a developer during debugging.
  - `info` - provides standard log level indicating operations within the Keda module. For example, it can show whether the workload scaling operation was successful or not.
  - `error` - shows error logs only. This means only log messages corresponding to errors and misconfigurations are visible in logs.
  - `warn` - shows warning logs only. This means only log messages corresponding to warnings and potential issues are visible in logs.

```
spec:
  logging:
    operator:
      level: "debug"
```

- To define the log output format, set the `logging.format` attribute to one of the following values:
  - `json` - outputs logs in JSON format, which is structured and machine-readable.
  - `console` - outputs logs in plain text format, which is human-readable.

```
spec:
 logging:
   operator:
     level: "info"
     format: "json"
   metricServer:
     level: "info"
     format: "json"
   admissionWebhook:
     level: "info"
     format: "json"
```

  For more information about logging configuration, see Keda logging configuration.

- To enable the Istio sidecar injection [page 1889] for `operator` and `metricServer`, set the value of `enabledSidecarInjection` to `true`. For example:

```
spec:
  istio:
    metricServer:
      enabledSidecarInjection: true
    operator:
      enabledSidecarInjection: true
```

- To change the resource consumption, enter your preferred values for `operator`, `metricServer`, and `admissionWebhook`. For example:

```
spec:
  resources:
    operator:
      limits:
        cpu: "1"
        memory: "200Mi"
      requests:
        cpu: "150m"
        memory: "150Mi"
    metricServer:
      limits:
        cpu: "1"
        memory: "1000Mi"
      requests:
```

```
        cpu: "150m"
        memory: "500Mi"
    admissionWebhook:
      limits:
        cpu: "1"
        memory: "1000Mi"
      requests:
        cpu: "50m"
        memory: "800Mi"
```

- To define custom annotations for KEDA workloads, enter your preferred values for `operator`, `metricServer` and `admissionWebhook`. For example:

```
spec:
  podAnnotations:
   operator:
     metrics.dynatrace.com/scrape: 'true'
     metrics.dynatrace.com/path: '/metrics'
   metricServer:
     metrics.dynatrace.com/scrape: 'true'
     metrics.dynatrace.com/path: '/metrics'
   admissionWebhook:
     metrics.dynatrace.com/scrape: 'true'
     metrics.dynatrace.com/path: '/metrics'
```

- To override the minimum TLS version used by KEDA (default is `TLS12`), set the `KEDA_HTTP_MIN_TLS_VERSION` environment variable. For example:

```
spec:
  env:
    - name: KEDA_HTTP_MIN_TLS_VERSION
      value: TLS13
```

- To enable the KEDA HTTP Add-on, which extends KEDA with the ability to scale HTTP workloads to and from zero based on incoming request rate, annotate the `Keda` CR:

```
kubectl annotate keda -n kyma-system default \
                         keda.kyma-project.io/addon-enabled=true
```

- Optionally, to enable Istio sidecar injection on the HTTP Add-on Deployments, set the following annotation:

```
kubectl annotate keda -n kyma-system default \
                         keda.kyma-project.io/addon-istio-injection=true
```

- Optionally, to install the HTTP Add-on in a different namespace than the default `kyma-system`, set the following annotation:

```
kubectl annotate keda -n kyma-system default \
                         keda.kyma-project.io/addon-namespace=my-new-
namespace --overwrite
```

Next Steps

For more information about the KEDA resources, see KEDA concepts .

4.3.1.6.2 Configuring Logging

Learn how to configure logging for Keda Manager and the Keda module components.

Prerequisites

You have kubectl installed.

Supported Log Levels

From the least to the most verbose: `fatal`, `panic`, `dpanic`, `error`, `warn`, `info` (default), `debug`.

Supported Log Formats

The supported log formats are the following:

- `json` - Structured JSON format (default)
- `console` (or `text`) - Human-readable console format

Configure Keda Manager Logging

Keda Manager (`keda-manager`) supports dynamic log-level reconfiguration using a ConfigMap. Changes take effect without requiring a Pod restart.

To change the log level, run:

```
kubectl patch configmap keda-log-configmap -n kyma-system --type merge -p
'{"data":{"log-config.yaml":"logLevel: debug"}}'
```

To change the log level and format, run:

```
kubectl patch configmap keda-log-configmap -n kyma-system --type merge -p
'{"data":{"log-config.yaml":"logLevel: debug\nlogFormat: json"}}'
```

To verify the change, run:

```
kubectl logs -n kyma-system -l app.kubernetes.io/name=keda-manager
```

Configure the Keda Module Components Logging

> → Remember
>
> Applying logging configuration changes to the `operator`, `metrics-server`, and `admission-webhooks` components triggers a restart to apply the new settings.

Supported Time Encodings

The supported time encodings are the following:

- `rfc3339` - RFC3339 format (default): `2006-01-02T15:04:05Z07:00`
- `rfc3339nano` - RFC3339 with nanoseconds: `2006-01-02T15:04:05.999999999Z07:00`
- `iso8601` - ISO8601 format: `2006-01-02T15:04:05.000Z0700`
- `epoch` - Unix timestamp in seconds
- `millis` - Unix timestamp in milliseconds
- `nano` - Unix timestamp in nanoseconds

Configuration

Update the Keda custom resource to configure logging for the Keda module components.

```
apiVersion: operator.kyma-project.io/v1alpha1
kind: Keda
metadata:
  name: default
  namespace: kyma-system
spec:
  logging:
    operator:
      level: "debug"
      format: "json"
      timeEncoding: "rfc3339"
    metricServer:
      level: "info"
      format: "json"
      timeEncoding: "rfc3339"
    admissionWebhook:
      level: "info"
      format: "json"
      timeEncoding: "rfc3339"
```

> ⓘ Note
>
> The zap logger used by the Keda module components (`operator, metrics-apiserver, admission-webhooks`) has the following fixed field names in JSON format that cannot be customized: `level`, `ts`, `logger`, `msg`, `caller`.

Verify the Changes

- To check the `keda-operator` logs, run:

```
kubectl logs -n kyma-system -l app=keda-operator
```

- To check the `keda-metrics-apiserver` logs, run:

```
kubectl logs -n kyma-system -l app=keda-operator-metrics-apiserver
```

- To check the `keda-admission-webhooks` logs, run:

```
kubectl logs -n kyma-system -l app=keda-admission-webhooks
```

Limitations

The klog logs (Kubernetes API server framework) used by `keda-operator-metrics-apiserver` are always in text format, prefixed with `I`, `W`, `E` for Info, Warning, Error.

```
I0203 12:03:29.253333       1 requestheader_controller.go:180] Starting
RequestHeaderAuthRequestController <- Actual log line
W0203 11:39:56.585970       1 this is a warning example
E0203 11:39:56.586030       1 this is an error example
```

Related Information

Configuring Keda Module [page 2122]
KEDA Logs

4.3.1.6.3 KEDA HTTP Add-on

The KEDA HTTP Add-on extends KEDA with the ability to scale HTTP workloads to and from zero based on incoming request rate.

Overview

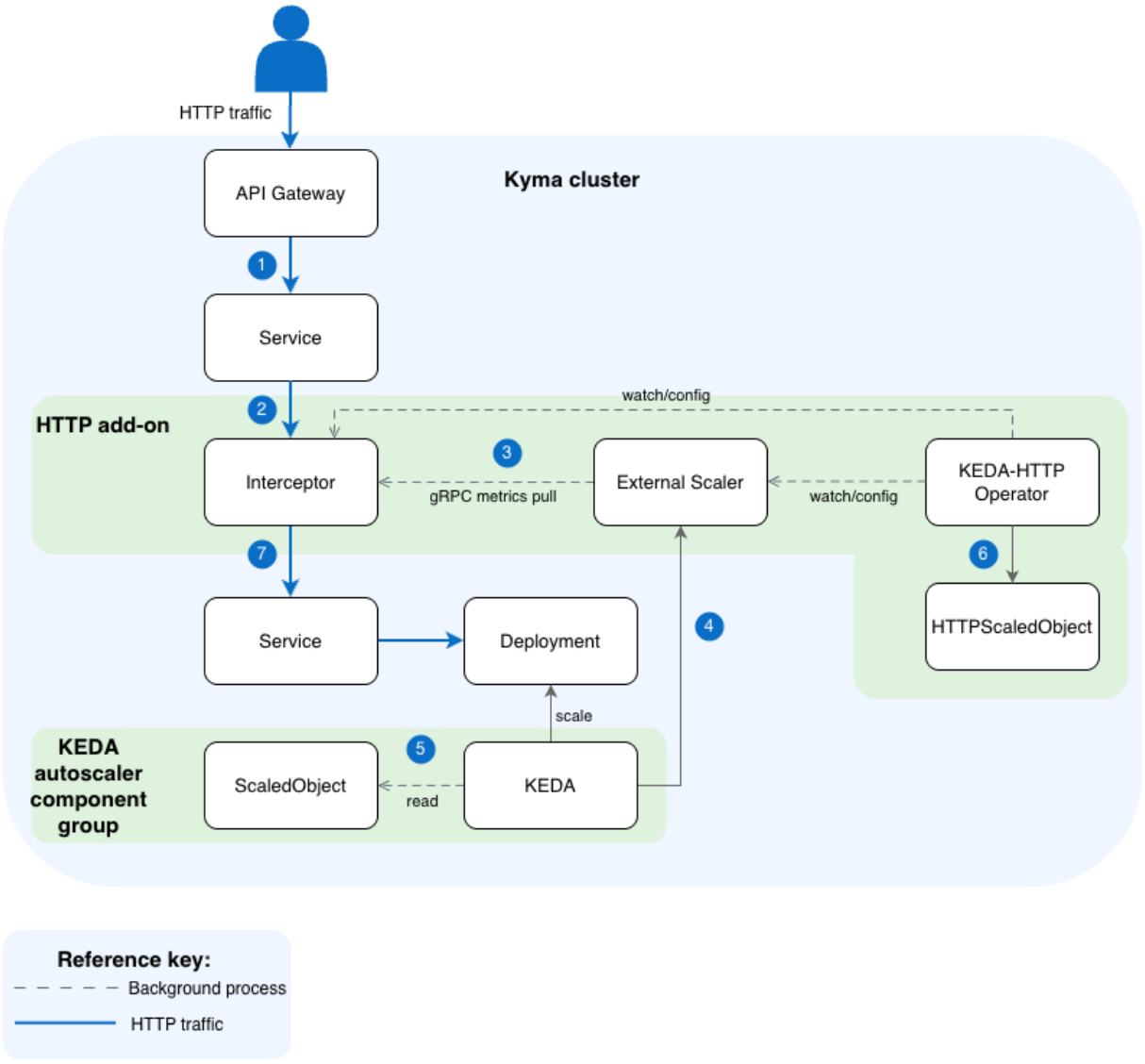
The KEDA HTTP Add-on extends KEDA with the ability to scale HTTP workloads to and from zero based on incoming request rate. It works by placing an **Interceptor proxy** in front of your application that counts, queues, and forwards HTTP requests — enabling true scale-to-zero without losing any requests.

Architecture

The HTTP Add-on consists of three components:

Component	Role
`Interceptor`	A reverse proxy that sits in front of your application. It counts incoming requests, queues them when the target has 0 replicas, and forwards them once the target is ready.
`Scaler` (External Scaler)	Exposes request-rate metrics to KEDA via gRPC so KEDA can make scaling decisions.
`Operator`	Watches `HTTPScaledObject` resources and configures the Interceptor routing and KEDA `ScaledObject`s.

Request flow:



1. An HTTP request arrives at the API Gateway, which routes traffic to the `Interceptor`'s `Service`.

2. The `Interceptor` receives the request, counts it, and queues it if the target workload has 0 replicas.
3. The `External Scaler` pulls request count metrics from the `Interceptor` via gRPC.
4. KEDA reads metrics from the `External Scaler` and scales the target `Deployment` accordingly (including to/from zero).
5. KEDA reconciles the `ScaledObject` (auto-created from `HTTPScaledObject`) to manage the scaling behavior.
6. The KEDA-HTTP Operator watches `HTTPScaledObject` resources and configures all add-on components (Interceptor routing, `ScaledObject`, `External Scaler`).
7. Once the `Deployment` has ready replicas, the `Interceptor` forwards the queued request to the application `Service`, which routes it to the running `Pod`.

Enabling and Disabling the HTTP Add-on

Enable the HTTP Add-on by annotating the `Keda` custom resource (CR):

```
kubectl annotate keda -n kyma-system default \
  keda.kyma-project.io/addon-enabled=true
```

Annotations Reference

Annotation	Required	Description
`keda.kyma-project.io/addon-enabled`	Yes	Set to true to install, false to uninstall.
`keda.kyma-project.io/addon-namespace`	No	Namespace where the add-on is installed. Defaults to `kyma-system`.
`keda.kyma-project.io/addon-istio-injection`	No	Set to true to enable Istio sidecar injection on the add-on `Deployment`s. Defaults to false — the add-on `Deployment`s are annotated with `sidecar.istio.io/inject`: "false" unless this annotation is explicitly set to true. When enabled, the `Interceptor` `Deployment` also receives `traffic.sidecar.istio.io/excludeInboundPorts`: "9090" to prevent the sidecar from intercepting internal gRPC traffic.

Changing the Installation Namespace

To move the HTTP Add-on to a different namespace, update the `addon-namespace` annotation:

```
kubectl annotate keda -n kyma-system default \
  keda.kyma-project.io/addon-namespace=my-new-namespace --overwrite
```

The controller detects the namespace change, removes only the HTTP Add-on resources from the old namespace (other `Deployments`, `Services`, and so on in that namespace are not affected), creates the new namespace, if it doesn't exist, with `istio-injection=enabled`, and installs the HTTP Add-on in the new namespace.

Disabling the HTTP Add-on

To disable the HTTP Add-on, run:

```
kubectl annotate keda -n kyma-system default \
  keda.kyma-project.io/addon-enabled=false --overwrite
```

This removes all add-on resources from the cluster. Only the resources managed by the HTTP Add-on are removed. Other workloads in the namespace are not affected.

Uninstall Protection

The Keda Manager refuses to disable or uninstall the HTTP Add-on while any `HTTPScaledObject` resources exist in the cluster. This protects user workloads from losing their scaling controller and ending up with orphaned resources after the CRD is removed.

If you try to disable the add-on or delete the Keda module CR while `HTTPScaledObject`s exist, the Keda CR's `Status.State` flips to Warning and the `Addon` condition is set to False with reason HTTPAddonInUse. The message tells you how many `HTTPScaledObject`s are blocking the uninstall.

To find and remove them:

```
kubectl get httpscaledobjects -A
         kubectl delete httpscaledobject -n <namespace> <name>
```

Once all `HTTPScaledObject`s are deleted, the Keda Manager retries automatically within about 30 seconds.

Configuring the HTTP Add-on

The HTTP Add-on components are configured using environment variables on their `Deployments`. You can customize them by patching the respective `Deployment` after installation.

Interceptor Timeouts

The most important configuration options are the `Interceptor`'s timeout settings. These control how long the `Interceptor` waits during a cold start and when to forward requests to your application.

Environment Variable	Default	Description
`KEDA_HTTP_REQUEST_TIMEOUT`	0s (unlimited)	Total wall-clock deadline from request arrival to response completion. When 0, there is no total request deadline — the request can wait indefinitely for scale-up.
`KEDA_HTTP_READINESS_TIMEOUT`	0s (unlimited)	How long to wait for the backing workload to have ≥1 ready replicas before giving up. When 0, the readiness wait is bounded only by the request timeout, giving the full request budget to cold starts.
`KEDA_HTTP_RESPONSE_HEADER_TIMEOUT`	300s	How long to wait for response headers from the backend after the request is forwarded. Acts as a safety net against hung backends. Set to 0 to disable.
`KEDA_HTTP_CONNECT_TIMEOUT`	500ms	Per-attempt TCP dial timeout when connecting to the backend. Bounded by the request context deadline.

> ### Note
>
> If `KEDA_HTTP_REQUEST_TIMEOUT` is set to 0 (default), the `Interceptor` waits indefinitely for the target to scale up. This is the recommended setting when using the EnvoyFilter retry policy, as the retry policy on the Ingress Gateway side handles client-facing timeouts.

Interceptor Connection Pool

The following settings control the `Interceptor`'s internal HTTP connection pool to backend services:

Environment Variable	Default	Description
`KEDA_HTTP_MAX_IDLE_CONNS`	1000	Max idle connections across all backend services. Increase if you proxy to many unique backends.
`KEDA_HTTP_MAX_IDLE_CONNS_PER_HOST`	200	Max idle connections per backend service. Increase if you observe many new connection establishments under load.
`KEDA_HTTP_FORCE_HTTP2`	false	Force HTTP/2 for all upstream connections.

Interceptor Behavior

Environment Variable	Default	Description
KEDA_HTTP_ENABLE_COLD_START_HEADER	true	When enabled, the `Interceptor` adds an `X-KEDA-HTTP-Cold-Start: true` response header if the request triggered a scale-from-zero. Useful for observability.
KEDA_HTTP_LOG_REQUESTS	false	Log every incoming request (for debugging).

Scaler Configuration

The `External Scaler` component has the following key settings:

Environment Variable	Default	Description
KEDA_HTTP_QUEUE_TICK_DURATION	500ms	How often the scaler fetches request counts from the `Interceptor`. Lower values give faster scaling reactions but increase gRPC traffic.
KEDA_HTTP_SCALER_STREAM_INTERVAL_MS	200	Interval (ms) between metric stream updates sent to KEDA.

Usage: HTTPScaledObject

After the add-on is installed, create an `HTTPScaledObject` to configure scaling for your workload:

```
apiVersion: http.keda.sh/v1alpha1
kind: HTTPScaledObject
metadata:
  name: my-app
  namespace: my-namespace
spec:
  hosts:
  - "my-app.example.com"
  pathPrefixes:
  - /
  scaleTargetRef:
    name: my-app
    kind: Deployment
    apiVersion: apps/v1
    service: my-app
    port: 8080
  replicas:
    min: 0
    max: 10
  scalingMetric:
    requestRate:
      targetValue: 10
```

```
      granularity: "1s"
      window: "1m"
```

Key fields:

- `replicas.min`: 0 — enables scale-to-zero.
- `scalingMetric.requestRate.targetValue` — number of requests per second per replica that triggers scale-out.
- `scalingMetric.requestRate.window` — time window over which request rate is averaged.

Limitations and Throughput Considerations

Interceptor Is a Proxy in the Data Path

Every request to your application goes through the `Interceptor`. This adds:

- **Latency:** Approximately 1–5ms per request in steady state (non-cold-start).
- **Resource overhead:** The `Interceptor` consumes CPU and memory proportional to the request rate.

Interceptor Queue Capacity

When the target is at 0 replicas, the `Interceptor` queues incoming requests **in memory**. Limits:

- There is **no configurable queue size limit** — all requests are queued until the `Pod` comes up or the request times out.
- **Memory pressure:** Under high burst traffic to a scaled-to-zero workload, the `Interceptor` may consume significant memory. If it OOMs, all queued requests are lost.
- **Request timeout:** The `Interceptor` has a default forwarding timeout (configurable via `KEDA_HTTP_DEFAULT_TIMEOUT`, default 3000ms for the connect phase). Requests exceeding this timeout after forwarding are dropped.

Scaling Latency (Cold Start Time)

The time from first request to successful response depends on:

- **KEDA polling interval:** Default 15 seconds (configured via `pollingInterval` on the `ScaledObject`). This is the delay before KEDA detects pending requests.
- **Pod startup time:** Container pull, init containers, readiness probes, Istio sidecar injection (typically 10–60 seconds).
- **Cooldown period:** After traffic stops, KEDA waits the `cooldownPeriod` (default 300 seconds) before scaling to zero.

Total cold-start latency: Typically 15–90 seconds depending on your `Pod`'s startup time.

In an Istio mesh, the cold-start window can also cause `502` or `503` errors returned to the client. For details and the required EnvoyFilter configuration, see HTTP Add-on Returns 503 Errors During Cold Start [page 2134].

No Persistent Queue

Queued requests are stored in-memory. If the `Interceptor` `Pod` restarts or is evicted:

- All queued requests are **lost**.
- There is no at-least-once delivery guarantee.

Summary Table

Aspect	Value / Behavior
Added latency (steady state)	~1–5ms
Cold-start latency	15–90 seconds (depends on `Pod` startup)
Queue persistence	In-memory only (lost on restart)
Max queue size	Unlimited (bounded by memory)
KEDA polling interval	15s default (configurable)
Cooldown before scale-to-zero	300s default (configurable)

Related Information

KEDA HTTP Add-on GitHub
Scale-to-Zero Example
HTTP Add-on Returns 503 Errors During Cold Start [page 2134]

4.3.1.6.4 Troubleshooting for the Keda Module

Troubleshoot problems related to the Keda module.

4.3.1.6.4.1 HTTP Add-on Returns 503 Errors During Cold Start

Symptom

When your application is scaled to zero, and a new request arrives, the client receives a `502` or `503` error instead of a successful response.

Cause

The HTTP Add-on `Interceptor` queues requests while the target `Deployment` has zero replicas. Once it detects at least one ready endpoint, it immediately forwards the queued request without retrying on failure. In an Istio mesh, this creates a race condition during cold start:

1. The `Interceptor` receives the request and queues it.

2. KEDA detects pending requests and scales the `Deployment` from 0 to 1.
3. The `Pod` starts: container pull, init containers, readiness probes, Istio sidecar injection.
4. The readiness probe passes, but the Istio sidecar or the application is not yet fully ready to handle traffic.
5. The `Interceptor` sees a ready endpoint and forwards the queued request.
6. The upstream returns `503` or refuses the connection.

Additionally, in some edge cases the `Istio Ingress Gateway` can return a `502` or `503` from the `Interceptor` if it is temporarily overloaded during scale-up.

Solution

Configure an `EnvoyFilter` on the `Istio Ingress Gateway` to transparently retry failed requests during the cold-start window:

```
apiVersion: networking.istio.io/v1alpha3
kind: EnvoyFilter
metadata:
  name: my-app-keda-retry
  namespace: istio-system
spec:
  workloadSelector:
    labels:
      istio: ingressgateway
  configPatches:
  - applyTo: HTTP_ROUTE
    match:
      context: GATEWAY
      routeConfiguration:
        vhost:
          name: "my-app.example.com:443"
          route:
            name: ""
    patch:
      operation: MERGE
      value:
        route:
          retry_policy:
            retry_on: "5xx,connect-failure,reset,refused-stream"
            num_retries: 100
            per_try_timeout: 3s
```

> **Note**
>
> The `vhost.name` must match the actual virtual host name in Envoy's configuration. To find the correct name, run **`istioctl proxy-config routes <ingressgateway-pod> -o json`**. The format is typically `hostname:port` (for example, `my-app.example.com:443`), not `https://hostname`.

With this retry policy in place, failed attempts (`502`, `503`, connection refused) are transparently retried. The client receives a successful response once the `Pod` is ready — up to 100 × 3s = 5 minutes of retries.

Also set `KEDA_HTTP_REQUEST_TIMEOUT` to 0 (the default) on the `Interceptor` `Deployment`. This makes the `Interceptor` wait indefinitely for the target to scale up, letting the `EnvoyFilter` retry policy handle all client-facing timeouts.

4.3.1.7 Eventing Module

Use the Eventing module to set up event-driven communication between applications in your Kyma cluster using a publish-subscribe model.

What is Eventing?

The Eventing module enables event-driven communication between applications in your Kyma cluster. One application publishes an event (the publisher), and other applications (subscribers) subscribe to receive it.

This decouples your services, as publishers and subscribers don't need to know about each other. They can communicate asynchronously and evolve independently.

Features

The Eventing module provides the following features:

- Publish-subscribe (pub/sub) messaging: Decouples applications so you can build resilient and scalable event-driven systems.
- Flexible backend support: Use the in-cluster NATS backend (see NATS Module [page 2152]) or configure SAP Event Mesh (see SAP Event Mesh) for enterprise messaging.
- Standardized event format: All events follow the CloudEvents specification, ensuring a consistent and portable format.
- Automatic legacy event conversion: Converts older, non-standard Kyma event formats into valid CloudEvents automatically. For details, see Event Name Cleanup [page 2139].
- At-least-once delivery: Ensures that each event is delivered at least one time when you use the NATS backend, preventing message loss during temporary failures.
- Declarative subscriptions: Manage event subscriptions with a simple Subscription custom resource (CR).
- Built-in observability: Exposes key health and performance metrics in Prometheus format (see Eventing Metrics). Use the Telemetry module to collect, process, and forward these metrics to your observability backend.

Scope

The Eventing module focuses on in-cluster, asynchronous communication using the CloudEvents standard. It uses only one stream, with `file` storage, for all the events.

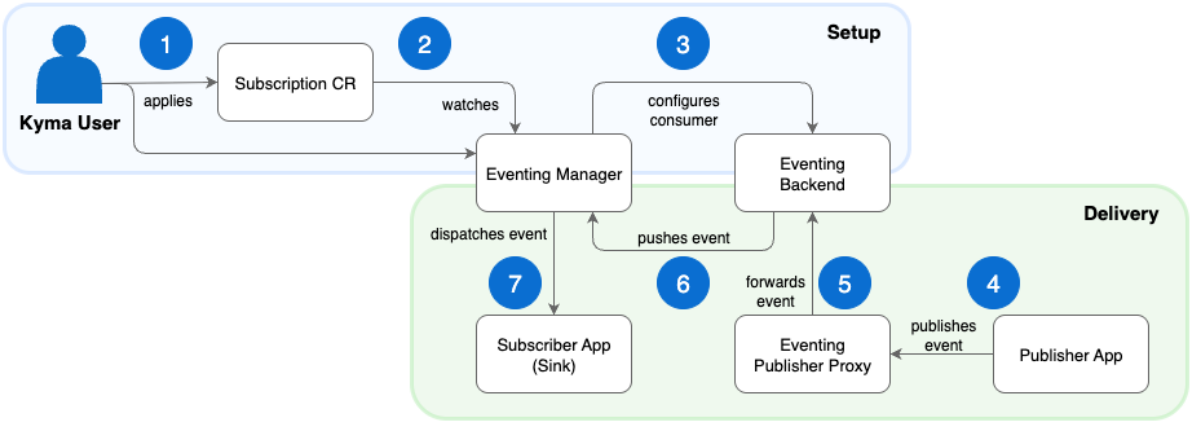
For cross-cluster or hybrid scenarios, configure SAP Event Mesh as the backend.

Architecture

The Eventing module uses the Eventing Manager, a Kubernetes operator , to manage the components that process and deliver events within your cluster.

The architecture involves the following flows:

- Subscription: The Eventing Manager watches for Subscription custom resources and configures the eventing infrastructure.
- Delivery: The Eventing Publisher Proxy receives events, and the configured backend (NATS or SAP Event Mesh) delivers them.



1. You create a Subscription CR. In the resource manifest, you specify the type of event you want to receive and the sink where to send it.
2. The Eventing Manager, continuously watching for Subscription resources, detects your Subscription CR.
3. Based on the backend configured in the Eventing CR, the Eventing Manager connects to the backend (NATS or SAP Event Mesh) and creates a subscription mechanism (a consumer or webhook).
4. A publisher app sends an event to the Eventing Publisher Proxy.
5. The Eventing Publisher Proxy forwards the event in CloudEvents format to your eventing backend.
6. The eventing backend processes the event and pushes it to the Eventing Manager.
7. The Eventing Manager receives the event, looks up the sink from the Subscription CR, and sends the event to your subscriber app. Your application can now process the event payload.

Eventing Manager

The Eventing Manager is the module's controller. It watches for Subscription custom resources and configures the underlying eventing infrastructure.

When you create or update a Subscription, the Eventing Manager performs the following tasks:

- Configures the selected eventing backend to manage event streams and consumers for subscriptions.
- Ensures that events are routed from the correct publisher to the specified subscriber (the sink).
- Creates and manages Kubernetes resources, such as `ConfigMap`, `Services`, and `DestinationRules`, adapting them to the desired state.

Eventing Publisher Proxy

The Eventing Publisher Proxy provides a single, stable endpoint for your applications to publish events using a standard HTTP POST request (endpoint: `/publish` for CloudEvents and `<application_name>/v1/`

`events` for legacy events). This simplifies integration, as you can use common tools like curl or any standard HTTP client.

The proxy performs the following tasks:

- Receives inbound events from your applications.
- Converts events from legacy formats into the standard CloudEvents format.
- Forwards validated CloudEvents to the configured eventing backend for delivery.

API/Custom Resource Definitions

You configure the Eventing module by creating and applying Kubernetes Custom Resource Definitions (CRD), which extend the Kubernetes API with custom additions:

- To understand and configure the module's global settings, see the Eventing CRD .
- To create a subscriber, define a Subscription CRD . You cannot delete the Eventing module as long as Subscription CRs exist.

Authorization

To assign access permissions to the Eventing module resources, use the following aggregated ClusterRoles :

- `kyma-eventing-view` - Grants read-only access to all Eventing module custom resources and their statuses.
- `kyma-eventing-edit` - Grants full access to all Eventing module custom resources and read-only access to their statuses.

Resource Consumption

To learn more about the resources used by the Eventing module, see Kyma Modules' Sizing [page 3518].

4.3.1.7.1 Event Naming and Cleanup

The Eventing module uses event names to identify and route events to subscribers. Learn about the supported event types, the naming conventions, and how the Eventing module handles event names with prohibited characters.

Event Types

Event names depend on the type of event. Eventing supports the following event types:

- CloudEvents: Events that conform to the CloudEvents specification . This is the recommended standard for describing event data in a common way. The specification is currently under the CNCF .
- Legacy events: Events or messages published to Kyma that do not conform to the CloudEvents specification. The Eventing Publisher Proxy converts all legacy events into CloudEvents before processing them.

Event Name Format

The event name format ensures compatibility with CloudEvents specifications and underlying messaging backends.

For a Subscription custom resource (CR), the fully qualified event name follows a structure like `order.created.v1` or `Account.Root.Created.v1`.

An event type consists of the following components:

- Event: Typically, has two or three segments separated by a dot (`.`), like `order.created` or `Account.Root.Created`
- Version: A version identifier, typically `v1`

For publishers, the event type takes these sample forms:

- `order.created` or `Account.Root.Created` for legacy events coming from the `commerce` application
- `order.created.v1` or `Account.Root.Created.v1` for CloudEvents

Event Name Cleanup

The Eventing Publisher Proxy modifies event names to filter out prohibited characters. This ensures compliance with CloudEvents specifications and the underlying messaging backend (such as NATS JetStream specifications).

If an event name contains prohibited characters, the Eventing Publisher Proxy removes these characters and uses the cleaned name for internal processing and routing. For example, if an event type is `order.payment*success.v1`, the Eventing module cleans it to `order.paymentsuccess.v1`.

To check the Subscription's cleaned event type, run:

```
kubectl get subscriptions {SUBSCRIPTION_NAME} -o=jsonpath="{.status.types}"
```

This cleanup happens internally; you can still publish and subscribe using the original event names (with prohibited characters). However, the Eventing module processes the cleaned version.

> ⚠ Caution
>
> This can lead to a naming collision if two different original event names clean up to the same internal name. For example, both `system>prod` and `systemprod` become `systemprod`. While this does not result in an error, a naming collision can cause subscribers to receive irrelevant events or miss expected events. For details, see Subscriber Receives Irrelevant Events [page 2151].

When you verify event delivery in your Function logs, the received event type reflects the cleaned name, not the original name you defined in the `Subscription` or published with.

4.3.1.7.2 Choose an Eventing Backend

The Eventing module supports two backends: NATS and SAP Event Mesh. Learn how to set up your preferred eventing backend.

Prerequisites

- You have the Eventing module in your Kyma cluster. See Adding and Deleting a Kyma Module [page 3511].
- If you want to use NATS, you have added the NATS module.
- If you want to use SAP Event Mesh, you have configured it for Kyma Eventing. See Configure SAP Event Mesh for Kyma Eventing [page 2141].

Context

You can send and receive events in Kyma with the Eventing module. By default, Kyma clusters have no eventing backend defined. You must set up an eventing backend, either based on the NATS technology or SAP Event Mesh.

Procedure

1. Log in to Kyma dashboard. The URL is in the *Overview* section of your subaccount.
2. Go to the `kyma-system` namespace.
3. Go to *Kyma* > *Eventing*, open the `Eventing` resource, and choose *Edit*.

4. Under *Backend Type*, select your preferred backend.
5. If you selected *EventMesh*, go to *Event Mesh Secret*, and select the namespace and name of your SAP Event Mesh service binding.

   > → Remember
   >
   > If you selected *NATS*, make sure the NATS module is added.

6. Choose *Save*.

Results

You have set up your eventing backend for Kyma.

If you want to choose another backend, edit the `Eventing` resource again, select the preferred backend, and save your changes.

You can no longer return to an empty eventing backend.

Related Information

SAP Event Mesh

4.3.1.7.3 Configure SAP Event Mesh for Kyma Eventing

If you want to use SAP Event Mesh as backend for the Eventing module, you must first set up the credentials.

Prerequisites

- You have the Eventing module and the SAP BTP Operator module in your Kyma cluster (see Adding and Deleting a Kyma Module [page 3511]).
- You are using an enterprise account.

Procedure

1. Log in to Kyma dashboard. The URL is in the *Overview* section of your subaccount.
2. To generate an SAP Event Mesh Secret, select a namespace and go to *Service Management* > *Service Instances*.

3. Choose *Create*, and enter the following information:
    - *Name* - enter the name of your instance.
    - *Offering Name* - type: **`enterprise-messaging`**.
    - *Plan Name* - type: **`default`**.
4. In *Parameters*, paste the following sample `config.json`:

```
{
  "options": {
      "management": true,
      "messagingrest": true
  },
  "rules": {
      "topicRules": {
          "publishFilter": [
              "${namespace}/*"
          ],
          "subscribeFilter": [
              "${namespace}/*"
          ]
      },
      "queueRules": {
          "publishFilter": [
              "${namespace}/*"
          ],
          "subscribeFilter": [
              "${namespace}/*"
          ]
      }
  },
  "resources": {
    "units": "10"
  },
  "version": "1.1.0",
  "emname": "<{EVENT_MESH_NAME}>",
  "namespace": "<{EVENT_MESH_NAMESPACE}>"
}
```

5. Replace `<{EVENT_MESH_NAME}>` and `<{EVENT_MESH_NAMESPACE}>` with the values you want your SAP Event Mesh instance to have.

> ### Note
>
> The `<{EVENT_MESH_NAMESPACE}>` must follow the SAP Event Specification; for example, it cannot exceed 64 characters, or begin or end with a dot or hyphen.

6. Go to *Service Management* > *Service Bindings* and choose *Create*.
7. Provide the name of your binding, select the name of your instance from the list, and choose *Create*.

Results

You have set up SAP Event Mesh for Kyma Eventing.

Next Steps

Choose SAP Event Mesh as your backend for the Eventing module.

Related Information

Choose an Eventing Backend [page 2140]

4.3.1.7.4 Subscribe to Multiple Event Types

Configure a single Subscription to receive events of multiple types.

Prerequisites

- You have the Eventing module in your Kyma cluster.
- You have access to Kyma dashboard. Alternatively, if you prefer CLI, you need kubectl and curl.
- Optionally, you have the CloudEvents Conformance Tool for publishing events.
- You have an inline Function as event sink (see Create and Modify an Inline Function).

> → Tip
>
> Use Istio sidecar proxies for reliability, observability, and security (see Istio Service Mesh [page 1889]).

Context

To subscribe to multiple events, you must create a Subscription custom resource (CR). The following example shows how to subscribe to events of two types: `order.received.v1` and `order.changed.v1`.

If you want to subscribe to more event types, add more types to your subscription.

Procedure

1. Create a subscription with multiple event types.
   - In Kyma dashboard, find the default namespace, go to *Configuration* and create a `Subscription` with the following parameters:
     - *Subscription name*: **`lastorder-sub`**

- *Types*: **`order.received.v1`** and **`order.changed.v1`**
- *Service*: **`lastorder`** (the sink is populated automatically)
- *Type matching*: **`standard`**
- *Source*: **`myapp`**

- With kubectl, run:

```
cat <<EOF | kubectl apply -f -
 apiVersion: eventing.kyma-project.io/v1alpha2
 kind: Subscription
 metadata:
  name: lastorder-sub
  namespace: default
 spec:
  sink: 'http://lastorder.default.svc.cluster.local'
  source: myapp
  types:
    - order.received.v1
    - order.changed.v1
EOF
```

2. Verify that the subscription is ready.
   - In Kyma dashboard, the status must be `READY`.
   - Alternatively, run:

     ```
     kubectl get subscriptions lastorder-sub -o=jsonpath="{.status.ready}"
     ```

     The response must be `true`.
3. Port-forward the Eventing Publisher Proxy to `localhost`, using port `3000`:

   ```
   kubectl -n kyma-system port-forward service/eventing-publisher-proxy 3000:80
   ```
4. To publish the first event (type `order.received.v1`) to trigger your Function, open another terminal window:
   - With the CloudEvents Conformance Tool, run:

     ```
     cloudevents send http://localhost:3000/publish \
        --type order.received.v1 \
        --id cc99dcdd-6f6d-43d6-afef-d024eb276584 \
        --source myapp \
        --datacontenttype application/json \
        --data "{\"orderCode\":\"3211213\", \"orderStatus\":\"received\"}" \
        --yaml
     ```
   - With *curl*, run:

     ```
     curl -v -X POST \
        -H "ce-specversion: 1.0" \
        -H "ce-type: order.received.v1" \
        -H "ce-source: myapp" \
        -H "ce-eventtypeversion: v1" \
        -H "ce-id: cc99dcdd-6f6d-43d6-afef-d024eb276584" \
        -H "content-type: application/json" \
        -d "{\"orderCode\":\"3211213\", \"orderStatus\":\"received\"}" \
        http://localhost:3000/publish
     ```
5. Publish the second event (type `order.changed.v1`) to trigger your Function.

- With the CloudEvents Conformance Tool, run:

```
cloudevents send http://localhost:3000/publish \
   --type order.changed.v1 \
   --id 94064655-7e9e-4795-97a3-81bfd497aac6 \
   --source myapp \
   --datacontenttype application/json \
   --data "{\"orderCode\":\"3211213\", \"orderStatus\":\"changed\"}" \
   --yaml
```

- With curl, run:

```
curl -v -X POST \
   -H "ce-specversion: 1.0" \
   -H "ce-type: order.changed.v1" \
   -H "ce-source: myapp" \
   -H "ce-eventtypeversion: v1" \
   -H "ce-id: 94064655-7e9e-4795-97a3-81bfd497aac6" \
   -H "content-type: application/json" \
   -d "{\"orderCode\":\"3211213\", \"orderStatus\":\"changed\"}" \
   http://localhost:3000/publish
```

6. To verify that the event was delivered, check the logs of the Function.
   - In Kyma dashboard, return to the view of your `lastorder` Function and go to *Code* > *Replicas of the Function*. Select your replica and under *Containers*, view the logs.
   - With kubectl, run:

```
kubectl logs \
-n default \
-l serverless.kyma-project.io/function-name=lastorder,serverless.kyma-
project.io/resource=deployment \
-c function
```

Results

You see the received event in the logs:

```
Received event: { orderCode: '3211213', orderStatus: 'received' }
Received event: { orderCode: '3211213', orderStatus: 'changed' }
```

4.3.1.7.5 Manage Subscriber Workload with Max-In-Flight

Manage your subscriber's workload by configuring the max-in-flight limit to control concurrent event delivery.

Prerequisites

- You have the Eventing module in your Kyma cluster.
- You have access to Kyma dashboard. Alternatively, if you prefer CLI, you need kubectl and curl.

- Optionally, you have the CloudEvents Conformance Tool for publishing events.
- You have an inline Function as event sink (see Create and Modify an Inline Function). Replace the default code sample with the following code. To simulate prolonged event processing, the Function waits for 5 seconds before returning the response.

```
module.exports = {
  main: async function (event, context) {
    console.log("Processing event:", event.data);
    // sleep/wait for 5 seconds
    await new Promise(r => setTimeout(r, 5 * 1000));
    console.log("Completely processed event:", event.data);
    return;
  }
}
```

> → Tip
>
> Use Istio sidecar proxies for reliability, observability, and security (see Istio Service Mesh [page 1889]).

Context

Your subscriber applications, such as Functions or microservices, have finite resources. To prevent overload, configure the `maxInFlightMessages` limit on your `Subscription` CRD. This limit controls the maximum number of events delivered to your subscriber without waiting for a response. The Eventing module resumes delivery after the subscriber acknowledges processing some of the existing in-flight events.

Procedure

1. Create a subscription that forwards a maximum of 5 events in parallel to the sink without waiting for a response.
   - In Kyma dashboard, find the default namespace, go to *Configuration* and create a subscription with the following parameters:
     - *Subscription name*: **`lastorder-sub`**
     - *Types*: **`order.received.v1`** and **`order.changed.v1`**
     - *Service*: **`lastorder`** (the sink is populated automatically)
     - *Type matching*: **`standard`**
     - *Source*: **`myapp`**
   - With kubectl, run:

```
cat <<EOF | kubectl apply -f -
  apiVersion: eventing.kyma-project.io/v1alpha2
  kind: Subscription
  metadata:
    name: lastorder-sub
    namespace: default
  spec:
    config:
      maxInFlightMessages: "5"
```

```
    sink: 'http://lastorder.default.svc.cluster.local'
    source: myapp
    types:
      - order.received.v1
EOF
```

2. Verify that the subscription is ready.
   - In Kyma dashboard, the status must be `READY`.
   - Alternatively, run `kubectl get subscriptions lastorder-sub -o=jsonpath="{.status.ready}"` and see if the response is `true`.
3. Port-forward the Eventing Publisher Proxy to `localhost`, using port 3000:

```
kubectl -n kyma-system port-forward service/eventing-publisher-proxy 3000:80
```

4. Publish 15 events at once and see how the Eventing module triggers the workload. In another terminal window, run:
   - With the CloudEvents Conformance Tool:

```
for i in {1..15}
do
  cloudevents send http://localhost:3000/publish \
    --type order.received.v1 \
    --id e4bcc616-c3a9-4840-9321-763aa23851f${i} \
    --source myapp \
    --datacontenttype application/json \
    --data "{\"orderCode\":\"$i\"}" \
    --yaml
done
```

   - With curl:

```
for i in {1..15}
do
  curl -v -X POST \
    -H "ce-specversion: 1.0" \
    -H "ce-type: order.received.v1" \
    -H "ce-source: myapp" \
    -H "ce-eventtypeversion: v1" \
    -H "ce-id: e4bcc616-c3a9-4840-9321-763aa23851f${i}" \
    -H "content-type: application/json" \
    -d "{\"orderCode\":\"$i\"}" \
    http://localhost:3000/publish
done
```

5. To verify that the event was delivered, check the logs of the Function.
   - In Kyma dashboard, return to the view of your `lastorder` Function and go to *Code* > *Replicas of the Function*. Select your replica and under *Containers*, view the logs.
   - With kubectl, run:

```
kubectl logs \
  -n default \
  -l serverless.kyma-project.io/function-name=lastorder,serverless.kyma-project.io/resource=deployment \
  -c function
```

Results

In the logs, you see that only 5 events were delivered to the Function in parallel. As soon as the Function completed the processing of the event and returns a response, the Eventing module delivers the next in-line event to the Function.

See the following example:

```
Processing event: { orderCode: '1' }
Processing event: { orderCode: '2' }
Processing event: { orderCode: '3' }
Processing event: { orderCode: '4' }
Processing event: { orderCode: '5' }
Completely processed event: { orderCode: '1' }
Processing event: { orderCode: '6' }
Completely processed event: { orderCode: '2' }
Processing event: { orderCode: '7' }
Completely processed event: { orderCode: '3' }
Processing event: { orderCode: '8' }
Completely processed event: { orderCode: '4' }
Processing event: { orderCode: '9' }
Completely processed event: { orderCode: '5' }
Processing event: { orderCode: '10' }
Completely processed event: { orderCode: '6' }
Processing event: { orderCode: '11' }
Completely processed event: { orderCode: '7' }
Processing event: { orderCode: '12' }
Completely processed event: { orderCode: '8' }
Processing event: { orderCode: '13' }
Completely processed event: { orderCode: '9' }
Processing event: { orderCode: '14' }
Completely processed event: { orderCode: '10' }
Processing event: { orderCode: '15' }
Completely processed event: { orderCode: '11' }
Completely processed event: { orderCode: '12' }
Completely processed event: { orderCode: '13' }
Completely processed event: { orderCode: '14' }
Completely processed event: { orderCode: '15' }
```

4.3.1.7.6 Troubleshooting for the Eventing Module

Troubleshoot problems related to the Eventing module.

- General Diagnostics: Event Not Delivered [page 2149]: Start here if your events are not reaching their destination or your `Subscription` is not `Ready`.
- Subscriber Receives Irrelevant Events [page 2151]: Use this guide if a subscriber receives events it did not subscribe to.
- NATS Backend Storage Is Full [page 2151]: Follow these steps if the Eventing Publisher Proxy returns a `507 Insufficient Storage` error.

For issues with the NATS cluster, see the NATS troubleshooting guides:

- General Diagnostics: NATS Module Readiness and Connectivity [page 2156]: Refer to this guide for issues related to the NATS module itself, such as Pod health or stream configuration.
- Published Events Are Pending in the Stream [page 2158]: Use this guide if events are stuck in the NATS stream and are not being delivered.

If you can't find a solution, don't hesitate to create a GitHub issue .

4.3.1.7.6.1 General Diagnostics: Event Not Delivered

Symptom

- You publish an event, but the subscriber doesn't receive it.
- A `Subscription` custom resource (CR) is not in the `Ready` state.
- You can't publish events, and the publisher receives an error.

Cause

This issue can have several causes, from misconfiguration of the Eventing module or `Subscription` to problems with the event payload or the subscriber workload.

Solution

Follow these steps to detect the source of the problem:

1. Verify That the Eventing Module Is Healthy

1. Check the state of the `Eventing` CR:

   ```
   kubectl get eventing -n kyma-system
   ```

2. In the output, look for `STATE: Ready`. If the state is not `Ready` (for example, `Warning` or `Error`), inspect the CR's status for detailed error messages:

   ```
   kubectl get eventing eventing -n kyma-system -o yaml
   ```

3. To understand the issue, review the `status.conditions` and `status.state`. For common states and recommended actions, see Eventing Module Status and Event Flow .

2. Check the Subscription Status

If the `Eventing` CR is ready, check the status of your `Subscription`.

1. Check if the `Subscription` is ready. Replace the placeholders with your values.

   ```
   kubectl get subscription {SUBSCRIPTION_NAME} -n {NAMESPACE}
   ```

2. In the output, look for `READY: true`. If it's `false`, inspect the `Subscription`'s status for details:

   ```
   kubectl get subscription {SUBSCRIPTION_NAME} -n {NAMESPACE} -o yaml
   ```

3. Examine the `status.conditions` array for error messages. A common issue is an invalid sink, which must be a valid, reachable HTTP endpoint (for example, `http://my-service.my-namespace.svc.cluster.local`).

3. Check the Publisher and Event Payload

If the `Subscription` CR is ready, verify that you are publishing the event correctly.

After sending an event, check the HTTP response code.

- `4xx` error: A client-side error indicates a problem with your request. Ensure that your event conforms to the CloudEvents specification or the legacy event format. For details, see Event Name Format [page 2139].
- `5xx` error: A server-side error indicates a problem with the Eventing Publisher Proxy. Check its logs for errors:

  ```
  kubectl logs -n kyma-system -l app.kubernetes.io/name=eventing-publisher-proxy
  ```

- `2xx` status: Verify that the event `type` attribute in your published event exactly matches one of the types defined in your `Subscription`'s `spec.types` field.

4. Check the Eventing Manager Logs

The Eventing Manager is responsible for dispatching events from the backend to the subscriber.

1. Check the logs for dispatch errors or other warnings:

   ```
   kubectl logs -n kyma-system -l app.kubernetes.io/name=eventing-manager
   ```

2. Look for logs related to your subscription. If you use the NATS backend, a successful dispatch log contains `"message":"event dispatched"`. If this log is missing, it could mean that the subscriber is unreachable or the NATS server is unavailable.

5. Check the Subscriber's Health

Ensure the subscriber application (the sink) is running and can receive HTTP requests.

1. Get the sink URL from your `Subscription`:

   ```
   kubectl get subscription {SUBSCRIPTION_NAME} -n {NAMESPACE} -o
   jsonpath='{.spec.sink}'
   ```

2. Run a temporary curl pod inside the cluster to test connectivity to the sink. Replace `<{SINK_URL}>` with the output from the previous command:

   ```
   # Run the test
   kubectl run sink-test --image=curlimages/curl --restart=Never -n {NAMESPACE}
   -- curl -v -X POST {SINK_URL}
   # Check the logs for the HTTP response
   kubectl logs sink-test -n {NAMESPACE}
   # Clean up the pod
   kubectl delete pod sink-test -n {NAMESPACE}
   ```

If the response code is not `2xx`, the problem is with your subscriber application. Check its logs to diagnose the issue.

6. Check the NATS Backend

If all previous steps show no errors, the issue may be within the NATS backend itself. Continue with General Diagnostics: NATS Module Readiness and Connectivity [page 2156].

4.3.1.7.6.2 Subscriber Receives Irrelevant Events

Symptom

A subscriber receives events that don't match the types defined in its `Subscription` custom resource.

Cause

This issue occurs due to a naming collision after event type cleanup. To conform to Cloud Event specifications, Eventing modifies the event names to filter out prohibited characters. For details, see Event Name Cleanup [page 2139].

Solution

1. To find the cleaned event type that the backend uses, check the status of the Subscription:

   ```
   kubectl get subscription {SUBSCRIPTION_NAME} -n {NAMESPACE} -o
   jsonpath='{.status.types}'
   ```

   This command returns the mapping between the original type and the cleaned type. Note the `cleanType` value.
2. Search all Subscription resources in the cluster to see if another one uses the same cleaned type. Replace `<{CLEAN_TYPE}>` with the value from the previous step.

   ```
   kubectl get subscriptions -A -o yaml | grep "cleanType: {CLEAN_TYPE}" -B 1
   ```

   If the output shows more than one `Subscription` with the same `cleanType`, you have a naming collision.
3. To fix the issue, modify the `spec.types` in one of the colliding `Subscription` resources and update the corresponding publisher to use a unique event type that doesn't cause a collision.

4.3.1.7.6.3 NATS Backend Storage Is Full

Symptom

- When you publish an event, the Eventing Publisher Proxy returns status `507 Insufficient Storage`.
- The Eventing Publisher Proxy logs show the error: `cannot send to stream: nats: maximum bytes exceeded`.

Cause

In Kyma, the default retention policy for NATS JetStream is Interest . This retention policy keeps messages in the stream if they can't be delivered to the sink, as long as there are consumers in the stream that match the published event's subject.

If there are too many undelivered events, the message backlog grows until it fills the available storage. To prevent event loss, the backend stops receiving events, and no further events can be persisted to the stream.

This often happens if you delete a subscriber's deployment without first deleting the corresponding `Subscription`.

Solution

To free up storage and resume event flow, choose one of the following solutions:

- Temporarily reduce the events' publish rate until the events are delivered.
- If the subscriber is no longer needed, delete its `Subscription` resource. This action removes the NATS consumer and discards all pending messages for that subscription, freeing up space. The `Interest` retention policy specifies that events published to the subject are not kept in the stream if they don't match any consumer filter.
- If a subscriber is unhealthy or slow, fix the underlying issue in the subscriber's application. When the subscriber is healthy and starts acknowledging messages, the NATS stream begins to clear.

To diagnose the problem, inspect the NATS JetStream status (see General Diagnostics: NATS Module Readiness and Connectivity [page 2156]) and verify that the subscriber's sink is healthy and reachable (see General Diagnostics: Event Not Delivered [page 2149].

- If the events' publish rate is very high (more than 1.5k events per second), the subscriber may not be able to keep up. Speed up the event dispatching by increasing the `maxInFlightMessages` configuration of the `Subscription` (default is set to 10).
- As a long-term solution, scale the NATS backend with additional replicas.

4.3.1.8 NATS Module

Learn more about the NATS Module. Use it to manage and configure the message-oriented middleware, NATS.

What Is NATS?

NATS is an infrastructure that enables the exchange of data in the form of messages. One feature of NATS is JetStream. JetStream is a distributed persistence system that provides more functionalities and higher qualities of service on top of `Core NATS`.

The NATS module includes the NATS Manager, which manages the lifecycle of a NATS JetStream deployment. It observes the state of the NATS cluster and reconciles it with the desired state.

For more information about NATS and NATS JetStream, see the official NATS documentation .

The Eventing Module [page 2136] can use NATS as a backend to process events and send them to subscribers.

> ⚠ Caution
>
> Reaching the NATS storage size limits may cause the Eventing module to stop receiving events or delay them. For more information on the troubleshooting, see NATS Backend Storage Is Full [page 2151].

Features

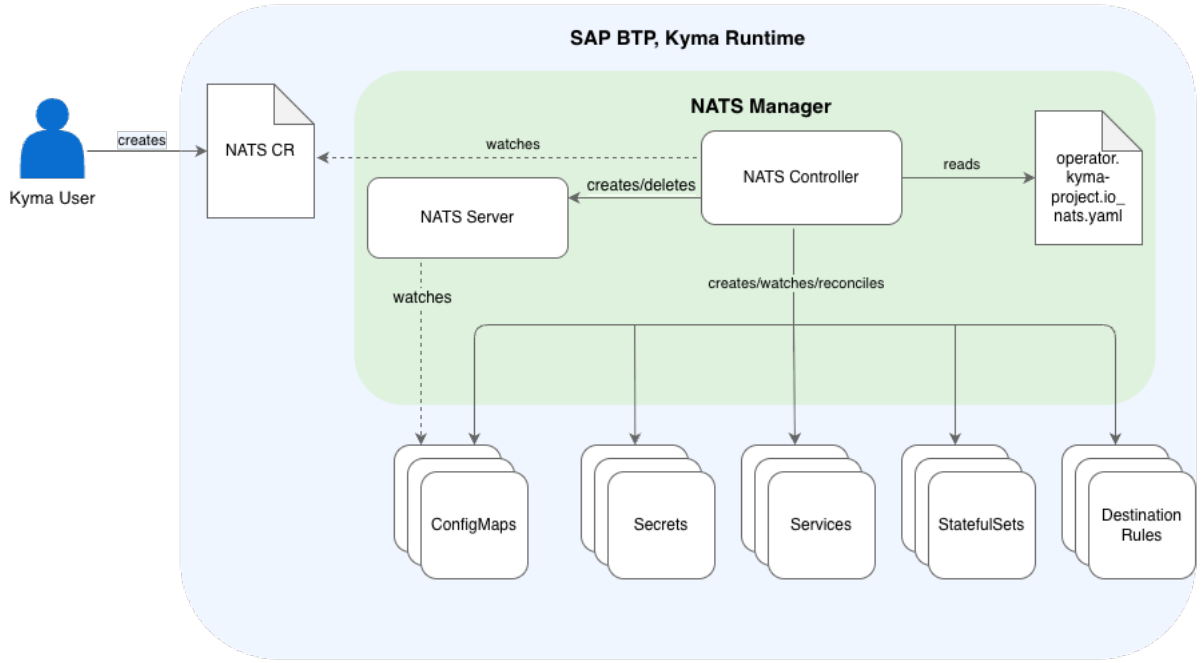
- Automated NATS JetStream Deployment: Deploys a production-ready NATS JetStream cluster without manual setup.
- Persistent Messaging: Use file-based storage to ensure messages are retained even if a Pod restarts. Memory-based storage is available for higher throughput scenarios.
- Declarative Configuration: Manage your NATS cluster configuration, including cluster size and storage options, through a simple Kubernetes custom resource.
- Configurable Resource Allocation: Define specific CPU and memory requests and limits for the NATS Pods to fit your cluster's capacity.
- Seamless integration with the Eventing module.

High Availability

For high availability, set up NATS servers across different availability zones for uninterrupted operation and uptime. NATS Manager deploys the NATS servers in the availability zones where your Kubernetes cluster has Nodes. If the Kubernetes cluster has Nodes distributed across at least three availability zones, NATS Manager automatically distributes the NATS servers across these availability zones. If the Kubernetes cluster doesn't have Nodes distributed across at least three availability zones, high availability is compromised.

Architecture

The NATS module uses a Kubernetes operator -based architecture.



1. You create a NATS CR.
2. The NATS Manager starts the Controller, which creates, watches, and reconciles the relevant resources.
3. The Controller reacts to changes in the NATS CR to adapt the resources to the desired state.
4. The Controller creates or deletes the NATS server.

NATS Manager

The NATS Manager is responsible for starting the Controller which creates, watches, and reconciles the relevant resources:

- ConfigMaps
- Secrets
- Services
- StatefulSets
- DestinationRules

API/Custom Resource Definitions

The `nats.operator.kyma-project.io` CustomResourceDefinition (CRD) describes the NATS CR that NATS Manager uses to manage the module. See NATS Custom Resource.

Authorization

To assign access permissions to the NATS module resources, use the following aggregated ClusterRoles :

- `kyma-nats-view` - Grants read-only access to the NATS custom resources and their statuses.
- `kyma-nats-edit` - Grants full access to the NATS custom resources and read-only access to their statuses.

Resource Consumption

To learn more about the resources used by the NATS module, see NATS [page 3520].

4.3.1.8.1 Accessing the NATS Server Using CLI

Interact directly with your NATS server deployment in Kyma using the NATS command-line interface (CLI). Use the CLI to inspect server status, manage streams and consumers, and troubleshoot message flow.

Prerequisites

- You have the NATS module in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- You have installed kubectl and NATS CLI .

Context

Accessing certain resources in NATS requires system account privileges . Kyma automatically generates a `system account` user using a Secret named `eventing-nats-secret` in the `kyma-system` namespace.

Procedure

1. To get the credentials, run:

   ```
   kubectl get secrets -n kyma-system eventing-nats-secret -ogo-
   template='{{index .data "resolver.conf"|base64decode}}'| grep 'user:'| tr -d
   '{}'
   ```

   If you changed the default NATS instance name from `eventing-nats`, replace `eventing-nats-secret` with `{your_NATS_CR_name}-secret`.

You receive the credentials for the `system account` user in the following format:

```
user: admin, password: <your password>
```

2. To access the NATS server with the NATS CLI tool, forward its port:

```
kubectl port-forward -n kyma-system svc/eventing-nats 4222
```

3. To send your NATS commands, pass the credentials:

```
nats server info --user admin --password <your password>
```

4.3.1.8.2 Troubleshooting for the NATS Module

Troubleshoot problems related to the NATS module.

- General Diagnostics: NATS Module Readiness and Connectivity [page 2156]: Refer to this guide for issues related to the NATS module itself, such as Pod health or stream configuration.
- Published Events Are Pending in the Stream [page 2158]: Use this guide if events are stuck in the NATS stream and are not being delivered.

For issues with the Eventing module, see the Eventing troubleshooting guides:

- General Diagnostics: Event Not Delivered [page 2149]: Start here if your events are not reaching their destination or your Subscription is not Ready.
- Subscriber Receives Irrelevant Events [page 2151]: Use this guide if a subscriber receives events it did not subscribe to.
- NATS Backend Storage Is Full [page 2151]: Follow these steps if the Eventing Publisher Proxy returns a 507 Insufficient Storage error.

If you can't find a solution, don't hesitate to create a GitHub issue .

General Diagnostics: NATS Module Readiness and Connectivity [page 2156]

Published Events Are Pending in the Stream [page 2158]

4.3.1.8.2.1 General Diagnostics: NATS Module Readiness and Connectivity

Symptom

- The NATS module is not in a `Ready` state.
- The Eventing module reports that it cannot connect to the NATS backend.
- NATS Pods are in a `CrashLoopBackOff` or `Pending` state.

Cause

Issues with the NATS module can stem from misconfigurations in the NATS custom resource (CR), problems with the underlying Kubernetes Nodes, or storage issues with Persistent Volume Claims (PVCs).

Solution

1. Check the NATS CR Status

1. To verify the health of the NATS cluster, check the NATS CR.

```
kubectl get nats -n kyma-system
```

2. Look for `STATE: Ready`. If the state is `Error` or `Processing`, inspect the CR for detailed error messages.

```
kubectl get nats {NATS_CR_NAME} -n kyma-system -o yaml
```

3. Review the `status.conditions` to understand the root cause.

2. Check the NATS Pods

1. Ensure all NATS Pods are running correctly.

```
kubectl get pods -n kyma-system -l nats_cluster=eventing-nats
```

2. If any Pods are not in the `Running` state, use `kubectl describe pod` and `kubectl logs` to investigate further.

3. Check the Persistent Volume Claims (PVCs)

1. If you use file storage, a common issue is a problem with the PVCs.

```
kubectl get pvc -n kyma-system -l nats_cluster=eventing-nats
```

2. Check the `STATUS` column. If a PVC is `Pending`, there may be no available Persistent Volume that satisfies its request. If it is `Bound`, check if it is full by referring to the NATS Backend Storage Is Full guide.

4. Inspect the NATS JetStream

If the cluster appears healthy, you can inspect the JetStream components directly using the NATS CLI .

1. Ensure that you have access to the NATS server (see Accessing the NATS Server Using CLI).
2. Port-forward to a NATS Pod.

```
kubectl -n kyma-system port-forward svc/eventing-nats 4222
```

3. Verify that the `kyma` stream exists.

```
      $ nats stream ls

╭──────────────────────────────────────────────────────────────────────────────────────────╮
│                                                                                          │
│                                         Streams                                          │
├──────┬──────┬─────────────┬─────────────────────┬──────────┬──────┬──────────────────────┤
│      │ Name │ Description │ Created             │ Messages │ Size │ Last Message         │
```

```
├─────┼──────────┼─────────────────────┼──────────┼───────┼──────────┤
  | sap  |          | 2022-05-03 00:00:00 | 0        | 318 B |
5.80s       |
└─────┴──────────┴─────────────────────┴──────────┴───────┴──────────┘
```

4. If the stream exists, check the timestamp of the `Last Message` that the stream received. A recent timestamp would mean that the event was published correctly.
5. Check if the consumers were created and have the expected configurations.

   ```
   nats consumer info
   ```

   To correlate the consumer to the Subscription and the specific event type, check the `description` field of the consumer.
6. If the PVC storage is fully consumed and matches the stream size as shown above, the stream can no longer receive messages. Either increase the PVC storage size (see NATS Backend Storage Is Full [page 2151]) or set the `MaxBytes` property which removes the old messages.

4.3.1.8.2.2 Published Events Are Pending in the Stream

Symptom

You publish events, but some of them are not received by the subscriber and stay pending in the stream.

Cause

When the NATS `EventingBackend` has more than one replica, and the `Clustering` property on the NATS Server is enabled, a leader is elected for each stream and consumer (see NATS: JetStream Clustering). When the leader is elected, all the messages are replicated across the replicas.

Sometimes replicas can go out of sync with the other replicas. As a result, messages on some consumers can stop being acknowledged and start piling up in the stream.

Solution

1. Trigger Consumer Leader Reelection

To fix the "broken" consumers with pending messages, trigger a leader reelection. You can do this either on the consumers that have pending messages, or if that fails, on the stream level.

You need the latest version of NATS CLI and access to the NATS server (see Accessing the NATS Server Using CLI [page 2155]).

1. Port forward to a NATS replica.

   ```
   kubectl port-forward -n kyma-system eventing-nats-0 4222
   ```

2. Run this Shell script:

```
for consumer in $(nats consumer list -n sap) #  sap is the stream name
do
  nats consumer info sap $consumer -j | jq -c '{name: .name,
pending: .num_pending, leader: .cluster.leader}'
done
```

   The output shows each consumer, its pending message count, and its current leader Pod.

```
{"name":"ebcabfe5c902612f0ba3ebde7653f30b","pending":25,"leader":"eventing-
nats-1"}
{"name":"c74c20756af53b592f87edebff67bdf8","pending":0,"leader":"eventing-
nats-0"}
```

3. Note the name of the consumer with a non-zero pending count and the name of its leader (in this example, `ebcabfe5c902612f0ba3ebde7653f30b`).
4. Port-forward to the identified leader Pod.

```
kubectl port-forward -n kyma-system eventing-nats-1 4222
```

5. Trigger the leader reelection for that broken consumer.

```
nats consumer cluster step-down sap ebcabfe5c902612f0ba3ebde7653f30b
```

   After execution, you see a message like the following:

```
New leader elected "eventing-nats-2"
Information for Consumer sap > ebcabfe5c902612f0ba3ebde7653f30b created
2022-10-24T15:49:43+02:00
```

6. Check the consumer and confirm that the pending messages start being dispatched.

2. Stream Leader Reelection

Sometimes triggering the leader reelection on the broken consumers doesn't work. In that case, you must restart the NATS Pods to trigger leader reelection on the stream level.

Command the stream to step down.

```
nats stream cluster step-down sap
```

Result:

You should get a result similar to the following example:

```
11:08:22 Requesting leader step down of "eventing-nats-1"in a 3 peer RAFT group
11:08:23 New leader elected "eventing-nats-0"
Information for Stream sap created 2022-10-24 15:47:19
             Subjects: kyma.>
             Replicas: 3
              Storage: File
```

3. Restart the NATS Pods

If none of the previous steps work, perform a restart of the NATS Pods.

```
# assuming we have 3 NATS instances
$ kubectl delete pod -n kyma-system eventing-nats-0 eventing-nats-1 eventing-
nats-2 --wait=false
```

4.3.1.9 Registry Cache Module

The Registry Cache module adds a caching layer for container image registries in SAP BTP, Kyma runtime instances.

What Is Registry Cache?

Registry Cache reduces outbound traffic to upstream registries, improving image pull performance. With Registry Cache, you can also cache images from private registries. To do so, provide credentials for the caching layer to use when authenticating against those registries.

The Registry Cache module is built on top of the Gardener's registry cache extension. See Configuring the Registry Cache Extension .

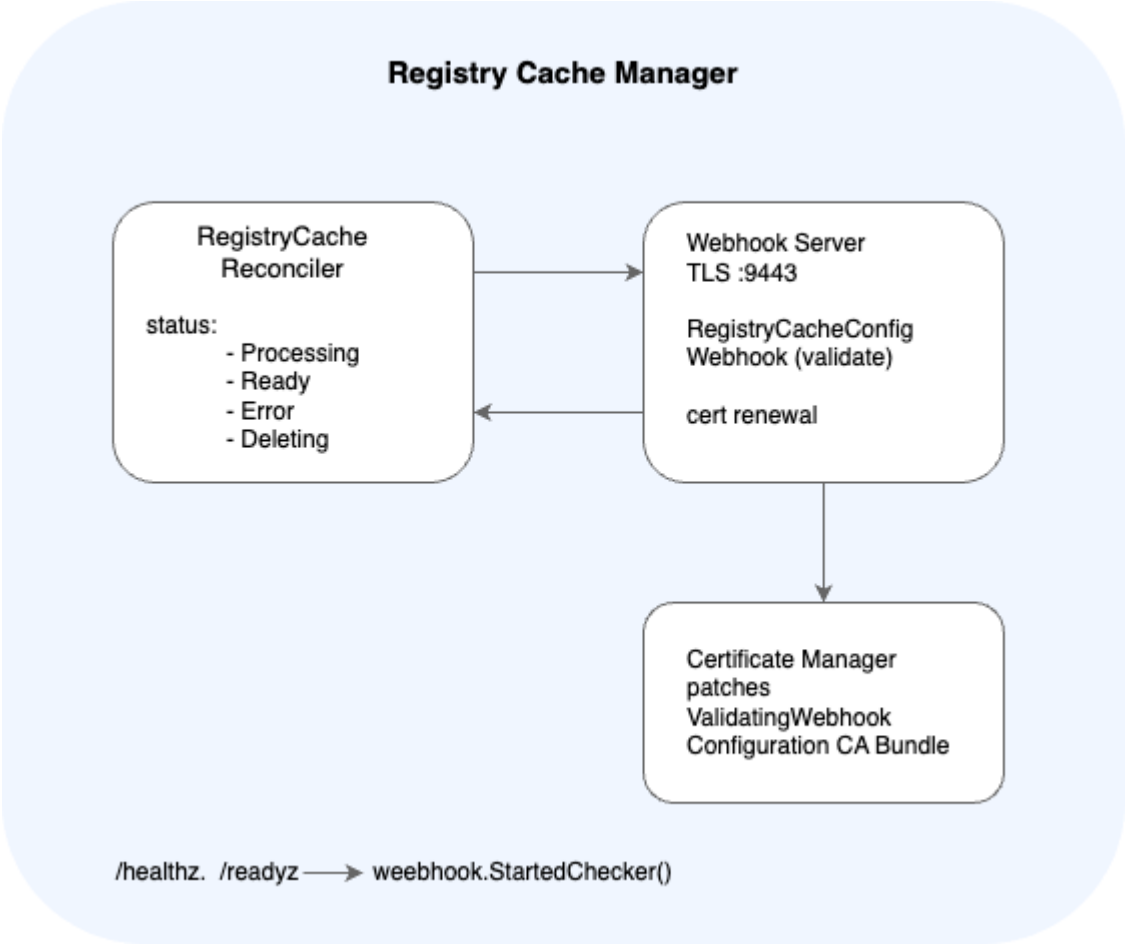
Features

The Registry Cache module provides the following features:

- Caches container images from upstream registries to reduce outbound network traffic.
- Supports private registries using credential Secrets referenced in `RegistryCacheConfig`.
- Configurable cache volume size and storage class per upstream registry.
- Configurable garbage collection Time to Live (TTL); you can disable garbage collection.
- Proxy support for HTTP and HTTPS connections used by the cache.
- TLS-enabled HTTP server for the Registry Cache endpoint.

Architecture

The Registry Cache module consists of two main runtime components: the `RegistryCache` controller and the `RegistryCacheConfig` admission webhook. Both run in the same Registry Cache Manager process.



- `RegistryCache` controller - reconciles `RegistryCache` custom resources (CRs) and drives status transitions.
- Webhook Server - TLS server on port 9443 that validates `RegistryCacheConfig` resources on creation and update.
- Certificate Manager - watches TLS certificate files and rotates the CA bundle in `ValidatingWebhookConfiguration` on renewal.

API / Custom Resource Definitions

The Registry Cache module defines the following custom resources:

Custom Resource Definitions

CRD	Scope	Description
`RegistryCache`	Namespaced	Module CR managed by the lifecycle infrastructure. Tracks the installation health of the Registry Cache module. See RegistryCache Custom Resource .
`RegistryCacheConfig`	Namespaced	User-created CR that configures a caching layer for a specific upstream container image registry. See RegistryCacheConfig Custom Resource .

4.3.1.9.1 Configuring Registry Cache

Create a `RegistryCacheConfig` custom resource (CR) to enable caching for an upstream container image registry in your SAP BTP, Kyma runtime instance.

Prerequisites

- You have a Kyma instance created.
- You have administrative access to the Kyma runtime with kubeconfig and kubectl. See Access a Kyma Instance Using kubectl.
- You have the Registry Cache module added. See Adding and Deleting a Kyma Module.

Context

`RegistryCacheConfig` is a namespace-scoped resource. You can create it in any namespace.

Procedure

1. Create the `test` namespace if it doesn't exist.

   ```
   kubectl create namespace test
   ```

2. Create a `RegistryCacheConfig` CR.

   ```
   kubectl create -f - <<EOF
   apiVersion: core.kyma-project.io/v1beta1
   kind: RegistryCacheConfig
   metadata:
     name: config1
     namespace: test
   spec:
     upstream: docker.io
     volume:
       size: 100Gi
   EOF
   ```

   When applied, Kyma Control Plane (KCP) processes the resource and configures a caching layer for the specified upstream registry (in this case, `docker.io`). The `volume.size` field specifies the size of the persistent volume used to store cached images.

   You can create multiple `RegistryCacheConfig` resources to cache different upstream registries. Each resource must have a unique name, and each upstream registry must be unique across all resources in the cluster.

3. To verify that KCP processed the resource successfully, check its status.

   ```
   kubectl get registrycacheconfig <name> -n <namespace> -o
   jsonpath='{.status.state}'
   ```

   The expected output values are the following:

   - Pending - KCP is processing the configuration.
   - Ready - the caching layer has been configured successfully.
   - Error - KCP encountered an issue and is retrying. The state transitions to `Ready` automatically when processing succeeds.

4.3.1.9.2 Providing Credentials for a Private Upstream Registry

Create a Kubernetes Secret and reference it in a `RegistryCacheConfig` resource to enable Registry Cache to authenticate against a private upstream registry.

Context

If the upstream registry requires authentication, create a Kubernetes Secret in the same namespace as the `RegistryCacheConfig` resource and reference it in the `spec.secretReferenceName` field. The Secret must be immutable and of type `generic`.

Procedure

1. Set environment variables with the upstream registry credentials.

```
export USERNAME=<your username>
export PASSWORD=<your password>
```

2. Create the `test` namespace if it doesn't exist.

```
kubectl create namespace test
```

3. To create the credential Secret, use the procedure that matches your registry type.

   - For registries other than Google Artifact Registry, create an immutable Secret named `rc-secret` in the `test` namespace. The credential Secret must exist in the cluster before applying the `RegistryCacheConfig` resource.

     ```
     kubectl create -f - <<EOF
     apiVersion: v1
     kind: Secret
     metadata:
       name: rc-secret
       namespace: test
     type: Opaque
     immutable: true
     data:
       username: $(echo -n $USERNAME | base64 | tr -d '\n')
       password: $(echo -n $PASSWORD | base64 | tr -d '\n')
     EOF
     ```

   - For Google Artifact Registry , the username is `_json_key` and the password is the service account key in JSON format.

     1. Base64-encode the service account key.

        ```
        export PASSWORD=$(echo -nE $SERVICE_ACCOUNT_KEY_JSON | base64 | tr -d
        '\n')
        ```

     2. Create an immutable Secret with the encoded key as the password.

        ```
        kubectl create -f - <<EOF
        apiVersion: v1
        kind: Secret
        metadata:
          name: rc-secret
          namespace: test
        type: Opaque
        immutable: true
        data:
          username: $(echo -n "_json_key" | base64 | tr -d '\n')
          password: $PASSWORD
        EOF
        ```

4. Apply the Registry Cache configuration referencing the created Secret.

```
kubectl create -f - <<EOF
apiVersion: core.kyma-project.io/v1beta1
kind: RegistryCacheConfig
metadata:
  name: config2
  namespace: test
spec:
  upstream: <protected registry URL>
```

```
  secretReferenceName: rc-secret
  volume:
    size: 100Gi
EOF
```

> **Note**
>
> When using a private registry, store the same credentials in the following Kubernetes Secrets:
>
> - The Secret referenced in `spec.secretReferenceName`, which Registry Cache uses to authenticate against the upstream registry when pulling images to cache.
> - An `imagePullSecret` on each workload, which containerd uses to authenticate directly against the upstream registry as a fallback when Registry Cache is unavailable.
>
> Do not remove the `imagePullSecret` from your workloads when configuring credentials for Registry Cache. If the cache is unavailable, containerd falls back to the upstream registry and requires the credentials directly.

4.3.1.9.3 Validation of Registry Cache Configuration

Learn how the Registry Cache webhook validates `RegistryCacheConfig` custom resources (CRs), how Kyma Control Plane (KCP) processes them, and what validation rules apply to each field.

When you apply a `RegistryCacheConfig` resource, the Registry Cache webhook validates the configuration on the Kyma runtime side before the Kubernetes API accepts it. If the configuration is invalid, the API rejects the request and returns an error - no custom resource is created.

Example error message:

```
admission webhook "registrycacheconfig-v1beta1.kb.io" denied the request:
spec.upstream: Invalid value: "dockerrrrr.io": upstream is not DNS resolvable
```

If the CR is accepted, KCP processes it. The status transitions from Pending to Ready on success, or to Error if KCP-side processing fails. For details, check `status.conditions`.

KCP periodically reconciles `RegistryCacheConfig` resources. During reconciliation, a CR in Ready state transitions back to Pending and then returns to Ready once reconciliation completes. This is expected behavior.

The following table describes the validation rules for each field.

Validation Rules

Field	Validation
`spec.upstream`	Must be a valid DNS-resolvable host (no scheme). Must be unique across all `RegistryCacheConfig` resources in the cluster. Port, if specified, must be in the range 1–65535.
`spec.remoteURL`	Must have the format `<scheme><host>[:<port>]` where `<scheme>` is `https://` or `http://` and `<host>[:<port>]` corresponds to the upstream. Must be DNS resolvable.

Field	Validation
`spec.secretReferenceName`	The referenced Secret must exist in the same namespace as the `RegistryCacheConfig` resource, be immutable, and contain exactly the `username` and `password` data keys.
`spec.volume.size`	Must be a positive value in a format recognized by Go's `resource.Quantity` (for example, `10Gi`). Immutable after creation.
`spec.volume.storageClassName`	The referenced storage class must be available. Immutable after creation.
`spec.garbageCollection.ttl`	Must be in a format recognized by Go's `time.ParseDuration` (for example, `24h`). Set to `0s` to disable garbage collection. Cannot be re-enabled once disabled.
`spec.proxy.httpProxy`	Must be a valid URL starting with `http://` or `https://`.
`spec.proxy.httpsProxy`	Must be a valid URL starting with `http://` or `https://`.
`spec.http.tls`	Must be a valid boolean indicating whether TLS is enabled.

4.3.1.9.4 Managing Registry Cache Configuration

List and delete `RegistryCacheConfig` resources in your cluster.

Procedure

- To list all `RegistryCacheConfig` resources across all namespaces, run:

  ```
  kubectl get registrycacheconfig -A
  ```

- To list resources in a specific namespace, run:

  ```
  kubectl get registrycacheconfig -n <namespace>
  ```

- To delete a `RegistryCacheConfig` resource, run:

  ```
  kubectl delete registrycacheconfig <name> -n <namespace>
  ```

  For example:

  ```
  kubectl delete registrycacheconfig config -n test
  ```

4.3.1.9.5 Rotating Credentials

Update the credentials used by Registry Cache to authenticate against a private upstream registry.

Context

Credential Secrets are immutable and cannot be updated in place.

Procedure

1. Create a new Secret with the updated credentials. Use a different name (for example, `rc-secret-v2`).

```
kubectl create -f - <<EOF
apiVersion: v1
kind: Secret
metadata:
  name: rc-secret-v2
  namespace: <namespace>
type: Opaque
immutable: true
data:
  username: $(echo -n $USERNAME | base64 | tr -d '\n')
  password: $(echo -n $PASSWORD | base64 | tr -d '\n')
EOF
```

2. To reference the new Secret, update `spec.secretReferenceName` in the existing `RegistryCacheConfig` resource.

```
kubectl patch registrycacheconfig <name> -n <namespace> \
  --type=merge -p '{"spec":{"secretReferenceName":"rc-secret-v2"}}'
```

3. When the `RegistryCacheConfig` is in Ready state, delete the old Secret.

```
kubectl delete secret rc-secret -n <namespace>
```

4.3.1.9.6 Troubleshooting for the Registry Cache Module

Troubleshoot problems related to the Registry Cache module.

Registry Cache Does Not Cache Images from a Private Registry [page 2168]
Diagnose and resolve a misconfigured credential Secret that prevents Registry Cache from caching images from a private upstream registry.

4.3.1.9.6.1 Registry Cache Does Not Cache Images from a Private Registry

Diagnose and resolve a misconfigured credential Secret that prevents Registry Cache from caching images from a private upstream registry.

Symptom

You configured Registry Cache with credentials for a private upstream registry. Image pulls in your workloads still succeed, but you suspect or observe that images are not being served from the cache.

Cause

When Registry Cache credentials are incorrect, the registry cache Pod in the `kube-system` namespace receives an error response from the upstream registry. Depending on the upstream registry, the error may be indistinguishable from a missing image at the log level - for example, JFrog Artifactory returns `404` instead of an authentication error. Meanwhile, image pulls in workloads continue to succeed using the `imagePullSecret` fallback, so the misconfiguration is not immediately visible.

> ⓘ Note
>
> Registry Cache is designed not to impair operations if its configuration is incorrect. If you have configured an `imagePullSecret` on your workloads (recommended), image pulls still succeed using direct fallback to the upstream registry even when the Registry Cache credentials are incorrect. This means misconfigured credentials are not immediately visible. To verify that the cache is working correctly, check the registry cache Pod logs as described in this document.

Solution

The Gardener extension creates the registry cache Pods in `kube-system`. They are named after the upstream registry host they cache.

1. List the Registry Cache Pods for the affected upstream.

   ```
   kubectl get pods -n kube-system | grep registry-<upstream-host>
   ```

2. Check the logs of the relevant Pod.

   ```
   kubectl logs -n kube-system <pod-name> --tail=50
   ```

   A pull failure due to incorrect credentials produces log output similar to the following:

   ```
   level=error msg="response completed with error" err.code="manifest
   unknown" err.detail="unknown tag=<tag>" err.message="manifest unknown" ...
   http.response.status=404
   ```

3. If you see this pattern repeating, verify that the credentials in the Secret referenced by `spec.secretReferenceName` are correct and that the Secret is up to date.
   To learn how to update the credentials, see Rotating Credentials [page 2167].

4.3.1.10 Serverless Module

Learn more about the Serverless module. Use it to run lightweight Functions in a cost-efficient and scalable way using JavaScript and Node.js.

What Is Serverless?

The Kyma Serverless module is a "code-as-a-service" tool designed to accelerate the process of prototyping applications by removing the technical overhead necessary to obtain the first testable versions. It allows developers to focus on the coding and easily run it in the context of the SAP BTP without dealing with Kubernetes orchestration or containerization complexities. The fact that no Kubernetes knowledge is required to use it makes Serverless a useful tool for beginners who want to experiment with loosely coupled microservice architecture on Kubernetes or for business consultants who need to provide extension logic for an existing Kubernetes-based system with minimal overhead.

With the Serverless module, you can build, run, and manage serverless applications in Kubernetes. These applications are called Functions, and they are based on the Function custom resource (CR) objects. They contain simple code snippets that implement a specific business logic. For example, you can define that you want to use a Function as a proxy that saves all incoming event details to an external database.

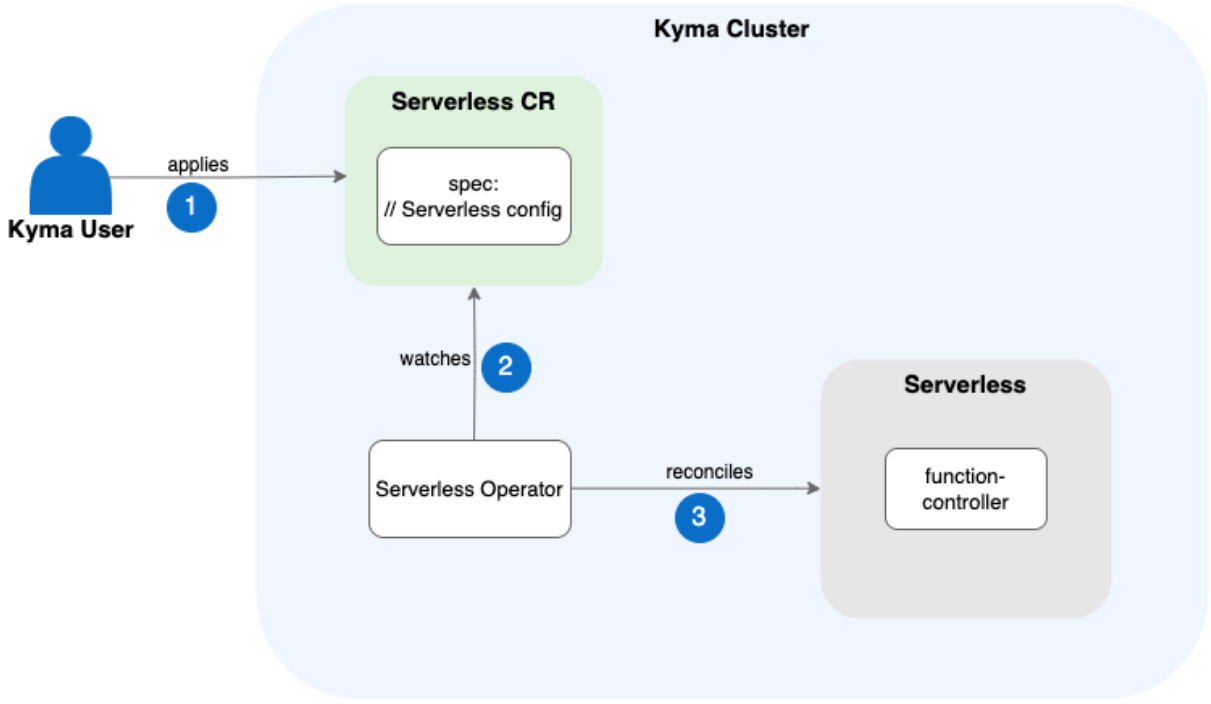
A Function can be triggered by other workloads in the cluster (in-cluster events) or business events coming from external sources. You can subscribe to them using a Subscription CR . You can expose your Function to an external endpoint (HTTPS). With an APIRule CR , you can define who can reach the endpoint and what operations they can perform on it.

Features

The Serverless module provides the following features:

- Quick deployments following a Function approach
- Loosely coupled microservice approach
- Event-driven programming model
- Reduced implementation and operation effort
- Function management with Kubernetes resources like Deployments , Services , and HorizontalPodAutoscalers

Architecture



1. User applies the Serverless CR.
2. Serverless Operator watches the Serverless CR.
3. Serverless Operator reconciles the Serverless workloads.

Serverless Operator

When you add the Serverless module, Serverless Operator takes care of installation and configuration of Serverless in your cluster. It manages the Serverless lifecycle based on the dedicated Serverless CR.

API/Custom Resource Definitions

The API of the Serverless module is based on Kubernetes CustomResourceDefinitions (CRDs), which extend the Kubernetes API with custom additions. To inspect the specification of the Serverless module API, see:

- Function CRD
- Serverless CRD

Security Considerations

To learn how to avoid any potential threats while using Functions, see Function Security [page 3657].

Authorization

To assign access permissions to the Serverless module resources, use the following aggregated ClusterRoles :

- `kyma-serverless-view` - Grants read-only access to the Serverless custom resource.
- `kyma-serverless-edit` - Grants full access to the Serverless custom resource.
- `kyma-functions-view` - Grants read-only access to the Function custom resources.
- `kyma-functions-edit` - Grants full access to the Function custom resources.

Resource Consumption

To learn more about the resources used by the Serverless module, see Serverless [page 3520].

Related Information

Function Security [page 3657]
kyma-project.io: Serverless module tutorials
kyma-project.io: Serverless module technical reference
Troubleshooting for the Serverless Module [page 2181]

4.3.1.10.1 From Code to Function

Pick the programming language for your Function and decide where you want to keep the source code. The Serverless module uses it to create the workload.

Serverless Runtimes

Functions support programming languages by using the underlying execution environments known as runtimes. You can create both Node.js and Python Functions in Kyma.

> → Tip
>
> See sample Functions for each available runtime.

Source Code

You can also choose where you want to keep your Function's source code and dependencies. You can either place them directly in the Function CR under the `spec.source` and `spec.deps` fields as an inline Function, or store the code and dependencies in a public or private Git repository (Git Functions). Choosing the second option ensures your Function is versioned and gives you more development freedom in the choice of a project structure or an IDE.

> → Tip
>
> Read more about Git Functions .

4.3.1.10.2 Serverless Limitations

Controller Limitations

Function Controller does not serve time-critical requests from users. It reconciles Function custom resources (CRs) stored at the Kubernetes API Server, and has no persistent state on its own.

Function Controller doesn't serve Functions using its allocated runtime resources. It delegates this work to the dedicated Kubernetes workloads. Refer to the architecture diagram for more details.

Having this in mind, also remember that Function Controller does not require horizontal scaling. It scales vertically up to `1Gi` of memory and `500m` of CPU time.

Namespace Setup Limitations

Be aware that if you apply LimitRanges in the target namespace where you create Functions, the limits also apply to the Function workloads and may prevent Functions from being run. In such cases, ensure that resources requested in the Function configuration are lower than the limits applied in the namespace.

Limitation for the Number of Functions

There is no upper limit of Functions that you can run on Kyma. Once you define a Function, its runtime Pods are always requested by Function Controller. It's up to Kubernetes to schedule them based on the available memory and CPU time on the Kubernetes worker nodes. This is determined mainly by the number of the Kubernetes worker nodes (and the node auto-scaling capabilities) and their computational capacity.

Runtime Phase Limitations

> **Note**
>
> All measurements were taken on Kubernetes with three Azure worker nodes of type Standard_D2s_v5 (two vCPU amd64 cores, ~8 GiB memory), distributed across availability zones westeurope-1, westeurope-2, and westeurope-3, running Garden Linux 1877.10 with kernel `6.12.66-cloud-amd64` and Kubernetes `v1.34.3`.
>
> The values in the following tables are averages from three test runs. Last updated: 2026-04-02.

Functions serve user-provided logic wrapped in the web framework, Express for Node.js and Bottle for Python. Taking the user logic aside, those frameworks have limitations and depend on the selected runtime profile and the Kubernetes nodes specification.

The following tables present the response times of the selected runtime profiles for a "Hello World" Function across three load scenarios. This describes the overhead of the serving framework itself. Any user logic added on top of that adds extra milliseconds and must be profiled separately.

Tests are implemented using k6 and consist of the following scenarios:

- **Constant load** — 50 virtual users send one request per second each (with a 1-second sleep between calls) for 2 minutes. Represents a steady, moderate traffic baseline.
- **Max load** — 100 virtual users send requests as fast as possible (no sleep) for 2 minutes. Represents sustained high concurrency.

Constant Load

Node.js 22

response time [ms]	XS	S	M	L	XL
median	1.7	1.4	1.3	0.9	1.2
95 percentile	4.9	4.4	4.4	3.7	4.8
99 percentile	93	61	22	12	12

Node.js 24

response time [ms]	XS	S	M	L	XL
median	1.5	1.6	1.3	1.3	1.8
95 percentile	3.7	4.1	3.4	3.2	4.1
99 percentile	13	12	7.2	5.4	7.7

Python 3.12

response time [ms]	XS	S	M	L	XL
median	2.5	2.4	3.7	3.4	3.7
95 percentile	16	7.6	9.8	9.2	9.4

response time [ms]	XS	S	M	L	XL
99 percentile	147	47	21	30	20

Max Load

Node.js 22

response time [ms]	XS	S	M	L	XL
median	104	97	50	18	15
95 percentile	300	204	104	57	28
99 percentile	390	293	156	69	40

Node.js 24

response time [ms]	XS	S	M	L	XL
median	86	12	8.2	6.8	6.3
95 percentile	100	93	65	23	17
99 percentile	191	102	73	31	27

Python 3.12

response time [ms]	XS	S	M	L	XL
median	902	699	384	137	119
95 percentile	1157	846	397	213	163
99 percentile	1540	1100	585	300	228

The bigger the runtime profile, the more resources are available to serve the response quicker. Consider these limits of the serving layer as a baseline because this does not take your Function logic into account.

Scaling

Function runtime Pods can be scaled horizontally from zero up to the limits of the available resources at the Kubernetes worker nodes. See the Use External Scalers tutorial for more information.

4.3.1.10.3 Configuring Serverless

By default, the Serverless module comes with the default configuration. You can change the configuration using the Serverless CustomResourceDefinition (CRD), which manages Serverless custom resource (CR).

Prerequisites

- You have the Serverless module added. See Adding and Deleting a Kyma Module [page 3511].
- You have access to Kyma dashboard. Alternatively, to use CLI instructions, you must install kubectl.

Context

The Serverless CR is an API to configure the Serverless module. You can use it to perform the following actions:

- Configuring the trace endpoint.
- Configuring the Eventing endpoint.
- Configuring the Function requeue duration.
- Configuring the healthz liveness timeout.
- Configuring the default runtime Pod preset.
- Configuring the log level.
- Configuring the log format.

The default configuration of the Serverless module is the following:

```
apiVersion: operator.kyma-project.io/v1alpha1
kind: Serverless
metadata:
  name: serverless-sample
spec: {}
```

Procedure

1. Go to Kyma dashboard. The URL is in the Overview section of your subaccount.
2. Choose *Modify Modules*, and in the *View* tab, choose *serverless*.
3. Go to *Edit*, and provide your configuration changes. You can use the *Form* or *YAML* tab.

Configuring the Trace Endpoint

By default, the Serverless operator checks if there is a trace endpoint available. If available, the detected trace endpoint is used as the trace collector URL in Functions. If no trace endpoint is detected, Functions are

configured with no trace collector endpoint. You can configure a custom trace endpoint so that Function traces are sent to any tracing backend you choose. The currently used trace endpoint is visible in the Serverless CR status.

```
spec:
  tracing:
    endpoint: http://jaeger-collector.observability.svc.cluster.local:4318/v1/
traces
```

Configuring the Eventing Endpoint

You can configure a custom Eventing endpoint to publish events sent from your Functions. The currently used trace endpoint is visible in the Serverless CR status. By default `http://eventing-publisher-proxy.kyma-system.svc.cluster.local/publish` is used.

```
spec:
  eventing:
    endpoint: http://eventing-publisher-proxy.kyma-system.svc.cluster.local/
publish
```

Configuring the Function Requeue Duration

By default, the Function associated with the default configuration will be requeued every 5 minutes.

```
   spec:
      functionRequeueDuration: 5m
```

Configuring the healthz Liveness Timeout

By default, Function is considered unhealthy if the liveness health check endpoint does not respond within 10 seconds.

```
   spec:
      healthzLivenessTimeout: "10s"
```

Configuring the Default Runtime Pod Preset

You can configure the default runtime Pod preset to be used.

```
   spec:
      defaultRuntimePodPreset: "M"
```

For more information on presets, see Available Presets .

Configuring the Log Level

You can configure the desired log level to be used.

```
spec:
   logLevel: "debug"
```

Configuring the Log Format

You can configure the desired log format to be used.

```
spec:
   logFormat: "json"
```

For more details, see Configuring Logging [page 2178].

4.3.1.10.4 Serverless Buildless Mode

Learn how to accelerate your development with Serverless buildless mode.

From the beginning, Kyma Serverless has aimed to accelerate the development of fast prototypes by allowing users to focus on business logic rather than containerization and Kubernetes deployment. Our goal is to remove operational barriers so developers can iterate quickly and efficiently.

With the introduction of buildless mode, we significantly shortened the feedback loop during prototype development by eliminating the image build step in Kyma runtime. In buildless mode, instead of building and pushing custom Function images into the in-cluster registry, your code and dependencies are mounted into Kyma-provided runtime images. This approach positions Kyma Serverless as a more efficient development tool, enabling even faster iteration. Additionally, it eliminates the architectural complexities and limitations of deploying Serverless Functions on Kubernetes.

Features

- **Simplified architecture**: By separating the Docker Registry into its own module, the Serverless module is now more lightweight and easier to manage.
- **Faster deployment**: The removal of the build job reduces the time required to deploy Functions.
- **Dynamic dependency resolution**: Dependencies are resolved at runtime, allowing for more flexibility in managing library versions.
- **Improved flexibility**: Injecting Function code into the runtime Pod simplifies the deployment process and reduces image management overhead.
- **Reduced resource consumption**: Eliminating build jobs means Serverless no longer requires computational resources from worker nodes for image building.

- **Enhanced security**: By removing build jobs, Functions can run in namespaces with more restrictive Pod security levels enabled.

Outcomes of Switching to Buildless Mode

- Your Serverless module no longer includes an internal Docker Registry.
- Function builds are eliminated. Your Functions use a base image that mounts dependencies dynamically at runtime.
- Function dependencies are downloaded each time a Function Pod starts. This means different replicas of the same Function may use different dependency versions if you don't pin exact versions.
- Your Function code is injected directly into runtime Pods without requiring pre-built container images.

Using Fixed Dependency Versions

- **Avoid using `latest` versions of Function dependencies**: Since dependencies are resolved at the Function's Pod start time in buildless mode, using `latest` versions can lead to inconsistencies between replicas of the same Function. This may be the case when the dependency provider releases a new version after one replica is already running and before another replica is created due to auto-scaling. Always specify exact versions of dependencies to ensure stability and predictability.
- **Dependency resolution behavior**: Be aware that each replica of a Function may resolve and use a different dependency version if the version is not explicitly pinned.

Disabling Buildless Mode

To learn how to disable Serverless buildless mode, see Configuring Serverless [page 2175].

4.3.1.10.5 Configuring Logging

Learn how to configure logging for the Serverless components.

Prerequisites

You have kubectl installed.

Supported Log Levels

From the least to the most verbose: `fatal`, `panic`, `dpanic`, `error`, `warn`, `info` (default), `debug`.

Supported Log Formats

The supported log formats are the following:

- `json` - Structured JSON format (default)
- `console` - Human-readable console format

Configuring Serverless Controller

To update the log configuration, modify the Serverless custom resource (CR) in the `kyma-system` namespace.

> ### Note
>
> The `logLevel` changes are applied dynamically without restarting the Serverless Pods. The `logFormat` changes automatically restart the Serverless Pods to apply the new format.

To change the log level, run:

```
kubectl patch serverless -n kyma-system default --type merge -p '{"spec":
{"logLevel":"debug"}}'
```

To change the log format, run:

```
kubectl patch serverless -n kyma-system default --type merge -p '{"spec":
{"logFormat":"console"}}'
```

To change the log level and format, run:

```
kubectl patch serverless -n kyma-system default --type merge -p '{"spec":
{"logLevel":"debug","logFormat":"console"}}'
```

To verify the change, run:

```
kubectl logs -n kyma-system -l app.kubernetes.io/name=serverless
```

Configuring Serverless Operator

Update the log configuration in the `serverless-operator-log-config` ConfigMap in the `kyma-system` namespace.

> **Note**
>
> You can't dynamically change the log format for the Serverless Operator. If you want to change it, update the ConfigMap and restart the Pods.

To change the log level, run:

```
kubectl patch configmap serverless-operator-log-config -n kyma-system --type
merge -p '{"data":{"log-config.yaml":"logLevel: debug"}}'
```

To change the log level and format, run:

```
kubectl patch configmap serverless-operator-log-config -n kyma-system --type
merge -p '{"data":{"log-config.yaml":"logLevel: debug\nlogFormat: console"}}'
```

To verify the change, run:

```
kubectl logs -n kyma-system -l control-plane=operator
```

4.3.1.10.6 Function Runtime Deprecation Schedule

This document outlines the planned deprecation and end-of-life (EOL) dates for supported Function runtimes in Kyma Serverless.

Supported Runtimes and Deprecation Timeline

Runtime	Planned Deprecation	Estimated EOL	Status
Node.js 22	July 2026	November 2026	Deprecated
Node.js 24	TBD	TBD	
Node.js 26	TBD	TBD	
Python 3.12	September 2026	March 2027	
Python 3.14	TBD	TBD	

Deprecation History

Node.js 22

`Status`: Deprecated

`Deprecation Version`: `v1.14.0`

Node.js 22 is deprecated. For more information, see #2681 . Migrate to Node.js 24 or newer.

Node.js 20

`Status`: Removed

`Deprecation Version`: `v1.10.0`

`Removal Version`: `v1.14.0`

Node.js 20 reached end-of-life. For more information, see #2231 and #2682 . Migrate to Node.js 22 or newer.

> **Note**
>
> The deprecation and EOL dates listed in this document are predictions based on current release cadence and Node.js/Python LTS schedules. These dates are subject to change and may be adjusted based on the following:
>
> - Changes in the Kyma Serverless release schedule
> - Updates to upstream Node.js and Python LTS timelines
> - Community feedback and requirements
> - Security considerations
>
> Always check the release notes for announcements regarding runtime deprecations and EOL timelines.

Recommendations

- Plan upgrades to newer runtimes well in advance of deprecation dates.
- Monitor release notes for any changes to this schedule.
- For Functions using deprecated runtimes, migrate before the EOL date to avoid service disruption.

4.3.1.10.7 Troubleshooting for the Serverless Module

Troubleshoot problems related to the Serverless module.

Serverless Periodically Restarting [page 2181]

Function Fails to Start With Customized OpenTelemetry Dependencies [page 2182]

Git Source Certificate Signed by Unknown Authority [page 2183]

4.3.1.10.7.1 Serverless Periodically Restarting

Symptom

When reconciling Git-sourced Functions, Serverless restarts every 10 minutes.

Cause

Function Controller is polling for changes in referenced Git repositories. If you have a lot of Git-sourced Functions and they were deployed together at approximately the same time, their git sources will be checked out in a synchronized pulse (every 10 minutes). If you happen to reference large repositories (multi-repositories), there will be rhythmical, high demand on CPU and I/O resources necessary to check out repositories. This may cause Function Controller to crash and restart.

Solution

Avoid using multi-repositories or large repositories to source your git Functions. Using small, dedicated Function repositories decreases the CPU and I/O resources used to check out Git sources, and hence improves the stability of Function Controller.

4.3.1.10.7.2 Function Fails to Start With Customized OpenTelemetry Dependencies

Symptom

After customizing OpenTelemetry dependencies, the Serverless Function is not starting.

Cause

You can configure your own dependencies as part of the Function's dependencies. While this allows customization, it also introduces the risk of dependency conflicts, especially with OpenTelemetry packages, which have strict version requirements for their dependencies. For example, `opentelemetry-instrumentation` requires specific versions of `opentelemetry-api` and `opentelemetry-sdk`. A version mismatch can lead to runtime errors. Downgrading or upgrading one package without aligning the others can lead to runtime errors or import issues.

Solution

Avoid downgrading OpenTelemetry packages in your Function's configuration. Use the dependencies provided by the Serverless module for Python and Node.js. These dependencies are regularly maintained and upgraded. They are also tested to ensure compatibility.

If you must customize OpenTelemetry dependencies, ensure that all related packages (e.g., `opentelemetry-api`, `opentelemetry-sdk`, `opentelemetry-instrumentation`) are aligned to compatible versions. Refer to the PyPi and npm pages to verify compatibility of the desired version.

4.3.1.10.7.3 Git Source Certificate Signed by Unknown Authority

Symptom

When using a Git repository as a Function source over HTTPS, the Function status shows a condition with reason `SourceUpdateFailed` and a message similar to:

```
certificate signed by unknown authority
```

The Function remains in a non-ready state and its source code cannot be fetched or updated.

Cause

The Serverless manager cannot verify the TLS certificate of the Git repository server because the certificate is signed by a Certificate Authority (CA) that is not trusted in the cluster.

This typically happens when:

- The Git server uses a certificate signed by a private or internal CA (for example, an on-premise GitLab or GitHub Enterprise instance).
- The cluster's root CA bundle does not include the CA that signed the Git server's certificate.
- The cluster or landscape root CAs are managed externally and are not under the control of the Serverless module.

Solution

The root CA bundle used by the cluster must include the CA that signed your Git server's certificate. This is a cluster-level or landscape-level configuration — it cannot be adjusted within the Serverless module itself.

Depending on your environment, take one of the following steps:

- For SAP BTP, Kyma runtime, contact your landscape administrator to ensure the required root CA is included in the cluster's trusted CA bundle.
- For a self-managed cluster, add the CA certificate to the cluster's system trust store or configure the relevant node/Pod CA bundle to include the missing CA. Refer to your Kubernetes distribution documentation for the exact procedure.

After the CA bundle is updated, the Serverless manager automatically retries fetching the Git source, and the Function status should return to `Ready`.

4.3.1.11 Telemetry Module

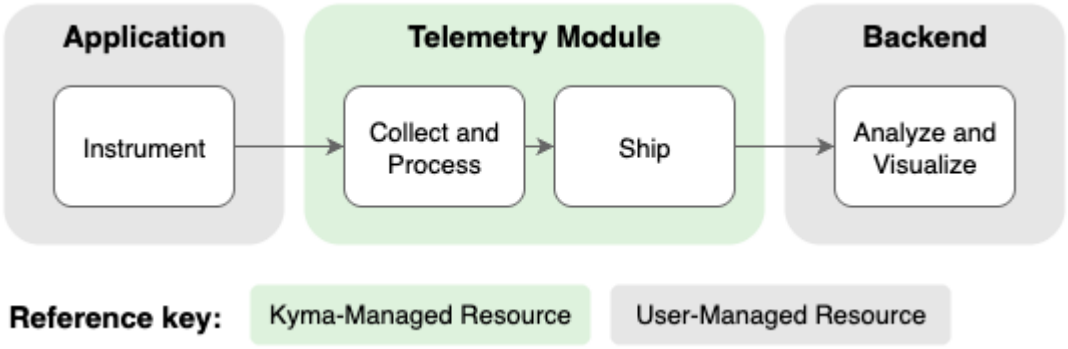
Use the Telemetry module to collect telemetry signals (logs, traces, and metrics) from your applications and send them to your preferred observability backend.

What is Telemetry?

With telemetry signals, you can understand the behavior and health of your applications and infrastructure. The Telemetry module provides a standardized way to collect these signals and send them to your observability backend, where you can analyze them and troubleshoot issues.

The Telemetry module processes three types of signals:

- Logs: Time-stamped records of events that happen over time.
- Traces: The path of a request as it travels through your application's components.
- Metrics: Aggregated numerical data about the performance or state of a component over time.



Telemetry signals flow through the following stages:

1. You instrument your application so that its components expose telemetry signals.
2. The signals are collected and enriched with infrastructural metadata.
3. You send the enriched signals to your preferred observability backend.
4. The backend stores your data, where you can analyze and visualize it.

The Telemetry module focuses on the collection, processing, and shipment stages of the observability workflow. It offers a vendor-neutral approach based on OpenTelemetry and doesn't force you into a specific backend. This means you can integrate with your existing observability platforms or choose from a wide range of available backends that best suit your operational needs.

> → Tip
>
> Build your first telemetry pipeline with the hands-on lesson Collecting Application Logs and Shipping them to SAP Cloud Logging.

Features

To support telemetry for your applications, the Telemetry module provides the following features:

- **Consistent Telemetry Pipeline API**: Use a streamlined set of APIs based on the OTel Collector to collect, filter, and ship your logs, metrics, and traces (see Telemetry Pipeline API [page 2187]). You define a pipeline for each signal type to control how the data is processed and where it's sent. For details, see Collecting Logs [page 2194], Collecting Traces [page 2199], and Collecting Metrics [page 2203].
- **Flexible Backend Integration**: The Telemetry module is optimized for integration with SAP BTP observability services, such as SAP Cloud Logging. You can also send data to any backend that supports the OpenTelemetry protocol (OTLP), giving you the freedom to choose your preferred solution (see Integrate With Your OTLP Backend [page 2231]).

  > → Recommendation
  >
  > For production deployments, we recommend using a central telemetry solution located outside your cluster. For an example, see Integrate With SAP Cloud Logging [page 2235].
  >
  > For testing or development, in-cluster solutions may be suitable. For examples such as Dynatrace (or to learn how to collect data from applications based on the OpenTelemetry Demo App), see Integration Guides.

- **Seamless Istio Integration**: The Telemetry module seamlessly integrates with the Istio module when both are present in your cluster. For details, see Istio Integration [page 2263].
- **Custom Filtering and Processing**: You can drop any unnecessary Telemetry data right when it's collected (see Filter Logs [page 2214], Filter Traces [page 2217], and Filter Metrics [page 2218]). For advanced data manipulation, such as redacting sensitive information or creating complex filters, you can use the OpenTelemetry Transformation Language (see Transform and Filter with OTTL [page 2220]).
- **Automatic Data Enrichment**: The Telemetry module adds resource attributes as metadata, following OTel semantic conventions. This makes your data more consistent, meaningful, and ready for analysis in your observability backend. For details, see Automatic Data Enrichment [page 2228].
- **Instrumentation Guidance**: To generate telemetry data, you must instrument your code. Based on Open Telemetry (OTel), you get community samples on how to instrument your code using the Open Telemetry SDKs in most programming languages.
- **Simplified Permission Management**: To manage access to Telemetry resources, you can use the module's aggregated ClusterRoles (see Authorization [page 2186]).

Scope

The Telemetry module focuses only on the signals of application logs, distributed traces, and metrics. Other kinds of signals are not considered. Also, audit logs are not in scope.

Supported integration scenarios are neutral to the vendor of the target system.

Architecture

The Telemetry module is built around a central controller, Telemetry Manager, which dynamically configures and deploys data collection components based on your pipeline resources.

To understand how the core components interact, see Telemetry Architecture [page 2252].

To learn how this model applies to each signal type, see:

- Logs Architecture [page 2256]
- Traces Architecture [page 2258]
- Metrics Architecture [page 2260]

API/Custom Resource Definitions

You configure the Telemetry module and its pipelines by creating and applying Kubernetes CustomResourceDefinitions (CRDs), which extend the Kubernetes API with custom additions.

To understand and configure the module's global settings, refer to the Telemetry CRD .

To define how to collect, process, and ship a specific signal, use the pipeline CRDs:

- LogPipeline CRD
- TracePipeline CRD
- MetricPipeline CRD

Authorization

To assign access permissions to the Telemetry module resources, use the following aggregated ClusterRoles :

- `kyma-telemetry-view` - Grants read-only access to the Telemetry custom resources.
- `kyma-telemetry-edit` - Grants write access to the Telemetry custom resources

Resource Consumption

To learn more about the resources used by the Telemetry module, see Kyma Modules' Sizing [page 3518].

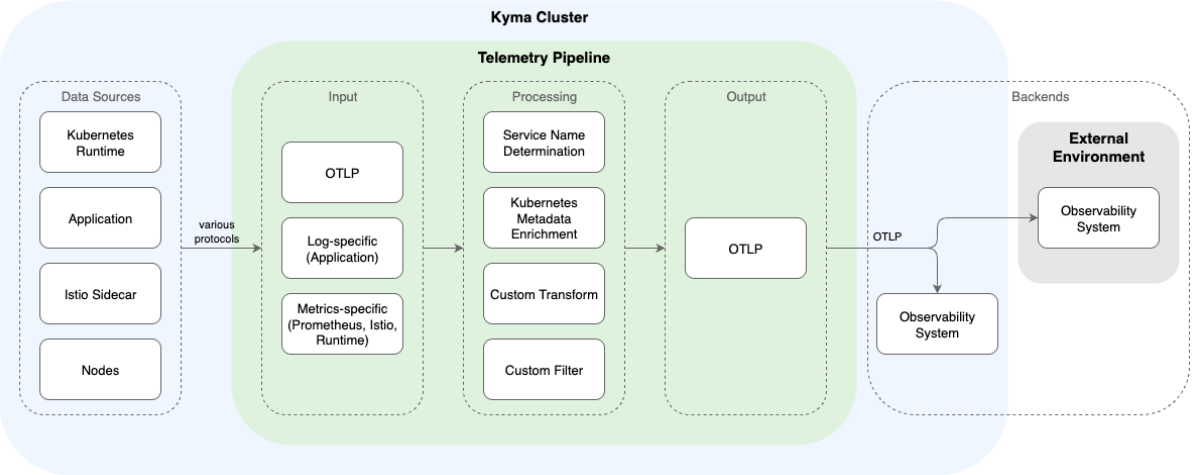
4.3.1.11.1 Telemetry Pipeline API

To collect and export telemetry data from your Kyma cluster, you define one or more pipelines for each signal type (logs, traces, metrics). You choose which data to collect and to which backend it's sent.

Pipeline Structure

You define Telemetry pipelines using three dedicated Kubernetes CRDs that extend the Kubernetes API: `LogPipeline`, `TracePipeline`, and `MetricPipeline`.

A pipeline defines how the data flows from the original data sources through the respective inputs and a series of processing steps to the backend you defined as output.



The pipelines use the OpenTelemetry Protocol (OTLP) as the primary method to ingest and export data, which gives you the flexibility to integrate with a wide range of observability backends.

While each pipeline is tailored to a specific signal, they all share a common structure:

Sample Code

```
apiVersion: telemetry.kyma-project.io/v1beta1
kind: <LogPipeline | TracePipeline | MetricPipeline>     # Choose pipeline
kind depending on signal type
metadata:
  name: my-observability-backend
spec:
  input:                # Enable additional inputs depending on signal type
    otlp:
      ...
  output:
    otlp:               # Integrate with your observability backend
      endpoint:
      ...
```

Pipeline Types

The `kind` attribute in the CRD specifies the type of telemetry data that the pipeline handles.

- `LogPipeline`: Collects logs from your application containers' standard output (`stdout`) and standard error (`stderr`), and from OTLP sources. It parses these logs, extracts useful information, and forwards them to your configured backend.
- `TracePipeline`: Collects trace data from OTLP sources, which show how requests flow between different components of your application. To collect Istio traces generated by applications or the service mesh, you must enable tracing within Istio using the Istio `Telemetry` CRD.
- `MetricPipeline`: Collects metrics from OTLP sources, Prometheus-annotated workloads, the Istio service mesh, and the Kubernetes runtime.

Input

In the `spec.input` section, you define the sources of your telemetry data. This section is the primary difference between the pipeline types.

All pipelines share `otlp` as the default input and can be configured with additional, signal-specific inputs:

By default, the `otlp` input is enabled for all signal types, which provisions a cluster-internal endpoint accepting OTLP data. For details, see Set Up the OTLP Input [page 2192].

Additionally, you can apply specific `input` configurations for each signal type:

- `LogPipeline`: The `runtime` input is enabled by default. Additionally, you can collect Istio access logs through the default `otlp` input. For both inputs, you can restrict from which Kubernetes resources you want to collect signals. For details, see Configure Application Logs [page 2196] and Configure Istio Access Logs [page 2197].
- `TracePipeline`: Tracing is a push-based model, so `otlp` is the only available input. The pipeline's OTLP endpoint receives span data pushed from your applications and Istio proxies. For Istio tracing, you can configure the sampling rate and apply individual settings to namespaces or workloads (see Configure Istio Tracing [page 2201]).
- `MetricPipeline`: You can select which metrics are collected by enabling inputs: `prometheus` (for scraping annotated workloads), `runtime` (for Kubernetes resource metrics), and `istio` (for service mesh metrics). You can filter all inputs by namespace. For details, see Collect Prometheus Metrics [page 2206], Collect Istio Metrics [page 2209], and Collect Runtime Metrics [page 2210].

Filtering and Processing

You can control the volume and focus of your incoming telemetry data by filtering it based on Kubernetes resources like namespaces, containers, and workloads. For details, see Filter Logs [page 2214], Filter Traces [page 2217], and Filter Metrics [page 2218].

Furthermore, you can transform and filter the received data by defining custom rules with the OpenTelemetry Transformation Language (OTTL). For details, see Transform and Filter with OTTL [page 2220].

All pipelines automatically enrich telemetry data with Kubernetes resource attributes, such as Pod name, namespace, and labels. With this context information, you can easily identify the source of telemetry data in your backend. For details, see Automatic Data Enrichment [page 2228].

Output

In the `spec.output` section, you define the destination for your telemetry data. Each pipeline resource supports exactly one output, which sends data using OTLP.

You must specify the endpoint address of your observability backend. You can also configure the protocol (gRPC or HTTP) and the authentication details required to connect securely. For details, see Integrate With Your OTLP Backend [page 2231].

To send the same signal to multiple backends, create a separate pipeline resource for each destination. For details, see Route Specific Inputs to Different Backends [page 2193].

4.3.1.11.1.1 Migrate Telemetry Pipelines to v1beta1

The Telemetry module now uses the stable v1beta1 API for pipeline resources. In some cases, you must update your pipeline manifests to use the new API version.

Prerequisites

- You have the Telemetry module in your cluster.
- One or more of your Telemetry pipelines use the v1alpha1 API version.

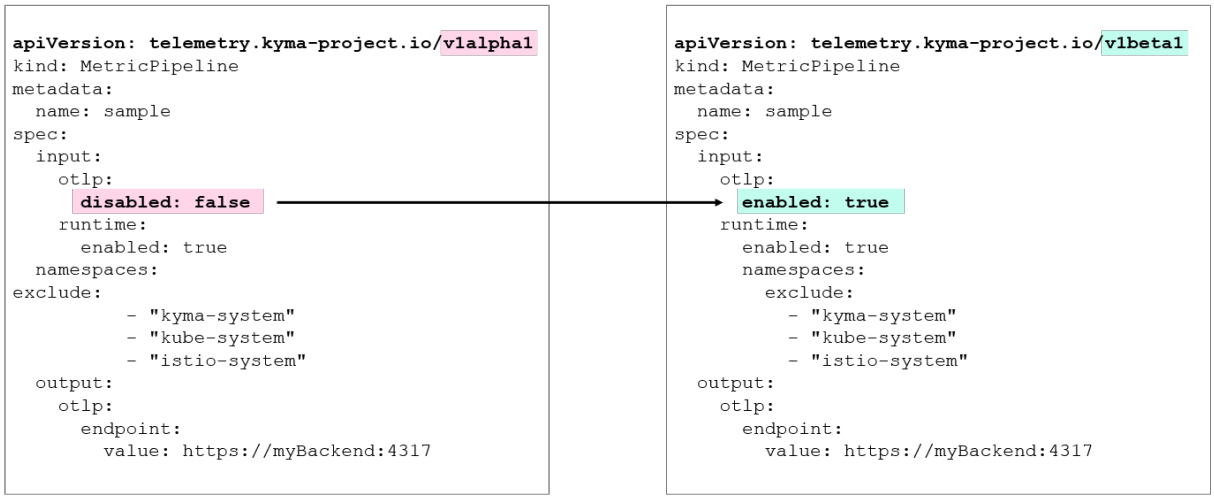
Context

> **Note**
>
> If your pipelines are already running in the cluster and you don't store their manifests, you don't have to take any action. The Telemetry module automatically migrates your existing pipeline resources to v1beta1.
>
> To see what your converted pipeline looks like in v1beta1, fetch the resource directly from your cluster:
>
> ```
> kubectl get logpipeline {PIPELINE_NAME} -o yaml
> kubectl get metricpipeline {PIPELINE_NAME} -o yaml
> kubectl get tracepipeline {PIPELINE_NAME} -o yaml
> ```

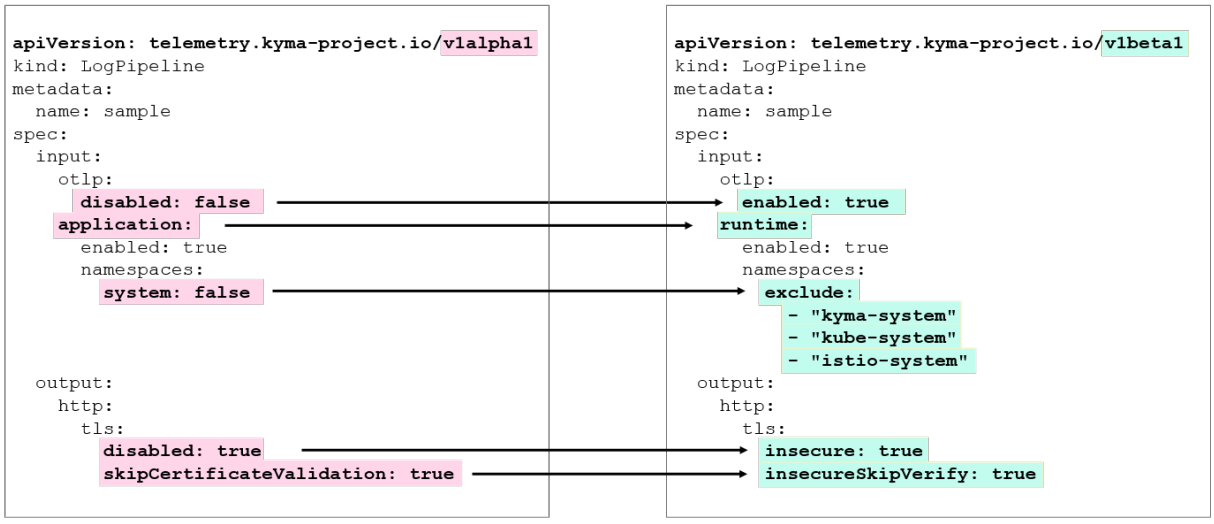
If you manage your Telemetry pipeline resources declaratively, you must manually adjust them; for example, in the following cases:

- You use GitOps, CI/CD, or tools like Argo CD to deploy your pipelines.
- You store pipeline manifests in a Git repository that you reapply.
- You plan to create new pipelines and want to use the v1beta1 API.

This migration involves breaking changes: To align your pipeline manifests with the new version, you must update the `apiVersion`, rename several fields, and, for `LogPipeline` resources, adjust how you configure namespace selection. The migration from v1alpha1 to v1beta1 mostly affects `LogPipeline` and `MetricPipeline` resources. For `TracePipeline` resources, only the `apiVersion` changes.



MetricPipeline Migration to v1beta1



LogPipeline Migration to v1beta1

To identify the required updates for your manifests, review the breaking changes in the following table:

Pipeline	v1alpha1 Field	v1beta1 Field	Migration Action
`LogPipeline`, `MetricPipeline`	`spec.input.otlp.disabled`	`spec.input.otlp.enabled`	Rename the field and invert the boolean value (for example, false becomes true).
`LogPipeline`	`spec.input.application`	`spec.input.runtime`	Rename the field.
`LogPipeline`	`spec.input.application.namespaces.system`	(Removed)	To include system namespaces, use `spec.input.runtime.namespaces: {}`.
`LogPipeline`	`spec.output.http.tls.disabled`	`spec.output.http.tls.insecure`	Rename the field.
`LogPipeline`	`spec.output.http.tls.skipCertificateValidation`	`spec.output.http.tls.insecureSkipVerify`	Rename the field.

Procedure

1. In each of your `LogPipeline`, `MetricPipeline`, and `TracePipeline` YAML files, change the `apiVersion` to telemetry.kyma-project.io/v1beta1.
2. For `LogPipeline` and `MetricPipeline` resources, update the OTLP input:
   a. Replace `spec.input.otlp.disabled` with `spec.input.otlp.enabled`.
   b. Invert the boolean value (for example, `disabled: false` becomes `enabled: true`).
3. For `LogPipeline` resources, replace `spec.input.application` with `spec.input.runtime`. To include system namespaces for application logs, also update the system namespace selection.

   By default, system namespaces are excluded (as in v1alpha1), but v1beta1 removes the `spec.input.application.namespaces.system` field. To include application logs from system namespaces (like `kyma-system`), you must now provide an empty object (`{}`) for the `namespaces` selector. For details, see Filter Application Logs by Namespace [page 2215].

   ```
   spec:
    input:
      runtime:
        enabled: true
        namespaces: {}  # This includes system namespaces
   ```

4. For `LogPipeline` resources using the `http` output, update the following fields:
   a. Replace `spec.output.http.tls.disabled` with `spec.output.http.tls.insecure`.
   b. Replace `spec.output.http.tls.skipCertificateValidation` with `spec.output.http.tls.insecureSkipVerify`.

> → Recommendation
>
> Consider switching to the `otlp` output (see Migrate Your LogPipeline From HTTP to OTLP [page 2241]). For OTLP, use the fields `spec.output.otlp.tls.insecure` and `spec.output.otlp.tls.insecureSkipVerify`.

5. Validate and apply your updated configuration with kubectl.

Results

Your pipelines are now updated to the v1beta1 API. The Telemetry module begins using the new configuration.

To confirm the migration was successful, check that your new pipelines are healthy (see Check Pipeline Status [page 2243]).

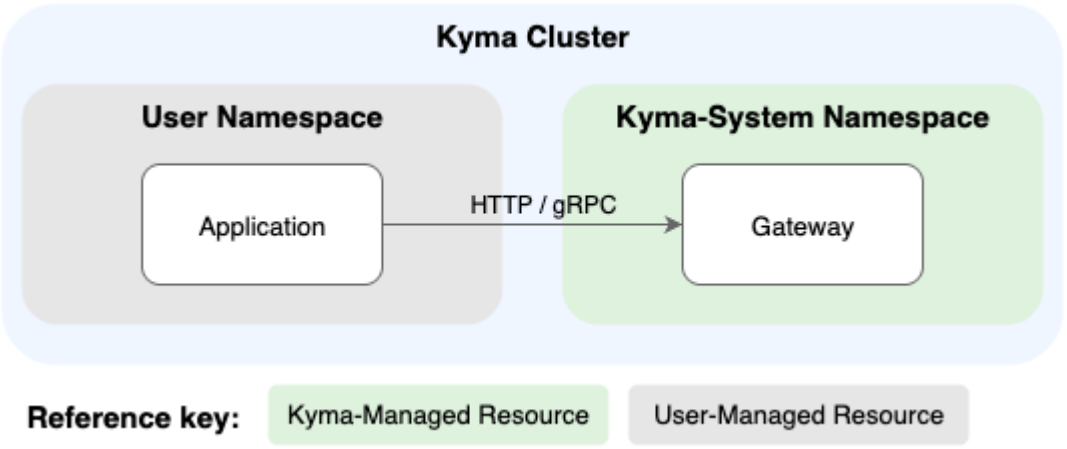
4.3.1.11.2 Set Up the OTLP Input

Use the default OTLP input to collect telemetry data from your instrumented applications and customize how that data is processed by the pipeline. You can specify which input data goes to which backend. If you're using Istio, data is collected from the service mesh.

Overview

When you create any `LogPipeline`, `TracePipeline`, or `MetricPipeline`, the Telemetry module automatically deploys the OTLP Gateway. This opens a stable, cluster-internal OTLP endpoint, ready to receive data from your applications.

The endpoint listens on port `4317` for gRPC (default) and on port `4318` for HTTP.



Configure Your Application's OTLP Exporter

To send data from your application, first instrument your code using an OTel SDK for your programming language. The SDK's OTLP exporter sends the collected telemetry data to a backend.

It's recommended that you configure the exporter's destination by setting the standard environment variables in your application's deployment. This method avoids hardcoding endpoints in your application code.

Use the following environment variable to set the harmonized OTLP endpoint, which accepts all signal types:

- gRPC: `export OTEL_EXPORTER_OTLP_ENDPOINT="http://telemetry-otlp.kyma-system:4317"`
- HTTP: `export OTEL_EXPORTER_OTLP_ENDPOINT="http://telemetry-otlp.kyma-system:4318"`

> ### Note
>
> If your cluster uses Istio, communication with these endpoints is automatically secured with mTLS. For details, see Istio Integration [page 2263].

Verify the Endpoints

To see whether you've set up your OTLP Gateway and its push endpoints successfully, check the status of the default `Telemetry` resource:

```
kubectl -n kyma-system get telemetries.operator.kyma-project.io default -oyaml
```

The output shows the available endpoints and the pipeline health under the `status.endpoints` section:

> ### Output Code
>
> ```
> endpoints:
>     otlp:
>       grpc: http://telemetry-otlp.kyma-system:4317
>       http: http://telemetry-otlp.kyma-system:4318
> ```

Route Specific Inputs to Different Backends

For logs and metrics: If you have multiple pipelines sending data to different backends, you can specify which inputs are active for each pipeline. This is useful if you want one pipeline to handle only OTLP data and another to handle only data from a different source.

> ### Tip
>
> For more granular control, you can also filter incoming OTLP data by namespace. For details, see Filter Logs [page 2214] and Filter Metrics [page 2218].

For example, if you want to analyze `otlp` input data in one backend and only data from the log-specific `runtime` input in another backend, then disable the `otlp` input for the second backend. By default, `otlp` input is enabled.

Sample Code

```
...
  input:
    runtime:
      enabled: true
    otlp:
      enabled: false
```

4.3.1.11.3 Collecting Logs

With the Telemetry module, you can observe and debug your applications by collecting, processing, and exporting logs. To begin collecting logs, you create a `LogPipeline` resource. It automatically collects OTLP logs and application logs from the `stdout/stderr` channel. You can also activate Istio log collection.

Overview

A `LogPipeline` is a Kubernetes custom resource (CR) that configures log collection for your cluster. When you create a `LogPipeline`, the Telemetry Manager automatically deploys the necessary components (for details, see Logs Architecture [page 2256]):

- The OTLP Gateway provides a central OTLP endpoint for receiving logs pushed from your applications.
- A Log Agent that runs on each cluster node to collect logs written to `stdout` and `stderr` by your application containers.

The pipeline enriches all collected logs with Kubernetes metadata and transforms them into the OTLP format before sending them to your chosen backend.

Log collection is optional. If you don't create a `LogPipeline`, no logs are collected.

Prerequisites

- Before you can collect logs from a component, it must emit the logs. Typically, it uses a logger framework for the used language runtime (like Node.js) and prints them to the `stdout` or `stderr` channel (see Kubernetes: How nodes handle container logs). Alternatively, you can use the OTel SDK to use the push-based OTLP format .
- If you want to emit the logs to the `stdout/stderr` channel, use structured logs in a JSON format with a logger library like log4J. With that, the Log Agent can parse your log and enrich all JSON attributes as log attributes, and a backend can use that.

- If you prefer the push-based alternative with OTLP, also use a logger library like log4J. However, you must additionally instrument that logger and bridge it to the OTel SDK. For details, see OpenTelemetry: New First-Party Application Logs .

Minimal LogPipeline

For a minimal setup, you only need to create a `LogPipeline` that specifies your backend destination (see Integrate With Your OTLP Backend [page 2231]):

```
apiVersion: telemetry.kyma-project.io/v1beta1
kind: LogPipeline
metadata:
  name: backend
output:
    otlp:
      endpoint:
        value: http://<myEndpoint>:4317
```

By default, this minimal pipeline enables the following types of log collection:

- Application logs: Collects `stdout` and `stderr` logs from all containers running in non-system namespaces (such as `kyma-system` and `kube-system`).
- OTLP logs: Activates cluster-internal endpoints to receive logs in the OTLP format. Your applications can push logs directly to these URLs:
  - gRPC: `http://telemetry-otlp.kyma-system:4317`
  - HTTP: `http://telemetry-otlp.kyma-system:4318`

Configure Log Collection

You can customize your `LogPipeline` using the available parameters and attributes (see LogPipeline: Custom Resource Parameters):

- Configure or disable the collection of `application` logs from the `stdout/stderr` channel (see Configure Application Logs [page 2196]).
- Set up the collection of Istio access logs (see Configure Istio Access Logs [page 2197]).
- Choose from which specific namespaces you want to include or exclude logs (see Filter Logs [page 2214]).
- If you have more than one backend, specify which input source sends logs to which backend (see Route Specific Inputs to Different Backends [page 2193]).

Limitations

- **Throughput**:
  - When pushing OTLP logs of an average size of 2 KB to the OTLP Gateway, the Telemetry module can process approximately 12,000 logs per second (LPS) per node. The OTLP Gateway runs one instance per cluster node.

    - The Log Agent, running one instance per node, handles tailing logs from `stdout` using the `runtime` input. When writing logs of an average size of 2KB to `stdout`, a single Log Agent instance can process approximately 9,000 LPS.
- **Unavailability of Output**: For up to 5 minutes, a retry for data is attempted when the destination is unavailable. After that, data is dropped.
- **No Guaranteed Delivery**: The used buffers are volatile. If any gateway or agent instance crashes, logs data can be lost.
- **Multiple LogPipeline Support**: The maximum amount of `LogPipeline` resources is 5.

4.3.1.11.3.1 Configure Application Logs

To collect logs that your applications write to `stdout` and `stderr`, create a `LogPipeline`. The `runtime` input is enabled by default and uses an agent on each node to tail container log files. You can control which namespaces and containers to include or exclude.

Prerequisites

- You have the Telemetry module in your cluster.
- You have access to Kyma dashboard. Alternatively, if you prefer CLI, you need kubectl .

Context

Use the `runtime` input section to restrict or specify which resources you want to include. You can define the namespaces to include in the input collection, exclude namespaces from the input collection, or choose that only system namespaces are included. For details, see LogPipeline: Custom Resource Parameters .

When you apply the `LogPipeline` resource to your Kubernetes cluster, a log agent is deployed and starts collecting the log data, transforms them to OTLP, and sends them to your backend. For details, see Transformation to OTLP Logs [page 2226].

Enable or Disable Log Collection

The `runtime` input is enabled by default. To create a pipeline that only accepts logs pushed with OTLP, you can disable it.

```
...
input:
  runtime:
    enabled: false     # Default is true
```

By default, input is collected from all namespaces, except the system namespaces `kube-system`, `istio-system`, `kyma-system`, which are excluded by default.

> → Tip
>
> To select logs from specific namespaces and containers, or to include system namespaces, see Filter Logs [page 2214].

Discard the Original Log Body

By default, the log agent preserves the original JSON log message by moving it to the `attributes."log.original"` field after parsing. For details, see Transformation to OTLP Logs [page 2226].

To reduce data volume, you can disable this behavior. Set the parameter to **`false`** to discard the original JSON string after its contents are parsed into attributes.

```
...
  input:
    runtime:
      keepOriginalBody: false     # Default is true
```

4.3.1.11.3.2 Configure Istio Access Logs

To monitor traffic in your service mesh, configure Istio to send access logs. The `LogPipeline` automatically receives these logs through its default OTLP input.

Prerequisites

- You have the Istio module in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- You have access to Kyma dashboard. Alternatively, if you prefer CLI, you need kubectl .

Context

Istio access logs help you monitor the "four golden signals" (latency, traffic, errors, and saturation) and troubleshoot anomalies.

By default, these logs are disabled because they can generate a high volume of data. To collect them, you apply an Istio `Telemetry` resource to a specific namespace, for the Istio Ingress Gateway, or for the entire mesh.

> ⚠ Caution
>
> Enabling access logs, especially for the entire mesh, can significantly increase log volume and may lead to higher storage costs. Enable this feature only for the resources or components that you want to monitor.

After enabling Istio access logs, reduce data volume and costs by filtering them (see Filter Logs [page 2214]).

The Istio module provides a preconfigured extension provider called `kyma-logs`, which tells Istio to send access logs to the Telemetry module's OTLP endpoint. If your `LogPipeline` uses the legacy `http` output, you must use the `stdout-json` provider instead.

> **Note**
>
> You can only have one mesh-wide Istio `Telemetry` resource. If you also plan to enable Istio tracing (see Configure Istio Tracing [page 2201]), configure both access logging and tracing in this single resource.

Enable Istio Logs for a Namespace

Apply the Istio `Telemetry` resource to a specifc namespace:

```
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: access-config
  namespace: $YOUR_NAMESPACE
spec:
  accessLogging:
    - providers:
      - name: kyma-logs
```

Enable Istio Logs for the Ingress Gateway

To monitor all traffic entering your mesh, enable access logs on the Istio Ingress Gateway (instead of the individual proxies of your workloads).

Apply the Istio `Telemetry` resource to the `istio-system` namespace, selecting the gateway Pods:

```
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: mesh-default
  namespace: istio-system
spec:
  selector:
    matchLabels:
      istio: ingressgateway
  accessLogging:
    - providers:
      - name: kyma-logs
```

Enable Istio Logs for the Entire Mesh

To enable access logs globally for all proxies in the mesh, apply the Istio `Telemetry` resource to the `istio-system` namespace. Use this option with caution due to the high data volume.

```
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: mesh-default
  namespace: istio-system
spec:
  accessLogging:
    - providers:
      - name: kyma-logs
```

4.3.1.11.4 Collecting Traces

With the Telemetry module, you can collect distributed traces to understand the flow of requests through your applications and infrastructure. To begin collecting traces, you create a `TracePipeline` resource. It automatically collects OTLP traces and can be configured to collect traces from the Istio service mesh.

Overview

A `TracePipeline` is a Kubernetes custom resource (CR) that configures trace collection for your cluster. When you create a `TracePipeline`, the OTLP Gateway provides a central OTLP endpoint receiving traces pushed from applications (for details, see Traces Architecture [page 2258]).

The pipeline enriches all collected traces with Kubernetes metadata before sending them to your chosen backend.

Trace collection is optional. If you don't create a `TracePipeline`, no traces are collected.

Prerequisites

For the recording of a distributed trace, every involved component must propagate at least the trace context. For details, see Trace Context .

- In Kyma, all modules involved in users' requests support the W3C Trace Context protocol. The involved Kyma modules are, for example, Istio, Serverless, and Eventing.
- Your application also must propagate the W3C Trace Context for any user-related activity. This can be achieved easily using the Open Telemetry SDKs available for all common programming languages. If your application propagates the W3C Trace Context and is part of the Istio service mesh, Istio automatically generates its own spans for the traffic entering and leaving your application.
- Furthermore, your application must enrich a trace with additional span data and send this data to the cluster-central telemetry services. You can achieve this with Open Telemetry SDKs .

With the default configuration, the OTLP Gateway collects push-based OTLP traces of any container running in Kyma runtime, and the data is shipped to your backend.

Minimal TracePipeline

For a minimal setup, you only need to create a `TracePipeline` that specifies your backend destination (see Integrate With Your OTLP Backend [page 2231]):

```
apiVersion: telemetry.kyma-project.io/v1beta1
kind: TracePipeline
metadata:
  name: backend
output:
    otlp:
      endpoint:
        value: http://<myEndpoint>:4317
```

By default, this minimal pipeline collects push-based OTLP traces of any container running in Kyma runtime.

It activates cluster-internal endpoints to receive traces in the OTLP format. Applications can push traces directly to these URLs:

- gRPC: `http://telemetry-otlp.kyma-system:4317`
- HTTP: `http://telemetry-otlp.kyma-system:4318`

Configure Trace Collection

You can adjust the `TracePipeline` using runtime configuration with the available parameters and attributes (see TracePipeline: Custom Resource Parameters).

- If you use Istio, activate Istio tracing. For details, see Configure Istio Tracing [page 2201]. You can adjust which percentage of the trace data is collected.
- The Serverless module integrates the OpenTelemetry SDK by default. It automatically propagates the trace context for chained calls and reports custom spans for incoming and outgoing requests. You can add more spans within your Function's source code. For details, see Customize Function Traces .
- The Eventing module uses the CloudEvents protocol, which natively supports W3C Trace Context propagation. It ensures that the trace context is passed along but doesn't enrich a trace with more advanced span data.

Limitations

- **Throughput**: Assuming an average span with 40 attributes with 64 characters, the maximum throughput is 4200 span/sec ~= 15.000.000 spans/hour. If this limit is exceeded, spans are refused. The OTLP Gateway runs one instance per cluster node
- **Unavailability of Output**: For up to 5 minutes, a retry for data is attempted when the destination is unavailable. After that, data is dropped.

- **No Guaranteed Delivery**: The used buffers are volatile. If any gateway instance crashes, trace data can be lost.
- **Multiple TracePipeline Support**: The maximum amount of `TracePipeline` resources is 5.
- **System Span Filtering**: System-related spans reported by Istio are filtered out without the opt-out option, for example:
  - Any communication of applications to the OTLP Gateway
  - Any communication from the OTLP Gateway to backends

4.3.1.11.4.1 Configure Istio Tracing

To get an end-to-end view of requests in your service mesh, configure Istio to send trace data. The `TracePipeline` receives these traces through its default OTLP input. You can adjust the sampling rate and apply settings to specific namespaces or workloads.

Prerequisites

- You have the Istio module in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- You have access to Kyma dashboard. Alternatively, if you prefer CLI, you need kubectl and curl.

Context

By default, Istio traces are disabled because they can generate a high volume of data. To collect them, you create an Istio `Telemetry` resource in the `istio-system` namespace. When you enable this feature, the Istio proxy sidecars automatically propagate the trace context and report spans for traffic between your services.

The Istio module provides a preconfigured extension provider called `kyma-traces` to send this data to the Telemetry module's trace gateway.

Istio plays a key role in distributed tracing. Its Ingress Gateway is typically where external requests enter your cluster. If a request doesn't have a trace context, Istio adds it. Furthermore, every component within the Istio service mesh runs an Istio proxy, which propagates the trace context and creates span data. When you enable Istio tracing, and it manages trace propagation in your application, you get a complete picture of a trace, because every component automatically contributes span data. Also, Istio tracing is preconfigured to use the vendor-neutral W3C Trace Context protocol.

> ⚠ Caution
>
> Enabling Istio traces can significantly increase data volume and might quickly consume your trace storage. Start with a low sampling rate in production environments.

Enable Istio Tracing for the Entire Mesh

To enable tracing for all workloads in the service mesh, apply an Istio `Telemetry` resource to the `istio-system` namespace. Use this option to establish a baseline configuration for your mesh.

> ⓘ Note
>
> You can only have one mesh-wide Istio `Telemetry` resource in the `istio-system` namespace. If you also want to configure Istio access logs, combine both configurations into a single resource (see Configure Istio Access Logs [page 2197]).

The following example enables tracing with a default sampling rate of 1%:

```
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: mesh-default
  namespace: istio-system
spec:
  tracing:
  - providers:
    - name: "kyma-traces"
    randomSamplingPercentage: 1.00
```

> → Tip
>
> After setting a mesh-wide default in the `istio-system` namespace, you can apply more specific tracing configurations for an entire namespace or for individual workloads within a namespace. This is useful for debugging a particular service by increasing its sampling rate without affecting the entire mesh. For details, see Filter Traces [page 2217].

Configure the Sampling Rate

By default, Istio samples 1% of traces to reduce data volume. To change it, set the `randomSamplingPercentage` value. The sampling decision is propagated within the trace context to ensure that either all or no spans for a given trace are reported.

> ⓘ Note
>
> For production environments, a low sampling rate (1–5%) is recommended to manage costs and performance. For development or debugging, you can set it to 100.00 to capture every trace.

```
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: mesh-default
  namespace: istio-system
spec:
  tracing:
  - providers:
    - name: "kyma-traces"
    randomSamplingPercentage: 5.00 # Samples 5% of all traces
```

Propagate Trace Context Without Reporting Spans

In some cases, you may want Istio to propagate the W3C Trace Context for context-aware logging but not report any trace spans. This approach enriches your access logs with `traceId` and `spanId` without the overhead of full distributed tracing.

To achieve this, set `randomSamplingPercentage` to `0.00` in your mesh-wide configuration.

> → Tip
>
> For guidance on Istio access logs, see Configure Istio Access Logs [page 2197].

```
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: mesh-default
  namespace: istio-system
spec:
  tracing:
  - providers:
    - name: "kyma-traces"
    randomSamplingPercentage: 0
```

4.3.1.11.5 Collecting Metrics

With the Telemetry module, you can collect metrics from your workloads and Kubernetes resources to monitor their health, performance, and behavior. To begin collecting metrics, you create a `MetricPipeline` resource. You can collect Prometheus, Istio, and runtime metrics.

Overview

A `MetricPipeline` is a Kubernetes custom resource (CR) that configures metric collection for your cluster. When you create a `MetricPipeline`, it automatically provisions the necessary components (for details, see Metrics Architecture [page 2260]):

- The OTLP Gateway provides a central OTLP endpoint for receiving metrics pushed from applications.
- A metric agent that runs on each cluster node to pull (scrape) metrics from applications and Kubernetes resources.

The pipeline enriches all collected metrics with Kubernetes metadata. It also transforms non-OTLP formats (like Prometheus) into the OTLP standard before sending them to your chosen backend.

Metric collection is optional. If you don't create a `MetricPipeline`, the metric collection components are not deployed.

Prerequisites

- Before you can collect metrics data from a component, it must expose (or instrument) the metrics. Typically, it instruments specific metrics for the used language runtime (like Node.js) and custom metrics specific to the business logic. Also, the exposure can be in different formats, like the pull-based Prometheus format or the push-based OTLP format .
- If you want to use Prometheus-based metrics, you must have instrumented your application using a library like the Prometheus client library , with a port in your workload exposed serving as a Prometheus metrics endpoint.
- If you want to scrape the metric endpoint with Istio, your Service `port` definition must define the app protocol. For details, see Scrape Metrics from Istio-enabled Workloads [page 2208].
- For the instrumentation, you typically use an SDK, namely the Prometheus client libraries or the Open Telemetry SDKs . Both libraries provide extensions to activate language-specific auto-instrumentation like for Node.js, and an API to implement custom instrumentation.

Minimal MetricPipeline

For a minimal setup, you only need to create a `MetricPipeline` that specifies your backend destination (see Integrate With Your OTLP Backend [page 2231]):

```
apiVersion: telemetry.kyma-project.io/v1beta1
kind: MetricPipeline
metadata:
  name: backend
output:
    otlp:
      endpoint:
        value: http://myEndpoint:4317
```

By default, this minimal pipeline collects the following types of metrics:

- OTLP Metrics: Activates cluster-internal endpoints to receive metrics in the OTLP format. Your applications can push metrics directly to these URLs:
  - gRPC: `http://telemetry-otlp.kyma-system:4317`
  - HTTP: `http://telemetry-otlp.kyma-system:4318`
- Health Metrics: Collects health and performance metrics about the Telemetry module's components. This input is always active and cannot be disabled. For details, see Monitor Pipeline Health [page 2243].

To collect metrics from Kyma modules like Istio, Eventing, or Serverless, enable additional inputs.

Configure Metrics Collection

You can adjust the `MetricPipeline` using runtime configuration with the available parameters (see MetricPipeline: Custom Resource Parameters).

- Scrape `prometheus` metrics from applications that expose a Prometheus-compatible endpoint (see Collect Prometheus Metrics [page 2206]).

- Collect `istio` service mesh metrics from Istio proxies and control plane components (see Collect Istio Metrics [page 2209]).
- Collect `runtime` resource usage and status metrics from Kubernetes components like Pods, Nodes, and Deployments (see Collect Runtime Metrics [page 2210]).
- Use diagnostic metrics to debug your `prometheus` and `istio` configuration.
- Choose from which specific namespaces you want to include or exclude metrics (see Filter Metrics [page 2218]).
- Avoid redundancy by dropping push-based OTLP metrics that are sent directly to the OTLP Gateway (see Route Specific Inputs to Different Backends [page 2193]).
- Reduce or increase metric collection frequency for all pull-based inputs or for a specific input type by changing the collection interval (see Configure Collection Interval [page 2205]).

Configure Collection Interval

By default, the metric agent scrapes all pull-based inputs (Prometheus, Istio, and runtime) every 30 seconds. You can change this interval in the Telemetry CR.

To set a global collection interval that applies to all pull-based inputs, use the `metric.collectionInterval` field:

```
apiVersion: operator.kyma-project.io/v1beta1
kind: Telemetry
metadata:
  name: default
  namespace: kyma-system
spec:
  metric:
    collectionInterval: 60s
```

To override the interval for a specific input type, use the input-specific `collectionInterval` field. The following example sets a global interval of 60s but overrides it to 15s for the Prometheus input:

```
apiVersion: operator.kyma-project.io/v1beta1
kind: Telemetry
metadata:
  name: default
  namespace: kyma-system
spec:
  metric:
    collectionInterval: 60s
    prometheus:
      collectionInterval: 15s
```

The input-specific override takes precedence over the global `metric.collectionInterval`, which takes precedence over the default of 30s.

For details on the available parameters, see Telemetry Custom Resource .

Limitations

- **Throughput**: Assuming an average metric with 20 metric data points and 10 labels, the default OTLP Gateway setup has a maximum throughput of 34K metric data points/sec per node. If more data is sent to the gateway, it is refused. The OTLP Gateway runs one instance per cluster node.
- **Scraping**: The metric **agent** enforces the following limits when scraping metrics from applications and Istio proxies:
  - Timeseries limit: 50000 timeseries per scrape
  - Request body size limit: 20 MB per scrape

  If a scrape exceeds either limit, the metric agent drops the scrape and logs a warning message. To resolve this issue, try reducing the number of exported metrics.
- **Unavailability of Output**: For up to 5 minutes, a retry for data is attempted when the destination is unavailable. After that, data is dropped.
- **No Guaranteed Delivery**: The used buffers are volatile. If any gateway or agent instance crashes, metric data can be lost.
- **Multiple MetricPipeline Support**: The maximum amount of `MetricPipeline` resources is 5.

4.3.1.11.5.1 Collect Prometheus Metrics

To collect metrics from applications that expose a Prometheus-compatible endpoint, enable the `prometheus` input in your `MetricPipeline` and annotate your Pods or Services for discovery. You can enable diagnostic metrics and control from which namespaces metrics are collected.

Prerequisites

Instrument your application using a library like the Prometheus client library or the OTel SDK (with Prometheus exporter). Expose a port in your workload as a Prometheus metrics endpoint.

Activate Prometheus Metrics

By default, the `prometheus` input is disabled. If your applications emit Prometheus metrics, enable the collection of Prometheus-based metrics:

```
...
input:
  prometheus:
    enabled: true
```

> → Tip
>
> To validate or debug your configuration, use diagnostic metrics.

To select metrics from specific namespaces or to include system namespaces, see Filter Metrics [page 2218].

To change the scrape interval, see Configure Collection Interval [page 2205].

Enable Metrics Collection With Annotations

The metric agent automatically discovers Prometheus endpoints in your cluster by looking for specific annotations on your Kubernetes Services or Pods.

To enable automatic metrics collection, apply the following annotations. If your Pod has an Istio sidecar, annotate the Service. Otherwise, annotate the Pod directly.

> **Note**
>
> If your service mesh enforces STRICT mTLS, the agent scrapes the endpoint over HTTPS automatically. If you don't use STRICT mTLS, add the annotation `prometheus.io/scheme: http` to force scraping over plain HTTP.

Annotation Key	Values	Description
`prometheus.io/scrape` (mandatory)	true, false (no default value)	Set to true to enable scraping for this target.
`prometheus.io/port` (mandatory)	8080, 9100 (no default value)	Specify the port on the Pod where your application exposes metrics.
`prometheus.io/path`	/metrics (default), /custom_metrics	Set the HTTP path for the metrics endpoint.
`prometheus.io/scheme`	https (default with Istio), http (default without Istio)	Define the protocol for scraping: Either HTTPS with mTLS, or plain HTTP.
`prometheus.io/param_<name>`	Example: `format:` prometheus (no default)	Add a URL parameter to the scrape request. For example, `prometheus.io/param_format:` prometheus adds `?format=prometheus` to the URL.

For example, see the following `Service` configuration:

```
apiVersion: v1
kind: Service
metadata:
  annotations:
    prometheus.io/port: "8080"
    prometheus.io/scrape: "true"
  name: sample
spec:
  ports:
  - name: http-metrics
    appProtocol: http
    port: 8080
    protocol: TCP
    targetPort: 8080
  selector:
```

```
    app: sample
  type: ClusterIP
```

Scrape Metrics from Istio-enabled Workloads

If your application is part of an Istio service mesh, you must consider service port naming and mutual TLS (mTLS) configuration:

- Istio must be able to identify the `appProtocol` from the Service port definition. Otherwise, Istio may block the scrape request.
  You must either prefix the port name with the protocol like in `http-metrics`, or explicitly define the `appProtocol` attribute.
- The metric agent can scrape endpoints from workloads that enforce mutual TLS (mTLS). For scraping through HTTPS, Istio must configure the workload using STRICT mTLS mode.
  If you can't use STRICT mTLS mode, you can set up scraping through plain HTTP by adding the following annotation to your Service: `prometheus.io/scheme: http`. For related troubleshooting, see MetricPipeline: Failed to Scrape Prometheus Endpoint [page 2249].

Collect Diagnostic Metrics

To validate or debug your scraping configuration for the `prometheus` and `istio` input, you can use diagnostic metrics. By default, they are disabled.

> **Note**
>
> Unlike the `prometheus` and `istio` inputs, the `runtime` input gathers data directly from Kubernetes APIs instead of using a scraping process, so it does not generate scrape-specific diagnostic metrics.

To use diagnostic metrics, enable the `diagnosticMetrics` for the input in your `MetricPipeline`:

```
...
input:
  <istio | prometheus>:
    enabled: true
    diagnosticMetrics:
      enabled: true
```

When enabled, the metric agent generates metrics about its own scrape jobs, such as the following:

- `up`: The scraping was successful
- `scrape_duration_seconds`: Duration of the scrape
- `scrape_samples_scraped`: The number of samples the target exposed
- `scrape_samples_post_metric_relabeling`: The number of samples remaining after metric relabeling was applied
- `scrape_series_added`: The approximate number of new series in this scrape

4.3.1.11.5.2 Collect Istio Metrics

To monitor the health and performance of your service mesh, enable the `istio` input in your `MetricPipeline`. This scrapes metrics directly from Istio proxies (sidecars) and the control plane. You can enable Envoy and diagnostic metrics, and control from which namespaces metrics are collected.

Prerequisites

- The Istio module is added in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- You have access to Kyma dashboard. Alternatively, if you prefer CLI, you need kubectl and curl .

> ⓘ Note
>
> This configuration collects metrics generated by the Istio proxies themselves. To collect custom metrics exposed by your own applications running in the service mesh, you must configure your application accordingly. For details, see Scrape Metrics from Istio-enabled Workloads [page 2208].

Activate Istio Metrics

By default, the `istio` input is disabled. If you are using Istio, enable the collection of Istio metrics:

```
...
input:
  istio:
    enabled: true
```

With this, the Metric Agent starts collecting all Istio metrics from Istio sidecars in all namespaces except the system namespaces `kyma-system`, `kube-system`, and `istio-system`.

> → Tip
>
> To validate or debug your configuration, use diagnostic metrics.
>
> To select metrics from specific namespaces, see Filter Metrics [page 2218].
>
> To change the scrape interval, see Configure Collection Interval [page 2205].

Collect Envoy Metrics

By default, the Metric Agent collects only Istio metrics (prefixed with `istio_`) and ignores Envoy metrics (prefixed with `envoy_`).

Envoy metrics help you understand the performance and behavior of your Envoy proxy, providing details like request rates, latencies, and error counts. For details, see Envoy metrics and server metrics .

To use Envoy metrics to observe and troubleshoot service mesh traffic, enable the `envoyMetrics` section under the `istio` input:

```
...
input:
  istio:
    enabled: true
    envoyMetrics:
      enabled: true
```

Collect Diagnostic Metrics

To validate or debug your scraping configuration for the `prometheus` and `istio` input, you can use diagnostic metrics. By default, they are disabled.

> **Note**
>
> Unlike the `prometheus` and `istio` inputs, the `runtime` input gathers data directly from Kubernetes APIs instead of using a scraping process, so it does not generate scrape-specific diagnostic metrics.

To use diagnostic metrics, enable the `diagnosticMetrics` for the input in your `MetricPipeline`:

```
...
input:
  <istio | prometheus>:
    enabled: true
    diagnosticMetrics:
      enabled: true
```

When enabled, the metric agent generates metrics about its own scrape jobs, such as the following:

- `up`: The scraping was successful
- `scrape_duration_seconds`: Duration of the scrape
- `scrape_samples_scraped`: The number of samples the target exposed
- `scrape_samples_post_metric_relabeling`: The number of samples remaining after metric relabeling was applied
- `scrape_series_added`: The approximate number of new series in this scrape

4.3.1.11.5.3 Collect Runtime Metrics

To monitor the health and resource usage of your Kubernetes cluster, enable the `runtime` input in your `MetricPipeline`. This uses an agent on each node to gather metrics for resources like Pods, Nodes, and

Deployments. You can choose the specific resources to monitor and control from which namespaces metrics are collected.

Activate Runtime Metrics

By default, the `runtime` input is disabled. If you want to monitor your Kubernetes resources, enable the collection of runtime metrics:

```
...
  input:
    runtime:
      enabled: true
```

With this, the Metric Agent starts collecting all runtime metrics from all resources (Pod, container, Node, Volume, DaemonSet, Deployment, StatefulSet, and Job).

> → Tip
>
> To select metrics from specific namespaces or to include system namespaces, see Filter Metrics [page 2218].
>
> To change the scrape interval, see Configure Collection Interval [page 2205].

Select Resource Types

By default, metrics for all supported resource types are collected. To enable or disable the collection of metrics for a specific resource, use the `resources` section in the `runtime` input.

The following example collects only DaemonSet, Deployment, StatefulSet, and Job metrics:

```
...
  input:
    runtime:
      enabled: true
      resources:
        pod:
          enabled: false
        container:
          enabled: false
        node:
          enabled: false
        volume:
          enabled: false
        daemonset:
          enabled: true
        deployment:
          enabled: true
        statefulset:
          enabled: true
        job:
          enabled: true
```

See a summary of the types of information you can gather for each resource:

Resource	Metrics Collected
`pod`	CPU, memory, filesystem, and network usage; current Pod phase
`container`	CPU/memory requests, limits, and usage; container restart count
`node`	Aggregated CPU, memory, filesystem, and network usage for the Node
`volume`	Filesystem capacity, usage, and inode statistics for persistent volumes
`deployment`	Number of desired versus available replicas
`daemonset`	Number of desired, current, and ready Nodes
`statefulset`	Number of desired, current, and ready Pods
`job`	Counts of active, successful, and failed Pods

To learn which specific metrics are collected from which source (`kubletstatsreceiver` or `k8sclusterreceiver`), see Runtime Metrics .

Collect Additional Metrics

You can collect additional metrics emitted from the `kubeletstatsreceiver` or `k8sclusterreceiver`. To collect them, add the metric names to the `additionalMetrics` list in the `runtime` input.

The following example collects the metrics `k8s.pod.memory_request_utilization` (from `kubeletstatsreceiver`) and `k8s.container.status.state` (from `k8sclusterreceiver`):

```
...
    input:
      runtime:
        enabled: true
        additionalMetrics:
          - k8s.pod.memory_request_utilization
          - k8s.container.status.state
```

To learn which metrics can be added to the `additionalMetrics` list, see Runtime Additional Metrics .

The `additionalMetrics` list overrides the `resources` section. For example, if you disable `pod` metrics in the `runtime` input but add a specific Pod metric to the `additionalMetrics` list, that specific metric is still collected.

The following example collects the `k8s.pod.memory_request_utilization` metric even though the `pod` metrics are disabled:

```
...
    input:
      runtime:
```

```
        enabled: true
        resources:
          pod:
            enabled: false
        additionalMetrics:
          - k8s.pod.memory_request_utilization
```

4.3.1.11.6 Filtering and Processing Data

The Telemetry module provides several ways to refine your telemetry data. You can control what data is collected, enrich it with valuable context, modify its structure, and drop irrelevant information before it reaches your backend.

Input-Level Filtering

You can select or reject data at the source, before it enters a pipeline. This is useful for controlling data volume from specific namespaces, applications, or Kubernetes resources. This is the most efficient way to drop large volumes of unwanted data at the earliest stage of the pipeline.

The Telemetry module supports the following input-level filtering mechanisms:

- Filtering within a pipeline: In your `LogPipeline` and `MetricPipeline`, you can configure the `input` section to select or reject data before it is processed by the agent. This is the most common method for filtering application logs, runtime metrics, and Prometheus metrics.
- Configuring the data source: For Istio-generated data (access logs and traces), you configure the Istio `Telemetry` resource itself. This controls which workloads generate data and at what volume (sampling rate), before it is even sent to a pipeline.

Advanced Processing with OTTL

Next, you can use the OpenTelemetry Transformation Language (OTTL) for content-based control over your telemetry data. OTTL rules are applied after basic input filtering. You can modify fields, redact sensitive information, or drop data based on complex conditions.

Automatic Processing

By default, Telemetry pipelines perform some automatic processing to standardize your data and make it easier to analyze:

- Application logs from containers are automatically transformed into structured OpenTelemetry (OTLP) log records.

- All pipelines automatically enrich telemetry data with Kubernetes resource attributes, such as Pod name, namespace, and labels. With this context information, you can easily identify the source of telemetry data in your backend.

4.3.1.11.6.1 Filter Logs

Filter logs from the OTLP, application, and Istio input to control which data your pipeline processes. You can define filters to include or exclude logs based on their source namespace, container, and other attributes.

Overview

> → Tip
>
> The following settings filter data by source. For advanced, content-based filtering and transformation, use the OpenTelemetry Transformation Language (OTTL). For details, see Transform and Filter with OTTL [page 2220].

Source	Granularity	Behavior without `namespaces` Block	Collect from All Namespaces	Collect from Specific Namespaces
OTLP (default)	Namespace	**includes** system namespaces	This is the default, no action needed.	Use the `include` or `exclude` filter
Application	Namespace, Container*	**excludes** system namespaces	Add `namespaces: {}` to the input's configuration	Use the `include` or `exclude` filter
Istio	Namespace, Workload (`selector`), Log content (`filter.expression`)	n/a	Apply the Istio `Telemetry` resource mesh-wide	Apply the Istio `Telemetry` resource to specific namespaces

* The `runtime` input provides an additional `containers` selector that behaves the same way as the `namespaces` selector.

Filter OTLP Logs by Namespaces

You can filter incoming OTLP logs by namespace. By default, all system namespaces are included.

These filters only apply to logs that have an associated namespace. The pipeline always collects any logs that do not have a namespace.

The `include` and `exclude` filters are mutually exclusive.

- To collect OTLP logs from specific namespaces, use the `include` filter:

  Sample Code

  ```
  ...
  input:
    otlp:
      namespaces:
        include:
          - namespaceA
          - namespaceB
  ```

- To collect OTLP logs from all namespaces except specific ones, use the `exclude` filter:

  Sample Code

  ```
  ...
  input:
    otlp:
      namespaces:
        exclude:
          - namespaceA
          - namespaceB
  ```

Filter Application Logs by Namespace

You can control which namespaces to collect logs from using `include`, `exclude`, and `system` filters. The `include` and `exclude` filters are mutually exclusive.

- To collect logs from specific namespaces, use the `include` filter:

  ```
  ...
  input:
    runtime:
      namespaces:
        include:
          - namespaceA
          - namespaceB
  ```

- To collect logs from all namespaces except specific ones, use the `exclude` filter:

  ```
  ...
  input:
    runtime:
      namespaces:
        exclude:
          - namespaceC
  ```

Collect Application Logs From System Namespaces

By default, application logs from `kube-system`, `istio-system`, and `kyma-system` are excluded. To override this and collect logs from them, set the `namespaces` attribute to an empty object::

```
  ...
  input:
    runtime:
      enabled: true
      namespaces: {}
```

Filter Application Logs by Container

You can also filter logs based on the container name with `include` and `exclude` filters. These filters apply in addition to any namespace filters.

The following pipeline collects input from all namespaces excluding `kyma-system` and only from the `istio-proxy` containers:

```
...
  input:
    runtime:
      enabled: true
      namespaces:
        exclude:
          - myNamespace
      containers:
        exclude:
          - myContainer
    otlp:
      ...
```

Select Istio Logs From a Specific Application

To limit logging to a single application within a namespace, configure label-based selection for this workload with a selector in the Istio `Telemetry` resource.

```
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: access-config
  namespace: $YOUR_NAMESPACE
spec:
  selector:
    matchLabels:
      service.istio.io/canonical-name: $YOUR_LABEL
  accessLogging:
    - providers:
      - name: kyma-logs
```

Filter Istio Logs by Content

To reduce noise and cost, filter your Istio access logs. This is especially useful for filtering out traffic that doesn't use an HTTP-based protocol, as those log entries often lack useful details.

Add a `filter` expression to the same `accessLogging` block that you used to enable the logs. The expression uses Envoy attributes to define which log entries to keep.

The following example enables mesh-wide logging but only keeps logs that have a defined request protocol, effectively filtering out most non-HTTP traffic.

```
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
 name: mesh-default
 namespace: istio-system
spec:
 accessLogging:
 - filter:
     expression: 'has(request.protocol)'
   providers:
   - name: kyma-logs
```

4.3.1.11.6.2 Filter Traces

`TracePipeline` resources have no `input` specification. You can configure Istio trace collection by applying the Istio `Telemetry` resource to specific namespaces.

> → Tip
>
> The following settings filter data before it reaches the `TracePipeline`. For advanced, content-based filtering and transformation, use the OpenTelemetry Transformation Language (OTTL). For details, see Transform and Filter with OTTL [page 2220].

Override Tracing for a Namespace or Workload

After setting a mesh-wide default, apply more specific tracing configurations for an entire namespace or for individual workloads within a namespace. Use this to debug a particular service by increasing its sampling rate without affecting the entire mesh.

To do this, create an Istio `Telemetry` resource in the workload's namespace. To apply a tracing configuration to a specific workload within the namespace, add a `selector` that matches the workload's labels.

For example, increase the sampling rate (in this example, to `100.00`) to debug a specific application without affecting the entire mesh:

```
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: tracing
```

```
  namespace: my-namespace
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: "my-app"
  tracing:
  - providers:
    - name: "kyma-traces"
    randomSamplingPercentage: 100.00
```

Disable Tracing for a Specific Workload

To completely disable Istio span reporting for a specific workload while keeping it enabled for the rest of the mesh, create a `Telemetry` resource that targets the workload and set `disableSpanReporting` to **`true`**.

```
apiVersion: telemetry.istio.io/v1
kind: Telemetry
metadata:
  name: $YOUR_APP_NAME-tracing-disable
  namespace: $YOUR_NAMESPACE
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: "$YOUR_APP_NAME"
  tracing:
  - providers:
    - name: "kyma-traces"
    disableSpanReporting: true
```

4.3.1.11.6.3 Filter Metrics

Filter metrics from the OTLP, Istio, Prometheus, and runtime input to control which data your pipeline processes. You can define filters to include or exclude metrics based on their source namespace and resource type.

Overview

> → Tip
>
> The following settings filter data by source. For advanced, content-based filtering and transformation, use the OpenTelemetry Transformation Language (OTTL). For details, see Transform and Filter with OTTL [page 2220].

Source	Granularity	Behavior without `namespaces` Block	Collect from All Namespaces	Collect from Specific Namespaces
OTLP (default)	Namespace	**includes** system namespaces	This is the default, no action needed.	Use the `include` or `exclude` filter
Istio	Namespace	**excludes** system namespaces	Add `namespaces: {}` to the input's configuration	Use the `include` or `exclude` filter
Prometheus	Namespace	**excludes** system namespaces	Add `namespaces: {}` to the input's configuration	Use the `include` or `exclude` filter
Runtime	Namespace, Resource Type*	**excludes** system namespaces	Add `namespaces: {}` to the input's configuration	Use the `include` or `exclude` filter

* The `runtime` input provides additional filters for Kubernetes resources such as Pods or Nodes. For details, see Select Resource Types [page 2211].

Filter Metrics by Namespace

For the all inputs (`otlp`, `prometheus`, `istio`, and `runtime`), you can filter incoming metrics by namespace.

These filters only apply to metrics that have an associated namespace. The pipeline always collects any metrics that do not have a namespace.

The `include` and `exclude` filters are mutually exclusive.

- To collect metrics from specific namespaces, use the `include` filter:

  Sample Code

  ```
  ...
  input:
    <otlp | prometheus | istio | runtime>:
      enabled: true
      namespaces:
        include:
          - namespaceA
          - namespaceB
  ```

- To collect metrics from all namespaces except specific ones, use the `exclude` filter:

  Sample Code

  ```
  ...
  input:
    <otlp | prometheus | istio | runtime>:
      enabled: true
      namespaces:
        exclude:
          - namespaceA
          - namespaceB
  ```

Collect Metrics From System Namespaces

For `otlp` metrics, system namespaces are included by default.

To include system namespaces for `prometheus`, `istio`, and `runtime` metrics without specifying any other namespaces, explicitly configure an empty namespace object: `namespaces: {}`:

Sample Code

```
...
input:
  <prometheus | istio | runtime>:
    enabled: true
    namespaces: {}
```

4.3.1.11.6.4 Transform and Filter with OTTL

To apply custom transformations and filters to your telemetry data, use the OpenTelemetry Transformation Language (OTTL). You can modify, enrich, and filter logs, metrics, and traces before they're sent to your backend.

Overview

You define OTTL rules in the `spec` of your `LogPipeline`, `MetricPipeline`, and `TracePipeline` resources.

1. With the `transform` section, you can standardize and enrich your data: Add, rename, or modify fields to conform to a standard schema, add valuable context, or redact sensitive information. For details, see Transform with OTTL [page 2222].
2. With the `filter` section, you can reduce data volume and cost: Drop unwanted logs, metrics, or traces to lower your ingestion costs and reduce noise in your observability backend. For details, see Filter with OTTL [page 2224].

When you define rules, the Telemetry module configures processors in the underlying OpenTelemetry Collector to apply your rules.

Note

- The underlying OpenTelemetry Transformation Language (OTTL) is a beta feature. Its syntax and function signatures may change in future releases. Always test complex expressions in a non-production environment first.
- Each processing rule consumes CPU and memory. Complex rules can significantly impact the pipeline performance.
- This feature requires Telemetry module v1.52.0 or later.

Processing Order

The Telemetry module processes data in a fixed sequence. Understanding this order is critical for designing efficient and correct pipelines:

1. Input Filtering: First, the pipeline applies any basic filters defined in the `spec.input` section of your pipeline. These filters operate on the source of the data, such as its namespace or container. This is the most efficient way to drop large, unwanted blocks of telemetry data before any complex processing occurs. For details, see Filter Logs [page 2214], Filter Traces [page 2217], and Filter Metrics [page 2218].
2. OTTL Transformation: Next, the pipeline applies the transformation rules from the `transform` section. These rules modify the data that passed the input filters. You can add, rename, or delete fields.
3. OTTL Filtering: Finally, the pipeline evaluates the transformed data against the filter rules from the `filter` section. Any data matching a filter condition is dropped.

This sequence means that your OTTL rules only operate on data that has already passed the initial input filters. Furthermore, your OTTL `filter` conditions must use the final, transformed state of your data, not its original state. For example, if you rename a field in a transformation rule, your OTTL filter must use the new name.

Limitations

Always Use the Full Context Path

You must specify the full path for every field. Short-hand references are not supported.

- Correct: `resource.attributes["k8s.namespace.name"] == "default"`, `log.attributes["level"]`, `datapoint.value_int`, `span.name`
- Incorrect: `attributes["k8s.namespace.name"] == "default"`, `attributes["level"]`, `value_int`, `name`

> → Tip
>
> For details on the underlying implementation details of context inference, see OTel Transform Processor Context Inference .

Limited OTTL Functions

You cannot use the OTTL function `IsRootSpan()` as a standalone function because the processor cannot infer the context from a function without path prefixes. If you need this condition, use `span.parent_span_id == SpanID(0x0000000000000000)` instead.

4.3.1.11.6.4.1 Transform with OTTL

Use transformations to modify telemetry data as it flows through a pipeline. You can modify telemetry fields to enrich data or conform to a specific schema.

Overview

You define these rules in the `transform` section of your Telemetry pipeline's `spec`.

Each rule in the `transform` list contains:

- `statements`: One or more OTTL functions that modify the data. These are the actions that you want to perform.
- `conditions` (optional): One or more OTTL conditions that must be met. If you provide conditions, the statements only run on data that matches at least one of the conditions.
  If you don't provide any conditions, the statements apply to all telemetry data passing through the pipeline.

> **Note**
>
> - The pipeline applies all transformation rules **before** it evaluates any filter rules. Any change you make during the transformation affects the data that your filters see.
> - This feature is based on the OpenTelemetry Transform Processor , with some limitations (see Limitations [page 2221]).

Example: Add a Global Attribute to All Metrics

You can add a `deployment.environment.name` attribute with the value production to all metrics. This is useful for tagging all data from a specific cluster.

```
# In your MetricPipeline spec
spec:
  input:
    prometheus:
      enabled: true
  output:
    otlp:
      endpoint:
        value: http://metrics.example.com:4317
  transform:
    - statements:
        - 'set(resource.attributes["deployment.environment.name"], "production")'
```

Example: Conditionally Set a Span's Status

If you want to identify system-related spans in your observability backend, mark all spans from workloads in system namespaces by adding a `system` attribute with the value true.

```
# In your TracePipeline spec
spec:
  output:
    otlp:
      endpoint:
        value: http://traces.example.com:4317
  transform:
    - conditions:
        - 'IsMatch(resource.attributes["k8s.namespace.name"], ".*-system")'
      statements:
        - 'set(span.attributes["system"], "true")'
```

Example: Parse Custom Log Formats

If your application writes logs to stdout in a custom format, you can use a series of OTTL statements to parse these logs into structured data, so you can easily query and analyze them in your observability backend. The following example parses the payload and enriches core attributes:

```
# In your LogPipeline spec
spec:
  input:
    runtime:
      enabled: true
  output:
    otlp:
      endpoint:
        value: http://traces.example.com:4317
  transform:
    # Try to parse the body as custom parser (default spring boot logback)
    # 2025-11-12T14:40:36.828Z  INFO 1 --- [demo] [           main]
c.e.restservice.RestServiceApplication   : Started Application
    - conditions:
        - log.body != nil
      statements:
        - merge_maps(log.attributes, ExtractPatterns(log.body,"^(?
P<timestamp>\\d{4}-\\d{2}-\\d{2}T\\d{2}:\\d{2}:\\d{2}\\.\\d+Z)\\s+(?P<level>[A-Z]
+)\\s+(?P<pid>\\d+)\\s+---\\s+\\[(?P<mdc>[^\\]]+)\\]\\s+\\[\\s*(?P<thread>[^\\]]
+)\\s*\\]\\s+(?P<logger>[^\\s:]+)\\s*:\\s*(?P<msg>.*)$"), "upsert")
    # Try to enrich core attributes if custom parsing was successful
    - conditions:
        - log.attributes["msg"] != nil
      statements:
        - set(log.body, log.attributes["msg"])
        - delete_key(log.attributes, "msg")
    - conditions:
        - log.attributes["timestamp"] != nil
      statements:
        - set(log.time, Time(log.attributes["timestamp"],
"2006-01-02T15:04:05.000Z"))
        - delete_key(log.attributes, "timestamp")
    - conditions:
        - log.attributes["level"] != nil
      statements:
        - set(log.severity_text, ToUpperCase(log.attributes["level"]))
```

```
        - delete_key(log.attributes, "level")
```

Example: Redact Sensitive Data in Spans

To improve security and meet compliance requirements, you can redact sensitive information from your telemetry data before it leaves the cluster.

```
# In your TracePipeline spec
spec:
  output:
    otlp:
      endpoint:
        value: http://traces.example.com:4317
  transform:
      - statements:
          - replace_pattern(span.attributes["http.url"], "client_id=[^&]+",
"client_id=[REDACTED]")
          - replace_pattern(span.attributes["http.url"], "client_secret=[^&]+",
"client_secret=[REDACTED]")
```

4.3.1.11.6.4.2 Filter with OTTL

Use filters to drop unwanted telemetry data from a pipeline. Filtering helps you reduce noise, lower storage and processing costs, and focus on the data that matters most.

Overview

You define these rules in the `filter` section of your Telemetry pipeline's `spec`.

Each rule in the `filter` list contains one or more `conditions`.

The pipeline drops any log, metric, or trace that matches **at least one** of the conditions you define. This means that multiple conditions are always combined with a logical `OR`. If any single condition evaluates to true, the data is dropped.

> **Note**
>
> - Filters run **after** all transformations. Your filter conditions must operate on the final, modified state of your data, not its original state.
> - This feature is based on the OpenTelemetry Filter Processor, with some limitations and differences (see Limitations [page 2221]).

Example: Drop Low-Severity Logs

You can filter out `DEBUG` and `INFO` logs to reduce noise and storage costs:

```
# In your LogPipeline spec
spec:
  input:
    runtime:
      enabled: true
  output:
    otlp:
      endpoint:
        value: http://logs.example.com:4317
  filter:
    - conditions:
        - 'log.severity_number < SEVERITY_NUMBER_WARN'
```

Example: Keep Only Specific Envoy Metrics

> → Tip
>
> To start collecting envoy metrics, see Collect Envoy Metrics [page 2209].

You can keep all standard Istio metrics but filter the verbose Envoy metrics (those prefixed with `envoy_`) to retain only those related to outlier detection (circuit breaking) and host ejections in your service mesh.

```
# In your MetricPipeline spec
spec:
  input:
    istio:
      enabled: true
      envoyMetrics:
        enabled: true
  output:
    otlp:
      endpoint:
        value: http://metrics.example.com:4317
  filter:
    - conditions:
        - 'IsMatch(metric.name, "^envoy_") == true and IsMatch(metric.name,
".*outlier_detection.*") == false'
```

4.3.1.11.6.5 Transformation to OTLP Logs

Learn how the log agent processes the original container logs and transforms them into structured OpenTelemetry (OTLP) log records.

Original Log Message

The following example shows a container `myContainer` in Pod `myPod`, running in namespace `myNamespace`, logging to `stdout` with the following JSON message:

```
{
  "level": "warn",
  "message": "This is the original message",
  "tenant": "myTenant",
  "trace_id": "123"
}
```

Log Tailing

The log agent reads the log message from a log file managed by the container runtime. The file name contains namespace, Pod, and container information that becomes available later as log attributes. The raw log record looks like the following example:

```
{
  "time": "2022-05-23T15:04:52.193317532Z",
  "stream": "stdout",
  "_p": "F",
   "log": "{\"level\": \"warn\",\"message\": \"This is the original
message\",\"tenant\": \"myTenant\",\"trace_id\": \"123\"}"
}
```

After the tailing, the created OTLP record looks like the following example:

```
{
  "time": "2022-05-23T15:04:52.100000000Z",
  "observedTime": "2022-05-23T15:04:52.200000000",
  "attributes": {
   "log.file.path": "/var/log/pods/myNamespace_myPod-<containerID>/myContainer/
<containerRestarts>.log",
   "log.iostream": "stdout"
  },
  "resourceAttributes": {
    "k8s.container.name": "myContainer",
    "k8s.container.restart_count": "<containerRestarts>",
    "k8s.pod.name": "myPod",
    "k8s.namespace.name": "myNamespace"
  },
  "body": "{\"level\": \"warn\",\"message\": \"This is the original
message\",\"tenant\": \"myTenant\",\"trace_id\": \"123\"}"
}
```

The log agent enriches all information identifying the log source (such as container, Pod, and namespace name) as resource attributes, following Kubernetes conventions . It enriches further metadata, like the

original file name and channel, as log attributes, following log attribute conventions . The agent uses the `time` value from the container runtime's log entry as the `time` attribute in the new OTel record, as it closely matches the actual log event time. Additionally, the agent sets `observedTime` with the time it actually reads the log record, as the OTel log specification recommends. The agent moves the log payload to the OTLP `body` field.

JSON Parsing

If the `body` value is a JSON document, the agent parses the value and enriches all JSON root attributes as additional log attributes. The agent moves the original body into the `log.original` attribute (managed with the `LogPipeline` attribute `input.application.keepOriginalBody: true`).

After JSON parsing, the OTLP record looks like the following example:

```
{
  "time": "2022-05-23T15:04:52.100000000Z",
  "observedTime": "2022-05-23T15:04:52.200000000",
  "attributes": {
   "log.file.path": "/var/log/pods/myNamespace_myPod-<containerID>/myContainer/
<containerRestarts>.log",
   "log.iostream": "stdout",
   "log.original": "{\"level\": \"warn\",\"message\": \"This is the original
message\",\"tenant\": \"myTenant\",\"trace_id\": \"123\"}",
   "level": "warn",
   "tenant": "myTenant",
   "trace_id": "123",
   "message": "This is the original message"
  },
  "resourceAttributes": {
    "k8s.container.name": "myContainer",
    "k8s.container.restart_count": "<containerRestarts>",
    "k8s.pod.name": "myPod",
    "k8s.namespace.name": "myNamespace"
  },
  "body": ""
}
```

Severity Parsing

Typically, a log message includes a log level in the `level` or `log.level` field. If both are present, `log.level` takes precedence. Based on the matching field, the agent parses it with a severity parser. If parsing succeeds, the agent transforms the log attribute into the OTel attributes `severityText` and `severityNumber`.

Trace Parsing

OTLP natively attaches trace context to log records. If possible, the log agent parses the following log attributes according to the W3C-Tracecontext specification :

- `trace_id`

- `span_id`
- `trace_flags`
- `traceparent`

Log Body Determination

Because the original log message typically resides in the `body` attribute, the agent moves a log attribute called `message` (or `msg`) into the body.

At this point, before further enrichment, the resulting overall log record looks like the following example:

```
{
  "time": "2022-05-23T15:04:52.100000000Z",
  "observedTime": "2022-05-23T15:04:52.200000000",
  "attributes": {
   "log.file.path": "/var/log/pods/myNamespace_myPod-<containerID>/myContainer/
<containerRestarts>.log",
   "log.iostream": "stdout",
   "log.original": "{\"level\": \"warn\",\"message\": \"This is the original
message\",\"tenant\": \"myTenant\",\"trace_id\": \"123\"}",
   "tenant": "myTenant",
  },
  "resourceAttributes": {
    "k8s.container.name": "myContainer",
    "k8s.container.restart_count": "<containerRestarts>",
    "k8s.pod.name": "myPod",
    "k8s.namespace.name": "myNamespace"
  },
  "body": "This is the original message",
  "severityNumber": 13,
  "severityText": "warn",
  "traceId": "123"
}
```

This structured record is now ready to be shipped to your observability backend. For details on further enrichment, see Automatic Data Enrichment [page 2228].

4.3.1.11.6.6 Automatic Data Enrichment

The Telemetry module automatically enriches your data with OTel resource attributes, so you can easily identify the source of the data in your backend.

> → Tip
>
> For custom enrichment, such as adding your own business-specific attributes, see Transform and Filter with OTTL [page 2220].

Service Attributes

By default, the gateway enriches telemetry data with service attributes following OTel conventions. It sets `service.namespace`, `service.name`, `service.version`, and `service.instance.id` based on Kubernetes metadata.

If you don't provide a service name, or if its value follows the pattern `unknown_service:<process.executable.name>` as described in the specification, the gateway generates it from Kubernetes metadata.

Be aware of these OTel-specific edge case limitations.

> ⚠ Caution
>
> The legacy strategy for service name enrichment is deprecated and will be removed in a future release. The legacy strategy is active if the annotation `telemetry.kyma-project.io/service-enrichment` is set to kyma-legacy or is missing on the Telemetry CR. It determines the service name based on the following hierarchy:
>
> 1. `app.kubernetes.io/name`: Pod label value
> 2. `app`: Pod label value
> 3. Deployment/DaemonSet/StatefulSet/Job name
> 4. Pod name
> 5. If none of the above is available, the value is unknown_service
>
> To migrate, set the annotation `telemetry.kyma-project.io/service-enrichment`: otel on your Telemetry CR.

Kubernetes Metadata

`k8s.*` attributes encapsulate various pieces of Kubernetes metadata associated with the Pod, such as:

- `k8s.pod.name`: The Kubernetes Pod name of the Pod that emitted the data.
- `k8s.pod.uid`: The Kubernetes Pod ID of the Pod that emitted the data.
- `k8s.<workload kind>.name`: The Kubernetes workload name to which the emitting Pod belongs. Workload is either Deployment, DaemonSet, StatefulSet, Job, or CronJob.
- `k8s.namespace.name`: The Kubernetes namespace name with which the emitting Pod is associated.
- `k8s.cluster.name`: A logical identifier of the cluster, which, by default, is the API Server URL. To set a custom name, configure the `enrichments.cluster.name` field in the Telemetry CRD.
- `k8s.cluster.uid`: A unique identifier of the cluster, realized by the UID of the `kube-system` namespace.
- `k8s.node.name`: The Kubernetes node name to which the emitting Pod is scheduled.

Pod Label Attributes

In the Telemetry CRD, you can also specify your own enrichments of telemetry data based on Pod labels.

To capture custom application metadata (for example, for filtering, grouping, or correlation), configure specific label keys or label key prefixes to include in the enrichment process. The gateway adds all matching Pod labels to the telemetry data as resource attributes, using the label key format `k8s.pod.label.<label_key>`.

The following example configuration enriches the telemetry data with Pod labels that match the specified keys or key prefixes:

- `k8s.pod.label.app.kubernetes.io/name`: The value of the exact label key `app.kubernetes.io/name` from the Pod.
- `k8s.pod.label.app.kubernetes.io.*`: All labels that start with the prefix `app.kubernetes.io` from the Pod, where * is replaced by the actual label key.

```
apiVersion: operator.kyma-project.io/v1beta1
kind: Telemetry
metadata:
  name: default
  namespace: kyma-system
spec:
  enrichments:
    extractPodLabels:
    - key: "<myExactLabelKey>" # for example, "app.kubernetes.io/name"
    - keyPrefix: "<myLabelPrefix>" # for example, "app.kubernetes.io"
```

Cloud Provider Attributes

If data is available, the gateway automatically adds cloud provider attributes to the telemetry data:

- : Cloud provider name. Sourced from the Gardener `shoot-info` ConfigMap in `kube-system`. Only available on Gardener-managed clusters.
- `cloud.region`: Region where the Node runs (from Node label `topology.kubernetes.io/region`)
- `cloud.availability_zone`: Zone where the Node runs (from Node label `topology.kubernetes.io/zone`)

Host Attributes

If data is available, the gateway automatically adds host attributes to the telemetry data:

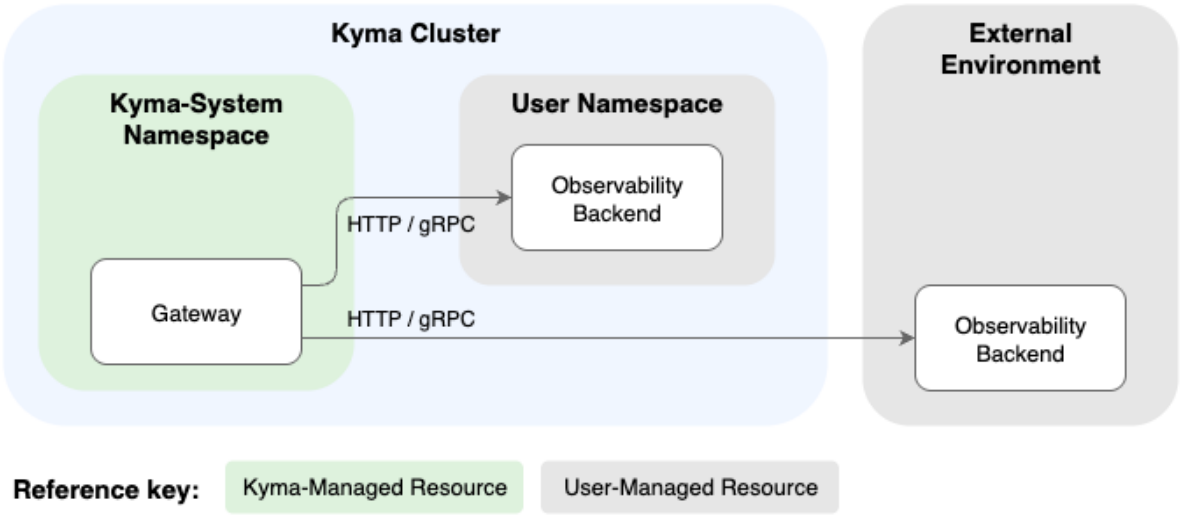
- `host.type`: Machine type of the Node (from Node label `node.kubernetes.io/instance-type`)
- `host.arch`: CPU architecture of the system the Node runs on (from Node label `kubernetes.io/arch`)

4.3.1.11.7 Integrate With Your OTLP Backend

Define the backend destination for your telemetry data by configuring the OTLP output, including the protocol and authentication method.

Overview

In every pipeline, you must configure an `output` section, which defines the destination for your telemetry data using OTLP. This is where you specify your chosen observability backend (see OpenTelemetry: Vendors).



> ### Note
>
> Each pipeline resource supports exactly one backend. However, you can send specific inputs to different backends by setting up designated pipelines. For details, see Route Specific Inputs to Different Backends [page 2193].

Specify the OTLP Endpoint

For the minimal pipeline configuration, you only specify an OTLP endpoint; though it's recommended that you use authentication.

```
...
  output:
    otlp:
      endpoint:
        value: https://backend.example.com:4317
```

Choose a Protocol

The default protocol for shipping the data to a backend is gRPC. If your backend requires the HTTP protocol instead, set the `protocol` attribute to **`http`**. Based on this setting, the gateway chooses the exporter: `otlp_grpc` for gRPC (the default) or `otlp_http` for HTTP.

Ensure the port in your endpoint URL is correct for the chosen protocol.

```
...
 output:
    otlp:
      protocol: http
      endpoint:
        value: https://backend.example.com:4318
```

Configure Compression

By default, the Telemetry gateway uses `gzip` compression when exporting data. To use a different compression algorithm, set the `compression` attribute to one of the supported values: gzip, snappy, zstd, or none (to disable compression explicitly).

```
...
  output:
    otlp:
      compression: none
      endpoint:
        value: https://backend.example.com:4317
```

Set Up Authentication

For each pipeline, add authentication details (like user names, passwords, certificates, or tokens) to connect securely to your observability backend. You can use mutual TLS (mTLS), custom headers, OAuth2, or Basic Authentication.

While you can choose to add your authentication details from plain text, it's recommended to store these sensitive details in a Kubernetes `Secret` and reference the Secret's keys in your pipeline configuration. When you rotate the `Secret` and update its values, Telemetry Manager detects the changes and applies the new `Secret` to your setup.

> → Tip
>
> If you use a Secret owned by the SAP BTP Service Operator , you can configure an automated rotation policy with a specific rotation frequency and don't have to intervene manually.

- To use client certificates for mTLS, configure the `tls` section with your public certificate and private key.

  ```
  ...
  output:
    otlp:
      endpoint:
  ```

```
      value: https://backend.example.com/otlp:4317
    tls:
      cert:
        valueFrom:
          secretKeyRef:
            name: backend
            namespace: default
            key: cert
      key:
        valueFrom:
          secretKeyRef:
            name: backend
            namespace: default
            key: key
```

- To send an authentication token (such as a bearer token) in an HTTP header, configure the `headers` section.

```
...
output:
  otlp:
    endpoint:
      value: https://backend.example.com:4317
    headers:
    - name: Authorization
      prefix: Bearer
      valueFrom:
        secretKeyRef:
            name: backend
            namespace: default
            key: token
```

- To use OAuth2 for authentication, configure the `authentication.oauth2` section.

```
...
output:
  otlp:
    endpoint:
      valueFrom:
        secretKeyRef:
            name: backend
            namespace: default
            key: endpoint
    authentication:
      oauth2:
        clientId:
          valueFrom:
            secretKeyRef:
              name: backend
              namespace: default
              key: clientId
        clientSecret:
          valueFrom:
            secretKeyRef:
              name: backend
              namespace: default
              key: clientSecret
        tokenUrl:
          valueFrom:
            secretKeyRef:
              name: backend
              namespace: default
              key: tokenUrl
```

> ### Note
>
> If you want to use OAuth2 with gRPC, you must configure TLS for your backend connection as well.

- To use a username and password for authentication, configure the `authentication.basic` section.

```
...
output:
  otlp:
    endpoint:
      valueFrom:
        secretKeyRef:
            name: backend
            namespace: default
            key: endpoint
    authentication:
      basic:
        user:
          valueFrom:
            secretKeyRef:
              name: backend
              namespace: default
              key: user
        password:
          valueFrom:
            secretKeyRef:
              name: backend
              namespace: default
              key: password
```

- If you want to configure authentication details from plain text, use the following pattern. The example shows mTLS, but you can also use Basic Authentication or custom headers:

```
...
output:
  otlp:
    endpoint:
      value: https://backend.example.com/otlp:4317
    tls:
      cert:
        value: |
          -----BEGIN CERTIFICATE-----
          ...
      key:
        value: |
          -----BEGIN RSA PRIVATE KEY-----
          ...
```

> ### Note
>
> If your backend is running inside the cluster and is part of the Istio service mesh, the gateways automatically secure the connection and you don't have to configure the `authentication` block. For details, see Sending Data to In-Cluster Backends [page 2265].

4.3.1.11.7.1 Integrate With SAP Cloud Logging

Configure the Telemetry module to send logs, metrics, and traces from your cluster to an SAP Cloud Logging instance. By centralizing this data in your SAP Cloud Logging instance, you can store, visualize, and analyze the observability of your applications.

Prerequisites

- Kyma as the target deployment environment, with the following modules added (see Adding and Deleting a Kyma Module [page 3511]):
  - Telemetry module
  - To collect data from your Istio service mesh: Istio module (default module)
  - SAP BTP Operator module (default module)
- An instance of SAP Cloud Logging with OpenTelemetry ingestion enabled. For details, see Ingest via OpenTelemetry API Endpoint.

  > → Tip
  >
  > Create the SAP Cloud Logging instance with the SAP BTP service operator (see Create an SAP Cloud Logging Instance through SAP BTP Service Operator), because it takes care of creation and rotation of the required Secret. However, you can choose any other method of creating the instance and the Secret, as long as the parameter for OTLP ingestion is enabled in the instance. For details, see Configuration Parameters.

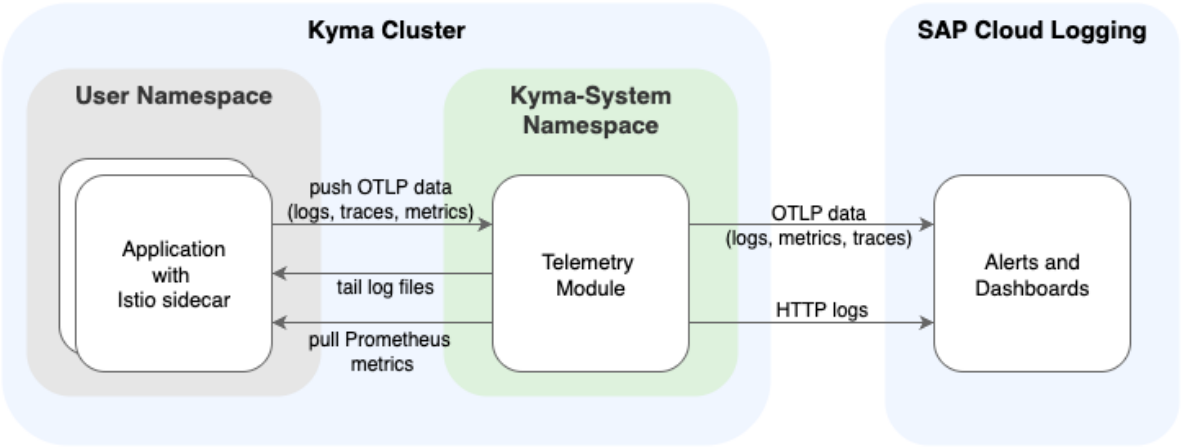
- A Secret in the respective namespace in your Kyma cluster, holding the credentials and endpoints for the instance. It's recommended that you rotate your Secret (see SAP BTP Security Recommendation BTP-CLS-0003). In the following example, the Secret is named "sap-cloud-logging" and the namespace "sap-cloud-logging-integration", as illustrated in the secret-example.yaml .
- Kubernetes CLI (kubectl) (see Install the Kubernetes Command Line Tool).
- UNIX shell or Windows Subsystem for Linux (WSL) to execute commands.

Context

The Telemetry module supports shipping logs and ingesting distributed traces as well as metrics from applications and the Istio service mesh to SAP Cloud Logging.

First, set up the Telemetry module to ship the logs, traces, and metrics to your backend by deploying the pipelines and other required resources. Then, you configure Kyma dashboard integration. Finally, set up SAP Cloud Logging alerts and dashboards.

SAP Cloud Logging is an instance-based and environment-agnostic observability service to store, visualize, and analyze logs, metrics, and traces.



Ship Logs To SAP Cloud Logging

You can set up ingestion of logs from applications and the Istio service mesh to the OTLP endpoint of the SAP Cloud Logging service instance.

Procedure

Set Up Application Logs

1. Deploy a `LogPipeline` for application logs:

```
kubectl apply -f - <<EOF
apiVersion: telemetry.kyma-project.io/v1beta1
kind: LogPipeline
metadata:
  name: sap-cloud-logging
spec:
  input:
    runtime:
      enabled: true
    otlp:
      enabled: true
  output:
    otlp:
      endpoint:
        valueFrom:
          secretKeyRef:
            name: sap-cloud-logging
            namespace: sap-cloud-logging-integration
            key: ingest-otlp-endpoint
      tls:
        cert:
          valueFrom:
            secretKeyRef:
              name: sap-cloud-logging
              namespace: sap-cloud-logging-integration
              key: ingest-otlp-cert
        key:
```

```
          valueFrom:
            secretKeyRef:
              name: sap-cloud-logging
              namespace: sap-cloud-logging-integration
              key: ingest-otlp-key
EOF
```

2. Verify that the `LogPipeline` is running:

```
kubectl get logpipelines
```

Set Up Istio Access Logs

By default, Istio sidecar injection and Istio access logs are disabled in Kyma. To analyze them, you must enable them:

3. Enable Istio sidecar injection for your workload (see Enabling Istio Sidecar Proxy Injection [page 1892]).
4. Configure the Istio `Telemetry` resource (see Configure Istio Access Logs [page 2197]) with the `kyma-logs` extension provider.

Ship Traces To SAP Cloud Logging

You can set up ingestion of distributed traces from applications and the Istio service mesh to the OTLP endpoint of the SAP Cloud Logging service instance.

Procedure

Set Up Traces

1. Deploy a `TracePipeline`:

```
kubectl apply -f - <<EOF
apiVersion: telemetry.kyma-project.io/v1beta1
kind: TracePipeline
metadata:
  name: sap-cloud-logging
spec:
  output:
    otlp:
      endpoint:
        valueFrom:
          secretKeyRef:
            name: sap-cloud-logging
            namespace: sap-cloud-logging-integration
            key: ingest-otlp-endpoint
      tls:
        cert:
          valueFrom:
            secretKeyRef:
              name: sap-cloud-logging
              namespace: sap-cloud-logging-integration
              key: ingest-otlp-cert
        key:
          valueFrom:
            secretKeyRef:
              name: sap-cloud-logging
              namespace: sap-cloud-logging-integration
```

```
              key: ingest-otlp-key
EOF
```

2. Verify that the `TracePipeline` is running:

```
kubectl get tracepipelines
```

Set Up Istio Tracing

By default, Istio sidecar injection and Istio tracing are disabled in Kyma. To analyze them, you must enable them:

3. Enable Istio sidecar injection for your workload (see Enabling Istio Sidecar Proxy Injection [page 1892]).
4. Configure the Istio `Telemetry` resource to use the `kyma-traces` extension provider based on OTLP (see Configure Istio Tracing [page 2201]).

Ship Metrics To SAP Cloud Logging

You can set up ingestion of metrics from applications and the Istio service mesh to the OTLP endpoint of the SAP Cloud Logging service instance.

Procedure

1. Deploy a `MetricPipeline`:

```
kubectl apply -f - <<EOF
apiVersion: telemetry.kyma-project.io/v1beta1
kind: MetricPipeline
metadata:
  name: sap-cloud-logging
spec:
  input:
    prometheus:
      enabled: false
    istio:
      enabled: false
    runtime:
      enabled: false
  output:
    otlp:
      endpoint:
        valueFrom:
          secretKeyRef:
            name: sap-cloud-logging
            namespace: sap-cloud-logging-integration
            key: ingest-otlp-endpoint
      tls:
        cert:
          valueFrom:
            secretKeyRef:
              name: sap-cloud-logging
              namespace: sap-cloud-logging-integration
              key: ingest-otlp-cert
        key:
          valueFrom:
            secretKeyRef:
              name: sap-cloud-logging
```

```
              namespace: sap-cloud-logging-integration
              key: ingest-otlp-key
EOF
```

The default configuration creates a gateway to receive OTLP metrics from your applications.

2. **Optional:** To collect additional metrics, such as those from the runtime or Istio, configure the presets in the `input` section of the `MetricPipeline`. For the available options, see Collecting Metrics [page 2203].
3. Verify that the `MetricPipeline` is running:

```
kubectl get metricpipelines
```

Set Up Kyma Dashboard Integration

You can add direct links from Kyma dashboard to SAP Cloud Logging.

Procedure

1. If your Secret has a name or namespace different from the example, download the file and edit the `dataSources` section before you apply it.
2. Apply the `ConfigMap`:

```
kubectl apply
-f https://raw.githubusercontent.com/kyma-project/telemetry-manager/main/docs/
user/integration/sap-cloud-logging/kyma-dashboard-configmap.yaml
```

Use SAP Cloud Logging Alerts

You can import predefined alerts for SAP Cloud Logging to monitor the health of your telemetry integration.

Procedure

1. In the SAP Cloud Logging dashboard, define a “notification channel” to receive alert notifications.
2. To import a monitor, use the development tools of the SAP Cloud Logging dashboard.
3. Execute `POST _plugins/_alerting/monitors`, followed by the contents of the respective JSON file.
4. Depending on the pipelines you are using, enable some or all of the following alerts:

   The alerts are based on JSON documents defining a `Monitor` for the alerting plugin.

Monitored Component	File	Description
SAP Cloud Logging	alert-health.json	Monitors the health of the underlying OpenSearch cluster in SAP Cloud Logging using the cluster health API . Triggers if the cluster status becomes **red**.
SAP Cloud Logging	alert-rejection-in-progress.json	Monitors the `cls-rejected-*` index for new data. Triggers if new rejected data is observed.
Kyma Telemetry Integration	alert-telemetry-status.json	Monitors the status of the Telemetry module. Triggers if the module reports a non-ready state.
Kyma Telemetry Integration	alert-log-ingestion.json	Monitors the `LogPipeline`. Triggers if log data stops flowing.
Kyma Telemetry Integration	alert-trace-ingestion.json	Monitors the `TracePipeline`. Triggers if trace data stops flowing to SAP Cloud Logging.
Kyma Telemetry Integration	alert-metric-ingestion.json	Monitors the `MetricPipeline`. Triggers if metric data stops flowing to SAP Cloud Logging.

5. After importing, edit the monitor to attach your notification channel or destination and adjust thresholds as needed.
6. Verify that the new monitor definition is listed among the SAP Cloud Logging alerts.

Use SAP Cloud Logging Dashboards

You can view logs, traces, and metrics in SAP Cloud Logging dashboards. Several dashboards come with SAP Cloud Logging, and you can import additional dashboards as needed.

Procedure

- For the status of the SAP Cloud Logging integration with the Telemetry module, import the file dashboard-status.ndjson .
- For application logs and Istio access logs: Use the preconfigured dashboard *Overview*.
- For traces, use the OpenSearch plugin “Observability”.
- For runtime metrics, import the file dashboard-runtime.ndjson .
- For Istio Pod metrics, import the file dashboard-istio.ndjson .

4.3.1.11.7.2 Migrate Your LogPipeline From HTTP to OTLP

To use the OpenTelemetry Protocol (OTLP) for sending logs, you must migrate your `LogPipeline` from the `http` output to the `otlp` output. With OTLP, you can correlate logs with traces and metrics, collect logs pushed directly from applications, and use features available only for the OTLP-based stack.

Prerequisites

- You have an active Kyma cluster with the Telemetry module added.
- Your observability backend has an OTLP ingestion endpoint. If your backend doesn't support OTLP natively, you must run a custom OTel Collector as gateway between the Telemetry module and the target backend.
- You have one or more `LogPipeline` resources that use the `http` output.

> → Recommendation
>
> If your LogPipeline still uses the `v1alpha1` API, migrate it to `v1beta1`. For details, see Migrate Telemetry Pipelines to v1beta1 [page 2189].

Context

When you want to migrate to the `otlp` output, create a new `LogPipeline`. To prevent data loss, run it in parallel with your existing pipeline. After verifying that the new pipeline works correctly, you can delete the old one.

You can't modify an existing `LogPipeline` to change its output type. You must create a new resource.

In the following sample `LogPipeline`, see the fields that you must change or remove for the migration:

```
apiVersion: telemetry.kyma-project.io/v1beta1
kind: LogPipeline
metadata:
  name: my-http-pipeline
spec:
  input:
    runtime:            # OTLP supports the runtime input, but you must replace
the dropLabels and keepAnnotation flags
      dropLabels: true       # Configure label enrichment centrally
      keepAnnotations: true  # No longer supported
        ...
  output:
    http:                    # Switch to OTLP
      endpoint:
        value: "my-backend:4317"
```

See how the deprecated fields map to their new OTLP-based counterparts:

Deprecated Field	Migration Action
`spec.output.http`	Replace with `spec.output.otlp`.
`spec.output.http.uri`	OTLP uses a predefined index that you can't change. If you need a custom index, use reindexing (see OpenSearch: Reindex Data).
`spec.input.runtime.dropLabels`	Configure label enrichment in the central Telemetry resource instead.
`spec.input.runtime.keepAnnotations`	Remove this functionality. It is not supported with the OTLP output.

Procedure

1. Create a new `LogPipeline` that uses the `otlp` output.

   Pay special attention to the following settings (for details, see Integrate With Your OTLP Backend [page 2231]):

   - Endpoint URL: Use the OTLP-specific ingestion endpoint from your observability backend. This URL is different from the one used for the legacy `http` output.
   - Protocol: The `otlp` output defaults to the gRPC protocol. If your backend uses HTTP, you must include the protocol in the endpoint URL (for example, `https://my-otlp-http-endpoint:4318`).
   - Authentication: The OTLP endpoint often uses different credentials or API permissions than your previous log ingestion endpoint. Verify that your credentials have the necessary permissions for OTLP log ingestion.

   ```
   apiVersion: telemetry.kyma-project.io/v1beta1
   kind: LogPipeline
   metadata:
     name: <my-otlp-pipeline>
   spec:
     output:
       otlp:
         endpoint:
           value: "<my-backend>:4317"
   ```

2. **Optional:** To enrich logs with Pod labels, configure the central `Telemetry` resource (Telemetry CRD).

   In contrast to a Fluent Bit `LogPipeline`, the `otlp` output doesn't automatically add all Pod labels. To continue enriching logs with specific labels, you must explicitly enable it in the `spec.enrichments.extractPodLabels` field.

   > ### Note
   >
   > Enrichment with Pod annotations is no longer supported.

3. Deploy the new `LogPipeline`:

   ```
   kubectl apply -f logpipeline.yaml
   ```

4. Check that the new `LogPipeline` is healthy:

   ```
   kubectl get logpipeline <my-otlp-pipeline>
   ```

5. Check your observability backend to confirm that log data is arriving.
6. Delete the old `LogPipeline`:

   ```
   kubectl delete logpipeline <my-old-pipeline>
   ```

Results

Your cluster now sends logs exclusively through your new OTLP-based `LogPipeline`. Your enrichment logic is preserved.

4.3.1.11.8 Monitor Pipeline Health

The Telemetry module is designed to be reliable and resilient. However, there may be situations when the instances drop data or cannot handle the load, and you must take action.

Overview

The Telemetry module automatically handles temporary issues to prevent data loss and ensure that the OTel Collector instances of your pipelines are operational and healthy. For example, if your backend is temporarily unavailable, the module buffers your data and attempts to resend it when the connection is restored.

The Telemetry module continuously monitors the health of your pipelines (see Self Monitor [page 2255]). To ensure that your Telemetry pipelines operate reliably, you can monitor their health data in the following ways:

- Perform manual checks by inspecting the status conditions of your pipeline resources with `kubectl`.
- Set up continuous monitoring by using a `MetricPipeline` to export health metrics to your observability backend, where you can set up dashboards and alerts.

Check Pipeline Status

For a quick check, you can inspect the `status` of a pipeline resource directly.

1. Run `kubectl get` for the pipeline that you want to inspect:
   - For `LogPipeline`: **`kubectl get logpipeline <your-pipeline-name>`**
   - For `TracePipeline`: **`kubectl get tracepipeline <your-pipeline-name>`**
   - For `MetricPipeline`: **`kubectl get metricpipeline <your-pipeline-name>`**
2. Review the output. A healthy pipeline shows `True` for all status conditions.

Output Code

```
NAME      CONFIGURATION GENERATED   GATEWAY HEALTHY   FLOW HEALTHY
backend   True                      True              True
```

3. If any condition is `False`, investigate the problem and fix it.

To understand the meaning of each status condition, see the detailed reference for each pipeline type:

- LogPipeline Status
- TracePipeline Status
- MetricPipeline Status

Set Up Health Monitoring and Alerts

For production environments, set up continuous monitoring by exporting the health metrics to your observability backend, where you can create dashboards and configure alerts. For an example, see Integrate With SAP Cloud Logging [page 2235].

> ⚠ Caution
>
> Do not scrape the metrics endpoint of the OpenTelemetry Collector instances. These metrics are an internal implementation detail and are subject to breaking changes when the underlying Collector is updated. For stable health monitoring, rely on the status conditions of your `LogPipeline`, `MetricPipeline`, or `TracePipeline` custom resources.

To collect these health metrics, you must have at least one active `MetricPipeline` in your cluster. This pipeline automatically collects and exports health data for all of your pipelines, including `LogPipeline` and `TracePipeline` resources.

The Telemetry module emits the following metrics for health monitoring:

- `kyma.resource.status.conditions`: Represents the status of a specific condition on a resource. It is available for all pipelines and the main `Telemetry` resource.
  Values: `1` (“True”), `0` (“False”), or `-1` (“Unknown”)
  Specific attributes:
  - `metric.attributes.type`: The type of the status condition
  - `metric.attributes.status`: The status of the condition
  - `metric.attributes.reason`: A programmatic identifier indicating the reason for the condition's last transition
  - `metric.attributes.name`: The name of the resource
  - `metric.attributes.namespace`: The namespace of the resource
  - `metric.attributes.group`: The API group of the resource
  - `metric.attributes.version`: The API version of the resource
  - `metric.attributes.kind`: The kind of the resource
- `kyma.resource.status.state`: Represents the overall state of the main `Telemetry` resource.
  Values: `1` (“Ready”) or `0` (“Not Ready”)

Specific attributes:

- `metric.attributes.state`: The value of the `status.state` field
- `metric.attributes.name`: The name of the resource
- `metric.attributes.namespace`: The namespace of the resource
- `metric.attributes.group`: The API group of the resource
- `metric.attributes.version`: The API version of the resource
- `metric.attributes.kind`: The kind of the resource

To create an alert, define a rule that triggers on a specific metric value. For example, to create an alert that fires if a pipeline's `TelemetryFlowHealthy` condition becomes “False” (indicating data flow issues), use the following PromQL query:

```
min by (name) ((kyma_resource_status_conditions{type="TelemetryFlowHealthy",
kind="metricpipelines"})) == 0
```

If there are issues with one of the pipelines, see Troubleshooting for the Telemetry Module [page 2245].

Deprecated Attributes

> ⚠ Caution
>
> The following resource attributes are deprecated and will be removed in a future release. Update your dashboards and alert rules to use the replacement `metric.attributes.*` attributes.

Deprecated Resource Attribute	Replacement Metric Attribute
`k8s.resource.name`	`metric.attributes.name`
`k8s.resource.namespace.name`	`metric.attributes.namespace`
`k8s.resource.group`	`metric.attributes.group`
`k8s.resource.version`	`metric.attributes.version`
`k8s.resource.kind`	`metric.attributes.kind`

4.3.1.11.9 Troubleshooting for the Telemetry Module

Troubleshoot problems related to the Telemetry module and its pipelines.

If you can't find a solution, don't hesitate to create a GitHub issue .

No Data Arrive at the Backend

Symptom

- No data arrive at the backend.
- In the pipeline status, the `TelemetryFlowHealthy` condition has status `GatewayAllTelemetryDataDropped` or `AgentAllTelemetryDataDropped`.

Cause

The pipeline cannot connect to the backend and drops all data, typically because of one of the following reasons:

- Authentication Error: The credentials in your `MetricPipeline` output are incorrect.
- Network Unreachable: The backend URL is wrong, a firewall is blocking the connection, or there's a DNS issue preventing the agent or gateway from reaching the backend.
- Backend is Down: The observability backend itself is not running or is unhealthy.

Solution

1. Identify the failing component.
   - If the status is `GatewayAllTelemetryDataDropped`, the problem is with the gateway.
   - If the status is `AgentAllTelemetryDataDropped`, the problem is with the agent.
2. To check the failing component's logs, call **`kubectl logs -n kyma-system <POD_NAME>`**:
   - For the gateway, check Pod `telemetry-otlp-gateway`.
   - For the agent, check Pod `telemetry-<log/metric>-agent`.

   Look for errors related to authentication, connectivity, and DNS.
3. Check if the backend is up and reachable.
4. Based on the log messages, fix the `output` section of your pipeline and re-apply it.

Not All Data Arrive at the Backend

Symptom

- The backend is reachable and the connection is properly configured, but some data points are refused.
- In the pipeline status, the `TelemetryFlowHealthy` condition has status `GatewaySomeTelemetryDataDropped` or `AgentSomeTelemetryDataDropped`.

Cause

This status indicates that the telemetry gateway or agent is successfully sending data, but the backend is rejecting some of it. Common reasons are:

- Rate Limiting: Your backend is rejecting requests because you're sending too much data at once.
- Invalid Data: Your backend is rejecting specific data due to incorrect formatting, invalid labels, or other schema violations.

Solution

1. Check the error logs for the affected Pod by calling `kubectl logs -n kyma-system {POD_NAME}`:
   - For `GatewaySomeTelemetryDataDropped`, check Pod `telemetry-otlp-gateway`.
   - For `AgentSomeTelemetryDataDropped`, check Pod `telemetry-<log/metric>-agent`.
2. Go to your observability backend and investigate potential causes.
3. If the backend is limiting the rate by refusing data, try the following options:
   - Increase the ingestion rate of your backend (for example, by scaling out your SAP Cloud Logging instances).
   - Reduce emitted data by re-configuring the pipeline (for example, by disabling certain inputs or applying filters).
   - Reduce emitted data in your applications.
4. Otherwise, fix the issues as indicated in the logs.

Gateway Throttling

Symptom

In the pipeline status, the `TelemetryFlowHealthy` condition has status `GatewayThrottling`.

Cause

The OTLP Gateway instance is receiving data faster than it can process and forward it.

Solution

Reduce the volume of telemetry data, either by rebalancing workloads across nodes or reconfiguring your pipeline(s) to filter out unused inputs and irrelevant data.

Log Buffer Filling Up

Symptom

In the `LogPipeline` status, the `TelemetryFlowHealthy` condition has status `AgentBufferFillingUp`.

Cause

The backend ingestion rate is too low compared to the export rate of the Log Agent, causing data to accumulate in its buffer.

Solution

You can either increase the capacity of your backend or reduce the volume of log data being sent. Try one of the following options:

- Increase the ingestion rate of your backend (for example, by scaling out your SAP Cloud Logging instances).
- Reduce emitted data by re-configuring the pipeline (for example, by disabling certain inputs or applying namespace filters).

- Reduce the amount of log data generated by your applications.

Transform or Filter Rule Has No Effect

Symptom

You have configured a `transform` or `filter` section in your pipeline, but the data arriving at your backend is not modified, or data you expect to be dropped is still present.

Cause

This usually happens for one of the following reasons:

- Incorrect execution order: You're filtering data based on a field's original value, but a transformation rule has already changed it. Transformation rules always run before filter rules.
- Condition never met: The condition in your rule is valid (otherwise, you'd see pipeline condition `ConfigurationGenerated`: `False` with the reason `OTTLSpecInvalid`) but never finds a match in the data. This is often due to a case-sensitive value mismatch or a flawed regular expression.

Solution

1. Review your rules and verify the execution order. For example, if you have a `transform` rule that renames `resource.attributes["foo"]` to `resource.attributes["bar"]`, your `filter` rule must check for “bar”, not “foo”.
2. Test your regex separately. Simplify complex conditions to a single comparison and re-apply.
3. To test your rules, temporarily remove all but one rule to confirm it works as expected. Then, add your other rules incrementally and isolate the rule that is causing the issue.

OTTL Spec Invalid with Unspecific Error Message

Symptom

- In the pipeline status, you see the condition `ConfigurationGenerated` with status `False` and reason `OTTLSpecInvalid`.
- The pipeline configuration fails with unclear error messages, for example, mentioning “unexpected token `<EOF>`” or EOF (End of File) parsing errors, such as the following example:

  Output Code

  ```
  'Invalid FilterSpec: condition has invalid syntax: 1:64: unexpected token
       "<EOF>" (expected <opcomparison> Value)'
  ```

Cause

If you get a generic EOF error instead of a specific error message, there's usually a syntax error in your OTTL transformation or filter rules. It occurs when the parser cannot diagnose the error precisely.

The following example uses the incorrect function name `isMatch` (it should be `IsMatch`, because the parser is case-sensitive):

```
# ...
filter:
    - conditions:
        - 'isMatch(resource.attributes["k8s.namespace.name"], ".*-system")'
```

Solution

Review the syntax of your transform and filter rules and ensure that the names of OTTL functions are spelled correctly (for example, `IsMatch()` instead of `isMatch()`).

MetricPipeline: Failed to Scrape Prometheus Endpoint

Symptom

- Custom metrics don't arrive at the destination.
- In the Metric Agent (OTel Collector) logs, you see entries saying `Failed to scrape Prometheus endpoint` like the following:

  Output Code

  ```
  2023-08-29T09:53:07.123Z warn internal/transaction.go:111 Failed to
  scrape Prometheus endpoint {"kind": "receiver", "name": "prometheus/
  app-pods", "data_type": "metrics", "scrape_timestamp": 1693302787120,
  "target_labels": "{__name__=\"up\", instance=\"10.42.0.18:8080\",
  job=\"app-pods\"}"}
  ```

Cause

There's a configuration or network issue between the Metric Agent and your application, such as:

- The Service that exposes your metrics port doesn't specify the application protocol.
- The workload is not configured to use `STRICT` mTLS mode, which the Metric Agent uses by default.
- A deny-all `NetworkPolicy` in your application's namespace prevents the agent from scraping metrics from annotated workloads.

Solution

- Define the application protocol in the Service port definition by either prefixing the port name with the protocol, or define the `appProtocol` attribute.
- If the issue is with mTLS, either configure your workload to use `STRICT` mTLS, or switch to unencrypted scraping by adding the `prometheus.io/scheme: "http"` annotation to your workload.
- Create a new `NetworkPolicy` to explicitly allow ingress traffic from the Metric Agent; such as the following example:

  ```
  apiVersion: networking.k8s.io/v1
  kind: NetworkPolicy
  metadata:
    name: allow-traffic-from-agent
  spec:
    podSelector:
  ```

```
    matchLabels:
      app.kubernetes.io/name: "annotated-workload" # <your workload here>
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kyma-system
      podSelector:
        matchLabels:
          telemetry.kyma-project.io/metric-scrape: "true"
  policyTypes:
  - Ingress
```

Custom Spans Don't Arrive at the Backend, but Istio Spans Do

Symptom

You see traces generated by the Istio service mesh, but traces from your own application code (custom spans) are missing.

Cause

The OpenTelemetry (OTel) SDK version used in your application is incompatible with the OTel Collector version.

Solution

1. Check which SDK version you're using for instrumentation.
2. Investigate whether it's compatible with the OTel Collector version.
3. If necessary, upgrade to a supported SDK version.

Too Few Traces Arrive at the Backend

Symptom

The observability backend shows significantly fewer traces than the number of requests your application receives.

Cause

By default, Istio samples only 1% of requests for tracing to minimize performance overhead (see Configure Istio Tracing [page 2201]).

For example, in low-traffic environments (for development or testing) or for low-traffic services, the request volume can be so low that a 1% sample rate may result in capturing zero traces.

Solution

- To see more traces in the backend, increase the percentage of requests that are sampled (see Configure the Sampling Rate [page 2202]).
- Alternatively, to trace a single request, force sampling by adding a `traceparent` HTTP header to your client request. This header contains a sampled flag that instructs the system to capture the trace, bypassing the global sampling rate (see Trace Context: Sampled Flag).

VPA Resources Are Not Created

Symptom

VPA resources are not created even though VPA is enabled.

Cause

The Vertical Pod Autoscaler (VPA) CRD is not installed in your cluster.

Solution

Install the VPA CRD in your cluster (see Vertical Pod Autoscaler).

Telemetry Components Run Out of Memory

Symptom

Telemetry components are running out of memory despite VPA being enabled.

Cause

The memory limits are too restrictive for your telemetry volume. VPA calculates limits based on the smallest node in your cluster, which might not provide enough memory for high-volume telemetry workloads.

Solution

If no data arrives at your backend, first check for backend-side issues (see Not All Data Arrive at the Backend [page 2246]).

If data arrives but components run out of memory, reduce memory pressure:

- Add nodes with more memory to increase the calculated maxAllowed value.
- Reduce telemetry volume by applying filters in your pipelines (see Filter Logs [page 2214], Filter Traces [page 2217], Filter Metrics [page 2218]).

As a workaround, you can disable VPA so that the system uses static resource limits:

1. Edit the Telemetry resource:

   ```
   kubectl edit telemetry default -n kyma-system
   ```

2. Add or change the `telemetry.kyma-project.io/enable-vpa` annotation under `metadata.annotations` to false:

   ```
   apiVersion: operator.kyma-project.io/v1beta1
   kind: Telemetry
   metadata:
     name: default
     namespace: kyma-system
     annotations:
       telemetry.kyma-project.io/enable-vpa: "false"
   spec:
     # your telemetry configuration
   ```

3. Save your changes.

4. Verify the configuration by listing VPA resources in the `kyma-system` namespace:

   ```
   kubectl get vpa -n kyma-system
   ```

   If VPA is disabled, no VPA resources appear in the namespace.
5. If you disabled VPA temporarily for debugging, you can re-enable it later by removing the annotation or setting its value to true.

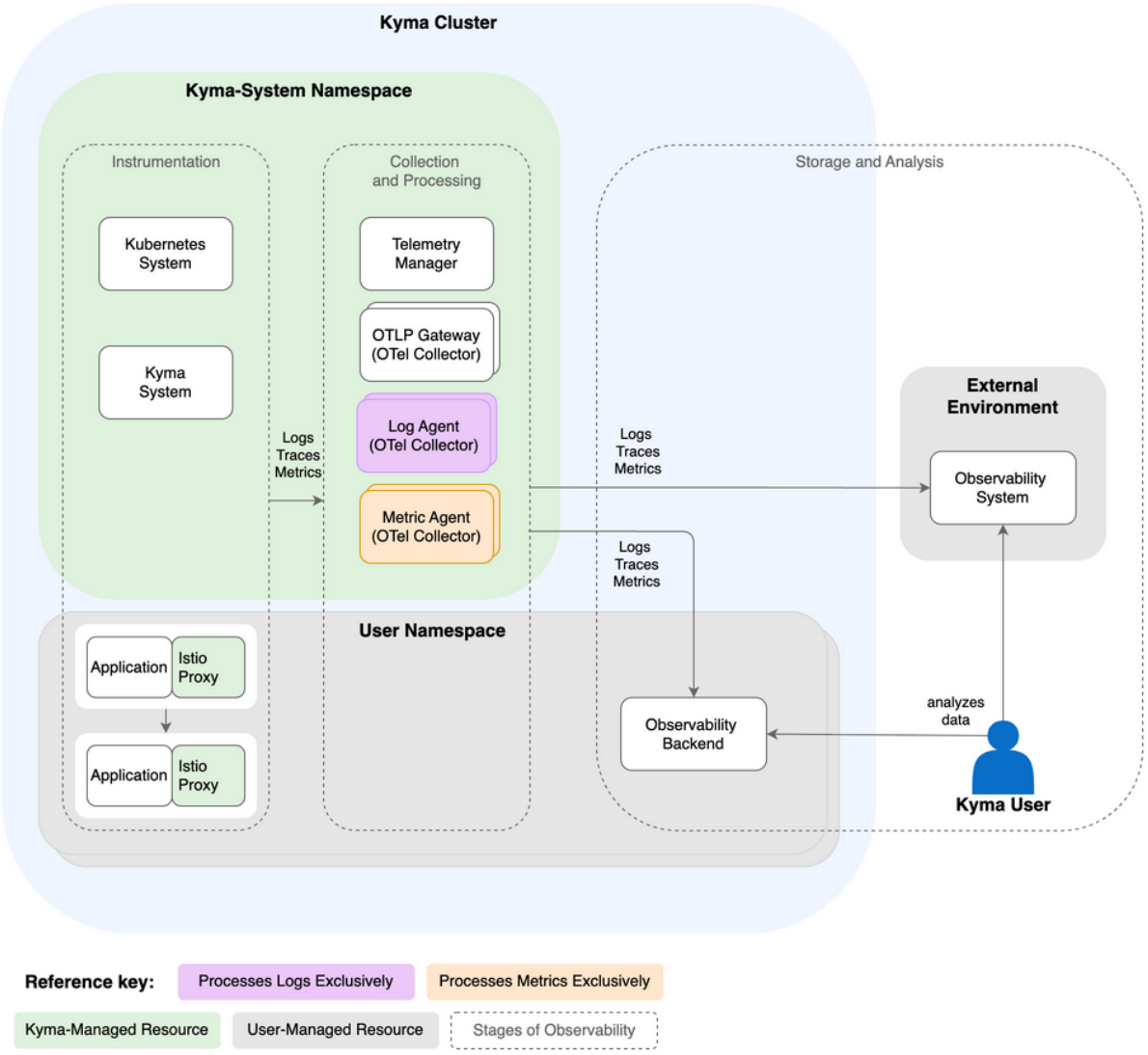
4.3.1.11.10 Telemetry Architecture

The Telemetry module consists of a manager component, which continuously watches the user-provided pipeline resources and deploys the respective OTel Collectors. Learn more about the architecture and how the components interact.

Overview

The Telemetry API provides a robust, preconfigured OpenTelemetry (OTel) Collector setup that abstracts its underlying complexities. This approach delivers several key benefits:

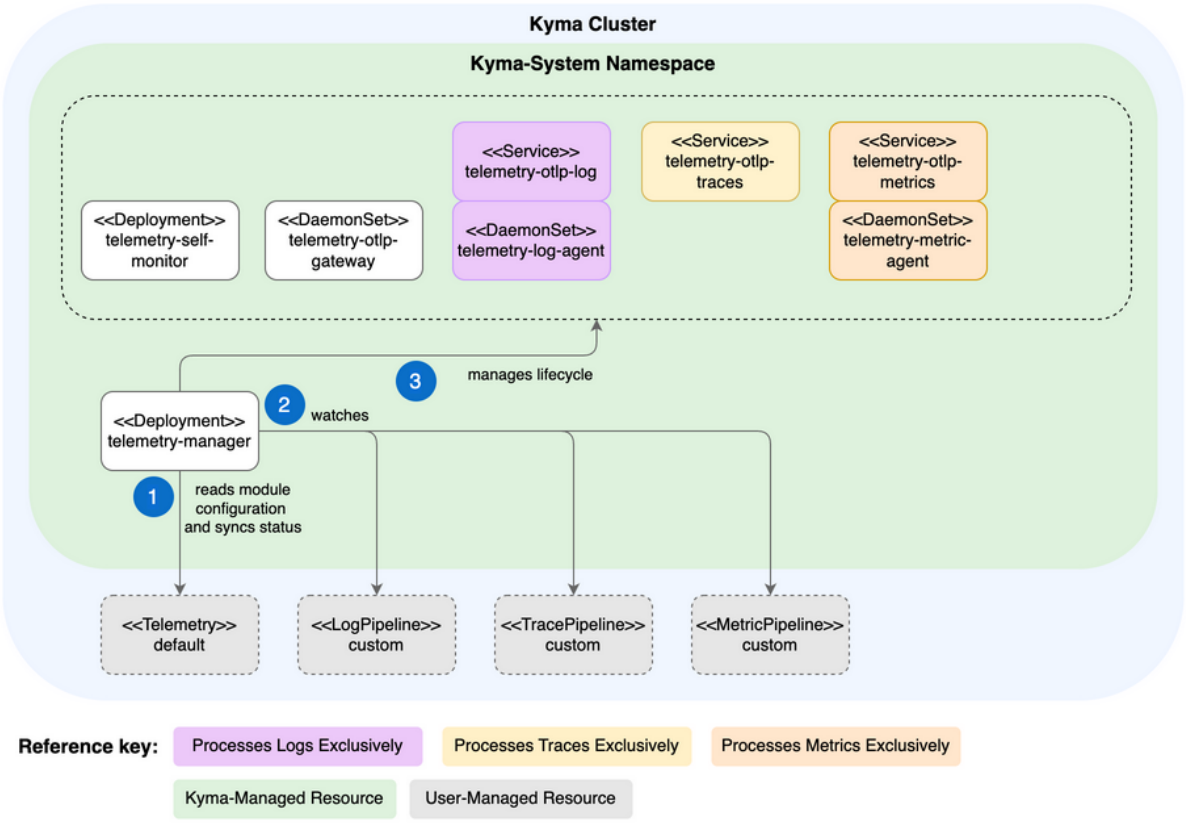
- Compatibility: Maintains stability and functionality even as underlying OTel Collector features evolve, reducing the need for constant updates on your end.
- Migratability: Facilitates smooth transitions when you switch underlying technologies or architectures.
- Native Kubernetes Support: Offers seamless integration with Secrets, for example, served by the SAP BTP Service Operator, and the Telemetry Manager automatically handles the full lifecycle of all components.
- Focus: Reduces the need to understand intricate underlying OTel Collector concepts, allowing you to focus on your application development.



Telemetry Manager

Telemetry Manager, the core component of the module, is a Kubernetes operator that implements the Kubernetes controller pattern and manages the whole lifecycle of all other Telemetry components. It performs the following tasks:

1. Watch the module configuration for changes and sync the module status to it.
2. Watch the user-created Kubernetes resources `LogPipeline`, `TracePipeline`, and `MetricPipeline`. In these resources, you specify what data of a signal type to collect and where to ship it.
3. Manage the lifecycle of the self monitor and of the components handling the incoming telemetry data: the OTLP Gateway and the signal-specific agents.



OTLP Gateway

If at least one valid pipeline of any type (`LogPipeline`, `TracePipeline`, or `MetricPipeline`) exists, Telemetry Manager deploys the OTLP Gateway.

The OTLP Gateway acts as the central endpoint where your applications push telemetry data (logs, traces, and metrics) using the OTLP format. It enriches and filters this data, then dispatches it to the backends you configure in your pipeline resources. The gateway runs as an OTel Collector -based DaemonSet with one instance per cluster node.

Applications can send data to the gateway using these service endpoints:

- gRPC: `http://telemetry-otlp.kyma-system:4317`
- HTTP: `http://telemetry-otlp.kyma-system:4318`

Agents

Agents collect telemetry data directly from each node in your cluster. Unlike the OTLP Gateway, which receives pushed data, agents actively pull data from sources on the node.

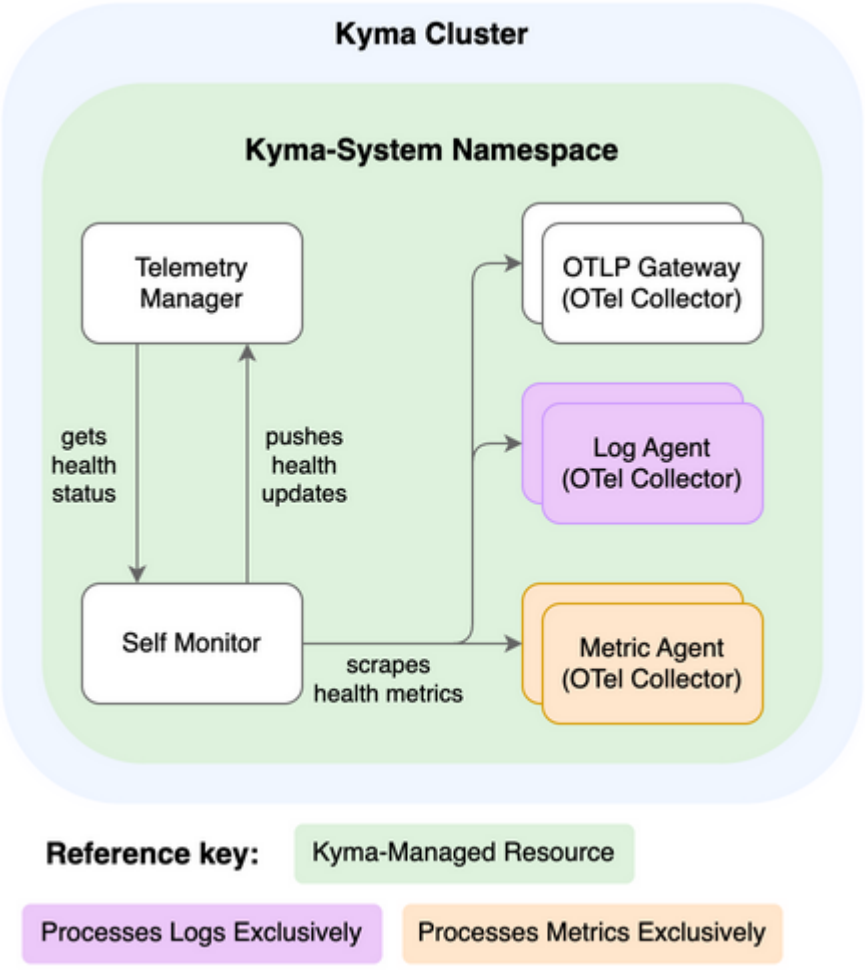
Depending on your `LogPipeline` and `MetricPipeline` configuration, Telemetry Manager deploys the following agents as an OTel Collector -based DaemonSet :

- The Log Agent collects logs from the stdout/stderr output of all containers on a node. For details, see Logs Architecture [page 2256].
- The Metric Agent scrapes Prometheus-annotated workloads on each node. For details, see Metrics Architecture [page 2260].

Self Monitor

The Telemetry module includes a Prometheus -based self monitor that collects and evaluates health metrics from the OTLP Gateway and signal-specific agents. Telemetry Manager uses this data to report the current health status in your pipeline resources.

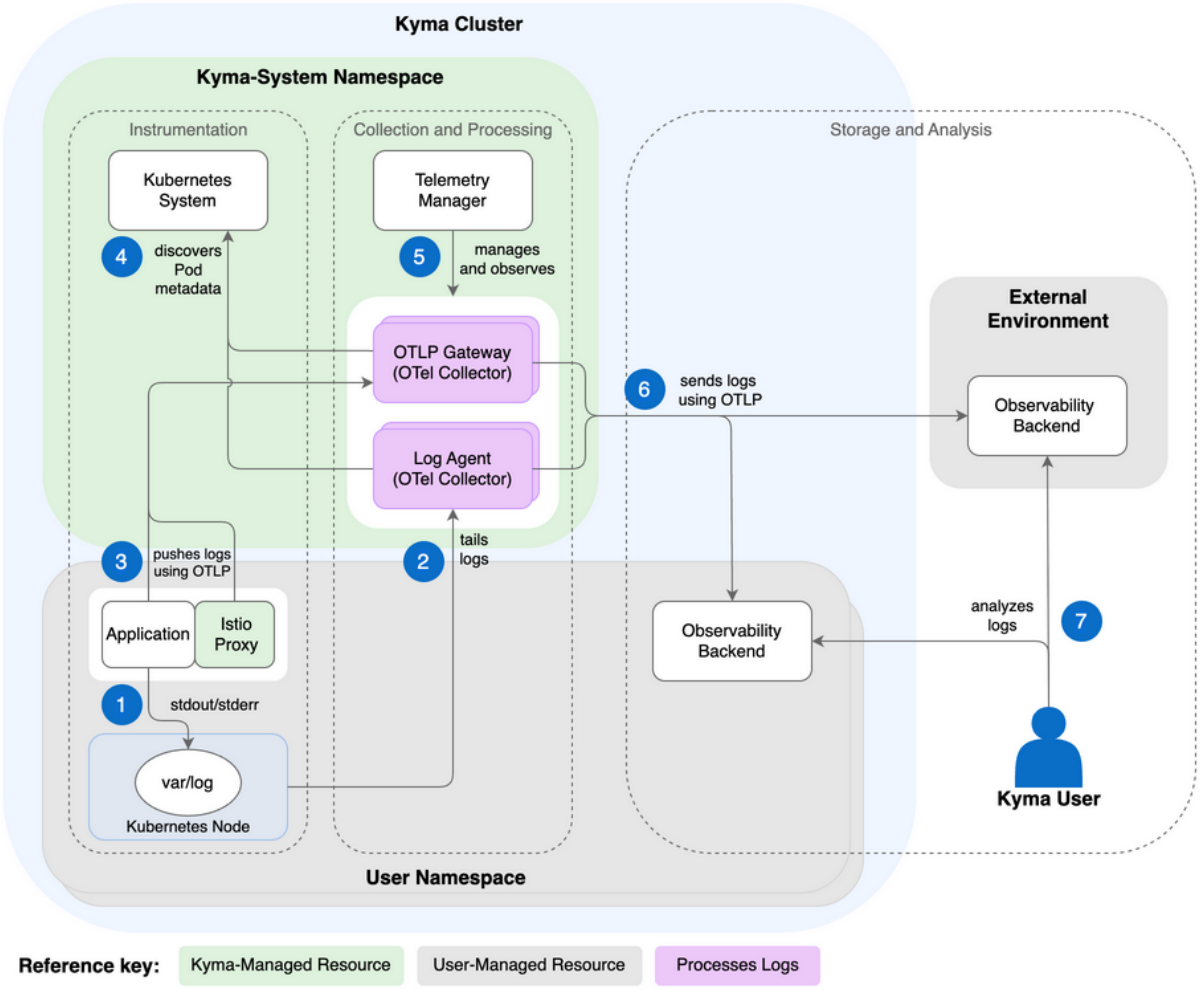
You can also use these health metrics in your own observability backend to set up alerts and dashboards for your telemetry pipelines. For details, see Monitor Pipeline Health [page 2243].



4.3.1.11.10.1 Logs Architecture

For log collection, the Telemetry module provides the OTLP Gateway and an optional Log Agent. To control their behavior and data destination, you define a `LogPipeline`. The OTLP Gateway is a DaemonSet with one instance per node that receives OTLP logs pushed from your applications. The Log Agent is a DaemonSet that pulls container logs from each node.

For details, see OTLP Gateway [page 2254] and Agents [page 2254].

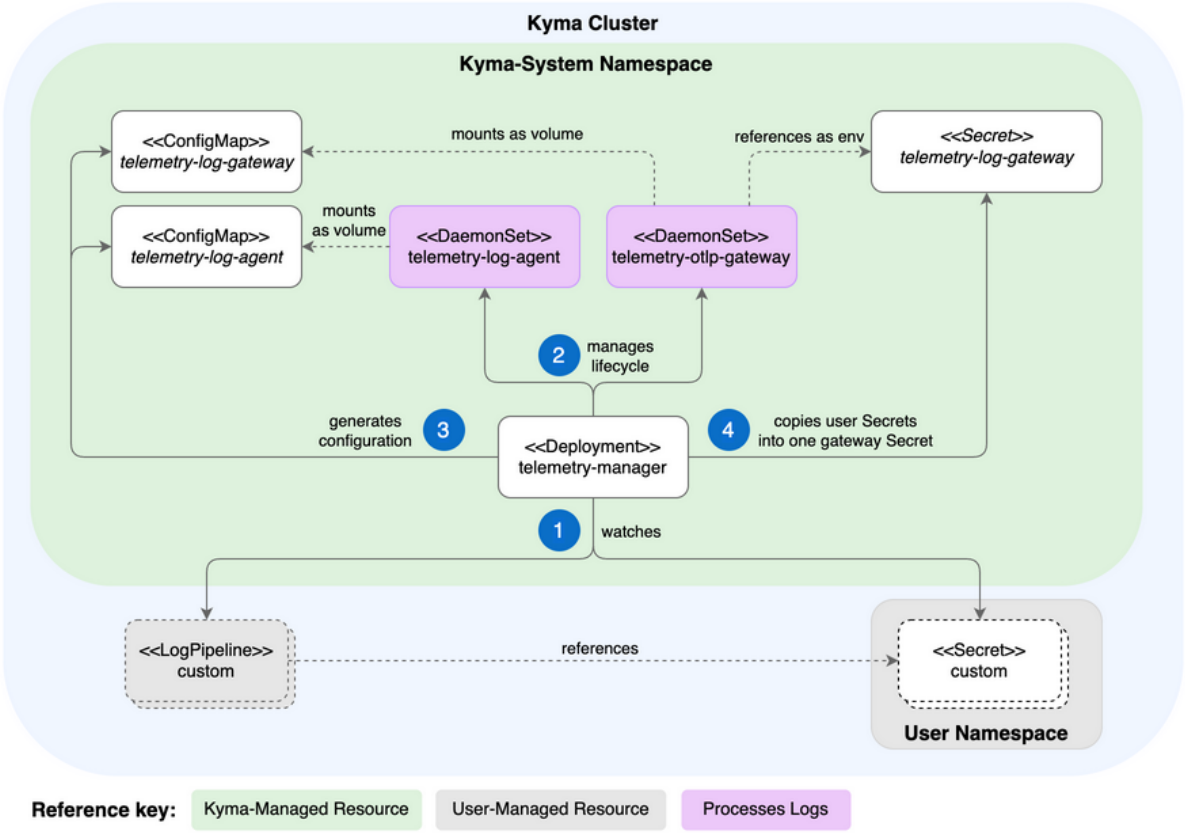


1. Application containers print JSON logs to the `stdout/stderr` channel and are stored by the Kubernetes container runtime under the `var/log` directory and its subdirectories at the related Node. Istio is configured to write access logs to `stdout` as well.
2. If you choose to use the agent, an OTel Collector runs as a DaemonSet (one instance per Node), detects any new log files in the folder, and tails and parses them.
3. An application (exposing logs in OTLP) sends logs to the OTLP Gateway using the `telemetry-otlp` service. Because the Service uses node-local routing, the OTLP Gateway instance on the same node always receives the data. Istio is configured to push access logs with OTLP as well.
4. The OTLP Gateway and Log Agent discover the metadata and enrich all received data with metadata of the source by communicating with the Kubernetes APIServer. Furthermore, they filter data according to the pipeline configuration.

5. Telemetry Manager configures the Log Agent and the OTLP Gateway according to the `LogPipeline` resource specification, including the target backend. Also, it observes the logs flow to the backend and reports problems in the LogPipeline status.
6. The OTLP Gateway and Log Agent send the data to the observability backend that's specified in your `LogPipeline` resource - either within your cluster, or, if authentication is set up, to an external observability backend.
7. You can analyze the logs data with your preferred observability backend.

Telemetry Manager

The `LogPipeline` resource is watched by Telemetry Manager, which is responsible for generating the configurations for the OTLP Gateway and the Log Agent.



1. Telemetry Manager watches all `LogPipeline` resources and related Secrets.
2. Furthermore, Telemetry Manager takes care of the full lifecycle of the OTLP Gateway DaemonSet and the Log Agent DaemonSet. Only if you defined a `LogPipeline`, the gateway and agent are deployed.
3. Whenever the user configuration changes, Telemetry Manager validates it and generates a single configuration for the OTLP Gateway and agent.
4. Referenced Secrets are copied into one Secret that is mounted to the OTLP Gateway as well.

OTLP Gateway

In your cluster, the OTLP Gateway is the central component to which all components can send their individual logs. The gateway collects, enriches, and dispatches the data to the configured backend. The OTLP Gateway handles all signal types (traces, metrics, and logs) in a single unified DaemonSet. For more information, see Set Up the OTLP Input [page 2192].

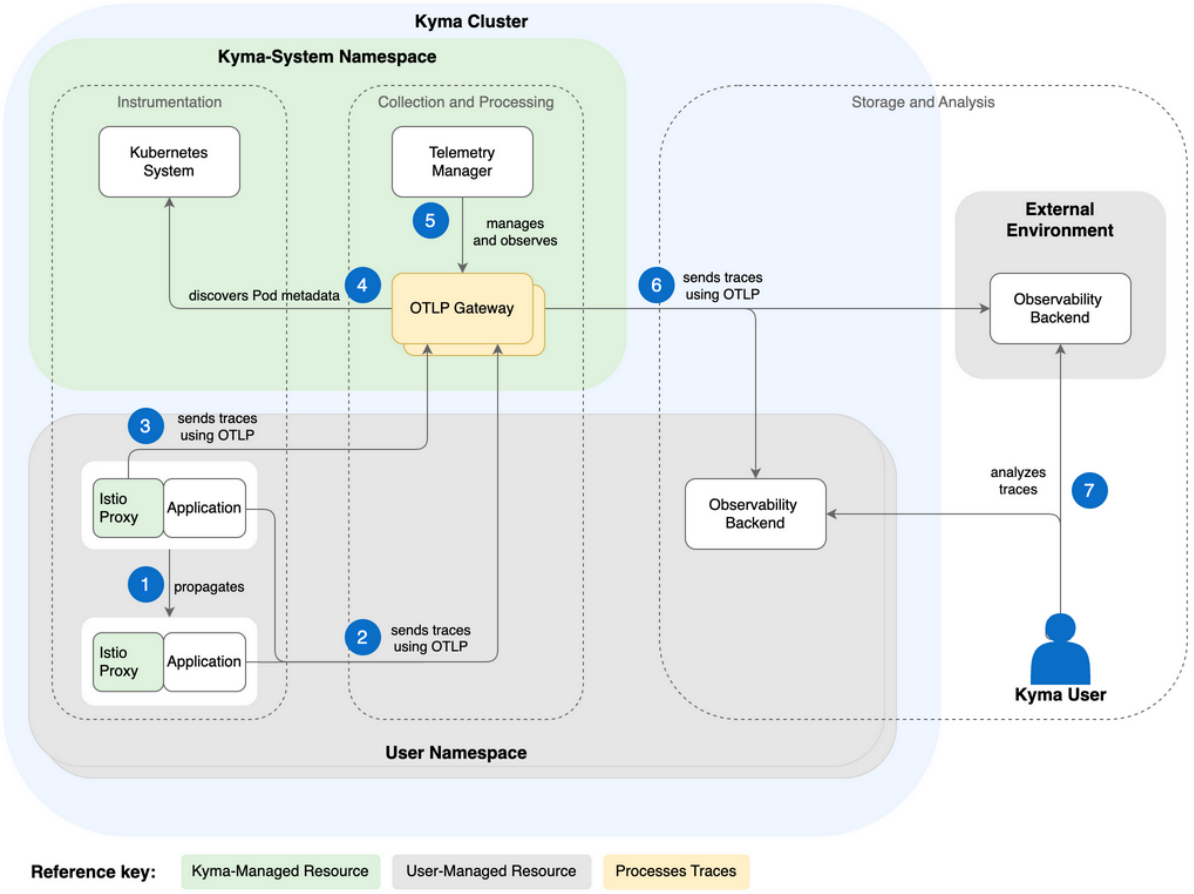
Log Agent

If you enable a log input in your `LogPipeline`, Telemetry Manager deploys a Log Agent. This agent is an OTel Collector -based DaemonSet that collects and converts logs from the container runtime. When your workload containers write structured logs to `stdout` or `stderr`, the agent collects, parses, and enriches them. Finally, it sends the logs in OTLP format to the backend you configured in the pipeline.

4.3.1.11.10.2 Traces Architecture

For trace collection, the Telemetry module provides the OTLP Gateway. To control its behavior and data destination, you define a `TracePipeline`. The OTLP Gateway is a DaemonSet with one instance per node that receives OTLP traces pushed from your applications.

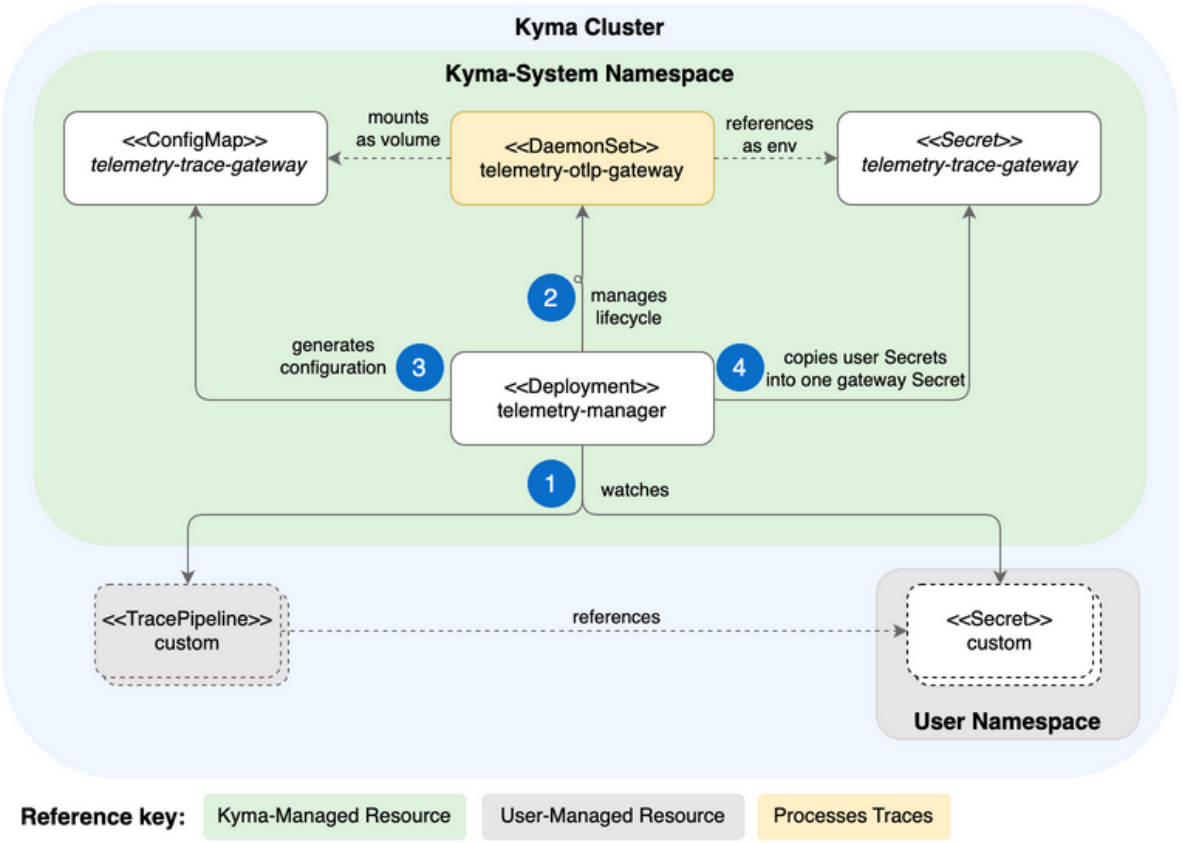
For details, see OTLP Gateway [page 2254].



1. An end-to-end request is triggered and populated across the distributed application. Every involved component propagates the trace context using the W3C Trace Context protocol.
2. After contributing a new span to the trace, the involved components send the related span data (OTLP) to the OTLP Gateway using the `telemetry-otlp` service. Because the Service uses node-local routing, the OTLP Gateway instance on the same node always receives the data.
3. Istio sends the related span data to the OTLP Gateway as well.
4. The OTLP Gateway discovers metadata that's typical for sources running on Kubernetes, like Pod identifiers, and then enriches the span data with that metadata.
5. Telemetry Manager configures the gateway according to the `TracePipeline` resource, including the target backend. Also, it observes the trace flow to the backend and reports problems in the `TracePipeline` status.
6. The OTLP Gateway sends the data to the observability backend that's specified in your `TracePipeline` resource - either within your cluster, or, if authentication is set up, to an external observability backend.
7. You can analyze the trace data with your preferred observability backend.

Telemetry Manager

The `TracePipeline` resource is watched by Telemetry Manager, which is responsible for generating the custom parts of the OTLP Gateway configuration.



1. Telemetry Manager watches all `TracePipeline` resources and related Secrets.

2. Furthermore, Telemetry Manager takes care of the full lifecycle of the OTLP Gateway DaemonSet.
3. Whenever the configuration changes, Telemetry Manager validates it and generates a new configuration for the OTLP Gateway, which is stored in a ConfigMap.
4. Referenced Secrets are copied into a single Secret that is mounted to the OTLP Gateway Pods.

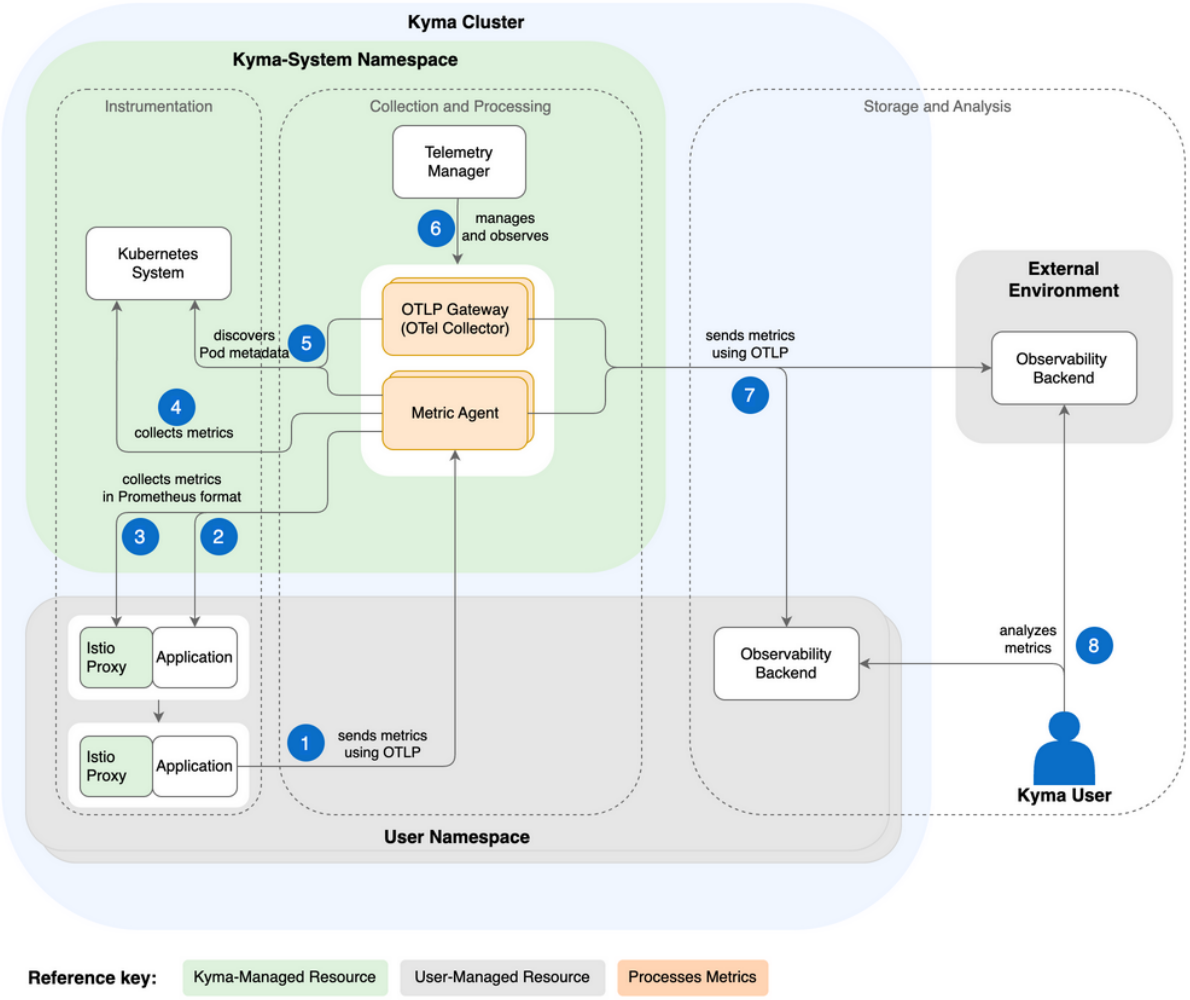
OTLP Gateway

In your cluster, the OTLP Gateway is the central component to which all components can send their individual spans. The gateway collects, enriches, and dispatches the data to the configured backend. The OTLP Gateway handles all signal types (traces, metrics, and logs) in a single unified DaemonSet. For more information, see Set Up the OTLP Input [page 2192].

4.3.1.11.10.3 Metrics Architecture

For metrics collection, the Telemetry module provides the OTLP Gateway and an optional Metric Agent. To control their behavior and data destination, you define a `MetricPipeline`. The OTLP Gateway is a DaemonSet with one instance per node that receives OTLP metrics pushed from your applications. The Metric Agent is a DaemonSet that pulls metrics from Prometheus-annotated endpoints.

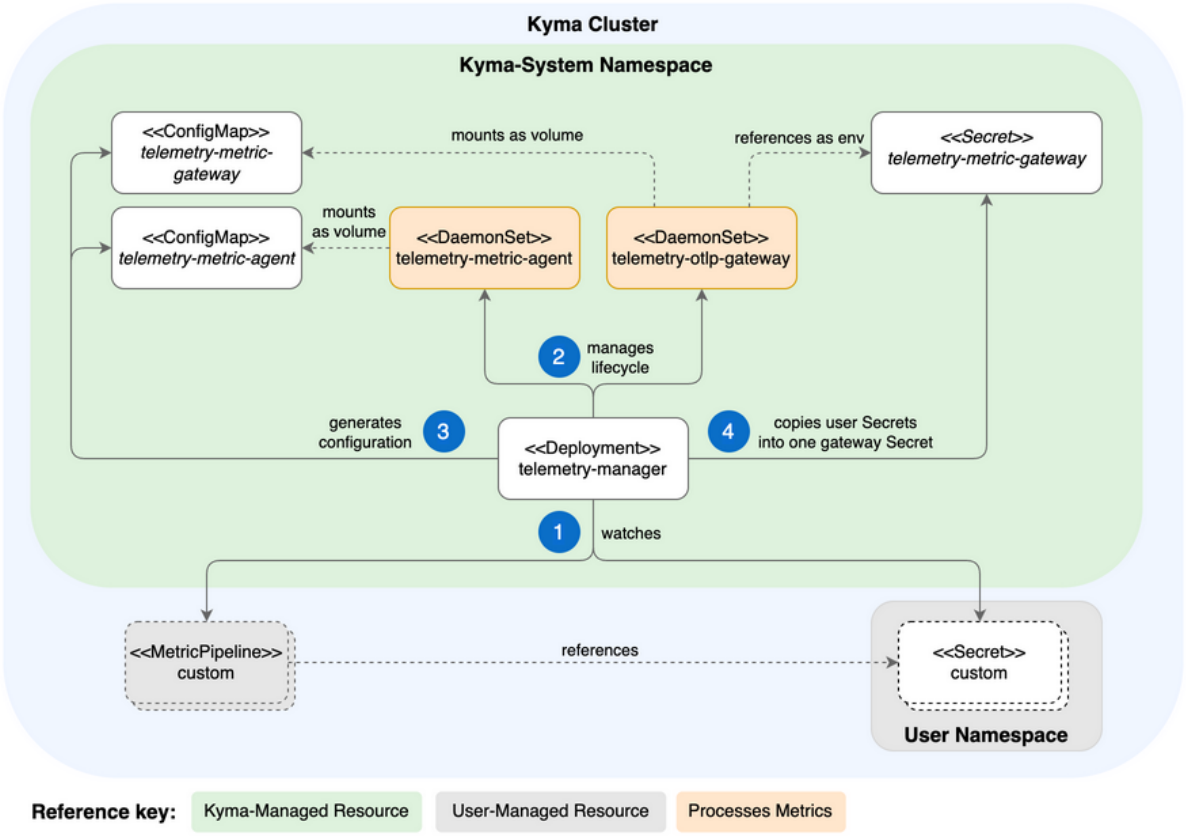
For details, see OTLP Gateway [page 2254] and Agents [page 2254].



1. An application (exposing metrics in OTLP) sends metrics to the OTLP Gateway using the `telemetry-otlp` service. Because the Service uses node-local routing, the OTLP Gateway instance on the same node always receives the data.
2. An application (exposing metrics in Prometheus protocol) activates the agent to scrape the metrics with an annotation-based configuration.
3. Additionally, you can activate the agent to pull metrics of each Istio sidecar.
4. The agent supports collecting metrics from the Kubelet and Kubernetes APIServer.
5. The OTLP Gateway and the Metric Agent discover the metadata and enrich all received data with typical metadata of the source by communicating with the Kubernetes APIServer. Furthermore, they filter data according to the pipeline configuration.
6. Telemetry Manager configures the agent and the OTLP Gateway according to the `MetricPipeline` resource specification, including the target backend. Also, it observes the metrics flow to the backend and reports problems in the `MetricPipeline` status.
7. The OTLP Gateway and the agent send the data to the observability backend that's specified in your `MetricPipeline` resource - either within your cluster, or, if authentication is set up, to an external observability backend.
8. You can analyze the metric data with your preferred observability backend.

Telemetry Manager

The `MetricPipeline` resource is watched by Telemetry Manager, which is responsible for generating the configurations for the OTLP Gateway and the Metric Agent.



1. Telemetry Manager watches all `MetricPipeline` resources and related Secrets.
2. Furthermore, Telemetry Manager takes care of the full lifecycle of the OTLP Gateway DaemonSet and the agent DaemonSet. Only if you defined a `MetricPipeline`, the gateway and agent are deployed.
3. Whenever the user configuration changes, Telemetry Manager validates it and generates a single configuration for the OTLP Gateway and agent.
4. Referenced Secrets are copied into one Secret that is mounted to the OTLP Gateway as well.

OTLP Gateway

In your cluster, the OTLP Gateway is the central component to which all applications can send their individual metrics. The gateway collects, enriches, and dispatches the data to the configured backend. The OTLP Gateway handles all signal types (traces, metrics, and logs) in a single unified DaemonSet. For more information, see Set Up the OTLP Input [page 2192].

Metric Agent

If you enable a metric input in your `MetricPipeline`, Telemetry Manager deploys a Metric Agent. This agent is an OTel Collector -based DaemonSet that collects and converts Prometheus-based metrics. To have the agent scrape your workload, you add a `prometheus.io/scrape` annotation to the Pod or Service specification. The agent then automatically discovers and collects metrics from the annotated endpoints.

4.3.1.11.10.4 Istio Integration

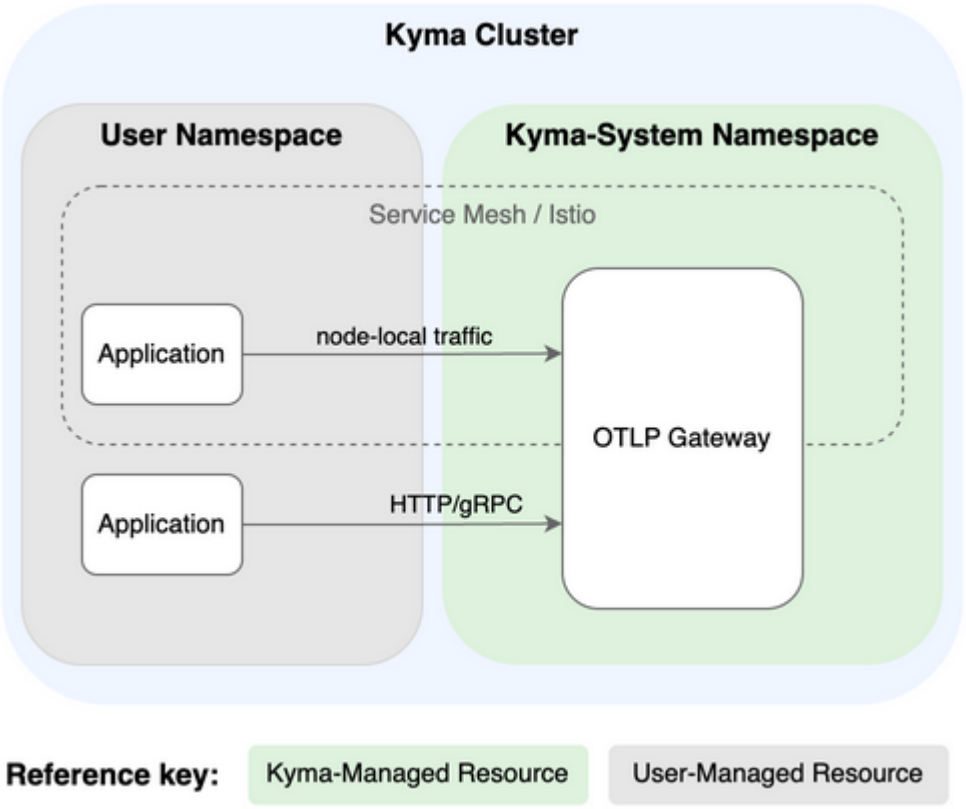
When you have the Istio module in your cluster, the Telemetry module automatically integrates with it. It detects the Istio installation and configures the Telemetry components accordingly, enabling secure mTLS communication for outbound data export by default.

Receiving Data from Your Applications

The OTLP Gateway runs as a DaemonSet, with one instance on each cluster node. To ensure that telemetry data from your applications stays on the same node, the ingestion Service uses the Kubernetes `internalTrafficPolicy` (see Service Internal Traffic Policy). By setting this policy to Local, data that your applications send is always received by the gateway instance on the same node.

Because the data path stays within a single node, Istio sidecars and mTLS are not required on the ingestion path. Applications can push data to the OTLP Gateway over a plain-text connection regardless of whether they are part of the Istio mesh.

> → Tip
>
> Learn more about Istio-specific input configuration for logs, traces, and metrics:
>
> - Configure Istio Access Logs [page 2197]
> - Configure Istio Tracing [page 2201]
> - Collect Istio Metrics [page 2209]



Node-Local Ingestion Security

The OTLP Gateway's node-local ingestion path provides the following security properties:

- No cross-node traffic on ingestion: The Service routes traffic only to the local node's gateway Pod. Telemetry data does not travel over the node-to-node network.
- Kernel-level network isolation: Pod-to-pod communication on the same node uses virtual network interfaces connected through a virtual bridge in the node's root network namespace. To intercept this traffic, an attacker needs root access to the node's network namespace or a man-in-the-middle position, which means the node is already compromised.
- No mTLS required for ingestion: Because the data path stays within a single node, mTLS encryption is not needed. This improves performance by removing the Istio sidecar from the ingestion path, without creating a security risk.

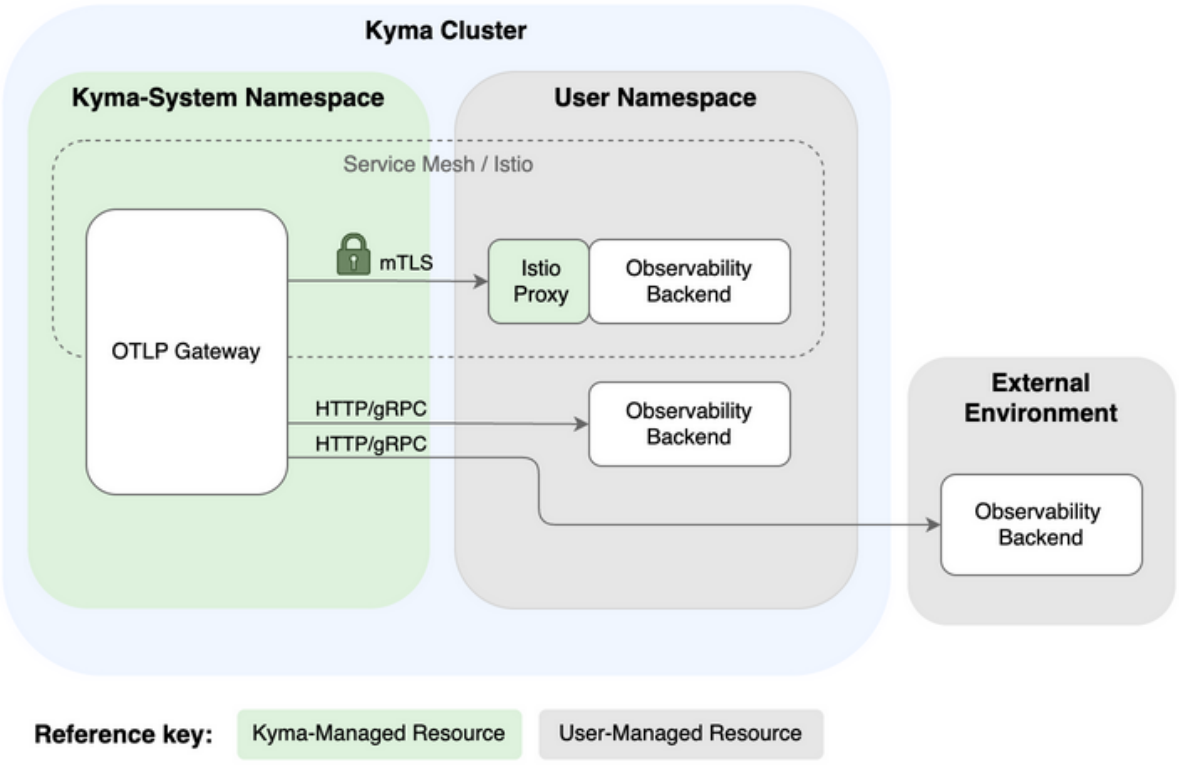
> ⓘ Note
>
> These properties apply only to the ingestion path (applications pushing data to the OTLP Gateway). For the export path (the OTLP Gateway sending data to your backend), mTLS is still used automatically when the backend is part of the Istio mesh. See Sending Data to In-Cluster Backends [page 2265].

Sending Data to In-Cluster Backends

The OTLP Gateway automatically secures the connection when sending data to your observability backends.

If you use an in-cluster backend that is part of the Istio mesh, the OTLP Gateway automatically uses mTLS to send data to the backend securely. You don't need any special configuration for this.

For sending data to backends outside the cluster, see Integrate With Your OTLP Backend [page 2231].



4.3.2 Access a Kyma Instance Using kubectl

As an alternative to managing Kyma with a graphical user interface, Kyma dashboard, you can also use the Kubernetes command-line tool, kubectl.

Prerequisites

You have a Kyma instance created in your subaccount of the SAP BTP cockpit.

Context

To start using kubectl, follow these steps:

Procedure

1. Install kubectl oidc-login . Follow the original instruction from the project repository. Use the relevant commands for macOS, Linux, or Windows users.

   > → Recommendation
   >
   > On Windows, we recommend using the release binaries to install the kubectl oidc-login plugin. The installation of the kubectl oidc-login plugin using Chocolatey and Krew could cause issues.

2. In the SAP BTP cockpit, in your subaccount *Overview*, go to the *Kyma Environment* section, and click on the *KubeconfigURL* link to download `kubeconfig.yaml`.
3. In the terminal, export the kubeconfig file.
   - On macOS, run:

     ```
     export KUBECONFIG={KUBECONFIG_FILE_PATH}
     ```

   - On Windows (PowerShell), run:

     ```
     $ENV:KUBECONFIG="{KUBECONFIG_FILE_PATH}"
     ```

Results

Now you can manage your Kyma instance using kubectl.

> → Tip
>
> If you authenticate in the Kyma environment with an Identity Provider using OpenID Connect, the downloaded `kubeconfig.yaml` can be used indefinitely to re-authenticate.

Related Information

Configuring a Custom Identity Provider for Kyma [page 3660]

4.3.3 Using SAP BTP Services in the Kyma Environment

With the Kyma environment, you can connect SAP BTP services to your cluster and manage them using the SAP BTP Operator module.

Context

To consume SAP BTP services and use their functionality for building and deploying your own applications, you must create and manage the service instances and bindings. To learn how to do it, see Creating Service Instances and Service Bindings [page 2077], Updating Service Instances [page 2082], and Deleting Service Bindings and Service Instances [page 2083].

For an example of how to use a service, see Create an SAP S/4HANA Cloud Extensibility Service Instance in the Kyma Environment.

Related Information

SAP BTP Operator Module [page 2070]
Preconfigured Credentials and Access [page 2072]
Customizing the Default Credentials and Access [page 2075]
Migrating Service Instances and Service Bindings from a Custom SAP BTP Service Operator [page 2084]
Working with Multiple Subaccounts [page 2087]
Instance-Level Mapping [page 2089]
Namespace-Level Mapping [page 2092]
Passing Parameters [page 2094]
Rotating Service Bindings [page 2095]
Formatting Service Binding Secrets [page 2097]

4.3.4 Deploy Workloads in the Kyma Environment to Extend SAP Systems

Access the Kyma environment and start creating extensions for SAP systems.

Prerequisites

- You have the Application Connector and Serverless modules in your cluster. See Adding and Deleting a Kyma Module [page 3511].

- If you want to expose and secure your workload, you must have the default Istio and API Gateway modules in your cluster.
- If you want your workload to react to events, you must have the Eventing module in your cluster.
- If you want to use CloudEvents , make sure that your ecosystem supports it.

Context

You can extend a given SAP system in the Kyma environment with workloads. There are two kinds of workloads - Functions and microservices:

- Functions are simple code snippets that are called when the Function is triggered. They implement the exact business logic you define in the code, and you don't need to worry about containerization and deployment in Kyma runtime. Because Kyma Serverless is taking care of that, using Functions is convenient and fast.
- You can also deploy your extension code as a microservice. In this variant, you must contain your microservice code in a Docker image and deploy it in Kyma runtime. You need more Kubernetes experience but get more control over the shape of your extension application, because you can separate the tasks into smaller pieces that interact with each other as loosely coupled, independently deployable units of code.

Whichever workload variant you choose, you can make it part of the SAP system extension. You can do this in two ways. Depending on your solution's needs, choose one or both of them:

- Expose your workload's API using API Rule and make it accessible to other components of your ecosystem with REST calls.
- Subscribe your workload to events emitted from the SAP systems and extend them.

> ### Note
>
> Most of the linked tutorials contain steps that you can perform both on the Kyma dashboard and in the terminal after downloading the kubeconfig file with the cluster configuration.

Follow these steps to get familiar with workloads and learn how to use them.

Procedure

1. Connect the SAP system you want to extend (Extending SAP Solutions Using Automated Configurations).
2. Choose a namespace in your cluster and create a simple Function (Create and Modify an Inline Function) or a microservice.
3. If you want to expose your workload outside the cluster, read Expose and Secure Workloads [page 1971].
4. If you want your workload to react to events, read Subscribe to Multiple Event Types [page 2143].

   To subscribe to events with Kyma, you must create a Subscription custom resource (CR) including the following parameters:

   - *Application name*: The name of the external Application connected to Kyma runtime. Typically, it starts with `mp-*`. This name can be found in the UI under *Integration* > *Applications*. It must be bound to the namespace.

- *Event name*: The event name depends on your CX solution:
  - SAP Commerce Cloud - for custom events, check the configuration of your Destination Target for the Kyma runtime
  - SAP Sales Cloud / SAP Service Cloud - you can check the event specification in *Event Notification Monitoring*.
- **Event version**: **v1** for legacy events or **v2** for CloudEvents

> ⚠ Caution
>
> Your workload must be error-free, otherwise event loss could occur. If you're facing problems, check:
> - Troubleshooting for the API Gateway Module [page 2052]
> - Troubleshooting for the Eventing Module [page 2148]
> - Troubleshooting for the Serverless Module

Related Information

API Gateway Module [page 1944]
Eventing Module [page 2136]
Serverless Module [page 2169]

4.3.5 Expose and Secure a Workload with a JWT Using SAP Cloud Identity Services

This procedure explains how to expose a workload on a custom domain and secure it with JSON Web Tokens (JWTs) issued by SAP Cloud Identity Services - Identity Authentication using the Client Credentials grant.

Prerequisites

- You have the Istio and API Gateway modules in your cluster. See Adding and Deleting a Kyma Module [page 3511].
- You have an SAP Cloud Identity Services tenant. See Initial Setup.

Context

Use this procedure to secure your workload with a short-lived JWT. To get the JWT, you must first register an OpenID Connect (OIDC) application in SAP Cloud Identity Services and enable the Client Credentials grant. This generates a client ID (public identifier) and a client secret (confidential credential). A calling system sends these credentials to the OIDC token endpoint over TLS, receiving a signed JWT.

When the client calls your exposed API, it includes the token in the Authorization header using the Bearer scheme. The API Gateway module validates the token based on the configuration you include in the APIRule custom resource (CR). If the validation fails, the Gateway returns `HTTP/2 403 RBAC: access denied` without forwarding the request to the backend Service.

If the validation is successful, the request proceeds to the Service behind the Gateway. At that point, you can implement optional, deeper authorization (examining scopes, audience, or custom claims) inside your application code.

Follow these steps:

1. Configure a TLS Gateway for Your Custom Domain [page 1976].
2. Create and Configure an OpenID Connect Application [page 1979].
3. Get a JWT [page 1979].
4. Configure JWT Authentication in Kyma [page 1980].

Configure a TLS Gateway for Your Custom Domain

To configure the flow in Kyma, you must first provide credentials for a supported DNS provider so Gardener can create and verify the necessary DNS records for your custom wildcard domain. After this, Let's Encrypt issues a trusted TLS certificate. The issued certificate is stored in a Kubernetes Secret referenced by an Istio Gateway, which terminates HTTPS at the cluster edge, so all downstream traffic enters encrypted.

Procedure

1. Create a namespace with enabled Istio sidecar proxy injection.

```
kubectl create ns test
kubectl label namespace test istio-injection=enabled --overwrite
```

2. Export the following domain names as environment variables. Replace `my-own-domain.example.com` with the name of your domain. You can adjust these values as needed.

```
PARENT_DOMAIN="my-own-domain.example.com"
SUBDOMAIN="tls.${PARENT_DOMAIN}"
GATEWAY_DOMAIN="*.${SUBDOMAIN}"
WORKLOAD_DOMAIN="httpbin.${SUBDOMAIN}"
```

Placeholder	Example domain name	Description
`PARENT_DOMAIN`	`my-own-domain.example.com`	The domain name available in your public DNS zone.
`SUBDOMAIN`	`tls.my-own-domain.example.com`	A subdomain created under the parent domain, specifically for the TLS Gateway.

Placeholder	Example domain name	Description
`GATEWAY_DOMAIN`	`*.tls.my-own-domain.example.com`	A wildcard domain covering all possible subdomains under the TLS subdomain. When configuring the Gateway, this allows you to expose workloads on multiple hosts (for example, `httpbin.tls.my-own-domain.example.com`, `test.httpbin.tls.my-own-domain.example.com`) without creating separate Gateway rules for each one.
`WORKLOAD_DOMAIN`	`httpbin.tls.my-own-domain.example.co`	The specific domain assigned to your workload.

3. Create a Secret containing credentials for your DNS cloud service provider.

   The information you provide to the data field differs depending on the DNS provider you're using. The DNS provider must be supported by Gardener. To learn how to configure the Secret for a specific provider, follow External DNS Management Guidelines and the External DNS Management examples.

   See an example Secret for the AWS Route 53 DNS provider. `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` are base-64 encoded credentials.

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     annotations:
       dns.gardener.cloud/class: garden
     name: aws-credentials
     namespace: test
   type: Opaque
   data:
     AWS_ACCESS_KEY_ID: <your AWS access key>
     AWS_SECRET_ACCESS_KEY: <your AWS secret access key>
     # Optionally, specify the region
     #AWS_REGION: <your AWS region>
     # Optionally, specify the token
     #AWS_SESSION_TOKEN: <your AWS session token>
   ```

4. Create a DNSProvider resource that references the Secret with your DNS provider's credentials.

   See an example Secret for the AWS Route 53 DNS provider:

   ```
   apiVersion: dns.gardener.cloud/v1alpha1
   kind: DNSProvider
   metadata:
     name: aws
     namespace: test
   spec:
     type: aws-route53
     secretRef:
       name: aws-credentials
     domains:
       include:
       - "${PARENT_DOMAIN}"
   ```

   For DNSProvider configuration for other providers, see the External DNS Management examples.

   To verify that the DNSProvider is ready, run:

Sample Code

```
kubectl get DNSProvider -n test {DNSPROVIDER_NAME}
```

5. Get the external access point of the `istio-ingressgateway` service.

   The external access point is either stored in the ingress Gateway's `ip` field (for example, on GCP) or in the ingress Gateway's `hostname` field (for example, on AWS).

```
LOAD_BALANCER_ADDRESS=$(kubectl get services --namespace istio-system istio-
ingressgateway --output jsonpath='{.status.loadBalancer.ingress[0].ip}')
if [[ -z $LOAD_BALANCER_ADDRESS ]]; then
    LOAD_BALANCER_ADDRESS=$(kubectl get services --
namespace istio-system istio-ingressgateway --output
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
fi
echo "The load balancer address is ${LOAD_BALANCER_ADDRESS}"
```

6. Create a DNSEntry resource.

```
cat <<EOF | kubectl apply -f -
apiVersion: dns.gardener.cloud/v1alpha1
kind: DNSEntry
metadata:
  name: dns-entry
  namespace: test
  annotations:
    dns.gardener.cloud/class: garden
spec:
  dnsName: "${GATEWAY_DOMAIN}"
  ttl: 600
  targets:
    - "${LOAD_BALANCER_ADDRESS}"
EOF
```

   To verify that the DNSEntry is ready, run:

Sample Code

```
kubectl get DNSEntry -n test dns-entry
```

7. Create the server's certificate.

   You use a Certificate CR to request and manage Let's Encrypt certificates from your Kyma cluster. When you create a Certificate CR, one of Gardener's operators detects it and creates an ACME request to Let's Encrypt requesting certificate for the specified domain names. The issued certificate is stored in an automatically created Kubernetes Secret, which name you specify in the Certificate's `secretName` field. For more information, see Manage certificates with Gardener for public domain .

```
cat <<EOF | kubectl apply -f -
apiVersion: cert.gardener.cloud/v1alpha1
kind: Certificate
metadata:
  name: domain-certificate
  namespace: "istio-system"
spec:
  secretName: custom-tls-secret
  commonName: "${GATEWAY_DOMAIN}"
  issuerRef:
    name: garden
```

```
EOF
```

To verify that the Secret with Gateway certificates is ready, run:

Sample Code

```
kubectl get secret -n istio-system custom-tls-secret
```

8. Create a TLS Gateway.

```
cat <<EOF | kubectl apply -f -
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: custom-tls-gateway
  namespace: test
spec:
  selector:
    app: istio-ingressgateway
    istio: ingressgateway
  servers:
    - port:
        number: 443
        name: tls
        protocol: HTTPS
      tls:
        mode: SIMPLE
        credentialName: custom-tls-secret
      hosts:
        - "${GATEWAY_DOMAIN}"
EOF
```

To verify that the TLS Gateway is ready, run:

Sample Code

```
kubectl get gateway -n test custom-tls-gateway
```

Create and Configure an OpenID Connect Application

You need an identity provider to issue JWTs. Create an OpenID Connect application to use SAP Cloud Identity Services as your JWT issuer that manages authentication for your workloads.

Procedure

1. Sign in to the administration console for SAP Cloud Identity Services. See Access Admin Console.
2. Create OpenID Connect Application.
3. Configure OpenID Connect Application for Client Credentials Flow.
4. Configure Secrets for API Authentication.

Get a JWT

Procedure

1. Export the following values as environment variables:

```
CLOUD_IDENTITY_SERVICES_INSTANCE="my-example-tenant.accounts.ondemand.com"
CLIENT_ID="{YOUR-CLIENT-ID}"
CLIENT_SECRET="{YOUR-CLIENT-SECRET}"
```

2. Export base 64 encoded client ID and client secret.

```
export ENCODED_CREDENTIALS=$(echo -n "$CLIENT_ID:$CLIENT_SECRET" | base64)
```

3. Get `token_endpoint`, `jwks_uri`, and `issuer` from your OpenID application, and save these values as environment variables:

```
TOKEN_ENDPOINT=$(curl -s https://$CLOUD_IDENTITY_SERVICES_INSTANCE/.well-
known/openid-configuration | jq -r '.token_endpoint')
echo token_endpoint: $TOKEN_ENDPOINT
JWKS_URI=$(curl -s https://$CLOUD_IDENTITY_SERVICES_INSTANCE/.well-known/
openid-configuration | jq -r '.jwks_uri')
echo jwks_uri: $JWKS_URI
ISSUER=$(curl -s https://$CLOUD_IDENTITY_SERVICES_INSTANCE/.well-known/openid-
configuration | jq -r '.issuer')
echo issuer: $ISSUER
```

4. Get the JWT access token:

```
ACCESS_TOKEN=$(curl -s -X POST "$TOKEN_ENDPOINT" \
    -d "grant_type=client_credentials" \
    -d "client_id=$CLIENT_ID" \
    -H "Content-Type: application/x-www-form-urlencoded" \
    -H "Authorization: Basic $ENCODED_CREDENTIALS" |  jq -r '.access_token')
echo $ACCESS_TOKEN
```

Configure JWT Authentication in Kyma

To configure JWT authentication, expose your workload using APIRule custom resource (CR) and configure `jwt` as the access strategy. You can either use kubectl or Kyma dashboard.

Procedure

- Use Kyma dashboard.
  a. Go to *Discovery and Network → API Rules* and choose *Create*.
  b. Provide all the required configuration details.
  c. For each path you want to secure with a JWT, add a rule with the following configuration.

Option	Description
Path	Specify the request path.
Methods	Check the allowed access methods.
Access Strategy	JWT
JWT → Authentications	In the *JWT* section, add an authentication with your issuer and JSON Web Key Set URIs.

See the following example of a sample HTTPBin Deployment exposed by an APIRule with JWT authentication:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: httpbin
  namespace: test
---
apiVersion: v1
kind: Service
metadata:
  name: httpbin
  namespace: test
  labels:
    app: httpbin
    service: httpbin
spec:
  ports:
  - name: http
    port: 8000
    targetPort: 80
  selector:
    app: httpbin
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
  namespace: test
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
      labels:
        app: httpbin
        version: v1
    spec:
      serviceAccountName: httpbin
      containers:
      - image: docker.io/kennethreitz/httpbin
        imagePullPolicy: IfNotPresent
        name: httpbin
        ports:
        - containerPort: 80
```

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
```

```
metadata:
  name: httpbin-tls
  namespace: test
spec:
  gateway: test/custom-tls-gateway
  hosts:
    - "${WORKLOAD_DOMAIN}"
  rules:
    - jwt:
        authentications:
          - issuer: ${ISSUER}
            jwksUri: ${JWKS_URI}
      methods:
        - GET
      path: /*
  service:
    name: httpbin
    port: 8000
```

- Use kubectl.

  To expose and secure your Service, create the APIRule CR. For each path you want to secure with a JWT, add a rule with the `jwt` field and specify the `issuer` and `jwksUri`.

  ```
  rules:
  ...
    - jwt:
        authentications:
          - issuer: ${ISSUER}
            jwksUri: ${JWKS_URI}
  ...
  ```

  See the following example of a sample HTTPBin Deployment exposed by an APIRule with JWT authentication:

  ```
  apiVersion: v1
  kind: ServiceAccount
  metadata:
    name: httpbin
    namespace: test
  ---
  apiVersion: v1
  kind: Service
  metadata:
    name: httpbin
    namespace: test
    labels:
      app: httpbin
      service: httpbin
  spec:
    ports:
    - name: http
      port: 8000
      targetPort: 80
    selector:
      app: httpbin
  ---
  apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: httpbin
    namespace: test
  spec:
    replicas: 1
    selector:
  ```

```
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
      labels:
        app: httpbin
        version: v1
    spec:
      serviceAccountName: httpbin
      containers:
      - image: docker.io/kennethreitz/httpbin
        imagePullPolicy: IfNotPresent
        name: httpbin
        ports:
        - containerPort: 80
```

```
apiVersion: gateway.kyma-project.io/v2
kind: APIRule
metadata:
  name: httpbin-tls
  namespace: test
spec:
  gateway: test/custom-tls-gateway
  hosts:
    - "${WORKLOAD_DOMAIN}"
  rules:
    - jwt:
        authentications:
          - issuer: ${ISSUER}
            jwksUri: ${JWKS_URI}
      methods:
        - GET
      path: /*
  service:
    name: httpbin
    port: 8000
```

Results

You have exposed the workload with APIRule custom resource.

1. First, test the connection without the JWT.

   ```
   curl -ik -X GET https://${WORKLOAD_DOMAIN}/headers
   ```

   You get the error `HTTP/2 403 RBAC: access denied`.
2. Now, access the secured workload using the correct JWT.

   ```
   curl -ik -X GET https://${WORKLOAD_DOMAIN}/headers --header
   "Authorization:Bearer $ACCESS_TOKEN"
   ```

   You get the `200 OK` response code.

4.3.6 Develop Resilient Applications in the Kyma Runtime

All SAP BTP, Kyma runtime production plans ensure high availability. To benefit from the high-availability setup, make sure the architecture and deployment of your application comply with resiliency best practices. Use the following guidelines for Kubernetes and microservice-based applications to develop a stable application. Read further to find some Istio-related resiliency tips.

Resilient Applications Deployed on Kubernetes

To develop resilient applications deployed on Kubernetes, use:

- Kubernetes built-in health checks. For more information, read how to Configure Liveness, Readiness and Startup Probes .
- Deployment across multiple availability zones for high availability. You can run your applications with multiple replicas and use topology spread constraints to distribute the replicas across multiple availability zones. For more information, see Pod Topology Spread Constraints . Additionally, you can create PodDisruptionBudgets to ensure the sufficient number of replicas during Kubernetes maintenance. For more information, see Disruptions .
- The Keda Module [page 2120] or Kubernetes autoscaling to automatically adjust the number of replicas based on resource usage.
- Rolling updates to minimize downtime during application updates. If you don't provide any configuration, rolling updates are by default enabled in your SAP BTP, Kyma runtime.
- SAP BTP backing services, such as PostgreSQL or Object Store on SAP BTP. If you have to store some application data on SAP BTP, Kyma runtime, use persistent storage to ensure data is not lost in the event of a pod failure.
- Resource quotas to prevent resource starvation and ensure fair resource allocation.
- The Telemetry Module [page 2184] to configure a logging and monitoring solution to identify and troubleshoot issues in real time.
- Pod Disruption Budgets (PDBs) to improve application resiliency and help maintain continuous service operation.

Resilient Microservice-Based Applications

The Istio Module [page 1887] and Istio traffic management features help you achieve inbuilt microservices resiliency. Moreover, to develop resilient microservice-based applications, use:

- Istio's circuit breaker and retry features to prevent service overloads and recover from service failures.
- Distributed tracing with Istio's observability features to monitor the flow of requests across microservices and identify performance and error issues by shipping logs to SAP Cloud Logging (for details, see Integrate With SAP Cloud Logging [page 2235]).
- The Eventing module for asynchronous communication patterns, such as messaging queues, to decouple services and reduce dependencies. See how to Configure SAP Event Mesh for Kyma Eventing [page 2141].

- Health check endpoints as a part of your microservice. For more details, read Kubernetes API health endpoints .
- A centralized logging and monitoring solution to identify issues across multiple microservices and quickly troubleshoot them.
- A clear and well-defined boundary in your microservice`s design to minimize the impact of failures and enable easy replacement or scaling of individual services.
- Istio's service mesh to implement mutual TLS and authorization policies to secure communication between microservices. For more details, read the blog post.
- Network resiliency using Istio's traffic management features to test how your microservices respond to various failure scenarios, such as slow or failing services.
- The Telemetry Module [page 2184] to ship your Istio's access logs and metrics to your logging backend to monitor traffic patterns and quickly troubleshoot issues.

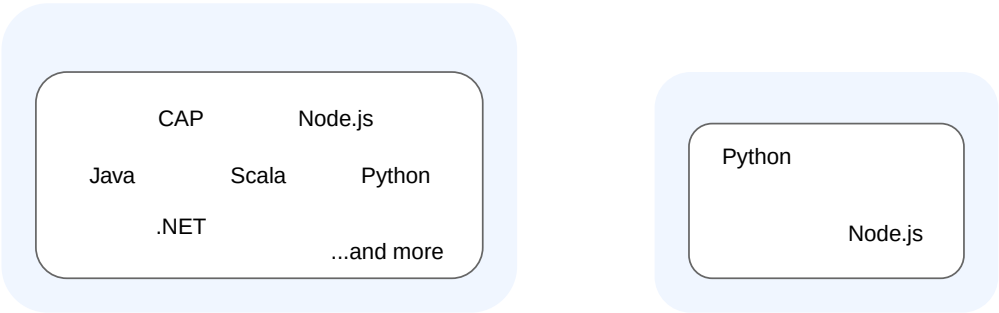
Related Information

Improve resiliency of your applications deployed on Kyma runtime by using multiple availability zones
Developing enterprise-grade applications with mutual TLS easy with SAP BTP, Kyma runtime and SAP BTP destinations
SAP BTP Kyma API Rules with destinations and a managed approuter
Protect your Kyma workloads from eavesdropping with JWT access strategy and SAP BTP destinations
Protect your Kyma workloads from eavesdropping with introspection and SAP BTP destinations

4.3.7 Available Technology

You can deploy your application in SAP BTP, Kyma runtime as a Docker container.

You can use any technology of choice as long as you can package it as a Docker container. The Docker container can be built using Cloud Native Buildpacks (CNB). It is also possible to build lightweight Serverless Functions in Python and JavaScript.



Text is not SVG - cannot display

With microservices, you build and maintain the Docker image.

When working with Serverless Functions, you write the code and Kyma takes care of creating and deploying the Docker image.

5 Extensions and Integrations

The extension capabilities of SAP Business Technology Platform (SAP BTP) enables developers to implement loosely coupled extension applications securely, thus implementing additional workflows or modules on top of the existing SAP cloud solution they already have.

Unified Customer Landscape

The Unified Customer Landscape service provides capabilities for automated extensibility and integration of SAP and third-party systems.

All standard SAP solutions are offered with customizing capabilities. Additionally, customers often have their own requirements for innovative or industry-specific extensions and the extension capabilities of SAP BTP can help them build, deploy, and operate their new functionality.

You can extend standard SAP solutions without disrupting their performance and core processes. When building extension applications, you can also benefit from the automation of the integration between the cloud platform and the extended SAP solutions.

You can also benefit from an automated integration between SAP systems or between SAP systems and a specific service in SAP BTP.

Using the Unified Customer Landscape service, you can maintain your customer landscape. The frontend representation of this service is the *System Landscape* page in the SAP BTP cockpit. The dedicated SAP S/4HANA Cloud Extensibility and SAP SuccessFactors Extensibility services are also part of the Unified Customer Landscape capabilities. See Maintaining Unified Customer Landscape [page 2283].

To get a full list of terms related to the extensibility and integration concepts in the Unified Customer Landscape area, see Extensibility and Integration Concepts [page 2284].

Registering and Deregistering Systems

To connect a system with a global account in SAP BTP, you need to have the system listed in the *System* page.

Adding, Registering and Deregistering Systems [page 2295]

Integrating SAP Solutions

You can integrate several systems and services as part of the common business scenario. To do that, you create a formation in the SAP BTP cockpit that includes all systems that the scenario requires. Every formation has a type related to its purpose. Using formations is a standard way to enable the communication between systems.

See Integrating SAP Solutions [page 2324].

Extending SAP Solutions

The extension capabilities of SAP BTP provide a standard way for extending SAP solutions and developing event-driven extensions and applications. This framework includes:

- Simplified, standardized and unified extensibility and configuration for the SAP solutions.
- Central catalog per customer for all connected SAP systems where data such as APIs, events, credentials and other is stored. You can benefit from business services and actionable events across end-to-end business processes.

You have the following options to extend your SAP solution using the SAP SuccessFactors Extensibility or SAP S/4HANA Cloud Extensibility services:

- Extensions with automated configurations in the Cloud Foundry runtime: applicable for SAP S/4HANA Cloud, SAP Marketing Cloud, and SAP SuccessFactors.
- Extensions with automated configurations in the Kyma runtime: applicable for SAP S/4HANA Cloud, SAP Marketing Cloud, SAP SuccessFactors, SAP Cloud for Customer, SAP Commerce Cloud, and SAP Field Service Management.

If you have to group the systems of different SAP solutions in the same business case, you can set up the connectivity between all these systems and a global account in SAP BTP in a single formation in the SAP BTP cockpit. See Automating Integrations Using Formations [page 2326].

See Extending SAP Solutions [page 2358].

Troubleshooting

If you encounter a problem when extending an SAP S/4HANA Cloud or an SAP SuccessFactors system, go through the following troubleshooting information first:

- Troubleshooting for SAP S/4HANA Cloud Extensibility Service [page 2424]
- Troubleshooting for SAP SuccessFactors Extensibility Service [page 2452]

Related Information

SAP BTP Developer's Guide: Extending Existing SAP Solutions Using SAP BTP

5.1 Maintaining Unified Customer Landscape

Use

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

Unified Customer Landscape is a service that offers customer landscape management capabilities. The frontend representation of this service is the *System Landscape* page in the SAP BTP cockpit. The dedicated *SAP S/4HANA Cloud Extensibility* and *SAP SuccessFactors Extensibility* services are also part of the Unified Customer Landscape capabilities.

Customer Landscape Discovery

The *System Landscape* page of the SAP BTP cockpit gives you a visual overview of your SAP systems associated with the given global account or a subaccount. There are different ways to add systems in the *Systems* > *Systems* page: manually or automatically. If a system of your solution is associated with your global account or through a subscription in SAP BTP cockpit associated with a given subaccount, it will appear in the list automatically. Otherwise, you have to add your system manually. Systems are added to the list in one of the following ways:

- Auto-Discovered
  An auto-discovered system is a system (associated with the given global account) that has been discovered and added automatically to the list based on information of the existing system landscape. Any SAP system of the supported system types that is associated with the same customer ID, with which your global account in SAP BTP is associated, will be added automatically in the system landscape list.
- Subaccount/<my-subaccount>
  Specifies that the system has been added through a subscription in SAP BTP cockpit associated with a given subaccount. The subscription has been discovered and added automatically through the subaccount.
- Manually-Added
  Specifies that the system has been added to the list manually by the global account administrator, using the *Add System* button and completing the wizard. The system has been associated with the global account in SAP BTP.
  See Adding, Registering and Deregistering Systems [page 2295].

Integration and Extensibility

Unified Customer Landscape also helps you integrate and extend your SAP S/4HANA Cloud, SAP Ariba, SAP SuccessFactors, and other SAP systems in one single experience.

In the SAP BTP cockpit, you get a comprehensive overview of all your systems associated with your customer ID. These systems can be registered or auto discovered. They are conveniently listed as a record in the *Systems* list. Moreover, Unified Customer Landscape lets you integrate one or more systems in a common business case by including these systems in a formation.

See:

- Automating Integrations Using Formations [page 2326]
- Extending SAP S/4HANA Cloud in the Cloud Foundry and Kyma Environment [page 2363]
- Extending SAP Marketing Cloud in the Cloud Foundry and Kyma Environment [page 2426]
- Extending SAP SuccessFactors in the Cloud Foundry and Kyma Environment [page 2430]
- Extending SAP Customer Experience Products in the Kyma Environment [page 2454]

5.2 Extensibility and Integration Concepts

When implementing your extensibility scenario, there are a couple of concepts you need to understand to be able to benefit from the functionality that SAP BTP offers.

Systems

A system is a specific instance of an SAP solution that is manually added or auto discovered and is listed in the *System Landscape* > *Systems* page in the SAP BTP cockpit.

When you want to add functionality to your SAP solution, you start by developing an extension application and deploying it in SAP BTP. Then, to allow the application to access the SAP solution, you add the system of this solution to the *Systems* page of the SAP BTP cockpit. If the system is one of the following types, you also have to register it using a registration token:

- SAP S/4HANA Cloud
- SAP Marketing Cloud
- SAP SuccessFactors

System Details

Status

The registration process can have the following status values:

- No status

The system has been added as a record to the list. You can get a registration token for it. However, the registration process on the corresponding solution's system side has not been performed or completed yet.

- *Registered*
  The registration token has been used and the automated registration process has been completed successfully. Now, the system can be used in extensibility business scenarios.
- *Error while Registering*
  The registration has failed.
- *Deregistering*
  A deregistration process has started. As a result, the connectivity between the system and SAP BTP is disabled and extension scenarios cannot be established. The system remains in the system landscape list and you can register it again later on.
- *Error while Deregistering*
  The deregistration has failed.
- *Removing*
  A system removal process has started in the SAP BTP cockpit. You can remove a system from the list only when this system:
  - Hasn't been registered yet
  - Has been deregistered first
  - Has been manually added

  Then, you can remove the system from the *Systems* list completely. To register the system again, first, you must add it to the list anew, and then, initiate the registration procedure.
- *Error while Removing*
  The system removal has failed.

Consumption Bundles, APIs, and Events

Consumption bundles group logically APIs and events intended for communication with the SAP system. This grouping means that the consumption of the APIs or the events can happen by using the same set of credentials later on. When you register a system, you can also preview the number and the content of consumption bundles that are exposed for communication with the given system.

You can explore the consumption bundles, the APIs, and the events configured for your system in *System Details*. To do that, go to the respective tab: *Consumption Bundles*, *APIs*, or *Events*.

URL

You can use the URL as an entry point to the corresponding system. Optionally, you can use it to identify duplicate systems added to the list.

Discovery Mechanisms

There are different ways to add systems to the *Systems* page in the SAP BTP cockpit: manually or automatically. If a system of your solution is associated with your global account or through a subscription in SAP BTP cockpit associated with a given subaccount, it will appear in the list automatically. Otherwise, you have to add your system manually. Systems are added to the list in one of the following ways:

Auto-Discovered

An auto-discovered system is a system (associated with the given global account) that has been discovered and added automatically to the list based on information of the existing system landscape. Any SAP system of the supported system types that is associated with the same customer ID, with which your global account in SAP BTP is associated, will be added automatically in the system landscape list.

> ### Note
>
> If a given SAP system is missing on the *Systems* page, it may be associated with a different customer ID on the SAP BTP global account you are working in. In this case, you need to add the system manually, and then, register it.

Subaccount/<my-subaccount>

Specifies that the system has been added through a subscription in SAP BTP cockpit associated with a given subaccount. The subscription has been discovered and added automatically through the subaccount.

Manually-Added

Specifies that the system has been added to the list manually by the global account administrator, using the *Add System* button and completing the wizard. The system has been associated with the global account in SAP BTP.

Business Type

When you have an auto-discovered SAP system, you can find information in the respective system details whether this system is used for test or production purposes for example. This information is loaded automatically and is presented in the *Business Type* field.

Actions

These are the things you can do with a system:

Adding

Adding a system to the list in the *Systems* page is just the first step of the system registration process. When you have only added a system, the system is not yet registered in the global account in SAP BTP. It appears in the system landscape list as a record with empty (or initialized) status. That is, the required configuration on the system side has not been performed, and therefore, the newly added system cannot exchange or expose its technical details, metadata, APIs, or events. Only when the registration process is complete and the system is registered with SAP BTP, it can exchange the relevant information and enable the extension scenario.

Registering

When registering an SAP system of type SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud and SAP systems from the SAP Customer Experience portfolio, the required configuration on the system side has been performed, and therefore, the newly added system can exchange or expose its technical details, metadata, APIs, or events. Only when the registration process is complete and the system is registered with SAP BTP, it can exchange the relevant information and enable the extension or integration scenario.

The registration of the newly added systems is based on a registration token. The token is used for the pairing of the system and the corresponding global account. After you add a system, you can get the token in the SAP BTP cockpit. Then, you can use it to configure the integration on the corresponding system side. By using the registration token on the system side and registering the system, the system administrator allows the integration of the system with SAP BTP.

The system gets a *Registered* status, only when a token is issued and the registration is complete on the corresponding system side. In general, the *Registered* status means that the communication between the system and SAP BTP has been established. However, depending on the system and its requirements, additional configuration might be needed for the enablement of a fully functional extension scenario. The additional configuration, depending on the system type, is outlined in the corresponding documents listed in the *Related Information* section. See Adding, Registering and Deregistering Systems [page 2295].

For system types different than SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud and SAP systems from the SAP Customer Experience portfolio, when adding a system in the *Systems* page, this system is registered directly but no status is displayed.

Refreshing

To ensure you have all your auto-discovered systems listed in the *Systems* page, choose *Refresh*. Refreshing the *Systems* list pulls all the systems and it might take some time.

Specifics When Registering SAP S/4HANA Cloud Systems

When you register an SAP S/4HANA Cloud system, you can automate some of the required configuration steps at a later point using communication scenario groups. The communication scenario groups are entities that are part of the registration token that you use to register your SAP S/4HANA Cloud systems and are associated with one or more communication scenarios in SAP S/4HANA Cloud.

For example, you can use the Event Mesh communication scenario group when you get the registration token for the SAP S/4HANA Cloud system. This allows the automatic enablement of the communication scenario `SAP_COM_0892` after the corresponding system is added to the formation of type *Eventing Between SAP Cloud Systems*.

You can configure the communication scenario groups when registering an SAP S/4HANA Cloud system in the SAP BTP cockpit, in *System Landscape* > *Systems*. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Merging

You have added your SAP systems manually at a given point in time and you have registered them. Then, you notice that you have the exact same systems in the list in the *Systems* page only this time they are auto-discovered and are not registered. To avoid duplicates and streamline the list of systems, you can merge the auto-discovered SAP systems with your manually added systems. The logic automatically detects duplicates based on their URL and system type. Note that you can only merge auto-discovered SAP systems without assigned status in a target system that has already been registered. That is, you cannot merge registered auto-discovered systems. See Merge SAP Systems [page 2323].

Deregistering

Deregistering a system means that the connectivity between this system and the global account is disabled and extension scenarios cannot be established. You can deregister a system from the *Actions* column, or from the *System Details* page that you access when selecting the system from the *System* list. See Deregister or Remove a System [page 2321].

Removing

Removing a system means that this system is no longer part of the system landscape list. To remove a system from the list, first you have to deregister it. You can remove a system from the *Actions* column, or from the *System Details* page that you access when selecting the system from the *System* list. See Deregister or Remove a System [page 2321].

You can see the discovery mechanism and access all the actions related to the systems listed in the SAP BTP cockpit, in *System Landscape* > *Systems*. See Adding, Registering and Deregistering Systems [page 2295].

Formations

A formation is a logical grouping of SAP systems that can be extended in a business scenario. Formations allow you to combine SAP solution systems and a subaccount in SAP BTP to simplify the connectivity setup and to provide a unified view of all components required for the implementation of your extension scenario. To create a fully functional formation, you can use a two-step wizard. At the first step, you specify a custom formation name and assign a subaccount to it. At the second step, you can include an SAP solution system in the formation. You do this configuration once and you can change it anytime.

Extension business cases often involve extending several SAP solutions at a time. For example, for a given business case you might need to extend the functionality or the UI as follows:

- An SAP SuccessFactors system, and an SAP S/4HANA Cloud system. First, you need to configure the connectivity of each of these systems to Cloud Foundry, Kyma, or both environments. Extension applications of both solutions are part of the same business need.
- An SAP Commerce Cloud system, and an SAP S/4HANA Cloud system. Again, you first configure the connectivity of each of these systems to the Kyma environment.
- A single system of the supported SAP solutions.
- SAP systems that expose event data, which can be shared and exchanged with the systems included in the formation.

When creating a formation in the *System Landscape* > *Formations* page in the SAP BTP cockpit, you include the systems of the different SAP solutions you want to extend. If your business case features more than one system, you can use the corresponding button to include additional systems in the formation. You can start the dialog as many times and add systems to your formation as you want.

Systems Details in a Formation

Consumption Bundle APIs

A consumption bundle can group logically a set of APIs for communication with the SAP system.

Consumption Bundle Events

A consumption bundle can group logically a set of events for communication with the SAP system.

Types

The formation type defines the use case. Therefore, depending on the use case, you have different formation types. To a get a list of the available formation types, see Integrating SAP Solutions [page 2324].

Formation Status

A formation can have the following status values:

- *Ready*
  The formation has been added as a record to the list and all registered systems are included successfully.
- *Draft*
  Not all the systems were included when creating the formation. Include all the missing systems and choose *Finalize Formation*.
- *Action Required*
  The formation has been created but you cannot use it productively yet.
  Based on the formation type and the subaccount that you specified, it might require an SAP BTP, Kyma environment instance or an SAP Business Application Studio subscription. Although the SAP BTP cockpit

allows you to create such a formation, to enable and make use of it, you must also create the respective instance or subscription.

- *Synchronizing*
  Systems that are included in the formation are currently synchronizing in the background.
- *Error*
  An error occurred while some of the systems that are included in the formation were synchronizing in the background.

Actions

Creating Formations

Create a logical grouping of the systems that you want to extend.

Including/Excluding Systems in a Formation

Include additional systems to a given formation or exclude already included systems from a formation.

If your business case features more than one system, you can include additional systems in the formation. You can include as many systems as you want to your formation.

Resynchronizing Systems in a Formation

When you include systems in a formation, these systems are synchronized in the background. If an error occurs, you can resynchronize these systems to restart the synchronization process. To do that, choose *Resynchronize* for the particular formation. You will have this action available only for formations that are in status *Error*.

Reseting and Resynchronizing Systems in a Formation

If you want to reconfigure the systems in a formation from scratch without excluding them, you have to choose *Reset and Resynchronize*. You will have this action available only for formations that have *Ready* status and their formation type is Side-by-Side Extensibility with Kyma.

Finalizing a Formation

You can finalize a formation that is in *Draft* state which means that not all the required systems were included when creating the formation. When you include all the missing systems, choose *Finalize Formation*.

Deleting Formations

Detach the systems, unassign the subaccount, and delete the formation at one go.

When you start a formation deletion process, first the systems are excluded from the formation, then, the subaccount is unassigned, and last, the formation is deleted from the list completely.

You can create and configure formations in the SAP BTP cockpit, in *System Landscape* > *Formations*. See Automating Integrations Using Formations [page 2326].

5.3 Requesting Consent for a System or System Type

A consent request helps you get additional authorization for a specific system or system type. There are different scenarios for which you need to request consent and depending on these scenarios you specify a

consent scope. Once the consent request has been sent, the owner of the system or system type gets an incoming consent request to approve or reject.

Consent Scopes

Consent scopes define the purpose of the consent request, that is the reason why you need additional authorization to a specific system. You specify one or more scopes when you request consent.

Consent Scopes for Systems

Consent Scope	Description
System Sharing	Use this scope when you are subscribed to an application and you want this application to appear as a system added to the *Systems* tab in the *Systems* page.
Integration	Use this scope when you have the system in the *Systems* page and you need additional authorizations to include it in a formation. If the system you want to include in a formation is not listed in the *Systems* page and it corresponds to an application you are subscribed to, use this scope together with the *System Sharing* scope which will also add this system in the *Systems* page.

Consent Scopes for System Types

Consent Scope	Description
System Type Sharing	Use this scope when you want to add a system of a specific type but this type is not available in the *Systems* page. Once you have your consent request approved, you will be able to add systems of this type.

Scenarios

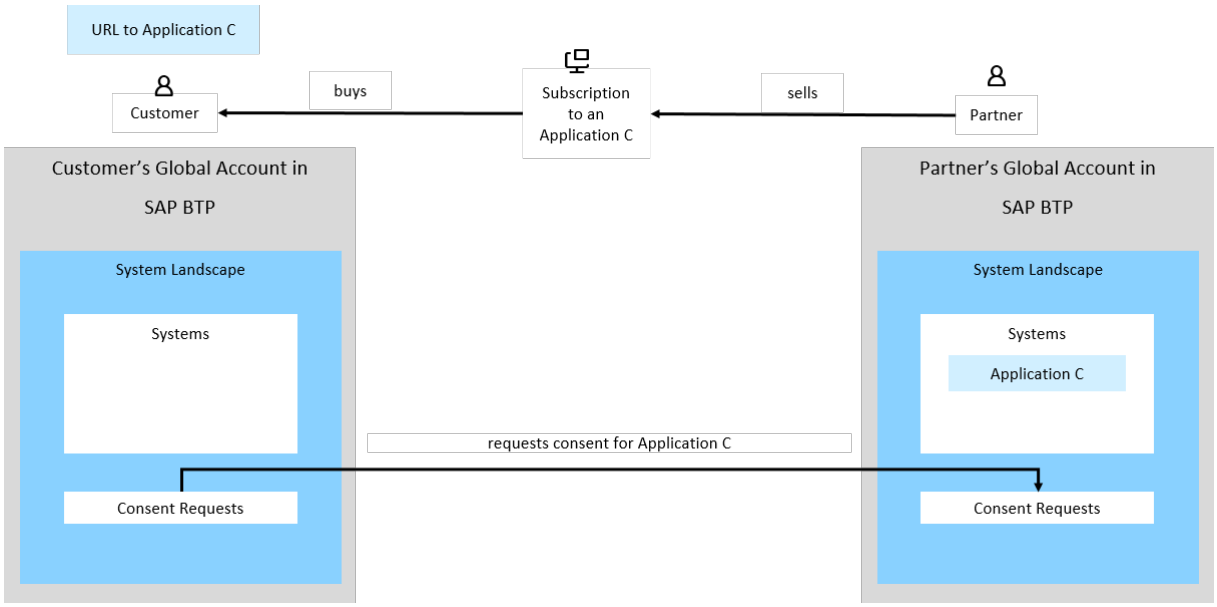
There are various scenarios in which you would need to send consent requests. Pay attention on the scopes defined in each consent request. If you select the wrong scope, your scenario will not work and you would have to send a new consent request with the correct scope. You may need to select more than one scope in the same consent request for a system.

Systems with System Sharing Scope: Have an Application Subscription as a System in the System Landscape

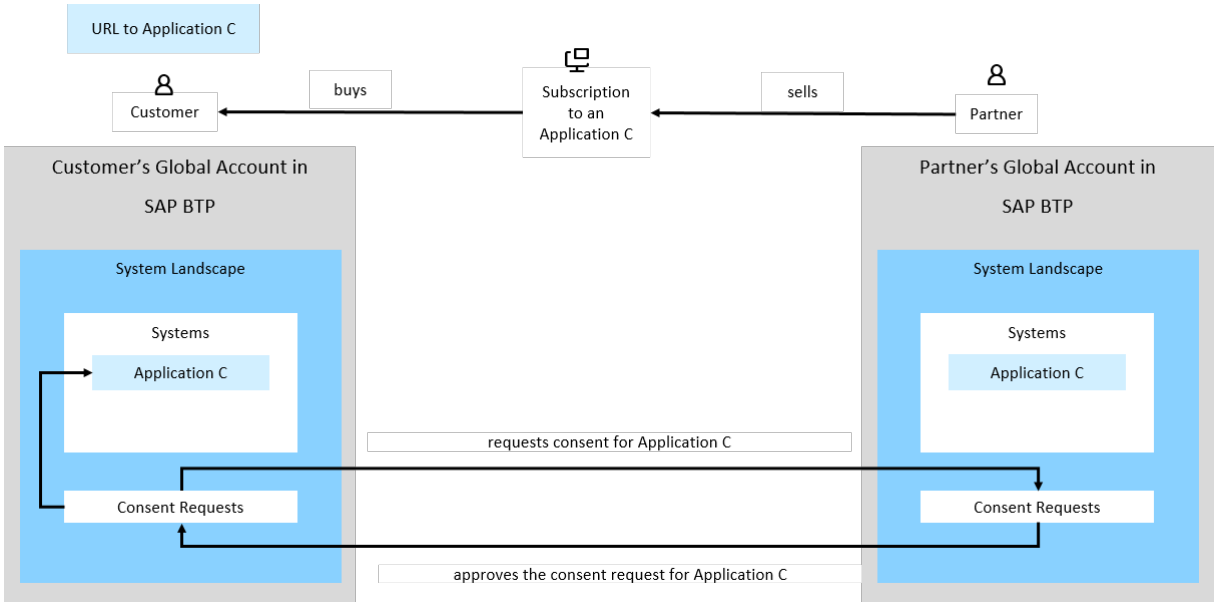
SAP partners develop multitenant applications and sell subscriptions to their customers. These applications are deployed to the global account in SAP BTP of the SAP partner. When customers buy a subscription to such an application, the SAP partner has a dedicated subaccount for each customer and creates a subscription to the application in every subaccount for every customer. Then, the SAP partner takes the respective URL from *Services* > *Instances and Subscriptions* of every subscription and sends it to the respective customer.

As an administrator of a customer, you do not have access to the subscription in your global account in SAP BTP. However, you want to add as a system in your system landscape in your global account the subscription to the application your company has purchased from the SAP partner. To do that, you need to send a consent request to the application owner and after they approve it, you will have the system corresponding to your application subscription listed in the *Systems* page.

In this scenario, use the *System Sharing* consent scope. See Request Consent [page 2293].



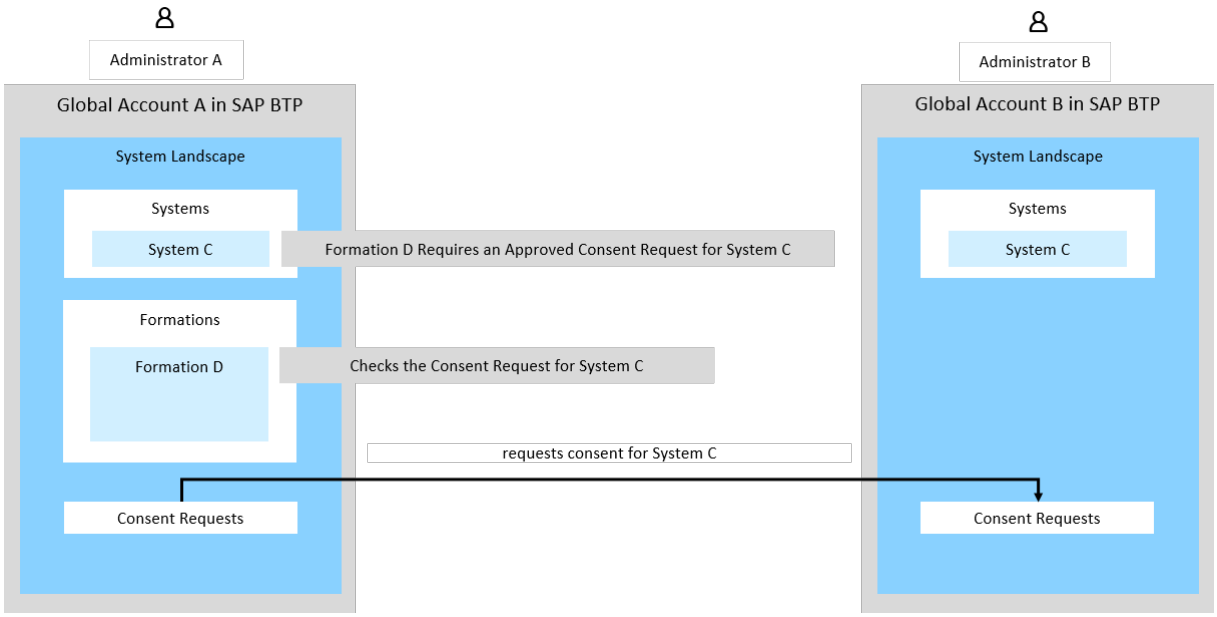
The SAP partner sees the request and then approves or rejects it. When the request has been approved, you see the system of this application listed in the *Systems* page of your global account in SAP BTP. See Approve or Reject a Consent Request [page 2294].



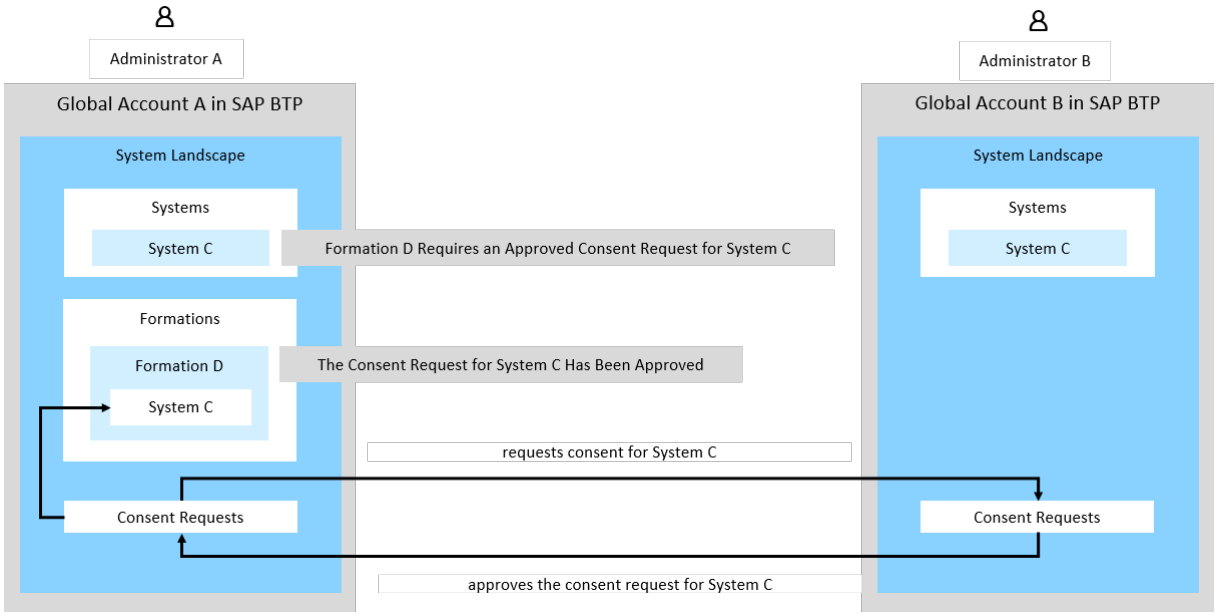
Systems with Integration Scope: Permission to Include a System in a Formation

You are an administrator of a global account in SAP BTP and you have system C added in the *Systems* page. You want to include system C in a dedicated formation, but this system requires additional authorization so

you need to request a consent. In this scenario, use the consent scope *Integration*. See Request Consent [page 2293].



After the administrator responsible for the system approves the request, you can include system C in formation D. See Approve or Reject a Consent Request [page 2294].



See Integrating SAP Solutions [page 2324].

System Types with Sharing System Type Scope: Add Systems of a System Type Which Is Not Part of Your System Landscape

You might need an authorization to have a specific system type shared with you so you can add systems of this type. To do that, you have to send a consent request with the namespace of the system type and the **Sharing System Type** scope.

5.3.1 Request Consent

Context

There are a couple of scenarios which include requesting consent to work with a dedicated system or system type in the *System Landscape*. See Requesting Consent for a System or System Type [page 2289].

You request consent for a system in one of the following cases:

- As an administrator of a customer who has bought a subscription of an SAP partner application. You have received from the partner a URL to directly use the application. To have a system corresponding to your application subscription in the *Systems* page of your global account in SAP BTP, you have to request a consent by providing the URL of the application.
- As an administrator of a global account in SAP BTP, you have a system listed in the *System* page but you need additional authorizations to include this system in a formation. In this case, you provide the system ID and namespace in the consent request.

You request consent for a system type when you went to add a system of this system type and you don't have it in your system landscape by default.

Procedure

1. Sign in to the SAP BTP cockpit and navigate to *System Landscape* > *Consent Requests*.
2. In *My Consent Requests* tab, choose *Request Consent*.
3. In the *Request Consent For* dropdown menu, select one of the following:
   - *System*: when you want to add a system in the *Systems* page or include a system in a formation.
   - *System Type*: when you want to add a system of a specific system type in the *Systems* page and you don't have this system type by default.
4. In the *Request Consent By* dropdown menu, select one of the following:

   For a system:

   - *System URL*
     In the *URL* field, enter the URL of the application you are subscribed to.

     > **Note**
     >
     > If the URL is not unique, you might get an error. If you subscribe to an application provided by an SAP partner and you have only a URL, you have to get back to the SAP partner and ask for the system ID and namespace of the application and use the *System ID and Namespace* option.

   - *System ID and Namespace*
     1. In the *ID* field, enter the ID of the system.
     2. In the *Namespace* field, enter the namespace of the system which is the namespace of the system type.

You get the system ID and namespace from the system details in the *Systems* page. To open the details, choose *System Landscape* > *Systems* and then choose the system for which you are requesting consent.

For a system type, in the *Namespace* field, enter the namespace of the system type.

5. In the *Scopes* dropdown menu, select the scope that you need:
   - *System Sharing*: when you want to add a system corresponding to your application subscription in the *Systems* page of your global account in SAP BTP.
   - *Integration*: when you want to include a system in a formation and this formation requires consent for this.
   - *System Type Sharing*: when you to add a system of this system type and you don't have it in your system landscape by default.

   See Requesting Consent for a System or System Type [page 2289].
6. Choose *Request*.

Next Steps

Once you've sent the consent request, wait for the owner of the system or system type to approve or reject it. You can monitor the state of your requests in the *For Systems* and *For System Types* tabs in *My Consent Requests*.

5.3.2 Approve or Reject a Consent Request

Context

You approve or reject consent for a system or system type in one of the following scenarios:

- As an SAP partner, you develop multitenant applications, deploy them to SAP BTP and sell subscriptions to your customers. When customers have subscribed to your application, they may want to request a consent from you so they have a respective system in their *Systems* page in their global accounts.
- As a system administrator giving additional authorizations for a system so the system can be included in a formation.
- As a system administrator giving additional authorizations for a system type so the customer can add a system of this type in their *Systems* page.

To learn more about the use cases of consent requests for systems or system types, see Requesting Consent for a System or System Type [page 2289].

Procedure

1. Sign in to the SAP BTP cockpit and navigate to *System Landscape* > *Consent Requests*.
2. In the *Incoming Consent Requests* tab, choose the *For Systems* and *For System Types* tabs to check for incoming consent requests.
3. Choose *Approve* or *Reject* for each request.

5.4 Adding, Registering and Deregistering Systems

To connect an SAP system with a global account in SAP BTP, you need to register the system. You can also add and work with a third-party systems.

Prerequisites

- You are a global account administrator, or you are a system landscape administrator of the global account where you want to register your SAP system. See Working with Role Collections [page 2612].
- Before adding your system, make sure it's not already auto-discovered and listed automatically in the *System Landscape* > *Systems* page.

Registering SAP Systems

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

For some system types, such as SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud and SAP systems from the SAP Customer Experience portfolio, adding a system to the list in the *Systems* page is just the first step of the system registration process. When you have only added a system, the system is not yet registered in the SAP BTP global account. That is, the required configuration on the system side has not been performed, and therefore, the newly added system cannot exchange or expose its technical details, metadata, APIs, or events. Only when the registration process is complete and the system is registered with SAP BTP, it can exchange the relevant information and enable the extension scenario.

If your system is associated with the given global account, this system is discovered and added automatically to the *Systems* page based on information of the existing system landscape. Any SAP system that is associated with the same customer ID, with which your global account in SAP BTP is associated, will be auto-discovered.

> **Note**
>
> If a given SAP system is missing on the *Systems* page, it may be associated with a different customer ID on the SAP BTP global account you are working in. In this case, you need to add the system manually, and then, register it.

> ### Note
>
> You cannot migrate the registered SAP systems between global accounts.
>
> If you want to start using another global account, you will have to register your SAP systems again.

For systems of type SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud and SAP systems from the SAP Customer Experience portfolio, the registration of the newly added systems is based on a registration token. The registration token is used for the pairing of the system and the corresponding global account. After you add a system, you can get the token in the SAP BTP cockpit. Then, you can use it to configure the integration on the corresponding system side. By using the registration token on the system side and registering the system, the system administrator allows the integration of the system with SAP BTP.

When you add an SAP system of type SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud and SAP systems from the SAP Customer Experience portfolio, it appears in the system landscape list as a record with empty (or initialized) status. The system gets a *Registered* status, only when a token is issued and the registration is complete on the corresponding system side. In general, the *Registered* status means that the communication between the system and SAP BTP has been established. However, depending on the system and its requirements, additional configuration might be needed for the enablement of a fully functional extension scenario. The additional configuration, depending on the system type, is outlined in the corresponding documents listed in the *Related Information* section.

> ### Note
>
> When registering SAP systems of the same type, you can have up to 1000 tokens per global account ready to be used. Tokens that are already used to register an SAP system are not included in this number.
>
> This means that you cannot have more than 1000 systems in the Systems list of the same type with an empty status and generated token that is not used yet.

For system types different than SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud and SAP systems from the SAP Customer Experience portfolio, when adding a system in the *Systems* page, this system is registered directly but no status is displayed.

The registration process has the following states displayed in the cockpit:

- No status displayed in the *Status* column - the registration token for an SAP system of type SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud or SAP systems from the SAP Customer Experience portfolio has been created but the registration on the respective SAP system side has not been performed or completed yet. For systems with type other than SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud and SAP systems from the SAP Customer Experience portfolio are registered when added to the *Systems* page but no status has been displayed. Only systems that require a registration token to be registered have the status *Registered* when this token has been used.
- *Registered* - the registration token for systems of type SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud and SAP systems from the SAP Customer Experience portfolio has been used and the automated registration process has been successfully completed. The systems of these types can be assigned to a formation on the *Formations* page in the cockpit. All other system types are registered without a registration token and can be assigned to a formation.
- *Error while Registering* - the registration has failed.

> ### Note
>
> If a system is in status *Error while Registering*, delete it and register it again.

If the registration error persists, report a case in one of the following components depending on the system type:

- SAP SuccessFactors - `BC-NEO-EXT-SF`
- SAP S/4HANA Cloud - `BC-NEO-EXT-S4C`
- All other system types - `BC-CP-MP`

> **Note**
>
> When registering a system or creating a formation, the data you provide in the given input fields is not encrypted with your customer managed key. The data you enter is only encrypted at rest.

Procedure

1. In the cockpit, navigate to your global account, and then choose *System Landscape* > *Systems*.
2. On the *Systems* page, choose *Add System*.

   > **Note**
   >
   > Before adding your system, make sure it's not already auto-discovered and listed automatically in the *Systems* page. Some of the SAP systems have additional configurations described in dedicated procedures. See:
   >
   > - Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299]
   > - Register an SAP Marketing Cloud System in a Global Account in SAP BTP [page 2304]
   > - Register an SAP SuccessFactors System in a Global Account in SAP BTP [page 2312]
   > - Register an SAP Customer Experience System [page 2316]

3. In the *Add System* wizard:
   1. In the *Type* dropdown list, select the system type.
   2. Enter a name for the system you want to register.

      > **Note**
      >
      > Use only printable ASCII characters.

      > **→ Tip**
      >
      > We recommend that you indicate the type of the system when specifying the system name. For example, **`<mysystem>-commerce-cloud`**. This helps you identify the system type when assigning systems to a formation.

   3. Choose *Add*.
   4. For systems of type SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud and SAP systems from the SAP Customer Experience portfolio, choose *Get Token*.
      A registration token is generated for this system.
      Systems of type different than SAP SuccessFactors, SAP S/4HANA Cloud and SAP systems from the SAP Customer Experience portfolio don't need a registration token, they are automatically registered when added to the *Systems* page. Their status is empty but you can consider them as registered.
   5. Copy the registration token. You need the token to complete the integration on the respective SAP solution system side.
   6. Close the wizard.

Result

The system has been added as a record to the list on the *Systems* page in the SAP BTP cockpit. For systems of type SAP SuccessFactors, SAP S/4HANA Cloud and SAP systems from the SAP Customer Experience portfolio, a registration token for connecting the corresponding SAP solution with the global account in SAP BTP has been generated. Systems of type other than SAP SuccessFactors, SAP S/4HANA Cloud and SAP systems from the SAP Customer Experience portfolio, are automatically registered, no additional configurations are required.

> **Note**
>
> Registration tokens have different validity periods that depend on the system type. For more information about token expiry, see the corresponding documentation at the *Related Links* section.

Next Steps

- For SAP systems of type SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud and SAP systems from the SAP Customer Experience portfolio, use the registration token to complete the registration on the respective SAP system side. See:
  - Trigger the Registration in the SAP S/4HANA Cloud Tenant [page 2303]
  - Trigger the Registration in the SAP Marketing Cloud Tenant [page 2307]
  - Register an SAP SuccessFactors System in a Global Account in SAP BTP [page 2312]
  - Register an SAP Customer Experience System [page 2316]
- You can assign the system to a formation on the *Formations* page, as follows:
  - SAP systems of type *SAP Commerce Cloud*, *SAP Cloud for Customer*, and *SAP Field Service Management* can be assigned to a formation directly before the integration is complete on the respective SAP Customer Experience system side. However, to enable the API access, you first need to complete the integration.
  - For systems of type *SAP S/4HANA Cloud*, *SAP Marketing Cloud* and *SAP SuccessFactors*, you first need to configure the integration on the respective SAP system side.
  - For systems of type other than *SAP S/4HANA Cloud*, *SAP Marketing Cloud* and *SAP SuccessFactors* and SAP systems from the SAP Customer Experience portfolio, after adding these systems in the *Systems* list, you can directly include them in a formation.

  See Automating Integrations Using Formations [page 2326].
- If you no longer need it, you can deregister or remove the system depending on its status. See Deregister or Remove a System [page 2321].

Deregistering Systems

Deregistering an SAP or third-party system means that the connectivity between this system and the global account is disabled and extension scenarios cannot be established, while removing a system means that this system is no longer part of the system landscape list. You can deregister or remove a system from the *Actions* column, or from the *System Details* page that you access when selecting the system from the system landscape list.

See Deregister or Remove a System [page 2321].

Merging Systems

If you have already added these systems manually at a given point in time and you have registered them, the auto-discovered systems might create duplicate entries in the list. To avoid duplicates and streamline the list of systems, you can merge automatically the auto-discovered SAP systems with your manually added systems.

See Merge SAP Systems [page 2323].

Related Information

Extending SAP S/4HANA Cloud in the Cloud Foundry and Kyma Environment [page 2363]
Extending SAP Marketing Cloud in the Cloud Foundry and Kyma Environment [page 2426]
Extending SAP SuccessFactors in the Cloud Foundry and Kyma Environment [page 2430]
Extending SAP Customer Experience Products in the Kyma Environment [page 2454]
Register an SAP Integrated Business Planning System in a Global Account in SAP BTP [page 2308]
Register a Microsoft 365 Copilot for Joule System [page 2319]
Register a System of Type SAP BTP Application [page 2320]

5.4.1 Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP

To connect an SAP S/4HANA Cloud Public Edition system with a global account in SAP BTP, you need to register the system in the corresponding global account.

Prerequisites

- See Adding, Registering and Deregistering Systems [page 2295].
- You are an administrator of the global account where you want to register your SAP S/4HANA Cloud system.
- To configure the integration on the SAP S/4HANA Cloud system side, you need to be an administrator of the SAP S/4HANA Cloud tenant.

Context

> **Note**
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

The registration process is based on a registration token that is used for the pairing of the SAP S/4HANA Cloud system and the corresponding global account. You create the token in the global account, and then the tenant administrator of the respective SAP S/4HANA Cloud system uses the token to start the automated registration process on the SAP S/4HANA Cloud system side.

> ### Note
>
> When registering SAP S/4HANA Cloud systems, you can have up to 1000 tokens per global account ready to be used. Tokens that are already used to register an SAP S/4HANA Cloud system are not included in this number.
>
> This means that you cannot have more than 1000 systems in the *Systems* page of type SAP S/4HANA Cloud with an empty status and generated token that is not used yet.

> ### Note
>
> You cannot migrate the registered SAP S/4HANA Cloud systems between global accounts.
>
> If you want to start using another global account, you will have to register your SAP S/4HANA Cloud systems again.

> ### Note
>
> You can have the same SAP S/4HANA Cloud system registered in two global accounts:
>
> - If the two global accounts belong to the same Customer ID, then the SAP S/4HANA Cloud system will be auto discovered in both global accounts.
>   In this case, if you register the SAP S/4HANA Cloud system in one of the global accounts, it will appear registered in the other global account as well. If the communications scenario group with which the SAP S/4HANA Cloud system has been registered in the first global account works for the use case in the second global account, then you can use the same system in a formation in the second global account as well.
>   If the use case in the second global account requires a different communication scenario group, you have to add the SAP S/4HANA Cloud system manually in this global account and register it with the necessary communication scenario group.
> - If the two global accounts belong to different Customer IDs, the SAP S/4HANA Cloud system will be auto discovered in one of the global account and need to be added manually in the other. Then, you register the system for each of the global accounts.

The registration process has the following states displayed in the cockpit:

- No status displayed in the *Status* column - the registration token for an SAP system has been created but the registration on the respective SAP solution system side has not been performed or completed.
- *Registered* - the registration token has been used and the automated registration process has been successfully completed. The system can be assigned to a formation on the *Formations* page in the cockpit.
- *Error while Registering* - the registration has failed. Remove the system and then add it to the *Systems* list and try to register it again.
- *Deregistering* - a deregistration process has started in the SAP BTP cockpit. As a result, the connection between the SAP solution system and the global account in SAP BTP is removed. The system remains in the list and you can register it again later on.
  Once a system is registered, you can deregister it only after removing it from all entitlement configurations and formations it takes part in.

> **Note**
>
> You will not be able to deregister a system if its status is one of the following:
>
> - *Error while Registering*
> - *Deregistering*

- *Error while Deregistering* - the deregistration has failed. Try to deregister the system again. If the problem persists, report a case in the `BC-NEO-EXT-S4C` component.
- *Removing* - a system removal process has started in the SAP BTP cockpit. As a result, the link between the SAP solution and SAP BTP is destroyed and the system is removed from the list. To register the system again, first you must add it to the list anew, and then initiate the registration procedure.
  Once a system is registered, you can only remove it if you first deregister it.

  > **Note**
  >
  > You will not be able to remove a system if its status is one of the following:
  >
  > - *Registered*
  >   You first need to deregister the system.
  > - *Deregistering*
  > - *Error while Deregistering*
  >   Try to deregister the system again.

- *Error while Removing* - the system removal has failed. Try to remove the system again. If the problem persists, report a case in the `BC-NEO-EXT-S4C` component.

> **Note**
>
> When registering a system or creating a formation, the data you provide in the given input fields is not encrypted with your customer managed key. The data you enter is only encrypted at rest.

Procedure

1. In the cockpit, navigate to your global account, and then choose *System Landscape* > *Systems*.
2. Check if your SAP S/4HANA Cloud system has been auto-discovered and is already in the *Systems* page. If yes, then continue with *step 3.e.* and get the registration token.

   > **Note**
   >
   > If your SAP S/4HANA Cloud system is auto-discovered and you still add it manually and then finish the registration process with the manually added system, you can merge the two entries. See Merge SAP Systems [page 2323].

3. On the *Systems* page, choose *Add System*.
   a. In the *Type* dropdown list, select *SAP S/4HANA Cloud*.
   b. Enter a name for the system you want to register.

      > **Note**
      >
      > Use only printable ASCII characters.

> → Tip
>
> We recommend that you indicate the type of the system when specifying the system name. For example, **<mysystem>-S/4HANA-cloud**. This helps you identify the system type when assigning systems to a formation.

c. Choose *Add*.

d. In the *Communication Scenario Groups* dropdown menu, select one of the following options:

- *All Communication Scenarios*: to create any of the communication scenarios in SAP S/4HANA Cloud.
- *Eventing Between SAP Cloud Systems*: to allow the automatic enablement of the SAP_COM_0892 communication scenario in SAP S/4HANA Cloud.
- *Integration with SAP Start*: to allow the automatic enablement of the SAP_COM_0647 communication scenario in SAP S/4HANA Cloud.
- *Integration with SAP Ariba Buying*: to allow the automatic enablement of the SAP_COM_0545 and SAP_COM_0A00 communication scenarios in SAP S/4HANA Cloud.
- *Integration with SAP Master Data Integration*: to allow the automatic enablement of the SAP_COM_0008, SAP_COM_0091, SAP_COM_0659, and SAP_COM_0594 communication scenarios in SAP S/4HANA Cloud.
- *Integration with SAP Ariba Central Invoice Management*: to allow the automatic enablement of the SAP_COM_0897, SAP_COM_0516, SAP_COM_0531, SAP_COM_0008, SAP_COM_0091, SAP_COM_0659, and SAP_COM_0594 communication scenarios in SAP S/4HANA Cloud.
- *Integration with SAP Subscription Billing*: to allow the automatic enablement of the SAP_COM_0642, SAP_COM_0845, SAP_COM_0861, SAP_COM_0095, SAP_COM_0008, SAP_COM_0009, SAP_COM_0092, SAP_COM_0884, SAP_COM_0892, and SAP_COM_0165 communication scenarios in SAP S/4HANA Cloud.
- *Integration with SAP Advanced Financial Closing*: to allow the automatic enablement of the SAP_COM_0566 communication scenario in SAP S/4HANA Cloud.
- *Integration with SAP Master Data Governance*: to allow the automatic enablement of the SAP_COM_0659 communication scenario in SAP S/4HANA Cloud.

e. Choose *Get Token*.

The system generates the registration token.

f. Copy the registration token and send it to the tenant administrator for the respective SAP S/4HANA Cloud system. You need it for configuring the integration on the extended SAP S/4HANA Cloud system side.

You can also get the registration token later, once the system appears in the list on the *Systems* page.

The registration token is valid for 7 days after it has been generated. When a token is not used within its validity period, it is no longer valid and cannot be used for registering an SAP S/4HANA Cloud system. If the validity of the token expires before you use it to configure the integration on the SAP S/4HANA Cloud system side and complete the registration, you need to get a new token. You can then copy it and use it to complete the registration.

> ⓘ Note
>
> A registration token can be used only once, for registering a single SAP S/4HANA Cloud system.

g. Close the wizard.

The SAP S/4HANA Cloud system appears in the list of systems on the *Systems* page. Its *Status* field is empty because the registration process is not yet completed.

4. Start the automated registration process on the SAP S/4HANA Cloud system side. To do so, proceed as described in as described in Trigger the Registration in the SAP S/4HANA Cloud Tenant [page 2303].

   > **Note**
   >
   > You can register a system with the same name only once per global account. Once you have started a registration process for a system with a specified name you can no longer register a system with the same name and connect it with the same global account.

5. Check the status of the registration process. To do so, in the cockpit navigate to your global account, and on the *Systems* page, check if the status of the SAP S/4HANA Cloud system has changed to *Registered*.

   If you are already on the *Systems* page, refresh the page to check if the status has changed.

Next Steps

Trigger the Registration in the SAP S/4HANA Cloud Tenant [page 2303]

Related Information

Adding, Registering and Deregistering Systems [page 2295]
Deregister or Remove a System [page 2321]

5.4.1.1 Trigger the Registration in the SAP S/4HANA Cloud Tenant

Use this procedure to trigger the registration process for an SAP S/4HANA Cloud system that you want to pair with your global account in SAP BTP.

Prerequisites

You are an SAP S/4HANA Cloud tenant administrator.

Procedure

1. Log on to the SAP S/4HANA Cloud tenant, go to *Home*, *Communication Management* tab and then choose the *Maintain Extensions on SAP BTP* tile.

2. In the *Maintain Extensions on SAP BTP* screen, in the *Extensions* section, choose *New*.
3. In the *Integration Token* field, enter the content of the registration token from the SAP BTP cockpit.
4. Choose *Save*. A new entry in the table appears with status *Enabling*.
5. After the integration has finished successfully, you can refresh the table.

   The status of the integration should have changed to *Enabled*.

5.4.2 Register an SAP Marketing Cloud System in a Global Account in SAP BTP

To connect an SAP Marketing Cloud system with a global account in SAP BTP, you need to register the system in the corresponding global account.

Prerequisites

- See Adding, Registering and Deregistering Systems [page 2295]
- You are an administrator of the global account where you want to register your SAP Marketing Cloud system.
- To configure the integration on the SAP Marketing Cloud system side, you need to be an administrator of the SAP Marketing Cloud tenant.

Context

The registration process is based on a registration token that is used for the pairing of the SAP Marketing Cloud system and the corresponding global account. You create the token in the global account, and then the tenant administrator of the respective SAP Marketing Cloud system uses the token to start the automated registration process on the SAP Marketing Cloud system side.

> ⓘ Note
>
> When registering SAP Marketing Cloud systems, you can have up to 1000 tokens per global account ready to be used. Tokens that are already used to register an SAP Marketing Cloud system are not included in this number.
>
> This means that you cannot have more than 1000 systems in the *Systems* page of type SAP Marketing Cloud with an empty status and generated token that is not used yet.

The registration process has the following states displayed in the cockpit:

- No status displayed in the *Status* column - the registration token for an SAP system has been created but the registration on the respective SAP solution system side has not been performed or completed.
- *Registered* - the registration token has been used and the automated registration process has been successfully completed. The system can be assigned to a formation on the *Formations* page in the cockpit.

- *Error while Registering* - the registration has failed. Remove the system and then add it to the *Systems* list and try to register it again
- *Deregistering* - a deregistration process has started in the SAP BTP cockpit. As a result, the connection between the SAP solution system and the global account in SAP BTP is removed. The system remains in the list and you can register it again later on.
  Once a system is registered, you can deregister it only after removing it from all entitlement configurations and formations it takes part in.

  > ### Note
  >
  > You will not be able to deregister a system if its status is one of the following:
  >
  > - *Error while Registering*
  > - *Deregistering*

- *Error while Deregistering* - the deregistration has failed. Try to deregister the system again. If the problem persists, report a case in the `BC-NEO-EXT-S4C` component..
- *Removing* - a system removal process has started in the SAP BTP cockpit. As a result, the link between the SAP solution and SAP BTP is destroyed and the system is removed from the list. To register the system again, first you must add it to the list anew, and then initiate the registration procedure.
  Once a system is registered, you can only remove it if you first deregister it.

  > ### Note
  >
  > You will not be able to remove a system if its status is one of the following:
  >
  > - *Registered*
  >   You first need to deregister the system.
  > - *Deregistering*
  > - *Error while Deregistering*

- *Error while Removing* - the system removal has failed. Try to remove the system again. If the problem persists, report a case in the `BC-NEO-EXT-S4C` component.

> ### Note
>
> When registering a system or creating a formation, the data you provide in the given input fields is not encrypted with your customer managed key. The data you enter is only encrypted at rest.

> ### Note
>
> You cannot migrate the registered SAP Marketing Cloud systems between global accounts.
>
> If you want to start using another global account, you will have to register your SAP Marketing Cloud systems again.

> ### Note
>
> You can have the same SAP Marketing Cloud system registered in two global accounts:
>
> - If the two global accounts belong to the same Customer ID, then the SAP Marketing Cloud system will be auto discovered in both global accounts.
>   In this case, if you register the SAP Marketing Cloud system in one of the global accounts, it will appear registered in the other global account as well. If the communications scenario group with which the

SAP Marketing Cloud system has been registered in the first global account works for the use case in the second global account, then you can use the same system in a formation in the second global account as well.
If the use case in the second global account requires a different communication scenario group, you have to add the SAP Marketing Cloud system manually in this global account and register it with the necessary communication scenario group.

- If the two global accounts belong to different Customer IDs, the SAP Marketing Cloud system will be auto discovered in one of the global account and need to be added manually in the other. Then, you register the system for each of the global accounts.

Procedure

1. In the cockpit, navigate to your global account, and then choose *System Landscape* > *Systems*.
2. On the *Systems* page, choose *Add System*.
   a. In the *Type* dropdown list, select *SAP Marketing Cloud*.
   b. Enter a name for the system you want to register.

      > ⓘ Note
      >
      > Use only printable ASCII characters.

      > → Tip
      >
      > We recommend that you indicate the type of the system when specifying the system name. For example, **`<mysystem>-S/4HANA-cloud`**. This helps you identify the system type when assigning systems to a formation.

   c. Choose *Add*.
   d. Choose *Get Token*.

      The system generates the registration token.
   e. Copy the registration token and send it to the tenant administrator for the respective SAP Marketing Cloud system. You need it for configuring the integration on the extended SAP Marketing Cloud system side.

      You can also get the registration token later, once the system appears in the list on the *Systems* page.

      The registration token is valid for 7 days after it has been generated. When a token is not used within its validity period, it is no longer valid and cannot be used for registering an SAP Marketing Cloud system. If the validity of the token expires before you use it to configure the integration on the SAP Marketing Cloud system side and complete the registration, you need to get a new token. You can then copy it and use it to complete the registration.

      > ⓘ Note
      >
      > A registration token can be used only once, for registering a single SAP Marketing Cloud system.

   f. Close the wizard.

The SAP Marketing Cloud system appears in the list of systems on the *Systems* page. Its *Status* field is empty because the registration process is not yet completed.

3. Start the automated registration process on the SAP Marketing Cloud system side. To do so, proceed as described in as described in Trigger the Registration in the SAP Marketing Cloud Tenant [page 2307].

   > ⓘ Note
   >
   > You can register a system with the same name only once per global account. Once you have started a registration process for a system with a specified name you can no longer register a system with the same name and connect it with the same global account.

4. Check the status of the registration process. To do so, in the cockpit navigate to your global account, and on the *Systems* page, check if the status of the SAP Marketing Cloud system has changed to *Registered*.

   If you are already on the *Systems* page, refresh the page to check if the status has changed.

Next Steps

Trigger the Registration in the SAP Marketing Cloud Tenant [page 2307]

Related Information

Adding, Registering and Deregistering Systems [page 2295]
Deregister or Remove a System [page 2321]

5.4.2.1 Trigger the Registration in the SAP Marketing Cloud Tenant

Use this procedure to trigger the registration process for an SAP Marketing Cloud system that you want to pair with your global account in SAP BTP.

Prerequisites

You are an SAP Marketing Cloud tenant administrator.

Procedure

1. Log on to the SAP Marketing Cloud tenant, go to *Home*, *Communication Management* tab and then choose the *Maintain Extensions on SAP BTP* tile.

2. In the *Maintain Extensions on SAP BTP* screen, in the *Extensions* section, choose *New*.
3. In the *Integration Token* field, enter the content of the registration token from the SAP BTP cockpit.
4. Choose *Save*. A new entry in the table appears with status *Enabling*.
5. After the integration has finished successfully, you can refresh the table.

   The status of the integration should have changed to *Enabled*.

5.4.3 Register an SAP Integrated Business Planning System in a Global Account in SAP BTP

To connect an SAP Integrated Business Planning system with a global account in SAP BTP, you need to register the system in the corresponding global account.

Prerequisites

- See Adding, Registering and Deregistering Systems [page 2295]
- You are an administrator of the global account where you want to register your SAP Integrated Business Planning system.
- To configure the integration on the SAP Integrated Business Planning system side, you need to be an administrator of the SAP Integrated Business Planning tenant.

Context

The registration process is based on a registration token that is used for the pairing of the SAP Integrated Business Planning system and the corresponding global account. You create the token in the global account, and then the tenant administrator of the respective SAP Integrated Business Planning system uses the token to start the automated registration process on the SAP Integrated Business Planning system side.

> **Note**
>
> When registering SAP Integrated Business Planning systems, you can have up to 1000 tokens per global account ready to be used. Tokens that are already used to register an SAP Integrated Business Planning system are not included in this number.
>
> This means that you cannot have more than 1000 systems in the *Systems* page of type SAP Integrated Business Planning with an empty status and generated token that is not used yet.

The registration process has the following states displayed in the cockpit:

- No status displayed in the *Status* column - the registration token for an SAP system has been created but the registration on the respective SAP solution system side has not been performed or completed.
- *Registered* - the registration token has been used and the automated registration process has been successfully completed. The system can be assigned to a formation on the *Formations* page in the cockpit.

- *Error while Registering* - the registration has failed. Remove the system and then add it to the *Systems* list and try to register it again
- *Deregistering* - a deregistration process has started in the SAP BTP cockpit. As a result, the connection between the SAP solution system and the global account in SAP BTP is removed. The system remains in the list and you can register it again later on.
  Once a system is registered, you can deregister it only after removing it from all entitlement configurations and formations it takes part in.

  > **Note**
  >
  > You will not be able to deregister a system if its status is one of the following:
  >
  > - *Error while Registering*
  > - *Deregistering*

- *Error while Deregistering* - the deregistration has failed. Try to deregister the system again. If the problem persists, report a case in the `BC-NEO-EXT-S4C` component..
- *Removing* - a system removal process has started in the SAP BTP cockpit. As a result, the link between the SAP solution and SAP BTP is destroyed and the system is removed from the list. To register the system again, first you must add it to the list anew, and then initiate the registration procedure.
  Once a system is registered, you can only remove it if you first deregister it.

  > **Note**
  >
  > You will not be able to remove a system if its status is one of the following:
  >
  > - *Registered*
  >   You first need to deregister the system.
  > - *Deregistering*
  > - *Error while Deregistering*

- *Error while Removing* - the system removal has failed. Try to remove the system again. If the problem persists, report a case in the `BC-NEO-EXT-S4C` component.

> **Note**
>
> When registering a system or creating a formation, the data you provide in the given input fields is not encrypted with your customer managed key. The data you enter is only encrypted at rest.

> **Note**
>
> You cannot migrate the registered SAP Integrated Business Planning systems between global accounts.
>
> If you want to start using another global account, you will have to register your SAP Integrated Business Planning systems again.

> **Note**
>
> You can have the same SAP Integrated Business Planning system registered in two global accounts:
>
> - If the two global accounts belong to the same Customer ID, then the SAP Integrated Business Planning system will be auto discovered in both global accounts.
>   In this case, if you register the SAP Integrated Business Planning system in one of the global accounts, it will appear registered in the other global account as well. If the communications scenario group with

which the SAP Integrated Business Planning system has been registered in the first global account works for the use case in the second global account, then you can use the same system in a formation in the second global account as well.
If the use case in the second global account requires a different communication scenario group, you have to add the SAP Integrated Business Planning system manually in this global account and register it with the necessary communication scenario group.

- If the two global accounts belong to different Customer IDs, the SAP Integrated Business Planning system will be auto discovered in one of the global account and need to be added manually in the other. Then, you register the system for each of the global accounts.

Procedure

1. In the cockpit, navigate to your global account, and then choose *System Landscape* > *Systems*.
2. On the *Systems* page, choose *Add System*.
   a. In the *Type* dropdown list, select *SAP Integrated Business Planning*.
   b. Enter a name for the system you want to register.

      > ⓘ Note
      >
      > Use only printable ASCII characters.

      > → Tip
      >
      > We recommend that you indicate the type of the system when specifying the system name. For example, **`<mysystem>-S/4HANA-cloud`**. This helps you identify the system type when assigning systems to a formation.

   c. Choose *Add*.
   d. In the *Communication Scenario Groups* dropdown menu, select one of the following options:
      - *All Communication Scenarios*: to create any of the communication scenarios in SAP Integrated Business Planning.
      - *Eventing Between SAP Cloud Systems*: to allow the automatic enablement of the `SAP_COM_0892` communication scenario in SAP Integrated Business Planning.
      - *Integration with SAP Start*: to allow the automatic enablement of the `SAP_COM_0647` communication scenario in SAP Integrated Business Planning.
      - *Integration with SAP Ariba Buying*: to allow the automatic enablement of the `SAP_COM_0545` and `SAP_COM_0A00` communication scenarios in SAP Integrated Business Planning.
      - *Integration with SAP Master Data Integration*: to allow the automatic enablement of the `SAP_COM_0008`, `SAP_COM_0091`, `SAP_COM_0659`, and `SAP_COM_0594` communication scenarios in SAP Integrated Business Planning.
      - *Integration with SAP Ariba Central Invoice Management*: to allow the automatic enablement of the `SAP_COM_0897`, `SAP_COM_0516`, `SAP_COM_0531`, `SAP_COM_0008`, `SAP_COM_0091`, `SAP_COM_0659`, and `SAP_COM_0594` communication scenarios in SAP Integrated Business Planning.
      - *Integration with SAP Subscription Billing*: to allow the automatic enablement of the `SAP_COM_0642`, `SAP_COM_0845`, `SAP_COM_0861`, `SAP_COM_0095`, `SAP_COM_0008`,

`SAP_COM_0009`, `SAP_COM_0092`, `SAP_COM_0884`, `SAP_COM_0892`, and `SAP_COM_0165` communication scenarios in SAP Integrated Business Planning.

- *Integration with SAP Advanced Financial Closing*: to allow the automatic enablement of the `SAP_COM_0566` communication scenario in SAP Integrated Business Planning.

e. Choose *Get Token*.

The system generates the registration token.

f. Copy the registration token and send it to the tenant administrator for the respective SAP Integrated Business Planning system. You need it for configuring the integration on the extended SAP Integrated Business Planning system side.

You can also get the registration token later, once the system appears in the list on the *Systems* page.

The registration token is valid for 7 days after it has been generated. When a token is not used within its validity period, it is no longer valid and cannot be used for registering an SAP Integrated Business Planning system. If the validity of the token expires before you use it to configure the integration on the SAP Integrated Business Planning system side and complete the registration, you need to get a new token. You can then copy it and use it to complete the registration.

> **Note**
>
> A registration token can be used only once, for registering a single SAP Integrated Business Planning system.

g. Close the wizard.

The SAP Integrated Business Planning system appears in the list of systems on the *Systems* page. Its *Status* field is empty because the registration process is not yet completed.

3. Start the automated registration process on the SAP Integrated Business Planning system side. To do so, proceed as described in as described in Trigger the Registration in the SAP Integrated Business Planning Tenant [page 2312].

> **Note**
>
> You can register a system with the same name only once per global account. Once you have started a registration process for a system with a specified name you can no longer register a system with the same name and connect it with the same global account.

4. Check the status of the registration process. To do so, in the cockpit navigate to your global account, and on the *Systems* page, check if the status of the SAP Integrated Business Planning system has changed to *Registered*.

If you are already on the *Systems* page, refresh the page to check if the status has changed.

Next Steps

Trigger the Registration in the SAP Integrated Business Planning Tenant [page 2312]

5.4.3.1 Trigger the Registration in the SAP Integrated Business Planning Tenant

Use this procedure to trigger the registration process for an SAP Integrated Business Planning system that you want to pair with your global account in SAP BTP.

Prerequisites

You are an SAP Integrated Business Planning tenant administrator.

Procedure

1. Log on to the SAP Integrated Business Planning tenant, go to *Home*, *Communication Management* tab and then choose the *Maintain Extensions on SAP BTP* tile.
2. In the *Maintain Extensions on SAP BTP* screen, in the *Extensions* section, choose *New*.
3. In the *Integration Token* field, enter the content of the registration token from the SAP BTP cockpit.
4. Choose *Save*. A new entry in the table appears with status *Enabling*.
5. After the integration has finished successfully, you can refresh the table.

   The status of the integration should have changed to *Enabled*.

5.4.4 Register an SAP SuccessFactors System in a Global Account in SAP BTP

To connect an SAP SuccessFactors system with a global account in SAP BTP, you need to register the system in the corresponding global account.

Prerequisites

- See Adding, Registering and Deregistering Systems [page 2295].
- You are an administrator of the global account where you want to register your SAP SuccessFactors system.
- You have a dedicated SAP SuccessFactors company instance.
- You have a user with permissions to access *Extension Center* in SAP SuccessFactors Admin Center that include:
  - *Admin Access to MDF OData API* permission from the *Metadata Framework* category
  - *Create Integration with SAP BTP* permission from the *Manage Extensions on SAP BTP* category

To assign these permissions to your user, you might need the help of an administrator.
To get a user with the respective permissions, follow these steps:

1. Make sure you have an access to the SAP SuccessFactors Admin Center. See Permission to Access Admin Center.
2. Add your user to an already existing group or create a new dedicated group and add your user. See Creating Dynamic Permission Groups.
3. Use an already existing role or create a new dedicated role. See Creating Permission Roles.
4. Assign the respective permissions to your role. See Assigning Permissions to a Role.
5. Add your role to the dedicated group. See Assigning Roles to Groups.
6. To fully activate the permissions, log out and log in again to the SAP SuccessFactors company instance.

Context

The registration process is based on a registration token that is used for the pairing of the SAP SuccessFactors company and the corresponding global account in SAP BTP. You create the token in the global account, and then start the automated registration process on the SAP SuccessFactors company side using this token.

> **Note**
>
> When registering SAP SuccessFactors systems, you can have up to 1000 tokens per global account ready to be used. Tokens that are already used to register an SAP SuccessFactors system are not included in this number.
>
> This means that you cannot have more than 1000 systems in the *Systems* list of type SAP SuccessFactors with an empty status and generated token that is not used yet.

The registration process has the following states displayed in the cockpit:

- No status displayed in the *Status* column - the registration token for an SAP system has been created but the registration on the respective SAP solution system side has not been performed or completed.
- *Registered* - the registration token has been used and the automated registration process has been successfully completed. The system can be assigned to a formation on the *Formations* page in the cockpit.
- *Error while Registering* - the registration has failed. Remove the system and then add it to the *Systems* list and try to register it again
- *Deregistering* - a deregistration process has started in the SAP BTP cockpit. As a result, the connection between the SAP solution system and the global account in SAP BTP is removed. The system remains in the list and you can register it again later on.
  Once a system is registered, you can deregister it only after removing it from all entitlement configurations and formations it takes part in.
- *Error while Deregistering* - the deregistration has failed. Try to deregister the system again. If the problem persists, report a case in the `BC-NEO-EXT-SF` component.
- *Removing* - a system removal process has started in the SAP BTP cockpit. As a result, the link between the SAP solution and SAP BTP is destroyed and the system is removed from the list. To register the system again, first you must add it to the list anew, and then initiate the registration procedure.
  Once a system is registered, you can only remove it if you first deregister it.

> **Note**
>
> You will not be able to remove a system if its status is one of the following:
>
> - *Registered*
>   You first need to deregister the system.
> - *Deregistering*
> - *Error while Deregistering*
>   Try to deregister the system again.

- *Error while Removing* - the system removal has failed. Try to remove the system again. If the problem persists, report a case in the `BC-NEO-EXT-SF` component.

> **Note**
>
> When registering a system or creating a formation, the data you provide in the given input fields is not encrypted with your customer managed key. The data you enter is only encrypted at rest.

Procedure

1. In the SAP BTP cockpit, navigate to your global account, and then choose *System Landscape* > *Systems*.
2. On the *Systems* page, choose *Add System*.
   a. In the *Type* dropdown list, select SAP SuccessFactors.
   b. Enter a name for the system you want to register.

      > **Note**
      >
      > Use only printable ASCII characters.

      > **→ Tip**
      >
      > We recommend that you indicate the type of the system when specifying the system name. For example, **`<mysystem>-SuccessFactors`**. This helps you identify the system type when assigning systems to a formation.

   c. Choose *Add*.
   d. Choose *Get Token*.

      The system generates the registration token.
   e. Copy the registration token and send it to the tenant administrator for the respective SAP SuccessFactors system. You need it for configuring the integration on the extended SAP SuccessFactors system side.

      You can also get the registration token later, once the system appears in the list on the *Systems* page.

      The registration token is valid for 7 days after it has been generated. When a token is not used within its validity period, it is no longer valid and cannot be used for registering an SAP SuccessFactors system. If the validity of the token expires before you use it to configure the integration on the SAP

SuccessFactors system side and complete the registration, you need to get a new token. You can then copy it and use it to complete the registration.

> **Note**
>
> A token can be used only once, for registering a single SAP SuccessFactors system.

f. Close the wizard.

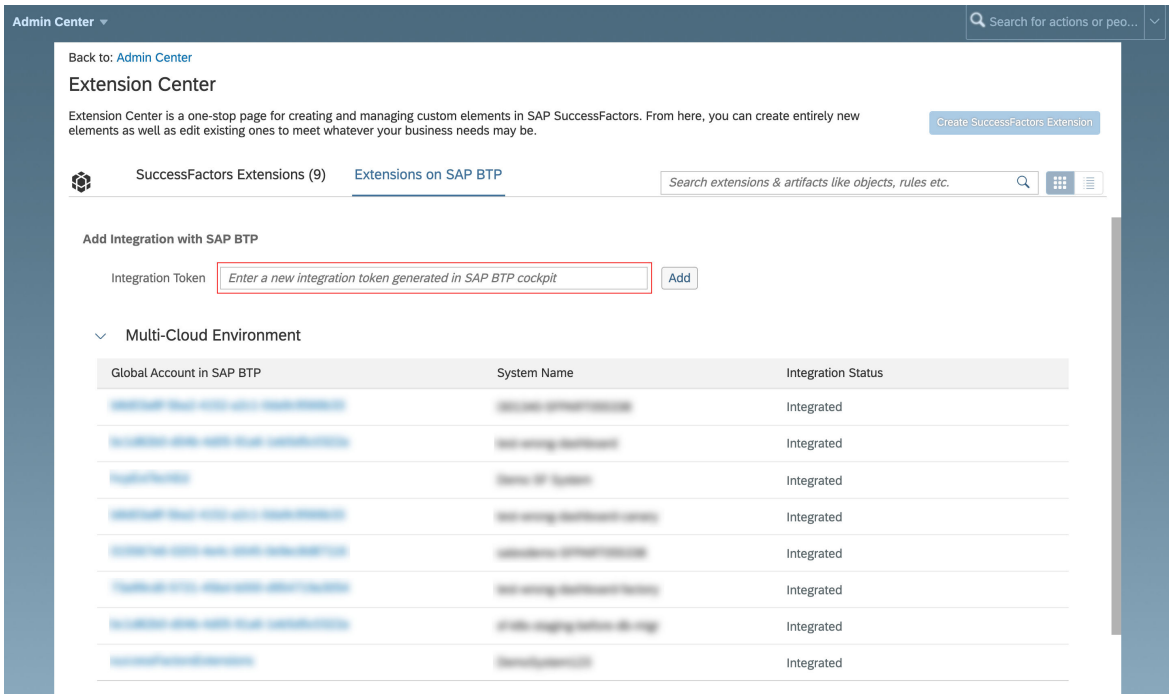
The SAP SuccessFactors system appears in the list of systems on the *Systems* page. Its *Status* field is empty because the registration process is not yet completed.

3. Start the automated integration process on the SAP SuccessFactors company side:

> **Note**
>
> If you do not have permissions to access the Extension Center for the corresponding SAP SuccessFactors system, you need to send the registration token to a user with such permissions who will configure the integration on the SAP SuccessFactors system side. For the requires permissions, check the prerequisites.

1. In SAP SuccessFactors *Admin Center*, navigate to *Extension Center*.
2. On the *Extensions on SAP BTP* tab page, navigate to the *Add Integration with SAP BTP* screen area, and paste the registration token in the *Integration Token* input field.
3. Choose *Add*.

The system appears in the integration list in the *Multi-Cloud Environment* screen area, and the status of the integration is displayed in the *Integration Status* column. To refresh the status of the process, choose the *Check Status* icon. Wait for the integration to finish.



4. In the cockpit, check the status of the registration process. To do so, navigate to your global account, and on the *Systems* page, check if the status of the SAP System has changed to *Registered*.

If you are already on the *Systems* page, refresh the page to check if the status has changed.

> **Note**
>
> You can register a system only once with the same name per global account.

> **Note**
>
> You cannot migrate the registered SAP SuccessFactors systems between global accounts.
>
> If you want to start using another global account, you will have to register your SAP SuccessFactors systems again.

Related Information

Adding, Registering and Deregistering Systems [page 2295]
Deregister or Remove a System [page 2321]

5.4.5 Register an SAP Customer Experience System

Register an SAP Customer Experience system to connect it with a global account in SAP BTP.

Prerequisites

- See Adding, Registering and Deregistering Systems [page 2295].
- You are an administrator of the global account where you want to register your SAP Customer Experience system.
- To configure the integration on the SAP Customer Experience system side, you need to be an administrator of the SAP Customer Experience tenant.

Context

The registration process is based on a registration token that is used for the pairing of the SAP Customer Experience system and the corresponding global account. You create the token in the global account, and then the tenant administrator of the respective SAP Customer Experience system uses the token to start the automated registration process on the SAP Customer Experience system side.

The registration process has the following states displayed in the cockpit:

- No status displayed in the *Status* column - the registration token for an SAP system has been created but the registration on the respective SAP solution system side has not been performed or completed.
- *Registered* - the registration token has been used and the automated registration process has been successfully completed. The system can be assigned to a formation on the *Formations* page in the cockpit.

- *Error while Registering* - the registration has failed. Remove the system and then add it to the *Systems* list and try to register it again
- *Deregistering* - a deregistration process has started in the SAP BTP cockpit. As a result, the connection between the SAP solution system and the global account in SAP BTP is removed. The system remains in the list and you can register it again later on.
  Once a system is registered, you can deregister it only after removing it from all entitlement configurations and formations it takes part in.
- *Error while Deregistering* - the deregistration has failed. Try to deregister the system again. If the problem persists, report a case in the `BC-CP-MP` component.
- *Removing* - a system removal process has started in the SAP BTP cockpit. As a result, the link between the SAP solution and SAP BTP is destroyed and the system is removed from the list. To register the system again, first you must add it to the list anew, and then initiate the registration procedure.
  Once a system is registered, you can only remove it if you first deregister it.

  > **Note**
  >
  > You will not be able to remove a system if its status is one of the following:
  > - *Registered*
  >   You first need to deregister the system.
  > - *Deregistering*
  > - *Error while Deregistering*
  >   Try to deregister the system again.

- *Error while Removing* - the system removal has failed. Try to remove the system again. If the problem persists, report a case in the `BC-CP-MP` component.

> **Note**
>
> When registering a system or creating a formation, the data you provide in the given input fields is not encrypted with your customer managed key. The data you enter is only encrypted at rest.

Procedure

1. In the cockpit, navigate to your global account, and then choose *System Landscape* > *Systems*.
2. On the *Systems* page, choose *Add System*.
   a. In the *Type* dropdown list, select the system type.

      The following SAP Customer Experience systems are supported:

      - SAP Commerce Cloud
      - SAP Field Service Management
      - SAP Cloud for Customer

   b. Enter a name for the system you want to register.

      > **Note**
      >
      > Use only printable ASCII characters.

> → Tip
>
> We recommend that you indicate the type of the system when specifying the system name. For example, **`<mysystem>-S/4HANA-cloud`**. This helps you identify the system type when assigning systems to a formation.

c. Choose *Add*.

d. Choose *Get Token*.

The system generates the registration token.

e. Copy the registration token and send it to the tenant administrator for the respective SAP Customer Experience system. You need it for configuring the integration on the extended SAP Customer Experience system side.

You can also get the registration token later, once the system appears in the list on the *Systems* page.

The integration token is valid for 10 minutes after it has been generated. When a token is not used within its validity period, it is no longer valid and cannot be used for registering a system. If the validity of the token expires before you use it to configure the integration on the SAP Customer Experience system side and complete the registration, you need to get a new token. You can then copy it and use it to complete the registration.

> ⓘ Note
>
> A registration token can be used only once, for registering a single SAP Customer Experience system.

f. Close the wizard.

The SAP Customer Experience system appears in the list of systems on the *Systems* page. Its *Status* field is empty because the registration process is not yet completed.

3. (Recommended) Follow the steps in Automating Integrations Using Formations [page 2326] before proceeding with the registration on the SAP Customer Experience side.
4. Configure the integration on the SAP Customer Experience system side. See Extending SAP Customer Experience Products in the Kyma Environment [page 2454].
5. Check the status of the registration process. To do so, in the cockpit navigate to your global account, and in the *Systems* panel, check if the status of the SAP Customer Experience system has changed to *Registered*.

   If you are already in the *Systems* panel, refresh the page to check if the status has changed.

Results

Once you use the integration token to connect your SAP Customer Experience system, all of the exposed services and events are propagated to the Kyma runtime.

Related Information

Adding, Registering and Deregistering Systems [page 2295]

Deregister or Remove a System [page 2321]

5.4.6 Register a Microsoft 365 Copilot for Joule System

Prerequisites

You are a global account administrator, or you are a system landscape administrator of the global account where you want to add a system of type Microsoft 365 Copilot for Joule. See Working with Role Collections [page 2612].

Context

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

You can integrate Microsoft 365 Copilot with Joule so that your users have conversational interactions that include content about integrated SAP systems. To do that, you have to add a system of type *Microsoft 365 Copilot for Joule* in a formation of type *Integration with Joule*. See Integrating Joule with Microsoft 365 Copilot.

Procedure

1. In the cockpit, navigate to your global account, and then choose *System Landscape* > *Systems*.
2. On the *Systems* page, choose *Add System*.
3. In the *Add System* wizard:
   a. In the *Type* dropdown list, select *Microsoft 365 Copilot for Joule*.
   b. Enter a name for the system you want to register.

      > **Note**
      >
      > Use only printable ASCII characters.

      > **→ Tip**
      >
      > We recommend that you indicate the type of the system when specifying the system name.

   c. In the *System ID* field, enter the ID which corresponds to the Microsoft Entra Tenant ID. To see how to get this ID, see section **Get the Tenant ID of Your Microsoft Entra ID Tenant** of the Configuring SAP Cloud Identity Services and Microsoft Entra ID for Joule blog post.

d. Choose *Add*.

Results

The system has been added as a record to the list on the *Systems* page in the SAP BTP cockpit and you have added the respective APIs or events.

If you no longer need it, you can remove the system. See Deregister or Remove a System [page 2321].

Next Steps

Create a new *Integration with Joule* formation or use an already existing one and include in the system in this formation. See Enabling Joule [page 2348].

5.4.7 Register a System of Type SAP BTP Application

Prerequisites

You are a global account administrator, or you are a system landscape administrator of the global account where you want to add your *SAP BTP Application* system. See Working with Role Collections [page 2612].

Context

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

You can connect an application you've developed and deployed in SAP BTP with a global account in SAP BTP. To do that, you add a system representing your application to the list in the *System Landscape* > *Systems* page. The system type you use is *SAP BTP Application*. You also have to provide the namespace which is unique for this system. A unique system ID is automatically assigned to any system of type *SAP BTP Application*. For systems of type SAP BTP Application, this completes the registration process and you have your system registered with SAP BTP. Even though the system of type SAP BTP Application is registered directly, no status is displayed.

When you have a systems of your application listed in the *Systems* page, you can include it in different formations depending on your needs.

Procedure

1. In the cockpit, navigate to your global account, and then choose *System Landscape* > *Systems*.
2. On the *Systems* page, choose *Add System*.
3. In the *Add System* wizard:
   a. In the *Type* dropdown list, select *SAP BTP Application*.
   b. Enter a name for the system you want to register.

      > ⓘ Note
      >
      > Use only printable ASCII characters.

   c. In the *System ID* field, there is an automatically assigned ID which is unique for this system.
   d. In the *System Namespace*, enter a unique technical identifier to avoid global ID and name conflicts. It covers vendor, system (application or service), authority namespaces and optionally further sub-context namespaces.
   e. Choose *Add*.

Related Information

Declaring System APIs and Events as Dependencies for Business Scenarios [page 2360]

5.4.8 Deregister or Remove a System

When you no longer need the system to be paired with your global account, you can deregister or remove.

Prerequisites

- You are a global account administrator, or you are a system landscape administrator of the global account where you want to deregister or remove your system. See Working with Role Collections [page 2612].
- You have manually added your system as a record to the list on the **Systems** page. See Adding, Registering and Deregistering Systems [page 2295].

Context

> ⓘ Note
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

Deregistering an SAP systems of type SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud and SAP systems from the SAP Customer Experience portfolio means that the connectivity between this system and the global account is disabled and extension scenarios cannot be established, while removing a system means that this system is no longer part of the systems list. You can deregister or remove a system from the *Actions* column, or from the *System Details* page that you access when selecting the system from the system landscape list. You can remove only manually added systems.

The deregistration and the removal processes have the following states displayed in the cockpit:

- *Deregistering* - a deregistration process has started. As a result, the connectivity between the SAP system of type SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud or SAP systems from the SAP Customer Experience portfolio and SAP BTP is disabled and extension scenarios cannot be established. The system remains in the systems list and you can register it again later on.
  Once a system of type SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud and SAP systems from the SAP Customer Experience portfolio is registered, you can deregister it only after deleting its entitlement configurations and excluding it from the formations it takes part in.
- *Error while Deregistering* - the deregistration has failed. Try to deregister the system again. If the problem persists, you have to report a case in one of the components mentioned below.
- *Removing* - a system removal process has started in the SAP BTP cockpit. As a result, the SAP systems of type SAP SuccessFactors, SAP S/4HANA Cloud, SAP Marketing Cloud and SAP systems from the SAP Customer Experience portfolio is deregistered, and then, it is removed from the systems list completely. To register the system again, first, you must add it to the list anew, and then, initiate the registration procedure. This state also applies to other manually added SAP systems that you are removing.
  Once a system is registered, you can only remove it if you first deregister it.
- *Error while Removing* - the system removal has failed. Try to deregister the system again. If the problem persists, you have to report a case in one of the components mentioned below.

> **Note**
>
> If the deregistration, or removal errors persist, report a case in one of the following components depending on the system type:
>
> - SAP SuccessFactors - `BC-NEO-EXT-SF`
> - SAP S/4HANA Cloud - `BC-NEO-EXT-S4C`
> - All other system types - `BC-CP-MP`

Procedure

1. In the cockpit, navigate to your global account, and then choose *System Landscape* > *Systems*.
2. On the *Systems* page, select the SAP system you want to deregister or remove.
   - To deregister a system, choose *(Deregister system)* from the *Actions* column, or choose *Deregister System* from the *System Details* page that you access when selecting the system from the system landscape list.
     You can deregister only registered systems.

> **Note**
>
> If your system is of type SAP S/4HANA Cloud, SAP SuccessFactors, or part of the SAP Customer Experience portfolio and you deregister this system, you can register back the same system's tenant and not a different one.

- To remove a system, choose *(Remove system)* from the *Actions* column, or choose *Remove System* from the *System Details* page that you access when selecting the system from the system landscape list.
  Before removing a system from the list, first you have to check all configurations that this system is part of, such as entitlements and formations. Then, you have to deregister it and after that you will be able to remove it from the *Systems* page. If you try to remove a system before deregistering it, a dialog will appear and will ask you to deregister the system first.

  > **Note**
  >
  > You can only remove manually added systems. Depending on their discovery mode the systems that are added to the list either are *Manually added* (registered manually by following the procedure in the prerequisites), or *Auto-discovered* (added to the list automatically, based on information of the existing system landscape), or part of the *Subaccount/<my-subaccount>* (automatically added, based on the information of the SAP BTP subaccount).
  >
  > You cannot remove *Auto-discovered* systems from the list.

Results

The system has been deregistered or removed as a record from the list on the *Systems* page in the SAP BTP cockpit.

5.4.9 Merge SAP Systems

Merging the SAP systems allows you to reduce the duplicate systems in the list. You can merge auto-discovered SAP systems without assigned status in a target system that has already been registered.

Prerequisites

- You are a global account administrator, or you are a system landscape administrator of the global account where you want to deregister or remove your SAP system. See Working with Role Collections [page 2612].
- You have registered the target SAP system. See Adding, Registering and Deregistering Systems [page 2295].

Context

The logic of the SAP BTP cockpit enables it to discover SAP systems across your landscape and add them automatically on the *Systems* page. If you have already added these systems manually at a given point in time and you have registered them, the auto-discovered systems might create duplicate entries in the list. To avoid duplicates and streamline the list of systems, you can merge the auto-discovered SAP systems with your manually added systems. The logic automatically detects duplicates based on their URL and system type. Note that you can only merge auto-discovered SAP systems without assigned status in a target system that has already been registered. That is, you cannot merge registered auto-discovered systems.

Procedure

1. In the cockpit, navigate to your global account, and then choose *System Landscape* > *Systems* .
2. On the *Systems* page, find the auto-discovered SAP system you want to merge.
3. Under the *Actions* column, choose the *(Merge system)* button.

   Alternatively, you can choose the system from the list to open the *System Details* view, and then, choose the *Merge System* button on the top-right corner.

   The *Merge System* dialog opens.
4. In the *Target System* dropdown menu, find and select the SAP system, in which you want to merge your system.
5. Choose *Merge*.

Results

The auto-discovered system has been merged in the target system. That is, all individual properties of the system were recorded in the target system and the common properties were preserved. At the end, the duplicate (auto-discovered) system has been removed as a record from the list on the *Systems* page in the SAP BTP cockpit.

5.5 Integrating SAP Solutions

You can integrate several systems and services as part of the common business scenario. To do that, you create a formation in the SAP BTP cockpit that includes all systems that the scenario requires.

A formation is a logical grouping of SAP systems that can be integrated as part of a business scenario. Formations allow you to combine SAP solution systems to simplify the connectivity setup and to provide a unified view of all components required for the implementation of your integration or extension scenario. To create a fully functional formation, you have to include SAP solution systems. You do this configuration once and you can change it anytime.

See Automating Integrations Using Formations [page 2326].

Use Cases

Integrate SAP Systems to Achieve a Business Scenario

This formation use case enables some of the SAP systems included in such a formation to communicate with each other.

These are the formations types following this use case:

- *Integration with SAP Ariba Buying*. See Automating Integrations with SAP Ariba Buying [page 2329].
- *Integration with SAP Start*. See Automating Integrations with SAP Start [page 2331].
- *Data Ingestion for Industry Cloud Solutions*. See Automating Integrations with Data Ingestion for Industry Cloud Solutions [page 2335].
- *Integration with SAP Ariba Central Invoice Management*. See Automating Integrations with SAP Ariba Central Invoice Management [page 2336].
- *Integration with SAP Subscription Billing*. See Automating Integrations with SAP Subscription Billing [page 2338].
- *Integration with SAP Collaboration Manager*. See Automating Integrations with SAP Collaboration Manager [page 2340].
- *Integration with SAP Ariba Central Invoice Management*. See Automating Integrations with SAP Advanced Financial Closing [page 2342].

Integrate SAP Systems with a Service in SAP BTP

There are formations dedicated to automate the integration between supported SAP systems and a service in SAP BTP. In this use case the topology is a star topology where the service is in the center and is communicating with every SAP system that is included in the formation. The formation type is related to the service. The SAP systems that are included do not communicate between each other.

These are the formations types following this use case:

- *Integration with Joule*. See Enabling Joule [page 2348].
- *Eventing Between SAP Cloud Systems*. See Enabling Events Exchange Between SAP Cloud Systems [page 2344].
- *Integration with SAP Master Data Integration*. See Enabling Integration with SAP Master Data Integration [page 2346].

Integrate SAP Systems to Achieve an Extensibility Scenario

Automated configuration allows you to extend SAP solutions without affecting their core functions. It also provides a central repository for APIs and data, ensuring easy access to services for developing extensions.

These are the formations types following this use case:

- *Side-by-Side Extensibility with Kyma*. See Setting Up System Landscape for Kyma [page 2351].
- *Integration with SAP Build*. See Setting Up System Landscape for SAP Build [page 2352].
- *Integration with SAP Integration Suite*. See Setting Up System Landscape for SAP Integration Suite [page 2354].

- *Developing with SAP Business Application Studio*. See Setting Up System Landscape for SAP Business Application Studio [page 2355].

5.5.1 Automating Integrations Using Formations

You can use the automated integration that allows you to include various SAP systems into a formation and thus combine diverse SAP solutions into an extended business scenario.

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].
- The SAP solution systems that you want to include to a formation must be listed in the *Systems* page. See Adding, Registering and Deregistering Systems [page 2295].

Context

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

A formation is a logical grouping of SAP systems following a business scenario. Formations allow you to combine SAP solution systems to simplify the connectivity setup and to provide a unified view of all components required for the implementation of your integration or extension scenario. There are three formation types: formations for business scenarios, for integrating with a specific service in SAP BTP, and for extensibility.

When creating a formation in the SAP BTP cockpit, you include the systems of the different SAP solutions you want to extend. If your business case features more than one SAP solution system, you can use the corresponding button to include additional systems in the formation. You can start the dialog as many times and add systems to your formation as you want.

A formation can have the following status values:

- *Ready*
  The formation has been added as a record to the list and all registered systems are included successfully.
- *Draft*
  Not all the systems were included when creating the formation. Include all the missing systems and choose *Finalize Formation*.
- *Action Required*
  The formation has been created but you cannot use it productively yet.
  Based on the formation type and the subaccount that you specified, it might require an SAP BTP Kyma environment instance or an SAP Business Application Studio subscription. Although the SAP BTP cockpit

allows you to create such a formation, to enable and make use of it, you must also create the respective instance or subscription.

- *Synchronizing*
  Systems that are included in the formation are currently synchronizing in the background.
- *Error*
  An error occurred while some of the systems that are included in the formation were synchronizing in the background. In this case, report an incident in the `BC-CP-MP` component.

When you include systems in a formation, these systems are synchronized in the background. If an error occurs, you can resynchronize these systems to restart the synchronization process. To do that, choose *Resynchronize* for the particular formation. You will have this action available only for formations that are in status *Error*.

If you want to reconfigure the systems in a formation from scratch without excluding them, you have to choose *Reset and Resynchronize*. You will have this action available only for formations that have no status in SAP BTP cockpit and their formation type is *Side-by-Side Extensibility with Kyma*.

> **Note**
>
> When registering a system or creating a formation, the data you provide in the given input fields is not encrypted with your customer managed key. The data you enter is only encrypted at rest.

Procedure

1. Open the SAP BTP cockpit and navigate to your global account.
2. Choose *System Landscape* > *Formations*.
3. Choose *Create Formation*.

   The *Create Formation* dialog opens. There, in a wizard, you can create a formation and include one or more SAP systems to it.
4. On the *General Information* step of the wizard, enter the following information:
   a. Enter a unique formation name.

      The formation name must be up to 128 symbols which can be lowercase or uppercase Latin letters, numbers, hyphens, spaces, or underscores only. The name must start and end with an alphanumeric character.
   b. Specify a type for the formation.

      The formation type defines the use case.

      Depending on the type, a list of systems that can be included in this formation is loaded at the following step of the wizard.
5. Choose *Next Step*.
6. On the *Include Systems* step, select one or more systems that you want to include in the newly created formation, and then, choose *Next Step*.

   The wizard prefilters the systems that were added to the *Systems* page and are valid for the formation type that you specified at the previous step.

7. On the *Review* step, double check your entries before you create the formation.

   If you want to make any changes, either you can edit the corresponding section directly, or use the *Previous* button.
8. Choose *Create*.
9. Optionally, you can include additional systems to the newly created formation, by choosing *Include System*.

   On the systems list that opens, select a system, and then, choose *Include*.

Results

> **Note**
>
> When you delete a formation, several activities are performed at one go. First, the systems are excluded from the formation. Then, the subaccount is unassigned. Finally, the formation is deleted from the *Formations* page completely.
>
> To restore a deleted formation, first you must create it anew, and then, include all of its systems again, one by one.

Next Steps

Once the formation is ready, you might need to configure additionally some of the systems that are part of this formation. You get the configuration details from the formation and then you go to the respective system and add these details. Such a system is the SAP S/4HANA Cloud Private Edition. To get the configuration details, you have to:

1. In the *Formations* page, select the formation you are working with.
2. Choose the system that needs additional configuration to open its details. In this example, this is the SAP S/4HANA Cloud Private Edition system.
3. Go to the *Configuration for System... Provided By* section and in the *Actions* column, choose {·} *(Syntax)*.
4. You get the configuration details in a visual or in a JSON format. Copy the details that you need.

Depending on the system type, use the details you've copied to configure the system.

Related Information

Automating Integrations with SAP Ariba Buying [page 2329]
Automating Integrations with SAP Start [page 2331]
Automating Integrations with Data Ingestion for Industry Cloud Solutions [page 2335]
Automating Integrations with SAP Ariba Central Invoice Management [page 2336]
Automating Integrations with SAP Subscription Billing [page 2338]
Automating Integrations with SAP Collaboration Manager [page 2340]

Automating Integrations with SAP Advanced Financial Closing [page 2342]
Enabling Joule [page 2348]
Enabling Events Exchange Between SAP Cloud Systems [page 2344]
Enabling Integration with SAP Master Data Integration [page 2346]
Setting Up System Landscape for Kyma [page 2351]
Setting Up System Landscape for SAP Build [page 2352]
Setting Up System Landscape for SAP Integration Suite [page 2354]
Setting Up System Landscape for SAP Business Application Studio [page 2355]

5.5.1.1 Formations for Business Scenarios

5.5.1.1.1 Automating Integrations with SAP Ariba Buying

Use

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

SAP Ariba Buying offers procurement users a unique buying experience through a guided process. SAP Ariba Buying provides buyers an easy process to shop for items. Buyers can search for, view and purchase items that are available in the connected shopping sites, or request items from suppliers. SAP Ariba Buying also helps buyers to track their purchase requests.

To set up SAP Ariba Buying with SAP S/4HANA Cloud on SAP BTP you need to have all the necessary systems included in a formation of type *Integration with SAP Ariba Buying* in the *System Landscape* > *Systems* page of the SAP BTP cockpit.

> **Note**
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

Rules

When creating *Integration with SAP Ariba Buying* formations, keep in mind the following rules:

- SAP S/4HANA Cloud systems can be included in at most one *Integration with SAP Ariba Buying* formation.
- Only SAP S/4HANA Cloud systems that are registered with *All Communication Scenarios* or *Integration with SAP Ariba Buying* communication scenario group can be part of a *Integration with SAP Ariba Buying* formation. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].
- You have an Identity Authentication tenant. See Indentity Authentication: Initial Setup.
- (Optional: if you are using the *Set Up SAP Ariba Buying* booster in the SAP BTP cockpit, this step will be automatically done by the booster.) Check that you have a system of type *SAP Ariba Buying* in the *Systems* page. This system is added through a subscription in SAP BTP cockpit associated with a given subaccount. The subscription has been discovered and added automatically through the subaccount. To have a system of type *SAP Ariba Buying* in the *Systems* page, you need to:
  - You have subscribed to SAP Ariba Buying. See Get a Subscription to SAP Ariba Buying.
  - You have subscribed to the services required for SAP Ariba Buying. See Subscribe to Services Required for SAP Ariba Buying.
- Check that you have a system of type *SAP S/4HANA Cloud* in the *Systems* page. This system is auto-discovered because it is associated with your global account and has been discovered and added automatically to the list based on information of the existing system landscape.

  > ### Note
  >
  > If you don't find your system of type *SAP S/4HANA Cloud* in the *Systems* page, add this system manually. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

- Register the SAP S/4HANA Cloud system in the *Systems* page.
  When you register an SAP S/4HANA Cloud system, use the *All Communication Scenarios* or the *Integration with SAP Ariba Buying* communication scenario group when you get the registration token for the SAP S/4HANA Cloud system. This allows the automatic enablement of the communication scenarios `SAP_COM_0545` and `SAP_COM_0A00` after the corresponding system is added to the formation of type *Integration with SAP Ariba Buying*. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Procedure

If you have SAP Ariba Buying set up in your subaccount in SAP BTP, follow these steps to configure the integration between SAP Ariba Buying and SAP S/4HANA Cloud using a formation of type *Integration with SAP Ariba Buying*.

If you want to have the end-to-end setup automatically, including creating the formation, use the *Set Up SAP Ariba Buying* booster in the SAP BTP cockpit to guide you through the steps to configure SAP Ariba Buying in your subaccount and integrate it with SAP S/4HANA Cloud.

1. In the *Systems* page, find the systems of type *SAP Ariba Buying* and *SAP S/4HANA Cloud* that you want to include in a formation.
   The *SAP S/4HANA Cloud* system you are going to include in this formation must not be part of another formation.
2. In the *System Landscape* > *Formations* page, create a formation of type *Integration with SAP Ariba Buying* and include the relevant systems in it.
   1. Add any name that helps you identify your formation.

2. In the *Formation Type* dropdown menu, select *Integration with SAP Ariba Buying*.
3. Select the *SAP Ariba Buying* and *SAP S/4HANA Cloud* systems that you want to include in the formation.
4. Review your selections and create the formation.

Next Steps

When the formation is created, you have SAP Ariba Buying set up with SAP S/4HANA Cloud on SAP BTP.

5.5.1.1.2 Automating Integrations with SAP Start

Use

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

SAP Build Work Zone, standard edition is a predefined central entry point for accessing SAP cloud business solutions that are integrated with it, such as SAP S/4HANA Cloud.

To set up SAP Build Work Zone, standard edition with SAP S/4HANA Cloud on SAP BTP you need to have all the necessary systems included in a formation of type *Integration with SAP Start* in the *System Landscape* > *Systems* page of the SAP BTP cockpit.

> **Note**
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

Rules

When creating *Integration with SAP Start* formations, keep in mind the following rules:

- SAP Build Work Zone system can be included in at most one *Integration with SAP Start* formation.
- At most one SAP Build Work Zone system can be included in an *Integration with SAP Start* formation.
- Only SAP S/4HANA Cloud systems that are registered with *All Communication Scenarios* or *Integration with SAP Start* communication scenario group can be part of a *Integration with SAP Start* formation. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].
- You have an Identity Authentication tenant. See Indentity Authentication: Initial Setup.
- (Optional: if you are using the *Integration with SAP Start* booster in the SAP BTP cockpit, this step will be automatically done by the booster.) Check that you have a system of type *SAP Build Work Zone* in the *Systems* page. This system is added through a subscription in SAP BTP cockpit associated with a given subaccount. The subscription has been discovered and added automatically through the subaccount.
- Check that you have a system of type *SAP S/4HANA Cloud* in the *Systems* list in the *Systems* page. This system is auto-discovered because it is associated with your global account and has been discovered and added automatically to the list based on information of the existing system landscape.

  > ### Note
  >
  > If you don't find your system of type *SAP S/4HANA Cloud* in the *Systems* list, add this system manually. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

- Register the SAP S/4HANA Cloud system in the *Systems* list in the *System Landscape* page.
  When you register an SAP S/4HANA Cloud system, use the *Integration with SAP Start* communication scenario group when you get the registration token for the SAP S/4HANA Cloud system. This allows the automatic enablement of the communication scenario `SAP_COM_0647` after the corresponding system is added to the formation of type *Integration with SAP Start*.
  See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Procedure

If you have SAP Build Work Zone set up in your subaccount in SAP BTP, follow these steps to configure the integration between SAP Build Work Zone and SAP S/4HANA Cloud using a formation of type *Integration with SAP Start*.

If you want to have the end-to-end setup automatically, including creating the formation, use the *Integration with SAP Start* booster in the SAP BTP cockpit to guide you through the steps to configure SAP Build Work Zone in your subaccount and integrate it with SAP S/4HANA Cloud.

1. In the SAP BTP cockpit, in the *Systems* page, find the systems of type *SAP Build Work Zone* and *SAP S/4HANA Cloud* that you want to include in a formation.
   The *SAP Build Work Zone* system you are going to include in this formation must not be part of another formation.
2. In the *System Landscape* > *Formations* page, create a formation of type *Integration with SAP Start* and include the relevant systems in it.
   1. Add any name that helps you identify your formation.
   2. In the *Formation Type* dropdown menu, select *Integration with SAP Start*.
   3. Select the *SAP Build Work Zone* and *SAP S/4HANA Cloud* systems that you want to include in the formation.
   4. Review your selections and create the formation.

Next Steps

When the formation is created, you have SAP Build Work Zone set up with SAP S/4HANA Cloud on SAP BTP. Then, set up SAP Task Center. See Post-Setup Tasks.

5.5.1.1.3 Automating Integrations with SAP Task Center

Use

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

SAP Task Center helps you integrate tasks into a central solution. The SAP Task Center service enables integration with SAP and non-SAP applications to provide a single entry point for end users to access all their assigned tasks. For example, you can work with task from SAP S/4HANA Cloud Public Edition on SAP BTP, Cloud Foundry runtime. See What Is SAP Task Center?.

To set up SAP Task Center with SAP S/4HANA Cloud on SAP BTP you need to have all the necessary systems included in a formation of type *Integration with Customer-Managed SAP Task Center* in the *System Landscape* > *Systems* page of the SAP BTP cockpit.

> **Note**
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

Rules

When creating *Integration with Customer-Managed SAP Task Center* formations, keep in mind the following rules:

- SAP Task Center system can be included in at most one *Integration with Customer-Managed SAP Task Center* formation.
- At most one SAP Task Center system can be included in an *Integration with Customer-Managed SAP Task Center* formation.
- Only SAP S/4HANA Cloud systems that are registered with *All Communication Scenarios* communication scenario group can be part of a *Integration with Customer-Managed SAP Task Center* formation. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].
- Check that you have a system of type *SAP S/4HANA Cloud* in the *Systems* list in the *Systems* page. This system is auto-discovered because it is associated with your global account and has been discovered and added automatically to the list based on information of the existing system landscape.

  > **Note**
  >
  > If you don't find your system of type *SAP S/4HANA Cloud* in the *Systems* list, add this system manually. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

- Register the SAP S/4HANA Cloud system in the *Systems* list in the *System Landscape* page.
  When you register an SAP S/4HANA Cloud system, use the *All Communication Scenarios* communication scenario group when you get the registration token for the SAP S/4HANA Cloud system. This allows the automatic enablement of the communication scenario `SAP_COM_0501` after the corresponding system is added to the formation of type *Integration with Customer-Managed SAP Task Center*.
  See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Procedure

If you have SAP Task Center set up in your subaccount in SAP BTP, follow these steps to configure the integration between SAP Task Center and SAP S/4HANA Cloud using a formation of type *Integration with Customer-Managed SAP Task Center*.

1. In the SAP BTP cockpit, in the *Systems* page, find the systems of type *SAP Task Center* and *SAP S/4HANA Cloud* that you want to include in a formation.
   The *SAP Task Center* system you are going to include in this formation must not be part of another formation of type *Integration with SAP Task Center*.
2. In the *System Landscape* > *Formations* page, create a formation of type *Integration with Customer-Managed SAP Task Center* and include the relevant systems in it.
   1. Add any name that helps you identify your formation.
   2. In the *Formation Type* dropdown menu, select *Integration with Customer-Managed SAP Task Center*.
   3. Select the *SAP Task Center* and *SAP S/4HANA Cloud* systems that you want to include in the formation.
   4. Review your selections and create the formation.

Next Steps

You have set up the connection to SAP Task Center and can receive SAP S/4HANA Cloud Public Edition tasks in the SAP Task Center Web app. For more information on the Web app, see SAP Task Center Web App.

5.5.1.1.4 Automating Integrations with Data Ingestion for Industry Cloud Solutions

Use

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

SAP's industry cloud provides specialized industry-focused solutions to help you optimize, extend, and transform your core business processes. Developed by SAP and SAP partners and based on SAP Business Technology Platform (SAP BTP), industry cloud solutions integrate with SAP S/4HANA Cloud and SAP Business Network. They help you take advantage of the latest industry-relevant innovations and extend existing capabilities in an efficient way. For a list of all industry cloud solutions published by SAP, see SAP Store.

Data ingestion for industry cloud solutions allows supported industry cloud solutions to consume data from a shared data foundation instead of integrating with the source systems individually. This helps to simplify solution implementation, increase operational visibility, and improve data quality. For a list of all industry cloud solutions published by SAP that support data ingestion, see Data Integration Overview.

To set up on SAP BTP systems that support data ingestion and are part of the industry cloud solutions published by SAP, you need to include them in a formation of type *Data Ingestion for Industry Cloud Solutions*. When you create a new formation of type *Data Ingestion for Industry Cloud Solutions*, a system of type *Data Ingestion for Industry Cloud Solutions* is automatically added to the *System Landscape* > *Systems* page and included in this formation.

Rules

When creating *Data Ingestion for Industry Cloud Solutions* formations, keep in mind the following rules:

- Only systems that are part of the industry cloud solutions, support data ingestion, and are provisioned from SAP for Me can be included in formations of type *Data Ingestion for Industry Cloud Solutions*.
- System that is part of the industry cloud solutions published by SAP are included in at most one *Data Ingestion for Industry Cloud Solutions* formation.
- System of type *Data Ingestion for Industry Cloud Solutions* is automatically registered and included in the *Data Ingestion for Industry Cloud Solutions* formation.
- System of type *Data Ingestion for Industry Cloud Solutions* cannot be excluded from the *Data Ingestion for Industry Cloud Solutions* formation. Once you delete this formation, the *Data Ingestion for Industry Cloud Solutions* system is automatically deleted.

Prerequisites

You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].

Procedure

1. In the *System Landscape* > *Systems* page of the SAP BTP cockpit, browse the already added systems that are part of the industry cloud solutions published by SAP in your customer system landscape.
   The system that is part of the industry cloud solutions published by SAP and that you want to include in this formation must not be part of another formation.
2. In the *System Landscape* > *Formations* page, create a formation of type *Data Ingestion for Industry Cloud Solutions* and include the relevant systems in it.
   1. Add any name that helps you identify your formation.
   2. In the *Formation Type* dropdown menu, select *Data Ingestion for Industry Cloud Solutions*.
   3. Select the systems that you want to include in the formation.
   4. Review your selections and create the formation.

Related Information

Data Integration Overview

5.5.1.1.5 Automating Integrations with SAP Ariba Central Invoice Management

Use

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

SAP Ariba Central Invoice Management on SAP BTP provides a unified solution for receiving and managing supplier invoices, with connection to different backend systems like SAP S/4HANA Cloud.

Using a formation of type *Integration with SAP Ariba Central Invoice Management*, you automatically set up an integration between systems of type *SAP Ariba Central Invoice Management* and *SAP S/4HANA Cloud*.

> **Note**
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

Rules

When creating formations of type *Integration with SAP Ariba Central Invoice Management*, keep in mind the following rules:

- System of type *SAP Ariba Central Invoice Management* can be included in at most one *Integration with SAP Ariba Central Invoice Management* formation.
- Formation of type *Integration with SAP Ariba Central Invoice Management* can contain exactly one system of type *SAP Ariba Central Invoice Management*.
- System of type *SAP S/4HANA Cloud* can be included in at most one *Integration with SAP Ariba Central Invoice Management* formation when creating this formation.
- When registering the *SAP S/4HANA Cloud* system, you have to choose one of the following communication scenario groups:
  - All Communication Scenarios
  - Integration with SAP Ariba Central Invoice Management

  See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].
- You have a system of type *SAP Ariba Central Invoice Management* in the *Systems* list. This system is auto-discovered because it is associated with your global account and has been discovered and added automatically to the list based on information of the existing system landscape.
- You have registered the *SAP S/4HANA Cloud* system that you are going to include in the formation of type *Integration with SAP Ariba Central Invoice Management*.
  When you register an SAP S/4HANA Cloud system, use the *All Communication Scenarios* or the *Integration with SAP Ariba Central Invoice Management* communication scenario groups when you get the registration token for the SAP S/4HANA Cloud system. This allows the automatic enablement of the `SAP_COM_0897`, `SAP_COM_0516`, and `SAP_COM_0531` communication scenarios after the corresponding system is added to the formation of type *Integration with SAP Ariba Central Invoice Management*. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].
- You have created a formation of type *Integration with SAP Master Data Integration* and have included in it the *SAP Ariba Central Invoice Management* and the *SAP S/4HANA Cloud* systems. See Enabling Integration with SAP Master Data Integration [page 2346].

Procedure

As an alternative to the steps that follow, you can use the *Set Up SAP Ariba Central Invoice Management* booster in the SAP BTP cockpit and have the end-to-end scenario set up. The booster includes:

- Creating new subaccounts
- Creating required service instances and subscriptions
- Configuring all the required business integrations with the Identity Authentication service, SAP Master Data Integration, and SAP S/4HANA Cloud

> → Tip
>
> We recommend that you use the *Set Up SAP Ariba Central Invoice Management* booster.

1. In the *System Landscape* > *Systems* page of the SAP BTP cockpit, browse the already added systems in your system landscape.
   The *SAP S/4HANA Cloud* and the *SAP Ariba Central Invoice Management* systems you are going to include in this formation must not be part of another formation.
2. In the *System Landscape* > *Formations* page, create a formation of type *Integration with SAP Ariba Central Invoice Management* and include the relevant systems in it.
   1. Add any name that helps you identify your formation.
   2. In the *Formation Type* dropdown menu, select *Integration with SAP Ariba Central Invoice Management*.
   3. Choose *Next Step*.
   4. Select the *SAP Ariba Central Invoice Management* system that you want to include in the formation.
   5. Select the other systems that you want to include. In the list you can select only systems that are allowed to be included in this formation.
   6. Choose *Next Step*.
   7. Review your selections and create the formation.

5.5.1.1.6 Automating Integrations with SAP Subscription Billing

Use

> ⓘ Note
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

SAP Subscription Billing is a public cloud solution that supports innovative, subscription-based business models. It enables flexible definition of product prices and automates processes such as subscription generation, rating, and preparation of billing data. SAP Subscription Billing offers comprehensive management of subscriptions throughout their life cycle. See SAP Subscription Billing.

To set up SAP Subscription Billing with SAP S/4HANA Cloud on SAP BTP you need to have all the necessary systems included in a formation of type *Integration with SAP Subscription Billing* in the *System Landscape* > *Systems* page of the SAP BTP cockpit.

> **Note**
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

Rules

When creating *Integration with SAP Subscription Billing* formations, keep in mind the following rule:

- Formation of type *Integration with SAP Subscription Billing* can contain exactly one system of type SAP Subscription Billing and one system of type SAP S/4HANA Cloud.
- System of type SAP Subscription Billing can be included in at most one *Integration with SAP Subscription Billing* formation.
- System of type SAP S/4HANA Cloud can be included in at most one *Integration with SAP Subscription Billing* formation when creating this formation.
- Only SAP S/4HANA Cloud systems that are registered with *All Communication Scenarios* or the *Integration with SAP Subscription Billing* communication scenario groups, can be part of an *Integration with SAP Subscription Billing* formation. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].
- Register the SAP S/4HANA Cloud system in the *Systems* page.
  When you register an SAP S/4HANA Cloud system, use the *All Communication Scenarios* or the *Integration with SAP Subscription Billing* communication scenario groups when you get the registration token for the SAP S/4HANA Cloud system. This allows the automatic enablement of the `SAP_COM_0845` and `SAP_COM_0642` communication scenarios after the corresponding system is added to the formation of type *Integration with SAP Subscription Billing*. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Procedure

If you have SAP Subscription Billing set up in your subaccount in SAP BTP, follow these steps to configure the integration between SAP Subscription Billing and SAP S/4HANA Cloud using a formation of type *Integration with SAP Subscription Billing*.

1. In the *Systems* page, find the systems of type *SAP Subscription Billing* and *SAP S/4HANA Cloud* that you want to include in a formation.
   The **SAP S/4HANA Cloud** and the *SAP Subscription Billing* systems you are going to include in this formation must not be part of another formation.
2. In the *System Landscape* > *Formations* page, create a formation of type *Integration with SAP Subscription Billing* and include the relevant systems in it.
   1. Add any name that helps you identify your formation.
   2. In the *Formation Type* dropdown menu, select *Integration with SAP Subscription Billing*.
   3. Select the *SAP Subscription Billing* and *SAP S/4HANA Cloud* systems that you want to include in the formation.
   4. Review your selections and create the formation.

Next Steps

When the formation is created, you have SAP Subscription Billing set up with SAP S/4HANA Cloud on SAP BTP. Continue with your setup activities for the integration scope item 57Z or 5IK. See:

- Setup Guide for 57Z (Subscription Management with Sales Billing)
- Setup Guide for 5IK (Subscription Management with Convergent Invoicing)

5.5.1.1.7 Automating Integrations with SAP Collaboration Manager

Use

> ⓘ Note
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

SAP Collaboration Manager is a chat solution that is integrated into the SAP Fiori launchpad, enabling you to accomplish tasks quickly in your business applications. Collaborate with colleagues on activities, create notes and chats, and share attachments and screenshots.

To set up SAP Collaboration Manager with SAP S/4HANA Cloud on SAP BTP, you need to have all the necessary systems included in a formation of type *Integration with SAP Collaboration Manager* in the *System Landscape* > *Systems* page of the SAP BTP cockpit.

> ⓘ Note
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

Rules

When creating *Integration with SAP Collaboration Manager* formations, keep in mind the following rule:

- Only SAP S/4HANA Cloud systems that are registered with the *All Communication Scenarios* communication scenario group can be part of a *Integration with SAP Collaboration Manager* formation. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].
- You have an Identity Authentication tenant. See Identity Authentication: Initial Setup.
- You have an SAP Collaboration Manager subscription. See Subscribe to SAP Collaboration Manager.
- Register the SAP S/4HANA Cloud system in the *Systems* page.
  When you register an SAP S/4HANA Cloud system, use the *All Communication Scenarios* communication scenario group when you get the registration token for the SAP S/4HANA Cloud system. This allows the automatic enablement of the `SAP_COM_0835` communication scenario after the corresponding system is added to the formation of type *Integration with SAP Collaboration Manager*. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Procedure

If you have SAP Collaboration Manager set up in your subaccount in SAP BTP, follow these steps to configure the integration between SAP Collaboration Manager and SAP S/4HANA Cloud using a formation of type *Integration with SAP Collaboration Manager*.

As an alternative to the steps that follow, you can use the *SAP Collaboration Manager* booster in the SAP BTP cockpit and have the end-to-end scenario set up. See Run the SAP Collaboration Manager Booster.

> **Note**
>
> We recommend that you use the *SAP Collaboration Manager* booster.

1. In the *Systems* page, find the systems of type *SAP Collaboration Manager* and *SAP S/4HANA Cloud* that you want to include in a formation.
2. In the *System Landscape* > *Formations* page, create a formation of type *Integration with SAP Collaboration Manager* and include the relevant systems in it.
   1. Add any name that helps you identify your formation.
   2. In the *Formation Type* dropdown menu, select *Integration with SAP Collaboration Manager*.
   3. Select the *SAP Collaboration Manager* and *SAP S/4HANA Cloud* systems that you want to include in the formation.
   4. Review your selections and create the formation.

Next Steps

When the formation is created, you have SAP Collaboration Manager set up with SAP S/4HANA Cloud on SAP BTP. Then, follow the next steps at Run the SAP Collaboration Manager Booster: Next Steps.

5.5.1.1.8 Automating Integrations with SAP Advanced Financial Closing

Use

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

SAP Advanced Financial Closing supports you in planning, processing, monitoring, and analyzing financial closing tasks for the entities of your group. See SAP Advanced Financial Closing User Guide.

To set up SAP Advanced Financial Closing with SAP S/4HANA Cloud on SAP BTP you need to have all the necessary systems included in a formation of type *Integration with SAP Advanced Financial Closing* in the *System Landscape* > *Systems* page of the SAP BTP cockpit.

> **Note**
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

Rules

When creating *Integration with SAP Advanced Financial Closing* formations, keep in mind the following rule:

- Only SAP S/4HANA Cloud systems that are registered with *All Communication Scenarios* or the *Integration with SAP Advanced Financial Closing* communication scenario groups, can be part of a *Integration with SAP Advanced Financial Closing* formation. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].

- You have an Identity Authentication tenant. See Indentity Authentication: Initial Setup.
- Register the SAP S/4HANA Cloud system in the *Systems* page.
  When you register an SAP S/4HANA Cloud system, use the *All Communication Scenarios* or the *Integration with SAP Advanced Financial Closing* communication scenario groups when you get the registration token for the SAP S/4HANA Cloud system. This allows the automatic enablement of the `SAP_COM_0566` communication scenario after the corresponding system is added to the formation of type *Integration with SAP Advanced Financial Closing*. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Procedure

If you have SAP Advanced Financial Closing set up in your subaccount in SAP BTP, follow these steps to configure the integration between SAP Advanced Financial Closing and SAP S/4HANA Cloud using a formation of type *Integration with SAP Advanced Financial Closing*.

As an alternative to the steps that follow, you can use the *Set Up SAP Advanced Financial Closing* booster in the SAP BTP cockpit and have the end-to-end scenario set up. See SAP Advanced Financial Closing Administration Guide: Automated Setup.

> **Note**
>
> We recommend that you use the *Set Up SAP Advanced Financial Closing* booster.

1. In the *Systems* page, find the systems of type *SAP Advanced Financial Closing* and *SAP S/4HANA Cloud* that you want to include in a formation.
2. In the *System Landscape* > *Formations* page, create a formation of type *Integration with SAP Advanced Financial Closing* and include the relevant systems in it.
   1. Add any name that helps you identify your formation.
   2. In the *Formation Type* dropdown menu, select *Integration with SAP Advanced Financial Closing*.
   3. Select the *SAP Advanced Financial Closing* and *SAP S/4HANA Cloud* systems that you want to include in the formation.
   4. Review your selections and create the formation.

Next Steps

When the formation is created, you have SAP Advanced Financial Closing set up with SAP S/4HANA Cloud on SAP BTP. See the *Next Steps* section at SAP Advanced Financial Closing Administration Guide: Automated Setup.

5.5.1.2 Formations for Services in SAP BTP

5.5.1.2.1 Enabling Events Exchange Between SAP Cloud Systems

Use

> ⓘ Note
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

Systems that are included in *Eventing Between SAP Cloud Systems* formations on the *System Landscape* page of the SAP BTP cockpit can publish and consume events. The process of publishing and consuming events is in fact an exchange of event information across the customer system landscape and is driven by the system formations on the one hand side and the SAP Cloud Application Event Hub on the other.

> ⓘ Note
>
> By creating a formation of type *Eventing Between SAP Cloud Systems*, keep in mind that there may be event distribution charges incurred if you exceed the free quota limit when using SAP Cloud Application Event Hub. To learn more about usage and consumption costs, see Usage and Consumption Costs.

Rules

When creating *Eventing Between SAP Cloud Systems* formations, keep in mind the following rule:

- Every system can be included in at most one *Eventing Between SAP Cloud Systems* formation.
- If you register your SAP S/4HANA Cloud system using the *All Communication Scenarios* or *Eventing Between SAP Cloud Systems* communication scenario group when you get the registration token for this SAP S/4HANA Cloud system, the communication scenario `SAP_COM_0892` will be enabled automatically after this SAP S/4HANA Cloud system is added to the formation of type *Eventing Between SAP Cloud Systems* in the *Formations* page. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].
- You have subscribed to the SAP Cloud Application Event Hub application. See Subscribing to SAP Cloud Application Event Hub Application.

Procedure

The following procedure outlines the steps you need to perform to enable the exchange of events across the systems within the system landscape.

1. In the SAP BTP cockpit, in the *System Landscape* > *Systems* page of the SAP BTP cockpit, browse the already added systems in your customer system landscape or manually add and register any missing systems.
   Every system you are going to include in this formation must not be part of another formation.
   The customer system landscape features systems that are added to the list in one of the following ways:
   - *Auto-discovered*
     An auto-discovered system is a system (associated with the given global account) that has been discovered and added automatically to the list based on information of the existing system landscape. Any SAP system of the supported system types that is associated with the same customer ID, with which your global account in SAP BTP is associated, will be added automatically in the system landscape list.
   - *Subaccount/<my-subaccount>*
     A subscription in SAP BTP cockpit associated with a given subaccount. The subscription has been discovered and added automatically through the subaccount. Check the value of the *Discovery* column to see the subaccount where your system is subscribed.
   - *Manually added*
     Specifies that the system has been added to the list manually by the global account administrator, using the *Add System* button and completing the wizard. The system has been associated with the global account in SAP BTP.

   > ### Note
   >
   > If a given SAP system is missing on the *Systems* page, it may be associated with a different customer ID on the SAP BTP global account you are working in. In this case, you need to add the system manually, and then, register it.

   > ### → Tip
   >
   > When you register an SAP S/4HANA Cloud system, you can automate some of the required configuration steps at a later point. To do this, you can use the *All Communication Scenarios* or *Eventing Between SAP Cloud Systems* communication scenario groups when you get the registration token for the SAP S/4HANA Cloud system. This allows the automatic enablement of the communication scenario `SAP_COM_0892` after the corresponding system is added to the formation of type *Eventing Between SAP Cloud Systems*.

   See Adding, Registering and Deregistering Systems [page 2295].
2. Create a formation of type *Eventing Between SAP Cloud Systems* and include the relevant systems in it.
   1. Add any name that helps you identify your formation.
   2. In the *Formation Type* dropdown menu, select *Eventing Between SAP Cloud Systems*.
   3. Select the systems that you want to include in the formation. One of these systems must be of type *SAP Cloud Application Event Hub*.

      > ### Note
      >
      > Systems can only be added to one formation of type *Eventing Between SAP Cloud Systems* in a global account.

> Also, a formation of type *Eventing Between SAP Cloud Systems* can contain only one system of type *SAP Cloud Application Event Hub*.

4. Review your selections and create the formation.

Next Steps

When the formation is created, navigate to the SAP Cloud Application Event Hub application and enable the event subscriptions. See Enabling SAP Event Subscriptions.

Remove a System from the Formation

If you want to remove an SAP system from a formation of type *Eventing Between SAP Cloud Systems*, make sure that all active event subscriptions are disabled first in SAP Cloud Application Event Hub. You can check your subscriptions in the SAP Cloud Application Event Hub application on the *Subscriptions* page. See Disabling SAP Event Subscriptions.

5.5.1.2.2 Enabling Integration with SAP Master Data Integration

Use

> ⓘ Note
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

SAP Master Data Integration service is a central master data hub. Applications integrate with SAP Master Data Integration to synchronize their local master data databases with the master data database of the central hub. In a typical setup, there is exactly one tenant of SAP Master Data Integration for each landscape. See What Is Master Data Integration?.

Using a formation of type *Integration with SAP Master Data Integration*, you automatically set up an integration between systems of type SAP Master Data Integration and other systems or applications, for example SAP S/4HANA Cloud.

> ⓘ Note
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

Rules

When creating formations of type *Integration with SAP Master Data Integration*, keep in mind the following rules:

- System of type *SAP Master Data Integration* can be included in at most one *Integration with SAP Master Data Integration* formation.
- Formation of type *Integration with SAP Master Data Integration* can contain exactly one system of type *SAP Master Data Integration*.
- System of type *SAP S/4HANA Cloud* can be included in at most one *Integration with SAP Master Data Integration* formation.
- When registering the *SAP S/4HANA Cloud* system, you have to choose one of the following communication scenario groups:
  - All Communication Scenarios
  - Integration with SAP Master Data Integration
  - Integration with SAP Ariba Central Invoice Management

  See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].
- You have a system of type *SAP Master Data Integration* in the *Systems* list. This system is auto-discovered because it is associated with your global account and has been discovered and added automatically to the list based on information of the existing system landscape.
- You have registered the *SAP S/4HANA Cloud* system that you are going to include in the formation of type *Integration with SAP Master Data Integration*.
  When you register an SAP S/4HANA Cloud system, use the *All Communication Scenarios* or the *Integration with SAP Master Data Integration* communication scenario groups when you get the registration token for the SAP S/4HANA Cloud system. This allows the automatic enablement of the `SAP_COM_0008`, `SAP_COM_0091`, `SAP_COM_0659`, and `SAP_COM_0594` communication scenarios after the corresponding system is added to the formation of type Integration with SAP Master Data Integration.

Procedure

1. In the *System Landscape* > *Systems* page of the SAP BTP cockpit, browse the already added systems in your system landscape.
   The **SAP S/4HANA Cloud** and the *SAP Master Data Integration* systems you are going to include in this formation must not be part of another formation.
2. In the *System Landscape* > *Formations* page, create a formation of type *Integration with SAP Master Data Integration* and include the relevant systems in it.
   1. Add any name that helps you identify your formation.

2. In the *Formation Type* dropdown menu, select *Integration with SAP Master Data Integration*.
3. Choose *Next Step*.
4. Select the *SAP Master Data Integration* system that you want to include in the formation.
5. Select the other systems that you want to include. In the list you can select only systems that are allowed to be included in this formation.
6. Choose *Next Step*.
7. Review your selections and create the formation.

Next Steps

Depending on the specific scenario, the next steps would be different. For example, if an SAP Ariba Central Invoice Management system is included in a formation of type *Integration with SAP Master Data Integration*, then continue with the steps described in Automating Integrations with SAP Ariba Central Invoice Management [page 2336].

5.5.1.2.3 Enabling Joule

Use

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

Leveraging its generative AI capabilities, Joule aims to understand your requests, retrieve information, and complete your tasks in a conversational way. You can describe the current task to Joule, who can help automate tasks or take you to places where you can find desired data. By using Joule, you can easily interact with the system and increase your productivity.

To use Joule in SAP systems, you have to integrate these systems with Joule. To do that, you have to include the SAP systems in a formation of type *Integration with Joule* on the *System Landscape* > *Formations* page of the SAP BTP cockpit. The integration is between the Joule system and every SAP system in the formation, but not between each and every SAP system.

To see which SAP systems are supported, see Joule Capabilities.

Rules

When creating *Integration with Joule* formations, keep in mind the following rules:

- At most one system of type *Joule* can be included in one *Integration with Joule* formation.
- A system of type *Joule* can be included in at most one *Integration with Joule* formation.
- Applicable to systems of type SAP Integrated Business Planning: you can include these systems in a formation of type *Integration with Joule* only if they are registered using the *All Communication Scenarios* communication scenario group. See Register an SAP Integrated Business Planning System in a Global Account in SAP BTP [page 2308].
- Applicable to systems of type SAP SuccessFactors: you can include these systems in a formation of type *Integration with Joule* only after they have been registered. See Register an SAP SuccessFactors System in a Global Account in SAP BTP [page 2312].
- Applicable to systems of type SAP S/4HANA Cloud: you can include these systems in a formation of type *Integration with Joule* only if they are registered using the *All Communication Scenarios* communication scenario group. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

> **Note**
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].
- You have a Joule system and at least one other system of the supported types that fulfill the rules in the *Systems* page. The Joule system is auto-discovered.

Procedure

The following procedure outlines the steps you need to perform to enable the integration between Joule and SAP systems.

As an alternative to the steps that follow, you can use the *Setting Up Joule* booster in the SAP BTP cockpit and have the end-to-end scenario set up. See Run the Joule Booster.

> **Note**
>
> We recommend that you use the *Setting Up Joule* booster.

1. In the SAP BTP cockpit, in the *System Landscape* > *Systems* page of the SAP BTP cockpit, browse the already added systems in your customer system landscape or manually add and register any missing systems.
   The *Joule* system you are going to include in this formation must not be part of another formation.

The customer system landscape features systems that are added to the list in one of the following ways:

- *Auto-discovered*
  An auto-discovered system is a system (associated with the given global account) that has been discovered and added automatically to the list based on information of the existing system landscape. Any SAP system of the supported system types that is associated with the same customer ID, with which your global account in SAP BTP is associated, will be added automatically in the system landscape list.
- *Subaccount/<my-subaccount>*
  A subscription in SAP BTP cockpit associated with a given subaccount. The subscription has been discovered and added automatically through the subaccount. Check the value of the *Discovery* column to see the subaccount where your system is subscribed.
- *Manually added*
  Specifies that the system has been added to the list manually by the global account administrator, using the *Add System* button and completing the wizard. The system has been associated with the global account in SAP BTP.

> **Note**
>
> If a given SAP system is missing on the *Systems* page, it may be associated with a different customer ID on the SAP BTP global account you are working in. In this case, you need to add the system manually, and then, register it.

See Adding, Registering and Deregistering Systems [page 2295].

2. Create a formation of type *Integration with Joule* and include the relevant systems in it.
   1. Add any name that helps you identify your formation.
   2. In the *Formation Type* dropdown menu, select *Integration with Joule*.
   3. Select the systems that you want to include in the formation. One of these systems must be of type *Joule*.

      > **Note**
      >
      > Systems can only be added to one formation of type *Integration with Joule* in a global account.
      >
      > Also, a formation of type *Integration with Joule* can contain only one system of type *Joule*.

   4. Review your selections and create the formation.

Next Steps

The next steps of the different SAP solutions integrated with Joule are listed at Integrating Joule with SAP Solutions.

5.5.1.3 Formations for Extensibility

5.5.1.3.1 Setting Up System Landscape for Kyma

Use

> ⓘ Note
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

You want to extend the functionality on top of your existing SAP solution because your specific business needs require it. You also want to use the SAP BTP Kyma environment. You can build an extension application that contains your business logic and UI changes and deploy it in SAP BTP Kyma runtime. For the users of your SAP solution, this change will be seamless, it will look as if it's part of the original functionality. At the same time, you can use the extension capabilities of SAP BTP to implement the additional workflows or modules. This is called side-by-side extensibility wth Kyma.

Procedure

To enable the side-by-side extensibility with Kyma for your SAP solution, you have to:

1. Create an Kyma environment instance in your subaccount in SAP BTP.
2. Add the Application Connector module on your Kyma runtime.
3. Connect the corresponding SAP system with the global account in SAP BTP. Use the automated configurations for the following SAP solutions:
   - Extending SAP S/4HANA Cloud in the Cloud Foundry and Kyma Environment [page 2363]
   - Extending SAP Marketing Cloud in the Cloud Foundry and Kyma Environment [page 2426]
   - Extending SAP SuccessFactors in the Cloud Foundry and Kyma Environment [page 2430]
   - Extending SAP Customer Experience Products in the Kyma Environment [page 2454]
4. Create a formation of type *Side-by-Side Extensibility with Kyma* where you can include one or more SAP systems depending on your use case.

5.5.1.3.2 Setting Up System Landscape for SAP Build

Use

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

A registered SAP S/4HANA Cloud system in the SAP BTP cockpit can expose consumption bundles that contain APIs and events. You can easily discover and consume the APIs exposed by the SAP S/4HANA Cloud systems in your system landscape when you develop and extend applications on SAP BTP, Cloud Foundry environment, using SAP Build. To do this, first you need to enable connectivity between your system landscape in SAP BTP cockpit and SAP Build. The integration requires performing several configuration steps starting with configuration on a global account level, and then, configuration on a subaccount level.

> **Note**
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].
- You have an SAP Build subscription.

Procedure

The following procedure outlines the steps you need to perform to consume the APIs of registered SAP S/4HANA Cloud systems within SAP Build.

1. Add and then register an SAP system of type *SAP S/4HANA Cloud* in the *System Landscape* > *Systems* page of SAP BTP cockpit.
   To expose information about its APIs and events and show this information on the *Systems* page, an SAP system of type *SAP S/4HANA Cloud* must be registered in the SAP BTP cockpit. Only when registered, the system communicates information about its APIs and other technical details across the landscape. See Extending SAP S/4HANA Cloud in the Cloud Foundry and Kyma Environment [page 2363].
2. In the *Formations* page, create a formation of type *Integration with SAP Build* and include the SAP Build system and the SAP S/4HANA Cloud systems you want to expose in SAP Build.

> **Note**
>
> In the formation type *Integration with SAP Build*, you can include only SAP systems of type *SAP Build* and *SAP S/4HANA Cloud*.

To enable connectivity between given SAP systems of type *SAP S/4HANA Cloud* from the *Systems* page of SAP BTP cockpit and SAP Build, you must create a formation of the corresponding type and include the SAP S/4HANA Cloud systems in it. See Automating Integrations Using Formations [page 2326].

3. Create a destination and make sure it has the corresponding system and consumption bundle properties.
   - Create a destination manually.
     You can create the destination manually via the SAP Destination service page in SAP BTP cockpit by specifying one of the following additional properties variants in it.

Additional Properties (variant 1)	Additional Properties (variant 2)	Description
`x-correlation-id`	`x-correlation-id`	The property identifies the consumption bundle and its APIs that are exposed by the SAP S/4HANA Cloud system. The destination also provides the required credentials to consume the bundle and use it for further development in SAP Build.
`x-system-id`	`x-system-name`	Properties that uniquely identify the local tenant of the registered SAP S/4HANA Cloud system.
`x-system-type`	`x-system-base-url`	

     All of the required properties of a given SAP S/4HANA Cloud system are accessible in the corresponding *System Details* section on the *Systems* page.

   - Create a destination automatically.
     Alternatively, the destination can be created automatically. You can do this, by creating a service instance of the SAP S/4HANA Cloud Extensibility service after you register an SAP S/4HANA Cloud system. See Extending SAP S/4HANA Cloud in the Cloud Foundry and Kyma Environment [page 2363] and Create a Service Instance to Consume the SAP S/4HANA Cloud APIs [page 2368].

Next Steps

Develop applications on SAP BTP, Cloud Foundry environment using SAP Build and the system landscape.

When you enable the connectivity, the SAP S/4HANA Cloud systems and their APIs are accessible in SAP Build. Next you can use the APIs to develop new or extend the existing functionality with the help of SAP Build.

Related Information

SAP Build Process Automation: Using SAP Systems

5.5.1.3.3 Setting Up System Landscape for SAP Integration Suite

Use

> ⓘ Note
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

A registered SAP S/4HANA Cloud system in the SAP BTP cockpit can expose consumption bundles that contain APIs and events. You can easily discover and consume the APIs exposed by the SAP S/4HANA Cloud systems in your system landscape when you develop and extend applications on SAP BTP, Cloud Foundry environment. To do this, first you need to enable connectivity between your system landscape in SAP BTP cockpit and SAP Integration Suite. The integration requires performing several configuration steps starting with configuration on a global account level, and then, configuration on a subaccount level.

> ⓘ Note
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

Rules

When creating *Integration with SAP Integration Suite* formations, keep in mind the following rule:

- At most one system of type *SAP Integration Suite* can be included in one *Integration with SAP Integration Suite* formation.

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].
- You have a subscription to SAP Integration Suite. See Subscribing and Configuring Initial Access to SAP Integration Suite.

  > ⓘ Note
  >
  > Every time you create a subscription to a new SAP Integration Suite tenant, the system type for the tenant is automatically defined as SAP Integration Suite.

Procedure

The following procedure outlines the steps you need to perform to consume the APIs of registered SAP S/4HANA Cloud systems within SAP Integration Suite.

1. Add and then register an SAP system of type *SAP S/4HANA Cloud* in the *System Landscape* > *Systems* page of SAP BTP cockpit.
   To expose information about its APIs and events and show this information on the *Systems* page, an SAP system of type *SAP S/4HANA Cloud* must be registered in the SAP BTP cockpit. Only when registered, the system communicates information about its APIs and other technical details across the landscape. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].
2. In the *System Landscape* > *Formations* page, create a formation of type *Integration with SAP Integration Suite* and include the SAP S/4HANA Cloud systems you want to expose to SAP Integration Suite. See Automating Integrations Using Formations [page 2326].

   > ### Note
   >
   > In the formation type *Integration with SAP Integration Suite*, you can include only SAP systems of type *SAP S/4HANA Cloud*.

5.5.1.3.4 Setting Up System Landscape for SAP Business Application Studio

Use

> ### Note
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

A registered SAP S/4HANA Cloud system in the SAP BTP cockpit can expose consumption bundles that contain APIs and events. You can easily discover and consume the APIs exposed by the SAP S/4HANA Cloud systems in your system landscape when you develop and extend applications on SAP BTP, Cloud Foundry environment, using SAP Business Application Studio. To do this, first you need to enable connectivity between your system landscape in SAP BTP cockpit and SAP Business Application Studio. The integration requires performing several configuration steps starting with configuration on a global account level, and then, configuration on a subaccount level.

> ### Note
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

Prerequisites

- You are a global account administrator, or you are a system landscape administrator. See Working with Role Collections [page 2612].
- You have an SAP Business Application Studio subscription. See Subscribe to SAP Business Application Studio.

Procedure

The following procedure outlines the steps you need to perform to consume the APIs of registered SAP S/4HANA Cloud systems within the SAP Business Application Studio.

1. In the *System Landscape* > *Systems* page of SAP BTP cockpit, add and then register an SAP system of type *SAP S/4HANA Cloud*.
   To expose information about its APIs and events and show this information on the *Systems* page, an SAP system of type *SAP S/4HANA Cloud* must be registered in the SAP BTP cockpit. Only when registered, the system communicates information about its APIs and other technical details across the landscape. See Extending SAP S/4HANA Cloud in the Cloud Foundry and Kyma Environment [page 2363].
2. In the *System Landscape* > *Formations* page, create a formation of type *Developing with SAP Business Application Studio* and include the SAP Business Application Studio system and the SAP S/4HANA Cloud systems you want to expose in SAP Business Application Studio.

   > ### Note
   >
   > In the formation type *Developing with SAP Business Application Studio*, you can include only SAP systems of type *SAP Business Application Studio* and *SAP S/4HANA Cloud*.

   To enable connectivity between given SAP systems of type *SAP S/4HANA Cloud* from the *Systems* page of SAP BTP cockpit and the SAP Business Application Studio, you must create a formation of the corresponding type and include the SAP S/4HANA Cloud systems in it. See Automating Integrations Using Formations [page 2326].
3. Create a destination and make sure it has the corresponding system and consumption bundle properties.
   - Create a destination manually.
     You can create the destination manually via the SAP Destination service page in SAP BTP cockpit by specifying one of the following additional properties variants in it.

Additional Properties (variant 1)	Additional Properties (variant 2)	Description
`x-correlation-id`	`x-correlation-id`	The property identifies the consumption bundle and its APIs that are exposed by the SAP S/4HANA Cloud system. The destination also provides the required credentials to consume the bundle and use it for further development in the SAP Business Application Studio.
`x-system-id`	`x-system-name`	Properties that uniquely identify the local tenant of the registered SAP S/4HANA Cloud system.
`x-system-type`	`x-system-base-url`	

All of the required properties of a given SAP S/4HANA Cloud system are accessible in the corresponding *System Details* section on the *Systems* page.

- Create a destination automatically.
  Alternatively, the destination can be created automatically. You can do this, by creating a service instance of the SAP S/4HANA Cloud Extensibility service after you register an SAP S/4HANA Cloud system. See Extending SAP S/4HANA Cloud in the Cloud Foundry and Kyma Environment [page 2363] and Create a Service Instance to Consume the SAP S/4HANA Cloud APIs [page 2368].

Next Steps

Develop applications on SAP BTP, Cloud Foundry environment using SAP Business Application Studio and the system landscape.

When you enable the connectivity, the SAP S/4HANA Cloud systems and their APIs are accessible in the SAP Business Application Studio. Next you can use the APIs to develop new or extend the existing functionality with the help of SAP Business Application Studio. See Unified Customer Landscape Service Provider.

5.6 Extending SAP Solutions

The extension capabilities of SAP Business Technology Platform (SAP BTP) enables developers to implement loosely coupled extension applications securely, thus implementing additional workflows or modules on top of the existing SAP cloud solution they already have.

Overview

> ⓘ Note
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

With the automated configuration you have the following key benefits:

- A way of extending standard SAP solutions without disrupting their performance and core processes
- Frameworks that offer simplified, standardized, and unified extensibility and configuration for SAP solutions
- A central repository for solutions' APIs, events, credentials, and other data, thereby ensuring easy access to services while creating your extensions

You can also use dedicated formations that help you integrate automaticatically the SAP solution you want to extend with runtimes and tools you might need. See:

- Setting Up System Landscape for Kyma [page 2351]
- Setting Up System Landscape for SAP Build [page 2352]
- Setting Up System Landscape for SAP Integration Suite [page 2354]
- Setting Up System Landscape for SAP Business Application Studio [page 2355]

Process Flow

You can use the extension capabilities of SAP BTP to implement additional workflows or modules on top of your existing SAP solutions. You can extend one or more SAP solutions grouped together in a common business case.

To enable the integration, you need to:

1. Connect the corresponding SAP system with the global account.
   - For systems of type SAP S/4HANA Cloud, SAP Marketing Cloud, SAP SuccessFactors and SAP systems from the SAP Customer Experience portfolio, during the pairing process you create an registration token which is then used by the SAP system administrator to configure the registration on the SAP system side. See Adding, Registering and Deregistering Systems [page 2295].
   - For systems of type different than SAP S/4HANA Cloud, SAP Marketing Cloud, SAP SuccessFactors and SAP systems from the SAP Customer Experience portfolio, the SAP system is already registered when it's part of the *Systems* page. However, in the *Status* column, there is no status shown. See Adding, Registering and Deregistering Systems [page 2295].

> **Note**
>
> You cannot migrate the registered SAP systems between global accounts.
>
> If you want to start using another global account, you will have to register your SAP systems again.

2. Optional: Group SAP systems so that they can be extended in a business scenario at one go.
   To do so, you create a formation containing one or more different systems assigned to a common subaccount. See Automating Integrations Using Formations [page 2326].
3. Make the SAP system accessible in the subaccounts in which you want to build your extension applications.
   For systems of type SAP S/4HANA Cloud, SAP Marketing Cloud, and SAP SuccessFactors, configure the entitlements and assign the corresponding quota and service plans to the subaccounts where the extension applications will reside. The service plans define the access to the corresponding SAP solution APIs. See:
   - Configure the Entitlements for the SAP S/4HANA Cloud Extensibility Service [page 2367]
   - Configure the Entitlements for the SAP SuccessFactors Extensibility Service [page 2432]

   For the systems of type different than SAP S/4HANA Cloud, SAP Marketing Cloud, and SAP SuccessFactors, you can continue with developing your extension application.
4. For systems of type SAP S/4HANA Cloud, SAP Marketing Cloud, and SAP SuccessFactors, configure the communication flow for the extension application.
   To be able to consume the SAP S/4HANA Cloud, SAP Marketing Cloud, and SAP SuccessFactors APIs, you need to create a service instance for the corresponding SAP solution system. During the creation of the service instance, you configure the communication flow between the subaccount and the corresponding SAP solution. An HTTP destination which contains the binding properties for establishing the connection is automatically generated.
   After you have created the service instance, you have two options to configure the extension application's connectivity to the corresponding SAP solution:
   - Consume the HTTP destination
   - Bind the service instance to the extension application

Related Information

Extending SAP S/4HANA Cloud in the Cloud Foundry and Kyma Environment [page 2363]
Extending SAP SuccessFactors in the Cloud Foundry and Kyma Environment [page 2430]
Extending SAP Customer Experience Products in the Kyma Environment [page 2454]
Extending SAP Marketing Cloud in the Cloud Foundry and Kyma Environment [page 2426]

5.6.1 Declaring System APIs and Events as Dependencies for Business Scenarios

Context

For certain formation types, systems of type *Other System Type*, *SAP BTP Application*, or *SAP Integration Suite* that are part of the *System Landscape* page may need to describe their runtime dependencies so other systems can comply with these dependencies to be able to interact with these systems in a common scenario.

The integration dependency includes a list of requirements, which point out which API and event resources are involved. Each requirement describes one aspect and can be used to express alternatives using OR conditions for achieving the same outcome.

To add the necessary API and event resources to an integration dependency, systems use the Open Resource Discovery standard. The Open Resource Discovery standard is about adding missing or common high-level information and standardizing the discovery aspects. See Why ORD .

Every integration dependency has the following properties:

- Name: a human-readable name that does not exceed 255 characters. For example, **`Integration Dependency to realize Customer Order data product`**.
- Description: a plain text that does not exceed 255 characters. For example, **`SAP S/4HANA Cloud, our next generation cloud ERP suite designed for in-memory computing.`**
- Version: the version is assigned automatically when adding a new integration dependency.

> **Note**
>
> If you add a new integration dependency using the same name as an existing integration dependency, the version of the new integration dependency will be automatically increased. Then, in the given system, there will be two integration dependencies with the same name and different versions.

Depending on your use case, there are different templates that help you set up the necessary integration dependency:

- Blank template
  You write integration dependencies from scratch inluding the aspects that contain a set of APIs and/or events of a system that define the dependencies other systems need to comply with to be able to be part of an integration scenario together.
- Simplified Business Eventing template
  Integration dependencies are exposed as available event subscriptions in SAP Cloud Application Event Hub. This template helps you write simplified integration dependencies that identify the types of events a subscribing application wants to consume. Here's an example. Let's say SAP SuccessFactors publishes a set of events. You want to receive a subset of those events. You can use the *Simplified Business Eventing Template* to identify the event types that SAP Cloud Application Event Hub makes available to your subscribing system.

5.6.1.1 Add an Integration Dependency Using a Blank Template

Prerequisites

You have a system of type *SAP BTP Application* listed in the *Systems* page of your global account in SAP BTP. See Register a System of Type SAP BTP Application [page 2320].

Context

Integration dependencies contain aspects. An aspect contains a set of APIs and/or events of a system that define the dependencies other systems need to comply with to be able to be part of an integration scenario together.

Procedure

1. In the cockpit, navigate to your global account, and then choose *System Landscape* > *Systems*.
2. On the *Systems* page, select a system of type *Other System Type* to which you want to add an integration dependency and open its system details.
3. In the *Integration Dependencies* tab, choose *Add*.
4. Select *Blank Template* to write an integration dependency from scratch.
5. Define a name and a description for the new integration dependency. Choose *Next Step*.

   > **Note**
   >
   > The version is assigned automatically when adding a new integration dependency. If you add a new integration dependency using the same name as an existing integration dependency, the version of the new integration dependency will be automatically increased.

6. Define a name and a description for the first aspect and add more if necessary. Choose *Next Step*.
7. Add at least one API or event to each aspect and choose *Review*.
8. Check if all the aspects are configured properly and choose *Add*.

   > **Note**
   >
   > Once you add an integration dependency to a system, you can no longer edit it and add or delete aspects, APIs or events. You can only delete an integration dependency.

5.6.1.2 Add an Integration Dependency Using a Simplified Business Eventing Template

Prerequisites

- An SAP Cloud system is registered in the *Systems* page in SAP BTP cockpit and has a catalog of events that it plans to publish. Refer to the product documentation for the publishing system for more details. For example, an SAP SuccessFactors system can be configured to publish events. See SAP Cloud Application Event Hub Service Guide: Integration Use Cases.
  The SAP system you plan to use as the subscriber is registered in the *Systems* page. If you want to use SAP Integration Suite integration flows to consume events from SAP Cloud Application Event Hub, then the tenant with the integration flow must be registered in the *Systems* page. See Integration Example using SAP Integration Suite.
- Your subscribing system is listed in the *Systems* page of your global account in SAP BTP.
  For example, you have a system of type *SAP Integration Suite* listed in the *Systems* page. It appears as auto-discovered.

Context

You can subscribe to events with extension applications. For example, you use SAP SuccessFactors to publish events and an Integration Suite tenant to consume them. The SAP SuccessFactors system is publishing these events and you add the event types that you want this system to receive using the *Simplified Business Eventing Template* of the integration dependencies. Integration dependencies are exposed as available event subscriptions in SAP Cloud Application Event Hub. The *Simplified Business Eventing Template* helps you write simplified integration dependencies specifying only event-specific information.

Procedure

1. In the cockpit, navigate to your global account, and then choose *System Landscape* > *Systems*.
2. On the *Systems* page, select a system of type *Integration Suite* to which you want to add an integration dependency and open its system details.
3. In the *Integration Dependencies* tab, choose *Add*.
4. Select *Simplified Business Eventing Template* to identify the system that is publishing events and add the event types that you want to receive. Integration dependencies are exposed as available event subscriptions in SAP Cloud Application Event Hub.
5. Define a name and a description for the new integration dependency.

> ### Note
>
> The version is assigned automatically when adding a new integration dependency. If you add a new integration dependency using the same name as an existing integration dependency, the version of the new integration dependency will be automatically increased.

6. In the *Publishing System Namespace* field, provide the system namespace of the system that is publishing events. You can find the system namespace in the *System Details* of the system.

   The system namespace has to follow these rules:

   - Use only lowercase ASCII letters (a-z) and digits (0-9)
   - Include at least one character
   - Use the dot (.) only to separate fragments
   - Include only two fragments separated by a dot
     Example: sap.foo

7. In the *Event Type* field, list the event types published by the system that you want to make available to your subscribing system.

   You can write or paste the event types separated by comma, semicolon or new row. Preferably, each event type should be on a new row. You have a counter giving the number of event types that are already filled in.

8. Choose *Next Step*.
9. Check if all the event types are configured properly and choose *Add*.

> ### Note
>
> Once you add an integration dependency to a system, you can no longer edit it and add or delete event types. You can only delete the integration dependency itself.

5.6.2 Extending SAP S/4HANA Cloud in the Cloud Foundry and Kyma Environment

Extend SAP S/4HANA Cloud with extension applications running on SAP BTP using automated integration configuration.

Overview

> ### Note
>
> The SAP S/4HANA Cloud Extensibility service is also available for global accounts marked with EU Access. See Regions.

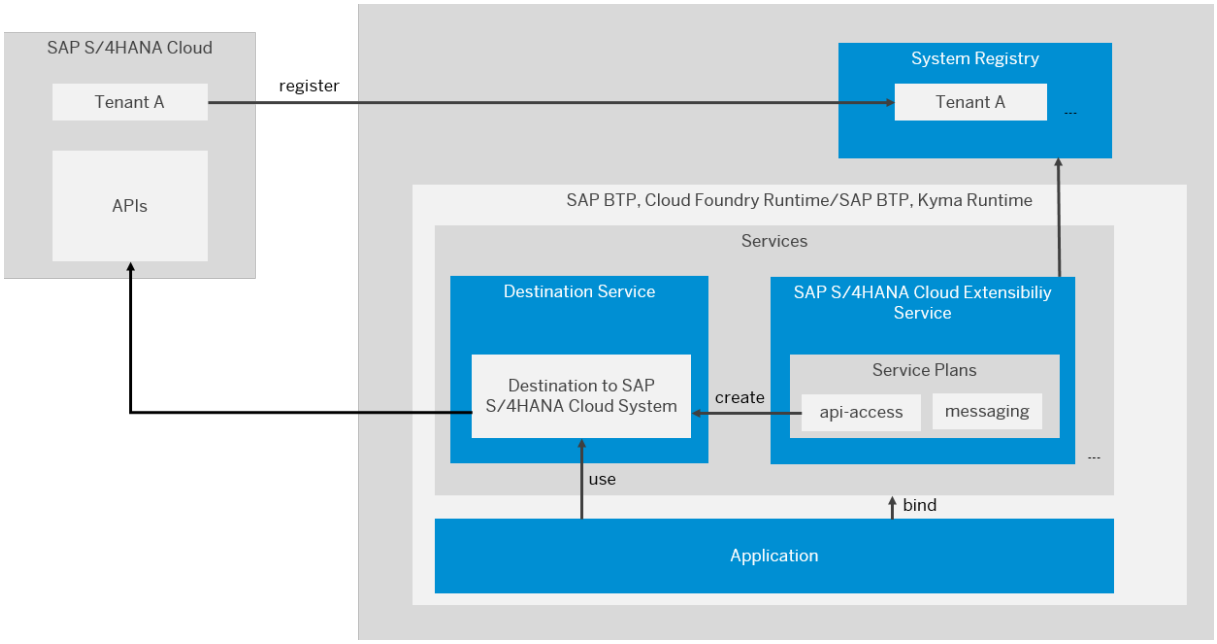
> ### Note
>
> This documentation refers to SAP S/4HANA Cloud Public Edition. See Introduction to the Universe of SAP S/4HANA Cloud Public Edition.

The extension capabilities of SAP BTP offer a standard way for extending SAP S/4HANA Cloud and developing event-driven extensions and applications.

You can extend SAP S/4HANA Cloud without disrupting its performance and core processes. When building extension applications, you can also benefit from the automation of the integration between the cloud platform and SAP S/4HANA Cloud.

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

The following graphic provides a high-level overview of the integration between the cloud platform and SAP S/4HANA Cloud:



Process Flow

To integrate the cloud platform and SAP S/4HANA Cloud so that you can build extension applications, you need to:

Process Step	Related Documentation
1. Connect the SAP S/4HANA Cloud system you want to extend with the corresponding global account in SAP BTP. During the pairing process you create an registration token which is then used by the SAP S/4HANA Cloud system tenant administrator to configure the integration on the SAP S/4HANA Cloud system side.	Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299] **Note** You cannot migrate the registered SAP S/4HANA Cloud systems between global accounts. If you want to start using another global account, you will have to register your SAP S/4HANA Cloud systems again.
2. Make the SAP S/4HANA Cloud system accessible in the SAP BTP subaccounts in which you want to build your extension applications. To do so, you configure the entitlements and assign the corresponding quota and service plans to the subaccounts where the extension applications will reside for the system you registered in the previous step.	Configure the Entitlements for the SAP S/4HANA Cloud Extensibility Service [page 2367]

Process Step	Related Documentation

3. Configure the communication flow.
 	
You have the following options:
 	
• Consume the SAP S/4HANA Cloud APIs (inbound connection) or consume APIs exposed by the extension application from SAPS/4 HANA Cloud (outbound connection)
To do so, you create a service instance of the *SAP S/4HANA Cloud Extensibility* service using the *api-access* service plan.
For communication arrangements with inbound connections, an HTTP destination on a subaccount level is automatically generated in this subaccount during the service instance creation. It contains all instance binding properties which are sufficient to establish connection to the SAP S/4HANA Cloud system. When creating the service instance, you configure the communication arrangement and the authentication type for the connection.
The following authentication scenarios for SAP S/4HANA Cloud are supported:
  • Basic Authentication (inbound and outbound connections)
  • OAuth 2.0 SAML Bearer Assertion (inbound connections)
  • OAuth 2.0 Client Credentials (outbound Connections)
  • No Authentication (outbound connections)
Both predefined and custom communication scenarios are supported.
• Enable the consumption of SAP S/4HANA Cloud events.
If you want to create event-based extensions for SAP S/4HANA Cloud using the SAP Event Mesh, you have to create a service instance of the *SAP S/4HANA Cloud Extensibility* service using the *messaging* service plan.
• A combination of both
	• Creating a Service Instance to Consume the SAP S/4HANA Cloud APIs [page 2368]
	• Enable the Consumption of SAP S/4HANA Cloud Events [page 2390]

5.6.2.1 Configure the Entitlements for the SAP S/4HANA Cloud Extensibility Service

Configure the required entitlements to make the APIs of the registered SAP S/4HANA Cloud system accessible in your subaccount in which your extension applications will reside.

Prerequisites

- You are an administrator of the global account in SAP BTP.
- The subaccount is in the SAP BTP, Cloud Foundry environment with enabled Cloud Foundry, or Kyma, or both capabilities.
- You have registered an SAP S/4HANA Cloud system. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Context

An entitlement is your right to provision and consume a resource. In other words, the entitlement is the respective service plan, *api-access* or *messaging*, that you're entitled to use.

Procedure

1. In the SAP BTP cockpit, navigate to your global account.
2. In the navigation area, choose *Entitlements* > *Entity Assignments*.
3. On the *Entity Assignments* screen, select your subaccount in the *Subaccounts/Directories* field.
4. Choose *Edit*, and then choose *Add Service Plans*.
5. Select the *SAP S/4HANA Cloud Extensibility* service.

   > **Note**
   >
   > To have the *SAP S/4HANA Cloud Extensibility* service in the list, you need to have registered at least one SAP S/4HANA Cloud system.

6. In the *Service Details* area, select the system you have registered. In the *Available Plans* area, select the required service plans, and then choose *Add Service Plan*.

   > **Note**
   >
   > For more information about the available service plans for SAP S/4HANA Cloud Extensibility service, see Supported Service Plans for SAP S/4HANA Cloud [page 2419].

7. Choose *Save*.

Related Information

Configure Entitlements and Quotas for Subaccounts [page 2529]

5.6.2.2 Create a Service Instance to Consume the SAP S/4HANA Cloud APIs

To enable the integration of your extension applications with the SAP S/4HANA Cloud system you have registered in the SAP BTP global account, and to configure the communication flow, you create a service instance of the SAP S/4HANA Cloud Extensibility service.

Prerequisites

- You have configured the required entitlements to make the APIs of the registered SAP S/4HANA Cloud system accessible in your subaccount in which your extension applications will reside. See Configure the Entitlements for the SAP S/4HANA Cloud Extensibility Service [page 2367].
- In case of principal propagation, you first need to configure single-sign on (SSO) with the Identity Authentication service and protect your application. See Integrating the Service with SAP S/4HANA Cloud, SAP Integrated Business Planning and SAP Analytics Cloud.

Context

In both Cloud Foundry and Kyma environments, you consume services by creating a service instance. Service instances are created using a specific service plan.

To allow applications running on SAP BTP to consume SAP S/4HANA Cloud APIs, you need to create a service instance of the SAP S/4HANA Cloud Extensibility service using the *api-access* service plan.

> **Note**
>
> These service plans have been deprecated:
>
> - *sap_com_0109*
> - *sap_com_0009*
> - *sap_com_0008*
>
> However, you can still enable these communication scenarios using the *api-access* service plan:
>
> - *Sales Order Integration (SAP_COM_0109)*: allows you to integrate your extension applications with sales order processing in SAP S/4HANA Cloud. For more information, see https://api.sap.com/api/API_SALES_ORDER_SRV/overview.
> - *Product Integration (SAP_COM_0009)*: enables you to replicate product master data from client system to SAP S/4HANA system. For more information, see https://api.sap.com/api/PRODUCTMDMBULKREPLICATEREQUEST/overview.

- *Business Partner, Customer and Supplier Integration (SAP_COM_0008)*: allows you to consume the Business Partner API which enables you to create, read, update, and delete master data related to Business Partners, Suppliers, and Customers in an SAP S/4HANA system. For more information, see https://api.sap.com/api/API_BUSINESS_PARTNER/overview.

For a sample JSON for these communication scenarios, see Communication Arrangement JSON/YAML File - Properties [page 2374].

The *api-access* service plan define the access to the corresponding SAP S/4HANA Cloud APIs. It supports both predefined and custom communication scenarios for consuming the SAP S/4HANA Cloud APIs and integrating your extension applications. See:

- Supported Service Plans for SAP S/4HANA Cloud [page 2419]
- Custom Communication Scenarios

You create the service instance in your subaccount with the respective environment enabled. When creating the service instance, you configure the connectivity by specifying the required configurations in a JSON format. The following authentication scenarios are supported for the communication flow between the extension application and SAP S/4HANA Cloud:

- Basic Authentication (inbound and outbound connections)
- OAuth 2.0 Client Credentials (outbound connections)
- OAuth 2.0 SAML Bearer Assertion (inbound connections)
  To communicate with SAP S/4HANA Cloud the extension application can use principal propagation which is done using OAuth 2.0 SAML Bearer Assertion flows. Principal propagation means you forward the identity of the logged-in cloud users when accessing or updating data in the SAP S/4HANA Cloud system. This is useful in scenarios where you need to have restricted data access based on the logged-in user from your extension. Or, you want to ensure only users with the right permissions are able to update the system via extensions deployed in SAP BTP.
  To use this authentication scenario, you first need to configure single-sign on (SSO) with the Identity Authentication service and protect your application. See Integrating the Service with SAP S/4HANA Cloud, SAP Integrated Business Planning and SAP Analytics Cloud.
- Client Certificate Authentication (inbound and outbound connections)
- OAuth2mTLS (outbound connections)

Depending on whether you are using Cloud Foundry or Kyma environment, you have to follow different steps to create an SAP S/4HANA Cloud Extensibility service instance:

- Create an SAP S/4HANA Cloud Extensibility Service Instance in the Cloud Foundry Environment [page 2370]
- Create an SAP S/4HANA Cloud Extensibility Service Instance in the Kyma Environment [page 2372]

Related Information

Communication Arrangement JSON/YAML File - Properties [page 2374]

5.6.2.2.1 Create an SAP S/4HANA Cloud Extensibility Service Instance in the Cloud Foundry Environment

Prerequisites

- Before creating an SAP S/4HANA Cloud Extensibility service instance in the Cloud Foundry environment, see Create a Service Instance to Consume the SAP S/4HANA Cloud APIs [page 2368].
- Have enabled Cloud Foundry environment for your subaccount. See Enable Environment or Create Environment Instance [page 2515].
- Have registered an SAP S/4HANA Cloud system. SeeRegister an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].
- Have configured the entitlements to the SAP S/4HANA Cloud Extensibility service. See Configure the Entitlements for the SAP S/4HANA Cloud Extensibility Service [page 2367].

Context

To consume the SAP S/4HANA Cloud APIs (inbound connection) or consume APIs exposed by the extension application from SAP S/4HANA Cloud (outbound connection), you create an SAP S/4HANA Cloud Extensibility service instance. When creating the service instance, you configure the communication arrangement and the authentication type for the connection.

For communication arrangements with inbound connections, a destination on a subaccount level with the same name as the service instance name is automatically created in this subaccount. It contains all instance binding properties which are sufficient to establish connection to the SAP S/4HANA Cloud system.

> **Note**
>
> Make sure that you don't already have a destination with the same name as the service instance. If you do, you will not be able to create the service instance.

> **Note**
>
> Multitenant applications, that this subaccount is subscribed to, can access the destination and connect to the SAP S/4HANA Cloud system without binding to the created service instance.
>
> See Developing Multitenant Business Applications in the Cloud Foundry Environment.

Procedure

1. In the SAP BTP cockpit, navigate to the subaccount in which you want to create a service instance.
2. In the navigation area, choose *Services* > *Service Marketplace*.

   All services available to you appear.
3. To enable the integration with an SAP S/4HANA Cloud system that you have registered in SAP BTP global account, choose *SAP S/4HANA Cloud Extensibility*.
4. In the *SAP S/4HANA Cloud Extensibility* page, choose *Create*.
5. In the *New Instance or Subscription* wizard:
   a. In the *Service* dropdown list, ensure you have selected the *SAP S/4HANA Cloud Extensibility* service.
   b. In the *Plan* dropdown list, select the *api-access* service plan.
   c. In the *Runtime Environment* dropdown list, select *Cloud Foundry*.
   d. In the *Space* dropdown list, select your space. If you haven't create a space yet, you can do it at this point.
   e. In the *System Name* dropdown list, select the SAP S/4HANA Cloud system you have registered.
   f. In the *Instance Name* field, enter a name for your instance. Choose *Next*.
   g. To define the communication arrangement and the authentication type for the API access, specify a JSON file or specify parameters in the JSON format. If you decide to define the communication arrangement and the authentication type later on, you have to delete this service instance and create it again. Choose *Next*.

      For more information about the structure of the JSON file, see Communication Arrangement JSON/YAML File - Properties [page 2374].
   h. Choose *Create*.

   After you have created the service instance:
   - The newly created instance appears in the list of instances in the *Instance and Subscriptions* page.
   - An HTTP destination on a subaccount level with the same name as the service instance name is automatically generated in this subaccount.

   Alternatively, you can use the Cloud Foundry Command Line Interface (cf CLI) to create the service instance using the technical name of the SAP S/4HANA Cloud Extensibility service which is **`s4-hana-cloud`**.

   For more information, see Create Service Instances Using the Cloud Foundry Command Line Interface.

   > **Note**
   >
   > You can use the cf CLI to troubleshoot if the creation of the service instance fails. To do that, use this command in the cf CLI:
   >
   > ```
   > cf service <service_instance_name>
   > ```

Next Steps

After you have created the service instance:

- The newly created instance appears in the list of instances in the *Instance and Subscriptions* page.
- An HTTP destination on a subaccount level with the same name as the service instance name is automatically generated in this subaccount.

Alternatively, you can use the Cloud Foundry Command Line Interface (cf CLI) to create the service instance using the technical name of the SAP S/4HANA Cloud Extensibility service which is **`s4-hana-cloud`**.

For more information, see Create Service Instances Using the Cloud Foundry Command Line Interface.

> ⓘ Note
>
> You can use the cf CLI to troubleshoot if the creation of the service instance fails. To do that, use this command in the cf CLI:
>
> ```
> cf service <service_instance_name>
> ```

After creating the *SAP S/4HANA Cloud Extensibility* service instance, you have the following options for configuring the connectivity for your extension application:

- Bind the instance to an application, and it will be assigned an access URL and credentials to the corresponding API. For more information about binding applications to service instances, see Binding Service Instances to Applications in the SAP BTP documentation.
- Consume the automatically generated destination.
  To consume the destination, you use the destination service. You can either consume the Destination service directly, or configure the application router to consume it.

  > ⓘ Note
  >
  > The name of the destination is the same as the name of the service instance you have created.

  - For more information about consuming the destination service using the application router, see Application Routes and Destinations.
  - For more information about consuming the destination service directly, see Consuming the Destination Service (Cloud Foundry Environment).

5.6.2.2.2 Create an SAP S/4HANA Cloud Extensibility Service Instance in the Kyma Environment

Prerequisites

- Before creating an SAP S/4HANA Cloud Extensibility service instance in the Kyma environment, see Create a Service Instance to Consume the SAP S/4HANA Cloud APIs [page 2368].
- Have the Kyma environment enabled for the subaccount you are using. See Creating Kyma Instances [page 3452].
- Configure the roles in the Kyma environment. See Assign Roles in the Kyma Environment [page 3666].

- The SAP BTP Operator module is enabled. See Adding and Deleting a Kyma Module [page 3511]
- Have the entitlements of the SAP S/4HANA Cloud Extensibility service configured. See Configure the Entitlements for the SAP S/4HANA Cloud Extensibility Service [page 2367].

Context

To consume the SAP S/4HANA Cloud APIs (inbound connection) or consume APIs exposed by the extension application from SAP S/4HANA Cloud (outbound connection), you create an SAP S/4HANA Cloud Extensibility service instance. When creating the service instance, you configure the communication arrangement and the authentication type for the connection.

For communication arrangements with inbound connections, a destination on a subaccount level with the same name as the service instance name is automatically created in this subaccount. It contains all instance binding properties which are sufficient to establish connection to the SAP S/4HANA Cloud system.

> **Note**
>
> Make sure that you don't already have a destination with the same name as the service instance. If you do, you will not be able to create the service instance.

Procedure

1. Navigate to the subaccount for which you want to create an SAP S/4HANA Cloud Extensibility service instance.
2. On the subaccount *Overview* page, in the *Kyma Environment* section, open the Kyma dashboard.
3. Choose *Namespaces* from the left-hand side navigation and open the namespace in which you want to create a service instance.
4. Choose *Service Management* > *Service Instances* from the left-hand side navigation.
5. In the *Service Instances* page, choose *Create Service Instance* in the upper right-hand corner. A new dialog opens.
6. Choose the *Simple* tab and fill in the following fields:
   - Give a meaningful name of the new SAP S/4HANA Cloud Extensibility service instance.
   - In the *Offering Name* field, enter **`s4-hana-cloud`**, which is the technical name of the SAP S/4HANA Cloud Extensibility service.
   - In the *Plan Name* field, enter **`api-access`**.
7. Choose the *YAML* tab.

   To define the communication arrangement and the authentication type for the API access, specify in *parameters:* in the *spec:* section, the parameters listed in this YAML file: Communication Arrangement JSON/YAML File - Properties [page 2374].

   For specific examples, see Communication Arrangement YAML File - Examples [page 2386].
8. Choose *Create*.

Next Steps

After creating the *SAP S/4HANA Cloud Extensibility* service instance, you must bind the instance to an application, and it will be assigned an access URL and credentials to the corresponding API. See Using SAP BTP Services in the Kyma Environment [page 2267].

5.6.2.2.3 Communication Arrangement JSON/YAML File - Properties

Learn how to construct a JSON or YAML file for the SAP S/4HANA Cloud APIs, including parameters such as system name, communication arrangement, authentication type, communication system, outbound services, and additional properties. It also provides guidelines and rules for each property.

Context

To construct the JSON file for the Cloud Foundry environment or the YAML file for the Kyma environment for the SAP S/4HANA Cloud APIs you want to use, you can use these parameters. To access the specific documentation of these APIs, see SAP S/4HANA Cloud APIs at SAP API Business Hub.

The information that you need to construct the JSON or the YAML file is available in the Display Communication Scenario application in the corresponding SAP S/4HANA Cloud system. It contains information such as scenario details and properties, and supported inbound and outbound authentication methods. See Display Communication Scenarios.

For specific sample JSON files, see Communication Arrangement JSON File - Examples [page 2382].

For specific sample YAML files, see Communication Arrangement YAML File - Examples [page 2386].

Properties

Parameter	Description
`systemName`	The name of the system you have registered in SAP BTP global account. **Note** The system must be in status *Registered*. **Rules/Guidelines** • Required: Yes

Parameter	Description
`communicationArrangement`	Represents a communication arrangement in SAP S/4HANA Cloud. **Rules/Guidelines** • Required: Yes
`communicationArrangementName`	A `communicationArrangement` property. Meaningful name of the communication arrangement that will be created for the SAP S/4HANA Cloud tenant. **Rules/Guidelines** • Allowed characters: [a-zA-Z0-9_-] • Max length: 80 • Required: Yes
`scenarioId`	A `communicationArrangement` property. The ID of the SAP S/4HANA Cloud communication scenario. **Rules/Guidelines** • Required: Yes • Allowed characters: [A-Z0-9_] • Pattern: **`SAP_COM_<number>`**

Parameter	Description
`inboundAuthentication`	A `communicationArrangement` property. The authentication type for the SAP S/4HANA Cloud API access. **Note** Currently, the following authentication methods are supported: - Basic Authentication - SAML Bearer Assertion Authentication **Note** The generated X.509 certificate that is used to configure the trust between the cloud platform and SAP S/4HANA Cloud system expires two years after you have created the instance. - Client Certificate Authentication **Rules/Guidelines** - Required: Yes, if there is no `outboundAuthentication` defined - Allowed values: `OAuth2SAMLBearerAssertion`, `BasicAuthentication`, and `ClientCertificateAuthentication`

Parameter	Description
`outboundAuthentication`	A `communicationArrangement` property. The type of the authentication used by SAP S/4HANA Cloud to call SAP BTP APIs. **Note** Currently, the following authentication methods are supported: • Basic Authentication • OAuth 2.0 Client Credentials • No Authentication • OAuth2mTLS • Client Certificate Authentication **Rules/Guidelines** • Required: Yes, if there is no `inboundAuthentication` defined • Allowed values: `BasicAuthentication`, `OAuth2ClientCredentials`, `NoAuthentication`, `OAuth2mTLS`, and `ClientCertificateAuthentication`
`communicationSystem`	A `communicationArrangement` property. This represents the *Communication System* view of the communication arrangement. **Rules/Guidelines** • Required: No
`communicationSystemHostname`	A `communicationSystem` property. The URL of the remote system hosting the APIs that will be consumed in case the scenario contains outbound communication. For SAP S/4HANA Cloud extensions on SAP BTP, this is the URL of the extension application running on SAP BTP. This is equivalent to *Technical Data* > *General* > *Host Name* in the *Communication System* view. **Rules/Guidelines** • Required: Yes, if `communicationArrangement` is specified.

Parameter	Description
`port`	A `communicationSystem` property. The port for outbound calls to the remote system hosting the APIs that will be consumed in case the scenario contains outbound communication. **Rules/Guidelines** • Required: No If not specified, the default *443* port is used for the communication. • Type: String • Allowed values: positive integer [*1-65535*]
`oAuthAuthEndpoint`	A `communicationSystem` property. The OAuth authorization endpoint of the remote OAuth service in case the communication scenario contains outbound communication. This is equivalent to *Technical Data* > *OAuth 2.0 Settings* > *Auth. Endpoint* in the *Communication System* view. **Rules/Guidelines** • Required: No
`oAuthTokenEndpoint`	A `communicationSystem` property. The OAuth token endpoint. This is equivalent to *Technical Data* > *OAuth 2.0 Settings* > *Token Endpoint* in the *Communication System* view. **Rules/Guidelines** • Required: No
`outboundCommunicationUser`	A `communicationSystem` property. The communication user used for outbound authentication. This is equivalent to an entry under the *Users for Outbound Communication* table in the *Communication System* view in the SAP S/4HANA Cloud system. **Note** Currently, only users with authentication method *User Name and Password* are supported. **Rules/Guidelines** • Required: No

Parameter	Description
`username`	An `outboundCommunicationUser` property. The username of the communication user. **Rules/Guidelines** • Required: Yes, if the `outboundCommunicationUser` property is specified.
`password`	An `outboundCommunicationUser` property. The password of the communication user. **Rules/Guidelines** • Required: Yes, if the `outboundCommunicationUser` property is specified.
`outboundServices`	A `communicationArrangement` property. A list of outbound service objects. This is equivalent to the *Outbound Services* section in the SAP S/4HANA Cloud *Communication Arrangement* view. **Rules/Guidelines** • Required: No
`outboundService`	An `outboundServices` object. A specific outbound service object. **Rules/Guidelines** • Required: No
`id`	An `outboundService` property. The id of the outbound service. It must match the *Outbound Service ID* displayed in the SAP S/4HANA Cloud system. **Rules/Guidelines** • Required: Yes, if the `outboundServices` property is specified. **Note** When you specify the `id` property of the outbound service, you can skip the `name` property.

Parameter	Description
`name`	An `outboundService` property. The name of the outbound service. It must be an exact match of the name displayed in the SAP S/4HANA Cloud system. **Rules/Guidelines** • Required: Yes, if the `outboundServices` property is specified, and the `id` property is not specified. > **Note** > If the `id` property is specified, the `name` property is not required.
`urlPath`	An `outboundService` property. This is equivalent to the *Path* field in the SAP S/4HANA Cloud system. It is used to configure the API path in the outbound service URL. **Rules/Guidelines** • Required: No
`isServiceActive`	An `outboundService` property. This is equivalent to the *Service Status* checkbox in the SAP S/4HANA Cloud system. In some communication scenarios, some outbound services must be disabled. You can achieve that by setting this parameter to *false*. **Rules/Guidelines** • Required: No • Default value: **`true`**
`attributes`	An `outboundService` parameter. A list of attribute objects. This is equivalent to the *Additional Properties* section of the outbound service. **Rules/Guidelines** • Required: No

Parameter	Description
`attribute`	An `attributes` object. A specific attribute object. Represents an additional property in the outbound service. **Rules/Guidelines** • Required: No
`name`	An `attribute` property. The name of the additional property of the outbound service. It is equivalent to the *Technical Property Name* column in the *Additional Properties* section in the Display Communication Scenarios app in the SAP S/4HANA Cloud system. See Display Communication Scenarios. **Rules/Guidelines** • Required: Yes, if an `attribute` object is specified.
`value`	An `attribute` property. Enter value for the additional property of the outbound service. **Rules/Guidelines** • Required: Yes, if an `attribute` object is specified.
`attributes`	A `communicationArrangement` parameter. A list of attribute objects. This is equivalent to the *Additional Properties* section of the communication arrangement in SAP S/4HANA Cloud. **Rules/Guidelines** • Required: No
`attribute`	An `attributes` object. A specific attribute object. Represents an additional property in the communication arrangement. **Rules/Guidelines** • Required: No

Parameter	Description
`name`	An `attribute` property. It is equivalent to the *Technical Property Name* section in the *Additional Properties* table in the Display Communication Scenarios application in the SAP S/4HANA Cloud system. See Display Communication Scenarios. **Rules/Guidelines** • Required: Yes, if an `attribute` object is specified for `communicationArrangement`.
`value`	An `attribute` property. Enter a value for the additional property of the communication arrangement. **Rules/Guidelines** • Required: Yes, if an `attribute` object is specified for `communicationArrangement`.

Related Information

Communication Arrangement JSON File - Examples [page 2382]
Communication Arrangement YAML File - Examples [page 2386]

5.6.2.2.4 Communication Arrangement JSON File - Examples

The examples in this section will help you to create the service JSON descriptor used for defining the communication arrangement and the authentication type for the SAP S/4HANA Cloud API access.

The information that you need to create the JSON file is available in the *Display Communication Scenario* app in the corresponding SAP S/4HANA Cloud system. It contains information such as scenario details and properties, and supported inbound and outbound authentication methods. See Display Communication Scenarios.

> **Note**
>
> The purpose of these examples is just to give you an idea how you construct your JSON file. For more details of the properties of these JSON files, see Communication Arrangement JSON/YAML File - Properties [page 2374].

Example of Enabling Communication Scenario of Type Basic Authentication

This is an example of a JSON file for a communication arrangement with an inbound connection with *Basic Authentication* and an outbound connection with *Basic Authentication*.

```
{
    "systemName": "DEMO",
    "communicationArrangement": {
        "communicationArrangementName": "INBOUND_COMMUNICATION_ARRANGEMENT",
        "scenarioId": "SAP_COM_0008",
        "inboundAuthentication": "BasicAuthentication",
        "outboundAuthentication": "BasicAuthentication",
        "outboundServices": [
            {
                "id": "DEBMAS_IDOC",
                "isServiceActive": false
            },
            {
                "id": "CREMAS_IDOC",
                "isServiceActive": false
            },
            {
                "id": "ADRMAS_IDOC",
                "isServiceActive": false
            },
            {
                "id": "ADR3MAS_IDOC",
                "isServiceActive": false
            },
            {
                "id": "ADR2MAS_IDOC",
                "isServiceActive": false
            },
            {
                "id": "CO_MDG_BP_RPLCTRQ_SPRX",
                "isServiceActive": false
            },
            {
                "id": "CO_MDG_BP_RELATIONSHIP_OUT_SPRX",
                "isServiceActive": false
            },
            {
                "id": "CO_MDG_BP_RPLCTCO_SPRX",
                "isServiceActive": false
            },
            {
                "id": "CO_MDG_BP_RELATIONSHIP_CNF_OUT_SPRX",
                "isServiceActive": false
            }
        ],
        "communicationSystem": {
            "communicationSystemHostname": "default.com",
            "outboundCommunicationUser": {
                "username": "DefaultUser",
                "password": "DefaultPassword"
            }
        }
    }
}
```

Example of Enabling Communication Scenario of Type OAuth2ClientCredentials

This is an example of a JSON file for a communication arrangement with an outbound connection with authentication type *OAuth2ClientCredentials*.

```
{
    "systemName":"DEMO",
    "communicationArrangement":{
        "outboundAuthentication":"OAuth2ClientCredentials",
        "communicationArrangementName":"0219_ARRANGEMENT",
        "scenarioId":"SAP_COM_0219",
        "communicationSystem":{
            "communicationSystemHostname":"default.com",
            "oAuthAuthEndpoint":"oauth.com/oauth/authorize",
            "oAuthTokenEndpoint":"oauth.com/oauth/token",
            "outboundCommunicationUser":{
                "username":"DefaultUser",
                "password":"DefaultPassword"
            }
        }
    }
}
```

Example for Enabling Communication Scenario of Type OAuth2SAMLBearerAssertion

This is an example of a JSON file for a communication arrangement with an inbound connection with *OAuth2SAMLBearerAssertion* and an outbound connection with *NoAuthentication*.

To communicate with SAP S/4HANA Cloud the extension application can use Principal Propagation which is done using OAuth 2.0 SAML Bearer Assertion flows. Principal Propagation means you forward the identity of the logged-in cloud users when accessing or updating data in the SAP S/4HANA Cloud system.

This is useful in scenarios where you need to have restricted data access based on the logged-in user from your extension. Or, you want to ensure only users with the right permissions are able to update the system via extensions deployed in SAP BTP, Cloud Foundry runtime.

```
{
    "systemName": "DEMO",
    "communicationArrangement": {
        "scenarioId": "SAP_COM_0213",
        "communicationArrangementName": "0213_ARRANGEMENT",
        "inboundAuthentication": "OAuth2SAMLBearerAssertion",
        "communicationSystem": {
            "communicationSystemHostname": "default.com"
        }
    }
}
```

Example for Enabling Communication Scenario of Type NoAuthentication

This is an example of a JSON file for a communication arrangement with an outbound connection with authentication type *NoAuthentication*.

```
{
    "systemName": "DEMO",
    "communicationArrangement": {
        "outboundAuthentication": "NoAuthentication",
        "communicationArrangementName": "0215_ARRANGEMENT",
        "scenarioId": "SAP_COM_0215",
        "outboundServices": [
            {
                "id": "SAP_COM_0215_0001_REST",
                "isServiceActive": true
            }
        ],
        "communicationSystem": {
            "communicationSystemHostname": "default.com"
        }
    }
}
```

Example for Enabling Communication Scenario of Type Client Certificate Authentication with Inbound Connection

This is an example of a JSON file for a communication arrangement with an inbound connection with authentication type *ClientCertificateAuthentication*.

```
{
    "systemName": "DEMO",
    "communicationArrangement": {
        "scenarioId": "SAP_COM_0724",
        "communicationArrangementName": "CommunicationArrangementName",
        "inboundAuthentication": "ClientCertificateAuthentication"
    }
}
```

Example for Enabling Communication Scenario of Type Client Certificate Authentication with Outbound Connection

This is an example of a JSON file for a communication arrangement with an outbound connection with authentication type *ClientCertificateAuthentication*.

```
{
    "systemName": "DEMO",
    "communicationArrangement": {
        "communicationArrangementName": "CommunicationArrangementName",
        "scenarioId": "SAP_COM_0047",
        "outboundAuthentication": "ClientCertificateAuthentication",
        "communicationSystem": {
            "communicationSystemHostname": "default.com"
```

```
        }
    }
}
```

Example for Enabling Communication Scenario of Type OAuth2mTLS

This is an example of a JSON file for a communication arrangement with an outbound connection with authentication type *OAuth2mTLS*.

```
{
    "systemName": "DEMO",
    "communicationArrangement": {
        "outboundAuthentication": "OAuth2mTLS",
        "communicationArrangementName": "CommunicationArrangementName",
        "scenarioId": "SAP_COM_0080",
        "communicationSystem": {
            "communicationSystemHostname": "default.com",
            "oAuthAuthEndpoint":"oauth.com/oauth/authorize",
            "oAuthTokenEndpoint":"oauth.com/oauth/token",
            "outboundCommunicationUser": {
                "username": "DefaultUser"
            }
        }
    }
}
```

5.6.2.2.5 Communication Arrangement YAML File - Examples

The examples in this section will help you to create the service YAML descriptor used for defining the communication arrangement and the authentication type for the SAP S/4HANA Cloud API access.

The information that you need to create the YAML file is available in the *Display Communication Scenario* app in the corresponding SAP S/4HANA Cloud system. It contains information such as scenario details and properties, and supported inbound and outbound authentication methods. See Display Communication Scenarios.

> **Note**
>
> The purpose of these examples is just to give you an idea how you construct your YAML file. For more details of the properties of these YAML files, see Communication Arrangement JSON/YAML File - Properties [page 2374].

Example of Enabling Communication Scenario of Type Basic Authentication

This is an example of a YAML file for a communication arrangement with an inbound connection with *Basic Authentication* and an outbound connection with *Basic Authentication*.

```
spec:
  externalName: ''
  serviceOfferingName: s4-hana-cloud
  servicePlanName: api-access
  parameters:
    systemName: DEMO
    communicationArrangement:
      communicationArrangementName: INBOUND_COMMUNICATION_ARRANGEMENT
      scenarioId: SAP_COM_0008
      inboundAuthentication: BasicAuthentication
      outboundAuthentication: BasicAuthentication
      outboundServices:
        - id: DEBMAS_IDOC
          isServiceActive: false
        - id: CREMAS_IDOC
          isServiceActive: false
        - id: ADRMAS_IDOC
          isServiceActive: false
        - id: ADR3MAS_IDOC
          isServiceActive: false
        - id: ADR2MAS_IDOC
          isServiceActive: false
        - id: CO_MDG_BP_RPLCTRQ_SPRX
          isServiceActive: false
        - id: CO_MDG_BP_RELATIONSHIP_OUT_SPRX
          isServiceActive: false
        - id: CO_MDG_BP_RPLCTCO_SPRX
          isServiceActive: false
        - id: CO_MDG_BP_RELATIONSHIP_CNF_OUT_SPRX
          isServiceActive: false
      communicationSystem:
        communicationSystemHostname: default.com
        outboundCommunicationUser:
          username: DefaultUser
          password: DefaultPassword
```

Example of Enabling Communication Scenario of Type OAuth2ClientCredentials

This is an example of a YAML file for a communication arrangement with an outbound connection with authentication type *OAuth2ClientCredentials*.

```
spec:
  externalName: ''
  serviceOfferingName: s4-hana-cloud
  servicePlanName: api-access
  parameters:
    systemName: DEMO
    communicationArrangement:
      outboundAuthentication: OAuth2ClientCredentials
      communicationArrangementName: 0219_ARRANGEMENT
      scenarioId: SAP_COM_0219
      communicationSystem:
        communicationSystemHostname: default.com
```

```
        oAuthAuthEndpoint: oauth.com/oauth/authorize
        oAuthTokenEndpoint: oauth.com/oauth/token
        outboundCommunicationUser:
          username: DefaultUser
          password: DefaultPassword
```

Example for Enabling Communication Scenario of Type OAuth2SAMLBearerAssertion

This is an example of a YAML file for a communication arrangement with an inbound connection with *OAuth2SAMLBearerAssertion* and an outbound connection with *NoAuthentication*.

To communicate with SAP S/4HANA Cloud the extension application can use Principal Propagation which is done using OAuth 2.0 SAML Bearer Assertion flows. Principal Propagation means you forward the identity of the logged-in cloud users when accessing or updating data in the SAP S/4HANA Cloud system.

This is useful in scenarios where you need to have restricted data access based on the logged-in user from your extension. Or, you want to ensure only users with the right permissions are able to update the system via extensions deployed in SAP BTP, Kyma runtime.

```
spec:
  externalName: ''
  serviceOfferingName: s4-hana-cloud
  servicePlanName: api-access
  parameters:
    systemName: DEMO
    communicationArrangement:
      communicationArrangementName: 0213_ARRANGEMENT
      scenarioId: SAP_COM_0213
      inboundAuthentication: OAuth2SAMLBearerAssertion
      communicationSystem:
        communicationSystemHostname: default.com
```

Example for Enabling Communication Scenario of Type NoAuthentication

This is an example of a YAML file for a communication arrangement with an outbound connection with authentication type *NoAuthentication*.

```
spec:
  externalName: ''
  serviceOfferingName: s4-hana-cloud
  servicePlanName: api-access
  parameters:
    systemName: DEMO
    communicationArrangement:
      communicationArrangementName: 0215_ARRANGEMENT
      scenarioId: SAP_COM_0215
      outboundAuthentication: NoAuthentication
      outboundServices:
        - id: SAP_COM_0215_0001_REST
          isServiceActive: true
      communicationSystem:
        communicationSystemHostname: default.com
```

Example for Enabling Communication Scenario of Type Client Certificate Authentication with Inbound Connection

This is an example of a YAML file for a communication arrangement with an inbound connection with authentication type *ClientCertificateAuthentication*.

```
spec:
  externalName: ''
  serviceOfferingName: s4-hana-cloud
  servicePlanName: api-access
  parameters:
    systemName: DEMO
    communicationArrangement:
      communicationArrangementName: CommunicationArrangementName
      scenarioId: SAP_COM_0724
      inboundAuthentication: ClientCertificateAuthentication
```

Example for Enabling Communication Scenario of Type Client Certificate Authentication with Outbound Connection

This is an example of a YAML file for a communication arrangement with an outbound connection with authentication type *ClientCertificateAuthentication*.

```
spec:
  externalName: ''
  serviceOfferingName: s4-hana-cloud
  servicePlanName: api-access
  parameters:
    systemName: DEMO
    communicationArrangement:
      communicationArrangementName: CommunicationArrangementName
      scenarioId: SAP_COM_0018
      outboundAuthentication: ClientCertificateAuthentication
      outboundServices:
        - id: CO_MSM_PROJECT_ERPCREATE_REQUE_SPRX
          isServiceActive: true
        - id: CO_MSM_PROJECT_ERPUPDATE_REQUE_SPRX
          isServiceActive: false
        - id: CO_MSM_PROJECT_ERPBY_IDQUERY_R_SPRX
          isServiceActive: false
      communicationSystem:
        communicationSystemHostname: default.com
```

Example for Enabling Communication Scenario of Type OAuth2mTLS

This is an example of a YAML file for a communication arrangement with an outbound connection with authentication type *OAuth2mTLS*.

```
spec:
  externalName: ''
  serviceOfferingName: s4-hana-cloud
  servicePlanName: api-access
```

```
parameters:
  systemName: DEMO
  communicationArrangement:
    communicationArrangementName: CommunicationArrangementName
    scenarioId: SAP_COM_0080
    outboundAuthentication: OAuth2mTLS
    outboundServices:
      - id: SAP_COM_0080_0001_REST
        isServiceActive: true
    communicationSystem:
      communicationSystemHostname: default.com
      oAuthAuthEndpoint: oauth.com/oauth/authorize
      oAuthTokenEndpoint: oauth.com/oauth/token
      outboundCommunicationUser:
          username: DefaultUser
```

5.6.2.3 Enable the Consumption of SAP S/4HANA Cloud Events

To create event-based extensions for SAP S/4HANA Cloud you need to set up the messaging between the SAP S/4HANA Cloud system and the SAP Event Mesh.

Prerequisites

- You have registered an SAP S/4HANA Cloud tenant in the global account in SAP BTP. See Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].

Context

To consume SAP S/4HANA Cloud events, you need to configure the connectivity between SAP Event Mesh and the SAP S/4HANA Cloud tenant. To do so, you need to create and configure service instances for both SAP S/4HANA Cloud Extensibility service and SAP Event Mesh.

When you create a service instance in a subaccount, an event bus for this subaccount is created. All service instances in a subaccount point to the same event bus and are nothing else but clients providing access credentials for the event bus. You configure the clients using JSON descriptors. Each client has its own name which can be configured with the `emname` parameter in the JSON descriptor file. Thus, the client is identified with a human readable name instead of a GUID. The name of each client must be unique. As all of the clients point to the same event bus, they must have different namespaces in which to publish events. Each namespace must be unique per subaccount. Each client should publish only in its own unique namespace.

Communication flow

The SAP S/4HANA Cloud Extensibility service instance is the client which creates the SAP S/4HANA Cloud integration and provides the credentials for the SAP S/4HANA Cloud system so that the SAP S/4HANA Cloud system can send events to that event bus. The purpose of this client is to send messages to a topic. It sends messages to the namespace which was configured while creating the service instance of S/4HANA Cloud

Extensibility service. See Define SAP S/4HANA Cloud Extensibility Service Descriptor JSON/YAML File [page 2398]

The SAP Event Mesh service instance is the client which receives the events. This service instance must have a different name and a different namespace than the ones configured for the SAP S/4HANA Cloud extensibility service instance. Since the purpose of this client is to receive messages from a different client the rules have to take care of that. So, for the SAP Event Mesh service instance, you need to configure the subscribe filter to the SAP S/4HANA Cloud Extensibility service instance. See Define SAP Event Mesh Service Descriptor JSON/YAML File [page 2411].

After both instances are created and configured, you can create the topic-to-queue subscription. To do so, you first create a queue in the client that refers to the SAP Event Mesh service instance and then subscribe the topic of the namespace of the SAP S/4HANA Cloud Extensibility service to that queue. See Manage Queues in the SAP Event Mesh documentation

The service instance of SAP Event Mesh is the client which consumes the message and should be bound to the extension application.

Procedure

To configure the connectivity between SAP Event Mesh and the SAP S/4HANA Cloud tenant so that SAP S/4HANA Cloud events can be produced and then consumed by applications running on SAP BTP, you need to perform the following tasks:

1. Add the required quotas to your subaccount as follows:
    1. Assign the *messaging* SAP S/4HANA Cloud Extensibility service plan to the subaccount in SAP BTP you want to pair with the SAP S/4HANA Cloud tenant. See Configure the Entitlements for the SAP S/4HANA Cloud Extensibility Service [page 2367].

        > **Note**
        >
        > You can use the service plan on regions where the SAP Event Mesh service is available.

        > **Note**
        >
        > The *messaging* service plan enables the *Enterprise Eventing Integration (SAP_COM_0092)* communication scenario which allows you to consume SAP S/4HANA Cloud events and create event-based extensions.
        >
        > This service plan has been renamed from *sap_com_0092*.

    2. Assign the required number of SAP Event Mesh instances to the subaccount in SAP BTP that you want to pair with the SAP S/4HANA Cloud tenant. See Configure the Entitlements for SAP Event Mesh [page 2392].
2. Configure the connectivity between SAP Event Mesh and the SAP S/4HANA Cloud tenant, and then configure event topics for the channel inside SAP S/4HANA Cloud tenant. See Set Up the Connectivity Between Event Mesh and the SAP S/4HANA Cloud Tenant [page 2393].
3. Enable the SAP Event Mesh for your subaccount in SAP BTP, create a queue that is specific to your application, and then subscribe this queue to the channel topic that the SAP S/4HANA Cloud tenant uses to produce events. See Enable the SAP Event Mesh Service for Your Subaccount in SAP BTP [page 2407].

Related Information

What Is SAP Event Mesh?

5.6.2.3.1 Configure the Entitlements for SAP Event Mesh

To be able to consume SAP S/4HANA Cloud events, you need to configure the entitlements for SAP Event Mesh for the subaccount in SAP BTP.

Context

The *messaging* service plan connects SAP S/4HANA Cloud tenant to SAP Event Mesh for the subaccount where the SAP Event Mesh service entitlement is configured. This SAP Event Mesh service instance allows you to consume events from SAP S/4HANA Cloud.

You need to configure the entitlements for the SAP Event Mesh service to create an SAP Event Mesh service instance. Then, you bind this service instance to an application to consume events from this application.

Procedure

1. In SAP BTP cockpit, navigate to the global account that contains the subaccount in which you want to make your SAP system accessible.
2. In the navigation area, choose *Entitlements* > *Entity Assignments*.
3. On the *Entity Assignments* screen, select your subaccount in the *Subaccounts/Directories* field.
4. Choose *Edit*, and then choose *Add Service Plans*.
5. Select the *SAP Event Mesh* service.
6. In the *Service Details* area, select the system you have registered. In the *Available Plans* area, select the *default* service plan, and then choose *Add Service Plan*.
7. Choose *Save*.

Related Information

Managing Entitlements and Quotas Using the Cockpit [page 2521]

5.6.2.3.2 Set Up the Connectivity Between Event Mesh and the SAP S/4HANA Cloud Tenant

Overview

To configure the connectivity between the SAP S/4HANA Cloud tenant and SAP Event Mesh and to enable the exchange of credentials between the two systems, you first need to create an *SAP S/4HANA Cloud Extensibility* service instance with service plan `messaging`. For more information about the `messaging` service plan, see Supported Service Plans for SAP S/4HANA Cloud [page 2419].

When creating this service instance, you create the required configurations in both the SAP S/4HANA Cloud tenant and the Event Mesh system associated with the subaccount in SAP BTP, so that events can flow. Depending on the SAP BTP environment, to create an `SAP S/4HANA Cloud Extensibility` service instance, you have to choose one of the following options:

- Create an SAP S/4HANA Extensibility Service Instance in the Cloud Foundry Environment [page 2394]
- Create an SAP S/4HANA Cloud Extensibility Service Instance in the Kyma Environment [page 2396]

After you have created the `SAP S/4HANA Cloud Extensibility` service instance, either in the Cloud Foundry or Kyma environment, you need to configure event topics for the channel inside the SAP S/4HANA Cloud tenant, and then you need to create an SAP Event Mesh service instance, respectively in the Cloud Foundry or Kyma environment, for the application to consume SAP S/4HANA Cloud events. You do this in a dedicated JSON file that you add either when creating the service instance, or after that. To construct this JSON file, see Define SAP S/4HANA Cloud Extensibility Service Descriptor JSON/YAML File [page 2398].

Process Flow

Process Step	Related Documentation
1. Create an `SAP S/4HANA Cloud Extensibility` service instance with service plan `messaging`.	• Create an SAP S/4HANA Extensibility Service Instance in the Cloud Foundry Environment [page 2394] • Create an SAP S/4HANA Cloud Extensibility Service Instance in the Kyma Environment [page 2396]
2. Define a dedicated JSON file that you add either when creating the `SAP S/4HANA Cloud Extensibility` service instance, or after that.	Define SAP S/4HANA Cloud Extensibility Service Descriptor JSON/YAML File [page 2398]
3. Configure event topics for the channel inside SAP S/4HANA Cloud tenant.	Configure Event Topics in SAP S/4HANA Cloud [page 2406]

5.6.2.3.2.1 Create an SAP S/4HANA Extensibility Service Instance in the Cloud Foundry Environment

Use this procedure to configure the communication between SAP S/4HANA Cloud and SAP Event Mesh.

Prerequisites

- Before creating an SAP S/4HANA Cloud Extensibility service instance in the Cloud Foundry environment, see Create a Service Instance to Consume the SAP S/4HANA Cloud APIs [page 2368].
- Have enabled Cloud Foundry environment for your subaccount. See Enable Environment or Create Environment Instance [page 2515].
- Have registered an SAP S/4HANA Cloud system. SeeRegister an SAP S/4HANA Cloud System in a Global Account in SAP BTP [page 2299].
- Have configured the entitlements to the SAP S/4HANA Cloud Extensibility service. See Configure the Entitlements for the SAP S/4HANA Cloud Extensibility Service [page 2367].

Context

To configure the connectivity between the SAP S/4HANA Cloud tenant and Event Mesh and to enable the exchange of credentials between the two systems, you first need to create an *SAP S/4HANA Cloud Extensibility* service instance with service plan `messaging`. For more information about the `messaging` service plan, see Supported Service Plans for SAP S/4HANA Cloud [page 2419].

When creating this service instance, you create the required configurations in both the SAP S/4HANA Cloud tenant and the Event Mesh system associated with the subaccount in SAP BTP, so that events can flow.

Procedure

1. In the SAP BTP cockpit, navigate to the subaccount in which you want to create a service instance.
2. In the navigation area, choose *Services* > *Service Marketplace*.

   All services available to you appear.
3. To enable the integration with an SAP S/4HANA Cloud system that you have registered in the global account in SAP BTP, choose *SAP S/4HANA Cloud Extensibility*.
4. In the *SAP S/4HANA Cloud Extensibility* page, choose *Create*.
5. In the *New Instance or Subscription* wizard:
   a. In the *Service* dropdown list, ensure you have selected the *SAP S/4HANA Cloud Extensibility* service.
   b. In the *Plan* dropdown list, select the *messaging* service plan.
   c. In the *Runtime Environment* dropdown list, select *Cloud Foundry*.
   d. In the *Space* dropdown list, select your space. If you haven't create a space yet, you can do it at this point.

e. In the *System Name* dropdown list, select the SAP S/4HANA Cloud system you have registered.
f. In the *Instance Name* field, enter a name for your instance. Choose *Next*.
g. Specify a JSON file or specify parameters in the JSON format to define the communication arrangement for the communication scenario *Enterprise Eventing Integration (SAP_COM_0092)* in the SAP S/4HANA Cloud tenant and to configure the parameters for the Enterprise Messaging service. If you decide to define the communication arrangement later on, you have to delete this service instance and create it again. Choose *Next*.

   For more information about the structure of the JSON file, see Define SAP S/4HANA Cloud Extensibility Service Descriptor JSON/YAML File [page 2398].
h. Choose *Create*.

The newly created instance appears in the list of instances in the *Instance and Subscriptions* page.

Alternatively, you can create the instance using cf CLI. To do so, execute the following command

```
cf create-service s4-hana-cloud messaging emsconnect -c '{"systemName":
"<system_name>","communicationArrangement": {"communicationArrangementName":
"<communication_arrangement_name>","attributes": [{"name": "Channel","value":
"<channel_name>"},{"name": "Description","value": "<short_description>"},
{"name": "Topic Space","value": "<topic_to_be_used_by_events>"},
{"name": "QoS","value": "<quality_of_service>"},{"name":
"Reconnect Attempts","value": "<number_of_reconnect_attempts>"},
{"name": "Reconnect wait time(sec)","value":
"<time_before_trying_to_reconnect>"}]},"ems": {"parameters":
{"emname": "enterprise_messaging_client_name","namespace": "enterprise-
messaging_client_namespace":{"options": {<required_options>},"rules":
{<required_rules>}}'
```

For more information, see Create Service Instances Using the Cloud Foundry Command Line Interface.

> **Note**
>
> You can use the cf CLI to troubleshoot if the creation of the service instance fails. To do that, use this command in the cf CLI:
>
> ```
> cf service <service_instance_name>
> ```

Next Steps

Configure Event Topics in SAP S/4HANA Cloud [page 2406]

5.6.2.3.2.2 Create an SAP S/4HANA Cloud Extensibility Service Instance in the Kyma Environment

Use this procedure to configure the communication between SAP S/4HANA Cloud and SAP Event Mesh in the Kyma environment.

Prerequisites

In the SAP BTP cockpit, you have assigned the `messaging` SAP S/4HANA Cloud service plan to the subaccount you want to pair with the SAP S/4HANA Cloud tenant. See Configure the Entitlements for the SAP BTP Subaccount [page 2367].

Context

To configure the connectivity between an SAP S/4HANA Cloud tenant and Event Mesh and to enable the exchange of credentials between the two systems, you first need to create an SAP S/4HANA Cloud Extensibility service instance with `messaging` service plan in the Kyma dashboard. For more information about the `messaging` service plan, see Supported Service Plans for SAP S/4HANA Cloud [page 2419].

When creating this service instance, you create the required configurations in both the SAP S/4HANA Cloud tenant and the Event Mesh system associated with the subaccount in SAP BTP, so that the events can flow from one system to the other.

Procedure

1. In the SAP BTP cockpit, navigate to the subaccount for which you want to create an SAP S/4HANA Cloud Extensibility service instance.
2. On the subaccount *Overview* page in the *Kyma Environment* section, open the Kyma dashboard.
3. Navigate to the *default* namespace from the drop-down list in the top navigation.
4. Choose *Service Management* > *BTP Service Instances* from the left-hand side navigation.
5. In the *Service Instances* page, choose *Create Service Instance* in the upper right-hand corner. A new dialog opens.
6. Choose the *Simple* tab and fill in the following fields:
   - Give a meaningful name of the new SAP S/4HANA Cloud Extensibility service instance.
   - In the *Offering Name* field, enter **`s4-hana-cloud`**, which is the technical name of the SAP S/4HANA Cloud Extensibility service.
   - In the *Plan Name* field, enter **`messaging`**.
7. Choose the *YAML* tab.

   In *parameters:* in the *spec:* section, specify the parameters to define the communication arrangement for the communication scenario *Enterprise Eventing Integration (SAP_COM_0092)* in the SAP S/4HANA

Cloud tenant and to configure the parameters for the Event Mesh service. Make sure you include at least these two required parameters: `systemName` and `emclientId`. The parameters are listed in this YAML file: Define SAP S/4HANA Cloud Extensibility Service Descriptor JSON/YAML File [page 2398].

8. Choose *Create*.

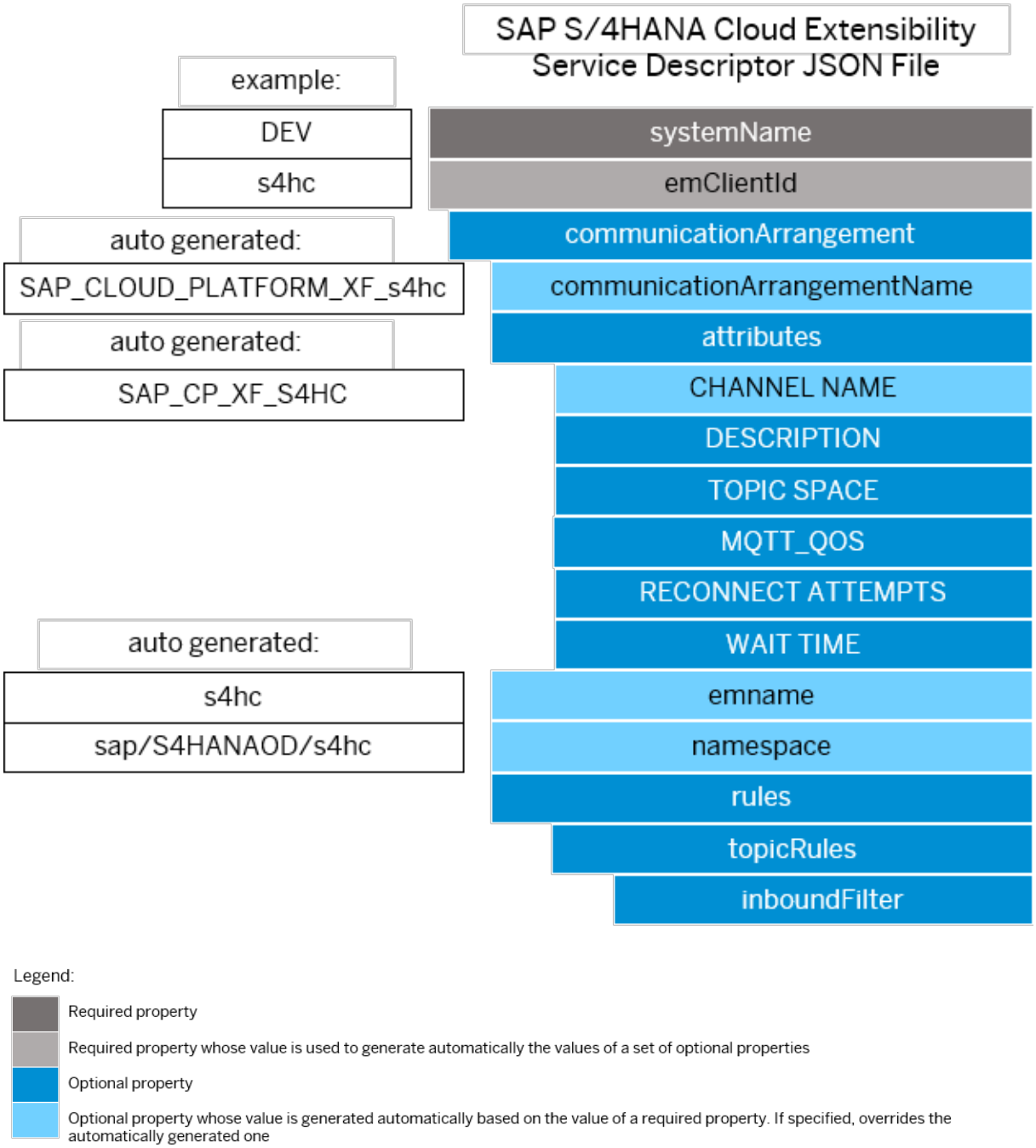
Next Steps

After creating the SAP S/4HANA Cloud Extensibility service instance with the `messaging` plan, a respective messaging client is created for you in the `sap/S4HANAOD/{emClientId}` namespace of the Event Mesh service. As the next step, you must relate the two clients (SAP S/4HANA Cloud Extensibility and Event Mesh) in the Kyma dashboard. To do this, create an instance of the Event Mesh service with the details of the automatically created Event Mesh namespace. Follow the steps in Create an SAP Event Mesh Service Instance in the Kyma Environment [page 2409].

5.6.2.3.2.3 Define SAP S/4HANA Cloud Extensibility Service Descriptor JSON/YAML File

The SAP S/4HANA Cloud Extensibility service descriptor defines details of a message client and needs to be provided when provisioning new SAP S/4HANA Cloud Extensibility service instances with service plan `messaging`.

Procedure



Define the SAP S/4HANA Cloud Extensibility service descriptor in a JSON structure for the Cloud Foundry environment and in a YAML structure for the Kyma environment. It has to contain two sections:

- For the parameters needed to activate the communication arrangement for the communication scenario *Enterprise Eventing Integration (SAP_COM_0092)* in the SAP S/4 HANA Cloud tenant.
  There are only two required parameters, *emClientId* and *systemName*. The rest of the parameters are automatically generated. However, you can still provide the optional parameters in the descriptor to override the automatically generated values.
- For the parameters for configuring the SAP Event Mesh service.
  If this section is not explicitly added to the JSON/YAML file, the values of its parameters are automatically generated based on the values of the *emClientId* and *systemName* parameters.

Parameters required to activate the communication arrangement in SAP S4/HANA Cloud tenant

Parameter	Description
`systemName`	The name of the system you have registered in the global account in SAP BTP. **Rules/Guidelines** • Required: Yes • The name must be the same as the one you have used when registering the SAP S/4HANA Cloud system during the pairing process.
`emClientId`	Using default patterns, it generates **communication arrangement name**, **channel name**, **emname**, and **namespace**, when these parameters are not explicitly provided. The default values are described in the table bellow containing the parameters required to configure SAP Event Mesh service. **Note** The values of the emname and namespace parameters must be the same as the values of the emname and namespace parametes in the SAP Event Mesh service descriptor. See Define SAP Event Mesh Service Descriptor JSON/YAML File [page 2411]. **Rules/Guidelines** • Required: Yes • Allowed characters: `[a-zA-Z0-9]` • Maximum length: 4
`communicationArrangement`	Defines the communication arrangement for the SAP S4/HANA Cloud tenant.

Parameter	Description
`communicationArrangementName`	A `communicationArrangement` property. The name of the communication arrangement for the SAP S/4HANA Cloud tenant. **Rules/Guidelines** • Required: No • Allowed characters: **`[a-zA-Z0-9_-]`** • Maximum length: 80 • Default value: **`"SAP_CLOUD_PLATFORM_XF_<emClientId>"`**
`attributes`	A `communicationArrangement` property. Defines the configuration properties for the communication arrangement. The name of the property is equivalent to the *Technical Property Name* column in the properties table in the Display Communication Scenarios app in the SAP S/4HANA Cloud system. See Display Communication Scenarios.
`CHANNEL NAME`	An `attributes` property. The name of the communication channel. **Rules/Guidelines** • Required: No • Allowed characters: **`[A-Z0-9_-]`** • Default value: **`"SAP_CP_XF_<emclientId>"`** > **Note** > Have in mind that the **`<emclientId>`** will be automatically capitalized. • Must be unique within the SAP S/4 HANA Cloud tenant.

Parameter	Description
`DESCRIPTION`	An `attributes` property. Short description. **Rules/Guidelines** • Required: No • Maximum length: 60 • Allowed characters: **`[a-zA-Z0-9_-]`** • Default value: **`"Integration with Enterprise Messaging for EM Client: <emclientId>"`**
`TOPIC SPACE`	An `attributes` property. The identifier for the events that originate from the same source. This is the topic that the events should use. **Rules/Guidelines** • Required: No • Maximum length: 24 • Allowed characters: [a-zA-Z0-9//] • Contains exactly three segments, for example **`a/b/c`**. • Default value: **`"sap/S4HANAOD/<emclientId>"`** • The `subscribeFilter` in the SAP Event Mesh configuration must be use the topic space as a prefix.
`MQTT_QOS`	An `attributes` property. Defines the quality of service. **Rules/Guidelines** • Required: No • Allowed values: **`0`**, **`1`** QoS=**`0`**, at most once delivery. The message is delivered according to the capabilities of the underlying network. No response is sent by the receiver and no retry is performed by the sender. The message arrives at SAP Event Mesh either once or not at all. QoS=**`1`**, at least once delivery. This quality of service ensures that the message arrives at SAP Event Mesh at least once. • Default value: **`1`**

Parameter	Description
`RECONNECT ATTEMPTS`	An `attributes` property. The number of attempts the Enterprise Event Enablement framework tries to reestablish the connection if the connection is lost. **Rules/Guidelines** • Required: No • Default value: **`0`** • If no value is entered or the entered value is 0, the framework tries to connect until connection is established. If the connection is not established after reaching reconnect attempts, the communication arrangement and the underlying channel is deactivated.
`WAIT TIME`	An `attributes` property. Specifies the time (in seconds) for which the Enterprise Event Enablement framework waits before trying to reconnect. If the attempts fail, framework increases the wait time until the reconnect wait time (1800 seconds) is reached. **Rules/Guidelines** • Required: No • Default value: **`10`**

Parameters required to configure SAP Event Mesh service

Parameter	Description
`emname`	Specifies the name of the SAP Event Mesh client. It is used by SAP Event Mesh to identify clients. **Rules/Guidelines:** • Required: No • Allowed characters: **`[a-zA-Z0-9_-]`** • Maximum length: 100 • Default value: **`<emClientId>`** • It is unique within a subaccount.

Parameter	Description
`namespace`	Namespace for the message client. **Rules/Guidelines:** The namespace in the SAP Event Mesh configuration must be the same as the first three segments of the topic space in the configuration of the communication arrangement in SAP S4/HANA Cloud tenant. **Rules/Guidelines:** - Required: No - Allowed characters: **`[a-zA-Z0-9//]`** - Maximum length: 24 - It is unique within a subaccount - Contains exactly three segments, for example **`a/b/c`**. - Default value: **`"sap/S4HANAOD/<emclientId>"`**.
`rules`	Defines the access privileges of the message client. **Guidelines/Rules:** - Required: No - In order to allow access to a queue or a topic the namespace of the corresponding owner client has to be added. The placeholder `${namespace}` can be used instead of the defined namespace.
`topicRules`	A `rules` attribute. **Guidelines/Rules:** - Type: object
`inboundFilter`	A `topicRules` attribute. Defines if a client (publisher/producer) is allowed to send messages to the defined topics. **Guidelines/Rules:** - Required: No - Allowed characters: **`[a-zA-Z#?/]`** - Type: array - Default value: **`${namespace}/#`** - Example values: **`${namespace}/foo/bar`**, **`${namespace}/#`**.

Parameter	Description
`resources`	A collection of messaging resources such as queues, connections, and so on, required for a message client. **Rules/Guidelines** • Required: No
`units`	A `resources` attribute. Specify this value in the service descriptor to allocate messaging resources based a specific business scenario. See Syntax for Service Descriptor. **Guidelines/Rules:** • Required: No • Allowed values: **`10-50`** • Type: attribute • Default value: **`10`**

JSON File Example

In this example, only the required properties are provided.

```
{
    "systemName": "DEV",
    "emClientId": "s4hc"
}
```

This descriptor is equivalent to:

> **Note**
>
> All properties can be overridden explicitly.

```
{
  "systemName": "DEV",
  "emClientId": "s4hc",
  "communicationArrangement": {
    "communicationArrangementName": "SAP_CLOUD_PLATFORM_XF_s4hc",
    "attributes": [
        {
           "name": "CHANNEL NAME",
           "value": "SAP_CP_XF_S4HC"
        },
        {
           "name": "DESCRIPTION",
           "value": "Integration with Enterprise Messaging for EM Client: s4hc"
        },
        {
           "name": "TOPIC SPACE",
           "value": "sap/S4HANAOD/s4hc"
        },
```

```
        {
           "name": "MQTT_QOS",
           "value": "1"
        },
        {
           "name": "RECONNECT ATTEMPTS",
           "value": "0"
        },
        {
           "name": "WAIT TIME",
           "value": "10"
        }
     ]
  },
  "ems": {
    "parameters": {
      "emname": "s4hc",
      "namespace": "sap/S4HANAOD/s4hc",
      "rules": {
        "topicRules": {
          "inboundFilter": [
            "${namespace}/#"
          ]
        }
      },
      "resources": {
        "units": "10"
      }
    }
  }
}
```

YAML File Example

In this example, only the required properties are provided.

```
parameters:
    systemName: DEV
    emClientId: s4hc
```

This descriptor is equivalent to:

> ### Note
>
> All properties can be overridden explicitly.

```
spec:
  externalName: ''
  serviceOfferingName: s4-hana-cloud
  servicePlanName: messaging
  parameters:
    systemName: DEV
    emClientId: s4hc
    communicationArrangement:
      communicationArrangementName: SAP_CLOUD_PLATFORM_XF_s4hc
      attributes:
      - name: CHANNEL NAME
        value: SAP_CP_XF_S4HC
      - name: DESCRIPTION
```

```
        value: 'Integration with Enterprise Messaging for EM Client: s4hc'
      - name: TOPIC SPACE
        value: sap/S4HANAOD/s4hc
      - name: MQTT_QOS
        value: '1'
      - name: RECONNECT ATTEMPTS
        value: '0'
      - name: WAIT TIME
        value: '10'
    ems:
      parameters:
        emname: s4hc
        namespace: sap/S4HANAOD/s4hc
        rules:
          topicRules:
            inboundFilter:
            - "${namespace}/#"
        resources:
          units: '10'
```

5.6.2.3.2.4 Configure Event Topics in SAP S/4HANA Cloud

Configure event topics for the channel inside the SAP S/4HANA Cloud tenant.

Prerequisites

You have created the `SAP S/4HANA Cloud Extensibility` service instance in the Cloud Foundry or Kyma environment. See:

- Create an SAP S/4HANA Extensibility Service Instance in the Cloud Foundry Environment [page 2394]
- Create an SAP S/4HANA Cloud Extensibility Service Instance in the Kyma Environment [page 2396]

Context

After you have created the `SAP S/4HANA Cloud Extensibility` service instance, you need to configure event topics for the channel inside the SAP S/4HANA Cloud tenant, and then you need to create an SAP Event Mesh service instance for the application to consume SAP S/4HANA Cloud events.

Procedure

Configure event topics for the channel inside SAP S/4HANA Cloud tenant. Use the channel you have specified in SAP S/4HANA Cloud Extensibility service descriptor JSON file when configuring the parameters for the communication arrangement in SAP S4/HANA Cloud tenant.

As an example, you can use the *sap/s4/beh/businesspartner/v1/BusinessPartner/Changed/v1* outbound topic.

Next Steps

Create an SAP Event Mesh service instance in the Cloud Foundry or the Kyma environment. See:

- Create an SAP Event Mesh Service Instance in the Cloud Foundry Environment [page 2408]
- Create an SAP Event Mesh Service Instance in the Kyma Environment [page 2409]

5.6.2.3.3 Enable the SAP Event Mesh Service for Your Subaccount in SAP BTP

Prerequisites

- You have created the `SAP S/4HANA Cloud Extensibility` service instance in the Cloud Foundry or Kyma environment. See:
  - Create an SAP S/4HANA Extensibility Service Instance in the Cloud Foundry Environment [page 2394]
  - Create an SAP S/4HANA Cloud Extensibility Service Instance in the Kyma Environment [page 2396]
- You have configured event topics for the channel inside SAP S/4HANA Cloud tenant. See Configure Event Topics in SAP S/4HANA Cloud [page 2406].

Overview

To enable the SAP Event Mesh for your subaccount in SAP BTP, you have to create an instance of the SAP Event Mesh service with service plan `default`. This instance will specify the details of the namespace that you created in SAP Event Mesh for the SAP S/4HANA Cloud Extensibility service instance with the `messaging` plan. This way, you are setting up a contract between SAP Event Mesh and the Cloud Foundry, or the Kyma environment, and connect them to enable seamless event flow from the given SAP S/4HANA Cloud tenant.

Depending on the SAP BTP environment, to create an SAP Event Mesh service instance, you have to choose one of these:

- Create an SAP Event Mesh Service Instance in the Cloud Foundry Environment [page 2408]
- Create an SAP Event Mesh Service Instance in the Kyma Environment [page 2409]

Define the details of your message client in SAP Event Mesh as parameters in a dedicated JSON file. To map the message client to the client defined in the SAP S/4HANA Cloud JSON file, in the SAP Event Mesh JSON file, you must include the `subscribeFilter` parameter which specifies the `sap/S4HANAOD/{emClientId}` namespace you created for the SAP S/4 HANA Cloud Extensibility instance of the `messaging` plan. For more information about the structure of the JSON file, see Define SAP Event Mesh Service Descriptor JSON/YAML File [page 2411].

Process Flow

Process Step	Related Documentation
1. Create an SAP Event Mesh service instance with service plan `default`.	• Create an SAP Event Mesh Service Instance in the Cloud Foundry Environment [page 2408] • Create an SAP Event Mesh Service Instance in the Kyma Environment [page 2409]
2. Define a dedicated JSON file that you add either when creating the SAP Event Mesh service instance, or after that.	Define SAP Event Mesh Service Descriptor JSON/YAML File [page 2411]
3. Create a queue in the message client that refers to the SAP Event Mesh service instance and then subscribe the topic of the namespace of the SAP S/4HANA Cloud Extensibility service to that queue.	Create Queues and Subscribe to Them [page 2417]

5.6.2.3.3.1 Create an SAP Event Mesh Service Instance in the Cloud Foundry Environment

Use this procedure to enable the SAP Event Mesh service for the subaccount where your extension application will reside.

Context

To enable the SAP Event Mesh service for your subaccount in SAP BTP, you have to go through the following steps.

Procedure

1. Prepare a JSON file that contains details of a message client. See Define SAP Event Mesh Service Descriptor JSON/YAML File [page 2411].
2. Create an SAP Event Mesh service instance for the application to consume SAP S/4HANA Cloud events.
   a. In the SAP BTP cockpit, navigate to the space in which you want to create a service instance.
   b. In the navigation area, choose *Services* > *Service Marketplace*, and then choose *Event Mesh* in the *Service Marketplace* panel.
   c. From the *Event Mesh* service tile, choose *Create* and follow the steps in the wizard to subscribe to the service.
   d. In the *New Instance or Subscription* wizard:

1. In the *Service* dropdown list, select *Event Mesh*.
2. In the *Plan* dropdown list select the *default* service plan.
3. In the *Runtime Environment* dropdown list, select *Cloud Foundry*.
4. In the *Space* dropdown list, select your Cloud Foundry space.
5. In the *Instance Name* field, enter a name for your instance. Choose *Next*.
6. Specify the JSON file you prepared in **Step 1**. Choose *Next*.
7. Review and verify the instance details, and choose *Create*.

Next Steps

Create Queues and Subscribe to Them [page 2417]

5.6.2.3.3.2 Create an SAP Event Mesh Service Instance in the Kyma Environment

Use this procedure to enable the SAP Event Mesh service for the subaccount where your extension application will reside.

Prerequisites

- In the SAP BTP cockpit, you have assigned the `messaging` SAP S/4HANA Cloud Extensibility service plan to the SAP BTP subaccount that you want to pair with the SAP S/4HANA Cloud tenant. See Configure the Entitlements for the SAP S/4HANA Cloud Extensibility Service [page 2367].
- In the Kyma dashboard, you have created the SAP S/4HANA Cloud Extensibility service instance with the `messaging` plan. See Create an SAP S/4HANA Cloud Extensibility Service Instance in the Kyma Environment [page 2396].
- In the SAP BTP cockpit, you have assigned the `default` SAP Event Mesh service plan to the SAP BTP subaccount that you want to pair with the SAP S/4HANA Cloud tenant. See Configure the Entitlements for the SAP S/4HANA Cloud Extensibility Service [page 2367].

Context

In the Kyma dashboard, create an instance of the SAP Event Mesh service. This instance will specify the details of the namespace that you created in SAP Event Mesh for the SAP S/4HANA Cloud Extensibility service instance with the `default` plan. This way, you are setting up a contract between SAP Event Mesh and the Kyma environment, and connect them to enable seamless event flow from the given SAP S/4HANA Cloud tenant to Kyma.

Procedure

1. In the SAP BTP cockpit, navigate to the subaccount for which you want to create an SAP Event Mesh service instance.
2. On the subaccount *Overview* page in the *Kyma Environment* section, open the Kyma dashboard.
3. Navigate to the *default* namespace from the drop-down list in the top navigation.
4. Choose *Service Management* > *BTP Service Instances* from the left-hand side navigation.
5. In the *Service Instances* page, choose *Create Service Instance* in the upper right-hand corner. A new dialog opens.
6. Choose the *Simple* tab and fill in the following fields:
   - Give a meaningful name of the new SAP Event Mesh service instance.
   - In the *Offering Name* field, enter **`enterprise-messaging`**, which is the technical name of the SAP Event Mesh service.
   - In the *Plan Name* field, enter **`default`**.
7. Choose the *YAML* tab.

   In *parameters:* in the *spec:* section, specify the parameters to define the details of your message client in the SAP Event Mesh service. You must include the `subscribeFilter` parameter which specifies the `sap/S4HANAOD/{emClientId}` namespace you created for the SAP S/4 HANA Cloud Extensibility instance of the `messaging` plan in SAP Event Mesh. For more information about the structure of the YAML file, see Define SAP Event Mesh Service Descriptor JSON/YAML File [page 2411].
8. Choose *Create*.

Results

Once the service instance is provisioned, you can see it under *Service Management* > *Instances* in the left-hand side navigation of the Kyma dashboard.

Next Steps

Create Queues and Subscribe to Them [page 2417]

5.6.2.3.3.3 Define SAP Event Mesh Service Descriptor JSON/YAML File

The SAP Event Mesh service descriptor defines details of a message client and needs to be provided when provisioning new SAP Event Mesh service instances with service plan `default`.

Prerequisites

- You have created an SAP S/4HANA Cloud Extensibility service instance with defined SAP S/4HANA Cloud Extensibility descriptor. See Define SAP S/4HANA Cloud Extensibility Service Descriptor JSON/YAML File [page 2398].
- Get to know the additional parameters you can have in the SAP Event Mesh service descriptor. In this page only the parameters that are related to this use case are described. See Syntax for Service Descriptor in SAP Event Mesh documentation.

Context

You set up service instance of the SAP Event Mesh service to receive the events which are sent by the SAP S/4HANA Cloud Extensibility service. In the JSON file for the Cloud Foundry environment and in the YAML file for the Kyma environment, this service instance needs to be created with a different `emname` as well as a different `namespace`. The rule set here is important as each client should only publish in its own namespace. Since this message client's purpose is to receive messages from a different client, the rules of the `subscribeFilter` for topics need to take care of that. So the value of the message client defined in the SAP Event Mesh JSON/YAML file has to be configured to have the value of the `subscribeFilter` to be the same as the value of the `namespace` of the other client defined in the SAP S/4HANA Cloud Extensibility JSON/YAML file.

After both message clients are set up, you have to create a queue in the message client which refers to the SAP Event Mesh service instance. You can name the queue as you like and it will be created in the namespace of the client. This queue can now be subscribed to the topic configured **step 7** in the Create an SAP S/4HANA Extensibility Service Instance in the Cloud Foundry Environment [page 2394] page. This topic in defined in the namespace of the S/4HANA Cloud Extensibility service descriptor, for example **`sap/S4HANAOD/s4hc/myTopic`**.

The service instance of SAP Event Mesh is the message client which consumes the messages and should be bound to the actual application. These are the important aspects of the whole configuration:

- When creating a service instance of the SAP S/4HANA Cloud Extensibility service:
  - Use unique emname per subaccount
  - Use unique namespace per subaccount, for example **`a/b/c`**
- When creating a service instance of the SAP Event Mesh service:
  - Use unique emname per subaccount
  - Use unique namespace per subaccount, for example **`d/e/f`**
  - Use the namespace defined in the SAP S/4HANA Cloud Extensibility JSON/YAML file and put it in the `subscribeFilter` parameter

JSON:

Sample Code

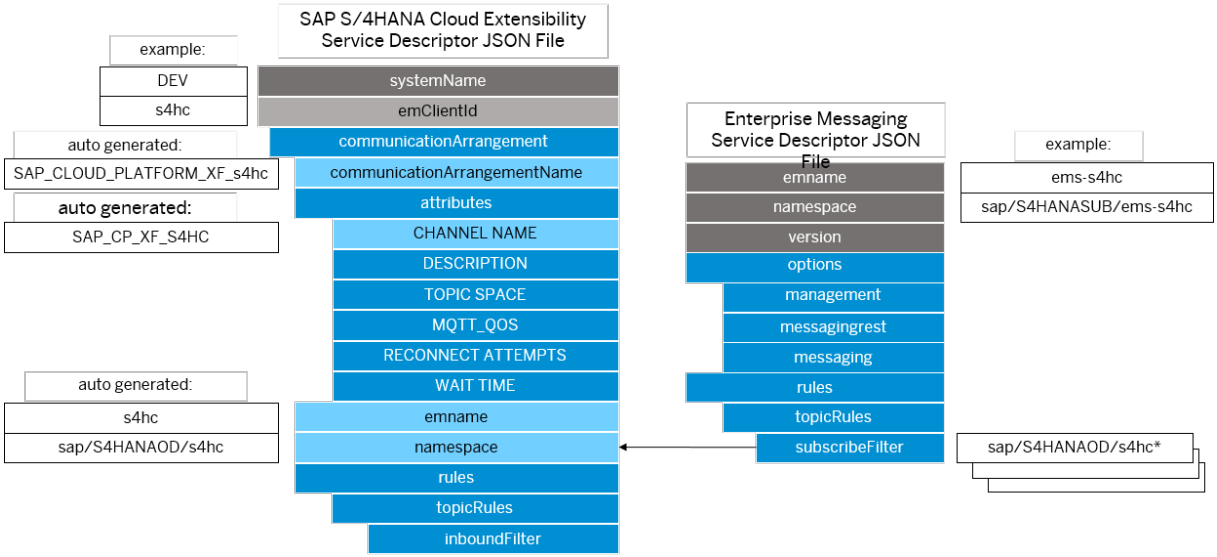
```
 "subscribeFilter": [
 "a/b/c/*",
"${namespace}/*"
 ]
```

YAML:

Sample Code

```
subscribeFilter:
- a/b/c/*
- "${namespace}/*"
```

- Create queue for this client
- Subscribe this queue to the topic defined in the namespace of the SAP S/4HANA Cloud Extensibility JSON/YAML file



Legend:

- Required property
- Required property whose value is used to generate automatically the values of a set of optional properties
- Optional property
- Optional property whose value is generated automatically based on the value of a required property. If specified, overrides the automatically generated one

Procedure

Define the JSON file for the SAP Event Mesh service instance following the requirements for the properties and their values listes below:

Parameter	Description
`emname`	Specifies the name of the SAP Event Mesh client. It is used by SAP Event Mesh to identify clients. **Note** The `emname` parameter in this descriptor must be unique per subaccount, so it must have the different value than the `emname` parameter in the SAP S/4HANA Cloud service descriptor. See Define SAP S/4HANA Cloud Extensibility Service Descriptor JSON/YAML File [page 2398]. **Rules/Guidelines** • Required: Yes • Allowed characters: [a-zA-Z0-9_-] • Max length: 100 **Rules/Guidelines:** It is unique within a subaccount.
`version`	It specifies the version of the service descriptor. **Note** The version parameter is optional when you use the deprecated syntax. See Syntax for Service Descriptor. **Rules/Guidelines** • Required: Yes • Allowed value: 1.1.0

Parameter	Description
`namespace`	Namespace for the messaging client. **Note** The `namespace` parameter in this descriptor must be unique per subaccount, so it must have the different value than the `namespace` parameter in the SAP S/4HANA Cloud service descriptor. See Define SAP S/4HANA Cloud Extensibility Service Descriptor JSON/YAML File [page 2398]. **Rules/Guidelines** • Required: Yes • Allowed characters: [a-zA-Z0-9//] • Max length: 24 • It is unique within a subaccount • Contains exactly three segments, for example **`d/e/f`**.
`options`	Defines the access channels for the message client.
`management`	An `options` attribute. Enables/disables the usage of the management REST APIs. **Rules/Guidelines** • Default: false • Allowed values: true | false
• `messagingrest`	An `options` attribute. Enables/Disables the usage of the messaging REST APIs. **Rules/Guidelines** • Default: false • Allowed values: true | false
`messaging`	An `options` attribute. Enables/disables the usage of the messaging gateway. **Rules/Guidelines** • Default: true • Allowed values: true | false

Parameter	Description
`rules`	Defines the access privileges of the messaging client. **Rules/Guidelines** • Required: true • **Guidelines/Rules:** In order to allow access to a queue or a topic, the namespace of the corresponding owner client has to be added. The placeholder `${namespace}` can be used instead of the defined namespace.
`topicRules`	A `rules` attribute. **Rules/Guidelines** • Type: Object
`subscribeFilter`	A `topicRules` attribute. Filters from which topics a client (receiver) is allowed to receive messages. **Rules/Guidelines** • Type: Array • Allowed characters: [a-zA-Z*?/] • Default: no default • Example values: **`${namespace}/foo/bar/*`**, **`${namespace}/*`**. **Note** The `namespace` part of the value must be specified exactly as the `namespace` value of the SAP S/4HANA Cloud Extensibility service descriptor JSON File described in Define SAP S/4HANA Cloud Extensibility Service Descriptor JSON/YAML File [page 2398].

JSON File Example

```
{
    "emname": "ems-s4hc",
    "namespace": "sap/S4HANAOD/ems-s4hc",
   "version": "1.1.0",
    "options": {
        "management": true,
        "messagingrest": true,
        "messaging": true
    },
    "rules": {
```

```
        "queueRules": {
            "subscribeFilter": [
                "${namespace}/*"
            ]
        },
        "topicRules": {
            "subscribeFilter": [
                "${namespace}/*",
                "sap/S4HANAOD/s4hc/*"
            ]
        }
    }
}
```

YAML File Example

```
spec:
  externalName: ''
  serviceOfferingName: enterprise-messaging
  servicePlanName: default
  parameters:
    emname: ems-s4hc
    namespace: sap/S4HANAOD/ems-s4hc
    version: 1.1.0
    options:
      management: true
      messagingrest: true
      messaging: true
    rules:
      queueRules:
        subscribeFilter:
        - "${namespace}/*"
    topicRules:
      subscribeFilter:
        - "${namespace}/*"
        - sap/S4HANAOD/s4hc/*
```

Related Information

Syntax for Service Descriptor in SAP Event Mesh documentation

5.6.2.3.3.4 Create Queues and Subscribe to Them

Prerequisites

You have created the `SAP S/4HANA Cloud Extensibility` service instance in the Cloud Foundry or Kyma environment. See:

- Create an SAP S/4HANA Extensibility Service Instance in the Cloud Foundry Environment [page 2394]
- Create an SAP S/4HANA Cloud Extensibility Service Instance in the Kyma Environment [page 2396]

You have created the SAP Event Mesh service instance in the Cloud Foundry or Kyma environment. See:

- Create an SAP Event Mesh Service Instance in the Cloud Foundry Environment [page 2408]
- Create an SAP Event Mesh Service Instance in the Kyma Environment [page 2409]

Context

After both instances are created and configured, you can create the topic-to-queue subscription. To do so, you first create a subscription to SAP Event Mesh with plan `standard`. Then, create a queue in the client that refers to the SAP Event Mesh service instance and then subscribe the topic of the namespace of the SAP S/4HANA Cloud Extensibility service to that queue.

> **Note**
>
> You can subscribe to the SAP Event Mesh service once per subaccount.

Procedure

1. Create a subscription to SAP Event Mesh.
   a. In the SAP BTP cockpit, navigate to the space in which you want to create a service instance.
   b. In the navigation area, choose *Services* > *Service Marketplace*, and then choose *Event Mesh* in the *Service Marketplace* panel.
   c. From the *Event Mesh* service tile, choose *Create* and follow the steps in the wizard to subscribe to the service.
   d. In the *New Instance or Subscription* wizard:
      1. In the *Service* dropdown list, select *Event Mesh*.
      2. In the *Plan* dropdown list, select the *standard* service plan. Choose *Create*.
2. Assign the necessary roles to your user.
   a. In the SAP BTP cockpit, navigate to your subaccount.

b. In the navigation area, choose *Security* > *Trust Configuration*, and then choose *SAP ID Service* in the *Trust Configuration* panel.
c. On the *Role Collection Assignment* page, enter the e-mail of your user in the empty field and choose *Show Assignments*. Then, choose *Assign Role Collection*.
d. In the *Assign Role Collection* wizard, select the following role collections:
   - *Enterprise Messaging Administrator*
   - *Enterprise Messaging Developer*
   - *Enterprise Messaging Display*
   - *Enterprise Messaging Subscription Administrator*
e. Choose *Assign Role Collection*.

3. Open the SAP Event Mesh application.
   a. In the SAP BTP cockpit, navigate to the space in which you want to create a service instance.
   b. In the navigation area, choose *Services* > *Instances and Subscriptions*.
   c. In the *Subscriptions* tab, choose the *Event Mesh* application.
   d. From the *Event Mesh* page, choose *Go to Application* and open the SAP Event Mesh application.

> **Note**
>
> If you are using the sample JSON files when creating the SAP S/4HANA Extensibility and the Event Mesh service instances, you should see 2 message clients in the SAP Event Mesh application, *s4hc* and *ems-s4hc*.

4. Create a queue that is specific to your application.
   a. Open the message client created when creating the Event Mesh service instance. For example, choose the *ems-s4hc* tile.
   b. Choose the *Queues* tab in the message client page and then choose *Create Queue*.
   c. Enter a queue name. Note that in the blue field of the *Create a New Queue* wizard, the final queue name is displayed. For example, if you write **`my-queue`** in the *Queue Name* field, the final queue name would be *sap/S4HANAOD/ems-s4hc/my-queue*. Choose *Create*.

5. Subscribe this queue to the channel topic that SAP S/4HANA Cloud tenant uses to produce events.
   a. From the *Actions* of your newly created queue, select *Queue Subscriptions*.
   b. In the *Topic* field, enter the topic you configured in the SAP S/4HANA Cloud tenant. For example, `sap/S4HANAOD/s4hc/ce/sap/s4/beh/businesspartner/v1/BusinessPartner/Changed/v1`.
   c. Choose *Add*.

Related Information

Manage Queues
REST APIs for Events

5.6.2.4 Supported Service Plans for SAP S/4HANA Cloud

The following service plans are available for subaccounts in SAP BTP paired with an SAP S4/HANA Cloud tenant.

api-access

This service plan enables all communication scenarios, both predefined and custom, which allow you to consume SAP S/4HANA Cloud APIs and integrate your extension applications with the respective SAP S/4HANA Cloud functionality.

- To access the specific documentation of these APIs, see SAP S/4HANA Cloud APIs at SAP Business Accelerator Hub.
- For more information about SAP S/4HANA Cloud communication scenarios see:
  - Communication Management
  - Custom Communication Scenarios

> **Note**
>
> These service plans have been deprecated:
>
> - *sap_com_0109*
> - *sap_com_0009*
> - *sap_com_0008*
>
> However, you can still enable these communication scenarios using the *api-access* service plan:
>
> - *Sales Order Integration (SAP_COM_0109)*: allows you to integrate your extension applications with sales order processing in SAP S/4HANA Cloud. For more information, see https://api.sap.com/api/API_SALES_ORDER_SRV/overview.
> - *Product Integration (SAP_COM_0009)*: enables you to replicate product master data from client system to SAP S/4HANA system. For more information, see https://api.sap.com/api/PRODUCTMDMBULKREPLICATEREQUEST/overview.
> - *Business Partner, Customer and Supplier Integration (SAP_COM_0008)*: allows you to consume the Business Partner API which enables you to create, read, update, and delete master data related to Business Partners, Suppliers, and Customers in an SAP S/4HANA system. For more information, see https://api.sap.com/api/API_BUSINESS_PARTNER/overview.
>
> For a sample JSON for these communication scenarios, see Communication Arrangement JSON/YAML File - Properties [page 2374].

messaging

This service plan enables the *Enterprise Eventing Integration (SAP_COM_0092)* communication scenario which allows you to consume SAP S/4HANA Cloud events and create event-based extensions.

> **Note**
>
> This service plan has been renamed from *sap_com_0092*.

5.6.2.5 Configuring Backup

You can back up and restore the destination and the trust configuration settings for the integration between the SAP S/4HANA Cloud and the global account in SAP BTP.

You can back up and restore the following configuration settings:

- The HTTP destination configured during the creation of the SAP S/4HANA Cloud Extensibility service instance. For more, see Export Destinations.
- Your subaccount X.509 certificates. For more information, see Set up Trust Between Systems
- Your subaccount-specific IdP metadata.
  To download your SAP SuccessFactors IdP SAML 2.0 metadata, in the SAP BTP cockpit, navigate to *Connectivity* > *Destinations* > *Download IdP Metadata*.
- Your identity provider configurations.
  You can back up identity provider configurations with the Identity Provider Management API. For more information see: Identity Provider Management API.

5.6.2.6 Auditing and Logging Information

Here you can find a list of the events that are logged by SAP S/4HANA Cloud Extensibility service. To retrieve the audit logs stored for SAP S/4HANA Cloud Extensibility create a support ticket in component BC-NEO-EXT-S4C.

Events written in audit logs

Event grouping	What events are logged	How to identify related log events	Additional information
Creating an integration	Start creating integration	• `"type":"s4Integration"` • `"attributes":[{"name":<object_name>,"new":<integration_details>}` • `"status":"BEGIN"`	Event that signifies an integration of a global account with an SAP S/4HANA Cloud system has been started. The log contains integration details such as the SAP S/4HANA Cloud tenant host name and ID, the Identity Authenticattion tenant ID, the integration ID, and other. The `customDetails` attribute contains the SAP S/4HANA Cloud user ID of the user who has triggered the integration process.
	Created integration	• `"type":"s4Integration"` • `"attributes":[{"name":<object_name>,"new":<integration_details>}` • `"status":"END"`	Event that signifies an integration of a global account with an SAP S/4HANA Cloud system has been completed. The log contains integration details such as the SAP S/4HANA Cloud tenant host name and ID, the Identity Authenticattion tenant ID, the integration ID, and other. The `customDetails` attribute contains the SAP S/4HANA Cloud user ID of the user who has triggered the integration process.

Event grouping	What events are logged	How to identify related log events	Additional information
	Delete integration	• `"type":"s4Integration"` • `"attributes":[{"name":<object_name>,"old":<integration_details>}` • "status":"BEGIN"	Event that signifies a deletion of an integration of a global account with an SAP S/4HANA Cloud system has been triggered. The log contains integration details such as the SAP S/4HANA Cloud tenant host name and ID, the Identity Authenticattion tenant ID, the integration ID, and other. The `customDetails` attribute contains the SAP S/4HANA Cloud user ID of the user who has triggered the deletion process.
	Deleted integration	• `"type":"s4Integration"` • `"attributes":[{"name":<object_name>,"old":<integration_details>}` • "status":"END"	Event that signifies a deletion of an integration of a global account with an SAP S/4HANA Cloud system has been completed. The log contains integration details such as the SAP S/4HANA Cloud tenant host name and ID, the Identity Authentication tenant ID, the integration ID, and other. The `customDetails` attribute contains the SAP S/4HANA Cloud user ID of the user who has triggered the deletion process.
Setting up the connectivity	Start creating connectivity configuration	• `"type":"s4Configuration"` • `"attributes":[{"name":<object_name>,"new":<connection_details>}` • `"status":"BEGIN"`	Event that signifies a creation of a service instance of the SAP S/4HANA Cloud Extensibility service has been triggered. The log contains details such as the global account ID, the subaccount ID, the communication arrangement, and the authentication mechanism.

Event grouping	What events are logged	How to identify related log events	Additional information
	Created connectivity configuration	• `"type":"s4Configuration"` • `"attributes":[{"name":<object_name>,"old":<connection_details>}` • `"status":"BEGIN"`	Event that signifies a creation of a service instance of the SAP S/4HANA Cloud Extensibility service has been completed. The log contains details such as the global account ID, the subaccount ID, the communication arrangement, and the authentication mechanism.
	Start deleting connectivity configuration	• `"type":"s4Configuration"` • `"attributes":[{"name":<object_name>,"old":<connection_details>}` • `"status":"BEGIN"`	Event that signifies the deletion of a service instance of the SAP S/4HANA Cloud Extensibility service has been started. The log contains details such as the global account ID, the subaccount ID, the communication arrangement, and the authentication mechanism.
	Deleted connectivity configuration	• `"type":"s4Configuration"` • `"attributes":[{"name":<object_name>,"old":<connection_details>}` • `"status":"END"`	Event that signifies the deletion of a service instance of the SAP S/4HANA Cloud Extensibility service has been completed. The log contains details such as the global account ID, the subaccount ID, the communication arrangement, and the authentication mechanism.

The following information is described in the table columns:

- *Event grouping* - Events that are logged with a similar format or are related to the same entities.
- *What events are logged* - Description of the security or data protection and privacy related event that is logged.
- *How to identify related log events* - Search criteria or key words, that are specific for a log event that is created along with the logged event.
- *Additional information* - Any related information that can be helpful.

Related Information

Audit Logging in the Cloud Foundry Environment

5.6.2.7 Accessibility Features in SAP S/4HANA Cloud Extensibility Service

To optimize your experience of SAP S/4HANA Cloud Extensibility service, SAP Business Technology Platform (SAP BTP) provides features and settings that help you use the software efficiently.

> **Note**
>
> SAP S/4HANA Cloud Extensibility service runs on the SAP BTP cockpit. For this reason, accessibility features for SAP BTP cockpit also apply. See the accessibility documentation for SAP BTP cockpit on SAP Help Portal at Accessibility Features in SAP BTP cockpit [page 2459].

For more information on screen reader support and keyboard shortcuts, see Accessibility for End Users.

5.6.2.8 Troubleshooting for SAP S/4HANA Cloud Extensibility Service

Information about troubleshooting issues you might experience when extending SAP S/4HANA Cloud on SAP BTP using the SAP S/4HANA Cloud Extensibility service.

Cloud Foundry Environment

These are some issues you might experience when extending SAP S/4HANA Cloud on SAP BTP, Cloud Foundry environment:

- Cannot create an SAP S/4HANA Cloud Extensibility Service Instance of the Messaging Plan [page 2424]
- My SAP S/4HANA Cloud Extensibility Service Instance Failed to Create and It Didn't Say Why [page 2425]

5.6.2.8.1 Cannot create an SAP S/4HANA Cloud Extensibility Service Instance of the Messaging Plan

Cannot create an SAP S/4HANA Cloud Extensibility service instance of the messaging plan.

Issue/Symptom

You are trying to create an SAP S/4HANA Cloud Extensibility service instance of the messaging plan and you get the following error message:"Creation Failed."

Reason

You are probably providing in the JSON file the same *emClientId* that was already used in the subaccount.

Solution

Make the value of the *emClientId* parameter unique within the subaccount.

Related Information

Set Up the Connectivity Between Event Mesh and the SAP S/4HANA Cloud Tenant [page 2393]

5.6.2.8.2 My SAP S/4HANA Cloud Extensibility Service Instance Failed to Create and It Didn't Say Why

My SAP S/4HANA Cloud Extensibility service instance failed to create and it didn't say why.

Issue/Symptom

You are trying to create an SAP S/4HANA Cloud Extensibility service instance but the creation fails with no error message.

Solution

This is what you can do:

- You can use the Cloud Foundry command line interface to get the error message by executing the command:
  ```
  cf service <service_instance_name>
  ```
  Check for any error messages there.
- Can you reach the SAP S/4HANA Cloud tenant? Check that the tenant is not in maintenance or being updated.

If this didn't help, you can try some more specific actions. What service plan are you using?

I am using the messaging service plan

Issue/Symptom

Your SAP S/4HANA Cloud Extensibility service instance of the messaging plan failed to create and it didn't say why.

Solution

Try to do the following:

- Is the Enterprise Messaging Integration scenario (SAP_COM_0092) enabled on your SAP S/4HANA Cloud tenant? Check whether you see it in the **Display Communication Scenarios** application in your SAP S/4HANA Cloud tenant.
- The clients for SAP Event Mesh require unique values (within your subaccount in SAP BTP) for the **emname** and **namespace** properties. In the *messaging* plan, you have either provided these manually, or you have only set the **emClientId** property and let it generate them. If the latter, then these values would be **emClientId>** for **emname** and **sap/S4HANAOD/emClientId>** for **namespace**. Check whether your subaccount already has existing clients with such **emname** or **namespace** values. You can do this by using the **Enterprise Messaging Business Application** from the SAP Event Mesh subscription of your subaccount. See View Event Catalog for a Subaccount.

I am using the api-access service plan

Issue/Symptom

Your SAP S/4HANA Cloud Extensibility service instance of the api-access plan failed to create and it didn't say why.

Solution

Try to do the following:

- Is the communication scenario with the **scenarioId** that you provided enabled on that tenant? Check whether you see it in the **Display Communication Scenarios** application in your SAP S/4HANA Cloud tenant.
- Are you using supported authentication types? In the scenario definition, you can see which authentication types are supported for both outbound and inbound connectivity. Verify that the types you are providing in the descriptor are supported by the scenario.

5.6.3 Extending SAP Marketing Cloud in the Cloud Foundry and Kyma Environment

Overview

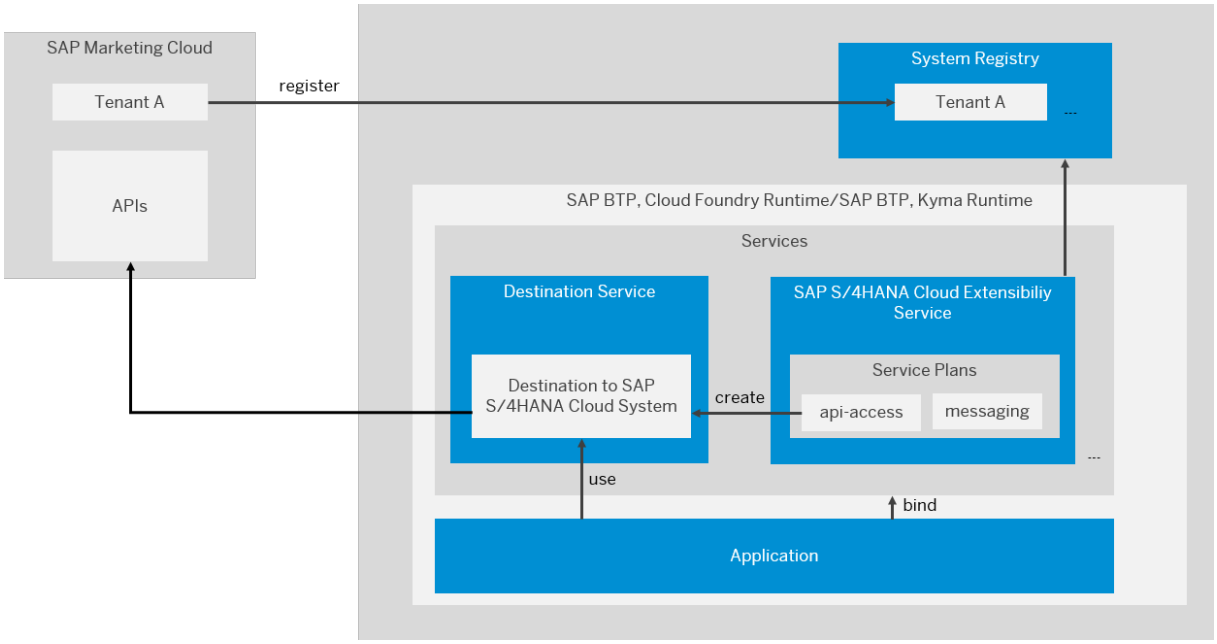
> **Note**
>
> The SAP S/4HANA Cloud Extensibility service is also available for global accounts marked with EU Access. See Regions.

SAP BTP offers a standard way for extending SAP solutions.

You can extend SAP Marketing Cloud in the Cloud Foundry or the Kyma environment without disrupting the performance and the core processes. When building extension applications, you can also benefit from the automation of the integration between the cloud platform and SAP Marketing Cloud.

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

The following graphic provides a high-level overview of the integration between SAP BTP and SAP Marketing Cloud:



Process Flow

To integrate SAP BTP and SAP Marketing Cloud so that you can build extension applications, you have to use the SAP S/4HANA Cloud Extensibility service. All the steps are the same from the SAP BTP side. You register an SAP Marketing Cloud system in the SAP BTP cockpit. Then, you take the generated token and use it to configure the connectivity on the SAP Marketing Cloud side. These are the tasks you need to follow:

Integrating SAP BTP and SAP Marketing Cloud

Process Step	Related Documentation
1. Connect the SAP Marketing Cloud system you want to extend with the corresponding global account in SAP BTP. During the pairing process you create an registration token which is then used by the SAP Marketing Cloud system tenant administrator to configure the integration on the SAP Marketing Cloud system side.	Register an SAP Marketing Cloud System in a Global Account in SAP BTP [page 2304] **Note** You cannot migrate the registered SAP Marketing Cloud systems between global accounts. If you want to start using another global account, you will have to register your SAP Marketing Cloud systems again.
2. Make the SAP Marketing Cloud system accessible in the subaccounts in SAP BTP in which you want to build your extension applications. To do so, you configure the entitlements and assign the corresponding quota and service plans to the subaccounts where the extension applications will reside for the system you registered in the previous step.	Configure the Entitlements for the SAP S/4HANA Cloud Extensibility Service [page 2367]

Process Step	Related Documentation

3. Configure the communication flow.
 	
You have the following options:
 	
• Consume the SAP Marketing Cloud APIs (inbound connection) or consume APIs exposed by the extension application from SAP S/4 HANA Cloud (outbound connection)
To do so, you create a service instance of the *SAP S/4HANA Cloud Extensibility* service using the *api-access* service plan.
During the service instance creation an HTTP destination on a subaccount level is automatically generated in this subaccount. It contains all instance binding properties which are sufficient to establish connection to the SAP Marketing Cloud system. When creating the service instance, you configure the communication arrangement and the authentication type for the connection. The following authentication scenarios for SAP Marketing Cloud are supported:
  • Basic Authentication (inbound and outbound connections)
  • OAuth 2.0 SAML Bearer Assertion (inbound connections)
  • OAuth 2.0 Client Credentials (outbound Connections)
  • No Authentication (outbound connections)
Both predefined and custom communication scenarios are supported.
• Enable the consumption of SAP Marketing Cloud events.
If you want to create event-based extensions for SAP Marketing Cloud using the SAP Enterprise Messaging service, you have to create a service instance of the *SAP S/4HANA Cloud Extensibility* service using the *messaging* service plan.
• A combination of both
	• Creating a Service Instance to Consume the SAP S/4HANA Cloud APIs [page 2368]
	• Enable the Consumption of SAP S/4HANA Cloud Events [page 2390]

5.6.4 Extending SAP SuccessFactors in the Cloud Foundry and Kyma Environment

Use SAP BTP to extend SAP SuccessFactors with extension applications running on the cloud platform.

Overview

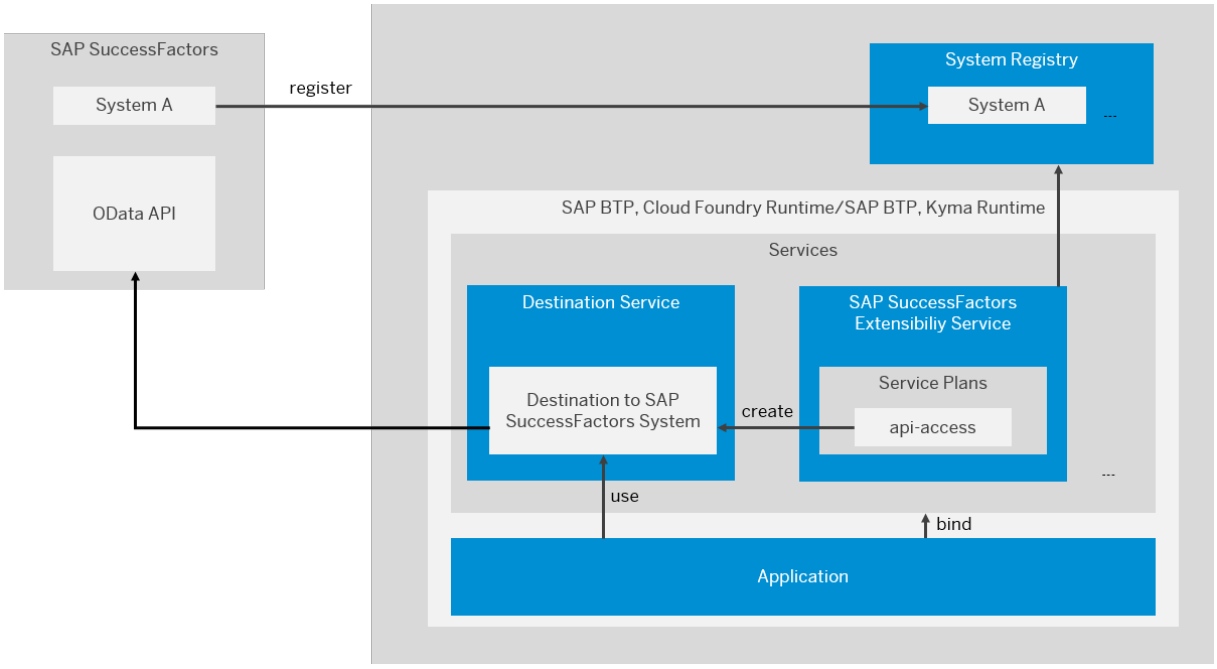
> **Note**
>
> The SAP SuccessFactors Extensibility service is also available for global accounts marked with EU Access. See Regions.

SAP BTP offers a standard way for extending SAP solutions.

You can extend SAP SuccessFactors systems without disrupting the performance and the core processes. When building extension applications, you can also benefit from the automation of the integration between the cloud platform and SAP SuccessFactors.

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

The following graphic provides a high-level overview of the integration between SAP BTP and SAP SuccessFactors:



Process Flow

To integrate SAP BTP and SAP SuccessFactors so that you can build extension applications, you need to perform the following tasks:

Integrating SAP BTP and SAP SuccessFactors

Process Step	Related Documentation
1. Connect the SAP SuccessFactors system that you want to extend with the corresponding global account in SAP BTP. During the pairing process you create a registration token which is then used by the SAP SuccessFactors system tenant administrator to configure the integration on the SAP SuccessFactors system side.	Register an SAP SuccessFactors System in a Global Account in SAP BTP [page 2312]. **Note** You cannot migrate the registered SAP SuccessFactors systems between global accounts. If you want to start using another global account, you will have to register your SAP SuccessFactors systems again.
2. Make the SAP SuccessFactors system accessible in the subaccounts in which you want to build your extension applications. To do so, you configure the entitlements and assign the corresponding quota where the extension applications will reside.	Configure the Entitlements for the SAP SuccessFactors Extensibility Service [page 2432]
3. Configure the communication flow. To do so, create a service instance of the SAP SuccessFactors Extensibility service using the *api-access* service plan. During the service instance creation an HTTP destination on a subaccount level is automatically generated in this subaccount. It contains all instance binding properties which are sufficient to establish connection to the SAP SuccessFactors system. SAP BTP supports the following authentication scenarios for SAP SuccessFactors: • OData access with OAuth 2.0 SAML bearer assertion • OData access with OAuth 2.0 SAML bearer assertion with technical user	Create a Service Instance to Consume the SAP SuccessFactors HXM Suite OData API [page 2433]

Process Step	Related Documentation
4. Configure the Single-Sign On (SSO) between the subaccount in SAP BTP and the SAP SuccessFactors system. To ensure the required security for accessing the applications, you need to configure the single sign-on between the subaccount in SAP BTP and the SAP SuccessFactors system using a SAML identity provider.	Single Sign-On Between a Subaccount in SAP BTP and SAP SuccessFactors [page 2440]
5. If you have performed an automated instance refresh or your cloud operators have performed a manual instance refresh, you need to restore some of the extension configuration settings.	Restore Configuration Settings After an Instance Refresh [page 2445]

5.6.4.1 Configure the Entitlements for the SAP SuccessFactors Extensibility Service

Configure the required entitlements to make the SAP SuccessFactors HXM Suite OData APIs of the registered SAP SuccessFactors system accessible in your subaccount in which your extension applications will reside.

Prerequisites

- You are an administrator of the global account in SAP BTP.
- The subaccount is in the SAP BTP, Cloud Foundry environment with enabled Cloud Foundry, or Kyma, or both capabilities.
- You have registered an SAP SuccessFactors system. See Register an SAP SuccessFactors System in a Global Account in SAP BTP [page 2312].

Context

An entitlement is your right to provision and consume a resource. In other words, the entitlement is the *api-access* service plan that you're entitled to use, and the *sso-configuration* service plan that allows you to register an assertion consumer service in SAP SuccessFactors automatically.

Procedure

1. In the SAP BTP cockpit, navigate to your global account.
2. In the navigation area, choose *Entitlements* > *Entity Assignments*.
3. On the *Entity Assignments* screen, select your subaccount in the *Subaccounts/Directories* field.
4. Choose *Edit*, and then choose *Add Service Plans*.
5. Select the *SAP SuccessFactors Extensibility* service.

   > **Note**
   >
   > To have the *SAP SuccessFactors Extensibility* service in the list, you need to have registered at least one SAP SuccessFactors system.

6. In the *Service Details* area, select the system you have registered. In the *Available Plans* area, select the *api-access* and *sso-configuration* service plans, and then choose *Add 2 Service Plans*.

Related Information

Configure Entitlements and Quotas for Subaccounts [page 2529]

5.6.4.2 Create a Service Instance to Consume the SAP SuccessFactors HXM Suite OData API

To enable the integration of your extension applications with the SAP SuccessFactors system you have registered in the global account in SAP BTP, you first need to create a service instance of the corresponding service.

Context

In both Cloud Foundry and Kyma environment, you consume services by creating a service instance. Service instances are created using a specific service plan.

To allow applications running on SAP BTP to consume SAP SuccessFactors HXM Suite OData APIs, you need to create a service instance of the SAP SuccessFactors Extensibility service using the *api-access* service plan.

You create the service instance in your subaccount with the respective environment enabled. When creating the service instance, you configure the authentication type for the communication by specifying the required configurations in a JSON format. SAP BTP supports the following authentication scenarios for SAP SuccessFactors:

- OData access with OAuth 2.0 SAML bearer assertion
  To use this authentication type, you must protect your application. See Adding Authentication and Authorization [page 546].

- OData access with OAuth 2.0 SAML bearer assertion with technical user

The process for creating a service instance depends on the environment you are using:

- For Cloud Foundry environment, see Create an SAP SuccessFactors Extensibility Service Instance in the Cloud Foundry Environment [page 2434].
- For Kyma environment, see Create an SAP SuccessFactors Extensibility Service Instance in the Kyma Environment [page 2436].

5.6.4.2.1 Create an SAP SuccessFactors Extensibility Service Instance in the Cloud Foundry Environment

To enable the integration of your extension applications with the SAP SuccessFactors system you have registered in the global account in SAP BTP, you first need to create a service instance of the corresponding service.

Prerequisites

- You have registered an SAP SuccessFactors system. See Register an SAP SuccessFactors System in a Global Account in SAP BTP [page 2312].
- Before creating an SAP SuccessFactors Extensibility service instance in the Cloud Foundry environment, see Create a Service Instance to Consume the SAP SuccessFactors HXM Suite OData API [page 2433].
- Have enabled Cloud Foundry environment for your subaccount. See Enable Environment or Create Environment Instance [page 2515].
- Have configured the entitlements to the SAP SuccessFactors Extensibility service. See Configure the Entitlements for the SAP SuccessFactors Extensibility Service [page 2432].

Context

In the Cloud Foundry environment, you consume services by creating a service instance. Service instances are created using a specific service plan. The services are offered in the Service Marketplace, from which you can create service instances to provision the reserved resources.

To allow applications running on SAP BTP to consume SAP SuccessFactors HXM Suite OData APIs, you need to create a service instance of the SAP SuccessFactors Extensibility service using the *api-access* service plan.

You create the service instance in a space of your subaccount. When creating the service instance, you configure the authentication type for the communication by specifying the required configurations in a JSON format. SAP BTP supports the following authentication scenarios for SAP SuccessFactors:

- OData access with OAuth 2.0 SAML bearer assertion
  To use this authentication type, you must protect your application. See Authentication for Applications.

- OData access with OAuth 2.0 SAML bearer assertion with technical user

During the creation of the service instance, a destination on a subaccount level with the same name as the service instance name is automatically created in this subaccount. It contains all instance binding properties which are sufficient to establish connection to the SAP SuccessFactors system.

> **Note**
>
> Make sure that you don't already have a destination with the same name as the service instance. If you do, you will not be able to create the service instance.

Procedure

1. In the SAP BTP cockpit, navigate to the subaccount in which you want to create a service instance.
2. In the navigation area, choose *Services* > *Service Marketplace*.

   All services available to you appear.
3. To enable the integration with an SAP SuccessFactors system that you have registered in SAP BTP global account, choose *SAP SuccessFactors Extensibility*.
4. In the *SAP SuccessFactors Extensibility* page, choose *Create*.
5. In the *New Instance or Subscription* wizard:
   a. In the *Service* dropdown list, ensure you have selected the *SAP SuccessFactors Extensibility* service.
   b. In the *Plan* dropdown list, select the *api-access* service plan.
   c. In the *Runtime Environment* dropdown list, select *Cloud Foundry*.
   d. In the *Space* dropdown list, select your space. If you haven't create a space yet, you can do it at this point.
   e. In the *System Name* dropdown list, select the SAP SuccessFactors system you have registered.
   f. In the *Instance Name* field, enter a name for your instance. Choose *Next*.
   g. (Optional) To define a technical user the access to the SAP SuccessFactors HXM Suite API, specify a JSON file or specify parameters in the JSON format. If you decide to add a technical user later on, you have to delete this service instance and create it again. Choose *Next*.

      For more information about the structure of the JSON file, see API Access Configuration JSON File [page 2438].
   h. Choose *Create*.

Results

After you have created the service instance:

- The newly created instance appears in the list of instances in the *Instance and Subscriptions* page.
- An HTTP destination on a subaccount level with the same name as the service instance name is automatically generated in this subaccount.

Alternatively, you can use the Cloud Foundry Command Line Interface (cf CLI) to create the service instance using the technical name of the SAP SuccessFactors Extensibility service which is **`sap-successfactors-extensibility`**.

For more information, see Create Service Instances Using the Cloud Foundry Command Line Interface.

> **Note**
>
> You can use the cf CLI to troubleshoot if the creation of the service instance fails. To do that, use this command in the cf CLI:
>
> ```
> cf service <service_instance_name>
> ```

Next Steps

After creating the *SAP SuccessFactors Extensibility* service instance, you have the following options for configuring the connectivity for your extension application:

- Bind the instance to an application, and it will be assigned an access URL and credentials to the corresponding API. For more information about binding applications to service instances, see Binding Service Instances to Applications.
- Consume the automatically generated destination.
  To consume the destination, you use the Destination service. You can either consume the Destination service directly, or configure the application router to consume it.

  > **Note**
  >
  > The name of the destination is the same as the name of the service instance you have created.

  - For more information about consuming the destination service using the application router, see Application Routes and Destinations.
  - For more information about consuming the destination service directly, see Consuming the Destination Service (Cloud Foundry Environment).

5.6.4.2.2 Create an SAP SuccessFactors Extensibility Service Instance in the Kyma Environment

Prerequisites

- Before creating an SAP SuccessFactors Extensibility service instance in the Kyma environment, see Create a Service Instance to Consume the SAP SuccessFactors HXM Suite OData API [page 2433].
- Have the Kyma environment enabled for the subaccount you are using. See Creating Kyma Instances [page 3452].

- Configure the roles in the Kyma environment. See Assign Roles in the Kyma Environment [page 3666].
- The SAP BTP Operator module is enabled. See Adding and Deleting a Kyma Module [page 3511]
- Have the entitlements of the SAP SuccessFactors Extensibility service configured. See Configure the Entitlements for the SAP S/4HANA Cloud Extensibility Service [page 2367].

Context

During the creation of the service instance, a destination on the subaccount level with the same name as the service instance name is automatically created in this subaccount. It contains all instance binding properties which are sufficient to establish connection to the SAP SuccessFactors system.

> **Note**
>
> Make sure that you don't already have a destination with the same name as the service instance. If you do, you will not be able to create the service instance.

Procedure

1. Navigate to the subaccount for which you want to create an SAP SuccessFactors Extensibility service instance.
2. On the subaccount *Overview* page, in the *Kyma Environment* section, open the Kyma dashboard.
3. Choose *Namespaces* from the left-hand side navigation and open the namespace in which you want to create a service instance.
4. Choose *Service Management* > *Service Instances* from the left-hand side navigation.
5. In the *Service Instances* page, choose *Create Service Instance* in the upper right-hand corner. A new dialog opens.
6. Choose the *Advanced* tab and fill in the following fields:
   - Give a meaningful name of the new SAP SuccessFactors Extensibility service instance.
   - In the *Offering Name* field, enter **`sap-successfactors-extensibility`**, which is the technical name of the SAP SuccessFactors Extensibility service.
   - In the *Plan Name* field, enter **`api-access`**.
   - Expand *Instance Parameters*.
     To define the authentication type for the API access, specify the parameters listed in this JSON file: API Access Configuration JSON File [page 2438].
7. Choose *Create*.

Next Steps

After creating the *SAP SuccessFactors Extensibility* service instance, you have to bind the instance to an application, and it will be assigned an access URL and credentials to the corresponding API. See Using SAP BTP Services in the Kyma Environment [page 2267].

5.6.4.2.3 API Access Configuration JSON File

Use the authentication type JSON descriptor to define the authentication type for the inbound connectivity to the SAP SuccessFactors HXM Suite OData API.

Parameter	Description
`systemName`	The name of the system you have registered in a global account in SAP BTP. **Note** The system must be in status *Registered*. **Rules/Guidelines** • Required: Yes

Parameter	Description
`technicalUser`	The name of the technical user for consuming the SAP SuccessFactors HXM Suite OData API without a logged-in user. **⚠ Caution** This property specifies the *systemUser* property in the destination that is created as a result of creating the SAP SuccessFactors Extensibility service instance. The *systemUser* is deprecated and will be removed soon. We recommend that you work on behalf of specific (named) users instead of working with a technical user. **ⓘ Note** The technical user can be any user with the respective permissions. These permissions depend on the use case and the API you want to access. To find out which permission you need to assign to the technical user, go to SAP Business Accelerator Hub, find the SAP SuccessFactors API you want to access and from the *Overview* tab go to the *Documentation* section and open the *help.sap.com* link. There you find the right information for each API. **Rules/Guidelines** • Required: No • Allowed characters: [a-zA-Z0-9_-] • Max length: 100

Example

```
{
   "systemName":"MY_SYSTEM",
   "technicalUser":"technicalonboarder"
}
```

5.6.4.3 Single Sign-On Between a Subaccount in SAP BTP and SAP SuccessFactors

Use this procedure to configure the single sign-on (SSO) between SAP BTP and the SAP SuccessFactors system.

When a user attempts to access SAP SuccessFactors for the first time, the system redirects the user to the identity provider for authentication. From then on, the user session is kept active, and the user is no longer prompted for credentials when trying to use the SAP SuccessFactors application. This is called single sign-on (SSO).

> **Note**
>
> For SAML-based single sign-on integrations, we recommend using Identity Authentication as the primary identity provider instead of using SAP SuccessFactors as the identity provider.

Using Identity Authentication

You can use Identity Authentication to set up the single sign-on (SSO) between SAP SuccessFactors and SAP BTP. Identity Authentication provides you with controlled cloud-based access to business processes, applications, and data. It simplifies your user experience through authentication mechanisms, single sign-on, on-premise integration, and convenient self-service options. We recommend using Identity Authentication as a central point to integrate SAP applications for a seamless single sign-on experience.

- Authentication: All SAP cloud applications can offer their users the same authentication mechanisms , as well as strong authentication with configurable multi-factor (MFA) enforcement; easy separation mechanism for multiple user stores and flexible configuration where to validate user's credentials.
- Single Sign-On: Identity Authentication offers central SSO endpoint for all SAP cloud applications and pre-configured or semi-automated trust configuration.
- Integrating SAP applications: Identity Authentication offers common identity for users, as well as a unified way for user management and security token service for protection of system-to-system communication. Data across applications can be correlated.

See SAP SuccessFactors Integration Scenario.

Using SAP SuccessFactors as a Trusted Identity Provider

Prerequisites

You have the *UI and Role Administrator* role assigned to your user. See Security Administration: Managing Authentication and Authorization [page 2537].

Context

The authentication to SAP SuccessFactors applications is restricted to the authorized users. The identification of a user is verified by the identity provider, as specified by SAML 2.0. The identity provider stores a list of all

users that are allowed to access the SAP SuccessFactors system along with their credentials. The integration between the SAP SuccessFactors and the identity provider is based on a trust configuration.

To ensure the required security for accessing the applications, you need to configure the single sign-on between the subaccount in SAP BTP and the SAP SuccessFactors system using a SAML identity provider. The single sign-on requires both solutions to be configured as trusted SAML service providers for the identity provider, and at the same time, the identity provider to be configured as trusted identity provider for the two solutions.

Use a subaccount in SAP BTP with the SAP BTP, Cloud Foundry runtime enabled and follow these steps to set up the single sign-on (SSO):

1. Configure SAP SuccessFactors as a Trusted Identity Provider in SAP BTP [page 2441]
2. Configure the Subaccount as a Trusted Service Provider in SAP SuccessFactors [page 2442]

5.6.4.3.1 Configure SAP SuccessFactors as a Trusted Identity Provider in SAP BTP

Use this procedure to set up the trust configuration of the subaccount in SAP BTP and add SAP SuccessFactors as an identity provider.

Procedure

1. Download SAML metadata from the SAP SuccessFactors system.
   a. Go to `https://<sap_successfactors_system>/idp/samlmetadata?company=<company_id>&cert=sha2` where:
      - `<sap_successfactors_system>` is the hostname of your SAP SuccessFactors system
      - `<company_id>` is the ID of your SAP SuccessFactors company
   b. When you are prompted, save the file on your local file system and change its extension to **.xml**.
2. Register the SAP SuccessFactors identity provider in the SAP BTP cockpit.
   a. Open the cockpit and navigate to your subaccount.
   b. Choose *Security* > *Trust Configuration*.
   c. Choose *New SAML Trust Configuration*.
   d. To upload the SAML metadata you downloaded in step 1, choose *Upload*. Browse to the XML file you saved and select it. Some of the fields are automatically filled in.
   e. In the *Name* field, enter a meaningful name for the trust configuration.
   f. Save the changes.
3. Make the trust configuration to the SAP SuccessFactors identity provider the only configuration that is active.

5.6.4.3.2 Configure the Subaccount as a Trusted Service Provider in SAP SuccessFactors

To configure the subaccount as a trusted service provider in SAP SuccessFactors, you have to create an SAP SuccessFactors Extensibility service instance using the *sso-configuration* service plan.

Prerequisites

- Register an SAP SuccessFactors System in a Global Account in SAP BTP [page 2312]
- Configure the Entitlements for the SAP SuccessFactors Extensibility Service [page 2432]
- Configure SAP SuccessFactors as a Trusted Identity Provider in SAP BTP [page 2441]

Context

With the SAP SuccessFactors Extensibility service instance using the *sso-configuration* service plan, you can create automatically an assertion consumer service in SAP SuccessFactors for the subaccount in which this service instance is created and a logout URL for each extension application you have deployed in SAP BTP that is extending the functionality of the registered SAP SuccessFactors system. To create an assertion consumer service, you either go to the SAP SuccessFactors Provisioning and create these assertion consumer services manually, or create them automatically in the SAP BTP cockpit via an SAP SuccessFactors Extensibility service instance using the *sso-configuration* service plan without loggin in to the Provisioning.

To create an assertion consumer service automatically, you create an SAP SuccessFactors Extensibility service instance of plan *sso-configuration* providing all the necessary information in a JSON file. You can have only one service instance of plan *sso-configuration* per subaccount.

In the JSON file you provide, in the *systemName* parameter, you specify the SAP SuccessFactors system where you want the assertion consumer services to be created. Then, you automatically get an assertion consumer service for the subaccount where the service instance is created and assertion consumer services for each logout URL you provided.

If you want to add or delete assertion consumer services with logout URLs, you have to update the SAP SuccessFactors Extensibility service instance of plan *sso-configuration* and provide a JSON file with the respective information. Before updating the service instance, you have the option to view the JSON that was previously passed.

When updating the service instance:

- All the new assertion consumer services with logout URLs will be created in SAP SuccessFactors.
- All the assertion consumer services with logout URLs that were previously created but are not part of the new JSON file will be deleted.
- All the assertion consumer services with logout URLs that were previously created and are part of the new JSON file will be recreated.

When you delete the SAP SuccessFactors Extensibility service instance of plan *sso-configuration*, all the assertion consumer services with logout URLs created by this service instance will also be deleted.

> **Note**
>
> You cannot bind the service instance of *sso-configuration* service plan.

Procedure

1. In the SAP BTP cockpit, navigate to the subaccount in which you want to create a service instance.
2. In the navigation area, choose *Services* > *Service Marketplace*.

   All services available to you appear.
3. To enable the integration with an SAP SuccessFactors system that you have registered in the global account, choose *SAP SuccessFactors Extensibility*.
4. In the *SAP SuccessFactors Extensibility* page, choose *Create*.
5. In the *New Instance or Subscription* wizard:
   a. In the *Service* dropdown list, ensure you have selected the *SAP SuccessFactors Extensibility* service.
   b. In the *Plan* dropdown list, select the *sso-configuration* service plan.
   c. In the *Runtime Environment* dropdown list, select *Other*.
   d. In the *System Name* dropdown list, select the SAP SuccessFactors system you have registered.
   e. In the *Instance Name* field, enter a name for your instance. Choose *Next*.
   f. To configure the assertion consumer service of the subaccount and the respective applications, specify a JSON file or specify parameters in a JSON format. You can also do this later on. Choose *Next*.

      > **Note**
      >
      > The content of the JSON file must be up to 5120 characters without spaces.

      For more information about the structure of the JSON file, see Single Sign-On Configuration JSON File [page 2444].
   g. Choose *Create*.

Results

You have the assertion consumer services created in SAP SuccessFactors and have the SSO between your subaccount in SAP BTP and your SAP SuccessFactors system.

You can check the assertion consumer services. To do so, go to the *Admin Center* of you SAP SuccessFactors company and in the *Search* field, enter *Authorized SP Assertion Consumer Service Settings*. This is a read-only information that allows you to verify if all the assertion consumer services that you need are created.

5.6.4.3.3 Single Sign-On Configuration JSON File

Use the single sign-on (SSO) configuration JSON file to define the assertion consumer services in SAP SuccessFactors for the subaccount in SAP BTP and for each extension application you have deployed in SAP BTP that is extending the SAP SuccessFactors system functionality.

> **Note**
>
> The content of the JSON file must be up to 5120 characters without spaces.

Parameter	Description
`systemName`	The name of the system you have registered in the global account in SAP BTP **Rules/Guidelines** The name must be the same as the one defined during the system registration in the global account. See Register an SAP SuccessFactors System in a Global Account in SAP BTP [page 2312]. > **Note** > The system must be in status *Registered*.
`logoutURLs`	(Optional) This is a list of the logout URLs. A logout URL is the absolute URL that corresponds to the URL of the application router with the appended value of the `logoutEndpoint` property. See logout [page 482]. > **Note** > Make sure that the value of the `logoutMethod` property is set to **GET** in the *xs-app.json* file. > **Note** > The `SameSite` attribute of the `Set-Cookie` HTTP response header needs to be set to `None` so that the cookies are sent in all responses to both first-party and cross-origin requests. You also need to set the cookie `Secure` attribute, because it requires a secure context/ HTTPS. See Environment Variables [page 510]. > **Note** > If you don't specify the `logoutURLs` parameter, only an assertion consumer service for the subaccount in SAP BTP will be created.

Sample Code

```
{
    "systemName": "MY_SYSTEM",
    "logoutURLs": ["https://my-first-site.com/logout", "https://my-second-
site.com/logout"]
}
```

5.6.4.4 Restore Configuration Settings After an Instance Refresh

After an instance refresh, you must restore some of your extension integration artifacts and configuration settings for the SAP SuccessFactors target company.

Prerequisites

- You have a dedicated SAP SuccessFactors company instance.
- You have a registered SAP SuccessFactors system in your global account in SAP BTP.
- You have performed the instance refresh with the *Instance Refresh* tool or your cloud operators have performed an instance refresh manually.

Context

In SAP SuccessFactors, instance refresh is a procedure in which the data and the settings of a source company instance are copied into another company instance. During the instance refresh the data of the target company instance is deleted and replaced by the data of the source company instance. Therefore, after an instance refresh, if the target company has been integrated with a global account in SAP BTP, the extension integration configuration settings and artifacts in the target company are overwritten by the configuration settings and data coming from the source company.

You can trigger the instance refresh automatically with the *Instance Refresh* tool or your cloud operators can trigger it manually.

If you have performed an instance refresh with the *Instance Refresh* tool, the OAuth clients, and the Assertion Consumer Services (ACS) created by the cloud platform are recreated. In *Provisioning*, the *Extension Management Configuration* page displays the integration details for the target system.

Note

The OAuth clients are assigned new IDs. This does not affect the configuration settings.

However, if you have the following artifacts, you must reconfigure them in the SAP SuccessFactors target company instance:

- Custom home page tiles
- Permission roles
- Permission groups

> **Note**
>
> After a manual instance refreshed, no artifacts are reconfigured. If you want to restore the integration settings for the target company instance, you need to recreate the *SAP SuccessFactors Extensibility* service instance.

Procedure

To restore the configuration settings after an instance refresh, proceed as follows:

- After an automated instance refresh:
  1. If you have configured custom home page tiles in the target company instance, reconfigure them. See Custom Home Page Tiles.
  2. If you have configured permission roles and permission groups, you need to reconfigure them. See What Are Role-Based Permissions?.
- After a manual instance refresh:
  1. For SAP SuccessFactors, Second Half 2021 Release or later, proceed as follows:
     1. Recreate the *SAP SuccessFactors Extensibility* service instance. See Create a Service Instance to Consume the SAP SuccessFactors HXM Suite OData API [page 2433].
     2. If you have configured permission roles and permission groups, you need to reconfigure them. See Custom Home Page Tiles.
     3. Reconfigure the permission roles and permission groups. See What Are Role-Based Permissions?.

Results

You have restored the extension integration artifacts and configuration setting of the target company instance.

5.6.4.5 Auditing and Logging Information

Here you can find a list of the events that are logged by SAP SuccessFactors Extensibility service. To retrieve the audit logs stored for SAP SuccessFactors Extensibility create a support ticket in component BC-NEO-EXT-SF.

Events written in audit logs

Event grouping	What events are logged	How to identify related log events	Additional information
Creating an integration	Start of create-integration process	• `"type":"integration"` • `"attributes":[{"name":"<object_name>","new":"<integration_details>"}` • `"status":"BEGIN"`	Event that signifies an integration of a global account with an SAP SuccessFactors system has been started. The `customDetails` attribute contains details such as the issuer of the integration token, the global account ID, the name of the SAP SuccessFactors system and the SAP SuccessFactors company ID, the SAP SuccessFactors host name, and other.
	End of create-integration process	• `"success":true` • `"type":"integration"` • `"attributes":[{"name":"<object_name>","new":"<integration_details>"}` • `"status":"END"`	Event that signifies an integration of a global account with an SAP SuccessFactors system has been completed. The `customDetails` attribute contains details such as the issuer of the integration token, the global account ID, the name of the SAP SuccessFactors system and the SAP SuccessFactors company ID, the SAP SuccessFactors host name, and other.

Event grouping	What events are logged	How to identify related log events	Additional information
	Failure of create-integration process	• `"success":true` • `"type":"integration"` • `"attributes":[{"name":"<object_name>","new":"<integration_details>"}` • `"status":"END"`	Event that signifies an integration of a global account with an SAP SuccessFactors system could not be completed. The `customDetails` attribute contains details such as the issuer of the integration token, the global account ID, the name of the SAP SuccessFactors system and the SAP SuccessFactors company ID, the SAP SuccessFactors host name, and other.
	Start of delete-integration	• `"type":"integration"` • `"attributes":[{"name":"<object_name>","old":"<integration_details>"}` • `"status":"BEGIN"`	Event that signifies a deletion of the integration of a global account with an SAP SuccessFactors system has been triggered. The `customDetails` attribute contains details such as the issuer of the integration token, the global account ID, the name of the SAP SuccessFactors system and the SAP SuccessFactors company ID, the SAP SuccessFactors host name, and other.

Event grouping	What events are logged	How to identify related log events	Additional information
	End of delete-integration	• `"success":true` • `"type":"integration"` • `"attributes":[{"name":"<object_name>","old":"<integration_details>"}` • `"status":"END"`	Event that signifies a deletion of the integration of a global account with an SAP SuccessFactors system has been completed. The `customDetails` attribute contains details such as the issuer of the integration token, the global account ID, the name of the SAP SuccessFactors system and the SAP SuccessFactors company ID, the SAP SuccessFactors host name, and other.
	Failure of delete-integration	• `"success":false` • `"type":"integration"` • `"attributes":[{"name":"<object_name>","old":"<integration_details>"}` • `"status":"END"`	Event that signifies a deletion of the integration of a global account with an SAP SuccessFactors system could not be completed. The `customDetails` attribute contains details such as the issuer of the integration token, the global account ID, the name of the SAP SuccessFactors system and the SAP SuccessFactors company ID, the SAP SuccessFactors host name, and other.
Setting up the connectivity	Start of create-inbound-connection process	• `"type":"inboundConnection"` • `"attributes":[{"name":<object_name>,"new":"<connection_details>"` • `"status":"BEGIN"`	Event that signifies a creation of a service instance of the SAP SuccessFactors Extensibility service has been triggered. The `customDetails` attribute contains details such as the subaccount ID, the global account ID, SAP SuccessFactors host name, and other.

Event grouping	What events are logged	How to identify related log events	Additional information
	End of create-inbound-connection process	• `"type":"inboundConnection"` • `"attributes":[{"name":<object_name>,"new":"<connection_details>"` • `"status":"END"`	Event that signifies a creation of a service instance of the SAP SuccessFactors Extensibility service has been completed. The `customDetails` attribute contains details such as the subaccount ID, the global account ID, SAP SuccessFactors host name, and other.
	Start of delete-inbound-connection	• `"type":"inboundConnection"` • `"attributes":[{"name":<object_name>,"old":"<connection_details>"` • `"status":"BEGIN"`	Event that signifies the deletion of a service instance of the SAP SuccessFactors Extensibility service has been started. The `customDetails` attribute contains details such as the subaccount ID, the global account ID, SAP SuccessFactors host name, and other.
	End of delete-inbound-connection	• `"success":true` • `"type":"inboundConnection"` • `"attributes":[{"name":<object_name>,"old":"<connection_details>"` • `"status":"END"`	Event that signifies the deletion of a service instance of the SAP SuccessFactors Extensibility service has been completed. The `customDetails` attribute contains details such as the subaccount ID, the global account ID, SAP SuccessFactors host name, and other.

Event grouping	What events are logged	How to identify related log events	Additional information
	Failure of delete-inbound-connection	• `"success":false` • `"type":"inboundConnection"` • `"attributes":[{"name":<object_name>,"old":"<connection_details>"` • `"status":"END"`	Event that signifies the deletion of a service instance of the SAP SuccessFactors Extensibility service could not be completed. The `customDetails` attribute contains details such as the subaccount ID, the global account ID, SAP SuccessFactors host name, and other.

The following information is described in the table columns:

- *Event grouping* - Events that are logged with a similar format or are related to the same entities.
- *What events are logged* - Description of the security or data protection and privacy related event that is logged.
- *How to identify related log events* - Search criteria or key words, that are specific for a log event that is created along with the logged event.
- *Additional information* - Any related information that can be helpful.

Related Information

Audit Logging in the Cloud Foundry Environment

5.6.4.6 Configuring Backup

You can back up and restore the destination and the trust configuration settings for the integration between the SAP SuccessFactors system and the global account in SAP BTP.

You can back up the following configuration settings:

- The HTTP destination configured during the creation of the SAP SuccessFactors Extensibility service instance. For more, see Export Destinations.
- Your subaccount X.509 certificates. For more information, see Set up Trust Between Systems
- Your SAP SuccessFactors IdP SAML 2.0 metadata.
  To download your SAP SuccessFactors IdP SAML 2.0 metadata, in the SAP BTP cockpit, navigate to *Connectivity* > *Destinations* > *Download IdP Metadata*.
- Your identity provider configurations.
  You can back up identity provider configurations with the Identity Provider Management API. For more information see: Identity Provider Management API.

5.6.4.7 Accessibility Features in SAP SuccessFactors Extensibility Service

To optimize your experience of SAP SuccessFactors Extensibility service, SAP Business Technology Platform (SAP BTP) provides features and settings that help you use the software efficiently.

> **Note**
>
> SAP SuccessFactors Extensibility service runs on the SAP BTP cockpit. For this reason, accessibility features for SAP BTP cockpit also apply. See the accessibility documentation for SAP BTP cockpit on SAP Help Portal at Accessibility Features in SAP BTP cockpit [page 2459].

For more information on screen reader support and keyboard shortcuts, see Accessibility for End Users.

5.6.4.8 Troubleshooting for SAP SuccessFactors Extensibility Service

Information about troubleshooting issues you might experience when extending SAP SuccessFactors on SAP BTP using the SAP SuccessFactors Extensibility service.

Cloud Foundry Environment

These are some issues you might experience when extending SAP SuccessFactors on SAP BTP, Cloud Foundry environment:

- My SAP SuccessFactors OData Destination Does Not Work [page 2452]
- My SAP SuccessFactors Extensibility Service Instance Failed to Create and It Didn't Say Why [page 2453]

5.6.4.8.1 My SAP SuccessFactors OData Destination Does Not Work

SAP SuccessFactors OData destination does not work.

Issue/Symptom

You are extending SAP SuccessFactors on SAP BTP, Cloud Foundry environment using the SAP SuccessFactors Extensibility service. You are trying to configure the integration between SAP SuccessFactors and SAP Business Technology Platform (SAP BTP) and your SAP SuccessFactors OData destination does not work.

Solution

To fix your SAP SuccessFactors OData destination, you can try the following:

- Make sure that the names of the destination properties, and their values match the names described in Create an HTTP Destination in SAP BTP.
- Make sure that the values of the *Client Key* and *apiKey* properties match the *API Key* value specified in the SAP SuccessFactors OAuth client.
- Make sure that the X509 certificate specified in the SAP SuccessFactors OAuth client is the same as the one you have in the Cloud Foundry environment. See Configuring the Extension Application's Connectivity to SAP SuccessFactors.
- Make sure that you have the *uaa.user* authorization scope defined in the *xs-security.json* file. See:
  - Update a Service Instance
  - UAA Scopes
- Make sure that the *uaa.user* authorization scope is defined in the role template in the *xs-security.json* file. See Accessing Business Service Data.
- Make sure you consume the destination correctly, as described in Consume the Destination.
- Make sure that you use the *user_token* OAuth grant for retrieving the destination. See OAuth User Token Exchange Authentication.
- Make sure that the name of the destination that you use in your application matches the name of the one created in the SAP BTP cockpit.
- Note that if you have a destination defined as an environment variable, it will overrule the one created in the cockpit.
- Check the logs in Kibana. If you do not have the application logging service, you can add it. See Access and Analyze Application Logs and Container Metrics.
- Make sure that the name of the destination is the same as the name of the SAP SuccessFactors Extensibility service instance you have created. See Access and Analyze Application Logs, Container Metrics and Custom Metrics.

5.6.4.8.2 My SAP SuccessFactors Extensibility Service Instance Failed to Create and It Didn't Say Why

My SAP SAP SuccessFactors extensibility service instance failed to create and it didn't say why.

Issue/Symptom

You are trying to create an SAP SuccessFactors Extensibility service instance but the creation fails without an error message.

Solution

This is what you can do:

- You can use the Cloud Foundry command line interface to get the error message by executing the command:

```
cf service <service_instance_name>
```

Check for any error messages there. See Managing Service Instances .

- Can you reach the SAP SuccessFactors company? Check that the tenant is not in maintenance or being updated.
- Check that there is not an already existing destination with the same name as the SAP SuccessFactors Extensibility service instance you are creating. When creating this service instance, a destination with the same name will be automatically created. See Create an SAP SuccessFactors Extensibility Service Instance in the Cloud Foundry Environment.

5.6.5 Extending SAP Customer Experience Products in the Kyma Environment

You can configure the integration between SAP BTP and SAP Customer Experience automatically to extend SAP Customer Experience products with applications running on the cloud platform.

Overview

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and Government Cloud (US) regions.

SAP BTP offers a standard way for extending SAP solutions.

You can extend SAP Customer Experience products in SAP BTP, Kyma runtime without disrupting the performance and the core processes. You can build extension applications for SAP Commerce Cloud, SAP Field Management, and SAP Cloud for Customer. When building extension applications, you can also benefit from the automation of the integration between the cloud platform and SAP Customer Experience.

Process Flow

To integrate SAP BTP and SAP Customer Experience products so that you can build extension applications, you need to perform the following tasks:

Integrating SAP BTP and SAP Customer Experience Products

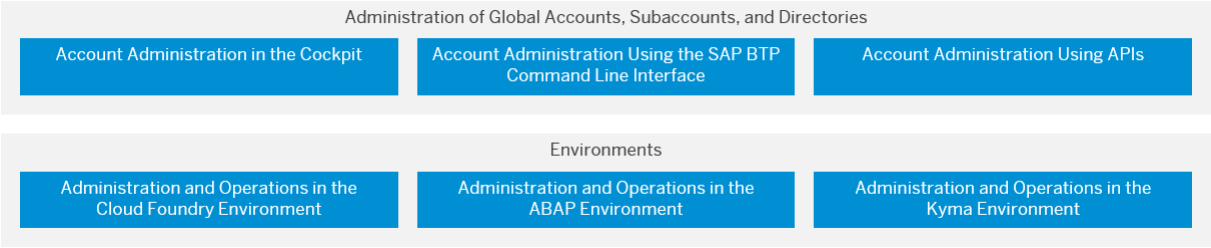
Process Step	Related Documentation
1. Add the Application Connector module, and connect the SAP Customer Experience system you want to extend with the corresponding global account in SAP BTP. During the registration process you create an integration token which is then used in the SAP Customer Experience system for configuring the integration on the SAP Customer Experience side.	Register an SAP Customer Experience System [page 2316]
2. Configure the integration on SAP Customer Experience side. To do so, you pair the integration token with your SAP Customer Experience system.	• For an SAP Commerce Cloud system, see **steps 2-7** in Retrieving Client Certificate. • For an SAP Field Service Management system, see SAP BTP Extensions. • For an SAP Cloud for Customer system, see *SAP BTP Extensions* > *Step 2 Configuration in SAP Cloud for Customer Event Notifications* in Configure an Event Notification .
3. Assign the SAP Customer Experience system to a formation to enable the API access to the corresponding SAP Customer Experience product's APIs.	Automating Integrations Using Formations [page 2326]
4.Call a registered external service using Central Application Gateway URL.	Call a registered external service from Kyma

Related Information

Getting Started in the Kyma Environment [page 247]
Development in the Kyma Environment [page 1882]
Administration and Operations in the Kyma Environment [page 3450]

6 Administration and Operations

Administration and operation tasks for SAP BTP include management and configuration of global accounts and subaccounts as well as the operation of applications.



- Account Administration Using the SAP BTP Command Line Interface (btp CLI) [page 2664]
- Account Administration in the Cockpit [page 2458]
- Administration and Operations in the Cloud Foundry Environment [page 2779]
- Administration and Operations in the ABAP Environment [page 2914]
- Administration and Operations in the Kyma Environment [page 3450]
- Account Administration Using APIs [page 2729]

Administration of Global Accounts, Directories, and Subaccounts

Some of the typical responsibilities of SAP BTP administrators include maintenance of global accounts, management of directories and subaccounts, and configuration of entitlements. To do this, you can use either the SAP BTP cockpit, the SAP BTP command line interface, or APIs.

- Account Administration in the Cockpit [page 2458]
- Account Administration Using the SAP BTP Command Line Interface (btp CLI) [page 2664]
- Account Administration Using APIs [page 2729]

Administration and Operations in Different Environments

Environments constitute the actual platform-as-a-service offering of SAP BTP that allows for the development and administration of business applications. Each environment offers different approaches to manage administration and operations. For more information, see Environments [page 62].

Administration and Operations in the Cloud Foundry Environment

In the Cloud Foundry environment, you can manage orgs, spaces, and space quota plans using the SAP BTP cockpit or the SAP BTP command line interface.

Learn more about the different application operations that you can perform in the Cloud Foundry environment.

- Org Administration Using the Cockpit [page 2792]
- Org Administration Using the Cloud Foundry CLI [page 2812]
- Application Operations in the Cloud Foundry Environment [page 2860]
- Audit Logging in the Cloud Foundry Environment [page 2878]

Administration and Operations in the ABAP Environment

When using the ABAP environment, the account administration tasks are performed in the Cloud Foundry environment.

The application operations are specific to the ABAP environment. For more information, see SAP Fiori Apps in the ABAP Environment [page 2916].

Administration and Operations in the Kyma Environment

The administrators of the Kyma environment take care of setting it up and make sure it's ready for developers to work with.

- Assign Roles in the Kyma Environment [page 3666]
- Creating Kyma Instances [page 3452]
- Kyma Environment Backup [page 3516]

Related Information

Account Model [page 121]

6.1 Account Administration

Learn how to manage global accounts, directories, and subaccounts on SAP BTP using different tools.

- Account Administration in the Cockpit [page 2458]
- Account Administration Using the SAP BTP Command Line Interface (btp CLI) [page 2664]
- Account Administration Using Joule in the Cockpit [page 2658]
- Account Administration Using APIs [page 2729]
- Account Administration Using Infrastructure as Code [page 2777]

Once your account model is set up, you can move on to managing the different environments you plan to use:

- Administration and Operations in the Cloud Foundry Environment [page 2779]
- Administration and Operations in the ABAP Environment [page 2914]
- Administration and Operations in the Kyma Environment [page 3450]

Related Information

Account Model [page 121]
Entitlements and Quotas [page 127]
User and Member Management [page 133]
Environments [page 62]

6.1.1 Account Administration in the Cockpit

Learn about frequent administrative tasks you can perform using the SAP BTP cockpit, such as managing global accounts, directories, subaccounts, entitlements, and members.

Access the Cockpit [page 2459]
: Learn how to access the SAP BTP cockpit.

In-App Onboarding and User Analytics with WalkMe [page 2460]
: WalkMe, the AI-powered Digital Adoption Platform (DAP), is a digital layer added on top of Web sites or enterprise applications to guide users through workflows, automate actions, and monitor user engagement with the content.

Navigate in the Cockpit [page 2461]
: Learn how to navigate to your global accounts, directories, and subaccounts in the SAP BTP cockpit.

Managing Global Accounts Using the Cockpit [page 2464]
: Your SAP BTP global account is the entry point for managing the resources, landscape, and entitlements for your departments and projects in a self-service manner.

Managing Directories Using the Cockpit [page 2501]
: Learn how to organize and manage your subaccounts according to your technical and business needs by using directories in the SAP BTP cockpit.

Managing Subaccounts Using the Cockpit [page 2509]
: Learn how to structure a global account according to your organization's and project's requirements with regard to members, authorizations, and entitlements by managing subaccounts.

Managing Entitlements and Quotas Using the Cockpit [page 2521]
: When you purchase an enterprise account, you are entitled to use a specific set of resources, such as the amount of memory that can be allocated to your applications.

Subscribe to Multitenant Applications Using the Cockpit [page 2534]
: Subscribe to multitenant applications from the *Subscriptions* page in the SAP BTP cockpit.

Security Administration: Managing Authentication and Authorization [page 2537]
: This section describes the tasks of administrators of SAP BTP. Administrators ensure user authentication and assign authorization information to users and user groups.

Related Information

Account Model [page 121]

6.1.1.1 Access the Cockpit

Learn how to access the SAP BTP cockpit.

Context

Navigate to https://cockpit.btp.cloud.sap to access the cockpit. Depending on your geographic location, this URL redirects you to the closest regional cockpit URL.

> → Tip
>
> Use the `idp` URL parameter to choose which identity provider with which you want to log on.
>
> For example: `https://cockpit.btp.cloud.sap?idp=mytenant.accounts.ondemand.com`
>
> For more information, see Log On with a Custom Identity Provider to the SAP BTP cockpit [page 2591].

Accessing the cockpit from China

Users accessing SAP BTP cockpit from China must use a dedicated regional entry point.

Use the following URL to access the cockpit: https://cockpit.cn40.platform.sapcloud.cn

You will receive an email from **ias-notification@sapcloudplatform.cndatacom.com** to activate your user before logging in.

6.1.1.1.1 Accessibility Features in SAP BTP cockpit

To optimize your experience of the SAP Business Technology Platform (SAP BTP), SAP BTP cockpit provides features and settings that help you use the software efficiently. Learn more about these features and settings and find out how to use them.

> ⓘ Note
>
> SAP BTP cockpit is based on SAPUI5. For this reason, accessibility features for SAPUI5 also apply. See the accessibility documentation for SAPUI5 on SAP Help Portal at Accessibility for End Users.

For more information on screen reader support and keyboard shortcuts, see Screen-Reader Support for SAPUI5 Controls and Keyboard Handling for SAPUI5 Elements.

Additionally, SAP BTP cockpit offers a high contrast black theme. See Use High Contrast Black Theme [page 2460].

Use High Contrast Black Theme

Context

This feature is recommended for users with need for high contrast themes.

Steps

1. In the header toolbar choose *Your Name* > *Settings*.
2. Select the *High Contrast Black* theme and click *Apply*.
3. Once you've saved your changes, the cockpit starts with the theme of your choice.

6.1.1.2 In-App Onboarding and User Analytics with WalkMe

WalkMe, the AI-powered Digital Adoption Platform (DAP), is a digital layer added on top of Web sites or enterprise applications to guide users through workflows, automate actions, and monitor user engagement with the content.

> **Note**
>
> The content in this section is not relevant for China (Shanghai) and SAP Souvereign Cloud regions.

The SAP BTP cockpit offers in-app onboarding tours and contextual information, powered by WalkMe, for free. The content is developed and managed by SAP.

For more information about WalkMe, see WalkMe Digital Adoption.

Language Support

The WalkMe walk-thrus are available in English only at the current time. If you're using the SAP BTP cockpit in another language, you'll see SAP Companion static in-app help only.

WalkMe End User Experience

When you launch the SAP BTP cockpit for the first time, a welcome screen offers you options to follow a walk-thru to get a global understanding of the SAP BTP cockpit or to follow available tours that guide the user through key tasks. You can choose your preferred option or simply close the welcome screen.

You can view the list of available walk-thrus by clicking the *Question Mark* button in the banner at the top of the page, and then select the tour that maps to a task you want to accomplish.

WalkMe and SAP Companion

WalkMe walk-thrus replace the former SAP Companion guided tours in the SAP BTP cockpit. SAP Companion static in-app help is accessible via the WalkMe menu.

WalkMe Analytics

WalkMe analytics monitor the usage of WalkMe content. This information is used by SAP to improve the WalkMe content.

Enabling WalkMe analytics by a global account administrator is a prerequisite for collecting WalkMe analytics data.

> **Note**
>
> Data sent to SAP is anonymized. SAP does not collect personal, user-specific, company identifying data, or user input.

6.1.1.3 Navigate in the Cockpit

Learn how to navigate to your global accounts, directories, and subaccounts in the SAP BTP cockpit.

Prerequisites

- Sign up for an enterprise or a trial account and receive your logon data. For more information, see Try Out SAP BTP for Free [page 162] or Sign up for a Customer Account [page 163].

Procedure

1. Navigate to a global account.
   - **You have only one global account:**
     When you log on to the cockpit you're automatically taken into your global account, to the *Account Explorer* page.
   - **You have multiple global accounts:**
     When you log on to the cockpit, a dialog opens up containing the list of all global accounts that you're part of. In this dialog, select the global account you want to navigate to.

> → Tip
>
> If you have one global account you usually navigate to, you can choose *Remember my selection* in the initial dialog. This means you'll be automatically redirected to your preferred global account after logging on, without seeing the dialog with all the options each time.
>
> If you've chosen a default global account, you can change or remove it anytime. To do so, navigate to the *Account Explorer* page of any global account, choose *Switch Global Account*.
>
> To change it, choose the desired new default global account from the list, select *Save new selection as default global account* and choose *Continue*. Your new default is savedand you're redirected to that global account.
>
> To delete the default global account and go back to seeing the selection dialog after each logon, simply choose the ⊗ icon next to your default global account name in the dialog and choose *Close*.

You can see which global account you are in at any time by looking at the first item in the breadcrumbs. It looks like this: 🌐 *<global account name>*

2. Navigate to a different global account.
   - Navigate to the *Account Explorer* page at global account level and choose *Switch Global Account*.
   - Use the dropdown menu next to the 🌐 *<global account name>*.
3. (Optional) Navigate to directories.
   a. When you enter your global account, you are by default taken to the *Account Explorer* page of that global account. To navigate to a directory, choose the corresponding entry in one of the views in the *Directories & Subaccounts* tab.

   > ⓘ Note
   >
   > If a directory has the **user management** capability enabled, then you can access this directory (or its subdirectories) only if you are a global account admin or you are assigned as a member of the directory. See Manage Users in Directories [page 2505].
   >
   > The 🔒 *(Locked)* icon is shown next to directories that you aren't authorized to access.

4. Navigate to subaccounts.
   a. When you enter your global account, you are by default taken to the *Account Explorer* page of that global account. To navigate to a subaccount, choose the corresponding entry from one of the views.

   Once you've entered a subaccount, the breadcrumbs look like this: 🌐 *<global account name>* / 🖧 *<subaccount name>*

   > ⓘ Note
   >
   > To access a subaccount your user must either have authorizations in the subaccount tenant or, in the case of a subaccount that has the Cloud Foundry environment enabled, have authorizations in the Cloud Foundry org or space of the subaccount. See About User Management in the Cloud Foundry Environment [page 2787].
   >
   > The 🔒 *(Locked)* icon is shown next to subaccounts that you aren't authorized to access.

6.1.1.3.1 Personalizing the Home Screen

Tailor the home screen of the SAP BTP cockpit to your needs by choosing which cards are displayed and removing the ones you do not use.

Context

The home screen is the page you see after logging on to your global account. Its content is organized in cards, such as *Favorites*, *Subaccount Quick Access*, and *What's New for the Services*. You can personalize the home screen so that it shows only the cards that are relevant to you.

Personalization is done in an edit mode that you enter from the home screen. In edit mode you can remove individual cards directly, or open a dialog to show or hide any of the available cards. Your changes apply only to your own view and take effect after you save them.

Procedure

1. On the home screen, choose *Edit Home*.

   The home screen switches to edit mode. Each card shows a remove icon in its top-right corner, and *Save* and *Cancel* buttons appear.
2. To remove a card from the home screen, choose the *Remove from Home* icon (*X*) in the top-right corner of that card.
3. To choose which cards are displayed, choose *Edit Cards*.

   The *Edit Cards* dialog opens, listing all available cards, each with a toggle switch:

   - *Frequently Used Instances and Subscriptions*
   - *Subaccount Quick Access*
   - *Favorites*
   - *Information Resources*
   - *What's New for the Services*
   - *SAP Community*
   - *Helpful Cockpit Hints and News*
4. In the dialog, use the toggle switch next to each card to show or hide it, and then choose *Save*.

   Choose *Cancel* to close the dialog without changing which cards are shown.
5. Choose *Save* to apply your changes to the home screen, or *Cancel* to discard them.

Results

The home screen now displays only the cards you selected. You can return to edit mode at any time by choosing *Edit Home* to further adjust your personalization.

6.1.1.4 Managing Global Accounts Using the Cockpit

Your SAP BTP global account is the entry point for managing the resources, landscape, and entitlements for your departments and projects in a self-service manner.

Global Account Settings

As a global account administrator, you can access the global account settings by clicking ⚙ *(Global Account Settings)* in the SAP BTP cockpit.

In the *General* tab of the global account settings, you can obtain general information about your global account, including your global account ID, global account name, account type, payment models, contract status, global account subdomain, geographic access restrictions, and when your account was created.

> **Note**
>
> In this tab, you cannot change any of the values. The only value that you can edit is the global account name, and you can do this in the *Account Explorer* page (see Change the Display Name of Your Global Account [page 2466]).

In the *Subaccounts* tab in the global account settings, you can

- Define default settings that are used when creating a new subaccount in this global account.
  You can set the supported providers, the default provider, and the default region.
- Set the option to enable the force deletion of subaccounts.
  If this option is selected, all global account admins can force delete subaccounts that are not marked as "used for production". The force deletion of a subaccount permanently deletes all its data, including applications, service instances, spaces, active subscriptions, and members. The force deletion is permanent and irreversible. This option also enables the force deletion of subaccounts using the btp CLI or the SAP Cloud Management APIs. Subaccounts that are marked as "used for production" cannot be force deleted. When the force deletion of a subaccount has been requested, a seven-day waiting period begins during which access to the subaccount and its services is partially blocked and the subaccount is marked as *Pending Deletion*. During this waiting period, you can still cancel the scheduled deletion. After the waiting period, the subaccount and all its data will be permanently deleted.

Related Information

Setting Up Your Account Model
Managing Directories Using the Cockpit [page 2501]
Managing Subaccounts Using the Cockpit [page 2509]

6.1.1.4.1 Log On to Your Global Account

Use the SAP BTP cockpit to log on to your global account and start working in SAP BTP.

Prerequisites

You have a user that is a global account administrator.

If you don't have a global account administrator or one isn't available, there is a workaround using SAP for Me.

For more information, see 2669325 - How to add a user as a Global Account administrator for SAP Business Technology Platform.

Context

Procedure

1. Access the SAP BTP cockpit.

   For more information, see Access the Cockpit [page 2459].
2. Choose a global account.

Results

The *Account Explorer* page for that global account opens.

Next Steps

- Track and monitor usage of services in your global account:
  - Use the filters to specify which usage information to display in the tables and monthly usage charts. The *Period* filter is applied to the chart display.
  - Some rounding or shortening is applied to large values. Mouse over values in the table to view the exact values in the tooltips.
  - Choose a row in the table to view its historic information in the *Monthly Usage* chart. To display a larger view of a chart, choose the *(Zoom)* button.
  - To download the information in the *Overview* page to a Microsoft Excel file, choose the *Export* button.

- Access your subaccounts and create new subaccounts.
  See Create a Subaccount [page 2509].

Related Information

Account Model [page 121]
Global Accounts [page 121]
Create a Subaccount [page 2509]

6.1.1.4.2 Change the Display Name of Your Global Account

Change the display name for the global account using the SAP BTP cockpit.

Prerequisites

You are an admin of the global account that you'd like to edit.

Context

> → Tip
>
> You can also change the account's display name using the SAP BTP command line interface (btp CLI) with the `btp update accounts/global-account` command. See Account Administration Using the SAP BTP Command Line Interface (btp CLI) [page 2664].

Procedure

1. Open your global account in the SAP BTP cockpit.
2. Navigate to the *Account Explorer* page.
3. In the *Directories & Subaccounts* view, locate the first entry in your account structure, which is your global account, and click ••• *(Actions)*. Then select *Edit*.
4. In the *Edit Global Account* dialog box, enter the new human-readable name for the global account.
5. Save your changes.

Related Information

Account Model [page 121]

6.1.1.4.3 Add Members to Your Global Account

Add users as global account members using the SAP BTP cockpit.

Prerequisites

- You must be a global account administrator to add other global account members.
  If you don't have a global account administrator or one isn't available, there is a workaround using SAP for Me.
  For more information, see 2669325 - How to add a user as a Global Account administrator for SAP Business Technology Platform.
- The users exist in a trusted platform identity provider.
  All users of SAP BTP are stored in identity providers, either in the default or in a custom identity provider. SAP BTP needs a copy of the user, sometimes called a shadow user. You assign the shadow user authorizations to access resources in SAP BTP. When a user authenticates, SAP BTP forwards the request to the identity provider.
  For more information, see Trust and Federation with Identity Providers [page 2538].

Context

Assign predefined or custom role collections to users who need to manage or view the global account in SAP BTP cockpit. Examples of predefined role collections include the following:

- *Global Account Administrator*
- *Global Account Viewer*

For more information about these role collections, see Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138].

Procedure

1. Navigate to your global account.
2. Add a user to your global account.

   For more information, see Create Users [page 2608].

> ⓘ Note
>
> If the user is already a user in a subaccount and has logged on to SAP BTP cockpit, SAP BTP adds the user at the global account level automatically, but without any role collections.

3. Assign a role collection to the user.

   For more information, see Assign Users to Role Collections [page 2617].

Results

The next time this user logs on to the SAP BTP cockpit, the user can access this global account.

Related Information

User and Member Management [page 133]

6.1.1.4.4 Managing Resource Providers

SAP BTP allows you to connect your global account in the SAP BTP cockpit to your provider account from a non-SAP cloud vendor, and consume remote service resources that you already own and which are supported by SAP through this channel.

> ⓘ Note
>
> The use of this functionality is subject to the availability of the supported non-SAP cloud vendors in your country or region.

Context

For example, if you are subscribed to Amazon Web Services (AWS) and have already purchased services, such as PostgreSQL, you can register the vendor as a resource provider in SAP BTP and consume this service across your subaccounts together with other services offered by SAP.

SAP BTP currently supports the following vendors and their consumable services:

Cloud Vendor	Supported Services
Amazon Web Services	Amazon Relational Database Service (RDS) - PostgreSQL
Microsoft Azure	Azure Database for PostgreSQL

More information: PostgreSQL on SAP BTP - Product Documentation

Configure a New Resource Provider

Context

To consume the resources provisioned by your provider account, you need to first create a resource provider instance in the cockpit.

Procedure

1. Navigate to your global account in the cockpit.
2. Choose *Resource Providers* from the left hand-side navigation panel.
3. Choose *New Provider*.
4. Choose the provider from the list of supported vendors.
5. Enter a display name for the provider.

   You can create more than one instance of a given resource provider, each with its unique configuration properties. In such cases, the display name (and technical name) should be descriptive enough so that you and developers can easily differentiate between each instance.
6. Enter a unique technical name for the provider.

   The technical name can contain only letters (a-z), digits (0-9), and underscore (_) characters.

   This name can be used by developers as a parameter when creating service instances from this provider.

   > ### Note
   >
   > After you save the settings for the resource provider, you cannot change its technical name.
7. Choose either *Manually enter provider configuration properties* or *Add provider configuration properties by JSON file*, depending on how you prefer to provide the necessary configuration properties for the given provider.

   > ### → Tip
   >
   > Each supported vendor has its own unique configuration properties. The *New Resource Provider* form provides a description of each property and indicates whether they are mandatory or optional.

   If you choose to configure the properties by uploading a JSON file, which is typically provided by the vendor, make sure that the file contains the required configuration properties for connecting to the provider, and is correctly formatted.

   > ### {·} Sample Code
   >
   > Sample JSON for configuring Amazon Web Services (AWS) as a resource provider:
   >
   > ```
   > {
   >    "access_key_id": "AWSACCESSKEY",
   >    "secret_access_key": "SECRETKEY",
   >    "vpc_id": "vpc-test",
   >    "region": "eu-central-1"
   > ```

```
}
```

8. Save your changes.

   The new resource provider is added to the resource providers table.

Results

After you configure a new resource provider, its supported services are added as entitlements in your global account. In the *Entitlements* page in the cockpit, you can then allocate the required services and quotas to the relevant directories and subaccounts in your global account.

Manage Resource Providers and Service Entitlements

Context

Follow these steps to manage the resource providers that you have already configured in the cockpit.

Procedure

1. Navigate to your global account in the cockpit.
2. Open the *Resource Providers* page.

   The page displays the resources providers that you have already configured.
3. In the *Actions* columns, you can perform the following for each resource provider:

Action	Description
Manage Entitlements and Quotas	View the subaccount entitlements and quotas of services that are consumed from a resource provider. This action opens the *Entitlements* > *Service Assignments* page in the cockpit, and is prefiltered for the selected resource provider. → Tip Whenever you need to manage assigned entitlements, you can navigate directly to the *Service Assignments* or *Subaccount Assignments* pages in the cockpit without going through the *Resource Providers* page.
Edit Resource Provider	Edit the display name, description, and configuration properties of a resource provider.

Action	Description
	Note You cannot change the technical name of a resource provider.
 Delete	Delete a resource provider. When you delete an existing resource provider, all the services offered by this instance of the resource provider will no longer be available on SAP BTP. **Note** You cannot delete a resource provider that already has service entitlements assigned to subaccounts in your global account. To delete the provider, you need to first remove its subaccount assignments in the *Subaccount Assignments* page.

Related Information

Configure Entitlements and Quotas for Subaccounts [page 2529]

6.1.1.4.5 Monitoring Usage and Consumption Costs in Your Global Account

SAP BTP cockpit supports advanced usage and cost monitoring of services in your global account. You can compare the usage and costs of multiple services and subaccounts, see monthly trends, and drill down into subaccounts and service plans for detailed information.

Note

- The use of the consumption-based commercial model is subject to its availability in your country or region.
- If your global account uses only the subscription-based commercial model, then refer only to information described here about usage data and the *Usage* tab. You can ignore any information in this topic about costs, cloud credits, and the *Overview* and *Billing* tabs.

The *Costs and Usage* page in the cockpit provides a central and flexible experience for the following cost control flows:

- Continuously monitor and analyze the monthly usage and costs of services consumed by subaccounts and directories in your global account.
- Help you to verify your monthly billing statement against actual resource consumption.
- Help you to verify periodic cross-charges between your various business units based on subaccount and directory allocation/distribution.

Check out also this blog for more information.

The *Costs and Usage* page uses the same terms that are printed in your monthly balance statement, which facilitates better contract-to-billing traceability and verification for cost controllers. Get your latest balance statement from SAP for Me.

→ Remember

- The monthly balance statement, which is provided separately, contains legally binding information regarding your monthly costs. Details about costs on the *Costs and Usage* page in the cockpit are provided for informational purposes only. Any discrepancy between the information displayed in the cockpit and the information in your balance statement will be resolved in favor of the balance statement.
- Costs are displayed according to your contract currency.
- Global accounts are the only contractual billable entity for SAP BTP. Directories and subaccounts are used as structural entities in global accounts. The usage and cost data displayed for directories and subaccounts are estimations and may differ from the actual global account metrics. Hence, you should use their data only for internal cost estimations.
  The relative calculation per billable usage within each subaccount is an estimation only as it is based on certain measures, which in some cases can either be different from the metrics that are presented on the global account level, or that use different formulas than the ones used for billing.
- Cloud credit information and monthly costs apply to all regions used by the subaccounts in your global account. Usage and cost data is updated after your monthly balance statement has been generated.
- Usage data is updated after your monthly balance statement has been generated. For new global accounts, data is updated with the first billing cycle.
- Prices are listed as estimates if a consumable item has not yet been billed.
- In the cost and usage charts, estimated values are used for the period between the last balance statement and the current date. These are displayed as striped bars.
  These estimates are based on resource usage values before computation for billing and might change after the next balance statement is issued. The estimated values are not projected or forecast values.
- Some SAP BTP services report their billing and usage at the global account level, and not at the subaccount level. Such services are listed in the *Costs and Usage* page under a reserved technical entity named `REPORTED-AT-GLOBAL-ACCOUNT-LEVEL` with the ID `DEFAULT_SA`.

Accessing the Costs and Usage Page

To access the *Costs and Usage* page, open the global account in the cockpit and choose *Costs and Usage* in the navigation area.

ⓘ Note

The *Costs and Usage* page is available only if your global account uses the consumption-based commercial model.

If your global account uses the subscription-based commercial model exclusively, then you must access the *Usage* page instead. The *Usage* page resembles the *Costs and Usage* page, except you cannot see any data relating to costs or cloud credits since your eligible services are already prepaid and you aren't charged per usage. See Using the Usage Tab [page 2484].

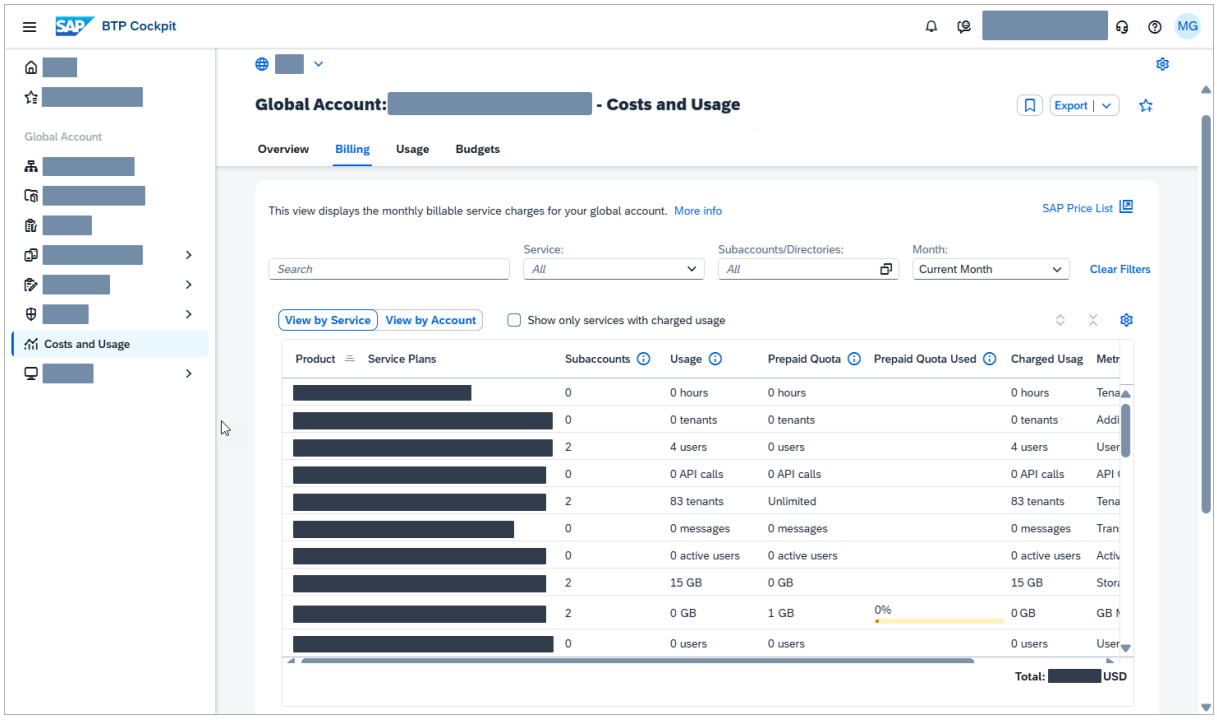
Understanding the Layout of the Costs and Usage Page

Overview, Billing, Usage, and Budget Tabs

The tab row at the top of the *Costs and Usage* page allows you to quickly navigate to the different features designed to help you monitor your usage and consumption costs across your global account.

Tab row of the Costs and Usage page

Tab	Description
Overview	Shows several cards that provide you with key insights into your monthly costs, prepaid subscription quota, cost distribution and breakdown, and cost-related recommendations. For more information, see Using the Overview Tab [page 2476].
Billing	Shows the monthly billable service charges in your global account, based on the aggregation of your resource's actual usage. The aggregation of usage from all your subaccounts is calculated according to the pricing structure and legally billable metric of each service, as mentioned in the SAP BTP Service Description Guide. **Note** This tab relevant only if your global account uses a consumption-based commercial model. If your global account uses only the subscription-based commercial model, then all your eligible services and entitled quota are prepaid. For more information, see Using the Billing Tab [page 2480].
Usage	Shows the data representing your actual non-aggregated monthly usage for services consumed in your subaccounts within your global account. This tab applies to both the consumption-based and subscription-based commercial models. For more information, see Using the Usage Tab [page 2484].
Budgets	In this tab, you can set up budget limits for your global account so that you can better manage your global account spending and plan for future consumption. Here you can also opt to have automatic e-mail alerts sent to all global account administrators to notify them when spending exceeds defined budget thresholds. For more information, see Managing Budgets in Your Global Account [page 2490].



Billing tab in the Costs and Usage page

Filter and Search Bar

At the top of the *Billing* and *Usage* tabs, you can find the filter and search bar. Apply the filters to choose which services, subaccounts, directories, or billing month you want to view in these two tabs.

You can also use the *Search* option. When you start typing, the search offers results that apply to service names, plan names, subaccount names, directory names, labels assigned to subaccounts and directories, product IDs (SKU), and metrics. The search offers suggestions only for items that are currently listed.

> → Tip
>
> If you've set up labels in your subaccounts and directories that reflect the distribution of these entities to your company structure or projects, you can easily track and manage cross-charges by entering the labels of your subaccount and directory labels in the search.

When you enter filter and search criteria in either the *Billing* or *Usage* tab, the same filters and search terms are applied to both tabs, allowing you to view consistent data across both sections more efficiently.

> → Tip
>
> To share or save a filtered tab, click the *(Bookmarks)* option in the header area. The URL of the tab and its filters is saved to your clipboard.
>
> You can then save this URL as a bookmark in your browser, or you can share it with a colleague so they can quickly open the same tab and filter settings.

General Tips for the Billing and Usage Tabs

In the following table, you can find some useful tips that are common to both the *Billing* and *Usage* tabs:

To Do This...	Do This...
Find out what each table column means	Click the ⓘ *(More Info)* buttons in the column headings, where available.
Sort the data in ascending or descending order	Click a column heading in the table and choose the sort order you want. Sorting the tables can be useful for determining which service plans and subaccounts have the highest and lowest costs.
Show or hide table columns	Click ⚙ *(Configure Table Columns)* to configure which table columns you'd like to show and hide. The tables in each tab have columns that are hidden by default. Open this setting to check if any of these columns apply to your needs.
Maximize the tables across the entire page	Collapse the navigation panel to maximize the tables across the entire page.
Adjust the width of table columns	Drag the column separators to the left or right.
Manage the custom labels of your directories and subaccounts	Choose either the *Add Labels* or the *Edit Labels* option in the ••• *(Actions)* context menu of any listed directory or subaccount in either the *Billing* or *Usage* tab. In the *View by Account* view of the *Billing* tab, these context menu options are available in the main table, while in the *View by Service* view, you need to open the details view of a service plan. Existing labels of directories and subaccounts are listed under the *Labels* columns in the *Billing* or *Usage* tabs. You can hide this column if you have no use for it. → Remember You can also add, remove, and edit the labels of your directories and subaccounts on the *Account Explorer* page. Adding custom labels to your directories and subaccounts is optional, but they can help you narrow down cost results by focusing on specific labels in the *Search* fields. For more information about labels, see Labels [page 125].

Using the Overview Tab

The *Overview* tab is where you can get key insights into your monthly costs, prepaid subscription quota, cost distribution and breakdown, and cost-related recommendations in the following cards:

Card	What Information is Displayed
Monthly Cost Summary	Shows key indicators of your global account's monthly consumption costs over the past 12 months, including: • Estimated costs so far this month (based on raw usage data before computation for billing; values may update after your next balance statement) • Forecasted total cost for the current month • Total cost for the previous month • Months with the highest and lowest costs • Average monthly cost **Note** The available indicators in this card depend on which consumption-based model is used by your global account. For example, the *Total forecasted cost for current month* indicator is not available for the Pay-As-You-Go for SAP BTP model. Some indicators also require a minimum amount of historical data.

Card	What Information is Displayed
SAP BTP Enterprise Agreement	Shows information relating to your cloud-credit usage and costs per month relative to your total cloud credits for the current cloud credits period. It also shows your monthly trend of cloud-credit usage and costs. This card is displayed only if your global account uses the SAP BTPEA flavor of the consumption-based commercial model. In this card, you'll see warnings if you're approaching or exceeded your cloud credit limit for the current cloud credits period. Any overages are billed at list price, so make sure to contact SAP if you need more cloud credits. → Tip To allow you to better manage your global account spending and plan for future consumption, turn on the *Show forecast* option. When turned on, you can view a projection of your anticipated cloud-credit costs and usage for the next 6 months or the end of current contract period, whichever is shorter. For more information about this option, see Estimated and Forecasted Data [page 2486]. Note the following: • Cloud-credit usage and cost information is displayed only for the current cloud credits period. The total contract duration is split into cloud credit periods (typically one year each) and the total cloud credits are divided between these periods. • Cost information applies to all regions used by the subaccounts in your global account. Data is updated after your monthly balance statement has been generated. • Your cloud credit balance is calculated each month by deducting the corresponding costs of all SAP BTP services for the previous month. • If your global account has received a cloud-credit refund at any time during the current cloud credits period, you may see a difference between your total costs and the monthly costs in the chart. • If you've previously over-consumed your available cloud credits and then purchased additional credits, you may see a shift in your monthly cloud credits cost graph from overage to within limits. This change indicates that the extra credits have balanced out any previous overages, and it is an expected part of managing your cloud credits usage. • If your global account uses both consumption-based and subscription-based models, this card only displays usage charged under the consumption-based model. For services included in your subscription, only usage beyond your prepaid quota is shown here and is billed per your contract's consumption-based terms. For instance, if your subscription covers 100 site visits and you have 151, this card reflects data for the extra 51 visits that exceed your subscription-based quota. → Tip To see more details about your contract, cloud credit or cost details, click *(Expand)*.

Card	What Information is Displayed
	→ Tip To extract the data to a spreadsheet document, use the **Export** option. For more information, see Exporting Usage and Cost Data Information [page 2486]..
Cloud Platform Enterprise Agreement	This card has the same functionality as the **SAP BTP Enterprise Agreement** card (see previous row) and is displayed only if your global account uses the CPEA flavor of the consumption-based commercial model.
Pay-As-You-Go for SAP BTP	Shows the cumulative cost trend and monthly cost trend of your global account over the last 12 months according to the licensing of your Pay-As-You-Go for SAP BTP commercial model agreement. This card is displayed only if your global account uses the Pay-As-You-Go for SAP BTP flavor of the consumption-based commercial model. With this commercial model, you pay only for the SAP BTP services that you use, and you are billed monthly in arrears. If your global account uses both the Pay-As-You-Go for SAP BTP consumption-based commercial model and the subscription-based model, this card displays only the usage that is charged under the consumption-based model. For services included in your subscription, only usage beyond your prepaid quota is shown here and is billed per your contract's consumption-based terms. For instance, if your subscription covers 100 site visits and you have 151, this card reflects data for the extra 51 visits that exceed your subscription-based quota. If your global account has received a refund at any time during the current cloud credits period, you may see a difference between the displayed monthly costs and your billing document. Cost information applies to all regions used by the subaccounts in your global account. Data is updated after your monthly balance statement has been generated. → Tip To see more details about your contract, cloud credit or cost details, click *(Expand)*. → Tip To extract the data to a spreadsheet document, use the **Export** option. For more information, see Exporting Usage and Cost Data Information [page 2486]..

Card	What Information is Displayed
Cost and Usage Recommendations	Provides actionable insights and suggestions related to your global account's costs and usage patterns. These recommendations can help you to proactively manage your cloud costs and usage, prevent unexpected expenses, and optimize resource planning. The types of recommendations you may see include: • **Budget Alerts:** Notifications if budgets amounts or budget thresholds have been exceeded, prompting review and assessment of potential overruns and additional costs. • **Budget Management:** Suggestions to set up new budgets or reactivate expired ones to maintain control over future spending. • **Cloud Credit Monitoring:** Alerts if cloud credits are forecasted to be exceeded soon or if current cloud credits have been surpassed, advising monitoring or purchasing additional cloud credits. • **Cost Spikes:** Notifications about unusual increases in costs for specific services, subaccounts, or the entire global account, encouraging your to review detailed cost data and investigate causes.
Last Month's Prepaid Quota Consumption	Shows how many service plans were used during the previous month within the limits of your prepaid quota as defined by your subscription-based commercial model agreement. You can use this card to understand the utilization of service usage relative to the subscription-based plans associated with your global account so you can manage your quota more effectively. Note that services used within your prepaid quota are billed as part of your subscription, with no additional charges. However, any usage exceeding your prepaid quota is billed according to your consumption-based commercial model terms as outlined in your agreement. This card is displayed only if your global account uses both consumption-based and subscription-based models. **ⓘ Note** The total number of service plans shown in this card reflects only the services that had recorded usage in your global account during the previous month. To see all services included in your subscription, regardless of whether they were used in the past month, follow these steps: 1. Open the *Entitlements* > *Service Assignments* page. 2. Toggle on the *Show commercial info* option. 3. In the *Entitlement Source* filter, select all services listed under *All Subscription Based*.
Cost Breakdown	Shows a breakdown of your costliest service plans or subaccounts over the last 12 months. **→ Tip** The card shows 5 items initially. To see additional service plans and subaccounts, click *(Expand)*.

Card	What Information is Displayed
Cost Distribution	Shows the distribution of your costliest service plans or subaccounts in the current month and over the last 12 months. → Tip The card shows 5 items initially. To see additional service plans and subaccounts, click *(Expand)*.

Using the Billing Tab

You can use the *Billing* tab to display, monitor, and analyze your monthly charges of billable services, which have been consumed by the subaccounts in your global account over the last 12 months.

The billing data represents the aggregation of usage and its rating, which is determined by the pricing structure of each service. You can view the bill items for previous months or the estimated charge for the current month, to monitor your consumption costs.

The parameters for charged usage can include the amount of data processed, the number of users, the duration of use, or the specific functionality or modules used by each service. The specifics of these metrics are outlined in your contract.

→ Tip

This tab is available only if your global account uses a consumption-based commercial model.

→ Remember

In certain instances, you may observe charged usage at the subaccount level that reflects **fractional** amounts for metrics that are usually presented as whole numbers, such as user counts, tenants, documents, API calls, or connections.

Although billing is always computed at the global account level, we display the cost allocation across your subaccounts based on their proportional actual usage. This top-down proportional calculation may result in fractional units that you see in the *Costs and Usage* page.

Keep in mind that your monthly balance statement is not impacted by the proportional cost distribution to your subaccounts.

In the *Billing* tab, you can switch between the *View by Service* and *View by Account* views:

View by Service

This view provides an overview of your charged usage across different services and their associated plans, and is useful for verifying billing details.

Here are some useful tips for working in this view:

- To drill down and display more details about a particular service, click on an entry in the table or its › *(View Details)* button. When you drill down, a new pane opens and you can see, for example, exactly which subaccounts consume the selected service and the subtotal of charges and usage per directory and subaccount.
  In this details view, subaccounts are displayed in the context of their account hierarchy. If you want to hide their directories and display the subaccounts as a flat list, choose the *Show subaccounts only* checkbox.

  > → Tip
  >
  > In this details view, you can use the *(Switch)* button for any subaccount to quickly open the *View by Account* view with the specific subaccount selected and displaying the plans it consumes.
  >
  > You can also click *(Configure Table Columns)* to display and hide columns.

- To filter the main table for specific services, use the *Service* dropdown list.
- If the display is cluttered by too many service plans or account entities that have no charged usage, you can choose the *Show only services with charged usage* checkbox to hide them.
- To filter the main table for specific subaccounts or directories, use the *Subaccounts/Directories* dropdown list.
- To view a visual chart of the monthly cost trend, cumulative costs, or breakdown of the costliest subaccounts for a specific service plan over the last 12 months, first select the service plan from the main table and then scroll down past the main table to the chart area. Then, select the chart type from the *View By* dropdown list that is located adjacent to the chart area.

  > → Tip
  >
  > These charts are useful for seeing in which months a service plan has usage and costs, which service plans have increased or decreased usage/costs over time, or the months that have the highest and lowest usage/costs per service plan.

- To view a visual chart with a summary of the costs of all service plans in your global account over the last 12 months, make sure you do not have a specific service plan selected in the main table. Then, scroll past the table to the *Global Account - Cost Summary* chart area.
  In the *View By* dropdown list, select a chart type; for example, *List Price*, *Cost Breakdown by Service Plans*, or *Cumulative List Price*.

  > → Tip
  >
  > In the cost breakdown chart, the 8 costliest service plans in your global account over the last 12 months are displayed. Click *Show More* below the chart legend to expand the chart and to view the cost breakdown of additional plans.

- To open the SAP Price List for a specific service, choose *Open in SAP Price List* from the service plan's ••• *(Actions)* context menu, or you can open the service plan's details view and then click *SAP Price List* located in the header area.

See other useful tips, such as table sorting, filtering, and searching, under *General Tips for the Billing and Usage Tabs* in the Understanding the Layout of the Costs and Usage Page [page 2473] section.

View by Account

This view provides an overview of your charged usage across different subaccounts and directories within your account hierarchy. It is useful for verifying cross-charges within your company or organization.

Here are some useful tips for working in this view:

- To drill down and display more details about a particular subaccount or directory, click on an entry in the table or its › *(View Details)* button. When you drill down, a new pane opens and you can see, for example, exactly which plans are consumed by the selected subaccount or directory and the subtotal of the charges and usage per plan.

  > → Tip
  >
  > In this details view, you can use the *(Switch)* button to quickly open the *View by Services* view with the specific plan selected and displaying the subaccounts that consume it.
  >
  > You can also click *(Configure Table Columns)* to display and hide columns.

- To filter the main table for specific subaccounts or directories, use the *Subaccounts/Directories* dropdown list.
- If your account hierarchy includes directories, you can hide them and display your subaccounts as a flat list by choosing the *Show only subaccounts* checkbox.
- You can quickly expand or collapse the entire account hierarchy by clicking the *(Expand All)* and *(Collapse All)* buttons.
- To filter the main table for specific services, use the *Services* dropdown list.
- To view a visual chart of the monthly cost trend, cumulative costs, or breakdown of the costliest service plans for a specific subaccount or directory over the last 12 months, first select the subaccount or directory from the main table and then scroll down past the main table to the chart area. Then, select the chart type from the *View By* dropdown list that is located adjacent to the chart area.

  > → Tip
  >
  > These charts are useful for seeing in which months a subaccount or directory has usage and costs, which subaccounts or directories have increased or decreased usage/costs over time, or the months that have the highest and lowest usage/costs per subaccount or directory.
  >
  > When you choose a directory, the cost information applies to all subaccounts under the directory.

- To view a visual chart with a summary of the costs of all subaccounts in your global account over the last 12 months, make sure you do not have a specific subaccount or directory selected in the main table. Then, scroll past the table to the *Global Account - Cost Summary* chart area.
  In the *View By* dropdown list, select a chart type; for example, *List Price*, *Cost Breakdown by Subaccounts*, or *Cumulative List Price*.

  > → Tip
  >
  > In the cost breakdown chart, the 8 costliest subaccounts in your global account over the last 12 months are displayed. If your global account has more than 8 subaccounts, click

> *Show More* below the chart legend to expand the chart and to view the cost breakdown of additional subaccounts.

- To open the SAP Price List for a specific service used in a directory or subaccount, first open the details view of the directory or subaccount, then locate the service, and choose *Open in SAP Price List* from the service's ••• *(Actions)* context menu.

See other useful tips, such as table sorting and searching, under *General Tips for the Billing and Usage Tabs* in the Understanding the Layout of the Costs and Usage Page [page 2473] section.

If your global account has subaccounts that have been deleted but have charges in the last 12 months while they were still active, they'll still be listed. Such subaccounts are marked with *(deleted)* after their name. If these subaccounts were originally located under a directory, you'll see them listed directly under the root global account.

If your global account uses both a subscription and a consumption-based commercial model, the *Usage* column shows the combined total usage of services, both subscription-based and consumption-based. Note that:

- The subscription-based consumption includes usage of services that falls within the prepaid quota that is specified in your subscription-based commercial model agreement. This part of the combined usage is shown in the *Prepaid Quota* column. There is no additional billing for consumption of the services that falls within this prepaid quota.
- The consumption-based usage includes the usage of services that exceeds the prepaid subscription-based quota. This part of the combined usage is shown in the *Charged Usage* column. For this usage, you are charged based on the terms outlined in your consumption-based commercial model agreement.

Using the Usage Tab

Use the *Usage* tab to display, monitor, and analyze the distribution of the actual (also called raw) monthly usage metrics for services consumed by subaccounts in your global account over the last 12 months.

> ### Note
>
> The *Usage* tab displays data prior to aggregation for billing and can contain metrics that will not be aggregated since they are always free or belong to free services. Hence, you may notice some discrepancies when comparing the data in the *Usage* tab to the overall usage data shown in the *Billing* tab. Billable usage data is aggregated at the global account level and then processed according to accounting formulas to generate your monthly billing statement.

Pay attention also to the following:

- If your global account uses only a consumption-based commercial model, such as SAP BTP Enterprise Agreement (SAP BTPEA), Cloud Platform Enterprise Agreement (CPEA), and Pay-As-You-Go for SAP BTP, this tab includes all usage data, including non-rated services, such as free service plans.
- If your global account uses both the consumption-based and subscription-based commercial models, this tab combines all usage data, which falls under your consumption-based plans, prepaid subscription quota, and non-rated data sets, such as free service plans. You can see the distribution of charged and prepaid usage and costs in the *Billing* tab.

To Do This...	Do This...
Drill down and display more details about a particular service	Click on an entry in the table or its › *(View Details)* button. When you drill down, a new pane opens and you can see, for example, which subaccounts are consuming the selected service and the subtotal of usage per directory and subaccount. In this details view, subaccounts are displayed in the context of their account hierarchy. If you want to hide their directories and display the subaccounts as a flat list, choose the *Show subaccounts only* checkbox. **→ Tip** In this details view, you can click ⚙ *(Configure Table Columns)* to display and hide columns.
Filter by specific services	Choose one or more in the *Service* dropdown list. **→ Tip** If the display is cluttered by too many service plans that have no usage, choose the *Show only used services* checkbox to quickly hide them.
Filter by specific subaccounts and directories	Choose one or more subaccounts or directories in the *Subaccounts/Directories* dropdown list.
View the monthly usage trend of a specific service plan over the last 12 months	Select the service plan from the main table and scroll past the table to the chart area. Then, choose *Actual Usage* in the *View By* dropdown list that is located adjacent the chart area. This chart is useful for seeing changes in usage trends over time, including the months with the highest and lowest usage per service.
View a monthly breakdown of subaccounts that have used a specific service plan the most over the last 12 months	Select the service plan from the main table and scroll past the table to the chart area. Then, choose *Usage Breakdown by Subaccounts* in the *View By* dropdown list. The top 8 subaccounts that use the selected plan the most are displayed initially. If there are more than 8 subaccounts that use the plan, click *Show More* to expand the chart and to view the breakdown of additional subaccounts.

→ Tip

See other useful tips, such as table sorting, filtering, and searching, in the Understanding the Layout of the Costs and Usage Page [page 2473] section.

Using the Budgets Tab

For detailed information about using this tab to help you manage your global account spending and to plan for future consumption, see Managing Budgets in Your Global Account [page 2490].

Estimated and Forecasted Data

In addition to actual usage that you've already been charged for, the **Costs and Usage** page also shows estimated and forecasted values in the cloud credits usage and cost charts:

- **Estimated Data:** The estimated values reflect your accrued service usage during the period between the last balance statement and the current date.
  The values are estimated because they are based on resource usage values before computation for billing and might change after the next balance statement is issued. The estimated values are not forecast values.
- **Forecasted Data:**The SAP BTP Enterprise Agreement and Cloud Platform Enterprise Agreement cards in the *Overview* tab offer a *Show forecast* option that allows you to display a forecast of your cloud-credit costs and usage for the next 6 months or until the end of current contract period, whichever is shorter.
  The forecast is based on certain assumptions and the historical data taken from your global account. To make a reliable forecast, you need to have consistent data from the recent past covering at least 9 months. If the available historical data in your global account doesn't allow for a reliable prediction, you cannot turn on the forecasting option.
  The forecast does not include the costs and usage of the current month (since this estimated data only), nor does it take into consideration any changes to future usage patterns or costs. Additionally, you cannot turn on the forecasting when you're in the last month of the current cloud credits period.

> ### Note
>
> The forecast data provided by our software is intended for informational purposes only; however, it can you help to anticipate and plan for future usage needs and to budget accordingly.
>
> SAP is not responsible for and cannot guarantee the accuracy or reliability of the forecast data, and it should not be considered as financial, legal, professional, or any other advice. It is your responsibility to independently review and validate the forecast information before making any decisions or taking any actions based on it.

Exporting Usage and Cost Data Information

To export your usage and cost data to a Microsoft Excel spreadsheet document, use the *Export* button menu in the header of the *Costs and Usage* page.

> ### Note
>
> Cost information is exported only if your global account uses the consumption-based commercial model.

The *Export* button menu has various options depending on the period and type of data that you want to export:

- If you want to export all data for the last 12 months, click the main *Export* button.
- If you want to export all data for the last 12, 24, or 36 months, click the button's dropdown menu and choose the relevant period.
- If you want to export only the data that you've currently filtered in the main table for the current month or last 12, 24, or 36 months, choose *Export* > *Custom*.

→ Tip

Use this option also if you want to export all your cost and usage data for the current month without considering any currently set filters. To do this, deselect the *Export only filtered data* checkbox in the *Custom Export* dialog box.

In the *Custom Export* dialog box, you can also choose any label names (keys) that you want to add as dedicated columns to the `Subaccount Costs by Service` and `Actual Usage` sheets (tabs) in the exported Microsoft Excel file. Start typing to list the available label names for your existing subaccounts and directories. In the exported spreadsheet, the values for each selected label name are displayed in the data rows under their respective label column.

The exported document contains several sheets (tabs). The sheets that are included depend on the commercial model that is used by your global account as shown in the following table:

		Commercial Model	
Sheet Name	**Description**	**Subscription-Based**	**Consumption-Based**
Global Account Info	Provides general information about your global account. If there is a cloud-credit balance for the global account, then its cloud-credit usage, per month as a percentage of your total cloud credits for the current cloud credits period, is also shown.	Yes	Yes
Global Account Costs	Allows you to view total monthly usage data and costs for all billable services and plans consumed at the level of your global account. The items listed are all the billed items that are created in the accounting system and deducted from the cloud-credit balance of your global account.	No	Yes

Sheet Name	Description	Commercial Model	
		Subscription-Based	Consumption-Based
Subaccount Costs by Service	Allows you to view monthly usage data and costs of all billable services consumed by plan and subaccount. All subaccount calculations are estimated and proportionate to the total global account usage: `[Subaccount usage / Global account usage x Rate plan per SKU]` **Note** Global accounts are the only contractual billable entity for SAP BTP. Directories and subaccounts are used as structural entities in the global account, and their usage and cost data should only be used for your internal cost estimations. The relative calculation per billable usage within each subaccount is an estimation only as it is based on certain measures which in some cases can either be different from the metrics that are presented in the **SAP BTP Enterprise Agreement**, **Cloud Platform Enterprise Agreement**, and**Pay-As-You-Go for SAP BTP** cards in the *Overview* tab, or which use different formulas than the ones used for billing.	No	Yes
Actual Usage	Allows to you to view the actual monthly usage data of all consumed services by plan, subaccount, and space. Actual or raw usage data is different to the billed usage that is used in your billing document, and includes non-billable services. The aggregated usage for each service is based on a formula that is specific to each service, for example, MIN, MAX, or AVG.	Yes	Yes

Sheet Name	Description	Commercial Model	
		Subscription-Based	Consumption-Based
CF Space Costs by Service	Allows you to view the monthly cost distribution of your service plans across the Cloud Foundry spaces in your global account. **Note** This is a beta feature. Beta features aren't part of the officially delivered scope that SAP guarantees for future releases. For more information, see Important Disclaimers and Legal Information. To include this information in the spreadsheet, you must choose the *Export cost distribution by Cloud Foundry spaces* option in the *Custom Export* dialog box. The cost distribution in the exported spreadsheet is estimated and proportionate to the usage of each service across the Cloud Foundry spaces within a subaccount. For example, if the distribution of the total usage of a service is 20% in one space and 80% in another space within the same subaccount, the cost per space is calculated proportionately as 20% and 80% of the total cost of the service in that subaccount for the two spaces, respectively. When a specific service **instance** is used by multiple spaces in Cloud Foundry, the full cost of that service is only charged to the space where it is provisioned. So, the cost breakdown by spaces in the exported Excel file works best when the service is used in the same space it was set up in. However, you can use the exported subaccount data to split the costs differently to better fit your setup.	No	Yes
Labels	Lists the user-defined labels that are assigned directly to each subaccount that has usage data. Also lists the labels that are inherited by the subaccounts from their parent directories, and the IDs of those subaccounts. This sheet does not display any usage data; however, you can use this information to determine usage by label by extrapolating the subaccount usage data in the other sheets to the labels assigned to the subaccounts in this sheet.	Yes	Yes

Related Information

Managing Budgets in Your Global Account [page 2490]
Commercial Models [page 113]
Trial Accounts and Free Tier [page 109]
Using Free Service Plans [page 118]
Managing Entitlements and Quotas Using the Cockpit [page 2521]
View Subaccount Usage Analytics [page 2518]
View Directory Usage Analytics [page 2507]
Labels [page 125]
Monitoring Usage Information Using APIs of the SAP Usage Data Management Service [page 2766]

6.1.1.4.6 Managing Budgets in Your Global Account

Budgets allow cost controllers and global account admins in SAP BTPto better control their global account spending and plan for future consumption by setting up budget limits for global accounts.

Automatic email alerts notify global account administrators when spending exceeds defined thresholds, allowing for timely intervention. You can also use the SAP Alert Notification service for SAP BTP to create custom alerts according to your specific monitoring needs.

> ⓘ Note
>
> The use of budgets in your global account is subject to its availability in your country or region. Additionally, even where budgets are available, the option to send automatic email alerts to global account administrators when spending exceeds defined thresholds, may not be supported in your country or region.

> → Tip
>
> Remember to periodically review your budget settings and thresholds to ensure proactive financial management.

Budget Limitations and Responsibilities

When you use budgets in your global account, note the following important details:

- Costs and usage values between the last balance statement and the current date are estimates and may change at the end of the month when the actual overall global account usage is aggregated for billing purposes. SAP does not guarantee the accuracy or reliability of real-time alerts based on these estimates. It is your responsibility to review and validate alerts before you make any decisions based on them.
- The system doesn't suspend costs and service usage when a budget amount or threshold is exceeded. It's your responsibility to take the appropriate action — for example, by reducing active service usage, disabling non-essential services, adjusting the budget amount, or contacting the global account team — to prevent overages.

- There may be a delay between the time a budget threshold was exceeded and the time that you receive a notification. This may be due to delays in the update cycles of costs and service usage, as well as the periodic evaluations of budgets against your global account costs and usage.
- If your global account combines both subscription and consumption-based commercial models, then any usage that falls within your prepaid quota, as defined by your subscription-based commercial agreement, is not evaluated by your budget.

6.1.1.4.6.1 Accessing Your Global Account's Budgets

Access and manage your global account's budgets in SAP BTP cockpit. Monitor spending, view active and expired budgets, and configure table columns.

Prerequisites

- Your global account uses the SAP BTP Enterprise Agreement (SAP BTPEA), Cloud Platform Enterprise Agreement (CPEA), or Pay-As-You-Go for SAP BTP flavor of the consumption-based commercial model, or it combines one of these consumption-based flavors with the subscription-based commercial model. Note that the use of the consumption-based commercial model is subject to its availability in your country or region.
- You must also be a global account administrator or global account viewer.

Context

Procedure

1. Open SAP BTP cockpit.
2. From the global account level, go to the *Costs and Usage* page.
3. Click on the *Budgets* tab.

   The *Budgets* tab shows your global account's budgets in two separate tables:

   - **Active Budgets**: All budgets that are currently active and monitoring spending in your global account.

     > → Tip
     >
     > Choose the *View in Billing Tab* action in the ••• *(Actions)* context menu of a budget to quickly open the Billing tab. This tab is then automatically filtered by the subaccounts and service plans defined in the budget's scope.

   - **Expired Budgets**: All budgets whose end date surpasses the current date. Expired budgets are no longer active in your global account and don't send alerts or events.

> → Tip
>
> Click ⚙ *(Configure Table Columns)* to set the columns you want to view in each table.

4. Click on a budget to view its configuration and *History* chart.

6.1.1.4.6.2 Creating a Budget

Creating a budget allows you to monitor and manage spending in your global account. You can create a budget from scratch or from an existing budget that is active or has expired.

Context

Each global account is allowed up to 30 active budgets. Active budgets have an end date in the future or no end date. Budgets expire automatically when the current date surpasses the end date of the budget.

Procedure

1. In SAP BTP cockpit, open your global account and then navigate to the *Costs and Usage* page.
2. In the *Budgets* tab, click *Create*.

   > → Tip
   >
   > To duplicate a previous budget configuration, choose *Create as New Budget* in the budget's ••• *(Actions)* context menu. Note that when you use this option to create a new budget, you cannot change the budget type.

   > ⓘ Note
   >
   > The *Create* and *Create as New Budget* buttons are disabled when your global account has reached the 30-budget limit.

3. Configure the budget's settings.

   For more information about the budget settings, see Understanding the Budget Settings [page 2493].

   a. Enter the budget name and an optional description.
   b. Choose the budget type: *Cost* or *Charged Usage*.
   c. (Optional) Define the budget's scope by selecting the applicable subaccounts and service plans.
   d. Specify the budget amount.
   e. Set the time period by entering the start date and optionally the end date. Enter a start date from today onward.
   f. (Optional) Set up to three budget thresholds.

      See also Setting Up Budget Threshold Alerts [page 2496].

4. View the budget amount and threshold in the *History Preview* chart below the *Scopes* section.
5. Review your configuration and then click *Create*.

6.1.1.4.6.3 Understanding the Budget Settings

A budget configuration includes these settings:

Setting	Description
Budget Name and Description	A unique display name and short description for the budget.
Monthly Budget Amount	The amount allocated to the budget per month. • When the budget type (see `Budget Type`) is set to *Cost*, the specified amount represents a monetary value in the same currency as that of the global account. • When the budget type is set to *Charged Usage*, the specified amount corresponds to the unit of the metric (for example: `API Calls`) of the selected service plan (see `Scope`).
Time Period	Includes the budget start date and optional end date. A budget without an end date never expires. Note however, that you are allowed up to 30 active budgets per global account.
Budget Type	Determines the type of data that the budget is based on: • *Cost*: The budget is based on list price data as shown in the *Billing* tab. • *Charged Usage*: The budget is based on charged usage data as shown in the *Billing* tab.

Scope	Determines which subaccounts and service plans in the global account are monitored by the set budget:
	• *Subaccounts*: By default, the budget's scope covers all subaccounts, but you can choose specific subaccounts to monitor.
	• *Service Plans*: The service plans that are monitored by the budget depend on the budget's type:
	  • When the budget type is set to *Cost*, then by default all services and their plans are covered in the budget's scope, but you can choose specific plans to monitor.
	  • When the budget type is set to *Charged Usage*, you must specify a single service plan to be monitored by the budget. If a particular plan has multiple metrics, then you must choose a specific metric.
	Note
	You can only choose from services that currently have registered usage in your global account.

Setting	Description
Budget Thresholds	(Optional) Define up to three threshold percentages ranging from 1% to 120% relative to the monthly budget amount. When you configure a budget threshold and enable it (by selecting its checkbox), the threshold value is displayed over the *History* chart when you view a budget's details in the cockpit. Additionally, you can: • Opt-in to send automatic email notifications to global account administrators when a threshold is exceeded. > **Note** > This option is subject to availability in your country or region. • Use the SAP Alert Notification service for SAP BTP to set up customized alerts for your preferred recipients, communication channels, and monitoring solutions. For more information, see Setting Up Budget Threshold Alerts [page 2496].

6.1.1.4.6.4 Viewing a Budget's Details

Viewing a budget provides detailed information about the budget's configuration and status.

Context

Procedure

1. In SAP BTP cockpit, open your global account and then navigate to the *Costs and Usage* page.
2. In the *Budgets* tab, click on a budget to display more details.

   For more information about the budget settings, see Understanding the Budget Settings [page 2493].

   Use the *History* chart below the *Scopes* section to view the costs or charged usage in your global account over the past 12 months. The budget amount and threshold values are overlaid on the chart data, including budget exceedance information, if relevant.

6.1.1.4.6.5 Editing a Budget

Editing a budget allows you to make changes to the budget's settings.

Context

> ⓘ Note
>
> When you make changes to an existing budget, previously sent alerts may not accurately reflect the new budget settings.
>
> Any changes you make to budget amount will trigger alerts from the current month onward, not retroactively.

Procedure

1. In SAP BTP cockpit, open your global account and then navigate to the *Costs and Usage* page.
2. In the *Budgets* tab, go to the Active Budgets table.
3. Choose *Edit* in the budget's ••• *(Actions)* context menu.
4. Make changes to the budget's editable settings.

   Note that you cannot change the start date and budget type of an existing budget.

   For more information about the budget settings, see Understanding the Budget Settings [page 2493].
5. Review your configuration and then click *Update*.

   > → Tip
   >
   > Whenever you change a budget's settings, we recommend you review and update your threshold values carefully to ensure they align with the new budget settings.

6.1.1.4.6.6 Deleting a Budget

Delete a budget when it is no longer needed — for example, to free capacity when you reach the 30-active-budget limit, or to remove a budget created in error.

Context

Before deleting a budget, check with the users who created and last modified it to confirm the budget is no longer needed. Open the budget's details and look for the *Created By* and *Modified By* fields.

> ⚠ Caution
>
> Deleting a budget removes it permanently from your global account. Deleted budgets cannot be retrieved.

Procedure

1. In SAP BTP cockpit, open your global account and then navigate to the *Costs and Usage* page.
2. Go to the *Budgets* tab.
3. Choose *Delete* in the budget's ••• *(Actions)* context menu.
4. Confirm the delete action.

6.1.1.4.6.7 Setting Up Budget Threshold Alerts

Besides regularly checking the status and history graphs of active budgets for up-to-date information on spending, you can also configure alerts to notify relevant users and/or channels in your organization when a budget's threshold is exceeded.

Before you can set up the threshold alerts for an active budget, you first need to configure the threshold values in your budget's configuration. You can define up to three budget thresholds per budget. For example: 70%, 90%, and 100% of the monthly budget amount, respectively.

Then, once you've enabled these threshold values (by selecting their checkboxes), you can set up the following alert types:

Alert Type	Description	Action Required
Automatic Alerts ⓘ Note This option is subject to availability in your country or region.	Whenever a budget's threshold value is exceeded, the system automatically sends an email to all administrators of the global account. This option is free of charge and requires no additional setup besides enabling the opt-in switch in the budget's configuration. By default, it is turned off per budget. ⓘ Note You cannot customize the content of the email and to which global account administrators the email is sent.	Edit your budget, and then in the *Budget Thresholds* section, turn on the *Send alerts automatically to global account admins* option. → Tip If you don't receive any emails, please check your spam or junk folder. You can also add the `btp.budgets@notifications.sap.com` to your address book or safe senders list.

Alert Type	Description	Action Required
Custom Alerts	Whenever a budget's threshold value is exceeded, we also automatically send an event to the SAP Alert Notification service for SAP BTP. You can integrate with SAP Alert Notification service to send custom alerts to your preferred recipients, communication channels, and monitoring solutions. This is a paid service. For pricing information, go to the SAP Discovery Center. **Note** If you turn off the *Send alerts automatically to global account admins* option, budget events are still sent to the SAP Alert Notification service when an enabled threshold is exceeded.	Subscribe to the budget events (`BudgetAlertThresholdExceeded`) using your own SAP Alert Notification service instance and configure the custom alerts, as needed. For more information, see: • SAP Alert Notification for SAP BTP • Budget Threshold Exceedance Event

When working with budget threshold alerts, consider the following:

- The two alert types are independent of one another, you can use either one or both.
- When you enable a threshold or change the budget amount in an existing budget, alerts will be triggered from the current month onward, not retroactively for previous months.
- The automated mail and budget events are sent only once when a budget threshold is exceeded. We do not send a daily recurring email or event while the budget remains in excess.
- If a set budget amount is exceeded and you haven't set a 100% threshold value nor opted to use any of the available alert types, you won't be automatically notified about the budget exceedance.
- There may be a delay between the time a budget threshold was exceeded and the time that you receive an alert. This is due to delays between cost and service usage update cycles, along with the interval-based evaluations between budgets and your global account's overall costs and usage.

→ Remember

The system doesn't suspend costs and service usage when a budget amount or threshold is exceeded. It's your responsibility to take the appropriate action to prevent overages.

Related Information

Monitoring Usage and Consumption Costs in Your Global Account [page 2471]
SAP Alert Notification Service for SAP BTP
Budget Threshold Exceedance Event

6.1.1.4.7 Configure Legal Information

Administrators can define legal links per enterprise global account in the SAP BTP cockpit.

Prerequisites

You have the Administrator role for the global account for which you'd like to define legal links.

Context

You can define the privacy link relevant for a global account so the members of this global account can view this information.

Procedure

1. Choose the global account for which you'd like to make changes.
2. In the left-hand navigation, choose *Legal Information* > *Configure Privacy Link*.
3. Enter the relevant URL.
4. Save your changes.

   The links you configured are available in the *Legal Information* menu.

Related Information

Navigate in the Cockpit [page 2461]
Global Accounts

6.1.1.4.8 Enable Analytics in the Cockpit

Learn about how to enable analytics in the SAP BTP Cockpit to improve the overall user experience.

Prerequisites

You have the Administrator role for the global account for which you'd like to enable analytics.

> **Note**
>
> If you are an Administrator for multiple global accounts, you must enable analytics for each global account.

Context

To help improve user experience in the SAP BTP Cockpit, global account administrators can allow global account users to send anonymized usage data to SAP. By enabling analytics in the cockpit, we'll identify and fix issues more quickly, making for an improved cockpit experience. We use analytics to understand how users interact with the platform. With this information, we can make improvements, such as adapting navigation patterns, enhancing customer support, and prioritizing features.

What kind of data do we collect?

- Cockpit usage data: How often a page is opened or which functionality (buttons, links) is used or not used
- Click paths: The navigational paths a user takes to navigate through the cockpit

What data don't we collect?

- Personal, user-specific, or company identifying data
- User input

Your data is used to improve the overall user experience and cannot be used for marketing or advertising purposes.

Procedure

1. Open the global account for which you'd like to enable analytics.

   If you are the Global Account Owner and analytics has not yet been enabled, a dialog box will automatically appear when you access the global account. Here you can enable analytics by clicking "Allow for this Global Account." You can also choose to no longer see this information or decide later following the steps below.
2. Navigate to the global account settings by clicking *(Global Account Settings)*.
3. Select **Consent Management**.
4. Toggle the button for **Help improve the BTP cockpit experience**.
5. Save your changes.

Results

After you have enabled analytics in the global account, each global account user must also provide their consent before anonymized usage data is sent to SAP. All users of a global account can change their consent choice at any time by accessing Cookie Preferences in the Legal Information menu located in the lower left. Global account administrators can also update or withdraw global account consent at any time in the global account settings.

Related Information

SAP BTP Cookie Statement
SAP BTP Cockpit Privacy Statement

6.1.1.4.9 Enabling and Modifying Notifications

Control which notifications you receive and how they are delivered by configuring your notification settings for the services associated with your account.

Context

Notification settings let you decide which events you are informed about and through which channels. The settings are organized in three levels, so you can control notifications broadly or fine-tune them for a single event:

- **Categories** – Each category represents a service that is associated with your global account or subaccount. You see only the services that are available to you, so the list differs from account to account. You can enable or disable notifications for an entire service from this level.
- **Events** – Within a service, individual events represent the specific situations that can trigger a notification. You can enable or disable each event separately.
- **Delivery channels** – For each event, you choose how you are notified, for example through in-app notifications or e-mail.

At every level, a toggle switch turns notifications on or off, and a navigation arrow opens the next level of detail. The steps below describe the general process; Cloud Foundry is used as an example service.

> **Note**
>
> For now, only global account administrators can receive and modify notifications.

Procedure

1. Open the *Settings* dialog in the user menu and choose *Notifications*.

   You will see the services that are associated with your account. Each row shows a toggle switch and a navigation arrow.
2. To enable or disable notifications for an entire service, use the toggle switch of the corresponding category.
3. To fine-tune the events of a service, select its navigation arrow, for example *Cloud Foundry*.

   The detail view for the selected service opens. A toggle at the top enables or disables all notifications for the service, and the individual events are listed below.

4. Use the toggle switch of an event to enable or disable notifications for that event.
5. To choose how you are notified about an event, select its navigation arrow.

   The event detail view opens and describes when the notification is sent.
6. Use the event toggle to enable or disable the event, and under *Choose how you get notified*, enable or disable each delivery channel.

   Typical delivery channels are:

   - *In-App notifications* – Delivered inside the SAP BTP cockpit.
   - *E-mail* – Sent to your registered e-mail address.
7. Choose *Save* to apply your changes, or *Cancel* to discard them.

Results

Your notification preferences are saved. You are notified only about the services and events you enabled, through the delivery channels you selected. You can return to *Settings* at any time to modify these preferences.

6.1.1.5 Managing Directories Using the Cockpit

Learn how to organize and manage your subaccounts according to your technical and business needs by using directories in the SAP BTP cockpit.

Context

Procedure

1. Learn how to Create a Directory [page 2502].
2. Learn how to Manage the Account Explorer Hierarchy [page 2504].
3. Learn how to Manage Users in Directories [page 2505].
4. Learn how to View Directory Usage Analytics [page 2507].

Related Information

Directories [page 123]
Managing Subaccounts Using the Cockpit [page 2509]

6.1.1.5.1 Create a Directory

Create a directory using the SAP BTP cockpit to organize and manage your subaccounts. For example, you can group subaccounts by project, team, or department.

Context

> → Recommendation
>
> Before creating your subaccounts, we recommend you learn more about Setting Up Your Account Model.

For more information about directories, see Directories [page 123].

Procedure

1. In your global account, navigate to the *Account Explorer* page.
2. In the *Create* dropdown, choose *Directory*.

   > → Tip
   >
   > You can also create a directory by selecting *Create Directory* from the ••• *Actions* menu button of the global account or an existing directory.

3. In the *Create Directory* dialog box, enter a display name for your new directory and choose a parent. The parent can be the global account or another directory.
4. **Optional:** Under *Advanced*, choose the *Enable entitlement management* option if you want to manage the entitlements that are used by the subaccounts under this directory. This allows you to reserve quota for service plans so that they cannot be assigned to other subaccounts or directories. For more information, see Entitlements and Quotas [page 127] and Configure Entitlements and Quotas for Directories [page 2524].

   You can enable this feature later from the *Entitlements* > *Entity Entitlements* page or by editing the directory in the *Account Explorer* page.

   If you don't enable this feature, then subaccounts in this directory use the entitlements and quota from the global account level.

   > ⓘ Note
   >
   > Only a single directory in any given directory path can have the user management (see next option) and/or entitlement management features enabled.
   >
   > For example, if you have 3 stacked directories in your account hierarchy and the middle directory has both the user and entitlement management features enabled, then neither of these features can be enabled for its parent or child directory since these two directories are in the same direct path as the middle directory.

5. **Optional:** Under *Advanced*, choose the *Enable user management* option if you want to assign specific users as administrators or viewers of this directory. For more information, see Manage Users in Directories [page 2505].

   You can enable this feature later from the directory's *Users* page or by editing the directory in the *Account Explorer* page.

   If you don't enable this feature, then any user that has access to the directory in the account hierarchy can view and work in it.

   > ⓘ Note
   >
   > - The user management feature can be enabled only in combination with the entitlement management feature on the same directory in given path.
   > - Only a single directory in any given directory path can have the user management and/or entitlement management features enabled.
   >   For example, if you have 3 stacked directories in your account hierarchy and the middle directory has both the user and entitlement management features enabled, then neither of these features can be enabled for its parent or child directory since these two directories are in the same direct path as the middle directory.

6. **Optional:** Under *Advanced* > *Labels*, assign labels to the directory to make organizing and filtering your directories and their subaccounts easier. For more information, see Labels [page 125].

   > → Tip
   >
   > - When adding multiple values to a label, press `Enter` after each value.
   > - When you start typing, any matching label names and values that are already assigned to other directories in your global account are offered as suggestions.

   > ⓘ Note
   >
   > All subaccounts in the path of this directory also inherit all labels that are assigned to this directory.

7. Review the details of your directory and choose *Create* to finalize the process.

Results

Your directory is created.

Next Steps

Once you've created a directory, you can perform the following actions:

- Edit the directory - you can edit the name, description, entitlements, and labels of the directory.
- Add subaccounts to the directory - Manage the Account Explorer Hierarchy [page 2504].
- Assign users and entitlements to the directory (if you've enabled the user and entitlement management features, respectively).

- Delete the directory - you can only delete a directory that does not contain any subaccounts or subdirectories.

Related Information

Directories [page 123]
Manage the Account Explorer Hierarchy [page 2504]
Manage Users in Directories [page 2505]
Configure Entitlements and Quotas for Directories [page 2524]
View Directory Usage Analytics [page 2507]
Labels [page 125]

6.1.1.5.2 Manage the Account Explorer Hierarchy

Create an account structure by creating a hierarchy of directories and subaccounts using the SAP BTP cockpit. Add, move, and delete subaccounts and directories in your structure.

Context

Here, you'll learn how to:

- Add directories and subaccounts to a directory
- Move directories and subaccounts in the account structure
- Delete directories and subaccounts from the account structure
- Export the account structure to Microsoft Excel format.

A subaccount moves with its assigned service plans and quota. The entitlements of the source and target directories are adjusted accordingly.

If any entitlement for a plan in the target directory is configured with the auto-assign option, it is then automatically applied to the moved subaccount. Automatic quota assignments are subject to available quota in the target directory.

You can create a hierarchical structure that is up to 7 levels deep. The highest level of a given path is always the global account and the lowest is a subaccount, which means that you can have up to 5 levels of directories between the global account and the lowest level subaccount.

Procedure

- Add directories and subaccounts to a directory.

- In the *Create* dropdown, choose *Directory*.
  When creating the new directory, you choose its location in the account structure by selecting its parent directory or the global account directly in the creation dialog box.
- You can also create a directory by selecting *Create Directory* from the ••• *Actions* menu button for the global account or a specific parent directory.

- Move subaccounts in the account structure.
  - Click ••• *Actions* and select *Edit*. Then select a new parent.
  - Click ••• *Actions* and select *Move*. Then select a new parent.
  - Click ✣ *Move* to move subaccounts by drag and dropping.
- Delete directories and subaccounts from the account structure.
  - Click ••• *Actions* and select *Delete*.
  - To delete a subaccount, you can also navigate into the subaccount and click *Delete Subaccount*.
- Export the account structure to Microsoft Excel format.
  - Click *Export*.
    The exported Microsoft Excel file contains a sheet (tab) called *Account Structure*, which lists all the subaccounts and directories in the account structure, and includes information about each entity.
    Each subaccount and directory displays its name, ID, description, path, creation/change dates, and the user that created it.
    Subaccounts also display information about their environment, provider, region, used for production setting, and whether the beta feature option is enabled. Directories also display whether they are configured to manage users and entitlements.

Related Information

Directories [page 123]
Create a Directory [page 2502]
Create a Subaccount [page 2509]
Delete a Subaccount [page 2520]

6.1.1.5.3 Manage Users in Directories

Manage members in your directory using the SAP BTP cockpit.

Prerequisites

- You're a directory administrator.
- User management is enabled for this directory.
  If the feature isn't enabled already, you can do so using one of the following methods:
  - Go to the *Account Explorer* page, edit the directory, and then under *Advanced*, choose the *Enable User Management* option.

- Navigate into the directory from the *Account Explorer* page, and then in the *Users* page, choose the *Enable Entitlement and User Management* option (if entitlement management is already enabled for the directory, then choose the *Enable User Management* option instead).

> **Note**
>
> - The user management feature can be enabled only in combination with the entitlement management feature on the same directory in given path.
> - Only a single directory in any given directory path can have the user management and/or entitlement management features enabled. See Configure Entitlements and Quotas for Directories [page 2524].
>
> For example, if you have 3 stacked directories in your account hierarchy and the middle directory has both the user and entitlement management features enabled, then neither of these features can be enabled for its parent or child directory since these two directories are in the same direct path as the middle directory.

- Your platform user exists in a trusted identity provider.
  All users of SAP BTP are stored in identity providers, either in the default or in a custom identity provider. SAP BTP needs a copy of the user, sometimes called a shadow user. You assign the shadow user authorizations to access resources in SAP BTP. When a user authenticates, SAP BTP forwards the request to the identity provider.
  For more information, see Trust and Federation with Identity Providers [page 2538].

Context

Assign predefined or custom role collections to users who need to manage or view the directory in SAP BTP cockpit. Examples of predefined role collections include the following:

- *Directory Administrator*
- *Directory Viewer*

For more information about these role collections, see Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138].

Procedure

1. Navigate to your directory.
2. Add a user to your directory.

   For more information, see Create Users [page 2608].
3. Assign a role collection to the user.

   For more information, see Assign Users to Role Collections [page 2617].

Results

The next time this user logs on to the SAP BTP cockpit, the user can access this directory.

Related Information

Working with Users [page 2603]
Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138]

6.1.1.5.4 View Directory Usage Analytics

You can explore, compare, and analyze all your actual usage data for the services and applications that are available in your directory.

To monitor and track usage in a directory, open your directory from the *Account Explorer* in the SAP BTP cockpit, and then go to the *Usage Analytics* page in the navigation area.

> **Note**
>
> This cockpit page is available only for directories that have user management enabled.
>
> You must also be assigned as an admin of the directory. For the more information, see Create a Directory [page 2502] and Manage Users in Directories [page 2505].

Views in the Directory 'Usage Analytics' Page

The directory *Usage Analytics* page contains two views that display the same usage information; however, your entry point is different – either from a service/subscription or a directory/subaccount:

View	Description
Service Usage	Displays high-level usage information for a selected service or business application subscriptions according to the directory from which you accessed the *Usage Analytics* page. You can also drill down and view usage data for specific subdirectories and subaccounts that are located in this directory.
Directory/Subaccount Usage	Displays high-level usage information for all services and business application subscriptions according to the directory from which you accessed the *Usage Analytics* page. You can also drill down and view usage data for specific subdirectories and subaccounts that are located in this directory.

Information is shown only for those services and subscriptions whose metered consumption in the directory is greater than zero.

Usage values are updated every 24 hours. Usage data displayed in this cockpit page refers to actual usage, not billed usage.

The *Space* filter is available only when you've selected a subaccount and the subaccount has the Cloud Foundry environment enabled.

> → Tip
>
> - If your directory is in a global account that uses the consumption-based commercial model, you can view information about your billed usage for billable services in your global acccount's *Usage Analytics* page. The global account's *Usage Analytics* page also provides information about cloud-credit usage. For more information, see Monitoring Usage and Consumption Costs in Your Global Account [page 2471].
> - You can also view usage information by subaccount directly in a subaccount's *Usage Analytics* page. For more information, see View Subaccount Usage Analytics [page 2518].

Working with the Tables and Charts

Both the *Service Usage* and *Directory/Subaccount Usage* views display the usage information in tabular and chart formats.

- The tables present accumulated usage values based on the aggregation logic of each service plan and its metric over the selected time period. The usage values are broken down by resource.
- The charts present usage values for one or more service plans that you select in the adjacent tables.

Select a table row to display the usage information in the chart. You can also compare usage between service plans in the same service by selecting multiple rows. To compare usage, the rows items must also share the same metric.

In the charts, you can view the data as a column chart or as a line chart.

To display a larger view of a chart, choose the *(Zoom)* button.

Some rounding or shortening is applied to large values. You can mouse over values in the table to view the exact values as tooltips.

Related Information

Org Administration Using the Cockpit [page 2792]
Monitoring Usage and Consumption Costs in Your Global Account [page 2471]
What Is the Consumption-Based Commercial Model? [page 114]
View Subaccount Usage Analytics [page 2518]
Monitoring Usage Information Using APIs of the SAP Usage Data Management Service [page 2766]
Manage Users in Directories [page 2505]

6.1.1.6 Managing Subaccounts Using the Cockpit

Learn how to structure a global account according to your organization's and project's requirements with regard to members, authorizations, and entitlements by managing subaccounts.

Context

To learn more about the concept of subaccounts, see Subaccounts [page 122].

Procedure

1. Learn how to create subaccounts.

   Create a Subaccount [page 2509]
2. Learn how to manage users.

   Add Members to Your Subaccount [page 2513]
3. Learn how to administer your subaccount.
   - Change Subaccount Details [page 2516]
   - View Subaccount Usage Analytics [page 2518]
   - Delete a Subaccount [page 2520]

6.1.1.6.1 Create a Subaccount

Create subaccounts in your global account using the SAP BTP cockpit.

Create a Subaccount

Prerequisites

You are a global account administrator.

> **Note**
>
> If you are creating a subaccount in an entitlement-managed or user-managed directory, then you must be either a global account administrator or a directory administrator. See Configure Entitlements and Quotas for Directories [page 2524] and Manage Users in Directories [page 2505].

→ Recommendation

Before creating your subaccounts, we recommend you learn more about Setting Up Your Account Model.

Context

You create subaccounts in your global account. Once you create a new subaccount, you see a tile for it in the global account view, and you are automatically assigned to it as an administrator.

Procedure

1. From your global account, navigate to the *Account Explorer* page.
2. Select *Create* > *Subaccount* to create a subaccount from scratch.

   → Tip

   You can also create a subaccount from scratch by selecting *Create Subaccount* from the ••• *(Actions)* context menu of the global account or a directory.

   → Tip

   Alternatively, you can create a new subaccount from an existing subaccount. For more information, see the Create a Subaccount from an Existing Subaccount procedure below.
3. Specify a display name.
4. **Optional:** Enter a description.
5. Enter a subdomain for your subaccount. This will become part of the URL for accessing applications that you subscribe to from this subaccount.

   ⓘ Note

   After selecting a multi-environment region, an auto-generated subdomain is entered for you in the *Subdomain* field. You can modify this subdomain to suit your preferences, provided it adheres to the naming guidelines.

   The subdomain name can contain up to 63 characters such as letters, digits and hyphens (not allowed in the beginning or at the end), and must be unique across all subaccounts in the same region. Uppercase and lowercase letters can be used, however that alone does not qualify as a means to differentiate subdomains (e.g. **`SUBDOMAIN`** and **`subdomain`** are considered to be the same).
6. Choose the desired region for your subaccount.
7. Select a parent for your subaccount. The parent can either be the global account or a directory.
8. If your subaccount is to be used for production purposes, select the *Used for production* option.

   ⓘ Note

   This does not change the configuration of your subaccount. Use this flag for your internal use to operate your production subaccounts in your global account and systems more efficiently. Your cloud

operator may also use this flag to take appropriate action when handling incidents related to mission-critical accounts in production systems.

You can change your selection at any time by editing the subaccount properties. Do not select this option if your account is used for non-production purposes, such as development, testing, and demos.

9. **Optional:** To use beta services and applications in the current subaccount, under *Advanced*, select *Enable beta features*.

> ⚠ Caution
>
> You shouldn't use SAP BTP beta features in subaccounts that belong to productive enterprise accounts. For more information, see Important Disclaimers and Legal Information.

> ⓘ Note
>
> Once you have enabled this setting in a subaccount you cannot disable it.

10. **Optional:** Under *Advanced* > *Labels*, assign labels to the subaccount to make organizing and filtering your subaccounts easier. For more information, see Labels [page 125].

> → Tip
>
> - When adding multiple values to a label, press `Enter` after each value.
> - When you start typing, any matching label names and values that are already assigned to other subaccounts in your global account are offered as suggestions.

11. Save your changes.

Next Steps

Once your new subaccount is set up, navigate to it to enable the environment that you wish to use and add users.

Create a Subaccount from an Existing Subaccount

Prerequisites

You are a global account administrator.

Context

Instead of creating a subaccount from scratch, you can create one from an existing subaccount that is similar to the one you want to create. This saves you valuable time since all the properties, labels, and service plan assignments are copied from the existing subaccount to the new subaccount.

> **Note**
>
> This option is available only on the AWS, Azure, or GCP region.

Procedure

1. From your global account, navigate to the *Account Explorer* page.
2. Do one of the following:
   - Select *Create* > *Create from Existing Subaccount* and then choose the source subaccount.
   - Locate the source subaccount in your account hierarchy and then choose *Create as New Subaccount* option in its ••• *(Actions)*context menu.
3. Edit the new subaccount's properties and labels, as needed.

   Refer to Create a Subaccount above to find out more information about the various subaccount properties.
4. **Optional:** If you don't want all the assigned service plans in the selected subaccount to be copied to the new subaccount, turn off the *Copy service plan assignments* option.
5. Click *Create*.

   > **Note**
   >
   > If you selected the option to copy the assigned service plans from the source subaccount to your new subaccount, and any of the required plans are missing in the target location of the new subaccount, or if there is insufficient quota, you'll be informed before the subaccount creation process begins so that you or another global account admin can make the necessary adjustments before you try again.

Next Steps

After the new subaccount has been created, you can complete its setup, including its authorization configuration, instances, subscriptions, and environment setups.

Related Information

Global Accounts [page 121]
Navigate in the Cockpit [page 2461]
Account Model [page 121]

6.1.1.6.2 Add Members to Your Subaccount

Add members to your subaccount to enable users to access resources available there. Platform users manage subaccounts with cloud management tools, while business users consume applications and services.

Prerequisites

- You must be a subaccount administrator to add other subaccount members.
- The users exist in a trusted identity provider.
  All users of SAP BTP are stored in identity providers, either in the default or in a custom identity provider. SAP BTP needs a copy of the user, sometimes called a shadow user. You assign the shadow user authorizations to access resources in SAP BTP. When a user authenticates, SAP BTP forwards the request to the identity provider.
  For more information, see Trust and Federation with Identity Providers [page 2538].

> **Note**
>
> This procedure explains how to add users to multi-environment subaccounts. To add members to Neo subaccounts, see Add Members to Your Neo Subaccount.

Context

There are two types of users in subaccounts:

- Platform users
  These users are members of your team. Platform users manage the subaccount with the cloud management tools.
  Assign predefined or custom role collections to users who need to manage or view the subaccount in SAP BTP cockpit. Examples of predefined role collections include the following:
  - *Subaccount Administrator*
  - *Subaccount Viewer*

  For more information about the available role collections, see Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138].
  To work in the Cloud Foundry, ABAP, Kyma, and Neo environments, your team members need additional authorizations.
  For more information, see Giving Access Rights to Platform Users in the *SAP BTP Best Practices Guide*.

  > **Note**
  >
  > If the subaccount has a Cloud Foundry environment, SAP BTP checks Cloud Foundry users with org or space roles. If there's no corresponding user at the subaccount level, SAP BTP creates one automatically.
  >
  > For more information, see About User Management in the Cloud Foundry Environment [page 2787].

- Business users
  These users consume applications and services. Business users don't access cloud management tools like the SAP BTP cockpit. Assign role collections required by the applications and services that the business users consume.

Procedure

> → Tip
>
> If you're a global account administrator, you can quickly assign yourself as an administrator of a subaccount by going to the *Account Explorer* page and then choosing the *Add Me as Admin* option in the *Actions* (•••) context menu of the subaccount. To assign other users, use the following instructions.
>
> This feature isn't available if it has been disabled for your global account.
>
> To disable this feature, send an opt-out request using the component BC-NEO-CIS-OPS. See Getting Support [page 3683] for information about opening a case and getting technical assistance.

1. Navigate to your subaccount.
2. Add a user to your subaccount.

   For more information, see Create Users [page 2608].
3. Assign role collections to the user.

   For more information, see Assign Users to Role Collections [page 2617].

Results

The user can log on to SAP BTP and access resources according to the role collections you've assigned to the user.

> ⓘ Note
>
> If the new subaccount platform user logs on to SAP BTP cockpit and there's no corresponding platform user at the global account level, SAP BTP creates one automatically.

Related Information

User and Member Management [page 133]
Navigate in the Cockpit [page 2461]

6.1.1.6.3 Enable Environment or Create Environment Instance

Enable an environment or create an environment instance using the SAP BTP cockpit.

Prerequisites

- You're managing a multi-environment subaccount.
- You're a subaccount administrator.
- You have free quota assigned to your subaccount.

Context

On a subaccount level, environments constitute the actual platform-as-a-service (PaaS) offering of SAP BTP that allows for the development and administration of business applications.

Each environment comes equipped with specific tools, technologies, and runtimes that you need to build applications. The availability of different environments allows for greater flexibility in your development process.

For more information, see Environments [page 62].

Procedure

1. Navigate into the subaccount.
2. Choose your method: **Enable Environment** or **Create an Environment Instance**.
    - **Enable Environment**: On the *Subaccount - Overview* page, click *Enable <Environment>*.
    - **Create an Environment Instance**: See Environment Instances.

Results

You can use the environment or environment instance in your subaccount.

6.1.1.6.4 Change Subaccount Details

Edit subaccounts using the SAP BTP cockpit.

Context

You edit a subaccount by choosing the relevant action on its tile. It's available in the global account view, which shows all its subaccounts.

The subaccount technical name is a unique identifier of the subaccount on SAP BTP that is generated when the subaccount is created.

You can change to the following subaccount details:

Editable Subaccount Details

Field	Details
Display Name	Specify a human-readable name for your subaccount and change it later on, if necessary. This way you can distinguish between multiple subaccounts.
Description	(Optional) Specify a short descriptive text about the subaccount.
Parent	Change the parent of your subaccount. When you create a subaccount, this is added as a direct child of the global account you created it in. You can change your subaccount's parent and add it to a directory, move it from one directory to another, or remove from a directory by making the global account its parent once again.
Used for production	(Optional) Select this option if your subaccount is being used for production purposes. This does not change the configuration of your subaccount. Use this flag for your internal use to operate your production subaccounts in your global account and systems more efficiently. Your cloud operator may also use this flag to take appropriate action when handling incidents related to mission-critical accounts in production systems. Since subaccounts that are marked as used for production cannot be force deleted, this option also prevents an accidental forced deletion of the account. Do not select this option if your account is used for non-production purposes, such as development, testing, and demos.

Field	Details
Labels	(Optional) Assign, change, or remove labels from your subaccount. Labels help to make organizing and filtering your subaccounts easier. For more information, see Labels [page 125]. → Tip • When adding multiple values to a label, press Enter after each value. • When you start typing, any matching label names and values that are already assigned to other subaccounts in your global account are offered as suggestions.
Enable beta features	(Optional) Enable the subaccount to use services and applications which are occasionally made available by SAP for beta usage on SAP BTP. This option is available to administrators only and is, by default, unselected. ⚠ Caution You shouldn't use SAP BTP beta features in subaccounts that belong to productive enterprise accounts. For more information, see Important Disclaimers and Legal Information. ⓘ Note Once you have enabled this setting in a subaccount you cannot disable it. ⓘ Note If you enable this setting in a subaccount that is already consuming services and subscriptions, these consumed resources will not be automatically updated to beta versions if they exist.

Procedure

1. Choose the subaccount for which you'd like to make changes and choose on its tile.

   You can view more details about the subaccount such as its description and additional attributes by choosing *Show More*.
2. Make your changes and save them.

Related Information

Navigate in the Cockpit [page 2461]
Change the Default Database System

6.1.1.6.5 View Subaccount Usage Analytics

You can explore, compare, and analyze all your actual usage data for the services and applications that are available in your subaccount.

To monitor and track usage in a subaccount, open the subaccount in the SAP BTP cockpit and choose *Usage Analytics* in the navigation area.

Views in the Subaccount 'Usage Analytics' Page

The subaccount *Usage Analytics* page contains views that display usage at different levels of detail:

View	Description
Subaccount	Displays high-level usage information for your subaccount relating to services, business application subscriptions, and Cloud Foundry org spaces. Some information in this view is displayed only for global account admins.
Services	Displays usage per service plan for the region of the subaccount, and the selected metric and period. Information is shown for all services whose metered consumption in the subaccount is greater than zero.
Spaces	(Cloud Foundry environment only) Displays service plan usage per space for the services shown in the *Services* view.

> ⓘ Note
>
> The information displayed in this page depends on the environments that are enabled for your subaccount. For example, information about spaces is displayed only for subaccounts in the Cloud Foundry environment.

Usage values are updated every 24 hours.

Usage data displayed in this cockpit page refers to actual usage, not billed usage.

> → Tip
>
> - If your subaccount is in a global account that uses the consumption-based commercial model, you can view information about your billed usage for billable services in your global acccount's *Usage Analytics* page. The global account's *Usage Analytics* page also provides information about cloud-credit usage. A

directory's *Usage Analytics* page also allows you to view usage information for each subaccount that is located in the directory.

- If you use directories to group your subaccounts, you can view usage information by directory in your global account's *Usage Analytics* page or directly in the directory's *Usage Analytics* page.

For more information, see Monitoring Usage and Consumption Costs in Your Global Account [page 2471] and View Directory Usage Analytics [page 2507].

Working with the Tables and Charts

In the *Services* and *Spaces* views, the usage information is displayed in both tabular and chart formats.

- The tables present accumulated usage values based on the aggregation logic of each service plan and its metric over the selected time period. The usage values are broken down by resource.
- The charts present usage values for one or more service plans or spaces that you select in the adjacent tables. The resolution of the charts is automatically set to days, weeks, or months, depending on the range of the selected time period.

You can perform various actions within the tables and charts:

- In the *Services* and *Spaces* views, select a table row to display the usage information in the chart. You can also select multiple rows to compare usage in the following ways:
  - In the *Services* view, you can compare usage between service plans in the same service. Multi-row selection in the table is possible only when you have selected a single service in the *Service* filter and the row items share the same metric.
  - In the *Spaces* view, you can compare usage between spaces for the same service and service plan. Multi-row selection in the table is possible only when you have selected a single service plan in the *Service Plan* filter.
- In the charts, you can view the data as a column chart or as a line chart.
- To display a larger view of a chart, choose the *(Zoom)* button.
- Some rounding or shortening is applied to large values. You can mouse over values in the table to view the exact values as tooltips.

Using the Filters

Use the filters in the *Services* and *Spaces* views to choose which usage information to display.

The *Spaces* view inherits the filter settings, except for the *Period*, from the *Services* view above it. In other words, when you modify filters in the *Services* view, it affects the information that is displayed in the *Spaces* view. You can apply the *Period* filter independently to each view.

In the *Services* view, you can apply the *Metric* filter only when you have selected a service with more than one metric.

Click the *(Reset)* icon in each view to reset the filters to their default settings. If you reset the filters in the *Services* view, the filters in the *Spaces* are also reset.

Related Information

Org Administration Using the Cockpit [page 2792]
Monitoring Usage and Consumption Costs in Your Global Account [page 2471]
What Is the Consumption-Based Commercial Model? [page 114]
View Directory Usage Analytics [page 2507]
Monitoring Usage Information Using APIs of the SAP Usage Data Management Service [page 2766]

6.1.1.6.6 Delete a Subaccount

Delete subaccounts using the SAP BTP cockpit to clean up your account hierarchy, free up quota used by services in your subaccounts, and to reduce overall costs.

Delete a Subaccount

Prerequisites

- You're a global account administrator.

Procedure

1. In the cockpit, go to the *Account Explorer* page of your global account.
2. In the account hierarchy, locate the subaccount that you want to delete.
3. In the *Actions* (•••) context menu of the subaccount, choose *Delete*.

   The cockpit first checks if the subaccount contains data, such as active subscriptions, service instances, brokers, or platforms.
4. If the subaccount is empty, then confirm the operation, after which the subaccount is deleted.

   If you are a subaccount administrator and the subaccount contains data, such as applications, service instances, spaces, active subscriptions, brokers, and members, then we strongly recommended that you visit the *Instances and Subscriptions* page to review the content in the subaccount and to manually remove the content from there. Then you can come back and safely delete the subaccount.

   > ⓘ Note
   >
   > If you are not a subaccount administrator, then you won't be able to access the *Instances and Subscriptions*. Instead, you'll have to ask a subaccount administrator in your organization to review the content in this subaccount and to manually remove the content from there.

> ### Note
>
> The deletion may take up to 12 hours to complete depending on the amount of content in your subaccount. If your subaccount contains service instances, subscriptions, and runtime environments, these need to be deprovisioned as part of the purge operation, which is a timely process. After the subaccount is deleted, it could take a few more days for some related services to be deleted. Note that you won't be charged for any continued usage of these services in the subaccount during the deletion cleanup.

> ### Caution
>
> If the subaccount contains data, you may have the option to force delete it. This option deletes the subaccount and all its data. Force deletion is available only if the force deletion of subaccounts is enabled in the global account settings and if the subaccount isn't marked as used for production.
>
> You must use this option with extreme caution because any data in the subaccount is deleted permanently, including productive data, without any recovery option.
>
> It is your responsibility to confirm that the subaccount is safe to delete before using the force-delete option. If you cannot check the content of the subaccount, you can get the subaccount reviewed by other qualified members of your organization.
>
> When the force deletion of a subaccount has been requested, the subaccount isn't immediately deleted. Instead, a seven-day begins during which access to the subaccount and its services is partially blocked and the subaccount is marked as *Pending Deletion*. During this waiting period, you can still cancel the scheduled deletion. You can do this on the *Account Explorer* page in the *Actions* menu for the subaccount. After the waiting period, the subaccount and all its data will be permanently deleted.

Related Information

Navigate in the Cockpit [page 2461]
Relationship Between Global Accounts, Subaccounts, and Directories [page 125]

6.1.1.7 Managing Entitlements and Quotas Using the Cockpit

When you purchase an enterprise account, you are entitled to use a specific set of resources, such as the amount of memory that can be allocated to your applications.

An entitlement equals your right to provision and consume a resource. A quota represents the numeric quantity that defines the maximum allowed consumption of that resource.

Entitlements are managed at the global account level, where services plans and their allowed quota are distributed to subaccounts, and then consumed by the subaccounts. When quota is freed at the subaccount level, it becomes available again at the global account level.

Optionally, you can assign service plans and quota from the global account to directories, and from there distribute these assignments to subaccounts under the directory. Before you can assign plans and quota to

a directory, the directory must first be configured to manage entitlements. See Configure Entitlements and Quotas for Directories [page 2524]. When quota is freed at the subaccount level, it becomes available again at the directory level.

> **Note**
>
> Only global account administrators can configure entitlements and quotas for subaccounts and managed directories.
>
> Note that if you're assigning plans to a subaccount that is under a managed directory that also has the user management feature enabled, then you must be either the directory administrator or a global account administrator.

There are two places in the SAP BTP cockpit where you can view entitlements and assign service plans and quotas for subaccounts - at the global account level and at the subaccount level. Depending on your permissions, you may only have access to one of these pages. You can find more details below:

Entitlements Pages

Page in cockpit	Navigation Level	Visible to	Permissions
Entitlements > *Entity Assignments*	Global account level	Global account administrators only	• View & Edit - Global account administrators
Entitlements > *Service Assignments*	Global account level	Global account administrators only	• View - Global account administrators (you cannot make changes to entitlements or quota assignments from this page)
Entitlements	Subaccount level	Subaccount members (regardless of whether they are also global account administrators or not)	• View - Subaccount members who are not global account administrators • View & Edit - Global account administrators who are also subaccount members

The *Entitlements* > *Entity Assignments* is where you view and edit assignments and quota from the perspective of subaccounts and directories. This page contains these views (tabs):

View	Description
Manage Assignments	Allows global account admins to distribute the entitled service plans of their global account to subaccounts and directories.
View Commercial Information	Allows global account admins and viewers to view the commercial source of their service entitlements. This feature is not available for trial accounts.

For information about setting assignments and quota, see:

- Configure Entitlements and Quotas for Directories [page 2524]
- Configure Entitlements and Quotas for Subaccounts [page 2529]

The *Entitlements* > *Service Assignments* is where you view assignments and quota from the perspective of services. This page contains these views (tabs):

View	Description
All Services	Provides an overview of all available service entitlements of your global account. **→ Tip** The *Assigned To* column in this view shows you the total number of directories and subaccounts to which each service plan is assigned in your global account. You can click on the links in this column to quickly navigate to the selected service in the *Assignments by Service* view and display more details about these assignments. Turn on the *Show commercial info* option to display additional columns in the main table. These columns can help you to gain a better understanding of the relationship between contractual service entitlements and the technical assets (services and plans) in your global account.
Assignments by Service	Allows you to choose specific services and see how their service plans are assigned to subaccounts and directories within your global account. **→ Tip** Turn on the *Show commercial info* option to display additional columns in the main table. These columns can help you to gain a better understanding of the relationship between contractual service entitlements and the technical assets (services and plans) in your global account.

ⓘ Note

- To subscribe to a multitenant application, you must first assign its plan to the specific subaccount (in one of the *Entitlements* pages). If you remove a subaccount's entitlement to a multitenant application, any subscriptions to it from that subaccount will stop working.
- Before a subaccount admin can enable a quota-based environment, such as Kyma, the subaccount admin must first assign the environment's plan to the subaccount (in one of the *Entitlements* pages). Other non-quota-based environments, such as Cloud Foundry, are available by default to all subaccounts, and therefore are not available as entitlements.

In the Cloud Foundry environment, you can further distribute the quotas that are allocated to a subaccount. This is done by creating space quota plans and assigning them to the spaces. For more information on space quota plans in the Cloud Foundry environment, see:

- Managing Space Quotas [page 2806]
- https://docs.cloudfoundry.org/adminguide/quota-plans.html

→ Tip

Optionally, you can configure assignments and quotas for supported services that you own or subscribe to from supported non-SAP cloud vendors. These services can be consumed in SAP BTP in your subaccounts alongside your self-developed services and those provided by SAP. This functionality requires first setting

up a resource provider instance for your provider account in the cockpit. For more information, see Managing Resource Providers [page 2468].

> ⓘ Note
>
> In global accounts that use the consumption-based commercial model, a default runtime memory quota of 200 GB for the standard plan is automatically assigned to every subaccount. This quota represents a limit on how much runtime memory the applications in the subaccount can use at any given time. When you use the standard plan for SAP BTP, Cloud Foundry runtime in the consumption-based commercial model, you're billed based on the amount of runtime memory that has actually been consumed. For more information, see Service Plans and Metering for Cloud Foundry Runtime and Consumption Monitoring.
>
> A global account administrator can change the default quota as described in (optional) Adjusting Runtime Memory Quota for Cloud Foundry Runtime (Consumption-Based).

Related Information

Entitlements and Quotas [page 127]
Configure Entitlements and Quotas for Subaccounts [page 2529]
Configure Entitlements and Quotas for Directories [page 2524]
Create Space Quotas [page 2808]
Create Space Quota Plans Using the Cloud Foundry Command Line Interface [page 2857]
Tutorial: Manage Entitlements on SAP BTP Trial
Troubleshooting: Entitlement and Quota Problems

6.1.1.7.1 Configure Entitlements and Quotas for Directories

Distribute entitlements that are available in your global account to directories by adding service plans and their allowed quotas by using SAP BTP cockpit.

Enable Entitlement Management for a Directory

Before you can assign plans and quotas to a directory so that they can be further distributed to the subaccounts in the directory, you must first enable the **entitlement management** feature for the directory. This configuration only needs to be done once for each directory. In this documentation, we'll refer to these types of directories as "managed directories".

> ⓘ Note
>
> - In the account hierarchy, each directory path can have only one directory enabled with the entitlement management feature (or the user management feature). Consequently, if a directory in the path

already has one of these features enabled, it is not possible to enable either of them for other parent or child directories within the same path as this directory.

- Enabling entitlement management for a directory that already contains subaccounts will result in the automatic assignment of any existing assignments and quotas (for the subaccounts) to the directory.

Prerequisites

- You are a global account administrator.

Procedure

- There are several ways you can enable the entitlement management feature for a directory:
- In the *Account Explorer*, click the ••• *Actions* button for a specific directory and select *Edit*. In the *Edit Directory* dialog box, under *Advanced*, choose the *Enable entitlement management* option.
- Navigate to the directory's *Entitlements* page and select *Enable Entitlement Management*.
- Navigate to your global account and open the *Entitlements* > *Entity Assignments* from the navigation panel. Select the icon in the *Subaccounts/Directories* dropdown menu.

  Then, in the *Select Subaccounts and Directories* dialog box, choose the directory for which you want to manage assignments. Click *Select* and then *Enable Entitlement Management*.

  → Tip

  To disable entitlement management for a directory, redo these steps from the global account's *Entitlements* > *Entity Assignments* page, but choose *Disable Entitlement Management* at the end. When you disable entitlement management, any assignments and quota that are currently assigned to the directory are "returned" to the global account and are then available for distribution to other directories and subaccounts.

Next Steps

Once you've enabled entitlement management for a directory, you can now configure its service plan and quota assignments as described below.

Configure Assignments and Quotas for Managed Directories

Use this procedure to configure the service plan assignments and quotas for one or more managed directories.

> **Note**
>
> To get an overview of all the services and plans available in your global account, navigate to the *Entitlements* > *Service Assignments* page. There you can see the global usage of each service plan in the *All Services* view, as well as the detailed assignments of each service across subaccounts and directories in the *Assignments by Service* view.
>
> - In the *All Services* view of this page, the *Assigned To* column shows you the total number of directories and subaccounts to which each service plan is assigned in your global account. You can click on the links in this column to quickly navigate to the selected service in the *Assignments by Service* view and display more details about these assignments.
> - You can turn on the *Show commercial info* toggle option in the *Service Assignments* page to display additional table columns that can help you to gain a better understanding of the relationship between contractual service entitlements and the technical assets (services and plans) in your global account; in other words, how each entitled plan was added to your global account.
> - The *Service Assignments* page is for viewing purposes only; you cannot make any assignment changes from it.
>
> You can also access the *Entitlements* > *Service Assignments* page from the directory level (only for directories that are configured to manage entitlements) to display the directory's assignments.

Prerequisites

- You are a global account administrator.
  Note that if the directory also has the user management feature enabled, then you must be either the directory administrator or a global administrator. See Manage Users in Directories [page 2505].
- The service and applications that you intend to assign to your directories must be entitled to your global account.
  - For enterprise accounts that use the subscription-based commercial model, your account receives only the services that you've already purchased and subscribed to according to your SAP BTP contract. To access additional services, at an extra cost, you can modify your contract via your sales representative or account executive.
  - For enterprise accounts that use the consumption-based commercial model, your global account receives all pay-for-use and free services that are eligible for this commercial model.
  - If you have a trial account, you'll receive a restricted set of free platform resources and services for a limited time period. To access additional services, you'll have to switch to an enterprise account.

Procedure

1. Navigate to your global account.
2. Choose *Entitlements* > *Entity Assignments* from the left hand-side navigation.

   Alternatively, you can navigate to a directory from the *Account Explorer*, and then access the *Entitlements* > *Entity Assignments* page for that specific directory.
3. Open the *Manage Assignments* view (tab).

   Skip this step if you have a trial account.

   > → Tip
   >
   > You can use the *View Commercial Information* view in this page to display comprehensive commercial information about the entitlements in your global account (you cannot make assignments in this view). Note that this view is not available for trial accounts.
4. Select the icon in the *Directories / Subaccounts* dropdown menu.
5. In the *Select Subaccounts and Directories* dialog box, choose the directories for which you want to manage assignments.
6. Click *Select*.

   > ⓘ Note
   >
   > If you select a directory for which entitlement management is not enabled, you can click *Enable Entitlement Management* to enable the feature. Remember that you can enable this feature only for a single directory in a path.

   You get a table for each of the directories you selected, displaying the current assignments and quotas. You can choose *Show subaccount assignments* next to the table title to see the assignments and quotas for the subaccounts in a specific directory.

   > → Tip
   >
   > Under the *Service Technical Name* column, click on the ⓘ *(Service Details)* icon of any service to access its service-related links to documentation, support information, and SAP Discovery Center.
7. Choose *Edit* to start editing assignments for a particular directory.

   > ⓘ Note
   >
   > You can only edit assignments for one directory at a time.
8. You can now edit the assignments table:

Action	Steps
Add new service plans to the directory	Choose *Add Service Plans* and from the dialog box select the services and the plans from each service that you would like to add to the directory. Choose *Add <x> Service Plans* in the dialog box to confirm.

Action	Steps
	Remember, the services that you see depend on the type of global account you have and your contract details (see the prerequisites above for more information). Once you've added new service plans, you can also change the quota for those plans as described in the following row and enable the option to auto-assign quota to new subaccounts that are added to the directory. For service plans where quota can be increased or decreased, you can also specify what amount should be auto-assigned to each subaccount that is created or added to the directory. **Note** To subscribe a subaccount to a multitenant application, you must first assign the application to the specific subaccount using the process described in Configure Entitlements and Quotas for Subaccounts [page 2529]. **→ Tip** If your global account is configured to consume remote services from a non-SAP cloud vendor (resource provider), an additional dropdown list is displayed in the *Service Details* pane, where you can choose a configured resource provider. You can then choose the required service plans that are available for the selected resource provider to add them to the subaccount. For more information, see Managing Resource Providers [page 2468].
Edit the quota for one or more service plans	Use + *(Plus)* and − *(Minus)* in the *Quota Assignment* column to increase or decrease the quota for each service plan. Note some service plans do not have a numeric quota assignment. In other words, you do not assign a specific quantity but grant access to the plan by simply assigning it to the subaccount. See *Quotas* in Managing Entitlements and Quotas Using the Cockpit [page 2521]. **→ Tip** If you would like to have quota automatically assigned to subaccounts that are created or moved to the directory, select the *Auto-Assign to Subaccounts* checkbox and follow the same process to set the amount that should be auto-assigned to each subaccount. • The auto-assign feature doesn't apply to subaccounts that are already in the directory when the feature is enabled. • You cannot use the auto-assign feature with beta services and applications.
Set a quota limit to unlimited service plans that use the consumption-based commercial model	The usage of most service plans is unlimited by default in global accounts that use the consumption-based commercial model. To control costs of such plans, you can set a quota limit on the service plans for this directory by first selecting the checkbox in the *Set Quota Limit* column. Now, you can increase or decrease the quota for this service plan in the *Quota Assignment* column by using + *(Plus)* and − *(Minus)*.

Action	Steps
	Note some service plans do not have a numeric quota assignment. In other words, you do not assign a specific quantity but grant access to the plan by simply assigning it to the directory (and then subsequently to the subaccount). See *Quotas* in Managing Entitlements and Quotas Using the Cockpit [page 2521].
Delete a service plan and its quota from the directory	Choose *(Remove Service Plan)* from the *Actions* column.

9. Once you're done, choose *Save* to save the changes and exit edit mode for that directory.
10. **Optional:** Repeat steps 7 to 9 to configure assignments for the other selected directories.

Related Information

Entitlements and Quotas [page 127]
Directories [page 123]
Manage Users in Directories [page 2505]
Create a Directory [page 2502]
Managing Entitlements and Quotas Using the Cockpit [page 2521]
Configure Entitlements and Quotas for Subaccounts [page 2529]
Using Free Service Plans [page 118]
Trial Accounts and Free Tier [page 109]

6.1.1.7.2 Configure Entitlements and Quotas for Subaccounts

Distribute the entitlements that are available in your global account by adding service plans and their allowed quotas to your subaccounts using SAP BTP cockpit.

Prerequisites

- You must be a global account administrator to configure subaccount assignments.
  Note that if the subaccount is under a directory that is configured to manage users, then you must be either the directory administrator or a global account administrator to configure the subaccount's assignments.
- The service and applications that you intend to assign to your subaccounts must be entitled with the required quota to your global account. If you're assigning a service plan to a subaccount, which is under a directory that is configured to manage entitlements, then the managed directory must have the relevant service plans and needed quota assigned to it so that you can distribute them to the directory's subaccounts, as required.

- For enterprise accounts that use the subscription-based commercial model, your account receives only the services that you've already purchased and subscribed to according to your SAP BTP contract. To access additional services, at an extra cost, you can modify your contract via your sales representative or account executive.
- For enterprise accounts that use the consumption-based commercial model, your global account receives all pay-for-use and free services that are eligible for this commercial model.
- If you have a trial account, you'll receive a restricted set of free platform resources and services for a limited time period. To access additional services, you'll have to switch to an enterprise account.

- To assign beta service plans, your subaccount must have the *Enable beta features* option selected. See Change Subaccount Details [page 2516]. Only applies to non-production subaccounts.

Context

You can distribute assignments and quotas across subaccounts within a global account from these places in the cockpit:

- The *Entitlements* > *Entity Assignments* page at the global account level (visible only to global account administrators).
- The *Entitlements* > *Entity Assignments* page at the directory level by first navigating into the directory from the *Account Explorer*. This applies only to subaccounts that reside in the path of a directory that is configured to manage entitlements.
- The *Entitlements* page at the subaccount level (visible to all subaccount members, but editable only by global account administrators) by first navigating into the subaccount from the *Account Explorer*.

For more information, see Managing Entitlements and Quotas Using the Cockpit [page 2521].

> → Tip
>
> To get an overview of all the services and plans available in your global account, navigate to the *Entitlements* > *Service Assignments* page. There you can see the global usage of each service plan in the *All Services* view, as well as the detailed assignments of each service across subaccounts and directories in the *Assignments by Service* view.
>
> - The *Service Assignments* page is for viewing purposes only; you cannot make any assignment changes from it.
> - In the *All Services* view of this page, the *Assigned To* column shows you the total number of directories and subaccounts to which each service plan is assigned in your global account. You can click on the links in this column to quickly navigate to the selected service in the *Assignments by Service* view and display more details about these assignments.
> - You can turn on the *Show commercial info* toggle option in this page to displays additional table columns that can help you to gain a better understanding of the relationship between contractual service entitlements and the technical assets (services and plans) in your global account; in other words, how each entitled plan was added to your global account.
>
> If you navigate to a subaccount from the *Account Explorer*, you view its assignments in its *Overview* page view.

> ### Note
>
> - To subscribe to a multitenant application, you must first assign it to the specific subaccount using the same process explained below. If you remove a subaccount's assignments to a multitenant application, any subscriptions to it from that subaccount stop working.
> - Before a subaccount admin can enable a quota-based environment, such as Kyma, the subaccount admin must first assign the environment's plan and quota to the subaccount. Other non-quota-based environments, such as Cloud Foundry, are available by default to all subaccounts, and therefore aren't available as entitlements.

Configure Assignments and Quotas from Your Global Account

Procedure

1. Navigate to your global account.
2. Choose *Entitlements* > *Entity Assignments* from the left hand-side navigation.

   > ### → Remember
   >
   > There are other ways you can access the subaccount to manage its assignments, such as navigating to the subaccount or its managed directory from the *Account Explorer*. See above for more information.

3. Open the *Manage Assignments* view (tab).

   Skip this step if you have a trial account.

   > ### → Tip
   >
   > You can use the *View Commercial Information* view in this page to display comprehensive commercial information about the entitlements in your global account (you cannot make assignments in this view). Note that this view is not available for trial accounts.

4. Select the icon in the *Directories / Subaccounts* dropdown menu.
5. In the *Select Subaccounts and Directories* dialog box, select all the subaccounts that you want to manage assignments.
6. Choose *Select* to apply the filter.

   You get a table for each of the subaccounts you selected, displaying the current assignments and quota.

   > ### → Tip
   >
   > Under the *Service Technical Name* column, click on the *(Service Details)* icon of any service to access its service-related links to documentation, support information, and SAP Discovery Center.

7. Choose *Edit* to start editing assignments for a particular subaccount.

   > ### Note
   >
   > You can only edit assignments for one subaccount at a time.

8. You can now edit the assignments table:

Action	Steps
Add new service plans to the subaccount	Choose *Add Service Plans* and from the dialog box, select the services and the service plans from each service that you would like to add to the subaccount. Choose *Add <x> Service Plans* in the dialog box to confirm. Remember, the services that you see depend on the type of global account you have and your contract details (see the prerequisites above for more information). → Tip If your global account is configured to consume remote services from a non-SAP cloud vendor (resource provider), an additional dropdown list is displayed in the *Service Details* pane, where you can choose a configured resource provider. You can then choose the required service plans that are available for the selected resource provider to add them to the subaccount. For more information, see Managing Resource Providers [page 2468].
Edit the quota for one or more assigned service plans	Use + *(Plus)* and − *(Minus)* in the *Quota Assignment* column to increase or decrease the quota for each service plan. Note some service plans do not have a numeric quota assignment. In other words, you do not assign a specific quantity but grant access to the plan by simply assigning it to the subaccount. See *Quotas* in Managing Entitlements and Quotas Using the Cockpit [page 2521].
Set a quota limit to unlimited service plans that use the consumption-based commercial model	The usage of most service plans is unlimited by default in global accounts that use the consumption-based commercial model. To control costs of such plans, you can set a quota limit on the service plans for this subaccount by first selecting the checkbox in the *Set Quota Limit* column. Now, you can increase or decrease the quota for this service plan in the *Quota Assignment* column by using + *(Plus)* and − *(Minus)*.
Delete a service plan and its quota from the subaccount	Choose 🗑 *(Remove Service Plan)* from the *Actions* column.

9. Once you're done, choose *Save* to save the changes and exit edit mode for that subaccount.
10. **Optional:** Repeat steps 7 to 9 to configure assignments for the other selected subaccounts.

Related Information

Entitlements and Quotas [page 127]

Configure Assignments and Quotas from a Subaccount

If you're working in a subaccount and realize you're missing assignments or quota, you can edit the assignments directly from that subaccount.

Prerequisites

In addition to being a global account administrator, you must also be a member of the subaccount to be able to access that subaccount.

Procedure

1. Navigate into the subaccount where you would like to configure the assignments.
2. Choose *Entitlements* from the left hand-side navigation to see the assignments for your subaccount.
3. Choose *Edit*.
4. You can now edit the assignments table as described in the procedure above.
5. Once you're satisfied with the changes, choose *Save* to save and exit edit mode.

Related Information

Entitlements and Quotas [page 127]
Managing Entitlements and Quotas Using the Cockpit [page 2521]
Configure Entitlements and Quotas for Directories [page 2524]
Using Free Service Plans [page 118]
Trial Accounts and Free Tier [page 109]

6.1.1.8 Subscribe to Multitenant Applications Using the Cockpit

Subscribe to multitenant applications from the *Subscriptions* page in the SAP BTP cockpit.

Prerequisites

- If you're subscribing to an application provided by SAP:
  - If your global account uses the subscription-based commercial model, then you must have purchased Software-as-a-Service (SaaS) licenses for the applications you want to consume. See https://cloudplatform.sap.com/pricing.html. You can also contact us on SAP BTP or via an SAP sales representative.

    > ⓘ Note
    >
    > If your global account is licensed with the consumption-based commercial model, then you are eligible to subscribe to any multitenant application that is available to this model.

  - You have created a multi-environment subaccount. See Create a Subaccount [page 2509].
- If you are an application owner who is subscribing consumer tenants to your own multitenant application that you created in the Cloud Foundry environment:
  - The application must be deployed to your provider subaccount, configured, and registered as described in Developing Multitenant Applications in the Cloud Foundry Environment [page 401].
  - You have created a multi-environment subaccount for each application consumer in the region in which the application is deployed. See Providing Multitenant Applications to Consumers in the Cloud Foundry Environment [page 419].
- You are an admininistrator of the subaccount.
- Your subaccount is entitled to the multitenant application. See Managing Entitlements and Quotas Using the Cockpit [page 2521].

Context

The instructions provided here apply whether you are an SAP customer subscribing to an application provided by SAP or you are an application owner who is sharing your multitenant application with your consumers.

Procedure

1. Open your global account in the cockpit.
2. Do one of the following:
   - If you are subscribing to an SAP application, open your subaccount.

- If you are sharing your multitenant application with other consumers in your global account, open the subaccount of each consumer.

See Navigate in the Cockpit [page 2461].

3. In the navigation area of each subaccount, choose *Services* > *Service Marketplace*.

   All resources you are entitled to consume in the subaccount appear as tiles.

   > ### Note
   >
   > To create a subscription, you need to open the *Create* wizard. There are 2 ways to reach this wizard, either from the *Service Marketplace* or from the *Instances and Subscriptions* page.
   >
   > In steps 3, 4, 5, and 6, we describe how to reach the wizard from the *Service Marketplace* page.
   >
   > This way is useful if you want to get an overview of all the applications your subaccount is entitled to use and if you want to get more details about the available plans before you create a subscription.
   >
   > If you already know these details, select *Services* > *Instances and Subscriptions*, click on *Create* in the top-right corner, and skip to step 7 to learn about the wizard.

4. In the filters area, select the *All Types* filter, and from the dropdown menu, select *Application*.

   You can use other filters to narrow down your search by the environments for which the application is available, capabilities, and its status.

   Active status means you are currently subscribed to that application.

5. Click the application name to open its *Overview* page.

   There, you can find more information about the application including all the available plans.

   > ### → Tip
   >
   > You can also create a new subscription without opening its overview by selecting the three dots in the top-right corner of the tile.

6. Choose *Create*.
7. In the wizard that opens, select one of the plans of type subscription and choose *Create*.

   > ### Note
   >
   > The wizard contains more than one step if additional subscription configuration input parameters are provided by the app provider.

   A confirmation popup appears with the link to the *Instances and Subscriptions* page, where you can follow the status of your new subscription under the *Subscriptions* table.

   The *Go to Application* link becomes available once the subscription is activated. Choose the link to launch the application and obtain its URL.

   To find the link, either click on the application's name or click on the *(Go to Application)* icon in the *Application* column.

> → Tip
>
> To update an existing subscription plan, select the three dots at the end of the subscription row, and from the menu, select *Update*.

> ⓘ Note
>
> You can update a subscription plan only if additional plans for the subscription are entitled to the subaccount you're using and if your subscription is eligible for a plan update.

> → Tip
>
> To remove an existing subscription, select the three dots at the end of the subscription row, and from the menu, select *Delete*. All data related to the multitenant application will be deleted in the respective subaccount.

8. **Optional:** Assign labels to the subscription to make it searchable and organized within the subaccount according to various criteria.
   a. In the navigation area, choose *Services* > *Instances and Subscriptions*.
   b. Find the subscription to which you want to assign labels under the *Subscriptions* section of the page.
   c. Select the actions (•••) menu and from the dropdown list, choose *Add Labels*.

      > ⓘ Note
      >
      > If there are already labels assigned to the subscription, the action becomes *Change Labels*.
      >
      > You can see the existing labels under the *Labels* column.
      >
      > If you do not see the column, unhide it by clicking on ⚙ *(Configure Table Columns)*.

   For more information, see Labels [page 125].

Next Steps

1. To make sure that the users can access the application, assign the relevant authorizations. Most applications manage authorizations using role collections in the SAP Authorization and Trust Management service. Other applications use authorization policies in SAP Cloud Identity Services.
   For role collections, see Assigning Role Collections to Users or User Groups [page 2616].
   For authorization policies, see Assign Authorization Policies.
2. If the application consumer isn't the subaccount owner or administrator, provide the consumer-specific URL of the application to the consumer or the users of the application.

Related Information

Working with Multitenant Applications Using the BTP CLI [page 2704]

6.1.1.9 Security Administration: Managing Authentication and Authorization

This section describes the tasks of administrators of SAP BTP. Administrators ensure user authentication and assign authorization information to users and user groups.

Since identity providers provide the users or user groups, you make then sure that there is a trust relationship between your subaccount and the identity provider. This is a prerequisite for authentication. You can manage the authorizations of the users.

Authentication

A user account corresponds to a particular user in an identity provider. The user is always authenticated using an external identity provider.

> → Recommendation
>
> We recommend to use a custom tenant of SAP Cloud Identity Services. You can connect SAP Cloud Identity Services to your corporate identity provider.

SAP BTP distinguishes two types of users. Platform users are usually administrators, operator, or developers. They have full access and give permissions at global account, directory, and subaccount level. Business users use the applications deployed to SAP BTP. They are, for example, end users of SaaS apps or of custom applications.

For more information, see the related links.

Authorization

Application developers create and deploy application-based authorization artifacts for business users. Administrators use this model to manage roles, build role collections, and assign these collections to users or user groups. In this way, they control the users' permissions.

To perform the functions related to authorization artifacts, account administrators can have multiple options. Here are some of the options:

- The SAP BTP cockpit covers all authorization functions. Its user interface offers easy-to-use and clear navigation.
- There is also a command line option to manage most authorization artifacts. If you prefer working in a terminal or automating operations, use the SAP BTP command line interface (BTP CLI). It's suitable for repetitive tasks.
- Especially if you need to perform bulk operations or programmatically access the authorizations, we recommend to use the REST API for authorizations of the SAP Authorization and Trust Management service.
- Administrators can also use the Terraform Provider for SAP BTP within Infrastructure as Code to manage some of the authorization functions.

You find the all available options and tools for managing authorizations in the account administration overview. See Account Administration [page 2457].

Setting Up Authorization Artifacts (Account Administrators)

Task	Links
Assign the role collection to the users provided by an identity provider	Working with Role Collections [page 2612]
(If you do use a custom identity provider) Assign the role collections to user groups	Map Role Collections to User Groups [page 2621]
Assign the role collections to users and user groups, manage attribute mappings	Mapping Role Collections [page 2620]
Create a role collection and assign roles to it	Define a Role Collection [page 2614]
Use an existing role or create a new one using role templates	Add Roles to Role Collections on the Application Level [page 2632]

> **Note**
>
> When users log on, their authorizations are stored in each user's current session. These authorizations are not dynamically updated and are removed from there only when the session is terminated. This means that, after changes of role collection assignments of a user, these changes only become effective after the user logged out and logged on again.

Related Information

Trust and Federation with Identity Providers [page 2538]
Monitoring and Troubleshooting [page 3552]
SAP Authorization and Trust Management Service [page 3538]
Default Identity Provider [page 2594]
User and Member Management [page 133]

6.1.1.9.1 Trust and Federation with Identity Providers

When setting up accounts you need to assign users. While we provide you with your first users from the default identity provider to get you started, your organization has identity providers that you want to integrate.

SAP BTP supports identity federation, a concept of linking and reusing digital identities of a user base across loosely coupled systems. Identity federation frees applications on SAP BTP as well as the platform itself from the need to obtain and store the credentials of users and to authenticate them. Instead, the user base is reused from identity providers, which support the administration of digital user identities, authentication, and authorizations in a centralized and decoupled manner. To enable communication between SAP BTP

and identity providers, you must cross-configure the communication endpoints of the involved systems, establishing a trust relationship between them.

→ Recommendation

We strongly recommend that you use a custom tenant of SAP Cloud Identity Services. Using SAP Cloud Identity Services eases integration with other SAP solutions. When setting up accounts you need to assign users. We provide you with your first users from the default identity provider. By using SAP Cloud Identity Services as a proxy to your corporate identity provider and hosting your users there, you gain a number of advantages:

- Integrate the management of these users with your broader identity management strategy, hosted on your own identity providers. You control your own user lifecycle and single sign-on strategies throughout the entire landscape.
- Enforce your own password and authentication policies, such as stronger passwords or multifactor authentication.

For platform users, the use of SAP Cloud Identity Services is mandatory. If you don't have a tenant yet, check Getting a Tenant.

To connect your corporate identity provider to SAP Cloud Identity Services, see Corporate Identity Providers and Configure Conditional Authentication for an Application in What Is Identity Authentication and SAP Cloud Identity Services

Identity Provider and the SAP Authorization and Trust Management Service in SAP BTP Architecture

SAP Cloud Identity Services is a multitenancy-enabled identity provider for all SAP cloud applications and optionally on-premise applications. The service provides capabilities for authentication, single sign-on, authorizations, identity lifecycle management, and on-premise integration as well as self-services like self-registration or password reset.

SAP has its own SAP Cloud Identity Services tenant, SAP ID service. SAP ID service is the default identity provider of SAP BTP and where you register to get initial access to SAP BTP. Trust to SAP ID service is preconfigured by default.

We recommend that you request your own SAP Cloud Identity Services tenant (see Getting a Tenant). To establish trust with your identity provider, proceed as follows.

For business users: Business Users [page 2540]

For platform users: Platform Users [page 2575]

Related Information

Map User Attributes from a Corporate Identity Provider for Business Users [page 2543]
Map User Attributes from a Corporate Identity Provider for Platform Users [page 2581]
Settings for Default SAML Federation Attributes of Identity Providers for Business Users [page 2554]

6.1.1.9.1.1 Business Users

User accounts enable users to log on to SAP BTP, access subaccounts, and to use applications according to the permissions granted to them. In this context, it's important to understand the difference between the two types of users that we refer to: business users and platform users.

Business users use the applications that are deployed to SAP BTP. For example, the end users of SaaS apps or services, such as SAP Build Work Zone, or end users of your custom applications are business users.

In the Cloud Foundry environment, application developers (platform users [page 2575]) create and deploy application-based security artifacts for business users. Administrators use these artifacts to assign roles, build role collections, and then assign those role collections to business users or user groups. The assignment of role collections enables administrators to control the permissions that the users have in the deployed applications.

For business users, there's a default identity provider [page 2594]. We expect that you have your own user base. We recommend that you configure the SAP Cloud Identity Services service as the identity provider and connect SAP Cloud Identity Services to your own corporate identity provider.

Related Information

Platform Users [page 2575]
Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2540]
Manually Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2547]
Establish Trust and Federation with UAA Using Any SAML Identity Provider [page 2551]
Using Multiple Identity Providers from the Same Subaccount [page 2556]

6.1.1.9.1.1.1 Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services

Use your SAP Cloud Identity Services tenant as an identity provider or a proxy to your own identity provider hosting your business users. This method avoids the upload and download of SAML meta data by using OpenID Connect (OIDC) to establish trust.

Prerequisites

- You have subaccount administrator permissions.
- You have a tenant of SAP Cloud Identity Services.
  For more information, see Get Your Tenant in the documentation for SAP Cloud Identity Services.

- The SAP Cloud Identity Services tenant is associated with the customer IDs of the relevant global account of SAP BTP.
  For more information, see Reuse SAP Cloud Identity Services Tenants for Different Customer IDs in the documentation for SAP Cloud Identity Services.

Context

Consider the following restrictions and tips before you start.

> ⚠ Restriction
>
> - You can only establish trust with a single tenant of SAP Cloud Identity Services per subaccount using this method.
> - Your SAP Cloud Identity Services tenant can be changed only when no SAP Cloud Identity Services-based subscriptions (for example, SAP Build Work Zone using authentication with SAP Cloud Identity Services, SAP Build Apps, or SAP Integration Suite, advanced event mesh) exist.
> - If you have an SAML trust configuration for a certain tenant already, you can't add an OIDC trust configuration to the same tenant in addition. Instead, migrate the SAML trust configuration to OIDC (see Migration from SAML Trust to OpenID Connect Trust with SAP Cloud Identity Services [page 2561]).
>   Consider the upper limits for trust configurations in the subaccount. See Limits for the Subaccount [page 3606].

> → Tip
>
> - We recommend that you use SAP Cloud Identity Services as identity provider and connect a potential corporate identity provider there. If you don't have a tenant yet, check Getting a Tenant.
>   For more information, see Corporate Identity Providers and Configure Conditional Authentication for an Application in What Is Identity Authentication and SAP Cloud Identity Services
> - We provide APIs so you can perform this procedure programmatically. For more information, see the Identity Provider Management API on SAP Business Accelerator Hub.

Procedure

1. In the SAP BTP cockpit, go to your subaccount (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Trust Configuration*.
2. Choose *Establish Trust*.
   The *Choose Tenant* wizard opens.
3. Choose the SAP Cloud Identity Services tenant and choose *Next*.

   The identity providers listed are the SAP Cloud Identity Services tenants associated with your customer ID.

   For more information, see the preceding prerequisites.
4. You can change the name and the description of the tenant, display and change the origin key.

The origin key can have 36 characters at maximum. Only use the following characters for the origin key of the trust configuration.

`aA–zZ`, `0–9`, `-` (hyphen), _ (underscore), `.` (period)

5. Choose the domain configured with the SAP Cloud Identity Services tenant and continue with *Next*. It's also possible to use `Default` as domain.
6. *Available for User Logon* is switched on. Provide a link text for user logon (see Using Multiple Identity Providers from the Same Subaccount [page 2556]).
7. If unsure, keep the parameter *Create Shadow Users on User Logon* switched on. After logon, SAP BTP automatically creates a shadow user. Logon failures would occur if the shadow user doesn't exist. An administrator can then assign role collections to the shadow user.
8. If unsure, keep the parameter *Default for Noninteractive Logon* as suggested by the wizard (only for logon to a command line interface). If you switch on this parameter, it sets another identity provider as default for noninteractive logon (for more information,see `btp update security/settings`. Continue with *Next*.
9. Review your configuration and confirm using *Finish*.

Results

You've configured trust in your tenant of the SAP Cloud Identity Services, which is your identity provider. SAP Cloud Identity Services creates an application with the prefix *SAP BTP subaccount* and the display name of your subaccount in the administration console for SAP Cloud Identity Services. Some attributes of the application are managed by SAP, such as the application type, and are read-only.

SAP BTP automatically creates applications in SAP Cloud Identity Services for subscriptions to SAP BTP applications and service instances of the Identity service of SAP BTP. Newly created applications in SAP Cloud Identity Services are automatically set as child applications of the application that represents the relevant SAP BTP subaccount. Child applications inherit configurations such as the subject name identifier, authentication configuration, and branding from the parent application representing the subaccount.

Example

If your subaccount is named *My Subaccount*, the resulting application in SAP Cloud Identity Services would be *SAP BTP subaccount My Subaccount*.

Older applications start with *XSUAA_*.

→ Tip

To troubleshoot problems with tokens from SAP Cloud Identity Services, see Logging OpenID Connect Tokens in the documentation for SAP Cloud Identity Services.

⚠ Restriction

If the OIDC issuer is changed in the trust configuration, the trust breaks. To update the trust configuration on the SAP BTP side, open the details of the trust configuration and choose *Refresh*. This button refreshes the trust configuration to reflect changes in the SAP Cloud Identity Services tenant, for example the issuer value.

Next Steps

If you don't need the default identity provider anymore, set it to inactive or hide the logon link.

Related Information

Default Identity Provider [page 2594]

6.1.1.9.1.1.1.1 Map User Attributes from a Corporate Identity Provider for Business Users

When you enable trust with a tenant of SAP Cloud Identity Services, you get an OpenID Connect (OIDC) application in SAP Cloud Identity Services to represent your subaccount, in the context of business users. When SAP Cloud Identity Services authenticates users using a corporate identity provider, map the user attributes provided by the corporate identity provider to the attributes required by your applications.

Finding the Application for Your Subaccount

The name of the application in the administration console of SAP Cloud Identity Services that represents your subaccount has the prefix **SAP BTP subaccount** or `XSUAA_` and the display name of your subaccount.

If your subaccount is named `My Subaccount`, the resulting application in SAP Cloud Identity Services is **SAP BTP subaccount My Subaccount** or `XSUAA_My Subaccount`.

Customizing Attribute Mappings

There are several options to customize attribute mappings in SAP Cloud Identity Services, depending on whether identity federation is enabled or disabled.

For more information about identity federation, see Configure Identity Federation.

- When identity federation is disabled, SAP Cloud Identity Services always propagates all the attributes received from the corporate identity provider to all the applications, on a 1:1 basis. You have the following options:
  - Configure the corporate identity provider to directly send the attributes.
  - If needed, use **enriched token claims** or **enriched assertion attributes** (depending on whether the corporate identity provider is connected with SAML or OIDC) to map the attributes sent by the corporate identity provider, to the attribute names needed by SAP BTP.
    Enriched token claims or **enriched assertion attributes** add additional attributes, either based on the corporate identity provider attributes (for example, by renaming them) or on static values.

- When identity federation is enabled, SAP Cloud Identity Services doesn't automatically propagate any attributes from the corporate identity provider to the application. This option requires mappings in the SAP Cloud Identity Services application, for each attribute that is needed by the application.
  Use the *Attributes* in the SAP Cloud Identity Services application representing your subaccount.
  SAP BTP expects the following attributes. The default configuration of the trust configuration sets up the values in the following table in the SAP Cloud Identity Services tenant.

Self-Defined Attributes	Identity Directory
`email`	`Email`
`family_name`	`Last Name`
`given_name`	`First Name`
`groups`	`Groups`
`user_uuid`	`Global user ID`

  If the corporate identity provider sends user attributes with other names, set the source to the corporate identity provider and the value to the correct attribute name.

  Example

  If your corporate identity provider sends users' last names as the `sn` attribute, add the corporate identity provider as source to the `last_name` attribute with the value `sn`.

  Customized Last Name Attribute Configuration in SAP Cloud Identity Services

Attribute Name	Source	Attribute Value
`email`	Identity Directory	`Email`
`family_name`	Identity Directory	`Last Name`
`given_name`	Identity Directory	`First Name`
`groups`	Identity Directory	`Groups`
`last_name`	Corporate Identity Provider	**Example** `sn`
`mail`	Identity Directory	`Email`
`user_uuid`	Identity Directory	`Global User ID`

  For more information, see User Attributes in the documentation of SAP Cloud Identity Services.

The subject name identifier attribute is used by SAP BTP to uniquely identify the application user.

For more information, see Configure the Subject Name Identifier Sent to the Application in the documentation of SAP Cloud Identity Services.

Default Configuration of ID Tokens

In the default configuration, the attributes provided in the ID token issued by SAP Cloud Identity Services are described in the following table.

Default Attributes of SAP Cloud Identity Services Tokens

User Attribute of SAP Cloud Identity Services	Assertion Attribute	Description
NA	`azp`	The authorized party identifies the client ID to which the ID token was issued.
`mail`	`email`	E-mail address of the subject. By default, this value is used for the subject name identifier. See the table *Default Configurations of the Subaccount in SAP Cloud Identity Services* following this table.
`lastName`	`family_name`	Last name of the subject.
`firstName`	`given_name`	First name of the subject.
`companyGroups`	`groups`	Any groups the subject is assigned to in the identity provider.
NA	`sap_id_type`	An SAP claim that identifies the type of token.

User Attribute of SAP Cloud Identity Services	Assertion Attribute	Description
`userUuid`	`user_uuid`	An identifier for a user that's unique across technology layers such as user interface, APIs, and security tokens, as well as across products and lines of businesses contributing to a business process in the Intelligent Enterprise. Business applications can use this identifier to correlate information about the user. While not necessary for platform users, the attribute doesn't hinder such users either. **Note** SAP Authorization and Trust Management service supports global user identifiers. When SAP Cloud Identity Services sends a global user identifier, it's included in the SAP Authorization and Trust Management service tokens, which means that you can use it in scenarios where you need to use global user identifiers. For more information, see Global User ID in Integration Scenarios.

Default Configuration of the SAP Cloud Identity Services Application

In the application of SAP Cloud Identity Services that represents the subaccount, the configuration sets defaults as shown in the following table.

Default Configurations of the Subaccount in SAP Cloud Identity Services

Configuration	Description
Subject name identifier	This attribute is used by the SAP Authorization and Trust Management service to identify the user for authentication. Default value: `E-Mail`. For more information, see Configure the Subject Name Identifier Sent to the Application in the documentation of SAP Cloud Identity Services.

Configuration	Description
List of allowed redirect URIs	The list of URIs to which SAP Cloud Identity Services is allowed to redirect from the application that represents your subaccount. Default value: `https://<subdomain>.authentication.<landscape>/login/callback/<origin>`. For example: `https://mysubdomain.authentication.us10.hana.ondemand.com/login.callback/sap.custom` For more information, see OpenID Connect Application Configurations in the documentation of SAP Cloud Identity Services.
Post logout redirect URIs	The list of URIs to which SAP Cloud Identity Services is allowed to direct users when logging out. Default value: `https://<subdomain>.authentication.<landscape>/*`. For example: `https://mysubdomain.authentication.us10.hana.ondemand.com/*` For more information, see OpenID Connect Application Configurations in the documentation of SAP Cloud Identity Services.

Related Information

Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2540]

6.1.1.9.1.1.2 Manually Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services

Use your SAP Cloud Identity Services tenant as an identity provider or a proxy to your own identity provider hosting your business users. Exchange SAML metadata to establish trust with the SAP Cloud Identity Services

tenant and then register your subaccount with the tenant. To complete federation, maintain the federation attributes of the user groups.

Context

> **Note**
>
> Instead of manually exchanging SAML metadata, we recommend that you use the automatic method based on the OpenID Connect (OIDC) protocol.
>
> For more information, see Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2540].

6.1.1.9.1.1.2.1 Establish Trust with an SAML 2.0 Identity Provider in a Subaccount

You want to use an SAML 2.0 identity provider, for example, SAP Cloud Identity Services. The identity provider authenticates business users for SAP BTP.

Prerequisites

- You have subaccount administrator permissions.
- You have downloaded the SAML 2.0 metadata file from the SAP Cloud Identity Services tenant using the URL `https://<sap_cloud_identity_services_tenant>.accounts.ondemand.com/saml2/metadata`.

  > **Example**
  >
  > `https://my-ias-tenant.accounts.ondemand.com/saml2/metadata`

  For more information, see Tenant SAML 2.0 Configuration in the SAP Cloud Identity Services.

Context

The following procedure describes how to establish trust with an SAP Cloud Identity Services tenant.

> **Restriction**
>
> You've already created a trust configuration with a custom identity provider for applications. In this case, you can't add a trust configuration with the same SAP Cloud Identity Services tenant using another protocol.

> ### Example
>
> If there's already a trust configuration with OpenID Connect, you can't create one using the SAML protocol.

> ### Restriction
>
> Consider the upper limits for trust configurations in the subaccount. See Limits for the Subaccount [page 3606].

> ### Restriction
>
> We recommend a single trust configuration per subaccount with SAP Cloud Identity Services using the OpenID Connect (OIDC) protocol. If this isn't sufficient, you may use a trust configuration with SAML 2.0.
>
> Nevertheless, we recommend migrating existing SAML 2.0 trusts according to the documentation Migration from SAML Trust to OpenID Connect Trust with SAP Cloud Identity Services [page 2561].

Procedure

1. In the SAP BTP cockpit, go to your subaccount (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Trust Configuration*.
2. Choose *Add SAML Trust*.
3. Enter a name and a description that make it clear to which identity provider that the trust configuration refers.

   > ### Note
   >
   > Make sure that the users who are supposed to log on to this identity provider understand the name of the trust configuration.

4. Download the relevant metadata (see prerequisites) and save it in an XML file.
5. Choose the *Upload* button to insert the SAML 2.0 metadata.
6. To validate the metadata, choose *Parse*. The system fills the *Subject* and *Issuer* fields with the relevant data from the SAP Cloud Identity Services - your SAML 2.0 identity provider. You see the fields when you choose *Show Details*.

   The name of the new trust configuration now shows the value `<sap_cloud_identity_services_tenant>.accounts.ondemand.com`. It represents the custom identity provider SAP Cloud Identity Services.

   This action also fills the fields for the single sign-on URLs and the single logout URLs.
7. Provide a link text for user logon (see Using Multiple Identity Providers from the Same Subaccount [page 2556]).
8. Choose *Add SAML Trust*.
9. To get the SAML metadata of your subaccount, choose *Download SAML Metadata*.
   You download an XML file with the SAML metadata of your subaccount. Its name is `saml-<subdomain>-sp.xml`. Use this file to import the SAML metadata into your identity provider.

Next Steps

Register your SAP BTP subaccount in the SAML 2.0 Identity Provider

6.1.1.9.1.1.2.2 Register SAP BTP Subaccount in the SAML 2.0 Identity Provider

An SAML service provider interacts with an SAML 2.0 identity provider to authenticate users signing in by means of a single sign-on (SSO) mechanism. In this scenario, the SAP Authorization and Trust Management service (XSUAA) acts as a service provider representing a single subaccount. To establish trust between an identity provider and a subaccount, you must register your subaccount by providing the SAML metadata to the identity provider. The identity provider is SAP Cloud Identity Services.

Context

Administrators must configure trust on both sides, in the service provider and in the identity provider. This description covers the side of the identity provider (SAP Cloud Identity Services). The trust configuration on the side of the SAP Cloud Identity Services must contain the following items:

- Metadata for web-based authentication or the relevant configuration information
  The metadata contains the configuration information, including the signing certificate and the required URLs. Create the metadata file of your subaccount using the *Download SAML Metadata* button in *Security* > *Trust Configuration* of your subaccount.
- Use e-mail as the unique name ID attribute, and map the user attribute `Groups` to the assertion attribute **`Groups`** (case-sensitive). This assertion attribute is required for the assignment of roles.

We illustrate the process of configuring trust in the service provider by describing how administrators use the administration console of SAP Cloud Identity Services to register the subaccount.

To establish trust from a tenant of SAP Cloud Identity Services to a subaccount, assign a metadata file and define attribute details. The SAML 2.0 assertion includes these attributes. With the SAP Authorization and Trust Management service as SAML 2.0 service provider, they're used for automatic assignment of SAP Authorization and Trust Management service authorizations based on information maintained in the identity provider.

Procedure

1. Open the administration console of SAP Cloud Identity Services.

   `https://<sap_cloud_identity_services_tenant>.accounts.ondemand.com/admin`
2. Choose *Applications & Resources* > *Applications* in the menu or the *Applications* tile.
3. To add a new SAML service provider, choose *Create*.

4. Enter the required data and choose *+ Create*.
    - Enter a name for the application that clearly identifies it as your new service provider.
      Users see this name in the logon screen when the authentication is requested by the SAP Authorization and Trust Management service. Seeing the name, they know which application they currently access after authentication.
    - Choose *SAML 2.0* as the protocol type.
5. Choose *Trust* > *SAML 2.0 Configuration*.
6. Under *Define from Metadata*, upload the relevant metadata XML file and choose *Save*.

    > ⓘ Note
    >
    > Use the `saml-<subdomain>-sp.xml` metadata file of your subaccount. You can create the file using the *Download SAML Metadata* button in *Security* > *Trust Configuration* of your subaccount.

    If the contents of the metadata XML file are valid, the parsing process extracts the information required to populate the remaining fields of the SAML configuration. It provides the name, the URLs of the assertion consumer service and single logout endpoints, and the signing certificate.
7. Choose *Default Name ID Format* and select *E-Mail* as a unique attribute. Save your changes.
8. Choose *Attributes*, use *Add* to add an attribute named **`Groups`** with a source **`Identity Provider`** and the value **`Groups`** (case-sensitive). Save your changes.

    > ⓘ Note
    >
    > Be aware that the X.509 certificates exposed in the SAML 2.0 metadata have specific validities. This applies to both SAML service providers and SAML identity providers. Authentication may fail with an error once the X.509 certificates expire.
    >
    > Ensure that the SAML 2.0 metadata is updated before the X.509 certificate expiry dates.
    >
    > Sign the certificate with a new signing key. For more information, see Rotate Signing Keys of SAML Token [page 2652].

6.1.1.9.1.1.3 Establish Trust and Federation with UAA Using Any SAML Identity Provider

Integrate trust between the SAP BTP subaccount and any SAML 2.0 identity provider.

Context

> → Tip
>
> Avoid multiple trust configurations in the same subaccount. We recommend that you use SAP Cloud Identity Services as identity provider and connect a potential corporate identity provider there. We strongly recommend OpenID Connect (OIDC). If you use SAML 2.0, connect an SAP Cloud Identity Services tenant and then add your corporate identity provider. If you don't have a tenant yet, check Getting a Tenant.

For more information, see the announcement of December 31, 2024 in the release notes.

To establish trust, configure the trust configuration of the SAML 2.0 identity provider in your subaccount using the SAP BTP cockpit. Next, register your subaccount in User Account and Authentication service using the administration console of your SAML 2.0 identity provider. To complete federation, maintain the federation attributes of the SAML 2.0 user groups. This makes sure that you can assign authorizations to user groups.

6.1.1.9.1.1.3.1 Establish Trust with Any SAML 2.0 Identity Provider in a Subaccount

You want to use an SAML 2.0 identity provider. This is where the business users for SAP BTP are stored.

Prerequisites

- You have subaccount administrator permissions.
- You have downloaded the SAML 2.0 metadata file from your SAML 2.0 identity provider using the relevant URL. For more information, see the documentation of your identity provider.

Context

You want to establish a trust relationship with an SAML 2.0 identity provider in your subaccount in SAP BTP. The following procedure guides you through the trust configuration.

Procedure

1. Go to your subaccount (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Trust Configuration* in the SAP BTP cockpit.
2. Choose *Add SAML Trust*.
3. Enter a name and a description that make it clear that the trust configuration refers to your identity provider.

   > **Note**
   >
   > Make sure that the users who are supposed to log on to this identity provider understand the name of the trust configuration.

4. To get the relevant metadata, get to the metadata of your identity provider.
5. Copy the SAML 2.0 metadata and paste it into the *Metadata* field.

6. To validate the metadata, choose *Parse*. Take care that the *Subject* and *Issuer* fields are filled with the relevant data from your SAML 2.0 identity provider.

   The name of the new trust configuration now shows the value of your identity provider. It represents your custom identity provider.

   Check whether the fields for the single sign-on URLs and the single logout URLs are filled.
7. Choose *Add SAML Trust*.
8. To get the SAML metadata of your subaccount, choose the *Download SAML Metadata* button.

   You download an XML file with the SAML metadata of your subaccount. Its name is `saml-<subdomain>-sp.xml`. Use this file to import the SAML metadata into your identity provider.

6.1.1.9.1.1.3.2 Register SAP BTP Subaccount in Any SAML 2.0 Identity Provider

An SAML service provider interacts with an SAML 2.0 identity provider to authenticate users signing in by means of a single sign-on (SSO) mechanism. In this scenario, the User Account and Authentication (UAA) service acts as a service provider representing a single subaccount. To establish trust between an identity provider and a subaccount, you must register your subaccount by providing the SAML details for web-based authentication in the identity provider itself.

Context

Administrators must configure trust on both sides, in the service provider and in the SAML identity provider. This description tries to guide you through the configuration of your identity provider. The trust configuration on the side of the SAM 2.0 identity provider must contain the following items:

- Metadata for web-based authentication or the relevant configuration information
  If available, the metadata contains the configuration information, including the signing certificate and the required URLs. You can create the metadata file of your subaccount using the *SAML Metadata* button in *Security* > *Trust Configuration* of your subaccount.
- Use e-mail as the unique name ID attribute, and map the user attribute `Groups` to the assertion attribute **`Groups`** (capitalized). This attribute is required for the assignment of roles.
  This makes sure that there is a trust relationship between your SAM 2.0 identity provider and the subaccount.

Your administrators use the administration console of your SAML 2.0 identity provider to register the subaccount.

To establish trust from a tenant of your identity provider to a subaccount, assign a metadata file and define attribute details. The SAML 2.0 assertion includes these attributes. With the UAA as SAML 2.0 service provider, they are used for automatic assignment of UAA authorizations based on information maintained in the identity provider.

Procedure

1. Open the administration console of your SAML 2.0 identity provider.
2. Go to the configuration of your SAML 2.0 identity provider.
3. Add a new SAML service provider as described in the documentation of your identity provider.
4. (If applicable) Choose a name for your identity provider that clearly identifies it as your new service provider. Save your changes.

   Users see this name in the logon screen when the authentication is requested by the UAA service. Seeing the name, they know which application they currently access after authentication.
5. Choose the SAML 2.0 configuration and import the relevant metadata XML file. Save your changes.

   > ### Note
   >
   > Use the `saml-<subdomain>-sp.xml` metadata file of your subaccount. You can create the file using *Download SAML Metadata* in *Security* > *Trust Configuration* of your subaccount.

   Take care that the fields of the SAML configuration are filled. The metadata provides information, such as the name, the URLs of the assertion consumer service and single logout endpoints, and the signing certificate.
6. Choose or create the name ID attribute and select E-mail as a unique attribute. Save your changes.
7. Choose or create a user attribute, and enter **`Groups`** (capitalized) as attribute name for the *Groups* user attribute. Save your changes.

Related Information

Settings for Default SAML Federation Attributes of Identity Providers for Business Users [page 2554]

6.1.1.9.1.1.3.3 Settings for Default SAML Federation Attributes of Identity Providers for Business Users

This table shows the attribute settings of the identity provider and the values administrators use to establish trust between the SAML 2.0 identity provider and a subaccount.

Since there are multiple identity providers you can use, we display the parameters and values of SAP Cloud Identity Services. Use the information in this table as a reference for the configuration of your identity provider.

Settings of SAP Cloud Identity Services for SAML 2.0 Trust

Section in the User Interface	Description	Recommended Value
SAML 2.0 Configuration	Configures the trust with a service provider using an uploaded metadata file.	Upload the metadata file from the SAP Authorization and Trust Management service.
Default Name ID Format	Configures the attribute that the identity provider uses to identify the users. The attribute is sent as the name ID for the authenticated user in SAML assertions. Possible settings: • *User ID* • *E-Mail* • *Display Name* • *Login Name* • *Employee Number*	*E-Mail* **Note** We recommend that you use exactly this name ID format.
Attributes	To assign authorizations indirectly, provide the user groups in the assertion attribute **`Groups`** (capitalized). You can change the default names of the assertion attributes that the application uses to recognize the user attributes. You can use multiple assertion attributes for the same user attribute. Some possible settings: • *Groups* • *first_name* • *last_name* • *email* You can choose from a number of user attributes and add them.	Select the *Groups* user attribute and enter **`Groups`** as assertion attribute. You must set this attribute to enable that the assignment of role collection to user groups. For more information, see the related link. **Caution** Use exactly this spelling: **`Groups`**

Example

In the following, you see what John Doe's SAML 2.0 assertion looks like if *Default Name ID Format* was set to *E-Mail* and *Assertion Attribute* to *Groups*.

Sample Code

```
<Assertion xmlns="urn:oasis:names:tc:SAML:2.0:assertion"
xmlns:ns2="http://www.w3.org/2000/09/xmldsig#" xmlns:ns3="http://www.w3.org/
2001/04/xmlenc#" ID="A-de4246c0-397e-4df0-baf0-cde399d55422"
IssueInstant="2017-10-23T08:52:39.494Z" Version="2.0">
      <Issuer>company-security.accounts.ondemand.com</Issuer>
      <ds:Signature xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
            ...
      </ds:Signature>
```

```
      <Subject>
            <NameID Format="urn:oasis:names:tc:SAML:1.1:nameid-
format:emailAddress">john.doe@example.com</NameID>
            <SubjectConfirmation
Method="urn:oasis:names:tc:SAML:2.0:cm:bearer">
                  <SubjectConfirmationData
InResponseTo="a3h3d679ae07c8cj2ih97d22i7d79a0"
NotOnOrAfter="2017-10-23T09:02:39.494Z"
Recipient="https://authentication.example.hana.ondemand.com/saml/SSO/alias/
company-prod-example"/>
            </SubjectConfirmation>
      </Subject>
      <Conditions NotBefore="2017-10-23T08:47:39.494Z"
NotOnOrAfter="2017-10-23T09:02:39.494Z">
            <AudienceRestriction>
                  <Audience>aws-live-eu10</Audience>
            </AudienceRestriction>
      </Conditions>
      <AuthnStatement AuthnInstant="2017-10-23T08:52:39.494Z"
SessionIndex="S-SP-83d68a13-25fe-44b8-a139-649ca3e99363"
SessionNotOnOrAfter="2017-10-23T20:52:39.494Z">
      ..
      </AuthnStatement>
      <AttributeStatement>
        <Attribute Name="Groups">
            <AttributeValue xmlns:xs="http://www.w3.org/2001/
XMLSchema" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:type="xs:string">iot-acme-monitoring</AttributeValue>
            <AttributeValue xmlns:xs="http://www.w3.org/2001/
XMLSchema" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:type="xs:string">new-group-01</AttributeValue>
            <AttributeValue xmlns:xs="http://www.w3.org/2001/
XMLSchema" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:type="xs:string">new-group-02</AttributeValue>
        </Attribute>
        <Attribute Name="email">
            <AttributeValue xmlns:xs="http://www.w3.org/2001/
XMLSchema" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:type="xs:string">john.doe@example.com</AttributeValue>
        </Attribute>
      </AttributeStatement>
</Assertion>
```

Related Information

Map Role Collections to User Groups [page 2621]

6.1.1.9.1.1.4 Using Multiple Identity Providers from the Same Subaccount

You need to use multiple identity providers for different groups of business users. You want to guide business users to the right identity provider for logon.

Usually there's a single identity provider, which is needed for most users.

Additional identity providers are only needed in exceptional cases.

Basic Considerations Before You Provide a Logon Link for Business Users

Recommendation	Description
(Preferred) Try to avoid multiple trust configurations at all.	Hide the default identity provider and connect a single SAP Cloud Identity Services tenant. Connect further identity providers as corporate identity providers in the SAP Cloud Identity Services tenant and use conditional authentication rules in SAP Cloud Identity Services.
(Feasible) If multiple trusts are really needed in the subaccount, avoid that business users must pick a specific trust by giving them dedicated application URLs, which choose the right identity provider (only supported by some applications).	Some applications can be accessed using a URL that includes a query parameter to define the trust configuration for user login, instead of asking the user to choose one. **Example** `https://application.cfapps.eu10.hana.ondemand.com/?sap_idp=some-origin-key` For information about how to create such URLs, see Create URLs to Access Applications with Specific Identity Providers. Options 1. Make sure all users access applications with such URLs. 2. Choose one standard trust configuration as the one to be used when applications are accessed without such a query parameter. Only use the parameter when any other trust configuration should be used. To achieve this, disable *Available for User Logon* for all trust configurations other than the one you want to use as standard trust configuration. For more information, see Provide Logon Link to Identity Provider for Business Users [page 2558]. If your custom applications use application router, you can enable support for the parameter by following the application router documentation. See Dynamic Identity Provider Configuration or routes [page 469]. **Note** For SaaS applications, check the application documentation to find out whether or not they support the parameter.

Recommendation	Description
(Exceptional) If business users must really choose the identity provider. Make sure the respective link texts on the login page makes sense to them, so that they can easily choose the right one.	Review the link texts of all trust configurations, where *Available for User Logon* is enabled, so that business users understand which one to choose. For more information, see Rename the Logon Link Text for Custom Identity Providers [page 2560].

6.1.1.9.1.1.4.1 Provide Logon Link to Identity Provider for Business Users

You want to provide an understandable link on the logon page and guide business to the right identity provider.

You want to make life easy for business users and provide a logon link that they can easily recognize as such. This link provides an identity provider logon that enables them to log on to the application that they want to work with. To make it even easier, you can hide the logon of the default trust configuration (`SAP ID service`) for platform users, although it remains active. That way, your business users know at once which link to use for their logon.

We strongly recommend to use the following procedure:

1. Hide the default identity provider (`SAP ID service`) from the logon page.
   Hide Logon Link for Default Identity Provider [page 2558]
2. Display the custom identity provider at logon time.
   Display Logon Link for Custom Identity Provider for Business Users [page 2559]
3. (Optional) Give the logon link of the custom identity provider a name the business users understand.
   Rename the Logon Link Text for Custom Identity Providers [page 2560]

6.1.1.9.1.1.4.2 Hide Logon Link for Default Identity Provider

You use one or multiple custom identity providers for business users as well as the default identity provider primarily for platform users. To provide a good logon experience for your business users, you want to hide the default identity provider, which remains active.

Prerequisites

- You have configured a custom trust configuration for a custom identity provider, for example SAP Cloud Identity Services, and set it to active.
- You've checked the *Available for User Logon* checkbox in your custom trust configuration.
- The default trust configuration (`SAP ID service`) is active.

Context

To hide the default identity provider at logon time, proceed as follows:

Procedure

1. Go to your subaccount (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Trust Configuration* in the SAP BTP cockpit.

   You see the list of trust configurations: the default trust configuration and the custom trust configuration(s).
2. Choose (Edit) for the default trust configuration. It has the status *Default*.
3. To hide the default trust configuration (`SAP ID service`) for logon, uncheck the *Available for User Logon* checkbox.
4. Save your changes.

   The default trust configuration is now hidden but remains active.

6.1.1.9.1.1.4.3 Display Logon Link for Custom Identity Provider for Business Users

You want to display a logon link of the custom identity provider that business users should use to log on to an application.

Context

Procedure

1. Go to your subaccount (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Trust Configuration* in the SAP BTP cockpit.

   You see the list of trust configurations: the default trust configuration and the custom trust configuration(s).
2. Go to the desired trust configuration with the status *Custom*.
3. Choose (Edit) for the custom trust configuration.
4. Make sure that the status of your custom trust configuration is *Active*.

5. Check the *Available for User Logon* checkbox.
6. Save your changes.

Results

Whenever business users want to log on to their application, they see a logon link for the custom identity provider that they should use.

6.1.1.9.1.1.4.4 Rename the Logon Link Text for Custom Identity Providers

You can provide a logon link and give the link a name the business users understand. That way, they know which link they should use to log on.

Prerequisites

- You have already made your custom identity provider available for user logon (see the related link).

Context

You want to define an understandable name for the logon link of the custom identity provider that the business users should use.

Procedure

1. Go to your subaccount (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Trust Configuration* in the SAP BTP cockpit.

   You see the list of trust configurations: the default trust configuration and the custom trust configuration(s).
2. Choose the link of your custom identity provider. It has the *Custom* status.
3. Choose *Edit*.
4. Go to *Link Text for User Logon* and give the logon link an understandable name.
5. Save your changes.

Results

Whenever business users want to log on to their application, they see the logon link of the custom identity provider that they should use. It has the caption you entered in *Link Text for User Logon*.

Related Information

Display Logon Link for Custom Identity Provider for Business Users [page 2559]

6.1.1.9.1.1.5 Migration from SAML Trust to OpenID Connect Trust with SAP Cloud Identity Services

To use or get the most out of some applications, such as SAP Build Work Zone and SAP Build Apps, your subaccount must use SAP Cloud Identity Services with OpenID Connect (OIDC) as the custom identity provider. This process helps you change an SAML trust configuration to an OIDC configuration with as little impact to your application users as possible, especially in productive subaccounts.

The goal of this process is as follows:

- Preserve user records, including their authorizations, and also all user-related data in applications and services. User IDs don't change. We update the existing trust configuration instead of creating a new one.
- Preserve role collection mappings to groups or other attributes.
- Ensure that applications receive exactly the same information about users as they did before. We provide guidance and verification steps for when you reconfigure authentication along the chain of subaccount, SAP Cloud Identity Services tenant, and potentially corporate identity provider.

> ⚠ Caution
>
> You could achieve a similar result by deleting your current SAML trust configuration and setting up a new trust with OIDC. However, this method can cause an interruption in the service to your business users. Deleting the trust deletes any local shadow users in the subaccount and their role collection assignments. You would need to re-create those users and their role collection assignments. You would also permanently lose any connection to application data for those users. The internal IDs of the re-created users will never be the same as with the original trust configuration.

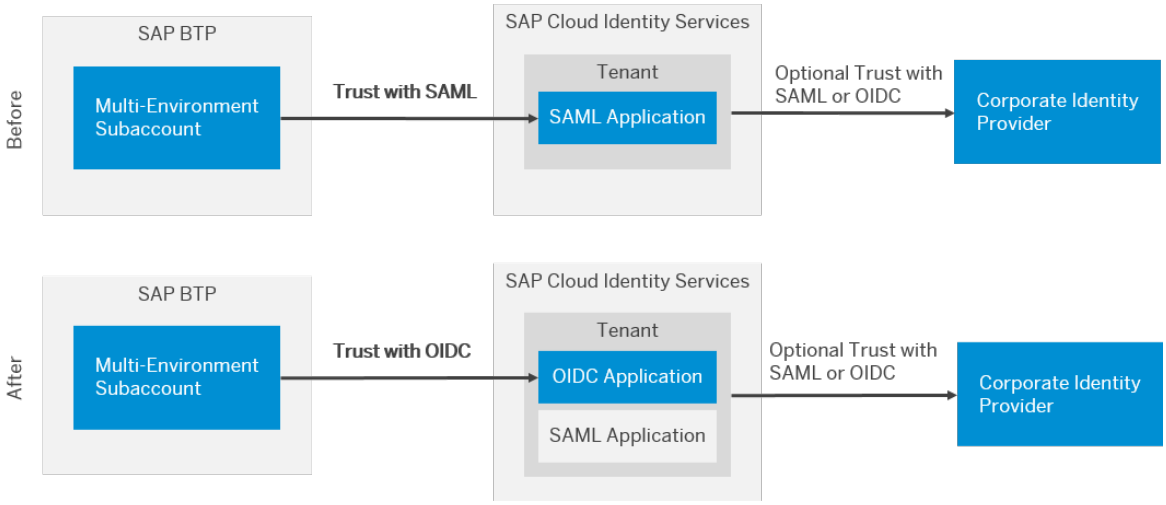
Prerequisites

In the SAP BTP cockpit under *Custom Identity Provider for Applications*, there are no trust configurations with protocol OpenID Connect and no more than one with SAML.

Architecture

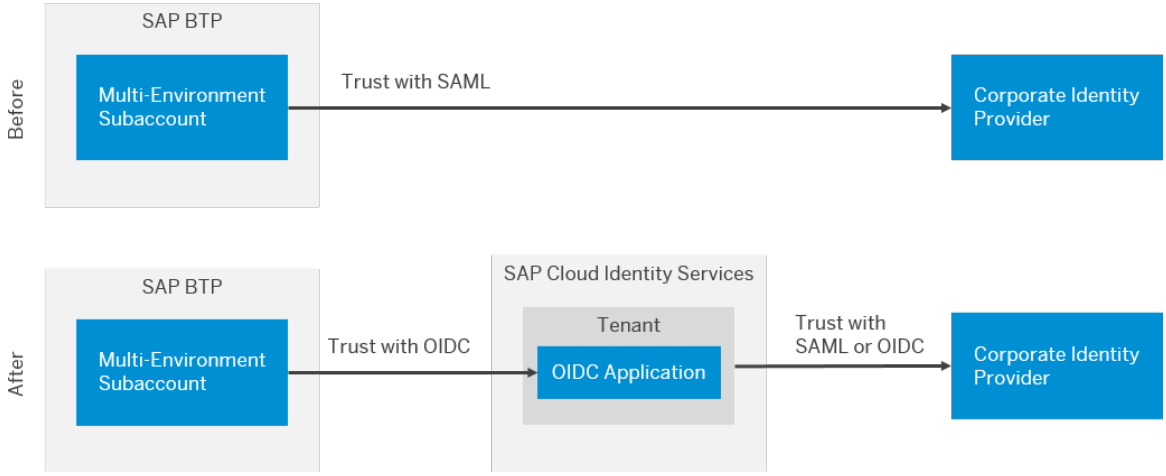
You have one of the following configurations:

- You've already configured SAP Cloud Identity Services as a trusted identity provider, but with SAML. Optionally the SAP Cloud Identity Services tenant trusts a corporate identity provider.
  For this case, you don't change the architecture, only the protocol between the subaccount and the SAP Cloud Identity Services tenant. If you use a corporate identity provider, configure the new SAP Cloud Identity Services application to authenticate with it.



Update of Existing Trust Configuration for SAP Cloud Identity Services

- You've connected your corporate identity provider directly with your subaccount as the trusted identity provider with SAML.
  For this case, you insert trust to SAP Cloud Identity Services in between and configure SAP Cloud Identity Services as a proxy for your corporate identity provider.



Insertion of SAP Cloud Identity Services in Trust Chain of Identity Providers

Risks and Downtime

Expect a downtime between the time when you start the migration and when you've re-established and configured the chain of trust with your identity providers, including the new SAP Cloud Identity Services application representing your subaccount.

> → Recommendation
>
> Practice this procedure in your dev or test subaccounts before carrying it out in your productive subaccounts. Practice helps you to learn how the migration works, enabling you to minimize the risk of misconfiguration and downtime in your productive environments.
>
> If you use the same SAP Cloud Identity Services tenant for nonproductive and productive subaccounts, the integration between SAP Cloud Identity Services and your corporate identity provider is already complete as part of the preparation for the nonproductive subaccounts. This setup reduces the risk and downtime for productive subaccounts.

We provide a rollback function if you encounter any problems.

For more information, see Restore SAML Trust Configuration [page 2573].

Process Overview

1. Prepare for the migration to minimize the downtime starting in step 2.
    1. Collect and save the user data with the existing SAML trust. After the migration, use this information to verify that everything works correctly.
    2. If you use a corporate identity provider and it isn't yet trusted by your SAP Cloud Identity Services tenant, establish trust now.

    For more information, see Prepare for Migration from SAML Trust to OpenID Connect [page 2564].
2. Migrate the subaccount's trust configuration, updating the existing SAML trust to an OIDC trust with SAP Cloud Identity Services.
   For more information, see Migrate from SAML Trust Configuration to OpenID Connect Trust with SAP Cloud Identity Services [page 2565].
3. Configure your new OIDC application for your subaccount.
   Adjust the user subject and any attributes used by your new application in SAP Cloud Identity Services to match your old trust configuration.
   If you use a corporate identity provider, configure the application in SAP Cloud Identity Services to use the corporate identity provider for user authentication.
   For more information, see Configuration of SAP Cloud Identity Services After Migration from SAML to OIDC [page 2568].
4. Verify that user authentication works as before the migration.
   For more information, see Test the Trust Configuration After the Migration [page 2571].

6.1.1.9.1.1.5.1 Prepare for Migration from SAML Trust to OpenID Connect

Before migrating your trust configuration, check which user data are sent today, so you can validate the new configuration after the migration. If you use a corporate identity provider, prepare the trust between your SAP Cloud Identity Services tenant and your corporate identity provider ahead of time.

Procedure

1. Check what attributes your users have.

   To see what attributes your user has, open the following page of your subaccount:

   ```
   https://<subaccount_subdomain>.authentication.<region>.hana.ondemand.com/config?
   action=who&details=true&trustMigrationOnly=true
   ```

   For example

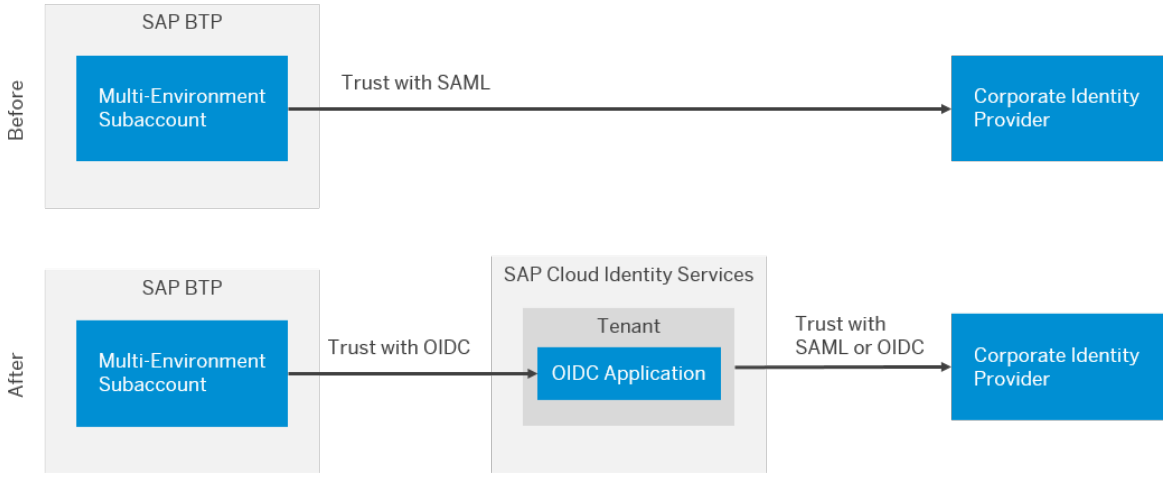
   ```
   https://my-subdomain.authentication.eu10.hana.ondemand.com/config?
   action=who&details=true&trustMigrationOnly=true
   ```

   If at logon you see multiple identity providers, choose the one you want to migrate from for your applications.

   > → Remember
   >
   > Save the results. You'll need to compare the attributes with the attributes you get after migrating the trust configuration.
   >
   > For an example, see Test the Trust Configuration After the Migration [page 2571].

2. Determine if you need to establish trust between your corporate identity provider and your SAP Cloud Identity Services tenant.

   If you previously had configured your subaccount trust to your corporate identity provider directly, configure your SAP Cloud Identity Services tenant to trust your corporate identity provider.



Insertion of SAP Cloud Identity Services in Trust Chain of Identity Providers

In the preceding figure, we're configuring the SAML or OIDC trust between the SAP Cloud Identity Services tenant and the corporate identity provider. Configure the corporate identity provider to send the same value for subject name identifier and the attributes. Keep the attribute names the same, too, except for the attribute names listed in the following table.

From User Attribute	To Assertion Attribute
first_name	given_name
last_name	family_name
mail	email

For more information, see Corporate Identity Providers in the documentation of SAP Cloud Identity Services.

6.1.1.9.1.1.5.2 Migrate from SAML Trust Configuration to OpenID Connect Trust with SAP Cloud Identity Services

If your subaccount uses the SAML protocol to connect to your custom identity provider, you can't consume applications or application features that require direct authentication with SAP Cloud Identity Services, such as SAP Build Work Zone and SAP Build Apps. To consume such applications or features, switch to using OpenID Connect (OIDC) with SAP Cloud Identity Services.

Prerequisites

- The prerequisites for working with the SAP BTP command line interface (CLI) are fulfilled (see Log in [page 2679]).

- In the SAP BTP cockpit under *Custom Identity Provider for Applications*, there are no trust configurations with the OpenID Connect protocol and no more than one with SAML trust configuration that is used for interactive user login (*Available for User Logon*); namely the configuration that you want to migrate.

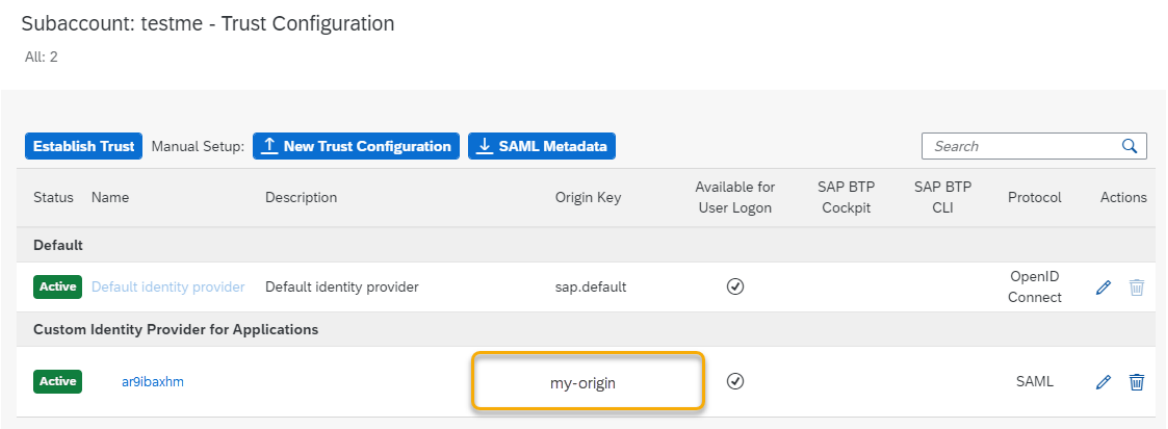
  > **Note**
  >
  > SAML trust configurations that are used for principal propagation from other applications are not relevant and can remain unchanged.

  For more information, see Migration from SAML Trust to OpenID Connect Trust with SAP Cloud Identity Services [page 2561].

Procedure

1. Find the origin key of the identity provider that you want to switch.

   You can look up the origin key in the trust configuration in the SAP BTP cockpit, using the SAP BTP command line interface (BTP CLI), or the Identity Provider Management API.

   Finding the Origin Key in the Cockpit

2. To migrate the trust configuration from SAML to OIDC, use the BTP CLI.

   For more information, see Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707].

3. Log on to the BTP CLI (see Log in [page 2679]).
4. Make sure that you set the target to the relevant subaccount.
5. Find the name of the SAP Cloud Identity Services tenant you want to migrate to OpenID Connect. Use the following command:

   ```
   btp list security/available-idp
   ```

   You receive a list of all SAP Cloud Identity Services tenants that are available in this subaccount. It's a good idea to copy the name of the SAP Cloud Identity Services tenant that you want to migrate.

6. Migrate your trust configuration using the following command:

   ```
   btp migrate security/trust ORIGIN --idp TENANT
   ```

> ### Example
>
> ```
> btp migrate security/trust my-origin-key --idp myidp.accounts.ondemand.com
> ```

The BTP CLI returns that the trust configuration is active and that the protocol is OIDC.

7. Check that your SAML trust configuration was migrated to a trust configuration with the OpenID Connect protocol. Go to your subaccount in the SAP BTP cockpit and choose *Security* > *Trust Configuration*.

Results

In the cockpit, there's a new OIDC trust configuration for your SAP Cloud Identity Services tenant. The original SAML trust configuration has been set to inactive and has received a new `oidc-migration-backup` origin key. If you need to roll back the change, the service uses this configuration to restore your original configuration.

In your SAP Cloud Identity Services tenant, there's a new application that represents the trust configuration. The name is `SAP BTP subaccount <subaccount_display_name>`.

Next Steps

Configure your new SAP Cloud Identity Services application to match your old configuration. Check that your users can log on with all the required attributes. If necessary, configure your new application to consume your corporate identity provider.

> ### Note
>
> If everything works, you can delete the original SAML trust configuration. If you do so, you can't restore it.

Related Information

Configuration of SAP Cloud Identity Services After Migration from SAML to OIDC [page 2568]
Restore SAML Trust Configuration [page 2573]

6.1.1.9.1.1.5.3 Configuration of SAP Cloud Identity Services After Migration from SAML to OIDC

You replaced an SAML trust configuration to your custom identity provider with an OIDC trust configuration to SAP Cloud Identity Services. Now, you need to make sure that the subaccount gets the same user attributes (names and values) as before.

How you proceed depends on which architecture you're trying to achieve. We provide guidance for the configuration according to your target architecture.

- Business users authenticate directly with SAP Cloud Identity Services
  See SAP Cloud Identity Services Without Corporate Identity Provider [page 2568].
- Business users authenticate with a corporate identity provider. SAP Cloud Identity Services is a proxy:
  - Identity federation is disabled (default flow)
  - Identity federation is enabled (advanced)

  See SAP Cloud Identity Services with Corporate Identity Provider [page 2569].

Related Information

Migration from SAML Trust to OpenID Connect Trust with SAP Cloud Identity Services [page 2561]
Restore SAML Trust Configuration [page 2573]
Map User Attributes from a Corporate Identity Provider for Business Users [page 2543]

6.1.1.9.1.1.5.3.1 SAP Cloud Identity Services Without Corporate Identity Provider

You've only changed the protocol of the trust configuration between your SAP Business Technology Platform subaccount and SAP Cloud Identity Services.

Copy the following configurations from the old SAML-based SAP Cloud Identity Services application to your new OIDC-based SAP Cloud Identity Services application:

- Subject name identifier
  See Configure the Subject Name Identifier Sent to the Application in the documentation of SAP Cloud Identity Services.
- Attributes
  Change the names of the following attributes as listed in the following table.

From User Attribute	To User Attribute
first_name	given_name
last_name	family_name
mail	email

See Configuring User Attributes from the Identity Directory.

Related Information

Configuration of SAP Cloud Identity Services After Migration from SAML to OIDC [page 2568]
Test the Trust Configuration After the Migration [page 2571]

6.1.1.9.1.1.5.3.2 SAP Cloud Identity Services with Corporate Identity Provider

You want to insert your SAP Cloud Identity Services tenant in the user authentication flow between your SAP BTP subaccount and your corporate identity provider. So far, you've migrated the trust configuration from your subaccount to use SAP Cloud Identity Services. Now, you configure SAP Cloud Identity Services to delegate user authentication to the corporate identity provider.

We already asked you to configure trust between your SAP Cloud Identity Services tenant and your corporate identity provider is this step: Prepare for Migration from SAML Trust to OpenID Connect [page 2564].

How you configure the connection depends on whether you're using identity federation or not. Identity federation is a configuration of SAP Cloud Identity Services. If you haven't connected SAP Cloud Identity Services to the corporate identity provider yet, use the disabled approach. If your SAP Cloud Identity Services tenant already trusts your corporate provider, then choose the approach that fits to the existing trust configuration.

For more information about identity federation, see Configure Identity Federation in the documentation of SAP Cloud Identity Services.

- Identity federation is disabled (default option)
  See Choose Default Identity Provider for an Application in the documentation of SAP Cloud Identity Services.
- Identity federation is enabled (advanced)
  See Configure Conditional Authentication for an Applicationin the documentation of SAP Cloud Identity Services.

Identity Federation Is Disabled

As you were preparing for this migration, you saw what subject and attributes you need for your users.

For more information, see Prepare for Migration from SAML Trust to OpenID Connect [page 2564].

Attributes are passed to SAP BTP just as they're received from the corporate identity provider. Subject and attribute configurations on the SAP Cloud Identity Services application are ignored. The subject **must** be provided by the corporate identity provider as required. Attributes **ideally** come from the corporate identity provider with the attribute names used by the subaccount so far.

Compare the attributes you got before the migration to the attributes you get now.

For more information, see Test the Trust Configuration After the Migration [page 2571].

If the assertion attributes don't come from the corporate identity provider as you expected, change the names of the following assertion attributes as listed in the following table.

From User Attribute	To User Attribute
first_name	given_name
last_name	family_name
mail	email

Map the attribute names between what the corporate identity provider provides and what the subaccount expects, according to the protocol of the trust configuration:

- For OIDC: See Enrich Token Claims Coming from Corporate IdP in the documentation of SAP Cloud Identity Services.
- For SAML: Enrich Assertion Attributes Coming from Corporate IdP in the documentation of SAP Cloud Identity Services.

Identity Federation Is Enabled

As you were preparing for this migration, you saw what subject and attributes you need for your users.

For more information, see Prepare for Migration from SAML Trust to OpenID Connect [page 2564].

Configure your new OIDC-based SAP Cloud Identity Services application to send the same information to your SAP BTP subaccount as before.

- Subject name identifier
  See Configure the Subject Name Identifier Sent to the Application in the documentation of SAP Cloud Identity Services.
- Assertion attributes

  > → Recommendation
  >
  > Change the names of the following assertion attributes as listed in the following table.
  >
  > | From User Attribute | To User Attribute |
  > | --- | --- |
  > | first_name | given_name |
  > | last_name | family_name |
  > | mail | email |

  See Configuring User Attributes from a Corporate Identity Provider in the documentation of SAP Cloud Identity Services.

Compare the attributes you got before the migration to the attributes you get now.

For more information, see Test the Trust Configuration After the Migration [page 2571].

6.1.1.9.1.1.5.4 Test the Trust Configuration After the Migration

Check that your users can still access their applications and have all the attributes they require. Use these steps to validate your new configuration.

Prerequisites

You saved the list of attributes you get before the migration.

For more information, see Prepare for Migration from SAML Trust to OpenID Connect [page 2564].

Procedure

1. Check what attributes your user has now.

   ```
   https://<subaccount_subdomain>.authentication.<region>.hana.ondemand.com/config?
   action=who&details=true&trustMigrationOnly=true
   ```

   For example

   ```
   https://my-subdomain.authentication.eu10.hana.ondemand.com/config?
   action=who&details=true&trustMigrationOnly=true
   ```

2. Compare with the results you had before the migration.

   In the following example, the user has a few special attributes in addition to the standard ones:

   - Custom attribute `cost_center`
   - Group assignment `my-group`
   - Authorization assignment over the group to the role collection for reading audit logs

Before (SAML)	After (OIDC)

`userName P000011`
`origin my-origin-key`
`zoneId 9fcd13d0-77d7-4cfa-86b0-a41b02e662a6`
`User Attributes {`
`  user_uuid=[7ebf5c32-cacd-4ad2-a208-6bcb3e764f6f],`
`  email=[maria.fontes@example.com],`
`  cost_center=[my-cost-center],`
`  Groups=[my-group],`
`  family_name=[Fontes],`
`  given_name=[Maria]`
`  }`
`email maria.fontes@example.com`
`userId 128af760-ad57-4213-b063-2dd381a4d92e`
`externalId P000011`
`given_name Maria`
`family_name Fontes`
`verified true`
`authorities [`
`  openid,`
`  user_attributes,`
`  auditlog-management!`
`b8.ReadAuditLogs,`
`  uaa.user`
`]`
	`userName P000011`
	`origin my-origin-key`
	`zoneId 9fcd13d0-77d7-4cfa-86b0-a41b02e662a6`
	`User Attributes {`
	`  given_name=[Maria],`
	`  user_uuid=[7ebf5c32-cacd-4ad2-a208-6bcb3e764f6f],`
	`  cost_center=[my-cost-center],`
	`  groups=[my-group],`
	`  family_name=[Fontes],`
	`  email=[maria.fontes@example.com]`
	`  }`
	`email maria.fontes@example.com`
	`userId 128af760-ad57-4213-b063-2dd381a4d92e`
	`externalId P000011`
	`given_name Maria`
	`family_name Fontes`
	`verified true`
	`authorities [`
	`  openid,`
	`  user_attributes,`
	`  auditlog-management!`
	`b8.ReadAuditLogs,`
	`  uaa.user`
	`]`

> **Note**
>
> In the new OIDC application, uppercase or lower case G in the `groups` attribute name doesn't matter.

3. If there are any remaining differences, adjust the configuration of the SAP Cloud Identity Services tenant or applications or of the corporate identity provider until you get the same attributes.

   For more information, see Configuration of SAP Cloud Identity Services After Migration from SAML to OIDC [page 2568].

   Use the OIDC token logging in SAP Cloud Identity Services to help troubleshoot any problems.

   For more information, see Logging OpenID Connect Tokens in the documentation of SAP Cloud Identity Services.

   If you can't get the configuration to work, restore the SAML trust configuration.

   For more information, see Restore SAML Trust Configuration [page 2573].

Next Steps

Once everything is working as you expected and are certain you don't need it anymore, delete your old SAML trust configuration with the origin `oidc-migration-backup`.

> **Caution**
>
> If you delete the old SAML trust configuration too early, you can't use the restore feature anymore.

Related Information

Restore SAML Trust Configuration [page 2573]

6.1.1.9.1.1.5.5 Restore SAML Trust Configuration

You replaced a SAML trust configuration to your custom identity provider with an OpenID Connect (OIDC) trust configuration to SAP Cloud Identity Services, and the authentication of application users in the subaccount isn't working as you expected. Restore your SAML configuration to get your applications working again.

Prerequisites

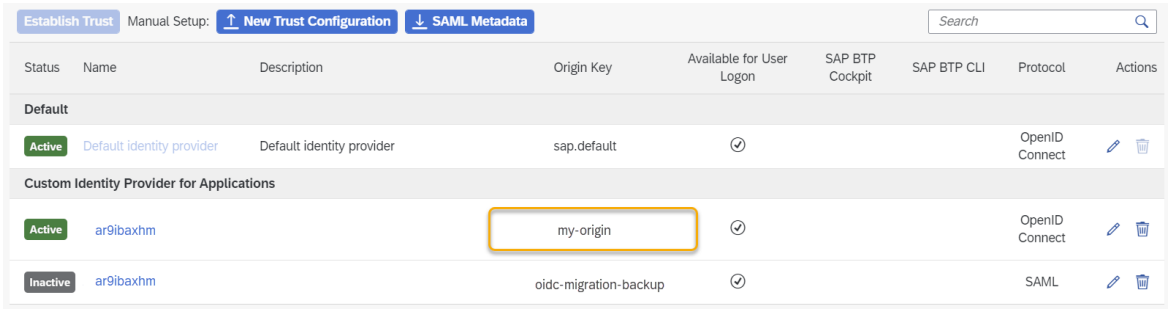
You previously migrated your trust configuration from SAML to OIDC.

For more information, see Migrate from SAML Trust Configuration to OpenID Connect Trust with SAP Cloud Identity Services [page 2565].

Procedure

1. Find the inactive SAML trust configuration with the `oidc-migration-backup` origin key. There's an active trust configuration with the same name and a different origin key. This trust is the one you can restore to a SAML trust configuration.

   You can look up the origin key in the trust configuration in the SAP BTP cockpit, using the SAP BTP command line interface (BTP CLI), or the Identity Provider Management API.



Finding the Origin Key in the Cockpit

2. To restore the trust configuration from OIDC to SAML, use the BTP CLI.

   For more information, see Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707].

3. Log on to theBTP CLI (see Log in [page 2679]).
4. Make sure that you set the target to the relevant SAP BTP subaccount.

5. Find the name of the (currently active) OIDC trust configuration that you want to restore to the SAML protocol.

   > **Note**
   >
   > There's also an inactive trust configuration with the same name, but it has the `oidc-migration-backup` origin key and the SAML protocol. It still has the data and assignments of the original SAML trust configuration.

6. Restore your trust configuration. Use the original key of the active OIDC trust configuration using the following command:

   ```
   btp restore security/trust my-origin-key
   ```

   The BTP CLI returns that the restored trust configuration is active and that the protocol is SAML.

7. Check whether your trust configuration with the OpenID Connect protocol has been restored to SAML. Go to your subaccount in the SAP BTP cockpit and choose *Security* > *Trust Configuration*. You see only one SAML trust configuration - the restored one. The OIDC trust configuration has disappeared.

Results

The original SAML trust configuration has been restored, with its original role collection assignments and origin key. The respective OIDC trust configuration no longer exists. In your SAP Cloud Identity Services tenant, the application for the OIDC trust configuration, `SAP BTP subaccount <subaccount_display_name>` has been deleted.

6.1.1.9.1.1.6 Safely Switch to Another OIDC Identity Provider

Safely move an OIDC trust configuration to another SAP Cloud Identity Services tenant without losing user data, such as authorizations or user-related data in subscribed applications.

Context

As a subaccount administrator, you want to consolidate your landscape. You want to use a different SAP Cloud Identity Services tenant than the one you used so far. Be aware that data provided by identity providers and key data of trust configurations are used to store user-related data, so you want to ensure that this change doesn't impact any user data. To avoid any impact, safely update your trust configuration and switch to another identity provider.

For more information, see SAP Note 3468982.

This change requires planned downtime. Reconfigure the trust in SAP BTP and SAP Cloud Identity Services so that SAP BTP receives exactly the same user data as before. Some time is needed before sign-in is available again.

> ⚠ Caution
>
> For such a change, it's not a good idea to delete your old trust configuration and then create a new identity provider. When you delete a trust configuration, you lose all role collection assignments and the shadow users in the subaccount. User-related data in applications and services might be affected as well.

> → Recommendation
>
> Practice this procedure in your development or test subaccounts before carrying it out in your productive subaccounts.

Procedure

1. To switch the SAP Cloud Identity Services tenant of your trust configuration, go to the custom trust configuration for applications that you want to switch to another identity provider.
2. Select the trust configuration.
3. Choose the *Switch Tenant* button. A popup is displayed that lists all resources of this subaccount. Multiple cases are possible.
    1. The subaccount doesn't contain any applications, which use SAP Cloud Identity Services directly. You can perform a tenant switch.
    2. The applications support a tenant switch and can be migrated to the selected tenant. Review, re-create the user group assignments, and adapt the configuration of the applications.
    3. Some of the applications don't support a tenant switch. In this case, you can't perform a tenant switch.

    For more information, see SAP Note 3468982.
4. If a tenant switch is possible, confirm that you want to perform the tenant switch. Don't forget to adjust the configuration of the applications afterwards.
5. (If required) With the new tenant, the cockpit automatically suggests you a domain of the new tenant. Check the domain. To adapt it, choose *Edit*.
6. (If required) Save your domain changes.

    The origin, user names, and shadow user IDs remain the same. Make sure you keep the same subject identifier and user attributes.

6.1.1.9.1.2 Platform Users

User accounts enable users to log on to SAP BTP, access subaccounts, and to use applications according to the permissions granted to them. In this context, it's important to understand the difference between the two types of users that we refer to: Platform users and business users.

Platform users are usually developers, administrators or operators who deploy, administer, and troubleshoot accounts, applications and services on SAP BTP.

Platform users who were added as members and have administrative permissions can view or manage the list of global accounts, directories, subaccounts, and Cloud Foundry orgs and spaces that are available to them.

Members access them using the SAP BTP cockpit, the SAP BTP command line interface (BTP CLI), or the Cloud Foundry command-line interface (CF CLI).

> **Note**
>
> The content in this section is not relevant for Government Cloud (US) regions.

For platform users, there's a default identity provider [page 2594]. We expect that you have your own user base. We recommend that you configure SAP Cloud Identity Services as the custom identity provider and connect SAP Cloud Identity Services to your own corporate identity provider.

Related Information

Business Users [page 2540]
Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576]
Supported Tools and Services When Using Custom Identity Providers for Platform Users [page 2585]
Log On with a Custom Identity Provider to the Cloud Foundry Environment Using the Cloud Foundry Command-Line Interface [page 2587]
Log on with a Browser to the Cloud Foundry CLI and Service Dashboards [page 2592]

6.1.1.9.1.2.1 Establish Trust and Federation of Custom Identity Providers for Platform Users

You want to use a custom identity provider for the platform users of SAP BTP in different environments and at the different account levels: global account, directory, and subaccount. By default, platform users in multi-environment subaccounts are users in the default identity provider.

Prerequisites

- You have a tenant of SAP Cloud Identity Services.

  > **Note**
  >
  > For the custom identity provider for platform users, you can have up to three SAP Cloud Identity Services tenants per global account. Multiple global accounts can share the same SAP Cloud Identity Services tenant.

  For more information, see Tenant Model and Licensing in the documentation for SAP Cloud Identity Services.
- The SAP Cloud Identity Services tenant is associated with the customer IDs of the relevant global account of SAP BTP.
  For more information, see Reuse SAP Cloud Identity Services Tenants for Different Customer IDs in the documentation for SAP Cloud Identity Services.

- Make sure that the email addresses of all users in your identity provider are unique and correct.

> **Note**
>
> The email address of the user is the user identifier for all account levels (global account, directory, multi-environment subaccount) and for the Cloud Foundry environment.

Context

Platform users perform technical development, deployment, and administration tasks. For example, they perform subaccount administration in the SAP BTP cockpit or access the Cloud Foundry command line interface (CF CLI). By hosting these users in your own identity provider, you gain a number of advantages over hosting them in the default identity provider.

- Integrate the management of these users with your corporate identity management strategy, hosted on your own identity providers. You control your own user lifecycle and single sign-on strategies throughout the entire landscape.
- Enforce your own password and authentication policies, such as stronger passwords or multifactor authentication.

> **Note**
>
> The content in this section is only relevant for **platform users** and **not** business users.
>
> For more information about establishing trust for business users, see Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2540].

Establish a trust relationship with a custom identity provider in your global account in SAP BTP. The following procedure guides you through the trust configuration with your custom identity provider.

Procedure

1. Go to your global account (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Trust Configuration* in the SAP BTP cockpit.
2. Choose *Establish Trust*.

   The identity providers listed are the SAP Cloud Identity Services tenants associated with your customer ID.
3. Select an identity provider from the list of available tenants and choose *Next*.
4. Enter a name and a description of the new trust configuration. If possible, set the origin key. The origin key always ends with *-platform*. Continue with *Next*.

> **Note**
>
> The origin key is a technical identifier of an identity provider for platform users. Administrators need it when managing users. Platform users from the identity provider need the origin key when signing in with certain tools, such as the Cloud Foundry runtime command-line interface or service dashboards.

You can only set the origin key once for the whole landscape. You can't change the origin key later.

The whole origin key including `-platform` can have at maximum 36 characters. Only use the following characters for the origin key of the trust configuration.

`aA`–`zZ`, `0`–`9`, `-` (hyphen)

5. Choose your desired domain of the tenant and continue with *Next*. For a good single sign-on experience, choose the same domain for all SAP BTP accounts and non SAP BTP applications that use this tenant.
6. The wizard shows you a preview of your configuration. To complete your new trust configuration, choose *Finish*.

→ Remember

If the OIDC issuer was changed in the trust configuration, the trust breaks, but the global account automatically repairs the trust after 24 hours.

To update the trust configuration on the SAP BTP side, open the details of the trust configuration and choose *Refresh*. This button immediately refreshes the trust configuration to reflect changes in the SAP Cloud Identity Services tenant, for example the issuer value.

Results

You've configured trust in your tenant of SAP Cloud Identity Services, which is your identity provider. SAP Cloud Identity Services creates an application with the name *SAP Business Technology Platform*. Administrators can change this name. Some attributes of the application are managed by SAP, such as the application type, and are read-only.

→ Tip

To troubleshoot problems with tokens from SAP Cloud Identity Services, see Logging OpenID Connect Tokens in the documentation for SAP Cloud Identity Services.

Next Steps

- Add platform users to your orgs and spaces.
  For more information about adding members with the SAP BTP cockpit, see Add Org Members [page 2795] and Add Space Members [page 2803].
  For more information about adding members with the command-line interface, see Add Organization Members Using the Cloud Foundry Command Line Interface [page 2853] and Add Space Members Using the Cloud Foundry Command Line Interface [page 2855].
- Add platform users at different levels (global account, directory, subaccount) from the custom identity provider.
  For more information, see Working with Users [page 2603].

> ### Note
>
> For subaccounts in the Neo environment, the identity provider is offered in the value help only if a user in that identity provider has created at least one Neo subaccount in the corresponding global account and Neo region.
>
> > ### → Tip
> >
> > If the identity provider isn't available in the value help for Neo subaccount members, log on to the global account with a user from that identity provider and create a new Neo subaccount. If it's not needed otherwise, you can delete it.

- Log on to the SAP BTP cockpit as a user from the custom identity provider. In the *Trust Configuration* page, the *Open* link in the SAP BTP cockpit column contains the URL (for example, *https://emea.cockpit.btp.cloud.sap/cockpit/?idp=cidppuxhm.accounts.ondemand.com*) for the user to log on with the custom identity provider. Copy the link and send it to your platform user colleagues. Remember if you have the SAP BTP cockpit open and if you want to work in parallel, your current session may be shared by your browser. Open the link in a private or incognito browsing mode.
  For more information, see Log On with a Custom Identity Provider to the SAP BTP cockpit [page 2591].
- If you want to impose, for example, two-factor authentication for platform users, you must configure two-factor authentication in **all** the SAP Cloud Identity Services applications involved. Multifactor authentication is used as an example.
  For more information, see Multi-Factor Authentication in the documentation for the SAP Cloud Identity Services service. Keep all settings for SAP Cloud Identity Services applications the same.

  > ### → Tip
  >
  > Instead of configuring two-factor authentication in all the SAP Cloud Identity Services applications, you could also configure it once in your corporate identity provider.

  > ### Note
  >
  > For all the Neo datacenters where you have subaccounts, you might have to configure a separate SAML application (for example, *SAP Cloud Platform hhw9g3jrxa*) in SAP Cloud Identity Services.

- Integrate your SAP Cloud Identity Services tenant with your identity authentication management solution. For more information, see Corporate Identity Provider in the documentation for SAP Cloud Identity Services.
- Work in the SAP BTP command line interface (BTP CLI).
  To log on to a global account, you need to provide the hostname of the tenant (for example, *ar9ibaxhm*) with the `--idp` parameter. See Log in with a Custom Identity Provider [page 2684].
  To work with users and role collections on global account, directory, or subaccount level, you need to provide the origin key with the `--of-idp` parameter in the following commands: `btp list security/user`, `btp get security/user`, `btp delete security/user`, `btp assign security/role-collection`, `btp unassign security/role-collection`.
  Both values can be found in the SAP BTP cockpit under *Security* → *Trust Configuration* → *Custom Platform Identity Providers*.
  As an administrator, you want to determine which of the SAP Cloud Identity Services tenants' domains SAP BTP should use for platform user logon. For this reason, you specify a custom domain for an SAP Cloud Identity Services tenant. You can use the `btp update security/trust` command and specify the domain in the `--domain` parameter.

- Work in the Cloud Foundry command-line interface (CF CLI).
  For more information, see Log On with a Custom Identity Provider to the Cloud Foundry Environment Using the Cloud Foundry Command-Line Interface [page 2587].
- The Kyma environment has a separate configuration for the identity provider. We recommend that you also configure your Kyma environment to use SAP Cloud Identity Services as your custom identity provider.
  For more information, see Configure a Custom Identity Provider for Kyma.

Related Information

User and Member Management [page 133]

6.1.1.9.1.2.1.1 Restrictions When Using Custom Identity Providers for Platform Users

The following is a list of restrictions that apply to the use of custom identity providers with platform users.

Supported with Restrictions When Using Custom Identity Providers for Platform Users

Supported with Restrictions	Description
Maximum number of parallel sessions per user per identity provider	Each user is allowed a maximum of 10 parallel sessions, per identity provider. This number considers all tools, including the cockpit and CLIs. **Note** When accessing the cockpit, a user is allowed one session in each region. For example, if you access https://emea.cockpit.btp.cloud.sap, it counts as the first session and https://amer.cockpit.btp.cloud.sap as the second one.
Single logout (SLO)	For platform users of custom identity providers, logging out from the SAP BTP cockpit (including Neo cockpit) terminates the session in the used SAP Cloud Identity Services tenant and sessions of other applications that connect to the same tenant. What is required is that the sessions support this kind of logout. This requirement doesn't apply for other instances of the SAP BTP cockpit except for instances where the user initially logged out from. In this case, sessions remain active.

Supported with Restrictions for Neo Subaccounts Before the Changes in the Trust Configuration

The following is a list of restrictions that only apply for Neo subaccounts when using custom identity providers for platform users.

- All individual Neo subaccounts that have been created before July 2023.
- Neo subaccounts in global accounts that have custom identity providers for platform users. For these subaccounts, SAP Note 3330671 hasn't been applied yet.

Supported with Restrictions	Description
Working with custom domains for an SAP Cloud Identity Services tenant	SAP BTP always uses the default domain of the SAP Cloud Identity Services tenant, regardless of a potentially configured custom domain. Therefore, when you use this tenant as a platform identity provider: • Single sign-on (SSO) doesn't work between applications that use this custom domain and cloud management tools. Exception: if you use the same SAP Cloud Identity Services tenant for both platform and business users, as custom domain is a tenant setting. • The OpenID Connect (OIDC) issuer in the *Name* field of the SAP Cloud Identity Services tenant must be the default domain (<origin>.accounts.ondemand.com). For more information, see Tenant OpenID Connect Configurations.
OpenID Connect (OIDC) issuer in the *Name* field of the SAP Cloud Identity Services tenant	⚠ Caution Don't change the *Name* field after configuring trust. Changing the issuer breaks the trust between the systems.
Neo CLI	No restriction for new Neo subaccounts if this prerequisite is fulfilled. For basic authentication, the Neo CLI has limited support for existing Neo subaccounts when using custom identity providers for platform users.
Neo Git service	Logging on with a password to the Neo Git service doesn't work with custom identity providers for platform users.
Cloud connector	Logging on with a password to the Cloud connector doesn't work with custom identity providers for platform users.
SAP HANA studio	No restriction for new Neo subaccounts if this prerequisite is fulfilled.

6.1.1.9.1.2.1.2 Map User Attributes from a Corporate Identity Provider for Platform Users

When you enable trust with a tenant of SAP Cloud Identity Services, you get an OpenID Connect (OIDC) application in SAP Cloud Identity Services to represent SAP BTP, in the context of platform users. When you

authenticate users using a corporate identity provider, map the user attributes provided by the corporate identity provider to the attributes required by SAP BTP. The following information explains which attributes SAP BTP needs for which purpose, and how you can map those attributes.

Prerequisites

- Ensure that your corporate identity provider allows local users from SAP Cloud Identity Services. If local users are not allowed, you must enrich the attributes coming from the corporate identity provider. For more information, see Enrich Assertion Attributes Coming from Corporate IdP.

Finding the Application for Your Platform Users

The name of the application in the administration console of SAP Cloud Identity Services that represents SAP BTP in the context of platform users has the name, *SAP Business Technology Platform*.

For more information, see OpenID Connect in the documentation of SAP Cloud Identity Services.

Customizing Attribute Mappings

There are several options to customize attribute mappings in SAP Cloud Identity Services, depending on whether identity federation is enabled or disabled. For more information about identity federation, see Configure Identity Federation.

- When identity federation is disabled, SAP Cloud Identity Services always propagates all the attributes received from the corporate identity provider to all the applications, on a 1:1 basis. You have the following options:
  - Configure the corporate identity provider to directly send the attributes (see the table below).
  - If needed, use **enriched token claims** or **enriched assertion attributes** (depending on whether the corporate identity provider is connected with SAML or OIDC) to map the attributes sent by the corporate identity provider, to the attribute names needed by SAP BTP.
    Enriched token claims or **enriched assertion attributes** add additional attributes, either based on the corporate identity provider attributes (for example, by renaming them) or on static values.
- When identity federation is enabled, SAP Cloud Identity Services doesn't automatically propagate any attributes from the corporate identity provider to the application. This option requires mappings in the SAP Cloud Identity Services application, for each attribute that is needed by the application.
  Use the attributes in the SAP Cloud Identity Services application representing SAP BTP.

> ### Note
>
> You can add attribute sources or disable the default application attributes. You can also add self-defined attributes for mapping to role collections.
>
> For example, you have so many groups being added to your token and you're running into size limits. You can disable the standard groups configuration and add a regular expression to include only those

groups, which begin with “BTP”. So, to the *groups* attribute, you add a source of type **`Expression`** with the value **`${companyGroups:regex[BTP.*]}`**.

For more information, see Configuring Attributes Based on Flexible Expressions.

To check which groups SAP Cloud Identity Services actually sends, use the troubleshooting logs for OpenID Connect. For more information, see Logging OpenID Connect Tokens in the documentation for SAP Cloud Identity Services.

Note

Ensure that you enter the accurate value names for the attributes as they are provided by your corporate identity provider.

Default Configuration of Application Attributes in SAP Cloud Identity Services

Attribute Name	Source	Attribute Value
`email`	Identity Directory	`Email`
	Corporate Identity Provider	`email`
`groups`	Identity Directory	`All Groups`
`mail`	Identity Directory	`Email`
	Corporate Identity Provider	`email`
`uuid`	Identity Provider	`User ID`

If the corporate identity provider sends user attributes for email address, first and last name with other names than *mail*, *first_name*, or *last_name*, set the right attribute name by replacing those values.

Example

If your corporate identity provider sends users' last names as the `sn` attribute, add the corporate identity provider as source to the `last_name` attribute with the value `sn`.

For more information, see User Attributes in the documentation of SAP Cloud Identity Services.

The following table provides the information needed for mapping the attributes.

The subject name identifier attribute is used by SAP BTP to uniquely identify the user in Neo subaccounts. The email address is used as the user identifier for the global account, directory, multi-environment subaccount, and the Cloud Foundry environment.

For more information, see Configure the Subject Name Identifier Sent to the Application in the documentation of SAP Cloud Identity Services.

Attribute Mapping in SAP Cloud Identity Services Tokens

User Attribute Expected by SAP BTP	Purpose
`Subject name identifier`	User identifier for Neo subaccounts. Default value: `User ID`
`email`	Email address of the user. **Note** User identifier for all the account levels (global account, directory, multi-environment subaccount), and for the Cloud Foundry environment. The email addresses of all the users in the SAP Cloud Identity Services tenant must be unique. For more information, see the prerequisites [page 2576] for establishing trust and federation of custom identity providers for platform users.
`first_name`	First name of the user.
`last_name`	Last name of the user.
`groups`	Any groups the subject is assigned to in the identity provider.

Related Information

Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2540]

Map User Attributes from a Corporate Identity Provider for Business Users [page 2543]

6.1.1.9.1.2.2 Supported Tools and Services When Using Custom Identity Providers for Platform Users

Not all tools and services of SAP BTP support the use of custom identity providers with platform users. We provide a list of tools and services, which support this feature and any restrictions that apply.

Supported When Using Identity Providers for Platform Users

Supported	Description
Cloud Connector	You can connect a Cloud Connector with multi-environment and Neo subaccounts of SAP BTP. For more information, see Use a Custom IDP for Subaccount Configuration. **Note** In addition to the initial connection from Cloud Connector to SAP BTP, the regular rotation of the client certificate that is used for the ongoing connection, is also affected.
SAP BTP command line interface (BTP CLI)	Once trust has been configured on the global account level in the cockpit, platform users can log on using the custom identity provider and use the full range of the BTP CLI. To log on to a global account, you need to provide the hostname of the tenant (for example, *ar9ibaxhm*) with the `--idp` parameter. See Log in with a Custom Identity Provider [page 2684]. To work with users and role collections on global account, directory, or subaccount level, you need to provide the origin key with the `--of-idp` parameter in the following commands: `btp list security/user`, `btp get security/user`, `btp delete security/user`, `btp assign security/role-collection`, `btp unassign security/role-collection`. Both values can be found in the cockpit under *Security* → *Trust Configuration* → *Custom Platform Identity Providers*.
Cloud Foundry command-line interface (CF CLI)	Make sure you use the Cloud Foundry command-line interface (CF CLI v7). For more information, see https://docs.cloudfoundry.org/cf-cli/v7.html . **Restriction** With v6 of the CF CLI, you can't use the **`--origin`** option in **`cf set-org-role`** and **`cf set-space-role`**. Use the *Members* tab in the SAP BTP cockpit. Go to the SAP BTP cockpit under `<my_global_account>` > `<my_subaccount>` > *Security* > *Members* to set the user's permissions. For more information, see Log On with a Custom Identity Provider to the Cloud Foundry Environment Using the Cloud Foundry Command-Line Interface [page 2587]. See also Using the Cloud Foundry CLI with SAP HANA Cloud

Supported	Description
SAP Application Logging service and Kibana	Users logged on using custom identity providers can analyze logs written using the SAP Application Logging service. For more information, see the SAP Application Logging service documentation.
SAP BTP cockpit	Platform users logged on using custom identity providers can use the full range of the SAP BTP cockpit.
SAP Cloud Transport Management	Platform users logged on to SAP Cloud Transport Management service using custom identity providers can use all functions of the service as described in the SAP Cloud Transport Management documentation. For more information, see SAP Cloud Transport Management.
SAP HANA Cloud Cockpit	Platform users logged on using custom identity providers can use the full range of SAP HANA Cloud Cockpit. For more information, see SAP HANA Cockpit.
SAP Job Scheduling service	Platform users logged on using custom identity providers can use the SAP Job Scheduling service dashboard.
SAP Mobile Services	SAP Mobile Services uses platform users to log on to the admin cockpit and manage apps and configurations. Based on their space member roles, platform users can log on using a custom identity provider and access the SAP Mobile Services admin cockpit. For more information, see Set Up Customer Accounts in the SAP Mobile Services documentation.

Supported with Restrictions When Using Platform Identity Providers

Supported with Restrictions	Description
SAP Business Application Studio	SAP Business Application Studio supports custom identity providers with the following restrictions. • When you log on with the SAP Business Application Studio web-based user interface, you authenticate with the identity provider for business users. • When you log on to the Cloud Foundry environment from the terminal mode of SAP Business Application Studio with the Cloud Foundry environment command-line interface (CF CLI), you can authenticate with the identity provider for platform users. For more information, see Terminal in the SAP Business Application Studio documentation.
SAP HANA Service Dashboard	This tool doesn't support log on with a platform user from a custom identity provider. You can only log on with a user from the default identity provider. • Open the SAP HANA Service Dashboard in AWS and Google Cloud regions • Open the SAP HANA Service Dashboard in the China (Shanghai) region

Supported with Restrictions	Description
SAP Rapid Application Development by Mendix	You can't perform operations on multi-environment subaccounts, like deployments with SAP Rapid Application Development by Mendix when using a custom identity provider for platform users. For more information, see SAP Rapid Application Development by Mendix .
Service dashboards of services that aren't listed as supported.	If you use services with dashboards, you can't access these dashboards with a platform user from a custom identity provider. Using service dashboards is only possible with the default identity provider (SAP ID service).
SAP Web IDE	SAP Web IDE supports identity providers with the following restrictions. • When you log on with the WebIDE web-based user interface, you authenticate with the identity provider for business users. • When you deploy applications to the Cloud Foundry or Neo environment from SAP Web IDE, you can authenticate with the identity provider for platform users. For more information, see User Authentication and Authorization. → Recommendation Use SAP Business Application Studio.

Related Information

Administration and Operations in the Cloud Foundry Environment [page 2779]

6.1.1.9.1.2.3 Log On with a Custom Identity Provider to the Cloud Foundry Environment Using the Cloud Foundry Command-Line Interface

Learn how to use different methods to log on to Cloud Foundry using a custom identity provider (IdP).

Prerequisites

- You've created at least one subaccount and enabled the Cloud Foundry environment in this subaccount. For more information, see Create a Subaccount [page 2509].
- You've downloaded and installed the Cloud Foundry command-line interface (cf CLI). For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813].

- Your administrator has configured your Cloud Foundry environment to use a custom identity provider. For more information, see Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576].
- You've configured the login screen to display your custom identity provider, see Log on with a Browser to the Cloud Foundry CLI and Service Dashboards [page 2592] (relevant for the manual login process).

Context

The cf CLI provides different options to log on using a custom identity provider. One scenario is recommended for an interactive logon because you must switch from the CLI to the browser to log on. This scenario supports multifactor authentication and other passwordless authentication methods, as well as single sign-on. The other scenario focuses on automation scenarios where switching between the CLI and a browser isn't possible.

Procedure

Decide which scenario applies to you according to this table.

Scenario	See
You can open a browser during the logon process.	Log On Manually With a Custom Identity Provider [page 2589].
The logon process is automated, for example with a script or there's no possibility to open a browser during logon. ⚠ Restriction **This scenario is only supported if the users exist directly in your tenant of SAP Cloud Identity Services and not in a corporate identity provider.**	Log On as a Technical User With a Custom Identity Provider [page 2590]

Related Information

Log on with a Browser to the Cloud Foundry CLI and Service Dashboards [page 2592]

6.1.1.9.1.2.3.1 Log On Manually With a Custom Identity Provider

To log on to Cloud Foundry, using a custom identity provider, use the single sign-on option of the CF CLI.

Prerequisites

- You've configured the login screen to display your custom identity provider.
  For more information, see Log on with a Browser to the Cloud Foundry CLI and Service Dashboards [page 2592].
- You know the origin key of the identity provider.
  For more information, see Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576].

Context

During logon, the system provides you with a URL to your custom identity provider for you to enter in a web browser. If you're already logged on to your custom identity provider, the system provides a temporary authentication code to complete your logon. If you don't have an active session with your custom identity provider, you need to log on to receive the temporary authentication code.

Procedure

1. Open a command line.
2. Set the target API endpoint to endpoint of your subaccount.

   ```
   cf api https://api.cf.<region>.hana.ondemand.com
   ```

   {·} Sample Code

   ```
   cf api https://api.cf.eu10.hana.ondemand.com
   ```

3. Log on to the Cloud Foundry environment with single sign-on:

   ```
   cf login --sso
   ```

   The command-line interface displays a login URL and prompts you to enter a temporary authentication code.
4. Open the URL that is provided on the screen to go to the logon screen.

   If this is the first time you're logging on or your browser cache has been cleaned of cookies, you're prompted to provide the origin key of your custom identity provider or choose the default identity provider.

Otherwise, the sign-in page lists your accounts on custom identity providers with which you previously logged on.

5. Choose the account of your identity provider.

   If you don't have an active session yet, you're prompted to log on to your identity provider.

   After you've successfully logged on, you receive a temporary authentication code.

6. Enter the passcode.

   You've logged on successfully. You can now choose your org.

Related Information

Log on with a Browser to the Cloud Foundry CLI and Service Dashboards [page 2592]

6.1.1.9.1.2.3.2 Log On as a Technical User With a Custom Identity Provider

To log on to Cloud Foundry, using a custom identity provider, use the `--origin` option of the Cloud Foundry command-line interface (cf CLI).

Prerequisites

The users exist:

- Directly in your tenant of the SAP Cloud Identity Services.
- In a corporate identity provider with SAP Cloud Identity Services working as a proxy.
  - Trust to your SAP Cloud Identity Services tenant must be configured with OIDC and not SAML.
  - Your corporate identity provider must support the password grant flow.

  For more information, see Configure Trust with OpenID Connect Corporate Identity Provider in the documentation for SAP Cloud Identity Services.

Context

We recommend this method of logging on if you want to use an automated script and can't open a browser during the logon process.

Procedure

Set up a script with the following code:

```
cf api https://api.cf.<region>.hana.ondemand.com
cf login --origin <origin> -u <user> -p <password>
```

Sample Code

```
cf api https://api.cf.eu10.hana.ondemand.com
cf login --origin sap.ids -u julie.armstrong@sap.com -p mysecurepassword
```

- Find the `<region>` value, that applies to you in the section Regions [page 17].
- Find the `<origin>` value for the user that you want to use in the cockpit.
  1. Navigate to the subaccount of your user.
  2. Choose *Cloud Foundry* > *Org Members*.
  3. Find the `<origin>` value of your user in the table.

6.1.1.9.1.2.4 Log On with a Custom Identity Provider to the SAP BTP cockpit

All users can log on to the SAP BTP cockpit with a custom identity provider.

Prerequisites

- Your user has administration or viewer rights in this subaccount or global account. For more information, see Security Administration: Managing Authentication and Authorization [page 2537] or Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138].
- You have established trust with a custom identity provider for platform users.
  For more information, see Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576].

Context

Use the *Open* link in the SAP BTP Cockpit column. It contains a URL for the user to log on with the custom identity provider.

Example

https://emea.cockpit.btp.cloud.sap/cockpit/?idp=mytenant.accounts.ondemand.com

Otherwise platform users access the cockpit without this URL parameter. For more information, see Access the Cockpit.

Procedure

1. Go to your global account or subaccount (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Trust Configuration* in the SAP BTP cockpit.
2. Use the *Open* link in the SAP BTP *Cockpit* column.
   - To enable other platform users to log on with a custom identity provider, send the link to these users.
   - To work with your user accounts from multiple identity providers at the same time, open the link in a private or incognito browser window or tab.

     > **Note**
     >
     > A browser that isn't in private or incognito mode has one single session with the cockpit. If you have an existing cockpit session with one identity provider and log on with another one, the existing session closes and creates a new session. For this reason, you can't have parallel sessions with different identity providers.
3. Log on using the custom identity provider.

6.1.1.9.1.2.5 Log on with a Browser to the Cloud Foundry CLI and Service Dashboards

Platform users of the Cloud Foundry environment have the option to log on with a custom identity provider or the default identity provider.

Prerequisites

- You know the origin key of the identity provider.
  For more information, see Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576].
- You don't already have a session with the Cloud Foundry environment.
  If you're already authenticated, you skip the logon process and go straight to your application.

Context

When accessing a web application, like a service dashboard (for example `logs.cf.eu10.hana.ondemand.com`) or you log into your Cloud Foundry account using the CLI with `cf`

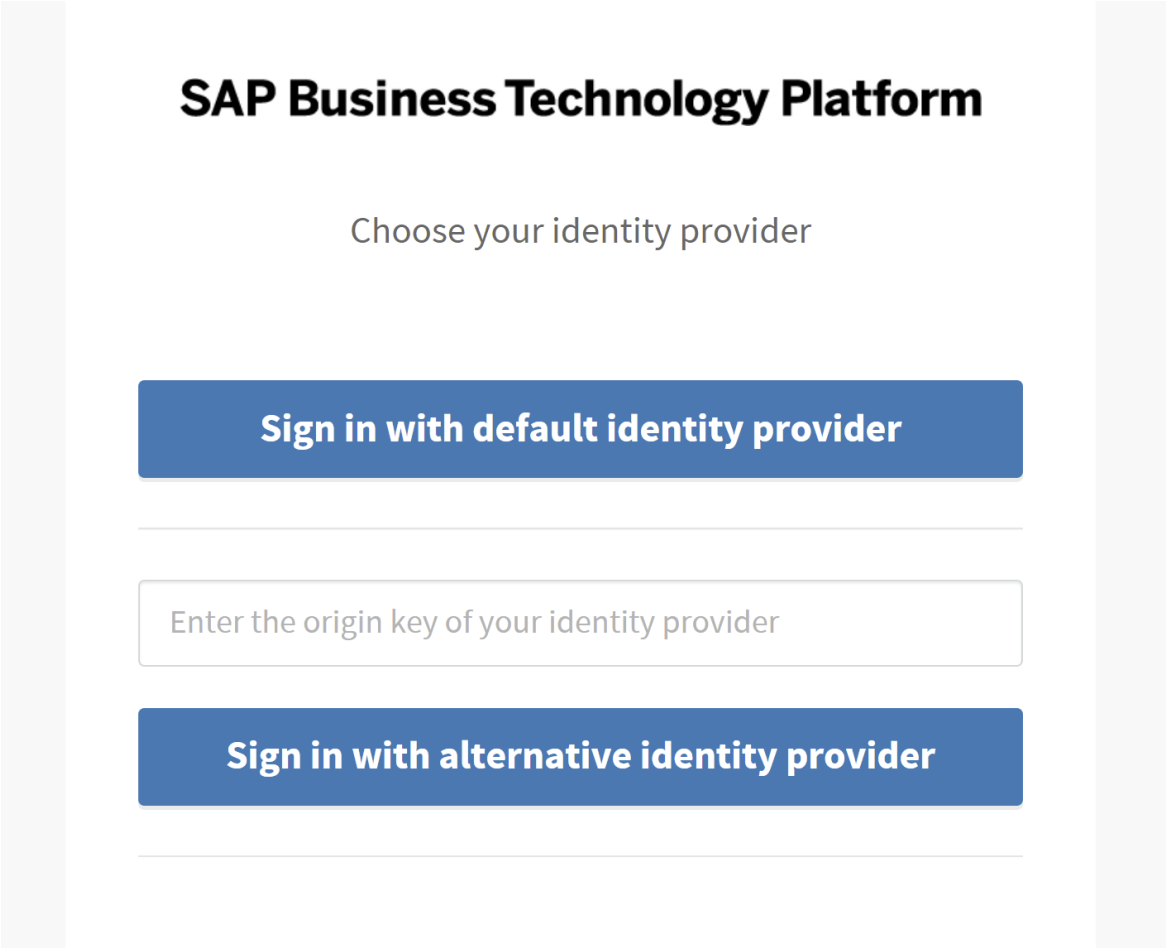
`login -sso`, you're prompted by a login screen. This method enables you to log on to a custom identity provider or makes it easier to log on when multifactor authentication is required.

Procedure

1. Navigate to the URL for your service dashboard or provided by the CF CLI.

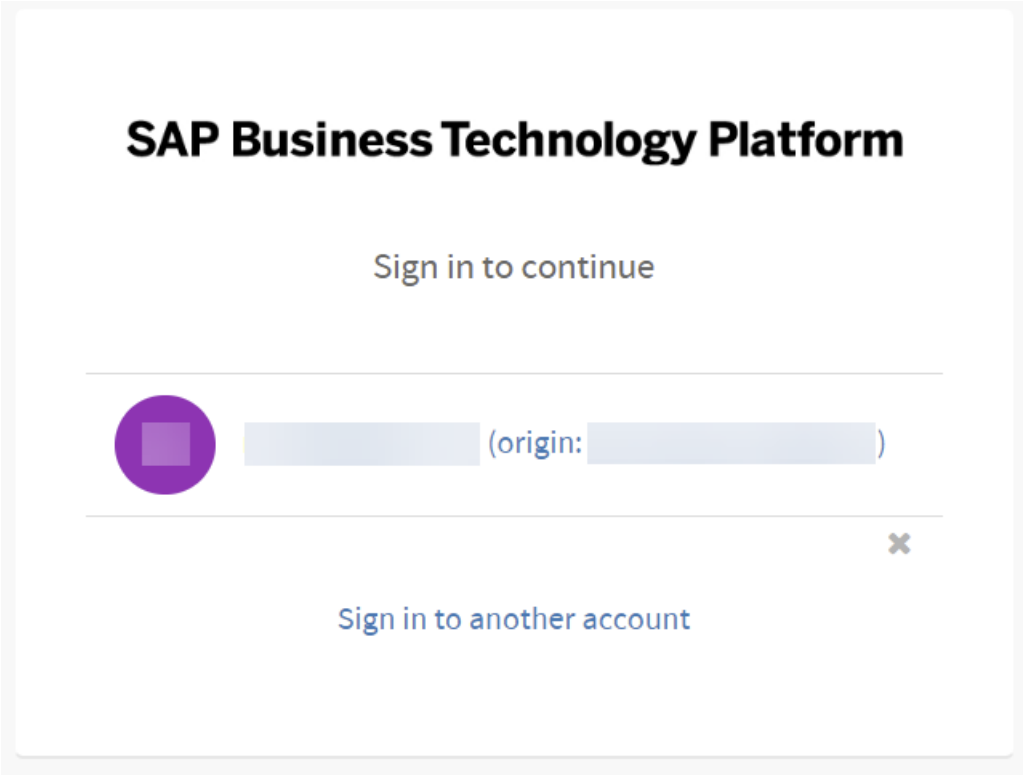
   The login screen appears.

   If this is the first time you're logging on or your browser cache has been cleaned of cookies, you're prompted to provide the origin key of your custom identity provider or choose the default identity provider.



Logon Page Without Cookies

   Otherwise, the sign-in page lists your accounts on custom identity providers with which you previously logged on.



Logon Page With Previous Logon

2. Choose your identity provider and log on.

Related Information

Log On with a Custom Identity Provider to the Cloud Foundry Environment Using the Cloud Foundry Command-Line Interface [page 2587]

6.1.1.9.1.3 Default Identity Provider

SAP ID service is the default identity provider for both platform users and business users (in applications) at SAP BTP. You can start using it without further configuration.

> → Recommendation
>
> We strongly recommend that you configure your custom tenant of SAP Cloud Identity Services as the identity provider and connect SAP Cloud Identity Services to your own corporate identity provider.
>
> For more information, see Trust and Federation with Identity Providers [page 2538].

> ⓘ Note
>
> For China (Shanghai) region, a different default identity provider is used.

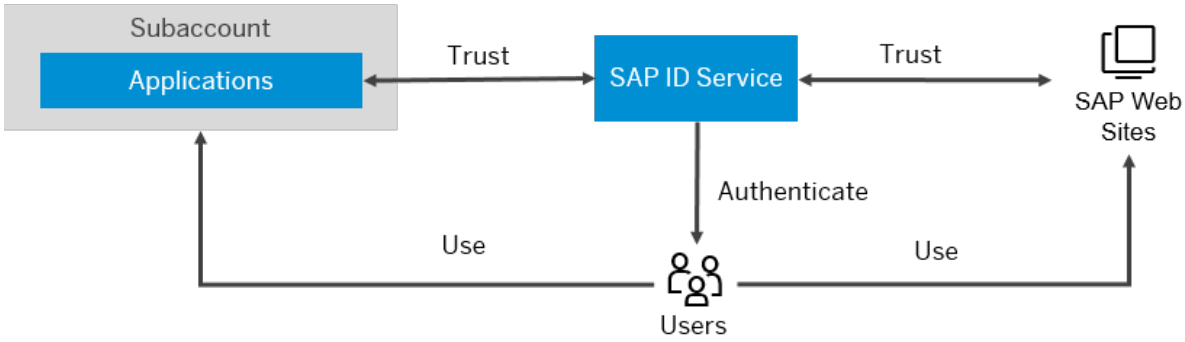
For more information, see this blog article on *SAP Community*.

For Government Cloud (US) region, a different default identity provider is used.

SAP ID service provides:

- A central user store
- A Single Sign-On (SSO) service. It enables users to log on once and get access to all your applications.

Use SAP ID service as a preconfigured user store in your starter scenarios or for testing. You can also use the default identity provider as a backup identity provider if access to your custom identity provider fails. SAP ID service is the place where you register to get initial access to SAP BTP.



SAP ID Service Authenticates Users

If you're a new user, you can use the self-service registration option at the SAP Web site The SAP website registers you with your SAP user ID, which also registers you with SAP ID service.

Managing Users

To add users to a subaccount, the users must exist in an identity provider.

For more information about adding users to our default identity providers, see Create SAP User Accounts [page 2597].

To add new users to a subscribed app or service, such as Web IDE, add those users to your subaccount.

For more information, see Add Users from SAP ID Service for Multi-Environment Subaccounts [page 2598].

Multifactor Authentication

For SAP ID service, you can ask users hosted by this identity provider to enable multifactor authentication as a self-service. There's **no** mechanism to enforce users in SAP ID service to use this function.

→ Recommendation

To enforce multifactor authentication, we recommend that you trust SAP Cloud Identity Services - Identity Authentication and configure enforcement. Optionally, integrate SAP Cloud Identity Services with your corporate identity provider and enforce the policy there.

For more information, see:

- Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576]
- Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2540]

For more information about your users of SAP ID service can enable multifactor authentication themselves, see How to Enable Multi-Factor Authentication (MFA) on the *SAP Support Portal*.

> **Note**
>
> Some user interfaces don't offer an interactive way to support multifactor authentication, such as time-based one time passwords (TOTP). Instead, such tools offer fixed logon ID and password fields. For such tools, enter your password directly followed, without any spaces or dividers, by the TOTP offered by your multifactor device.
>
> `User ID: MylogoniD`
>
> `Password: MystrongpassworDMytotppasscodE`
>
> For some tools, this behavior affects log on to the tool itself:
>
> - Cloud Foundry command-line interface (cf CLI)
> Alternatively, you can log on through a browser instead.
> For more information, see Log On Manually With a Custom Identity Provider [page 2589].
> - SAP BTP command line interface (BTP CLI)
> Alternatively, you can log on through a browser instead.
> For more information, see Log in with Single Sign-On [page 2683].
>
> For other tools, this behavior doesn't affect log on to the tool itself, but log on to the platform when establishing connections or deploying software and such. For example:
>
> - Cloud Connector
> - SAP Business Application Studio
> - SAP Web IDE

Trust and Identity Providers

The system preconfigures trust between your subaccount and SAP ID service by default, so you can start using the service without further configuration.

For web-based access, such as through a browser, you can hide the default trust.

For more information, see Hide Logon Link for Default Identity Provider [page 2558].

For non-interactive logons, such as with CLI tools or password grant flow, you can make a custom identity provider the default.

For more information, see btp update security/settings.

To use a custom identity provider, establish trust with your custom identity provider. We describe a custom trust configuration using the example of SAP Cloud Identity Services.

For more information, see Trust and Federation with Identity Providers [page 2538].

Related Information

Security [page 3536]
Platform Identity Provider for Tools
Working with Users [page 2603]

6.1.1.9.1.3.1 Create SAP User Accounts

To grant authorizations to people from the default identity provider in your subaccount, ensure that they have a user account.

Context

If the person in question already has a user account on `sap.com` Web sites, then that person already has a user in the default identity provider. Add this user to your accounts, if you know this person's e-mail address.

> → Recommendation
>
> We strongly recommend that you configure your custom tenant of SAP Cloud Identity Services as the identity provider and connect SAP Cloud Identity Services to your own corporate identity provider.
>
> For more information, see Trust and Federation with Identity Providers [page 2538].

> ⓘ Note
>
> For technical users, create the user in your custom identity provider. In your custom identity provider, you control the access policies of these users independently from the policies we set for the default identity provider. Our default identity provider is meant for **human** users, who interact with SAP and our solutions.

Procedure

Send your colleagues the self-registration URL.

```
https://account.sap.com/core/create/register?
redirectURL=https%3A%2F%2Femea.cockpit.btp.cloud.sap%2Fcockpit%2F
```

The Web site registers you with SAP Universal ID, which also registers you with the SAP ID service. If you already have a user in the SAP ID service, associate this user with your new SAP Universal ID account. SAP Universal ID manages the users of official SAP sites, including the SAP developer and partner community.

> → Tip
>
> After your colleagues sign up, add them to a subaccount. Assign role collections to them. Then your colleagues can log on to the SAP BTP cockpit.

Related Information

Add Users from SAP ID Service for Multi-Environment Subaccounts [page 2598]
Default Identity Provider [page 2594]
Trust and Federation with Identity Providers [page 2538]

6.1.1.9.1.3.2 Add Users from SAP ID Service for Multi-Environment Subaccounts

Before you can assign role collection to a user from SAP ID service, ensure that this user exists in your subaccount.

Prerequisites

The user you want to add to your subaccount must have an SAP user account (for example, an S-user or P-user).

For more information, see Create SAP User Accounts [page 2597].

Context

When you create your own trial account, your SAP user is automatically created and assigned to SAP ID service. When you onboard new members to your subscribed app, add them to your subaccount. Then you can assign a role collection to the user.

Procedure

Create users and assign role collections to users in *Security* > *Users*.

It's possible to create users in custom identity providers and in the SAP ID service.

For more information, see Working with Users [page 2603].

Related Information

Platform Identity Provider for Tools
Mapping Role Collections [page 2620]

6.1.1.9.1.3.3 Deactivate the Default Identity Provider

You can deactivate the default identity provider for an individual global account and subaccount.

Prerequisites

The prerequisites differ depending on the account type. Make sure you fulfill all conditions for your account type before proceeding.

Account Type	Prerequisites
Global account	• The global account administrator must have signed in using a custom platform trust configuration, not the default trust configuration.

Subaccount	• The subaccount administrator must have signed in using a custom platform trust configuration, not the default trust configuration.
	• At least one custom trust configuration for application users is available in the subaccount.
	• **Note**
	To deactivate the default trust configuration, you can also use the following command of the SAP BTP command line interface (BTP CLI).
	`btp [OPTIONS] update security/trust ORIGIN [--subaccount [ID]] [--status [STATUS]]`
	Example
	`btp update security/trust sap.default --subaccount 195a6e-c379-5041-87b2-2baa7e7fdea3 --status inactive`
	For more information, see Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707].

Context

To deactivate the default identity provider for a global account and subaccount, take the following steps.

Procedure

1. Go to your global account or subaccount (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Trust Configuration* in the SAP BTP cockpit.

   The list of trust configurations appears showing the default trust configuration and the custom trust configurations.
2. Choose the default identity provider.
3. Choose *Edit* in the details screen.
4. To deactivate the default identity provider, go to *Status*, and choose *Inactive*.
5. Save your changes.

6.1.1.9.1.4 Switch Off Automatic Creation of Shadow Users

To switch off the creation of shadow users in the trust configuration of custom identity providers, administrators must explicitly allow users to log on. Administrators then have full control over who is allowed to log on.

Context

Whenever a user authenticates at an application in your subaccount using any identity provider, the User Account and Authentication service always stores user-related data provided by the identity provider in the form of shadow users. The UAA uses details of the shadow users to issue tokens that refer to a certain user.

By default, the UAA allows any user of any connected identity provider to authenticate to applications in the subaccount. When there's no corresponding shadow user, it automatically creates one based on the information received from the identity provider.

> ⓘ Note
>
> With new subaccounts created after 24 September 2020, automatic creation of shadow users is switched off by default for the default identity provider, SAP ID service.

Usually, you want your administrators to be fully aware of which users they allow to log on. If you've switched off automatic creation of shadow users for a certain identity provider, you enforce that only those users can log on where shadow users have been created explicitly. You can create them in the SAP BTP cockpit, typically when you assign the first role collection to them. For more information, see Create Users [page 2608].

To switch off creation of shadow users during logon, proceed as follows:

Procedure

1. In the SAP BTP cockpit, go to your subaccount (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Trust Configuration*.
2. Choose your custom trust configuration.
3. Choose *Edit*.
4. Go to *Create Shadow Users on User Logon* and remove the checkmark.
5. Save your changes.

   From now on, the User Account and Authentication service doesn't automatically create shadow users for a certain identity provider during logon.

Related Information

Deletion [page 3638]
Delete Users [page 2611]
Delete Shadow Users for Data Protection and Privacy Using APIs [page 3642]

6.1.1.9.1.5 Refresh for Trusted Identity Providers

Trust refresh updates the SAP BTP trust configuration to reflect changes in the SAP Cloud Identity Services tenant. Use it to repair broken trust configurations while ensuring custom assertion attribute mappings are saved beforehand.

Prerequisites

You established trust with an identity provider.

For more information, see:

- Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2540]
- Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576]

Context

When you change the OIDC issuer in your SAP Cloud Identity Services tenant, the trust configuration in SAP BTP breaks. To restore trust, you refresh the trust configuration. Refreshing the trust updates SAP BTP to reflect the latest settings from your SAP Cloud Identity Services tenant, including the issuer value.

Note: For platform users, the global account automatically repairs the trust after 24 hours. To restore the trust immediately, you manually refresh the trust configuration.

Procedure

1. Back up attribute mappings of any applications.

   Refreshing the trust configuration resets the assertion attributes of associated applications in SAP Cloud Identity Services to their default values. You lose any customizations you made to the assertion attribute mappings. To preserve your customizations, save a copy of your current assertion attributes before you refresh.

   a. Retrieve the current assertion attributes.

      You can retrieve the current assertion attributes by accessing the following application URL of SAP Cloud Identity Services:

      `https://<tenant>.accounts.ondemand.com/Applications/v1/<application-ID>`

      You can retrieve the application ID from the SAP Cloud Identity Services admin console.

      For more information, see Configuring User Attributes from the Identity Directory.

      You receive a JSON file of the application configuration.

   b. In the results, locate `assertionAttributes`.

      The list shows the current attribute mappings. The following example illustrates this:

      ```
      "assertionAttributes": [
        { "assertionAttributeName": "first_name", "userAttributeName":
      "firstName", "inherited": false },
        { "assertionAttributeName": "last_name", "userAttributeName":
      "lastName", "inherited": false },
        { "assertionAttributeName": "mail", "userAttributeName": "mail",
      "inherited": false },
        { "assertionAttributeName": "user_uuid", "userAttributeName":
      "userUuid", "inherited": false },
        { "assertionAttributeName": "locale", "userAttributeName": "language",
      "inherited": false }
      ]
      ```

   c. Save a copy of the `assertionAttributes` list.

      After the refresh, you use this copy to restore your attribute mappings if needed.

2. In the SAP BTP cockpit, go to the trust configuration you want to refresh.
3. Choose the trust configuration to open its details.
4. Choose *Refresh*.

The trust configuration updates to reflect the latest settings from your SAP Cloud Identity Services tenant, including the issuer value.

5. In the SAP Cloud Identity Services admin console, go to the application associated with your trust configuration.
6. Compare the current assertion attributes with the copy you saved before the refresh.
7. Restore any missing or changed attribute mappings using your saved copy.

6.1.1.9.2 Working with Users

In the SAP BTP cockpit, you can see the users of your global account or subaccount, user-related identity provider information, and their authorizations. In a user's overview, you can create and delete users, and assign role collections. You can also display an overview of the role collections, where you can drill down all the way to the role, and see the application that the role belongs to.

Overview

All users in the global account, subaccount, directories, and environments are stored in identity providers. Users in the subaccounts of SAP BTP are either stored in the default identity provider or in a custom identity provider.

SAP BTP creates a copy of the user in the global account or in the subaccount when a user-related action happens. This copy of the user is called a shadow user. Global account or subaccount administrators of SAP BTP assign role collections to shadow users. To comply with data protection and privacy regulations, administrators can be obliged to delete shadow users who belonged to employees who left the company.

> → Remember
>
> For access tokens of subaccounts created after September 23, 2020, user IDs preserve the case that they are created in. This means that both uppercase and lowercase letters can be used. However, this does not qualify as a means to differentiate between them; for example, denise.smith@example.com and Denise.Smith@example.com are considered to be the same.
>
> Subaccounts before this date convert the user IDs to lowercase.

> ⓘ Note
>
> User data of shadow users is regularly updated after authentication at the identity provider.
>
> When a user authenticates against SAP BTP, an authentication request is forwarded to the trusted identity provider (for example, SAP Cloud Identity Services). After successful authentication, the identity provider creates a user token based on its configuration and sends it back to SAP BTP. In SAP BTP, the user data is updated according to the token.

Prerequisites

- You have administration rights in this global account, subaccount, or directory. For more information, see Security Administration: Managing Authentication and Authorization [page 2537] or Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138].
- The users are stored in identity providers that are connected to SAP BTP.
  - Default identity provider. For more information, see Default Identity Provider [page 2594].
  - Custom identity provider. For more information, see Trust and Federation with Identity Providers [page 2538].

Use Case	Description
Enable a new hire to log on to the subaccount and have access to necessary resources.	If automatic creation of shadow users was switched off for this identity provider, shadow users aren't created automatically in the subaccount (see the related link). This means that the new user can't log on to the subaccount. As an administrator, create a shadow user that corresponds to the user in the identity provider and assign the necessary role collections to this user. See Create Users [page 2608].
	If automatic creation of shadow users was switched off for this identity provider, shadow users aren't created automatically in the subaccount (see the related link). Add the user group to the role collections that are necessary for this user if the user belongs to a user group in the identity provider. See Assign User Groups to Role Collections [page 2618].
Enable a new hire to have access to necessary resources.	A shadow user is automatically created in the subaccount. This means that the new user can log on to the subaccount, but the user doesn't have any authorizations. As an administrator, you have created new role collections or you want to use existing ones for the new hire. You grant authorizations by assigning the role collections to this user. See Assign Users to Role Collections [page 2617].
Enable a new hire to log on.	If automatic creation of shadow users was switched off for this identity provider, shadow users aren't created automatically in the subaccount (see the related link). As an administrator, create a new shadow user in the subaccount. See Create Users [page 2608].
	If your subaccount uses the default trust configuration, you can also create a shadow user there. See Add Users from SAP ID Service for Multi-Environment Subaccounts [page 2598].

Use Case	Description
Review authorizations of individual users.	You want to find out which authorizations have been granted to users. To do so, access the list of all users, find the users you want to review, and check the role collection assignments. From the user's role collection, you can drill down to the roles of the role collection. You can also assign and remove role collections. See Find Users and Their Role Collection Assignments [page 2606].
Delete users of employees who have left the company.	You want to find and delete users of employees who have left the company, but their users still exist in the system. How to find users, see Find Inactive Users [page 2606]. How to delete users, see Delete Users [page 2611].
Automate the provisioning of users.	You want to provision users to SAP cloud solutions, such as SAP BTP so that you can manage your identities centrally. How to provision users to SAP BTP, see Provision Users to SAP BTP Accounts [page 2609].
Find users who have never logged on.	You want to find users who never logged on. For example, they could be manually created shadow users with a typo in their user ID, so they don't match the user ID provided by the identity provider. They were never used, and it might make sense to delete them. See Delete Users [page 2611].
Delete shadows user to comply with data protection and privacy regulations.	For data protection and privacy reasons, administrators might be legally obliged to delete shadow users who belonged to employees who left the company. You can find and delete the shadow users in the SAP BTP cockpit. See Delete Users [page 2611].
	You can also delete the users using APIs. See Delete Shadow Users for Data Protection and Privacy Using APIs [page 3642].

Related Information

Switch Off Automatic Creation of Shadow Users [page 2600]
Security Administration: Managing Authentication and Authorization [page 2537]
Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138]

6.1.1.9.2.1 Find Users and Their Role Collection Assignments

You want to find the authorizations granted to specific users in your global account and subaccount. You can search or sort the list of users, and view their role collection assignments.

Context

You want to find out which authorizations have been granted to single or multiple users. To do so, access the list of all users, find the users you want to review, and check the role collection assignment. From the user's role collection, you can drill down to the roles of the role collection.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and/or subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* > *Users*.

   You see all the users of your subaccount.

   > → Tip
   >
   > If you want to see users that were updated before a certain point of time, use the *Last Updated* field. You can filter the list according to the point of time of the last update.

4. To search for users, choose (Search).
5. Enter a string occurring in the user name, last name, first name, or e-mail address. The SAP BTP cockpit returns a list of all the users containing this string.
6. Select a user and open the overview section.

   You can see all of the role collection assignments of this user. You can also drill down into the roles.

6.1.1.9.2.2 Find Inactive Users

As an administrator, you want to have an overview of the users in your subaccount, no matter which identity provider stores them. You want to clean them up if necessary.

Context

- You want to find users that haven't been used for some time. The reason could be that employees left the company, but the users still exist in the system.

- You want to find users who never logged on. For example, they could be manually created shadow users with a typo in their e-mail address, so they didn't match the user ID provided by the identity provider. They were never used, and it might make sense to delete them.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and/or subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* > *Users*.

   You see all the users in your subaccount.
4. If you want to see users that were updated at a certain point in time, use the *Last Updated* field. You can filter the list according to the following filter criteria:

   - All
   - Today
   - This week
   - This month
   - This year

   If you want to see the users who haven't logged on at a certain point in time, use the *Hasn't Logged On* field. You can filter the list according to the following filter criteria:

   - All
   - Last 7 Days
   - Last 30 Days
   - Last 180 Days
   - Last 365 Days
   - Choose a Date

   The SAP BTP cockpit displays the filtered list of users. The *Last Updated* column shows you the point in time when the user was updated last. The *Last Logon* column shows you the point of time of the user's last logon.
5. (Optional) To see whether a user has never logged on, choose the user to display the user's overview section.

   The SAP BTP cockpit displays the user's overview section. If the user has never logged on, the *Last Logon* field shows the message *User has never logged on*.
6. (Optional) To delete the user, close the overview section and choose (Delete).

   > → Remember
   >
   > You can't undo the deletion of a user.

6.1.1.9.2.3 Create Users

As an administrator, you can create shadow users in your subaccount. When you create a shadow user, you must know and specify which identity provider stores the user.

Prerequisites

You've administration rights in this subaccount. For more information, see Security Administration: Managing Authentication and Authorization [page 2537] or Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138].

Context

All users in the subaccounts of SAP BTP are stored in identity providers, either in the default or in a custom identity provider. SAP BTP creates a copy of the user in the subaccount when a user-related action happens. The copy of the user in the subaccount is called shadow user. It's also possible to switch off the creation of shadows users (see the related link).

A new hire starts to work in your company next month. As an administrator, you want to provide a user, enable logon to the SAP BTP subaccount, and provide the necessary authorizations for this new hire. You create the actual user in the identity provider. However, you can't assign subaccount authorizations to an individual user in an identity provider. You need a shadow user in the SAP BTP subaccount to assign authorizations. After having created a shadow user, you can assign role collections. Thus, you make sure that the new hire can log on to the subaccount and that the necessary authorizations are in place when the new hire starts.

If you switched off the creation of shadow users in the trust configuration of your custom identity providers, new users created in the identity provider don't have a user copy (shadow user) in the respective SAP BTP subaccount. You, as a subaccount administrator, have full control of who can log on to this subaccount. The new user can log on to the subaccount if you've created a shadow user with the same email address in the SAP BTP subaccount. Now, you can grant authorizations by assigning role collections.

You can alternatively grant authorizations in the subaccount by mapping role collections to the user group if the user belongs to a user group in the identity provider.

As an administrator, you can create shadow users in your subaccount and you must determine which identity provider stores the user. You can then give the user authorizations by assigning role collections to the user.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and/or subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* > *Users*.
4. Choose *Create*.

The SAP BTP cockpit displays a new row where you can enter the user data.

5. Enter the user ID and email address.

> **Note**
>
> The email address of the user is the user identifier for all account levels (global account, directory, multi-environment subaccount) and for the Cloud Foundry environment.

> **Note**
>
> To specify the last name and first name of the user, maintain the names in the identity provider that stores the user. This information is automatically added to the shadow users when the user logs on with their identity provider using a URL such as:
>
> `https://<myapplication>.cfapps.<region>.hana.ondemand.com/`
>
> For example: `https://my-application.cfapps.eu10.hana.ondemand.com/`
>
> Logging on with the SAP BTP cockpit or the command-line interface isn't enough.

6. Choose the identity provider where the user is stored. The dropdown list displays the identity providers configured in the trust configuration of your subaccount.
7. Save your changes.

   You can now proceed to assign role collections to the new user.

Related Information

Switch Off Automatic Creation of Shadow Users [page 2600]

6.1.1.9.2.4 Provision Users to SAP BTP Accounts

As people join, change roles, and eventually leave your organization, you care for the onboarding, maintenance, and offboarding of their users in your systems. If you're a small organization, you can use manual processes; otherwise, you should consider automating your processes.

Prerequisites

- You have the required authorizations.
  For more information, see Security Administration: Managing Authentication and Authorization [page 2537] or Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138].
- For global account, directory, and subaccount levels, you have API credentials.
  For more information, see Accessing Administration Using APIs of the SAP Authorization and Trust Management Service [page 2773].

Context

We recommend creating users within the SAP Cloud Identity Services. The persistency layer of SAP Cloud Identity Services is the Identity Directory. Use SAP Cloud Identity Services - Identity Provisioning to provision users to SAP cloud solutions, such as SAP BTP. Identity Provisioning has connectors to distribute user artifacts needed at different levels of your SAP BTP accounts. By managing your identities centrally, you also ensure that when you offboard users, they're removed from target systems.

Procedure

Integrate with your identity and access management solution.

- To integrate with SAP Cloud Identity Services, see SAP BTP Integration Scenario for more information.
- To integrate with another solution, use the User Management SCIM API. See User Management (System for Cross-domain Identity Management (SCIM)) for more information.

> **Note**
>
> For user management of Cloud Foundry organizations, use the provisioning connectors of SAP Cloud Identity Services - Identity Provisioning. For more information, see SAP BTP Integration Scenario.

6.1.1.9.2.5 Export Users

To synchronize SAP BTP user data with other systems, you can export the data from a global account, subaccount, or directory to a CSV file.

Prerequisites

You have the relevant administrator authorizations for the account, from which you want to export users.

Procedure

1. Open the SAP BTP cockpit and navigate to your global account, subaccount, or directory.
2. From the side navigation, select *Security* > *Users*.
3. Select *(Export)*.
4. Choose *Export All Users* or *Export All Users with Role Collections*.

   The SAP BTP cockpit exports user data into a CSV file, which is downloaded to your browser's default download folder.

The CSV file contains these columns:

- User Name
- Identity Provider
- Last Name
- First Name
- E-Mail
- Last Updated
- Last Login
- Assigned Role Collection (only available if you choose *Export All Users with Role Collections*)

6.1.1.9.2.6 Delete Users

As an administrator, you can delete users from your subaccount. When you delete a user, you also delete the user's role collection assignments.

Prerequisites

You have the required authorizations.

For more information, see Security Administration: Managing Authentication and Authorization [page 2537] or Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138].

Context

All users in the subaccounts of SAP BTP are shadow users. As an administrator, you might want to clean up users from your subaccount. Or for data protection and privacy reasons, it might be a legal obligation that administrators delete shadow users who belonged to employees who left the company. For more information, see Working with Users [page 2603].

It's also possible to delete users using APIs. For more information, see the related links.

Keep in mind that you also delete the user's role collection assignments.

> → Remember
>
> When you delete a user at the Cloud Foundry level, SAP BTP doesn't delete the corresponding shadow user at the subaccount level.
>
> When you delete a user at the subaccount level, ensure that you delete the user at the Cloud Foundry level too. Otherwise, if the Cloud Foundry user logs on again, the system automatically creates the corresponding user at the subaccount level.
>
> So ensure that you delete the shadow users on all levels.

For more information, see About User Management in the Cloud Foundry Environment [page 2787].

> ⚠ Caution
>
> You cannot undo the deletion of a user.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account, directory, or subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* > *Users*.

   > ⓘ Note
   >
   > For directories, choose *Users*. There is no *Security* menu item.

4. Choose from the following options:

Options	Procedure
Delete a single user	1. Select the row of a user. 2. Choose *Delete*.
Delete multiple users	1. Choose *Select multiple users* and select users (see the related link). 2. Choose *Delete*.

Related Information

Working with Users [page 2603]
Deletion [page 3638]
Delete Shadow Users for Data Protection and Privacy Using APIs [page 3642]

6.1.1.9.3 Working with Role Collections

As an administrator, you group application roles in role collections. You then assign role collections to application users.

Application developers have defined application-specific role templates in the security descriptor file. The role templates contain the role definitions. You can assign the role to a role collection.

→ Tip

Application developers can even directly define role collections with default roles. For more information, see Create Role Collections with Predefined Roles [page 585].

Typically, these role collections provide authorizations for certain types of users, for example, sales representatives.

Once you have created a role collection, you can pick the roles that apply to the typical job of a sales representative. Since the roles are application-based, you must reference the application to see which roles come with the role template of this application. You are free to add roles from multiple applications to your role collection.

Finally, you assign the role collection directly to users or indirectly to attributes such as groups. For more information, see the related link.

ⓘ Note

A warning icon marks role collections that include critical security authorizations (see scopes). Review the authorizations carefully before you assign such a role collection.

Prerequisites

- Your user has administration rights in this subaccount and or global account. For more information, see Security Administration: Managing Authentication and Authorization [page 2537] or Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138].
- The roles defined by your application developers in the application security descriptor are available in the SAP BTP cockpit. For more information, see Adding Authentication and Authorization [page 546].
- The users are stored in identity providers that are connected to SAP BTP.
  - Default identity provider (`SAP ID service`). For more information, see Default Identity Provider [page 2594].
  - Custom identity provider. For more information, see User Management in SAP Cloud Identity Services.

ⓘ Note

SAP BTP provides default role collections. You can use the default role collections, but you can't change or delete them. For this reason, the *Delete* icon of a default role collection is grayed out.

You can perform the following tasks:

- Define a Role Collection [page 2614]
  - Add Roles to a Role Collection [page 2615]
  - Delete Roles from a Role Collection [page 2616]
- Assigning Role Collections to Users or User Groups [page 2616]
  - Assign Users to Role Collections [page 2617]
  - Delete Users from Role Collections [page 2618]
  - Assign User Groups to Role Collections [page 2618]

- Delete User Groups from Role Collections [page 2619]

Related Information

Security Administration: Managing Authentication and Authorization [page 2537]
Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138]
Limits for Technical Artifacts of the SAP Authorization and Trust Management Service [page 3605]

6.1.1.9.3.1 Define a Role Collection

To define a role collection, you can create a new role collection or copy an existing one. Add the roles, users, and user groups as needed.

Context

One way to define a role collection is to create a new role collection. After you've entered a name and description, you find the new role collection in the list of role collections. You can then add the roles you need and assign users and user groups.

If you have an existing role collection that you want to use as a template, it's a good idea to copy it. The copied role collection includes all of the roles of the origin. However, it doesn't include the users or user groups. You can give it a new name and description. You can find the copy in the list of role collections and assign users and user groups.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* > *Role Collections*.
4. To create a new role collection, choose *Create*. To copy an existing role collection, choose *Copy* at the end of the row.
5. Enter a new name and description. If you copied an existing role collection, you can see the included roles.
6. Save your changes.

   You can now add or remove roles and assign users or user groups.

6.1.1.9.3.1.1 Add Roles to a Role Collection

You can add roles to a role collection.

Context

The roles are derived from role templates that are provided by applications. For more information, see the related links.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* > *Role Collections*.
4. To add roles, choose the role collection name.
5. Go to the *Roles* section and choose *Edit*.
6. To add a role to the role collection, choose the input field. The role selection screen opens.
7. Use the dropdown list or the F4 function key under *Role Name* to display the roles that are available. The roles are sorted alphabetically. Choose the role you want to add. You can also select multiple roles from the list below.
8. Choose *Add*.
9. Save your changes.

Related Information

Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138]
Protecting Your Application [page 549]

6.1.1.9.3.1.2 Delete Roles from a Role Collection

You can delete roles from a role collection.

Context

The roles are derived from role templates that are provided by applications. For more information, see the related links.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* > *Role Collections*.
4. To delete roles, choose the role collection name.

   To drill down into the role details, choose the chevron on the right side of the row.
5. Choose *Edit*.
6. Choose the (Select multiple roles) button.
7. Select the roles you want to delete from the role collection.
8. Choose *Remove*.
9. Confirm the popup.
10. Save your changes.
11. Repeat to delete more roles.

6.1.1.9.3.2 Assigning Role Collections to Users or User Groups

You can assign role collections to users and user groups.

Prerequisites

- You have administration rights in this subaccount or global account. For more information, see Security Administration: Managing Authentication and Authorization [page 2537] or Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138].
- The roles defined by your application developers in the application security descriptor are available in the SAP BTP cockpit. For more information, see Adding Authentication and Authorization [page 546].

- The users are stored in identity providers that are connected to SAP BTP.
  - Default identity provider (`SAP ID service`). For more information, see Default Identity Provider [page 2594].
  - Custom identity provider. For more information, see User Management in SAP Cloud Identity Services.

You perform the following tasks to assign role collections to users and user groups stored by identity providers. The identity providers are connected to SAP BTP using trust configurations.

> **Note**
>
> Custom identity providers can provide user groups and users whereas default identity providers can only provide individual users.

- Assign Users to Role Collections [page 2617]
- Delete Users from Role Collections [page 2618]
- Assign User Groups to Role Collections [page 2618]
- Delete User Groups from Role Collections [page 2619]

6.1.1.9.3.2.1 Assign Users to Role Collections

You can assign users to role collections by adding them to the desired role collection in the SAP BTP cockpit. Use this function to manage user authorizations and access control within your global account or subaccount in the cloud platform.

Context

You can assign users to a role collection. After having entered the user's user ID, choose the name of the trust configuration and the e-mail address.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account or subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* > *Role Collections*.
4. Choose the role collection to which you want to assign users.
5. Go to the *Users* section and choose *Edit*.
6. Enter the user ID, identity provider, and e-mail address of the user that you want to assign to the role collection.
7. (Optional) To add more users, choose + (Add a user).
8. Save your changes.

   You've now assigned this user to the role collection. The user has the authorizations of the role collection.

6.1.1.9.3.2.2 Delete Users from Role Collections

You can unassign users from a role collection by deleting them from the role collection.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* > *Role Collections*.
4. Choose the role collection from which you want to unassign users.
5. Choose *Edit*.
6. Choose (Delete) in the row of the user you want to unassign.
7. Save your changes.

   You've now unassigned this user by deleting the user from the role collection.

6.1.1.9.3.2.3 Assign User Groups to Role Collections

You can assign users groups to a role collection by adding them to the role collection.

Prerequisites

- You're using a custom identity provider. For more information, see Trust and Federation with Identity Providers [page 2538].

Context

You can assign user groups from custom identity providers to a role collection. After having chosen the identity provider, enter the user group name.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and subaccount (see Navigate in the Cockpit [page 2461]).

3. Choose *Security* > *Role Collections*.
4. Choose the role collection to which you want to assign user groups.
5. Go to the *User Groups* section and choose *Edit*.
6. Select the identity provider where the user group is stored.
7. Enter the name of the user group.

> **Note**
>
> If you're using SAP Cloud Identity Services, you can find the name of the user group in the administration console under *Users & Authorizations* > *User Groups*. For more information, see the related link.

8. (Optional) To add other user groups, choose + (Add a user group).
9. Save your changes.

Related Information

User groups in the SAP Cloud Identity Services

6.1.1.9.3.2.4 Delete User Groups from Role Collections

You can unassign users groups from a role collection by deleting them from the role collection.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* > *Role Collections*.
4. Choose the role collection from which you want to unassign a user group.
5. Go the to *User Groups* section and choose (Delete).
6. Confirm the popup and save your changes.

6.1.1.9.3.3 Mapping Role Collections

You've arranged roles in role collections, and now want to assign or map these role collections to business users.

How you assign users to their authorizations depends on the type of trust configuration and on whether or not you prefer to maintain the authorizations of individual users rather in the identity provider or in SAP BTP. The following options are available:

- Directly assign role collections to users.
- Map role collections to user groups or other user attributes defined in the identity provider. You initially maintain the mapping between user groups or other user attributes and role collections once in SAP BTP, and maintain group memberships or other attributes of users in the identity provider.

> ### Note
>
> If you're using the default trust configuration with the default identity provider, you directly assign users to role collections. For more information, see Default Identity Provider [page 2594].
>
> However, if you're using a custom trust configuration, for example, with SAP Cloud Identity Services, you can use more options. For more information about configuring the trust between your subaccount and a custom identity provider, see Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2540]. For more information about configuring the trust between your global account and a custom identity provider, see Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576].
>
> If you use the same SAP Cloud Identity Services tenant for both platform and business users, the service saves the mapping once per tenant. The mapping applies to both platform users and business users.

Options for Assignment of Role Collections

Trust Configuration	Assignment Options
Default trust configuration (SAP ID service)	Assign Users to Role Collections [page 2617]
Custom trust configuration (for example: a tenant of SAP Cloud Identity Services)	• Assigning Role Collections to Users or User Groups [page 2616] • Map Role Collections to User Groups [page 2621] • Map Role Collections to User Attributes [page 2622]

Related Information

Working with Role Collections [page 2612]
Trust and Federation with Identity Providers [page 2538]

6.1.1.9.3.3.1 Map Role Collections to User Groups

You want to assign a role collection to a user group provided by an identity provider that has a custom trust configuration in SAP BTP. In this case, the assignment is a mapping of a user group to a role collection. Your identity provider provides the user groups using the attribute called `Groups`. Each value of the attribute is mapped to a role collection as described in this procedure.

Prerequisites

- You've configured your custom identity provider and established trust with your subaccount.

  > → Remember
  >
  > The name of the trust configuration is different from SAP ID service. The name of a custom trust configuration to SAP Cloud Identity Services could be as follows:
  >
  > `https://<tenant_id>.accounts.ondemand.com`

- You've configured the identity provider so that it conveys the user's group memberships in the *groups* attribute.
  - For business users, see Map User Attributes from a Corporate Identity Provider for Business Users [page 2543].
  - For platform users, see Map User Attributes from a Corporate Identity Provider for Platform Users [page 2581].
- You've created role collections.

Context

The identity provider provides the users, who can belong to user groups. It's efficient to map user groups to role collections. The role collection as a reusable element contains the authorizations that are necessary for this user group. Mappings save time when you want to add a new user. Simply add the user to the respective user group or groups, and the user automatically gets all the authorizations that are included in the role collections.

Procedure

To map a user group to a role collection, see Assign User Groups to Role Collections [page 2618].

You can also map role collections to attributes other than groups. For more information, see the related links.

Related Information

Trust and Federation with Identity Providers [page 2538]
Map Role Collections to User Attributes [page 2622]

6.1.1.9.3.3.2 Map Role Collections to User Attributes

Map role collections to users dynamically through the use of user attributes. When user agents present the attributes of their users, the SAP Authorization and Trust Management service can assign role collections based on the values of those attributes.

Prerequisites

- You've configured your custom identity provider and established trust with your subaccount.
- You've configured the identity provider so that it conveys the user attributes in the assertion attributes.
  - For business users, see Map User Attributes from a Corporate Identity Provider for Business Users [page 2543].
  - For platform users, see Map User Attributes from a Corporate Identity Provider for Platform Users [page 2581].
- You've created role collections.

Context

The identity provider hosts the users and their attributes. When a user is authenticated by the SAP Authorization and Trust Management service, a user agent presents a number of attributes of the user, such as an e-mail address. The SAP Authorization and Trust Management service uses this attribute to identify the user. You can use such attributes to assign role collections. For example, you can use the `company_region` attribute to assign role collections specific to the provinces or states your users are located. More use cases are to have different role collections for different cost centers or job functions. Mapping role collections to the attribute `Groups` is so common that we have a separate use case for it. For more information, see *Map Role Collections to User Groups* in the related links.

> → Tip
>
> Mappings save time when you add new users. Simply set the relevant attributes in the identity provider for the users, and the users automatically gets all the authorizations that are included in the role collections.

Procedure

1. Go to your subaccount (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Role Collections*.
2. Search for and choose a role collection.
3. In the overview page for the role collection, choose the *Edit* button.
4. Under *Attribute Mapping*, select an identity provider.
5. Enter the name of the attribute and the value for the SAP Authorization and Trust Management service to use to map the role collection to users. The attributes are combined with logical OR operations. If you assign multiple attributes to the role collection, and one of them is true, the users with this attribute are assigned to this role collection.

   > → Tip
   >
   > Provide the **exact** name of the attribute and its value as provided by the identity provider. These values are case-sensitive.
   >
   > For more information, see Settings for Default SAML Federation Attributes of Identity Providers for Business Users [page 2554] and Configure the User Attributes Sent to the Application in the documentation of SAP Cloud Identity Services.

6. Save your entries.

Related Information

Trust and Federation with Identity Providers [page 2538]
Map Role Collections to User Groups [page 2621]
Create Role Collections with Predefined Roles [page 585]

6.1.1.9.4 Building Roles and Role Collections for Applications

As an administrator, you can maintain application roles and role collections which can be used in user management.

The user roles are derived from role templates that are defined in the security description (`xs-security.json`) of applications that have been registered as OAuth 2.0 clients at the User Account and Authentication service during application deployment. The application security-descriptor file also contains details of the authorization scopes that are used for application access and defines any attributes that need to be applied. The roles you create can be added to role collections.

You can perform the following tasks:

- Create and delete user roles
  User roles define authorization scopes and are based on the role templates and scopes defined in the application's security descriptor file

- Add user roles to one or more “role collections”
- Configure and manage role collections

> → Tip
>
> Using the SAP BTP cockpit, you can assign the role collections to users directly or indirectly over attributes, such as groups.

Related Information

Add Roles to Role Collections on the Application Level [page 2632]
Mapping Role Collections [page 2620]
Protecting Your Application [page 549]
Working with Role Collections [page 2612]
Limits for Technical Artifacts of the SAP Authorization and Trust Management Service [page 3605]
Adding Authentication and Authorization [page 546]

6.1.1.9.4.1 Roles and Role Collections

Usually a role collection consists of one or multiple roles. You can use the SAP BTP cockpit to add or remove roles.

Role

A role is an instance of a role template; you can build a role based on a role template and assign the role to a role collection. The SAP BTP cockpit helps you to display information about the selected application and any related roles in the following windows, tabs, and panes:

- Roles
- Scopes
- Attributes
- Role templates

Role Collection

Roles are assigned to role collections which are assigned in turn to users or user groups if an SAML 2.0 identity provider stores the users. Using the SAP BTP cockpit, you can display information about the role collections that have been maintained as well as the roles available in a role collection. Additional information includes:

which templates the roles are based on, and which applications the roles apply to. Role collections enable you to group together the roles you create. The role collections you define can be assigned as follows:

- To users logged on to the `SAP ID service`.
- To user groups containing users logging on with SAML 2.0 assertions.

Related Information

Attributes [page 2628]
Security Administration: Managing Authentication and Authorization [page 2537]

6.1.1.9.4.2 Create Roles for Applications Using Existing Role Templates

Use the SAP BTP cockpit to create a role using an existing role template. You can refine the role by assigning attributes and add the role to role collections.

Context

You can use existing role templates to create new roles. An SAP BTP cockpit wizard helps you to configure new roles.

> **Note**
>
> You can only create roles from roles templates that come with attributes.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* in the navigation pane.
4. Choose *Roles*.

   Here you see a complete list of all existing roles sorted by application name. It also contains the role template, role names, and role description. On the right side, you find the action buttons.
5. Choose *Create Role* in the row of the role template that you want to use.

   The *Create Role* wizard opens.
6. Enter a role name and a description, and choose *Next*.

7. Configure the attributes and choose *Next*. The attributes further refine the role. For more information, see the related link.
8. Select the available role collections for your new role and choose *Next*.
9. You can review your role configuration in the next window and complete the creation of the role by choosing *Finish*.

Related Information

Attributes [page 2628]

6.1.1.9.4.3 Create Roles for Subscribed Applications Using Role Templates

Use the SAP BTP cockpit to create a role for a subscribed application using a role template. You can refine the role by assigning attributes and add the role to role collections.

Context

You can use role templates to create new roles. An SAP BTP cockpit wizard helps you configure new roles.

> **Note**
>
> You can only create roles from role templates that come with attributes.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your subaccount (see Navigate in the Cockpit [page 2461]).
3. In the navigation pane, choose *Services* > *Instances and Subscriptions*.
4. To view your subscribed application, choose the *Subscriptions* tab.
5. Choose ••• *(Actions)* next to your subscribed application, then choose *Manage Roles*.

   A complete list of all roles opens, sorted by the role template. The list also contains role names and role descriptions.
6. Choose *Create*.

   The *Create Role* wizard opens.
7. Enter a role name and description, then choose *Next*.

8. Configure the attributes, then choose *Next*. Attributes further refine the role. For more information, see the related link.
9. Select the available role collections for your new role, then choose *Next*.
10. Review your role configuration, then choose *Finish*.

Next Steps

If needed, use + (Add to role collection) to add a role to a role collection.

Related Information

Attributes [page 2628]
Create Roles for Applications Using Existing Role Templates [page 2625]

6.1.1.9.4.4 Delete Roles

Use the SAP BTP cockpit to delete a role. If this role has been added to a role collection, it's also deleted from the role collection.

Context

You can use existing role templates to create new roles. An SAP BTP cockpit wizard helps you to configure new roles.

> **Note**
>
> You cannot delete roles that belong to default role collections delivered by SAP BTP.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* in the navigation pane.
4. Choose *Roles*.

   Here you see a complete list of all existing roles sorted by application name. It also contains the role template, role names, and role description. On the right side, you find the action buttons.

5. Choose the (Delete) action button in the row of the role that you want to delete. If the role has been added to a role collection, it's also deleted from the role collection.
6. Confirm that you want to delete the role.

6.1.1.9.4.5 Attributes

Attributes use information that is specific to the user, for example the user's country. If the application developer in the Cloud Foundry environment of SAP BTP has created a country attribute to a role, this restricts the data a business user can see based on this attribute.

> → Recommendation
>
> We recommend implementing instance-based authorizations with SAP Cloud Identity Services – Authorization Management. This topic remains relevant for managing instance-based authorizations for applications that use the "attributes" concept of the SAP Authorization and Trust Management service. Note that some applications might use authorization policies instead, which you manage in SAP Cloud Identity Services. For more information, see Configuring Authorization Policies.

A lot of applications provide purely functional role templates which grant access for all data of a certain type within your subaccount. Roles for such role templates are generated automatically. Some other applications also provide the possibility for administrators to restrict access not only by functional authorizations, but also by instance-based authorizations. That means that users can only work with a certain subset of the data in your subaccount.

The restriction can be either based on information within the respective role, or on user-specific information provided by the identity provider. This makes instance-based authorizations specific for each customer because the respective roles cannot be generated automatically. Instead, administrators must create them. Typical restrictions depend on information like the user's country or cost center.

Each restriction is represented by a dedicated attribute which belongs to a role template of the application.

> ⓘ Note
>
> Application developers can define attribute references in the application security descriptor file (`xs-security.json`): For more information, see Application Security Descriptor Configuration Syntax [page 592].

For each attribute, administrators have multiple options to specify the value which restricts data access:

- Static attributes
  They are stored in the role. The user is given the attribute value when the administrator assigns the role collection with this role to the user.
- Attributes from a custom identity provider
  Since an identity provider provides the business users, you can dynamically reference all attributes that come with the SAML 2.0 assertion. You define the attributes and the attribute values in the identity provider. In the Cloud Foundry environment of SAP BTP, administrators can use the attributes to refine the roles.

> **Note**
>
> If you want to reference attributes from your identity provider, you must know the exact identifier of the attribute. Go to the SAML 2.0 or OpenID Connect configuration of your identity provider and use the attributes as they are defined there.

- Unrestricted attributes
  In this case, you want to express that it is not necessary to set a specific value for this attribute. The behavior is the same as if the attribute would not exist for this role.

Related Information

Specify Attributes in a Role [page 2629]
Specify Attributes in a New Role [page 2631]

6.1.1.9.4.5.1 Specify Attributes in a Role

As an administrator of the Cloud Foundry environment, you can specify attributes in roles to refine authorizations of the business users. Depending on these attributes, business users with this role have restricted access to data.

Prerequisites

You have maintained the attributes of the users in your identity provider.

> **Note**
>
> In SAP Cloud Identity Services or any identity provider, you find the attributes in the configuration.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your subaccount.

   For more information, see Navigate in the Cockpit [page 2461].
3. Choose your space in *Cloud Foundry* > *Spaces*.
4. Choose the application.
5. Choose *Security* > *Roles*.
6. Select a role.

The role overview pane displays the attribute name and fields where you can select the source and enter a value.

7. Choose *Edit*.
8. To specify an attribute, choose the source of the attribute. The following sources are available:

Attribute Sources

Source	Value/Attribute
Static	Enter a static value, for example **USA** to refine the role depending on the country. → Remember Follow entries with the `Enter` key. Otherwise you can't save the role.
Identity Provider	Enter an attribute as defined in your identity provider. Check in your identity provider for the exact syntax of the attribute identifier. For SAP Cloud Identity Services, you find the attribute identifier in the settings of the attributes of your identity provider under *Applications & Resources* > *Applications* > `<Application_Name>` > *Trust* > *Attributes*. Example To use the attribute for cost center, you must enter the value **`cost_center`**.
Unrestricted	In this case, you want to express that it is not necessary to set a specific value for this attribute. The behavior is the same as if the attribute would not exist for this role.

9. Save your changes.

Related Information

Subscribe to Multitenant Applications Using the Cockpit [page 2534]
Attributes [page 2628]

6.1.1.9.4.5.2 Specify Attributes in a New Role

As an administrator, you can specify attributes in a new role to refine authorizations of business users. Depending on these attributes, business users with this role have restricted access to data.

Prerequisites

You have maintained the attributes of the users in your identity provider.

> **Note**
>
> In SAP Cloud Identity Services or any identity provider, you find the attributes in the configuration.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account (China (Shanghai) region) or subaccount. For more information, see Navigate in the Cockpit [page 2461].
3. Choose your space in *Cloud Foundry* > *Spaces*.
4. Choose the application.
5. Choose *Security* > *Roles*.
6. To create a new role, choose *Create*.

   A wizard guides you through the role creation process.
7. Enter a name and a description of the new role.
8. Select the role template that you want to use.
9. Choose *Next*.
10. To specify an attribute, choose the source of the attribute. The following sources are available:

    Attribute Sources

Source	Value/Attribute
Static	Enter a static value, for example **USA** to refine the role depending on the country/region.

Source	Value/Attribute
Identity Provider	Enter an attribute as defined in your identity provider. Check in your identity provider for the exact syntax of the attribute identifier. For SAP Cloud Identity Services, you find the attribute identifier in the settings of the attributes under *Applications & Resources* > *Applications* > `<Application Name>` > *Trust* > *Attributes*. **Example** To use the attribute for cost center, you must enter the value **`cost_center`**.
Unrestricted	In this case, you want to express that it isn't necessary to set a specific value for this attribute. The behavior is the same as if the attribute wouldn't exist for this role.

11. Choose *Next*.
12. Select the role collections for your new role. For more information, see the related link.
13. Choose *Next* and *Finish*.

    You have now created a new role with attributes.

Related Information

Subscribe to Multitenant Applications Using the Cockpit [page 2534]
Attributes [page 2628]
Specify Attributes in a New Role [page 2631]

6.1.1.9.4.6 Add Roles to Role Collections on the Application Level

Roles are used to define the type of access granted to an application.

Context

A role is an instance of a role template; you can build a role based on a role template and assign the role to a role collection. Role collections are then assigned to SAML 2.0 groups or users. The role template defines the type of access permitted for an application, for example: the authorization scope, and any attributes that need

to be applied. Attributes define information that comes with the respective user, for example 'cost center' or 'country' (see the related link). This information can only be resolved at run time.

> **Note**
>
> The User Account and Authentication service automatically creates default roles for all role templates that do not include attribute references. They have the same name as the role template. You can't delete them. You can recognize them because their description contains *Default Instance*.

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* in the navigation pane or go to *Cloud Foundry* > *Spaces* > *<your_space>* > *<your_application>*.
4. Choose *Security* > *Roles* in the navigation pane.

   Here you see a complete list of all roles sorted by role templates. It also contains information about attributes used in this role and about the role collections the role has been added to. On the right side, you find the action buttons.

   > **Note**
   >
   > You cannot edit or delete predefined roles for default role collections. For this reason, the action buttons are grayed out. For more information, see the related link.

5. To directly assign a role to role collections, choose the action button + (Add to role collection) in the same row.

   A new window shows all role collections that are available in your global account or subaccount.
6. Select the role collections to which you want to add your role.
7. Choose *Add*.

   The number in the role collections column counts up. You have now assigned this role to the role collections you selected earlier.

Related Information

Attributes [page 2628]
Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138]

6.1.1.9.5 Managing Secrets of the SAP Authorization and Trust Management Service

The SAP Authorization and Trust Management service maintains a number of secrets to ensure secure operation of the service. Your organization can have policies that require you change secrets or you may need to respond to the loss of a secret.

The SAP Authorization and Trust Management service uses the following secrets in its operation:

- Service Instance Secrets [page 2634]
- Rotate Binding Secrets [page 2636]
- Migrate from Instance Secrets to Binding Secrets [page 2638]
- Rotating Instance Secrets [page 2640]
- Binding Parameters of SAP Authorization and Trust Management Service [page 2640]
- Rotate Signing Keys of Access Tokens [page 2646]
- Rotate Signing Keys of SAML Token [page 2652]

6.1.1.9.5.1 Service Instance Secrets

When an application consumes a service instance of the SAP Authorization and Trust Management service (XSUAA), the application identifies itself to the service instance with a client ID and a secret. The client ID and secret are the credentials with which an application authenticates itself to the service instance.

The SAP Authorization and Trust Management service (XSUAA) creates these credentials either when you bind the application to the service instance or when you create a service key for the service instance.

The service instance can use multiple secrets in the `application` plan.

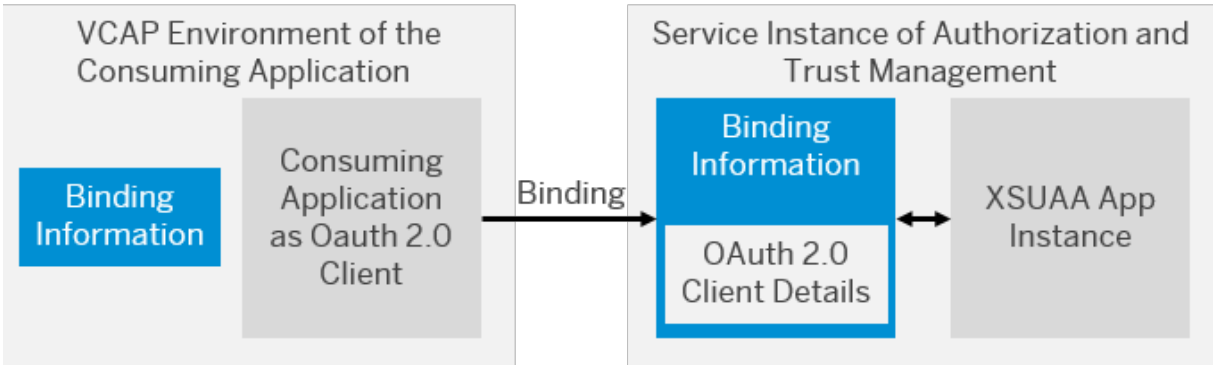
- The instance secret is the same for all bindings of the service instance. The secret remains valid as long as the instance exists.

  > ### Note
  >
  > Starting with January, 19, 2026, the `instance-secret` credential type isn't supported anymore for new bindings and service keys. We recommend to remove old bindings with this credential type.

- The binding secret is the default secret type. Binding secrets must be enabled in the application security descriptor (`xs-security.json`) when you create the service instance. When you bind an application to a service instance or create a service key, you can pass a `parameters.json` to use a binding secret. The secret remains valid as long as the binding or the service key exists.
- X.509 secrets must be enabled in the application security descriptor (`xs-security.json`) when you create the service instance. When you bind an application to a service instance or create a service key, you can pass a `parameters.json` to use an X.509 secret. SAP Authorization and Trust Management service can generate an X.509 certificate for you or you can provide your own. The X.509 secret remains valid as long as the certificate itself is valid.

> ### Note
>
> The service doesn't check for certificate revocation. To stop the service from accepting a certificate that is still valid, delete the relevant bindings or service keys. As soon as the binding is deleted, the service stops accepting the certificate.
>
> For bindings with self-managed certificates and the `certificate-pinning` parameter set to `false`, you can rotate the secrets without deleting bindings. Just use a new certificate with the same subject and issuer distinguished name (DN). The service saves the new validity date of the new certificate.
>
> For more information, see Parameters for Self-Managed X.509 Certificates [page 2643].

The following figure illustrates the XSUAA app and the binding information about the OAuth 2.0 client as part of a service instance of the SAP Authorization and Trust Management service. A consuming application, functioning as an OAuth 2.0 client is bound to the SAP Authorization and Trust Management service instance. The secret is part of the environment of the consuming application and the information about the OAuth 2.0 client saved with the XSUAA app. Alternatively, this information is saved as part of a service key.



Binding Between an SAP Authorization and Trust Management Service Instance and a Consuming Application

The `credential-types` parameter of the OAuth client configuration in the application security descriptor (`xs-security.json`) determines which types of secrets that bindings support.

In the following example, the service instance creates a binding secret for all new bindings. It allows the creation of X.509 secrets.

> ### Example
>
> #### Sample Code
>
> ```
> "oauth2-configuration": {
>     "credential-types": ["binding-secret","x509"]
> }
> ```

In the following example, the service instance creates a binding secret for all new bindings, but doesn't accept the instance secret or the use of X.509 secrets.

> ### Example
>
> #### Sample Code
>
> ```
> "oauth2-configuration": {
> ```

```
    "credential-types": ["binding-secret"]
}
```

Related Information

Rotate Binding Secrets [page 2636]
Rotating Instance Secrets [page 2640]
Migrate from Instance Secrets to Binding Secrets [page 2638]
Application Security Descriptor Configuration Syntax [page 592]
Limits for Technical Artifacts of the SAP Authorization and Trust Management Service [page 3605]

6.1.1.9.5.2 Rotate Binding Secrets

Service instances of the SAP Authorization and Trust Management service use different binding secrets for each binding. To rotate binding secrets, unbind and rebind any consuming applications.

Prerequisites

- You've enabled support for binding secrets.
  For more information, see Migrate from Instance Secrets to Binding Secrets [page 2638].
- If you have to replace any service keys, create the new service keys before proceeding.
  For more information, see Creating Service Keys [page 278].

Procedure

1. Unbind the application that consumes your service instance.

   Use the following syntax:

   `cf unbind-service <APP_NAME> <SERVICE_INSTANCE>`

   ```
   cf unbind-service my-app my-xsuaa-service
   ```

2. Rebind the application that consumes your service instance.

   Use the following syntax:

   `cf bind-service <APP_NAME> <SERVICE_INSTANCE> [-c <PARAMETERS_AS_JSON>]`

   ```
   cf bind-service my-app my-xsuaa-service
   ```

Results

When you rebind the application, the system creates a new binding secret.

Next Steps

Delete any old service keys that you don't need anymore.

Related Information

Service Instance Secrets [page 2634]

6.1.1.9.5.3 Rotate X.509 Secrets

When your private key is exposed or your certificates are about to expire, it's time to rotate secrets. Rotating secrets can be as easy as unbinding and rebinding your application or service key.

Prerequisites

- If you provide your own certificates, you've prepared new certificates.
- If you need to replace any service keys, create the new service keys before proceeding.
  For more information, see Creating Service Keys [page 278].

Context

> **Note**
>
> The service doesn't check for certificate revocation. To stop the service from accepting a certificate that is still valid, delete the relevant bindings or service keys. As soon as the binding is deleted, the service stops accepting the certificate.
>
> For bindings with self-managed certificates and the `certificate-pinning` parameter set to `false`, you can rotate the secrets without deleting bindings. Just use a new certificate with the same subject and issuer distinguished name (DN). The service saves the new validity date of the new certificate.
>
> For more information, see Parameters for Self-Managed X.509 Certificates [page 2643].

Procedure

1. Unbind the application that consumes your service instance.

   Use the following syntax:

   ```
   cf unbind-service <APP_NAME> <SERVICE_INSTANCE>
   ```

   ```
   cf unbind-service my-app my-xsuaa-service
   ```

2. Rebind the application that consumes your service instance.

   Use the following syntax:

   ```
   cf bind-service <APP_NAME> <SERVICE_INSTANCE> [-c <PARAMETERS_AS_JSON>]
   ```

   ```
   cf bind-service my-app my-xsuaa-service -c parameters.json
   ```

   If you provide your own certificates, include them in a `parameters.json` file.

Related Information

Binding Parameters of SAP Authorization and Trust Management Service [page 2640]
Binding Service Instances to Applications [page 276]
Service Instance Secrets [page 2634]

6.1.1.9.5.4 Migrate from Instance Secrets to Binding Secrets

To simplify the management of secrets for service instances of the SAP Authorization and Trust Management service, we recommend that you configure service instances to use binding secrets.

Context

By default, service instances of the SAP Authorization and Trust Management service use the instance secret for all bindings of the service instance. In the application security descriptor (`xs-security.json`), enable binding secrets for service instances. All bindings have their own secret. You can enable both at once for the following plans:

- Application plan

The API access plan only uses binding secrets.

Procedure

1. Modify the application security descriptor (`xs-security.json`) to support both instance secrets and binding secrets.

   Edit the OAuth client configuration of the `xs-security.json` as follows:

   > {·} Sample Code

   ```
   "oauth2-configuration": {
       "credential-types": ["binding-secret","instance-secret"]
   }
   ```

2. Update the service instance with the new application security descriptor.
3. Unbind and rebind any consuming applications.

   With each new binding, the system creates a new binding secret.
4. Replace any service keys with new service keys.

   At this point, none of the applications consuming your service instance need the instance secret anymore.

   > → Remember
   >
   > Before continuing with the next step, verify whether instance secrets are currently in use to ensure a safe transition. Use the REST API endpoint `GET /sap/rest/authorization/v2/apps/{id}` and check the response parameter `instanceSecretLastUsedAt`.
   >
   > Why this matters: Removing the `instance-secret` credential type from your application's security descriptor invalidates all existing instance secret bindings and service keys.

5. Modify the application security descriptor (`xs-security.json`) to disable support for instance secrets.

   Edit the OAuth client configuration of the `xs-security.json` as follows:

   > {·} Sample Code

   ```
   "oauth2-configuration": {
       "credential-types": ["binding-secret"]
   }
   ```

6. Update the service instance with the new application security descriptor.

Related Information

Update a Service Instance [page 577]
Creating Service Keys [page 278]
Service Instance Secrets [page 2634]

6.1.1.9.5.5 Rotating Instance Secrets

When configured for instance secrets, a service instance of the SAP Authorization and Trust Management service uses the same instance secret for all bindings. You can't really rotate instance secrets, but must rotate the applications and service instance together.

> → Recommendation
>
> Migrate to using binding secrets instead of instance secrets.
>
> Using instance secrets forces you to redeploy the entire application under a new `xsappname` and repeat the assignment of any role collections.
>
> For more information, see Migrate from Instance Secrets to Binding Secrets [page 2638].

Related Information

Service Instance Secrets [page 2634]

6.1.1.9.5.6 Binding Parameters of SAP Authorization and Trust Management Service

When binding applications or creating service keys for services instances of the SAP Authorization and Trust Management service (XSUAA), provide configuration parameters in JSON format.

Set parameters according to the following use cases:

- For instance or binding secrets, you want to supress the creation of client secrets.
- For X.509 secrets, set parameters according to the following scenarios:
  - The SAP Authorization and Trust Management service generates certificates for you.
  - You already have your own public key infrastructure (PKI), with certificates issued from one of the supported certificate authorities (CA). See the links in the related information.

> → Remember
>
> The types of secrets that a service instance can accept in a binding or service key is configured in the application security descriptor (`xs-security.json`).
>
> For more information, see Application Security Descriptor Configuration Syntax [page 592].

Related Information

Parameters for Suppressing Client Secrets [page 2641]

Parameters for X.509 Certificates Managed by SAP Authorization and Trust Management Service [page 2642]
Parameters for Self-Managed X.509 Certificates [page 2643]
Trusted Authorities for X.509 Certificates [page 2646]
Limits for Technical Artifacts of the SAP Authorization and Trust Management Service [page 3605]

6.1.1.9.5.6.1 Parameters for Suppressing Client Secrets

When binding or creating a service key for an `xsuaa` service instance, you can pass some parameters in JSON format or in a JSON file (see **`cf bind-service`** and **`cf create-service-key`** in the related links). The `"hide-secret"` element enables you to suppress the client secret when binding or creating a service key. It's useful if some applications only want to bind the `xsuaa` service for authorization purposes.

Prerequisites

- In the application security descriptor file (`xs-security.json`), you've included the "credential-type" key with `"binding-secret"` when you created or updated the service instance. The `"hide-secret"` element is only taken into account if the credential type is `"binding-secret"`.

> ⓘ Note
>
> If the `"credential-type"` element isn't defined in the binding descriptor, it defaults to the first value of the array passed as `"credential-types"` in `xs-security.json` for the service instance or for the first value of the default list.

For more information, see Application Security Descriptor Configuration Syntax [page 592].

Parameters for Suppressing Client Secrets

The **`cf create-binding`** or **`cf create-service-key`** commands expect a valid `parameters.json` file containing the service-specific configuration parameters in the following format.

> {·} Sample Code

```
{
  "credential-type": "binding-secret",
  "hide-secret": "true"
}
```

The `hide-secret` element defines whether the client secret is omitted.

Element for Suppressing Client Secrets

Element	Value	Description
hide-secret	`true`	The client secret is omitted in the binding information.
	`false` (default)	The client secret is included in the binding information.

Related Information

Cloud Foundry CLI Reference

6.1.1.9.5.6.2 Parameters for X.509 Certificates Managed by SAP Authorization and Trust Management Service

Use the parameters to have the service generate X.509 certificates for you.

Prerequisites

You've enabled the service instance to use X.509 secrets.

For more information, see Application Security Descriptor Configuration Syntax [page 592].

Parameters for Certificates Managed by the Service

The *create-binding* or *create-service-key* commands expect a `parameters.json` in the following format.

```
{
  "credential-type": "x509",
  "x509": {
    "key-length": 2048,
    "validity": 8,
    "validity-type": "DAYS"
  }
}
```

The following table describes the supported parameters.

Parameters for Service-Managed Certificates

Parameter	Description
`credential-type`	Only required to use X.509 when X.509 isn't specified as default in the application security descriptor (`xs-security.json`). For more information, see Application Security Descriptor Configuration Syntax [page 592].
`x509`	In this parameter, you define the certificate options of key length, validity period, and unit of length. This parameter is optional.
`key-length`	Specifies the byte length of the generated private key. The default length is 2048 bytes. You can also choose 4096 bytes or 8192 bytes.
`validity`	Specifies the number of time units for `validity-type`. The default value is 7. Together with the `validity-type` the range of validity runs from `1 DAYS` to `30 DAYS`.
`validity-type`	Specifies the time unit for validity. Supported value is **DAYS**. The default value is **DAYS**.

Related Information

Service Instance Secrets [page 2634]

6.1.1.9.5.6.3 Parameters for Self-Managed X.509 Certificates

Use these parameters to provide your own certificates for a binding or service key to service instances of the SAP Authorization and Trust Management service (XSUAA).

Prerequisites

- You've enabled the service instance to use X.509 secrets.
  For more information, see Application Security Descriptor Configuration Syntax [page 592].
- To use your own public key infrastructure (PKI), the certificates must be issued from one of the following certificate authorities (CA).
  For more information, see Trusted Authorities for X.509 Certificates [page 2646].

Parameters for Self-Managed Certificates

The create binding or create service key command expects a `parameters.json` in the following format.

```
{
  "credential-type": "x509",
  "x509": {
    "certificate": "-----BEGIN CERTIFICATE-----...-----END CERTIFICATE-----",
    "ensure-uniqueness": false,
    "certificate-pinning": true,
    "hide-certificate": true
  }
}
```

Parameters for Self-Managed Certificates

Parameter	Description
`credential-type`	Only required to use X.509 when X.509 isn't specified as default in the application security descriptor (`xs-security.json`). For more information, see Application Security Descriptor Configuration Syntax [page 592].
`x509`	In this parameter, you include the certificate and the options of uniqueness and pinning. This parameter is required.
`certificate`	Provide the certificate string in privacy-enhanced mail (PEM) format. This parameter is required. **⚠ Restriction** You can only use each certificate once per service instance. The uniqueness of the certificate among all bindings and keys of a service instance is dependent on the value of `certificate-pinning`: • `true` The certificate hash must be unique. • `false` The combination of subject and issuer DN must be unique.
`ensure-uniqueness`	Ensures that the certificate is unique among all following clone instances. The default value is `false`. **ⓘ Note** This parameter only applies to the broker plan of the service.

Parameter	Description
`certificate-pinning`	Set to `false` for applications that are rarely updated or deployed. The incoming certificate's subject and issuer DN is compared to the subject and issuer of the stored certificate. If the values match and the certificate was issued on or after the stored issuer date, the authentication is accepted. Afterwards the incoming certificate's issuer date is stored for future authentication attempts. **Exception:** Matching certificates with a `Valid from` parameter that is less than 10 minutes old are always accepted. Set to `true` for blue and green deployments, where you deploy a new application or version of an application regularly and rotate the certificate in parallel. For this use case, you need a new certificate to use in the binding of the new application. The default value is `true`.
`hide-certificate`	If this parameter is set to `true` (default is `false`), the X.509 certificate is not returned in the binding or service key. This is useful to reduce the size of the environments when the Cloud Foundry environment limit of 130k has been reached. This parameter is optional. **Note** This parameter works only with self-managed certificates as otherwise, the public key would not be known to the user.

Related Information

Service Instance Secrets [page 2634]

6.1.1.9.5.6.4 Trusted Authorities for X.509 Certificates

Service instances of the SAP Authorization and Trust Management service (XSUAA) trust the following certificate authorities (CA). To use your own public key infrastructure (PKI) for bindings, the certificates must be issued from one of these CAs.

Supported CAs for X.509 Certificates

Subject	Issuer	Validity
`C=US, O=DigiCert Inc, OU=www.digicert.com, CN=DigiCert Global Root G2`	`C=US, O=DigiCert Inc, OU=www.digicert.com, CN=DigiCert Global Root G2`	Jan 15 12:00:00 2038 GMT
`C=DE, L=Walldorf, O=SAP AG, CN=SAP Global Root CA`	`C=DE, L=Walldorf, O=SAP AG, CN=SAP Global Root CA`	Apr 26 15:46:27 2032 GMT
`C=US, O=DigiCert Inc, OU=www.digicert.com, CN=DigiCert Global Root CA`	`C=US, O=DigiCert Inc, OU=www.digicert.com, CN=DigiCert Global Root CA`	Nov 10 00:00:00 2031 GMT
`C=US, O=DigiCert Inc, OU=www.digicert.com, CN=DigiCert SHA2 Secure Server CA`	`C=US, O=DigiCert Inc, OU=www.digicert.com, CN=DigiCert Global Root CA`	Sep 22 23:59:59 2030 GMT
`C=US, O=Entrust, Inc., OU=www.entrust.net/legal-terms, CN=Entrust Root Certification Authority - G2`	`C=US, O=Entrust, Inc., OU=www.entrust.net/legal-terms, CN=Entrust Root Certification Authority - G2`	Dec 07 17:55:54 2030 GMT

Related Information

Parameters for Self-Managed X.509 Certificates [page 2643]
Service Instance Secrets [page 2634]

6.1.1.9.5.7 Rotate Signing Keys of Access Tokens

Components of SAP BTP use the digital signature of the access tokens to verify the validity of access tokens. Periodically rotate the signing keys of access tokens, either automatically or manually (see Security Recommendations for SAP Authorization and Trust Management Service [page 3599]).

Automatic Signing Key Rotation

We recommend that you use the automatic signing key rotation for access tokens. Before you activate it, make sure that your services and applications can handle automatic rotation of signing keys.

> ⚠ Restriction
>
> Automatic signing key rotation isn't possible with SAP HANA Cloud with principal propagation and Cloud connector lower than version 2.19.0 (see Set Up Trust).
>
> For subaccounts created after January 19, 2026, automatic signing key rotation is enabled by default. If you use SAP HANA Cloud with principal propagation and Cloud connector, you must disable automatic signing key rotation.

If the automatic signing key rotation is enabled, SAP BTP rotates the signing key periodically. The first signing key rotation is initiated immediately, and the new key becomes active 48 hours after the enablement. Use the SAP BTP cockpit or the SAP BTP command line interface (BTP CLI).

- Enable the automatic signing key rotation in the SAP BTP cockpit:
  Choose your subaccount and go to the *Signing Keys* tab in *Security* > *Settings*. Choose *Enable*.
- Enable the automatic signing key rotation using the following command of the SAP BTP command line interface (BTP CLI):
  `btp update security/settings --rotate-signing-key-automatically TRUE` (Boolean)

> → Recommendation
>
> We recommend to enable automatic rotation first in a non-productive environment and test all applications and services once the first new key becomes active. This occurs within 48 hours.
>
> If issues arise, you can disable the automatic signing key rotation.

> ⓘ Note
>
> You may also initiate the signing key rotation manually, for example, if needed for testing. Note that the rotation is completed within 48 hours. Use the following command of the SAP BTP command line interface (BTP CLI):
>
> ```
> btp rotate security/token-key
> ```

Manual Rotation of Signing Keys

You can also manually rotate the signing keys. You can create a new signing key and enable it after a grace period. The latter is required so that signing key caches (for example, in libraries) can retrieve the new keys. Keep in mind that only two signing keys are allowed at the same time.

- To rotate the signing keys directly, use the following command of the SAP BTP command line interface (BTP CLI). The signing key is rotated after 48 hours.
  `btp rotate security/token-key --key ID`

For more information on how to manually manage the rotation using the SAP BTP cockpit or the SAP BTP command line interface (BTP CLI), see the related links.

Related Information

Manually Rotate Signing Keys of Access Tokens with the BTP CLI [page 2648]
Manually Rotate Signing Keys of Access Tokens with the Security Setting API [page 2650]

6.1.1.9.5.7.1 Manually Rotate Signing Keys of Access Tokens with the BTP CLI

The BTP CLI enables you to manually rotate the signing keys of access tokens.

Prerequisites

Plan for waiting time in hours between the steps of this procedure.

After activating a new key, wait for access tokens signed by the old key to expire before deleting the old key. The default lifetime of access tokens is 12 hours.

To support this delay in rotation, the service enforces a minimum 12-hour delay between rotation and the activation.

> **Note**
>
> In emergency cases or in automated test setups, you can bypass the delay in activation and deletion by using the `--force` option. Forcing changes can also cause service interruption for access tokens that haven't expired yet.

Context

To rotate the signing key of access tokens, use the SAP BTP command line interface (BTP CLI). You can rotate signing keys for the global account, directory, or subaccount.

For more information, see Managing Signing Keys for Access Tokens [page 2712].

Procedure

1. Log on to the BTP CLI.

   For more information, see Log in [page 2679].

2. Check that there's space for a new signing key.

   You can store two signing keys per subaccount.

   Use the following command:

   ```
   btp list security/token-key
   ```

   The service lists any keys.

   ```
   Key ID                      Enabled   Created                      Fips
   Compliant
   default-jwt-key-491aae9c42            Fri Feb 02 14:48:51 UTC 2024 yes
   default-jwt-key-ca81ebbb43 yes        Mon Feb 05 14:42:26 UTC 2024 yes
   2 entries
   OK
   ```

   Delete the inactive key if you already have two. In the previous example, delete `default-jwt-key-491aae9c42`. For more information on how to delete keys, see step 6.

   > ⚠ Caution
   >
   > If the `keyID` is `key-id-0` or `key-id-1`, these keys are legacy signing keys. When you enable the first signing key after a legacy signing key, the legacy signing key is immediately invalid. Clients with access tokens issued within the lifetime of an access token can't use their access tokens anymore. Affected clients must reauthenticate to get a new token.
   >
   > In addition, the issuer of the key changes as well. Update the configuration of systems depending on these keys, such as SAP HANA systems and cloud connector connections. For more information, see:
   >
   > - Managed Certificate Collections in the *SAP HANA Cloud, SAP HANA Database Security Guide*
   > - SAP BTP Connectivity:Configure Trusted Entities in the Cloud Connector
   >
   > Plan for possible service interruption when you add your first signing key.

3. Add a new signing key for the access token.

   Use the following command:

   ```
   btp create security/token-key
   Created default signing key
   ```

   A new signing key is generated.

4. Enable the new signing key.

   Use the following command:

   ```
   btp enable security/token-key --key <Key_ID>
   ```

   For example:

   ```
   btp enable security/token-key --key default-jwt-key-ca81ebbb43
   Enabled signing key 'default-jwt-key-ca81ebbb43'
   ```

New access tokens are now signed with the new signing key.

5. To verify that the new signing key is enabled, use the following command:

   ```
   btp list security/token-key
   ```

6. Wait for access tokens signed by the old key to expire.

   As long as the old signing key exists, the system still accepts digital signatures signed by that key. Once you've waited out the lifetime of any access tokens signed by the old key, you can delete the old key.

7. Delete the old signing key for the access token.

   Use the following command:

   ```
   btp delete security/token-key --key <Key_ID>
   ```

   For example:

   ```
   btp delete security/token-key --key default-jwt-key-491aae9c42
   Do you really want to delete signing key 'default-jwt-key-491aae9c42'? [no]>
   Yes
   Deleted signing key 'default-jwt-key-491aae9c42'
   ```

6.1.1.9.5.7.2 Manually Rotate Signing Keys of Access Tokens with the Security Setting API

You can use the Security Setting API of the SAP Authorization and Trust Management service to manually rotate the signing keys of access tokens.

Prerequisites

- You've enabled API access to the service.
  For more information, see Get Access to the APIs [page 2774].
- Plan for waiting time in hours between the steps of this procedure.
  After activating a new key, wait for access tokens signed by the old key to expire before deleting the old key. The default lifetime of access tokens is 12 hours.
  To support this delay in rotation, the service enforces a minimum 12-hour delay between rotation and the activation (`UPDATE`) or deletion (`DELETE`) of signing keys.

> **Note**
>
> In emergency cases or in automated test setups, you can bypass the delay in activation and deletion by using the `FORCE_UPDATE` and `FORCE_DELETE` change modes. Forcing changes can also cause service interruption for access tokens that haven't expired yet.

Context

Procedure

1. Check that there's space for a new signing key.

   You can store two signing keys per subaccount. Get the settings of your subaccount to see how many keys are stored there.

   Call the *GET* method of the Security Settings API at the following endpoint:

   `https://api.authentication.<region>.hana.ondemand.com/sap/rest/authorization/v2/securitySettings`

   For more information, check the security settings API on the SAP Business Accelerator Hub.

   The API returns a JSON with the security settings. Examine the key IDs in the token policy settings.

   ```
   {
       …
       "tokenPolicySettings": {
           …
           "activeKeyId": "jwt-sig-2022-12-01",
           "keyIds": [
               "jwt-sig-2022-09-10",
               "jwt-sig-2022-12-01"
           ]
       },
   …
   }
   ```

   Delete the inactive key if you already have two. In the previous example, delete `jwt-sig-2022-09-10`. For more information on how to delete keys, see step 5.

   > ⚠ Caution
   >
   > If the `keyID` is `key-id-0` or `key-id-1`, these keys are legacy signing keys. When you enable the first signing key after a legacy signing key, the legacy signing key is immediately invalid. Clients with access tokens issued within the lifetime of an access token, can't use their access tokens anymore. Affected clients must reauthenticate to get a new token.
   >
   > In addition, the issuer of the key changes as well. Update the configuration of systems depending on these keys, such as SAP HANA systems and cloud connector connections. For more information, see:
   >
   > - Managed Certificate Collections in the *SAP HANA Cloud, SAP HANA Database Security Guide*
   > - Configure a CA Certificate in the *SAP BTP Connectivity* documentation
   >
   > Plan for possible service interruption when you add your first signing key.

2. Add a new signing key for the access token.

   Call the *PATCH* method of the same endpoint.

   In the body, include a key ID and set `changeMode` to **`ADD`** in the token policy settings.

Sample Code

```
{
    "tokenPolicySettings": {
        "keyId": "jwt-sig-2023-01-30",
        "changeMode":"ADD"
    }
}
```

A new signing key is generated.

3. Enable the new signing key.

   Call the *PATCH* method of the same endpoint, but set `changeMode` to **UPDATE**.

```
{
    "tokenPolicySettings": {
        "keyId": "jwt-sig-2023-01-30",
        "changeMode":"UPDATE"
    }
}
```

   New access tokens are now signed with the new signing key.

4. Wait for access tokens signed by the old key to expire.

   As long as the old signing key exists, the system still accepts digital signatures signed by that key. Once you've waited out the lifetime of any access tokens signed by the old key, you can delete the old key.

5. Delete the old signing key for the access token.

   Call the *PATCH* method of the same endpoint, but set `keyID` to the old key ID and `changeMode` to **DELETE**.

Sample Code

```
{
    "tokenPolicySettings": {
        "keyId": "jwt-sig-2022-12-01",
        "changeMode":"DELETE"
    }
}
```

Related Information

Managing Secrets of the SAP Authorization and Trust Management Service [page 2634]

6.1.1.9.5.8 Rotate Signing Keys of SAML Token

The SAP Authorization and Trust Management service uses an X.509 certificate to sign SAML 2.0 protocol messages between itself, a SAML service provider, and an SAML identity provider. To rotate the signing keys for SAML tokens, use the Security Settings API of the SAP Authorization and Trust Management service and then establish trust by exchanging metadata between the service provider and identity provider.

Prerequisites

- You've enabled API access to the SAP Authorization and Trust Management service.
  For more information, see Get Access to the APIs [page 2774].
- Optionally, to use your own signing key for SAML tokens, you've obtained an X.509 certificate for the SAP Authorization and Trust Management service.
- You've checked that you've space for an additional signing key.
  You can store two signing keys per subaccount. Get the settings of your subaccount to see how many signing keys you've stored there. Delete an inactive signing key if you already have two.

Context

The signed SAML tokens are used to validate and authenticate the protocol messages between the SAP Authorization and Trust Management service instance and the identity provider.

Procedure

1. Add a new signing key for SAML tokens.

   Call the *PATCH* method of the Security Settings API at the following endpoint:

   `https://api.authentication.<region>.hana.ondemand.com/sap/rest/authorization/v2/securitySettings`

   In the body, include a key ID and set `changeMode` to **ADD** in the SAML configuration settings.

   {·} Sample Code

   ```
   {
       "samlConfigSettings": {
           "keyId": "my-new-key",
           "changeMode":"ADD"
       }
   }
   ```

   A new signing key is generated.

   Optionally, provide your own signing key with the `key` parameter. The signing key is your private key of the certificate key pair. Provide the certificate.

   → Recommendation

   We recommend providing an empty passphrase. We believe that there's no reason to view the private key.

   ```
   {
       "samlConfigSettings": {
           "keyId": "my-new-key",
   ```

```
        "changeMode":"ADD",
        "key": {
               "key": "<key_data>",
               "passphrase": "",
               "certificate": "<certificate_data>"
        }
    }
}
```

2. Enable the new signing key.

   Call the *PATCH* method of the same endpoint, but set `changeMode` to **UPDATE**.

```
{
    "samlConfigSettings": {
        "keyId": "my-new-key",
        "changeMode":"UPDATE"
    }
}
```

   New SAML tokens are now signed with the new signing key.

3. Establish trust with the identity providers you consume.

   Upload the new metadata information to your identity providers.

   For more information, see the related documentation:

   - Manually Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2547]
   - Establish Trust and Federation with UAA Using Any SAML Identity Provider [page 2551]

> ⚠ Caution
>
> Until you upload the metadata to your identity provider, the identity provider rejects the signed SAML tokens.

4. Delete the old signing key for SAML tokens.

   Call the *PATCH* method of the same endpoint, but set `keyID` to the old key ID and `changeMode` to **DELETE**.

> {·} Sample Code

```
{
    "samlConfigSettings": {
        "keyId": "my-old-key",
        "changeMode":"DELETE"
    }
}
```

Related Information

Managing Secrets of the SAP Authorization and Trust Management Service [page 2634]

6.1.1.9.6 Configure Trusted Domains for Multi-Environment Subaccounts

By default, applications of the subaccount, including login pages of the SAP Authorization and Trust Management service (XSUAA), can't be framed by other applications in different domains for security reasons.

Prerequisites

- → Recommendation

  Review the security implications of using inline frames (IFrames) with the SAP Authorization and Trust Management service.

  For more information, see Implications of Using IFrames [page 3592].
- Check that your application supports the trusted domains of multi-environment subaccounts.
  The documentation of your application indicates whether you need to make this configuration or not.
  The login pages of the SAP Authorization and Trust Management service (XSUAA) supports this feature.

Context

To prevent clickjacking or overlay attacks, web browsers follow a same origin policy. The same origin policy means you can only embedded content in your web page if the content shares the same protocol, host, and port as your web page. If the origins don't match, you must maintain a list of origins that allows for exceptions to the same origin policy. The same origin policy also applies to cookies. Cookies enable single sign-on scenarios. If the cookies have a different origin, the browser rejects them like any other content.

→ Recommendation

Instead of configuring lists of trusted domains, ensure that your application runs under the same domain as the framing application.

For more information about the SAP Custom Domain service, see the documentation of the SAP Custom Domain Service.

If your custom identity provider runs under a different domain as the framing application, you must configure the identity provider to trust the domain of the framing application, too.

Procedure

1. Go to your subaccount (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Settings*.
2. Choose the *Trusted Domains* tab.

3. Choose *Add*.
4. Enter the host name for the trusted domain.

   For example: `https://store.example.com`

   Wildcards are allowed for the subdomain.

   For example: `https://*.example.com`

   > ⓘ Note
   >
   > All the trusted domains together, including their space separators, can't exceed 2048 characters.

5. Save your entries.

Next Steps

Ensure that other components framed by the host application also trust the framing domain. Typical components include the application router and the identity provider.

Related Information

Application Router [page 453]
Configure Trust Domains (in SAP Cloud Identity Services - Identity Authentication Documentation)

6.1.1.9.7 Configure Token Policy for SAP Authorization and Trust Management Service

Set the token policy in the SAP BTP cockpit for SAP Authorization and Trust Management service by configuring the validity of the OpenID Connect (OIDC) tokens the service issues.

Prerequisites

> → Recommendation
>
> Review the security implications of token policies for SAP Authorization and Trust Management service.
>
> For more information, see Setting Token Policy [page 3594].

Context

On request, SAP Authorization and Trust Management service issues access and refresh tokens.

With a valid access token, you can access a protected resource. Once an access token expires, you can get new access tokens with a refresh token. Once the refresh token expires, you must reauthenticate and request new access and refresh tokens.

> → Recommendation
>
> Relaxing the token policy means that users reauthenticate less. However, increasing the token validity also means that if a malicious user manages to steal a token, that malicious user has access until the token expires. Keep token validity as short as possible, but not less than 30 minutes.

To change the token validity, access the security settings. The change applies to all service instances in the subaccount that haven't set a specific value in the application security descriptor (`xs-security.json`).

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account and subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* > *Settings*.
4. Choose the *Token Validity* tab.

   You see the setting of the current access token validity and of the current refresh token validity.
5. Use the slider to set the validity for each token. The tooltip shows the selected value in seconds.

   > Example
   >
   > - Set the access token validity to 1800 seconds
   > - Set the refresh token validity to 43200 seconds.

6. To save the value, choose *Set Token Validity*.

   > Note
   >
   > It is also possible to change the token validity in the Security Settings API.
   >
   > For more information, see Security Settings API on SAP Business Accelerator Hub.

Related Information

Application Security Descriptor Configuration Syntax [page 592]

6.1.2 Account Administration Using Joule in the Cockpit

Learn about the administrative tasks that SAP's generative AI copilot, Joule, can assist you with SAP BTP cockpit.

> **Note**
>
> A Global Account Administrator must enable Joule in the Global Account settings by accepting the legal Terms and Conditions for Generative AI. Only then will the Joule icon appear in the cockpit tool header. Check out Enable Joule in the Cockpit to learn more.

> **Note**
>
> This version of Joule is provided free of charge in non-trial BTP Global Accounts and is limited to supporting BTP administration capabilities only within SAP BTP cockpit.

Joule makes it easy to quickly access information about your accounts and users in the cockpit.

- Ask Joule to perform an administrative action for you from a set of pre-defined capabilities.
- Ask Joule a question about SAP BTP services or the cockpit. Joule references SAP BTP documentation and provides a summary of what it finds, as well as links to relevant help content.

Want to see Joule in action? Check out this short demo video for a brief overview of how to work with Joule.

New features and capabilities for Joule in the SAP BTP Cockpit are announced on the What's New for SAP Business Technology Platform page on SAP Help Portal (filtered for **Joule**). For more information about what's in store for Joule, see the SAP Road Map Explorer.

Enable Joule in the Cockpit [page 2659]
: Enable Joule in SAP BTP cockpit to access Generative AI capabilities.

Access Joule in the Cockpit [page 2660]
: Launch Joule directly from the SAP BTP cockpit.

Supported Administrative Tasks with Joule in the Cockpit [page 2661]
: Learn about the administrative tasks that Joule can perform for you in the SAP BTP cockpit.

Best Practices for Joule in the Cockpit [page 2662]
: Get the most our of Joule in SAP BTP Cockpit.

Troubleshooting Joule in the Cockpit [page 2663]
: When you work with Joule, you could have questions about the things you see in the chat conversation. A list of common questions and answers are provided here.

Related Information

What is Joule?

Using Joule

6.1.2.1 Enable Joule in the Cockpit

Enable Joule in SAP BTP cockpit to access Generative AI capabilities.

Prerequisites

- You have the Administrator role for the global account for which you'd like to enable Joule in the cockpit.
- Joule is available for your global account (P-/S-users). Make sure you are accessing the cockpit from the SAP default identity provider.

> **Note**
>
> This version of Joule is provided free of charge in non-trial BTP Global Accounts and is limited to supporting BTP administration capabilities only within SAP BTP cockpit.

Procedure

1. Open the global account for which you'd like to enable Joule in SAP BTP cockpit.
2. Navigate to the global account settings by selecting *(Global Account Settings)*.
3. In the global settings, select **Joule**.
4. Accept the Terms and Conditions for Generative AI and toggle the button **Enable Joule**.
5. Save your changes.

Results

After a global account administrator has enabled Joule for the global account, the Joule diamond icon will become visible in the cockpit tool header for all global account users.

Related Information

Access Joule in the Cockpit
Supported Browsers

6.1.2.2 Access Joule in the Cockpit

Launch Joule directly from the SAP BTP cockpit.

Joule is accessed directly from the cockpit when you click the diamond icon in the cockpit toolbar. When the chat window opens, you can ask Joule to perform one of the supported capabilities or you can ask Joule a question about SAP BTP.

Authentication

Joule in SAP BTP cockpit only supports SAP-managed IAS tenants (P-/S-users) for authentication at this time. Ensure that you are accessing the cockpit using the SAP default identity provider.

Authorization

Because you're already logged into SAP BTP cockpit, Joule recognizes you and can perform administrative actions on your behalf. It's important to know that Joule is bound to your cockpit authorization. In other words, Joule can only accomplish tasks for you that you're also authorized to perform.

Language Settings

Joule in SAP BTP cockpit is available in 8 languages: English, Chinese, Japanese, Korean, German, Portuguese, French, and Spanish.

Joule automatically adopts the language settings in the cockpit. To change the language, navigate to your profile settings by clicking on your avatar and selecting *(Settings)* in the user menu. Select the *Language* tab and choose the language from the drop-down menu. Select *Save*.

When you return to Joule, it will first appear in the previous language. Joule will switch to your new language when you enter a message or select *New Conversation*.

Related Information

Supported Browsers
Supported Administrative Tasks with Joule in the Cockpit [page 2661]
Best Practices for Joule in the Cockpit [page 2662]
Using Joule

6.1.2.3 Supported Administrative Tasks with Joule in the Cockpit

Learn about the administrative tasks that Joule can perform for you in the SAP BTP cockpit.

You can ask Joule to assist you with administrative scenarios related to accounts and users. If Joule recognizes the task as a supported capability, it walks you through the steps to complete the task.

> **Note**
>
> Joule doesn't support administrative actions in the Neo environment.

You can ask Joule what it can do for you, either by typing the question directly or selecting the quick start button.

Joule can help with **user management**:

- Add a new user to a subaccount
- Get details for global account or subaccount users
- View users and their permissions
- List role collections for your global account or subaccount
- Assist you in exporting subaccount users by guiding you to the subaccount users page

Joule can provide you with **information about your global accounts or subaccounts**:

- List your global accounts or subaccounts and see details
- See available regions in a global account
- Check for services entitlements
- View information about consumption costs in your global account

Joule in the BTP cockpit can also assist you with administrative tasks related to **Cloud Foundry**:

- View available Cloud Foundry environments
- List Cloud Foundry instances, spaces, or applications
- See the Cloud Foundry org or space users
- Add a user to a Cloud Foundry org or space

Joule is also capable of helping you to find the root causes of runtime issues with your applications hosted by the **Application Frontend** service and suggest ways to resolve there errors. See Analyzing Application Issues with Joule.

Joule can also **navigate** you to select pages within the cockpit:

- Service entitlements in your global account or a subaccount
- Trust configuration for your global account or a subaccount
- Cloud Foundry spaces
- Applications, Events, Service Instances, Service Marketplace, Security Groups, or Routes in a Cloud Foundry space
- HANA Cloud instance in a subaccount

Related Information

Best Practices for Joule in the Cockpit [page 2662]
Troubleshooting Joule in the Cockpit

6.1.2.4 Best Practices for Joule in the Cockpit

Get the most our of Joule in SAP BTP Cockpit.

Joule is your AI copilot in SAP BTP Cockpit. You can ask questions using natural language and it will find information or perform tasks for you.

> **Note**
>
> Joule is an addition to in-app help content, guided tours, and embedded learning content.

New to Joule? Here are some tips to help you get started:

- To see **what Joule can do for you**, just ask! For example, you can type, What can you do for me?
- You can **start a new the conversation** at any time by selecting the *New Conversation* button in the Joule menu.
- You can **hide the Joule chat window** by selecting the X and pick up right where you left off by selecting the Joule icon in the cockpit tool header.
- **Conversations expire after 8 hours**. When this happens, Joule will notify you and give you the option to start a new conversation.
- You may find that **selecting a suggested action or option** will provide better results than typing.
- Want to give feedback about your experience with Joule in the BTP cockpit? Simply type or select the quick start button *give feedback*.

Best practices when writing prompts

Joule can help you with a number of different scenarios, from finding information to completing certain tasks. The responses depend strongly on how you write your prompt.

When you want Joule to **perform a task**, it's best to treat your prompt like a command. Using verbs such as "add", "show" or "go to" will help Joule understand you want it to perform an action. Adding keywords like "subaccount" or "space" can also influence how Joule interprets the task.

When you need to **find information** about something, write your prompt as an open question. Inquiries like "How to create a CF space?" and "Where can I find instructions for entitlements?" will help Joule look for document-based answers. In this case, avoid using too many keywords in the same prompt that might cause Joule to misinterpret your request.

Joule also responds best to **single requests**. For example, if you want to see the users in your global account and in a specific subaccount, send these requests as two separate queries.

Finally, giving additional **context** may also help Joule provide a better answer. For example, if you want to see users of a specific subaccount, Joule may ask for the subaccount ID or name.

Quick reference guide:

Your goal	Prompt suggestions	Joule's response
Perform an action	Use a command: "Add", "Show me", "Go to", etc.	Executes the action
Learn an information	Ask an open question: "How to...", "Where is...", etc.	Provides instructions or explanations
Ask multiple requests	Write separate prompts for each request	Gives a more accurate response to each prompt and minimizes misinterpretation

Related Information

Supported Administrative Tasks with Joule in the Cockpit [page 2661]
Troubleshooting Joule in the Cockpit

6.1.2.5 Troubleshooting Joule in the Cockpit

When you work with Joule, you could have questions about the things you see in the chat conversation. A list of common questions and answers are provided here.

I don't see my subaccount in the subaccount list

- Check to make sure any newly created subaccounts were successfully created. If the create action failed, you'll see a red tag on the subaccount that says *Creation Failed*.

I'm unable to add users to a subaccount

- Make sure you are asking Joule to add one new user at a time to the subaccount.

I'm seeing an error message

- Try restarting the conversation with Joule by selecting the *New Conversation* button in the Joule menu.
- Rephrasing your query may help Joule better understand what you want it to do.

Joule has stopped responding

- If Joule does not respond to your prompt, try asking again or asking "are you still there?"
- Try restarting the conversation with Joule by selecting the *New Conversation* button in the Joule menu.

I'm getting an Identity Provider error message

- If you're seeing an error message about the Identity Provider configuration, try enabling third-party cookies in your browser.

Joule is in the wrong language

- If you recently changed the language settings in the cockpit user menu and Joule is still in the previously set language, simply write something to Joule or select *New Conversation* to update the language in Joule.

- To learn more about Joule and languages, check out Joule Multi Language Support.

Get Support

If you need more help, see Getting Support. Please use ticket component **BC-CP-CPT-AI** when contacting support. It may be helpful to include screen shots of your interactions with Joule in the cockpit.

6.1.3 Account Administration Using the SAP BTP Command Line Interface (btp CLI)

Use the SAP BTP command line interface (BTP CLI) for all account administration tasks, such as creating or updating subaccounts, authorization management, and working with service brokers and platforms. It is an alternative to the SAP BTP cockpit for users who like to work in a terminal or want to automate operations using scripts.

The BTP CLI offers convenient features for interactive use as well as for scripting: For the interactive user, there's login via SSO, command autocompletion, as well as several interactive commands with prompts. But all commands can also be run with parameters, and the command output can be set to json format, so the BTP CLI is perfect for scripting.

If you want to use the BTP CLI for scripting, we recommend to use the `--format json` option or to persistently set the output format to json with `btp set config --format json`. See Change the Output Format to JSON [page 2694] and Change Configuration Settings [page 2691].

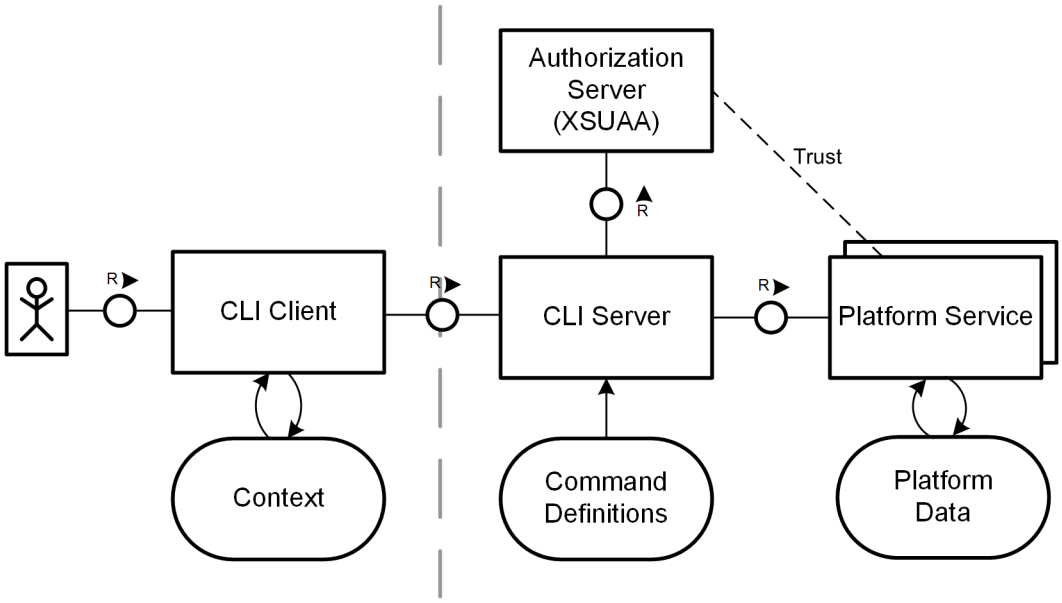
Documentation

In this documentation, you learn how to download, install, update, and use the btp CLI. Additionally, see the following:

- BTP CLI Command Reference: The command reference contains descriptions of all commands with their parameters, as well as examples, tips, and links to further conceptual documentation.
- Get Started with the SAP BTP Command Line Interface (BTP CLI): This tutorial walks you through downloading and installing the BTP CLI client, understanding the syntax, and running your first commands.
- Setting Up a Global Account via the Command Line [page 180]: This chapter lists the commands of the BTP CLI and the cf CLI that you need to create a subaccount and enable Cloud Foundry.
- What's New for SAP BTP Command Line Interface: Release notes for the BTP CLI.

When working with the BTP CLI in your terminal, you can access the help by executing `btp help` or by appending `--help` to a command.

How the BTP CLI Works



You download the BTP CLI client to your local desktop and access it through the shell of your operating system. The client then accesses all required platform services through its backend, the CLI server, where the command definitions are stored. The CLI server delegates authentication and authorization to the authorization server, and forwards trust to the platform services, which then take care of authorization at the execution of each command.

Related Information

BTP CLI Command Reference

6.1.3.1 Download and Start Using the BTP CLI Client

To use the SAP BTP command line interface (BTP CLI), you need to download the client first.

Context

Each client version is supported for at least one year. Most of the updates to the BTP CLI don't require a new client installation, but are made available through the BTP CLI server so that you can use them in your installed version of the client. Using the latest version of the client ensures that you don't miss out on any new features or security updates. If you already have the client installed, you can see which version you have by simply entering `btp`.

To stay informed about features, client releases, as well as client deprecations and discontinuations, check out the What's New for SAP Business Technology Platform page on SAP Help Portal (filtered for *SAP BTP Command Line Interface*).

The client is available for 64-bit versions of the following operating systems:

- Microsoft Windows (amd64)
- Apple macOS (amd64 and arm64)
- Linux (amd64 and arm64)

There are various ways of downloading the client for each operating system, as explained below.

Related Information

How to Work with the btp CLI [page 2676]
Command Syntax of the btp CLI [page 2671]
Set a Target for Subsequent Commands with btp target [page 2688]
BTP CLI Command Reference

6.1.3.1.1 Get the BTP CLI Client for Microsoft Windows (amd64)

Context

Procedure

1. To get the BTP CLI client, you can either run the installation script or download the client manually.
   - Option 1: **Installation Script (Windows PowerShell)**
     When you run the installation script in Windows PowerShell, you will be guided through the installation process:

     ```
     Invoke-RestMethod 'https://cli.btp.cloud.sap/btpcli-install.ps1' | Invoke-
     Expression
     ```

     To uninstall the BTP CLI:

     ```
     Invoke-RestMethod 'https://cli.btp.cloud.sap/btpcli-uninstall.ps1' |
     Invoke-Expression
     ```

   - Option 2: **Windows Package Manager**

You can use Windows Package Manager (WinGet) to download the latest version of the BTP CLI. WinGet downloads the client from SAP Development Tools. In your terminal, run:

```
winget install SAP.btp --source winget
```

- Option 3: **Manually install the latest Microsoft Windows (amd64) version**
  1. Download the latest Microsoft Windows client from SAP Development Tools.
  2. Extract the client from `tar.gz` archive.
     Use PowerShell or an external program, such as WinRar, to extract the tar.gz file. Tip: Once you've unpacked the executable file, enter `cmd` or `powershell` in the address bar of the folder. This opens the command prompt or PowerShell in this folder. Or proceed with the next step to enable calling `btp` from any location on your system.
  3. Copy the client executable from the unpacked folder to a directory of your choice. We recommend the following, because it ensures that you can call `btp` system-wide, i.e., that it is in your PATH:
     `C:/Users/<your-user>`
     In the Windows search, enter **`Environment Variable`** and open the *System Properties*. On the *Advanced* tab, open *Environment Variables*. Under *User Variables*, open the *Path* entry and add the file location of the btp.exe (`C:\Users<your-user>`).
  4. **Optional:** If you have a proxy server configured in your environment, you need to specify its address and port as environment variable `HTTPS_PROXY` or `https_proxy` to access SAP BTP. For example, `HTTPS_PROXY=https://my-https-proxy:1234`.
     The specified proxy server is then used for HTTPS requests, unless overridden by the NO_PROXY or no_proxy, environment variables, which define a comma-separated list of hosts to be excluded from proxying.

2. In your terminal, run `btp`. Note that you need read and write permissions in the target folder to run this executable.
3. Log into your global account with `btp login` and enter the required information. You must confirm the BTP CLI server (`https://cli.btp.cloud.sap/`) and, if you have more than one, select your global account. For details, see Log in.
4. Once you're logged in, familiarize yourself with the BTP CLI, for example with How to Work with the BTP CLI, Command Syntax of the BTP CLI, or simply by trying out a few commands, such as the following:

   ```
   btp list accounts/subaccount
   ```

   ```
   btp list security/user
   ```

   ```
   btp get security/user "name@example.com"
   ```

   ```
   btp get accounts/global-account"
   ```

   > → Tip
   >
   > Use the command autocompletion feature in the BTP CLI to save keystrokes when entering command actions, group-object combinations, and their parameters in the command line. For more information, see Enable Command Autocompletion [page 2686].

5. If you're going to work in a subaccount of this global account, consider setting the target to this subaccount using `btp target` and select the subaccount. See Set a Target for Subsequent Commands with btp target.
6. To view the current context, target, and version, use `btp`.

6.1.3.1.2 Get the BTP CLI Client for Apple macOS (amd64 and arm64)

Context

Procedure

1. To get the BTP CLI client, you can either run the installation script or download the client manually.
   - Option 1: **Installation Script**
     When you run the installation script in your Apple macOS terminal, you will be guided through the installation process:

     ```
     curl https://cli.btp.cloud.sap/btpcli-install.sh | zsh
     ```

     To uninstall the BTP CLI:

     ```
     curl https://cli.btp.cloud.sap/btpcli-uninstall.sh | zsh
     ```
   - Option 2: **Homebrew Package Manager**
     You can use Homebrew open source package management software to download the latest version of the BTP CLI. Homebrew downloads the client from SAP Development Tools. In your terminal, run:

     ```
     brew install --cask btp
     ```
   - Option 3: **Manually install the latest Apple macOS (amd64 and arm64) version**
     1. Download the latest Apple macOS client (amd64 or arm64) from SAP Development Tools.
     2. Extract the client from `tar.gz` archive.
        Open the tar.gz file by double-clicking it.
     3. Copy the client executable from the unpacked folder to a directory of your choice. We recommend the following, because it ensures that you can call `btp` system-wide, i.e., that it is in your PATH: `/usr/local/bin`
     4. **Optional:** If you have a proxy server configured in your environment, you must specify its address and port as environment variable `HTTPS_PROXY` or `https_proxy` to access SAP BTP. For example, `HTTPS_PROXY=https://my-https-proxy:1234`.
        The specified proxy server is then used for HTTPS requests, unless overridden by the NO_PROXY or no_proxy, environment variables, which define a comma-separated list of hosts to be excluded from proxying.
2. In your terminal, run `btp`. Note that you need read and write permissions in the target folder to run this executable.
3. Log into your global account with `btp login`and enter the required information. You must confirm the BTP CLI server (`https://cli.btp.cloud.sap/`) and, if you have more than one, select your global account. For details, see Log in.
4. Once you're logged in, familiarize yourself with the BTP CLI, for example with How to Work with the BTP CLI, Command Syntax of the BTP CLI, or simply by trying out a few commands, such as the following:

   ```
   btp list accounts/subaccount
   ```

```
btp list security/user
```

```
btp get security/user "name@example.com"
```

```
btp get accounts/global-account"
```

> → Tip
>
> Use the command autocompletion feature in the BTP CLI to save keystrokes when entering command actions, group-object combinations, and their parameters in the command line. For more information, see Enable Command Autocompletion [page 2686].

5. If you're going to work in a subaccount of this global account, consider setting the target to this subaccount using `btp target` and select the subaccount. See Set a Target for Subsequent Commands with btp target.
6. To view the current context, target, and version, use `btp`.

6.1.3.1.3 Get the BTP CLI Client for Linux (amd64 and arm64)

Context

Procedure

1. To get the BTP CLI client, you can either run the installation script or download the client manually.
   - Option 1: **Installation Script**
     When you run the installation script in your Linux terminal, you will be guided through the installation process:

     ```
     curl https://cli.btp.cloud.sap/btpcli-install.sh | bash
     ```

     To uninstall the BTP CLI:

     ```
     curl https://cli.btp.cloud.sap/btpcli-uninstall.sh | bash
     ```

   - Option 2: **Homebrew Package Manager**
     You can use Homebrew open source package management software to download the latest version of the BTP CLI. Homebrew downloads the client from SAP Development Tools. In your terminal, run:

     ```
     brew install --cask btp
     ```

   - Option 3: **Manually install the latest Linux (amd64 and arm64) version**
     1. Download the latest Linux client (amd64 or arm64) from SAP Development Tools.
     2. Extract the client from `tar.gz` archive.

Use the terminal to open the tar.gz file with `tar -xzf btp-cli-linux-amd64-latest.tar.gz`

3. Copy the client executable from the unpacked folder to a directory of your choice. We recommend the following, because it ensures that you can call `btp` system-wide, i.e., that it is in your PATH: `/usr/local/bin`
4. **Optional:** If you have a proxy server configured in your environment, you must specify its address and port as environment variable `HTTPS_PROXY` or `https_proxy` to access SAP BTP. For example, `HTTPS_PROXY=https://my-https-proxy:1234`.
   The specified proxy server is then used for HTTPS requests, unless overridden by the NO_PROXY or no_proxy, environment variables, which define a comma-separated list of hosts to be excluded from proxying.

2. In your terminal, run `btp`. Note that you need read and write permissions in the target folder to run this executable.
3. Log into your global account with `btp login` and enter the required information. You must confirm the BTP CLI server (`https://cli.btp.cloud.sap/`) and, if you have more than one, select your global account. For details, see Log in.
4. Once you're logged in, familiarize yourself with the BTP CLI, for example with How to Work with the BTP CLI, Command Syntax of the BTP CLI, or simply by trying out a few commands, such as the following:

```
btp list accounts/subaccount
```

```
btp list security/user
```

```
btp get security/user "name@example.com"
```

```
btp get accounts/global-account"
```

> → Tip
>
> Use the command autocompletion feature in the BTP CLI to save keystrokes when entering command actions, group-object combinations, and their parameters in the command line. For more information, see Enable Command Autocompletion [page 2686].

5. If you're going to work in a subaccount of this global account, consider setting the target to this subaccount using `btp target` and select the subaccount. See Set a Target for Subsequent Commands with btp target.
6. To view the current context, target, and version, use `btp`.

6.1.3.2 Get Updates

Updating the BTP CLI client is essentially replacing the old executable file with a newer version.

Context

Each BTP CLI client version is supported for at least a year after its release, but we recommend to regularly update the client. This way, you won't miss any new features or security updates. The What's New for SAP

Business Technology Platform page on SAP Help Portal announces new features and informs you if you need a client update to use them. If your client version is deprecated, an update hint is displayed in the client as well.

To find out the version of the CLI client you are using, run `btp --info` or simply `btp`.

Procedure

1. Get the latest version either by re-running the installation script or downloading the client manually.
2. Extract the client file (for example: `btp.exe`) and replace the old file with this new one.
3. Work with the btp CLI as usual and enjoy the new features.

Related Information

Log in [page 2679]
Commands in the btp CLI [page 2698]

6.1.3.3 Command Syntax of the btp CLI

Each command consists of the base call `btp` followed by a verb (the action), a combination of group and object, and parameters.

The btp CLI uses the following syntax:

```
btp [OPTIONS] ACTION GROUP/OBJECT [PARAMS]
```

The commands are ordered in groups and you need to specify the object on which you want to carry out an action by the group/object combination. Words in caps are placeholders, and brackets [ ] denote optionality. Optionally, you can pass options with a command call. Here's is one example with the verbose option and no parameters before we outline the entire syntax:

Sample Code

```
btp --verbose list accounts/subaccount
```

- `btp` is the base call to start each command
- OPTIONS can be added to each command, for example `--format json` to change the output format to json, or `--verbose` to execute a command in verbose mode. For details, see How to Work with the btp CLI [page 2676].
- ACTION is the verb. Depending on the GROUP/OBJECT combination, different verbs are available, such as `get`, `list`,`create`, `delete`, `assign`, `unassign`, `add`, `remove`. For a complete list, use `btp help`. To find out which commands are available for a specific action, use `btp help ACTION`, for example, `btp help list`.

- Special ACTION: the `help` ACTION. You can always place `help` at the beginning of a command and still add further parts of the command, such as the ACTION or GROUP or GROUP/OBJECT combination for which you want to call help. See Get Help [page 2678].
- The GROUP/OBJECT combination specifies the entity that the action is carried out on. For example, all commands related to users and their authorizations belong to the **security** group, in which **objects** such as role, role-collection, and user are available. There are currently three groups:
  - accounts: Objects related to the account model, subscriptions, and environments
  - security: Authorization objects and users
  - services: Objects related to SAP Service Manager

  To get help on a particular group, use `btp help GROUP`, for example `btp hep accounts`. This will display all objects and related actions available in that group.
- PARAMETERS are passed with most commands. With `btp login`, for example, you don't have to pass parameters up front, but you'll be prompted to enter them. The same applies to the `btp target` command. And `btp logout` does not need parameters, as it will log out the current user from the global account. Some commands have one positional parameter, which is entered directly after the command. All further parameters have a key and can be optional. The command help specifies the optional parameters as such. For example, in `btp assign security/role-collection "Global Account Administrator" --to-user example@mail.com --of-idp my-idp`, "Global Account Administrator" is the positional parameter, and the other two parameters have keys.
  You may encounter issues with certain parameter values that start with a `-`, as they can be mistaken for parameter names instead of values. It is recommended to use a = to separate parameters and values when a value begins with `-`. For example, if your value for parameter `--user` is `-username-`, please use `--user="-username-"`.

> ⓘ Note
>
> The commands that you type into the command line are interpreted and executed by the shell. Make sure you're familiar with your shell to avoid unexpected interferences. For examples of correct escaping, see Passing JSON Parameters on the Command Line [page 2673].

> → Tip
>
> You can use the command autocompletion feature in the btp CLI to save keystrokes when entering commands actions, group-object combinations, and their parameters in the command line. For more information, see Enable Command Autocompletion [page 2686].

A Few Commands to Get Started

Here are a few commands for you to try out once you're logged in (Log in [page 2679]):

```
btp list accounts/subaccount
```

```
btp list security/user
```

```
btp get security/user "name@example.com"
```

```
btp list accounts/subscription
```

Example

In this example, we assign the **Global Account Administrator** role collection to user **name@example.com** and try out some options.

Use `help` or `--help` to display command-specific help to learn how to use the command:

```
btp help assign security/role-collection
```

```
btp assign security/role-collection --help
```

```
btp --help assign security/role-collection
```

Command with positional parameter and one mandatory parameter:

```
btp assign security/role-collection "Global Account Administrator" --to-user
"name@example.com"
```

Command with positional parameter, mandatory parameter, and optional parameter:

```
btp assign security/role-collection "Global Account Administrator" --to-user
"name@example.com" --of-idp "my-idp"
```

The same command in verbose mode:

```
btp --verbose assign security/role-collection "Global Account Administrator" --
to-user "name@example.com" --of-idp "my-idp"
```

Related Information

btp CLI Command Reference

6.1.3.3.1 Passing JSON Parameters on the Command Line

Depending on the shell you use, the escaping rules are different. Here you'll find examples of correct escaping and quotes when passing JSON objects on the command line using different shells.

The table below gives an overview of the escaping rules in these commonly used shells. VALUE refers to one single line in json.

Shell	Correct Escaping and Quotes
Bash	Use `--parameter "VALUE"` and escape quotes with `\"` within `VALUE` or Use `--parameter 'VALUE'` and do not escape quotes within `VALUE`
Windows Command Prompt	Use `--param "VALUE"` and escape quotes with `\"` within `VALUE`
Windows PowerShell	Use `--param "VALUE"` and escape quotes with `\"` within `VALUE`

→ Tip

The CLI client provides examples in the command help for all commands. The examples that include JSON parameters use formatting that is compliant with Bash (Unix/Linux operating system, macOS) and Windows Command Prompt.

The command-line examples in this chapter are based on the following JSON:

```
{
    "Key1": ["Value1"],
    "Key2": ["Value1", "Value2"]
}
```

Bash (Linux, macOS)

Option 1

```
--parameter "{\"Key1\":[\"Value1\"],\"Key2\":[\"Value1\", \"Value2\"]}"
```

For example, when creating a subaccount (`btp create accounts/subaccount`) with labels `Department` = Sales and `Contacts` = name1@example.com and name2@example.com, use the following syntax:

{·} Sample Code

```
btp create account/subaccount --display-name "my-subaccount" --region us10
--subdomain "my-subdomain" --labels "{\"Department\":[\"Sales\"],\"Contacts\":
[\"name1@example.com\", \"name2@example.com\"]}"
```

Option 2

```
--parameter '{"Key1": ["Value1"], "Key2": ["Value1", "Value2"]}'
```

For example, when creating a subaccount (`btp create accounts/subaccount`) with labels `Department` = Sales and `Contacts` = name1@example.com and name2@example.com, use the following syntax:

Sample Code

```
btp create account/subaccount --display-name "my-subaccount" --region us10
--subdomain my-subdomain --labels '{"Department": ["Sales"],"Contacts":
["name1@example.com", "name2@example.com"]}'
```

Windows Command Prompt

```
--parameter "{\"Key1\":[\"Value1\"],\"Key2\":[\"Value1\", \"Value2\"]}"
```

For example, when creating a subaccount (`btp create accounts/subaccount`) with labels `Department` = Sales and `Contacts` = name1@example.com and name2@example.com, use the following syntax:

Sample Code

```
btp create account/subaccount --display-name "my-subaccount" --region us10
--subdomain "my-subdomain" --labels "{\"Department\":[\"Sales\"],\"Contacts\":
[\"name1@example.com\", \"name2@example.com\"]}"
```

Windows PowerShell

```
--parameter '{\"Key1\":[\"Value1\"],[\"Key2\":\"Value1\", \"Value2\"]}'
```

For example, when creating a subaccount (`btp create accounts/subaccount`) with labels `Department` = Sales and `Contacts` = name1@example.com and name2@example.com, use the following syntax:

Sample Code

```
./btp create account/subaccount --display-name "my-subaccount" --region us10
--subdomain my-subdomain --labels '{\"Department\":[\"Sales\"],\"Contacts\":
[\"name1@example.com\", \"name2@example.com\"]}'
```

Passing JSON Parameters as Files

You can also transfer JSON parameters by providing the full path to a JSON file. For example:

```
btp create services/binding --name my-binding-name --instance-name my-service-
instance-name --parameters "<your-user-directory>/Documents/parameters.json"
```

Related Information

Command Syntax of the btp CLI [page 2671]
Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
Working with External Resource Providers Using the btp CLI [page 2705]
Working with Environments Using the BTP CLI [page 2703]

6.1.3.4 How to Work with the btp CLI

Learn how to work with the SAP BTP command line interface (btp CLI). For example, how to log in, get help, and set a default context for commands.

General Commands

```
btp help
```

```
btp feedback
```

```
btp login
```

```
btp logout
```

```
btp target
```

```
btp enable autocomplete
```

```
btp disable autocomplete
```

```
btp list config
```

```
btp set config
```

```
btp reset config
```

Options for Each Command

The following options are available for each command. They need to be typed right after the base call `btp`, and they can be combined (for example, `btp --verbose --help list accounts/subaccount`). The `--help` option also works at the end of a command call.

```
btp [OPTIONS] ACTION GROUP/OBJECT [PARAMS]
```

Options	
--config	Specifies the location of the configuration file. See Specify the Location of the Configuration File [page 2696].
--info	Displays client and server versions, target, and context. Note that this option only works on its own (`btp --info`) and cannot be added to other command calls. You can also just use `btp` to display this info. See View Version and Current Context [page 2679].
--help	Displays help. See Get Help [page 2678].
--verbose	Prints tracing information for support or for your own troubleshooting. See Troubleshooting for the btp CLI [page 2720]. To set the command output to verbose persistently, you can change the configuration settings with `btp set config --verbose true`. See Change Configuration Settings [page 2691]. **⚠ Caution** The `--verbose` option prints input data that you pass as parameters through files. For example, if you pass a .json file as a parameter without the `--verbose` option, its content is sent directly to the backend service to be processed. If you use the `--verbose` option, the btp CLI prints the content of the file to the terminal. If there is sensitive information in such a file, the btp CLI cannot filter it out, as it doesn't understand its semantics. Use the verbose output only if you can control the output of your terminal, i.e. only locally. It should not be used in production or shared environments, where logs may be persisted or even transported out of the system.
--format	Changes the output format of a command to JSON. See Change the Output Format to JSON [page 2694]. To set the command output to json persistently, you can change the configuration settings with `btp set config --format json`. See Change Configuration Settings [page 2691].
--version	Prints the version of the client.

Related Information

btp CLI Command Reference

6.1.3.4.1 Get Help

There is extensive help in the btp CLI about every command. You can get help with the `help` action or the `--help` option.

There are several ways to call help in the btp CLI, and there is help on all levels, from a basic entry page thats explains the syntax to very detailed command-specific help that explains the usage of a command and its parameters in detail.

The easiest way to get help is probably the `help` action (for basic help, use `btp help`, as shown in the examples below. But you can also place `--help` at the end or beginning of a command call, for example, `btp list help`, `btp --help list`, and `btp list --help` are interchangeable.

Here's the help syntax:

- `btp help`
- `btp help <ACTION>`
- `btp help <GROUP>`
- `btp help <GROUP>/<OBJECT>`

Example Help Calls (progressing in level of detail)	What to Expect
`btp help`	Explains the syntax and shows plenty of examples.
`btp help all`	Displays an overview of all available commands, ordered according to their group/object combinations.
`btp help list`	Displays *list* commands with short summaries.
`btp help accounts`	Displays all objects in *accounts* group, and all available actions per group/object combination.
`btp help accounts/subaccount`	Displays all available commands for this group/object combination.
`btp help list accounts/subaccount`	Displays command-specific help, such as usage, a list of all parameters with descriptions, helpful tips, examples, and links to further documentation.

Related Information

Command Syntax of the btp CLI [page 2671]

6.1.3.4.2 View Version and Current Context

To find out the current context you're working in, run the command `btp --info` or simply `btp`.

Context

Procedure

Use `btp --info` or just `btp`.

The client displays its own version, usage information, the CLI server URL and version, the current user, the location of the configuration file, as well as the global account, directory, or subaccount you're targeting in their account hierarchy.

Related Information

Set a Target for Subsequent Commands with btp target [page 2688]

6.1.3.4.3 Log in

Log in with the BTP CLI is on global account level.

Prerequisites

- You need to pass the CLI server URL. Usually, it is proposed during login and you can confirm with ENTER: `https://cli.btp.cloud.sap/` .
  Note that in clients up until version 2.49.0, the old server URL `https://cpcli.cf.eu10.hana.ondemand.com` is proposed. For the time being, you can use either one, but we recommend to switch to `https://cli.btp.cloud.sap/` . If you are in a private cloud and your operator has provided you with a different server URL, you'll have to enter that one.
- To log in successfully, you need access to a global account. See Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138].
- Log on with the password associated with your account (S-user or P-user) in the default identity provider, SAP ID service. If you have multiple user accounts with the same email address, log on with your user ID (S-user or P-user) instead of your email address. If you've forgotten this password, reset your password. For more information, see SAP Note 3085908 and Log in with Single Sign-On [page 2683].

Context

When you log in with the BTP CLI, a session token is saved in your local configuration. This allows you to keep your authentication over multiple command calls or after restarting your computer. The maximum session length is 12 hours. After that, you'll have to log in again. If you want to end your login earlier, use `btp logout`.

Login with single sign-on

We recommend using single sign-on and authenticating directly at your identity provider through a web browser:

```
btp login --sso
```

See Log in with Single Sign-On [page 2683].

Login with a custom identity provider

To login with a custom identity provider, use:

```
btp login --sso --idp <TENANT>
```

See Log in with a Custom Identity Provider [page 2684].

Procedure

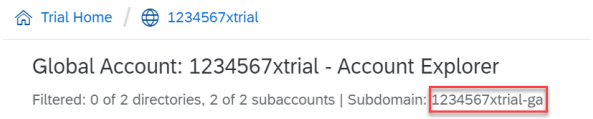
To log in manually, use `btp login`. The BTP CLI prompts for all login information, but optionally, you can provide the required information as parameters.

Usage: `btp [OPTIONS] login [PARAMS]`

Parameters

`--sso`	Opens a browser for single sign-on at the identity provider. The BTP CLI doesn't prompt for this parameter. To suppress automatic browser opening, use `--sso manual`.

`--idp <TENANT>`	This parameter is only needed if you want to log in through a custom identity provider. The CLI doesn't prompt for it. If trust is configured between your global account and a custom identity provider, use this parameter to log in through this identity provider by providing its tenant ID. To retrieve the required value, you can run `btp list security/trust` and use the value from the *Tenant* column. Or you can use the Global Account view in the cockpit, under *Security* → *Trust Configuration* → *Custom Platform Identity Providers*. The value is displayed in the *BTP CLI* column. **Note** To work with users from a custom identity provider, you need to specify the `--of-idp` parameter by providing the origin key of the custom identity provider. This is applicable to the following commands: `btp list security/user`, `btp get security/user`, `btp delete security/user`, `btp assign security/role-collection`, `btp unassign security/role-collection`, and you find this origin key in the cockpit under *Security*. For more information about using a custom identity provider, see Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576].
`--url <URL>`	The client proposes this CLI server URL: `https://cli.btp.cloud.sap/`, which you can confirm by pressing ENTER. If your operator has provided you with a different server URL, you can specify it here. Note that when you enter a new server URL for the first time, you're asked to confirm that you trust it.
`--user <USER>`	Your user name, usually an email address.
`--password <PASSWORD>`	Your password. If you have enabled Two-Factor-Authentication, append your token to your password. **→ Tip** We don't recommend to provide the password with this parameter, as it appears in plain text and may be recorded in your shell history. Rather, enter it when you're prompted.

`--subdomain <GLOBALACCOUNT>`	In the interactive login, after successful authentication, the BTP CLI will offer all of your global accounts so you can select the one to log in to. You can also provide the global account as a parameter by specifying its subdomain. You should have obtained the subdomain from your operator; but you can also find it in the cockpit in the global account view. 

```
btp login
```

If you've logged in before, the server URL, the subdomain, and the user from the last login are suggested. You can then press `Enter` to confirm, or type in different values.

```
CLI server URL [https://cli.btp.cloud.sap/]>
Subdomain [my-global-account]>
User [name@example.com]>
```

Results

Upon successful login, the BTP CLI creates a folder (`btp`) and a configuration file (`config.json`) in the default location of your user data directory:

- Microsoft Windows: `C:\Users\<username>\AppData\Roaming\SAP\btp\config.json`
- Apple macOS ; `~/Library/Application Support/.btp/config.json`
- Linux: `~/.config/.btp/config.json`

To change this location, use the `--config` option or the environment variable. See Specify the Location of the Configuration File [page 2696].

> → Tip
>
> You've logged in to the global account and all commands are executed in this global account, unless you provide a subaccount or directory ID with the command. To change this default context for subsequent commands, you can target a subaccount or directory, or even a different global account by using `btp target`. See Set a Target for Subsequent Commands with btp target [page 2688].

Related Information

BTP CLI Command Reference

6.1.3.4.3.1 Log in with Single Sign-On

Use the single sign-on parameter (`btp login --sso`) to log in to your identity provider through a browser instead of passing username and password on the command line.

Prerequisites

Login through a browser is only available if you use the default server URL that is proposed by the CLI during login (`https://cli.btp.cloud.sap/`). This is the case unless your operator has provided you specifically with a different server URL.

Context

If you have a custom identity provider configured in your global account, use `btp login --sso --idp <TENANT>`. See Log in with a Custom Identity Provider [page 2684].

> → Tip
>
> If you like logging in through single sign-on in a browser, you can configure this as the default behavior of `btp login` to save a few keystrokes at login. Use `btp set config --login.sso browser`. See Change Configuration Settings [page 2691] .

Procedure

1. Enter the following and press `ENTER` to be prompted for the server URL and the subdomain of your global account:

   ```
   btp login --sso
   ```

   Alternatively, you can pass the server URL and the subdomain of your global account as parameters:

   ```
   btp login --sso --url https://cli.btp.cloud.sap/ --subdomain <my-subdomain>
   ```

   To suppress the automatic browser-opening, you can run the command as follows:

   ```
   btp login --sso manual
   ```

2. Authenticate directly at your identity provider through your browser.

   If you already have an active session, the identity provider immediately transfers a token to the client, which completes your login. Otherwise, you need to enter your credentials.

Results

You've successfully logged in. You can return to the command line to work in your global account or - at first login - to select the global account for login.

Related Information

Log in [page 2679]
Log in with a Custom Identity Provider [page 2684]

6.1.3.4.3.2 Log in with a Custom Identity Provider

Context

If trust is configured between your global account and a custom identity provider, you need to use the `--idp` parameter to log in through this identity provider. As value, you provide its tenant ID. To retrieve the required value, you can use `btp list security/trust` and check the *Tenant* column. Or you can use the Global Account view of the cockpit under *Security* → *Trust Configuration* → *Custom Platform Identity Providers* and use the value from the *BTP CLI* column.

> **Note**
>
> To work with users from a custom identity provider, you need to specify the `--of-idp` parameter by providing the origin key of the custom identity provider. This is applicable to the following commands: `btp list security/user`, `btp get security/user`, `btp delete security/user`, `btp assign security/role-collection`, `btp unassign security/role-collection`, and you find this origin key in the cockpit under *Security*.

To learn how to configure trust to a custom identity provider, see Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576] or the command help of btp create security/trust.

> **Restriction**
>
> Keep in mind that each user is allowed a maximum of 10 parallel sessions per identity provider. This number takes into account all tools, including the cockpit and CLIs.
>
> For more information, see Restrictions When Using Custom Identity Providers for Platform Users [page 2580].

Procedure

1. To make use of single sign-on, log in with:

   ```
   btp login --sso --idp <TENANT>
   ```

2. To provide user and password on the command line, log in with:

   ```
   btp login --idp <TENANT>
   ```

   You will be promped for all required login information.

Related Information

Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576]
Trust and Federation with Identity Providers [page 2538]
Log in [page 2679]

6.1.3.4.4 Log out

Logging out of the configured server removes all user-specific data from the configuration file.

Context

Once you're finished using the btp CLI and you want to ensure that your locally stored credentials are immediately deleted, you can run the logout command. If you choose not to log out, your credentials will expire 24 hours after you last command execution, but the next time you log in, the btp CLI will propose your current subdomain and user so you won't have to type it in again.

Procedure

To log out, use `btp logout`.

This terminates your active logout session and ensures that all user-specific data is removed. The next time you log in, you will have to type in the subdomain of the global account and your user.

Related Information

Log in [page 2679]

6.1.3.4.5 Enable Command Autocompletion

Use command autocompletion to save keystrokes when entering command actions, group-object combinations, and their parameters in the SAP BTP command line interface (BTP CLI).

Context

Autocompletion in BTP CLI currently supports the following shells:

- Bash
- PowerShell
- Zsh

> **Note**
>
> The respective shell must be installed on your operating system before enabling autocomplete. Additionally, the BTP CLI executable must be available on the `path` for autocompletion to initialize. To install it on the `path`, follow the steps outlined inDownload and Start Using the BTP CLI Client [page 2665].

Once autocomplete is enabled (it's disabled by default), you use the autocomplete feature as follows in the command line:

- Enter a partial command action, group-object combination, or parameter, and then press the `Tab` key. The command line either automatically completes your command or, when there's more than one option available, it displays a list of suggested command actions/options/parameters.
- When a suggestion list is displayed, depending on your shell, use the `Tab` or arrow keys to move through the list and press `Enter` to make a selection.

Examples

The following examples show various ways that you can use autocompletion:

- Enter `btp` and press `Tab` to display all available actions:

```
./btp TAB
add           create       enable       list         logout       register
subscribe     unassign     unsubscribe
assign        delete       get          login        move         remove
target        unregister   update
```

- Partially enter `btp cre` and press `Tab` to autocomplete the command to `btp create`. Then, press `Tab` again to display a suggested list of group-object combinations:

```
./btp create TAB
accounts/directory             accounts/resource-provider     security/
role                  services/binding
accounts/environment-instance  accounts/subaccount            security/role-
collection       services/instance
```

- Partially enter a group and press `Tab` to display a suggested list of objects:

```
./btp create accountsTAB
accounts/directory             accounts/environment-instance  accounts/
resource-provider     accounts/subaccount
```

- Partially enter a parameter and press `Tab` to display a suggested list of parameters:

```
./btp create accounts/subaccount -TAB
--beta-enabled
--custom-properties
--description
--directory
--display-name
--global-account
--region
--subaccount-admins
--subdomain
--used-for-production
```

Procedure

1. Use `btp enable autocomplete <SHELL>` to enable command autocompletion for a specified shell.

   The valid values for the supported shells are:

   - `bash`
   - `powershell`
   - `zsh`

   > {·} Sample Code
   >
   > ```
   > btp enable autocomplete zsh
   > ```

2. Enter the option of your choice.

   To enable autocompletion, the startup file (RC file) of your shell is modified to load the available commands and parameters. The client looks for this RC file in your system and proposes possible files. You can choose to accept a proposal or specify a different RC file. To specify a custom path, first choose "Custom" and then enter the full path of the RC file to use. If no RC file is found, the clients prints an error message with the full path of the missing file so that you can create it and run the enable command again.

   For example:

   ```
   ./btp enable autocomplete powershell
   Which RCFile should be used for the installation?
   1: C:\Documents\WindowsPowerShell\Microsoft.PowerShell_profile.ps1
   2: Custom
   Enter option>2
   Enter the full path of
   your RCFile>C:\Documents\WindowsPowerShell\Microsoft.PowerShell_my-custom-
   profile.ps1
   ```

3. Start a new terminal session to activate the installed autocomplete script.

Results

The autocompletion option remains enabled in future sessions in your current client, until you disable it. To disable command autocompletion, run the following command:

```
btp disable autocomplete <SHELL>
```

You can run either `btp` or `btp --info` to see if command autocompletion is currently enabled and where the autocomplete RC file for your shell is located. If you don't see a line specifying the location of the autocomplete script, then it's disabled.

> → Tip
>
> If you see a discrepancy between the version of the autocomplete script and the client, the update of the autocomplete script might have failed. In such a case, try to disable and enable the autocomplete feature again.

Whenever you start a new BTP CLI terminal session, the autocomplete suggestions are automatically updated to include the latest commands.

If disabling the command autocompletion fails or you have uninstalled the BTP CLI client without disabling autocompletion, you can manually remove traces of the autocomplete installation in your shell initialization file (RC or profile file depending on your shell) by deleting the section in your RC file from `Begin SAP BTP command line interface autocomplete` to `End SAP BTP command line interface autocomplete`.

6.1.3.4.6 Set a Target for Subsequent Commands with `btp target`

Set the target for command calls to a subaccount, a directory, or a global account with the `btp target` command.

Context

During login, the btp CLI asks you for the global account you want to log in to and this global account is then the target. This means that commands are executed in this global account unless you specify otherwise via a parameter. There are two typical use cases for the target command:

- If you know that you need to work in a particular subaccount or directory, you can change the target, so that you won't have to specify that subaccount's or directory's ID as a parameter with every command call.
- If you want to switch to another global account, you can do so by setting the target to this global account without logging in again.

By setting the target to a subaccount or directory, you also target its parent entities, that is, the **target hierarchy**. If no parameter is specified in a command call, the btp CLI will execute the command in the innermost targeted entity where a command is applicable. If a subaccount is targeted that is inside a directory

and the command is not available on subaccount level, it will be executed in the parent directory (if it's a directory command) or in the parent global account (if it's a global account command). This can be useful, for example, if you have targeted a subaccount, but want to update its parent directory. You can then execute `btp update accounts/directory` without specifying the directory ID.

To see the current target, use `btp --info` or simply `btp`.

To explicitly execute a command in a parent entity instead of in the target, you can specify this with '-dir' or '-ga' parameters without a value. The value is then taken from the target hierarchy. This can be useful, for example, if you have targeted a subaccount, but want to list the users of the parent directory. You can then execute `btp list security/user -dir`. Note, however, that this command is only available on a directory level if the directory is enabled to manage authorizations.

Usage: `btp [OPTIONS] target [PARAMS]`

Procedure

1. Use `btp target` to set the target for subsequent commands.
   - Run the command without parameters to see the children, siblings and parent of you current target. You can then select your target or navigate up and down.
   - Enter `--hierarchy` as parameter to see all your global accounts with their entire account hierarchies. You can then select your target from this list.

     > → Tip
     >
     > If you like the `--hierarchy` parameter and would like to make it the default for the target command, you can define it as a configuration setting with `btp set config --target.hierarchy true`. If you then run `btp target`, the entire hierarchy of all your global accounts is shown.

   - Specify the subaccount, directory, or global account you want to target as described in the next step.
2. Enter one of the following parameters:

Parameter	Description
`--global-account, -ga <SUBDOMAIN>`	The subdomain of the global account to be targeted.
`--directory, -dir<ID>`	The ID of the directory to be targeted. You can find the directory ID by using `btp get accounts/global-account --show-hierarchy`.
`--subaccount, -sa <ID>`	The ID of the subaccount to be targeted. You can find the subaccount ID by using `btp list accounts/subaccount`.

Results

The CLI client displays the current target as a hierarchy. To execute a command in the targeted entity, you can omit the corresponding parameter.

Example

A global account can group together different directories and subaccounts that the global account administrator makes available to users. CLI commands can be executed on all levels of this account hierarchy, that is in the global account, a directory, or a subaccount, usually specified by the above-mentioned parameters. The table below shows some example commands, and explains in which account entity they can be executed and how the target mechanism can be used.

Let's look at a global account that contains a directory with a targeted subaccount. With `btp`, the current target hierarchy is displayed like this:

```
Current target:
  My Global Account (global account, subdomain: cee12xx112345-ga)
   └─ My Directory (directory, ID: 371eXXXX-55XX-40XX-b3XX-e9947ed9XXXX)
      └─ My Subaccount (subaccount, ID: d8aeXXXX-74XX-49XX-89XX-f058029eXXXX)
```

Now, when you run commands without specifying an account entity as parameter, they are executed in the targeted subaccount or in one of its parents.

Example Command	Command is Available for	Command is Executed in
`btp get accounts/available-environment`	`--subaccount`	The targeted subaccount
`btp update accounts/directory`	`[directory] ID` Note that the directory ID is a positional parameter in this command. You can omit it when a subaccount inside the directory is targeted: `btp update accounts/directory [directory ID]`.	The parent directory of the targeted subaccount
`btp list accounts/available-region`	`--global-account`	The parent global account of the targeted subaccount

`btp list security/user`	`--global-account`
	`--directory`
	`--subaccount`
		The targeted subaccount
		→ Tip
		To execute this command in a parent entity, use the `-dir` or `-ga` parameter without a value.

Example Command	Command is Available for	Command is Executed in

`btp list security/user --subaccount "1111XXXX-22XX-33XX-44XX-55555555XXXX"`	`--global-account`
	`--directory`
	`--subaccount`
		The specified subaccount, which overrides the target

Related Information

btp CLI Command Reference
Global Accounts [page 121]

6.1.3.4.7 Change Configuration Settings

Change the configuration settings to customize the behavior of the btp CLI.

Context

There are a few configuration settings that you can set to customize how the btp CLI works.

These settings are saved to the configuration file, which governs how the btp CLI works. You can have more than one configuration file for working in different accounts at the same time by logging in with the `--config PATH` option and then also providing it with each command call. See Specify the Location of the Configuration File [page 2696].

To find out where the configuration file is stored, use `btp --info`.

Procedure

1. To change a setting, run:

   ```
   btp [OPTIONS] set config [--format FORMAT] [--verbose BOOL] [--login.sso MODE] [--login.securestore BOOL] [--login.showglobalaccounts BOOL] [--target.hierarchy BOOL] [--theme THEME]
   ```

Parameter	Description
`--format FORMAT`	The output format of commands. Valid values: text (default), json.

`--verbose BOOL`	If set to true, command output includes tracing information for support. Valid values: false (default), true.
`--login.sso MODE`	Single sign-on mode of `btp login`. Valid values: none (default), browser, manual. For example, if you want `btp login` to always open a browser for login at your identity provider, use `btp set config --login.sso browser`.
`--login.securestore BOOL`	If true, your login session will be stored in your OS secure store (only macOS/Windows).
`--login.showglobalaccounts BOOL`	If set to true, `btp login` will always prompt you to choose a global account.
`--target.hierarchy BOOL`	If set to true, `btp target` displays the full hierarchy of all global accounts. Valid values: false (default), true See Set a Target for Subsequent Commands with btp target [page 2688].
`--theme THEME`	Theme setting for the CLI. Use environment variable `NO_COLOR=true` to force the theme to 'none', regardless of this setting.

2. To list the current settings, run:

   ```
   btp [OPTIONS] list config
   ```

3. To reset individual or all settings back to their default values, run:

   ```
   btp [OPTIONS] reset config [--format] [--verbose] [--login.sso] [--
   login.securestore] [--login.showglobalaccounts] [--target.hierarchy] [--theme]
   [--all]
   ```

Related Information

Specify the Location of the Configuration File [page 2696]

6.1.3.4.8 Theme Configuration for the BTP CLI

Customize the visual theme of the SAP BTP Command Line Interface (btp CLI).

Context

You can control the visual appearance of the SAP BTP Command Line Interface (BTP CLI) output by using the theme configuration parameter.

Procedure

1. **Available Themes**

   The following values are supported:

   default (default): applies the SAP-themed visual style to the CLI output.
   none: applies the standard CLI appearance without additional theming.

2. **Set the Theme**

   To configure the theme, use:

   ```
   btp set config --theme=<value>
   ```

   Examples:

   ```
   btp set config --theme=default
   btp set config --theme=none
   ```

3. **Reset the Theme Configuration**

   To reset the theme setting to its default value, run:

   ```
   btp reset config –theme
   ```

4. **NO_COLOR Environment Variable**

   If the NO_COLOR environment variable is set to true, any configured theme is ignored and the CLI output is displayed without theming.

   When `NO_COLOR=true` is set, it overrides the `--theme` configuration.

Related Information

Change Configuration Settings [page 2691]

6.1.3.4.9 Change the Output Format to JSON

Use the `--format json` option to change the output format to JSON. This is the recommended output format for automation with the btp CLI.

Context

The standard output format of the btp CLI is plain text, formatted in a way that an interactive user of the btp CLI can easily read it and therefore often limited to the most relevant information. We don't recommend using this plain text output for automatic parsing and extracting information. The text output is not stable enough, as, for example, column width or the appearance of new lines may depend on the actual content of a response.

If you're using responses of the btp CLI for automation, such as in scripts, we recommend the JSON output format, as JSON is a data format that can be parsed by most programming languages. The JSON output is much more stable than the plain text output and it often includes more information in the JSON response than the respective text output. Because it is optimized for post-processing, it doesn't inlude non-essential information such as an OK at the end of a successful call.

> → Tip
>
> To set the command output to json persistently, you can change the configuration settings with `btp set config --format json`. This makes using the `--format json` option obsolete. See Change Configuration Settings [page 2691].

See this Getting BTP resource GUIDs with the btp CLI blog post for some examples.

Procedure

Start commands with `--format json`. Currently, the only valid value is `json`.

```
btp --format json list accounts/subaccount
```

Results

Example with text output

```
btp list accounts/subaccount
subaccounts in global account c8XXXX3d-6XX7-4XXb-8dXX-89XXXX372eXX...
subaccount id:                          display
name:                                subdomain:
region:        beta-enabled:   parent id:
state:            state
message:

88XXXX80-9844-4XX7-8XXd-beXXXX42bfb2   my-
subaccount-1                            my-CF-subdomain1
```

```
us10-TEST      false           c8XXXX3d-6XX7-4XXb-8dXX-89XXXX372eXX
OK                Subaccount
created.

ebXXXX3e-f3a2-4XXf-a080-3eXXXX9b9ced   my-
subaccount-2                            my-CF-subdomain2
us10-TEST      false           c8XXXX3d-6XX7-4XXb-8dXX-89XXXX372eXX
OK                Subaccount created.
```

Example with JSON output

```
btp --format json list accounts/subaccount
{
  "value": [
    {
      "guid": "88XXXX80-9844-4XX7-8XXd-beXXXX42bfb2",
      "displayName": "my-subaccount-1",
      "globalAccountGUID": "c8XXXX3d-6XX7-4XXb-8dXX-89XXXX372eXX",
      "parentGUID": "c8XXXX3d-6XX7-4XXb-8dXX-89XXXX372eXX",
      "parentType": "ROOT",
      "region": "us10-TEST",
      "subdomain": "my-CF-subdomain1",
      "betaEnabled": false,
      "usedForProduction": "UNSET",
      "description": null,
      "expiryDate": null,
      "state": "OK",
      "stateMessage": "Subaccount created.",
      "createdDate": 1602759737909,
      "createdBy": "name@email.com",
      "modifiedDate": 1602759737909,
      "zoneId": null
    },
    {
      "guid": "ebXXXX3e-f3a2-4XXf-a080-3eXXXX9b9ced",
      "displayName": "CF TEST",
      "globalAccountGUID": "c8XXXX3d-6XX7-4XXb-8dXX-89XXXX372eXX",
      "parentGUID": "c8XXXX3d-6XX7-4XXb-8dXX-89XXXX372eXX",
      "parentType": "ROOT",
      "region": "us10-TEST",
      "subdomain": "my-CF-subdomain2",
      "betaEnabled": false,
      "usedForProduction": "UNSET",
      "description": null,
      "expiryDate": null,
      "state": "OK",
      "stateMessage": "Subaccount created.",
      "createdDate": 1602760244477,
      "createdBy": "name@email.com",
      "modifiedDate": 1602760244477,
      "zoneId": null
    }
  ]
}
```

Related Information

Change Configuration Settings [page 2691]

6.1.3.4.10 Specify the Location of the Configuration File

You can change the location of the configuration file by using the `--config` option or the environment variable.

Context

Upon successful login, the BTP CLI creates a configuration file (`config.json`) in the default location of your user config directory:

- Microsoft Windows: `C:\Users\<username>\AppData\Roaming\SAP\btp\config.json`
- Apple macOS ; `~/Library/Application Support/.btp/config.json`
- Linux: `~/.config/.btp/config.json`

This folder serves as the working directory of the BTP CLI, that is, it's necessary for its proper functioning, and is used with each command execution.

If you want the configuration file to be created in a different folder, you can use the `--config` option in your login command and then specify this location in each command call with this `--config` option.

Procedure

1. Specify the location of the configuration file with your login command:

   ```
   btp --config "<file path>" login
   ```

2. Specify this location either with the `BTP_CLIENTCONFIG` environment variable, or use the `--config` option with each subsequent command call.

   {·} Sample Code

   ```
   btp --config "<file path>"
   ```

   ⓘ Note

   In version 2.14, the environment variable `BTP_CLIENTCONFIG` was introduced. If you use an older client version, you need to use `SAPCP_CLIENTCONFIG`. Although the old one is kept, we recommend to switch to `BTP_CLIENTCONFIG`.

Example

Let's assume you want to work in two subaccounts in parallel, using two terminals. For example, with the first terminal (A), you want to work in a subaccount with ID 1000, with the second terminal (B) you want to work in a subaccount with ID 2000, and you want to list the users in each subaccount.

Terminal A	Terminal B
1. Log in to your global account using the default location of the configuration file: `btp login` Run all commands as usual. The BTP CLI uses the default configuration file.	1. Log in to your global account using a different location of the configuration file: `btp --config "C:\my-cli-folder" login` Use this option with each command call.
2. Set the target to subaccount 1000: `btp target --subaccount 1000`	2. Set the target to subaccount 2000: `btp --config "C:\my-cli-folder" target --subaccount 2000`
3. List the users of subaccount 1000: `btp list security/user`	3. List the users of subaccount 2000: `btp --config "C:\my-cli-folder" list security/user`

6.1.3.4.11 Provide Feedback about the btp CLI

With `btp feedback`, you can open a feedback survey in a web browser in which you can share your opinion about the btp CLI with the development team.

Context

A short survey will open in your web browser, in which you're asked how satisfied you are with the btp CLI, and where you can share additional comments. This feedback is carefully analyzed by the development team and a valuable source of input for further improving the btp CLI.

You can also fill it out now: Take our survey about the btp CLI

Thanks a lot for your participation. Your feedback is greatly appreciated.

Procedure

Open the feedback survey with the following command:

```
btp feedback
```

6.1.3.5 Commands in the btp CLI

Find a full reference of all btp CLI commands and their parameters here: btp CLI Command Reference.

Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
: Use the SAP BTP command line interface (btp CLI) to manage operations with global accounts, directories, and subaccounts.

Setting Entitlements Using the btp CLI [page 2702]
: Use the SAP BTP command line interface (btp CLI) to set entitlements to define the functionality or permissions available for users of global accounts, directories, and subaccounts.

Working with Environments Using the BTP CLI [page 2703]
: Use the SAP BTP command line interface (BTP CLI) to manage runtime environment instances in a subaccount. For example, enable the Cloud Foundry environment by creating a Cloud Foundry org (environment instance).

Working with Multitenant Applications Using the BTP CLI [page 2704]
: Use the SAP BTP command line interface (BTP CLI) to manage the multitenant applications to which a subaccount is entitled to subscribe.

Working with External Resource Providers Using the btp CLI [page 2705]
: Use the SAP BTP command line interface (btp CLI) to get details, or to create or delete resource provider instances in a global account.

Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707]
: SAP BTP supports identity federation. Its concept is to reuse the user bases of identity providers. To use a custom identity provider, your global account or subaccount in SAP BTP must have a trust relationship to the identity provider you want to use.

Managing Users and Their Authorizations Using the btp CLI [page 2709]
: User authorizations are managed by assigning role collections to users (for example, Subaccount Administrator). Use the SAP BTP command-line interface (btp CLI) to manage roles and role collections, and to assign role collections to users.

Managing Signing Keys for Access Tokens [page 2712]
: Use the SAP BTP command line interface (btp CLI) to manage signing keys for access tokens in the subaccount.

Managing Security Settings [page 2713]
: Use the SAP BTP command line interface (btp CLI) to display and update the security settings for the subaccount.

Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service [page 2714]
: Use the SAP BTP command line interface (btp CLI) to manage API credentials, which enable you to access the REST APIs of the SAP Authorization and Trust Management service.

Working with Resources of the SAP Service Manager Using the btp CLI [page 2715]
: Use the SAP BTP command line interface to perform various operations related to your platforms, attached service brokers, service instances, and service bindings.

Working with Multi-Region Disaster Recovery Using the BTP CLI [page 2719]
: Use the SAP BTP command line interface (BTP CLI) to manage Multi-Region Disaster Recovery setups, such as creating, viewing, or deleting subaccount pairs.

6.1.3.5.1 Working with Global Accounts, Directories, and Subaccounts Using the btp CLI

Use the SAP BTP command line interface (btp CLI) to manage operations with global accounts, directories, and subaccounts.

Working with Global Accounts

> → Tip
>
> These global account commands are always executed in the global account you're logged in to. You don't need to pass the global account parameter, even if you've targeted a subaccount or directory. See Set a Target for Subsequent Commands with btp target [page 2688].

Task	Run the command...	Command help
Get details about a global account, and the account structure (directories and subaccounts) of the global account.	`btp get accounts/global-account`	btp get accounts/global-account
Update the display name and/or description of a global account.	`btp update accounts/global-account`	btp update accounts/global-account

For more information, see Global Accounts [page 121].

Working with Directories

Directories allow you to organize and manage your subaccounts according to your technical and business needs.

> → Tip
>
> By default, all commands are executed in the global account you're logged in to. To change the target to a directory, use `btp target -dir <my-directory-id>`. See Set a Target for Subsequent Commands with btp target [page 2688].

Task	Run the command...	Command help
Get details about a directory and list the subaccounts and subdirectories in the directory	`btp get accounts/directory`	btp get accounts/directory
Create a directory	`btp create accounts/directory`	btp create accounts/directory

Task	Run the command...	Command help
Update a directory	`btp update accounts/directory`	btp update accounts/directory
Delete a directory	`btp delete accounts/directory`	btp delete accounts/directory
Change the set of enabled features (user and entitlement management) for a directory	`btp enable accounts/directory`	btp enable accounts/directory
List the user-defined labels assigned to a directory	`btp list accounts/labels`	btp list accounts/label
List the custom properties assigned to a directory **Note** Custom properties are deprecated. Custom properties support only single values per key and are now replaced by the string array `labels`, which supports multiple values per key. Use `btp list accounts/labels` instead.	`btp list accounts/custom-property`	btp list accounts/custom-property

For more information, see Directories [page 123].

Working with Subaccounts

→ Tip

By default, all commands are executed in the global account you're logged in to. To change the target to a subaccount, use `btp target -sa <my-subaccount-id>`. See Set a Target for Subsequent Commands with btp target [page 2688].

Task	Run the command...	Command help
List all subaccounts in a global account	`btp list accounts/subaccount`	btp list accounts/subaccount
Get details about a subaccount	`btp get accounts/subaccount`	btp get accounts/subaccount
Create a subaccount	`btp create accounts/subaccount`	btp create accounts/subaccount

Task	Run the command...	Command help
Update a subaccount	`btp update accounts/subaccount`	btp update accounts/subaccount
Delete a subaccount	`btp delete accounts/subaccount`	btp delete accounts/subaccount
Move a subaccount	`btp move accounts/subaccount`	btp move accounts/subaccount
List the user-defined labels assigned to a subaccount	`btp list accounts/labels`	btp list accounts/label
List the custom properties assigned to a subaccount **Note** Custom properties are deprecated. Custom properties support only single values per key and are now replaced by the string array `labels`, which supports multiple values per key. Use `btp list accounts/labels` instead.	`btp list accounts/custom-property`	btp list accounts/custom-property
Get all available regions for global account	`btp list accounts/available-region`	btp list accounts/available-region

For more information, see Subaccounts [page 122].

Parent topic: Commands in the btp CLI [page 2698]

Related Information

Setting Entitlements Using the btp CLI [page 2702]
Working with Environments Using the BTP CLI [page 2703]
Working with Multitenant Applications Using the BTP CLI [page 2704]
Working with External Resource Providers Using the btp CLI [page 2705]
Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707]
Managing Users and Their Authorizations Using the btp CLI [page 2709]
Managing Signing Keys for Access Tokens [page 2712]
Managing Security Settings [page 2713]
Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service [page 2714]
Working with Resources of the SAP Service Manager Using the btp CLI [page 2715]
Working with Multi-Region Disaster Recovery Using the BTP CLI [page 2719]
Set a Target for Subsequent Commands with btp target [page 2688]

Labels [page 125]
btp CLI Command Reference

6.1.3.5.2 Setting Entitlements Using the btp CLI

Use the SAP BTP command line interface (btp CLI) to set entitlements to define the functionality or permissions available for users of global accounts, directories, and subaccounts.

> → Tip
>
> By default, all commands are executed in the global account you're logged into. To change this to a directory or subaccount, use `btp target`. See Set a Target for Subsequent Commands with btp target [page 2688].

Task	Run the command...	Command help
Get all the entitlements and quota assignments for a global account, directories, and subaccounts	`btp list accounts/entitlement`	btp list accounts/entitlement
Assign or update an entitlement to a subaccount or a directory	`btp assign accounts/entitlement`	btp assign accounts/entitlement

Parent topic: Commands in the btp CLI [page 2698]

Related Information

Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
Working with Environments Using the BTP CLI [page 2703]
Working with Multitenant Applications Using the BTP CLI [page 2704]
Working with External Resource Providers Using the btp CLI [page 2705]
Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707]
Managing Users and Their Authorizations Using the btp CLI [page 2709]
Managing Signing Keys for Access Tokens [page 2712]
Managing Security Settings [page 2713]
Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service [page 2714]
Working with Resources of the SAP Service Manager Using the btp CLI [page 2715]
Working with Multi-Region Disaster Recovery Using the BTP CLI [page 2719]
btp CLI Command Reference
Entitlements and Quotas [page 127]

6.1.3.5.3 Working with Environments Using the BTP CLI

Use the SAP BTP command line interface (BTP CLI) to manage runtime environment instances in a subaccount. For example, enable the Cloud Foundry environment by creating a Cloud Foundry org (environment instance).

> → Tip
>
> By default, all commands are executed in the global account you're logged in to. To change this target to a subaccount, use `btp target -sa <my-subaccount-id>`. See Set a Target for Subsequent Commands with btp target [page 2688].

Task	Run the command ...	Command help
Get all available environments for a subaccount	`btp list accounts/available-environment`	btp list accounts/available-environment
Get details about an environment available for a subaccount	`btp get accounts/available-environment`	btp get accounts/available-environment
Get all environment instances of a subaccount	`btp list accounts/environment-instance`	btp list accounts/environment-instance
Get a specific environment instance of a subaccount	`btp get accounts/environment-instance`	btp get accounts/environment-instance
Create an environment instance in a subaccount	`btp create accounts/environment-instance`	btp create accounts/environment-instance
Update the plan and/or configuration parameters of an environment in a subaccount	`btp update accounts/environment-instance`	btp update accounts/environment-instance
Delete environment instances of a subaccount	`btp delete accounts/environment-instance`	btp delete accounts/environment-instance

> → Tip
>
> For information about using these BTP CLI commands with the Cloud Foundry environment, see Org Management Using the SAP BTP Command Line Interface (BTP CLI) [page 2780].
>
> For information about using BTP CLI with the project "Kyma" environment, see the Enable SAP BTP, Kyma Runtime Using the Command Line tutorial and the Creating SAP BTP, Kyma runtime via the BTP CLI blog.

Parent topic: Commands in the btp CLI [page 2698]

Related Information

Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
Setting Entitlements Using the btp CLI [page 2702]

Working with Multitenant Applications Using the BTP CLI [page 2704]
Working with External Resource Providers Using the btp CLI [page 2705]
Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707]
Managing Users and Their Authorizations Using the btp CLI [page 2709]
Managing Signing Keys for Access Tokens [page 2712]
Managing Security Settings [page 2713]
Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service [page 2714]
Working with Resources of the SAP Service Manager Using the btp CLI [page 2715]
Working with Multi-Region Disaster Recovery Using the BTP CLI [page 2719]
BTP CLI Command Reference

6.1.3.5.4 Working with Multitenant Applications Using the BTP CLI

Use the SAP BTP command line interface (BTP CLI) to manage the multitenant applications to which a subaccount is entitled to subscribe.

> → Tip
>
> By default, all commands are executed in the global account you're logged in to. To change this target to a subaccount, use `btp target -sa <my-subaccount-id>`. See Set a Target for Subsequent Commands with btp target [page 2688].

Task	Run the command...	Command help
Get all applications to which a subaccount is entitled to subscribe	`btp list accounts/subscription`	btp list accounts/subscription
Get details of a multitenant application in a subaccount	`btp get accounts/subscription`	btp get accounts/subscription
Subscribe to an application from a subaccount	`btp subscribe accounts/subaccount`	btp subscribe accounts/subaccount
Update the plan of an existing subscription > ⓘ Note > You can update a subscription plan only if additional plans for the application are entitled to the subaccount you're using and if your subscription is eligible for a plan update.	`btp update accounts/subscription`	btp update accounts/subscription

Task	Run the command...	Command help
Unsubscribe an application from a sub-account	`btp unsubscribe accounts/subaccount`	btp unsubscribe accounts/subaccount

For more information, see Subscribe to Multitenant Applications Using the Cockpit [page 2534].

Parent topic: Commands in the btp CLI [page 2698]

Related Information

Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
Setting Entitlements Using the btp CLI [page 2702]
Working with Environments Using the BTP CLI [page 2703]
Working with External Resource Providers Using the btp CLI [page 2705]
Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707]
Managing Users and Their Authorizations Using the btp CLI [page 2709]
Managing Signing Keys for Access Tokens [page 2712]
Managing Security Settings [page 2713]
Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service [page 2714]
Working with Resources of the SAP Service Manager Using the btp CLI [page 2715]
Working with Multi-Region Disaster Recovery Using the BTP CLI [page 2719]
BTP CLI Command Reference

6.1.3.5.5 Working with External Resource Providers Using the btp CLI

Use the SAP BTP command line interface (btp CLI) to get details, or to create or delete resource provider instances in a global account.

> **Note**
>
> The use of this functionality is subject to the availability of the supported non-SAP cloud vendors in your country or region.

Creating a resource provider instance allows your global account to connect to your provider account on a non-SAP cloud vendor. Through this channel, you can then consume remote service resources that you already own and which are supported by SAP BTP.

Task	Run the command...	Command help
List all resource provider instances in a global account	`btp list accounts/resource-provider`	btp list accounts/resource-provider
Get details about a resource provider instance	`btp get accounts/resource-provider`	btp get accounts/resource-provider
Create a resource provider instance	`btp create accounts/resource-provider`	btp create accounts/resource-provider
Update a resource provider instance	`btp update accounts/resource-provider`	btp update accounts/resource-provider
Delete a resource provider instance	`btp delete accounts/resource-provider`	btp delete accounts/resource-provider

For more information, see Managing Resource Providers [page 2468].

Parent topic: Commands in the btp CLI [page 2698]

Related Information

Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
Setting Entitlements Using the btp CLI [page 2702]
Working with Environments Using the BTP CLI [page 2703]
Working with Multitenant Applications Using the BTP CLI [page 2704]
Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707]
Managing Users and Their Authorizations Using the btp CLI [page 2709]
Managing Signing Keys for Access Tokens [page 2712]
Managing Security Settings [page 2713]
Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service [page 2714]
Working with Resources of the SAP Service Manager Using the btp CLI [page 2715]
Working with Multi-Region Disaster Recovery Using the BTP CLI [page 2719]
btp CLI Command Reference

6.1.3.5.6 Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant

SAP BTP supports identity federation. Its concept is to reuse the user bases of identity providers. To use a custom identity provider, your global account or subaccount in SAP BTP must have a trust relationship to the identity provider you want to use.

> ⓘ Note
>
> We recommend that you always use SAP Cloud Identity Services as a single identity provider for SAP BTP. If you use corporate identity providers, connect them to your SAP Cloud Identity Services tenant, which then acts as a hub.

Prerequisites

- You have a tenant of SAP Cloud Identity Services.
  For more information, see Tenant Model and Licensing in the documentation for SAP Cloud Identity Services.
- The SAP Cloud Identity Services tenant is associated with the customer IDs of the relevant global account of SAP BTP.
  For more information, see **Reuse SAP Cloud Identity Services Tenants for Different Customer IDs** in the documentation for SAP Cloud Identity Services.
- Make sure that the email addresses of all users in your identity provider are unique and correct.

> ⓘ Note
>
> We recommend that you request your own SAP Cloud Identity Services tenant (see Getting a Tenant).

Managing Trust Configurations

> → Tip
>
> All of these commands can be executed in the global account or in a subaccount. To choose the level, use the `btp target` command. See Set a Target for Subsequent Commands with btp target [page 2688].

To manage trust with your identity provider, use the following commands.

```
btp create security/trust --idp my-tenant --subaccount my-subaccount-id --name
my-trust-configuration --description "My Trust Configuration"
```

See Trust and Federation with Identity Providers [page 2538].

Task	Run the command...	Command help
List all trust configurations that are configured for your global account or subaccount tenants to which you can connect this global account or subaccount	`btp list security/trust`	btp list security/trust
Get details about a trust configuration	`btp get security/trust`	btp get security/trust
Establish trust from a global account or subaccount to an SAP Cloud Identity Services tenant to which you can connect this global account or subaccount	`btp create security/trust`	btp create security/trust
Update a trust configuration of a global account or subaccount	`btp update security/trust`	btp update security/trust
Delete a trust configuration from a global account or subaccount	`btp delete security/trust`	btp delete security/trust
Migrate from SAML trust to OpenID Connect trust	`btp migrate security/trust`	btp migrate security/trust
Roll back the migration of an existing SAML trust with any identity provider to OpenID Connect trust with an SAP Cloud Identity Services tenant	`btp restore security/trust`	btp restore security/trust

Finding Available SAP Cloud Identity Services Tenants

Task	Run the command...	Command help
List all SAP Cloud Identity Services	`btp list security/available-idp`	btp list security/available-ipd
Get details about an SAP Cloud Identity Services tenant that is available for a global account or a subaccount	`btp get security/available-idp` To see available tenants, run `btp list security/available-idp` .	btp get security/available-ipd

Parent topic: Commands in the btp CLI [page 2698]

Related Information

Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
Setting Entitlements Using the btp CLI [page 2702]
Working with Environments Using the BTP CLI [page 2703]
Working with Multitenant Applications Using the BTP CLI [page 2704]
Working with External Resource Providers Using the btp CLI [page 2705]
Managing Users and Their Authorizations Using the btp CLI [page 2709]
Managing Signing Keys for Access Tokens [page 2712]
Managing Security Settings [page 2713]
Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service [page 2714]
Working with Resources of the SAP Service Manager Using the btp CLI [page 2715]
Working with Multi-Region Disaster Recovery Using the BTP CLI [page 2719]

6.1.3.5.7 Managing Users and Their Authorizations Using the btp CLI

User authorizations are managed by assigning role collections to users (for example, Subaccount Administrator). Use the SAP BTP command-line interface (btp CLI) to manage roles and role collections, and to assign role collections to users.

> → Tip
>
> All of these commands can be executed in the global account, a directory, or in a subaccount. To choose the level, use the `btp target` command. See Set a Target for Subsequent Commands with btp target [page 2688].

Managing Users and Assigning Role Collections

Role collections are user-related authorizations that allow access to resources and services. You give users permissions by assigning role collections to them. All users in the global accounts, directories, and subaccounts are stored in identity providers, either in the default or in a custom identity provider. When the first role collection assignment to a user happens, SAP BTP creates a copy of this user in the global account, directory, or subaccount. This copy of the user is called shadow user.

When you do the first role collection assignment to a user through the btp CLI, such a shadow user is created by default. If you want to prevent this, you need to pass the `--create-user-if-missing` parameter with value `false`.

See Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138] and User and Member Management [page 133].

Task	Run the command ...	Command help
List users	`btp list security/user`	btp list security/user
Get details about a specific user, including role collections	`btp get security/user`	btp get security/user
Delete a user	`btp delete security/user`	btp delete security/user
Assign a role collection to a user, user group, or to an attribute	`btp assign security/role-collection`	btp assign security/role-collection
Unassign a role collection from a user, user group, or from an attribute	`btp unassign security/role-collection`	btp unassign security/role-collection

Managing Roles

A role is an instance of a role template; you can build a role based on a role template and assign the role to a role collection. See Add Roles to Role Collections on the Application Level [page 2632].

Task	Run the command ...	Command help
List apps	`btp list security/app`	btp list security/app
Get details about a specific application	`btp get security/app` To get the ID of a specific app, run `btp list security/app` .	btp get security/app
List roles	`btp list security/role`	btp list security/role
Get details about a specific role	`btp get security/role`	btp get security/role
Create a role	`btp create security/role`	btp create security/role
Delete a role	`btp delete security/role`	btp delete security/role
Add a role to a role collection	`btp add security/role`	btp add security/role
Remove a role from a role collection	`btp remove security/role`	btp remove security/role

Managing Role Collections

Role collections consist of roles, which, in turn, are based on role templates. Role collections are specific to account entities, that is, there are different role collections in global accounts, subaccounts, and directories. There are predefined role collections, such as *Global Account Administrator* and *Subaccount Viewer*. For more information, see Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138] and Working with Role Collections [page 2612].

Task	Run the command ...	Command help
List role collections	`btp list security/role-collection`	btp list security/role-collection
Get details about a specific role collection	`btp get security/role-collection`	btp get security/role-collection
Create a role collection	`btp create security/role-collection`	btp create security/role-collection
Change the description of a role collection	`btp update security/role-collection`	btp update security/role-collection
Delete a role collection	`btp delete security/role-collection`	btp delete security/role-collection

Parent topic: Commands in the btp CLI [page 2698]

Related Information

Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
Setting Entitlements Using the btp CLI [page 2702]
Working with Environments Using the BTP CLI [page 2703]
Working with Multitenant Applications Using the BTP CLI [page 2704]
Working with External Resource Providers Using the btp CLI [page 2705]
Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707]
Managing Signing Keys for Access Tokens [page 2712]
Managing Security Settings [page 2713]
Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service [page 2714]
Working with Resources of the SAP Service Manager Using the btp CLI [page 2715]
Working with Multi-Region Disaster Recovery Using the BTP CLI [page 2719]
Security Administration: Managing Authentication and Authorization [page 2537]
Set a Target for Subsequent Commands with btp target [page 2688]

Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138]
btp CLI Command Reference

6.1.3.5.8 Managing Signing Keys for Access Tokens

Use the SAP BTP command line interface (btp CLI) to manage signing keys for access tokens in the subaccount.

You can create and manage signing keys for access tokens.

> **Note**
>
> The newly created key only becomes active once you enable it. The number of keys is restricted to 2 per subaccount.

When a signing key is enabled, all newly requested tokens are signed with this key and the existing signing key is disabled. It's also possible to list all signing keys, no matter whether they're enabled or disabled. You can also delete a disabled signing key.

For more information, see **Rotate Signing Keys of Access Tokens**.

Task	Run the command ...	Command help
Create a new signing key for access tokens	`btp create security/token-key`	btp create security/token-key
Enable an existing key as signing key for access tokens	`btp enable security/token-key`	btp enable security/token-key
Delete a disabled signing key for access tokens	`btp delete security/token-key`	btp delete security/token-key
List the existing signing keys for access tokens and indicates which key is currently enabled	`btp list security/token-key`	btp list security/token-key

Parent topic: Commands in the btp CLI [page 2698]

Related Information

Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
Setting Entitlements Using the btp CLI [page 2702]
Working with Environments Using the BTP CLI [page 2703]
Working with Multitenant Applications Using the BTP CLI [page 2704]

Working with External Resource Providers Using the btp CLI [page 2705]
Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707]
Managing Users and Their Authorizations Using the btp CLI [page 2709]
Managing Security Settings [page 2713]
Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service [page 2714]
Working with Resources of the SAP Service Manager Using the btp CLI [page 2715]
Working with Multi-Region Disaster Recovery Using the BTP CLI [page 2719]

6.1.3.5.9 Managing Security Settings

Use the SAP BTP command line interface (btp CLI) to display and update the security settings for the subaccount.

The security settings comprise configuration settings, such as information about signing keys, URLs for cross origin resource sharing or iframing, and about bindings.

For more information, see Configure Trusted Domains for Multi-Environment Subaccounts [page 2655] and Configure Token Policy for SAP Authorization and Trust Management Service [page 2656].

Task	Run the command ...	Command help
Show the security settings of a subaccount	`btp list security/settings`	btp list security/settings
Update the security settings of a subaccount	`btp update security/settings`	btp update security/settings

Parent topic: Commands in the btp CLI [page 2698]

Related Information

Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
Setting Entitlements Using the btp CLI [page 2702]
Working with Environments Using the BTP CLI [page 2703]
Working with Multitenant Applications Using the BTP CLI [page 2704]
Working with External Resource Providers Using the btp CLI [page 2705]
Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707]
Managing Users and Their Authorizations Using the btp CLI [page 2709]
Managing Signing Keys for Access Tokens [page 2712]
Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service [page 2714]
Working with Resources of the SAP Service Manager Using the btp CLI [page 2715]

Working with Multi-Region Disaster Recovery Using the BTP CLI [page 2719]

6.1.3.5.10 Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service

Use the SAP BTP command line interface (btp CLI) to manage API credentials, which enable you to access the REST APIs of the SAP Authorization and Trust Management service.

If you want to access the REST APIs of SAP Authorization and Trust Management service in multi-environment subaccounts, global accounts, or directories, you must provide the required API credentials. One main use case is for customers who want to provision users including their authorizations, for example into a global account or a directory. They need credentials (with a client ID) for provisioning of users, especially when using SAP Cloud Identity Services - Identity Provisioning.

The API credentials are only passed on when they are created. They can't be retrieved later. If lost, new credentials must be generated.

See Application Security Descriptor Configuration Syntax [page 592] for the credential types and SAP Business Accelerator Hub for the APIs of the SAP Authorization and Trust Management service.

Task	Run the command ...	Command help
Create API credentials	`btp create security/api-credential`	btp create security/api-credentials
Show all API credentials	`btp list security/api-credential`	btp list security/api-credential
Show details about specific API credentials	`btp get security/api-credential`	btp get security/api-credential
Delete API credentials	`btp delete security/api-credential`	btp delete security/ api.credential

Parent topic: Commands in the btp CLI [page 2698]

Related Information

Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
Setting Entitlements Using the btp CLI [page 2702]
Working with Environments Using the BTP CLI [page 2703]
Working with Multitenant Applications Using the BTP CLI [page 2704]
Working with External Resource Providers Using the btp CLI [page 2705]

Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707]
Managing Users and Their Authorizations Using the btp CLI [page 2709]
Managing Signing Keys for Access Tokens [page 2712]
Managing Security Settings [page 2713]
Working with Resources of the SAP Service Manager Using the btp CLI [page 2715]
Working with Multi-Region Disaster Recovery Using the BTP CLI [page 2719]

6.1.3.5.11 Working with Resources of the SAP Service Manager Using the btp CLI

Use the SAP BTP command line interface to perform various operations related to your platforms, attached service brokers, service instances, and service bindings.

You can also get information about the service plans and service offerings associated with your subaccount.

For more information about the SAP Service Manager, see SAP Service Manager.

> → Tip
>
> All of these commands are executed in subaccounts. If you know you are working in a specific subaccount, we recommend using the `btp target` command to set the target to this subaccount. Then you won't have to provide the subaccount ID with every command call. See Set a Target for Subsequent Commands with btp target [page 2688].

For detailed descriptions of all SAP Service Manager CLI commands, see SAP Service Manager Commands for SAP BTP command line interface .

Managing Platforms

Task	Run the command ...	Command help
List all registered platforms in the current subaccount	`btp list services/platform`	btp list services/platform
Get details about a specific platform registered in the current subaccount	`btp get services/platform`	btp get services/platform
Register a new platform in the current subaccount	`btp register services/platform`	btp register services/platform
Update an existing platform registered in the current subaccount	`btp update services/platform`	btp update services/platform

Task	Run the command ...	Command help
Unregister an existing platform in the current subaccount	`btp unregister services/platform`	btp unregister services/platform

Managing Service Brokers

Task	Run the command ...	Command help
List all registered brokers in the current subaccount	`btp list services/broker`	btp list services/broker
Get a specific service broker in the current subaccount	`btp get services/broker`	btp get services/broker
Register a new service broker in the current subaccount	`btp register services/broker`	btp register services/broker
Update an existing service broker in the current subaccount	`btp update services/broker`	btp update services/broker
Unregister an existing service broker in the current subaccount	`btp unregister services/broker`	btp unregister services/broker

Managing Service Instances

Task	Run the command ...	Command help
List all service instances associated with the current subaccount.	`btp list services/instance`	btp list services/instance
Get details about a specific service instance associated with the current subaccount.	`btp get services/instance`	btp get services/instance
Create a new service instance of the service you want to consume.	`btp create services/instance`	btp create services/instance
Delete an existing service instance.	`btp delete services/instance`	btp delete services/instance
Share a service instance	`btp share services/instance`	

Task	Run the command ...	Command help
Unshare a service instance	`btp unshare services/instance`	

> ### Note
>
> The optional `--show-parameters` available for `btp get services/instance` and `btp get services/binding` commands returns parameters only if the service offering associated with the instance or binding for which to view parameters has the `instances_retrievable` or `bindings_retrievable` parameter values respectively set to **true**.

> ### → Tip
>
> To check associated service offering's `instances_retrievable` value, run the following commands:
>
> 1. `btp get services/instance` and find the `plan_id` for the instance. Use the plan ID in
> 2. `btp get services/plan` and check the `offering_id` for the instance. Use the offering ID in
> 3. `btp get services/offering` and see whether the `instances_retrievable` parameter's value is set to true.
>
> Start with `btp get services/binding` and then use the same commands for plan and offering details if you're checking the `bindings_retrievable` value.

Managing Service Bindings

Task	Run the command ...	Command help
List all service bindings associated with the current subaccount.	`btp list services/binding`	btp list services/binding
Get details about a specific service binding associated with the current subaccount.	`btp get services/binding`	btp get services/binding
Create a new binding between an existing service instance and an application.	`btp create services/binding`	btp create services/binding
Delete an existing binding between a service instance and an application.	`btp delete services/binding`	btp delete services/binding

Viewing Service Plans

Task	Run the command ...	Command help
List all service plans of services available for consumption that are associated with your current subaccount.	`btp list services/plan`	btp list services/plan
Get details about a specific service plan of a service that is available for consumption and associated with your current subaccount.	`btp get services/plan`	btp get services/plan

Viewing Service Offerings

Task	Run the command ...	Command help
List all service offerings associated with your current subaccount.	`btp list services/offering`	btp list services/offering
Get details about a specific service offering associated with your subaccount.	`btp get services/offering`	btp get services/offering

Parent topic: Commands in the btp CLI [page 2698]

Related Information

Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
Setting Entitlements Using the btp CLI [page 2702]
Working with Environments Using the BTP CLI [page 2703]
Working with Multitenant Applications Using the BTP CLI [page 2704]
Working with External Resource Providers Using the btp CLI [page 2705]
Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707]
Managing Users and Their Authorizations Using the btp CLI [page 2709]
Managing Signing Keys for Access Tokens [page 2712]
Managing Security Settings [page 2713]
Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service [page 2714]
Working with Multi-Region Disaster Recovery Using the BTP CLI [page 2719]

6.1.3.5.12 Working with Multi-Region Disaster Recovery Using the BTP CLI

Use the SAP BTP command line interface (BTP CLI) to manage Multi-Region Disaster Recovery setups, such as creating, viewing, or deleting subaccount pairs.

> → Tip
>
> You must be assigned to Central Disaster Recovery Administrator role by being added to Subaccount Administrator role collection in both subaccounts. Each subaccount can only be paired to one subaccount. You can create instance pairs and subscription pairs in paired subaccounts.

For more information about the Multi-Region Disaster Recovery, see SAP BTP Multi-Region Guide.

Task	Run the command...	Command help
Pair a subaccount with another subaccount in the paired region	`btp create disaster-recovery/subaccount-pair`	btp create disaster-recovery/subaccount-pair
Get a subaccount pair	`btp get disaster-recovery/subaccount-pair`	btp get disaster-recovery/subaccount-pair
Unpair a subaccount with its paired subaccount	`btp delete disaster-recovery/subaccount-pair`	btp delete disaster-recovery/subaccount-pair

Parent topic: Commands in the btp CLI [page 2698]

Related Information

Working with Global Accounts, Directories, and Subaccounts Using the btp CLI [page 2699]
Setting Entitlements Using the btp CLI [page 2702]
Working with Environments Using the BTP CLI [page 2703]
Working with Multitenant Applications Using the BTP CLI [page 2704]
Working with External Resource Providers Using the btp CLI [page 2705]
Managing Trust from SAP BTP to an SAP Cloud Identity Services Tenant [page 2707]
Managing Users and Their Authorizations Using the btp CLI [page 2709]
Managing Signing Keys for Access Tokens [page 2712]
Managing Security Settings [page 2713]
Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service [page 2714]
Working with Resources of the SAP Service Manager Using the btp CLI [page 2715]

6.1.3.6 Troubleshooting for the btp CLI

Troubleshooting and support information for the btp CLI.

Make sure that you are using a recent version of the btp CLI client

We recommend checking for updates of the client on a regular basis. Most new functionality is added via the CLI server, so no update from your side is required. But if new functionality is added to the client, the only way to get it is by downloading and installing the latest version.

Run the command `btp` or `btp --info` to see which version of the client you are using. To get the latest client, you can run the installation script for your operating system. Alternatively, you can check on our download page for the current version and manually install the latest client.

Use `btp --help` to display an overview of all commands. If new commands are available but cannot be used with your client version, these commands are marked with a notice to update your client. If you run such a command, you get an error message pointing you to the client version that you need.

I can't log on

To log on, we recommend to run `btp login --sso` or `btp login` without parameters. Then confirm the server URL that is proposed by the CLI and provide your credentials when you are prompted. After successful authentication, the btp CLI will log you into your global account, or, if you have more than one, will show all available global accounts for you to select the one you need.

To log on with SAP Universal ID, you need to use the `--sso` parameter. Otherwise log on with the password associated with your account (S-user or P-user) in the default identity provider, SAP ID service. If you've forgotten this password and this user is associated with your SAP Universal ID user, reset your password.

For more information, see SAP Note 3085908 and Log in with Single Sign-On [page 2683].

I'm getting an error message about connecting to the server

If you get an error message about connecting to the server, you should update your root CA or install the custom CA for the login URL you used. The btp CLI uses system defaults to locate the installed CA certificates. You should therefore follow the standard instructions for your operating system.

I'd like to setup a global account with the command line

If you don't know how to get started, you might want to have a look at the appropriate workflow: Setting Up a Trial Account From the Command Line [page 172] or Setting Up a Global Account via the Command Line [page 180].

I'd like to see a more detailed output (verbose mode)

If you get an error message that is not helpful enough, or if you want to find out more about what exactly happened after you ran a command, use the `btp --verbose` option with your command and run it again. The btp CLI will try to execute the command again, but also print information about each step of the command execution. This might help you understand what exactly went wrong and provide hints for you to solve the issue. If you need to create a support request, providing this information helps us analyze the error and provide a solution more quickly.

Here is an example of the `btp list security/role-collection` command call without the option:

{·} Sample Code

```
btp list security/role-collection
name                              description
Global Account Administrator      Administrative access to the global account
Global Account Viewer             Read-only access to the global account
OK
```

If you use `btp --verbose list security/role-collection`, the output is much more lengthy. It contains information such as client version and the current context, the correlation ID, and request and response details. This output may include sensitive information, so we recommend to use the verbose mode only for troubleshooting, not as default, and only on your local machine.

⚠ Caution

The `--verbose` option prints input data that you pass as parameters through files. For example, if you pass a .json file as a parameter without the `--verbose` option, its content is sent directly to the backend service to be processed. If you use the `--verbose` option, the btp CLI prints the content of the file to the terminal. If there is sensitive information in such a file, the btp CLI cannot filter it out, as it doesn't understand its semantics.

Use the verbose output only if you can control the output of your terminal, i.e. only locally. It should not be used in production or shared environments, where logs may be persisted or even transported out of the system.

{·} Sample Code

```
btp --verbose list security/role-collection
2019/12/18 10:35:21.309235 main.go:34: Support trace activated.
2019/12/18 10:35:21.309284 main.go:35: Client version 1.x.x running with
command line: --verbose list security/role-collection
2019/12/18 10:35:21.309349 clifiles.go:65: Config file path was set to /
Users/my-user/Library/Caches/.btp/config.json
2019/12/18 10:35:21.309386 login.go:138: Reading configuration from &{/
Users/my-user/Library/Caches/.btp config.json commands.json}
2019/12/18 10:35:21.309578 main.go:60: CLI server URL:            http://
localhost:8080
2019/12/18 10:35:21.309591 main.go:60: Global account subdomain:  my-ga-
subdomain
2019/12/18 10:35:21.309594 main.go:60: Subaccount:                not set
2019/12/18 10:35:21.309597 main.go:60: Configuration file:        /Users/my-
user/Library/Caches/.btp/config.json
2019/12/18 10:35:21.309603 login.go:138: Reading configuration from &{/
Users/my-user/Library/Caches/.btp config.json commands.json}
2019/12/18 10:35:21.309663 main.go:68: Successfully loaded login config.
```

```
2019/12/18 10:35:21.311758 protocol.go:41: Loading command metadata from
http://localhost:8080/commandMetadata/v-1
2019/12/18 10:35:21.311860 protocol.go:248: Adding correlation ID
1234abcde-ab12-12ab-34cd-123456abcdef to request http://localhost:8080/
commandMetadata/v-1
2019/12/18 10:35:21.319441 protocol.go:53: Received response 200 from request
http://localhost:8080/commandMetadata/v-1
2019/12/18 10:35:21.322441 protocol.go:161: Executing command list security/
role-collection with parameters map[subaccount:{0 }]
2019/12/18 10:35:21.322622 protocol.go:205: Sending request:
{POST http://localhost:8080/command/v-1/security/role-collection?list
HTTP/1.1 1 1 map[Content-Type:[application/json;charset=UTF-8] X-Cpcli-
Subdomain:[my-ga-subdomain]] {{"paramValues":{"subaccount":null}}} 0x12c6380
35 [] false localhost:8080 map[] map[] <nil> map[]   <nil> <nil> <nil>
0xc0000ae070}
to http://localhost:8080/command/v-1/security/role-collection?list
2019/12/18 10:35:21.322642 protocol.go:248: Adding correlation ID 1234abcde-
ab12-12ab-34cd-123456abcdef to request http://localhost:8080/command/v-1/
security/role-collection?list
2019/12/18 10:35:24.683689 protocol.go:214: Received response 200 from
request http://localhost:8080/command/v-1/security/role-collection?list
2019/12/18 10:35:24.690916 main.go:196: Getting response mapping for command
result code 200
2019/12/18 10:35:24.690943 main.go:198: Response mapping: {[200] ok {table
[name description]}}
name                          description
Global Account Administrator Administrative access to the global account
Global Account Viewer        Read-only access to the global account
OK
```

I need support

Use component `BC-CP-TOOLS-CLI` to contact support.

If a correlation ID is printed with your error message, please provide it with your support ticket. The correlation ID is created for each command execution, and is passed with all of its steps. It allows the identification of all log messages related to a command execution.

See Getting Support [page 3683].

6.1.4 Account Administration Using MCP Servers

Use the MCP Server for SAP BTP Administration to manage your SAP BTP account structure using natural language through a compatible AI assistant.

The Model Context Protocol (MCP) is an open standard that defines how AI assistants connect to external tools and data sources.

The MCP Server for SAP BTP Administration exposes SAP BTP account administration operations as tools that a compatible AI assistant can discover and invoke on your behalf. The server aggregates four administration domains — accounts, security, services, and connectivity — behind a single endpoint. You do not need to configure individual operations; the AI assistant discovers what is available and determines which tools to call based on your request.

Any AI client that supports the MCP standard can connect to the MCP Server for SAP BTP Administration. The following are examples of compatible AI clients:

- Claude Code
- GitHub Copilot in VS Code
- OpenCode

Documentation

In this documentation, see the following:

- Connect to the MCP Server for SAP BTP Administration [page 2723]: Prerequisites and step-by-step instructions for registering the MCP server in your AI client.
- Troubleshooting the MCP Server for SAP BTP Administration [page 2728]: Solutions to common errors when connecting to and using the MCP server.

6.1.4.1 Connect to the MCP Server for SAP BTP Administration

Register the MCP Server for SAP BTP Administration in your AI client to start managing your BTP account using natural language.

Prerequisites

Ensure that you meet the following requirements before connecting:

- You have an MCP compatible AI client installed. Examples of supported clients include Claude Code, GitHub Copilot in VS Code, and OpenCode.
- You have access to an SAP BTP global account.
- You have determined which authentication mode applies to your setup:

Authentication mode	When to use	What you need
Single Sign-On (SSO)	Your BTP platform user comes from the Default Identity Provider (`accounts.sap.com`).	An SAP user account and browser access for the one-time login flow.
Direct Connection	Your BTP platform user comes from a custom identity provider configured as a platform trust in your global account.	Your IAS username and password (Base64-encoded), and your IAS tenant subdomain. If your IAS tenant delegates login to a corporate identity provider (for example, PingID or Microsoft Entra ID), use your IAS credentials.

> **Note**
>
> After the initial login, your session is maintained automatically:
>
> - *Access tokens* expire after 30 minutes.
> - *Refresh tokens* are valid for 1 day and rotate on every use.
>
> Your MCP client renews access tokens silently in the background.
>
> Your session ends if the server goes unused for a full day, or if you explicitly log out of your SAP account from the browser (SAP Identity Service at `accounts.sap.com`). In either case, your client opens the browser login again automatically.

Context

The MCP Server for SAP BTP Administration is available at the following endpoint:

Authentication mode	URL
Single Sign-On	`https://sso.mcp.btp.cloud.sap/mcp`
Direct Connection	`https://proxy.c-769d49e.kyma.ondemand.com/mcp`

For SSO, you must also provide the OAuth client ID when registering the server: `e789ba01-5612-47ee-bfe7-79e26411c1ca`

> **Caution**
>
> The MCP Server operates with the full scope of your SAP user identity. If your trial account is registered under the same email address as your productive accounts, the server will have access to all global accounts associated with that identity. To restrict access to a trial account only, register it under a separate email address.

Procedure

Follow the instructions for your AI client.

AI client	Instructions
Claude Code	**Single Sign-On** 1. Open a terminal and run the following command: `claude mcp add --transport http BTP-Administration \ "https://sso.mcp.btp.cloud.sap/mcp" \ --client-id e789ba01-5612-47ee-bfe7-79e26411c1ca`

AI client	Instructions

	You only need to run this command once. The server remains registered across sessions.
	2. Start Claude Code: **`claude`**
	3. Type **`/mcp`** at the prompt and select the server you registered.
	4. Complete the SAP login in the browser that opens, then return to your terminal.
	Direct Connection
	1. Compute the Base64 encoding of your IAS credentials in the format `<username>:<password>` and set it as an environment variable along with your IAS tenant subdomain.
	macOS and Linux:
	`export BTP_CREDENTIALS=$(echo -n "<your-username>:<your-password>" | base64)`
	`export BTP_ORIGIN="<your-ias-tenant-subdomain>"`
	Windows (PowerShell):
	`$Env:BTP_CREDENTIALS = [Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes('<your-username>:<your-password>'))`
	`$Env:BTP_ORIGIN = '<your-ias-tenant-subdomain>'`
	2. Register the server:
	`claude mcp add --transport http BTP-Administration \ "https://proxy.c-769d49e.kyma.ondemand.com/mcp" \ --header 'Authorization: Basic ${BTP_CREDENTIALS}' \ --header 'X-Platform-Origin: ${BTP_ORIGIN}'`
	3. Start Claude Code: **`claude`**

GitHub Copilot in VS Code	**Single Sign-On**
	1. Open or create `.vscode/mcp.json` in your project and add the following entry:
	`{ "servers": { "BTP Administration": { "type": "streamableHttp", "url": "https://sso.mcp.btp.cloud.sap/mcp" } } }`
	2. When VS Code prompts for the OAuth client ID, enter `e789ba01-5612-47ee-bfe7-79e26411c1ca`. VS Code stores this value and will not ask again.
	3. Complete the SAP login in the browser that opens, then return to VS Code.
	Direct Connection
	1. Before configuring VS Code, compute the Base64 encoding of your IAS credentials in the format `<username>:<password>`.
	macOS and Linux:
	`echo -n "<your-username>:<your-password>" | base64`

AI client	Instructions

	Windows (PowerShell):
	`[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes('<your-username>:<your-password>'))`
	Copy the output — you will enter it in the next step.
	2. Open or create `.vscode/mcp.json` and add the following. VS Code prompts for credentials once per session and does not store them on disk.
	`{ "inputs": [{ "id": "btp-credentials", "type": "promptString", "description": "Base64-encoded BTP credentials (<username>:<password>)", "password": true }, { "id": "btp-origin", "type": "promptString", "description": "IAS tenant subdomain" }], "servers": { "BTP Administration": { "type": "streamableHttp", "url": "https://proxy.c-769d49e.kyma.ondemand.com/mcp", "headers": { "Authorization": "Basic ${input:btp-credentials}", "X-Platform-Origin": "${input:btp-origin}" } } } }`

OpenCode	**Single Sign-On** 1. Open your `opencode.json` configuration file and add the following entry under the `mcp` key: `{ "mcp": { "BTP Administration": { "type": "remote", "url": "https://sso.mcp.btp.cloud.sap/mcp", "oauth": { "clientId": "e789ba01-5612-47ee-bfe7-79e26411c1ca" } } } }` 2. Start OpenCode. Complete the SAP login in the browser that opens, then return to OpenCode. **Direct Connection**

AI client	Instructions

	1. Compute the Base64 encoding of your IAS credentials in the format `<username>:<password>` and set it as an environment variable along with your IAS tenant subdomain.
	macOS and Linux:
	`export BTP_CREDENTIALS=$(echo -n "<your-username>:<your-password>" | base64)`
	`export BTP_ORIGIN="<your-ias-tenant-subdomain>"`
	Windows (PowerShell):
	`$Env:BTP_CREDENTIALS = [Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes('<your-username>:<your-password>'))`
	`$Env:BTP_ORIGIN = '<your-ias-tenant-subdomain>'`
	2. Add the following to your `opencode.json`:
	`{ "mcp": { "BTP Administration": { "type": "remote", "url": "https://proxy.c-769d49e.kyma.ondemand.com/mcp", "headers": { "Authorization": "Basic {env:BTP_CREDENTIALS}", "X-Platform-Origin": "{env:BTP_ORIGIN}" } } } }`

Results

Your AI client is now connected to the MCP Server for SAP BTP Administration and can invoke BTP administration operations on your behalf.

Next Steps

If you encounter errors during or after connecting, see Troubleshooting the MCP Server for SAP BTP Administration [page 2728].

Related Information

Account Administration Using MCP Servers [page 2722]

6.1.4.2 Troubleshooting the MCP Server for SAP BTP Administration

Find solutions to common errors that occur when connecting to or using the MCP Server for SAP BTP Administration.

Error	Cause	Solution
Authentication fails with an error indicating the session has expired or the token is no longer valid.	The authentication token issued during your last login has expired. Tokens are short-lived by design.	Re-authenticate using your AI client: • **Claude Code:** Type `/mcp` at the prompt, select the server, and complete the browser login again. • **GitHub Copilot in VS Code:** VS Code detects the expired session and prompts you to log in again automatically. • **OpenCode:** Restart OpenCode and complete the browser login when prompted.
Authentication fails with an error indicating the identity provider is not recognized or the tenant cannot be found.	The custom identity provider value in your connection configuration is incorrect or missing. This applies to Direct Connection only.	Check the IAS tenant subdomain value in your configuration: • **Claude Code:** Verify that the `BTP_CLI_ORIGIN` environment variable is set to the correct IAS tenant subdomain and is exported in the shell where you run `claude`. • **GitHub Copilot in VS Code:** Check the `X-Platform-Custom-IDP` header value in your `mcp.json`. • **OpenCode:** Check the `X-Platform-Custom-IDP` header value in your `opencode.json`. You can find the correct IAS tenant subdomain in the SAP BTP cockpit under your global account trust configuration. If your platform user comes from the Default Identity Provider (`accounts.sap.com`), omit this value entirely.
A tool call returns a `403` error or a message indicating the operation is not authorized.	Your BTP user does not have the role collection required for the requested operation. The MCP server enforces the same role-based authorization as the SAP BTP cockpit.	Check the error message returned by the assistant for the name of the required role collection. Then ask your global account or subaccount administrator to assign that role collection to your user in the SAP BTP cockpit. Once the role collection is assigned, retry your request. No reconnection is needed.

Error	Cause	Solution
The MCP server does not appear in the AI client after registration, or the client reports that it cannot reach the server.	The server URL or client ID entered during registration is incorrect, or the required environment variables are not available in the current shell session.	Verify your registration configuration: • **Claude Code:** Run `claude mcp list` to check the registered URL. If it is incorrect, remove the entry with `claude mcp remove <server-name>` and register again using the correct URL and client ID from Connect to the MCP Server for SAP BTP Administration [page 2723]. For Direct Connection, confirm that the `BTP_USERNAME`, `BTP_PASSWORD`, and `BTP_CLI_ORIGIN` environment variables are exported in your current shell. • **GitHub Copilot in VS Code:** Check the URL in `.vscode/mcp.json` against the values in Connect to the MCP Server for SAP BTP Administration [page 2723]. • **OpenCode:** Check the URL and, for SSO, the `clientId` value in `opencode.json`.

Reporting an Issue

If the steps above do not resolve your issue, create a support request using SAP component `BC-CP-ADMINMCP`. See Getting Support to know more.

6.1.5 Account Administration Using APIs

SAP BTP provides REST APIs that enable you to perform administrative tasks on the global account, directory, and subaccount level, such as creating or updating subaccounts, monitoring usage information, managing access, and managing service resources.

Related Information

Account Administration Using APIs of the SAP Cloud Management Service [page 2730]
Monitoring Usage Information Using APIs of the SAP Usage Data Management Service [page 2766]
Accessing Administration Using APIs of the SAP Authorization and Trust Management Service [page 2773]
Managing Service Resources Using the APIs of the SAP Service Manager [page 2777]

6.1.5.1 Account Administration Using APIs of the SAP Cloud Management Service

Provides information about using the APIs of the SAP Cloud Management service for SAP BTP (technical name: `cis`) to manage some of the administrative operations in your accounts.

The APIs of the core services for SAP BTP allow you to manage, build, and extend the core capabilities of SAP BTP. Each core service focuses on a different aspect of the platform, such as the management of accounts and product entitlements, and the registration and provisioning of subscriptions for multitenant applications and services.

The APIs of the SAP Cloud Management service are included in the core services for SAP BTP.

The following table provides an overview of the SAP Cloud Management service APIs:

API / Microservice	Description	Region Availability[1]
`Accounts`	Provides functions to manage the directories and subaccounts in your global accounts' structure in SAP BTP.	Central region[2]
`Entitlements`	Provides functions to manage product entitlements and assignments across your global account, directories, and subaccounts.	Central region[2]
`Provisioning`	Provides functions to manage the provisioning of your environment instances, multitenant application subscriptions, and services for subaccounts in their corresponding region.	All regions
`Events`	Provides functions to get information about events relating to administrative operations in your accounts.	All regions

[1] For more information about available regions, see Regions [page 17].

[2] Refers to the eu10 Europe (Frankfurt) region, unless your global account is located in the China (Shanghai) and Government Cloud (US) regions then use the landscape domain of your central region.

For detailed information about the APIs, including their specifications and endpoints, go to SAP Core Services for SAP BTP on SAP Business Accelerator Hub

Alternatively, go to `https://events-service.<app domain>.<landscape domain>/swagger-index.html` where you can try out the APIs.

> ### Example
>
> If your global account is running on the US East (VA) region, use this URL:
>
> `https://events-service.cfapps.us10.hana.ondemand.com/swagger-index.html`

> ### → Remember
>
> - When using the `Accounts` and `Entitlements` APIs, make sure the endpoint you are using points to the landscape domain of the central region.

- To call the core platform API methods, you must obtain an access token. See Getting an Access Token for SAP Cloud Management Service APIs [page 2731].

For information about the APIs of the SAP Service Manager service, see Working with Service Management APIs.

For additional information about using the SAP Cloud Management service APIs with subaccounts in the Neo environment, see Working with Cloud Management Tools Feature Set B in the Neo Environment.

Related Information

Error Response Format [page 2757]
Asynchronous Jobs [page 2759]
Using the Events Service APIs [page 2739]
Rate Limiting [page 2759]
Monitoring Usage Information Using APIs of the SAP Usage Data Management Service [page 2766]
Using SAP SaaS Provisioning Service APIs to Manage Multitenant Applications [page 421]
Using Platform APIs
Account Model [page 121]
Environments [page 62]
Entitlements and Quotas [page 127]
Data Protection and Privacy [page 3633]

6.1.5.1.1 Getting an Access Token for SAP Cloud Management Service APIs

The APIs of the SAP Cloud Management service for SAP BTP are protected with the OAuth 2.0 Password grant type and, in some cases, also the Client Credentials grant type. This procedure guides you through the steps to create an OAuth client and obtain an access token from SAP Authorization and Trust Management service (`xsuaa`) to call the APIs of the SAP Cloud Management service.

> **Note**
>
> The Client Credentials grant type is currently available for the SAP Cloud Management service only when creating the instances of this service on a subaccount level by using the SAP Service Manager API, CLI, or when creating an instance of an SAP Cloud Management service by using the SAP BTP cockpit and selecting the *Other* environment.
>
> For more information, see Consuming Services in Other Environments Using the Service Management Instances.

The scopes included in the access token depend on the service plan you chose for the SAP Cloud Management service. For the list of the available service plans, see SAP Cloud Management Service - Service Plans [page 2736].

Prerequisites

Your global account admin has entitled at least one of the plans of the SAP Cloud Management service in your subaccount. See SAP Cloud Management Service - Service Plans [page 2736] and Managing Entitlements and Quotas Using the Cockpit [page 2521].

Procedure

1. Create a service instance for the SAP Cloud Management service (`cis`).
   When you create the service instance, specify the following:
   - The name of the service plan you want to use. See SAP Cloud Management Service - Service Plans [page 2736].
   - A name for the new service instance.
   - (Optional) Using the `directoryId` parameter, you can specify the ID of a directory in the account hierarchy for which you want to allow directory administrators to perform account management actions, such as creating subaccounts and setting entitlements in the directory, using the APIs of the Accounts and Entitlements services. The specified directory must exist in the current global account and the `AUTHORIZATIONS` feature must be enabled for the directory. You can apply this parameter only with the `central` plan of the SAP Cloud Management service (`cis`).

   There are several options available to create instances depending on the environment you use.
   - To create a service instance in the Cloud Foundry environment using the SAP BTP cockpit or the cf CLI, see Creating Service Instances [page 273].
   - To create a service instance in other environments using the Service Manager Control (SMCTL) CLI or SAP Service Manager APIs, see Consuming Services in Other Environments Using the Service Management Instances.
     You can also create and manage service instances and bindings in other environments by using the following APIs in the Accounts service:

URL	Name
`POST/accounts/v1/subaccounts/{subaccountGUID}/serviceManagementBinding`	`Create a Service Manager binding`
`GET/accounts/v1/subaccounts/{subaccountGUID}/serviceManagementBinding`	`Get a Service Manager binding`
`DELETE/accounts/v1/subaccounts/{subaccountGUID}/serviceManagementBinding`	`Delete a Service Manager binding`

     For more information about the APIs, see the `Subaccount Operations` section of the `Accounts` service in the SAP Business Accelerator Hub.

     > **Note**
     >
     > If you use the SAP Service Manager API, CLI, or the SAP BTP cockpit to create the service instance of the SAP Cloud Management service (`cis`), you can get a Client Credentials grant

type token by specifying the following parameter during the instance creation: `{"grantType": "clientCredentials"}`.

If you don't specify this parameter, the Password grant type is chosen by default.

> → Remember
>
> For `central-viewer` or `local-viewer` service plans, you can only use the Client Credentials grant type.

- To create a service instance in Kyma using the Kyma dashboard, see Using SAP BTP Services in the Kyma Environment [page 2267].

> → Recommendation
>
> If you aren't working in Cloud Foundry, Kyma, or Kubernetes, use the SAP Service Manager to create and manage service instances. These instances are platform agnostic and can be deployed and integrated with any other environment of your choice.

2. Create a service key.
   When you create the service key, specify:
   - The name of the service instance for which to create the service key.
   - A name for the service key.

   There are several options available to create service keys depending on the environment you use.
   - To create a service key in the Cloud Foundry environment using the SAP BTP cockpit or the cf CLI, see Creating Service Keys [page 278].
   - To create bindings in other environments, see Consuming Services in Other Environments Using the Service Management Instances.
   - To create credentials for calling the service and retrieving information in the Kyma environment, see Using SAP BTP Services in the Kyma Environment [page 2267].

   > {·} Sample Code
   >
   > The following example shows the service key information displayed in the Cloud Foundry environment:

```
{
   "endpoints": {
      "external_provider_registry_url": <external provider registry URL>,
      "accounts_service_url": <accounts service URL>,
      "entitlements_service_url": <entitlements service URL>,
      "events_service_url": <events service URL>,
      "order_processing_url": <order processing service URL>,
      "provisioning_service_url": <provisioning service URL>,
      "saas_registry_service_url": <saas registry service URL>,
   },
 "grant_type": "user_token",
 "uaa": {
  ...
  "clientid": <client_id>,
  "clientsecret": <client_secret>,
  "url": <uaa_url>,
  ...
 }
}
```

3. Use the `uaa_url`, `clientid`, and `clientsecret` to request an access token using the following commands:

Sample Code

For Windows OS (Password grant type):

```
curl -L -X POST "<uaa_url>/oauth/token" ^
-H "Content-Type: application/x-www-form-urlencoded" ^
-u "<clientid>:<clientsecret>" ^
-d "grant_type=password" ^
-d "username=<user email>" ^
-d "password=<password>"
```

Sample Code

For Windows OS (Client Credentials grant type):

```
curl -L -X POST "<uaa_url>/oauth/token" ^
-H "Content-Type: application/x-www-form-urlencoded" ^
-u "<clientid>:<clientsecret>" ^
-d "grant_type=client_credentials" ^
```

Sample Code

For Mac OS (Password grant type):

```
curl -L -X POST '<uaa_url>/oauth/token' \
-H 'Content-Type: application/x-www-form-urlencoded' \
-u '<clientid>:<clientsecret>' \
-d 'grant_type=password' \
-d 'username=<user email>' \
-d 'password=<password>'
```

Sample Code

For Mac OS (Client Credentials grant type):

```
curl -L -X POST '<uaa_url>/oauth/token' \
-H 'Content-Type: application/x-www-form-urlencoded' \
-u '<clientid>:<clientsecret>' \
-d 'grant_type=client_credentials' \
```

Note

If two-factor authentication (2FA) is activated on the SAP BTP landscape, then you need to append the passcode generated by the SAP Authenticator to your password.

For example, if your password is `Abcd` and the authenticator-generated passcode is `1234`, enter the password as **`Abcd1234`**.

Two-factor authentication is only relevant for Password grant type authorization.

> ### Note
>
> The access token received also contains the scopes that are granted for this access token. Therefore, only APIs that require one of these scopes can be used with this access token.
>
> ```
> {
>   "access_token": "<access_token>",
>   "token_type": "bearer",
>   "expires_in": 43199,
>   "scope": "<xsappname>.job.read <xsappname>.event.read"
> }
> ```

See Get Access to the APIs Using the apiaccess Service Plan

4. Call the SAP Cloud Management service APIs in SAP API Business Accelerator Hub with the `access_token` that you received in the previous step:
    1. Choose one of the endpoints of your instance. See step 2 and the sample code for a service instance in Cloud Foundry.
    2. Browse to `<endpoint url>/swagger-ui.html`.
    3. Enter the `api_key` and choose *Authorize*.

    > ### Note
    >
    > Use the `<access_token>` that you received in the step 3 to compose the `api_key` in the format: `bearer <access_token>`

5. Choose the API that you want to call, provide the relevant parameters, and choose *Try it out!*

Trying Out SAP Cloud Management Service API in SAP Business Accelerator Hub

To learn more about SAP Business Accelerator Hub, see What Is the SAP Business Accelerator Hub?

> ### Note
>
> In the SAP Business Accelerator Hub, you find SAP Cloud Management Service APIs under the **Core Services for SAP BTP** API Package.

To start working with SAP Cloud Management Service APIs in SAP Business Accelerator Hub, you'll need to configure an environment. See Trying Out APIs in a Configured Environment.

> ### Note
>
> You configure an environment by populating all mandatory fields in the *Create New Environment* wizard.
>
> Refer to steps 2 and 3 of this topic to see how to get the values for Client ID, Client Secret, and Token URL fields.

6.1.5.1.1.1 SAP Cloud Management Service - Service Plans

Describes the service plans for the SAP Cloud Management service for SAP BTP and the scopes that they provide. The techncial name for this service is `cis`.

Service Display Name	Technical Name	Service Plan	Scopes	Additional Configuration Parameters
Cloud Management Service	`cis`	`central`: Service plan for using SAP Cloud Management service APIs to manage your global accounts, subaccounts, directories, and entitlements.	• global-account.read • global-account.update • global-account.subaccount.read • global-account.subaccount.create • global-account.subaccount.update • global-account.subaccount.delete • global-account.account-directory.read • global-account.account-directory.create • global-account.account-directory.update • global-account.account-directory.delete • global-account.entitlement • global-account.entitlement.subaccount.update • global-account.region.read • catalog.product.update • catalog.product.delete • directory.entitlement.update	• `grantType`: Choose whether to get a Client Credentials or Password grant type token when using the SAP Service Manager API, CLI, or the SAP BTP cockpit to create the service instance of the SAP Cloud Management service (`cis`). If you don't specify this parameter, the Password grant type is chosen by default. • `directoryId`: ID of a directory to allow directory administrators to use the APIs of the Accounts and Entitlements services to perform account management actions, such as creating subaccounts and setting entitlements, in the directory. For more information, see Getting an Access Token for SAP Cloud Management Service APIs [page 2731].

Service Display Name	Technical Name	Service Plan	Scopes	Additional Configuration Parameters
			• directory.entitlement.read • job.read • cis-central.event.read • account-automation-request.read	
Cloud Management Service		`local`: Service plan for using SAP Cloud Management service APIs to manage your environments and subscriptions to multitenant applications.	• subaccount.entitlement.read • subaccount.environment.read • subaccount.environment.create • subaccount.environment.delete • subaccount.application.subscription.read • subaccount.application.subscription.update • job.read • cis-local.event.read	`grantType`: Choose whether to get a Client Credentials or Password grant type token when using the SAP Service Manager API, CLI, or the SAP BTP cockpit to create the service instance of the SAP Cloud Management service (`cis`). If you don't specify this parameter, the Password grant type is chosen by default. For more information, see Getting an Access Token for SAP Cloud Management Service APIs [page 2731].

Service Display Name	Technical Name	Service Plan	Scopes	Additional Configuration Parameters
Cloud Management Service		`central-viewer`: Service plan for using SAP Cloud Management service APIs to read your global accounts, subaccounts, directories, and service entitlements and assignments.	• global-account.read • global-account.subaccount.read • global-account.account-directory.read • global-account.entitlement.read • global-account.region.read • directory.entitlement.read • job.read • automation-module.read • automation-solution.read • automation-request.read • account-automation-request.read • account-automation-solution.read	`grantType`: Only the Client Credentials grant type is available when using the SAP Cloud Management service APIs to read the SAP BTP entities included with this plan. For more information, see Getting an Access Token for SAP Cloud Management Service APIs [page 2731].
Cloud Management Service		`local-viewer`: Service plan for using SAP Cloud Management service APIs to read your environments and subscriptions to SaaS multitenant applications.	• subaccount.entitlement.read • subaccount.environment.read • subaccount.application.subscription.read • event.read • job.read	`grantType`: Only the Client Credentials grant type is available when using the SAP Cloud Management service APIs to read the SAP BTP entities included with this plan. For more information, see Getting an Access Token for SAP Cloud Management Service APIs [page 2731].
SaaS Provisioning Service	`saas-registry`	`application`: Service plan for application owners to manage the lifecycle of multitenant applications with SAP Software-as-a-Service Provisioning service APIs.	• job.read • subscription.read • subscription.write • entitlement.read	See configuration JSON file properties in Register the Multitenant Application to the SAP SaaS Provisioning Service [page 413].

For information about assigning these plans to a subaccount, see Managing Entitlements and Quotas Using the Cockpit [page 2521].

Related Information

Getting an Access Token for SAP Cloud Management Service APIs [page 2731]

6.1.5.1.2 Using the Events Service APIs

The Events service provides REST APIs that collect information about events relating to account administrative operations in the microservices of the SAP Cloud Management service for SAP BTP, such as Accounts, Entitlements, Provisioning, and the SAP SaaS Provisioning service, within central and local regions.

You can filter the events by various query parameters, such as a specific time frame, event type, or the type of entity associated with the event.

You can also use the APIs to get all available event types.

Here are some examples of scenarios that can be implemented or built by reacting to account administrative events:

- A global account admin can implement an automatic tagging system for newly created subaccounts by polling for subaccount creation events, `Subaccount_Creation`, from the Accounts service and then adding custom property to each one of the subaccounts according to predefined criteria.
- Set up automatic notifications when a global account admin assigns quota to subaccounts. This scenario uses the `SubaccountEntitlements_Update` event from the Entitlements service.
- Send automatic alerts when a directory is deleted. This scenario uses the event `AccountDirectory_Deletion` from the Accounts service.
- Use the subscription events information, `SubaccountAppSubscription_Creation`, from the SAP SaaS Provisioning service, to send emails notifying when applications are subscribed to from a subaccount.

Permissions: A service instance of the SAP Cloud Management service is required. The scopes required to get the events are available for all of the plans for the SAP Cloud Management service. See SAP Cloud Management Service - Service Plans [page 2736].

For more information about permissions, see Getting an Access Token for SAP Cloud Management Service APIs [page 2731].

Base URI: `https://events-service.<app domain>.<landscape domain>`

Methods

HTTP Method	Action	URI
GET	Get events	`events-service.<app domain>.<landscape domain>/cloud-management/v1/events`
GET	Get event types	`events-service.<app domain>.<landscape domain>/cloud-management/v1/events/types`

Related Information

Getting an Access Token for SAP Cloud Management Service APIs [page 2731]

6.1.5.1.2.1 Get Events

Get all events associated with administrative operations in your accounts.

The events you get depend on the scopes you used to access the API.

To learn more about the scopes, see SAP Cloud Management Service - Service Plans [page 2736].

Prerequisites

You have obtained an access token with the `$XSAPPNAME.event.read` scope.

Request

URI: `https://events-service.<app domain>.<landscape domain>/cloud-management/v1/events`

HTTP Method: *GET*

Query Parameters

Parameter Name	Required	Parameter Type	Description
`entityId`	No	Array of strings	The ID of the entity associated with the event.
`entityType`	No	Array of strings	The type of entity associated with the event. For example Subaccount, Directory, or Tenant.

Parameter Name	Required	Parameter Type	Description
`eventType`	No	Array of strings	The type of the event that was triggered. There are two groups of event types, Central Events and Local Events group. The group you get depends on the scopes granted to you after you authorized to use the API. You can query any event listed in the group that is relevant for your scope. To retrieve all available event types, use the Event Types API. See Get Event Types [page 2752]. The examples of some of the events for both groups: • **Central Events group:** GlobalAccount_Update, AccountDirectory_Creation, AccountDirectory_Update, AccountDirectory_Update_Type, AccountDirectory_Deletion, Subaccount_Creation, Subaccount_Deletion, Subaccount_Update, Subaccount_Move, AccountDirectoryTenant_Creation, AccountDirectoryTenant_Deletion, GlobalAccountEntitlements_Update, EntityEntitlements_Update, EntityEntitlements_Move • **Local Events group:** SubaccountAppSubscription_Cre-

Parameter Name	Required	Parameter Type	Description
			ation, SubaccountAppSubscription_Deletion, SubaccountAppSubscription_Update, AppRegistration_Creation, AppRegistration_Deletion, AppRegistration_Update, SubaccountTenant_Creation, SubaccountTenant_Update, SubaccountTenant_Deletion, EnvironmentInstance_Creation, EnvironmentInstance_Deletion, EnvironmentInstances_Deletion
`fromActionTime`	No	Integer	Start date and time to query the events by the action that triggered them. Use the Unix epoch time in milliseconds (you can find an online converter from a regular date-time format to the Unix epoch time format). **Example** Monday, June 1, 2020 9:40:22 AM is 1590993622000 in Unix epoch milliseconds time.

Parameter Name	Required	Parameter Type	Description
`fromCreationTime`	No	Integer	Start date and time to query the events by when they were created. Use the Unix epoch time in milliseconds (you can find an online converter from a regular date-time format to the Unix epoch time format). **Example** Monday, June 10, 2020 04:32:22 AM is 1591752742000 in Unix epoch milliseconds time.
`id`	No	Array of integers	The ID of the event.
`pageNum`	No	Integer	The page number to retrieve.
`pageSize`	No	Integer	The number of events to retrieve per page (max = 150).
`searchParams`	No	JSON Object	Additional search parameters that depend on the type of the events.
`sortField`	No	String	Field by which to sort the events.
`sortOrder`	No	String	Sort order for the events. Can be ascending or descending. Available values: **ASC**,**DESC**.

Parameter Name	Required	Parameter Type	Description
`toActionTime`	No	Integer	End date and time to query the events by the action that triggered them. Use the Unix epoch time in milliseconds (you can find an online converter from a regular date-time format to the Unix epoch time format). **Example** Monday, June 4, 2020 11:40:22 AM is 1591260022000 in Unix epoch milliseconds time.
`toCreationTime`	No	Integer	End date and time to query the events by when they were created. Use the Unix epoch time in milliseconds (you can find an online converter from a regular date-time format to the Unix epoch time format). **Example** Monday, June 6, 2020 12:32:22 AM is 1591392742000 in Unix epoch milliseconds time.

Response

Returns the list of events, and its associated objects.

Content Type: *JSON*

Response Status and Error Codes

Code	Description
200	Found events (OK)
400	Bad Request Possible reasons: • The requested event type doesn't belong to the events group associated with the scope you used to access the API • Requested event type is unknown • Query arguments are invalid
401	Unauthorized Possible reasons: • Invalid token • A token with the scopes for central region was used to access the Events APIs in a local region.
403	Forbidden
404	Not Found
429	Rate Limit Exceeded
500	Internal Server Error Possible reasons: • Failed to obtain global account or subaccount info

Response Objects and Their Parameters

`BusinessEventsResponseCollection`

A collection of the event objects associated with the API call and the used scopes.

Parameter Name	Parameter Type	Description
`events`	Array of JSON objects: `BusinessEventResponseObject`	List of the event objects. See the next table `BusinessEventResponseObject` for its parameters.
`morePages`	Boolean	Whether there are more pages with event objects to return.
`pageNum`	Integer	The current page number.
`total`	Integer	Total number of results.
`totalPages`	Integer	Total number of pages.

`BusinessEventResponseObject`

Includes details about an event in the list.

Parameter Name	Parameter Type	Description
`actionTime`	String	The time the action triggered the event. The format is Unix epoch time in milli-seconds.
`creationTime`	String	The time the event record was created. The format is Unix epoch time in milli-seconds.
`details`	JSON object	Contains description and details about the requested events.
`entityId`	String	The ID of the entity associated with the event.
`entityType`	String	The type of the entity associated with the event.
`eventOrigin`	String	The service that reported the event.

Parameter Name	Parameter Type	Description
`eventType`	String	The type of the event that was triggered. There are two groups of event types: Local Events and Central Events group. Only event types that belong to one of the groups is returned as the result of a single API call. The event types group you get depends on the scope you used to access the API. The examples of some of the events for both groups: • Central Events group: GlobalAccount_Update, AccountDirectory_Creation, AccountDirectory_Update, AccountDirectory_Update_Type, AccountDirectory_Deletion, Subaccount_Creation, Subaccount_Deletion, Subaccount_Update, Subaccount_Move, AccountDirectoryTenant_Creation, AccountDirectoryTenant_Deletion, GlobalAccountEntitlements_Update, EntityEntitlements_Update, EntityEntitlements_Move • Local Events group: SubaccountAppSubscription_Creation, SubaccountAppSubscription_Deletion, SubaccountAppSubscription_Update, AppRegistration_Creation, AppRegistration_Deletion, AppRegistration_Update, SubaccountTenant_Creation, SubaccountTenant_Update, SubaccountTenant_Deletion, EnvironmentInstance_Creation, EnvironmentInstance_Deletion, EnvironmentInstances_Deletion
`globalAccountGUID`	String	The unique ID of the global account associated with the event.
`id`	Integer	The ID of the event.

Response Example


```
{
    "total": 3,
    "totalPages": 1,
    "pageNum": 0,
    "morePages": false,
    "events": [
        {
            "id": 584299,
            "actionTime": 1593494674668,
            "creationTime": 1593494674709,
            "details": {
                "description": "Subaccount created.",
                "guid": "6699b187-d17c-4783-bc8c-3263690d4aac",
                "parentGuid": "a7410cf4-84a5-46ac-9c15-52e34d283eb2",
                "displayName": "trial",
                "subaccountDescription": null,
                "region": "us10-staging",
                "jobLocation": null,
                "subdomain": "6a193bbetrial",
                "betaEnabled": false,
                "expiryDate": null
            },
            "globalAccountGUID": "a7410cf4-84a5-46ac-9c15-52e34d283eb2",
            "entityId": "6699b187-d17c-4783-bc8c-3263690d4aac",
            "entityType": "Subaccount",
            "eventOrigin": "accounts-service",
            "eventType": "Subaccount_Creation"
        },
        {
            "id": 584300,
            "actionTime": 1593494675583,
            "creationTime": 1593494676435,
            "details": {
                "description": "Addition and update of entitlements in a global
account",
                "entitlementItems": {
                    "entitlements": [
                        {
                            "productId": "trial",
                            "name": "TRIAL",
                            "amount": 1,
                            "effectiveDate": null,
                            "allowedSubaccountIDs": null
                        }
                    ],
                    "origin": null
                }
            },
            "globalAccountGUID": "a7410cf4-84a5-46ac-9c15-52e34d283eb2",
            "entityId": "a7410cf4-84a5-46ac-9c15-52e34d283eb2",
            "entityType": "GlobalAccount",
            "eventOrigin": "entitlements-service",
            "eventType": "GlobalAccountEntitlements_Update"
        },
        {
            "id": 584315,
            "actionTime": 1593494726523,
            "creationTime": 1593494727151,
            "details": {
                "description": "Assignment of entitlements from global account
to subaccount",
                "servicePlanAssignments": "{\"destinationlite\":{\"decreased\":
{},\"increased\":{\"6699b187-d17c-4783-bc8c-3263690d4aac\":
{\"amount\":10,\"servicePlanAssignmentId\":21589}},\"serviceName\":\"destination\
",\"servicePlanName\":\"lite\",\"servicePlanUniqueIdentifier\":\"destinationlite\
",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"mongodbm
```

```
edium\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21590}},\"serviceName\":\"mongodb\",\"s
ervicePlanName\":\"medium\",\"servicePlanUniqueIdentifier\":\"mongodbmedium\",\"i
sEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"applicationru
ntime\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":100,\"servicePlanAssignmentId\":21591}},\"serviceName\":\"APPLICATION
_RUNTIME\",\"servicePlanName\":\"MEMORY\",\"servicePlanUniqueIdentifier\":\"appli
cationruntime\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT
\"},\"cis-local\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":5,\"servicePlanAssignmentId\":21592}},\"serviceName\":\"cis\",\"servi
cePlanName\":\"local\",\"servicePlanUniqueIdentifier\":\"cis-
local\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"ht
ml5-apps-repo-app-host\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":10,\"servicePlanAssignmentId\":21593}},\"serviceName\":\"html5-apps-
repo\",\"servicePlanName\":\"app-host\",\"servicePlanUniqueIdentifier\":\"html5-
apps-repo-app-
host\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"rab
bitmqsmall\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21594}},\"serviceName\":\"rabbitmq\",\"
servicePlanName\":\"small\",\"servicePlanUniqueIdentifier\":\"rabbitmqsmall\",\"i
sEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"rabbitmq-
virtualhost\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":5,\"servicePlanAssignmentId\":21595}},\"serviceName\":\"rabbitmq\",\"
servicePlanName\":\"virtualhost\",\"servicePlanUniqueIdentifier\":\"rabbitmq-
virtualhost\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"
},\"postgresqlmedium\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21596}},\"serviceName\":\"postgresql\",
\"servicePlanName\":\"medium\",\"servicePlanUniqueIdentifier\":\"postgresqlmedium
\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"postgre
sqllarge\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21597}},\"serviceName\":\"postgresql\",
\"servicePlanName\":\"large\",\"servicePlanUniqueIdentifier\":\"postgresqllarge\"
,\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"screening
hits-application\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21598}},\"serviceName\":\"screeninghits
\",\"servicePlanName\":\"saas-
application\",\"servicePlanUniqueIdentifier\":\"screeninghits-
application\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"
},\"redis-dev\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":10,\"servicePlanAssignmentId\":21599}},\"serviceName\":\"redis\",\"se
rvicePlanName\":\"dev\",\"servicePlanUniqueIdentifier\":\"redis-
dev\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"redi
slarge\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21600}},\"serviceName\":\"redis\",\"ser
vicePlanName\":\"large\",\"servicePlanUniqueIdentifier\":\"redislarge\",\"isEntit
lementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"rabbitmq-dev\":
{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-bc8c-3263690d4aac\":
{\"amount\":10,\"servicePlanAssignmentId\":21601}},\"serviceName\":\"rabbitmq\",\
"servicePlanName\":\"dev\",\"servicePlanUniqueIdentifier\":\"rabbitmq-
dev\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"mong
odblarge\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21602}},\"serviceName\":\"mongodb\",\"s
ervicePlanName\":\"large\",\"servicePlanUniqueIdentifier\":\"mongodblarge\",\"isE
ntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"rabbitmqlarge\"
:{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21603}},\"serviceName\":\"rabbitmq\",\"
```

```
servicePlanName\":\"large\",\"servicePlanUniqueIdentifier\":\"rabbitmqlarge\",\"i
sEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"featureflagsl
ite\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-bc8c-3263690d4aac\":
{\"amount\":10,\"servicePlanAssignmentId\":21604}},\"serviceName\":\"feature-
flags\",\"servicePlanName\":\"lite\",\"servicePlanUniqueIdentifier\":\"featurefla
gslite\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"a
uditlog-viewer_standard_\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":10,\"servicePlanAssignmentId\":21605}},\"serviceName\":\"auditlog-
viewer\",\"servicePlanName\":\"standard\",\"servicePlanUniqueIdentifier\":\"audit
log-
viewer_standard_\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCO
UNT\"},\"redismedium\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21606}},\"serviceName\":\"redis\",\"ser
vicePlanName\":\"medium\",\"servicePlanUniqueIdentifier\":\"redismedium\",\"isEnt
itlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"rabbitmqmedium\":
{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21607}},\"serviceName\":\"rabbitmq\",\"
servicePlanName\":\"medium\",\"servicePlanUniqueIdentifier\":\"rabbitmqmedium\",\
"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"postgresqls
mall\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-bc8c-3263690d4aac\":
{\"amount\":2147483647,\"servicePlanAssignmentId\":21608}},\"serviceName\":\"post
gresql\",\"servicePlanName\":\"small\",\"servicePlanUniqueIdentifier\":\"postgres
qlsmall\",\"isEntitlementUnlimited\":true,\"assignmentLevel\":\"SUBACCOUNT\"},\"m
ongodb_xsmall\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21609}},\"serviceName\":\"mongodb\",\"s
ervicePlanName\":\"xsmall\",\"servicePlanUniqueIdentifier\":\"mongodb_xsmall\",\"
isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"connectivity
lite\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-bc8c-3263690d4aac\":
{\"amount\":10,\"servicePlanAssignmentId\":21610}},\"serviceName\":\"connectivity
\",\"servicePlanName\":\"lite\",\"servicePlanUniqueIdentifier\":\"connectivitylit
e\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"rabbit
mqxsmall\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":2147483647,\"servicePlanAssignmentId\":21611}},\"serviceName\":\"rabb
itmq\",\"servicePlanName\":\"xsmall\",\"servicePlanUniqueIdentifier\":\"rabbitmqx
small\",\"isEntitlementUnlimited\":true,\"assignmentLevel\":\"SUBACCOUNT\"},\"por
talservicessite\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":10,\"servicePlanAssignmentId\":21612}},\"serviceName\":\"portal-
services\",\"servicePlanName\":\"site\",\"servicePlanUniqueIdentifier\":\"portals
ervicessite\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"
},\"redisxsmall\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21613}},\"serviceName\":\"redis\",\"ser
vicePlanName\":\"xsmall\",\"servicePlanUniqueIdentifier\":\"redisxsmall\",\"isEnt
itlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"mongodbsmall\":
{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21614}},\"serviceName\":\"mongodb\",\"s
ervicePlanName\":\"small\",\"servicePlanUniqueIdentifier\":\"mongodbsmall\",\"isE
ntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"redissmall\":
{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21615}},\"serviceName\":\"redis\",\"ser
vicePlanName\":\"small\",\"servicePlanUniqueIdentifier\":\"redissmall\",\"isEntit
lementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"postgresqlxsmall\":
{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21616}},\"serviceName\":\"postgresql\",
\"servicePlanName\":\"xsmall\",\"servicePlanUniqueIdentifier\":\"postgresqlxsmall
\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"mongodb
-dev-large\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21617}},\"serviceName\":\"mongodb\",\"s
ervicePlanName\":\"dev-large\",\"servicePlanUniqueIdentifier\":\"mongodb-dev-
large\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"sc
reening-application\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
```

```
{\"amount\":1,\"servicePlanAssignmentId\":21618}},\"serviceName\":\"screening\",\
"servicePlanName\":\"saas-
application\",\"servicePlanUniqueIdentifier\":\"screening-
application\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"
},\"postgresql-dev\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":150,\"servicePlanAssignmentId\":21619}},\"serviceName\":\"postgresql\
",\"servicePlanName\":\"dev\",\"servicePlanUniqueIdentifier\":\"postgresql-
dev\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"samp
le-saas-app-i337243-test\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":1,\"servicePlanAssignmentId\":21620}},\"serviceName\":\"sample-saas-
app-
i337243\",\"servicePlanName\":\"test\",\"servicePlanUniqueIdentifier\":\"sample-
saas-app-i337243-
test\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},\"job
scheduler-standard\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":5,\"servicePlanAssignmentId\":21621}},\"serviceName\":\"jobscheduler\
",\"servicePlanName\":\"standard\",\"servicePlanUniqueIdentifier\":\"jobscheduler
-standard\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"},
\"cis-central\":{\"decreased\":{},\"increased\":{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":
{\"amount\":5,\"servicePlanAssignmentId\":21622}},\"serviceName\":\"cis\",\"servi
cePlanName\":\"central\",\"servicePlanUniqueIdentifier\":\"cis-
central\",\"isEntitlementUnlimited\":false,\"assignmentLevel\":\"SUBACCOUNT\"}}",
                "subaccountIdToRegion": "{\"6699b187-d17c-4783-
bc8c-3263690d4aac\":\"us10-staging\"}"
            },
            "globalAccountGUID": "a7410cf4-84a5-46ac-9c15-52e34d283eb2",
            "entityId": "a7410cf4-84a5-46ac-9c15-52e34d283eb2",
            "entityType": "GlobalAccount",
            "eventOrigin": "entitlements-service",
            "eventType": "SubaccountEntitlements_Update"
        }
    ]
}
```


Related Information

Getting an Access Token for SAP Cloud Management Service APIs [page 2731]
Using the Events Service APIs [page 2739]
Get Event Types [page 2752]

6.1.5.1.2.2 Get Event Types

Get all available event types, including their categories and their available search parameters.

The event types you get are either for a central or for a local region, and the region you get depends on the scopes you used to access the API.

To learn more about the scopes, see SAP Cloud Management Service - Service Plans [page 2736].

Prerequisites

You have obtained an access token with the `$XSAPPNAME.event.read` scope.

Request

URI: `https://events-service.<app domain>.<landscape domain>/cloud-management/v1/events/types`

HTTP Method: *GET*

Response

Returns the event types, their descriptions and categories.

Content Type: *JSON*

Response Status and Error Codes

Code	Description
200	OK
401	Unauthorized Possible reasons: • Invalid token
403	Forbidden
404	Not Found
429	Rate Limit Exceeded
500	Internal Server Error Possible reasons: • Failed to obtain global account or subaccount info

Response Objects and Their Parameters

`BusinessEventTypeResponseObject`

A JSON object that contains details about event types.

Parameter Name	Parameter Type	Description
`category`	String	Category to which the event type belongs. Possible values: • LOCAL: The event is associated with the local region within a multi-region universe. • CENTRAL: The event is associated with the central region within a multi-region universe.
`description`	String	The description of the event type.
`searchParams`	Array	List of all the search parameters for the event type. Provided inline. **Example** `<URL HOST>`?searchParamKey1=searchParamValue1

Parameter Name	Parameter Type	Description
`type`	String	The type of the event that was triggered. There are two groups of event types: Local Events and Central Events group. Only event types that belong to one of the groups are returned as the result of a single API call. The event types group you get depends on the scope you used to access the API. The examples of some of the events for both groups: • Central Events group: GlobalAccount_Update, AccountDirectory_Creation, AccountDirectory_Update, AccountDirectory_Update_Type, AccountDirectory_Deletion, Subaccount_Creation, Subaccount_Deletion, Subaccount_Update, Subaccount_Move, AccountDirectoryTenant_Creation, AccountDirectoryTenant_Deletion, GlobalAccountEntitlements_Update, EntityEntitlements_Update, EntityEntitlements_Move • Local Events group: SubaccountAppSubscription_Creation, SubaccountAppSubscription_Deletion, SubaccountAppSubscription_Update, AppRegistration_Creation, AppRegistration_Deletion, AppRegistration_Update, SubaccountTenant_Creation, SubaccountTenant_Update, SubaccountTenant_Deletion, EnvironmentInstance_Creation, EnvironmentInstance_Deletion, EnvironmentInstances_Deletion

Response Example

```
{
    "SubaccountAppSubscription_Update": {
        "searchParams": [
```

```
            "saasApplicationId",
            "saasApplicationName",
            "consumerTenantId"
        ],
        "description": "A subaccount's subscription to a multitenant
application, including its dependencies, was updated.",
        "category": "LOCAL",
        "type": "SubaccountAppSubscription_Update"
    },
    "SubaccountAppSubscription_Creation": {
        "searchParams": [
            "saasApplicationId",
            "saasApplicationName",
            "consumerTenantId"
        ],
        "description": "A subaccount was subscribed to a multitenant application
and its dependencies.",
        "category": "LOCAL",
        "type": "SubaccountAppSubscription_Creation"
    },
    "AppRegistration_Creation": {
        "searchParams": [
            "saasApplicationId",
            "saasApplicationName"
        ],
        "description": "A multitenant application was registered in the SaaS
registry",
        "category": "LOCAL",
        "type": "AppRegistration_Creation"
    },
    "SubaccountTenant_Creation": {
        "searchParams": [],
        "description": "The tenant of a subaccount was created.",
        "category": "LOCAL",
        "type": "SubaccountTenant_Creation"
    },
    "EnvironmentInstance_Creation": {
        "searchParams": [],
        "description": "An environment instance was created. For example, a
Cloud Foundry org was enabled for a subaccount.",
        "category": "LOCAL",
        "type": "EnvironmentInstance_Creation"
    },
    "AppRegistration_Update": {
        "searchParams": [
            "saasApplicationId",
            "saasApplicationName"
        ],
        "description": "The registration of a multitenant application in the
SaaS registry was changed.",
        "category": "LOCAL",
        "type": "AppRegistration_Update"
    },
    "AppRegistration_Deletion": {
        "searchParams": [
            "saasApplicationId",
            "saasApplicationName"
        ],
        "description": "A multitenant application was unregistered from the SaaS
registry.",
        "category": "LOCAL",
        "type": "AppRegistration_Deletion"
    },
    "EnvironmentInstances_Deletion": {
        "searchParams": [],
        "description": "All environment instances in a subaccount were deleted.",
        "category": "LOCAL",
        "type": "EnvironmentInstances_Deletion"
```

```
    },
    "SubaccountAppSubscription_Deletion": {
        "searchParams": [
            "saasApplicationId",
            "saasApplicationName",
            "consumerTenantId"
        ],
        "description": "The subscription of a subaccount to a multitenant
application including its dependencies was cancelled.",
        "category": "LOCAL",
        "type": "SubaccountAppSubscription_Deletion"
    },
    "SubaccountTenant_Deletion": {
        "searchParams": [],
        "description": "The tenant of a subaccount was deleted.",
        "category": "LOCAL",
        "type": "SubaccountTenant_Deletion"
    },
    "SubaccountTenant_Update": {
        "searchParams": [],
        "description": "The tenant of a subaccount was changed.",
        "category": "LOCAL",
        "type": "SubaccountTenant_Update"
    },
    "EnvironmentInstance_Deletion": {
        "searchParams": [
            "environmentType",
            "tenantId",
            "subaccountGuid",
            "platformId"
        ],
        "description": "An environment instance was deleted. For example, a
Cloud Foundry org, including its contents, was removed from a subaccount.",
        "category": "LOCAL",
        "type": "EnvironmentInstance_Deletion"
    }
}
```

Related Information

Getting an Access Token for SAP Cloud Management Service APIs [page 2731]
Using the Events Service APIs [page 2739]
Get Events [page 2740]

6.1.5.1.3 Error Response Format

Describes the response format for the errors from the SAP Cloud Management service for SAP BTP.

Standard HTTP codes are used to indicate successful execution or if errors occur. The response includes additional information about the error, including an application error code and human readable error description. This example represents a common error response resulting from a failed API request.

```
HTTP/1.1 400 Bad Request
{
  "error": {
    "code": 11000,
```

```
    "message": "Request payload is invalid",
    "target": "/accounts/v1/subaccounts/125e2778-e457-4c24-9db8-e3e10a3a0ed5",
    "details": [
      {
        "code": 11001,
        "message": "description: length must be between 0 and 255"
      }
    ]
  }
}
```

Parameter	Description	Type
`error.code`	A unique application error code. Common values are listed below.	Number
`error.message`	A description of the error.	String
`error.target`	The resource URL of the request.	String
`error.details`	An array of additional error codes and descriptions with further details about the error and its root cause. If this field exists, the array will contain at least one element.	Array

An application error code is not necessarily coupled with a specific HTTP status code.

Error Codes

Provides a list of error codes and the details of the errors.

Error Code	Description
10XXX	Server errors
11XXX	General request errors
12XXX	General operations failures
2XXXX	Accounts service errors
3XXXX	Entitlements service errors
40XXX	Tenants operations failures
41XXX	Environment instances operations failures
42XXX	Provisioning service errors
5XXXX	Service Management service proxy errors
6XXXX	Order Processing service errors
7XXXX	CLI backend errors
8XXXX	Asynchronous jobs failures
9XXXX	SaaS Provisioning service errors
10XXXX	Events service errors

6.1.5.1.4 Asynchronous Jobs

Describes the asynchronous calls that are supported by some functionality from the SAP Cloud Management service for SAP BTP.

When the API responds with the status code *202*, the request can be processed as an asynchronous job.

The status of the request can be fetched using the path provided in the *Location* header. The response of the API provided in the *Location* header is normally:

```
{
  "status": "<IN_PROGRESS/COMPLETED/FAILED>",
  "description": "<meaningful description in case one needed>"
}
```

> ### Note
>
> The response includes the `status` (always) and `description` (optional) fields.

For example, you can trigger a brand new job.

6.1.5.1.5 Rate Limiting

Describes how all API requests to the SAP Cloud Management service for SAP BTP adhere to rate limiting rules.

Every API endpoint defines its own custom rate limit rule for the number of requests per time window and per identified caller.

Authenticated requests are associated either with the authenticated username, tenant ID or with the OAuth client ID. Unauthenticated requests are associated with the originating IP address, and not the user making requests.

When the rate limit is exceeded, the client receives the HTTP *429* Too Many Requests response status code.

Response Example

For example, if you try to perform more than five GET requests to the `/accounts/v1/globalAccounts` endpoint in the `accounts-service` in less than a three second time window, you will receive the following error:

```
HTTP/1.1 429 Too Many Requests
{
  "error": {
    "code": 11006,
    "message": "Request rate limit exceeded.",
    "target": "/accounts/v1/globalAccounts",
    "details": [
      {
        "code": 11007,
        "message": "Maximum request rate limit is <X> requests in <Y> seconds"
```

```
      }
    ]
  }
}
```

> **Note**
>
> - The error code for the rate limit exceeded error is always *11006* and the details contains the list of exceeded rate limit rules with the *11007* error code.
> - No HTTP headers are returned to show your current rate limit status (for example, the number of requests remaining in the current rate limit window).

6.1.5.1.6 Auditing and Logging Information

Here you can find a list of the audit log events that are logged by the SAP Cloud Management service.

Audit log events written in audit logs

Event grouping	What audit log events are logged	How to identify related audit log events	Additional information
Events related to the management of global accounts and the creation and management of directories and subaccounts (`Accounts` API)	Update of a global account	The message contains the texts: • `\"type\":\"UPDATE\"` • `\"entity\":\"GlobalAccount`	To retrieve audit logs for a global account, use the audit log retrieval API. For more information, see Audit Log Retrieval API for Global Accounts in the Cloud Foundry Environment [page 2886].
	Creation of a directory	The message contains the texts: • `\"type\":\"CREATE\"` • `\"entity\":\"Directory`	
	Update of a directory	The message contains the texts: • `\"type\":\"UPDATE\"` • `\"entity\":\"Directory`	

Event grouping	What audit log events are logged	How to identify related audit log events	Additional information
	Deletion of a directory	The message contains the texts: • `\"type\":\"DELETE\"` • `\"entity\":\"Directory`	
	Creation of a subaccount	The message contains the texts: • `\"type\":\"CREATE\"` • `\"entity\":\"Subaccount`	
	Update of a subaccount	The message contains the texts: • `\"type\":\"UPDATE\"` • `\"entity\":\"Subaccount`	
	Deletion of a subaccount	The message contains the texts: • `\"type\":\"DELETE\"` • `\"entity\":\"Subaccount`	
Events related to the creation and management of environment instances (`Provisioning` API)	Creation of an environment instance (for example, Cloud Foundry)	The message contains the texts: • `\"type\":\"CREATE\"` • The `\"entity\"` attribute contains the text `.ServiceInstance\"`	

Event grouping	What audit log events are logged	How to identify related audit log events	Additional information
	Update of an environment instance	The message contains the texts: • `\"type\":\"UPDATE\"` • The `\"entity\"` attribute contains the text `.ServiceInstance\"`	
	Deletion of an environment instance	The message contains the texts: • `\"type\":\"DELETE\"` • The `\"entity\"` attribute contains the text `.ServiceInstance\"`	
Events related to the management of product entitlements and assignments across your global account, directories, and subaccounts (`Entitlements` API)	Assignment of a service plan to a subaccount	The message contains the texts: • `\"type\":\"CREATE\"` or `\"type\":\"UPDATE\"` • The `"entityType"` attribute contains the text `ServicePlanAssignment`. • The `"entityId"` attribute contains the subaccount GUID.	To retrieve audit logs for a global account, use the audit log retrieval API. For more information. see Audit Log Retrieval API for Global Accounts in the Cloud Foundry Environment [page 2886].

Event grouping	What audit log events are logged	How to identify related audit log events	Additional information
	Removal of a service plan from a subaccount	The message contains the texts: • `\"type\":\"DELETE\"` • The `"entityType"` attribute contains the text `ServicePlanAssignment`. • The `"entityId"` attribute contains the subaccount GUID.	
	Assignment of an entitlement to a directory	The message contains the texts: • `\"type\":\"CREATE\"` or`\"type\":\"UPDATE\"` • The `"entityType"` attribute contains the text `AssignedEntitlement`. • The `"entity_Id"` attribute contains the directory GUID in the `"entityGUID"` field and `"entityType":"DIRECTORY"`.	

Event grouping	What audit log events are logged	How to identify related audit log events	Additional information
	Update of an assignment of an entitlement to a directory	The message contains the texts: • `\"type\":\"UPDATE\"` • `"entityType"` attribute contains the text `AssignedEntitlement`. • The `"entityGUID"` attribute contains the directory GUID and updated quota amounts.	
	Removal of an entitlement from a directory	The message contains the texts: • \"type\":\"DELETE\" • The `"entityType"` attribute contains the text `AssignedEntitlement`. • The `"entityGUID"` attribute contains the directory GUID and updated quota amounts.	
Event related to subscriptions of consumers to multitenant applications in the Cloud Foundry environment. The multitenant applications must be registered to the SaaS Provisioning Service. See Register the Multitenant Application to the SAP SaaS Provisioning Service [page 413]. (SaaS Provisioning Service, technical name: `saas-registry`)	Creation of a subscription to an application for a subaccount	The message contains the texts: • `\"type\":\"CREATE\"` • `\"entity\":\"RootSubscription` • The `"entity"` attribute contains the subaccount GUID in `"subaccountId"` field, the app name in the `"appName"` field , and subscription state details, including `"state"` and `"code"` (plan).	To retrieve audit logs for subscriptions of applications to subaccounts, use the audit log retrieval API. For more information, see Audit Log Retrieval API Usage for Subaccounts in the Cloud Foundry Environment [page 2880].

Event grouping	What audit log events are logged	How to identify related audit log events	Additional information
	Update of a subscription (when parameters or a plan have been changed)	• `\"type\":\"UPDATE\"` • `\"entity\":\"RootSubscription` • The `"entity"` attribute contains the subaccount GUID in `"subaccountId"` field, the app name in the `"appName"` field , and subscription state details, including `"state"` and `"code"` (plan).	
	Deletion of a subscription	The message contains the texts: • `\"type\":\"UPDATE\"` • `\"entity\":\"RootSubscription` • The `"entity"` attribute contains the subaccount GUID in `"subaccountId"` field, the app name in the `"appName"` field , and subscription state details, including `"state"` and `"code"` (plan).	
Events related to the management of budgets in your global account. See Managing Budgets in Your Global Account [page 2490]. (`Budgets` service)	Creation of a budget	The message contains the texts: • `\"type\":\"CREATE\"` • `\"entity\":\"Budget`	To retrieve audit logs for a global account, use the audit log retrieval API. For more information, see Audit Log Retrieval API for Global Accounts in the Cloud Foundry Environment [page 2886].

Event grouping	What audit log events are logged	How to identify related audit log events	Additional information
	Update of a budget	The message contains the texts: • `\"type\":\"UPDATE\"` • `\"entity\":\"Budget`	
	Deletion of a budget	The message contains the texts: • `\"type\":\"DELETE\"` • `\"entity\":\"Budget`	

The following information is described in the table columns:

- *Event grouping* - Events that are logged with a similar format or are related to the same entities.
- *What audit events are logged* - Description of the security or data protection and privacy related event that is logged.
- *How to identify related log events* - Search criteria or key words, that are specific for a log event that is created along with the logged event.
- *Additional information* - Any related information that can be helpful.

Related Information

Audit Logging in the Cloud Foundry Environment

6.1.5.2 Monitoring Usage Information Using APIs of the SAP Usage Data Management Service

Provides information about using the *Resource Consumption* APIs of the SAP Usage Data Management service for SAP BTP for gathering, storing, and making usage information available for all services and applications in all regions in a cloud deployment. This information is for the purpose of central analysis, reporting, and license auditing.

The service accumulates the information and provides reports in SAP BTP cockpit via APIs for resource planning and cross-billing purposes.

> ### Note
>
> The SAP Usage Data Management service is not sold to SAP BTP customers as a service; however, customers can see their usage data in the cockpit, for example, in the Usage Analytics pages.

Accessing the API

For more information about the APIs, including their specifications and endpoints, go to Resource Consumption for the list of all SAP Usage Data Management service APIs in the SAP Business Accelerator Hub.

> ### → Remember
>
> To call the core platform API methods, you must obtain an access token. See Configuration for Using the Resource Consumption APIs in the SAP Business Accelerator Hub [page 2768].

Base URI: `https://uas-reporting.cfapps.<landscape domain>/reports/v1`

Your global account admin has entitled the service plan for the SAP Usage Data Management service in your subaccount. See SAP Usage Data Management Service - Service Plans [page 2771].

Methods

HTTP Method	Action	URI
GET	Get the cloud credit data history for a global account.	`/cloudCreditsDetails`
GET	Get monthly usage reporting data for a directory and a specified time period.	`/monthlyDirectoryUsage`
GET	Get monthly usage-reporting data for a global account and a specified time period.	`/monthlyUsage`
GET	Get usage-reporting data for a subaccount.	`/subaccountUsage`
GET	Get monthly cost reporting data for all subaccounts for the Cloud Platform Enterprise Agreement (CPEA) consumption-based commercial model.	`/monthlySubaccountsCost`

Related Information

Configuration for Using the Resource Consumption APIs in the SAP Business Accelerator Hub [page 2768]
SAP Usage Data Management Service - Service Plans [page 2771]

6.1.5.2.1 Configuration for Using the Resource Consumption APIs in the SAP Business Accelerator Hub

The **Resource Consumption** APIs of the SAP Usage Data Management service for SAP BTP are protected with OAuth 2.0 Client Credentials grant type and in some cases, also the Password grant type.

> ⓘ Note
>
> OAuth 2.0 Password grant type is only mandatory for the `subaccountUsage` API.

Prerequisites

- To use the *Resource Consumption* APIs of the SAP Usage Data Management service
  - You must have service access to the relevant **uas** service plan. For information about available service plans, see SAP Usage Data Management Service - Service Plans [page 2771].
  - When using **Password grant type** the user must be assigned one of the following role collections:

API	Role Collection Required
`monthlyUsage`, `/monthlySubaccountsCost`, and `subaccountUsage`	Global Account Administrator and Global Account Viewer

    For more information, see Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138].

> ⓘ Note
>
> To create a service instance in other environments using the Service Manager Control (SMCTL) CLI, SAP Service Manager APIs, or the SAP BTP cockpit, see Consuming Services in Other Environments Using the SAP Service Manager Instances.

Procedure

1. Create a service instance for the service plans that correspond to the APIs that you want to use.
   When you create the service instance, specify the following parameters:
   - `PLAN:` The name of the service plan that you want to use. See SAP Usage Data Management Service - Service Plans [page 2771].
   - `SERVICE_INSTANCE:` Name of the service instance.

   There are several options available to create instances depending on the environment you use.
   - To create a service instance in other environments using the Service Manager Control (SMCTL) CLI or SAP Service Manager APIs, see Consuming Services in Other Environments Using the SAP Service Manager Instances.

- To create a service instance in the Cloud Foundry environment using the SAP BTP cockpit or the cf CLI, see Creating Service Instances [page 273].
- To create a service instance in Kyma using the Kyma dashboard, see Using SAP BTP Services in the Kyma Environment [page 2267].

> → Recommendation
>
> If you are not working in Cloud Foundry, Kyma, or Kubernetes, use the SAP Service Manager to create and manage service instances. These instances are platform-agnostic and can be deployed and integrated with any other environment of your choice.

2. Get the OAuth 2.0 client information and `uas` service endpoints by creating a service key or binding. There are several options available to create service keys or bindings depending on the environment you use.
   - To create bindings in other environments, see Consuming Services in Other Environments Using the SAP Service Manager Instances.
   - To create a service key in the Cloud Foundry environment using the SAP BTP cockpit or the cf CLI, see Creating Service Keys [page 278].
   - To create credentials for calling the service and retrieving information in the Kyma environment, see Using SAP BTP Services in the Kyma Environment [page 2267].
3. Use `uaa_url`, `clientid`, and `clientsecret` to request an access token using the following commands:

> {·} Sample Code
>
> For Windows OS (Password grant type):

```
curl -L -X POST "<uaa_url>/oauth/token" ^
-H "Content-Type: application/x-www-form-urlencoded" ^
-u "<clientid>:<clientsecret>" ^
-d "grant_type=password" ^
-d "username=<user email>" ^
-d "password=<password>"
```

> {·} Sample Code
>
> For Windows OS (Client Credentials grant type):

```
curl -L -X POST "<uaa_url>/oauth/token" ^
-H "Content-Type: application/x-www-form-urlencoded" ^
-u "<clientid>:<clientsecret>" ^
-d "grant_type=client_credentials" ^
```

> {·} Sample Code
>
> For Mac OS (Password grant type):

```
curl -L -X POST '<uaa_url>/oauth/token' \
-H 'Content-Type: application/x-www-form-urlencoded' \
-u '<clientid>:<clientsecret>' \
-d 'grant_type=password' \
-d 'username=<user email>' \
-d 'password=<password>'
```

> ### Sample Code
>
> For Mac OS (Client Credentials grant type):
>
> ```
> curl -L -X POST '<uaa_url>/oauth/token' \
> -H 'Content-Type: application/x-www-form-urlencoded' \
> -u '<clientid>:<clientsecret>' \
> -d 'grant_type=client_credentials' \
> ```

> ### Note
>
> The access token that you receive also contains the scopes that are granted for this access token. Therefore, only APIs that require one of these scopes can be used with this access token.
>
> ```
> {
>   "access_token": "<access_token>",
>   "token_type": "bearer",
>   "expires_in": 43199,
>   "scope": "<xsappname>.UAS.reporting.GA_Admin"
> }
> ```

4. Using the `<access_token>` that you received in the previous step you can call the Resource Consumption APIs using bearer authentication or you can try them directly in the SAP Business Accelerator Hub using the following steps:
   1. Got to Resource Consumption APIs in the SAP Business Accelerator Hub.
   2. Choose *Select Environment* and select the *Add New Environment* option
   3. In the *Authentication Type* field, select *OAuth 2.0 Application Flow*.
   4. Provide the parameters that you have retrieved in the following fields: `clientid`, `clientsecret`, and `identityzone`.
   5. Save your changes.
5. Choose the API that you want to call, provide the relevant parameters, and choose *Run*.

To learn more about the SAP Business Accelerator Hub, see What Is the SAP API Business Hub?

6.1.5.2.2 Rate Limiting Rules

API requests to the SAP Usage Data Management service for SAP BTP are subject to rate limits.

Rate limits restrict the number of requests made by a caller within a defined time period. For authenticated requests, callers are identified by the authenticated user name, the tenant ID or the OAuth client ID. For non-authenticated requests, callers are identified by the originating IP address.

When you exceed the rate limit, you receive the HTTP response status code *429 Too Many Requests*.

The rate limits are:

- The rate limit per authentication token or IP is 120 requests per second.
- The maximum payload body size per request is 80 MB.

Example

For example, if you try to perform more than 120 GET requests to the `/reports/v1/monthlyUsage` endpoint in less than 1 second, you receive the following error:

```
<html>
<head><title>429 Too Many Requests</title></head>
<body>
<center><h1>429 Too Many Requests</h1></center>
<hr><center>openresty</center>
</body>
</html>
```

6.1.5.2.3 SAP Usage Data Management Service - Service Plans

Describes the plans available for the SAP Usage Data Management service for SAP BTP.

Note

The SAP Usage Data Management service APIs can be found in the SAP Business Accelerator Hub.

Display Name	Technical Service Name	Service Plans	Scopes
Usage Data Management Service (UDM)	`uas`	`reporting-ga-admin`: Service plan for using the SAP Usage Data Management service APIs. This microservice is used to generate reports based on resource and cost consumption of a global account.	`Reporting.GA_Admin`

Display Name	Technical Service Name	Service Plans	Scopes
		`reporting-directory`: Service plan for using the SAP Usage Data Management service APIs. This microservice is used to generate reports based on resource and cost consumption of a directory. When you create a service instance of this plan, you have to pass the directory ID in the following format: • `{"directoryGuid": "f4c920c1-efb7-4bda-8d0e-81616ab9a574"}` - if you create the service instance using the SAP BTP cockpit • `-c "{\"directoryGuid\": \"f4c920c1-efb7-4bda-8d0e-81616ab9a574\"}"` - if you create the service instance using the cf CLI **⚠ Restriction** The directory must be located under `globalAccount` and contain the subaccount from which you are creating the service instance. User management must be enabled for the directory, see Manage Users in Directories [page 2505].	`$XSAPPNAME.reporting.Directory_Admin`

Related Information

Configuration for Using the Resource Consumption APIs in the SAP Business Accelerator Hub [page 2768]

6.1.5.3 Accessing Administration Using APIs of the SAP Authorization and Trust Management Service

The REST services of the SAP Authorization and Trust Management service (XSUAA) provide APIs that enable you to manage entities, such as roles, shadow users, and access tokens in global accounts, directories, and multi-environment subaccounts.

> **Note**
>
> To manage Neo subaccounts, use the Neo APIs. For more information, see Using Platform APIs.

API Overview

The following table provides an overview of the APIs. For more information about the APIs and their API endpoints, see *SAP Business Accelerator Hub* in the related links.

APIs of SAP Authorization and Trust Management Service

API	Description
User Management (System for Cross-domain Identity Management (SCIM))	Provides functions to administrate the SAP Authorization and Trust Management service of the Cloud Foundry environment. Provision users from identity providers and manage roles and role collections. Use this API to manage shadow users; users the service provisions from your identity provider to the subaccount. See SAP BTP XS Advanced UAA (Cloud Foundry) for more information.
Authorization	Provides functions to administrate the SAP Authorization and Trust Management service of Cloud Foundry environment. Manage service instances of the SAP Authorization and Trust Management service. You can also manage roles, role templates, and role collections of your subaccount.
Identity Provider Management	Provides functions to manage identity providers in in the Cloud Foundry environment.
Security Settings	Provides functions to manage the security settings of an account in Cloud Foundry environment. Use this API to manage access token validity and signing keys.

Related Information

APIs of SAP Authorization and Trust Management Service service on SAP Business Accelerator Hub
Using Platform APIs
Rate Limiting [page 3600]
Limits for Technical Artifacts of the SAP Authorization and Trust Management Service [page 3605]

6.1.5.3.1 Get Access to the APIs

To enable programmatic access to the SAP Authorization and Trust Management service (XSUAA) in your global account, directory, or multi-environment subaccount, you need API credentials.

Context

To get access to the APIs of the SAP Authorization and Trust Management service, use the SAP BTP command-line interface (btp CLI) to create API credentials. You can get API credentials that apply for the global account, directory, or subaccount.

For more information, see security/api-credential.

Procedure

1. Log on to the SAP BTP CLI, targeting the relevant global account, directory, or subaccount.
2. Enter the following command:

   ```
   btp create security/api-credential --name my-credential
   ```

   You get API credentials, which enable you to access the REST APIs of the SAP Authorization and Trust Management service on Business Accelerator Hub.

   The btp CLI shows the API credential as follows:

   Output Code

   ```
   Name: my-credential
   Credential Type: binding-secret
   Read-only: false
   Client Secret: <secret>
   Certificate: <null>
   Client ID: sb-full-access!a76125
   ```

   With the client ID and client secret, or certificate, you can request an access token for the APIs in the targeted global account, directory, or subaccount.

Related Information

Managing API Credentials for Calling REST APIs of SAP Authorization and Trust Management Service [page 2714]
btp CLI Command Reference

6.1.5.3.2 Call an API

Use the different endpoints of the SAP Authorization and Trust Management service APIs to manage users roles and other authorization configurations.

Context

Actually, you need two separate URLs. First, you need the token URL to fetch an access token. Next, you can use the API URL to call an API.

To call an API of the SAP Authorization and Trust Management service, you need an access token. See the following example how to retrieve an access token and how to call an endpoint with a cURL request.

On the *SAP Business Accelerator Hub*, we provide a sandbox system for you to make example calls to the different APIs and their endpoints.

Use the following API URLs:

`https://api.authentication.<region>.hana.ondemand.com`

> ⓘ Note
>
> To find the region, use a btp CLI command, for example **`btp get accounts/subaccount subaccount-id`**.

Procedure

1. Configure or program your application to request a token from the OAuth authorization server.

   In your call to the OAuth server, you send the client ID and client secret to the token URL appended with the `/oauth/token` endpoint. To illustrate the call, we use cURL in the following example.

   > {·} Sample Code
   >
   ```
   curl --request POST \
     --url 'https://my-subdomain.authentication.eu10.hana.ondemand.com/oauth/
   token' \
     --header 'Accept: application/json' \
     --data 'client_id=sb-full-access!a76125' \
     --data 'client_secret=<secret>' \
   ```

```
  --data 'grant_type=client_credentials' \
  --data 'response_type=token'
```

The authorization server returns a token along with other related information.

Output Code

```
{"access_token":"eyJhbGciOiJSUzI1N...",
"token_type":"bearer",
"expires_in":43199,
"scope":"xs_user.write uaa.resource xs_authorization.read
 xs_idp.write xs_user.read xs_idp.read xs_authorization.write",
"jti":"be340353ac694b4cb504c6823f938647"}
```

2. To retrieve the token, use the following URL:

   `https://<subdomain>.authentication.<region>.hana.ondemand.com`

   Note

   To find the subdomain of the global account, directory, and subaccount, go to the SAP BTP cockpit and display the overview in the *Account Explorer*.

   When you have a token, you can use this token to call the APIs until its validity expires. In this case, simply get a new access token.
3. Use the value of the `access_token` property to make calls to the various API endpoints.
4. Call the endpoint of an API.

   Sample Code

   In this example, we request the list of roles of from the authorization API. With the authorization header, use the access token you received in the prerequisites after the key word `bearer`.

```
curl --request GET \
  --url https://api.authentication.eu10.hana.ondemand.com/sap/rest/
authorization/v2/roles \
  --header 'Accept: application/json' \
  --header 'Authorization: bearer eyJhbGciOiJSUzI1N...'
```

Results

The API returns the list of roles.

Output Code

```
[
    {
        "roleTemplateName": "Viewer",
        "roleTemplateAppId": "myapp!t1111",
        "name": "Viewer",
        "attributeList": [],
        "roleCapabilityIDList": [],
        "description": "Default instance",
```

```
        "scopes": [
            {
                "description": "Display forecast",
                "name": "myapp!t1111.Tourist",
                "custom-granted-apps": [],
                "granted-apps": [],
                "grant-as-authority-to-apps": [],
                "custom-grant-as-authority-to-apps": []
            }
        ]
    }
]
```

Related Information

Authorization and Trust Management APIs on the SAP Business Accelerator Hub

Implementing Custom Token Retrieval from SAP Authorization and Trust Management Service with mTLS [page 573]

6.1.5.4 Managing Service Resources Using the APIs of the SAP Service Manager

Use the APIs of the SAP Service Manager to work with environments, service brokers, service instances, service bindings, service plans, and service offerings.

SAP Service Manager is the central registry for service brokers and platforms in SAP BTP.

The service allows you to consume platform capabilities in any connected runtime environment, track the creation and managing of service instances, and share services and service instances between different environments.

For the list of the available APIs, see Working with SAP Service Manager APIs.

6.1.6 Account Administration Using Infrastructure as Code

Infrastructure as Code (IaC) automates infrastructure provisioning and management using code. It helps ensure consistency, scalability, versioning, collaboration, and documentation. The Terraform Provider for SAP BTP allows you to provision and manage SAP BTP resources as code, with open-source support.

What is Infrastructure as Code?

Infrastructure as Code (IaC) is an approach to managing and provisioning infrastructure resources as code, using machine-readable configuration files or scripts, instead of manually configuring them. Infrastructure

configurations are written in a programming language and usually stored in version control systems, so they can easily be shared, reviewed, and modified, enabling collaboration and reproducibility.

The following are some benefits of the IaC approach:

- **Automation**: It allows for the automation of infrastructure provisioning and management processes. Infrastructure can be created, modified, and destroyed programmatically, reducing the need for manual intervention and minimizing human error.
- **Consistency**: By defining infrastructure as code, you can ensure that your infrastructure is consistently provisioned and configured across different environments, such as development, testing, and production. This helps in reducing configuration drift and improves reliability.
- **Scalability**: IaC enables the ability to scale infrastructure resources up or down based on demand. By defining infrastructure configurations in code, you can easily replicate and deploy infrastructure resources in a consistent and repeatable manner.
- **Versioning and Collaboration**: IaC configurations can be stored in version control systems, allowing you to track changes over time and roll back to previous versions if needed. It also enables collaboration among team members, as multiple people can work on the same infrastructure codebase simultaneously.
- **Documentation**: IaC serves as a form of documentation, providing a clear and concise representation of the desired infrastructure state. It helps in understanding the infrastructure architecture and facilitates knowledge sharing within the team.

Overall, Infrastructure as Code brings the principles and practices of software development to infrastructure management, with the goal of enabling organizations to achieve greater agility, scalability, and reliability in their infrastructure operations.

Infrastructure as Code with the Terraform Provider for SAP BTP

There is an open-source solution for using IaC for SAP BTP: The **Terraform Provider for SAP BTP**. It focuses on administration capabilities within SAP BTP global accounts. For example, it lets you manage subaccounts and directories, entitlements, environment instances, subscriptions, service instances and bindings, users and role collections, and trust configurations to custom identity providers.

Where to Find the Terraform Provider forSAP BTP

- You find the Terraform Provider for SAP BTP in Hashicorp's Terraform registry, the central provider repository: https://registry.terraform.io/providers/SAP/btp .
- You find the source code and documentation in this GitHub repository: https://github.com/SAP/terraform-provider-btp

Support

The Terraform Provider for SAP BTP was built by SAP as an open-source project under the Apache 2.0 license . Note that this means that there is no official SAP support for the provider. Feedback, issues, pull requests, and feature requests are handled through the open repository on github.com .

Documentation

For a complete overview of the scope of the Terraform Provider for SAP BTP, see the documentation in Hashicorp's Terraform Registry: https://registry.terraform.io/providers/SAP/btp/latest/docs

To learn how to get started, see the tutorial: Get Started with the Terraform Provider for SAP BTP

To check out different use cases and options, look at our samples: Repository with Samples for the Terraform Provider for SAP BTP .

For blog posts in SAP Community, search for "infrastructure as code" in the Technology Blogs by SAP.

6.2 Administration and Operations in the Cloud Foundry Environment

Learn about the different account administration and application operation tasks which you can perform in the Cloud Foundry environment.

- Account Administration Using the SAP BTP Command Line Interface (btp CLI) [page 2664]
- Account Administration in the Cockpit [page 2458]
- Account Administration Using APIs of the SAP Cloud Management Service [page 2730]
- About Roles in the Cloud Foundry Environment [page 2788]
- Org Administration Using the Cockpit [page 2792]
- Org Management Using the SAP BTP Command Line Interface (BTP CLI) [page 2780]
- Org Administration Using the Cloud Foundry CLI [page 2812]
- Application Operations in the Cloud Foundry Environment [page 2860]
- Log on with a Browser to the Cloud Foundry CLI and Service Dashboards [page 2592]
- Accessing Administration Using APIs of the SAP Authorization and Trust Management Service [page 2773]
- Audit Logging in the Cloud Foundry Environment [page 2878]

Related Information

Cloud Foundry Environment [page 63]
Getting Started in the Cloud Foundry Environment [page 165]
Development in the Cloud Foundry Environment [page 256]
Tutorial Navigator: Learn How to Manage Your Cloud Foundry Environment in the SAP BTP Cockpit

6.2.1 Org Management Using the SAP BTP Command Line Interface (BTP CLI)

The Cloud Foundry environment allows you to create polyglot cloud applications in Cloud Foundry. To manage the lifecycle of an org in the Cloud Foundry environment, use the `accounts/environment-instance` command in the BTP CLI.

Available Plans

To create a Cloud Foundry org in a subaccount, you use the Cloud Foundry Runtime service. The Cloud Foundry Runtime service offers the following plans for this purpose:

Plan Name	Description
`standard`	This is an enterprise-grade plan that allows you to create an org in your Cloud Foundry environment to start developing polyglot cloud-native applications. By default, a Cloud Foundry org that is created with this plan does not have any application runtime. To assign application runtime, the global account admin must assign the Cloud Foundry Runtime service with the `MEMORY` plan to the subaccount so that the org has memory for its applications.
`free`	Use this plan to try out the Cloud Foundry environment without an additional charge before switching to the `standard` plan that supports enterprise-grade productivity. By default, a Cloud Foundry org that is created with this plan is given a fixed application-runtime quota. There is no need to assign the Cloud Foundry Runtime service with the `MEMORY` plan to the subaccount as it will have no effect on the org quota. For more information about the free tier model for SAP BTP and its availability, see Using Free Service Plans [page 118].
`trial`	Use this plan to utilize your trial period. **Note** Your subaccount's type must be trial to use this plan.

For more information, about the features that each plan offers, see Cloud Foundry Runtime on SAP Discovery Center.

Command Reference for CRUD Operations

The btp CLI offers the following environment instance command actions to manage Cloud Foundry orgs in SAP BTP:

Action	Command	Description	Parameters	Additional Info
`list`	`btp list accounts/environment-instance --subaccount <SUBACCOUNT_ID>`	List all the Cloud Foundry orgs and other environment instances in a subaccount.	`SUBACCOUNT_ID`: The ID of the subaccount.	
`get`	`btp get accounts/environment-instance <ID> --subaccount <SUBACCOUNT_ID>`	Get the details of a specific Cloud Foundry org in a subaccount.	`ID`: The ID of the environment instance to view. `SUBACCOUNT_ID`: The ID of the subaccount.	

Action	Command	Description	Parameters	Additional Info
`create`	`btp create accounts/environment-instance --subaccount <SUBACCOUNT_ID> --display-name <DISPLAY_NAME> --service <SERVICE> --plan <PLAN> --environment cloudfoundry --parameters "{\"instance_name\":\"<ORG_NAME>\"}"` **Note** If the logged-in region has multiple landscapes, then you must specify the name of the landscape using the `--landscape` parameter in the command. You can obtain the valid values using the `btp list accounts/available-environment` command.	Create a Cloud Foundry org in a subaccount.	`SUBACCOUNT_ID`: The ID of the subaccount in which to create the Cloud Foundry org. `DISPLAY_NAME`: The SAP BTP name of the environment instance. `PLAN`: Specify either `standard` or `free` as the value. `ORG_NAME`: The name of the org in the Cloud Foundry environment. Spaces are not allowed in the org name. Once the org is created, you cannot change its name.	For examples that show how to pass JSON parameters in the command line with different operating systems and shells, see the Passing JSON Parameters on the Command Line [page 2673].

Action	Command	Description	Parameters	Additional Info
`update`	`btp update accounts/environment-instance <ID> --subaccount <SUBACCOUNT_ID> --plan <PLAN>`	Update an existing Cloud Foundry org in a subaccount.	`ID`: The ID of the environment instance to update `SUBACCOUNT_ID`: The ID of the subaccount. `PLAN`: Specify either `standard` or `free` as the value.	For the Cloud Foundry environment, you can use this command to change only the plan of an existing Cloud Foundry environment instance. In other words, update an org created with the `free` plan of the Cloud Foundry Runtime to the `standard` plan (not vice versa). **Note** Before updating the plan to `standard`, make sure that the `MEMORY` plan for the Cloud Foundry Runtime entitlement is assigned to your subaccount, and that you've assigned sufficient application runtime quota to cover the current workload in the subaccount.
`delete`	`btp delete accounts/environment-instance <ID> --subaccount <SUBACCOUNT_ID>`	Delete a Cloud Foundry org created in a subaccount.	`ID`: The ID of the environment instance to delete. `SUBACCOUNT_ID`: The ID of the subaccount where the environment instance exists.	The Cloud Foundry org and all its data will be lost.

> ### Note
>
> You can obtain the subaccount ID by running `btp list accounts/subaccount`. You can also target the subaccount using `btp target --subaccount <ID>`, and then omit the `subaccount` parameter from the rest of the commands.

Adding and Removing Org Managers

You can also use the BTP CLI to add or remove org managers.

Action	Command	Description	Parameters	Additional Info

`update`	For the default identity provider:
	`btp update accounts/environment-instance <ID> --subaccount <SUBACCOUNT_ID> --plan <PLAN> --parameters "{\"usersToAdd\": [{\"id\":\"myUserID\",\"email\":\"name@example.com\"}]}"`
	For a custom identity provider:
	`btp update accounts/environment-instance <ID> --subaccount <SUBACCOUNT_ID> --plan <PLAN> --parameters "{\"usersToAdd\": [{\"id\":\"myUserID\",\"email\":\"name@example.com\",\"origin\":\"<your custom IDP>\"}]}"`
		Add one or more org managers to a Cloud Foundry org in a subaccount.
		`usersToAdd` adds the OrgManager role to the user, and, if it doesn't exist yet, it creates the user in the org.
			`ID`: The ID of the environment instance for which to add org managers.
			`SUBACCOUNT_ID`: The ID of the subaccount.
			`PLAN`: Specify either `standard` or `free` as the value.
				For examples that show how to pass JSON parameters in the command line with different operating systems and shells, see the Passing JSON Parameters on the Command Line [page 2673].

Action	Command	Description	Parameters	Additional Info

`update`	For the default identity provider:
	`btp update accounts/environment-instance <ID> --subaccount <SUBACCOUNT_ID> --plan <PLAN> --parameters "{\"usersToRemove\":[{\"id\":\"myUserID\",\"email\":\"name@example.com\"}]}"`
	For a custom identity provider:
	`btp update accounts/environment-instance <ID> --subaccount <SUBACCOUNT_ID> --plan <PLAN> --parameters "{\"usersToRemove\":[{\"id\":\"myUserID\",\"email\":\"name@example.com\",\"origin\":\"<your custom IDP>\"}]}"`
		Remove one or more org managers from a Cloud Foundry org in a subaccount.
		`usersToRemove` removes the OrgManager role from the user, but it doesn't remove the user from the org.
			`ID`: The ID of the environment instance for which to remove org managers.
			`SUBACCOUNT_ID`: The ID of the subaccount.
			`PLAN`: Specify either `standard` or `free` as the value.
				To learn how to remove a user from the Cloud Foundry org and the subaccount, see About User Management in the Cloud Foundry Environment [page 2787].

Related Information

Account Administration Using the SAP BTP Command Line Interface (btp CLI) [page 2664]
About Roles in the Cloud Foundry Environment [page 2788]
Org Administration Using the Cockpit [page 2792]

Add Org Members [page 2795]
Managing Spaces [page 2799]
About User Management in the Cloud Foundry Environment [page 2787]

6.2.2 About User Management in the Cloud Foundry Environment

The Cloud Foundry environment has its own store for user data within SAP BTP. Understanding the relationship between SAP BTP and the Cloud Foundry environment is useful.

All users of Cloud Foundry and SAP BTP are stored in identity providers. Both require a local copy of the users so that you can assign authorizations to those users. When a user tries to log on, the user is forwarded to the identity provider to authenticate. When you add a user as a member to an org or space, there's no check whether or not this user actually exists in the identity provider you specified.

Org members can only be added by an `Org Manager`. If you only have the `Space Manager` role, you can only add space members that are known to the org. To do that, you must first ask your `Org Manager` to add the users as org members with no roles.

> ⓘ Note
>
> All org members implicitly have the `Org User` role.

If you're the `Org Manager`, you already have the authorizations to add users to the org. When you add a new user as a space member, that user is added automatically as part of the org as well.

Cloud Foundry and Shadow Users

Whenever you add a user at the org or space level with the SAP BTP cockpit, SAP BTP also creates the user at the subaccount level, also known as a shadow user. When you use the SAP BTP cockpit to add a user, SAP BTP assigns at least one Cloud Foundry role to the user, namely *Org User* if you don't add any other role. SAP BTP checks for Cloud Foundry users with a role and creates corresponding shadow users at the subaccount level if no such user exists. Creating users with the Cloud Foundry CLI or Cloud Foundry APIs doesn't add any Cloud Foundry roles by default. Once you add a role to the user, the system ensures that there's a corresponding user at the subaccount level.

> → Remember
>
> When you delete a user at the Cloud Foundry level, SAP BTP doesn't delete the corresponding shadow user at the subaccount level.
>
> When you delete a user at the subaccount level, delete the user at the Cloud Foundry org and space levels, too. Otherwise, the system automatically creates the corresponding user at the subaccount level if the user has a Cloud Foundry role.

For more information about shadow users, see Working with Users [page 2603].

User Provisioning

For user management of Cloud Foundry organizations, use the provisioning connectors of SAP Cloud Identity Services - Identity Provisioning. For more information, see SAP BTP Integration Scenario.

> ⚠ Restriction
>
> If user provisioning through the Cloud Foundry environment is active in SAP Cloud Identity Services - Identity Provisioning, you can provision a maximum of 1000 platform users.

Related Information

User and Member Management [page 133]
About Roles in the Cloud Foundry Environment [page 2788]
Add Org Members [page 2795]
Add Organization Members Using the Cloud Foundry Command Line Interface [page 2853]
Add Space Members [page 2803]
Add Space Members Using the Cloud Foundry Command Line Interface [page 2855]

6.2.3 About Roles in the Cloud Foundry Environment

Roles determine which features users can view and access, and which actions they can initiate.

Cloud Foundry includes predefined roles that are specific to the navigation level in the SAP BTP cockpit; for example, the roles at the level of the organization differ from the ones for the space. Each role comes with a set of permissions. Roles apply to all operations that are associated with the organization or the space, irrespective of the tool used (Eclipse-based tools, SAP BTP cockpit, and cf CLI).

The following roles can be assigned to users in the Cloud Foundry environment on SAP BTP:

Level	Role	Role Description
Organization	Org Manager	Manages the organization. When you create an org, you get the Org Manager role in that org by default.
	Org Auditor	Provides read-only access to user information and org quota usage information.

Level	Role	Role Description
	Org User	Provides read-only access to the list of other organization users and their roles. **Note** When creating an org member, you can choose one or more of the following roles: • Org Manager • Org User • Org Auditor When creating a space member, they get the Org User role by default.
Space	Space Manager	Manages a space within an organization.
	Space Developer	Manages applications, services, and space-scoped service brokers in a space. **Caution** The Space Developer role has broad rights within Cloud Foundry and, in particular, has access to the credentials used in various services and app bindings as well as other sensitive data. For more information, see Giving Access Rights to Platform Users.
	Space Auditor	Provides read-only access to a space.
	Space Supporter	Allows you to troubleshoot and debug applications and service bindings in a space. **Note** To add the Space Supporter role using the `set-space-role` command, you need to have installed version 8 of the Cloud Foundry Command Line Interface (cf CLI). See https://docs.cloudfoundry.org/cf-cli/install-go-cli.html .

Note

Managing members and roles for an organization takes place on the navigation level of the subaccount to which the organization is assigned.

For more information about organization and space roles in Cloud Foundry, see https://docs.cloudfoundry.org/concepts/roles.html .

Related Information

About User Management in the Cloud Foundry Environment [page 2787]
Add Org Members [page 2795]
Add Organization Members Using the Cloud Foundry Command Line Interface [page 2853]
Add Space Members [page 2803]
Add Space Members Using the Cloud Foundry Command Line Interface [page 2855]

6.2.4 Migrating Cloud Foundry Environment Instance to SAP Build

Update your Cloud Foundry environment instance to migrate it to the `build-runtime` plan.

Prerequisites

- You're using an enterprise global account. For more information, see Enterprise Accounts [page 111].
- You're a subaccount administrator. For more information, see Find Users and Their Role Collection Assignments [page 2606].
- The following entitlement has been assigned to your subaccount:

Service	Technical Name	Service Plan
Cloud Foundry Environment	`cloudfoundry`	`build-runtime`

  For more information, see Upgrading your service to the build-default or build-runtime plan.

- In your subaccount, there's an active instance of the Cloud Foundry environment that uses one of the following plans:
  - `standard`: see Standard Plan for Cloud Foundry Runtime
  - `free`: see Free Plan for Cloud Foundry Runtime
  - `build-code`: see What Is SAP Build Code

Context

Here are the possible options for migrating your Cloud Foundry environment instance to SAP Build:

- `standard` (together with `MEMORY`) → `build-runtime`
- `free` → `build-runtime`
- `build-code` → `build-runtime`

For more information, see What Is SAP Build.

> ### Note
>
> When you migrate to the `build-runtime` plan, 400 GB of runtime memory is automatically assigned to your subaccount. This value overwrites any original runtime memory assignment and can't be changed.
>
> The 400 GB of runtime memory is a technical quota that represents an upper limit on the runtime memory consumption. You're billed based on the runtime memory you actually consume, not for the full 400 GB technical quota.

Procedure

> ### Caution
>
> The migration procedure is irreversible. You can't move the Cloud Foundry environment instance back to your original plan after switching to `build-runtime`.

1. Log in to your global account in the SAP BTP cockpit.
2. Go to your subaccount and choose *Instances and Subscriptions*.
3. Under *Environments*, find your Cloud Foundry environment instance and do the following:
   a. Choose the ••• *Actions* icon on the right.
   b. Choose *Update*.
   c. In the drop-down list *Plan*, select `build-runtime`.
   d. Check the box with a warning that has appeared.
   e. Choose *Update Instance*.
4. Wait for your Cloud Foundry environment instance to update.

Results

If the status of your Cloud Foundry environment instance after the procedure is *Created*, you've successfully migrated the instance to SAP Build.

> ### Note
>
> After the migration, the runtime memory consumption of your Cloud Foundry environment instance is going to be reflected in the *SAP Build - Cloud Foundry Runtime* plan, instead of its original plan. The consumption of the SAP Build capacity units is going to be reflected in the *SAP Build - Runtime* plan.

Related Information

Service Plans and Metering for Cloud Foundry Runtime [page 69]

6.2.5 Org Administration Using the Cockpit

In the Cloud Foundry environment, manage orgs, spaces and space quota plans using the SAP BTP cockpit.

In addition to global accounts and subaccounts, the Cloud Foundry environment includes another hierarchical level represented by orgs and spaces.

When you enable Cloud Foundry in a subaccount, you create a Cloud Foundry org in which you can then create multiple spaces. Each subaccount can contain only one Cloud Foundry org. There is no limit to how many spaces you can have within one org.

In the Cloud Foundry environment, you deploy applications and consume services at space level. Similar to subaccounts, spaces enable you to once again structure and sub-divide quota if you want to. You can do this by managing space quota plans.

- Managing Orgs [page 2793]
- Managing Spaces [page 2799]
- Managing Space Quotas [page 2806]

6.2.5.1 Navigate to Orgs and Spaces

To administer your Cloud Foundry environment, navigate to orgs, and spaces in the SAP BTP cockpit.

Prerequisites

- Sign up for an enterprise or a trial account and receive your logon data.
  For more information, see Get a Free Trial [page 162] or Sign up for a Customer Account [page 163].
- Create the org or space to which you want to navigate.
  For more information, see Create Orgs [page 2793] and Create Spaces [page 2800].

Procedure

Entity You'd Like to Navigate to	Navigation Path
Org	Navigate to the subaccount that contains the Cloud Foundry org. If you've already enabled the Cloud Foundry environment in your subaccount, you see the name of your organization, the number of its spaces and members, and its API endpoint on the *Overview* page of your subaccount. If you haven't enabled Cloud Foundry yet, choose *Enable Cloud Foundry* to create a Cloud Foundry org.

Entity You'd Like to Navigate to	Navigation Path
	Note Note that your subaccount and your org have a 1:1 relationship. They have the same name and therefore also the same navigation level in the cockpit.
Space	Navigate to the subaccount that contains the space you'd like to navigate to. 1. On the subaccount *Overview* page, you have table with all your spaces. Choose the space you want to navigate to. 2. In the navigation area, choose *Cloud Foundry* > *Spaces*.

Related Information

Navigate in the Cockpit [page 2461]

6.2.5.2 Managing Orgs

In the SAP BTP cockpit, learn how to create a Cloud Foundry org, manage its members, and later delete it, if needed.

- Create Orgs [page 2793]
- Delete Orgs [page 2794]
- Managing Org Members [page 2795]

Note

If you work with the Cloud Foundry CLI (cf CLI), be aware that you are responsible for all actions performed using the CLI. Also, you won't always be able to mirror the same data in the SAP BTP cockpit.

6.2.5.2.1 Create Orgs

To enable Cloud Foundry in your subaccount, you must create an org in order to use it.

Procedure

1. Navigate to your subaccount *Overview* page in the SAP BTP cockpit.
2. Choose *Enable Cloud Foundry*.

After you choose *Enable Cloud Foundry*, follow the steps provided by the wizard to complete the creation of your org.

3. Choose a plan from the drop-down list and enter a name for your org.

> **Note**
>
> Once you've created the org, you can't change its name afterwards in the SAP BTP cockpit. However, you can rename the org using the Cloud Foundry CLI (cf CLI), and the new org name will be reflected in the cockpit.

As an optional step, you can customize the instance name of your environment. Otherwise, the wizard uses the automatically generated instance name.

If you decide to enter an environment instance name of your choice, make sure that it's CLI-friendly. A CLI-friendly name is a string of up to 32 characters: alphanumeric (A-Z, a-z, 0-9), periods, underscores, and hyphens. Having a CLI-friendly name allows you to use it in BTP CLI commands.

4. Choose *Create*.

Related Information

Navigate to Orgs and Spaces [page 2792]
Create a Subaccount [page 2509]

6.2.5.2.2 Delete Orgs

You can delete a Cloud Foundry org from a subaccount using the SAP BTP cockpit. Once the org is deleted, you can create a new one.

Prerequisites

You are a global account administrator, as well as a member of the subaccount containing the org you want to delete.

> **Note**
>
> You can delete an org using only the SAP BTP cockpit or the BTP CLI. You cannot use the Cloud Foundry CLI to perform this task.

Procedure

1. Navigate to the subaccount containing the org.
2. Choose *Disable Cloud Foundry* and confirm the operation.

Results

The org is deleted. All data in the org including spaces, applications, service instances, and member information is lost. You can now choose *Enable Cloud Foundry* to create a new org in the subaccount.

Related Information

Navigate to Orgs and Spaces [page 2792]

6.2.5.3 Managing Org Members

Learn how to add, edit, and delete org members in the SAP BTP cockpit.

- Add Org Members [page 2795]
- Edit Org Members [page 2797]
- Delete Org Members [page 2798]

6.2.5.3.1 Add Org Members

In the SAP BTP cockpit, add users as org members and assign roles to grant the users access to information, such as user and quota information in a Cloud Foundry org.

Prerequisites

- You have the Org Manager role for the org in question.

  > **Note**
  >
  > When you enable Cloud Foundry and the org is created, the Org Manager role is assigned automatically to you.

  For more information, see About Roles in the Cloud Foundry Environment [page 2788].

- The users exist in a trusted platform identity provider.
  For more information, see About User Management in the Cloud Foundry Environment [page 2787].

Procedure

1. Navigate to your subaccount in the cockpit. Make sure that Cloud Foundry is enabled. For more information, see Navigate to Orgs and Spaces [page 2792].
2. In the navigation area, choose *Cloud Foundry* > *Org Members*.
   The screen displays all members currently assigned to the org in a list.
3. Choose *Add Members*.
4. Enter one or more e-mail addresses.

   Use commas (`,`), spaces ( ), semicolons (`;`), or line breaks to separate members.
5. Enter the *Origin* for the identity provider, which hosts the members you just added.

   The default identity provider is `sap.ids`. For more information, see Default Identity Provider [page 2594].

   If the new members are platform users from a custom identity provider, enter the origin.

   Select the corresponding origin of the Identity Authentication tenant.

   > ### Note
   >
   > The option to select an origin appears only if you have two or more identity providers.

   For more information about finding the origin, see Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576].
6. Select the org roles you want to assign.

   For more information, see About Roles in the Cloud Foundry Environment [page 2788].
7. Choose *Add*.

Next Steps

- To change the roles of a member, see Edit Org Members [page 2797].
- To remove all the roles of a member, see Delete Org Members [page 2798].
  This action removes the member from the organization and any spaces in the org.
- If the users need to view or manage applications in spaces, add the users to the relevant spaces, too.
  For more information, see Add Space Members [page 2803].

Related Information

Add Organization Members Using the Cloud Foundry Command Line Interface [page 2853]

6.2.5.3.2 Edit Org Members

Remove existing roles or assign new roles to an org member in the SAP BTP cockpit.

Prerequisites

- You have the Org Manager role for the org in question.

  > **Note**
  >
  > When you enable Cloud Foundry and the org is created, the Org Manager role is assigned automatically to you.

  For more information, see About Roles in the Cloud Foundry Environment [page 2788].
- The users exist in a trusted platform identity provider.
  For more information, see About User Management in the Cloud Foundry Environment [page 2787].

Procedure

1. In the navigation area in the cockpit, choose *Cloud Foundry* > *Org Members*.
   The screen displays all members currently assigned to the org in a list.
2. Find the org member in the list and choose *(Edit)*.
3. Select or unselect the roles you want to assign or remove.

   For more information, see About Roles in the Cloud Foundry Environment [page 2788].
4. Choose *Save*.

   The changes you make to the roles of a member take effect immediately.

Related Information

Add Org Members Using the Cockpit [page 2795]
Delete Org Members Using the Cockpit [page 2798]

6.2.5.3.3 Delete Org Members

Delete all roles of an org member in the SAP BTP cockpit. As a consequence, the member is removed from the org and any spaces in that org.

Prerequisites

- You have the Org Manager role for the org in question.

  > **Note**
  >
  > When you enable Cloud Foundry and the org is created, the Org Manager role is assigned automatically to you.

  For more information, see About Roles in the Cloud Foundry Environment [page 2788].
- The users exist in a trusted platform identity provider.
  For more information, see About User Management in the Cloud Foundry Environment [page 2787].

Procedure

1. In the navigation area in the cockpit, choose *Cloud Foundry* > *Org Members*.
   The screen displays all members currently assigned to the org in a list.
2. Find the org member in the list. To remove all the roles of that member, choose *(Delete)*.

   This action removes the member from the organization and any spaces in the org.

Related Information

Managing Org Members [page 2795]

6.2.5.4 Managing Spaces

Learn what a Cloud Foundry space is, what type of information you will find on the *Spaces* page in the SAP BTP cockpit, and what you can do with or within a space.

What Is a Space?

A space is a shared location on the platform in which you can deploy and maintain applications, and connect them to services. You can create multiple spaces within one org. There is no limit to how many spaces you can have in a single org.

By default, a space inherits the org quota. If you don't want your space to consume the entire org quota, you can use space quota instead. For more information, see Managing Space Quotas [page 2806].

Prerequisites

If you want to see any spaces on the *Spaces* page in the cockpit, you must have either the Org Manager role or any one of the space roles.

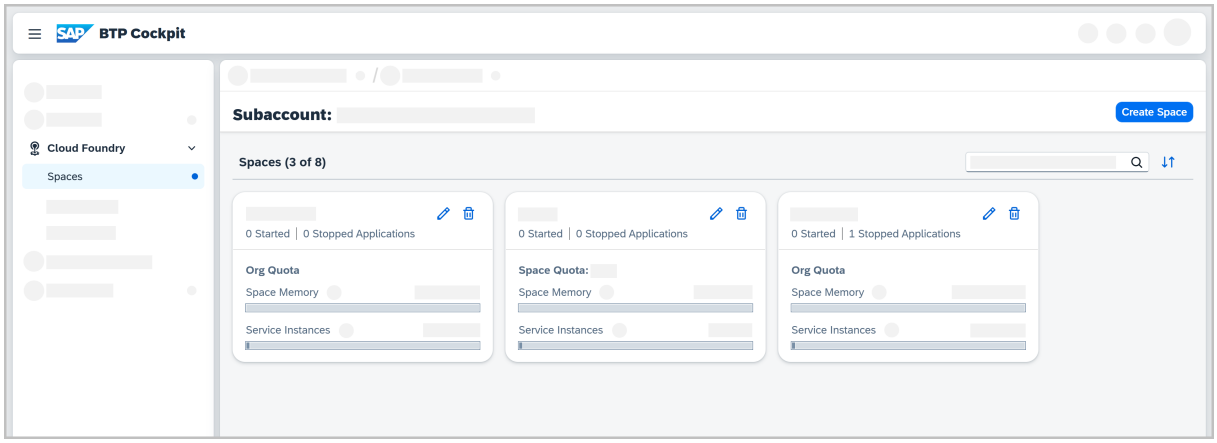
The Org Manager role gives you access to all spaces in an org. A space role gives you access to a specific space.

For more information, see About Roles in the Cloud Foundry Environment [page 2788].

What Can You Find on the Spaces Page?

To visit this page in the SAP BTP cockpit:

1. Navigate to a subaccount of your choice.
2. Navigate to the *Cloud Foundry* > *Spaces* page.



You find the following on this page:

- A list of all spaces you have access to.
- Details about each space tile, such as:
  - Number of started and stopped applications in a space
  - Name of the space quota if a space quota is assigned to the space, or *Org Quota* if the space is currently using the org quota

    To manage these settings, navigate to *Cloud Foundry* > *Space Quotas* > *Quota Assignments*. For more information, see Assign Space Quotas to Spaces [page 2810] and Edit Space Quotas [page 2811].
  - Available resources such as memory and services with their upper consumption limit and current usage statuses

What Can You Do With or Within a Space?

Here are some of the space-related actions you can do:

- Create Spaces [page 2800]
- Managing Space Members [page 2802]
- Deploy an Application [page 2862]
- Edit Spaces

  To edit a space, choose *(Edit) on the space tile. This option lets you change the name of the space.* Org Managers and Space Managers are allowed to rename spaces.
- Delete Spaces [page 2802]

Related Information

Org Administration Using the Cockpit [page 2792]
Org Administration Using the Cloud Foundry CLI [page 2812]

6.2.5.4.1 Create Spaces

Create spaces in your Cloud Foundry organization using the SAP BTP cockpit. In a space, you can deploy and maintain applications, and connect them to services.

Prerequisites

You have a Cloud Foundry organization in your subaccount, and you have the Org Manager role in that organization.

See Create Orgs [page 2793] and About Roles in the Cloud Foundry Environment [page 2788].

Context

There are two ways to create a space in the cockpit. The main one is to do it from the *Spaces* page in your subaccount. The second one is from the *Overview* page of your subaccount. You can choose either one of them.

Procedure

Create a Space from the Spaces Page	Create a Space from the Subaccount Overview Page
1. Navigate to the subaccount that contains the Cloud Foundry organization in which you'd like to create a space. 2. Navigate to the *Cloud Foundry* > *Spaces* page. 3. Choose *Create Space* from the top right hand-side corner of that page. 4. Enter a space name and choose the space roles you want to assign to yourself. 5. Then, choose *Save*.	1. Navigate to the subaccount that contains the Cloud Foundry organization in which you'd like to create a space. 2. On the subaccount *Overview* page, you have a *Spaces* table. Choose *Create Space* from the top right hand-side corner of that table. 3. Enter a space name and choose the space roles you want to assign to yourself. 4. Then, choose *Save*.

Next Steps

As an optional step, you can assign space quotas to spaces. If you don't assign a space quota, the space you've just created will use the consumption limits of the org quota.

For more information, see Assign Space Quotas to Spaces [page 2810].

Related Information

Create Spaces Using the Cloud Foundry Command Line Interface [page 2854]

6.2.5.4.2 Delete Spaces

Delete spaces in your Cloud Foundry org using the SAP BTP cockpit.

Prerequisites

- You have the Org Manager role in the org from which you want to delete a space. For more information about roles and permissions, see About Roles in the Cloud Foundry Environment [page 2788].
- You've ensured that the data in the space you're going to delete is no longer needed.

> ⚠ Caution
>
> Please be aware that you won't be able to access your SAP HANA database system if you delete a space in an organization to which an SAP HANA database system is assigned.

Procedure

1. Navigate to *Cloud Foundry* > *Spaces* in your Cloud Foundry subaccount and choose *(Delete)* on the tile of the space you want to delete.
2. Confirm your change.

Related Information

Create Spaces Using the Cockpit [page 2800]
Delete Spaces Using the Cloud Foundry Command Line Interface [page 2859]

6.2.5.5 Managing Space Members

Learn how to add, edit, and delete space members in the SAP BTP cockpit.

- Add Space Members [page 2803]
- Edit Space Members [page 2804]
- Delete Space Members [page 2805]

6.2.5.5.1 Add Space Members

You can add space members and assign roles to them at the space level in the SAP BTP cockpit.

Prerequisites

- You have the Space Manager or Org Manager role.
  If you only have the Space Manager role, the users you want to add to the space must already have the Org User role.
  If you have the Org Manager role, you can assign space roles to users even if they're not members of the org yet.

  > **Note**
  >
  > When you add users who aren't yet members of an org to a space in that org, they get the Org User role by default. This makes them members of the org as well.

- The users exist in a trusted platform identity provider.
  For more information, see About User Management in the Cloud Foundry Environment [page 2787].

Procedure

1. Navigate to *Cloud Foundry* > *Spaces* in the SAP BTP cockpit. For more information, see Navigate to Orgs and Spaces [page 2792].
2. Choose a space to which you'd like to add members. For more information, see Navigate to Orgs and Spaces [page 2792].
3. In the navigation area, choose *Space Members*.
   The screen displays all members currently assigned to the space in a list.
4. Choose *Add Members*.
5. Enter one or more e-mail addresses.

   Use commas (`,`), spaces ( ), semicolons (`;`), or line breaks to separate members. The changes you make to the roles of a member take effect immediately.
6. Enter the *Origin* for the identity provider, which hosts the members you just added.

   The default identity provider is `sap.ids`.

   For more information, see Default Identity Provider [page 2594].

   If the new members are platform users from a custom identity provider, enter the origin.

   Select the corresponding origin of the Identity Authentication tenant.

   For more information about finding the origin, see Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576].
7. Select the space roles you want to assign.

For more information, see About Roles in the Cloud Foundry Environment [page 2788].

8. Choose *Add*.

Next Steps

You also have the following options:

- Edit Space Members [page 2804]
- Delete Space Members [page 2805]

Related Information

Add Org Members Using the Cockpit [page 2795]
Add Space Members Using the Cloud Foundry Command Line Interface [page 2855]

6.2.5.5.2 Edit Space Members

Remove existing roles or assign new roles to a space member in the SAP BTP cockpit.

Prerequisites

- You have the Space Manager or Org Manager role.
- The users exist in a trusted platform identity provider.
  For more information, see About User Management in the Cloud Foundry Environment [page 2787].

Procedure

1. Navigate to *Cloud Foundry* > *Spaces* in the SAP BTP cockpit. For more information, see Navigate to Orgs and Spaces [page 2792].
2. Choose the space in which you'd like to edit the space roles of any of its members.
3. In the navigation area, choose *Space Members*.
   The screen displays all members currently assigned to the space in a list.
4. Choose *(Edit)*.
5. Select or unselect the space roles you want to assign or remove.

   For more information, see About Roles in the Cloud Foundry Environment [page 2788].

6. Choose *Save*.

   The changes you make to the roles of a member take effect immediately.

Related Information

Add Space Members [page 2803]
Delete Space Members [page 2805]

6.2.5.5.3 Delete Space Members

Delete all roles of a space member in the SAP BTP cockpit. As a consequence, the member is removed from the space.

Prerequisites

- You have the Space Manager or Org Manager role.
- The users exist in a trusted platform identity provider.
  For more information, see About User Management in the Cloud Foundry Environment [page 2787].

Procedure

1. Navigate to *Cloud Foundry* > *Spaces* in the SAP BTP cockpit. For more information, see Navigate to Orgs and Spaces [page 2792].
2. Choose the space from which you'd like to remove members.
3. In the navigation area, choose *Space Members*.
   The screen displays all members currently assigned to the space in a list.
4. To remove all the roles of a member, choose *(Delete)*.

   This action removes the member from the space.

Related Information

Managing Space Members [page 2802]

6.2.5.6 Managing Space Quotas

Learn what a space quota is, what type of information you will find on the *Space Quotas* page in the SAP BTP cockpit, and what you can do with a space quota in the Cloud Foundry environment.

What Is a Space Quota?

A space quota defines the upper consumption limit for all resources in a space, such as:

- Space memory
- Route
- Service instances
- Memory per instance
- Application instances

> **Note**
>
> Assigning a space quota to a space is optional. A space without an assigned space quota shares the consumption limits defined in the org quota with all the other spaces part of the org.

When working with space quotas, keep in mind the following additional specifics:

- You can create unlimited space quotas in an organization (org), but you can assign only one space quota to a space at a time.
- If a space quota is larger than the org quota, the org quota limit is applied.
  Example: If you create a space quota with space memory of 2048MB but the org quota has a total memory of 1024MB, then the space is going to use the org quota instead.
- Also, you can have only one space quota named "dev" in an org, but two orgs can have two separate space quotas with the same name "dev".
- When creating a space quota, you can define whether a space quota allows you to use paid services within a space.

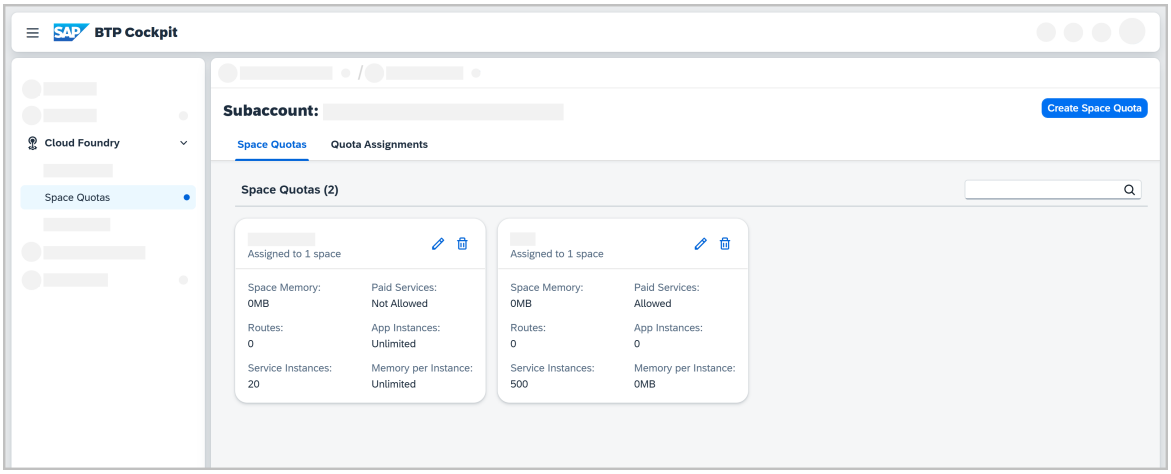
Prerequisites

To create, edit, or assign space quotas in an org, you must have the Org Manager role.

What Can You Find on the Space Quotas Page?

You can switch between the following views on this page:

- *Space Quotas*



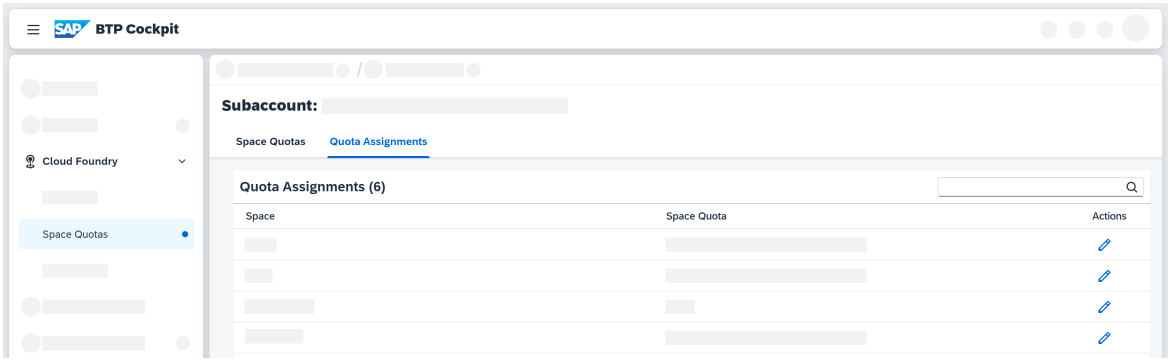
You find the following in this view:

- A list of all space quotas you have created and whether they are assigned to a space.
- Details about each space quota tile related to the space, such as:
  - *Space Memory (MB)*: Total amount of memory
  - *Routes*: Total number of routes
  - *Service Instances*: Total number of service instances
  - *Paid Services*: Selected if you'd like to allow the provisioning of instances of paid service plans
  - *App Instances*: Total number of application instances
  - *Memory per Instance (MB)*: Maximum amount of memory an application instance can have

  > **Note**
  >
  > The value of `-1` represents an unlimited amount.

- The option to create a space quota.
  For more information, see Create Space Quotas [page 2808].
- The option to edit a space quota.
  For more information, see Edit Space Quotas [page 2811].
- The option to delete a space quota.
  You can delete a space quota only if it's not assigned to a space.

- *Quota Assignments*



You find the following in this view:

- A list of all the available spaces in the subaccount you have access to and whether they have a space quota assigned to them or if they are using the org quota instead.

- The option to *(Edit)* a quota assignment.

For more information, see Assign Space Quotas to Spaces [page 2810].

What Can You Do with a Space Quota?

- Create Space Quotas [page 2808]
- Assign Space Quotas to Spaces [page 2810]
- Edit Space Quotas [page 2811]

Related Information

Managing Spaces [page 2799]
About Roles in the Cloud Foundry Environment [page 2788]

6.2.5.6.1 Create Space Quotas

You can use the SAP BTP cockpit to create space quotas.

Prerequisites

You have the Org Manager role for the org in which you want to create a space quota.

Procedure

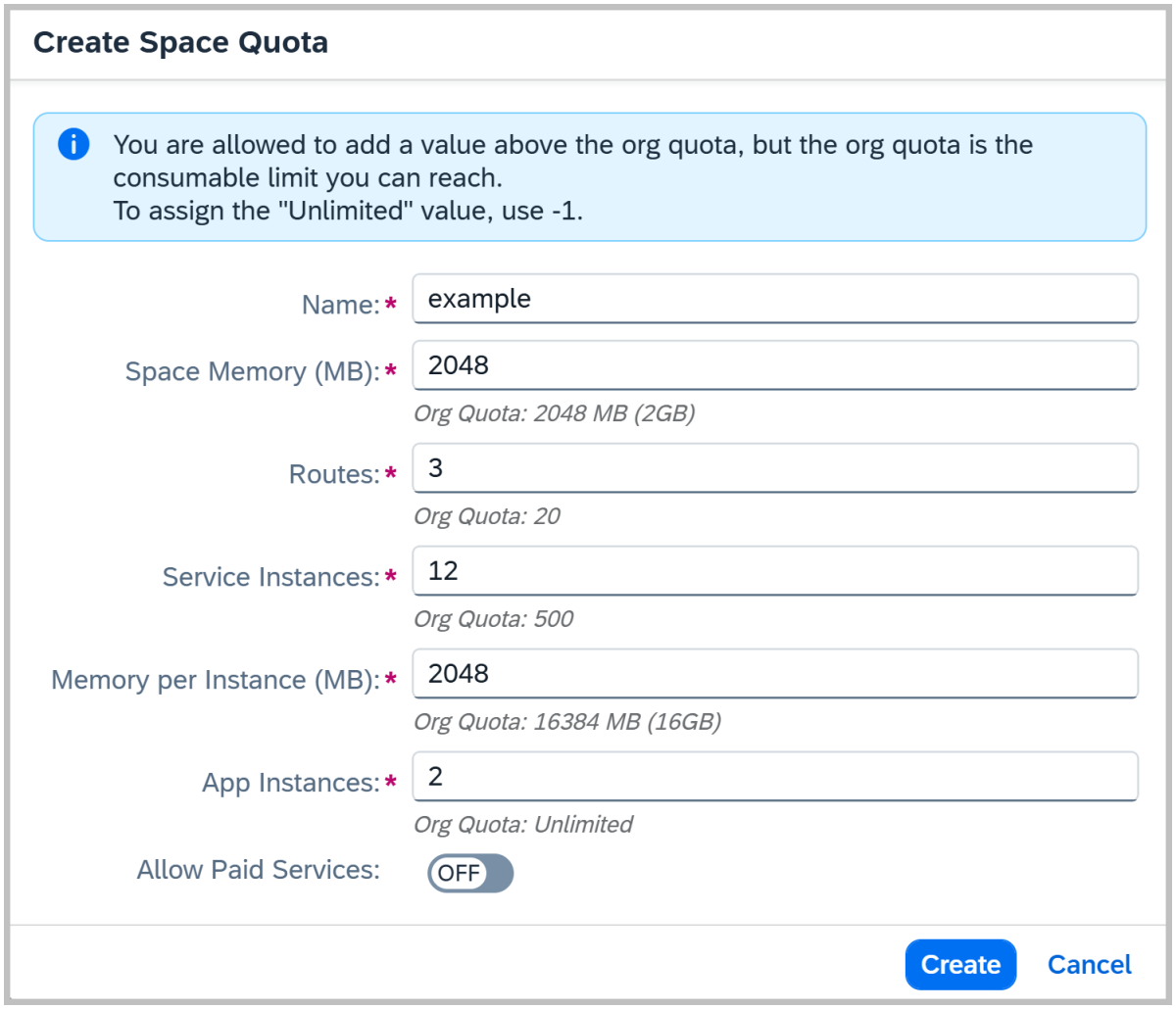
1. Navigate to the subaccount that contains the spaces to which you want to add quotas.
2. In the navigation menu, choose *Cloud Foundry* > *Space Quotas*.
3. Choose *Create Space Quota*.
4. Enter a name for your space quota and add limits for the following:
   - *Space Memory (MB)*: Total amount of memory a space can have
   - *Routes*: Total number of routes
   - *Service Instances*: Total number of service instances
   - *Memory per Instance (MB)*: Maximum amount of memory an application instance can have
   - *App Instances*: Total number of application instances
   - (Optional) *Allow Paid Service*: Switch on if you'd like to allow the provisioning of instances of paid service plans.

> **Note**
>
> The value of `-1` represents an unlimited amount.

> **Note**
>
> If the space quota limit for a resource exceeds the org quota limit, the org quota limit applies.

5. Choose *Create*.

Example: Creating a Space Quota in the Cockpit



Related Information

Entitlements and Quotas [page 127]

Create Space Quota Plans Using the Cloud Foundry Command Line Interface [page 2857]

6.2.5.6.2 Assign Space Quotas to Spaces

You can use the SAP BTP cockpit to assign space quotas to spaces.

Prerequisites

- You have the Org Manager role for the org in which you want to create and assign a space quota.
- You have at least one space quota. For more information, see Create Space Quotas [page 2808].

Procedure

1. Navigate to the subaccount that contains the spaces to which you want to assign space quotas.
2. In the navigation menu, choose *Cloud Foundry* > *Space Quotas*.
3. Open the *Quota Assignments* tab.
4. Select one of the existing quota assignments and choose *(Edit)* to make changes.
5. Select either a specific *Space Quota* from the drop-down or the *Org Quota*.

   If you select *Org Quota*, the space will use the consumption limits defined in the org quota.
6. To assign the selected space or org quota to the space, choose *Save*.

Related Information

Entitlements and Quotas [page 127]
Create Space Quotas Using the Cockpit [page 2808]
Assign Quota Plans to Spaces Using the Cloud Foundry Command Line Interface [page 2858]

6.2.5.6.3 Edit Space Quotas

Edit space quotas in the Cloud Foundry environment using the SAP BTP cockpit.

Prerequisites

You have the Org Manager role for the org in which you want to edit space quotas.

Procedure

1. Choose the subaccount that contains the spaces for which you'd like to edit the space quota.
2. In the navigation menu, choose *Cloud Foundry* > *Space Quotas*.
3. Choose *(Edit)* for an existing space quota.
4. Adjust the values in the text fields as needed.

   For more information about the different values, see https://docs.cloudfoundry.org/adminguide/quota-plans.html#space .
5. Choose *Save*.

Related Information

Change Space Quota Plans Using the Command Line Interface [page 2858]
Create Space Quotas [page 2808]
Assign Space Quotas to Spaces [page 2810]
Configure Entitlements and Quotas for Subaccounts [page 2529]
Entitlements and Quotas [page 127]

6.2.5.7 Using Space Events

The *Space Events* page at space level in the SAP BTP cockpit shows the order in which actions are performed in a space.

When Are Space Events Created?

A space event is created when you perform an action in the space.

For example, if you trigger the creation of a new route, the `audit.route.create` space event is logged. The route is the target resource of this action. In this example, you are the *Actor*, while the route is the *Target*.

How Are Space Events Most Commonly Used?

You can use space events to:

- Troubleshoot a failed action
- Keep track of the history of certain resources
- For auditing and compliance purposes

What Are the Types of Space Events?

Depending on the resource that is the target of the action, a space event can be specific to an application, service, route, organization, among others.

To get the full list of all space event types, see https://docs.cloudfoundry.org/running/managing-cf/audit-events.html#types .

Searching for a Specific Space Event

You can search for a specific space event by actor or target.

> **Note**
>
> Not all search results are loaded immediately. To add more results to your search, choose **Load More**. Each **Load More** action adds up to 200 space events to your search results.

6.2.6 Org Administration Using the Cloud Foundry CLI

Use the Cloud Foundry command line interface (CF CLI) for managing subaccounts in the Cloud Foundry environment, such as creating orgs and spaces, or managing quota.

- Download and Install the Cloud Foundry Command Line Interface [page 2813]
- CF CLI: Plug-ins [page 2815]
- Create Spaces Using the Cloud Foundry Command Line Interface [page 2854]
- Add Organization Members Using the Cloud Foundry Command Line Interface [page 2853]
- Add Space Members Using the Cloud Foundry Command Line Interface [page 2855]
- Create Space Quota Plans Using the Cloud Foundry Command Line Interface [page 2857]
- Assign Quota Plans to Spaces Using the Cloud Foundry Command Line Interface [page 2858]

- Change Space Quota Plans Using the Command Line Interface [page 2858]

> **Note**
>
> If you work with the Cloud Foundry CLI (cf CLI), be aware that you are responsible for all actions performed using the CLI. Also, you won't always be able to mirror the same data in the SAP BTP cockpit.
>
> You must delete an org using only the SAP BTP cockpit or the SAP BTP command line interface (BTP CLI).

6.2.6.1 Working with the Cloud Foundry Command Line Interface

Find out how to get and use the Cloud Foundry command line interface.

Related Information

Download and Install the Cloud Foundry Command Line Interface [page 2813]
Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814]
CF CLI: Plug-ins [page 2815]

6.2.6.1.1 Download and Install the Cloud Foundry Command Line Interface

Download and set up the Cloud Foundry Command Line Interface (cf CLI) to start working with the Cloud Foundry environment.

Procedure

1. Download the latest version of cf CLI from GitHub at the following URL: https://github.com/cloudfoundry/cli#downloads
   - For Microsoft Windows, download the Windows 64-bit installer.
   - For Mac OS, download the PKG file. You can use the Homebrew open source package management software to download the latest available version of the cf CLI.
2. Install the cf CLI:
   - For Microsoft Windows, unpack the ZIP file.
   - For Mac OS, proceed as follows:
     1. Open the PGK file.

2. In the installation wizard, choose *Continue*, and then select the destination folder for the cf CLI installation.
3. Choose *Continue*, and when prompted, choose *Install*.

For more information, see http://docs.cloudfoundry.org/devguide/installcf/install-go-cli.html .

3. (Optional) If you have an HTTP proxy server, configure the proxy settings. For more information, see http://docs.cloudfoundry.org/cf-cli/http-proxy.html .

6.2.6.1.2 Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface

Use the Cloud Foundry Command Line Interface (cf CLI) to log on to the Cloud Foundry space.

Prerequisites

- The Cloud Foundry environment is enabled. You can enable it in the Cloud Platform Cockpit. For more information, see Create a Subaccount [page 2509].
- (Enterprise accounts only) You have created at least one subaccount and enabled the Cloud Foundry environment in this subaccount. For more information, see Create a Subaccount [page 2509].

> ⓘ Note
>
> In a Cloud Foundry trial account, the Cloud Foundry environment has been activated for you automatically and a first space "dev" has been created for you.

- Download and install the cf CLI and log on to Cloud Foundry. For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813].

Procedure

1. Open a command line.
2. Set the target API endpoint to the cloud controller.

> ⓘ Note
>
> There is no specific endpoint for trial accounts. Both enterprise and trial accounts use the same API endpoints.

```
cf api https://api.cf.eu10.hana.ondemand.com
```

For more information, see Regions and API Endpoints Available for the Cloud Foundry Environment [page 19].

3. Log on to the Cloud Foundry environment:

```
cf login
```

> **Note**
>
> - Log on with the password associated with your account (S-user or P-user) in the default identity provider, SAP ID service. If you have multiple user accounts with the same email address, log on with your user ID (S-user or P-user) instead of your email address.
>   If you've forgotten your password, reset your password.
>   For more information, see SAP Note 3085908.
> - To log on with a specific identity provider, use the option `--origin`.
>   For more information, see Log On as a Technical User With a Custom Identity Provider [page 2590].

4. When prompted, enter your user credentials (email and password).
5. To view the help for the CLI, execute **`cf help`**, which lists the most common CLI commands with a short description, or **`cf help -a`**, which lists all commands. To get help for a specific command, execute **`cf help <command>`**.

6.2.6.1.3 CF CLI: Plug-ins

A list of additional commands that have been implemented as plug-ins to extend the base CF CLI client.

- Autoscaler Plugin for Cloud Foundry CLI [page 2815]
- Custom Domain Plugin for the Cloud Foundry Environment [page 2822]
- Java Plugin for Cloud Foundry Command Line Interface [page 2831]
- Multitarget Application Commands for the Cloud Foundry Environment [page 2834]
- Service Fabrik Plugin in Cloud Foundry [page 2849]

6.2.6.1.3.1 Autoscaler Plugin for Cloud Foundry CLI

The Autoscaler service offers a CLI plugin for the Cloud Foundry Command Line Interface (CLI).

The plugin enables users to:

- attach or detach scaling policies to apps
- view attached scaling policies and view the scaling histories
- view the metrics sent by apps

This is useful:

- to get some insights into how current scaling policies work and if changes are reasonable.
- for debugging, especially when introducing custom-metrics to scale an app. It lets you check if the sent metrics are correctly handled by the Application Autoscaler, and it helps you understand why your app is scaling out or in or why this does not happen.

This documentation explains how to use the plugin on Cloud Foundry in BTP. For details not related to BTP, see https://github.com/cloudfoundry/app-autoscaler-cli-plugin/blob/main/README.md .

Command List

Cloud Foundry Commands of Application Autoscaler [page 2817]

Related Information

Initial Setup
Cloud Foundry Commands of Application Autoscaler [page 2817]

6.2.6.1.3.1.1 Cloud Foundry Commands of Application Autoscaler

The Application Autoscaler plugin for the Cloud Foundry Command Line Interface (cf CLI) includes commands that you can use to get some insights into your scaling policies and for debugging.

Commands for the Application Autoscaler CLI Plugin

Command	Alias	Usage	Options	Description	Examples

`cf autoscaling-policy`	`asp`	`cf autoscaling-policy APP_NAME [--output PATH_TO_FILE]`	`--output` : dump the policy to a file in JSON format	Retrieve the scaling policy of an application. The policy will be displayed in JSON format.	View scaling policy, replace APP_NAME with the name of your application:
					`$ cf autoscaling-policy APP_NAME`
					
					`Showing policy for app APP_NAME...`
					`{`
					`"instance_min_count": 1,`
					`"instance_max_count": 5,`
					`"scaling_rules": [`
					`{`
					`"metric_type": "memoryused",`
					`"breach_duration_secs": 120,`
					`"threshold": 15,`
					`"operator": ">=",`
					`"cool_down_secs": 120,`
					`"adjustment": "+1"`
					`},`
					`{`
					`"metric_type": "memoryused",`
					`"breach_duration_secs": 120,`
					`"threshold": 10,`
					`"operator": "<",`

Command	Alias	Usage	Options	Description	Examples

					<pre>"cool_down_secs": 120,
					
					"adjustment": "-1"
					        }
					    ]
					}</pre>
					Dump the scaling policy to a file in JSON format:
					<pre>$ cf asp APP_NAME --output PATH_TO_FILE
					
					Saving policy for app APP_NAME to PATH_TO_FILE...
					OK</pre>

`cf attach-autoscaling-policy`	`aasp`	`cf attach-autoscaling-policy APP_NAME PATH_TO_POLICY_FILE`		Attach a scaling policy to an application. The policy file must be a JSON file. See policy specification for the policy format.	<pre>$ cf attach-autoscaling-policy APP_NAME PATH_TO_POLICY_FILE
					
					Attaching policy for app APP_NAME...
					OK</pre>

`cf detach-autoscaling-policy`	`dasp`	`cf detach-autoscaling-policy APP_NAME`		Detach the scaling policy from an application. The policy will be deleted when detached.	<pre>$ cf detach-autoscaling-policy APP_NAME
					
					Detaching policy for app APP_NAME...
					OK</pre>

Command	Alias	Usage	Options	Description	Examples

`cf autoscaling-metrics`	`asm`	`cf autoscaling-metrics APP_NAME METRIC_NAME [--start START_TIME] [--end END_TIME] [--asc] [--output PATH_TO_FILE]`	• METRIC_NAME : default metrics "memoryused, memoryutil, responsetime, throughput, cpu" or customized name for your own metrics.
			• --start : start time of metrics collected with format yyyy-MM-ddTHH:mm:ss+/-HH:mm or yyyy-MM-ddTHH:mm:ssZ, default to very beginning if not specified.
			• --end : end time of the metrics collected with format yyyy-MM-ddTHH:mm:ss+/-HH:mm or yyyy-MM-ddTHH:mm:ssZ, default to current time if not spefi-cied.
			• --asc : display in ascending order, default to descending order if not specified
			• --output : dump the metrics to a file
				Retrieve the aggregated metrics of an application. You can specify the start/end time of the returned query result and the display order (ascending or descending). The metrics are shown in a table.	`$ cf autoscaling-metrics APP_NAME memoryused --start 2018-12-27T11:49:00+08:00 --end 2018-12-27T11:52:20+08:00 --asc`
					
					`Retreiving aggregated metrics for app APP_NAME...`
					`Metrics Name Value Timestamp`
					`memoryused 62MB 2018-12-27T11:49:00+08:00`
					`memoryused 62MB 2018-12-27T11:49:40+08:00`
					`memoryused 61MB 2018-12-27T11:50:20+08:00`
					`memoryused 62MB 2018-12-27T11:51:00+08:00`
					`memoryused 62MB 2018-12-27T11:51:40+08:00`
					
					Description of table-columns:
					• Metrics Name: name of the current metric item
					• Value: the value of the current metric item with unit
					• Timestamp: collect time of the current metric item

Command	Alias	Usage	Options	Description	Examples

`cf autoscaling-history`	`ash`	`cf autoscaling-history APP_NAME [--start START_TIME] [--end END_TIME] [--asc] [--output PATH_TO_FILE]`	• --start : start time of the scaling history with format yyyy-MM-ddTHH:mm:ss+/-HH:mm or yyyy-MM-ddTHH:mm:ssZ, default to very beginning if not specified.
			• --end : end time of the scaling history with format yyyy-MM-ddTHH:mm:ss+/-HH:mm or yyyy-MM-ddTHH:mm:ssZ, default to current time if not spefiicied.
			• --asc : display in ascending order, default to descending order if not specified
			• --output : dump the scaling history to a file
				Retrieve the scaling event history of an application. You can specify the start/end time of the returned query result, and the display order (ascending or descending). The scaling event history is shown in a table.	`$ cf autoscaling-history APP_NAME --start 2018-08-16T17:58:53+08:00 --end 2018-08-16T18:01:00+08:00 --asc`
					
					`Showing history for app APP_NAME...`
					`Scaling Type Status Instance Changes Time`
					
					`Action`
					
					`Error dynamic`
					
					`failed 2->-1`
					
					`2018-08-16T17:58:53+08:00 -1 instance(s) because throughput 10rps for 120 seconds app does not have policy set dynamic`
					
					`succeeded 2->3`
					
					`2018-08-16T17:59:33+08:00 +1 instance(s) because memoryused >= 15MB for 120 seconds scheduled`
					
					`succeeded 3->6`
					
					`2018-08-16T18:00:00+08:00 3 instance(s) because limited`

Command	Alias	Usage	Options	Description	Examples

					`by min instances 6`
					
					Description of table-columns:
					• Scaling Type: the trigger type of the scaling action, possible scaling types: dynamic and scheduled
					  • dynamic: the scaling action is triggered by a dynamic rule (memoryused, memoryutil, responsetime or throughput)
					  • scheduled: the scaling action is triggered by a recurring schedule or specific date rule
					• Status: the result of the scaling action: succeeded or failed
					• Instance Changes: how the instances number get changed (e.g. 1->2 means the application was scaled out from 1 instance to 2)
					• Time: the finish time of scaling action, no mater succeeded or failed
					• Action: the detail information about why and how the application scaled
					• Error: the reason why scaling failed

Related Information

What Is Application Autoscaler?
Initial Setup
Defining a Scaling Policy

6.2.6.1.3.2 Custom Domain Plugin for the Cloud Foundry Environment

The Custom Domain CLI plugin provides functions for creating private keys and certificate signing requests, as well as additional commands for managing your custom domains.

To use the functionality of the Custom Domain plugin, use the extended Cloud Foundry commands in the command-line interface.

> **Note**
>
> However, we recommend using the UI over the CLI because the Custom Domain Manager supports TLS configurations and extension landscapes. For more information, see Custom Domain Manager.

Related Information

Install the Custom Domain Plugin [page 2822]
Extended Cloud Foundry Commands of Custom Domains [page 2824]
Configuring Custom Domains

6.2.6.1.3.2.1 Install the Custom Domain Plugin

Use the Custom Domain CLI plugin to configure and manage your custom domains.

Prerequisites

Install the Cloud Foundry command-line interface (CLI) for the plugin to work. You can find the installation instructions here: Download and Install the Cloud Foundry Command Line Interface [page 2813].

Procedure

1. Download the latest version of the plugin that is compatible with your operating system. On the Web page, you'll find the plugin under the **SAP BTP Cloud Foundry CLI plugins** section with the name Custom Domain.
2. Untar or unzip the downloaded archive.
3. Open the command-line interface or terminal.
4. To install the plugin, enter the following command:

- Mac and Linux:

  ```
  cf install-plugin <path to the downloaded folder>/custom-domain-cli
  ```

- Windows:

  ```
  cf install-plugin <path to the downloaded folder>\custom-domain-cli.exe
  ```

> **Note**
>
> If you are reinstalling the plugin, first uninstall the previous version using: `cf uninstall-plugin "Custom Domain"`

5. Verify that the plugin has been installed successfully by entering:

   ```
   cf plugins
   ```

   You see a list of plugins that now includes Custom Domain. The output also displays commands that are specific to the Custom Domain plugin.

Results

You have installed the Custom Domain plugin for the Cloud Foundry CLI and can now use the extended commands that are available from the Custom Domain service.

Related Information

Download and Install the Cloud Foundry Command Line Interface [page 2813]
Configuring Custom Domains

6.2.6.1.3.2.2 Extended Cloud Foundry Commands of Custom Domains

The Custom Domain plugin includes commands that you can use to configure and manage your custom domains.

Commands for the Custom Domain CLI Plugin

Command	Alias	Usage	Options		Description
`cf custom-domain-create-key`	`cdck`	`cf custom-domain-create-key KEY SUBJECT DOMAIN [DOMAIN ...] [options]`	`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	Generate a new private and public key pair.
			`-verbose`	Show verbose information.	
			`-force`	Force key creation without confirmation.	
`cf custom-domain-get-csr`	`cdgc`	`cf custom-domain-get-csr KEY [FILE] [options]`	`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	Download the certificate signing request corresponding to a new key.
			`-verbose`	Show verbose information.	

Command	Alias	Usage	Options		Description
`cf custom-domain-upload-certificate-chain`	`cducc`	`cf custom-domain-upload-certificate-chain KEY FILE [options]`	`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	Upload a certificate chain for a given key.
			`-verbose`	Show verbose information.	
			`-force`	Force uploading the certificate chain without confirmation.	
`cf custom-domain-activate`	`cda`	`cf custom-domain-activate KEY DOMAIN [DOMAIN ...] [options]`	`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	Activate the server certificate for specified domains.
			`-verbose`	Show verbose information.	

Command	Alias	Usage	Options		Description
`cf custom-domain-list`	`cdl`	`cf custom-domain-list`	`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	List domains and their configuration status.
			`-verbose`	Show verbose information.	
`cf custom-domain-show-certificates`	`cdsc`	`cf custom-domain-show-certificates KEY [FILE] [options]`	`-dump`	Dump the certificates in PEM format.	Show certificates for a specified key.
			`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	
			`-verbose`	Show verbose information.	

Command	Alias	Usage	Options		Description
`cf custom-domain-enable-client-authentication`	`cdeca`	`cf custom-domain-enable-client-authentication FILE DOMAIN [DOMAIN ...] [options]`	`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	Activate client authentication for specified domains.
			`-verbose`	Show verbose information.	
`cf custom-domain-disable-client-authentication`	`cddca`	`cf custom-domain-disable-client-authentication DOMAIN [options]`	`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	Disable client authentication for a domain.
			`-verbose`	Show verbose information.	
			`-force`	Force deactivation of client authentication without confirmation.	

Command	Alias	Usage	Options		Description
`cf custom-domain-show-trusted-certificates`	`cdstc`	`cf custom-domain-show-trusted-certificates DOMAIN [FILE] [options]`	`-dump`	Dump the certificates in PEM format.	Show the trusted certificates for a domain.
			`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	
			`-verbose`	Show verbose information.	
`cf custom-domain-deactivate`	`cdd`	`cf custom-domain-deactivate DOMAIN [options]`	`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	Deactivate the certificate for a specified domain.
			`-verbose`	Show verbose information.	
			`-force`	Force deactivation without confirmation.	

Command	Alias	Usage	Options		Description
`cf custom-domain-delete-key`	`cddk`	`cf custom-domain-delete-key KEY [options]`	`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	Delete the private key and its certificates.
			`-verbose`	Show verbose information.	
			`-force`	Force key deletion without confirmation.	
`cf custom-domain-version`	`cdv`	`cf custom-domain-version`	`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	Show the Custom Domain plugin version.
			`-verbose`	Show verbose information.	

Command	Alias	Usage	Options		Description
`cf custom-domain-map-route`	`cdmr`	`cf custom-domain-map-route STANDARD_ROUTE CUSTOM_ROUTE`	`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	Map a custom domain route to a SaaS application.
			`-verbose`	Show verbose information.	
`cf custom-domain-unmap-route`	`cdur`	`cf custom-domain-unmap-route CUSTOM_ROUTE`	`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	Unmap a custom domain route from a SaaS application.
			`-verbose`	Show verbose information.	
`cf custom-domain-routes`	`cdr`	`cf custom-domain-routes`	`-skip-ssl-validation`	Do not attempt to validate SSL certificate.	List all configured routes of SaaS applications with custom domains.
			`-verbose`	Show verbose information.	

Related Information

Configuring Custom Domains
Managing Custom Domains

6.2.6.1.3.3 Java Plugin for Cloud Foundry Command Line Interface

Use the Java plugin for the Cloud Foundry command line interface to get heap dumps, thread dumps and profiling data (JFR or async-profiler).

Before using the extended commands, you first need to install the Java plugin in the Cloud Foundry environment.

See: Install the Java CLI Plugin in the Cloud Foundry Environment [page 2831]

6.2.6.1.3.3.1 Install the Java CLI Plugin in the Cloud Foundry Environment

The Java plugin provides convenience utilities to work with Java applications deployed on Cloud Foundry.

Prerequisites

You have installed the Cloud Foundry command line interface (cf CLI). See: Download and Install the cf CLI

Procedure

1. Open the command line interface or terminal.
2. Check if you already have the Java plugin installed. Run:

   ```
   cf plugins
   ```

   If the `java` plugin is installed, you have to uninstall it first. Run:

   ```
   cf uninstall-plugin java
   ```
3. Before installing the latest available version of the plugin, make sure you have the Cloud Foundry community repository in your cf CLI.

If it is not available there, you need to add it. Run:

```
cf add-plugin-repo CF-Community https://plugins.cloudfoundry.org
```

4. Install the latest Java plugin version. Run:

```
cf install-plugin -r CF_Community java
```

> → Tip
>
> In case the latest version is not available in the Cloud Foundry community repository, you can perform a manual installation:
>
> 1. Download the binary file for your target operating system from the latest release .
> 2. If you've already installed the plugin and you're now updating it, run: `cf uninstall-plugin java`
> 3. Now install the plugin. Run: `cf install-plugin [cf-cli-java-plugin]`
>
> **Note:** Replace `[cf-cli-java-plugin]` with the *actual* binary name you'll use, which depends on your operating system.

5. Verify that the plugin has been installed successfully. Run:

```
cf plugins
```

Results

You see a list of cf CLI plugins that now includes the `java` one. The output also displays commands that are specific to this plugin.

6.2.6.1.3.4 Multitarget Application Plugin for Cloud Foundry Command Line Interface

Use the Multitarget application plugin for the Cloud Foundry command line interface to deploy, remove, and view MTAs, among other possible operations.

Before using the extended commands, you first need to install the MTA plugin in the Cloud Foundry environment.

See: Install the MultiApps CLI Plugin in the Cloud Foundry Environment [page 2833]

Related Information

Multitarget Applications in the Cloud Foundry Environment [page 615]

6.2.6.1.3.4.1 Install the MultiApps CLI Plugin in the Cloud Foundry Environment

The MultiApps command line interface plugin (formerly known as the MTA plugin) for the Cloud Foundry command line interface lets you deploy, remove, and view MTAs, among other possible operations, by extending Cloud Foundry commands.

Prerequisites

You have installed the Cloud Foundry command line interface version 6.40 or higher. See Download and Install the Cloud Foundry Command Line Interface [page 2813].

Procedure

1. Open the command line interface or terminal.
2. Check if a previous version is installed by using the command **`cf plugins`**. If the `MtaPlugin` is already installed, you have to uninstall it using the command **`cf uninstall-plugin MtaPlugin`**.

   If you do not uninstall the previous version of the plugin and try to install a new one, you receive the following error:

   Output Code

   ```
   Plugin multiapps v<version> could not be installed as it contains commands
   with names and aliases that are already used:
   bg-deploy, deploy, download-mta-op-logs, mta, mta-ops, mtas, purge-mta-
   config, undeploy, dmol
   ```

3. To install the latest available version of the plugin, proceed as follows:
   1. Make sure that you have the Cloud Foundry community repository in your Cloud Foundry command line interface. If it is not available there, add it by executing the following command:
      `cf add-plugin-repo CF-Community https://plugins.cloudfoundry.org`
   2. To install the plugin, enter the following command:
      `cf install-plugin multiapps -f`

   Note

   Alternatively, if you want to install a specific version of the plugin, proceed as follows:

   1. Download the preferred version of the plugin that is compatible with your operating system.
   2. Extract the downloaded archive if required.
   3. To install the plugin, enter the following command:
      - Mac and Linux: **`cf install-plugin<path to the downloaded plugin> -f`**
      - Windows: **`cf install-plugin<path to the downloaded plugin> -f`**

4. Verify that the plugin has been installed successfully by entering `cf plugins`.

   You see a list of plugins that now includes the MultiApps CLI Plugin for the Cloud Foundry command line interface. The output also displays commands that are specific to this plugin.

Related Information

SAP BTP, Cloud Foundry CLI Plugins
Download and Install the Cloud Foundry Command Line Interface [page 2813]
MultiApps CLI Plugin

6.2.6.1.3.4.2 Multitarget Application Commands for the Cloud Foundry Environment

A list of additional commands to deploy multitarget applications (MTA) to the Cloud Foundry environment.

> **Note**
>
> The expiration time for all MTA operations in Cloud Foundry is 3 days. If an operation is still active when the time limit is reached, it is automatically aborted.

> **Caution**
>
> If you are using a version of the MultiApps CF CLI plugin that is older than 3.0.0, you need to make some additional configuration for deployments in regions China (Shanghai) and China (North 3). See MTA Deployment in Regions China (Shanghai) and China (North 3) [page 833].

Commands for the Cloud Foundry Environment Overview

Command	Alias	Description
`deploy`		Deploy a new multitarget application (MTA) or synchronize changes to an existing MTA.
`bg-deploy`		Deploy a multitarget application using “blue-green” (zero-downtime) deployment.
`rollback-mta` [page 2843]		Roll back to a previously deployed multitarget application (MTA) version with zero downtime.
`undeploy`		Undeploy a multitarget application (MTA).
`mta`		Display information about a deployed multitarget application (MTA).
`mtas`		List all deployed multitarget applications (MTA).
`mta-ops`		List all active operations for multitarget applications.
`download-mta-op-logs`	`dmol`	Download the log files for one or more operations concerning multitarget applications.

Command	Alias	Description
`purge-mta-config`		Purge all configuration entries and subscriptions, which are no longer valid.

> **Note**
>
> By default, large multitarget applications are not uploaded as a single unit, but are split up into smaller chunks of 45 MB that are uploaded separately. Configure the chunk size as described in Configuration .

deploy

Deploy a new MTA or synchronize changes to an existing one. You have the following options for doing this:

Usage

- **Deployment using a path to a directory**
  You have the option to deploy or synchronize an MTA, the source of which is contained in a local directory:

```
cf deploy MTA_ARCHIVE | DIRECTORY_PATH [-e EXT_DESCRIPTOR[,...]] [-t TIMEOUT]
[--version-rule VERSION_RULE] [-u URL] [-f] [--retries RETRIES] [--no-start]
[--namespace NAMESPACE]
[--apply-namespace-app-names true/false] [--apply-namespace-service-names
true/false]
[--apply-namespace-app-routes true/false] [--apply-namespace-as-suffix true/
false ]
[--delete-services] [--delete-service-keys] [--delete-service-brokers] [--
keep-files]
[--no-restart-subscribed-apps] [--do-not-fail-on-missing-permissions] [--
abort-on-error]
[--strategy STRATEGY] [--skip-testing-phase] [--skip-idle-start] [--apps-
start-timeout TIMEOUT]
[--apps-stage-timeout TIMEOUT] [--apps-upload-timeout TIMEOUT] [--apps-task-
execution-timeout TIMEOUT]
```

- **Deployment using a URL to the MTA archive**
  You have the option to deploy or synchronize an MTA, the source of which is contained at a URL address. When you use this command, the request prompts the backend to download the archive and then deploy it:

```
<write MTA archive URL to STDOUT> | cf deploy [-e EXT_DESCRIPTOR[,...]]
[-t TIMEOUT] [--version-rule VERSION_RULE] [-u MTA_CONTROLLER_URL] [--retries
RETRIES] [--no-start]
[--namespace NAMESPACE] [--apply-namespace-app-names true/false] [--apply-
namespace-service-names true/false]
[--apply-namespace-app-routes true/false] [--apply-namespace-as-suffix true/
false ] [--delete-services]
[--delete-service-keys] [--delete-service-brokers] [--keep-files] [--no-
restart-subscribed-apps]
[--do-not-fail-on-missing-permissions] [--abort-on-error] [--strategy
STRATEGY] [--skip-testing-phase]
[--skip-idle-start] [--apps-start-timeout TIMEOUT] [--apps-stage-timeout
TIMEOUT]
[--apps-upload-timeout TIMEOUT] [--apps-task-execution-timeout TIMEOUT]
```

> ### Note
>
> - The URL to the MTA archive must include the `<https://>` prefix.
> - You can use a locally present extension descriptor along with this deployment method.

> ### Restriction
>
> The feature has the following limitations:
>
> - There is no possibility for you to check if there are any ongoing operations on the MTA that you want to deploy using a URL.
> - The remote URL must have a standard PKI certificate. It is not possible to use self-signed certificates.
> - The URL cannot point to a protected GitHub URL, as GitHub does not support Basic authentication.

- **Deployment from your current directory**
  You have the option to deploy the content of the directory you are currently in. The MultiApps CLI plugin packages the content in accordance with the deployment descriptor definition, so that the Cloud Deployment service can then upload and deploy the MTA.

```
cf deploy [-e <EXT_DESCRIPTOR_1>[,<EXT_DESCRIPTOR_2>]]
[-u <URL>] [-t <TIMEOUT>] [-v <VERSION_RULE>]
[--no-start] [--namespace]
[--delete-services] [--delete-service-keys] [--delete-service-brokers]
[][--keep-files] [--no-restart-subscribed-apps] [--do-not-fail-on-missing-
permissions] [--skip-idle-start]
cf deploy ...[--retries <RETRIES>]
```

In addition to deployment, you can also interact with an active MTA deploy operation, for example, by performing an action:

```
cf deploy  [-i <OPERATION_ID>] [-a <ACTION>]
```

Arguments

Based on the contents of the `mtad.yaml` descriptor, the Cloud Deployment service assembles an MTA archive in that directory before deploying it.

Command Arguments Overview

Argument	Description
`<MTA_ARCHIVE>`	The path to (and name of) the archive or the directory containing the Multitarget Application to deploy. The application archive must have the format (and file extension) `mtar`, for example, `MTApp1.mtar`. If the value of this argument is a directory, the deploy-service checks if an mtad.yaml file exists in that directory, and assembles an MTA archive based on the file before deploying it. If no argument exists, the current directory is used.
`<DIRECTORY_PATH>`	The Cloud Deployment service also accepts a path to a directory where an `mtad.yaml` is maintained with path elements, for example, `cf deploy ./`.
`<--retries>`	Retry the operation a specified number of times in case a non-content error (default 3).

Options

Command Options Overview

Option	Description
`-e <EXT_DESCRIPTOR_1>[,<EXT_DESCRIPTOR_2>]`	Define one or more extensions to the deployment descriptors; multiple extension descriptors must be separated by commas.
`-u <URL>`	Specify the URL for the deploy-service endpoint that is to be used for the deployment operation.
`-t <TIMEOUT>`	Specify the maximum amount of time (in seconds) that the service must wait for before starting the deployed application.
`-v <VERSION_RULE>`	Specify the rule to apply to determine how the application version number is used to trigger an application-update deployment operation, for example: "`HIGHER`", "`SAME_HIGHER`", or "`ALL`".
`-i <OPERATION_ID>`	Specify the ID of the deploy operation that you want to perform an action on.
`-a <ACTION>`	Specify the action to perform on the deploy operation, for example, "abort", "retry", "monitor", or "resume".
`-f`	Force deployment without requiring any confirmation about aborting any conflicting processes.
`--no-start`	Do not start the application after deployment.
`--apps-upload-timeout <TIMEOUT>`	Defines how long, in seconds, you can upload your application binary before the MTA operation times out. See Application-Specific Timeouts [page 750] . **Note** This applies to all applications.
`--apps-stage-timeout <TIMEOUT>`	Defines how long, in seconds, your application can take during staging before the MTA operation times out. See Application-Specific Timeouts [page 750] . **Note** This applies to all applications.
`--apps-start-timeout <TIMEOUT>`	Defines how long, in seconds, your application can take to start before the MTA operation times out. See Application-Specific Timeouts [page 750] . **Note** This applies to all applications.

Option	Description
`--apps-task-execution-timeout <TIMEOUT>`	Defines how long, in seconds, your application can take to execute a task before the MTA operation times out. See Application-Specific Timeouts [page 750] . **Note** This applies to all applications.
`--delete-services`	Re-create changed services and delete discontinued services.
`--delete-service-keys`	Delete the existing service keys and apply the new ones.
`--delete-service-brokers`	Delete discontinued service brokers.
`--keep-files`	Keep the files used for deployment.
`--no-restart-subscribed-apps`	Do not restart the subscribed applications that are updated during the deployment.
`--do-not-fail-on-missing-permissions`	Perform the deployment, even if required administrator permissions are missing for some operations (for example, the creation of service brokers).
`--abort-on-error`	If an operation fails, the corresponding process is automatically aborted and cannot be retried using the option -a retry. However, if you run a new operation for the same MTA, you will not receive an error message that there is an ongoing process for the MTA, and you will not be asked if you want to abort it.
`-m <module name>`	Deploy only the module with the specified name. **Note** It can be used multiple times.
`--all-modules`	Deploy all modules. **Note** Any `-m` options are ignored.
`-r <resource name>`	Deploy only the resource with the specified name **Note** It can be used multiple times.
`--all-resources`	Deploy all resources. **Note** Any `-r` options are ignored.

Option	Description
`--strategy`	Announce to the platform a special deployment approach, for example when performing a “blue-green” or an “incremental-blue-green” deployment. See Blue-Green Deployment Strategy [page 788] and Incremental Blue-Green Deployment Strategy [page 791]. Using this option, you will not be asked to manually confirm the deletion of the older version of the MTA applications. This means that the deployment process is performed without any interruptions and you are not prompted to confirm the switch of routes to the new version of the MTA applications.
`--skip-testing-phase`	When using the --strategy option for “blue-green” or “incremental-blue-green” deployment, you can choose to skip the testing phase and you will not be asked to manually confirm the deletion of the older version of the MTA applications. The deployment process is performed without any interruptions and you are not prompted to confirm the switch of routes to the new version of the MTA applications. **Note** This option is equivalent to --no-confirm from “bg-deploy”.
`--namespace`	Namespace for the MTA. They are also applied to the application and service names. For more information, see Namespaces [page 798].
`--apply-namespace-app-names true/false`	Apply namespace to application names. If the namespace value is not provided in the CLI options, it is not applied. **Note** This applies to all applications.
`--apply-namespace-service-names true/false`	Apply namespace to service names. If the namespace value is not provided in the CLI options, it is not applied. **Note** This applies to all services.
`--apply-namespace-app-routes true/false`	Apply namespace to application routes. If the namespace value is not provided in the CLI options, it is not applied. **Note** This applies to all application routes.

Option	Description
`--apply-namespace-as-suffix true/false`	Apply namespace as suffix. If the namespace value is not provided in the CLI options, it is not applied. **Note** This applies to all applications, services, and application routes.
`--skip-idle-start`	When using the `--strategy` option for “blue-green” deployment, you can choose to skip the starting of the newly deployed applications on idle routes. This means that the newly deployed applications will be mapped directly to live routes. Under the hood, this option includes `--skip-testing-phase` .
`--require-secure-parameters`	Retrieves sensitive values from locally declared environment variables and ensures their secure handling during MTA deployment. For more information, see Sensitive Data Handling During MTA Deployment [page 811].
`--dependency-aware-stop-order`	When `--strategy` with "blue-green" or "incremental-blue-green" is used, `--dependency-aware-stop-order` ensures that during the final phase of the deployment, idle applications are stopped and restarted in a dependency-aware order based on the dependencies defined in the `deployed-after` attribute.
`--disposable-user-provided-service`	Determines that a disposable user-provided service with a randomly generated encryption key will be used for sensitive data encryption during MTA deployment. Sensitive Data Handling During MTA Deployment [page 811].

Note

- If any of the options `-m`, `--all-modules`, `-r`, `--all-resources` is used, only the specified modules and resources will be deployed. Otherwise, everything will be deployed by default.
- If the options for module deployment (`-m`, `--all-modules`) are used, the modules need to contain a `path` element on module level, which points to the content of the module. If the module has some `requires` dependency section to a resource that needs a configuration file, then the `requires` section should have a `path` parameter, which points to the configuration file.
- If the options for the resource deployment (`-r`, `--all-resources`) are used, then any resources that have some configuration files need to contain a `path` parameter in their `parameters` section, which points to the configuration file.
- The `path` element or parameter value should be relative to the MTA directory.

An example of an MTA deployment descriptor, containing all variants of the `path` elements and parameters can be found at Defining Multitarget Application Deployment Descriptors for Cloud Foundry [page 623].

bg-deploy

Deploy a new Multitarget Application (MTA) using “blue-green” (zero-downtime) deployment.

> ### Note
>
> You can also perform this deployment type using the `deploy` command by using the `--strategy blue-green` flag. See above for details.

"Blue-green" deployment is a release technique that reduces application downtime and the resulting risk by running two identical target deployment environments called "blue" and "green". Only one of the two target environments is "live" at any point in time and it is much easier to roll back to a previous version after a failed (or undesired) deployment.

Usage

Deploy a new Multitarget Application using "blue-green" deployment:

```
cf bg-deploy <MTA_ARCHIVE>
[-e <EXT_DESCRIPTOR_1>[,<EXT_DESCRIPTOR_2>]]
[-u <URL>] [-t <TIMEOUT>] [-v <VERSION_RULE>]
[--no-start] [--use-namespaces] [--no-namespaces-for-services]
[--delete-services] [--delete-service-keys] [--delete-service-brokers]
[--keep-files] [--no-restart-subscribed-apps] [--no-confirm] [--do-not-fail-on-
missing-permissions][--skip-idle-start]
cf deploy ...[--retries <RETRIES>]
```

Interact with an active MTA deploy operation, for example, by performing an action:

```
cf bg-deploy  [-i <OPERATION_ID>] [-a <ACTION>]
```

Arguments

Command Arguments Overview

Argument	Description

`<MTA_ARCHIVE>`
or
`<DIRECTORY_PATH>`
	The path to (and name of) the archive or the path to the directory containing the Multitarget Application to deploy. The application archive must have the format (and file extension) `mtar`, for example, `MTApp1.mtar`. The directory can be specified as a path (for example, `myApp/` or `.` (current directory)).

`<--retries>`	Retry the operation a specified number of times in case a non-content error (default 3).

Options

Command Options Overview

Option	Description
`-e <EXT_DESCRIPTOR_1>[,<EXT_DESCRIPTOR_2>]`	Define one or more extensions to the deployment descriptors; multiple extension descriptors must be separated by commas.
`-u <URL>`	Specify the URL for the deploy-service endpoint that is to be used for the deployment operation.
`-t <TIMEOUT>`	Specify the maximum amount of time (in seconds) that the service must wait for before starting the deployed application.

Option	Description
`-v <VERSION_RULE>`	Specify the rule to apply to determine how the application version number is used to trigger an application-update deployment operation, for example: “`HIGHER`”, “`SAME_HIGHER`”, or “`ALL`”.
`-i <OPERATION_ID>`	Specify the ID of the deploy operation that you want to perform an action on.
`-a <ACTION>`	Specify the action to perform on the deploy operation, for example, “abort”, “retry”, “monitor”, or “resume”.
`-f`	Force deploy without requiring any confirmation for aborting any conflicting processes.
`--no-start`	Do not start application after deployment.
`--use-namespaces`	Use namespaces in application (and service) names during application deployment.
`--no-namespaces-for-services`	Do not use namespaces in service names.
`--delete-services`	Re-create changed services and delete discontinued services.
`--delete-service-keys`	Delete the existing service keys and apply the new ones.
`--delete-service-brokers`	Delete discontinued service brokers.
`--keep-files`	Keep the files used for deployment.
`--no-restart-subscribed-apps`	Do not restart subscribed applications that are updated during the deployment.
`--no-confirm`	Use this option to turn off the manual confirmation for deleting the older version of the MTA applications. Thus the deployment process is performed from start to finish uninterrupted, and you are not prompted to confirm the switch of routes to the new version of the MTA applications.
`--do-not-fail-on-missing-permissions`	Perform the deployment, even if required administrator permissions are missing for some operations (for example, the creation of service brokers).
`--abort-on-error`	If an operation fails, the corresponding process is automatically aborted and cannot be retried using the option -a retry. However, if you run a new operation for the same MTA, you will not receive an error message that there is an ongoing process for the MTA, and you will not be asked if you want to abort it.
`-m <module name>`	Deploy only the module with the specified name. **Note** It can be used multiple times.
`--all-modules`	Deploy all modules. **Note** Any `-m` options are ignored.

Option	Description
`-r <resource name>`	Deploy only the resource with the specified name. **Note** It can be used multiple times.
`--all-resources`	Deploy all resources. **Note** Any `-r` options are ignored.
--skip-idle-start	You can choose to skip the starting of the newly deployed applications on idle routes. This means that the newly deployed applications will be mapped directly to live routes. Under the hood, this option includes --no-confirm.

Note

- If any of the options `-m`, `--all-modules`, `-r`, `--all-resources` is used, only the specified modules and resources will be deployed. Otherwise, everything will be deployed by default.
- If the options for the module deploying (`-m`, `--all-modules`) are used, the modules need to contain a `path` element on an MTA module level, which points to the content of the module. In case the module has some `requires` dependency section to a resource that needs a configuration file, then the `requires` section should have a `path` parameter, which points to the configuration file.
- If the options for the resource deploying (`-r`, `--all-resources`) are used, then any resources that have some configuration files need to contain a `path` parameter in their parameters section, which points to the configuration file.
- The `path` element or parameter value should be relative to the MTA directory.

See the complete procedure in section Legacy Blue-Green Deployment [page 785].

An example of an MTA deployment descriptor, containing all variants of the `path` elements and parameters can be found at Defining Multitarget Application Deployment Descriptors for Cloud Foundry [page 623].

rollback-mta

Roll back to a previously deployed multitarget application (MTA) version with zero downtime.

Note

The rollback of multitarget applications is currently experimental. We advise you first try it out in a non-productive environment.

Note

If you plan to use the backup and rollback features, make sure you have the MultiApps CLI Plugin version 3.5.0 or higher.

Rollback of MTA aims to quickly switch the productive traffic from the current MTA version to a previously deployed MTA version. For more information, see (Experimental) Rollback of Multitarget Applications [page 794].

Usage

Roll back your MTA to a previously deployed version:

```
cf rollback-mta MTA_ID [-t TIMEOUT] [-f] [--retries RETRIES]
[--namespace NAMESPACE] [--do-not-fail-on-missing-permissions] [--abort-on-
error]
[--apps-start-timeout TIMEOUT] [--apps-stage-timeout TIMEOUT] [--apps-upload-
timeout TIMEOUT]
[--apps-task-execution-timeout TIMEOUT]
```

Options

Command Options Overview

Option	Description
`--process-user-provided-services`	Enable processing of user-provided services during rollback.
`--namespace`	Namespace for the MTA. It is also applied to the application and service names. For more information, see Namespaces [page 798].
`-u <URL>`	Specify the URL for the deploy-service endpoint that is to be used for the deployment operation.
`-t <TIMEOUT>`	Specify the maximum amount of time (in seconds) that the service must wait for before starting the deployed application.
`--apps-stage-timeout <TIMEOUT>`	Define how long, in seconds, your application can take during staging before the MTA operation times out. See Application-Specific Timeouts [page 750]. **Note** This applies to all applications.
`--apps-start-timeout <TIMEOUT>`	Defines how long, in seconds, your application can take to start before the MTA operation times out. See Application-Specific Timeouts [page 750]. **Note** This applies to all applications.
`--apps-task-execution-timeout <TIMEOUT>`	Defines how long, in seconds, your application can take to execute a task before the MTA operation times out. See Application-Specific Timeouts [page 750]. **Note** This applies to all applications.
`-i <OPERATION_ID>`	Specify the ID of the deploy operation that you want to perform an action on.

Option	Description
`-a <ACTION>`	Specify the action to perform on the deploy operation, for example, “abort”, “retry”, “monitor”, or “resume”.
`-f`	Force deploy without requiring any confirmation for aborting any conflicting processes.
`--do-not-fail-on-missing-permissions`	Perform the deployment, even if required administrator permissions are missing for some operations (for example, the creation of service brokers).
`--abort-on-error`	If an operation fails, the corresponding process is automatically aborted and cannot be retried using the option `-a retry`.

undeploy

Undeploy a Multitarget Application (MTA), or interact with an undeploy MTA operation.

Usage

Undeploy an MTA.

```
cf undeploy <MTA_ID>
[-u <URL>] [-f]
[--delete-services] [--delete-service-keys] [--delete-service-brokers] [--no-restart-subscribed-apps]
[--do-not-fail-on-missing-permissions]
cf deploy ...[--retries <RETRIES>]
```

Interact with an undeploy MTA operation, for example, by performing an action.

```
cf undeploy [-i <UNDEPLOY_ID>] [-a <ACTION>]
```

Arguments

Command Arguments Overview

Argument	Description
`<MTA_ID>`	The ID of the MTA you want to undeploy
`<--retries>`	Retry the operation a specified number of times in case a non-content error (default 3).

Options

Command Options Overview

Option	Description
`-u <URL>`	Specify the URL for the service endpoint that is to be used for the undeployment operation.
`-i <OPERATION_ID>`	Specify the ID of the undeploy operation on which you want to perform an action.

Option	Description
`-a <ACTION>`	Specify the action to perform on the undeploy operation, for example, “abort”, “retry”, or “monitor”.
`-f`	Force completion of the undeploy operation without any system prompt or confirmation.
`--delete-services`	Delete any related services.
`--delete-service-brokers`	Delete discontinued service brokers.
`--no-restart-subscribed-apps`	Do not restart subscribed applications that are updated during the undeployment.
`--do-not-fail-on-missing-permissions`	Perform the undeployment, even if required administrator permissions are missing for some operations (for example, the creation of service brokers).
`--abort-on-error`	If an operation fails, the corresponding process is automatically aborted and cannot be retried using the option -a retry. However, if you run a new operation for the same MTA, you will not receive an error message that there is an ongoing process for the MTA, and you will not be asked if you want to abort it.
`--delete-service-keys`	Delete the existing service keys.

mta

Display information about a Multitarget Application (MTA). The information displayed includes the requested state, the number of instances, information about allocated memory and disk space, as well as details regarding the bound service (and service plan).

Usage

```
cf mta MTA_ID
[-u <URL>]
```

Arguments

Command Arguments Overview

Argument	Description
`<MTA_ID>`	The ID of the MTA whose details you want to display

Options

Command Options Overview

Option	Description
`-u <URL>`	Specify the URL for the deploy-service endpoint to use to obtain details of the selected MTA.

mtas

Display information about all available Multitarget Applications (MTA).

Usage

```
cf mtas [-u <URL>]
```

Options

Command Options Overview

Option	Description
`-u <URL>`	Specify the URL for the deploy-service endpoint to use to obtain details of the selected MTA.

mta-ops

Display information about all **active** operations for Multitarget Applications (MTA). The information includes the ID, type, status, the time the MTA-related operation started, as well as the name of the user that started the operation.

Usage

```
cf mta-ops [--mta <MTA>] [-u <URL>] [--last <NUM>] [--all]
```

Options

Command Options Overview

Option	Description
`-u <URL>`	Specify the URL for the deploy-service endpoint to use to obtain details of the selected MTA operations.
`--last <NUM>`	List the last `<NUM>` active MTA operations.
`--all`	List all active MTA operations.
`– mta <MTA>`	Specify the MTA whose operations you want to list.

download-mta-op-logs

Download the log files for one or more operations concerning Multitarget Applications.

Usage

You have the following usage options:

- `cf download-mta-op-logs -i <OPERATION_ID> [-u <URL>] [-d <DIRECTORY>]`
- `cf download-mta-op-logs --mta <MTA> [ --last <NUM>] [ -u <URL>] [ -d <DIRECTORY>]`

> → Tip
>
> You can use the alias `dmol` in place of the `download-mta-op-logs` command.

Options

Command Options Overview

Option	Description
`-u <URL>`	Specify the URL for the deploy-service endpoint to use to obtain details of the selected MTA operations.
`-i <OPERATION_ID>`	Specify the identity (ID) of the MTA operation whose logs you want to download.
`-d <DIRECTORY>`	Specify the path to the location where you want to save the downloaded MTA operation logs; by default, the location is `./mta-op-<OPERATION_ID>/`.
`--mta <MTA>`	Specify the MTA whose logs you want to download. **Note** This option is not compatible with the -i option.
`--last<NUM>`	Download the last <NUM> logs for the specified MTA.

purge-mta-config

Purge all configuration entries and subscriptions, which are no longer valid.

Usage

```
cf purge-mta-config
[-u <URL>]
```

Invalid configuration entries are often produced when the application that is providing configuration entries is deleted by the user without using the deploy-service, for example, the `cf delete` command. In this case, the configuration remains in the deploy-service database even though the corresponding application is no longer available. This could lead to a failure during subsequent attempts to resolve the configuration entries.

Options

Command Options Overview

Option	Description
`-u <URL>`	The URL of the deploy-service

Related Information

MultiApps CLI Plugin
Cloud Foundry MTA Examples Repository
Deploy an MTA archive referenced by a remote URL

6.2.6.1.3.4.2.1 Application Versions and Deployment Consistency

The SAP Cloud Deployment service follows the rules specified in the Semantic Versioning Specification (Semver) when comparing the versions of a deployed multitarget application (MTA) with the version that is to be deployed. If an MTA that is submitted for deployment has a version that is lower than or equal to an already deployed version of the same MTA, the deployment might fail if their is a conflict with the version rule specified in the deployment operation. The version rule is a parameter of the deployment process, which specifies which relationships to consider between the existing and the new version of an MTA before proceeding with the operation. The version rules supported by the deploy service are as follows:

- HIGHER - Only MTAs with versions that are bigger than the version of the currently deployed MTA, are accepted for deployment.
- SAME_HIGHER - Only MTAs with versions that are higher than (or the same as) the version of the currently deployed MTA are accepted for deployment. This is the **default** setting for the version rule.
- ALL - All versions are accepted for deployment.

6.2.6.1.3.5 Service Fabrik Plugin in Cloud Foundry

Service Fabrik CF CLI plugin is used for performing various backup and restore operations on service-instances in Cloud Foundry, such as starting/aborting a backup, listing all backups, removing backups, starting/aborting a restore, etc.

This CF CLI plugin is only available for ServiceFabrik broker, so it can only be used with CF installations in which this service broker is available. You can also list all available commands and their usage with 'cf backup'. You can now manage your backups of service instance and restore them using the backup and restore functionality.

To use the functionality, use the SAP BTP cockpit or the extended Cloud Foundry commands in the command line interface.

6.2.6.1.3.5.1 Install the Service Fabrik Plugin

The Service Fabrik plugin lets you manage backups of a service instance by extending Cloud Foundry commands.

Prerequisites

You need to have Cloud Foundry Command Line Interface (CF CLI) installed for the plugin to work since it is built on CF CLI. The installation instructions for CF CLI can be found here . The minimum version of CF CLI, on which the plugin has been tested successfully is v6.20.

Procedure

1. Download the latest version of the plugin that is compatible with your operating system. On the Web page, you will find the plugin under the **SAP Cloud Platform Cloud Foundry CLI Plugins** section with the name **Service Fabrik based B&R**.
2. Untar or unzip the downloaded archive.
3. Open the command line interface or terminal.
4. To install the plugin, enter the following command:
   - Mac and Linux: `cf install-plugin <path to the downloaded folder></service-fabrik-cli-plugin [Linux/Mac]>`
   - Windows : `cf install-plugin<path to the downloaded folder><\service-fabrik-cli-plugin.exe [Windows]>`

   > **Note**
   >
   > If you are reinstalling the plugin, first uninstall the previous version using: `cf uninstall-plugin ServiceFabrikPlugin`

5. Verify that the plugin has been installed successfully by entering `cf plugins`.

   You see a list of plugins that now includes Service Fabrik. The output also displays commands that are specific to the Service Fabrik plugin.

Related Information

https://tools.hana.ondemand.com/#cloud
Download and Install the Cloud Foundry Command Line Interface [page 2813]

6.2.6.1.3.5.2 Extended Cloud Foundry Commands of Service Fabrik

The Service Fabrik plugin provides commands that support backup and restore operations.

The various commands described in the table below are functionalities that facilitate these operations:

Usage	Description	Common Error Scenarios
`cf list-backup`	Shows a list of all the service instance backups. These backups are specific to the spaceyou are logged on to. If you have permission to access multiple spaces and need to view backups for a specific space, log on to that space in Cloud Foundry.	**Unauthorized Access:** if you do not have permission to access the space containing the service instance or the service instance itself. Verify that you have the required permission.
`cf list-backup <service_instance_name>`	Shows a list of backups that are specific to the service instance within a space.	**Unauthorized Access:** if you do not have permission to access the space containing the service instance or the service instance itself. Verify that you have the required permission.
`cf backup <backup_id>`	Provides additional information about a specific backup such as the service instance name for which backup was taken, the org and the space name to which the backup belongs, the type of backup, the status of backup, the start and finish time of backup, and so on.	none
`cf list-backup --guid <service_instance_guid>`	Shows the list of all backups for the given service-fabrik service instance. The argument has to be the guid of the service instance. (Works even for a deleted instance.)	**Unauthorized Access:** if you do not have permission to access the space containing the service instance or the service instance itself. Verify that you have the required permission.
`cf list-backup<service_instance_name> --deleted`	Shows the list of all backups for a deleted service-fabrik service instance. (Works only for a deleted service-instance.)	**Unauthorized Access:** if you do not have permission to access the space containing the service instance or the service instance itself. Verify that you have the required permission. **Deleted instance not found error:** If the deleted instance name is incorrect. **Multiple Guid for a deleted service instance error:** When plugin encounters multiple instance Guid's associated with the given instance name

Usage	Description	Common Error Scenarios
`cf start-restore <service_instance_name> <backup_id>`	Restores a service instance from the specified instance name and backup ID. Before providing a restore command, ensure that the backup is available for the service instance. You can verify the state of the restore process using `cf restore <service_instance_name>`.	**Unauthorized Access:** if you do not have permission to access the space containing the service instance or the service instance itself. Verify that you have the required permission. **Another concurrent access:** if another operation is already in progress for the service instance. You might need to try again after the current operation is completed.
`cf abort-restore <service_instance_name>`	Aborts an ongoing restore operation of the specified service instance. You can verify the state of the abort process using `cf restore <service_instance_name>`.	**No restore in progress:** if no restore operation is currently in progress for the specified service instance. The restore activity might have completed or you might not have initiated a restore. **Unauthorized Access:** if you do not have permission to access the space containing the service instance or the service instance itself. Verify that you have the required permission.
`cf instance-events --delete`	List all delete service instance events in the space.	**Unauthorized Access:** if you do not have permission to access the space containing the service instance or the service instance itself. Verify that you have the required permission.
`cf restore<service_instance_name>`	Check the progress of the last restore operation triggered on the specified instance name. Provides additional information about the restore operation such as the service instance name for which restore was taken, the org and the space name to which the instance belongs, the status of restore, the start and finish time of restore, and so on.	**No restore in progress**: if no restore operation is currently in progress for the specified service instance. The restore activity might have completed or you might not have initiated a restore. **Unauthorized Access**: if you do not have permission to access the space containing the service instance or the service instance itself. Verify that you have the required permission.

6.2.6.2 Add Organization Members Using the Cloud Foundry Command Line Interface

You can use the Cloud Foundry Command Line Interface (cf CLI) to add organization members and assign roles to them.

Prerequisites

- The Cloud Foundry environment is enabled.
  Enable it in the SAP BTP cockpit. For more information, see Create a Subaccount [page 2509].
- The Org Manager role for the org in question.

  > **Note**
  >
  > You automatically have the Org Manager role in an org that you created.

- The user exists in the UAA.
  All users are stored in identity providers, either in the default or in a custom identity provider. Cloud Foundry needs a copy of the user, sometimes called a shadow user. To create the shadow user, choose one of the following options:
  - The user has been added from the cockpit.
    For more information, see Add Org Members [page 2795].
  - The user has already logged on to the Cloud Foundry environment.
    For example: `cf login -a https://api.cf.eu20.hana.ondemand.com`

    > **Note**
    >
    > The landscape the user logs on to must be the same as the landscape where the subaccount is located. See the overview page of your subaccount for the API endpoint.

- You have the e-mail addresses of the members that you want to add.
  The user must already exist in the connected identity provider. For more information, see Trust and Federation with Identity Providers [page 2538].
- Download and install the cf CLI and log on to Cloud Foundry. For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813] and Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].

Context

This task is meant for the `OrgManager` and `OrgAuditor` roles. To add members with the `OrgUser` role, follow the procedure described in Add Space Members Using the Cloud Foundry Command Line Interface [page 2855].

Procedure

Enter the following command, specifying the user name, the name of the organization, and the role:

```
cf set-org-role <USERNAME> <ORG> <ROLE>
```

```
cf set-org-role julie.armstrong@example.com account_subaccount-47v2w8dj
OrgAuditor
```

Next Steps

To remove an org role from a user, enter the following string, specifying the user name, the name of the organization, and the role:

```
cf unset-org-role <USERNAME> <ORG> <ROLE>
```

Related Information

Supported Tools and Services When Using Custom Identity Providers for Platform Users [page 2585]
About Roles in the Cloud Foundry Environment [page 2788]
Managing Org Members Using the Cockpit [page 2795]

6.2.6.3 Create Spaces Using the Cloud Foundry Command Line Interface

Use the `cf create-space` command to create spaces in your Cloud Foundry organization using the Cloud Foundry Command Line Interface (cf CLI).

Prerequisites

- (Enterprise accounts only) Create at least one subaccount and enable the Cloud Foundry environment in this subaccount. For more information, see Create a Subaccount [page 2509] and Create Orgs [page 2793].

  > **Note**
  >
  > In a trial account, the Cloud Foundry environment is automatically activated, and a first space named *dev* is created.

- You have the Org Manager role in the organization in which you want to create a space. For more information about roles and permissions, see https://docs.cloudfoundry.org/concepts/roles.html .
- Download and install the cf CLI and log on to Cloud Foundry. For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813] and Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].

Procedure

1. Make sure that you are in the Cloud Foundry organization you want to add a space to.

   ```
   cf target -o ORG
   ```

   To get a list of your organizations, call `cf orgs`.
2. Enter the following string, specifying a name for the new space, the name of the organization, and the quota you'd like to assign to it:

   ```
   cf create-space SPACE [-o ORG] [-q SPACE_QUOTA]
   ```
3. (Optional) Set this space as target by executing: **`cf target -o ORG -s SPACE`**

   > **Note**
   >
   > If you are assigned to only one Cloud Foundry organization and space, the system automatically targets you to the relevant Cloud Foundry organization and space when you log on.

Related Information

Delete Spaces Using the Cloud Foundry Command Line Interface [page 2859]
Create Spaces Using the Cockpit [page 2800]

6.2.6.4 Add Space Members Using the Cloud Foundry Command Line Interface

You can use the Cloud Foundry Command Line Interface (cf CLI) to add space members and assign roles to them.

Prerequisites

- The Space Manager role for the space.

- The users exist in a trusted platform identity provider.
  All users of Cloud Foundry UAA are stored in identity providers, either in the default or in a custom identity provider. UAA needs a copy of the user, sometimes called a shadow user. You assign the shadow user authorizations to access resources in UAA. When a user authenticates, UAA forwards the request to the identity provider.
  To specify which identity provider hosts the user, use the `--origin` option. For more information about finding the origin key, see the trust configuration documentation.
  For more information, see Trust and Federation with Identity Providers [page 2538].
- Download and install the cf CLI and log on to Cloud Foundry. For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813] and Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].

Context

Space members are organization members who have specific roles in a space.

Org members can only be added by an Org Manager. This means that if you only have the Space Manager role, you can't add space members that aren't known to the org. To do that, you must first ask your Org Manager to add the users as org members with no roles. Once this is done, you can add them as space members following the steps below.

If you're the Org Manager, you don't need to first add the users as org members with no roles. Since you have the permissions necessary to add users to the org, when you add a new user as a space member, that user automatically becomes part of the org as well.

Procedure

Enter the following string, specifying the user name, the name of the organization, the name of the space, and the role:

```
cf set-space-role USERNAME ORG SPACE ROLE
```

Next Steps

To remove a space role from a user, enter the following string, specifying the user name, the name of the organization, the name of the space, and the role:

```
cf unset-space-role USERNAME ORG SPACE ROLE
```

Related Information

Supported Tools and Services When Using Custom Identity Providers for Platform Users [page 2585]
Add Space Members Using the Cockpit [page 2803]

6.2.6.5 Create Space Quota Plans Using the Cloud Foundry Command Line Interface

You can use the Cloud Foundry Command Line Interface to create space quota plans.

Prerequisites

- The Org Manager role for the org that contains the spaces to which you want to assign quotas.
- Download and install the cf CLI and log on to Cloud Foundry. For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813] and Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].

Procedure

Open a command line and enter the following string, replacing `QUOTA` with the name for your space quota plan:

```
cf create-space-quota QUOTA [-i INSTANCE_MEMORY] [-m MEMORY] [-r ROUTES] [-s
SERVICE_INSTANCES] [--allow-paid-service-plan | --disallow-paid-service-plans]
```

Specify the following parameters:

- `-i`: Maximum amount of memory an application instance can have (`-1` represents an unlimited amount)
- `-m`: Total amount of memory a space can have
- `-r`: Total number of routes
- `-s`: Total number of service instances
- `--allow-paid-service-plans`: Can provision instances of paid service plans
- `--disallow-paid-service-plans`: Can not provision instances of paid service plans

Related Information

Create Space Quotas [page 2808]

6.2.6.6 Assign Quota Plans to Spaces Using the Cloud Foundry Command Line Interface

You use the Cloud Foundry Command Line Interface to assign the quotas available in your global account to your subaccounts.

Prerequisites

- The Org Manager role for the org that contains the spaces to which you want to assign quotas.
- Download and install the cf CLI and log on to Cloud Foundry. For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813] and Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].
- A space quota plan. For more information, see Create Space Quota Plans Using the Cloud Foundry Command Line Interface [page 2857].

Procedure

Open a command line and enter the following string:

```
cf set-space-quota SPACE_NAME SPACE_QUOTA_NAME
```

6.2.6.7 Change Space Quota Plans Using the Command Line Interface

Change space quota plans in the Cloud Foundry environment using the Cloud Foundry command line interface (cf CLI).

Prerequisites

- You have the Org Manager role in the organization in which you'd like to change space quota plans. For more information about roles and permissions, see https://docs.cloudfoundry.org/concepts/roles.html .
- Download and install the cf CLI and log on to your Cloud Foundry instance. For more information, see Download and Install Cloud Foundry Command Line Interface and Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].

Procedure

1. (Optional) Enter the following string to identify the names of all space quota plans available in your org:

```
cf space-quotas
```

2. To modify a quota plan, enter the following string:

```
cf update-space-quota SPACE-QUOTA-NAME [-i MAX-INSTANCE-MEMORY] [-m MEMORY]
[-n NEW_NAME] [-r ROUTES] [-s SERVICES] [--allow-paid-service-plans | --
disallow-paid-service-plans]
```

> ⓘ Note
>
> For more information about managing space quota plans, see https://docs.cloudfoundry.org/adminguide/quota-plans.html#space .

Related Information

Edit Space Quotas [page 2811]
Configure Entitlements and Quotas for Subaccounts [page 2529]
Managing Entitlements and Quotas Using the Cockpit [page 2521]

6.2.6.8 Delete Spaces Using the Cloud Foundry Command Line Interface

Use the `cf delete-space` command to delete spaces in your Cloud Foundry organization using the Cloud Foundry Command Line Interface (cf CLI).

Prerequisites

- You have the Org Manager role in the organization from which you want to delete a space. For more information about roles and permissions, see https://docs.cloudfoundry.org/concepts/roles.html .
- Download and install the cf CLI and log on to Cloud Foundry. For more information, see Download and Install the Cloud Foundry Command Line Interface [page 2813] and Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].

Procedure

1. Make sure that you are in the Cloud Foundry organization you want to delete a space in.

   ```
   cf target -o ORG
   ```

   To get a list of your organizations, call `cf orgs`.
2. Enter the following string, specifying a name for the space, and the name of the organization:

   ```
   cf delete-space SPACE [-o ORG] [-f]
   ```

   To get the list of your spaces, call `cf spaces`.

Related Information

Create Spaces Using the Cloud Foundry Command Line Interface [page 2854]
Delete Spaces Using the Cockpit [page 2802]

6.2.7 Application Operations in the Cloud Foundry Environment

Learn more about the different application operations that you can perform in the Cloud Foundry environment.

- Manage Environment Variables [page 2861]
- Deploy an Application [page 2862]
- Start, Stop, and Restart Applications [page 2864]
- Restage an Application [page 2865]
- Manage Application Instances [page 2867]
- Bind an Application to a Service [page 2867]
- Troubleshoot Applications [page 2869]
  - Using Application Events [page 2870]
  - Using Application Logs [page 2871]
- Configuring Application URLs in the Cloud Foundry CLI [page 2871]
- About Routes in the Cockpit [page 2873]
  - Create Routes [page 2874]
  - Map Routes to Applications [page 2875]
  - Bind Routes to Service Instances [page 2876]
- Security Groups [page 2877]
- Using Custom Domains [page 2878]

6.2.7.1 Manage Environment Variables

You can use the SAP BTP cockpit to view system-provided and create user-provided variables. The Cloud Foundry environment uses these variables to communicate with the applications deployed in it.

Environment Variables

In the Cloud Foundry environment, there are two types of environment variables:

- System-provided
  The system-provided variables are provided by the Cloud Foundry environment.
  They are split into `VCAP_APPLICATION` and `VCAP_SERVICES`:
  - `VCAP_APPLICATION` contain the attributes of your deployed application.
  - `VCAP_SERVICES` contain information about the service instances bound to your application, if any.
- User-provided
  You can create your own custom environment variables as key-value pairs. For more information, see Create User-Provided Variables [page 2862].

Environment Variable Groups

There are two types of environment variable groups:

- Running
  All variables available to your application at runtime.
- Staging
  All variables available to your application at the time of staging.

View System-Provided Variables and Environment Variable Groups

Prerequisites

- You must have at least one application deployed in your Cloud Foundry space.
- You have the Space Developer or Space Supporter role.

Procedure

1. In the SAP BTP cockpit navigate to the *Applications* page in your Cloud Foundry space.
2. Choose the application whose environment variables you want to view.
3. Choose *Environment Variables*.

Create User-Provided Variables

Prerequisites

- You must have at least one application deployed in your Cloud Foundry space.
- You have the Space Developer or Space Supporter role.

Procedure

1. In the SAP BTP cockpit, navigate to the *Applications* page in your Cloud Foundry space.
2. Choose the application for which you want to create a user-provided variable.
3. Choose *User-Provided Variables*.
4. Choose *Create Variable*.
5. Provide a *Key* and a *Value*.
   These values are case-sensitive.
6. Choose *Create*.

Related Information

https://docs.cloudfoundry.org/devguide/deploy-apps/environment-variable.html

6.2.7.2 Deploy an Application

You can use the SAP BTP cockpit to deploy a new application in the Cloud Foundry environment.

Prerequisites

- You have the Space Developer role. For more information, see About Roles in the Cloud Foundry Environment [page 2788].
- You have enough space quota, or enough org quota if no space quota is assigned to the space. For more information, see Managing Space Quotas [page 2806].

Procedure

1. In your Cloud Foundry subaccount, navigate to the space where you would like to deploy your application.
2. On the *Applications* page, choose *Deploy Application*.
3. Choose the location of the file which contains your application.
4. (Optional) If you would like to use a manifest, choose the location of your `manifest.yml` file.

> ### Note
>
> To avoid any unexpected behavior, specify the attributes of your choice and their respective values in the `manifest.yml` file. For reference, see https://docs.cloudfoundry.org/devguide/deploy-apps/manifest-attributes.html .

5. (Optional) If you don't want to use a manifest, select the *Custom Settings* radio button and enter the application details.
   a. Enter a name for your application.
   b. (Optional) Edit the amount of memory and disk space available to each instance of your application, as well as the number of instances.

      The amount of memory per instance that you assign, in this step or in the `manifest.yml` file during deployment, is the value that is metered when using the Cloud Foundry runtime service. For more information, see the examples in Monitoring and Troubleshooting.

      > ### Note
      >
      > By default, each instance of a new application is assigned 1024 MB of memory and 512 MB of disk space, and each application starts with 1 instance. If you require more or less resources, edit the prefilled fields in the form to suit your needs.

   c. (Optional) If you don't need a route for your application, switch off *Create Route*.
   d. (Optional) If you would like to create a route for your application, leave *Create Route* switched on and choose a *Host* name (if different from your application name) and a *Domain* for your application.

      > ### Note
      >
      > When you enter an application name, the *Host* field is automatically filled with the same name. You can make changes to it or leave it as is. After deciding on a host and domain, you can see a preview of your final *Application Route* at the bottom of the form.

6. Choose *Deploy*.

Results

The file containing your new application is uploaded and your application is deployed.

> ### Note
>
> If an application is displayed as *Started* according to the *Requested State* column, it doesn't automatically mean that the application is started. This is the desired state, but there might be an issue with an application instance. For any warning alerts, check the *Instances* column.
>
> To get more detailed information about the current state of your application instances, navigate to the *Application Overview* page.

Related Information

Deploying to the Cloud Foundry Environment [page 614]
About Routes in the Cockpit [page 2873]
Create Routes [page 2874]

6.2.7.3 Start, Stop, and Restart Applications

You can start and stop applications in the Cloud Foundry environment in the SAP BTP cockpit to control whether they can be accessed by end users.

Prerequisites

- You have deployed an app in your Cloud Foundry space.
- You have the Space Developer or Space Supporter role.

Context

A user can only reach your application if the application is started.

> ⓘ Note
>
> On the *Applications* page in the cockpit, if an application is displayed as *Started* according to the *Requested State* column, it doesn't automatically mean that the application is started. This is the desired state, but there might be an issue with an application instance. For any warning alerts, check the *Instances* column.
>
> To get more detailed information about the current state of your application instances, navigate to the *Application Overview* page of the application.

The application routes do not work when the application is stopped.

The first start of the application occurs when you deploy the app, if enough quota is available in the space.

Procedure

1. In the SAP BTP cockpit, navigate to the *Applications* page in your space.
2. You can perform these actions either directly from the *Applications* page, or by navigating into your application, to the *Application Overview* page:

Page	Actions
Applications **page**	For the application that you'd like to start or stop, choose the respective icon from the *Actions* column: • ▷ (*Start*) - This starts all instances of your application and makes them available to users. • ⦿ (*Stop*) - This stops all running instances of the application. You cannot directly restart an application from this page. You can restage your application by choosing *Restage Application*. For more information, see Restage an Application [page 2865]. ⚠ Caution Restaging applications causes downtime.
Application Overview **page**	Choose the button corresponding to the desired action: • *Start* - This starts all instances of your application and makes them available to users. • *Stop* - This stops all running instances of the application. • *Restart* - This restarts the application. ⚠ Caution Restarting applications causes downtime.

Related Information

Deploy an Application [page 2862]

6.2.7.4 Restage an Application

You can use the SAP BTP cockpit to restage an application deployed to the Cloud Foundry environment.

Prerequisites

- You have either the Space Developer or Space Supporter role. For more information, see About Roles in the Cloud Foundry Environment [page 2788].
- You've prepared for the downtime caused by restaging the application.

Context

You need to restage your application when environmental changes might affect how it runs, even if you have not changed the application code. Restaging is useful when runtime dependencies change, such as when you bind your application to a new service or you modify environment variables. You also use it when buildpacks or stacks are updated.

Restaging:

- Integrates updated configurations or dependencies with your current code. This process re-compiles the application so that the new droplet reflects these changes.
- Helps ensure that the performance of your application matches the latest environment settings and service configurations.

Procedure

1. In your Cloud Foundry subaccount, navigate to the space where you would like to restage your application.
2. On the *Applications* page, choose *Restage Application*.
3. Choose an application from the dropdown menu.

   The dropdown contains all applications deployed in the space.
4. Choose *Restage* to confirm the action.

Results

Similar to restarting, it may take several minutes for the application to start working again.

Related Information

Application Operations in the Cloud Foundry Environment [page 2860]

6.2.7.5 Manage Application Instances

To increase the availability of your Cloud Foundry application, you can run multiple instances of it. In the SAP BTP cockpit, you can add or remove application instances, or change the size of their memory and disk.

Prerequisites

- You must have one application deployed in your Cloud Foundry space.
- You have the Space Developer or Space Supporter role.

Procedure

1. In the SAP BTP cockpit, navigate to the *Applications* page in your Cloud Foundry space.
2. Choose the application whose instances you want to manage.
3. On the *Application Overview* page, you can:
   - *Change Configuration* to update the memory size and the disk size of all instances of your application.

     > ⓘ Note
     >
     > Each instance of the application receives the size in megabytes you have specified. You can allocate instance memory up to the available space memory limit.

   - *Bind Application Autoscaler* to scale your application by binding it to the Application Autoscaler service. Once it's bound, you can access the Application Autoscaler dashboard from the *Instances Configuration* section.
   - *Add Instance* to start an additional instance of your application.
   - *Remove Instance* to remove one of the running instances of your application.

Related Information

Deploy an Application [page 2862]

6.2.7.6 Bind an Application to a Service

In the SAP BTP cockpit, you can bind applications in the Cloud Foundry environment to services by binding them to available service instances. If no service instances are available, you can create a new service instance and bind the application to it directly from the *Service Bindings* page at the application level. Service bindings are used for configuring and delivering access credentials to your applications.

Prerequisites

- You have the Space Developer or the Space Supporter role. For more information, see About Roles in the Cloud Foundry Environment [page 2788].
- You have deployed an application in the same space where you plan to create the service binding. For more information, see Deploy an Application [page 2862].
- You have created a service instance and haven't bound your application to it yet.

Context

On the *Service Bindings* page in the cockpit, you bind an application to a service using the *Bind Service Instance* wizard. To start the wizard, you choose *Bind Service Instance*.

The first step, *Choose Service Instance*, provides a table with all services instances you can select and bind to your application.

> → Tip
>
> If you're looking for a specific service instance in the table and it isn't there, it might already be bound to your application or it may no longer exist.

To summarize, you use the wizard to either:

- Find the service instance you need in the table with unbound service instances and bind it.
- Create a new service instance and then bind it.

Here is a detailed description of both procedures:

Option 1: Bind an Existing Service Instance

1. In the SAP BTP cockpit, navigate to the *Applications* page in your Cloud Foundry space.
2. Choose the application for which you want to create a service binding.
3. Choose *Service Bindings*.
4. Choose *Bind Service Instance*. This action starts the *Bind Service Instance* wizard.
5. Find the service instance in the table and choose *Bind*.
   If you can't find it, create and bind a new service instance as instructed in **Option 2**.
6. (Optional) Provide JSON parameters either manually or by importing a JSON file.
7. Choose *Next Step*.
8. Review all the details you've provided so far. You can always go back and make changes, if necessary.
9. Choose *Bind*.

Option 2: Create and Bind a New Service Instance

1. In the SAP BTP cockpit, navigate to the *Applications* page in your Cloud Foundry space.

   > ⓘ Note
   >
   > Alternatively, you can create and bind service instances from the *Services* > *Instances and Subscriptions* page in the cockpit. For more information, see Service Instances and Binding Service Instances to Cloud Foundry Applications.

2. Choose the application for which you want to create a service binding.
3. Choose *Service Bindings*.
4. Choose *Bind Service Instance*. This action starts the *Bind Service Instance* wizard.
5. Choose *Create Service Instance*.
6. Select a *Service* and a *Plan*, and provide an *Instance Name*. Then, choose *Next Step*.
7. (Optional) Provide JSON parameters either manually or by importing a JSON file.
8. Choose *Next Step*.
9. Review all the details you've provided so far. You can always go back and make changes, if necessary.
10. Choose *Create and Bind*.

Results

The newly created service binding appears as the top entry in the *Service Bindings* table.

Related Information

Bind Service Instances to Applications Using the Cloud Foundry Command Line Interface [page 277]
Service Instances
Binding Service Instances to Cloud Foundry Applications

6.2.7.7 Troubleshoot Applications

You can use application events and logs to troubleshoot your application in the SAP BTP cockpit.

Procedure

1. In the SAP BTP cockpit, navigate to the *Applications* page in your Cloud Foundry space.
2. Choose the application whose events or logs you want to manage.
3. Navigate to the *Events* or the *Logs* page.

   For more information, see:
   - Using Application Events [page 2870]
   - Using Application Logs [page 2871]

6.2.7.7.1 Using Application Events

The *Application Events* page at application level in the SAP BTP cockpit shows the order in which actions are performed on an application. You can use these events to debug your application and analyze its state.

UI Reference

UI Component	Role
Time	Shows the timestamp at which an event is created and logged.
Event	Shows the type of the event. The application events use the `audit.app.*` naming pattern. For more information, see https://docs.cloudfoundry.org/running/managing-cf/audit-events.html#app-lifecycle .
Actor	Shows who triggered the event. It can be either you or Cloud Foundry.
Target	Shows where the event was triggered. Provides the name of the application.
Description	Provides technical details about the event.

Related Information

Troubleshoot Applications [page 2869]
Using Application Logs [page 2871]

6.2.7.7.2 Using Application Logs

The *Logs* page at application level in the SAP BTP cockpit shows the application logs and logs generated by various Cloud Foundry components that relate to the application. You can use these logs to identify and troubleshoot application-specific problems.

UI Reference

UI Component	Role
Time	Shows the timestamp at which the log is generated.
Log Type	Shows the type of the log according to Cloud Foundry. It can be either *STDOUT* or *STDERR*.
Source	Shows the Cloud Foundry component code and the instance number. For example, *[APP/1]* indicates that application instance 1 has generated the log.
Message	Allows you to view the log message in a message box. To open the message box, choose *More Details*.

Related Information

https://docs.cloudfoundry.org/devguide/deploy-apps/streaming-logs.html
Troubleshoot Applications [page 2869]
Using Application Events [page 2870]

6.2.7.8 Configuring Application URLs in the Cloud Foundry CLI

By default, all applications running on SAP BTP are accessed on the **default landscape** domain. According to your needs, you can change the default application URL by configuring additional application domains.

The URL for an application deployed on SAP BTP in the Cloud Foundry environment is `https://<application>.cfapps.<region>.hana.ondemand.com`. The domain depends on your location, in the European region, for example, the domain is `cfapps.eu10.hana.ondemand.com`. So, if you're deploying an application with the name "myapp", the default application URL is `https://myapp.cfapps.eu10.hana.ondemand.com`.

Running on the China (Shanghai) region:

There's no default URL available in China, therefore you can't deploy an application without configuring a custom domain first. Please refer to the related information link on how to use custom domains.

> **Note**
>
> To check what domains are available in your Cloud Foundry organization, use the command `cf domains`.

Custom Domains

Use custom domains to reach your applications on your own domain instead of the default domain, for example, `https://myapp.mydomain.com` instead of `https://myapp.cfapps.eu10.hana.ondemand.com`. When you use a custom domain, both the domain name and the TLS/SSL certificate of the server are owned by the customer.

You can configure custom domains using the Cloud Foundry command-line interface with the plugin for custom domains.

For more information, read the Custom Domain service user guide.

Application Routes

If you want to make your application reachable on another route, you can add additional routes using the Cloud Foundry CLI. If your application is available under the route `https://myapp.cfapps.eu10.hana.ondemand.com`, you could add a route that reads `https://expenses.cfapps.eu10.hana.ondemand.com` that leads to the same application.

Related Information

Using Custom Domains [page 2878]
About Routes in the Cockpit [page 2873]
Configuring Application URLs (Neo environment)

6.2.7.9 About Routes in the Cockpit

To enable your end users to reach your application, create a route and map it to the application in the SAP BTP cockpit.

What Is a Route?

A route is the URL that enables your end users to reach your application. Routes belong to a space, and therefore are managed at space level.

The Router component in Cloud Foundry is responsible for routing routes. It maintains a list of mapped applications and compares each request with the list to find the best match. Based on this comparison, it then routes requests to the appropriate application instance.

Currently, you can create, map, and delete HTTP routes. An HTTP route includes a domain, a host name (or subdomain), and an optional path.

Creating Routes

The number of routes you can create in a space depends on your subaccount entitlements and quotas, or on the space quota assigned to that space.

For more information, see Create Routes [page 2874].

Mapping Routes to Applications

You can only map a route to an application that belongs to the same space.

It is possible to map a single route to multiple applications, as well as multiple routes to a single application.

For more information, see Map Routes to Applications [page 2875].

Binding Routes to Service Instances

To bind a route to a service instance in the SAP BTP cockpit, see Bind Routes to Service Instances [page 2876].

6.2.7.9.1 Create Routes

You can configure the URLs through which end users can reach your applications.

Prerequisites

You must have the Space Developer or the Space Supporter role.

Context

Routes belong to a space but they're globally unique, regardless of the organization that controls a space. If a route with a URL exists, you can't create a route with the same URL.

Procedure

1. Navigate to your Cloud Foundry space.
2. Choose *Routes* from the left hand-side navigation.
3. Choose *Create Route*.
4. In the dialog, enter the following parameters:

Parameter	Details
Domain	From the dropdown menu, you can choose either a shared domain (for example, the default **`cfapps.<region>.hana.ondemand.com`**) or a private domain that you've previously created using the CF CLI. From the dropdown menu, you can choose either a shared domain or a private domain that you've previously created using the CF CLI. For more information on private domains, see https://docs.cloudfoundry.org/devguide/deploy-apps/routes-domains.html#private-domains .
Host Name	The host name is your desired subdomain. In the URL, it's added before the selected domain, as follows: `https://<host name>.<domain>` The host name can contain up to 63 characters.
Path (Optional)	In addition to the domain and subdomain, you can also add a path. You can use paths if you want to create routes for multiple applications available for the same host name and domain. The path becomes part of the URL as follows: `https://<host name>.<domain>/<path>`

You can see the preview of your route at the bottom of the dialog.

5. When you're happy with your route, choose *Create*.

Next Steps

- Once you've created a route, you must map it to your application. For more information, see Map Routes to Applications [page 2875].
- Additionally, you also have the option to bind it to a route service instance by choosing *(Bind Route Service)* from the *Actions* column. For more information, see Bind Routes to Service Instances [page 2876].

Related Information

About Routes in the Cockpit [page 2873]

6.2.7.9.2 Map Routes to Applications

Once a route has been created, you must map it to an application to make this application reachable for end users.

Prerequisites

- You have at least one route and one deployed application in the same Cloud Foundry space.
- You must have the Space Developer or the Space Supporter role.

Procedure

1. Navigate to the *Routes* page in your Cloud Foundry space in the cockpit.

   > **Note**
   >
   > You can also map routes from the *Application Overview* page. To unmap routes, use the *Application Overview* page. The *Routes* page does not support unmapping routes.

2. For the route that you wish to map to an application, choose *(Map Route)* from the *Actions* column and select the application you want to map it to.

Results

Your application can now be accessed via the route mapped to it. You can launch your mapped route from two different places in the cockpit:

- *Routes* page:
  Choose *(Launch Route)* from the *Actions* column.
- *Overview* page of the mapped application:
  Choose the URL from the *Application Routes* section.

Related Information

About Routes in the Cockpit [page 2873]
Create Routes [page 2874]

6.2.7.9.3 Bind Routes to Service Instances

You can bind a route to a service instance in the SAP BTP cockpit.

Prerequisites

In SAP BTP, Cloud Foundry environment, only user-provided and fully-brokered services are currently supported.

- You must have the Space Developer or the Space Supporter role.
- The service instance and the route are in the same Cloud Foundry space.
- To bind a route to a user-provided service instance, the service instance must include the URL of the route service. To include it, set the `route_service_url` property.
- To bind a route to a managed service instance, make sure this service can be bound by setting route forwarding in the `requires` property:
  `requires=[route_forwarding]`

Context

Before you proceed with the route binding, keep in mind the following:

- You can have a route bound to an application and a service instance at the same time.
- You can bind one or multiple routes to a single service instance.

Now that you're aware of these specifics, let's continue with the route binding steps.

Procedure

1. Navigate to the *Routes* page in your Cloud Foundry space in the cockpit.
2. Choose *(Bind Route Service)* from the *Actions* column.
3. Select a service instance from the dropdown.

   The dropdown contains a list of all service instances that you can bind to the route. If an existing service instance can't be bound to a route, you won't see it in the dropdown.

   If you want to create a new service instance from your services available in the Service Marketplace, see Creating Service Instances in Cloud Foundry.

   To create a new user-provided service instance, see Creating User-Provided Service Instances in Cloud Foundry Environment.
4. (Optional) If you want to customize the route binding service, mark the *Specify Parameters* checkbox.

   You have the option to either upload your own JSON file or you can directly enter JSON parameters and their values in the field below.

   For more information about what parameters you can use, see https://v3-apidocs.cloudfoundry.org/version/3.138.0/index.html#service-route-binding .
5. Choose *Bind*.

Related Information

About Routes in the Cockpit [page 2873]

6.2.7.10 Security Groups

In the SAP BTP, Cloud Foundry environment, application security groups represent a collection of rules that specify the protocols, ports, and IP address ranges where an application or a task instance sends traffic. You can access this information in the SAP BTP cockpit.

Overview

In the SAP BTP, Cloud Foundry environment, the security groups are space-scoped and dynamic. They are also split into:

- Default security groups:
  - `dns`
  - `public_networks`

For more information, see https://docs.cloudfoundry.org/concepts/asg.html .

How to Find the Security Groups Page?

1. In the SAP BTP cockpit, navigate to your subaccount.
   For more information, see Navigate in the Cockpit.
2. In the navigation menu, choose *Cloud Foundry* > *Spaces*.
3. Go to one of your spaces.
4. In the navigation menu, choose *Security Groups*.

What Can You Do on the Security Groups Page?

The security groups manage the outgoing traffic of your application by allowing access to the network based on a list of specific protocols, ports, and IP address ranges. You have read-only access to them.

If your application is trying to access an endpoint not specified by the security group, the access is denied. On the *Security Groups* page in the cockpit, you can check the information in the *Rules* table to verify that the application is using the correct protocols, ports, and IP address ranges.

> **Note**
>
> Instead of using the cockpit, you can get information about a security group by running the corresponding CLI command. For more information, see https://cli.cloudfoundry.org/en-US/v8/ .

6.2.7.11 Using Custom Domains

SAP BTP allows subaccount owners to make their SAP BTP applications reachable and secure via a custom domain that is different from the default domain – for example, **subdomain.mydomain.com**.

See the documentation for the Custom Domain service for more information about configuring and managing custom domains.

6.2.8 Audit Logging in the Cloud Foundry Environment

In this section, you can find information for audit log functionalities in the Cloud Foundry environment.

SAP Audit Log is a core, security, and compliance-based SAP BTP service to provide means for audit purposes. The following features of SAP Audit Log service are available for SAP BTP Applications and Services:

- With the default features, you have:
  - Written compliance audit data from SAP BTP services and applications, through oAuth2 service plan.
  - Audit data securely stored for a default retention time of 90 days with no additional costs applied.
  - Retrieval of audit data within the default retention period ensured as part of the SAP Audit Log Management service.

- With the advanced features you can:
  - Write audit data from your owned SAP BTP applications.
  - Configure the retention period.

The advanced features enable you to comply with the SAP Product Standards and Business Industry regulations you're subject to. To enable the advanced features, you need to enable the premium edition service plan, where additional costs are applied based on the consumed volumes.

Audit logs are a special type of logs. They represent security-relevant chronological records that provide documentary evidence for an event or activity. The following table gives you a better understanding what is the difference between audit logs, activity logs, and application logs. The SAP Audit Log only stores audit logs written by SAP BTP services, when you take action over your account data.

	Audit logs	Activity logs	Application logs
Scope	Evidence	History	Fix Issues
Used by	Auditor	User	Developer
Tamper proof	✓		
UI performance		✓	
Frequent data access		✓	
Long data retention	✓		
Secure low cost architecture	✓		

The SAP Audit Log stores audit logs representing different actions taken over your account and/or data. There are predefined audit categories, which represent such kinds of actions:

- Data protection and privacy related
  - *audit.data-access* read-access logging records for access to sensitive personal data;
  - *audit.data-modification* data modification logging records for sensitive personal data.

  Security related
  - *audit.security-events* logging of general security events like login, logout, and other;
  - *audit.configuration* logging of security critical configuration changes.

See Change Logging and Read-Access Logging [page 3637].

If your subaccount audit logs contain other log types or information, create a support ticket with component `BC-CP-CF-SEC-AUDITLG`.

Related Information

Audit Log Retrieval API Usage for Subaccounts in the Cloud Foundry Environment [page 2880]
Audit Log Retention for the Cloud Foundry Environment [page 2893]
SAP Audit Log Viewer service for the Cloud Foundry Environment [page 2907]
Guided Answers for Audit Logging in the Cloud Foundry Environment

6.2.8.1 Audit Log Retrieval API Usage for Subaccounts in the Cloud Foundry Environment

The audit log retrieval API allows you to retrieve the audit logs for your SAP BTP Cloud Foundry environment subaccount.

> **Note**
>
> After the audit logs are generated and sent to the Audit Log service, the logs are not immediately accessible for retrieval.Retrieving of audit logs via the Audit Log Retrieval API is limited to the size of the audit logs generated for the subaccount. The audit log results are provided as a collection of JSON entities. The possible request rate is now limited to:

- 4 requests per second per auth-token/tenant on CF-EU11, CF-EU30, CF-US20, CF-US21, CF-JP10, CF-JP20, CF-BR10. There is also a burst queue that can temporarily allow up to 20 requests per second.
- 8 requests per second per auth-token/tenant on CF-EU10, CF-EU20, CF-US10, CF-US30. There is also a burst queue that can temporarily allow up to 40 requests per second.

The API returns a HTTP 429 response code when the limits are exceeded.

The audit log retrieval API is protected by OAuth. To consume the audit log retrieval API, you need your system to generate and use a valid OAuth.

Prerequisites

To execute the procedure, you need to have the `Space Developer` role for the corresponding Space.

> **Note**
>
> For security reasons, we recommend you to re-create the `auditlog-management` Service-Instance bindings and service keys at least every 90 days.

Create Instance of the `auditlog-management` Service

1. Create a Cloud Foundry Org and Space, in case you don't have any. See Create Spaces [page 2800].

2. Log in the Cloud Foundry landscape using the corresponding Cloud Foundry API (Infrastructure/ Landscape Overview).

```
cf login -a <API_URL> -o <ORG> -s <SPACE> -u <USER>
```

3. Create a service instance of the service `auditlog-management`.

> ### Note
>
> For security reasons, we strongly recommend you to adopt the provided mutual TLS authentication (mTLS). The mTLS authentication relies on X.509 certificates for the verification of the parties in the network connection. Create the `auditlog-managenment` service instances with the additional parameters, as explained in the following mTLS points.

- (Recommended) For mTLS authentication using X.509 certificates, use:

```
cf create-service auditlog-management default <SERVICE_INSTANCE> -c '{
    "xs-security": {
        "xsappname": "auditlog",
        "oauth2-configuration": {
            "credential-types": [
                "x509"
            ],
            "grant-types": [
                "client_credentials"
            ]
        }
    }
}'
```

> ### Note
>
> The `xsappname` attribute from the service parameters JSON needs to have unique instance for each service. The `xsappname` attribute can hold an arbitrary string as value, which is not required to be `auditlog`. The code block serves only as example.

- For non-mTLS authentication, use

```
cf create-service auditlog-management default <SERVICE_INSTANCE>
```

> ### Note
>
> If there's a previously created auditlog-management service instance without the security parameters for mTLS, we recommend you to delete that instance and create a new instance with mTLS authentication using the parameters from the previous (recommended) scenario.

4. Create a key for the service instance:
   - For service instances with mTLS, use:

```
cf create-service-key <SERVICE_INSTANCE> <SERVICE_KEY> -c '{
    "xsuaa": {
        "credential-type": "x509",
        "x509": {
            "key-length": 2048,
            "validity": 1,
            "validity-type": "DAYS"
        }
    }
}'
```

> ### Note
>
> You can specify the validity period of the service key for each use case. To use the service instance, once the key validity expires, create a new key. By default and for security reasons, the key from this code example is created with a validity of 1 day.

- For service instances without mTLS, use:

```
cf create-service-key <SERVICE_INSTANCE> <SERVICE_KEY>
```

5. List the key of the service instance:

```
cf service-key <SERVICE_INSTANCE> <SERVICE_KEY>
```

6. Extract data from the service key.
   - For mTLS scenario:
     - Extract the values for `uaa.clientid` and `uaa.certurl` of the key of the service instance for access token creation.
     - Extract the value for `url` and use it for later request to retrieve audit logs.
     - Extract the value for `uaa.certificate`, remove all `\n` entries from the X.509 certificate, and save it as a file in the `.pem` format.
     - Extract the value for `uaa.key`, remove all `\n` entries from the RSA private key, and save it as a file in the `.pem` format.

     > ### Note
     >
     > To extract of the values and avoid errors while copying or removing characters, you can use the `sed` and `jq` tools:
     >
     > ```
     > cf service-key <SERVICE_INSTANCE> <SERVICE_KEY> | sed '/Getting key/d'
     > | jq --raw-output .credentials.uaa.certurl
     > cf service-key <SERVICE_INSTANCE> <SERVICE_KEY> | sed '/Getting key/d'
     > | jq --raw-output .credentials.uaa.clientid
     > cf service-key <SERVICE_INSTANCE> <SERVICE_KEY> | sed '/Getting key/d'
     > | jq --raw-output .credentials.url
     > # the following two commands save the output to .pem files on the
     > location, where the command is executed
     > cf service-key <SERVICE_INSTANCE> <SERVICE_KEY> | sed '/Getting key/d'
     > | jq --raw-output .credentials.uaa.certificate > mtls-certificate.pem
     > cf service-key <SERVICE_INSTANCE> <SERVICE_KEY> | sed '/Getting key/d'
     > | jq --raw-output .credentials.uaa.key > mtls-private-key.pem
     > ```

   - For non-mTLS scenario:
     Extract values for `uaa.url`, `uaa.clientid` and `uaa.clientsecret` of the key of the service instance for access token creation. Extract the `url` to be used for later request to retrieve audit logs.

Create an OAuth Access Token

1. Request an OAuth access token
   - For mTLS scenario
     To request the OAuth access token via the mTLS extracted X.509 certificates and RSA private key flow, execute an HTTP POST request with the following parameters:

     ```
     URL: <value of "uaa.certurl">/oauth/token?
     grant_type=client_credentials&client_id=<value of "uaa.clientid">
     ```

```
Method: POST
Authentication Method: Client Certificates
Certificate: <Value of "uaa.certificate">
Key: <Value of "uaa.key">
grant_type: client_credentials
client_id: <Value of "uaa.clientid">
```

Example:

```
curl --cert <path to certificate.pem> --key <path to key.pem>
--request POST https://<value of "uaa.certurl">/oauth/token -d
'grant_type=client_credentials&client_id=<Value of "uaa.clientid">'
```

- For non-mTLS scenario

Request the OAuth access token via the OAuth client credentials flow, by executing an HTTP POST request with the following parameters:

```
URL: <value of "uaa.url">/oauth/token?grant_type=client_credentials
Method: POST
Authentication Method: Basic Authentication
Username: <Value of "uaa.clientid">
Password: <Value of "uaa.clientsecret">
```

2. Extract the value of the `access_token` attribute from the JSON response.
   This token grants access to the audit logs of the subaccount on the landscape where the service instance is created.

> **Note**
>
> The token is valid for 1 hour.

> **Note**
>
> This token grants access to the audit logs of the subaccount on the landscapeThe content of the access token can be displayed using a standard JWT decoder

Audit Log Retrieval

The Audit Log Retrieval API incorporates server-side paging to address the efficient handling of queries producing substantial result sets.

> **Note**
>
> The logs are not immediately accessible for retrieval

In instances where the query produces a result with significant size, the API systematically partitions the output into manageable chunks. The response includes an HTTP header featuring a handle, designed to facilitate the retrieval of subsequent result segments. To obtain additional portions of the result, pass the handle as a URL parameter in the subsequent retrieval request. The size of each chunk is predetermined, with a page size fixed at 500.

Supported request parameters:

- `time_from and time_to` – if no time filter is specified the default timeframe of 30 days back is returned. Use the following format: YYYY-MM-DDTHH:MM:SS, for example 2018-05-11T10:42:00. The time is in UTC.
- `handle` - in case the result set is too large, it's chunked, and a handle is returned for reading the next chunk. The handle is returned as a Response Header in the form: Paging: `handle=<value>`. Then you can provide `handle=<value>` as a request parameter in the subsequent retrieval request.
- `category` - this parameter allows you to filter the audit logs by predefined event types. If no event type is specified, all event types are returned. The available event types are:
  - **`audit.security-events`**
  - **`audit.configuration`**
  - **`audit.data-access`**
  - **`audit.data-modification`**

  To filter by a specific event type, provide the event type name as the value for this parameter, for example `category=audit.data-access`. Multiple event types can be specified separated by commas, for example `category=audit.data-access,audit.configuration`.

> **Note**
>
> Filterning by category doesn't work for custom retention periods and Sovereign cloud. In such case, an error code 501 is returned with the following format and message:
>
> ```
> {
>   "error": {
>     "code": "filter_category_not_supported",
>     "message": "Filtering by category not available for custom retention
> periods. Remove categories to proceed."
>   }
> }
> ```

Example: Get audit logs for the last 30 days

Execute the following HTTP GET request:

```
<url_from_service_key>/auditlog/v2/auditlogrecords
```

Provide the OAuth access token as an `"Authorization"` header: `"Authorization: Bearer <access_token>"`

The response is in JSON format, containing the audit log entries, split on pages if needed.

Example: Get audit logs filtering by time

Execute the following HTTP GET request:

```
<url_from_service_key>/auditlog/v2/auditlogrecords?
time_from=2018-05-10T10:42:00&time_to=2018-05-11T10:46:00
```

Example: Get audit logs filtered by event type

You can query for one of the predefined audit log event types for the `category` parameter.

Execute the following HTTP GET request:

```
<url_from_service_key>/auditlog/v2/auditlogrecords?
time_from=2018-05-10T10:42:00&time_to=2018-05-11T10:46:00&category=audit.data-
access
```

Example: Get audit logs next chunk determined by the server-side paging

Execute the following HTTP GET request:

```
<url_from_service_key>/auditlog/v2/auditlogrecords?
handle=2018-06-14T10:11:18.968<4f932695-8616-4e1f-
ac9a-1cdfb758f01d<2018-06-14T10:11:00.000
```

Response codes:

```
HTTP Codes

HTTP 200 OK

HTTP 204 NO_CONTENT

HTTP 401 UNAUTHORIZED

HTTP 400 BAD_REQUEST

HTTP 403 FORBIDDEN

HTTP 429 Too Many Requests

HTTP 501 NOT_IMPLEMENTED
```

Results

```
{
    "message_uuid": "e3a533c1-57b3-42b5-b514-8934ec5b6a6a",
    "time": "2018-10-04T08:12:00.239Z",
    "tenant": "c6da83a8-dc72-4374-b8f7-42b37a99XXXX",
    "org_id": "82f1da92-e5b3-4cc5-8c90-964165afXXXX",
    "space_id": "82f1da92-e5b3-4cc5-8c90-964165aXXXX",
    "app_or_service_id": "5fb40c3a-d36a-42c7-adc8-e4abd83dXXXX",
    "als_service_id": "c18f9b6d-a8af-431c-a187-749ebc59XXXX",
    "user": "test",
    "category": "audit.security-events",
    "format_version": "",
    "message": {
        "uuid": " e3a533c1-57b3-42b5-b514-8934ec5b6a6a ",
        "user": "test",
        "time": "2018-10-04T08:12:00.239Z",
        "ip": "10.58.183.15",

"data": "{\"level\":\"INFO\",\"origin\":null,\"msgNo\":1,\"msgId\":\"a2cf08ee-
eedd-455b-bbd6-    400d6b611116\",\"message\":\"ClientAuthenticationSuccess
('Client authentication success'): principal=sb-40ae21f3-5034-40dd-
ac0d-0c9d3e0ebb06!b3034|auditlog-management!b3034,
origin=[remoteAddress=52.58.183.15, clientId=sb-40ae21f3-5034-40dd-
ac0d-0c9d3e0ebb06!b3034|auditlog-management!b3034], identityZoneId=[c6da83a8-
dc72-4374-b8f7-42b37a99db2b]\",\"user\":null,\"version\":\"1.0\"}",
        "id": "cb046a0f-cc23-406b-a1d2-22ee6cf89a4d",
        "category": "audit.security-events",
```

```
        "tenant": "c6da83a8-dc72-4374-b8f7-42b37a99XXXX"
      }
}
```

Related Information

Audit Log Retrieval API for the Cloud Foundry Environment
API Documentation

6.2.8.2 Audit Log Retrieval API for Global Accounts in the Cloud Foundry Environment

On Central regions, Audit Log Retrieval API allows you to retrieve audit logs written on behalf of your SAP BTP Global Account.

Your central region is the eu10 Europe (Frankfurt) region, unless your global account is located in the China (Shanghai) and Government Cloud (US) regions. Then use the landscape domain of your central region.

Audit Log Retrieval API is protected by OAuth, and to consume it, you need your system to generate and use a valid OAuth.

> **Note**
>
> After the audit logs are generated and sent to the Audit Log service, the logs are not immediately accessible for retrieval.

Retrieving of audit logs via Audit Log Retrieval API is limited to the size of the audit logs written on behalf of the global account. The possible request rate is now limited to:

- 4 requests per second per auth-token/tenant on CF-EU11, CF-EU30, CF-US20, CF-US21, CF-JP10, CF-JP20, CF-BR10. There is also a burst queue that can temporarily allow up to 20 requests per second.
- 8 requests per second per auth-token/tenant on CF-EU10, CF-EU20, CF-US10, CF-US30. There is also a burst queue that can temporarily allow up to 40 requests per second.

The API returns a HTTP 429 response code when the limit is exceeded.

> **Note**
>
> Custom managed key (CMK) is not supported for audit logs generated on Global Account level.

Prerequisites

You do the following procedure through Cloud Foundry Environment, which is part of a subaccount. To enable Audit Log Retrieval API for Global Account , you must perform the following prerequisite steps as a global account administrator:

1. You need to have one or more subaccounts in the Central region. Create one if needed.

   > ### Note
   >
   > All space developer users can access the service instance in the subaccount where it's entitled. To restrict access to your global account audit logs we recommend you to create a new subaccount in the Central regions and add only users eligible to read global account audit logs to it.

2. Follow the steps to entitle one or more of your subaccounts with the **Central plan** of service Auditlog Management:
   1. Navigate to your global account.
   2. Choose *Entitlements* > *Entity Assignments* from the left-hand side menu.
   3. In the pop-up window, select all the subaccounts for which you would like to configure or display entitlements.
   4. Choose *Select* to apply the filter.
      You get a table for each of the subaccounts you selected, which displays the current entitlement and quota assignments.
   5. Choose *Configure Entitlements* to start editing entitlements for a particular subaccount. You can now edit the entitlements table.

      > ### Note
      >
      > You can edit entitlements for only one subaccount at a time.

   6. Choose *Add Service Plans*.
   7. Select *Auditlog Management* from the list on the left-hand side.
   8. Choose the checkbox of *Central plan* on the right-hand side.
   9. Choose *Add 1 Service Plan*.
   10. Choose *Save* and exit edit mode for that subaccount.
3. To execute the procedure, you need to have the `Space Developer` role for the corresponding Space.

> ### Note
>
> For security reasons, we strongly recommend you to re-create the `auditlog-management` Service-Instance bindings and service keys at least every 90 days.

Create Instance of the `auditlog-management` Service

1. Create a Cloud Foundry Org and Space, in case you don't have any. See Create Spaces [page 2800].
2. Log in the Cloud Foundry landscape using the corresponding Cloud Foundry API (Infrastructure/ Landscape Overview).

   ```
   cf login -a <API_URL> -o <ORG> -s <SPACE> -u <USER>
   ```

3. Create a service instance of the service `auditlog-management`:

> **Note**
>
> For security reasons, we strongly recommend you to adopt the provided mutual TLS authentication (mTLS). The mTLS authentication relies on X.509 certificates for the verification of the parties in the network connection. Create the `auditlog-managenment` service instances with the additional parameters, as explained in the following mTLS points.

- (Recommended) For mTLS authentication using X.509 certificates, use:

```
cf create-service auditlog-management central <SERVICE_INSTANCE> -c '{
    "xs-security": {
        "xsappname": "auditlog",
        "oauth2-configuration": {
            "credential-types": [
                "x509"
            ],
            "grant-types": [
                "client_credentials"
            ]
        }
    }
}'
```

> **Note**
>
> The `xsappname` attribute from the service parameters JSON needs to have unique instance for each service. The `xsappname` attribute can hold an arbitrary string as value, which is not required to be `auditlog`. The code block serves only as example.

- For non-mTLS authentication, use

```
cf create-service auditlog-management central <SERVICE_INSTANCE>
```

> **Note**
>
> If there's a previously created auditlog-management service instance without the security parameters for mTLS, we recommend you to delete that instance and create a new instance with mTLS authentication using the parameters from the previous (recommended) scenario.

4. Create a key for the service instance:

- For service instances with mTLS, use:

```
cf create-service-key <SERVICE_INSTANCE> <SERVICE_KEY> -c '{
    "xsuaa": {
        "credential-type": "x509",
        "x509": {
            "key-length": 2048,
            "validity": 1,
            "validity-type": "DAYS"
        }
    }
}'
```

> **Note**
>
> You can specify the validity period of the service key for each use case. To use the service instance, once the key validity expires, create a new key. By default and for security reasons, the key from this code example is created with a validity of 1 day.

- For service instances without mTLS, use:

```
cf create-service-key <SERVICE_INSTANCE> <SERVICE_KEY>
```

5. List the key of the service instance:

```
cf service-key <SERVICE_INSTANCE> <SERVICE_KEY>
```

6. Extract data from the service key.
   - For mTLS scenario:
     - Extract the values for `uaa.clientid` and `uaa.certurl` of the key of the service instance for access token creation.
     - Extract the value for `url` and use it for later request to retrieve audit logs.
     - Extract the value for `uaa.certificate`, remove all `\n` entries from the X.509 certificate, and save it as a file in the `.pem` format.
     - Extract the value for `uaa.key`, remove all `\n` entries from the RSA private key, and save it as a file in the `.pem` format.

     > **Note**
     >
     > To extract of the values and avoid errors while copying or removing characters, you can use the `sed` and `jq` tools:
     >
     > ```
     > cf service-key <SERVICE_INSTANCE> <SERVICE_KEY> | sed '/Getting key/d'
     > | jq --raw-output .uaa.certurl
     > cf service-key <SERVICE_INSTANCE> <SERVICE_KEY> | sed '/Getting key/d'
     > | jq --raw-output .uaa.clientid
     > cf service-key <SERVICE_INSTANCE> <SERVICE_KEY> | sed '/Getting key/d'
     > | jq --raw-output .url
     > # the following two commands save the output to .pem files on the
     > location, where the command is executed
     > cf service-key <SERVICE_INSTANCE> <SERVICE_KEY> | sed '/Getting key/d'
     > | jq --raw-output .uaa.certificate > mtls-certificate.pem
     > cf service-key <SERVICE_INSTANCE> <SERVICE_KEY> | sed '/Getting key/d'
     > | jq --raw-output .uaa.key > mtls-private-key.pem
     > ```

   - For non-mTLS scenario:

   Extract values for `uaa.url`, `uaa.clientid` and `uaa.clientsecret` of the key of the service instance for access token creation. Extract the `url` to be used for later request to retrieve audit logs.

Create an OAuth Access Token

1. Request an OAuth access token
   - For mTLS scenario

     To request the OAuth access token via the mTLS extracted X.509 certificates and RSA private key flow, execute an HTTP POST request with the following parameters:

     ```
     URL: <value of "uaa.certurl">/oauth/token?
     grant_type=client_credentials&client_id=<value of "uaa.clientid">
     Method: POST
     Authentication Method: Client Certificates
     Certificate: <Value of "uaa.certificate">
     Key: <Value of "uaa.key">
     grant_type: client_credentials
     client_id: <Value of "uaa.clientid">
     ```

Example:

```
curl --cert <path to certificate.pem> --key <path to key.pem>
--request POST https://<value of "uaa.certurl">/oauth/token -d
'grant_type=client_credentials&client_id=<Value of "uaa.clientid">'
```

- For non-mTLS scenario

Request the OAuth access token via the OAuth client credentials flow, by executing an HTTP POST request with the following parameters:

```
URL: <value of "uaa.url">/oauth/token?grant_type=client_credentials
Method: POST
Authentication Method: Basic Authentication
Username: <Value of "uaa.clientid">
Password: <Value of "uaa.clientsecret">
```

2. Extract the value of the `access_token` attribute from the JSON response.
   This token grants access to the audit logs of the subaccount on the landscape where the service instance is created.

> **Note**
>
> The token is valid for 12 hours.

> **Note**
>
> The content of the access token can be displayed using a standard JWT decoder

Audit Log Retrieval

The Audit Log Retrieval API incorporates server-side paging to address the efficient handling of queries producing substantial result sets.

> **Note**
>
> The logs are not immediately accessible for retrieval

In instances where the query produces a result with significant size, the API systematically partitions the output into manageable chunks. The response includes an HTTP header featuring a handle, designed to facilitate the retrieval of subsequent result segments. To obtain additional portions of the result, pass the handle as a URL parameter in the subsequent retrieval request. The size of each chunk is predetermined, with a page size fixed at 500.

Supported request parameters:

- `time_from and time_to` – if no time filter is specified the default timeframe of 30 days back is returned. Use the following format: 2018-05-11T10:42:00. The time is in UTC.
- `handle` – in case the result set is too large, it's chunked, and a handle is returned for reading the next chunk. The handle is returned as a Response Header in the form: Paging: `handle=<value>`. Then you can provide `handle=<value>` as a request parameter in the subsequent retrieval request.

> ### Note
>
> The `handle` parameter is a base 64 encoded string. Don't attempt to decode it. Send the `handle` in the subsequent request exactly as received in the previous one, without any modification.

Example: Get audit logs for the last 30 days

Execute the following HTTP GET request:

```
<url_from_service_key>/auditlog/v2/auditlogrecords
```

Provide the OAuth access token as an `"Authorization"` header: `"Authorization: Bearer <access_token>"`

The response is in JSON format, containing the audit log entries, split on pages if needed.

Example: Get audit logs filtering by time

Execute the following HTTP GET request:

```
<url_from_service_key>/auditlog/v2/auditlogrecords?
time_from=2018-05-10T10:42:00&time_to=2018-05-11T10:46:00
```

Example: Get audit logs next chunk determined by the server-side paging

Execute the following HTTP GET request:

```
<url_from_service_key>/auditlog/v2/auditlogrecords?
time_from=2018-05-10T10:42:00&time_to=2018-05-11T10:46:00&handle= <handle value>
```

Response codes:

```
HTTP Codes

HTTP 200 OK

HTTP 204 NO_CONTENT

HTTP 401 UNAUTHORIZED

HTTP 400 BAD_REQUEST

HTTP 403 FORBIDDEN

HTTP 429 Too Many Requests
```

Predefined audit log message categories:

```
'audit.security-events'

'audit.configuration'

'audit.data-access'

'audit.data-modification'
```

Results


```
{
    "message_uuid": "e3a533c1-57b3-42b5-b514-8934ec5b6a6a",
    "time": "2018-10-04T08:12:00.239Z",
    "tenant": "c6da83a8-dc72-4374-b8f7-42b37a99XXXX",
    "org_id": "82f1da92-e5b3-4cc5-8c90-964165afXXXX",
    "space_id": "82f1da92-e5b3-4cc5-8c90-964165aXXXX",
    "app_or_service_id": "5fb40c3a-d36a-42c7-adc8-e4abd83dXXXX",
    "als_service_id": "c18f9b6d-a8af-431c-a187-749ebc59XXXX",
    "user": "test",
    "category": "audit.security-events",
    "format_version": "",
    "message": {
        "uuid": " e3a533c1-57b3-42b5-b514-8934ec5b6a6a ",
        "user": "test",
        "time": "2018-10-04T08:12:00.239Z",
        "ip": "10.58.183.15",

"data": "{\"level\":\"INFO\",\"origin\":null,\"msgNo\":1,\"msgId\":\"a2cf08ee-
eedd-455b-bbd6-    400d6b611116\",\"message\":\"ClientAuthenticationSuccess
('Client authentication success'): principal=sb-40ae21f3-5034-40dd-
ac0d-0c9d3e0ebb06!b3034|auditlog-management!b3034,
origin=[remoteAddress=52.58.183.15, clientId=sb-40ae21f3-5034-40dd-
ac0d-0c9d3e0ebb06!b3034|auditlog-management!b3034], identityZoneId=[c6da83a8-
dc72-4374-b8f7-42b37a99db2b]\",\"user\":null,\"version\":\"1.0\"}",
        "id": "cb046a0f-cc23-406b-a1d2-22ee6cf89a4d",
        "category": "audit.security-events",
        "tenant": "c6da83a8-dc72-4374-b8f7-42b37a99XXXX"
      }
}
```


6.2.8.3 Audit Events Logged by SAP Audit Log service

The SAP Audit Log service records specific audit events related to data access and security activities when interacting with the Retrieval API.

Audit Events

Audit Log Type	Type	When	Message
`data-access`	Data read event	Logged on every audit log retrieval attempt, regardless of whether records were returned, including whether the read was successful.	Attribute with name “data read event” was read/not read successfully. The attribute is a part of an object with type “data read event” and ID consisting of: tenant_id “`<subaccount_id>`”. It belongs to a subject with type “account”, role “account”, and ID consisting of: id “`<uua-client-id>`”.
`security-events`	Successful login	Logged each time a successful authentication is made against the Retrieval API.	Security event message “successful login event” on `<timestamp>`. Security event was related to user “`<uua-client-id>`”.

Audit Log Type	Type	When	Message
`security-events`	Unsuccessful login	Logged on failed authentication attempts.	Security event message “unsuccessful login event” on `<timestamp>`. Security event was related to user “`<uua-client-id>`”.

6.2.8.4 Audit Log Retention for the Cloud Foundry Environment

The audit log data is stored on a subaccount level. The access to the stored audit log data is strictly restricted - only authorized stakeholders can preview, retrieve, and download their audit log data.

> **Note**
>
> Currently, configuring retention period is allowed only on a subaccount level. Configuring custom retention period on a global account level is not available yet.

There are two main categories of retention of audit data, divided by the source of generation:

- Retention of Audit Data written from SAP BTP Applications and Services [page 2893] (SAP-generated)
- Retention of Audit Data written from Customer's BTP Applications [page 2895] (Customer-generated)

6.2.8.4.1 Retention of Audit Data written from SAP BTP Applications and Services

Free Retention Period

By default, you can retain the securely stored SAP-generated audit log data for your subaccount for 90 days, free of charge. After this period, the data is automatically deleted. Within this free default retention period, as an authorized stakeholder you can access, retrieve, and download the stored audit log data.

Configurable Retention Period

Enable *Audit Log service, premium edition*

To follow the Industry and Business Regulations requirements for a longer retention and storage of audit log data written on behalf of a customer's subaccount, and to be able to configure a custom retention period longer than the free 90 days, you must enable *Audit Log service, premium edition* service plan for your subaccount as follows:

1. Entitle your subaccount with service auditlog and plan premium. For more information, see Configure Entitlements and Quotas for Subaccounts [page 2529]
2. Create a service instance *auditlog* of plan *premium*. For more information, see Creating Service Instances [page 273]

Configure a Retention period

When you enable the *Audit Log service, premium edition* service plan, you can create a support ticket with component BC-CP-CF-SEC-AUDITLG to the Audit Log service to configure a flexible retention period for storing your audit data. In the support ticket provide the following data:

- Subaccount ID(s) and Tenant ID(s) whose retention you want to extend.
- Your desired retention period in days.
- The date from which you want the configurable retention to be effective.

Pricing

The premium edition with its advanced functionalities is a commercialized consumption based service plan, which prices per GB you can check in SAP Discovery Center.

Revoking Custom Retention of SAP-Generated Audit Data

Revoke Custom retention of an Existing Subaccount

To revoke custom retention, create a support ticket with component BC-CP-CF-SEC-AUDITLG to the Audit Log service to configure a flexible retention period for storing your audit data. In the support ticket provide the following data:

- Subaccount ID(s) and Tenant ID(s) whose custom retention you want to revoke.
- The date from which you want the revokation to be effective.

When you revoke custom retention all stored audit data which is older than 90 days is removed and charging for extra storage stops.

Revoke Custom retention of a Terminated Subaccount

SAP-generated audit log data for a terminated subaccount is deleted when it is older than 90 days. The 90-day retention period is kept free of charge for audit purposes.

> **Note**
>
> When you delete a subaccount, all audit log data older than 90 days is automatically deleted.

> **Note**
>
> When you delete an auditlog premium service instance, the charging for storage doesn't stop automatically. To avoid accumulating further costs, you must either create a support ticket to revoke the custom retention period, or delete the subaccount.

6.2.8.4.2 Retention of Audit Data written from Customer's BTP Applications

Initial retention period

There is no free retention period for any customer-generated audit data. If you enable **Audit Log service, premium edition** service plan, the default period in which you can retain all audit data written from your customer applications built on top of SAP BTP is 90 days. For more information, see Audit Log Write API for Customers [page 2897].

Configurable Retention Period for Subaccounts

When you have enabled the **Audit Log service, premium edition** service plan, you can create a support ticket with component BC-CP-CF-SEC-AUDITLG to the Audit Log service to configure a custom retention period for storing your audit data.

Availability of the Write API for Customers

The functionality for writing customer-generated audit data as part of the **Audit Log service, premium edition** service plan, is currently available in the following regions:

- Australia (Sydney)
- Australia Southeast (Sydney)
- Brazil (São Paulo)
- Brazil South
- Canada (Toronto)
- Europe (Eemshaven)
- Europe (Frankfurt)
- Europe (Frankfurt, EU Access)
- Europe (Netherlands)
- India (Mumbai)
- Israel (Tel Aviv)
- Japan (Osaka)
- Japan (Tokyo)
- KSA (Dammam – KSA Non-Regulated Customers)
- KSA (Dammam – KSA Regulated Customers)
- Singapore
- South Korea (Seoul)

- Switzerland (EU Access)
- US Central (IA)
- US East (VA)
- US West (WA)

If you want to use this functionality in another region, contact the Service owner to discuss the conditions for storing large volumes of audit data. **Audit Log service, premium edition** with its advanced functionalities is a commercialized consumption-based service plan. You can find the cost per Customer Written Volume stored in GB in SAP Discovery Center.

Revoking Custom Retention of SAP-Generated Audit Data

Revoke Custom retention of an Existing Subaccount

To revoke custom retention, create a support ticket with component BC-CP-CF-SEC-AUDITLG to the Audit Log service to configure a flexible retention period for storing your audit data. In the support ticket provide the following data:

- Subaccount ID(s) and Tenant ID(s) whose custom retention you want to revoke.
- The date from which you want the revokation to be effective.

When you revoke custom retention all stored audit data which is older than 90 days is removed and charging for extra storage stops.

> ⓘ Note
>
> Audit log data written from Customer's SAP BTP Applications is still charged per GB at the price in SAP Discovery Center no matter the retention period in days.

Revoke Custom retention of a Terminated Subaccount

Audit log data written by Customer's SAP BTP Applications is removed immediately in case of a subaccount termination, as noted in Delete a Subaccount [page 2520].

> ⓘ Note
>
> When you delete a subaccount, all audit log data written from your Customer's SAP BTP Applications is automatically deleted.

> ⓘ Note
>
> When you delete an auditlog premium service instance, the charging for storage doesn't stop automatically. To avoid accumulating further costs, you must either create a support ticket to revoke the custom retention period, or delete the subaccount.

6.2.8.5 Audit Log Write API for Customers

The SAP Audit Log provides an Audit Log Write API for customers, which enables the recording of audit relevant events.

Prerequisites for Using the Audit Log Write API for Customers

Do the Enablement and Entitlements

As a first step to use the Audit Log Write API, make sure that the service is enabled for your subaccount and that there is an entitlement for **Audit Log service, premium edition** service plan.

If the *Audit Log Service* of plan *premium* is not visible from the account cockpit service marketplace for your subaccount, do the following to entitle the service:

- To add entitlements on subaccount level, navigate to the subaccount via the account cockpit and choose *Entitlements* > *Configurate Entitlements*
- To add entitlements on global account level via the account cockpit, navigate to the account cockpit global account view and choose *Entitlements* > *Service Assignments*
- To add entitlements for a global account via the SAP BTP Control Center, choose the global account and go to *Edit* > *Assign Services*. To select the Auditlog Service checkbox choose *Next* > *Set Entitlements* > *Select Auditlog Service* and ensure that there is quota for the “premium” plan of the service in the desired region(s).

Create a Service Instance

The *premium* plan of the auditlog service uses mTLS authentication via X.509 certificates. To create a service instance via the account cockpit, do the following:

1. Select the *premium* plan for the service instance creation from the Service Marketplace inside the SAP BTP cockpit.
2. Choose a runtime and a name for the service instance.
3. Specify a unique *xsappname* per subaccount for each instance and provide the x509 authentication parameters during service instance creation.
   You can also create a service instance via the CF client with the following sequence of commands:

```
cf api <cf-domain>
cf login
cf target -o <desired-org> -s <desired-space>
cf create-service auditlog premium <my-service-instance> -c '{
  "xs-security": {"xsappname": "<some-unique-name>",
        "oauth2-configuration": {
            "credential-types": ["x509"],
            "grant-types": ["client_credentials"]
    }
  }
}'
```

> ### Note
>
> Deleting the service instance doesn't delete the customer audit data already stored in the Audit Log Viewer. For more information, see Audit Log Retention for the Cloud Foundry Environment [page 2893]

Create a Service Key

The service keys provide credentials and URLs for interacting with your service instance. You can create a service key in one of the following ways: from the account cockpit or via CF command.

- From your account cockpit, select the created *premium* service instance from the *Instances and Subscriptions* section. Depending on the chosen runtime environment, you can see a *Create* button for either a service key or a service binding. In the service key/binding creation box, specify the validity of the X.509 certificates used for authentication.
- In the CF client, use the following command. The example parameters sets the certificate credentials validity to 30 days. After that period, the credentials expire and need to be rotated. For more information about the rotation, see the *Credentials rotation* section.

```
cf create-service-key <my-service> <my-key> -c '{
  "xsuaa": {
  "credential-type": "x509",
  "x509": {
    "key-length": 2048,
    "validity": 30,
    "validity-type": "DAYS"
   }
  }
}'
```

Create a Service Binding

Like service keys, the classical service bindings provide credentials and endpoints for using a service instance and are mainly intended to be used with SAP BTP, Cloud Foundry runtime applications. You can bind a SAP BTP, Cloud Foundry runtime application to one or more service instances. As a result, in the VCAP_SERVICES application environment variable there is a dedicated JSON object for each service binding.

You can perform the binding operation directly from the account cockpit settings of the application, via the CF client, or via the application deployment manifest. Specifically for the Audit Log Write API “premium” plan, it is important to provide the credentials parameter to enable the provisioning of X.509 certificates. Examples:

- Create a service binding via the CF client:

```
cf bind-service <sample-application> <auditlog-premium-service-instance> -c '{
  "xsuaa": {
  "credential-type": "x509",
    "x509": {
      "key-length": 2048,
      "validity": 30,
  "validity-type": "DAYS"
    }
  }
}'
```

- Create a service binding through a simple deployment manifest:

```
---
applications:
- name: java-auditlog-sample
  memory: 512M
  instances: 1
  path: target/java-auditlog-sample.war
  env:
    TARGET_RUNTIME: tomcat
  services:
    - name: "<auditlog-premium-service-instance-name>"
      parameters:
        xsuaa:
```

```
          credential-type: "x509"
          x509:
            key-length: 2048
            validity: 30
            validity-type: "DAYS"
```

- Create a service binding through an MTA deployment manifest:

```
modules:
- name: <module using the Audit Log Service>
  requires:
    - name: <Audit Log Service Premium Instance Name>
      parameters:
        config:
          xsuaa:
            credential-type: x509
            x509:
              key-length: 2048
              validity: 2
              validity-type: MONTHS
 - name: <other modules>
resources:
- name: <Audit Log Service Premium Instance Name>
    type: org.cloudfoundry.managed-service
    parameters:
      service: auditlog
      service-plan: premium
```

Obtain Credentials for Authentication and Authorization

This section describes the approach to obtain credentials for authentication and authorization towards the Audit Log Write API.

- Retrieving service key information
  When you create a service key or binding, do one of the following:
  - Download the service key data from the account cockpit - select the three dots next to the service key/binding and then *Download*.
  - Use the following command to extract the data from the service key:

    ```
    cf service-key <my-service> <my-key>
    ```

    Example of a sample service key or binding object:

    ```
    {
      "uaa": {
        "apiurl": "https://api.authentication.<cf_domain>",
        "certificate": "<x509_certificate>",
        "certificate-pinning": true,
        "certurl": "https://<subdomain>.authentication.cert.<cf_domain>",
        "clientid": "<client_id>",
        "clientx509enabled": true,
        "credential-type": "x509",
        "identityzone": "<identitiy_zone>",
        "identityzoneid": "<identitiy_zone_id>",
        "key": "<private_key>",
        "sburl": "https://internal-xsuaa.authentication.<cf_domain>",
        "subaccountid": "<subaccount_id>",
        "tenantid": "<tenant_id>",
        "tenantmode": "dedicated",
        "uaadomain": "authentication.<cf_domain>",
        "url": "https://<subdomain>.authentication.<cf_domain>",
        "verificationkey": "<verification_key>",
        "xsappname": "<xsappname>",
        "zoneid": "<zone_id>"
    ```

```
  },
  "url": "https://api.auditlog.<cf_domain>:6081",
  "vendor": "SAP"
}
```

- Obtaining credentials for using the Audit Log Write API
  The service key/binding information you retrieve in the previous step, contains the issued client certificates for authentication and the URLs to query the service. You need to use those certificate credentials to issue a JWT token for authentication towards the Write API.
  To prepare a request for issuing a JWT token, extract the PEM certificates and the relevant service data. Extract clientid, certurl, certificate and key from the credentials for the audit log service instance. Save the extracted certificate and key to files in `.pem` format by removing all `\n` entries.
  You can use the following example, considering that the service key/binding information is saved to a json file and use the `cat` and `jq` commands can be used for parsing the file:

  ```
  cat customer-write-api-demo-sk.json | jq --raw-output .uaa.certurl
  cat customer-write-api-demo-sk.json | jq --raw-output .uaa.clientid
  cat customer-write-api-demo-sk.json | jq --raw-output .url
  cat customer-write-api-demo-sk.json | jq --raw-output .uaa.certificate > mtls-
  certificate.pem
  cat customer-write-api-demo-sk.json | jq --raw-output .uaa.key > mtls-private-
  key.pem
  ```

  Then execute a POST request to `<uaa.certurl>oauth/token` using the client certificates. Use the following parameters in the request:

  ```
  URL: <value of "uaa.certurl">/oauth/token?
  grant_type=client_credentials&client_id=<value of "uaa.clientid">
  Method: POST
  Authentication Method: Client Certificates
  Certificate: <Value of "uaa.certificate">
  Key: <Value of "uaa.key">
  grant_type: client_credentials
  client_id: <Value of "uaa.clientid">
  ```

  Example of such a request using `curl`:

  ```
  curl --cert <path to certificate.pem> --key <path to key.pem>
  --request POST https://<value of "uaa.certurl">/oauth/token -d
  'grant_type=client_credentials&client_id=<Value of "uaa.clientid">'
  ```

  You can also execute the JWT token request via REST client, like Postman. In such case, make sure to configure the certifciates in the Postman settings as described at https://learning.postman.com/docs/sending-requests/certificates/ .
  Pass the issued JWT token in the Authorization header when calling the Audit Log Server Write REST API: `Authorization: Bearer <token-value>`. Example:

  ```
  curl --location --request POST 'https://api.auditlog.<url-host>:6081/audit-
  log/oauth2/v2/configuration-changes' \
  --header 'Content-Type: application/json' \
  --header 'Authorization: Bearer <token>' \
  --data-raw '{message-content}'
  ```

  Fore more examples, see section *Example requests for the Audit Log Write API*.
  The JWT token has a limited validity. When the token validity expires, you can issue a new one with the same set of certificate credentials.

Rotate credentials

Once the validity of the certificate credentials from the service key and service binding expire, the credentials for using the Audit Log service become invalid until the certificates are rotated. Make sure to properly rotate the credentials before they expire to avoid issues using the Audit Log service.

- For the case of SAP BTP, Cloud Foundry runtime apps with service bindings, do the credentials rotation by recreating the service binding with the auditlog service instance (`cf unbind-service`; `cf bind-service`). On each bind operation a new set of x509 certificates is generated.
- For the case of apps using auditlog credentials from an auditlog service key, rotate the credentials by creating a new service key and extracting the certificates and private key from the newly created key.

For security reasons, don't use very long validity period for the mTLS credentials and rotate the certificates as frequently as possible (depending on the use case and application lifecycle).

Audit Log Write REST API specifics

Endpoints

The endpoints exposed for the *premium* service plan for the 4 types of audit log categories are the following, where the base URL can be obtained from the service key/service binding:

- AWS IAAS Provider
  - https://api.auditlog.<cf-domain>:6081/audit-log/oauth2/v2/security-events
  - https://api.auditlog.<cf-domain>:6081/audit-log/oauth2/v2/configuration-changes
  - https://api.auditlog.<cf-domain>:6081/audit-log/oauth2/v2/data-accesses
  - https://api.auditlog.<cf-domain>:6081/audit-log/oauth2/v2/data-modifications
- SAP Converged Cloud
  - https://api.auditlog.<cf-domain>/audit-log/premium/v2/security-events
  - https://api.auditlog.<cf-domain>/audit-log/premium/v2/configuration-changes
  - https://api.auditlog.<cf-domain>/audit-log/premium/v2/data-accesses
  - https://api.auditlog.<cf-domain>/audit-log/premium/v2/data-modifications

Request Encoding

Declare the encoding in the Content-Type header:

```
Content-Type: application/json; charset=UTF-8
```

Response Status Codes

The following response status codes can be returned from the Audit Log Write API:

Code	Description
201 (Created)	Audit log successfully created.

Code	Description
400 (Bad Request)	Bad request for one of the following reasons: • URI syntax error • Invalid attributes • Cannot parse the received audit log message • Connection shutdown
401 (Unauthorized)	Missing or invalid *Authorization* header.
413 (Payload Too Large)	The Audit log message size exceeds 10KB.
429 (Too Many Requests)	Rate limit per tenant (provider_tenant, org, space) is hit.
500 (Internal Server Error)	An unexpected exception occurred.
502 (Bad Gateway)	Load balancer issue.
503 (Service Unavailable)	Global rate limit is hit.
504 (Gateway Timeout)	Load balancer issue.

Audit Log Message Format

The audit log record contains the following two main parts - the first one is filled in by the user of the audit log service and the second one, mainly in the metadata part of the message, is filled in by the audit logging framework.

- **Filled by the audit log user**

Field	Semantics
`uuid`	Unique identifier of the message.
`time`	The time when the auditable event has been executed.
`user`	The user that is logged in and performs the auditable action. To let the server-side extract the user from the JWT token, set the "$USER" string, or set a custom user and set a custom field.
`data_subject`	The persona that is the owner of the personal data, which is being accessed or modified in Log read access to sensitive personal data events and Log changes to personal data.
`object`	The unique identificator for the location of the personal data that is being accessed or modified in case of Log read access to sensitive personal data events and Log changes to personal data, as well as the unique identificator of the location and configuration that is being changed.
`attributes`	The list of attributes that are either added, deleted, or modified. There are three scenarios, for example: 1. a new address is added - only fill in the new value for the attribute with the newly written value (skip the old value); 2. an existing address is modified – add the changed value as new and the previous value as old; 3. an existing address is deleted - fill in only the deleted value as the old value for the attribute (skip the new value field).
`attachment`	The list of attachments that contain a personal data and has been opened for reading by a user, and which must be reflected in an audit log entry.

Field	Semantics
`ip`	The IP where the audited security-relevant event is executed (only available for security audit log events).
`data`	The message that contains the needed information for the logged security event (only available for security audit log events).
`tenant`	The identifier of the owner of the audit log message. Each audit log entry can have only single tenant equal to the zone ID, which determines the subaccount that is the owner of the message. The message is visible only for users in a role that has permissions to read the audit logs for this subaccount. The retention of the audit log message is determined based on the setup retention for this subaccount. For the premium plan, set the string "$PROVIDER", which lets the servers extract the tenant zone ID automatically from the used JWT token.
`success`	The field is used for transactional messages only. When you mean to log the action as intention, skip the success field. If with the audit log message you mean to confirm the success of the action execution or to state that it was not successful, add the "success" field with values TRUE or FALSE.

- **Filled by the framework**

Field	Semantics
`service_instance_id`	The GUID of the audit log instance that is used to write audit logs.
`platform`	The platform identifier together with the region information from where the audit log is written.
`org_name`	Empty in the newest version for security reasons.
`org_guid`	The organization GUID of the component that is using the audit log service to write audit logs.
`space_name`	Empty in the newest version for security reasons.
`space_guid`	The space GUID of the component that is using the audit log service to write audit logs.
`app_name`	Empty in the newest version for security reasons.
`app_guid`	The GUID of the application that is using the audit log service to write audit logs.
`category`	Message category is set implicitly by the framework.

Example Requests for the Audit Log Write API

This section contain examples for writing audit log events using `curl` demo purposes. You can use any other client, the examples serve to demonstrate the structure of the required payload per each audit log type.

Writing a Security Event

Mandatory fields: `uuid`, `user`, `time`, `data`, `tenant`.

```
curl --location --request POST 'https://api.auditlog.cf.<cf-domain>:6081/audit-
log/oauth2/v2/security-events' \
--header 'Content-Type: application/json; charset=UTF-8' \
--header 'Authorization: Bearer <JWT-token-value>' \
--data-raw '{
  "uuid": "<uuid-value>",
  "user": "$USER",
```

```
  "time": "2023-06-30T00:00:00.000Z",
  "ip": "127.0.0.1",
  "data": "Demo security event message.",
  "tenant": "$PROVIDER"
}'
```

Writing a Configuration Change Event

Mandatory fields: `object_id`, `uuid`, `user`, `tenant`, `time`, `attributes`.

```
curl --location --request POST 'https://api.auditlog.cf.<cf-domain>:6081/audit-
log/oauth2/v2/configuration-changes' \
--header 'Content-Type: application/json; charset=UTF-8' \
--header 'Authorization: Bearer <JWT-token-value>'\
--data-raw '{
  "uuid": "<uuid-value>",
  "user": "test-als-user@sap.com",
  "time": "2023-06-30T00:00:00.000Z",
  "id": "<id-value>",
  "tenant": "$PROVIDER",
  "object": {
    "type": "name of an attribute",
    "id": {
      "key1": "value1",
      "key2": "value2"
    }
  },
  "attributes": [
    {
      "name": "cfg attribute 1",
      "old": "old cfg attribute 1 value",
      "new": "new cfg attribute 1 value"
    },
    {
      "name": "cfg attribute 2",
      "old": "old cfg attribute 2 value",
      "new": "new cfg attribute 2 value"
    }
  ],
  "category": "audit.configuration",
  "customDetails": {
      "customKey": "customValue",
      "arbitraryKey": "arbitraryValue",
      "customJson": {
          "testString1": "testValue1",
          "testString2": "testValue2"
      }
  }
}'
```

Writing a Data Modification Event

Mandatory fields: `object_id`, `user`, `tenant`, `time`, `attributes`.

```
curl --location --request POST 'https://api.auditlog.cf.<cf-domain>:6081/audit-
log/oauth2/v2/data-accesses' \
--header 'Content-Type: application/json; charset=UTF-8' \
--header 'Authorization: Bearer <JWT-token-value>'\
--data-raw '{
  "uuid": "<uuid-value>",
  "user": "some-user-id",
  "identityProvider" : "$IDP",
  "time": "2023-06-30T00:00:00.000Z",
  "tenant" : "$PROVIDER",
  "object": {
      "type": "online system",
```

```
      "id": {
        "name" : "Students info system",
        "module" : "Admin module"
      }
  },
  "data_subject": {
      "type": "student",
      "role": "foreign student",
      "id": {
        "student_id":"12345678",
        "first name":"Jane",
        "last name":"Smith"
      }
  },
  "attributes": [
    {
      "name": "name of attribute 1"
    },
    {
      "name": "name of attribute 2",
      "successfull": true
    }
  ],
  "customDetails" : {
    "customKey" : "customValue",
    "anotherCustomKey" : "another custom value",
    "customJson": {
      "testString1": "testValue1",
      "testString2": "testValue2"
    }
  },
  "attachments": [
      {
          "id": "someId",
          "name": "attachmentName",
          "content": "attachmentContent"
      }
  ]
}'
```

Writing a Data Access Event

Mandatory fields: `object_id`, `user`, `tenant`, `time`, `attributes`.

```
curl --location --request POST 'https://api.auditlog.cf.<cf-domain>:6081/audit-
log/oauth2/v2/data-modifications' \
--header 'Content-Type: application/json; charset=UTF-8' \
--header 'Authorization: Bearer <JWT-token-value>' \
--data-raw '{
  "uuid": "<uuid-value>",
  "user": "$USER",
  "time": "2023-06-30T00:00:00.000Z",
  "object": {
    "type": "DEMO Instance",
    "id": {
      "serviceInstanceId": "<serviceInstanceId-value>"
    }
  },
  "data_subject": {
    "type": "credentials",
    "role": "authentication",
    "id": {
      "key1": "value1",
      "key2": "value2"
    }
  },
  "attributes": [
```

```
    {
      "name": "some attribute",
      "new": "new value"
    }
  ],
  "id": "<id-value>",
  "category": "audit.data-modification",
  "tenant": "$PROVIDER",
  "customDetails": {}
}'
```

Getting Support

If you need support for Audit Log Write API, open a SNOW support ticket to the component `BC-CP-CF-SEC-AUDITLG`.

Related Information

Getting Support [page 3683]

6.2.8.5.1 Categories of Audit Relevant Events

Find explained the four categories of Audit Relevant Events.

Category	Description	Examples
Log read access to sensitive personal data	Use a data-accesses audit log record for the cases where an access to personal data is executed. The audit log contains information on the accessed personal data, as well as the full information of the user that is accessing the personal data, the owner of the personal data (the Data Subject), and the identifier of the place where the access to the personal data have been executed (the Object).	Examples of personal data access events are all the read accesses sensitive personal data, which means information on racial or ethnic origin, political opinions, religious or philosophical beliefs, trade-union membership, health or sex life, bank account and credit card data, genetic data and biometric data for the purpose of uniquely identifying a natural person.
Log changes to personal data	Use a data-modification audit log record for the events that are related to modification of personal data. The audit log contains information on the modified personal data, as well as the full information of the user that is accessing the personal data, the owner of the personal data (the Data Subject), and the identifier of the place where the access to the personal data have been executed (the Object).	Examples of personal data modification events are writing, deleting, or changes on an address, e-mail address, phone number, and any other data that can lead to a persona identification.

Category	Description	Examples
Security event log	Log security data for all events that might have an impact over the system security.	Examples for such events are successful and non-successful login and logout events, changes in access control permissions, changes in credentials, changes in sensitive configurations, and so on.
Configuration change log	Use the configuration change audit log events for various changes in configurations. The audit log contains the changed configuration parts with old and new value. Log only the changed parts with the old and new value. If old or new value doesn't exist in case of adding a new property or deleting one respectively, skip the missing values or set them as null.	Examples of configuration change events are creation/deletion of service instances, binding/unbinding service instances, software update, changes in configurations, and so on.

6.2.8.6 SAP Audit Log Viewer service for the Cloud Foundry Environment

The SAP Audit Log Viewer service displays the audit logs associated with your Cloud Foundry account. These are the activity logs generated by the SAP applications and services to which you are subscribed.

The Audit Log Viewer is available for your SAP BTP Global Account and all your Cloud Foundry environment subaccounts.

- To view the audit logs for your global account, open theSAP Audit Log Viewer service in the header of the Account Explorer.
- To view the audit logs for a subaccount, open the SAP Audit Log Viewer service on the subaccount Overview page or in the subaccount Actions context menu in the Account Explorer.

The UI allows you to:

- Read the audit logs written for your account in selected time frame. The SAP Audit Log Viewer service displays 500 audit log records per request. If you want to view more audit log records, specify a shorter time frame.
- Display more detailed information on each single audit log record.
- Filter the retrieved client-side chunk for certain strings.
- Select which of the four audit log message categories to be returned. If you don't select any category, all are returned.
- Download the audit logs in the selected time frame.

The default displayed columns are:

- User - the person that has executed the event reflected in the written audit log.
- Time - creation time for the audit log message.
- Message - summary of the audit log message that is written.
- Category - audit log message category. The four-supported audit log message categories are:
  - `audit.security-events`

- `audit.configuration`
- `audit.data-access`
- `audit.data-modification`

The appearance of the UI can be modified by selecting the rows to be displayed on a single page, as well as the columns that you want to be visible.

Related Information

Accessibility Features in Audit Log Viewer [page 2908]

6.2.8.6.1 Accessibility Features in Audit Log Viewer

To optimize your experience of Audit Log Viewer, SAP Business Technology Platform (SAP BTP) provides features and settings that help you use the software efficiently.

> **Note**
>
> Audit Log Viewer is based on SAPUI5. For this reason, accessibility features for SAPUI5 also apply. See the accessibility documentation for SAPUI5 on SAP Help Portal at Accessibility for End Users.

6.2.8.7 Audit Events Logged by the Cloud Foundry Services

This document contains a list of the Cloud Foundry services that log audit events. The logged events are service specific and determined by the service itself. The Audit Log service does not know what audit events the services log, it only provides the infrastructure for services to write the logs, and for customers to read them by using the Audit Log Retrieval API and the Audit Log Viewer. For more information about the audit events logged by a service, follow the corresponding link in the table.

Audit Log Events by Service

Service	Audit log events page of the service
Authorization and Trust Management	Auditing and Logging Information for SAP Authorization and Trust Management Service
Business Entity Recognition	Auditing and Logging Information
Business Logging	Auditing and Logging
Business Rules	Auditing and Logging Information
Custom Domain (Cloud Foundry)	Auditing and Logging Information
Data Attribute Recommendation	Auditing and Logging Information
Document Classification	Auditing and Logging Information

Service	Audit log events page of the service
Document Information Extraction	Auditing and Logging Information
Document Management	Auditing and Logging Information
Identity Authentication	Auditing and Logging Information
Invoice Object Recommendation	Auditing and Logging Information
Process Visibility	Auditing and Logging Information
SAP AI Core	Auditing and Logging Information
SAP AI Launchpad	Auditing and Logging Information
SAP Alert Notification Service	Auditing and Logging Information
SAP API Management	Auditing and Logging Information
SAP Audit Log	Audit Events Logged by SAP Audit Log service
SAP Build Apps	Auditing and Logging Information
SAP Build Process Automation	Audit Logging
SAP Build Work Zone, standard edition	Auditing and Logging Information
SAP Build Work Zone, advanced edition	Auditing and Logging Information
SAP Business Application Studio	Auditing and Logging Information
SAP Cloud ALM	Integration of Audit Log Service
SAP Cloud Identity Access Governance	Integration of Audit Log Service
SAP Cloud Integration (Cloud Foundry)	Auditing and Logging Information
SAP Cloud Integration for data services	Security Log
SAP Cloud Management service for SAP BTP	Auditing and Logging Information [page 2760]
SAP Cloud Portal service	Auditing and Logging Information
SAP Cloud Transport Management	Auditing and Logging Information
SAP Credential Store	Auditing and Logging Information
SAP Conversational AI	Auditing and Logging Information
SAP Data Privacy Integration	Access to Audit Logs
SAP Datasphere Cloud	Audit Logging
SAP Enterprise Architecture Designer, Cloud Edition	Auditing and Logging
SAP Edge Services, Cloud Edition	Auditing and Logging Information
SAP Event Mesh	Auditing and Logging Information
SAP HANA Service for SAP BTP in AWS and Google Cloud Regions	Auditing
SAP HANA Service for SAP BTP in SAP Regions	Auditing
SAP Integration Advisor	Auditing and Logging Information
SAP Job Scheduling Service	https://help.sap.com/docs/job-scheduling/sap-job-scheduling-service/auditing-and-logging-information

Service	Audit log events page of the service
SAP Leonardo Machine Learning Foundation	Logging
SAP Market Rates Management, Bring Your Own Rates data option	Auditing and Logging Information
SAP Market Rates Management, Refinitiv data option	Auditing and Logging Information
SAP Mobile Services (Cloud Foundry)	Viewing Audit Logs
SAP Monitoring Service	Auditing and Logging Information
SAP Omnichannel Promotion Pricing	Auditing and Logging Information
SAP Print Service	Auditing and Logging Information
SAP Private Link Service (Beta)	Audit and Logging Information
SAP S/4HANA Cloud Extensibility	Auditing and Logging Information
SAP Secure Login Service for SAP GUI	Auditing and Logging Information
SAP Start	Security Events Logged by the Interest Cards Service
SAP SuccessFactors Extensibility	Auditing and Logging Information
SAP Task Center	Auditing and Logging Information
SAP Variant Configuration and Pricing	Audit Logging
SAP Web Analytics	Auditing and Logging Information
SAP Web IDE Full-Stack	Auditing and Logging Information
Service Ticket Intelligence	Auditing and Logging Information
UI Theme Designer	Auditing and Logging Information
UI5 Flexibility for Key Users	Auditing and Logging Information
Unit of Measure	Audit Logging
Workflow	Audit Logs

6.2.8.8 Checking Who Created an Application or a Service Instance

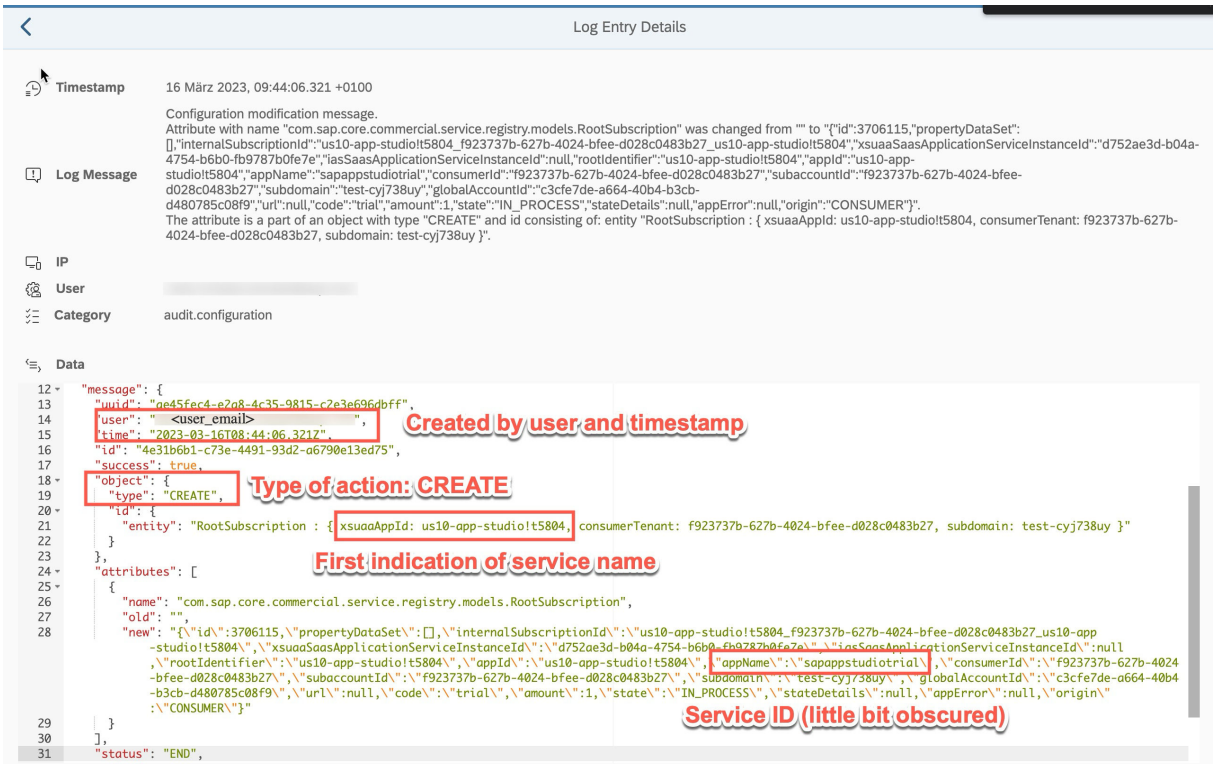
When you want to know for your subaccount which user created an application or a service instance, you need to look into the event logs.

- Check Who Created an Application [page 2910]
- Check Who Created a Service Instance [page 2912]

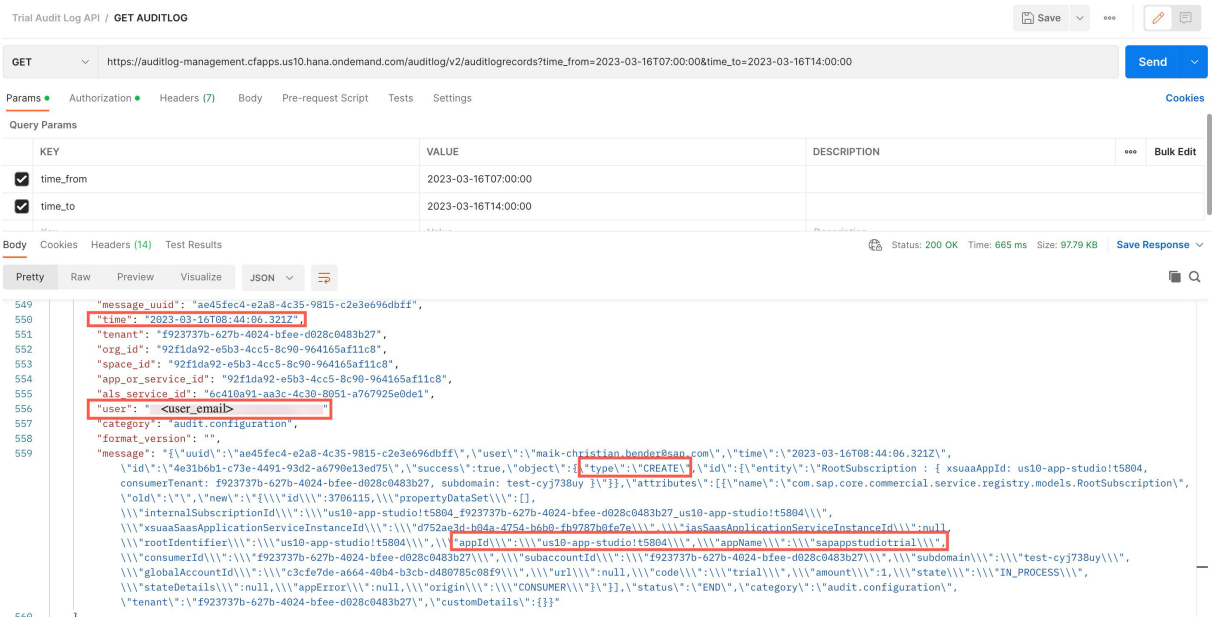
6.2.8.8.1 Check Who Created an Application

Creating an application is logged by SAP Audit Log service. You can see this information in the Audit Log Viewer or retrieve it by the Audit Log API.

Example in Audit Log Viewer for SAP Business Application Studio created by <user_email> on 16.03.2023 08:44:06.



The same information retrieved by Audit Log API:

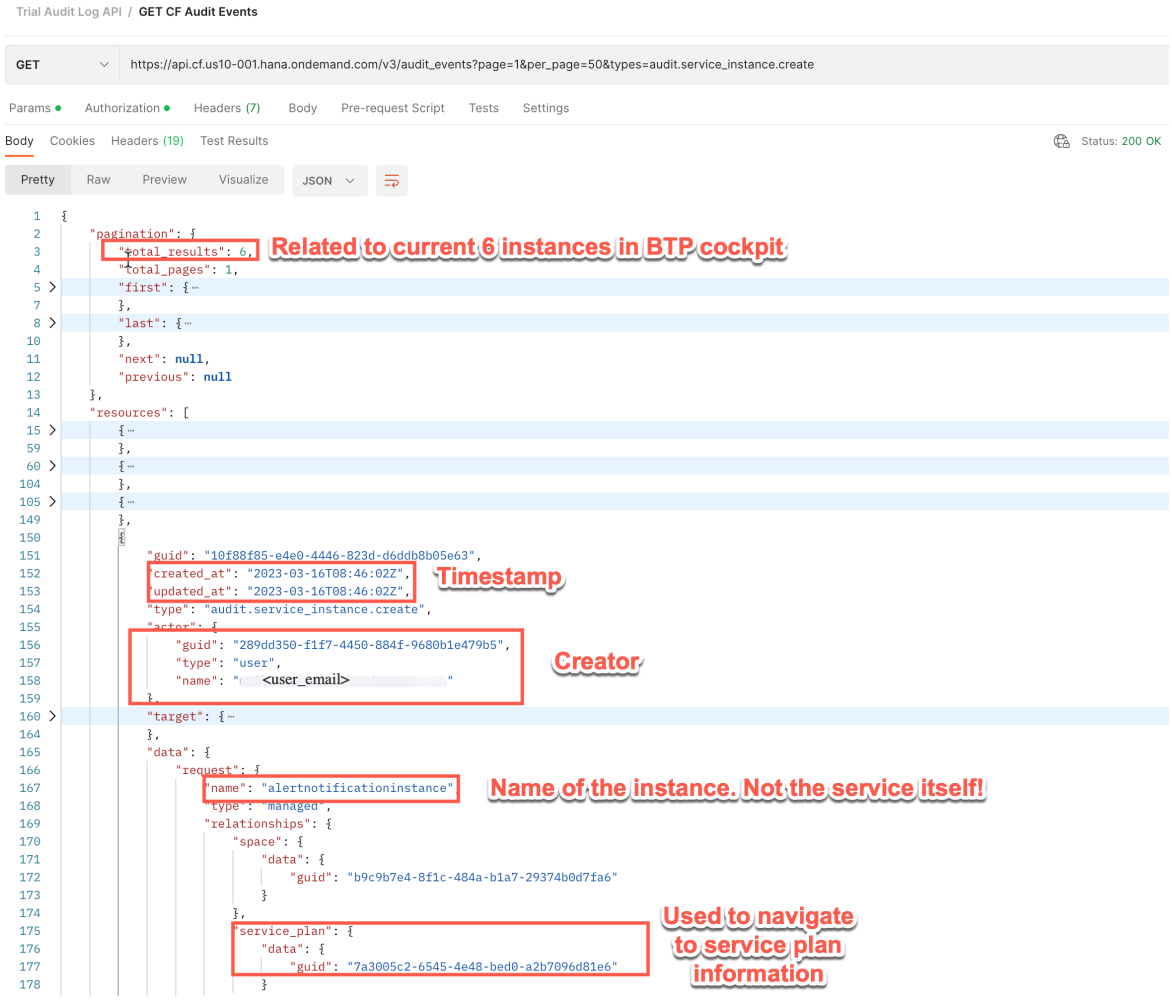


6.2.8.8.2 Check Who Created a Service Instance

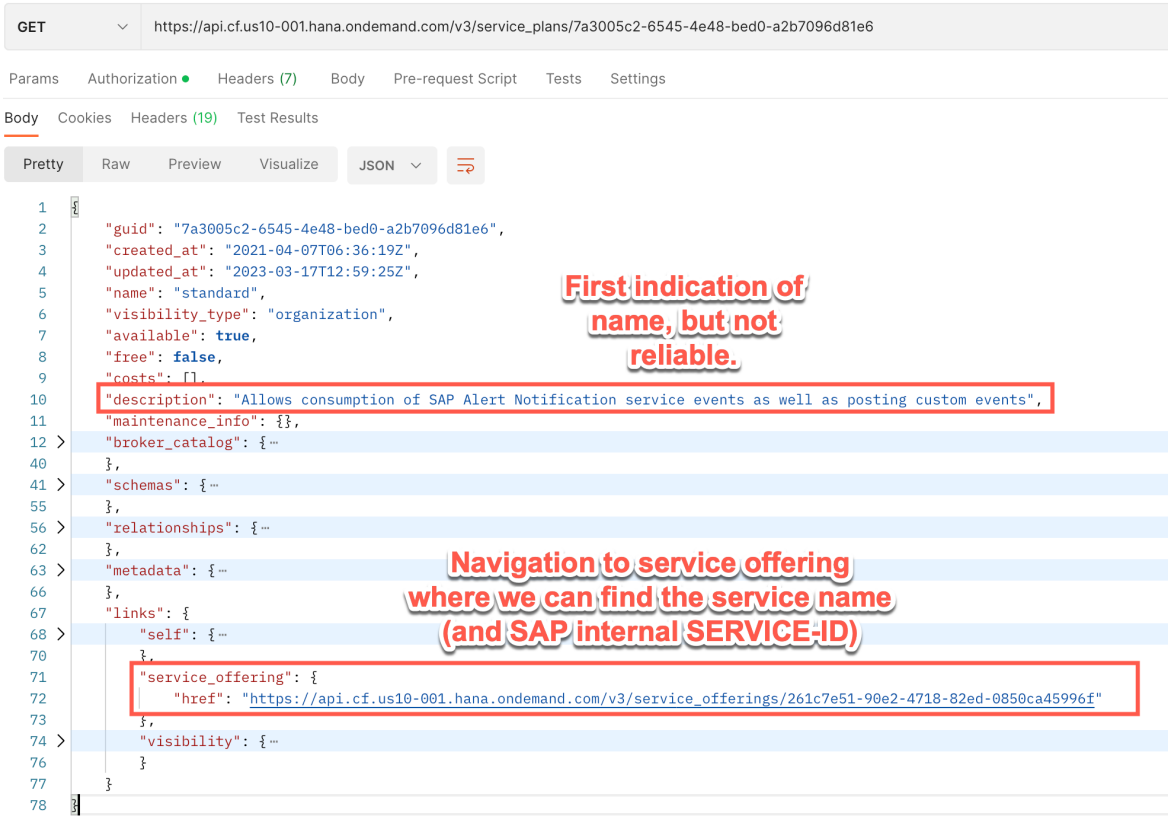
The following steps help you get information about who created a service instance. These steps are valid only for Cloud Foundry subaccounts.

To get the information for a service instance, use the Cloud Foundry API , which is available for all Cloud Foundry subaccounts. The following steps describe a path of API calls. Combining all information from the three steps, you get a list of all services created by a specific user at a specific time.

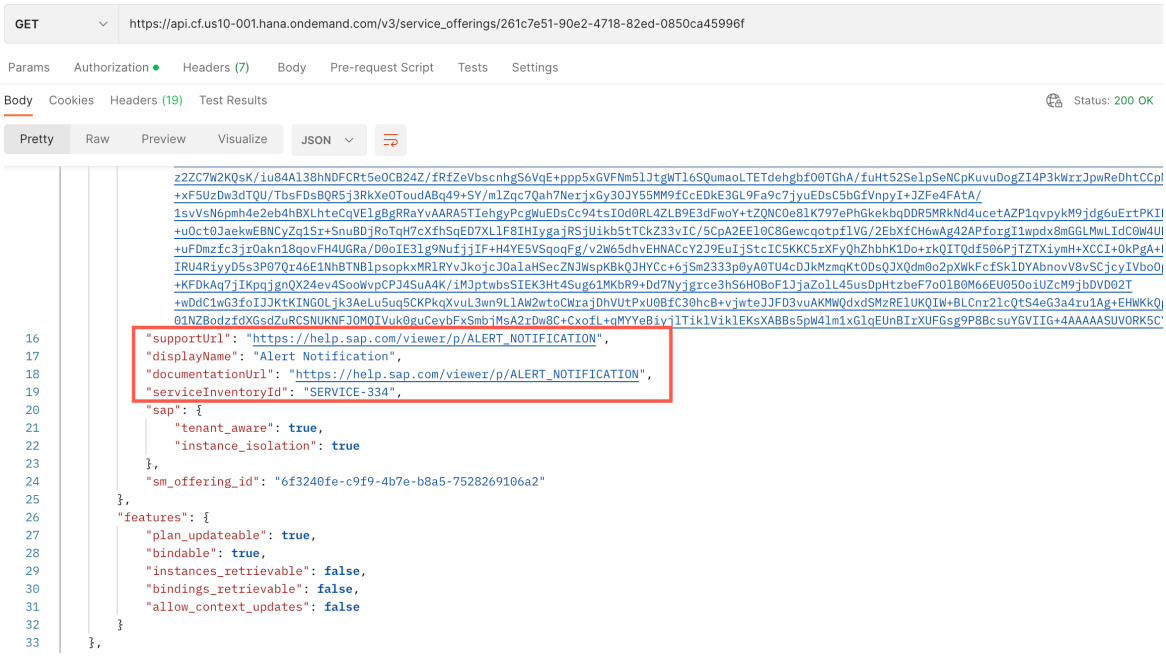
1. Get a list of all audit events from type `audit.service_instance.create`.



2. Follow the link to the service plan.



3. See the service offering parameters to get the name, documentation URL and other parameters of the service.



6.2.8.9 Troubleshooting

You can use the Guided Answers for Audit Logging in the Cloud Foundry Environment page as self service to troubleshoot your issues.

If you don't manage to solve your issue with the Guided Answers, you can open a support ticket in SAP4Me, in which you need to provide:

- Your GA ID
- Your subaccount ID
- Step by step description of the issue with screenshots.

6.3 Administration and Operations in the ABAP Environment

As an administrator in the ABAP environment, you have different tools and features available that support you in your daily work.

Task Examples

As an administrator in the ABAP environment, you focus on administration and operational tasks that aren't managed by SAP. Such tasks can include, for example:

- Account and organization management, including identity and access management for your users
- Monitoring of performance issues and exceptions in your own applications
  Such a monitoring includes response times of application-related communication endpoints, SQL statements issued by the application, application jobs, or ABAP runtime errors originating from custom code.
- System sizing, including a regular review of the overall resource utilization
  Such a system sizing can include a daily check of resource utilization peaks and a monthly check of resource utilization trends.
- Software lifecycle of custom applications (add-on updates or custom transports)
- Tenant management if you manage multi-tenant custom applications
- Monitoring of integrations with other cloud or on-premise based services

Tools, Apps, and Functions: Overview

Here's an overview of tools, apps, and functions that are available to you to perform administration and operations tasks, such as the following:

Task	Tools, Apps, or Functions
Managing accounts and organizations	As an administrator in the ABAP environment, you can create additional accounts for users. In addition, you can manage orgs, spaces, and space quota plans to organize your subaccount into smaller units. You use the tools for account and org administration from the Cloud Foundry environment for this purpose. For more information, see the following: • Account Administration in the Cockpit • Org Administration Using the Cockpit • Org Administration Using the Cloud Foundry CLI For more information about business roles in the ABAP environment, see Business Catalogs and Business Roles [page 2917].
Performing administration tasks, such as identity and access management, business configuration, communication management, and more	In the Fiori launchpad, you can find apps that support you as an administrator or developer with various operational tasks. For more information, see SAP Fiori Apps in the ABAP Environment [page 2916].
Performing technical operations, such as monitoring the performance of your systems as well as the exhaustion of your purchased service volume	You can use tools such as the technical monitoring cockpit, health monitoring, and more. There's also guidance available for the most common tasks in technical operations and for system sizing. For more information, see Technical Operations [page 3378].
Operating communications when you integrate your system or solution with other systems to enable data exchange in your ABAP environment	For more information, see Communication Operations [page 3286].
Performing lifecycle management operations and system administration tasks such as • stopping currently unused systems and scheduling (regularly recurring) stop/starts for a system to save resource usage. • restoring recently deleted tenants that are still in retention time. • monitoring operations.	TheLandscape Portal functions as a central tool for providers to perform lifecycle management operations and system admininistation tasks, seeManage System Hibernation,Restore Consumer Tenants, Operations Dashboard.
Getting notifications about planned maintenance events and about unplanned service outages	Register for the Cloud Availability Center. For more information, see the information about the Cloud Availability Center on SAP Support Portal.

Task	Tools, Apps, or Functions
Operating cloud-centric SAP solution landscapes	SAP Cloud ALM is the central entry point to operate cloud-centric SAP solution landscapes. You can use it for the following: • Central monitoring of service level indicators (real user monitoring, job monitoring, exception monitoring) • Central monitoring of key performance indicators for system and applications (health monitoring) • Alert notifications (currently limited to SAP-defined metrics of the health monitoring framework) For more information about the available metrics for health monitoring, see Central Health Monitoring Using SAP Focused Run and SAP Cloud ALM [page 3411].

6.3.1 SAP Fiori Apps in the ABAP Environment

In the SAP Fiori launchpad for the ABAP environment, you can use apps to help you with your tasks as administrator and developer.

Apps for the ABAP environment help you with the following, for example:

- Manage lifecycle maintenance of business users
- Maintain certificate trust lists
- Integrate your systems in the ABAP environment with other systems to enable data exchange
- Manage software components in your system landscape

Prerequisites

To be able to view and use the apps in the SAP Fiori launchpad, you need a business role with specific business catalogs. You can either use the business roles that are available with the ABAP environment or you can create business roles that include the relevant business catalogs. For more information about the business catalogs and business roles, see Business Catalogs and Business Roles [page 2917]. For more information about creating business roles from available business role templates in the ABAP environment, see also How to Create a Business Role from a Template [page 3072].

Related Information

SAP Fiori Launchpad - User Guide

6.3.1.1 Business Catalogs

Business Catalogs and Business Roles [page 2917]
Get an overview about which business catalogs and business roles are required for applications in the ABAP environment.

Business Catalogs for Development Tasks [page 2928]
Get an overview of available business catalogs for development tasks and their restrictions.

Business Catalogs for Key User Tasks [page 2930]
Get an overview of available business role catalogs and their restrictions.

6.3.1.1.1 Business Catalogs and Business Roles

Get an overview about which business catalogs and business roles are required for applications in the ABAP environment.

> → Tip
>
> You can also use the *Business Catalogs* app to view the details of each business catalog, including their descriptions and the apps they provide access to.

Area	Application	Business Role Template ID	Business Catalog ID
Intelligent Scenario Lifecycle Management	Manage Gen AI Scenarios [page 964]	`SAP_BR_ANALYTICS_SPECIALIST`	`SAP_CORE_BC_ISM_ISC` `SAP_CORE_BC_ISM_ISO`
Extensibility	Configure Predefined Custom Fields [page 3030]	`SAP_BR_EXTENSIBILITY_SPEC` *Administrator*	`SAP_CORE_BC_EXT_PCF_PC` *Extensibility - Predefined Custom Fields*
Extensibility	Custom Logic [page 3018]	`SAP_BR_EXTENSIBILITY_SPEC` *Administrator*	`SAP_CORE_BC_EXT_BLE` *Extensibility - Key User Adaptation*
Extensibility	Adapting SAP Fiori UIs at Runtime - Key User Adaptation	`SAP_BR_EXTENSIBILITY_SPEC` *Administrator*	`SAP_CORE_BC_EXT_FLX_CUS_PC` *Extensibility - Key User Adaptation Customizing*
Extensibility	Simplified BRF+ Workbench: API for Consumption of Business Rules [page 1832]	`SAP_BR_EXTENSIBILITY_SPEC` *Administrator*	`SAP_CORE_BC_EXT_FDT` *Extensibility - Business Rules*

Area	Application	Business Role Template ID	Business Catalog ID
Employee Master Data	Maintain Employees [page 3016]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_HCM_BC_EMP_MDT_PC *Employee - Master Data*
Read Access Logging	Read Access Logging Configuration [page 3145]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_RAL_CONF *Read Access Logging - Configuration*
Read Access Logging	Read Access Logging: Monitor [page 3168]	SAP_BR_DATA_PRIVACY_SPECIALIST *Data Privacy Specialist*	SAP_CORE_BC_RAL *Read Access Logging - Monitoring*
Business Event Logging	Business Event Logging Configuration [page 3265]	SAP_BR_BPC_EXPERT *Configuration Expert - Business Process Configuration*	SAP_CA_BC_IC_LND_BEL_BCO_PC *Business Event Log - Configuration*
Business Event Logging	Display Business Event Logs [page 3266]	SAP_BR_BUSINESS_PROCESS_SPEC *Business Event Log - Monitoring*	SAP_CA_BEL_PC *Business Event Log - Monitoring*
Business Event Logging	Display Business Events by Objects [page 3267]	SAP_BR_BUSINESS_PROCESS_SPEC *Business Process Specialist*	SAP_CA_BEL_PC *Business Event Log - Monitoring*
Business Event Logging	Display Changes to Objects [page 3268]	SAP_BR_BUSINESS_PROCESS_SPEC *Business Process Specialist*	SAP_CA_BEL_PC *Business Event Log - Monitoring*
User Interface Configuration	Managing Launchpad Spaces and Pages	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_UI_FLD *User Interface - Fiori Launchpad Design*
User Interface Configuration	Manage Launchpad Settings	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_UI *User Interface - Configuration*
User Interface Monitoring	Display Launchpad Content Exposure Logs [page 3260]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_UI_MON *User Interface - Monitoring*
ABAP Test Cockpit	Configure ABAP Test Cockpit [page 2933]	SAP_BR_ADMINISTRATOR_SW_DEV *Administrator - Software Development*	SAP_CORE_BC_ATC_CONFIG *ABAP Test Cockpit Configuration*

Area	Application	Business Role Template ID	Business Catalog ID
ABAP Test Cockpit	Manage API Snapshots [page 2936]	SAP_BR_ADMINISTRATOR_SW_DEV *Administrator - Software Development*	SAP_CORE_BC_ARS_PC *Lifecycle Management - API Snapshots*
ABAP Test Cockpit	Approve ATC Exemptions [page 2934]	SAP_BR_QUALITY_MNGR_SW_DEV *Quality Manager - Software Development*	SAP_A4C_BC_DEV_XMPT_APR_PC *Quality Management - ABAP Test Cockpit Exemption Approval (Central Check System)*
Business Configuration	Business Configuration Change Logs [page 2968]	SAP_BR_BPC_EXPERT *Configuration Expert - Business Process Configuration*	SAP_CA_BC_IC_LND_PC *Business Configuration - Customizing*
Business Configuration	Manage Number Range Intervals [page 2972]	SAP_BR_BPC_EXPERT *Configuration Expert - Business Process Configuration*	SAP_CA_BC_IC_LND_PC *Business Configuration - Customizing*
Business Configuration	Upload Business Configuration [page 2975]	SAP_BR_BPC_EXPERT *Configuration Expert - Business Process Configuration*	SAP_CA_BC_IC_LND_PC *Business Configuration - Customizing*
Business Configuration	Custom Business Configurations App [page 2970]	SAP_BR_BPC_EXPERT *Configuration Expert - Business Process Configuration*	SAP_CA_BC_IC_LND_PC *Business Configuration - Customizing*
Business Configuration	Export Customizing Transports [page 2978]	SAP_BR_BPC_EXPERT *Configuration Expert - Business Process Configuration*	SAP_CORE_BC_BCT_TRN_MNG_PC *Business Configuration - Transport Management* SAP_CORE_BC_BCT_TRN_REL_PC *Business Configuration - Transport Release Management*
Application Jobs	Application Jobs [page 2955]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_APJ_JCE *Application Jobs*
Application Jobs	Maintain Job Users [page 2964]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_APJ_JCE *Application Jobs*

Area	Application	Business Role Template ID	Business Catalog ID
Application Jobs	Application Job Templates [page 2960]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_APJ_TPL *Application Job Templates*
System Management	Maintain User Sessions [page 3194]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_CST_DP_MAS_PC *System Management - Sessions*
Factory Calendar	Maintain Holidays [page 3033]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CA_BC_IC_LND_CAL_CA1_PC *Factory Calendar - Configuration*
Factory Calendar	Maintain Holiday Calendars [page 3035]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CA_BC_IC_LND_CAL_CA1_PC *Factory Calendar - Configuration*
Factory Calendar	Maintain Factory Calendars [page 3037]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CA_BC_IC_LND_CAL_CA1_PC *Factory Calendar - Configuration*
Health Monitoring	Health Monitoring [page 3203]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_TMC *Technical Monitoring*
Identity and Access Management	Business Catalogs [page 3092]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_IAM_RM *Role Management*
Identity and Access Management	Business Role Templates [page 3094]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_IAM_RM *Role Management*
Identity and Access Management	Display Authorization Trace [page 3104]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_IAM_RM *Role Management* SAP_CORE_BC_IAM_RA *Role Assignment*
Identity and Access Management	Display Restriction Types [page 3097]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_IAM_RM *Role Management* SAP_CORE_BC_IAM_RA *Role Assignment*

Area	Application	Business Role Template ID	Business Catalog ID
Identity and Access Management	Display Technical Users [page 3053]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_IAM_UM *User Management*
Identity and Access Management	IAM Information System [page 3102]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_IAM_RM *Role Management* SAP_CORE_BC_IAM_RA *Role Assignment* SAP_CORE_BC_IAM_UM *User Management*
Identity and Access Management	Maintain Business Roles [page 3068]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_IAM_RA *Role Assignment* SAP_CORE_BC_IAM_RM *Role Management*
Identity and Access Management	Maintain Business Users [page 3044]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_IAM_RA *Role Assignment* SAP_CORE_BC_IAM_UM *User Management*
Identity and Access Management	Maintain Business User Groups [page 3049]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_IAM_UM *User Management* SAP_CORE_BC_IAM_UM_DISP_PC *Identity and Access Management - User Management - Display* SAP_CORE_BC_IAM_GRP_PC *Identity and Access Management - Group Management*

Area	Application	Business Role Template ID	Business Catalog ID
Identity and Access Management	Maintain Business Role Groups [page 3090]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_IAM_RM *Role Management* SAP_CORE_BC_IAM_RA *Role Assignment* SAP_CORE_BC_IAM_RM_DISP_PC *Identity and Access Management - Role Management - Display* SAP_CORE_BC_IAM_GRP_PC *Identity and Access Management - Group Management*
Identity and Access Management	Maintain Deleted Business Users [page 3051]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_IAM_UMD *Identity and Access Management - User Management of Deleted Users*
Identity and Access Management	IAM Key Figures [page 3103]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_IAM_UM *User Management* SAP_CORE_BC_IAM_RA *Role Assignment* SAP_CORE_BC_IAM_RM *Role Management*
Security	Maintain Certificate Trust List [page 3186]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_SEC *Security*
Security	Maintain Protection Allow-lists [page 3178]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_SEC *Security*
Security	Manage Content Security Policy [page 3180]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_SEC *Security*
Security	Maintain Client Certificates [page 3187]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_SEC *Security*

Area	Application	Business Role Template ID	Business Catalog ID
Security	Display Security Audit Log [page 3190]	SAP_BR_ADMINISTRATOR *Administrator* SAP_BR_DATA_PRIVACY_SPECIALIST *Data Privacy Specialist* SAP_BR_EXTERNAL_AUDITOR *External Auditor*	SAP_CORE_BC_SEC_SAL *Security Audit Log*
Communication Management	Communication Arrangements [page 2997]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_COM *Communication Management*
Communication Management	Display Communication Scenarios [page 2996]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_COM *Communication Management*
Communication Management	Communication Systems [page 3001]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_COM *Communication Management*
Communication Management	Maintain Communication Users [page 2993]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_COM *Communication Management*
Communication Management	Maintain Extensions on SAP BTP [page 3007]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_COM *Communication Management*
Communication Management	Maintain SAML 2.0 Configuration [page 3014]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_BCR_CORE_SEC_IDP_PC *Security Identity Provider*
Communication Management	Display Connectivity Trace [page 3011]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_COM *Communication Management*
Communication Management	Monitor bgRFC Queues [page 3012]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_COM *Communication Management*
Communication Management	Display Inbound Services [page 3008]	SAP_BR_ADMINISTRATOR *Administrator*	*Communication Management* (SAP_CORE_BC_COM) *Communication Management - Display* (SAP_CORE_BC_COM_DISP_PC)

Area	Application	Business Role Template ID	Business Catalog ID
Communication Management	Display Outbound Services [page 3009]	SAP_BR_ADMINISTRATOR *Administrator* SAP_BR_CONF_EXPERT_BUS_NET_INT *Configuration Expert - Business Network Integration*	*Communication Management* (SAP_CORE_BC_COM) *Communication Management - Display* (SAP_CORE_BC_COM_DISP_PC)
i18n Services	Language Configuration [page 3039]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_I18N_LANG *Language Configuration*
Output Management	Maintain Print Queues [page 3128]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_OM_PRT *Output Management - Printing*
Output Management	Monitor Email Transmissions [page 3139]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_OM_EQM_PC *Output Management - Monitor Email Transmissions*
Additional Software	Install Additional Software [page 2954]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_IAS *Additional Software*
Custom Code Migration	Custom Code Analysis [page 2941]	SAP_BR_IT_PROJECT_MANAGER *Project Manager - IT*	SAP_CORE_BC_CCM *Custom Code Migration*
Software Component Lifecycle Management	Manage Software Components [page 3222]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_A4C_BC_MSCL_PC *Lifecycle Management - Software Components*
Message Monitoring	SOAP Error Log, OData Error Log, and Event Error Log [page 3125]	SAP_BR_CONF_EXPERT_BUS_NET_INT *Configuration Expert - Business Network Integration*	SAP_CA_BC_COM_TECH_ERR_PC *Communication Management - Technical Message Monitoring*
Message Monitoring	Message Dashboard [page 3120]	SAP_BR_CONF_EXPERT_BUS_NET_INT *Configuration Expert - Business Network Integration*	SAP_CA_BC_COM_TECH_ERR_PC *Communication Management - Technical Message Monitoring*

Area	Application	Business Role Template ID	Business Catalog ID
Message Monitoring	Message Monitoring Overview [page 3121]	SAP_BR_CONF_EXPERT_BUS_NET_INT *Configuration Expert - Business Network Integration*	SAP_CA_BC_COM_TECH_ERR_PC *Communication Management - Technical Message Monitoring*
Message Monitoring	Message Monitoring for Integration Experts [page 3122]	SAP_BR_CONF_EXPERT_BUS_NET_INT *Configuration Expert - Business Network Integration*	SAP_CA_BC_COM_TECH_ERR_PC *Communication Management - Technical Message Monitoring*
Message Monitoring	Assign Recipients to Users [page 3124]	SAP_BR_CONF_EXPERT_BUS_NET_INT *Configuration Expert - Business Network Integration*	SAP_CA_BC_COM_TECH_ERR_PC *Communication Management - Technical Message Monitoring*
Message Monitoring	Automatic Reprocessing [page 3127]	SAP_BR_CONF_EXPERT_BUS_NET_INT *Configuration Expert - Business Network Integration*	SAP_CA_BC_COM_EXT_ERR_PC *Communication Management - Message Monitoring and Extended Error Handling*
Message Monitoring (Emergency Correction)	Message Monitoring (Emergency Correction) [page 3124]	SAP_BR_CONF_EXPERT_BUS_NET_INT *Configuration Expert - Business Network Integration*	SAP_CA_BC_COM_EMC_PC *Communication Management - Emergency Message Monitoring*
Development	SQL Trace Analysis [page 3015]	SAP_BR_DEVELOPER *Developer* SAP_BR_APPL_SUP_ENG_DEV_SUP *Application Support Engineer - Development Support*	SAP_A4C_BC_DEV_SUP_PC *Development - Analysis and Support*
Development	Display Publishing Processes [page 3015]	SAP_BR_DEVELOPER *Developer* SAP_BR_APPL_SUP_ENG_DEV_SUP *Application Support Engineer - Development Support*	SAP_A4C_BC_DEV_SUP_PC *Development - Analysis and Support*

Area	Application	Business Role Template ID	Business Catalog ID
Technical Monitoring	Application System Overview [page 3197]	SAP_BR_ADMINISTRATOR *Administrator* SAP_BR_DEVELOPER *Developer*	SAP_CORE_BC_TMC *Technical Monitoring*
Technical Monitoring	System Workload [page 3200]	SAP_BR_ADMINISTRATOR *Administrator* SAP_BR_DEVELOPER *Developer*	SAP_CORE_BC_TMC *Technical Monitoring*
Technical Monitoring	HANA Table Analysis [page 3198]	SAP_BR_ADMINISTRATOR *Administrator* SAP_BR_DEVELOPER *Developer*	SAP_CORE_BC_TMC *Technical Monitoring*
Technical Monitoring	Capture Request Statistics [page 3202]	SAP_BR_ADMINISTRATOR *Administrator* SAP_BR_DEVELOPER *Developer*	SAP_CORE_BC_TMC *Technical Monitoring*
Technical Monitoring	Perform System Sizing [page 3202]	SAP_BR_ADMINISTRATOR *Administrator* SAP_BR_DEVELOPER *Developer*	SAP_CORE_BC_TMC *Technical Monitoring*
Technical Monitoring	System Outbound Communication [page 3201]	SAP_BR_ADMINISTRATOR *Administrator* SAP_BR_DEVELOPER *Developer*	SAP_CORE_BC_TMC *Technical Monitoring*
Technical Monitoring	Sampled Work Process Data [page 3199]	SAP_BR_ADMINISTRATOR *Administrator* SAP_BR_DEVELOPER *Developer*	SAP_CORE_BC_TMC *Technical Monitoring*
Technical Monitoring	Partition HANA Tables [page 3204]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_COT_PC *Technical Monitoring*

Area	Application	Business Role Template ID	Business Catalog ID
Technical Monitoring	Schedule Metric Provider Collection [page 3204]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_TMC *Technical Monitoring*
Technical Monitoring	Displaying and Analyzing ABAP Runtime Errors [page 3425]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_TMC *Technical Monitoring*
Technical Monitoring	HANA Thread Samples [page 3199]	SAP_BR_ADMINISTRATOR *Administrator* SAP_BR_DEVELOPER *Developer*	SAP_CORE_BC_TMC *Technical Monitoring*
Technical Monitoring	HANA Memory Analysis [page 3207]	SAP_BR_ADMINISTRATOR *Administrator* SAP_BR_DEVELOPER *Developer*	SAP_CORE_BC_TMC *Technical Monitoring*
Technical Monitoring	SQL Statement Analysis [page 3208]	SAP_BR_ADMINISTRATOR *Administrator* SAP_BR_DEVELOPER *Developer*	SAP_CORE_BC_TMC *Technical Monitoring*
Translation	Maintain Translations [page 3209]	SAP_BR_DEVELOPER *Administrator*	SAP_A4C_BC_DEV_TRA_PC *Translation*
Customizing Translation	Maintain Customizing Translations [page 2985]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_BCT_TRA_PC *Customizing Translation*
ABAP Dictionary	Manage Database Cache Configuration [page 2932]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_A4C_DIC *Administration - ABAP Dictionary*
ABAP Dictionary	Repair CDS Views [page 2931]	SAP_BR_ADMINISTRATOR *Administrator*	SAP_CORE_BC_A4C_DIC *Administration - ABAP Dictionary*

Related Information

Business Catalogs for Development Tasks [page 2928]

6.3.1.1.2 Business Catalogs for Development Tasks

Get an overview of available business catalogs for development tasks and their restrictions.

You assign business catalogs to business roles that are assigned to business users. Business catalogs contain authorizations that define what a business user with a certain business role is allowed to do.

Certain business catalogs are only available in development systems (see Creating an ABAP System [page 198]).

Business Catalogs for Development Tasks

Business Catalog	Authorizations	Restrictions	Availability
Development - Joule with ABAP Developer Capabilities Quota SAP_A4C_BC_DEV_AIQ_PC	Using Joule for ABAP	No transport request management (transport tasks only)	In development systems only
Development - ABAP Development Tools SAP_A4C_BC_DEV_PC	ADT Development	-	In development systems only
Development - Analysis and Support SAP_A4C_BC_DEV_SUP_PC	Troubleshooting tools such as logs, traces, and the debugger. **Note** To enable the read-only debugging authorization, set the access category Write, Read, Value Help to *No Access* and the access category Read, Value Help to *Unrestricted* access. To enable debugging with value change in customer code, set the access category Write, Read, Value Help to *Unrestricted*. See How to Define Authorizations Based on Restrictions [page 3075].	-	In all systems
Development - API Test SAP_A4C_BC_DEV_TST_PC	Testing ABAP-based APIs released by SAP	-	In development systems only

Business Catalog	Authorizations	Restrictions	Availability
Development - Class Runner Execution SAP_A4C_BC_DEV_CLA_RUN_PC	Executing class runners in ABAP Development Tools	-	In all systems
Development - Data Preview - Released Objects SAP_A4C_BC_DEV_DAT_PRV_PC	Using data preview in ABAP Development Tools	-	In all systems
Development - Development Objects Display SAP_A4C_BC_DEV_OBJ_DIS_PC	Viewing (read-only) development objects in ABAP Development Tools	-	In all systems
Development - Transport Management SAP_A4C_BC_TRN_MNG_PC	ADT Transport Management	No release of transport requests and no customizing requests	In development systems only
Development - Transport Release Management SAP_A4C_BC_TRN_REL_PC	ADT Transport Release Management	No customizing requests	In development systems only
Development - UI Deployment SAP_A4C_BC_DEV_UID_PC	Deployment of UIs into the ABAP system repository	-	In development systems only
Development Support - Data Preview - Business Data SAP_A4C_BC_DAT_PRV_DFT_PC	Usage of the data preview in ABAP Development Tools for objects that are considered to contain business data: • CDS views in language version 5 • Client-dependent tables in language version 5 with delivery class A and L Data of CDS views can be viewed with or without application of the DCL.	-	In all systems

Business Catalog	Authorizations	Restrictions	Availability
Development Support - Data Preview - Cross-Client and Customizing Data SAP_A4C_BC_DAT_PRV_RDO_PC	Usage of the data preview in ABAP Development Tools for objects that are considered to contain customizing and cross-client data: • Client-independent tables in language version 5 with delivery class A and L • Tables in language version 5 with delivery class S, E, and W (metadata) • Tables in language version 5 with delivery class C and G (customizing)	-	In all systems
Extensibility - Custom Apps and Services SAP_CORE_BC_EXT_TST	• Previewing UIs in SAP Business Application Studio and Microsoft Visual Studio Code • Previewing service bindings • Testing custom apps and custom services	Only services that have their original in the current system	In development systems only

6.3.1.1.3 Business Catalogs for Key User Tasks

Get an overview of available business role catalogs and their restrictions.

You assign business catalogs to business roles that are assigned to business users. Business catalogs contain authorizations that define what a business user with a certain business role is allowed to do.

Certain business catalogs are only available in tenants of specific Tenant Business Types [page 1350].

Business Catalogs for Key User Tasks

Business Catalog	Authorizations	Restrictions	Availability
Extensibility - Key User Adaptation SAP_CORE_BC_EXT_FLX_CUS_PC	Key user adaptation		All tenant business types
SAP_CORE_BC_EXT_FDT	Maintain `Business Role Framework+` applications		All tenant business types

Business Catalog	Authorizations	Restrictions	Availability
SAP_CORE_BC_EXT_PCF_PC	Configuration of custom fields		Test tenant, partner customer test tenant, partner customer production tenant
SAP_CORE_BC_EXT_BLE	Implementation of custom logic		Test tenant, partner customer test tenant, partner customer production tenant
Extensibility - Custom Apps and Services SAP_CORE_BC_EXT_TST	Testing of custom apps and custom services	Only services that have their original in the current system	Partner development tenant

6.3.1.2 ABAP Dictionary

Repair CDS Views [page 2931]

Manage Database Cache Configuration [page 2932]

6.3.1.2.1 Repair CDS Views

With this app you can view and repair the CDS views that are in inconsistent state and update entities of the logical external schemas.

Key Features

You can use this app to:

- **View** the details of the inconsistent CDS views such as the application component, the error category, the person who is responsible, and the package details
- **Search** for a specific view based on the view name or filter the views based on the error category
- **Repair** the inconsistent views
- **Update** entities of the logical external schemas

In addition, the app allows you to export the CDS views to spreadsheets.

Supported Device Types

- Desktop

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-DWB-DIC-ADM`.

6.3.1.2.2 Manage Database Cache Configuration

With this app you can view and manage the database cache configurations.

Key Features

a) You can use this app to:

- View the list of caches in the system.
- Search for a particular cache based on the name or the data source.
- View the cache-specific information such as total number of records cached and size available for consumption.
- Create or drop a cache by choosing *Create* or *Drop* respectively.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-DWB-DIC-ADM`.

6.3.1.3 ABAP Test Cockpit

Configure ABAP Test Cockpit [page 2933]
SAP Fiori app for editing ATC configurations.

Approve ATC Exemptions [page 2934]

Manage API Snapshots [page 2936]

Custom Code Analysis [page 2941]

6.3.1.3.1 Configure ABAP Test Cockpit

SAP Fiori app for editing ATC configurations.

Purpose

The *Configure ABAP Test Cockpit* app enables you to maintain ATC configurations using SAP Fiori launchpad.

> **Note**
>
> The *Configure ABAP Test Cockpit* app enables you to maintain multiple configurations. Edits done in transactions `ATC` and `SCI` only affect the **active** configuration.

Key Features

1. Assign a check variant.
2. Change priorities.
3. Set priority levels that block or interrupt transport releases.
4. Set a default configuration.
5. Set handling of pseudo comments and pragmas.

Access Information

For more information about how to provide access to users and how to implement this app, see Enable Usage of the Configure ABAP Test Cockpit App [page 2934].

Component for Customer Incidents

BC-DWB-TOO-ATF

Supported Device Types

- Desktop

Related Information

Enable Usage of the Configure ABAP Test Cockpit App [page 2934]

6.3.1.3.1.1 Enable Usage of the Configure ABAP Test Cockpit App

This is the enablement topic for the Configure ABAP Test Cockpit Fiori app.

Prerequisites

- Access to SAP Fiori launchpad with administrator authorization

Overview

1. Create a business role either from a template or from scratch.
   - To use a template, see How to Create a Business Role from a Template [page 3072].
     Choose the template `SAP_BR_ADMINISTRATOR_SW_DEV`.
   - To create a role from scratch, see How to Create a New Business Role [page 3070].
     Choose `ABAP Test Cockpit Configuration` (business catalog ID: `SAP_CORE_BC_ATC_CONFIG`) as catalog for this role (step 3).
2. Assign the business role you created to a business user.
   (See also: How to Maintain Business Users [page 3045].)

When you log on to SAP Fiori launchpad with the assigned business user, the tile *Configure ABAP Test Cockpit* will be available.

6.3.1.3.2 Approve ATC Exemptions

With this app you can approve or reject ATC exemptions via the SAP Fiori launchpad.

Key Features

With this app you can:

- Approve exemption requests.

- Reject exemption requests.
- See a list of exemptions of all states.
- Apply filters to the list of exemptions, such as exemption state, processor and requester.
- See a justification for an exemption request.
- Add or remove columns to or from the list of exemptions to customize your view.
- See the scope the requester would like their exemption request to be valid for through the object scope (finding, object, sub-object and all objects of a package) (granularity).
- See the scope the requester would like their exemption request to be valid for through the check scope (finding, check message, check) (validity).
- See the system group for which an exemption would be valid for.
- See the expiry date of an exemption. The processor of the request can change the expiry date after request creation.
- Give an assessment for an exemption request.
- Replicate exemptions from an on-premise system to SAP BTP.
- Subscribe/unsubscribe to e-mail notifications informing you that a developer has requested a new ATC exemption.

> ⓘ Note
>
> Use transaction `SATC_EXPORT_XMPT` to download exemptions from an on-premise ATC system. For customers that are on a release between 7.52 and 7.58 with software component SAP_BASIS, apply SAP Note 3480722 first to install this functionality.

> → Remember
>
> The Approve ATC Exemptions app only supports exemptions when the SAP BTP ABAP environment acts as the central check system and when exemptions are requested via the ATC developer scenario (see Configuring the ATC Developer Scenario). Exemptions of type `CHKE` are not supported. To learn more about how to approve or reject `CHKE` exemptions in ABAP development tools for Eclipse, see Approving and Rejecting ATC Exemptions.

Access Information

For more information on how to enable this app, see Enable Usage of the Approve ATC Exemptions App [page 2936].

Component for Customer Incidents

If you need support or are experiencing issues, please report an incident under component `BC-DWB-TOO-ATF`.

Supported Device Types

- Desktop
- Tablet
- Smartphone

6.3.1.3.2.1 Enable Usage of the Approve ATC Exemptions App

Prerequisites

You have access to the SAP Fiori launchpad with a quality manager authorization and you've configured the ATC Developer Scenario. For more information, see Using an SAP BTP System as ATC Central Check System.

Procedure

1. Create a business role either from a template or from scratch.
   - To use a template, see How to Create a Business Role from a Template [page 3072].
     Choose the template `SAP_BR_QUALITY_MNGR_SW_DEV`.
   - To create a role from scratch, see How to Create a New Business Role [page 3070].
     Choose `Approve ATC Exemptions` (business catalog ID: `SAP_A4C_BC_DEV_XMPT_APR_PC`) as catalog for this role (step 3).
2. Assign the business role you created to a business user.
   (See also: How to Maintain Business Users [page 3045].)

When you now log on to the SAP Fiori launchpad with the assigned business user, the tile *Approve ATC Exemptions* will be available.

Related Information

Approve ATC Exemptions [page 2934]
SAP Note 3480722 - Downport Program to Export ATC Exemptions

6.3.1.3.3 Manage API Snapshots

With this app you can create API snapshots of your software components. An API snapshot is a snapshot of the external interface of all released objects within a software component. This snapshot should be taken

when the released objects within the software component must be kept stable. API snapshots allow you to run compatibility checks for your released objects using the ATC check *API Release: Compatibility of Released APIs* (`API_COMPATIBILITY`).

Key Features

You can use this app to:

- Create API snapshots to be able to use them in the compatibility check
- Set the check-relevance of API snapshots
- Regenerate the snapshots of selected APIs
- Regenerate the snapshots of all failed APIs
- Regenerate already existing API snapshots completely
- Download and upload API snapshots
- Display the changes made to your API snapshots
- Display the included APIs

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-EXT-REL`.

6.3.1.3.3.1 Enable Usage of the Manage API Snapshots App

Find out how to enable your *Manage API Snapshots* app in the `SAP Fiori launchpad`.

Prerequisites

You already have access to the `SAP Fiori launchpad` and you have the administrator authorization.

Overview

1. Create a business role either from a template or from scratch.
   - To use a template, see How to Create a Business Role from a Template [page 3072].
     Choose the template `SAP_BR_ADMINISTRATOR_SW_DEV`.
   - To create a role from scratch, see How to Create a New Business Role [page 3070].
     Choose `API Snapshots` (business catalog ID: `SAP_CORE_BC_ARS_PC`) as catalog for this role.
2. Assign the business role you created to a business user.
   (See also: How to Maintain Business Users [page 3045].)

When you log on to `SAP Fiori launchpad` with the assigned business user, the tile `Manage API Snapshots` will be available.

6.3.1.3.3.2 Creating API Snapshots

Find out how to create API snapshots using the *Manage API Snapshots* app.

Context

To be able to run compatibility checks for your released objects using the ATC check *API Release: Compatibility of Released APIs* (`API_COMPATIBILITY`), you need to create API snapshots of your software components beforehand. This app allows you to generate and manage your API snapshots. It's recommended to create snapshots in the test system, because your released objects exist in a consolidated version and are not disturbed by any development activities. To detect incompatible changes to your released APIs early in the development process, download your snapshots from the test system and upload them in the development system. This way, the ATC check can detect incompatible changes already before they are transported.

If the test system acts as a product version build system, snapshots are created automatically by the system during the build process.

Procedure

1. Open the *Manage API Snapshots* app.
2. Click *Create*. Choose a name for your snapshot and select the software component from the value help.
3. Click *Create* again to create the snapshot metadata only, or select *Create and Generate* to generate a snapshot, too. Once the snapshot generation has finished, the action status will change to *Completed*. You can view the snapshot details by clicking on the snapshot.

   > **Note**
   >
   > If you release your object and a snapshot doesn't exist already, it's automatically created. This default snapshot appears in your list of snapshots as *SAP Default*. An SAP default snapshot has limited use: you can't regenerate or delete this snapshot, but you can regenerate single APIs of the SAP default

snapshot in the *APIs* tab, however. You can set the check relevance of the snapshot as usual. Mind that if at the time of its creation, the SAP default snapshot is the only available snapshot in the respective software component, or if no other snapshot of the respective software component is set to check-relevant, it'll be set to check-relevant automatically.

4. In the *Snapshot API Details*, you can choose to maintain your snapshot in the following way:
   a. Change the name of your snapshot by clicking *Edit* in the top right corner of your screen in the *General Information* tab.
   b. Regenerate single APIs of your snapshot in the *APIs* tab. To regenerate your APIs, select the ones in question and click *Regenerate APIs*. Remember that the regeneration of APIs is irreversible.

      > ⓘ Note
      >
      > The APIs of uploaded snapshots can't be regenerated.

   c. Regenerate your snapshot completely by choosing *Regenerate Snapshot* in the top right corner of your screen. Remember that the regeneration of a snapshot is irreversible.

      > ⓘ Note
      >
      > Uploaded snapshots can't be regenerated.

   d. Regenerate all the non-extracted APIs in your snapshot by choosing *Regenerate Snapshot* in the top right corner of your screen, and then, in the button menu, choose *Regenerate Failed APIs*.

      > ⓘ Note
      >
      > The non-extracted APIs of uploaded snapshots can't be regenerated.

   e. Control the check relevance of your snapshot by choosing *Set Check-Relevance* in the top right corner of your screen. In the button menu, set your snapshot to either check-relevant or not check-relevant.

      > ⓘ Note
      >
      > Only one snapshot per software component can be set as check-relevant.

   f. Download your snapshot for use in another system by clicking *Prepare Download*. When the snapshot is ready for download, choose *Download*.
   g. Create a note for other people to view by selecting the *Notes* tab and posting a note. Notes are meant for the system in which you created the snapshot. They are not included in downloaded snapshots.
   h. Delete your snapshot by choosing *Delete*.
   i. View the change history of your snapshot by selecting the *History* tab.
   j. View your error logs in the *Generation Log* tab.

You've now created a snapshot that you can use for your compatibility check.

6.3.1.3.3.3 Downloading API Snapshots

Find out how to download API snapshots using the *Manage API Snapshots* app.

Context

To be able to run compatibility checks for your released objects using the ATC check *API Release: Compatibility of Released APIs* (`API_COMPATIBILITY`), you need to create API snapshots of your software components beforehand. This app allows you to download API snapshots, preferably in the test system, and to upload them again, typically in the development system. This way, incompatible changes to released objects can be prevented early in the development process. If the test system acts as a product version build system, snapshots are created automatically by the system during the build process.

Procedure

1. Open the *Manage API Snapshots* app.
2. Select the snapshot you want to download.
3. Once selected, choose *Download* and then click on *Prepare*. Wait until the download preparation has been completed.

   > ⓘ Note
   >
   > When you make changes to your snapshot, by regenerating it, for example, the preparation step needs to be performed again.

4. Once the preparation is completed, choose *Download* and download your snapshot. Depending on your browser settings, a zip file is downloaded directly to the configured destination, or you are asked to select a destination for the zip file first.
   You've now downloaded a snapshot that you can use for your compatibility check in another system.

6.3.1.3.3.4 Uploading API Snapshots

Find out how to upload API snapshots using the *Manage API Snapshots* app.

Context

To be able to run compatibility checks for your released objects using the ATC check *API Release: Compatibility of Released APIs* (`API_COMPATIBILITY`), you need to create API snapshots of your software components beforehand. This app allows you to download API snapshots, typically from the test system, and to upload

them again, typically in the development system. This way, incompatible changes to released objects can be prevented early in the development process.

Procedure

1. Open the *Manage API Snapshots* app.
2. Click *Upload*. Alternatively, drag and drop a valid snapshot zip file on the list of snapshots, thereby skipping steps 3 and 4.

   > ⓘ Note
   >
   > The upload process allows for a maximum of five snapshots per software component. If five or more snapshots exist already, you can't upload the snapshot. In this case, if you want to upload another snapshot, you need to delete one first. You can only upload snapshots for software components belonging to you.

3. In the file upload dialog, select a valid snapshot zip file that you've downloaded beforehand.
4. Confirm the file selection to start uploading the snapshot.

   > ⓘ Note
   >
   > If a snapshot with the same name already exists, a unique snapshot name is created by the app. You can change the name of the snapshot on the *API Snapshot Details* screen.

5. If you want the uploaded snapshot to be check-relevant, select the snapshot, click *Set Check-Relevance* and set the snapshot to check-relevant.
   You've now uploaded a snapshot that you can use for your compatibility check.

6.3.1.3.4 Custom Code Analysis

Purpose

As of releases SAP S/4HANA Cloud Public Edition 2508, SAP BTP ABAP environment 2508 and SAP S/4HANA Cloud Private Edition and SAP S/4HANA 2025, the Custom Code Migration app is split into two tiles. There's the **Analyze Custom Code** app for the custom code analysis use case and the **Custom Code Migration** app for the migration to SAP S/4HANA use case. If you have the `Quality Manager – Software Development` business role, you'll find the Analyze Custom Code app on the Fiori launchpad. If you have the `Project Manager – IT` business role, you'll find the Custom Code Migration app.

These apps enable you to create three different kinds of projects:

- SAP S/4HANA Migration Project:
  Analyze custom code that shall be migrated from an existing product like SAP Business Suite to a new product such as SAP S/4HANA 2025. To evaluate the custom objects to be adapted, it performs the SAP S/4HANA custom code checks.
- SAP BTP Analysis Project:

Analyze custom code for readiness to run in SAP Business AI Platform, ABAP environment.

- Custom Code Analysis Project:
  Analyze custom code with arbitrary ATC check variants. See Performing a Custom Code Analysis [page 2946].

In addition, this app supports you with identifying unused custom code based on your collected usage data. This enables you to remove unused custom code during a system conversion to SAP S/4HANA.

Key Features

Analysis of custom code:

- Graphical representation of custom code analysis results
- Results can be filtered by various categories, such as quick fix availability, scope and usage data.
- Options to handle findings after anaylsis (see Process Findings After Analysis [page 2952])

Scoping:

- Based on usage data, you can define the ABAP custom code that needs to be migrated to SAP S/4HANA.
- This app creates a deletion transport in order to delete unused ABAP source code during the system conversion to SAP S/4HANA.

SAP Joule for Developers, ABAP AI Capabilities:

As of SAP BTP ABAP environment release 2602, you have the option to activate an AI-powered feature in the Custom Code Analysis app: The *ATC Explain* capability enables you to get an explanation of your ATC findings in-app.

> **Note**
>
> To leverage Joule's capabilities, you need to purchase an additional license: For SAP Business AI Platform, ABAP environment, SAP S/4HANA Cloud Public Edition, and SAP S/4HANA Cloud Private Edition, see SAP note 3571857.

Access Information

For more information on how to enable this app, see Enable Usage of the Analyze Custom Code App [page 2943].

Component for Customer Incidents

BC-DWB-CEX

Supported Device Types

- Desktop

6.3.1.3.4.1 Enable Usage of the Analyze Custom Code App

Prerequisites

- Your company has a global account for *SAP Business Technology Platform*.
- You need permission to access the *Cloud Connector*.

Procedure

Maintain Business Role

To enable business users to access the Analyze Custom Code aka Custom Code Migration app, create the `Quality Manager – Software Development` business role and assign one or more business user(s) to it. If you only intend to use the Custom Code Migration app, you can also use the `Project Manager – IT` business role. To create these roles, proceed as follow (taking `Project Manager – IT` as an example):

1. From the SAP Fiori launchpad of your ABAP environment, open the *Maintain Business Role* tile.
2. Create the `Project Manager – IT` business role using the `SAP_BR_IT_PROJECT_MANAGER` template. For more information see, How to Create a Business Role from a Template.
3. Maintain the restrictions of the business role.

   > ⓘ Note
   >
   > To enable write access for this business role, ensure that it is set to *Unrestricted*.

4. Open the new business role and add the *Assigned Business Users*.
5. *Save* the changes.

Maintain Communication Arrangement

To enable communication from your ABAP environment to your on-premise systems using Remote Function Calls (RFC), you need to create the communication arrangement SAP Custom Code Migration Integration (`SAP_COM_0464`) in your ABAP environment.

To set up the connection from the ABAP system in the ABAP environment to your on-premise system, proceed as follows:

1. Log on to the *SAP Fiori launchpad* of your ABAP environment.
2. In the *Communication Management* section, select the *Communication Arrangement* tile.
3. Create a new communication arrangement using the `SAP_COM_0464` scenario.

4. If not yet available or defined, create a communication system for the communication arrangement to define an endpoint for your checked system.
    1. For the *Communication System*, choose *New*.
    2. In the *New Communication System* dialog, enter the *System ID* and *System Name* of the checked system.
    3. Choose *Create*.
    4. In the *Technical Data* tab under *General*, enter the virtual host as specified in your *Cloud Connector* as *Host Name*. The field *Port* has already been filled in automatically with the default *443*. The value of the field *UI Host* can be changed if it differs from that of the host.
    5. Switch on the slider for *Cloud Connector*.
    6. Filling in the field *SCC Location ID* is optional.
    7. Under *RFC Settings*, fill in the fields *Client*, *Instance Number* and *Target Host* as specified as virtual host in your *Cloud Connector*.
    8. Under *User for Outbound Communication*, choose + to assign the RFC user you created in the checked system to the communication system. See Configuring Remote ATC Using a Central Check System to learn how to create such an RFC user.
    9. Choose *Create* and then *Save* to save the communication system and to be navigated back to the Communication Arrangement creation page.
    10. Confirm with *Save*.
5. Under *Additional Properties*, fill out the fields *Object Provider* and *System Group*. An object provider defines the RFC connection to be used for analysis in a remote SAP system and must be assigned to a system group. This is the name you then select as your *Object Provider to Remote System* for your custom code migration project in the Analyze Custom Code app. Enter a name for each field. This name can be defined by you and can be 20 characters long (25 for system groups). The object provider will be created on the spot. The system group will either be created right then as well or, if it already exists, the object provider will be grouped within the system group that already groups together other systems and object providers in the SAP BTP system.

    > **Note**
    >
    > A system group subsumes multiple SAP systems (typically, the productive system, the test system(s), and the development system(s)), which all represent a part of a system landscape of one and the same SAP release. If you're using an existing system group, your new object provider will be grouped with the existing object providers in this system group.

6. Under *Outbound Services* > *Retrieve Custom Code*, ensure that the *Service Status* is set to *Active*. Optionally, you can test the connection from the SAP BTP ABAP environment system to the checked system by choosing the *Check Connection* button.
7. Under *Outbound Services* > *Display Custom Code*, ensure that the *Service Status* for displaying custom code via UI link is set to *Active*. You may also need to enter a value for *Port* if the system number is not 00.
8. Choose *Save* to save the communication arrangement. A message should now pop up at the bottom of the screen informing you that the activation was successful.

You can now select the communication arrangement in the *Object Provider to Remote System* field in the Analyze Custom Code app to establish the connection to your on-premise system.

On-Premise System: Install Remote Stubs

To analyze your custom code in your on-premise system using the Custom Code Migration app, see SAP Note 2599695 Custom Code Migration Fiori App: Remote Stubs for the Checked System.

Related Information

SAP Note 2436688
Custom Code Migration Guide for SAP S/4HANA
How to check your custom ABAP code for SAP Business AI Platform, ABAP environment

6.3.1.3.4.1.1 Configuration

You can perform remote code analysis in ATC which allows to analyze custom code remotely with the latest checks.

When using SAP BTP ABAP environment as central check system, you will use the **Custom Code Migration** and **Analyze Custom Code** app to scope and analyze custom code in an on-premise SAP system.

To use this SAP Fiori app, your SAP administrator must enable the app provide access for the relevant users. In addition, you also need to establish an RFC connection for the Cloud Connector. This enables you to access the on-premise system as checked system from the ABAP environment as central check system remotely.

To find out more about this configuration, see Configuring Remote ATC Using a Central Check System.

6.3.1.3.4.1.2 Administrating User Assignments

The user assignment describes the relevant roles that a user requires to access SAP Business AI Platform, ABAP environment using the Analyze Custom Code app.

Context

You want to enable business users to access the Analyze Custom Code app.

For more information see, How to Create a Business Role from a Template.

> **Note**
>
> If you use `Prepare an Account for ABAP Development` booster, you can skip this step. The booster will perform this task.

Procedure

1. In the SAP Fiori launchpad, navigate to the *Identity and Access Management* section and open the `Maintain Business Roles` app.
2. On the initial screen, select *Create from Template* from the bottom of the page.

   The *Create Business Role from Template* wizard is opened.
3. Use the value help to choose *SAP_BR_IT_PROJECT_MANAGER* as the underlying *Template*.
4. Enter *BR_IT_PROJECT_MANAGER* as the name of the *New Business Role ID*.
5. Enter a *New Business Role Description*.
6. The checkbox *Create and Assign Spaces Based on SAP-Delivered Spaces* provides you with the following two options:
   a. **Mark** the checkbox to create a new space for the Analyze Custom Code app in your subaccount. Define and enter a *New Space ID*.
   b. **Unmark** the checkbox to use an existing space for the Analyze Custom Code app. Consequently, you must allocate the Analyze Custom Code app to the existing space and page in your subaccount. For more information, see Working with Predefined Spaces and Pages.
7. Confirm the creation with *OK*.

   The new business role is created and opened in the *Maintain Business Role* page.
8. Select *Maintain Restrictions* from the title toolbar.

   The relevant subpage is opened.
9. In the *Write, Read, Value Help* tab, choose *Unrestricted* from the drop-down list to enable write access for this business role.
10. Navigate back to the main page of your new business role.
11. Open the *Assigned Business Users* tab and add the relevant users using the *Search* field or the *Add* button.
12. *Save* the changes from the bottom.

Results

The business role is created and added. The assigned users now have access to the Analyze Custom Code app.

6.3.1.3.4.2 Performing a Custom Code Analysis

This guide describes how to perform remote custom code analysis using the Analyze Custom Code app.

The Analyze Custom Code app in the SAP BTP ABAP environment can act as the ABAP Test Cockpit (ATC), a central check system analyzing the ABAP source code located in your on-premise system. It enables you to identify functional or security vulnerabilities in your ABAP source code. This includes security checks performed by the SAP Code Vulnerability Analyzer (CVA). Further cloud use cases for ATC in the Analyze Custom Code app involve the migration to S/4HANA, performance checks, and ABAP Cloud readiness checks.

In the following chapters, you will learn how to create a custom code analysis project, analyze the findings, and, in addition, perform dependency and complexity analyses.

Related Information

Code Vulnerability Analyzer Checks

6.3.1.3.4.2.1 Complexity Analysis

In this topic, you will learn more about complexity analysis in the Analyze Custom Code app.

Purpose

Sometimes, your ABAP code can contain complex objects that very often also experience the most frequent changes. With the Analyze Custom Code app it is possible to identify the most complex objects via complexity analysis and help you decide whether you want to redesign and therefore simplify them.

> ⓘ Note
>
> Please be aware that, in this topic, the focus is on reducing the complexity of packages only. Nevertheless, generally, what applies to packages also applies to request entry points.

In your Custom Code Migration project, choose the bubble chart to display all packages that contain custom code. With this chart, you can also display the Halstead difficulty of a package. The Halstead complexity measures are software metrics that describe the effort to develop and understand the complexity of a program.

The chart enables you to identify packages that contain complex coding and have often been changed in the last year, for example.

The default representation of the chart displays the following information:

The X axis indicates the complexity/difficulty of the custom code within a package. The higher the number signifies the complexity. The app calculates the complexity based on the Halstead complexity measures. For packages, the Halstead difficulty is aggregated using an H-index algorithm.

> ⓘ Note
>
> Only if you have multiple complex objects in your code does the H-index rise.
>
> An H-index of 42 means that the package contains 42 procedures with a Halstead difficulty of 42 or higher.
>
> An H-index of 80 means that the package contains 80 procedures with a Halstead difficulty of 80 or higher.
>
> The Y axis displays how many changes – based on transport requests – you have made to the objects of a package.

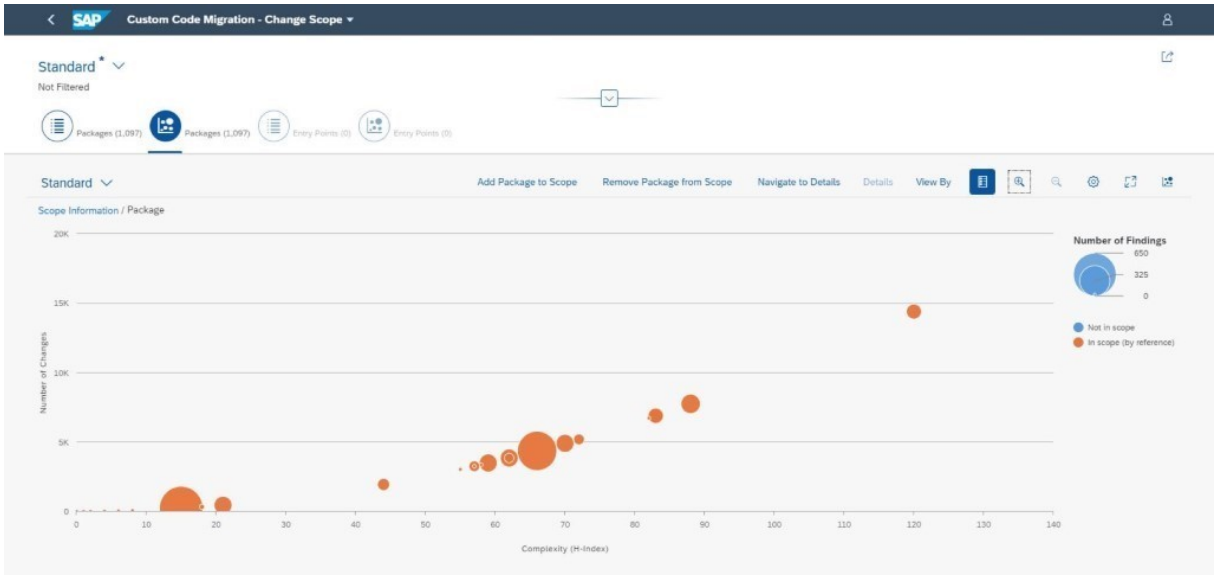
→ Recommendation

SAP recommends you have a closer look at packages that appear in the top-right corner of this chart, as those are the packages that are very complex and have often been changed.

The size of the bubbles indicates the number of findings. Hence, the bigger the bubble, the higher the number of findings.

You can change the default representation of the chart by choosing the chart preferences (⚙).

Your chart could now look as follows:



Now that you are aware of the complexity of your code, it is time to take action and reduce said complexity. There is no standard solution for fixing complex code, however, SAP can provide you with suggestions as to how to start this process:

- Check whether you can replace your code with SAP standard functionalities.
- Check whether there are smaller units in your code that you can reuse.
- Check out the **Clean ABAP** style guide to learn more about clean code.

Related Information

SAP Note 2436688

Custom Code Migration Guide for SAP S/4HANA

How to check your custom ABAP code for SAP Business AI Platform, ABAP environment

6.3.1.3.4.2.2 Dependency Analysis

In this topic, you will learn more about dependency analysis in the Analyze Custom Code app.

Purpose

You can utilize dependency analysis to identify dependencies from your custom code to the SAP standard and thus which part of your custom code can be transformed to the SAP BTP ABAP environment. To do this, you can use the *Scoping* app within the Analyze Custom Code app. This app provides you with a set of measures that analyze how likely it is for you to be able to transform your code to the SAP BTP ABAP environment.

> ⓘ Note
>
> Please be aware that as of the SAP BTP ABAP environment 2411 release, you now need to initiate scope calculation manually after project creation. Scope calculation no longer starts automatically at project creation.

This is how you start a dependency analysis in the Scoping app:

1. In your project, select the *Scope* tab in the lower half of the project overview page.
2. Select *Change Scope*.
3. Select the second *Packages* tab from the left. This tab is marked with a bubble chart icon.
4. Select the *Settings* (⚙) button.
5. Under *Measures*, choose your metrics for the chart axes.

> → Tip
>
> This way you can set up your own individualized chart to perform dependency analysis of your custom code with SAP standard. You can determine, for example, how often your custom code calls SAP application APIs or uses DB accesses to SAP application tables such as the `MARA` table.

The following measures are available:

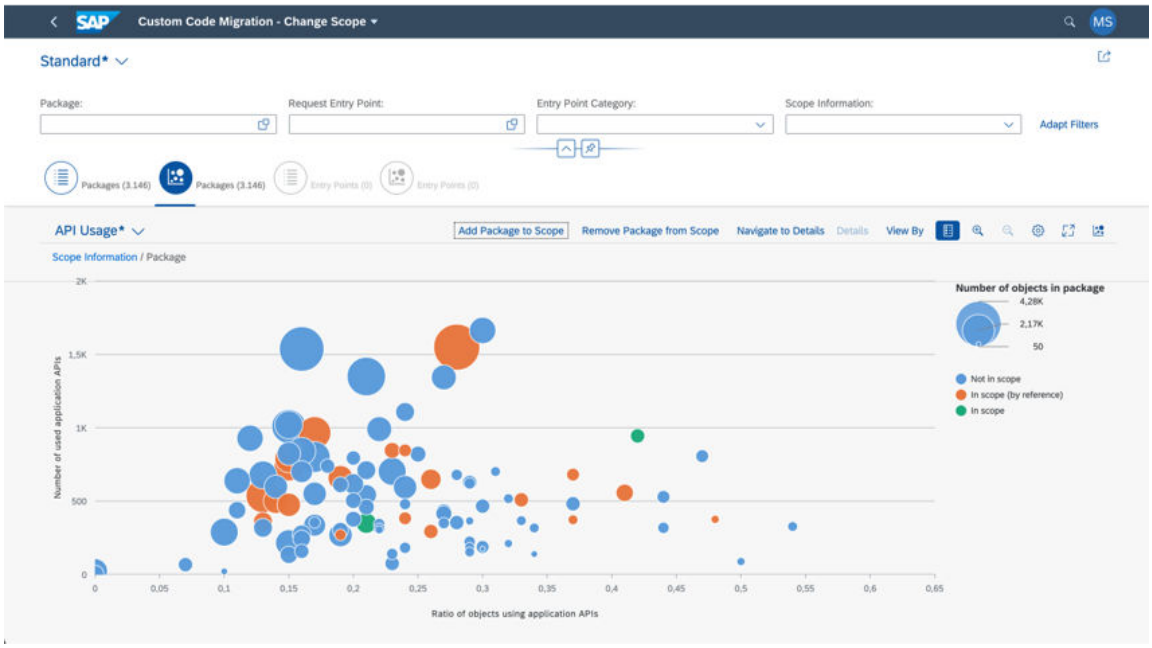
The measures and their meanings

Measure	Explanation
Number of Changes	Number of changes per object within the last year
Number of application API usages	All usages of application APIs such as classes, interfaces, and function modules
Number of application DB API usages	All database operations on database tables or views; i.e., only the number of different database tables and views are counted, not the overall number of accesses
Number of distinct app. DB APIs used	Database operations on distinct database tables or views

Measure	Explanation
Number of distinct application APIs used*	Distinct usages of application APIs; i.e., only the number of different APIs are counted, not the overall number of accesses
Number of objects in package	Overall number of custom code objects within a package or request entry point
Number of objects using appl. DB APIs	Number of objects within a package or request entry point accessing application-specific database tables or views
Number of objects using application APIs	Number of objects within a package or request entry point using application APIs
Proportion of objects using app. APIs	Ratio between objects using application APIs and all objects within the package/request entry point
Proportion of objects using app. DB APIs	Ratio between objects using application-specific database tables or views and all objects within the package/request entry point

*With this measure, for example, you can check how many dependencies to different APIs your code has. In the best case, both this number and the number of locations where you are accessing those APIs are small.

After adjusting your settings, your chart could look as follows:



You can then also save your individual settings as your own view. This way, you won't have to adjust your settings again the next time you want to use the chart.

Related Information

Custom Code Analysis [page 2941]
Complexity Analysis [page 2947]

6.3.1.3.4.2.3 Analyzing the Findings

When you create a custom code analysis project, the custom code analysis is performed automatically. The Analyze Custom Code app then provides you with an analytical representation of the findings.

Procedure

1. Choose *Analysis* to get an overview of the results, as soon as the first analysis has been finished.
2. Choose *Analyze Findings* to get a detailed analysis.
3. In the *Analyze Custom Code - Analysis - Findings* view, you can specify various filters, such as:
   - *Quick Fix Availability*: Analyze which findings can be solved by a Quick Fix and which findings have to be solved manually.
     Scope Information: Filter the results to only show findings for objects that are in scope.
4. Choose ⚙ in the *Findings* section to change the settings for the chart.

   For example, to show the Quick Fix availability per Check Message, you need to specify the following settings for the chart in the *Chart* tab under *Dimensions*:
   - Check Title (as Category)
   - Check Message (as Category)
   - Quick Fix Availability (as Series)

Results

You now have a graphical and tabular representation of findings and can take action to resolve these findings in your development environment.

6.3.1.3.4.2.4 Creating a Custom Code Analysis Project

With a custom code analysis project, you can analyze custom code with arbitrary ATC check variants.

Procedure

1. Start the tile *Analyze Custom Code* from the SAP Fiori launchpad.
2. Choose *Create*, then choose *Custom Code Analysis Project*.
3. Fill in the fields under *Project Data* for your new project.
   a. *Project Description*: Provide further details about your custom code migration project.
   b. *Check Variant*: Use the value help to select a check variant available in your system.
   c. *Location of Custom Code*: Use the drop down menu to specify the system in which the custom code to be analyzed is located.
   d. *Object Provider to Remote System*: An object provider defines the RFC connection to be used for analysis in a remote SAP system and must be assigned to a system group.

> ⓘ Note
>
> If you collected usage data, choose the *Add* button to add usage data to your custom code migration project.
>
> Under *Advanced Configuration*, you can limit your custom code analysis by specifying the packages you want to analyze. This can be helpful to save time and resources.
>
> These steps are optional.

4. In the *Advanced Configuration* tab, under *Analysis Configuration*, you can either leave the check box checked and start the custom code analysis automatically at project creation or un-check it and start the analysis manually at a later point.
5. To confirm your input, choose *Create*.

6.3.1.3.4.2.5 Process Findings After Analysis

Find out how you can process your findings after custom code analysis.

Purpose

In case you've been wondering what the next steps would be after custom code analysis and what to do with your findings in the Analyze Custom Code app, this topic is for you. Here, you will learn about your options for after custom code analysis.

1. **ABAP Test Cockpit (ATC) Baseline**

With the baseline functionality, it's now possible to add/remove custom code project check results to/from the baseline and choose whether you would like to:

1. Exempt your findings: ATC findings from the baseline will be marked as exempted in the check results list.
2. Suppress your findings: After a new ATC check run, all ATC findings from the baseline are excluded from the ATC run results list.
3. Reduce findings' priority: The priority will be reduced to low (priority 3) for all ATC findings of the relevant ATC check result.
   Result: After adding findings to the baseline, they won't appear in the subsequent check results of the custom code project.

> ⓘ Note
>
> Please be aware that:
>
> - Once you have added findings to the baseline, the custom code migration project cannot be deleted. Remove your findings from the baseline first and then proceed with deleting your project.
> - The analysis cannot be started again after the findings were added to the baseline.

2. **SAP Readiness Check**
   You can download the check results to upload them later onto SAP Readiness Check 2.0. To do so, in the *Analyze Findings* section of your project, choose the arrow next to the spreadsheet icon and select *Export for SAP Readiness Check*. To learn more about SAP Readiness Check for SAP S/4HANA, see SAP Note 2913617.

6.3.1.3.4.3 Schedule Custom Code Analysis

Learn more about the *Schedule Custom Code Analysis* app.

Purpose

The *Schedule Custom Code Analysis* app enables you to schedule projects created in the *Custom Code Migration* app as application jobs. With this functionality, the analyses can be performed once or periodically. The *Schedule Custom Code Analysis* app is available to all users that have access to the *Custom Code Migration* app and have been assigned the `Project Manager – IT` business role.

Key Features

With this app you can schedule ABAP Test Cockpit (ATC) mass runs for your custom code migration project in a SAP Business AI Platform, ABAP environment system through a predefined custom code analysis application job template.

Access Information

For more information how to provide access to users and how to implement this app, see Enable Usage of the Analyze Custom Code App [page 2943].

Component for Customer Incidents

BC-DWB-TOO-ATF

Supported Device Types

- Desktop

6.3.1.4 Additional Software

Install Additional Software [page 2954]

6.3.1.4.1 Install Additional Software

With this app you can only display the list of the apps that are available for download. This helps you to better integrate your apps with other programs you need for your daily business.

Key Features

You can use this app to:

- Download the current version of the SAP Cloud Print Manager

  > → Tip
  >
  > For more information, see the *SAP Cloud Print Manager Quick Guide* that you can call up directly in the application by clicking *Help*. This document is only available if you have installed SAP Cloud Print Manager.

- Download the current version of ABAP Development Tools

Supported Device Types

- Desktop

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-CCM-PRN-OM`.

6.3.1.5 Application Jobs

Find out more about application jobs in SAP and how they can be helpful.

Many business activities have to be run periodically. To avoid starting these activities manually each time, you can create application job templates and schedule them as application jobs in the *Application Jobs* app.

An application job template can be scheduled as an application job. In the scheduling dialog, you as a business user can determine a periodic start condition (such as every first Monday of the month at 1:00), and you can set appropriate parameter values for the application job execution. The application job will then be run automatically without further manual interaction, whenever the start condition is fulfilled.

SAP also offers the *Application Job Templates* app. In this app, you can create new templates by saving an existing template with new parameter values. You can also connect existing templates to multistep templates. Running a multistep template means running all steps sequentially.

Application Jobs [page 2955]

Application Job Templates [page 2960]

Maintain Job Users [page 2964]

6.3.1.5.1 Application Jobs

App ID: F1240

You can use this app to monitor all application jobs from all specialized application jobs apps (example: Schedule Overhead Accounting Jobs,...) that business users have scheduled. You can also cancel application jobs.

Key Features

> **Note**
>
> You can use the scheduling, copying and restarting features in the *Application Jobs* app only if you have been granted the necessary authorization via a business role. For more information on how to get the necessary authorizations for this app, see Maintaining Authorizations [page 2959].

You can use this app to:

- Schedule jobs based on a job template: Select a job template, define the description, start date, start time and recurrence of the job, and enter job-specific selection criteria and parameters
- Work with personalized job templates: Create and personalize a job template, save personalized job template for later use, and share personalized template

  > **Note**
  >
  > The parameters under *Scheduling Information* can't be saved as a part of a template.

- Schedule jobs with a custom factory calendar: In the *Application Jobs* app, define the scheduling information, change an existing calendar, create a new one, and define execution days
- Arrange job series that take different time zones into account
- Monitor jobs
- Display logs and results: You can navigate to application job logs and application job results by clicking on the icon in the *Log* or the *Results* column. The icon can be either an error icon, a success icon, or an info icon. Selecting the icon, you'll navigate to the specific logs or the specific results of the selected job.
- Copy a job
- Cancel a job
- Restart Job Chains

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-APJ`.

6.3.1.5.1.1 Creating an Application Job

Find out how to create your own application jobs in the *Application Jobs* app.

Context

You can use the *Application Jobs* app to schedule and monitor application-related jobs. If you have manual activities that you often need to do at a specific time, the app can reduce your workload by running these tasks smoothly in the background. You can plan regular jobs which allows you to concentrate on other tasks.

Procedure

1. Open the *Application Jobs* app.
2. Choose *Create* to create a new application job.
3. Select a job template and enter a job name.
4. Specify your preferred scheduling options. You can define the recurrence of your job by selecting *Define Recurrence Pattern*.
5. In the next step, you can adjust the parameters of your application job.
6. You can choose to let the system check if your entries are consistent. If possible, the system fills in default values for the parameters. To choose this option, click *Check*.
7. You can choose to save the application job as a template, or manage your templates. To do so, select *Template*.
8. Finally, to schedule your application job, select *Schedule*.

6.3.1.5.1.2 Job Cancellation

Job cancellation details

A job can be canceled either by the system or by a user. If a job is canceled by the system, it gets the status *Failed*. If you want to cancel a job, please keep in mind that the system behavior depends on what exactly you want to cancel:

- If you cancel a single job with the status *Scheduled*, this job will be canceled and removed from the job list.
- If you cancel a single job with the status *In Process*, this job will be canceled, will get the status *Canceled* and will remain in the job list.
- If you cancel a job that has the status *Scheduled* and is a part of a job series, all jobs with the status *Scheduled* will be canceled and removed from the job list. All jobs with other statuses will remain in the list and won't be canceled.
- If you cancel a job that has the status *In Process* and is a part of a job series, a job with the status *In Process* will be canceled and will get the status *Canceled*. All other jobs of the series with other statuses will remain in the list.

To cancel a job, please select the job to be removed and choose *Cancel* from the main toolbar.

> **Note**
>
> Please note that the jobs with the status *Finished* cannot be canceled because they have already been run.

> **Note**
>
> You can choose to skip a single execution of a periodic job chain. To do so, select *Skip*. A skipped job can be restored by clicking *Restore*.

Related Information

Application Jobs [page 2955]

6.3.1.5.1.3 Regular Job Deletion

Get an overview of the regular job deletion

All jobs that are not removed by a user remain in the system until a technical cleaning job runs. The time when this job is run by the system depends on the periodic granularity of the jobs that should be deleted. Currently, the following timeframes apply:

Regular Job Deletion

Periodic Granularity of the Job to Be Deleted	Job is Automatically Deleted After
Not periodic	14 days
Minutes	7 days
Hours	14 days
Days	Periodic value*24 days
Weeks	Periodic value*7*24 days
Months	Periodic value*31*24 days
Every X week in every Y month	Periodic value*31*24 days

Example

You have scheduled a job that runs every two weeks and ends after 20 runs. The first run starts in the calendar week 2 and ends in the calendar week 40. Each job run stays in the system for the certain retention time.

Retention period is always set to 24 runs. Retention time=periodic value * retention periods * weekdays [days] = 2 * 24 *7 [days] = 336 [days] = 48 [weeks]. The first job run will be deleted a day after 48 weeks since this job run took place.

> **Note**
>
> This is the default behaviour. If the application chooses to have different schemes, this is documented in the application-specific documentation.

6.3.1.5.1.4 Restart Job Chains

Find out how to restart job chains.

When an application job or a chain of application jobs consisting of one or more steps has the status *Failed*, *Canceled* or *User Error*, you can restart it. To do so, select the job and choose one of the following options from the drop-down menu:

- From Error Step: All steps before the error step will be skipped and all other steps (including the error step) will be executed.
- After Error Step. All steps before the error step including the error step will be skipped and all other steps after the error step will be executed. This option is not available for single step jobs.

The new job will be executed immediately with the original job user.

> **Note**
>
> You can't change the starting or application parameters.

Related Information

Application Jobs [page 2955]

6.3.1.5.1.5 Maintaining Authorizations

Find out which authorizations you need to maintain in order to use the *Application Jobs* app.

Context

To change access to the objects of other users used in the *Application Jobs* app, create a new business role assigning the business catalog `SAP_CORE_BC_APJ_JCE` to it. You can modify the provided restriction types according to your needs.

Procedure

1. Create a new business role assigning the business catalog `SAP_CORE_BC_APJ_JCE` to it. The restriction type `Job Catalog Entry/Application Job Part` contains the two fields `Application Job Catalog Entry` and `Application Job Part`. For more information, see Maintain Business Roles [page 3068].

   > **Note**
   >
   > With this restriction type, you'll be able to provide the necessary authorizations for certain actions on the application jobs of other users that are based on a specified job catalog entry. You can display the result lists of these application jobs.
   >
   > Alternatively, you can also create a display only-role using this restriction type.

2. Next, define your business role with the restriction fields `Application Job Catalog Entry` and `Application Job Part` mentioned above. To do so, select *Maintain Restrictions*. In the `Application Job Catalog Entry` field, you can choose the job catalog entries, and in the `Application Job Part` field, you can choose for which job part access should be granted. You can choose the values `JOB` (Job Details), `SPOOL` (Result List), and `APPLOG` (Log) by clicking the *Edit* icon next to the fields.
3. If you want to have different business users for the job owner and the job user, the job owner needs the authorization to schedule the application job for the job user. In this case, the job owner needs the restriction type `Create Application Jobs for Other Users`.
4. Go back and select *Save*. Now, you've maintained your business role, and you can assign the role to a business user. For more information on authorizations, see 3028505.

   > **Note**
   >
   > The private templates of other users aren't shown in the *Application Jobs* app, even if the authorization to view these private templates was granted via `SAP_CORE_BC_APJ_TPL`. You can view the private templates of other users only in the *Application Job Templates* app. For more information, see Application Job Templates [page 2960].

Related Information

Maintain Business Users [page 3044]
Maintain Business Roles [page 3068]

6.3.1.5.2 Application Job Templates

App ID: F2058

You can use this app to create your own application job templates. The application job templates consist of a set of parameters that you can set for the assigned job catalog entry. This app is mandatory to create multi-step templates which can then be submitted via the *Application Jobs* app. A multi-step template consists

of several other templates that run in a series, one after the other. Each step could be a single-step template, or another multi-step template that was created with the *Application Job Templates* app before.

Key Features

You can use this app to:

- Create and customize a job template
- Create multi-step templates
- Change the template parameters
- Transport job templates, if supported: For more information, see 2999966
- Display a list of templates you have read access to
- Display the layer code of the template:
  - *Global:* a template with this layer code is predelivered by SAP and therefore can't be edited or deleted
  - *Private:* a template with this layer code is visible for all users within the *Application Job Templates* and the *Application Jobs* apps, but only the author can edit or delete the template

    > **Note**
    >
    > With the new restriction type, you can define a full authorization for job templates, such that a system administrator can display, change or delete private templates of other users

  - *Shared:* a template with this layer code is visible to all users that have the required authorizations to use the *Application Job Templates* and the *Application Jobs* apps
  - *Reference:* a template with this layer code is visible for all users within the *Application Job Templates* and the *Application Jobs* apps, but only the author can edit or delete the template. It can't be scheduled in the *Application Jobs* app. It can be used as a copy pattern to create other templates.
- Delete a job template

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-APJ`.

Related Information

Creating an Application Job Template [page 2962]

6.3.1.5.2.1 Creating an Application Job Template

Find out how to create your own application job templates in the *Application Job Templates* app.

Context

You can use the *Application Job Templates* app to create a job template for a specific business purpose so that you can schedule concrete jobs by reusing the job template.

Procedure

1. Open the *Application Job Templates* app.
2. Choose *Create* to create a new job template.
3. Enter a name and specify your preferred scheduling options.
4. Tick the *Shared Template* checkbox if you want other users to be able to view or edit the job template. If you share the template, all users who have access to the *Application Job Templates* app, and who have the appropriate authorizations, can make the same changes as you.
5. Select *Maintain Steps* and choose *Add* to add an existing template as step. This step can be repeated many times to create a multi-step template.
6. Click *Save Job Template*. Now, you can view each parameter of your selected templates in the *Parameter Section* and edit the values according to your needs.
7. If you are done editing, you can now choose whether to check your parameters, or display, save, or cancel your job template. When clicking *Save* to save your job template, you can choose to save and reload the page to be able to view the values you've entered previously by selecting *Save and Reload*. Mind that this may trigger a longer reload of new values. If you want to get back to the main page and continue working, choose *Save and Back*.

> **Note**
>
> If you want to schedule multistep templates with the *Application Jobs* app, you'll need the appropriate authorizations for all steps.

6.3.1.5.2.2 Maintaining Authorizations

Find out which authorizations you need to maintain in order to use the *Application Job Templates* app.

Context

In order to grant access and authorizations to the *Application Job Templates* app, create a new business role assigning the business catalog `SAP_CORE_BC_APJ_TPL` to it.

Procedure

1. Create a new business role assigning the business catalog `SAP_CORE_BC_APJ_TPL` to it. For more information, see *Maintain Business Roles* linked below.
2. **(Optional)** If you want to grant read or write access to templates of other users, select *Maintain Restrictions* to provide *Write*, *Read*, or *No Access* using the restriction type `APPLICATION JOB TEMPLATE LAYER`.
3. Go back to *Save* your template.

> ### Note
>
> You will always be able to see your own templates and the templates delivered by your software provider without having to maintain further authorizations.

> ### Note
>
> The possible values for the layer field are *Global*, *Shared*, and *Private*. These are described in *Application Job Templates* linked below. The restriction type *Global* is predelivered by SAP and therefore can't be edited or deleted, but you can modify the restriction types *Shared* and *Private* to your needs.

> ### Note
>
> If you want to schedule multistep templates with the *Application Jobs* app, you'll need the appropriate authorizations for all steps.

Related Information

Maintain Business Users [page 3044]
Maintain Business Roles [page 3068]
Application Job Templates [page 2960]

6.3.1.5.3 Maintain Job Users

App ID: F6156

With this app, you can change the owner and the user of application jobs that you've created from a job template. This can be helpful if, for example, a person changes the department within a company or leaves the company, and the corresponding business user has to be deleted.

Key Features

You can use this app to:

- Change either the job owner or the job user of an application job, or change both
- Delete an application job

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-APJ`.

6.3.1.5.3.1 Editing Job Users

Find out how to change the user of your application job, or delete your job.

Prerequisite

Before changing the user of a job, ensure that the target user has the correct authorizations to run the job. Otherwise, the job will fail.

To be able to see the app, make sure that the target user is assigned to a business role that has the business catalog `SAP_CORE_BC_APJ_USR_PC` included.

Procedure

1. Open the *Maintain Job Users* app.
2. Select the scheduled jobs that you want to work with, and choose one of the following actions:
   - *Change Job Owner* to change the owner of your application job
   - *Change Job User* to change the user of your application job
   - *Change Job Owner and User* to change both the owner and the user of your application job
   - *Delete* to delete your application job

   When clicking on an application job, you'll be navigated to the *Details* page of the *Application Jobs* app.
3. If you've decided to change the owner or the user of your application job, or if you want to change both, a dialog opens. Enter the new job owner and user by selecting them from the value help. Mind that in case of a job user change, you can only select users that have the appropriate start authorization for a job. Job owners don't need this authorization, but they need the restriction type `Create Application Jobs for Other Users` instead. For more information, see **Maintaining Authorizations** of the *Application Jobs* documentation. App-specific authorizations are not checked. You can choose to only display these users by filtering for *Only Valid*.

   > ⓘ Note
   >
   > If a job owner is a communication user, the owner can't be changed, since the connection to the corresponding communication arrangement of the type `SAP_COM_0326` would be broken.
4. You can change multiple application job owners and users at the same time. To do this, select all users you want to change and choose *Change Job Owner and User*. If the job owner differs from the job user, the job owner must have the appropriate authorization given by the restriction type `Create Application Jobs for Other Users` of business catalog `SAP_CORE_BC_APJ_JCE`.
5. You can choose to display the change history of your application job by selecting *Change History*. All changes that were made to the owner and user in the past are shown in the change history. The change history shows the user type, and the new and old user IDs. You can view who changed the user, and when it was changed. The type of change is shown, too.
6. You can delete multiple application jobs at the same time. To do this, select all jobs you want to delete and choose *Delete*.

You've now maintained your job user.

Related Information

Application Jobs [page 2955]
Maintaining Authorizations [page 2959]

6.3.1.6 Business Configuration

Find out which lifecycle management tasks a business process configuration expert (business role SAP_BPC_EXPERT) needs to perform for the administration of business configurations.

Context

Business configuration is the customization of business applications. In enterprise software, business configuration refers to a predefined set of configuration options that affect its functionality and behavior. You can maintain custom business configuration content in your `SAP BTP ABAP Environment` to enrich your extensibility scenarios. This includes both functionalities provided by SAP and custom business configuration objects developed using `ABAP Development Tools` (ADT).

If you're looking for more information on how to implement custom business configuration objects, please refer to Custom Business Configurations App [page 2970].

Requirements

The administration of your business configuration has the following requirements:

- The business configuration is client-dependent. After import, it's only available in the client where the import took place. In contrast, development objects are not client-dependent.
- Business configuration content is usually not directly maintained in a productive system, but created in a development system, and then transported to the test system and productive system. For more information on how to set up your system landscape, please refer to the scenarios described in Setting Up and Working with Your Landscape.
- Changes to business configuration content are recorded onto transport requests of the type `Customizing`. They can be released in the source system and imported in any target system. For more information on how to set up a suitable software component for transport, please refer to Software Components.

Procedure

You as a business process configuration expert are responsible for the lifecycle management of business configuration content. First, you need to create transport requests of the type `Customizing` that contain corresponding transport tasks. Once a suitable transport task has been assigned, you can proceed with the creation and adjustment of content. When the content is ready, the transport tasks and the transport requests can be released. The changes will be written to the remote repository corresponding to the software component that was used for transport. The content can then be imported into target systems simply by pulling the changes for the software component that was used. This procedure is valid regardless of the specific landscape setup or whether software components of the type `Development` or `Business Configuration` are used.

> **Note**
>
> It's possible to maintain local business configuration changes which can't be transported to other systems. If no software component of the type `Business Configuration` is cloned in the source system, you can still create a customizing transport request in the *Export Customizing Transports* app. This request will have no target. It can be used to record business configuration changes, but its release will have no effect.

The following sections will introduce the applications needed for both the maintenance and lifecycle management of business configuration content.

6.3.1.6.1 Business Configuration Change Logs Overview

You can display change logs for objects you're authorized to access. Use change logs to check when data was changed, what type of data was changed, or who changed it.

Context

To get an overview of changes made to all objects you're authorized for, use this app to download the change logs. You can navigate to this app from the Business Configuration Change Logs [page 2968] app by selecting the menu option *Download Change Logs*.

This app complements the *Business Configuration Change Logs* app. Together, these apps provide an overview of all configuration changes made in your system, including changes made using the *Custom Business Configurations* app.

Access Information

To access the app, you need to have the following business catalog assigned to your user: **SAP_CORE_BC_BCT_LOG_DIS_PC** - Business Configuration - Change Logs Overview. This business catalog is available in the business role template: **SAP_BR_BPC_EXPERT**.

Key Features

- You can use this app to display which changes were made, who made the changes and when the changes were made.
- You can restrict the changes displayed by time interval.
- You can download and save the output as a PDF document.

> **Note**
>
> The downloaded output displays the domain description in addtion to the domain values. Under the column *Changed By* the full user name is displayed.
>
> The downloaded output can contain a large number of change logs depending on the time interval. This can potentially lead to memory issues. The app issues a warning before scheduling when the number of change logs exceeds 10,000. In these cases, restrict the selection to shorter intervals.

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-CUS-TOL-ALO`.

6.3.1.6.2 Business Configuration Change Logs

With this app, business process configuration experts can keep track of content changes in your business configuration tables. Table data change logs can be used to check when data was changed, which kind of data was changed, or by whom. This is made visible in a table.

Access Information

To access the app, you need to have the following business catalog assigned to your user: `SAP_CORE_BC_BCT_LOG_PC` - Business Configuration - Change Logs.

This business catalog is available in the business role template: `SAP_BR_BPC_EXPERT`.

To display the change logs for tables, users must have the authorization for the object `S_TABU_NAM` with `Display Change Documents` for `ACTVT` and the name of the table for `TABLE`.

Key Features

You can use this app to:

- Display which changes were made to the records of your business configuration tables.
- Display when changes were made to the records of your business configuration tables.
- Display who made changes to the records of your business configurations tables.

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-CUS-TOL-ALO`.

6.3.1.6.2.1 Working in the Business Configuration Change Logs App

Prerequisites

The *Log Changes* checkbox has to be enabled in the technical settings for the table:

- For tables delivered by SAP: SAP defines whether change logging is enabled.
- For tables you have created: You need to enable change logging.

> ⓘ Note
>
> *Log Changes* checkbox can be enabled or disabled in the Technical Table Settings ADT editor by expanding the entry of your table in the project explorer.

Displaying a Change Log

Find out how to display the change log of your business configuration tables.

Access Information

To access the option, you need to have the following business catalog assigned to your user: `SAP_CORE_BC_BCT_LOG_DIS_PC`

This business catalog is available in the business role template `SAP_BR_BPC_EXPERT`.

Procedure

1. Open the *Business Configuration Change Logs* app.
2. Use the search bar to find a configuration table. You can search based on the table name or a specified date range.
3. Select a configuration table to see its change log details.
4. You can filter the change log details based on criteria such as field name, changed date, changed by, and more.

Download Change Logs

Find out how to display the change log of your business configuration tables.

Access Information

To access this option, you need to have the business catalog `SAP_CORE_BC_BCT_LOG_DIS_PC` assigned to your user

This business catalog is available in the business role template: `SAP_BR_BPC_EXPERT`.

Procedure

1. Open the *Business Configuration Change Logs* app.
2. Select the button *Download Change Logs*. Once selected you will be navigated to the *Business Configuration Change Logs Overview* app.
3. Select *Create* to schedule a job.
4. Select a *Job Template* and a *Job Name*, then select *Step 2*.
5. You can either start the job immediately by selecting it or select a specific start date and time. Select *Step 3* to continue
6. In the *Parameters* section select an *Evaluation Begin Date*, *Evaluation Begin Time*, *Evaluation End Date* and an *Evaluation End Time*. All the change logs given in this time window will be downloaded.
7. To download the change logs for a specific user, provide their *Username*. When no username is provided, all change logs from every user will be downloaded
8. The *Exclude Text Tables* option allows to filter out the change logs for the text tables.
9. Once all the Parameters are entered, select *Schedule* to start the job immediately or at the provided time given at *Step 5*.

Result

In the Business *Configuration Change Logs Overview* app, you can see the scheduled jobs. Once the job is finished you can preview the change logs or download the change logs.

If you need support or experience issues, please report an incident under component `BC-CUS-TOL-ALO`.

6.3.1.6.3 Custom Business Configurations App

Find out how to use the *Custom Business Configurations* app.

Purpose

The *Custom Business Configurations* app serves as an entry point to the Business Configuration Maintenance Object provided by the custom applications or partners. You can use the app to adjust these configuration objects to change and influence the system behavior. For more information, see also Creating Business Configuration Apps with ABAP RESTful Application Programming Model and Custom Business Configurations App.

Access Information

The following business catalog needs to be assigned to your user to access the app: `SAP_CORE_BC_BCT_MBC_PC`.

The business catalog is contained in the business role template: `SAP_BR_BPC_EXPERT`.

For a business configuration maintenance object to be listed in the *Custom Business Configurations* app, you require the necessary authorizations regarding the service of the business configuration: Business Configuration Maintenance Object > **Provide Authorizations for a Business Configuration**.

Procedure

1. Open the *Custom Business Configurations* app.
2. You'll see a list of all business configurations that can be adapted.
3. Use the *Search* bar to find a certain business configuration or select the one you would like to work on from the list.

   > ### Note
   >
   > Depending on your respective role and access rights, you may only be able to edit certain business configurations.

4. You can now adjust the business configuration you have selected. Simply add, update, or delete entries.

Intent Navigation

You can use the parameter `TechnicalIdentifier` for the semantic object `BusinessConfiguration` with the action `maintain`. Intent navigation can be used to directly navigate to the maintenance view of the business configuration specified with the parameter `TechnicalIdentifier`. The attribute `TechnicalIdentifier` is available as a column in the list report of the *Custom Business Configurations* app.

When all technical key fields are provided as parameters to uniquely identify a record on an object or subobject page, the page opens directly.

Example

Name of the Business Configuration Maintenance Object: `ZPLANT`

Technical Key of the CDS entity Plant: `PlantID`

Navigation to the maintenance view of ZPLANT: `#BusinessConfiguration-maintain?TechnicalIdentifier=ZPLANT`

Navigation to the active page of Plant 201: `#BusinessConfiguration-maintain?TechnicalIdentifier=ZPLANT&PlantID=201`

Display Change Logs

For a selected business configuration, the action *Display Change Logs* is available in the header section if:

- at least one database table of the *RAP BO* of the *Business Configuration Maintenance Object Table Logging* is active.
- the *Business Configuration Change Logs* app is configured as a valid navigation target for the user.

With this action, you can navigate to the Business Configuration Change Logs app.

Show Documentation

For a selected business configuration, the action *Show Documentation* is available in the header section if a knowledge transfer document exists for the business configuration maintenance object. With this action the content of the knowledge transfer document is displayed. For more information, see Documentation of Business Configuration Maintenance Objects.

To provide context-sensitive in-app help you can use SAP Companion.

Grouping

In the table view settings dialog you can group the business configuration maintenance objects by their attribute `Configuration Group`. This allows the list to be structured in a simple hierarchical manner. If this attribute is not maintained for a business configuration maintenance object, it will be part of a generic group with the header *Not assigned*.

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-CUS-TOL-MBC`. If you experience issues updating a specific business configuration, please use the component that is displayed in the error message.

6.3.1.6.4 Manage Number Range Intervals

You can use this app to get an overview of number range objects and maintain number range intervals.

Key Features

You can use this app to:

- Display properties of number range objects
- Display, create, change and delete number range intervals
- Change the number range status of a number range interval
- Display the change history

Supported Device Type

- Desktop

Prerequisites

To access the *Manage Number Range Intervals* app, assign the `SAP_CA_BC_IC_LND_NUM_PC` (Number Range Management – Configuration) business catalog to a proper business role and - if required - restrict it to specific number range objects.

To maintain number range intervals and to create transport requests, assign business role `BR_BPC_EXPERT`.

For more information, see:

- Maintaining Number Range Intervals [page 2974]
- Changing the Number Range Status [page 2975]
- Checking Properties of a Number Range Object [page 2973]

6.3.1.6.4.1 Checking Properties of a Number Range Object

To check the properties of a number range object, proceed as follows:

1. Start the *Manage Number Range Intervals* app from the *Business Configuration* app group on thr fiori launchpad.
2. Choose a number range object from the list and click the arrow-button on the right-hand side of the entry. The *Properties* section is displayed.

   > **Note**
   >
   > You can only display properties. You don't have the option of changing them.

   The following parameters are displayed:
   - Subobject exists: The number range object distinguishes between further subobjects.
   - Year-Dependent: This parameter is set to *Yes* if the number range intervals are distinguished according to to-fiscals.

- Subobject as Prefix: This parameter is set to *Yes* if determined numbers consist of the prefix (name of subobject) and the numbers.
- Number of Digits: This parameter determines the length of an interval.
- Rolling: This parameter is set to *No* to prevent the number range object intervals from starting from the upper limit automatically.
- Buffering Type: This parameter shows whether main memory, parallel, or no buffering is supported.
- Number: In case of parallel and main memory, this parameter determines how many numbers are reserved in the buffer for the intervals.
- Warning at Remaining: This parameter displays the percentage of numbers remaining in a number range. When reaching the percentage in number assignment, a warning is given. Assuming an interval from 1 to 1000 and a percentage of 10 (%) is used, a notification will be triggered if number 900 has been reached.

You can maintain number range objects using the ABAP Development Tool (ADT) or APIs. For more information, see

- ADT:
  Working with Number Range Objects
- API:
  Maintaining Number Range Objects

6.3.1.6.4.2 Maintaining Number Range Intervals

You can structure the defined character set of a number range object in intervals. For more information on intervals, see Intervals.

To maintain a number range interval, proceed as follows:

1. Start the *Manage Number Range Intervals* app from the *Business Configuration* app group on the fiori launchpad.
2. Choose a number range object from the list and click the arrow-button on the right-hand side of the entry. The *Properties* section is displayed.
3. In the *Number Ranges* section, press *Create* to create a new interval. Assign a two-character ID as interval number and a lower and upper limit. If the current number range object supports subtypes or is year-dependent, assign a subtype and a year accordingly.

> ### Note
>
> Don't set the *External Interval* flag if you want to use an internal number assignment. That means that during number determination within an application program using that interval, the number range framework automatically assigns a number from the associated number range object to the data record created by the user. Only digits are assigned from internal intervals of the number range object.
>
> For an external number assignment, an application selects a number for a new data record by their own. This can be useful if you want more information to be included in the structure of the number. The number range framework then checks whether the selected number is in an interval of the number range object that is defined as external. However, it does not check whether this number has already been used. Digits and also letters and special characters are assigned from external intervals of a number range object.

4. Press *Save* to save the new interval.

If you want to delete an interval mark it by using the radio button and press *Delete*.

If you want to change an interval, open it by clicking the arrow-button on the right-hand side of the entry. Then press *Edit* if you want to change the limits or the external flag.

6.3.1.6.4.3 Changing the Number Range Status

The *Number Range Status* or *Number Range Level* is the last number used by an interval.

> ⓘ Note
>
> Only change this number in case of inconsistencies.

To change the number range status, proceed as follows:

1. Start the *Manage Number Range Intervals* app from the *Business Configuration* app group on thr fiori launchpad.
2. Choose a number range object from the list and click the arrow-button on the right-hand side of the entry. The *Properties* section is displayed.
3. Choose a number range interval from the list and click the arrow-button on the right-hand side of the entry.
4. Click the *Change Number Range Status* button. In the pop-up, enter a new interval level an press *Change Number Range Status*.

6.3.1.6.5 Upload Business Configuration

Context

You want to maintain Business Configuration data in the cloud system. However, you do not want to manually maintain it, for instance, due to data volume. You may have already maintained your own configuration tables in the backend of your ERP system and now simply want to import the configuration content from a file into the corresponding tables in your system.

Purpose

The **Upload Business Configuration** app offers an easy way to import the configuration content via ".xslx" file upload. Such a file may have been generated from a leading ERP system.

Access Information

The following business catalogue needs to be assigned to your user to access and use the app:

- SAP_CA_BC_IC_LND_PC - Business Configuration – Customizing

In addition, you need an open customizing transport, which can be created by a user who has the following business catalogues assigned to them:

- SAP_CORE_BC_BCT_TRN_MNG_PC (Business Configuration - Transport Management) – Assigned to new role SAP_BCR_CORE_BCT_TRN_MNG_PC
- 2. SAP_CORE_BC_BCT_TRN_REL_PC (Business Configuration - Transport Release Management) – Assigned to new role SAP_BCR_CORE_BCT_TRN_REL_PC

All of these business catalogs are contained in the business role template SAP_BR_BPC_EXPERT.

Prerequisites

Use case 1: Export content from an SAP system on-premise, e.g. SAP S/4HANA, and import it using this app.

- The file to be uploaded via the *Upload Business Configuration* app is available in an external format. Here's how you can generate such a file from SAP systems based on the ABAP platform:
  - The configuration content of the maintenance object is exported via transaction *SM30* > *Table View* > *Export (Spreadsheet)* in ".xslx" file format. If the *Export (Spreadsheet)* menu is not available, you can export the configuration content using *Print*. (*SM30* > *Table View* > *Print* > *Local File* > *Save List in File*.
  - Text tables can be exported via transaction *SE16* > *Spreadsheet*. Keep in mind that the *client (MANDT)* field needs to be removed from the exported spreadsheet.

Use case 2: Import content for your custom C tables using this app.

- Create a spreadsheet with table fields as column headers, excluding the field of type "client".
- To upload the content to objects which are in a customer namespace, you need to have the authority for authorization object `S_TABU_NAM. TABLE = <Table Name> ACTVT = '02'`. Please contact your administrator or IAM expert.

> **Note**
>
> The *Upload Business Configuration* app only accepts files with the extension ".xlsx". The uploaded file can contain a maximum of 5000 rows. If there are more rows in the exported file, it needs to be split as per the 5000 row limitation. Apart from explicitly enabled SAP namespace objects, the app only allows uploads for customer namespace tables that have delivery class "C", i.e. "customizing".

> **Note**
>
> Business Configurations for tables with field types `STRING` or `XSTRING` are not supported and therefore can't be uploaded.

Component for Customer Incidents

If you need support or experience issues, please report an incident under component BC-CUS-TOL-BCD.

6.3.1.6.5.1 How to Upload Business Configurations

You want to import your configuration content via ".xlsx" file upload.

1. Open the *Upload Business Configuration* app.
2. In the *Selecht Object* step, open the value help to select the *Object Name* from the allowed list of objects.
3. Select one of the following operations from the drop-down menu:
   - *Upsert (Update or/and Insert)*: Updates existing entries and inserts new entries.
   - *Insert (Insert Only)*: Inserts new entries but doesn't overwrite/update existing entries.
   - *Replace (Replace Existing Records)*: Replaces any existing entries with the new list of entries. Keep in mind that existing entries will be deleted. This action cannot be undone.

   Click *Step 2* to continue.
4. Select *Download File Template* to download the file with a templatized example of the object selected. See the *Download File Template* section below for details.
5. Upload the adjusted downloaded file which contains the configuration content for the selected object. Once you uploaded it, select *Step 3*.
6. In the *Select Configuration Data* step, the configuration content from the file you uploaded is displayed. You can unselect the rows you don't want upload and click on *Review* to continue.
7. Review the information. If everything is correct, select *Deploy*.
8. Based on the system and object settings, you may get a pop-up with a defaulted transport request or no pop-up appears. You can change the transport request by selecting another one from the list. For more informations see *Transport Request* section below.
9. Select your transport request and confirm it with the OK button. The changes will be sent for deployment to be captured in the provided transport request.

Transport Request

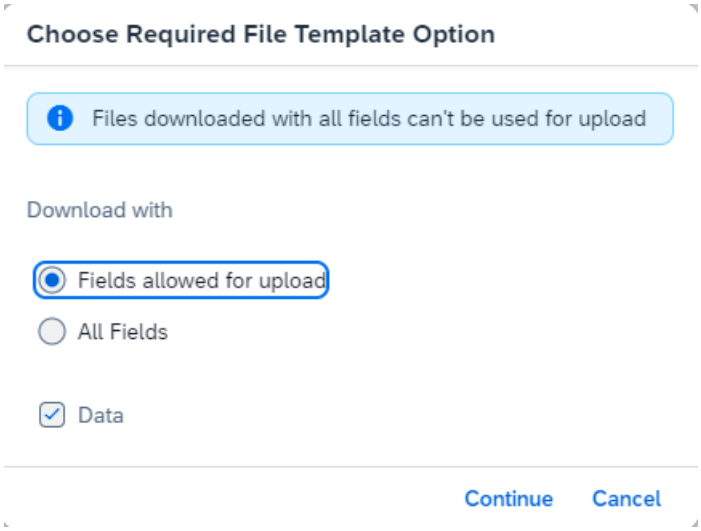
The transfer request pop-up is based on the system and object settings. If a transport request is mandatory for the selected object and the transport is not hidden, then the transport request which is selected by default is either the *Open Default Customizing/Cross-Client Customizing Request* in the *Export Customizing Transports* app or the task which was last used by the current user. If no pop-up appears, the default request was selected and you'll get a confirmation message.

In case there is no available transport request in the transport dialog, or you want to create a new request, select *Create* in the transport dialog. You'll then be redirected to the *Export Customizing Transports* app in a new tab. Once redirected, you can then create a new request.

After creating a new transport request in the *Export Customizing Transports* app, you can find it in the *Upload Business Configuration* app by using the value help.

Download File Template

By selecting *Download File Template* a pop-up will appear.



There are two options to download the file

1. *Fields allowed for upload* downloads the file which can be used for uploading configuration content. You can always adjust the content according to your requirements before uploading it.
2. *All Fields* provides the additional capability to download the file with all the fields of the selected object, including read-only field

If the*Data* option is selected the file will be downloaded with the current configuration. If this option is deselected a blank file will be downloaded.

6.3.1.6.6 Export Customizing Transports

With this app, business process configuration experts can manage business configuration changes recorded in requests.

Business configuration changes are recorded in requests, depending on the category to which the customizing objects belong. Unlike ABAP repository objects, customizing objects recorded in a request are accessible to other business users.

Access Information

The following business catalogs are available to manage the customizing requests:

- Display/Edit/Delete: `SAP_CORE_BC_BCT_TRN_MNG_PC` - Business Configuration - Transport Management
- Display/Edit/Delete/Release: `SAP_CORE_BC_BCT_TRN_REL_PC` - Business Configuration - Transport Release Management

These business catalogs are contained in the business role template: `SAP_BR_BPC_EXPERT`.

Key Features

You can use this app to:

- Display a list of customizing requests
- Create new customizing transport requests
- Display the customizing tasks, objects, attributes, notes and logs of a customizing transport request
- Display the objects recorded in a customizing task
- Display the table keys recorded for an object
- Add notes to a customizing request
- Check customizing transport requests
- Release customizing transport requests
- Assign a transport request to your user
- Create tasks for other users
- Change the transport category
- Take over tasks and transports from other users and assign them to your user
- Check the consistency of all open requests
- Release all open requests

Default Customizing Request

Since configuration changes recorded in requests of the type `Customizing Request` are not lockable, it might happen that several business users record their configuration changes in different requests.

Recording the configuration changes in different requests might create dependencies between these requests, however to avoid such dependencies, a new transport category is introduced.

Any new request created from this app is categorized as *Default*: In a client, there can be only one open default request of the type `Customizing Request`.

Business users can record their configuration changes to this default request to avoid any dependencies.

If you need to perform an emergency fix, record the configuration changes in a request which is not categorized as *Default*. Default requests are listed in the column *Transport Category*.

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-CUS-TOL-CTO`.

6.3.1.6.6.1 Working in the Export Customizing Transports App

Find out how to create, release, or merge customizing requests using the *Export Customizing Transports* app.

- If a software component of the type `Business Configuration` is available in the system, the created request will target this software component. If no such software component is available, the request will be created without any target.

Changing Client-Specific Business Configurations

The client settings determine whether changes to business configurations are allowed or recorded automatically in a request.

If the automatic recording for the client is activated, you have to select a request number for the recording whenever you save changed client-specific configurations. If there is no request number selected, the configurations can't be saved.

If the automatic recording for the client is not allowed, you can save changed client-specific configurations without selecting a request number.

Display Request

1. Navigate to a request's object page to display its tasks, objects, attributes, notes and logs.
2. Select a customizing task to display the objects recorded in it.
3. Select an object to display the table keys recorded in it.

Show Logs

1. To display the logs of a transport request either select a transport request that belongs to you and select *Release > Show Logs* or navigate to the object page of the transport requests and click on the *Logs* section.
2. The logs of a transport request can also be filtered based on the type of the message.
3. In case the logs contain an error message, select the message *Description* to see a detailed information about the error and how to resolve it.

Create Request

1. Open the *Export Customizing Transports* app. A list of customizing requests is displayed, with the latest request shown first at the top.
2. Use the search bar to filter for specific requests based on search criteria like the *Request No*, the *Owner*, or the *Status*.
3. Click *Create* to create a new request.

> **Note**
>
> Note that there can only be one default customizing request.

4. (Optional) Add business configurations to this request by, for instance, using the *Upload Business Configurations* app or the *Manage Number Range Intervals* app.

Check All

1. Open the *Export Customizing Transports* app. A list of customizing requests is displayed, with the latest request shown first at the top.
2. You may put a filter on *Status* for modifiable and checking status. Additionally you can sort the transports in ascending order to see the current status of those transports sequentially.
3. Select *Check All > Start* to trigger consistency check for all open transports.
4. To check remaining open transports for which the consistency checks are yet to be performed, select *Check All > Show Progress*.

> ### Note
>
> Consistency check gets triggered in thebackground and the UI shows the latest transport request status as soon as *Check All* is triggered. The UI gets refreshed everytime after certain interval showing the latest status. The transport with status *Checking* shows the request currently being processed.
>
> As soon as the consistency check is complete, a momentarily message gets displayed at the bottom.
>
> If the *Check all* is already running, you can't trigger another and it will give you an error message

Release All

1. Open the *Export Customizing Transports* app. A list of customizing requests is displayed, with the latest request shown first at the top.
2. You may put a filter on Status for *modifiable* and *checking status*. Additionally, you can sort the requests in ascending order to see the current status of those requests sequentially.
3. Select *Release All > Start* to trigger the release of all open requests.
4. When the release all process is running, select on *Release All > Show Progress* to see the currently active phase.
5. Once the release all process is completed, select *Release All > Show Progress* to check if any phase has failed and to view the list of requests with errors, if there are any.
6. The release of all open requests consists of the following phases:

> ### Note
>
> If any phase fails, the process does not proceed to the next phase.

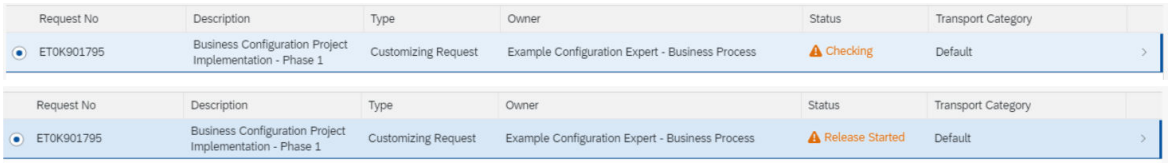
1. **Suppression of dependency check errors for all open requests.**
   - The dependency check errors are suppressed for the open requests since all open requests are released together.
   - If any error occurs in this phase, the message *Error occurred in suppression of request dependency check errors* is displayed. This message appears when you select *Release All > Show Progress* after the run is completed. The message also contains a detailed description, which you can refer to.

2. **Consistency check of all open requests.**
    - Consistency check is performed for all the open requests.
    - If any error occurs in this phase, the message *Error occurred in Check phase* is displayed. This message appears when you select *Release All > Show Progress* after the run is completed. Additionally the individual requests with error in the consistency check are also displayed. You can navigate to those requests to see the logs of the error messages.
3. **Check for changes to open requests during the consistency check phase**
    - The open requests are checked for any modifications made during the consistency check phase. Ensure that no modifications are made to the open requests and no changes are recorded to them during the process.
    - Changes to open requests, such as adding or deleting tasks under these requests or deleting any open requests, are not allowed during this process. This is because the dependency check errors have been suppressed with the assumption that the dependent requests are also released together. Therefore, changes to open requests could result in an inconsistent state.
    - If any error occurs due to any open request being modified or changed during the consistency check phase, the message *Error occurred in suppression of request dependency check errors* is displayed. This message appears when you select *Release All > Show Progress* after the run is completed. The message also contains a detailed description, which you can refer to.
4. **Assign ownership of all open requests to your user (Assign To Me)**
    - All open requests that are not already owned by you will be assigned to your user.
    - If any error occurs in this phase, the message *Error occurred in Assign To Me phase* is displayed. This message appears when you select *Release All > Show Progress* after the run is completed. Additionally the individual requests with errors in assigning the ownership are also displayed. You can navigate to those requests to see the logs of the error messages.
5. **Determining the sequence for releasing open requests**
    - The sequence for releasing open requests is determined.
    - If any error occurs in this phase, the message *Error occured in determination of release sequence for open requests* is displayed. This message appears when you select *Release All > Show Progress* after the run is completed. The message also contains a detailed description, which you can refer to.
6. **Release of all open requests.**
    - In the final phase, all the open requests are released in the order determined in the previous phase.
    - If any error occurs in this phase, the message *Error occured in Release phase* is displayed. This message appears when you select *Release All > Show Progress* after the run is completed. Additionally the individual requests with an error during the release are also displayed. You can navigate to those requests to see the logs of the error messages.

> **Note**
>
> Consistency check gets triggered in thebackground and the UI shows the latest transport request status as soon as *Release All* is triggered. The UI gets refreshed everytime after certain interval showing the latest status. The transport with status *Checking* shows the request currently being processed.
>
> As soon as the consistency check is complete, a momentarily message gets displayed at the bottom.
>
> If the *Release all* is already running, you can't trigger another and it will give you an error message

Release Request

1. Release any tasks belonging to your transport request. A request can't be released until all its tasks have been released.
2. Check the consistency of your request by triggering a release simulation: Select the request and click *Release* > *Check*.
3. You can also release your request directly without triggering a simulation: Select the request and click *Release* > *Execute*.
4. To refresh the status of a released or simulated request: Select the request with the status *Checking* or *Release Started* and click *Refresh*.



5. View the results of the simulation or direct release in the *Logs* tab.

Merge Request

If there are dependencies between different requests or several business users have recorded their configuration changes in different requests, you can choose to merge these different requests into one.

1. Select the request you want to merge.
2. Select *Merge*.
3. In the selection dialog, enter the target request number to which the selected request should be merged, and click *Merge*.

> ### Note
>
> The configuration changes recorded in the request you selected to merge are copied to the target request. After merging, the previously selected request will be deleted.

Create Copy

It can happen that for technical reasons, it's necessary for you to create a copy of an exported request. This can be the case if, for example, your exported request containing the business configurations didn't synchronize to the Git Repository. In order to synchronize these business configurations with the Git Repository again, a copy is needed.

1. Select the request you want to copy.
2. Click on *Create Copy*.

Repository Id

You can export the business configuration recorded in a request to a specific software component by assigning the *Repository Id* when creating a request.

1. Select *Create* to create a new request.
2. Using the value help, check the allowed repository ids which can be assignet to the request.
3. Select the suitable repository id and save the changes.

6.3.1.6.6.2 Semantic Business Configuration Lock Token

When critical business configuration changes are made in a customizing tenant of the development system, a check is triggered in the test and production systems to ensure that the configurations can be changed consistently. To count as a consistent change, the new configuration must meet the following critera:

- It doesn't violate existing configurations or business processes in the production system.
- It can be imported into the test and production systems, offering the new or changed business processes.

If the new business configuration matches the existing configuration or application data, it generally returns either a **SUCCESS** or a **ERROR** messages. The **SUCCESS** status allows to save the business configuration in the customizing tenant of the development system. This business configuration can then be sent to the targeted test and production systems with a transport request to make the changes active. A **Semantic Business Configuration Lock Token** safeguards this verified configuration until the transport reaches the test and production systems.

This generic mechanism is defines into three phases:

1. **Setting the Semantic Business and Configuration Locks**
   After the checks are successful in the test and production systems, a lock is saved with the status **Preliminary | On hold**. Once the configuration changes are recorded to a transport and saved in the customizing tenant of the development system, the locks will be finalized and changed from **Preliminary | On hold** to **Active** in the test and production systems.
2. **Checking the Business Processes for existing Locks**
   When a lock is set in the test and production system, all related business processes are checked for active locks. If a business process is blocked due to an existing lock, the user receives detailed information explaining why the process could not be completed as usual.
3. **Release of Semantic Business & Configuration Locks**
   When an active lock exists, the business processes are blocked. To release this lock, import the transport containing the critical configuration changes into the test and production systems. This import will successfully release the locks.

> ⓘ Note
>
> Critical business configuration changes create a lock in the test and production systems, so these changes should no longer be imported into these environments. To resolve this, revert the configuration changes

in the customizing tenant of the development system, and then import the transport into the test and production systems to release the locks.

6.3.1.6.7 Maintain Customizing Translations

The *Maintain Customizing Translations* app provides an easy way to translate customizing extensions into other languages.

Context

You've built a customizing extension and would now like to translate it into other languages. The *Maintain Customizing Translations* app allows you to select your text sources and generate an XLIFF (1.2) file containing the translatable texts as well as useful notes on them. XLIFF (XML Localization Interchange File Format) is a standard used in localization. The XLIFF file can then be forwarded to a dedicated translator or translation agency. Once the XLIFF file has been translated, you can use the *Maintain Customizing Translations* app to upload it and publish the translated texts into your system.

Prerequisites

The following business catalogs are required:

Business Catalogs

Primary	Secondary
SAP_CORE_BC_BCT_TRA_PC (for the *Maintain Customizing Translations* app)	SAP_CORE_BC_BCT_TRN_MNG_PC (required to create customizing transport requests)
	SAP_CORE_BC_BCT_TRN_REL_PC (required to release customizing transport requests)

Key Features

You can use this app to:

- generate an XLIFF file of the texts that need to be translated.
- publish the translated texts to the system.

> **Note**
>
> If you want to have your texts translated by SAP, you can use the `SAP Translation Hub`. For more information, see

SAP Translation Hub.

Supported Device Types

- Desktop

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-DOC-TTL-MTR`.

6.3.1.6.7.1 Create a Translation Project

Context

You want to translate your customizing extension. Before you can select your text sources and target languages, you need to create a translation project.

> **Note**
>
> Please assign the business catalog `SAP_CORE_BC_BCT_TRN_MNG_PC` to ensure that changes can be recorded on transport requests.

Procedure

1. Open the *Maintain Customizing Translations* app.
2. Click + (Create) to create a new translation project. Choose a project type from the dropdown.
3. Fill in an identifier for your project, a name, and a description.
4. Click *Create* at the bottom right of your screen.
5. You now need to add your translation project to a transport. You can create a customizing transport request by using the *Export Customizing Transports* app. For more information, see Export Customizing Transports [page 2978] .

Results

Your project has now been created and added to the overview list.

Related Information

Select Your Text Sources [page 2987]

6.3.1.6.7.2 Select Your Text Sources

Context

Select the text sources that you want to have translated.

Procedure

1. In the *Maintain Customizing Translations* app, open the project you want to add text sources to.
2. Scroll down to *Text Sources* and click *Add*.
3. Here you can select which text source types to add. The following text source types are supported:
   - UI runtime adaptations

     > **Note**
     >
     > Mind that only the translation of those UI runtime adaptations which have been recorded on a Customizing transport request are supported. UI runtime adaptations which have not been recorded at all or have been recorded on a workbench request will not be considered
     >
     > For more information on how to work with UI runtime adaptations, see Adapting SAP Fiori UIs at Runtime - Key User Adaptation

   - `SAP Fiori launchpad` Pages
   - `SAP Fiori launchpad` Spaces

     > **Note**
     >
     > For more information on how to manage launchpad pages and spaces, see Manage Launchpad Pages and Manage Launchpad Spaces.

   - Text tables

> **Note**
>
> A database table needs to fulfill the following requirements to be considered a text table by the app:
>
> - The delivery class of the text table has to be C
> - There must be exactly one language key field
> - Every character-like, non key field with a length greater than one will be treated as a text attribute
> - The translation of text tables that contain key fields of type p (packed number), i (integer) or x (byte field) is not supported. That includes any kind of UUID and raw types, such as `SYSUUID_X16`.

4. Select *Add* to open the *Add Text Source* dialog which displays all text sources that can be added to the translation project. By filtering the type and name, you can choose the text sources you want to add. Select *Add* to add the text sources you selected to your translation project.

> **Note**
>
> The total amount of texts in a translation project must not exceed 1000. Be aware that a text source might contain more than one text.

Related Information

Select Your Source and Target Language [page 2988]

6.3.1.6.7.3 Select Your Source and Target Language

Context

Select your source language as well as the languages you want to translate your texts into.

Procedure

1. In the *Maintain Customizing Translations* app, open the project for which you want to create a translation from a source language to a target language. You can choose to select either a single language or more than one language.
2. Scroll down to *Translations* and click *Create*.
3. Select a *Source Language* and *Target Language* from the dropdown menus and confirm with *Create*.

> **Note**
>
> You can select from all of the languages that are installed in the system. For a given text source, it's only possible to maintain translations for languages other than the original language.
>
> Note that deleting a previously created translation entry for a given source and target language pair will only delete the corresponding entry in the translation project. Already published texts for the target language of the translation will not be deleted from the system.

4. A new entry has now been added to the list.
5. Repeat these steps for all source and target languages.

6.3.1.6.7.4 Generate and Download the XLIFF File

Context

You can generate an XLIFF file for your source and target language(s) that you can download and send to your dedicated translator or translation agency.

Procedure

1. In the *Maintain Customizing Translations* app, open the project you want to have translated and scroll down to *Translations*.
2. Select the entry for the source and target language for which you want to generate the XLIFF file.
3. Navigate to *Download*. Here you can select whether the XLIFF file should contain all texts (*All Texts*) or only those texts that were changed since you last published a translated XLIFF file in these languages (*Changed Texts*). If this is the first time you are generating an XLIFF file for the project for the given source and target language, choose *All Texts*.
   The XLIFF file will now be downloaded to your computer. You can send it to your dedicated translator or translation agency to be translated.

Related Information

Upload and Publish the Translated XLIFF File to the System [page 2990]

6.3.1.6.7.5 Upload and Publish the Translated XLIFF File to the System

Context

Your texts have been translated and a new XLIFF file has been sent back to you. You can now upload and publish the translated texts into the system. Here's how:

Procedure

1. In the *Maintain Customizing Translations* app, open your project and scroll down to *Translations*.
2. Select the entry for the source and target language for which you want to upload and publish the translated XLIFF file.
3. Click *Upload*.
4. Search for the translated file and confirm with *Upload*.
5. Now the translations have been uploaded to the system. They still need to be published, however. Select the entry and click *Publish*.
6. Select a transport to which you want to add the corresponding language (`LANG`) transport entries and click *Select*.
   Your translations have now been published.

6.3.1.6.8 Configure Units of Measurement

With this app, you can create, update, and delete units of measurement (UoM).

Prerequisites

To configure units of measurement, create a business role and assign the catalog `SAP_CA_BC_IC_LND_UOM_BCO_PC` to the role. For more information, see How to Create a New Business Role. You can also create a business role using the `SAP_BR_BPC_EXPERT` template. For more information, see How to Create a Business Role from a Template.

Key Features

Use this app to display, create, change, and delete:

- Units of measurement
- Dimensions for units of measurement
- ISO codes for units of measurement

Access Information

1. To access the app, create a business role and assign it to your user as described in the prerequisites above.
2. Open the `Custom Business Configurations` app. A list of all business configurations, which can be adapted, is displayed.
3. Choose the *Settings* button.
4. In the *View Settings* dialog window, go to the *Group* tab, group by *Configuration Group*, and choose *OK*. The three units of measurement configurations are now displayed together in the *Check Units of Measurement* group. Alternatively, you can use the regular search function to search for the UoM configurations individually, but we recommend using the group function to find them more easily.

Procedure

Configuring Units of Measurement

1. In the `Custom Business Configurations` app, search and open *Units of Measurement*.
2. To create a new entry, choose *Edit*, then *Create*.
3. Enter the internal unit of measurement, select the dimension, and choose *Continue*.
4. Fill in the required fields *Commercial Unit* and *Technical Unit*, and enter a text in the *Unit of Measurement Text* field for the new unit.
5. Maintain the other configuration sections as needed: *Display*, *Conversion*, *ALE/EDI*, *Application Parameters*, and *Temperature/Pressure Specifications*.
6. Save your entries.

Configuring Dimensions for Units of Measurement

1. In the `Custom Business Configurations` app, search and open *Dimensions for Units of Measurement*.
2. To create a new entry, choose *Edit*, then *Create*.
3. Enter the dimension and choose *Continue*.
4. Enter a text in the *Dimension Text* field and fill in the configuration sections *Dependency* and *Dimension Exponents* as needed.
5. Save your entries.

Once units are defined for a dimension, you can use the app to assign the official SI Unit to that dimension.

Configuring ISO Codes for Units Measurement

1. In the `Custom Business Configurations` app, search and open *ISO Codes for Units of Measurement*.
2. To create a new entry, choose *Edit*, then *Create*.
3. In the newly created row at the end, enter the ISO code and ISO code text.
4. Save your entries.

Supported Device Types

- Desktop

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-ASF-UOM`.

Related Information

Custom Business Configurations App [page 2970]

6.3.1.7 Communication Management

The communication management apps allow you to integrate your system or solution with other systems to enable data exchange.

Prerequisites

- Predefined communication scenarios are available for different use cases, for example the integration for employee data. Decide which scenario you are going to use to create a communication arrangement.

Process Overview

The communication management apps allow you to establish secure communication between your solution and other systems. The best practice to organize efficient data exchange is to proceed as follows:

1. Create communication users for inbound communication according to your needs using the *Maintain Communication Users* app.
2. Create a communication system using the *Communication Systems* app that represents the system you want to communicate with. If the selected scenario contains inbound services, you select the user for inbound communication that you have created earlier.
3. Create a communication arrangement using the *Communication Arrangements* app. You select the communication system that you have created earlier. The information about the user for inbound communication will be populated automatically. If several users for inbound communication exist for the system, you can select the appropriate one. Otherwise the first user in the list will be selected automatically. If the selected scenario contains outbound services, you have to enter the outbound user information manually.
4. Select *Save* to activate the communication arrangement.

> ⓘ Note
>
> A communication scenario is a design-time description of how two communication partners communicate with each other. It consists of inbound and/or outbound services as well as supported authentication methods. It provides technical information, such as the used inbound and outbound services and their service type, for example OData or SOAP, and the number of allowed communication arrangement instances. If the scenario exposes inbound services, it specifies the authorizations that are required to execute the services. The following types of communication scenarios are available:
>
> - *Managed by SAP*, where SAP provides a ready-to-use communication scenario and you create and maintain a communication arrangement.
> - *Managed by Customer*, where you develop a communication scenario and create and maintain a communication arrangement.

Related Information

Maintain Communication Users [page 2993]
Communication Arrangements [page 2997]
Communication Systems [page 3001]
IAM Information System [page 3102]

6.3.1.7.1 Maintain Communication Users

You can use this app to create and edit communication users. Communication users are used by solutions to authenticate themselves to be able to post data.

With this app you can manage communication users for different integration scenarios with other solutions. A communication user enables the integration with other solutions. To be able to post data, the solutions have to authenticate themselves with the user and password you create here. The communication users are assigned to the communication system you want to use.

Key Features

You can use this app to:

- Create a user
- Edit a user
- Lock or unlock a user
- Delete a user
- Display communication systems that use the selected communication user
- Display communication arrangements for the systems that use the selected communication user
- Download a list of all communication users to a spreadsheet

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-COM`.

Related Information

Communication Systems [page 3001]
How to Create Communication Users [page 2995]

6.3.1.7.1.1 How to Create Communication Users

Context

To create a new communication user, perform the following steps:

Procedure

1. On the initial screen select *New*.
2. Enter a valid user name and description.
3. Either provide a password (basic authentication) or upload certificates for certificate-based authentication.

   > **Note**
   >
   > We recommend that you use certificate-based authentication.

   1. Configure password-based authentication

      > **Note**
      >
      > We recommend that you use the *Propose Password* button.

      > **Note**
      >
      > Passwords for communication users need to comply with the following requirements: the password length must be at least 20 characters, the password must contain at least one special character. The user will be locked after three unsuccessful login attempts.

   2. Configure certificate-based authentication by uploading a X.509 client certificate.
      The customer system must use a client certificate signed by an appropriate certification authority (CA). A list of all root CAs approved by SAP Global Security is available in SAP Note 2801396 (SAP Global Trust List).
4. Select *Create* to save the user.

   You can now assign the created user to a communication system. Use the *Communication Systems* app for this purpose.

Related Information

Maintain Communication Users [page 2993]
Communication Systems [page 3001]

6.3.1.7.1.2 How to Display Change Documents for Communication Users

Context

To get an overview of the changes that were made to communication users, you can display a list of change documents by completing the following steps:

Procedure

1. Click *Display Changes* in the *Maintain Communication Users* list view if you want to display change documents for several communication users. Alternatively, click *Display Changes* in the *Maintain Communication User* detail screen if you want to display the change documents for one communication user.
2. Select the required change categories. The following change categories are available:
   - *User Created*
   - *User Deleted*
   - *Password Status*
   - *Password*
   - *Lock Changed*
   - *User Alias Changed*
   - *Communication Scenario Assignment Changed*
   - *Certificate*
3. If required, filter for certain communication users in the *User ID* area.
4. If required, filter for certain date ranges or for specific points in time in the *Changed At* area.
5. If required, filter for changes made by specific users in the *Changed By* area.
6. Click *Go* to view the relevant change documents.

6.3.1.7.2 Display Communication Scenarios

You can use this app to get an overview of available communication scenarios.

With this app you can display communication scenarios used for integrations, and the scenario status.

Key Features

You can use this app to:

- Display all available communication scenarios
- Display scenario details and properties
- Display supported inbound authentication methods and inbound services used in a communication scenario
- Display supported outbound authentication methods, certificates, and outbound services used in a communication scenario
- Download certificates for specific purposes
- Display communication arrangements in which a communication scenario is used
- Create a new communication arrangement based on a communication scenario
- Display scope items: check which scope items the communication scenarios depend on. Communication scenarios that do not depend on any scope items are always visible in the system. For these communication scenarios, *Scope Items (0)* is displayed on the relevant tab and the following message appears in the table: *The communication scenario is not scope-dependent*.
- Display component: check under which component you can create an incident if required.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-COM`.

6.3.1.7.3 Communication Arrangements

With this app you can create and edit communication arrangements that your company has set up with a communication partner. The system provides communication scenarios for inbound and outbound communication that you can use to create communication arrangements. Inbound communication defines how business documents are received from a communication partner, whereas outbound communication defines how business documents are sent to a communication partner. The communication scenario determines the authorizations, inbound and outbound services and the supported authentications methods, that are required for the communication.

Key Features

You can use this app to:

- Display all existing communication arrangements
- Display detailed information for the selected communication arrangement
- Create new communication arrangements
- Maintain general information
- Select the required communication system. The user information and authentication method are filled in automatically after you have selected a communication system.
- Delete communication arrangements that you have created earlier

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-COM`.

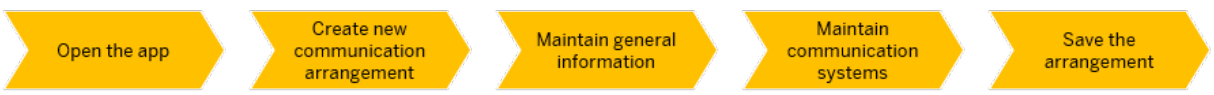
6.3.1.7.3.1 How to Create a Communication Arrangement

Prerequisites

- You have already created a communication system with inbound and outbound users.
- You have already created a communication user with a supported authentication type that is defined in the selected communication scenario.

Context

Process Steps



To create a communication arrangement, proceed as follows:

Procedure

1. Open the *Communication Arrangements* app from the SAP Fiori Launchpad. Already existing communication arrangements are listed on the initial screen.
2. Select *New*. In the *New Communication Arrangement* window, select a communication scenario and define the arrangement name. Select *Create*.
3. The new *Communication Arrangements* screen is now open. In the *Communication System* field under *Common Data*, select a communication system that you want your system to connect to. Alternatively, you can click *New* to create a new communication system.
4. Depending on the underlying communication scenario, maintain the *Additional Properties*.
5. If the communication scenario provides inbound services, you have to select the desired communication user and authentication method from the assigned communication system. The defined communication user then has the authorization to call these services. In the *Inbound Services* section, the URLs to the service endpoints are displayed. Depending on the underlying scenario, maintain the required inbound parameters.

   > ### Note
   >
   > The inbound section of the communication arrangement provides the correct URLs for API consumption. Depending on the protocol and scenario, the host name or path might differ. Always use the URLs provided in this section when configuring the calling service. For security reasons, we have different URLs for each user type.

6. If the communication scenario provides outbound services, you have to select the desired outbound user and authentication method from the assigned communication system. In the *Outbound Services* section, the URL path and port need to be defined. Depending on the underlying scenario, maintain the required outbound parameters and job execution details.

   > ### Note
   >
   > If the required outbound service is of the type *Outbound SQL Access*, you need to make the following additional settings:
   >
   > - *Remote Database Schema Name*: Here you enter the schema in the remote system containing the objects to be accessed.
   > - If required click *Use Default*. This reads the *Remote Database Schema Name* from the Outbound SQL access service.
   > - *Remote Database Name*: The database in the remote system containing the objects to be accessed. Usually you can leave the field empty unless the remote system hosts multiple databases.

7. Save the arrangement.

Related Information

Maintain Communication Users [page 2993]
Communication Systems [page 3001]

6.3.1.7.3.2 How to Display Change Documents for Communication Arrangements

Context

To get an overview of the changes that were made to communication arrangements, you can display a list of change documents by completing the following steps:

Steps

1. Click *Display Changes* in the *Communication Arrangements* list view if you want to display change documents for several communication arrangements. Alternatively, click *Display Changes* in the *Communication Arrangment* detail screen if you want to display the change documents for one communication arrangement.
2. Select the required change categories. The following change categories are available:
   - *Communication Arrangement*
   - *Communication Arrangement Root Property*
   - *Communication Arrangement Root Property Value*
   - *Communication Arrangement Root Property Structure Instance*
   - *Communication Arrangement Inbound Service Defaults*
   - *Communication Arrangement Inbound Service*
   - *Communication Arrangement Inbound Service Property*
   - *Communication Arrangement Inbound Service Property Value*
   - *Communication Arrangement Outbound Service Defaults*
   - *Communication Arrangement Outbound Service*
   - *Communication Arrangement Outbound Service Property*
   - *Communication Arrangement Outbound Property Value*
3. If required, filter for certain communication arrangements in the *Arrangement Name* area.
4. If required, filter for certain date ranges or for specific points in time in the *Changed At* area.
5. If required, filter for changes made by specific users in the *Changed By* area.
6. Click *Go* to view the relevant change documents.

6.3.1.7.4 Communication Systems

You can use this app to create communication systems. Communication systems are created to enable the communication among different systems.

With this app you can create new communication systems that you can later use to establish communication arrangements. To enable communication between different systems you have to register these systems in the *Communication Systems* app. The communication system represents the communication partner within a communication. For inbound communication, this is the system that calls services provided by your system. For outbound communication, this is the system that provides services called by your system.

Key Features

You can use this app to:

- Create communication systems
- Display detailed information about the already existing communication systems
- Delete communication systems

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-COM`.

Related Information

Maintain Communication Users [page 2993]

6.3.1.7.4.1 How to Create Communication Systems

Context

To create a communication system perform the following steps:

Procedure

1. On the initial screen of the *Communication Systems* app, select *New*.
2. In the *New Communication System* dialog define an ID and a name of the new system.
3. Fill in the required fields under:

    - *General Data*
      Enter system ID and system name.
      Contact Information
      Enter name, email or phone number of a contact person.
    - *Technical Data*
      General
      If you want your communication system to be used for a communication arrangement based on a communication scenario that only contains inbound services, we recommend that you select the *Inbound Only* checkbox. If this checkbox is selected, the input fields that are only required for outbound communication disappear.
      If your communication system is intended for outbound communication, host, port, user and authentication method are used as specified in the destination service.
    - *Cipher Suites*
      Here you can define a set of encryption mechanisms you want to use as a basis for your communication. You can choose between different versions of TLS (Transport Layer Security). A default value is defined by the system and you can change this if required.
    - *Destination Service* (Only for Outbound Communication)
      Because SAP BTP for the ABAP environment uses destination services of SAP BTP for outbound communication, you need to enter the name of the required destination service. For more information, see Set Up the Destination Service. You also need to select the instance the destination service is based on. For more information, see Creating the Service Instance for the ABAP Environment.

      > ⓘ Note
      >
      > A predefined instance is selected by default. If you want to add your own instance, deselect this instance.

    - *Cloud Connector* switch
      Indicates that SAP Cloud Platform Cloud Connector is used for communication with this system.
      SSC Location ID: Enter the SAP Cloud Platform Cloud Connector ID

If your scenario is uses RFC modules, and SAP Cloud Platform Cloud Connector is not activated, the system uses WebSocket RFC. Therefore the remote system needs to support WebSocket RFC and you need to specify the client in the *RFC Settings* section.

- *RFC Settings*
  Define additional settings needed for RFC communication.
- *OAuth 2.0 Settings*
  Define additional settings if oAuth 2.0 is used for outbound communication.

> ### Note
>
> The propagation of technical users from the cloud application towards on-premise systems can be enabled in the *Communication Systems* app. To propagate the technical user, you have to select the *Cloud Conn. Technical User Propagation* checkbox in the *OAuth 2.0 Settings* area (the checkbox is only active if the *Cloud Connector* switch is on). This is similar to principal propagation, but in this case, a technical user is propagated instead of a business user.

- *Remote SQL Access* switch
  Define additional settings needed to access remote databases via SQL:
  Select the required *Adapter Name*. This specifies the corresponding HANA Smart Data Access adapter and the type of access method to be used by the SAP HANA database to access the data. Use the field input help for the list of adapter names you can specify.
  Enter the required *Adapter Configuration*. This specifies the connection parameters for a given adapter.
- *OAuth 2.0 Identity Provider* switch
  Before you can authenticate and get an access token to access resources using an OAuth 2.0 client, you have to configure a trusted relationship to the required *SAML Identity Provider* (token issuer). Upload the signing certificate of the trusted provider and define the provider name.
  Define the *User ID Mapping Mode* if you don't use e-mail addresses, but either logon name or global user ID.
  The *User ID Mapping Mode* is required when the *SAML Identity Provider* is creating SAML assertions with the *NameIdentifierFormat* **`unspecified`**.
  This setting is ignored when then SAML assertion is using the *NameIdentifierFormat* **`eMailAddress`**.
  Please note that the required authorization for calling the service is granted to the business user and not to the technical user. When you set up a communication arrangement using oAuth, you can click *OAuth 2.0 Details*, select the scope and click *Granted by Business Catalogs* to find the necessary details. This way you can retrieve the business catalogs that are required for calling the service.
- *SAML Bearer Assertion Provider*
  Before you can authenticate using SAML2 Bearer assertions transmitted via the authorization header, you have to configure a trusted relationship to the required *SAML Identity Provider* (token issuer).
  Proceed as follows:
  1. Navigate to your *BTP Cockpit* subaccount
  2. Go to ▶ *Connectivity* > *Destination Trust* > *SAML IDP Trust Configuration* > *Active Trust Certificate* >
  3. Choose *Export* to download the signing certificate of the SAML Identity Provider.

  Upload the signing certificate of the trusted provider and define the provider name in your ABAP system. Proceed as follows:
  1. Open the *Communication Systems* app.
  2. Create a new communication system or open an existing one.
  3. On the *General* tab, activate the *SAML Bearer Assertion Provider* switch.

4. Choose *Upload Signing Certificate* and upload the certificate that you exported from the SAP BTP Cockpit.
5. After uploading, locate the newly displayed *Signing Certificate Subject* field.
6. Copy the *CN (Common Name)* value from the certificate subject.
7. Paste this value into the *SAML Bearer Issuer* field.
8. Save your changes.

Define the required *User ID Mapping Mode* if it is unspecified and you don't use e-mail addresses, but either logon name or global user ID.
The *User ID Mapping Mode* is required when the *SAML Identity Provider* is creating SAML assertions with the *NameIdentifierFormat* **`unspecified`** .
This setting is ignored when then SAML assertion is using the *NameIdentifierFormat* **`eMailAddress`**.

- *OpenID Connect* switch
  Before you can authenticate using OpenID Connect, you have to configure a trusted *OpenID Connect Provider*.
  Define the required *User ID Mapping Mode*, the *OIDC Token Issuer* and *Client ID*. Click *Import Metadata*. This is necessary to ensure that the configuration is validated and complete.
  If the automatic configuration fails, you can configure the OpenID Connect settings manually. For more information, see *Manual Configuration - OpenID Connect (OIDC) Provider* in the *Related Information* section.

  > **Note**
  >
  > You can select the *User Mapping Claim* to define the authorization attributes that are used by the identity provider. The default selection is *sub*.

4. Add technical users for inbound communication. You can either select a user from the list or create a new user. If you decide to create a new user, you will be redirected to the *Maintain Communication User* app.

   > **Note**
   >
   > Inbound users are communication users provided by your system and are used by the communication system to call the inbound services.

5. Save the changes.

   You can now establish a communication arrangement with the created system. Use the *Communication Arrangements* app for this purpose.

Related Information

Communication Systems [page 3001]
How to Create Communication Users [page 2995]
Manual Configuration - OpenID Connect (OIDC) Provider [page 3006]

6.3.1.7.4.2 Authentication Methods for Outbound Users

You can define how your solution authenticates itself when it sends business documents to the external system. For security reasons, we recommend the use of certificate-based authentication.

You can choose between the following authentication methods:

- User name and password
- SSL Client Certificate
- OAuth 1.0
- OAuth 2.0
  You can choose between the following authentication methods:
  - Basic
    If you use this method, you need to enter a client secret.
  - mTLS
    If you use this method, you need to select a client certificate.
  - JWT
    You can use JWT (JSON Web Tokens) to authenticate an OAuth client at the token endpoint of an OAuth authorization server (instead of using a client ID and a client secret).
    If you use this method, you need to download the corresponding certificate to your local device.
  - None

If you want to use a SSL Client Certificate, you can either use the default certificate of your solution or a client certificate you have defined in the *Maintain Client Certificates* app.

6.3.1.7.4.3 Enabling Public OAuth Clients (PKCE) for Inbound Communication Scenarios

Context

Communication scenarios that support OAuth 2.0 as authentication method for inbound communication can now be consumed by public OAuth clients using PKCE (Proof Key for Code Exchange). This allows desktop applications (acting as OAuth Clients) to consume OAuth-protected OData services with the identity of a business user. The business user will be prompted for providing consent to this action once (a web browser is required for this).

To enable consumption by public OAuth clients using PKCE, proceed as follows:

Procedure

1. In the *Communication Systems* app, enter the *Client Redirect URI* under Inbound OAuth 2.0 Client Settings. Thereby you define the URL to which the client will redirect the browser after authorization has been granted by the user.
2. Under *Users for Inbound Communication*, click Add.
3. In the *New User for Inbound Communication* pop-up that appears, select *OAuth 2.0* in the *Authentication Method* dropdown list, enter the *OAuth 2.0 Client ID* and click *OK*.

Related Information

How to Create a Communication Arrangement [page 2998]

6.3.1.7.4.4 Manual Configuration - OpenID Connect (OIDC) Provider

When you have made the required settings under *OpenID Connect (OIDC) Provider* – *Basic Configuration* in the *Edit* mode of the communication system, the system attempts to automatically configure OpenID Connect for the provider you have defined. If the provider does not convey the required information, you receive an error message. In this case, you can use manual configuration as a fallback. Activate the *Manual Configuration* switch if this is required.

As a first step, you can download web keys from the provider. Should this fail too, you can directly enter the web keys as defined in your OpenID Connect specification.

6.3.1.7.4.5 How to Display Change Documents for Communication Systems

Context

To get an overview of the changes that were made to communication systems, you can display a list of change documents by completing the following steps:

Procedure

1. Click *Display Changes* in the *Communication Systems* list view if you want to display change documents for several communication systems. Alternatively, click *Display Changes* in the *Communication System* detail screen if you want to display the change documents for one communication system.
2. Select the required change categories. The following change categories are available:
   - *Communication System*
   - *Users for Inbound Communication*
   - *Users for Outbound Communication*
   - *Business Partners*
3. If required, filter for certain communication systems in the *System ID* area.
4. If required, filter for certain date ranges or for specific points in time in the *Changed At* area.
5. If required, filter for changes made by specific users in the *Changed By* area.
6. Click *Go* to view the relevant change documents.

6.3.1.7.5 Maintain Extensions on SAP BTP

You can use this app to create *SAP Business Technology Platform (SAP BTP)* extensions to build extension applications.

With this app you can create SAP BTP extensions for your SAP S/4HANA Cloud system. These automatically connect the two systems and enable you to build extension applications for SAP S/4HANA Cloud on SAP BTP.

Key Features

You can use this app to:

- Create new SAP BTP extensions

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support while setting up the app (for example if the content could not be activated), please report an incident under component `BC-SRV-APS-COM`.

If you need support or experience issues that relate to the UI, please report an incident under component `BC-NEO-EXT-S4C`.

Related Information

Deregister or Remove a System
Trigger the Registration in the SAP S/4HANA Cloud Tenant
Register an SAP S/4HANA Cloud System in a Global Account in SAP BTP

6.3.1.7.6 Display Inbound Services

With this app you can display all available inbound services defined by you or delivered by SAP. You can also display further information about the services, such as if and where a deprecated inbound service is used and a list of its successors. You can also check whether the service is deprecated system-wide or only in a scenario.

Key Features

You can use this app to:

- Display documentation for the service
- Display authorization objects (this is only relevant in the custom communication scenarios environment)
- Display successors
- Display all communication scenarios that use this service

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-COM`.

6.3.1.7.7 Display Outbound Services

With this app you can display all available outbound services defined by you or delivered by SAP. You can also display further information about the services, such as if and where a deprecated outbound service is used and a list of its successors. You can also check whether the service is deprecated system-wide or only in a scenario.

Key Features

You can use this app to:

- Display documentation for the service
- Display successors
- Display all communication scenarios that use this service

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-COM`.

6.3.1.7.8 Business Catalogs for Communication Management

You assign business catalogs to business roles that are assigned to business users. Business catalogs contain authorizations that define what a business user with a certain business role is allowed to do.

Business Catalogs for Communication Management Apps

Business Role Catalog	Authorizations	Restrictions
Communication Management SAP_CORE_BC_COM	All authorizations for the COM apps	None

6.3.1.7.9 Glossary for Communication Management

Overview of terminology used for communication management

Terminology Overview

Term	Description
Communication arrangement	A communication arrangement describes a communication scenario with a remote system during configuration time. It provides the necessary metadata for service configuration.
Communication system	A communication system represents the communication partner with all technical information that is required for communication, that is hostname, identity, user information, certificates, etc.
Communication user	A communication user is a special type of technical users that is assigned to a communication system. You create a communication user for particular communication scenarios.
Communication scenario	A communication scenario defines the communication between external systems and your own SAP cloud system. It bundles inbound and outbound services and additional properties that are required for configuration settings.

6.3.1.7.10 Display Connectivity Trace

With this app you can analyze inbound connectivity issues, such as failed SSL handshakes, malformed HTTP requests or failed log-in.

Key Features

You can use this app to:

- Display connectivity trace details
- Cross-navigate to the *Maintain Business Users* app and the *Maintain Communication Users* app.
- Maintain trace criteria

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-COM`.

6.3.1.7.10.1 How to Create a Trace

Context

In order to start a trace, you need to define the trace criteria.

Procedure

1. Select the *Display Connectivity Trace* tile on the SAP Fiori Launchpad to open the app. On the initial screen, click *Define Trace Criteria* in the top-right corner.
2. Click *Create*.

3. Enter a description if required. We recommend using a description that is easily identifiable in the filter.
4. Change the validity if required. The default setting is *10 Minutes*. Unlike a log, a trace is only available for a limited time frame.
5. Change the *Request Method* if required. The default setting is *All*.
6. Click *Save*.

Results

The trace is now activated for the defined criteria. You can filter for it in the *Trace Criterion* dropdown list. Please note that there might be a time delay until the activation takes effect because all the affected components need to be notified.

6.3.1.7.11 Monitor bgRFC Queues

With this app you can monitor the bgRFC (background RFC) queues together with the associated units and inbound destinations. As well as displaying queues and units that belong to a destination, the app provides you with functions that let you intervene in their processing, such as *Stop*, *Start*, or *Delete*.

Please note the following when using the *Monitor bgRFC Queues* app.

- Deleting a queue or unit can be risky. If you delete a queue, the included units are automatically removed. This can cause issues in the processing because dependencies between units and queues might exist.
- Queues and units can be stopped from the monitor. This function is only intended for analysis and not for regular operations, because it has consequences for runtime. If a queue is to be stopped, this also refers to the top unit that uses this queue. As soon as the unit is restarted, the queue is released again.
- Transactional (*T-Type*) units can be displayed to check whether any errors occurred. They are listed below the *BGPF* (Background Processing Framework) *Inbound Destination*. To call them up, you need to click the *Standard* list entry of the relevant queue.
- In the *Queue* table, the *Start All* and *Delete All* pushbuttons enable batch operations. When clicked, they apply their respective actions to all current entries in the table based on the filters that have been set. Therefore, if no filters are in place, all queues in the table will be affected. Conversely, if filters have been set, only those queues matching the filter criteria will be impacted. If you click one of these pushbuttons, a confirmation dialog box appears indicating the number of queues that will be impacted by the action.

Key Features

You can use this app to:

- Save units with errors for later analysis to move them into a separate a queue and execute the blocked units in this queue that were waiting to be processed.

> ⚠ Caution
>
> We recommend that you use this feature with caution and process only the units that are independent from the one with errors, for example, units for sales orders with multiple line items.

- Stop and start destinations, queues, units
- Delete queues and units if they are no longer needed
- Check for errors
- Display the first function module for each unit
- Navigate to Message Monitoring to view more details if required

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-COM`.

Related Information

6.3.1.7.11.1 Monitor bgRFC Queues: Further Information

Please note the following when using the *Monitor bgRFC Queues* app.

- Deleting a queue or unit can be risky. If you delete a queue, the included units are automatically removed. This can cause issues in the processing because dependencies between units and queues might exist.
- Queues and units can be stopped from the monitor. This function is only intended for analysis and not for regular operations, because it has consequences for runtime. If a queue is to be stopped, this also refers to the top unit that uses this queue. As soon as the unit is restarted, the queue is released again.
- Transactional (*T-Type*) units can be displayed to check whether any errors occurred. They are listed below the *BGPF* (Background Processing Framework) *Inbound Destination*. To call them up, you need to click the *Standard* list entry of the relevant queue.
- In the *Queue* table, the *Start All* and *Delete All* pushbuttons enable batch operations. When clicked, they apply their respective actions to all current entries in the table based on the filters that have been set. Therefore, if no filters are in place, all queues in the table will be affected. Conversely, if filters have been

set, only those queues matching the filter criteria will be impacted. If you click one of these pushbuttons, a confirmation dialog box appears indicating the number of queues that will be impacted by the action.

6.3.1.7.12 Maintain SAML 2.0 Configuration

You can use this app to upload the primary and secondary signing certificates you have maintained within the SAP Cloud Identity Services. Please note that Base64 ASCII encoding is required for this.

Key Features

This app provides the following key features:

- Upload primary signing certificates
- Upload secondary signing certificates
- Delete secondary signing certificates
- Display detailed information for the certificates, such as validity, subject, or issuer

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-COM`.

Related Information

Tenant SAML 2.0 Configurations

6.3.1.8 Development

SQL Trace Analysis [page 3015]

Display Publishing Processes [page 3015]

6.3.1.8.1 SQL Trace Analysis

With this app you can access and analyze SQL trace records after you have used the SQL trace function of the ABAP performance trace in the ABAP Development Tools (ADT).

Key Features

This app provides the following key features:

- Display of all SQL traces that are available under your user name
- View of the SQL trace entries in chronological order, together with information about the ABAP source code location, duration of the execution, and the number of accessed records
- View and recalculation of the prepared and executed access plan
- Display of the origin of SQL statements in the source code

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

6.3.1.8.2 Display Publishing Processes

With this app you can monitor publishing processes for custom communication scenarios and business catalog extensions.

Key Features

You can use this app to:

- Filter for a certain status, such as *Successful*
- Filter for a certain object type, such as *Custom Communication Scenario*
- View details of the publishing process

- Restart a publishing process if required

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

6.3.1.9 Employee Master Data

Business User Management [page 3016]
Business User Management enables and supports the lifecycle maintenance of business users.

Maintain Employees [page 3016]

6.3.1.9.1 Business User Management

Business User Management enables and supports the lifecycle maintenance of business users.

6.3.1.9.2 Maintain Employees

With this app, you can create employees and modify employee information.

Key Features

You can use this app to:

- Create employees directly in the app or with CSV file import
- Edit employee information
- Retrieve the change log to view modified employee data

- Select employees for archiving
- Export employee data

In addition, the app supports the following technical features and options:

Navigate to *Maintain Business User*

Supported Device Types

- Desktop
- Tablet
- Smartphone

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `CA-GTF-BUM`.

6.3.1.9.2.1 How to Select an Employee for Archiving

Context

You can't delete an employee in the *Maintain Employees* app. You can only select an employee for archiving.

> **Note**
>
> It's not possible to select an employee for archiving in the *Maintain Employees* app, if the employee has a user assigned to it. In this case, you have to delete the user in the *Maintain Business Users* app first.

The employee consists of a business partner in the role category `BUP003` (Employee), a workplace address, and a user. An employee cannot be deleted immediately and in one step for business process reasons. If you need to permanently delete an employee, you have to delete the user first. Afterwards, you're able to delete the business partner, which also contains the workplace address.

In the *Maintain Employees* app, the *Select for Archiving* filter is set to *No* by default. This means, only employees that are not selected for archiving are displayed. Choose the arrow icon (⌄) to expand the header and see all filter options. If you want to see employees that are selected for archiving choose *Yes* from the dropdown and *Go*. To see both, select the empty field from the dropdown.

This is how you can select an employee for archiving:

Prerequisite:

The employee doesn't have a user assigned.

Procedure

1. Open the *Maintain Employees* app and select the respective employee from the list.
2. Choose *Edit*.
3. On the *General Information* tab, in the *Validity Period* section, mark the *Selected for Archiving* checkbox.
4. Choose *Save*.

   Alternatively, choose the *Select for Archiving* button.

6.3.1.10 Extensibility

Custom Logic [page 3018]

Custom Logic (Deprecated) [page 3025]

Configure Predefined Custom Fields [page 3030]

6.3.1.10.1 Custom Logic

With this app, you can implement BAdIs that have been released by SAP partners.

> ⚠ Caution
>
> The management of data in an extension scenario deviates from the management of data in the standard scenarios. You are responsible for ensuring that the data used in an extension scenario is managed in accordance with any applicable legal requirements or business needs, such as data protection legislation or data life cycle requirements. The extensibility framework should not be used for the processing of personal data if this processing falls under any applicable data protection legislation.

Key Features

You can use this app to do the following:

- Create your own implementation
  - Specify filter conditions to define when an implementation is executed

- Publish your implementation to your test system
- Write custom code by using ABAP for key users
  - Test your code with predefined test variants, for example
  - Compare your code with older code versions
- Edit filter conditions of existing implementations
- Delete implementations

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-EXT-BL`.

Related Information

Creating Implementations [page 3019]
Creating Custom Code [page 3020]

6.3.1.10.1.1 Creating Implementations

Get an overview of how to create and publish your implementations.

Process Steps

This is a generic description about how to create implementations. For specific documentation, refer to the individual extension point documentation in the *Custom Logic* app.

1. To create a new implementation, open the *Custom Logic* app and click *Create*.
2. Select the extension point (BAdI) that you want to implement. Perform a free text search by using the search field. The extension point descriptions, IDs and documentation will be searched. To view the documentation of an extension point, click *View Documentation*. When you have selected an extension point, click *Step 2*.
3. Depending on the extension point you've selected, you can enter the implementation attributes next. Open the *Edit Filter Condition* dialog by clicking *Add*.

4. All BAdI filter conditions added in the dialog are connected with **AND**. Click *Save* to add the filter condition to the list of filters. You can add more rows to the filter list by opening the dialog again. The filter list rows are connected with **OR**. Click *Step 3* once you've maintained the filters.
5. Enter the implementation description and the ID. The description must start with a non-whitespace character. The ID starts with a fixed prefix and its total length must not exceed 30 characters. Click *Review* once you've maintained valid values.
6. Review the data you entered. If you want to make changes, click *Edit* in the respective section. Once you're finished, click *Create*. The creation process starts in the background and you're taken to the implementation detail page.
7. Once the implementation is created, the status will change from *Creating* to *Created*. You can now edit the implementation attributes if your implementation has any, or publish your implementation to start writing code. After publishing, click *Open Code Editor* to write custom code.

6.3.1.10.1.2 Creating Custom Code

With the *ABAP Cloud Editor*, you can write your own code using ABAP for key users.

The editor is divided into three tabs that focus on different aspects of dealing with code: *Develop*, *Test*, and *Compare*.

Develop

On the *Develop* tab, you can write, save, and publish ABAP code. Saving means writing a draft version of the code that only you can see. Once you publish your code, it'll be available in the system and visible for all users. Depending on the method you're implementing, there is a button *Show Sample Code*. Click this button to open a dialog that shows the sample code provided by SAP. Click *Expand* to increase the vertical space for the code.

The method signature on the right provides information about the method's parameters.

Put the cursor on a class, interface, or parameter to view the documentation displayed in the element information at the bottom of the page.

In edit mode, you can add prebuilt code blocks by selecting *Insert Code Snippet*. For more information, see Inserting Code Snippets [page 3021].

You can use the entity manipulation language (EML) to implement behavior extensions. For more information, see Entity Manipulation Language.

You can use keyboard shortcuts while working with the ABAP Cloud editor. You can view a list of all possible shortcuts by clicking *Show Keyboard Shortcuts*.

Test

On the *Test* tab, you can test your code with different parameter values. Use the parameter kind dropdown to navigate to **importing**, **exporting**, **changing**, and **returning** parameters, depending on what the method offers.

Use the input fields in the table rows to enter parameter values. For table parameters, click *Add* to add a new table row or *Delete* to delete a table row.

On the right, you can choose what code version will be tested: either *Draft* or *Latest Published*. Click *Test* to run the test. If the test was successful, a toast will appear. You can then view the outcome in the different parameter tables.

If you want to save the test data you entered, click *Save As*, enter a test variant name, and click *Save*. If you have already selected an existing test variant, just click *Save*. Use *Manage* to delete saved test variants.

Compare

On the *Compare* tab, you can compare different code versions. If you have a draft, it can be selected on the left. This is useful if you want to copy parts from older code versions to a new draft, for example. On the *Compare* tab, you can only save code but not publish it. Use the *Develop* tab for all code editing features.

> ⓘ Note
>
> Please make sure that your draft logic is syntactically correct and tested. Otherwise, the draft logic might be lost when your system is upgraded to another release.

6.3.1.10.1.2.1 Inserting Code Snippets

Find out how to insert predefined or custom code snippets into your code.

Code snippets are small blocks of source code for recurring coding tasks such as calling a method of a custom reuse library. Some code snippets offer parameters for filling in placeholders in the code. There are two kinds of code snippets: Predefined code snippets and custom code snippets. The predefined code snippets are provided by SAP, while the custom code snippets depend on the custom entities you have created in your system.

Predefined Code Snippets

Predefined code snippets are calls to the custom business objects Write API. These code snippets will only be shown if the method you are implementing has the Write API as a parameter.

Procedure

1. Select a code snippet from the list.
2. Fill in the placeholders by maintaining the parameters on the *Details* page.

> ⓘ Note
>
> The parameters are not mandatory. You can also insert the code snippet with placeholders, but then syntax errors will appear.

3. Select *Insert Code Snippet* to insert the code snippet into your code at the last cursor position.

Custom Code Snippets

Custom code snippets are calls to the methods of your custom reuse libraries. The parameters of the method call are prefilled with sample data.

6.3.1.10.1.3 Statements in ABAP for Key Users

This topic provides you with an overview of ABAP keywords and ABAP system fields that are available with the ABAP version for key users.

You can search the tables for specific ABAP keywords and system fields that are available with the ABAP version for key users. If you need more information and examples about how to use the ABAP version for key users, you can refer to our tutorial blog linked under *Related Information*

ABAP for Key Users - ABAP System Fields

ABAP System Field	Restriction
sy-dbcnt	read-only access
sy-fdpos	read-only access
sy-index	read-only access
sy-subrc	read-only access
sy-tabix	read-only access

ABAP for Key Users - ABAP Keywords

ABAP Keyword	Subordinate Keyword
APPEND	
ASSIGN	
CASE	
CATCH	
CHECK	
CLEANUP	
CLEAR	
COLLECT	
CONCATENATE	
CONDENSE	
CONSTANTS	
CONTINUE	

ABAP Keyword	Subordinate Keyword
CONVERT	**Restricted to following keywords:** CONVERT-DATE-TIME CONVERT-TEXT CONVERT-TIME-STAMP
DATA	
DELETE	**Restricted to following keywords:** DELETE-ITAB
DO	
ELSE	
ELSEIF	
ENDCASE	
ENDDO	
ENDIF	
ENDLOOP	
ENDSELECT	
ENDTRY	
ENDWHILE	
EXIT	
EXPORT	**Restricted to following keywords:** EXPORT-DATA-BUFFER EXPORT-ITAB
FIELD-SYMBOLS	
FIND	
GET	**Restricted to following keywords:** GET-BIT GET-TIME-STAMP
IF	
IMPORT	**Restricted to following keywords:** IMPORT-DATA-BUFFER IMPORT-ITAB
INSERT	**Restricted to following keywords:** INSERT-ITAB

ABAP Keyword	Subordinate Keyword
LOOP	**Restricted to following keywords:** LOOP-ITAB
MESSAGE	**Restricted to following keywords:** MESSAGE-INTO
MODIFY	**Restricted to following keywords:** MODIFY-ITAB
MOVE-CORRESPONDING	
OVERLAY	
RAISE	**Restricted to following keywords:** RAISE-EXCEPTION
READ	**Restricted to following keywords:** READ-ITAB
REPLACE	
RESUME	
RETRY	
RETURN	
SELECT	
SET	**Restricted to following keywords:** SET-ASSOCIATION SET-BIT
SHIFT	
SORT	**Restricted to following keywords:** SORT-ITAB
SPLIT	
TRANSLATE	
TRY	
TYPES	
UNASSIGN	
WHEN	
WHILE	

Related Information

SAP S/4HANA Extensibility Tutorial

6.3.1.10.1.4 Keyboard Shortcuts in Web-Based Editor

You can use keyboard shortcuts while working in the web-based editor.

The following key combinations are provided in the web-based editor:

Key Combination	What Is It Used for?
`Ctrl` + `Space`	To turn on code completion
`Ctrl` + `/` `Ctrl` + `#`	To uncomment or comment out a code block
`Shift` + `F1`	To use pretty printer

6.3.1.10.2 Custom Logic (Deprecated)

With this app you can write your own implementations to extend SAP partner code.

> ⚠ Caution
>
> The management of data in an extension scenario deviates from the management of data in the standard scenarios. You are responsible for ensuring that the data used in an extension scenario is managed in accordance with any applicable legal requirements or business needs, such as data protection legislation or data life cycle requirements. Please note that the extensibility framework is currently not integrated in the privacy-by-default functionality. Therefore, the extensibility framework should not be used for the processing of personal data if this processing falls under any applicable data protection legislation.

Key Features

You can use this app to:

- Create your own implementation by using ABAP for key users
  - Implement your custom logic
  - Test your custom logic, for example with predefined test variants
  - Create filter conditions to define when an implementation is executed
- Publish implementation to your test system
- Edit implementation descriptions that have already been published
- Delete implementations

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component BC-SRV-APS-EXT-BL.

6.3.1.10.2.1 Creating Custom Logic

Get an overview of how to create custom logic, and how to publish your custom logic.

Business Benefit

This app enables you to create and maintain custom logic for business add-ins (BAdIs) that have been released by SAP partners.

Process Steps

This is a generic description about how to implement custom logic.

1. To find a Cloud BAdI, open the *Custom Logic* app.
2. Click + *(Create)* . Select a BAdI Description of the delivered BAdI you want to implement. Add an implementation description and an implementation ID.
3. Choose *Create*.
4. Now you can modify and test the draft logic using ABAP for key users in a web-based editor. You can click on *Example* to see a sample implementation of the BAdI. Choose the fields for testing on the *Available Fields* tab, enter the test data and click *Test* to check if your custom logic is working as expected.

   > **Note**
   >
   > Please make sure that any of your draft logic is syntactically correct and tested. Otherwise, the draft logic might be lost when your system is upgraded to another Cloud release.

5. To activate your BAdI implementation, choose *Publish*.

Related Information

Statements in ABAP for Key Users [page 3027]
Keyboard Shortcuts in Web-Based Editor [page 3030]
Extensibility [page 3018]

6.3.1.10.2.2 Statements in ABAP for Key Users

This topic provides you with an overview of ABAP keywords and ABAP system fields that are available with the ABAP version for key users.

You can search the tables for specific ABAP keywords and system fields that are available with the ABAP version for key users. If you need more information and examples about how to use the ABAP version for key users, you can refer to our tutorial blog linked under *Related Information*.

ABAP for Key Users - ABAP System Fields

ABAP System Field	Restriction
sy-dbcnt	read-only access
sy-fdpos	read-only access
sy-index	read-only access
sy-subrc	read-only access
sy-tabix	read-only access

ABAP for Key Users - ABAP Keywords

ABAP Keyword	Subordinate Keyword
APPEND	
ASSIGN	
CASE	
CATCH	
CHECK	
CLEANUP	
CLEAR	
COLLECT	
CONCATENATE	
CONDENSE	
CONSTANTS	
CONTINUE	

ABAP Keyword	Subordinate Keyword
CONVERT	**Restricted to following keywords:** CONVERT-DATE-TIME CONVERT-TEXT CONVERT-TIME-STAMP
DATA	
DELETE	**Restricted to following keywords:** DELETE-ITAB
DO	
ELSE	
ELSEIF	
ENDCASE	
ENDDO	
ENDIF	
ENDLOOP	
ENDSELECT	
ENDTRY	
ENDWHILE	
EXIT	
EXPORT	**Restricted to following keywords:** EXPORT-DATA-BUFFER EXPORT-ITAB
FIELD-SYMBOLS	
FIND	
GET	**Restricted to following keywords:** GET-BIT GET-TIME-STAMP
IF	
IMPORT	**Restricted to following keywords:** IMPORT-DATA-BUFFER IMPORT-ITAB
INSERT	**Restricted to following keywords:** INSERT-ITAB

ABAP Keyword	Subordinate Keyword
LOOP	**Restricted to following keywords:** LOOP-ITAB
MESSAGE	**Restricted to following keywords:** MESSAGE-INTO
MODIFY	**Restricted to following keywords:** MODIFY-ITAB
MOVE-CORRESPONDING	
OVERLAY	
RAISE	**Restricted to following keywords:** RAISE-EXCEPTION
READ	**Restricted to following keywords:** READ-ITAB
REPLACE	
RESUME	
RETRY	
RETURN	
SELECT	
SET	**Restricted to following keywords:** SET-ASSOCIATION SET-BIT
SHIFT	
SORT	**Restricted to following keywords:** SORT-ITAB
SPLIT	
TRANSLATE	
TRY	
TYPES	
UNASSIGN	
WHEN	
WHILE	

Related Information

SAP S/4HANA Extensibility Tutorial

6.3.1.10.2.3 Keyboard Shortcuts in Web-Based Editor

You can use keyboard shortcuts while working in the web-based editor.

The following key combinations are provided in the web-based editor:

Key Combination	What Is It Used for?
`Ctrl` + `Space`	To turn on code completion
`Ctrl` + `/` `Ctrl` + `#`	To uncomment or comment out a code block
`Shift` + `F1`	To use pretty printer

6.3.1.10.3 Configure Predefined Custom Fields

You, as a system administrator can use this app to configure your predefined custom fields to customize applications and their UIs.

Different companies and business processes require different document layouts or application functionalities. To address these specific needs, you can adapt predelivered fields accordingly. This app offers the option to configure predelivered custom fields in your apps to your needs.

Key Features

You can use this app to:

- Configure a predefined field
- Provide UI texts in other languages
- Enable field usage
- Edit fields that have already been published
- Discard changes you have done to a field that was already published, and thus reset all modifications to get back to the state of the last published version of a field with one click
- Delete new fields or fields that have already been published

> ⓘ Note
>
> The provided functionality of predefined custom fields does not provide the technical capabilities to support the collection, processing, and storage of personal data.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-EXT-FLD`.

Related Information

Configuring Predefined Custom Fields [page 3031]

6.3.1.10.3.1 Configuring Predefined Custom Fields

Get an overview of how to configure predefined custom fields, and how to publish your configured custom fields.

Context

This process enables you to configure custom fields that were predefined by your application provider. They can be used to enhance applications which contain APIs and data sources that have been released by your software provider to configure custom fields.

Procedure

1. Open the *Configure Predefined Custom Fields* app. You can open the app either directly from the SAP Fiori launchpad, or from an extensible application by clicking *+ (Add Custom Field)* in the UI adaptation mode of an application.

   > **Note**
   >
   > To use new fields from a predefined custom field in an SAP Fiori app, you have to restart the app and the UI adaptation mode. For more information about how to adapt SAP Fiori UIs during runtime,see Adapting SAP Fiori UIs at Runtime

2. Create a new predefined custom field configuration in the business process where it is needed. To do so, select a business object, a business object node, and the technical type of the configuration. Complete

this step by selecting the basic properties of this type. For example, if you selected the technical type `Character`, you need to choose one of the available field lengths your application provider has predefined for you.

3. Next, you can adapt further properties of your custom field. Enter a label and a tooltip, and choose a semantic type for your field. For example, if you selected the semantic type `Email Address`, you will be able to open your email client directly from the link.

   > ⚠ Caution
   >
   > Once created, the semantic type of the custom field can't be changed anymore.

   You can translate your field label and your field tooltip. To do so, select *Translation* in your custom field. Now, you can proceed to making your custom field available in other languages.

   > ⓘ Note
   >
   > If your field is of type `Code List`, you can also translate the individual list value descriptions that belong to that field.

4. Finally, you can decide whether you want to create and publish your configured field directly, or if you want to create and edit your custom field. By selecting *Create and Edit*, you will be able to define where and how your configured field should appear in the app before publishing it.
5. You can decide which code values to enable for your custom field by using the toggles displayed in the table. Also, you can choose whether you want to make your new fields visible to the UI and app services. After you have finished editing your custom field, select *Publish*.

   > ⚠ Caution
   >
   > Once published, you won't be able to delete the code values from the type `Code List` anymore. However, you can still enable or disable your code values.

   > ⓘ Note
   >
   > If a change is made to the custom field after publishing, the status of the custom field will change to *Revised*. The changes in your custom field will only be visible in the system after selecting *Publish* once more.
   >
   > You can only *Discard Changes* if the revised custom field hasn't been published yet. After publishing, you can still enable or disable specific code values in your custom field as described in step 5.
   >
   > If a custom field is published or deleted, the extended application has to restart. This may result in a prolonged loading time of the extended application when it is launched for the first time.

   > ⚠ Caution
   >
   > When you publish a field with a certain length, you can't decrease the field length later on.

   Your configured custom fields will now appear in your applications and their UIs.

6. There are applications that use remote OData services to fetch data. However, there might be extension fields which are not mapped in the standard service consumption model. You can also enable remote OData services and maintain the mapping between the internal field name and the field name in the remote OData service. To do this, you can either use an existing field or create a new one at the specific

extension point. Find the available service consumption model entities in the *Services* section and enable the respective one. This navigates to the field mapping page, where you can maintain the entity data model field name. That field name corresponds to the OData extension field name on the remote service side. Finally, save and publish the field.

Related Information

Configure Predefined Custom Fields [page 3030]

6.3.1.11 Factory Calendar

You can create and maintain holiday and factory calendars using the following apps:

- Maintain Holidays [page 3033]
- Maintain Holiday Calendars [page 3035]
- Maintain Factory Calendars [page 3037]

6.3.1.11.1 Maintain Holidays

You can use this app to display and maintain holidays according to your location, industry, or other requirements.

Key Features

You can use this app to:

- display existing holidays.
- edit existing holidays.
- create new holidays.
- delete holidays.

To maintain holidays, log on to the SAP Fiori launchpad in the *SAP Business Technology Platform (SAP BTP)* system and open the *Maintain Holidays* app.

> **Note**
>
> To start the Fiori apps, you must have the `SAP_BR_BPC_EXPERT` business role containing the `SAP_CA_BC_IC_LND_CAL_CA1_PC` business catalog assigned.

> ⚠ Caution
>
> The following HANA SQL functions are currently not supported:
>
> - `ADD_WORKINGDAYS`
> - `WORKDAYS_BETWEEN`

Displaying Existing Holidays

If you want to display existing holidays, choose one of the following options:

- Enter your search criteria in the header section and press *Go* to display a specific holiday.
- Choose a status from the *Editing Status* drop-down menu and press *Go* to display a list of holidays.

Click a holiday from the list. The holiday details screen opens.

Creating a New Holiday

You can create a new holiday. To do so, proceed as follows:

1. Press *Create*. The *Create* dialog box opens.
2. Enter a unique holiday ID and choose one of the following holiday types from the drop-down list:
   - Fixed date: make entries in the *Month* and *Day* fields.
   - Floating dates: press *Create* and choose a date from the drop-down list.
   - Distance to easter: make an entry for the *Distance to Easter Sunday* field.
   - Fixed day from date: make entries in the *Month*, *Day* and *Weekday* fields.
3. Fill all mandatory fields and press *Create*.

Editing Existing Holidays

You can edit an existing holiday. To do so, proceed as follows:

1. Press *Go* to display the list of existing holidays.
2. Click a holiday from the list. The holiday details screen opens.
3. Click *Edit* and make your changes in the relevant sections.
4. Press *Save* to save your entries.

Deleting Holidays

You can delete a holiday. To do so, proceed as follows:

1. Press *Go* to display the list of existing holidays.
2. Choose a holiday from the list by checking the checkbox next to the holiday calendar ID.
3. Press *Delete*.
4. In the *Delete* dialog box, press *Delete*.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-ASF-CAL`.

6.3.1.11.2 Maintain Holiday Calendars

You can use this app to display and maintain holiday calendars according to your location, industry, or other requirements.

Key Features

You can use this app to:

- display existing holiday calendars.
- edit existing holiday calendars.
- create new holiday calendars.
- delete holiday calendars.

To maintain holiday calendars, log on to the SAP Fiori launchpad in the *SAP Business Technology Platform (SAP BTP)* system and open the *Maintain Holiday Calendars* app.

> **Note**
>
> To start the Fiori apps, you must have the `SAP_BR_BPC_EXPERT` business role containing the `SAP_CA_BC_IC_LND_CAL_CA1_PC` business catalog assigned.

> ⚠ Caution
>
> The following HANA SQL functions are currently not supported:
>
> - `ADD_WORKINGDAYS`
> - `WORKDAYS_BETWEEN`

Displaying Existing Holiday Calendars

If you want to display existing holiday calendars, choose one of the following options:

- Enter your search criteria in the header section and press *Go* to display a specific calendar.
- Choose a status from the *Editing Status* drop-down menu and press *Go* to display a list of calendars.

Click a holiday calendar from the list to find more information, such as:

- Details
- Assignments
- Holiday Calendar Texts

Creating a New Holiday Calendar

You can create a new holiday calendar. To do so, proceed as follows:

1. Press *Create*. The *Create* dialog box opens.
2. Enter a unique holiday calendar ID (*Holiday. Cal. ID*) and press *Continue*.
3. Enter values for the *Valid from* and *Valid to* fields.
4. Add an assignment and a holiday calendar text. To do so, proceed as follows:
    1. To add an assignment in the *Assignment* section, press *Create*. Choose a holiday from the drop-down list and press *Continue*. Enter a *Valid To* date and press *Continue*. Enter a *Valid From* date and press *Continue*. Press *Apply*.
    2. To add a text in the *Holiday Calendar Texts* section, press *Create*. Choose a language key from the drop-down list and press *Continue*. Enter a description in the *Description Text* field and press *Apply*.
5. Press *Save*.

Editing Existing Holiday Calendars

You can edit an existing holiday calendar. To do so, proceed as follows:

1. Press *Go* to display the list of existing holiday calendars.
2. Click a holiday calendar from the list. The holiday calendar overview screen opens and the *Details*, *Assignments*, and *Holiday Calendar Texts* sections are displayed.
3. Click *Edit* and make your changes in the relevant sections.

4. Press *Save* to save your entries.

Deleting a Holiday Calendar

You can delete a holiday calendar. To do so, proceed as follows:

1. Press *Go* to display the list of existing holiday calendars.
2. Choose a holiday calendar from the list by checking the checkbox next to the holiday calendar ID.
3. Press *Delete*.
4. In the *Delete* dialog box, press *Delete*.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-ASF-CAL`.

6.3.1.11.3 Maintain Factory Calendars

You can use this app to display and maintain factory calendars according to your location, industry, or other requirements.

Key Features

You can use this app to:

- display existing factory calendars.
- edit existing factory calendars.
- create new factory calendars.
- delete factory calendars.

To maintain factory calendars, log on to the SAP Fiori launchpad in the *SAP Business Technology Platform (SAP BTP)* system and open the *Maintain Factory Calendars* app.

> **Note**
>
> The business catalog `SAP_CA_BC_IC_LND_CAL_CA1_PC` - Factory Calendar - Configuration needs to be assigned to your user to access and use the app.
>
> The business catalog is included in the business role template `SAP_BR_BPC_EXPERT`.

> **Note**
>
> When the communication arrangement for the **Factory Calendar Integration** scenario (`SAP_COM_0834`) is active and configured (Integrating Factory Calendar), the *Maintain Factory Calendars* app is switched to read-only.

> **Caution**
>
> The following HANA SQL functions are currently not supported:
>
> - `ADD_WORKINGDAYS`
> - `WORKDAYS_BETWEEN`

Displaying Existing Factory Calendars

If you want to display existing factory calendars, choose one of the following options:

- Enter your search criteria in the header section and press *Go* to display a specific calendar.
- Choose a status from the *Editing Status* drop-down menu and press *Go* to display a list of calendars.

Click a factory calendar from the list to find more information, such as:

- Details
- Exception Rules
- Factory Calendar Texts

Creating a New Factory Calendar

You can create a new factory calendar. To do so, proceed as follows:

1. Press *Create*. The *Create* dialog box opens.
2. Enter a unique factory calendar ID (*Fac. Cal. ID*) and press *Continue*.
3. Enter values for the *Valid from* and *Valid to* fields.
4. Press *Save*.

Editing Existing Factory Calendars

You can edit an existing factory calendar. To do so, proceed as follows:

1. Press *Go* to display the list of existing factory calendars.
2. Click a factory calendar from the list. The factory calendar overview screen opens and the *Details*, *Exception Rules*, and *Factory Calendar Texts* sections are displayed.
3. Click *Edit*.
    1. To add a factory exception rule in the *Exception Rule* section, press *Create*. Choose a language key from the drop-down list and press *Continue*. Enter a description in the *Description Text* field and press *Apply*.
    2. To add a new text in the *Factory Calendar Texts* section, press *Create*. Choose a language key from the drop-down list and press *Continue*. Enter a description in the *Description Text* field and press *Apply*.
4. Press *Save* to save your entries.

Deleting a Factory Calendar

You can delete a factory calendar. To do so, proceed as follows:

1. Press *Go* to display the list of existing factory calendars.
2. Choose a factory calendar from the list by checking the checkbox next to the factory calendar ID.
3. Press *Delete*.
4. In the *Delete* dialog box, press *Delete*.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-ASF-CAL`.

6.3.1.12 i18N Services

Language Configuration [page 3039]

6.3.1.12.1 Language Configuration

With this app you can configure the set of active languages that can be used by applications.

> **Note**
>
> *Active languages* refer to the set of languages available for user interaction within applications. By default, a fixed list of 41 languages is available via the value help for languages. This list can be customized using the *Language Configuration* app.

> **Note**
>
> *Logon Languages* refer to the set of languages available for logging into the system. This set is fixed and cannot be modified using the *Language Configuration* app.

Key Features

The available language set is divided into two parts: active languages and inactive languages.

- Active languages
  This list defines the set of languages that are available in the system and offered for user selection.
- Inactive languages
  This list defines the set of languages that are available in the system but are **not** offered for user selection.

Activities

If the list is displayed empty after starting the app, click button *Go* to refresh the view.

The first tab shows all currently active languages. You can remove one or more languages from this list by selecting them and then clicking the *inactivate* button.

The second tab shows all currently inactive languages. You can add one or more languages from this list to the list of active languages by selecting them and then clicking the *activate* button.

On both tabs you can use the fulltext *Search* function to find one or more languages in the list.

On both tabs you can filter the displayed languages by language code. To refresh the tabs display click button *Go*.

Supported Device Types

- Desktop
- Tablet
- Smartphone

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-I18`.

6.3.1.13 Identity and Access Management

Identity and Access Management (IAM) enables you to control user access to apps and specify what business users can do and see in the apps.

The main elements of IAM are business catalogs, business roles, and business users. The IAM apps secure the access to your solution based on these elements. For more information about how these elements interrelate, refer to Business Roles, Business Catalogs, App Authorization Variants, Restrictions.

Access to business apps is controlled by a role-based authorization management. That means you assign business roles to users and these business roles provide access to certain business tasks.

You can also assign business users to business roles in the **Maintain Business Roles** app. The main purpose of the app though is to create and adapt business roles. You create business roles by combining pre-defined business catalogs that contain the actual authorizations that allow users to access apps for a specific business area. If necessary, you can change the restrictions for the access categories value help, read, and write on field level. Once you have created a business role, you can assign it to multiple business users who perform similar business tasks in the **Maintain Business Users** app. Business users are all persons that need access to your solution, such as employees or contractors.

As opposed to business users, technical users aren't persons, but rather services that are used to automate technical tasks in the system, for example, a communication user. The **Display Technical Users** app shows you what technical users are available in your solution.

> ⓘ Note
>
> We recommend that you use the *Manage Business Role Changes After Upgrade* app as a starting point to carry out regular release activities.

> ⓘ Note
>
> To find out which business roles grant access to which apps, you can use the *IAM Information System* app. On the *Business Role - Application* tab, you can display all apps per business role as well as the corresponding business catalog IDs. If you select *Application* in the *Main Entity* dropdown list, you can search for apps and see which business roles and business catalogs provide access to them. For more information, refer to IAM Information System.
>
> If you don't have access to the *IAM Information System* app, you can still find out which business catalogs grant access to which apps using the SAP Fiori Apps Reference Library.

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

Access to the Apps

To have access to these apps, your user needs to be assigned to a business role which contains one of the following business catalogs:

- *User Management* (SAP_CORE_BC_IAM_UM)
- *Role Management* (SAP_CORE_BC_IAM_RM)
- *Role Assignment* (SAP_CORE_BC_IAM_RA)

For more information, see sections *Maintain Business Users* and *Business Catalogs for Identity and Access Management Apps* linked below or ask your administrator.

Related Information

Maintain Business Users [page 3044]
Display Technical Users [page 3053]
Business Catalogs [page 3092]
IAM Information System [page 3102]
Business Role Templates [page 3094]
Business Catalogs for IAM Administration Apps [page 3115]
Maintain Deleted Business Users [page 3051]
Manage Business Role Changes After Upgrade [page 3107]
IAM Key Figures [page 3103]
Display Authorization Trace [page 3104]
Display Restriction Types [page 3097]
Maintain Business Roles [page 3068]
Business Roles, Business Catalogs, App Authorization Variants, Restrictions [page 3060]
Manage Business Role Changes After Upgrade [page 3107]
IAM Information System [page 3102]
SAP Fiori Apps Reference Library - User Guide

6.3.1.13.1 Users

There are different categories of users for different purposes and with different capabilities. Business users represent end users performing day-to-day business tasks. They constitute personal data and their lifecycle is governed by retention and deletion policies.

Technical users are used in the background for system tasks such as inter-system communication, printing, and support.

Business Users

Relation of Employees and Business Users

Employees are metadata-records containing personal and employment-related data. Employees do not have access to the system directly, but only via corresponding business users.

A business user corresponds to a real-life person who works with the system interactively. It is a user with a business partner assignment (either an employee or a contingent worker). Business users are created to allow a person access to the system. You can maintain business users in the *Maintain Business Users* app. Authentication and logon for business users happens at the identity provider.

Business User Authorizations and Monitoring

You can assign business users to business roles in the *Maintain Business Roles* app. The main purpose of the app though is to create and adapt business roles.

Information about currently assigned business roles can be found in the *IAM Information System* app and the *IAM Key Figures* app.

Business User Lifecycle

Business users that haven't been used for a certain time can be automatically locked. For more information, see *How to Lock Unused Business Users* in the *Related Information* section.

Retention and ultimate deletion of blocked business users is governed by the Information Lifecycle Management (ILM) component.

> ⓘ Note
>
> Please note that ILM is not available in SAP BTP ABAP Environment.

Technical Users

Technical users correspond to a local or remote process which is typically part of the cloud management process (such as system provisioning, support) or intrinsic system processes. There are technical users that belong to the software or service provider and there are technical users that belong to the customer.

You can get an overview of all technical users in the *Display Technical Users* app.

Communication Users

The communication user is a technical user type aimed at technical communication. A communication user corresponds to a remote system that connects to your own system. You can define communication users in the *Maintain Communication Users* app.

Support Users

Support users are a separate type of technical user for temporary access. These users are intended for SAP employees who access the system based on a support ticket from the customer. The *Display Technical Users* app shows which support users were created for the system on which incident.

Related Information

Maintain Business Users [page 3044]
Maintain Business Roles [page 3068]
IAM Information System [page 3102]
IAM Key Figures [page 3103]
How to Lock Unused Business Users [page 3047]
Display Technical Users [page 3053]
Maintain Communication Users [page 2993]

6.3.1.13.1.1 Maintain Business Users

You use this app to provide business users with access rights and to maintain business user settings.

Business users gain access to Fiori apps through business roles. A business role can comprise one or several business catalogs which in turn comprise several apps.

Key Features

You can use this app to:

- Edit business user data
- Assign business roles to a user and remove existing assignments
- Update user role assignments
- Lock users so that they cannot log on to the system
- Unlock users
- Download list of users
- Set the language of the user interface for each business user

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

Related Information

Business Catalogs for IAM Administration Apps [page 3115]

6.3.1.13.1.1.1 How to Maintain Business Users

Context

You can change user data (for example, the user name) and regional settings (for example, the date and time format).

Procedure

- **Creating New Business Users**
- To create a new business user, proceed as follows:
  a. Choose *New* on the main screen of the app.
  b. Select the employee you want to create the user for, and fill in the required fields.

     For *User Name*, enter a name or ID that is identical with the login name of the same user in the on-premise identity provider.

     The name the user enters to log on to the SAP Fiori launchpad. The user name can only contain uppercase letters, numbers, underscores, periods and hyphens. It must have a maximum length of 40 characters and must not start with an underscore, or with SAP.
- **Assigning Roles to Business Users**
- To grant users access to applications, you assign the respective business roles to them.
  a. Select the required user.
  b. Click *Add Business Roles*.
  c. Search for the required roles and select them.
  d. Click *Apply* and *Save*.

     You can navigate from the Maintain Business Users app to the Maintain Business Roles app by choosing a business role ID in the list of assigned business roles.

     The user then sees the respective app tiles in the tile catalogs.
- **Removing Role Assignments from Business Users**
- To remove a role assignment from a user, proceed as follows:
  a. Select the business user and then the assigned business role.
  b. Choose *Remove* above the list or *Remove Business Roles* at the bottom of the screen.
- **Updating User Role Assignments**

- To update the assignments of roles to a user individually, edit the affected business user.
- To mass update user role assignments, upload a CSV file.

> **Note**
>
> The CSV file needs to be UTF-8-encoded and must comply with the following pattern:
>
> | User Name | User ID (Optional) | E-Mail (Optional) | Global User ID (Optional) | Role ID |
> |---|---|---|---|---|
> | <User name 1> | <Business user ID 1> | <Business user's mail address 1> | <Global user ID 1> | <Business role ID 1> |
> | <User name 2> | <Business user ID 2> | <Business user's mail address 2> | <Global user ID 2> | <Business role ID 2> |
> | <User name 3> | <Business user ID 3> | <Business user's mail address 3> | <Global user ID 3> | <Business role ID 3> |
> | <User name n> | <Business user ID n> | <Business user's mail address n> | <Global user ID n> | <Business role ID n> |
>
> In the app, you are provided with a CSV template that you can download and use for filling in the user role assignments. When uploading the CSV file, you can decide if you want to add the role assignments to a user, or if you want to overwrite them completely.
>
> You can find the CSV template as follows: Click the *Upload* button, then click *Upload User Role Assignments* in the menu that appears. This opens a dialog box containing the *Download CSV Template* link that you can use to download the template.
>
> The *Upload* button is only visible if your business user is assigned to the following IAM app:
>
> *Assign Business Users to Business Roles* (F1303_22_TRAN)
>
> The user ID is optional. Only if the user name is not unique, the user ID has to be maintained.

- **Downloading User Lists**
- You can download a list of all business users with an email address as CSV files and upload it to *SAP Cloud Identity Services - Identity Authentication*. To do so, proceed as follows:
  a. Open the *Maintain Business Users* app.
  b. Choose *Download*→ *Download for IDP (.csv)*.
  c. Save the generated CSV file locally. This file will be used for the user import in *Identity Authentication*.
  d. **Optional:** Adjust regional settings to view the CSV file correctly. To open the file, you have to set the list separator to **comma (,)** by setting the format in the regional settings of your operating system to *English (United States)*.
  e. Open *Identity Authentication* (IDP) system.
  f. Open the *Administration Console for SAP Cloud Identity Services – Identity Authentication*.
  g. Navigate to *User Management* > *Import Users* to upload the CSV file.

> **Note**
>
> Make sure that the CSV file is not opened with any program before the upload.

- **Changing the Default Settings for a Business User**
- To change the default settings for a business user, select it for editing and adjust the values as required. If you create a new business user, the following fields in the *User Data* area and the *Regional Settings* area are automatically filled with default values. You can adjust them if necessary.

Default Values

Field	Default Value
Valid From	01/01/1901
Valid To	12/31/9999
Decimal Format	This default value is derived from the standard setting that is valid in the country of the workplace address. If the workplace address is not maintained, the value is derived from the business user's private address.
Date Format	
Time Format	
Time Zone	
Language	

Related Information

Defaulting of User Attributes

6.3.1.13.1.1.2 How to Lock Unused Business Users

Lock inactive business users

Context

You can schedule a job to automatically lock business users who haven't logged on for a certain number of days.

Procedure

1. Open the *Schedule Lock for Unused Business Users* app.
2. Click *Create*.

3. Enter a job name and click *Step 2*.
4. Define the recurrence pattern of your job and click *Step 3*.
5. Under *Parameters*, enter the required number of days since the last logon you want to use for the automatic lock. Please note that business users that were unlocked within the entered number of days range are not getting locked. This prevents manually unlocked business users from being locked by the job again.
6. Add the business users you want to exclude.

   Carry out the following optional steps if required:

   1. Open the value help if you want to define further conditions.
      Go to the *Define Conditions* tab. Here you can define single users or whole user areas you want to exclude from the selection. You can enter multiple users at once by using copy and paste, for example to transfer users from lists.
      Please note that you can also restrict the list of users that should be processed by this job.
   2. Select *Lock User w/o Role Assignment* to also lock all business users that are not assigned to a business role.
   3. Select *Skip User with Appl. Jobs* to prevent the locking of business users with application jobs. By default, all business users with assigned application jobs are locked.
7. Click *Schedule*.

6.3.1.13.1.1.3 How to Make Mass Changes to Business Users

Context

You can use the mass change wizard to change multiple business users at once. This is helpful, for example, if you want to apply the same validity period to more than one business user.

Procedure

1. Select the business users you want to change and click *Mass Change*. The *Mass Change Wizard* appears. It has the following pattern:
2. Select the required attributes. You can make the following changes:

Area	Attributes
User Data	Select one or more attributes (such as timezone or language) you want to change in all selected business users.
Role Assignment	Add Business Roles Remove Business Roles

3. Apply the required attributes to your business users.
4. Review and confirm your changes.

6.3.1.13.1.2 Maintain Business User Groups

With this app you can create business user groups and assign multiple business users to them. This helps you to organize your area and easily search for all business users of a certain category (for example to assign them to business roles). Grouping also facilitates maintenance of authorizations. If you are the super administrator for all areas, you can delegate this task to administrators for the relevant areas, such as Financials. In this case, you would create a business user group for Financials.

Please note that the name of business user group is created automatically in namespace *ZCB*.

Key Features

You can use this app to:

- Create business user groups and assign the required business users to them
- Reassign or remove business users to or from business user groups
  Note: You can also use drag and drop to move business users from one group to another.
- Upload business user assignments
- Add business user groups to transports
- Export business user groups to a spreadsheet (including description and number of business users)

Prerequisites

The following restriction types (in the following business catalogs) are required for maintaining or displaying business role groups and business user groups:

Business Catalogs	Restriction Types
SAP_CORE_BC_IAM_RM	*Business Role* (S_BRL)
SAP_CORE_BC_IAM_UM	*Business User* (CLASS) *Business Role* (S_BRL)
SAP_CORE_BC_IAM_RA	*Business Role* (S_BRL) *Business Role User Assignment* (S_BRL_ASG) *Business User* (CLASS)
SAP_CORE_BC_IAM_RM_DISP_PC	*Business Role* (S_BRL)
SAP_CORE_BC_IAM_UM_DISP_PC	*Business User* (CLASS)

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

6.3.1.13.1.2.1 How to Import and Export Business User Groups

Context

You can use the *Export Software Collection* app and the *Import Collection* app to transport business user groups from your source system to your target system.

We recommend that you do not change the business user group after it has been transported. It should not be changed locally but only updated with a new transport. Local changes should only be carried out in urgent cases.

Procedure

1. In your source system, open the *Export Software Collection* app.
2. Create a software collection and give it a unique name.
3. Click *Add Items* and select the business user groups you want to export.
4. Click *Export* to export the required business user groups to your target system. You can view in the log if the export has been finished.
5. In your target system, open the *Import Collection* app.
6. Select the previously exported software collection and click *Import* to import the required business user groups to your target system. You can view in the log if the import has been finished.

Related Information

6.3.1.13.1.3 Maintain Deleted Business Users

With this app you can display details of deleted business users, such as the retention period, and allow or block their re-creation.

Prerequisites

To access the app, you need to have the `SAP_CORE_BC_IAM_UMD` business catalog assigned to your user.

Key Features

You can use this app to:

- Display a list of all deleted users including deletion time, end of re-creation period and re-creation status
- Resolve the identity of deleted business users by displaying contact details such as email address or phone number
- Change the re-creation status from *Allowed* to *Not Allowed* or the other way round.
- View system deletion checks to find out if a user can be permanently deleted or if it is still used or to check why blocked user data wasn't destroyed.

Supported Device Types

- Desktop
- Tablet
- Smartphone

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

6.3.1.13.1.3.1 How to Enable the Recreation of Blocked Business Users

Sometimes the person with whom a deleted business user was associated must access the system again.

Context

The system protects users in the blocking area from being recreated. The system prevents blocked business users from being recreated to prevent inconsistencies in the user data. You don't want the same business user associated with two different people. But there are cases where you want to recreate business users, which have been deleted. Perhaps a person who left your organization has come back.

> ⓘ Note
>
> If you have the authorization to allow the recreation of users and you are recreating a blocked user, the recreation is automatically allowed.

Procedure

1. In the app *Maintain Deleted Business Users*, select the business user you want to recreate.
2. Click *Allow Re-creation*.

Results

Use your standard processes for creating this business user again in the system. When the business user is recreated, the system removes the business user entry from the app *Maintain Deleted Business Users*.

> ⓘ Note
>
> The system does not transfer the address data from the app *Maintain Deleted Business Users*. Reenter the address information in your process for business user creation.

6.3.1.13.1.4 Display Technical Users

This app shows all technical users that exist in the system.

Purpose

This app has two tabs:

- Technical User
- SAP Support User Request Log

In the *Technical User* tab, you can display technical users that can be services that are used to automate technical tasks in the system, for example, a print queue user who pulls print jobs remotely. In addition, the service and support users of the software provider or hosting provider are technical users.

In the *SAP Support User Request Log* tab, you can display more information about when and why SAP support users accessed your customer system in the past 12 months. For each support user, the relevant incident ID, access level, access category, customer user, request date and validity date is displayed when you click on the required entry in the *Users* list. SAP support user IDs are pseudonymized to respect the data subject rights of SAP employees according to GDPR.

Key Features

You can use this app to:

- Lock or unlock the following types of technical users: Print users and communication users
- Change the user name and password of some types of technical users
- Display details of which support users were created for the system
- Display all technical users in the system
- Export user data (such as *User ID* or *User Group*) to a spreadsheet if required

> **Note**
>
> To lock and unlock communication users, use the *Maintain Communication Users* app. See section Related Information.

All users are assigned to one of the following user groups:

User Groups

Group	Explanation
SAP Technical Users	These users are predefined by SAP to run tasks in the system for cloud operation activities and automated processes. SAP technical users are also required for customer-specific tasks, such as running and scheduling jobs. The required job template contains the SAP technical user needed to run the job by default.
Communication Users	In this group, all communication users used in communication arrangements that are managed by customers are displayed. Communication users are technical users required for communication between your system and any integrated system. The following types of communication users are available: • SAP communication users: Predefined by SAP • Communication users: Created by customers Custom communication users are created in the *Maintain Communication Users* app and are assigned to systems for inbound and outbound communication. These technical users either use basic authentication or certificates to communicate between your system and the integrated system. The communication system, together with a communication scenario, is assigned to a communication arrangement. Thus, all communication scenario roles are assigned to the communication user which provides this user with all permission required for the connection between your system and the integrated system.
Support Users	SAP uses support users to provide customers with support if issues occur in the system. These users only exist temporarily and are deleted by SAP after the support process is completed.

Supported Device Types

- Desktop
- Tablet

SAP Technical Users

Group	Explanation
SAP*	This user has always been used in on-premise systems to log on to newly created tenants. This user is locked in your tenant and it'sn't used for any system operation.
SAP_CUST_BUS	This user is a reference user assigned to all customer business users and SAP support users. General authorizations required for basic system access are assigned through this reference (for example to access SAP Fiori launchpad). The user can't be used by itself. It's locked and has no logon credentials.
SAP_LMADM	This user is used for system internal, automated software lifecycle management across all tenants. It's used in processes that need to update tenant-specific data after changes to tenant-independent data. This user is unlocked. External system logon isn't possible.
SAP_SMTP_IN	This user is used as the SMTP mail server user.
SAP_SPC	This user is used for SAP-internal automated system operation.
SAP_SYSTEM	This user is the default user to run system-internal technical jobs.
SAP_WFRT	This user is used by SAP Business Workflow for the automated background processing of work items.
SAP_WSRT	This user is used for system internal processing in the Web service runtime.

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

Related Information

Business Catalogs for IAM Administration Apps [page 3115]
Maintain Communication Users [page 2993]
Access Levels [page 3056]
Access Categories [page 3058]

6.3.1.13.1.4.1 SAP Support User Request Log

The *SAP Support User Request Log* is part of the *Display Technical Users* app.

SAP personnel need access to customer systems for support and operations. Customers require details about this access for audits and the fiscal year-end closing process. In the *Display Technical Users* app, you can display more information about when and why SAP support users accessed your customer system in the past 12 months. For each support user, the relevant incident ID, access level, access category, customer user, request date and validity date is displayed when you choose the desired entry in the *Users* list. SAP support user IDs are pseudonymized to respect the data subject rights of SAP employees according to GDPR.

> ### Note
>
> The SAP Support User Request Log is also evaluable with the CDS view entity `I_IAMSupportUserRequestLog`.

Related Information

Access Levels [page 3056]
Access Categories [page 3058]

6.3.1.13.1.4.1.1 Access Levels

Authorizations of SAP support users

Access levels define the authorizations of SAP support users when accessing customer systems.

Access Levels

Name	Use
SUPPORT_DEFAULT (Platform Analysis, Display Configuration, No Business Authorization)	The *SUPPORT_DEFAULT* access level provides display access to ABAP Platform transactions, display of Customizing tables and views, display of system tables with content delivered by SAP or transported in customer systems. It also provides authorization to display, activate and deactivate transient logs and traces without access to payload data. It grants authorization for the debugging of the own user and external debugging of other users. Authorization to change values of variables or to alter the code flow in the debugger is not granted. In case business authorizations are required for support, they are agreed in customer support cases and are copied from a specific customer business user or customer communication user as additional authorization.

Name	Use
SUPPORT_DEFAULT_APP (Platform Analysis, Display Configuration, Display Business Data)	The *SUPPORT_DEFAULT_APP* access level inherits all the authorizations from the *SUPPORT_DEFAULT* access level. Additionally, it grants display access to application tables and selected business transactions that enable analysis. It includes display of most business application logs as well as display, activation and deactivation of transient traces with access to payload data. Authorization to change customer data is not granted.
SUPPORT_EXTENDED (Platform Analysis, Limited Administration, Full Business Authorization)	The *SUPPORT_EXTENDED* access level inherits all the authorizations from the *SUPPORT_DEFAULT_APP* access level. Additionally, it grants full business application authorization. It includes all backend transactions and reports of the application layer. It grants limited ABAP Platform administration authorization, such as deletion of caches. Authorization to change system configuration, Customizing data, or development objects is not granted.
SUPPORT_CUSTOMIZING (Limited Platform Customizing, Full Business Customizing)	The *SUPPORT_CUSTOMIZING* access level inherits all the authorizations from the *SUPPORT_DEFAULT_APP* access level. Additionally, it grants authorization to maintain selected ABAP Platform Customizing as well as unrestricted business application Customizing.
SUPPORT_CONTENT_ACT (Content Activation)	The *SUPPORT_CONTENT_ACT* access level inherits all the authorizations from the *SUPPORT_EXTENDED* and the *SUPPORT_CUSTOMIZING* access levels. Additionally, it grants administrative authorization to the content framework which manages the Customizing content lifecycle.
SUPPORT_DEVELOP_LOCAL (Local Development)	The *SUPPORT_DEVELOP_LOCAL* access level inherits all the authorizations from the *SUPPORT_EXTENDED* access level. Additionally, it grants authorization to create and execute local development objects. This access level may need to be used to support frameworks that generate local development objects in the customer name space (such as extensibility or data migration). It allows to test execute all function modules and static methods. It also allows changing field values and altering the code flow in the debugger. Authorizations to modify development objects delivered by SAP are not granted.

Name	Use
SUPPORT_DEVELOP (Unrestricted Development)	The *SUPPORT_DEVELOP* access level inherits all the authorizations from the *SUPPORT_DEVELOP_LOCAL* access level. Additionally, it grants authorization to modify development objects delivered by SAP. Standard procedure for such changes is a hotfix or an emergency patch. This access level may only be used in emergency situations where that process is not applicable.
SUPPORT_USER_ADMIN (User and Role Administration)	The *SUPPORT_USER_ADMIN* access level inherits all the authorizations from the *SUPPORT_DEFAULT* access level. Additionally, it grants authorization for local user and role administration. This access level may be used for support cases related to business user or communication user management.
SUPPORT_SYSTEM_CONFIG (System Configuration)	The *SUPPORT_SYSTEM_CONFIG* access level inherits all the authorizations from the *SUPPORT_EXTENDED* access level. Additionally, it grants authorization to manually configure SAP managed communication scenarios. Automatic setup is mandatory for such communication scenarios in customer systems. This access level is used in situations where automatic setup failed.
SUPPORT_SYSTEM_ADMIN (Unrestricted System Administration)	The *SUPPORT_SYSTEM_ADMIN* access level grants unrestricted administrative access. It includes authorization profile SAP_ALL. Cloud operations may need to use it for system lifecycle management. Support teams may need to request it as the final escalation of privileges for support cases that cannot be resolved with other access levels. Root case analysis is mandated for such support cases. The root cause must be resolved.

Related Information

SAP Support User Request Log [page 3056]
Access Categories [page 3058]

6.3.1.13.1.4.1.2 Access Categories

Access categories classify the purpose of SAP support user access to customer systems.

Access Categories

Name	Use
Operations	The *Operations* access category is used for system access by central cloud operations teams for system lifecycle management, reactive system management and proactive system management. Such access is not based on individual support cases. Taking over authorization from customer business users or customer communication users is not possible.
Customer Support	The *Customer Support* access category is used for system access based on support cases raised by the customer. The purpose of the system access is documented in a customer support case and can be reviewed in customer service management. System access is restricted to a processor of a customer case that is not completed. Support cases created for productive systems also allow access to non-productive systems. Support cases created for non-productive systems do not allow system access to productive systems. System access to a productive system requires a case for that specific system. Customers may allow the processor to copy authorizations from one specific customer business user or customer communication user. Consent to that is documented in the customer support case. The processor of the case can then copy these authorizations in addition to a support access level. The *SUPPORT_DEFAULT* and *SUPPORT_DEFAULT_APP* support access levels are automatically approved for processors of customer support cases to access customer systems. The *SUPPORT_EXTENDED* support access level is also automatically approved for processors of customer support cases to access non-productive customer systems. To access productive customer systems using the *SUPPORT_EXTENDED* support access level, an SAP internal approval process is in place for SAP S/4HANA Cloud as well as SAP BTP ABAP environment. Any requests for higher privileged support access levels in customer systems enforce the principle of dual control using SAP internal approval processes.
Health Check Support	The *Health Check Support* access category is used for system access based on support incidents raised internally by automatic health checks. The purpose of the system access is documented by SAP internally via the incidents. Procedures for approval of access levels match the procedures of the *Customer Support* access category. The same is true for restrictions of access to productive systems versus non-productive systems.

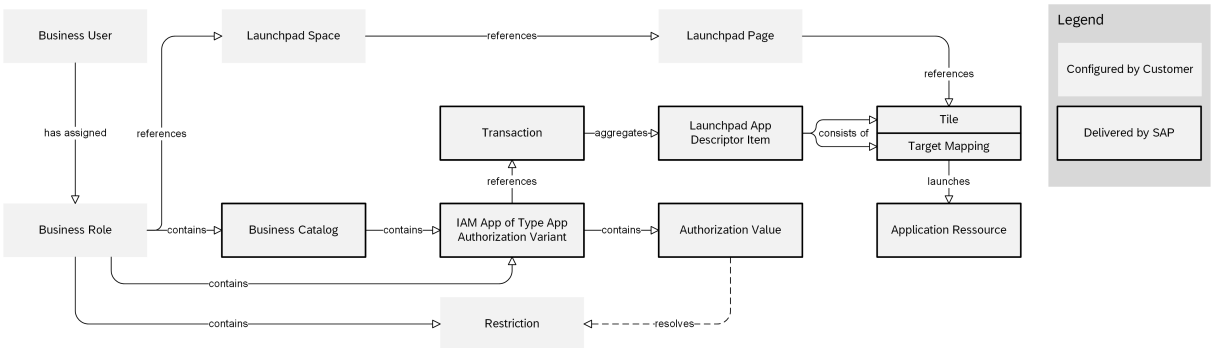
Name	Use
Emergency Support	Support user access is controlled by a platform for central cloud management. The *Emergency Support* access category is used for locally created support users in situations where the connection to the central cloud management is interrupted. The purpose of the emergency support access is documented in a customer support case and can be reviewed in customer service management.
SAP Internal Support	The *SAP Internal Support* access category is used for system access to SAP internal test or demo systems. It is not intended for system access to customer systems. We recommend that you create an incident for cloud operations to validate the configuration of support user access if there is any such access in customer systems.

Related Information

SAP Support User Request Log [page 3056]
Access Levels [page 3056]

6.3.1.13.2 Business Roles, Business Catalogs, App Authorization Variants, Restrictions

SAP divides the business functionality into semantically meaningful business catalogs, representing tasks or sub-processes within a business process. Business catalogs grant access to an app, a set of apps, or individual aspects of an app. Some business catalogs have restrictions. These restrictions give you the option of further specifying the way the user might interact with the app, for example, they may grant write or read access.



Business catalogs are grouped into collections called business roles. A business role generally contains multiple business catalogs and corresponds to a set of authorizations required to perform the tasks of a particular job description, for example, a warehouse clerk. On the business role level, restriction values of the

contained business catalogs are defined. A business catalog might be contained in different business roles and might have different restriction values assigned in these different business roles.

Launchpad spaces and pages structure the layout of your SAP Fiori launchpad. Tiles can be referenced in launchpad pages. Pages are grouped into launchpad spaces. You can assign launchpad spaces to business roles so that the launchpad space is visible to all users with a given business role.

Business roles are assigned to business users. Multiple business roles might be assigned to a single business user. And a business role might of course be assigned to different business users.

To facilitate the process of business role creation, SAP delivers business role templates. These templates are collections of business catalogs suitable for a particular user persona. You can create a business role by simply copying the business role template and specifying restriction values.

> ⓘ Note
>
> It is not recommended to use copies of business role templates for productive use.

IAM Apps - App Authorization Variant

IAM apps of the app authorization variant type (app authorization variants) are the smallest entity for business role design and grant you flexibility to manage authorizations for business users.

> ⚠ Caution
>
> IAM apps of the app authorization variant type are not to be confused with app variants. An app variant is an adapted version of an original SAP Fiori app. A key user can adapt an original app in UI adaptation mode and then save it as an app variant to be used by other users. For more information, see *App Variants: All You Need to Know* in the *Related Information* section.
>
> IAM apps of the app authorization variant type should be differentiated from IAM apps used in developer extensibility. You can use the latter to define necessary authorizations for business users for one or more services created in ABAP Development Tools (ADT). For more information, see *Authorization Basics* in the *Related Information* section.

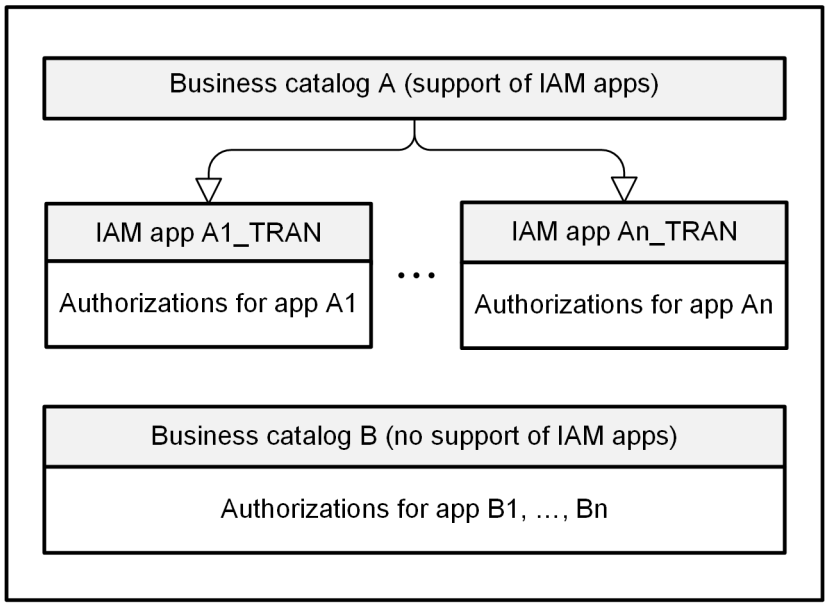
Concept

Business catalogs are a well-established concept to pre-bundle authorizations. They combine authorizations for business functionality (selection of apps) into semantically meaningful building blocks as best practices delivered by SAP that represent tasks or sub-processes for a certain workplace within an overall business process. With the introduction of app authorization variants, business catalogs change from indivisible entities into entities that bundle app authorization variants that can be activated or deactivated. Consequently, app authorization variants become the smallest entity for business role design, providing greater flexibility to manage authorizations for business users including which apps they can use.

> ⓘ Note
>
> To check whether a business catalog supports IAM apps, open the *Business Catalogs* app and use the *Supports IAM Apps* filter.

The graphic shows how authorizations for business users are managed differently depending on whether the business catalog supports IAM apps. Business catalogs that support IAM apps bundle app authorization

variants. After these business catalogs have been assigned to a business role the referenced IAM apps can be activated or deactivated. If business catalogs do not yet support IAM apps or the app authorization variants feature is not activated, business catalogs yet remain the smallest entity for business role design.



Components for Customer Incidents

If you need support or experience issues, please report an incident under the respective component:

- If you experience issues with any app used to administrate IAM-related tasks, report an incident under component `BC-SRV-APS-IAM`.
- If you experience issues with a business catalog, open the *Business Catalogs* app, select the business catalog and find the assigned component in the header section.
- If you experience issues with an IAM app, open the *Display IAM Apps* app, select the IAM app, go to the *General* tab, and find the assigned component.

Implementation Considerations

During the implementation phase of an IAM concept, the following considerations might be useful. After implementation, mechanisms for monitoring and analysis become more important.

The lifecycle of IAM elements is influenced by system upgrades. For more information, see *Manage Changes and Deprecations After Upgrade* in the *Related Information* section.

For more information about monitoring of the IAM configuration, see *Monitoring and Analysis* in the *Related Information* section.

Business Role Implementation

If you want to model business roles sharing common properties, you can make use of the inheritance mechanism of leading and derived business roles.

In the context of inheritance, you can maintain the intended level on which a field should be filled in a business role using the field maintenance status.

Transport

Changes to business roles should not be carried out in production systems directly.

We recommend that you define business roles in a development system and transport them into the production system instead of making local changes.

Related Information

Business Catalogs
Maintain Business Roles
Work with IAM Apps of the App Authorization Variant Type
App Variants: All You Need to Know
Authorization Basics
Manage Changes and Deprecations After Upgrade [page 3106]
Monitoring and Analysis [page 3101]

6.3.1.13.2.1 Work with IAM Apps (App Authorization Variants)

Learn how to use IAM apps of the app authorization variant type to manage authorizations as an administrator. Discover the impact of IAM apps on the end-user experience.

> ⚠ Caution
>
> IAM apps of the app authorization variant type are not to be confused with app variants. An app variant is an adapted version of an original SAP Fiori app. A key user can adapt an original app in UI adaptation mode and then save it as an app variant to be used by other users. For more information, refer to App Variants: All You Need to Know.
>
> IAM apps of the app authorization variant type should be differentiated from IAM apps used in developer extensibility. You can use the latter to define necessary authorizations for business users for one or more services created in ABAP Development Tools (ADT). For more information, refer to Authorization Basics.

6.3.1.13.2.1.1 Administrator

Understand the architecture and app views on authorization management for business users. As an administrator, this will help you work with IAM apps of the app authorization variant type.

Architecture View

The graphic provides an architecture view on the entities involved in authorization management and the relationships between them using cardinalities to indicate how many instances of one entity relate to instances of another (for example, one-to-one or one-to-many).

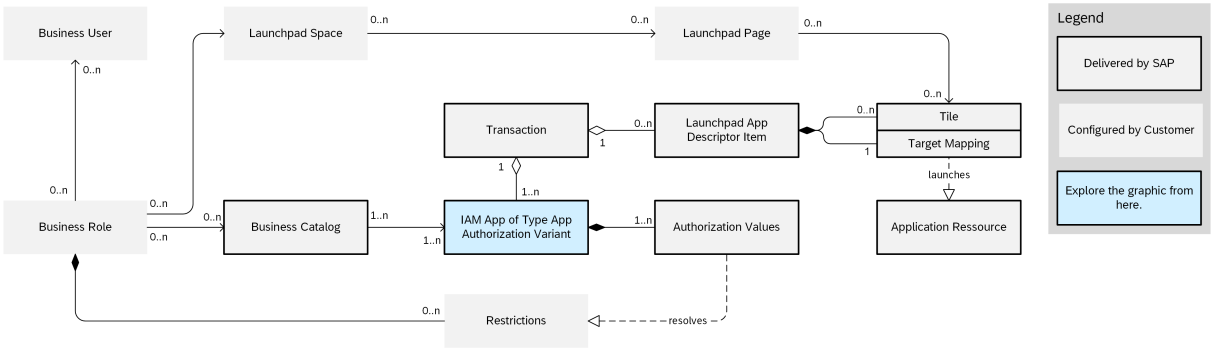
Generally, IAM apps are units that grant access to a set of services and authorizations. IAM apps of the app authorization variant type are IAM apps based on transactions containing services, authorizations, restriction types, and dependencies.

A transaction is an entity in the SAP system that represents an application or an executing program. Each transaction is identified by a transaction code, which is a technical ID used to call or identify the transaction. For SAP Fiori apps, for example, the transaction code equals the SAP Fiori ID.

Each transaction can be associated with multiple launchpad app descriptor items. A launchpad app descriptor item consists of a target mapping and one or more tiles. Apps need a launchpad app descriptor item to be accessed from SAP Fiori launchpad.

Target mappings are a prerequisite for navigating to an app in the SAP Fiori launchpad. Tiles serve as the visual representation of an app. It is possible that only a target mapping is provided if the app is not launched from a tile and only from another app (app-to-app navigation). Tiles can be referenced in SAP Fiori launchpad pages. Pages are grouped into SAP Fiori launchpad spaces that are then assigned to business roles.

A transaction can relate to several IAM apps that define different access rights through the defined authorization values, for example, to differentiate read and write access. IAM apps are bundled in business catalogs and provide sets of authorization values. Business catalogs can be assigned to business roles. The related restrictions become available for the business roles through the authorization values of the active IAM apps contained in the business role. Business roles are then assigned to a business user. The business user can access the related tiles and target mappings of the associated transactions from the active IAM apps available through the business role.



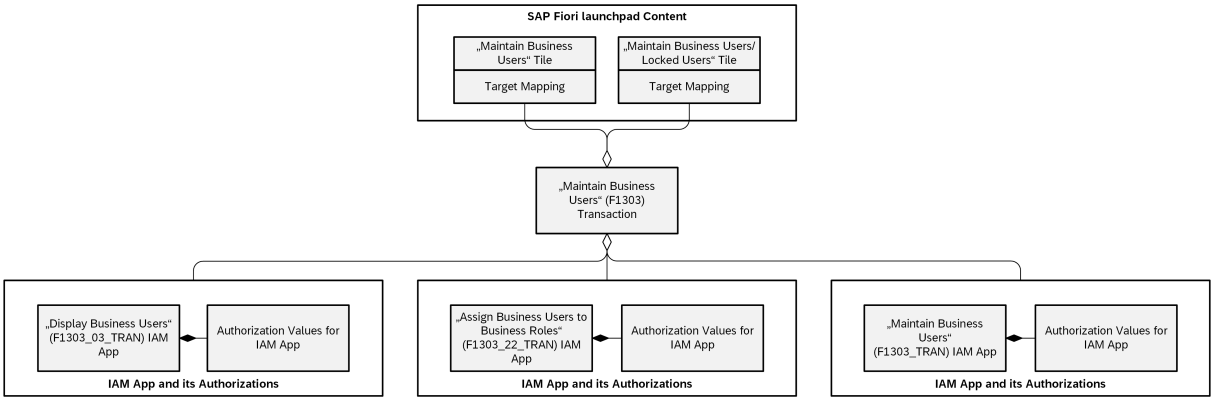
Let's focus more on the relationship between transactions and app authorization variants as illustrated in the following graphic using an example. An app authorization variant relates to one transaction, but a transaction can relate to several app authorization variants to define different access rights. The different app authorization variants grant access to the same SAP Fiori launchpad content through the transaction.

Example

The *Maintain Business Users* app has the Fiori ID `F1303` and, for example, the following app authorization variants:

- an app authorization variant called *Display Business Users* (IAM app ID: `F1303_03_TRAN`) with read authorization
- an app authorization variant called *Assign Business Users to Business Roles* (IAM app ID: `F1303_22_TRAN`) with write authorization
- an app authorization variant called *Maintain Business Users* (IAM app ID: `F1303_TRAN`) with write authorization

All three app authorization variants belong to the transaction code `F1303`. They grant access to the same tiles and target mappings in SAP Fiori launchpad but have separate IAM app IDs with the prefix `F1303`. If you only want to grant read access to business users assigned to a specific business role, for example, you activate the *Display Business Users* app authorization variant (`F1303_03_TRAN`).



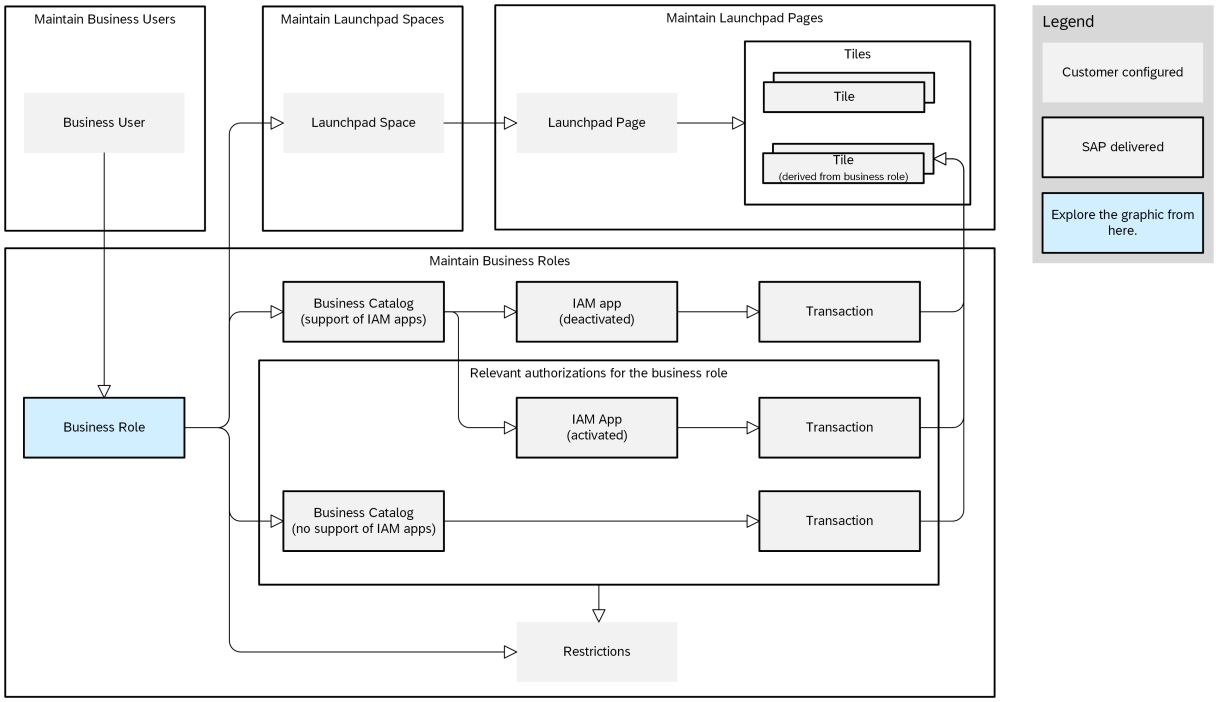
App View

In contrast to the architecture view on authorization management, the following graphic highlights the apps you can use as an administrator to manage authorizations for business roles using app authorization variants.

Use the *Maintain Business Users* app to provide business users with access rights to SAP Fiori apps through business roles.

Use the *Maintain Business Roles* app to define business roles that control the access to your applications by combining predefined business catalogs. If the business catalogs support IAM apps, they bundle app authorization variants. After such business catalogs are assigned to a business role, you have the option to deactivate individual IAM apps according to your business requirements. This allows you to assign only the apps that are required for a business role to it giving you the flexibility to manage authorizations for business users on a more granular level. For more information, refer to How to Activate or Deactivate IAM Apps (App Authorization Variants) [page 3089].

Use the *Maintain Launchpad Spaces* and *Maintain Launchpad Pages* apps to define the SAP Fiori launchpad of the business user. In the *Maintain Launchpad Pages* app you can display all tiles that are referenced by transactions and available through the corresponding business role for a specific page using the role context feature. The activation status of IAM apps is not considered for this feature.



Related Information

Managing Launchpad Spaces and Pages

Maintain Business Users

Maintain Business Roles

Manage Launchpad Spaces

Manage Launchpad Pages

Personas

6.3.1.13.2.1.2 End User

Business roles assigned to your business user define the access to apps and the layout of your SAP Fiori launchpad (spaces, pages, tiles) through provided authorizations. There are two main effects for you as an end user:

- To manage authorizations more granularly, administrators can deactivate IAM apps in business roles assigned to your business user. Authorizations for deactivated IAM apps assigned to a business role are removed. This means that you no longer have access to apps you previously accessed by default.
- An app can have multiple tiles as access points on SAP Fiori launchpad. This can be the case if tiles have been provided with different parameters to display different information in the app.
  All tiles for apps you are authorized to access are available in the app finder. On an SAP Fiori launchpad page, tiles are visible based on the authorization of your business user and based on the tiles the administrator has selected during the creation of the SAP Fiori launchpad page.

Related Information

App Finder

Personas

6.3.1.13.2.2 Effects of Using IAM Apps (App Authorization Variants)

When you work with IAM apps of the app authorization variant type, it affects how you use the IAM administration apps to manage IAM-related tasks and authorizations for business users.

The following apps contain features you can use to work with IAM apps:

App	Feature
Display IAM Apps	Display IAM apps for all supported IAM app types. Keep an overview of the assigned business roles, deprecated and non-deprecated IAM apps, their successors, and more.
IAM Information System	Discover all the details about IAM apps, including the business roles they're assigned to, the business catalogs they're bundled in, and any restrictions defined for them. You can also find information on successor IAM apps for both deprecated and non-deprecated IAM apps. This helps with early app replacement and deprecation planning, among other things.
Business Role Templates	The *IAM App* tab is available displaying all assigned IAM apps per business role template.
Business Catalogs	On the *Applications* tab, you can view all assigned IAM apps if the *Supports IAM Apps* checkbox is selected in the corresponding business catalog. For business catalogs that do not support IAM apps, the tile and target mapping title is displayed.
Display Restriction Types	On the *Used in IAM Apps* tab, you can view all IAM apps in which the corresponding restriction type is used.
Manage Launchpad Pages	You can display all tiles referenced by transactions and available through the corresponding business roles for specific pages using the role context feature. This feature doesn't consider the activation status of IAM apps.
Manage Business Role Changes After Upgrade	How To View and Manage Changed IAM Apps After an Upgrade [page 3113]
Maintain Business Roles	Effects on the Maintain Business Roles App [page 3088]

Additionally, be aware of the following effects when working with IAM apps:

- **Transaction Authorizations in Business Catalogs with and without IAM App Support**
  Transactions and their authorizations can be referenced in two types of business catalogs: those that support IAM apps and a small number that do not. If both types of business catalogs are assigned to a business role, the business catalog without IAM app support determines the authorizations for that business role. For these few non-migrated business catalogs, the corresponding tiles are not available on their own. However, if they are combined with migrated business catalogs that include the same transaction, the tiles become available. Still, these tiles do not appear in the app finder for the non-migrated business catalogs.

  > ### Example
  >
  > Let's picture the following scenario: A transaction is assigned to both types of business catalogs and both types of business catalogs are assigned to a business role. Even if the IAM app is deactivated in the business catalog that supports IAM apps, the transaction authorizations remain included because of the business catalog that doesn't support IAM apps and their deactivation.

- **IAM App Deactivation Limitations in Dependent Business Catalogs**
  If a business catalog that supports IAM apps has a dependent business catalog that does not support IAM apps, it is not possible to deactivate IAM apps in the dependent business catalog.
- **Dependent IAM Apps**
  Not all business catalogs currently support IAM apps. As a result, some dependent IAM apps are not maintained for those that have already been delivered.
- **Tiles in App Finder**
  For more information, refer to End User.

Related Information

Display IAM Apps
IAM Information System
Business Role Templates
Business Catalogs
Display Restriction Types
Manage Launchpad Pages
SAP Community Blog Post – Precise Control of Fiori App Access with the App Authorization Variants

6.3.1.13.2.3 Maintain Business Roles

You can use this app to create and edit business roles, add business catalogs to the roles, and maintain access restrictions. If the assigned business catalogs support IAM apps, the corresponding apps are assigned too. You can also directly assign IAM apps (for example app authorization variants) to the business role if you require more flexible authorization assignment.

With the *Maintain Business Roles* app you define business roles by assigning business catalogs and, if necessary, define value help, read and write access by maintaining the allowed values for fields. You use business roles to control the access to your applications. The predefined catalogs contain the actual

authorizations via the assigned IAM apps that allow users to access apps and allow to define instance-based restrictions where necessary. Business catalogs bundle authorizations for a specific business area via the assigned IAM apps. Once you have created a business role, you can assign it to multiple business users who perform similar business tasks.

Key Features

You can use this app to:

- Create business roles from scratch (based on selected business catalogs)
- Create business roles from a template delivered by SAP
- Download business roles from the test system and then upload them to the productive system per `XML` file
- Assign one or more business roles to a business user
- Maintain restrictions
- Display the *Restrictions Overview*
- Activate or deactivate IAM apps that are assigned via business catalogs
- Add or remove IAM apps that are directly assigned

  > **Note**
  >
  > Direct IAM app assignment is only possible for IAM apps of the *App Authorization Variant* type.

- Copy business roles
- Delete business roles

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

Related Information

Maintain Restrictions UI [page 3078]
Restrictions Overview [page 3080]

6.3.1.13.2.3.1 Best Practices for Business Roles

We recommend the following procedures in connection with business roles:

- Adhere to the 'principle of least privilege' when you assign business users to business roles.
- We recommend that you design your business role in a particularly specific way so that only a limited number of business users can be assigned to them. This means that only one business role should be available per specifically designed job or work environment. For example a finance developer role that is intended for about 35 developers or an IAM developer role that is intended for about 30 developers.
- We recommend assigning a business user to no more than five business roles, though ideally just to one.
- Do not edit a business role in the target system after the transport. We recommend that you only edit business roles in the source system before the tranport.
- Do not rename business roles (or change the role ID) after transport, as this is handled as deleting the old role and creating a new one in the target system. Existing business user assignments will be lost.
- Choose *Unrestricted ('*')* instead of selecting all restriction field values if the assigned values exceed a manageable limit.
- Not more than 100 business catalogs should be assigned to a business role.

Related Information

How to Define Authorizations Based on Restrictions [page 3075]

6.3.1.13.2.3.2 How to Create a New Business Role

Get an overview of how to create a new business role from scratch based on selected business catalogs.

Context

You use business roles to control the access to your applications. To create a business role, you add one or multiple business catalogs to it. These predefined catalogs contain the actual authorizations that allow users to access apps and allow to define instance based restrictions where necessary. Business catalogs bundle authorizations for a specific business area. Once you have created a business role, you can assign it to multiple business users who perform similar business tasks.

> **Note**
>
> Please note that this document describes how to create a business role without a template. For more information on how to create a business role from a template, see the *Related information* section.

Procedure

1. Open the *Maintain Business Roles* app.
2. Choose *New*.
3. Add general role details, such as business role name, ID and description.
4. On the *Business Catalogs* tab, select *Add* to add the required business catalogs. Select the business catalogs according to the business activities that the users with this role need to perform. Choose *Apply*.

   > **Note**
   >
   > Some business catalogs require additional dependent catalogs to be assigned to enable access to associated master data information (for example, for business partners or customers). These additionally required catalogs ensure access to all business objects used with the SAP Fiori apps of the main catalog. When you select the business catalogs you want to add to the business role and click *Add*, a list of dependent business catalogs is displayed. You can then select all the dependencies you want to add.

5. By default, the value help and read access for each business catalog is set to unrestricted and there is no write access. If you want to change these restrictions, choose *Maintain Restrictions*.
6. Maintain instance-based restrictions for all required business objects (following the requirements of your local authorization concept).
7. If required, choose *Launchpad Space* to assign a space to the business role.
8. On the *Business Users* tab, you can assign the business users to your new business role. These users will receive the authorizations as defined in the business role.
9. To activate the business role, save it.

   > **Note**
   >
   > If you go back to the business roles overview **without saving** the business role, the business role will automatically be saved in a draft status. You can access it again and edit it from the business roles overview. Before saving the role, consider double-checking the following document Best Practices for Business Roles [page 3070].

Related Information

Step by Step: Create a New Space and Page for a Business Role
How to Create a Business Role from a Template [page 3072]

6.3.1.13.2.3.3 How to Create a Business Role from a Template

Get an overview of how to create a business role from a template.

Context

Instead of creating a business role from scratch, you can also create it from a business role template.

> ⓘ Note
>
> A business role template is defined by SAP to make it easier for you to find the business catalogs that might be relevant for the corresponding role in your company. Usually, the business role templates have a very broad job profile to show all options. SAP does not recommend using them in their full scope because the business catalogs might conflict with the roles in your company. Instead, adjust them to the tasks of the role in your company by choosing only the relevant business catalogs.

> ⚠ Caution
>
> Business role templates delivered by SAP are intentionally broad and meant for testing. Before productive use, business roles based on business role templates must be customized and restricted to actual company roles to comply with the principle of least privilege.

If changes to the template were included in an upgrade, the *Business Role Templates* app informs you about these changes and how they affect your business roles. For more information, see *Business Role Templates* (Related Information).

When you create a role based on a template, as a default the read and value help access are unrestricted, and write access is not granted. You can change this and define for each field to which a catalog provides access what a user is allowed to see and to edit. This allows you to shape your business roles in a very detailed way. For example, you can create two roles that comprise the same catalogs, but role 1 is restricted to work with data for the US, and role 2 is restricted to work only with data for Germany.

Process Steps

Procedure

1. Select the tile of the *Maintain Business Roles* app on the SAP Fiori launchpad to open the app. On the initial screen select *Create From Template*.
2. Select the required template. Define the ID and the name of the new business role. If required, go to the *Assigned Launchpad Spaces* tab, click *Add*, select *Use Predefined Space*, select the required launchpad space and click Assign Space (for more information, see the *Related Information* section below). Click *OK*.

3. A template already contains one or more business catalogs that the business role will be assigned to. Adjust the displayed template to your requirements, for example, change the general role details, and add or delete catalogs.
4. By default, the value help and read access for each business catalog is set to unrestricted and there is no write access. If you want to change these restrictions, select *Maintain Restrictions*.
5. Maintain instance-based restrictions for all required business objects (following the requirements of your local authorization concept).
6. On the *Assigned Business Users* tab, you can assign the business users to your new business role. These users will receive the authorizations as defined in the business role.
7. Save the business role to activate it.

> ⓘ Note
>
> If you go back to the business roles overview **without saving** the business role, the business role is automatically saved in a draft status. You can access it again and edit it from the business roles overview.

Related Information

Business Role Templates [page 3094]
Step by Step: Create a New Space and Page for a Business Role

6.3.1.13.2.3.4 How to Create a Business Role for the Administrator

Get an overview of how to create a business role for the administrator if you haven't used the system before.

Context

If you as an administrator haven't used the Fiori apps before, you have to create an administrator business role first. Otherwise the app tiles are not visible on the Fiori Launchpad. With this role, you can create all other business roles according to the needs of your company.

> ⚠ Caution
>
> The SAP_BR_ADMINISTRATOR role is pre-defined by SAP and only intended for the initial set-up of a system. Please create your own business roles based on the required business role template.

To create a business role for the administrator, proceed as follows:

Process Steps

Procedure

1. Use the initial credentials that you have received separately e.g. via email, to log in to the system. On the SAP Fiori Launchpad select the tile *Maintain Business Roles* to open the app.
2. Select *Create From Template*. The *Create Business Role from Template* window opens. In the field *Template*, search for `SAP_BR_ADMINISTRATOR`. Define the ID and the name of the new business role.
3. A predelivered template contains a number of catalogs that in this case an administrator could need to perform tasks. These catalogs are displayed on the *Assigned Business Catalogs* tab. By default for each catalog the read access is unrestricted and there is no write access. If you want to change the restrictions, for example, to allow unrestricted write access for all catalogs, proceed as follows:
   a. Select *Maintain Restrictions*.
   b. On the new screen, select the restriction you would like to change.
   Under *Write*, select *Unrestricted*.
   c. Select *Back to Main Page* to save the changes.
4. On the *Assigned Business Users* tab, all users that are assigned to the selected catalogs are listed. To add a business user, proceed as follows:
   a. Select *Add*.
   b. In the *Add Business Users* window, search for your own business user and select *Apply*.
5. Save the business role to activate it.

   > ⓘ Note
   >
   > If you go back to the business roles overview **without saving** the business role, the business role will automatically be saved in a draft status. You can access it again and edit it from the business roles overview.

   You can now log out from the system and log in with your own credentials and proceed creating further business roles according to the needs of your company

6.3.1.13.2.3.5 How to Edit Active Business Roles

Context

The editing of an active business role takes place in a draft version of this business role. This draft version is created by the system once you enter the edit mode of the active business role. Proceed as follows to edit an active business role:

Procedure

1. Select the tile of the *Maintain Business Roles* app on the SAP Fiori Launchpad to open the app.
2. On the initial screen, open the business role you need to edit by clicking on it in the *Business Roles* list. Alternatively, you can either use the *Search* or filter for the business role ID, for example.
3. To enter the edit mode, click *Edit*.

   A draft version of the business role is created in which the editing takes place. The active business role remains.
4. Make the required changes to the business role.
5. Save the business role.

   Once the changes are saved, they are written into the active business role and the draft version disappears. **If the dialog for editing a business role is left without saving the business role**, the active business role as well as the draft version of the business role will be available in the business roles overview. In this case, you can continue editing the draft version later.

   Please note that the *Check Dependencies* function is only active in edit mode.

6.3.1.13.2.3.6 How to Define Authorizations Based on Restrictions

Context

The authorization concept for business roles is based on the concept of granting authorizations (as opposed to denying them). By adding business catalogs to a business role and assigning the business role to a business user, you can control which applications the business user is authorized to carry out. As a next step, you define how the customer data can be accessed. You do this by adding authorization values to the restriction fields. Restriction fields are organized in restriction types. Restriction types that contain general organizational restriction fields can be found in the top section called *General*. These restriction types contain only one

restriction field. The settings you make for the single restriction types sum up to the authorization that is granted to the business role and therefore to the assigned business users.

> ⚠ Caution
>
> Assigning multiple business roles to a business user increases the risk of overriding existing authorizations.

> Example
>
> You have created the business roles 1 and 2 that both include the business catalog A. In business role 1, you restricted the access rights for sales organization to A. In business role 2, you allowed to work with data for all sales organizations. The business user to whom you assign both roles will then have full access to the data for all sales organizations.
>
> This overriding effect can also be caused if one restriction type is used in different assigned business roles of different business catalogs.

Using the *Maintain Business Roles* app, you have the following options for maintaining restrictions:

Procedure

- In the *Maintain Restrictions* UI, you can maintain restrictions for business roles. These restrictions are the basis of authorizations that are granted to the business users who are assigned to this business role. The business catalog defines which access categories are available for maintaining and for which fields restriction values can be maintained.

  The following access categories are available:
  - *Write, Read, Value Help* (write access)
  - *Read, Value Help* (read access)
  - *Value Help* (value help access)

  The business role aggregates the authorizations of the assigned catalogs.

  The available restriction fields represent the authorization-relevant attributes of the business objects that are used in a role. Authorization for these fields can be granted on write, read or value help level (for example for a particular sales area).

  You can select or enter single values (pre-defined by SAP or customer-specific configuration) and ranges.

  For more information, refer to *Display Restriction Types* (Related Information).

  > ⓘ Note
  >
  > When you set an access category to *Restricted* you can define the data access for each restriction type and restriction field according to your process requirements. You can do this on *Read*, *Write* and *Value Help* level.

- **Write**
- The default value access category when a role is created is *No Access*. This means that this business role has no write authorizations at all (display only). You can add specific authorizations (*Restricted*) or in cases where you want to grant full access for all restriction types and restriction fields, you can choose *Unrestricted* (‘*’).

Switching the write access to *Restricted* allows you to define which data can be edited by the users assigned to this business role.

- In the *Values* area, you can define the authorization values for the desired restriction fields.

  If you don't want to grant access to a restriction field on purpose, you can choose the status *Not maintained*.
- Every authorization you define in the *Write* access category is inherited by the *Read* and *Value Help* access category if these categories are not explicitly set to *Unrestricted* access.
- **Read**
- The default status of the access category *Read* is *Unrestricted*.

  Switching the read access to *Restricted* allows you to define which data can be seen by the users assigned to this business role.

  In the *Values* section, you can define the instance-based restrictions for the desired restriction fields used for value helps.

  For more information about the *Values* area, refer to the *Write* section above.
- Every authorization you define in the *Read* access category is inherited by the *Value Help* access category if this category is not explicitly set to *Unrestricted* access.
- **Value Help**
- You can define authorizations for value helps that are used in a business role. These value help authorizations will not influence the defined restrictions for read access.

  In the context of a business role, you can authorize the value help access, for example to business partners that belong to certain authorization groups.
- *Leading Restriction*
- You can reduce the maintenance effort of restriction fields that are used in multiple restriction types of an access category.
- Restriction types that contain general organizational restriction fields can be found in the top section called *General*. These restriction types contain only one restriction field.

  The status of the restriction fields that belong to this section can be changed to *Leading Restriction*. You can do this by selecting the *Leading Restriction* checkbox. This status is visible through the *Leading Restriction* symbol in the *Restrictions Overview*. The respective symbol looks like this:

  That means the value in this field is automatically inherited to other restriction types the field is used in as well.

  > **Example**
  >
  > If you want to ensure that the values for the country/region templates for Austria and Switzerland apply to all restriction types where the *Company Code* field is used, you can select the values **`AU01`** for Austria and **`CH01`** for Switzerland. Then, switch *Leading Restriction* on. These values are automatically inherited to all occurrences of the *Company Code* field.
- *Default Values from Business Catalogs*
- Business catalogs can contain default values. In rare cases, these might overwrite the values defined by you in the *Restrictions* and *Values* section. The system then displays a message listing the default values

that are added from the business catalog. To avoid this, you then need to remove the respective business catalog from the business role.

The respective symbol that is visible in the *Restrictions Overview* looks like this:

Related Information

Display Restriction Types [page 3097]
Maintain Restrictions in Business Role
Maintain Restrictions UI [page 3078]
Restrictions Overview [page 3080]

6.3.1.13.2.3.6.1 Maintain Restrictions UI

Maintain Restrictions

The *Maintain Restrictions UI* combines all restrictions of a restriction type of all access categories per business role in one place. It includes a main view showing all access categories and assigned restriction types and a detail view where you can change the assigned restrictions and values if required.

Main View

At the top of the main view, name and ID of the business role and the defined settings of the access categories are displayed. You can change these if required in the dropdown lists shown directly below. Underneath, the assigned restriction types are listed.

The *More Actions* area in the header includes the following options:

- *Maintain Empty Restrictions*
  When you are in the edit mode you can apply the selected field settings to all empty restrictions of a selected access category.
- *Display Restrictions Overview*
  You can navigate to the *Restrictions Overview* from here and from the object view in both edit and display mode

The following colour scheme of the assigned restriction types helps you to organize your work:

- Green: all restrictions of this type have been processed (either maintained with values or set to *Not Maintained* intentionally)
- Orange: at least one field is still empty

Please note that there is one exception to this colour scheme. If a restriction is not directly maintained for a specific access category but filled via inheritance from a higher access category it is shown as green even if it was not processed manually.

The restriction type you're currently working on is displayed under focus.

Detail Area

The detail area consists of the following four tabs:

- *Values*
- *Description* (if available)
- *Business Catalogs*
  Here you can see which business catalogs the restrictions come from.
- *IAM Apps*
  Here you can see which IAM apps the restrictions come from.

You can carry out the following activities in the detail view:

- Display all restrictions and values of the restriction type that is currently under focus.
- Edit restrictions and values by clicking the pencil icon when you are in edit mode.
  - Maintain the field settings *Restricted*, *Unrestricted Access* or *Not maintained* by selecting the radio buttons. The option *Maintained in Derived Role* is only available if the business role you're currently editing is a leading business role.
  - Select and deselect values from the value help list
  - Add number ranges if number ranges are supported
    Edit or paste multiple values in a comma-separated form in the text area

    > **Note**
    >
    > Please note that any change in the text area will be processed once you leave the text area.
  - Display an overview of all assigned values

  > **Note**
  >
  > Mass maintenance is supported as well. You do this by selecting multiple restrictions and then clicking on one of the following buttons in the header of the restrictions list:
  >
  > *Unrestricted Access*
  >
  > *Not maintained*
  >
  > *Maintained in derived role* (this option is only available for leading business roles)
- Easily change the access category of a restriction by selecting the following radio buttons: *Write, Read, Value Help*, *Read, Value Help*, or *Value Help*.
- Add other instances of this restriction type of the highest access category (which is automatically selected) by pressing the *Add* button in the header of the restrictions list.
- Define a restriction of the *General type* as leading by selecting the *Leading Restriction* checkbox.
  If required, you can define several general restrictions as leading. You can do this by selecting the required restrictions and then clicking *Leading*.
- Remove restrictions if required by selecting the restrictions and clicking the *Remove button* in the header of the restrictions list.

- Hide fields you have set as *Unrestricted*. This makes it easier for you to gain an overview of all restrictions. You can do this clicking *Hide Unrestricted* in the very top right corner of the screen.

6.3.1.13.2.3.6.2 Restrictions Overview

Restrictions Overview

You can use the *Restrictions Overview* to display all restrictions of different access categories and values that are maintained for the business role. The following filtering options are available:

- *Restriction Field*
- *Restriction Type*
- *Restriction Type ID*
- *Access Category*
- *Field Settings*

Clicking on a restriction leads you directly to the maintenance screen where you can make adjustments if required. You can also open the maintenance screen by clicking *Maintain Restrictions* in the top right corner of the screen.

6.3.1.13.2.3.7 How to Download and Upload Business Roles

Download business roles from the source system and then upload them to the target system to make them available there.

Context

You can download one or more business roles from the source system and then upload them to the target system using an `XML` file.

Procedure

1. To download the required business roles, go to your source system and select them.
2. Click *Download* and save the `XML` file on your hard drive.
3. To upload the required business roles, go to your target system and click *Upload*.
4. Browse for the `XML` file and click *OK*.

> ⚠ Caution
>
> Please do not modify the XML file on your hard drive before uploading it to the target system. This can lead to technical issues.

> ⓘ Note
>
> Some business catalogs with tenant-specific functions might not be in scope in all target tenants. For example, the business catalog *Extensibility - Transport Management - Export (SAP_CORE_BC_SL_EXP)* provides authorizations for exporting key user extensibility. As you can't export key user extensibility transports from the production system (but only import them), the business catalog may not be available in your production system.

> ⓘ Note
>
> Uploading business roles from a newer release to an older release is not supported and may lead to inconsistencies.

> ⓘ Note
>
> Business roles that have already been **transported via a Software Collection** to a subsequent system **cannot be uploaded again using the Upload function** in that target system.
>
> A **re-upload** of previously transported business roles is technically prevented and will be rejected by the system.

6.3.1.13.2.3.8 How to Create Leading and Derived Business Roles

Context

If you need to create several business roles with common characteristics, you can define one business leading role and then derive other business roles from it. This leading role contains the basic settings such as access restrictions, the assigned business catalogs and common restrictions, such as *General Accountant* or *General Planner*. The values defined in the leading business role can't be changed in the derived business role. You can, however, define additional values for the derived role.

Procedure

1. Open the business role you want to define as leading business role and click *Edit*.
2. Select the *Is Leading Business Role* checkbox and click *Save*. The business role now has a *LEADING BUSINESS ROLE* label.

3. Go back to the list view, select this leading business role and click *Create Derived Business Role*.
4. The system suggests a business role ID you can enhance.
5. Enter a role description and click *OK*. The system displays the business role maintenance screen. The business role has a *DERIVED BUSINESS ROLE* label.
6. Add further values to the derived business role as required.
7. Save your entries.

6.3.1.13.2.3.9 How to Compare Business Roles

Context

To display the common features and the differences between business roles, you can compare them in the *Maintain Business Roles* app. On the *Compare Business Roles* screen, you can display the restrictions, business users, business catalogs and launchpad spaces, which the business roles have in common, or those that the business roles don't have in common. You can only compare a maximum of two business roles. They can be active or in draft mode.

Procedure

1. On the *Maintain Business Roles* initial screen, select the two business roles you want to compare.
2. Click *Compare*. The system displays the *Compare Business Roles* screen for these two business roles including information about the assigned restrictions, business users, business catalogs and launchpad spaces.

   If the *Write*, *Read* or *Value Help* access category is restricted in both roles, you can navigate further to display the assigned restriction types for that access category.

   Restrictions

   You have the following options for comparing restrictions:

   Display All

   Display all restriction types assigned to one of the roles or to both.

   Display Same Restriction Types with Same Values

   Display the restriction types that are assigned to both roles and have the same values in both roles.

   Display Same Restriction Types with Different Values

   Display the restriction types that are assigned to both roles and have different values in each role.

   Display only Differences

Display only the differences between the two roles, for example if a restriction type is only assigned to one role but not to the other role.

The restriction types are coloured as follows:

Colour	Explanation
Green	The restriction type is assigned to both roles and in both roles, the values are the same.
Orange	The restriction type is assigned to both roles but the values are different.
Red	The restriction type is only assigned to one role but not to the other.

If the restriction types are colored in green or orange, you can navigate further and display more details.

The system displays the restriction types assigned to the roles you want to compare next to each other. If there are differences between the two roles, the order of the instances reflects the degree of similarity. You can use drag and drop if you want to compare two other restriction types than the ones suggested by the system.

Business Users, Business Catalogs and Launchpad Spaces

You have the following options for comparing business users, business catalogs and launchpad spaces:

Display All

Display all business users, business catalogs and launchpad spaces that are assigned to one of the roles or both.

Display Only Common Business Users / Display Only Common Business Catalogs / Display Only Common Launchpad Spaces

Display only the business users, business catalogs or launchpad spaces the two business roles have in common or that are assigned to both.

Display only Different Business Users / Display only Different Business Catalogs / Display Only Different Launchpad Spaces

Display only the business users, business catalogs or launchpad spaces that differ from one business role to the other or that are assigned to one business role but not to the other.

6.3.1.13.2.3.10 How to Create Spaces and Pages for a Business Role

Context

When creating a new business role, you can create a new launchpad space, use an existing launchpad space or launchpad spaces based on a predefined space. If required, you can add multiple spaces to the role by clicking

Add on the *Assigned Launchpad Spaces* tab. If a launchpad space is no longer relevant for a role, you can click *Remove*. Please note that you can assign an existing launchpad space to several business roles at once. For more information, see the *Related Information* section.

Related Information

Step by Step: Create a New Space and Page for a Business Role
How to Make Mass Changes to Business Roles [page 3086]

6.3.1.13.2.3.11 How to Transport Business Roles

Context

You can export business roles from your quality system to your productive system using customizing transports.

> ⓘ Note
>
> - Once you have transported a business role, no change documents will be written for this business role in the productive system. Change documents for transported business roles are only available in the quality system.
> - If you transport a derived business role, the leading business role and all other derived business roles need to be added as dependencies to the transport request as well.
>   If you transport a leading business role, all derived business roles need to be added as dependencies to the transport request as well.
> - Please note that business roles that are pre-delivered by SAP (business roles whose ID starts with SAP*) cannot be transported.

Procedure

1. In the *Business Roles* overview screen of the *Maintain Business Roles* app, select one or more business roles.
2. Click *Add to Transport*. A list of available customizing transports appears.

   > ⓘ Note
   >
   > The default transport is always displayed as well as your own transports. If no transport exists, a confirmation dialog is shown. If you confirm, a new transport is created.

3. Select the required transport and click *OK*. The selected business roles are then added to the transport.
4. Open the *Export Customizing Transports* app and release your request. For more information, see Export Customizing Transports [page 2978].
5. Import your business role to the target system:
   a. Open the *Manage Software Components* app.
   b. Select *Business Configuration* in the *Type* dropdown and click *Go*.
   c. Select the component to which the relevant configuration was exported.
   d. In the detail view of the component, check out the change that contains your business role(s). For more information, see How to Work with Branches.

> **Note**
>
> If you want to delete a business role that was previously transported or is currently included in an open transport request, the system offers you to add it directly to an available transport.
>
> If the business role was previously transported and there is currently no transport available, a confirmation dialog is shown. If you confirm, a new transport is created.

Related Information

Export Customizing Transports [page 2978]
Working in the Export Customizing Transports App [page 2980]
Manage Software Components
How to Delete Previously Transported Business Roles [page 3085]

6.3.1.13.2.3.11.1 How to Delete Previously Transported Business Roles

Context

If you want to delete a business role that was previously transported or is currently included in an open transport request, the system offers you to write it directly to an available transport.

Procedure

1. In the *Business Roles* overview screen of the *Maintain Business Roles* app, select one or more business roles you want to delete.

2. Click *Delete* and confirm the dialog. A list of available customizing transports appears.

   > ⓘ Note
   >
   > The default transport is always displayed as well as your own transports. If no transport exists, a confirmation dialog is shown. If you confirm, a new transport is created.

3. Select the required transport and click *OK*. The deletion of the selected business roles is then added to the transport and the selected roles will be deleted.

   > ⓘ Note
   >
   > Your selection may contain business roles that were not previously deleted. These will be deleted directly without a transport. The system also offers you to delete the business role without transporting the deletion. The role will not be deleted in the target system in this case.

4. Open the *Export Customizing Transports* app and release your request. For more information, see Export Customizing Transports [page 2978].
5. Import your business role to the target system:
   a. Open the *Manage Software Components* app.
   b. Select *Business Configuration* in the *Type* dropdown and click *Go*.
   c. Select the component to which the relevant configuration was exported.
   d. In the detail view of the component, check out the change that contains your business role(s). For more information, see How to Work with Branches.

Related Information

Export Customizing Transports [page 2978]
Working in the Export Customizing Transports App [page 2980]
Manage Software Components
How to Transport Business Roles [page 3084]

6.3.1.13.2.3.12 How to Make Mass Changes to Business Roles

Context

You can use the mass change wizard to change multiple business roles at once. This is helpful, for example, if you have switched from using custom spaces to predefined spaces or if you start using spaces. In that case, you can select all affected business roles and choose *Assign Space Templates*. The system then automatically assigns predefined spaces to all of the business roles in question. If you choose *Remove Launchpad Spaces Already Assigned*, the custom spaces that were previously assigned to the business roles are automatically removed.

> **Note**
>
> Please note that SAP-defined business roles can only be modified for business user assignment in the initial setup. Apart from this, they can be updated by SAP only. We recommend that you create your own business roles from templates so that you can modify them as required.

Select the business roles you want to change and click *Mass Change*. The *Mass Change Wizard* appears. The pattern of the wizard is described in the next section.

> **Caution**
>
> If you have selected a combination of leading, non-leading, and derived business roles, some attributes might not be selectable. For example, business catalogs in derived business roles can't be edited. Therefore, business catalogs are not included in the mass change if a derived business role is selected.
>
> To change all of the listed attributes, select only leading business roles.

Procedure

1. Select the required attributes. You can make the following changes:

Area	Attributes
Business Role Data	Role Group Expose to SAP BTP Inherit Spaces in Derived Business Roles
Business User Assignment	Add Business Users Remove Business Users
Business Catalog Assignment	Add Business Catalogs Remove Business Catalogs
IAM App Activation	Activate IAM Apps Deactivate IAM Apps
IAM App Assignment	Add IAM Apps Remove IAM Apps
Launchpad Space Assignment	Add Launchpad Spaces Remove Launchpad Spaces
Assign Space Templates	Assign space templates to your business roles You see the results and can filter for error or success statuses in the *Mass Change Overview*.

Area	Attributes
	Note Please note that this option is only relevant for business roles that were created from business role templates.
Access Categories	Change access categories as required.
Restrictions	Select the required access category, such as *Value Help* and select the required restriction change, such as *Add Restrictions*.
Copy Restrictions	1. Click *Add Restrictions* or *Remove Restrictions*. 2. Copy the required restrictions using the *Copy Restriction Values From* and the *Copy Restriction Values To* value help. 3. Select additional access categories if needed. **Note** You can use this option in case a restriction type is deprecated to copy restrictions from the deprecated restriction type to the successor restriction type(s). You can either add the copied restriction values in the fields to existing ones or remove all and replace them with the copied restrictions.

2. Apply the required attributes to your business roles.
3. Review and confirm your changes.

Related Information

Assigning Spaces to Several Business Roles at Once

6.3.1.13.2.3.13 Effects on the Maintain Business Roles App

Learn which information about IAM apps of the app authorization variant type you can view and manage in the *Maintain Business Roles* app.

IAM apps can either be assigned directly to a business role or through a business catalog that bundles several IAM apps. Some business catalogs might not yet support IAM apps. To check if a business catalog supports IAM apps, open the *Business Catalogs* app and use the *Supports IAM Apps* filter. To view the IAM apps assigned to a business role, select a business role and go to the *IAM Apps* tab. On this tab, you can activate or deactivate the assigned IAM app for this business role.

To view the history of changes to a business role regarding the IAM apps assigned, select a business role and choose *Display Changes*. On the *Change Documents* screen, select *Assigned IAM Apps* for the *Change Category* filter and choose *Go*.

To activate or deactivate IAM apps for more than one business role at once, select the relevant business roles, choose *Mass Change* and use the *Mass Change Wizard*.

If the business role is created from a business role template, the *Activate IAM Apps* checkbox is selected on the *Create Business Role from Template* screen.

Related Information

Business Catalogs [page 3092]
How to Activate or Deactivate IAM Apps (App Authorization Variants) [page 3089]
Work with IAM Apps of the App Authorization Variant Type

6.3.1.13.2.3.14 How to Activate or Deactivate IAM Apps (App Authorization Variants)

With IAM apps of the app authorization variant type, you have the flexibility to manage authorizations for business users on a more granular level. You can activate or deactivate individual IAM apps according to your business requirements.

If new or successor IAM apps are added to a business catalog with a new release, they are not automatically activated. Even if the business catalog is already in use, these IAM apps remain inactive until you choose to enable them. This gives you full control over if and when to activate new capabilities and helps prevent unexpected changes.

1. Open the *Maintain Business Roles* app.
2. Select a business role.
3. Go to the *IAM Apps* tab.
   All IAM apps that are assigned to the selected business role, for example of the app authorization variant type, are displayed.
4. If certain IAM apps are required for the selected business role, choose *Activate*.

> ⓘ Note
>
> You can deactivate IAM apps again at any time if required.
>
> To activate or deactivate IAM apps for more than one business role at once, select the relevant business roles, choose *Mass Change* and use the *Mass Change Wizard*.

> ✤ Example
>
> The *Maintain Business Users* app has the Fiori ID `F1303`. If you open the *Maintain Business Roles* app, select a specific business role, go to the *IAM Apps* tab, enter **`F1303`** in the search field, and choose *Go*. All existing IAM apps are displayed, for example:

- IAM app called *Display Business Users* (`F1303_03_TRAN`) with read authorization
- IAM app called *Assign Business Users to Business Roles* (`F1303_22_TRAN`) with write authorization
- IAM app called *Maintain Business Users* (`F1303_TRAN`) with write authorization

All three IAM apps belong to the transaction code `F1303`. However, they have distinct IAM app IDs with the prefix `F1303`. If you want to grant read access to business users assigned to a specific business role, activate the *Display Business Users* IAM app (`F1303_03_TRAN`).

Related Information

Work with IAM Apps (App Authorization Variants)

6.3.1.13.2.4 Maintain Business Role Groups

With this app you can create business role groups and assign multiple business roles to them. This helps you to organize your area and easily search for all business roles of a certain category (for example to assign business users to them). Grouping also facilitates maintenance of authorizations. If you are the super administrator for all areas, you can delegate this task to administrators for the relevant areas, such as Financials. In this case, you would create a business role group for Financials and allow to assign these business roles only to particular user groups.

Please note that the name of business role group is created automatically in namespace *ZCB*.

Key Features

You can use this app to:

- Create business role groups and assign the required business roles to them
- Reassign or remove business roles to or from business role groups
  Note: You can also use drag and drop to move business roles from one group to another.
- Upload business role assignments
- Add business role groups to transports
- Export business role groups to a spreadsheet (including description and number of business roles)

Prerequisites

The following restriction types (in the following business catalogs) are required for maintaining or displaying business role groups and business user groups:

Business Catalogs	Restriction Types
SAP_CORE_BC_IAM_RM	*Business Role* (S_BRL)
SAP_CORE_BC_IAM_UM	*Business User* (CLASS) *Business Role* (S_BRL)
SAP_CORE_BC_IAM_RA	*Business Role* (S_BRL) *Business Role User Assignment* (S_BRL_ASG) *Business User* (CLASS)
SAP_CORE_BC_IAM_RM_DISP_PC	*Business Role* (S_BRL)
SAP_CORE_BC_IAM_UM_DISP_PC	*Business User* (CLASS)

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

6.3.1.13.2.4.1 How to Transport Business Role Groups

Context

You can export business role groups from your quality system to your productive system using customizing transports.

Procedure

1. In the *Business Role Groups* overview screen of the *Maintain Business Role Groups* app, select one or more business role groups.
2. Click *Add to Transport*. A list of available customizing transports appears.

> **Note**
>
> The default transport is always displayed as well as your own transports. If no transport exists, a confirmation dialog is shown. If you confirm, a new transport is created.

3. Select the required transport and click *OK*. The selected business role groups are then added to the transport.
4. Open the *Export Customizing Transports* app and release your request. For more information, see Export Customizing Transports [page 2978].
5. Import your business role groups to the target system:
   a. Open the *Manage Software Components* app.
   b. Select *Business Configuration* in the *Type* dropdown and click *Go*.
   c. Select the component to which the relevant configuration was exported.
   d. In the detail view of the component, check out the change that contains your business role group(s). For more information, see How to Work with Branches.

> **Note**
>
> If you want to delete a business role group that was previously transported or is currently included in an open transport request, the system offers you to add it directly to an available transport.
>
> If the business role group was previously transported and there is currently no transport available, a confirmation dialog is shown. If you confirm, a new transport is created.

6.3.1.13.2.5 Business Catalogs

You use this app to display all available business catalogs.

With this app you can get an overview of the business catalogs, their status (for example *Deprecated*), and their usage within business roles. You use this application to see which applications and business catalogs delivered by SAP have changed, for example after an upgrade. They may also be deprecated. As a key user, you need to have an overview of these changes to adapt the affected business roles accordingly.

Key Features

You can use this app to:

- Display business catalogs, their usage in roles, and general description.
- Display restriction types and in which business catalog the restriction type is used.
- Display business catalog changes: Check if the business catalogs or apps in your area have changed (for example, a transaction has been replaced with an SAP Fiori app), and which business roles are affected by these changes.
- Display assigned apps: You can display apps that have their own tile access (classified as *Standard Application*) as well as ones that can be started indirectly via app-to-app navigation (classified as *Standard Navigation Target*).

- Display deprecations: Check if any of the business catalogs in your area are deprecated and which successors they will be replaced with. Select the affected business role and transfer the changes.
- Display dependencies: Check if a business catalog depends on other business catalogs or if other business catalogs depend on a business catalog in order to be fully operational.
- Display scope items: check which scope items the business catalogs depend on. Business catalogs that do not depend on any scope items are always visible in the system. For these business catalogs, *Scope Items (0)* is displayed on the relevant tab and the following message appears in the table: *The business catalog is not scope-dependent*.
- Display component: check under which component you can create an incident if required.
- Check whether a business catalog supports IAM apps using the *Supports IAM Apps* filter.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

Related Information

Business Catalogs for IAM Administration Apps [page 3115]
Display Restriction Types [page 3097]

6.3.1.13.2.5.1 How to Manage Deprecated Business Catalogs

Deprecated business catalogs are deleted from the systems around 12 months after the deprecation announcement. Review upcoming changes and define the resulting IAM requirements together with authorization coordinators (key users or domain experts) from the line of businesses.

Context

Due to ongoing development of new features and new apps, we need to periodically revise existing business catalogs. This means that some business catalogs are deprecated and replaced by one or more successors, and you need to assign business roles and business users to these new catalogs. Use the *Business Catalogs*

app to identify and replace deprecated business catalogs in business roles **four weeks before the test system is upgraded**.

> ⚠ Caution
>
> To prevent authorization issues, it's important to replace deprecated business catalogs as soon as possible. SAP usually provides around 12 months between a deprecation announcement and the actual deletion, but this period can vary. It's best to replace deprecated business catalogs soon after the announcement. However, if you missed this or intentionally deferred it, you can still prepare before the release upgrade. Four weeks before an upgrade, review the preliminary information to identify which deprecated business catalogs are still in use and scheduled for removal. Make the necessary last-minute business catalog adjustments to mitigate authorization issues during the upgrade cycle and maintain uninterrupted access.

Procedure

1. In the *Business Catalogs* app, adapt the filter settings to search for deprecated business catalogs that are used in a business role. In the *Status* search filter, select *Deprecated*. In the *Used in Business Roles* search filter, enter **>=1**.
2. Focus on the business catalogs first that have been announced to be deprecated in the last release and that will now finally be removed. Select a business catalog to see its description, its deprecation release, the list of business roles it affects, and any successor(s). If more than one successor is assigned, confirm whether all successors are required. If any successors are not required, you need to manually delete them in the updated business roles **after the upgrade**. Identify the business roles that require the deprecated business catalog to be replaced and apply the update.
3. Ensure that also the assignments of custom catalog extensions are changed.

6.3.1.13.2.6 Business Role Templates

The app provides a summary of business role templates delivered by SAP and helps determine if adaptations are needed after an upgrade.

With this app you can get an overview of the business role templates delivered by SAP, any changes included in upgrade, and whether you need to adapt your business roles to these changes. For example, after an upgrade, you can check if the business role templates have changed and are therefore different from the existing business role - a new business catalog might have been added or an existing catalog replaced by another. You can see which business roles were affected by the changes and adapt them if required.

> ⚠ Caution
>
> Business role templates delivered by SAP are intentionally broad and meant for testing. Before productive use, business roles based on business role templates must be customized and restricted to actual company roles to comply with the principle of least privilege.

Key Features

You can use this app to:

- Display business role templates
- Display usage of business role templates in business roles
- Display differences between business role templates and business roles
- Check which business roles are affected by these changes
- Adapt business role to changes
- Create a business role
- Display scope items: check which scope items the business role templates depend on. Business role templates that do not depend on any scope items are always visible in the system. For these business role templates, *Scope Items (0)* is displayed on the relevant tab and the following message appears in the table: *The business role template is not scope-dependent*.
- Display component: check under which component you can create an incident if required.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

Related Information

How to Create a New Business Role [page 3070]
How to Create a Business Role from a Template [page 3072]

6.3.1.13.2.7 Display IAM Apps

With this app you can display IAM apps of all supported IAM app types. This helps you to keep an overview of the assigned business roles.

Key Features

You can use this app to:

- View general information, such as the IAM app ID or the transaction
- View the IAM app description
- Display the business catalogs to which an IAM app is assigned
- Display the business roles to which an IAM app is assigned
- Identify successor IAM apps for deprecated or non-deprecated IAM apps
- For external apps, you can view the related authorizations. These include authorization objects, authorization instances, and authorization fields.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

6.3.1.13.2.7.1 IAM App Types

The following IAM app types are available:

IAM App Type Description	IAM App Type	Explanation
External App	*EXT*	An external app represents an app extension. It defines what is included in a tile (this is determined by the services). You can use the external app to bundle services, define authorizations and assign business catalogs.
Business Configuration App	*MBC*	A business configuration app serves as an entry point to the configuration objects provided by the different application areas or partners. The configuration objects can be adjusted to change the system behaviour.

IAM App Type Description	IAM App Type	Explanation
Custom Inbound Service App	*IBS*	This application type is only created in the development system and is only required for authorization purposes to allow the developers to test the inbound services locally. The included objects only exist in the development system and authorize developers to call the OData services with the developer users for testing purposes via the following business catalog: *SAP_CORE_BC_EXT_TST - Extensibility - Custom Apps and Services*.
UI5 Adaptation App	*UI5A*	An IAM app for a developer adaptation project. It is similar to an external app but release state checks for services inherited from the base app are disabled.

6.3.1.13.2.8 Display Restriction Types

You can use this app to display restriction types and their validity.

With this app you can display restriction types, the assigned fields, and in which business catalog the restriction type is used.

Key Features

You can use this app to:

- Display all available restriction types and their fields.
- Check whether the restriction fields support number ranges.
- Display how these restriction types can be used in which business catalogs.
- Display component: check under which component you can create an incident if required.

Restriction types bundle one or more restriction fields. The restriction type *Organizational Area*, for example, contains the following fields: *Purchasing Group*, *Purchasing Organization*, *Division*, *Sales Organization*, *Distribution Channel*.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

6.3.1.13.3 IAM Roles

IAM roles are a new role concept that replaces business roles with improved performance and scalability through simplified design and modern frameworks. Use IAM roles to manage larger numbers of roles and more complex role definitions while migrating existing business roles during the transition period.

General Information

The IAM role is a new role concept that gradually replaces the business role. Its simplified design and use of modern frameworks improve performance and scalability, supporting a larger number of roles and more complex role definitions.

Business roles and IAM roles coexist for several releases during the transition period. Over time, the system deprecates and eventually removes business roles. Therefore, migrate all existing business roles to the new IAM role concept during the transition phase.

IAM Role Features and Migration Details

- After you migrate a business role, the original business role remains available to ensure reliability and backward compatibility. However, use IAM roles for all future role maintenance activities.
- To increase transparency and control, the system doesn't update IAM roles automatically during upgrades or transport imports. You must apply any required changes manually.
- Transporting IAM roles is faster and more stable because you can't modify IAM roles directly in target systems. You maintain all changes exclusively in the development system.
- You can migrate a business role to an IAM role. However, the system doesn't synchronize changes you make later in the IAM role back to the original business role.
- IAM role authorizations are based on authorization objects and authorization fields rather than restriction types and restriction fields. During migration, the system converts business role restrictions one-to-one into the corresponding IAM role authorizations.

- Thoroughly test all migrated IAM roles after you complete the migration process.

6.3.1.13.3.1 How to Prepare for the Migration from Business Roles to IAM Roles

The migration from business roles to IAM roles is introduced in two phases:

- *Key User Extensibility Migration*
- *Developer Extensibility Migration*

> **Note**
>
> Key user extensibility migration is not yet available and will be delivered with a later release.

Related Information

Key User Extensibility Migration [page 3099]
Developer Extensibility Migration [page 3100]

6.3.1.13.3.1.1 Key User Extensibility Migration

This task describes upcoming support for key user extensibility migration.

Context

Key user extensibility migration *is not yet available* and will be delivered with a later release.

The following steps describe how the migration process will work once it becomes available.

Procedure

1. Open the *Maintain Business Roles* app.
2. Select the business roles you want to migrate.
3. Choose *Migrate* and select one of the following options:
   - *Migrate Selected* to migrate only the roles you have selected
   - *Migrate All* to migrate all roles

4. The *IAM Role Migration Wizard* opens.

> **Note**
>
> This wizard is currently for information purposes only and does not execute the migration process. The guidelines below describe the steps that can be performed in the future.

5. In *Step 1: General Information*, review the migration concept.
6. Choose *Next*.
7. In *Step 2: Related Objects*, review the selected roles and their related objects.
8. Choose *Next*.
9. In *Step 3: Pre-Requisites Check*, review the *Business Role Readiness* status.
   - *Ready*: No action required.
   - *Action Required*: Additional steps are necessary. Select the status to display the migration readiness details.
10. Proceed as follows:
    - Choose *Next* if no action is required.
    - If the status indicates that a business catalog needs to be migrated, continue with the Developer Extensibility Migration [page 3100].
11. In *Step 4: Summary*, review the migration summary.
12. Choose *Migrate* to complete the migration.

> **Caution**
>
> This option will be available in a future release.

Results

Business roles are prepared for migration to IAM roles using key user extensibility once the feature becomes available.

6.3.1.13.3.1.2 Developer Extensibility Migration

This task describes developer extensibility migration.

Procedure

1. Navigate to *Identity and Access Management* in your system in ABAP Development Tools (ADT)
2. Locate the relevant business catalog and check the migration status of its associated IAM apps.
3. For each IAM app that is not yet migrated, choose *Migrate IAM App*.

4. In the migration wizard:
    - Choose *Next*
    - Select the restriction types to migrate
    - Choose *Next* to confirm

    The IAM app status changes to *Migrated*.
5. After all IAM apps are migrated, choose *Migrate Business Catalog*.
6. Complete the wizard by choosing *Next* and then *Finish*.

    The business catalog status changes to *Migrated*.
7. Navigate to the *Maintain Business Roles* app and recheck the readiness status in the *IAM Role Migration Wizard*. See, Key User Extensibility Migration [page 3099].

    The business catalog should now be marked as *Ready*.

Results

The selected business roles are migrated to IAM roles. Associated IAM apps and business catalogs are also migrated, and restriction types are assigned at the IAM app level.

6.3.1.13.4 Monitoring and Analysis

Monitoring and analysis tasks are part of the daily work as administrator.

Common activities in this context include the following:

- Find business users with unnecessarily high privileges (for example, too many admin users)
- Find business users with unnecessary or critical authorization combinations (the result of privilege creep)
- Find business roles granting access to data too generously (that is, having unmaintained restrictions)
- Analyze the root cause for authorization errors

You can use the *IAM Information System* app to monitor the relationships between business users, business roles, business catalogs, and so on.

The *IAM Key Figures* app enables you to monitor KPIs for your authorization system. It shows the number of locked business users, for example. Also, the number of unrestricted business roles is shown. Thresholds can be defined to visualize critical deviations from the desired state.

For the analysis of issues with privileges you can use the *Display Authorization Trace* app.

An overview of the technical users in the system – both communication users and SAP support users – is provided by the *Display Technical Users* app. The *Maintain User Sessions* app enables you to analyze sessions containing a lock and to find the associated business user.

Related Information

IAM Information System [page 3102]
IAM Key Figures [page 3103]
Display Authorization Trace [page 3104]
Maintain User Sessions [page 3194]

6.3.1.13.4.1 IAM Information System

With this app you can get an overview of business users in your system and what roles and restrictions are assigned to them.

With this app you can display information about the usage of business roles, IAM apps, business catalogs, business users and restrictions, and how they are related.

If you want to look up more information about a business role, derived business role, business user, business catalog, business role template or restriction, you can jump directly to the respective app by clicking the entity.

You can use this app for administrative tasks as part of your daily work.

Key Features

You can use this app to:

- Check the usage of the following entities and how they are related: Business role, derived business role, business role template, IAM app, business user, business catalog, restriction, launchpad space, launchpad page.
- Check, for example, which business roles are assigned to a business user, which business catalogs and restrictions are therefore assigned to the business user and to which apps a user has access. You can also download a list of business users and business catalogs.
- Check, for example, which business roles and business catalogs are required to make a certain app visible for a business user on the SAP Fiori Launchpad.

  > ⓘ Note
  >
  > You can also display the scope items the business catalogs depend on. Business catalogs that do not depend on any scope items are always visible in the system. For these business catalogs, **Scope Items (0)** is displayed on the relevant tab and following message appears in the table: *The business catalog is not scope-dependent*.

- Display assigned apps: You can display apps that have their own tile access (classified as *Standard Application*) as well as ones that can be started indirectly via app-to-app navigation (classified as *Standard Navigation Target*).

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

6.3.1.13.4.2 IAM Key Figures

You can use this app to display the following information:

- Number of business users assigned to business roles
- Month of the business user's last log-on
- Number of locked and unlocked business users
- Validity of business users
- Number of business roles with unmaintained restrictions
- Number of business roles with unrestricted access

  > **Note**
  >
  > You can filter for the productive business roles in your area, for example by adding a certain namespace followed by an asterisk (*) in the *Filter by Business Role ID* field in the tile configuration.

- Business user price categories
- Business roles with default values from business catalogs
  The default values are delivered automatically and can't be overwritten. This means either that an asterisk (*) overwrites all defined values or that fixed values are added to the restrictions you have defined.
  Business roles affected by these non-changeable values are displayed in red in the overview chart.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

6.3.1.13.4.2.1 IAM Key Figures - Background

The charts in the **IAM Key Figures** app support you to increase the security in your area and to reduce costs by displaying which business users are inactive and can be removed.

They also provide a helpful overview that can be useful before go-live. For example, it helps you to evaluate whether the business user and role distribution in your area is ready for use.

To define threshold values, you can use color codes for the bars in your charts. The following colors are available:

- Green: **Accepted**
- Yellow: **Warning**
- Red: **Critical**

> ### Example
>
> You can define the colors based on certain time frames. If a user has not logged on to the system for 6 months and you want to use this point in time as a threshold value, you can determine that the corresponding bar in the chart is red.

You can also jump directly to the **Maintain Business Users** app or **Maintain Business Roles** app if you need more information about the business users or business roles that are shown in the overview charts.

Related Information

Maintain Business Roles [page 3068]
Maintain Business Users [page 3044]

6.3.1.13.4.3 Display Authorization Trace

With this app you can enable an authorization trace for a business user. This helps you to analyze if any authorizations are missing or are insufficient.

Key Features

You can use this app to:

- Activate or deactivate a trace
- Display authorization check results including already assigned authorizations and failed checks
- Display all business roles granting access to selected fields and values

A maximum of 10.000 data sets is possible, therefore we recommend to consider this when defining the selection criteria, especially the date range.

The following authorization check statuses are possible:

Status	Meaning
Successful	The authorization check was successful.
Failed	The authorization check failed.
Filtered	When reading an object, an authorization check is taking place and certain data is filtered out defined by a DCL (Data Control Language). **Note** When the *Authorization Check Status* is set to *Filtered*, no authorization check is carried out for any value. Instead, access filtering occurs. In this case, *No Value* is displayed for the corresponding restriction fields. Filtering means that the authorizations of the user for the specified restriction type/authorization object and for the specified field values are read and these authorization values are used as additional filter conditions during the selection of the business data. This way, the user only has access to the data for which he or she has appropriate authorizations.

If an authorization check resulted in a *Failed* and *Filtered* status, you can check which business roles expose the affected restriction type. The root cause could be that the business user that has been checked is not assigned to the required business role or that the required value has not been maintained yet.

Note

The trace entries are deleted automatically within a certain time frame.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

6.3.1.13.5 Manage Changes and Deprecations After Upgrade

Your system is upgraded regularly to deliver new features and improvements.

These changes may include changes or deprecation of apps, business catalogs, and business role templates.

Deprecation Process for Apps

If an app is outdated or its functionality is covered by a successor app, it is deprecated. This means that even though you can still use the app, SAP no longer recommends its use, and it will be deleted from the SAP Fiori launchpad in an upcoming release. There's a period of at least six months between the initial announcement of an app's deprecation and its deletion. The deprecated app may no longer be available by default on the SAP Fiori launchpad. In this case, you can find it in the app finder until it is deleted.

The deprecation of an app may require you to recheck the business catalogs that are assigned to your business roles. For more details about this and changes to restriction types, see *Manage Business Role Changes After Upgrade* in the *Related Information* section.

Deprecation of Business Catalogs

Due to ongoing development of new features and new apps, you need to revise existing business catalogs with every system upgrade. This means that some business catalogs are deprecated and replaced by new ones, and you need to assign business roles and business users to these new business catalogs.

Rather than disappearing, deprecated business catalogs are identified as being obsolete, which allows you to identify them at a glance. For more information, see *How to Handle Deprecated Business Catalogs* in the *Related Information* section.

Business Role Changes after Upgrades

It is important for you to know how the release changes impact your productive business roles.

Business role changes often have the technical reason that new developments may require changes to existing applications, business catalogs, restriction types, and/or restriction fields. The affected business roles need to be adapted accordingly to ensure that existing apps behave consistently before and after the upgrade.

The following typical changes might occur:

- Business catalogs were deprecated, or split, or have new dependencies so new business catalogs need to be assigned.
- New apps were added to existing business catalogs.
- New restriction types were added, removed, or access categories were changed.

For more information about change activation, see *Phase-In/Phase Out Status* in the *Related Information* section.

Related Information

Manage Business Role Changes After Upgrade [page 3107]
How to Manage Deprecated Business Catalogs [page 3093]

6.3.1.13.5.1 Manage Business Role Changes After Upgrade

With this app you can display all relevant changes to business catalogs and restriction types after an upgrade. If a new restriction type was added to a business catalog for example, you can maintain the corresponding restrictions using this app.

Key Features

You can use this app to:

- Display a list of changed restriction types
- Display details of the change
- Filter for certain change types (such as *Added* or *Removed*)
- Display the affected business catalogs and their dependencies
- Display deprecated business catalogs, their successors and adopt the respective changes
- Display the number of affected business roles
- Cross-navigate to the affected business roles in the *Maintain Business Roles* app to view further details
- Mass-maintain restrictions of the same access categories

Supported Device Types

- Desktop
- Tablet
- Smartphone

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

Related Information

How to Manage Changed Restriction Types After an Upgrade [page 3109]
How to Manage Changed Business Catalog Dependencies After an Upgrade [page 3110]
How to Manage Deprecated Business Catalogs After an Upgrade [page 3111]

6.3.1.13.5.1.1 Manage Business Role Changes After Upgrade: Background

It is important for you to know how the quarterly release changes impact your productive business roles.

Business role changes often have the technical reason that new developments may require changes to existing applications, business catalogs, restriction types and/or restriction fields. The affected business roles need to be adapted accordingly to ensure that existing apps behave consistently before and after the upgrade.

The following typical changes might occur:

- Business catalogs were deprecated or split, or have new dependencies so new business catalogs need to be assigned.
- New apps were added to existing business catalogs.
- New restriction types were added, removed, or access categories were changed.

6.3.1.13.5.1.2 Best Practices for Managing Business Role Changes After an Upgrade

We recommend downloading a list of all the changes that have taken place after an upgrade before you adjust your business roles using the *Manage Business Roles After an Upgrade* app. You can do this by clicking *Download* (csv format is supported). This helps you to keep an overview of what has changed. Once you have modified your business roles, the respective changed objects disappear from the list in the app. It is therefore advisable to keep a separate worklist on your local machine.

We also recommend adjusting your business roles using the *Manage Business Roles After an Upgrade* app only if one change applies to one business role. If there is more than one change, we recommend making the required changes manually in the *Maintain Business Roles* app.

Restriction Types / Business Catalog Dependencies

An affected business role is no longer shown in the list once the business role has been saved the first time after the upgrade/update.

The change itself will no longer be visible in the app after all listed affected business roles have been saved the first time after the upgrade/update.

Deprecated Business Catalogs / Business Role Template

An affected business role is no longer shown in the list once the respective change has been incorporated in the business role.

The change itself will no longer be visible in the app after the respective change has been incorporated in all affected business roles.

6.3.1.13.5.1.3 How to Manage Changed Restriction Types After an Upgrade

Context

If you want to have transparency over all of the changes to restriction types after an upgrade, and you want to maintain the corresponding restrictions in your business roles, proceed as follows:

Procedure

1. Select the *Manage Business Role Changes After Upgrade* tile on the SAP Fiori Launchpad to open the app. On the initial screen, view the *Restriction Types* area to get an overview of what has changed with the last upgrade. Filter for certain change types if required, for example if you only want to see the removed or the added restriction types.
2. Download a csv file containing all recently changed restriction types if required.
3. Click on the required restriction type to check how the changed restriction types affect your business roles. The business roles affected by the changes to this restriction type are displayed in a list. By clicking on the restriction type *Name* above the list, you can view a description of the restriction and a list of business catalogs in which this restriction type has been changed (for example added or removed). Please note that any derived roles will be filtered out from the list of the affected business roles to allow for synchronization with the master role.
4. To find more information about the affected business roles or to make further changes, you can cross-navigate to the *Maintain Business Roles* app by clicking on the link listed in the *Business Role ID* column. When you click on the *Information* pushbutton next to the link, a list of all other changes that have affected the business role after the upgrade and that might be relevant for you as well is displayed.
5. To make the necessary changes in your business roles, select the required business role and click *Maintain (Write, Read, Value Help) Restrictions*. Restrictions can be mass-maintained for multiple business roles if the roles have the same access categories. If they have different access categories, the pushbutton is disabled. On the *Maintain Restrictions* screen, you can also view the related business catalogs and the business roles selected on the previous screen. You see the restriction fields of the restriction you are currently maintaining. To see other restrictions these business roles might include, click the glasses icon (*Display Restrictions*) to cross-navigate to the *Maintain Business Roles* app.
6. Click the *Edit* icon and assign the required values or ranges. Click *Not maintained* if you want to make clear that you have not maintained a field on purpose and not by accident.
7. Click *Apply*.

Results

The system displays the results of the update in a pop-up. If the update has been successful, a green icon is visible. If issues occurred, a warning message is displayed.

When you close the pop-up, you will be directed to the main view again.

6.3.1.13.5.1.4 How to Manage Changed Business Catalog Dependencies After an Upgrade

Context

If you want to have transparency over all the changes to business catalog dependencies after an upgrade, proceed as follows:

Procedure

1. Select the *Manage Business Role Changes After Upgrade* tile on the SAP Fiori Launchpad to open the app. On the initial screen, view the *Business Catalog Dependencies* area to get an overview of what has changed with the last upgrade. Filter for certain change types if required, for example if you only want to see the removed or the added dependencies.
2. Download a csv file containing all recently changed business catalog dependencies if required.
3. Click on the required business catalog dependency to check how the changed dependency affects your business roles. The business roles affected by the changes to this dependency are displayed in a list. By clicking on the business catalog *Name* above the list, you can view a description of the business catalog and its newly added dependencies or recently removed dependencies after an upgrade.
4. To find more information about the affected business roles or to make further changes, you can cross-navigate to the *Maintain Business Roles* app by clicking on the link listed in the *Business Role ID* column.

6.3.1.13.5.1.5 How to Manage Deprecated Business Catalogs After an Upgrade

Context

If you want to view all deprecated business catalogs after an upgrade, you want to assign the successor business catalogs and adopt the relevant changes in your business roles, proceed as follows:

Procedure

1. Select the *Manage Business Role Changes After Upgrade* tile on the SAP Fiori Launchpad to open the app. On the initial screen, view the *Deprecated Business Catalogs* area to get an overview of what has changed with the last upgrade.
2. Download a csv file containing all recently deprecated business catalogs if required.
3. Click on the required business catalog to check how the deprecation affects your business roles. The business roles affected by the deprecated business catalog are displayed in a list. By clicking on the business catalog *Name* above the list, you can view a description of the deprecated business catalog and the successor business catalog.
4. Select the required business role and click *Adopt Changes* if you want to take over the changes to your business roles. Please note that you might lose your maintained restrictions by doing this. We recommend that you validate the business role after the automatic adoption. By adopting the changes, you add all the successors business catalogs to the selected roles. The deprecated catalog will be removed.

   > ⓘ Note
   >
   > If the deprecated business catalogs contain extensions and you click *Adopt Changes*, the included extensions will be removed. If you want to keep these extensions, you need to re-assign them.

5. Select the *Add the dependent business catalogs of the successor business catalogs* checkbox if required.
6. Click *OK*. The system displays a progress bar and a list of change results when it is done.
7. Click *Close* to return to the *Deprecated Business Catalogs* area.

6.3.1.13.5.1.6 How to Manage Business Role Template Changes After an Upgrade

Context

To get an overview of changes in the business role template area since the last upgrade, you can display any differences between business role and business role template in the *How to Manage Business Role Changes After an Upgrade* app. Either the template or the role that was based on the template might have changed. For example, business catalogs might have been added or removed. To apply these changes in a detailed maintenance screen, you can cross-navigate to the *Business Role Templates* app.

Procedure

1. Select the *Manage Business Role Changes After Upgrade* tile on the SAP Fiori Launchpad to open the app. On the initial screen, the *Business Role Templates* area provides an overview of what has changed with the latest upgrade.
2. Download a csv file containing all recently changed business templates if required.
3. Click on the required business role template for details of the changes. The type of change (for example *Different from Business Roles*) is already displayed on the initial screen.
4. To display a detailed comparison, select one or more affected business roles and click *Compare with Business Role Template*. This takes you to the *Business Role Templates* app.
5. All deviations of the business roles from the business role templates are displayed in a table. You can click *Apply All* if you want to take over all of the changes. Alternatively, you can go through the list and select deviations you would like to apply.

   You can define one role as leading and all changes you make to this role will be automatically transferred to the roles you selected in the *Deviations* table.
6. Click *Save* when you're done.

   Please note that you can also adjust your business roles directly in the *Business Role Templates* app.

Related Information

Business Role Templates [page 3094]

6.3.1.13.5.1.7 Phase-In / Phase-Out Status

If a restriction is in status *Phase-In*, the values have no effect in the system at the moment. Therefore, you can prepare your system for new restrictions that will be active in the next release.

The *Phase-Out* status denotes that the restriction will be removed in the next release.

6.3.1.13.5.1.8 How To View and Manage Changed IAM Apps After an Upgrade

Learn how to review IAM app changes after an upgrade. This includes added or removed IAM apps, updates to access categories and restriction types, dependency changes, and deprecations with successor IAM apps. Use the dedicated tabs to analyze impacted business catalogs and business roles. You can export change lists in CSV format to assess the impact and take corrective action.

Restriction Types

On the *Restriction Types* tab, you can view the restriction types that changed not only in business catalogs but also in IAM apps.

You can view the following types of changes:

- *Added*
- *Removed*
- *Access Category Changed*
- *Phase-Out*

In the *Affected Object Type* column, you can distinguish between business catalogs and IAM apps.

In the *Affected Objects* column, you can view the object names of the business catalogs or IAM apps.

To get more information about the change and a list of the *Affected Business Roles*, select a restriction type.

Business Catalogs that Support IAM App

On the *Business Catalogs that Support IAM App* tab, you can view all business catalogs that support IAM apps and download this list as a CSV file.

IAM Apps

On the *IAM Apps* tab, you can view IAM apps that have been added to or removed from a business catalog after an upgrade. You can also view the affected business catalogs or download all changed IAM apps as a CSV file. To get more information about the affected business roles, select an IAM app.

IAM App Dependencies

On the *IAM App Dependencies* tab, you can view dependencies that have been added to or removed from IAM apps after an upgrade. You can download all changed IAM app dependencies as a CSV file. To get more information about the affected business roles, select an IAM app.

Deprecated IAM Apps

On the *Deprecated IAM Apps* tab, you can view deprecated IAM apps, the deprecation release, and the successors. You can download all deprecated IAM apps as a CSV file. To get more information about the affected business roles, select an IAM app.

Affected Business Roles

On the *Affected Business Roles* tab, you can view the following information about IAM apps:

- Changed restriction types in IAM apps
- Changed IAM apps in business catalogs
- Changed IAM app dependencies
- Deprecated IAM apps

In the *Affected Object Type* column, you can see whether business catalogs or IAM apps are affected.

In the *Changes Object Type* column, you can see whether, for example, a business catalog or an IAM app are changed.

Related Information

Work with IAM Apps (App Authorization Variants)

6.3.1.13.6 Business Catalogs for IAM Administration Apps

Find out which business catalogs are required to access the IAM administration apps and what restrictions apply.

To grant a business user access to the IAM administration apps, include one of the following business catalogs or IAm apps in their assigned business role.

The following two types of business catalogs are available:

- Business catalogs that directly include authorizations specifying the actions a business user with a specific business role is permitted to perform.
- Business catalogs that serve as containers for IAM apps, which manage authorizations. These business catalogs are designated by the *Supports IAM Apps* checkbox.

> ### Note
>
> You can also directly assign IAM apps to business roles in the *Maintain Business Roles* app.

Business Catalogs for Identity and Access Management Apps

Business Role Catalog	IAM App	IAM App ID	Transaction	Transaction Code	Restrictions
User Management `SAP_CORE_BC_IAM_UM`	*Display Technical Users*	`F1301_TRAN`	Display Technical Users	`F1301`	None
User Management `SAP_CORE_BC_IAM_UM`	*Maintain Business Users*	`F1303_TRAN`	Maintain Business Users	`F1303`	Not possible to assign business roles to the business users
User Management `SAP_CORE_BC_IAM_UM`	*Display Business Roles*	`F1492_03_TRAN`	Maintain Business Roles	`F1492`	Not possible to assign business roles to the business users.
User Management `SAP_CORE_BC_IAM_UM`	*IAM Information System*	`F2450_TRAN`	IAM Information System	`F2450`	None
User Management `SAP_CORE_BC_IAM_UM`	*Display Restriction Types*	`F3816_TRAN`	Display Restriction Types	`F3816`	None
User Management `SAP_CORE_BC_IAM_UM`	*IAM Key Figures*	`F4474_TRAN`	IAM Key Figures	`F4474`	None

Business Role Catalog	IAM App	IAM App ID	Transaction	Transaction Code	Restrictions
User Management SAP_CORE_BC_IAM_UM	*Schedule Locks for Unused Business Users*	F6288_TRAN	Schedule Lock for Unused Business Users	F6288	None
Role Management SAP_CORE_BC_IAM_RM	*Maintain Business Roles*	F1492_02_TRAN	Maintain Business Roles	F1492	Not possible to assign business roles to the business users.
Role Management SAP_CORE_BC_IAM_RM	*IAM Information System*	F2450_TRAN	IAM Information System	F2450	None
Role Management SAP_CORE_BC_IAM_RM	*Business Catalogs*	F2471_TRAN	Business Catalogs	F2471	None
Role Management SAP_CORE_BC_IAM_RM	*Business Role Templates*	F2820_TRAN	Business Role Templates	F2820	None
Role Management SAP_CORE_BC_IAM_RM	*Display Restriction Types*	F3816_TRAN	Display Restriction Types	F3816	None
Role Management SAP_CORE_BC_IAM_RM	*Display Authorization Trace*	F4106_TRAN	Display Authorization Trace	F4106	None
Role Management SAP_CORE_BC_IAM_RM	*IAM Key Figures*	F4474_TRAN	IAM Key Figures	F4474	None
Role Management SAP_CORE_BC_IAM_RM	*Display IAM Apps*	F7500_TRAN	Display IAM Apps	F7500	None
Role Assignment SAP_CORE_BC_IAM_RA	*Assign Business Users to Business Roles*	F1303_22_TRAN	Maintain Business Users	F1303	Only possible to assign business roles to the business users. Not possible to change the business roles.

Business Role Catalog	IAM App	IAM App ID	Transaction	Transaction Code	Restrictions
Role Assignment SAP_CORE_BC_IAM_RA	*Assign Business Roles to Business Users*	F1492_22_TRAN	Maintain Business Roles	F1492	None
Role Assignment SAP_CORE_BC_IAM_RA	*IAM Information System*	F2450_TRAN	IAM Information System	F2450	None
Role Assignment SAP_CORE_BC_IAM_RA	*Display Business Catalogs*	F2471_TRAN	Business Catalogs	F2471	None
Role Assignment SAP_CORE_BC_IAM_RA	*Display Restriction Types*	F3816_TRAN	Display Restriction Types	F3816	None
Role Assignment SAP_CORE_BC_IAM_RA	*Display Authorization Trace*	F4106_TRAN	Display Authorization Trace	F4106	None
Role Assignment SAP_CORE_BC_IAM_RA	*IAM Key Figures*	F4474_TRAN	IAM Key Figures	F4474	None
Role Assignment SAP_CORE_BC_IAM_RA	*Display IAM Apps*	F7500_TRAN	Display IAM Apps	F7500	None
Role Management - Display SAP_CORE_BC_IAM_RM_DISP_PC	*Display Business Roles*	F1492_03_TRAN	Maintain Business Roles	*F1492*	None
Role Management - Display SAP_CORE_BC_IAM_RM_DISP_PC	*IAM Information System*	F2450_TRAN	IAM Information System	*F2450*	None
Role Management - Display SAP_CORE_BC_IAM_RM_DISP_PC	*Display Business Catalogs*	F2471_TRAN	Business Catalogs	F2471	None

Business Role Catalog	IAM App	IAM App ID	Transaction	Transaction Code	Restrictions
Role Management - Display `SAP_CORE_BC_IAM_RM_DISP_PC`	*Business Role Templates*	`F2820_TRAN`	Business Role Templates	F2820	None
Role Management - Display `SAP_CORE_BC_IAM_RM_DISP_PC`	*Display Restriction Types*	`F3816_TRAN`	Display Restriction Types	F3816	None
Role Management - Display `SAP_CORE_BC_IAM_RM_DISP_PC`	*IAM Key Figures*	`F4474_TRAN`	IAM Key Figures	F4474	None
Role Management - Display `SAP_CORE_BC_IAM_RM_DISP_PC`	*Display IAM Apps*	`F7500_TRAN`	Display IAM Apps	F7500	None
User Management - Display `SAP_CORE_BC_IAM_UM_DISP_PC`	*Display Technical Users*	`F1301_TRAN`	Display Technical Users	F1301	None
User Management - Display `SAP_CORE_BC_IAM_UM_DISP_PC`	*Display Business Users*	`F1303_03_TRAN`	Maintain Business Users	F1303	None
User Management - Display `SAP_CORE_BC_IAM_UM_DISP_PC`	*IAM Information System*	`F2450_TRAN`	IAM Information System	F2405	None
User Management - Display `SAP_CORE_BC_IAM_UM_DISP_PC`	*Display Restriction Types*	`F3816_TRAN`	Display Restriction Types	F3816	None

Business Role Catalog	IAM App	IAM App ID	Transaction	Transaction Code	Restrictions
User Management - Display `SAP_CORE_BC_IAM_UM_DISP_PC`	*IAM Key Figures*	`F4474_TRAN`	IAM Key Figures	F4474	None
Group Management `SAP_CORE_BC_IAM_GRP_PC`	*Maintain Business User Groups*	`F6399_TRAN`	Maintain Business User Groups	`F6399`	None
Group Management `SAP_CORE_BC_IAM_GRP_PC`	*Maintain Business Role Groups*	`F6461_TRAN`	Maintain Business Role Groups	`F6461`	None

6.3.1.13.7 Glossary for Identity and Access Management

Get an overview of the terminology used for identity and access management.

Terminology Overview

Term	Description
Access category	A category that defines what kind of access is granted to the users assigned to a business role, for example, *Read*, *Write*, or *Value Help*.
Business role	A business role provides users with authorizations to access apps.
Business catalog	Business catalogs grant access to an app, a set of apps, or individual aspects of an app.
Business role template	A pre-defined set of authorizations and assigned business catalogs you can use as a basis for creating new business roles.
Business user	Any person who can log on to the system, including administrators.
Restriction	An access limitation that is defined on business role level. You can determine what kind of access is granted to specific objects, such as *company code*.

Term	Description
Restriction field	A field that is used to restrict the access to a specific business object, for example, *organizational area*.
Restriction type	An authorization entity that bundles the available restriction fields to a logical definition, for example, company code.
Leading restriction	A property that defines a certain restriction field as leading restriction. No matter in which restriction types the restriction field is being used, the field will have the same restriction assigned in all types.
Technical user	A technical user corresponds to a local or remote process which is typically part of the cloud management process (e.g. system provisioning, support) or intrinsic system processes (e.g. periodic clean-up of logs). There are technical users that belong to the software or service provider and there are technical users that belong to the customer.
IAM app	A unit that grants access to a set of services and authorizations. An IAM app can be grouped in business catalogs and assigned to business roles.
External app	An IAM app based on services, for example derived from SAP Fiori launchpad app descriptor items. It contains services and authorizations.

6.3.1.14 Message Monitoring

Message Dashboard [page 3120]

Message Monitoring Overview [page 3121]

Message Monitoring for Integration Experts [page 3122]

Assign Recipients to Users [page 3124]

Message Monitoring (Emergency Correction) [page 3124]

SOAP Error Log, OData Error Log, and Event Error Log [page 3125]

Assigning the Business Catalog for SOAP Error Log, OData Error Log, and Event Error Log to Your Business Role [page 3126]

Automatic Reprocessing [page 3127]

6.3.1.14.1 Message Dashboard

With this app, you can get an overview of the interfaces you're responsible for and their current status. You can analyze the root causes of errors, restart or cancel data messages.

Key Features

You can use this app to:

- See all interfaces you've been assigned to using recipients.
- See the general status of your interfaces over time and restrict the time range.
- Check for active alerts and confirm them in the column *Alert*.
- Configure what notifications you want to receive for a specific interface in the column *Notification*. You can choose between *E-Mail for the Next Alert* (default), *E-Mail for Every Single Error*, and *No Mail*.
- See the message numbers and statuses (error, warning, success) by namespace and interface.
- Navigate to the message summary for an overview of how often a certain log message was written to the application log and how many data messages used a certain log message.
- Navigate to the monitoring and error handling to analyze the log messages in detail.
- Display the contents of the data messages.
- Create custom hints, message texts, and functions for certain log messages.
- Cancel data messages with issues that don't require you to take action (this functionality is only available for selected message formats).
- Solve errors by correcting the underlying data content of the messages.
- Restart corrected data messages or messages that had a temporary issue (this functionality is only available for selected message formats).
- Download payload for SOAP, IDoc, and XML messages.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-AIF`.

6.3.1.14.2 Message Monitoring Overview

With this app, you can see an overview of all the interfaces that you're responsible for monitoring. The intuitive card-based view provides a holistic overview of your message interfaces and also provides additional details like the number of messages in different statuses for each interface. When you click on the specific status for a specific interface, you're redirected to *Message Monitoring*. Here, you can see a filtered view of the messages in the status that you selected for the specific interface containing the log information and key field information. For example, in the *Message Monitoring Overview* app, if you're monitoring an interface *AIF_XXX_MMO* and in the card specific to *AIF_XXX_MMO*, you click on the status *Error*, you're redirected to the *Message Monitoring*

where you can see all the messages in status *Error* for the interface *AIF_XXX_MMO*. You also see the log information and key field information for the messages in the *Error* status. This improves the efficiency of you finding the list of messages in a specific status for a given interface.

> → Tip
>
> If you don't see the interface you're assigned to, verify that your user is assigned to the corresponding recipient using the *Assign Recipients to Users* app.

Key Features

This app provides the following key features:

- Monitoring overview of all the responsible interfaces
- Quick overview of key parameters of the selected messages
- Filter or search for messages with key content as the search parameters
- Navigate to *Message Monitoring* with appropriate filter conditions for specific interfaces and message status, significantly improving the user experience
- Order the cards to keep an eye on mission-critical interfaces using drag-and-drop mechanism
- Identify interfaces with an active alert

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-AIF`.

Related Information

Assign Recipients to Users [page 3124]

6.3.1.14.3 Message Monitoring for Integration Experts

With this app, you can display an overview of your interfaces and get details on all messages processed through them.

→ Tip

If you don't see the interface you're assigned to, verify that your user is assigned to the corresponding recipient using the *Assign Recipients to Users* app.

Key Features

You can use this app to:

- Get a tabular overview of your interfaces and all messages (with specific status) processed through them.
- Filter your overview by time and/or interface, and restrict it to show only interfaces with messages or errors.
- Drill down on the messages of a specific interface and filter them by time, status, and key fields (if defined).
- Get more information on an individual message (such as log messages, message details, and data structure).
- Download the payload of a specific message.
- Edit, restart, or cancel a message (if in status *Error* or *In Process*) (this functionality is only available for selected message formats).
- Perform mass error handling for multiple messages of an interface at once.
- Set process information for a message (such as defining a process status).
- Identify interfaces with an active alert and confirm the alert.
- Define data links and dynamic data links for log messages without a predefined link.
- Display the original payload of a message in the message's data content.
- View the log history of a message (only available for interfaces that use the application log engine)

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-AIF`.

Related Information

Assign Recipients to Users [page 3124]

6.3.1.14.4 Assign Recipients to Users

With this app you can assign your business users to the recipients that were defined for the communication scenarios. Once the users are assigned they can use the *Message Monitoring Overview* and *Message Dashboard* to monitor the interfaces and data messages they are responsible for.

Key Features

You can use this app to:

- Find and add users.
- Find and assign pre-defined recipients to these users.
- Define which types of log messages the user can see in the Message Dashboard, for example, warnings only.
- Define if the messages are displayed or hidden on the Message Dashboard.
- Define if the user can see messages that are in process or that had a technical error using *Advanced Error Handling*.
- Unassign recipients from users.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-AIF`.

6.3.1.14.5 Message Monitoring (Emergency Correction)

With this app, you can edit non-customized fields of a message payload in an emergency.

Key Features

You can use this app to:

- Get a tabular overview of your interfaces and all messages (with specific status) processed through them.
- Filter your overview by time and/or interface, and restrict it to show only interfaces with messages or errors.
- Drill down on the messages of a specific interface and filter them by time, status, and key fields (if any are defined).
- Get more information on an individual message, such as log messages, message details, and data structure.
- Download the payload of a specific message.
- Edit, restart, or cancel a message (if in status *Error* or *In Process*) (this functionality is only available for selected message formats).
- Set process information for a message (such as defining a process status).
- Identify interfaces with an active alert and confirm the alert.
- Edit the payload of messages, even if the messages aren't customized to be changeable.

Despite having similar functionalities as *Message Monitoring for Integration Experts*, *Message Monitoring (Emergency Correction)* doesn't support navigating to the *Message Monitoring* app or performing mass error handling.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-AIF`.

6.3.1.14.6 SOAP Error Log, OData Error Log, and Event Error Log

With these apps, you can view and filter error logs related to SOAP web services, OData services, and events using SAP Application Interface Framework.

Key Features

You can use these apps to:

- Get a comprehensive overview of failed communications relating to **SOAP web services**, **OData services**, and **events**
- Filter the SOAP Error Log, OData Error Log, and Event Error Log for various capabilities, such as Message ID, User Name, or Processing Date

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-AIF`.

6.3.1.14.7 Assigning the Business Catalog for SOAP Error Log, OData Error Log, and Event Error Log to Your Business Role

Context

If you want to display the technical monitoring apps SOAP Error Log, OData Error Log, and Event Error Log in SAP Fiori launchpad, your user needs business roles that are authorized for the relevant business catalog.

Procedure

1. In SAP Fiori launchpad, go to *Identity and Access Management* > *Maintain Business Roles*.
2. Select *Business Role*, then click *Edit*.
3. In the tab *Assigned Business Catalogs*, select *Add*.
4. In the pop-up *Assigned Business Catalogs*, select the following business catalog: *Communication Management – Technical Message Monitoring SAP_CA_BC_COM_TECH_ERR_PC*.

5. Click *OK*, then select *Save*. The tiles *SOAP Error Log*, *OData Error Log*, and *Event Error Log* are now visible in SAP Fiori launchpad.

6.3.1.14.8 Automatic Reprocessing

With this app you can configure the automatic reprocessing that is executed when a data message fails with a specific log message.

Key Features

You can use this app to:

- Create, edit, and delete automatic reprocessing settings for a specific combination of log message and interface. Whenever a data message fails with the selected log message, the configuration you defined is executed.
- Define a maximum number of reprocessing attempts that can be executed for a failed data message.
- Define a minimum waiting time that must pass before the application can trigger a reprocessing attempt. The waiting time can take longer depending on the interval of the background job that executes the reprocessing.
- Define the intermediate status (In Process or Error) the failed data message displays while it's waiting to be reprocessed.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-AIF`.

6.3.1.15 Output Management

Maintain Print Queues [page 3128]

Monitor Email Transmissions [page 3139]

6.3.1.15.1 Maintain Print Queues

Purpose

Using this app, you can set up print queues to manage the printing of documents and monitor the print jobs in each queue.

This helps you to identify and analyze errors and gives you a sense of direction for doing troubleshooting to solve them.

Key Features

With this app you can:

- Create a new print queue
- Delete a print queue
- Pause a print queue
  This allows you to stop print items from being collected by the printing process while you are carrying out repairs.
- Restart a print queue
- Check the status (such as *New* , *Failed* , *Transmitted* , *Successful*, *Warning*) of queues or individual print items in the queue and display detailed information regarding the status
- Reassign a print item to another print queue

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-CCM-PRN-OM-PQ` or `BC-CCM-PRN-OM-PM`.

Related Information

Install Additional Software (SAP BTP ABAP Environment)
Install Additional Software (SAP S/4HANA Cloud)
Integrating SAP BTP ABAP Environment and Local Printers
Integrating SAP S/4HANA and Local Printers
Printing (SAP BTP ABAP Environment)
Printing (SAP S/4HANA Cloud)

6.3.1.15.1.1 Before Getting Started

Background Information for Print Queues

In a Cloud environment, the back-end system does not have a connection to local printers in your network (no virtual private network access is available, for example). To establish this connection, you need to create a print queue in the Cloud system representing an output channel to a local printer. To do so, open the *Install Additional Software* app and install the `SAP Cloud Print Manager for Pull Integration` (hereafter abbreviated as CPM) in your network. The CPM can then regularly check whether new print items are in the print queue. If this is the case, the CPM retrieves these items and sends them to the locally configured printer. In addition to that, you need to define communication scenario `SAP_COM_0466`. For more information on the CPM, see 3048273 or Working with the SAP Cloud Print Manager for Pull Integration [page 3130].

Watch the following video to learn how to get started with printing:

Related Information

SAP_COM_0466 (SAP S/4HANA Cloud)
SAP_COM_0466 (SAP BTP ABAP Environment)

6.3.1.15.1.2 Creating Print Queues

You want to set up print queues to manage the printing of documents.

Procedure

1. Open the *Maintain Print Queues* app.
2. Click *New* to create a new print queue.
3. Select a print manager.
4. Enter the required information:
   - **Queue**: Print queue name
   - **Description**: Print queue desciption
   - **Format**: The format is the printer language type of the queue. The following language types are available: `PDF, CAB 203 dpi, CAB 300 dpi, Datamax 203dpi, Datamax 300 dpi, Datamax 406 dpi, Datamax 600 dpi, HP Laserjet 4350 PCL 5e, HP LaserJet 4350 PS, PCL 5c (Color), PCL 5e (Monochrome), Intermec 203 dpi, Intermec 300 dpi, Intermec 400 dpi, Lexmark T644 PCL 5e, Lexmark T644 PS, PostScript 2, Postscript 3, Toshiba 203 dpi, Toshiba 305 dpi, XMLDATA, Zebra 203 dpi, Zebra 300 dpi, Zebra 600 dpi.`
   - **Communication User**: Technical user with which the *SAP Cloud Print Manager for Pull Integration* logs on to the system. You have to define a communication user in the communication scenario

`SAP_COM_0466` first. You can connect several queues to the same *SAP Cloud Print Manager for Pull Integration* - in this case all queues must use the same communication user.

- **Retention**: Spool Retention Period. The retention period is the number of days after which the print queue items that are in final state (*successful* or *failed*) will be deleted automatically.

5. Click *Create* to create a new print queue.

Results

You have created a print queue you can use in the *SAP Cloud Print Manager for Pull Integration*.

> ⓘ Note
>
> The *Maintain Print Queues* app always includes a print queue called *DEFAULT*. It's the only print queue provided by SAP and serves as a sample queue that can't be used for productive output as it can't be connected to the *SAP Cloud Print Manager for Pull Integration*.
>
> To connect to your physical printers, you therefore need to create one or more custom print queues.
>
> Keep in mind that you can't define custom retention time for items in the *DEFAULT* queue and that SAP might adapt the standard retention time at any time to avoid misuse. The *DEFAULT* print queue should also not be used as storage for all documents that are printed from the system, as SAP might restrict the given quota at any time to avoid misuse. A cleanup job will delete all items in the *DEFAULT* print queue after eight days, regardless of their status. This will not affect the behavior of other print queues.

Related Information

SAP_COM_0466 (SAP S/4HANA Cloud)
SAP_COM_0466 (SAP BTP ABAP Environment)

6.3.1.15.1.3 Working with the SAP Cloud Print Manager for Pull Integration

You can use the `SAP Cloud Print Manager for Pull Integration` (from now on abbreviated as CPM) to establish a connection between an SAP Cloud-based solution and printers available in your network. It requires the CPM to enable automatic transmission and printing of documents from the cloud-based system to local printers.

You should install the CPM once in your network. Download the CPM installation package from the *Install Additional Software* app and select *Run as Administrator*.

> ⓘ Note
>
> Keep in mind that you need to have administrator rights in order to proceed with the installation. The installation should be performed on a Windows server.

Start the downloaded installer executable file. The `SAP Front-End Installer` opens. Select CPM and click *Next*. Wait until the installation has finished and confirm the final dialog. The CPM is now installed.

As a Windows® service, CPM is available for print queue output from the cloud-based system as long the Windows® computer is up and running and the CPM Windows® service has not been stopped manually. Thus, we recommend to install CPM on a central print server that is up and running all the time or at least during the time in which printing is needed.

You only need to install CPM more than once if you have multiple separate networks with printers that you want to use for process-integrated printing.

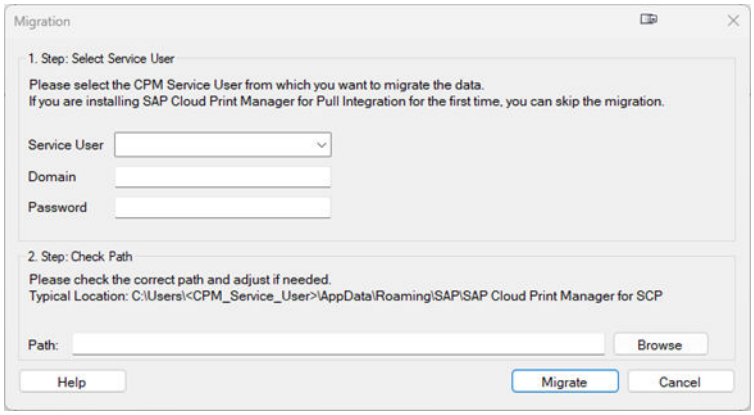
Prerequisites

- **Prerequisites on the Print Server**
  You have administration authorization for the computer on which you install CPM.
  Note that when CPM collects documents from a print queue and sends them to a printer, the documents are stored temporarily on the server CPM is installed on. If you are using CPM to print sensitive documents, it is recommended that you ensure that only authorized users can access this folder. The folder, called the working directory, can be found in the following path on the Windows® computer running CPM:
  `C:\ProgramData\SAP\Cloud Print Manager for PI`
  Since the CPM can run under a different user than the `SAP Cloud Print Service`, the working directory is stored under the `ProgramData` folder as it can be accessed from both users.
  For the storage of the temporary documents and the configuration, the embedded `NoSQL` database `LiteDB` is used. `LiteDB` is a serverless database fully written in `.NET C#` managed code. The stored data can be viewed with the LiteDB.Studio .
- **Prerequisites in the SAP Cloud Systems**
  In S/4HANA Cloud the integration with CPM is done with an appropriate communication scenario. Please follow the instructions in Integrating SAP S/4HANA Cloud and Local Printers. For SAP Business Technology Platform please get more information from the `SAP Print Service` documentation on how to integrate your system with SAP Cloud Print Manager for Pull Integration.

Migrate to Release 800

Starting with release 800 of the CPM some major changes have been made that require a migration. For this, a new migration tool was introduced. At the first start the migration tool is launched automatically.

> **Note**
>
> Please make sure that the print service is stopped, and that the migration process is started during a phase with low printing amount.



1. Here you must select the CPM service user that you have previously used and from which you want to migrate the data. It is also required to type in the corresponding password to perform the migration.
2. Please verify that the path to the CPM application data is filled and correct. Usually, nothing needs to be changed here.

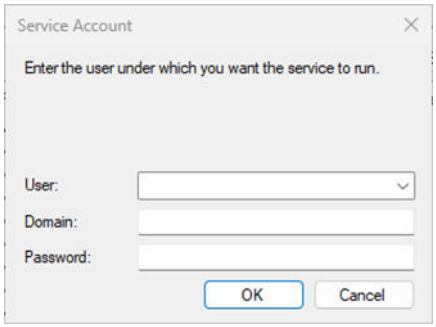
After the migration is done, the service user selected in the migration tool is automatically set as CPM service user as well. The service user can be changed as described here: Setting Up the SAP Cloud Print Manager for Pull Integration [page 3132].

6.3.1.15.1.3.1 Setting Up the SAP Cloud Print Manager for Pull Integration

In the following subsections, you find a description of the different steps necessary to set up the CPM.

Define Connection Settings

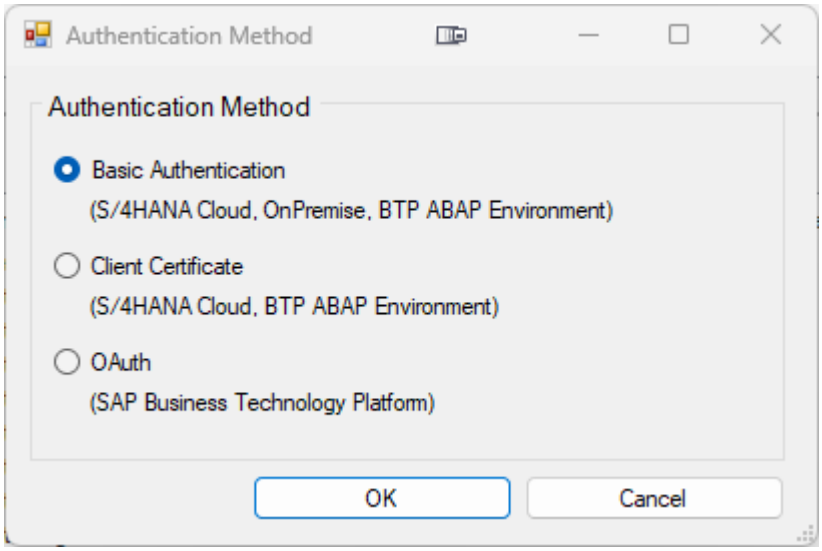
1. Start the CPM application from the Microsoft Windows® start menu.
2. Set or change the service user account (optional step) (This step is only required if the service user was not already set during the migration process.)
   At the first start of the CPM, the service account dialog opens automatically. In case the dialog doesn't open, or you want to change the service user, you can select *Service User* from the *Settings* menu. Enter a service user to run the Microsoft Windows® service. This user should have administration rights on the local computer as well as authorization to access your network printers. The user dropdown shows only local user accounts, but you can also use a domain user by simply filling in the required fields. We recommend to use a local user account as service user.



3. **Optional step:** From the settings menu, select *Proxy* to set up a connection to the internet. It is recommended to use the same configuration required by your company to access the internet. Mostly, the default setting is sufficient and there is no need to change anything.
   After applying these settings, the print service is started and runs as the Windows® service CPM. In addition, as a second Windows® service, you can start the CPM to check if the print service is running. If it's not running, for example, due to a crash, the CPM restarts the print service to ensure permanent availability of the service. If necessary, you can restart the print service if it has stopped for any reason. You can start both services in the top right corner of the screen.

Add a New Runtime System

From the runtime system menu, select *New* to add a new runtime system to the CPM.



Basic Authentication

This method can be used for `S/4HANA Cloud` systems, `OnPremise` or `BTP ABAP Environment` systems.

1. Select *Basic Authentication* and confirm the dialog.
2. Enter the following information:
   - Name: Enter a unique name for the system.
   - SAP Web Service URL: Enter the URL to your system provided by SAP.
   - User: Enter the communication user you created while setting up the communication scenario for CPM.
   - Password: Enter the communication user's password.
3. Click *Test* to test the connection to the system and the user credentials.
4. Click *OK* to save the new configuration.

Client Certificate

This method can be used for `S/4HANA Cloud` systems or `BTP ABAP Environment` systems.

1. Select *Client Certificate* and confirm the dialog.

2. Enter the following information:
    - Name: Enter a unique name for the system.
    - SAP Web Service URL: Enter the URL to your system provided by SAP.
    - Certificate: Select the client certificate that should be used to establish the connection. The dropdown list displays all the certificates installed in the *Personal* folder of the Windows® machine certificate store. The client certificate must be signed by an appropriate certification authority (CA) and uploaded to the communication user in the S/4HANA Cloud system. A list of all root CAs approved by SAP Global Security is available in SAP Note 2801396
3. Click *Test* to test the connection to the system and the user credentials.
4. Click *OK* to save the new configuration.

OAuth

This method can be used for `SAP Business Technology Platform`.

1. Select *OAuth* (SAP Business Technology Platform) and confirm the dialog.
2. Enter the following information:
    - Name: Enter a unique name for the system.
    - SAP Web Service URL: Enter the URL to your system provided by SAP.
    - Client ID: Enter the client ID you received during the registration.
    - Client Secret: Enter the client secret you received during the registration.
    - Token URL: Enter the token URL you received during the registration.
3. Click *Test* to test the connection to the system and the user credentials.
4. Click *OK* to save the new configuration.
   As soon as you have added the runtime system, CPM starts to collect print queues from the SAP Cloud system. All print queues that have been assigned to the communication scenario or user are displayed in CPM. You now need to assign Windows® printers to the print queues to start printing. If you don't immediately see any print queues, select *Check Now* from the *Runtime System* menu to start the connection.

Remarks

- It is not supported to connect to the same runtime system with the same communication user multiple times.

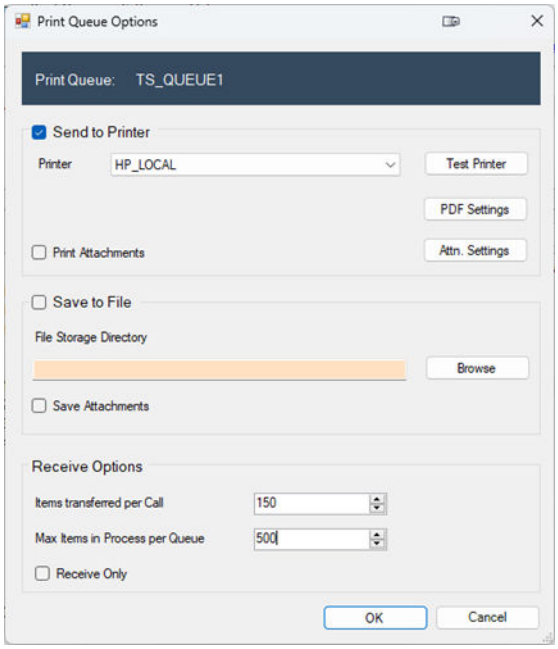
Assign a Printer to a Print Queue

1. Under *Runtime Systems*, select a system from the list. All print queues that have been assigned to the technical user are displayed in this list.
2. Under *Print Queues*, select a print queue and choose *Queue Options* from the *Print Queue* menu or use the context menu or a double click on the queue. The *Print Queue Options* dialog opens. Mark the checkbox *Send to Printer* and save the file.
3. Select a Windows® printer and browse or enter a path and click *OK* to save the configuration.

Note that the printer that you assign to the queue should support the printer language defined in the queue. You can view this information in the *Format* column of the *Print Queues* list. Do not select a printer that requires user interaction for each print job. For example, you should not select a printer that prints to a file with a file selection popup. There is no user interface to the service, so the job wouldn't print.

As soon you have assigned a printer to the print queue, the documents that have been sent to this print queue will be collected and printed.

CPM can print or save attachments which were added to the document. Attachments can be saved in any format, but you can only print PDF, DOCx, XLSx and PPTx attachments. To save attachments to the specified path, select the checkbox *Save Attachments*.

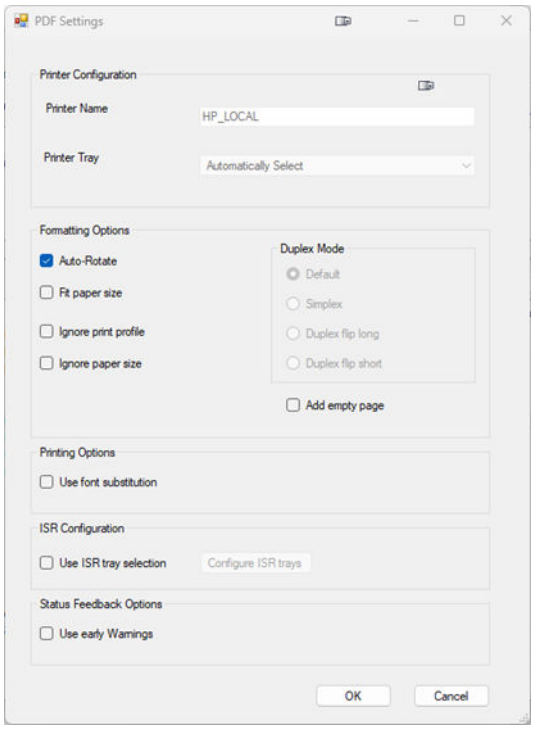


> ### Note
>
> Mapped network drives might not work correctly if chosen from the *Browse* dialog. In this case, enter the UNC path name (`\\<server_name>\<directory_name>`) manually in the entry field for the file storage directory on the left side.

Test Printer

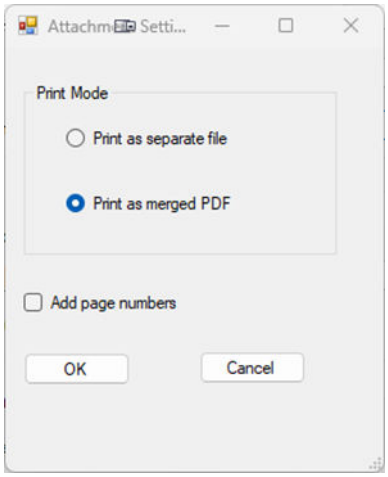
1. Under *Print Queues*, select a *Print Queue* and choose *Output Options* from the print queue menu.
2. Ensure that *Send to Printer* is checked and a valid printer is selected.
3. Press the *Test Printer* button. A test page is sent to the defined printer. The test page contains a test sentence in English, Chinese, French, and German.

PDF Settings



1. Click on *PDF Settings* to define print settings for your PDF document. This button is only available for queues with a PDF format.
2. Choose between the following options:
    - Auto-Rotate: the print settings are adapted in accordance with the orientation of the document that you want to print.
    - Fit paper size: the print settings are scaled in accordance with the paper size in the paper tray while preserving the aspect ratio of your original document.
    - Ignore print profile: CPM sends printer specific properties of the attached printer to the print queue in the SAP Cloud system. If supported from the application, these can be used to define print profiles for a print queue item. Print profiles can consist of settings like duplex mode, color mode, tray selection and others. To use the global default settings of CPM, select this option.
    - Ignore paper size: sending a document with invalid paper size (for instance, a letter to an A4 tray) usually causes the printer to ask for a manual confirmation to print. To prevent this, select this option.
    - Duplex Mode: global print property (see ‘Ignore print profile’)
    - Add empty page: When selecting this option, an empty page is added after a document with odd pages is printed in duplex mode.
    - Use font substitution: see SAP note 2959085 for details.
    - Use ISR tray selection: configure swiss payment form printing. Do not use this for any other printing.
    - Use early warnings: sends early status feedback to the S/4HANA Cloud or SAP BTP print queue in case of a printer issue. For example: If the printer is out of paper, all items will be set to warning.
3. Confirm with *OK*.

Attachment Settings



1. Click on *Attachment Settings* to define print settings for your attachments.
2. Choose between the following options from the print mode:
    - Print as separate file: All attachments are printed separately keeping their original document format.
    - Print as merged PDF: All attachments are merged into one PDF and are printed in accordance with the selected PDF settings (see section PDF Settings).
3. Additionally, you can select *Add Page Numbers* for merged PDFs to add page numbering.
4. Confirm with *OK*.

Receive Options

Below, you'll find a list of all available Receive options:

- *Items transferred per Call* specifies the number of print queue items which are transferred in one request using the new print API.
- *Max. Items in Process per Queue* specifies the maximum number of print queue items that can exist in the internal queue of CPM. If the limit is reached, no new items will be received until some of them were processed. This behavior should prevent memory overflow in case of problems with a printer.
- *Receive only* is a test option to measure the performance of the item transfer from the SAP Cloud system to CPM. Do not select this option in a productive environment. Nothing is printed or saved. Create many items in a test queue, set the *Log Level* to *Info* (see section Other Functions) and select *Receive only*. After all print queue items were received, you can find a time stamp for each block of items in the *Error Log*. The block size is the number specified in *Items transferred per Call*.

Other Functions

View Log Information

1. From the Help menu, select *About*.

2. Using the links provided in the *About* CPM dialog, you can access the following information:
    - Document Log: All documents retrieved from the configured runtime system.
    - Error Log: General errors.
    - Printing Log: Errors during print job processing.
3. If you need to provide more detailed log information, select *Options...* from the *Settings* menu, set the *Log Level* to *Info* or *Debug* and mark the checkbox named *Keep Job-Specific Log Files*. Then reproduce the error. This will add more detailed error information to the log files.
4. If required, click *Save Support Information* to create an archive file containing all log information, which can be attached to a support incident on the application component `BC-CCM-PRN-OM-PM`.

Cleanup Directories

1. From the settings menu, select *Clean up Directories*.
2. All documents and log files stored in CPM's working directories will be deleted.

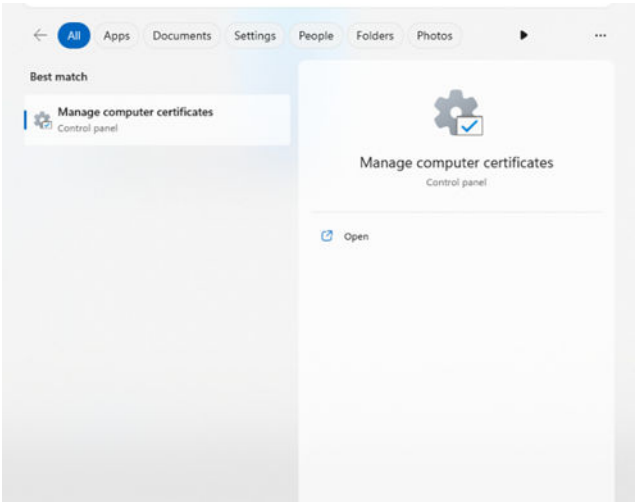
Reset to Factory Defaults

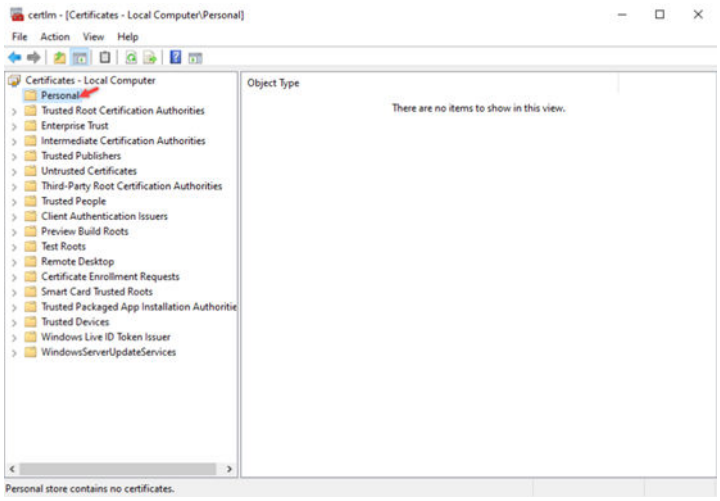
1. From the *Settings* menu, select *Reset to factory defaults*.
2. All documents and log files, including the configuration, will be deleted. After performing this action, all the system information, including user and password, must be entered again.

Appendix

Self-signed Certificate

To use self-signed certificates, they must be installed in the Windows® computer Certificate Store under the *Personal* folder. After the certificate is installed successfully, the CPM service must be restarted. You can then add connections to the system using the self-signed certificate.





6.3.1.15.2 Monitor Email Transmissions

Use

You can use this app to display successful and failed email transmissions and to provide background information. This app is read-only and doesn't allow any data changes.

Key Features

With this app, you can:

- Search for a specific email by adapting filters
- Sort, add, or remove columns
- Display all outgoing email requests
- Export a results list as a spreadsheet
- Get the header data and information about the sending status on a specific email request
- Send a test email to check if the email outbound in a system is working

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-COM`.

Related Information

Monitoring Email Transmissions [page 3140]

6.3.1.15.2.1 Monitoring Email Transmissions

Find out how to monitor your email transmissions.

Procedure

Start the *Monitor Email Transmissions* app. On the starting page, you can search for a specific email by adapting the following filters:

- Subject
- Status
- Sender
- Created at
- Recipient Address

In addition, you can

- Sort columns
- Add or remove columns

Click *Go* to display the search results. In the results list, all email requests are displayed according to the filters you've set. You can export the results list as a spreadsheet.

To get more information on a specific email request, click a request in the results list. On the *Details* page, the following information is displayed:

- Status
- Sender
- Status Code
- Subject
- Created at
- Response

You can also get information about the recipients, such as:

- Latest status

- Recipient address
- Copy (BCC or TO)
- Latest status changed at

You can get additional information about the present and former recipient's status by selecting a recipient.

Email Reorganization

Due to data protection reasons, all displayed emails will be deleted from the table view after 30 days.

Testmail Functionality

You can send out a test email to any recipient chosen by you. The subject and the text are fixed values that are set by the app. The mail sender will automatically be set as the email address of the user currently logged on, or, if that user has no email address, as the default email address. After the successful creation of the send requests, the email can be viewed in the main table of the *Monitor Email Transmissions* app.

Sending test emails can help administrators to check if the email outbound of their system is working. If the email outbound is not working, you'll either directly receive an error message, or you can see the error message in the main view of the *Monitor Email Transmissions* app.

Related Information

Monitor Email Transmissions [page 3139]
Configuration of the System Email Outbound

6.3.1.16 Read Access Logging

Definition

Read Access Logging (RAL) is used to monitor and log read access to sensitive data. This data may be categorized as sensitive by law, by external company policy, or by internal company policy. The following typical questions might be of interest for an application that uses Read Access Logging:

- Who accessed the data of a given business entity, for example a bank account?
- Who accessed personal data, for example of a business partner?
- Which employee accessed personal information, for example religion?

- Did anyone search, for example, if VIPs were admitted to hospital?
- Which accounts or business partners were accessed by which users?

These questions can be answered using information about who accessed particular data within a specified time frame. Technically, this means that all remote API and UI infrastructures (that access the data) must be enabled for logging. Read Access Logging is available for different channels such as Dynpro and Web Dynpro.

Main Purposes of Read Access Logging

Read Access Logging is often required to comply with legal regulations or public standards such as data protection and privacy, for example in banking or healthcare applications. Data protection and privacy is about protecting and restricting access to personal data. In some countries, data protection and privacy regulations even require that access to certain personal data is reported. Companies and public institutions may also want to monitor access to classified or other sensitive data for their own reasons. If no trace or log is kept on who accesses data, it is difficult to track the person responsible for any data leaks to the outside world. Read Access Logging provides this information.

Read Access Logging is always based on a logging purpose that is freely defined according to the requirements of an organization (for example, data protection and privacy). This logging purpose is then assigned to each log entry as an attribute, which allows the log data to be classified and organized according to the logging purpose. For example, various archiving rules or reportings can be created based on logging purposes.

The Read Access Logging framework can thus be used to fulfill legal or other regulations, to detect fraud or data theft, for auditing purposes, or for any other internal purpose.

Background Information on the Architecture of Read Access Logging

When an application is started, the Read Access Logging configuration is read. It indicates whether, for example, the current function module, operation, or UI element is log-relevant and to what extent. For example, should only the access itself be logged, or the content too? Log entries can be structured according to their semantics.

SAP applications may contain predefined configurations. However, the customer's key user will usually have to adapt them to his or her needs in order to fulfill the legal requirements of his or her organization - requirements that are not all known to SAP.

> ⓘ Note
>
> - As with all logging, your need to log data must be balanced with the effect logging will have on the performance of your system. The performance of your system will depend upon the amount of data you log, as well as the complexity of the conditions you specify for which data is logged.
> - You can find information on Read Access Logging configurations and templates in specific areas in the documentation for those areas.

Activities

Configuration Tasks

1. Determine which data must be logged under which circumstances.
   The organization must define which legal or security requirements to apply and which data must be logged on a semantic level. For example, the legal department might determine that access to the social security number must be logged.

2. Define purposes for Read Access Logging.
   Depending on the logging purpose and the laws and regulations that must be fulfilled, the application can define reports and rules for saving and processing the data. For more information, see How to Define Purposes of Read Access Logging [page 3146].
3. Determine the channels (such as Dynpro or Web Dynpro) through which the data can be accessed.
4. Define log domains.
   Log domains are groups of semantically related data fields that need to be logged. For example, *Gross salary* and *Net salary* might be grouped together in the *Salary* log domain. Log domains bundle different technical representations of the same semantic entity. Log domains are channel-independent. For more information, see How to Define Semantic Grouping of Fields to Be Logged [page 3147].
5. Define which data needs to be logged and whether only the access is logged or also the content.
   For more information, see How to Define What to Log [page 3150].

Tasks During Operations

- Defining a user exclusion list for Read Access Logging configurations.
  If you want to exclude certain users from Read Access Logging, they need to be added to a user exclusion list. This is useful for any automatic processing without user interaction, for example, background jobs. For more information, see How to Exclude Specific Users from Read Access Logging [page 3166].
- Displaying changes made to Read Access Logging configuration and evaluating errors and warnings.
  For more information, see How to Check Runtime Errors and See Changes to Configurations [page 3166]

Monitoring

You can view all log entries using the Read Access Logging monitor. For more information, see How to Monitor the Read Access Log [page 3169].

For monitoring in external systems, for example Security Information and Event Management (SIEM) systems, you can replicate read access logs by integrating the Read Access Logs API [page 3170].

Related Information

Data Protection and Privacy (62Y)

6.3.1.16.1 Read Access Logging for Data Protection

You can use Read Access Logging (RAL) to monitor and log access to personal data. The information provided may include, for example, which business users accessed business partner personal data, and in which time frame.

SAP delivers default configurations, which assign dedicated fields to log domains. A log domain is a category that groups semantically identical or related data fields. Logging occurs for all fields disclosed on the UI that are related to these domains. The domain is displayed in the log.

You can activate or deactivate the available RAL configurations in the *Read Access Logging Configuration* app, or make changes. You should carefully consider which information is relevant for logging. Configure your system to log only what you really control. If you maintain a wide scope of information to be logged, you will

end up with a lot of data that will be more difficult to process than if you are more specific in your logging configurations.

The following table lists the log domains that are part of the delivered sample configuration content. You can use these log domains or define your own, according to your needs.

Log Domain Name	Description
BANK	Data referring to a bank account
BIOMETRIC*	Data referring to biometric data
CRIME*	Data referring to criminal or administrative offenses or suspected criminal or administrative offenses
ETHNIC_ORIGIN*	Data referring to racial or ethnic origin
GENETIC*	Data referring to genetic data
HEALTH*	Data referring to health data
POLITICAL_OPINION*	Data referring to political opinion
PROFILE*	Data which is usually based on profiling like: scoring, rating, unwanted customer
RELIGION*	Data referring to religious or philosophical beliefs
SECRECY*	Data referring to professional secrecy
SEX_LIFE*	Data referring to sex life
SEXUAL_ORIENTATION*	Data referring to sexual orientation
SSN	Data referring to social security number
TRADE_UNION*	Data referring to trade union membership

These log domains are logged with fields related to business partner (`BP`), customer (`CUSTOMER`), supplier (`VENDOR`), legal entity (`LEGAL_ENTITY`), employee (`EMPLOYEE`), or student (`STUDENT`) because they are only identifiable with this additional personal data. For example, `TRADE_UNION` data requires details on the `EMPLOYEE`.

Log conditions are used, if required, to limit the logging of data (in technical terms, these fields are considered as **and** conditions). For example, you could configure RAL to log a field related to the employee in a specific transaction only if the employee's religion is shown. If this field is only visible on a tab where the employee's religion is not displayed, access to this field is not logged.

RAL is switched off by default. You can activate it in the respective section of the *Read Access Logging Configuration* app. In the *Read Access Logging: Monitor* app, you can view created logs. If you need a downloaded version of your RAL logs, please contact the Service Center. Also contact the Service Center if you need to create new configurations.

More Information

For more information on Read Access Logging and the related apps, see Read Access Logging [page 3141].

6.3.1.16.2 Read Access Logging Configuration

With this app you can configure Read Access Logging to determine which read access to data is logged and under which conditions.

> ⚠ Restriction
>
> RAL items can not be created in the P system, only in the Q system. If you try to create configurations, log domains, purposes or recordings in the P system, you will get an error message saying `Items of type SRAL_<RAL Item> must not be edited in this system`. You can still activate and deactivate RAL items in the P system.

Key Features

You can use this app to:

- Define purposes of Read Access Logging
- Define semantic grouping of fields to be logged
- Manage recordings of application user interfaces
- Change predelivered configurations of what to log
- Enable Read Access Logging in current client
- Check runtime errors and see changes to configurations

Prerequisites

To access the app, you need to have the `SAP_CORE_BC_RAL_CONF` business catalog assigned to your user. This business catalog is contained in the `SAP_BR_ADMINISTRATOR` business role template.

Supported Device Types

- Desktop
- Tablet
- Smartphone

Component for Customer Incidents

If you need support or experience issues, please report an incident under component BC-SEC-RAL.

6.3.1.16.2.1 How to Define Purposes of Read Access Logging

Read Access Logging is always based on a logging purpose that is freely defined according to the requirements of an organization. It describes why specific data is logged.

Context

In the configuration, you specify the logging purpose and each log entry in the log is assigned its purpose as an attribute. This configuration enables you to organize the log data by the logging purpose. For example, various archiving rules or reportings can be created based on logging purposes.

In each Read Access Logging configuration, assign a logging purpose. When you define archiving rules, use the logging purpose as the basis.

Procedure

1. In the *Read Access Logging Configuration* app, choose *Logging Purposes*.
2. Decide whether to create a purpose or modify one of the templates delivered by SAP.

   SAP delivers default purposes you can copy and modify to your needs or you can create your own.

   - To create your own purpose, choose *Create* under *Search Results*.
   - To copy a purpose from SAP, do the following.

   a. Under *Search Criteria*, choose *Search*.
   b. From the search results, select a template from SAP.

      Templates from SAP use the *(SAP Template)* icon under *Owner* or list SAP under *Created By* or *Changed By*.
   c. Under *Actions*, choose *(Copy)*.
3. Enter the required data.

   The ID is limited to ten characters. We recommend that you use an abbreviation of the purpose name as ID.

   The purpose name appears on all UIs.
4. Save your entries.

Results

Assign logging purposes during the creation of Read Access Logging configurations. Logging purposes help you with the definition of archiving rules.

6.3.1.16.2.2 How to Define Semantic Grouping of Fields to Be Logged

Within an application, the data to be logged must be defined on a semantic level, before the actual fields and rules are defined. Log domains are semantic descriptions of semantically identical or related fields that have different technical representations.

Context

In Read Access Logging, you define a log domain. When changing Read Access Logging configurations, assign a log domain to each field to be logged.

For a log domain, specify a name and a business area that the data element is related to. This definition is necessary because different applications might use the same log domain. For example, a log domain *account* can be something different in the Human Resources application than it is in the Banking application.

Procedure

1. In the *Read Access Logging Configuration* app, choose *Log Domains*.
2. Decide whether to create a log domain or modify one of the templates delivered by SAP.

   SAP delivers default log domains you can copy and modify to your needs or you can create your own.

   - To create your own log domain, choose *Create* under *Search Results*.
   - To copy a log domain from SAP, do the following.

   a. Under *Search Criteria*, choose *Search*.
   b. From the search results, select a template from SAP.

      Templates from SAP use the *(SAP)* icon under *Owner* or list SAP under *Created By* or *Changed By*.
   c. Under *Actions*, choose *(Copy)*.
3. Enter the required data.

   You can freely define the business area. The business area functions as a namespace for the data element.

   The description appears in the detailed view of the read access log and can be helpful for the person evaluating the log to identify the log domain.
4. Save your entries.

Results

When you change a configuration, you can assign a log domain to each field to be logged. The log domain is displayed in the read access log and helps the log evaluator to understand the semantics of the field. It enables the evaluator to search for a given semantic field regardless of the technical representation or ID of this field.

6.3.1.16.2.3 How to Manage Recordings of Application User Interfaces

To use Read Access Logging with user interface technologies like Web Dynpro and Dynpro, you first identify the log-relevant fields. Read Access Logging provides a user interface recorder to identify those fields.

Context

When read access to remote APIs is logged, their contents are already known at design time. All fields of a Web service are therefore available for read access logging configuration. For user interface technologies however, the elements that are visible are not known until the application is started. Therefore, it is not possible to offer all fields for read access logging configuration. First, determine which fields are needed for read access configuration. A user interface recorder, which is started from the *Read Access Logging Configuration* app, helps you. When the recorder runs, the key user navigates the user interface and records every UI element that is relevant for read access logging.

> ⓘ Note
>
> The value of the fields that are displayed on the UI during the recording (the so-called sample values) become visible in the recording. The sample values and UI labels help the key user to identify the fields when changing a configuration. However, you can rerecord or delete the sample values. For example, if you find it more helpful for the key user, you can replace them with a description of the field. You can also search for one or more recordings and clear all sample values at once. Select one or more recordings from the search results and choose *Clear Sample Values*.

> → Tip
>
> During the recording, the field labels are recorded in the UI language in which the recording is made. These labels are displayed in the configuration using the same language of the user interface recording. We recommend that you perform the recording in a language that the key user, who creates the configuration, understands.

Procedure

1. In the *Read Access Logging Configuration* app, choose *Recordings*.

2. Decide whether to create a recording or modify one of the default recordings delivered by SAP.

   SAP delivers default recordings you can copy and modify to your needs or you can create your own.

   - To create your own recording, choose *Create* under *Search Results*.
   - To copy a recording from SAP, do the following.

   a. Under *Search Criteria*, choose *Search*.
   b. From the search results, select a template from SAP.

      Templates from SAP use the *(SAP Template)* icon under *Owner*.
   c. Under *Actions*, choose *(Copy)*.
3. Enter the required data.
4. Save your entries.
   - If you copied a recording, you can use the copy as is or modify the recording to fit your needs. You can skip the rest of the steps in this procedure.
   - If you created your own recording, the recording is created and started. You can now start the user interface you want to record. The recorder stays in the *Recording* state until you stop it (choosing the *Stop Recording* icon). You can restart the recording at a later point in time by choosing the *Start Recording* icon.
     Perform the steps as follows.
5. Start the application that you want to record.

   > **Note**
   >
   > If the application is already open when you create the recording, you must restart it. The check if Read Access Logging is activated or not is performed when an application is started.
6. Right-click (Web Dynpro) or `Ctrl` + right-click (Dynpro) each field that you want to collect for the recording and choose *Read Access Logging* > *Record Field* from the context menu.
7. To remove a field from a recording, right-click the field and choose *Read Access Logging* > *Remove Field from Recording*.
8. When you have recorded all fields, close the application and return to the Read Access Logging Configuration app and choose *Stop Recording* in the list of recordings.

   If you do not close the Web Dynpro session, the recording is locked and you cannot change it.

Results

A recording for a Dynpro or Web Dynpro application is created. If the recording includes errors or is missing values, restart the recording.

> **→ Tip**
>
> When you restart a recording, you can select another user to be recorded. This function enables you to have another user carry out the steps you want to record with their authorizations in the system. You avoid running the recording with missing or too many authorizations.

You can also download and upload the recording.

> **Note**
>
> If you record fields from different applications (Web Dynpro) or programs (Dynpro) within one recording, the recording is separated into the programs and applications that were used. Create configurations for each program or application within the recording.
>
> For Dynpro, single transactions can contain more than one program, which may not be apparent to you when you create the recording. In this case too, your recording is separated into the programs that were used.

6.3.1.16.2.4 How to Define What to Log

To define what to log, use a read access logging configuration.

Context

There are pre-existing configurations delivered by SAP that you can use immediately. If you need a configuration that is more customized to your needs, you can either create a configuration from scratch or change an existing one.

The configuration can then be activated and used for different applications.

Related Information

How to Create a New Configuration [page 3150]
How to Activate or Deactivate the Correct Read Access Logging Configuration [page 3152]

6.3.1.16.2.4.1 How to Create a New Configuration

You can define what to log by creating your own read access logging configuration or modifying a template delivered by SAP.

Procedure

1. In the *Read Access Logging Configuration* app, choose *Configuration*.
2. Select a channel.
3. Decide whether to create a configuration or modify one of the templates delivered by SAP.

SAP delivers default configurations you can copy and modify to your needs or you can create your own.

- To create your own configuration, select a channel and choose *Create* under *Search Results*.
  For channels that use recordings, choose a recording.
- To copy a configuration from SAP, do the following.

a. Under *Search Criteria*, filter with **Owner** is **SAP Template**.

a. From the search results, select a template from SAP.

   Templates from SAP use the *(SAP)* icon under *Owner*.

b. Under *Actions*, choose *(Copy)*.

   For channels that use recordings, copy the recording that comes with the template under the same name or enter a new name.

c. Find the channel you just copied and choose *(Edit Configuration)*.

4. Depending on the channel, select the object you want to create a configuration for.

   For example, an operation for a Web Service, a function module for an RFC, or an application for a Web Dynpro. If you are editing an existing channel, the object is already selected for you.

   For more information, see Channel-Specific Information [page 3154].

5. Enter the required data.
   - For Dynpro, choose *Application / Software Component*.
   - For Dynpro and Web Dynpro, specify a *Log Context*.

6. Create log groups.

> ### Note
>
> For a logging purpose, only create one logging group.

a. Under *Log Groups*, choose *(Create Log Group)*.

b. Specify a logging purpose and a description for the log group.

c. Specify the fields that you want to log together in the log group.

   Move fields in the *Field list* to the log group.

d. In the *Fields* list, define how each field should be logged.
   - The *Logging Type* determines if only the field name is logged when accessed or if the value of that field is also logged.
   - To save space in the database, set *Exclude if Initial* to have Read Access Logging ignore a field if its value is initial.

     > ### Note
     >
     > The initial value is not the default value, but the technical initial value. For example, the empty value for string or character-based fields or the number zero for integers. The default value is logged in any case.
     >
     > If this option is set and all fields within a Web service, RFC, or Web Dynpro UI are initial, no log entry is created for any of the fields.

   - For Dynpro and Web Dynpro, specify the *Field Type*, which is whether an input field, an output (display) field, or both.

> **Note**
>
> Values for password fields cannot be included in logs. The *Logging Type* is *Without Value* and cannot be changed.

7. Create conditions.
   a. Under *Conditions* choose *(Create Condition)*.

      If you do not assign any conditions, select the *Without Condition* checkbox in the log group attributes and continue with the next step.
   b. Specify a name and a description for the condition.
   c. Create one or more expressions for the condition.

      Expressions are joined using logical `AND`.
   d. Specify the fields that you want to define rules for.

      Move fields in the *Field list* to the expression.
   e. For each field, specify the select option (the selection criteria).

      For example, *Username Inclusive Equals John Smith*.

      Select options are joined using logical `OR`.
   f. Assign conditions to a log group or set the *Without Condition* option.

      To assign a condition, select the Log Group, choose *Assign Condition* and select a condition. Activate the assignment if you want to apply the condition to the log group. The log groups and conditions do not need to be activated, however.
8. To check the configuration for consistency and function: choose *(Check Configuration) Check*.
9. Save your entries.

6.3.1.16.2.4.2 How to Activate or Deactivate the Correct Read Access Logging Configuration

Context

To activate or deactivate Read Access Logging for certain apps, follow the steps described below.

Procedure

1. Go to the app for which you want to activate or deactivate Read Access Logging.
2. Open the app and navigate to your user area (*User Actions Menu*).
3. Click the *About* icon.

- If the app has an app ID such as `F1492`, please proceed with the steps in the section *Activating and Deactivating Read Access Logging for Apps with App ID*.

Activating and Deactivating Read Access Logging for Apps with App ID:

4. Copy the app ID.
5. Go to the SAP Fiori apps reference library and select *All Apps*.
6. In the *Search* bar, paste the app ID you have just copied.
7. Click the app entry in the results list.
8. On the *App Details* screen, select *SAP S/4HANA Cloud* from the dropdown list.
9. In the section *Implementation Information*, select your current SAP S/4HANA Cloud release.
10. Open the subsection *Configuration* and copy the OData service name/s and the OData service version of the app.
11. Go back to the SAP Fiori launchpad and open the app Read Access Logging Configuration.
12. Go to *Configuration*.
13. Select the channel *SAP Gateway*.
14. Paste the OData service name that you have just copied in the field *Service ID*.

    If you have copied more than one OData service name, you can add further *Service ID* fields to your search criteria.
15. Paste the OData service version that you have just copied in the field *Service Version*.
16. Click *Search*.

    In the search results list, you can see if Read Access Logging configurations exist for the service/s and, thus, for the app, and if the Read Access Logging is active or inactive.
17. To activate or deactive Read Access Logging, select the service/s, and click either *Activate* or *Deactivate*.

    Read Access Logging is now activated or deactivated for the respective app.

Related Information

How to Define What to Log [page 3150]

6.3.1.16.2.4.3 How to Compare an Existing Configuration With an SAP Template

You can compare an existing configuration that has been created based on an SAP template with the current SAP template to see if there are any deviations.

Context

Because SAP templates are updated from time to time, it can be useful to compare existing configurations that are based on an SAP template. To check for templates with deviations and to see what has changed, follow these steps:

Procedure

1. Under *Administration* open *Configuration*.
2. Choose the channel that you want to search through.
3. Under *Search Criteria* select **Deviations from SAP Template** is **True**.
4. Choose *Search*.
5. Choose one of the search results.
6. Choose *More* and select *Compare with SAP Template*.

> **Note**
>
> If you want to rebuild a configuration based on an updated version of the SAP Template, close the pop-up window, choose the configuration, then choose *More* and select *Rebuild from SAP Template*.

Results

A pop-up window opens where you can check which values are different between the SAP Template (left) and the existing configuration (right).

6.3.1.16.2.4.4 Channel-Specific Information

The Read Access Logging framework handles the channels generically, but each channel configuration is specific.

The following table shows the objects for each channel that can be logged.

Channels Available to the Read Access Logging Framework

Channel	Data Logged
Remote Function Call	Remote-enabled function modules: Read Access Logging takes place when data leaves the system. • Asynchronous outgoing request calls on client side. Logging takes place before the call is made. • Synchronous requests on the provider side are logged with their response and fault messages to ensure that the response can be understood. Logging takes place before the call is made.
Dynpro	Configuration is based on Dynpro recordings that list the UI elements from a Dynpro application. Read access to these UI elements is then logged. For more information, see: • Dynpro Screen Elements that Can Be Logged [page 3160] • Tips for Configurations with Log Contexts [page 3163]
Web Dynpro	Configuration is based on Web Dynpro recordings that contain UI elements from a Web Dynpro application. When recording, UI elements from more than one Web Dynpro application can be gathered, but each configuration is based on UI elements from only one Web Dynpro application within a recording. **Note** Web Dynpro applications are technically identified by their Web Dynpro application configurations. For more information, see: • Web Dynpro UI Elements that Can Be Logged [page 3157] • Tips for Configurations with Log Contexts [page 3163]

Channel	Data Logged
Web Service	Web service operations: • Synchronous and asynchronous outgoing request messages on consumer side. • Synchronous requests on the provider side are logged with their response and fault messages in a single log entry to ensure that the response can be understood. **Note** Synchronous requests on the provider side are logged with their response and fault messages in a single log entry to ensure that the response can be understood. Asynchronous requests on the provider side are logged after the message passes authentication. Waiting until after authentication prevents denial of service (DOS) attacks from being logged. Log entries are created independently of the processing of messages by applications. Log entries are created and remain in the log, irrespective of whether an error is subsequently caused in the application or if the message is later canceled. Local shortcut Web service calls are not logged, since these calls do not leave the system.
ALE/IDoc Communication	IDoc outbound Supported are the following default IDoc outbound protocols: • tRFC 3.0/3.1,4.x • qRFC • File, XML file • http, SOAP Read Access Logging takes place when IDoc data leaves the system, right before the data is handed over to the single communication protocols.
SAP BW	• Query requests • Query results or outputs • Value help • Master data maintenance data • Data preview of InfoProviders
KPro	Read Access Logging takes place when Knowledge Provider (KPro) is used to access the documents. Logging is done for specific PHIO classes for which the Read Access Logging is activated.

Channel	Data Logged
Output Forms (DDIC Interface)	Read Access Logging takes place when an Output Form is rendered for Print Output, Preview or next processing by application. You can configure logging for the name of the Output Form as well as for its fields. Note that the name of the form is logged as the field `FORM_NAME`.
SAP Gateway (OData)	For OData version 2 (V2) and version 4 (V4) both output logging and input logging is supported: In the RAL configuration users can define the requests for which a service should be logged, so the data that is returned by an OData service. The data which is marked as confidential, is logged. For OData V4 compared to OData V2, some channel fields have been changed/added. For more information, see: • Read Access Logging (RAL) and OData V2 • Read Access Logging (RAL) and OData V4

Electronic Document	• Administrative eDocument data
	• Administrative eDocument file data
	• eDocument type-specific fields
	• eDocument file content data:
	  • If the file structure is filled, then all the fields from the file structure are available for logging.
	  • If no file structure is defined, then all the data can be logged without its structure (for example, a PDF file). The Electronic Document channel provides three fields for domain assignment.

6.3.1.16.2.4.4.1 Web Dynpro UI Elements that Can Be Logged

Structure

> **Note**
>
> - Both output and input elements can be logged, but they must be context-bound UI elements.
> - UI elements that are generated automatically (for example, when using F4 Help) can be logged. They do not need to have a UI element ID assigned during application development. In these cases, a temporary ID is assigned during the recording.

The table below shows all UI elements that can be logged in Web Dynpro applications. It shows the properties that can be recorded for each UI element.

UI Element	Property
`TextEdit`	Value
`TextView`	Text
`FormattedTextEdit`	Value
`FormattedTextView`	Text If *value suggest* is active, all proposed texts for this UI element that are displayed on the UI are logged.
`InputField`	Value If *value suggest* is active, all proposed texts for this UI element that are displayed on the UI are logged. Historical values are not recorded.
`ToolbarInputField`	Value
`DropdownByIndex`	Texts
`Caption`	Text
`Button`	Text
`ToolbarButton`	Text
`ToolbarDropdownByIndex`	Texts
`LinkChoice`	Text
`ToolbarLinkChoice`	Text
`ButtonChoice`	Text
`ToolbarBtnChoice`	Text
`ToggleButton`	Text
`ToolbarToggleButton`	Text
`Checkbox`	Text
`Label`	Text
`LinkToAction`	Text
`LinkToURL`	Text

UI Element	Property
`ToolbarLinkToAction`	Text
`ToolbarLinkToURL`	Text
`FileDownload`	fileName, mimeType, text, data
`Panel`	Title
`RadioButton`	Text Selected Key
`RadioButtonGroupByIndex`	Text
`RadioButtonGroupByKey`	Selected Key
`ToggleLink`	Text
`TriStateCheckbox`	Text
`SectionHeader`	Text
`DropdownByKey`	Selected Key
`ToolbarDropdownByKey`	Selected Key
`DropdownListBox`	ItemText ItemDescription Texts
`ToolbarDropdownListBox`	ItemText ItemDescription Texts

The constituents of the following multiple UI elements can be logged with Read Access Logging:

- `Table`
- `CTable`
- `RowRepeater`
- `MultiPane`
- `Accordion` and `MultiAccordionItem`

Related Information

Channel-Specific Information [page 3154]
Channel-Specific Information [page 3154]

Tips for Configurations with Log Contexts [page 3163]

6.3.1.16.2.4.4.2 Dynpro Screen Elements that Can Be Logged

A list of screen elements in Dynpro applications that can have read access logged.

The main scope of Read Access Logging is to log output fields on result screens. Optionally, also the input fields of the corresponding selection or query screens can be logged. Logging and defining a log context are supported for such selection or query screens and their result screens if both are implemented and processed in the same program. Please note that one business transaction may be processed using more than one program. In such cases, logging of the input is not possible.

The table below shows the basic screen elements that can be logged in Dynpro applications.

Basic Screen Element	Comment
Input fields	Value of input field
Output fields	Value of output field
Radio buttons	For each radio button, a value is logged (true/false). Note that each radio button is logged separately. There is no logging of entire groups of radio buttons.
Checkboxes	Value of checkbox
Dropdown lists	Technical key value with marker for selected key. Note that the texts from the user interface are not logged.
Table controls	Columns of table control
Parameters	Value of the parameters of a selection screen
Subscreens	As subscreens consist of the same elements as screens, logging takes place for the elements of subscreens in the same way as for screens

Basic Screen Element	Comment
Select-Options	Value of each select-option of a selection screen (list of sign, option, low/high values). A select-option `SELOPT` is displayed as: • a `LOW` or `HIGH` field for a single selection • a push-button (arrow icon) for a multiple selection that is represented as a table of fields (`SIGN`, `OPTION`, `LOW`, `HIGH`) To be able to log `SELOPT-LOW` or `SELOPT-HIGH` fields, they have to be configured. You can enter the data to `SELOPT-LOW` or `SELOPT-HIGH` manually: • If a program was executed without a screen round trip, the fields `SELOPT-LOW` and `SELOPT-HIGH` can be logged as input fields. • The multiple selection is attached to the `SELOPT-LOW` field: if the field `SELOPT-LOW` is configured for logging, the multiple selection is configured for logging as well. If a screen round trip was done (e.g. pressing `Enter`), or if the multiple selection was used, the `SELOPT` data is added to the multiple selection table, therefore the `SELOPT-LOW` field can be logged as an input or output field. If a variant that is used to provide the input data for a selection-option shall be logged, it has to be configured for output logging. Input data provided for the multiple selection can be logged, even if it was not displayed. Input data provided via variant for other parameter fields on a selection screen can be logged using the input logging. **Note** To be sure that all the data related to the select-option field is logged, it should be configured for both input and output logging.
Value help (F4)	Entries from the value help (F4 help) can be recorded by choosing the read access logging function from the context menu.

The table below shows the screen attributes that can be logged:

Screen Attribute	Comment
Screen title	The title of a screen

Screen Attribute	Comment
Transaction code	The value of a transaction code at runtime
OK code	Code that is set when triggering an action (for example, clicking a button)
Message	The status message displayed on a screen

The table below shows the controls that can be logged:

Control	Comment
ALV grid	Columns of a display-only ALV grid control
Tab strip	Same logging behavior as for subscreens. If switching tabs happens on front-end, logging is done together for all tabs. If, on the other hand, switching tabs happens on application level, the tabs are logged separately. In addition, an OK code can only be logged when switching tabs happens on application level.

The table below shows the additional screen elements that can be logged:

Screen Element	Comment
ABAP Lists	ABAP lists can be recorded by choosing the read access logging function from the context menu. The pseudo field `$_LIST_CODE` will be added to the created recording. Using this pseudo field, all lists of a program can be configured for logging. It is not possible to log input fields and OK codes of a list.

It is not possible to log other screen objects, such as the following:

- Input-enabled ALV grid
- ALV tree
- File download

For more information about the availability of features of Read Access Logging, see SAP Note 1969086.

Related Information

Channel-Specific Information [page 3154]
Channel-Specific Information [page 3154]
How to Manage Recordings of Application User Interfaces [page 3148]

6.3.1.16.2.4.4.3 Tips for Configurations with Log Contexts

Definition

In a Read Access Logging configuration for Dynpro and Web Dynpro channels, you might need to use a log context to make sure that your log can be interpreted correctly. The log context is the UI element that other UI elements within the logging session depend on. When read access is being logged and the field that is defined as the log context changes on the user interface, all other fields are deleted from memory before the next log entry is created. Whether or not you need a log context depends on the user interface of your application. The graphic below shows a screen sequence taken from a simple HR application and the accompanying table gives recommendations on how to configure Read Access Logging for this screen sequence with the help of the log context.

> ⓘ Note
>
> Log contexts are based on single applications or programs, like configurations. Log contexts cannot be applied across several configurations or applications/programs.
>
> For Dynpro, single transactions can contain more than one program, which may not be apparent to you when you create a recording. In this case too, log contexts cannot be spread over multiple programs used in the transaction.

Example

The graphic below shows a sequence of three screens. On Screen 1, the user enters an employee number. Screen 2 displays the social security number and religion of the employee. Screen 3 displays the salary of the employee.

Example screen sequence

The table below shows how to configure the screen sequence depicted in the graphic above. Note that the field *Employee Number* is a different field on the three screens; on the first screen, it is an input field and on the second and third field, it is an output field like the other fields that cannot be edited.

Screen Number	UI Element	Recommended Logging Type	Comment
Screen 1	Employee No	Log with value and define as log context	The employee number is an input field, we recommend that you log it. As the other fields on Screen 2 depend on the employee number, we recommend defining it as the log context. In the log, the field that is the log context is marked for easy identification.
Screen 2	Employee No	No logging necessary	The employee number has been entered on Screen 1 and is repeated here as an output field. As you have already specified it as the log context, you do not need to log this field. **Note** Even if the employee number is not repeated on each screen, the log context is still logged as part of the log entry of these screens.
	Social Security No	Log without value (for example, legal regulations may prohibit logging social security numbers)	The social security number of this employee is displayed within the same log entry as the log context. If you configure to log it without the value, only the fact that the field was accessed is logged, but not the value.
	Religion	Log with value	The religion of this employee is displayed within the same log entry. If you configure to log it with value, the religion is displayed in the log.

Screen Number	UI Element	Recommended Logging Type	Comment
Screen 3	Employee No	No logging necessary	As you have specified the employee number from Screen 1 as the log context, you do not need to log this field. **Note** Even if the employee number is not repeated on each screen, the log context is still logged as part of the log entry of these screens.
	Salary	Log with value	As it is the only field that is configured on this screen, the salary of this employee will be displayed with the log context from Screen 1 in one log entry.

Result

When a user clicks through this screen sequence when it is configured as described above, two log entries are created, one for Screen 2 and one for Screen 3. On Screen 1, no output field exists that can be logged. On Screen 3, the employee number does not have to be logged as the employee number field from screen 2 is the log context and is thus automatically logged during a screen sequence even if it is not displayed on subsequent screens.

Related Information

Channel-Specific Information [page 3154]
How to Define What to Log [page 3150]

6.3.1.16.2.5 How to Exclude Specific Users from Read Access Logging

Context

You cannot exclude specific users from Read Access Logging by using the app Read Access Logging Configuration.

Procedure

- To let users be excluded from Read Access Logging, please contact the Service Center.

6.3.1.16.2.6 How to Check Runtime Errors and See Changes to Configurations

Context

The Administrative Log of Read Access Logging provides information about different types of information such as:

- Action - changes made to Read Access Logging configuration.
- Error - errors that occur within the Read Access Logging framework.

Action Log Entries

The action log records the following Read Access Logging changes:

- Configuration
- User exclusion list
- Enabling and disabling of Read Access Logging in the current client

> ⓘ Note
>
> Only configuration changes performed manually are recorded in the log. Configurations that have been transported into the system are not included in the Administrative Log.

Error Log Entries

Error entries could be errors in the configuration, or problems that occur during logging. The log entry may recommend action to resolve the error and/or provide other information relevant for key users or SAP Active Global Support.

Procedure

1. In the app Read Access Logging Configuration, open *Administrative Log*.
2. Select a *Data Source*, either the database or the archive.
3. For *Databases*, specify a time filter. For *Archives*, select the *Archive Name*.
4. Enter a *User Name* to search the log for configuration actions or read access performed by a specific user.
5. Specify a log *Type* for archives or *Event Type* for databases (*Error*, *Warning*, *Message*, *Action* or *Logging Error*).

6.3.1.16.2.7 Transporting Read Access Logging Data

RAL objects that you've created can be transported from a Q system to a P system (development system to production system) with the *Export Customizing Transports* app

In your RAL application, you can create different customizing objects like configurations or recordings. These objects are added to the default customizing request that you can export from a Q system and import into a P system. The objects that come from the RAL application are:

- Configurations
  Refer to SAP note 2963664 for details.
- Log domains
- Purposes
- Recordings

To transport your changes from the development system to the productive system, perform the following steps:

In your development system:

1. Use the *Manage Software Components* app to make sure there is a cloned business configuration software component. See Manage Software Components.
   In case it is not available yet, you need to create a business configuration software component and clone it. See How to Create Software Components and How to Clone Software Components.
2. Make sure that there is an open customizing transport request for your system and that the customizing transport request belongs to the cloned business configuration software component.
   To create a customizing transport request, you use the *Export Customizing Transport* app. You need the SAP_BR_BPC_EXPERT role to access this app. See Export Customizing Transports [page 2978].
   If a software component of the type Business Configuration is available in the system, the created request will target this software component. If no such software component is available, the request will be created without any target.

3. Make the actual changes to the RAL objects.
4. Release the task and customizing transport request in the *Export Customizing Transport* app.

In your productive system:

1. In the Manage Software Components app, pull the changes for the business configuration. See How to Pull Software Components.
2. Check in the *Read Access Logging Configuration* app if the changes are visible.

Related Information

Working in the Export Customizing Transports App [page 2980]

6.3.1.16.3 Read Access Logging: Monitor

With this app you can use the read access log for evaluation purposes. In it, all log entries as well as all errors that occurred in Read Access Logging are displayed.

Key Features

You can use this app to:

- Choose a data source
- Search for log entries
- Evaluate log entries

Prerequisites

To access the app, you need to have the `SAP_CORE_BC_RAL` business catalog assigned to your user. This business catalog is contained in the `SAP_BR_DATA_PRIVACY_SPECIALIST` business role template.

Supported Device Types

- Desktop
- Tablet
- Smartphone

Component for Customer Incidents

If you need support or experience issues, please report an incident under component BC-SEC-RAL.

6.3.1.16.3.1 How to Monitor the Read Access Log

Context

You use the read access log for evaluation purposes. In it, all log entries as well as all errors that occurred in Read Access Logging are displayed. There are the following data sources from which you can display log entries:

- *Expanded Database* (default)
- *Expanded Archive*

Procedure

1. Open the Read Access Logging: Monitor app.
2. Select the data source from which you want to display log entries.
3. Search for the log entries you want to evaluate.

   To search for values of fields that have been accessed, you must use the *Expanded Database* as the source.

   For data source *Archive*, select the archive in which you want to search for log entries. Additionally, there are various search criteria that you can use (choose the *Add Line* icon). Typically, you would search for a time range and a log domain or a logging purpose. These search criteria are also available if you search using the database as the source.

   > ⓘ Note
   >
   > Searching the archive may be time-consuming. It is much faster if you search an archive with an index. In this case, the search criteria that you have specified for the index are available.

   When you choose *Search*, all log entries are displayed that match your search criteria. By default, only a few attributes are displayed per log entry. You can switch between the default and the extended view.

   Within the search, you can clear the values you have entered for search criteria by choosing *Clear*, and you can reset the search criteria to the default by choosing *Reset*.

   > ⓘ Note
   >
   > You can use the button *Download* at the top left of the search result table to save your search results in an Excel file.

Search Operators: “is” vs. “covers”.

In many search functions within *SRALMANAGER* and *SRALMONITOR*, two comparison operators are available:

- *“is”*: Used for an **exact match**. The entered value must exactly match the stored value.

  > ✤ Example
  >
  > Searching for Status is **`"Open"`** will return only entries with the exact status **`Open`**.

- *“covers”*: Used for a **pattern-based search** with **wildcards**. The most common wildcard is **`"*"`**, which represents any sequence of characters.

  > ✤ Example
  >
  > Searching for Name covers **`"Error"`** will return all entries whose name starts with **`Error*`**, such as **`Error123`**, **`Error - Timeout`**, etc.

These operators allow for flexible data filtering depending on the desired level of precision.

4. Select a log entry to display detailed information at the bottom.

   On the *Details* tab, all fields are displayed with their log domain and, if logged, the field values. On the *Access Environment* tab, you find more information about the underlying object that was logged.

6.3.1.16.4 Integrating Read Access Logs

Service name: `SRAL_API_RAW_LOG`

This service enables you to retrieve the raw Read Access Logs. You can use the log data to integrate them into your Security and Event Management solution (SIEM) to detect security relevant event situations.

Technical Details

Service Group (incl. Namespace if Existent)	Repository ID	Service Name (incl. Namespace if Existent)	Version
`SRAL_API_RAW_LOG`	`SRVD_A2X`	`SRAL_API_RAW_LOG`	`0002`

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Necessity	Link to Details
ReadAccessLogRAW	Read Access Log in RAW format		Read Access Logging [page 3141]

Service Response

The details included in the response vary according to the type of operation. For more information, see Operation for Read Access Log Integration.

Integration with SIEM Solution

Service name: `SRAL_API_RAW_LOG`

Communication scenario: `SAP_COM_0915`

This service enables you to retrieve the raw Read Access Logs. You can use the log data to integrate them into your Security and Event Management solution (SIEM) to detect security relevant event situations.

6.3.1.16.4.1 Operations for Read Access Log Integration - Read

The Read Access Log Integration - Read offers the following operations:

Operation	HTTP Method	HTTP Method
Retrieving Read Access Log [page 3172]	GET	Example getting the first 1000 audit log entries: GET**<host>/sap/opu/odata4/sap/sral_api_raw_log_04/srvd_a2x/sap/sral_api_raw_log/0002/ReadAccessLogRAW?top=1000**

Example getting the first 1000 audit log entries:

6.3.1.16.4.1.1 Retrieving Read Access Log

With `Read Access Log` you can retrieve the Read Access Log by using the HTTP method `GET`.

Request

You have to include the following properties in the URL of the request:

Property	Property	Comment
`LogID`	Optional	Unique identfier of each logt • String($uuid) Example: **`01234567-89ab-cdef-0123-456789abcdef`**
`LogTimestamp`	Optional	Timestamp of the read access log event • String($date-time) Example: **`2017-04-13T15:51:04`**
`slgmand`	Optional	Client within the SAP System • String <=3 characters Example: 100
`sid`	Optional	System ID • String <=3 characters Example: YI3
`counter`	Optional	Message counter • integer($int16)
`slgtc`	Optional	Transaction Code • String <=20 characters Example: VA01
`slgrepna`	Optional	Program Name • String <= 40 characters Example: SAPMSSY8

Response

The operation returns the Read Access Log.

Example

Get events within a specific timeframe:

```
GET <host>/sap/opu/odata4/sap/rsau_log_api/srvd_a2x/sap/rsau_log_api/0001/
ReadAccessLog?
$filter=(log_tstmp%20ge%202021-05-08T10%3A25%3A09Z%20and%20log_tstmp%20le%202021-
05-09T10%3A25%3A09Z)
```

Response


```
{
"eventID": "AU1",
"log_tstmp": "2021-05-08T10:26:22.740611Z",
"slgmand": "100",
"sid": "ABC",
"counter": 0,
"terminal_name": "",
"user_fullname": "Example Administrator",
"slgtc": "S000",
"slgrepna": "RSBTCRTE",
"rsau_text": "Logon successful (type=B, method=A)",
"UserDescription": "Example Administrator"
},
{
"eventID": "AU1",
"log_tstmp": "2021-05-08T10:28:22.924215Z",
"slgmand": "100",
"sid": "ABC",
"counter": 0,
"terminal_name": "",
"user_fullname": "Example Administrator",
"slgtc": "S000",
"slgrepna": "RSBTCRTE",
"rsau_text": "Logon successful (type=B, method=A)",
"UserDescription": "Example Administrator"
},
```


6.3.1.17 Security

Secure Communication [page 3174]

Display Security Audit Log [page 3190]

6.3.1.17.1 Secure Communication

SAP Business AI Platform, ABAP environment offers secure communication features for securing both machine-to-machine communication (server communication security) as well as communication between systems and end-user UIs (frontend communication security).

Communication security is maintained on multiple levels.

Establishing the Communication Channel to a Host

The connection is initiated by a client calling a host to set up a cryptographically secured communication channel. This is known as a TLS (or SSL) handshake. During the handshake, the host proves its identity by presenting a server certificate. The client validates the server certificate to gain assurance that the connected host coincides with the host the client intended to call. This prevents man-in-the-middle attacks.

In case of a web UI, the validation of server certificates is done in the browser using a list of trusted root certificates provided by the browser or the operating system.

The certificate of your system is signed by DigiCert Global Root CA.

In other cases, the client may also be a backend server talking to another backend server. Here, the server initiating the connection takes the role of the client and the other one is the host.

Authentication of the Client/User

After the communication channel is established, the application on the host side may verify the caller's identity if the application requires authentication. The most common method for this is still basic authentication (user/password). A more secure option is certificate-based authentication (mTLS). Here, the client presents an X.509 certificate to the application to prove its identity.

Additional Protection of Web Clients

For web clients, there are additional mechanisms protecting the user from attacks like cross site scripting (XSS) or redirect attacks. These mechanisms are defined by the server, but enforced in the user's client.

6.3.1.17.1.1 Frontend Communication Security

Communications between the customer browser and the system landscapes of SAP Business AI Platform, ABAP environment are secured by industry best practices and state-of-the-art open cryptographic standards. Customers use a unique, customer-specific URL. Communication is carried out via the Reverse Proxy (RP)

component. The communication channels are secured by using Transport Layer Security (TLS) protocol version 1.2.

Client-side security controls implemented by the browser and by SAP Business AI Platform, ABAP environment mitigate the risk of various attacks, including cross-site scripting, data injection, and clickjacking.

Some of these controls can be configured with allowlists such as the following:

- Trusted sources of scripts, stylesheets, and fonts can be added to the default Content Security Policy.
- Trusted hosts can be defined for the clickjacking protection, and for safe redirects.

6.3.1.17.1.1.1 Content Security Policy

Content Security Policy (https://www.w3.org/TR/CSP3) is a standard which allows you to disable certain HTML/JavaScript features to reduce the attack surface of applications running in a browser (for example, as a second line of defense against cross-site scripting attacks). The policy explicitly lists all allowed sources from where resources (scripts, styles and fonts) can be loaded.

Servers can define different content security policies (CSPs) for different paths (applications) and different resource types to provide very granular control.

CSPs can be defined in reporting or blocking mode. In reporting mode, a violation will be reported to a URI specified within the CSP. In blocking mode, violations are not only reported but execution of violating code or loading of the violating resource is additionally blocked by the browser.

Related Information

Manage Content Security Policy [page 3180]

6.3.1.17.1.1.2 Clickjacking Protection

Clickjacking (or UI redressing) is a type of attack that tricks the user into triggering actions within an application by hijacking mouse clicks or other user inputs.

In the simplest attack scenario, an invisible iFrame containing the attacked page is positioned over an attacker-controlled page. The user thinks the UI is original and is unaware of the actual actions triggered on the invisible page in the frame. To protect against this attack, you need to control whether to render your application within a frame and which pages are allowed.

The allowlist service is an ABAP-wide service and supports HTML-based frameworks to implement protections. By default, clickjacking protection is active.

As soon as the protection is enabled, a special check is performed every time an application is rendered. If the application is embedded into another one, the check determines whether the other application is secure. If the check fails, the embedded application is not framed. An error message appears.

An application is considered secure if one of the following applies:

- The application itself is not embedded in another frame
- The host of the application is part of the same domain as the embedding applications (same origin policy).
- The host of the application is part of the allowlist. For multi-domain scenarios, you have to make sure that the host name of the application in which your application is embedded, is part of the allowlist.

Related Information

Maintain Protection Allowlists [page 3178]

6.3.1.17.1.1.3 Trusted Network Zones

Redirection Attacks

The primary use of trusted network zones is to prevent redirection attacks.

In a redirection attack, the attacker tricks the victim into sending a specially crafted request to the server. The server then redirects the request to a malicious website based on the information in the request.

This is used for phishing. The victim might not be aware of the redirect because the initial request was sent to a trustworthy system and the malicious site might be built to resemble the trustworthy system.

The configuration of trusted network zones prevents such attacks by limiting which URLs the server can redirect to.

Upload Limitation

The trusted network zones configuration also restricts uploads defined by URL.

Where file upload is possible by specifying a URL rather than a file on the filesystem, the entered URL is checked against the trusted network zones list. Note that host, port, and scheme of the URL must be maintained in *Trusted Network Zones* for the upload to be permitted.

Related Information

Maintain Protection Allowlists [page 3178]

6.3.1.17.1.1.4 Session Timeout

Both web applications in the SAP Fiori Launchpad and in SAP Cloud Identity Services - Identity Authentication log out users after a certain period of inactivity. The length of this period can be changed independently for the two.

SAP Fiori Launchpad

Users are automatically signed out from the SAP Fiori Launchpad after a period of inactivity that is configurable in the *Manage Launchpad Settings* app.

> ### Note
>
> The following timeout parameters are visible in the *Manage Launchpad Settings* app. If you need to make changes, we suggest you keep the value of the `SESSION_TIMEOUT_INTERVAL_IN_MINUTES` parameter synchronized with the timeout in your Identity Authentication tenant so as to avoid inconsistencies in session management.
>
> - `SESSION_TIMEOUT_INTERVAL_IN_MINUTES`
> - `SESSION_TIMEOUT_REMINDER_IN_MINUTES`
> - `SESSION_TIMEOUT_STOP_DATA_REFRESH`

Related Information

Manage Launchpad Settings
Signing In and Signing Out
Configuring the Automatic Sign-Out
Configure Session Timeout (SAP Cloud Identity Services - Identity Authentication)

6.3.1.17.1.1.5 Apps for Configuring Frontend Communication Security

The following apps are used for configuring frontend communication security:

- Maintain Protection Allowlists [page 3178]
- Manage Content Security Policy [page 3180]

6.3.1.17.1.1.5.1 Maintain Protection Allowlists

With this app you can maintain a list of trusted hosts or URL patterns.

With this app you can define secure applications by adding trusted hosts to allowlists. The following allowlists are available as separate tabs in the app:

- *Clickjacking Protection*
  By default, clickjacking protection is active to protect your systems against malicious clickjacking. If the system is embedded into another application, the check determines whether the other application is allowed as a framing parent (that means it is allowed to embed the UI within frames in external pages). To add trusted hosts, you have to enter specific details, such as schema and port, for each trusted host to make sure that malicious hosts are identified and prevented from calling your system.
- *Trusted Network Zones*
  Here you can list URL patterns for which a redirect is allowed or blocked. Trusted network zones configuration also restricts uploads defined by URLs.
- *Trusted CSS Style Sheets*
  Here you can list URL patterns for loading CSS style sheets. This may be necessary if customer extensions are using external CSS style sheets.
- *Cross-Origin Resource Sharing*
  Here you can add trusted hosts using Cross-Origin Resource Sharing. For more information, see the *Related Information* section.

Key Features

You can use this app to:

- Add trusted hosts to the required allowlist
- Edit trusted hosts or URL patterns
- Delete trusted hosts or URL patterns

> ⓘ Note
>
> You can also define wildcards if required, such as `*test.com`.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-COM`.

Related Information

How to Add Trusted Hosts for Cross-Origin Resource Sharing (CORS) [page 3179]

6.3.1.17.1.1.5.1.1 How to Add Trusted Hosts for Cross-Origin Resource Sharing (CORS)

Context

CORS is a mechanism that allows web browsers or other web clients access to your sites. Access of this kind is usually forbidden by the Same-Origin-Policy (SOP). To add trusted hosts using CORS, proceed as follows:

Procedure

1. Open the *Maintain Protection Allowlists* app on the SAP Fiori Launchpad.
2. Open the *Cross-Origin Resource Sharing* tab.
3. Enter the *Trusted Host Name* (SAP Analytics Cloud, for example, could have the following pattern: *mytenant.us1.sapbusinessobjects.cloud*).
4. Enter the *HTTP Service Path* (for example **`/sap/opu/odata/sap/APS_IAM_API_BROLE_CDOC`**).
5. Select the allowed HTTP methods .
6. Enter the allowed headers (for example **`content-type`**)

   Only the following response headers are generally accessible in a CORS query:
   - *Cache Control*
   - *Content Type*
   - *Last Modified*
   - *Content Language*
   - *Expires*
   - *Pragma*

   If the server wants to give the client access to further headers, it has to use the *Exposed Headers* option.
7. In addition, you can define exposed headers under *Optional Response Headers* if required. Please select the required checkboxes.
8. Click *Add*.

Related Information

Understanding the Same-Origin Policy and CORS

6.3.1.17.1.1.5.2 Manage Content Security Policy

Content Security Policy is a standard that allows to disable certain HTML/Javascript features to reduce the attack surface of applications running in a browser (for example as second line of defense against cross-site scripting attacks).

With this app you can view an allowlist of trusted sources. You can add your own trusted content for example if you have developed your own SAPUI5 app that loads external resources such as fonts, scripts or styles. Moreover, you can display any violations of the policy in a log.

Key Features

You can use this app to:

- Display an allowlist of trusted sources
- Add new trusted sources to the allowlist
  For fonts, use UI_RESOURCES_FONTS; for scripts, use UI_RESOURCES_SCRIPTS, for styles, use UI_RESOURCES_STYLES.
- Display logs listing violations
  Please note the following information about browser-related effects that in some cases might result in additional violations while in other cases violations are not listed:
  - The violation logs might contain records indicating that an EVAL violation has occurred. These violations can be ignored. They are caused by unexpected behavior in Google Chrome.
  - Firefox does not send the session cookie with the reporting requests. These requests will therefore be rejected by the backend, and are thus not included in the log.
  - Browser extensions sometimes insert non-policy conform code into an HTML page. This results in violation log entries that are not caused by the applications themselves.

> **Note**
>
> You can switch the reporting endpoint delivered by SAP on and off. To do this, activate or deactivate the *Reporting Endpoint Status* switch on the *Settings* tab.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-MID-ICF`.

6.3.1.17.1.2 Server Communication Security

In integration scenarios, the SAP Business AI Platform, ABAP environment backend exchanges data with other systems. These systems may be SAP or third-party systems in the cloud or on-premise.

To secure these system-system communication channels, authentication based on certificates can be used.

We distinguish between outbound and inbound integrations.

Outbound Integrations

In outbound integrations, the SAP Business AI Platform, ABAP environment calls an external system. To establish the connection, the external system must present a trusted (server) certificate.

The SAP Business AI Platform, ABAP environment may in turn authenticate towards the external system by presenting a client certificate that the external system trusts. Depending on the capabilities and configuration of the external system, other authentication methods like OAuth or user/password are possible.

Inbound Integrations

In inbound integrations, an external system calls the SAP Business AI Platform, ABAP environment backend system. The authentication method is specified as part of the communication arrangement. In case of certificate-based authentication, the external system must present a specific certificate that is maintained in the communication arrangement and signed by an approved CA.

6.3.1.17.1.2.1 Secure Communication for Outbound Integration

Secure communication is required in all integration scenarios that connect SAP Business AI Platform, ABAP environment to other systems.

These outbound integration scenarios may include:

- other SAP cloud systems
- your on-premise systems
- third party systems (cloud, non-cloud)

When a connection between SAP Business AI Platform, ABAP environment and the external system is established, this happens in two steps. In the first step, the SAP Business AI Platform, ABAP environment validates the identity of the external system to ensure there's no man in the middle impersonating the external system. In the second step, SAP Business AI Platform, ABAP environment authenticates against the external system (if required by the external system).

Step 1: SAP Business AI Platform, ABAP environment Verifies the Identity of the External System

When establishing the secure communication, the external system must prove its identity using a server certificate that is signed by a trusted certificate authority (CA).

For secure communication to SAP-owned systems and services, SAP Business AI Platform, ABAP environment contains a preconfigured list of trusted CAs (marked as *Managed By SAP*, not changeable by customers).

For integration to non-SAP systems, you can maintain the list of trusted CAs (*Managed By Customer*) in the *Maintain Certificate Trust List* app.

Expiration of Certificates of Trusted CAs

When a certificate from the list of trusted certificates reaches its expiration date, it needs to be deleted from the list in your SAP Business AI Platform, ABAP environment. You can do this using the *Maintain Certificate Trust List* app.

In case of certificates that are managed by SAP, you need to update the list of certificates in the app. You can choose between manual and automatic updating. When you check for updates manually, you will be prompted to select which updates you'd like to apply, if any are available. If you choose to set up automatic updating instead, you can select if you'd like to apply updates regarding both newly introduced and deleted certificates, or only new certificates. These changes will be applied by the system each time the global certificate trust list is updated by SAP.

Certificates that have been added manually need to be deleted manually as well.

> → Remember
>
> If your external system that SAP Business AI Platform, ABAP environment communicates with uses a certificate that is signed with an expiring root CA certificate, remember to change that certificate. Otherwise, the communication will fail.

Step 2: SAP Business AI Platform, ABAP environment Authenticates as Client

Different methods can be used to authenticate. The availability of these options depends on the capabilities of the external system.

The credentials for outbound communication are configured in the *Maintain Communication Systems* app.

Authentication Using a Client Certificate (mTLS)

For authentication against an external system, SAP Business AI Platform, ABAP environment may use the certificate-based authentication method. In this method, SAP Business AI Platform, ABAP environment presents a client certificate to the external system.

The client certificate is linked to a private key that serves as a secret for identifying SAP Business AI Platform, ABAP environment. The client certificates must be trusted by the target system or be signed by a CA the target system trusts.

Expiration of Client Certificates and the Default Client Certificate

When client certificates expire, the external system will not accept them for authentication anymore. Consequently, your SAP Business AI Platform, ABAP environment will not be able to connect to the external system anymore. You must update the trust between the system in time before this happens by uploading a new client certificate (private key) in the *Maintain Client Certificates* app. The corresponding public key must be made known to the external system.

Authentication Using OAuth

OAuth-based authentication is a two-step process (the so-called "client credential flow"). In the first step, the SAP Business AI Platform, ABAP environment contacts a token provider to obtain an access token. During this step, the SAP Business AI Platform, ABAP environment authenticates against the token provider using basic authentication using client ID and client secret or mTLS using a client certificate (only available in OAuth 2.0). The authentication endpoints of the token provider are maintained in the *Communication Systems* app.

As the second step, the call to the external system is made and the access token is passed to the external system.

Authentication Using User Name and Password

SAP Business AI Platform, ABAP environment also supports basic authentication (user name and password). As there is a high risk of the credentials leaking, we recommend using certificate-based authentication instead.

No Authentication

Some external systems do not require authentication. They just offer public endpoints available to everybody.

Related Information

Maintain Certificate Trust List [page 3186]
Maintain Client Certificates
Communication Systems [page 3001]

6.3.1.17.1.2.2 Secure Communication for Inbound Integration

Integration scenarios from a customer system to SAP Business AI Platform, ABAP environment (inbound integration) require secure communication.

You specify and configure the authentication method in the *Communication Arrangements* app when you select a user for inbound communication. The underlying communication scenario defines the authentication methods that are available for a communication arrangement.

Communication scenarios can offer the following types of authentication method:

- Token-based
- Certificate-based
- Basic (User ID and Password)

Token-based authentication methods enable principal propagation, which means that the resulting sessions run in the context of a business user. In contrast, sessions using certificate-based or basic authentication run in the context of a technical communication user.

Token-Based Authentication

Token-based authentication methods are configured in the *Communication System* app, where you establish trust with the token provider and define which part of the token contains the user name for principal propagation.

> **Note**
>
> In system-to-system communication of the kind covered here, SAML and OIDC authentication flows use the bearer flow mechanism. These authentication flows differ from flows implemented in browser-based SAML 2.0 and OIDC authentication because there's no UI or user interaction in system-to-system communication.

OAuth-Based Authentication

When the external system authenticates to SAP Business AI Platform, ABAP environment using OAuth, it presents a short-lived token to SAP Business AI Platform, ABAP environmentt. This token is cryptographically signed by a trusted token provider.

To establish the trust relationship between SAP Business AI Platform, ABAP environment and the token provider, you must upload the token provider's certificate (public key) to your SAP Business AI Platform, ABAP environment in the *Communication Systems* app.

SAML-Based Authentication

In SAML-based authentication, the external system authenticates by passing a SAML assertion (a cryptographically signed piece of text) to SAP Business AI Platform, ABAP environment.

For authentication to succeed, the issuer of the SAML assertion must be trusted by the SAP Business AI Platform, ABAP environment. To ensure trust, you must upload the certificate (public key) of the SAML issuer in the *Communication Systems* app. This enables SAP Business AI Platform, ABAP environment to verify the authenticity of the SAML assertion.

OpenID Connect (OIDC) Authentication

OpenID Connect standardizes the way in which user information is transferred via JSON Web Token (JWT) during login. To ensure trust with the token provider, you must upload a document containing the provider's metadata. This document can include the URL for fetching the OIDC provider's private keys (Web Keys),

allowing trust to be established automatically. Alternatively, you can configure the URL for downloading the OIDC provider's public key manually or enter the keys directly.

Certificate-Based Authentication

For certificate-based authentication, the client certificate (public key) of the external system must be uploaded to the communication user used in the communication arrangement.

Such client certificates must be signed by approved certification authorities (CAs).

If the certificate is signed by an untrusted CA, establishing a connection fails during the TLS handshake.

If the external system presents a certificate that is signed by a trusted CA, but doesn't match the certificate configured in the communication arrangement, authentication fails at the application level.

A list of all root CAs approved by SAP Global Security is available in SAP Note 2801396 (SAP Global Trust List).

Certificate Expiration

Every certificate is valid for a certain period. Once a certificate expires, it can't be used for authentication anymore.

When a certificate of a root CA expires and is replaced by a new certificate, you also need to consider any certificates that were signed by the expiring CA certificate. These certificates have to be replaced by ones that signed with the new CA certificate. Remember to update your communication users accordingly.

Basic Authentication (User ID and Password)

To use password-based authentication, set a secure password for the communication user in the *Maintain Communication Users* app.

> ⚠ Caution
>
> SAP highly recommends using strong authentication methods such as token- or certificate-based authentication for greater security.

Related Information

Maintain Communication Users [page 2993]
Communication Systems [page 3001]
SAP Note 2801396

6.3.1.17.1.2.3 Apps for Configuring Server Communication Security

The following apps are used for configuring server communication security:

- Maintain Certificate Trust List [page 3186]
- Maintain Client Certificates [page 3187]

6.3.1.17.1.2.3.1 Maintain Certificate Trust List

With this app you can maintain a list of trusted certificates. If certificates of communication partners are classified as trusted, outbound communication to these partners can be enabled.

With this app you can monitor all available trusted certificates.

Key Features

You can use this app to:

- Display a list of all already existing trusted certificates
- Upload a new certificate
- Display detailed information
- Check for updates of certificate trusts and adopt the changes if required.
- Automatically apply all changes to the SAP trust list in your trust list after an upgrade. You can either just add new certificates or add new certificates and simultaneously remove deleted certificates.
- Delete trusted certificates from the list. This feature is enabled only for the certificate type *Managed By Customer*. Certificates of the type *Managed By SAP* cannot be deleted and therefore the *Delete* button is disabled.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-COM`.

6.3.1.17.1.2.3.2 Maintain Client Certificates

With this app you can upload client certificates for your area to enable secure outbound certificate-based authentication. You can maintain one central list of certificates you can use when defining communication systems for outbound communication.

Key Features

You can use this app to:

- Upload certificates in PKCS #12 format
- Download certificates
- Display general information and usage in communication arrangements and systems
- Replace client certificates
- Create signing requests

Note:

Once you have uploaded your certificate in the *Maintain Client Certificates* app, you have to edit it to enable it for usage in SAP BTP. You do this by selecting *Client Default for ABAP on BTP* in the *Certificate Category* dropdown.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-COM`.

Related Information

Communication Arrangements [page 2997]

6.3.1.17.1.2.3.2.1 Using Client Certificate Authentication with SAP BTP Destination Service

To enable client certificate-based authentication in SAP Business AI Platform, ABAP environment, the following prerequisites have to be met:

SAP BTP cockpit:

For Web Socket RFC

1. Select your Web Socket destination within your used destination service.
2. Under *Additional Properties*, apply the *jco.client.tls_client_certificate_logon property* and set it to value **1**.
3. Choose *Save*.

For HTTP

1. Select your HTTP destination within your used destination service.
2. In the *Authentication* dropdown list of the *Destination Configuration* area, select *Client Certificate Authentication*.
3. Choose *Save*.

6.3.1.17.1.2.3.2.2 How to Handle Default Client Certificate Renewal

If you use the default client certificate for certificate-based authentication in outbound integration scenarios, you must update the trust between the systems when a new default client certificate is issued.

Context

In outbound integration scenarios that use certificate-based authentication, the SAP Business AI Platform, ABAP environment system needs to authenticate itself against the external system with a client certificate. You can use the default client certificate provided by SAP Business AI Platform, ABAP environment for this purpose. This certificate will be rotated twice a year by SAP, regardless of its remaining validity.

The new certificate needs to be configured in your external system that communicates with SAP Business AI Platform, ABAP environment.

Rotation Process

The rotations occur every year on March 1 and September 1, and the process is divided into two stages.

One Month Before Rotation

On the first day of the month preceding the rotation (February 1 and August 1), a new default client certificate will be issued.

The new default client certificate will be available for export in the **Maintain Client Certificates** app and will be labeled **Client Default**. The expiring certificate will be renamed to **Client Default Expiring**.

Once the new default client certificate is available, follow these steps:

1. **Upload the New Certificate to the External System:**If the external system supports multiple certificates simultaneously, keep the old one until the replacement process is complete:
   - Open the **Maintain Client Certificates** app.
   - Export the new **Client Default** certificate in the format required by the external system.
   - Upload the certificate to the external system.
2. **Rotate the Used Certificate in the Communication System:**
   - In the **Maintain Client Certificates** app you can see by which Communication Systems the **Client Default Expiring** certificate is used.
   - Open the **Communication Systems** app.
   - For each communication system using the **Client Default Expiring** certificate:
     - Navigate to the **Users for Outbound Communication** section.
     - Select the user associated with the **Client Default Expiring** certificate.
     - In the dialog, change the **Client Certificate** value to **Client Default**.

> ### Note
>
> Switch to the new certificate as soon as possible. This ensures that if any issues arise, there will be time to troubleshoot within the 30-day overlap of the certificate validity periods.

> ### Note
>
> When exporting the **Client Default** certificate from the **Maintain Client Certificates** app, only the low-level client certificate is included; the certificate chain (**Intermediate CA** and **Root CA** certificates) is missing. To maintain trust relationships, you must separately obtain the Root CA certificate from the SAP Cloud Root CA Certificate and upload it to your external system.
>
> Ensure the **Root CA** certificate is trusted in your external system to prevent trust issues during the client certificate renewal process.

On the Rotation Date

Once the old certificate is rotated on either March 1 or September 1, it will be removed from the list in the **Maintain Client Certificates** app and all remaining Communication Systems using the old certificate will be migrated to the new Client Default certificate.

What Happens If You Don't Act

If you do not update your communication users and the new certificate in your external system, the outbound integration scenarios which use the default client certificate for authentication might break. You will get either the `401 Unauthorized` or the `403 Forbidden` HTTP status code message when trying to connect, depending on the implementation of the external system.

6.3.1.17.2 Display Security Audit Log

With this app, you can display information about security-relevant events that occur in your SAP system. This can be necessary in case of an audit.

Key Features

This app provides the following key features:

- Recording of security-relevant events in your SAP system.
- Access to previously specified log files in the form of an audit analysis report.

Scope

The Security Audit Log records the following information:

- Security-related changes to the ABAP Platform (for example, change of system changeability)
- Information that provides a higher level of transparency (for example, successful and unsuccessful logon attempts)
- Information that enables the reconstruction of a series of events (for example, successful or unsuccessful transaction starts)
- Security Audit Log events are retained according to the configured retention period (45 days by default). For details about configuring the retention period, see Manage Security Audit Log.

The app *Display Static System Audit* (available for the external auditor role) displays a detailed list of the Security Audit Log events, which are visible in the Security Audit Log app. SAP Note 2903873 provides a list of currently recorded Security Audit Log events.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, report an incident under component BC-SEC-SAL.

6.3.1.17.2.1 Integrating Security Audit Logs

Service name: RSAU_LOG_API

This service enables you to retrieve the security audit log data. You can use the audit log data to integrate them into your Security and Event Management solution (SIEM) to detect security relevant event situations.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

A service group contains services belonging to the same business object model and so it shares similar environment conditions. This means the configuration and administration of a service group apply to all services in a service group, that is, routing, authorization, and so on, so you only have to do them once.

In the OData version 4 (V4) runtime implementation of the SAP Gateway Foundation, framework services originate from repositories.

Service Group (incl. Namespace if Existent)	Repository ID	Service Name (incl. Namespace if Existent)	Version
RSAU_LOG_API	SRVD_A2X	RSAU_LOG_API_SERVICE	0001

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Necessity	Link to Details
Security Audit Log	Security Audit Log		Retrieving Security Audit Log [page 3192]

Service Response

The details included in the response vary according to the type of operation. For more information, see Operation for Security Audit Log Integration.

Integration with SIEM Solution

Service name: `RSAU_LOG_API`

Communication scenario: `SAP_COM_0750`

This service enables you to retrieve the security audit log data. You can use the audit log data to integrate them into your Security and Event Management solution (SIEM) to detect security relevant event situations.

Additional Information

> **Note**
>
> For more details about Communication Management, see Communication Management [page 2992].

6.3.1.17.2.1.1 Operations for Security Audit Log Integration - Read

The Security Audit Log Integration - Read offers the following operations:

Operation	HTTP Method	Sample URL
Retrieving Security Audit Log [page 3192]	GET	Example getting the first 1000 audit log entries: GET <host>/sap/opu/odata4/sap/rsau_log_api/srvd_a2x/sap/rsau_log_api/0001/SecurityAuditLog?$top=1000

6.3.1.17.2.1.1.1 Retrieving Security Audit Log

With `SecurityAuditLog` you can retrieve the Security Audit Log by using the HTTP method `GET`.

Request

You have to include the following properties in the URL of the request:

Property	Necessity	Comment
`eventID`	Optional	Security Audit Log Event • String <= 3 characters Example: AU1, AU2
`log_tstmp`	Optional	Timestamp of the security audit log event • String($date-time) Example: 2017-04-13T15:51:04Z
`slgmand`	Optional	Client within the SAP System • String <=3 characters Example: 100
`sid`	Optional	System ID • String <=3 characters • Example: YI3
`counter`	Optional	Message counter • integer($int16)
`slgtc`	Optional	Transaction Code • String <= 20 characters Example: VA01
`slgrepna`	Optional	Program Name • String <= 40 characters Example: SAPMSSY8

Response

The operation returns the Security Audit Log.

Examples

Get events within a specific timeframe:

```
GET <host>/sap/opu/odata4/sap/rsau_log_api/srvd_a2x/sap/rsau_log_api/0001/
SecurityAuditLog?
```

```
$filter=(log_tstmp%20ge%202021-05-08T10%3A25%3A09Z%20and%20log_tstmp%20le%202021-
05-09T10%3A25%3A09Z)
```

Response


```
{
"eventID": "AU1",
"log_tstmp": "2021-05-08T10:26:22.740611Z",
"slgmand": "100",
"sid": "ABC",
"counter": 0,
"terminal_name": "",
"user_fullname": "Example Administrator",
"slgtc": "S000",
"slgrepna": "RSBTCRTE",
"rsau_text": "Logon successful (type=B, method=A)",
"UserDescription": "Example Administrator"
},
{
"eventID": "AU1",
"log_tstmp": "2021-05-08T10:28:22.924215Z",
"slgmand": "100",
"sid": "ABC",
"counter": 0,
"terminal_name": "",
"user_fullname": "Example Administrator",
"slgtc": "S000",
"slgrepna": "RSBTCRTE",
"rsau_text": "Logon successful (type=B, method=A)",
"UserDescription": "Example Administrator"
},
```


6.3.1.18 System Management

Maintain User Sessions [page 3194]

6.3.1.18.1 Maintain User Sessions

With this app administrators can monitor and control users' logon sessions in the system. You can filter by user ID or name, see when each session was last used, the session index, how many locks it holds, and the client/device IP address. From the list, admins can select sessions and delete/end them to free locks, resolve stuck processes, reclaim resources, and support security or troubleshooting activities.

> ⚠ Caution
>
> If sessions are related to drafts they cannot be deleted.

Prerequisites

To access the app, you need to have the `SAP_CORE_BC_CST_DP_MAS_PC` business catalog assigned to your user.

Key Features

You can use this app to:

- View details of users' logon sessions in the system, such as user IDs, user names, or number of locks, and client/device IP addresses.
- Delete sessions if required.

  > ⚠ Caution
  >
  > The business user does not get a warning message when a session is deleted. Therefore, we recommend that you do this with caution.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SRV-APS-IAM`.

6.3.1.19 Manage Banks - Master Data

App ID: F6437

With this app, you can display and create master data for the banks your company, your business partners, customers, and your suppliers use to conduct business.

Key Features

You can use this app to:

- Display banks and view their basic information, such as bank name, SWIFT/BIC code, bank number and address information.
- Create and edit both standard banks and internal banks.
  For more information about the bank categories the app supports, see Banks and Bank Categories.
- Create and edit international address versions.
  For more information, see International Address Versions.

> **Note**
>
> Additional features relating to *Cash Management* can be accessed in app *Manage Banks – Cash Management*. For more information, see *Manage Banks - Cash Mangement* under *Feature Comparison for Bank Management Apps*.

> **Note**
>
> This app contains in-app help for key fields and concepts. To display the help while working in the app, press F1 or choose the question mark displayed in the app header.

In addition, following technical features and options are supported:

- Banks can be replicated to other systems using SAP Master Data Integration. For more information, see Master Data Replication Using SAP Master Data Integration (Related Links).
- Banks can be imported from external files with other app *Import Bank Directories*. For more information, see Uploading Bank Data (Related Links).

Access Information

For information on what business role and business catalog need to be assigned to your user to access the app, see the overview table in the chapter Applications for General Functions for Key Users.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `CA-BK-BNK` (*Bank Master Data*).

Related Information

Master Data Replication Using SAP Master Data Integration
Uploading Bank Data with *Import Bank Directories*
Manage Banks - Cash Management

6.3.1.20 Technical Monitoring Apps

Get an overview of the SAP Fiori apps for technical monitoring.

Application System Overview [page 3197]
HANA Table Analysis [page 3198]
Sampled Work Process Data [page 3199]
HANA Thread Samples [page 3199]
System Workload [page 3200]
System Outbound Communication [page 3201]
Capture Request Statistics [page 3202]
Perform System Sizing [page 3202]
Health Monitoring [page 3203]
Schedule Metric Provider Collection [page 3204]
Partition HANA Tables [page 3204]
ABAP Runtime Errors [page 3207]
HANA Memory Analysis [page 3207]
SQL Statement Analysis [page 3208]

6.3.1.20.1 Application System Overview

App ID: F4031

With this app you can get access to a set of essential data to monitor the application system.

Key Features

This app provides the following key features:

- Navigation to detail views, such as memory utilization, work process utilization, dialog step times, and top transactions by CPU utilization

- Thumbnail previews of essential application system metrics

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

More Information

The application system overview is part of the technical monitoring cockpit. For more information about the technical monitoring cockpit, see the documentation at https://help.sap.com/viewer/tmc_cloud.

6.3.1.20.2 HANA Table Analysis

App ID: F8726

With this app you can get in-depth information about tables, for example, about their characteristics and resource consumption.

Key Features

This app provides the following key features:

- View of the total memory consumption of the biggest tables
- Identification where you should optimize the tables and indexes in your system, for example, to save disk space
- For a selected table, detailed information about partitions, indexes, memory, and load

Supported Device Types

- Desktop

- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

6.3.1.20.3 Sampled Work Process Data

App ID: F4031

With this app, you can check, for example, how much ABAP CPU time and main memory your work processes have consumed and find out the dialog and background work process utilization. This helps you to identify the top consumer workloads that might slow down your system and cause bottlenecks.

Key Features

This app provides the following key features:

- Identification of resource utilization and main workload
- Analysis of concurrent workload
- Detail views of single work process samples
- Analysis of the root cause of an identified expensive workload or a bottleneck by further drilling down to single code lines via sampled stack traces

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

6.3.1.20.4 HANA Thread Samples

App ID: F4031

With this app, you can estimate how much HANA CPU time your custom applications consume. This helps you to identify your top workload that consumes most of the HANA CPU time, from where you can drill down to a detailed view of HANA thread samples and the related single SQL statements.

Key Features

This app provides the following key features:

- Identification of main workload for HANA CPU utilization
- Detail views of single HANA thread samples
- Analysis of the root cause of expensive workload or resource bottlenecks by drilling down to the ABAP coding that triggered the database activity
- Navigation to the SQL statement that triggered the thread activity

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

6.3.1.20.5 System Workload

App ID: F4031

With this app you can get an overview of your system workload and see how it behaves over time.

Key Features

This app provides the following key features:

- Response time distribution in terms of, for example, CPU, database, and processing times
- Time series of the response time distribution of the system
- Drilldown to request processing details and ABAP statistics records

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

6.3.1.20.6 System Outbound Communication

App ID: F4031

With this app you can get an overview of the system outbound communication over time, which helps you identify obvious expensive outbound communication.

Key Features

This app provides the following key features:

- It shows the top 10 communication scenarios by calling time.
- The *System Outbound Communication* screen and its related drill-down screens provide sample records of expensive system outbound communication.
- Outbound communication of type RFC, HTTP, and Web service is supported.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

6.3.1.20.7 Capture Request Statistics

App ID: F5699

With this app you can capture request statistics to find out which activities are running in your ABAP system. With the activation of a capture profile, you can capture system activities. Requests are recorded with technical information, such as response time, program name, or CPU time. Using the request statistics, you can investigate the performance of a system or of a tenant in more detail.

Key Features

This app provides the following key features:

- Creation of capture profiles that define which request statistics are captured when, and whether they are stored locally or are exported
- Display of profiles for capturing request statistics defined by SAP
- Activation and deactivation of profiles for capturing request statistics
- Navigation to the analysis of request statistics
- Activation and deactivation of SQL traces

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

6.3.1.20.8 Perform System Sizing

App ID: F5700

With this app, you can calculate how many ABAP compute units are consumed based on captured request statistics of selected business scenarios. With the result of the system sizing, you can estimate how many ABAP compute units the production system will need.

Key Features

This app provides the following key features:

- Navigation from the definition of profiles for capturing request statistics to system sizing based on these statistics
- Estimation of required total ABAP memory, work process time, and ABAP CPU time in ABAP compute units for selected ABAP statistics records
- Fine-tuning of system sizing by changing the selected ABAP statistics records, the process executions per minute, or the available ABAP compute units

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

6.3.1.20.9 Health Monitoring

App ID: F6967

With this app you can check the health of your ABAP system. The KPIs and time series of various metrics are displayed on individual cards. These metrics indicate the health state of the system.

Key Features

This app provides the following key features:

- Overview of the metrics for the monitored system
  Health metrics are represented by cards with a KPI that summarizes the current status.
- Display of the history of the metrics to, for example, compare trends, identify performance bottlenecks, or investigate resource shortages
- Navigation to related apps from some cards for further analysis or issue fixing

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

6.3.1.20.10 Schedule Metric Provider Collection

App ID: F7021

With this app you can schedule and monitor application jobs that collect metric providers that have been created in ABAP development tools in Eclipse (ADT) using ABAP for Cloud Development. After you've scheduled your application job and set up the communication to SAP Cloud ALM, your metrics will also be available in SAP Cloud ALM Health Monitoring.

Key Features

This app provides a view on the *Application Jobs* app. Therefore, its key features are identical with the key features of the *Applications Jobs* app.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

6.3.1.20.11 Partition HANA Tables

App ID: F5466

With this app you can break down large columnar tables into partitions. By default, database tables on SAP HANA are not partitioned. The SAP HANA database allows for a maximum number of approximately two billion records per table. If this threshold is exceeded, your application stops working. To avoid this, you need to partition your tables in time.

Key Features

This app provides the following key features:

- Search for large tables that can be partitioned according to certain filter criteria
- Partition large tables
- Undo the partitioning of a table by merging the partitions of a table back into one table

> ⓘ Note
>
> With this app you can only partition your own tables or tables that have explicitly been classified for partitioning by SAP.

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

More Information

For more information about the partitioning limits of the SAP HANA database, see SAP Note 2154870 and Partitioning Limits.

6.3.1.20.11.1 Partitioning a Large HANA Table

Break down large columnar tables into partitions before the SAP HANA database threshold of two billion records per table is reached.

Context

> **Note**
>
> Using this app may cause an application downtime because the table is locked and inserts are not possible.

Procedure

1. On the SAP Fiori launchpad of your ABAP environment, search for the *Partition HANA Tables* app.
2. To search for tables that contain a large number of records, enter your search criteria.

   By default, the search criteria include the table name, the number of records, the total memory in MB and the number of partitions. If you want to include other search criteria, choose the *Adapt Filters* button.
3. Choose the *Go* button.

   A list of tables that match the search criteria and that are eligible for partitioning is displayed, such as your own tables or tables released for partitioning by SAP. You can identify large tables in this list by checking the number of records and the total memory of the individual tables.

   > **Note**
   >
   > You can add or remove columns in the list by choosing the *Settings* button.
4. Choose the table that you want to partition and choose the *Partition Table* button.
5. In the *Change Partitioning Schema* dialog box, select the partitioning key and enter the number of partitions that you want to create. Then choose *Partition Table*.
6. In the *Confirm Table Partitioning* dialog box, choose *OK*.

   A background job is scheduled on the backend system that carries out the table partitioning.
7. To check whether the job has finished and your table was partitioned successfully, refresh the list of tables by choosing the *Go* button.

   > **Note**
   >
   > You can always undo the partitioning of a table by choosing a partitioned table in the list and then choosing the *Merge Partitions* button. All partitions of this table will then be merged back into one partition.

6.3.1.20.12 ABAP Runtime Errors

App ID: F7770

With this app, you can display and analyze the ABAP runtime errors that have occurred in your current tenant.

Key Features

This app provides the following key features:

- List of all ABAP runtime errors that have occurred in your current tenant, which you can personalize
- Display of summary information, the source code, and the call stack for each ABAP runtime error that has occurred
- Option to navigate to ABAP development tools (ADT) to fix the error or to display the error in the Web browser (HTML)

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

6.3.1.20.13 HANA Memory Analysis

App ID: F8727

With this app, you can get an overview of the memory consumption of the HANA memory components over time. This helps you to understand how the SAP HANA database requests, uses and manages memory, which is crucial to understand how the system workload is processed.

Key Features

This app provides the following key features:

- Identification of the components in your HANA memory that approach the critical memory limit that leads to botthenecks in your HANA memory

- Overview of the total memory consumption of column store and row store tables
- Overview of the content of the SQL plan cache

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

6.3.1.20.14 SQL Statement Analysis

App ID: F8784

With this app, you can analyze the performance and resource consumption of SQL statements within a selectable time range.

Key Features

This app provides the following key features:

- List of the most expensive SQL statements in the SQL plan cache
- Display of the application/execution sources of the SQL statement with navigation links to the source code location
- Historical overview of performance metrics such as execution time and CPU time
- Download of PlanViz (PLV) files for analysis with the SAP HANA SQL Analyzer for Visual Studio Code

Supported Device Types

- Desktop
- Tablet

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-OP-MON`.

6.3.1.21 Translation

Maintain Translations [page 3209]

6.3.1.21.1 Maintain Translations

The *Maintain Translations* app provides an easy way to translate developer extensions into other languages.

Context

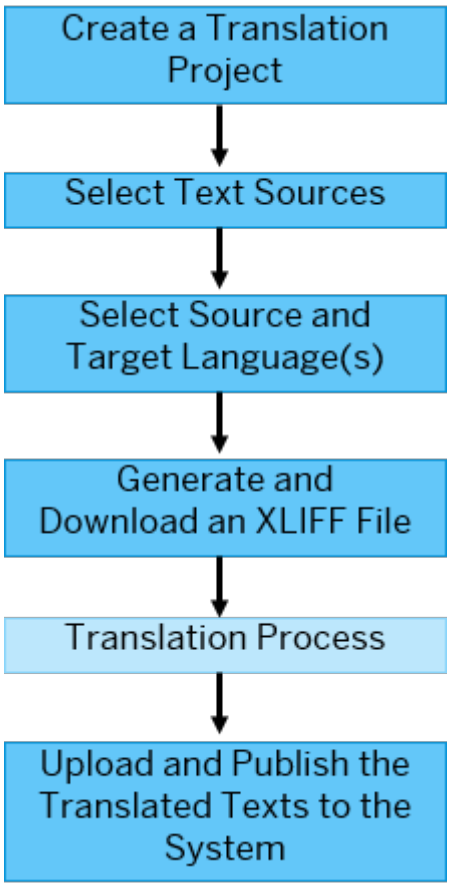
You've built a developer extension and would now like to translate it into other languages. The *Maintain Translations* app allows you to select your text sources and generate an XLIFF (1.2) file containing the translatable texts as well as useful notes on them. XLIFF (XML Localization Interchange File Format) is a standard used in localization. The XLIFF file can then be forwarded to a dedicated translator or translation agency. Once the XLIFF file has been translated, you can use the *Maintain Translations* app to upload it and publish the translated texts into your system.

Prerequisites

The following business catalogs are required:

Business Catalogs

Primary	Secondary
SAP_A4C_BC_DEV_TRA_PC (for the *Maintain Translations* app)	SAP_A4C_BC_TRN_MNG_PC (required to create transport requests)
	SAP_A4C_BC_TRN_REL_PC (required to release transport requests)



- Create a Translation Project [page 3211]
- Select Your Text Sources [page 3212]
- Select Your Source and Target Language [page 3214]
- Generate and Download the XLIFF File [page 3214]
- Upload and Publish the Translated XLIFF File to the System [page 3215]

Key Features

You can use this app to:

- generate an XLIFF file of the texts that need to be translated.
- publish the translated texts to the system.

> ⓘ Note
>
> If you want to have your texts translated by SAP, you can use the SAP Translation Hub. For more information, see SAP Translation Hub.

Supported Device Types

- Desktop

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-DOC-TTL-MTR`.

6.3.1.21.1.1 Create a Translation Project

Context

You want to translate your developer extension. Before you can select your text sources and target languages, you need to create a translation project.

> **Note**
>
> Please assign the business catalog `SAP_A4C_BC_TRN_MNG_PC` to ensure that changes can be recorded on transport requests.

Procedure

1. Open the *Maintain Translations* app.
2. Click + (Create) to create a new translation project.
3. Fill in an identifier for your project, a name and description, and the name of the package that should contain your translation project.
4. Click *Create* at the bottom right of your screen.
5. You now need to add your translation project to a transport. You can either use an existing transport request that matches the transport target of the package you have selected for the translation project, or create a new one by clicking *Create*. Save your changes by clicking *Select*.

Results

Your project has now been created and added to the overview list.

Related Information

Select Your Text Sources [page 3212]

6.3.1.21.1.2 Select Your Text Sources

Context

Select the text sources that you want to have translated.

Procedure

1. In the *Maintain Translations* app, open the project you want to add text sources to.
2. Scroll down to *Text Sources* and click *Add*.
3. Here you can select which text source types to add. The following text source types are supported:
   - Application job catalog entries
   - Application job templates
   - Application log objects
   - Business configuration objects
   - CDS type definitions
   - Data definitions (view entities, projection views, abstract entities and custom entities)
   - Data elements
   - Domains
   - Email templates
   - Fiori launchpad app descriptor items
   - Fiori launchpad page templates
   - Fiori launchpad space templates
   - IAM business catalogs
   - Message classes
   - Metadata extensions
   - Text pools
   - Text tables

   It's also possible to specify a package from which all supported translatable elements will be pulled into the translation project. Please note that translatable elements in sub packages are not included.
4. Select *Add* to open the *Add Text Source* dialog which displays all text sources that can be added to the translation project. By filtering the type and name, you can choose the text sources you want to add. Select *Add* to add the text sources you selected to your translation project.

   > ⓘ Note
   >
   > Text sources that can be translated by a translation project need to reside in the same software component as the translation project.

> **Note**
>
> - If you want to test the translated texts of IAM business catalogs directly in the SAP Fiori launchpad of the development system, you have to publish the IAM business catalog once again locally in ADT.
> - If you want to test the translated texts of data definitions or metadata extensions directly in the development system with a SAP Fiori app built on top of these objects, you should first reactivate the objects in ADT to ensure that all caches will be invalidated.

> **Note**
>
> **Text Tables**
>
> Text tables are database tables that were created as part of a business object and in which language-dependent texts are stored. The translation of text table entries is only available in environments that support transporting cross-client data.
>
> A database table needs to fulfill the following requirements to be considered a text table by the *Maintain Translations* app:
>
> - The database table must not be client-dependent, meaning that is must not start with a key field of ABAP Dictionary built-in type `CLNT`.
> - The database table has exactly one key field of ABAP Dictionary Built-in Type LANG.
> - The database table is of delivery class C or S.
> - The database table is located in the same software component as the translation project.
> - The translation of text tables that contain key fields of type p (packed number), i (integer) or x (byte field) is not supported. That includes any kind of UUID and raw types, such as `SYSUUID_X16`.
>
> If these requirements are fulfilled, the database table can be added to a translation project as a text table. Additionally, the following rules apply:
>
> - Every non-key field of the text table that is of an ABAP Dictionary built-in type that is character-like and has a minimum length of 1 will be interpreted as a text attribute for which translations can be maintained.
> - When the translations are published, the rows containing the texts in the target language will be added to the selected transport via R3TR TABU.
>
> In this tutorial (Create an SAP Fiori-based Table Maintenance App with SAP BTP ABAP Environment) on how to create a business configuration application to maintain error code definitions, for instance, the language-dependent texts for the respective error code definitions are stored as entries in the database table `ZERROR_CODE_T###` with `###` being replaced by a number chosen by the developer, such as `000`. The non-key field for which translations can be maintained is field `DESCRIPTION`.

Related Information

Select Your Source and Target Language [page 3214]

6.3.1.21.1.3 Select Your Source and Target Language

Context

Select your source language as well as the languages you want to translate your texts into.

Procedure

1. In the *Maintain Translations* app, open the project for which you want to select the source language and target language. You can choose to select either a single language or more than one language.
2. Scroll down to *Translations* and click *Create*.
3. Select a *Source Language* and *Target Language* from the dropdown menus and confirm with *Create*.

> **Note**
>
> You can select from all of the languages that are installed in the system. In case of ABAP repository objects and text pools, it's only possible to maintain translations for languages other than the original language.
>
> Note that deleting a previously created translation entry for a given source and target language pair will only delete the corresponding entry in the translation project. Already published texts for the target language of the translation will not be deleted from the system.

4. A new entry has now been added to the list.
5. Repeat these steps for all source and target languages.

6.3.1.21.1.4 Generate and Download the XLIFF File

Context

You can generate an XLIFF file for your source and target language(s) that you can download and send to your dedicated translator or translation agency.

Procedure

1. In the *Maintain Translations* app, open the project you want to have translated and scroll down to *Translations*.
2. Select the entry for the source and target language for which you want to generate the XLIFF file.
3. Navigate to *Download*. Here you can select whether the XLIFF file should contain all texts (*All Texts*) or only those texts that were changed since you last published a translated XLIFF file in these languages (*Changed Texts*). If this is the first time you are generating an XLIFF file for the project in this target language, choose *All Texts*.

The XLIFF file will now be downloaded to your computer. You can send it to your dedicated translator or translation agency to be translated.

Related Information

Upload and Publish the Translated XLIFF File to the System [page 3215]

6.3.1.21.1.5 Upload and Publish the Translated XLIFF File to the System

Context

Your texts have been translated and a new XLIFF file has been sent back to you. You can now upload and publish the translated texts into the system. Here's how:

Procedure

1. In the *Maintain Translations* app, open your project and scroll down to *Translations*.
2. Select the entry for the source and target language for which you want to upload and publish the translated XLIFF file.
3. Click *Upload*.
4. Search for the translated file and confirm with *Upload*.
5. Now the translations have been uploaded to the system. They still need to be published, however. Select the entry and click *Publish*.
6. Select a transport to which you want to add the corresponding language (`LANG`) transport entries or `R3TR TABU` entries (in case of text tables) and click *Select*.

   > **Note**
   >
   > If the translation project is already locked on a transport, this transport must be used for publishing the translation.

   Your translations have now been published.

6.3.1.22 Software Component Lifecycle Management

The software component lifecycle management allows you to manage the lifecycle of software components that are available in your ABAP environment landscape.

Purpose

For the lifecycle management of software components you can use the app *Manage Software Components* in your launchpad that displays available software components, and imports them to service instances in your ABAP environment landscape. With the app, you can create new software components and delete them. Due to similarity to the Git protocol and its workflows, the following documentation uses the term **pull** as synonym for **import**.

One software component is comparable to a repository in Git. These "repositories" are centrally stored in separate units and cannot be referenced by other software components, which means the transport of development objects between the components is not possible. All software components are managed by SAP.

Once you have pulled a new software component to a service instance, a new structure package is created. The structure package name corresponds to the software component name. A software component itself is developed in ABAP packages in ABAP Developments Tools for Eclipse. The development objects are then uploaded to the structure package, and the software component is made available for the import to other service instances.

> ⓘ Note
>
> In your system you will see all software components linked to the same global account that is the relevant layer for software component access control. Furthermore, only the global account is relevant for the visibility of the software components. Region or infrastructure provider do not matter. You only need to ensure that all systems are created within the same global account.
>
> This is contrary to the overall SAP approach, where you can use the subaccount level inside a global account as a relevant access control layer (entity separation).

> ⚠ Caution
>
> It is strongly discouraged to perform ABAP cloud development on a software component across multiple development systems. The reason for this is the potential for critical merge conflicts. When multiple developers work on the same software component simultaneously, edit it in different development systems, and release the transport requests afterward, inconsistencies and conflicts can arise in the target system.
>
> This can lead to significant problems in the further development and maintenance of the software. It can even result in data loss. To avoid these issues, performing ABAP development on a software component in only one development system is highly recommended.

Related Information

Manage Software Components [page 3222]

6.3.1.22.1 Supported ABAP Object Types

The following table contains all released ABAP object types that are supported in Git-enabled Change and Transport System (gCTS).

Object Name	Description	Generated
ADSO	DataStore Object	
ADVC	App Variant (LRep cross-client content)	
AIFC	Objects from AIF content	
AIFN	Namespace	
AIFR	Recipient	
AOBJ	Archiving Object	
APCO	Application Components	
APIS	API Release State of Objects	
APLO	Application Log Object	
AREA	InfoArea	
ATOM	ATO Manifest	
AUTH	Authorization Check Fields	
BDEF	Behavior Definition	
BRLC	Business Role Custom	
BUGR	Business User Group	
CDAT	View Cluster Maintenance: Data	
CDBO	Customer Data Browser Object	
CFDA	Custom Fields: BAdIs of Business Context	
CFDF	Custom Field	
CHDO	Change Document	
CHKC	ATC Check Category	
CHKE	Exemptions	
CHKO	ATC Check Object	
CHKV	ATC Check Variant	
CLAS	Class	

Object Name	Description	Generated
CSNM	Core Schema Notation Model	
CTRT	Currency Translation Type	
DCLS	ABAP Data Control Language Sources	
DDLS	Data Definition Language Source	
DDLX	Core Data Services Metadata Extensions	
DEVC	Package	
DMOD	Data Flow	
DOBJ	Data Destruction Object	
DOMA	Domain	
DPTA	Data Protection Template	
DRAS	CDS Aspects	
DRTY	Dictionary: CDS Type Definitions	
DSFD	CDS Scalar Function Definition	
DSFI	CDS Scalar Function Implementation Reference	
DTDC	Dictionary Dynamic Cache	
DTEB	Dictionary Tuning: Entities Buffer	
DTEL	Data Element	
DTPA	Data Transfer Process: Active Version	
EEEC	Enterprise Event Enablement - Event Consumer	
ENHO	Enhancement Implementation Object	
ENHS	Enhancement Spot Object	
ENQU	Lock Object	
ESHA	Enterprise Search: DDL Activation	
EVTB	Event Binding	
EVTO	RAP Event Object	
FUGR	Function Group	
FUNC	Function Module	
G4BA	OData v4 – BEP - Service Group and Assignments	
G4BS	OData v4 – BEP - Service	
GCPM	SAP Gateway OData Client Proxy - Proxy Model	
GSMP	Metric Provider	

Object Name	Description	Generated
HCPR	SAP HANA Composite Provider	
HTTP	HTTP Service	
INA1	InA Service	
INTF	Interface	
INTM	Intelligent Scenario Model	
INTS	Intelligent Scenario	
IOBJ	InfoObject	
IWMO	SAP Gateway Business Suite Enablement - Model	
IWOM	SAP Gateway: Model Metadata	Yes
IWSG	SAP Gateway: Service Groups Metadata	Yes
IWSV	SAP Gateway Business Suite Enablement - Service	Yes
IWVB	SAP Gateway Business Suite Enablement - Vocabulary Annotation	Yes
LRCC	LRepository cross-client content	
LRCD	LRepository client-dependent content	
LRST	LRepository client-dependent content - customizing requests	
MSAG	Message Class	
NONT	SAP Object Note Type	
NROB	Number Range Object	
NTTY	Note Type	
OA2S	OAuth2 Scope Yes	
PCFN	Predefined Field: Extensible Node	
PROG	Program	
RABR	Read Access Logging: Recording	
RADB	Read Access Logging: Dynpro	
RAGB	Read Access Logging: Generic Channel	
RARB	Read Access Logging: Remote Function Call	
RASB	Read Access Logging: Web-Service	
RAWB	Read Access Logging: Web Dynpro	
RDOM	Read Access Logging: Log Domain	
RGRP	Business Role Group	
RONT	SAP Object Type	

Object Name	Description	Generated
RPUR	Read Access Logging: Purpose	
RSDS	DataSource in BW	
RSPC	Process chain	
RSPI	Interrupt Process	
RSPT	Trigger Process	
RSPV	Process variants	
RVBC	Review Booklet Configuration Model	
SAIA	ABAP Development Tools IDE Action	
SAJC	Application Job Catalog Entry	
SAJT	Application Job Template	
SCO1	Communication: Communication Scenario	
SCO2	Communication: Inbound Service	Yes
SCO3	Outbound Service	
SCP1	BC Set or Customizing Profile	
SIA1	IAM: Business Catalog	
SIA2	IAM: Restriction Type	
SIA3	IAM: Authorization Object Extension	
SIA5	IAM: Restriction Field	
SIA6	IAM: App	
SIA7	IAM: Business Catalog App Assignment	Yes
SIA8	IAM: Business Role Template	
SIA9	IAM: Business Role Templ. Bus. Catalog Assignment	
SIAD	Business Role Template Fiori Space Assignment	
SICF	ICFService	
SKTD	Knowledge Transfer Document	
SMBC	Maintainable Business Configuration	
SMIM	MIME Object	Yes
SMTG	OM: Email Template	
SOD1	API Package	
SOD2	API Package Assigment	
SPRV	SOAP Provider Model	
SUCO	Authorization Default Variant	

Object Name	Description	Generated
SUSI	Assignemnt: Service -> Auhtorization Objects (with TADIR)	
SPRX	Proxy Object	Yes
SQL1	ABAP SQL Service	
SRVB	Service Binding	
SRVC	Service Consumption Model	
SRVD	Service Definition	
SUSH	Assignment: Service --> Authorization Objects	
SUSO	Authorization Object	
TABL	Table Definition	
TABU	Table Contents	
TDAT	Customizing: Table Contents	
TOBJ	Definition of a Maintenance and Transport Object	
TRAN	Transaction	
TRCS	Communication Structure for Transformation	
TRFN	Transformation	
TRPC	Translation Project - Customizing	
TRPR	Translation Project	
TTYP	Table Type	
TYPE	Type Group	
UIAD	Application Descriptors for the Fiori Launchpad Enviornment	
UIPC	Fiori Launchpad Page - Customizing	
UISC	Fiori Launchpad Space - Customizing	
UOMT	Quantity Conversion Type	
VDAT	View Maintenance: Data	
WAPA	BSP Page	Yes
WEBI	Service definition - WEBI	Yes
XINX	Ext. Index	Yes
XSLT	Transformation	

> ### Note
>
> Object types flagged as "generated" cannot be created by customers directly but they are secondary objects that are necessary for the functionality of some objects.

Customizing

Object Name	Description
CDAT	View Cluster Maintenance: Data
SNUMS	Number Range Configuration
TABU	Table Entries
TDAT	Customizing: Table Contents
VDAT	View Maintenance: Data

6.3.1.22.2 Manage Software Components

You can use this app to create, display, clone, delete and configurate software components in your ABAP environment landscape. Moreover, you can pull (import) changes from the central software component into other instances.

Purpose

This app allows you to create a software component on a service instance and make with a 'Clone' and import changes (=commits = transport requests) on other instances after an initial clone using a 'Pull'.

Key Features

You can use this app to:

- display available software components
- create new software components
- pull software components and display pulls
- delete software components
- display logs for performed actions like Clone, Pull, and other
- create and delete branches
- display pushed commits (= released transport request)
- create and delete tags on commits
- configure the repository role (Source or Target)
- enable or disable a rollback mechanism for failed pull operations

In addition, the app supports the integration with Git.

> **Note**
>
> Do not restrict the business catalog or role when a custom role is created.

Context: You can create custom business roles and assign the business catalog of the Manage Software Components app as well as restrict the role to a read-only access. However, the Manage Software Compoents app requires write privileges (unrestricted access).

Supported Device Types

- Desktop
- Tablet
- Smartphone

Related Information

Display Software Components [page 3223]
Create Software Components [page 3224]
Pull Software Components [page 3227]
Delete Software Components [page 3229]

6.3.1.22.2.1 Software Components

6.3.1.22.2.1.1 Display Software Components

Context

You want to see which software components exist in the global account and which are already available locally, meaning in a **Cloned** state. To display available software components, perform the following steps:

Procedure

1. Open the **Manage Software Components** Fiori app to view all created software components in the global account.
2. Click on a software component to display more detailed information.

Table Explanation

Column	Description
Name	Name of the software component that has to be unique per service instance. The maximum length of the name is restricted to 18 characters including a namespace.

Column	Description
Description	Description of the software component, which can only contain 60 characters.
Type	Type of the software component. The software component must either be **Development** or **Business Configuration**.
Checked Out Branch	The name of the branch that is currently checked out (active). Empty if not cloned yet.
Git Repository URL	If a customer-managed Git repository is used with the Bring-your-own-Git feature, the repository URL will be displayed.
Cloned	Shows whether or not the software component has been cloned on the current system instance.
Relative Creation Date	Date and time when the software component was created, relative to the current date and time, e.g., '10 days ago'.
Created On	Date and time when the software component was created.
Created By	Email address of the business user who created the software component.
Changed On	Date and time when last changes were made to the software component.
Changed By	Email address of the business user who performed the last changes on the software component.
Repository Role	Shows the selected role of the software component in the current system instance. Either **Source** or **Target**.

6.3.1.22.2.1.2 Create Software Components

Context

To create a new software component, perform the following steps:

Procedure

1. On the *Manage Software Components* screen, select *Create*.
2. In the *New Software Component* popup, select your namespace from the drop-down menu and enter a name for the software component. Optionally you can also enter a description (max. 60 characters), which explains the functions of the component and is displayed in the app. Choose a type for your software component: Select either *Development* or *Business Configuration* from the drop-down menu.

Software Component Type	Description
Development	Used for standard ABAP application development.

Software Component Type	Description
Business Configuration	Used to transport customizing content from one ABAP environment system to another ABAP environment system. **Note** You can create multiple business configuration software components. You can only clone one business configuration software component to a system. You can only release customizing transports to a business configuration software component

3. Click *Save* to make the software component centrally available.
4. Back on the *Manage Software Component Lifecycle* screen, choose *Go* to display the new software component in the list.
    1. As soon as your software component is cloned into a service instance, a structure package with the same name as the software component is created there. The structure package can contain several ABAP packages. See Clone Software Components [page 3226].
    2. The instance is integrated with the ABAP Development Tools (ADT), where ABAP development objects are created in development packages and uploaded to the structure package.

        Note

        Note that you have to create a development package first. Afterwards, create ABAP development objects in this newly created development package. The development package is a sub-package of the structure package dependent on the software component.

        → Tip

        We recommend that you add all your cloned software components (structure packages) to your *Favorite Packages* in eclipse to a have an easy and quick access to it.
    3. After the transport of the structure package, the software component becomes available for all instances and can be cloned to other system instances.
    As soon as the structure package is transported, for example, to provide a new version of the software component, the date and time of the transport as well as the user who triggered the transport are displayed in the *Branching* section of the detail page of a software component. More information about the released content can be found when navigating on the used branch.

Note

Note that a maximum of 99 software components can be created per customer.

6.3.1.22.2.1.3 Clone Software Components

Context

If a software component is not locally in your system yet, you will first need to clone it once to import it into your system.

You can now deactivate the rollback mechanism for clone actions in the Manage Software Components Fiori App. The clone rollback should only be deactivated in exceptional cases.

Example of a use case: If you have a faulty remote repository (software component) but no development system on which the repository (software component) is still available locally. Then you need a way to clone the faulty software component and fix the errors.

> ⓘ Note
>
> Cloning a software component is only necessary once per system instance.

Procedure

1. Go to the details page of the software component you would like to clone.
2. Click *Clone*.
3. A pop-up opens. You can now select which branch you would like to import. If the software component does not have branches yet, the master or main branch (dependent on when the repository was created) is automatically selected. Select *By Tag* if you want to clone a specific git tag, which has been assigned to a commit. Select *By Commit* if you want to clone a software component with a specific commit ID. Confirm by clicking *Clone*.

   > ⓘ Note
   >
   > There is no input validation for the entered values for the options *By Tag* and *By Commit*. If the entered values are not valid the cloning process will start, but will be shown as failed in the end of the cloning process. In this case, the clone *Latest* option is automatically called as a fallback. With this, the repository is still useable, but not in the orginally requested version.

   > ⓘ Note
   >
   > If the initial clone of a software component runs into errors, the partly imported objects will be removed again from the system instance and the system will be kept clean. The clone rollback is now available for all system types.
4. The software component with the selected branch is now imported into your current system instance. The *Clone* button turns into a *Pull* button. From now on, you can use it to pull (remotely available) changes of your software component to your system.

> ⓘ Note
>
> It is recommended to clone multiple software components one by one. Please clone software components with objects that are reused by other components first.

Related Information

Pull Software Components [page 3227]

6.3.1.22.2.1.4 Pull Software Components

Context

You can pull (remotely available) changes of your software component to the service instance you are currently on. To do the pull, perform the following steps:

> ⓘ Note
>
> Keep in mind that if a software component is not in your system yet, you will first need to clone it once. For more information see Clone Software Components [page 3226].

Procedure

1. Navigate to the detail page of a software component from the list and choose *Pull* in the page header.
2. Select how you want to pull the software component. By selecting *Latest*, the last remote commit will be pulled into the system. Select *By Tag* if you want to pull a specific git tag, which has been assigned to a commit. Select *By Commit* if you want to pull a software component with a specific Commit ID.

   > ⓘ Note
   >
   > To pull a specific tag, at least one tag has to exist on a commit on the currently active branch.

   > ⓘ Note
   >
   > A rollback mechanism is activiated which allows an automatic roll back to the last valid commit of the repository, in case an invalid commit is pulled. We define an invalid commit as a commit that can cause import errors during a pull operation. The pull of invalid commits is allowed for development and test systems only. If only one out of multiple commits causes an issue, none of the commits will be pulled.
   >
   > Nevertheless, this mechanism cannot reverse any data loss that may occur due to ABAP Dictionary changes. For example, in the case of field shortening of a database table, the field length can be rolled back, but the lost content remains irretrievable. Therefore, ensure that deployments to the production systems are checked in the advance (via CI/CD processes), for example on quality systems.
3. Display the list of all pulls, checkouts and other actions by navigating to the *History* tab.
   To display pulls and checkouts that have been triggered for a particular software component, first select the component from the list and then scroll down to the *History* section where all actions are listed in the *Recent Actions* tables. In both cases, you will be navigated to the *Software Component Details* screen.
4. To display or refresh the list of *Recent Actions*, choose *Go* or click on the refresh button inside the search bar.
   Field Description

Column Name	Description
ID	A unique key (UUID) of the software component. This column is initially hidden.
Branch Name	The branch which was used for the executed action.
Short Commit ID	The short commit id that was used for the executed action.
Tag	The name of the tag which is eventually assigned to the used commit.
Last Status Change	The relative time from current system time and the timestamp of the last status change.
Status	Displays whether the pull or checkout process has been successful, is still running, or there has been an error. **Note** Only one pull can be in status *Running*. If you trigger several pulls, other software components will switch to status *Error*. If an error has occurred during a pull, you need to click Pull again.
Started By	E-Mail address of the user who started the pull.
Start Time	Date and time when the pull has been triggered.
Change Time	Date and time of the changes during the pull procedure.

Note

The `<ID>`, which is a unique key (UUID) of the software component is not in the table, but visible in the action detail page.

5. You can display the pull details by choosing the entry from the list.
   The *Execution Log* and *Log Overview* are displayed as a table. You can find the new tables when navigating to the detail page of every action entry. Additionally, every log line is classified with a criticality level.
   A logline can have the criticality level "Information", "Success", "Warning" or "Error". Each level is represented with a colored criticality icon.
   Furthermore, you can also download the logs as an Excel-file. This may be helpful when communicating with SAP in the context of error analysis.

> **Note**
>
> Note that the execution logs and transport logs are deleted periodically. Please use the download feature in the UI or the OData service `MANAGE_GIT_REPOSITORY` (see Pulling Git Repositories to an ABAP Environment System [page 948]) to download the logs.

Field Description

Column Name	Description
Index	Index number to sort the table properly.
Log	Name of the log.
Severity	Can have the status success, warning, error or other.
Transport Request	Transport request ID which was used for the log.
File inlcuding Path	The system file path where the log files are saved.
Timestamp	Date and time when the log was created.

Related Information

Clone Software Components [page 3226]
Configure Software Components [page 3235]

6.3.1.22.2.1.5 Delete Software Components

Context

You want to delete a software component in order to:

- get rid of software components that are no longer used.
- get rid of software components that have been created by mistake (such as a typo in the software component name).

For every delete activity, the executing business user is documented in a log.

Procedure

To delete a software component, perform the following steps:

1. In the *Manage Software Components* app, pick the software component that you want to delete out of the list.
2. Click onto the software component to be redirected to its detail page.
3. Choose the *Delete* button in the upper right corner.

4. You will be prompted to confirm the action and are able to choose if you want to delete only the local repository or the local repository in combination with the remote repository.

Software Component Deletion in a Service Instance

Types of Software Component Deletion

Empty Software Component	Used Software Component
The software component contains no object locally in the service instance and no changes are committed to the remote repository. The remote repository, the associated structure package of the software component, and the software component itself are deleted. The deletion of the ABAP structure package and software component is executed without creating a transport request.	The software component contains objects locally and possible changes are committed to the remote repository. The remote repository is then deleted and the software component is locked. Since the software component is locked, no object changes can be released (pushed to remote) anymore.

Local and Remote Deletion - SAP-managed Software Components

	Local Deletion	Local and Remote Deletion	Safe Deletion
Use Case	Only the local representation of the software component is deleted. Local means all ABAP objects including the parent structure package.	When deleting local and remote representations, local means only the current system instance is deleted. *The software component will be remotely deleted. As a consequence, no actions can be performed anymore. Please make sure to first delete the software component locally on all system instances where it's cloned. The component can't be deleted from the other systems after it's remotely erased. This action can't be undone.*	The Safe Remote Deletion feature ensures that: • No repository or branch is deleted while still in active use. • All systems maintain consistent software component and branch states. • You can manage repositories safely without risking data inconsistency or broken links.

Consequences	You can re-clone the software component again. **⚠ Caution** A local deletion can be critical when database tables are deleted. The local (not transported) table entries contained in this table will be lost. This is a possible scenario in test or production systems, since development systems usually don't contain any business data (like table entries).	This step should be performed as a last step once all local deletions on other system instances have been performed. **ⓘ Note** A distributed local deletion (deleting local software components on other system instances than the current one) is not possible.	This utility prevents the remote deletion of a software component if it is still cloned on any local system.

Local and Remote Deletion - Customer-managed Software Components (Bring Your Own Git)

	Local Deletion	**Remote Deletion**
Use Case	Only the local representation of the software component is deleted. Local means all ABAP objects including the parent structure package.	You can delete the software component remotely if you select *Unregister*. This option appears only for components that are not cloned. The linked Git repository will be unregistered from the Manage Software Components app. You cannot use the linked Git repository with the software component again, nor can you link it with new software components. This action is irreversible. **ⓘ Note** Make sure to first delete the software component locally on all system instances where it's cloned. Your git repository in your selected git provider will not be deleted.
Consequences	You can re-clone the software component again.	The remote Git repository cannot be used again or linked with a new software component.

Resulting behavior in service instances where the deleted software component is still imported

1. No further Git actions, including pull, switch branch and others, are possible.
2. Changes to ABAP objects of the deleted software component can no longer be released.
3. Non-released requests, in which object changes have already been recorded before the software component was deleted, must be handled specially.

For example, you can delete the recorded object changes from the transport request. Alternatively, the transport layer of the transport request can be deleted.

Result

The selected software component is deleted centrally if the checkbox *remote deletion* for SAP-managed or the *unregister* option for customer-managed components was selected. Otherwise it is only a local deletion, affecting the current system instance.

> ⚠ Caution
>
> Here, centrally means the repository visible throughout the global account is deleted.

You can no longer pull or clone the software component to a service instance.

If this software component has already been cloned to a service instance, objects are not deleted in the ABAP instances but the status of the software component and all belonging objects are changed to *read-only*. You're not able to make any changes to the dedicated structure package and the development package that is contained in there. If you want to delete the objects, you must delete them and import the deletion into other systems, before you delete the software component.

Restoring Deleted Software Components

> ⓘ Note
>
> Currently, you can't restore software components that have been deleted. Use new software component names instead of reusing previously deleted software components or software component names.

Safe Deletion

Overview

The Safe Remote Deletion feature in the Manage Software Components app is designed to ensure system integrity by preventing the accidental deletion of active repositories or branches. This feature protects against the creation of ghost software components (SC): local clones that become disconnected from their remote repositories and lose pull and push capabilities.

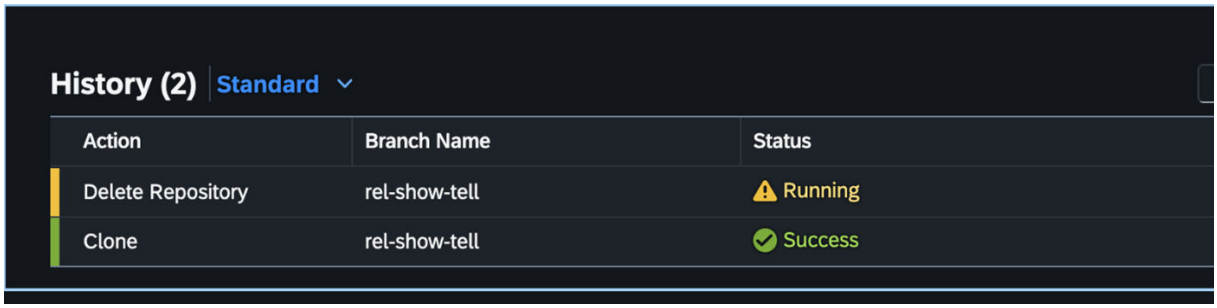
> ⚠ Caution
>
> Currently, this feature is available only for SAP managed software components.

Behavior Demonstration

Repository Deletion Scenario

1. Create a new software component in the Manage Software Components Fiori app.
    1. Navigate to *Manage Software Components* in your Fiori launchpad.
    2. Click on the *Create* button.
    3. Fill out all the necessary information.
2. Clone the software component into two different systems.
    1. Click on *Clone* for the newly created software component.
    2. Fill out the required information, such as the branch to be checked out, the repository role and rollback mechanism.
    3. Clone the software component on another system instance in the same global account as well.
3. Attempt to delete the remote software component from one system.
    1. Click on the *Delete* button.
    2. Select the checkbox: *Delete remote repository*.
    3. Enter the name of the software component.
    4. Select the checkbox: "I understand the consequences".
    5. In the end click on *Delete*.

The deletion will be triggered asynchronously in the background. Search for the software component and open it. Under the tab *History*, you will now be able to see the running deletion process as shown in the picture below.



After a few minutes, refresh the table and you will see *Error* as the status. To see more details, click onto the action, where you will be able to navigate to the *Execution Log* for more detailed error messages.

Expected Result:

The deletion fails because the software component is still cloned in another system.

6.3.1.22.2.1.6 Display the Action History

Context

You want to see which actions were already executed for a specific software component. To display the action history, perform the following steps.

Procedure

1. Open the **Manage Software Components** Fiori app.
2. Select the software component you are interested in.
3. Scroll down to the section **History** to display the table.

Table Explanation

Column	Description
Action	Type of action that was executed. This could be a **Create Repository**, **Delete Repository**, **Checkout**, **Pull**, **Create Tag** and more.
Branch Name	Name of the branch that was used for this action, e.g., the branch that was check out (switched to).
Status	This attribute indicates whether the action is currently running, was successfully completed, or encountered errors (stopped).
Started By	Name of the user who executed the action.
Short Commit ID	ID of the Commit that was used for this action, e.g., the commit that was pulled.
Tag	Name of the tag that was used for this action.
Start Time	Date and time when the action was started.
Change Time	Date and time when the status (or executed step) of the action was changed.
Last Status Change	Date and time when the status (or executed step) of the action was changed in relation to thecurrent date and time, e.g., '10 days ago'.

6.3.1.22.2.1.7 Display Open Transport Requests

Displaying Open Transport Requests: Use this to easily view all open transport requests for a specific software component. It provides the unique identifier, description, owner, and last update of each request.

Context

To view all open transport requests for the selected software component on the current system instance, follow these steps. Please note that open transport requests from other system instances will not be displayed.

Procedure

1. Open the **Manage Software Components** Fiori app.
2. Select the software component you are interested in.
3. Scroll down to the section **Open Transports** to display the table.

Table Explanation

Column	Description
Transport Request	ID of the ABAP transport request.
Description	Description of the ABAP transport request.
Owned By	The name of the ABAP user who currently owns the transport request.

Column	Description
Updated At	Date and time of the most recent change to the transport request.

Navigate from Transport Request to ABAP Development Tools for Eclipse

You can navigate to the open transport request in ABAP development tools for Eclipse by clicking on the transport request in the software components application table. This link opens up a popup.

Here, you can click on the *TransportRequest* button to diyplay the respective tansport request in ABAP development tools for Eclipse.

In case nothing opens up when clicking on *TransportRequest*, the following must be activated in ABAP development tools for Eclipse:

ABAPDevelopmentTools > *Settings* > *General* > *Link Handlers* > *enable ADT Scheme entry*. The link can then be opened.

6.3.1.22.2.1.8 Configure Software Components

Context

You can configurate software components in the *Repository Settings*, which can be opened by clicking on the *Settings* button.

Key Features

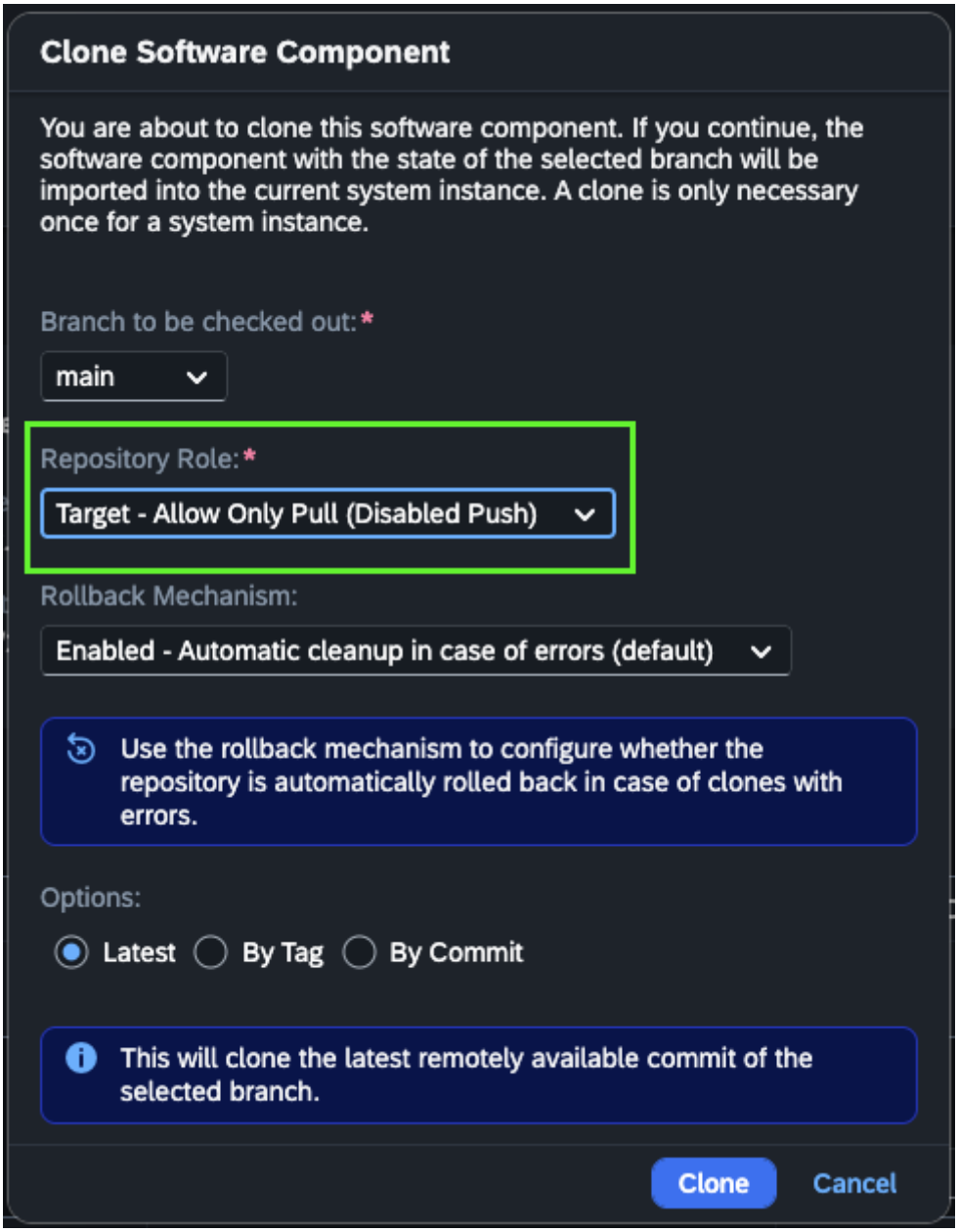
You can do the following adjustments to the selected software component:

- Repository Role:
  Use the repository role to configure how transport requests behave when they're released. If the option *Target* is selected, the transport requests will not be pushed to the remote repository. However, if the *Source* option is selected, the transport request can be pushed to the remote repository.
- Rollback Mechanism:
  This rollback mechanism can be enabled and disabled. If enabled, only valid commits can be imported in the repository. If disabled, all commits can be imported even if they are invalid. For more information about this mechanism, see: Pull Software Components [page 3227].
- Update Credentials (BYOG software components only):
  Use this option to update your Git user credentials that were saved when the software component was cloned. This option is enabled only for Bring Your Own Git components that have already been linked to a Git repository. For more information on that, see Bring Your Own Git [page 3249].

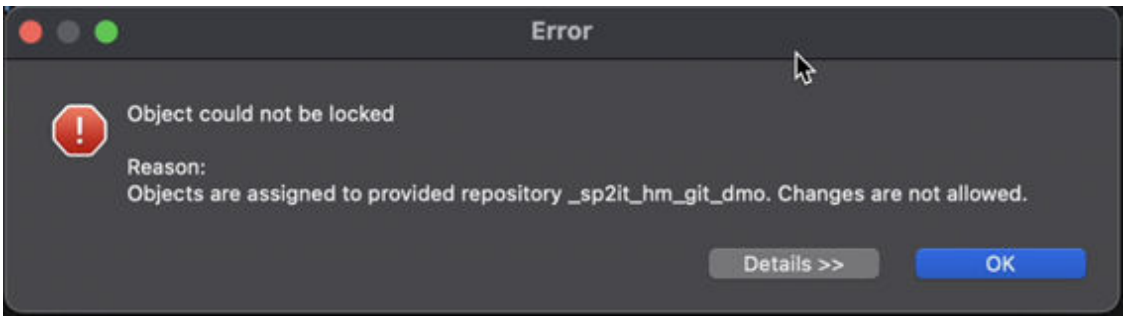
Repository Role: Target

Software components configured with the role *Target* are now protected from changes and cannot be recorded on transports. This feature is designed to prevent inadvertent modifications to components, ensuring their integrity and stability.

1. *Detailed Goal Description*
    - Importance: Protecting components from unauthorized or accidental modifications is crucial in maintaining system stability and reliability.
    - Benefits for Users: This feature ensures that objects associated with *Target* role software components remain unchanged, reducing the risk of potential errors and preserving the intended configuration.
2. *Behavior Demonstration*
    - The following steps demonstrate the behavior of software components with the *Target* role:
    1. Clone a software component (SC) configured with the role *Target*.



2. Open an object (e.g., a class) within the software components package using ABAP development tools for Eclipse.
3. Attempt to modify the object.
4. An error window should appear, blocking the modification and providing a reason for the restriction.



3. *Summary*
   The *Target* role feature safeguards software components from accidental changes by locking their objects against modification. Implemented to enhance system integrity, this feature is essential in environments where stability and consistency are of utmost importance.

   > → Tip
   >
   > If you need to modify the contents of a software component, clone it in the *Source* role instead.

6.3.1.22.2.2 Branches

6.3.1.22.2.2.1 Work with Branches

Context

To allow you to easily work on and switch between different versions of code, you can create branches, a copy of your existing code at a certain point in time, that you can check out and work on without changing your original code.

Find out how to work on branches below. To find out how to create, check out and switch between different branches, please refer to the following documentation pages.

Procedure

1. Upon opening the **Manage Software Components** app, you'll see a list of all available software components. Select the one you would like to work on.
2. Scroll down to *Branching* to see a list of all available branches for your component. The table shows you which branch the different branches originally derived from and which branch is currently checked out. The first branch is always named 'main'. The main branch will be created directly with the creation of the software component. If a branch was successfully created, it should directly be visible on all other system instances.
   The column *Delta* shows information about the currently active/checked-out branch. It displays how many commits the branch is behind the remote repository (1 commit behind remote) or just informs, that the branch is up to date (No new commits). This information is only valid for the currently active branch.
   If the branch is not active, the column value will be set to "Please checkout first". If the branch is multiple commits behind, the column value will be set to "X commits behind remote", where X is the number of new commits on the remote branch.
3. Note that you need to clone the repository once before you can check out a branch. Clone the repository by clicking *Clone*.

Merging of Branches - SAP Managed Software Components

> ⓘ Note
>
> The merging of branches of SAP-managed software components is currently not supported.

Merging of Branches - Customer-managed Software Components

It is possible to merge branches of a software component with Bring Your Own Git. However, this will have implications on the usability of your software component and is not officially supported using the

Manage Software Components Fiori app. If you choose to merge branches of a customer-managed software component, please refer to the two options below:

Option 1: Merging of Branches Using the Git Provider UI

You need to do a double maintenance for all the changes made to a branch that should be merged. This double maintenance needs to be performed on an SAP Business AI Platform, ABAP environment, so that the changes exist in both the source branch and the target branch.

If the double maintenance has not been done correctly, and changes to the same object are present in both branches, Git might detect a merge conflict.

In case there are merge conflicts produced during the merge attempt of the two branches, please maintain your changes in both branches respectively. Once all changes are available in both branches, they can be merged on the remote repository.

It is essential not to resolve merge conflicts using a merge conflict resolver by your Git provider. If the individual line content was altered during the merge conflict resolution, this commit might not be able to get pulled into an ABAP environment.

Option 2: Merging of Branches Using the Git Command Line Interface (CLI)

With this approach, it is mandatory to clone the remote Git repository to your local machine.

After the repository is cloned to your local machine, you need to perform the merge of the branches using either the `ours` or `theirs` strategy option. This approach ensures that the merging of branches does not alter individual line contents, butcontents but takes full objects from one side to the other instead. This way, the resulting commit from the merge is importable into an SAP Business AI Platform, ABAP environment. As a reference, please take a look at the example below on how to perform a merge using the `theirs` strategy.

Sample Code

```
git clone <Repository_URL>
cd <Repository_Name>
git checkout <baseBranch>
git merge -X theirs <mergeBranch>
git push origin <baseBranch>
```

Note

Currently, the merging of branches is not officially supported in the Manage Software Components Fiori app using customer-managed software components.

6.3.1.22.2.2.2 Create a Branch

Context

You want to create a new branch for your software component.

Procedure

1. Open the **Manage Software Components** Fiori app.

2. Choose the software component you want to work on.
3. Navigate to the section **Branching** and click on **Create**.
4. Choose a name for your new branch and select the parent branch from the drop down menu.
5. Confirm with **Create**.
   You can also create a branch based on a specific commit state. This referenced commit must be available in the current system, i.e. included in the currently selected branch. To do this, simply fill in the Commit ID in the Create New Branch popup. The commit ID can be provided in short or long format. Note that short commit IDs should only be used if the software component has already been pulled. Otherwise, the long ID needs to be used.

   > ⓘ Note
   >
   > When you delete a branch and recreate it with the same name, the creation will be denied with an error popup saying "Branch cannot be recreated". As a consequence, a branch name has to be unique once it has been used. Please choose another name in that case.
6. Refresh the page. Your new branch has now been created and added to the table.

6.3.1.22.2.2.3 Switch a Branch (Checkout)

Context

You previously created a new branch and added it to the table. To learn how to create a new branch, please navigate to the previous page. Now you want to work on the new branch by checking it out. Please be aware that a new checkout is not possible while a checkout or pull is already in progress.

> ⓘ Note
>
> All objects in the target branch will overwrite existing objects in the system without further notice. It is important to ensure that all open transport requests in the current branch are released before switching to a new branch.

Procedure

1. After you created a new branch and refreshed the page, you navigate to it.
2. Choose the **Checkout** button to switch to this new branch.

6.3.1.22.2.2.4 Display Branches

Context

You want to view all branches of a software component, including both remote and local branches, regardless of the currently active branch. To display the list of branches, follow these steps:

Procedure

1. Open the **Manage Software Components** Fiori app.
2. Select the software component you are interested in.
3. Scroll down to the section **Branching** to display the table.

Table Explanation

Column	Description
Branch	Name of the branch. The **main** branch is created by default.
Checked Out	Shows if the branch is currently active in the system. Only one branch of a software component can be active (checked out) in a system instance. Is either **Yes** or **No**.
Delta	This column indicates whether the currently active local branch is in sync with the remote repository. If it is up to date, it will display **No new commits**. If there are new commits, it will show a message similar to **2 commits behind**. To retrieve the latest changes from the remote repository, you can carry out a pull action. Please note that a pull includes all commits that occurred before the specified commit; there is no option for cherry-picking or skipping commits.
Last Remote Commit ID	The commit ID of the most recent commit on the branch of the remote repository.
Last Remote Commit Message	The commit message of the most recent commit on the branch of the remote repository.
Last Remote Commit	Relative date and time of the most recent commit pushed to the remote repository on this branch.
Last CommitON	Date and time of the most recent commit pushed to the remote repository on this branch.
Local Short Commit ID	The currently checked-out commit ID in short format.
Created By	Name of the user who created this branch. Note that this information is only available for SAP managed repositories and not for customer-managed repositories.
Created On	Date and time when this branch was created. Note that this information is only available for SAP managed repositories and not for customer-managed repositories.

6.3.1.22.2.2.5 Delete a Branch

> ⚠ Caution
>
> **Branch Deletion**:
>
> While it is possible to delete branches, please consider the possible negative effects of branch deletion, like losing the commit history.
>
> Please make sure that the branch you are deleting is no longer in use on other systems. If you delete the branch while it's in use in other systems, this code can no longer be released there, even if there are open transport requests.
>
> The branch will be deleted locally as well as on the remote repository. After the branch deletion, a pull of this branch is no longer possible on all system instances.

Safe Deletion

Overview

The Safe Remote Deletion feature in the Manage Software Components app is designed to ensure system integrity by preventing the accidental deletion of active repositories or branches. This feature protects against the creation of ghost software components: local clones that become disconnected from their remote repositories and lose pull and push capabilities.

> ⚠ Caution
>
> Currently, this feature is available only for SAP managed software components.

Summary

The Safe Remote Deletion feature ensures that:

- No repository or branch is deleted while still in active use.
- All systems maintain consistent software component and branch states.
- Developers can manage repositories safely without risking data inconsistency or broken links.

Safe Branch Deletion

This feature prevents the remote deletion of a branch that is currently checked out (active) in any local repository.

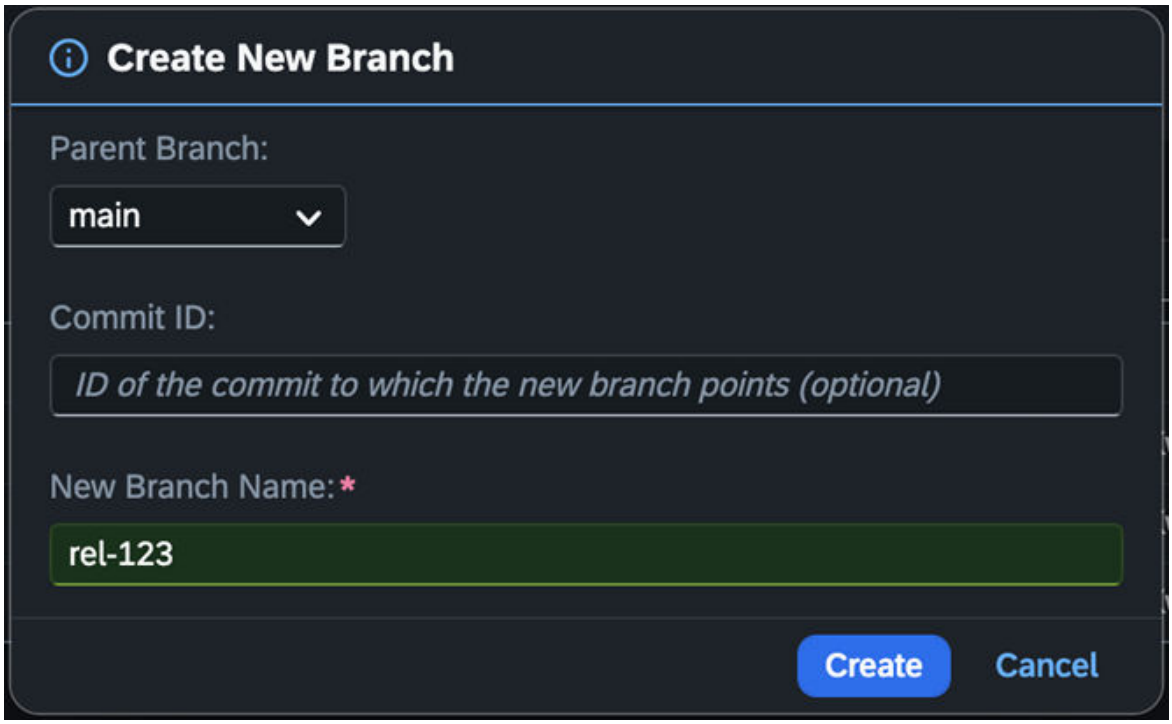
Ensures:

- No local repository references a non-existent branch.
- Ongoing development remains consistent across systems.

Branch Deletion Scenario

Steps:

1. Create a software component in the Manage Software Components app.
2. Clone the software component into two different systems.
3. In one system, create and check out a new branch (e.g., `rel-123`).



4. In the second system, refresh the branch list.
5. Attempt to delete the branch `rel-123` remotely.

Expected Result:

The deletion fails because the branch is active in another system.

Safe Remote Deletion Procedure

To safely delete a branch, you need to...

1. Check out another branch in all local repositories where the branch is active.
2. Initiate the remote deletion once the branch is no longer active anywhere.

> → Tip
>
> Before performing any deletion operation, always verify clone and branch usage in all systems to ensure a clean and safe deletion process.

6.3.1.22.2.3 Commits

6.3.1.22.2.3.1 Display Commits

Context

You want to view all commits of a branch. To display the list of commits, follow these steps:

Procedure

1. Open the **Manage Software Components** Fiori app.
2. Select the software component you are interested in.
3. Scroll down to the section **Branching** to display the table.
4. Select the branch you are interested in.
5. Scroll down to the section **List of Commits** to display the table.

Table Explanation

Column	Description
Pulled	Shows if the changes of a commit are in the current system instance. Either **Yes** or **No**. **Note** If a branch is not active, all commits are indicated as **No**, despite the commit may also be in the commit history of the active branch.
Short Commit ID	ID of the commit in short form.
Message	Commit message that begins with the transport request ID followed by the transport request descirption.
Author	Commit author, meaning the e-mail address of the ABAP business user (CB-user) that owned and released the transport request.
Pushed to Remote	Relative date and time when the transport request was released from the source system and therefore pushed to the remote repository.
Pushed to Remote On	Date and time when the transport request was released from the source system and therefore pushed to the remote repository.
Tag	Shows the last tag name if assigned.
Tag Description	Shows the last tag description if assigned.
Tagged By	Shows the last tag creator if assigned.

6.3.1.22.2.3.2 Display Commit Objects

Context

You want to view all objects that were changed with a specific commit. To display the list of objects, follow these steps:

Procedure

1. Open the **Manage Software Components** Fiori app.
2. Select the software component you are interested in.
3. Scroll down to the section **Branching** to display the table.
4. Select the branch you are interested in.
5. Scroll down to the section **List of Commits** to display the table.
6. Select the commit you are interested in.
7. Scroll down to the section **Committed Objects** to display the table.

Table Explanation

Column	Decsription
Object Type	ABAP object types like `CLAS`, `INTF`, `APIS` and many others. For more information, see ABAP Object Types.
Object Name	Name of the ABAP object that was touched.
Action Type	Shows what kind of action was performed on the object. Can be either **Created**, **Modified** or **Deleted**.

6.3.1.22.2.3.3 Export Commits using SAP Cloud Transport Management

Context

1. A developer creates and modifies ABAP coding in the SAP Business AI Platform, ABAP environment by using the Eclipse-based IDE (ABAP development tools for Eclipse) with a rich debugger, troubleshooting, and testing tool support.
2. The changes are recorded in an ABAP transport request that is finally released. During the release, the changes are committed and pushed to the configured development branch of the corresponding Git repository (there is a one-to-one relation between an ABAP software component and a Git repository).
3. After testing and after the release decision, a release branch is created from the main branch.
4. The resulting combination of the corresponding software component and either the commit ID, Git tag, or the branch name is used as a reference by the following export call to the SAP Cloud Transport Management service. It is recommended using the commit ID, because the latest commit of a branch may change, resulting in unreproducible deployments.
5. In the SAP Cloud Transport Management system a transport request is created containing the reference and added to the import queue of the following SAP Cloud Transport Management node (given by the configured service transport route).

6. Finally, a transport administrator uses the cloud transport management service UI to generate the import to the SAP Business AI Platform, ABAP environment system configured on the service transport route. In doing so, the service calls an import API of the target SAP Business AI Platform, ABAP environment system by passing the reference.
7. The import is done asynchronously, and the progress status is monitored by the service. When the import is finished, the service receives the transport logs from the target SAP Business AI Platform, ABAP environment instance.

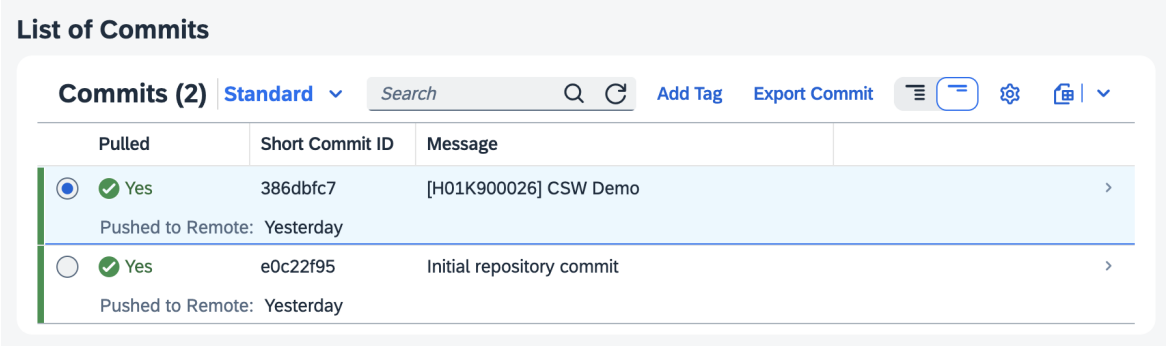
Procedure

Before we can start releasing transport requests that generate commits, a new communication arrangement is needed in order to use an endpoint.

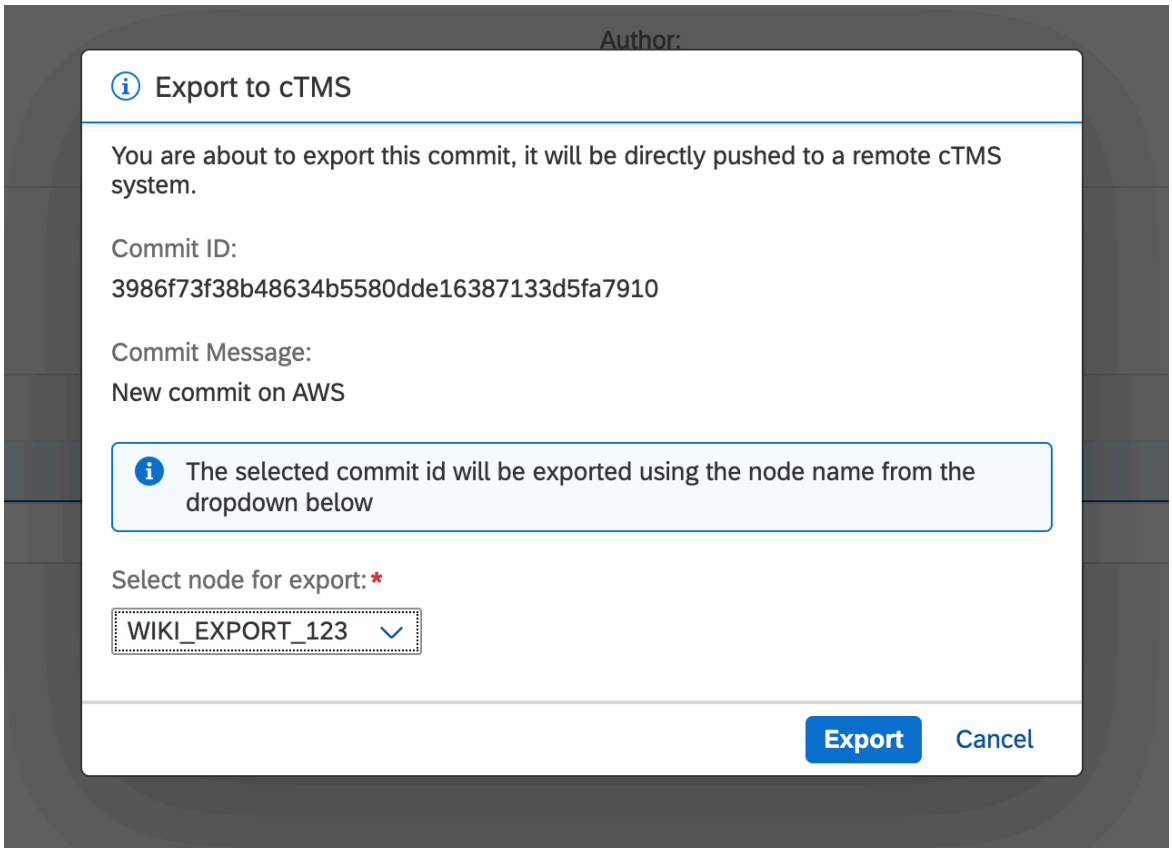
1. Create a service instance and a service key of SAP Cloud Transport Management service. For more information, see Creating a Service Instance and a Service Key.
2. Navigate to the Communication Arrangements Fiori app.
3. Click on *New* and search for `SAP_COM_0599` communication scenario in the list. Type in the name. Click on *Create*.
4. On the following screen, a new communication system should be created. Click on *New*. Type in your System ID, which is the URI from your service key. Click on *Create*.
5. On the communication system creation page find general properties and define the *Host Name* parameter according to the URI property in the service key created for SAP Cloud Transport Management service. For example *https://transport-service-app-backend.ts.cfapps.eu10.hana.ondemand.com*
6. Then find *OAuth 2.0 Settings* and put in a *Token Endpoint*, which is the URL from your service key. For example tmststest.authentication.sap.hana.ondemand.com/oauth/token?grant_type=client_credentials .
7. Now it's time to create a user for outbound communication. Click on + and select oAuth 2.0 from the authentication method dropdown.
8. Type in your OAuth 2.0 client ID and client secret that can be taken from the service key for the basic authentication. Click on *Create*.
9. Define a SAP Cloud Transport Management node name that you will need for the export. The selected transport node must allow uploads and you should use the same name when creating the communication arrangement and creating the same in the SAP Cloud Transport Management.
   For more information, see Create Transport Nodes. Click *Save* afterwards to complete the process.

Export to SAP Cloud Transport Management using the Manage Software Components app

1. Open the *Manage Software Components* app and navigate to the desired software component and a branch.
2. Select a commit for the export and click on the *Export Commit* button.



3. A new dialog will open, where you will be prompted to select a SAP Cloud Transport Management node you want to use for this export.



4. Click on *Export* after you are ready with the selection. A status modal in the center of the screen will let you know if the process was completed successfully.
5. In case of an error, you will get an HTTP error code with a description.

Troubleshooting

Do not use a service key created for the SAP Cloud Transport Management instance of the plan **transport_operator** to establish integration with cTMS. This plan is not permitted to perform an export. The error shown in the **Manage Software Components** Fiori app for this is **access denied**.

To run an export, you need to use a service key created for the SAP Cloud Transport Management instance of either the **standard** or **export** plan.

Related Information

Set Up the Environment to Transport Content Archives directly in an Application
Configuring the Landscape
Using the Import Queue
Creating Destinations for Deployment of References of SAP BTP ABAP environment

6.3.1.22.2.4 Tags

6.3.1.22.2.4.1 Create a Tag

Context

You can assign a tag name and tag description to individual commits.

Procedure

1. In the *Manage Software Components* app, open your software component.
2. Navigate to a branch. Commits on this branch are displayed in the *List of Commits*.
3. Select a commit and click *Add Tag* to add a tag name and a tag description.
4. **(Optional)** Navigate to the *Tagging* tab of your software component to see a table listing all the tags on the branch that is currently checked out/active. Log entries for the creation and deletion of tags are also displayed in the *Recent Actions* table in the *History* tab.

> ⓘ Note
>
> Note that a commit can have multiple tags assigned to it. There is no restriction on how many tags a commit can contain.

6.3.1.22.2.4.2 Display Tags

Context

You want to view all tags that are assigned to commits of all branches, independently of what branch is active. To display the list of tags, follow these steps:

Procedure

1. Open the **Manage Software Components** Fiori app.
2. Select the software component you are interested in.
3. Scroll down to the section **Tagging** to display the table.

Table Explanation

Column	Description
Tag	Name of the tag assigned to a commit
Description	Description of the tag assigned to a commit.
Commit ID	ID of the commit the tag is assigned to.
Created On	Date and time when the tag was created.
Created By	Email address of the business user who created the tag.

Column	Description
Changed On	Date and time when last changes were made to the software component.
Changed By	Email address of the business user who performed the last changes on the software component.

6.3.1.22.2.4.3 Delete Tags

Procedure

1. Open the **Manage Software Components** app.
2. Choose the software component where you want to delete the tag.
3. Scroll down to the section **Tagging** to display the table.
4. Here, you can select the tag you want to remove and click the **delete** button.

6.3.1.22.2.5 Bring Your Own Git

Up until this point, the ABAP repositories in SAP Business AI Platform, ABAP environment were managed by the Manage Software Components Fiori app, which was the only interface where users could see the status of their remote repository. Bring Your Own Git (BYOG) allows users to connect their own Git repositories with the Manage Software Components app. In doing so, the user has a better overview of their own code, branches, and commits.

> ⚠ Caution
>
> However, the user is not allowed to make code changes in their remote Git repository and import these changes into the local SAP Business AI Platform, ABAP environment system instance, as this might violate any guidelines for ABAP cloud development.

See Supported ABAP Object Types

Prerequisites

- You have created a new repository with the preferred Git provider. The repository does not have any objects except for a readme file.
- You have created a branch in your repository before cloning it.
- Your repository is publicly accessible on the internet. Repositories hosted on a private company network are currently not supported.
- You have administrative authorizations to push, pull and clone from the repository. Credentials must include a username and a password or token.

- The developer users have administrative authorizations to the **Manage Software Components** app.
- Production, development and/or test system instances are in the same global account.
- The Git provider must trust a (root) certificate that is available in the SAP standard certificate trust list. Please check the current trust list here and compare it with your Git provider certificate.

Restrictions

- Moving a BYOG software component from one global account to another is not supported.
- Changing the URL of the remote repository after it has been linked to a software component is not allowed. After changing the URL, the repository can no longer be used.
- It is important to distinguish between the user (pusher), whose credentials are entered during the clone process, and any other ABAP users (committers) that develop and release objects to be pushed to the remote repository. Although the committer will be noted in the commit history, the release and later the push will technically be performed using the credentials of the pusher. Therefore, it is recommended thtat the pusher does have push, pull and clone authorizations.

> **Note**
>
> If you are the actual owner of the repositories and credentials used for your BYOG scenario, please acknowledge the following task. It is necessary for you to update or rotate the used credentials regularly on a two to four week basis. This is an important step to increase security, minimize risk and prevent possible credential misuse. You can do this by navigating to the *Settings* button in the Manage Software Components app. This is not applicable for SAP-managed repositories (non-BYOG).

Procedure

Follow the steps below to create and manage a BYOG (Bring Your Own Git) repository in the Manage Software Components app.

Creating a remote repository

To create a BYOG repository, first create a new repository in your preferred Git provider.

Make sure to create a branch as well, as a default branch is necessary for the clone later on.

Next, copy the HTTPS link (must end with `.git`) to clone the repository in the Manage Software Components app.

Create a Software Component and link the remote repository

In the Manage Software Components app, create a new software component and enter the URL from the previous step in the *Repository URL* field.

> ### Note
>
> When specifying the repository URL, please note that it must be case-insensitive and unique. This prevents multiple software components from being linked to a single or identical Git repository.

Select the correct Git Provider for your repository. The currently supported providers are GitHub, GitLab, Azure and Bitbucket. Choose **'Git (others)'** if none of the other options apply. This selection is important for the correct repository configuration and cannot be changed at a later stage.

Add a name for your software component and choose the component type as development. Afterwards, click on *Create*. The software component is linked to the remote repository and is now ready to be cloned into the system.

Cloning the Software Component

To clone the software component, click on the *Clone* button on the top right corner. A dialog will pop up:

Enter your user credentials that you would normally use to clone the repository, respectively a username and a password or token.

The option to use a token without a user name in the *Repository Credentials* dialog is only available for cloning a software component in Github.

> ### Note
>
> Remember to enter the credentials of a user that has administrative rights to the repository and has authorizations to push and pull changes directly into the repository.

Select the radio button which describes your method of authentication, either a password or an OAuth token. Then click on *Next*. If the credentials are correct, the next step *Branch Selection*, will be displayed. Otherwise, re-enter the correct credentials before you move forward.

From the dropdown select the name of the branch you want to clone. Click on *Next*.

Select the repository role and import options. Choose *Latest* to import the latest commit.

Click on *Clone*. When the clone is finished, the status should be green.

After the clone is finished, notice that there will be a new commit *Modify repository layout* generated in the remote repository by commit author `noreply@example.com` and a newly added file. This file is necessary for pushing and pulling changes into the SAP system.

> ### Caution
>
> Do not edit or delete this file at any given time.

Creating Branches

To create a branch, you should do so using the native interface (e.g., Git interface) of your git provider linked to your repository.

In the Manage Software Components app simply refresh the branch table to see the new branch. You can proceed as usual with the checkout.

> **Note**
>
> To be able to work with an alternative branch, you should create the branch after the initial clone. It is important that any new branches are created based on the *Modify repository layout* commit, as mentioned above. The file generated from this commit will allow import into the local system instance.

Deleting a Software Component

Deleting a software component in the Manage Software Components app with the default settings will only delete the local representation of the software component in the ABAP system instance. To delete the component locally, simply proceed by clicking *Delete*. Releasing transport requests will no longer be possible. Selecting the *Unregister* option will unlink the remote repository from the software component and delete the software component completely from your global account. Unlinking the repository from the software component should only be done in the cases where the software component is no longer needed. The repository cannot be used again or linked anew with another software component. This action is irreversible, therefore ensure to first delete the software component on all system instances where it is cloned. Your remote Git repository will not be deleted in any case.

Creating and deleting tags

Tagging functionality is the same as in classic non-BYOG software components. To create a tag, go to the commit view of your selected branch. The created tag will also be visible on your remote repository. Deleting a tag will also delete the tag in the remote repository.

Related Information

Development in the ABAP Environment

6.3.1.22.3 API for Managing Software Components

You can use the `MANAGE_SOFTWARE_COMPONENTS` API to work with software components on SAP Business AI Platform, ABAP environment systems. The `MANAGE_SOFTWARE_COMPONENTS` API belongs to the communication scenario `SAP_COM_0948` and is an integral part of the partner add-on build pipeline.

6.3.1.22.3.1 Cloning Software Components to an ABAP Environment System

- You have an ABAP environment system.
- You have created a communication user as described in How to Create Communication Users.
- You've created a communication system as described in How to Create Communication Systems.
- You've created a communication arrangement as described in How to Create a Communication Arrangement.

- You have selected the communication scenario SAP_COM_0948 for your communication arrangement and have mapped it to your communication system.

If a software component is not in your system yet, you will first need to clone it once to import it into your system. Please note that you can clone a software component only once into your system.

1. *Authentication on the server*
   The first step serves the authentication on the server. The response header contains an `x-csrf-token`, which is used as authentication for the `POST` request following in step 2.

   Sample Code

   ```
   Request
   GET /sap/opu/odata4/sap/a4c_mswc_api/srvd_a2x/sap/
   manage_software_components/0001 HTTP/1.1
   Accept: application/json
   X-Csrf-Token: fetch
   Host: host.sap
   Response
   HTTP/1.1 200 OK
   X-csrf-token: xCsrfToken
                          that was retrieved in the first request in the
   header parameters. The
                          software component is passed in the path of the
   request URL. The branch you
                          want to clone is passed in the body of the
   request. If you don’t enter a
                          branch name, the main branch is automatically
   selected. Alternatively, you
                          can clone a specific tag or commit ID by entering
   the tag name or commit ID.
                          Make sure that the tag or commit ID exists on the
   branch. If that's not the
                          case, no clone action will be executed.
   ```

2. *Cloning a Software Component*
   To trigger the clone, you need to insert the `x-csrf-token`

   Sample Code

   ```
   Request
       POST /sap/opu/odata4/sap/a4c_mswc_api/srvd_a2x/sap/
   manage_software_components/0001/SoftwareComponents/%2FDMO%2FCOMPONENT/
   SAP__self.clone HTTP/1.1
   Accept: application/json
   X-Csrf-Token: {{xcsrfToken}}
   Host: host.abapcp.int.sap
   Content-Length: 172
       {
           "commit_id": "",
           "branch_name": "main",
           "tag_name": "",
           "username": "myUsername",
           "password": "myToken",
           "auth_method": "token"
       }
   Response
       HTTP/1.1 200 OK
       Content-Type: application/json
       {
           "@odata.context": "../$metadata#Actions/$entity",
           "@odata.metadataEtag": "W/\"20240111143934\"",
   ```

```
        "uuid": "96bbf3e1-5202-1ede-ad8e-b46db5437410",
        "sc_name": "/DMO/COMPONENT",
        "import_type": "Clone",
        "branch_name": "main",
        "namespace": "",
        "status": "Q",
        "status_descr": "Scheduled",
        "criticality": 0,
        "user_name": "WebAPI - CC0000000009",
        "commit_id": "",
        "tag_name": "",
        "relative_date_change_time": "",
        "change_time": "2024-01-16T13:46:26Z",
        "start_time": "2024-01-16T13:46:26Z",
        "execution_mode": "",
        "import_mode": "",
        "batch_id": null,
        "batch_sequence": 0,
        "SAP__Messages": []
    }
```

- *2.1 Cloning Your Own Software Component from Git Repository Using Bring Your Own Git (BYOG)*
  Determine whether you have created a Bring Your Own Git (BYOG) software component as described in the according documentation.
  If the software component is to be cloned from an externally managed Git repository, the clone request must be adjusted. The request must be supplemented with the parameters `username`, `password` and `auth_method`. These parameters are used to authenticate with external Git providers.
  Two authentication options, which are specified with the `auth_method` parameter, are supported. Following, these two authentication options are listed.

auth_method	Meaning	Usage
basic	Authentification with username and password	Sample Code `{ "username": "myUsername", "password": "myPassword", "auth_method": "basic" }`

auth_method	Meaning	Usage
token	Authentification with username and token	Sample Code `{ "username": "myUsername", "password": "myToken", "auth_method": "token" }`

If `auth_method` is not defined, the token method is automatically selected.

Sample Code

```
Request
    POST /sap/opu/odata4/sap/a4c_mswc_api/srvd_a2x/sap/
manage_software_components/0001/SoftwareComponents/%2FDMO%2FCOMPONENT/
SAP__self.clone HTTP/1.1
Accept: application/json
X-Csrf-Token: {{xcsrfToken}}
Host: host.abapcp.int.sap
Content-Length: 172
    {
        "commit_id": "",
        "branch_name": "main",
        "tag_name": "",
        "username": "myUsername",
        "password": "myToken",
        "auth_method": "token"
    }
Response
    HTTP/1.1 200 OK
    Content-Type: application/json
    {
        "@odata.context": "../$metadata#Actions/$entity",
        "@odata.metadataEtag": "W/\"20240111143934\"",
        "uuid": "96bbf3e1-5202-1ede-ad8e-b46db5437410",
        "sc_name": "/DMO/COMPONENT",
        "import_type": "Clone",
        "branch_name": "main",
        "namespace": "",
        "status": "Q",
        "status_descr": "Scheduled",
        "criticality": 0,
        "user_name": "WebAPI - CC0000000009",
        "commit_id": "",
        "tag_name": "",
        "relative_date_change_time": "",
        "change_time": "2024-01-16T13:46:26Z",
        "start_time": "2024-01-16T13:46:26Z",
        "execution_mode": "",
        "import_mode": "",
        "batch_id": null,
        "batch_sequence": 0,
        "SAP__Messages": []
```

```
    }
```

3. *Tracking the Status of the Action*
   To track the status of the action, you can do a GET request using the UUID contained in the response.

   Sample Code

   ```
   Request
       GET /sap/opu/odata4/sap/a4c_mswc_api/
   srvd_a2x/sap/manage_software_components/0001/Actions/96bbf3e1-5202-1ede-
   ad8e-b46db5437410 HTTP/1.1
       Accept: application/json
       Host: host.abapcp.int.sap
   ```

   The response contains the same entity as the second request. The action is successful, when the *status* in the response body has the value *S*. The status description *status_descr* will then return *Successful* . In case of an error *status* will have the value *E* and *status_descr* the value *Error*.

6.3.1.22.3.2 Pulling Software Components to an ABAP Environment System

- You have an ABAP environment system.
- You've created a communication user as described in How to Create Communication Users.
- You've created a communication system as described in How to Create Communication Systems.
- You've created a communication arrangement as described in How to Create a Communication Arrangement.
- You have selected the communication scenario SAP_COM_0948 for your communication arrangement and have mapped it to your communication system.
- You have cloned the software component to the ABAP Environment system. See Cloning Software Components to an ABAP Environment System.

You can use the communication scenario SAP_COM_0948 to pull software components to an ABAP environment system.

1. *Authentication to the server*
   The first step serves the authentication on the server. The response header contains an `x-csrf-token`, which is used as authentication for the `POST` request following in step 2.

   Sample Code

   ```
   Request
   GET /sap/opu/odata4/sap/a4c_mswc_api/srvd_a2x/sap/
   manage_software_components/0001 HTTP/1.1
   Accept: application/json
   X-Csrf-Token: fetch
   Host: host.sap
   Response
   HTTP/1.1 200 OK
   X-csrf-token: xCsrfToken
   ```

2. *Pull a Software Component*
   To trigger the pull of a software component, you need to insert the `x-csrf-token` that was retrieved in the first request in the header parameters. The software component you want to pull is passed in path of the request URL.

   Sample Code

```
Request
    POST /sap/opu/odata4/sap/a4c_mswc_api/srvd_a2x/sap/
manage_software_components/0001/SoftwareComponents/%2FDMO%2FCOMPONENT/
SAP__self.pull HTTP/1.1
    Accept: application/json
    X-Csrf-Token: {{xcsrfToken}}
    Host: host.abapcp.int.sap
    Content-Length: 39
    {
    "commit_id": "",
    "tag_name": ""
    }
Response
 HTTP/1.1 200 OK
 Content-Type: application/json
    {
    "@odata.context": "../$metadata#Actions/$entity",
    "@odata.metadataEtag": "W/\"20240111143934\"",
    "uuid": "763eca2a-340b-1eee-ad8e-c864d7a77c38",
    "sc_name": "/DMO/COMPONENT",
    "import_type": "Pull",
    "branch_name": "master",
    "namespace": "",
    "status": "Q",
    "status_descr": "Scheduled",
    "criticality": 0,
    "user_name": "WebAPI - CC0000000009",
    "commit_id": "",
    "tag_name": "",
    "relative_date_change_time": "",
    "change_time": "2024-01-16T13:50:54Z",
    "start_time": "2024-01-16T13:50:54Z",
    "execution_mode": "",
    "import_mode": "",
    "SAP__Messages": []
    }
```

3. *Tracking the Status of the Action*
   To track the status of the action, you can do a GET request using the UUID contained in the response.

   Sample Code

```
Request
    GET /sap/opu/odata4/sap/a4c_mswc_api/
srvd_a2x/sap/manage_software_components/0001/Actions/96bbf3e1-5202-1ede-
ad8e-b46db5437410 HTTP/1.1
    Accept: application/json
    Host: host.abapcp.int.sap
```

   The response contains the same entity as the second request. The action is successful, when the *status* in the response body has the value *S*. The status description *status_descr* will then return *Successful* . In case of an error *status* will have the value *E* and *status_descr* the value *Error*.

6.3.1.22.3.3 Working with Branches on a Software Component

- You have an ABAP environment system.
- You've created a communication user as described in How to Create Communication Users.
- You've created a communication system as described in How to Create Communication Systems.
- You've created a communication arrangement as described in How to Create a Communication Arrangement.
- You have selected the communication scenario `SAP_COM_0948` for your communication arrangement and have mapped it to your communication system.
- You have cloned the software component to the ABAP environment system. See Cloning Software Components to an ABAP Environment System [page 3252].

You can use the communication scenario `SAP_COM_0948` to work with software components on a SAP Business AI Platform, ABAP environment system.

1. *Authentification on the server*
   The first step serves the authentication on the server. The response header contains an `x-csrf-token`, which is used as authentication for the POST request following in step 2 and 3.

   Sample Code

   ```
   Request
       GET /sap/opu/odata4/sap/a4c_mswc_api/srvd_a2x/sap/
   manage_software_components/0001 HTTP/1.1
       Accept: application/json
       X-Csrf-Token: fetch
       Host: host.sap
   Response
       HTTP/1.1 200 OK
       X-csrf-token: xCsrfToken
   ```

2. *Create a Branch*
   To create a branch, you perform a `POST` request on the `Branches` entity. In the header parameters, you should include the `x-csrf-token` that was retrieved in the first request. Fill in the following data in your request body:
   1. `sc_name`: the name of your software component
   2. `branch_name`: the name of your new branch
   3. `derived_from`: the branch that your new branch should derive from

   Sample Code

   ```
   Request
       POST /sap/opu/odata4/sap/a4c_mswc_api/srvd_a2x/sap/
   manage_software_components/0001/Branches HTTP/1.1
       Accept: application/json
       X-Csrf-Token: {{xcsrfToken}}
       Host: host.abapcp.int.sap
       Content-Length: 103
       {
           "sc_name" : "/DMO/COMPONENT",
           "branch_name" : "feature",
           "derived_from" : "main"
   ```

```
    }
Response
    HTTP/1.1 201 Created
    Content-Type: application/json
    {
        "@odata.context": "$metadata#Branches/$entity",
        "@odata.metadataEtag": "W/\"20240111143934\"",
        "sc_name": "/DMO/COMPONENT",
        "branch_name": "feature",
        "is_active": false,
        "criticality": 0,
        "delta": "Please checkout first",
        "delta_criticality": 0,
        "derived_from": "main",
        "name_of_new_branch": "",
        "commit_id": "",
        "commit_message": "",
        "last_commit_by": "",
        "relative_date_last_rem_commit": "",
        "last_commit_on": null,
        "commit_id_on_system": "-",
        "created_by": "CC0000000009",
        "created_on": "2024-01-16T14:00:00Z",
        "execution_mode": "",
        "import_mode": "",
        "SAP__Messages": []
    }
```

3. *Check Out a Branch*

   Simply creating a branch did not change the state of the system. In order to work with the newly created branch, you need to import the branch to the system. This action is called *checkout branch*. You need to include the software component and the name of the branch in the path of the request URL.

   {·} Sample Code

```
Request
      POST /sap/opu/odata4/sap/a4c_mswc_api/
srvd_a2x/sap/manage_software_components/0001/Branches/%2FDMO%2FCOMPONENT/
main/SAP__self.checkout_branch HTTP/1.1
        Accept: application/json
        X-Csrf-Token: {{xcsrfToken}}
        Host: host.abapcp.int.sap
        Content-Length: 44
        {
            "import_mode" : "",
            "execution_mode": ""
        }
Response
      HTTP/1.1 200 OK
      Content-Type: application/json
      {
        "@odata.context": "../
$metadata#com.sap.gateway.srvd_a2x.a4c_a2g_mswc_api.v0001.ActionsType",
        "@odata.metadataEtag": "W/\"20240111143934\"",
        "uuid": "96bbf3e1-5202-1ede-ad8f-001e560df4e6",
        "sc_name": "/DMO/FCOMPONENT",
        "import_type": "Checkout",
        "branch_name": "main",
        "namespace": "",
        "status": "Q",
        "status_descr": "Scheduled",
        "criticality": 0,
        "user_name": "WebAPI - CC0000000009",
        "commit_id": "",
        "tag_name": "",
        "relative_date_change_time": "",
```

```
        "change_time": "2024-01-16T14:03:22Z",
        "start_time": "2024-01-16T14:03:21Z",
        "execution_mode": "",
        "import_mode": "",
        "SAP__Messages": []
      }
```

4. *Tracking the Status of the Action*
   To track the status of the action, you can do a `GET` request using the UUID contained in the response.

   Sample Code

   ```
   Request
       GET /sap/opu/odata4/sap/a4c_mswc_api/
   srvd_a2x/sap/manage_software_components/0001/Actions/96bbf3e1-5202-1ede-
   ad8e-b46db5437410 HTTP/1.1
       Accept: application/json
       Host: host.abapcp.int.sap
   ```

   The response contains the same entity as the second request. The action is successful, when the *status* in the response body has the value *S*. The status description *status_descr* will then return *Successful* . In case of an error *status* will have the value *E* and *status_descr* the value *Error*.

6.3.1.23 User Interface Monitoring

Display Launchpad Content Exposure Logs [page 3260]

6.3.1.23.1 Display Launchpad Content Exposure Logs

With this app, you can check the application log created when publishing launchpad content to SAP BTP. You can check, for example, when the publishing run started, if the publishing was successful or if there were any issues.

> **Note**
>
> Log entries about apps that are not exposed and have a severity level of *Information* do not indicate an error. Applications may not be exposed for various reasons. For example, inactive applications may not be exposed in either version of exposure, or apps without a tile may not be displayed in exposure version 2.

Implementation Information

To use the app, you need the following:

- Business catalog: SAP_CORE_BC_UI_MON

- Business role: SAP_BR_ADMINISTRATOR

To get the key information, including all the technical data you need for the installation and configuration, go to the SAP Fiori apps reference library.

Related Information

Create Communication Arrangement

6.3.1.24 Manage Security Audit Log

With this app, you can manage Security Audit Log related parameters.

Key Features

You can use this app to:

- Each parameter can be active or inactive, so that changing the parameter value is possible without having the change immediately active
- Maintain parameter `DELETE`:
  - Maintain the default audit log retention time in days. After the number of days the Security Audit Log Events will be removed from database to reduce the consumption of the SAP HANA DB storage. The default retention period is 45 days. Confirm the default by activating the parameter `DELETE`, or define a custom retention period.

Mark the parameter you want to change and switch to *Edit* mode. Select the option for *Parameter Status* (active or inactive) and for *Parameter Value*. Save your settings.

Access Information

For information on what business role and business catalog need to be assigned to your user to access the app, see the overview table in the chapter Applications for General Functions for Key Users .

Supported Device Types

- Desktop
- Tablet

- Smartphone

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SEC-SAL`.

Related Information

Manage Security Audit Log (Deprecated) [page 3262]

6.3.1.25 Manage Security Audit Log (Deprecated)

With this app, you can manage Security Audit Log related parameters.

Key Features

You can use this app to:

- Each parameter can be active or inactive, so that changing the parameter value is possible without having the change immediately active
- Maintain parameter `DELETE`:
  - Maintain the default audit log retention time in days. After the number of days the Security Audit Log Events will be removed from database to reduce the consumption of the SAP HANA DB storage. The default retention period is 45 days. Confirm the default by activating the parameter `DELETE`, or define a custom retention period.

Mark the parameter you want to change and switch to *Edit* mode. Select the option for *Parameter Status* (active or inactive) and for *Parameter Value* (Keep, Replace, or Clear Value) and save your settings.

Access Information

For information on what business role and business catalog need to be assigned to your user to access the app, see the overview table in the chapter Applications for General Functions for Key Users.

Supported Device Types

- Desktop
- Tablet
- Smartphone

Component for Customer Incidents

If you need support or experience issues, please report an incident under component `BC-SEC-SAL`.

6.3.1.26 Business Event Logging

Business Event Logging enables you to log business events that are raised by activities of SAP S/4HANA applications or business users. Business events are messages that are sent to notify a consumer that an object has changed.

The events triggered during the execution of business activities can be:

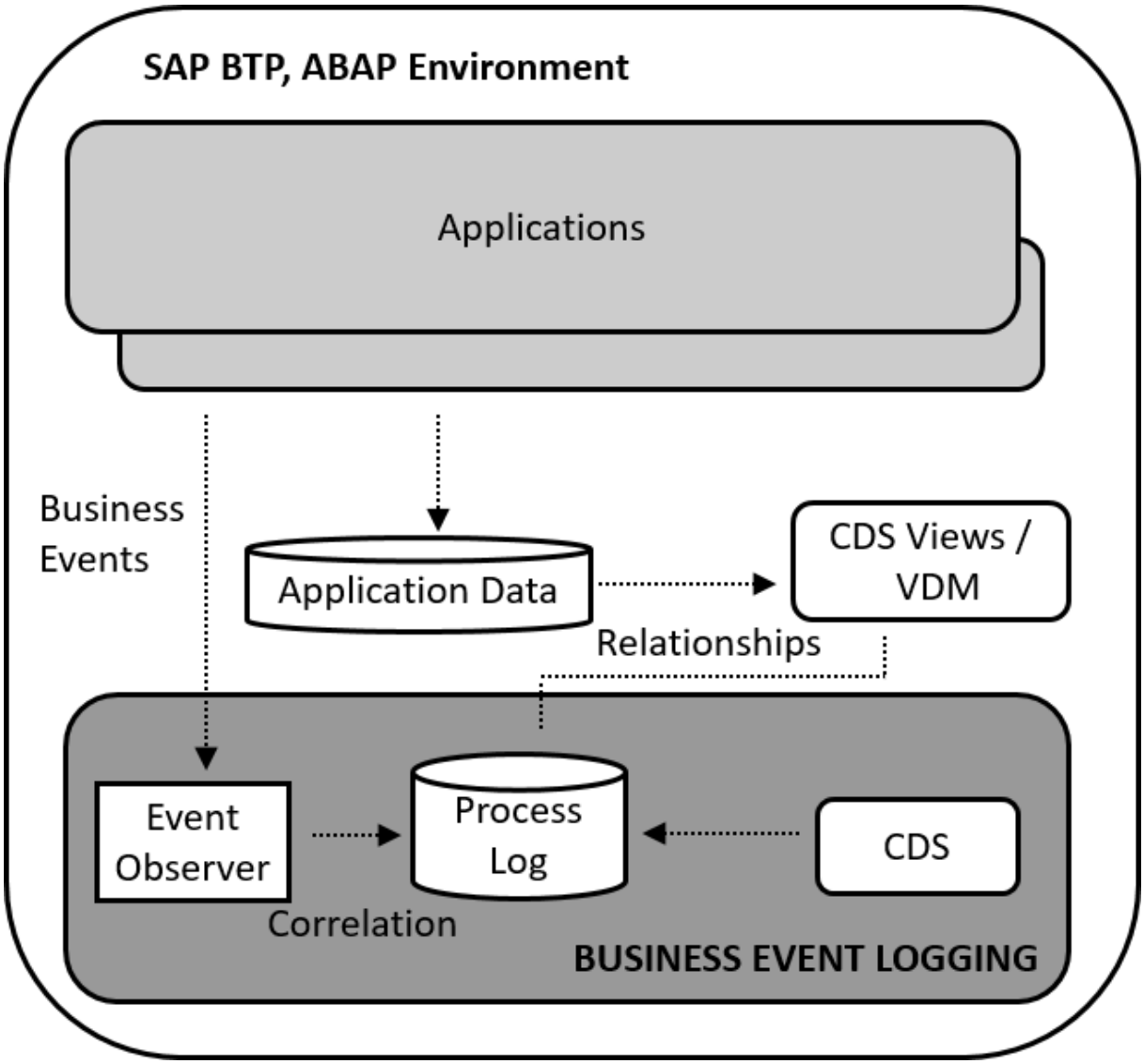
- Business events for remote consumption: C2-released events (external)
- Business events for local consumption: C1-released events (internal)

For example, you can find events such as **created**, **changed**, and **deleted** for Sales Order. The RAP-based events are listed in the SAP Business Accelerator Hub.

You can activate Business Event Logging using configuration activities for each object, such as Sales Order, Outbound Delivery, and so on.

The business event logs give you insights into the way processes run in the local system and how objects change. You can get an overview of all the logged business events, the number of events triggered, the types of events triggered, and information on changes to fields. Events of the same type occurring in the same session are grouped into activities.

This graphic displays the components of Business Event Logging:



Benefits

With Business Event Logging, you can collect records of these business events, irrespective of whether the consumer has subscribed to the event, to get insights into the way business processes run in the local system. You can either store the default information or the complete business event data, depending on your configuration settings.

Essential properties of an event are stored by default. These properties are:

- Objects
- Object components
- Business object key

- Date of event
- Event type (corresponding to performed action)
- User who performed the action
- Relevant field changes
- Qualifiers (for generic events)

Field changes are also recorded as long as they are defined in the business events.

6.3.1.26.1 Business Event Logging Configuration

Prerequisites

To configure business event logging, create a business role and assign the catalog `SAP_CA_BC_IC_LND_BEL_BCO_PC` to the role. For instructions, refer to How to Create a New Business Role [page 3070]. You can also create a business role using the template `SAP_BR_BPC_EXPERT`. For instructions, refer to How to Create a Business Role from a Template [page 3072] .

Configure the Application

Activate Business Event Logging

To activate logging for your objects:

1. Open the *Custom Business Configurations* app. You'll see a list of all business configurations that can be adapted.
2. Search for *Activate Business Event Logging* using the search bar.
3. Select the custom business configuration from the list and choose *Edit*.
4. To create a new entry, choose *New Entries*.
5. Enter the *Object* and select the *Activate Logging* checkbox to activate logging for the object.

   > ⓘ Note
   >
   > When you activate logging for an object, the keys and metadata such as event type, performed by, performed at, and other details of the object are logged and displayed in the business event logging apps. When you enable logging for all business event data, all changes made to the object is logged by the system. The fields captured by the system depend on how the event has been defined. For more information on the fields that get logged, refer to the SAP Business Accelerator Hub.

6. Choose *Save*.

For documentation about this item, refer to *Show Documentation*.

6.3.1.26.2 Business Event Logging Applications

With these apps, you can view business event logs and business events by object instances in your local system.

Key Features

Business Event Logging contains these features:

- Displaying Business Event Logs
- Displaying Business Events by Objects
- Display Changes to Objects

Components

All users can use the following applications in the ABAP environment for Business Event Logging:

- `Display Business Event Logs` app to view all the events logged in the SAP S/4HANA system.
- `Display Business Events by Objects` app to view all the sequences of business events in the context of the instances of object components.
- `Display Changes to Objects` app to view all changes for an object.

Prerequisites

To access applications, create a business role and assign business catalog **SAP_CA_BC_BEL_PC** to the role. For instructions, refer here.

To create a business role using a template, select the **SAP_BR_BUSINESS_PROCESS_SPEC** template. For instructions, refer here.

6.3.1.26.2.1 Display Business Event Logs

App ID: F6021

With this app, you can display logged business events. The events can be filtered by business event type for each object.

Key Features

You can use this app to:

- Get an overview of logged business events
- View details of logged business events
- Go to business event sequence for an instance

Supported Device Types

- Desktop
- Tablet
- Smartphone

6.3.1.26.2.2 Display Business Events by Objects

App ID: F6035

With this app, you can display object instances with their events grouped by activities. You can filter objects by processing variants. The detailed view provides information about the sequences of activities for instances of object components.

Key Features

You can use this app to view:

- Instances of object components containing business events
- Sequence of activities logged for an instance of object component. The sequence corresponds to a processing variant.
- Number of events for each instance of object component, and the processing cycle time

Supported Device Types

- Desktop
- Tablet
- Smartphone

6.3.1.26.2.3 Display Changes to Objects

App ID: F8211

With this app, you can view the recorded field changes for an object.

Key Features

You can use this app to view:

- The field changes that are recorded for an object
- Old and new values for a changed field

Supported Device Types

- Desktop
- Tablet
- Smartphone

6.3.1.26.3 Monitoring for Business Event Logging

You can monitor the problems that occur in business event logging using:

- Message Monitoring: You can use Message Monitoring Overview app to view runtime errors, to monitor processing of events, and to view missing events.
- Application Log: You can use Application Log to view additional information such as database errors, deletions, and other information related to business event logs.

6.3.1.26.3.1 Message Monitoring

You monitor the problems that occur in business event logging by using Application Interface Framework (AIF) . You can check the error status to identify failures. If there are failures in the queue, the business events aren't logged. However, the events aren't lost.

Prerequisites

To access Message Monitoring, create a business role and assign the role to a catalog. For more information about catalogs and other settings, refer to Message Monitoring [page 3120]. Use the BEL_IBD_QUEUE_MONITOR recipient to view Business Event Logging messages.

Message Monitoring

1. Log on to SAP Fiori launchpad.
2. Open the *Message Monitoring Overview* app. For more information, refer to Message Monitoring Overview.
3. Select *From* (Date/Time) and *To* (Date/Time) values to limit the time range for the monitoring records.

   > **Note**
   >
   > The default time range is "last 2 weeks". If you want a custom time range, you can also select the values in the *From* and *To* fields.

4. Choose *Go* to view the messages in this time range.
5. Select the following cards to monitor the processing of business events for the business event log:
   - Namespace: /BEL
   - Interface:
     - BEL_EY_TMP: Cleaning temporary tables of nodes
     - BEL_HDR_E: Processing empty header nodes
     - BEL_LG_MGR: Monitoring log manager
     - BEL_REL_IB: Monitoring association processing
     - BEL_REL_L1: BEL relationship processing
     - BEL_REL_L2: BEL relationship processing
     - BEL_TMP_CL: Cleaning up temporary nodes
     - BEL_REL_Q: Monitoring relationship inserts
     - BEL_VAR_DL: Deletion of unused variants
     - BEL_VAR_IN: Monitoring variant insert

   If you want to view only the error or success messages, choose *Error* or *Success*.
6. On the *Message Monitoring* screen, in the *Log Messages* column, you can see all business event logging messages for your selected time range. All types of messages, such as success, error, warning, canceled, and in process are included.
7. You can filter by *Status*.
8. Choose *Message Details* to see all the message logs.

If a queue fails, the user can restart the queue manually with the AIF user interface. If errors occur during restart, create an incident under component **CA-GTF-BEL**.

6.3.1.26.4 CDS Views for Business Event Logging

6.3.1.26.4.1 Key User Extensibility

6.3.1.26.4.1.1 Business Event Header Data (v2)

CDS View Name	`C_BUSEVTLOGEVENTDEX_2`
Analytical Data Category	@Analytics.dataCategory: #FACT
Represented Objects	This view represents the following objects: • `BusEvtLogEvent` (TechnicalObject)

Purpose

This CDS view is used to extract business event header data for C2-released “external” events, and direct APIs.

This CDS view provides the data to answer the following business questions:

- Which business event has been logged?

To help you decide which CDS view to use for your purposes, SAP has introduced the annotation `ObjectModel.supportedCapabilities` that indicates the most appropriate use cases for each CDS view. To find out what use cases are best supported by this CDS view, access the entry of the CDS view in the *View Browser* app and find the values for this annotation under the *Annotation* tab.

Structure

Important Fields

Important fields in this view include the following:

Field Name	Description
`EventOperation`	
`SAPObjectType`	The object type, a business object, or a technical object corresponding to the specified object component, for example: Outbound Delivery for Outbound Delivery Item.
`SAPObjectNodeType`	Object component, for example, for an Outbound Delivery object, the object components are Outbound Delivery (header) and Outbound Delivery Item (item).

Field Name	Description
SAPBusinessObjectNodeKey1	First part of the key of an object instance, for example, in one event for an Outbound Delivery Item, the first part of the key is 80013910 (outbound delivery number).
SAPBusinessObjectNodeKey2	Second part of the key of an object instance, for example, in one event for an Outbound Delivery Item, the second part of the key is 00020 (item number). If an event refers to an object component with only one key part (such as: Outbound Delivery [header]), then all subsequent key parts are empty. **Note** There can be many more object ID, depending on how many parts the object key has. For example: SAPBusinessObjectNodeKey3, SAPBusinessObjectNodeKey4, and so on. SAPObjectNodeType and SAPBusinessObjectNodeKey (1-7) identify an instance of an object. Outbound Delivery Item is an example of an instance.
BusEvtLogCreationDateTime	Event execution timestamp
CreatedByUser	Users who created the event
EventProducerNamespace	Provides the event's namespace. This can either be a Standard or Custom

Examples

An example of events include:

BusinessPartner

- Business partner created
- Business partner changed

For more information on other events, refer to the SAP Business Accelerator Hub.

6.3.1.26.4.1.2 Business Event Header Data (v3)

CDS View Name	C_BUSEVTLOGEVENTDEX_3
Analytical Data Category	Fact

Purpose

This CDS view is used to extract business event header data for C2-released external events, C1-released internal events, and direct APIs.

This CDS view provides the data to answer the following business questions:

- Which business event has been logged?

To help you decide which CDS view to use for your purposes, SAP has introduced the annotation `ObjectModel.supportedCapabilities` that indicates the most appropriate use cases for each CDS view. To find out what use cases are best supported by this CDS view, access the entry of the CDS view in the *View Browser* app and find the values for this annotation under the *Annotation* tab. For more information, see Supported Capabilities for CDS Views.

Structure

Important Fields

Important fields in this view include the following:

Field Name	Description
`EventOperation`	Event operation, for example: Created, Deleted, Changed, and so on.
`SAPObjectType`	The object type, a business object, or a technical object corresponding to the specified SAP object node type, for example: Outbound Delivery for Outbound Delivery Item.
`SAPObjectNodeType`	Node type of an object type, for example, for an Outbound Delivery SAP object type, the SAP object node types are Outbound Delivery (header) and Outbound Delivery Item (item).
`SAPBusinessObjectNodeKey1`	First part of the key of an object node instance, for example, in one event for an Outbound Delivery Item, the first part of the key is 80013910 (outbound delivery number).

Field Name	Description
SAPBusinessObjectNodeKey2	Second part of the key of an object node instance, for example, in one event for an Outbound Delivery Item, the second part of the key is 00020 (item number). If an event refers to an SAP object node type with only one key part (such as: Outbound Delivery [header]), then all subsequent key parts are empty. **Note** There can be many more node keys, depending on how many parts the object key has. For example: SAPBusinessObjectNodeKey3, SAPBusinessObjectNodeKey4, and so on. SAPObjectNodeType and SAPBusinessObjectNodeKey (1-7) identify a node instance of an SAP object. Outbound Delivery Item is an example of an SAP object node.
BusEvtLogCreationDateTime	Event execution timestamp
CreatedByUser	Users who created the event
EventProducerNamespace	Provides the event's namespace. This can either be a Standard or Custom
BusEvtLogBusinessActivity	Activity ID
BusinessEventLogLogicalSystem	Logical system
IsTechnicalUser	Is a technical user
BusEvtLogUserInteractionType	Application type
BusEvtLogIsDeleted	Deletion indicator
BusinessEventLogEventVersion	Business event version
BusEvtLogTransactionID	Identifies the session in which a business event was created
BusinessEventLogSource	Business event source
BusEvtLogEventHasFullPayload	Has all business event data

Data Extraction

Data Extraction Type

- Full
- Delta (change data capture)

> **Note**
>
> Data conversion using the Cloud Data Integration (CDI) channel can be different from that of the other channels (ODATA, ABAP SQL Service, and so on). For example:
>
> - The timestamps could appear in a different format
> - The IDs might not contain leading zeros
>
> There could be formatting changes for other data as well.

Related Information

Virtual Data Model and CDS Views in SAP S/4HANA Cloud Public Edition

6.3.1.26.4.1.3 Business Event Property Changes (v2)

CDS View Name	`C_BUSEVTLOGPAYLOADDEX_2`
Analytical Data Category	@Analytics.dataCategory: #FACT
Represented Objects	This view represents the following object: • `BusEvtLogEvent` (TechnicalObject)

Purpose

This CDS view provides a subset of event payload details.

This CDS view provides the data to answer the following business questions:

- What are the event qualifiers?
- What are the field changes?

To help you decide which CDS view to use for your purposes, SAP has introduced the annotation `ObjectModel.supportedCapabilities` that indicates the most appropriate use cases for each CDS view. To find out what use cases are best supported by this CDS view, access the entry of the CDS view in the *View Browser* app and find the values for this annotation under the *Annotation* tab.

Structure

Important Fields

Important fields in this view include the following:

Field Name	Description
`BusinessEventUUID`	Unique identifier for business events.
`SAPObjectType`	The object type, a business object, or a technical object corresponding to the specified object component. For example: Outbound Delivery for Outbound Delivery Item.
`BusEvtLogGlobalFieldName`	Field of an object. Example: Amount
`BusEvtLogFieldHasOldValue`	This is a Boolean flag that indicates whether there was a value for the field.
`BusEvtLogFieldIsQualifier`	This is a Boolean flag that indicates whether the field is an event qualifier. A qualifier is an annotated field in a generic event that defines the event in more detail. The value of the qualifier field describes a generic event in detail. Example: For the event Outb Deliv Goods Issue Status Changed the qualified field or Qualifier is OverallGoodsMovementStatus, with the values: Not yet started, Partially Completed, Completed, or Not Relevant
`BusEvtLogLastChangedDateTime`	The date and timestamp of when the payload entry was last updated.

6.3.1.26.4.1.4 Business Event Data

CDS View Name	`C_BuEvLgEvtFullPayloadJSONDEX`
Analytical Data Category	@Analytics.dataCategory: #FACT
Represented Objects	This view represents the following object: • `BusEvtLogEvent` (TechnicalObject)

Purpose

This CDS view provides full business event data (payload) in JSON format.

This CDS view provides the data to answer the following business questions:

- What is the full business data contained in the event?

To help you decide which CDS view to use for your purposes, SAP has introduced the annotation `ObjectModel.supportedCapabilities` that indicates the most appropriate use cases for each CDS view. To find out what use cases are best supported by this CDS view, access the entry of the CDS view in the *View Browser* app and find the values for this annotation under the *Annotation* tab.

Structure

Important Fields

Important fields in this view include the following:

Field Name	Description
`SAPObjectType`	The object type, a business object, or a technical object corresponding to the specified object component. For example: Outbound Delivery for Outbound Delivery Item.
`BusEvtLogPayloadJSONString`	Event Payload details in JSON format.
`BusEvtLogLastChangedDateTime`	Timestamp of when the event was changed.

6.3.1.26.4.1.5 Business Event Data v2

CDS View Name	`C_BuEvLgEvtFullPyldJSONDEX_2`
Analytical Data Category	Fact

Purpose

This CDS view provides full business event data (payload) in JSON format.

This CDS view provides the data to answer the following business questions:

- What is the full business data contained in the event?

To help you decide which CDS view to use for your purposes, SAP has introduced the annotation `ObjectModel.supportedCapabilities` that indicates the most appropriate use cases for each CDS view. To find out what use cases are best supported by this CDS view, access the entry of the CDS view in the *View Browser* app and find the values for this annotation under the *Annotation* tab. For more information, see Supported Capabilities for CDS Views.

Structure

Important Fields

Important fields in this view include the following:

Field Name	Description
SAPObjectType	The object type, a business object, or a technical object corresponding to the specified SAP object node type. For example: Outbound Delivery for Outbound Delivery Item.
BusEvtLogPayloadJSONString	Event Payload details in JSON format
BusEvtLogLastChangedDateTime	Timestamp of when the event was changed

Data Extraction

Data Extraction Type

- Full
- Delta (change data capture)

> ### Note
>
> Data conversion using the Cloud Data Integration (CDI) channel can be different from that of the other channels (ODATA, ABAP SQL Service, and so on). For example:
>
> - The timestamps could appear in a different format
> - The IDs might not contain leading zeros
>
> There could be formatting changes for other data as well.

Related Information

Virtual Data Model and CDS Views in SAP S/4HANA Cloud Public Edition

6.3.1.26.4.1.6 Business Event Activity

CDS View Name	C_BUSEVTLOGACTIVITYDEX
Analytical Data Category	Fact

Purpose

This CDS view retrieves activities.

This CDS view provides the data to answer the following business questions:

- What are the logged activities?

To help you decide which CDS view to use for your purposes, SAP has introduced the annotation `ObjectModel.supportedCapabilities` that indicates the most appropriate use cases for each CDS view. To find out what use cases are best supported by this CDS view, access the entry of the CDS view in the *View Browser* app and find the values for this annotation under the *Annotation* tab. For more information, see Supported Capabilities for CDS Views.

Structure

Important Fields

Important fields in this view include the following:

Field Name	Description
`BusEvtLogBusinessActivity`	Activity ID
`SAPObjectType`	RAP SOT: SAP Object Type
`SAPObjectNodeType`	RAP SOT: SAP Object Node Type
`EventOperation`	Business Event Operation
`SAPBusinessObjectNodeKey1` (1-8)	Object Key Part (1-8)
`CreatedByUser`	Users Who Executed the Action
`BusEvtLogCreationDateTime`	Action Execution Time Stamp
`BusinessEventLogLogicalSystem`	Logical system
`BusEvtLogTransactionID`	Identifies the session in which a business event was created
`IsTechnicalUser`	Is a Technical User
`BusEvtLogUserInteractionType`	Application Type
`BusEvtLgUsrIntactnTypeValue`	Application ID
`SAPBusinessObjectNodeKey1Name`(1-8)	Key Names

Data Extraction

This CDS view is enabled for data extraction in full mode as well as in delta mode. Deltas are determined automatically by change data capture.

Related Information

Virtual Data Model and CDS Views in SAP S/4HANA Cloud Public Edition

6.3.1.26.4.2 Developer Extensibility

6.3.1.26.4.2.1 Business Event Header Data

CDS View Name	`I_BusEvtLogBusinessEvent`

Purpose

This CDS view provides a structured representation of business event header data, capturing essential details about business events logged in the system. It serves as a composite view that aggregates information from detailed event logs, enabling users to analyze and track business events effectively.

This CDS view provides the data to answer the following business questions:

- When were specific business events created, and who created them?
- What operations were performed during these business events?
- What are the key attributes and names associated with the business object nodes involved in these events?
- What is the source and logical system associated with each business event?

Prerequisites

Authorizations

Users who want to use this CDS view must have the following authorization objects assigned:

- `BEL_SOT` (SAP Object Type)

Structure

Important Fields

Important fields in this view include the following:

Field Name	Description
BusinessEventUUID	Event GUID
SAPObjectType	The object type, a business object, or a technical object corresponding to the specified SAP object node type, for example: Outbound Delivery for Outbound Delivery Item..
BusEvtLogCreationDate	Action Executed Date
SAPObjectNodeType	Node type of an object type, for example, for an Outbound Delivery SAP object type, the SAP object node types are Outbound Delivery (header) and Outbound Delivery Item (item).
EventOperation	Business Event Operation. For example: Created, Deleted, Changed, and so on.
SAPBusinessObjectNodeKey1	First part of the key of an object node instance, for example, in one event for an Outbound Delivery Item, the first part of the key is 80013910 (outbound delivery number).
SAPBusinessObjectNodeKey2	Second part of the key of an object node instance, for example, in one event for an Outbound Delivery Item, the second part of the key is 00020 (item number). If an event refers to an SAP object node type with only one key part (such as: Outbound Delivery [header]), then all subsequent key parts are empty. **Note** There can be many more node keys, depending on how many parts the object key has. For example: SAPBusinessObjectNodeKey3, SAPBusinessObjectNodeKey4, and so on. SAPObjectNodeType and SAPBusinessObjectNodeKey (1-8) identify a node instance of an SAP object. Outbound Delivery Item is an example of an SAP object node.
CreatedByUser	Users Who Executed the Action
BusEvtLogCreationDateTime	Action Execution Time Stamp
BusinessEventLogLogicalSystem	Logical System
BusEvtLogIsDeleted	Deletion Indicator
BusinessEventLogEventVersion	Business Event Version
BusEvtLogTransactionID	Identifies the session in which a business event was created

Field Name	Description
BusEvtLogLastChangedDateTime	The date and timestamp of when the payload entry was last updated.
IsTechnicalUser	Is Technical User
BusinessEventLogSource	Business Event Source
EventProducerNamespace	Producer Event Namespace
BusEvtLogEventHasFullPayload	Has All Business Event Data
BusEvtLogUserInteractionType	Application Type
BusEvtLgUsrIntactnTypeValue	Application ID
ChangeDocument	Number of the Change Document
ChangeDocItemChangeType	Change Indicator of Change Document
SAPBusinessObjectNodeKey1Name	Key1 Name
SAPBusinessObjectNodeKey2Name	Key2 Name
SAPBusinessObjectNodeKey3Name	Key3 Name
SAPBusinessObjectNodeKey4Name	Key4 Name
SAPBusinessObjectNodeKey5Name	Key5 Name
SAPBusinessObjectNodeKey6Name	Key6 Name
SAPBusinessObjectNodeKey7Name	Key7 Name
SAPBusinessObjectNodeKey8Name	Key8 Name
BusEvtLogReferenceEventUUID	Event GUID
BusEvtLogBusinessActivity	Activity ID

6.3.1.26.4.2.2 Business Event Payload

CDS View Name	`I_BusEvtLogBusEventPayload`

Purpose

This CDS view is designed to capture and provide detailed information about changes in business event payloads. It tracks the old and new values of fields, along with their units and currencies, and records metadata about the changes, such as the last change date and the relevant database tables and fields.

This CDS view provides the data to answer the following business questions:

- Which fields in the business object have been modified, and what are their previous and current states?
- Which database tables and fields are associated with the changes in the business event payloads in case the change is related to change documents?

- Are there any fields qualifying the event semantics?

Prerequisites

Authorizations

Users who want to use this CDS view must have the following authorization objects assigned:

- `BEL_SOT` (SAP Object Type)

Structure

Important Fields

Important fields in this view include the following:

Field Name	Description
BusinessEventUUID	Unique identifier for business events.
SAPObjectType	The object type, a business object, or a technical object corresponding to the specified SAP object node type, for example: Outbound Delivery for Outbound Delivery Item.
BusEvtLogFieldName	Field of an SAP object. Example: Amount
BusEvtLogOldFieldValue	Previous Value of an Attribute
BusEvtLogOldFieldUnit	Previous Unit of Measure
BusEvtLogNewFieldValue	Updated Value of an Attribute
BusEvtLogNewFieldUnit	Updated Unit of Measure
BusEvtLogOldFieldCurrency	Previous Currency Key
BusEvtLogNewFieldCurrency	Updated Currency Key
BusEvtLogFieldHasOldValue	This is a Boolean flag that indicates whether there was a value for the field.
BusEvtLogFieldIsQualifier	This is a Boolean flag that indicates whether the field is an event qualifier. A qualifier is an annotated field in a generic event that defines the event in more detail. The value of the qualifier field describes a generic event in detail. Example: For the event Outb Deliv Goods Issue Status Changed the qualified field or Qualifier is OverallGoodsMovementStatus, with the values: Not yet started, Partially Completed, Completed, or Not Relevant
BusEvtLogLastChangedDateTime	The date and timestamp of when the payload entry was last updated.

Field Name	Description
BusEvtLogGlobalFieldName	Field of an SAP object. Example: Amount
BuEvLgGlobalFieldNameUpperCase	Property Name of the Object
ChangeDocumentDatabaseTable	Table Name
ChangeDocDatabaseTableField	Field Name

6.3.1.26.4.2.3 Business Event Data

CDS View Name	`I_BuEvLgBusEvtFullPyldJSON`

Purpose

This CDS view is designed to provide a comprehensive representation of business event data, including the full payload in JSON format. It serves as a transactional data source with high service quality and is capable of handling large volumes of data. The view is intended to facilitate the analysis and tracking of business events by capturing essential details such as the event's unique identifier, object type, payload content, and the last modification timestamp.

This CDS view provides the data to answer the following business questions:

- What is the complete business data associated with each business event?

Prerequisites

Authorizations

Users who want to use this CDS view must have the following authorization objects assigned:

- `BEL_SOT` (SAP Object Type)

Structure

Important Fields

Important fields in this view include the following:

Field Name	Description
BusinessEventUUID	Unique identifier for business events.

Field Name	Description
SAPObjectType	The object type, a business object, or a technical object corresponding to the specified SAP object node type, for example: Outbound Delivery for Outbound Delivery Item.
BusEvtLogPayloadJSONString	Business Event Payload
BusEvtLogLastChangedDateTime	The date and timestamp of when the payload entry was last updated.

6.3.1.26.4.2.4 Business Activity

CDS View Name	`I_BusEvtLogBusinessActivity`

Purpose

This CDS view is designed to provide a comprehensive overview of business event activities within a system. It captures detailed information about each event, including its type, creation date, associated user, and various identifiers related to the business object nodes involved in the event. This view is particularly useful for tracking and analyzing transactional data related to business events.

This CDS view provides the data to answer the following business questions:

- What activities were logged in the system?

Prerequisites

Authorizations

Users who want to use this CDS view must have the following authorization objects assigned:

- `BEL_SOT` (SAP Object Type)

Structure

Important Fields

Important fields in this view include the following:

Field Name	Description
BusEvtLogBusinessActivity	Activity ID
SAPObjectType	The object type, a business object, or a technical object corresponding to the specified SAP object node type, for example: Outbound Delivery for Outbound Delivery Item.
BusEvtLogCreationDate	Action Executed Date
BusEvtLogReferenceEventUUID	Event GUID
SAPObjectNodeType	Node type of an object type, for example, for an Outbound Delivery SAP object type, the SAP object node types are Outbound Delivery (header) and Outbound Delivery Item (item).
EventOperation	Business Event Operation. For example: Created, Deleted, Changed, and so on.
SAPBusinessObjectNodeKey1	First part of the key of an object node instance, for example, in one event for an Outbound Delivery Item, the first part of the key is 80013910 (outbound delivery number).
SAPBusinessObjectNodeKey2	Second part of the key of an object node instance, for example, in one event for an Outbound Delivery Item, the second part of the key is 00020 (item number). If an event refers to an SAP object node type with only one key part (such as: Outbound Delivery [header]), then all subsequent key parts are empty. **Note** There can be many more node keys, depending on how many parts the object key has. For example: SAPBusinessObjectNodeKey3, SAPBusinessObjectNodeKey4, and so on. SAPObjectNodeType and SAPBusinessObjectNodeKey (1-8) identify a node instance of an SAP object. Outbound Delivery Item is an example of an SAP object node.
CreatedByUser	Users Who Executed the Action
BusEvtLogCreationDateTime	Action Execution Time Stamp
BusinessEventLogLogicalSystem	Logical System
BusEvtLogTransactionID	Identifies the session in which a business event was created
BusEvtLogLastChangedDateTime	The date and timestamp of when the payload entry was last updated.

Field Name	Description
IsTechnicalUser	Is Technical User
BusEvtLogUserInteractionType	Application Type
BusEvtLgUsrIntactnTypeValue	Application ID
SAPBusinessObjectNodeKey1Name	Key1 Name
SAPBusinessObjectNodeKey2Name	Key2 Name
SAPBusinessObjectNodeKey3Name	Key3 Name
SAPBusinessObjectNodeKey4Name	Key4 Name
SAPBusinessObjectNodeKey5Name	Key5 Name
SAPBusinessObjectNodeKey6Name	Key6 Name
SAPBusinessObjectNodeKey7Name	Key7 Name
SAPBusinessObjectNodeKey8Name	Key8 Name
BusinessEventLogSource	Business Event Source

6.3.2 Communication Operations

Communication Management [page 3287]
Learn more about the basic principles of communication management when integrating your system or solution with other systems to enable data exchange in your ABAP environment.

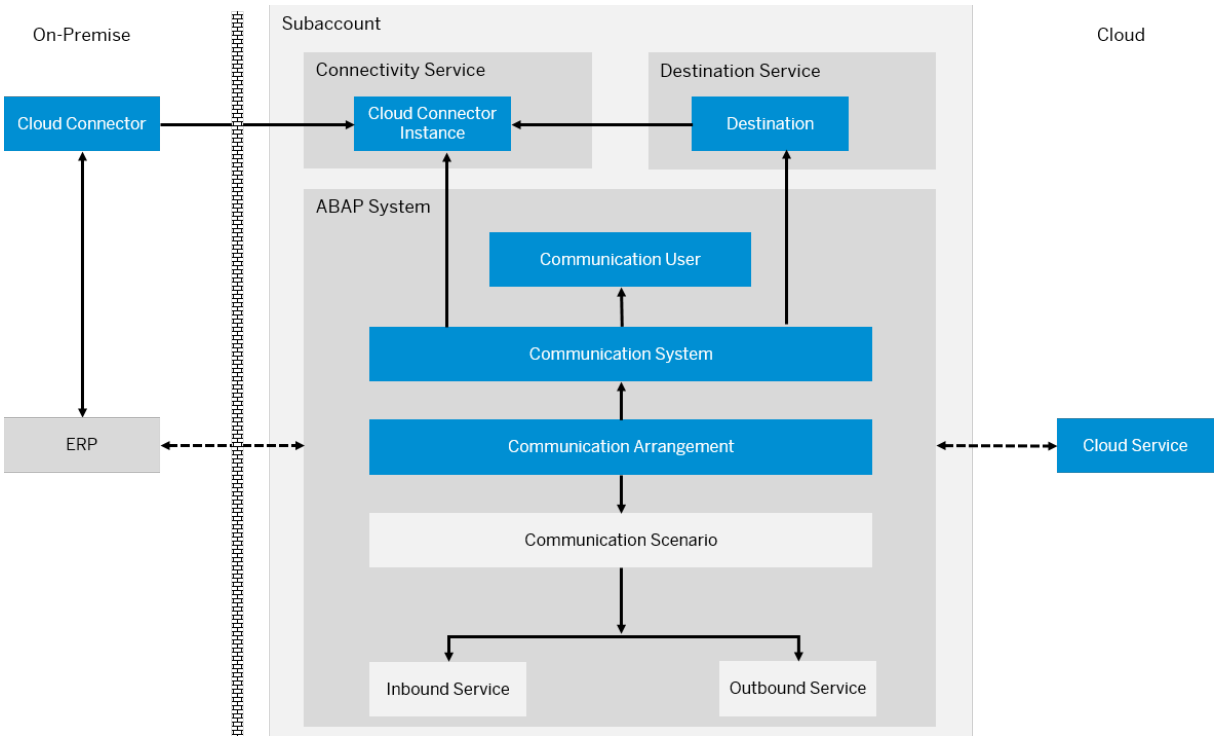
Integration in BTP Apps via SAP Destination Service [page 3290]
In this topic you learn how to configure destinations in the SAP Destination service that shall be used by applications to consume services provided by the ABAP environment, and how to maintain corresponding communication arrangements in the ABAP environment. For instance, how to configure a destination that is used by an SAP Fiori application from the HTML5 repository that accesses an OData service of the ABAP environment.

Overview of Communication Scenarios Managed by SAP [page 3305]
Find a quick overview of all the communication scenarios in the ABAP environment.

Integration Scenarios [page 3306]
You can integrate the ABAP environment with the following services.

6.3.2.1 Communication Management

Learn more about the basic principles of communication management when integrating your system or solution with other systems to enable data exchange in your ABAP environment.



6.3.2.1.1 Communication Scenario

A communication scenario, which is created in the development system using `ABAP Development Tools` and transported to other systems, is a design time description of how two communication partners communicate with each other. It consists of inbound and/or outbound services as well as supported authentication methods.

It provides technical information, such as the used inbound and outbound services and their service type, for example OData or SOAP, and the number of allowed communication arrangement instances. If the scenario exposes inbound services, it specifies the authorizations that are required to run the services.

The following types of communication scenarios are available:

- **Managed by SAP**, where SAP provides a ready-to-use communication scenario and you create and maintain a communication arrangement.
- **Managed by the customer**, where you develop a communication scenario and create and maintain a communication arrangement.

Related Information

Display Communication Scenarios [page 2996]
Overview of Communication Scenarios Managed by SAP
Outbound Communication

6.3.2.1.2 Communication System

A communication system is a specification of a system that represents a communication partner and the technical information required for the communication, such as the host name, port, users for inbound or outbound communication, certificates, etc.

If the communication system represents an on-premise system that is protected by a firewall, the system can be connected by assigning a cloud connector.

Instead of maintaining the credentials for outbound communication in the communication system itself, the communication system can also refer to a destination from the destination service. This can be a destination on subaccount level, or a destination from a destination service instance.

An administrator user in the ABAP environment has to create the communication system in the Communication Systems [page 3001] app in SAP Fiori launchpad. Note that the communication system is not transported between ABAP systems but created locally. The communication partner can vary for each system.

6.3.2.1.3 Communication User

A communication user is a specific type of technical user that is assigned to a communication system. The user can be assigned a password or X.509 certificate.

A communication user is added as a user for inbound communication to communication systems.

An administrator user in the ABAP environment creates the communication user in the Maintain Communication Users [page 2993] app in SAP Fiori launchpad. Note that the communication user is not transported between systems but created locally. The technical users and their credentials can vary for each system.

6.3.2.1.4 Communication Arrangement

A communication arrangement is a runtime description of a specific communication scenario. It describes which communication partners communicate with each other in the scenario, and how they communicate.

To describe this runtime behavior, you have to create an arrangement for a scenario, assign the communication system and communication users, and select the authentication method that shall be used.

If the communication scenario exposes inbound services, the communication user is granted the authorizations that have been specified for the communication scenario.

An administrator user in the ABAP environment creates the communication arrangement in the Communication Arrangements [page 2997] app in SAP Fiori launchpad. Note that it is not transported between systems but created locally. For certain SAP-managed scenarios that integrate an SAP BTP service, you can also create a communication arrangement from a service key of the service instance that you would like to connect.

For inbound-only scenarios, you can create a communication arrangement by creating a corresponding service key for the ABAP environment service instance.

Related Information

Maintain a Communication Arrangement for Inbound Communication

6.3.2.1.5 Destination Service

Using the SAP destination service, you can retrieve and store technical information about the target resource (destination) that you want to connect with your application to a remote service or a system.

The destination service allows you to read and manage the address of a remote service and the user authentication information for the connection on subaccount and service instance level.

Related Information

Destination Service
Create a Destination [page 1219]

6.3.2.1.6 Destination

A destination is stored in the SAP destinations service and contains the connection details for the communication partner.

You can use a destination to:

- Connect your application to the Internet (via HTTP, RFC, or WebSocket RFC) or to an on-premise system (via HTTP or RFC)
- Send and retrieve emails by configuring a mail destination

> ⓘ Note
>
> You can create a destination for a destination service on instance level or subaccount level.
>
> A destination can be referenced in a communication system. To refer a destination from a destination service instance, a communication arrangement for SAP-managed scenario `SAP_COM_0276` needs to be

maintained to enable communication with the destination service instance. In this case, the destination name and the service instance name need to be maintained in the communication system.

Related Information

Managing Destinations

6.3.2.1.7 Cloud Connector

The Cloud Connector serves as a link between cloud applications and on-premise systems.

In the Cloud Connector, the subaccount of the ABAP system is connected so that the Cloud Connector can be used within a communication system or destination in the subaccount.

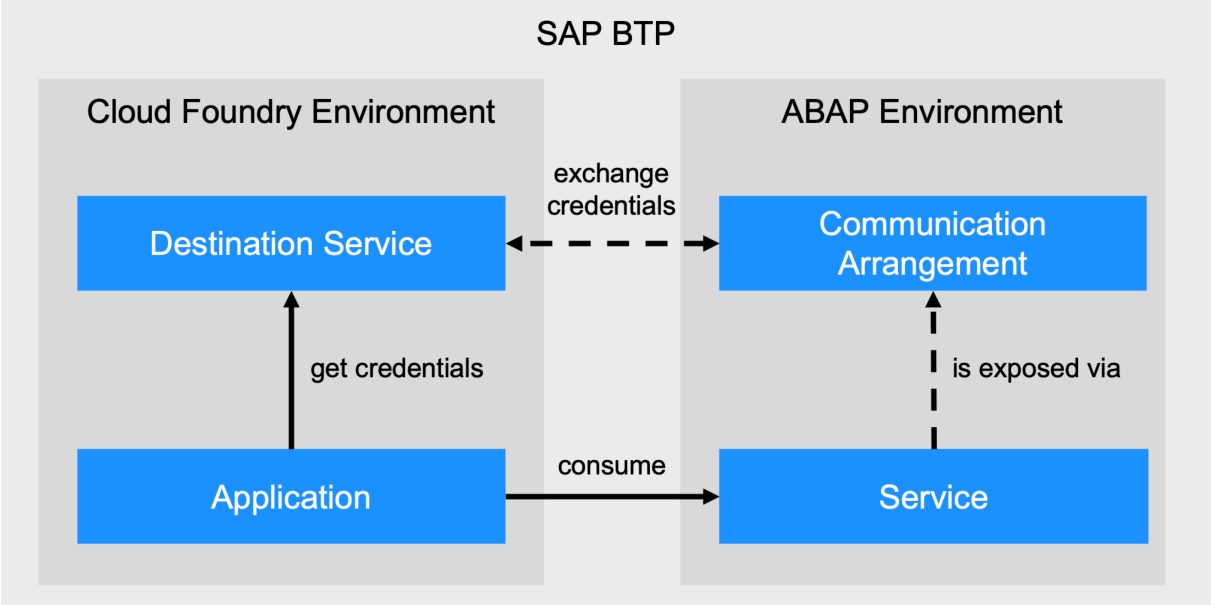
It provides an easy setup with a clear configuration of the systems that are exposed to SAP BTP.

Related Information

Cloud Connector
Integrating On-Premise Systems [page 1214]

6.3.2.2 Integration in BTP Apps via SAP Destination Service

In this topic you learn how to configure destinations in the SAP Destination service that shall be used by applications to consume services provided by the ABAP environment, and how to maintain corresponding communication arrangements in the ABAP environment. For instance, how to configure a destination that is used by an SAP Fiori application from the HTML5 repository that accesses an OData service of the ABAP environment.



Choosing the Right Authentication Method

Before you start with the configuration, you need to decide on the right authentication method first. The right authentication method can be decided by answering the following questions:

What Authentication Methods Are Supported Technically?

The authentication method needs to be supported by both parties. The following authentication methods are commonly supported by the SAP Destination service and the ABAP environment. For more information, see Supported Protocols and Authentication Methods.

- Authentication via communication user
  - Basic authentication (user name & password)
  - Client certificate authentication (X.509 certificate)
- Authentication via business user (principal propagation)
  - SAML Assertion authentication
  - OAuth 2.0 SAML Bearer Assertion grant
    - Client secret authentication (client ID & secret)
    - Mutual TLS (mTLS) client authentication (client ID & X.509 certificate)

The following applies to each authentication method:

- Basic authentication: A user name and password are sent to the ABAP environment for authentication.
- Client certificate authentication: An X.509 certificate is used.
- SAML assertion authentication: A SAML bearer assertion is sent.

In all three cases the credentials are directly sent to the ABAP environment. However, when using OAuth 2.0, then two endpoints are involved: the authorization server and the resource server. First, the SAP Destination service calls the token endpoint of the authorization server and provides an authorization grant to obtain an access token. To authenticate at the authorization server, a client secret or an X.509 certificate can be used.

The supported authorization grant is SAML bearer assertion. Second, the client application uses the access token to authenticate at the ABAP environment to access the actual resource.

Who Shall Be Acting?

Decide if the actions in the ABAP environment shall be executed under a communication user independent from the user of the client application, or if the actions shall be performed under an individual business user corresponding to the user of the client application (principal propagation).

If the service shall be processed as communication user, then choose one of the following authentication methods:

- Basic authentication
- Client certificate authentication

If the principal shall be propagated to the target, then use one of the following authentication methods:

- SAML assertion authentication
- OAuth 2.0 SAML Bearer Assertion

Although OAuth SAML 2.0 Bearer Assertion is a well-known standard and enables additional access restriction using OAuth scopes, it also has some drawbacks compared to the proprietary SAML assertion authentication method:

- It requires a corresponding communication scenario and arrangement to provide the scope to the OAuth client.
- Only OData services are supported.
- It requires an additional round trip to exchange the authorization grant against an access token.
- The configuration is much more complicated and, thus, more error prone.

If you don't have any special requirements to use the OAuth standard, then the usage of SAML assertion authentication is preferred over OAuth 2.0 SAML Bearer Assertion.

How Strong Are the Security Requirements?

Certificate-based authentication is more secure than using a password-based authentication method: a password is a shared secret and is known by both communication partners. Certificate-based client authentication uses public key infrastructure. The private key, which is owned by the client, isn't shared with the communication partner. Moreover, a password change between the communication partners is disruptive, while a non-disruptive change can be achieved with certificate-based authentication since more than one client certificate can be assigned to a communication user.

Hence, client certificate authentication is preferred over Basic Authentication and OAuth 2.0 SAML Bearer Assertion Grant with mTLS-based OAuth client authentication is preferred over OAuth 2.0 SAML Bearer Assertion Grant with client secret authentication.

The client certificates can be provided by the application, can be uploaded to the SAP Destination service, or generated by the SAP Destination service. The ABAP environment needs to trust the certificate authority (CA) of the client certificate. The list of trusted root certificate authorities can be found in SAP Note 2801396.

Configure the Destination and the Communication Arrangement

Once you have chosen the appropriate authentication method, you can continue to configure the destination, communication user, communication system, and communication arrangement accordingly:

- Inbound communication via communication user
  - Configuring Basic Authentication [page 3293]
  - Configuring client certificate authentication (X.509 certificate)
- Inbound communication via business user
  - Configuring SAML Assertion Authentication [page 3298]
  - Configure OAuth 2.0 SAML Bearer Assertion Grant [page 3301]
    - Client secret authentication (client ID & secret)
    - Mutual TLS (mTLS) client authentication (client ID & X.509 certificate)

6.3.2.2.1 Inbound Communication via Communication User

Configuring Basic Authentication [page 3293]
: When using the basic authentication method, the client application provides the name of a communication user and its password to authenticate against the ABAP environment.

Configuring Client Certificate Authentication [page 3295]
: When using the client certificate authentication method, the client application uses a client certificate to authenticate against the ABAP environment as corresponding communication user.

6.3.2.2.1.1 Configuring Basic Authentication

When using the basic authentication method, the client application provides the name of a communication user and its password to authenticate against the ABAP environment.

Procedure

1. Create a communication user using the Maintain Communication Users app in your ABAP environment system. Enter the following data:

Field	Input
User Name	Provide a user name, for example, **`MY_BASIC_AUTH_USER`**.
Description	Provide a description.

Field	Input
Password	Provide a password or generate one.

2. Create a communication system.

   Create a communication system using the Communication Systems app in your ABAP environment system. Enter the following data:

   General Data

Field	Input
System ID	Provide a system ID, for example, **MY_COMMUNICATION PARTNER**.
System Name	Provide a system name.

   Technical Data

Field	Input
General: *Inbound Only*	Activate

   Users for Inbound Communication

Authentication Method	User Name/Client ID
User ID and Password	User name of the communication user, for example, **MY_BASIC_AUTH_USER**.

3. Create a communication arrangement.

   Create a communication arrangement using the Communication Arrangements app in your ABAP environment system. Enter the following data:

Field	Input
Scenario ID	Select the scenario, for example, **Z_MY_SCENARIO**.
Arrangement Name	Provide a name, for example, **MY_ARRANGEMENT**.
Communication System	Select the communication system, for example, **MY_COMMUNICATION_PARTNER**.
Inbound Communication: *User Name*	Select the communication user, for example, **MY_BASIC_AUTH_USER**.
Inbound Communication: *Authentication Method*	**User ID and Password**

4. Get technical data of the own communication system.

   Open the own communication system in the Communication Systems app in your ABAP environment system. To open your own communication system choose *Own SAP Cloud System*.

   Denote the host name, for example `1a354373-d200-46f6-9d5c-daab9a65d9b6.abap.eu10.hana.ondemand.com`.

5. Create a destination.

Create a destination using the Destinations editor in the SAP BTP cockpit. For more information, see Using the Destination Editor in the Cockpit. Provide the following data:

Field	Input
Name	Enter the name of the destination that is used by the application, for example, **`my-basic-destination`**.
Type	*HTTP*
URL	Enter **`https://<hostname of the own communication system>`**, for example, **`https://1a354373-d200-46f6-9d5c-daab9a65d9b6.abap.eu10.hana.ondemand.com`**.
Proxy Type	*Internet*
Authentication	*BasicAuthentication*
User	Enter the name of the communication user.
Password	Enter the password of the communication user.

Related Information

Maintain Communication Users [page 2993]
Communication Systems [page 3001]
Communication Arrangements [page 2997]

6.3.2.2.1.2 Configuring Client Certificate Authentication

When using the client certificate authentication method, the client application uses a client certificate to authenticate against the ABAP environment as corresponding communication user.

Context

The client certificate can be provided by the application itself or by the SAP Destination service. The ABAP environment needs to trust the signing root certificate authority (CA). The list of trusted root certificate authorities can be found in SAP Note 2801396.

Procedure

1. Maintain the key store.

   If the client application does not provide its own client certificate, then either upload a key store including the client certificate, the private key, and the certificate chain to the SAP Destination service, or generate one, for instance **`my-keystore.p12`**. For more information, see Use Destination Certificates.

2. Get technical data of the own communication system.

   Open the own communication system in the Communication Systems app in your ABAP environment system. To open your own communication system choose *Own SAP Cloud System*.

   Denote the host name, for example, `1a354373-d200-46f6-9d5c-daab9a65d9b6.abap.eu10.hana.ondemand.com`.

3. Create a destination.

   Create a destination using the Destinations editor in the SAP BTP cockpit. For more information, see Using the Destination Editor in the Cockpit. Provide the following data:

Field	Input
Name	Enter the name of the destination that is used by the application, for example, **`my-client-certificate-destination`**.
Type	*HTTP*
URL	Enter **`https://<hostname of the own communication system>`**, for example, **`https://1a354373-d200-46f6-9d5c-daab9a65d9b6.abap.eu10.hana.ondemand.com`**.
Proxy Type	*Internet*
Authentication	**`ClientCertificateAuthentication`**
Use client provided certificate	Activate, if the client provides the client certificate. Otherwise, deactivate.
Key Store Location	If the certificate is provided by the SAP Destinations service and not the client itself, then select the name of the corresponding key store, for instance **`my-keystore.p12`**.
Key Store Password	If the certificate is provided by the SAP Destinations service and not the client itself, then provide the password of the key store.

4. Create a communication user.

   Create a communication user using the Maintain Communication Users app in your ABAP environment system. Enter the following data:

Field	Input
User Name	Provide a user name, for example, **MY_CERT _USER**.
Description	Provide a description.
Certificate	Upload the client certificate to the communication user. During runtime, the service will be processed by the communication user that is assigned to the certificate.

5. Create a communication system.

   Create a communication system using the Communication Systems app in your ABAP environment system. Enter the following data:

   General Data

Field	Input
System ID	Provide a system ID, for example, **MY_COMMUNICATION_PARTNER**.
System Name	Provide a system name.

   Technical Data

Field	Input
General: *Inbound Only*	Activate

   Users for Inbound Communication

Authentication Method	User Name/Client ID
SSL Client Certificate	User name of the communication user, e.g. **MY_CERT_USER**

6. Create a communication arrangement.

   Create a communication arrangement using the Communication Arrangements app in your ABAP environment system. Enter the following data:

Field	Input
Scenario ID	Select the scenario, for example, **Z_MY_SCENARIO**.
Arrangement Name	Provide a name, for example, **MY_ARRANGEMENT**.
Communication System	Select the communication system, for example, **MY_COMMUNICATION_PARTNER**.
Inbound Communication: *User Name*	Select the communication user, for example, **MY_CERT_USER**.
Inbound Communication: *Authentication Method*	*SSL Client Certificate*

Related Information

Maintain Communication Users [page 2993]

Communication Systems [page 3001]
Communication Arrangements [page 2997]

6.3.2.2.2 Inbound Communication via Business User (Principal Propagation)

Configuring SAML Assertion Authentication [page 3298]
: When using the SAML assertion authentication method, the client application sends a signed SAML bearer assertion containing information about the business user to authenticate against the ABAP environment.

Configure OAuth 2.0 SAML Bearer Assertion Grant [page 3301]
: When using the OAuth 2.0 SAML Bearer Assertion Grant authentication method, the SAP Destination service sends a signed SAML Bearer Assertion containing information about the business user to the token endpoint of the ABAP environment to obtain an access token.

6.3.2.2.2.1 Configuring SAML Assertion Authentication

When using the SAML assertion authentication method, the client application sends a signed SAML bearer assertion containing information about the business user to authenticate against the ABAP environment.

Context

You can provide your own certificate to sign the SAML assertion or use the standard subaccount-wide signing certificate. The ABAP environment needs to trust the issuer of the SAML bearer assertion.

Procedure

1. Maintain the key store for the signing certificate.

   If the SAML assertion shall not be signed by the standard subaccount-wide signing certificate, then either upload a key store including the signing certificate, the private key, and the certificate chain, or generate one, for instance **`my-signing-keystore.p12`**. For more information, see Use Destination Certificates.
2. Download the signing certificate.

   If you use the standard subaccount-wide signing key, then download the signing certificate by choosing *Download Trust* in the SAP Destination editor.
3. Get data of the own communication system.

   Open the own communication system in the Communication Systems app in your ABAP environment system. To open your own communication system, choose *Own SAP Cloud System*.

Denote the following values:

- Host Name, for example, `1a354373-d200-46f6-9d5c-daab9a65d9b6.abap.eu10.hana.ondemand.com`
- SAML2 Audience, for example, `https://1a354373-d200-46f6-9d5c-daab9a65d9b6.abap-web.eu10.hana.ondemand.com`

4. Create a destination.

   Create a destination using the Destinations editor in the SAP BTP cockpit. For more information, see Using the Destination Editor in the Cockpit. Provide the following data:

Field	Input
Name	Enter the name of the destination, for example, **`my-SAML-assertion-destination`**.
Type	*HTTP*
URL	Enter **`https://<hostname of the own communication system>`**, for example, **`https://1a354373-d200-46f6-9d5c-daab9a65d9b6.abap.eu10.hana.ondemand.com>`**.
Proxy Type	*Internet*
Authentication	*SAMLAssertion*
Key Store Location	If you don't use the standard signing certificate, then select the name of the corresponding key store, for instance **`my-signing-keystore.p12`**.
Key Store Password	If you don't use the standard signing certificate, provide the password for the keystore.
Audience	Enter the OAuth 2.0 SAML2 Audience from the own communication system, for example, **`https://1a354373-d200-46f6-9d5c-daab9a65d9b6.abap-web.eu10.hana.ondemand.com`**.
AuthnContextClassRef	**`urn:oasis:names:tc:SAML:2.0:ac:classes:PreviousSession`**

   Enter the following additional properties:

   Additional Properties

Property	Input
`nameIdFormat`	Enter **`urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress`** if the e-mail is propagated or **`urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified`** if the user name is propagated. For more information, see User Propagation via SAML 2.0 Bearer Assertion Flow.

5. Create a communication user.

   For this authentication method, a communication user isn't needed since it is solely based on the trust relationship to the issuer of the SAML assertion – which is configured in the communication system.

6. Create a communication system.

   Create a communication system using the Communication Systems app in your ABAP environment system. Enter the following data:

   General Data

Field	Input
System ID	Provide a system ID, for example, **`MY_COMMUNICATION_PARTNER`**.
System Name	Provide a system name.

   Technical Data

Field	Input
General: *Inbound Only*	Activate

   Identity Provider

Field	Input
SAML Bearer Assertion Provider	Activate
User ID Mapping Mode	Select *User Name*. If the **`NameIdFormat`** is unspecified, then the value of the **`NameID`** element in the SAML assertions' subject is mapped to the user name.
SAML Bearer Issuer	Enter the SAML entity ID of the SAML bearer issuer, which corresponds to the common name (the string after **`CN=`**) of the signing certificate subject.
Upload Signing Certificate	Upload the signing certificate.

7. Create a communication arrangement.

   For this authentication method, a communication arrangement isn't needed since it's solely based on the trust relationship to the issuer of the SAML assertion – which is configured in the communication system.

Related Information

Maintain Communication Users [page 2993]
Communication Systems [page 3001]
Communication Arrangements [page 2997]

6.3.2.2.2.2 Configure OAuth 2.0 SAML Bearer Assertion Grant

When using the OAuth 2.0 SAML Bearer Assertion Grant authentication method, the SAP Destination service sends a signed SAML Bearer Assertion containing information about the business user to the token endpoint of the ABAP environment to obtain an access token.

Context

You can provide your own certificate to sign the SAML assertion or use the standard subaccount-wide signing certificate. The ABAP environment needs to trust the issuer of the SAML Bearer Assertion.

To authenticate the OAuth client as communication user at the token endpoint, a client secret (password of the communication user) or client certificate can be used. In the latter case, the client certificate needs to be provided by the SAP Destination service and the ABAP environment needs to trust the signing root certificate authority (CA). The list of trusted root certificate authorities can be found in SAP Note 2801396.

Finally, the client application uses the access token to authenticate as a business user at the ABAP environment to access the actual resource.

Procedure

1. Maintain the key store for the signing certificate.

   If the SAML assertion shall not be signed by the standard subaccount-wide signing key, then either upload a key store including the signing certificate, the private key, or generate one, for instance **`my-signing-keystore.p12`**. For more information, see Use Destination Certificates.
2. Download the signing certificate.

   If you use the standard subaccount-wide signing key, then download the signing certificate by choosing *Download Trust* in the SAP Destination service.
3. Maintain the key store for the client certificate.

   If the OAuth client shall authenticate via client ID and client certificate, then either upload a key store including the client certificate, the private key, and the certificate chain to the SAP Destination service, or generate one, for instance **`my-keystore.p12`**. For more information, see Use Destination Certificates.
4. Create a communication user.

   Create a communication user using the Maintain Communication Users app in your ABAP environment system. In this context the communication user represents the OAuth client. Enter the following data:

Field	Input
User Name	Provide a user name, for example, **`MY_OAUTH_CLIENT`**.
Description	Provide a description.

Field	Input
Password	If the OAuth client shall authenticate via client secret, provide a password.
Certificate	If the OAuth client shall authenticate via client certificate, then upload the client certificate to the communication user. During runtime, the service is processed by the communication user that is assigned to the certificate.

5. Create a communication system.

   Create a communication system using the Communication Systems app in your ABAP environment system. Enter the following data:

   General Data

Field	Input
System ID	Provide a system ID, for example, **`MY_COMMUNICATION_PARTNER`**.
System Name	Provide a system name.

   Technical Data

Field	Input
General – Inbound Only	Yes

   Identity Provider

Field	Input
OAuth 2.0 Identity Provider	On
User ID Mapping Mode	Select *User Name*. Thereby, if the NameIdFormat is unspecified, then the value of the NameID element in the SAML assertions' subject will be mapped to the user name.
SAML Bearer Issuer	Enter the SAML entity ID of the SAML bearer issuer, which corresponds to the common name (the string after CN=) of the signing certificate subject.
Signing Certificate	Upload the signing certificate.

   Users for Inbound Communication

Authentication Method	User Name/Client ID
OAuth 2.0	Add a user. User name of the communication user (= OAuth client ID), for example, **`MY_OAUTH_CLIENT`**

6. Create a communication arrangement.

   Create a communication arrangement using the Communication Arrangements app in your ABAP environment system. Enter the following data:

Field	Input
Scenario ID	Select the scenario, for example, **`Z_MY_SCENARIO`**.
Arrangement Name	Provide a name, for example, **`MY_ARRANGEMENT`**.
Communication System	Select the communication system, for example, **`MY_COMMUNICATION_PARTNER`**.
Inbound Communication – User Name	Select the communication user, for example, **`MY_OAUTH_CLIENT`**.
Inbound Communication – Authentication Method	**`OAuth 2.0`**

7. Get technical data of the own communication system.

   Open the own communication system in the Communication Systems app in your ABAP environment system. To open your own communication system, choose *Own SAP Cloud System*.

   Denote the following values:

   - Host name, for example, `1a354373-d200-46f6-9d5c-daab9a65d9b6.abap.eu10.hana.ondemand.com`
   - OAuth 2.0 confidential client token service URL, for example, `https://1a354373-d200-46f6-9d5c-daab9a65d9b6.abap.eu10.hana.ondemand.com/sap/bc/sec/oauth2/token`
   - SAML2 audience, for example, `https://1a354373-d200-46f6-9d5c-daab9a65d9b6.abap-web.eu10.hana.ondemand.com`

8. Create a Destination

   Create a destination using the Destinations editor in the SAP BTP cockpit. For more information, see Using the Destination Editor in the Cockpit. Provide the following data:

Field	Input
Name	Enter the name of the destination, for example **`my-oauth-saml-bearer-assertion-destination`**.
Type	HTTP
URL	Enter **`https://<hostname of the own communication system>`**, for example, **`https://1a354373-d200-46f6-9d5c-daab9a65d9b6.abap.eu10.hana.ondemand.com`**.
Proxy Type	Internet
Authentication	**`OAuth2SAMLBearerAssertion`**
Key Store Location	If you don't use the standard signing certificate, then select the name of the corresponding key store, for example **`my-signing-keystore.p12`**.
Key Store Password	Enter the password for the keystore.

Field	Input
Audience	Enter the SAML2 Audience from the own communication system, for example, **`https://1a354373-d200-46f6-9d5c-daab9a65d9b6.abap-web.eu10.hana.ondemand.com`**.
AuthnContextClassRef	**`urn:oasis:names:tc:SAML:2.0:ac:classes:PreviousSession`**
Use mTLS for token retrieval	Activate if the OAuth client shall be authenticated at the token endpoint using client certificate.
Client Key	Enter the user name of the communication user (= OAuth Client ID), for example, **`MY_OAUTH_CLIENT`**.
Token Service Key Store Location	If the OAuth client shall be authenticated at the token endpoint using client certificate, then select the name of the corresponding key store, for example **`my-keystore.p12`**.
Token Service Key Store Password	If the OAuth client shall be authenticated at the token endpoint using client certificate, then enter the password for the key store.
Token Service URL Type	Choose *Dedicated*.
Token Service URL	Enter the OAuth 2.0 confidential client token service URL from the own communication system, for example, **`https://1a354373-d200-46f6-9d5c-daab9a65d9b6.abap.eu10.hana.ondemand.com/sap/bc/sec/oauth2/token`**.
Token Service User	If the OAuth client shall be authenticated at the token endpoint using client ID and secret, then enter the name of communication user, for example, **`MY_OAUTH_CLIENT`**.
Token Service Password	If the OAuth client shall be authenticated at the token endpoint using client ID and secret, then enter the password of the communication user.
nameIdFormat	Enter **`urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress`** if the e-mail is propagated or **`urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified`** if the user name is propagated. See User Propagation via SAML 2.0 Bearer Assertion Flow for more information.

Related Information

Maintain Communication Users [page 2993]
Communication Systems [page 3001]
Communication Arrangements [page 2997]

6.3.2.3 Overview of Communication Scenarios Managed by SAP

Find a quick overview of all the communication scenarios in the ABAP environment.

Communication Scenarios

Communication Scenario	Documentation Link
SAP Enable Now Manager Integration (SAP_COM_0011)	Integrating SAP Enable Now Manager [page 3360]
SAP Analytics Cloud Integration (SAP_COM_0065)	Integrating SAP Analytics Cloud [page 3315]
Identity Management Integration (SAP_COM_0093)	Inbound Service: Business User [page 1070]
SAP Cloud Identity Services - Identity Provisioning (SAP_COM_0193)	SAP BTP ABAP Environment
SAP BTP Cloud Foundry Destination Service Integration (SAP_COM_0276)	Creating a Destination
External Sheduler - Application Job Administration Integration	Integrating External Sheduler
Business User Change Document Integration (SAP_COM_0327)	Integrating Business User Change Documents [page 1112]
Business Role Change Document Integration (SAP_COM_0366)	Integrating Business Role Change Documents [page 1109]
Application Monitoring Integration (SAP_COM_0454)	Central Health Monitoring Using SAP Focused Run and SAP Cloud ALM [page 3411]
Custom Code Migration Integration (SAP_COM_0464).	Enable Usage of the Analyze Custom Code App [page 2943]
Printing - Pull Integration (SAP_COM_0466)	Configuring the SAP Cloud Print Manager for Pull Integration
Printing - Notification Integration (SAP_COM_0467)	Configuring the SAP Cloud Print Manager for Pull Integration
Adobe Document Services (SAP Forms service by Adobe) Integration (SAP_COM_0503)	Print Forms [page 1595]
SAP Business AI Platform, ABAP environment - Software Component Test Integration (SAP_COM_0510)	(Deprecated) Test Integration (SAP_COM_0510) [page 945]
SAP Cloud ALM for Operations - Application Monitoring Push Integration (SAP_COM_0527)	Central Health Monitoring Using SAP Focused Run and SAP Cloud ALM [page 3411]
SAP BTP Workflow Integration (SAP_COM_0542)	Integrating SAP Workflow Management [page 3371]
E-Mail Integration (SAP_COM_0548)	Sending Mails Using SMTP
SAP Business AI Platform, ABAP environment - Software Assembly Integration (SAP_COM_0582)	Software Assembly Integration (SAP_COM_0582) [page 955]
SAP BTP Transport Management - Outbound Integration (SAP_COM_0599)	Manage Software Components
UI Theme Designer Integration (SAP_COM_0623)	Integrating UI Theme Designer [page 3361]
Communication Management - RFC Metadata Integration (SAP_COM_0636)	Providing an RFC Service
FLP Content Exposure Integration (SAP_COM_0647)	Expose Launchpad Content to Launchpads on SAP BTP

Communication Scenario	Documentation Link
Enabling Notifications (SAP_COM_0683)	Enabling Notifications on SAP Business AI Platform, ABAP environment in Launchpads Running on SAP BTP [page 3314]
ABAP Test Cockpit - Test Integration (SAP_COM_0901)	Running ATC Check Runs [page 928]
ABAP Unit Test Integration (SAP_COM_0735)	Automate the Software Lifecycle Management Process
Security Information and Event Management Integration (SIEM) (SAP_COM_0750)	Security Audit Log Integration
ABAP Test Cockpit Configuration Integration (SAP_COM_0763)	Reading and Deploying ATC Configurations
Factory Calendar - Pull Integration (SAP_COM_0834)	How to export using SAP Cloud Transport Management

6.3.2.4 Integration Scenarios

You can integrate the ABAP environment with the following services.

Integrating Security Audit Logs [page 3307]

Integrating Business Role Change Documents [page 3311]
You can use this scenario to read the change documents of business roles.

Integrating Business User Change Documents [page 3312]
You can use this scenario to read the change documents of business users.

Integrating Content with SAP Build Work Zone, standard edition [page 3313]
Get an overview of the process for integrating content to the SAP Build Work Zone, standard edition. This process is composed of a content exposure flow and a content consumption flow.

Integrating SAP Analytics Cloud [page 3315]
SAP Analytics Cloud is used as analytics client consuming ABAP environment data exposed via ABAP environment analytical queries.

Integrating SQL Services Using SAP Datasphere [page 3318]
You need the integration scenario `SAP_COM_0532` if you want to create a replication flow in SAP Datasphere to subscribe to CDS view entities from an ABAP system and consume data from these views. With a communication arrangement based on `SAP_COM_0532` combined with a communication arrangement for exposing the SQL service, you can consume data from custom CDS view entities that are exposed using the SQL service.

Integrating SAP Document Management Service [page 3320]
You can integrate the SAP Business AI Platform, ABAP environment with SAP Document Management Service to establish a communication scenario.

Integrating SAP Business AI Platform, ABAP environment and Local Printers [page 3321]
To set up the integration between SAP Business AI Platform, ABAP environment and local printers, you can use two communication scenarios, `SAP_COM_0466` and `SAP_COM_0467`.

Integrating ABAP Test Cockpit Checked Systems [page 3331]
Learn how to integrate ABAP Test Cockpit (ATC) checked systems and how to enable the Code Explain functionality in the Analyze Custom Code/Custom Code Migration app.

Integrating External Scheduler [page 3336]
: This inbound OData service enables external systems to schedule an application job based on an application job template.

Integrating Factory Calendar [page 3358]
: Learn how to integrate an ABAP Platform system with the factory calendar.

Integrating Outbound Emails Using SMTP [page 3359]
: Learn how to integrate outbound emails using the Simple Message Transfer Protocol (SMTP).

Integrating SAP Enable Now Manager [page 3360]
: The ABAP environment supports the extended content scenario of SAP Enable Now that allows you to add an additional source for help content to the SAP Companion plugin. You can edit the context-sensitive in-app help delivered by SAP and adapt it to your needs or include your own content.

Integrating UI Theme Designer [page 3361]
: To set up the integration between SAP Business AI Platform, ABAP environment and UI theme designer, you can use the communication scenario `SAP_COM_0623`.

Integrating SAP Build Process Automation [page 3365]
: You can integrate your ABAP environment with SAP Build Process Automation.

Integrating SAP Workflow Management [page 3371]
: You can enable the communication between the ABAP environment and the workflow capability within the SAP Workflow Management service.

Integrating Business Event Logging [page 3377]

6.3.2.4.1 Integrating Security Audit Logs

Service name: `RSAU_LOG_API`

This service enables you to retrieve the security audit log data. You can use the audit log data to integrate them into your Security and Event Management solution (SIEM) to detect security relevant event situations.

This is an OData version 4 service. This version aims to improve processing time and resource consumption of clients and servers. This includes a lightweight JSON format that reduces the size of every response.

Technical Details

A service group contains services belonging to the same business object model and so it shares similar environment conditions. This means the configuration and administration of a service group apply to all services in a service group, that is, routing, authorization, and so on, so you only have to do them once.

In the OData version 4 (V4) runtime implementation of the SAP Gateway Foundation, framework services originate from repositories.

Service Group (incl. Namespace if Existent)	Repository ID	Service Name (incl. Namespace if Existent)	Version
RSAU_LOG_API	SRVD_A2X	RSAU_LOG_API_SERVICE	0001

Service Structure

Service Header (optional)

The service header contains information about the service.

Entities

The entities contain the business data of the service.

Entity	Description	Necessity	Link to Details
Security Audit Log	Security Audit Log		Retrieving Security Audit Log [page 3192]

Service Response

The details included in the response vary according to the type of operation. For more information, see Operation for Security Audit Log Integration.

Integration with SIEM Solution

Service name: RSAU_LOG_API

Communication scenario: SAP_COM_0750

This service enables you to retrieve the security audit log data. You can use the audit log data to integrate them into your Security and Event Management solution (SIEM) to detect security relevant event situations.

Additional Information

> **Note**
>
> For more details about Communication Management, see Communication Management [page 2992].

6.3.2.4.1.1 Operations for Security Audit Log Integration - Read

The Security Audit Log Integration - Read offers the following operations:

Operation	HTTP Method	Sample URL
Retrieving Security Audit Log [page 3192]	GET	Example getting the first 1000 audit log entries: GET <host>/sap/opu/odata4/sap/rsau_log_api/srvd_a2x/sap/rsau_log_api/0001/SecurityAuditLog?$top=1000

6.3.2.4.1.1.1 Retrieving Security Audit Log

With `SecurityAuditLog` you can retrieve the Security Audit Log by using the HTTP method `GET`.

Request

You have to include the following properties in the URL of the request:

Property	Necessity	Comment
`eventID`	Optional	Security Audit Log Event • String <= 3 characters Example: AU1, AU2
`log_tstmp`	Optional	Timestamp of the security audit log event • String($date-time) Example: 2017-04-13T15:51:04Z
`slgmand`	Optional	Client within the SAP System • String <=3 characters Example: 100
`sid`	Optional	System ID • String <=3 characters • Example: YI3

Property	Necessity	Comment
`counter`	Optional	Message counter • integer($int16)
`slgtc`	Optional	Transaction Code • String <= 20 characters Example: VA01
`slgrepna`	Optional	Program Name • String <= 40 characters Example: SAPMSSY8

Response

The operation returns the Security Audit Log.

Examples

Get events within a specific timeframe:

```
GET <host>/sap/opu/odata4/sap/rsau_log_api/srvd_a2x/sap/rsau_log_api/0001/
SecurityAuditLog?
$filter=(log_tstmp%20ge%202021-05-08T10%3A25%3A09Z%20and%20log_tstmp%20le%202021-
05-09T10%3A25%3A09Z)
```

Response

```
{
"eventID": "AU1",
"log_tstmp": "2021-05-08T10:26:22.740611Z",
"slgmand": "100",
"sid": "ABC",
"counter": 0,
"terminal_name": "",
"user_fullname": "Example Administrator",
"slgtc": "S000",
"slgrepna": "RSBTCRTE",
"rsau_text": "Logon successful (type=B, method=A)",
"UserDescription": "Example Administrator"
},
{
"eventID": "AU1",
"log_tstmp": "2021-05-08T10:28:22.924215Z",
"slgmand": "100",
"sid": "ABC",
"counter": 0,
"terminal_name": "",
"user_fullname": "Example Administrator",
"slgtc": "S000",
```

```
"slgrepna": "RSBTCRTE",
"rsau_text": "Logon successful (type=B, method=A)",
"UserDescription": "Example Administrator"
},
```

6.3.2.4.2 Integrating Business Role Change Documents

You can use this scenario to read the change documents of business roles.

Context

This scenario allows you to read the change documents of business roles. We recommend that you limit the number of change documents according to the *Changed On* and the *Business Role ID* fields to avoid a high volume of data.

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Business Role Change Document Integration* scenario (SAP_COM_0366) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

Related Information

How to Create a Communication Arrangement [page 2998]
How to Create Communication Users [page 2995]

6.3.2.4.3 Integrating Business User Change Documents

You can use this scenario to read the change documents of business users.

Context

This scenario allows you to read the change documents of business users. We recommend that you limit the number of change documents according to the *Changed On* and the *Business User ID* fields to avoid a high volume of data.

Procedure

1. Check if a communication user already exists for this scenario. If not, create a communication user in the *Maintain Communication Users* app.

   For more information, see the *Related information* section.
2. Check if a communication system already exists for this scenario. If not, create a communication system in the *Communication Systems* app.

   For more information, see the *Related information* section.
3. Check if a communication arrangement already exists for this scenario. If not, create a communication arrangement for the *Business User Change Document Integration* scenario (SAP_COM_0327) in the *Communication Arrangements* app.

   For more information, see the *Related information* section.

Related Information

How to Create a Communication Arrangement [page 2998]
How to Create Communication Users [page 2995]

6.3.2.4.4 Integrating Content with SAP Build Work Zone, standard edition

Get an overview of the process for integrating content to the SAP Build Work Zone, standard edition. This process is composed of a content exposure flow and a content consumption flow.

Exposing Content to SAP Build Work Zone, standard edition

Communication scenario `SAP_COM_0647`- SAP Fiori Launchpad Content Exposure Integration is needed to supply the SAP Build Work Zone, standard edition with content from the connected system.

For a description of the steps required to set up the communication scenario `SAP_COM_0647`, see Expose Launchpad Content to Launchpads on SAP BTP.

Consuming Content in SAP Build Work Zone, standard edition

Prerequisite

Launchpad content from the same system/client has been transferred to SAP Build Work Zone, standard edition using `SAP_COM_0647` - SAP Fiori Launchpad Content Exposure Integration.

Content Consumption - Cloud Solutions

Content Consumption - Portal Service

Content Consumption - SAP Work Zone

Related Information

Set SAP BTP ABAP Environment as a Content Provider for SAP Build Work Zone, standard edition

6.3.2.4.4.1 Enabling Notifications on SAP Business AI Platform, ABAP environment in Launchpads Running on SAP BTP

Get an overview of the communication scenario `SAP_COM_0683` - Enabling Notifications from SAP Business AI Platform, ABAP environment in launchpads running on SAP BTP. The launchpads running on SAP BTP include the SAP Build Work Zone, standard edition and SAP Cloud Portal service.

Prerequisites

- To receive notifications, every user must have the same email configured in the SAP BTP system user management section and in the Identity Provider (IdP) used by the launchpad running on SAP BTP, Cloud Foundry environment.
- If you want to deliver push notifications to mobile devices, the target user is identified using the email ID they provided. Currently, it is not possible to use other types of identifiers.
- You've generated and noted the following parameters from the launchpad running on SAP BTP:
  - Host
  - OAuth 2.0 Client ID
  - Client Secret
  - Authorization Endpoint
  - Token Endpoint

Procedure

1. Log on to the SAP Fiori launchpad in your SAP Business AI Platform, ABAP environment system.
2. Open the *Communication Arrangements* app.
3. Create a communication arrangement. Choose scenario `SAP_COM_0683`.
4. Enter a name for the communication arrangement.
5. Create a new communication system by choosing *New* next to the communication system.
6. Provide values for *System ID* and *System Name* and then choose *Create*.
7. In the *Technical Data* section, enter the value for *Host* obtained from the launchpad running on SAP BTP.
8. In the *OAuth 2.0 Settings* section, enter values for *Auth. Endpoint* and *Token Endpoint* obtained from the launchpad running on SAP BTP.
9. Create a new user for Outbound Communication by entering the following details:
   - Authentication Method: OAuth 2.0
   - Client Authentication: Basic
   - OAuth 2.0 Client ID: Client ID obtained from the launchpad running on SAP BTP
   - Client Secret: Client Secret obtained from the launchpad running on SAP BTP
10. Save the Communication System and Communication Arrangement.

Related Information

Working with Notifications

6.3.2.4.5 Integrating SAP Analytics Cloud

SAP Analytics Cloud is used as analytics client consuming ABAP environment data exposed via ABAP environment analytical queries.

Prerequisites

- You have the *BI Admin* role in the SAP Analytics Cloud system. For more information, see Requesting Roles.
- You have the *Administrator* role (role template `ID SAP_BR_ADMINISTRATOR`). For more information, see Maintain Business Roles.
- Your end users' web browsers must be configured to:
  - Allow pop-up windows from the SAP Analytics Cloud domain: `[*.]sapbusinessobjects.cloud`, if you'll be choosing the single sign-on (SSO) authentication method in step 2.
  - Allow 3rd party cookies from the Tomcat server's domain. For example, in Internet Explorer 11, go to *Internet Options* > *Security* > *Trusted Sites*, add your domain name, then select *Enable Protected Mode*.

Context

You can configure a live data connection in SAP Analytics Cloud and create an SAP Analytics Cloud model based on an ABAP environment analytical query to consume the result set in an SAP Analytics Cloud Story. The analytical query is delivered as CDS content and is exposed via the analytical engine and the InA protocol via the REST services under `/sap/bw/ina/...` to SAP Analytics Cloud.

SAP BTP destinations are created by the SAP Analytics Cloud app when you define an SAP Analytics Cloud connection of the type ABAP environment.

You set up the scenario using the following tasks:

1. Configure Identity Authentication [page 3316]
2. Create a Communication System [page 3316]
3. Create a Communication Arrangement [page 3317]
4. Configure Live Data Connection [page 3318]

6.3.2.4.5.1 Configure Identity Authentication

Configure your Identity Authentication tenant as the identity provider.

Prerequisites

You have the *System Owner* role.

Procedure

1. Log on to the SAP Analytics Cloud system via the default identity provider tenant.

   The identity provider is the Identity Authentication tenant that you get with the ABAP environment tenant.
2. Go to *System* > *Administration* > *Security* and change the *Authentication Method* from *SAP Cloud Identity (Default)* to *SAML Single Sign-On (SSO)*, then follow the steps for *SAML Single Sign-On (SSO) Configuration*.
3. Under *Step 2: Upload Identity Provider metadata*, deselect *Identity Provider will also be used...* if this checkbox is visible.
4. Under *Step 3*, set the *SAML User Mapping* to *Custom SAML User Mapping*. Under *Security* > *Users*, the value in the *Custom SAML User Mapping* column must equal the *User Data* > *User Name* of the corresponding business user in the ABAP environment.

6.3.2.4.5.2 Create a Communication System

You can create a communication system in the ABAP environment.

Prerequisites

You have the *Administrator* role (role template `ID SAP_BR_ADMINISTRATOR`).

Procedure

1. Log on to the SAP Fiori launchpad in the ABAP environment.
2. Open the *Communication Systems* app from the SAP Fiori Launchpad.
3. Choose *New*.

4. Enter *System ID* and *System Name*. The system ID should be unique.
5. Turn off the *Destination Service* to view the *Host Name*.
6. Under *Technical Data* > *General*, enter the *Host Name* of the SAP Analytics Cloud tenant and *Port* as **443**. SAP Analytics Cloud follows the host name schema `xxx.sapanalytics.cloud`.
7. Choose *Save*.

6.3.2.4.5.3 Create a Communication Arrangement

You create a communication arrangement in the ABAP environment.

Prerequisites

You have the *Administrator* role (role template `ID SAP_BR_ADMINISTRATOR`).

Procedure

1. Log on to the SAP Fiori launchpad in the ABAP environment.
2. Open the *Communication Arrangement* app.
3. Choose *New*.
4. Select the communication scenario `SAP_COM_0065`.
5. Enter a name for the arrangement.
6. Choose *Create*.
7. In the *Common Data* section, select the *Communication System* that you created in the previous section.
8. Under *Additional Properties* > *Tenant ID* maintain the tenant ID that is visible in the URL of the SAP Analytics Cloud app.

   When you call `https://xxx.sapanalytics.cloud`, you will be redirected to the full URL where you can find the tenant ID as shown in this example: `https://xxx.sapanalytics.cloud/sap/fpa/ui/tenants/«tenant ID»/app.html`. You can also find the tenant ID under *Menu* > *SAC Tenant* > *System* > *About* > *System Name*.
9. Under *Outbound Services* > *UI Link Navigation*, check the *Service Status* as *Active*.
10. Under *Retrieve Stories*, uncheck the *Service Status*.

    > **Note**
    >
    > Retrieving stories is not supported.

11. Choose *Save*.

> **Note**
>
> Do not create more than one communication arrangement with communication scenario `SAP_COM_0065` in your ABAP Environment tenant.

6.3.2.4.5.4 Configure Live Data Connection

You configure a live data connection in SAP Analytics Cloud.

Prerequisites

You have the *BI Admin* role in the SAP Analytics Cloud system.

Procedure

1. Login to the SAP Analytics Cloud tenant.
2. Choose *Main Menu* > *Connection* > +.
3. In the *Select a data source* dialog, expand *Connect to Live Data* and select *SAP S/4 HANA*.
4. In the *S/4 HANA Live Connection* dialog, enter the name and description.
5. Select the *Connection Type* as **`Direct Connection`**.
6. Enter the host name as `xxx.abap-web.stagingaws.hanavlab.ondemand.com`.
7. Enter *HTTPS Port* as **`443`** and *Client* as **`100`**.
8. Select the *Authentication Method* as **`SAML Single Sign-on (Standard Compliant)`**.
9. Choose *OK*.

6.3.2.4.6 Integrating SQL Services Using SAP Datasphere

You need the integration scenario `SAP_COM_0532` if you want to create a replication flow in SAP Datasphere to subscribe to CDS view entities from an ABAP system and consume data from these views. With a communication arrangement based on `SAP_COM_0532` combined with a communication arrangement for exposing the SQL service, you can consume data from custom CDS view entities that are exposed using the SQL service.

The connection that you establish for this scenario is shared by all SAP Datasphere users, and SAP Datasphere controls the usage of the connection and accessed data assets through its integrated policy management agent.

SAP Datasphere connects to the CDC (Change Data Capture) functionality for CDS view entities that records changes of exposed CDS view entities. This component allows multiple consumers to subscribe to a CDS view entity to get the CDS data including the option for delta recording and delta consumption.

SAP Datasphere either adds another subscription to an existing CDS extraction process (if the CDS view entity is already processed by another subscriber) or creates a new extraction process and adds a subscription (if the CDS view entity is configured for extraction the first time), followed by the consumption of the data.

Prerequisites

- You have entered the user and password for your user (typically a technical user) and have selected *SAP ABAP* as the connection type for the connection to the SAP system in the *Connections* area.
- The user in the SAP system that you use to complete the configuration steps should have administrator privileges or at least Business Catalog ID `SAP_CORE_BC_COM` for access to *Communication Management*.
- You are familiar with the steps required to expose CDS view entities. For more information, see Data Consumption Using SAP Datasphere.

Procedure

1. Create a communication user in the *Maintain Communication Users* app. You will later need this user name and password to establish the connection in SAP Datasphere.
2. Create a communication system in the *Communication Systems* app and select the checkbox *Inbound only*.
   Background information: A communication system is important for outbound services, as it is used to register a destination for the communication system in the ABAP system. In the given scenario, requests always originate from the external system, so selecting the checkbox *Inbound only* is sufficient.
3. For a production scenario, switch on *OAuth 2.0 Identity Provider*, enter a provider name and upload a signing certificate. Note that this step is not relevant if you are creating a test scenario.
4. Add the communication user that you created in step 1 with authentication method *User Name and Password* to *Users for Inbound Communication*.
5. Create a communication arrangement for communication scenario `SAP_COM_0532` in the *Communication Arrangement* app and enter the name of the communication system that you created in step 2. Note that you do not need to specify an authorization group ID.
6. In SAP Datasphere, choose *Connections*. On the *Local Connections* tab, choose the *Create* button.
7. Enter the necessary information. Choose the connection type *SAP ABAP*. In the *Protocol* field, select the value *Web Socket RFC*. In the *SAP Logon Connection Type* field, select the value *Application Server*. In the *Use Cloud Connector* field, ensure the value *False* is selected. Use the inbound user data from step 4.
8. Test the connection by choosing the *Validate* button. Ensure that the status is *OK* before proceeding.
9. Create a communication arrangement based on a communication scenario with object privileges so that the communication user created in step 1 can access the exposed CDS view entities. For more information, see Creating a Communication Arrangement for Exposing the SQL Service.

You can now create a replication flow in SAP Datasphere to consume data from CDS view entities.

6.3.2.4.7 Integrating SAP Document Management Service

You can integrate the SAP Business AI Platform, ABAP environment with SAP Document Management Service to establish a communication scenario.

Prerequisites

- You've subscribed to the service Document Management Service, Integration Option and created an instance in the Cloud Foundry environment. See Initial Setup for Document Management Service, Integration Option.
- The global administrator of your SAP BTP account has added the service plan for Document Management Service, Repository Option to your subaccount via entitlements. See Configure Entitlements for Subaccounts.
- You've onboarded the storage capability provided by Document Management. See Connect to Document Management Service, Repository Option Using API.
- You've created and copied the service key in the same subaccount of Cloud Foundry environment where Document Management, integration option instance is created. For more information about creating a service key, see Create Service Keys Using the Cockpit.
- You have the *Administrator* role (role template `ID SAP_BR_ADMINISTRATOR`). See Maintain Business Roles.

Context

SAP Document Management Service helps companies with managing records or attachments that are linked to business objects or a specific business context.

For each tenant of SAP Business AI Platform, ABAP environment, a service instance of SAP Document Management Service in Cloud Foundry environment has to be created.

You can build and edit communication arrangements that your organization has set up with a communication partner using the *Communication Arrangement* application. The system provides communication scenarios for inbound and outbound communication that you can use to create communication arrangements. Inbound communication describes how a communication partner receives business documents, while outbound communication describes how a communication partner sends business documents to a communication partner. The communication scenario specifies the authorizations, inbound and outbound services, and supported authentication methods that are necessary for the communication.

> **Note**
>
> There is a collection of sample classes available on Github: **`https://github.com/SAP-samples/abap-cmis-client-sample`**. They help you use SAP Document Management Service, Integration option, in the SAP Business AI Platform, ABAP environment.

Related Information

Create a Communication Arrangement [page 3321]

6.3.2.4.7.1 Create a Communication Arrangement

Create a communication arrangement in the ABAP environment.

Prerequisites

You've the *Administrator* role (role template `ID SAP_BR_ADMINISTRATOR`).

Procedure

1. Log on to the SAP Fiori launchpad in the ABAP environment.
2. Open the *Communication Arrangement* app from the SAP Fiori Launchpad.
3. Choose *New*.
4. Select the communication scenario `SAP_COM_0668`.
5. Enter *Service Key* that you generated. For more information about creating service key, see Create Service Keys Using the Cockpit.
6. Choose *Create*.

   A communication system created with the name `ZSAP_COM_0668`.

   > ⓘ Note
   >
   > Don't create more than one communication arrangement with communication scenario `SAP_COM_0668` in your tenant in the ABAP environment.

6.3.2.4.8 Integrating SAP Business AI Platform, ABAP environment and Local Printers

To set up the integration between SAP Business AI Platform, ABAP environment and local printers, you can use two communication scenarios, `SAP_COM_0466` and `SAP_COM_0467`.

Both communication scenarios are designed to connect third-party output management systems (OMS). OMS provide you with more refined printing functionalities. With `SAP_COM_0466`, the OMS use polling to check for the availability of new items to print. The communication scenario only has inbound communication. The *SAP Cloud Print Manager for Pull Integration* is supported by this communication scenario.

With `SAP_COM_0467`, the SAP Business AI Platform, ABAP environment system sends a notification to the OMS if new items are available to print. Therefore, the communication scenario has in- and outbound communication. This communication scenario doesn't support the use of the *SAP Cloud Print Manager for Pull Integration*.

Item retrieval and status update is the same for both methods. The only difference is the determination of new items to print.

Please check the documentation of your OMS first to see which method is supported before creating a communication arrangement of any type. An example of an implementation of these communication scenarios is Microsoft Universal Print. See this blog for more details.

Related Information

SAP_COM_0466 [page 3322]
SAP_COM_0A86 [page 3326]
SAP_COM_0467 [page 3324]
Maintain Print Queues [page 3128]
Printing [page 1594]

6.3.2.4.8.1 SAP_COM_0466

This document describes the configuration steps that have to be carried out by customers to set up the integration between SAP Business AI Platform, ABAP environment and local printers using SAP_COM_0466.

Purpose

The communication scenario `SAP_COM_0466` enables you to print from an system on your locally installed printers. The communication scenario is designed to connect the *SAP Cloud Print Manager for Pull Integration* and third-party output management systems (OMS). OMS provide you with more refined printing functionalities.

> **Note**
>
> The term 'OMS' will stand for both an OMS as well as the *SAP Cloud Print Manager for Pull Integration* for the remainder of this document.

Prerequisites

- You've installed an OMS.

- You have the following business role assigned to your user:
  - `SAP_BR_ADMINISTRATOR`
- This business role contains the following business catalogs:
  - `Communication Management (SAP_CORE_BC_COM)`
  - `Output Management (SAP_CORE_BC_OM_PRT)`

Process Steps

(SAP Business AI Platform, ABAP environment)

1. Log in to your system and navigate to the *Display Communication Scenarios* app.
2. Search for scenario ID `SAP_COM_0466` and click on it.
3. Now click on *Create Communication Arrangement* in the top right corner.
4. Choose a name for your communication arrangement and click *Create*.
5. You will now need to create a communication system. You can do this by clicking *New* next to the field *Communication System*. For more information on general concepts of communication management, see Communication Management.
6. Choose a new *System ID* and click *Create*.

   > **Note**
   >
   > The *System Name* is filled in automatically with the same name as the ID. We recommend leaving this as it is to prevent confusion.

7. In the *General* section, mark the *Inbound Only* checkbox.
8. Now scroll down to *Users for Inbound Communication* and click "+" to add a new communication user.
9. Click on *New User*. Enter a username and a description. You can then choose to either enter a new password or upload a client certificate by clicking on *Upload*. Click *Create* to save the user, then click *OK*.
10. Save your communication system by clicking *Save* on the bottom right, then save your communication arrangement the same way.
11. Now search for the *Maintain Print Queues* app, open it and select *New* to create a print queue of the type *Pull Integration Queue*. You can also use the operations of communication scenario `A086`. For more information, see SAP_COM_0A86 [page 3326].

    > **Note**
    >
    > The selection of the print queue type is only possible if more types are available. If this is not the case, *Pull Integration* is automatically selected. In case more options are available, it is important to select the *Pull Integration* queue type for the scenario. You won't see the print queue in your OMS system if you choose the wrong queue type.

12. The following steps are only valid if you use the *Maintain Print Queues* app to create print queues: Enter a name and description for the queue. Choose a format and make sure to select the name you chose for your communication arrangement in step 3. Leave the other settings as they are and save your test queue by clicking *Create*.
13. Copy the *API-URL* provided to you in the *Communication Arrangements* app. You'll need the URL later on.
14. Open the print queue and create a couple of test pages by clicking *Create Test Page*. They'll have the status *New*.

15. **(OMS)**: Open your OMS and log in to the SAP Business AI Platform, ABAP environment system by using the system URL you copied in step 13 as well as the user and password created in step 8 and assign a physical printer to your print queue. Now you can print the test pages from your OMS. For details, please refer to the individual product documentation.
16. **(SAP Cloud Print Manager for Pull Integration)**: For more information on how to configure your SAP Cloud Print Manager for Pull Integration, see Communication Management.
17. **(SAP Business AI Platform, ABAP environment)** : In the *Maintain Print Queues* app, your documents will now be labeled *Successful*.

6.3.2.4.8.2 SAP_COM_0467

This document describes the configuration steps that have to be carried out by customers to set up the integration between SAP Business AI Platform, ABAP environment and local printers using SAP_COM_0467.

Purpose

The communication scenario `SAP_COM_0467` enables you to print from an SAP Business AI Platform, ABAP environment system on your locally installed printers. The scenario consists of an inbound OData API which is designed for third-party output management systems (OMS) to connect, as well as a REST outbound API for the notification of new documents to be printed. OMS provide you with more refined printing functionalities.

Prerequisites

- You've installed an OMS.
- You've configured a user in your OMS.
- You have the following business role assigned to your user:
  - `SAP_BR_ADMINISTRATOR`
- This business role contains the following business catalogs:
  - `Communication Management (SAP_CORE_BC_COM)`
  - `Output Management (SAP_CORE_BC_OM_PRT)`

Process Steps

1. Log in to your SAP S/4HANA system. Go to the *Communication Management* section and start the app **Display Communication Scenarios**.
2. Search for scenario ID `SAP_COM_0467` and click on it.
3. Now click on *Create Communication Arrangement* in the top right corner.
4. Choose a name for your communication arrangement and click *Create*.

5. You will now need to create a communication system. You can do this by clicking *New* next to the field *Communication System*.
6. Choose a new *System ID* and click *Create*.

> **Note**
>
> The *System Name* is filled in automatically with the same name as the ID. We recommend leaving this as it is to prevent confusion.

7. In the *General* section enter the host name, the IP address, or the URL of your OMS. Leave the HTTPS port as it is.
8. Now scroll down to *Users for Inbound Communication* and click "+" to add a new communication user.
9. Click on *New User*. Enter a username and a description. You can then choose to either enter a new password or upload a client certificate by clicking on *Upload*. Click *Create* to save the user, then click *OK*.
10. Now scroll down to *Users for Outbound Communication* and click "+" to add a new communication user.
11. Enter the username and password you created in your OMS. Click *Create* to save the user, then click *OK*.
12. Save your communication system by clicking *Save* on the bottom right.
13. Scroll down to the *Outbound Services* section and enter the service path you got from your OMS provider in the field *Path*. The service url will be configured automatically.
14. Save your communication arrangement by clicking *Save* on the bottom right.
15. Now search for the *Maintain Print Queues* app, open it and select *New* to create a print queue of the type **OMS for Notification Integration Queue**. You can also use the operations of communication scenario `A086`. For more information, see SAP_COM_0A86 [page 3326].
16. The following steps are only valid if you use the *Maintain Print Queues* app to create print queues: Enter a name and a description for the queue. Choose a format and make sure to select the name you chose for your communication arrangement in step 4. Set the notification type: When print queue items are added to the queue, a notification call is sent out. Here you can decide whether you want notifications for a certain number of items or for every single new item. Leave the other settings as they are and save your test queue by clicking *Create*.
17. Copy the *API-URL* provided to you in the *Communication Arrangements* app. You'll need the URL later on.
18. Open the test queue and create a couple of test pages by clicking *Create Test Page*. They'll have the status *New*.
19. **(OMS)**: Open your OMS and log in to the S/4 HANA system by using the system URL you copied in step 17 as well as the user and password created in step 9 and assign a physical printer to your print queue. Now you can print the test pages from your OMS. For details, please refer to the individual product documentation.
20. **(SAP Business AI Platform, ABAP environment)**: In the *Maintain Print Queues* app, your documents will now be labeled *Successful*.

6.3.2.4.8.3 SAP_COM_0A86

This document describes the print queue administrator integration. It provides the configuration steps that have to be carried out by customers to create and delete print queues using SAP_COM_0A86.

Purpose

The communication scenario `SAP_COM_0A86` enables you to create and delete print queues directly using APIs without the need to enter the *Maintain Print Queues* app. This is especially helpful for defining print queues across different systems. With the API, you don't need to log in separately. You can create print queues from a single location using this API.

Prerequisites

- You have the following business role assigned to your user:
  - `SAP_BR_ADMINISTRATOR`
- This business role contains the following business catalogs:
  - `Communication Management (SAP_CORE_BC_COM)`
  - `Output Management (SAP_CORE_BC_OM_PRT)`
- Make sure you've followed integration scenario `SAP_COM_0466` until step 11 or `SAP_COM_0467` until step 15. See SAP_COM_0466 [page 3322] or SAP_COM_0467 [page 3324], respectively.

Process Steps

1. Log in to your system and navigate to the *Display Communication Scenarios* app.
2. Search for scenario ID `SAP_COM_0A86` and click on it.
3. Now click on *Create Communication Arrangement* in the top right corner.
4. Choose a name for your communication arrangement and click *Create*.
5. You will now need to create a communication system. You can do this by clicking *New* next to the field *Communication System*. For more information on general concepts of communication management, see Communication Management [page 2992].
6. Choose a new *System ID* and click *Create*.

   > **Note**
   >
   > The *System Name* is filled in automatically with the same name as the ID. We recommend leaving this as it is to prevent confusion.

7. In the *General* section, mark the *Inbound Only* checkbox.
8. Now scroll down to *Users for Inbound Communication* and click "+" to add a new communication user.
9. Click on *New User*. Enter a username and a description. You can then choose to either enter a new password or upload a client certificate by clicking on *Upload*. Click *Create* to save the user, then click *OK*.

10. Save your communication system by clicking *Save* on the bottom right, then save your communication arrangement the same way.
11. Now, use the operations of communication scenario `0A86` mentioned below to create print queues.

Operations

Operations for the print queue administrator integration

Operation	HTTP Method	Sample URL
Get Queues	GET	`<host>/sap/bc/http/sap/APS_OM_PQ/queues`
Get Queue Formats	GET	`<host>/sap/bc/http/sap/APS_OM_PQ/queue-formats`
Get Queue Context	GET	`<host>/sap/bc/http/sap/APS_OM_PQ/queue-context`
Create Queue	POST	`<host>/sap/bc/http/sap/APS_OM_PQ/queues`
Delete Queue	DELETE	`<host>/sap/bc/http/sap/APS_OM_PQ/queues/{QNAME}`
Set Status of Queues	POST	`<host>/sap/bc/http/sap/APS_OM_PQ`

Get Queues

Read existing queues in a tenant using the HTTP method `GET`.

Request: You can retrieve all print queues in tenant with one call: `GET /sap/bc/http/sap/APS_OM_PQ/queues`

Response: The operation returns a list of print queues.

Examples: If the request is `GET /sap/bc/http/sap/APS_OM_PQ/queues`, the response is:

Sample Code

```
[
                                {
"MANDT" : "100",
"QNAME" : "ZTEST01",
"QDESCRIPTION" : "",
"QFORMAT" : "acrobat6.xdc",
"QFORMAT_DESCRIPT" : "PDF",
"TECH_USER_NAME" : "CC*",
"QSTATUS" : "R",
"CREATE_TIME" : 20250212162704,
"LAST_RETRIEVE_TIME" : 20250214061838,
"CLEANUP_PRD" : 1,
"LOCATION_ID" : "",
"LOCATION_ID_TYPE" : "",
"OMS_TYPE" : 466,
"QPARAMS" : "",
"DESTINATION" : "",
"NOTIFY_TYPE" : ""
```

```
},
{
"MANDT" : "100",
"QNAME" : "ZTEST02",
"QDESCRIPTION" : "",
"QFORMAT" : "acrobat6.xdc",
"QFORMAT_DESCRIPT" : "PDF",
"TECH_USER_NAME" : "CC*2",
"QSTATUS" : "H",
"CREATE_TIME" : 20241103110846,
"LAST_RETRIEVE_TIME" : 20241103124912,
"CLEANUP_PRD" : 0,
"LOCATION_ID" : "",
"LOCATION_ID_TYPE" : "",
"OMS_TYPE" : 466,
"QPARAMS" : "",
"DESTINATION" : "",
"NOTIFY_TYPE" : ""
}
]
```

Get Queue Formats

Read existing queue formats in a tenant using the HTTP method `GET`. When a queue is created, the queue format should be part of the input.

Request: You can retrieve all print queue formats in tenant with one call: `GET/sap/bc/http/sap/APS_OM_PQ/queue-formats`

Response: The operation returns a list of print queue formats.

Examples: If the request is `GET /sap/bc/http/sap/APS_OM_PQ/queue-formats`, then the response is:

Sample Code

```
[
{
"QFORMAT" : "acrobat6.xdc",
"QFORMAT_DESC" : "PDF"
},
{
"QFORMAT" : "cab203.xdc",
"QFORMAT_DESC" : "CAB 203 dpi"
}

]
```

Get Queue Context

Read the existing queue context in a tenant using the HTTP method `GET`. The queue context provides information on which users could be used as print queue owners, and in which queue type the users could be used (such as queue type 466 or 467, for example). When a queue is created, the queue owner should be part of the input.

Request: You can retrieve the entire print queue context (which is the queue owner and the corresponding queue type) in a tenant with one call: `GET /sap/bc/http/sap/APS_OM_PQ/queue-context`

Response: The operation returns a list of the entire print queue context.

Examples: If the request is `GET /sap/bc/http/sap/APS_OM_PQ/queue-context`, then the reponse is:

Sample Code

```
[
                                {
                                "MANDT" : "100",
                                "OWNER" : "CC001",
                                "DESCRIPTION" : "",
                                "DESTINATION" : "",
                                "USERALIAS" : "TEST466",
                                "OMSTYPE" : 466
                                },
                                {
                                "MANDT" : "100",
                                "OWNER" : "CC002",
                                "DESCRIPTION" : "",
                                "DESTINATION" : "......",
                                "USERALIAS" : "TEST467",
                                "OMSTYPE" : 467
                                }
                                ]
```

Create Queue

Create a new print queue in a tenant using the HTTP method `POST`.

Request: You can create a new print queue in a tenant with one call: `POST /sap/bc/http/sap/APS_OM_PQ/queues`. The body is: `{"queuename":"", "queuedescription":"","queueformat":"","queueowner":"","queuetype":"","queueretention":"", "queuenotification":"", "queuelocation":"","queuelocationtype":""}`.

Response: The operation returns the status code `200`.

Examples: If the request is `POST /sap/bc/http/sap/APS_OM_PQ/queues` with the body `{"queuename":"ZTEST", "queuedescription":"TEST...","queueformat":"acrobat6.xdc","queueowner":"CC_PRINT_USER","queuetype":"466","queueretention":"2"}`, then the response is

Sample Code

```
~status_code    200
                                ~status_reason    OK
```

Delete Queue

Delete a print queue in a tenant using the HTTP method `DELETE`.

Request: You can delete a print queue in a tenant with one call: `DELETE /sap/bc/http/sap/APS_OM_PQ/queues/{QNAME}`.

Response: The operation returns the status code `200`.

Examples: If the request is `POST /sap/bc/http/sap/APS_OM_PQ/queues/ZTEST`, then the response is

Sample Code

```
~status_code    200
                                ~status_reason    OK
```

Set Status of Queues

Set the status of print queues in a tenant using the HTTP method `POST`.

Request: You can set the status *Resume* or *Hold* of print queues in a tenant with one call: `POST /sap/bc/http/sap/APS_OM_PQ` using the body `[{"queuename":"","queuestatus":""}...]`.

Response: The operation returns status code `200`.

Examples: If the request is `POST /sap/bc/http/sap/APS_OM_PQ` with the body `[{"queuename":"ZTEST2","queuestatus":"H"},{"queuename":"ZTEST3","queuestatus":"R"}]`, then the response is

Sample Code

```
~status_code    200
                          ~status_reason    OK
```

6.3.2.4.8.4 Managing Print Profiles

You can manage your print profiles for `SAP_COM_0466` and `SAP_COM_0467` in the *Maintain Print Queues* app.

Procedure

1. Assign a printer to a 0466 or 0467 print queue in your OMS or in the *SAP Cloud Print Manager for Pull Integration*. Mind that print profiles in the print manager are only supported when in PDF format. Make sure to ignore the print profile in the *PDF Settings* of your print manager in order to secure a correct runtime of the tool.

   > **Note**
   >
   > If you've used SAP_COM_0467 to integrate your local printers to the SAP Business AI Platform, ABAP environment system, you can only choose a local printer from your external OMS instead.

2. Now, open the *Maintain Print Queues* app and select the print queue for which you want to define your print profiles.
3. Select *Manage Print Profiles* in the taskbar to set your print profile.
4. In the *Print Options* dialog, you can manage your print settings. After you've selected your individual print settings from the dropdown menu, select *Save As*.
5. Choose a profile name. Select *Use as favorite* to save the settings of your print profile. If no favorite has been selected, a default will be used. Each print job going into the same print queue will use these same settings. Click *OK*. Your new print profile has now been created. To delete a print profile, select *Delete* from the *Print Options* dialog.

   > **Note**
   >
   > You can only define print profiles if you have administrator rights.

Related Information

SAP_COM_0466 [page 3322]
SAP_COM_0467 [page 3324]

6.3.2.4.9 Integrating ABAP Test Cockpit Checked Systems

Learn how to integrate ABAP Test Cockpit (ATC) checked systems and how to enable the Code Explain functionality in the Analyze Custom Code/Custom Code Migration app.

SAP_COM_0464: This communication scenario connects the Analyze Custom Code/Custom Code Migration app running in an SAP Business AI Platform, ABAP environment, with the customer's SAP S/4HANA or SAP S/4HANA Cloud Private Edition ABAP system. Via this Remote Function Call (RFC) connectivity, the Analyze Custom Code/Custom Code Migration app analyzes the custom code in the connected system using the ABAP Test Cockpit (ATC). One ABAP tenant can be connected to multiple such ABAP systems, but only one connection can be made to the same ABAP system.

SAP_COM_0B22: This communication arrangement describes the configuration steps that have to be carried out to enable the Code Explain functionality in the Analyze Custom Code/Custom Code Migration app on the SAP Fiori launchpad.

SAP_COM_0A68: This communication scenario provides the integration of SAP Solution Manager with an ATC central check system running on SAP BTP ABAP environment to perform Retrofit checks via ChaRM (Change Request Management).

Related Information

SAP_COM_0464 [page 3332]
SAP_COM_0B22 [page 3333]
SAP_COM_0A68 [page 3335]
Custom Code Analysis [page 2941]
Enable Usage of the Analyze Custom Code App [page 2943]

6.3.2.4.9.1 SAP_COM_0464

Learn how to establish a connection between the Analyze Custom Code/Custom Code Migration app running in an SAP Business AI Platform, ABAP environment, with the customer's SAP ECC, SAP S/4HANA or SAP S/4HANA Cloud Private Edition ABAP system.

Prerequisites

- You have the `ADMINISTRATOR` role.
- You've created a destination to your SAP S/4HANA or SAP S/4HANA Cloud Private Edition system in the *Cloud Connector* that is connected to your SAP BTP subaccount.
- You have set up SAP cloud connector to use the Custom Code Migration app in the SAP Business AI Platform, ABAP environment. See SAP note 2861842.

Purpose

This communication scenario connects the Analyze Custom Code/Custom Code Migration app running in an SAP Business AI Platform, ABAP environment, with the customer's SAP S/4HANA or SAP S/4HANA Cloud Private Edition ABAP system. Via this RFC connectivity, the Analyze Custom Code/Custom Code Migration app analyzes the custom code in the connected system using the ABAP Test Cockpit (ATC). One ABAP tenant can be connected to multiple such ABAP systems, but only one connection can be made to the same ABAP system.

Procedure

1. Log on to the *SAP Fiori launchpad* of your ABAP environment.
2. In the *Communication Management* section, select the *Communication Arrangements* tile.
3. Choose *New*.
4. Use the value help to choose scenario `SAP_COM_0464`.
5. Define and enter an *Arrangement Name*.
   If not yet available or defined, create a new communication system for the communication arrangement to define an endpoint for your checked system.
   1. For the *Communication System*, choose *New*.
   2. In the *New Communication System* dialog, enter the *System ID* and *System Name* of the checked system.
   3. Choose *Create*.
   4. In the *Technical Data* tab under *General*, enter the virtual host as specified in your *Cloud Connector* as *Host Name*. The field *Port* has already been filled in automatically with the default *443*.
   5. Turn on the slider for *Cloud Connector*.
   6. Filling in the field *SCC Location ID* is optional.

7. Under *RFC Settings*, fill in the fields *Client*, *Instance Number* and *Target Host* as specified as virtual host in your *Cloud Connector*.
8. Under *User for Outbound Communication*, choose + to assign the RFC user you created in the checked system to the communication system.
9. Choose *Create* and then *Save* to save the communication system and to be navigated back to the Communication Arrangement creation page.

6. Under *Additional Properties*, fill out the fields *Object Provider* and *System Group* (both freely selectable). This is the name you then select as your *Object Provider to Remote System* for your custom code migration project in the Analyze Custom Code/Custom Code Migration app.
7. Under *Outbound Services* > *Retrieve Custom Code* , ensure that the *Service Status* is set to *Active*.
8. Choose *Save* to save the communication arrangement. A message should now pop up at the bottom of the screen informing you that the activation was successful.

Results: You can now select the communication arrangement in the *Object Provider to Remote System* field in the Analyze Custom Code/Custom Code Migration app to establish the connection to your on-premise system.

Related Information

Integrating ABAP Test Cockpit Checked Systems [page 3331]
Custom Code Analysis [page 2941]

6.3.2.4.9.2 SAP_COM_0B22

This document describes the configuration steps that have to be carried out by customers to enable the Code Explain functionality in the Analyze Custom Code/Custom Code Migration app on the SAP Fiori launchpad.

Prerequisites

- You have the `ADMINISTRATOR` role.
- You've defined a communication arrangement for the communication scenario `SAP_COM_0464` to enable communication from your ABAP environment to your on-premise systems using remote function calls (RFC). See SAP_COM_0464 [page 3332].
- You have all the roles required for your SAP BTP subaccount and the cloud connector is installed. See Initial Configuration.

Purpose

You need to follow the steps below to define a communication arrangement for communication scenario `SAP_COM_0B22` for your cloud system to enable the **Code Explain** feature in the **Anaylze Custom Code/ Custom Code Migration** app.

Procedure

1. In the SAP BTP Cockpit, log on to the relevant subaccount.
    1. In the *Connection* section, select *Cloud Connectors*.
    2. Select *Download authentication data* to download the subaccount data.
2. Log on to the cloud connector. See Configure the Cloud Connector.
    1. Choose the *Add Subaccount* button.
    2. Choose *Configure using authentication data* and upload the file you just downloaded.
    3. Add the Location ID for the cloud connector.
    4. Choose *Save*.
    5. In the *Cloud to On-Premise* section in the *Access Control* tab, select +.
    6. Add the details of the on-premise system, *Host* and *Port*, and choose *Next*.
    7. Provide the details on *Virtual Host* and *Port*, and choose *Save*.
    8. Provide details on the resources you'd like to access.
    9. Download the ZIP file attached in SAP Note 2861842 and upload them in the *Resources Of [Virtual Host]* list by selecting *Import resources belonging to a scenario*.
3. Configure Principal Propagation. See Configure Principal Propagation.
4. Configure the ABAP system. See Configure Identity Propagation for RFC.
5. Map short-lived certificates to users in the ABAP server. See Rule-Based Mapping of Certificates.
6. Log on to your SAP BTP subaccount. See also Configure the Destination in Your SAP BTP Subaccount.
    1. In the *Connectivity* tab, select *Destinations*.
    2. Select *Create* and choose *Create from scratch*.
    3. Provide the name of the *Destination*. For *Type*, choose RFC. For *Authentication*, choose Principal Propogation. And for *Proxy Type*, choose On-Premise.
    4. Provide the Location ID from Step 2 c.
    5. In the *Target System Configuration* section, provide the *Server Host* and *System Number* as defined in Step 2 g.
    6. The client will be the original on-premise system client.
    7. Choose *Save*.
7. Log on to the *SAP Fiori launchpad* of your ABAP environment.
8. In the *Communication Management* section, select the *Communication Arrangements* tile.
9. Choose *New*.
10. Use the value help to choose scenario `SAP_COM_0B22`.
11. Define and enter an *Arrangement Name*.
    If not yet available or defined, create a new communication system for the communication arrangement to define an endpoint for your checked system.
    1. For the *Communication System*, choose *New*.
    2. In the *New Communication System* dialog, enter the *System ID* and *System Name* of the checked system.
    3. Activate the *Destination Service* toggle.
    4. Enter the name from Step 6 c.
    5. Choose *Create*.
12. Under *Additional Properties*, select the *Object Provider* you've defined in the communication arrangement for `SAP_COM_0464`.
13. Choose *Save*.

14. Now, on the *SAP Fiori launchpad*, in the *IT Project Management* section, select the *Custom Code Migration* app.

    > **Note**
    >
    > If you have the `Quality Manager – Software Development` role instead of the `Project Manager – IT` role, you'll find the Analyze Custom Code app in the *Custom Code Analysis* tab.

15. Choose *Create*, then *SAP S/4HANA Migration Project*.
16. In the Project Data dialog, enter *Project Description*, *Transition Scenario*, *Material Number Length*, *Location of Custom Code*, and *Target Release*.
17. The value for the *Object Provider*, if *Location of Custom Code* is chosen, is the value of the object provider defined in `SAP_COM_0464`.
18. Choose *Create*.
19. Once the project is created, you can choose *Start Analysis Run* to initiate the analysis.
20. After the finished run, choose *Number of Findings*.
21. Select any of the findings at the bottom, then choose the *Code Explain* button to get an explanation of your code.

Related Information

Integrating ABAP Test Cockpit Checked Systems [page 3331]
Custom Code Analysis [page 2941]

6.3.2.4.9.3 SAP_COM_0A68

You, as a Solution Manager administrator, want to use ATC on an SAP Business AI Platform, ABAP environment system as your central check system to perform Retrofit checks via ChaRM.

Prerequisites

- You have the `ADMINISTRATOR` role.

Purpose

This communication scenario provides the integration of SAP Solution Manager with an ATC central check system running on SAP Business AI Platform, ABAP environment to perform Retrofit checks via ChaRM (Change Request Management).

Procedure

1. Log on to the *SAP Fiori launchpad* of your ABAP environment.
2. In the *Communication Management* section, select the *Communication Arrangements* tile.
3. Choose *New*.
4. Use the value help to choose the `SAP_COM_0A68` scenario.
5. Define and enter an **Arrangement Name**.
6. If not yet available or defined, create a new communication system for the communication arrangement to define an endpoint for your checked system.
    1. For the **Communication System**, choose **New**.
    2. In the **New Communication System** dialog, enter the **System ID** and **System Name** of the checked system.
    3. Choose **Create**.
    4. In the **Technical Data** tab under **General**, select the **Inbound Only** check box.
    Be aware that you should only select the **Inbound Only** check box if you're using a new communication system. Do not check this box when you're re-using a communication scenario.
    5. In the **Users for Inbound Communication** tab, choose **+** to add a new inbound communication user.
    6. Choose **User Name and Password** as **Authentication Method**, and enter a user name, description, and password for the user.
    7. Choose **Create**.
7. Choose *Save*.

Results: You've now set up communication scenario `SAP_COM_0A68` that provides the integration of SAP Solution Manager with an ATC central check system to perform Retrofit checks via ChaRM (Change Request Management). For more information, see SAP S/4HANA Upgrade Scenario.

The class `CL_SATC_API_FACTORY_REMOTE` provides a local proxy API on SAP_BASIS 7.40 (or higher) that calls the APIs of this communication scenario remotely. This local proxy API is used locally in the SAP Solution Manager system to call the remote ATC checks.

When you want to use the Custom Code Migration app running on a BTP ABAP environment system to analyze your code with ATC in other systems, the communication scenario SAP_COM_0464 [page 3332] is needed in addition for further integration with checked systems.

6.3.2.4.10 Integrating External Scheduler

This inbound OData service enables external systems to schedule an application job based on an application job template.

The technical name of this inbound OData service is `BC_EXT_APPJOB_MANAGEMENT`. Please see the description of the operation *JobSchedule* for restrictions. Additionally, the service offers other elementary operations on application jobs.

Service Structure

Entities

The entities contain the business data of the service.

Entity	Description
Jobinfo	Detailed information on a single application job.
JobTemplate	Information on an application job template.
JobScheduleStatus	Information on a scheduled job.
FuncImpReturn	Return code.

Service Response

The details included in the response vary according to the type of operation. For more information, see Operations for External Job Scheduling Service [page 3337].

6.3.2.4.10.1 Operations for External Job Scheduling Service

The external job scheduling service API offers the following operations:

Operation	HTTP Method	Sample URL
Read an Application Job Without Parameters [page 3343]	GET	<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobHeaderSet(JobName='JobName',JobRunCount='JobCount')?$expand=JobStepSet/JobStepLogInfoSet/JobLogMessageSet
Read Application Job Templates [page 3343]	GET	<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobTemplateSet
Schedule an Application Job [page 3344]	POST	<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobSchedule?JobTemplateName='TemplateName'&JobText='Test'&JobUser='BusinessUser'&TestModeInd=false&JobParameterValues=''

Operation	HTTP Method	Sample URL
Read Application Job Status [page 3346]	GET	<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobStatusGet?JobName='JobName'&JobRunCount='JobCount'
Abort a Running Application Job [page 3347]	POST	<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobAbort?JobName='JobName'&JobRunCount='JobCount'
Cancel an Application Job [page 3348]	POST	<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobCancel?JobName='JobName'&JobRunCount='JobCount'
Read Detailed Job and Step Information [page 3349]	GET	<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobinfoGet?JobName='JobName'&JobRunCount='JobCount'
Read Application Job Status Texts [page 3350]	GET	<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobStatusInfoSet
Restart a Failed Application Job [page 3351]	POST	<host>/sap/opu/odata/sap/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobRestart?JobName='JobName'&JobRunCount='JobCount'&JobRestartMode='E'
Check Parameters for Scheduling a Job [page 3352]	POST	<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobScheduleCheck?JobTemplateName='TemplateName'&JobParameterValues='ParameterValueString'
Retrieve Application Job Parameters [page 3353]	GET	<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobParamValuesGet?JobName='JobName'&JobCount='JobCount' and <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobParamValuesStructGet?JobName='JobName'&JobCount='JobCount'

Operation	HTTP Method	Sample URL
Retrieve Application Job Template Parameters [page 3355]	GET	<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/TemplateValuesGet?JobTemplateName='TemplateName' and <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/TempalteValuesStructGet?JobTemplateName='TemplateName'

For more information on these operations, see 3383044.

Working with $Batch

The OData implementation can optionally support running multiple operations sent in a single HTTP request through the use of batching. This is especially useful when scheduling application jobs with a large amount of application job parameters, as these parameters are added to the request URL and may exceed the maximum number of allowed characters. For details, see the chapter **Batch** on page 17 of 3383044.

Error Handling

In case of an error, the external job scheduling service API sets the http status code to 400 and returns a message of an SAP message class. In the http response, you'll see the message class, the message number, and the message text. One example:

Sample Code

```
<code>APJ_RT/003</code><message xml:lang="en">Job doesn't exist.</message>
```

In this example, the message with number 003 of message class APJ_RT and the text "Job doesn't exist" was returned.

In most cases, a message of the message class APJ_RT is used. The following table lists the message numbers and the corresponding texts:

Message code	Message constant	Text
001	TECHNICAL_ERROR	Job was aborted due to a technical error.
002	no_appl_log_found	No application log found
003	no_job_exists	Job doesn't exist.
004	cancel_job_failed	Job &1 couldn't be canceled.
005	no_cancel_privilege	User &1 has no authorization to cancel job &2.
006	delete_job_failed	Job &1 couldn't be deleted.

Message code	Message constant	Text
007	no_job_delete_privilege	User &1 has no authorization to delete job &2.
008	job_cancel_internal_error	Job &1 couldn't be canceled due to an internal error.
009	job_cancellation_succesful	Job &1 has been canceled.
010	job_deletion_succesful	Job &1 has been deleted.
011	job_wrong_state_for_deletion	You can't delete job &1 due to its status.
012	no_runs_for_job_found	Job &1 has no runs.
013	job_schedule_successfully	Job &1 has been scheduled.
014	job_catalog_entry_not_found	Job catalog entry &1 doesn't exist.
015	job_template_entry_not_found	Job template &1 doesn't exist.
016	report_for_job_not_found	Job &1 couldn't be created.
017	job_creation_failed	Report for job &1 couldn't be found.
018	job_metadata_inconsistent	Job &1 metadata are inconsistent.
019	read_job_log_failed	Log for job &1 couldn't be read.
020	job_start_immediate_failed	Job &1 couldn't start immediately
021	job_invalid_startdate	Job &1 has an invalid start date.
022	job_close_failed	Job &1 couldn't be scheduled.
023	job_list_internal_error	Internal error occurred during job list processing.
024	user_no_auth_report	User &1 not authorized to schedule report &2. Report on BC-SRV-APS-APJ.
025	future_schedules_restricted	Job &1 was restricted to 1000 scheduled runs.
026	job_completed_wo_info	Job completed without errors
027	no_auth_appl_log	User &1 is not authorized to read the log.
028	no_auth_job_details	User &1 is not authorized to read the job details.
029	no_auth_job_log	User &1 is not authorized to read the job log.
030	no_auth_job_delete	User &1 is not authorized to delete the job.
031	no_auth_job_cancel	User &1 is not authorized to cancel the job.
032	job_schduling_failed	Job couldn't be scheduled.
033	job_no_end_date	No end date supplied for job &1.
034	job_start_before_end_date	Job &1 must start before the end date.

Message code	Message constant	Text
035	job_no_max_iterations	Enter a number of iterations for the job &1.
036	job_end_cond_ignored	End date of job &1 will be ignored due to missing recurrence pattern.
037	job_aborted	An error occurred. Please create an incident on component &1.
038	job_cancel_checking_failed	Job &1 couldn't be canceled: Check of job failed
039	job_cancel_not_exist	Job couldn't be canceled: Job does not exist.
040	JOB_CANCEL_NOT_RUNNING	Job couldn't be canceled: Job is not running.
041	job_delete_event_failed	Job &1 couldn't be deleted: Event scheduling can't be deleted
042	job_delete_steps_failed	Job &1 couldn't be deleted: Job steps couldn't be deleted
043	job_delete_time_failed	Job &1 couldn't be deleted: Deadline scheduling can't be deleted
044	job_delete_derelease	Job &1 couldn't be deleted: Previous job can't be modified
045	job_delete_enq_pre	Job &1 couldn't be deleted: Succeeding job can't be locked
046	job_delete_enq_suc	Job &1 couldn't be deleted: Previous job can't be locked
047	job_delete_enq_tbtco	Job &1 couldn't be deleted: Job can't be locked
048	job_delete_update_pre	Job &1 couldn't be deleted: Previous job can't be modified
049	job_delete_update_suc	Job &1 couldn't be deleted: Succeeding job can't be modified
050	job_delete_commit_failed	Job &1 couldn't be deleted: A technical error occurred.
051	job_delete_already_run	Job &1 couldn't be deleted: Job has already been published.
052	illegal_report_or_variant	Report name or variant name contains invalid characters.
053	illegal_variantname	Variant couldn't be created: Its name is empty, or contains '&' or '%'.
054	no_auth_variant_create	User &1 is not authorized to create a variant.
055	variant_not_executed	Variant couldn't be created: Variant can't be executed.

Message code	Message constant	Text
056	variant_no_report_exists	Variant couldn't be created: Report doesn't exist.
057	variant_no_report_supplied	Variant couldn't be created: Report is not of type 1.
058	variant_locked	Variant couldn't be created: Variant is locked.
059	variant_lock_enqueue_error	Variant couldn't be created: Internal error from enqueue server
060	job_must_not_periodic	Job &1 must not be periodic. Report incident on component BC-SRV-APS-APJ.
061	job_wrong_periodic_value	Value &1 for recurrence pattern is invalid for job &2.
062	invalid_factory_calendar	Job &1 is not scheduled due to invalid calendar.
063	invalid_restriction_code	Job &1 is not scheduled due to invalid restriction code
064	job_list_filter_failed	Filter for job list couldn't be applied.
065	job_delete_internal_error	Job couldn't be deleted due to an internal error.
066	INVALID_MONTH_SHIFT_DIR	Invalid shift direction for days of month
067	WEEKDAY_WRONG_STARTDATE	Start date and day of recurrence must be the same weekday.
068	INVALID_TIMEZONE	Time zone &1 is invalid.
069	INVALID_FC_ID	Calendar &1 is invalid.
070	SCHEDULING_DATE_ADJUSTED	Start date was adjusted to fit the recurrence pattern.
071	INVALID_SCHEDULING_START_TS	Job must start before the end date.
072	SELECT_AT_LEAST_ONE_WEEKDAY	For weekly recurrence, select at least one weekday.
073	CREATE_JOB_VARIANT_FAILED	Job variant couldn't be created.
074	VALUES_OF_READ_ONLY_PARAM_REM	Parameter &1 has been removed because it is read-only
075	USER_NO_AUTH_TO_SCHEDULE_JOB	Parameter &1 has been removed because it's read-only
076	JOB_CANCELED_BY_USER	Job was canceled by user &1.
077	NO_AUTH_SCHEDULE_FOR_USER	User &1 has no authorization to schedule the job.
078	PARAM_CHANGE_BEFORE_SCHEDULE	Invalid change of parameter when scheduling. Please open a ticket on &1
079	INVALID_JOB_RUN_STATUS	Invalid job run status.

Message code	Message constant	Text
080	JOBCATALOG_IS_NOT_BASIC	Job catalog is not basic.
081	JOB_RESTART_INCORRECT_STATUS	Can't restart the job due to the status
082	JOB_RESTART_INCORRECT_MODE	Can't restart the job after the last step
083	INVALID_USER	User &1 doesn't exist or is invalid.

6.3.2.4.10.1.1 Read an Application Job Without Parameters

Read an application job including step and log data, without parameters, using the HTTP method `GET`. An application job consists of a job header, a job step, a log header, and a log message. Via `$expand` all these job data can be read with one request.

Request

You can retrieve all entities of an application job
with one call: GET `<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobHeaderSet(JobName='JobName',JobRunCount='JobCount')?$expand=JobStepSet/JobStepLogInfoSet/JobLogMessageSet`

You can include the following properties in the URL of the request:

Property	Necessity	Comment
JobName	Mandatory	This is the name of the application job.
JobRunCount	Mandatory	This is the ID of the application job.

Example

GET `<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobHeaderSet(JobName='JobName',JobRunCount='JobCount')?$expand=JobStepSet/JobStepLogInfoSet/JobLogMessageSet`

For more information, see 3383044.

6.3.2.4.10.1.2 Read Application Job Templates

Get a list of all application job templates using the HTTP methode `GET`.

Request

Use the URL <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobTemplateSet without parameters.

Response

The operation returns a list of all application job templates.

Examples

Request

```
GET <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobTemplateSet
```

Response

```
{
  "d": {
    "results": [
      {
        "JobTemplateName": "string",
        "JobTemplateVersion": "string",
        "JobTemplateStepCount": "Unknown Type: integer,null",
        "JobPeriodicGranularity": "Unknown Type: string,null",
        "JobReportName": "Unknown Type: string,null",
        "JobUserName": "Unknown Type: string,null",
        "JobPeriodicValue": "Unknown Type: string,null",
        "JobTemplateText": "Unknown Type: string,null",
        "CreationDateTime": "/Date(1492098664000)/",
        "CreationUserName": "Unknown Type: string,null",
        "LastChangeDateTime": "/Date(1492098664000)/",
        "LastChangeUserName": "Unknown Type: string,null",
        "SupportsTestModeInd": "Unknown Type: boolean,null"
      }
    ]
  }
}
```

6.3.2.4.10.1.3 Schedule an Application Job

With `JobSchedule`, you can schedule an application job based on a job template using the HTTP method `POST`. No start condition can be specified, it's always an immediate start.

Request

You can include the following properties in the URL of the request:

Property	Necessity	Comment
TestModeInd	Optional	Reserved should be set to false.
JobParameterValues	Optional	These are the parameter values of the application job.
JobParameterValuesSimple	Optional	This is the parameter string containing the step number. You can schedule application jobs with a simplified format, containing the step number, the application job name, and the T_VALUE.
JobTemplateName	Mandatory	This is the technical ID of the application job template from which the new application job will be generated.
JobText	Mandatory	This is the human-readable name for the application job.
JobUser	Mandatory	This is the user ID of the user performing the job.
JobUserName	Obsolete	This is the user name of the user performing the job.
JobUserID	Obsolete	This is the ID of the application job user.

Response

The operation schedules an application job.

Examples

Request

```
POST <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobSchedule?
JobTemplateName='TemplateName'&JobText='Test'&JobUser='BusinessUser'&TestModeInd=
false&JobParameterValues=''
```

Response

```
{
  "d": {
    "results": [
      {
        "JobName": "string"
        "JobRunCount": "string"
        "JobStatus": "char"
```

```
        "ReturnCode": "integer" (0 = ok, <> 0 = error)
      }
    ]
  }
}
```

Example of `JobParameterValuesSimple`

Sample Code

```
'{"VALUES":[{"STEP_NR":StepNum","NAME":"ParameterName","T_VALUE":
[{"SIGN":"I","OPTION":"EQ","LOW":"Value or X depending
on parameter Type","HIGH":""}]}]}'
```

6.3.2.4.10.1.4 Read Application Job Status

Read the status of an application job using the HTTP method `GET`.

Request

You can include the following properties in the URL of the request:

Property	Necessity	Comment
JobRunCount	Mandatory	This is the ID of an application job.
JobName	Mandatory	This is the name of an application job.

Response

The operation returns the status of an application job.

Additional Information

Please note that to obtain the status of jobs that weren't scheduled using the external scheduler API, you should assign your communication user not only to SAP_COM_0064 but also to SAP_COM_0326. For more information on this issue, see 3358110.

Examples

Request

```
GET <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobStatusGet?
JobName='JobName'&JobRunCount='JobCount'
```

Response

```
{
  "d": {
    "results": [
      {
        "JobName": "string"
        "JobRunCount": "string"
        "JobStatus": "char"
        "ReturnCode": "integer" (0 = ok, <> 0 = error)
      }
    ]
  }
}
```

6.3.2.4.10.1.5 Abort a Running Application Job

Abort a running application job using the HTTP metthod POST.

Request

You can include the following properties in the URL of the request:

Property	Necessity	Comment
JobRunCount	Mandatory	This is the ID of an application job.
JobName	Mandatory	This is the name of an application job.

Examples

Request

```
POST <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobAbort?
JobName='JobName'&JobRunCount='JobCount'
```

Response

```
{
  "d": {
```

```
    "results": [
      {
        "ReturnCode": "integer" (0 = ok, <> 0 = error)
      }
    ]
  }
}
```

6.3.2.4.10.1.6 Cancel an Application Job

Cancel an application job in general using the HTTP method POST.

This operation is more general than JobAbort. JobCancel treats a job depending on its status:

- If a job is already running, JobCancel will cancel the job like JobAbort.
- If a job has not yet started, JobCancel will delete it.

Request

You can include the following properties in the URL of the request:

Property	Necessity	Comment
JobRunCount	Mandatory	This is the ID of an application job.
JobName	Mandatory	This is the name of an application job.

Examples

Request

```
POST <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobCancel?
JobName='JobName'&JobRunCount='JobCount'
```

Response

```
{
  "d": {
    "results": [
      {
        "ReturnCode": "integer" (0 = ok, <> 0 = error)
      }
    ]
  }
}
```

6.3.2.4.10.1.7 Read Detailed Job and Step Information

Read detailed job and step information using the HTTP method `GET`.

This operation returns one line for each job step, where each line contains some global information of the job (like status, start time, end time) and step specific information like step status and step start time.

Request

You can include the following properties in the URL of the request:

Property	Necessity	Comment
JobRunCount	Mandatory	This is the ID of an application job.
JobName	Mandatory	This is the name of an application job.

Examples

Request

```
GET <host><host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobinfoGet?
JobName='JobName'&JobRunCount='JobCount'
```

Response

```
{
  "d": {
    "results": [
      {
        "JobName": "string",
        "JobRunCount": "string",
        "JobStatus": "string",
        "JobSdlDateTime": "DateTime",
        "JobStartDateTime": "DateTime",
        "JobEndDateTime": "DateTime",
        "JobAppRC": "Integer",
        "StepCount": "DateTime",
        "StepStatus": "String",
        "StepStartDateTime": "DateTime",
        "StepAppRC": "Integer",
      }
    ]
  }
}
```

6.3.2.4.10.1.8 Read Application Job Status Texts

Get a list of all possible application job statuses and their corresponding status texts using the HTTP method `GET`.

Request

Use the URL `<host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobStatusInfoSet` without parameters.

Response

The operation returns a list of all application job statuses and their texts.

Examples

Request

```
GET <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobStatusInfoSet
```

optional with language settings:

```
GET <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobStatusInfoSet?sap-
language=EN
```

Response

```
<d:JobStatus>C</d:JobStatus>
<d:JobStatusText>Canceled</d:JobStatusText>
```

or

```
<d:JobStatus>S</d:JobStatus>
<d:JobStatusText>Scheduled</d:JobStatusText>
```

and so on.

6.3.2.4.10.1.9 Restart a Failed Application Job

Create a copy of the failed application job and start it from or after the failed step using HTTP method `POST`.

Request

You can include the following properties in the URL of the request:

Property	Necessity	Comment
JobRunCount	Mandatory	This is the ID of an application job
JobName	Mandatory	This is the name of an application job
JobRestartMode	Mandatory	This is the type of restart you can trigger

There are two restart modes:

- JobRestartMode='E': Restart the job from the failed step. The job steps before the failed step will be skipped.
- JobRestartMode='A': Restart the job after the failed step. The failed step and the steps before will be skipped.

Response

The operation restarts a failed application job.

Examples

Request

```
POST <host>/sap/opu/odata/sap/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobRestart?
JobName='JobName'&JobRunCount='JobCount'&JobRestartMode='E'
```

Response

```
<d:JobName>JobName</d:JobName>
<d:JobRunCount>JobCount</d:JobRunCount>
<d:JobStatus>R</d:JobStatus>
<d:ReturnCode>0</d:ReturnCode>
```

6.3.2.4.10.1.10 Check Parameters for Scheduling a Job

Check the parameter string that you intend to schedule a job with. This call makes the same checks as the *Check* button contained within the *Application Jobs* app.

Request

You can include the following properties in the URL of the request:

Property	Necessity	Comment
JobTemplateName	Mandatory	This is the name of the application job.
JobParameterValues	Mandatory	These are the parameter values of the application job. For more information, see 3383044.
JobUser	Optional	This is the user under whom the application job is supposed to run. This field is kept optional for compatibility reasons only. It's recommended to always provide the job user when making the call.

Response

The operation returns:

- Parameter String: An overview of the supplied parameter string. (In some applications, the supplied parameters can be changed by the framework. In such cases, a string with the changed parameters is returned in the response).
- Successful Indicator: Returns `true` when the supplied parameters are allowed. Otherwise it will return `false`.
- Changed Indicator: Returns `true` if the parameters have been adjusted. Otherwise it will return `false`.
- Dynamic Properties: `<Reserved>`
- Error Messages: Shows errors or warnings regarding the supplied parameter string.

Examples

Request

```
POST <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobScheduleCheck?
JobTemplateName='TemplateName'&JobParameterValues='ParameterValueString'&JobUser=
'UserName'
```

Response

```
{
                    "d": {
                    "results": [
                    {
                    "JobParameterValues": "ParameterValueString",
                    "SuccessfulInd": "boolean",
                    "ChangedInd": "boolean",
                    "DynamicProperties": "",
                    "ErrMessages": "ErrorMessagesString",
                    }
                    ]
                    }
                    }
```

6.3.2.4.10.1.11 Retrieve Application Job Parameters

Retrieve the application job parameters as a JSON string. This string can directly be used for scheduling an application job.

Request (As a String)

You can include the following properties in the URL of the request:

Property	Necessity	Comment
JobName	Mandatory	This is the name of the application job
JobCount	Mandatory	This is an alphanumerical job count value

Response

The operation returns:

- Parameter String: A JSON string containing the application job parameters.

Examples

Request

```
POST <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/JobParamValuesGet?
JobName='JobName'&JobCount='JobCount'
```

Response

```
<d:ParameterValues>{JSONSTRING}</d:ParameterValues>
```

Request (Standard)

Retrieve the application job parameters. You can include the following properties in the URL of the request:

Property	Necessity	Comment
JobName	Mandatory	This is the name of the application job
JobCount	Mandatory	This is an alphanumerical job count value

Response

The operation returns:

- Step Number: The number of the step where the parameter exists
- Parameter Name: The name of the parameter
- Sign/Option/Low/High: Values according to ABAP ranges

Examples

Request

```
POST <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/
JobParamValuesStructGet?JobName='JobName'&JobCount='JobCount'
```

Response

```
{
  "d": {
    "results": [
      {
       "StepNr": StepNumber,
    "JobParameterName": "ParameterName",
    "Sign": "I",
    "Option": "EQ",
    "Low": "LowValue",
    "High": "HighValue"    }
    ]
  }
}
```

6.3.2.4.10.1.12 Retrieve Application Job Template Parameters

Retrieve application job template parameters as a JSON string. This string can directly be used for scheduling an application job.

Request (As a String)

You can include the following properties in the URL of the request:

Property	Necessity	Comment
JobTemplateName	Mandatory	This is the name of the application job template.

Response

The operation returns:

- Parameter String: A JSON string containing the application job template parameters. The parameters not pre-defined in the application job template are also delivered in the response.

Examples

Request

```
GET <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/TemplateValuesGet?
JobTemplateName='TemplateName'
```

Response

```
<d:ParameterValues>{JSONSTRING}</d:ParameterValues>
```

Request (Standard)

Retrieve the application job parameters. You can include the following properties in the URL of the request:

Property	Necessity	Comment
JobTemplateName	Mandatory	This is the name of the application job template.

Response

The operation returns:

- Application job template parameter values

Examples

Request

```
GET <host>/sap/opu/odata/SAP/BC_EXT_APPJOB_MANAGEMENT;v=0002/
TemplateValuesStructGet?JobTemplateName='TemplateName'
```

Response

```
{
  "d": {
    "results": [
      {
       "StepNr": StepNumber,
    "JobParameterName": "ParameterName",
    "Sign": "I",
    "Option": "EQ",
    "Low": "LowValue",
    "High": "HighValue"    }
    ]
  }
}
```

6.3.2.4.10.2 SAP_COM_0326

This communication scenario concerns application job scheduling authorizations.

Purpose

This inbound OData service enables external systems to schedule an application job based on an application job template.

Context

If you create a communication arrangement based on SAP_COM_0326, your communication user will have the authorization to schedule any application job template under any business user. Furthermore, you'll also have the authorization to run all available operations on application jobs that were created by other users. This means that you can delete or cancel any application job created by any business user.

Prerequisites

- You've installed an external scheduler
- You have the following business roles assigned to your user:
  - **Communication Management (**`SAP_CORE_BC_COM`

Process Steps

1. Open the *Display Communication Scenarios* app.
2. Search for the scenario ID `SAP_COM_0326` and click on it.
3. Select *Create Communication Arrangement* in the top right corner.
4. Choose a name for your communication arrangement and click *Create*.
5. You'll now need to create a communication system. To do this, click *New* next to the field *Communication System*. For more information on general concepts of communication management, see Communication Management [page 2992].
6. Choose a new *System ID* and click *Create*.

   > ⓘ Note
   >
   > The *System Name* is filled automatically with the same name as the ID. We recommend leaving this as it is to prevent confusion.

7. In the *General* section, mark the *Inbound Only* checkbox.
8. Now, scroll down to *Users for Inbound Communication* and click "+" to add a new communication user.
9. Select *New User*. Enter a user name, a description, and a new password or select *Propose Password*. Click *Create* to save the user, then click *OK*.

   > ⓘ Note
   >
   > Please don't use the *Certificate* section. At the moment, only basic authentication has been implemented.

10. Select *Save* to save your communication system.
11. You'll be forwarded to your communication arrangement. In the *Additional Properties* section, you can enter an external job scheduler. These fields are not mandatory.
12. Finally, select *Save* to save your communication arrangement.

6.3.2.4.11 Integrating Factory Calendar

Learn how to integrate an ABAP Platform system with the factory calendar.

Context

This communication scenario allows you to set up a connection to the import factory calendar customizing from another ABAP client. There can be at most one source client for an ABAP tenant.

Prerequisites

You have to connect your ABAP system via RFC. To do this, follow the steps described in 3148310 or install the required support package. The following RFC-enabled function modules are called during import:

- `SCAL_TO_FHC_MIG_COMPLETED`
- `SCAL_READ_CUSTOMIZING`
- `SFHC_READ_CUSTOMIZING`

Procedure

1. Create a communication user in the *Maintain Communication Users* app. For more information, see How to Create Communication Users [page 2995].
2. Create a communication system in the *Communication Systems* app. For more information, see How to Create Communication Systems [page 3002].
3. Create a communication arrangement for the **Factory Calendar Integration** scenario (`SAP_COM_0834`) in the *Communication Arrangements* app. For more information, see How to Create a Communication Arrangement [page 2998].

After creating the communication arrangement, assign the business catalog `SAP_CA_BC_IC_LND_CAL_CA0_PC` **or** `SAP_CA_BC_IC_LND_CAL_CA1_PC` to schedule an application job. The application job is required to pull the data of the three related IMG activities via the connection. The check logic of the application job and the transaction prevent the scheduling and execution in clients where the selected customizing objects are not editable.

> **Note**
>
> All existing factory calendar customizing will be overwritten by the import. In case the automatic pull destination is configured, a warning is issued in the customizing maintenance UIs to prevent concurrent changes.

All changes are recorded on a task.

The source of the factory calendar depends on the migration state of the source client:

- If the source client has not yet been migrated, the cross-client calendar will be pulled
- If the source client has been migrated, the client-dependent calendar will be pulled

6.3.2.4.12 Integrating Outbound Emails Using SMTP

Learn how to integrate outbound emails using the Simple Message Transfer Protocol (SMTP).

Procedure

You must define the following configurations in a communication system. See How to Create Communication Systems for more information.

There are two ways to connect an SMTP server.

Connect publicly available SMTP server:

- Host
- Port
- Disable Cloud Connector

Advanced - Connect internal SMTP server via Cloud Connector:

- Host
- Port
- Enable Cloud Connector
- *SCC Location ID* of the `SAP BTP Cloud Connector`

> **Note**
>
> This parameter is only required if several SAP BTP, Cloud connectors are used in one subaccount (to define the target SAP BTP, Cloud Connector with the same *SCC Location ID*).

> **Note**
>
> If you want to use OAuth 2.0 as authentication method you must define the Token Endpoint for your email provider.

> **Note**
>
> Define a user for outbound communication according to te authentication method you are using.

Afterwards, create a communication arrangement for the `SAP_COM_0548` communication scenario using this communication system.

Communication Arrangement:

1. Log in to your system and follow the steps as described in How to Create a Communication Arrangement.
2. Enter the previously created Communication System.
3. If you are using OAuth 2.0 for the outbound configuration, use the authorized user from the email provider as additional property.
4. If you are using OAuth 2.0 as authentication method, use the scope address from the email provider in the field **Additional Scopes**. This could be something like https://outlook.office365.com/.default.
5. Save your communication arrangement by clicking **Save** on the bottom right.

Testing email outbound functionality:

To create a test email, search for the Fiori app **Monitor Email Transmission** and open it. Use the **Send Testmail** button. Enter your company email address to check if the email arrives in the recipient's inbox.

If an error message appears stating that the recipient address is not allowed, set the address allowlists accordingly, as defined in Configuration of the System Email Outbound.

Related Information

Communication Systems [page 3001]
How to Create a Communication Arrangement [page 2998]
Integrating On-Premise Systems [page 1214]
Configure Access Control (TCP)
Sending Mails Using SMTP

6.3.2.4.13 Integrating SAP Enable Now Manager

The ABAP environment supports the extended content scenario of SAP Enable Now that allows you to add an additional source for help content to the SAP Companion plugin. You can edit the context-sensitive in-app help delivered by SAP and adapt it to your needs or include your own content.

To use the SAP Enable Now Manager, you have to create a communication arrangement that uses communication scenario `SAP_COM_0011`.

In addition, you have to install the SAP Enable Now Manager.

If you need further information, see the SAP Companion Integration guide. The specifics for the ABAP environment are described in section *Set Up of SAP Companion for SAP S/4HANA Cloud and Similar Systems*.

Related Information

SAP Enable Now
SAP Companion Authoring

6.3.2.4.14 Integrating UI Theme Designer

To set up the integration between SAP Business AI Platform, ABAP environment and UI theme designer, you can use the communication scenario `SAP_COM_0623`.

You can use `SAP_COM_0623` to establish a connection to the Cloud Foundry environment.

Create a communication arrangement in SAP Business AI Platform, ABAP environment to establish a connection to Cloud Foundry for fetching custom themes which can then be used in SAP BTP ABAP.

Prerequisites

Your user needs a business role with the business catalog *Communication Management* (`SAP_CORE_BC_COM`) as well as *Security* (`SAP_CORE_BC_SEC`) assigned (e.g., the administrator user).

In Cloud Foundry, you need to have the environment for the UI theme designer prepared as described in Portal Scenario.

A custom theme needs to be created and published in UI theme designer to be available for consumption in SAP Business AI Platform, ABAP environment. For more information, see Create Themes — End to End Flow. The theme ID shown in UI theme designer will be used to define default theme or additional themes to be used in SAP Business AI Platform, ABAP environment.

An entitlement for the UI theme designer (theming) service is required in your global account and needs to be assigned to the subaccount.

Creating an Instance for UI Theme Designer Service in Cloud Foundry

To consume published themes in SAP Business AI Platform, ABAP environment, an instance of UI theme designer service needs to be created in Cloud Foundry.

1. Navigate to the *Service Marketplace* in the Cloud Foundry subaccount. Select service *UI Theme Designer* (name: theming) and click *Create*.
2. In the creation dialog, enter basic info for the service instance
    1. Service: UI Theme Designer
    2. Plan: standard
    3. Runtime Environment: Cloud Foundry
    4. Space

5. Instance Name
3. In the next screen, no further parameters need to be configured.
4. Review and verify the instance details in the following screen and confirm the creation with *Create*.
5. Make sure that a new instance for the *UI Theme Designer (theming)* service is successfully created.

Creating a Service Key in Cloud Foundry

The simplest way to enable communication between SAP Business AI Platform, ABAP environment and Cloud Foundry is to create a service key in Cloud Foundry and use its content for configuring the communication arrangement.

1. Navigate to the space of your Cloud Foundry subaccount. Go to *Services* > *Instances* and select the instance for the UI theme designer.
2. Now you can create a service key. For more information, see Creating Service Keys [page 278].
3. Open the service key and copy its content (e.g., by clicking *Copy JSON*).

Using mTLS for the Communication

To enable mTLS for the communication between SAP Business AI Platform, ABAP environment and the UI theme designer in Cloud Foundry, the steps are a little bit different:

1. In SAP Business AI Platform, ABAP environment, go to *Maintain Client Certificates*.
2. Export the Client Default certificate. Choose one where *Valid to* is a date in the future.
3. In Cloud Foundation, navigate to the space of your subaccount. Go to *Services* > *Instances* and select the instance for the UI theme designer.
4. Now create a service key with mTLS information. For more information, see section Creating a Service Key in Cloud Foundry. When doing this via the Cockpit, enter the following text into the parameters field in the *New Service Key* wizard. Replace the xxxxxxxxxxxxxxxxx with the text you fin in your exported certificate.

   Sample Code

   ```
   {
           "xsuaa": {
               "credential-type": "x509",
               "x509": {
               "certificate": "-----BEGIN CERTIFICATE----- xxxxxxxxxxxxxxxxx
   -----END CERTIFICATE-----",
               "ensure-uniqueness": false,
               "certificate-pinning": true
               }
           }
       }
   ```

5. After creating the service key, open it and copy its content (e.g. by clicking on *Copy JSON*).

If the client certificate is not valid anymore, you need to repeat the described steps.

1. As there can be only one Service Key at the same time for a Service Instance with a client certificate, you either need to delete the old one and create a new one, or you use two Service Instances where one is used active and the other can be used to update the Service Key.
2. Download the new Client Default certificate as described above in steps 1 and 2.
3. Recreate the Service Key as described above in steps 3 and 4. Copy this new key.
4. Then go to the SAP Business AI Platform, ABAP environment, and in the launchpad choose *Communication Management* > *Communication Arrangements*.
5. Select the previously created Communication Arrangement for `SAP_COM_0623`. Choose *Update by Service Key*, paste the newly created Key and then choose *Update*.

Creating a Communication Arrangement in SAP Business AI Platform, ABAP environment

1. Log on to the SAP Business AI Platform, ABAP environment system as administator (or as a user with the necessary role/catalog assigned).
2. In the launchpad, navigate to the group *Communication Management* and choose the tile *Communication Arrangements*.
3. In the *Communication Arrangements* dialog, choose *New*.
4. Select communication scenario `SAP_COM_0623` (*UI Theme Designer Integration*). If you want, change the proposed arrangement name.
5. Paste the service key into the field *Service Key*.
6. In the *Additional Properties* section, enter the custom theme as a default theme. (You can do or change this later in the detail screen).
7. Now click *Create*. With the help of the service key, everything is created in the background - communication system and communication arrangement - and all fields relevant for enabling a connection to Cloud Foundry are filled.

> ⓘ Note
>
> In a SAP Business AI Platform, ABAP environment system, there is only one communication arrangement for `SAP_COM_0623` (connection to the Cloud Foundry environment) per client possible. If you have already an existing connection for custom themes and want to create a new connection, you must delete the old connection first.

8. Now you can maintain the fields for the communication arrangement. Besides the default theme ID, you can also maintain the field *Additional Theme IDs*. Here you can provide additional themes (resp. their IDs) from which the users can select one in the SAP Fiori launchpad personalization.

> → Tip
>
> We strongly recommend that you have a test phase first and enter the custom theme only as an additional theme. If tests by selected users are successful, you can enter your theme as the default theme (which applies to all users as soon as you save the communication arrangement).

9. Click on *Save*.

Maintain Content Security Policy in SAP Business AI Platform, ABAP environment

To be able to correctly display custom themes in SAP Business AI Platform, ABAP environment, the runtime endpoint of UI theme designer (theming) service needs to be maintained as trusted site.

1. Log on to the SAP Business AI Platform, ABAP environment system as administrator (or as a user with the necessary role/catalog assigned).
2. In the launchpad, navigate to the group *Security* and choose te tile *Manage Content Security Policy*.
3. In the app, select the *Trusted Sites* tab and navigate into the *UI_RESOURCES_FONTS* allow list.
4. Press *Add*, then press *New* in the *Managed by Customer* section.
5. Fill in the theming runtime URL for your landscape and press *Save* (e.g. https://theming-runtime.cfapps.eu10.hana.ondemand.com)

> → Tip
>
> The theming runtime URL can be retrieved from the service key of the UI theme designer service instance, where it is the value of the "endpoints" → "runtime" property.

Results

After a refresh in the FLP of SAP Business AI Platform, ABAP environment, your custom theme is available according to your configuration, as follows:

- If you have entered the custom theme as an *additional theme*, the SAP Fiori launchpad is unchanged but the theme is now available in the *User Actions* menu under *Settings* > *Appearance*.
- If you have set the custom theme as a *default theme* (only after thorough tests!), the SAP Fiori launchpad is displayed with the new theme for all users.
  Exception: Users who have switched their theme previously in the user settings of the SAP Fiori launchpad keep their theme. But they can switch to the new default theme in their user settings since it is listed there.

> ⓘ Note
>
> If there were changes within your instance on Cloud Foundry and you need to update the service key, you can copy the new content of the service key and use *Update by Service Key* within the communication arrangement.

Troubleshooting

- **Theming runtime not included in Content Security Policy**
  Custom themes are largely shown correctly, but many icons are not displayed.
  Icons are loaded via fonts. This is reflected in the Developer Tools console (e.g. F12 in Google Chrome) as an error of the type "Refused to load the font ... because it violates the following Content Security Policy directive ..."

Solution: Theming runtime URL needs to be included into the *UI_RESOURCE_FONTS* allow list in *Manage Content Security Policy* app.

- **Custom Theme in Theming Service is not available and FLP in ABAP Environment is broken**
  FLP in SAP Business AI Platform, ABAP environment is not usable because default theme or theme selected for business user cannot be loaded.
  Solution: Is it possible to specify theme ID to be used for FLP in SAP Business AI Platform, ABAP environment using an URL parameter `sap-theme=<theme-id>`
  See *Usage of the sap-theme URL parameter* in SAP Note 2043817.
  This is especially useful if the FLP is broken, and the business user needs to select another theme to resolve the issue e.g., while using SAP-provided theme `sap-theme=sap_fiori_3`.

6.3.2.4.15 Integrating SAP Build Process Automation

You can integrate your ABAP environment with SAP Build Process Automation.

6.3.2.4.15.1 Configuring the SAP Build Process Automation Integration

You can enable the communication between the ABAP environment and SAP Build Process Automation.

This scenario enables ABAP environment applications to extend their business workflows or business processes with workflow capability of SAP Build Process Automation. It allows the provisioning of an API in the ABAP environment to start and control workflow instances running on workflow capability.

You set up the scenario using the following tasks:

1. Create a Communication Arrangement Using a Service Key [page 3365] (recommended)
2. Create Destinations [page 3368]
3. AIF Monitoring [page 3369]

6.3.2.4.15.1.1 Create a Communication Arrangement Using a Service Key

To connect the ABAP environment and SAP Build Process Automation, you need a communication arrangement.

Prerequisites

- You have already created a communication user, or you know how to create a new one through the Maintain Communication Users app on the SAP S/4HANA.

- You have created an SAP Build Process Automation instance and service key on the SAP BTP cockpit of your subaccount.
- If you are using a shared environment, not the "Public" environment (for which the usage is deprecated) you need to update the service instance.
- For this, you must add an API Key for your shared environment and store the API key together with the environment ID in the service instance.
- For more information, see:
  - Add API Keys to an Environment
  - Update a Service Instance

Procedure

1. Get the service key.
   a. In the SAP BTP cockpit, navigate to your space. See Navigate to Orgs and Spaces.
   b. From the navigation pane, choose *Services* > *Instances and Subscriptions*.
   c. Click the name of your SAP Build Process Automation instance and choose the side arrow ›.
   d. From the navigation pane, choose *Service Keys*.
   e. Select one entry, and copy the service key.
2. In your SAP S/4HANA Cloud system, go to *Administration* > *Communication Management*. Open the *Communication Arrangements* app.
3. Choose *New* to create a communication arrangement.
4. In the *New Communication Arrangement* window, in the *Scenario* field enter *SAP_COM_0863*.
5. In the *Arrangement Name* field, enter a name for the arrangement.
6. Choose *Additional Properties* and in the following fields enter:

Property Name	Action
SAP Process Visibility Service Tenant ID	• If you do not require integration with the process visibility capability, leave this field **blank**. • If integration is needed, enter the Tenant ID of the SAP BTP sub-account.
Consumer Type (Workflow)	Choose *Default*.

Property Name	Action

Consumer Type (Visibility)	• Select the customer type corresponding to the scenarios for which you would like to create an extension workflow.
	• The consumer type is used to identify the communication arrangement. Ensure that you assign the customer type to exactly one communication arrangement. You can assign multiple consumer types to one communication arrangement.
	• Based on how the extension workflow is integrated, the customer type may vary. If the intergration method is:
	  • Flexible Workflow Extension of Business Workflow: Use the scenario identifier as the consumer type. For example, WS12345678.
	  • Directly integrated through API: Choose the consumer type provided and documented by the SAP S/4HANA Cloud application.

API (Optional)	• Provide an SAP Build Process Automation API Key if API Triggers are used in relevant SAP Build Process Automation processes. • API keys are created in the Control Tower of SAP Build Process Automation. Ensure that the API keys have the scopes *trigger_read* and *trigger_execute*. • If the API key is already stored in the service instance, you do not need to provide it again in the communication arrangement. **Note** If you want to add an API key, you cannot do it on the initial Communication Arrangement window. You can do it on the next Communication Arrangement page.

7. Select a communication user (of which you know the password) or create a new one to use for inbound communication (from SAP BTP to your ABAP environment).
8. Paste the service key into the corresponding field, and choose *Create*.

   To set up the destination configuration in SAP BTP, you need the inbound username, password, and the host URL (with port) of the inbound services.

Results

You've created the communication arrangement, the communication system, and an inbound user.

6.3.2.4.15.1.2 Create Destinations

In the SAP BTP cockpit, you need to create destinations to enable the communication to the ABAP environment. In addition, the destination must be enabled for use in SAP Build.

Procedure

1. In the SAP BTP cockpit, navigate to your subaccount. See Navigate to Orgs and Spaces.
2. Choose *Connectivity* > *Destinations.*.
3. Choose *New Destination* > *Blank Template*, and enter the following data:

   Process Visibility Capability

Field	Value
Name	Enter the destination name.
Type	Select *HTTP*.
Description	Enter a description.
URL	API-URL of the ABAP environment. The format is: https://*****.onde-mand.com/. Set the destination URL to the API endpoint URL found in the service key that was created for the service instance of SAP Build Process Automation and which was used creating the communication arrangement. Access the com-munication arrangement and copy the API URL displayed under *Common Data* > *API-URL*.
Proxy Type	*Internet*
Authentication	Select *BasicAuthentication*.
User	(Inbound) communication user used in the communication arrangement
Password	Set the password to the secret. Password of the (inbound) communication user used in the communication arrangement.
Additional Properties	Add the following properties: • “sap.applicationdevelopment.actions.enabled” → true • “sap.processautomation.enabled ” → true

4. Save your entries.
5. Open the Settings of SAP Build on your subaccount.
6. Choose *Destinations* > *New Destination*.
7. Select the destination that you have created in the SAP BTP cockpit and click *Add*.

Results

You have created a destination in your SAP BTP subaccount that can be used for the callback to the SAP BTP ABAP environment. The callback indicates the completion of a process on SAP Build Process Automation.

Related Information

Configure SAP Build Process Automation Destinations

6.3.2.4.15.2 AIF Monitoring

The Application Interface Framework (AIF) enables you to monitor background communication with your connected SAP Build Process Automation tenant.

Procedure

First, you need to assign business roles to a business user and then to assign recipients to this user.

6.3.2.4.15.2.1 Maintain Business Roles and Business Users

As a preparation to monitor messages for the integration of the SAP Build Process Automation, you assign business roles to a business user.

Procedure

1. Log on to the SAP Fiori launchpad as an administrator user.
2. With the *Maintain Business Roles* app, create and edit business roles and add business catalogs to the roles.
   a. To grant access to the *Assign Recipients to Users* app, add the business catalog *Communication Management – Message Monitoring Configuration* (SAP_CA_BC_COM_CONF_PC) to a business role.
   b. To grant access to the *Message Dashboard* app, add the business catalog *Business Network Integration* (SAP_BR_CONF_EXPERT_BUS_NET_INT) to a business role.

   Both business catalogs are also contained in the business role template *Configuration Expert – Business Network Integration* (SAP_BR_CONF_EXPERT_BUS_NET_INT).
3. With the *Maintain Business User* app, provide business users with access rights.

6.3.2.4.15.2.2 Monitor Messages for SAP Build Process Automation Integration

You can monitor the background processes for the integration of the SAP Build Process Automation.

Prerequisites

You have assigned business roles and business users.

Context

With the *Message Dashboard* or *Message Monitoring Overview*, you can monitor and administrate the SAP Build Process Automation background processing. You can manually trigger or cancel monitored background tasks that have issues.

AIF messages are deleted after a retention time, which can be configured in AIF customizing. The default value for all workflow-relevant namespaces and interfaces is set to 7 days.

Procedure

1. Log on to the SAP Fiori launchpad as an administrator user.
2. With the *Assign Recipients to Users* app, assign the corresponding recipients to a business user.
   a. Select the user for which you want to perform the assignment.
   b. Repeatedly choose *Assign* and successively enter the following recipients:

Namespace	Recipient
/SWFCP(for workflow capability)	*REC_PROC_BGRFC (process bgRFC)*
	REC_RAISE_EVT (Recipient for raise of event)
/SWFPV (for process visibility capability)	*EVENT_PUBL (Process bgRFC)*

   Also choose the message types that the user is allowed to view.

   For more information, see Assigning Users to Recipients.

3. To view the results, open the *Message Dashboard* app.
4. To view the triggered messages, use the *Calendar Monitor* to select the date range and choose *Search*.
5. Alternatively, view the results with the *Message Monitoring Overview* app.

   The overview displays the interfaces and message types to which you're subscribed. To see more details, click a message type.

Related Information

Blog on Working with the Message Dashboard

6.3.2.4.16 Integrating SAP Workflow Management

You can enable the communication between the ABAP environment and the workflow capability within the SAP Workflow Management service.

This scenario enables the ABAP environment applications to extend their business processes by using the workflow capability in the Cloud Foundry environment. It allows the provisioning of an API in the ABAP environment to start and control workflow instances running on workflow capability.

You set up the scenario using the following tasks:

1. Create a Communication Arrangement Using a Service Key [page 3371] (recommended)
2. Create a Destination [page 3372]
3. Maintain Business Roles and Business Users [page 3376]

6.3.2.4.16.1 Create a Communication Arrangement Using a Service Key

To connect the ABAP environment and the SAP BTP cockpit, you need a communication arrangement.

Procedure

1. Get the service key.
   a. In the SAP BTP cockpit, navigate to your space. See Navigate to Orgs and Spaces.
   b. From the navigation pane, choose *Services* > *Service Instances* > .
   c. Click the name of your workflow capability instance.
   d. From the navigation pane, choose *Service Keys*.
   e. Select one entry, and copy the service key.
2. Open the *Communication Arrangements* app in your ABAP environment.
3. Create a communication arrangement.
4. Choose scenario *SAP_COM_0542*.
5. Enter a name for the arrangement.
6. Select a communication user to use for inbound communication (from SAP Business Technology Platform to your ABAP environment).
7. Paste the service key into the corresponding field, and choose *Create*.

The password of the newly created user for the inbound communication is shown in an information message in the status bar. You need the user and password for the destination in theSAP Business Technology Platform.

Results

You've created the communication arrangement, the communication system, and an inbound user. The communication arrangement is assigned to the DEFAULT consumer type. The consumer type is used to identify the communication arrangement. Therefore, you must only assign it to exactly one communication arrangement. You can assign one communication arrangement to multiple consumer types.

Adjust the Consumer Type

Procedure

1. Select the communication arrangement, and choose *Edit*.
2. Set the consumer type to DEFAULT.

6.3.2.4.16.2 Create a Destination

In the SAP BTP cockpit, you need to create a destination to enable the communication to the ABAP environment.

Procedure

1. In the SAP BTP cockpit, navigate to your subaccount. See Navigate to Orgs and Spaces.
2. Choose *Connectivity* > *Destinations.*.
3. Choose *New Destination*, and enter the following data:

Field	Value
Name	Enter the destination name. This name is also used in the SAP Web IDE as the destination name in the service task properties.
Type	Select *HTTP*.
Description	Enter a description.

Field	Value
URL	URL for the ABAP environment. The format is: `https://*****.ondemand.com/`. Set the destination URL to the authorization endpoint URL found in the service key that was created for the workflow capability when creating the communication arrangement. Access the communication arrangement and copy the root URL under *Inbound Services* > *Service URL/Service Interface*.
Proxy Type	*Internet*
Authentication	Select *BasicAuthentication*.
User	Set the user to the client ID as specified for the inbound communication.
Password	Set the password to the secret.

4. Save your entries.

6.3.2.4.16.3 Set Scopes for the Service Instance Using the Command Line Interface

To set the necessary scopes of a service instance, you need to install a new service interface with all scopes needed or you update an already existing service interface.

Procedure

1. To update the workflow capability instance with the necessary scopes, save the following JSON file that specifies all necessary scopes:

```
{
    "authorities": [
        "WORKFLOW_INSTANCE_START",
        "WORKFLOW_DEFINITION_GET",
        "WORKFLOW_INSTANCE_GET",
        "WORKFLOW_INSTANCES_UPDATE",
        "WORKFLOW_INSTANCE_CANCEL",
        "WORKFLOW_INSTANCE_GET_ERROR_MESSAGES",
        "WORKFLOW_INSTANCE_GET_CONTEXT",
        "WORKFLOW_INSTANCE_GET_EXECUTION_LOGS",
        "MESSAGE_SEND",
        "TASK_GET"
    ]
}
```

   To set these scopes, see *Update an existing service instance* in Enable Technical Authentication.

2. Model the workflow as described in Developing Applications with Workflow Capability.

6.3.2.4.16.4 Model Service Task to Complete the Workflow in the ABAP Environment

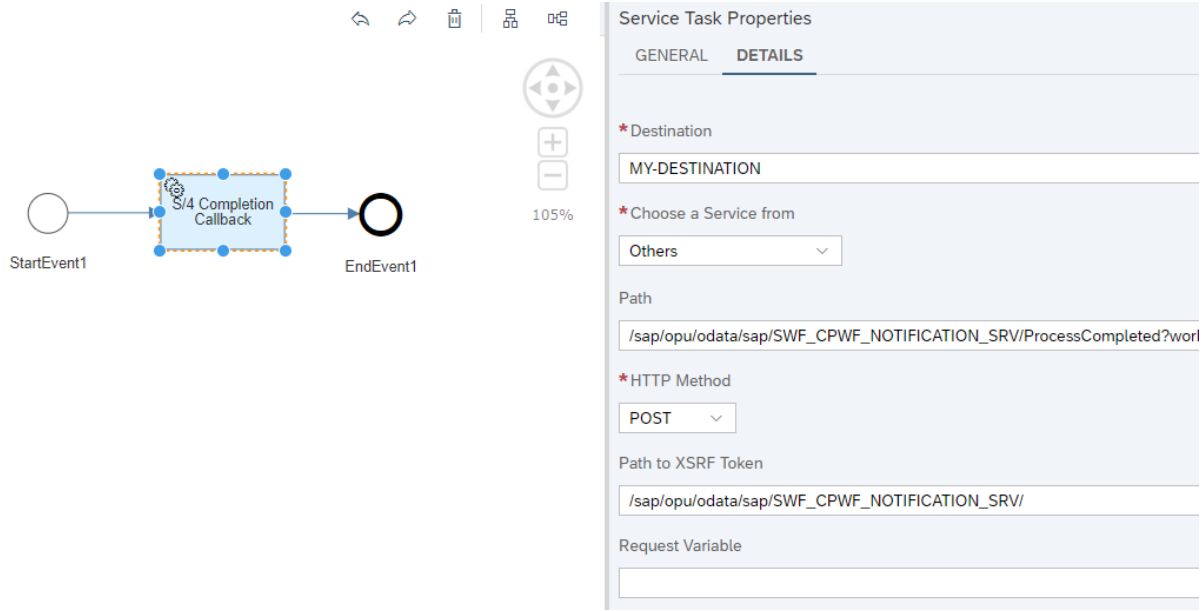
As the workflow capability doesn't support the enterprise messaging service, you must model an additional service task right before each end event for each workflow definition, that was started from the ABAP environment.

Prerequisites

You have modeled a workflow. See Modeling a Workflow.

Context

This task triggers the completion in the ABAP environment tenant and must be called exactly once.



With this approach, cancellations on workflow instances directly from the workflow capability API aren't communicated to the ABAP environment tenant.

Procedure

1. Open the workflow that you have modeled in the workflow editor.
2. Select the service task, and go to the *DETAILS* tab.

3. Enter the following data:

Field	Value
Destination	Enter the name of the destination you've created in Cloud Foundry.
Choose a Service from	*Others*
Path	`/sap/opu/odata/sap/SWF_CPWF_NOTIFICATION_SRV/ProcessCompleted?workflowInstanceId='${info.workflowInstanceId}'`
HTTP Method	*POST*
Path to XSRF Token	`/sap/opu/odata/sap/SWF_CPWF_NOTIFICATION_SRV/`

6.3.2.4.16.5 AIF Monitoring

AIF (Application Interface Framework) enables you to monitor inbound and outbound messages in the ABAP environment and allows you to analyze communication issues.

Procedure

First, you need to assign business roles to a business user and then to assign recipients to this user.

Related Information

Maintain Business Roles and Business Users [page 3376]
Monitor Messages for Workflow Capability Integration [page 3376]

6.3.2.4.16.5.1 Maintain Business Roles and Business Users

As a preparation to monitor messages for background workflow processes, you create a business role and a business user.

Procedure

1. Log on to the SAP Fiori launchpad as an administrator user.
2. With the *Maintain Business Roles* app, create and edit business roles and add business catalogs to the roles.
   a. To grant access to the *Assign Recipients to Users* app, add the business catalog *Communication Management – Message Monitoring Configuration* (SAP_CA_BC_COM_CONF_PC) to a business role.
   b. To grant access to the *Message Dashboard* app, add the business catalog *Business Network Integration* (SAP_CA_BC_COM_ERR_PC) to a business role.
3. With the *Maintain Business User* app, provide business users with access rights.

6.3.2.4.16.5.2 Monitor Messages for Workflow Capability Integration

You can monitor the background processes for workflow capability integration.

Prerequisites

You have assigned business roles and business users.

Context

With the *Message Dashboard* or *Message Monitoring Overview*, you can monitor and administrate the SAP Business Workflow background processing. You can manually trigger or cancel monitored background tasks that have issues.

AIF messages are deleted after a retention time, which can be configured in AIF customizing. The default value for all workflow-relevant namespaces and interfaces is set to 7 days.

Procedure

1. Log on to the SAP Fiori launchpad as an administrator user.
2. With the *Assign Recipients to Users* app, assign the corresponding interface to a business user.
   a. In the *Namespace* field, enter the namespace **`/SWFCP`** for the workflow capability integration.
   b. Select the interfaces using the recipient names:
      - REC_PROC_BGRFC (Process bgRFC)
      - REC_RAISE_EVT (Recipient for raise of event)
   c. Choose the message types the user is allowed to view:
      - Application Error or Technical Error
      - Info
      - Success
      - Warning
      - Application Error
      - Technical Error
      - None

   For more information, see Assigning Users to Recipients.
3. To view the results, open the *Message Dashboard* app.
4. To view the triggered messages, use the *Calendar Monitor* to select the date range and choose *Search*.
5. Alternatively, view the results with the *Message Monitoring Overview* app.

   The overview displays the interfaces and message types to which you're subscribed. To see more details, click a message type.

Related Information

How to enable users to work with the message dashboard

6.3.2.4.17 Integrating Business Event Logging

Context

There are two integration scenarios supporting data flow from Business Event Logging to the target systems:

- SAP_COM_0A54: This integration scenario is for data federation/extraction from Business Event Logging to target systems.
- SAP_COM_0532: This integration scenario is for data replication from Business Event Logging to target systems. You can use the schema `API_BusEvtLogEvent` from SAP_COM_A054 communication scenario.

For more information on this communication scenario, see Integrating SQL Services using SAP Datasphere.

6.3.2.4.17.1 Prerequisites

- Create a communication user in the Maintain Communication Users app. For more information, see How to Create Communication Users [page 2995].

  > **Note**
  >
  > Ensure that you use the same communication user for SAP_COM_A054 and SAP_COM_532 for replication scenario.

- Create a communication system in the Communication Systems app. For more information, see How to Create Communication Systems [page 3002].
- Create a communication arrangement in the Communication Arrangements app. For more information, see How to Create a Communication Arrangement [page 2998].

6.3.2.4.17.2 Consumption of Data

Data can be consumed using either External ODBC Clients or using SAP Datasphere:

Data Consumption using External ODBC Clients

For more information on data consumption, see Consumption of the SQL Service for Data Federation.

Data Consumption using SAP Datasphere

For more information on data consumption, see Consumption of the SQL Service for Data Replication.

6.3.3 Technical Operations

Learn more about what you can do to operate your ABAP environment.

As an administrator in the ABAP environment, you need metrics and alerts to monitor the availability and performance of your systems. You also need an indication whether your purchased service volume has been exhausted. Therefore, the following metrics and alerts are particularly relevant for the ABAP environment:

- Utilization metrics for computing, storage, and network resources
- Performance metrics
- Exceptional situations such as aborted jobs or runtime errors

In the ABAP environment, you have different tools available to support you with your tasks.

Local Monitoring

For local monitoring of your ABAP environment, you can use the technical monitoring cockpit. In the technical monitoring cockpit, multiple monitoring screens help you identify and solve bottlenecks and performance issues by giving you an in-depth view on important technical data of the database and application server, such as CPU, memory, network, storage, performance, time spent, workload, and configuration. You can use the technical monitoring cockpit also for a profound analysis of the most important entities, such as work processes, tables, SQL traces, and SQL statements.

For more information, see Technical Monitoring (Cloud).

System Landscape Monitoring

For monitoring an entire system landscape, consider using a central monitoring and alerting infrastructure. If you use SAP Focused Run or SAP Cloud ALM for monitoring your system landscape, you can use the integrated health monitoring for the ABAP environment.

For more information, see Central Health Monitoring Using SAP Focused Run and SAP Cloud ALM [page 3411].

6.3.3.1 Common Tasks: Quick Reference

Learn how you can perform typical tasks during technical operations using the available apps in the ABAP environment.

6.3.3.1.1 Analyzing the ABAP Resource Utilization of the ABAP Environment

Context

When you're planning, for example, system capacities or facing performance issues with SAP or partner cloud services running on the ABAP enviroment, you want to understand the resource utilization of your ABAP environment.

Procedure

1. On the SAP Fiori launchpad of your ABAP environment, search for *Application System Overview*.

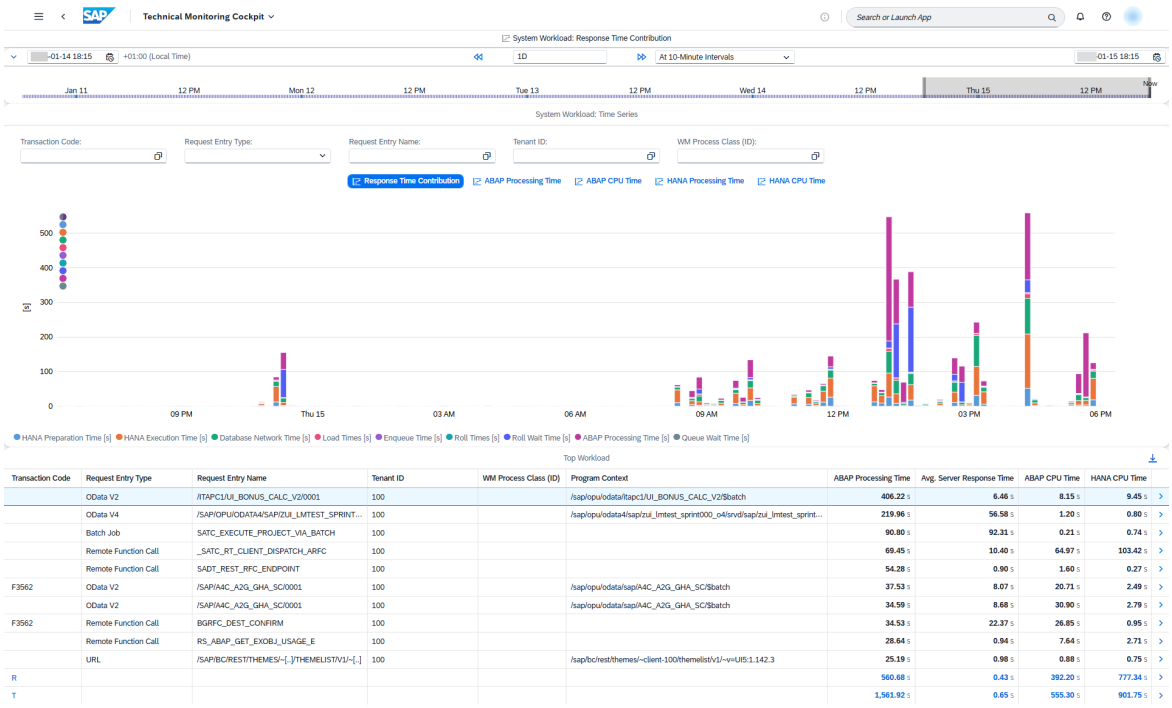
The technical monitoring cockpit opens.

2. To gain insights into the memory utilization of the ABAP application server, choose the *Memory Utilization* tile.
3. To gain further insights into the work process utilization of the ABAP application server, choose the *Sampled Work Process Data: Backgr. Work Process Utilization* and the *Sampled Work Process Data: Dialog Work Process Utilization* tiles.

   A high load on the background work processes isn't critical if it's only for a short duration. If the background work process utilization is at 100%, jobs are started delayed.

   For more information about the work process utilization screens, see Sampled Work Process Data: Background Work Process Utilization and Sampled Work Process Data: Dialog Work Process Utilization.
4. To gain more insights into the application workload of the system, choose the *System Workload* tile from the application system dashboard.

   Click on any peak in the chart to get the top 10 most contributing ABAP requests in the *System Workload: Details* section.



Transaction Code	Request Entry Type	Request Entry Name	Tenant ID	WM Process Class (ID)	Program Context	ABAP Processing Time	Avg. Server Response Time	ABAP CPU Time	HANA CPU Time
	OData V2	/ITAPC1/UI_BONUS_CALC_V2/0001	100		/sap/opu/odata/itapc1/UI_BONUS_CALC_V2/$batch	406.22 s	6.46 s	8.15 s	9.45 s
	OData V4	/SAP/OPU/ODATA4/SAP/ZUI_LMTEST_SPRINT...	100		/sap/opu/odata4/sap/zui_lmtest_sprint000_o4/srvd/sap/zui_lmtest_sprint...	219.96 s	56.58 s	1.20 s	0.80 s
	Batch Job	SATC_EXECUTE_PROJECT_VIA_BATCH	100			90.80 s	92.31 s	0.21 s	0.74 s
	Remote Function Call	_SATC_RT_CLIENT_DISPATCH_ARFC	100			69.45 s	10.40 s	64.97 s	103.42 s
	Remote Function Call	SADT_REST_RFC_ENDPOINT	100			54.28 s	0.90 s	1.60 s	0.27 s
F3562	OData V2	/SAP/A4C_A2G_GHA_SC/0001	100		/sap/opu/odata/sap/A4C_A2G_GHA_SC/$batch	37.53 s	8.07 s	20.71 s	2.49 s
	OData V2	/SAP/A4C_A2G_GHA_SC/0001	100		/sap/opu/odata/sap/A4C_A2G_GHA_SC/$batch	34.59 s	8.68 s	30.90 s	2.79 s
F3562	Remote Function Call	BGRFC_DEST_CONFIRM	100			34.53 s	22.37 s	26.85 s	0.95 s
	Remote Function Call	RS_ABAP_GET_EXOBJ_USAGE_E	100			28.64 s	0.94 s	7.64 s	2.71 s
	URL	/SAP/BC/REST/THEMES/~[..]/THEMELIST/V1/~[..]	100		/sap/bc/rest/themes/~client-100/themelist/v1/~v=UI5:1.142.3	25.19 s	0.98 s	0.88 s	0.75 s
R						560.68 s	0.43 s	392.20 s	777.34 s
T						1,561.92 s	0.65 s	555.30 s	901.75 s

For more information, see System Workload.

6.3.3.1.2 Analyzing the System Workload

You want to know more about the ABAP system workload.

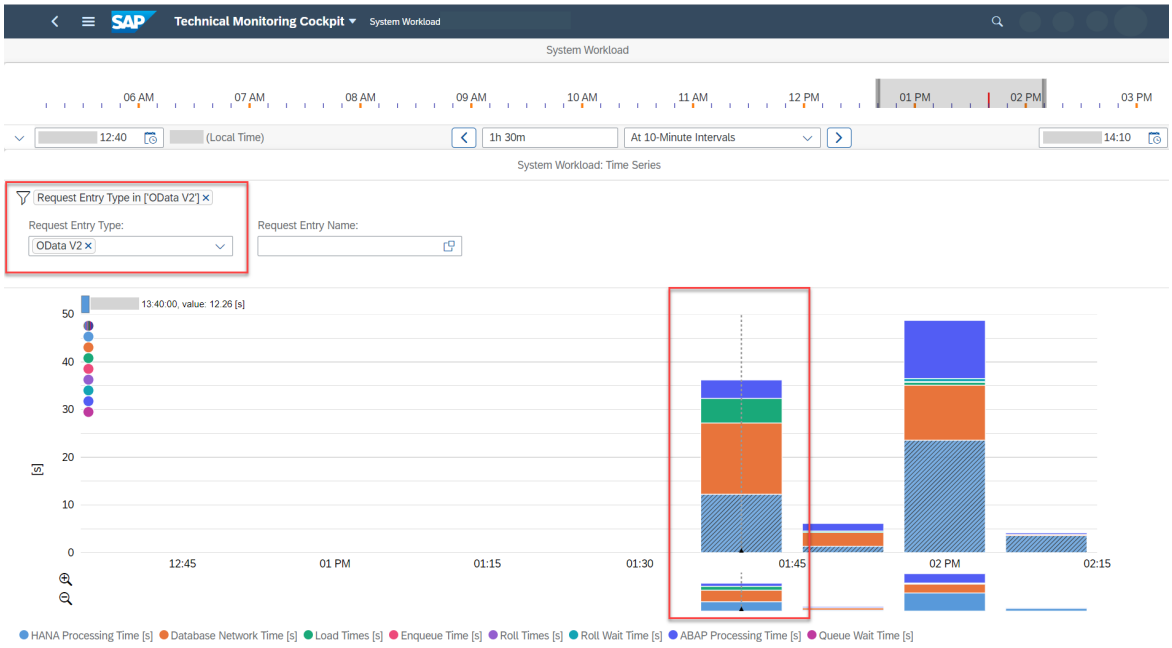
Context

Using the *System Workload* app, you benefit from a seamless connection between an overview of your system workload and single ABAP statistics records (main records and subrecords) down to SQL statements and their prepared plan. This gives you an excellent starting point for further performance analysis on SAP HANA level in the case of high database response times, which is true especially in the example below. Of course, you might also find records with a high ABAP CPU time where you don't need to further investigate the execution of SQL statements. In that case, a performance analysis on SAP HANA level would not be needed.

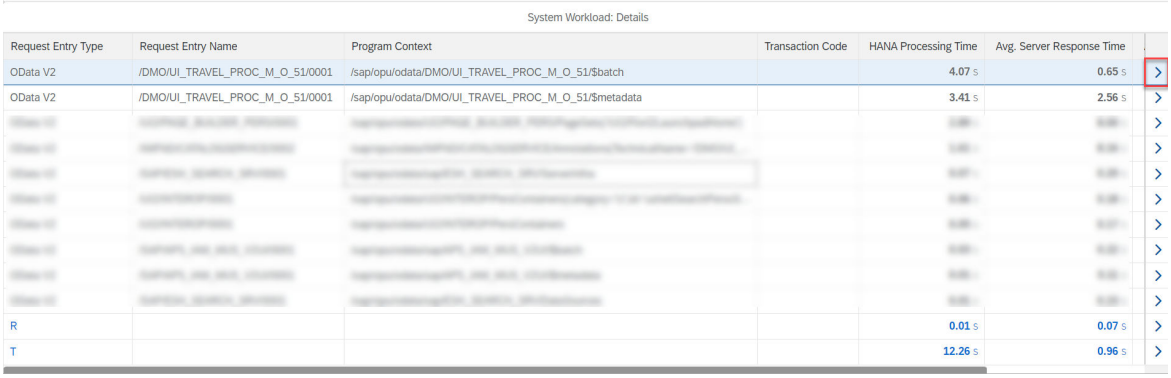
In the following example, let's analyze the usage of an application built according to the ABAP RESTful Application Programming Model (RAP). We're going to use the well-known demo application for flight booking, which is often used as a reference scenario in SAP contexts.

Procedure

1. On the SAP Fiori launchpad of your ABAP environment, search for the *System Workload* app and open it.
2. Let's assume you're interested in the behavior of the OData V2 services and set a filter on *Request Entry Type* for *OData V2*.
3. In the chart, click on the HANA processing time that seems high, for example, at 01:40 p.m.

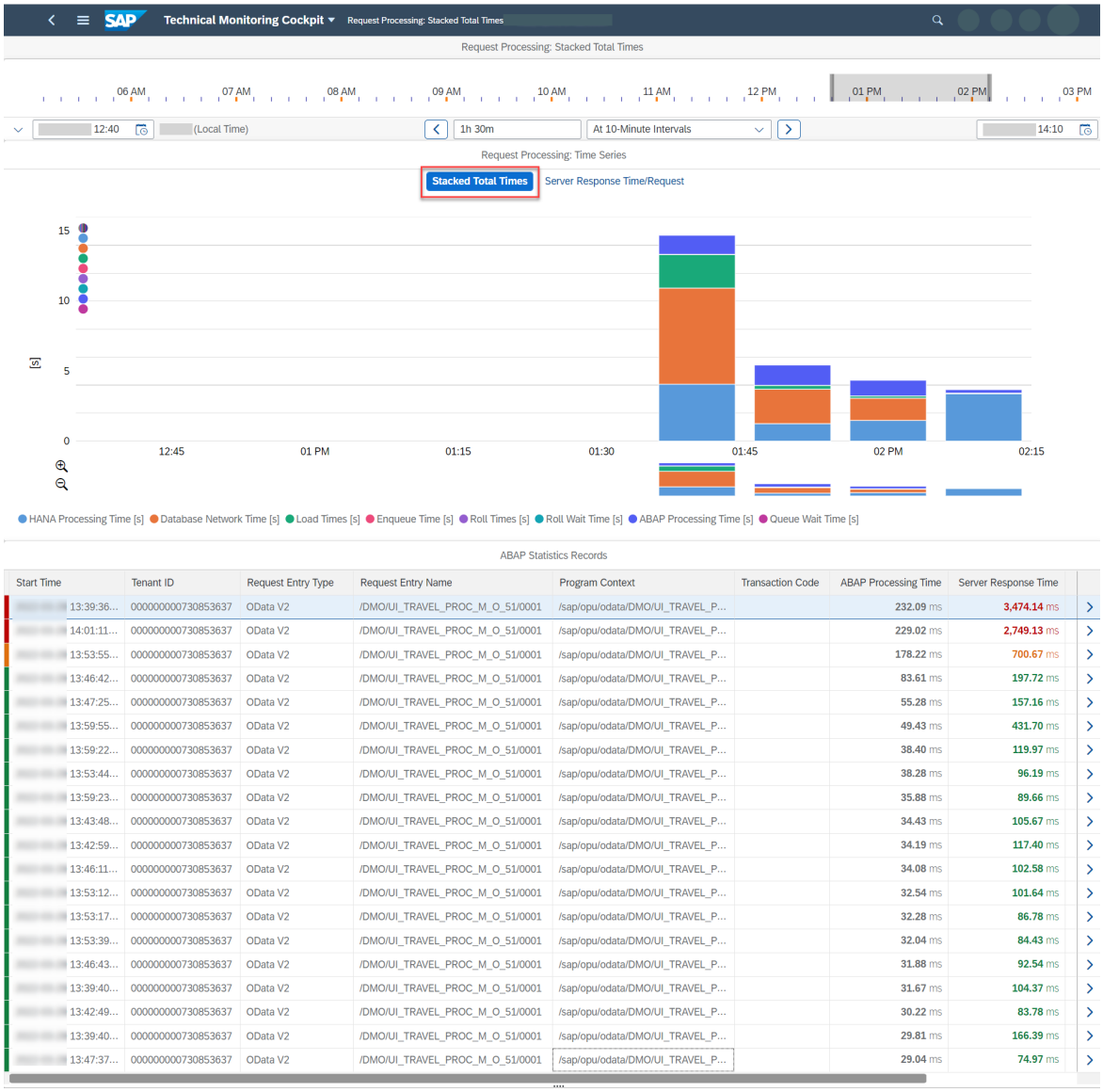


The *System Workload: Details* section below the chart changes accordingly and now displays the top contributors to the HANA processing time at the time you clicked in the chart. The contributors are grouped by request entry type and request entry name.


System Workload: Details

Request Entry Type	Request Entry Name	Program Context	Transaction Code	HANA Processing Time	Avg. Server Response Time
OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_PROC_M_O_51/$batch		4.07 s	0.65 s
OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_PROC_M_O_51/$metadata		3.41 s	2.56 s
R				0.01 s	0.07 s
T				12.26 s	0.96 s

4. To get more details, let's choose that first highlighted entry in the list. The subsequent *Request Processing* screen shows the behavior of this specific workload (the flight processing service) over time:

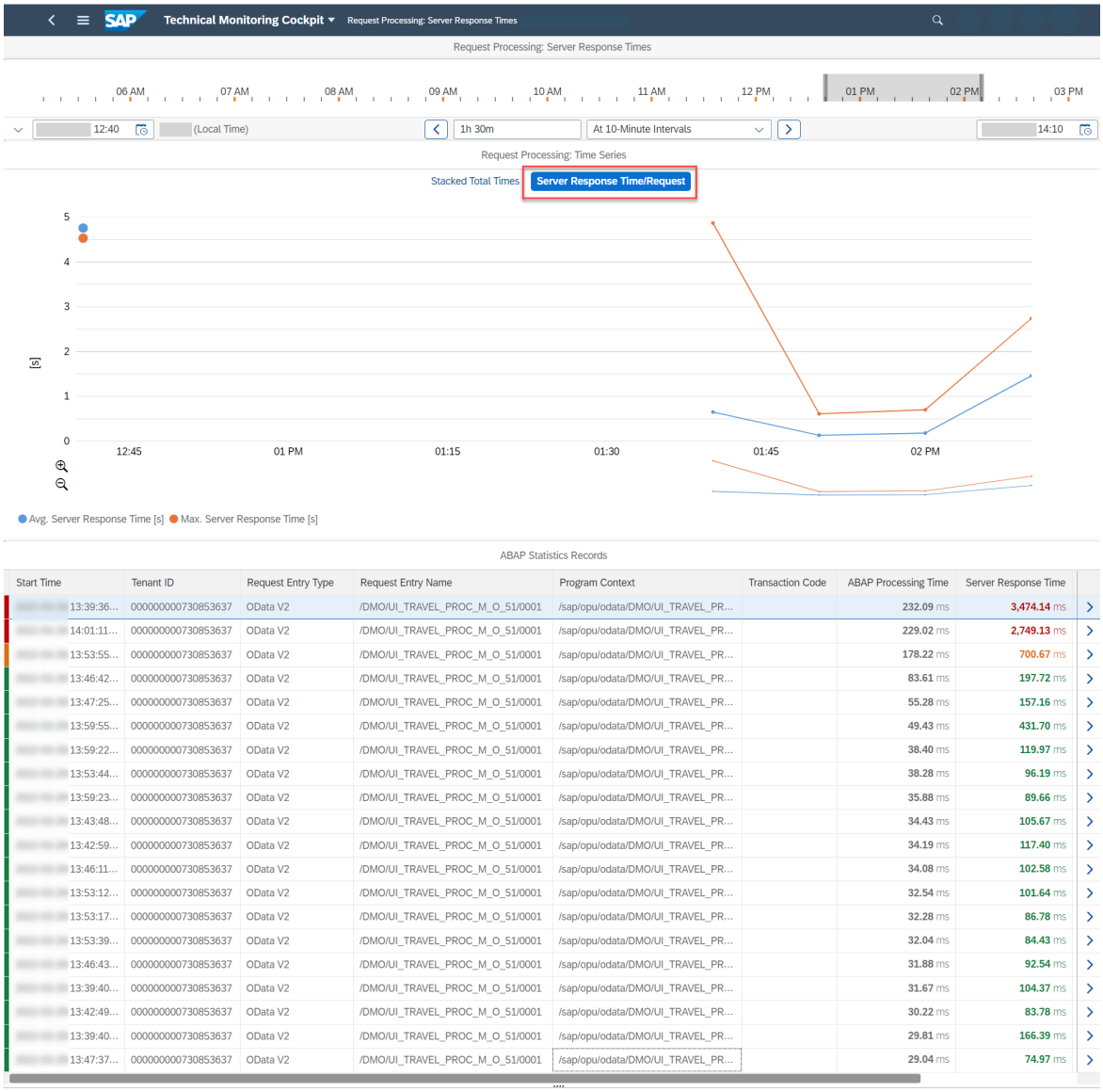


Start Time	Tenant ID	Request Entry Type	Request Entry Name	Program Context	Transaction Code	ABAP Processing Time	Server Response Time
13:39:36...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		232.09 ms	3,474.14 ms
14:01:11...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		229.02 ms	2,749.13 ms
13:53:55...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		178.22 ms	700.67 ms
13:46:42...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		83.61 ms	197.72 ms
13:47:25...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		55.28 ms	157.16 ms
13:59:55...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		49.43 ms	431.70 ms
13:59:22...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		38.40 ms	119.97 ms
13:53:44...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		38.28 ms	96.19 ms
13:59:23...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		35.88 ms	89.66 ms
13:43:48...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		34.43 ms	105.67 ms
13:42:59...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		34.19 ms	117.40 ms
13:46:11...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		34.08 ms	102.58 ms
13:53:12...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		32.54 ms	101.64 ms
13:53:17...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		32.28 ms	86.78 ms
13:53:39...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		32.04 ms	84.43 ms
13:46:43...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		31.88 ms	92.54 ms
13:39:40...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		31.67 ms	104.37 ms
13:42:49...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		30.22 ms	83.78 ms
13:39:40...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		29.81 ms	166.39 ms
13:47:37...	000000000730853637	OData V2	/DMO/UI_TRAVEL_PROC_M_O_51/0001	/sap/opu/odata/DMO/UI_TRAVEL_P...		29.04 ms	74.97 ms

The chart shows the stacked total times of the metrics, and - depending on what you click in the chart - the table below shows the relevant detail view, that is, a list of a maximum of 20 single ABAP statistics records for this workload sorted by the metric you chose in the chart. This is very helpful for your further analysis: You're coming from a workload perspective and now get the single records the workload consists of.

The color coding of the single records is based on their deviation from the average server response times (top to bottom): Red for very high, orange for above normal, and green for normal server response times.

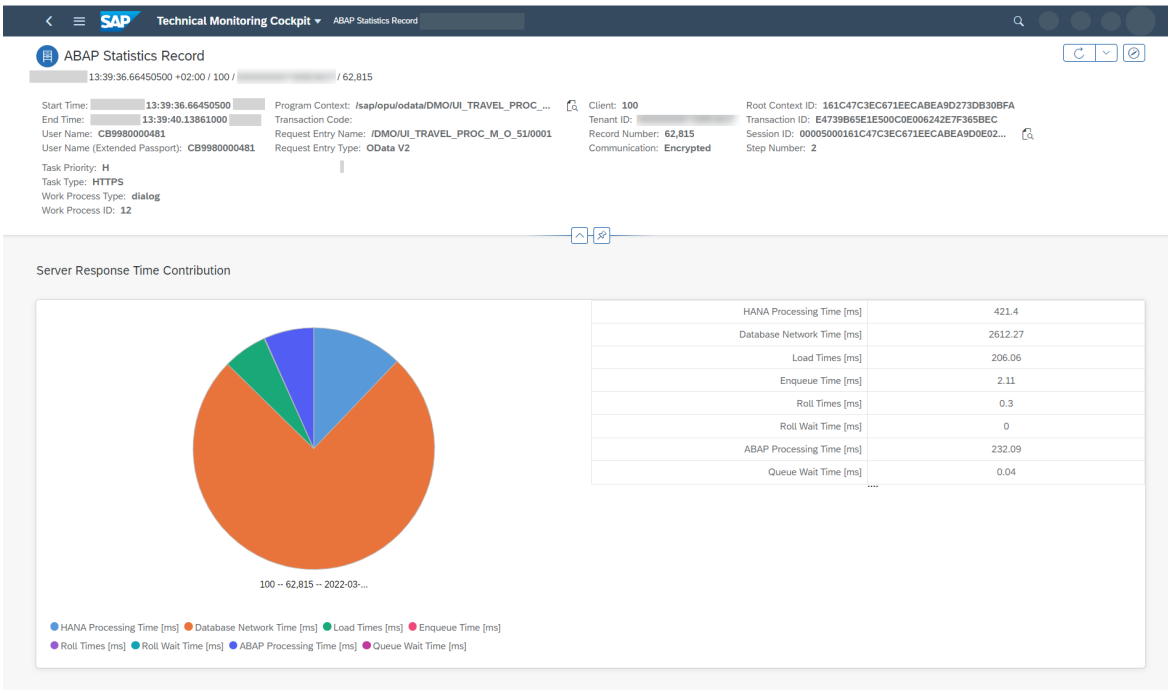
5. Let's switch views by choosing the *Server Response Time/Request* button:



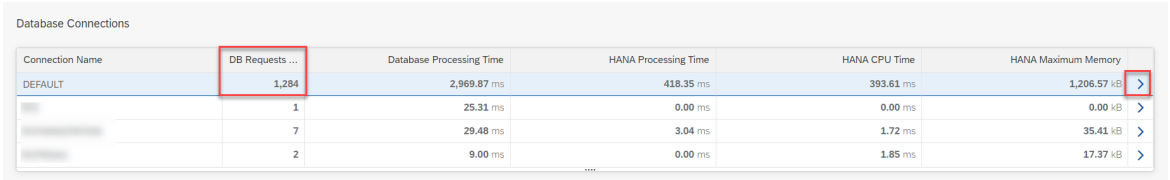
Like in the *Stacked Total Times* view, the color rating of the single records is based on their deviation from the average server response times. The average server response time per request is around one second here. However, there are some requests with higher times, so let's have a closer look at a "bad" request.

6. In the table, choose the first red entry (the one with 3,474.14 ms).

On the following *ABAP Statistics Record* screen, you get more detailed information about this record. The pie chart for the server response time contributions clearly shows that the response time is dominated by the database network time:

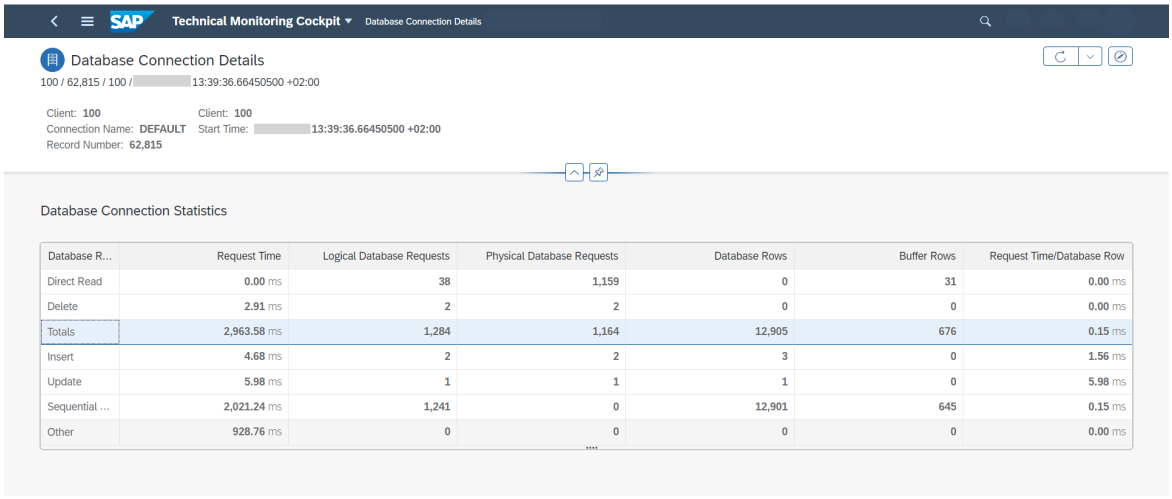


What's also noticeable is the DEFAULT database connection that apparently fired 1,284 database requests:



7. Let's dig deeper: In the *Database Connections* section of the *ABAP Staticstics Record* screen, choose the first entry (connection name DEFAULT) to get more information.

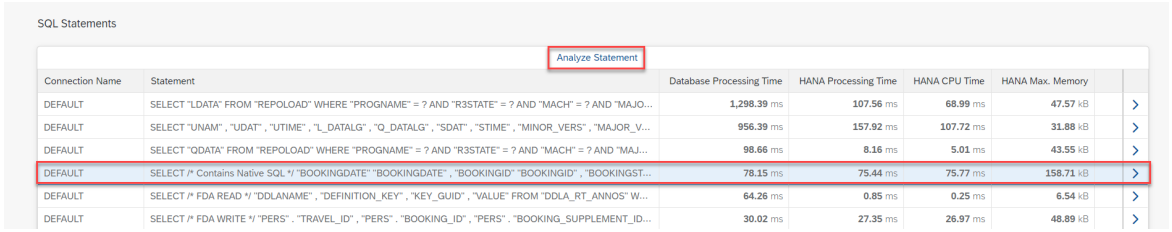
   The *Database Connection Details* screen opens and shows that 1,284 logical database requests were fired to fetch almost 13,000 database rows, which explains the high network time: It was caused by the many database requests.



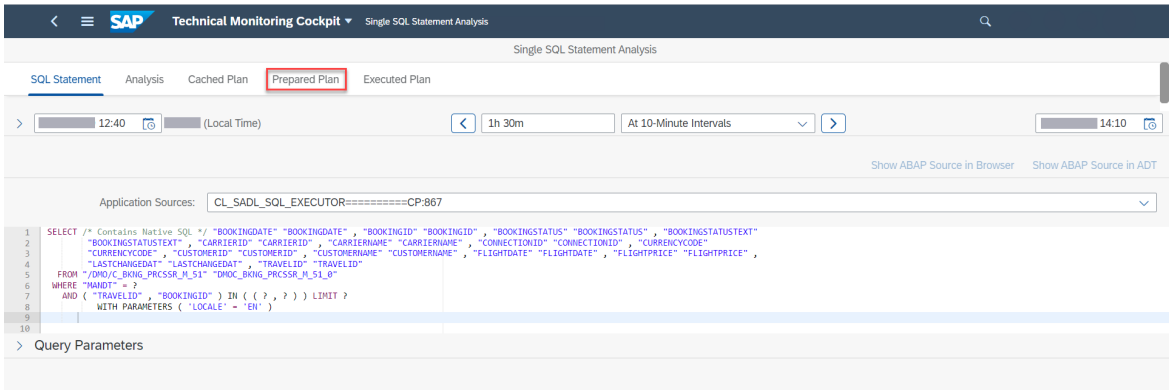
8. Another thing you may want to check in this example is the *SQL Statements* section on the *ABAP Statistics Record* screen: The table shows the top SQL statements contributing to the HANA processing time.

> **Note**
>
> The SQL statements are available in the single record by default. You do **not** need to activate any tracing explicitly.

Let's see if the access on CDS views is OK (in this case on the booking view). Place the cursor on the entry so that it's highlighted and simply choose *Analyze Statement* to resolve the SQL statement:



The *Single SQL Statement Analysis* screen opens and shows the SQL statement as well as its prepared access plans:

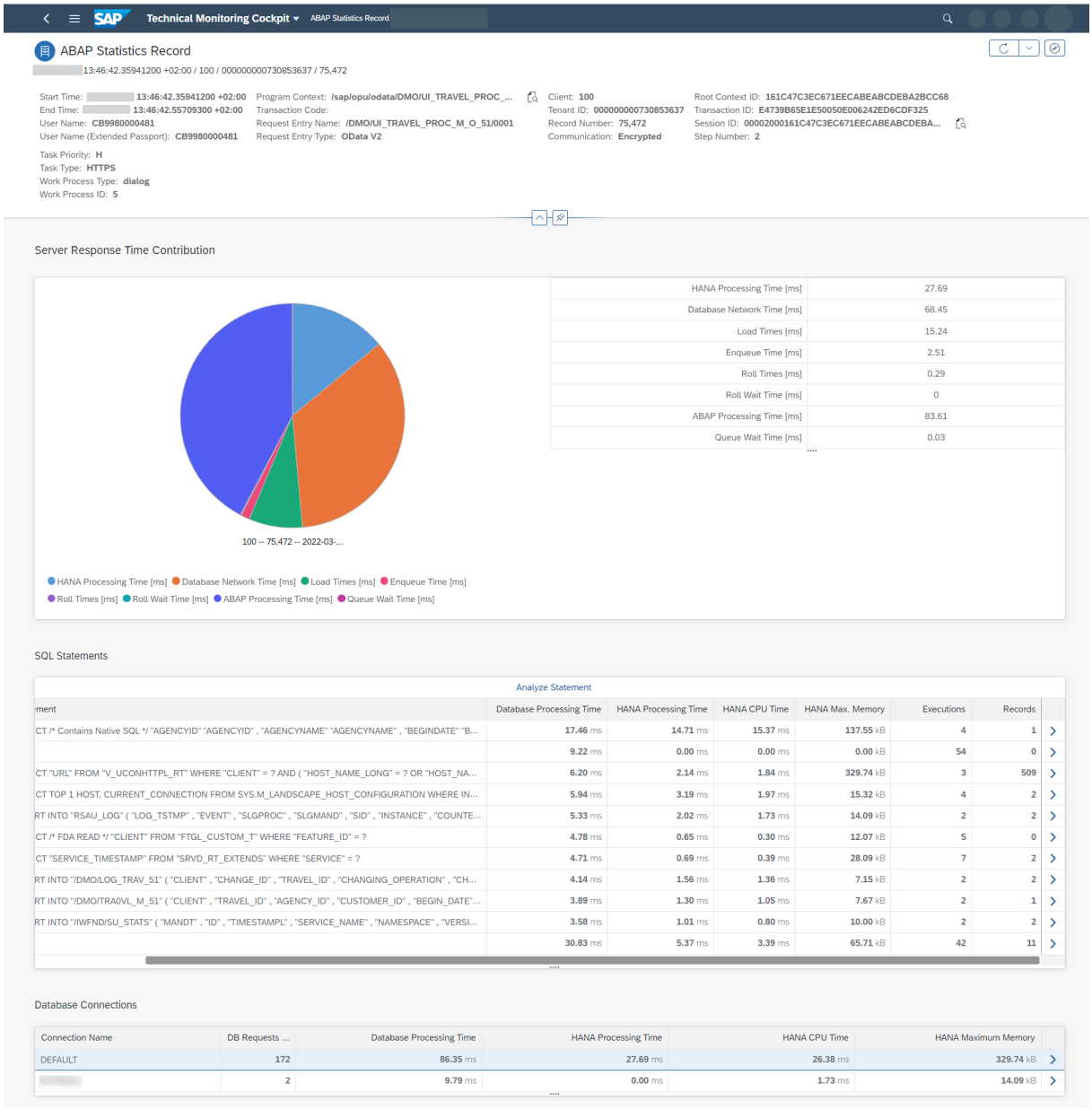


From the prepared plan, a HANA expert could now dig deeper and start analyzing the statement in detail.

Results

With the drilldown options to all kinds of details that are all in one place in the technical monitoring cockpit, you're able to identify several possible reasons for the high HANA processing time and network time.

Are you interested in a "good" example of this kind of workload for comparison, with a server response time per request of under one second? The *ABAP Statistics Record* screen in such a case might look like this:



Here, everything's fine: The server response time per request is only 0.197 seconds. The number of database requests is low, and therefore the database network time and processing times are low, too.

Related Information

https://blogs.sap.com/2023/04/24/analyzing-performance-degradations-in-the-abap-environment-in-the-cloud/

6.3.3.1.3 Capturing ABAP Statistics Records to Analyze System Activities

Capture ABAP statistics records to find out which activities are running in your ABAP and database system.

ABAP Request Statistics

By default, in the ABAP environment, requests are recorded with technical information, such as response time, program name, or CPU time. The resulting individual ABAP statistics records are written to the file system of the relevant tenant. ABAP statistics records are well suited to investigating the performance or resource consumption of a system or of a tenant in more detail, or, if performance problems occur, to identifying the area causing the problems.

Use Cases

You typically use ABAP statistics records in the following scenarios:

- You want to investigate your system workload to find out about top consumers and investigate in detail the overall response time contributions or the database connections. In this case, you can use the ABAP statistics records that are captured by default and visualized in the *System Workload* app.
  For more information, see Navigating from System Workload to Single ABAP Statistics Records.
- You want to analyze a specific workload, for example, the request statistics of an individual user or program name at a particular time.
  For more information, see Capturing Request Statistics [page 3389] and Analyzing Captured Request Statistics [page 3393].

Related Information

ABAP Statistics Records

6.3.3.1.3.1 Capturing Request Statistics

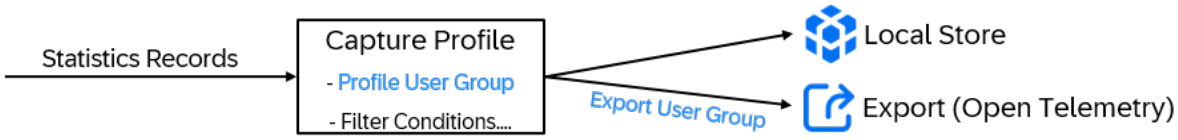
With the *Capture Request Statistics* app, you can capture request statistics to find out which activities are running in your ABAP system.

Context

With the activation of a capture profile, you can capture system activities. Requests are recorded with technical information, such as response time, program name, or CPU time. Using the request statistics, you can investigate the performance or the resource consumption of a system or of a tenant in more detail.

In the *Capture Request Statistics* app, you can also find predefined profiles for capturing expensive requests (with names starting with SAP_). These predefined capture profiles are active by default, and you can't edit or deactivate them.

With this app, you can store captured statistics records not only locally, but you can also export them. In a nutshell, this is the process flow:



> **Note**
>
> The *Profile User Group* affects the whole profile. A statistics record is only taken into account by a profile if its user group matches the profile user group.
>
> The *Export User Group* setting, however, only applies to what has already been defined by the profile user group and to what you want to export.

You can view the locally stored data from all profiles in the technical monitoring cockpit on the *Captured Request Statistics* and *Request Processing* screens. You can get to the *Request Processing* screens by choosing *System Workload* from the menu. In addition, the captured ABAP statistics records are the basis for ABAP system sizing in the *Perform System Sizing* app.

Procedure

1. On the SAP Fiori launchpad of your ABAP environment, search for the *Capture Request Statistics* app and open it.
2. To create a new capture profile, choose *Create*.
3. Enter the following data:

Field	What to Enter
Profile ID	Enter a short, meaningful name for your profile. Don't use the prefix SAP_ because this prefix is reserved for profiles predefined by SAP.
Description	If needed, you can enter a longer description of your profile.
Start/End Time	The start and end time specify when request statistics were captured by the app. These fields are automatically filled after you have activated the profile.
Profile User Group	Choose one of the profile user groups starting with *Customer*.
Profile Type	Choose *ABAP Statistics Records (Static)*. Only static profile types (with fixed values) are possible for ABAP statistics records.
Profile Owner	The profile owner is automatically set to *Customer* when you create a new profile. Predefined profiles have the profile owner *SAP*.
Health Monitoring	You can select the checkbox *Health Monitoring* for any capture profile that you create. As a result, the total number of locally stored ABAP statistics records for these profiles is shown in Health Monitoring of SAP Cloud ALM.

4. Choose *Create*.

   The profile is created, but is still inactive and has no filter conditions and local store or export settings yet.

5. To define the filter conditions under which request statistics are captured by this profile, choose *Create* in the *Conditions for ABAP Statistics Records* section.

6. With *Field ID*, *Operator*, *High*, and *Low*, you can select a filter condition that determines what kind of data is captured. For example, you might be interested in requests with a high ABAP CPU or network time. Alternatively, you can also select a program name to capture requests relating to that program, for example

   > ### Note
   >
   > If you don't need to define a range (lower and upper limit), but only a single value, enter the value in the *Low* field.

   For example, you can select *User ID* as field ID, *include equal to* as operator and your business user name to capture all request statistics relating to your business user.

7. Choose *Create*.

   A new filter condition has been created.

8. Depending on whether you want to store the captured activities locally or whether you want to export some or all captured activities to an exporter target, enter the relevant settings in the *Local Store* and in the *Export* section.

   > ### Note
   >
   > Via the sampling rate setting, you can activate or deactivate the local store.

Local Store section:

Field	What to Enter
Retention Time	Select how long you want to keep the captured request statistics stored in the system.
Record Limit per Minute	Enter the maximum number of records that you want the system to store per minute. The record limit prevents that too much database memory is consumed by captured request statistics. The maximum possible record limit per minute is 1,000. **Note** To get a realistic idea of the workload, consider a relatively high record limit. However, keep in mind that you generate load on your system while request statistics are captured.
Sampling Rate	Enter a sampling rate, which is the probability of storing one single ABAP statistics record in percent. With the sampling rate, you ensure that a number of random records are selected during the capturing of request statistics. If you don't want anything to go to the local store, simply enter 0,00 (zero) in this field.

Export section:

Field	What to Enter
Target Type and *Target Name*	Choose the target type and name where you want to export captured statistics records to.
Export User Group	Choose an export user group. This is a subset of the profile user group, which was defined in the app header. While the profile user group applies to the whole profile, the export user group only applies to what is already defined by the profile user group and to what should be exported.
Record Limit per Minute	Enter the maximum number of records that you want the system to export per minute.

Field	What to Enter
Sampling Rate	Enter a sampling rate, which is the probability of exporting one single ABAP statistics record in percent. With the sampling rate, you ensure that a number of random records are selected during the capturing of request statistics. If you don't want to connect to the telemetry exporter framework, leave this section empty. You can delete single targets here using the *Delete* button, or deactivate a target by setting its sampling rate to zero.

> **Note**
>
> The *Export* section is empty and cannot be filled if the relevant communication arrangements don't exist.

9. To start capturing request statistics during the time period that's defined in the profile, choose *Activate*.

   Make sure that you enter a start time that's right now or in the future. You can't activate profiles with a start time in the past.

Results

Request statistics are captured and processed by a collector that runs every minute. Therefore, you must expect that locally stored request statistics are displayed with a delay of a minute in the *System Workload* screens. Capturing request statistics is finished when the status of relevant capture profile has changed to *Finished*.

> **Note**
>
> If you have a long-running request, ABAP statistics records for this request will only show up in the technical monitoring cockpit after the request is done. The collector then captures the request and calculates its workload over its runtime.

6.3.3.1.3.2 Analyzing Captured Request Statistics

Get an overview of the request statistics that you have captured using the *Capture Request Statistics* app. View the server response time contribution or the database connections of a single request.

Context

With the *Capture Request Statistics* app, you can define conditions for which request statistics are collected in the system (see Capturing Request Statistics [page 3389]). You can view the captured data by choosing the *Analyze Request Statistics* button that leads you to the *Captured Request Statistics* screen.

> **Note**
>
> Before ABAP statistics records are shown, they are captured and processed by a collector that runs every minute, so you must expect that captured request statistics are displayed with a delay.

Procedure

1. In the *Capture Request Statistics* app, choose a profile for which you want to analyze the captured data.
2. Choose *Analyze Captured Request Statistics*.

   The *Captured Request Statistics* screen opens. The captured request statistics shown are filtered; only the results for the selected profile are shown.
3. Analyze the data for different aspects like identifying expensive requests and their context (program name, transaction code, or path of an HTTP request), so that you can find the area causing performance issues or resource shortages.

   For more information, see Captured Request Statistics.

6.3.3.1.4 Monitoring Expensive Outbound Communication

The *Capture Request Statistics* and the *System Outbound Communcation* apps, optionally in combination with SAP Cloud ALM, help you to identify service requests that cause expensive outbound communication (RFC, HTTP, and Web service calls).

Use Cases

You might want to find out what the requests with the most expensive system outbound communication are:

- Which communication arrangements have a long calling time?

- How is the calling time distributed among communication scenarios and how does it behave over time?
- Are there any outliers?

After you have found answers to these questions, you might want to dig deeper into the details of one particular communication arrangement.

Alternatively, you might already have a good idea which communication arrangements are associated with a longer calling time. For example, you notice that one of your customer applications does not run smoothly because the related system communication takes a long time.

In both cases, you want to find out the root cause for the long calling times. Therefore, you might want a heads-up when the long calling times occur. In addition, you also need to find the affected service requests and you want to collect their ABAP statistics records.

In these cases, a combined use of the technical monitoring cockpit, the *Capture Request Statistics* app and SAP Cloud ALM is beneficial.

Process Flow

1. If you want to get an overview of the system outbound communication first, use the technical monitoring cockpit to find expensive service requests and their related communication arrangements (see also Monitoring System Outbound Communication in General [page 3395]).
   In the technical monitoring cockpit, the top 10 communication scenarios by calling time are shown, which helps you identify obvious expensive outbound communication. If you have identified a particular communication arrangement, you can now get more detailed data.
   However, sometimes the issues during outbound communication occur too infrequently, so you might miss them with only an occasional check. It might also happen that critical service requests do not appear in the top 10 communication scenarios by calling time, or they do, but the related detailed statistics are not part of the samples that are collected by default for the technical monitoring cockpit. Therefore, you now turn to the *Capture Request Statistics* app to collect more specific, user-defined data.
2. To collect ABAP statistics records relating to expensive outbound communication, for example, for a specific communication arrangement or for particularly long calling times, define a profile in the *Capture Request Statistics* app. Make sure that the *Health Monitoring* checkbox is selected in your profile (see Capturing Request Statistics Relating to Expensive Outbound Communication [page 3397]).
3. In Health Monitoring in SAP Cloud ALM, use the metric *Captured Request Statistics* to find out whether service requests exceed a defined threshold in their calling time (see Getting Alerted About Expensive Outbound Communication Using SAP Cloud ALM [page 3399]). You can also use the settings in SAP Cloud ALM to get alerted with an e-mail notification.
4. After you have received a notification, use the *Analyze Request Statistics* button in the *Capture Request Statistics* app which leads you to the *Captured Request Statistics* screen where you can analyze the captured data for outbound communication (see Analyzing Expensive Outbound Communication Using the System Outbound Communication App [page 3401]).

6.3.3.1.4.1 Monitoring System Outbound Communication in General

Get an overview of the system outbound communication over time.

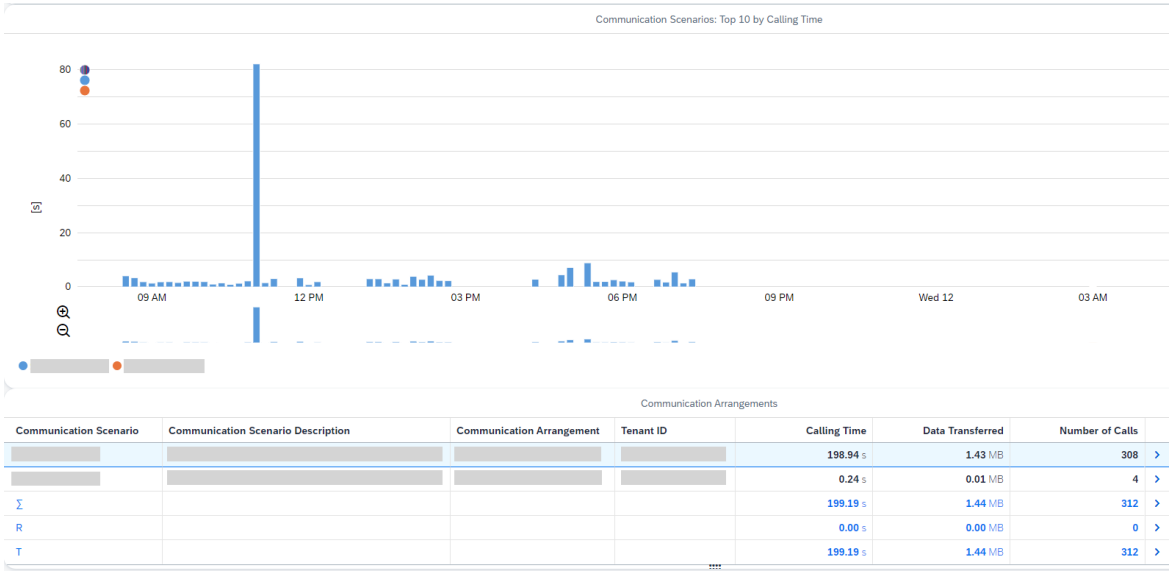
Context

In the *System Outbound Communication* app, the top 10 communication scenarios by calling time are shown, which helps you identify obvious expensive outbound communication. The focus is on outbound communication of type RFC, HTTP, and Web service.

Procedure

1. On the SAP Fiori launchpad of your ABAP environment, search for the *Application System Overview* app and open it.
2. On the dashboard shown, choose the *System Outbound Communication* tile.

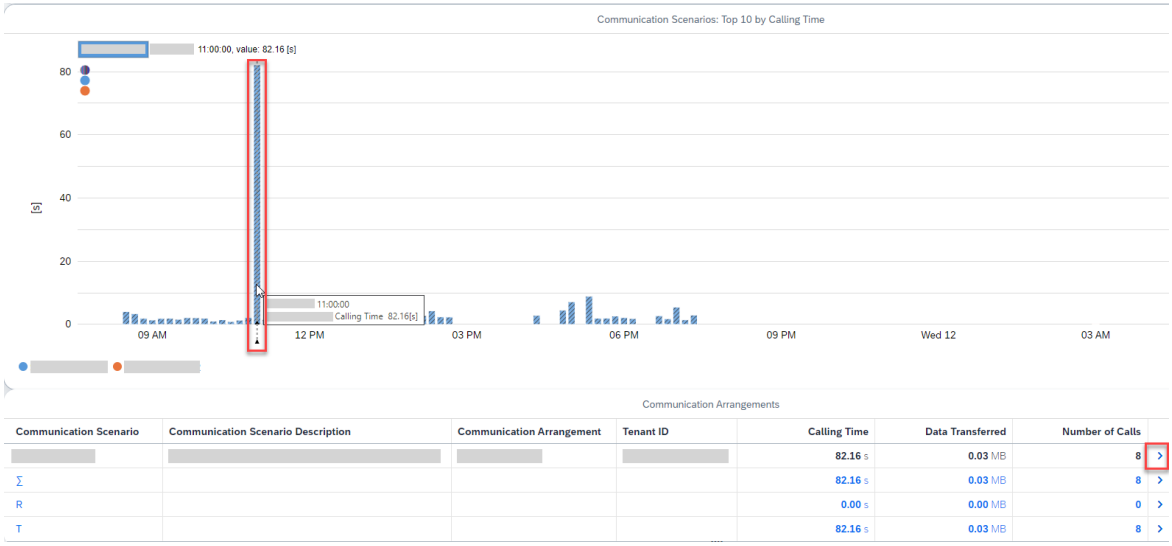
   The top 10 communication scenarios by calling time are shown, including the related communication arrangements. In the chart, the development over time is shown. Only customer workload using customer communication arrangements is displayed; SAP workload and SAP communication arrangements are filtered out.



3. In the chart, you can also click on a communication scenario at a particular time.

   As a result, the table of communication arrangements is updated with the single communication arrangements at this particular time.

   In this rather quiet test system, workload is low, but one communication scenario before 12 p.m. in the chart stands out with a long calling time.

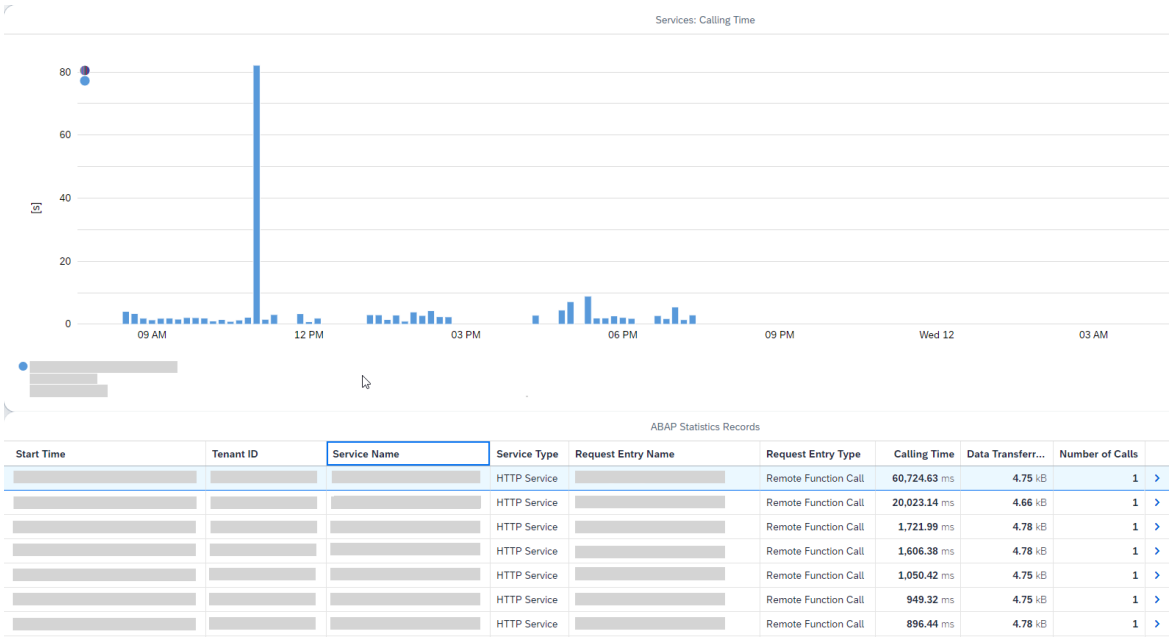


4. To analyze such a specific communication arrangement in more detail, choose its entry from the list of communication arrangements.

   The *Communication Arrangement* screen opens, where the selected communication arrangement with its related outbound services is shown. The chart indicates how the outbound services perform over time. You can now check whether calling time is distributed evenly or whether there are any outliers.

   In the list, you can find the single ABAP statistics records for the top outbound-calling services. The records include the calling time, the amount of transferred data and the number of calls.

   Again, in the example shown here, there's an outlier among the services that merits further investigation.



5. To show more details of a single ABAP statistics record, choose it from the list.

   As a result, you navigate to the *ABAP Statistics Record* screen, where more details, such as the server response time and multiple subrecords are shown.

6. To see the details of the HTTP subrecord, choose *HTTP*.

Here, you can find the single outbound HTTP client calls that were made during the service request.

7. To drill down to even more details, choose an entry from the table of HTTP client calls.

   You now get the HTTP client call details, including its calling time, and also the related communication arrangement.

Results

The *System Outbound Communication* screen and its related drill-down screens are configured by default in such a way that they provide sample records of expensive system outbound communication. If you want to get a full set of ABAP statistics records for a particular communication arrangement with a long calling time, use the *Capture Request Statistics* app. With this app, you can capture request statistics relating to defined expensive outbound communication arrangements.

Related Information

Capturing Request Statistics Relating to Expensive Outbound Communication [page 3397]
System Outbound Communication

6.3.3.1.4.2 Capturing Request Statistics Relating to Expensive Outbound Communication

Use the *Capture Request Statistics* app to collect statistics of service requests with an expensive outbound communication (RFC, HTTP, or Web service).

Prerequisites

You need a system administrator role (based on `SAP_BR_ADMINISTRATOR`) to perform these tasks.

Context

With the *Capture Request Statistics* app, you can capture single ABAP statistics records of dedicated workloads in your ABAP system (see also Capturing Request Statistics [page 3389]). You can also use this app for monitoring expensive outbound communication. For this purpose, you configure a capture profile to monitor the calling time of outbound calls of type HTTP, RFC, and Web service that exceed a defined threshold. In this procedure, you can capture records related to a high calling time. Alternatively, you might already have an idea which communication arrangements are related to expensive outbound communication, and you would like to find out the corresponding service requests.

With the checkbox *Health Monitoring* selected, you can also automatically export the number of captured ABAP statistics records of the profile to Health Monitoring in SAP Cloud ALM so that you get alerted when expensive outbound communication occurs.

Procedure

1. On the SAP Fiori launchpad of your ABAP environment, search for the *Capture Request Statistics* app.
2. Choose *Create* and enter the basic header information for the profile.

   If you want to export the number of capture request statistics to SAP Cloud ALM, select the *Health Monitoring* checkbox.

   For more information about how to fill out the individual fields in the header, see the built-in SAP Companion documentation.
3. To define a filter for high outbound calling times, choose *Create* and enter the following data:

Option	Description
Field ID	Choose *Calling Time [ms]* from the dropdown list.
Operator	Select *Include greater than or equal to*. **Note** When you enter a metric in the *Low* field, it doesn't matter which operator you choose here because it's always interpreted as threshold. The operator is therefore always interpreted as *Include greater than or equal to*.
Low	Enter a threshold for the calling time, for example, 1000. As a result, data for all service requests is captured whose calling time exceed 1,000 ms.
High	Leave this field empty.

4. Choose *Create* to save the filter and go back to the header data.

   If you don't want to filter for any specific communication arrangement, you can stop defining filters here and activate your capture profile. The app will then capture any service request that makes at least one outbound RFC, HTTP, or Web consumer call and whose **total** calling time for all calls of each type exceeds the defined threshold.
5. To define a filter for a specific communication arrangement, choose *Create* and enter the following data:

Option	Description
Field ID	Select *Communication Arrangement* from the dropdown list.
Operator	Select *Include equal to*.
Low	From the dropdown list, select the communication arrangement that you want to monitor.

Option	Description
High	Leave this field empty.

6. Choose *Create* to save the filter and go back to the header data.

   Optionally, you can define more filters for additional communication arrangements.

   If you have also defined filters for communication arrangements, the *Capture Request Statistics* app will then only consider the service requests for the communication arrangements in the filters. In addition, data for a request using one of these communication arrangements is only captured if the total of at least one of the request's outbound RFC, HTTP, or Web consumer calls exceeds the defined threshold for the calling time.
7. Activate the capture profile.

6.3.3.1.4.3 Getting Alerted About Expensive Outbound Communication Using SAP Cloud ALM

If you use SAP Cloud ALM to monitor your applications in a Cloud system landscape, you can also use health metrics to get alerted about expensive outbound communication.

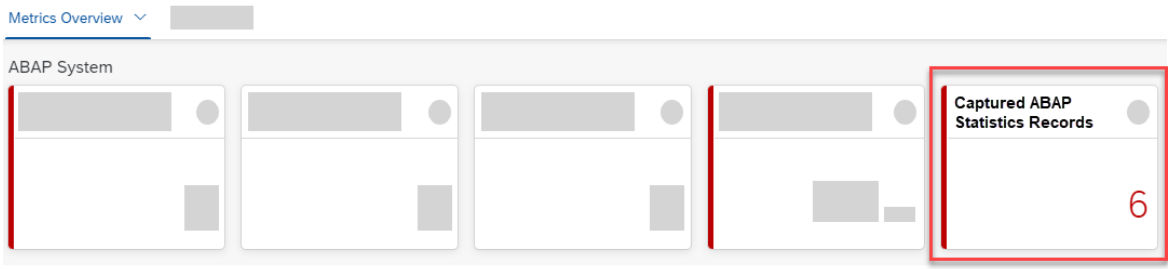
Prerequisites

You have SAP Cloud ALM set up (see Central Health Monitoring Using SAP Focused Run and SAP Cloud ALM [page 3411]).

You have captured request statistics relating to outbound communication using the *Capture Request Statistics* app, and you have selected the checkbox *Health Monitoring* in the relevant capture profiles (see Capturing Request Statistics Relating to Expensive Outbound Communication [page 3397]).

Procedure

1. Log on to SAP Cloud ALM and call up Health Monitoring.
2. On the Health Monitoring dashboard for your ABAP system, check the metric *Captured ABAP Statistics Records*.



The number of captured ABAP statistics records that is shown in SAP Cloud ALM is the total number of captured records for profiles with the checkbox *Health Monitoring* selected.

3. If the number of records is above zero, you have a clear indication that there was expensive outbound communication in the ABAP system during the last 5 minutes.
4. Choose the *Captured ABAP Statistics Records* tile and then *History*.

   On the *History* popup, you can find out how many records were captured by the profile that you've chosen and how the number of records changed over time.

> **Note**
>
> A high number of captured ABAP statistics records with a high calling time are an indication that the performance of outbound communication gets critical. Use the settings in SAP Cloud ALM if you want to get notified with an e-mail as soon as a critical number of ABAP statistics records have been captured.

SAP Cloud ALM Notification

Health Monitoring Event

Critical Workload Records Captured

Service Name	
Service Type	SAP BTP ABAP Environment
Object Details	
Reported at	11-Oct- 11:13:44
Rating	Critical

Additional Information

Name	Profile	Timestamp	Rating	Value
Number of Captured Requests		11-Oct- 11:10:00	Critical	6.0

Click here to view this alert

Click here to view the service/system in Health Monitoring

This is an automated email, please do not reply.

If you do not want to receive further notifications from SAP Cloud ALM, you can click to unsubscribe

Let's Run Better Together!

Your SAP Cloud ALM Team

Results

You can now go to the *System Outbound Communication* app to analyze the expensive outbound communication in more detail.

Related Information

Analyzing Expensive Outbound Communication Using the System Outbound Communication App [page 3401]

6.3.3.1.4.4 Analyzing Expensive Outbound Communication Using the System Outbound Communication App

With the *System Outbound Communication* app, you can analyze expensive outbound communication in detail.

Prerequisites

You have set up a profile to capture request statistics relating to outbound communication using the *Capture Request Statistics* app (see Capturing Request Statistics Relating to Expensive Outbound Communication [page 3397]).

Procedure

1. On the SAP Fiori launchpad of your ABAP environment, search for the *Capture Request Statistics* app.
2. From the list of capture profiles, choose the profile that you have defined to collect ABAP statistics records for expensive system outbound communication.
3. Choose the *Analyze Request Statistics* button.

   On the *Captured Request Statistics* screen, you can find all workloads that were captured using your profile in the *Capture Request Statistics* app. You can now investigate in more detail which service requests were responsible for expensive outbound communication and what details their ABAP statistics records contain, such as the related outbound HTTP, RFC, or Web service calls.

Related Information

ABAP Statistics Records

6.3.3.1.5 Analyzing the Performance of SQL Statements Running on Your ABAP Environment

Learn how to use SQL statement and trace analysis in the ABAP environment.

Data retrieval and modification through SQL statements are the main functions of the SAP HANA database in your ABAP environment. When you develop new applications or operate them productively, you need to analyze the performance of SQL statements executed by your applications on the database.

When analyzing statement performance, you typically encounter these scenarios:

- You're focused solely on the performance of SQL statements within your application. You don't need to consider other SQL activities on your database.
  - To trace SQL statements, you have the following options:
    You may prefer the lightweight option integrated into ABAP Development Tools for Eclipse (ADT) if you are in a developer role. For more information, see Tracing SQL Statements for Developers [page 3404].
    Administrators can use an option with more functions available on the Fiori launchpad. For more information, see Tracing SQL Activities for Administrators [page 3402].
  - To deepen your performance analysis of SQL statements, you can obtain HANA executed plans in two ways:
    You can switch on the HANA plan trace for the statement hash of a specific SQL statement. For more information, see Tracing HANA Executed Plans Using the Capture Request Statistics App [page 3406].
    You can also use ABAP Development Tools for Eclipse (ADT) to get HANA executed plans of SQL statements issued by SAP Fiori apps. For more information, see Tracing HANA Executed Plans Using the ABAP Cross Trace in ADT [page 3408].
- You want to view the performance of SQL statements along with other SQL activities on your database. Start with the `SQL Statement Analysis` app that you can find on the SAP Fiori Launchpad. For more information, see SQL Statement Analysis.

6.3.3.1.5.1 Tracing SQL Activities for Administrators

With the *Capture Request Statistics* app, you can activate and capture SQL traces for a later analysis.

Context

The *Capture Request Statistics* app allows you to trace the SQL activity of your ABAP environment from the SAP Fiori Launchpad if you're in the role of an administrator (business role template `SAP_BR_ADMINISTRATOR`).

To run an SQL trace, you create a profile of type *SQL Trace* and activate it (the app does not offer predefined SQL trace profiles). All SQL activities of your ABAP environment that fulfill the filter conditions defined in the profile are then recorded until you deactivate the profile. You can view the trace results on the *SQL Trace Analysis* screen in the technical monitoring cockpit.

Procedure

1. On the SAP Fiori launchpad of your ABAP environment, search for the *Capture Request Statistics* app and open it.

   On the first page of the app, you'll find a list of existing profiles. The relevant profiles for your use case are capture profiles of type *SQL Trace*.
2. To create a new capture profile, choose *Create*.
3. Enter the following data in the header section that opens:

Field	What to Enter
Profile ID	Enter a short, meaningful name for your profile. Don't use the prefix SAP_ because this prefix is reserved for profiles predefined by SAP.
Description	If needed, you can enter a longer description of your profile.
Target User Group	Choose one of the target groups starting with *Customer*. **Note** You can only define user filters on users that match the target user group of the profile.
Profile Type	Choose *SQL Trace*.

4. To confirm your input, choose *Create*.

   Now the profile is created, but is still inactive and has no filter conditions yet apart from a filter on your own user name, which you can keep or remove according to your preferences.
5. To define filter conditions under which SQL statements are captured by the profile, choose *Create* in the *Conditions for SQL Trace* section.
6. In the dialogue that opens, choose a *Field ID* from the dropdown list, such as *CDS View or Table*, *Program Name*, or *User Name*. Also choose an *Operator* (for example, *include equal to*), and in the *Low* field, enter your specific filter value.

   Note

   You can set almost all conditions only with the *include equal to* operator. Only for the field *CDS View or Table* can you choose between *include* and *exclude equal to*.

   There's a limit for the number of operators:

   For *CDS View or Table*, you can enter up to **five** *include equal to* and up to five *exclude equal to* conditions. Such a combination results in a hit if one of the *include* and one of the *exclude equal to* conditions match. It's possible to use a wildcard (*).

   For *Program Name*, *RFC Function Name*, and *Transaction Name*, you can only set one condition per profile.
7. Choose *Create*.

   A new filter condition has been created. If you want to set more filter conditions for the profile, repeat steps 5 to 7.

8. To finally start the SQL trace, choose *Activate*.

   The following popup tells you whether another user is currently running an SQL trace in the system. If so, you cannot start a new one because it's only possible to run one trace at a time. The running trace must first be stopped before you can start a new one.

   > **Note**
   >
   > You cannot stop a running trace if it was started on a different tenant. You can only stop running traces of users of your own user group.

9. To stop SQL tracing, choose *Deactivate*.
10. To view and analyze the captured SQL statements for your profile, choose *View Trace Directory*.

    This leads you to the *SQL Trace Analysis* in the technical monitoring cockpit. If you activate an SQL trace profile that you've previously run, you'll trigger a new trace run, which is then available again on the *SQL Trace Analysis* screen as a new set of trace records.

6.3.3.1.5.2 Tracing SQL Statements for Developers

Context

> **Note**
>
> The following procedure is for developers. If you have the role of an administrator, see Tracing SQL Activities for Administrators [page 3402].

During developing and operating applications on your ABAP environment, these situations are typical when you take an SQL trace of your ABAP application and turn to the SQL trace analysis:

- You want to analyze the performance of the SQL statements of your ABAP application in the sequence as they run.
- When analyzing the performance of an ABAP application, you want to clarify whether the SQL activity of the application is a relevant contribution to the processing time.

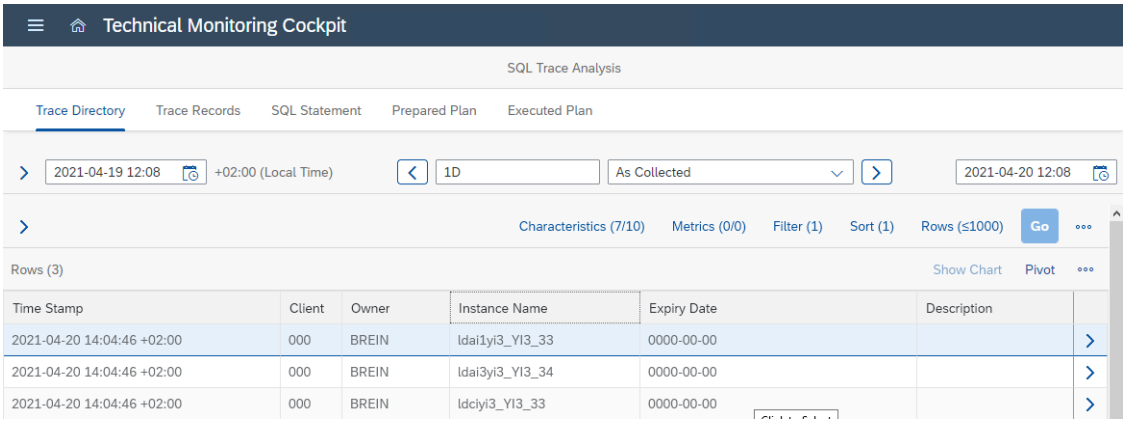
If you're familiar with on-premise products from SAP and know the SAP GUI transaction `ST05` (performance trace), you'll be happy to see that the SQL trace functions described in the following procedure are similar to what you know from the SQL trace in transaction `ST05`.

Procedure

1. Take an SQL trace of your ABAP application using ABAP development tools for Eclipse (ADT):
   a. Right-click on your ABAP project on the project explorer and choose *SQL Trace* from the context menu.

b. Activate the trace.
c. Run the ABAP application that you want to trace.
d. Deactivate the trace again and choose *View Trace Directory*.

   This opens the *SQL Trace Analysis* app.



2. On the *Trace Directory* tab page of the SQL trace analysis, choose the new trace you just took.

   The default view shows you all SQL traces taken by your ABAP user in the last 24 hours.

   The concept of the *SQL Trace Analysis* app is: With each move to a tab page to the right, you get more information about what you've selected on the previous tab pages to the left.
3. Choose the trace that you want to look at and change to the *Trace Records* tab page.
4. On the *Trace Records* tab page, single out expensive or otherwise interesting SQL statements.

   By default, you see all trace records in chronological order. However, you can apply filter conditions, for example, on the SQL statement text, or sort the entries by duration to put the longer running statements to the top of the list. Note that you can display additional columns that aren't shown by default by choosing *Characteristics* or *Metrics*.

   To go further into detail from here, you have three options on the following three tab pages to the right (*SQL Statement*, *Prepared Plan*, *Executed Plan*), which are mostly independent of each other. You can, for example, jump from here directly to the calculation of an executed access plan for your statement.
5. Choose the *SQL Statement* tab page to get a decent view of your selected SQL statement, pretty-printed and with syntax highlighting.

   The *Application Source Position* field shows you the ABAP code location (if available) where the SQL statement originates from. You can choose one of the *Show ABAP Source in ...* buttons to display the ABAP source in the web browser or in ABAP Development Tools.
6. Choose the *Prepared Plan* tab page to display and analyze a newly calculated prepared access plan for your SQL statement.
7. Choose the *Executed Plan* tab page to display and analyze a newly calculated executed access plan for your SQL statement.

   You can also download the PlanViz (PLV) file of the executed access plan containing the comprehensive plan information to display it with other HANA plan visualizing tools like HANA SQL Analyzer.

6.3.3.1.5.3 Tracing HANA Executed Plans Using the Capture Request Statistics App

If you identify a problematic SQL statement that uses many resources or runs slowly on your ABAP tenant, you might need to analyze the HANA execution plan.

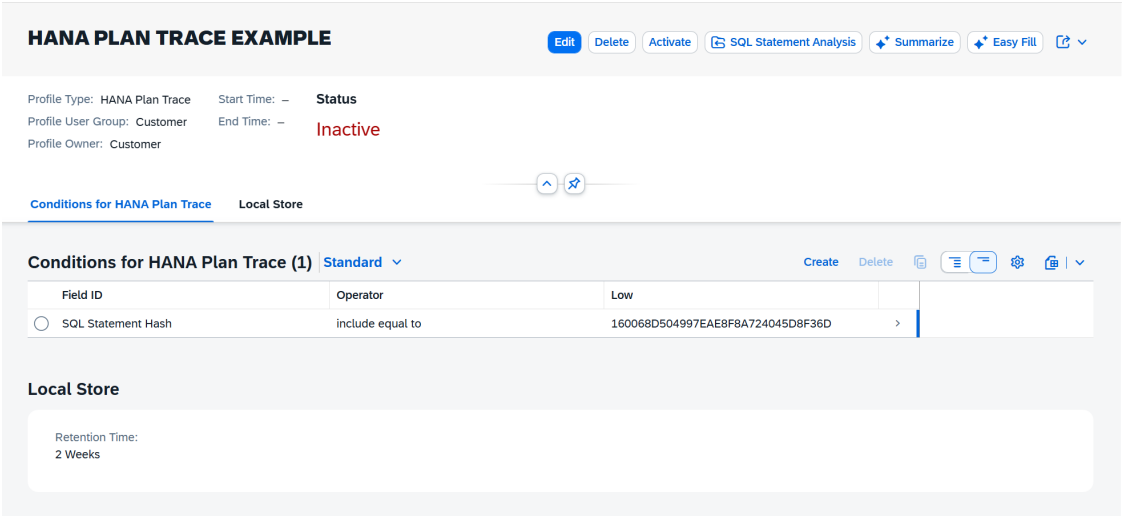
To do this, you can use the `Capture Request Statistics` app and trace these plans during future executions. In the app, create your own HANA plan trace profile or use an existing one to capture PlanViz (PLV) files for SQL statement executions. You can then download the PLV files in the `SQL Statement Analysis` app and analyze them using the SQL Analyzer Tool for SAP HANA in Visual Studio Code.

Prerequisites

You need the `SAP_CORE_BC_TMC_MON_PC` business catalog, which is part of the `SAP_BR_ADMINISTRATOR` business role template.

Procedure

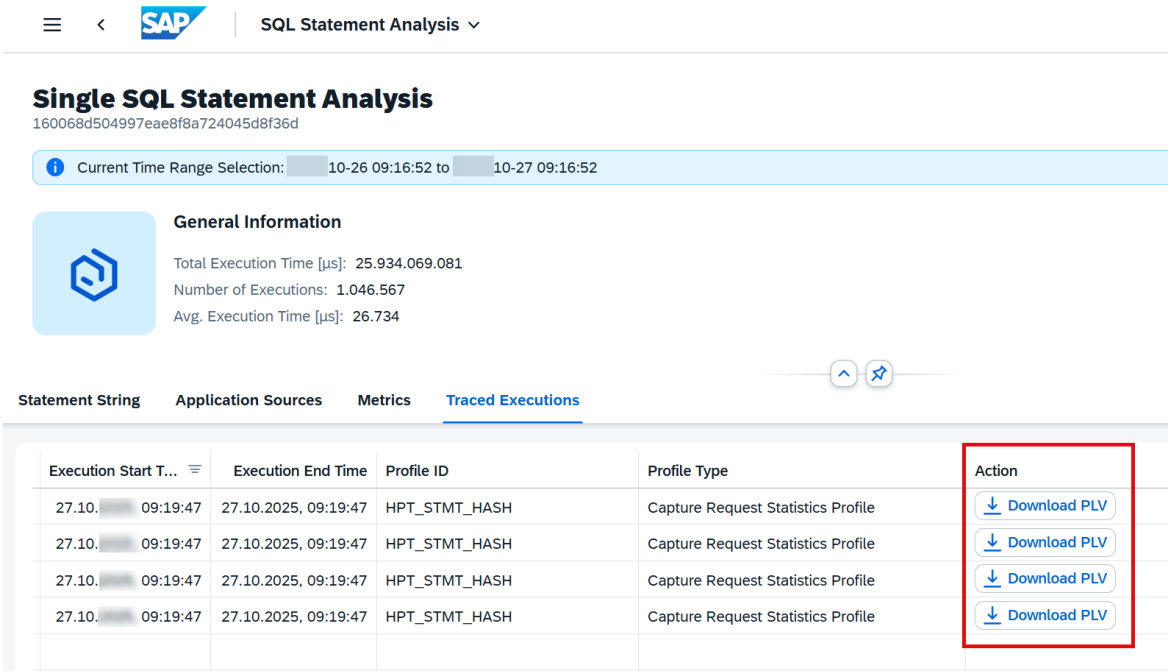
1. Search for the `Capture Request Statistics` app on the SAP Fiori Launchpad and open it, or navigate to the *Technical Monitoring* page on the SAP Fiori Launchpad and find the app there.
2. On the home screen of the `Capture Request Statistics` app, choose *Create* or an existing HANA plan trace profile from the list.
3. To create a new profile, follow these steps:
    1. Choose a unique *Profile ID* to ensure your trace profile is clearly recognizable. The same applies to the *Description* field.
    2. As *Profile Type*, choose *HANA Plan Trace*.
    3. In the *Profile User Group* field, choose the user group you want to trace, for example, *Customer*.
    4. Choose the *Create* button. The new profile is created and displayed with status *inactive*:



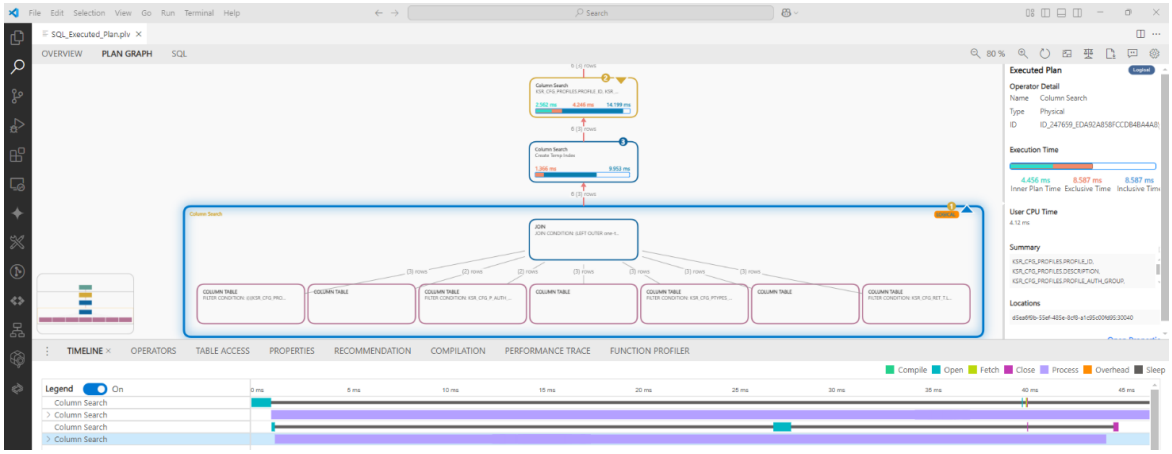
4. In the *Conditions for HANA Plan Trace* section, choose a valid statement hash as a filter condition for your trace profile.
   Note that you can capture HANA executed plans only for SQL statement hashes that have been executed at least once before and therefore can be found, for example, in the `SQL Statement Analysis` or `System Workload` app.
5. In the *Local Store* section of your profile, adjust the retention time to set how long the traced records will be accessible.
6. To start the HANA plan trace, choose *Activate*. Trace record generation is limited to collecting 20 traces and to a trace duration of one hour. You can manually deactivate the HANA plan trace profile at any time to stop collecting trace records.

> **Note**
>
> If one of these limits is reached, the profile run stops automatically. A small time delay might occur. After the profile stops, the traced records become available. You cannot access the traced records of a profile that is still running.

7. To download the PLV files, choose the *SQL Statement Analysis* button. The `SQL Statement Analysis` app opens.
8. In the `SQL Statement Analysis` app, go to the *Traced Executions* tab for the relevant statement hash and download the PLV files.



9. To analyze the PLV files, open them in the SQL analyzer tool for SAP HANA (Visual Studio Code extension):



Related Information

SQL Analyzer Tool for SAP HANA

Tracing HANA Executed Plans Using the ABAP Cross Trace in ADT [page 3408]

SQL Statement Analysis

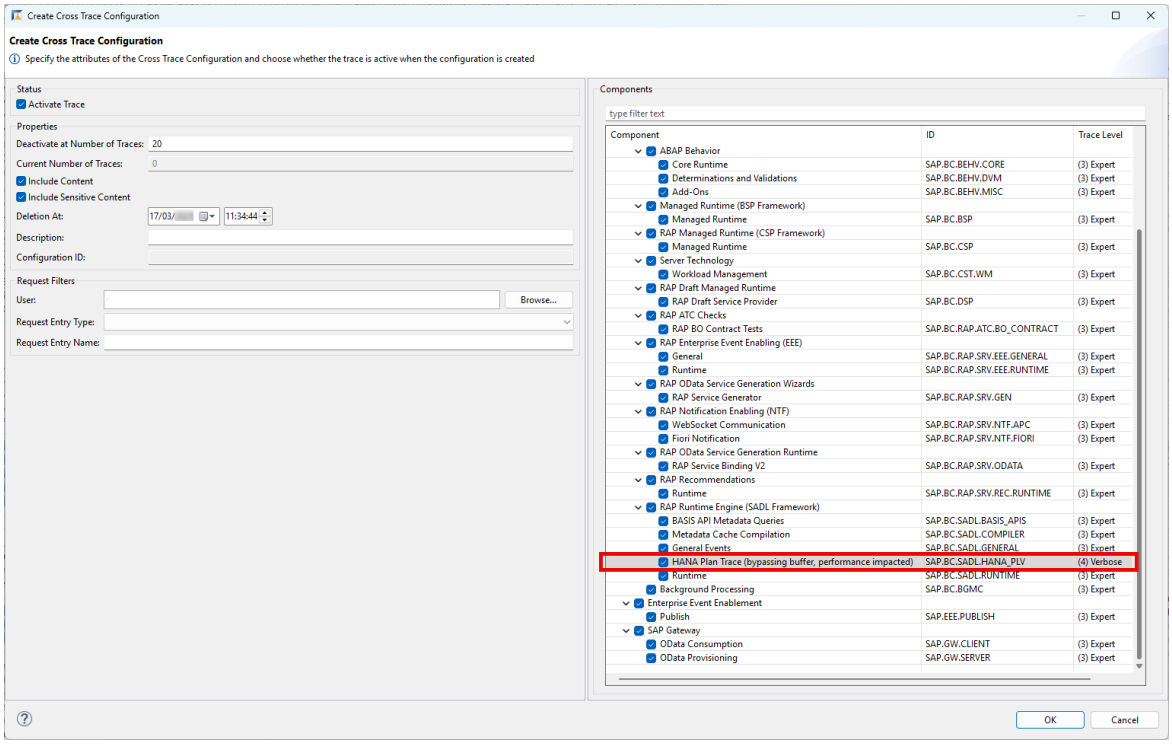
6.3.3.1.5.4 Tracing HANA Executed Plans Using the ABAP Cross Trace in ADT

Analyze the SQL statement execution in your system to identify potential bottlenecks or performance issues.

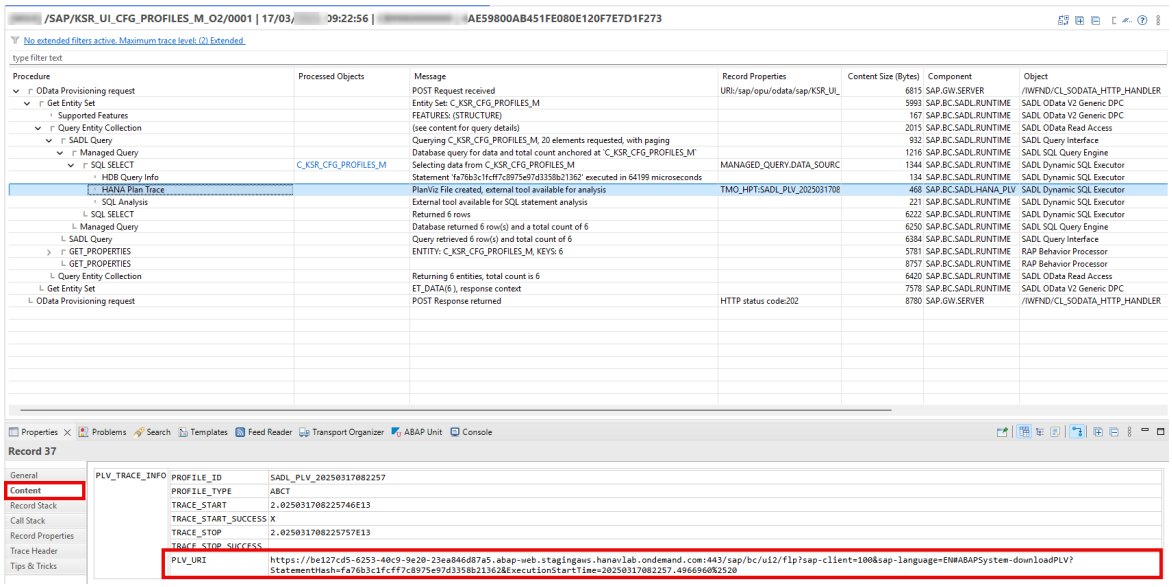
To do so, in the ABAP cross trace in ABAP development tools for Eclipse (ADT), you can capture PlanViz (PLV) files for SQL statement executions using the integrated HANA Plan Trace feature and download and view the PLV files in the SQL analyzer tool for SAP HANA.

Procedure

1. To open the ABAP Cross Trace view, choose *Window* > *Show View* > *Other* > *ABAP Cross Trace* in ABAP development tools for Eclipse (ADT).
2. Choose *Create Cross Trace Configuration*.
3. In the *Request Filters* section, enter your transaction criteria in the fields for *User*, *Request Entry Type* and *Request Entry Name*.
4. In the *Components* window on the right, make sure that *HANA Plan Trace* is selected and the *Trace Level* for this component is set to *Verbose*.

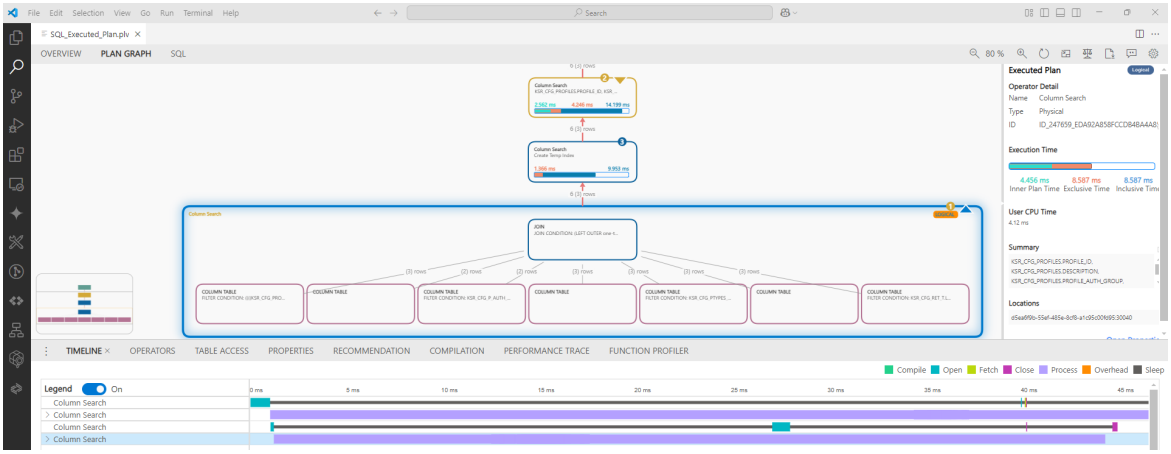


5. To start the trace for your user, choose *OK*. The generation of trace records will stop after the maximum number, which is set in the properties section, is reached.
6. On the *Trace Results* tab page, right-click on your user, choose *Refresh*, and you'll get an overview of the captured trace records for the duration the trace was running. The trace records that include a captured PLV file have a comment in the properties starting with *TMO_HPT: SADL_PLV*.
7. For details of a trace record, doubleclick an entry on the *Trace Results* tab page. In the tree, you find HANA Plan Trace results within the *SADL Query* nodes in *SQL SELECT* subnodes.
8. Choose a trace result entry named *HANA Plan Trace*. The properties for this entry are displayed in the *Properties* section.



9. Chose *Content* to display the `PLV_TRACE_INFO`.

10. Copy the URL displayed under *PLV_URI* into your web browser and open it. This will start the PLV download.
11. To view and analyze the PLV files, open them in the SQL Analyzer Tool for SAP HANA (Visual Studio Code extension).



Related Information

SQL Analyzer Tool for SAP HANA

6.3.3.2 Monitoring the System Health

With health monitoring metrics, you can check the health of your ABAP system from an application and customer perspective.

Metrics are collected on a regular basis and give you an idea of the system health with a special focus on performance, events, configuration, and the quota used by the ABAP system.

For the ABAP environment, you can choose between central monitoring tools such as SAP Cloud ALM or SAP Focused Run and an SAP Fiori app for local health monitoring:

	SAP Cloud ALM or SAP Focused Run	**SAP Fiori App**
Use cases	Monitoring multiple systems in a customer system landscape	Monitoring the local ABAP system
Prerequisites and constraints	SAP Cloud ALM or SAP Focused Run available	Local monitoring of only the system where the user is logged on
Benefits	• Display of health metrics for multiple systems • Alerts and notifications	Overview of metrics on one single screen

In addition to metrics provided by SAP, you can also define your own metrics for health monitoring using ABAP development tools for Eclipse (ADT). You can make these metrics automatically available in SAP Cloud ALM or SAP Focused Run.

Related Information

Central Health Monitoring Using SAP Focused Run and SAP Cloud ALM [page 3411]
Local Health Monitoring [page 3418]
Health Monitoring Using Your Own Metrics [page 3423]

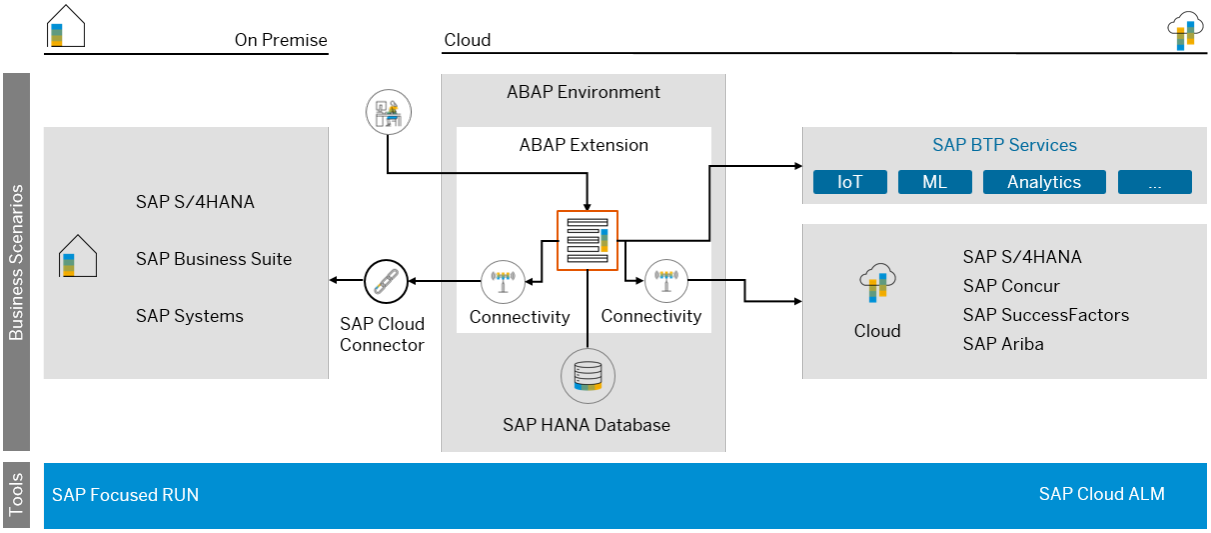
6.3.3.2.1 Central Health Monitoring Using SAP Focused Run and SAP Cloud ALM

Learn more about how you can monitor the ABAP environment using SAP Focused Run and SAP Cloud ALM.

Context

If you run SAP Business AI Platform, ABAP environment as part of a hybrid landscape with on-premise and cloud systems, you might already have a central monitoring and alerting infrastructure in place. For example, the service offering SAP Focused Run is designed specifically for businesses that need high-volume system and application monitoring, alerting, and analytics, especially for hybrid landscapes including on-premise SAP solutions.

In addition, for cloud-centric customers, there's SAP Cloud ALM as an application lifecycle management offering.



If you use SAP Focused Run or SAP Cloud ALM as monitoring and alerting infrastructure, you can integrate monitoring of the ABAP environment into your existing infrastructure. Using the health monitoring in SAP Focused Run or SAP Cloud ALM, you can watch whether your ABAP environment is still up and running and whether any exceptional situations occurred. In addition, you can use the real-user monitoring to monitor requests coming from business users and integration monitoring to watch the communication between integrated systems.

Available Metrics for the ABAP Environment

In the health monitoring of SAP Focused Run and SAP Cloud ALM, the following metrics for application checks are available for the ABAP environment:

Metrics for the ABAP Environment in Health Monitoring (Application Check)

Area	Metric Label	Technical Metric Name	Description
ABAP System	ABAP Runtime Errors	abap_core_dump_count_5m	The number of ABAP runtime errors during the last 5 minutes
ABAP System	Current Unique Users	abap_system_users_count	The number of current unique users in the ABAP system
ABAP System	Locked Business Users	abap_system_locked_bus_usr_count	The number of currently locked business users
ABAP System	Locked Communication Users	abap_system_locked_com_usr_count	The number of currently locked communication users
ABAP System	Current Sessions	abap_system_sessions_count	The number of current sessions in the ABAP system
ABAP System	Captured ABAP Statistics Records	abap_system_ksr_captured_count_5m	The number of ABAP statistics records that were captured by the *Capture Request Statistics* app during the last 5 minutes In the *Capture Request Statistics* app, you can select the checkbox *Health Monitoring* for any capture profile that you create. The number of records that is shown in SAP Cloud ALM is the total number of captured ABAP statistics records during the last 5 minutes for profiles with the checkbox *Health Monitoring* selected. For more information about one use case of this metric, see Monitoring Expensive Outbound Communication [page 3393].

Area	Metric Label	Technical Metric Name	Description
ABAP System	ABAP Compute Units	abap_acu_used_count_5m (type: memory)	Used quota for ABAP system memory during the last 5 minutes A quota represents the available system size and therefore the maximum allowed consumption of a resource. A resource is measured against the quota independently. For ABAP system resources, the quota is measured in ABAP compute units (ACUs).
ABAP System	ABAP Compute Units	abap_acu_used_count_5m (type: cpu)	Used quota for ABAP CPU utilization during the last 5 minutes A quota represents the available system size and therefore the maximum allowed consumption of a resource. A resource is measured against the quota independently. For ABAP system resources, the quota is measured in ABAP compute units (ACUs).
ABAP System	ABAP Compute Units	abap_acu_used_count_5m (type: allocation)	Resources (CPU and memory) that were allocated to your ABAP system during the last five minutes For ABAP system resources, the allocation is measured in ABAP compute units (ACUs). In a static service plan, the allocation corresponds to the quota limit of the service plan. In an elastic service plan, the allocation of resources is handled in an adaptive way.

Area	Metric Label	Technical Metric Name	Description
ABAP System	Expiry of Client Certificates	abap_system_client_cert_expiry_d	Expiry of client certificates in days With this metric, you can monitor whether any client certificates expire that you have uploaded to the ABAP system for the communication with systems or services outside the ABAP system, for example, SAP Business AI Platform services. Make sure that your client certificates are valid, so that communication doesn't break off. If the client certificate expires within 7 days, it's marked red. If it expires within 30 days, it's marked yellow.
ABAP System	Expiry of Communication System Certificates	abap_system_comsys_cert_expiry_d	Expiry of communication system certificates in days With this metric, you can monitor whether any communication system certificates uploaded to the ABAP system expire for the communication with systems or services outside the ABAP system. Make sure that your communication system certificates are valid, so that communication doesn't break off. If the communication system certificate expires within 7 days, it's marked red. If it expires within 30 days, it's marked yellow.

Area	Metric Label	Technical Metric Name	Description
ABAP System	Expiry of Trust List Certificates	abap_system_trusts_cert_expiry_d	Expiry of trust list certificates in days With this metric, you can monitor certificates from communication partners that you have added to the certificate trust list in your ABAP system. The metric shows whether any of these trusted certificates expire for the communication with systems or services outside the ABAP system. Make sure that your trusted certificates are valid, so that communication doesn't break off. If the trusted certificate expires within 7 days, it's marked red. If it expires within 30 days, it's marked yellow.
ABAP System	Critical Number Range Intervals	abap_system_nr_critical_interval_pct	Fill ratio of number range intervals Only critical number range intervals are shown. When you create a number range interval, you can define the critical remaining fill ratio when a warning is generated. If a critical number range was reported and is extended, it's reported once again with its now uncritical, good fill ratio to report the fixed state. After that, this number range is dropped from reported number range intervals because it's not critical anymore.
ABAP System	E-Mail Transmissions (Last Day)	abap_system_email_error_count_1d	The total number of e-mails and the number of e-mails with errors during the last day

Area	Metric Label	Technical Metric Name	Description
ABAP System	E-Mail Transmissions (Last 5 Minutes)	abap_system_email_error_count_5m	The total number of e-mails and the number of e-mails with errors during the last 5 minutes
ABAP System	Table Change Deltas	abap_system_bc_dbtablog_delta_count_1d	Table changes during the last day per database table for database tables with more than 10,000 changes
ABAP System	Table Changes	abap_system_bc_dbtablog_count_1d	Total table changes during the last day per database table for database tables with more than 50,000 changes
HANA	HANA Out-of-Memory Events	hana_db_oom_event_count_5m	The number of out-of-memory events on the SAP HANA index server during the last 5 minutes
HANA	HANA Compute Units (CPU)	hana_hcu_used_count_5m (type: cpu)	Used quota for CPU utilization during the last 5 minutes A quota represents the available system size and therefore the maximum allowed consumption of a resource. Each resource is measured against the quota independently. For SAP HANA database resources, the quota is measured in HANA compute units (HCUs).
HANA	HANA Compute Units (Disk)	hana_hcu_used_count_5m (type: disk)	Used quota for disk space consumption during the last 5 minutes A quota represents the available system size and therefore the maximum allowed consumption of a resource. Each resource is measured against the quota independently. For SAP HANA database resources, the quota is measured in HANA compute units (HCUs).

Area	Metric Label	Technical Metric Name	Description
HANA	HANA Compute Units (Memory)	hana_hcu_used_count_5m (type: memory)	Used quota for memory utilization during the last 5 minutes A quota represents the available system size and therefore the maximum allowed consumption of a resource. Each resource is measured against the quota independently. For SAP HANA database resources, the quota is measured in HANA compute units (HCUs).
Application Jobs	Average Application Job Delay	abap_system_appl_jobs_avg_delay_s_5m	Average application job delay in seconds during the last 5 minutes
Application Jobs	Maximum Application Job Delay	abap_system_appl_jobs_max_delay_s_5m	Maximum application job delay in seconds during the last 5 minutes
Application Jobs	Delayed Application Jobs	abap_system_appl_jobs_delayed_count_5m	The number of delayed application jobs during the last 5 minutes. An application job is counted as delayed when its delay exceeds a certain threshold. The default configuration uses a threshold of 60 seconds.
Application Jobs	Failed Application Jobs	abap_system_appl_jobs_failed_count_5m	The number of failed application jobs during the last 5 minutes
Application Jobs	Running Application Jobs	abap_system_appl_jobs_running_count	The number of currently running application jobs
Application Jobs	Finished Application Jobs	abap_system_appl_jobs_success_count_5m	The number of successfully finished application jobs during the last 5 minutes
Application Jobs	Application Logs	abap_system_appl_logs_count	The total number of application logs
Application Jobs	Application Logs with Errors	abap_system_appl_logs_errors_count_5m	The number of application logs with errors during the last 5 minutes

Setup of ABAP Environment Monitoring with SAP Cloud ALM

For more information about setting up ABAP environment monitoring with SAP Cloud ALM, see the tutorial Monitor An SAP BTP ABAP Environment Service Using SAP Cloud ALM and the documentation Setup for SAP BTP ABAP Environment.

More Information

SAP Focused Run on SAP Support Portal

Health Monitoring in SAP Focused Run

SAP Cloud ALM on SAP Support Portal

Health Monitoring in SAP Cloud ALM

Monitoring the Health of the ABAP System in the Cloud (blog post on SAP Community)

6.3.3.2.2 Local Health Monitoring

With the *Health Monitoring* app, you can check the health of your local ABAP system.

Context

This app allows you to monitor your system health **locally** directly on your ABAP system.

You get an overview of the metrics for the monitored system on one overview page. The health metrics are represented on individual cards with a KPI that summarizes the current status and with a history of the metrics to, for example, compare trends, identify performance bottlenecks, or investigate resource shortages.

The *Health Monitoring* app offers the following features:

- The time filter settings apply globally to all cards on the screen, that is, to all KPIs and charts.
- All charts are displayed on one screen and you can easily compare metrics over time.
- Multiple characteristics are visible in one chart if applicable.
- From some cards, you can navigate to related apps for further analysis or issue fixing. For more information, see the in-app help for the individual cards.

> ⓘ Note
>
> If you want to monitor the health of multiple systems **centrally**, you can use SAP Cloud ALM. For more information, see Central Health Monitoring Using SAP Focused Run and SAP Cloud ALM [page 3411].

Available Metrics for the ABAP Environment

In the *Health Monitoring* app, the following metrics for application checks are available for the ABAP environment:

Metrics for the ABAP Environment in Health Monitoring

Metric Card	Technical Metric Name	Description
ABAP Compute Units	`abap_acu_used_count_5m`	Used quota for ABAP system resources and resources that were allocated during the last 5 minutes A quota represents the available system size and therefore the maximum allowed consumption of a resource. Each resource (memory or CPU) is measured against the quota independently. For ABAP system resources, the quota is measured in ABAP compute units (ACUs). The allocation of ABAP system resources (memory and CPU) is also measured in ABAP compute units (ACUs). In a static service plan, the allocation corresponds to the quota limit of the service plan. In an elastic service plan, the allocation of resources is handled in an adaptive way. The number of ACUs on this card refers to the resource (memory or CPU) with the highest quota usage.
ABAP Runtime Errors	`abap_core_dump_count_5m`	The number of ABAP runtime errors during the last 5 minutes

Metric Card	Technical Metric Name	Description
Application Jobs	• Delayed application jobs: `abap_system_appl_jobs_delayed_count_5m` • Failed application jobs: `abap_system_appl_jobs_failed_count_5m` • Running application jobs: `abap_system_appl_jobs_running_count` • Successful application jobs: `abap_system_appl_jobs_success_count_5m`	The number of application jobs, which includes the following: • The number of delayed application jobs during the last 5 minutes. An application job is counted as delayed when its delay exceeds a certain threshold. The default configuration uses a threshold of 60 seconds. • The number of failed application jobs during the last 5 minutes • The number of currently running application jobs • The number of successfully finished application jobs during the last 5 minutes The KPI on this card refers to the number of recently failed application jobs. You can compare this value with the successfully finished application jobs to get an overview of your overall application job health.
Application Logs	`abap_system_appl_logs_count`	The total number of application logs
Application Logs with Errors	`abap_system_appl_logs_errors_count_5m`	The number of application logs with errors during the last 5 minutes
Average Application Job Delay	`abap_system_appl_jobs_avg_delay_s_5m`	Average application job delay in seconds during the last 5 minutes
Captured ABAP Statistics Records	`abap_system_ksr_captured_count_5m`	The number of ABAP statistics records that were captured by the *Capture Request Statistics* app during the last 5 minutes In the *Capture Request Statistics* app, you can select the checkbox *Health Monitoring* for any capture profile that you create. The KPI on this card is the total number of captured ABAP statistics records during the last 5 minutes for profiles with the checkbox *Health Monitoring* selected. For more information about one use case of this metric, see Monitoring Expensive Outbound Communication [page 3393].

Metric Card	Technical Metric Name	Description
Critical Number Range Intervals	`abap_system_nr_critical_interval_pct`	Fill ratio of number range intervals Only critical number range intervals are shown. When you create a number range interval, you can define the critical remaining fill ratio when a warning is generated. If a critical number range was reported and is extended, it's reported once again with its now uncritical, good fill ratio to report the fixed state. After that, this number range is dropped from reported number range intervals because it's not critical anymore.
Current Unique Users	• Sessions: `abap_system_sessions_count` • Users: `abap_system_users_count`	The number of current unique users and sessions in the ABAP system
E-Mail Transmissions (Last Day)	`abap_system_email_error_count_1d`	The total number of e-mails and the number of e-mails with errors during the last day
E-Mail Transmissions (Last 5 Minutes)	`abap_system_email_error_count_5m`	The total number of e-mails and the number of e-mails with errors during the last 5 minutes
Expiry of Client Certificates	`abap_system_client_cert_expiry_d`	Expiry of client certificates in days With this metric, you can monitor whether any client certificates expire that you have uploaded to the ABAP system for the communication with systems or services outside the ABAP system, for example, BTP services. Make sure that your client certificates are valid, so that communication doesn't break off. If the client certificate expires within 7 days, it's marked red. If it expires within 30 days, it's marked yellow.

Metric Card	Technical Metric Name	Description
Expiry of Communication System Certificates	`abap_system_comsys_cert_expiry_d`	Expiry of communication system certificates in days With this metric, you can monitor whether any communication system certificates uploaded to the ABAP system expire for the communication with systems or services outside the ABAP system. Make sure that your communication system certificates are valid, so that communication doesn't break off. If the communication system certificate expires within 7 days, it's marked red. If it expires within 30 days, it's marked yellow.
Expiry of Trust List Certificates	`abap_system_trusts_cert_expiry_d`	Expiry of trust list certificates in days With this metric, you can monitor certificates from communication partners that you have added to the certificate trust list in your ABAP system. The metric shows whether any of these trusted certificates expire for the communication with systems or services outside the ABAP system. Make sure that your trusted certificates are valid, so that communication doesn't break off. If the trusted certificate expires within 7 days, it's marked red. If it expires within 30 days, it's marked yellow.
HANA Compute Units (Memory, CPU)	`hana_hcu_used_count_5m`	Used quota for memory or CPU utilization during the last 5 minutes A quota represents the available system size and therefore the maximum allowed consumption of a resource. Each resource (memory or CPU) is measured against the quota independently. For SAP HANA database resources, the quota is measured in HANA compute units (HCUs). The number of HCUs on this card refers to the resource (memory or CPU) with the highest quota usage.

Metric Card	Technical Metric Name	Description
HANA Compute Units (Disk)	`hana_hcu_used_count_5m`	Used quota for disk space consumption during the last 5 minutes A quota represents the available system size and therefore the maximum allowed consumption of disk space. Disk space consumption is measured against the quota independently. For SAP HANA database resources, the quota is measured in HANA compute units (HCUs). The number of HCUs on this card refers to the disk with the highest quota usage.
HANA Out-of-Memory Events	`hana_db_oom_event_count_5m`	The number of out-of-memory events on the SAP HANA index server during the last 5 minutes
Locked Business Users	`abap_system_locked_bus_usr_count`	The number of locked business users
Locked Communication Users	`abap_system_locked_com_usr_count`	The number of locked communication users
Maximum Application Job Delay	`abap_system_appl_jobs_max_delay_s_5m`	Maximum application job delay in seconds during the last 5 minutes
Table Change Deltas	`abap_system_bc_dbtablog_delta_count_1d`	Table changes during the last day per database table for database tables with more than 10,000 changes
Table Changes	`abap_system_bc_dbtablog_count_1d`	Total table changes during the last day per database table for database tables with more than 50,000 changes

6.3.3.2.3 Health Monitoring Using Your Own Metrics

In addition to using the metrics for health monitoring provided by SAP, you can define your own application-specific metrics for the health monitoring of your customer and partner solutions running on the ABAP environment. The new metrics can be consumed by Health Monitoring in SAP Cloud ALM.

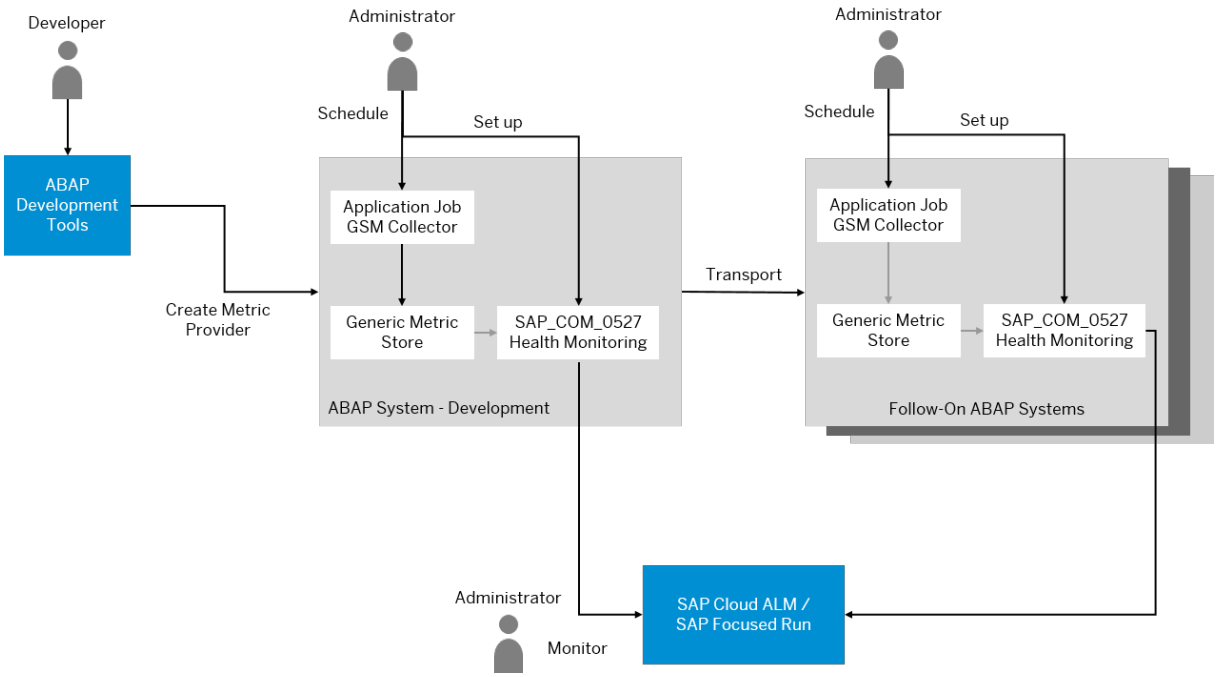
Background and Use Case

As a customer or partner, you can use SAP Cloud ALM or SAP Focused Run to monitor the health of your systems. As part of the health monitoring in SAP Cloud ALM and SAP Focused Run, you get metrics defined by

SAP that support you to watch technical issues such as the number of ABAP runtime errors, failed application jobs, or locked business users, for example, in the ABAP environment.

For your own customer or partner solutions running on the ABAP environment, you might want to define your own application-specific metrics to be able to monitor the health of these solutions. As a partner, you might also want to provide additional health monitoring metrics for your partner solution to your customers.

Architecture Overview



To define, set up, and use your own metrics, you proceed as follows:

1. Using ABAP development tools for Eclipse (ADT), a developer at your customer or partner company defines the relevant metrics for your solution.
2. As an administrator, you schedule an application job using the *Schedule Metric Provider Collection* app. The application job collects the metrics that you have defined and stores the values in the generic metric store.

   > **Note**
   >
   > Make sure that you schedule this job for the development system and all follow-up systems.

3. As an administrator, you define a communication arrangement based on the communication scenario `SAP_COM_0527` (*Health Monitoring*) in your ABAP system.
   As a result, a technical background job pushes the measured metrics automatically to SAP Cloud ALM or SAP Focused Run.

Related Information

Central Health Monitoring Using SAP Focused Run and SAP Cloud ALM [page 3411]
Developing Metrics for Health Monitoring [page 1223]

6.3.3.3 Displaying and Analyzing ABAP Runtime Errors

With the *ABAP Runtime Errors* app, you can display and analyze the ABAP runtime errors that have occurred in your current tenant.

Displaying ABAP Runtime Errors

When you open the app, you get a list of all ABAP runtime errors that have occurred in your current tenant and that have written a short dump.

You can search this list for certain dates and times, users, runtime errors, exceptions, and programs. In addition, you can filter the content of the list by using the corresponding filters.

> **Note**
>
> You can store the full content of an ABAP runtime error in a `zip` file on your local drive by choosing the *Download* button.

Analyzing ABAP Runtime Errors

To analyze an ABAP runtime error, click on a row in the list. This takes you to detailed information about this runtime error:

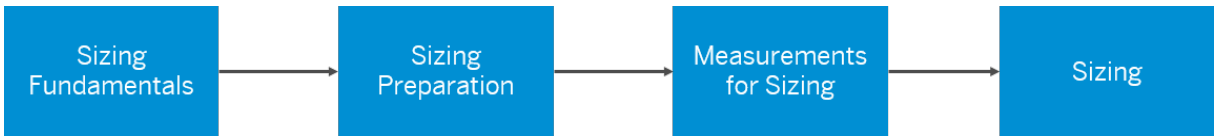
- The *General* section shows a description of what happened, an analysis of the error and information on where the program terminated.
- The *Source Code Extract* section shows an extract of the ABAP source code where the error occurred. You can work in the source code now and return to the source code position where the error occurred at any time by choosing *Go to Line with Runtime Error*. You can now choose whether you want to go to the ABAP code in the ABAP development tools in Eclipse (ADT) to debug or change the development object or whether you want to display the code read-only in HTML.
- The *Active Calls/Events* section lists the call stack with information on where the active calls and events occurred. From here, you can also navigate to the ABAP code in the ABAP development tools (ADT) or display it in HTML.
- Some HTTP requests from applications can result in failures that are recorded as ABAP runtime errors. For these cases you can find more information about the application call in the *Application Information* section.

6.3.3.4 ABAP System Sizing

This documentation describes the principles of how to perform system sizing for a custom application operated in the ABAP environment.

About this Documentation

This documentation introduces you to the following:



- Sizing Fundamentals [page 3427]
- Sizing Preparation [page 3431]
- Measurements for Sizing [page 3434]
- Sizing [page 3438]

1. **Sizing fundaments**
   You learn more about sizing occasions and methods, especially about expert sizing and how sizing works for scalable applications in the ABAP environment.
2. **Sizing preparation**
   Important preparatory steps are to identify the business processes that are relevant for sizing and to define test cases for them.
3. **Measurements for sizing**
   You can perform measurements for sizing using the *Capture Request Statistics* app.
4. **Sizing**
   You can use the *Perform System Sizing* app to perform the actual sizing of your ABAP system based on what you have measured before.

As the types of applications developed and operated in the ABAP environment are highly diverse, use this documentation as a rough guideline. This documentation assumes that in your business scenario, most business processes comprise the synchronous processing of single transactions. These business processes are the basis for measuring and calculating sizing values. If your business application mainly comprises other types of processing, for example, the mass processing of background jobs at specified dates and times, you need other sizing methods to achieve meaningful results.

This documentation focuses on the sizing of the ABAP layer. It doesn't include sizing for the SAP HANA database.

6.3.3.4.1 Sizing Fundamentals

Sizing describes the process of translating business requirements into hardware requirements.

Sizing determines resource requirements such as CPU processing power, physical memory, as well as disk and network bandwidth. The outcome is influenced by both business and technological aspects. This means that the number of users and their application usage is as important as the behavior of the application and the underlying technology stack.

6.3.3.4.1.1 Sizing Occasions and Methods

Sizing makes sense at different occasions, such as before a go-live or after an increased usage of applications. You can use different sizing methods, ranging from educated guess to expert sizing.

Occasions for Sizing

Sizing can be performed at different occasions:

- Before the initial go-live of an application (greenfield sizing)
- After the initial go-live and after an increased usage of applications, or after functionality has been added or changed (delta sizing or resizing)

Reliable sizing can only be achieved if the hardware as well as the platform layer used are scalable, but most importantly, the application software must be scalable. Load tests are used to check the scalability of all involved components.

In a typical productive environment, 80% of the workload is caused by 20% of the business processes. Consequently, detailed sizing focuses on these relevant business processes. Sizing for the remaining business processes is typically done using an educated guess.

As a rule of thumb, sizing should target an average resource utilization of no more than 70%. Keep extra space available for peak times as well as for anticipated future growth because of increasing business activity.

> → Recommendation
>
> We recommend that you consider a resizing of the environment if the resource utilization frequently exceeds 80%. Going beyond 90% resource utilization in normal operation mode usually causes resource contention in individual components and can considerably impact user experience with longer response times.

Sizing Methods

There are different sizing methods that allow you to perform sizing with different efforts required, but also resulting in different levels of accuracy. Sophisticated sizing methods are applied in environments where

hardware resources are the main cost driver. Simple and less elaborate sizing methods can be applied in smaller environments, or to get a starting point for less critical scenarios.

Sizing Method	Advantages	Disadvantages
Educated guess (Rule of thumb)	• Easy to apply • Delivers a quick rough estimate	• Depends on many assumptions • Highly depends on the experience of the person who makes the guess
T-shirt sizing (Predefined application model and categories of workload sizes)	• Easy to apply • Easy to understand	• Depends on many assumptions • Requires an underlying sizing model that fits well to the actual application
Expert sizing (Manual measurement and creation of sizing formula)	• Delivers the most realistic results • Allows for more variables • Transparent sizing model	• Suggests accuracy that sizing can't deliver • Requires the highest efforts

6.3.3.4.1.2 Expert Sizing

As the ABAP environment is a platform-as-a-service offering, sizing in this context refers to applications that aren't standard SAP coding but are custom development of either an SAP partner or an SAP customer. This fact usually prevents the usage of the educated guess or t-shirt sizing method, as there's typically no reference available. This documentation therefore focuses on the expert sizing method.

Expert sizing involves the following steps:

1. Identify the sizing-relevant (main) business scenarios and their variable dimensions.
2. Determine the throughput requirements during defined time periods and their day-time distribution. As part of the requirement determination, separate scenarios that are time-critical from those scenarios that can be postponed to times with lower load.
3. Define test cases for each relevant scenario with a reasonable number of input parameters and assumptions. These test cases must be self-contained and idempotent, which means repeatable with a stable load profile.
4. Execute each test case in single-user mode to measure its individual workload profile.
5. Create sizing models based on the results of the previous steps.
6. Create a structured load analysis to verify and refine the sizing model regarding scalability aspects (mass user testing).

   > ⓘ Note
   >
   > This step isn't in the scope of this document.

7. To simplify the consumption of the sizing formulas resulting from expert sizing, you can also define and precalculate representative t-shirt sizes for the system.

> → Recommendation
>
> Changes to the underlying technology stack (for example, configuration changes in the ABAP environment), as well as changes in the application behavior (for example, enhanced functionality in a

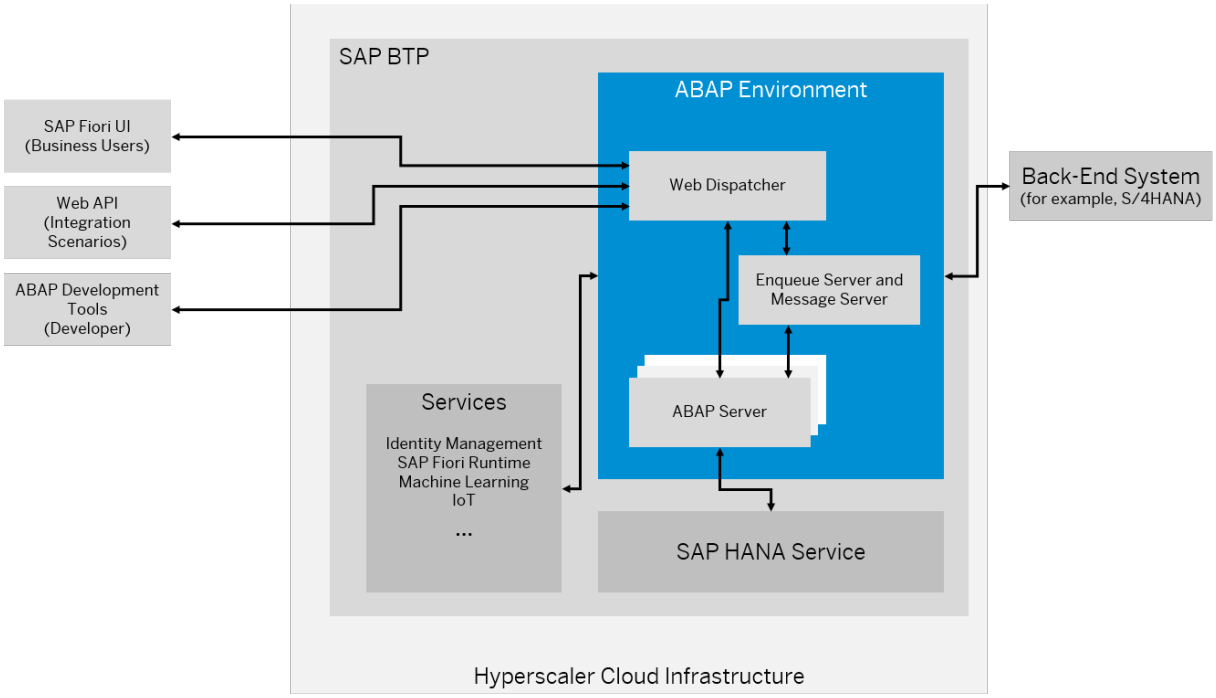
new software release) can invalidate your sizing results. We recommend that you validate from time to time whether the sizing is still adequate for your system workload.

6.3.3.4.1.3 Scalable Applications in the ABAP Environment

Get an overview of the technical architecture of an ABAP environment. It's important to understand which aspects are conditioned by this architecture and are therefore relevant for creating scalable applications in the ABAP environment and for the sizing of such applications.

Architecture of the ABAP Environment

The following figure provides an overview of the technical architecture of the ABAP environment:



Both the developer and the business user access the ABAP environment using HTTPS OData calls. The web dispatcher of the ABAP system dispatches these calls to an ABAP server. The respective ABAP server processes the request and might access the SAP HANA database to read or modify data. Besides user-driven calls, the web API can be used to integrate ABAP environment-based applications with other systems, for example, to consume IoT data.

While there's only one web dispatcher and one enqueue and message server, the number of ABAP servers can be increased based on demand (scale-out approach). There are always at least two ABAP servers.

The ABAP environment itself consumes reuse services of SAP BTP, for example, identity management or the SAP Fiori UI runtime. The ABAP environment developer can consume additional reuse services such as machine learning. Furthermore, predefined connectivity services of SAP BTP can be consumed to connect an ABAP environment with an SAP S/4HANA (or SAP ERP) back-end system.

Relevance for Sizing

The following aspects are important regarding the scalability of the ABAP environment-based applications:

- The ABAP environment is designed to scale primarily via the number of ABAP servers (scale-out), and not via the size of individual ABAP servers (scale-up). It's therefore important to split larger work packages into smaller ones, and to process these smaller packages in parallel on many rather small ABAP servers .
- With the in-memory architecture of SAP HANA and its support for heavily parallelized operations, it has become popular to push down application logic into the database layer (ABAP supports this pushdown using ABAP-Managed Database Procedures). While this pushdown remains a valid use case for operations that are time-critical or benefit greatly from being executed close to the data, care must be taken not to overload the central (and expensive) SAP HANA instance. In many cases, scaling out by using more (comparatively cheap) ABAP servers provides a better total cost of ownership.
- The ABAP environment and SAP HANA as a cloud service are co-located in a single virtual data center (one hyperscaler region, but not necessarily the same data center). Therefore, the network latency between these components is still slightly higher than in a typical on-premise environment. Consequently, performance anti-patterns such as small `SELECT` queries within loops are even more problematic in an ABAP environment than in an on-premise setup.
- As for any cloud service, network latency is also a limiting factor for clients interacting with the ABAP environment. Avoid performing many small calls to the ABAP environment.

Keep these aspects in mind when you plan scalable applications in the ABAP environment and their sizing.

6.3.3.4.1.4 Native Storage Extension for the ABAP Environment

You can use SAP HANA Native Storage Extension for selected database tables to save in-memory storage.

SAP HANA Native Storage Extension

SAP HANA Native Storage Extension (NSE) is a general-purpose, built-in warm data store in SAP HANA that lets you manage less-frequently accessed data without fully loading it into memory. It integrates disk-based or flash-drive based database technology with the SAP HANA in-memory database for an improved price-performance ratio.

For more information, see SAP HANA Storage Extension in the SAP HANA administration guide.

Enabling Native Storage Extension for Selected Database Tables

When you create or alter database tables in the ABAP Development Tools, go to the technical table settings. In the *Load Unit* field, select *Page Enforced*.

6.3.3.4.2 Sizing Preparation

Before you start any measurements for system sizing, it's important that you prepare system sizing carefully.

Such preparations include first and foremost identifying the main business processes that use the custom application for which you want to perform sizing. As a second step, formulate test cases that you want to use for the actual measurements.

6.3.3.4.2.1 Defining the Sizing-Relevant Business Processes

Before the actual sizing, identify sizing-relevant business processes and determine the throughput requirements for each of these business processes.

Context

> ⓘ Note
>
> Examples used in this sizing documentation refer to the ABAP flight reference scenario of the ABAP RESTful Programming Model. For more information, see Downloading the ABAP Flight Reference Scenario.

Procedure

1. Identify the main business processes of the application as well as their variable dimensions.

   The main business processes included processes that are critical for the company, for example, from revenue, reputation, or legal perspective.

   Examples of variable dimensions are the number of items in a sales order, or the number and size of documents uploaded by a user. It's unavoidable to make some assumption on these variables to limit the complexity of the resulting sizing model. As an example, it can be sufficient to assume an average of 10 items per sales order, instead of making items per sales order an input parameter of the sizing model.
2. Determine throughput requirements for each sizing-relevant business process, with its distribution over a typical business day. Determining throughput requirements also includes the separation of business processes that are time-critical from those processes that can be postponed to times of low load. For example, a background job performing some reporting activity can be shifted into non-business hours.

Results

The resulting overview of sizing-relevant business processes can look as follows, for example:

Flight Reference Example: Sizing-Relevant Business Processes

Business Process	Load Type	Process Executions per Minute	Criticality	Variable Dimensions
Creation of a new travel request	HTTPS / User-driven	160 per minute during main business hours (8 am to 7 pm)	High	Number of bookings per travel
Approval of a travel request	HTTPS / User-driven	160 per minute during main business hours (8 am to 7 pm)	High	--
Calculation and sending of metrics for reporting	Batch Job / Scheduled	Once a day	Low	Number of travel requests created per day

In this example, variable dimensions have been turned into assumptions to simplify the sizing model and to reduce complexity of the overall sizing procedure. For the flight reference example, we can assume an average of 2 bookings per travel (inbound and outbound flight). With eliminating this variable dimension, the only remaining input parameter of the respective business process is the number of process executions expected per minute.

6.3.3.4.2.2 Defining Test Cases for Sizing

After you've identified the sizing-relevant business processes, define test cases for each business process for baseline measurements.

It's important that these test cases are self-contained and idempotent, which means they're repeatable and return the same resource requirements after each repetition. Such a careful design of test cases is important to avoid side effects because, for example, data volume processed by SAP HANA increases during query execution (empty table versus table with high volume of irrelevant data).

Furthermore, make sure that each test case is limited to a single business process. For example, split the creation and the approval of a travel request into two test cases because creation and approval are performed by different users and at different points in time.

Example

A test case description for the business process *Creating a new travel request* can look as follows:

Test Case Example: Travel Request

Step ID	Description	Input Fields	Input Values
Preparation Steps (Not Measured)			

Step ID	Description	Input Fields	Input Values
SystemLogon	Link to SAP Fiori launchpad	User Password	`<tester name>`, `<tester password>`
Test Case (Measured)			
OpenApp	Open the travel management app from the launchpad (or using the Fiori Elements preview).	--	--
CreateTravel_1a	Choose *Create* from the table toolbar of the travel overview.	--	--
CreateTravel_1b	1. Open the search help of field *Customer ID*. 2. Choose *Go* without entering any search data. 3. Select an arbitrary entry from the persons listed.	Customer ID	`<arbitrary, using search help>`
CreateTravel_1c	1. Open the search help of field *Agency ID*. 2. Choose *Go* without entering any search data. 3. Select an arbitrary entry from the agencies listed.	Agency ID	`<arbitrary, using search help>`
CreateTravel_1d	1. Open the search help of the currency field next to *Total Price*. 2. In the *Currency* field, enter **EUR** and choose *Go*. 3. Select *EUR*.	Currency	EUR

CreateTravel_1e	Enter the remaining input and choose *Save*.	Starting date
		End date
		Total price
		Booking fee
		Status
		Description
			`<arbitrary>`
			`<arbitrary>`
			0
			9.99
			O
			`<optional>`

Cleanup Steps (Not Measured)			
DeleteTravel	Delete the created travel from either the travel overview or the travel details.	--	--

6.3.3.4.3 Measurements for Sizing

After you've defined test cases for sizing-relevant business processes, you can now measure the workload that each business process generates.

> **Note**
>
> This section of the documentation describes the steps necessary to prepare and execute a single test case for a business process. To obtain a combined sizing of all relevant business processes (see Defining the Sizing-Relevant Business Processes [page 3431]), perform the steps described in this section for each business process individually and add up the results.

6.3.3.4.3.1 Measurement Prerequisites

To ensure meaningful and reproducible results, make sure that prerequisites for the test system and test organization are met before measurements for sizing are taken.

Before you start testing, check the following:

- The setup and configuration of the test environment should be equivalent to a production environment. As you only perform single user tests, the test environment doesn't need the same hardware resources, but make sure that the following is the same as in a production system:
  - Software releases of all software components involved in the stack
  - Settings of the application that influence its behavior, for example, Customizing settings that control internal data verification steps
- Moreover, prevent any concurrent workload that consumes a noticeable amount of resources during the test. Having some house-keeping jobs active in the background is unproblematic. You should, however, avoid that tests compete with other, resource-intensive business processes.
- The test environment should contain a defined amount of existing business data. The SAP HANA database can consume different amounts of resources in a system with no business data compared to a system with a significant amount of existing business data.
- All test cases should be executed with a specific test user. A dedicated test user makes it much easier to separate calls originating from the test case execution from other activities, for example, calls originating from measurement tools. This test user should have the same authorizations that a corresponding business user has in the relevant business process.
- Consider caching effects of the application. The initial load of an application can consume more resources than a later use of the same application. To ensure a measurement that considers caching effects, execute the business scenario once without capturing the workload and a second time with capturing it.

6.3.3.4.3.2 Defining a Capture Profile for Sizing

With the *Capture Request Statistics* app, you can capture request statistics to find out which activities are running in your ABAP system. For sizing, you can define a capture profile that records defined activities of a business user or business process.

Prerequisites

You've defined one or multiple test cases to measure the workload of sizing-relevant business processes (see Defining Test Cases for Sizing [page 3432]).

Context

With the definition and activation of a capture profile, you can capture system activities. Requests are recorded with technical information, such as response time, program name, or CPU time. The request statistics serve as input for ABAP system sizing using the *Perform System Sizing* app (see Performing System Sizing [page 3438]).

> **Note**
>
> In this documentation, the assumption is that workload for a particular business process is measured by capturing the request statistics generated by the activities of a business user who acts as a tester. With such a measurement approach, you define the profile that contains the technical user name of the tester and a time period reserved for testing.

Procedure

1. In the SAP Fiori launchpad of the ABAP environment, under *Technical Monitoring*, choose the *Capture Request Statistics* tile.
2. Choose *Create*.
3. Enter the following data:

Field	What to enter
Profile ID	Enter a short, meaningful name for your profile. Don't use the prefix SAP_ because this prefix is reserved for profiles predefined by SAP.
Description	If needed, you can enter a longer description of your profile.

Field	What to enter
Start/End Time	The start and end time specify when request statistics were captured by the app. These fields are automatically filled after you have activated the profile.
Retention Time	Select how long you want to keep the captured request statistics stored in the system, for example, 14 days.
Record Limit	Enter the maximum number of records that you want the system to capture per minute, for example, 1,000 records. The record limit prevents that too much database memory is consumed by captured request statistics. The maximum possible record limit per minute is 1,000. **Note** To get a realistic idea of the workload, consider a relatively high record limit. However, you must keep in mind that you generate load on your system while request statistics are captured.
Sampling Rate	Enter a sampling rate, which is the probability of capturing one single ABAP statistics record in percent. **Note** To get a realistic idea of the workload, consider a relatively high sampling rate, if possible 100. However, you must keep in mind that you generate load on your system while request statistics are captured.
Target User Group	Choose one of the target groups starting with *Customer*.
Profile Type	This field is automatically populated by the system. Only static profile types (with fixed values) are possible.
Profile Owner	The profile owner is automatically set to *Customer* when you create a new profile.

4. Choose *Create*.

   The profile has been created, but is still inactive and has no filter conditions yet.
5. To define the filter conditions under which request statistics are captured, choose *Create*.
6. With *Field ID*, *Operator*, *High*, and *Low*, you can select a filter condition that determines what kind of data is captured.

   For example, if you want to capture all request statistics relating to your tester, select *User ID* as field ID, *include equal to* as operator and the technical user name of the tester.
7. Choose *Create*.
8. To enable capturing request statistics using the profile, choose *Activate*.

   If you use the capturing of request statistics to measure sizing, enter the planned start and end time of your test case.

6.3.3.4.3.3 Executing the Test Cases

Instruct your testers to perform the steps described in the test cases to measure the workload for sizing.

Prerequisites

Before testing starts, make sure that the following prerequisites are met:

- Testers have the authorizations that correspond to that of a business user.
- A capture profile exists, and it is set to *Active* (see also Defining a Capture Profile for Sizing [page 3435]).
- Testers perform their tests after the start time and before the end time defined in the capture profile.
- When you define and activate a profile, the start time of the profile must be later than or equal to now. The testers can start testing immediately. However, note that it takes up to two minutes before the records are processed by a collector job that runs in the background. Therefore, it can take up to two minutes until the first request statistics from your test are shown in the sizing app. In addition, it can take up to two minutes after the defined end time of the profile until all records have been processed and are shown in the sizing app.

Context

The testers' system activities are captured to estimate the workload as a basis for system sizing.

Procedure

1. Instruct your testers to execute the preparation steps as documented in Defining Test Cases for Sizing [page 3432].
2. Instruct your testers to execute the steps of the test case within the defined time period for testing.
3. After testing, wait for two minutes before you set the profile to *Inactive* again.

> ⓘ Note
>
> If a profile is set to *Inactive* before the collector of the request statistics runs another time, all ABAP statistics records get lost that would have been captured with the last collector run.

Results

The ABAP statistics records relating to the tester activities in the ABAP system have been captured. To ensure stable results, consider defining multiple capture profiles with different test times for one test case and repeat the measurement 2-3 times.

6.3.3.4.4 Sizing

After you have prepared measurements for sizing and a tester has executed the relevant test cases, you can now perform system sizing for each relevant business process.

With the help of the *Perform System Sizing* app, you can get sizing results for individual test cases in ABAP compute units. If you combine your results for all business processes, you can get a sizing for the overall business scenario.

6.3.3.4.4.1 ABAP Compute Units

ABAP Compute Units (ACUs) represent the runtime size in an SAP Business AI Platform, ABAP environment. Each ABAP Compute Unit comprises 16 GB of memory and approximately four virtual CPUs (vCPUs).

For availability reasons, we recommend using more instances instead of larger ones. However, large servers are more efficient when usage is constantly high, such as in multi-tenancy systems. Multi-tenancy systems consume more resources for runtime (ACU) and persistence (HANA Compute Units, HCUs).

The minimum number of ACUs and HCUs depends on the number of tenants. For every 100 tenants, at least one ACU and two HCUs are required.

The maximum number of ACUs is limited by:

- Twice the number of HCUs
- 32 times the size of application servers (0.5 or 2 ACUs)

The memory limit for a single session is 4 GB, regardless of the application server size.

Large servers allow memory exemptions (of up to 16 GB) requested programmatically using `cl_ssi_rp_session_memory`.

6.3.3.4.4.2 Performing System Sizing

With the *Perform System Sizing* app, you can calculate the required total ABAP memory, the work process time, and the ABAP CPU time in ABAP compute units for a custom application operated in the ABAP environment. You can use these calculations as a basis for ABAP system sizing.

Prerequisites

Before you start a system sizing, you've used the *Capture Request Statistics* app to capture a sample of the expected workload of the custom application.

> **Note**
>
> In the *Capture Request Statistics* app, you must create your own profile for capturing the relevant request statistics. For system sizing, you can't use the predefined SAP capture profiles.

Context

You want to use the captured request statistics as a basis to perform the system sizing of a production system. The *Perform System Sizing* app uses the captured request statistics to calculate the required total ABAP memory, the work process time per minute, and the ABAP CPU time per minute for the custom application and translates these KPIs into ABAP compute units (ACUs).

Procedure

1. In the SAP Fiori launchpad, under *Technical Monitoring*, choose the *Capture Request Statistics* app.
2. In the list of capture profiles, check whether the profile that you've defined before to capture the request statistics of the custom application is set to *Finished*.
3. Choose the profile from the list.

   The profile with its settings is shown.
4. To navigate to the system sizing, choose the *Perform System Sizing* button.

   The *Perform System Sizing* app opens with the captured request statistics of the profile that you've chosen.

   When you open the app, all captured ABAP statistics records are selected for system sizing, and a system sizing for 10 parallel process executions is shown. Initially, one ABAP compute unit is entered as available, but you can also enter a higher number of available ABAP compute units.
5. Replace the default value of 10 parallel process executions with the expected volume as assumed in your sizing-relevant business process definition (see Defining the Sizing-Relevant Business Processes [page 3431]).

   The process execution is the unit to describe the execution of a specific business process by a single business user.
6. Deselect ABAP statistics records from the table that aren't part of the test case definition (see Defining Test Cases for Sizing [page 3432]).

   > ### Note
   >
   > You can use the *User ID* field to deselect requests from users that you want to exclude from system sizing. By default, the field is hidden, but you can display it by choosing it in the table settings of the *ABAP Statistics Records* table.
7. Check whether the required ABAP compute units for total ABAP memory, work process time, and ABAP CPU time are still below acceptable thresholds.

   If total ABAP memory, work process time, and ABAP CPU time consume more than 70% of the available ABAP compute units, a warning sign next to the calculated ABAP compute units is shown. If 100% or more is consumed, an error sign is shown.
8. If there are still warnings or errors, increase the number of available ABAP compute units until all error or warning signs disappear and the calculated ABAP compute units (ACUs) for total ABAP memory, work process time, and ABAP CPU time are displayed with a green OK sign.

   > ### Note
   >
   > If you notice an entry in the captured ABAP statistics records that is responsible for a high workload and considerable resource consumption, don't simply accept a high number of required ABAP

compute units for your business process. Consider reviewing the relevant request for optimization and, after the optimization, repeat the sizing.

Results

Combine the results for multiple custom applications that run in the production system to get an idea of the required system size. After you have found the appropriate number of ABAP compute units for your business scenario, you can proceed with using this number for requesting an ABAP system for production.

Related Information

Combining System Sizing Results [page 3440]

6.3.3.4.4.3 Combining System Sizing Results

After performing a system sizing for each business process, combine the results for multiple custom applications that are planned to run in the production system. As a result, you get an idea of its required system size in ABAP compute units (ACUs).

Let's use the flight reference scenario and let's assume you've performed system sizing for its business processes *Create a new travel request*, *Approve a travel request*, and *Calculate and send metrics for reporting*. After performing system sizing for each business process, the results can look as follows:

Flight Reference Scenario: Example Sizing Results

Business Process	ABAP Total Memory	Work Process Time	ABAP CPU Time	Process Executions/Minute
Create a new travel request	0.108 ACUs	0.306 ACUs	0.414 ACUs	180
Approve a travel request	0.072 ACUs	0.162 ACUs	0.234 ACUs	180
Calculate and send metrics for reporting	0.025 ACUs	0.331 ACUs	0.092 ACUs	1

The process execution is the unit to describe the execution of a specific business process by a single business user. In the flight reference scenario, we assume that up to 180 travel requests are created and approved each minute, and metrics are calculated and sent for reporting once a minute.

The workload generated by a business process can vary depending on the daytime. In the flight scenario example, let's assume that travel requests are only created and approved between 8 a.m. and 8 p.m., but the

reporting runs in the evening after 8 p.m. Combining the business processes with the volumes expected at different times of day, you can create a summary such as the following:

Here, the ABAP total memory consumption, the work process time, and the ABAP CPU time for the travel requests and approvals must be combined. They add up to more than 0.2 ACUs for ABAP total memory, almost 0.5 ACUs for work process time, and more than 0.6 ACUs for ABAP CPU time. The reporting runs after 8 p.m. and its sizing is lower than the sum of the figures for travel request and approvals. Consequently, the limiting resource for the overall business scenario is the ABAP CPU time, with a maximum requirement of 0.648 ACUs. This ABAP CPU time still fits into the smallest available service plan of the ABAP environment (while also reserving some space for smaller business processes considered irrelevant for sizing).

To simplify the consumption of the sizing formulas that you created, you can optionally define some feasible t-shirt sizes and precalculate their resource requirements. Remember, sizing targets a mean resource utilization of no more than 70%:

Flight Reference Scenario: T-Shirt Sizes

T-Shirt Size	Description	Required ABAP Compute Units
S	Up to 180 travels created and approved per minute	1
M	Up to 360 travels created and approved per minute	2
L	Up to 540 travels created and approved per minute	3

6.3.3.5 Managing Growing Business Data

If you experience or expect a significant data growth in your application's persistency (SAP HANA database), it's essential that you define a data retention strategy to manage the amount of relevant data.

A good data retention and archiving strategy keeps your system lean, which increases the performance and lowers the total cost of ownership since storing and maintaining the data consumes resources. In other words, if your application persists a significant amount of data, it's important that you do some lifecycle management

for this data. If you don't, your database will grow indefinitely by default, which will negatively impact the performance and TCO. This is why you need to analyze the relevancy of especially aged data and how long you need to keep it.

As a first step, it's important that you need to understand the relevance of the data created by your business application. A simple categorization could look like this:

- Short retention time (30-90 days, depending on operational needs)
  - Log and monitoring data
  - Temporary data (caches, session data, processing queues)
- Medium retention time (2–7 years, depending on policy)
  - Operational data (transaction and master data)
  - Business communications
- Long retention time (7–30 years, sometimes indefinitely, depending on regulations)
  - Regulated data (financial records, tax documents, legal contracts, HR records)
  - Intellectual property and R&D data
  - Compliance & legal hold data

Once you have classified the importance of your data and the required retention time, you need to evaluate the different options that you have to manage your business data according to your needs.

Options to Delete Data or Move Data out of the System

- **Managing Data Retention Using Application Jobs**
  You can implement and schedule an application job to delete the data after a certain period of time. This application job defines the retention period and is executed on a regular basis.
  For more information about creating application jobs, see Application Jobs [page 1461].
- **Archiving Your Data**
  If you need to keep important data for a long time, for example, for legal reasons, you should consider archiving your data.
  For more information, see Data Archiving [page 1576]

Options to Keep Aged Data Available to the Application

- **Unloading Data from the SAP HANA Memory Using Nearline Storage Extension (NSE)**
  If you want to keep the data available within the system, consider using the nearline storage extension (NSE) of the SAP HANA database. Using NSE, you can move major parts of your data out of the HANA memory. The data will remain mostly on disk, which is a much cheaper way to retain your data. You can switch on the NSE feature in ABAP development tools for Eclipse (ADT).
  For more information about NSE, see Native Storage Extension for the ABAP Environment [page 3430].
- **Optimizing the Delta Merge by Partitioning HANA Tables**
  SAP HANA requires memory to perform delta merge operations on tables. If you expect a table in your SAP HANA database to grow considerably to **over 100 million records** (the limit is 2^{31} records), you should consider optimizing the delta merge by partitioning the table. If you choose a very selective partition

key that is also used frequently as a filter criterion by the application, this may also increase query performance.
For more information about partitioning your table, see Partition HANA Tables [page 3204].

- **Using Inverted Individual Indexes**
  You can reduce the size of the primary index by changing the index to an inverted individual index, which is a special type of SAP HANA index with more than one column for column store tables. Using an inverted individual index significantly reduces the memory footprint of specifically tables with many primary key columns. You can switch on the feature to change an index to an inverted individual index in ABAP development tools for Eclipse (ADT).
  For more information, see SAP Note 2600076.
- **Compressing LOBs**
  If your application data contains a high volume of LOB data (XSTRING or STRING fields with a length of more than 1000 bytes), you should consider compressing such LOBs. This provides significant space savings and improves query performance because less I/O is required to access the same amount of data. In addition, the compression of LOBs reduces time and space requirements for backup, restore, and log archiving. The SAP HANA database doesn't compress LOB data that exceeds 1000 bytes. In your application, you can compress and decompress LOB data using the ABAP class CL_ABAP_ZIP.
  For more information about compressing and decompressing your data, see Compression/Decompression [page 1369].

6.3.3.6 Resilience of an ABAP Environment System

Learn about the resilience features that are provided to all ABAP Cloud applications built on the SAP Business AI Platform, ABAP environment.

Instances deployed to the SAP Business AI Platform, ABAP environment without the multi-availability zone option receive the following resilience features:

- The components of a single ABAP system and the corresponding SAP HANA Cloud database preferably run together inside the same availability zone of a region.
- Automated monitoring and restart of services in case of failures is ensured for the ABAP system components and the SAP HANA Cloud database.
- A restart usually happens within minutes if hardware is available. A restarted ABAP application server still needs some more minutes to warm up before it can serve traffic.
- In case of failures, components are restarted on other hardware (subject to availability).
- Enqueue locks are stored redundantly.

With the multi-availability zone option for an ABAP environment system inside a region, the following additional features apply:

- The application servers for ABAP are distributed across the availability zones inside the region, in a round-robin fashion.
- Two instances of the SAP HANA Cloud database will be set up in different availability zones and configured with synchronous and transaction-safe data replication. This is covered with the multizone synchronous replication feature of the SAP HANA Cloud service.
- In case of failures, the ABAP system components are restarted on other hardware with higher priority than components of systems without the multi-availability zone option.
- In most cases, the underlying infrastructure handles outages of components automatically and restarts them on functional hardware within minutes.

- In severe cases, like complete availability zone outages, a fully automatic outage handling can't be guaranteed. For such cases, a manual declaration of a zone outage by SAP's operations team will lead to the evacuation of the defective zone and will bring up the system with the multi-availability zone option in the remaining zones, within the limits of hardware availability.
- In case of database-related failures, a takeover to the secondary database instance takes place automatically, and the application servers for ABAP reconnect to the secondary database instance.
- The multi-zone option doesn't affect the costs for the ABAP runtime: the consumed ABAP Compute Units (ACUs) don't change compared to a system without the multi-availability zone option.
- The second database instance affects the costs for the ABAP persistence: the consumed HANA Compute Units (HCUs) increase by factor 2.

To switch on the multi-availability zone option, set the parameter `multi_availability_zones` to `true`. See Updating an ABAP System [page 206].

> ⓘ Note
>
> The multi-availability zone option is based on the principles of the multi-availability zone architecture for SAP BTP *In-Metro Disaster Recovery* as described in Resilience, High Availability, and Disaster Recovery.

> → Tip
>
> The distribution of ABAP application servers across multiple availability zones requires at least two application servers. For example: with application servers of the size 0.5 ACU, the minimum *Total ABAP runtime size* is 1 ACU. A more resilient setup requires a minimum of three ABAP application servers. This configuration ensures that at least two servers remain running if one availability zone fails. For example: with 0.5 ACU servers, in this case, the minimum *Total ABAP runtime size* would be 1.5 ACU, and with 2 ACU servers, the minimum *Total ABAP runtime size* would be 6 ACU.
>
> An exception is the SAP Cloud Infrastructure region `eu01`, which currently has only two availability zones. If you want at least two servers to remain running in the event of an availability zone failure, you need at least four application servers. For example, with 0.5 ACU servers, this corresponds to a minimum *Total ABAP runtime size* of 2 ACUs.

> ⓘ Note
>
> The multi-availability zone option does not reduce the technical downtime for upgrades or updates and hotfix collection imports.

> ⓘ Note
>
> SAP Business AI Platform, ABAP environment provides protection against denial-of-service attacks via various defense mechanisms. These are continuously improved to keep a decent protection level. However, a full protection against DoS attacks can't be guaranteed.

Related Information

SAP HANA Database Availability Zone and Replicas
Resilience, High Availability, and Disaster Recovery

6.3.3.7 Virus Scanning

SAP Business AI Platform, ABAP environment applies virus scans on uploaded external data via OData service calls for fields of type `binary` to detect malicious or suspicious content.

In case malicious code is reported for binary content, it will be rejected and can't be processed any further. We also highly recommend common client protection tools, such as virus scanners on the clients.

> ### Note
>
> Note that for the following scenarios automatic virus scanning is not available:
>
> - During the processing of requests of object type `HTTP Service`
> - During `Read` operations
> - For data types other than `binary` (like `string`, for example)

6.3.3.8 Backups

The ABAP environment relies on backups from SAP HANA Cloud:

- Backups are retained up to 30 days.
- The recovery point objective (RPO), which is the maximum data loss, is no more than 15 minutes.
- Backups are stored encrypted and redundantly in different availability zones.

> ### Note
>
> A point-in-time recovery requires a coordinated plan between you as a customer and SAP that clarifies the target point-in-time and the target time of execution.

You can initiate a restore via a service request by using component `BC-CP-ABA`. To perform a restore, please provide the following information:

- The ID (GUID) of the ABAP environment service instance
- The target point-in-time to which the instance should be restored specified in coordinated universal time (UTC)

> ### Caution
>
> All data stored between the specified time and the time at which the service request is processed by SAP can't be retrieved.

> ### Caution
>
> A point-in-time recovery changes data stored in the ABAP environment service database only.
> Other data will not be changed, which may lead to inconsistencies with communication integration configurations or integrated services.

> ⚠ Restriction
>
> In case of SAP-managed software updates that are part of scheduled maintenance, the target point-in-time can only be set after the last software update.

6.3.4 Troubleshoot Custom Apps

After developing or releasing an application to a test or productive system, you might need to troubleshoot your application, for example to debug your business logic, display data of your application, and to analyze short dumps as well as logs and traces.

Prerequisites

As an administrator, assign business role `Application Support Engineer - Development Support` to a business user. With this role, you enable business users to troubleshoot applications. In addition to this business role, you might need to assign the business role of the application to be analyzed to the business user.

Business Role Template Application Support Engineer - Development Support

Business role template `Application Support Engineer - Development Support` (`BR_APPL_SUP_ENG_DEV_SUP`) contains all the business catalogs that are required for troubleshooting activities, such as:

- Debugging capability
- Running the data preview
- Executing classrun
- Accessing Feed Reader
- SQL Trace Analysis & SQL Explain Analysis capability

Business Catalogs

Business role template `Application Support Engineer - Development Support` includes the following business catalogs:

Business Catalog	Authorization
Development Support - Data Preview - Business Data (SAP_A4C_BC_DAT_PRV_DFT_PC)	This business catalog enables you to use the data preview in ABAP development tools for Eclipse for objects that are considered to contain business data, such as: • CDS views in language version 5 • Client-dependent tables in language version 5 with delivery class A and L • Data of CDS views that can be viewed with or without application of the DCL
Development Support - Data Preview - Cross-Client and Customizing Data (SAP_A4C_BC_DAT_PRV_RDO_PC)	This business catalog enables you to use the data preview in ABAP development tools for Eclipse for objects that are considered to contain customizing and cross-client data, such as: • Client-independent tables in language version 5 with delivery class A and L • Tables in language version 5 with delivery class S, E, and W (metadata) • Tables in language version 5 with delivery class C and G (customizing)
Development - Class Runner Execution (SAP_A4C_BC_DEV_CLA_RUN_PC)	This business catalog enables you to execute class runners in ABAP development tools for Eclipse.
Development - Data Preview - Released Objects (SAP_A4C_BC_DEV_DAT_PRV_PC)	This business catalog enables you to use the data preview in ABAP development tools for Eclipse for objects that are released for language version 5: • SAP-delivered CDS views • CDS views that are released for usage outside of their own software component • Data of CDS views that can be viewed with or without application of the DCL
Development - Development Objects Display (SAP_A4C_BC_DEV_OBJ_DIS_PC)	This business catalog enables you to sign in to ABAP development tools for Eclipse and view development objects with read-only authorization.

Business Catalog	Authorization
Development - Analysis and Support (SAP_A4C_BC_DEV_SUP_PC)	This business catalog gives you access to troubleshooting tools such as logs, traces, and the debugger. To enable the read-only debugging authorization, set the access category Write, Read, Value Help to *No Access* and the access category Read, Value Help to *Unrestricted* access. To enable debugging with value change in customer code, set the access category Write, Read, Value Help to *Unrestricted*. Source code changes are not permitted with this catalog. Nevertheless, business users can run the class runner in a test or production system, unless this is restricted by the administrator.

Connect to the System

As the Application Support Engineer, you have to connect to the ABAP system using ABAP development tools for Eclipse. This allows you to use the troubleshooting functionality and tools.

> ⓘ Note
>
> Certain analysis tools, such as the SQL Trace Analysis and SQL Explain SAP Fiori app are only accessible in SAP Fiori launchpad.

See Connect to the ABAP System [page 851].

Troubleshoot Using Debugging

The most common use case for troubleshooting to analyze the implemented business logic is debugging. The following debugging scenarios exist:

- Debug your own user
- Debug another business user
- Debug a communication user

See ABAP Debugger.

> ⓘ Note
>
> To find the business user and communication user ID for debugging, navigate to the *Debug Properties View* in ABAP development tools for Eclipse and search for business users (**CB***) and communication users (**CC***).

Debug Your Own User

To debug the application with your own Application Support Engineer user, make sure that you have the business application role assigned.

Debug Another Business User

To debug the application of another business user, you need to change the breakpoint settings in the debug properties view in ABAP development tools for Eclipse to the business user (**CB***).

Debug a Communication User

To debug the application of the communication user, you need to change the breakpoint settings in the debug properties view in ABAP development tools for Eclipse to the communication user (**CC***).

Related Information

Troubleshooting Tools [page 3449]

6.3.4.1 Troubleshooting Tools

Apart from debugging, you can also use other troubleshooting tools, such as the Feed Reader view in ABAP development tools for Eclipse, ABAP Profiler as well as the SQL Explain and SQL Trace Analysis SAP Fiori apps.

Feed Reader

ABAP Development Tools provides a dedicated view, the Feed Reader View, for runtime errors (ABAP short dumps) and gateway errors. See Feed Reader View.

ABAP Profiler

ABAP Trace

The ABAP Profiler allows you to examine the performance of your implemented business logic by using function modules and global ABAP classes. You can use the ABAP trace, which is the result of the ABAP Profiler, to identify runtime-intensive statements and to follow the hierarchy of program calls. See Profiling ABAP Code.

ABAP Cross Trace

If you develop OData services or other ABAP functionality related to the ABAP RESTful application programming model , the ABAP Cross Trace gives you insights into the ABAP RESTful Application Programming Model runtime framework. This includes the processing of OData requests, for example in SAP Fiori applications, and contains functionality similar to the payload trace of SAP Gateway trace in SAP GUI.

SQL Trace Analysis SAP Fiori App

With the SQL Trace Analysis app, you can access and analyze SQL trace records after you have used the SQL trace function of the ABAP performance trace in ABAP development tools for Eclipse. See SQL Trace Analysis [page 3015].

6.4 Administration and Operations in the Kyma Environment

This is the managed offering of SAP BTP, Kyma runtime (based on the open-source project "Kyma"). The administrators of the Kyma environment take care of setting it up and make sure it is ready for developers to work with. Create your Kyma instance to build applications and extensions to SAP and third-party solutions, manage roles, have your Kubernetes objects backed up, and view metrics and logs.

> ⚠ Caution
>
> If you apply cluster-wide security tools or policies to your cluster, configure them to exclude Kyma-managed namespaces and components. Policies that go beyond the platform defaults can block module installation and reconciliation. Kyma modules are not modified to comply with custom security policies.
>
> Do not make changes in any Kyma-managed namespaces, the `kube-system` namespace, or to any resources marked with *Do Not Edit*. Changes to those resources could lead to exclusion or disruptions from the agreed SLA. If you decide to make changes despite this, consult support in advance, and proceed with caution. To identify a Kyma-managed namespace, check if it has the `operator.kyma-project.io/managed-by: kyma` label.

Creating Kyma Instances [page 3452]
Set up a Kubernetes cluster with SAP BTP, Kyma runtime and use it to build applications and extensions to your SAP and third-party solutions. You can create one or multiple Kyma clusters in a single SAP BTP subaccount.

Tracking Kubeconfig and Cluster Associations in Kyma [page 3454]
You can identify which kubeconfig is associated with a particular Kyma cluster by the value of the *Cluster Name* parameter.

Managing Kyma Runtime Using the Provisioning Service API [page 3455]
The SAP Cloud Management service for SAP BTP (technical name: `cis`) provides the Provisioning Service API to create and manage available environments. Use the Provisioning Service API to automatically manage and access SAP BTP, Kyma runtime.

Available Plans in the Kyma Environment [page 3459]
Depending on your global account type, you have access to a different plan that specifies the cluster parameters for the Kyma environment.

Provisioning and Updating Parameters in the Kyma Environment [page 3470]
When creating a Kyma cluster, you can configure various parameters to adjust it to your specific needs.

Adding and Deleting a Kyma Module [page 3511]
To use a Kyma module, you must add it first. Use Kyma dashboard or Kyma CLI to do that. If you don't need the module anymore, delete it to save resources.

Setting Your Module to the Unmanaged and Managed State [page 3513]
In some cases, for example, for testing, you may need to modify your module beyond what is supported by its configuration. By default, when a module is in the managed state, Kyma Control Plane governs its Kubernetes resources, reverting any manual changes during the next reconciliation loop. To modify Kubernetes objects directly without them being reverted, you must set the module to the unmanaged state. In this state, reconciliation is disabled, ensuring your manual changes are preserved.

Access Kyma Application Logs [page 3515]
Get insights into your applications, microservices, and Functions by viewing the respective logs. To check out real-time logs immediately, use the Kubernetes functionalities - either in Kyma dashboard, or with kubectl.

Kyma Environment Backup [page 3516]

Kyma Modules' Sizing [page 3518]
The following tables show the resource consumption, such as memory, CPU, and storage, of Kyma modules.

Change Storage Size in Kyma [page 3521]
If the amount of data for the applications in your Kyma environment grows, you can expand the storage size for your customer data by resizing the respective Persistent Volume Claim (PVC).

Assigning Workloads to Worker Node Pools [page 3522]
In addition to the mandatory worker node pool existing in each Kyma runtime, you can add and configure custom worker node pools. These additional worker nodes are reserved for running your workloads with special computing requirements on optimized machines. To ensure your workload runs on the right worker node pool, assign workloads to particular worker nodes.

Adapting Your Workloads to Kyma Landscape Requirements [page 3525]
Learn how to use Runtime Bootstrapper, a built-in Kyma component, to automatically adapt your Pods to landscape-specific requirements such as private registries, custom TLS certificates, and FIPS mode signaling.

Admission Policies [page 3530]
Learn about the validating admission policies deployed in SAP BTP, Kyma runtime clusters, why they block specific actions, and what you can do if you need an exception. These policies are enforced at admission time and protect cluster stability, security, and operational correctness.

Kyma Runtime with Dual-Stack Support [page 3533]
Learn how to apply dual-stack support in Kyma instances.

Ingress and Egress IP Addresses [page 3534]
Learn how to check ingress and egress IP addresses for your Kyma runtime cluster. These addresses are needed to integrate with partner systems, enforce IP-based access controls, connect to external databases, and more.

6.4.1 Creating Kyma Instances

Set up a Kubernetes cluster with SAP BTP, Kyma runtime and use it to build applications and extensions to your SAP and third-party solutions. You can create one or multiple Kyma clusters in a single SAP BTP subaccount.

Prerequisites

Your subaccount has entitlements for Kyma runtime configured. See Configure Entitlements and Quotas for Subaccounts [page 2529].

Context

To set up Kyma environment in your subaccount, you must create an instance of it. You can create it from your subaccount *Overview* section by choosing *Enable Kyma* and following the wizard steps to configure the provisioning parameters. You can also create a Kyma environment instance from *Service Marketplace* in the same way as any other SAP BTP service or application.

If you prefer to work in a terminal or want to automate operations using scripts, you can create Kyma environment with the SAP BTP command line interface (btp CLI). See Enable SAP BTP, Kyma Runtime Using the Command Line.

In the process of provisioning a Kyma cluster, an instance of SAP Service Manager is created (see Credentials [page 2072]). Therefore, each Kyma instance has an instance of Service Manager assigned to it. If you have multiple Kyma clusters in your subaccount, you can easily trace which Service Manager instance is assigned to which Kyma instance by the Service Manager instance's display name, which is the same as its matching Kyma instance's ID.

> **Note**
>
> If the creation of an SAP Service Manager instance is unsuccessful, Kyma provisioning cannot begin. If the situation persists, create a support ticket (see Getting Support).

> **Note**
>
> To indicate that your SAP BTP, Kyma runtime is used for production, select *Used for production* in your subaccount details. This setting allows Kyma runtime operators to prioritize incidents and support cases affecting production subaccounts over subaccounts used for non-production purposes. See Change Subaccount Details [page 2516].

Procedure

1. In the SAP BTP cockpit, navigate to your subaccount *Overview*.
2. In the *Kyma Environment* section, choose *Enable Kyma* when creating your first Kyma instance. When provisioning the second and subsequent Kyma clusters, choose *Create*.
3. In the *Basic Info* view of the wizard window, perform the following actions:
   - Choose one of the plans assigned to your account.
   - You can keep the default instance and cluster names, but customizing them enables you to identify particular instances more easily.
   - Choose a region from the list.
4. To continue the configuration, choose *Next*.
5. In the *Additional Parameters* view, provide the required details in the *Form*. Alternatively, switch to the *JSON* tab and upload your configuration file or specify the parameters in JSON format.

   For more information on the configurable parameters, see Provisioning and Updating Parameters in the Kyma Environment [page 3470].
6. To review your configuration, choose *Next*.
7. To confirm changes, choose *Create*.
8. Wait until the instance is created. The instance creation may take several minutes. In the *Kyma Environment* section of your subaccount *Overview*, you can monitor the instance creation status and view the operation details.

   > ⓘ Note
   >
   > If the Kyma instances quota assigned to your subaccount has been used up, you get a message informing you about it. To increase the quota, go to *Entitlements* and make the necessary changes or contact your administrator.

Results

You have created a Kyma environment instance. If needed, you can repeat the procedure to create more Kyma instances.

Next Steps

- In the *Kyma Environment* section of your subaccount *Overview*, you can see all the Kyma instances created in your subaccount. If there is more than one, choose an instance from the dropdown list to view the following details:
  - Kyma instance ID
  - Kyma instance name
  - Plan
  - Kyma dashboard link

- Kubeconfig link
- Cluster name

You can also view the details of your Kyma instances under *Instances and Subscriptions*.

- To manage your Kyma instances, go to *Instances and Subscriptions*. Choose the action you want to perform from an instance's actions menu.
- To manage access to Kyma environment and Kyma dashboard, assign roles as needed. See Assign Roles in the Kyma Environment [page 3666].
- To use a variety of functionalities, such as telemetry and eventing, or to use SAP BTP services, add the respective Kyma modules. See Kyma Modules [page 88].

> → Tip
>
> To manage a Kyma instance automatically, you can create a Kyma service binding. The binding enables getting a Kyma kubeconfig, which in turn allows for accessing a Kyma cluster, deploying applications, running tests, and deleting the resources in a fully automated way. See Managing Kyma Runtime Using the Provisioning Service API [page 3455].

Related Information

Available Plans in the Kyma Environment [page 3459]
Configuring a Custom Identity Provider for Kyma [page 3660]
Adding and Deleting a Kyma Module [page 3511]
Account Administration Using the SAP BTP Command Line Interface (btp CLI) [page 2664]

6.4.2 Tracking Kubeconfig and Cluster Associations in Kyma

You can identify which kubeconfig is associated with a particular Kyma cluster by the value of the *Cluster Name* parameter.

The Cluster Name* [page 3480] (technical name: `name`) parameter specifies the name of your cluster. Its value is also used in the kubeconfig of a Kyma instance as a context name and a user name (if OIDC authentication is configured). With the {CLUSTER_NAME} value used in this way, you can track the kubeconfig - cluster associations.

If multiple OIDC configurations are provided, user entries are generated following this incremental naming convention:

- {CLUSTER_NAME}
- {CLUSTER_NAME-2}
- {CLUSTER_NAME-3}

The pattern continues for additional users. Without any OIDC configuration, the `users` section is omitted.

To view the kubeconfig file, you can download it from the SAP BTP cockpit. In your subaccount *Overview*, go to the *Kyma Environment* section and choose the *KubeconfigURL* link.

See how the {CLUSTER_NAME} value is used in a typical kubeconfig file:

```
---
apiVersion: v1
kind: Config
current-context: {CLUSTER_NAME}
clusters:
  - name: {CLUSTER_NAME}
    cluster:
      certificate-authority-data: {CERTIFICATE_DATA}
      server: {SERVER_URL}
contexts:
  - name: {CLUSTER_NAME}
    context:
      cluster: {CLUSTER_NAME}
      user: {CLUSTER_NAME}
users:
  - name: {CLUSTER_NAME}
    user:
      exec:
        apiVersion: client.authentication.k8s.io/v1beta1
        args:
          - get-token
          - "--oidc-issuer-url={ISSUER_URL}"
          - "--oidc-client-id={CLIENT_ID}"
          - "--oidc-extra-scope=email"
          - "--oidc-extra-scope=openid"
        command: kubectl-oidc_login
        installHint: |
          kubelogin plugin is required to proceed with authentication
          # Homebrew (macOS and Linux)
          brew install int128/kubelogin/kubelogin
          # Krew (macOS, Linux, Windows and ARM)
          kubectl krew install oidc-login
          # Chocolatey (Windows)
          choco install kubelogin
```

6.4.3 Managing Kyma Runtime Using the Provisioning Service API

The SAP Cloud Management service for SAP BTP (technical name: `cis`) provides the Provisioning Service API to create and manage available environments. Use the Provisioning Service API to automatically manage and access SAP BTP, Kyma runtime.

Prerequisites

- Your subaccount must have entitlements for SAP BTP, Kyma runtime and the SAP Cloud Management service for SAP BTP. See Managing Entitlements and Quotas Using the Cockpit [page 2521].
- Command line interface (CLI) tools:
  - kubectl configured to communicate with your Kyma instance (see Access a Kyma Instance Using kubectl [page 2265])
  - jq

- curl
- SAP BTP command line interface (btp CLI). See Download and Start Using the BTP CLI Client [page 2665].

Context

To manage a Kyma instance automatically, create a Kyma service binding. The Kyma service binding enables getting a Kyma kubeconfig, which in turn allows for accessing a Kyma cluster, deploying applications, running tests, and deleting the resources in a fully automated way.

Procedure

1. Provision the SAP Cloud Management service instance with the `local` plan and create a binding to get the credentials for the Provisioning Service API. To do that, you can use one of the following methods:
   - SAP BTP cockpit, as described in Getting an Access Token for SAP Cloud Management Service APIs [page 2731].
   - btp CLI and follow these steps:
   a. Set the `CIS_INSTANCE_NAME` environment variable with the name of the SAP Cloud Management service instance.

      ```
      export CIS_INSTANCE_NAME={CIS_INSTANCE_NAME}
      ```

   b. Provision the SAP Cloud Management service instance with the Client Credentials grant type passed as parameters.

      ```
      btp create services/instance --offering-name cis --plan-name local --name
      ${CIS_INSTANCE_NAME} --parameters {\"grantType\":\"clientCredentials\"}
      ```

   c. Create a binding for the instance.

      ```
      btp create services/binding --name ${CIS_INSTANCE_NAME}-binding --instance-
      name ${CIS_INSTANCE_NAME}
      ```

2. Set the `CLIENT_ID`, `CLIENT_SECRET`, `UAA_URL`, and `PROVISIONING_SERVICE_URL` environment variables using the credentials from the binding stored in the `clientid`, `clientsecret`, `url`, and `provisioning_service_url` fields. Use the btp CLI to get the credentials.

   ```
   export CLIENT_ID=$(btp --format json get services/binding --name $
   {CIS_INSTANCE_NAME}-binding | jq -r '.credentials.uaa.clientid')
   export CLIENT_SECRET=$(btp --format json get services/binding --name $
   {CIS_INSTANCE_NAME}-binding | jq -r '.credentials.uaa.clientsecret')
   export UAA_URL=$(btp --format json get services/binding --name $
   {CIS_INSTANCE_NAME}-binding | jq -r '.credentials.uaa.url')
   export PROVISIONING_SERVICE_URL=$(btp --format json get services/
   binding --name ${CIS_INSTANCE_NAME}-binding | jq -r
   '.credentials.endpoints.provisioning_service_url')
   ```

3. Get the access token for the Provisioning Service API using the client credentials.

```
TOKEN=$(curl -s -X POST "${UAA_URL}/oauth/token" -H "Content-Type:
application/x-www-form-urlencoded" -u "${CLIENT_ID}:${CLIENT_SECRET}" --data-
urlencode "grant_type=client_credentials" | jq -r '.access_token')
```

4. Check if Kyma runtime is available for provisioning.

```
curl -s "$PROVISIONING_SERVICE_URL/provisioning/v1/availableEnvironments" -H
"accept: application/json" -H "Authorization: bearer $TOKEN" | jq
```

5. Set the `ENVIRONMENT_TYPE` and `SERVICE_NAME` environment variables to const values kyma and kymaruntime, and provide values for the `NAME`, `REGION`, `PLAN`, and `USER_ID` environment variables.

```
export ENVIRONMENT_TYPE="kyma"
export SERVICE_NAME="kymaruntime"
export NAME={RUNTIME_NAME}
export REGION={CLUSTER_REGION}
export PLAN={KYMA_RUNTIME_PLAN_NAME}
export USER_ID={USER_ID}
```

6. Provision the Kyma runtime instance and save the instance ID in the `INSTANCE_ID` environment variable.

```
INSTANCE_ID=$(curl -s -X POST "$PROVISIONING_SERVICE_URL/provisioning/v1/
environments" -H "accept: application/json" -H "Authorization: bearer $TOKEN"
-H "Content-Type: application/json" -d
"{\"environmentType\":\"$ENVIRONMENT_TYPE\",\"parameters\":
{\"name\":\"$NAME\",\"region\":\"$REGION\"},\"planName\":\"$PLAN\",\"serviceNa
me\":\"$SERVICE_NAME\",\"user\":\"$USER_ID\"}" | jq -r '.id')
```

7. **Optional:** Set the `EXPIRATION_SECONDS` environment variable to the number of seconds (from 600 to 86400) after which a binding expires.

```
export EXPIRATION_SECONDS={EXPIRATION_SECONDS}
```

8. After the provisioning is completed, create the binding and save the binding ID in the `BINDING_ID` environment variable.

```
[ -z "$EXPIRATION_SECONDS" ] && \
BINDING_ID=$(curl -sS -D - -X PUT "$PROVISIONING_SERVICE_URL/provisioning/v1/
environments/$INSTANCE_ID/bindings" -H "accept: application/json" -H
"Authorization: bearer $TOKEN" -H "Content-Type: application/json" -d
"{\"parameters\":{\"expiration_seconds\":600}}" -o /dev/null | sed -n 's/
^.*location: //p' | sed 's/\r$//g') || \
BINDING_ID=$(curl -sS -D - -X PUT "$PROVISIONING_SERVICE_URL/provisioning/v1/
environments/$INSTANCE_ID/bindings" -H "accept: application/json" -H
"Authorization: bearer $TOKEN" -H "Content-Type: application/json" -d
"{\"parameters\":{\"expiration_seconds\":$EXPIRATION_SECONDS}}" -o /dev/null
| sed -n 's/^.*location: //p' | sed 's/\r$//g')
```

> ### Note
>
> You can have a maximum of 10 non-expired bindings. If you try to create more, you get the message stating that you've reached the maximum number of non-expired bindings.

9. Get the binding credentials and save them in a kubeconfig file.

```
curl -s -X GET "$PROVISIONING_SERVICE_URL/provisioning/v1/environments/
$INSTANCE_ID/bindings/$BINDING_ID" -H "accept: application/json" -H
"Authorization: bearer $TOKEN" | jq -r '.credentials.kubeconfig' >
kubeconfig.yaml
```

10. To access the cluster through kubectl, set the `KUBECONFIG` environment variable to the path of the kubeconfig file.

    ```
    export KUBECONFIG=kubeconfig.yaml
    ```

11. Verify the connection to the cluster. Run a kubectl command to get Pods.

    ```
    kubectl get pods
    ```

    kubectl should return the list of Pods in the `default` namespace running in the cluster, which means that the cluster is accessible.

12. **Optional:** To view the details of the binding you have created, list all bindings for the instance.

    ```
    curl -s "$PROVISIONING_SERVICE_URL/provisioning/v1/environments/$INSTANCE_ID/
    bindings" -H "accept: application/json" -H "Authorization: bearer $TOKEN"
    ```

13. **Optional:** For extra security, revoke the credentials by deleting the binding sooner than it is set to expire in the `EXPIRATION_SECONDS` environment variable.

    ```
    curl -s -X DELETE "$PROVISIONING_SERVICE_URL/provisioning/v1/environments/
    $INSTANCE_ID/bindings/$BINDING_ID" -H "accept: application/json" -H
    "Authorization: bearer $TOKEN"
    ```

    Try to access the cluster using kubectl. The connection should be refused, which means that the binding was successfully deleted and credentials revoked.

    ```
    kubectl get pods
    ```

    > ### Note
    >
    > If you skip this step, the binding is automatically deleted after the maximum allowed expiration time (86400 seconds) passes.

Next Steps

To deprovision Kyma runtime, run:

```
curl -s -X DELETE "$PROVISIONING_SERVICE_URL/provisioning/v1/environments/
$INSTANCE_ID" -H "accept: application/json" -H "Authorization: bearer $TOKEN"
```

> ### Note
>
> You can delete the runtime independently of the bindings. Existing bindings do not block the runtime deprovisioning.

Related Information

Account Administration Using APIs of the SAP Cloud Management Service [page 2730]

6.4.4 Available Plans in the Kyma Environment

Depending on your global account type, you have access to a different plan that specifies the cluster parameters for the Kyma environment.

Trial

The trial version of SAP BTP, Kyma runtime is available only on request. For more information, see Getting Started with a Trial Kyma Instance [page 249].

For details on the trial cluster specification, see Scope and Limitations [page 252].

Free

The technical name of the plan is `free`. With the free service plans for Kyma, you can try out services in global accounts without any additional cost for 30 days. However, the free model account has certain limitations.

- The free plan offers you a one-node cluster and is only available on AWS.
- Only community support is available for free tier service plans and these are not subject to SLAs.
- Before you enable the Kyma environment, you must first assign it as an entitlement to your subaccount.
- You can use the free plan only once in a global account for up to 30 days.

  > → Remember
  >
  > Once you have started the free plan in a subaccount, you cannot use it in another subaccount within the same global account even though the 30-day period has not ended.

- The services you plan to use must be available in the same region as the subaccount for the Kyma runtime. If necessary, change the default subaccount region.
- The upgrade to the paid plan is not yet supported.

For more information, read Using Free Service Plans [page 118].

> → Remember
>
> The parameters marked with an asterisk "*" are mandatory.

Free Plan Specification

Parameter	Description	Supported Operation	More Information
*Cluster Name** btp CLI parameter: `name`	Defines the name of your cluster.	Provisioning	Cluster Name* [page 3480]

Parameter	Description	Supported Operation	More Information
*Region** btp CLI parameter: `region`	Defines a region (set of data-centers) where your cluster runs.	Provisioning	Region* [page 3498]
Modules btp CLI parameter: `modules`	Defines which Kyma modules are provisioned in your cluster.	Provisioning	Modules [page 3488]
Networking btp CLI parameter: `networking`	Provides a custom IP range for worker nodes.	Provisioning	Networking [page 3491]
OpenID Connect btp CLI parameter: `oidc`	Provides a custom Open ID Connect (OIDC) configuration.	Provisioning Updating	OpenID Connect (OIDC) [page 3494]
Administrators btp CLI parameter: `administrators`	Specifies the list of runtime administrators.	Provisioning Updating	Administrators [page 3476]

Standard: Amazon Web Services, Google Cloud, Microsoft Azure, and Alibaba Cloud

The technical names of the standard enterprise plans are `aws`, `gcp`, `azure`, and `alicloud`.

Within these plans, Kyma offers the high availability feature. For more information, see Availability Zones in the Kyma Environment [page 100].

> ### Note
>
> To indicate that your Kyma runtime is used for production, select Used for production in your subaccount details. This setting allows SAP BTP, Kyma runtime operators to prioritize incidents and support cases affecting production subaccounts over subaccounts used for non-production purposes. See Change Subaccount Details [page 2516].

After your Kyma instance is created, you can switch from standard enterprise plans to build runtime plans within the same cloud provider. However, you can't switch from build runtime plans to the standard plans. For more information, see Updating Kyma Environment Plans [page 3468].

> ### → Remember
>
> The parameters marked with an asterisk "*" are mandatory.

Standard Plan Specification

Parameter	Description	Supported Operation	More Information
*Cluster Name** btp CLI parameter: `name`	Defines the name of your cluster.	Provisioning	Cluster Name* [page 3480]
*Region** btp CLI parameter: `region`	Defines a region (set of data-centers) where your cluster runs.	Provisioning	Region* [page 3498]
Colocate Control Plane btp CLI parameter: `colocateControlPlane`	Allows you to have both the control plane and worker nodes in the same region.	Provisioning	Colocate Control Plane [page 3480]
Machine Type btp CLI parameter: `machineType`	Specifies the provider-specific virtual machine type.	Provisioning Updating	Machine Type [page 3481]
Auto Scaler Min btp CLI parameter: `autoScalerMin`	Specifies the minimum number of virtual machines to create.	Provisioning Updating	Auto Scaler Min [page 3479]
Auto Scaler Max btp CLI parameter: `autoScalerMax`	Specifies the maximum number of virtual machines to create.	Provisioning Updating	Auto Scaler Max [page 3478]
Modules btp CLI parameter: `modules`	Defines which Kyma modules are provisioned in your cluster.	Provisioning	Modules [page 3488]
Networking btp CLI parameter: `networking`	Provides a custom IP range for worker nodes.	Provisioning	Networking [page 3491]
OpenID Connect btp CLI parameter: `oidc`	Provides a custom Open ID Connect (OIDC) configuration.	Provisioning Updating	OpenID Connect (OIDC) [page 3494]
Administrators btp CLI parameter: `administrators`	Specifies the list of runtime administrators.	Provisioning Updating	Administrators [page 3476]
Additional Worker Node Pools btp CLI parameter: `additionalWorkerNodePools`	Defines a custom list of additional worker node pools.	Provisioning Updating	Additional Worker Node Pools [page 3473]

Parameter	Description	Supported Operation	More Information
Access Control List [1, 2] `accessControlList`	Specifies the IP ranges that can access the Kubernetes API.	Provisioning Updating	Access Control List [page 3471]
Additional Volume Size [2] btp CLI parameter: `additionalVolumeSizeGi`	Specifies extra disk space on top of the default volume size for a worker node pool.	Provisioning Updating	Additional Volume Size [page 3472]
Audit Log Access [2] btp CLI parameter: `auditLogAccess`	Enables direct read access to your own audit log data using the SAP Audit Log Retrieval API v2.	Provisioning Updating	Audit Log Access [page 3477]

> **Note**
>
> [1] Not available in the standard Google Cloud (`gcp`) plan.

> **Note**
>
> [2] Not available in the standard Alibaba Cloud (`alicloud`) plan or in the China (North 3) (cf-cn20) region in the Microsoft Azure (`azure`) plan.

SAP Cloud Infrastructure

The technical name of this enterprise plan is `sap-converged-cloud`.

> **→ Remember**
>
> The parameters marked with an asterisk "*" are mandatory.

SAP Cloud Infrastructure Plan Specification

Parameter	Description	Supported Operation	More Information
*Cluster Name** btp CLI parameter: `name`	Defines the name of your cluster.	Provisioning	Cluster Name* [page 3480]
*Region** btp CLI parameter: `region`	Defines a region (set of data-centers) where your cluster runs.	Provisioning	Region* [page 3498]

Parameter	Description	Supported Operation	More Information
Colocate Control Plane btp CLI parameter: `colocateControlPlane`	Allows you to have both the control plane and worker nodes in the same region.	Provisioning	Colocate Control Plane [page 3480]
Machine Type btp CLI parameter: `machineType`	Specifies the provider-specific virtual machine type.	Provisioning Updating	Machine Type [page 3481]
Auto Scaler Min btp CLI parameter: `autoScalerMin`	Specifies the minimum number of virtual machines to create.	Provisioning Updating	Auto Scaler Min [page 3479]
Auto Scaler Max btp CLI parameter: `autoScalerMax`	Specifies the maximum number of virtual machines to create.	Provisioning Updating	Auto Scaler Max [page 3478]
Modules btp CLI parameter: `modules`	Defines which Kyma modules are provisioned in your cluster.	Provisioning	Modules [page 3488]
Networking btp CLI parameter: `networking`	Provides a custom IP range for worker nodes.	Provisioning	Networking [page 3491]
OpenID Connect btp CLI parameter: `oidc`	Provides a custom Open ID Connect (OIDC) configuration.	Provisioning Updating	OpenID Connect (OIDC) [page 3494]
Administrators btp CLI parameter: `administrators`	Specifies the list of runtime administrators.	Provisioning Updating	Administrators [page 3476]
Additional Worker Node Pools btp CLI parameter: `additionalWorkerNodePools`	Defines a custom list of additional worker node pools.	Provisioning Updating	Additional Worker Node Pools [page 3473]
Access Control List btp CLI parameter: `accessControlList`	Specifies the IP ranges that can access the Kubernetes API.	Provisioning Updating	Access Control List [page 3471]

Parameter	Description	Supported Operation	More Information
Additional Volume Size btp CLI parameter: `additionalVolumeSizeGi`	Specifies extra disk space on top of the default volume size for a worker node pool.	Provisioning Updating	Additional Volume Size [page 3472]
Audit Log Access btp CLI parameter: `auditLogAccess`	Enables direct read access to your own audit log data using the SAP Audit Log Retrieval API v2.	Provisioning Updating	Audit Log Access [page 3477]

Build Runtime: Amazon Web Services, Google Cloud, Microsoft Azure, and Alibaba Cloud

The technical names of these enterprise plans are `build-runtime-aws`, `build-runtime-gcp`, `build-runtime-azure`, and `build-runtime-alicloud`. Use them to integrate Kyma's functionalities within SAP Build. See the SAP Build documentation.

You can switch from the standard enterprise plans to build runtime plans within the same cloud provider. For more information, see Updating Kyma Environment Plans [page 3468].

You can also use the build runtime plans within the *Pay-As-You-Go for SAP BTP for cloud test, demo, and development* commercial model. For details, see Service Plans and Metering.

> → Remember
>
> The parameters marked with an asterisk "*" are mandatory.

Build Runtime Plan Specification

Parameter	Description	Supported Operation	More Information
*Cluster Name** btp CLI parameter: `name`	Defines the name of your cluster.	Provisioning	Cluster Name* [page 3480]
*Region** btp CLI parameter: `region`	Defines a region (set of data-centers) where your cluster runs.	Provisioning	Region* [page 3498]
Colocate Control Plane btp CLI parameter: `colocateControlPlane`	Allows you to have both the control plane and worker nodes in the same region.	Provisioning	Colocate Control Plane [page 3480]

Parameter	Description	Supported Operation	More Information
Machine Type btp CLI parameter: `machineType`	Specifies the provider-specific virtual machine type.	Provisioning Updating	Machine Type [page 3481]
Auto Scaler Min btp CLI parameter: `autoScalerMin`	Specifies the minimum number of virtual machines to create.	Provisioning Updating	Auto Scaler Min [page 3479]
Auto Scaler Max btp CLI parameter: `autoScalerMax`	Specifies the maximum number of virtual machines to create.	Provisioning Updating	Auto Scaler Max [page 3478]
Modules btp CLI parameter: `modules`	Defines which Kyma modules are provisioned in your cluster.	Provisioning	Modules [page 3488]
Networking btp CLI parameter: `networking`	Provides a custom IP range for worker nodes.	Provisioning	Networking [page 3491]
OpenID Connect btp CLI parameter: `oidc`	Provides a custom Open ID Connect (OIDC) configuration.	Provisioning Updating	OpenID Connect (OIDC) [page 3494]
Administrators btp CLI parameter: `administrators`	Specifies the list of runtime administrators.	Provisioning Updating	Administrators [page 3476]
Additional Worker Node Pools btp CLI parameter: `additionalWorkerNodePools`	Defines a custom list of additional worker node pools.	Provisioning Updating	Additional Worker Node Pools [page 3473]
Access Control List [3, page 3466] [4 page 3466] `accessControlList`	Specifies the IP ranges that can access the Kubernetes API.	Provisioning Updating	Access Control List [page 3471]
Additional Volume Size [4 page 3466] btp CLI parameter: `additionalVolumeSizeGi`	Specifies extra disk space on top of the default volume size for a worker node pool.	Provisioning Updating	Additional Volume Size [page 3472]

Parameter	Description	Supported Operation	More Information
Audit Log Access [4] [page 3466] btp CLI parameter: `auditLogAccess`	Enables direct read access to your own audit log data using the SAP Audit Log Retrieval API v2.	Provisioning Updating	Audit Log Access [page 3477]

> **Note**
>
> [3] Not available in the Build Runtime: Google Cloud (`build-runtime-gcp`) plan.

> **Note**
>
> [4] Not available in the Build Runtime: Alibaba Cloud (`build-runtime-alicloud`) plan.

Kyma Test Demo and Development (Azure Lite)

The technical name of the plan is `azure_lite`. The Kyma Test Demo and Development plan is offered to Partners, within the *Pay-As-You-Go for SAP BTP for cloud test, demo, and development* commercial model. You can use this plan for testing, development, and demo purposes. In the SAP BTP cockpit, the plan is called "Kyma Runtime Partner TDD".

> **→ Remember**
>
> The parameters marked with an asterisk "*" are mandatory.

Kyma Test Demo and Development Plan Specification

Parameter	Description	Supported Operation	More Information
*Cluster Name** btp CLI parameter: `name`	Defines the name of your cluster.	Provisioning	Cluster Name* [page 3480]
*Region** btp CLI parameter: `region`	Defines a region (set of data-centers) where your cluster runs.	Provisioning	Region* [page 3498]
Machine Type btp CLI parameter: `machineType`	Specifies the provider-specific virtual machine type.	Provisioning Updating	Machine Type [page 3481]
Auto Scaler Min btp CLI parameter: `autoScalerMin`	Specifies the minimum number of virtual machines to create.	Provisioning Updating	Auto Scaler Min [page 3479]

Parameter	Description	Supported Operation	More Information
Auto Scaler Max btp CLI parameter: `autoScalerMax`	Specifies the maximum number of virtual machines to create.	Provisioning Updating	Auto Scaler Max [page 3478]
Modules btp CLI parameter: `modules`	Defines which Kyma modules are provisioned in your cluster.	Provisioning	Modules [page 3488]
Networking btp CLI parameter: `networking`	Provides a custom IP range for worker nodes.	Provisioning	Networking [page 3491]
OpenID Connect btp CLI parameter: `oidc`	Provides a custom Open ID Connect (OIDC) configuration.	Provisioning Updating	OpenID Connect (OIDC) [page 3494]
Administrators btp CLI parameter: `administrators`	Specifies the list of runtime administrators.	Provisioning Updating	Administrators [page 3476]
Additional Worker Node Pools btp CLI parameter: `additionalWorkerNodePools`	Defines a custom list of additional worker node pools.	Provisioning Updating	Additional Worker Node Pools [page 3473]
Colocate Control Plane btp CLI parameter: `colocateControlPlane`	Allows you to have both the control plane and worker nodes in the same region.	Provisioning	Colocate Control Plane [page 3480]
Audit Log Access btp CLI parameter: `auditLogAccess`	Enables direct read access to your own audit log data using the SAP Audit Log Retrieval API v2.	Provisioning Updating	Audit Log Access [page 3477]

Related Information

Provisioning and Updating Parameters in the Kyma Environment [page 3470]
Regions for the Kyma Environment [page 51]
Kyma Modules [page 88]
Account Administration Using the SAP BTP Command Line Interface (btp CLI) [page 2664]
Service Plans and Metering for Kyma Runtime [page 103]

What Is the Consumption-Based Commercial Model? [page 114]

6.4.4.1 Updating Kyma Environment Plans

Update standard enterprise Kyma environment plans to build runtime plans.

Prerequisites

- Assign entitlements for Kyma runtime with the `build-runtime-aws`, `build-runtime-gcp`, or `build-runtime-azure` plan to your subaccount. For more information, see Configure Entitlements and Quotas for Subaccounts.

Context

You can switch from standard enterprise plans to build runtime plans within the same cloud provider. Options include:

- `aws` → `build-runtime-aws`
- `gcp` → `build-runtime-gcp`
- `azure` → `build-runtime-azure`

> **Note**
>
> You can't switch from build runtime plans to standard ones.

To switch plans, you must update your environment instance.

Procedure

1. In the SAP BTP cockpit, navigate to your subaccount *Instances and Subscriptions*.
2. Go to *Environments*, and from the three-dot menu at the end of the Kyma environment row, select *Update*.
3. In the *Update Instance* window, choose the new plan, and select *Update Instance*.

Results

You have updated your Kyma environment service plan.

Related Information

What Is SAP Build?

6.4.4.2 Assigning Kyma Instances to IaaS Provider Accounts

Assignment rules for SAP BTP, Kyma runtime to IaaS provider accounts determine how Kyma instances are mapped to specific cloud provider account pools based on provider type, plan, and, in some cases, region.

Each IaaS provider (Amazon Web Services, Google Cloud, Microsoft Azure, SAP Cloud Infrastructure) provides pools of accounts to which your Kyma instances are assigned. The assignment is primarily based on the IaaS provider type. This means that Kyma instances in one global account are assigned to a single account of a particular IaaS provider. The IaaS provider corresponds to a Kyma instance's plan and, in some cases, other requirements, such as EU Access. For example, a Kyma instance with the standard `aws` plan is assigned to an Amazon Web Services account, and a Kyma instance with the `build-runtime-azure` plan is assigned to a Microsoft Azure account.

IaaS Provider Account Assignment Rules

The specific assignment options are the following:

- Amazon Web Services offers the following account pools:
  - An account pool for Kyma instances created with the service plans `aws`, `build-runtime-aws`, or `free`
  - An account pool for Kyma instances created with the service plans `aws` or `build-runtime-aws` in the subaccount region cf-eu11, Europe (Frankfurt) EU Access
  - A pool of shared accounts for trial Kyma instances
- Microsoft Azure offers the following account pools:
  - An account pool for Kyma instances created with the service plans `azure`, `build-runtime-azure`, test demo and development plan `azure-lite`, and `free`
  - An account pool for Kyma instances created with the service plans `azure` or `build-runtime-azure` in the subaccount region cf-ch20, Switzerland (Zurich) EU Access
- Google Cloud offers the following account pools:
  - An account pool for Kyma instances created with the service plans `gcp` or `build-runtime-gcp`
  - An account pool for Kyma instances created with the service plans `gcp` or `build-runtime-gcp` in the subaccount region cf-sa30, KSA (Dammam) GCP public sector, requiring Assured Workloads Kingdom of Saudi Arabia (KSA) control package.
- SAP Cloud Infrastructure offers one pool of shared accounts for each of the cluster regions available for the service plan `sap-converged-cloud`

For example, once you create the first Kyma instance with the standard `aws` plan in a non-EU region in your global account, it is automatically assigned to an account provided by Amazon Web Services. After that, all your Kyma instances in all subaccounts within the same global account created with the `aws` or `build-`

`runtime-aws` plans in non-EU regions are assigned to the same Amazon Web Services account. Suppose you have no Kyma instances created in the `aws` EU region in the same global account. In that case, this is your only assignment to an Amazon Web Services account, regardless of the number of existing Kyma instances with the standard `aws` plan. However, if, in the same global account, there are also Kyma instances created with the `aws` or `build-runtime-aws` plans in the `aws` EU region, they are all assigned to another single account provided by Amazon Web Services. Therefore, your Kyma instances in this particular global account are assigned to two IaaS provider accounts. Adding more Kyma instances with other plans results in more assignments to the corresponding IaaS provider accounts.

Optimized Cluster Assignment for Amazon Web Services Accounts

To optimize resource consumption and avoid exceeding Amazon Web Services limits in your global account, your Kyma instances are created in accounts that support up to 100 clusters. When an account reaches this limit, new Kyma instances are automatically assigned to a different Amazon Web Services account. When multiple Amazon Web Services accounts are available in your global account, new instances are placed in the account with the most clusters until that account reaches the limit.

IaaS Account Assignments After Subaccount Transfers to Another Global Account

Within SAP Business Technology Platform (BTP), you can transfer solutions from one global account to another. See 2498151. However, the transfer does not affect your Kyma instance(s) assignment to IaaS provider accounts. The assignment is permanent and cannot be changed. This is also true when you migrate subaccounts with Kyma instances assigned to a specific IaaS provider account from one global account to another. The original account assignments remain unchanged even if the migrated Kyma instances and those existing in the target global account have the same characteristics.

For example, if you migrate Kyma instances created with the standard service plan `aws`, all assigned to a single account provided by Amazon Web Services, to another global account with existing Kyma instances also created with the standard service plan `aws`, all assigned to another account provided by Amazon Web Services, the assignments remain unchanged. Even though all your Kyma instances in this particular global account share the same characteristics, they remain assigned to two different IaaS provider accounts.

6.4.5 Provisioning and Updating Parameters in the Kyma Environment

When creating a Kyma cluster, you can configure various parameters to adjust it to your specific needs.

Overview

To configure the cluster parameters, you can use your preferred interface, the SAP BTP cockpit, or the SAP BTP command line interface (btp CLI).

To check which parameters are available for configuration in a particular plan, see Available Plans in the Kyma Environment [page 3459].

Access Control List

Access Control List (`accessControlList`) specifies the IP ranges that can access the Kubernetes API. Internally, the list of IP ranges includes additional entries necessary for the continuous operation of your cluster.

> ⚠ Caution
>
> Enabling *Access Control List* blocks access to Kyma dashboard. You can then manage your cluster only through kubectl.

Access Control List Parameter

Parameter	Supported Operation	Default Value	Allowed Input
Access Control List btp CLI parameter: `accessControlList`	Provisioning Updating	By default, the feature is disabled, and your cluster is created with no IP restrictions.	See the Access Control List Nested Parameters table.

> → Remember
>
> The parameters marked with an asterisk "*" are mandatory.

Access Control List Nested Parameters

Nested Parameter	Default Value	Allowed Input
*Allowed CIDRs**	Empty list (`[]`)	List of IP ranges or an empty list.
btp CLI parameter: `allowedCIDRs`		
type: list of strings		

Configuration

To define your access control list, provide the `accessControlList` parameter with `allowedCIDRs` listing any correct IP ranges in the provisioning request. See the example configuration:

```
"accessControlList": {
    "allowedCIDRs": ["1.2.3.0/24", "2.3.4.0/24"]
    }
```

To modify the set of IP ranges after the cluster is provisioned, send an update request with the new set of IP ranges. If the update request does not contain the `accessControlList` parameter, the existing access control list remains unchanged.

To remove your access control list, set `allowedCIDRs` to an empty list.

```
"accessControlList": {
    "allowedCIDRs": []
```

```
    }
```

Additional Volume Size

With the *Additional Volume Size* (`additionalVolumeSizeGi`) parameter, you can add extra disk space on top of the default volume size for your worker nodes. The total volume size is the sum of the default volume size and `additionalVolumeSizeGi`.

Additional Volume Size Parameter

Parameter	Supported Operation	Default Value	Allowed Input
Additional Volume Size btp CLI parameter: `additionalVolumeSizeGi` type: integer	Provisioning Updating	0	Integer between 0 and 100.

Configuration

> → Tip
>
> Before requesting additional disk space, check your current default volume size. See Machine Type [page 3481].

You can set `additionalVolumeSizeGi` on the main Kyma worker pool and on additional worker node pools. See Additional Worker Node Pools [page 3473].

To add extra disk space to the main Kyma worker pool, set `additionalVolumeSizeGi` at the root level of the request.

```
"additionalVolumeSizeGi": 50
```

To add extra disk space to an additional worker node pool, set `additionalVolumeSizeGi` within the respective entry in the `additionalWorkerNodePools` array.

```
{
  "additionalWorkerNodePools": [
    {
      "name": "worker-1",
      "machineType": "Standard_D4s_v5",
      "haZones": true,
      "autoScalerMin": 3,
      "autoScalerMax": 10,
      "additionalVolumeSizeGi": 50
    }
  ]
}
```

When updating an existing cluster, the behavior differs depending on the worker pool:

- Main Kyma worker pool: If `additionalVolumeSizeGi` changes, the total volume size is recomputed as the sum of the default volume size and the new `additionalVolumeSizeGi` value. If you don't include `additionalVolumeSizeGi` in the update request, the existing volume is preserved.
- Additional worker node pools: To update the additional volume size of an existing pool, you must explicitly provide the new `additionalVolumeSizeGi` value in the update request. To remove the additional volume size, either omit `additionalVolumeSizeGi` from the update request or set it to `0`.

Additional Worker Node Pools

With the *Additional Worker Node Pools* (`additionalWorkerNodePools`), parameter, you can add customized worker node pools to your Kyma runtime and introduce worker nodes optimized and reserved for your particular workload requirements.

Additional Worker Node Pools Parameter

Parameter	Supported Operation	Default Value	Allowed Input
Additional Worker Node Pools btp CLI parameter: `additionalWorkerNodePools` type: array	Provisioning Updating	None (if not provided, no additional worker node pools are created)	See the Additional Worker Node Pools Nested Parameters table.

> → Remember
>
> The parameters marked with an asterisk "*" are mandatory.

Additional Worker Node Pools Nested Parameters

Nested Parameter	Description	Supported Operation	Allowed Input
*Name** btp CLI parameter: `name` type: string	Specifies the unique name of your additional worker node pool.	Provisioning Updating	Short string of up to 15 characters that contains only alphanumeric lowercase characters (a-z, 0–9), and hyphens (-). It must begin and end with an alphanumeric character, and can't contain whitespace characters.
*Machine Type** btp CLI parameter: `machineType` type: string	Specifies the provider-specific virtual machine type.	Provisioning Updating [1] [page 3475]	See Machine Type [page 3481].

Nested Parameter	Description	Supported Operation	Allowed Input
*High Availability Zones** btp CLI parameter: `haZones` type: boolean	Specifies if high availability zones are supported. This setting is permanent and cannot be updated [2] [page 3475]. If enabled, your resources are distributed across three zones to enhance fault tolerance. If you disable high availability, all resources are placed in a single, randomly selected zone. Disabling `haZones` is not recommended for production environments. High availability is not supported in the `azure_lite` plan.	Provisioning Updating [2] [page 3475]	`true` or `false`
*Auto Scaler Min** btp CLI parameter: `autoScalerMin` type: integer	Specifies the minimum number of virtual machines to create.	Provisioning Updating	With high availability disabled, you can set it to `0`. With high availability enabled, you must set it to at least `3`. You can also set it to `0` in `azure_lite` because the plan does not support high availability. See Auto Scaler Min [page 3479].
*Auto Scaler Max** btp CLI parameter: `autoScalerMax` type: integer	Specifies the maximum number of virtual machines to create.	Provisioning Updating	With high availability disabled, you can set it to `1`. With high availability enabled, you must set it to at least `3`. You can also set it to `1` in `azure_lite` because the plan does not support high availability. See Auto Scaler Max [page 3478].

Nested Parameter	Description	Supported Operation	Allowed Input
Taints btp CLI parameter: `taints` type: list	Specifies which workloads are scheduled to nodes in a given worker pool.	Provisioning Updating	See Taints Configuration [page 3502].
Additional Volume Size btp CLI parameter: `additionalVolumeSizeGi`	Specifies extra disk space added on top of the default volume size for a worker node pool. See Additional Volume Size [page 3472].	Provisioning Updating	Integer between 0 and 100.
Annotations btp CLI parameter: `annotations`	Attaches arbitrary non-identifying metadata to worker nodes, such as tooling configuration or operational notes.	Provisioning Updating	Key-value pairs where each key is a non-empty string and each value is a string.
Labels btp CLI parameter: `labels`	Attaches identifying metadata to worker nodes that you can use to identify, filter, and organize them.	Provisioning Updating	Key-value pairs where each key is a non-empty string and each value is a string.

[1] You can update your virtual machine type only within the general-purpose machine types. You cannot perform updates on compute-intensive machine types. For details, see Machine Type [page 3481].

[2] You can only use this parameter to update your Kyma runtime by creating a new additional worker node pool. You cannot use it to update an existing additional worker node pool.

Configuration

If you do not provide the `additionalWorkerNodePools` array in the provisioning request, no additional worker node pools are created.

If you do not provide the `additionalWorkerNodePools` array in the update request, the saved additional worker node pools stay unchanged. However, if you provide an empty array in the update request, all existing additional worker node pools are removed. If you rename your existing additional worker node pool, it is deleted and a new one is created.

See also Assigning Workloads to Worker Node Pools [page 3522].

When updating optional parameters of an existing additional worker node pool, the following rules apply:

- If you omit an optional field in the update request, the existing setting for that pool is removed.
- If you set an optional field to an empty object ([]), the existing setting for that pool is also removed.
- To update an optional field, provide the new setting - the update overwrites the existing setting for that pool.

See the example configuration:

Sample Code

```
{
  "additionalWorkerNodePools": [
```

```
    {
      "name": "worker-1",
      "machineType": "Standard_D2s_v5",
      "haZones": true,
      "autoScalerMin": 3,
      "autoScalerMax": 20,
      "labels": {
        "env": "prod",
        "team": "platform"
      },
      "annotations": {
        "owner": "team-platform",
        "cost-center": "12345"
      }
    },
    {
      "name": "worker-2",
      "machineType": "Standard_D4s_v5",
      "haZones": false,
      "autoScalerMin": 1,
      "autoScalerMax": 1,
      "taints": [
        {
          "key": "dedicated",
          "value": "gpu",
          "effect": "NoSchedule"
        },
        {
          "key": "dedicated",
          "value": "gpu",
          "effect": "NoExecute"
        }
      ]
    }
  ]
}
```

See also Assigning Workloads to Worker Node Pools [page 3522].

Administrators

With the *Administrators* (`administrators`) parameter, you can define the list of runtime administrators.

Administrators Parameter

Parameter	Supported Operations	Default Value	Allowed Input
Administrators btp CLI name: `administrators` type: an array of strings	Provisioning Updating	If you don't provide the parameter or provide an empty array during provisioning, the email address of the provisioning user is used.	A list of administrators' email addresses.

Configuration

See an example of the JSON input:

Sample Code

```
"administrators": [
        "example_1@mail.com",
        "example_2@mail.com",
        "example_3@mail.com"
    ]
```

If you don't include `administrators` or provide an empty array in an update request, the existing list of administrators remains unchanged.

To revoke all administrators, set the parameter to a list with a single entry. The entry does not have to correspond to an existing user.

Audit Log Access

With the *Audit Log Access* (`auditLogAccess`) parameter, you can gain direct read access to your own audit log data using the SAP Audit Log Retrieval API v2. When you enable the parameter, Kyma stores the required credentials in a Kubernetes Secret named `auditlog-read-credentials` in the `kyma-system` namespace of your cluster.

Caution

Enabling *Audit Log Access* is irreversible. If you enable the feature, you cannot disable it.

When you enable *Audit Log Access* during an update, you can only access the logs created after enabling the feature.

Audit Log Access Parameter

Parameter	Supported Operations	Default Value	Allowed Input
Audit Log Access btp CLI name: `auditLogAccess` type: boolean	Provisioning Updating	false	`true` or `false`

See an example of the JSON input:

Sample Code

```
"auditLogAccess": true
```

See also Accessing Your Audit Log Data [page 3501].

Auto Scaler Max

With the *Auto Scaler Max* (`autoScalerMax`) integer parameter, you can set the maximum number of virtual machines to create.

> ⚠ Caution
>
> Cluster autoscaling is not subject to the Service Level Agreement (SLA). Successful autoscaling is not guaranteed. The mechanism may fail or take longer than expected due to constraints of the underlying cloud providers.

Auto Scaler Max Parameter

Plan	Supported Operation	Default Value	Maximum Value	Allowed Input
Standard: • Amazon Web Services (`aws`) • Google Cloud (`gcp`) • Microsoft Azure (`azure`) • Alibaba Cloud (`alicloud`) Build Runtime: • Amazon Web Services (`build-runtime-aws`) • Google Cloud (`build-runtime-gcp`) • Microsoft Azure (`build-runtime-azure`)	Provisioning Updating	20	300	Number between 3 and 300, but greater than or equal to *Auto Scaler Min*. Within the *Additional Worker Node Pools* array, with high availability disabled, you can set it to `1`. See Additional Worker Node Pools [page 3473].
Kyma Test Demo and Development (Azure Lite) technical name: `azure_lite`	Provisioning Updating	10	40	Number between 2 and 40, but greater than or equal to *Auto Scaler Min*. Within the *Additional Worker Node Pools* array, you can set it to `1`. See Additional Worker Node Pools [page 3473].
SAP Cloud Infrastructure technical name: `sap-converged-cloud`	Provisioning Updating	20	300	Number between 3 and 300, but greater than or equal to *Auto Scaler Min*. Within the *Additional Worker Node Pools* array, with high availability disabled, you can set it to `1`. See Additional Worker Node Pools [page 3473].

See the default JSON input:

> {·} Sample Code

```
"autoScalerMax": 20
```

Auto Scaler Min

With the *Auto Scaler Min* (`autoScalerMin`) integer parameter, you can set the minimum number of virtual machines to create.

Auto Scaler Min Parameter

Plan	Supported Operation	Default Value	Minimum Value	Allowed Input
Standard: • Amazon Web Services (`aws`) • Google Cloud (`gcp`) • Microsoft Azure (`azure`) • Alibaba Cloud (`alicloud`) Build Runtime: • Amazon Web Services (`build-runtime-aws`) • Google Cloud (`build-runtime-gcp`) • Microsoft Azure (`build-runtime-azure`)	Provisioning Updating	3	3	Number between 3 and the current value set in *Auto Scaler Max*. Within the *Additional Worker Node Pools* array, with high availability disabled, you can set it to `0`. See Additional Worker Node Pools [page 3473].
Kyma Test Demo and Development (Azure Lite) technical name: `azure_lite`	Provisioning Updating	2	2	Number between 2 and the current value set in *Auto Scaler Max*. Within the *Additional Worker Node Pools* array, you can set it to `0`. See Additional Worker Node Pools [page 3473].

Plan	Supported Operation	Default Value	Minimum Value	Allowed Input
SAP Cloud Infrastructure technical name: `sap-converged-cloud`	Provisioning Updating	3	3	Number between 3 and the current value set in *Auto Scaler Max*. Within the *Additional Worker Node Pools* array, with high availability disabled, you can set it to `0`. See Additional Worker Node Pools [page 3473].

See an example of the JSON input:

Sample Code

```
"autoScalerMin": 3
```

Cluster Name*

Use the *Cluster Name* (`name`) parameter to set the name of your cluster.

→ Remember

The parameters marked with an asterisk "*" are mandatory.

Cluster Name Parameter

Parameter	Supported Operation	Default Value	Allowed Input
*Cluster Name** btp CLI parameter: `name` type: string	Provisioning Updating (for updates, the *Cluster Name* parameter is optional)	Defaults to your subaccount's subdomain.	Short string of 1 to 64 alphanumeric characters (A-Z, a-z, 0–9) and hyphens (-). It must not contain whitespace characters.

See also Tracking Kubeconfig and Cluster Associations in Kyma [page 3454].

Colocate Control Plane

With the *Colocate Control Plane* (`colocateControlPlane`) parameter, you can specify that your control plane and worker nodes are in the same region.

Colocate Control Plane Parameter

Parameter	Supported Operations	Default Value	Allowed Input
Colocate Control Plane btp CLI name: `colocateControlPlane` type: boolean	Provisioning	`false`	`true` (control plane in the same region as cluster's worker nodes) `false` (control plane can be deployed in a different region than worker nodes)

Configuration

If you set it to `true`, it ensures the location of the control plane in the same region where your cluster's worker nodes are deployed. With this setting, you can control where your sensitive data is stored. If the control plane cannot be colocated in the selected region, the provisioning process fails. The error message offers you a list of regions supporting the control plane colocation.

If you set the parameter to `false` or leave the field empty, your control plane can sometimes be deployed in a different region than the worker nodes.

To learn which regions support the control plane colocation, see the Region* [page 3498] section.

See an example of the JSON input:

Sample Code

```
"colocateControlPlane": true
```

Machine Type

With the *Machine Type* (`machineType`) parameter, you can specify the IaaS provider-specific virtual machine type.

Machine Type Parameter

Parameter	Supported Operations	Default Value	Allowed Input
Machine Type btp CLI parameter: `machineType` type: string	Provisioning Updating	Varies by IaaS provider and service plan. Expand the table for your preferred provider to see the default.	Varies by IaaS provider and service plan. Expand the table for your preferred provider for the full list. To ensure smooth updates and avoid disruptions during upgrades, choose version-agnostic machine type names, such as `Standard_D2s` or `mi.large`. The version-agnostic machine type name represents the underlying instance family that powers this machine type. The most optimized underlying instance families are assigned to the version-agnostic machine type names and can be updated to newer generations during maintenance windows without affecting your configurations. Where present, the parenthetical values in the tables show the specific instance types that are actually provisioned.

See an example input for the *Machine Type* parameter:

Sample Code

```
"machineType": "mi.large"
```

The available categories of machine types are the following:

- General-purpose — available for use in both the mandatory Kyma worker node pool and in your additional worker node pools.
- Compute-intensive — available for use only in additional worker node pools. See Machine Types in Additional Worker Node Pools [page 3486].

To see the general-purpose machine types available for specific service plans, expand the table for your preferred IaaS provider.

Amazon Web Services

You can use the machine types listed in the table with the following plans:

- Standard: Amazon Web Services (technical name: `aws`)
- Build Runtime: Amazon Web Services (technical name: `build-runtime-aws`)

The `m5` and `m6i` machine types have been deprecated and can no longer be provisioned. To avoid breaking existing configurations, these inputs are automatically mapped to the equivalent `m7i` generation, so you do not need to adjust your configurations.

AWS Machine Types

Default Value	Allowed Input (Resolved Machine Type)	Virtual Machine Size	Default Volume Size
`mi.large`	`mi.large` (`m7i.large`)	2 vCPU, 8 GB RAM	80 Gi
	`mi.xlarge` (`m7i.xlarge`)	4 vCPU, 16 GB RAM	80 Gi
	`mi.2xlarge` (`m7i.2xlarge`)	8 vCPU, 32 GB RAM	80 Gi
	`mi.4xlarge` (`m7i.4xlarge`)	16 vCPU, 64 GB RAM	94 Gi
	`mi.8xlarge` (`m7i.8xlarge`)	32 vCPU, 128 GB RAM	158 Gi
	`mi.12xlarge` (`m7i.12xlarge`)	48 vCPU, 192 GB RAM	222 Gi
	`mi.16xlarge` (`m7i.16xlarge`)	64 vCPU, 256 GB RAM	250 Gi
	`m6i.large` (`m7i.large`)	2 vCPU, 8 GB RAM	80 Gi
	`m6i.xlarge` (`m7i.xlarge`)	4 vCPU, 16 GB RAM	80 Gi
	`m6i.2xlarge` (`m7i.2xlarge`)	8 vCPU, 32 GB RAM	80 Gi
	`m6i.4xlarge` (`m7i.4xlarge`)	16 vCPU, 64 GB RAM	94 Gi
	`m6i.8xlarge` (`m7i.8xlarge`)	32 vCPU, 128 GB RAM	158 Gi
	`m6i.12xlarge` (`m7i.12xlarge`)	48 vCPU, 192 GB RAM	222 Gi
	`m6i.16xlarge` (`m7i.16xlarge`)	64 vCPU, 256 GB RAM	250 Gi
	`m5.large` (`m7i.large`)	2 vCPU, 8 GB RAM	80 Gi
	`m5.xlarge` (`m7i.xlarge`)	4 vCPU, 16 GB RAM	80 Gi
	`m5.2xlarge` (`m7i.2xlarge`)	8 vCPU, 32 GB RAM	80 Gi
	`m5.4xlarge` (`m7i.4xlarge`)	16 vCPU, 64 GB RAM	94 Gi
	`m5.8xlarge` (`m7i.8xlarge`)	32 vCPU, 128 GB RAM	158 Gi
	`m5.12xlarge` (`m7i.12xlarge`)	48 vCPU, 192 GB RAM	222 Gi
	`m5.16xlarge` (`m7i.16xlarge`)	64 vCPU, 256 GB RAM	250 Gi

Google Cloud

You can use the machine types listed in the table with the following plans:

- Standard: Google Cloud (technical name: `gcp`)
- Build Runtime: Google Cloud (technical name: `build-runtime-gcp`)

Google Cloud Machine Types

Default Value	Allowed Input	Virtual Machine Size	Default Volume Size
`n2-standard-2`	`n2-standard-2`	2 vCPU, 8 GB RAM	80 Gi
	`n2-standard-4`	4 vCPU, 16 GB RAM	80 Gi
	`n2-standard-8`	8 vCPU, 32 GB RAM	80 Gi
	`n2-standard-16`	16 vCPU, 64 GB RAM	94 Gi
	`n2-standard-32`	32 vCPU, 128 GB RAM	158 Gi
	`n2-standard-48`	48 vCPU, 192 GB RAM	222 Gi
	`n2-standard-64`	64 vCPU, 256 GB RAM	250 Gi

Microsoft Azure

You can use the machine types listed in the table with the following plans:

- Standard: Microsoft Azure (technical name: `azure`)
- Build Runtime: Microsoft Azure (technical name: `build-runtime-azure`)

The `Standard_D_v3` machine types have been deprecated by Microsoft Azure and can no longer be provisioned. To avoid breaking existing configurations, these inputs are automatically mapped to the equivalent `Standard_Ds_v5` generation, so you do not need to adjust your configurations.

Microsoft Azure Machine Type Parameter

Default Value	Allowed Input (Resolved Machine Type)	Virtual Machine Size	Default Volume Size
`Standard_D2s`	`Standard_D2s` (`Standard_D2s_v5`)	2 vCPU, 8 GB RAM	80 Gi
	`Standard_D4s` (`Standard_D4s_v5`)	4 vCPU, 16 GB RAM	80 Gi
	`Standard_D8s` (`Standard_D8s_v5`)	8 vCPU, 32 GB RAM	80 Gi
	`Standard_D16s` (`Standard_D16s_v5`)	16 vCPU, 64 GB RAM	94 Gi
	`Standard_D32s` (`Standard_D32s_v5`)	32 vCPU, 128 GB RAM	158 Gi
	`Standard_D48s` (`Standard_D48s_v5`)	48 vCPU, 192 GB RAM	222 Gi
	`Standard_D64s` (`Standard_D64s_v5`)	64 vCPU, 256 GB RAM	250 Gi

Default Value	Allowed Input (Resolved Machine Type)	Virtual Machine Size	Default Volume Size
	`Standard_D2s_v5`	2 vCPU, 8 GB RAM	80 Gi
	`Standard_D4s_v5`	4 vCPU, 16 GB RAM	80 Gi
	`Standard_D8s_v5`	8 vCPU, 32 GB RAM	80 Gi
	`Standard_D16s_v5`	16 vCPU, 64 GB RAM	94 Gi
	`Standard_D32s_v5`	32 vCPU, 128 GB RAM	158 Gi
	`Standard_D48s_v5`	48 vCPU, 192 GB RAM	222 Gi
	`Standard_D64s_v5`	64 vCPU, 256 GB RAM	250 Gi
	`Standard_D4_v3` (`Standard_D4s_v5`)	4 vCPU, 16 GB RAM	80 Gi
	`Standard_D8_v3` (`Standard_D8s_v5`)	8 vCPU, 32 GB RAM	80 Gi
	`Standard_D16_v3` (`Standard_D16s_v5`)	16 vCPU, 64 GB RAM	94 Gi
	`Standard_D32_v3` (`Standard_D32s_v5`)	32 vCPU, 128 GB RAM	158 Gi
	`Standard_D48_v3` (`Standard_D48s_v5`)	48 vCPU, 192 GB RAM	222 Gi
	`Standard_D64_v3` (`Standard_D64s_v5`)	64 vCPU, 256 GB RAM	250 Gi

You can use the machine types listed in the table with the Test Demo and Development (Azure Lite) (technical name: `azure_lite`) plan.

Azure Lite Machine Types

Default Value	Allowed Input (Resolved Machine Type)	Virtual Machine Size	Default Volume Size
`Standard_D4s`	`Standard_D2s` (`Standard_D2s_v5`)	2 vCPU, 8 GB RAM	80 Gi
	`Standard_D4s` (`Standard_D4s_v5`)	4 vCPU, 16 GB RAM	80 Gi
	`Standard_D2s_v5`	2 vCPU, 8 GB RAM	80 Gi
	`Standard_D4s_v5`	4 vCPU, 16 GB RAM	80 Gi
	`Standard_D4_v3` (`Standard_D4s_v5`)	4 vCPU, 16 GB RAM	80 Gi

SAP Cloud Infrastructure

You can use the machine types listed in the table with the SAP Cloud Infrastructure (`sap-converged-cloud`) plan.

SAP Cloud Infrastructure Machine Types

Default Value	Allowed Input	Virtual Machine Size	Default Volume Size
`g_c2_m8`	`g_c2_m8`	2 vCPU, 8 GB RAM	80 Gi
	`g_c4_m16`	4 vCPU, 16 GB RAM	80 Gi
	`g_c6_m24`	6 vCPU, 24 GB RAM	80 Gi
	`g_c8_m32`	8 vCPU, 32 GB RAM	80 Gi
	`g_c12_m48`	12 vCPU, 48 GB RAM	80 Gi
	`g_c16_m64`	16 vCPU, 64 GB RAM	94 Gi
	`g_c32_m128`	32 vCPU, 128 GB RAM	158 Gi
	`g_c64_m256`	64 vCPU, 256 GB RAM	250 Gi

Alibaba Cloud

You can use the machine types listed in the table with the standard Alibaba Cloud (technical name: `alicloud`) plan.

Alibaba Cloud Machine Types

Default Value	Allowed Input	Virtual Machine Size	Default Volume Size
`ecs.g9i.large`	`ecs.g9i.large`	2 vCPU, 8GB RAM	80 Gi
	`ecs.g9i.xlarge`	4 vCPU, 16GB RAM	80 Gi
	`ecs.g9i.2xlarge`	8 vCPU, 32GB RAM	80 Gi
	`ecs.g9i.4xlarge`	16 vCPU, 64GB RAM	94 Gi
	`ecs.g9i.8xlarge`	32 vCPU, 128GB RAM	158 Gi
	`ecs.g9i.12xlarge`	48 vCPU, 192GB RAM	222 Gi
	`ecs.g9i.16xlarge`	64 vCPU, 256GB RAM	250 Gi

Machine Types in Additional Worker Node Pools

In your additional worker node pools, you can use both the general-purpose and compute-intensive virtual machines. To see the compute-intensive machine types available for specific service plans, expand the table for your preferred IaaS provider.

Amazon Web Services

You can use the machine types listed in the table with the following plans:

- Standard: Amazon Web Services (technical name: `aws`)
- Build Runtime: Amazon Web Services (technical name: `build-runtime-aws`)

AWS Machine Types in Additional Worker Node Pools

Allowed Input (Resolved Machine Type)	Virtual Machine Size	Default Volume Size	Availability Regions

`c7i.large`	2 vCPU, 4 GB RAM	80 Gi	`eu-central-1`
			`eu-west-2`
			`eu-south-1`
			`ca-central-1`
			`sa-east-1`
			`us-east-1`
			`ap-northeast-1`
			`ap-northeast-2`
			`ap-south-1`
			`ap-southeast-1`
			`ap-southeast-2`
			`us-west-2`

`c7i.xlarge`	4 vCPU, 8 GB RAM	80 Gi	
`c7i.2xlarge`	8 vCPU, 16 GB RAM	80 Gi	
`c7i.4xlarge`	16 vCPU, 32 GB RAM	94 Gi	
`c7i.8xlarge`	32 vCPU, 64 GB RAM	158 Gi	
`c7i.12xlarge`	48 vCPU, 96 GB RAM	222 Gi	
`c7i.16xlarge`	64 vCPU, 128 GB RAM	250 Gi	

`ri.large` (`r8i.large`)	2 vCPU, 16 GB RAM	80 Gi	`eu-central-1`
			`eu-west-2`
			`us-east-1`
			`us-west-2`
			`ap-south-1`
			`ap-southeast-2`

`ri.xlarge` (`r8i.xlarge`)	4 vCPU, 32 GB RAM	80 Gi	
`ri.2xlarge` (`r8i.2xlarge`)	8 vCPU, 64 GB RAM	94 Gi	
`ri.4xlarge` (`r8i.4xlarge`)	16 vCPU, 132 GB RAM	158 Gi	
`ri.8xlarge` (`r8i.8xlarge`)	32 vCPU, 256 GB RAM	250 Gi	
`ri.12xlarge` (`r8i.12xlarge`)	48 vCPU, 384 GB RAM	250 Gi	
`ri.16xlarge` (`r8i.16xlarge`)	64 vCPU, 512 GB RAM	250 Gi	

Google Cloud

You can use the machine types listed in the table with the following plans:

- Standard: Google Cloud (technical name: `gcp`)
- Build Runtime: Google Cloud (technical name: `build-runtime-gcp`)

Google Cloud Machine Types in Additional Worker Node Pools

Allowed Input (Resolved Machine Type)	Virtual Machine Size	Default Volume Size	Availability Regions

`c2d-highcpu-2`	2 vCPU, 4 GB RAM	80 GI	`europe-west3`
			`europe-west4`
			`us-central1`

`c2d-highcpu-4`	4 vCPU, 8 GB RAM	80 Gi	
`c2d-highcpu-8`	8 vCPU, 16 GB RAM	80 Gi	
`c2d-highcpu-16`	16 vCPU, 32 GB RAM	94 Gi	

Allowed Input (Resolved Machine Type)	Virtual Machine Size	Default Volume Size	Availability Regions

`c2d-highcpu-32`	32 vCPU, 64 GB RAM	158 Gi	`us-west1`
			`us-east4`
			`asia-southeast1`
			`asia-south1`

`c2d-highcpu-56`	56 vCPU, 112 GB RAM	250 Gi	

Microsoft Azure

You can use the machine types listed in the table with the following plans:

- Standard: Microsoft Azure (technical name: `azure`)
- Build Runtime: Microsoft Azure (technical name: `build-runtime-azure`)

Microsoft Azure Machine Types in Additional Worker Node Pools

Allowed Input (Resolved Machine Type)	Virtual Machine Size	Default Volume Size	Availability Regions
`Standard_F2s_v2`	2 vCPU, 4 GB RAM	80 Gi	All Microsoft Azure regions. See Region* [page 3498].
`Standard_F4s_v2`	4 vCPU, 8 GB RAM	80 Gi	
`Standard_F8s_v2`	8 vCPU, 16 GB RAM	80 Gi	
`Standard_F16s_v2`	16 vCPU, 32 GB RAM	94 Gi	
`Standard_F32s_v2`	32 vCPU, 64 GB RAM	158 Gi	
`Standard_F48s_v2`	48 vCPU, 96 GB RAM	222 Gi	
`Standard_F64s_v2`	64 vCPU, 128 GB RAM	250 Gi	

Modules

With the *Modules* (`modules`) object, you can define which Kyma modules you want to provision in your cluster. You can also use it to create a cluster without any modules.

API for module configuration is built on the `oneOf` feature from the JSON schema. If the `modules` object is passed to API, it must have only one valid option: *Default* (`default`) or *Custom* (`list`).

Even if you omit the `modules` object from the request, the default Kyma modules are provisioned in your cluster.

> → Remember
>
> The parameters marked with an asterisk "*" are mandatory.

Modules Nested Parameters

Nested Parameter	Description	Supported Operation	Default Value
Default btp CLI parameter: `default` type: boolean	Defines whether to use the default list of Kyma modules. Check the Default Kyma Modules to find out which modules are in the default modules list.	Provisioning	`true`
Custom btp CLI parameter: `list` type: array of objects	Defines a custom list of Kyma modules. Leave your custom list of Kyma modules empty if you don't want any modules provisioned.	Provisioning	[ ] (empty array)
Default Module Channel btp CLI parameter: `channel` type: string	Specifies your preferred release channel, regular or fast, for the whole default or custom list of modules. If needed, you can override this setting for individual modules in your custom list of modules.	Provisioning	Taken from the `runtimeConfiguration` setting, where the Kyma resource spec channel is specified per plan.

The *Custom* (`list`) parameter includes three nested parameters: *Name**, *Channel*, and *Custom Resource Policy*.

Custom list Parameters

Nested Parameter	Description	Supported Operation	Default Value
*Name** btp CLI parameter: `name` type: string	Defines the module's technical name. To check the available Kyma modules and their technical names, see Kyma Modules [page 88].	Provisioning	n/a
Channel btp CLI parameter: `channel` type: string	Defines the preferred release channel. The default is `regular`. The other option is `fast`. See Kyma Release Channels [page 91].	Provisioning	"" (an empty string)

Nested Parameter	Description	Supported Operation	Default Value
Custom Resource Policy btp CLI parameter: `customResourcePolicy` type: string	Specifies whether Kyma manages the module's default custom resource (CR). If you set `customResourcePolicy` to `CreateAndDelete` or leave the field empty, Kyma configures the module with default settings. The default configuration is defined in the module CR, which you can edit at any time and Kyma doesn't overwrite your changes. When you delete the module, Kyma also removes the module CR. If you set `customResourcePolicy` to `Ignore`, the module is added to your cluster without default configuration. Module CustomResourceDefinitions (CRDs) are not installed until you apply the module CR with your own configuration. For example, you can't create VirtualServices with the Istio module until you apply the Istio CR. Before deleting the module, you must delete the CR first. Use this setting when you want to provide your module configuration from the start, for example in automated provisioning workflows where you manage CRs as YAML files.	Provisioning	"" (an empty string)

Configuration

If you don't provide the `modules` object in the JSON payload, or if you use the following input, you have the default Kyma modules provisioned in your cluster :

Sample Code

```
"modules": {
    "default": true
}
```

See an example of JSON input for a custom list of Kyma modules.

Sample Code

```
"modules": {
    "list": [
        {
            "name": "btp-operator"
        },
        {
            "name": "keda",
            "channel": "regular",
            "customResourcePolicy": "CreateAndDelete"
        }
    ]
```

```
}
```

If you use the following JSON input, Kyma doesn't create the Istio module's default CR and doesn't install Istio CRDs (such as VirtualServices or Gateways). To complete the setup, you must apply your own Istio CR. You can set `customResourcePolicy` to `Ignore` for any Kyma module. To configure the module CR, find reference documentation on kyma-project.io. For example, Istio Custom Resource .

{·} Sample Code

```
"modules": {
    "list": [
        {
            "name": "istio",
            "customResourcePolicy": "Ignore"
        }
    ]
}
```

If you apply the following values, your Kyma runtime is provisioned without any Kyma modules.

{·} Sample Code

```
"modules": {
    "list": []
}
```

{·} Sample Code

```
"modules": {
    "default": false
}
```

Networking

With the *Networking* (`networking`) object, you provide networking configuration. These values are immutable and cannot be updated later.

→ Remember

The parameters marked with an asterisk "*" are mandatory.

Networking Nested Parameters

Nested Parameter	Description	Supported Operation	Default Value	Allowed Input
Enable dual stack btp CLI parameter: `dualStack` type: boolean	Enables dual-stack networking (IPv4 and IPv6). **⚠ Caution** The Istio module does not support the dual-stack mode. For more information, see Kyma Runtime with Dual-Stack Support [page 3533].	Provisioning	`false`	`true` or `false`
*CIDR range for Nodes** btp CLI parameter: `nodes` type: string	Defines a custom IP range for worker nodes.	Provisioning	10.250.0.0/16	The CIDR range for nodes must not overlap with the following CIDRs: 10.242.0.0/16, 10.64.0.0/11, 10.254.0.0/16, 10.243.0.0/16, 192.168.123.0/24, 240.0.0.0/8. Also, the range for nodes must not overlap with those used for Pods or Services. The same restriction applies to the default ranges set for Pods or Services if you don't provide your own.

Nested Parameter	Description	Supported Operation	Default Value	Allowed Input
CIDR range for Pods btp CLI parameter: `pods` type: string	Defines a custom IP range for Pods.	Provisioning	10.96.0.0/13	CIDR range for Pods must not overlap with the following CIDRs: 10.242.0.0/16, 10.64.0.0/11, 10.254.0.0/16, 10.243.0.0/16, 192.168.123.0/24, 240.0.0.0/8. Also, the range for Pods must not overlap with those used for nodes or Services. The same restriction applies to the default range set for Services if you don't provide your own.
CIDR range for Services btp CLI parameter: `services` type: string	Defines a custom IP range for Services.	Provisioning	10.104.0.0/13	CIDR range for Services must not overlap with the following CIDRs: 10.242.0.0/16, 10.64.0.0/11, 10.254.0.0/16, 10.243.0.0/16, 192.168.123.0/24, 240.0.0.0/8. Also, the range for Services must not overlap with those used for nodes or Pods. The same restriction applies to the default range set for Pods if you don't provide your own.

See the default JSON input for the `networking` object:

Sample Code

```
"networking": {
        "nodes": "10.250.0.0/16",
        "pods": "10.96.0.0/13",
        "services": "10.104.0.0/13"
    }
```

OpenID Connect (OIDC)

With the *OpenID Connect* (OIDC) (`oidc`) parameter, you can configure a custom identity provider. You can configure the property in the following ways:

- As a list of `oidc` objects (recommended)
- As a single `oidc` object

OIDC Configured as a List of `oidc` Objects

The*OpenID Connect* property is a list of `oidc` objects. You can use it to configure one or several `oidc` objects.

> → Remember
>
> The parameters marked with an asterisk "*" are mandatory.

OIDC Parameters for a List of oidc Objects

Nested Parameter	Description	Supported Operation	Default Value	Allowed Input
*Client ID** btp CLI parameter: `clientID` type: string	The client ID for the OpenID Connect client.	Provisioning Updating	n/a	n/a
*Issuer URL** btp CLI parameter: `issuerURL` type: string	Provides the URL of the OpenID issuer.	Provisioning Updating	n/a	Only HTTPS scheme is accepted.
*Groups Claim** btp CLI parameter: `groupsClaim` type: string	If provided, specifies the name of a custom OIDC claim for identifying user groups.	Provisioning Updating	n/a	n/a
*Groups Prefix** btp CLI parameter: `groupsPrefix` type: string	If specified, causes claims mapping to group names to be prefixed with the provided value. If not provided, the prefix defaults to `-` (dash character without additional characters), which disables prefixing.	Provisioning Updating	n/a	For example, the value `oidc:` results in groups like `oidc:engineering` and `oidc:marketing`. To skip any prefixing, provide the value `-` (dash character without additional characters).

Nested Parameter	Description	Supported Operation	Default Value	Allowed Input
*Username Claim** btp CLI parameter: `usernameClaim` type: string	Provides the OpenID claim to use as the user name.	Provisioning Updating	n/a	n/a
Required Claims btp CLI parameter: `requiredClaims` type: array	Describes a required claim in the ID Token. If set, the claim is verified to be present in the ID Token with the matching value.	Provisioning Updating	n/a	List of key=value pairs. To remove a previously set value, leave the array empty.
*Signing Algs** btp CLI parameter: `signingAlgs` type: array	Provides the OIDC signing algorithms for Kyma runtime.	Provisioning Updating	n/a	Comma-separated list of allowed JOSE asymmetric signing algorithms, for example, `RS256`, `ES256`.
*Username Prefix** btp CLI parameter: `usernamePrefix` type: string	Provides an OIDC username prefix for Kyma runtime.	Provisioning Updating	n/a	To skip any prefixing, provide the value `-` (dash character without additional characters).
Encoded JWKS Array btp CLI parameter: `encodedJwksArray` type: string	The JSON Web Key Set (JWKS) array encoded in base64. Use it when your OIDC metadata discovery endpoint is inaccessible from the public internet.	Provisioning Updating	n/a	Base64 encoded string. To remove a previously set value, leave the input empty.

The following example shows the default configuration of a list of `oidc` objects. To revert your changes to the default settings, copy and paste the following values:

Sample Code

```
"oidc": {
  "list": [
    {
      "clientID": "12b13a26-d993-4d0c-aa08-5f5852bbdff6",
      "groupsClaim": "groups",
      "issuerURL": "https://kyma.accounts.ondemand.com",
      "groupsPrefix": "-",
      "signingAlgs": ["RS256"],
      "usernameClaim": "sub",
      "usernamePrefix": "-",
      "requiredClaims": [],
      "encodedJwksArray": ""
```

```
    }
  ]
}
```

For more configuration options, see Custom OpenID Connect Configuration [page 3505].

OIDC Configured as a Single `oidc` Object

This approach is not recommended. Use it only to maintain backward compatibility with existing automations.

> → Remember
>
> The parameters marked with an asterisk "*" are mandatory.

OIDC Parameters for a Single oidc Object

Nested Parameter	Description	Supported Operation	Default Value	Allowed Input
*Client ID** btp CLI parameter: `clientID` type: string	The client ID for the OpenID Connect client.	Provisioning Updating	n/a	n/a
*Issuer URL** btp CLI parameter: `issuerURL` type: string	Provides the URL of the OpenID issuer.	Provisioning Updating	n/a	Only HTTPS scheme is accepted.
Groups Claim btp CLI parameter: `groupsClaim` type: string	If provided, specifies the name of a custom OIDC claim for specifying user groups.	Provisioning Updating	`groups`	n/a
Groups Prefix btp CLI parameter: `groupsPrefix` type: string	If specified, causes claims mapping to group names to be prefixed with the provided value. If not provided, the prefix defaults to `-` (dash character without additional characters), and disables prefixing.	Provisioning Updating	`-`	For example, the value `oidc:` results in groups like `oidc:engineering` and `oidc:marketing`. To skip any prefixing, provide the value `-` (dash character without additional characters).

Nested Parameter	Description	Supported Operation	Default Value	Allowed Input
Username Claim btp CLI parameter: `usernameClaim` type: string	Provides the OpenID claim to use as the user name.	Provisioning Updating	n/a	n/a
Required Claims btp CLI parameter: `requiredClaims` type: array	Describes a required claim in the ID Token. If set, the claim is verified to be present in the ID Token with the matching value.	Provisioning Updating	n/a	List of key=value pairs. To remove a previously set value, provide only the value `-` (dash character without additional characters).
Signing Algs btp CLI parameter: `signingAlgs` type: array	Provides the OIDC signing algorithms for Kyma runtime.	Provisioning Updating	RS256	Comma-separated list of allowed JOSE asymmetric signing algorithms, for example, RS256, ES256.
Username Prefix btp CLI parameter: `usernamePrefix` type: string	Provides an OIDC username prefix for Kyma runtime.	Provisioning Updating	-	To skip any prefixing, provide the value `-` (dash character without additional characters).
Encoded JWKS Array btp CLI parameter: `encodedJwksArray` type: string	The JSON Web Key Set (JWKS) array encoded in base64. Use it when your OIDC metadata discovery endpoint is inaccessible from the public internet.	Provisioning Updating	n/a	Base64 encoded string. To remove a previously set value, provide only the value `-` (dash character without additional characters).

The following example shows the default configuration of an `oidc` object. To revert your changes to the default settings, copy and paste the following values:

```
{
  ...
  "oidc" : {
    "clientID" : "12b13a26-d993-4d0c-aa08-5f5852bbdff6",
    "issuerURL" : "https://kyma.accounts.ondemand.com",
    "groupsClaim" : "groups",
    "groupsPrefix" : "-",
    "signingAlgs" : ["RS256"],
    "usernamePrefix" : "-",
    "usernameClaim" : "sub",
    "requiredClaims" : ["-"],
    "encodedJwksArray": "-"
  }
  ...
```

```
}
```

For more configuration options, see Custom OpenID Connect Configuration [page 3505].

Region*

Use the *Region** (`region`) parameter (string) to define a region where your cluster runs.

> → Remember
>
> The parameters marked with an asterisk "*" are mandatory.

Region Parameter

Parameter	Supported Operation	Default Value	Allowed Input
*Region** btp CLI parameter: `region` type: string	Provisioning	None	Varies by plan. Expand the table for your preferred IaaS provider for the full list of available regions.

Here is an example of the JSON input for the *Region* parameter:

> {·} Sample Code
>
> ```
> "region": "us-east-1"
> ```

The available regions vary by IaaS provider. Expand the table for your preferred provider to see the supported regions for specific service plans.

Amazon Web Services

You can use the regions listed in the table with the following plans:

- Standard: Amazon Web Services (`aws`)
- Free (`free`)
- Build Runtime: Amazon Web Services (`build-runtime-aws`)

Amazon Web Services Regions

Region Technical Key	Region Name
`eu-central-1`	Europe (Frankfurt)[7 [page 3501]]
`eu-west-2`	Europe (London) [7 [page 3501]]
`eu-south-1`	Europe (Milan) [7 [page 3501]]
`ca-central-1`	Canada (Montreal) [7 [page 3501]]
`sa-east-1`	Brazil (São Paulo)
`us-east-1`	US East (VA)[7 [page 3501]]

Region Technical Key	Region Name
`us-west-2`	US West (Oregon)[7 [page 3501]]
`ap-northeast-1`	Japan (Tokyo)
`ap-northeast-2`	South Korea (Seoul)[7 [page 3501]]
`ap-south-1`	India (Mumbai) [7 [page 3501]]
`ap-southeast-1`	Singapore[7 [page 3501]]
`ap-southeast-2`	Australia (Sydney)[7 [page 3501]]

Google Cloud Regions

You can use the regions listed in the table with the following plans:

- Standard: Google Cloud (`gcp`)
- Build Runtime: Google Cloud (`build-runtime-gcp`)

Google Cloud Regions

Region Technical Key	Region Name
`europe-west3`	Europe (Frankfurt) [7 [page 3501]]
`europe-west4`	Europe (Netherlands)
`us-central1`	US Central (IA)[7 [page 3501]]
`us-west1`	North America (Oregon)
`us-east4`	North America (Virginia)[7 [page 3501]]
`southamerica-east1`	Brazil (São Paulo)
`asia-south1`	India (Mumbai)[7 [page 3501]]
`asia-south2`	India (Delhi) [7 [page 3501]]
`asia-northeast2`	Japan (Osaka)
`asia-northeast1`	Japan (Tokyo) [7 [page 3501]]
`asia-southeast1`	Singapore (Jurong West)
`australia-southeast1`	Australia (Sydney)
`me-central2`	KSA (Dammam) [7 [page 3501]]
`me-west1`	Israel (Tel Aviv)

Microsoft Azure Regions

You can use the regions listed in the table with the following plans:

- Standard: Microsoft Azure (`azure`)
- Kyma Test Demo and Development (Azure Lite) (`azure_lite`)
- Build Runtime: Microsoft Azure (`build-runtime-azure`)

Microsoft Azure Regions

Region Technical Key	Region Name
`eastus`	US East (VA)[7] [page 3501]
`eastus2`	US East 2 (VA)
`centralus`	US Central (IA)
`westus2`	US West (WA)[7] [page 3501]
`uksouth`	UK South (London)
`germanywestcentral`	Europe (Frankfurt)
`northeurope`	North EU (Ireland)
`westeurope`	Europe (Netherlands)[7] [page 3501]
`switzerlandnorth`	Switzerland (Zurich)[7] [page 3501], [8] [page 3500]
`japaneast`	Japan (Tokyo) [7] [page 3501]
`southeastasia`	Singapore
`australiaeast`	Australia (Sydney)[7] [page 3501]
`brazilsouth`	Brazil (São Paulo)
`canadacentral`	Canada (Toronto)
`chinanorth3`	China (North 3) [9] [page 3500]

[8] Not available with the `azure_lite` plan.

[9] This region is available only in the subaccount region cf-cn20 and is the sole region available within the Microsoft Azure (`azure`) plan in China.

SAP Cloud Infrastructure Regions

You can use the regions listed in the table with the SAP Cloud Infrastructure (`sap-converged-cloud`) plan.

For the exact mapping between the subaccount regions and the available IaaS regions, see Regions for the Kyma Environment [page 51].

Region Parameter

Region Technical Key	Region Name
`eu-de-1`	Germany (Rot)[7] [page 3501]
`eu-de-2`	Germany (Frankfurt)
`na-us-1`	US East (Sterling)[7] [page 3501]
`na-us-2`	US West (Colorado)
`ap-au-1`	Australia (Sydney)
`ap-jp-1`	Japan (Tokyo)
`ap-ae-1`	UAE (Dubai)

Alibaba Cloud Regions

You can use the regions listed in the table with the Alibaba Cloud (`alicloud`) plan.

Alibaba Cloud Regions

Region Technical Key	Region Name
`cn-shanghai`	China (Shanghai)

[7] Supports the *Colocate Control Plane* feature. For more information, see Colocate Control Plane [page 3480].

Related Information

Creating Kyma Instances [page 3452]
Available Plans in the Kyma Environment [page 3459]
Regions for the Kyma Environment [page 51]
Kyma Modules [page 88]
Account Administration Using the SAP BTP Command Line Interface (btp CLI) [page 2664]

6.4.5.1 Accessing Your Audit Log Data

Use the credentials stored in your Kyma cluster to query your audit log data using the SAP Audit Log Retrieval API v2.

Prerequisites

- You have a Kyma instance created. See Creating Kyma Instances [page 3452].
- You have the *Audit Log Access* (`auditLogAccess`) parameter enabled. See Audit Log Access [page 3477].

Context

When you enable the *Audit Log Access* parameter, Kyma stores the audit log read credentials in a Kubernetes Secret named `auditlog-read-credentials` in the `kyma-system` namespace of your cluster. The Secret contains the following fields:

Field	Description
`url`	Base URL of the SAP Audit Log Retrieval API
`clientID`	OAuth 2.0 client ID
`clientSecret`	OAuth 2.0 client Secret
`tokenUrl`	OAuth 2.0 token endpoint

> ⚠ Caution
>
> Enabling *Audit Log Access* is irreversible. Once you enable the feature, you cannot disable it.

Procedure

1. Connect to your Kyma cluster using kubectl. See Access a Kyma Instance Using kubectl.
2. To retrieve the Secret, run:

   ```
   kubectl get secret -n kyma-system auditlog-read-credentials -o yaml
   ```

3. Use the credentials from the Secret to authenticate against the SAP Audit Log Retrieval API v2 using OAuth 2.0 client credentials. See Create an OAuth Access Token.
4. Retrieve your audit log records. See Audit Log Retrieval.

Results

You can read your audit log data from the SAP Audit Log Retrieval API v2 using the credentials stored in the `auditlog-read-credentials` Secret.

Related Information

Audit Log Retrieval API for Global Accounts in the Cloud Foundry Environment
Retrieve Adit Log Data Using Audit Log CLI

6.4.5.2 Taints Configuration

Learn how to use taints to effectively schedule your workloads to nodes in additional worker node pools.

The Additional Worker Node Pools [page 3473] (`additionalWorkerNodePools`) parameter supports an optional *Taints* list. Use it to control which workloads are scheduled to nodes in a given worker pool.

> ⓘ Note
>
> Taints alone dictate scheduling restrictions on nodes, but these restrictions take effect only when combined with tolerations. While you configure taints on nodes, you set tolerations on your workloads (Pods) to allow them to run on tainted nodes. For more information on configuring tolerations, see Taints and Tolerations .

Taints Parameters

The *Taints* (`taints`) parameter includes three nested parameters: *Key**, *Value*, and *Effect**.

> → Remember
>
> The parameters marked with an asterisk "*" are mandatory.

Taints Nested Parameters

Nested Parameter	Description	Allowed Input
*Key** btp CLI parameter: `key`	Identifies the taint.	String
Value btp CLI parameter: `value`	Provides additional context.	String
*Effect** btp CLI parameter: `effect`	Defines what happens to workloads that do not tolerate the taint.	`NoSchedule` (excludes certain types of Pods from running on a specific set of nodes) `PreferNoSchedule` (discourages scheduling Pods on certain nodes, but allows it as a last resort, such as if no other nodes are available, to maintain availability or resource utilization) `NoExecute` (prevents any new workloads that cannot tolerate the taint from being scheduled onto a given node and evicts any existing Pods that already violate the taint and do not have the required toleration)

For each taint within a single worker node pool, the combination of the `key` and `effect` parameters must be unique. However, you can use the same `key` value with different `effect` values.

Provisioning a Cluster with Taints

To provision a cluster with taints in an additional worker node pool, use a configuration similar to the following in your request:

```
{
  "additionalWorkerNodePools": [
    {
      ...
      "taints": [
        {
          "key": "dedicated",
          "value": "gpu",
```

```
          "effect": "NoSchedule"
        },
        {
          "key": "dedicated",
          "value": "gpu",
          "effect": "NoExecute"
        }
      ]
    }
  ]
}
```

Updating Taints

To update or overwrite the existing taints for your worker node pool, you must explicitly provide a complete list of desired taints, for example:

```
{
  "additionalWorkerNodePools": [
    {
      ...
      "taints": [
        {
          "key": "dedicated",
          "value": "gpu",
          "effect": "NoSchedule"
        }
      ]
    }
  ]
}
```

Removing Taints

To remove existing taints for a worker node pool, either omit the `taints` field in the update request or set it to an empty list (`[]`), for example:

```
{
  "additionalWorkerNodePools": [
    {
      ...
      "taints": []
    }
  ]
}
```

Related Information

Assigning Workloads to Worker Node Pools [page 3522]

6.4.5.3 Custom OpenID Connect Configuration

During SAP BTP, Kyma runtime provisioning or update, you can provide your custom Open ID Connect (OIDC) configuration as a list of `oidc` objects (recommended) or as a single `oidc` object. You can also have no OIDC configuration.

Configuring the OIDC Property as a List of `oidc` Objects

It is recommended to configure the `oidc` property as a list of `oidc` objects. In this way, you can define zero, one, or multiple OIDC configurations. When using the list of `oidc` objects, you must provide values for each element except for `requiredClaims`. By configuring the OIDC details yourself, you gain full visibility of your settings within the SAP BTP cockpit. For your convenience, in the SAP BTP cockpit, one `oidc` object has the predefined values filled in, but you can change them. For each additional `oidc` object, you must provide your own settings.

Provisioning Kyma Runtime with the `oidc` List

When configuring your OIDC setup, you have the following options:

- If you use an `oidc` list, you must explicitly define values for all its elements, because they do not inherit default values.
- If you provide an empty `oidc` list with zero elements, no OIDC configuration is applied to the instance.

Example

- Configuring the `oidc` list.
  You can define one or multiple OIDC configurations, depending on the number of lists of `oidc` objects you add to your provisioning request. See an example of a request with one `oidc` object in the list:

```
{
  "service_id" : "47c9dcbf-ff30-448e-ab36-d3bad66ba281",
  "plan_id" : "4deee563-e5ec-4731-b9b1-53b42d855f0c",
  "context" : {
    "globalaccount_id" : {GLOBAL_ACCOUNT_ID}
  },
  "parameters" : {
    "region": {REGION},
    "name" : {CLUSTER_NAME},
    "oidc" : {
      "list": [
        {
          "clientID" : "custom-client-id",
          "issuerURL" : "https://custom.issuer.com",
          "groupsClaim" : "groups",
          "groupsPrefix" : "-",
          "signingAlgs" : ["RS256"],
          "usernamePrefix" : "-",
          "usernameClaim" : "sub",
          "requiredClaims" : ["first-claim=value", "second-claim=value"],
          "encodedJwksArray": ""
        }
      ]
```

```
    }
  }
}
```

You see a configuration similar to this one:

```
{
  ...
  "oidc" : {
    "list": [
      {
        "clientID" : "custom-client-id",
        "issuerURL" : "https://custom.issuer.com",
        "groupsClaim" : "groups",
        "groupsPrefix" : "-",
        "signingAlgs" : ["RS256"],
        "usernamePrefix" : "-",
        "usernameClaim" : "sub",
        "requiredClaims" : ["first-claim=value", "second-claim=value"],
        "encodedJwksArray": ""
      }
    ]
  }
  ...
}
```

Updating Kyma Runtime with the `oidc` List

You can update the OIDC configuration from a single `oidc` object to an `oidc` list, but updating from an `oidc` list to a single `oidc` object is not supported.

When updating your OIDC configuration, you have the following options:

- If you are updating your `oidc` list, you must explicitly define values for all its elements, because they do not inherit default values.
- The update operation overwrites the OIDC configuration values provided in JSON for the `oidc` list, so if you provide OIDC parameters with empty values, they are considered valid and replace the existing settings.
- If you provide an empty `oidc` list, it clears the OIDC configuration for the instance.

Example

- Updating an instance with a list of `oidc` objects.
  Suppose your current OIDC configuration is the following:

```
[
  {
   "clientID": "first-custom-client-id",
   "issuerURL": "https://first.custom.issuer.com",
   "groupsClaim": "groups",
   "groupsPrefix": "-",
   "usernameClaim": "sub",
   "usernamePrefix": "-",
   "signingAlgs": ["RS256"],
   "requiredClaims": ["first-claim=value", "second-claim=value"],
   "encodedJwksArray": ""
  },
  {
   "clientID": "second-custom-client-id",
   "issuerURL": "https://second.custom.issuer.com",
```

```
    "groupsClaim": "groups",
    "groupsPrefix": "-",
    "usernameClaim": "sub",
    "usernamePrefix": "acme-",
    "signingAlgs": ["RS256"],
    "requiredClaims": ["example-claim=value"],
    "encodedJwksArray": ""
  }
]
```

To update this configuration, send an HTTP PATCH request with the following payload:

```
{
  "service_id": "47c9dcbf-ff30-448e-ab36-d3bad66ba281",
  "plan_id": "4deee563-e5ec-4731-b9b1-53b42d855f0c",
  "context": {
   "globalaccount_id": "{GLOBAL_ACCOUNT_ID}"
  },
  "parameters": {
   "name": "{CLUSTER_NAME}",
   "oidc": {
    "list": [
      {
       "clientID": "new-client-id",
       "issuerURL": "https://new.issuer.com",
       "groupsClaim": "groups",
       "groupsPrefix": "-",
       "signingAlgs": ["RS256"],
       "usernameClaim": "sub",
       "usernamePrefix": "-",
       "requiredClaims": [],
       "encodedJwksArray": ""
      },
      {
       "clientID": "updated-client-id",
       "issuerURL": "https://updated.issuer.com",
       "groupsClaim": "groups",
       "groupsPrefix": "-",
       "usernameClaim": "sub",
       "usernamePrefix": "acme-",
       "signingAlgs": ["RS256"],
       "encodedJwksArray": ""
      }
    ]
   }
  }
}
```

You see the configuration with the values provided in the request:

```
[
  {
   "clientID": "new-client-id",
   "issuerURL": "https://new.issuer.com",
   "groupsClaim": "groups",
   "groupsPrefix": "-",
   "usernameClaim": "sub",
   "usernamePrefix": "-",
   "signingAlgs": ["RS256"],
   "requiredClaims": [],
   "encodedJwksArray": ""
  },
  {
   "clientID": "updated-client-id",
   "issuerURL": "https://updated.issuer.com",
   "groupsClaim": "groups",
   "groupsPrefix": "-",
```

```
    "usernameClaim": "sub",
    "usernamePrefix": "acme-",
    "signingAlgs": ["RS256"],
    "requiredClaims": [],
    "encodedJwksArray": ""
  }
]
```

Configuring the OIDC Property as a Single `oidc` Object

This solution is not recommended, and is only supported for maintaining backward compatibility with existing automations. With this approach you cannot have multiple OIDC configurations for your Kyma runtime.

In the SAP BTP cockpit, you can only see the parameters that you configure yourself. Since you must provide values only for `clientID` and `issuerURL` in a single `oidc` object, the visibility of your complete OIDC configuration may be limited.

Provisioning Kyma Runtime with a Single `oidc` Object

When configuring your OIDC setup, you have the following options:

- If you do not include an `oidc` object in the provisioning request at all, the default OIDC configuration is applied.
- If you leave the parameters of a single `oidc` object empty, it defaults to the predefined values.

Example

- See a request with an `oidc` object with empty keys:

```
{
  "service_id" : "47c9dcbf-ff30-448e-ab36-d3bad66ba281",
  "plan_id" : "4deee563-e5ec-4731-b9b1-53b42d855f0c",
  "context" : {
    "globalaccount_id" : {GLOBAL_ACCOUNT_ID}
  },
  "parameters" : {
    "region": {REGION},
    "name" : {CLUSTER_NAME},
    "oidc" : {
      "clientID" : "",
      "issuerURL" : "",
      "groupsClaim" : "",
      "groupsPrefix" : "",
      "signingAlgs" : [],
      "usernamePrefix" : "",
      "usernameClaim" : "",
      "requiredClaims" : ["-"],
      "encodedJwksArray": "-"
    }
  }
}
```

  The result is the default OIDC configuration:

```
{
```

```
  ...
  "oidc" : {
    "clientID" : "12b13a26-d993-4d0c-aa08-5f5852bbdff6",
    "issuerURL" : "https://kyma.accounts.ondemand.com",
    "groupsClaim" : "groups",
    "groupsPrefix" : "-",
    "signingAlgs" : ["RS256"],
    "usernamePrefix" : "-",
    "usernameClaim" : "sub",
    "requiredClaims" : ["-"],
    "encodedJwksArray": "-"
  }
  ...
}
```

Updating Kyma Runtime with a Single `oidc` Object

You can update the OIDC configuration from a single `oidc` object to an `oidc` list, but updating from an `oidc` list to a single `oidc` object is not supported.

When updating your OIDC configuration, you have the following options:

- If you provide no `oidc` object in the update request, the existing OIDC configuration remains unchanged.
- If you update using a single `oidc` object, empty keys do not change the configuration, and only the provided values are updated.

Example

- Updating an instance with a single `oidc` object.
  Suppose your current OIDC configuration is the following:

  ```
  {
    "clientID": "some-client-id",
    "issuerURL": "https://some.issuer.com",
    "groupsClaim": "groups",
    "groupsPrefix": "-",
    "usernameClaim": "sub",
    "usernamePrefix": "-",
    "signingAlgs": ["RS256"],
    "requiredClaims": ["-"],
    "encodedJwksArray": "-"
  }
  ```

  To update this configuration, send an HTTP PATCH request with the following payload:

  ```
  {
    "service_id": "47c9dcbf-ff30-448e-ab36-d3bad66ba281",
    "plan_id": "4deee563-e5ec-4731-b9b1-53b42d855f0c",
    "context": {
     "globalaccount_id": "{GLOBAL_ACCOUNT_ID}"
    },
    "parameters": {
     "name": "{CLUSTER_NAME}",
     "oidc": {
      "clientID": "new-client-id",
      "issuerURL": "https://new.issuer.com",
      "groupsClaim": "",
      "groupsPrefix": "",
      "signingAlgs": [],
      "usernamePrefix": "",
  ```

```
    "usernameClaim": "",
    "requiredClaims": ["-"],
    "encodedJwksArray": "-"
   }
  }
}
```

You see the configuration with the values provided in the request, similar to this one:

```
{
  "clientID": "new-client-id",
  "issuerURL": "https://new.issuer.com",
  "groupsClaim": "groups",
  "groupsPrefix": "-",
  "usernameClaim": "sub",
  "usernamePrefix": "-",
  "signingAlgs": ["RS256"],
  "requiredClaims": ["-"],
  "encodedJwksArray": "-"
}
```

Configuration with no OIDC Property

You can provision or update your Kyma runtime without providing the OIDC property at all.

Provisioning Kyma Runtime with No OIDC Property

If you do not include an `oidc` parameter in the provisioning request at all, the default OIDC configuration is applied.

Example

- See an example of a provisioning request without specifying any `oidc` configuration:

```
{
  "service_id" : "47c9dcbf-ff30-448e-ab36-d3bad66ba281",
  "plan_id" : "4deee563-e5ec-4731-b9b1-53b42d855f0c",
  "context" : {
    "globalaccount_id" : {GLOBAL_ACCOUNT_ID}
  },
  "parameters" : {
    "region": {REGION},
    "name" : {CLUSTER_NAME}
  }
}
```

The resulting configuration has default values like in the following example. You can only view this configuration in your cluster.

```
{
  ...
  "oidc" : {
    "clientID" : "12b13a26-d993-4d0c-aa08-5f5852bbdff6",
    "issuerURL" : "https://kyma.accounts.ondemand.com",
    "groupsClaim" : "groups",
```

```
    "groupsPrefix" : "-",
    "signingAlgs" : ["RS256"],
    "usernamePrefix" : "-",
    "usernameClaim" : "sub",
    "requiredClaims" : ["-"],
    "encodedJwksArray": "-"
  }
  ...
}
```

Updating Kyma Runtime with No OIDC Property

If you provide no `oidc` property in the update request, the existing OIDC configuration remains unchanged.

Related Information

OpenID Connect (OIDC) [page 3494]
Configuring a Custom Identity Provider for Kyma [page 3660]

6.4.6 Adding and Deleting a Kyma Module

To use a Kyma module, you must add it first. Use Kyma dashboard or Kyma CLI to do that. If you don't need the module anymore, delete it to save resources.

Context

To see which modules you have in your cluster, go to Kyma dashboard and under *Cluster Overview*, choose *Modify Modules*.

If you prefer Kyma CLI, run `kyma module list`.

Adding a Kyma Module

Context

> ⚠ Caution
>
> If you apply cluster-wide security tools or policies to your cluster, configure them to exclude Kyma-managed namespaces and components. Policies that go beyond the platform defaults can block module installation and reconciliation. Kyma modules are not modified to comply with custom security policies.

> Do not make changes in any Kyma-managed namespaces, the `kube-system` namespace, or to any resources marked with *Do Not Edit*. Changes to those resources could lead to exclusion or disruptions from the agreed SLA. If you decide to make changes despite this, consult support in advance, and proceed with caution. To identify a Kyma-managed namespace, check if it has the `operator.kyma-project.io/managed-by: kyma` label.

Procedure

- Use Kyma dashboard
  a. Go to Kyma dashboard. The URL is in the Overview section of your subaccount.
  b. Choose *Modify Modules* > *Add*.
  c. In the *Add Modules* section, check the modules you want to add, and select *Add*.
  d. **Optional:** At the module level, you can overwrite the default release channel for the modules you are adding. Under the *Advanced* options, choose your preferred release channel.

  This process may take a while, depending on the number of modules. The operation was successful when the module status changes to `READY`.
- Use Kyma CLI
  a. Check the list of modules that are available to be added:

  ```
  kyma module catalog
  ```

  b. Add a new module:
     - To add a new module with the default configuration, use the following command:

     ```
     kyma module add {MODULE-NAME} --default-config-cr
     ```

     - To add a module with your specific configuration, create a YAML file containing your module custom resource (CR) settings, and use the `--cr-path={CR-FILEPATH}` flag:

     ```
     kyma module add {MODULE-NAME} --cr-path={CR-FILEPATH}
     ```

     - To add a module with the specified release channel, use the `-c {CHANNEL-NAME}` flag:

     ```
     kyma module add {MODULE-NAME} -c {CHANNEL-NAME} --default-config-cr
     ```

  To see if your module is added, run the following command.

  ```
  kyma module list
  ```

Related Information

Kyma Dashboard [page 93]
Kyma CLI [page 99]

Deleting a Kyma Module

Procedure

- Use Kyma dashboard
  a. Go to Kyma dashboard. The URL is in the Overview section of your subaccount.
  b. Choose *Modify Modules*, and delete your module.
- Use Kyma CLI
  a. Check the list of installed modules:

     ```
     kyma module list
     ```

  b. To delete your module, use the following command:

     ```
     kyma module delete {MODULE-NAME}
     ```

     You should see the following message:

     ```
     {MODULE-NAME} module disabled
     ```

Related Information

Kyma Dashboard [page 93]
Kyma CLI [page 99]

6.4.7 Setting Your Module to the Unmanaged and Managed State

In some cases, for example, for testing, you may need to modify your module beyond what is supported by its configuration. By default, when a module is in the managed state, Kyma Control Plane governs its Kubernetes resources, reverting any manual changes during the next reconciliation loop. To modify Kubernetes objects directly without them being reverted, you must set the module to the unmanaged state. In this state, reconciliation is disabled, ensuring your manual changes are preserved.

> ⚠ Caution
>
> Unmanaged modules aren't subject to the Service Level Agreement (SLA). Additionally, setting a module back to the managed state does not guarantee its version is correctly updated.

Setting Your Module to the Unmanaged State

Procedure

- Use Kyma dashboard
  a. Go to Kyma dashboard. The URL is in the Overview section of your subaccount.
  b. Choose *Modify Modules*, and go to the *Edit* tab.
  c. In the *Form* view, uncheck the *Managed* checkbox under your module.
  d. Save the changes.

  In the *View* tab, you see your module in the `Unmanaged` state.
- Use Kyma CLI
  - To set a module to the unmanaged state, use the following command:

    ```
    kyma module unmanage {MODULE-NAME}
    ```

  You should see the following message:

  ```
  Module {MODULE-NAME} set to unmanaged
  ```

Related Information

Kyma Dashboard [page 93]
Kyma CLI [page 99]

Setting Your Module to the Managed State

Procedure

- Use Kyma dashboard
  a. Go to Kyma dashboard. The URL is in the Overview section of your subaccount.
  b. Choose *Modify Modules*, and go to the *Edit* tab.
  c. In the *Form* view, check the *Managed* checkbox under your module.
  d. Save the changes.

  In the *View* tab, you see your module in the `Ready` state.
- Use Kyma CLI
  - To set a module to the managed state, use the following command:

    ```
    kyma module manage {MODULE-NAME}
    ```

You should see the following message:

```
Module {MODULE-NAME} set to managed
```

Related Information

Kyma Dashboard [page 93]
Kyma CLI [page 99]

6.4.8 Access Kyma Application Logs

Get insights into your applications, microservices, and Functions by viewing the respective logs. To check out real-time logs immediately, use the Kubernetes functionalities - either in Kyma dashboard, or with kubectl.

Context

- If you prefer to see the logs in your terminal, replace the placeholders in the following command and run it:

  ```
  kubectl logs <{POD_NAME}> --namespace <{NAMESPACE_NAME}> --container
  <{CONTAINER_NAME}>
  ```

- To see real-time logs in Kyma dashboard, follow these steps:

Procedure

1. Open Kyma dashboard and select the namespace.
2. Access the Pod.
3. Find the container and select *View Logs*.

Results

You have access to short-term logs of every container in your Kyma runtime.

Next Steps

> → Recommendation
>
> If you want long-term storage of logs with advanced querying, dashboarding, and alerting capabilities, set up SAP Cloud Logging to ingest observability data with Kyma runtime.

Related Information

Auditing and Logging Information in Kyma [page 3678]
Create an SAP Cloud Logging Instance through SAP BTP Service Operator
SAP Cloud Logging: Ingest via Kyma Runtime

6.4.9 Kyma Environment Backup

The user load in a Kyma cluster typically consists of various Kubernetes objects and volumes.

- Objects hold the configuration of user's application applied to the API server using Kubernetes resources
- Volumes hold the actual data that a user may store in a persistent way

Kyma does not provide backup, neither for objects nor volumes, that can replace the configuration or data from a specific point in time.

Object Backup for Kubernetes Configuration

Kyma environment relies on the managed Kubernetes cluster for periodic backups of Kubernetes objects. It means that it is possible to automatically restore Kubernetes objects in case of inconsistencies or data corruption. However, Kyma environment does not provide backup that can replace the configuration from a specific point in time. Automatic backup doesn't include Kubernetes volumes.

For example, project "Gardener" uses etcd as the Kubernetes' backing store for all cluster data. This means that all Kubernetes objects are stored on etcd. Gardener uses periodic jobs to take major and minor snapshots of the etcd database. A major snapshot (including all the resources) takes place every day, and each minor snapshot (including only the changes in between) takes place every 5 minutes. If the etcd database experiences any problems, Gardener automatically restores the Kubernetes cluster using the latest snapshot.

> → Tip
>
> Follow GitOps principles to reapply the desired state of your Kubernetes objects to any Kubernetes cluster at any time.
>
> - GitOps means managing your Kubernetes objects using Git .

- A Git repository becomes a single source of truth where you declare the desired state of your Kubernetes objects.
- Automation tooling of your choice pulls the desired state from that single source and applies the declared configuration to your cluster.

For more information, see OpenGitOps .

Volume Backup for Customer Data

Your customer data isn't backed up automatically. If you're using Kubernetes volumes to store data, we recommend that you use Kubernetes VolumeSnapshots to back up and recover your data. The backup and recovery using Kubernetes VolumeSnapshots are supported for AWS, Azure, and Google Cloud.

Related Information

Creating On-Demand Volume Snapshots [page 3517]
Change Storage Size in Kyma [page 3521]

6.4.9.1 Creating On-Demand Volume Snapshots

Prerequisites

A PersistentVolumeClaim (PVC) resource exists in the cluster.

Context

You can manually back up your volumes using the VolumeSnapshot Kubernetes resource. When you create a volume snapshot, you copy your volume's contents at that particular point in time. A VolumeSnapshot resource requires an existing source PVC and a VolumeSnapshotClass resource specified. Taking volume snapshots is possible thanks to Container Storage Interface (CSI) drivers , which allow third-party storage providers to expose storage systems in Kubernetes. The driver must be specified in the VolumeSnapshotClass resource.

Procedure

1. Create a VolumeSnapshot resource using the default VolumeSnapshotClass and your PVC name:

```
kubectl apply -n {NAMESPACE} -f <<EOF
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: snapshot
spec:
  volumeSnapshotClassName: default
  source:
    persistentVolumeClaimName: {YOUR_PVC_NAME}
EOF
```

   The VolumeSnapshot resource is created.

2. To verify that the snapshot was taken successfully, run `kubectl get -n {NAMESPACE} volumesnapshot -w` and check that the field *READYTOUSE* has status `true`.

6.4.10 Kyma Modules' Sizing

The following tables show the resource consumption, such as memory, CPU, and storage, of Kyma modules.

Istio

Istio Module's Sizing

Infrastructure size	Memory	CPU
Small cluster minimum	352 Mi	0.08
Large cluster minimum	1984 Mi	0.61

API Gateway

API Gateway Module's Sizing

Infrastructure size	Memory	CPU
Small cluster minimum	212 Mi	0.03
Large cluster minimum	416 Mi	0.34

SAP BTP Operator

SAP BTP Operator Module's Sizing

Infrastructure size	Memory	CPU
Minimum	53 Mi	0.02
Maximum	1000 Mi	2

Application Connector

Application Connector Module's Sizing

Infrastructure size	Memory	CPU
Minimum	64-128 Mi	0.2-0.5

Eventing

Eventing Manager's Sizing

Infrastructure size	Memory	CPU
Minimum	128 Mi	0.01
Maximum	512 Mi	0.5

Eventing Publisher Proxy's Sizing

Infrastructure size	Memory	CPU	Max Publisher instances
Low EPS (up to 200 Events per second)	64-512 Mi	0.04-0.5	2 (second instance for high availability)
High EPS (up to 3000 Events per second)	64-512 Mi	0.04-0.5	3

Keda

Keda Module's Sizing

Infrastructure size	Memory	CPU
Minimum	692 Mi	0.62
Maximum	928 Mi	1.8

NATS

By default, the NATS module creates a NATS-cluster with three nodes for data safety reasons. The values are given per NATS-cluster node:

NATS Module's Sizing

Infrastructure size	Memory	CPU	Storage
Minimum	64 Mi	0.04	1000 Mi
Maximum	1000 Mi	0.5	1000 Mi

Serverless

Serverless Module's Sizing

Infrastructure size	Memory	CPU	Storage
Minimum	400 Mi	0.03	
Maximum	2300 Mi	1.7	Up to 20000 Mi (for PVC for internal Docker registry)

Telemetry

Resource consumption depends on the APIs (pipelines) you are using. For details, see Telemetry Pipeline API [page 2187].

Using the Telemetry module without activating any pipelines results in the Telemetry Manager running with the following footprint:

Infrastructure size	Memory	CPU
Minimum	5 Mi	0.01
Maximum	100 Mi	0.1

Activation of the first `LogPipeline` causes the deployment of the log agent running an instance per node:

Infrastructure size	Memory	CPU
Minimum per node	50 Mi	0.1
Maximum per node	1000 Mi	1

Activation of the first `TracePipeline` causes the deployment of the trace gateway running with two replicas:

Infrastructure size	Memory	CPU
Minimum	2*32 Mi	2*0.2

Infrastructure size	Memory	CPU
Maximum	2*2000 Mi	2*1.2

Activation of the first `MetricPipeline` causes the deployment of the metric gateway running with two replicas and the metric agent running per node:

Infrastructure size	Memory	CPU
Minimum	2*30 Mi + 50 Mi per node	2*0.01 + 0.01 per node
Maximum	2*1000 Mi + 1200 Mi per node	2*1.00 + 1 per node

Related Information

Sizing

6.4.11 Change Storage Size in Kyma

If the amount of data for the applications in your Kyma environment grows, you can expand the storage size for your customer data by resizing the respective Persistent Volume Claim (PVC).

Prerequisites

- Your user has the authorizations to change PVC resources.
- Your workload uses a Persistent Volume, managed by a PVC.

Context

In a Kubernetes cluster, your customer data is stored in volumes. A “Persistent Volume Claim” (PVC) manages that volume for you and defines the size of storage. As the amount of customer data grows, you must adjust the PVC accordingly.

Procedure

1. Go to Kyma dashboard and select the correct namespace and workload.
2. Find the PVC you want to resize and enter the desired new size of the volume.

3. Save your changes.

   The PVC shows the condition type *Resizing*. Wait until the PVC switches to condition type *FileSystemResizePending*. This may take a few minutes.
4. Restart your workload by restarting all related Pods.

Results

The storage size for your customer data has been adjusted to your needs.

Next Steps

Remember to back up your customer data.

Related Information

Kubernetes: Expanding Persistent Volumes Claims
Kyma Environment Backup [page 3516]

6.4.12 Assigning Workloads to Worker Node Pools

In addition to the mandatory worker node pool existing in each Kyma runtime, you can add and configure custom worker node pools. These additional worker nodes are reserved for running your workloads with special computing requirements on optimized machines. To ensure your workload runs on the right worker node pool, assign workloads to particular worker nodes.

Prerequisites

You have at least one additional worker node pool in your Kyma runtime. To learn how to add and configure a custom worker node pool, see Additional Worker Node Pools [page 3473].

Context

Worker node pools are collections of one or more worker nodes. To assign your workloads (Pods) to a particular worker node pool, use the following worker node's label: `worker.gardener.cloud/pool=<worker-pool-name>`. The label specifies the worker pool to which the node belongs.

You can assign your workloads to worker node pools using one of the following recommended methods:

- Set a node selector .
- Configure node affinity .

Node affinity offers more flexibility for assigning workloads than node selector. You can define multiple affinity rules and configure them as hard or soft requirements by using one of the following strategies: `requiredDuringSchedulingIgnoredDuringExecution` or `preferredDuringSchedulingIgnoredDuringExecution`. You can also invert affinity by setting it as anti-affinity.

You can also assign your workloads to worker node pools using taints with tolerations. This method is useful for strict control over where workloads are placed, ensuring isolation or prioritization of specific workloads. For more information, see Taints and Tolerations .

Procedure

- Use a Node Selector.

  Add the `nodeSelector` to the Kubernetes manifest of your Pod.

  See an example of the manifest that schedules the Pod only on the `worker-pool-1` worker pool:

  ```
  apiVersion: v1
  kind: Pod
  metadata:
    name: worker-pool-nodeselector
    labels:
      testcase: nodeselector
  spec:
    nodeSelector:
      worker.gardener.cloud/pool: "worker-pool-1"
    # only needed if workloads should be spread over multiple availability
  zones:
    topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: "topology.kubernetes.io/zone"
        whenUnsatisfiable: ScheduleAnyway
        labelSelector:
          matchLabels:
            testcase: nodeselector
    containers:
    - name: pause
      image: registry.k8s.io/pause:3.8
  ```

  If the worker pool is configured with high availability and multiple Pod replicas are used, consider distributing the Pods over availability zones. For more information, see Kubernetes: Pod Topology Spread Constraints .

- Use Pod Affinity.

  Add `nodeAffinity` to the Kubernetes manifest of your Pod.

  The following example shows the worker pool configuration consisting of multiple availability zones. Using anti-affinity enables the sharing of Pods over multiple availability zones. For more information, see Kubernetes: Inter-pod affinity and anti-affinity .

  ```
  apiVersion: v1
  ```

```
kind: Pod
metadata:
  name: worker-pool-nodeaffinity
  labels:
    testcase: nodeaffinity
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: worker.gardener.cloud/pool
            operator: In
            values:
            - worker-pool-1
    # only needed if workloads should be spread over multiple availability
zones:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchLabels:
              testcase: nodeaffinity
          topologyKey: "topology.kubernetes.io/zone"
  containers:
  - name: pause
    image: registry.k8s.io/pause:3.8
```

Results

With a node selector or affinity constraint, Pods are only scheduled on nodes that match the defined label conditions.

Troubleshooting

Pod stuck in the Pending state

Symptom: Your Pod stays in the Pending state.

Cause: There's no suitable node.

Solution: Check the labels used to find matching nodes or use a soft node assignment by applying the node affinity type `preferredDuringSchedulingIgnoredDuringExecution`.

6.4.13 Adapting Your Workloads to Kyma Landscape Requirements

Learn how to use Runtime Bootstrapper, a built-in Kyma component, to automatically adapt your Pods to landscape-specific requirements such as private registries, custom TLS certificates, and FIPS mode signaling.

> **Note**
>
> The content in this topic is only relevant for China (Shanghai) and Government Cloud (US) regions.

When you deploy workloads on SAP BTP, Kyma runtime in certain regions, such as China (Shanghai) and Government Cloud (US), Pods need specific configurations. For example, Pods may need to pull images from a private registry or trust landscape-specific TLS certificates. Runtime Bootstrapper automatically applies these configurations to your Pods at creation time. This process is transparent and requires no changes to your application code.

Runtime Bootstrapper only modifies Pods. It doesn't affect any other Kubernetes resources, such as Deployments, Services, ConfigMaps, or Secrets.

Enabling Runtime Bootstrapper

To enable Runtime Bootstrapper features, annotate either a namespace or a Pod (or Pod template).

Annotating a Namespace

Add one or more feature annotations to your namespace. All Pods created in that namespace then receive the corresponding adjustments automatically. You don't need to change individual workload manifests.

```
apiVersion: v1
kind: Namespace
metadata:
  name: my-namespace
  annotations:
    rt-cfg.kyma-project.io/alter-img-registry: "true"
    rt-cfg.kyma-project.io/add-img-pull-secret: "true"
```

Annotating a Pod or a Pod Template

Add annotations directly to your Pod or to the `spec.template.metadata.annotations` section of a Deployment, StatefulSet, or a similar resource. As a result, only that specific Pod or Pods from that template are adjusted.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  template:
    metadata:
      annotations:
        rt-cfg.kyma-project.io/alter-img-registry: "true"
        rt-cfg.kyma-project.io/add-img-pull-secret: "true"
    spec:
      containers:
```

```
      - name: my-container
        image: my-registry.example.com/my-app:1.0.0
```

You can combine both methods. If you enable a feature on a namespace, all Pods in that namespace benefit from it. There is no way to opt out at the Pod level.

Supported Annotations

Each annotation enables one specific feature. To activate it, set the value to true.

The availability shown below may change. To confirm a feature is active in your landscape, see Confirming a Pod Was Modified [page 3529]. If a feature appears inactive despite being listed as available, contact your landscape operator.

Annotation	Typical use case	Relevant to Government Cloud (US)	Relevant to China (Shanghai)
`rt-cfg.kyma-project.io/add-cluster-trust-bundle` [page 3526]	Mounts custom CA certificates required to trust SAP backend endpoints	Yes, it's a primary feature. The CA bundle is managed by the landscape operator team. If TLS communication fails due to missing or outdated certificates, create a support ticket. See Getting Support.	No
`rt-cfg.kyma-project.io/add-img-pull-secret` [page 3528]	Only relevant when pulling images from a private registry that requires authentication	No	Yes
`rt-cfg.kyma-project.io/alter-img-registry` [page 3528]	Only required when the container registry hostname must be rewritten to a landscape-specific mirror	Yes	Yes
`rt-cfg.kyma-project.io/set-fips-mode` [page 3528]	Signals to workloads that FIPS 140-compliant cryptography should be used	Yes	No

> → Tip
>
> To enable all features available in your region at once, use the shorthand annotation `rt-cfg.kyma-project.io/all: "true"` [page 3529] on either a namespace or a Pod.

`rt-cfg.kyma-project.io/add-cluster-trust-bundle`

The annotation mounts the cluster's TLS certificate bundle into your containers and automatically restarts Pods when the CA bundle changes

Some landscapes use custom TLS certificates that are not included in the standard operating system trust store. When you enable this feature, Runtime Bootstrapper mounts the cluster's certificate bundle as a read-

only volume into every container (including init-containers) under the path `/etc/ssl/certs`. As a result, your application can trust landscape-specific HTTPS endpoints without any code changes.

The annotation accepts the following values:

Value	Behavior
"true"	Mount the CA bundle and automatically restart the Pod when the bundle changes
"false"	Explicit opt-out - no CA bundle mount, even if namespace defaults enable it

The annotation causes the following changes in your Pod:

- `.spec.volumes[]` - a projected volume named `rt-bootstrapper-certs` is added
- `.spec.containers[*].volumeMounts` - the volume is mounted read-only at `/etc/ssl/certs`
- `.spec.initContainers[*].volumeMounts` - same mount applied to init-containers
- `.metadata.annotations["rt-bootstrapper.kyma-project.io/ctb-hash"]` - a SHA-256 hash of the current CA bundle content, used to detect changes. This annotation is stamped on every Pod that receives the CA bundle, regardless of whether the opt-in came from a Pod annotation, namespace annotation, or namespace defaults.

Explicit Opt-Out

If you set the annotation to `"false"` on a Pod, it explicitly opts out of the CA bundle mount. This overrides namespace-level defaults. Use it when a specific workload must not receive the certificate bundle.

```
annotations:
  rt-cfg.kyma-project.io/add-cluster-trust-bundle: "false"
```

Automatic Restart on Certificate Rotation

Runtime Bootstrapper automatically restarts Pods when the `ClusterTrustBundle` CA changes. A built-in controller watches the `ClusterTrustBundle` resource and computes a SHA-256 hash of its content. When the hash changes, the controller scans all namespaces for Pods whose stamped hash no longer matches and deletes them. The owning controller (for example, a Deployment's ReplicaSet) recreates the Pods, and the webhook stamps the new hash during admission.

Only Pods managed by an owner (for example, a ReplicaSet or StatefulSet) are restarted. Standalone Pods without an owner still receive the CA bundle and the hash annotation but are never deleted to avoid data loss.

This means your application does not need to implement file watching or any other mechanism to handle certificate rotation. The restart ensures the new certificate is loaded at startup.

> → Tip
>
> If your application can reload TLS trust stores at runtime (for example, by watching for file changes under `/etc/ssl/certs`), it can pick up the new certificates without a restart. The automatic restart acts as a safety net for applications that load certificates only at startup.

Enabling Automatic Restart in Your Namespace

The restart controller can only operate in namespaces where the Runtime Bootstrapper ServiceAccount has Pods get, list, and delete permissions. By default, these permissions are configured for Kyma-managed namespaces (for example, `kyma-system`, `istio-system`). To enable automatic restarts in your own

namespace, create a RoleBinding that grants the required permissions. Replace `my-namespace` with your namespace name:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: rt-bootstrapper-pod-restarter-my-namespace
  namespace: my-namespace
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: rt-bootstrapper-pod-restarter-role
subjects:
- kind: ServiceAccount
  name: rt-bootstrapper-controller-manager
  namespace: kyma-system
```

Without this binding, the controller logs a warning and skips the namespace.

`rt-cfg.kyma-project.io/add-img-pull-secret`

The annotation injects image pull credentials into your Pod.

When a private container registry requires authentication, Runtime Bootstrapper adds a reference to the landscape's image pull Secret (`registry-credentials`) to your Pod. Your Pod can then pull images - you don't have to manage registry credentials yourself.

The annotation causes the following change in your Pod:

- `.spec.imagePullSecrets[]` - the `registry-credentials` entry is appended

If the Secret reference is already present, it is not added again.

`rt-cfg.kyma-project.io/alter-img-registry`

The annotation rewrites the container registry host in image references.

Some landscapes require container images to be pulled from a private or landscape-specific registry rather than the original public registry. When you enable this feature, Runtime Bootstrapper automatically rewrites the registry hostname in the `image` field of every container and init-container in your Pod. You don't need to know the target registry address, because it's configured centrally for the landscape.

The annotation causes the following changes in your Pod:

- `.spec.containers[*].image` - registry hostname is replaced
- `.spec.initContainers[*].image` - registry hostname is replaced

`rt-cfg.kyma-project.io/set-fips-mode`

The annotation signals FIPS 140 compliance mode to your workloads.

Federal Information Processing Standards (FIPS) mode restricts cryptographic operations to approved algorithms. When you enable this feature, Runtime Bootstrapper sets two environment variables in every container and init-container. Your application can read these variables to activate its own FIPS-compliant code paths.

The annotation causes the following changes in your Pod:

- `.spec.containers[*].env[]` - the following variables are added to every container:
  - `KYMA_FIPS_MODE_ENABLED=true`

    - `FIPS_MODE_ENABLED=true` (legacy compatibility)
- `.spec.initContainers[*].env[]` - the following variables are added to init-containers:
    - `KYMA_FIPS_MODE_ENABLED=true`
    - `FIPS_MODE_ENABLED=true` (legacy compatibility)

`rt-cfg.kyma-project.io/all`

The annotation enables all features available for your region at once.

Setting this annotation to true applies all features available for your region (see Supported Annotations [page 3526]) that are enabled in the operator's configuration. Use it when you want your namespace or Pod to receive all applicable landscape adaptations without listing them individually.

```
annotations:
  rt-cfg.kyma-project.io/all: "true"
```

Confirming a Pod Was Modified

After Runtime Bootstrapper processes a Pod, it adds the following annotation to the Pod metadata:

```
rt-bootstrapper.kyma-project.io/modified: "true"
```

To verify this, replace the placeholder and run:

```
kubectl get pod <pod-name> -o jsonpath='{.metadata.annotations.rt-
bootstrapper\.kyma-project\.io/modified}'
```

If the output is `true`, Runtime Bootstrapper applied at least one modification when the Pod was created. This annotation reflects the state at creation time only. If the landscape configuration changes later, recreated Pods might silently receive fewer modifications without automatic notification. If you suspect a feature is no longer active, contact your landscape operator and check the Runtime Bootstrapper logs for relevant warnings:

```
kubectl logs -n kyma-system -l app.kubernetes.io/name=rt-bootstrapper
```

If the output is empty or `false`, either no feature annotations were set on the Pod or its namespace, or the annotations used are not supported in this landscape. See Supported Annotations [page 3526].

6.4.14 Admission Policies

Learn about the validating admission policies deployed in SAP BTP, Kyma runtime clusters, why they block specific actions, and what you can do if you need an exception. These policies are enforced at admission time and protect cluster stability, security, and operational correctness.

Namespaces Delete Protection

This policy prevents the deletion of Kyma-managed namespaces unless the request comes from an allowed user, group, or service account.

The following namespaces host critical platform components and are protected from deletion:

- `istio-system`
- `kyma-system`
- `sap-transp-proxy-system`
- `ztis-agent-system`
- `docker-registry`
- `registry-proxy`

An accidental or unauthorized deletion of these namespaces can cause cluster-wide outages, break service mesh functionality, and interfere with platform lifecycle operations.

When you try to delete these namespaces, you receive an API error. The validating admission policy returns a rejection with the message that you cannot delete the Kyma-managed namespaces. This action requires an allowed user, group, or service account.

Verify that you're targeting the correct namespace and use application namespaces instead of the Kyma-managed system namespaces.

Kyma System Protection (Workload Deletion in `kyma-system`)

This policy prevents the deletion of workloads (Deployments, StatefulSets, DaemonSets, Jobs, CronJobs) inside the `kyma-system` namespace unless the request comes from an allowed user, group, or service account.

The `kyma-system` namespace runs managed Kyma components. Deleting workloads there can break the control plane, integrations, and user-facing services. This policy prevents accidental disruption and enforces that only trusted controllers or processes make such changes.

When you try to delete a workload in `kyma-system`, you see a rejection at the API admission step with the message that you cannot delete workloads in the `kyma-system` namespace. This action requires an allowed user, group, or service account.

When you delete workloads, verify that you're targeting the correct namespace. For changes to customer workloads, use their application namespaces rather than `kyma-system`.

Kube-System Istio Injection Prevention

This policy prevents adding the label `istio-injection=enabled` to the `kube-system` namespace.

Enabling Istio sidecar injection in `kube-system` can unintentionally inject proxies into critical system components. These components can be incompatible with sidecars, which could lead to degraded performance, bootstrap failures, or networking issues for platform services.

The label add or patch requests that attempt to set `istio-injection=enabled` on the `kube-system` namespace are rejected with the message that this label cannot be added here.

Double-check why you need injection in `kube-system`. Commonly, you can isolate your components by enabling sidecar injection in the specific application namespaces instead.

Dynakube Kyma Exclusion

This policy validates that Dynatrace Dynakube custom resources include namespace-exclusion rules so that Kyma-managed namespaces (for example, `kyma-system`, `istio-system`, `sap-transp-proxy-system`, `ztis-agent-system`, or namespaces with label `operator.kyma-project.io/managed-by=kyma`) are excluded from automatic instrumentation.

If the required exclusions are missing, the admission policy denies the resource.

Automatic instrumentation across Kyma-managed infrastructure may interfere with Kyma's lifecycle management, generate noisy metrics or logs, or cause unexpected side effects in platform-managed namespaces. Explicit exclusions ensure that monitoring is applied where safe and desired without interfering with platform-managed components.

The policy checks the following fields depending on your Dynakube API version and configuration:

- `spec.metadataEnrichment.namespaceSelector.matchExpressions` (if `metadataEnrichment` is enabled)
- `spec.oneAgent.cloudNativeFullStack.namespaceSelector.matchExpressions` (if `cloudNativeFullStack` is configured)
- `spec.oneAgent.applicationMonitoring.namespaceSelector.matchExpressions` (if `applicationMonitoring` is configured)
- `spec.namespaceSelector.matchExpressions` (for Dynakube API version v1beta1)

If your Dynakube setup is missing required exclusions, an error message tells you to which field you must add them.

For configuration examples, see Kyma: Dynatrace Setup and Dynatrace: DynaKube parameters for Dynatrace Operator.

Enforce Kyma Workload Traffic

This policy prevents the use of network policies with `default-deny-all` rules in the Kyma-managed namespaces. Such network policies block essential communication between Kyma components within a cluster, potentially leading to system failures.

If you attempt to create or update a network policy that violates this rule, the action is blocked, and you receive a warning. In such a case, review your network policy configuration and ensure it includes specific allow rules instead of denying all traffic. For applications that require strict network isolation, consider using your own namespaces instead of the Kyma-managed namespaces.

Kyma Module Label Protection

This policy prevents you from setting labels with the `kyma-project.io/` prefix on any resource in Kyma-managed namespaces.

The `kyma-project.io/` label prefix is reserved exclusively for Kyma module operators to identify and manage their resources. When your resource in a protected namespace carries a `kyma-project.io/` label, Kyma module operators may misidentify it as a module operator-managed resource. This misidentification causes the operator to enter an error state.

The following namespaces are protected:

- `kyma-system`
- `istio-system`
- `docker-registry`
- `registry-proxy`
- `sap-trans-proxy-system`
- `ztis-agent-system`

When you create or update a resource with a `kyma-project.io/` label in one of these namespaces, you receive an API error message stating that setting reserved labels isn't allowed.

- If you need to categorize or identify your resources, use your own label prefix instead of the reserved `kyma-project.io/` prefix.
- If you're deploying workloads that require such a label, deploy them into your own application namespaces rather than into Kyma-managed system namespaces.

Kyma Storage Protection

This policy prevents you from deleting PersistentVolumeClaims (PVCs) in the `kyma-system` namespace and PersistentVolumes (PVs) that are bound to a `kyma-system` PVC, unless the request comes from an allowed user, group, or service account.

The `kyma-system` namespace may use persistent storage for platform components. Deleting a PVC or its bound PV can result in permanent data loss for those components. This policy ensures that only trusted users, groups, or service accounts can remove storage resources tied to platform operations.

When you try to delete a PVC in `kyma-system`, or a PV that is bound to a PVC in `kyma-system`, you receive an API error. The error message states that this deletion isn't allowed and requires an allowed user, group, or service account.

Verify that you're targeting the correct resource. If you need to remove storage in `kyma-system`, contact your platform administrator to perform the operation with an allowed user, group, or service account.

Kyverno Namespace Protection

This policy validates that Kyverno ClusterPolicies and policies that affect workloads, such as Pods, Deployments, StatefulSets, DaemonSets, Jobs, CronJobs, ReplicaSets, include an `exclude` block that covers Kyma-managed namespaces.

Kyverno policies without explicit exclusions can inadvertently match resources in Kyma-managed namespaces, such as `kyma-system` and `istio-system`. This can cause mutations or validations to interfere with platform-managed components and lead to instability or failed reconciliation of Kyma modules.

When you create or update a Kyverno policy that affects workloads but is missing the required exclusions, you receive a warning. The operation isn't blocked, but you should address the warning to avoid unintended impact on Kyma components.

Add an `exclude` block to each rule that affects workloads. Use one of the following approaches:

- Label-based exclusion (recommended): Exclude namespaces with the selector `operator.kyma-project.io/managed-by NotIn [kyma]`. This label automatically covers all current and future Kyma-managed namespaces.
- Explicit namespace exclusion: List both `kyma-system` and `istio-system` in the `exclude` block's `namespaces` field.

6.4.15 Kyma Runtime with Dual-Stack Support

Learn how to apply dual-stack support in Kyma instances.

Context

SAP BTP, Kyma runtime can support both Internet Protocol (IP) versions, IPv4 and IPv6, simultaneously (dual stack), when deployed on Google Cloud and Amazon Web Services IaaS providers. Compared to IPv4, IPv6 offers a significantly greater number of unique IP addresses, which enhances network flexibility. To retrieve support for both Internet protocols, create your Kyma runtime with dual-stack support enabled.

The dual-stack feature is only available for newly created Kyma instances. Existing instances are limited to IPv4.

> ⚠ Caution
>
> The Kyma Istio module does not support dual-stack mode. Any traffic sent or received through the IPv6 network is not part of the Istio service mesh. Consequently, the Istio module does not protect IPv6 traffic, either by transparent encryption or by offering additional security mechanisms (for example, authentication).

Procedure

Create a service instance of type `LoadBalancer` in your Kyma cluster. In the service manifest, add the `ipFamilyPolicy` field with the value RequireDualStack, and add the value IPv6 in the `ipFamilies` field.

```
apiVersion: v1
kind: Service
...
spec:
  ipFamilyPolicy: RequireDualStack
  ipFamilies:
    - IPv6
    - IPv4
...
```

In your Kyma cluster with dual-stack support, each deployed Pod automatically receives two IP addresses: IPv4 and IPv6.

Related Information

Creating an IPv4/IPv6 (dual-stack) Ingress

6.4.16 Ingress and Egress IP Addresses

Learn how to check ingress and egress IP addresses for your Kyma runtime cluster. These addresses are needed to integrate with partner systems, enforce IP-based access controls, connect to external databases, and more.

Egress IP Address

How to Check the Egress IP

You can see the IP addresses for NAT Gateway that handles the egress traffic as values of the `cloud.natGatewayIps` parameter in the `kyma-info` ConfigMap in your Kyma instance.

To check the NAT IPs, use either Kyma dashboard or kubectl:

- In Kyma dashboard, go to the *Cluster Overview* section.
- With kubectl, run:

  ```
  kubectl --namespace kyma-system get configmap kyma-info -o json | jq -r
  '.data["cloud.natGatewayIps"]'
  ```

Egress IP Stability

The NAT Gateway IP address is assigned when you create the Kyma runtime, and remains unchanged throughout the cluster's lifecycle.

Ingress IP Address

How to Check the Ingress IP

You can see the IP addresses for Istio Ingress Gateway that, by default, handles the ingress traffic. To get the LoadBalancer IP for `istio-ingressgateway`, run:

```
LOAD_BALANCER_ADDRESS=$(kubectl get services --namespace istio-system istio-
ingressgateway --output jsonpath='{.status.loadBalancer.ingress[0].ip}')
if [[ -z $LOAD_BALANCER_ADDRESS ]]; then
    LOAD_BALANCER_ADDRESS=$(kubectl get services --namespace istio-system istio-
ingressgateway --output jsonpath='{.status.loadBalancer.ingress[0].hostname}')
fi
echo "The load balancer address is ${LOAD_BALANCER_ADDRESS}"
```

When the Ingress IP Is Assigned

When you create a Kyma runtime cluster and add the Istio module, the module deploys the `istio-ingressgateway` Service of type LoadBalancer. Then, Gardener creates a cloud LoadBalancer. When it is ready, its IP address appears in the `istio-ingressgateway` Service.

When you also enable the API Gateway module, it creates a default Gateway custom resource named `kyma-gateway`. This Gateway configures the `istio-ingressgateway` as the cluster's ingress traffic entry point.

If the Istio module isn't enabled, the cluster doesn't have the `istio-ingressgateway` Service, so there's no ingress LoadBalancer IP. If the Istio module is enabled but the API Gateway module isn't, the `istio-ingressgateway` exists, but there's no default Gateway resource that routes external traffic through it. In that case, you must manually configure your own Gateway and related routing resources.

Ingress IP Stability

The external ingress IP of the `istio-ingressgateway` LoadBalancer Service changes in these cases:

- You delete the `istio-ingressgateway` LoadBalancer Service and recreate it.
- You migrate from Elastic LoadBalancer to Network LoadBalancer.
- You remove the Istio module (which deletes the `istio-ingressgateway` Service) and then add it again, which creates a new LoadBalancer.

In other cases, the ingress IP remains the same.

7 Security

Use the security features and functions of SAP BTP to support the security policies of your organization.

Security Recommendations

We provide a list with our recommendations for the configuration of our services. These recommendations help you to meet your compliance goals and secure your business.

See SAP BTP Security Recommendations.

Our customer success organization uses these recommendations as a base to create a security baseline template.

For more information, go to https://support.sap.com/sos and choose *Media Library* > *SAP CoE Security Services - Security Baseline Template*.

User Model

SAP BTP distinguishes between **platform users** (account management, custom development, and operations) and **business users** (for the applications).

See User and Member Management [page 133].

Authorizations

You can configure authorizations using **roles** and **role collections** for your global account, subaccount, directory, or individual applications.

See Security Administration: Managing Authentication and Authorization [page 2537].

Identity Providers

All users of SAP BTP are stored in identity providers, either in the default or in a custom identity provider. SAP BTP needs a copy of the user, sometimes called a shadow user. You assign the shadow user authorizations to access resources in SAP BTP. When a user authenticates, SAP BTP forwards the request to the identity provider.

For more information, see Trust and Federation with Identity Providers [page 2538].

> **Note**
>
> For the China (Shanghai) region, a different default identity provider is used.
>
> For more information, see this blog article on *SAP Community*.

Default Identity Provider

We provide a default identity provider for both platform users and business users (in applications) at SAP BTP. The default identity provider enables single sign-on to your SAP applications and services.

Use the default identity provider as a preconfigured user store in your starter scenarios or for testing. You can also use the default identity provider as a backup identity provider if access to your custom identity provider fails.

See Default Identity Provider [page 2594].

Identity Authentication Service

Identity Authentication service provides authentication and single sign-on in the cloud.

We recommend that you configure the Identity Authentication service as the identity provider and connect Identity Authentication to your own corporate identity provider. Identity Authentication provides features that the default identity provider doesn't, such as the ability to connect your corporate identity provider or to define security policies.

See Trust and Federation with Identity Providers [page 2538].

For more information about Identity Authentication, see SAP Cloud Identity Services - Identity Authentication.

Transport Layer Security (TLS) Connectivity Support

SAP BTP uses encrypted communication channels based on HTTPS/TLS, supporting TLS version 1.2 or higher.

> **Note**
>
> TLS versions 1.0 and 1.1 are no longer supported.

Make sure you use HTTP clients (such as web browsers) that support TLS version 1.2 or higher for connecting to SAP BTP.

> **Note**
>
> You can optionally use TLS 1.3 in the Custom Domain Manager. This option allows the use of TLS 1.3 with applications running on SAP BTP. It's not allowed to use TLS 1.3, for example for the SAP BTP cockpit or SAP Cloud Identity Services. These services are still using TLS 1.2.

See What Is Custom Domain.

Audit Logging

Use the Audit Log Retrieval API to view the audit logs stored for your subaccount. Use the audit log viewer to display the audit logs for your Cloud Foundry account, produced by SAP applications and services you've subscribed to. See Audit Logging in the Cloud Foundry Environment [page 2878].

Credential Store

SAP Credential Store provides a repository for passwords and keys for applications that are running on SAP BTP, Cloud Foundry environment. It enables the applications to retrieve credentials and use them for authentication to external services, or to perform cryptographic operations and TLS communication.

See SAP Credential Store.

Malware Scanning

Use the SAP Malware Scanning service to scan business documents for malware. Integrate this service with your custom-developed apps running on the Cloud Foundry runtime. When your apps upload business documents, your apps can call the SAP Malware Scanning service to check for viruses or other malware.

For more information, see SAP Malware Scanning Service.

Related Information

SAP Authorization and Trust Management Service [page 3538]
Audit Logging in the Cloud Foundry Environment [page 2878]
Principal Propagation [page 3617]
Data Protection and Privacy [page 3633]
Security in the Kyma Environment [page 3644]

7.1 SAP Authorization and Trust Management Service

The global account and subaccounts get their users from identity providers. Administrators make sure that users can only access their dedicated subaccount by making sure that there is a dedicated trust relationship

only between the identity providers and the respective subaccounts. Developers configure and deploy application-based security artifacts containing authorizations, and administrators assign these authorizations using the SAP BTP cockpit.

> **Note**
>
> Before you start, make yourself familiar with the sections about authentication and authorization of the SAP BTP Planning and Lifecycle-Management Guide. See the Setting Up Your Security and Compliance Model section.

- #unique_392/unique_392_Connect_42_subsection-im1 [page 3540]
- #unique_392/unique_392_Connect_42_subsection-im2 [page 3540]
- #unique_392/unique_392_Connect_42_subsection-im3 [page 3541]
- #unique_392/unique_392_Connect_42_subsection-im4 [page 3542]
- #unique_392/unique_392_Connect_42_subsection-im5 [page 3543]
- #unique_392/unique_392_Connect_42_subsection-im6 [page 3544]
- #unique_392/unique_392_Connect_42_subsection-im7 [page 3544]

Hold your pointer over a box for a description. Select a box to display more information.

Overview of the SAP Authorization and Trust Management Service

Get a high-level overview of the concepts that underpin the SAP Authorization and Trust Management service for SAP BTP in the Cloud Foundry environment.

For more information, see What Is the SAP Authorization and Trust Management Service? [page 3545].

Tutorials for the SAP Authorization and Trust Management Service

Follow the tutorials below to get familiar with the SAP Authorization and Trust Management service in the Cloud Foundry environment of SAP BTP.

Tutorials for the SAP Authorization and Trust Management service in the Cloud Foundry environment	Language / Framework	Link
Learn how to secure a basic single-tenant Node.js application. Start with a Node.js application that uses the express framework and SAPUI5 to display a list of products and add the security components step by step.	Node.js	SAP Developers
Learn how to secure a basic java application. This tutorial starts with a Hello World Java application built with SAP Cloud SDK.	Java, SAP Cloud SDK	SAP Developers
Learn how to build a cloud-native Node.js application that features secured service-to-service communication. The application shows you two different ways of securing service-to-service-communication (by propagating a business user or using a technical user).	Node.js	GitHub
Learn how to secure microservices in SAP BTP using `spring-xsuaa` and Spring security. Furthermore, learn how to test the secured application using the `java-security-test` utilities.	Spring (Boot)	GitHub
Learn how to add multitenancy to a node.js application and make it available for other subaccounts using the SaaS Provisioning service and the XSUAA.	Node.js	SAP Developers

Tutorials for the SAP Authorization and Trust Management service in the Cloud Foundry environment	Language / Framework	Link
Learn how to secure microservices in SAP BTP. This sample provides J2EE Configuration using web.xml and uses the SAP Java Buildpack.	J2EE, SAP Java Buildpack	GitHub (SAP Java Buildpack version >=1.26.1) GitHub (SAP Java Buildpack version <= 1.26.0)
Learn how to use the `java-security` library to perform JWT Validation as part of your Java application. Furthermore, learn how to test the secured application using the `java-security-test` utilities.	Java	GitHub
Learn how to validate OAuth tokens using a Python library. Use this library to add authentication in your Python application. **Note** This library isn't part of an SAP BTP license. However, it belongs to a related open source project.	Python	GitHub
Learn in this reference application how the service fits into a complete architecture of microservices that interact with each other propagating user information.	Java	GitHub

Principal Propagation

Exchange user ID information between systems or environments in SAP BTP.

In This Section

- Principal Propagation from the Multi-Cloud Foundation to the Neo Environment [page 3624]
- Principal Propagation from the Neo Environment to the Multi-Cloud Foundation [page 3617]

Other Principal Propagation Scenarios

- On-Premise User Store
- Principal Propagation to OAuth-Protected Applications
- Connectivity in the Cloud Foundry Environment: Principal Propagation
- Connectivity in the Neo Environment: Principal Propagation

Trust and Federation

When setting up accounts you need to assign users. While we provide you with your first users from the default identity provider to get you started, your organization has identity providers that you want to integrate.

SAP BTP supports identity federation, a concept of linking and reusing digital identities of a user base across loosely coupled systems. Identity federation frees applications on SAP BTP as well as the platform itself from the need to obtain and store the credentials of users and to authenticate them. Instead, the user base is reused from identity providers, which support the administration of digital user identities, authentication, and authorizations in a centralized and decoupled manner. To enable communication between SAP BTP and identity providers, you must cross-configure the communication endpoints of the involved systems, establishing a trust relationship between them.

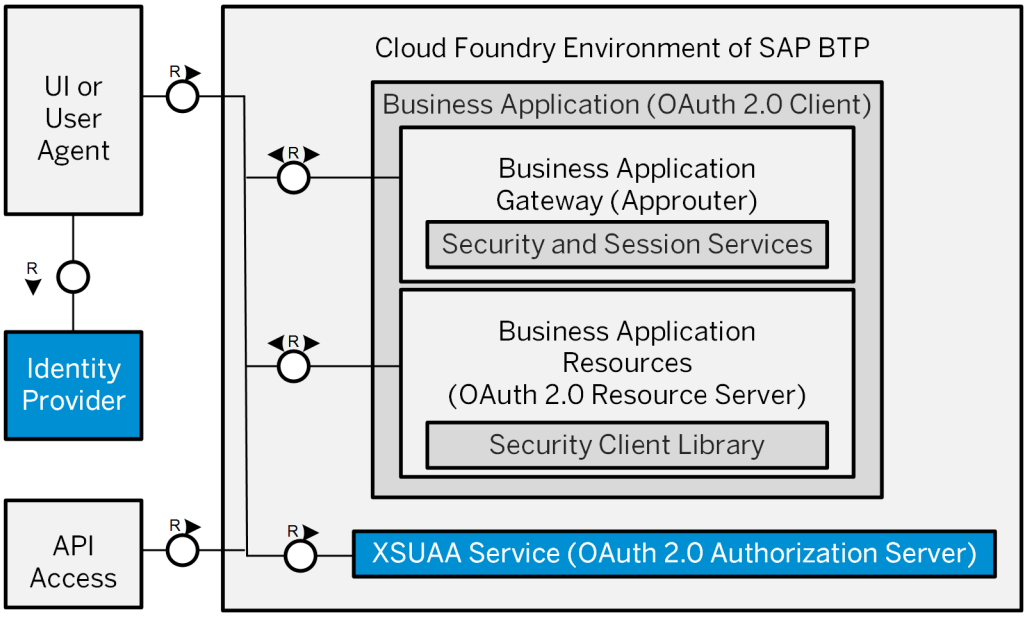
→ Recommendation

We strongly recommend that you use a custom tenant of SAP Cloud Identity Services. Using SAP Cloud Identity Services eases integration with other SAP solutions. When setting up accounts you need to assign users. We provide you with your first users from the default identity provider. By using SAP Cloud Identity Services as a proxy to your corporate identity provider and hosting your users there, you gain a number of advantages:

- Integrate the management of these users with your broader identity management strategy, hosted on your own identity providers. You control your own user lifecycle and single sign-on strategies throughout the entire landscape.
- Enforce your own password and authentication policies, such as stronger passwords or multifactor authentication.

For platform users, the use of SAP Cloud Identity Services is mandatory. If you don't have a tenant yet, check Getting a Tenant.

To connect your corporate identity provider to SAP Cloud Identity Services, see Corporate Identity Providers and Configure Conditional Authentication for an Application in What Is Identity Authentication and SAP Cloud Identity Services



Identity Provider and XSUAA in SAP BTP Architecture

SAP Cloud Identity Services is a multitenancy-enabled identity provider for all SAP cloud applications and optionally on-premise applications. The service provides capabilities for authentication, single sign-on, authorizations, identity lifecycle management, and on-premise integration as well as self-services like self-registration or password reset.

SAP has its own SAP Cloud Identity Services tenant, SAP ID service. SAP ID service is the default identity provider of SAP BTP and where you register to get initial access to SAP BTP. Trust to SAP ID service is preconfigured by default.

We recommend that you request your own SAP Cloud Identity Services tenant (see Getting a Tenant). To establish trust with your identity provider, proceed as follows.

For business users:

- Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2540]
- Manually Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2547]
- Establish Trust and Federation with UAA Using Any SAML Identity Provider [page 2551]

For platform users:

- Establish Trust and Federation of Custom Identity Providers for Platform Users [page 2576]

For default identity provider:

- Default Identity Provider

> **Note**
>
> How you assign users to their authorizations depends on the type of trust configuration. If you're using the default trust configuration via SAP ID service, you can assign users directly to role collections. For more information, see Default Identity Provider [page 2594].
>
> However, if you're using a custom trust configuration as described in this topic, you can assign individual users or groups to role collections. Assigning users to their authorizations is part of application administration, which is described here. For more information, see Mapping Role Collections [page 2620].

Administration: Managing Authentication and Authorization

Application developers create and deploy application-based authorization artifacts for business users. Administrators use this model to manage roles, build role collections, and assign these collections to users or user groups. In this way, they control the users' permissions.

Setting Up Authorization Artifacts (Account Administrators)

Task	Links
Assign the role collection to the users provided by an identity provider	Working with Role Collections [page 2612]
(If you do use a custom identity provider) Assign the role collections to user groups	Map Role Collections to User Groups [page 2621]

Task	Links
Assign the role collections to users and user groups, manage attribute mappings	Mapping Role Collections [page 2620]
Create a role collection and assign roles to it	Define a Role Collection [page 2614]
Use an existing role or create a new one using role templates	Add Roles to Role Collections on the Application Level [page 2632]

Troubleshooting

This section provides information on troubleshooting-related activities for the SAP Authorization and Trust Management service in the Cloud Foundry environment.

> → Tip
>
> We also recommend that you regularly check the SAP Notes and Knowledge Base for component `BC-CP-CF-SEC-IAM` in the SAP Support Portal. These contain information about program corrections and provide additional information.
>
> To help you troubleshoot your issue, we also recommend increasing the log verbosity of your application and application router. We provide a script to help you. If for some reason you can't use this script, increase the log verbosity manually, see related link.
>
> To troubleshoot problems with tokens from SAP Cloud Identity Services, see Logging OpenID Connect Tokens in the documentation for SAP Cloud Identity Services.

Our troubleshooting information can be found in Troubleshooting for SAP Authorization and Trust Management Service [page 3562]. Check the individual troubleshooting topics for your error message. If you can't find your problem, create an incident in the component `BC-CP-CF-SEC-IAM`. For more information, see Monitoring and Troubleshooting [page 3552].

Adding Authentication and Authorization

Developers create authorization information for business users in their environment and deploy this information in an application. They make this available to administrators, who complete the authorization setup and assign the authorizations to business users.

Developers store authorization information as design-time role templates in the `xs-security.json` security descriptor file. Using the cockpit, administrators of the environment assign the authorizations to business users.

The following sections contain the process of adding authentication and authorization checks for protecting your applications, links to a number of associated tutorials, extended tasks for creating authorization artifacts, as well as reference information, including the syntax required to set the properties and values defined in the application security descriptor file.

- Protecting Your Application [page 549]
- Tutorials for the SAP Authorization and Trust Management Service [page 574]
- Application Security Descriptor Configuration Syntax [page 592]

7.1.1 What Is the SAP Authorization and Trust Management Service?

Get a high-level overview of the concepts that underpin the SAP Authorization and Trust Management service for SAP BTP in the Cloud Foundry environment.

The SAP Authorization and Trust Management service lets you manage user authorizations and trust to identity providers. Identity providers are the user base for applications. We recommend that you use an SAP Cloud Identity Services tenant, an SAP on-premise system, or a custom corporate identity provider. User authorizations are managed using technical roles at the application level, which can be aggregated into business-level role collections for large-scale cloud scenarios.

Environment

The SAP Authorization and Trust Management service is available for consumption in the following SAP BTP environments:

- Cloud Foundry
- Kyma
- Neo

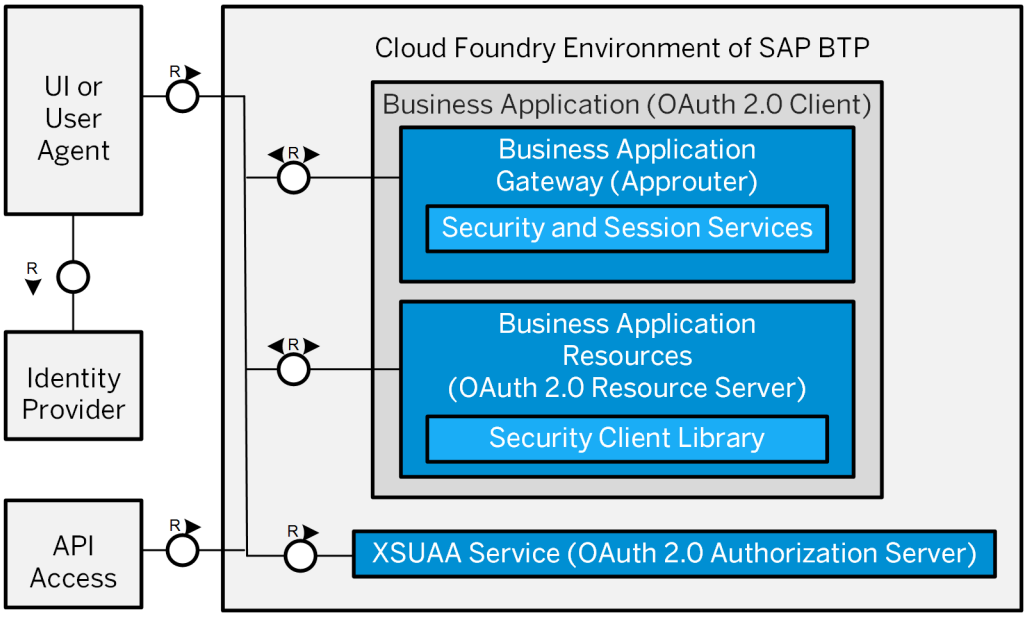
> **Note**
>
> This documentation refers to SAP BTP for multi-environment subaccounts.
>
> If you're looking for documentation about the Neo environment, see Authorization and Trust Management in the Neo Environment.

Features

Use your corporate or a default IdP	Enable user management for your applications by handling authentication to an external identity provider. Start with SAP ID service as a pre-configured easy-to-use identity provider. Switch to your corporate identity provider for customized user management.
Enable role-based access to applications	Enable different privileges to users accessing your applications based on roles.

Overview

The following figure shows a high-level overview of components, which comprise an SAP BTP business web applications and how these applications are embedded in the Cloud Foundry environment. Further details have been omitted for the sake of simplicity. It's further assumed that the Cloud Foundry environment is set up with basic configuration (that is, container-to-container networking isn't configured). The components and their interactions are depicted in the following block-diagram.



Application, Microservice, and App

The components and their interactions are depicted in the previous block-diagram. It shows a runtime platform for business web applications. These are referred to as applications. An SAP BTP application is implemented in an architectural style that structures the application as a collection of loosely coupled components, termed microservices. Microservices can be deployed independently from one another. This eliminates the need to deploy the complete application if only a subset of its microservices have received new features or a bug fix. In the terminology of the Cloud Foundry environment of SAP BTP, microservices are referred to as apps.

Application Architecture

The application consists of a distinct gateway app with at least one or more resource apps. The gateway serves as a reverse-proxy and provides functionality for security and session management. The application router app is a standard implementation of the gateway and is used as the single point-of-entry for the application. It also serves static content, initiates the authentication process, checks on cross-site request forgery (XSRF) attacks, and forwards requests to the resource apps while propagating user information.

The resource apps can use the security client library, which also provides security functionality. As stated in the previous section, all apps represent microservices, in the meaning of an app engineered and operated according to the 12-factor paradigm. All apps run in their dedicated runtime containers, which are hosted on the Cloud Foundry runtime platform.

OAuth 2.0, Resource Owner, Client, Resource Server, and the SAP Authorization and Trust Management Service

The security functionality of SAP BTP is based on the OAuth 2.0 specification. OAuth 2.0 defines how a user - the OAuth 2.0 resource owner - can delegate all or a subset of the authorizations to a third-party application - the OAuth 2.0 client - without the third-party application needing to know the credentials of the user.

The Cloud Foundry environment uses a standard implementation of OAuth 2.0 to protect its platform resources (orgs, spaces, and platform operations on those entities).

However, the OAuth 2.0 specification is reused for SAP BTP with a proprietary implementation to protect the resources of business web applications powered by the Cloud Foundry environment. The proprietary implementation exchanges the responsibilities of the OAuth 2.0 entities, client and resource owner: the OAuth

2.0 client - represented by the application - holds all the authorizations. A set or sub-set of these authorizations is assigned to the user after authentication in the system. The application also acts as the OAuth 2.0 resource server because it contains the resource apps. All apps of an application operate under the same OAuth 2.0 client.

The SAP Authorization and Trust Management service (XSUAA) provides functionality for administrating and assigning application authorizations. It acts as the OAuth 2.0 authorization server and represents a typical reuse service. The SAP Authorization and Trust Management servicebroker creates a service instance for each application. Each app that wants to enforce authorizations with the security client library is then bound to this SAP Authorization and Trust Management service instance of the corresponding application.

> ### Note
>
> The Cloud Foundry environment also supports the following token grant types of Cloud Foundry.
>
> - Authorization code grant
> - Client credentials grant
> - SAML 2.0 bearer grant
>
> Refresh tokens are supported as well.
>
> For more information, see the Cloud Foundry environment's API reference of the User Account and Authentication and the related link.

Service, Service Broker, and Service Instance

A service is an app that includes service broker functionality. The service broker must implement the Open Service Broker API specification, for which the Cloud Foundry environment is a client. The service broker is responsible for advertising its service offerings and service plans to the Cloud Foundry environment of SAP BTP and acting on requests from the platform for provisioning, binding, unbinding, and deprovisioning.

A service instance represents a reserved resource and is an instantiation of a service offering for a service plan. The service offering is the advertisement of a service that the service broker supports. The service plan is a variant of the service offering and usually represents the costs and benefits of that plan.

The service broker binds the service instance to the consuming apps. The service binding contains information about the service (for example, URL, credentials), which the app uses to consume the service. Apps and services communicate with one another indirectly, using the Cloud Foundry router.

Reuse Service, Backing Service

As a service is an app that includes service broker functionality, it can also run as a microservice component within an application. In this case, the service represents a reuse service. Services that don’t represent components within applications, but rather run standalone, are referred to as backing services.

Authentication Against Trusted Parties Only

The apps of the applications are accessed either using the UI of a user agent or the APIs that the app provides. All requests must first go through the Cloud Foundry router. Users of a user agent must first authenticate against a configured identity provider. The identity provider and the SAP Authorization and Trust Management service have a special trust relationship, which is established through crossover metadata configuration between the two systems (not depicted in the diagram above). Authentication is processed according to the SAML bearer assertion flow and initiated during the processing of the first request. A series of redirects leads to a request for authentication from the user agent towards the identity provider. After the user authenticates successfully, the identity provider responds with a SAML bearer assertion confirming the user's identity. This

SAML bearer assertion is presented to SAP Authorization and Trust Management service and the service determines the authorizations of that user. API clients receive their credentials directly from, and authenticate directly against, SAP Authorization and Trust Management service.

JSON Web Tokens

A JSON web token (JWT) (according to RFC 7519) is an open standard that defines a compact token format for transmitting information between parties as a JSON object. This information can be verified and trusted because it is digitally signed. JWT tokens can be signed using a secret key pair (with HMAC algorithm) or a public/private key pair using RSA.

The JWT token contains header and claims information (for example, issuer, subject, expiration time, consumer-defined information), and is digitally signed with the private key of the authorization server (UAA service).

The cloud or business application has a trust relationship with the authorization server. The trust is configured in the `<VCAP_SERVICES>` environment variable for the application router and each microservice of the business application. `<VCAP_SERVICES>` contains a credentials string for the UAA, which is created by the respective service broker when the application router and/or the microservice is bound to the UAA service instance. The credentials string contains, among other things, the public key corresponding to the private key of the UAA. This public key is used to verify the token signature.

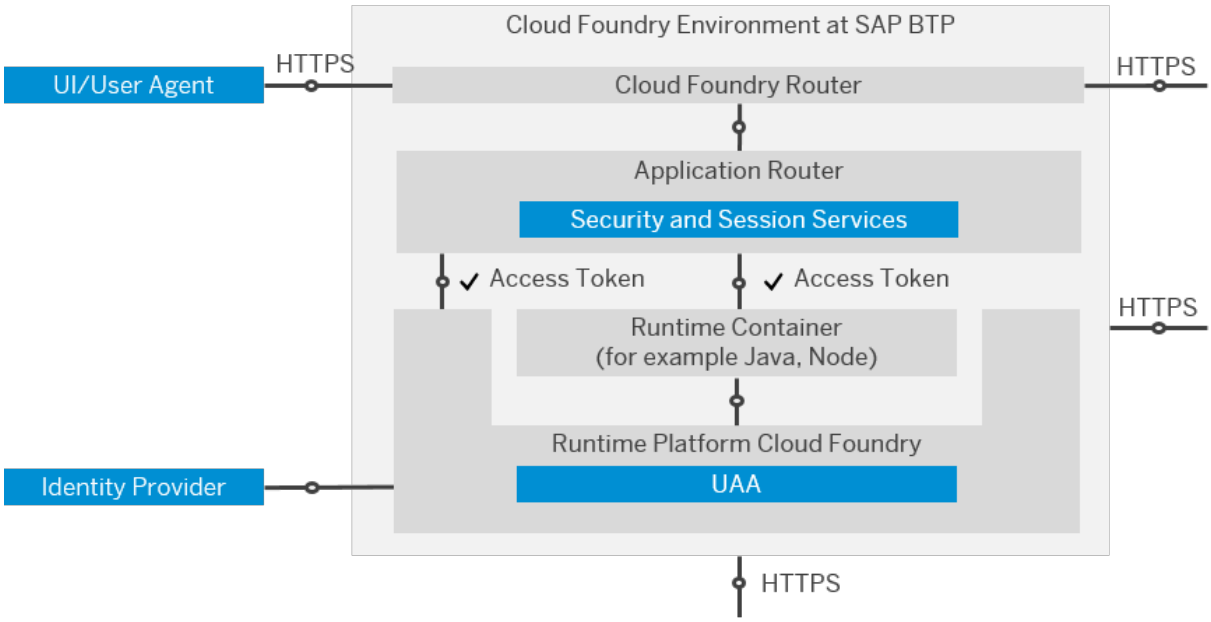
Related Information

http://saml.xml.org/saml-specifications
https://tools.ietf.org/html/rfc6749#section-1.4
SAP Authorization and Trust Management Service [page 3538]
Cloud Foundry API Reference for User Account and Authentication Server API
Security in the Kyma Environment [page 3644]

7.1.2 Web Access Control

The Cloud Foundry environment extends SAP BTP. It provides platform security functions such as business user authentication, authorization management, and other security functions for access to the applications in the runtime container. To access the runtime container, the business user can use a browser or a browser-based user interface.

The following diagram shows the architecture with the components that are responsible for business user authentication, authorization management, and security. It is not mandatory for applications to use the User Account and Authentication service and the application router.



The User Account and Authentication (UAA) component provides a programming model for business applications. It is the central infrastructure component of the runtime platform for business user authentication and authorization management. The users can be stored in the following identity providers:

- SAP ID service
- SAP Cloud Identity Services
- Any SAML 2.0 identity provider

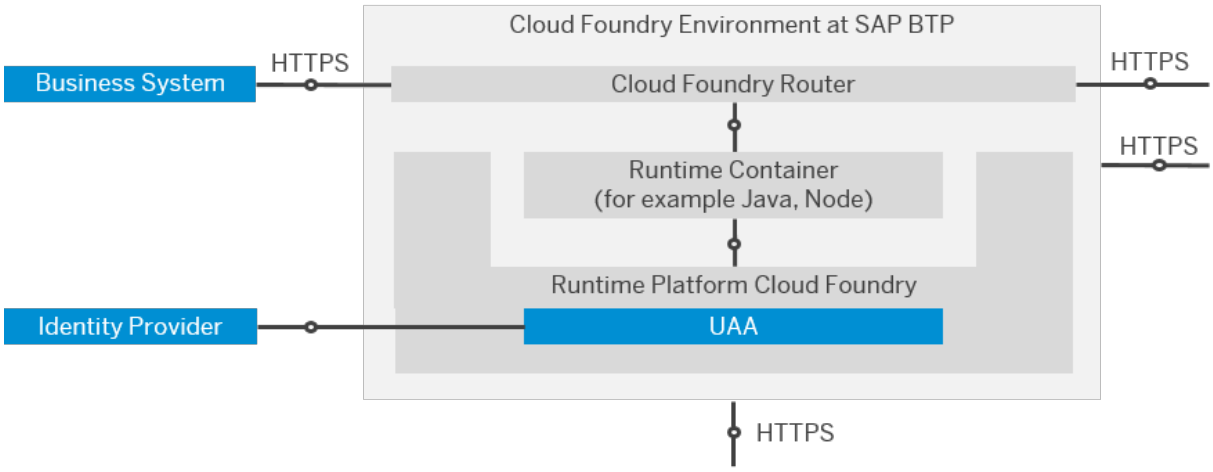
Applications authenticate using OAuth 2.0. When business users access an application, the application router acts as OAuth client and redirects their request to the OAuth authorization server for authentication (see the *Applications* section). Runtime containers act as resource servers, using the container security API of the relevant container (for example, Java) to validate the token issued by the OAuth authorization server.

7.1.3 API Access Control

The Cloud Foundry environment extends SAP BTP. It provides platform security functions such as business system authentication, authorization management, and other security functions to enable business systems to access the applications (for example, Java or Node.js) in the runtime container. Business systems use APIs to access the runtime container.

A business system uses APIs to directly access the resources in the runtime container.

The following diagram shows the architecture with the components that are responsible for business system authentication, authorization management, and security.



The User Account and Authentication (UAA) component is the central infrastructure component of the runtime platform for authentication and authorization management. The users can be stored in the following identity providers:

- SAP Cloud Identity Services
- Any identity provider

Business system use APIs to directly access the resources in the runtime container. The UAA acts as an OAuth authorization server and issues an appropriate access token. It enables the business system to directly access an application in the runtime container. Runtime containers act as OAuth resource servers, using the container security API of the relevant container (for example, Java) to validate the token issued by the OAuth authorization server.

Related Information

Protecting Your Application [page 549]
Tutorials for the SAP Authorization and Trust Management Service [page 574]

7.1.4 Authorization Entities

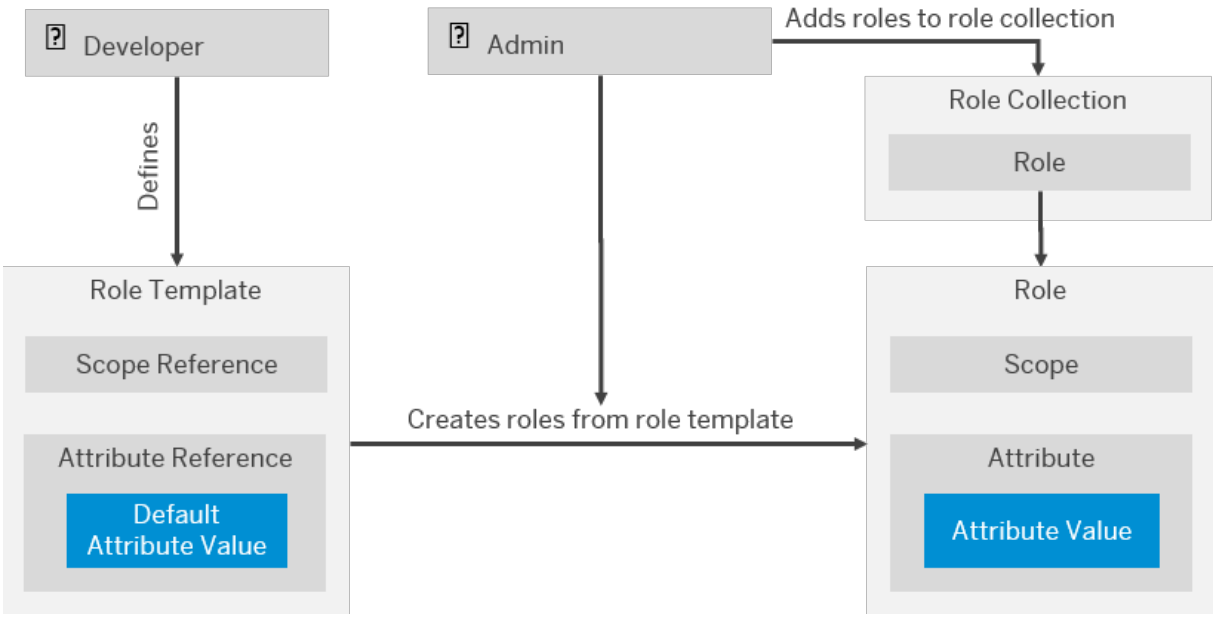
Business users in an application require different authorizations because they work in different jobs.

For example, in a leave request process, there are employees who want to create and submit leave requests, managers who approve or reject, and payroll administrators who need to see all approved leave requests to calculate the leave reserve. The authorization concept of a leave request application has to cover the needs of these employee groups. This authorization concept includes elements such as roles, scopes, and attributes.

A role is an instance of a role template; you can build a role based on a role template and assign the role to a role collection.

Role templates refer to attributes and scopes. The role templates contain the authorizations for activities, such as viewing, editing, or deleting data.

Information that is specific to the user is stored in attributes. For each attribute, administrators can specify the value that restricts data access. Static attributes are stored in the role, whereas in cases in which a custom identity provider provides the business users, you can dynamically reference all the attributes that come with the access token.



Relationship Between Authorization Components

Scopes for Functional Authorization Checks

The application defines scopes, which describe the authorizations required by the application to execute functions such as editing, viewing, and deleting.

The application security descriptor file (`xs-security.json`) defines the scopes used by the application.

Role Templates

A role template is a description of one or more roles (for example, employee or manager) and any attributes that apply to those roles.

If a role template contains attributes, the administrator must create different versions of the roles and fill in the attributes that are subject to customization; for example, one where country equals UK and another where country equals USA. This instantiates the role template. Else, the roles will have empty attributes (with no concrete values).

Role templates that contain only application-specific local scopes can be instantiated without the administrator having to do anything.

Role Collections

Role collections reference role templates.

After the developer has created the role templates and deployed them to the relevant application, it's the administrator's task to use those role templates to build roles with the required attributes, aggregate the roles into role collections, and then assign the role collections to business users in the application.

Attributes

Information retrieved from the user's identity (such as department or cost center) is stored in attributes. Attributes refine the authorizations of business users according to the attributes that come with the business users. For example, you can use static attribute values such as country equals USA, or dynamic attribute values from the access token or identity provider, such as group ID.

7.1.5 Monitoring and Troubleshooting

This section provides information on troubleshooting-related activities for the SAP Authorization and Trust Management service in the Cloud Foundry environment.

> → Tip
>
> We also recommend that you regularly check the SAP Notes and Knowledge Base for component `BC-CP-CF-SEC-IAM` in the SAP Support Portal. These contain information about program corrections and provide additional information.
>
> To help you troubleshoot your issue, we also recommend increasing the log verbosity of your application and application router. We provide a script to help you. If for some reason you can't use this script, increase the log verbosity manually, see related link.
>
> To troubleshoot problems with tokens from SAP Cloud Identity Services, see Logging OpenID Connect Tokens in the documentation for SAP Cloud Identity Services.

Our troubleshooting information can be found in Troubleshooting for SAP Authorization and Trust Management Service [page 3562]. Check the individual troubleshooting topics for your error message. If you can't find your problem, create an incident in the component `BC-CP-CF-SEC-IAM`. For more information, see Monitoring and Troubleshooting [page 3552].

Related Information

Getting Support [page 3683]
Enable and Provide Application Logs [page 3553]
Auditing and Logging Information for SAP Authorization and Trust Management Service [page 3555]

7.1.5.1 Enable and Provide Application Logs

If there are authentication problems in your application, enable logging for the container security library in question, reproduce the problem, and attach the application logs. To obtain more details, set the environment variables for the application.

Context

> → Tip
>
> We also provide a log collector script , that helps you to increase the log verbosity of your application and application router.

Procedure

1. Open a command prompt.
2. Log on to your Cloud Foundry environment using the Cloud Foundry command line interface (CLI). For more information, see Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface [page 2814].
3. Choose your organization.

   > ⓘ Note
   >
   > The Cloud Foundry command line interface prompts you to choose an org. To find the org of your subaccount, use the SAP BTP cockpit to go to your subaccount. You find the org in the Cloud Foundry tile under *Organization* (see Navigate to Orgs and Spaces [page 2792]).

4. Choose the space where the application is located.
5. To route the log messages to the standard output, use the following command:

   ```
   cf set-env <application_name> SAP_EXT_TRC stdout
   ```

   > ✤ Example
   >
   > ```
   > cf set-env your-app SAP_EXT_TRC stdout
   > ```

6. To set the log level, use the following command:

   ```
   cf set-env <application_name> SAP_EXT_TRL 3
   ```

   > ✤ Example
   >
   > ```
   > cf set-env your-app SAP_EXT_TRL 3
   > ```

7. (For Node.js) To set detailed logs of the Security API for Node.js, use the following command:

   ```
   cf set-env <application_name> DEBUG xssec*
   ```

   > Example
   >
   > ```
   > cf set-env your-app DEBUG xssec*
   > ```

8. Restage your application using the following command:

   ```
   cf restage <application>
   ```

   > Example
   >
   > ```
   > cf restage your-app
   > ```

   Restaging the application enables the display of the logs.

9. We recommend piping the output to a log file. Use the following command to do this:

   ```
   cf logs <application> > <log_file_name>
   ```

   > Example
   >
   > ```
   > cf logs your-app > your-app-log.txt
   > ```

10. Reproduce the problem in your application. The events that occur in your application are logged in your local log file.

    > Note
    >
    > You can stop the recording of the log messages using `CTRL+C`. You can revert the environment variables using the following command:
    >
    > ```
    > cf unset-env <application> SAP_EXT_TRC
    > ```
    >
    > > Example
    > >
    > > ```
    > > cf unset-env your-app SAP_EXT_TRC
    > > ```
    >
    > Restage your application using the following command:
    >
    > ```
    > cf restage <application>
    > ```
    >
    > > Example
    > >
    > > ```
    > > cf restage your-app
    > > ```

11. Create an incident for your local support. Create an incident using the component `BC-CP-CF-SEC-IAM`. Use the SAP Support Portal. For more information, see Monitoring and Troubleshooting [page 3552].
12. Attach the log files to the incident.

To obtain more details about the token validation, set the following environment variables of the container security library and Node.js (if applicable):

SAPSSOEXT Environment Variables

Environment Variables	Description
SAP_EXT_TRC	Enablement of logs
SAP_EXT_TRL	Log level ranging from off to high Values (integer): • **0** (off) • **1** (default) • **2** (medium) • **3** (high)

Node.js Environment Variables

Environment Variables	Description
DEBUG	Detailed logs of the security API for Node.js Value (string): **xssec***

7.1.5.2 Auditing and Logging Information for SAP Authorization and Trust Management Service

SAP Authorization and Trust Management service (XSUAA) logs security events. These events supplement the events of the Cloud Foundry User Account and Authentication service (UAA).

Troubleshoot Identity Provider Issues

To troubleshoot issues with the identity provider, use tools based on the trust configuration:

- For SAML identity providers, use the network trace of your browser.
  For more information, see 2461862.
- For OIDC identity providers, use the troubleshooting tools of the identity provider.
  For SAP Cloud Identity Services, see Logging OpenID Connect Tokens.

Security Events of the User Account and Authentication Service

For information about the security events of the UAA, see UAA Audit Requirements in the Cloud Foundry documentation.

Security Events of the SAP Authorization and Trust Management Service

The SAP Authorization and Trust Management service uses role collections to manage accounts.

SAP Authorization and Trust Management service records all changes in its database tables and summarizes them in the audit log. The following table summarizes the audit log entries.

> ### Note
>
> - When messages exceed 4 KB, the system breaks the messages into multiple entries. The system identifies each message with a `msgId` GUID and the parts with a `msgNo` index. To view the whole response, gather the parts and combine the contents together.
> - `UNKNOWN_USER` in an audit log message is a placeholder, that appears when the system writes for flows that don't involve users, such as client credential flows.

Security Events Written in Audit Logs

Event grouping	What events are logged	How to identify related log events	Additional information
Identity provider management	Trust identity provider.	`object with type "IdentityProvider" and id consisting of: crudType "CREATE"`	Attributes of the identity provider.
		`tableName "xs_tenant", crudType "UPDATE"`	When you trust an identity provider using OpenID Connect, the system triggers a change in the XSUAA tenant.
	Update trust with identity provider.	`object with type "IdentityProvider" and id consisting of: crudType "UPDATE"`	Old attributes of the identity provider and any attributes required to identify the changes.
		`tableName "xs_tenant", crudType "UPDATE"`	Updating trust in an identity provider using OpenID Connect can trigger a change in the XSUAA tenant.
	Delete trust with identity provider.	`object with type "IdentityProvider" and id consisting of: crudType "DELETE"`	Attributes of the service instance.

Event grouping	What events are logged	How to identify related log events	Additional information
		`tableName "xs_tenant", crudType "UPDATE"`	Removing trust in an identity provider using OpenID Connect triggers a change in the XSUAA tenant.

Instance management	Create instance.	`Attribute with name "complete" and value ""ServiceInstanceId : <Instance_ID>`
		...
		`object with type "<Instance_Name>" and id consisting of: crudType "CREATE"`
			Attributes of the service instance.

	Update instance.	`Attribute with name "complete" and value ""ServiceInstanceId : <Instance_ID>`
		
		`object with type "<Instance_Name>" and id consisting of: crudType "UPDATE"`
			Old attributes of the service instance and any attributes required to identify the changes.

	Delete instance.	`Attribute with name "complete" and value ""ServiceInstanceId : <Instance_ID>`
		...
		`object with type "<Instance_Name>" and id consisting of: crudType "DELETE"`
			Attributes of the service instance.

Event grouping	What events are logged	How to identify related log events	Additional information
	Note If you create a new binding secret for the service instance, the system creates a new binding entry in the SAP Authorization and Trust Management service. This entry then appears in the audit log of the subaccount of the service instance. The audit log of the subaccount of the reuse service itself remains unaffected.		
Role collection management	Create role collection.	`tableName "xsrolecollections", crudType "CREATE"` audit.configuration	Other attributes: • Timestamp • Origin key • Role collection name • Zone ID
	Assign users to role collection.	Direct assignment to users: `tableName "xs_rolecollection2user", crudType "CREATE"` Mapping to users with attributes: `tableName "xsrolecollection2samlattribute", crudType "CREATE"` audit.configuration	Other attributes for direct assignment: • Timestamp • Origin key • User SCIM ID • Zone ID • Role collection name Other attributes for mapping: • Timestamp • Origin key • Attribute name • Attribute value • Identity provider URL • Zone ID • Role collection name
	Assign roles to role collection.	`tableName "xsrolecollection2role", crudType "CREATE"`	• Timestamp • Origin key • Role name • Role zone ID • Role template name • Role collection zone ID • Role template app ID • Role collection name

Event grouping	What events are logged	How to identify related log events	Additional information
	Delete role collection.	`tableName "xsrolecollections", crudType "DELETE"` audit.configuration	Other attributes: • Timestamp • Origin key • Role collection name • Zone ID
Role management	Create role.	`tableName "xsrole", crudType "CREATE"` audit.configuration	Other attributes: • Timestamp • Origin key • App ID • Role name • Role template name • Zone ID
	Modify attribute values in role.	`tableName "xsattribute2role", crudType "CREATE"` audit.configuration	Other attributes: • Timestamp • Origin key • Role name • Zone ID • Attribute app ID • Role template name • Attribute name • Attribute value • Role template app ID
	Delete role.	`tableName "xsrole", crudType "DELETE"` audit.configuration	Other attributes: • Timestamp • Origin key • Role template app ID • Role name • Role template name • Zone ID

Event grouping	What events are logged	How to identify related log events	Additional information

Emergency administrator assignment	Assign global account or subaccount administrator in case of an emergency
	
	Example
	• Assignment of a subaccount administrator by a global account administrator
	• Assignment of a global account administrator by an authorized user
		`"crudType":"CREATE","type":"Global Account Administrator assignment"`
		
		or
		
		`"crudType":"CREATE","type":"Subaccount Administrator assignment"`
		
		audit.security-events
			The user who assigned the permission is identified by the user name and by the origin key (such as `sap.default` for the default identity provider).
			
			Further information included in the log:
			• Initiating user name
			• Initiating user origin
			• New administrator's user name
			• New administrator's origin

Token Embedding	Embedding error **Caution** The initial token retrieval might not include the corporate identity provider token. In this case, retry the operation. The retry might not yield the desired token, so ensure your application can handle this situation.	`Error during handling of IAS Tokens for embedding.` audit.security-events	This error occurs when attempting to exchange a token for a token with an embedded token from SAP Cloud Identity Services or a corporate identity provider. Check the configuration of the application. For more information, see Include Tokens from Corporate Identity Providers or SAP Cloud Identity Services in Tokens of the SAP Authorization and Trust Management Service [page 587].

Event grouping	What events are logged	How to identify related log events	Additional information

SAML authentication	Authentication error	`SAMLAuthenticationError`
		`Response issue time is either too old or with date in the future. Sync IdP to match skew <skew>`
		audit.security-events
			This error occurs when the time skew between SAP Authorization and Trust Management service and the identity provider is larger than 60 seconds. Or the authentication response took more than 60 seconds to reach the SAP Authorization and Trust Management service after being issued.
			Check the time skew between the identity provider and SAP Authorization and Trust Management service. Synchronize the clock of the identity provider.

		`SAMLAuthenticationError`
		`Unexpected AuthnResponse : Existing authentication - <user>`
		audit.security-events
			The user has probably chosen the back button on the browser, triggering a second authentication request to the identity provider with the same user ID. The identity provider issues a second authentication response for the same user ID. SAP Authorization and Trust Management service rejects duplicate responses.

		`SAMLAuthenticationError`
		`AuthnRequest expired - ID: <request_id> Destination: <identity_provider_destination>`
		audit.security-events
			This error occurs when an authentication response from an identity provider takes more than 15 minutes.
			If this error occurs consistently, investigate why your identity provider needs more than 15 minutes to issue an authentication response.

Event grouping	What events are logged	How to identify related log events	Additional information

		`SAMLAuthenticationError`
		`InResponseToField of Response doesn‘t correspond to the sent message`
		audit.security-events
			This error occurs when a user attempts to log on or refresh a session for which the authentication request has expired, for example, if this message is preceded by `AuthnRequest expired - ID`.

		`SAMLAuthenticationError`
		`No valid credential to evaluate the token`
		audit.security-events
			This error occurs when the certificate used to sign the SAML response isn't valid.

	Authentication success	`UserAuthenticationSuccess` audit.security-events	These entries are in addition to the entries made by the UAA. See the previous section *Security Events of the User Account and Authentication Service*. Authentication success includes: • User name • Principle (SCIM user ID) • Origin key • Zone ID
	SAML responses	`"msgNo":<index>,"msgId":"<message_id>",` audit.security-events	The system includes SAML responses in the audit log for web single sign-on and SAML bearer assertions.

7.1.5.3 Troubleshooting for SAP Authorization and Trust Management Service

This section contains troubleshooting information for the Cloud Foundry environment of SAP Business Technology Platform. It provides assistance with issues concerning platform security functions such as business user authentication, authentication of applications, authorization management, trust management, and other security functions.

If you need support or experience issues, please report an incident under component `BC-CP-CF-SEC-IAM`.

The Security Tab is Missing in the Subaccount [page 3564]

Access Is Denied or Forbidden [page 3565]

Identity Provider Could Not Process Authentication Request [page 3566]

Logon Screen Shows "SAP HANA XS Advanced" [page 3567]

Requested Route Does Not Exist [page 3568]

Subdomain Does Not Map to a Valid Identity Zone [page 3569]

No Client with Requested ID [page 3571]

Login Issues [page 3573]

Cannot Add Role Templates to Predefined Role Collections [page 3574]

409 Error Code on Deleting a Custom Identity Provider [page 3575]

Unexpected AuthnResponse : Existing authentication - <User> [page 3575]

AuthnRequest expired - ID: <RequestId> Destination: <IdPDestination> [page 3576]

InResponseToField of Response Doesn't correspond to the Sent Message [page 3577]

Response Issue Time is Either too Old or with Date in the Future. Sync IdP to Match Skew <skew> [page 3578]

Trust Establishment Issues [page 3578]

Token Retrieval Fails With Status Code 401 [page 3580]

Cockpit Displays HTTP Status 500 Error on Logon with Custom IdP User [page 3582]

IAS Application Reference isn't Created in Your IAS Tenant [page 3583]

User Base Doesn't Appear in Existing Neo Subaccounts [page 3583]

User from Corporate IdP Cannot Log on to Neo Subaccount [page 3584]

Share SAP Authorization and Trust Management Service Instances [page 3585]

Invalid "redirect_uri" [page 3585]

400 Error: Call to /oauth/token Was Not Successful [page 3586]

Missing Administrator Access to SAP Business Technology Platform [page 3587]

Unexpected User in Subaccount [page 3588]

There Was an Error When Authenticating Against the External Identity Provider [page 3589]

Common Authorization Issues in SAP Integration Suite [page 3590]

7.1.5.3.1 The Security Tab is Missing in the Subaccount

Symptom

If you navigate to your subaccount in the SAP BTP Cockpit, you can't see the Security tab.

Reason and Prerequisites

The problem here is that only the user that created the subaccount is granted the security administrator role. If other subaccount users should be able to see the security tab and make changes to the security configuration, the creator of the subaccount has to grant them the security administrator role.

The creator of the subaccount and the users that should be added must have at least one of the following member roles:

- They are members of the Cloud Foundry organization (if available) in the subaccount.
- They are members of any Cloud Foundry space that belongs to the organization.
- They are members of the global account that contains your subaccount.

Solution

The creator of the subaccount has to grant the security administrator role. To do this, follow these steps:

1. Open your SAP BTP Cockpit.
2. Navigate to your subaccount.
3. Choose the **Security** tab and choose **Administrators**.
4. Choose **Add Administrators**.
   A popup opens, where you can enter the user ID and assign roles.
5. Type in the ID (e-mail) of the user, that you want to add as a security administrator.
6. To assign roles, make sure that the **User & Role Administrator** checkbox is selected.
7. Choose **OK**.

After the role has been granted, the user needs to log out and log back in again to make sure the changes are applied to their subaccount user.

7.1.5.3.2 Access Is Denied or Forbidden

Symptom

After sending a request to a web application in the SAP BTP, Cloud Foundry runtime, you get a response containing a nested error element with the value `access_denied` and an `error_description` element.

The system responds with `Access is denied`.

The system might also respond with the single word `Forbidden`.

Output Code

```
<oauth>
  <error_description>Access is denied</error_description>
  <error>Access denied</error>
</oauth>
```

Reason and Prerequisites

A business user has tried to access the URL endpoint of a web application but is not authorized to do this. The authorizations (scopes) required for the URL endpoint must be assigned to this business user. Scopes (functional authorizations) allow the business user to perform operations provided by the web application. Attributes (instance-based authorizations) specify the data, for example, company code or cost center, which the business user is authorized to use. In this way, business users can perform the operations they are authorized for.

Solution

A role collection with the role that contains the required authorizations must be available. Assign this role collection to the business user.

If there are neither instance-based authorizations nor roles containing the required attribute values, create a role based on the corresponding role template. Maintain the attribute values of the new role to define the required data authorizations.

Note

- Role templates are predefined and deployed together with the web application. Role templates specify the scopes (functional operations) that the web application supports. You cannot change existing role templates or create new ones.
- After a role collection has been assigned, you might need to clear the cache and cookies and log on to the application again for the changes to take effect.

For more information about roles and role collections, see Building Roles and Role Collections for Applications.

7.1.5.3.3 Identity Provider Could Not Process Authentication Request

Symptom

After sending a request to a web application in the SAP BTP, Cloud Foundry runtime, the system responds as follows:

```
Error - Identity provider could not process the authentication request received.
Delete your browser cache and stored cookies, and restart your browser. If you
continue to experience issues, send an e-mail to sso@sap.com.
```

Reason and Prerequisites

SAP BTP, Cloud Foundry runtime uses the Identity Authentication service as its SAML 2.0 identity provider. The SAP Authorization and Trust Management service (XSUAA) forwards unauthenticated requests to the configured identity provider. This error occurs because the trust relationship between the SAP Authorization and Trust Management service and the identity provider hasn't been fully configured. The subaccount has not been registered in the SAML 2.0 identity provider.

Solution

Configure the missing part of the trust relationship in your identity provider by doing one of the following:

- Use the simplified trust configuration.
- Download the SAML metadata file from the SAP Authorization and Trust Management service, upload it to your SAML 2.0 identity provider, and complete the configuration by registering your subaccount. The following link provides instructions for configuring the Identity Authentication service as a trusted SAML 2.0 identity provider: Register SAP BTP Subaccount in the SAML 2.0 Identity Provider.

7.1.5.3.4 Logon Screen Shows "SAP HANA XS Advanced"

Symptom

After sending a request to a web application in the SAP BTP, Cloud Foundry runtime, you see a logon screen with the title SAP HANA XS Advanced.

Below the *Log On* field, the following domain is displayed: `xs2security.accounts400.ondemand.com`

> ⓘ Note
>
> **The expected system behavior is to return a logon screen with the title** `Welcome <IdP_tenant_name>!`.

Reason and Prerequisites

The request of the business user is not authenticated, so it needs to be redirected to the identity provider where the business user is defined. The trust of the identity provider is configured and assigned to the tenant to which the business user belongs. In this case, SAP BTP, Cloud Foundry runtime cannot correctly determine the tenant of the business user. As a result, it cannot find the correct identity provider and falls back to

https://authentication.sap.hana.ondemand.com

Solution

This error indicates a problem with the `TENANT_HOST_PATTERN` environment variable in the `manifest.yml` file of the web application. The tenant host pattern is a regular expression (regex) and is used to determine the tenant ID (which is identical to the subdomain) with which the request is associated. For more information, see Multitenancy. The system shows the behavior described earlier if the URL of the request does not match with the defined pattern.

The tenant host pattern should be defined as follows:

```
.
.
.
  env:
    XSAPPNAME: <application_name>
    TENANT_HOST_PATTERN: "^(.*)-<application-router-
hostname>.cfapps.<region>.hana.ondemand.com".
.
.
.
```

> ⓘ Note
>
> Separate the regular expression and the Cloud Foundry host name with a hyphen (-). The subdomain of the tenant is not a subdomain of the web application in a technical sense. The subdomain would be separated

from the Cloud Foundry host name with a dot (.). Since there is no automatic certificate creation process each time that a new tenant is onboarded, you must use a workaround.

The current workaround is to concatenate the subdomain of the tenant with the Cloud Foundry host name using the hyphen (-). All such concatenations share the same Cloud Foundry top level domain, together with the certificate of this top-level domain.

The following example shows the web application

`bulletinboard`

in the

`cfapps.eu10.hana.ondemand.com`

Cloud Foundry domain:

```
.
.
.
  env:
    XSAPPNAME: bulletinboard
    TENANT_HOST_PATTERN: "^(.*)-approuter.cfapps.eu10.hana.ondemand.com".
.
.
.
```

> ⓘ Note
>
> Recheck the definition of the `TENANT_HOST_PATTERN` environment variable in the `manifest.yml` file, and correct it, if necessary.**The route for the TENANT_HOST_PATTERN has to be lowercase.**

7.1.5.3.5 Requested Route Does Not Exist

Symptom

You send a request to a web application, which is running on SAP BTP, Cloud Foundry runtime and the browser responds with the message:

`404 Not Found: Requested route ('<cloudFoundryHostname>.<cloudFoundryDomain>') does not exist.`

Reason and Prerequisites

The URL of the request contains a route to an application for which no route mapping has been defined. This means that the Cloud Foundry router can't delegate the request to the respective web application and responds with an error message. Another reason might be that your application has stopped.

Solution

Create and Map the Missing Route to the Web Application

Create and map the missing route to the web application.

```
cf map-route <webApplication> <cloudFoundryDomain> --hostname
<cloudFoundryHostName>
```

> ### Example
>
> The web application's name is bulletinboard-123456 and the application was created in the European landscape eu10.hana.ondemand.com, the application should be reachable under the following domain: https://bulletinboard.eu10.hana.ondemand.com.
>
> 1. Use the following command to create this route:
>    ```
>    cf map-route bulletinboard-123456 cfapps.eu10.hana.ondemand.com --hostname
>    bulletinboard
>    ```
> 2. Check if the route was created:
>    ```
>    cf routes
>    ```

Check if Your Application is Still Running and Restart it

1. Log in to your Cloud Foundry account.
   ```
   cf routes
   ```
2. Choose your ORG and SPACE.
3. Check the status of your application.
   ```
   cf app <your application>
   ```
4. Restart your application.
   ```
   cf restart <your application>
   ```
5. Your application should now have the status running and you can call it through its route.

7.1.5.3.6 Subdomain Does Not Map to a Valid Identity Zone

Symptom

You send an unauthenticated request to a web application, which is running on SAP BTP, Cloud Foundry. The system responds with the message:

The subdomain does not map to a valid identity zone.

Reason and Prerequisites

The subdomain and/or the Cloud Foundry domain of the web application do not match the values provided in the URL of the web application.

Reason	Solution
The URL contains either an incorrect subdomain or an incorrect landscape domain (or both).	Check the the subdomain and landscape domain of your request
The subdomain belongs to a subaccount which has not yet been defined in Cloud Foundry.	Create the missing subaccount
The `TENANT_HOST_PATTERN` in the `manifest.yml` file of the web application is incorrect.	Correct the TENANT_HOST_PATTERN in the manifest.yml file of the web application
You are running your applications on the SAP BTP trial and your account has expired.	Extend your trial account

Solution

Check the Subdomain and Landscape Domain of Your Request

Check the subdomain and landscape domain of your request. You can find the subdomain in the SAP BTP cockpit in the **Overview** section of the subaccount that you are using for the request, under **Subaccount Details**.

You can find the landscape URL for your Cloud Foundry runtime here. For example, the subdomain for the European region is **eu10.hana.ondemand.com**.

Create the Missing Subaccount

Create the missing subaccount with the same subdomain in the Cloud Foundry runtime of the respective Cloud Foundry domain.

Correct the TENANT_HOST_PATTERN in the anifest.yml file of the web application

1. Check the `TENANT_HOST_PATTERN` environment variable in the `manifest.yml` file of the web application. The tenant host pattern is a regular expression (regex) and is used to determine the subdomain that the request is associated with. If the URL of the request does not match the pattern in the `manifest.yml`, you will receive an error message.
2. The tenant host pattern should usually be defined as follows:

```
env:
    XSAPPNAME: "<your-application-name>"
    TENANT_HOST_PATTERN: "^(.*)-<CloudFoundryHostName>.<landscape>".
```

The regular expression and the Cloud Foundry hostname are separated by a hyphen. The subdomain is not a subdomain of the web application in a technical sense. The subdomain would otherwise be separated from the Cloud Foundry hostname with a dot. That, however, would require the automation of the certificate creation process each time a new tenant is onboarded. The workaround for the time being is therefore to concatenate the subdomain of the tenant with the Cloud Foundry hostname with a hyphen. All of these concatenations share the same Cloud Foundry top level domain and also the certificate of that top level domain.

> ### Example
>
> The web application's name is **bulletinboard-123456** and the application was created in the European landscape **eu10.hana.ondemand.com**.

1. Change the `manifest.yml`.

```
 env:
                                    XSAPPNAME: "bulletinboard-123456"
                                    TENANT_HOST_PATTERN: "^(.*)-
approuter-123456.cfapps.eu10.hana.ondemand.com".
```

2. Recheck the definition of the `TENANT_HOST_PATTERN` environment variable in the `manifest.yml` file and correct it, if necessary.

> ### Note
>
> The route for the TENANT_HOST_PATTERN has to be lowercase.

Extend your trial account

Extend your trial account from within the SAP BTP cockpit.

1. Visit the trial page.
2. Choose **Enter Your Trial Account**.
3. Choose the calendar symbol in the upper right corner.
4. Choose **Extend Free Trial**.

Cloud Foundry trial accounts expire after 30 days. You can extend the trial period to a maximum of 90 days, after which your account is automatically deleted.

7.1.5.3.7 No Client with Requested ID

Symptom

You are not yet authenticated, and you send your first request to a web application, which is running on SAP BTP, Cloud Foundry environment. After you enter your credentials in the login screen, the system responds with the following error message:

```
No client with requested id: <OAuth2 Client-Id> - Authorization Request Error -
There was an error. The request for authorization was invalid.
```

The URL in the address-field of the browser contains the following URL-pattern **https://<subdomain>.authentication.<cloudFoundryDomain>/oauth/authorize?<urlParameters>**.

Reason and Prerequisites

Technical Reason

Even though you were authenticated successfully, **the SAP Authorization and Trust Management service (XSUAA) could not assign the authorizations** (scopes and attributes) of your business user to the web

application that you initially called. **The web application acts towards the SAP Authorization and Trust Management service as the OAuth2 client** and recognizes if the business user is sending an authorized request or not. If the request is not authorized, the web application first redirects the request to the authorized endpoint of the SAP Authorization and Trust Management service.

The SAP Authorization and Trust Management service acts towards the web application as the OAuth2 Authorization server and recognizes if the request to be authorized is authenticated or not. If the request is not authenticated, the service redirects the request to the configured identity provider (IdP) to initiate the authentication process.

The IdP sends a login screen to your browser so that you can authenticate yourself. The IdP then redirects your answer then back to the service with a SAML2 Bearer Assertion, which holds the authentication information of your business user. The SAP Authorization and Trust Management service can now retrieve the authorizations of your business user.

The service now wants to assign these authorizations to the OAuth2 client (the web application). The web application provides its OAuth2 client-id in the `client_id` URL parameter of the authorized endpoint. **The main point to consider here is that the SAP Authorization and Trust Management service accepts only the client-ids of those OAuth2 clients that it already knows**. That means that OAuth2 clients must register themselves to the service, before they call the authorized endpoint. This error occurs because **the web application was not registered as a valid OAuth2 client for the service**, before it called the authorized endpoint.

Reasons

Reason	Solution
You didn't create a service instance of the SAP Authorization and Trust Management service for your application.	Create a service instance for your application.
You're trying to access a SaaS application that you have unsubscribed from.	Subscribe to the SaaS application.

Solution

Create a service instance for your own application

1. Create a service instance in the space in which the web application is deployed.

   ```
   cf create-service xsuaa application my-uaa -c security/xs-security.json
   ```
2. Create an entry in the `manifest.yml` file, defining the service binding between the web application and the SAP Authorization and Trust Management service application plan service instance.

   ```
   services:
     -my-uaa
   ```
3. Redeploy the web application.

   ```
   cf push
   ```

Subscribe to the SaaS application

1. Log in to your Global Account.
2. Choose your subaccount.
3. Choose **Subscriptions**.

4. Choose the SaaS application that you want to subscribe to.
5. Choose **Subscribe**.
6. Retry accessing the SaaS application (it might be necessary to clear your cache).

7.1.5.3.8 Login Issues

Symptom

You log on to SAP BTP via the Cloud Foundry CLI or the browser and you get an HTTP 401 error or see messages on the screen that indicate that your login attempt has failed.

Reason and Prerequisites

Authentication in Cloud Foundry runs through the UAA service which tunnels the credentials supplied (such as username and password) towards a trusted Open ID Connect Identity Provider (IdP). SAP's default IdP is the SAP ID service, which is available at https://accounts.sap.com for SAP ID Service.

To narrow down the origin of the issue, you need to check the individual components involved in the login process.

Solution

Identify whether a second factor (a one-time token) is mandatory for this user/landscape

1. Open the UAA service in the browser directly, depending on your region: For example, for the European region, go to https://uaa.cf.eu10.hana.ondemand.com/ (For an overview of all Cloud Foundry landscape URLs, see Regions).
   If you are redirected to https://accounts.sap.com asking for two-factor authentication and a passcode, a second factor in the form of a one-time TOTP token is mandatory.
2. See Activate TOTP Two-Factor Authentication for information about how to configure a device to generate TOTP tokens; for example, with an authenticator app.
3. Log on to your Cloud Foundry runtime using two-factor authentication.
   1. Type `cf login`.
   2. Enter your e-mail address.
   3. Enter your password in the following format: <your global password><one-time-token>

7.1.5.3.9 Cannot Add Role Templates to Predefined Role Collections

Symptom

You want to update your SAP Authorization and Trust Management service (XSUAA) instance and add role templates to a previously defined role collection in the *xs-security.json*. The service instance update fails with the error message:

```
Cannot add or remove role-template-references from role collection if already in use (app subscribed or role collection assigned to user). Please create a new role-collection with a different name instead.
```

Reason and Prerequisites

You tried to add a role template to an existing role collection that is already in use. If your application was either subscribed (for a multi-tenant enabled application) or the role collection in question was already assigned to a user, an update of the role collection is not possible anymore. The security reason for this is that role collections might have already been audited by subscribers and/or the security administrator assigned these role collections to users based on the initial state of the role collections. Changes to the role collections at a later point in time would render these audits invalid.

In general, you should never modify any authorization object (role template, role) that is already in use in a way that it grants additional authorization without explicit approval of the user and role administrator.

Solution

Create a new role collection with a different name

You can add an additional role collection with a different name to your *xs-security.json*. Administrators can then audit the new role collection and migrate the users to the updated role collection. To do this, follow these steps:

1. Add a new role collection with a different name to the *xs-security.json*.
2. Add/modify the role templates within that role collection according to your needs.
3. Update the SAP Authorization and Trust Management service instance. See Update Service Instances Using the SAP BTP Cockpit or Cloud Foundry Command Line Interface.
4. Assign the new role collection to the application users.
5. (Optional) When all users have been migrated to the new role collection, remove the old role collection from the *xs-security.json*.
6. (Optional) Update the SAP Authorization and Trust Management service instance.

For more information about the creation of role collections and the *xs-security.json*, see Create Role Collections (with Predefined Roles) and Application Security Descriptor Configuration Syntax.

7.1.5.3.10 409 Error Code on Deleting a Custom Identity Provider

Symptom

- On deleting a custom identity provider (origin: *sap.custom*) in the SAP BTP cockpit, you get an error.
- If you open the developer console (F12) and try again, you will see a more detailed error message.

Reason and Prerequisites

- This issue occurs when there are applications still present in SAP Cloud Identity Services - Identity Authentication. For example, this can happen if you have created an identity service instance.

Solution

1. Check the Identity Authentication tenant for which you want to delete the trust configuration in the SAP BTP cockpit.
2. You need to delete all the applications of the error message.

Note: Delete only those applications that you are sure are not needed anymore.

See SAP Note 3148626 for more information.

7.1.5.3.11 Unexpected AuthnResponse : Existing authentication - <User>

Symptom

After logging on to an application with SAML, the user presses the back button of the browser. This leads to the *Unexpected AuthnResponse : Existing authentication - <User> error*.

Reason and Prerequisites

Using the back button leads to a second authentication request to the identity provider, with the same ID used for the initial logon. As a result, the identity provider issues a second authentication response for the same ID, which is then rejected.

Solution

The error message displayed is the correct outcome of this action. If you want to trigger a logout by pressing the back button, this needs to be handled on the application side.

7.1.5.3.12 AuthnRequest expired - ID: <RequestId> Destination: <IdPDestination>

Symptom

When a SAML session is dropped, the audit log shows AuthnRequest expired - ID: <RequestId> Destination: <IdPDestination>. This particular error message can be found only in the audit log.

Reason and Prerequisites

This error occurs when the identity provider takes more than 15 minutes to issue the authentication response or when the user waits with the logon action for more than 15 minutes on the identity provider side. After this time interval, the session information is dropped for performance and security reasons; and thus, the request and response do not match anymore.

Solution

Don't delay the logon action

Ensure that the user login is within the timeframe of 15 minutes.

Check the identity provider

The identity provider may be out of sync; or if it consistently takes a long time to issue the authentication response, reduce this time and check.

7.1.5.3.13 InResponseToField of Response Doesn‘t correspond to the Sent Message

Symptom

When the user tries to log on with SAML or by refreshing the session, they see the *InResponseToField of Response doesn‘t correspond to the sent message* error.

Reason and Prerequisites

This error occurs when handling an authentication response without a known corresponding request; for example, as the consequence of a session timeout. The timeout is a safeguard for performance and security reasons.

Solution

Don't delay the logon action

Ensure that the user login is within the timeframe of 15 minutes.

Check the identity provider

The identity provider may be out of sync; or if it consistently takes a long time to issue the authentication response, reduce this time and check.

Check the audit log

Check the audit log for the *AuthnRequest expired - ID: <RequestId> Destination: <IdPDestination>* error. If you find this error message, refer to AuthnRequest expired - ID: <RequestId> Destination: <IdPDestination> [page 3576].

For more information about the audit log, refer to Auditing and Logging Information for SAP Authorization and Trust Management Service - SAP Help Portal.

7.1.5.3.14 Response Issue Time is Either too Old or with Date in the Future. Sync IdP to Match Skew <skew>

Symptom

SAML logon doesn't work and the *Response issue time is either too old or with date in the future. Sync IdP to match skew <skew>* error message is displayed.

Reason and Prerequisites

This error occurs when the time skew between the SAP Authorization and Trust Management service and the identity provider is larger than 60 seconds in either direction, or when after being issued, the authentication response takes more than 60 seconds to reach the SAP Authorization and Trust Management service.

Solution

Check the time skew between the identity provider and the SAP Authorization and Trust Management service, and then sync the clock of the identity provider.

7.1.5.3.15 Trust Establishment Issues

Symptom

You could have trust establishment issues if you encounter one of the following symptoms:

- In the BTP Cloud Cockpit:
  - After clicking **Establish Trust**, an Identity Authentication (IAS) tenant is not displayed.
  - Failure after the IAS tenant is selected and confirmed by clicking **Establish Trust**.
  - **Establish Trust** button is not accessible (displayed in the inactive state).
  - Trust deletion fails.
- In the REST API:
  - Usage of the REST API (`/sap/rest/identity-providers`) returns status code `400`, `401`, or `404`.
  - Trust deletion fails.

Reason and Prerequisites

To establish trust in an automated manner, the customer ID of the IAS tenant has to match the customer ID of the global account.

Consequently, the issues in trust establishment may be because of the following:

- The customer ID maintained in the IAS tenant does not match what is in the global account.
- Trust has already been established (IdP trust with origin `sap.custom` exists.).

Solution

Check that the IAS tenant is displayed

If an IAS tenant is not displayed, verify the following:

- The customer ID maintained in the IAS tenant matches what is in the customer ID of the global account.
  - IAS admin UI: *Applications & Resources* > *Tenant Settings* > *Customer ID*
  - Global account: Verify the value in the control center.

In case you require a new IAS tenant, see the corresponding documentation. For more information, see this blog article.

Check that the "Establish Trust" button is accessible

If the Establish Trust button is grayed out, there is probably already an Identity Provider (IdP) trust with origin, sap.custom. To change the trust, you can delete the existing configuration and create a new one. Note that the trust deletion in the Cloud Cockpit implies the deletion of all existing shadow users (for this IdP).

If the Establish Trust button is not visible at all, it could be an issue with the Identity Authentication service (IAS) registry or the certificates. This is usually a temporary issue and you should retry accessing it later.

Check the usage of the REST API

Depending on the status code, verify the following points:

- `400`: Check the section, *Specific IAS tenant is not shown*. Further, verify that the `/sap/rest/identity-providers/ias` REST API returns the used IAS tenant.
- `401`: REST API requires a token of the *apiaccess* service plan from your subaccount.
- `404`: For newly created IAS tenants, it can take up to 2 hours until the DNS entry is propagated to all the DNS caches. Until that is done, you can get an error when trying to call the new IAS tenant. In such a case, wait and retry after 2 hours.

Check if trust has already been established

- Verify that you are deleting a custom IdP (the default IdP cannot be deleted).
- Ensure that if you use the identity broker (or a SaaS application that uses it), it does not contain any application references or other applications.

7.1.5.3.16 Token Retrieval Fails With Status Code 401

Symptom

You want to retrieve an access token from the Authorization and Trust Management service, but the request fails because of one of the following:

- Client authentication failed
- Bad credentials
- Client with requested ID does not exist
- The origin provided in *login_hint* does not match an active Identity Provider (IdP) that supports resource-owner password grant flow.

Reason and Prerequisites

The client or user authentication was not successful. This could happen if the client does not exist or if the client or user credentials are invalid.

- In the case of client authentication failing, during the authentication, the caller provides the *client_id* and the *client_secret*. If these are not correct, an error occurs. This is valid for both technical (*client_credentials*) and user authentication (e.g. *authorization_code*).
- Bad credentials could be the case in user flows and indicates that the user credentials (username/password) are incorrect.
- If a client with the requested ID does not exist, it signifies that the source of the *clientid* value should be checked.

You have to determine whether the issue is related to client or user authentication.

- For user authentication, verify whether token retrieval works for the `client_credentials` grant type.
- If not, the issue is related to client authentication.

Solution

User authentication

- Verify that the user and password combination is correct.
  - Open a browser and log on to the IdP (e.g. https://accounts.sap.com).
  - If the login fails, you need to use another user/password combination or you could change your password.
- In case of a Universal ID user:
  - Check Log On to the Cloud Foundry Environment Using the Cloud Foundry Command Line Interface and especially SAP Note 3085908. You can define a separate password for the SAP ID service user

that is linked to the Universal ID user, which you need to use for all the logins that involve the SAP ID service and where you have to provide a user name and password and cannot run the (redirected) login flow via the web browser.

Client authentication

Client authentication

Secrets

- Verify that the used secret is valid and that the corresponding binding/service key still exists.
- Depending on the credential type of the used binding/service key, do one of the following:
  - `binding-secret`: Ensure that the service instance of the client allows `binding-secret` in its security descriptor (within `credential-types`).
  - `instance-secret`:
    If you are using `instance-secret` in its security descriptor (within `credential-types`), we recommend that you migrate to `binding-secret`. For more information, see Migrate from Instance Secrets to Binding Secrets [page 2638].

    > **Note**
    >
    > Starting January, 19, 2026, the `instance-secret` credential type isn't supported anymore for new bindings and service keys.
- If the application to which the service instance belongs was subscribed to from another subaccount, ensure that the application or service is onboarded correctly via the SAP SaaS Provisioning service.
- If the secret contains the + sign:
  - Ensure that the secret is passed via POST body (and not in the `Authorization` header).
  - If it's not possible to pass the secret via POST body, ensure that the username and password are correctly URL encoded before being passed via the header. Also, you need to set the `X-CF-ENCODED-CREDENTIALS` custom header to `true` to indicate that the Authorization and Trust Management service supports URL-encoded credentials.
- If you've recently updated to Spring version 5.6 or newer and use a Spring client for the client credential flow:
  - Set the `X-CF-ENCODED-CREDENTIALS` custom header to `true.`
  - Customize the Spring client such that it does not URL-encode credentials.

X.509

- Verify that the certificate has been passed for authentication and that it is the correct one.
- Ensure that the service instance of the client allows `x509` in its security descriptor (within `credential-types`).
- If the application to which the service instance belongs was subscribed to from another subaccount, ensure that the application or service is onboarded correctly via the SAP SaaS Provisioning service.
- For issues with the SSL handshake, create an incident in the SAP Support Portal.

No IdP that supports resource-owner password grant flow

- Resource-owner password grant flow is only supported with OpenID Connect (OIDC) IdPs, not SAML.
- Make sure to use the `sap.default` or `sap.custom` origin key.
- Origin keys can be checked in the Cockpit, under **Trust Configuration**.

7.1.5.3.17 Cockpit Displays HTTP Status 500 Error on Logon with Custom IdP User

Symptom

Reason and Prerequisites

After successfully logging on with a custom identity provider (IdP) user, the cockpit displays an error page: *HTTP Status 500 – Internal Server Error*.

The following are the known reasons for this error:

- The mail claim in the OpenID Connect (OIDC) token from the Identity Authentication (IAS) tenant, is empty. This could happen in the following cases.
  - The corporate identity provider (for example, MS Azure AD) is connected to the IAS and the user in the corporate IdP does not have an email address assigned.
  - The Identity Authentication user store is enabled, and the corporate identity provider is configured to send the email address of its user in another SAML attribute or OIDC claim than what is defined in the default attributes of the application in the Identity Authentication tenant.
  - The Identity Authentication user store is disabled. This requires the corporate identity provider to send the user's email address in a SAML attribute or OIDC claim mail.
- The *mail*, *first_name*, or *last_name* claim in the OIDC token from IAS contains multiple values. This could happen in the following cases.
  - The IAS tenant or application is configured to use a corporate IdP with identity federation enabled and the user also exists in the IAS user store, or the corporate IdP sends multiple values in the first place.
  - In the case of the *mail* claim, the corporate IdPs OIDC token or SAML assertion does not send the email address in all lowercase. IAS always converts the email address to lowercase for local users.
- The issuer in the connected IAS tenant was changed after the trust to that tenant was established.

Solution

Configure the corporate identity provider

As typically, either some attribute is missing or has multiple values instead of a single one, configure the corporate identity provider to correctly send the mail claim or required attributes.

- If a platform user has problems when logging on with the a custom identity provider, refer to the IAS Troubleshooting Logs (OIDC) for that affected user.
- In particular, check if there is exactly one value (and the correct value) for the specified attributes.

Check the issuer in the IAS

The issuer in the IAS must not be changed after the trust has been established. This is also documented as a restriction here.

7.1.5.3.18 IAS Application Reference isn't Created in Your IAS Tenant

Symptom

When trying to log on to the global account with a custom identity provider (IdP) user, the Identity Authentication (IAS) tenant responds with the following error message: *OpenID provider cannot process the request because the configuration is incorrect. Please contact your system administrator.*

Reason and Prerequisites

Most likely, you configured a platform IdP earlier and deleted the resulting IAS application reference (SAP Business Technology Platform, *btp-cockpit*) from the IAS tenant.

Solution

Delete and recreate the trust to the IAS tenant

In your global account, delete and recreate the trust to the IAS tenant.

7.1.5.3.19 User Base Doesn't Appear in Existing Neo Subaccounts

Symptom

When trying to add new members, Neo member management doesn't have the custom identity provider configured in the global account, as the user base.

Reason and Prerequisites

For subaccounts in the Neo environment, the identity provider will be offered in the value help only if a user in that identity provider has created at least one Neo subaccount in the corresponding global account and Neo region. For more information about this, see here.

Solution

Create at least one new Neo subaccount

A user from the configured custom identity provider needs to create at least one new Neo subaccount in the Neo region where the user base selection is missing.

7.1.5.3.20 User from Corporate IdP Cannot Log on to Neo Subaccount

Symptom

When a user with a corporate identity provider (IdP) logs on to the cockpit and tries to navigate to a Neo subaccount, the Identity Authentication (IAS) tenant shows the logon screen with the *E-Mail or User Name* and *Password* fields. However, with a properly connected and configured corporate IdP, the IAS logon screen should show only the *E-Mail or User Name* field.

Reason and Prerequisites

For all the Neo datacenters where you have subaccounts, you might have to configure a separate SAML application (for example, *SAP Cloud Platform hhw9g3jrxa*) in the IAS tenant. This behavior is also documented here.

- A custom IdP with a corporate identity provider (e.g. MS Azure AD) is configured in the global account.
- The affected users from the corporate IdP can log on to the cockpit (e.g. https://cockpit.eu10.hana.ondemand.com/cockpit/?idp=ar9ibaxhm.accounts.ondemand.com).
- The affected users are configured as members of the Neo subaccount, with the correct user base.

Solution

Find the correct SAML application in the configured IAS tenant and configure conditional authentication accordingly. Each Neo region where custom IdP is configured has a separate SAML application in IAS that needs to be configured.

7.1.5.3.21 Share SAP Authorization and Trust Management Service Instances

Symptom

You want to share SAP Authorization and Trust Management service instances among different Cloud Foundry spaces or environments.

Reason and Prerequisites

- You use an SAP Authorization and Trust Management service instance of the *application* type.
- You want to use the service instance in multiple spaces or environments, without recreating it multiple times.

Solution

Use the SAP BTP command line interface (BTP CLI) to share the *application* plan.

For more information, see Sharing and Unsharing Service Instances.

7.1.5.3.22 Invalid "redirect_uri"

Symptom

After logging on, you are redirected to a *Where-To?* page with one of the following error messages:

- The *redirect_uri* is invalid
- The *redirect_uri* has an invalid domain
- Invalid redirect: *{URI}* does not match one of the registered values
- Client registration is missing *redirect_uri*
- Client registration contains *invalid redirect_uri*

Reason and Prerequisites

The `redirect_uris` configuration value for the used service instance has not been properly configured.

Solution

1. Identify the affected application. The affected application is the application on which you want to be redirected after the logon process.
2. Identify the affected service instance of the *xsuaa* type.
   - Cloud Cockpit: Find out which service instance of the *xsuaa* type was bound to the application.
   - CF CLI: Check the environment of the application, and then find *xsuaa* to retrieve the name of the service instance: *cf env <application-name>*
3. Identify the *xsappname* of the service instance (which is mandatory for *xs-security.json*).
   - Existing *xs-security.json*: You can find the *xsappname* of the *xs-security.json* file that was used to create the service instance.
   - Cloud Cockpit: Create a service key and retrieve the *xsappname* from it; remove the section after the exclamation mark - for example, *myapp!b123 -> xsappname: myapp*. For more information, see SAP Note 3319310.
   - CF CLI: Check the environment of the application (see step 2), retrieve the *xsappname* from the corresponding service, and then remove the section after the exclamation mark.
4. Prepare the security descriptor (*xs-security.json*).
   - For more information, see Application Security Descriptor Configuration Syntax.
5. Update the *redirect_uris* attribute of the corresponding service instance (within *oauth2-configuration*).
   - Cloud Cockpit: Click **Update** in the service, and then provide the *xs-security.json* file, as required.
   - CF CLI: Use *cf update-service* to update the service instance and provide the required *xs-security.json* file: *cf update-service <instance-name> -c <path-to-xs-security.json>*

Note: It is important to provide the complete *xs-security.json* file during the update as partial updates are not supported.

7.1.5.3.23 400 Error: Call to /oauth/token Was Not Successful

Symptom

You tried to log on to your application and got an error related to the token size exceeding some limits (for example, the HTTP header size).

Reason and Prerequisites

By default, access and refresh tokens include SAML groups and role collections. If you have a lot of SAML groups or role collections, the token size increases, and this can cause the JWT to exceed the size limit of 16 K. This results in a 400 error and the user not being able to log in.

Solution

Keep the JWT size to a minimum

Application security is maintained in the application security descriptor file (`xs-security.json`). In the syntax of the application security descriptor file, do the following to keep the JWT size to a minimum.

- Set the value of the `system-attributes` parameter to an empty array, like this: `"system-attributes": []`
- Configure only the required groups in the Identity Provider (IdP).

For more information regarding the `xs-security.json` file, see Application Security Descriptor Configuration Syntax.

7.1.5.3.24 Missing Administrator Access to SAP Business Technology Platform

You have issues with managing subaccounts and editing entitlements in SAP BTP. Add a user with global account administrator permissions. This administrator is then authorized to perform these tasks.

Symptom

You want to create or delete a subaccount in SAP BTP, but your user isn't authorized to do so.

You want to edit the entity assignments in the SAP BTP entitlements, but the *Edit* button is not available.

Reason and Prerequisites

- You don't have a user who has administrator persmissions for the global account in the SAP BTP cockpit.

Solution

Follow the steps described in the link below.

For more information, see SAP Note 3148626.

7.1.5.3.25 Unexpected User in Subaccount

Symptom

As a subaccount administrator, you find a user you didn't add to your subaccount and are concerned how it got there.

Reason and Prerequisites

- Another subaccount administrator added the user without your knowledge.
- The user was added by a background job, which syncs the Cloud Foundry users from orgs or spaces in that subaccount.
- The user was unexpectedly added by a provisioning job.

Solution

Check the audit log of the SAP Authorization and Trust Management service. See Auditing and Logging Information for SAP Authorization and Trust Management Service [page 3555].

There, you can see who created the user and coordinate your processes.

- If the creator was the `CLOUD_FOUNDRY_SNYC` user, the user was created by the background job for synchronizing Cloud Foundry users. If you don't want SAP BTP to create these users, you can disable this job for your subaccount. See Switch Off Automatic Creation of Shadow Users [page 2600].
- If the creator was a provisioning job, check with the solution used for provisioning and check for errors in configuration. For more information, see Provision Users to SAP BTP Accounts [page 2609].

Related Information

Working with Users [page 2603]
About User Management in the Cloud Foundry Environment [page 2787]

Add Members to Your Subaccount [page 2513]
Add Members to Your Global Account [page 2467]

7.1.5.3.26 There Was an Error When Authenticating Against the External Identity Provider

Symptom

You have configured a custom identity provider for applications in the SAP BTP cockpit, such as the SAP Cloud Identity Services.

When you open an SAP BTP application, such as a custom developed application or a subscribed application like SAP Business Application Studio, you receive the following error:

```
There was an error when authenticating against the external identity provider: The user account must be pre-created. Please contact your system administrator.
```

Reason and Prerequisites

The user from the custom identy provider for applications doesn't exist in the list of users of the SAP BTP cockpit.

When a user authenticates at an application in your subaccount using any identity provider, the User Account and Authentication always stores user-related data provided by the identity provider in the form of shadow users. It uses details of the shadow users to issue tokens that refer to a specific user.

By default, the User Account and Authentication allows any user of any connected identity provider to authenticate to applications in the subaccount. When there's no corresponding shadow user, it automatically creates one based on the information received from the identity provider.

However if *Create Shadow Users on User Logon* is turned off, shadow users aren't created, which causes the issue.

Solution

To resolve the issue, use one of the following options:

- Enable *Create Shadow Users on User Logon* for the custom identity provider:
  1. In the SAP BTP cockpit, go to your subaccount (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Trust Configuration*.
  2. Select your custom identity provider for applications.

3. Choose *Edit*.
4. Activate *Create Shadow Users on User Logon*.
5. Choose *Save*.

- Manually create a user for yor the respective identity provider (see Create Users [page 2608]).
  1. In the SAP BTP cockpit, go to your subaccount (see Navigate in the Cockpit [page 2461]) and choose *Security* > *Users*.
  2. Create a new user for the respective identity provider by using *Create*.
- Use the SAP Cloud Identity Services - Identity Provisioning to import users from a source system to the respective subaccount. Follow the instructions in SAP BTP XS Advanced UAA (Cloud Foundry).

> **Note**
>
> After creating the users in the subaccount, add the relevant role collection for users. The users must have the authorization to access the application (see Assign Users to Role Collections [page 2617]).

Related Information

Create Users [page 2608]
Switch Off Automatic Creation of Shadow Users [page 2600]
SAP BTP XS Advanced UAA (Cloud Foundry)

7.1.5.3.27 Common Authorization Issues in SAP Integration Suite

Find help about authorization issues with SAP Integration Suite.

I assigned the correct role collections to my user but still can't access SAP Integration Suite

After subscribing to the application and assigning roles, clear your browser cookies and cache. Then, sign out of the SAP BTP cockpit and SAP Integration Suite, and sign in again.

If the issue persists, raise a ticket under the component you're trying to access. See SAP Note 2904202.

I subscribed to SAP Integration Suite, but I want to know more about the role collections I need to work with the different capabilities

The following table lists the most common capabilities of SAP Integration Suite, and the role collections needed for them.

Role Collection	Capability
Integration_Provisioner	Gain initial access to SAP Integration Suite
PI_Read_Only *PI_Business_Expert* *PI_Integration_Developer* *PI_Administrator*	Cloud Integration
APIManagement.SelfService.Administrator *AuthGroup.SelfService.Admin*	API Management
PI_Integration_Expert *PI_Business_Expert* *PI_Administrator* *PI_Read_Only*	Trading Partner Management
iadv-content-developer *iadv-content-administrator* *iadv-content-read*	Integration Advisor

You can find more information and learn about other capabilities in the documentation for SAP Integration Suite:

- Subscribing and Configuring Initial Access to Integration Suite
- Configuring User Access

For more information about role templates, such as *AuthGroup_BusinessExpert* or *AuthGroup_Administrator*, see Identity and Access Management.

For more information on predefined technical roles, such as *ESBmessaging.send*, see the SAP Integration Suite documentation.

7.1.6 Security Considerations for the SAP Authorization and Trust Management Service

Decisions you make when using or administrating the SAP Authorization and Trust Management service (XSUAA) can have an impact on the security of your applications. The information provided is meant to help you decide.

Related Information

Configuration Options for the SAP Authorization and Trust Management Service [page 3596]

7.1.6.1 Implications of Using IFrames

By default, applications of the subaccount, including login pages of the SAP Authorization and Trust Management service can't be framed by other applications for security reasons.

As the security administrator of a subaccount, you can allow the embedding applications of the subaccount in an iFrame.

To configure the trusted domains, use one of the following methods:

- Enter trusted domains in the SAP BTP cockpit.
  For more information, see Configure Trusted Domains for Multi-Environment Subaccounts [page 2655].
- Access the Security Settings API.

  ```
  "iframeDomains": "https://store.example.com"
  ```

  For more information, see Security Settings API on SAP Business Accelerator Hub.

> → Remember
>
> To use iFrames with SAP Authorization and Trust Management service, configure the embedding of the login page of your identity provider. Not all identity providers support framing. This function only works if the identity provider supports framing and is configured, too.
>
> To use iFrames with other applications of the subaccount, which support this feature, configure the domains used by these applications.
>
> To configure iFrames for SAP Cloud Identity Services, see Configure Trusted Domains in the documentation of SAP Cloud Identity Services.

When configured, the SAP Authorization and Trust Management service sends a content security policy header allowing framing of the application for the domains you specified.

For more information about content security policies, see the World Wide Web Consortium (W3C) recommendation Content Security Policy .

This configuration has several security implications like:

- Clickjacking attacks on the application as the URL of the application isn't visible.
- JavaScript-based cross-site scripting vulnerabilities of older browser versions.
- Third-party cookies are required when using different domains for authentication with the SAP Authorization and Trust Management service, the identity provider, and the application.
  See also Third-Party Cookies and SAP Authorization and Trust Management Service [page 3598].

Ensure that no attacker can add malicious web pages or Javascript to any of the hosts allowed to frame the applications. This recommendation also includes hosts that act as reverse proxies, where an attacker can put their content on a different host behind the reverse proxy. Ensure that everything exposed by those framing hosts is safe.

7.1.6.2 Listing Allowed Redirect URIs

When developing your application, provide the list of the redirect URIs that the application needs when redirecting, for example during login or logout. Enter the list in the `redirect-uris` property of the application security descriptor (`xs-security.json`).

At runtime, the SAP Authorization and Trust Management service checks whether the redirect URI is listed in the property and rejects any other URIs. If your redirect isn't on this list, including wildcards, the service won't redirect users there.

The following is an example of a configuration of the `redirect-uris` property in an `xs-security.json`.

```
"oauth2-configuration": {
  …
  "redirect-uris": [
    "https://myapplication.cfapps.eu10-004.hana.ondemand.com",
    "https://myapplication.cfapps.eu10-004.hana.ondemand.com/my/content",
    "https://myapplication.mydomain.com",
    "https://myapplication.mydomain.com/my/content"
    ],
  …
}
```

> → Recommendation
>
> Set the *redirect-uris* property to restrict access as much as possible.

We support explicit wildcards, namely domain relaxing and arbitrary paths. For example: `"https://*.mydomain.com/callback/**"`

> ⚠ Caution
>
> If you use wildcards, we recommend that you make your URIs as specific as possible. By using wildcards, you open up the redirect for multiple websites. Wildcards increase the risk of redirecting to malicious websites.

> ⓘ Note
>
> In cloud landscapes, only `localhost` is always allowed by default as a redirect URI.

Related Information

Application Security Descriptor Configuration Syntax [page 592]
Configure Redirect URLs for Browser Logout [page 582]

7.1.6.3 Rotating Secrets

Starting January 19, 2026, all bindings created by the SAP Authorization and Trust Management service use binding secret or x509. However, there may be instance secrets created before. Take care that you remove the old instance secrets because they can't be rotated (see SAP BTP Security Recommendations).

We recommend to use the credential types `binding-secret` and/or `x509` so that you can rotate the secret of a binding without affecting the other bindings of the service instance. We also recommend that you rotate secrets regularly.

For more information, see Managing Secrets of the SAP Authorization and Trust Management Service [page 2634].

```
"oauth2-configuration": {
  …
  "credential-types": ["binding-secret"],
  …
}
```

For more information about the application security descriptor, see Application Security Descriptor Configuration Syntax [page 592].

7.1.6.4 Setting Token Policy

By default, SAP Authorization and Trust Management service sets the validity of tokens as shown in the following table.

Default Validity of Tokens

Token Type	Validity
Access tokens	Default: 43200 seconds (12 hours)
Refresh tokens	Default: 604800 seconds (7 days)

On request, SAP Authorization and Trust Management service issues access and refresh tokens.

With a valid access token, you can access a protected resource. Once an access token expires, you can get new access tokens with a refresh token. Once the refresh token expires, you must reauthenticate and request new access and refresh tokens.

See also Authorization for information on how tokens affect authorizations.

> → Recommendation
>
> Relaxing the token policy means that users reauthenticate less. However, increasing the token validity also means that if a malicious user manages to steal a token, that malicious user has access until the token expires. Keep token validity as short as possible, but not less than 30 minutes.

To change token validity, use one of the following methods:

- To target a specific instance of the service, change the values of the *token-validity* or *refresh-token-validity* parameters in the application security descriptor (`xs-security.json`).

  ```
  "oauth2-configuration": {
    …
    "token-validity": 1800,
    "refresh-token-validity": 43200,
    …
  }
  ```

  > ### Note
  >
  > This change applies to applications, which consume or subscribe to this instance. These values override the values set for the subaccount.

  For more information about the application security descriptor, see Application Security Descriptor Configuration Syntax [page 592].

- To target the subaccount in general, access the Security Settings API.

  ```
  "tokenPolicySettings": {
    …
    "accessTokenValidity": 1800,
    "refreshTokenValidity": 43200,
    …
  }
  ```

  > ### Note
  >
  > This change applies to all service instances in the subaccount that haven't set a specific value in the application security descriptor (`xs-security.json`).

  For more information, see Security Settings API on SAP Business Accelerator Hub.

7.1.6.5 Access to REST APIs

To get access to the APIs of the SAP Authorization and Trust Management service, use the SAP BTP command line interface (BTP CLI) to create API credentials. You can create read-only or read-write API credentials that apply to the global account, directory, or subaccount.

Related Information

Access to REST APIs [page 3595]

7.1.6.6 Training Business Users How to Handle Session Timeouts

Administrators should make business users aware of a possible security risk after single logoff. A business user might get a message saying that this user is being logged off from the application although the identity provider session was not terminated.

The reason for this message comes from a timeout mismatch between the application and the SAP Authorization and Trust Management service. The timeout of the SAP Authorization and Trust Management service is currently 30 minutes. After the timeout, the SAP Authorization and Trust Management service does not forward the logoff request to the identity provider. This situation incurs a security risk.

> → Recommendation
>
> To remedy this security risk, advise the business users to access the application URL again, log on to the application, and log off right away, or log off directly from the identity provider.

7.1.7 Configuration Options for the SAP Authorization and Trust Management Service

The following configuration options enable you to manipulate the operation of the SAP Authorization and Trust Management service (XSUAA). Set these options in the application security descriptor (`xs-security.json`) at design time for your application.

Related Information

Security Considerations for the SAP Authorization and Trust Management Service [page 3591]

7.1.7.1 Disabling System Attributes

Access tokens can become overloaded with too many SAML group or role collection attributes. By default, the SAP Authorization and Trust Management service includes all system attributes in the tokens the service issues.

If the number of attributes cause the token size to exceed 16k, you get HTTP error code 400. To avoid this problem, set the *system-attributes* parameter in the application security descriptor (`xs-security.json`).

```
"oauth2-configuration": {
  …
  "system-attributes ": [],
  …
}
```

For more information about the application security descriptor, see Application Security Descriptor Configuration Syntax [page 592].

See also 400 Error: Call to /oauth/token Was Not Successful in *Guided Answers*.

7.1.7.2 Preconfigured Role Collections

In the application security descriptor (`xs-security.json`) of your application, you define the authorizations needed to access your application. You define these authorizations as scopes within roles and role templates. From these entities, the administrator can build role collections and assign the role collections to users.

We recommend that you also define role collections for administrators to consume with your application. Delivering role collections with your application has the following advantages:

- Time savings for the administrator.
  The administrator of the subaccount can consume your application more quickly.
- Offer examples of well-defined role collections.
  The administrator can learn from your example, the security model of your application and design his or her own role collections based upon that model.

For more information about authorizations, see Authorization Entities [page 3550].

```
"role-collections": [
  {
    "name": "Employee",
    "description": "Employee roles",
    "role-template-references": [
      "$XSAPPNAME.Employee"
      ]
  }
]
```

For more information about the application security descriptor, see Application Security Descriptor Configuration Syntax [page 592].

7.1.7.3 Asynchronous Processing for Multi-Tenant Applications

After your application has more than 50 subscriptions, the time required to update instances of SAP Authorization and Trust Management service bound to your application increases. This delay can run into the timeouts set by the Cloud Controller of Cloud Foundry and SAP Service Manager.

To avoid timeout, enable asynchronous processing in your application security descriptor (`xs-security.json`). To enable asynchronous processing, set the *xsenableasyncservice* parameter to **`true`**.

```
"xsenableasyncservice":"true"
```

For more information about the application security descriptor, see Application Security Descriptor Configuration Syntax [page 592].

7.1.8 Third-Party Cookies and SAP Authorization and Trust Management Service

Browser manufacturers are phasing out support for third-party cookies. This change can prevent embedded SAP BTP applications and their logon screens from functioning.

To prevent your logon scenarios from failing, make sure that all multi-environment subaccounts, which contain these embedded applications, are configured to use a tenant of SAP Cloud Identity Services as the trusted identity provider.

> ⚠ Caution
>
> Migrate your trust before the browser vendors phase out support.
>
> Trusting your SAP Cloud Identity Services tenant enables your applications to integrate with the overall solution provided by SAP.
>
> For more information about the overall solution, see SAP BTP and Third-Party Cookies Deprecation and SAP Note 3409306.

To migrate an existing trust configuration, see Migration from SAML Trust to OpenID Connect Trust with Identity Authentication.

If your framing and framed applications run under different domains, you must still maintain trust between the domains as before. For more information, see Configure Trusted Domains for SAP Authorization and Trust Management Service [Feature Set B].

> → Recommendation
>
> Test your solutions before third-party cookies have been phased out. To test how your solutions function once third-party cookies have been phased out, enable the `partitionedCookies` property of the Security Settings API of the SAP Authorization and Trust Management service and run your tests. If your business scenarios fail, set `partitionedCookies` to false to help troubleshoot the source of the problem.
>
> For more information, see the following:
>
> - How to test: Preparing and Testing Your Solution for Third-Party Cookie Phase-Out
> - How to use the API: Accessing Administration Using APIs of the SAP Authorization and Trust Management Service
>
> In subaccounts created before May 30, 2024, the `partitionedCookies` property has a default value of `false`. In subaccounts created afterward, the property has a default value of `true`. When Google begins large-scale phase-out of third-party cookies, we plan to set this property to `true` and read-only in all subaccounts.

Related Information

Trust and Federation with Identity Providers [page 2538]

7.1.9 Security Recommendations for SAP Authorization and Trust Management Service

This service contributes to a central list of security recommendations for SAP BTP.

See the list of security recommendations for the SAP Authorization and Trust Management service.

The following sections present an overview of these recommendations according to your role.

Account Administrators

Account administrators manage access to SAP BTP for their organization. They also manage the apps to which their organization is subscribed.

Security Recommendations for Account Administrators

Summary	Recommendation
Use custom identity providers for better integration and policy enforcement.	BTP-UAA-0008
Check your custom identity providers.	BTP-UAA-0015
Synchronize the user data with a provisioning service.	BTP-UAA-0016
Check which users have critical roles.	BTP-UAA-0017
Check that only administrators have acccess to REST APIs.	BTP-UAA-0022
Remove users who haven't logged on for a long time.	BTP-UAA-0010
Implement a data deletion strategy.	BTP-UAA-0011
Check the validity configuration for access tokens.	BTP-UAA-0005
Rotate the signing keys for access tokens.	BTP-UAA-0019
Rotate the signing keys for the SAML protocol.	BTP-UAA-0020
Ensure understandable links for identity providers on logon pages.	BTP-UAA-0018
Archive audit log entries.	BTP-UAA-0012
Integrate the audit log with your corporate event management system.	BTP-UAA-0013
If you embed the login page of this service, check where you allow the page to be embedded.	BTP-UAA-0001

Summary	Recommendation
Train business users how to handle session timeouts	BTP-UAA-0023

Developers

Developers create new applications for their organization.

Security Recommendations for Developers

Summary	Recommendation
Enable individual secrets for each binding of a service instance.	BTP-UAA-0003
Set the `redirect-uris` parameter to restrict access as much as possible.	BTP-UAA-0002
Check your custom code for updates to client libraries offered by SAP in your dependencies.	BTP-UAA-0021

Platform-as-a-Service Operators

Platform-as-a-service operators manage applications developed by their organization.

Security Recommendations for Platform-as-a-Service Operators

Summary	Recommendation
Automate your deployment process to cope with the lifetime of the X.509 certificates.	BTP-UAA-0014

7.1.10 Rate Limiting

This section provides information on the rate limiting in the SAP Authorization and Trust Management service.

The SAP Authorization and Trust Management service acts as an OAuth authorization server and issues access tokens with a long validity. These tokens are meant to be reused. However, some OAuth clients don't reuse them and thus cause a high load. The SAP Authorization and Trust Management service is scaled per landscape to handle the load.

To protect the SAP Authorization and Trust Management service from overload of misbehaving OAuth clients, rate limiting has been introduced. The limit is per subaccount.

- Requests sent at a lower rate are processed directly.
- When the limit for a subaccount is exceeded, the requests are queued and sent to the SAP Authorization and Trust Management service at a maximum rate (see table). The response times increase because of the queuing.
- If too many requests are queued so that the response time due to queuing exceeds a certain time, the service sends an HTTP 429 response code. This response also contains the http `Retry-After` header, which indicates when the client can retry.

→ Recommendation

There might be company events that trigger a lot of users to log on and try to access one special resource, such as the launch of a special employee discount sale for a certain day. Therefore, it's very likely that a high load of users will try to log on to the special discount website at the announced time. This means that you can foresee that users might run into HTTP 429 error response codes.

- We recommend sending the announcement emails in waves. By doing so, you can avoid a situation where almost all users try to log on at about the same time.
- Additionally, it's helpful to handle the HTTP 429 error message by also sending a clear text message like `We are sorry, but the server is currently experiencing high demand. Please try again later.` Such a message gives the users guidance of what to do.

ⓘ Note

Since rate limiting is per subaccount, SaaS applications probably don't notice rate limiting because the load is distributed across different SaaS subaccounts.

⚠ Restriction

The following table lists sizing restrictions when using APIs.

Sizing Restrictions

Element	Maximum Size
HTTP header	16 KB
POST body	1 MB

Token Endpoint

The token endpoint enables you to authenticate with OAuth 2.0.

Token Endpoint

Endpoint	Subaccount Limit	Effect
`https://<subdomain>.authentication.<landscape>/oauth/token`	Up to 60 requests per second	The requests are executed at once.
`https://<subdomain>.authentication.<landscape>/oauth/token`	Exceeding 60 requests per second	The requests are queued and then executed.
`https://<subdomain>.authentication.<landscape>/oauth/token`	Significantly exceeding 60 requests per second	An `HTTP 429 Too Many Requests` response status code is sent.
`https://<subdomain>.authentication.cert.<landscape>/oauth/token`	Up to 60 requests per second	The requests are executed at once.
`https://<subdomain>.authentication.cert.<landscape>/oauth/token`	Exceeding 60 requests per second	The requests are queued and then executed.
`https://<subdomain>.authentication.cert.<landscape>/oauth/token`	Significantly exceeding 60 requests per second	An `HTTP 429 Too Many Requests` response status code is sent.

Authorization, Security Settings, and Identity Provider Management APIs

The authorization, security settings, and identity provider management APIs enable you to manage service instances, roles, role templates, role collections, and identity providers. You find the APIs of SAP Authorization and Trust Management service and of the security settings in the SAP Business Accelerator Hub.

Authorization, Security Settings, and Identity Provider APIs

Endpoint	Subaccount Limit	Effect
`https://api.authentication.<landscape>/sap/rest/authorization/v2` (Including all APIs with sub-paths, for example `sap/rest/authorization/v2/apps`)	Up to 45 requests per second	The requests are executed at once.
`https://api.authentication.<landscape>/sap/rest/authorization/v2` (Including all APIs with sub-paths, for example `sap/rest/authorization/v2/apps`)	Exceeding 45 requests per second	The requests are queued and then executed.
`https://api.authentication.<landscape>/sap/rest/authorization/v2` (Including all APIs with sub-paths, for example `sap/rest/authorization/v2/apps`)	Significantly exceeding 45 requests per second	An `HTTP 429 Too Many Requests` response status code is sent.
`https://api.authentication.<landscape>/sap/rest/identity-providers` (Including all APIs with subpaths, for example `sap/rest/identity-providers/{id}`)	Up to 30 requests per second	The requests are executed at once.
`https://api.authentication.<landscape>/sap/rest/identity-providers` (Including all APIs with subpaths, for example `sap/rest/identity-providers/{id}`)	Exceeding 30 requests per second	The requests are queued and then executed.

Endpoint	Subaccount Limit	Effect
`https://api.authentication.<landscape>/sap/rest/identity-providers` (Including all APIs with subpaths, for example `sap/rest/identity-providers/{id}`)	Significantly exceeding 30 requests per second	An `HTTP 429 Too Many Requests` response status code is sent.

User Management (SCIM) APIs

The user management (SCIM) APIs enable you to manage shadow users and role collections. You find the APIs of SAP Authorization and Trust Management service in the SAP Business Accelerator Hub.

User Management (SCIM) APIs

Endpoint	Subaccount Limit	Effect
`https://api.authentication.<landscape>/groups` (Including all APIs with subpaths, for example `sap/rest/groups/{id}`)	Up to 3 requests per second	The requests are executed at once.
`https://api.authentication.<landscape>/groups` (Including all APIs with subpaths, for example `sap/rest/groups/{id}`)	Exceeding 3 requests per second	The requests are queued and then executed.
`https://api.authentication.<landscape>/groups` (Including all APIs with subpaths, for example `sap/rest/groups/{id}`)	Significantly exceeding 3 requests per second	An `HTTP 429 Too Many Requests` response status code is sent.
`https://api.authentication.<landscape>/users` (Including all APIs with subpaths, for example `sap/rest/users/{id}`)	Up to 3 requests per second	The requests are executed at once.

Endpoint	Subaccount Limit	Effect
`https://api.authentication.<landscape>/users` (Including all APIs with subpaths, for example `sap/rest/users/{id}`)	Exceeding 3 requests per second	The requests are queued and then executed.
`https://api.authentication.<landscape>/users` (Including all APIs with subpaths, for example `sap/rest/users/{id}`)	Significantly exceeding 3 requests per second	An `HTTP 429 Too Many Requests` response status code is sent.

Related Information

Application Security Descriptor Configuration Syntax [page 592]
Accessing Administration Using APIs of the SAP Authorization and Trust Management Service [page 2773]

7.1.11 Limits for Technical Artifacts of the SAP Authorization and Trust Management Service

To improve the resiliency of the SAP Authorization and Trust Management service, we have introduced limitations on technical artifacts of the service.

Limits for the Service Broker

We allow a maximum of service instances per subaccount.

There is also a maximum of service bindings and service keys in total per service instance. The service rejects attempts to add more bindings or service keys. For more information, see Service Instance Secrets [page 2634].

Limits for the Service Broker

Service Broker Resource	Upper Limit
Service instances per subaccount	2500

Service Broker Resource	Upper Limit
Service bindings and service keys in total per service instances	1000

Limits for the Application Security Descriptor

There are limits that apply when you define properties in the application security descriptor.

You define properties in the `xs-security.json` application security descriptor file for creating or updating service instances. Take care that you don't define more properties than specified in the table.

Limits for Properties

Property	Upper Limit
`attributes`	50
`role-collections`	250
`role-templates`	500
`scopes`	500
`scopes` (granted)	250

> ### Note
>
> Granted means that the scope object refers to applications that are specified using `granted-apps` or `grant-as-authority-to-apps` properties

Limits for the Global Account

It is possible to configure up to 3 custom identity providers for platform users in a global account.

Limits for the Subaccount

There are limits that apply to the subaccount. Administrators can use application roles and create their own roles. They can create and manage role collections, add roles to them, and assign the role collections to business users.

For more information, see Building Roles and Role Collections for Applications [page 2623] and Working with Role Collections [page 2612].

Limits for Security Artifacts

Security Artifact	Upper Limit
Roles created by administrators **Note** Roles that are automatically generated when a service instance was created or updated are not counted here.	2500
Role collections created by administrators	5000

When establishing trust, subaccount administrators must observe the limits for identity providers.

Limits for Identity Providers

Identity Provider	Upper Limit
SAML identity providers	50
User attribute mappings defined in a subaccount	5000

7.1.12 Configuring Backup

For SAP Authorization and Trust Management Service, you can backup the service configuration, the authorizations, and the configuration of the identity provider using APIs.

You can back up your configurations with the SAP Authorization and Trust Management service API package available on the SAP Business Accelerator Hub. The following configurations are included.

- Roles: You can back up role configurations with the Authorization API.
- Role collections and role collection assignments: You can back up configurations for role collections and role collection assignments with the System for Cross-domain Identity Management (SCIM) API.
- Identity provider configurations: You can back up identity provider configurations with the Identity Provider Management API.

For more information on backup, see SAP BTP Service Configuration Backups Managed by Customers.

7.1.13 Accessibility Features in SAP Authorization and Trust Management Service

To optimize your experience of the SAP Authorization and Trust Management service, SAP Business Technology Platform provides features and settings that help you use the software efficiently.

> **Note**
>
> SAP Authorization and Trust Management service runs on the SAP BTP cockpit. For this reason, accessibility features for SAP BTP cockpit apply. For more information, see the accessibility documentation for SAP BTP cockpit on SAP Help Portal in Accessibility Features in SAP BTP cockpit [page 2459].

For more information on screen reader support and keyboard shortcuts, see the Accessibility for End Users.

7.2 Default Role Collections of SAP BTP

The following table displays the default role collections available with SAP BTP after initially deploying your accounts.

For more information about these role collections, see also the *Related Information*.

Default Role Collections Including Roles and Role Templates

Role Collection	Role Name	Role Template	App ID	Role Description
Global Account Administrator	Global Account Admin	GlobalAccount_Admin	cis-central!<suffix>	Role for global account members with read-write authorizations for core commercialization operations, such as updating global accounts, setting entitlements, and creating, updating, and deleting subaccounts. **Note** Users with this authorization don't automatically gain administrator authorizations in all subaccounts, but can grant themselves administrator-level access. You can opt out of this feature. For more information, see Add Members to Your Subaccount [page 2513].
Global Account Administrator	Global Account Usage Reporting Viewer	GlobalAccount_Usage_Reporting_Viewer	uas!<suffix>	Role for global account members with read-only authorizations for core commercialization operations, such as viewing global account usage information.
Global Account Administrator	User and Role Administrator	xsuaa_admin	xsuaa!<suffix>	Manage authorizations, trusted identity providers, and users. ⚠ : Includes security authorizations. See *Scopes with Critical Security Authorizations* under Scopes.
Global Account Administrator	System Landscape Administrator	GlobalAccount_System_Landscape_Administrator	cmp!<suffix>	Administrative access to systems and scenario-related resources.

Role Collection	Role Name	Role Template	App ID	Role Description
Global Account Administrator	Auditlog_Auditor	Auditlog_Auditor	auditlog-management!<suffix>	Read access to audit logs
Global Account Administrator	Auditlog_Auditor	Auditlog_Auditor	auditlog-viewer-initial-content!<suffix>	View audit logs
Global Account Viewer	System Landscape Viewer	GlobalAccount_System_Landscape_Viewer	extension-service-cmp!<suffix>	Viewer access to systems and scenario-related resources.
Subaccount Administrator	Cloud Connector Administrator	Cloud_Connector_Administrator	connectivity!<suffix>	Operate the data transmission tunnels used by the Cloud connector.
Subaccount Administrator	Destination Administrator	Destination_Administrator	destination-xsappname!<suffix>	Manage destination configurations, certificates and subaccount trust via the Destination editor in the SAP BTP cockpit.
Subaccount Administrator	Subaccount Admin	Subaccount_Admin	cis-local!<suffix>	Role for subaccount members with read-write authorizations for core commercialization operations, such as viewing subaccount entitlements, and creating and deleting environment instances.
Subaccount Administrator	User and Role Administrator	xsuaa_admin	xsuaa!<suffix>	Manage authorizations, trusted identity providers, and users. ⚠ : Includes security authorizations. See *Scopes with Critical Security Authorizations* under Scopes.
Subaccount Administrator	Subaccount Service Administrator	Subaccount_Service_Administrator	service-manager!<suffix>	Administrative access to service brokers and environments on a subaccount level.
Subaccount Administrator	Auditlog_Auditor	Auditlog_Auditor	auditlog-management!<suffix>	Read access to audit logs
Subaccount Administrator	Auditlog_Auditor	Auditlog_Auditor	auditlog-viewer-initial-content!<suffix>	View audit logs
Subaccount Administrator	Destination Fragment Administrator	Destination_Fragment_Administrator	destination-xsappname!<suffix>	Manage destination fragments

Role Collection	Role Name	Role Template	App ID	Role Description
Global Account Viewer	Global Account Viewer	GlobalAccount_Viewer	cis-central!<suffix>	Role for global account members with read-only authorizations for core commercialization operations, such as viewing global accounts, subaccounts, entitlements, and regions.
Global Account Viewer	Global Account Usage Reporting Viewer	GlobalAccount_Usage_Reporting_Viewer	uas!<suffix>	Role for global account members with read-only authorizations for core commercialization operations, such as viewing global account usage information.
Global Account Viewer	User and Role Auditor	xsuaa_auditor	xsuaa!<suffix>	Read-only access for authorizations, trusted identity providers, and users. ⚠ : Includes security authorizations. See *Scopes with Critical Security Authorizations* under Scopes.
Global Account Viewer	Auditlog_Auditor	Auditlog_Auditor	auditlog-management!<suffix>	Read access to audit logs
Global Account Viewer	Auditlog_Auditor	Auditlog_Auditor	auditlog-viewer-initial-content!<suffix>	View audit logs
Subaccount Viewer	Cloud Connector Auditor	Cloud_Connector_Auditor	connectivity!<suffix>	View the data transmission tunnels used by the Cloud connector to communicate with back-end systems.
Subaccount Viewer	Destination Viewer	Destination_Viewer	destination-xsappname!<suffix>	View destination configurations, certificates and subaccount trust via the Destination editor in the SAP BTP cockpit.

Role Collection	Role Name	Role Template	App ID	Role Description
Subaccount Viewer	Subaccount Viewer	Subaccount_Viewer	cis-local!<suffix>	Role for subaccount members with read-only authorizations for core commercialization operations, such as viewing subaccount entitlements, details of environment instances, and job results.
Subaccount Viewer	User and Role Auditor	xsuaa_auditor	xsuaa!<suffix>	Read-only access for authorizations, trusted identity providers, and users. ⚠ : Includes security authorizations. See *Scopes with Critical Security Authorizations* under Scopes.
Subaccount Viewer	Subaccount Service Auditor	Subaccount_Service_Auditor	service-manager!<suffix>	Read-only access to service brokers and environments on a subaccount level
Subaccount Viewer	Auditlog_Auditor	Auditlog_Auditor	auditlog-management!<suffix>	Read access to audit logs
Subaccount Viewer	Auditlog_Auditor	Auditlog_Auditor	auditlog-viewer-initial-content!<suffix>	View audit logs
Subaccount Viewer	Destination Fragment Viewer	Destination_Fragment_Viewer	destination-xsappname!<suffix>	View destination fragments
Subaccount Service Administrator	Subaccount Service Administrator	Subaccount_Service_Administrator	service-manager!<suffix>	Administrative access to service brokers and environments on a subaccount level.
Cloud Connector Administrator	Cloud Connector Administrator	Cloud_Connector_Administrator	connectivity!<suffix>	Operate the data transmission tunnels used by the Cloud connector.
Destination Administrator	Destination Administrator	Destination_Administrator	destination-xsappname!<suffix>	Manage destination configurations, certificates and subaccount trust via the Destination editor in the SAP BTP cockpit.
Connectivity and Destination Administrator	Cloud Connector Administrator	Cloud_Connector_Administrator	connectivity!<suffix>	Operate the data transmission tunnels used by the Cloud connector.

Role Collection	Role Name	Role Template	App ID	Role Description
Connectivity and Destination Administrator	Destination Administrator	Destination_Administrator	destination-xsapp-name!<suffix>	Manage destination configurations, certificates and subaccount trust via the Destination editor in the SAP BTP cockpit.
Destination Fragment Administrator	Destination Fragment Administrator	Destination_Fragment_Administrator	destination-xsapp-name!<suffix>	Manage destination fragments
Directory Administrator	Directory Admin	Directory_Admin	cis-central!<suffix>	Role for directory members with read-write authorizations for core commercialization operations, such as updating directories, setting entitlements, and creating, updating, and deleting subaccounts.
Directory Administrator	User and Role Administrator	xsuaa_admin	xsuaa!<suffix>	Manage authorizations, trusted identity providers, and users. ⚠ : Includes security authorizations. See *Scopes with Critical Security Authorizations* under Scopes.
Directory Administrator	Directory Usage Reporting Viewer	Directory_Usage_Reporting_Viewer	uas!<suffix>	Role for directory members with read-only authorizations for core commercialization operations, such as viewing directory usage information.
Directory Viewer	Directory Viewer	Directory_Viewer	cis-central!<suffix>	Role for directory members with read-only authorizations for core commercialization operations, such as viewing directories, subaccounts, entitlements, and regions.

Role Collection	Role Name	Role Template	App ID	Role Description
Directory Viewer	User and Role Auditor	xsuaa_auditor	xsuaa!<suffix>	Read-only access for authorizations, trusted identity providers, and users. ⚠ : Includes security authorizations. See *Scopes with Critical Security Authorizations* under Scopes.
Directory Viewer	Directory Usage Reporting Viewer	Directory_Usage_Reporting_Viewer	uas!<suffix>	Role for directory members with read-only authorizations for core commercialization operations, such as viewing directory usage information.
None	Application Subscriptions Viewer	Application_Subscriptions_Viewer	cis-local!<suffix>	Read-only access to an application's subscriptions.
None	Application Destination Administrator	Application_Destination_Administrator	destination-xsappname!<suffix>	Manage destination configurations, certificates and signing keys for SAML assertions issued by the Destination service.
None	Application Destination Viewer	Application_Destination_Viewer	destination-xsappname!<suffix>	View destination configurations, certificates and signing keys for SAML assertions issued by the Destination service.
None	Destination Administrator Instance	Destination_Administrator_Instance	destination-xsappname!<suffix>	Manage destination configurations, certificates and signing keys for SAML assertions issued by the Destination service on instance level.
None	Destination Certificate Administrator	Destination_Certificate_Administrator	destination-xsappname!<suffix>	Manage destination certificates.

Role Collection	Role Name	Role Template	App ID	Role Description
None	Destination Certificate Instance Administrator	Destination_Certificate_Instance_Administrator	destination-xsapp-name!<suffix>	Manage destination certificates on instance level.
None	Destination Certificate Viewer	Destination_Certificate_Viewer	destination-xsapp-name!<suffix>	View destination certificates.
None	Destination Configuration Administrator	Destination_Configuration_Administrator	destination-xsapp-name!<suffix>	Manage destination configurations.
None	Destination Configuration Instance Administrator	Destination_Configuration_Instance_Administrator	destination-xsapp-name!<suffix>	Manage destination configurations on instance level.
None	Destination Configuration Instance Viewer	Destination_Configuration_Instance_Viewer	destination-xsapp-name!<suffix>	View destination configurations on instance level.
None	Destination Configuration Viewer	Destination_Configuration_Viewer	destination-xsapp-name!<suffix>	View destination configurations.
None	Destination Subaccount Trust Administrator	Destination_Subaccount_Trust_Administrator	destination-xsapp-name!<suffix>	Manage destination subaccount trust via the Destination editor in the cloud cockpit.
None	Destination Subaccount Trust Viewer	Destination_Subaccount_Trust_Viewer	destination-xsapp-name!<suffix>	View destination subaccount trust via the Destination editor in the cloud cockpit.
None	Destination Viewer Instance	Destination_Viewer_Instance	destination-xsapp-name!<suffix>	View destination configurations, certificates and signing keys for SAML assertions issued by the Destination service on instance level.
None	Service Instances Viewer	Service_Instances_Viewer	service-manager!<suffix>	Read-only access to Service Manager instances, bindings, plans, and offerings.
None	Subaccount Service Admin	Subaccount_Service_Admin	service-manager!<suffix>	Administrative access to service brokers and environments on a subaccount level.

Role Collection	Role Name	Role Template	App ID	Role Description
None	Subaccount Service Viewer	Subaccount_Service_Viewer	service-manager!<suffix>	Read-only access to service brokers and environments on a subaccount level.
None	Destination Certificate Instance Viewer	Destination_Certificate_Instance_Viewer	destination-xsapp-name!<suffix>	View destination certificates on instance level
None	Destination Fragment All Properties Administrator	Destination_Fragment_All_Properties_Administrator	destination-xsapp-name!<suffix>	Allow inclusion of all properties in a fragment, even such properties that require the retype of other sensitive properties.
None	Destination Fragment Instance Administrator	Destination_Fragment_Instance_Administrator	destination-xsapp-name!<suffix>	Manage destination fragments on instance level
None	Destination Fragment Instance Viewer	Destination_Fragment_Instance_Viewer	destination-xsapp-name!<suffix>	View destination fragments on instance level
None	Destination Fragment Viewer	Destination_Fragment_Viewer	destination-xsapp-name!<suffix>	View destination fragments
None	Service Credentials Viewer	Service_Credentials_Viewer	service-manager!<suffix>	Read-only access to service bindings including the credentials.

Related Information

Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138]
User Roles in SAP BTP Connectivity

7.3 Trusted Certificate Authentication

All components of SAP BTP rely on a list of trusted Certificate Authorities (CAs), which differ depending on the runtimes used.

For your scenarios with SAP BTP, be aware of this list of CAs to ensure that you can provide trusted certificates that are signed by a trusted CA.

A list of all root CAs approved by SAP Global Security is available in SAP Note 2801396 (SAP Global Trust List).

7.4 Principal Propagation

Exchange user ID information between systems or environments in SAP BTP.

In This Section

- Principal Propagation from the Multi-Cloud Foundation to the Neo Environment [page 3624]
- Principal Propagation from the Neo Environment to the Multi-Cloud Foundation [page 3617]

Other Principal Propagation Scenarios

- On-Premise User Store
- Principal Propagation to OAuth-Protected Applications
- Connectivity in the Cloud Foundry Environment: Principal Propagation
- Connectivity in the Neo Environment: Principal Propagation

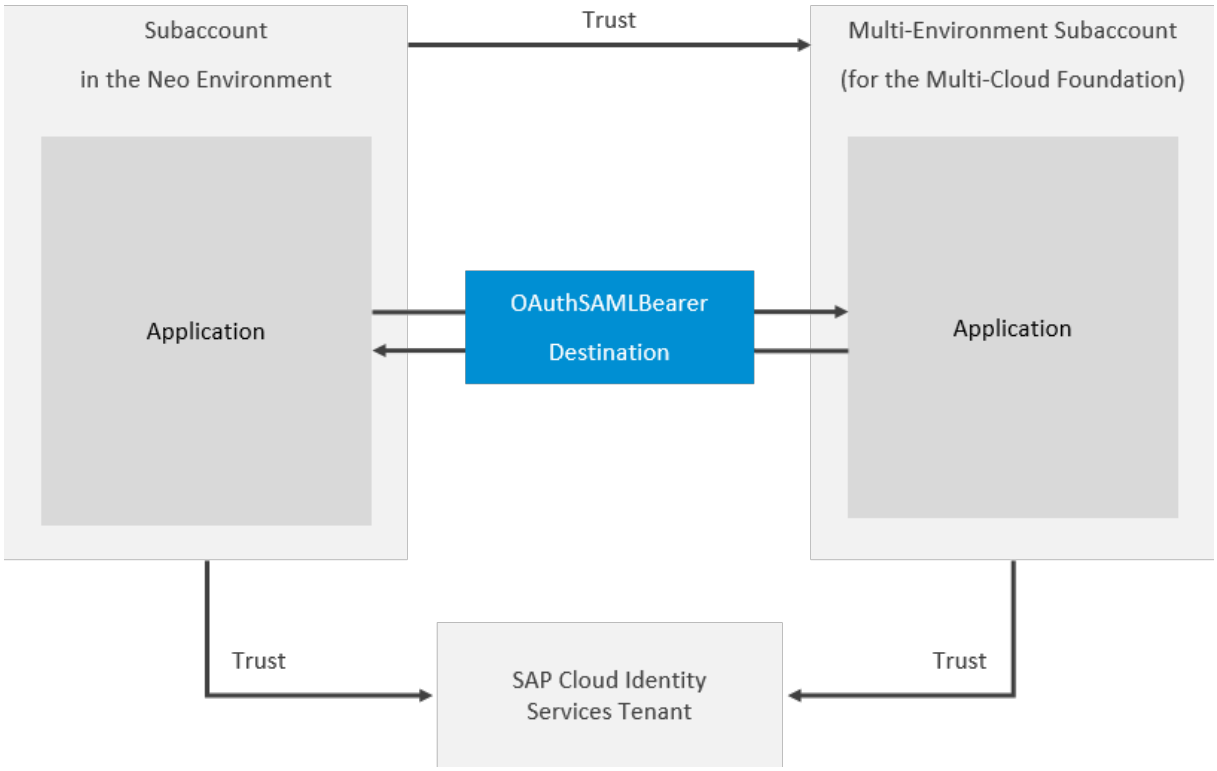
Related Information

Propagate User Information Between Applications or Services [page 555]

7.4.1 Principal Propagation from the Neo Environment to the Multi-Cloud Foundation

Enable an application in your subaccount in the Neo environment to access another application in a multi-environment subaccount (for the multi-cloud foundation) without user login (and user interaction) in the second application. For this scenario to work, the two subaccounts need to be in mutual trust, and in trust with the same SAP Cloud Identity Services tenant. The second application will propagate its logged-in user to the first application using an OAuth2SAMLBearer destination.

The graphic below illustrates the overall setup of the scenario.



Prerequisites

- You have a user account with *Administrator* role in both SAP BTP subaccounts. See Managing Member Authorizations in the Neo Environment.
- You have a custom local service provider configuration (signing keys and certificates, etc.) in your subaccount in the Neo environment. See Configure the Local Service Provider.
- Both accounts have a trust configuration to the same SAP Cloud Identity Services tenant. See:
  - Identity Authentication Tenant as an Application Identity Provider (for the Neo environment)
  - Manually Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2547] (for the multi-cloud foundation)
- You have developed and deployed both applications, each in the corresponding subaccount.

> **Note**
>
> All configuration steps described in this tutorial are done using the SAP BTP cockpit.

Requirements to the Application in the Neo Environment

The application is running on a Java Web Tomcat 9 runtime.

In the source code, the application needs to reference the destination that we are about to create as a later step. The sample source code below illustrates a complete servlet working with the destination with name `pptest`.

```
package com.sap.cloud.samples;
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.URL;
import java.net.URLConnection;
import java.security.KeyStore;
import java.util.List;
import javax.naming.Context;
import javax.naming.InitialContext;
import javax.net.ssl.HttpsURLConnection;
import javax.net.ssl.SSLContext;
import javax.net.ssl.SSLSocketFactory;
import javax.net.ssl.TrustManagerFactory;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import com.sap.core.connectivity.api.authentication.AuthenticationHeader;
import com.sap.core.connectivity.api.authentication.AuthenticationHeaderProvider;
import com.sap.core.connectivity.api.configuration.ConnectivityConfiguration;
import com.sap.core.connectivity.api.configuration.DestinationConfiguration;
@WebServlet("/neotocf")
public class NeoToCF extends HttpServlet {
  private static final long serialVersionUID = 1L;
  private static final Logger LOGGER = LoggerFactory.getLogger(NeoToCF.class);
  private static final String ON_PREMISE_PROXY = "OnPremise";
  protected void doGet(HttpServletRequest request, HttpServletResponse response)
           throws ServletException, IOException {
    response.getWriter().println("Served to: " +
request.getUserPrincipal().getName());
    try {
      // Look up the connectivity configuration API
      Context ctx = new InitialContext();
      ConnectivityConfiguration configuration = (ConnectivityConfiguration) ctx
                    .lookup("java:comp/env/connectivityConfiguration");
      // Get destination configuration
      DestinationConfiguration destConfiguration =
configuration.getConfiguration("pptest");
      if (destConfiguration == null) {
        response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR,
                        String.format(
                                "Destination %s is not found. Hint:" +
                                       " Make sure to have the destination
configured.",
                                "pptest"));
                return;
      }

      AuthenticationHeaderProvider authHeaderProvider =
(AuthenticationHeaderProvider) ctx
                    .lookup("java:comp/env/myAuthHeaderProvider");
      // retrieve the authorization header for OAuth SAML Bearer principal
propagation
      List<AuthenticationHeader> samlBearerHeader = authHeaderProvider
                    .getOAuth2SAMLBearerAssertionHeaders(destConfiguration);
      LOGGER.debug("JWT token from CF XSUAA: " +
samlBearerHeader.get(1).getValue());

      // get the configured truststore
```

```
      KeyStore trustStore = destConfiguration.getTrustStore();

      // create sslcontext
      TrustManagerFactory tmf =
TrustManagerFactory.getInstance(TrustManagerFactory.getDefaultAlgorithm());
      tmf.init(trustStore);

      SSLContext sslcontext = SSLContext.getInstance("TLS");
      sslcontext.init(null, tmf.getTrustManagers(), null);
      SSLSocketFactory sslSocketFactory = sslcontext.getSocketFactory();

      // get the destination URL
      String value = destConfiguration.getProperty("URL");
      URL url = new URL(value);

      // use the sslcontext for url connection
      URLConnection urlConnection = url.openConnection();
      ((HttpsURLConnection) urlConnection).setSSLSocketFactory(sslSocketFactory);

      urlConnection.setRequestProperty(samlBearerHeader.get(0).getName(),
samlBearerHeader.get(0).getValue());
      urlConnection.setRequestProperty(samlBearerHeader.get(1).getName(),
samlBearerHeader.get(1).getValue());

      urlConnection.connect();
      response.getWriter().println("Received from CF:");

      BufferedReader in = new BufferedReader(new InputStreamReader(
      urlConnection.getInputStream()));
      String inputLine;
      while ((inputLine = in.readLine()) != null)
                response.getWriter().append(inputLine);
            in.close();
    } catch (Exception e) {
        // Connectivity operation failed
        String errorMessage = e.getMessage();
        LOGGER.error("Connectivity operation failed", e);
        response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR,
errorMessage);
    }
  }
  protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
    doGet(request, response);
  }
}
```

Requirements to the Application in the Multi-Cloud Foundation

In the multi-cloud foundation, you need an application following the XSA security model (protected with SAML, UAA service binding using JWT token needed, roles configured in the *xs-security.json*).

See:

- Application Router [page 453]
- Building Roles and Role Collections for Applications [page 2623]

> **Note**
>
> You can use the XSA Security Sample Application in GitHub (instructions and code) to develop and deploy an application compliant with the above requirements.

7.4.1.1 Create a Destination

Prerequisites

Before you create the required destination, you need to note down a few properties that will be used as values in the destination settings.

1. In the cloud cockpit, navigate to the subaccount in the Cloud Foundry environment.
2. Navigate to the application router.
3. Enter the *Environment Variables* section.
4. Note down somewhere the values of the following properties:
   - clientid
   - clientsecret
   - url

Context

Connect the first subaccount to the second subaccount by describing the source connection properties in a destination. For more information see Modeling Destinations.

Procedure

1. Choose the global account and navigate to *Connectivity* > *Destinations*.
2. Choose *New Destination*.
3. In the new destination, provide the following information:

Field	Description
Name	Technical name of the destination. It can be used later on to get an instance of that destination. It must be unique for the global account. **Note** For the purposes of the example listed in this document, use *pptest* as value.
URL	The URL of the protected resource in the Cloud Foundry environment. See Configuring Application URLs. Example: `https://<tenant-specific-route-for-your-business-app>.cfapps.eu10.hana.ondemand.com/`

Field	Description
Authentication	OAuth2SAMLBearerAssertion
Proxy Type	Internet

Audience	Copy the value of `entityID` property of the SAML 2.0 metadata representing your subaccount in the Cloud Foundry environment.
	
	→ Tip
	
	You can open the metadata of the subaccount in the Cloud Foundry environment using the following URL:
	
	`https://<your subaccount's subdomain>.authentication.<region host>/saml/metadata`
	
	Example:
	
	`https://demo.authentication.eu10.hana.ondemand.com/saml/metadata`
	
	For the `<region host>`, see Regions and API Endpoints Available for the Cloud Foundry Environment [page 19].
	
	Example of audience/entityID:
	
	`demo.aws-live-eu10`

Client Key	In the cloud cockpit, navigate to the application in the Cloud Foundry environment (*<path to your subaccount>* > *Spaces* > *<your space>* > *Applications* > *<your application>*). Open *Environment Variables*. Copy the value of the *clientid* property in *VCAP_SERVICES* > *xsuaa* > *credentials*.

Field	Description

Token Service URL	Get the **token service URL** from the SAML 2.0 metadata representing your subaccount in the Cloud Foundry environment. The **token service URL** is defined in the `Location` attribute of the element marked as `AssertionConsumerService`, like this :
	`<md:AssertionConsumerService Location="<Token Service URL>" Binding="urn:oasis:names:tc:SAML:2.0:bindings:URI" index="1"/>`
	→ Tip
	You can open the metadata of the subaccount in the Cloud Foundry envirnoment using the following URL:
	`https://<your subaccount's subdomain>.authentication.<region host>/saml/metadata`
	Example:
	`https://demo.authentication.eu10.hana.ondemand.com/saml/metadata`
	For the `<region host>`, see Regions and API Endpoints Available for the Cloud Foundry Environment [page 19].
	Example of token service URL:
	`https://demo.authentication.eu10.hana.ondemand.com/oauth/token/alias/demo.aws-live-eu10`

Token Service User	In the cloud cockpit, navigate to the application in the Cloud Foundry environment (*<path to your subaccount>* > *Spaces* > *<your space>* > *Applications* > *<your application>*). Open *Environment Variables*. Copy the value of the *clientid* property in *VCAP_SERVICES* > *xsuaa* > *credentials* .
Token Service Password	In the cloud cockpit, navigate to the application in the Cloud Foundry environment (*<path to your subaccount>* > *Spaces* > *<your space>* > *Applications* > *<your application>*). Open *Environment Variables*. Copy the value of the *clientsecret* property in *VCAP_SERVICES* > *xsuaa* > *credentials* .
System User	Empty.

4. Save the changes.

7.4.1.2 Create Trust Between the Subaccounts

Context

Procedure

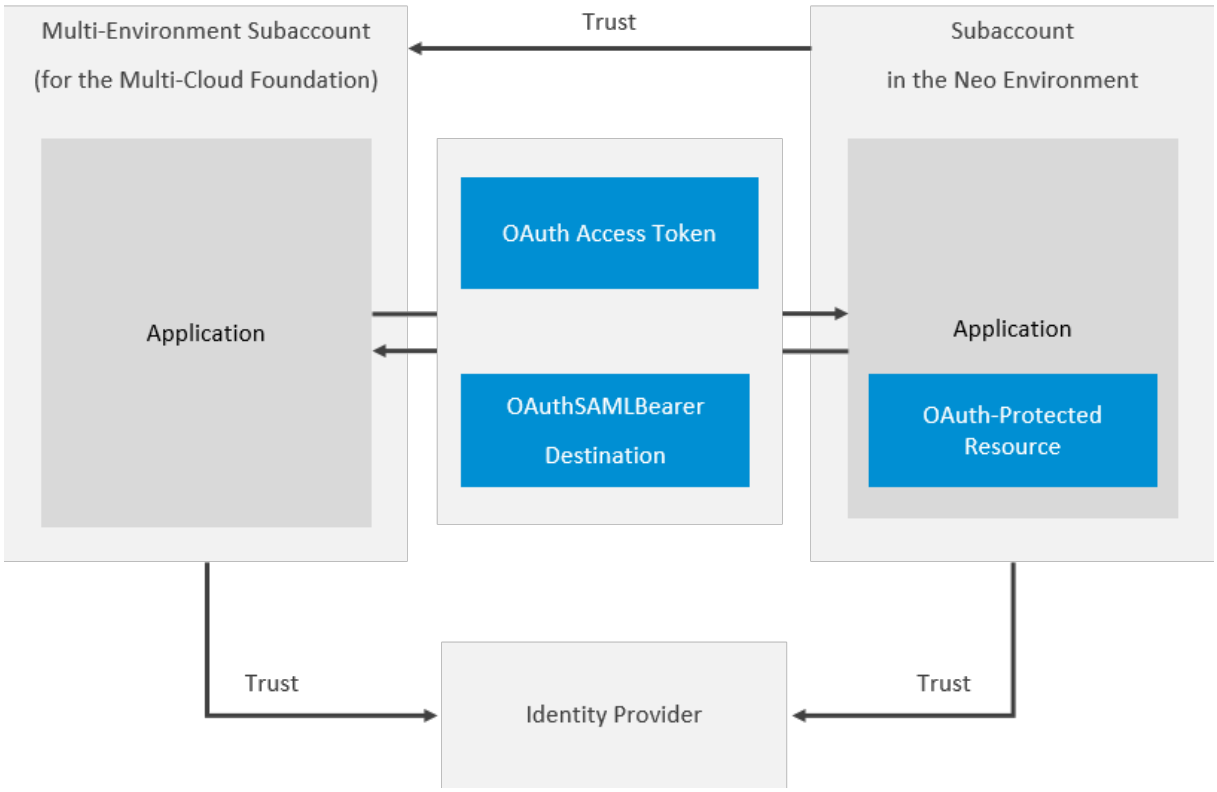
1. In the cloud cockpit, log on with the Administrator user.
2. Save locally the service provider metadata of the subaccount in the Neo environment.
   a. In SAP BTP cockpit, navigate to the subaccount in the Neo environment. See Navigate in the Cockpit [page 2461]
   b. Navigate to *Security* > *Trust* > *Local Service Provider*.
   c. Choose *Get Metadata* and save the metadata file representing this subaccount.
3. In the subaccount in the Cloud Foundry environment, create trust to the subaccount in the Neo environment.
   a. Navigate to the subaccount in the Cloud Foundry environment.
   b. Navigate to *Security* > *Trust Configuration*
   c. Choose *New Trust Configuration*.
   d. In the *Metadata* field, choose the *Upload* button.
   e. Upload the service provider metadata file representing the subaccount in the Neo environment.
   f. Save the new trust configuration.

   After this procedure, you can use the security context from the application in the Neo environment to the application in the Cloud Foundry environment. The assigned groups from the Neo environment can be used as role collections in the Cloud Foundry environment.

7.4.2 Principal Propagation from the Multi-Cloud Foundation to the Neo Environment

Enable an application in your multi-environment subaccount (for the multi-cloud foundation) to access an OAuth-protected application in a subaccount in the Neo environment without user login (and user interaction) in the second application. For this scenario to work, the two subaccounts need to be in mutual trust, and in trust with the same identity provider. The first application will propagate its logged-in user to the second application using an OAuth2SAMLBearer destination.

The graphic below illustrates the overall setup of the scenario.



Prerequisites

- You have a user account with *Administrator* role in both SAP BTP subaccounts. See Managing Member Authorizations in the Neo Environment.
- You have a *custom* local service provider configuration (this means in *cloud cockpit* > *Security* > *Trust* > *Local Service Provider* > you have chosen *Configuration Type* > *Custom*) in your subaccount in the Neo environment. See Configure the Local Service Provider.
- Both accounts have a trust configuration to the same identity provider. See:
  - Configure Trust to the SAML Identity Provider (for the Neo environment)
  - Establish Trust with Any SAML 2.0 Identity Provider in a Subaccount [page 2552] (for the multi-cloud foundation)
- The application in the Neo environment is protected using OAuth 2.0. See OAuth 2.0 Service.
- The application in the multi-environment subaccount is bound to an instance of the following services:
  - *Destination Service*. See Create and Bind a Destination Service Instance.
  - *xsuaa*
- You have deployed both applications, each in the corresponding subaccount.

> **Note**
>
> All configuration steps described in this tutorial are done using the cloud cockpit.

Contents

- Create Trust Between the Subaccounts [page 3626]
- Create an OAuth Client [page 3628]
- Create a Destination [page 3628]

7.4.2.1 Create Trust Between the Subaccounts

Exchange keys and certificates between the subaccounts, and configure trust between them. This will enable the subaccounts to communicate using HTTP destinations.

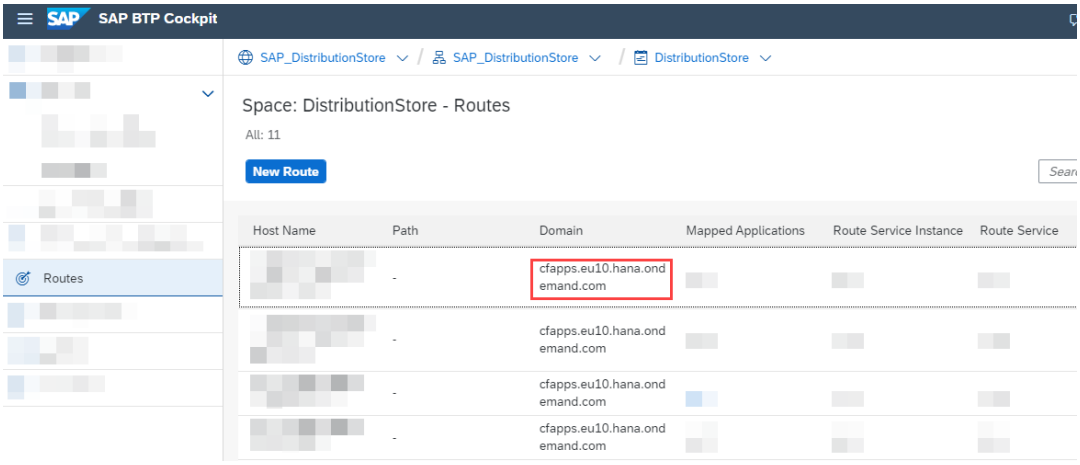
Context

Procedure

1. In the cloud cockpit, log on with the Administrator user.
2. Save locally the identifying X509 certificate of the subaccount in the Cloud Foundry environment.
   a. In SAP BTP cockpit, navigate to the subaccount in the Cloud Foundry environment.
   b. Navigate to *Connectivity* > *Destinations*.
   c. Choose *Download Trust* and save the X509 certificate identifying this subaccount.
3. In the subaccount in the Neo environment, create trust to the subaccount in the Cloud Foundry environment.
   a. Navigate to the subaccount in the Neo environment.
   b. Navigate to *Security* > *Trust* > *Application Identity Provider*.
   c. Choose *Add Trusted Identity Provider* and configure the new trust configuration settings:
      - In the *Name* field, enter the following:
        `<your Cloud Foundry domain host>/<Cloud Foundry subaccount ID>`

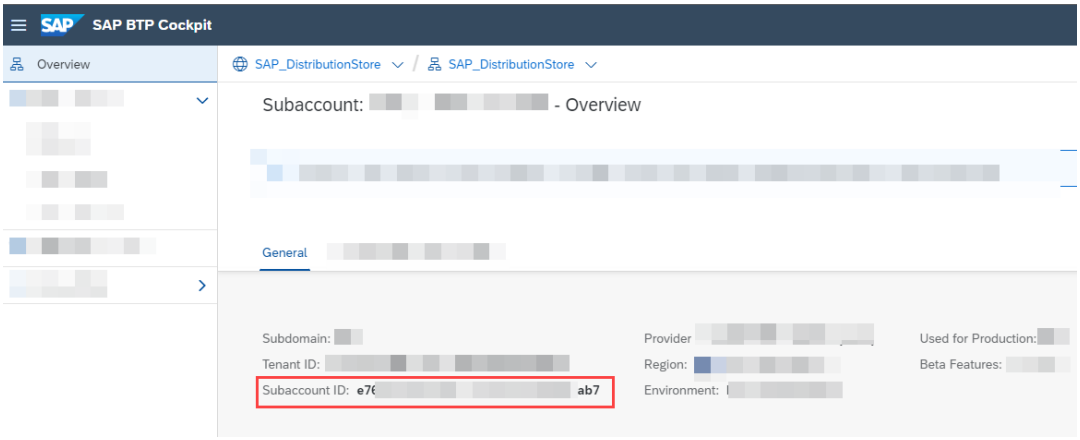
        → Tip

        You can view the **Cloud Foundry domain host** in *cockpit* > *<your global account>* > *<your subaccount>* > *<your space>* > *Routes*.



→ Tip

You can view the **subaccount ID** in *cockpit* > *<your global account>* > *<your subaccount>* > *Overview*.



- In the *Signing Certificate* field, enter the X509 certificate of the Cloud Foundry account.

  ⓘ Note

  Make sure you remove the `BEGIN CERTIFICATE` and `END CERTIFICATE` parts.

- Mark the *Only for IDP-initiated SSO* option.

d. Save the new trust configuration.

7.4.2.2 Create an OAuth Client

You need an OAuth client to get an access token for the OAuth-protected resources in the application in the Neo environment.

Context

For more information about working with OAuth clients, see Create an OAuth Client [page 3628].

Procedure

1. In the cockpit, navigate to the subaccount in the Neo environment.
2. Navigate to *Security* > *OAuth* > *Clients*, and choose *Register New Client*.
3. Create an OAuth client with the following configuration settings:
   - *Name* - the OAuth client name. You will need to provide this name as value of the *Token Service User* property of the destination below.
   - *Authorization Grant* - choose the *Authorization Code* option.
   - Mark the *Confidential* option, and provide a secret (password).
4. Save the client.

> **Note**
>
> When creating the required **OAuthSAMLBearer destination** later, you will need the following information from the OAuth client:
>
> - *ID*
> - *Secret*

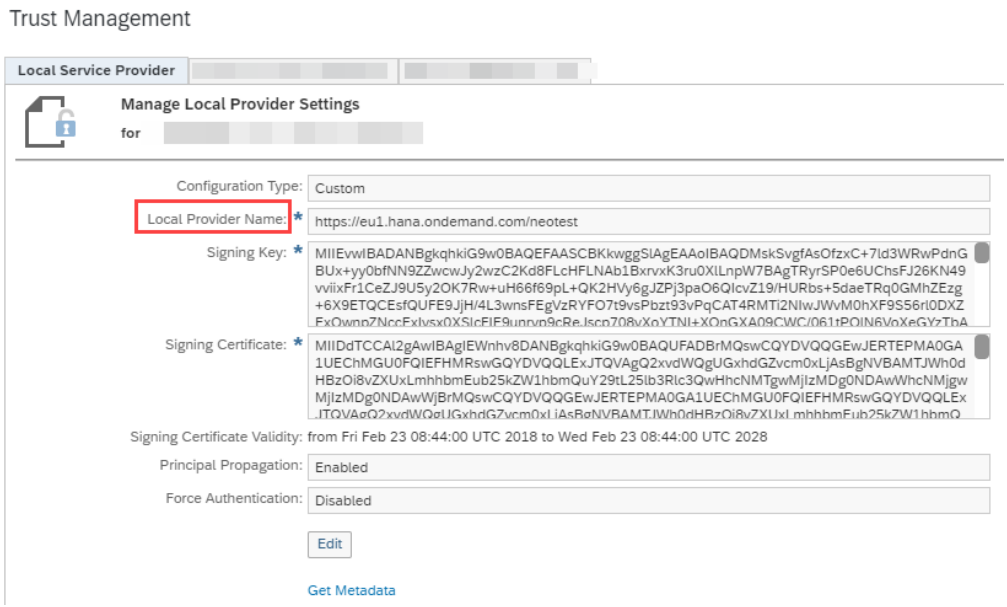
7.4.2.3 Create a Destination

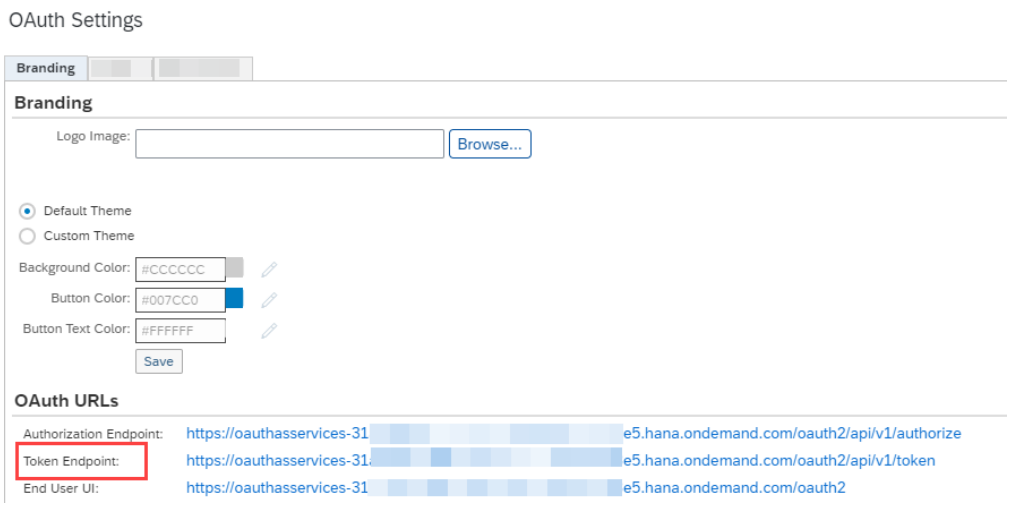
Context

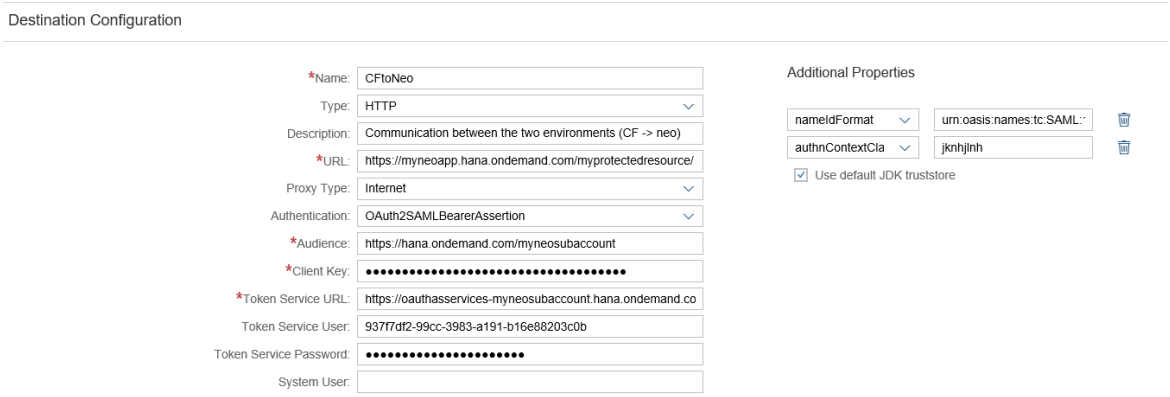
Connect the two subaccounts by describing the connection properties in a destination. For more information, see Modeling Destinations.

Procedure

1. Choose the subaccount in the Cloud Foundry environment, and navigate to *Connectivity* > *Destinations*.
2. Choose *New Destination*.
3. In the new destination, provide the following information:

Field	Description
Name	Technical name of the destination. It can be used later on to get an instance of that destination. It must be unique for the global account.
Description	Free-text description.
Type	HTTP
URL	The URL of the protected resource in the Neo environment. Example: `https://myneoapp.hana.ondemand.com/myprotectedresource/`
Authentication	OAuth2SAMLBearerAssertion
Proxy Type	Internet
Audience	The value of the local service provider name in the subaccount in the Neo environment. Copy the value from *cockpit* > *<your Neo subaccount>* > *Security* > *Trust* > *Local Service Provider* > *Local Service Provider Name*. 

Field	Description
Client Key	The ID of the OAuth client for the application in the Neo environment.
Token Service URL	Copy the value of *Token Endpoint* from the following place: *cockpit* > *<your Neo subaccount>* > *Security* > *OAuth* > *Branding*. 
Token Service User	The ID of the OAuth client for the application in the Neo environment.
Token Service Password	The secret from the OAuth client.
System User	Empty.
authnContextClassRef	*urn:oasis:names:tc:SAML:2.0:ac:classes:PreviousSession*
nameIdFormat	*urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified* if the user ID will be propagated to the Neo application or *nameIdFormat = urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress* if the user email will be propagated to the Neo application.



4. Save the changes.

7.4.3 Principal Propagation from SAP BTP Applications to On-Premise Systems Using Corporate IdP Tokens

You have a scenario where your SAP BTP application, such as SAP Build Work Zone, interacts with on-premise systems for which you don't want to accept tokens from SAP BTP identity and access management services, namely SAP Cloud Identity Services and SAP Authorization and Trust Management service. You can configure SAP BTP applications to include a token from the corporate identity provider you trust.

Prerequisites

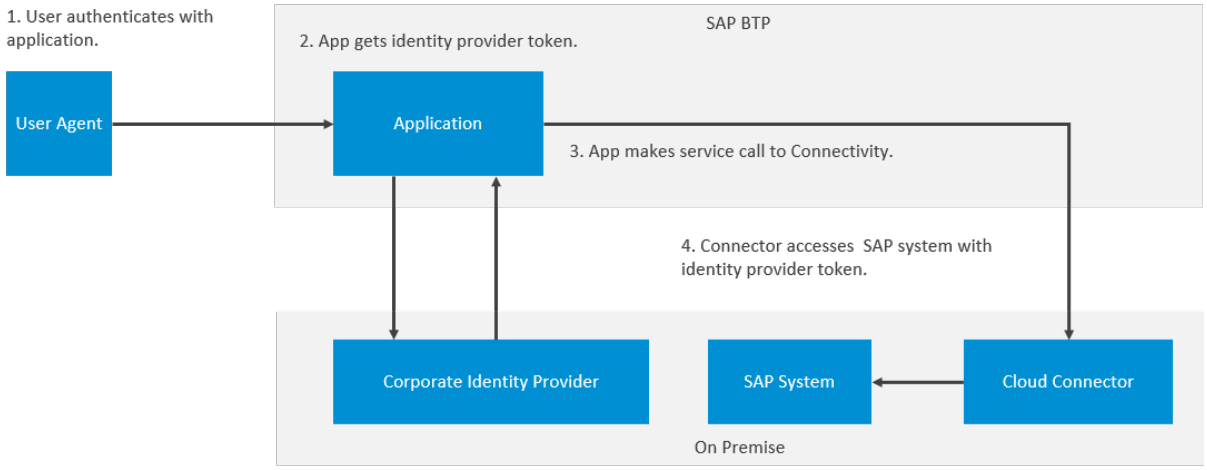
- You have configured trust between your subaccount and an SAP Cloud Identity Services tenant with OpenID Connect (OIDC).
  For more information, see Establish Trust and Federation Between SAP Authorization and Trust Management Service and SAP Cloud Identity Services [page 2540].
- You have configured trust between a corporate identity provider and your SAP Cloud Identity Services tenant with OIDC.
  For more information, see Configure Trust with OpenID Connect Corporate Identity Provider in the SAP Cloud Identity Services documentation.
- You have configured both your Cloud application and your on-premise application to authenticate with the same corporate identity provider.
  Set the corporate identity provider as the default identity provider.
  For more information, see Choose Default Identity Provider for an Application.

Context

In this scenario, the SAP BTP application passes the token of a corporate identity provider with the help of connectivity services to your on-premise SAP system. To configure this form of principal propagation, you must ensure SAP Cloud Identity Services and the relevant connectivity services are set up to support this scenario.

The following figure illustrates this scenario.

Cloud Connector Using Embedded Tokens



> ### ⚠ Restriction
>
> All functions that depend on principal propagation to back-end applications that require tokens from the corporate identity provider only work for users hosted by that corporate identity provider. Users from identity providers other than the corporate identity provider, including local users in the Identity Directory of SAP Cloud Identity Services can only perform the functions defined by the application.

Procedure

1. Subscribe to an application in your subaccount that supports this principal propagation scenario.
   Refer to the documentation of the application, which supports this scenario.
2. Enable SAP Cloud Identity Services to forward tokens of the corporate identity provider.
   - For pure proxy scenarios:
     Add the token claim `forward_corp_idp_token` with the static value **`id_token`** in the configuration of the corporate identity provider.
     For more information, see Enrich Token Claims Coming from Corporate IdP.
   - For federation scenarios:
     Add the token claim `forward_corp_idp_token` with the static value **`id_token`** to all relevant applications in the SAP Cloud Identity Services tenant.
     For more information, see Configuring Attributes Based on Flexible Expressions.

     > ### → Tip
     >
     > Apply this setting at least to the application that represents the subaccount trust. Other applications from the subaccount that were added after January 13, 2026 are children of the application representing the subaccount trust and inherit this setting.
     >
     > To find further applications from the subaccount that might have been added earlier, search for the applications by subaccount ID in SAP Cloud Identity Services. (For finding the subaccount ID, see here.) Add the same claim to the individual applications.

3. Check if you need to configure a destination to connect your application to the Cloud Connector.

   The application you subscribed to may have provisioned this destination for you. Check the configuration documentation of the application to which you subscribed.

   For more information, see Principal Propagation SSO Authentication for HTTP.
4. Check the Cloud Connector configuration to disable trust to SAP Cloud Identity Services and SAP Authorization and Trust Management service.

   For more information, see Configure Trusted Entities in the Cloud Connector.

Next Steps

You can now test the application you subscribed to. If the principal propagation doesn't work as expected, try the following:

- Troubleshooting logs of SAP Cloud Identity Services
  For more information, see Logging OpenID Connect Tokens.
- Logs of the Cloud Connector
  For more information, see Monitoring, Logging, And Troubleshooting.

7.5 Data Protection and Privacy

Data protection is associated with numerous legal requirements and privacy concerns. In addition to compliance with general data protection and privacy acts, it's necessary to consider compliance with industry-specific legislation in different countries/regions.

SAP provides specific features and functions to support compliance with regard to relevant legal requirements, including data protection. SAP does not give any advice on whether these features and functions are the best method to support company, industry, regional, or country/region-specific requirements. Furthermore, this information shouldn't be taken as advice or a recommendation regarding additional features that would be required in specific IT environments. Decisions related to data protection must be made on a case-by-case basis, taking into consideration the given system landscape and the applicable legal requirements.

> ⓘ Note
>
> SAP does not provide legal advice in any form. SAP software supports data protection compliance by providing security features and specific data protection-relevant functions. In many cases, compliance with applicable data protection and privacy laws will not be covered by a product feature. Definitions and other terms used in this document are not taken from a particular legal source.

> ⚠ Caution
>
> The extent to which data protection is supported by technical means depends on secure system operation. Network security, security note implementation, adequate logging of system changes, and appropriate usage of the system are the basic technical requirements for compliance with data privacy legislation and other legislation.

Generic Fields

You also need to make sure that no personal data enters the system in an uncontrolled or non-purpose related way, for example, in free-text fields, or customer extensions.

> ⚠ Caution
>
> Don't enter personal data in fields, such as global and subaccount names, Cloud Foundry orgs and spaces names, database names, and tenant names. Personal data in such fields can't be detected and removed during data deletion requests. Even if such cases could be identified, removing such data may lead to disruptive and unexpected behavior in our services. The previous list of example fields isn't complete.

SAP BTP

This documentation covers personal data relating to SAP BTP accounts and data stored in databases by SAP BTP. SAP BTP offers a number of capabilities, that is, services, buildpacks, application, and so on. Here we cover the core platform. For more information about data protection and privacy for capabilities you've purchased, see the data protection and privacy documentation for those capabilities.

To view the services consumed by a global account:

1. Navigate to the global account to which you'd like to view members.
   For more information, see Navigate to Orgs and Spaces [page 2792].
2. In the navigation area, choose *Entitlements*.

This documentation is written with the data protection officer of a company in mind. The processes described here may be required for a data protection officer or an administrator of the user accounts for your tenants or even business users of the tenants. In particular the processes for business users are described here so that you in your role of data protection officer or account administrator can communicate them to your business users if required.

Users are stored in the platform identity provider.

- Global account users are stored in platform identity provider or a tenant of SAP Cloud Identity Services.
- Platform users are stored in platform identity provider, a tenant of SAP Cloud Identity Services, or your own identity provider.
- Business users are stored in a tenant of SAP Cloud Identity Services or your own identity provider.

Related Information

Glossary for Data Protection and Privacy [page 3635]
Change Logging and Read-Access Logging [page 3637]
Personal Data Record [page 3637]
Deletion [page 3638]
Consent [page 3644]

7.5.1 Glossary for Data Protection and Privacy

The following terms are general to SAP products. Not all terms may be relevant for SAP BTP.

Term	Definition
Blocking	A method of restricting access to data for which the primary business purpose has ended.
Business purpose	The legal, contractual, or in other form justified reason for the processing of personal data to complete an end-to-end business process. The personal data used to complete the process is predefined in a purpose, which is defined by the data controller. The process must be defined before the personal data required to fulfill the purpose can be determined.
Consent	The action of the data subject confirming that the usage of his or her personal data shall be allowed for a given purpose. A consent functionality allows the storage of a consent record in relation to a specific purpose and shows if a data subject has granted, withdrawn, or denied consent.
Data subject	Any information relating to an identified or identifiable natural person ("data subject"). An identifiable natural person is one who can be identified, directly or indirectly, in particular by reference to an identifier such as a name, an identification number, location data, an online identifier, or to one or more factors specific to the physical, physiological, genetic, mental, economic, cultural, or social identity of that natural person.
Deletion	Deletion of **personal data** so that the data is no longer available.
End of business	Defines the end of active business and the start of residence time and retention period.
End of purpose (EoP)	The point in time when the processing of a set of personal data is no longer required for the primary business purpose, for example, when a contract is fulfilled. After the EoP has been reached, the data is blocked and can only be accessed by users with special authorizations (for example, tax auditors).
End of purpose (EoP) check	A method of identifying the point in time for a data set when the processing of **personal data** is no longer required for the primary **business purpose**. After the **EoP** has been reached, the data is **blocked** and can only be accessed by users with special authorization, for example, tax auditors.

Term	Definition
Personal data	Any information relating to an identified or identifiable natural person ("data subject"). An identifiable natural person is one who can be identified, directly or indirectly, in particular by reference to an identifier such as a name, an identification number, location data, an online identifier, or to one or more factors specific to the physical, physiological, genetic, mental, economic, cultural, or social identity of that natural person.
Purpose	The information that specifies the reason and the goal for the processing of a specific set of personal data. As a rule, the purpose references the relevant legal basis for the processing of personal data.
Residence period	The period of time between the end of business and the end of purpose (EoP) for a data set during which the data remains in the database and can be used in case of subsequent processes related to the original purpose. At the end of the longest configured residence period, the data is blocked or deleted. The residence period is part of the overall retention period.
Retention period	The period of time between the end of the last business activity involving a specific object (for example, a business partner) and the deletion of the corresponding data, subject to applicable laws. The retention period is a combination of the residence period and the blocking period.
Sensitive personal data	A category of personal data that usually includes the following type of information: • Special categories of personal data, such as data revealing racial or ethnic origin, political opinions, religious or philosophical beliefs, trade union membership, genetic data, biometric data, data concerning health or sex life or sexual orientation. • Personal data subject to professional secrecy • Personal data relating to criminal or administrative offenses • Personal data concerning insurances and bank or credit card accounts

Term	Definition
Technical and organizational measures (TOM)	Some basic requirements that support data protection and privacy are often referred to as technical and organizational measures (TOM). The following topics are related to data protection and privacy and require appropriate TOMs, for example: • **Access control:** Authentication features • **Authorizations:** Authorization concept • **Read access logging** • **Transmission control / Communication security** • **Input control / Change logging** • **Availability control** • **Separation by purpose:** Is subject to the organizational model implemented and must be applied as part of the authorization concept.

7.5.2 Change Logging and Read-Access Logging

Change logging records changes to personal data, while read-access logging records access to sensitive personal data. You may be required to gather this information for auditing purposes or legal requirements.

Use the appropriate API to retrieve logs.

- For the Cloud Foundry, see Audit Log Retrieval API Usage for Subaccounts in the Cloud Foundry Environment [page 2880].

> ### Note
>
> For any applications you develop, you must ensure they include logging functions. SAP BTP does not provide audit logging functions for custom developments.

Logs are deleted regularly.

7.5.3 Personal Data Record

A personal data record is a collection of data relating to a data subject. A data privacy specialist may be required to provide such a record or an application may offer a self-service.

To see the personal data that is used for membership management within SAP BTP, access the cloud cockpit.

1. Navigate to the global account to which you'd like to view members.
   For more information, see Navigate to Orgs and Spaces [page 2792].
2. In the navigation area, choose *Members*.

To see the personal data that is used for application logging within SAP BTP, access the cloud cockpit.

For more information, see Using Logs in the Cockpit or Analyze Logs from the Cockpit in the Application Logging service documentation.

If you do not use your own identity provider for identity federation, you can view the profiles available in SAP Cloud Identity Services - Identity Authentication.

For more information, see Information Report in the SAP Cloud Identity Services - Identity Authentication documentation.

For SAP BTP Cloud Foundry environment, the User Account and Authentication service creates shadow users to issue tokens for their corresponding users.

For more information about viewing shadow users and their attributes, see the User Management (SCIM) API on SAP Business Accelerator Hub.

For all other services, which persist data, such as databases or document services, retrieve the data you stored with the same APIs, protocols, or languages you used to store the data.

To view the services used in a global account, choose *Entitlements* in the navigation area.

7.5.4 Deletion

The processing of personal data is subject to applicable laws related to the deletion of this data when the specified, explicit, and legitimate purpose for processing this personal data has expired. If there is no longer a legitimate purpose that requires the retention and use of personal data, it must be deleted.

When deleting data in a data set, all referenced objects related to that data set must be deleted as well. Industry-specific legislation in different countries also needs to be taken into consideration in addition to general data protection laws. After the expiration of the longest retention period, the data must be deleted.

When accounts expire, we delete your data barring legal requirements that SAP retains your data. If your organization has separate retention requirements, you are responsible for saving this data before we terminate your account.

- For trial accounts in the Cloud Foundry environment, your account expires after 365 days.
- Productive accounts expire based on the terms of your contract.

To deactivate or delete users, see Erasure in the SAP Cloud Identity Services documentation.

For SAP BTP Cloud Foundry environment, the User Account and Authentication service creates shadow users to issue tokens for their corresponding users. Data privacy regulations or policies may require you to delete this data, for example, when the user has left your organization.

For more information about deleting shadow users, see:

- Delete Shadow Users for Data Protection and Privacy Using the Cockpit [page 3639]
- Delete Shadow Users for Data Protection and Privacy Using APIs [page 3642]

> → Remember
>
> When you delete a user at the Cloud Foundry level, SAP BTP doesn't delete the corresponding shadow user at the subaccount level.

When you delete a user at the subaccount level, ensure that you delete the user at the Cloud Foundry level too. Otherwise, if the Cloud Foundry user logs on again, the system automatically creates the corresponding user at the subaccount level.

So ensure that you delete the shadow users on all levels.

For all other services which persist data, you can retrieve the data you stored with the same APIs, protocols, or languages which you used to store the data.

To view the services used in a global account, choose *Entitlements* in the navigation area.

SAP BTP Data Retention Manager is a service available for the Cloud Foundry environment that helps you to identify data subjects for deletion as well as maintain rules for residence and retention.

For more information, see SAP BTP Data Retention Manager.

We maintain backups of the data for disaster recovery. When your account is deleted, we may have this data in our backup system for the length of our backup cycle.

> **Note**
>
> If your data is stored outside SAP BTP, we cannot guarantee that your data does not get reintegrated if you are pushing such data to our systems. You are responsible for terminating such integrations.
>
> We cannot restore data you have in your local system.

7.5.4.1 Delete Shadow Users for Data Protection and Privacy Using the Cockpit

Data privacy regulations or policies may require you to delete this data, for example, when the user has left your organization. SAP BTP cockpit offers an application to select and delete shadow users.

Prerequisites

You have the required authorizations.

For more information, see Security Administration: Managing Authentication and Authorization [page 2537] or Role Collections and Roles in Global Accounts, Directories, and Subaccounts [page 138].

Context

> **Note**
>
> When handling personal data, consider the legislation in the various countries and regions where your organization operates. After the data has passed the end of purpose, regulations might require you to delete the data. For more information on data protection and privacy, see the related link.

The User Account and Authentication service stores user-related data records in the form of shadow users. The UAA uses the information of the shadow users to issue tokens that refer to the specific user. If automatic shadow user creation is enabled, the UAA creates the shadow users when the user authenticates. Otherwise, the UAA creates the shadow user as soon as you assign the user a role collection. These conditions apply to platform users and business users. For more information about shadow users, see the Cloud Foundry documentation.

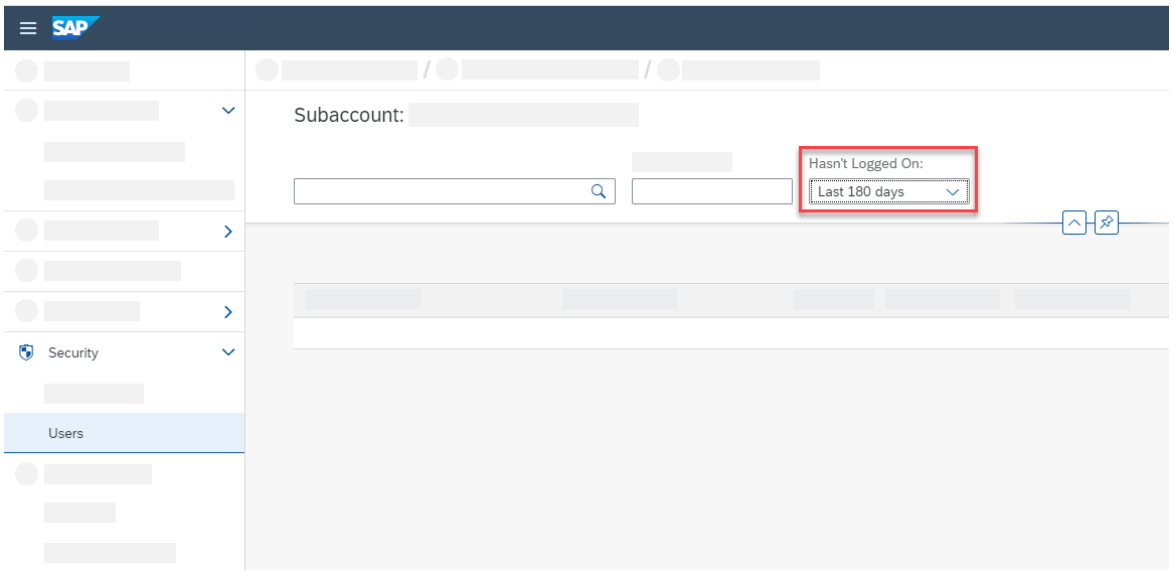
> **Note**
>
> You can also delete users using the User Management (SCIM) API.
>
> For more information, see Delete Shadow Users for Data Protection and Privacy Using APIs [page 3642].

Procedure

1. Open the SAP BTP cockpit.
2. Go to your global account, directory, or subaccount (see Navigate in the Cockpit [page 2461]).
3. Choose *Security* > *Users*.

   > **Note**
   >
   > For directories, choose *Users*. There is no *Security* menu item.

4. From the list of users, select the users to delete.



Filter for Users Who Haven't Logged On

> **→ Recommendation**
>
> Users, who have left your organization, haven't logged on recently. Use the *Hasn't Logged On* filter option to find users who haven't logged on in the last 180 days.

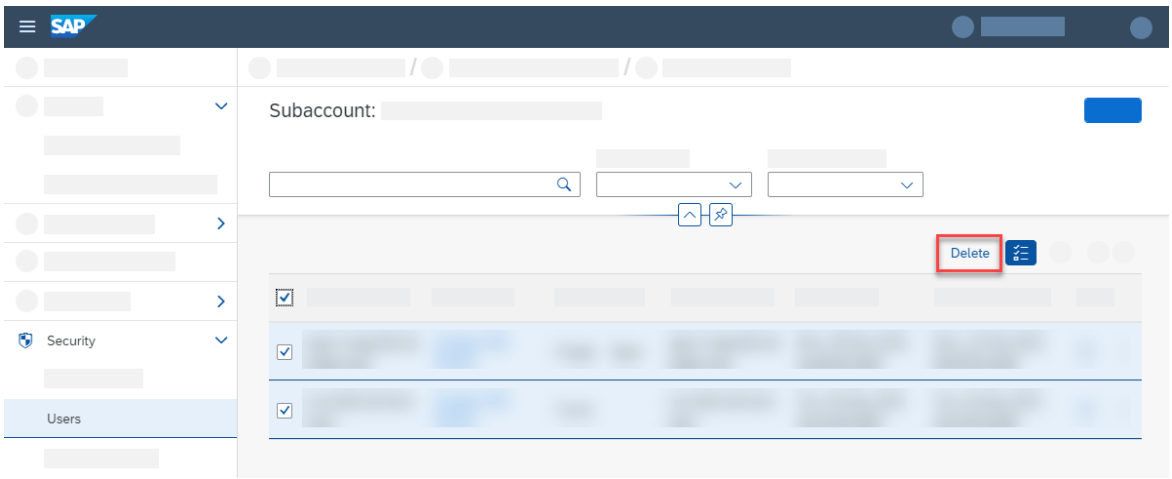
The last logon date you filter for varies from situation to situation. The filter value must meet the following requirements:

- Be long enough that the filter doesn't catch users who are simply on vacation or don't work regularly in the system.
- Not be so long that the filter no longer meets your data protection and privacy requirements.

Think about users on parental leave or on a longer leave of absence, too.

> ⚠ Caution
>
> You can't undo the deletion of a user. When you delete a user, you delete any direct assignments of role collections to that user. You also potentially invalidate any application referencing that user.
>
> For example, an application saves some data relating to a user. When you delete the user, that data points to an ID that no longer exists. You can recreate the user, but the new user has a different ID, even though other attributes, such as e-mail, first name, and last name, are identical.

5. Delete the selected users.



Delete Selected Users

Related Information

Delete Users [page 2611]
Data Protection and Privacy [page 3633]
Switch Off Automatic Creation of Shadow Users [page 2600]
Shadow Users in the Cloud Foundry Documentation

7.5.4.2 Delete Shadow Users for Data Protection and Privacy Using APIs

Data privacy regulations or policies may require you to delete this data, for example, when the user has left your organization. To delete shadow users using APIs, set up access to the API and then use the SCIM REST APIs to retrieve and delete the users.

Prerequisites

The security administrator must have the following scopes:

- `xs_user.read`
- `xs_user.write`

Context

> ### Note
>
> When handling personal data, consider the legislation in the various countries and regions where your organization operates. After the data has passed the end of purpose, regulations might require you to delete the data. For more information on data protection and privacy, see the related link.

The User Account and Authentication service stores user-related data records in the form of shadow users. The UAA uses the information of the shadow users to issue tokens that refer to the specific user. If automatic shadow user creation is enabled, the UAA creates the shadow users when the user authenticates. Otherwise, the UAA creates the shadow user as soon as you assign the user a role collection. These conditions apply to platform users and business users. For more information about shadow users, see the Cloud Foundry documentation.

> ### Note
>
> Administrators can also delete users using the SAP BTP cockpit. For more information, see Delete Users [page 2611].

Procedure

1. Enable API access to your subaccount.

   For more information, see Get Access to the APIs [page 2774].
2. Use the *GET* method for the User Management (SCIM) API of the SAP Authorization and Trust Management service to get a list of users.

For more information about the User Management (SCIM) API, see https://api.sap.com/package/authtrustmgmnt on SAP Business Accelerator Hub.

The API returns the list of users in JSON format.

3. Use your own tools to sort and identify the users in the JSON response to delete.

> → Recommendation
>
> Users, who have left your organization, haven't logged on recently. Use the `lastLogonTime` parameter to find users who haven't logged on in the last 180 days. The `lastLogonTime` parameter is in UNIX time.
>
> The last logon time you filter for varies from situation to situation. The filter value must meet the following requirements:
>
> - Be long enough that the filter doesn't catch users who are simply on vacation or don't work regularly in the system.
> - Not be so long that the filter no longer meets your data protection and privacy requirements.
>
> Account for users on parental leave or on a longer leave of absence, too.

> ⚠ Caution
>
> You can't undo the deletion of a user. When you delete a user, you delete any direct assignments of role collections to that user. You also potentially invalidate any application referencing that user.
>
> For example, an application saves some data relating to a user. When you delete the user, that data points to an ID that no longer exists. You can recreate the user, but the new user has a different ID, even though other attributes, such as e-mail, first name, and last name, are identical.

4. Use the *DELETE* method and the list of user IDs to delete the shadow users one at a time.

   For each user, call the `Users` endpoint with the delete method and include the user ID in the path. For example:

```
curl --location --request
DELETE 'https://api.authentication.eu20.hana.ondemand.com/Users/a0a67e6f-
c4b5-41e5-b871-6fa181d46599' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--header 'Accept: application/json' \
--header 'Authorization: bearer eyJqdGkiOiJmZmZl…'
```

   For more information about the User Management (SCIM) API, see https://api.sap.com/package/authtrustmgmnt on SAP Business Accelerator Hub.

Related Information

Data Protection and Privacy [page 3633]
https://docs.cloudfoundry.org/uaa/uaa-concepts.html#%23shadow
Switch Off Automatic Creation of Shadow Users [page 2600]
Delete Shadow Users for Data Protection and Privacy Using the Cockpit [page 3639]

7.5.5 Consent

SAP BTP supports you in collecting and managing the consent of data subjects in the following ways:

SAP Cloud Identity Services provides tools to manage privacy policies and terms of use agreements.

For more information, see Configuring Privacy Policies and Configuring Terms of Use in the SAP Cloud Identity Services documentation.

See also Consent in the SAP Cloud Identity Services documentation.

7.6 Security in the Kyma Environment

Security in SAP BTP, Kyma runtime comprises the following pillars:

- Kyma Security Concepts [page 3644]
- Secure Administration and Operations in the Kyma Environment [page 3658]
- Secure Development in the Kyma Environment [page 3651]

Go to the respective topics to learn more about the security features and configurations they propose.

7.6.1 Kyma Security Concepts

SAP BTP, Kyma runtime takes various security measures regarding your cluster and its underlying infrastructure. Benefit from the security setup that SAP BTP, Kyma runtime provides.

Cloud Infrastructure

All components required to run Kyma, namely Gardener, Kyma Control Plane, and a Kyma cluster, are deployed on SAP-managed IaaS provider accounts. SAP has strict guidelines to secure IaaS provider accounts and the resources deployed into them. The secure configuration is monitored regularly, and the following preventive controls are in place to revert critical configurations that weaken the security of the IaaS provider account:

Authentication

- A strict password policy is in place.
- Multi-factor authentication is enabled.

Authorization

- Permissions are assigned based on role-based access control (RBAC).
- The principle of least privilege is followed.

Storage

- Storage services are not publicly accessible.
- Encryption using Advanced Encryption Standard (AES) with a key length of at least 256 is used.

Kubernetes Control Plane

Kyma uses Gardener as its managed Kubernetes offering. With Gardener, the control plane of a Kubernetes cluster is hosted as a set of Pods in the Gardener environment. See the overview of the Gardener architecture . In order to harden the Kubernetes control plane as well as the Kubernetes components on the worker nodes, Gardener sets up the cluster in accordance with the Defense Information Systems Agency (DISA) Security Technical Implementation Guides (STIGs) requirements (V1R6).

The Kubernetes control plane fulfills the following DISA STIGs requirements:

General

- A kubectl version higher than 1.12.9 is used on the control plane (V-242396).
- The Kubernetes DynamicAuditing (V-242398) and DynamicKubeletConfig (V-242399) are disabled.
- Kubernetes manifests (V-242405), component manifests (V-242444), configuration files (V-242446), component PKI (V-242451), and `kubeadm.conf` (V-242454) are owned by root.
- The Kubernetes manifest files have restrictive file permissions following the least privileges principle (V-242408).
- Kubernetes doesn't store Secrets as environment variables (V-242415).
- The role-based access control (RBAC) is activated to control access (V-242435).
- Kubernetes is kept up to date with the latest patches, service packs, and hot fixes. A patch management process is in place (V-242443).
- The Kubernetes `kubeadm.conf` (V-242455), `admin.conf` (V-242460), and PKI CRT (V-242466) have restrictive file permissions.
- Access to the directory containing the Kubernetes PKI files is restricted (V-242467).
- Kubernetes endpoints use the approved organizational certificate and key pair to protect information in transit (V-245544).

API Server

The Kubernetes kube-apiserver fulfills the following DISA STIGs requirements:

- TLS 1.2, at a minimum, is used to protect the confidentiality of sensitive data during electronic dissemination (V-242378).
- The Alpha APIs are disabled (V-242400).
- The insecure bind address (V-242388) and insecure port (V-242386) are not set, and a secure API server port is configured (V-242389).
- The approved SSL Certificate Authority is set (V-242419).
- Only ports, protocols, and services (PPS) that adhere to the Ports, Protocols, and Services Management Category Assurance List (PPSM CAL) are configured (V-242410).
- Approved cipher suites are used (V-242418).
- A certificate for communication is used (V-242422).

- The `ValidatingAdmissionWebhook` is enabled (V-242436).
- Appropriate timeouts are set to limit attack surface (V-242438).
- Restrictive file permissions for the configuration files of the Control Plane services are used (V-242458).
- Only secure encryption protocols are used. SSL, TLS 1.0, and 1.1 are prohibited. (V-242468).

Kubernetes Scheduler

The Kubernetes scheduler, namely, kube-scheduler fulfills the following DISA STIGs requirements:

- TLS 1.2, at a minimum, is used to protect the confidentiality of sensitive data during electronic dissemination (V-242377).
- A secure binding to localhost (127.0.0.1) is used (V-242384).
- Only ports, protocols, and services (PPS) that adhere to the Ports, Protocols, and Services Management Category Assurance List (PPSM CAL) are configured (V-242411).

Controller Manager

The Kubernetes Controller Manager fulfills the following DISA STIGs requirements:

- TLS 1.2, at a minimum, is used to protect the confidentiality of sensitive data during electronic dissemination (V-242376).
- Unique service accounts for each work payload are used (V-242381).
- A secure binding to localhost (127.0.0.1) is used (V-242385).
- Profiling is disabled (V-242409).
- The SSL Certificate Authority is set (V-242421).
- Only ports, protocols, and services (PPS) that adhere to the Ports, Protocols, and Services Management Category Assurance List (PPSM CAL) are configured (V-242412).

etcd

The Kubernetes etcd fulfills the following DISA STIGs requirements:

- The usage of self-signed certificates for client-to-server and server-to-server communication is prohibited (V-242379, V-242380).
- Only ports, protocols, and services (PPS) that adhere to the Ports, Protocols, and Services Management Category Assurance List (PPSM CAL) are configured (V-242413).
- Certificate-base client authentication is enabled (V-242423), (V-242426).
- TLS encrypted client-to-server and server-to-server communication is configured (V-242427, V-242432, V-242433). The communication between the API server and etcd is encrypted with TLS (V-242431).
- Approved and valid certificates for the TLS encrypted connection between the API server and etcd are used (V-242428, V-242430). An SSL certificate authority is configured (V-242429).
- Files are owned by etcd:etcd (V-242445).
- Files have restrictive file permissions set (V-242459).

Kubernetes Worker Nodes

Kubernetes worker nodes use a kubectl version higher than 1.12.9 (V-242396).

Node Operating System

SAP BTP, Kyma runtime uses Garden Linux as its node operating system. Garden Linux is a Linux distribution with a minimal set of applications optimized for use in containerized environments.

The node operating system in Kyma has the following qualities:

- Node operating system images are secured and audited.
- Malware scans and vulnerability scans are performed as part of the release process of the node operating system images. For more information, see Security Vulnerability Management in the Kyma Environment [page 3649].
- Remote access to nodes (for example, SSH) is not permitted, and nodes are not exposed to the Internet.

kubelet

The Kubernetes kubelet fulfills the following DISA STIGs requirements:

- The read-only port flag (V-242387) and anonymous authentication (V-242391) are disabled.
- Explicit authorization is required (V-242392).
- Hostname override is denied (V-242404).
- An SSL certificate authority is configured (V-242420).
- TLS is enabled to enforce TLS encrypted communication sessions (V-242424, V-242425).
- Kernel protection is set (V-242434).
- A connection timeout is set (V-245541).
- No staticPodPath is configured (V-242397).
- The Kubernetes kubelet configuration file (V-242406), certificate authority (V-242450), and config (V-242453, V-242457) are owned by root.
- The Kubernetes kubelet configuration files (V-242407), certificate authority file (V-242449), config file (V-242452, V-242456) have restrictive file permissions set (V-242407).

kube-proxy

The Kubernetes kube-proxy fulfills the following DISA STIGs requirements:

- A kubeconfig file is not used (V-242447, V-242448).

Network Separation

Namespaces

Kyma components are deployed into the `kyma-system` and the `istio-system` namespaces. The default namespace is not used.

Network Policies

You, the customer, have full access to the cluster and can establish appropriate network policies to restrict network traffic. Because Istio is a default Kyma module, the customer can also establish PeerAuthentication policies to control the network traffic.

Encryption

All communication in the Kyma cluster is encrypted using TLS 1.2.

Authentication and Authorization

Authentication at the API Server

By default, a SAP BTP, Kyma runtime cluster is deployed with a shared tenant of SAP Cloud Identity Services provided by SAP. This identity provider (IdP) uses OIDC tokens to authenticate and is configured to enforce multi-factor authentication.

As a best practice, you, the customer, can change the identity provider to one controlled by you. This gives you control over the identity lifecycle of your accounts. For details, see Configuring a Custom Identity Provider for Kyma [page 3660].

The Kubernetes API Server fulfills the following DISA STIGs requirements:

- Anonymous authentication is disabled (V-242390).
- Basic authentication is disabled to protect information in transit (V-245542).
- Token authentication is disabled to protect information in transit (V-245543).

Role-Based Access Control

RBAC is enabled on Kyma clusters. The Roles and ClusterRoles deployed by Kyma follow the principle of least privilege.

Following DISA STIGs requirements, the Kubernetes API server enables RBAC as the authorization mode (V-242382).

High Availability

Kyma worker nodes are distributed over several availability zones in order to prevent outages of the cloud providers' availability zones.

Logging

Kubernetes-Native Audit Logging Configuration

Audit Logging of the Kubernetes API server is active, and the logs are written to SAP Platform Logging Service for SAP BTP.

The Kubernetes API server fulfills the following DISA STIGs requirements:

- An audit policy (V-242401), an audit log path (V-242402, V-242465), and audit log retention (V-242464) are set.
- Audit records that provide information about the type of event that has occurred, the source of the event, the results of the event, any users and containers associated with the event are generated (V-242403).
- Audit logs are enabled (V-242461).
- Audit log maximum size (V-242462), and maximum backup are configured (V-242463).

Container Logging

Kyma modules ship logs that contain information relevant to the containers.

Related Information

Security Vulnerability Management in the Kyma Environment [page 3649]
Distributed Denial-of-Service Protection in Kyma [page 3649]
Secure Administration and Operations in the Kyma Environment [page 3658]
Secure Development in the Kyma Environment [page 3651]

7.6.1.1 Security Vulnerability Management in the Kyma Environment

SAP employs a vulnerability management process to ensure the security of SAP BTP, Kyma runtime. Within this process, we identify, assess, prioritize, remedy, and monitor vulnerabilities. We also provide hotfixes between regular releases for critical vulnerabilities that our security organization identifies.

SAP regularly scans SAP-owned code, the container images, and the node operating system images that are part of SAP BTP, Kyma runtime, Kyma Control Plane, and Kyma dashboard for known and potential vulnerabilities. The following security scanning technologies are used:

- Static application security testing (SAST) for the proprietary code
- Open source vulnerability management (OSVM) for open-source software that is part of Kyma runtime

> ⓘ Note
>
> SAP runs security scans for Kyma runtime, and you, as a customer, are responsible for scanning your own workloads added to the Kyma cluster. For details, see Operating Model in the Kyma Environment. [page 3692]

The scanning results are constantly monitored and addressed. Vulnerabilities are handled with software updates. If an update is not available, the vulnerability is subject to further analysis to identify its associated risk and implement appropriate measures. For critical vulnerabilities identified by SAP's security organization, hotfixes are applied between regular releases.

> → Remember
>
> Kyma development teams analyze each vulnerability to assess its actual severity in the Kyma environment. Therefore, the severity score may be lowered or identified as false positive.

7.6.1.2 Distributed Denial-of-Service Protection in Kyma

A distributed denial-of-service (DDoS) attack is a malicious attempt to disrupt the normal traffic of a targeted server, service or network by overwhelming the target or its surrounding infrastructure with a flood of traffic.

With Kyma comes the default DDoS protection offered by the IaaS provider.

Default DDoS Protection

IaaS Provider	Default DDoS Protection
Microsoft Azure	Azure DDoS Protection Basic service
Amazon Web Services	AWS Shield Standard
Google Cloud	Standard network DDoS protection

> ⚠ Caution
>
> Because these are cloud providers' default offerings, they are subject to change as per the cloud providers' terms and conditions.

Web Application Firewalls

It is not possible to configure or enable web application firewalls in SAP BTP, Kyma runtime.

7.6.1.3 SAP's Access to Your Kyma Environment

SAP BTP, Kyma runtime provides reliable, secure infrastructure management while ensuring you maintain full control over your applications.

As a managed service, SAP BTP, Kyma runtime maintains a clear separation of responsibilities: SAP acts as the platform operator, and customers as the application owners. Customers retain full control over their applications and workloads, while SAP manages the underlying infrastructure, to ensure continuous platform stability. For more information, see Operating Model in the Kyma Environment. This model requires SAP to occasionally access your cluster for operational purposes.

SAP can access your environment exclusively for the following operational tasks:

- Maintenance activities: routine system upkeep and optimization
- Upgrades: platform version updates and feature deployments
- Patching: security fixes and vulnerability remediation
- Troubleshooting: investigation and resolution of platform-level issues

Access is carefully controlled, time-limited, and fully auditable to maintain security and compliance standards. These standards are outlined by the International Organization for Standardization (ISO), the Security Operations Center (SOC), and the General Data Protection Regulation (GDPR).

7.6.1.4 Payment Card Industry Data Security Standard in the Kyma Environment

Payment Card Industry Data Security Standard (PCI DSS) ensures businesses protect cardholder data, and SAP BTP, Kyma runtime's compliance attestation confirms its security measures.

PCI DSS is a set of rules and requirements for businesses that store, process, or transmit payment card information, designed to protect cardholder data from fraud. SAP BTP, Kyma runtime holds the PCI DSS

Attestation of Compliance (AoC) v4.0.1. The AoC defines how Kyma is compliant with the PCI standard. To request a copy of SAP Business Technology Platform Kyma Runtime 2025 PCI DSS AoC Report, visit SAP Trust Center.

Responsibility Matrix

Kyma, as a PCI service provider, defines the scope of responsibilities. Some responsibilities belong to Kyma, some to you as a customer, and some are shared.

To view the Responsibility Matrix, request access using the support procedure. To get support, report a case in SAP for Me with the `BC-CP-XF-KYMA` Kyma support component. For more information, see Getting Support.

Registering Your Kyma Cluster for PCI DSS

To fulfill its part of the shared responsibilities, the Kyma team must be able to identify which Kyma clusters are in scope for PCI DSS.

To register your Kyma cluster for PCI DSS, report a case in SAP for Me with the component `BC-CP-XF-KYMA`, providing the following details:

- Your subaccount ID
- Your Kyma environment instance ID or APIServerURL
- Desired registration date

You can find the subaccount ID, environment instance ID, and APIServerURL in your subaccount *Overview*.

To withdraw your Kyma cluster from PCI DSS, report another case in SAP for Me. To learn how to report a case, see Getting Support.

7.6.2 Secure Development in the Kyma Environment

Secure development focuses on Pod security, network traffic restriction, Istio sidecar proxy injection, and workload exposure. Learn about the security measures you can take to improve the security of your Kyma environment .

Pod Security

To learn how to secure your Kubernetes Pods in Kyma see Kubernetes Pod Security Recommendations [page 3652].

Restrict and Secure Network Traffic

- Set up Kubernetes Network Policies to restrict the network traffic between Pods in your namespaces.
- Enabling Istio Sidecar Proxy Injection [page 1892] for your namespaces.

Expose Workloads Securely

To securely expose workloads in SAP BTP, Kyma runtime, use the Istio and API Gateway modules. As a prerequisite, make sure that you have those two modules added to your Kyma cluster. Then, perform the following tasks:

- Set up a custom domain for a workload. See Istio Gateways [page 1946].
- Disable the default gateway. See Kyma Gateway [page 1969].
- Set up your own API gateway with one of the following options:
  - Configure a TLS Gateway [page 1948]
  - Configure an mTLS Gateway [page 1956]
- Create an APIRule Custom Resource to securely expose your workloads. See Expose and Secure a Workload with a JWT Using SAP Cloud Identity Services [page 1975].

Module-Specific Security Considerations

Serverless Module

For more information, see Function Security for the Serveless module.

Related Information

Kyma Security Concepts [page 3644]
Secure Administration and Operations in the Kyma Environment [page 3658]
Kubernetes Pod Security Recommendations [page 3652]

7.6.2.1 Kubernetes Pod Security Recommendations

Pods are the smallest deployable units of computing that you can create and manage in Kubernetes. This document shows how to use Pods securely and ultimately have a secure workload. The following steps guide you to use Pods securely and ultimately have a secure workload, depending on your needs.

1. Decide on the Basic Security Settings

Without Special Requirements: Highest Security

If you are not aware of any special requirements, set up your workload in the most secure way:

- Specify a user for the Pod and container. If your application requires a specific user, change values for the `runAsUser` and `runAsGroup` parameters.
- Run an unprivileged workload.
- Drop all capabilities.
- Use seccomp. For more information, see the Restrict a Container's Syscalls with seccomp .
- Run with a read-only root file system.

Use the following `securityContext` configuration for the Pod and container.

Sample Code

```
# Pod spec
  securityContext:
    runAsUser: 10001
    runAsGroup: 30001
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
# Container spec
      securityContext:
        privileged: false
        allowPrivilegeEscalation: false
        runAsNonRoot: true
        capabilities:
          drop:
            - ALL
        readOnlyRootFilesystem: true
```

If Your Container Must Run as User Root

If your container must run as user root, use the following setup:

- Do not use the `runAsUser` and `runAsGroup` parameters in the spec or set them to 0.
- Do not use the `runAsNonRoot` parameter.
- Run an unprivileged workload and do not allow privilege escalation.
- Drop all capabilities.
- Use seccomp.
- Run with a read-only root file system.

Use the following `securityContext` configuration for the Pod and container.

Sample Code

```
# Pod spec
  securityContext:
    seccompProfile:
      type: RuntimeDefault
# Container spec
      securityContext:
        privileged: false
        allowPrivilegeEscalation: false
```

```
        readOnlyRootFilesystem: true
        capabilities:
          drop:
            - ALL
```

2. Enable Additional Capabilities

Add Storage to Your Pod

If you need storage, follow these recommendations:

- Always use volumes instead of local storage in the container.
- Avoid using `hostPath` volumes. If you must use them, follow these rules:
  - **Whenever possible** mount `hostPath` volumes in the read-only mode.
  - **Never** mount the following sensitive paths:
    - `/proc` - a pseudo-file system that provides an interface to kernel data structures
    - `/` - file system's root
    - `/etc` - the directory that usually contains all system-related configuration files
    - `/root` - the home directory of the root user
    - `/var/run/docker.sock` - the Unix socket used to communicate with the Docker daemon
    - `/var/run/crio/crio.sock` - the Unix socket used to communicate with the CRI-O Container Engine
    - `/run/containerd/containerd.sock` - the Unix socket used to communicate with the Containerd container runtime
    - `/home/admin` - the home directory of the admin user
    - `/var/lib/kubelet` - the directory for kubelet-related configuration
    - `/var/lib/kubelet/pki` - the directory containing the certificate and private key of the kubelet
    - `/etc/kubernetes` - the directory containing Kubernetes-related configuration
    - `/etc/kubernetes/manifests`
  - Use only the `File` or `Directory` `hostPath.type`. Usually use `File`, and `Directory` as a fallback.
  - Some files and directories on the host can only be accessible by root. Read more in the Kubernetes documentation .

  Consider the following template for mounting `hostPath` volumes in order to create Kubernetes manifests:

  Sample Code

```
# Pod spec
  volumes:
    - name: <name>
      hostPath:
        path: <file or directory on the node>
        type: <File> or <Directory>
# Container spec
    volumeMounts:
      - mountPath: <path where hostpath has to be mounted>
        name: <name>
        readOnly: true
```

- Use `emptyDir` volumes for temporary file systems.
  If your application needs a temporary file system, for example, for caching purposes, use `emptyDir` volumes **instead of** mounting `hostPath` or making the root file system writable. Also, remember to always define a size limit for the volume. Use the following template:

  Sample Code

  ```
  # Pod spec
    volumes:
    - name: <name>
      emptyDir:
        sizeLimit: <size of the volume>
  # Container spec
      volumeMounts:
      - mountPath: <path where the file system has to be mounted>
        name: <name>
  ```

Add Other Capabilities

- Before you add new cabilities to your Pod or container configuration, **always drop all** capabilities. Each container runs with a fixed set of default capabilities. You can find the containerd v1.7 default capabilities in this code .
- Add only the capabilities you really need.
- If possible, avoid the following capabilities that can be exploited to escalate privileges:
  - CAP_CHOWN
  - CAP_DAC_OVERRIDE
  - CAP_DAC_READ_SEARCH
  - CAP_SETUID, CAP_SETGID
  - CAP_NET_RAW
  - CAP_SYS_ADMIN
  - CAP_SYS_PTRACE
  - CAP_SYS_MODULE
  - CAP_FORMER
  - CAP_SETFCAP

If your application doesn't work as expected, you may need some additional capabilities. Find the full list of the available capabilities on the Linux manual page . Check which capabilities are missing using the Inspector Gadget tools. You can install the toolset as a kubectl plugin using krew . After it's installed, use the `kubectl gadget trace capabilities` command.

3. Consider Special Features

Accessing Parts of the `/proc` File System

The `proc` file system provides a lot of critical information about the system and its running processes. To learn more, read Exploring the Linux `/proc` file system and Important Linux `/proc` file system files you need to know . By default, container runtimes mask certain parts of the `/proc` file system from inside a container in order to prevent potential security issues.

> ⚠ Caution
>
> You should change settings around the `/proc` file system only if you are aware of the impact. Changing the configuration may expose your system to security issues.

- Check if you really need to use the `/proc` file system. For example, if you want to use it for image building purposes, check the latest version of your build tool because many of them no longer need it.
- If you really need to use the `/proc` file system, only do so for a nested container. Never expose the `/proc` file system of your host system to a container.
- If your application needs access to parts of the `/proc` file system, you can set the `procMount` field in the `PodSpec` to unmasked to remove the restriction on the visibility of the `/proc` file system from inside the container.

Seccomp

Seccomp is a Linux kernel feature that allows filtering system calls made by a process. It is used to restrict the set of system calls that a process can make.

If, in `SecurityContext`, the `seccompProfile` is set to RuntimeDefault, the container uses the default seccomp profile provided by the container runtime.

Read more to learn about:

- Allowed system calls in the default profile of containerd.
- The full list of Linux system calls .
- How to work with Seccomp profiles in Kubernetes - Restrict a Container's Syscalls with seccomp .
- How to se tup a Custom Seccomp Profile in Gardener.

Host Namespaces

Host namespaces (Process ID namespace, Inter-Process Communication namespace, and network namespace) allow you to access shared information. You can also use host namespaces to elevate privileges. By removing namespaces from Pods, you reduce isolation and circumvent the protection models that containers are based on. It allows the processes in the Pod to perform tasks as if they were running natively on the host.

In the Pod spec, the following fields represent the host namespaces:

- `hostPID`: Process ID namespace
- `hostIPC`: Inter-Process Communication namespace
- `hostNetwork`: network namespace

The three fields are by default set to `false` and should remain so.

Usually, there is no need to remove this isolation and therefore, Pods should not have access to host namespaces.

> ⚠ Caution
>
> Removing namespaces from Pods is a very advanced feature. Use it only if you are certain you need it, for example, for low-level observation of other containers.

7.6.2.2 Function Security

To eliminate potential security risks when using Functions, bear in mind these facts:

- Kyma Serverless applies a strict securityContext configuration at the Pod and container level (non-root execution, minimal capabilities, and other hardening defaults), allowing Functions to run in namespaces where the restricted Pod security level is enforced. You can configure Functions' security context using `.spec.podSecurityContext` and `.spec.containerSecurityContext` fields. For more details, see Custom Resource Parameters .

  > ⚠ Caution
  >
  > Modifying the default security context can make Functions insecure. Use caution when changing `.spec.podSecurityContext` and `.spec.containerSecurityContext`, and review the Pod Security Standards , especially before deploying to production.

- By default, JSON Web Tokens (JWTs) issued by an OpenID Connect-compliant identity provider do not provide the **scope** parameter for Functions. This means that if you expose your Function and secure it with a JWT, you can use the token to validate access to all Functions within the cluster as well as other JWT-protected services.
- Kyma provides base images for serverless runtimes. Those default runtimes are maintained with regard to commonly known security advisories. It is possible to use a custom runtime image. For more information, see Override Runtime Image . In such a case, you are responsible for security compliance and assessment of exploitability of any potential vulnerabilities of the custom runtime image. Additionally, your custom runtime image must support running as a non-root user under the restricted Pod security level, for example, no dependency on root ownership of writable paths, privileged operations, or added capabilities.
- Kyma does not run any security scans against Functions and their images. Before you store any sensitive data in Functions, consider the potential risk of data leakage.
- Kyma does not define any authorization policies that would restrict Functions' access to other resources within the namespace. If you deploy a Function in a given namespace, it can freely access all events and APIs of services within this namespace.
- All administrators and regular users who have access to a specific namespace in a cluster can also access the source code of all Functions within this namespace.

7.6.3 Secure Administration and Operations in the Kyma Environment

Secure administration and operations focus on landscape setup, authentication, authorization, and Istio access logs. Learn about the security measures you can take to improve the security of your Kyma environment.

Landscape Setup

While creating a staged development environment is a good idea in any case, there are some considerations specific to Kyma you might want to take into account. For more information, see Cloud Foundry, Kyma, or Both?.

Authentication

SAP BTP, Kyma runtime uses OpenID Connect for authentication. Kyma runtime is configured to use a default shared SAP Cloud Identity Services tenant, SAP ID service, as an identity provider. This is a good starting point for development and testing purposes. However, for production scenarios, it's recommended to get your own SAP Cloud Identity Services tenant. For more information, see Authentication in the Kyma Environment [page 3659].

Authorization

Kyma uses Kubernetes role-based access control (RBAC) to define what resources users can access and what actions they can perform. When you create a Kyma cluster, your user account is automatically bound to the `cluster-admin` role. As a cluster admin, you can assign roles to other users and groups. For recommendations on setting up roles and permissions in Kyma, see Authorization in the Kyma Environment [page 3663].

For more information about user and member management in the SAP BTP cockpit, see User and Member Management [page 133].

Istio Access Logs

Collecting Istio access logs can help indicate the core concepts of monitoring (latency, traffic, errors, and saturation) and capture critical aspects of system behavior. You monitor and analyze the traffic in your Kyma cluster (see Configure Observability for the Istio Service Mesh [page 1917]) and then send the logs to a central platform, for example, a Security Information and Event Management (SIEM) system, for threat detection and incident response.

Related Information

Kyma Security Concepts [page 3644]
Secure Development in the Kyma Environment [page 3651]
Authentication in the Kyma Environment [page 3659]
Configuring a Custom Identity Provider for Kyma [page 3660]
Assign Roles in the Kyma Environment [page 3666]
Auditing and Logging Information in Kyma [page 3678]
ClusterRole Aggregation in Kyma [page 3664]

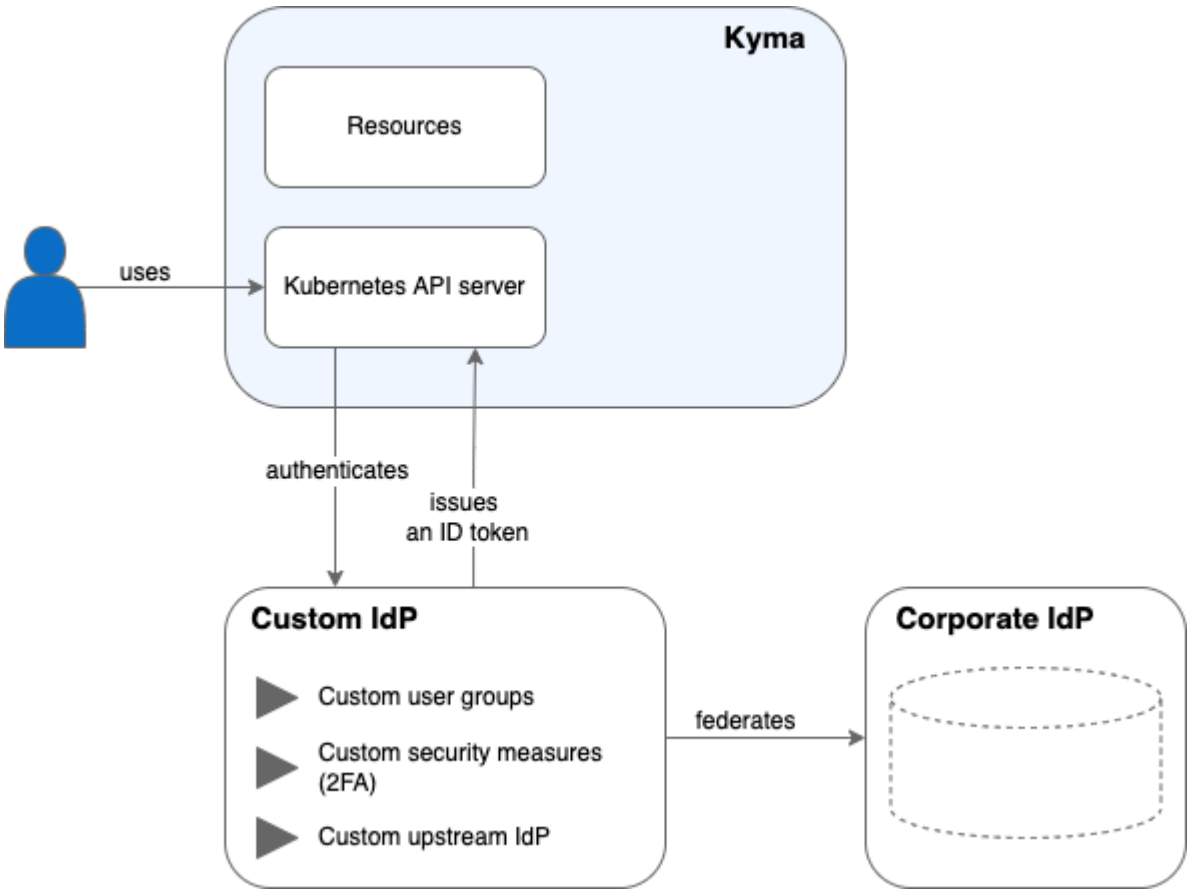
7.6.3.1 Authentication in the Kyma Environment

To authenticate in the Kyma environment, you can either use the default identity provider (IdP) or set up a custom identity provider.

Default vs. Custom Identity Provider

SAP BTP, Kyma runtime uses OpenID Connect for authentication. Kyma runtime is configured with a default identity provider: a shared SAP Cloud Identity Services tenant. This is a good starting point for development and testing purposes. For production scenarios, it's recommended that you set up your own SAP Cloud Identity Services tenant which provides the following features:

- Manages users
- Manages user groups and binds cluster roles to the user groups instead of individual usernames in your Kyma runtime
- Configures your own risk-based security measures, such as two-factor authentication, IP ranges, etc.
- Delegates authentication to an upstream corporate identity provider to prevent introducing redundant identities and to benefit from single sign-on, based on identity provided "centrally"



Learn how to Get Your Tenant from SAP Cloud Identity Services.

Alternatively, you can choose any other service compliant with OpenID Connect.

Having an SAP Cloud Identity Services tenant, you can configure it as the custom identity provider of your Kyma cluster. See Configuring a Custom Identity Provider for Kyma [page 3660].

7.6.3.1.1 Configuring a Custom Identity Provider for Kyma

Enable the Kyma environment with a custom identity provider (IdP).

Prerequisites

- If you choose to use SAP Cloud Identity Services as a custom identity provider, you have configured your tenant as an OpenID Connect (OIDC) provider for your Kyma cluster. For details, see Configure OpenID Connect Application for Authorization Code Flow.

  > → Tip
  >
  > When using SAP Cloud Identity Services as a custom identity provider, make sure you have public client flows enabled.

- You have the Kyma dashboard URL (`https://dashboard.kyma.cloud.sap`) and the localhost for kubectl authentication (`http://localhost:8000`) configured as allowed callback URLs at your identity provider, so that authenticated users can be redirected back to the Kyma application. See Redirect URIs, Post Logout Redirect URI Rules.

> → Tip
>
> It's recommended to use SAP Cloud Identity Services tenant as a custom identity provider. It allows you to use SAP Cloud Identity Services as a proxy to integrate your corporate identity provider. See Get Your Tenant. You could also configure SAP Cloud Identity Services to use a third-party identity provider if you already have one.

Context

When you create a new Kyma instance in the SAP BTP cockpit from the Service Marketplace, you can configure your custom OpenID Connect identity provider to authenticate users in your Kyma runtime.

If you've already created your Kyma environment, you can also apply the custom identity provider configuration and set up administrators during your Kyma instance update operation by providing the details as an array of strings in the respective fields.

Procedure

1. Go to *Services* > *Service Marketplace* and create a Kyma instance. For additional information, see Creating Kyma Instances [page 3452].
2. In the *Additional Parameters* view, go to *Oidc:*, choose *List*, and fill in the following fields:
   - *Client ID* - the client ID for the OpenID client
   - *Groups Claim* - the name of a custom OpenID Connect claim for specifying user groups

     > Example
     >
     > An identity provider token consists of fields (claims) that are included in it.
     >
     > ```
     > {
     > sub: "john.doe@sap.com"
     > groups: ["SAP fans", "Identity Provider", "OpenID"]
     > }
     > ```
     >
     > When you configure *Username Claim*, Kubernetes knows which field (claim) includes the user represented by a given token.
     >
     > Similarly, when you configure *Groups Claim*, Kubernetes knows to which groups the user is assigned.

   - *Issuer URL* - the URL of the OpenID issuer (use the `https` schema)
   - *Signing Algs* - the list of allowed cryptographic algorithms used for token signing. The allowed values are defined by RFC 7518 .

- *Username Claim* - the name of a custom OpenID Connect claim for specifying a username
- *Username Prefix* - the prefix for an OIDC username for Kyma runtime. If you don't provide it, username claims other than “email” are prefixed by the *issuerURL* to avoid clashes. To skip any prefixing, provide the value as `-`.
- *Groups Prefix* - if specified, causes claims mapping to group names to be prefixed with the provided value. To skip any prefixing, provide the value as `-`.
- *Required Claims* - describes a required claim in the ID Token. If set, the claim is verified to be present in the ID Token with the matching value.
- *Administrators* - the list of usernames meant to be administrators. Users identified with those usernames get the role *cluster-admin* during provisioning of your Kyma runtime.

You can also provide the configuration as a JSON object:

```
"oidc": {
  "list": [
    {
    "issuerURL": "{issuerURL}",
    "clientID": "{clientID}",
    "usernameClaim": "sub",
    "groupsClaim": "groups",
    "signingAlgs": ["RS256"],
    "usernamePrefix": "-",
    "groupsPrefix": "-",
    "requiredClaims": ["first-claim=value", "second-claim=value"]
  },
]
"administrators": [
  "example_1@mail.com",
  "example_2@mail.com"
  ]
}
```

> ### Note
>
> If you want to revert to the default settings, use the following configuration:
>
> ```
> "oidc": {
>   "list": [
>    {
>     "clientID": "12b13a26-d993-4d0c-aa08-5f5852bbdff6",
>     "groupsClaim": "groups",
>     "issuerURL": "https://kyma.accounts.ondemand.com",
>     "groupsPrefix": "-"
>     "signingAlgs": ["RS256"],
>     "usernameClaim": "sub",
>     "usernamePrefix": "-",
>     "requiredClaims": ["-"]
>    },
>  ]
> "administrators": [
>   "example_3@mail.com",
>   "example_4@mail.com"
>  ]
> }
> ```
>
> The email addresses must be recognized by `https://kyma.accounts.ondemand.com`.

3. Select *Create*.

Results

Your Kyma environment is instantiated with a custom identity provider.

Related Information

Authentication in the Kyma Environment [page 3659]
Custom OpenID Connect Configuration [page 3505]
OpenID Connect (OIDC) [page 3494]

7.6.3.2 Authorization in the Kyma Environment

You control access to your Kyma cluster using Kubernetes role-based access control (RBAC). Roles define what resources users can access and what actions they can perform.

Standard Roles

For access management, Kyma relies on standard Kubernetes RBAC, using standard Kubernetes ClusterRoles with different levels of access: `view`, `edit`, `admin`, and `cluster-admin`. For details, see Kubernetes: User-facing roles .

When you create a Kyma cluster, your user account is automatically bound to the `cluster-admin` role. As a cluster admin, you can assign roles to other users and groups.

You manage roles and bindings directly in the cluster using Kyma dashboard or kubectl. You don't use SAP BTP role collections for this task.

You assign roles to users or groups through RoleBindings (scoped to a namespace) and ClusterRoleBindings (scoped to the entire cluster). To assign roles to groups rather than individual users, you must configure a custom identity provider. The default shared SAP ID service tenant does not support group claims. For details, see Authentication in the Kyma Environment [page 3659].

As a starting point, separate operators from developers: operators manage the cluster infrastructure, and developers work within their namespaces. If you want to avoid permanent assignment of powerful roles, use Kubernetes impersonation. For details and manifest templates, see RBAC Manifest Templates for Kyma [page 3668].

Module Permissions

Kyma modules automatically extend the standard `view`, `edit`, and `admin` roles with module-specific permissions through ClusterRole aggregation. When you install a module, users assigned to these roles

immediately get the appropriate module permissions. You don't have to update the roles manually. For details, see ClusterRole Aggregation in Kyma [page 3664].

Related Information

Kubernetes: Using RBAC Authorization

Kubernetes: Role Based Access Control Good Practices

7.6.3.2.1 ClusterRole Aggregation in Kyma

Kyma modules use Kubernetes `ClusterRole` aggregation to automatically extend standard roles with module-specific permissions. When you install a module, users already bound to the standard `view`, `edit`, or `admin` roles immediately gain the appropriate permissions for the module's resources - no manual role updates are required.

How Kyma Modules Use Aggregation

Kubernetes provides a `ClusterRole` aggregation mechanism that allows roles to automatically merge permissions from other roles based on label selectors. For details, see Kubernetes: Aggregated ClusterRoles .

Kyma modules use this mechanism to extend the standard Kubernetes `view`, `edit`, and `admin` roles. For example, to grant read-only access to the module's resources, a module provides the `kyma-<module>-view` role with label `rbac.authorization.k8s.io/aggregate-to-view: "true"`.

When you install a module, its permissions automatically merge into the standard roles. Users bound to `view`, `edit`, or `admin` roles immediately gain the appropriate module permissions.

→ Tip

To inspect which permissions a ClusterRole provides, use kubectl:

```
# List all module-provided roles
kubectl get clusterrole | grep kyma-
# View a specific role's permissions
kubectl get clusterrole kyma-telemetry-view -o yaml
# See what's aggregated into the standard view role
kubectl get clusterrole view -o yaml
```

The aggregated permissions appear in the `rules` section of the standard role.

Module-Specific Roles

For information about Kyma modules and their features, see Kyma Modules [page 88].

Istio

- `kyma-istio-view` - Grants read-only access to the `Istio` custom resource.
- `kyma-istio-edit` - Grants full access to the `Istio` custom resource.
- `kyma-istio-resources-view` - Grants read-only access to resources from all API groups managed by Istio.
- `kyma-istio-resources-edit` - Grants full access to resources from all API groups managed by Istio.

API Gateway

- `kyma-api-gateway-view` - Grants read-only access to API Gateway resources.
- `kyma-api-gateway-edit` - Grants full access to API Gateway resources.

SAP BTP Operator

- `kyma-btp-operator-view` - Grants read-only access to BtpOperator, ServiceInstance, and ServiceBinding CRs and their status.
- `kyma-btp-operator-edit` - Grants full access to BtpOperator, ServiceInstance, and ServiceBinding CRs and read-only access to their status.

Keda

- `kyma-keda-view` - Grants read-only access to the Keda custom resource and resources from all API groups managed by KEDA.
- `kyma-keda-edit` - Grants full access to the Keda custom resource and resources from all API groups managed by KEDA.

Eventing

- `kyma-eventing-view` - Grants read-only access to all Eventing module custom resources and their statuses.
- `kyma-eventing-edit` - Grants full access to all Eventing module custom resources and read-only access to their statuses.

NATS

- `kyma-nats-view` - Grants read-only access to the NATS custom resources and their statuses.
- `kyma-nats-edit` - Grants full access to the NATS custom resources and read-only access to their statuses.

Serverless

- `kyma-serverless-view` - Grants read-only access to the Serverless custom resource.
- `kyma-serverless-edit` - Grants full access to the Serverless custom resource.
- `kyma-functions-view` - Grants read-only access to the Function custom resources.
- `kyma-functions-edit` - Grants full access to the Function custom resources.

Telemetry

- `kyma-telemetry-view` - Grants read-only access to the Telemetry custom resources.
- `kyma-telemetry-edit` - Grants write access to the Telemetry custom resources

7.6.3.2.2 Assign Roles in the Kyma Environment

Kyma uses roles to manage access within the cluster, which give the assigned users the permissions suitable for their purposes.

Prerequisites

The `cluster-admin` role is assigned to your account.

> **Note**
>
> After creating a Kyma cluster, you become an admin of this instance and the `cluster-admin` role is assigned to you by default. As the `cluster-admin`, you can assign roles to other users. Only the administrator who created the current subaccount can use the SAP BTP cockpit to assign the `cluster-admin` role to other members.

Context

You can assign roles to a group of users only if you use a custom identity provider. Assigning roles in Kyma is based on the Kubernetes role-based access control (RBAC) . You can see the list of available roles in Kyma dashboard when you create a RoleBinding. Once you become the admin, your user name is read from the SAP BTP (RBAC) concept and is passed to the Kyma provisioner to be bound to the `cluster-admin` role in Kyma.

Kyma modules provide aggregated ClusterRoles that automatically extend the standard Kubernetes roles. For details about how aggregation works and how to inspect role permissions, see ClusterRole Aggregation in Kyma [page 3664].

Procedure

1. Log in to Kyma dashboard. The URL is in the *Overview* section of your subaccount.
2. In Kyma dashboard, select or create the namespace in which you want to assign roles.
3. To create a role binding, go to *Configuration* > *Role Bindings* > *Create*. Fill in the required fields:
   a. As *Name*, insert the name of your binding.
   b. As *Role Type*, choose *ClusterRole*.

c. As *Role*, choose the required role from the list.
d. As *Kind*, choose *User*.
e. As *User name*, insert the user email address.

4. Select *Create*.

Results

The users have the required permissions within the specified namespace. If the users don't have additional cluster role binding to list the namespaces, they can still access the Kyma dashboard overview but must enter the required namespace name manually.

Next Steps

If the permissions of the default role aren't sufficient, clone the role, and add the missing resources.

Related Information

Authorization in the Kyma Environment [page 3663]
ClusterRole Aggregation in Kyma [page 3664]

7.6.3.2.3 Overwrite Kyma Administrators

In some cases, you may need to overwrite usernames of the current administrators, for example, if they forget their password or leave the organization and nobody can access your Kyma runtime.

Prerequisites

- You're the subaccount administrator.

Context

To overwrite the names for the `cluster-admin` role, update the Kyma instance and provide a new value for the *administrators* field.

> ⚠ Caution
>
> This procedure overwrites current administrators and shouldn't be used to add new ones. Treat it as an emergency self-service procedure in case of lost access. To provide access to new users, the `cluster-admin` should create a RoleBinding and/or a ClusterRoleBinding in the runtime. See RoleBinding and ClusterRoleBinding in the official Kubernetes documentation.

Procedure

1. In the SAP BTP cockpit, go to your Kyma instance and select *Update*.
2. Provide a new set of administrators' usernames as an array of strings.

   > → Remember
   >
   > Use administrators' email addresses as their usernames.

   In a JSON file, use the following structure:

   ```
   "administrators": [
           "example_1@mail.com",
           "example_2@mail.com",
           "example_3@mail.com"
       ]
   ```

3. Click *Update*.

Results

Your Kyma instance is updated with the new administrators' usernames.

Related Information

Provisioning and Updating Parameters in the Kyma Environment [page 3470]

7.6.3.2.4 RBAC Manifest Templates for Kyma

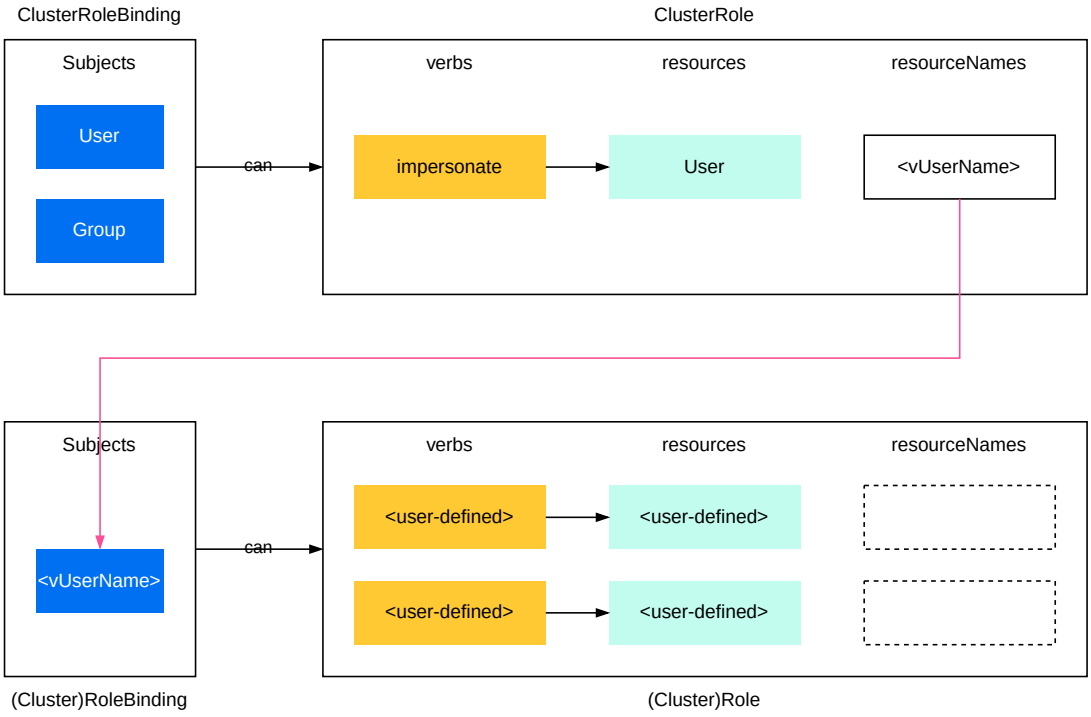
Use the following manifest templates to implement the recommended RBAC (Kubernetes role-based access control) patterns in your Kyma cluster.

1. Before applying the manifests, replace placeholder values (in angle brackets) with your actual configuration.
2. To apply a template, run `kubectl apply -f filename.yaml`.

Impersonation in Kubernetes

With Kubernetes User Impersonation , you can enable a least-privilege type of access to the Kubernetes cluster. With this feature, a subject (that is, users, groups, or service accounts) can acquire another Kubernetes user or group identity. These acquired identities are called “virtual users” or “vUser” because they don't need to exist.

Just create a ClusterRoleBinding that contains a virtual user (`vUserName`) and the administrative ClusterRole. For example, the operator or developer is allowed to impersonate the virtual user to perform administrative tasks.

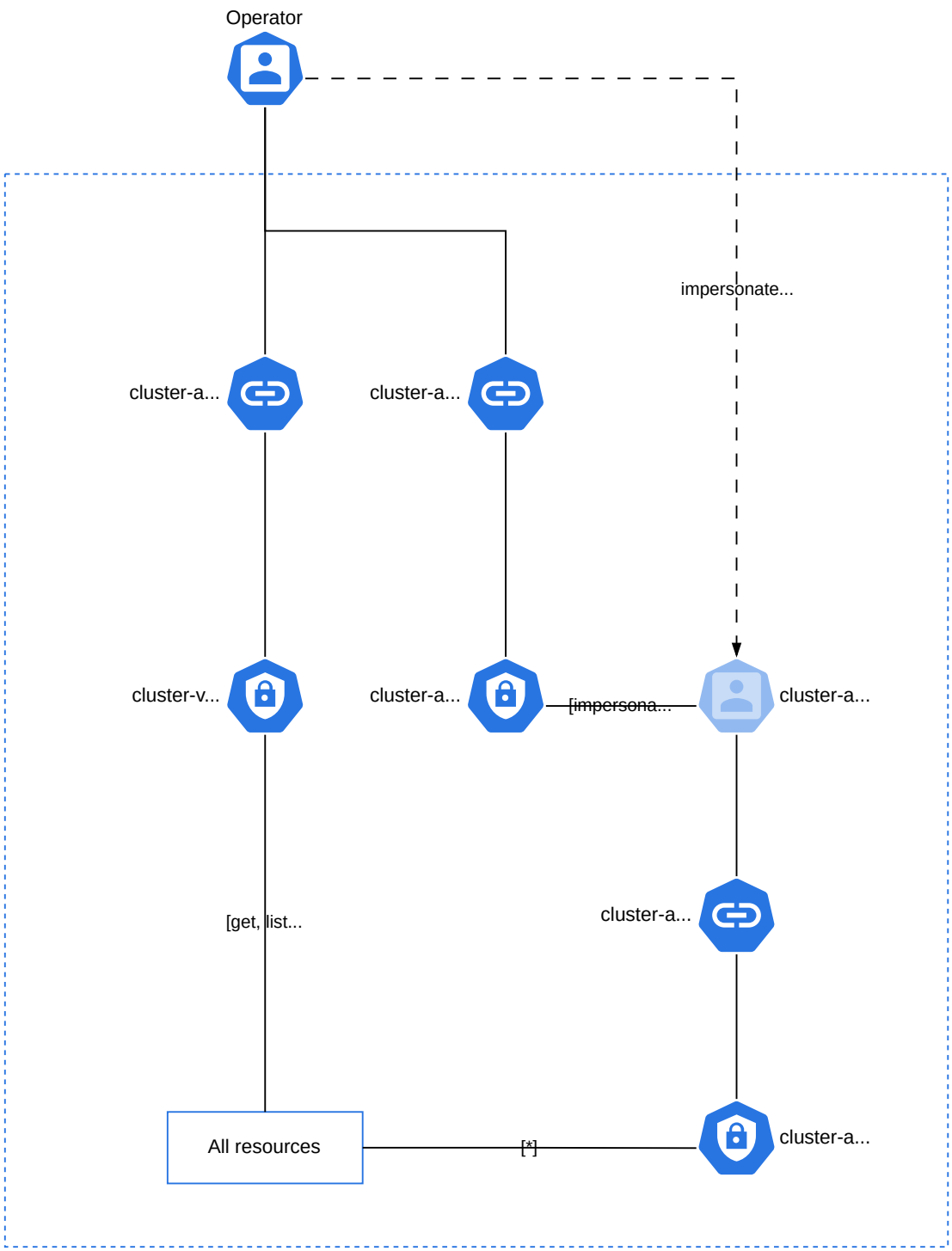


Text is not SVG - cannot display

Recommended Role Setup

Operators

- By default, operators only have view access in the clusters.
- By impersonating a virtual user (`cluster-admin`) bound to the ClusterRole `cluster-admin`, the operator can perform administrative tasks.

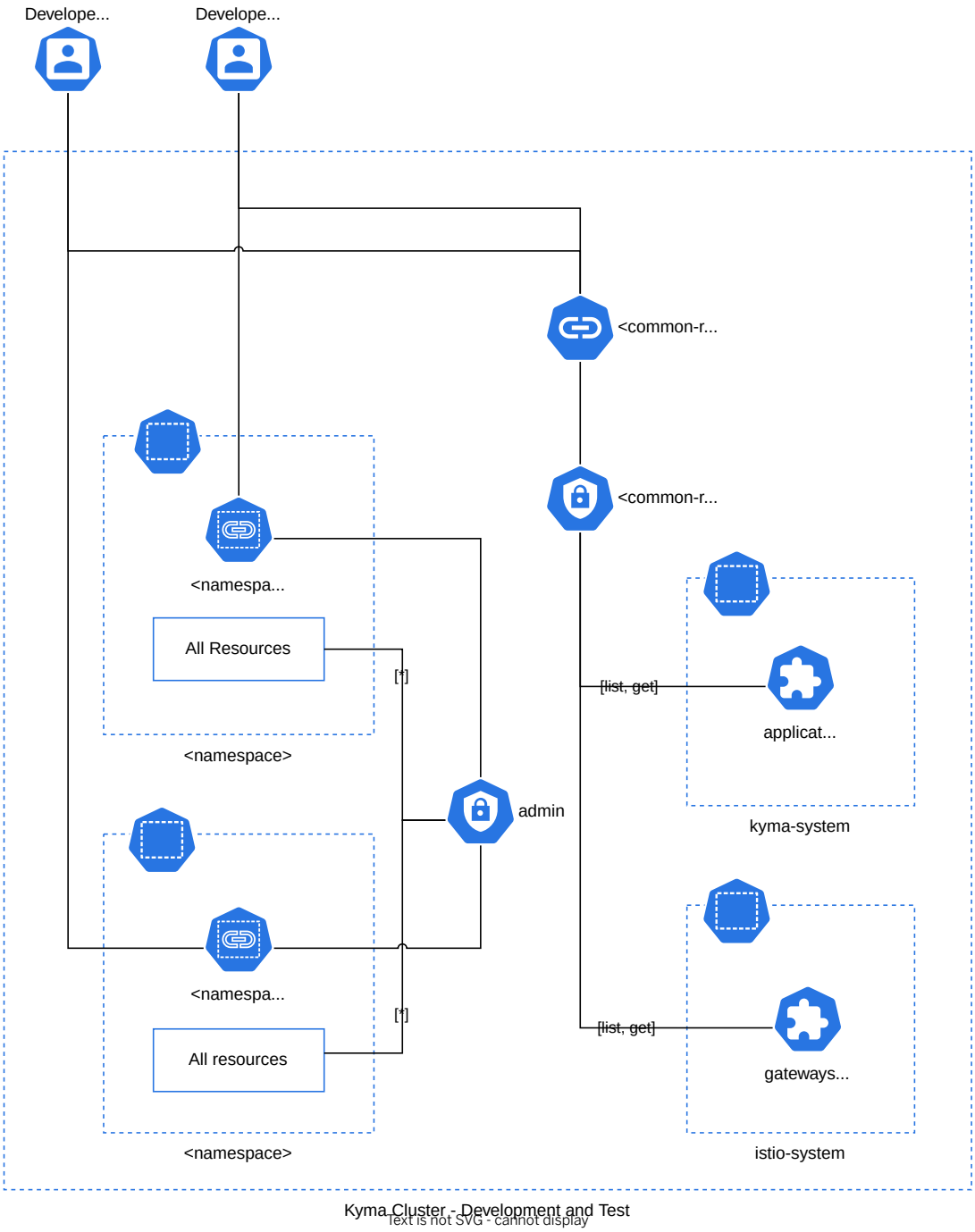


Text is not SVG - cannot display

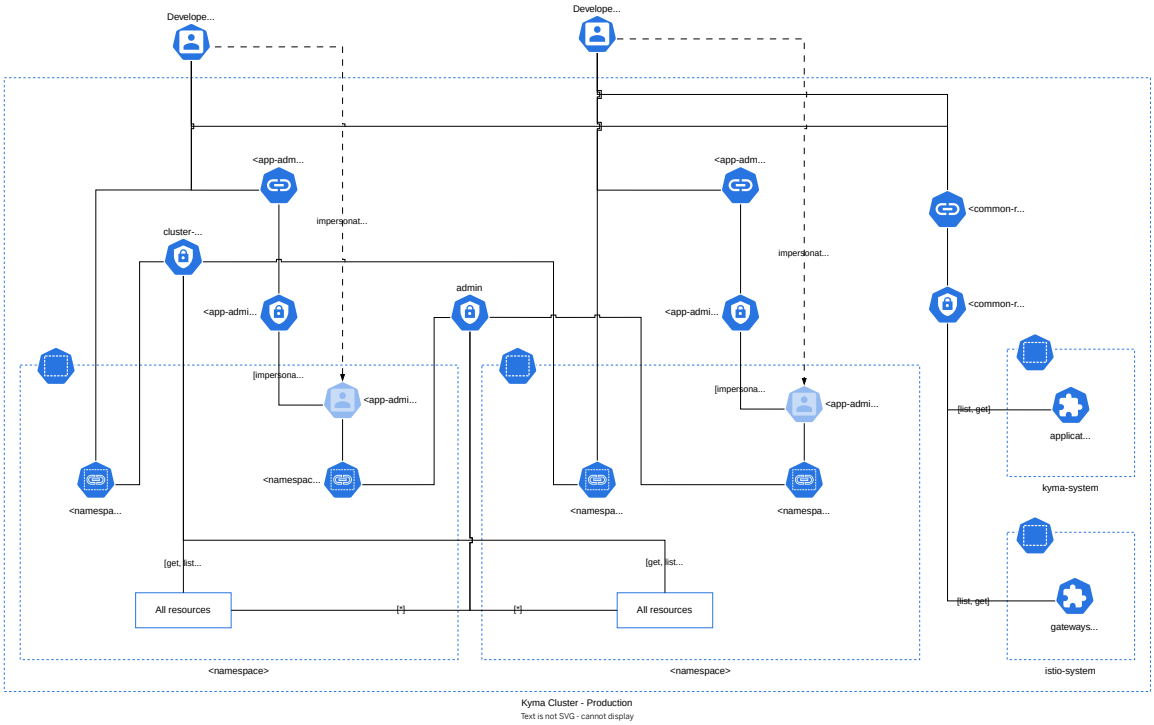
Role Setup: Operators in Dev, Test, and Prod

Developers

- In Dev and Test clusters, developers have full access to their namespaces. In the namespace, bind them to the ClusterRole `admin`.
- In Prod clusters, developers only have view access to their namespace. In the namespace, bind them to the ClusterRole `cluster-viewer`.
- By impersonating a virtual user (`app-admin`) bound to the ClusterRole `admin`, a developer can perform administrative tasks in their namespace.
- Developers typically also need read-only access to cluster-level resources in other namespaces, such as "Istio" or "Application Connector" configurations. Bind them to the ClusterRole `view` for this (see also the generic role template "View Common Kyma Objects in the Cluster").



Role Setup: Developers in Dev and Test



Role Setup: Developers in Prod

To implement these patterns in your cluster, create RoleBindings and ClusterRoleBindings for your users and groups. For details, see Assign Roles in the Kyma Environment [page 3666].

Kubernetes Manifest Templates for Generic Roles

View All Resources

Used in **Dev**, **Test**, and **Prod** clusters:

- ClusterRole `cluster-viewer`

  Sample Code

  ```
  # ClusterRole with the permission to view all resources
  kind: ClusterRole
  apiVersion: rbac.authorization.k8s.io/v1
  metadata:
    name: cluster-viewer
  rules:
    - apiGroups: ['*']
      verbs: ['get', 'list', 'watch']
      resources: ['*']
  ```

View Common Kyma Objects in the Cluster

Used in **Dev**, **Test**, and **Prod** clusters:

- ClusterRole `common-resource-viewer`

Sample Code

```
# ClusterRole with the permissions to view common Kyma objects in the
cluster
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: <common-resource-viewer>
rules:
  - verbs:
      - get
      - list
      - watch
    apiGroups:
      - applicationconnector.kyma-project.io
      - networking.istio.io
    resources:
      - applications
      - gateways
    resourceNames: []
    nonResourceURLs: []
```

- ClusterRoleBinding `common-resource-viewer-crb`

Sample Code

```
# ClusterRoleBinding to allow a group to view common Kyma objects in the
cluster
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: <common-resource-viewer-crb>
subjects:
  - kind: Group
    name: <group-name>
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: common-resource-viewer
  apiGroup: rbac.authorization.k8s.io
```

Kubernetes Manifest Templates for Operators

View Access for All Clusters

Used in **Dev**, **Test**, and **Prod** clusters:

- ClusterRoleBinding `cluster-admin-view-crb`

Sample Code

```
# Default permissions for the ops-team group to view all resources in the
cluster
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: cluster-admin-view-crb
roleRef:
  apiGroup: rbac.authorization.k8s.io
```

```
  kind: ClusterRole
  name: cluster-viewer
subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: Group
    name: <group-name>
```

Admin Access for All Clusters

Used in **Dev**, **Test**, and **Prod** clusters:

- ClusterRoleBinding `cluster-admin-crb`

  > ### Note
  >
  > The ClusterRole `cluster-admin` isn't directly bound to the group of the operations team; they can only impersonate it.

  #### Sample Code

  ```
  # ClusterRoleBinding to allow the (not really existing) user cluster-admin
  to have cluster-admin permissions on the cluster. The ClusterRole cluster-
  admin is NOT directly bound to the ops-team group.
  apiVersion: rbac.authorization.k8s.io/v1
  kind: ClusterRoleBinding
  metadata:
    name: cluster-admin-crb
  roleRef:
    apiGroup: rbac.authorization.k8s.io
    kind: ClusterRole
    name: cluster-admin
  subjects:
    - apiGroup: rbac.authorization.k8s.io
      kind: User
      name: cluster-admin
  ```

- ClusterRole `cluster-admin-impersonator`

  #### Sample Code

  ```
  # ClusterRole to allow the impersonation of the cluster-admin user
  apiVersion: rbac.authorization.k8s.io/v1
  kind: ClusterRole
  metadata:
    name: cluster-admin-impersonator
  rules:
    - apiGroups: ['']
      resources: ['users']
      verbs: ['impersonate']
      resourceNames: ['cluster-admin']
  ```

- ClusterRoleBinding `cluster-admin-impersonate-crb`

  #### Sample Code

  ```
  # ClusterRoleBinding to bind the impersonation capability to everyone in
  the ops-team group
  apiVersion: rbac.authorization.k8s.io/v1
  kind: ClusterRoleBinding
  metadata:
    name: cluster-admin-impersonate-crb
  ```

```
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin-impersonator
subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: Group
    name: <group-name>
```

Kubernetes Manifest Templates for Developers

Namespace Admin Access for Dev and Test

Used in **Dev** and **Test** clusters:

- RoleBinding `admin`

Sample Code

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: <name>
  namespace: <name of the namespace>
subjects:
  - kind: Group
    name: <group name>
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: admin
  apiGroup: rbac.authorization.k8s.io
```

Admin Access for Prod

Create the respective manifest files for each team and namespace, and adapt them accordingly.

Used in **Prod** clusters:

- ClusterRole `app-admin-impersonator`

Sample Code

```
# ClusterRole to allow the impersonation of the application admin user
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: <app-admin-impersonator>
rules:
  - apiGroups: ['']
    resources: ['users']
    verbs: ['impersonate']
    resourceNames: ['<name of the virtual user>']
```

- ClusterRoleBinding `app-team-impersonation-crb`

Sample Code

```
# ClusterRoleBinding to bind the impersonation capability to the app-team
group
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: <app-team-impersonation-crb>
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: <app-admin-impersonator>
subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: Group
    name: <group-name>
```

- RoleBinding `admin`

Note

The ClusterRole `admin` isn't directly bound to the groups of the development teams; they can only impersonate it.

Sample Code

```
# RoleBinding to allow a (not really existing) user to have admin
permissions in a specific namespace. The ClusterRole admin is NOT directly
bound to the developers groups.
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: <binding name, e.g. app1-admin-crb>
  namespace: <namespace>
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: admin
subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: User
    name: <name of the virtual user>
```

Read Access to Namespace for Prod

Create the respective manifest files for each team and namespace, and adapt them accordingly.

Used in **Prod** clusters:

- RoleBinding `cluster-viewer`

Sample Code

```
# Default permissions of an app-team group to view all resources in a
dedicated namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: <namespace-viewer-binding>
  namespace: <namespace>
roleRef:
```

```
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-viewer
subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: Group
    name: <group-name>
```

7.6.3.3 Auditing and Logging Information in Kyma

Kyma runtime collects audit and application logs.

Audit Logs

Audit logs are records that provide evidence of events, actions, or operations. The audit log records the following data:

- Security and system events that include administrative activities, potential threats, and evidence in the case of a breach.
- Changes introduced to the system, applications, and components.

Kyma runtime collects audit logs for configuration changes to the runtime setup itself and Kubernetes resources managed by the Kubernetes API server of the runtime. Usually, those logs contain the user account (technical account or email address) and the client IP address of the subject who triggered the changes. No other personal data is stored in the audit logs. The logs store the data for 90 days. If you want to store any other personal data, be cautious and bear in mind the recommendations provided in Data Protection and Privacy [page 3633].

Retrieving Audit Log Entries

As a user, you don't have direct access to the audit logs. If you want to get information stored in the audit log system, open a support ticket.

Events Written in Audit Logs

Here you can find a list of the security events that are logged by SAP BTP, Kyma runtime.

Events Written in Audit Logs

Event grouping	What events are logged	Additional information
Secrets, ConfigMaps, and TokenReviews	CREATE, UPDATE, PATCH, and DELETE operations on the metadata level	The payload is not logged. Resources: • "secrets" • "configmaps" • "tokenreviews"
All standard Kubernetes resources in core and extensions	CREATE, UPDATE, PATCH, and DELETE operations on the request level	Resource groups: • "" # core • "admissionregistration.k8s.io" • "apiextensions.k8s.io" • "apiregistration.k8s.io" • "apps" • "authentication.k8s.io" • "authorization.k8s.io" • "autoscaling" • "batch" • "certificates.k8s.io" • "coordination.k8s.io" • "extensions" • "metrics.k8s.io" • "networking.k8s.io" • "policy" • "rbac.authorization.k8s.io" • "scheduling.k8s.io" • "settings.k8s.io" • "storage.k8s.io"
All Kyma resources	CREATE, UPDATE, PATCH, and DELETE operations on the request level	Resource groups: • "applicationconnector.kyma-project.io" • "compass.kyma-project.io" • "eventing.kyma-project.io" • "gateway.kyma-project.io" • "serverless.kyma-project.io" • "telemetry.kyma-project.io" • "operator.kyma-project.io"

Event grouping	What events are logged	Additional information
All resources of Kyma dependencies	CREATE, UPDATE, PATCH, and DELETE operations on the request level	Resource groups: • "cert.gardener.cloud" • "ory.sh" • "services.cloud.sap.com" • "extensions.istio.io" • "install.istio.io" • "networking.istio.io" • "security.istio.io" • "telemetry.istio.io"

Application Logs of System Components

Kyma runtime collects application logs provided by the Kyma system components. Those logs reflect regular events that occurred in Kyma runtime and must not include any personal or sensitive data. These are typically system-related activities and operations you can analyze to get more information about performance or to inspect potential problems.

Related Information

https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/
Audit Logging in the Cloud Foundry Environment
Audit Logging in the Neo Environment
Access Kyma Application Logs [page 3515]
Collecting Logs [page 2194]

7.6.3.4 Verify Image Signatures in the Kyma Environment

Image signing and image signature verification are important countermeasures against security threats. They help you ensure the authenticity and integrity of the application you deploy in the runtime. You can be sure that what you deploy is what you have built in the build pipeline. While image signing happens in the automated Continuous Delivery (CD) workflow, image signature verification takes place at the time the workload is scheduled in the runtime. For this purpose, Kyma uses Warden, a mandatory security feature, which is added to your cluster by default and ensures that the Kyma workloads are authentic.

Warden is automatically enabled in the namespaces managed by Kyma, which can be identified by the `operator.kyma-project.io/managed-by: kyma` label. Therefore, you can be sure the Kyma-managed workloads are authentic. Use the `kubectl get namespace -l operator.kyma-project.io/managed-by=kyma` command to list the Kyma system namespaces, protected by Warden.

> ⚠ Caution
>
> When you enable third-party Kubernetes tools that mutate workloads (for example, Dynatrace that adds init containers), you must exclude the Kyma-managed namespaces, which are protected by Warden, from the tool config. Otherwise, Warden blocks unknown images. Mind that disrupting the Kyma-managed namespaces may lead to an inability to uphold the Service Level Agreement (SLA).

You can enable Warden also in your namespaces. All you need to do is add the `namespaces.warden.kyma-project.io/validate=user` label to a namespace and provide a notary service URL as an annotation. Warden then verifies if the images used for the workloads in these namespaces are signed by the configured notary service.

If an image is not signed or explicitly allowed by a dedicated annotation and is used to schedule a Pod in the protected namespace, the Pod admission is rejected.

Configuration

To enable Warden in your namespace, add the `namespaces.warden.kyma-project.io/validate: user` label to the given namespace. You can configure Warden on each namespace by adding the following annotations:

Name	Required	Description	Default value
`namespaces.warden.kyma-project.io/notary-url`	Yes	URL of the Notary server used for image verification.	""

`namespaces.warden.kyma-project.io/allowed-registries`	No	Comma-separated list of image URL prefixes for which the signature verification should be skipped. With that, you can configure exceptions for selected images, registry folders, or whole registries.
		Example
		To let Warden skip the verification of the `foo:0.0.1` and `bar:0.0.1` images (and tags), use the following value:
		`acme.common.repositories.cloud.sap/foo:0.0.1, acme.common.repositories.cloud.sap/bar:0.0.1`
		To skip all images from a given registry, use:
		`acme.common.repositories.cloud.sap/`
			""

`namespaces.warden.kyma-project.io/notary-timeout`	No	Timeout for the Notary server connection.	"30s"
`namespaces.warden.kyma-project.io/strict-mode`	No	If set to `true`, Warden rejects all images when the Notary server is unavailable. If set to `false`, Warden adds the label `pods.warden.kyma-project.io/validate: pending` to the Pod and retries the validation later.	"true"

Example Configuration

Example namespace configuration verified by Warden.

```
apiVersion: v1
kind: Namespace
metadata:
  name: my-namespace
  labels:
    namespaces.warden.kyma-project.io/validate: "user"
  annotations:
    namespaces.warden.kyma-project.io/notary-url: "https://
signing.repositories.cloud.sap"
    namespaces.warden.kyma-project.io/allowed-registries:
"acme.common.repositories.cloud.sap/foo:0.0.1,
acme.common.repositories.cloud.sap/bar:0.0.1"
    namespaces.warden.kyma-project.io/notary-timeout: "30s"
    namespaces.warden.kyma-project.io/strict-mode: "true"
```

8 Getting Support

To get help with technical issues, use the support channels and procedures available through SAP for Me. Before you begin, check the platform status and use the built-in support tools.

Prerequisites

Before you report a case, ensure the following:

- **Check Platform Status**
  - **Running on the AWS, Azure, or GCP regions:** visit SAP Trust Center. You can check:
    - availability by service on the *Cloud Status* tab page;
    - availability by region on the *Data Center* tab page.
  - **Running on the China (Shanghai) region:** check China (Shanghai) region status page .
  - **Running on the Government Cloud (US) region:** SAP sends planned downtime and outage communications via email to the initial administrator of your global account.
- **Check Tools Versions**
  Make sure that you have the latest versions of the tools you're using (recommended), or at least the versions you're using are still supported. For more information, see SAP Development Tools.

  > **Note**
  >
  > Trial users can ask for support by posting a question in SAP Community. However, they should first check for answers in SAP Community.
- **Try Out Built-In Support Within SAP BTP Cockpit**
  When using SAP BTP cockpit as an S-user, try Built-In Support to get customer help within the cockpit. See Built-In Support.
- **Learn the Best Practices and Offerings**
  Familiarize yourself with the guidelines on how to engage with support. See Support from SAP Best Practices and Offerings.

Context

> **Note**
>
> This section is applicable for all multicloud environments. To find which environments are available in your region, see Regions [page 17].

To report a case in SAP for Me, proceed as follows:

Procedure

1. Open SAP for Me and log on.
2. Open the *Services & Support* dashboard.
3. **Optional:** You can use the *Knowledge Search* tab for existing solutions in:
   - SAP Notes & Knowledge Base Articles (KBAs)
   - SAP Community
   - SAP Support Portal
   - Product Documentation
4. Use the *Get Support* application within SAP for Me to get assistance via the available support channels.
   - For more information about the specifics of each support channel and to learn how to create cases to report technical issues, see Get Support Application.
   - For any other non-technical assistance, see Customer Interaction Center.

Creating a Support User

Context

You may need to create a support user to troubleshoot issues in your system. See 3543065 - Best Practices for Creating Support Users in Your BTP System.

Related Information

Users and Authorizations for SAP One Support Launchpad
Gather Support Information [page 3686]
Platform Updates and Notifications [page 3687]
Cloud Availability Center
Road Map for Cloud Foundry Environment
Road Map for Kyma Runtime
Road Map for ABAP Environment

8.1 Providing Details for SAP HANA Service Database Problems

If your problem is related to a SAP HANA service database, the details you need to provide differ depending on the environment or infrastructure provider the database is provisioned in.

Infrastructure Provider	Details You Need to Provide	How to Find the Details You Need
Azure regions AWS regions (databases provisioned before June 4, 2018)	The URL of the space the database is provisioned in	1. In the SAP BTP cockpit, navigate to the org and space the database is provisioned in. 2. In the navigation area, go to *SAP HANA* > *Database Systems*. 3. Choose the affected *Database System*. 4. Copy the URL from the overview of the affected database system. Example URL: `https://account.hana.ondemand.com/cockpit#/globalaccount/<id>/subaccount/<id>/org/<id>/dbsystems/<id>/overview`
AWS regions (databases provisioned after June 4, 2018) Google Cloud regions	The service instance ID	1. In the SAP BTP cockpit, navigate to the space the SAP HANA service instance is provisioned in. 2. In the navigation area, choose *Services* > *Service Instances*. 3. In the list of available services, find your SAP HANA service instance and choose *Open Dashboard* from the *Actions* column. 4. Copy the service instance GUID under *Details* > *ID*. Example GUID: ad539b83-39bd-4ff4-8b41-1a5dfdfca7ea
China (Shanghai) region	See Getting Support	
Government Cloud (US) region	Please contact your operator.	

Related Information

Providing Details for Database Problems in the Neo Environment

8.2 Gather Support Information

The Eclipse tools come with a wizard for gathering support information in case you need help with a feature or operation (during deploying/debugging applications, logging, configurations, and so on).

Context

The wizard collects the information in a ZIP file, which can be later sent to the support team. This way, the support developers can get better understanding of your environment and process the issue faster.

Procedure

1. From the Eclipse IDE, choose *Help* > *Collect Support Information*.
2. The launched wizard lists the default components to be collected, depending on the tools you've installed. If you need the support team to look at specific resources, expand the *Additional Data* section and select the relevant items.

   > **Note**
   >
   > If you select *Screenshot*, your currently open Eclipse windows and views are snapped as a picture and added to the ZIP file. Make sure you don't reveal sensitive information.

3. In the *File Name* field, specify the ZIP file name and location.
4. Choose *Finish*.

Next Steps

You can create a support ticket, attach the files to it, and send it to the corresponding support team. For more information, see Getting Support [page 3683].

8.3 Platform Updates and Notifications

Platform updates for SAP Business Technology Platform include biweekly releases, immediate fixes, and major upgrades. To stay informed about updates and planned maintenance that may affect your applications, subscribe to notifications.

SAP BTP is a dynamic product, which has continuous production releases (updates). To get notifications for the new features and fixes every release, subscribe at What's New for SAP Business Technology Platform. For more information, see Subscribing to What's New Notifications.

Reasons for Region Updates

Regions are updated in the following cases:

- **Biweekly updates** (standard) - aligned with the contractual obligations to customers and partners. Such updates usually don't affect productive applications, because most services support zero downtime maintenance. For more information about the biweekly updates, see Consolidated Release Schedules for SAP BTP, Intelligent Enterprise Suite: Harmonized release calendar for SAP Cloud products, and Service Level Agreement for SAP Cloud Services.
- **Immediate updates** - fixes required for bugs that affect productive application operations, or due to urgent security fixes. In some cases, this might lead to downtime or application restart, for which the application groups receive a notification.
- **Major upgrades** - happen rarely, in a bigger maintenance window, up to four times per year. For the time frames of the services' major upgrades, see Service Level Agreement for SAP Cloud Services. We let you know about these upgrades 4 weeks in advance.

Announcements and Subscriptions [AWS, Azure, or GCP Regions]

You can follow the availability of the platform at SAP Trust Center. You can check:

- the availability by service on the *Cloud Status* tab page;
- the availability by region on the *Data Center* tab page.

In addition, you can use Cloud Availability Center (CAC) to check what cloud products and services are assigned to your customer ID for notifications. CAC provides you with a personalized dashboard based on SAP Fiori. It's designed to give you quick access to relevant information about the cloud products that you own and their availability and maintenance status. For more information, see CAC User Guide.

You can find information about the supported *Cloud Product* > *Cloud Service* at Supported Products.

> **Note**
>
> The foundational components of the platform are included in *SAP Extension Suite – Development Efficiency* > *Foundational Services for SAP BTP*.

To get notifications for updates and downtimes, create a subscription by specifying Cloud Product, Cloud Service, and Notification Type. For more information, see Cloud System Notification Subscriptions User Guide.

> **Example**
>
> To receive notifications about changes in the IP ranges of a region, subscribe to *SAP Extension Suite – Development Efficiency* > *Foundational Services for SAP BTP*.

> **Example**
>
> To receive notifications about what application processes to restart to apply new security patches, subscribe to *SAP Extension Suite – Development Efficiency* > *Foundational Services for SAP BTP*.

Announcements and Subscriptions [China (Shanghai) Region]

You can follow the availability of the platform and the announcements about upcoming updates and downtimes at China (Shanghai) region status page . Subscribe to the status page to get notifications for updates and downtimes.

Announcements and Subscriptions [Government Cloud (US) Region]

Planned downtimes and outage communication are sent through e-mail to the initial administrator of your global account.

Related Information

What's New
Regions [page 17]
Release Schedule for cross and foundational services of SAP BTP, Cloud Foundry environment
Release Schedule for SAP BTP, Kyma Runtime

8.4 Operating Model

The operating model defines the shared operational responsibilities and tasks between SAP and customers for Cloud Foundry and Kyma runtimes.

The operating model doesn't cover tasks that are part of SAP's inherent platform quality (such as infrastructure management, platform patching, network operations, or platform monitoring), nor does it list

standard support processes (such as case management or service requests) that are governed by your contractual agreement with SAP.

It isn't the intent of this document to supplement or modify the contractual agreement between SAP and the customer for the purchase of any of the services in scope. In the event of a conflict, the contractual agreement between SAP and the customer as set out in the Order Form, the General Terms and Conditions, the supplemental terms and conditions, and any resources referenced by those documents always takes precedence over this document.

Changes to this operating model are published using the *What's New* section of the platform. If critical changes are made that require action on the customer side, an explicit notification is sent by e-mail to the affected customers.

The following table defines the operational responsibilities shared between SAP and the customer. Tasks are classified by scope:

- Common tasks apply to both Cloud Foundry runtime and Kyma runtime.
- Cloud Foundry tasks apply only to the Cloud Foundry runtime.
- Kyma tasks apply only to the Kyma runtime.

Operating Model

Process	Scope	Task	Responsibility
Change Management and Operations	Common	Apply regular product increments and corrections to the infrastructure, systems, and services.	SAP
Change Management and Operations	Cloud Foundry	When using applications of lifecycle type “buildpack”, SAP provides new system buildpacks. However, customers must restage their application to apply changes in the buildpack.	Customer
Change Management and Operations	Cloud Foundry	When using a custom buildpack, update the buildpack version and restage the application to apply security fixes.	Customer
Change Management and Operations	Kyma	When using applications of lifecycle type “docker”, build the container image and apply all updates and security patches to the image's OS and dependencies.	Customer
Change Management and Operations	Cloud Foundry	When using applications of lifecycle type “buildpack”, SAP ensures the base operating system (stack) is updated with security patches. Customer applications are automatically restarted to consume the update.	SAP
Change Management and Operations	Kyma	Ensure that any custom configurations of modules are correct and updated as required by platform changes.	Customer
Application Management	Common	Design, develop, deploy, configure, maintain, and operate your applications within the subaccount. This includes application resource management and managing application availability and performance.	Customer

Process	Scope	Task	Responsibility
Application Management	Common	Provide tools and application programming interfaces (APIs) for the lifecycle management and operations of applications.	SAP
Application Management	Common	Regularly adopt the latest versions of the tools for lifecycle management and operations offered by SAP.	Customer
Application Management	Cloud Foundry	Ensure isolation between workloads and customers.	SAP
Application Management	Kyma	Ensure that your application configurations are hardened according to security best practices (for example, secure secrets management and, for Kyma runtime, Kubernetes Pod Security Recommendations).	Customer
Application Management	Common	Ensure that your applications correctly report their health and readiness status so that the platform can route and manage traffic as intended.	Customer
Application Management	Common	Scale applications based on demand using the platform's scaling features. Monitor application performance and resource usage to determine optimal scaling configurations. In the Kyma runtime, platform-level scaling may also require configuration of platform resources by the customer.	Customer
Network Management	Common	Create and manage your own custom domains for applications. Custom domains are especially recommended for production environments.	Customer
Account and Environment Management	Common	Create and manage subaccounts and runtime environments for different stages of your development lifecycle (for example, development, testing, and production). Configure appropriate permissions and resource quotas for each environment.	Customer
Authorization and Identity Management	Common	Manage users, permissions, and security configurations within the subaccount and runtime environments (for example, Cloud Foundry spaces or Kyma clusters).	Customer
Observability	Common	Monitor the resources and performance of your applications. Set up application monitoring and alerting for business-critical applications. Access and analyze application logs and metrics through the platform's observability tools.	Customer
Security and Compliance	Common	Ensure that platform and platform-provided assets are free of vulnerabilities and malware.	SAP

Process	Scope	Task	Responsibility
Security and Compliance	Common	Ensure that customer-provided applications and assets (for example, code, packages, and container images) are free of malware.	Customer
Security and Compliance	Common	Check regularly for vulnerabilities in your custom applications and their dependencies. If you find vulnerabilities, ensure they're mitigated in time.	Customer
Security and Compliance	Cloud Foundry	Inform SAP about any penetration testing that shall be performed for the customer account and ask for their approval. The results of the test are to be shared with SAP to mitigate or remedy any security issues.	Customer
Security and Compliance	Cloud Foundry	Inform SAP about any performance testing that shall be performed for the customer account and ask for their approval. Testing isn't allowed on any resources shared with other customers.	Customer
Backup and Restore	Kyma	Perform a backup of the database systems hosted in the subaccount. If you as a customer deploy your custom storage on Kyma runtime, you're responsible for backing it up. See Volume Backup for Customer Data.	SAP Customer
Backup and Restore	Kyma	Restore previously backed-up data to recover to a consistent state. If you as a customer deploy your custom databases on Kyma runtime, you're responsible for restoring them. See Volume Backup for Customer Data.	SAP Customer
Provisioning	Common	Provision resources and systems to customers in accordance with the ordered package and subscriptions, including the initial configuration of quotas, service subscriptions, permissions, and trust.	SAP
Provisioning	Common	Manage and assign the available quotas for resources and services within the global account and subaccounts.	Customer

Related Information

SLAs for Cloud Services

8.4.1 Operating Model in the Cloud Foundry Environment

Sorry, the content of this topic has moved. See Operating Model [page 3688].

8.4.2 Operating Model in the Kyma Environment

Sorry, the content of this topic has moved. See Operating Model [page 3688].

8.4.3 Comparison between the Operating Models of Kyma and Cloud Foundry Runtimes

Sorry, the content of this topic has moved. See Operating Model [page 3688].

8.5 Support Components

A list of support components for SAP BTP services and tools. Filter for the service you want to find the component for or have a look at the tools and software logistics section.

> ⓘ Note
>
> The table below lists the support components for services. If you are looking for components of tools or issues related to software logistics, see Additional Components [page 3783].

Service Components

Service Availability

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Java Server	Develop and run Java Web applications on SAP Cloud Platform Neo Environment.	BC-NEO-RT-JAV	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Identity Provisioning	Manage identity lifecycle processes for cloud and on-premise systems.	BC-IAM-IPS	Neo	SAP	UAE (Dubai) Australia (Sydney) Brazil (São Paulo) Canada (Toronto) Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) Japan (Tokyo) KSA (Riyadh) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP Fiori Cloud	Revamp your user experience with SAP Fiori on SAP Cloud Platform.	EP-CPP-NEO-SFC	Neo	SAP	Europe (Rot)* Europe (Frankfurt) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) Europe (Amsterdam) UAE (Dubai) KSA (Riyadh)	Yes	Available
Cloud Foundry Runtime	Operate polyglot cloud applications in Cloud Foundry.	BC-CP-CF	Cloud Foundry	AWS	US East (VA) Europe (Frankfurt) Europe (Frankfurt) EU Access Brazil (São Paulo) Japan (Tokyo) Canada (Montreal) Australia (Sydney) Singapore South Korea (Seoul)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Cloud Foundry Runtime	Operate polyglot cloud applications in Cloud Foundry.	BC-CP-CF	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Singapore Japan (Tokyo) Australia (Sydney)	Yes	Available
Cloud Foundry Runtime	Operate polyglot cloud applications in Cloud Foundry.	BC-CP-CF	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) Israel (Tel Aviv)	Yes	Available
Cloud Foundry Runtime	Operate polyglot cloud applications in Cloud Foundry.	BC-CP-CF	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
Data Enrichment Service	Create or enrich master data using trusted third-party data.	LOD-MDM-DE	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Singapore	Yes	Available
Internet of Things	Develop, customize, and operate IoT business applications in the cloud.	BC-NEO-SVC-IOT	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA)	Yes	Available
SAP Excise Tax Management	Help Excise Tax customers calculate, track, and comply with excise duty tax requirements in real time.	LOD-ET-INT	Cloud Foundry	AWS	Europe (Frankfurt)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Print Service	Manage print queues, connect print clients, and monitor print status.	BC-CCM-PRN-OM-SCP	Cloud Foundry	AWS	Europe (Frankfurt)	Yes	Available
Forms Service by Adobe	Generate print and interactive forms using Adobe Document Services.	BC-SRV-FP-NEO	Neo	SAP	Europe (Rot)* US East (Ashburn) US West (Chandler) US East (Sterling) Australia (Sydney) Japan (Tokyo) Brazil (São Paulo) Canada (Toronto) UAE (Dubai) Europe (Amsterdam) KSA (Riyadh) Europe (Frankfurt) US West (Colorado Springs)	Yes	Available
Process Visibility	Cloud Offering with End-to-End visibility on Business Processes	LOD-BPM-VIS	Cloud Foundry	SAP	Europe (Frankfurt) EUA only Europe (Rot) UAE (Dubai) Australia (Sydney) Japan (Tokyo) US West (Colorado) US (Sterling)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Process Visibility	Cloud Offering with End-to-End visibility on Business Processes	LOD-BPM-VIS	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Australia (Sydney) Singapore Japan (Tokyo) Brazil (São Paulo) South Korea (Seoul) Europe (Frankfurt) EU Access Canada (Montreal) US West (Oregon) Europe (Milan)	Yes	Available
Process Visibility	Cloud Offering with End-to-End visibility on Business Processes	LOD-BPM-VIS	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Singapore Japan (Tokyo) Australia (Sydney) Switzerland (Zurich) EU Access Brazil (São Paulo) Canada Central (Toronto) China (North 3)** UK South (London) Europe (Frankfurt)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Process Visibility	Cloud Offering with End-to-End visibility on Business Processes	LOD-BPM-VIS	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) India (Mumbai) Israel (Tel Aviv) KSA (Dammam) public sector Japan (Osaka) Australia (Sydney) KSA (Dammam) Japan (Tokyo)	Yes	Available
Process Visibility	Cloud Offering with End-to-End visibility on Business Processes	LOD-BPM-VIS	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
SAP Translation Hub	Translate UI texts and get suggestions for UI texts during development.	LOD-TH	Neo	SAP	Europe (Rot)* Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) Australia (Sydney) Japan (Tokyo) Europe (Frankfurt)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP Procurement Intelligence	Automate and simplify your business processes by applying machine learning intelligence.	CA-ML-PA	Cloud Foundry	AWS	Europe (Frankfurt)	Yes	Available
SAP Data Intelligence	Connect data from various sources to enable next-generation data management.	CA-DI	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Australia (Sydney) Japan (Tokyo)	Yes	Available
SAP Data Intelligence	Connect data from various sources to enable next-generation data management.	CA-DI	Cloud Foundry	Azure	Europe (Netherlands) US West (WA)	Yes	Available
Continuous Integration and Delivery	Configure and run predefined pipelines for continuous integration and delivery.	BC-CP-CF-CICD	Cloud Foundry	SAP	Europe (Frankfurt) EUA only Europe (Rot) US West (Colorado) Australia (Sydney) Japan (Tokyo) UAE (Dubai) US (Sterling)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Continuous Integration and Delivery	Configure and run predefined pipelines for continuous integration and delivery.	BC-CP-CF-CICD	Cloud Foundry	AWS	Canada (Montreal) Europe (Frankfurt) Europe (Frankfurt) EU Access Europe (Milan) US East (VA) US West (Oregon) Japan (Tokyo) Australia (Sydney) South Korea (Seoul) Brazil (São Paulo) Singapore	Yes	Available
Continuous Integration and Delivery	Configure and run predefined pipelines for continuous integration and delivery.	BC-CP-CF-CICD	Cloud Foundry	Azure	Canada Central (Toronto) Australia (Sydney) Japan (Tokyo) Europe (Netherlands) Europe (Frankfurt) US East (VA) Switzerland (Zurich) EU Access Brazil (São Paulo) Singapore UK South (London)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Continuous Integration and Delivery	Configure and run predefined pipelines for continuous integration and delivery.	BC-CP-CF-CICD	Cloud Foundry	GCP	Europe (Frankfurt) India (Mumbai) Japan (Osaka) Israel (Tel Aviv) US Central (IA) KSA (Dammam) public sector Australia (Sydney) Brazil (São Paulo) KSA (Dammam) Japan (Tokyo)	Yes	Available
SAP Customer Order Sourcing	Calculate sourcing results based on your own sourcing strategies.	LOD-CID-OSR	Cloud Foundry	AWS	Europe (Frankfurt)	Yes	Available
Object Store	Store and manage the blobs/objects on SAP BTP.	BC-CP-CF-OSAAS	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore South Korea (Seoul) Canada (Montreal) US East (VA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Object Store	Store and manage the blobs/objects on SAP BTP.	BC-CP-CF-OSAAS	Cloud Foundry	Azure	Japan (Tokyo) Europe (Netherlands) US West (WA) US East (VA) Australia (Sydney) Singapore	Yes	Available
Object Store	Store and manage the blobs/objects on SAP BTP.	BC-CP-CF-OSAAS	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) Israel (Tel Aviv)	Yes	Available
SAP Analytics Cloud, embedded edition	Analyze data via live connection to your business application's SAP HANA database.	LOD-ANA-OEM-CP	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Solution Lifecycle Management Service	Deploy, subscribe and transport Solutions using Multi-Target Applications (MTA).	BC-NEO-LCM-SRV	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) Australia (Sydney) Japan (Tokyo) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available
Variant Configuration	Configure your SAP ERP or SAP S/4HANA products interactively in the cloud	LOD-CPS	Cloud Foundry	SAP	Europe (Rot)	Yes	Available
Variant Configuration	Configure your SAP ERP or SAP S/4HANA products interactively in the cloud	LOD-CPS	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Singapore	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Variant Configuration	Configure your SAP ERP or SAP S/4HANA products interactively in the cloud	LOD-CPS	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) Australia (Sydney)	Yes	Available
Variant Configuration	Configure your SAP ERP or SAP S/4HANA products interactively in the cloud	LOD-CPS	Cloud Foundry	GCP	Europe (Frankfurt) US Central (IA) Australia (Sydney) India (Mumbai) KSA (Dammam) public sector KSA (Dammam)	Yes	Available
Business Rules	Enrich cloud offerings with decision modeling, management, and execution service.	LOD-BPM-RUL	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Canada (Toronto) Brazil (São Paulo) Australia (Sydney) Japan (Tokyo) UAE (Dubai) KSA (Riyadh)		Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Business Rules	Enrich cloud offerings with decision modeling, management, and execution service.	LOD-BPM-RUL	Cloud Foundry	SAP	Europe (Frankfurt) EUA only Europe (Rot) UAE (Dubai) Australia (Sydney) Japan (Tokyo) US West (Colorado) US (Sterling)	Yes	Available
Business Rules	Enrich cloud offerings with decision modeling, management, and execution service.	LOD-BPM-RUL	Cloud Foundry	AWS	Europe (Frankfurt) Australia (Sydney) US East (VA) Singapore Japan (Tokyo) Brazil (São Paulo) South Korea (Seoul) Europe (Frankfurt) EU Access Canada (Montreal) US West (Oregon) Europe (Milan)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Business Rules	Enrich cloud offerings with decision modeling, management, and execution service.	LOD-BPM-RUL	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Singapore Japan (Tokyo) Australia (Sydney) Switzerland (Zurich) EU Access Brazil (São Paulo) Canada Central (Toronto) China (North 3)** UK South (London) Europe (Frankfurt)	Yes	Available
Business Rules	Enrich cloud offerings with decision modeling, management, and execution service.	LOD-BPM-RUL	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) India (Mumbai) Israel (Tel Aviv) KSA (Dammam) public sector Japan (Osaka) Australia (Sydney) KSA (Dammam) Japan (Tokyo)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Business Rules	Enrich cloud offerings with decision modeling, management, and execution service.	LOD-BPM-RUL	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
UI Theme Designer	Apply your corporate branding to applications based on SAPUI5 technology.	CA-UI2-THD	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available
UI Theme Designer	Apply your corporate branding to applications based on SAPUI5 technology.	CA-UI2-THD	Cloud Foundry	AWS	Europe (Frankfurt) Brazil (São Paulo) Canada (Montreal) US East (VA) Australia (Sydney) Singapore Japan (Tokyo)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
UI Theme Designer	Apply your corporate branding to applications based on SAPUI5 technology.	CA-UI2-THD	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) Japan (Tokyo) Singapore US East (VA)	Yes	Available
UI Theme Designer	Apply your corporate branding to applications based on SAPUI5 technology.	CA-UI2-THD	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt)	Yes	Available
UI Theme Designer	Apply your corporate branding to applications based on SAPUI5 technology.	CA-UI2-THD	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
SAP Retail Execution	SAP Retail Execution is a mobile app that enables field sales executives to perform Field Sales activities effectively.	CEC-CRM-REX-IOS	Cloud Foundry	AWS	Europe (Frankfurt) EU Access	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Application Autoscaler	Automatically increase or decrease the number of application instances.	BC-CP-CF-AUTOSCALE	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access US East (VA) Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore Canada (Montreal)	Yes	Available
Application Autoscaler	Automatically increase or decrease the number of application instances.	BC-CP-CF-AUTOSCALE	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Singapore Japan (Tokyo) Australia (Sydney)	Yes	Available
Application Autoscaler	Automatically increase or decrease the number of application instances.	BC-CP-CF-AUTOSCALE	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) Israel (Tel Aviv)	Yes	Available
Application Autoscaler	Automatically increase or decrease the number of application instances.	BC-CP-CF-AUTOSCALE	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
Data Retention Manager	Manage retention and residence rules to block or delete personal data.	LOD-GDP-RM	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Australia (Sydney)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Data Retention Manager	Manage retention and residence rules to block or delete personal data.	LOD-GDP-RM	Cloud Foundry	Azure	Europe (Netherlands) US West (WA)	Yes	Available
Tax Service	Determine and calculate indirect taxes to support tax compliance.	LOD-LH-TAX	Neo	SAP	Europe (Rot)* US East (Sterling)		Available
Tax Service	Determine and calculate indirect taxes to support tax compliance.	LOD-LH-TAX	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA)	Yes	Available
Tax Service	Determine and calculate indirect taxes to support tax compliance.	LOD-LH-TAX	Cloud Foundry	Azure	US West (WA)	Yes	Available
Serverless Runtime	Use extensions on SAP Business Technology Platform and get runtime access to OData services in SAP Integration Suite.	BC-CP-XF-SRT	Cloud Foundry	AWS	US East (VA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Serverless Runtime	Use extensions on SAP Business Technology Platform and get runtime access to OData services in SAP Integration Suite.	BC-CP-XF-SRT	Cloud Foundry	Azure	Singapore	Yes	Available
SAP HANA Service	Create and consume SAP HANA databases.	BC-NEO-PERS	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)		Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP HANA Service	Create and consume SAP HANA databases.	HAN-CLS-DB	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore South Korea (Seoul)	Yes	Available
SAP HANA Service	Create and consume SAP HANA databases.	HAN-CLS-DB	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Singapore Japan (Tokyo) Australia (Sydney)	Yes	Available
SAP HANA Service	Create and consume SAP HANA databases.	HAN-CLS-DB	Cloud Foundry	GCP	US Central (IA)	Yes	Available
SAP HANA Service	Create and consume SAP HANA databases.	HAN-CLS-DB	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
SAP Secure Login Service for SAP GUI	Get client certificates via Secure Login Client (SLC) to log-on with SAP GUI.	BC-CP-CF-SEC-SLS		SAP	This service does not run on standard SAP Cloud Platform regions. Check out this note for further details.		Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Platform Identity Provider Service	Use your user base from the identity authentication tenant for admin tasks.	BC-NEO-SEC-IAM	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)		Available
Business Entity Recognition	Detect and highlight entities from unstructured text using machine learning.	CA-ML-BER	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Japan (Tokyo) Australia (Sydney) Europe (Frankfurt) EU Access	Yes	Available
SAP Build Work Zone, standard edition	Simplify access to applications by establishing a central entry point.	EP-WZ-SRB	Cloud Foundry	SAP	Europe (Frankfurt) EUA only UAE (Dubai) Europe (Rot) US West (Colorado) Australia (Sydney) Japan (Tokyo) US (Sterling)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP Build Work Zone, standard edition	Simplify access to applications by establishing a central entry point.	EP-WZ-SRB	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore Canada (Montreal) Europe (Frankfurt) EU Access South Korea (Seoul) US West (Oregon) Europe (Milan)	Yes	Available
SAP Build Work Zone, standard edition	Simplify access to applications by establishing a central entry point.	EP-WZ-SRB	Cloud Foundry	Azure	Singapore US West (WA) US East (VA) Europe (Netherlands) Japan (Tokyo) Australia (Sydney) Switzerland (Zurich) EU Access Brazil (São Paulo) Canada Central (Toronto) China (North 3)** UK South (London) Europe (Frankfurt)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP Build Work Zone, standard edition	Simplify access to applications by establishing a central entry point.	EP-WZ-SRB	Cloud Foundry	GCP	US Central (IA) India (Mumbai) Israel (Tel Aviv) Europe (Frankfurt) KSA (Dammam) public sector Japan (Osaka) Australia (Sydney) KSA (Dammam) Japan (Tokyo) Brazil (São Paulo)	Yes	Available
SAP Build Work Zone, standard edition	Simplify access to applications by establishing a central entry point.	EP-WZ-SRB	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
Service Ticket Intelligence	Build a self-driven customer service powered by machine learning.	CA-ML-STI	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access	Yes	Available
Service Ticket Intelligence	Build a self-driven customer service powered by machine learning.	CA-ML-STI	Cloud Foundry	GCP	US Central (IA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Redis, Hyperscaler Option	Create and consume in-memory cache with Redis based on hyperscalers.	BC-CP-BSB-REDIS	Cloud Foundry	AWS	US East (VA) Japan (Tokyo) Europe (Frankfurt) EU Access Europe (Frankfurt) Canada (Montreal) US East (VA) Brazil (São Paulo) Singapore Australia (Sydney)	Yes	Available
Redis, Hyperscaler Option	Create and consume in-memory cache with Redis based on hyperscalers.	BC-CP-BSB-REDIS	Cloud Foundry	Azure	US East (VA) US West (WA) Japan (Tokyo) Europe (Netherlands) Singapore Australia (Sydney)	Yes	Available
Redis, Hyperscaler Option	Create and consume in-memory cache with Redis based on hyperscalers.	BC-CP-BSB-REDIS	Cloud Foundry	GCP	Israel (Tel Aviv)	Yes	Available
SAP Document AI	Automate your document information extraction processes.	CA-ML-BDP	Cloud Foundry	SAP	UAE (Dubai) Europe (Frankfurt) EUA only Europe (Rot)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP Document AI	Automate your document information extraction processes.	CA-ML-BDP	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access Japan (Tokyo) US East (VA) Australia (Sydney) Singapore South Korea (Seoul) Brazil (São Paulo)	Yes	Available
SAP Document AI	Automate your document information extraction processes.	CA-ML-BDP	Cloud Foundry	Azure	Europe (Netherlands) US East (VA) Australia (Sydney) Singapore	Yes	Available
SAP Document AI	Automate your document information extraction processes.	CA-ML-BDP	Cloud Foundry	GCP	Europe (Frankfurt) US Central (IA) Australia (Sydney) KSA (Dammam) public sector India (Mumbai)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Mobile Service for SAP Fiori	Optimize, build, manage, and monitor SAP Fiori apps on mobile devices.	MOB-FM	Neo	SAP	Europe (Rot)* US East (Ashburn) US West (Chandler) Australia (Sydney) Japan (Tokyo) US East (Sterling) Canada (Toronto) Brazil (São Paulo)	Yes	Available
Agentry	Develop and run Agentry metadata-driven mobile applications.	MOB-CLD-AGS	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Brazil (São Paulo) Canada (Toronto) Japan (Tokyo) UAE (Dubai) KSA (Riyadh)		Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP Process Integration	Seamlessly orchestrate business processes and integrate data in real time.	LOD-HCI-PI-OPS-PROV	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) KSA (Riyadh) UAE (Dubai)		Available
Credential Store	Store and retrieve credentials such as cryptographic keys and passwords.	BC-CP-CF-SEC-CPG	Cloud Foundry	AWS	Europe (Frankfurt) Australia (Sydney) US East (VA) Brazil (São Paulo) Singapore Canada (Montreal) Japan (Tokyo)	Yes	Available
Credential Store	Store and retrieve credentials such as cryptographic keys and passwords.	BC-CP-CF-SEC-CPG	Cloud Foundry	Azure	Japan (Tokyo) Europe (Netherlands) US West (WA) US East (VA) Singapore	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Credential Store	Store and retrieve credentials such as cryptographic keys and passwords.	BC-CP-CF-SEC-CPG	Cloud Foundry	GCP	Europe (Frankfurt) Israel (Tel Aviv)	Yes	Available
Credential Store	Store and retrieve credentials such as cryptographic keys and passwords.	BC-CP-CF-SEC-CPG	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
Identity Authentication	Secure authentication and single sign-on for users in the cloud.	BC-IAM-IDS		SAP	This service does not run on standard SAP Cloud Platform regions. Check out this note for further details.		Available
SAP RealSpend	Manage and track expenses, and compare them against your budget in real time.	LOD-MAP-RS	Neo	SAP	Europe (Rot)* Europe (Frankfurt)		Available
SAP Intelligent Robotic Process Automation	Design, configure, and execute automation projects.	CA-ML-IPA	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access Australia (Sydney) Japan (Tokyo) US East (VA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP Intelligent Robotic Process Automation	Design, configure, and execute automation projects.	CA-ML-IPA	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
SAP ASE Service	Create and consume SAP ASE databases.	BC-NEO-PERS	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available
Personalized Recommendation	Personalized Recommendation draws insights from user data to deliver highly personalized recommendations.	CA-ML-PR	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Audit Log Service	A core, security and compliance based SAP BTP service to provide means for audit purposes.	BC-CP-CF-SEC-AU-DITLG	Cloud Foundry	AWS	Europe (Frankfurt) EU Access	Yes	Available
Audit Log Service	A core, security and compliance based SAP BTP service to provide means for audit purposes.	BC-CP-CF-SEC-AU-DITLG	Cloud Foundry	GCP	Israel (Tel Aviv)	Yes	Available
Custom Domain Service	Configure and expose your application under your own domain.	BC-NEO-INFR	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)		Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Custom Domain Service	Configure and expose your application under your own domain.	BC-CP-CF-SEC-DOM	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access US East (VA) Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore South Korea (Seoul) Canada (Montreal)	Yes	Available
Custom Domain Service	Configure and expose your application under your own domain.	BC-CP-CF-SEC-DOM	Cloud Foundry	Azure	Japan (Tokyo) Europe (Netherlands) US West (WA) US East (VA) Australia (Sydney) Singapore	Yes	Available
Custom Domain Service	Configure and expose your application under your own domain.	BC-CP-CF-SEC-DOM	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt)	Yes	Available
Custom Domain Service	Configure and expose your application under your own domain.	BC-CP-CF-SEC-DOM	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Serverless Runtime	Use extensions on SAP Business Technology Platform and get runtime access to OData services in SAP Integration Suite.	BC-CP-XF-SRT	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access Australia (Sydney) US East (VA) Japan (Tokyo) Australia (Sydney) Singapore South Korea (Seoul) Brazil (São Paulo) Canada (Montreal)	Yes	Available
Serverless Runtime	Use extensions on SAP Business Technology Platform and get runtime access to OData services in SAP Integration Suite.	BC-CP-XF-SRT	Cloud Foundry	Azure	Switzerland (Zurich) EU Access Brazil (São Paulo) Europe (Netherlands) Australia (Sydney) US West (WA) US East (VA) Japan (Tokyo) Singapore	Yes	Available
Serverless Runtime	Use extensions on SAP Business Technology Platform and get runtime access to OData services in SAP Integration Suite.	BC-CP-XF-SRT	Cloud Foundry	GCP	India (Mumbai) Israel (Tel Aviv) KSA (Dammam) public sector Europe (Frankfurt) US Central (IA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Open Connectors	Simplify integration via APIs.	LOD-OCN-OPS	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US West (Chandler) US West (Colorado Springs)	Yes	Available
Open Connectors	Simplify integration via APIs.	LOD-OCN-OPS	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Australia (Sydney) Singapore Canada (Montreal) Brazil (São Paulo) Japan (Tokyo)	Yes	Available
Open Connectors	Simplify integration via APIs.	LOD-OCN-OPS	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Singapore Japan (Tokyo) Australia (Sydney)	Yes	Available
Open Connectors	Simplify integration via APIs.	LOD-OCN-OPS	Cloud Foundry	GCP	US Central (IA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Cloud Portal Service	Create role based, multi-channel sites to access business apps and content.	EP-CPP-NEO-SFC	Neo	SAP	Europe (Rot)* Europe (Frankfurt) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) Europe (Amsterdam) UAE (Dubai) KSA (Riyadh)	Yes	Available
Cloud Portal Service	Create role based, multi-channel sites to access business apps and content.	EP-WZ-SRB	Cloud Foundry	SAP	Europe (Frankfurt) EUA only UAE (Dubai) Europe (Rot) US West (Colorado) Australia (Sydney) Japan (Tokyo) US (Sterling)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Cloud Portal Service	Create role based, multi-channel sites to access business apps and content.	EP-WZ-SRB	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore Canada (Montreal) Europe (Frankfurt) EU Access South Korea (Seoul) US West (Oregon) Europe (Milan)	Yes	Available
Cloud Portal Service	Create role based, multi-channel sites to access business apps and content.	EP-WZ-SRB	Cloud Foundry	Azure	Singapore US West (WA) US East (VA) Europe (Netherlands) Japan (Tokyo) Australia (Sydney) Switzerland (Zurich) EU Access Brazil (São Paulo) Canada Central (Toronto) China (North 3)** UK South (London) Europe (Frankfurt)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Cloud Portal Service	Create role based, multi-channel sites to access business apps and content.	EP-WZ-SRB	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) India (Mumbai) Israel (Tel Aviv) KSA (Dammam) public sector Japan (Osaka) Australia (Sydney) KSA (Dammam) Japan (Tokyo) Brazil (São Paulo)	Yes	Available
Cloud Portal Service	Create role based, multi-channel sites to access business apps and content.	EP-WZ-SRB	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
SAP API Business Hub	Discover, explore and test the APIs offered by SAP.	LOD-CHB		SAP	This service does not run on standard SAP Cloud Platform regions. Check out this note for further details.		Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Git Service	Store and version source code in Git repositories.	BC-NEO-GIT	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available
SAP Conversational AI	Build and deploy innovative chatbots using this comprehensive end-to-end platform.	CA-ML-CAI	Cloud Foundry	AWS	Europe (Frankfurt)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Java Profiling	Profile and analyze your Java applications.	BC-JVM	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Monitoring Service	Monitor Java, SAP HANA XS and HTML5 applications, and databases.	BC-NEO-MON	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available
Destination Service	Retrieve information about destinations in the Cloud Foundry environment.	BC-CP-DEST-CF	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore South Korea (Seoul) Canada (Montreal) US East (VA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Destination Service	Retrieve information about destinations in the Cloud Foundry environment.	BC-CP-DEST-CF	Cloud Foundry	Azure	Japan (Tokyo) Europe (Netherlands) US West (WA) US East (VA) Singapore Australia (Sydney)	Yes	Available
Destination Service	Retrieve information about destinations in the Cloud Foundry environment.	BC-CP-DEST-CF	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) Israel (Tel Aviv)	Yes	Available
Destination Service	Retrieve information about destinations in the Cloud Foundry environment.	BC-CP-DEST-CF	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Mobile Services, users	Build and run mobile apps for B2E and B2B use cases.	MOB-CLD-OPS	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Brazil (São Paulo) Canada (Toronto) KSA (Riyadh) UAE (Dubai)	Yes	Available
Mobile Services, users	Build and run mobile apps for B2E and B2B use cases.	MOB-CLD-OPS	Cloud Foundry	SAP	Europe (Frankfurt) EUA only Australia (Sydney) UAE (Dubai) US West (Colorado) Europe (Rot) Japan (Tokyo)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Mobile Services, users	Build and run mobile apps for B2E and B2B use cases.	MOB-CLD-OPS	Cloud Foundry	AWS	Australia (Sydney) Brazil (São Paulo) Japan (Tokyo) US East (VA) Europe (Frankfurt) Singapore Canada (Montreal) South Korea (Seoul) Europe (Frankfurt) EU Access US West (Oregon) Europe (Milan)	Yes	Available
Mobile Services, users	Build and run mobile apps for B2E and B2B use cases.	MOB-CLD-OPS	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) Japan (Tokyo) US East (VA) Singapore Switzerland (Zurich) EU Access Brazil (São Paulo) Canada Central (Toronto)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Mobile Services, users	Build and run mobile apps for B2E and B2B use cases.	MOB-CLD-OPS	Cloud Foundry	GCP	Europe (Frankfurt) US Central (IA) India (Mumbai) Israel (Tel Aviv) KSA (Dammam) public sector Japan (Osaka) Brazil (São Paulo) Australia (Sydney) KSA (Dammam) Japan (Tokyo)	Yes	Available
Mobile Services, users	Build and run mobile apps for B2E and B2B use cases.	MOB-CLD-OPS	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
OAuth 2.0	Protect applications and APIs with OAuth 2.0.	BC-NEO-SEC-IAM	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available
Cloud Transport Management	Manage transports of development artifacts and application-specific content.	BC-CP-LCM-TMS	Cloud Foundry	SAP	UAE (Dubai) Europe (Frankfurt) EUA only Europe (Rot) US (Sterling) US West (Colorado) Australia (Sydney) Japan (Tokyo)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Cloud Transport Management	Manage transports of development artifacts and application-specific content.	BC-CP-LCM-TMS	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access Europe (Milan) US East (VA) US West (Oregon) Australia (Sydney) Singapore South Korea (Seoul) Canada (Montreal) Brazil (São Paulo) Japan (Tokyo)	Yes	Available
Cloud Transport Management	Manage transports of development artifacts and application-specific content.	BC-CP-LCM-TMS	Cloud Foundry	Azure	Europe (Netherlands) Europe (Frankfurt) US West (WA) US East (VA) Australia (Sydney) Singapore Canada Central (Toronto) Japan (Tokyo) Switzerland (Zurich) EU Access UK South (London)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Cloud Transport Management	Manage transports of development artifacts and application-specific content.	BC-CP-LCM-TMS	Cloud Foundry	GCP	Europe (Frankfurt) US Central (IA) India (Mumbai) Japan (Tokyo) KSA (Dammam) public sector KSA (Dammam) Israel (Tel Aviv)	Yes	Available
Mobile Services, consumers	Build and run mobile apps for B2C use cases.	MOB-CLD-OPS	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Brazil (São Paulo) Canada (Toronto) KSA (Riyadh) UAE (Dubai)		Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Mobile Services, consumers	Build and run mobile apps for B2C use cases.	MOB-CLD-OPS	Cloud Foundry	SAP	Europe (Frankfurt) EUA only Australia (Sydney) UAE (Dubai) US West (Colorado) Europe (Rot) Japan (Tokyo)	Yes	Available
Mobile Services, consumers	Build and run mobile apps for B2C use cases.	MOB-CLD-OPS	Cloud Foundry	AWS	Australia (Sydney) Brazil (São Paulo) Japan (Tokyo) US East (VA) Europe (Frankfurt) Singapore Canada (Montreal) South Korea (Seoul) Europe (Frankfurt) EU Access US West (Oregon) Europe (Milan)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Mobile Services, consumers	Build and run mobile apps for B2C use cases.	MOB-CLD-OPS	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) Japan (Tokyo) US East (VA) Singapore Switzerland (Zurich) EU Access Brazil (São Paulo) Canada Central (Toronto)	Yes	Available
Mobile Services, consumers	Build and run mobile apps for B2C use cases.	MOB-CLD-OPS	Cloud Foundry	GCP	Europe (Frankfurt) US Central (IA) India (Mumbai) Israel (Tel Aviv) KSA (Dammam) public sector Japan (Osaka) Brazil (São Paulo) Australia (Sydney) KSA (Dammam) Japan (Tokyo)	Yes	Available
Mobile Services, consumers	Build and run mobile apps for B2C use cases.	MOB-CLD-OPS	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Java Debugging	Debug your Java application even through networks with high latency.	BC-JVM	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available
PostgreSQL, Hyperscaler Option	Create and consume relational databases with PostgreSQL based on hyperscalers.	BC-CP-BSB-POSTGRESQL	Cloud Foundry	AWS	Australia (Sydney) Singapore Brazil (São Paulo) Canada (Montreal) Europe (Frankfurt) Europe (Frankfurt) EU Access Japan (Tokyo) US East (VA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
PostgreSQL, Hyperscaler Option	Create and consume relational databases with PostgreSQL based on hyperscalers.	BC-CP-BSB-POSTGRESQL	Cloud Foundry	Azure	Australia (Sydney) Singapore Europe (Netherlands) Japan (Tokyo) US West (WA) US East (VA)	Yes	Available
PostgreSQL, Hyperscaler Option	Create and consume relational databases with PostgreSQL based on hyperscalers.	BC-CP-BSB-POSTGRESQL	Cloud Foundry	GCP	Israel (Tel Aviv)	Yes	Available
SAP Smart Business Service	Expose KPIs and OPIs as SAP Fiori applications without the need to write any code.	CA-GTF-SB-HCP	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP Smart Business Service	Expose KPIs and OPIs as SAP Fiori applications without the need to write any code.	CA-GTF-SB-HCP	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA)	Yes	Available
SAP SuccessFactors Extensibility	Automates the connectivity configuration of extension applications running on SAP BTP to SAP SuccessFactors systems.	BC-NEO-EXT-SF	Cloud Foundry	AWS	Australia (Sydney) Singapore South Korea (Seoul) Brazil (São Paulo) Canada (Montreal) Europe (Frankfurt) Europe (Frankfurt) EU Access Japan (Tokyo) US East (VA)	Yes	Available
SAP SuccessFactors Extensibility	Automates the connectivity configuration of extension applications running on SAP BTP to SAP SuccessFactors systems.	BC-NEO-EXT-SF	Cloud Foundry	Azure	Australia (Sydney) Singapore Brazil (São Paulo) Switzerland (Zurich) EU Access Europe (Netherlands) Japan (Tokyo) US West (WA) US East (VA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP SuccessFactors Extensibility	Automates the connectivity configuration of extension applications running on SAP BTP to SAP SuccessFactors systems.	BC-NEO-EXT-SF	Cloud Foundry	GCP	Australia (Sydney) Brazil (São Paulo) Europe (Frankfurt) Japan (Osaka) India (Mumbai) US Central (IA)	Yes	Available
HTML5 Application Repository Service	Develop and run HTML5 applications in a cloud environment.	BC-CP-CF-HTML5	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access US East (VA) Australia (Sydney) Singapore South Korea (Seoul) Brazil (São Paulo) Canada (Montreal) Japan (Tokyo)	Yes	Available
HTML5 Application Repository Service	Develop and run HTML5 applications in a cloud environment.	BC-CP-CF-HTML5	Cloud Foundry	Azure	Japan (Tokyo) Europe (Netherlands) US West (WA) US East (VA) Singapore	Yes	Available
HTML5 Application Repository Service	Develop and run HTML5 applications in a cloud environment.	BC-CP-CF-HTML5	Cloud Foundry	GCP	US Central (IA) Israel (Tel Aviv)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
HTML5 Application Repository Service	Develop and run HTML5 applications in a cloud environment.	BC-CP-CF-HTML5	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
SAP Document Center	Use uniform standard-based file access and mobilize your business content.	BC-NEO-ECM-APP	Neo	SAP	Europe (Rot)* Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available
Kyma Runtime	Develop and run containerized applications and extensions on Kubernetes.	BC-CP-XF-KYMA	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Europe (Frankfurt) EU Access Brazil (São Paulo) Australia (Sydney) Japan (Tokyo) Canada (Montreal) Singapore South Korea (Seoul)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Kyma Runtime	Develop and run containerized applications and extensions on Kubernetes.	BC-CP-XF-KYMA	Cloud Foundry	GCP	Israel (Tel Aviv)	Yes	Available
Data Enrichment Service	Create or enrich master data using trusted third-party data.	LOD-MDM-DE	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Singapore	Yes	Available
Invoicing option for Peppol	Meet compliance requirements by exchanging documents with the Peppol network.	LOD-LH-DCS-PAP	Neo	SAP	Europe (Rot)*		Available
SAP Intelligent Product Design	Generate, analyze and validate product requirements using intelligent tools.	PLM-DC	Neo	SAP	Europe (Rot)* Europe (Frankfurt)		Available
SAP Intelligent Product Design	Generate, analyze and validate product requirements using intelligent tools.	PLM-DC	Cloud Foundry	AWS	Europe (Frankfurt)	Yes	Available
Service Manager	The central registry for service brokers and platforms in SAP BTP.	BC-NEO-SVCMGR	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore Canada (Montreal)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Service Manager	The central registry for service brokers and platforms in SAP BTP.	BC-NEO-SVCMGR	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Japan (Tokyo) Singapore	Yes	Available
Service Manager	The central registry for service brokers and platforms in SAP BTP.	BC-NEO-SVCMGR	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) Israel (Tel Aviv)	Yes	Available
SAP Variant Configuration and Pricing	Configure and price your SAP ERP or SAP S/4HANA products interactively in the cloud	LOD-CPS	Cloud Foundry	SAP	Europe (Rot)	Yes	Available
SAP Variant Configuration and Pricing	Configure and price your SAP ERP or SAP S/4HANA products interactively in the cloud	LOD-CPS	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Singapore	Yes	Available
SAP Variant Configuration and Pricing	Configure and price your SAP ERP or SAP S/4HANA products interactively in the cloud	LOD-CPS	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) Australia (Sydney)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP Variant Configuration and Pricing	Configure and price your SAP ERP or SAP S/4HANA products interactively in the cloud	LOD-CPS	Cloud Foundry	GCP	Europe (Frankfurt) US Central (IA) Australia (Sydney) India (Mumbai) KSA (Dammam) public sector KSA (Dammam)	Yes	Available
SAP HANA Cloud	A single gateway to all your data.	HAN-CLS-HC	Cloud Foundry	AWS	Australia (Sydney) Singapore South Korea (Seoul) Brazil (São Paulo) Canada (Montreal) Europe (Frankfurt) Europe (Frankfurt) EU Access Japan (Tokyo) US East (VA)	Yes	Available
SAP HANA Cloud	A single gateway to all your data.	HAN-CLS-HC	Cloud Foundry	Azure	Australia (Sydney) Singapore Switzerland (Zurich) EU Access Europe (Netherlands) Japan (Tokyo) US West (WA) US East (VA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP HANA Cloud	A single gateway to all your data.	HAN-CLS-HC	Cloud Foundry	GCP	Europe (Frankfurt) India (Mumbai) US Central (IA)	Yes	Available
Mobile Service for App and Device Management	Manage your mobile devices.	MOB-SEC	Neo	SAP	Europe (Rot)* US East (Ashburn) US West (Chandler) Australia (Sydney) Japan (Tokyo) US East (Sterling)	Yes	Available
Document Classification	Classify business documents automatically using machine learning.	CA-ML-BDP	Cloud Foundry	AWS	Europe (Frankfurt)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Authorization and Trust Management Service	Manage application authorizations and trusted connections to identity providers.	BC-NEO-SEC-IAM	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available
Authorization and Trust Management Service	Manage application authorizations and trusted connections to identity providers.	BC-CP-CF-SEC-IAM	Cloud Foundry	AWS	US East (VA) Europe (Frankfurt) Europe (Frankfurt) EU Access Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore South Korea (Seoul) Canada (Montreal)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Authorization and Trust Management Service	Manage application authorizations and trusted connections to identity providers.	BC-CP-CF-SEC-IAM	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Australia (Sydney) Singapore Japan (Tokyo)	Yes	Available
Authorization and Trust Management Service	Manage application authorizations and trusted connections to identity providers.	BC-CP-CF-SEC-IAM	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) Israel (Tel Aviv)	Yes	Available
Authorization and Trust Management Service	Manage application authorizations and trusted connections to identity providers.	BC-CP-CF-SEC-IAM	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
SAP IoT	Put raw sensor data into business context and leverage it in analytical or transactional applications.	IOT-BSV-APB	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA)	Yes	Available
SAP IoT	Put raw sensor data into business context and leverage it in analytical or transactional applications.	IOT-BSV-APB	Cloud Foundry	Azure	Europe (Netherlands)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Event Mesh	Connect applications, services and systems across different landscapes.	BC-CP-EM-MES	Cloud Foundry	AWS	US East (VA) Australia (Sydney) Singapore Brazil (São Paulo) Japan (Tokyo) Europe (Frankfurt) Canada (Montreal)	Yes	Available
Event Mesh	Connect applications, services and systems across different landscapes.	BC-CP-EM-MES	Cloud Foundry	Azure	Japan (Tokyo) Europe (Netherlands) US West (WA) US East (VA) Singapore	Yes	Available
Event Mesh	Connect applications, services and systems across different landscapes.	BC-CP-EM-MES	Cloud Foundry	GCP	Europe (Frankfurt) Israel (Tel Aviv)	Yes	Available
Event Mesh	Connect applications, services and systems across different landscapes.	BC-CP-EM-MES	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Integration Suite	Integrate applications, services, and systems across landscapes.	LOD-HCI-PI	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Brazil (São Paulo) Canada (Montreal) Australia (Sydney) Singapore Japan (Tokyo)	Yes	Available
Integration Suite	Integrate applications, services, and systems across landscapes.	LOD-HCI-PI	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Singapore Japan (Tokyo)	Yes	Available
Integration Suite	Integrate applications, services, and systems across landscapes.	LOD-HCI-PI	Cloud Foundry	GCP	Israel (Tel Aviv)	Yes	Available
Integration Suite	Integrate applications, services, and systems across landscapes.	LOD-HCI-PI	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Streaming Analytics	Process continuous streams of event data in real time and act on the results.	HAN-SDS	Neo	SAP	Europe (Rot)* US East (Ashburn) US West (Chandler) US East (Sterling) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) KSA (Riyadh) Europe (Amsterdam) Europe (Frankfurt) UAE (Dubai)		Available
Document Management Service	Build API and UI based document management capabilities for your business apps.	BC-CP-CF-SDM	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Japan (Tokyo) Australia (Sydney) Singapore Brazil (São Paulo) US West (Oregon) Europe (Frankfurt) EU Access South Korea (Seoul) Canada (Montreal)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Document Management Service	Build API and UI based document management capabilities for your business apps.	BC-CP-CF-SDM	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Japan (Tokyo) Singapore Switzerland (Zurich) EU Access Australia (Sydney) Brazil (São Paulo)	Yes	Available
Document Management Service	Build API and UI based document management capabilities for your business apps.	BC-CP-CF-SDM	Cloud Foundry	GCP	Europe (Frankfurt) US Central (IA) India (Mumbai) Israel (Tel Aviv) KSA (Dammam) public sector Japan (Osaka) Australia (Sydney)	Yes	Available
Document Management Service	Build API and UI based document management capabilities for your business apps.	BC-CP-CF-SDM	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP Web IDE Full-Stack	Create and extend SAP full-stack applications for browsers and mobile devices.	CA-WDE-PLFRM	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Brazil (São Paulo) Canada (Toronto) UAE (Dubai) KSA (Riyadh)	Yes	Available
Feature Flags Service	Control the rollout of new features.	BC-CP-CF-FEATUREFLG	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access Australia (Sydney) US East (VA) Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore South Korea (Seoul) Canada (Montreal)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Feature Flags Service	Control the rollout of new features.	BC-CP-CF-FEATUREFLG	Cloud Foundry	Azure	Japan (Tokyo) Europe (Netherlands) US West (WA) US East (VA) Australia (Sydney) Singapore	Yes	Available
Feature Flags Service	Control the rollout of new features.	BC-CP-CF-FEATUREFLG	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt)	Yes	Available
Feature Flags Service	Control the rollout of new features.	BC-CP-CF-FEATUREFLG	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
Keystore Service	Manage cryptographic keys and certificates.	BC-NEO-SEC-CPG	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Administration Service	Administer the configuration and pricing services	LOD-CPS	Cloud Foundry	SAP	Europe (Rot)	Yes	Available
Administration Service	Administer the configuration and pricing services	LOD-CPS	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Singapore	Yes	Available
Administration Service	Administer the configuration and pricing services	LOD-CPS	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) Australia (Sydney)	Yes	Available
Administration Service	Administer the configuration and pricing services	LOD-CPS	Cloud Foundry	GCP	Europe (Frankfurt) US Central (IA) Australia (Sydney) India (Mumbai) KSA (Dammam) public sector KSA (Dammam)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Virtual Machine Service	Virtualized hardware resources (CPU, RAM, disk space, installed OS) to install and maintain the Linux-based software.	BC-NEO-INFR-VM	Neo	SAP	Europe (Rot)* Europe (Amsterdam) Europe (Frankfurt) Australia (Sydney) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh) Japan (Tokyo) Canada (Toronto)		Available
SAP S/4HANA Cloud Extensibility	Automates the connectivity configuration of extension applications running on SAP BTP to SAP S/4HANA Cloud systems.	BC-NEO-EXT-S4C	Cloud Foundry	AWS	Australia (Sydney) Singapore South Korea (Seoul) Brazil (São Paulo) Canada (Montreal) Europe (Frankfurt) Europe (Frankfurt) EU Access Japan (Tokyo) US East (VA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP S/4HANA Cloud Extensibility	Automates the connectivity configuration of extension applications running on SAP BTP to SAP S/4HANA Cloud systems.	BC-NEO-EXT-S4C	Cloud Foundry	Azure	Australia (Sydney) Singapore Brazil (São Paulo) Switzerland (Zurich) EU Access Europe (Netherlands) Japan (Tokyo) US West (WA) US East (VA)	Yes	Available
SAP S/4HANA Cloud Extensibility	Automates the connectivity configuration of extension applications running on SAP BTP to SAP S/4HANA Cloud systems.	BC-NEO-EXT-S4C	Cloud Foundry	GCP	Australia (Sydney) Brazil (São Paulo) Europe (Frankfurt) Japan (Osaka) India (Mumbai) US Central (IA)	Yes	Available
SAP Leonardo ML Foundation	Infuse your applications with intelligent, easy-to-use services based on Machine Learning.	CA-ML-PLT	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access US East (VA) Japan (Tokyo)	Yes	Available
SAP Leonardo ML Foundation	Infuse your applications with intelligent, easy-to-use services based on Machine Learning.	CA-ML-PLT	Cloud Foundry	GCP	US Central (IA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Alert Notification Service	Create and receive real-time alerts about your services	BC-CP-LCM-ANS	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Japan (Tokyo) Canada (Toronto) Australia (Sydney) KSA (Riyadh) UAE (Dubai) Brazil (São Paulo)	Yes	Available
Alert Notification Service	Create and receive real-time alerts about your services	BC-CP-LCM-ANS	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Japan (Tokyo) Singapore Australia (Sydney) Canada (Montreal) Brazil (São Paulo) Europe (Frankfurt) EU Access South Korea (Seoul)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Alert Notification Service	Create and receive real-time alerts about your services	BC-CP-LCM-ANS	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Singapore Japan (Tokyo) Australia (Sydney) Switzerland (Zurich) EU Access	Yes	Available
Alert Notification Service	Create and receive real-time alerts about your services	BC-CP-LCM-ANS	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) India (Mumbai) Israel (Tel Aviv)	Yes	Available
SAP Business Application Studio	Develop, debug, test, and deploy SAP business applications.	CA-BAS	Cloud Foundry	SAP	Europe (Frankfurt) EUA only UAE (Dubai) Europe (Rot) Japan (Tokyo) Australia (Sydney) US (Sterling) US West (Colorado)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP Business Application Studio	Develop, debug, test, and deploy SAP business applications.	CA-BAS	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Australia (Sydney) Canada (Montreal) Japan (Tokyo) Brazil (São Paulo) Singapore Europe (Frankfurt) EU Access South Korea (Seoul) US West (Oregon) Europe (Milan)	Yes	Available
SAP Business Application Studio	Develop, debug, test, and deploy SAP business applications.	CA-BAS	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Japan (Tokyo) Singapore Switzerland (Zurich) EU Access Brazil (São Paulo) Canada Central (Toronto) China (North 3)** Europe (Frankfurt)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP Business Application Studio	Develop, debug, test, and deploy SAP business applications.	CA-BAS	Cloud Foundry	GCP	US Central (IA) Israel (Tel Aviv) India (Mumbai) KSA (Dammam) public sector Japan (Osaka) Australia (Sydney) Europe (Frankfurt) KSA (Dammam) Japan (Tokyo)	Yes	Available
SAP Business Application Studio	Develop, debug, test, and deploy SAP business applications.	CA-BAS	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
Identity Authentication - Additional Tenant	Secure authentication and single sign-on for users in the cloud.	BC-IAM-IDS		SAP	This service does not run on standard SAP Cloud Platform regions. Check out this note for further details.		Available
SAP Landscape Management Cloud	Manage your SAP systems in infrastructure as a service (IaaS) environments.	BC-VCM-LMC	Cloud Foundry	AWS	Europe (Frankfurt)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Job Scheduling Service	Define and manage your jobs or Cloud Foundry tasks that run on one-time or recurring schedules.	BC-CP-CF-JBS	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access US East (VA) Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore South Korea (Seoul) Canada (Montreal)	Yes	Available
Job Scheduling Service	Define and manage your jobs or Cloud Foundry tasks that run on one-time or recurring schedules.	BC-CP-CF-JBS	Cloud Foundry	Azure	Japan (Tokyo) Europe (Netherlands) US West (WA) US East (VA) Australia (Sydney) Singapore	Yes	Available
Job Scheduling Service	Define and manage your jobs or Cloud Foundry tasks that run on one-time or recurring schedules.	BC-CP-CF-JBS	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) Israel (Tel Aviv)	Yes	Available
Job Scheduling Service	Define and manage your jobs or Cloud Foundry tasks that run on one-time or recurring schedules.	BC-CP-CF-JBS	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Document Service	Store and manage your documents.	BC-NEO-ECM-DS	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available
Workflow Management	Digitize workflows, manage decisions and gain end-to-end process visibility.	LOD-BPM-PFS	Cloud Foundry	SAP	Europe (Frankfurt) EUA only Europe (Rot) UAE (Dubai) Japan (Tokyo) Australia (Sydney) US West (Colorado) US (Sterling)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Workflow Management	Digitize workflows, manage decisions and gain end-to-end process visibility.	LOD-BPM-PFS	Cloud Foundry	AWS	Europe (Frankfurt) Australia (Sydney) US East (VA) Singapore Japan (Tokyo) Brazil (São Paulo) South Korea (Seoul) Europe (Frankfurt) EU Access Canada (Montreal) US West (Oregon) Europe (Milan)	Yes	Available
Workflow Management	Digitize workflows, manage decisions and gain end-to-end process visibility.	LOD-BPM-PFS	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Singapore Japan (Tokyo) Australia (Sydney) Switzerland (Zurich) EU Access Brazil (São Paulo) Canada Central (Toronto) China (North 3)** UK South (London) Europe (Frankfurt)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Workflow Management	Digitize workflows, manage decisions and gain end-to-end process visibility.	LOD-BPM-PFS	Cloud Foundry	GCP	US Central (IA) India (Mumbai) Israel (Tel Aviv) KSA (Dammam) public sector Japan (Osaka) Australia (Sydney) KSA (Dammam) Japan (Tokyo)	Yes	Available
Workflow Management	Digitize workflows, manage decisions and gain end-to-end process visibility.	LOD-BPM-PFS	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
UI5 flexibility for key users	Add UI adaptation to your UI5 applications.	CA-UI5-FL-CLS	Cloud Foundry	SAP	Europe (Frankfurt) EUA only Australia (Sydney) UAE (Dubai) US West (Colorado) Europe (Rot) Japan (Tokyo) US (Sterling)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
UI5 flexibility for key users	Add UI adaptation to your UI5 applications.	CA-UI5-FL-CLS	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Japan (Tokyo) Singapore South Korea (Seoul) Australia (Sydney) Canada (Montreal) Europe (Frankfurt) EU Access Brazil (São Paulo) US West (Oregon) Europe (Milan)	Yes	Available
UI5 flexibility for key users	Add UI adaptation to your UI5 applications.	CA-UI5-FL-CLS	Cloud Foundry	Azure	Europe (Netherlands) Australia (Sydney) US West (WA) US East (VA) Singapore Japan (Tokyo) Switzerland (Zurich) EU Access Brazil (São Paulo) China (North 3)** Canada Central (Toronto) UK South (London) Europe (Frankfurt)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
UI5 flexibility for key users	Add UI adaptation to your UI5 applications.	CA-UI5-FL-CLS	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) India (Mumbai) Japan (Osaka) KSA (Dammam) public sector Israel (Tel Aviv) Australia (Sydney) KSA (Dammam) Japan (Tokyo) Brazil (São Paulo)	Yes	Available
Java Apps Lifecycle Management	Manage the lifecycle of Java applications by using a REST API.	BC-NEO-DPL	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Pricing service	Calculate prices for configurable- and non-configurable products	LOD-CPS	Cloud Foundry	SAP	Europe (Rot)	Yes	Available
Pricing service	Calculate prices for configurable- and non-configurable products	LOD-CPS	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Singapore	Yes	Available
Pricing service	Calculate prices for configurable- and non-configurable products	LOD-CPS	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) Australia (Sydney)	Yes	Available
Pricing service	Calculate prices for configurable- and non-configurable products	LOD-CPS	Cloud Foundry	GCP	Europe (Frankfurt) US Central (IA) Australia (Sydney) India (Mumbai) KSA (Dammam) public sector KSA (Dammam)	Yes	Available
SAP HANA spatial services	SAP HANA spatial services provides a set of APIs for location-based services.	BC-CP-CF-HSS	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
API Management	Expose your data and processes as APIs and manage their lifecycles.	OPU-API-OD	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) Australia (Sydney) Japan (Tokyo) Brazil (São Paulo) Canada (Toronto) UAE (Dubai) KSA (Riyadh)	Yes	Available
API Management	Expose your data and processes as APIs and manage their lifecycles.	OPU-API-OD	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Singapore Japan (Tokyo) Australia (Sydney) Brazil (São Paulo) Canada (Montreal)	Yes	Available
API Management	Expose your data and processes as APIs and manage their lifecycles.	OPU-API-OD	Cloud Foundry	Azure	Japan (Tokyo) Europe (Netherlands) US West (WA) US East (VA) Singapore Japan (Tokyo)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
API Management	Expose your data and processes as APIs and manage their lifecycles.	OPU-API-OD	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
Workflow	Automate business processes using workflow technology.	LOD-BPM-WFS	Neo	SAP	UAE (Dubai) Australia (Sydney) Brazil (São Paulo) Canada (Toronto) Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) Japan (Tokyo) KSA (Riyadh) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs)		Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Workflow	Automate business processes using workflow technology.	LOD-BPM-WFS	Cloud Foundry	AWS	Australia (Sydney) Singapore South Korea (Seoul) Brazil (São Paulo) Canada (Montreal) Europe (Frankfurt) Europe (Frankfurt) EU Access Japan (Tokyo) US East (VA)	Yes	Available
Workflow	Automate business processes using workflow technology.	LOD-BPM-WFS	Cloud Foundry	Azure	Australia (Sydney) Singapore Europe (Netherlands) Japan (Tokyo) US West (WA) US East (VA)	Yes	Available
Workflow	Automate business processes using workflow technology.	LOD-BPM-WFS	Cloud Foundry	GCP	Europe (Frankfurt) US Central (IA)	Yes	Available
Workflow	Automate business processes using workflow technology.	LOD-BPM-WFS	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Cloud Logging	Store and analyze application logs, metrics, and traces based on OpenSearch.	BC-CP-CLS	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore South Korea (Seoul) Canada (Montreal) US East (VA)	Yes	Available
Cloud Logging	Store and analyze application logs, metrics, and traces based on OpenSearch.	BC-CP-CLS	Cloud Foundry	Azure	Japan (Tokyo) Europe (Netherlands) US West (WA) US East (VA) Australia (Sydney) Singapore	Yes	Available
Cloud Logging	Store and analyze application logs, metrics, and traces based on OpenSearch.	BC-CP-CLS	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) India (Mumbai)	Yes	Available
Cloud Logging	Store and analyze application logs, metrics, and traces based on OpenSearch.	BC-CP-CLS	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
SAP HANA spatial services	SAP HANA spatial services provides a set of APIs for location-based services.	BC-CP-CF-HSS	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA) Australia (Sydney)	Yes	Available
SAP HANA spatial services	SAP HANA spatial services provides a set of APIs for location-based services.	BC-CP-CF-HSS	Cloud Foundry	Azure	Europe (Netherlands)	Yes	Available
OData Provisioning	Access data in SAP Business Suite using OData services.	OPU-GW-OD-FW	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) KSA (Riyadh) Canada (Toronto) Brazil (São Paulo)	Yes	Available
OData Provisioning	Access data in SAP Business Suite using OData services.	OPU-GW-OD-FW	Cloud Foundry	AWS	Europe (Frankfurt) US East (VA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
OData Provisioning	Access data in SAP Business Suite using OData services.	OPU-GW-OD-FW	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) Japan (Tokyo)	Yes	Available
Cloud Integration for Data Services	Integrate data between on-premise and cloud applications on a scheduled (batch) basis.	LOD-HCI-DS	Neo	SAP	UAE (Dubai) Australia (Sydney) Europe (Rot)* Japan (Tokyo) KSA (Riyadh) US West (Colorado Springs)		Available
Connectivity Service	Establish connections between cloud applications and on-premise systems.	BC-NEO-CON	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Connectivity Service	Establish connections between cloud applications and on-premise systems.	BC-CP-CON-CF	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access US East (VA) Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore South Korea (Seoul) Canada (Montreal)	Yes	Available
Connectivity Service	Establish connections between cloud applications and on-premise systems.	BC-CP-CON-CF	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Singapore Japan (Tokyo) Australia (Sydney)	Yes	Available
Connectivity Service	Establish connections between cloud applications and on-premise systems.	BC-CP-CON-CF	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) Israel (Tel Aviv)	Yes	Available
Connectivity Service	Establish connections between cloud applications and on-premise systems.	BC-CP-CON-CF	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Data Quality Services	Embed data quality services to validate addresses and enrich with geocodes.	EIM-DQM-SVS	Neo	SAP	Europe (Rot)* Europe (Frankfurt)	Yes	Available
Application Logging Service	Create, store, access, and analyze application logs.	BC-NEO-LOG	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) Australia (Sydney) Japan (Tokyo) Canada (Toronto) Brazil (São Paulo) UAE (Dubai) KSA (Riyadh)	Yes	Available
Application Logging Service	Create, store, access, and analyze application logs.	BC-CP-CF-APPLOG	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access US East (VA) Brazil (São Paulo) Japan (Tokyo) Australia (Sydney) Singapore South Korea (Seoul) Canada (Montreal)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Application Logging Service	Create, store, access, and analyze application logs.	BC-CP-CF-APPLOG	Cloud Foundry	Azure	Europe (Netherlands) US West (WA) US East (VA) Singapore Japan (Tokyo) Australia (Sydney)	Yes	Available
Application Logging Service	Create, store, access, and analyze application logs.	BC-CP-CF-APPLOG	Cloud Foundry	GCP	US Central (IA) Europe (Frankfurt) Israel (Tel Aviv)	Yes	Available
Application Logging Service	Create, store, access, and analyze application logs.	BC-CP-CF-APPLOG	Cloud Foundry	Alibaba	China (Shanghai)**	Yes	Available
SAP Direct Tax Management	Manage your SAP Direct Tax Management inventory with ease.	LOD-PER-DTM	Cloud Foundry	AWS	Europe (Frankfurt)	Yes	Available
SAP Direct Tax Management	Manage your SAP Direct Tax Management inventory with ease.	LOD-PER-DTM	Cloud Foundry	Azure	Europe (Netherlands)	Yes	Available
SAP Direct Tax Management	Manage your SAP Direct Tax Management inventory with ease.	LOD-PER-DTM	Cloud Foundry	GCP	US Central (IA)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Data Attribute Recommendation	Apply machine learning to predict and classify data records.	CA-ML-DAR	Cloud Foundry	AWS	Europe (Frankfurt) Europe (Frankfurt) EU Access Australia (Sydney) US East (VA) Japan (Tokyo) Singapore Brazil (São Paulo)	Yes	Available
Data Attribute Recommendation	Apply machine learning to predict and classify data records.	CA-ML-DAR	Cloud Foundry	Azure	Europe (Netherlands) US East (VA) Australia (Sydney)	Yes	Available
Data Attribute Recommendation	Apply machine learning to predict and classify data records.	CA-ML-DAR	Cloud Foundry	GCP	Europe (Frankfurt) US Central (IA) India (Mumbai) KSA (Dammam) public sector Australia (Sydney)	Yes	Available

Service Name	Short Description	Support Component	Environment	Infrastructure	Region	Available as Trial	Release Status
Dynatrace Agent Activation	Connect your Java applications to a Dynatrace SaaS monitoring environment.	BC-NEO-MON-APM	Neo	SAP	Europe (Rot)* Europe (Frankfurt) Europe (Amsterdam) US East (Ashburn) US West (Chandler) US East (Sterling) US West (Colorado Springs) Australia (Sydney) Japan (Tokyo) Brazil (São Paulo) Canada (Toronto) UAE (Dubai) KSA (Riyadh)	Yes	Available

Additional Components

Tools

Name	Support Component
SAP BTP Cockpit	BC-CP-CPT BC-CP-CF-CPT [Cloud Foundry environment UI]
SAP BTP Command Line Interface	BC-CP-TOOLS-CLI
SAP BTP Console Client	BC-NEO-CMDTOOL
SAP UI Development Toolkit for HTML5 (SAPUI5)	CA-WDE
SDK for SAP BTP, Neo Environment	BC-NEO-SDK
SAP HANA Cockpit 2.0	HAN-CLS-CPT

Software Logistics

Name	Support Component
Deployment (Neo Environment)	BC-NEO-DPL
Deployment (Cloud Foundry Environment)	BC-XS-SL-DS

Other

Name	Support Component
Infrastructure (Neo Environment)	BC-NEO-IT-NW
Infrastructure (Cloud Foundry Environment)	BC-CP-CF
SAP Cloud Management Service	BC-NEO-CIS
Central Disaster Recovery services	BC-CP-CDR
SAP Usage Data Management Service	BC-NEO-MET-REP
SAP Audit Log	BC-NEO-AUDITLOG (Neo environment) BC-CP-CF-SEC-AUDITLG (Cloud Foundry environment)
SAP Streaming Analytics	HAN-SDS
SAP Cloud Application Programming Model (CAP)	BC-XS-CDX BC-XS-CDX-COR (Compiler and CDS Language) BC-XS-CDX-JAV (Java Runtime) BC-XS-CDX-NJS (Node.js Runtime) BC-XS-CDX-TLS (Tools, IDEs, Build, Deployment)

9 Glossary

SAP BTP Terminology

A-G

application	Software hosted on SAP BTP and used by business users to fulfill certain tasks. Applications are created by developers and might make use of services offered on the platform.
application router [page 453]	The single point-of-entry for an application running in the Cloud Foundry environment on SAP BTP. The application router is used to serve static content, authenticate users, rewrite URLs, and forward or proxy requests to other micro services while propagating user information.
availability	The durability and operational performance without failure of a system or component for an agreed amount of time, which is defined in the contract.
availability zone	A physically separate location with its own power supply, network, and cooling. Within a region, it serves as an individual failure domain. If one of the availability zones fails, the survival of the region is ensured.
block-hours	An hour regarded in terms of the run time of a single compute block. For example, a single compute block running for 10 hours would give 10 block-hours while 2 compute blocks running for the same period would give 20-block hours.
boosters [page 282]	A set of guided interactive steps that enable you to select, configure, and consume services on SAP BTP to achieve a specific technical goal.
buildpack [page 63]	In the Cloud Foundry environment, buildpacks provide framework and runtime support for apps.
business service [page 505]	Platform services that enable, facilitate, or accelerate the development of business process components and elements of an application.
cockpit [page 151]	The central point of entry to key information about your accounts and applications, and for managing all activities associated with your account.
connectivity	Provides a secure, reliable, and easy-to-consume access to business systems or remote services, running either on-premise or in the cloud.
Cloud Foundry CLI	The Cloud Foundry Command Line Interface (cf CLI) is used to deploy and manage your applications in the Cloud Foundry environment.
cloud connector	Cloud Connector serves as the link between on-demand applications in SAP BTP and existing on-premise systems. It combines an easy setup with a clear configuration of the systems that are exposed to SAP BTP.

cloud management tools	Cloud management tools represent the group of technologies designed for managing SAP BTP. Cloud management tools is a synonym for the internally used term Foundation.
disaster	An event declared by SAP when there is a loss of utilities and services, and uncertainty about whether they can be restored within a reasonable period of time. When a disaster is declared, a disaster recovery plan comes into action.
disaster recovery (DR)	A set of policies, tools, and procedures to protect applications by preserving and rapidly resuming their availability in case of a disaster.
durability	The ability of a system to permanently store data without loss or corruption.
enterprise account [page 111]	An enterprise account is usually associated with one SAP customer or partner and is typically subject to charges. It groups together different subaccounts that an administrator makes available to users for deploying applications.
environment [page 62]	Constitutes the SAP BTP actual Platform-as-a-Service offering that allows for the development and administration of applications. Each environment provides at least one application runtime, its own user and role management logic, and tools (for example, command line utility). Environments are integrated into the platform at the subaccount level.
failover	The automated or manually triggered process of switching from one system to another redundant system in case of an unexpected or planned downtime.
formation [page 2326]	A logical grouping of SAP systems that can be extended in a single business scenario.
global account [page 121]	The realization of a contract you made with SAP. A global account is region- and environment-independent, and it is used to manage subaccounts, members, entitlements and quotas.

H-R

identity provider [page 3536]	An authorization authority containing all user information and credentials. In SAP BTP, user information is provided by identity providers, not stored in SAP BTP itself.
member [page 133]	Indicates a user's assignment to an account. As an account member, a user automatically has the permissions required to use the SAP BTP functionality within the scope of the respective account and as permitted by their account member roles.
OAuth	Widely adopted security protocol for protection of resources over the Internet. It is used by many social network providers and by corporate networks.
org [page 133]	A hierarchical level in the account structure of SAP BTP using a Cloud Foundry subaccount. Each Cloud Foundry subaccount contains exactly one Cloud Foundry org. Within an org, you can create several spaces.

platform service	Software that enables, facilitates, or accelerates the development of applications and other platform services on SAP BTP. Platform services are integrated with applications and other cloud resources by developers. End users only interact with platform services via applications, never directly. All platform services provide an interface such as an API or a set of APIs. There are two types of platform services: business and technical services.
programming model	A set of concepts used to create applications on SAP BTP. For example, a programming model can include programming languages, runtimes, and APIs.
quota [page 127]	A numeric quantity that defines the maximum allowed consumption of a specific technical asset/resource.
region [page 17]	A geographical location that usually consists of two or more availability zones.
resilience	The ability to provide and maintain an acceptable level of service in the face of faults and challenges until normal operation is restored.
resilient software design	Building and preparing an application or service in a way that it can handle failures that occur during runtime. Its goal is not to reduce the probability of failure occurrence, but to maximize the availability of systems and system landscapes in such cases.
runtime	An engine or context for executing programs, such as Java Web Tomcat 8 or Node.js runtime.

S-Z

SAP BTP, Cloud Foundry environment [page 63]	An open Platform-as-a-Service, which provides a scalable runtime container and a choice of clouds, runtimes, and services.
SAP BTP, Neo environment [page 108]	An enterprise Platform-as-a-Service, providing a range of services to our customers
SAP BTP, Kyma runtime [page 86]	A runtime the developers can use to build cloud-native Kubernetes-based extensions to SAP by using microservices and serverless Functions.
Pay-As-You-Go for SAP BTP [page 114]	A variant to the consumption-based commercial model of SAP BTP in which customers of SAP BTP are billed and are paying for the exact services used. This differs from the CPEA payment model where customers purchase a committed amount of services in advance.
btp CLI [page 2664]	The command line tool for all tasks on global account, directory, and subaccount level, such as creating or updating subaccounts, authorization management, and working with service brokers and platforms.
SAP ID service [page 3536]	The default identity provider for SAP BTP applications. It manages the user base for SAP Community Network and other SAP Web sites. SAP ID service is also used for authentication in the cockpit and operations such as deploying, updating, and so on.
SAPUI5 [page 431]	A development toolkit providing UI controls for developing Web applications.

service broker [page 762]	When a developer provisions and binds a service to an application, the service broker for that service is responsible for providing the service instance and for binding services to applications. For example, the HANA service broker allows any application running on Cloud Foundry to connect to an SAP HANA database.
service plan [page 127]	A variant of a service; for example, a database may be configured with various “t-shirt sizes”, each of which is a different service plan.
SAP Java Virtual Machine	SAP's own implementation of a Java Virtual Machine on which the SAP BTP infrastructure runs.
space [page 2800]	In the Cloud Foundry environment, every application and service is scoped to a space. A space provides users with access to a shared location for application development, deployment, and maintenance. Each space role applies only to a particular space.
staging	The process in the Cloud Foundry environment by which the raw bits of an application are transformed into a droplet that is ready to execute.
subaccount [page 121]	Lets you structure a global account according to customer requirements with regards to members, authorizations and quotas. In the Cloud Foundry environment, a global account can have as many subaccounts as required; each subaccount can have an associated Cloud Foundry org.
subscription [page 117]	One of the SAP BTP commercial models ("subscription-based"), where the customer subscribes to SAP BTP services for a certain amount of time and for a fixed price in order to get access to these services.
technical service [page 16]	Platform services that enable, facilitate, or accelerate the generic development of an application, independent of the application's business process or task.
tool [page 151]	A means for users to develop, configure, monitor and administer a service or entities managed by a service. A tool can be part of the platform or a service but is not a service by itself.
user-provided service instances [page 275]	User-provided service instances enable you to use services that are not available in the marketplace with your applications running in the Cloud Foundry environment.

Important Disclaimers and Legal Information

Hyperlinks

Some links are classified by an icon and/or a mouseover text. These links provide additional information.
About the icons:

- Links with the icon : You are entering a Web site that is not hosted by SAP. By using such links, you agree (unless expressly stated otherwise in your agreements with SAP) to this:
  - The content of the linked-to site is not SAP documentation. You may not infer any product claims against SAP based on this information.
  - SAP does not agree or disagree with the content on the linked-to site, nor does SAP warrant the availability and correctness. SAP shall not be liable for any damages caused by the use of such content unless damages have been caused by SAP's gross negligence or willful misconduct.
- Links with the icon : You are leaving the documentation for that particular SAP product or service and are entering an SAP-hosted Web site. By using such links, you agree that (unless expressly stated otherwise in your agreements with SAP) you may not infer any product claims against SAP based on this information.

Videos Hosted on External Platforms

Some videos may point to third-party video hosting platforms. SAP cannot guarantee the future availability of videos stored on these platforms. Furthermore, any advertisements or other content hosted on these platforms (for example, suggested videos or by navigating to other videos hosted on the same site), are not within the control or responsibility of SAP.

Beta and Other Experimental Features

Experimental features are not part of the officially delivered scope that SAP guarantees for future releases. This means that experimental features may be changed by SAP at any time for any reason without notice. Experimental features are not for productive use. You may not demonstrate, test, examine, evaluate or otherwise use the experimental features in a live operating environment or with data that has not been sufficiently backed up.
The purpose of experimental features is to get feedback early on, allowing customers and partners to influence the future product accordingly. By providing your feedback (e.g. in the SAP Community), you accept that intellectual property rights of the contributions or derivative works shall remain the exclusive property of SAP.

Example Code

Any software coding and/or code snippets are examples. They are not for productive use. The example code is only intended to better explain and visualize the syntax and phrasing rules. SAP does not warrant the correctness and completeness of the example code. SAP shall not be liable for errors or damages caused by the use of example code unless damages have been caused by SAP's gross negligence or willful misconduct.

Bias-Free Language

SAP supports a culture of diversity and inclusion. Whenever possible, we use unbiased language in our documentation to refer to people of all cultures, ethnicities, genders, and abilities.

www.sap.com/contactsap

© 2026 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company. The information contained herein may be changed without prior notice.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors. National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. All other product and service names mentioned are the trademarks of their respective companies.

Please see https://www.sap.com/about/legal/trademark.html for additional trademark information and notices.